

Custom Compiler™ Tcl Reference Manual

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	80
Related Products, Publications, and Trademarks	80
Conventions	81
Customer Support	82
Statement on Inclusivity and Diversity	83

1. Introduction to Custom Compiler Tcl	84
Tcl Namespaces	84
Displaying Command and Preference Descriptions	86
Tcl Command Naming Conventions	86
Tcl Argument and Return Types	87
Basic Data Types	87
Object Types	88
Semantic Object Types	89
Enumerations	91
Lists	92
Creating Custom Tcl User Commands	92
Command Registration	92
Noninteractive Commands	92
Creating Custom Noninteractive Commands	93
Interactive Commands	93
Creating Custom Interactive Commands	94
Event Handler Procedures	95
Interactive Command Attributes	99
Interactive Command Engines	100
Example: Using Interactive Commands	100
Tcl Filtering	102
Standard Attribute Filters	102
Operators	103
Attributes	104
Tcl Variables and Tcl Commands	105
Filter Examples	105
OpenAccess Tcl Interface Limitations	105

Using Tcl in Tcl Bytecode Format	107
Executing Tcl Received in Tcl Bytecode Format	108
Exporting Tcl in Tcl Bytecode Format	108
Encrypting Tcl Files	109

2. Integrated Tool Tcl for CustomSim	110
Commands	110
cs::createBoundary	111
cs::exportCSV	111
cs::importRAResults	112
cs::importSPRESResults	113
cs::loadNpfForShortestPath	113
cs::mergeMultiCCDB	114
cs::showFlowSelection	115
cs::showInstancePins	115
cs::showSelectResults	116
cs::showSetupDialog	116
cs::showShortestPathOverlays	117
cs::showSPRESP2PQuery	117
cs::showTruncateDbDialog	118
cs::showWhatIfDialog	118
cs::traceLRP	119
cs::traceShortestPath	119
cs::truncateDb	120
ics::addResistors	121
Preferences	121

3. Constraint Management Tcl	140
Using Tcl Commands to Set Constraints	140
Commands	142
cm::checkIntents	144
cm::createAffinity	145
cm::createAlignment	145
cm::createBlockSize	146
cm::createBus	147
cm::createCapacitance	148
cm::createCluster	149

Contents

cm::createCMOS	150
cm::createColorMatching	153
cm::createColumn	153
cm::createDifferentialGroup	155
cm::createEdgeAssign	156
cm::createEnclosingShape	157
cm::createGuardRing	159
cm::createHalo	160
cm::createHierNetSpacing	161
cm::createInstMatching	162
cm::createIntent	165
cm::createIntentDef	166
cm::createIntentParamDef	168
cm::createLength	169
cm::createMaxCommonResistance	170
cm::createNetPriority	171
cm::createNetShielding	172
cm::createOrientations	175
cm::createPatternRoute	176
cm::createPlacementGrid	177
cm::createRelative	177
cm::createResistance	178
cm::createResistorBank	179
cm::createRoutingStripes	181
cm::createRows	182
cm::createShieldOverhang	183
cm::createStandardCells	184
cm::createSymmetric	186
cm::createTask	187
cm::createTemplate	187
cm::createWireMatching	188
cm::createWireStriping	189
cm::exportIntents	190
cm::getarea	191
cm::getallowedcutclass	192
cm::getConstraints	194
cm::getCutClass	195
cm::getCutSpacing	196
cm::getCutToMetalSpacing	199

Contents

cm::getEolExtension	201
cm::getExtension	203
cm::getIntentDefs	206
cm_getlength	206
cm::getPlacement	208
cm::getRegionLpps	210
cm::getRoutingVias	210
cm::getRowPlacement	211
cm::getShieldOverhangData	212
cm::getSizePolicy	213
cm::getSpacing	214
cm::getSpacingData	216
cm::getTasks	217
cm::getViaStackLimits	217
cm::getWidth	218
cm::importIntentDefinitions	219
cm::importIntents	220
cm::showConstraintManager	221
cm::showExportIntents	221
cm::showImportIntents	222
Preferences	222
4. Design Collaboration Tcl	225
Commands	225
dc::addAnnotation	226
dc::addComment	227
dc::addImage	228
dc::addPolicy	229
dc::addToolStateClient	230
dc::captureImage	231
dc::changeState	232
dc::createIssue	233
dc::createReview	234
dc::delete	235
dc::displayImages	236
dc::endReview	237
dc::exportReview	237
dc::getActivePolicy	238

Contents

dc::getActiveReview	238
dc::getReviews	239
dc::getToolState	240
dc::setActiveReview	240
dc::setPolicy	241
dc::setToolState	242
dc::showCreateReview	242
dc::showEditReview	243
dc::showReviewManager	243
dc::startReview	244
idc::addReviewAnnotation	244
idc::addReviewImage	245
Preferences	245
5. Design Insights Command Reference	249
di::closeResultsCellView	250
di::createResultsCellView	250
di::createParasiticView	251
di::getCapacitors	252
di::getCapAttr	252
di::getCorners	253
di::getElmoreDelayResult	253
di::getElmoreDelayResultAttr	254
di::getFanoutResult	254
di::getFanoutResultAttr	255
di::getInstAttr	255
di::getInsts	256
di::getLayers	257
di::getModels	257
di::getNetAttr	257
di::getNets	258
di::getNodeAttr	258
di::getResAttr	259
di::getResistanceContributionAttr	260

di::getResistanceResult	260
di::getResistanceResultAttr	261
di::getResistors	261
di::getTermAttr	262
di::getTerms	262
di::getViewType	263
di::listAttrs	263
di::loadResultsCellView	263
di::runElmoreDelayAnalysis	264
di::runFanoutAnalysis	264
di::runResistanceAnalysis	265
Preferences	265
6. Database Tcl	267
Commands	267
db::add	281
db::addAttr	281
db::addFigGroupMembers	282
db::addGroupMembers	283
db::addInit	284
db::addListener	285
db::addMarkerObject	286
db::append	287
db::appendLibertyEditorPinData	288
db::attachTech	289
db::batch	289
db::cat	290
db::checkDRD	290
db::checkFeature	292
db::checkForInterrupt	293
db::checkLicenseTier	293
db::checkoutFeature	294
db::checkVersion	295
db::cleanPcellRepository	295
db::closeCDF	296
db::closeDesign	297

Contents

db::copyLibertyData	298
db::copyParams	299
db::copyPrefs	300
db::createAssignAssignment	301
db::createAssignValue	302
db::createCallback	303
db::createCircularRuler	304
db::createCircularRulerDef	305
db::createCollection	306
db::createDesign	307
db::createDeviceMappingFileDialog	307
db::createEnum	308
db::createEvaluator	309
db::createExpressionMap	310
db::createFigGroup	310
db::createGenOptionSet	311
db::createGroup	313
db::createHistogram	314
db::createImage	315
db::createIncludeFileControl	316
db::createInstCache	317
db::createInstTerm	317
db::createInterfaceParam	318
db::createInterfacePin	320
db::createInterfaceSet	321
db::createInterruptHandler	322
db::createLibListInput	323
db::createLinearRuler	324
db::createList	325
db::createMarker	326
db::createNet	327
db::createNetlistInst	328
db::createNotifier	328
db::createObject	330
db::createPaperSize	331
db::createParamDef	332
db::createParamSandbox	334
db::createParasiticStencil	335
db::createPatternDef	336

Contents

db::createPcell	337
db::createPcellDef	338
db::createPin	341
db::createPref	342
db::createPrinter	344
db::createProgress	346
db::createProp	347
db::createScope	348
db::createScopeType	350
db::createScratchDesign	351
db::createStdViaVariant	351
db::createStreamNameMap	353
db::createSubCircularRuler	353
db::createWindowAnnotationSet	354
db::decodeSymbol	355
db::defineUdvDevice	355
db::defineVflagMaxDevice	356
db::defineVflagMaxMinDevice	356
db::defineVflagMinDevice	357
db::defineVsyncDevice	357
db::deleteBuiltInVias	358
db::deprecateAttr	358
db::destroy	359
db::detectNetlistType	361
db::enableTkSend	362
db::engToSci	362
db::eval	363
db::evalCallback	364
db::evalEnvironment	365
db::evalParam	366
db::exportImage	367
db::exportLibertyFile	368
db::exportMCellFile	369
db::exportParams	370
db::exportStream	371
db::exportTechFile	379
db::expr	380
db::filter	382
db::findCircuitMatches	382

Contents

db::findConnections	383
db::findConstraint	384
db::findContextConnectionDef	385
db::findContextConnections	386
db::findGenerator	387
db::findInterfaceReader	388
db::findPatternMatches	389
db::flushPcellCache	389
db::foreach	390
db::freezeRefs	391
db::generateBuiltinVias	391
db::generateCDL	392
db::generateCellView	393
db::generateConstraintPatterns	394
db::generateDeviceMappingTemplate	395
db::generateLayout	396
db::generateRule	397
db::generateSchematic	398
db::generateSPICE	401
db::generateSystemVerilog	402
db::generateVerilog	402
db::generateVerilogA	403
db::generateVerilogAMS	404
db::generateVHDL	405
db::getAnnotationSets	405
db::getAssignmentDefs	406
db::getAssignments	406
db::getAttr	407
db::getBuiltinNetlists	409
db::getCallbacks	410
db::getCellViewInterface	411
db::getCircularRulerDefs	411
db::getComponents	412
db::getConnectionDefState	413
db::getContextConnectionDefs	414
db::getCount	415
db::getCurrentContext	415
db::getCurrentEvalText	416
db::getCurrentParam	417

Contents

db::getCurrentRef	418
db::getCutClasses	419
db::getDeviceDisplays	419
db::getEffectiveNet	420
db::getEnums	420
db::getEquivalentNets	421
db::getEvaluators	423
db::getExportOasisOptions	424
db::getExportStreamOptions	424
db::getFeatures	424
db::getFigGroupMembers	425
db::getFigGroups	425
db::getFlightlineCoordinates	426
db::getGenOptionSets	427
db::getGroupMembers	427
db::getGroups	428
db::getICCCellViews	429
db::getIndex	429
db::getInstHeaders	430
db::getInsts	431
db::getInstTerms	432
db::getLayers	433
db::getLicenseTier	434
db::getLibertyEditorPinsData	434
db::getLPPHeaders	435
db::getMarkers	435
db::getNetlistInfo	436
db::getNets	437
db::getNext	439
db::getNotifiers	439
db::getOpenDesigns	441
db::getPaperSizes	442
db::getParamDefs	442
db::getParamFilter	443
db::getParams	444
db::getParamValue	445
db::getPatternDefs	447
db::getPatternInst	447
db::getPcellCacheLocation	448

Contents

db::getPcellCacheSize	448
db::getPcellDefs	449
db::getPcells	450
db::getPins	451
db::getPrefs	452
db::getPrefValue	454
db::getPrinters	454
db::getProcessInfo	455
db::getProps	456
db::getRegisteredNetlistInfo	456
db::getRulers	457
db::getScopes	458
db::getScopeTypes	459
db::getShapes	460
db::getTableValue	461
db::getTerms	462
db::getType	463
db::getVDRCDDeviceList	464
db::getVDRCDDeviceParamName	464
db::getVDRCDDeviceVoltage	464
db::getVersion	465
db::getViaDefs	465
db::getViaHeaders	466
db::getVias	467
db::getVirtualInstTerms	467
db::getWindowAnnotationSets	468
db::hideProps	469
db::importEDIF	470
db::importMCellFile	472
db::importTechFile	472
db::importText	474
db::includes	478
db::init	478
db::instCacheTriggerParamCallbacks	479
db::interrupt	480
db::isEmpty	480
db::isEncrypted	481
db::isObject	481
db::isProhibitedName	482

Contents

db::listAttrs	483
db::listCallbackTypes	484
db::loadDeviceDisplays	484
db::loadGenOptionSets	485
db::loadMCells	487
db::loadModelFile	487
db::loadPrefs	488
db::loadStreamNameMap	489
db::localizePrefs	489
db::makePcellEditable	491
db::mapStreamName	515
db::match	515
db::md5sum	516
db::monitorChannel	517
db::notify	517
db::openDesign	518
db::openHier	519
db::openDesignHierarchy	519
db::openParamDefEditor	520
db::openScope	521
db::pdaExplainCell	522
db::pdaShowCell	522
db::print	523
db::purgeCDF	527
db::purgePrefs	528
db::query	529
db::queryFeature	529
db::raiseLicenseTier	530
db::refreshICCCache	530
db::refreshMCells	531
db::registerCallbackRegsub	532
db::registerGenerator	532
db::registerInterfaceReader	534
db::registerNetlistInfo	535
db::registerParamEvalFunction	535
db::relativePath	536
db::releaseFeature	537
db::remove	538
db::removeAttr	538

Contents

db::removeFigGroupMembers	539
db::removeMarkerObject	540
db::removeLibertyEditorPinData	540
db::renderTemplatesParasitics	541
db::reopenDesign	542
db::reportCallbacks	542
db::reset	543
db::resetVDRCDefinedDevices	543
db::resolve	544
db::resolvePath	545
db::revertParam	546
db::revertPrefs	547
db::saveCDF	547
db::saveDesign	548
db::saveDesignAs	549
db::saveDeviceDisplays	549
db::saveGenOptionSets	550
db::savePrefs	551
db::sciToEng	552
db::setAnalogLibCompatibleVersion	553
db::setAttr	554
db::setColorShiftLayers	555
db::setDeviceDisplay	556
db::setInstName	557
db::setLicenseTier	557
db::setMaster	558
db::setNetlistInfo	559
db::setParamFilter	560
db::setParamValue	561
db::setPrefValue	563
db::setProgress	565
db::setViaDef	566
db::showAbstractGeneratorOptions	567
db::showAdvancedPrintOptions	567
db::showBindExternalNetlistFile	568
db::showBindExternalNetlistView	568
db::showCellViewFromCellView	569
db::showCellViewFromPinList	570
db::showDummyRules	571

Contents

db::showEditComponents	571
db::showExportDef	572
db::showExportImage	572
db::showExportLef	573
db::showExportLiberty	573
db::showExportMCellFile	574
db::showExportMilkyway	574
db::showExportNetlist	574
db::showExportNetlist::flattenDesign	575
db::showExportOasis	576
db::showExportStream	576
db::showFindReplace	576
db::showGenerateAbstract	577
db::showGenerateSchematic	578
db::showGenerateSchematicsFromText	578
db::showGenericPrint	578
db::showImportDef	579
db::showImportEDIF	579
db::showImportLef	580
db::showImportMCellFile	580
db::showImportMilkyway	580
db::showImportOasis	581
db::showImportSimulationDataDialog	581
db::showImportStream	582
db::showImportText	582
db::showImportVerilog	582
db::showLibertyDataEditor	582
db::showLoadPreferences	583
db::showManageTechnology	583
db::showPrint	584
db::showPrintQueueStatus	584
db::showPurgePreferences	585
db::showRefreshMCells	585
db::showSavePreferences	586
db::showUnitDisplayScale	586
db::showUpdateCellView	587
db::showVDRCManagerWindow	587
db::snapPoints	587
db::sort	588

Contents

db::source	589
db::startStatistics	590
db::stopStatistics	591
db::transform	591
db::teeLogFile	592
db::triggerParamCallback	593
db::unregisterGenerator	594
db::unregisterInterfaceReader	595
db::updateCDFParams	596
db::vdrcMgrAddNet	596
db::vdrcMgrAddNetPair	597
db::vdrcMgrAnnotateNet	597
db::vdrcMgrAnnotateNetPair	598
db::vdrcMgrCheckHierarchyMismatch	599
db::vdrcMgrCheckSymbolPlacement	599
db::vdrcMgrClearAllData	599
db::vdrcMgrDeleteNet	600
db::vdrcMgrDeleteNetPair	600
db::vdrcMgrImportLayout	601
db::vdrcMgrImportSchematic	601
db::vdrcMgrLoadDRCViolations	602
db::vdrcMgrRefresh	602
db::viaDefExists	603
db::viewExists	604
db::vwait	605
idb::createImage	605
idb::query	605
Enumerations	606
Objects	607
dbAssignAssignment	609
dbAssignmentDef	609
dbAssignValue	610
dbCallback	610
dbConnections	611
dbContextConnectionDef	611
dbContextConnections	612
dbDeviceDisplay	612
dbGenOptionSet	613

Contents

dbInstCache	613
dbInterfaceParam	614
dbInterfacePin	614
dbInterfaceSet	614
dbParam	615
dbParamDef	616
dbPcellDef	617
dbPref	617
dbPrinter	618
dbScope	618
dmCell	619
dmCellCategory	619
dmCellView	620
giWindow	620
heContext	621
leMPP	622
oaInst	622
oaInstTerm	623
oaNet	624
oaPin	625
oaShape	625
oaTerm	626
oaVia	627
saConvergenceAid	627
saDesignVariable	628
saDesignVariables	628
saDesignVariableSet	628
saIncludeFile	629
saModelFile	629
saOutput	629
saOutputs	630
saOutputSet	630
saPair	631
saSession	631
saTemperature	631
saTestbench	632
wfDataFile	633
xtJob	633
Preferences	634

7. Design Editing Tcl	716
Commands	716
de::abortCommand	726
de::activateTools	726
de::addCapability	727
de::addObject	728
de::addPoint	729
de::addTemplateView	730
de::addTool	731
de::applyOLPPreset	732
de::audit	732
de::cancelCommand	733
de::cancelTransaction	733
de::clearHighlights	734
de::clearUnusedDesigns	735
de::clone	735
de::close	736
de::commandOption	736
de::completeShape	738
de::connect	739
de::connectToServer	740
de::copy	740
de::copyBookmark	741
de::copyClipboardToLib	742
de::createActivationEngine	743
de::createApplication	746
de::createArgument	747
de::createArgumentGroup	750
de::createAssociation	751
de::createBookmark	752
de::createCanvas	753
de::createColorExpression	755
de::createCommand	756
de::createContext	758
de::createCrossProbeTool	759
de::createErrorViewerDatabase	760
de::createErrorViewerMarker	760
de::createFavoriteDevice	762

Contents

de::createFavoriteDeviceCategory	763
de::createGradient	763
de::createHierObject	764
de::createHighlightSelector	765
de::createHighlightSet	766
de::createHomePage	769
de::createInterfaceElement	770
de::createLPPInput	771
de::createLPPSet	773
de::createNetHighlightSet	774
de::createObjectFilter	775
de::createOLPPreset	776
de::createOnCanvasWidget	777
de::createOverlay	777
de::createProbe	779
de::createSelectionEngine	781
de::createShapeEngine	784
de::createTemplate	786
de::createTool	787
de::crossCheckViews	789
de::cut	791
de::cycleActiveFigure	792
de::cycleTapObject	793
de::deactivateTools	794
de::defineOrigin	794
de::deleteBookmarks	794
de::deprecateCommand	795
de::descend	796
de::descendInPlace	797
de::descendIntoGroup	798
de::deselect	798
de::deselectAll	799
de::deselectAllLogic	799
de::deselectLogic	800
de::disconnect	801
de::enableColorAnnotator	801
de::enablePartitioningDisplay	802
de::enableRotationAndScaling	803
de::endDrag	804

Contents

de::endLog	804
de::endTransaction	805
de::errorViewerLoadResult	806
de::exportBookmarks	807
de::exportProbeCellSettings	808
de::extractInterfaceElements	808
de::findMessage	809
de::finishEvalTextRendering	810
de::fit	810
de::getActiveCommand	812
de::getActiveContext	813
de::getActiveEditorWindow	813
de::getActiveFigure	814
de::getActiveLPP	815
de::getApplications	815
de::getAssociations	816
de::getBackgroundImporters	817
de::getBBox	817
de::getBinsForGradient	818
de::getBookmarks	818
de::getBreakpoints	819
de::getCapabilities	819
de::getClipboardLibs	820
de::getClips	821
de::getColorAnnotators	822
de::getCommands	822
de::getContexts	823
de::getCrossProbeTools	824
de::getCursor	824
de::getCursorSnapPoint	825
de::getDBLimits	825
de::getDesignTemplates	826
de::getEditors	826
de::getErrorViewerDatabases	827
de::getFavoriteDeviceCategories	828
de::getFavoriteDevices	828
de::getFigures	829
de::getHighlighted	831
de::getHighlightSets	832

Contents

de::getInterfaceElements	833
de::getLayers	834
de::getLookaheadMessages	835
de::getLPPs	835
de::getLPPSets	836
de::getMergedBBox	837
de::getObjectFilters	838
de::getOLPPresets	845
de::getOpenFiles	846
de::getOutdatedDesigns	847
de::getOutdatedDMData	847
de::getOverlays	848
de::getPartitioner	849
de::getProbeBitNames	849
de::getProbeFigures	850
de::getProbeNets	851
de::getProbes	853
de::getRecentFiles	854
de::getSelected	855
de::getSelectedLogic	855
de::getSelectedVertex	856
de::getTemplates	857
de::getTools	858
de::getTransactions	858
de::getViewport	860
de::gotoViewport	860
de::help	861
de::highlight	862
de::importBookmarks	863
de::importProbeCellSettings	864
de::isEditable	864
de::isHidden	865
de::isInUndo	865
de::isModified	866
de::linkContext	866
de::loadFavoriteDevices	867
de::loadInterfaceElements	868
de::loadOLPPresets	869
de::loadProbes	870

Contents

de::navigateTo	871
de::next	871
de::open	872
de::open3DICDesign	874
de::openCanvasPropertyEditor	875
de::openClipboardLib	875
de::openEditor	876
de::openFCDesign	877
de::openFile	878
de::openICC2Design	878
de::pan	879
de::paste	880
de::redoSelection	882
de::redoTransaction	883
de::redraw	884
de::registerHelp	884
de::removeBreakpoints	885
de::removeObject	886
de::removePoint	886
de::removeTool	887
de::renumberInstances	888
de::repeatCommand	888
de::replay	889
de::replayCommand	890
de::reportOpenTransactions	891
de::restoreBookmark	891
de::return	892
de::revert	893
de::run	894
de::runBackgroundImporter	894
de::save	894
de::saveAs	895
de::saveFavoriteDevices	896
de::saveInterfaceElements	897
de::saveOLPPresets	898
de::saveProbes	898
de::select	899
de::selectAll	900
de::selectLogic	901

Contents

de::send	901
de::sendComplete	902
de::sendMessage	902
de::setActiveLPP	903
de::setBreakpoint	904
de::setCursor	905
de::setEditMode	905
de::setErrorViewerCategory	906
de::setErrorViewerDetailInfo	907
de::setErrorViewerFieldName	908
de::setErrorViewerTip	909
de::setViewport	909
de::shiftCursor	910
de::showAbout	912
de::showBookmarkManager	912
de::showCloseDesigns	912
de::showConnectionsNavigator	913
de::showConnectionsNavigatorOptions	913
de::showCreateBookmark	913
de::showCrossCheckOptions	914
de::showDescendDialog	914
de::showDesignsPrompt	915
de::showGeneralOptions	917
de::showGradientEditor	917
de::showMakeDesignsReadOnly	918
de::showOpen3DICDesign	919
de::showOpenDesign	919
de::showOpenFCDesign	920
de::showOpenICC2Design	920
de::showParamFilter	921
de::showProbeBitSelectionDialog	921
de::showProperties	922
de::showRefreshDesigns	922
de::showRenumberInstances	923
de::showReturnToLevelDialog	923
de::showSaveDesignAs	924
de::sleep	924
de::snapToObject	925
de::startDrag	926

Contents

de::startLog	927
de::startRendererStatistics	927
de::startTransaction	928
de::stopBackgroundImporter	929
de::stopRendererStatistics	929
de::suppressMessage	930
de::transformPoint	930
de::undoSelection	931
de::undoTransaction	932
de::unhighlight	933
de::uniquify	934
de::zoom	935
ide::createProbe	936
ide::defineOrigin	936
ide::descend	936
ide::editCanvasText	937
ide::enterBox	938
ide::enterLine	939
ide::enterPoint	940
ide::enterPolygon	941
ide::highlight	942
ide::pan	943
ide::pick	943
ide::selectByName	944
ide::selectByRegion	945
ide::setViewport	946
ide::testEvents	947
ide::testStack	947
ide::zoom	947
Enumerations	948
Objects	949
deArgument	950
deArgumentGroup	951
deAssociation	951
deBookmark	952
deCanvas	953
deClipboard	954
deCommand	955

Contents

deContext	956
deEditor	957
deFigure	957
deHighlightSet	958
deLayer	958
deLPP	959
deLPPSet	960
deObjectFilter	961
deOverlay	961
deProbe	962
deTool	963
deTransaction	963
deViolation	964
Preferences	965
8. Data Management Tcl	1003
Commands	1003
dm::add3DICLibrary	1007
dm::addFCLibrary	1008
dm::addICC2Library	1009
dm::addICCRefLibs	1011
dm::addToCellCategory	1011
dm::addToLibDefs	1012
dm::associateVersionControl	1013
dm::checkIn	1014
dm::checkLock	1014
dm::checkOut	1015
dm::checkStatus	1016
dm::convertDMSystem	1017
dm::compress	1017
dm::copyCellCategory	1018
dm::copyCells	1019
dm::copyCellsUnique	1024
dm::copyCellViews	1025
dm::copyLib	1030
dm::copyProps	1034
dm::create3DICLibrary	1035
dm::createCell	1036

Contents

dm::createCellCategory	1037
dm::createCellInput	1038
dm::createCellView	1041
dm::createCellViewInput	1042
dm::createFCLibrary	1044
dm::createICC2Library	1045
dm::createLib	1047
dm::createLibInput	1048
dm::createMilkywayLib	1050
dm::createMultiCellViewInput	1051
dm::deleteFromCellCategory	1053
dm::findAutoLoadFile	1054
dm::findCell	1055
dm::findCellView	1055
dm::findDMData	1056
dm::findDMFile	1057
dm::get3DICLayerMap	1058
dm::get3DICRefLibs	1058
dm::getCategoriesFromCategory	1059
dm::getCategoriesOfMember	1060
dm::getCellCategories	1061
dm::getCells	1062
dm::getCellsFromCategory	1063
dm::getCellViews	1064
dm::getDMFileFollowers	1065
dm::getDMFileLeader	1066
dm::getDMFiles	1067
dm::getFCLayerMap	1068
dm::getFCRefLibs	1068
dm::getFollowers	1069
dm::getICC2LayerMap	1069
dm::getICC2RefLibs	1070
dm::getLeaders	1070
dm::getLibDefPath	1071
dm::getLibs	1071
dm::getMilkywayRefLibs	1072
dm::getOpenDesigns	1073
dm::getSelected	1073
dm::getStandardLibDefPath	1074

Contents

dm::getTopLibDefPath	1075
dm::getVersionControl	1075
dm::getVersionControlSystems	1076
dm::importAutoLoadFile	1076
dm::isDMFileLeader	1078
dm::isHidden	1078
dm::moveCells	1079
dm::moveCellViews	1081
dm::moveLib	1083
dm::openLibraryDefinitionsEditor	1085
dm::openLibraryManager	1086
dm::openLibs	1086
dm::refreshLibraryManager	1087
dm::refreshLibs	1088
dm::registerCopyProps	1089
dm::registerVersionControl	1090
dm::removeVersionControl	1094
dm::renameCellCategory	1095
dm::revert	1095
dm::saveAsFCRefLib	1096
dm::saveAsICC2RefLib	1096
dm::set3DICLCVMap	1097
dm::set3DICRefLibs	1097
dm::setFCLCVMap	1098
dm::setFCRefLibs	1098
dm::setFollowers	1099
dm::setICC2LCVMap	1100
dm::setICC2RefLibs	1100
dm::setICCTechSync	1101
dm::setSelected	1102
dm::showAddICC2Library	1102
dm::showAddLibrary	1103
dm::showCopyCell	1103
dm::showCopyCellProperties	1104
dm::showCopyCellView	1104
dm::showCopyLibrary	1104
dm::showCopyUnique	1105
dm::showDeleteViewsByName	1105
dm::showEditLibraryCompression	1106

Contents

dm::showEditProperties	1106
dm::showLibraryManager	1106
dm::showMoveCell	1107
dm::showMoveCellView	1107
dm::showMoveLibrary	1108
dm::showNewCell	1108
dm::showNewCellView	1108
dm::showNewLibrary	1109
dm::showRenameCellCategory	1110
dm::syncToVersion	1110
dm::uncompress	1111
dm::updateLib	1111
Enumerations	1112
Objects	1112
dmCell	1112
dmCellCategory	1113
dmCellView	1113
dmContainerPair	1114
dmDMFile	1115
Preferences	1116
9. Display Resource Tcl	1126
Commands	1126
dr::createColor	1127
dr::createColorSelector	1128
dr::createDisplay	1129
dr::createLineStyle	1131
dr::createLineStyleSelector	1131
dr::createPacket	1133
dr::createPacketAlias	1134
dr::createStipple	1135
dr::createTextPacket	1136
dr::deleteColor	1137
dr::deleteLineStyle	1137
dr::deleteStipple	1138
dr::getColors	1138
dr::getDisplays	1139

Contents

dr::getLineStyles	1140
dr::getPacketAliases	1140
dr::getPackets	1141
dr::getStipples	1142
dr::getTextPackets	1143
dr::loadDisplayResources	1144
dr::mergeDisplayResources	1145
dr::renameColor	1146
dr::renameLineStyle	1146
dr::renameStipple	1147
dr::saveDisplayResources	1147
dr::showDisplayResourceEditor	1148
Enumerations	1149
Objects	1149
drColor	1149
drDisplay	1150
drLineStyle	1150
drPacket	1150
drPacketAlias	1151
drStipple	1151
Preferences	1152
10. Design Technology Co-Optimization Tcl	1153
Commands	1153
dtco::checkAllTextLabelsInsidePRBoundary	1154
dtco::compileAllUDDs	1154
dtco::createCompactModel	1154
dtco::createInterconnectModel	1155
dtco::createIVCV	1156
dtco::createParasiticExtractedNetlist	1157
dtco::createSplit	1159
dtco::createUDD	1160
dtco::showSetupAndRunDialog	1161
Preferences	1161
11. Graphical Infrastructure Tcl	1169

Contents

Commands	1169
gi::addActions	1174
gi::addAssistant	1175
gi::addMenu	1176
gi::addToolBar	1178
gi::addWindow	1179
gi::applyWorkspace	1181
gi::closeFrames	1181
gi::closeWindows	1182
gi::collapse	1182
gi::configureTable	1183
gi::createAction	1184
gi::createActionCategory	1186
gi::createAppGrid	1187
gi::createAppIcon	1188
gi::createBinding	1189
gi::createBindingSet	1192
gi::createBooleanInput	1192
gi::createCarousel	1194
gi::createCarouselItem	1195
gi::createCheckableGroup	1197
gi::createColumn	1198
gi::createDataType	1200
gi::createDbTable	1202
gi::createDialog	1205
gi::createDrag	1208
gi::createDynamicList	1209
gi::createFileInput	1211
gi::createGroup	1213
gi::createInlineGroup	1215
gi::createLabel	1216
gi::createListInput	1218
gi::createMenu	1221
gi::createMutexInput	1222
gi::createNumberInput	1224
gi::createPointListInput	1227
gi::createPushButton	1229
gi::createRow	1231
gi::createScrollArea	1232

Contents

gi::createTabGroup	1233
gi::createTable	1234
gi::createTextInput	1237
gi::createTheme	1240
gi::createToolBar	1240
gi::createWebView	1242
gi::createWindow	1243
gi::createWorkspace	1244
gi::createX11Window	1245
gi::drop	1246
gi::execDialog	1247
gi::executeAction	1247
gi::expand	1248
gi::exportTable	1248
gi::findChild	1249
gi::getActionCategories	1250
gi::getActions	1251
gi::getActiveDialog	1252
gi::getActiveTab	1252
gi::getActiveWindow	1252
gi::getAppIcons	1253
gi::getAssistants	1254
gi::getBindings	1255
gi::getBindingSets	1255
gi::getCells	1256
gi::getChildren	1257
gi::getColumns	1257
gi::getContextMenu	1258
gi::getDataTypes	1259
gi::getDialogs	1260
gi::getField	1261
gi::getFrames	1261
gi::getMenus	1262
gi::getRows	1263
gi::getSectionSizes	1263
gi::getThemes	1264
gi::getToolbars	1265
gi::getWindows	1266
gi::getWindowTypes	1266

Contents

gi::getWorkspaces	1267
gi::getXidUsage	1268
gi::hideContextMenu	1268
gi::layout	1268
gi::loadBindings	1270
gi::loadWorkspaces	1270
gi::moveColumns	1271
gi::moveRows	1272
gi::openTerminal	1273
gi::pickWindow	1273
gi::pressButton	1274
gi::prompt	1274
gi::registerWebViewScheme	1276
gi::remove	1277
gi::reportWidgetState	1277
gi::saveBindings	1278
gi::saveWorkspaces	1278
gi::scrollTo	1279
gi::sendGUIEvent	1280
gi::setActiveDialog	1281
gi::setActiveTab	1281
gi::setActiveWindow	1282
gi::setContextMenu	1283
gi::setCurrentIndex	1284
gi::setDropAction	1284
gi::setField	1286
gi::setFocus	1287
gi::setItemSelection	1288
gi::setSectionSizes	1288
gi::showBusyCursor	1289
gi::showCommandsHelp	1290
gi::showConsole	1290
gi::showContextMenu	1290
gi::showCustomizeWindow	1291
gi::showDbTableFilter	1291
gi::showDbTableFilterExpressions	1293
gi::showDbTableSort	1293
gi::showKeyboardShortcuts	1295
gi::showSaveHistoryAs	1295

Contents

gi::sortItems	1296
gi::unregisterWebViewScheme	1296
Enumerations	1297
Objects	1297
dbScope	1298
giAction	1298
giAssistant	1300
giContainer	1301
giMenu	1302
giStatusBar	1303
giToolbar	1303
giWidget	1304
giWindow	1305
giWindowType	1306
Preferences	1306
12. Hierarchy Editing Tcl	1312
Commands	1312
he::addRule	1314
he::addViewGroup	1315
he::clearBindings	1316
he::clone	1316
he::closePropDB	1317
he::computeId	1318
he::createConfig	1318
he::createConfigTemplate	1320
he::createContext	1321
he::createLogicalSelectionEngine	1322
he::createNetInput	1323
he::createOverlayConfig	1325
he::createOverlayView	1325
he::createRule	1326
he::createViewGroup	1329
he::deleteRule	1330
he::deleteViewGroup	1330
he::expandViewList	1331
he::findPropDB	1332

Contents

he::findSwitchedMaster	1332
he::foreach	1334
he::getConfigTemplates	1334
he::getConfigView	1335
he::getExternalFileName	1335
he::getInstHeaders	1336
he::getInsts	1337
he::getProps	1337
he::getPropValue	1339
he::getRules	1340
he::getSelected	1343
he::getViewGroups	1343
he::isLeaf	1344
he::listPropDBs	1344
he::openConfig	1345
he::openPropDB	1346
he::popContext	1348
he::pushContext	1349
he::queryViewSelection	1350
he::saveAsPropDB	1351
he::saveConfig	1352
he::savePropDB	1353
he::setPropValue	1354
he::setRuleFromDE	1356
he::showExportToFile	1357
he::showHierarchyEditor	1357
he::showLogicalSelectionOptions	1358
he::showObject	1358
he::undoRedoHistory	1359
Enumerations	1359
Objects	1360
dmCell	1360
dmCellView	1360
giWindow	1361
heConfig	1362
heConfigTemplate	1363
heContext	1364
heInstHeader	1365

heRule	1365
heViewNameGroup	1366
oalnst	1366
Preferences	1367

13. iPDK Tcl Procedures	1374
iPDK_* Procedures	1374
iPDK_engToSci	1375
iPDK_getCurrentInst	1375
iPDK_getInstCellName	1375
iPDK_getInstLibName	1376
iPDK_getOrient	1376
iPDK_getParamDef	1377
iPDK_getParamValue	1377
iPDK_islvpcell	1378
iPDK_isLayout	1378
iPDK_isSchematic	1378
iPDK_paramExist	1378
iPDK_sciToEng	1379
iPDK_setParamValue	1379

14. Layout Editing Tcl	1381
Commands	1381
le::addDummy	1396
le::addToFigGroup	1397
le::addVirtualConnection	1398
le::align	1400
le::alignAngle	1401
le::autoFix	1402
le::autoVia	1403
le::boolean	1405
le::breakSync	1406
le::changePinRelationship	1407
le::checkAutoConfiguration	1407
le::checkCapacitance	1408
le::checkDRCInRegion	1408
le::checkDRCInViewRegion	1409

Contents

le::checkDRCViolations	1409
le::checkDRCViolationsByICV	1410
le::checkEMResConstraints	1411
le::checkOnModificationByICV	1412
le::checkOpens	1413
le::checkPinConnectivity	1414
le::checkShapeEM	1414
le::checkShield	1416
le::checkShorts	1417
le::checkTrackSnapping	1418
le::checkVias	1419
le::checkVoltageAnnotations	1420
le::chop	1422
le::clearFlightlines	1424
le::clearPinConnections	1425
le::clearVoltagePhysicalAnnotations	1426
le::clearVoltages	1426
le::cloneMaster	1426
le::closeICVLManager	1427
le::closeStarRC	1427
le::compactBus	1428
le::connectICCNet	1429
le::convertToAliasLayer	1430
le::convertToArray	1430
le::convertToPath	1431
le::convertToPolygon	1432
le::copy	1432
le::copyHighlighted	1435
le::copyLook	1436
le::copyTrackRegionDef	1436
le::correctOffgrid	1437
le::createAttributeLabel	1438
le::createAbutGroups	1440
le::createBlockage	1441
le::createBoundary	1442
le::createCellViewFromSelection	1443
le::createCoordinateMark	1445
le::createCorridorArea	1446
le::createDRDLineEdit	1447

Contents

le::createDRDTreeWidget	1448
le::createEllipse	1450
le::createGlobalTrackRegion	1451
le::createGuardRing	1452
le::createICCIInst	1453
le::createICNet	1455
le::createInst	1456
le::createLabel	1458
le::createMPP	1461
le::createMPPDef	1462
le::createMPPEnclosedPathDef	1463
le::createMPPEnclosedShapeDef	1465
le::createMPPMasterPathDef	1466
le::createMPPOffsetPathDef	1467
le::createMPPRingDef	1468
le::createMPPSubRectDef	1470
le::createNetGroup	1472
le::createOffTrackInfo	1473
le::createPath	1474
le::createPathSeg	1475
le::createPin	1478
le::createPinsFromText	1479
le::createPolygon	1480
le::createRectangle	1481
le::createRouteCorridor	1483
le::createRow	1483
le::createSiteDef	1485
le::createStripedWire	1486
le::createStripeSegment	1488
le::createSyncFigGroup	1489
le::createSyncGroup	1490
le::createTerm	1491
le::createTrack	1492
le::createTrackGroupDef	1494
le::createTrackPattern	1496
le::createTrackPatternDef	1497
le::createTrackPcell	1500
le::createTrackRegion	1501
le::createTrackRegionDef	1503

Contents

le::createVia	1506
le::createVoltagePhysicalAnnotations	1508
le::cutCorner	1509
le::cycleDensity	1510
le::delete	1511
le::deleteCutMetal	1511
le::deleteDensityResult	1512
le::deleteGeneratedPins	1512
le::deleteVirtualConnection	1513
le::designSummary	1513
le::displayFlightlines	1514
le::displayPinConnections	1515
le::distribute	1515
le::EMAccuracyTool	1517
le::EMRCDecryptFile	1517
le::EMRCCEMResultToEV	1518
le::EMRCRevertLogFile	1518
le::EMRCXoutToDSPFFile	1519
le::enableDebugLogModule	1519
le::evalViaParams	1520
le::exportResistanceQuery	1521
le::extractLayoutContext	1522
le::extendHandleSelection	1522
le::externalizeColor	1523
le::fill	1524
le::fixAcuteAngle	1526
le::fixMinWidthShapes	1527
le::fixTrackSnapping	1528
le::flatten	1528
le::generateAbstract	1530
le::generateAntennaRunset	1532
le::generateCutMetal	1533
le::generateGuardRing	1536
le::generateLabel	1538
le::generateLayout	1541
le::generateLayoutContext	1541
le::generatePGMarkers	1542
le::generatePins	1543
le::generateShapes	1546

Contents

le::generateSoftAbstract	1548
le::genPinFigureShorts	1549
le::getAbuttedInsts	1550
le::getAbuttedMPPs	1551
le::getAbutterAbuttee	1551
le::getAllAbutterAbutteeInDes	1552
le::getAreaAndPerimeter	1553
le::getBoundaryTrackRegionDefs	1553
le::getCommonPathPoints	1554
le::getCoordinateMarks	1555
le::getGateDistance	1555
le::getGateRectangles	1560
le::getGlobalTrackRegions	1561
le::getIncompleteNets	1562
le::getInstanceBBox	1562
le::getIntersections	1563
le::getMPPDefs	1565
le::getMPPs	1566
le::getNearestTracks	1567
le::getNetGroups	1568
le::getNetIslands	1570
le::getPinFigureShorts	1570
le::getPseudoParallelNets	1570
le::getRouteCorridors	1571
le::getRuleInfoByICV	1572
le::getStretchHandleCategorizedInfo	1573
le::getStripedWires	1574
le::getSyncGroups	1574
le::getTrackGroupDefs	1575
le::getTrackPatternDefs	1576
le::getTrackPatterns	1577
le::getTrackRegions	1578
le::getTrackRegionDefs	1579
le::getTracks	1580
le::getTracksInRegion	1581
le::getUnboundGlobalTrackRegions	1582
le::getUnboundTracks	1582
le::getUnboundTrackRegions	1583
le::getUnnettedFigs	1583

Contents

le::highlightConnected	1584
le::importCommandBindings	1586
le::importResistanceQuery	1586
le::importVoltages	1587
le::isInSync	1587
le::merge	1587
le::move	1589
le::optimize	1590
le::padLabel	1591
le::pasteLook	1592
le::propagateConnectivity	1592
le::refreshDensityResult	1593
le::regenerateMPP	1594
le::remaster	1594
le::removeFromFigGroup	1595
le::removeOverlappingCuts	1595
le::removeUnboundInstanceTerms	1596
le::repeatCopy	1596
le::replaceVia	1597
le::reportCapacitance	1597
le::reportCapacitanceFromEPA	1599
le::reportCELayers	1600
le::reportConnectivityPoints	1601
le::reportEM	1602
le::reportEMBackground	1604
le::reportNets	1604
le::reportNetsFromEPA	1605
le::reportRC	1605
le::reportResistance	1607
le::reportResistanceFromEPA	1611
le::reportShield	1613
le::resetVoltageOverrides	1614
le::reshape	1614
le::resize	1616
le::rotate	1618
le::scalarizeArrayInst	1619
le::selectStretchHandle	1619
le::setDRDOption	1620
le::setNetAnchor	1621

Contents

le::showAlignAssistOptions	1622
le::showCommandBindings	1622
le::showCreateTrack	1622
le::showCreateTrackGroupDefinition	1623
le::showCreateTrackRegionDefinition	1623
le::showDefineTerminals	1624
le::showDesignOptions	1624
le::showDRDOptions	1625
le::showDRDRuleSelection	1625
le::showEditBlackBox	1626
le::showEditFCBlackBox	1626
le::showEditICC2BlackBox	1627
le::showEditTrack	1627
le::showEditTrackGroupDefinition	1627
le::showEditTrackRegion	1628
le::showElectricalOptions	1628
le::showEMCalculator	1629
le::showImportLayout	1629
le::showLayoutGeneratorOptions	1630
le::showOLPOptions	1630
le::showTrackRegionOptions	1631
le::split	1631
le::splitArray	1632
le::splitIntoBoxes	1633
le::syncTo	1634
le::stretch	1634
le::toggleDRDTreeWidget	1636
le::trimWire	1636
le::validateColor	1637
le::yank	1638
ile::addDummy	1640
ile::addToFigGroup	1640
ile::adjustCursor	1641
ile::align	1641
ile::alignAngle	1642
ile::applyColor	1642
ile::attachDetach	1643
ile::autoChopGuardRing	1643
ile::autoFix	1644

Contents

ile::autoFlipHorizontal	1644
ile::autoFlipVertical	1644
ile::autoRotate90	1645
ile::boolean	1645
ile::bridgeInterconnect	1646
ile::changePinRelationship	1646
ile::checkCongestion	1646
ile::checkDensity	1647
ile::checkVias	1647
ile::checkVoltageAnnotations	1648
ile::chop	1648
ile::clearDensityDisplay	1649
ile::clearVoltagePhysicalAnnotations	1649
ile::clearVoltages	1649
ile::cloneMaster	1650
ile::collapseDensityTree	1650
ile::color	1651
ile::compactBus	1651
ile::continueBus	1651
ile::convertToArray	1652
ile::convertToPath	1652
ile::convertToPolygon	1653
ile::copy	1653
ile::copyLook	1654
ile::copyToClipboard	1654
ile::correctOffGrid	1655
ile::createAttributeLabel	1655
ile::createBlockage	1656
ile::createBoundary	1656
ile::createBus	1657
ile::createCellViewFromSelection	1657
ile::createCoordinateMark	1657
ile::createEllipse	1658
ile::createFigGroup	1658
ile::createGlobalTrackRegion	1659
ile::createGuardRing	1659
ile::createInst	1659
ile::createInterconnect	1660
ile::createLabel	1660

Contents

ile::createMPP	1661
ile::createPin	1661
ile::createPinsFromSelection	1662
ile::createPlane	1662
ile::createPolygon	1662
ile::createRectangle	1663
ile::createRouteCorridor	1663
ile::createRow	1664
ile::createRuler	1664
ile::createTopology	1665
ile::createTrackRegion	1665
ile::createTrackRegionPCell	1665
ile::createTransmissionLine	1666
ile::createVia	1666
ile::createVoltageConfig	1667
ile::createVoltagePhysicalAnnotations	1667
ile::cut	1667
ile::cutCorner	1668
ile::defineInFOLibrary	1668
ile::delete	1669
ile::deleteCutMetal	1669
ile::deleteFigGroup	1670
ile::distributeAlign	1670
ile::editFCBlackBox	1671
ile::editICC2BlackBox	1671
ile::editICCBlackBox	1671
ile::eraseLayer	1672
ile::expandDensityTree	1672
ile::fill	1673
ile::flatten	1673
ile::generateCutMetal	1673
ile::generateGuardRing	1674
ile::generateLabel	1674
ile::generateLayoutPins	1675
ile::generateMPP	1675
ile::generatePins	1676
ile::generatePlacementTrackRegion	1676
ile::generateRoutingTrackRegion	1676
ile::generateShapes	1677

Contents

ile::heal	1677
ile::hierarchicalCopy	1678
ile::highlightPath	1678
ile::importVoltages	1679
ile::inputCoord	1679
ile::lineSelect	1679
ile::measureAngle	1680
ile::measureDistance	1680
ile::merge	1681
ile::move	1681
ile::optimize	1682
ile::padLabel	1682
ile::partialToFull	1683
ile::paste	1683
ile::pasteLook	1684
ile::reassignNet	1684
ile::removeFromFigGroup	1685
ile::removeOverlappingCuts	1685
ile::replaceVia	1686
ile::reportCapacitance	1686
ile::reportEM	1687
ile::reportParasitic	1687
ile::reportResistance	1688
ile::reshape	1688
ile::reshapeInterconnect	1688
ile::resize	1689
ile::rotate	1689
ile::scalarizeArrayInst	1690
ile::selectByLPP	1690
ile::selectObjects	1690
ile::selectRouteObjects	1691
ile::showDensityDisplaySetting	1691
ile::showEMCalculator	1692
ile::split	1692
ile::splitArray	1692
ile::stackInterconnect	1693
ile::stretch	1693
ile::stretchArray	1694
ile::stretchWidth	1694

Contents

ile::stripeWire	1695
ile::tap	1695
ile::trimWire	1696
ile::validateColor	1696
ile::yank	1696
le::pkg::checkPlaneBoundary	1697
le::pkg::createDegassingHole	1698
le::pkg::createPlaneBoundary	1700
le::pkg::deleteDegassingHole	1701
le::pkg::deleteFloatingShapes	1701
le::pkg::deleteRedundantVias	1702
le::pkg::generatePlaneBoundary	1702
le::pkg::generateRedundantVias	1707
le::pkg::generateSealRing	1707
le::pkg::generateZigzag	1708
le::pkg::getPlaneBoundaries	1708
le::pkg::highlightPlaneBoundary	1709
le::pkg::realizePlane	1710
le::pkg::replacePlaneBoundary	1711
le::pkg::reportRedundantVias	1712
le::pkg::reportShieldingRate	1712
lepkg::refreshPlane	1713
ile::pkg::createDegassingHole	1714
ile::pkg::createPlaneBoundary	1714
ile::pkg::generateRedundantVias	1714
ile::pkg::generateZigzag	1715
ile::pkg::replacePlaneBoundary	1715
Enumerations	1716
Objects	1719
dbCircularRuler	1720
dbGenOptionSet	1720
dbLinearRuler	1721
dbSubCircularRuler	1721
dmCell	1722
dmCellView	1722
giWindow	1723
leCoordinateMark	1723
leDimensionMark	1724

leFlightline	1725
leMPP	1725
oaInst	1725
oaInstTerm	1726
oaNet	1727
oaPin	1728
oaShape	1729
oaTerm	1729
oaVia	1730
Preferences	1731
15. Placement Tcl	1875
Commands	1875
lp::adjustRelativeSpacing	1878
lp::align	1878
lp::applyTemplates	1880
lp::autoPinPlace	1880
lp::autoPlace	1881
lp::checkAreaPinPlacement	1882
lp::checkPinPlacement	1882
lp::checkSimilarity	1882
lp::cleanUpPins	1883
lp::convertCustomConstraint	1883
lp::copyResult	1884
lp::doDeviceGrouping	1884
lp::doStdCellGrouping	1885
lp::dumpDMProp	1885
lp::exportLayoutPlacement	1886
lp::extractAlignment	1886
lp::extractFigGroup	1887
lp::extractGRParams	1887
lp::extractGRPitchSpacing	1888
lp::extractLayoutParameters	1888
lp::extractOrigin	1889
lp::extractOriginFromFile	1889
lp::extractOriginInfoFromFile	1890
lp::extractOriginToFile	1890
lp::extractPinPlacement	1891

Contents

lp::extractPlacement	1891
lp::extractPRBoundary	1892
lp::extractSEDSUBGROUPS	1893
lp::extractSurroundCells	1893
lp::extractSymmetric	1894
lp::extractTopology	1894
lp::fastPlace	1895
lp::generateAreaPinPlacement	1896
lp::generateConstraint	1897
lp::generatePinPlacement	1898
lp::generatePlacement	1899
lp::generatePRBoundary	1900
lp::generateSurroundCells	1900
lp::getGroupVectorName	1901
lp::groupByGuardRing	1901
lp::groupBySurroundCells	1902
lp::groupDeviceByArea	1902
lp::handleConstraintArea	1903
lp::importPinPlacement	1903
lp::insertBoundary	1904
lp::insertColumn	1904
lp::insertDecapFiller	1906
lp::insertDiode	1906
lp::insertTap	1907
lp::isReadOnly	1909
lp::optimizePins	1909
lp::placementDebugger	1910
lp::placeToRows	1911
lp::rebuildFigGroup	1911
lp::saveDesignToPlacementLib	1912
lp::showOptions	1912
lp::testGenerateGrouping	1912
lp::updateConstraint	1913
lp::updateGRSpacing	1913
lp::updateLayoutParameters	1914
ilp::autoPinPlace	1915
ilp::autoPlace	1915
ilp::copyResult	1916
ilp::createConstraintArea	1916

ilp::createPlacementArea	1916
ilp::extractTopology	1917
ilp::optimizePins	1917
ilp::placeToRows	1918
Enumerations	1918
Preferences	1919
16. Schematic Driven Layout Tcl	1927
Commands	1927
lx::addPenalty	1934
lx::applyPresetParams	1934
lx::backAnnotateLDE	1935
lx::batchCreateTemplate	1936
lx::changeInstanceMapping	1936
lx::checkFeature	1936
lx::clearAnchors	1937
lx::clearCrossHighlight	1937
lx::clearHighlight	1938
lx::clearNetVoltage	1938
lx::clearRuler	1938
lx::createParasiticView	1939
lx::createPinsOnTop	1939
lx::createSession	1940
lx::dumpMismatchButton	1940
lx::ecoPostUpdateLayout	1941
lx::ecoPreUpdateLayout	1942
lx::ecoUpdateLayout	1942
lx::establishCorrespondence	1943
lx::establishCorrespondenceFromLVS	1944
lx::execParasiticExtraction	1945
lx::expandSchematic	1945
lx::exportSdlNetlist	1947
lx::extractFeatures	1947
lx::extractParasiticTrainingData	1948
lx::extractTemplate	1949
lx::findCloneCandidates	1950
lx::generateEdtext	1950
lx::generatePlacementCatalog	1951

Contents

lx::getAssistantCanvas	1952
lx::getAssociatedObject	1952
lx::getAssociatedObjectByName	1953
lx::getBrowserCanvas	1954
lx::getDesignFromSED	1954
lx::getDeviceInfo	1955
lx::getFoldingParam	1955
lx::getFoldingThreshold	1956
lx::getMismatchedConstraints	1957
lx::getPlacement	1957
lx::getPseudoParallelNets	1958
lx::getSchematicDummies	1958
lx::getSessions	1959
lx::highlightFlightline	1959
lx::incrementalRealizeConstraint	1960
lx::importSDLMappingFile	1960
lx::layoutHierarchyChangeECOUpdate	1961
lx::loadAddShapes	1961
lx::loadAddShapeSettings	1962
lx::loadCellTemplate	1962
lx::parasiticBackAnnotate	1963
lx::plePostProcess	1963
lx::predictParasitic	1964
lx::preview	1965
lx::realize	1966
lx::realizeConstraint	1967
lx::realizeTemplate	1967
lx::rechain	1968
lx::registerLayoutTemplate	1968
lx::reloadSymbolicEditor	1969
lx::removePeriphery	1969
lx::reopenLogicalSource	1970
lx::replacePlacement	1971
lx::resetLayoutNets	1971
lx::saveAddShapes	1972
lx::saveAddShapeSettings	1972
lx::saveCellTemplate	1973
lx::saveConstraint	1973
lx::saveTemplateConstraint	1973

Contents

Ix::sedCreateSession	1974
Ix::sedDestroySession	1975
Ix::sedGetDesign	1975
Ix::sedGetGridSize	1976
Ix::sedSetParam	1976
Ix::sedSetPreset	1977
Ix::setPlacement	1977
Ix::setupSDL	1978
Ix::showAddShapeSettingsDialog	1978
Ix::showAllNet	1979
Ix::showBatchCreateTemplate	1979
Ix::showCellTemplate	1979
Ix::showCheckerDialog	1980
Ix::showCheckOptions	1980
Ix::showCreateTemplate	1981
Ix::showDebugMessage	1981
Ix::showDefineSource	1982
Ix::showDefineTarget	1983
Ix::showExpandSchematic	1984
Ix::showGateConnected	1984
Ix::showGateNet	1984
Ix::showGenerateLayout	1985
Ix::showImportLayout	1985
Ix::showLHCGenerateInstanceMap	1986
Ix::showLHCGenerateSchematic	1986
Ix::showODNet	1987
Ix::showOptions	1987
Ix::showParasiticEstimationByMLDialog	1987
Ix::showPresetSetup	1988
Ix::showSDLCheckAndRepairWindow	1988
Ix::showSDLMapping	1988
Ix::showSymbolicDesignOptions	1989
Ix::showUpdateSchematic	1989
Ix::splitDevice	1990
Ix::symbolic	1990
Ix::templateDesigner	1991
Ix::testSaveTemplateConstraint	1991
Ix::unregisterLayoutTemplate	1991
Ix::updateConstraints	1992

Contents

lx::updateConstraintBySDL	1992
lx::updateInstanceCache	1993
lx::updateLayout	1993
lx::verify	1994
lx::verifyHierarchy	1995
lx::verifyNetVoltage	1996
ilx::add	1997
ilx::addDummy	1997
ilx::addIncremental	1997
ilx::addPeripheralDummy	1998
ilx::alignGates	1998
ilx::alignIndividual	1999
ilx::alignRow	1999
ilx::applyPresetParams	2000
ilx::associateSpareDummy	2000
ilx::calculateGridSize	2000
ilx::chainSerialRes	2001
ilx::clone	2001
ilx::copy	2002
ilx::customizeSpacing	2002
ilx::delete	2002
ilx::deleteTrack	2003
ilx::distance	2003
ilx::establishCorrespondence	2004
ilx::flip	2004
ilx::flipRow	2005
ilx::foldGates	2005
ilx::generateGroupPlacements	2005
ilx::generateGuardRings	2006
ilx::generatePattern	2006
ilx::generatePlacement	2007
ilx::highlightFlightline	2007
ilx::insertColumn	2008
ilx::insertRow	2008
ilx::insertShift	2008
ilx::insertTrack	2009
ilx::manualCorrespondence	2009
ilx::merge	2010
ilx::mergeGates	2010

Contents

ilx::mirror	2010
ilx::modifyDummy	2011
ilx::modifyFlightline	2011
ilx::modifyPode	2012
ilx::modifyPreset	2012
ilx::move	2013
ilx::moveFlightline	2013
ilx::moveInstance	2013
ilx::optimizeOD	2014
ilx::pickAndPlace	2014
ilx::reassignNet	2015
ilx::rechain	2016
ilx::removeColumn	2016
ilx::removeLayoutParam	2016
ilx::removeRow	2017
ilx::removeShift	2017
ilx::reportSummary	2018
ilx::resetCustomizedSpacing	2018
ilx::resetIndividualAlignment	2019
ilx::rotate	2019
ilx::sedRealize	2020
ilx::selectEmpty	2020
ilx::split	2020
ilx::splitDevice	2021
ilx::swapDevice	2021
ilx::swapInterdigitate	2021
ilx::swapSD	2022
ilx::swapTerm	2022
ilx::symbolicEditor	2023
ilx::transpose	2023
ilx::unalignGates	2024
ilx::unsetFingerGroup	2024
ilx::updateLayout	2024
ilx::viaTrackPlanner	2025
Enumerations	2025
Objects	2027
lxConnectionPoint	2028
lxSession	2029

Contents

	lxNetlist	2029
	lxInst	2030
	lxTerm	2031
	lxInstTerm	2031
	lxPair	2031
	lxNode	2032
	lxNet	2033
	Preferences	2033
17.	Netlist Editing Tcl	2082
	Commands	2082
	ne::getLogicals	2082
	ne::showDesignOptions	2083
	Enumerations	2084
	Objects	2084
	neLogical	2084
	Preferences	2084
18.	Netlist Tool Tcl	2086
	Commands	2086
	nl::addMapping	2090
	nl::addVariableReference	2092
	nl::addViewStopList	2092
	nl::append	2093
	nl::calculateMapping	2094
	nl::checkInstHeader	2096
	nl::checkSplitBuses	2096
	nl::checkVHDLPortConsistency	2097
	nl::checkVHDLUniqueEntityDefinitions	2097
	nl::clearBuffer	2098
	nl::computeTermOrder	2099
	nl::computeTermOrder::getTermsFromDesign	2101
	nl::computeTermOrder::sortTermsByType	2101
	nl::computeVirtualTerminalName	2102
	nl::createAMSNetlister	2102
	nl::createHierMap	2103

Contents

nl::createNameMap	2103
nl::createNetlister	2105
nl::createOANameMap	2106
nl::createParameterizedNameMap	2108
nl::createPartitioner	2110
nl::createTerm	2111
nl::extractDigitalPowerDomains	2112
nl::finalizeNameMap	2112
nl::findAMSDomain	2113
nl::findAMSDomainObjects	2113
nl::findInstNamePrefix	2114
nl::findModuleDir	2115
nl::findModuleName	2115
nl::findNetlistProcedure	2116
nl::findParamMap	2117
nl::findParamValue	2118
nl::flattenDesign	2119
nl::generateEvent	2120
nl::getAlwaysWriteParamDefault	2121
nl::getAMSNetlisters	2121
nl::getBuffer	2122
nl::getEffectiveNetlistInfo	2123
nl::getExternalEntities	2123
nl::getICCSupplyName	2124
nl::getInstTerms	2124
nl::getMappedContext	2125
nl::getMappings	2126
nl::getModuleDescription	2127
nl::getModuleNameMapForAMS	2128
nl::getNamesForMapping	2128
nl::getNetlisters	2129
nl::getNetMembers	2130
nl::getNetType	2132
nl::getParamNames	2132
nl::getReferencedVariables	2133
nl::getShortedNets	2134
nl::getSupplyType	2135
nl::getTerms	2135
nl::getWireBitOrder	2136

Contents

nl::getWorkerNetlisters	2137
nl::indent	2138
nl::instHeaderHandler	2138
nl::isLeaf	2139
nl::isModuleUpToDate	2140
nl::isNetNameGlobal	2140
nl::isStringParam	2141
nl::isTextModule	2141
nl::lookup	2142
nl::map	2143
nl::mapModuleNames	2144
nl::mapPath	2145
nl::mapTerminalNames	2146
nl::message	2146
nl::moduleBodyHandler	2147
nl::moduleHandler	2148
nl::modulePreprocessHandler	2149
nl::netlistBeginHandler	2150
nl::netlistBodyHandler	2150
nl::netlistEndHandler	2151
nl::netlistMainHandler	2151
nl::nlPrintCdInst	2152
nl::nlPrintCdInstInstance	2153
nl::openNameMap	2153
nl::preloadModuleNames	2154
nl::printSpectreInst	2155
nl::printSpiceInst	2156
nl::printSpiceInstTerms	2157
nl::refreshPartitioner	2157
nl::revertOutdatedDesigns	2158
nl::runNetlister	2158
nl::saveModuleState	2159
nl::saveNameMap	2160
nl::setTermOrder	2160
nl::setWireBitOrder	2162
nl::uniqueify	2162
nl::writeUPF	2163
Enumerations	2164

Contents

Objects	2165
dmCell	2165
dmCellView	2166
heContext	2166
heInstHeader	2167
nlAMSNetlister	2167
nlHierMap	2168
nlInstTerm	2168
nlNameMap	2169
nlNet	2170
nlNetlister	2171
nlOANameMap	2174
nlParameterizedNameMap	2175
nlTerm	2177
oalnst	2177
Netlisting Functions	2179
nl::cdl::printSpiceParam	2179
nl::cdl::setBadChars	2180
nl::HSPICE::getModelParamValue	2181
nl::HSPICE::getModuleNameForTextView	2181
nl::HSPICE::getSpiceInstTermName	2182
nl::HSPICE::spiceAppendTermNames	2183
Spectre::exprMapFuncProc	2183
VerilogAMS::printInst	2183
VerilogAMS::printInstParameters	2184
vlg::warnFilteredNetTypes	2184
Netlisting Procedures	2184
nlPrintauCdIBJT	2184
nlPrintauCdIDevice	2185
nlPrintauCdIDiode	2186
nlPrintauCdIMOSFET	2187
nlPrintauCdIPrimDevice	2188
nlPrintauCdISubckt	2189
Preferences	2189
Netlister Preference Examples	2216
nl<netlister>CompatiblePortOrder	2216
nlIgnoreCell	2219

19. Geometry Manipulation Tcl	2222
Commands	2222
po::and	2225
po::angle	2226
po::append	2228
po::area	2230
po::chop	2232
po::coinEdge	2233
po::compress	2238
po::convexEdge	2238
po::createCollection	2241
po::createPointSet	2243
po::cut	2244
po::cutCorner	2246
po::density	2247
po::donut	2250
po::enclose	2252
po::encloseRect	2254
po::enclosure	2255
po::expandEdge	2259
po::extent	2261
po::extents	2263
po::external	2264
po::getAngle	2268
po::getArea	2269
po::getContours	2270
po::getDistance	2271
po::getHoles	2272
po::getIntersection	2274
po::getMaxBox	2274
po::getPerimeter	2276
po::getPoints	2277
po::getSpecifiedBoxes	2277
po::inside	2279
po::insideEdge	2281
po::interact	2283
po::internal	2285
po::isClockwise	2289

Contents

po::isHorizontal	2290
po::isVertical	2291
po::length	2293
po::makeClockwise	2295
po::manhattanContour	2296
po::not	2297
po::offGrid	2299
po::or	2300
po::outside	2301
po::outsideEdge	2303
po::pathLength	2304
po::perimeter	2306
po::polyToTrap	2308
po::rectangle	2309
po::rectangles	2312
po::rectEnclosure	2313
po::reshape	2315
po::shapePoints	2315
po::shift	2317
po::size	2318
po::snapGrid	2322
po::split	2322
po::straddle	2324
po::touch	2324
po::touchEdge	2326
po::transform	2330
po::vertex	2333
po::withEdge	2334
po::withNeighbor	2336
po::withWidth	2338
po::xor	2340
Preferences	2342
Geometry Manipulation Example	2342
20. Relative Placement Language Tcl	2346
Introduction to Relative Placement Language Tcl	2346
Using Relative Placement Language	2347
A Cell With Schematic	2347

Contents

A Cell Without Schematic	2347
Relative Placement Language Basics	2348
Net and Instance Name Definition	2349
Using RPL Commands to Build a Circuit	2349
Commands	2356
rpl::addTracks	2358
rpl::align	2361
rpl::closeCell	2363
rpl::createBoundary	2364
rpl::createCell	2365
rpl::createInstance	2366
rpl::createInstPinConnection	2367
rpl::createLibrary	2368
rpl::createNet	2368
rpl::createTerm	2369
rpl::createTermLabel	2369
rpl::deleteCell	2370
rpl::discardDesigns	2371
rpl::error	2371
rpl::evaluate	2371
rpl::generateSchematic	2372
rpl::getTracks	2373
rpl::group::assign	2374
rpl::group::clear	2374
rpl::group::get	2375
rpl::pack	2375
rpl::param	2378
rpl::place	2379
rpl::reset	2379
rpl::setOrientation	2380
rpl::setup::addReferenceLibraries	2380
rpl::setup::getCurrentContext	2381
rpl::setup::getCurrentDesign	2381
rpl::setup::getOpenContexts	2381
rpl::setup::getOpenContextsCount	2382
rpl::setup::getReferenceLibraries	2382
rpl::setup::getWorkingLayoutLibrary	2382
rpl::setup::getWorkingLibrary	2383
rpl::setup::getWorkingSchematicLibrary	2383

Contents

rpl::setup::setReferenceLibraries	2383
rpl::setup::setWorkingLayoutLibrary	2384
rpl::setup::setWorkingLibrary	2384
rpl::setup::setWorkingSchematicLibrary	2385
rpl::tracks	2385
rpl::utils::createContext	2386
rpl::utils::deletePinFigs	2386
rpl::utils::expandNames	2387
rpl::utils::getBbox	2388
rpl::utils::getNets	2388
rpl::utils::layers::getMetalLayerStack	2389
rpl::utils::PRBoundaryFind	2389
Preferences	2390
21. Routing Tcl	2391
Commands	2391
rt::autoGenerateTrunk	2393
rt::cdIsRoute	2395
rt::checkPostRoute	2396
rt::checkPreRoute	2399
rt::checkRoute	2399
rt::checkRoutingStyle	2401
rt::checkTopology	2402
rt::connectCellRails	2402
rt::createFreeFormTopology	2404
rt::createGroupPattern	2405
rt::createP2PRoute	2406
rt::createPatternRoute	2409
rt::createPowerStaples	2410
rt::createPowerStripes	2413
rt::createTrunkTopology	2416
rt::createUniblock	2417
rt::createViaLadder	2418
rt::deleteCustomRouteShapes	2419
rt::deleteShield	2419
rt::deleteTopology	2420
rt::deleteTrunkAndShield	2420
rt::deleteTrunkBox	2421

Contents

rt::estimateRoute	2421
rt::exportTopology	2422
rt::extractShielding	2423
rt::generateTrunk	2424
rt::getStrappableNets	2433
rt::getTopologies	2433
rt::getTopologyInfo	2434
rt::getTopologyNets	2434
rt::importTopology	2435
rt::insertCutMetalPatch	2435
rt::insertVias	2437
rt::powerHookup	2438
rt::realizeGlobalRoute	2440
rt::reportSimilarity	2440
rt::route	2441
rt::routeHierarchy	2441
rt::shield	2442
irt::addStripe	2443
irt::autoRoute	2443
irt::autoShield	2444
irt::createIRoute	2444
irt::createP2PShield	2445
irt::createRoute	2445
irt::createRoutingArea	2445
irt::createTopology	2446
irt::deleteCustomRouteShapes	2446
irt::deleteMesh	2447
irt::deleteShield	2447
irt::generateMeshRoute	2447
irt::generateTrunk	2448
irt::previewPattern	2448
irt::query	2448
Preferences	2449
Route Pattern Description Reference	2497
Route Pattern Description Syntax	2497
trunk	2499
spine	2499
backbone / fishbone	2501

mirror	2501
intermittent / match	2503
track	2503
pin	2505
row	2507
tapoff	2507
layer	2508
width	2508
cluster	2509
22. Schematic Editing Tcl	2510
Commands	2510
se::activateSheet	2516
se::addSheets	2516
se::addToOverlayRendering	2517
se::align	2518
se::annotateOP	2519
se::attachText	2520
se::calculateVDRCDeltaV	2521
se::check	2521
se::chopWires	2523
se::clearNetColorizations	2524
se::clearUnboundOverlays	2525
se::colorizeNets	2525
se::compareDesigns	2526
se::connectWires	2527
se::copy	2528
se::createArc	2530
se::createBlock	2532
se::createCheckOption	2532
se::createConnectDef	2534
se::createConstraintNote	2535
se::createEllipse	2537
se::createEMDB	2539
se::createInst	2540
se::createLine	2541
se::createMask	2543
se::createNetColorizationDefinition	2543

Contents

se::createOPDB	2544
se::createPin	2545
se::createPolygon	2547
se::createRectangle	2548
se::createRule	2549
se::createSelectionShape	2553
se::createTable	2554
se::createText	2556
se::createVDRcdb	2558
se::createViolation	2559
se::createWire	2561
se::createWireName	2563
se::createWireStubs	2565
se::delete	2566
se::disconnectWires	2567
se::enableDynamicNetLabels	2567
se::extractConnectivity	2568
se::findMaskedInstances	2569
se::findReferenceDifferences	2569
se::generateEstRCSchematic	2570
se::generateSchematic	2570
se::generateSymbol	2572
se::generateSymbolTransformPCell	2574
se::getActiveSheet	2574
se::getBillOfMaterials	2575
se::getColorizationDefinitions	2575
se::getDynamicNetLabels	2576
se::getNetComponents	2576
se::getOverlayObjects	2577
se::getPinWiringPoint	2578
se::getReferenceCellView	2579
se::getRules	2579
se::getShapeStyle	2580
se::getSheets	2581
se::getTables	2582
se::importConfig	2582
se::importSymbol	2583
se::isDesignNetlistable	2584
se::isNetlistable	2584

Contents

se::isOverlay	2585
se::loadNetColorizationDefinitions	2585
se::move	2586
se::moveSheetToPosition	2587
se::populateDesignSummary	2588
se::propagatePortOrder	2590
se::registerLabelFunction	2591
se::registerParasiticTopology	2592
se::registerPinShape	2593
se::renameNet	2594
se::resizeSheets	2595
se::saveNetColorizationDefinitions	2596
se::schematicMigration	2596
se::setConnectivityValid	2598
se::setShapeStyle	2599
se::setSheetColumns	2600
se::showAnnotateOP	2600
se::showBillOfMaterials	2601
se::showCheckOptions	2601
se::showCompareDesigns	2602
se::showCreateTemplateFromSelected	2602
se::showEditSheets	2603
se::showEditTable	2603
se::showLibraryCheck	2604
se::showNetColorizationDefinitionDialog	2604
se::showPlotSignalSettings	2604
se::showPowerDomainAnalyzer	2605
se::showSchDesignOptions	2605
se::showSchematicMigration	2606
se::showSymDesignOptions	2606
se::showSynchronizePortOrder	2607
se::stretch	2607
se::syncOverlayDesign	2608
se::unregisterLabelFunction	2609
se::updatePinLabels	2609
ise::align	2610
ise::alignToColumn	2610
ise::attachText	2611
ise::check	2611

Contents

ise::chopWires	2612
ise::colorizeNets	2612
ise::connectWires	2612
ise::copy	2613
ise::copyToClipboard	2614
ise::createBlock	2614
ise::createColumn	2614
ise::createConnectDef	2615
ise::createConnector	2615
ise::createConstraintNote	2616
ise::createInst	2616
ise::createLabel	2617
ise::createMask	2617
ise::createNote	2618
ise::createOverlayInstanceWithMask	2618
ise::createRuler	2619
ise::createSchematicPin	2619
ise::createSelectionShape	2619
ise::createShape	2620
ise::createSymbolPin	2620
ise::createTable	2621
ise::createTemplate	2621
ise::createWire	2622
ise::createWireName	2623
ise::createWireStubs	2623
ise::cut	2623
ise::delete	2624
ise::disconnectWires	2624
ise::ignoreInst	2625
ise::importSymbol	2625
ise::manageLabels	2626
ise::move	2626
ise::paste	2627
ise::peek	2627
ise::plotSignal	2628
ise::printOP	2628
ise::renameInst	2629
ise::route	2629
ise::stretch	2629

Contents

Enumerations	2630
Objects	2632
dbGenOptionSet	2632
dbInterfaceSet	2633
dbScope	2633
deFigure	2633
deViolation	2634
dmCell	2635
dmCellView	2635
giWindow	2635
oaInst	2636
oaNet	2637
oaPin	2638
oaShape	2638
oaTerm	2639
seRule	2640
Preferences	2641
23. Simulation and Analysis Environment Scripting Tcl	2699
Commands	2699
ss::addGroupDefItem	2703
ss::addModelGroupDefFile	2703
ss::backAnnotateParasitics	2704
ss::beginCalibration	2705
ss::createAging	2705
ss::createAnalysis	2706
ss::createCorner	2707
ss::createGlobalModelGroup	2707
ss::createGroup	2708
ss::createGroupDef	2708
ss::createGroupSection	2709
ss::createModelGroup	2710
ss::createModelGroupDef	2710
ss::createModelGroupSection	2711
ss::createObject	2711
ss::deleteDuplicateCorners	2712
ss::deleteModelGroupDefFile	2712

Contents

ss::disable	2713
ss::editOutputs	2713
ss::enable	2714
ss::endCalibration	2715
ss::expr	2715
ss::findIncludeGroupDef	2716
ss::findModelGroupDef	2716
ss::findStimulusGroupDef	2717
ss::findVariableGroupDef	2717
ss::foreachSignal	2718
ss::generateCorners	2718
ss::getActiveSession	2719
ss::getActiveTestbench	2720
ss::getAnalyses	2720
ss::getCorners	2720
ss::getModelGroups	2721
ss::getOutputs	2721
ss::getTestbenches	2721
ss::listAnalyses	2722
ss::listCorners	2722
ss::listDesVars	2723
ss::listEnvOptions	2724
ss::listIncludeFiles	2724
ss::listMacros	2725
ss::listModelFiles	2725
ss::listModelGroupDefs	2726
ss::listModelGroups	2727
ss::listNodes	2727
ss::listOutputs	2728
ss::listResults	2728
ss::listSaveOptions	2729
ss::listSignals	2730
ss::listSimOptions	2730
ss::listStimulusSets	2731
ss::netlist	2731
ss::newSession	2732
ss::newTestbench	2733
ss::plot	2733
ss::print	2734

Contents

ss::printMeasurementSummary	2735
ss::printNoiseSummary	2736
ss::printTable	2738
ss::readOutputs	2740
ss::readTable	2740
ss::releaseNode	2741
ss::run	2741
ss::saveResults	2742
ss::selectDataDir	2743
ss::selectResults	2743
ss::setActiveNets	2744
ss::setActiveSession	2744
ss::setActiveTestbench	2745
ss::setAnalysisOptions	2745
ss::setDesign	2746
ss::setDesVars	2746
ss::setDeviceParameters	2747
ss::setEnvOptions	2747
ss::setGlobalDeviceParameters	2748
ss::setGlobalModelFiles	2748
ss::setGlobalTemperature	2749
ss::setGlobalVariables	2749
ss::setIncludeFiles	2750
ss::setJobOptions	2750
ss::setModelFiles	2751
ss::setNode	2751
ss::setPostRunProcs	2752
ss::setPreRunProcs	2752
ss::setSaveOptions	2753
ss::setSimOptions	2753
ss::setSimulator	2754
ss::setStimulusSet	2754
ss::setTemperature	2755
ss::setTestbenchName	2755
ss::simulateCalibration	2756
ss::trackActiveTestbench	2756
Enumerations	2756
Objects	2757

dmCell	2757
dmCellView	2758
saCorner	2758
saCornersSetup	2759
saOutput	2759
saResultFile	2760
saResults	2760
saSession	2761
saSimulator	2762
saSweep	2763
wfSignal	2763
24. MTB Simulation Scripting Tcl	2764
Commands	2764
ssm::createCorner	2765
ssm::getCornerName	2765
ssm::getCornerParams	2766
ssm::getCorners	2766
ssm::getDesVarVal	2766
ssm::getOutputs	2767
ssm::getScalarValue	2767
ssm::getSpecifications	2768
ssm::getSweepParams	2769
ssm::listDesVars	2769
ssm::setDesVars	2769
ssm::setScalarValue	2770
25. Text Editing Tcl	2771
Commands	2771
te::bindExternalNetlistView	2772
te::getActiveObject	2773
te::getCursor	2774
te::getTokens	2775
te::insertText	2776
te::removeText	2778
te::setCursor	2779
te::showDesignOptions	2779

	te::showExternalSyntaxCheckOptions	2780
	Objects	2780
	giDialog	2781
	teToken	2781
	Preferences	2782
26.	Technology Tcl	2786
	Preferences	2786
27.	UDD Editing Tcl	2787
	Commands	2787
	ue::cndbg	2788
	ue::compile	2788
	ue::debug	2789
	ue::getDesign	2790
	ue::preview	2790
	ue::searchAndReplace	2790
	ue::showOptions	2791
	ue::showSearchReplaceDialog	2791
	ue::updateCanvas	2792
	iue::align	2792
	iue::boolean	2793
	iue::copy	2793
	iue::debug	2793
	iue::fill	2794
	iue::move	2794
	iue::remove	2795
	iue::setOrigin	2795
	iue::size	2795
	iue::stretch	2796
	Preferences	2796
28.	Integrated Tool Tcl	2798
	Commands	2798
	xt::addJobClassPrefix	2806
	xt::addPlotPacket	2807

Contents

xt::annotateParasitics	2807
xt::cdplJobProc	2808
xt::cdplJobsProc	2809
xt::checkLegality	2809
xt::checkLVS	2810
xt::checkPGConnectivity	2811
xt::checkPGMissingVias	2812
xt::checkRoutability	2814
xt::checkRoutes	2816
xt::clearPlotPackets	2818
xt::clearViolations	2818
xt::closeSimRun	2818
xt::closeTextViewerTab	2819
xt::compareVersionString	2820
xt::configureCDPL	2820
xt::configureQueueEngine	2821
xt::connectProgressServer	2821
xt::createCDPL	2822
xt::createDebugSession	2823
xt::createFCFrame	2824
xt::createGPDRef	2827
xt::createICC2Frame	2828
xt::createICCFRAM	2831
xt::createIntegration	2833
xt::createJob	2833
xt::createJobClass	2836
xt::createJobGroup	2836
xt::createParDBfromQuery	2838
xt::createPDKConfig	2838
xt::createPlot	2839
xt::createPlotQueueItem	2840
xt::createPlotViewer	2840
xt::createPlotViewerDef	2841
xt::createPlotViewerInst	2841
xt::createPveJob	2842
xt::createPWJobClass	2842
xt::createQueueEngine	2843
xt::createSimRun	2845
xt::createSimulator	2846

Contents

xt::createTool	2846
xt::crossprobe	2847
xt::displayPlotSet	2848
xt::editFinalNetlist	2849
xt::editTemplate	2849
xt::exec3DIC	2849
xt::execICC	2850
xt::execICC2	2851
xt::execTerminal	2851
xt::executePve	2852
xt::exportFloorplan	2853
xt::exportToICC	2854
xt::exportToReport	2856
xt::findExecutable	2857
xt::findFeature	2858
xt::findFeatureManifest	2858
xt::generateSimOutputCbk	2859
xt::getAnalysisList	2859
xt::getCalibreSocketPort	2860
xt::getCallingJob	2860
xt::getCrossRefs	2860
xt::getCDPLStatus	2861
xt::getDebugSessions	2861
xt::getEquivalentNets	2862
xt::getGPDCorners	2862
xt::getGPDCouplingCapacitors	2863
xt::getGPDGroundCapacitors	2863
xt::getGPDLayers	2864
xt::getGPDOObjects	2865
xt::getGPDRefs	2865
xt::getGPDResistors	2866
xt::getInputFiles	2867
xt::getInputFilesTimeStamps	2867
xt::getIntegrations	2868
xt::getJobClasses	2868
xt::getJobClassOptionPrefName	2869
xt::getJobHostStatus	2869
xt::getJobs	2870
xt::getManifestFeatureValue	2870

Contents

xt::getNetlistInputFiles	2871
xt::getOpenFile	2871
xt::getOutOfDateFiles	2872
xt::getPackages	2873
xt::getPDKConfigs	2873
xt::getPDKLibs	2874
xt::getPlotViewers	2874
xt::getPveJobs	2875
xt::getPWJobClasses	2875
xt::getQueueEngines	2876
xt::getRunDirName	2876
xt::getSaveFile	2877
xt::getSelectedJobGroups	2877
xt::getSelectedJobs	2878
xt::getSimRuns	2878
xt::getSimulators	2879
xt::getTextViewerActiveTab	2879
xt::getTextViewerTabs	2880
xt::getTools	2880
xt::getWalkupPort	2881
xt::hideJobMonitorColumn	2881
xt::highlightViolation	2882
xt::initCalibreConnection	2883
xt::initializeQueueEngine	2883
xt::initIntegration	2884
xt::initRunDir	2884
xt::initSimulatorIntegration	2886
xt::invokeExecutableForVersion	2887
xt::isValidRunDir	2887
xt::launchJobs	2888
xt::launchPlotViewer	2888
xt::launchSimulatorCbk	2889
xt::launchViewerCbk	2889
xt::loadPackage	2890
xt::loadPlotSet	2890
xt::loadPlugin	2890
xt::loadPvePrefs	2891
xt::loadRunDir	2892
xt::loadSimRun	2893

Contents

xt::openFeatureManifest	2893
xt::openTextViewer	2894
xt::openTextViewerTab	2895
xt::plotWaveform	2895
xt::postProcessSimResults	2896
xt::processLPEResults	2897
xt::processMarkupTags	2898
xt::queryParasitics	2899
xt::quitPvDebugger	2901
xt::readGPDParasitics	2901
xt::readJob	2902
xt::refreshTextViewer	2903
xt::reloadJobs	2903
xt::removeGPDParasitics	2904
xt::removeJobClassPrefix	2904
xt::reportGPDConfig	2904
xt::reportGPDProperties	2905
xt::resetGPDConfig	2906
xt::runMultiplePveJobs	2906
xt::runNetListAndSimulate	2907
xt::runNetlister	2907
xt::runNetlisterCbK	2908
xt::runSimulator	2909
xt::runSimulatorCbK	2910
xt::saveJobClassOption	2910
xt::savePlotSet	2911
xt::saveSimRun	2912
xt::sendCommand	2912
xt::setGPDConfig	2913
xt::setGPDPathForRead	2914
xt_setSaveRequiredValue	2915
xt::show3DICLibraryAttributes	2915
xt::show3DICRefLibEditor	2916
xt::showCapReportDialog	2916
xt::showCheckRoutes	2917
xt::showColorDecompositionSetup	2917
xt::showCreateFrame	2918
xt::showCreateICC2Design	2918
xt::showCreateICCFRAM	2919

Contents

xt::showCreateNewClass	2919
xt::showCreateSchematicFromVerilog	2920
xt::showCrossTalkNetSelector	2920
xt::showDRCSetup	2921
xt::showExport	2921
xt::showFCFrontEndFlow	2922
xt::showFCLibraryAttributes	2922
xt::showFormalityVerification	2923
xt::showICC2LibraryAttributes	2923
xt::showICC2RefLibEditor	2923
xt::showICCPlaceAndRoute	2924
xt::showInitializeParasitics	2924
xt::showInitLoad	2925
xt::showInitNew	2925
xt::showJobClassesDefinition	2926
xt::showJobHostStatus	2926
xt::showJobManagementOptions	2927
xt::showJobMonitor	2928
xt::showLPESetup	2928
xt::showLVSSetup	2929
xt::showNetBasedCheck	2930
xt::showNetBasedShieldCheck	2930
xt::showNetBasedViaCheck	2930
xt::showOpenICC2	2931
xt::showParasiticsCompare	2931
xt::showParasiticsOptions	2932
xt::showPveAboutTools	2932
xt::showPVEDialog	2932
xt::showPveOptions	2933
xt::showPveResults	2933
xt::showQueryParasiticsReport	2934
xt::showReport	2935
xt::showRun3DIC	2935
xt::showRunICC2	2935
xt::showSessionJobOptions	2936
xt::showSetCalibreHighlightColorDialog	2936
xt::showTestbenchJobOptions	2937
xt::showValidateForDigital	2937
xt::showValidateForICC	2938

Contents

xt::signalJob	2938
xt::splitVersionString	2939
xt::startPvDebugger	2939
xt::testPackage	2940
xt::unloadPlugin	2941
xt::updateCDPL	2941
xt::updateJob	2942
xt::validateForDigital	2943
xt::validateForICC	2943
xt::viewRunLog	2944
xt::viewSimInput	2944
xt::viewSimOutput	2944
xt::wait	2944
xt::writeJob	2945
xt::writeParasitics	2945
ixt::crossProbe	2946
ixt::editPlotSet	2947
ixt::showGPDParasiticsOptions	2948
ixt::showGPDParasiticsOptions	2948
ixt::plotWaveform	2948
ixt::queryGPDParasitic	2949
ixt::queryParasitics	2949
Enumerations	2950
Objects	2951
deContext	2952
deProbe	2953
dmCell	2954
dmCellView	2954
giWindow	2955
nlNameMap	2955
nlNet	2956
nlNetlister	2957
oaInst	2960
oaInstTerm	2961
oaNet	2962
xtIntegration	2963
xtJob	2964
xtJobClass	2966

	xtJobGroup	2966
	xtJobLimit	2967
	xtPlot	2968
	xtPlotViewer	2969
	xtQueueEngine	2970
	xtSimRun	2971
	xtSimulator	2972
	Preferences	2976
29.	StarRC Tcl	3144
	Commands	3144
	rc::getCells	3145
	rc::getCouplingCapacitors	3145
	rc::getElmoreDelay	3146
	rc::getGPDCorners	3147
	rc::getGPDGroundCapacitors	3147
	rc::getGPDLayers	3148
	rc::getGroundCapacitors	3149
	rc::getNets	3150
	rc::getPins	3150
	rc::getPointToPointResistance	3151
	rc::getPorts	3152
	rc::getResistors	3152
	rc::readParasitics	3153
	rc::removeAnnotatedParasitics	3154
	rc::reportGPDConfig	3154
	rc::reportGPDProperties	3155
	rc::resetGPDConfig	3155
	rc::setGPDConfig	3156
	Preferences	3156
30.	Tcl Interface for Running Python Scripts, Blocks, and Expressions	3157
	py::eval	3157
	py::exec	3158
31.	Python Interface for Running Tcl Scripts	3160

tcl.eval()	3160
-------------------	------

32. Synopsys Live Recorder Tcl

Commands	3162
slr::flush	3162
slr::isActive	3163
slr::start	3163
slr::stop	3164

A. Deprecatd Elements

Deprecated Tcl Commands	3165
Deprecated Tcl Arguments	3167
Deprecated Tcl Argument Types	3170
Deprecated Objects	3171
Deprecated Object Attributes	3171
Deprecated Preferences	3172
Deprecated Actions	3181

About This Manual

The *Custom Compiler™ Tcl Reference Manual* covers Custom Compiler Tcl commands, objects, and preferences.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys® Custom Compiler™

Synopsys® PrimeWave™ Design Environment

PrimeSim™ Continuum

PrimeSim™

PrimeSim™ Pro

PrimeSim™ SPICE

PrimeSim™ HSPICE®

PrimeSim™ XA
PrimeSim™ CCK
PrimeSim™ MOSRA
PrimeSim™ EMIR
VCS® PrimeSim™ AMS
Synopsys® Custom Router
Synopsys® CustomSim™
Synopsys® CustomSim™ NanoSim®
Synopsys® CustomSim™ XA
Synopsys® WaveView™
Synopsys® IC Compiler™
Synopsys® FineSim®
Synopsys® Fusion Compiler™
Synopsys® HSPICE®
Synopsys® IC Validator
Synopsys® StarRC™
Synopsys® VCS®
Synopsys® Verdi®
Cadence® Spectre® Circuit Simulator
Cadence® Virtuoso® Spectre®
MathWorks® MATLAB
Mentor Graphics® Calibre®
Mentor Graphics® Calibre Interactive®

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
<code>Courier</code>	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to Custom Compiler Tcl

The Custom Compiler Tool Command Language (Tcl) supplements standard Tcl for designing and interacting with the Custom Compiler environment. Using the Custom Compiler Tcl, you can customize the user interfaces or automate design tasks.

This chapter covers the following topics:

- [Tcl Namespaces](#)
- [Displaying Command and Preference Descriptions](#)
- [Tcl Command Naming Conventions](#)
- [Tcl Argument and Return Types](#)
- [Creating Custom Tcl User Commands](#)
- [Tcl Filtering](#)
- [OpenAccess Tcl Interface Limitations](#)
- [Using Tcl in Tcl Bytecode Format](#)
- [Encrypting Tcl Files](#)

Tcl Namespaces

Custom Compiler Tcl provides commands in namespaces classified by usage category and named according to the type of usage, such as `de::` for design editing operations.

Note:

Si2 OpenAccess Tcl provides commands in the `oa::` namespace. For assistance with OpenAccess Tcl, see the Si2 documentation supplied with the Custom Compiler product installation. For information about accessing the OpenAccess Help, see the *Custom Compiler Environment User Guide*. For information about issues with OpenAccess Tcl interface, see [OpenAccess Tcl Interface Limitations](#).

Custom Compiler Tcl namespace names have two characters when the namespace provides commands for batch processing, and three characters, starting with `i`, when the namespace provides interactive commands. For example, the `ise::` namespace provides interactive schematic editing commands. An interactive command opens a toolbar to accept input during execution.

[Table 1](#) lists available namespaces and where to find more information in this manual.

Table 1 *Custom Compiler Tcl Namespaces and Usage Categories*

Namespace	For details, see:
cm	Constraint Management Tcl
cs	Integrated Tool Tcl for CustomSim
db, idb	Database Tcl and Technology Tcl
dc, idc	Design Collaboration Tcl
de, ide	Design Editing Tcl
dm	Data Management Tcl
dr	Display Resource Tcl
gi	Graphical Infrastructure Tcl
he	Hierarchy Editing Tcl
iPDK	iPDK Tcl Procedures
le, ile	Layout Editing Tcl
lp, ilp	Placement Tcl
lx, ilx	Schematic Driven Layout Tcl
ne	Netlist Editing Tcl
nl	Netlist Tool Tcl
po	Geometry Manipulation Tcl
rc	StarRC Tcl
rpl	Relative Placement Language Tcl
rt, irt	Routing Tcl
sa, isa	Simulation and Analysis Environment Tcl

Table 1 Custom Compiler Tcl Namespaces and Usage Categories (Continued)

Namespace	For details, see:
se, ise	Schematic Editing Tcl
ss	Simulation and Analysis Environment Scripting Tcl
ssm	MTB Simulation Scripting Tcl
te	Text Editing Tcl
ue	UDD Editing Tcl
wf	Waveform Tcl
xt, ixt	Integrated Tool Tcl

For more information, see [Tcl Command Naming Conventions](#) and [Tcl Argument and Return Types](#).

Displaying Command and Preference Descriptions

For descriptions of new or modified Tcl commands, type the following in the Console:

```
<command> -help
```

For example:

```
wf::createCustomPlotMode -help
```

For descriptions of new or modified Tcl preferences, type the following in the Console:

```
db::getAttr description -of [db::getPrefs <preference>]
```

For example:

```
db::getAttr description -of [db::getPrefs saMeasurementsCalculationMode]
```

Tcl Command Naming Conventions

Each Custom Compiler command is in a namespace and uses the following *camelBackNotation* convention as shown with the namespace qualifier:

```
<ns>::<verb><Object>
```

where <ns> is the namespace, <verb> is the action the command performs and <Object> (one or more) is the object type of the object upon which the command acts.

In this example:

```
sa::createCornersSetup
```

`sa` is the containing namespace, `create` is the verb action, and `CornersSetup` describes the object upon which the command acts.

Caution:

Private functions are all functions starting with an underscore (`_`) in the procedure or namespace. They are found in the `<product_installation>/auxx` directory. These procedures should be used as examples only, and Synopsys reserves the right to modify the procedures without warning.

For more information, see [Tcl Namespaces](#) and [Tcl Argument and Return Types](#).

Tcl Argument and Return Types

Custom Compiler Tcl commands accept the following data types for arguments and return values: basic data types, objects, enumerations.

This section covers the following topics:

- [Basic Data Types](#)
- [Object Types](#)
- [Enumerations](#)
- [Lists](#)

Basic Data Types

Custom Compiler Tcl basic data types support standard EDA data.

[Table 2](#) lists basic data types supported in Custom Compiler Tcl.

Table 2 Custom Compiler Tcl Supported Basic Data Types

Type	Description
<bool>	One of the following strings: "true", "on", "1", "yes" to represent true and "false", "off", "0" or "no" to represent logical false
<box>	A list of exactly two points representing diagonally opposite vertices of a rectangle. (See <point> in this table.)
<float>	Native Tcl floating point number

Table 2 *Custom Compiler Tcl Supported Basic Data Types (Continued)*

Type	Description
<int>	Native Tcl integer
<lpp>	Layer purpose pair. Either one element or a list of two elements. The first element is glob-style pattern representing layer name or layer number. The second element is a glob-style pattern representing purpose name or purpose number. If the second element is not specified, it defaults to a hardcoded value 'drawing' (-1).
<map>	A list of exactly two strings {key value} representing a mapping from a key to a value.
<matrix>	A list of lists of integers, where each sublist is the same length, representing a matrix of numbers in row-major format.
<point>	A list of exactly two floating point numbers: {x0 y0} representing coordinates in user units
<string>	Native Tcl string

Note:

Custom Compiler Tcl commands support value-less Boolean argument. If the default of the Boolean argument is true, you still need to specify false value explicitly if you want to change it.

For more information, see [Object Types](#) and [Tcl Argument and Return Types](#).

Object Types

Many Custom Compiler Tcl commands accept either OpenAccess or Custom Compiler objects as arguments. Find information about OpenAccess objects in the Si2 OpenAccess documentation. Find definitions for objects created for Custom Compiler use in each documented namespace in this manual.

Custom Compiler object type names follow the convention <ns><Type>, where 'ns' is the object's associated two-character namespace, and 'Type' is the object type.

For example, leMPP represents the MPP object, which is an oaAppDef object with an oaGroup of type MPP used in layout editing (le).

Attributes of objects can be retrieved using the db::getAttr command.

For more information, see [Semantic Object Types](#) and [Tcl Argument and Return Types](#).

Semantic Object Types

The Custom Compiler user interface adds additional semantics to some existing OpenAccess object types that derive from `oaFig`, effectively defining new object types in each editor. For example, in a schematic design, certain `oaPaths` or `oaLines` are treated as a *wire*, which is an object type that does not exist in the OA information model.

Wires and similar objects are collectively known as *Semantic Objects*. They represent a taxonomy that is more closely related to the design operations that are typically performed in the given editor.

The Binding Manager, the Property Editor assistant, the Object Layer Panel assistant and certain Tcl commands use semantic object types to represent object types in designs. For example, `db::getAttr type -of [de::getActiveFigure]` returns a semantic type name.

[Table 3](#) lists schematic and symbol semantic type objects and their prompts (if the prompt is different from the object name).

([Table 4](#) that follows lists layout semantic type objects and prompts.)

Table 3 *Tcl Semantic Object Types for Schematics and Symbols*

Object	Prompt (if different)
Arc	
Circle	
ConnectDefLabel	Connection Definition Label
DeviceShape	Device Shape
Ellipse	
FlightLine	Flight Line
Inst	Instance
InstConnectDefLabel	Instance Connection Definition Label
InstLabel	Instance Label
InstName	Instance Name
InstPin	Instance Pin
InstPinName	Instance Pin Name

Table 3 *Tcl Semantic Object Types for Schematics and Symbols (Continued)*

Object	Prompt (if different)
Label	
Line	
Marker	
Note	
NoteShape	Note Shape
Pin	
PinName	Pin Name
Polygon	
Rectangle	
SelectionShape	Selection Shape
Wire	

[Table 4](#) lists layout semantic objects and their prompts (if the prompt is different from the object name).

Table 4 *Tcl Semantic Object Types for Layouts*

Object	Prompt (if different)
AttributeText	Attribute Text
Blockage	
Boundary	
CircularRuler	Circular Ruler
CoordinateMark	Coordinate Mark
Ellipse	
FigureGroup	Figure Group
Flightline	Flight Line
Inst	Instance

Table 4 *Tcl Semantic Object Types for Layouts (Continued)*

Object	Prompt (if different)
LinearRuler	Linear Ruler
Marker	
MPP	Multi Part Path
Path	
PathSeg	Path Segment
Polygon	
PropertyText	Property Text
Rectangle	
Text	
Via	

For more information, see [Enumerations on page 91](#) and [Tcl Argument and Return Types](#).

Enumerations

Custom Compiler Tcl enumerations are fixed sets of predefined strings. Enumerations are named with the 'en' prefix followed the enumeration name.

For example, the `le::move` command requires an enumeration value to specify the transformation to be done during the move command operation.

The Custom Compiler `enOrient` enumeration offers values for the `le::move` command to use:

```
enOrient ::= R0 | R90 | R180 | R270 | MX | MY | MXR90 | MYR90
```

```
le::move {<dbObject>} -dx <int> -dy <int> -rotate <enOrient>
```

Enumeration values are case-sensitive strings.

For more information, see [Tcl Argument and Return Types](#).

Lists

Arguments and return values of Custom Compiler Tcl commands can be Tcl lists of basic data types or lists of lists. Note that any time a list is accepted for an argument, a single value is alternatively accepted, as a degenerate case of a single-element list.

In the following example, a list of points is passed to the `le::createBoundary` command:

```
le::createBoundary <points> -design <oaDesign> -type <enum>
```

For more information, see [Tcl Argument and Return Types](#).

Creating Custom Tcl User Commands

This section covers the following topics:

- [Command Registration](#)
- [Noninteractive Commands](#)
- [Interactive Commands](#)

Command Registration

Command registration of both interactive and noninteractive user commands includes the following:

- Command registration with the Tcl interpreter
- Command Line Input (CLI) integration to support tab completion and built-in help
- Argument name and value-type registration
- Licensing integration

Use `de::createCommand` to create and register a user-defined Tcl interactive or procedural command. See [de::createCommand on page 756](#) for syntax and examples.

Noninteractive Commands

Noninteractive commands, launched from the command line input or from your script, are fully configured by command-line options and do not require further inputs from you.

Registered noninteractive commands provide some advantages over a standard Tcl procedure:

- Named arguments
- Type checking
- Help text including `<command> -help`
- Mutually exclusive arguments

Creating Custom Noninteractive Commands

To implement a noninteractive command, you need:

- A Tcl procedure with the following signature:

```
proc myCmd::execute {args} {...}
```

- Previously created command option arguments. The list of argument name and value pair is passed to the `args` argument of the `myCmd::execute` procedure, and can be parsed as:

```
array set myArgs $args  
set optionValue $myArgs(-optionName)
```

- Argument groups for mutually exclusive arguments

Use the `de::createArgument` command to create a new argument definition for use with `de::createCommand`. See [de::createArgument on page 747](#) for syntax and examples.

The arguments provided to the command are passed directly to the procedure after type checking.

Interactive Commands

Interactive commands are event-driven. The set of events to which they are sensitive are called *semantic events* because they interpret GUI events as semantically meaningful instructions to the command.

An interactive command is activated in a design editor window by issuing the appropriate Tcl command, either directly at the command line input (CLI), by populating an action, or using a bindkey. The command returns nothing if the command is successfully activated or results in an error if not.

Note:

For further information about using Tcl interactive commands to create custom GUIs, see "Custom Graphic User Interfaces" in the *Custom Compiler Environment User Guide*.

This section covers the following topics:

- [Creating Custom Interactive Commands](#)
- [Event Handler Procedures](#)
- [Interactive Command Attributes](#)
- [Interactive Command Engines](#)
- [Example: Using Interactive Commands](#)

Creating Custom Interactive Commands

Interactive commands can be created either in the design or context scopes.

An interactive command has one nameless argument, the window id.

To implement an interactive command, use the following procedure:

1. Create a Tcl namespace whose name matches the name of the command and define event-handler procedures within that command-name namespace. For example:

```
namespace eval ins::commandName
    proc init { self context } {
        #Routine to initialize the command
    }
    proc addPoint { self context point } {
        #Routine to handle mouse click
    }
    ...
}
```

2. Register the interactive command using `de::createCommand`. For example:

```
de::createCommand ins::commandName -type
interactive -description "Command description" \
    -label "COT Label" -category layout
```

3. Create and add widget-type actions to the created Command Option Toolbar. For example:

```
proc _buildTextWidget { action } {
    return [gi::createText Input ...]
}

gi::createAction actionName -widgetProc
_buildTextWidget
gi::addAction actionName -to [gi::getToolbars
nsCommandNameOptions]
```

4. Create a Command Option Dialog by defining a Tcl procedure named `buildDialog` in the Tcl command namespace:

```
proc ins::commandName::buildDialog { self dialog } {  
    gi::createTextInput -parent $dialog ...  
}
```

Event Handler Procedures

The following sections describe namespace procedures that represent callbacks for the various state transactions in the command's lifespan.

- [init](#)
- [reinit](#)
- [isContextValid](#)
- [contextAdded](#)
- [contextRemoved](#)
- [toolbarCreated](#)
- [abort](#)
- [cancel](#)
- [addPoint](#)
- [removePoint](#)
- [completeShape](#)
- [startDrag](#)
- [endDrag](#)
- [addObject](#)
- [removeObject](#)
- [commandOption](#)

init

This is a required procedure, called when the command is invoked before any other procedures. The context argument corresponds to the context in which command was started. The command is not started if an error is thrown from the `init` procedure. The `init` procedure can be used to set command's arguments like `clientData`, `engine`, `canBeKilled`, and so forth.

Syntax:

```
proc init {deCommand context args} { ... }
```

Example:

```
proc init {self context} {  
    if {[db::getattr editDesign.viewType -of $context] !=  
        "maskLayout"} {  
        error "This command works only in layout editor"  
    }  
    db::setattr clientData -of $self -value { nPoints  
        point1 point2 0 }  
}
```

reinit

The `reinit` procedure is called when the command is invoked a second time. It is pushed out from the command stack.

Syntax:

```
proc reinit {deCommand args} { ... }
```

isContextValid

The `isContextValid` procedure can be set to return true for cross-context commands to allow the command to become active in the specified context. By default, the `isContextValid` procedure returns false in order not to allow the command become active for:

- All other designs for design-scoped commands
- All other contexts for context-scoped commands

Syntax:

```
proc isContextValid {deCommand deContext} { ... }
```

contextAdded

The `contextAdded` procedure is called for each additional context in which this command becomes active. An error issued by this procedure does not prevent activation within the context.

Syntax:

```
proc contextAdded {deCommand deContext} { ... }
```

contextRemoved

The `contextRemoved` procedure is called when the command is removed from a context. This happens immediately before being destroyed and when a cross-cellView command leaves a context.

Syntax:

```
proc contextRemoved {deCommand deContext} { ... }
```

toolbarCreated

The `toolbarCreated` procedure is called for each Command Option Toolbar in which the command has been activated. It can be used to enable or disable actions according to design state.

For example:

```
proc toolbarCreated {deCommand giToolbar} { ... }
```

abort

The `abort` procedure is called for each Command Option Toolbar in which the command has been activated. It can be used to enable or disable actions according to design state. It is called when the command is killed after being deactivated in all contexts in which it was activated. (Commands can also be terminated using the `de::abortCommand`.)

Syntax:

```
proc abort {deCommand} { ... }
```

cancel

The `cancel` procedure implements the semantic event handler that eliminates any runtime objects and returns the command to the start of its cycle. The event is generated by `de::cancelCommand`.

Syntax:

```
proc cancel {deCommand deContext} { ... }
```

addPoint

The `addPoint` procedure implements the add point semantic event handler. The event is generated by the `de::addPoint` command, which is bound to the left mouse button or space bar.

Syntax:

```
proc addPoint {deCommand deContext point} { ... }
```

removePoint

The `removePoint` procedure implements the remove point semantic event handler. The event is generated by the `de::removePoint` command, which is bound to the Backspace key.

Syntax:

```
proc removePoint {deCommand deContext point} { ... }
```

completeShape

The `completeShape` procedure implements the complete shape semantic event handler. The event is generated by the `de::completeShape` command, which is bound to the Enter key.

Syntax:

```
proc completeShape {deCommand deContext {point " "}} { ... }
```

startDrag

The `startDrag` procedure implements the start drag semantic event handler. The event is generated by the `de::startDrag` command.

Syntax:

```
proc startDrag {deCommand deContext point} { ... }
```

endDrag

The `endDrag` procedure implements the end drag semantic event handler. The event is generated by the `de::endDrag` command.

Syntax:

```
proc endDrag {deCommand deContext point} { ... }
```

addObject

The `addObject` procedure specifies an additional object to add to the selection set. The event is generated by the `de::addObject` command, which is bound to Shift + left mouse button.

Syntax:

```
proc addObject {deCommand deContext object} { ... }
```

removeObject

The `removeObject` procedure specifies an additional object to remove from the selection set. The event is generated by the `de::removeObject` command, which is bound to Ctrl + left mouse button.

Syntax:

```
proc removeObject {deCommand deContext object} { ... }
```

commandOption

The `commandOption` procedure implements the command option semantic event handler. The event is generated by the `de::commandOption` command. It can be used by any command that requires alternate semantics or event types in addition to those provided by the other semantic events.

Syntax:

```
proc removeObject {deCommand deContext point optionName}
{ ... }
```

Example:

```
namespace eval iuc::command {
  proc commandOption {self context point optionName} {
    if {"rotate" == $optionName}
      # do your operation
  }
}
gi: : createBinding -event Button3 -command "de::commandOption rotate" \
-activeOperation iuc::command
```

Interactive Command Attributes

The following sections describe the interactive command attributes.

- [canNest](#)
- [supportsInfix](#)

canNest

Permits the command to be used on top of other commands. Default: false

Syntax:

```
canNest- boolean
```

supportsInfix

Set to True to set an add point event at start if the infix mode preference is enabled.

Syntax:

```
supportsInfix- boolean
```

Interactive Command Engines

Interactive command response events are usually handled through command engines. Engines respond to these events by creating or manipulating runtime objects, modifying the command prompt and/or creating design objects.

Command engines are supplied for commonly used input models, including:

- Selecting an object
- Picking an object (without selecting it)
- Drawing shapes

A command engine can be created and set as an interactive command instantiation's `engine` attribute. For example:

```
db::setAttr engine -of $command -value [de::createSelectionEngine]
```

See the following sections for syntax and examples of creating command engines:

- [de::createShapeEngine](#)
- [de::createSelectionEngine](#)
- [de::createActivationEngine](#)

Example: Using Interactive Commands

The following example creates an `ile::createPathSeg` command for use in the second example.

```
namespace eval ile::createPathSeg {
    proc init {cmd context} {
        set vt [db::getAttr editDesign.viewType -of $context]
        if {$vt != "maskLayout"} {
            error"CreatePathSeg works sonly in layout editor"
        }
    }

    #Start shape engine
    db::setAttr engine -of $cmd -value \
        [de::createShapeEngine \
            -shapeType line -numPoints 2 \
            -angleMode [db::getPrefValue leCreateAngleMode \
                -scope $context] -pointsChangedProc \
                [namespace current]::prompt \
            -completeProc [namespace current]::shapeComplete ]
    db::setAttr cmd.prompt -value "Specify first point "
}
```



```

proc prompt {cmd} {
  set points [db::getAttr cmd.engine.points]
  if {[llength $points] == 1} {
    db::setAttr cmd.prompt -value "Specify next point \
      to complete"
  } else {
    db::setAttr cmd.prompt -value "Specify first point "
  }
}

proc shapeComplete { cmd } {
  set ctx [db::getNext [db::getAttr contexts -of $cmd]]
  set design [db::getAttr editDesign -of $ctx]
  set engine [db::getAttr engine -of $cmd]
  set points [db::getAttr points -of $engine ]
  set lpp [db::getAttr lpp -of [de::getActiveLPP \
    -design $design]]
  set width [db::getPrefValue leWidth -scope $ctx]
  set style [db::getPrefValue lePathSegstyle]
  le::createPathSeg $points -design $design -width \
    $width -lpp $lpp -beginStyle $style -endStyle $style
  db::setAttr cmd.prompt -value "Specify first point "

}

}
#Create preferences
db::createPref lePathSegstyle -value truncate

de::createCommand ile::createPathSeg -type interactive \
  -description "Create path seg" -category layout -label PathSeg

```

The following example decorates the `ile::createPathSeg` (created in the previous example) Command Options Toolbar with widget actions, allowing you to use different options.

```

proc _setAngleMode {widget} {
  set angleMode [db::getAttr widget.value]
  set cmd [de::getActiveCommand]
  db::setAttr engine -of $cmd -value \
    [de::createShapeEngine \
      -shapeType line -numPoints 2 \
      -angleMode $angleMode \
      -pointsChangedProc ile::createPathSeg::prompt \
      -completeProc ile::createPathSeg::shapeComplete ]
}

proc _buildWidthWidget {action} {
  return [gi::createNumberInput width -label "Width:" \
    -prefName leWidth -valueType float]
}

proc _buildAngleModeWidget {action} {

```

```
        return [gi::createMutexInput angle -label "Angle:" \
        -prefName leCreateAngleMode -enum {orthogonal diagonal} \
        -viewType combo -valueChangeProc _setAngleMode]
    }

    proc _buildExtStyleWidget {action} {
        return [gi::createMutexInput style -label "Style:" \
        -prefName lePathSegStyle -enum {truncate extend chamfer} \
        -viewType combo]
    }

    gi::createAction pathSegWidth -widgetProc _buildWidthWidget
    gi::createAction pathSegAngle -widgetProc _buildAngleModeWidget
    gi::createAction pathSegStyle -widgetProc _buildExtStyleWidget
    gi::addActions { pathSegWidth
                    giSeparator
                    pathSegAngle
                    giSeparator
                    pathSegStyle
    } -to [gi::getToolbars leCreatePathSegOptions]
```

Tcl Filtering

This section covers the following topic:

- [Standard Attribute Filters](#)

Standard Attribute Filters

Several Custom Compiler commands support a filter argument that allows the user to further refine the returned collection of objects. The filter argument takes an expression that is applied to each object potentially returned. Only the objects for which the expression evaluates to true is included in the returned collection.

The filter expression supports most operators supported by Tcl and also supports string matching with `=~` and `!~`.

The expressions can refer to attributes of Custom Compiler objects and can contain Tcl commands that may, for example, traverse relationships, or find OpenAccess attributes to be evaluated by the expression.

The filter expression is similar to a Tcl `expr` argument and is usually specified with braces such as in the following example:

```
-filter {(%attr == string) && (%otherAttr > $num)}
```

This section covers the following topics:

- [Operators](#)
- [Attributes](#)
- [Tcl Variables and Tcl Commands](#)
- [Filter Examples](#)

Operators

[Table 5 on page 103](#) lists Custom Compiler Tcl operators supported by the filter expression.

Table 5 Custom Compiler Tcl Operators Supported in Filter Expressions

Supported Operator Type	Operator	Notation
Comparison	equal	==
	not equal	!=
	greater than	>
	greater than or equal	>=
	less than	<
	less than or equal	<=
Regular expression comparisons	matches	=~
	does not match	!~
Logical operators	Boolean and	&&
	Boolean or	
	Boolean not	!
Arithmetic operators	addition	+
	subtraction	-
	multiplication	*
	division	/
Parentheses for grouping		()
Brackets for Tcl commands		[]

On numeric types, the comparison operators have the standard mathematical behavior. On strings, they compare with an alphanumerical ordering.

The alphanumerical ordering compares non-numeric characters with standard lexicographical ordering, but embedded numbers are compared numerically.

Examples of alphanumerical ordering are listed as follows:

- `a < b`
- `a2 < a10`
- `a1a < a12`

The operator precedence is the same as for Tcl `expr` commands, with `=~` and `!~` operators having the same precedence as `==` and `!=` operators.

The regular expressions supported by `=~` and `!~` also support the POSIX® standard. The equivalent functionality is roughly that of the Unix `egrep` command.

It is possible to embed an arbitrary Tcl command within a filtering expression using the standard Tcl square bracket construct. The output of such a command will be substituted for the string in brackets, as with any other Tcl command.

Note:

It is not allowed to use regular expression matching operators inside any bracketed commands. This is because the operators `=~` and `!~` are not supported natively by Tcl.

Return to [Standard Attribute Filters on page 102](#)

Attributes

Custom Compiler Tcl object attributes are available in the filter expression by preceding the attribute name by a percent sign.

For example, the following command:

```
de::getSelected -design [ed] -filter {%name =~ /^foo/ && %objType == "Inst"}
```

returns selected objects with a name that starts with foo and that have a type of Inst.

For object attributes that are int, string, or float, all the operators listed in [Table 5 on page 103](#) are available. If the attribute is an object, only equality operators (`==` and `!=`) are available. Attempts to use the other operators on objects return a `TCL_ERROR`.

Return to [Standard Attribute Filters on page 102](#)

Tcl Variables and Tcl Commands

Custom Compiler Tcl variables are available in the filter expressions. Reference them with `$variableName`. Tcl Commands can also be used by embedding them with `[ ]` as with other Tcl statements.

Return to [Standard Attribute Filters on page 102](#)

Filter Examples

This example filters the collection of all `instTerms` in a design down to those connected to global nets:

```
db::getInstTerms -filter {[oa::isGlobal [db::getAttr net -of \ %this]]}  
-of $design
```

This example filters the collection of all nets to exclude those nets whose names start with `vdd`:

```
db::getNets -filter {%name !~ /^vdd/} -of $design
```

This example filters the collection of all unsaved preferences that have a name starting with `sa`:

```
db::getPrefs sa* -filter {%modified}
```

Return to [Standard Attribute Filters on page 102](#)

OpenAccess Tcl Interface Limitations

The Tcl commands in the `oa::` namespace provide direct access to all OpenAccess classes and functions for Tcl writers. Although powerful, this capability can produce unexpected results.

The OpenAccess Tcl interface is a standard interface supplied by the Si2 OpenAccess Coalition. It is subject to problems associated with native C++ calls to OpenAccess.

In particular,

- There are many ways you can invoke Tcl commands that are deemed illegal in OpenAccess and consequently crash Custom Compiler.
- Other Tcl commands are known to have bugs that will crash the application.

The following list describes the most common causes of crash:

- Using a stale collection or iterator

When Tcl loops through a set of objects in OpenAccess it uses an iterator. For example, in the following command sequence the variable *shapes* is an iterator:

```
set block [oa::getTopBlock $design]
set shapes [oa::getShapes $block]
while {[set shape [oa::getNext $shapes]] != ""} {...
```

If this design is subsequently closed:

```
oa::close $design
```

then the use of *\$shapes* can cause a crash.

- Incorrect value type commands for parameters

OpenAccess requires that `getValue` calls for parameters to use the function that corresponds to the parameter's type. For example, it is illegal to use `oa::getIntVal` on a float Param. Using the wrong command throws an exception with the debug OA libraries and causes a crash with the optimized libraries. Scripts that refer to parameter values should always test the type of the parameter.

- Invalid array indexes

A number of OpenAccess classes are arrays. These classes inherit from `oaArrayBase` for example, `PointArrays` and `SitePatterns`. Using an out-of-range index with an array can result in bad data or cause a crash. Valid indexes are from 0 to

```
oa::getNumElements $array.
```

- Arguments of type `oaChar*` or `oaByte*`

Functions with arguments of these types do not work correctly in `oaTcl` and can crash Custom Compiler. For example, `oa::AppPropCreate`, as well as `setValue` and `getValue` on an `oaAppProp`, cannot be used. Consequently, there is no way to use `oaAppProps` in `oaTcl`. This applies as well to the constructor for an `oaParam` of type `appParam` and calling `oa::getAppVal` on such an `oaParam`.

- `getElements`

Each array has a `getElements` command that does not have a useful role in Tcl scripts. Using this command can crash Custom Compiler.

- Static functions using coordinates

The Tcl interface expects you to enter coordinates in user units (UU), where 2.2 might represent 2.2 microns and might be entered in the database as 2200 DBU (Data Base Units). In this case, the Tcl interface must find a technology database that specifies the `DBUPerUU` ratio to use.

However, there are a few commands that use coordinates (usually in Points or Boxes), but which do not refer to a database object. Consequently, the integration cannot find the DBUPerUU ratio to apply. These commands have been fixed to use the default maskLayout DBUPerUU, which is 1000.

- Creating a Marker with an empty PointArray

Attempting to use `oa::MarkerCreate` with an empty PointArray results in a crash.

- Using an incorrect object type

In most cases, the Tcl interface checks the types of the objects used for command arguments, and reports errors if the types are illegal. However, substituting a utility object (such as an enum object) where a managed object (persistent) is expected, results in a crash. In many cases, this includes using a string in place of the correct OA object.

- Values returned by reference

In several cases, a C++ function that returns a value in an argument by reference is handled incorrectly and results in a crash. This is known to occur in the special case iterators listed below that return an extra value:

```
oa::getNext DirIter
oa::getNext MemNetIter
oa::getNext ModMemNetIter
oa::getNext OccMemNetIter
oa::getNext EnumPropIter
```

- Iterating over OA collections

Collections returned by native OA functions, such as the collection returned by calling `oa::getShapes` on an `oaBlock`, are not Tcl collections. Using the Tcl for each of these objects can result in a crash. You must either set up a while loop that calls `oa::getNext` on the collection, or you can use `db::foreach` to loop over its objects.

- Several functions that require non empty strings as input, crash if handed an empty string. This include `oa::LibFind` and `oa::ScalarNetFind`.

Using Tcl in Tcl Bytecode Format

Tcl bytecode format protects the proprietary information contained in Tcl code. Bytecode format files have the file extension `.tbc`.

You can use Tcl delivered to you in Tcl bytecode format from other organizations just like you use Tcl in Tcl format.

You can also export your own Tcl in bytecode format to keep its proprietary information secret.

- [Executing Tcl Received in Tcl Bytecode Format](#)
- [Exporting Tcl in Tcl Bytecode Format](#)

These sections cover:

- [Executing Tcl Received in Tcl Bytecode Format on page 108](#)
- [Exporting Tcl in Tcl Bytecode Format on page 108](#)

Executing Tcl Received in Tcl Bytecode Format

Tcl bytecode files can be sourced like any Tcl script. The Custom Compiler environment ships with the bytecode interpreter, which executes the Tcl bytecode as if it were Tcl.

For example, you can source the file `myTclCode.tbc` by inputting `source myTclCode.tbc` to the Console.

You can also place Tcl bytecode files anywhere Tcl files are normally placed, for automatic interpretation on the Custom Compiler environment.

Exporting Tcl in Tcl Bytecode Format

When you ship Tcl to external organizations, you can keep its proprietary information secret by exporting it in Tcl bytecode format.

The Custom Compiler environment supports the protection of Tcl source code through the use of ActiveState's Tcl Dev Kit (TDK) Compiler product.

The ActiveState compiler generates Tcl bytecode (.tbc) files, which can be sourced like any Tcl script. The Custom Compiler environment also ships with the bytecode interpreter, which executes the Tcl bytecode as if it were Tcl.

To export Tcl in Tcl bytecode format:

1. From the ActiveState website, obtain the Tcl Dev Kit Compiler product.
Find instructions for downloading and using the ActiveState Tcl Dev Kit Compiler on the ActiveState website. The *Tcl Dev Kit User Guide* has additional information.
2. Use the Tcl Dev Kit Compiler to compile your Tcl into Tcl bytecode format.
3. Ship the bytecode format files as you would ship Tcl files.
4. Alert organizations receiving the bytecode that although the filename extension is different, bytecode format files can be treated in the same manner as Tcl.

Encrypting Tcl Files

Description

You can use the `synenc` shell command to encrypt files that contain Tcl commands. Encrypting Tcl files prevents users from seeing the file contents.

`synenc` produces encrypted Tcl files that have the `.tcl.e` file name extension. To source the files that `synenc` encrypts, use the `db::source` or the `db::init` command.

Syntax

```
synenc [-o <string>] [-lakerLicense] file1 [file2...fileN]
```

Arguments

`-o`

Name of the output file.

When sourcing the encrypted file, make sure you include the `.tcl.e` file name extension (for example, `db::source myEncryptedFile.tcl.e`).

`-lakerLicense`

Laker license information.

`file`

Name of the Tcl file that you want to encrypt. You must specify at least one file. You can optionally specify more files.

Examples

The following example shows how to use the `synenc` command in a linux shell to create a `utilities.tcl.e` encrypted file, which can be sourced using the `db::source` or `db::init` command:

```
synenc utilities.tcl
```

2

Integrated Tool Tcl for CustomSim

This chapter lists commands, enumerations, objects, and preferences in the Custom Compiler Tcl cs and ics namespaces. These namespaces provide Tcl for CustomSim and external tool integration. Interactive commands are listed at the end of the chapter.

For details, see:

- [Commands](#)
- [Preferences](#)

Commands

Interactive commands are listed after non-interactive commands in this section.

The commands covered are:

- [cs::createBoundary](#)
- [cs::exportCSV](#)
- [cs::importRAREsults](#)
- [cs::importSPRESResults](#)
- [cs::loadNpfForShortestPath](#)
- [cs::mergeMultiCCDB](#)
- [cs::showFlowSelection](#)
- [cs::showInstancePins](#)
- [cs::showSelectResults](#)
- [cs::showSetupDialog](#)
- [cs::showShortestPathOverlays](#)
- [cs::showSPRESP2PQuery](#)

- [cs::showTruncateDbDialog](#)
 - [cs::showWhatIfDialog](#)
 - [cs::traceLRP](#)
 - [cs::traceShortestPath](#)
 - [cs::truncateDb](#)
 - [ics::addResistors](#)
-

cs::createBoundary

Description

Creates a boundary based on the violation bounding area.

Syntax

```
cs::createBoundary [-context <deContext>] [-points [<point>]]
```

Arguments

-context

Context for editor window.

-points

Vertex coordinates of the boundary.

cs::exportCSV

Description

Exports current database table in the RA viewer to a CSV file.

Syntax

```
cs::exportCSV -parent <giWindow> -dbPath <string> -tableName <string>
```

Arguments

-parent

Parent window.

-dbPath

Database path.

`-tableName`

Name of the current reliability analysis results browser.

cs::importRAResults

Description

Imports CustomSim RA results into the OA database.

Syntax

```
cs::importRAResults <prefix: string> -runDir <string> [-sourceDesign  
  <string>] -net <string> -analysis <string> [-open <bool>] [-parent  
  <giWindow | int>] -libDestName <string>
```

Arguments

`prefix`

Prefix of the output files.

`-runDir`

CustomSim RA run directory.

`-sourceDesign`

Libray/Cell/View of the source design.

`-net`

Net or pattern.

`-analysis`

Analysis type.

`-open`

Opens the imported design in the design window. Default: 0

`-parent`

Custom Compiler Console window or Layout Editor window.

`-libDestName`

The Library name into which the results should be imported.

Returns

`<bool>` : Status of the import run.

cs::importSPRESResults

Description

Imports CustomSim SPRES results into the OA database.

Syntax

```
cs::importSPRESResults <prefix: string> -runDir <string> [-sourceDesign  
  <string>] -net <string> [-open <bool>] [-parent <giWindow | int>]  
  -libDestName <string>
```

Arguments

prefix

Prefix of the output files.

-runDir

SPRES run directory.

-sourceDesign

Library or Cell or View of the source design.

-net

Net or pattern.

-open

Opens the imported design in the design window (Default: 0).

-parent

Custom Compiler Console window or Layout Editor window.

-libDestName

The Library name into which the results should be imported.

Returns

<bool> : Status of the import run.

cs::loadNpfForShortestPath

Description

Loads an SPF or NPF file for calculating the shortest path traces.

Syntax

```
cs::loadNpfForShortestPath <file: string> [-net <string>] [-netDump  
    <string>]
```

Arguments

file

Path to the SPFor NPF file. An empty path will remove previously loaded data.

-net

Loads the SPF or NPF data for a specific net. Default: loads data for all nets

-netDump

Dumps net stats.

Returns

<none>

cs::mergeMultiCCDB

Description

Merges ccdb databases from multiple vectors.

Syntax

```
cs::mergeMultiCCDB (-emirView <string> | -outputFile <string>) -input  
    <string> [-scratchDir <string>] [-sourceDesign <string>]
```

Arguments

-emirView

Specifies the output EMIR library or cell or view.

-outputFile

Specifies the output file path.

-input

Specifies the input file that contains the label pattern and directory structure.

[-scratchDir <string>]

Specifies the scratch directory.

-sourceDesign

Specifies the source layout design library or cell or view.

Returns

<none>

cs::showFlowSelection

Opens the EMIR Flow Selection Options dialog box.

Syntax

```
cs::showFlowSelection -parent <giWindow | int>
```

Arguments

-parent

Opens the dialog box in the Layout Editor window.

Returns

<giDialog> : EMIR Flow Selection Options dialog box

cs::showInstancePins

Shows instance pins.

Syntax

```
cs::showInstancePins -parent <giWindow> -operationType <enum> [-netNames  
[<string>]]
```

Arguments

-parent

Reliability Result Browser window.

-operationType

Type of operation: { hide | inViewport | specifyRange }.

-netNames

Names of the nets.

Returns

<none>

cs::showSelectResults

Description

Opens the Reliability Analysis Results Importer.

Syntax

```
cs::showSelectResults -parent <giWindow | int> -resultsType  
  <csResultsTypeEnum> [-resultsDir <string>]
```

Arguments

-parent

Layout Editor window.

-resultsType

Type of results to open: RA or SPRES.

-resultsDir

Directory with database files.

Returns

<giDialog> : Import results dialog

cs::showSetupDialog

Description

Opens the EMIR Setup dialog box.

Syntax

```
cs::showSetupDialog -parent <giWindow | int>
```

Arguments

-parent

Layout Editor window

Returns

<giDialog> : EMIR Setup dialog box.

cs::showShortestPathOverlays

Description

Displays options for the shortest path overlays.

Syntax

```
cs::showShortestPathOverlays [-delete <bool>] [-context <deContext>]  
    [-show <bool>] [-toggle <bool>] [-showResistors <bool>]
```

Arguments

-delete

Deletes all overlays.

-context

Context of the design editor window.

-show

Show or hides all overlays. Default: true

-toggle

Switches overlays, showing one at a time.

-showResistors

Shows or Hides resistors.

Returns

<none>

cs::showSPRESP2PQuery

Description

Opens the CustomSim SPRES Point to Point Query dialog box.

Syntax

```
cs::showSPRESP2PQuery -parent <giWindow | int>
```

Arguments

-parent

Custom Compiler Console or the Layout Editor window.

Returns

<giDialog> : Dialog object

cs::showTruncateDbDialog

Description

Opens the **ccdb Database Truncate** dialog box.

Syntax

```
cs::showTruncateDbDialog [-dbPath <string> | -cellView <dmCellView |  
dmCellViewName>] -parent <giWindow | int>
```

Arguments

-parent

The Layout Editor window.

-dbPath

The path to the database folder.

-cellView

The results cellview.

Returns

<giDialog> : database truncate dialog

cs::showWhatIfDialog

Description

Opens the **What-If Analysis** dialog box.

Syntax

```
cs::showWhatIfDialog -parent <giWindow | int>
```

Arguments

-parent

Results Viewer Window.

Returns

<giDialog> : What-If dialog

cs::traceLRP

Description

Traces path of least resistance.

Syntax

```
cs::traceLRP -window <giWindow> [-src [<string>]] [-clean <bool>] [-show  
<bool>]
```

Arguments

-window

Results browser window.

-src

Names of the source points for resistance calculations.

-clean

Removes all traced paths and overlays. Default: false

-show

Displays the Viewer dialog box. Default: false

cs::traceShortestPath

Description

Highlights the path of least resistance.

Syntax

```
cs::traceShortestPath -net <string> [-file <string>] [-context  
<deContext>] -n1 [<string>] [-n2 [<string>]] [-debug <bool>] [-style  
<enum>]
```

Arguments

-net

Net name

-file

Path to the NPF file.

-context

Design context for showing the results.

-n1

Source node(s). Finds the shortest path from all source nodes to a single target node.

-n2

Target candidate nodes for the shortest path calculation. Default: pin nodes from the NPF file

-debug

Dumps traced path information.

-style

Highlight style. ({ box | line })

Returns

<string> : shortest path between subnodes.

cs::truncateDb

Description

Removes violations from the CustomSim RA database.

Syntax

```
cs::truncateDb (-analyses [<string>] | -netAnalysisPairs [<string>])  
  [-dbPath <string>] [-net [<string>]] -threshold <string>
```

Arguments

-dbPath

Opens with selected database.

-analyses

Specifies which analyses to process.

-net

Specifies the nets on which the analyses were run.

-netAnalysisPairs

List of net analysis pairs to truncate.

-threshold

Threshold in which to keep values. All values below the specified threshold will be removed.

ics::addResistors

Description

Selects resistor shapes.

Syntax

```
ics::addResistors [<context: deContext | unsigned int>] -parent  
  <giDialog> -mode <csWhatIfResSelModeEnum>
```

Arguments

context

The context in which this command is activated.

-parent

What-If Analysis dialog box.

-mode

Resistor Selection Mode: Add or Modify.

Returns

<none>

Preferences

[Table 6](#) lists the cs namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 6 Integrated Tool Preferences

Name	Type	Scope	Default	Description
csActiveViolationInfo	bool	user	false	Shows resistor information when the pointer rests on a resistor.
csEnableMultiAnalysisInterface	bool	user	true	Enables the MultiAnalysis Interface for PrimeSim EMIR and Static Res Check results. Valid values: true, false

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csRAAnalysis	string	cellview		PrimeSim EMIR analysis.
csRABin	string	cellview		PrimeSim EMIR bins.
csRADestLib	string	cellview		PrimeSim EMIR destination library.
csRAExtractedLayers	string	cellview		PrimeSim EMIR extracted layers.
csRAHiddenColumns	string	cellview		PrimeSim EMIR hidden table columns.
csRANet	string	cellview		PrimeSim EMIR net or pattern.
csRAPage	int	cellview	1	PrimeSim EMIR page.
csRAPageSize	int	cellview	50	PrimeSim EMIR results page size.
csRAResultsDialogGeometry	string	cellview	700x475+6+40	PrimeSim EMIR dialog width.
csRARun	string	cellview		PrimeSim EMIR run.
csRARunDir	string	cellview		PrimeSim EMIR run directory.
csRASetupCcdbacpcEnabled	bool	cellview	false	Enables the cddbmapacpc RA result filter option. Valid values: true, false
csRASetupCcdbacpcFilter	string	cellview	level	cddbmapacpc RA results filter option. Valid values: level, iimax, jjmax
csRASetupCdbacpcValue	string	cellview		Threshold of the RA cddbmapacpc result filter option.
csRASetupCdbbemEnableAll	bool	cellview	true	Includes electromigration data in all cddb output.

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csRASetupCcdbiabsEnabled	bool	cellview	false	Enables the <code>ccdbmapiabs</code> RA result filter option. Valid values: true, false
csRASetupCcdbiabsFilter	string	cellview	level	<code>ccdbmapiabs</code> RA results filter option. Valid values: level, iimax, jjmax
csRASetupCcdbiabsValue	string	cellview		Threshold of the <code>ccdbmapiabs</code> RA result filter option.
csRASetupCcdbiavgEnabled	bool	cellview	false	Enables the <code>ccdbmapiavg</code> RA result filter option. Valid values: true, false
csRASetupCcdbiavgFilter	string	cellview	level	<code>ccdbmapiavg</code> RA results filter option. Valid values: level, iimax, jjmax
csRASetupCcdbiavgValue	string	cellview		Threshold of the <code>ccdbmapiavg</code> RA result filter option.
csRASetupCcdbimaxEnabled	bool	cellview	false	Enables the <code>ccdbmapimax</code> RA result filter option. Valid values: true, false
csRASetupCcdbimaxFilter	string	cellview	level	<code>ccdbmapimax</code> RA results filter option. Valid values: level, iimax, jjmax
csRASetupCcdbimaxValue	string	cellview		Threshold of the <code>ccdbmapimax</code> RA result filter option.
csRASetupCcdbirdEnableAll	bool	cellview	true	Includes IR drop data for all ccdb output.

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csRASetupCcdbirmsEnabled	bool	cellview	false	Enables the <code>ccdbmapirms</code> RA result filter option. Valid values: true, false
csRASetupCcdbirmsFilter	string	cellview	level	<code>ccdbmapirms</code> RA results filter option. Valid values: level, iimax, jjmax
csRASetupCcdbirmsValue	string	cellview		Threshold of the <code>ccdbmapirms</code> RA result filter option.
csRASetupCcdbshEnableAll	bool	cellview	true	Includes information about shapes (including net names) that have EM or IR drop violations in all <code>ccdb</code> output.
csRASetupCcdbMerge	bool	cellview	false	Combines individual <code>.cc.db</code> files in one file or one cellview so that you can access multiple analyses in the Custom Compiler Reliability Analysis Results Browser. Valid values: true, false
csRASetupCcdbMergeAll	bool	cellview	false	Merges all output <code>.cc.db</code> files. Valid values: true, false
csRASetupCcdbMergeRemove	bool	cellview	false	Removes merged output <code>.cc.db</code> files. Valid values: true, false
csRASetupCcdbOpen	bool	cellview	true	Opens result database in the RA Results Browser after generating. Valid values: true, false
csRASetupCcdbOutputFile	string	cellview		Path of the results database.

Table 6 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
csRASetupCcdbOutputSelection	string	cellview	CellView	Saves Results Database in File or CellView
csRASetupCcdbResultsView	lcv	cellview		Saves Results Database in the CellView
csRASetupCcdbSourceDesign	lcv	cellview		Source Design of the Result Database
csRASetupCcdbvmaxEnabled	bool	cellview	false	Enables the <code>ccdbmapvmax</code> RA result filter option. Valid values: true, false
csRASetupCcdbvmaxFilter	string	cellview	level	Type of the <code>ccdbmapvmax</code> RA result filter option. Valid values: level, value
csRASetupCcdbvmaxValue	string	cellview		Threshold of the <code>ccdbmapvmax</code> RA result filter option.
csRASetupCornerName	string	cellview		Name of the corner analysis simulation. The default is the first corner analysis encountered in the Galaxy Parasitic Database (GPD).
csRASetupCustomOptionsFile	string	cellview		Custom options file to include.
csRASetupCustomPhase2Commands	string	cellview		Custom Step II (phase II) commands.
csRASetupDeltaTLayerMap	string	cellview		DeltaT layer map file.
csRASetupDevMaxDTemp	string	cellview		Maximum device temperature rise.

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csRASetupEmCCDBCols	string	cellview	{current current true} {curdns curdns true} {i_imax i_imax true} {j_jmax j_jmax true} {threshold threshold true} {time time true} {emlen emlen true} {width_new width_new true} {area_new area_new true} {emwirelen1 emwirelen1 true} {emwirelen2 emwirelen2 true} {emwirewidth1 emwirewidth1 true} {emwirewidth2 emwirewidth2 true} {pulse_width pulse_width true} {signal_period signal_period true} {dratio dratio true} {eflow eflow true} {resfit resfit true} {dtj dtj true} {dtepl dtepl true}	Columns to be listed in cddb output for electromigration analysis.
csRASetupEmFiles	string	cellview		Electromigration rule files.
csRASetupFinesimBAOptions	string	cellview		Back-annotation commands for the PrimeSim flow.
csRASetupFinesimConfigOptions	string	cellview		Commands for the PrimeSim flow.
csRASetupFineSimNetlist	string	cellview		Top-level PrimeSim netlist for PrimeSim one-pass flows.
csRASetupFineSimNetlistFormat	string	cellview		The format of the netlist for PrimeSim one-pass flows.

Table 6 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
csRASetupFineSimRunDir	string	cellview		PrimeSim run directory for PrimeSim one-pass flows.
csRASetupGdsLayerMap	string	cellview		Layer map for generating OD GDS.
csRASetupGndNets	string	cellview		List of ground nets.
csRASetupGpdPath	string	cellview		Path to the design Galaxy Parasitic Database (GPD).
csRASetupHirShapeExtension	string	cellview		HiR shape extension.
csRASetupInstPinTol	float	cellview	0.1	Activates the reuse flow.
csRASetupInternalPowerNets	string	cellview		Internal power nets.
csRASetupIrdCCDBCols	string	cellview	{dV dV true} {voltage voltage true} {time time true} {nodename nodename true} {dvperc dvperc true}	Columns to be listed in cddb output for IR drop analysis.
csRASetupIrdLowerB	string	cellview		Specify the IR drop lower boundary threshold
csRASetupIrdScaleRange	string	cellview		Vary the map scale range
csRASetupIrdTime	string	cellview		Specify the IR drop time duration
csRASetupIrdUpperB	string	cellview		Specify the IR drop upper boundary threshold
csRASetupItfLayerNames	bool	cellview	false	Uses ITF layer names. Valid values: true, false
csRASetupMapType	string	cellview		Map type.
csRASetupNetlistFormat	string	cellview	HSPICE	Input netlist format
csRASetupOdev	string	cellview		ODEV.

Table 6 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
csRASetupOdGdsFile	string	cellview		OD GDS file.
csRASetupOdGdsInput	string	cellview	File	Source of the OD GDS input: file or layout. Valid values: File, Layout
csRASetupOdLayerName	string	cellview		OD layer name for generating OD GDS.
csRASetupOdShapeExtension	string	cellview		OD shape extension.
csRASetupOutputFormat	string	cellview	wdf	Waveform output format.
csRASetupOverwriteSAEState	bool	cellview	true	Overwrites selected state. Valid values: true, false
csRASetupPhaseToRun	string	cellview	I + II	PrimeSim EMIR step (phase) to run. Valid values: I, II, I + II
csRASetupPodeSuffix	string	cellview		PODE suffix.
csRASetupPowerNets	string	cellview		List of power nets.
csRASetupPrintI	string	cellview		Resistor currents Saved.
csRASetupPrintIPad	string	cellview		Net pad currents saved.
csRASetupPrintV	string	cellview		Node voltage saved.
csRASetupProcessNode	string	cellview		Process node.
csRASetupPwraAnalyses	string	cellview	vmax irms iavg	List of analyses for PWRA.
csRASetupRaDtemp	string	cellview		RA dtemp.
csRASetupRaFormat	string	cellview	ccdb	Raformat.
csRASetupRaedfFile	string	cellview		PrimeSim RAEDF file for PrimeSim one-pass flow.
csRASetupRaLayers	string	cellview		Bin names.

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csRASetupRaRunDir	string	cellview	<file path>	RA run directory for PrimeSim one-pass flow
csRASetupRasimStart	string	cellview		RASIM start time.
csRASetupRasimStop	string	cellview		RASIM stop time.
csRASetupRaStep	string	cellview		RA step (ratau).
csRASetupRaStep	string	cellview		RA step (ratau).
csRASetupRaWinStart	string	cellview		RA start time in seconds.
csRASetupRaWinStop	string	cellview		RA stop time in seconds.
csRASetupRefVol	string	cellview		Reference Voltage for IR drop percentage calculation
csRASetupRefVoltage	string	cellview		Reference voltage for PrimeSim one-pass flow.
csRASetupResultsDir	string	cellview		results directory.
csRASetupReuseFlow	bool	cellview	false	Activates the reuse flow. Valid values: true, false
csRASetupSaeReuseMasterState	lcv	cellview	<file path>	Master state for the reuse flow.
csRASetupSaeState	lcv	cellview	<file path>	Simulation configuration state that is saved.
csRASetupSelfHeatingFlow	bool	cellview	false	Enables self-heating flow. Valid values: true, false

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csRASetupShCCDBCols	string	cellview	{resname resname true} {resistance resistance true} {layer layer true} {width width true} {area area true} {length length true} {nodes nodes true} {netname netname true}	Columns to be listed in cddb output for self-heating analysis.
csRASetupSheAnaTypes	string	cellview		List of analyses for self-heating flow.
csRASetupSheRatcl	string	cellview		SHE RATCL.
csRASetupSheXmlFile	string	cellview		SHE XML file.
csRASetupSigNets	string	cellview		List of signal nets.
csRASetupSigraAnalyses	string	cellview	irms iavg	List of analyses for SIGRA.
csRASetupSimEngine	string	cellview	PrimeSim	Simulator for phase1 b analysis.
csRASetupSimLvl	int	cellview	3	RA Sim level.
csRASetupSimStopTime	string	cellview		Simulation stop time.
csRASetupSimTimeStep	string	cellview		Simulation time step.
csRASetupSpfFile	string	cellview		SPF file to use.
csRASetupSpfpostOut	string	cellview		Analyses option for PrimeSim one-pass flow
csRASetupTestbench	lcv	cellview	<file path>	Testbench design on which to run the analyses.
csRASetupUseExistingSPF	bool	cellview	true	Uses existing SPF file. Valid values: true, false
csRASetupUseGPD	bool	cellview	false	Uses the Galaxy Parasitic Database (GPD) as the back-annotation file.

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csRASetupVmaxminavg	bool	cellview	false	Turns on the RA vmaxminavg option.
csRAViewerConfigFile	string	user		Configuration file that is loaded automatically by the Reliability Analysis Results Browser.
csRAViewerLRPShowNonPhysical	bool	user	false	Show non-physical shapes in LRP tracing.
csRAViewerOverlaySetPoolSize	unsigned int	user	10	The size of the pool of the overlay set.
csRASetupWireLenghAlg	string	cellview	longPath	Wire length calculation.
csRASetupWireMaxTemp	string	cellview		Maximum wire temperature.
csRASetupXbaOption	bool	cellview	false	Extends back-annotation. Valid values: true, false
csRASourceDesign	lcv	cellview	<file path>	PrimeSim EMIR source design.
csRAUseExistingCell	bool	cellview	false	Uses existing cell for PrimeSim EMIR. Valid values: true, false
csRAViewerAutoRaiseWindow	bool	user	true	Automatically raise the layout window or the result browser window, when the window is active.
csRAViewerGroupResultsByNets	bool	user	false	Displays the results grouped by net name when possible. Valid values: true, false
csRAViewOption	string	cellview	Zoom	PrimeSim EMIR view option.
csRAViolationType	string	cellview		PrimeSim EMIR violation type.

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csSPRESBin	string	cellview		PrimeSim Static Res Check bins.
csSPRESDestLib	string	cellview		PrimeSim Static Res Check destination library.
csSPRESExtractedLayers	string	cellview		PrimeSim Static Res Check extracted layers.
csSPRESHiddenColumns	string	cellview	Comment	PrimeSim Static Res Check hidden table columns.
csSPRESNet	string	cellview		PrimeSim Static Res Check net.
csSPRESP2PExtractedLayers	string	cellview		PrimeSim Static Res Check Extracted Layers Option.
csSPRESP2PFlightLines	string	cellview	1	PrimeSim Static Res Check FlightLine Option.
csSPRESP2PPadOpen	string	cellview	0	PrimeSim Static Res Check P2P What-If Pad Open Option.
csSPRESP2PSave	string	cellview	1	PrimeSim Static Res Check P2P Save Option.
csSPRESP2PSelect	string	cellview	1	PrimeSim Static Res Check P2P Select To Compute.
csSPRESP2PWhatIfGenerateResMap	string	cellview	0	PrimeSim Static Res Check P2P What-If Generate Resistance Map Option.
csSPRESP2PWhatIfHighlights	string	cellview	All	PrimeSim Static Res Check P2P What-If Highlight Option.
csSPRESP2PWhatIfOverwriteResMap	string	cellview	1	PrimeSim Static Res Check P2P What-If Overwrite Option.

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csSPRESP2PWhatIfSave	string	cellview	1	PrimeSim Static Res Check P2P What If Save Option.
csSPRESP2PWhatIfSelect	string	cellview	All	PrimeSim Static Res Check P2P What-If Select To Add or Modify.
csSPRESP2PZoom	string	cellview	None	PrimeSim Static Res Check P2P Zoom Option.
csSPRESPage	int	cellview	1	PrimeSim Static Res Check page.
csSPRESPageSize	int	cellview	50	PrimeSim Static Res Check results page size.
csSPRESResultsDialogGeometry	string	cellview	700x475+6+40	PrimeSim Static Res Check dialog width.
csSPRESRun	string	cellview		PrimeSim Static Res Check run.
csSPRESRunDir	string	cellview		PrimeSim Static Res Check run directory.
csSPRESSetupAdditionalTcl	string	cellview		Specifies the path to additional SPRES Tcl file.
csSPRESSetupGUIaddnetpinTable	string	cellview		Specifies data of addnetpin table.
csSPRESSetupGUIaddnetpinxyTable	string	cellview		Specifies data of addnetpinxy table.
csSPRESSetupGUIlayerfactorTable	string	cellview		Specifies data of layerfactor table.
csSPRESSetupGUIoutfiltresTable	string	cellview		Specifies data of outfiltres table.
csSPRESSetupMergeCCDB	bool	cellview	true	Turns on CCDB merging. Valid values: true, false
csSPRESSetupMergeCCDBDesignLCV	lcv	cellview	<file path>	The design LCV corresponding to the merged CCDB.

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csSPRESSetupMergeCCDBOpen	bool	cellview	false	Opens merged CCDB automatically. Valid values: true, false
csSPRESSetupMergeCCDBOutputFile	string	cellview		Merged CCDB file path.
csSPRESSetupMergeCCDBOutputLCV	lcv	cellview	<file path>	Merged CCDB LCV.
csSPRESSetupMergeCCDBType	mergeCCDBSaveType	cellview	file	Merged CCDB type.
csSPRESSetupOutputDir	string	cellview	spres_results	Specifies the directory to run and save the results.
csSPRESSetupchkmissstrapboxsize	string	cellview		Specifies the chkmissstrapboxsize command.
csSPRESSetupchkmissstrapdistance	string	cellview		Specifies distance option of the chkmissstrap command.
csSPRESSetupchkmissstraplayerlist	string	cellview		Specifies layer_list option of the chkmissstrap command.
csSPRESSetupchkmissstraplayers	string	cellview		Specifies the chkmissstraplayers command.
csSPRESSetupchkmissstrapresh	string	cellview		Specifies the resistance threshold option of the chkmissstrap command.
csSPRESSetupchkmissstrapub	string	cellview		Specifies upper boundary threshold option of chkmissstrap command.
csSPRESSetupchkmissviaboxsize	string	cellview		Specifies the chkmissviaboxsize command.

Table 6 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
csSPRESSetupchkmissvialayer1	string	cellview		Specifies layer1 option of the chkmissvia command.
csSPRESSetupchkmissvialayer2	string	cellview		Specifies layer2 option of the chkmissvia command.
csSPRESSetupchkmissvialayers	string	cellview		Specifies the chkmissvialayers command.
csSPRESSetupchkmissviaub	string	cellview		Specifies the upper boundary threshold option of the chkmissvia command.
csSPRESSetupfile	string	cellview		Specifies setup file command.
csSPRESSetupgdslabellsall	bool	cellview	true	Generates all labels. Valid values: true, false
csSPRESSetupgdslabellsmaxpin	bool	cellview	false	Specifies to generate maxpin label. Valid values: true, false
csSPRESSetupgdslabellspads	bool	cellview	false	Generates pad labels. Valid values: true, false
csSPRESSetupgdslayer0	string	cellview		Specifies gdslayer0 command.
csSPRESSetupgdslayers	string	cellview		Specifies the gdslayers command.
csGDSThreshType	gdsthreshType	cellview	boundary	Specifies the gdsthresh type.
csSPRESSetupgdsthreshlobnd	string	cellview		Specifies the lobnd option of the gdsthresh command.
csSPRESSetupgdsthreshnproc	string	cellview		Specifies the nproc option of the gdsthresh command.

Table 6 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
csSPRESSetupgdsthreshupbnd	string	cellview		Specifies the upbnd option of the gdsthresh command.
csSPRESSetupitflayernames	bool	cellview	false	Specifies the itflayernames command. Valid values: true, false
csSPRESSetuplayer0ohm	string	cellview		Specifies the layer0ohm command.
csSPRESSetupmergefinger	string	cellview		Specifies the mergefinger command.
csSPRESSetupnet	string	cellview		Specifies the net setup command.
csSPRESSetupnetdeletepad	string	cellview		Specifies the netdeletepad command.
csSPRESSetupoutputformatall	bool	cellview	false	Uses all output format. Valid values: true, false
csSPRESSetupoutputformatccdb	bool	cellview	true	Uses the ccdb output format. Valid values: true, false
csSPRESSetupoutputformatgds	bool	cellview	false	Uses the GDS output format. Valid values: true, false
csSPRESSetupoutputformatrve	bool	cellview	false	Uses the RVE output format. Valid values: true, false
csSPRESSetuprmin	string	cellview		Specifies the rmin command.
csSPRESSetuptemp	string	cellview		Specifies the temp command.
csSPRESSourceDesign	lcv	cellview	<file path>	PrimeSim Static Res Check source design.

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csSPRESUseExistingCell	bool	cellview	false	PrimeSim Static Res Check use existing cell Valid values: true, false
csSPRESViewOption	string	cellview	Zoom	PrimeSim Static Res Check view option.
csSPRESViolationType	string	cellview		PrimeSim Static Res Check violation type.
csTool	string	user	XA	CustomSim tool.
csViewResultsOnTop	bool	user	true	Keep the highlights on top. Valid values: true, false
csWhatIfRACmdFile	string	user		Command file path for EMIR What-If flow.
csWhatIfRACmdLineOption	string	user		Command line option for EMIR What-If flow.
csWhatIfRAEnableResistors	csWhatIfEnableEnum	user	All	Automatically selects resistors to enable. Valid values: All Add Modify None Custom
csWhatIfRALoadResults	bool	user	true	Load results in the Results Browser. Valid values: true, false
csWhatIfRANetlistFile	string	user		Netlist file path for EMIR What-If flow.
csWhatIfRAOverwrite	bool	user	true	Overwrites last results. Valid values: true, false
csWhatIfRARunDir	string	user		Run directory path for EMIR What-If flow.
csWhatIfRASaveConfig	bool	user	true	Saves dialog configuration to cellView. Valid values: true, false

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csWhatIfRASimulateSelectedNets	bool	user	false	If true, the tool imulates only the selected nets. Valid values: true, false
csWhatIfRAStepOneResultsDir	string	user		Step one result directory path for EMIR What-If flow.
csWhatIfRAZoomTo	csWhatIfZoomToEnum	user	None	Selects part of the resistor to zoom to. Valid values: Node1 Node2 Both None
csWhatIfSPRESCmdFile	string	user		Command file path for Static Res Check What-If flow.
csWhatIfSPRESCycle	bool	user	false	Cycles the display packet for node-to-node calculation. Valid values: true, false
csWhatIfSPRESEnableN2n	csWhatIfEnableN2nEnum	user	All	Selects and enables node- to-node pairs.
csWhatIfSPRESEnableResistors	csWhatIfEnableEnum	user	All	Automatically selects and enables resistors. Valid values: All, Add , Modify, None, Custom
csWhatIfSPRESFlightlines	bool	user	false	Enables flight line display for node-to-node calculation. Valid values: true, false
csWhatIfSPRESLoadResults	bool	user	true	Loads results in the Results Browser. Valid values: true, false
csWhatIfSPRESNetlistFile	string	user		Netlist file path for Static Res Check What-If flow.

Table 6 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
csWhatIfSPRESOverwrite	bool	user	true	Overwrites the last results. Valid values: true, false
csWhatIfSPRESPadOpen	bool	user	false	Enables open pad option for node-to-node calculation. Valid values: true, false
csWhatIfSPRESRunDir	string	user		Run directory path for Static Res Check What-If flow.
csWhatIfSPRESSaveConfig	bool	user	true	Saves dialog configuration to cellView. Valid values: true, false
csWhatIfSPRESSaveN2nConfig	bool	user	false	Saves node- to-node configuration and results to cellview. Valid values: true, false
csWhatIfSPRESZoomTo	csWhatIfZoomToEnum	user	None	Selects part of the resistor to zoom to. Valid values: Node1 Node2 Both None

3

Constraint Management Tcl

This chapter describes how to use Tcl commands and preferences to set design constraints and lists commands and preferences in the Custom Compiler Tcl `cm` namespace. These commands and preferences let you perform constraint management operations.

For details, see:

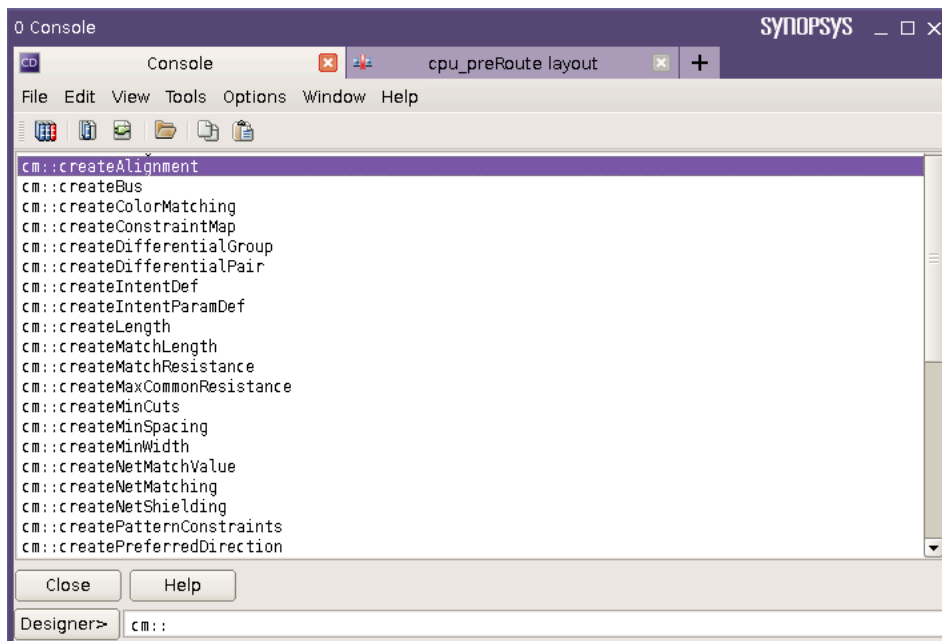
- [Using Tcl Commands to Set Constraints](#)
- [Commands](#)
- [Preferences](#)

Using Tcl Commands to Set Constraints

Instead of using the Constraint Editor, you can type Tcl commands in the Custom Compiler Console to set design-specific constraints (intents) for your design. Use the Tcl commands in the constraint manager (`cm::`) namespace.

To view the available commands, type `cm: :` in the Custom Compiler Console, and then press the Tab key. The list of constraint commands appears ([Figure 1](#)).

Figure 1 *Displaying Constraint Commands in the Custom Compiler Console*



The following is an example script that selects nets A and B in a design, uses cm:: commands to set the Differential Pair constraint for A and B, and then starts the routing operation.

```
# Activate the router tool
set routerTool [de::getTools "Router" ]
de::activateTools $routerTool -context [de::getActiveContext ]

# Get the nets A and B and create the collection
set netA [db::getNext [db::getNets "A" -of $design]]
set netB [db::getNext [db::getNets "B" -of $design]]
set netCollection [db::createCollection [list $netA $netB]]

# Creating the Differential Pair Constraint on nets A and B
cm::createDifferentialPair "inputDifferentialPair" -nets $netCollection

#Route the selected nets of the design
rt::route -nets $netCollection

# Route the remaining nets in the design
set design [oa::getDesign [ed]]
rt::route -design $design
```

Commands

The commands covered are:

- [cm::checkIntents](#)
- [cm::createAffinity](#)
- [cm::createAlignment](#)
- [cm::createBlockSize](#)
- [cm::createBus](#)
- [cm::createCapacitance](#)
- [cm::createCluster](#)
- [cm::createCMOS](#)
- [cm::createColorMatching](#)
- [cm::createColumn](#)
- [cm::createDifferentialGroup](#)
- [cm::createEdgeAssign](#)
- [cm::createEnclosingShape](#)
- [cm::createGuardRing](#)
- [cm::createHalo](#)
- [cm::createHierNetSpacing](#)
- [cm::createInstMatching](#)
- [cm::createIntent](#)
- [cm::createIntentDef](#)
- [cm::createIntentParamDef](#)
- [cm::createLength](#)
- [cm::createMaxCommonResistance](#)
- [cm::createNetPriority](#)
- [cm::createNetShielding](#)
- [cm::createOrientations](#)

- [cm::createPatternRoute](#)
- [cm::createPlacementGrid](#)
- [cm::createRelative](#)
- [cm::createResistance](#)
- [cm::createResistorBank](#)
- [cm::createRoutingStripes](#)
- [cm::createRows](#)
- [cm::createShieldOverhang](#)
- [cm::createStandardCells](#)
- [cm::createSymmetric](#)
- [cm::createTask](#)
- [cm::createTemplate](#)
- [cm::createWireMatching](#)
- [cm::createWireStriping](#)
- [cm::exportIntents](#)
- [cm::getarea](#)
- [cm::getallowedcutclass](#)
- [cm::getConstraints](#)
- [cm::getCutClass](#)
- [cm::getCutSpacing](#)
- [cm::getCutToMetalSpacing](#)
- [cm::getEolExtension](#)
- [cm::getExtension](#)
- [cm::getIntentDefs](#)
- [cm_getlength](#)
- [cm::getPlacement](#)
- [cm::getRegionLpps](#)

- [cm::getRoutingVias](#)
- [cm::getRowPlacement](#)
- [cm::getShieldOverhangData](#)
- [cm::getSizePolicy](#)
- [cm::getSpacing](#)
- [cm::getSpacingData](#)
- [cm::getTasks](#)
- [cm::getViaStackLimits](#)
- [cm::getWidth](#)
- [cm::importIntentDefinitions](#)
- [cm::importIntents](#)
- [cm::showConstraintManager](#)
- [cm::showExportIntents](#)
- [cm::showImportIntents](#)

cm::checkIntents

Description

Invokes the registered check procedure on the given intents.

Syntax

```
cm::checkIntents <intents: dbCollection>
```

Returns

<none>

Arguments

intents

Intent collection to be checked.

cm::createAffinity

Description

Creates an Affinity constraint (intent).

Syntax

```
cm::createAffinity [<name: string>] (-for <{<deFigure>} |  
{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |  
{<oaMarker>} | {<oaShape>} | {<oaVia>}}> | -in <oaDesign>) [-insts  
[<string>]] -strength <int>
```

Returns

<cmAffinity> : Affinity constraint

Arguments

name

Optional name of the constraint.

-for

Collection of instances.

-in

Design or cellview to search for patterns.

-insts

Tcl list of instance names.

-strength

Replusive (negative value) or attractive force (positive value) to apply between the instances.

Examples

In this example, the cm::createAffinity command creates attractive affinity intent on C1 instance in specified design.

```
set design [ed]  
cm::createAffinity -strength 9 -for $design -insts C1
```

cm::createAlignment

Description

Creates an alignment constraint (intent) for the group of objects given.

Syntax

```
cm::createAlignment <objects: {<deFigure>} | {<deSelectionSet>} |  
  {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |  
  {<oaVia>}}> -align <enum> [-groupName <string>] [-lpp <lpp>]
```

Arguments

objects

Collection of objects to align.

-align

Alignment geometry ({bottomEdge | center | leftEdge | rightEdge | topEdge}).

-groupName

The name for the created oaGroup.

-lpp

Layer and optional purpose for the shape. Default purpose is drawing.

Returns

<none>

Examples

In this example, the `cm::createAlignment` command creates bottom edge alignment constraint on instances those names starts with character R.

```
set design [ed]  
set insts [db::getInsts R* -of $design]  
cm::createAlignment $insts -align bottomEdges
```

cm::createBlockSize

Description

Creates a Block Size constraint (intent).

Syntax

```
cm::createBlockSize -for <oaDesign> -sizePolicy <cmSizePolicy>
```

Returns

<cmBlockSize> : Block Size constraint

Arguments

`-for`

Collection of instances.

`-sizePolicy`

Size policy for the design or block.

Examples

In this example, the `cm::createBlockSize` command creates aspect ratio intent for the specified design within 0.8–1.2.

```
set design [ed]
set aspectRatio [cm::getSizePolicy aspectRatio -minimum 0.8 -maximum 1.2]
cm::createBlockSize -for $design -sizePolicy $aspectRatio
```

cm::createBus

Description

Creates a bus constraint (intent), and defines the nets and their order in the bus trunk based on the specified order of the net collections or net groups.

Syntax

```
cm::createBus <groupName: string> -for <{<oaGroup>} | {<oaNet>}>
[-shieldPlacement <enum>] [-validLayers [<string>]] [-layerSpacing
<string>] [-layerWidth <string>] [-gap <float>] [-cornerType <enum>]
[-offGridLayers [<string>]] [-optimizeOrder <bool>] [-extendTrunk
<bool>]
```

Arguments

`groupName`

Name of bus constraint.

`-for`

Collection of nets or net groups.

`-shieldPlacement`

Where to add shields if required (`{default | doubleInterleave | interleave | outside}`). Default is *default*.

`-validLayers`

Optional list of layer names to use for the bus trunk.

`-layerSpacing`

Optional map of layer name and spacing pairs.

`-layerWidth`

Optional map of layer name and width pairs.

`-gap`

Optional gap spacing to use between the outermost bus bits and the other shapes in the design. Default is 0.

`-cornerType`

Optional corner type override to use for the bus ({auto | cross | default | river}. The default is default).

`-offGridLayers`

Optional list of layers allowed to go off grid for the bus trunk.

`-optimizeOrder`

Optional setting to control whether order-reversing is allowed.

The default is true.

`-extendTrunk`

Optional setting to control extension of bus trunk.

The default is true.

Examples

In this example, the `cm::createBus` command creates Bus intent on data bus net bits with specified layer options.

```
set design [ed]
set nets [db::getNets data<*> -of $design]
cm::createBus dataBus -for $nets -validLayers {m1 m2 m3} -layerSpacing
  {{m1 0.15} {m2 0.2} {m3 0.2}} -layerWidth {{m1 0.1} {m2 0.1} {m3 0.15}}
```

cm::createCapacitance

Description

Creates a capacitance intent for the given nets. The capacitance intent dictates that for a given net, the total or maximum capacitance to ground must be between the specified minimum and maximum capacitance.

Syntax

```
cm::createCapacitance -nets <{<oaNet>}> [-minValue <float>] -maxValue  
    <float>
```

Returns

<cmCapacitance> : Capacitance intent

Arguments

-nets

Collection of net

-minValue

Minimum capacitance permitted for the specified nets. The default is 0.

-maxValue

Maximum capacitance permitted for the specified nets.

Errors

[1] Empty collection is given.

[2] Objects are not all from the same design.

Examples

In this example, the `cm::createCapacitance` command creates capacitance intent on `clk` net with 8fF maximum value.

```
cm::createCapacitance -nets [db::getNets clk -of [ed]] -maxValue 8e-15
```

cm::createCluster

Description

Creates a Cluster constraint (intent) for the specified instances.

Syntax

```
cm::createCluster [<name: string>] (-for <{<deFigure>} |  
    {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |  
    {<oaMarker>} | {<oaShape>} | {<oaVia>}}> | -in <oaDesign>) [-insts  
    [<string>]] [-style <enum>] [-sizePolicy <cmSizePolicy>]
```

Returns

<cmCluster> : Cluster constraint

Arguments

name

Optional name of the constraint.

-for

Collection of instances.

-in

Design or cellview to search for patterns.

-insts

Tcl list of instance names.

-style

Style of cluster placement: normal, packed, symmetricX, symmetricXorY, symmetricY. Default: normal

-sizePolicy

Size policy to use.

Examples

In this example, the `cm::createCluster` command creates `ampCol` fig group from `PLL/amp/layout cellView` instantiations and applies one column size policy and symmetricX placement style to that group.

```
set design [ed]
set insts [db::getInsts -of [db::getInstHeaders PLL/amp/layout -of
    $design]
cm::createCluster ampCol -for $insts -sizePolicy [cm::getSizePolicy
    columns -target 1] -style symmetricX
```

cm::createCMOS

Description

Creates a CMOS constraint (intent) for the specified instances.

Syntax

```
cm::createCMOS [<name: string>] (-for <{<deFigure>} |
{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
{<oaMarker>}
| {<oaShape>} | {<oaVia>}} -in <oaDesign>) [-insts [<string>]]
[-gateAlignment <enum>] [-rowSpacings [<float>]] [-rowSeparations
    [<int>]]
[-leftDummies <int>] [-rightDummies <int>] [-sizePolicy <cmSizePolicy>]
```

```
[ -defaultPreset <string> ] [ -rowPattern <enum> ] [ -userPattern <string> ]  
[ -abutmentStyle <enum> ] [ -separation <enum> ] [ -sameRowSpacing <float> ]  
[ -boundaryStyle <enum> ] [ -sameRowLPP <string> ] [ -diffRowLPP <string>  
> ]
```

Returns

<cmCmos> : CMOS intent.

Arguments

name

Optional name of a new intent.

-for

A collection of instances.

-insts

The Tcl list of instance names.

-in

Design or cellview in which to search for patterns.

-gateAlignment

The alignment of gates for the same net. Valid values include center, left, none, and right. Default: center

-rowSpacings

A list of row spacing values.

-rowSeparations

A list of row separation values

-leftDummies

The number of dummy devices to be included on the left. Default: 0

-rightDummies

The number of dummy devices to be included on the right. Default: 0

-sizePolicy

The size policy for the design or block.

-defaultPreset

Optional preset cell name to use by default.

`-rowPattern`

Defines the pattern of row types ({ NP | NPPN | PN | PNNP | custom }). Default: PN

`-userPattern`

Defines a row pattern.

`-abutmentStyle`

Type of abutment performed on instances in the same-row spacing values ({ abut | none | optimal | withContacts }). Default: abut

`-separation`

Type of separation for same row spacing ({ drc | microns | polyPitch }). Default: drc

`-sameRowSpacing`

When `defaultSeparation` is fixed, defines the explicit spacing between adjacent instances in a row. Default: 0

`-boundaryStyle`

Type of device boundary: { activeLayer | instance | specify }.

Default: activeLayer

`-sameRowLPP`

When boundary is specified, defines the same-row layer-purpose pair (LPP).

`-diffRowLPP`

When boundary is specified, defines the different-row LPP.

Examples

In this example, the `cm::createCMOS` command creates CMOS intent for all instances in a design (assuming they are PMOS or NMOS devices) with 0.8u distance between P-diffusion and N-diffusion, and minimum 1, maximum 8 rows size policy.

```
cm::createCMOS -for [db::getInsts -of $design] -rowSpacings 0.8  
-sizePolicy [cm::getSizePolicy rows -minimum 1 -maximum 8]
```

cm::createColorMatching

Description

Creates a Color Matching constraint (intent) for the specified collection of nets or net groups. The Color Matching constraint lets you specify one of the following:

- All selected nets must be on a specific mask.
- All selected nets must be on the same color mask on a per layer basis regardless of which color mask is used.

Syntax

```
cm::createColorMatching [<name: string>] -for <{<oaGroup>} | {<oaNet>}>  
-color <string>
```

Returns

<cmColorMatching> : Color Matching constraint

Arguments

name

Name of the Color Matching constraint.

-for

Collection of nets or net groups.

-color

Name of color mask or color mask enumeration.

Examples

In this example, the cm::createColorMatching command creates Color Matching constraint on lfin net.

```
set design [ed]  
cm::createColorMatching netColor -for [db::getNets lfin -of $design]  
-color red
```

cm::createColumn

Description

Creates a Column constraint (intent).

Syntax

```
cm::createColumn [<name: string>] (-for <{<deFigure>} |  
<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |  
<oaMarker>} | {<oaShape>} | {<oaVia>}}> | -in <oaDesign>) [-insts  
[<string>]] [-alignment <enum>] [-ordering <enum>] [-separation <enum>]  
[-spacing <float>]
```

Returns

<cmColumn> : Column constraint

Arguments

name

Optional name of the constraint.

-for

Collection of instances.

-in

Design or cellview to search for patterns.

-insts

Tcl list of instance names.

-alignment

Alignment of instances in the column: center, left, right. Default: center

-ordering

Order of instances in the column: asls, autoOrdering. Default: autoOrdering

-separation

Type of separation between the instances: abut, drc, fixed. Default: abut

-spacing

(If -separation is fixed) Amount of separation, in user units. Default: 0

Examples

In this example, the cm::createColumn command creates Column intent for I0, I1 and I3 instances with 0.2u fixed spacing.

```
cm::createColumn -in [ed] -insts {I0 I1 I2} -separation fixed -spacing  
0.2
```

cm::createDifferentialGroup

Description

Creates a Differential Group constraint (intent) for two specified nets. An optional third net can be specified to produce a differential triplet. The Custom Router pairs the closest pins, one from each net, and routes from a common gather point between the pins, keeping the resulting routing of the nets together for as long as possible.

Syntax

```
cm::createDifferentialGroup <groupName: string> -nets <{<oaNet>}>
[-twistStyle <enum>] [-gap <float>] [-twistInterval <float>]
[-twistOffset <float>] [-shieldPlacement <enum>] [-validLayers
<string>]] [-layerSpacing <string>] [-layerWidth <string>]
```

Arguments

groupName

Name of the oaGroup that contains the specified nets.

-nets

Two or three nets that will be grouped as a differential group.

-twistStyle

Defines how the nets in the group will be twisted ({ default | diagonal | none | orthogonal }). Default: default

-gap

Optional gap spacing to use between the nets. Default: 0

-twistInterval

Optional setting for twist interval. Only valid if -twistStyle is not none. Default: 0

-twistOffset

Optional setting for twist offset. Only valid if -twistStyle is not none. Default: 0

-shieldPlacement

Where to add shields if required: ({ default | doubleInterleave | interleave | outside }). Default: default

-validLayers

Optional list of layer names to use for the trunk.

`-layerSpacing`

Optional map of layer name and spacing pairs.

`-layerWidth`

Optional map of layer name and width pairs.

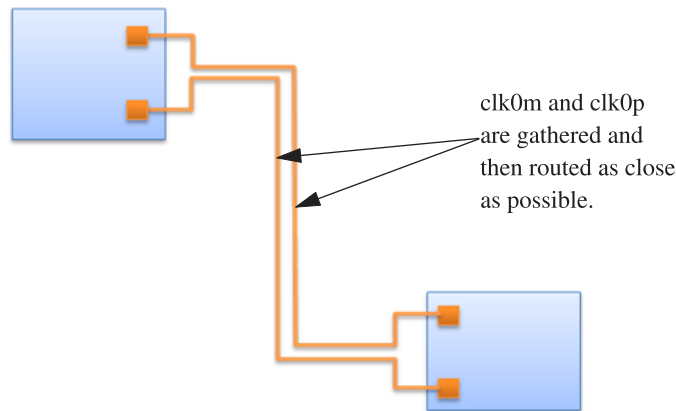
Examples

```
set nets [db::getNets -filter {%name=="PosIn"||%name=="NegIn"} \
    -of [ed]]
cm::createDifferentialGroup diffGroupPosNeg -nets $nets
```

In this example, `cm::createDifferentialGroup` creates a Differential Group constraint for the nets PosIn and NegIn. The created oaGroup is named diffGroupPosNeg.

```
set d [oa::DesignFind testLib top_design layout]
set nets {}
lappend nets [db::getNext [db::getNets clk0p -of $d]]
lappend nets [db::getNext [db::getNets clk0m -of $d]]
set netCollection [db::createCollection $nets]
cm::createDifferentialGroup clkDiffPair -nets $netCollection
```

In this example, `cm::createDifferentialGroup` creates a Different Pair constraint with the clk0m and clk0p nets.



cm::createEdgeAssign

Description

Creates an Edge Assign constraint (intent).

Syntax

```
cm::createEdgeAssign [<name: string>] (-for <{<deFigure>} |
    {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
```



```
{<oaMarker>} | {<oaShape>} | {<oaVia>}> | -in <oaDesign>) [-insts  
[<string>]]
```

Returns

<cmEdgeAssign> : Edge Assign constraint

Arguments

name

Optional name of the constraint.

-for

Collection of instances.

-in

Design or cellview to search for patterns.

-insts

Tcl list of the instance names.

cm::createEnclosingShape

Description

Creates an Enclosing Shape constraint (intent).

Syntax

```
cm::createEnclosingShape [<name: string>] (-for <{<deFigure>} |  
<{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |  
<{<oaMarker>} | {<oaShape>} | {<oaVia>}}> | -in <oaDesign>) [-insts  
[<string>]] -lpp <lpp> [-netName <string>] [-leftEnclose <enum>]  
[-leftMinimum <float>] [-bottomEnclose <enum>] [-bottomMinimum <float>]  
[-rightEnclose <enum>] [-rightMinimum <float>] [-topEnclose <enum>]  
[-topMinimum <float>]
```

Returns

<cmEnclosingShape> : Enclosing Shape constraint

Arguments

name

Name of the constraint.

-for

Collection of instances.

`-in`
Design or cellview to search for patterns.

`-insts`
Tcl list of the instance names.

`-lpp`
Layer and optional purpose for the shape. Default purpose: drawing

`-netName`
Optional net name for the shape.

`-leftEnclose`
Enclosure for the left edge: intent, layer. Default: intent

`-leftMinimum`
Minimum amount of enclosure for the left edge. Default: 0

`-bottomEnclose`
The enclosure for the bottom edge: intent, layer. Default: intent

`-bottomMinimum`
Minimum amount of enclosure for the bottom edge. Default: 0

`-rightEnclose`
Defines the enclosure for the right edge: intent, layer. Default: intent

`-rightMinimum`
Minimum amount of enclosure for the right edge. Default: 0

`-topEnclose`
Defines the enclosure for the top edge: intent, layer. Default: intent

`-topMinimum`
Minimum amount of enclosure for the top edge. Default: 0

Examples

In this example, the `cm::createEnclosingShape` command creates Enclosing Shape intent on M8 instance. So, auto placer encloses instance with shape on “border drawing” LPP, with 0.2u distance in horizontal direction and with 0.1u distance in vertical direction.

```
cm::createEnclosingShape -in [ed] -insts M8 -lpp {border drawing}  
-leftMinimum 0.2 -rightMinimum 0.2 -topMinimum 0.1 -bottomMinimum 0.1
```

cm::createGuardRing

Description

Creates a Guard Ring constraint (intent).

Syntax

```
cm::createGuardRing [<name: string>] (-for <{<deFigure>} |
    {<deSelectionSet>} |
    {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |
    {<oaVia>}}> |
    -in <oaDesign>) [-insts [<string>]] [-netName <string>] [-cellView
    <dmCellView>]
    [-style <enum>] [-minNotch <float>] [-cutRows <int>] [-bodyWidth <float>]
    [-individualWrap <bool>] [-mergeDistance <float>] [-params <string>]
    [-offset [<float>]]
```

Returns

<cmGuardRing> : Guard Ring intent

Arguments

name

Optional name of new intent.

-for

Collection of instances.

-insts

Tcl list of instance names.

-in

Design or cellview to search for patterns.

-netName

Net name to assign to rings.

-cellView

Library/cell/view names for PCell guard ring.

-style

Style of guard ring: ({ bottom | conformal | horizontal | left | rectangular | right | top | vertical}). Default: rectangular

-minNotch

Minimum notch size allowed for the body centerline. Default: 0

`-cutRows`

Number of cut rows to include for MCell guard rings. Default: 0

`-bodyWidth`

Minimum body width for MCell guard rings. Default: 0

`-individualWrap`

Indicates whether individual guard ring wrap should be applied to each instance separately. (Default is: false)

`-mergeDistance`

Merge distance (Default is: 0)

`-params`

Optional list of pairs of parameter names and values in the format of `{{<name> <value>}}`.

`-offset`

List of 1, 2, or 4 spacing values

Examples

In the following example, the `cm::createGuardRing` command creates a Guard Ring intent for the P0 instance, and the auto placer will surround instance with specified reference10/MPP_GRNWell/layout PCell guard ring.

```
cm::createGuardRing
```

cm::createHalo

Description

Creates a Halo constraint (intent).

Syntax

```
cm::createHalo [<name: string>] (-for <{<deFigure>} | {<deSelectionSet>}  
| {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |  
{<oaVia>}} | -in <oaDesign>) [-insts [<string>]] [-left <float>] [-right  
<float>] [-top <float>] [-bottom <float>]
```

Returns

`<cmHalo>` : Halo constraint

Arguments

name

Optional name of the constraint.

-for

Collection of instances.

-in

Design or cellview to search for patterns.

-insts

Tcl list of the instance names.

-left

Distance from the left edge. Default: 0

-right

Distance from the bottom edge. Default: 0

-top

Distance from the right edge. Default: 0

-bottom

Distance from top edge. Default: 0

Examples

In this example, the `cm::createHalo` command creates Halo intent on P0 and N1 instances. So, auto placer will keep out 0.2u distance in left and right direction, and 0.1u distance in bottom and top direction.

```
cm::createHalo -in [ed] -insts {P0 N1} -left 0.2 -bottom 0.1 -right 0.2  
-top 0.1
```

cm::createHierNetSpacing

Description

Creates a Hier Net Spacing constraint (intent).

Syntax

```
cm::createHierNetSpacing [<name: string>] -for <oaDesign> -nets1 <string>  
-nets2 <string> -spacings <dbCollection>
```

Returns

<cmHierNetSpacing> : Hier Net Spacing intent

Arguments

name

Optional name for the intent. If not given, the intent will be named automatically.

-for

Option to specify design for new intent.

-nets1

List of net names in net group 1.

-nets2

List of net names in net group 2.

-spacings

Collection of `cmSpacingData` objects that define the minimum spacing between the two nets.

Examples

In this example, the `cm::createHierNetSpacing` command creates Hier Net Spacing intent for FOUT and FOUT_B nets with defined spacing data collection.

```
set design [ed]
set spacing [db::createCollection ]
db::add [cm::getSpacingData -layer1 m1 -layer2 m2 -spacing 0.2
-runLengthThreshold 2.0] -to $spacing
db::add [cm::getSpacingData -layer1 m2 -layer2 m3 -spacing 0.2
-runLengthThreshold 2.0] -to $spacing
db::add [cm::getSpacingData -layer1 m3 -layer2 m4 -spacing 0.2
-runLengthThreshold 2.0] -to $spacing
cm::createHierNetSpacing -for $design -nets1 FOUT -nets2 FOUT_B
-spacings $spacing
```

cm::createInstMatching

Description

Creates an Instance Matching constraint (intent).

Syntax

```
cm::createInstMatching [<name: string>] (-for <{<deFigure>} |
{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
{<oaMarker>})
```

```
| {<oaShape>} | {<oaVia>}> |-in <oaDesign>) [-insts [<string>]]
[-style <enum>] [-groupCount <int>] [-firstGroup <int>] [-rowOrientations
<enum>]
[-sizePolicy <cmSizePolicy>] [-pattern <string>] [-differentRowSpacings
[<float>]]
[-rowSeparations [<int>]] [-separation <enum>] [-sameRowSpacing <float>]
[-defaultPreset <string>] [-chainStyle <enum>] [-startInstance <string>]
[-spacingType <enum>] [-abutmentStyle <enum>] [-boundaryStyle <enum>]
[-sameRowLPP <string>] [-diffRowLPP <string
>]
```

Returns

<cmInstMatching> : Instance Matching constraint

Arguments

name

Name of the constraint.

-for

Collection of instances.

-in

Design or cellview to search for patterns.

-insts

Tcl list of instance names.

-style

Defines how the nets in the group is twisted: balancedPattern, centralSymmetric, commonCentroid, custom, definedPattern, deviceGroup, interdigitate, seriesChain, textPattern. Default: deviceGroup

-groupCount

Number of fingers from the same instance used for a pattern grouping. Default: 1

-firstGroup

Number of fingers from the same instance used in first pattern group. Default: 1

-rowOrientations

Defines default row orientations ({ evenFlipped | flipped | none | oddFlipped }). Default: none

`-sizePolicy`

Sizing policy to use for the design or block when determining the number of rows and columns based on the pattern: aspectRatio, columns, height, none, policy_rows, width. Default: none

`-pattern`

(Style is custom) Defines the custom pattern.

`-differentRowSpacing`

Defines the explicit spacing to use between rows. Default: 0

`-separation`

Defines how adjacent instances in a row are separated: abut, abutKeepContacts, drc, fixed, microns, polyPitch. Default: drc

`-sameRowSpacing`

When defaultSeparation is fixed, defines the explicit spacing between adjacent instances in a row. Default: 0

`-defaultPreset`

Optional -preset cell name to use by default.

`-chainStyle`

Chaining style: ({ lowerLeftHorizontalSnake | lowerLeftHorizontalZigzag | lowerLeftVerticalSnake | lowerLeftVerticalZigzag | lowerRightHorizontalSnake | lowerRightHorizontalZigzag | lowerRightVerticalSnake | lowerRightVerticalZigzag | none | upperLeftHorizontalSnake | upperLeftHorizontalZigzag | upperLeftVerticalSnake | upperLeftVerticalZigzag | upperRightHorizontalSnake | upperRightHorizontalZigzag | upperRightVerticalSnake | upperRightVerticalZigzag }). Default: none

`-startInstance`

Optional start instance. Default: A

`-spacingType`

Indicates the units of the row spacing values: ({ microns | tracks }). Default: microns)

`-abutmentStyle`

Defines the type of abutment performed on instances in the same row spacing values: abut, none, optimal, withContacts. Default: abut

`-boundaryStyle`

Type of device boundary: { activeLayer | instance | specify }.

Default: activeLayer

-sameRowLPP

When boundary is specified, defines the same-row layer-purpose pair (LPP).

-diffRowLPP

When boundary is specified, defines the different-row LPP.

cm::createIntent

Description

Creates a custom constraint (intent) definition.

Syntax

```
cm::createIntent [<name: string>] (-for <{<deFigure>} |
    {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
    {<oaMarker>} | {<oaShape>} | {<oaVia>}}> | -in <oaDesign>) [-for
    <{<deFigure>} | {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>}
    | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}> | -groups
    [<string>]] [-for <{<deFigure>} | {<deSelectionSet>} | {<oaFig>} |
    {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>
    | -insts [<string>]] [-insts [<string>] | -nets <{<oaNet>}}>] [-for
    <{<deFigure>} | {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>}
    | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}> | -nets
    <{<oaNet>}}>] -defName <string> [-params [<string>]]
```

Arguments

name

Name of the new custom intent.

-defName

Definition name of custom intent.

-for

Collection of instances.

-insts

Tcl list of instance names.

-in

Design or cellview to search for patterns.

-terms

Tcl list of terminals or instance terminal names.

-nets

One or more net names if intent is net-based.

-groups

Tcl list of group names representing custom intent group name.

-params

Tcl list of paramName and paramValue pairs, for example: { {name1 val1 }
{ name2 val2 } }.

Returns

<cmCustom> : Newly created custom constraint (intent)

cm::createIntentDef

Description

Creates a constraint (intent) definition object for use in a session. The name of the constraint definition must be unique in the session.

Syntax

```
cm::createIntentDef <name: string> [-groupDef <oaGroupDef>] [-category  
  <enum>]  
[-title <string>] [-icon <string>] [-validateProc <string>] [-checkProc  
  <string>]  
[-menu <string>] -memberType [<string>] [-minMembersCount <int>]  
  [-maxMembersCount <int>]  
[-allowSharedMember <bool>] [-visibleInMenu <bool>] [-paramDefs  
  [<cmIntentParamDef>]]  
  
cm::createIntentDef <name: string> [-groupDef <oaGroupDef>] [-category  
  <enum>]  
[-title <string>] [-icon <string>] [-validateProc <string>] [-checkProc  
  <string>]  
[-menu <string>] -memberType <enum> [-minMembersCount <int>]  
  [-maxMembersCount <int>]  
[-allowSharedMember <bool>] [-visibleInMenu <bool>] [-paramDefs  
  [<cmIntentParamDef>]]
```

Arguments

name

Name of the constraint definition. The name must be unique.

-groupDef

Group definition object.

`-category`

Constraint definition category: ({electrical | placement | process | routing}).

`-title`

Title of the intent definition. The title is displayed in the Constraint Editor.

`-icon`

Icon of the intent definition. The icon should be preloaded in system.

`-validateProc`

Tcl procedure that validates whether the intent is in the correct state. If the intent is in the correct state, the procedure returns `true`. ({-onCreate | -onParamUpdate | -onMemberUpdate})

`-checkProc`

Tcl procedure to check if the intent is satisfied in design

`-menu`

Name that appears in the Constraint Editor **Create Constraint** menu.

`-memberType`

Tcl list is allowed for intent definition member types, combinations are possible.

`-minMembersCount`

Minimum number of intent definition members.

`-maxMembersCount`

Maximum number of intent definition members.

`-allowSharedMember`

Permits intents to share the same object as member item. Default: false

`-visibleInMenu`

Indicates if the intent name is visible in the Constraint Editor creation menu.
Default: true.

`-paramDefs`

A list of Tcl constraint parameter definitions.

Returns

<cmIntentDef>: Newly created constraint definition object.

Examples

In the example, `cm::createIntentDef` creates an intent which specifies that instances must have the same fixed aspect ratio. The `cm::createIntentDef` command also creates a parameter to hold the target aspect ratio.

```
set validDBs [oa::DBType Design]
set validTypes [oa::ManagedTypeArray]
oa::append $validTypes [oa::ManagedType $oa::oacInstType]
oa::append $validTypes [oa::ManagedType $oa::oacModInstType]
oa::append $validTypes [oa::ManagedType $oa::oacStringValueType]
set groupDef [oa::FlatGroupDefCreate matchAspectRatio \
    $validTypes $validDBs]
set params {}
lappend params [cm::createIntentParamDef aspectRatio -valueType\
    float
    -defaultValue 1.0 -title "Aspect Ratio"]
set intentDef [cm::createIntentDef matchAspectRatio -groupDef\
    $groupDef
    -category placement -paramDefs $params
    -description "Match Aspect Ratio"]
```

cm::createIntentParamDef

Description

Creates a constraint (intent) parameter definition for use by a constraint definition object. The name of the constraint definition must be unique in the session.

Syntax

```
cm::createIntentParamDef <name: string> [-title <string>] -defaultValue
<string>
-type <enum> [-validValues [<string>]] [-visible <bool>]
```

Returns

<cmIntentParamDef>: A new constraint parameter object.

Arguments

name

Name of intent parameter definition.

-title

Title of intent parameter definition.

-defaultValue

Default Value of the parameter.

`-type`

Name of the value type: { bool | enum | float | integer | multiEnum | multiString | string | userUnit }.

`-validValues`

List of string values that the parameter accepts.

`-visible`

Indicates if a parameter is shown or not on Constraint Editor. Default: true.

Examples

In this example, `cm::createIntentParamDef` creates a parameter `aspectRatio` for a new intent definition named `Match Aspect Ratio`.

```
set groupDef [oa::FlatGroupDefCreate matchAspectRatio\
    $validTypes $validDBs]
set params {}
lappend params [cm::createIntentParamDef aspectRatio -valueType\
    float
    -defaultValue 1.0 -title "Aspect Ratio"]
set intentDef [cm::createIntentDef matchAspectRatio -groupDef\
    $groupDef
    -category Placement -paramDefs $params
    -title "Match Aspect Ratio"]
```

cm::createLength

Description

Creates a Length constraint (intent) for the collection of nets or specified route pair.

Syntax

```
cm::createLength [<groupName: string>] [-first <oaInstTerm | oaPin |
    oaPinFig | oaTerm> | -nets <{<oaGroup>} | {<oaNet>} | {<oaRoute>}}>]
[-second <oaInstTerm | oaPin | oaPinFig | oaTerm> | -nets <{<oaGroup>} |
    {<oaNet>} | {<oaRoute>}}>] -maxValue <float> [-minValue <float>]
```

Returns

<cmLength> : Length constraint

Arguments

`groupName`

Name of the group if needed.

`-first`
First connection point.

`-second`
Second connection point.

`-maxValue`
Maximum value.

`-minValue`
Minimum value. Default: 0

`-nets`
Nets to add to match the constraint.

Examples

In this example, the `cm::createLength` command create Length intent on FOUT net.

```
set design [ed]
cm::createLength -maxValue 50.0 -nets [db::getNets FOUT -of $design]
```

cm::createMaxCommonResistance

Description

Creates a Maximum Common Resistance constraint (intent) for the given terminal or the instance terminal.

Syntax

```
cm::createMaxCommonResistance <resistance: float> -of <oaBitTerm |  
oaInstTerm> [-exclude [<string>]]
```

Returns

<cmMaxCommonResistance> : Max Common Resistance constraint

Arguments

`resistance`
Resistance value in ohms.

`-of`
Terminal or instance terminal to constrain.

`-exclude`
List of space-separated names of terminals and/or instance terminals.

Examples

In this example, the `cm::createMaxCommonResistance` command create Common Resistance intent on FOUT instance terminal of vco instance.

Example 1

```
set instTerm1 [db::getInstTerms FOUT -of [db::getInsts vco -of $design]]
cm::createMaxCommonResistance 10 -of $instTerm1
```

Example 2

In this example, the `cm::createMaxCommonResistance` command creates Max Common Resistance intent on FOUT terminal of the current design.

```
set term [db::getTerms FOUT -of $design]
cm::createMaxCommonResistance 10 -of $term
```

cm::createNetPriority

Description

Creates a Net Priority constraint (intent) for the specified nets or net group. The Net Priority intent assigns the given priority to the set of nets.

Syntax

```
cm::createNetPriority -for <{<oaGroup>} | {<oaNet>}> -priority <int>
```

Returns

<cmNetPriority> : Newly created intent.

Arguments

`-for`

Nets or net group.

`-priority`

Priority value.

Examples

In this example, the `cm::createNetPriority` command creates Net Priority intent for LFOUT net.

```
cm::createNetPriority -for [db::getNets LFOUT -of $design] -priority 64
```

cm::createNetShielding

Description

Creates Net Shielding constraint (intent) for the given group of nets. For more information, see Net Shielding Constraint.

Syntax

```
cm::createNetShielding [<groupName: string>] -nets <{<oaGroup>} |
  {<oaNet>}> [-shieldNet <string>] [-secondShieldNet <string>] [-style
  <enum>] [-gap <float>] [-width <float>] [-layerWidths <string>]
  [-layerGaps <string>] [-layerMaxGaps <string>] [-viaDefs [<string>]]
  [-disabledLayers [<string>]] [-encloseVia <enum>] [-enclosePin
  <enum>] [-minimumSegment <float>] [-sharing <enum>] [-defaultMaxGap
  <float>] [-groupShield <bool>] [-groupShield <bool>] [-topOffset
  <int>] [-bottomOffset <int>]
```

Returns

<cmNetShielding> Net shielding intent

Arguments

groupName

Name of the oaGroup if needed.

-nets

Nets or net groups to add shield constraint, nets are only supported on OA designs.

-shieldNet

Name of net to connect to the shield.

-secondShieldNet

Optional second shield net.

-style

Shielding style that must be used ({ above | aboveCenterCoaxial | aboveSolidCoaxial | aboveSplitCoaxial | below | belowCenterCoaxial | belowSolidCoaxial | belowSplitCoaxial | centerCoaxial | coaxial | offsetAbove | offsetAboveCenterCoaxial | offsetAboveSolidCoaxial | offsetAboveSplitCoaxial | offsetBelow | offsetBelowCenterCoaxial | offsetBelowSolidCoaxial | offsetBelowSplitCoaxial | offsetCenterCoaxial | offsetCoaxial | offsetSplitCoaxial | offsetTandem | parallel | parallelMultiLayer | splitCoaxial | tandem }). The default is parallel)

`-gap`

Layer-independent spacing between route objects and shield in user units.

`-width`

Layer-independent width for shielding material in user units.

`-layerWidths`

Optional list of layer-specific width values in the format of `{{<layerName>
<width>}}`.

`-layerGaps`

Optional list of layer-specific gap values in the format of `{{<layerName>
<gap>}}`.

`-layerMaxGaps`

Optional list of layer-specific max gap values in the format of `{{<layerName>
<maxGap>}}`.

`-viaDefs`

Optional list of viaDef names. If given, only specified viaDefs will be used to connect to the shield. Otherwise, all viaDefs are valid.

`-disabledLayers`

Optional list of layer names to be disabled from use for shielding.

`-encloseVia`

Optional override to indicate if vias are enclosed by the shield: `{ default | false | true }`. Default: default

`-enclosePin`

Optional override to indicate if pins are enclosed by the shield: `{ default | false | true }`. Default: default

`-minimumSegment`

Optional override defining the minimum segment length to shield.

`-sharing`

Shielding style that must be used: `{{ default | false | true }}`. Default: default

`-defaultMaxGap`

Layer-independent maximum spacing between route objects and shield in user units.

`-groupShield`

Indicates whether all nets in the shielding group should be shielded with a single set of shield shapes. Default: false

`-topOffset`

Top layer offset used in tandem shield.

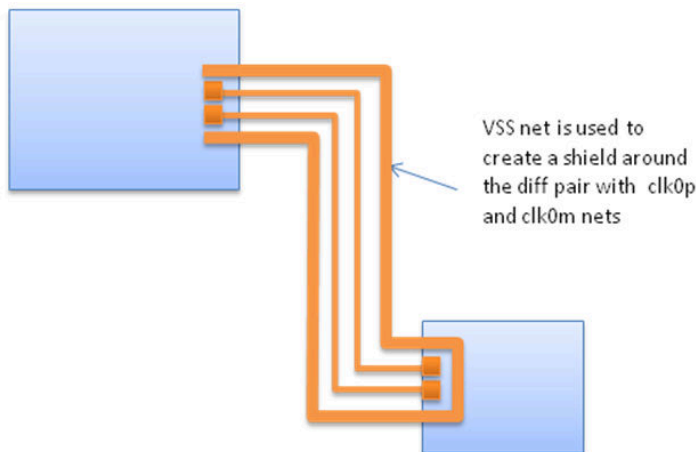
`-bottomOffset`

Bottom layer offset used in tandem shield.

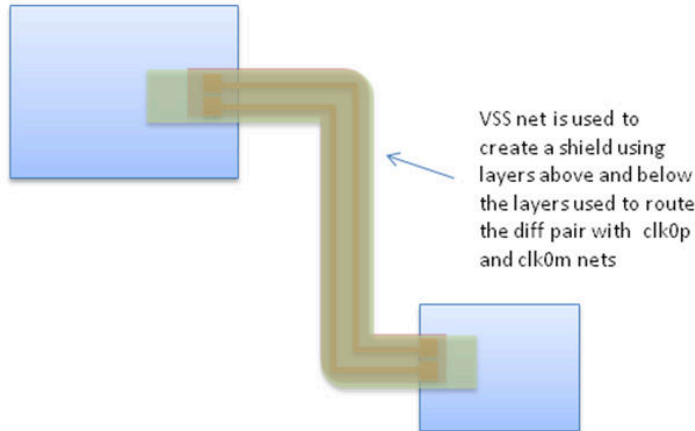
Examples

In all the three examples, `cm::createNetShielding` creates shielding constraint around the nets in the `clkDiffPair` group using VSS with 0.1u gap, and 0.3u width.

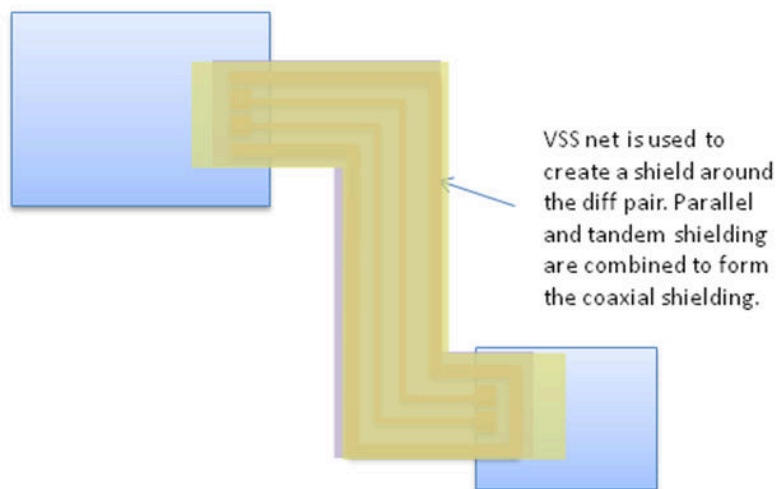
```
set d [oa::DesignFind testLib top_design layout]
set g [oa::getGroupsByName $d clkDiffPair]
cm::createNetShielding -nets $g -shieldNet VSS -gap 0.1 -width\
    0.3 -style parallel
```



```
set d [oa::DesignFind testLib top_design layout]
set g [oa::getGroupsByName $d clkDiffPair]
cm::createNetShielding -nets $g -shieldNet VSS -gap 0.1 -width\
    0.3 -style tandem
```



```
set d [oa::DesignFind testLib top_design layout]
set g [oa::getGroupsByName $d clkDiffPair]
cm::createNetShielding -nets $g -shieldNet VSS -gap 0.1 -width\
    0.3 -style coaxial
```



cm::createOrientations

Description

Creates an Orientations constraint (intent).

Syntax

```
cm::createOrientations [<name: string>] (-for <{<deFigure>} |
    {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
    {<oaMarker>} | {<oaShape>} | {<oaVia>}} -in <oaDesign>) [-insts
    [<string>]] -orientations <string>
```

Returns

<cmOrientations> : Orientations constraint

Arguments

name

Optional name of the constraint.

-for

Collection of instances.

-in

Design or cellview to search for patterns.

-insts

Tcl list of the instance names.

-orientations

Tcl list of the orientations.

Examples

In this example, the `cm::createOrientations` command creates orientation intent for all instances referencing from the `stdCellLib` library.

```
cm::createOrientations stdCells -for [db::getInsts -of $design -filter  
    {%libName == "stdCellLib"}] -orientations {R0 R180 MX MY}
```

cm::createPatternRoute

Description

Creates a Pattern Route intent from the given description string and applies it to the given net, intent, or intent associated with the given `lePlacementGroup` (`oaFigGroup`).

Syntax

```
cm::createPatternRoute <string> [-net <string | oaNet>] -for <oaNet |  
    cmIntent | lePlacementGroup>
```

Returns

<cmPatternRoute> : Pattern Route intent object.

Arguments

name

Name of the constraint.

`-net`

Name of the net or net object to route.

`-for`

Owner of the new Route Pattern intent. Required if `-net` is specified.

cm::createPlacementGrid

Description

Creates a Placement Grid decorator constraint (intent).

Syntax

```
cm::createPlacementGrid [<name: string>] [-insts [<string>]] -in  
  <oaDesign> -gridNames [<string>]
```

Returns

<cmPlacementGrid> : Placement Grid constraint

Arguments

`name`

Name of the constraint.

`-insts`

Tcl list of instance names.

`-in`

Design for the new constraint. If `-insts` is not given, new constraint applies to the design.

`-gridNames`

List of track region definition names to use for the placement grid.

cm::createRelative

Description

Creates a Relative constraint (intent).

Syntax

```
cm::createRelative [<name: string>] -for <{<deFigure>} |  
  {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>}}>
```

Returns

<cmRelative> : Relative constraint

Arguments

name

Name of the constraint.

-for

Collection of instances and/or lePlacementGroups to include in the placement constraint.

cm::createResistance

Description

Creates a resistance constraint (intent) for the collection of nets or between two specified connections.

Syntax

```
cm::createResistance [<groupName: string>] [-nets <{<oaNet>}>  
| -sources [<string>]] [-nets <{<oaNet>}> | -sourcesShorted <bool>]  
[-nets <{<oaNet>}> | -in <oaDesign>] [-nets <{<oaNet>}> |  
-destinationsShorted <bool>] -sources [<string>] [-nets <{<oaNet>}>  
| -destinations [<string>]] -maxValue <float> [-minValue <float>]
```

Returns

<cmResistance> : Resistance intent

Arguments

groupName

Name of group, if needed.

-maxValue

Maximum value.

-minValue

Minimum value.

Default: 0

-nets

Nets in the design to which the constraint applies.

`-sources`

List of design terminals, instance terminals, or pins on the same net as the `-destinations` terminals or pins. Use a single character space to separate the names.

`-destinations`

List of target design terminals, instance terminals, or pins on the same net as the `-sources` terminals or pins. Use a single character space to separate the names.

`-in`

Design or cellview in which to create the constraint.

`-sourcesShorted`

Treats multiple sources as shorts during calculation.

Default: true

`-destinationsShorted`

Treats multiple destinations as shorts during calculation.

Default: true

cm::createResistorBank

Description

Creates a Resistor Bank constraint (intent).

Syntax

```
cm::createResistorBank [<name: string>] (-for <{<deFigure>} |
{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
{<oaMarker>} |
{<oaShape>} | {<oaVia>}} -in <oaDesign>) [-rowSpacings [<float>] |
-rowSeparations [<int>]] [-insts [<string>]] -unitResistance <float>
-unitHeight <float> [-unitLength <float>] [-tolerance <float>]
[-defaultSeparation <enum>] [-gap <float>] [-sizePolicy <cmSizePolicy>]
[-defaultPreset <string>] [-chainStyle <enum>] [-startInstance <string>]
[-boundaryStyle <enum>] [-sameRowLPP <string>] [-diffRowLPP <string>]
>]
```

Returns

<cmResistorBank> : Resistor Bank constraint

Arguments

`name`

Optional name of the constraint.

`-for`

Collection of instances.

`-insts`

Tcl list of instance names.

`-in`

Design or cellview to search for patterns.

`-unitResistance`

Resistance of unit resistor piece.

`-unitHeight`

Height of the unit resistor. Default: 0

`-unitLength`

Length of the unit resistor. Default: 0

`-tolerance`

Tolerance of the unit resistor. Default: 0

`-rowSpacings`

List of row spacing values.

`-rowSeparations`

List of row separation values.

`-defaultSeparation`

Defines how adjacent instances in a row are separated: { abut | abutKeepContacts | drc | fixed | forceAbut | forceAbutKeepContacts }. Default: abut

`-gap`

When `-defaultSeparation` is fixed, defines the explicit spacing between adjacent instances in a row. Default: 0

`-sizePolicy`

Size policy for the design or block.

`-defaultPreset`

Optional preset cell name to use by default.

`-chainStyle`

Chaining style: lowerLeftHorizontalSnake, lowerLeftHorizontalZigzag, lowerLeftVerticalSnake, lowerLeftVerticalZigzag, lowerRightHorizontalSnake, lowerRightHorizontalZigzag, lowerRightVerticalSnake, lowerRightVerticalZigzag, none, upperLeftHorizontalSnake, upperLeftHorizontalW, upperLeftHorizontalZigzag, upperLeftVerticalSnake, upperLeftVerticalZigzag, upperRightHorizontalSnake, upperRightHorizontalZigzag, upperRightVerticalSnake, or upperRightVerticalZigzag. Default: none

`-startInstance`

Optional start instance. Default: A

`-boundaryStyle`

Type of device boundary: { activeLayer | instance | specify }.

Default: activeLayer

`-sameRowLPP`

When boundary is specified, defines the same-row layer-purpose pair (LPP).

`-diffRowLPP`

When boundary is specified, defines the different-row LPP.

cm::createRoutingStripes

Description

Creates a Routing Stripes constraint (intent).

Syntax

```
cm::createRoutingStripes [<name: string>] (-for <{<deFigure>} |
{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
{<oaMarker>} | {<oaShape>} | {<oaVia>}} -in <oaDesign>) [-insts
<string>]] [-locations <string>] [-order <enum>]
```

Returns

<cmRoutingStripes> : Routing Stripes constraint

Arguments

name

Optional name of the new constraint.

- for
Collection of instances.
 - in
Design or cellview to search for patterns.
 - insts
Tcl list of instance names.
 - locations
Sides to add routing stripes. Default: top bottom
 - order
Order to use for creating multiple stripes: auto, inward, outward. Default: auto
-

cm::createRows

Description

Creates a Rows constraint (intent).

Syntax

```
cm::createRows [<name: string>] (-for <{<deFigure>} | {<deSelectionSet>}  
| {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>}  
| {<oaVia>}} | -in <oaDesign>) [-insts [<string>]] [-sizePolicy  
<cmSizePolicy>] [-ordering <enum>] [-fillStyle <enum>] [-colAlignment  
<enum>] [-separation <enum>] [-spacing <float>] [-rowAlignment <enum>]  
[-rowSpacings <float>] [-rowSeparations [<int>]]
```

Returns

<cmRows> : Rows constraint

Arguments

- name
Optional name of the constraint.
- for
Collection of instances.
- in
Design or cellview to search for patterns.
- insts
Tcl list of the instance names.

`-rows`

Number of rows required. 0 lets the placement tool adjust the row count as necessary. Default: 0

`-ordering`

Order of instances in rows: asIs, autoOrdering, optimal. Default: auto

`-fillStyle`

Defines how the rows are filled when ordering is asIs: bottomThenTop, bottomToTop, leftThenRight, leftToRight, optimal, rightThenLeft, rightToLeft, topThenBottom, topToBottom. Default: leftToRight

`-colAlignment`

Alignment of columns when rows is greater than 1: center, left, right. Default: center

`-separation`

Separation of instances in a row: abut, drc, fixed. Default: abut

`-spacing`

When `-separation` is fixed, the minimum separation distance, in user units. Default: 0

`-rowAlignment`

Alignment of instances in a row: bottom, center, top. Default: center

`-rowSpacing`

List of row spacing values.

`-rowSeparation`

List of row separation values.

cm::createShieldOverhang

Description

Creates a Shield Overhang constraint (intent).

Syntax

```
cm::createShieldOverhang -for <oaDesign | oaGroup | oaNet>  
-shieldOverhangs <dbCollection>
```

Returns

<cmShieldOverhang> : Shield Overhang intent

Arguments

`-for`

oaNet, oaGroup, or oaDesign object.

`-shieldOverhangs`

Collection of cmShieldOverhangData objects.

Examples

In this example, the `cm::createShieldOverhang` command creates Shield Overhang intent for signal wires created on M3 and M4 layers in the specified design.

```
set shieldOver [db::createCollection ]
db::add [cm::getShieldOverhangData M3 -lowerOverhang 0.1 -upperOverhang
  0.1] -to $shieldOver
db::add [cm::getShieldOverhangData M4 -lowerOverhang 0.15 -upperOverhang
  0.15] -to $shieldOver
cm::createShieldOverhang -for [ed] -shieldOverhangs $shieldOver
```

cm::createStandardCells

Description

Creates a Standard Cells constraint (intent).

Syntax

```
cm::createStandardCells [<name: string>] (-for <{<deFigure>} |
  {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
  {<oaMarker>} | {<oaShape>} | {<oaVia>}} -in <oaDesign>) [-insts
  <string>] [-rowSpacing <float>] [-powerMode <enum>] [-UPFFile
  <string>] [-powerNetName <string>] [-groundNetName <string>]
  [-targetDensity <float>] [-rowFlip <enum>] [-auxCellSetName
  <string>] [-tapCellDistance <float>] [-tapCellEveryRow
  <bool>] [-tapCellDefaultLocation <enum>] [-sizePolicy
  <cmSizePolicy>] [-spacingType <enum>] [-allowYAxisFlip <bool>]
```

Returns

<none>

Arguments

`name`

Name of the constraint.

`-for`

Collection of instances.

`-in`

Design or cellview to search for patterns.

`-insts`

Tcl list of instance names.

`-rowSpacing`

Minimum row separation distance, in user units. Default: 0

`-powerMode`

Type of power mode: UPF, singleSupply. Default: singleSupply

`-UPFFile`

If powerMode is UPF, specifies the path to the UPF file.

`-powerNetName`

The power net name.

`-groundNetName`

The ground net name.

`-targetDensity`

Cell density, a value between 0 and 1. Default: 0

`-rowFlip`

Indicates how rows are flipped: all, even, none, odd. Default: none

`-auxCellSetName`

Name of the reference auxiliary cell.

`-tapCellDistance`

Tap cell distance. Default: 0

`-tapCellEveryRow`

If true, a tap cell is required in every row. Default: false

`-tapCellDefaultLocation`

Relative location of the tap cell: center, left, right. Default: center

`-sizePolicy`

Size policy for the design or block.

`-spacingType`

Specifies the type of value given in the Row Spacings parameter: microns, tracks. Default: microns

`-allowYAxisFlip`

Controls whether or not instances in a row are flipped on the y-axis. Default: true

Examples

In this example, the `cm::createStandardCells` command creates Standard Cell intent for instances those masters are core type cells. Placer will flip cells on even rows.

```
cm::createStandardCells [db::getInsts -of [ed] -filter {[oa::getName  
[oa::getCellType %master]] == "core"}] -powerNetName vdd! -groundNetName  
vss! -targetDensity 0.7 -rowFlip even -sizePolicy [cm::getSizePolicy  
rows -minimum 2 -maximum 4]
```

cm::createSymmetric

Description

Creates a Symmetric constraint (intent).

Syntax

```
cm::createSymmetric [<name: string>] (-for <{<deFigure>} |  
{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |  
{<oaMarker>} | {<oaShape>} | {<oaVia>}> | -in <oaDesign>) [-insts  
[<string>]]
```

Returns

<cmSymmetric> : Symmetric constraint

Arguments

`name`

Optional name of the constraint.

`-for`

Collection of instances.

`-in`

Design or cellview to search for patterns.

`-insts`

Tcl list of instance names.

cm::createTask

Description

Creates a new task.

Syntax

```
cm::createTask <name: string> -title <string> [-intents <dbCollection>]
```

Returns

<cmTask>: New task object.

Arguments

name

Name of the task.

-title

Displayable title string for the task.

-intents

A list of Tcl design-specific constraint definition objects to add to the task.

Examples

In this example, `cm::createTask` creates `My Task` that can be accessed from the Constraint Editor.

```
cm::createTask mytask -title "My Task" -intents [list $intentDef]
```

cm::createTemplate

Description

Creates a Template constraint (intent).

Syntax

```
cm::createTemplate [<name: string>] -insts <{<deFigure>} |  
<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |  
{<oaMarker>} | {<oaShape>} | {<oaVia>}}> -template <dbCore> [-viewName  
<string>] [-staticClone <bool>]
```

Returns

<cmTemplate> : The newly created template.

Arguments

name

Optional name for the intent, if not given the intent will be automatically named.

-insts

Collection of instances that match the given template netlist.

-template

The template being referenced.

-viewName

Optional view name in the template.

-staticClone

Optional setting defining whether the template should use the selected view as is.

cm::createWireMatching

Description

Creates a Wire Matching constraint (intent) for the collection of specified objects.

Syntax

```
cm::createWireMatching <groupName: string> -for <{<oaGroup>} |  
{<oaInstTerm>} | {<oaNet>} | {<oaRoute>}}> [-relative <bool>] [-matchType  
<enum>]  
-tolerance <float>
```

Returns

<cmWireMatching>: Wire Matching intent

Arguments

groupName

Name for created oaGroup holding given nets.

-for

Objects to add to the match constraint.

-relative

Indicates if tolerance value is relative or absolute. Default: false

-matchType

What characteristic or property should be matched: {capacitance | length | lengthPerLayer | resistance}). Default: length

-tolerance

Percent tolerance for consideration when nets match. If -relative is true, then the tolerance value is interpreted as a percent. If -relative is false, then the tolerance value is interpreted as an absolute value in ohms, micrometers, or picofarads.

For example, if -relative is true, and you specify -tolerance value as 5, then the tool interprets the tolerance value as 5%.

Example:

```
#####
Wire Matching
#####

set wmNetNames {sfr_addr<0> sfr_addr<1> sfr_addr<2>}
set wmNets [getNetColl $wmNetNames $design]

# Create a net group for the wire matching nets

set wmGroup [le::createNetGroup $wmNets -name "sfr_addr_gr"]

# Create the WM intent on the wire matching nets group

cm::createWireMatching "WM_sfr_addr" -for $wmGroup -matchType
"length" -tolerance 1.0 -relative true

#####
A proc to convert a list of net names to a collection of oaNets
#####

proc getNetColl { nets design } {
    set netObjs [list]
    foreach net $nets {
        set netObj [oa::NetFind [oa::getTopBlock $design]
            [oa::Name [oa::CdbaNS] $net]]lappend netObjs $netObj
    }
    return [db::createCollection $netObjs]
}
```

cm::createWireStripping

Description

Creates Wire Stripping constraint (intent) for the collection of nets or net groups.

Syntax

```
cm::createWireStriping [<name: string>] -for <{<oaGroup>} | {<oaNet>}>  
    -maxWidth <string> [-spacing <string>]
```

Returns

<dbCollection> : Collection of newly created constraints

Arguments

name

Name of the constraint.

-for

Collection of nets or net groups.

-maxWidth

Maximum width allowed for stripes per layer.

-spacing

Minimum spacing required between stripes per layer. If not given, foundry minimum spacing is used.

Errors

- Empty collection given with -for.
- Given name is already used.
- Width and/or spacing value given is less than foundry value.

cm::exportIntents

Description

Exports the specified intents to the specified XML file.

Syntax

```
cm::exportIntents <filePath: string> [-design <dmCellView | oaDesign>]  
    [-overwrite <bool>] [-category <enum>] [-intents  
    <cmConstraint>]
```

Arguments

filePath

Name of the file in which to store the intents.

`-intents`

Intent collection to be exported; if given, the `-XML` file format is used to store intents.

`-design`

The cellview or opened design from which to export intents.

`-overwrite`

Specifies whether or not to overwrite the existing file. Default: true

`-category`

Specifies the category type to export intents, including all or placement. Default: all

Examples

In this example, the `cm::exportIntents` command exports the specified design intents into the `intents.txt` file.

```
cm::exportIntents intents.txt -design [ed]
```

cm::getarea

Queries area rule

Syntax

```
cm::getArea [-regionLPPs [<lpp>] | -region <box | point> | -signature  
[<int>]] -layer <int> -scope <oaObject>  
[-purpose <int>] [-contextColor <int>] [-design <oaDesign>]  
-largestCoverWidth <float>  
-largestCoverLength <float> -longestEdgeLength <float> -width <float>  
-isRectangle <bool> [-viaLayers [<int>]]
```

Returns

`<cmValue>` : Value of area rule.

Arguments

`-layer`

Layer number of rule.

`-scope`

Scope of rule.

`-purpose`

Purpose number of rule.

-contextColor

Color of rule context.

-design

Design is required to get design scope information.

-regionLPPs

Region based rule related LPPs.

-region

Region to query region-based rule information under specified the given geometry region.

-signature

Layout context signature.

-largestCoverWidth

Width of the largest rectangle that can be covered by the shape.

-largestCoverLength

Length of the largest rectangle that can be covered by the shape.

-longestEdgeLength

Length of the longest edge.

-width

Width of shape.

-isRectangle

Indicates if the shape is rectangular.

-viaLayers

Set of layers, each defining a cut layer immediately above or below the metal layer.

cm::getallowedcutclass

Description

Reports allowed cut class rules.

Syntax

```
cm::getAllowedCutClass [-regionLPPs [<lpp>] | -region <box | point> |  
-signature [<int>]] -layer <int> -scope <oaObject> [-purpose <int>]  
[-contextColor <int>]  
[-design <oaDesign>] -lowerLayer <int> -upperLayer <int> -lowerMetalWidth  
<float>  
-upperMetalWidth <float> -lowerMetalDirection <enum> -upperMetalDirection  
<enum>
```

Arguments

-layer

Layer number of rule.

-scope

Scope of rule.

-purpose

Purpose number of rule.

-contextColor

Color of rule context.

-regionLPPs

Region based rule related LPPs.

-region

Region to query region-based rule information for specified given geometry region.

-signature

Layout context signature.

-design

Design is required to get design scope information.

-lowerLayer

Specifies lower layers.

-upperLayer

Specifies upper layers.

-lowerMetalWidth

Metal width of the lower layer.

`-upperMetalWidth`

Metal width of the upper layer.

`-lowerMetalDirection`

Direction of lower metal layer ({ horizontal | noDirection | vertical }).

`-upperMetalDirection`

Direction of upper metal layer ({ horizontal | noDirection | vertical }).

Returns

`<cmValue>`: List of allowed cut classes

cm::getConstraints

Description

Returns a collection of cmConstraint objects for the given owner, layer-purpose pairs (LPPs), and constraint names.

Syntax

```
cm::getConstraints [<expression: [string]>] [-lpps [<lpp | string>]]
  -of <oaDesign | oaShape | oaTech | {<deFigure>} | {<deSelectionSet>}
    | {<oaFig>} | {<oaFigGroup>} | {<oaGroup>} | {<oaInst>} | {<oaMarker>}
    | {<oaNet>} | {<oaRoute>} | {<oaShape>} | {<oaVia>}> [-local <bool>]
  [-constraintGroupType <enum>] [-filter <string>] [-match <enum>]
```

Returns

`<dbCollection>`: Collection of the cmConstraint objects

Arguments

`expression`

Tcl list of constraint type names.

`-lpps`

List containing LPP or string.

`-of`

Object to specify the scope of interest.

`-local`

Indicates whether constraints just from the `-of` object should be returned or if all child objects should also be considered. This option is applicable in all cases except for oaTech objects. Default: true

`-constraintGroupType`

This argument further qualifies which constraint group type to consider depending on the object given with `-of` (`{default | interChild | shielding | taper}`). If the constraint type `interChild` has a `runLength` setting, then the value is a Tcl list in the form used by array `get` or array `set` in which the keywords `spacing` and `runLength` are used (for example, `{spacing 1.2 runLength 2.5}`). Default: `default`

`-filter`

Standard filter expression. Default: `*`

`-match`

Defines whether all associated objects must exist in the `-of` collection (full) or just one (partial) for a constraint to be considered a match and returned (`{ full | partial }`). Default: `partial`

Examples

In the following example, `cm::getConstraints` gets the horizontal or verticalRouteGridPitch constraint for the ME7 layer, and sets the pitch value to 0.4.

```
set d [oa::DesignFind testLib top_design layout]
set c [cm::getConstraints horizontalRouteGridPitch -of $d -lpps\
{ME7} -local false]
db::setAttr value -of $c -value 0.4
set c [cm::getConstraints verticalRouteGridPitch -of $d -lpps\
{ME7} -local false]
db::setAttr value -of $c -value 0.4
```

In the following example, `cm::getConstraints` gets the minSpacing constraint to set the InterGroup Spacing constraint Run Length Threshold to 2.5. During the routing operation, when parallel routes reach 2.5 um, the router applies the intergroup spacing of 1.2 um.

```
set group [db::getGroups C -of [ed]]
set c_spacing [cm::getConstraints minSpacing -constraintGroupType interChild
-of $group -local false -lpps "m3"]
db::setAttr value -of $c_spacing -value {spacing 1.2 runLength 2.5}
set c_clearance [cm::getConstraints minClearance -constraintGroupType
interChild -of $group -local false \
-lpps "m2 m3"]
db::setAttr value -of $c_clearance -value {spacing 1.2 runLength 2.5}
```

cm::getCutClass

Description

Reports cut class rules.

Syntax

```
cm::getCutClass [-regionLPPs [<lpp>] | -region <box | point> | -signature  
[<int>]]  
-layer <int> -scope <oaObject> [-purpose <int>] [-contextColor <int>]  
[-design <oaDesign>]
```

Arguments

-layer

Layer number of rule.

-scope

Scope of rule.

-purpose

Purpose number of rule.

-contextColor

Color of rule context.

-regionLPPs

Region based rule related LPPs.

-region

Region to query region-based rule information for specified given geometry region.

-signature

Layout context signature.

-design

Design is required to get design scope information.

Returns

<cmValue>: List of allowed cut classes

cm::getCutSpacing

Description

Reports cut spacing rules.

Syntax

```
cm::getCutSpacing [-regionLPPs [<lpp>] | -region <box | point> |  
-signature [<int>]]
```



```
-layer1 <int> -scope1 <oaObject> [-purpose1 <int>] [-contextColor1 <int>]  
[-design <oaDesign>] -layer2 <int> -scope2 <oaObject> [-purpose2 <int>]  
[-contextColor2 <int>] -cut1 <box> -cut2 <box> -spacingDirection <enum>  
-sameShapes <bool> [-lowerWidth <float>] [-upperWidth <float>] [-sameVias  
<bool>]  
[-viaType1 <enum>] [-viaType2 <enum>] [-lowerLayer <int>] [-upperLayer  
<int>]  
[-selfAlignEdgeType1 <enum>] [-selfAlignEdgeType2 <enum>]  
[-lowerLayerInteractType <enum>]  
[-upperLayerInteractType <enum>] [-lowerLayerDirectionType <enum>]  
[-upperLayerDirectionType <enum>] -color <int>
```

Arguments

-layer1

First layer number of rule.

-scope1

First scope of rule.

-purpose1

First purpose number of rule.

-contextColor1

First color of rule context.

-regionLPPs

Region based rule related LPPs.

-region

Region to query region-based rule information for specified given geometry region.

-signature

Layout context signature.

-design

Design is required to get design scope information.

-layer2

Second layer number of rule.

-scope2

Second scope of rule.

`-purpose2`
Second purpose number of rule.

`-contextColor2`
Second color of rule context.

`-cut1`
Box of first cut shape.

`-cut2`
Box of second cut shape.

`-spacingDirection`
Spacing direction of wires. ({ any | horizontal | vertical }).

`-sameShapes`
Indicates whether the edges considered are from the same shape.

`-lowerWidth`
Width of lower layer shape that is below the cut.

`-upperWidth`
Width of upper layer shape that is above the cut,

`-sameVias`
Indicates whether the cuts considered are connected to the same metal shapes both above and below.

`-viaType1`
Via type of the first cut. ({ any | nonSelfAlignedViaType | selfAlignedViaType }) .

`-viaType2`
Via type of the second cut. ({ any | nonSelfAlignedViaType | selfAlignedViaType }).

`-lowerLayer`
Layer of the lower layer shape that is above the cut.

`-upperLayer`
Layer of the upper layer shape that is above the cut.

`-selfAlignEdgeType1`
Self aligned edge type for the first cut. ({ any | nonSelfAlignedEdgeType | selfAlignedEdgeType }).

`-selfAlignEdgeType2`

Self aligned edge type for the second cut. ({ any | nonSelfAlignedEdgeType | selfAlignedEdgeType }).

`-lowerLayerInteractType`

Interact type for lower layer shapes. ({ diffShapesInteractType | noneInteractType | sameShapeInteractType }).

`-upperLayerInteractType`

Interact type for upper layer shapes. ({ diffShapesInteractType | noneInteractType | sameShapeInteractType }) .

`-lowerLayerDirectionType`

Direction type for lower layer shapes. ({ diffDirectionType | horizontalDirectionType | noneDirectionType | verticalDirectionType }).

`-upperLayerDirectionType`

Direction type for upper layer shapes. ({ diffDirectionType | horizontalDirectionType | noneDirectionType | verticalDirectionType }).

`-color`

Color of rule.

Returns

`<cmValue>`: Value of spacing rule

cm::getCutToMetalSpacing

Description

Reports cut to metal spacing rule.

Syntax

```
cm::getCutToMetalSpacing [-regionLPPs [<lpp>] | -region <box | point> |
    -signature [<int>]]
-layer1 <int> -scope1 <oaObject> [-purpose1 <int>] [-contextColor1 <int>]
    [-design <oaDesign>]
-layer2 <int> -scope2 <oaObject> [-purpose2 <int>] [-contextColor2 <int>]
    -width <float>
-extension <float> -otherExtension <float> -box <box> [-length <float>]
    [-direction <enum>]
[-overLayers [<int>]] [-enclosedWidth <float>] [-isConcaveCorner <bool>]
[-isCornerContiguous <bool>] -enclosingShapeDirection <enum>
```

Arguments

`-layer1`

First layer number of rule.

`-scope1`

First scope of rule.

`-purpose1`

First purpose number of rule.

`-contextColor1`

First color of rule context.

`-regionLPPs`

Region based rule related LPPs.

`-region`

Region to query region-based rule information for specified given geometry region.

`-signature`

Layout context signature.

`-design`

Design is required to get design scope information.

`-layer2`

Second layer number of rule.

`-scope2`

Second scope of rule.

`-purpose2`

Second purpose number of rule.

`-contextColor2`

Second color of rule context.

`-width`

Width of a wire.

`-extension`

Extension of layer2 metal over the cut.

`-otherExtension`

Extension of layer1 or layer2 metal over the cut.

`-box`

Box of cut shape.

`-length`

Parallel run length of region under consideration.

`-direction`

Spacing direction. ({ any | horizontal | vertical }).

`-overLayers`

Over layer numbers, on which there are shapes that interact with both the cut and metal shapes.

`-enclosedWidth`

Width of the metal that encloses the cut.

`-isConcaveCorner`

Indicates whether there is a need to report concave corner spacing constraints.

`-isCornerContiguous`

Indicates whether the cut and the corner on contiguous.

`-enclosingShapeDirection`

Spacing direction of the enclosing shape. ({ any | horizontal | vertical })

Returns

<dbCollection> : List of value groups of cut to metal spacing rule.

cm::getEolExtension

Description

Reports end-of-line extension rules.

Syntax

```
cm::getEolExtension [-regionLPPs [<lpp>] | -region <box | point> |  
-signature [<int>]]  
-layer1 <int> -scope1 <oaObject> [-purpose1 <int>] [-contextColor1 <int>]  
[-design <oaDesign>]  
-layer2 <int> -scope2 <oaObject> [-purpose2 <int>] [-contextColor2 <int>]  
-parallelDistance <float> -eolDistance <float> -width <float> -length  
<float>
```

```
-enclosedWidth <float> -enclosedHeight <float> -shortEdgeParallel <bool>  
  [-wireDirection <enum>]  
-color <int>
```

Arguments

-layer1

First layer number of rule.

-scope1

First scope of rule.

-purpose1

First purpose number of rule.

-contextColor1

First color of rule context.

-regionLPPs

Region based rule related LPPs.

-region

Region to query region-based rule information for specified given geometry region.

-signature

Layout context signature.

-design

Design is required to get design scope information.

-layer2

Second layer number of rule.

-scope2

Second scope of rule.

-purpose2

Second purpose number of rule.

-contextColor2

Second color of rule context.

-parallelDistance

The larger of the two distances for parallel edges adjacent to a shape.

`-eolDistance`

The shortest distance of shapes opposite to the end-of-line edge contained in the check region.

`-width`

Width of the end-of-line shape.

`-length`

Smaller of the two-edge adjacent to end-of-line edge.

`-length`

Smaller of the two-edge adjacent to end-of-line edge.

`-enclosedWidth`

Width of the enclosed shape.

`-enclosedHeight`

Length of the enclosed shape.

`-shortEdgeParallel`

Indicates whether the enclosed shape is rectangular, and the short edge is parallel to the end-of-line edge.

`-wireDirection`

Direction of the wire with the end-of-line edge. ({ any | horizontal | vertical }) .

`-color`

Color of rule.

Returns

<cmValueGroup> : Groups of values of end of line extension rule.

cm::getExtension

Description

Reports extension rules.

Syntax

```
cm::getExtension [-regionLPPs [<lpp>] | -region <box | point> |  
-signature [<int>]]  
-layer1 <int> -scope1 <oaObject> [-purpose1 <int>] [-contextColor1 <int>]  
[-design <oaDesign>]  
-layer2 <int> -scope2 <oaObject> [-purpose2 <int>] [-contextColor2 <int>]  
-enclosedWidth <float>
```

```
-enclosedHeight <float> -enclosureWidth <float> -length <float> -width  
  <float> -distance <float>  
-color1 <int> [-color2 <int>] -otherLayer <int> -otherLayerWidth <float>  
[-cutShortEdgeParallelEol <bool>] [-wireDirection <enum>]  
  [-otherLayerDirection <enum>]  
[-selfAlignViaType <enum>]
```

Arguments

-layer1

First layer number of rule.

-scope1

First scope of rule.

-purpose1

First purpose number of rule.

-contextColor1

First color of rule context.

-regionLPPs

Region based rule related LPPs.

-region

Region to query region-based rule information for specified given geometry region.

-signature

Layout context signature.

-design

Design is required to get design scope information.

-layer2

Second layer number of rule.

-scope2

Second scope of rule.

-purpose2

Second purpose number of rule.

-contextColor2

Second color of rule context.

-enclosedWidth

Width of the enclosed shape.

-enclosedHeight

Length of the enclosed shape.

-enclosureWidth

Width of the shape enclosing or extending beyond the other shape.

-length

Length of longest enclosing shape covering other shape.

-width

Width of shape below cut when finding extension of layer over cut.

-distance

Shortest distance parallel or perpendicular to shape on same layer.

-color1

Mask color of over enclosing layer1 shape.

-color2

Mask color of over enclosing layer2 shape.

-otherLayer

Number of layer under immediately under the cut layer.

-otherLayerWidth

Width of shape below cut when finding extension of layer over cut.

-cutShortEdgeParallelEol

Indicates whether the shorter edge of the cut is parallel to the end of line edge.

-wireDirection

Direction of wire. ({ any | horizontal | vertical }).

-otherLayerDirection

Direction of other wire. ({ any | horizontal | vertical }).

-selfAlignViaType

Self align via type. ({ any | nonSelfAlignedViaType | selfAlignedViaType }).

Returns

<dbCollection> : List of value groups of extension rule

cm::getIntentDefs

Description

Gets all the constraint (intent) definition objects that are registered either explicitly from Tcl or from internal sources.

Syntax

```
cm::getIntentDefs [<expression: string>] [-category <enum>] [-filter  
  <string>]
```

Returns

{<dbCollection>}: Collection of the constraint (intent) definition objects

Arguments

expression

Glob-style expression to match against the constraint definition names.

-category

Returns the constraint definition for the given category ({electrical | placement | process | routing}).

-filter

Standard attribute filter.

Examples

In the example, `cm::getIntentDefs` gets and displays all the registered routing intent names.

```
set routingIntents [cm::getIntentDefs -category routing]
db::foreach intent $routingIntents {
    puts "Name is [db::getAttr name -of $intent]"
}
```

cm_getlength

Queries length rule

Syntax

```
cm::getLength [-regionLPPs [<lpp>] | -region <box | point> | -signature  
  [<int>]] -layer <int> -scope <oaObject> [-purpose <int>] [-contextColor  
  <int>] [-design <oaDesign>] -spacingDirection <enum> [-width <float>]  
  [-isRect <bool>] [-viaLayers [<int>]]
```

Returns

<cmValueGroup> : Value of length rule.

Arguments

-layer

Layer number of rule.

-scope

Scope of rule.

-purpose

Purpose number of rule.

-contextColor

Color of rule context.

-regionLPPs

Region based rule related LPPs.

-region

Region to query region-based rule information in the specified geometry region.

-signature

Layout context signature.

-design

Design is required to get design scope information.

-spacingDirection

Spacing direction of wires. ({any | horizontal | vertical})

-width

Width of wire.

-isRect

Indicates whether the rule should be applied only to rectangles and not to polygons.

-viaLayers

Set of layers, each defining a cut layer immediately above or below the metal layer.

cm::getPlacement

Description

Returns a cmPlacement data object.

Syntax

```
cm::getPlacement <item: string> [-orient <enum>] [-fingers <int>]  
    [-abutted <bool>]  
    [-splitDiffusion <bool>] [-grouping <int>] [-gateAlign [<int>]]  
    [-horizontalSpacing <float>]  
    [-gateWidth <float>] [-verticalAlignment <enum>] [-preset <string>]  
    [-dummyNets [<string>]]
```

Returns

<cmPlacement> : Placement Data

Arguments

item

Specifies the item used in this placement slot.

-orient

Specifies the orientation of the placement: R0, R90, MX, and MY ({ MX | MY | R0 | R90 }). Default: R0

-fingers

Specifies the number of fingers. If 0, then the default number of fingers applies. Default: 0

-abutted

Indicates if this placement abuts with the next placement. Default: false

-splitDiffusion

Indicates that the diffusion is split immediately after this placement slot. Default: false

-grouping

The grouping number. Default: 0

-gateAlign

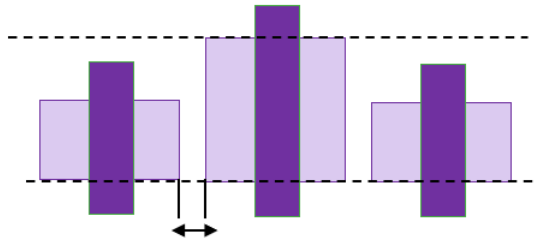
Two integers representing group number and gate index in the group.

-horizontalSpacing

Horizontal spacing between instances.

- > 0 adds spacing between adjacent instances.
- $= 0$ abuts adjacent instances.
- < 0 places the adjacent instances so that they overlap by the specified amount.

Default: 3.40282e+38



-

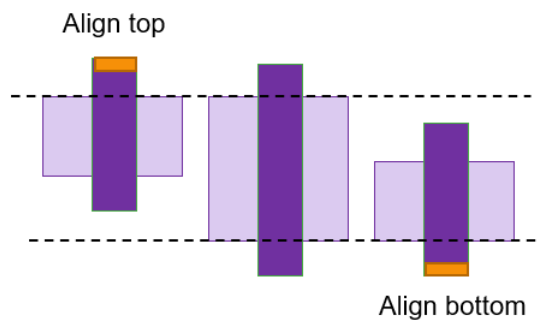
gateWidth

Gate Width (Default is 0).

-verticalAlignment

Vertical alignment of smaller instances to the largest instance in a row: none, top, center, or bottom.

Default: none



-preset

The name of the preset.

-dummyNets

A list of net names for left, gate, right, and bulk nets. This option is only valid if the `item` attribute is an asterisk (*).

cm::getRegionLpps

Description

Reports region rule LPPs.

Syntax

```
cm::getRegionLpps <tech: oaTech>
```

Arguments

tech

Technology to report LPPs.

Returns

<lpp> : All the region based-rule related LPPs.

cm::getRoutingVias

Description

Reports valid routing vias rules.

Syntax

```
cm::getRoutingVias [-regionLPPs [<lpp>] | -region <box | point> |  
-signature [<int>]]  
-layer <int> -scope <oaObject> [-purpose <int>] [-contextColor <int>]  
[-design <oaDesign>]  
-lowerWidth <float> -upperWidth <float>
```

Arguments

-layer

Layer number of rule.

-scope

Scope of rule.

-purpose

Purpose number of rule.

-contextColor

Color of rule context.

`-regionLPPs`

Region based rule related LPPs.

`-region`

Region to query region-based rule information for specified given geometry region.

`-signature`

Layout context signature.

`-design`

Design is required to get design scope information.

`-lowerWidth`

Width of the enclosing shape on the lower layer.

`-upperWidth`

Width of the enclosing shape on the upper layer.

Returns

`<cmValue>` : Value of routing vias rule.

cm::getRowPlacement

Description

Returns a cmRowPlacement data object.

Syntax

```
cm::getRowPlacement [-alignment <enum>] -placements [<dbCollection>]  
[-preset <string>]  
[-dummySize <string>] [-height <float>]
```

Returns

`<cmRowPlacement>` : Row Placement Data

Arguments

`-alignment`

The vertical alignment of instances in the row. Valid values: bottom, center, top.
Default: center

`-placements`

A collection of placements for the row.

`-preset`

Optional name of the default preset to use for the row.

`-dummySize`

Indicates the size of dummy devices for the row with values of (unset), 'm', 'M', or an explicit float value in user units.

`-height`

The height of the row. A value of zero means unset. Default: 0

cm::getShieldOverhangData

Description

Returns a cmShieldOverhangData data object

Syntax

```
cm::getShieldOverhangData <layer: string> [-width <float>]  
[-lowerOverhang <float>] [-upperOverhang <float>]
```

Returns

<cmShieldOverhangData> : Shield Overhang Data

Arguments

`layer`

Layer name.

`-width`

Width. Default: 0

`-lowerOverhang`

Lower layer shield overhang. Default: 0

`-upperOverhang`

Upper layer shield overhang. Default: 0

Examples

Example 1

In this example, the cm::getShieldOverhangData command returns Shield Overhang Data for signal wire on M1 layer, assuming that upper and lower shields of center coaxial type shielding must extends on 0.1u.

```
cm::getShieldOverhangData m1 -lowerOverhang 0.1 -upperOverhang 0.1
```


Example 2

In this example, the `cm::createShieldOverhang` command creates Shield Overhang intent for signal wires created on M3 and M4 layers in specified design.

```
set shieldOver [db::createCollection ]
db::add [cm::getShieldOverhangData M3 -lowerOverhang 0.1 -upperOverhang
  0.1] -to $shieldOver
db::add [cm::getShieldOverhangData M4 -lowerOverhang 0.15 -upperOverhang
  0.15] -to $shieldOver
cm::createShieldOverhang -for [ed] -shieldOverhangs $shieldOver
```

cm::getSizePolicy

Description

Returns a `cmSizePolicy` object with the given attribute values.

Syntax

```
cm::getSizePolicy <sizePolicy: enum> [-target <float> | -minimum <float>]
[-target <float> | -maximum <float>]
```

Returns

<cmSizePolicy> : Size policy

Arguments

`sizePolicy`

Size policy: aspectRatio, columns, height, none, row, Pairs, rows, or width.

`-target`

Target value. Default: is 0

`-minimum`

Minimum value. Default: 0

`-maximum`

Maximum value. Default: is 0

Examples

Example 1

The `cm::getSizePolicy` command returns policies that have an aspect ratio of 1.5.

```
cm::getSizePolicy aspectRatio -target 1.5
```

Example 2

The `cm::getSizePolicy` command returns policies that have widths within 0.8–1.2.

```
cm::getSizePolicy width -minimum 0.8 -maximum 1.2
```

cm::getSpacing

Description

Reports spacing rules.

Syntax

```
cm::getSpacing [-regionLPPs [<lpp>] | -region <box | point> | -signature  
[<int>]]  
-layer1 <int> -scope1 <oaObject> [-purpose1 <int>] [-contextColor1 <int>]  
[-design <oaDesign>]  
-layer2 <int> -scope2 <oaObject> [-purpose2 <int>] [-contextColor2 <int>]  
-box1 <box>  
-box2 <box> -spacingDirection <enum> -direction1 <enum> -direction2  
<enum> -sameShapes <bool>  
[-jointEOL <bool>] [-jointLength <float>] [-includeEndToEnd <bool>]  
[-color1 <int>]  
[-color2 <int>]
```

Arguments

`-layer1`

First layer number of rule.

`-scope1`

First scope of rule.

`-purpose1`

First purpose number of rule.

`-contextColor1`

First color of rule context.

`-regionLPPs`

Region based rule related LPPs.

`-region`

Region to query region-based rule information for specified given geometry region.

`-signature`

Layout context signature.

`-design`

Design is required to get design scope information.

`-layer2`

Second layer number of rule.

`-scope2`

Second scope of rule.

`-purpose2`

Second purpose number of rule.

`-contextColor2`

Second color of rule context.

`-box1`

Box of first wire.

`-box2`

Box of second wire.

`-spacingDirection`

Spacing direction of wires. ({ any | horizontal | vertical }).

`-direction1`

First wire direction. ({ any | horizontal | vertical }) .

`-direction2`

Second wire direction. ({ any | horizontal | vertical }) .

`-sameShapes`

Indicates whether the edges under consideration are from the same shape.

`-jointEOL`

Indicates whether rule applies to line ends and to merged line ends at wire joints.

`-jointLength`

Indicates the length of merged line ends at wire joints.

`-includeEndToEnd`

Indicates whether need to consider allowedEndToEndSpacing constraint.

`-color1`

First color of rule.

`-color2`

Second color of rule.

Returns

`<cmValue>` : Value of spacing rule.

cm::getSpacingData

Description

Returns a cmSpacingData data object.

Syntax

```
cm::getSpacingData -layer1 <int | string> [-layer2 <int | string>]  
-spacing <float> [-runLengthThreshold <float>]
```

Returns

`<cmSpacingData>` : Spacing Data

Arguments

`-layer1`

Layer number or name for layer1.

`-layer2`

Layer number or name for second layer if different than layer1.

`-spacing`

Spacing value.

`-runLengthThreshold`

Run length threshold value.

Examples

Example 1

In this example, the `cm::getSpacingData` command returns Spacing information for M1 layers, if parallel length exceeds 2.0u.

```
cm::getSpacingData -spacing 0.1 -layer1 m1 -runLengthThreshold 2.0
```

Example 2

In this example, the `cm::getSpacingData` command returns Spacing information for M1 and M2 layers.

```
cm::getSpacingData -spacing 0.1 -layer1 m1 -layer2 m2
```

cm::getTasks

Description

Gets all the task objects that are registered.

Syntax

```
cm::getTasks [<expression: string>] [-filter <string>]
```

Returns

<dbCollection>: Collection of task objects.

Arguments

`expression`

Glob expression to match the task names.

`-filter`

Standard attribute filter.

Examples

In the following example, `cm::getTasks` gets the task that matches the name 'default'.

```
cm::getTasks default
```

In the following example, `cm::getTasks` gets all the tasks that are registered.

```
cm::getTasks
```

cm::getViaStackLimits

Queries via stacking rule

Syntax

```
cm::getViaStackLimits [-regionLPPs [<lpp>] | -region <box | point> |  
-signature [<int>]]  
-layer <int> -scope <oaObject> [-purpose <int>] [-contextColor <int>]  
[-design <oaDesign>]  
[-lowerLayer <int>] [-upperLayer <int>]
```

Returns

<cmValueGroup> : Groups of values of via stack limits.

Arguments

-layer

Layer number of rule.

-scope

Scope of rule.

-purpose

Purpose number of rule.

-contextColor

Color of rule context.

-regionLPPs

Region based rule related LPPs.

-region

Region to query region-based rule information in the specified geometry region.

-signature

Layout context signature.

-design

Design is required to get design scope information.

-lowerLayer

Layer number of the bottom cut in the stack.

-upperLayer

Layer number of the top cut in the stack.

cm::getWidth

Description

Reports width rules.

Syntax

```
cm::getWidth [-regionLPPs [<lpp>] | -region <box | point> | -signature  
[<int>]]
```

```
-layer <int> -scope <oaObject> [-purpose <int>] [-contextColor <int>]  
[-design <oaDesign>]  
-spacingDirection <enum> [-length <float>]
```

Arguments

-layer

Layer number of rule.

-scope

Scope of rule.

-purpose

Purpose number of rule.

-contextColor

Color of rule context.

-regionLPPs

Region based rule related LPPs.

-region

Region to query region-based rule information for specified given geometry region.

-signature

Layout context signature.

-design

Design is required to get design scope information.

-spacingDirection

Spacing direction of wires. ({ any | horizontal | vertical }).

-length

Length of wire.

Returns

<cmValue> : Value of width rule.

cm::importIntentDefinitions

Imports intent definitions from file.

Syntax

```
cm::importIntentDefinitions -filePath [<string>] [-verbose <bool>]
```

Arguments

`-filePath`

Name of the intent definitions file.

`-verbose`

Prints detailed information when parsing the file. Default: false

cm::importIntents

Description

Imports all design-level intents from the specified file that is in the XML format. By default, the import process deletes all existing design constraints before importing the constraints from the file. You can use the `-merge` option to retain existing constraints and replace equivalent constraints in the design.

If a constraint in the file references components that do not exist in the design, such as layers, via definitions, or nets, the import process will skip those constraints and list the missing components in a message.

Syntax

```
cm::importIntents <filePath: string> [-design <dmCellView | oaDesign>]  
[-merge <bool>] [-category <enum>]
```

Returns

`<cmConstraint>`: Collection of the `cmConstraint` objects that are read from the specified file.

Arguments

`filePath`

File that contains the intents to be imported.

`-design`

Cellview or design into which you want to import the constraints.

`-merge`

Retains existing constraints and replaces equivalent constraints in the design.
Default: false

`-category`

Category type to import intents {all | placement}. Default: all

Examples

In this example, the `cm::importIntents` command imports intents specified in the `constraints.txt` file, merging with existing constraints in a design.

```
cm::importIntents constraints.txt -design $design -merge true
```

cm::showConstraintManager

Description

Shows the Constraint Manager window and associates it with the given layout editor window.

Syntax

```
cm::showConstraintManager -window <giWindow | int>
```

Returns

Newly created Constraint Manager window.

Arguments

`-window`

Window or window ID to associate with the Constraint Manager.

Examples

In this example, `cm::showConstraintManager` opens a Constraint Manager window for the design editor window with window ID 2.

```
cm::showConstraintManager -window 2
```

cm::showExportIntents

Description

Displays the Export Intents dialog box.

Syntax

```
cm::showExportIntents -window <giWindow | int>
```

Returns

<giDialog> : New Export Intents dialog box object

Arguments

-window
Window or window ID parent window.

cm::showImportIntents

Description

Displays the Import Intents dialog box.

Syntax

cm::showImportIntents -window <giWindow | int>

Returns

<giDialog> : New import intents dialog object

Arguments

-window
Window or window ID parent window.

Preferences

Table 7 on page 222 lists Custom Compiler cm namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 7 Constraint Management Preferences

Name	Type	Scope	Default	Description
cmConstraintEditorAutoSelect	<bool>	user	false	Automatically selects a single intent when a single object is logically selected. Valid values: true, false
cmConstraintEditorAttachNotesOnCreate	<bool>	user	false	Attach note shape and texts if available to the newly created intent. Valid values: true, false
cmConstraintEditorCurrentTask	<string>	user		Name of current task.
cmConstraintEditorFilterBySelection	<bool>	user	false	Enables or disables selection filtering. Valid values: true, false

Table 7 Constraint Management Preferences (Continued)

Name	Type	Scope	Default	Description
cmConstraintEditorHighlight	<bool>	user	true	Highlights associated objects in the canvas. Valid values: true, false
cmConstraintEditorShadowMode	<bool>	user	true	Uses Shadow mode for highlighting. Valid values: true, false
cmConstraintEditorShowAll	<bool>	cellview	false	Displays all constraints in the Constraint Editor. When set to <i>false</i> , displays only the highest level (or parent) constraints in the Constraint Editor and hides the child constraints. Valid values: true, false
cmConstraintEditorShowCategories	<bool>	user	true	Indicates whether the categories are the root items. Valid values: true, false
cmConstraintEditorShowFilter	<bool>	<user>	false	Shows or hides filter input. Valid values: true, false
cmConstraintEditorShowScalarizeBusNetsDialog	bool	user	false	Open the Scalarize Bus Nets dialog box during intent creation. Valid values: true, false
cmConstraintEditorZoom	<bool>	user	false	Indicates whether to adjust the viewport when objects are zoomed. Valid values: true, false
cmCreateConstraintGroupName	<string>	user		The name for the new constraint group.
cmExportDesignIntentsCellView	<lcv>	user	file path	Library/cell/view.
cmExportDesignIntentsFile	<string>	user		Exports a design intents file.
cmHighlightNotes	<bool>	cellview	false	Highlights intent notes in a schematic design. Valid values: true, false
cmImportDesignIntentsCellView	<lcv>	user	file path	Library/cell/view.

Table 7 *Constraint Management Preferences (Continued)*

Name	Type	Scope	Default	Description
cmlImportDesignIntentsFile	<string>	user		Imports a design intents file.
cmlImportDesignIntentsMode	Merge	user	Merge	Imports design intents mode.
cmlImportSheetResistanceCellView	<string>	user		Imports the sheet resistance data directly to the specified design cellView.
cmlImportSheetResistanceDefaultThickness	<string>	user		Specifies the default layer-thickness value to use. Valid values: Minimum, Nominal, Maximum
cmlImportSheetResistanceLayerThicknessFile	<string>	user		Specifies the layer-thickness file. The layer thickness file is an ASCII file that contains the list of the layers and the corresponding thickness values. The thickness value can be a floating point value in user units, or it is assigned one of the following keywords: Minimum, Nominal, or Maximum.
cmlImportSheetResistanceLibrary	<string>	user		Imports the sheet resistance data from the TLUPlus file directly to the specified technology library.
cmlImportSheetResistanceSourceFile	<string>	user		Specifies the TLUPlus file to import.

4

Design Collaboration Tcl

This chapter lists the commands and preferences in the Custom Compiler Tcl dc namespace. The dc Tcl lets you set up and control design reviews (or design collaborations). Interactive commands are listed after non-interactive commands in the Commands section.

For details, see:

- [Commands](#)
- [Preferences](#)

Commands

Interactive commands are listed after non-interactive commands in this section.

The commands covered are:

- [dc::addAnnotation](#)
- [dc::addComment](#)
- [dc::addImage](#)
- [dc::addPolicy](#)
- [dc::addToolStateClient](#)
- [dc::captureImage](#)
- [dc::changeState](#)
- [dc::createIssue](#)
- [dc::createReview](#)
- [dc::delete](#)
- [dc::displayImages](#)
- [dc::endReview](#)

- [dc::exportReview](#)
- [dc::getActivePolicy](#)
- [dc::getActiveReview](#)
- [dc::getReviews](#)
- [dc::getToolState](#)
- [dc::setActiveReview](#)
- [dc::setPolicy](#)
- [dc::setToolState](#)
- [dc::showCreateReview](#)
- [dc::showEditReview](#)
- [dc::showReviewManager](#)
- [dc::startReview](#)
- [idc::addReviewAnnotation](#)
- [idc::addReviewImage](#)

dc::addAnnotation

Description

Adds an annotation for an existing issue.

Syntax

```
dc::addAnnotation [<desc: string>] -issue <dbObject> -type  
  <enReviewAnnotation> -points [<point>]
```

Arguments

desc

Description of annotation.

-issue

Issue for new annotation.

-type

Annotation type to create.

`-points`

Coordinates of annotation.

Returns

`<dcIssue>` : Annotation object.

Examples

The example adds an annotation to the `dcIssue` object. After creation, if a Show Issue action is executed from the Review Assistant toolbar, an arrow pointing to {0 0} point will be shown.

```
# Start review for given design
set des [ed]
dc::startReview $des
# Create review object
set rvw [dc::createReview Initial_Review -design $des -assignee User1]
# Create issue object
set issue [dc::createIssue new_issue -review $rvw]
# Add annotation to issue
dc::addAnnotation new_annotation -issue $issue -type arrow -points {{0
  0} {1 1}}
# End a review session
dc::endReview $des
```

dc::addComment

Description

Appends a comment to the issue.

Syntax

```
dc::addComment <comment: string> -to <dbObject>
```

Arguments

`comment`

Comment to append to issue.

`-to`

Review or Issue to append comment.

Returns

`<none>`

Examples

The example adds a comment to the `dcIssue` object. Later, this comment can be found in the Discussion section of the Review Assistant.

```
# Start review for given design
set des [ed]
dc::startReview $des
# Create review object
set rvw [dc::createReview Initial_Review -design $des -assignee User1]
# Create issue object
set issue [dc::createIssue new_issue -review $rvw]
# Add comment to issue
dc::addComment "New comment" -to $issue
# End a review session
dc::endReview $des
```

dc::addImage

Description

Adds an image to an existing issue.

Syntax

```
dc::addImage [<caption: string>] -issue <dbObject> -imageData <string>
[-description <string>]
```

Arguments

`caption`

Caption for the image.

`-issue`

Issue for new annotation.

`-imageData`

Image for the issue.

`-description`

Description of the new image.

Returns

`<dcIssue>` : Image object.

dc::addPolicy

Description

Adds a design review policy.

The Custom Compiler design review system supports user-permission policy definitions. The definitions are used when registered procedures are called to validate whether the current user can create, modify, or delete design review components. See [Setting Up Design Review Policies](#).

After set up, design review policies will be activated during startup. The system assumes that you don't have to delete or redefine an existing policy.

Syntax

```
dc::addPolicy <string> -canCreateProc <string> -canModifyProc <string>  
-canDeleteProc <string>
```

Arguments

<string>

Unique policy name.

-canCreateProc

Tcl procedure to call before creating design review objects.

-canModifyProc

Tcl procedure to call before modifying an attribute of a design review object.

-canDeleteProc

Tcl procedure to call before deleting design review objects.

Returns

<none>

Errors

If policy name already exists.

Examples

In this example the new policy is added. By this policy only User1 can create reviews, only User2 can modify reviews and only User3 can delete issues.

```
# Define canCreateProc  
proc canCreate {user objectType parentObject} {  
    if {$user != "User1" && $objectType != "Review"} {  
        return "only User1 can create $objectType in \  

```

```
        '[db::getAttr review.description]''
    }
}
# Define canModifyProc
proc canModify {user object attrName newValue} {
    if {$user != "User2" && [db::getAttr object.selfType] == "dcReview"}{
        return "only User2 can modify $attrName in \
            '[db::getAttr object.description]'"
    }
}
# Define canDeleteProc
proc canDelete {user object} {
    if {$user != "User3" && [db::getAttr object.selfType] == "dcIssue"}{
        return false
    }
}
# Add new policy
dc::addPolicy newPolicy -canCreateProc canCreate \
    -canModifyProc canModify -canDeleteProc canDelete
```

dc::addToolStateClient

Description

Registers a tool state save or restore client.

Syntax

```
dc::addToolStateClient <name: string> -saveProc <string> -restoreProc
<string>
```

Arguments

name

Name of the client.

-saveProc

Name of the Tcl procedure to call to save tool state.

-restoreProc

Name of Tcl procedure to call to restore tool state.

Returns

<none>

Examples

In this example, a new tool state client is added to save and restore the Property Editor shown state. After creation, if a Show Issue action is executed from the **Review Assistant** toolbar, the Property Editor state will be restored.

```
# Create saveProc to save Property Editor state
proc savePEstate {wnd} {
    set pe [gi::getAssistants dePropertyEditor -from $wnd]
    return [db::getattr pe.shown]
}

# Create restoreProc to save Property Editor state
proc restorePEstate {wnd data} {
    set pe [gi::getAssistants dePropertyEditor -from $wnd]
    set shown [lindex $data 1]
    db::setattr pe.shown -value $shown
}

# Add tool state client
dc::addToolStateClient peState -saveProc savePEstate -restoreProc
    restorePEstate

# Start review for given design
set des [ed]
dc::startReview $des

# Create review object
set rvw [dc::createReview Initial_Review -design $des -assignee User1]

# Create issue named "new_issue" in $rvw review object and save tool
    state of window $id in the issue object
set issue [dc::createIssue new_issue -review $rvw -window [gi::getWindows
    $id]]

# End a review session
dc::endReview $des
```

dc::captureImage

Description

Displays the given image in a dialog box.

Syntax

```
dc::captureImage -captureProc <string> [-type <enImageType>]
```

Arguments

`-captureProc`

Name of Tcl procedure to call to save image, called with a single argument containing binary image data.

`-type`

Format of the image data. The default is png.

Returns

<none>

dc::changeState

Description

Changes the state of the specified review to the given state.

Syntax

```
dc::changeState -review <dbObject> [-comment <string>] -state  
                <enReviewState>
```

Arguments

`-review`

Review item to change the state.

`-comment`

Comment to append history.

`-state`

New state for review.

Returns

<none>

Examples

The example changes the state of the review to fixed.

```
# Start review for given design  
set des [ed]  
dc::startReview $des  
  
# Create review object  
set rvw [dc::createReview Initial_Review -design $des -assignee User1]
```

```
# Change review status to fixed
dc::changeState -review $rvw -state fixed

# End a review session
dc::endReview $des
```

dc::createIssue

Description

Creates an issue object for a review.

Syntax

```
dc::createIssue <desc: string> -review <dbObject> [-window <giWindow |
int>]
```

Arguments

desc

Description of the issue.

-review

Review to receive new issue.

-window

Window to capture.

Returns

<dcIssue> : Issue object.

Examples

The example creates an issue object for a review object. The description of the issue is `new_issue`. If the `-window` option is specified, the tool state of the window is saved in the issue object, so the tool state will be restored to the saved one at the Show Issue action from the Review Assistant toolbar.

```
# Start review for given design
set des [ed]
dc::startReview $des

# Create review object
set rvw [dc::createReview Initial_Review -design $des -assignee User1]

# Create issue named new_issue in $rvw review object and save tool state
of window $id in the issue object
set issue [dc::createIssue new_issue -review $rvw -window [gi::getWindows
$id]]
```

```
# End a review session
dc::endReview $des

# Print description and creator of the issue object
puts [db::getAttr issue.description]
puts [db::getAttr issue.creator]
```

dc::createReview

Description

Creates a review object.

Syntax

```
dc::createReview <desc: string> -design <oaDesign> -top <oaDesign> -path
<string>
-assignee <string> [-severity <enReviewSeverity>] [-creator <string>]
[-approvers [<string>]]
[-members [<string>]]
```

Arguments

`desc`

Description of a review item.

`-design`

Main design under review.

`-top`

Top-level design, which contains the main design of the review.

`-path`

The path of the main design, in the top-level design.

`-assignee`

Initial reviewer of a new item.

`-severity`

Severity of item (enhancement | critical | information). The default is information.

`-creator`

Creator of review. The default is current user.

`-approvers`

Optional list of user names to approve or close the review.

`-members`

Optional list of user names interested in the review item.

Returns

`<dcReview>` : Review object.

Examples

This example creates the review object with the description `Initial_Review` assigned to `User1` with `Critical` severity and defined lists of approvers (`User1` and `User2`) and members (`User3` and `User4`) of the review.

```
# Start review for given design
set des [ed]
dc::startReview $des
# Create review object
set rvw [dc::createReview Initial_Review -design $des -assignee User1
        -severity critical -approvers "User1 User2" -members "User3 User4"]
# End a review session
dc::endReview $des
# Print description and assignee of the review object
puts [db::getAttr rvw.description]
puts [db::getAttr rvw.assignee]
```

dc::delete

Description

Deletes a review, issue, annotation, or image object from the review.

Syntax

```
dc::delete <object: dbObject>
```

Arguments

`object`

Object to delete.

Returns

`<none>`

Examples

The example deletes the review object.

```
# Start review for given design
set des [ed]
dc::startReview $des
```

```
# Create two review objects
dc::createReview Initial_Review -design $des -assignee User1
dc::createReview New_Review -design $des -assignee User2
# Get one review and delete it
set review [dc::getReviews -of $des -filter {%description ==
  "Initial_Review"}}]
dc::delete [db::getNext $review]
# End a review session
dc::endReview $des
```

dc::displayImages

Description

Displays the given image in a dialog.

Syntax

```
dc::displayImages (-imageData <string> | -imageFiles [<string>])
  [-parent <giContainer>] [-captions <string>] [-acceptProc <string>]
  [-deleteOnClose <bool>]
```

Arguments

-imageData

Binary image data to display.

-imageFiles

List of image files to display.

-parent

Name of Tcl procedure to call to save the tool state.

-captions

Optional list of captions for images.

-acceptProc

If given, dialog is used to Accept or Discard an image called with an image data and user supplied caption.

-deleteOnClose

If given with -imageFiles, the given files will be deleted when the dialog is closed. The default is false.

Returns

<giDialog> : Dialog.

dc::endReview

Description

Ends a review on the given design and closes the review database.

Syntax

```
dc::endReview <design: oaDesign> [-save <bool>]
```

Arguments

design

Name of the design under review.

-save

Saves the review database before closing. Default: true

Returns

<none>

Examples

```
# Start a review on given design and open the review database
set des [ed]
dc::startReview $des
# End a review session and save the changes
dc::endReview $des -save true
```

dc::exportReview

Description

Exports specified reviews to an HTML file, thus allowing you to view the design review information without having to run the Custom Compiler tool.

Syntax

```
dc::exportReview <reviews: dbCollection> [-outputFile <string>]
[-exportImages <bool>]
```

Arguments

reviews

Collection of the dcReview objects.

`-outputFile`

Output file containing the review data. If you specify a full path, the directory is used for the output. If you do not specify a path, the default directory is used.

Default: `<current_working_directory>/synopsys_custom/reports`

`-exportImages`

Exports images with the review. Default: true

Returns

`<string>`: Path of the output HTML file.

dc::getActivePolicy

Description

Gets the current active design review policy.

Syntax

```
<string> dc::getActivePolicy
```

Arguments

`<string>`

Name of the active design review policy.

Returns

`<none>`

Errors

If policy name already exists.

dc::getActiveReview

Description

Gets the current active review object.

Syntax

```
dc::getActiveReview -design <oaDesign>
```

Arguments

`-design`

Retrieves the active review and displays it in the Design Review Assistant.

Returns

<dcReview> : Active review

Examples

The example creates a review object and gets active review.

```
# Start review for given design
set des [ed]
dc::startReview $des
# Create review object
dc::createReview Initial_Review -design $des -assignee User1
# Set review as active
set review [dc::getReviews -of $des -filter {%description ==
    "Initial_Review"}}
dc::setActiveReview [db::getNext $review]
# Get active review and print its description
set active [dc::getActiveReview -design [ed]]
puts [db::getAttr active.description]
# End a review session
dc::endReview $des
```

dc::getReviews

Description

Returns a collection of dcReview objects.

Syntax

```
dc::getReviews -of <dmCell | dmCellView | oaDesign | oaLib | string>
    [-filter <string>]
```

Arguments

-of

Container to search for reviews.

-filter

Filter expressions.

Returns

<dcReview> : Collection

Examples

The example gets all reviews from the given design.

```
# Start review for given design
set des [ed]
```

```
dc::startReview $des
# Create review object
dc::createReview Initial_Review -design $des -assignee User1
# Change another review
dc::createReview New_Review -design $des -assignee User2
# End a review session
dc::endReview $des
# Get all reviews and print their descriptions
set reviews [dc::getReviews -of $des]
puts [db::createList [db::getAttr description -of $reviews]]
```

dc::getToolState

Description

Returns tool state information.

Syntax

```
dc::getToolState [<window: giWindow | int>]
```

Arguments

window

Window to restore state.

Returns

<string> : Tool state data.

Examples

The example gets the tool state stored in a variable. It saves states for Tcl preferences, Object/Layer Panel, viewport, selection, highlights and probes.

```
# Get tool state for $id window
set toolState [dc::getToolState [gi::getWindows $id]]
```

dc::setActiveReview

Description

Sets review or issue object as active.

Syntax

```
dc::setActiveReview <review: dbObject>
```

Arguments

review

Review or issue to set as active.

Returns

<None> :

Examples

The example gets all reviews from the given design and sets one of them as active.

```
# Start review for given design
set des [ed]
dc::startReview $des
# Create review object
dc::createReview Initial_Review -design $des -assignee User1
# Create another review
dc::createReview New_Review -design $des -assignee User2
# Get one of the reviews and set it as active
set review [dc::getReviews -of $des -filter {%description ==
    "Initial_Review"}}]
dc::setActiveReview [db::getNext $review]
# End a review session
dc::endReview $des
```

dc::setPolicy

Description

Sets the specified design review policy as the active policy.

Syntax

```
dc::setPolicy <string>
```

Arguments

<string>

Name of an existing policy to set as the active Design Review policy.

Returns

<none>

Examples

```
# Set policy to previously defined policy with name newPolicy
dc::setPolicy newPolicy
```

dc::setToolState

Description

Restores the window using tool state information.

Syntax

```
dc::setToolState [<window: giWindow | int>] -toolState <string> [-exclude  
[<string>]]
```

Arguments

window

Window to restore state.

-toolState

Tool state data to restore.

-exclude

Tool state client names to exclude from restoration.

Returns

<none>

Examples

The example gets the tool state stored in a variable. After that, the viewport is changed and then restored using saved tool state.

```
# Get tool state for $id window
set toolState [dc::getToolState [gi::getWindows $id]]
# Zoom in the design
de::zoom -window $id -factor 2.0
# Set tool state to saved one
dc::setToolState -toolState $toolState
```

dc::showCreateReview

Description

Opens the **Create Review** dialog box to create a review.

Syntax

```
dc::showCreateReview <window: giWindow | int>
```

Arguments

window

Window or window ID of default design.

Returns

<giDialog> : Dialog object.

dc::showEditReview

Description

Opens the **Edit Review** dialog box to update a review.

Syntax

```
dc::showEditReview <window: giWindow | int>
```

Arguments

window

Window or Window ID of default design.

Returns

<giDialog> : Dialog object

dc::showReviewManager

Description

Creates the **Review Manager** window.

Syntax

```
dc::showReviewManager [-show <dmCellView | oaDesign | oaLib>]
```

Arguments

-show

Review repositories to display.

Returns

<giWindow> : Newly created window

dc::startReview

Description

Starts a review on the given design and opens the review database.

Syntax

```
dc::startReview <design: oaDesign>
```

Arguments

design

Name of the design under review.

Returns

<none>

Examples

This example starts a review on given design and opens the review database.

```
set des [ed]
dc::startReview $des
```

idc::addReviewAnnotation

Description

Adds an annotation to a review issue.

Syntax

```
idc::addReviewAnnotation [<context: deContext | unsigned int>] -window
<giWindow | int>
```

Arguments

context

The context in which to activate the command.

-window

Window or window ID.

Returns

<none>

idc::addReviewImage

Description

Adds an image to a review issue.

Syntax

```
idc::addReviewImage [<context: deContext | unsigned int>] -window  
                    <giWindow | int>
```

Arguments

context

The context in which to activate the command.

-window

Window or window ID.

Returns

<none>

Preferences

[Table 8](#) lists dc and idc namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 8 Design Collaboration Preferences

Name	Type	Scope	Default	Description
dcAddReviewAnnotationAngleMode	string	cellview	orthogonal	Angle mode for annotation shape creation in the <code>idc::addReviewAnnotation interactive</code> command.
dcAddReviewAnnotationCalloutText	string	cellview		Text to use for new annotation of type callout in the <code>idc::addReviewAnnotation interactive</code> command.

Table 8 Design Collaboration Preferences (Continued)

Name	Type	Scope	Default	Description
dcAddReviewAnnotationColor	int	cellview	-1	Purpose number of layer-purpose pair (LPP) on annotation layer for which color will be used for new annotation shape in the <code>idc::addReviewAnnotation interactive</code> command.
dcAddReviewAnnotationDialogMode	bool	user	true	<code>idc::addReviewAnnotation</code> command options dialog eject state. Valid values: true, false
dcAddReviewAnnotationIssue	string	cellview		Issue to add a new annotation to in the <code>idc::addReviewAnnotation interactive</code> command.
dcAddReviewAnnotationIssueDescription	string	cellview		Description of the new issue to create in the <code>idc::addReviewAnnotation interactive</code> command.
dcAddReviewAnnotationType	enReviewAnnotation	cellview	arrow	Type of annotation to create in the <code>idc::addReviewAnnotation interactive</code> command. Valid values: arrow, doubleArrow, dimensionInward, dimensionOutward, region, ellipse, wire, callout, calloutObject, calloutHighlight.
dcAddReviewAnnotationWireWidth	float	cellview	0.1	Width for wire annotation type in the <code>idc::addReviewAnnotation interactive</code> command.
dcAddReviewImageIssue	string	cellview		Issue to add new image to in the <code>idc::addReviewImage interactive</code> command.

Table 8 Design Collaboration Preferences (Continued)

Name	Type	Scope	Default	Description
dcAddReviewImageIssueDescription	string	cellview		Description of new issue to create in the <code>idc::addReviewImage</code> interactive command.
dcCreateReviewApprovers	string	user		List of approvers for the Create Review dialog box.
dcCreateReviewAssignee	string	user		Assignee for the Create Review dialog box.
dcCreateReviewDescription	string	user		Review description for the Create Review dialog box.
dcCreateReviewMembers	string	user		List of members for the Create Review dialog box.
dcCreateReviewSeverity	enReviewSeverity	user	information	Severity for the Create Review dialog box. Valid values: enhancement, critical information
dcReviewApprovers	string	cellview		List of UNIX user names that are considered approvers for review items.
dcReviewCellViewStatusProc	string	cellview		Name of Tcl procedure called with a single <code>oaDesign</code> (main design) argument that returns text for the <code>preview.html</code> file.
dcReviewCellViewSuffix	string	cellview	review	Suffix used when naming review cellviews.
dcReviewDefaultSortColumn	enReviewAssistantSortColumns	user	Created	Sets the column to be used by default for sorting. Valid values: Title, Severity, Status, Creator, Assignee, Approvers, Created, {Last Updated}.

Table 8 Design Collaboration Preferences (Continued)

Name	Type	Scope	Default	Description
dcReviewManagerDefaultLibraries	string	cellview		List of zero or more library names to select by default in the Search assistant of the Review Manager window.
dcReviewManagerGroupByColumn	enReviewManagerColumns	cellview	Library	The column to use as the grouping key to create a two level tree in Review Manager when displaying review items. Valid values: Library, Cell View, Assignee, Severity, Status, Creator {}.
dcReviewManagerLibraries	string	cellview		List of currently selected libraries in the Search assistant of the Review Manager window.
dcReviewMembers	string	cellview		List of unix user names that are members of the team responsible for the designs that are reviewed. Aids in selecting people to receive, approve, and be a part of a review.
dcReviewNotificationProc	string	user		Name of the Tcl procedure that sends design review notifications.

5

Design Insights Command Reference

This chapter details the Design Insights commands and preferences.

This chapter provides the list of Design Insights commands and preferences and detailed descriptions of the individual commands.

The commands in the Design Insights tool are grouped into the following categories:

- [di::closeResultsCellView](#)
- [di::createResultsCellView](#)
- [di::createParasiticView](#)
- [di::getCapacitors](#)
- [di::getCapAttr](#)
- [di::getCorners](#)
- [di::getElmoreDelayResult](#)
- [di::getElmoreDelayResultAttr](#)
- [di::getFanoutResult](#)
- [di::getFanoutResultAttr](#)
- [di::getInstAttr](#)
- [di::getInsts](#)
- [di::getLayers](#)
- [di::getModels](#)
- [di::getNetAttr](#)
- [di::getNets](#)
- [di::getNodeAttr](#)
- [di::getResAttr](#)

- [di::getResistanceContributionAttr](#)
- [di::getResistanceResult](#)
- [di::getResistanceResultAttr](#)
- [di::getResistors](#)
- [di::getTermAttr](#)
- [di::getTerms](#)
- [di::getViewType](#)
- [di::listAttrs](#)
- [di::loadResultsCellView](#)
- [di::runElmoreDelayAnalysis](#)
- [di::runFanoutAnalysis](#)
- [di::runResistanceAnalysis](#)
- [Preferences](#)

di::closeResultsCellView

Summary: Close the currently open design insights view.

Syntax: `di::closeResultsCellView`

Returns: `<none>`

Licenses: None.

di::createResultsCellView

Summary: Create design insights view by `rc_preprocessor` with `-path`.

Syntax: `di::createResultCellView[-selectNet <string> | -skipNet <string>]
-lcv<string> [-modelLibs <string>] [-modelSyntax <string>] [-overwrite
<bool>]-paths <string> -type <enum> [-useITFLayerName <bool>] [-vbias
<float>][-vbiasMappingFile <string>]`

Arguments:

`-netID` : Specifies the net id of the capacitors.

`-netName` : Specifies the net name of the capacitors.

-lcv: Target LCV name to create

-modelLibs: Specified the list of the lists to specify the corner, model libs and model sections.

-modelSyntax: Specifies the model syntax type. hspice, eldo, and spectre supported.

-overwrite: Specifies if overwrite parasitic view or not

-paths: Specifies the file paths rc_preprocessorread

-selectNet: Specifies the select net pattern

-skipNet: Specifies the skip net pattern

-type: Specifies the format the rc_preprocessorread. gpd and dspfformats supported. ({ dspf | gpd })

-useITFLayerName: Specify if the layer use ITF layer or not

-vbias: Specifies the vbias number

-vbiasMappingFile: Specified the list of the lists to specify the corner, model libs and model sections.

Returns:<xtJob>: Return the submitted job object

Licenses: None

```
di::createResultsCellView -lcv test2/test3/parasitic -paths p22.gpd -type  
gpd -useITFLayerName true -modelLibs  
{reference40_models.lib TT typical_25}{reference40_models.lib SS  
typical_125}{reference40_models.lib TT typical_M40}} -overwrite 1
```

di::createParasiticView

Summary: Create parasitic view by rc_preprocessor with -path.

Syntax: di::createParasiticView [-selectNet <string> | -skipNet <string>]
-lcv <string> [-modelLibs <string>] [-modelSyntax <string>] [-overwrite
<bool>] -paths <string> -type <enum> [-useITFLayerName <bool>] [-vbias
<float>] [-vbiasMappingFile <string>]

Arguments:

-lcv : Target LCV name to create

-modelLibs : Specified the list of the lists to specify the corner, model libs and model sections.

-modelSyntax : Specifies the model syntax type. hspice, eldo, and spectre supported.

`-overwrite` : Specifies if overwrite parasitic view or not

`-paths` : Specifies the file paths `rc_preprocessor read`

`-selectNet` : Specifies the select net pattern

`-skipNet` : Specifies the skip net pattern

`-type` : Specifies the format the `rc_preprocessor read`. `gpd` and `dspf` formats supported. `( { dspf | gpd } )`

`-useITFLayerName` : Specify if the layer use ITF layer or not

`-vbias` : Specifies the vbias number

`-vbiasMappingFile` : Specified the list of the lists to specify the corner, model libs and model sections.

Returns: `<xtJob>` : Return the submitted job object

Licenses: None

di::getCapacitors

Summary: Report the capacitors' ids from the specified net id/name.

Syntax: `di::getCapacitors [-netID <int> | -netName <string>]`

Arguments:

`-netID` : Specifies the net id of the capacitors.

`-netName` : Specifies the net name of the capacitors.

Returns: `<string>` : Return list of the capacitors' ids.

Licenses: None

di::getCapAttr

Summary: Get capacitor data by specified capacitor id and attribute name.

Syntax: `di::getCapAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getCorners

Summary: Get corners information

Syntax: di::getCorners

Arguments: Returns: <string> : corners information

Licenses: None

di::getElmoreDelayResult

Summary: Get the Elmore Delay Result for current loaded parasitic view.

Syntax: di::getElmoreDelayResult [-net <string>] -type <enum>

Arguments:

-net : Specifies the target net name.

-type : Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns: <string> : Return the ids of the summary.

Licenses: None

```
di::loadResultsCellView -lcv Test/aaa/parasitic
set net [di::getNets -name "FOUT"]
set job [di::runElmoreDelayAnalysis -mode automatic -nets $net -corners
"typical_M40 typical_125"]
xt::wait $job
set summary_ids [di::getElmoreDelayResult -type all]
foreach summary_id $summary_ids {
  set attrs [di::getElmoreDelayResultAttr -id $summary_id -attribute "net
elmore type bandwidth transitionTime drivers loads"]
  puts "net [lindex $attrs 0]"
  puts "elmore [lindex $attrs 1]"
  puts "type [lindex $attrs 2]"
  puts "bandwidth [lindex $attrs 3]"
  puts "transitionTime [lindex $attrs 4]"
  set drivers [lindex $attrs 5]
  set loads [lindex $attrs 6]
  puts "drivers info"
```

```
foreach driver $drivers {
  puts [di::getTermAttr -id $driver]
}
puts "loads info"
foreach load $loads {
  puts [di::getTermAttr -id $load]
}
}
```

di::getElmoreDelayResultAttr

Summary: Get the Elmore Delay Result for current loaded parasitic view.

Syntax: `di::getElmoreDelayResultAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: `<string>` : Return list of the attribute data.

Licenses: None

di::getFanoutResult

Summary: Get the Fanout Result for current loaded parasitic view.

Syntax: `di::getFanoutResult [-net <string>] -type <enum>`

Arguments:

`-net` : Specifies the target net name.

`-type` : Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns: `<string>` : Return the ids of the summary.

Licenses: None

```
di::loadResultsCellView -lcv Test/aaa/parasitic
set net [di::getNets -name "FOUT"]
set job [di::runFanoutAnalysis -mode automatic -nets $net]
xt::wait $job
```

```
set summary_ids [di::getFanoutResult -type all]
foreach summary_id $summary_ids {
    set attrs [di::getFanoutResultAttr -id $summary_id -attribute "net
fanout type driver_c interconnect_c load_c drivers loads"]
    puts "net [lindex $attrs 0]"
    puts "fanout [lindex $attrs 1]"
    puts "type [lindex $attrs 2]"
    puts "driver_c [lindex $attrs 3]"
    puts "interconnect_c [lindex $attrs 4]"
    puts "load_c [lindex $attrs 5]"
    set drivers [lindex $attrs 6]
    set loads [lindex $attrs 7]
    puts "drivers info"
    foreach driver $drivers {
        puts [di::getTermAttr -id $driver]
    }
    puts "loads info"
    foreach load $loads {
        puts [di::getTermAttr -id $load]
    }
}
```

di::getFanoutResultAttr

Summary: Get the Fanout Result Attribute data from current fanout result.

Syntax: `di::getFanoutResultAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: `<string>` : Return list of the attribute data.

Licenses: None

di::getInstAttr

Summary: Get instance data by specified instance id and attribute name.

Syntax: `di::getInstAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getInsts

Summary: Report the list of the instances' ids from the netlist.

Category: Design Insights

Syntax: `di::getInsts [-name <string> | -netId <int> | -netName <string> | -termID <int> | -termName <string>]`

Arguments:

`-name` : Specifies the name of the instance.

`-netId` : Specifies net name of the instance.

`-netName` : Specifies net name of the instance.

`-termID` : Specifies term id of the net.

`-termName` : Specifies term name of the instance.

Returns: <string> : Return list of the instances' ids.

Licenses: None

```
di::loadResultsCellView -lcv di_test/di/parasitic
set instances [di::getInsts -name XI1|MM0]
foreach instance $instances {
    set instance_attribute [di::getInstAttr -id $instance -attribute
        "name model type parameters"]
    puts "instance name [lindex $instance_attribute 0] information"
    puts "-----"
    puts "model [lindex $instance_attribute 1]"
    puts "type [lindex $instance_attribute 2]"
    puts "parameters [lindex $instance_attribute 3]"
}
di::closeResultsCellView
```

di::getLayers

Summary: Get layers information

Syntax: `di::getLayers [-layerName <string>]`

Arguments:

`-layerName` : specified layer name

Returns: `<string>` : layers information

Licenses: None

di::getModels

Summary: Get models information

Syntax: `di::getModels [-modelName <string>]`

Arguments:

`-modelName` : specified model name

Returns: `<string>` : models information

Licenses: None

di::getNetAttr

Summary: Get net data by specified net id and attribute name.

Syntax: `di::getNetAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: `<string>` : Return list of the attribute data.

Licenses: None

di::getNets

Summary: Report the list of the nets' ids from the netlist.

Syntax: `di::getNets [-name <string> | -termID <int> | -termName <string>]`

Arguments:

`-name` : Specifies the name of the net

`-termID` : Specifies term id of the net.

`-termName` : Specifies term name of the net.

Returns: `<string>` : Return list of the nets' ids.

Licenses: None

```
di::loadResultsCellView -lcv di_test/di/parasitic
set nets [di::getNets -name "ck*"]
foreach net $nets {
    set net_attribute [di::getNetAttr -id $net -attribute "name cg cc
interconnect_c term_c total_c r_count term_count max_r max_r_layer"]
    puts "Net name [lindex $net_attribute 0] information"
    puts "-----"
    puts "cg [lindex $net_attribute 1]"
    puts "cc [lindex $net_attribute 2]"
    puts "interconnect_c [lindex $net_attribute 3]"
    puts "term_c [lindex $net_attribute 4]"
    puts "total_c [lindex $net_attribute 5]"
    puts "r_count [lindex $net_attribute 6]"
    puts "term_count [lindex $net_attribute 7]"
    puts "max_r [lindex $net_attribute 8]"
    puts "max_r_layer [lindex $net_attribute 9]"
    puts "-----"
}
```

di::getNodeAttr

Summary: Get node data by specified node id and attribute name.

Syntax: `di::getNodeAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getResAttr

Summary: Get resistor data by specified resistor id and attribute name.

Syntax: `di::getResAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

```
di::loadParasiticDB -lcv di_test/di/parasitic
set resistances [di::getResistors -netName net193]
puts "dump res"
foreach resistance $resistances {
    # just get net node1 node2 position(llx lly urx ury) and value
    set resistance_attribute [di::getResAttr -id $resistance -attribute
"net node1 node2 llx lly urx ury value"]
    puts "net [lindex $resistance_attribute 0]"
    puts "node1 [lindex $resistance_attribute 1]"
    puts "node2 [lindex $resistance_attribute 2]"
    puts "position \{[lindex $resistance_attribute 3] , [lindex
$resistance_attribute 4], [lindex $resistance_attribute 5], [lindex
$resistance_attribute 6]\}"
    puts "value [lindex $resistance_attribute 7]"
}

puts "dump caps"

set capacitances [di::getCapacitors -netName net193]
foreach cap $capacitances {
    set cap_attribute [di::getCapAttr -id $cap -attribute "net1 node1
net2 node2 value"]
    puts "net1 [lindex $cap_attribute 0]"
    puts "node1 [lindex $cap_attribute 1]"
    puts "net2 [lindex $cap_attribute 2]"
    puts "node2 [lindex $cap_attribute 3]"
    puts "value [lindex $cap_attribute 4]"
}
```

```
}  
di::closeParasiticDB
```

di::getResistanceContributionAttr

Summary: Get the Effective Resistance Contribution Attribute data from opened effective resistance result.

Syntax: `di::getResistanceContributionAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: `<string>` : Return list of the attribute data.

Licenses: None

di::getResistanceResult

Summary: Get the Effective Resistance Result for current loaded parasitic view.

Syntax: `di::getResistanceResult [-net <string>] -type <enum>`

Arguments:

`-net` : Specifies the target net name.

`-type` : Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns: `<string>` : Return the ids of the summary.

Licenses: None

```
di::loadResultsCellView -lcv Test/aaa/parasitic  
set from [di::getTerms -name pfd|inv1|M1:GATE]  
set to [di::getTerms -name pfd|inv1|M3:GATE]  
set job [di::runResistanceAnalysis -mode p2p -drivers $from -loads $to]  
xt::wait $job  
set summary_ids [di::getResistanceResult -type all]  
foreach summary_id $summary_ids {  
    set attrs [di::getResistanceResultAttr -id $summary_id -attribute "net r  
    lrp_load lrp_driver lrp_r type drivers loads contributions"]
```



```
puts "net [lindex $attrs 0]"
puts "r [lindex $attrs 1]"
puts "lrp_load [lindex $attrs 2]"
puts "lrp_driver [lindex $attrs 3]"
puts "lrp_r [lindex $attrs 4]"
puts "type [lindex $attrs 5]"
set drivers [lindex $attrs 6]
set loads [lindex $attrs 7]
set contributions [lindex $attrs 8]
puts "drivers info"
foreach driver $drivers {
    puts [di::getTermAttr -id $driver]
}
puts "loads info"
foreach load $loads {
    puts [di::getTermAttr -id $load]
}
puts "contribution infos"
foreach contribution $contributions {
    puts [di::getResistanceContributionAttr -id $contribution]
}
}
```

di::getResistanceResultAttr

Summary: Get the Effective Resistance Result for current loaded parasitic view.

Syntax: `di::getResistanceResultAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: `<string>` : Return list of the attribute data.

Licenses: None

di::getResistors

Summary: Report the resistors' ids from the specified net id/name.

Category: Design Insights

Syntax: `di::getResistors [-netID <int> | -netName <string>]`

Arguments:

-netID : Specifies the net id of the resistors.

-netName : Specifies the net name of the resistors.

Returns: <string> : Return list of the resistors' ids.

Licenses: None

di::getTermAttr

Summary: Get term data by specified term id and attribute name.

Syntax: `di::getTermAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getTerms

Summary: Report the list of the terms' ids from the netlist.

Syntax: `di::getTerms [-instID <int> | -instName <string> | -name <string> | -netID <int> | -netName <string>]`

Arguments:

-instID : Specifies instance id of the terms.

-instName : Specifies instance name of the terms.

-name : Specifies the name of the terms.

-netID : Specifies net id of the terms.

-netName : Specifies net name of the terms.

Returns: <string> : Return list of the terms' ids.

Licenses: None

```
di::loadResultsCellView -lcv di_test/di/parasitic
set terms [di::getTerms -netName net193]
foreach term $terms {
    set term_attribute [di::getTermAttr -id $term -attribute "name nodes
net cap ron type"]
    puts "term name [lindex $term_attribute 0] information"
    puts "-----"
    puts "nodes [lindex $term_attribute 1]"
    puts "net [lindex $term_attribute 2]"
    puts "cap [lindex $term_attribute 3]"
    puts "ron [lindex $term_attribute 4]"
    puts "type [lindex $term_attribute 5]"
}
di::closeResultsCellView
```

di::getViewType

Summary: Get view type.

Syntax: di::getViewType

Returns: <string> : View type

Licenses: None

di::listAttrs

Summary: List all attributes for specified type of the di::get*Attr commands

Syntax: di::listAttrs -type <enum>

Arguments:

-type : List all attributes for specified type of the di::get*Attr commands. ({ cap | elmoreDelayResult | fanoutResult | inst | net | node | res | resistanceContributionResult | resistanceResult | term })

Returns: <string> : The list of the attribute name

Licenses: None

di::loadResultsCellView

Summary: Open a PA view.

Syntax: di::loadResultsCellView -lcv <dmCellView | dmCellViewName>

Arguments:

-lcv : Specify a PA cell view to be opened

Returns: <none>

Licenses: None

di::runElmoreDelayAnalysis

Summary: Get the Elmore delay result for currently loaded parasitic view.

Syntax: di::runElmoreDelayAnalysis [-corners <string>] [-drivers <string>]
[-includeDeviceTermCapacitance <bool>] [-loads <string>] -mode <enum>
[-nets <string>]

Arguments:

-corners : Specify the list of the corner name.

-drivers : Specifies the list of the driver's term id.

-includeDeviceTermCapacitance : Specify if the analysis have to include term cap or not.

-loads : Specifies the list of the load's term id.

-mode : Specifies mode of the command. ({ automatic | p2p })

-nets : Specifies list of the net id.

Returns: <xtJob> : Return the submitted job object

Licenses: None

di::runFanoutAnalysis

Summary: Get the fanout result for currently loaded parasitic view.

Syntax: di::runFanoutAnalysis [-corners <string>] [-drivers <string>]
[-loads <string>] -mode <enum> [-nets <string>]

Arguments:

-corners : Specify the list of the corner name.

-drivers : Specifies the list of the driver's term id.

-loads : Specifies the list of the load's term id.

`-mode` : Specifies mode of the command. ({ automatic | p2p }) `-nets` : Specifies list of the net id.

Returns: <xtJob> : Return the submitted job object

Licenses: None

di::runResistanceAnalysis

Summary: Get the effective resistance result for current loaded parasitic view.

Syntax: `di::runResistanceAnalysis [-corners <string>] [-drivers <string>] [-loads <string>] -mode <enum> [-nets <string>]`

Arguments:

`-corners` : Specify the list of the corner name.

`-drivers` : Specifies the list of the driver's term id.

`-loads` : Specifies the list of the load's term id.

`-mode` : Specifies mode of the command. ({ automatic | instP2p | p2p })

`-nets` : Specifies list of the net id.

Returns: <xtJob> : Return the submitted job object

Licenses: None

Preferences

The following table provides the preferences supported for Design Insights.

diCanvasClear	int	cellview	0	Clear	
diCanvasCrossSelectMode	bool	cellview	false	Cross select mode	true, false
diCanvasFitMode	bool	cellview	false	Fit mode	true, false
diCanvasShadowMode	bool	cellview	false	Shadow mode	true, false
diHighlightPacket	string	cellview		Highlight packet	

diPreprocessorOaOutputDir	string	cellview		OA output directory	
diPreprocessorRonMode	int	cellview	0	R_on Mode	
diPreprocessorSqlOutputDir	string	cellview		SQL output directory	
diPwdeHighlightToSchematic	bool	cellview	false	Highlight to schematic	true, false
diQueryNavigatorExpandAllWithConnectivity	bool	cellview	true	Expand all with connectivity	true, false
diShowNonPhysicalResistors	bool	cellview	false	Show non-physical resistors	true, false

6

Database Tcl

This chapter lists commands, enumerations, objects, and preferences in the Custom Compiler Tcl db namespace. This namespace provides Tcl for database operations.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

Interactive commands are listed after non-interactive commands in this section.

The commands covered are:

- [db::add](#)
- [db::addAttr](#)
- [db::addFigGroupMembers](#)
- [db::addGroupMembers](#)
- [db::addInit](#)
- [db::addListener](#)
- [db::addMarkerObject](#)
- [db::append](#)
- [db::appendLibertyEditorPinData](#)
- [db::attachTech](#)
- [db::batch](#)

- [db::cat](#)
- [db::checkDRD](#)
- [db::checkFeature](#)
- [db::checkForInterrupt](#)
- [db::checkLicenseTier](#)
- [db::checkoutFeature](#)
- [db::checkVersion](#)
- [db::cleanPcellRepository](#)
- [db::closeCDF](#)
- [db::closeDesign](#)
- [db::copyLibertyData](#)
- [db::copyParams](#)
- [db::copyPrefs](#)
- [db::createAssignAssignment](#)
- [db::createAssignValue](#)
- [db::createCallback](#)
- [db::createCircularRuler](#)
- [db::createCircularRulerDef](#)
- [db::createCollection](#)
- [db::createDesign](#)
- [db::createDeviceMappingFileDialog](#)
- [db::createEnum](#)
- [db::createEvaluator](#)
- [db::createExpressionMap](#)
- [db::createFigGroup](#)
- [db::createGenOptionSet](#)
- [db::createGroup](#)

- [db::createHistogram](#)
- [db::createImage](#)
- [db::createIncludeFileControl](#)
- [db::createInstCache](#)
- [db::createInstTerm](#)
- [db::createInterfaceParam](#)
- [db::createInterfacePin](#)
- [db::createInterfaceSet](#)
- [db::createInterruptHandler](#)
- [db::createLibListInput](#)
- [db::createLinearRuler](#)
- [db::createList](#)
- [db::createMarker](#)
- [db::createNet](#)
- [db::createNetlistInst](#)
- [db::createNotifier](#)
- [db::createObject](#)
- [db::createPaperSize](#)
- [db::createParamDef](#)
- [db::createParamSandbox](#)
- [db::createParasiticStencil](#)
- [db::createPatternDef](#)
- [db::createPcell](#)
- [db::createPcellDef](#)
- [db::createPin](#)
- [db::createPref](#)
- [db::createPrinter](#)

- [db::createProgress](#)
- [db::createProp](#)
- [db::createScope](#)
- [db::createScopeType](#)
- [db::createScratchDesign](#)
- [db::createStdViaVariant](#)
- [db::createStreamNameMap](#)
- [db::createSubCircularRuler](#)
- [db::createWindowAnnotationSet](#)
- [db::decodeSymbol](#)
- [db::defineUdvDevice](#)
- [db::defineVflagMaxDevice](#)
- [db::defineVflagMaxMinDevice](#)
- [db::defineVflagMinDevice](#)
- [db::defineVsyncDevice](#)
- [db::deleteBuiltInVias](#)
- [db::deprecateAttr](#)
- [db::destroy](#)
- [db::detectNetlistType](#)
- [db::enableTkSend](#)
- [db::engToSci](#)
- [db::eval](#)
- [db::evalCallback](#)
- [db::evalEnvironment](#)
- [db::evalParam](#)
- [db::exportImage](#)
- [db::exportLibertyFile](#)

- `db::exportMCellFile`
- `db::exportParams`
- `db::exportStream`
- `db::exportTechFile`
- `db::expr`
- `db::filter`
- `db::findCircuitMatches`
- `db::findConnections`
- `db::findConstraint`
- `db::findContextConnectionDef`
- `db::findContextConnections`
- `db::findGenerator`
- `db::findInterfaceReader`
- `db::findPatternMatches`
- `db::flushPcellCache`
- `db::foreach`
- `db::freezeRefs`
- `db::generateBuiltInVias`
- `db::generateCDL`
- `db::generateCellView`
- `db::generateConstraintPatterns`
- `db::generateDeviceMappingTemplate`
- `db::generateLayout`
- `db::generateRule`
- `db::generateSchematic`
- `db::generateSPICE`
- `db::generateSystemVerilog`

- [db::generateVerilog](#)
- [db::generateVerilogA](#)
- [db::generateVerilogAMS](#)
- [db::generateVHDL](#)
- [db::getAnnotationSets](#)
- [db::getAssignmentDefs](#)
- [db::getAssignments](#)
- [db::getAttr](#)
- [db::getBuiltinNetlists](#)
- [db::getCallbacks](#)
- [db::getCellViewInterface](#)
- [db::getCircularRulerDefs](#)
- [db::getComponents](#)
- [db::getConnectionDefState](#)
- [db::getContextConnectionDefs](#)
- [db::getCount](#)
- [db::getCurrentContext](#)
- [db::getCurrentEvalText](#)
- [db::getCurrentParam](#)
- [db::getCurrentRef](#)
- [db::getCutClasses](#)
- [db::getDeviceDisplays](#)
- [db::getEffectiveNet](#)
- [db::getEnums](#)
- [db::getEquivalentNets](#)
- [db::getEvaluators](#)
- [db::getExportOasisOptions](#)

- [db::getExportStreamOptions](#)
- [db::getFeatures](#)
- [db::getFigGroupMembers](#)
- [db::getFigGroups](#)
- [db::getFlightlineCoordinates](#)
- [db::getGenOptionSets](#)
- [db::getGroupMembers](#)
- [db::getGroups](#)
- [db::getICCCCellViews](#)
- [db::getIndex](#)
- [db::getInstHeaders](#)
- [db::getInsts](#)
- [db::getInstTerms](#)
- [db::getLayers](#)
- [db::getLicenseTier](#)
- [db::getLibertyEditorPinsData](#)
- [db::getLPPHeaders](#)
- [db::getMarkers](#)
- [db::getNetlistInfo](#)
- [db::getNets](#)
- [db::getNext](#)
- [db::getNotifiers](#)
- [db::getOpenDesigns](#)
- [db::getPaperSizes](#)
- [db::getParamDefs](#)
- [db::getParamFilter](#)
- [db::getParams](#)

- [db::getParamValue](#)
- [db::getPatternDefs](#)
- [db::getPatternInst](#)
- [db::getPcellCacheLocation](#)
- [db::getPcellCacheSize](#)
- [db::getPcellDefs](#)
- [db::getPcells](#)
- [db::getPins](#)
- [db::getPrefs](#)
- [db::getPrefValue](#)
- [db::getPrinters](#)
- [db::getProcessInfo](#)
- [db::getProps](#)
- [db::getRegisteredNetlistInfo](#)
- [db::getRulers](#)
- [db::getScopes](#)
- [db::getScopeTypes](#)
- [db::getShapes](#)
- [db::getTableValue](#)
- [db::getTerms](#)
- [db::getType](#)
- [db::getVDRCDDeviceList](#)
- [db::getVDRCDDeviceParamName](#)
- [db::getVDRCDDeviceVoltage](#)
- [db::getVersion](#)
- [db::getViaDefs](#)
- [db::getViaHeaders](#)

- [db::getVias](#)
- [db::getVirtualInstTerms](#)
- [db::getWindowAnnotationSets](#)
- [db::hideProps](#)
- [db::importEDIF](#)
- [db::importMCellFile](#)
- [db::importTechFile](#)
- [db::importText](#)
- [db::includes](#)
- [db::init](#)
- [db::instCacheTriggerParamCallbacks](#)
- [db::interrupt](#)
- [db::isEmpty](#)
- [db::isEncrypted](#)
- [db::isObject](#)
- [db::isProhibitedName](#)
- [db::listAttrs](#)
- [db::listCallbackTypes](#)
- [db::loadDeviceDisplays](#)
- [db::loadGenOptionSets](#)
- [db::loadMCells](#)
- [db::loadModelFile](#)
- [db::loadPrefs](#)
- [db::loadStreamNameMap](#)
- [db::localizePrefs](#)
- [db::makePcellEditable](#)
- [db::mapStreamName](#)

- [db::match](#)
- [db::md5sum](#)
- [db::monitorChannel](#)
- [db::notify](#)
- [db::openDesign](#)
- [db::openHier](#)
- [db::openDesignHierarchy](#)
- [db::openParamDefEditor](#)
- [db::openScope](#)
- [db::pdaExplainCell](#)
- [db::pdaShowCell](#)
- [db::print](#)
- [db::purgeCDF](#)
- [db::purgePrefs](#)
- [db::query](#)
- [db::queryFeature](#)
- [db::raiseLicenseTier](#)
- [db::refreshICCCache](#)
- [db::refreshMCells](#)
- [db::registerCallbackRegsub](#)
- [db::registerGenerator](#)
- [db::registerInterfaceReader](#)
- [db::registerNetlistInfo](#)
- [db::registerParamEvalFunction](#)
- [db::relativePath](#)
- [db::releaseFeature](#)
- [db::remove](#)

- [db::removeAttr](#)
- [db::removeFigGroupMembers](#)
- [db::removeMarkerObject](#)
- [db::removeLibertyEditorPinData](#)
- [db::renderTemplatesParasitics](#)
- [db::reopenDesign](#)
- [db::reportCallbacks](#)
- [db::reset](#)
- [db::resetVDRCDefinedDevices](#)
- [db::resolve](#)
- [db::resolvePath](#)
- [db::revertParam](#)
- [db::revertPrefs](#)
- [db::saveCDF](#)
- [db::saveDesign](#)
- [db::saveDesignAs](#)
- [db::saveDeviceDisplays](#)
- [db::saveGenOptionSets](#)
- [db::savePrefs](#)
- [db::sciToEng](#)
- [db::setAnalogLibCompatibleVersion](#)
- [db::setAttr](#)
- [db::setColorShiftLayers](#)
- [db::setDeviceDisplay](#)
- [db::setInstName](#)
- [db::setLicenseTier](#)
- [db::setMaster](#)

- [db::setNetlistInfo](#)
- [db::setParamFilter](#)
- [db::setParamValue](#)
- [db::setPrefValue](#)
- [db::setProgress](#)
- [db::setViaDef](#)
- [db::showAbstractGeneratorOptions](#)
- [db::showAdvancedPrintOptions](#)
- [db::showBindExternalNetlistFile](#)
- [db::showBindExternalNetlistView](#)
- [db::showCellViewFromCellView](#)
- [db::showCellViewFromPinList](#)
- [db::showDummyRules](#)
- [db::showEditComponents](#)
- [db::showExportDef](#)
- [db::showExportImage](#)
- [db::showExportLef](#)
- [db::showExportLiberty](#)
- [db::showExportMCellFile](#)
- [db::showExportMilkyway](#)
- [db::showExportNetlist](#)
- [db::showExportOasis](#)
- [db::showExportStream](#)
- [db::showFindReplace](#)
- [db::showGenerateAbstract](#)
- [db::showGenerateSchematic](#)
- [db::showGenerateSchematicsFromText](#)

- [db::showGenericPrint](#)
- [db::showImportDef](#)
- [db::showImportEDIF](#)
- [db::showImportLef](#)
- [db::showImportMCellFile](#)
- [db::showImportMilkyway](#)
- [db::showImportOasis](#)
- [db::showImportSimulationDataDialog](#)
- [db::showImportStream](#)
- [db::showImportText](#)
- [db::showImportVerilog](#)
- [db::showLibertyDataEditor](#)
- [db::showLoadPreferences](#)
- [db::showManageTechnology](#)
- [db::showPrint](#)
- [db::showPrintQueueStatus](#)
- [db::showPurgePreferences](#)
- [db::showRefreshMCells](#)
- [db::showSavePreferences](#)
- [db::showUnitDisplayScale](#)
- [db::showUpdateCellView](#)
- [db::showVDRCManagerWindow](#)
- [db::snapPoints](#)
- [db::sort](#)
- [db::source](#)
- [db::startStatistics](#)
- [db::stopStatistics](#)

- `db::transform`
- `db::teeLogFile`
- `db::triggerParamCallback`
- `db::unregisterGenerator`
- `db::unregisterInterfaceReader`
- `db::updateCDFParams`
- `db::vdrcMgrAddNet`
- `db::vdrcMgrAddNetPair`
- `db::vdrcMgrAnnotateNet`
- `db::vdrcMgrAnnotateNetPair`
- `db::vdrcMgrCheckHierarchyMismatch`
- `db::vdrcMgrCheckSymbolPlacement`
- `db::vdrcMgrClearAllData`
- `db::vdrcMgrDeleteNet`
- `db::vdrcMgrDeleteNetPair`
- `db::vdrcMgrImportLayout`
- `db::vdrcMgrImportSchematic`
- `db::vdrcMgrLoadDRCViolations`
- `db::vdrcMgrRefresh`
- `db::viaDefExists`
- `db::viewExists`
- `db::vwait`
- `idb::createImage`
- `idb::query`

db::add

Description

Adds an object to a collection. Only supports collections built on concrete realizations. The ordering of the destination collection after the execution of the command is not defined and any `db::getNext` iterators are invalidated.

Syntax

```
db::add <collection: dbIterator> -to <dbIterator>
```

Arguments

`collection: {oaObjectdbObject}`

An iterator over the `dbCollection`.

`-to {oaObjectdbObject}`

A `dbCollection` to which to add elements.

Returns

<none>

Examples

The following example adds the net with name `enable` to the collection of nets whose names start with `analog`.

```
set analogNets [db::getNets analog* -of [ed]]
set enable [db::getNets enable -of [ed]]
db::add $enable -to $analogNets
```

db::addAttr

Description

Adds the specified attribute to the given object.

Syntax

```
db::addAttr <name: dbVarAttrName> [-of <dbCollection | dbIterator>]
[-valueType <enum>] -value <variant> [-type <enum>]
```

Arguments

`name`

The attribute name or hierarchical sequence of names to add separated by periods (`"."`).

`-of`

The objects on which the attribute is added.

`-valueType`

The type of the attribute value to create. Only used for prop attribute type:
(`{boolean | double | float | hierarchical | integer | list | string}`, default is: `string`).

`-value`

The attribute value to add.

`-type`

Forces only attributes of the specified type to be added (`{all | appDef | intrinsic | param | prop }`, default is: `all`).

Returns

`<none>`

Examples

```
# Create an object of dbObject type
set obj [db::createObject ]

# Add attributes to the object
db::addAttr name -of $obj -valueType string -value "vdd!"
db::addAttr nettype -of $obj -valueType string -value "power"
db::addAttr isGlobal -of $obj -valueType boolean -value "true"

# Print attributes of the object
puts [db::listAttrs -of $obj]
```

db::addFigGroupMembers

Description

Creates a new figure group member (`figGroupMem`) object making the figure a member of the `figGroup`.

Syntax

```
db::addFigGroupMembers <objects: {<deFigure>} | {<deSelectionSet>} |  
{<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |  
{<oaVia>}} -group <deFigure | oaFigGroup | string> [-design <oaDesign>]
```

Arguments

`objects`

The figure to add.

`-group`

The figGroup in which the figure should be added.

`-design`

The design in which the group belongs.

Returns

<none>

Examples

The following example creates a figure group object and adds an instance to that group.

```
set design [db::getAttr editDesign -of [de::getActiveContext]]
set inst [db::getInsts I0 -of $design]
set figGroup [db::createFigGroup instGroup -design $design]
db::addGroupMembers $inst -group $figGroup
```

db::addGroupMembers

Description

Creates group members for the specified objects in the specified group.

Syntax

```
db::addGroupMembers -group <oaGroup> -objects <{<deFigure>} |
{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaGroup>} |
{<oaInst>} | {<oaInstHeader>} | {<oaInstTerm>} | {<oaLPPHeader>} |
{<oaMarker>} | {<oaNet>} | {<oaPin>} | {<oaProp>} | {<oaShape>} |
{<oaTerm>} | {<oaVia>} | {<oaViaHeader>}} [-isLeader <bool>]
```

Arguments

`-group`

The group to add object to.

`-objects`

The object to add to group. If multiple objects are passed, only the first one is created.

`-isLeader`

Specifies this group member as the leader of the group. (Default is: false)

Returns

<none>

Examples

The following example creates groups and adds a rectangle object as a leader and a text object as a follower.

```
set design [db::getAttr editDesign -of [de::getActiveContext]]
set transaction [de::startTransaction "Create Group" -design $design]
set group [db::createGroup -design $design ]
set rect [le::createRectangle {{0 0} {0.8 0.8}} -lpp m1 -design $design]
db::addGroupMembers -group $group -isLeader 1 -objects $rect
set label [le::createLabel VDD -lpp text -parent $design -origin {0.4 0.4}
    -height 0.4]
db::addGroupMembers -group $group -isLeader 0 -objects $label
de::endTransaction $transaction
```

db::addInit

Description

Evaluates a Tcl expression when a specific object is requested. This command can be used to defer loading Tcl support routines until specific functionality is needed. The identifier is a unique identifier for the specified object type and is frequently the object name.

Syntax

```
db::addInit -type <string> -id <string> -eval <script>
```

Arguments

-type

Object type.

-id

Object identifier, frequently its a name.

-eval

Tcl expression to evaluate when the object is requested.

Returns

<none>

Errors

Unsupported object type. Supported types include giWindowType, giAssistant, giAction, giMenu, giToolbar, deTool, deApplication and dbPref.

Examples

```
# Define proc which creates new action
proc createNewAction {} {gi::createAction newAction -command "ise::copy"
    -title "New Copy"}

# createNewAction proc will be evaluated when newAction is requested
db::addInit -type giAction -id newAction -eval {createNewAction}

# request newAction by using gi::addAction command
gi::addActions newAction -to [gi::getMenus seSchEditMenu -from
    [gi::getActiveWindow ]]
```

db::addListener

Description

Adds a listener to a notifier. Each notifier has a unique set of arguments passed to the registered procedure. In addition, it is possible to bind additional arguments by embedding them within the specified procedure.

Syntax

```
db::addListener <proc: [variant]> -to <dbNotifier>
```

Arguments

proc

Procedure to invoke.

-to

Notifier to listen to.

Returns

<dbListener> : Newly created listener.

Examples

```
# Create an object
set obj [db::createObject ]

# Add attributes to the object
db::addAttr name -of $obj -valueType string -value "nmos"
db::addAttr l -of $obj -valueType double -value 1u
db::addAttr w -of $obj -valueType double -value 0.1u

# Create notifiers for each attribute. Each notifier notifies the object to
# set its attribute value
set n1 [db::createNotifier nmosAttrSet_l -to $obj -args string \
    -description "NMOS L parameter set notifier"]
set n2 [db::createNotifier nmosAttrSet_w -to $obj -args string \
    -description "NMOS W parameter set notifier"]
```

```
# Set listeners to the notifiers
db::addListener [list nmosValueSet_l $obj ] -to $n1
db::addListener [list nmosValueSet_w $obj ] -to $n2

# Create procedures which handles object attribute
proc nmosValueSet_l {obj value} {
    puts "This is nmos l attribute set listener proc, the value is $value"
    if {$value >= 0} {
        db::setAttr obj.l -value $value
    }
}

# Create procedures which handles object attribute
proc nmosValueSet_w {obj value} {
    puts "This is nmos w attribute set listener proc, the value $value"
    if {$value >= 0} {
        db::setAttr obj.w -value $value
    }
}

# Get the object notifier
set n [db::getNotifiers nmosAttrSet_l -of $obj]
# Notify the object to set the l attribute
db::notify $n -args 5u
# Check if the attribute is set
puts [db::getAttr obj.l]

# Get the object notifier
set n [db::getNotifiers nmosAttrSet_w -of $obj]
# Notify the object to set the w attribute
db::notify $n -args 0.5u
# Check if the attribute is set
puts [db::getAttr obj.w]
```

db::addMarkerObject

Description

Adds the specified object to the list of objects associated with the marker.

Syntax

```
db::addMarkerObject <objects: oaBlockObject | {<deFigure>} |
    {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
    {<oaMarker>} | {<oaShape>} | {<oaVia>}> -marker <deFigure | oaMarker>
```

Arguments

objects

The objects to add to marker.

-marker

The marker for the objects.

Returns

<none>

Examples

The following example removes m1 shape object from the marker.

```
set design [db::getAttr editDesign -of [de::getActiveContext]]
set obj [db::getNext [db::getShapes -lpp m1 -of $design ]]
set marker [db::getMarkers -of $design \
-filter {[db::includes %objects -object $obj]]}]
set transaction [de::startTransaction "Remove Marker " -design $design]
db::removeMarkerObject $obj -marker $marker
de::endTransaction $transaction
```

db::append

Description

Merges two collections into a single collection. The position of the getNext iterators on both collections is irrelevant; the entire first collection is appended to the entire second collection. The input collections are not modified. The ordering of the output collection is not defined.

Syntax

```
db::append <collection: dbCollection | dbIterator> -to <dbCollection
|dbIterator>
```

Arguments

collection: {oaObjectdbObject}

An iterator over the dbCollection.

-to {oaObjectdbObject}

A dbCollection to which to append elements.

Returns

The merged dbCollection ({oaObjectdbObject})

Examples

```
set des [ed]

# Create a collection of the nets of the design, which name attribute starts with
# O
set nets1 [db::getNets O* -of $des]

# Print the name of nets from the collection nets1
puts [db::createList [db::getAttr nets1.name]]
```

```
# Create a collection of the nets of the design, which name attribute starts with
I
set nets2 [db::getNets I* -of $des]

# Print the name of nets from the collection nets1
puts [db::createList [db::getAttr nets2.name]]

# Create a new collection appending net1 collection to net2 collection
set res [db::append $nets1 -to $nets2]

# Print the name of nets from the resulting collection
puts [db::createList [db::getAttr res.name]]
```

The following example creates a merged collection from the two input collections. The input collections do not change.

```
set analogNets [db::getNets analog* -of [ed]]
set enable [db::getNets enable -of [ed]]
set mergedCollection [db::append $enable -to $analogNets]
```

db::appendLibertyEditorPinData

Description

Appends pin data to the pins table in the Liberty Editor dialog box. The passed pin data should have the format described in [db::getLibertyEditorPinsData](#). The pin name from passed data is checked to not be present in the already existing data of table.

Syntax

```
db::appendLibertyEditorPinData -dialog <giDialog> -pinData <string>
```

Arguments

-dialog

The Liberty Editor dialog box to add the pin data to table.

-pinData

Pin data as list of values: {<name> <direction> <capacitance> <min. capacitance> <max. capacitance> <min. transition> <max. transition> <related power> <related ground> <function>}.

Returns

<none>

db::attachTech

Description

Sets or clears references to other libraries using the “techLibName” property. To clear techLibName property, you should skip the `-ref` argument or provide an empty string as a reference. This command performs automatic backup and restore of technology as well.

Syntax

```
db::attachTech <lib: oaLib | string> [-refLib <oaLib> | -refLibName  
    <string>]
```

Arguments

lib

OA library, for which reference should be set.

-refLib

OA library, to which reference should be set.

-refLibName

Name of OA library, to which reference should be set.

Returns

<none>

Examples

The following example attaches "reference90RF" library to "PLL" library as a technology reference.

```
set newLib [dm::createLib PLL -path ./directoryToBeCreatedByCD] \  
db::attachTech $newLib -refLibName reference90RF
```

db::batch

Description

Evaluates a Tcl expression in the scope of one design transaction.

Syntax

```
db::batch <script: variant>
```

Arguments

`script`

Tcl expression to evaluate.

Returns

<none>

db::cat

Given a path relative to any startup directory, returns the content of the first file match from any startup directory.

Syntax

```
db::cat <path: [string]> [-all <bool>] [-noWarning <bool>]
```

Returns

<string>: File content

Arguments

`path`

Path relative to any startup directory or file system path.

`-all`

Output the content of all matching files. (Default: false).

`-noWarning`

Do not issue a warning when no file is found. (Default: false).

db::checkDRD

Description

Checks rules in the specified rule deck file. Optional arguments `-lib`, `-cell`, `-view`, and `-map` can also be specified in the rule deck file using the `vr::input` command. See Design Rule Verification Tcl in the *Custom Compiler Technology File and Display Resource File Reference Manual*.

You must specify `-lib` and `-cell` arguments either in the `db::checkDRD` command or in the `vr::input` command in the rule deck file.

Caution:

An error message is issued on the console, if `-lib` and `-cell` are not specified either in the `db::checkDRD` command or in the `vr::input` command.

If `-lib` and `-cell` arguments are specified both in the `vr::checkDRD` command and in the `vr::input` command, the one that is specified in the `vr::checkDRD` command is used for checking.

Note:

The `-view` and `-map` arguments must be specified together with the `-lib` and `-cell` arguments.

After the `db::checkDRD` command is executed, the tool generates a result file in an industry standard format which can be loaded into most types of layout editors. The result file is saved in the current working directory with the name `$cellName.results`, which is defined using the commands in the Tcl `vr` namespace.

Syntax

```
db::checkDRD -rule <string> [-lib <string>] [-cell <string>]  
[-view <string>] [-map <string>] [-objectMap <string>]
```

Arguments

`-rule`

Specifies the rule deck file. It can be specified with its file path.

`-lib`

A library name.

`-cell`

A cell name.

`-view`

A view type.

Default: layout.

`-map`

Layer mapping file, the same as the stream IO layer mapping file. It can be specified with its file path.

`-objectMap`

The object mapping file.

Returns

<none>

Examples

```
db::checkDRD -rule drd.rule -lib Alib -cell Bcell
```

db::checkFeature

Description

Determines whether a feature is defined on the license server. If the optional `-exists` argument is provided, its Tcl expression will be executed if the mentioned feature exists at the license server. If the optional `-doesNotExist` argument is provided, its Tcl expression will be executed if the mentioned feature does not exist at the license server.

Syntax

```
db::checkFeature <name: string>    [-exists <variant>]    [-doesNotExist  
    <variant>]
```

Arguments

name

Name of the feature.

-exists

Tcl expression to evaluate if the specified feature exists.

-doesNotExist

Tcl expression to evaluate if the specified feature does not exist.

Returns

<none>

Errors

The feature does not exist.

Examples

```
db::checkFeature CoDesign -exists {  
    set m [gi::createMenu xtICCMenu -title ICC]  
    gi::addActions {  
        xtRunICC  
        xtOpenICC  
    } -to $m  
    gi::addMenu $m -to [gi::getWindowTypes leLayout]  
}
```

db::checkForInterrupt

Description

Breaks the current loop if an interrupt (Ctrl +C) was detected while evaluating a script with `db::eval`.

Syntax

```
db::checkForInterrupt [<id: string>] -handler <dbInterruptHandler |  
dbProgress>
```

Arguments

`id`

Optional loop id for replaying the interrupt.

`-handler`

Interrupt handler object.

Returns

<none>

Examples

```
puts "Start"  
db::eval {  
    # Create Interrupt handler  
    set edifInterrupt [db::createInterruptHandler]  
    set i 0  
    while { $i < 10 } {  
        de::sendMessage "Iteration $i"  
  
        # Check the Ctrl+C binding which interrupts the script  
        db::checkForInterrupt -handler $edifInterrupt  
        incr i  
        exec sleep 1  
    }  
}  
puts "End"
```

db::checkLicenseTier

Description

Checks that the current tier is satisfied.

Syntax

```
db::checkLicenseTier -atLeast <enum>
```

Arguments

`-atLeast`

Required license tier. ({ advanced | advancedPlus | apex | backend | basic | elite
| frontend | mtm | router | schematic })

Returns

`<bool>` : If true, the current tier is sufficient. If false, the current tier is insufficient.

Examples

The following example returns true if the backend tier or one of the tiers above backend is already checked out

```
db::checkLicenseTier -atLeast backend
```

db::checkoutFeature

Description

Checks out an available licensing feature from the license pool. If the feature could not be checked out, the command throws an error. If the feature is already checked out, this command does nothing.

Note that the license name is an internally defined license name which may refer to one or more features on the actual license server. Use the `db::getFeatures` command to determine which features can be checked out.

Syntax

```
db::checkoutFeature <name: string> [-implicit <bool>]
```

Arguments

`name`

Name of the licensing feature to check out.

`-implicit`

If true, the implicit check-out count is incremented. (Default is: false.)

Returns

`<none>`

db::checkVersion

Description

Determines that the product version is at least the specified version.

Syntax

```
db::checkVersion (-is <string> | -exact <string> | -atLeast <string>)
```

Arguments

-is

Requested product version; allows for additional patches to the specified release.

-exact

Requested product version, must be an exact match.

-atLeast

Requested product version, must be at least the specified version.

Returns

<bool>: True if the version is sufficient, false otherwise.

Examples

```
puts [db::checkVersion -is O-2018.09]  
puts [db::checkVersion -atLeast N-2017.06]
```

db::cleanPcellRepository

Description

Rewrites all PCell repository map and data files for the specified CellView's repository. The command opens the design in that cellView (if needed) and then traverses all its PCell references to find all references that are still active. It copies subMaster images from the old repository files to a new copy that has no unused space. The subMaster images are not saved from memory so that this is not equivalent to a new save.

Syntax

```
db::cleanPcellRepository <object: dmCellView | dmCellViewName | oaDesign>  
[-saveBackup <bool>]  
[-listDetails <bool>] [-checkOnly <bool>]
```

Arguments

`object`

`dmCellView`, `oaDesign`, or `dmCellViewName` that contains a PCell repository to cleanup.

`-saveBackup`

If true, back up the old PCell repository. The default is false.

`-listDetails`

If true, information is shown for each repository `superMaster`. The default is false.

`-checkOnly`

If true, report information on the repository without changing anything. Nothing will be written or backed up. The default is false.

Returns

`<none>`

Errors

- A PCell repository does not exist in this `cellView`.
- UNIX protections prevented the operation.

Examples

```
# Clean pcell repository for the current edit design with saving backup
and printing details for each repository
db::cleanPcellRepository [ed] -saveBackup 1 -listDetails 1
```

db::closeCDF

Description

Closes component description format file for a cell or a library.

Syntax

```
db::closeCDF <data: dmCell | oaLib>
```

Arguments

`data`

Specifies the location for storing the CDF data as `oaDMData`.

Returns

<none>

Examples

The following example closes the CDF for a specified cell.

```
# Create a parameter for specified cell. This will also create a CDF for
a specified cell.
db::createParamDef param -data [dm::getCells bias -libName DesignLib]

# Check CDF
db::foreach data [oa::DMDDataGetOpenDMDDataObjs] {
if {"bias" == [db::getAttr data.cellName]} {
puts [db::getAttr data.type]
}
}

# Close the CDF for a specified cell
db::closeCDF [dm::getCells bias -libName DesignLib]

# Check CDF
db::foreach data [oa::DMDDataGetOpenDMDDataObjs] {
if {"bias" == [db::getAttr data.cellName]} {
puts [db::getAttr data.type]
}
}
```

db::closeDesign

Description

Closes the current design.

Syntax

```
db::closeDesign <design: oaDesign> [-force <bool>]
```

Arguments

design

OA Design object

-force

Forces the design to close. The default is false.

Returns

<none>

Examples

The following example closes the specified design.

```
# Open the DemoPLL/and2/layout design
set context [de::open DemoPLL/and2/layout]

# Get the edit design from context
set design [db::getAttr context.editDesign]

# Close the specified design
db::closeDesign $design
```

db::copyLibertyData

Description

Copies Liberty Data of a given design with a given corner name, to another corner name of the same design or to another design. The optional arguments `-to` and `-toCorner`, both cannot be omitted.

Syntax

```
db::copyLibertyData <name: dmCellView | dmCellViewName | oaDesign>
    [-corner <string>] [-to <oaDesign>] [-toCorner <string>] [-force <bool>]
```

Arguments

`name`

Cellview or design which contains Liberty Data to be copied.

`-corner`

Name of Liberty Data corner to be copied. (Default is: nominal)

`-to`

Design to which the Liberty Data is copied.

`-toCorner`

Name of target Liberty Data corner.

`-force`

Overwrite existing data. (Default is: false)

Returns

<none>

Errors

- The corner with the specified name does not exist in source design.
- The target corner exists and `-force` is false.
- The target design and target corner name are omitted.

Examples

```
# Export liberty data of the current edit design from "nominal" corner to  
"newCorner" corner  
db::copyLibertyData [ed] -corner nominal -toCorner newCorner
```

db::copyParams

Description

Merges parameters from the source property to the destination oaLib/oaCell's DM data. It first opens the DMData of the passed in container for "append". Next it merges the contents of the source property to the property named "cdfData" in the destination DMData.

Parameter definitions are copied from the source to the destination. If both the source and destination have a named parameter definition, the source's definition overwrites the destination's definition.

The Top Level CDF information, such as "fieldWidth," is copied into the destination replacing same named fields with the values from the source.

The propList section is then copied into the destination replacing same named fields with the values from the source. For example, if the source contains a modelLabelSet and the destination contains a modelLabelSet, then the final value of the destination modelLabelSet is identical to the value in the source data.

The Netlist Info section is also merged into existing data by creating new netlister sections if necessary and overwriting conflicting sections.

Finally the DMData is saved and closed if it was not already opened before the function was called.

This function is registered via `dm::registerCopyProps` in the Custom Compiler startup scripts and is typically not called directly by the user.

Syntax

```
db::copyParams <parameters: oaProp | {<dbParam>}> -to <dmCell | oaLib |  
oaStdViaDef | oaTech>
```

Arguments

parameters

The source property to copy/merge to the destination.

-destination

The destination container.

Returns

<none>

Examples

```
# Add two new parameters to the testLib/nmos cell
set nmos [dm::findCell nmos -libName testLib]
db::createParamDef param1 -data $nmos
db::createParamDef param2 -data $nmos

# Copy these parameters to the testLib/nmos4 cell
set nmos4 [dm::findCell nmos4 -libName testLib]
set params [db::getParams -of $nmos]
db::copyParams $params -to $nmos4
```

db::copyPrefs

Description

Copies a collection of preferences to a new environment or copies the collection to the active environment. This lets you create a sandbox within which any preference value can be modified and then restored on completion.

If the -env argument is omitted, the collection of preferences is copied to the system environment. If specified, the preferences are copied into the specified environment. If it does not already exist, it is created. The preference copy is a merge operation. If a preference with the same name and scope exists in the destination environment, it is overwritten, otherwise it is created.

You can destroy the environment with the `db::purgePrefs` command.

Syntax

```
db::copyPrefs <dbpref: dbCollection>
```

Arguments

{dbPref}

Collection of preferences to copy.

Returns

{<dbPref>}: Collection of newly copied preferences in the target environment.

Errors

Invalid preference object.

db::createAssignAssignment

Description

Creates an assignAssignment on an instance or a design.

Syntax

```
db::createAssignAssignment <name: string> (-inst <oaInst> | -design  
<oaDesign>) -newName <string> -defaultNetName <string>
```

Arguments

name

The name of a connection definition to assign.

-newName <string>

New name for connection definition.

-inst

Instance on which to create the assign assignment.

-design

Design on which to create the assign assignment.

-defaultNetName <string>

Name of the default net.

Returns

Object describing assignment on the instance or design. ([dbAssignAssignment](#))

Errors

- Invalid argument.
- Design not opened for write.

Examples

```
# Create new assignment on net "vdd" with name "power"
db::createAssignAssignment power -inst [db::getInsts I0 -of [ed]]
    -newName VDD -defaultNetName vdd
```

db::createAssignValue

Description

Creates an assignValue on an instance or a design.

Syntax

```
db::createAssignValue <name: string> (-inst <oaInst> | -design
    <oaDesign>) -netName <string>
```

Arguments

-name

Name of connect definition to assign.

-inst

Instance on which to create the assign value.

-design

Design on which to create the assign value.

-netName <string>

Name of net which exists in the given design, or the design containing given instance.

Returns

Object describing value assignment on the instance or design.

Errors

- Invalid argument.
- Design not opened for write.

Examples

```
# Create new assignment on I0 instance
db::createAssignValue power -netName vdd -inst [db::getInsts I0 -of [ed]]
```

db::createCallback

Description

Registers a user callback for any registered callbackType.

If a callback with the same name already exists, the new callback overwrites the existing callback.

The callback can be unregistered by calling `db::destroy` on the callback object.

Syntax

```
db::createCallback <callbackName: string> -callbackType <string>  
-procedure <dbExecObject | string> [-priority <int>] [-viewType  
<string>] [-clientData <string>] [-enableInUndo <bool>]
```

Arguments

`callbackName <string>`

Callback name.

Default: none.

`-callbackType <string>`

Event that triggers this callback.

Default: none.

`-procedure <dbExecObject | string>`

Tcl procedure to call when executing the callback.

Default: none.

`-priority <int>`

Priority of execution relative to other callbacks for the event. The highest priority has the lowest value. If there are callbacks with the same priority, then they are evaluated in the order that they were registered; first registered is first triggered.

Default: 1000.

`-viewType <string>`

View type name, if applicable, for which to trigger the callback. For callbackTypes with specific viewTypes, the default value, the empty string (""), indicates the callback will be triggered for all viewTypes.

Default: "".

`-clientData [<string>]`

Any additional parameters to pass to the callback.

Default: [].

`-enableInUndo <bool>`

Enable this callback during undo operations for database events

The default is false.

Returns

New callback object ([dbCallback](#)).

Errors

- Undefined callbackType.
- Tcl procedure does not have the appropriate number of arguments as required by the supplied callbackType.

Examples

```
proc designInfo {design} {  
    puts [oa::getCellName $design]  
}  
db::createCallback open -callbackType onFirstDesignOpened \  
    -procedure designInfo -priority 1
```

In this example, the callback object “open” calls the procedure designInfo, which prints the name of the opened cell each time the design opens. The dbCallback object is a user-defined callback for a system event exported to Tcl.

db::createCircularRuler

Description

Creates a circular ruler in the specified layout, symbol, or the schematic design.

Syntax

```
db::createCircularRuler <point: point> (-rulerDef <dbCircularRulerDef |  
    string> | -subRulers [<dbSubCircularRuler>]) -design <oaDesign> [-angle  
    <float>] [-persistent <bool>]
```

Arguments

point

The center point for all circles.

`-design`

Design in which to create the ruler.

`-rulerDef`

A ruler definition or definition name to define the circular ruler instance.

`-subRulers`

List of sub-rulers which define the circular ruler instance.

`-angle`

Radial measure scribe direction [0...360]. (Default is: 0)

Default: 0.

`-persistent`

Keeps a linear ruler in the design. (Default is: true)

Returns

<dbCircularRuler>: Circular ruler created.

Examples

The following example creates a circular ruler object.

```
# Create a sub-ruler object
set subCR [db::createSubCircularRuler -radius 2.4 -label 2.4 -packet ruler1]

# Create a circular ruler definition
set CRDef [db::createCircularRulerDef CRDefName -subRulers $subCR]

# Create a circular ruler object
set point [oa::Point 0 0]
db::createCircularRuler $point -design $design -rulerDef $CRDef
```

db::createCircularRulerDef

Description

Defines a circular ruler definition.

Syntax

```
db::createCircularRulerDef <name: string> -subRulers
    [<dbSubCircularRuler>] [-angle <float>]
```

Arguments

name

Ruler definition name.

`-subrulers`

List of sub-rulers associated with this definition object.

`-angle`

Radial measure scribe direction [0...360].

Default: 0.

Returns

<dbCircularRulerDef>: Ruler definition created.

Examples

The following example creates a circular ruler definition.

```
# Create a sub-ruler object
set subCR [db::createSubCircularRuler -radius 2.4 -label 2.4 -packet ruler1]

# Create a circular ruler definition
db::createCircularRulerDef CRDefName -subRulers $subCR
```

db::createCollection

Description

Converts a Tcl list to a dbCollection.

Syntax

```
db::createCollection [<object: [attrObject]>]
```

Arguments

`object`

A list of objects that will be converted.

Returns

<dbCollection | dbIterator> : A newly created dbCollection from list.

Examples

The following example creates a collection of instances in the design, and then selects the instances.

```
db::foreach inst [db::getInsts -of [ed]] {
  set instArr($inst) 1
}
de::select [db::createCollection [array names instArr]]
```

db::createDesign

Description

Creates a design.

Syntax

```
db::createDesign <cellView: dmCellView | dmCellViewName> -viewType <enum>  
[-overwrite <bool>]
```

Arguments

cellView

Cellview for which to create design

-viewType

View type of design. ({ layout | schematic | symbol })

-overwrite

Overwrites the existing design. The default is false.

Returns

<oaDesign> : Newly created design.

Examples

The following example creates a new design DemoPLL/and2/layout2 with view type layout.

```
db::createDesign DemoPLL/and2/layout2 -viewType layout
```

The following example recreates existing design DemoPLL/and2/layout with view type layout by overwriting it.

```
db::createDesign DemoPLL/and2/layout -viewType layout -overwrite true
```

db::createDeviceMappingFileDialog

Description

Creates the **Device Mapping File Template Generation** dialog box.

Syntax

```
db::createDeviceMappingFileDialog [-shown <bool>]
```

Arguments

`-shown`

Initial shown state for dialog. The default is true.

Returns

`<giDialog>` : Device Mapping Template Generation Dialog.

db::createEnum

Description

Create and return a new dbEnum object.

Syntax

```
db::createEnum <name: string> (-values [<string>] | -valueDescPairs  
    [<string>] | -valueProc [<variant>] | -valueDescProc [<variant>])  
    [-caseSensitive <bool>] [-exclusive <bool>] [-validator [<variant>]]
```

Arguments

`name`

Enum Name

`-values`

List of valid values.

`-valueDescPairs`

List of {value description} pairs.

`-valueProc`

Procedure which returns a list of valid values.

`-valueDescProc`

Procedure which returns a list of {value description} pairs.

`-caseSensitive`

Case matching behavior against valid values. (Default is: false)

`-exclusive`

If false, permit values other than valid values. (Default is: true)

`-validator`

Validator procedure for use with `-exclusive` false and an unrecognized value.

Returns

<dbEnum> : New dbEnum object

Examples

```
db::createEnum enTermType \  
    -values {input output inputOutput switch jumper unused tristate}  
db::createPref crTerm -type enTermType -value tristate  
db::setPrefValue crTerm -value j  
# type j and press the Tab key to auto complete the value to "jumper"
```

db::createEvaluator

Description

Creates an evaluator.

Syntax

```
db::createEvaluator <name: string> -annotationSet <string> -type <enum>  
    [-evaluatorProc <string>] [-paramListProc <string>] [-paramLabelSetProc  
    <string>]
```

Arguments

name

Name of the evaluator object, which will be shown in Device Label Editor.

-annotationSet

Annotation set name to which the evaluator should belong.

-type

Display type: {name | param | term}.

-evaluatorProc

Name of evaluator procedure.

-paramListProc

Name of Tcl procedure for instance parameters being retrieved.

-paramLabelSetProc

Name of Tcl procedure used for default parameter label set retrieval.

Returns

<dbLabelEvaluator>: Evaluator object used for cds term evaluation.

db::createExpressionMap

Description

Creates an expression map.

Syntax

```
db::createExpressionMap
```

Returns

<dbExpressionMap >: The expression map object.

db::createFigGroup

Description

Creates a new figure group (figGroup) in the given design with the given attributes.

Syntax

```
db::createFigGroup <name: string> -design <oaDesign> [-origin <point>]  
[-orient <enum>] [-members <{<deFigure>} | {<deSelectionSet>} |  
{<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |  
{<oaVia>}}>]
```

Arguments

name

The name of the figGroup to be created.

-design

Design in which to create the figure group.

-origin

Location of the origin of figure group.

-orient

Orientation of the figure group. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270
| R90 }, default is: R0)

-members

Creates figure group members for the specified figures to this figGroup.

Returns

<oaFigGroup> : Newly created figure group.

Examples

The following example creates a hierarchical object.

```
set context [de::getActiveContext]
set ho [de::createHierObject [db::getInsts I0 -of [db::getAttr
context.editDesign]] -hierPath $context]
db::listAttrs -of [db::getAttr ho.hierPath]
=> depth levels this transform type
```

db::createGenOptionSet

Description

Creates a new generator option set object using the given name and type. If a set with the given name and type already exists, an error is generated. Any modifications to the returned option set object update the internal session wide option set. In other words, the option set object is the interface to the internal definition.

The option set exists throughout the session. To remove the named option set from the session, use the `db::destroy` command and pass it a `dbGenOptionSet` object. Note that Tcl variables holding references to destroyed option sets become invalid.

Syntax

```
db::createGenOptionSet <name: string> -type <string> [-value <string>]
[-loadDefault <bool>] [-src <oaDesign>] [-fromTech <bool>]
```

Arguments

`name`

Name of the option set.

`-type <string>`

Generator option set type name.

`-value <string>`

Initial value for the generator options.

`-loadDefault`

Loads default values for abstractGenerator. (Default is: false)

`-src`

The source design.

`-fromTech`

Get constraints from technology database, otherwise get them from the design.
(Default is: false)

Returns

New dbGenOptionSet object ([dbGenOptionSet](#)).

Errors

Set with same name already exists.

Examples

The following example creates an Abstract Generator option set named *m1Set* with the specified values for M1 layer.

```
db::createGenOptionSet m1Set
-type abstractGenerator \
-value "M1 Use true
      M1 Cover true
      M1 Pin true
      M1 Blockage true
      M1 pinCutout false
      M1 effectiveWidth effectiveWidth
      M1 Value 0.15
      M1 ignoreAccess false
      M1 Oversize 0.022"]
```

The following example finds the default option set for Symbol Generator.

```
# Find default option set for Symbol Generator
set os [db::getGenOptionSets default -filter
  {%setType=="symbolGenerator"}}
# Get values from default option set
array set val [db::getattr os.value]
set os [db::getNext [db::getGenOptionSets custom -filter
  {%setType=="symbolGenerator"}}]
if {""==$os} {
  # Create new options set
  set os [db::createGenOptionSet custom -type symbolGenerator]
}
# Set values
set val(pinSpacing) 4
set val(aspectRatio) 0.5
set val(pinToEdgeSpacing) 2
set val(pinStubLength) 10
db::setattr os.value -value [array get val]
# Generate symbol view from schematic design
db::generateCellView -cellView testLib/vco/schematic -dest
  testLib/vco/symbol -update false -viewType schematicSymbol -options $os
```

The following example creates an option set for antenna mode.

```
set optSetValue {
  AntennaMode true
  PVEToolName ICValidator
}
```

```
}
set optSet [db::createGenOptionSet mySet -src [ed] -loadDefault true
-value $optSetValue]

le::generateAbstract -src [ed] -options $optSet -force 1
```

db::createGroup

Description

Creates a new group in the given design with the given attributes.

Syntax

```
db::createGroup [-name <string>] -design <oaDesign> [-isUniqueName
<bool>] [-deleteWhen <enum>] [-members <{{<deFigure>}} |
{{<deSelectionSet>}} | {{<oaFig>}} | {{<oaFigGroup>}} | {{<oaGroup>}} |
{{<oaInst>}} | {{<oaInstHeader>}} | {{<oaInstTerm>}} | {{<oaLPPHeader>}} |
{{<oaMarker>}} | {{<oaNet>}} | {{<oaPin>}} | {{<oaProp>}} | {{<oaShape>}} |
{{<oaTerm>}} | {{<oaVia>}} | {{<oaViaHeader>}}>}] [-leader <oaFig | oaFigGroup
| oaGroup | oaInst | oaInstHeader | oaInstTerm | oaLPPHeader | oaMarker
| oaNet | oaPin | oaProp | oaShape | oaTerm | oaVia | oaViaHeader>]
```

Arguments

-name

The name of the group to be created. (Default is:
__CDBA_PARENTCHILD_ONLY_GROUP)

-design

Design in which to create the group.

-isUniqueName

Specifies whether the group name must be unique. (Default is: false)

-deleteWhen

Sets the event type that determines when, if ever, this group will be deleted.
({ never | onFirst | onLast }, default is: onLast)

-members

Creates figure group members for the specified figures to this figGroup.

-leader

Specifies the leader for the group. If multiple objects are passed, the first one is
set as the leader.

Returns

<oaGroup> : Newly created oaGroup.

Examples

The following example creates a group from rectangle and text objects as the leader and follower respectively.

```
set design [db::getattr editDesign -of [de::getActiveContext]]
set transaction [de::startTransaction "Create Group" -design $design]
set rect [le::createRectangle {{0 0} {0.8 0.8}} -lpp m1 -design $design]
set label [le::createLabel VDD -lpp text -parent $design -origin {0.4 0.4}]
db::createGroup -design $design -leader $rect -members $label
de::endTransaction $transaction
```

db::createHistogram

Description

Creates a new dbHistogram widget and adds it to the specified parent.

Syntax

```
db::createHistogram [<{none}>: string] [-parent <giAssistant |
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]
[-xAxisLabel <string>] [-yAxisLabel <string>] [-bins <dbCollection>]
[-precision <unsigned int>]
```

Arguments

{none}

Widget name.

-parent

Dialog, group or other container into which this widget should be placed.

-enabled

Initial enabled state. (Default is: true)

-shown

Initial shown state. (Default is: true)

-xAxisLabel

X-axis label.

-yAxisLabel

Y-axis label.

`-bins`

Collection of bin objects.

`-precision`

The display precision. The default is 5.

Returns

`<dbHistogram>` : New object.

db::createImage

Description

Creates an image object.

Syntax

```
db::createImage <filePath: string> -design <oaDesign> [-display <enum>]  
[-attachTo <oaFig>] -origin <point> [-just <enum>] -size <float>
```

Arguments

`filePath`

Image file path.

`-design`

Design in which to create the image.

`-display`

Image display mode ({ collapsed | expanded }, default is: expanded)

`-attachTo`

Parent figure to attach the image to.

`-origin`

Location of the origin of image.

`-just`

Justification of the image with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }, default is: lowerCenter)

`-size`

Image size.

Returns

<oaRect> : The new image added to the given design.

Examples

```
db::createImage ./image.png -design [ed] -origin {0 0} -size 5
```

db::createIncludeFileControl

Description

Creates a new include file control widget.

Syntax

```
db::createIncludeFileControl [<{none}>: string] [-parent <giAssistant |  
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]  
[-prefName <dbPrefName>] [-toolTip <string>] [-exceptionsEnabled <bool>]  
[-includeModeEnabled <bool>]
```

Arguments

{none}

Widget name.

-parent

Dialog, group or other container into which this widget should be placed.

-enabled

Initial enabled state. The default is true.

-shown

Initial shown state. The default is true.

-prefName

Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model.

-toolTip

Optional widget toolTip.

-exceptionsEnabled

Flag indicating whether or not exceptions for specific files should be supported. The default is false.

`-includeModeEnabled`

Flag indicating whether or not the include mode should be supported. The default is true.

Returns

`<dbIncludeFileControl>` : New object.

Examples

```
# Create dialog and add include file control widget to it
set dialog [gi::createDialog newDialog -title "Include File"]
db::createIncludeFileControl -parent $dialog -exceptionsEnabled true
```

db::createInstCache

Description

Creates cache for parameters.

Syntax

`db::createInstCache`

Returns

`<dbInstCache>` : The newly created `dbInstCache` object.

Examples

```
set cache [db::createInstCache]
set inst [de::getSelected -design [ed]]
set instance [db::getAttr object -of $inst]
db::instCacheTriggerParamCallbacks $cache -of $instance -triggerParamList
{contRows finger}
```

db::createInstTerm

Description

Creates an instance terminal.

Syntax

```
db::createInstTerm <terminal: oaTerm | string> (-inst <oaInst> |
  -instName <string>) -net <oaNet>
```

Arguments

`terminal`

Name of terminal or oaTerm inside instance master.

`-net`

Name of oaNet.

`-inst`

Name of oaInst.

`-instName`

instName.

Returns

<oaInstTerm> : Newly created instTerm.

Examples

The following example creates instance terminal object.

```
# Get edit design from active context
set context [de::getActiveContext]
set design [db::getattr context.editDesign]

# Get single instance in the layout
set instance [db::getNext [db::getInsts -of $design]]

# Get net object of existing clk net in the top design
set net [db::getNets clk -of $design]

# Create instance terminal for exiting net in top design
db::createInstTerm clk -inst $inst -net $net
```

db::createInterfaceParam

Description

Adds a dbInterfaceParam object. The object can be explicitly destroyed by calling `db::destroy`. A collection of interface param objects can be used by the `db::createInterfaceSet` command.

For the time type property, an integer value is required to be given for `defValue` and also for `minValue` and `maxValue` for `timeRange` types.

The `-enums` argument should only be given if the given type is enum.

Syntax

```
db::createInterfaceParam <name: string> -type <enum> [-defValue <string>]  
[-isCDF <bool>] [-minValue <string>] [-maxValue <string>] [-enums  
[<string>]]
```

Arguments

<string>

Name of the parameter.

-type

Type of value for this parameter: {bool | double | doubleRange | enum | float | floatRange | int | intRange | string | time | timeRange}.

-defValue

Optional default value. If not given, then the parameter default value is considered 'unset'.

-isCDF

Indicates whether this parameter should be created as a CDF parameter or a design property.

The default is false.

-minValue

Minimum value for the range type property.

-maxValue

Maximum value for the range type property.

-enums

List of strings for the property of type enum.

Returns

New interface parameter object ([dbInterfaceParam](#)).

Errors

- maxValue is given without supplying minValue.
- minValue is given without supplying maxValue.
- minValue and/or maxValue not given when a range type is given.
- Incorrect minValue, maxValue, or defValue value given for the supplied type.
- -enums is given, but the type is not enum.

Examples

```

set inputPinsCount 10
set outputPinsCount 10
set interfacePinList [db::createCollection]
for {set k 1} {$k <= $inputPinsCount} {incr k} {
db::add [db::createInterfacePin "in${k}" -type input -side left] \
-to $interfacePinList
}
for {set k 1} {$k <= $outputPinsCount} {incr k} {
db::add [db::createInterfacePin "out${k}" -type input -side right] \
-to $interfacePinList
}
set param [db::createInterfaceParam length -type string -isCDF true
-defValue "2u"]
set interface [db::createInterfaceSet -pins $interfacePinList -parameters
$param]
se::generateSymbol -interface $interface -update 0 -dest PLL/cell1/symbol
\
-options [db::getGenOptionSets default -filter
{%setType=="symbolGenerator"}]

```

db::createInterfacePin

Description

Creates an interface pin object. The object can be explicitly destroyed by calling `db::destroy`. A collection of interface pins can be used by the `db::createInterfaceSet` command.

Syntax

```

db::createInterfacePin <string: string> -type <enum> [-side <enum>]
[-position <int>] [-assignmentName <string>] [-defaultConnection
<string>]

```

Arguments

<string>

Name of the pin.

-type

Type of the pin: {input | inputOutput | output | switch | tristate}.

-side

Side of the core rectangle on which to place the pin: {bottom | left | right | top | unknown}.

Default: unknown.

-position

Relative order of the pin on the specified side.

-assignmentName

Assignment name for connect definition to be added to pin.

-defaultConnection

Name of default net for connection definition.

Returns

New interface pin object ([dbInterfacePin](#)).

Errors

- Invalid enum is given.
- assignmentName or defaultConnection argument is given without the other.

Examples

```
set inputPinsCount 10
set outputPinsCount 10
set interfacePinList [db::createCollection]
for {set k 1} {$k <= $inputPinsCount} {incr k} {
    db::add [db::createInterfacePin "in${k}" -type input -side left] \
        -to $interfacePinList
}
for {set k 1} {$k <= $outputPinsCount} {incr k} {
    db::add [db::createInterfacePin "out${k}" -type input -side right] \
        -to $interfacePinList
}
set interface [db::createInterfaceSet -pins $interfacePinList]
se::generateSymbol -interface $interface -update 0 -dest PLL/cell11/symbol \
    -options [db::getGenOptionSets default -filter {%setType=="symbolGenerator"}]
```

db::createInterfaceSet

Description

Creates an interface set object. If multiple pins have the same side and position specifications, the behavior is dependent of the final generator used to create a template of this interface set.

Syntax

```
db::createInterfaceSet [-pins <dbCollection>] [-parameters
    <dbCollection>] [-cellView <dmCellView | dmCellViewName>] [-filePath
    <string>]
```

Arguments

`-pins`

Collection of interface pins.

`-parameters`

Collection of interface parameters.

`-cellView`

Source cellView.

`-filePath`

Source file.

Returns

New interface set object ([dbInterfaceSet](#))

Errors

- If both the `-cellView` and `-filePath` arguments are specified.

Examples

```
set inputPinsCount 10
set outputPinsCount 10
set interfacePinList [db::createCollection]
for {set k 1} {$k <= $inputPinsCount} {incr k} {
    db::add [db::createInterfacePin "in${k}" -type input -side left] \
        -to $interfacePinList
}
for {set k 1} {$k <= $outputPinsCount} {incr k} {
    db::add [db::createInterfacePin "out${k}" -type input -side right] \
        -to $interfacePinList
}
set interface [db::createInterfaceSet -pins $interfacePinList]
se::generateSymbol -interface $interface -update 0 -dest PLL/cell1/symbol \
    -options [db::getGenOptionSets default -filter {%setType=="symbolGenerator"}]
```

db::createInterruptHandler

Description

Creates a new interrupt handler which processes Ctrl+C events during the evaluation of a script with `db::eval`.

Syntax

```
db::createInterruptHandler [<name: string>][-safe <bool>]
```

Arguments

name

Optional identifier for replaying the interrupt.

Default: "".

-safe

Safe mode will not process events when checking for interrupts. (Default is: true)

Returns

<dbInterruptHandler>: Interrupt handler object.

Examples

```
puts "Start"
db::eval {
    # Create Interrupt handler
    set edifInterrupt [db::createInterruptHandler]
    set i 0
    while { $i < 10 } {
        de::sendMessage "Iteration $i"

        # Check the Ctrl+C binding which interrupts the script
        db::checkForInterrupt -handler $edifInterrupt
        incr i
        exec sleep 1
    }
}
puts "End"
```

db::createLibListInput

Description

Creates a library list input widget

Syntax

```
db::createLibListInput [<{none}>: string] [-label <string>] [-parent  
    <giAssistant | giContainer | giDialog | giWindow>] [-enabled <bool>]  
    [-shown <bool>] [-prefName <dbPrefName>] [-toolTip <string>]
```

Arguments

{none}

Widget name.

-label

Widget label, shown in the left column.

`-parent`

Dialog, group or other container into which this widget should be placed.

`-enabled`

Initial enabled state. The default is true.

`-shown`

Initial shown state. The default is true.

`-prefName`

Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model.

`-toolTip`

Optional widget tooltip.

Returns

`<dbLibraryListInput>` : New object.

Examples

Example 1

The following example creates dialog box and adds LibListInputs widget to it.

```
set dialog [gi::createDialog newDialog -title "Library Input"]
db::createLibListInput -parent $dialog
```

Example 2

The following example creates LibListInputs widget, which contains an Open button to open the modal dialog allowing to select the desired libraries.

```
set dialog [gi::createDialog libs -title Libraries]
db::createLibListInput libsInput -parent $dialog
```

db::createLinearRuler

Description

Creates a linear ruler in the specified layout, symbol, or schematic design.

Syntax

```
db::createLinearRuler <points: [point]> -design <oaDesign> [-vectorOnly  
  <bool>] [-persistent <bool>]
```


Arguments

`points`

List of points for positioning the linear ruler.

`-design`

Design in which to create the linear ruler.

`-vectorOnly`

Draws all measure values or only end measure value. (Default is: false)

`-persistent`

Keeps a linear ruler in the design. (Default is: true)

Returns

`<dbLinearRuler>`: Linear ruler created.

Examples

```
set design [ed]
db::createLinearRuler {{1 1} {2 1} {3 4}} -design $design
```

db::createList

Description

Converts a collection to a Tcl list.

Syntax

```
db::createList <collection: dbCollection | dbIterator>
```

Arguments

`collection`

An iterator over the dbCollection.

Returns

Tcl list of objects created. (oaObjectdbObject).

Errors

The collection contains one or more objects unsupported as Tcl object types.

Examples

The following example returns net names of all nets in the edited design.

```
db::createList [db::getAttr name -of [db::getNets -of [ed]]]
```

db::createMarker

Description

Creates a new marker in the specified design with the given attributes. There is no check if a marker already exists with the specified attributes.

Syntax

```
db::createMarker <points: [point]> -design <oaDesign> [-message  
  <string>] [-shortMessage <string>] [-tool <string>] [-severity <enum>]  
  [-deleteWhen <enum>] [-closed <bool>]
```

Arguments

`points`

The list of points that form marker.

`-design`

Design in which to create the marker.

`-message`

Sets the message string associated with this marker.

`-shortMessage`

Sets the short message string associated with this marker.

`-tool`

Sets the tool string associated with this marker.

`-severity`

Sets the `severity` attribute of this marker. ({ acknowledgedWarning | annotation | criticalError | error | fatalError | info | signedOffCriticalError | signedOffError | warning }, default is: error)

`-deleteWhen`

Sets the `deleteWhen` attribute of this marker to the specified `deleteWhen` value. ({ deleteFirst | deleteLast | deleteModified | deleteNever }, default is: deleteFirst)

`-closed`

Used to indicate whether the pointArray associated with this marker is closed. (Default is: true)

Returns

<oaMarker> : Newly created marker.

Examples

The following example creates marker object with provided points.

```
set design [db::getattr editDesign -of [de::getActiveContext]]
set transaction [de::startTransaction "Create Marker" -design $design]
db::createMarker {{5.1 10.3} {5.9 10.3} {5.9 10.5} {5.1 10.5}} \
-deleteWhen deleteLast -design $design \
-severity warning -message "Different color overlap" \
-shortMessage Colorshort -tool "Color Checker" -closed 1
de::endTransaction $transaction
```

db::createNet

Description

Creates a net.

Syntax

```
db::createNet <name: string> -design <oaDesign> [-global <bool>]
[-sigType <enum>]
```

Arguments

name

Net name.

-design

Design in which to create the net.

-global

Global. The default is false.

-sigType

Net signal type. ({ analog | clock | ground | power | reset | scan | signal | tieHi | tieLo | tieoff }). The default is signal.)

Returns

<oaNet> : Newly created oaNet

Examples

The following example creates net object in specified design.

```
# Get edit design from active context
set context [de::getActiveContext]
set design [db::getAttr context.editDesign]

# Create rst global net in design
db::createNet rst -design $design -global true -sigType reset
```

db::createNetlistInst

Description

Creates a dbNetlistInst object.

Syntax

```
db::createNetlistInst <name: string> -deviceType <enum> -connections
    [<map>][[-otherDeviceName <string>]]
```

Arguments

name

The instance name.

-deviceType

The type of device, { cap | diode | ind | nmos | npn | other | pmos | pnp | res })

-connections

List of net/port pairs.

-otherDeviceName

Name of other device.

Returns

<dbNetlistInst> : Netlist instance object.

db::createNotifier

Description

Adds a notifier to an object created with db::createObject.

Syntax

```
db::createNotifier <name: string> -to <dbObject> [-args [<string>]]  
-description <string>
```

Arguments

name

Notifier name.

-to

Object to which the new notifier is attached.

-args

Tcl list of argument types.

-description

Description of this notifier.

Returns

<dbNotifier> : Newly created notifier.

Errors

- Invalid type %s specified in -args.

Examples

```
# Create an object  
set obj [db::createObject ]  
  
# Add attributes to the object  
db::addAttr name -of $obj -valueType string -value "nmos"  
db::addAttr l -of $obj -valueType double -value 1u  
db::addAttr w -of $obj -valueType double -value 0.1u  
  
# Create notifiers for each attribute. Each notifier notifies the object to  
# set its attribyte value  
set n1 [db::createNotifier nmosAttrSet_l -to $obj -args string \  
-description "NMOS L parameter set notifier"]  
set n2 [db::createNotifier nmosAttrSet_w -to $obj -args string \  
-description "NMOS W parameter set notifier"]  
  
# Set listeners to the notifiers  
db::addListener [list nmosValueSet_l $obj ] -to $n1  
db::addListener [list nmosValueSet_w $obj ] -to $n2  
  
# Create procedures which handles object attribute  
proc nmosValueSet_l {obj value} {  
  puts "This is nmos l attribute set listener proc, the value is $value"  
  if {$value >= 0} {  
    db::setAttr obj.l -value $value  
  }  
}
```

```
# Create procedures which handles object attribute
proc nmosValueSet_w {obj value} {
    puts "This is nmos w attribute set listener proc, the value $value"
    if {$value >= 0} {
        db::setAttr obj.w -value $value
    }
}

# Get the object notifier
set n [db::getNotifiers nmosAttrSet_l -of $obj]
# Notify the object to set the l attribute
db::notify $n -args 5u
# Check if the attribute is set
puts [db::getAttr obj.l]

# Get the object notifier
set n [db::getNotifiers nmosAttrSet_w -of $obj]
# Notify the object to set the w attribute
db::notify $n -args 0.5u
# Check if the attribute is set
puts [db::getAttr obj.w]
```

db::createObject

Description

Creates an empty dbObject and returns it. This command takes no arguments.

Syntax

```
db::createObject
```

Returns

<dbObject>: New empty dbObject.

Examples

```
# Create two object of dbObject type
set netObj [db::createObject ]
set termObj [db::createObject ]

# Add attribites to the termObj object
db::addAttr name -of $termObj -valueType string -value "clk"
db::addAttr type -of $termObj -valueType string -value "INPUT"
db::addAttr isScalar -of $termObj -valueType boolean -value true
db::addAttr Imax -of $termObj -valueType double -value 0.1u
db::addAttr Cf -of $termObj -valueType double -value 100f

# Add attribites to the netObj object
db::addAttr name -of $netObj -valueType string -value "CLK"
db::addAttr nettype -of $netObj -valueType string -value "signal"
db::addAttr isGlobal -of $netObj -valueType boolean -value "false"
db::addAttr Term -of $netObj -value $termObj

# Print attributes of the netObj object
puts [db::getAttr netObj.name]
```

```
puts [db::getattr netObj.Term.Imax]  
puts [db::getattr netObj.Term.Cf]  
puts [db::getattr netObj.Term.isScalar]
```

db::createPaperSize

Description

Creates the PaperSize Object.

Syntax

```
db::createPaperSize <name: string> -width <string> -height <string>  
[-offsetX <string>] [-offsetY <string>] [-spoolProcOptions <string>]
```

Arguments

name

The name of the paper size to create.

-width

The width of the paper.

-height

The height of the paper.

-offsetX

The X offset of the origin.

Default: 0.

-offsetY

The Y offset of the origin.

Default: 0.

-spoolProcOptions

Options to be passed to the spool procedure when this paper size is used
(for example: if using lpr as a spool procedure, then for Letter paper size this
attribute can be "-o media=Letter").

Returns

<dbPaperSize>: The size of the paper that is created.

Examples

```
db::createPaperSize Custom1 -width 10 -height 20
```

db::createParamDef

Description

Creates a component description format (CDF) parameter at either the library or the cell level (depending on the passed oaDMDData object).

Typical values for “units” are: lengthMetric, capacitance, conductance, inductance, resistance, voltage, current, power, time, and frequency.

If the type is either “radio” or “cyclic” and a defValue is not provided, then the default value will be set to the first item in the -choices and a warning will be printed to the log file.

Syntax

```
db::createParamDef <name: string> -level <enum> -data <dmCell |
oaCellDMDData | oaLib | oaLibDMDData | oaStdViaDef | oaTech> [-prompt
<string>] [-defValue <string>] [-type <enum>] [-callback <string>]
[-units <string>] [-choices <string>] [-parseAsNumber <bool>]
[-parseAsPEL <bool>] [-editable <string>] [-display <string>] [-use
<string>] [-storeDefault <bool>] [-propList <string>] [-mode <string>]
[-filePrompt <string>] [-relativePath <bool>] [-fileType <string>]
[-fileMasks <string>] [-includeQuotes <bool>] [-includeParenthesis
<bool>]
```

Arguments

name

The name of the dbParamDef object.

-data

The storage location of dbParamDef.

-prompt

Prompt in the Property Editor. (Default is:)

-defValue

Default value of the parameter. (Default is:)

-type

Parameter Value Type. ({ bool | boolean | button | cyclic | filePath | float | int | integer | lpp | netSet | radio | real | string }, default is: string)

-callback

Callback for the parameter. (Default is:)

-units

Units for the parameter. (Default is:)

`-choices`

The list of choices to use for the enumeration parameter types. (Default is:)

`-parseAsNumber`

Specifies if the parameter should be interpreted as a number if referenced from PEL expressions. (Default is: true)

`-parseAsPEL`

Specifies if the parameter should be interpreted as a PEL expression. (Default is: false)

`-editable`

Specifies if the parameter should be editable in the **Create Instance** and **Edit Properties** dialog boxes. (Default is: t)

`-display`

Specifies if the parameter should be displayed in the **Create Instance** and **Edit Properties** dialog boxes. (Default is: t)

`-use`

Specifies if the parameter should be displayed in the **Create Instance** and **Property Editor** dialog boxes. (Default is: t)

Also used by netlisters to determine if the parameter should be printed. This is commonly used for providing an interface that presents a variable number of elements like the Time/Voltage pairs of a piecewise linear voltage source.

`-storeDefault`

Is parameter storeDefault. (Default is: false)

`-propList`

The list of attributes for the specified CDF parameter. If you specify `-propList nil`, then `propList` will not appear in the **Parameter Definition Editor** dialog box. (Default is: nil)

`-mode`

Specifies the file mode. (Default is: open)

`-filePrompt`

Specifies the file prompt. (Default is:)

`-relativePath`

Defines the relative path. (Default is: false)

`-fileType`

Specifies the file type. (Default is: file)

`-fileMasks`

File masks. (Default is:)

`-includeQuotes`

Specifies whether to include quotes. (Default is: true)

The default is true.

`-includeParenthesis`

Specifies whether to include parenthesis. (Default is: false)

`-level`

The CDF level in which parameter will be created. ({ base | memory | overlay }, default is: overlay)

Returns

<dbParamDef>: The newly created dbParamDef object.

Errors

If the named dbParamDef object already exists.

Examples

The following example creates a parameter definition with the name `w`.

```
db::createParamDef w -data [dm::findCell buffer\  
  -libName "DemoPLL"]\  
  -prompt "Width"\  
  -defValue 1u\  
  -type string\  
  -units lengthMetric\  
  -parseAsNumber true\  
  -parseAsPEL true\  
  -
```

db::createParamSandbox

Description

Creates a sandbox for parameters.

Syntax

```
db::createParamSandbox
```

Returns

<dbParamSandbox>: The newly created dbParamSandbox object.

Examples

```
db::eval {
    set sb [db::createParamSandbox]
    db::setParamValue param1 -of $inst -value $val1
    db::setParamValue param2 -of $inst -value $val2
    .....
}
```

In this example, `db::createParamSandbox` creates a <dbParamSandbox> object which is used in the `db::eval` block during parameter evaluation.

db::createParasiticStencil

Description

Creates parasitic stencil file for template from DSPF file.

Syntax

```
db::createParasiticStencil -fromFile <string> -cellView <dmCellView |
dmCellViewName> [-instanceMap <string>] [-netMap <string>] [-lineage
<string>]
```

Arguments

-fromFile

Path of the source DSPF file.

-cellView

Template cell view for which the stencil is generated.

-instanceMap

Instance names map between DSPF file and target template design.

-netMap

Net names map between DSPF file and target template design.

-lineage

Hierarchy lineage to the module in DSPF file in which the template is located.

Returns

<string> : The path of newly created file.

db::createPatternDef

Description

Creates a session based dbPatternDef object to define a circuit topology pattern.

Use the `db::getPatternDefs` command to retrieve all registered patterns in the session.

Use the `db::destroy` command to remove a pattern definition from the current session.

Syntax

```
db::createPatternDef <name: string> -insts <dbCollection> [-label  
  <string>] [-expansionType <enum>] [-expansionRange <string>]
```

Arguments

`name`

Name of the pattern definition.

`-insts`

List of dbPatternInst objects defining the topology of the pattern definition.

`-label`

Label to display in the user interface.

`-expansionType`

Whether the pattern definition supports expansion ({ none | replicate | template }).

Default is: none.

`-expansionRange`

Pair of integers defining the start and end expansion numbers.

Returns

<dbPatternDef> : Pattern definition.

Errors

- Pattern definition exists with the given name.
- `expansionRange` given with `expansionType` not given or *none*.

db::createPcell

Description

Creates a PCell with the specified name, using the specified PCell definition. The PCell is created in the library in which the definition resides and within the specified cell and view, which are created if they do not exist. Although users can construct their own `oaViewTypes`, valid `viewTypes` are restricted to a subset of `oaReservedViewTypes`: those that support `oaDesign` databases: `oacMaskLayout`, `oacSchematic`, `oacSchematicSymbol`, or `oacNetlist`. An exception is thrown if `viewType` is not one of these reserved design `viewTypes`.

The PCell is actually an `oaDesign` that is created in the `CellView` indicated by the `lib/cell/view` and marked as a `superMaster`. It contains a reference to the PCell plug-in to use when evaluating an instance of the `superMaster`, including a link to the appropriate evaluation routine to provide to the plug-in order to generate the database objects for the instance's `subMaster`, and a set of parameters and their default values supported by the PCell. It also contains a 'PcellDef' `oaStringProp` that is used to store the name of the PCell definition it is dependent on.

Both a 'getParams' and an 'eval' routine, prefixed by the PCell name, must exist in the definition file associated with the supplied PCell definition.

The `overwrite` argument controls whether the command fails if the `CellView` already contains an `oaDesign`. If set to `true`, the existing design is replaced.

In the event of any error, the `superMaster` `oaDesign` will not be created or overwritten.

Syntax

```
db::createPcell <name: string> -def <dbPcellDef> -cellName <string>  
[-viewName <string>] [-viewType <oaReservedViewType | string>]  
[-overwrite <bool>]
```

Arguments

`name`

The name of the PCell.

`-def`

The PCell definition.

`-cellName`

The cell in which to create the PCell.

`-viewName`

The view in which to create the PCell.

Default: layout.

`-viewType`

The view type of the PCell being defined.

Default: maskLayout.

`-overwrite`

If true, overwrites an existing PCell superMaster, otherwise, the command fails.

The default is false.

Returns

<oaDesign> : The PCell superMaster design.

Errors

- dmcPcellDesignExists (only if overwrite is false).
- oacLibAccessDenied.
- dmcPcellUnableToLoadDefFile.
- dmcPcellUnableToLoadSupportFile.
- dmcPcellGetParamsNotDefined.
- dmcPcellEvalNotDefined.
- dmcPcellParamDoesNotExist.
- oacPcellEvaluatorNotFound.
- oacDesignCannotGetWriteAccess.

Examples

The following example creates a PCell in a library with specified cell name.

```
set pcellDef [db::createPcellDef pcellDefName -libName pcellLib \  
    -def ./pcellDef.tcl]  
db::createPcell pcellName -def $pcellDef -cellName pcell_mos
```

db::createPcellDef

Description

Creates a PCell Definition with the specified name in the specified library.

Both a 'getParams' and an 'eval' routine, prefixed by the PCell name, must exist in the supplied PCell definition file for each PCell this definition file is meant to support. To

verify this, the list of PCell names this definition file supports can be supplied in the `pcellNames` argument, in which case, the definition file will be loaded and the existence of the 'getParams' and 'eval' routines for each PCell name will be verified. If the `pcellName` argument is not supplied, the definition file is not loaded and no verification occurs. Common support routines can be supplied in separate support files.

Because the C++ plug-in requires a separate shared object for each platform that will be supported, both the 'def' and 'support' arguments support a list of file paths. Every file in the list will be attached to the supplied library as DMFiles. The DMFile names that are created for the supplied set of definition and support files are generated by prefixing either 'pcellDef_' or 'pcellSupport_' to the base name of each file.

To prevent multiple definitions from using the same definition file, the DMFile name(s) of the definition file(s) supplied will be verified to not be in use by any other PCell definition. If a conflict is found, the command fails and you must delete the conflicting PCell definition before creating the new one that uses the same definition file.

This does not prevent multiple PCell 'getParams' and 'eval' routines from existing within the same definition file. In that case a single PCell definition should be created and referenced by each PCell, using the `db::createPcell` command, whose 'getParams' and 'eval' routines reside in that file.

You must identify the type of PCell plug-in which the PCell definition and support files require, either `oaTcl` or `oaCPP`.

If the `oaCPP` type is specified, then a shared object for each definition and support file for every platform/compiler combination on which the PCell will be supported needs to be supplied.

If the `oaTcl` type is specified, then only a single definition file can be specified and the filenames do not require an identifier.

The `overwrite` argument controls whether the command fails if a PCell definition or DMFile of the same name already exists. If set to true, an existing definition is replaced by destroying it and all of the DMFiles it references that are not referenced by any other PCell definition. The new definition then overwrites any remaining DMFiles that still exist.

If set to false and an attempt to create a new PCell definition is made with DMFiles that already exist, the command fails.

In the event of any error, the PCell definition will not be created and the definition and support files will not be attached to the library.

Syntax

```
db::createPcellDef <name: string> -libName <string> -def [<string>]  
[-plugin <enum>] [-pcellNames [<string>]] [-support [<string>]]  
[-overwrite <bool>]
```

Arguments

`<string>`

The name of the PCell.

`-libName`

The library in which to create the PCell definition.

`-def`

The list of definition file paths to store in the supplied library.

`-plugin`

The PCell plug-in associated with this definition: {oaCPP | oaTcl}.

Default: oaTcl.

`-pcellNames`

The list of PCell names this definition supports.

Default: "".

`-support`

The list of support file paths to store in the supplied library.

Default: "".

`-overwrite`

If true, overwrites an existing definition, otherwise, the command fails.

The default is false.

Returns

`<dbPcellDef>` The PCell definition object.

Errors

- oacInvalidLibName.
- dmcPcellDefExists (only if overwrite is false).
- dmcPcellDataExists (only if overwrite is false).
- dmcPcellTclOnlySupportsSingleDefFile.
- dmcPcellDefFileNotFound.
- dmcPcellSupportFileNotFound.
- dmcPcellDefFileAlreadyInUse.

- dmcPcellInvalidPlugIn.
- dmcPcellUnableToLoadDefFile (only if pcellNames is supplied).
- dmcPcellGetParamsNotDefined (only if pcellNames is supplied).
- dmcPcellEvalNotDefined (only if pcellNames is supplied).
- dmcPcellUnsupportedPlatform.
- oacLibAccessDenied.
- dmcPcellCPPSharedObjectAlreadyLoaded.

Examples

The following example creates a PCell definition in a library by providing PCell definition Tcl file.

```
db::createPcellDef pcellDefName -libName pcellLib -def ./pcellDef.tcl
```

db::createPin

Description

Creates a pin.

Syntax

```
db::createPin [<name: string>] -term <oaTerm> [-accessDir <int>]
```

Arguments

name

Pin name. The default is P_0.

-term

oaTerm on which the pin will be created.

-accessDir

Access direction. The default is 0.

Returns

<oaInstTerm> : Newly created instTerm

Examples

The following example creates pin object in specified design.

```
# Get edit design from active context
set context [de::getActiveContext]
set design [db::getattr context.editDesign]

# Create rst term with rst net name net in design
set term [le::createTerm rst -design $design -net rst]

# Create pin object in design
db::createPin rst -term $term
```

db::createPref

Description

Creates a new preference definition within all environments and defines its default value. The default value will be set at the install scope within all environments. Because it is impossible to save preferences at install scope, a preference definition created with this command would have to be recreated at each session start by executing this command from the appropriate startup script.

An optional validation procedure may be specified which will be called to validate preference values as they are set. It is called with two arguments: name and value. It should return 0 for invalid data and 1 for valid data. The signature is as follows:

```
proc validProc {name value} {...}
```

The `-defaultScope` argument lets you specify a scope type where this preference will be automatically localized. All preferences are localized at the user scope by default, so this provides an additional localization. For example, if the preference should have distinct values for each `cellView`, specifying `-defaultScope cellView` here will cause it to be automatically localized to each `cellView` scope, thereby ensuring distinct values.

Note:

By providing the default value here, the preference value may be later reverted.

Syntax

```
db::createPref <name: string> [-deprecated <bool> | -defunct <bool>]
-value <string> [-description <string>] [-validationProc <string>]
[-labels [<string>]] [-type <enPrefType>] [-defaultScope <dbScopeType
| string>] [-autoLocalizeUser <bool>] [-replacement <string>]
[-valueChangeProc [<variant>]]
```

Arguments

`name`

Preference name.

`-value`

Preference default value.

`-description`

Optional preference description.

Default: "".

`-validationProc`

Optional procedure called to validate new preference values.

Default: "".

`-labels`

Tcl list of arbitrary string labels for this preference.

Default: {}.

`-type`

Value type which may be used to restrict invalid inputs or provide guided graphical input: {bool | float | int | lcv | multilcv | point | string | enum}.

Default: <string>.

`-defaultScope`

Scope type, in addition to the user scope, where preference will be automatically localized: {cell | cellview | context | custom | library | user}.

Default: user.

`-autoLocalizeUser`

If true, this preference will be automatically localized to user scope when created and on application startup. This option should only be set to false when another interface controls the localizations.

The default is true.

`-deprecated`

True if the preference is deprecated.

The default is false.

`-defunct`

True if the preference is defunct.

The default is false.

`-replacement`

Name of the replacement preference to see the deprecating or defuncting status of this preference.

`-valueChangeProc`

Optional procedure to invoke when this preference is modified.

Returns

`<dbPref>`: The newly created preference object at the install scope regardless of any default localizations.

Examples

The following example creates a `verilogIncludePath` preference with the current directory value in the user scope.

```
db::createPref verilogIncludePath -value ./ -description \
    "Include Files Path"
```

db::createPrinter

Description

Creates a printer object and defines its properties.

The `paperSize` argument takes a list of values to indicate the paper size name, width, height, and origin X and Y offset. A width or height of 0 indicates an unlimited width or height dimension. When the optional X and Y offsets are not provided, the origin is assumed to be {0, 0}.

If a display and/or Object Layer Panel Group argument is supplied, the Print dialog box automatically updates its Display and/or OLP Group widgets to the OLP group argument value, if it exists when the printer is selected.

If a printer by the same name already exists, the `replace` argument must be set to true, otherwise an error occurs.

Syntax

```
db::createPrinter <name: string> (-paperSize [<string>] | -paperSizes
    [<string | {<dbPaperSize>}>]) -resolution <int> -spoolProc <string>
    -queryProc <string> -removeProc <string> [-description <string>]
```

```
[-doubleSided <bool>] [-color <bool>] [-display <string>] [-replace  
<bool>]
```

Arguments

`name: string`

The name of the printer to create.

`-resolution`

The number of addressable units per inch (that is, DPI or PPI).

`-paperSize`

A list describing paper sizes of the printer. Each paper size has a name and four attributes: width, height, offsetX and offsetY. The specified list should contain strings with the format {<name>, <attr>} followed by the value of the attribute <attr> of the paper size named <name>.

`-spoolProc`

Send the print request to the printer queue.

`-queryProc`

Query the printer's queue.

`-removeProc`

Remove a queued print job.

`-description`

A description of the printer.

Default: "".

`-doubleSided`

Indicates if the printer supports double-sided printing.

The default is false.

`-color`

Indicates if the printer supports color printing.

The default is false.

`-display`

Indicates the default DRF Display to use with this printer.

Default: "".

`-replace`

Indicates if an existing printer definition should be replaced with this one.

The default is false.

Returns

The printer that was created ([dbPrinter](#)).

Errors

- Printer with supplied name already exists and replace is false.
- paperSize argument format is invalid.

Examples

```
db::createPrinter lj637 -paperSize [list "A4,width" 8.27 \  
    "A4,height" 11.69 "Letter,width" 8.5 "Letter,height" 11] \  
    -spoolProc "lpr -P lj637" \  
    -queryProc "lpq"\  
    -removeProc "lprm" -resolution 600
```

This example defines a printer named lj637 with size A4 paper, and allows users to submit, query, and remove print jobs.

db::createProgress

Description

Creates a new progress dialog to process Ctrl+C events.

Syntax

```
db::createProgress <name: string> -title <string> [-label <string>]  
    [-maxValue <unsigned int>] [-shown <bool>] [-parent <dbProgress>] [-safe  
    <bool>] [-hideCancel <bool>] [-minDuration <unsigned int>]
```

Arguments

`name`

Unique identifier, used for logging and replay.

`-title`

Title for dialogs.

`-label`

Current suboperation label. (Default is:)

`-maxValue`
Maximum value, if this task is bounded. (Default is: 0)

`-shown`
Show or hide the progress dialog. (Default is: true)

`-parent`
Optional parent progress.

`-safe`
Safe mode will not process events when checking for interrupts. (Default is: true)

`-hideCancel`
Hides the **Cancel** button on the progress dialog. (Default is: false)

`-minDuration`
The time (milliseconds) that must pass before dialog appears. (Default is: 4000)

Returns

`<dbProgress>` : New progress object.

Examples

```
db::eval {
  # Create a progress bar
  set progress [db::createProgress inprogress -label "Lable" \
    -title "Example Progress" -maxValue 100]
  set i 0
  while {$i < 100} {
    incr i

    # Set the progress bar value
    db::setProgress $i -label Label -of $progress

    # Check the Cnt+C binding
    db::checkForInterrupt $i -handler $progress
    puts $i
    after 200
  }
}
```

db::createProp

Description

Creates a prop.

Syntax

```
db::createProp <object: oaObject> -name <string> -value <string> [-type  
  <enum>] [-replace <bool>]
```

Arguments

`object`

An object for which property should be created.

`-name`

Property name

`-value`

Property value

`-type`

propEnum. ({ bool | double | float | int | string }). The default is string)

`-replace`

Property value. The default is true.

Returns

<oaProp> : Newly created oaProp

Examples

The following example creates property for specified design.

```
# Get edit design from active context
set context [de::getActiveContext]
set design [db::getattr context.editDesign]

# Get single instance in the layout
set instance [db::getNext [db::getInsts -of $design]]

# Create lxIgnore Boolean property with value true for instance
db::createProp $instance -name lxIgnore -value true -type bool
```

db::createScope

Description

Creates a new named scope in all environments, returning the one in the system environment. The `-inherits` argument lets you specify the scope from which the newly created scope will inherit preferences, but the newly created scope will not initially have any preferences localized in it.

It is an error to create a scope with the same name as an existing scope or one of the system names (install, site, project, user). The scope name is case insensitive. You can destroy a scope with the `db::destroy` command.

Syntax

```
db::createScope <name: string> [-type <dbScopeType>] [-inherits  
    <dbScope>] [-filePath <string>]
```

Arguments

`name`

Scope name (unique, case insensitive).

`-type`

Specifies the scope type.

`-inherits`

Specifies a scope from which values are inherited.

Default: User scope.

`-filePath`

Default save/load target for this scope.

Returns

Newly created scope ([dbScope](#)).

Errors

- The scope to inherit from does not exist.
- The scope with the same name already exists.

Examples

```
# Create a new scope type
set sct [db::getScopeTypes myScopeType]

# Create a new scope
set sc [db::createScope userScope -type $sct]

# get preference abd ist value from installation scope
set pref [db::getPrefs dbCapacitanceSuffix -scope [db::getScopes install]]

# localize the preference to the new created scope
db::localizePrefs $pref -scope $sc

# Set the preference value another from default
db::setPrefValue dbCapacitanceSuffix -value f

# Check the preference value in the netw scope
```

```
db::getPrefValue dbCapacitanceSuffix -scope userScope

# Check the preference value in the current scope
db::getPrefValue dbCapacitanceSuffix

# Check the preference value in the installation scope, which is the default value
db::getPrefValue dbCapacitanceSuffix -scope install
```

In this example, the `db::createScope` command creates `currentScope` scope, and the following commands update `colorPref` preference value in that scope, and save updated preferences in `./runDir/prefs.xml` file.

```
file mkdir ./runDir
db::createPref colorsPref -defaultScope custom
-value redset scope [db::createScope currentScope -filePath
runDir/prefs.xml]
db::localizePrefs [db::getPrefs colorsPref] -scope
$scope
db::setPrefValue colorsPref -value green -scope $scope
db::savePrefs [db::getPrefs colorsPref -scope $scope]
```

db::createScopeType

Description

Creates a new scope type from which scopes can be created.

Syntax

```
db::createScopeType <{name}: string>
```

Arguments

`name`

Unique, case insensitive scope type name.

Returns

<dbScopeType>: Newly created scope type.

Examples

The following example creates `svnScopeType` scope type. The `svnUrlType` preference will be automatically localized, when `svnScope` scope of this type is created.

```
set scopeType [db::createScopeType svnScopeType]
db::createPref svnUrlType
-defaultScope svnScopeType
-value http://
set scope [db::createScope svnScope -type $scopeType]
db::setPrefValue svnUrlType -value svn://
-scope $scope
#Prints default value
puts [db::getPrefValue svnUrlType]
#Prints updated value
puts [db::getPrefValue svnUrlType -scope $scope]
```

db::createScratchDesign

Description

Creates a scratch design.

Syntax

```
db::createScratchDesign -lib <oaLib | string> -viewType <enum>
```

Arguments

-lib

Lib in which scratch design will be created.

-viewType

View type of scratch design. ({ layout | schematic | symbol })

Returns

<oaDesign> : Newly created scratch design.

Examples

The following example creates scratch layout design for existing library LIB.

```
db::createScratchDesign -lib LIB -viewType layout
```

db::createStdViaVariant

Description

Creates a standard via variant.

Syntax

```
db::createStdViaVariant <name: string> -definition <oaStdViaDef |  
string> -design <oaDesign> [-layer1Enc <point>] [-layer2Enc <point>]  
[-implant1Enc <point>] [-implant2Enc <point>] [-layer1Offset <point>]  
[-layer2Offset <point>] [-cutSpacing <point>] [-originOffset <point>]  
[-cutWidth <float>] [-cutHeight <float>] [-cutLayer <string | unsigned  
int>] [-cutColumns <unsigned int>] [-cutRows <unsigned int>]
```

Arguments

name

The name of the oaStdViaVariant to be created.

-definition

The definition of the oaStdViaVariant to be created.

`-design`

Design in which to create the `std_via_variant`.

`-layer1Enc`

Specifies the width and height of the layer1 enclosure.

`-layer2Enc`

Specifies the width and height of the layer2 enclosure.

`-implant1Enc`

Specifies the width and height of the implant enclosure for the implant1 layer.

`-implant2Enc`

Specifies the width and height of the implant enclosure for the implant2 layer.

`-layer1Offset`

Specifies the current x and y offsets for the layer1 enclosure rectangle.

`-layer2Offset`

Specifies the current x and y offsets for the layer2 enclosure rectangle.

`-cutSpacing`

Specifies the x and y spacing between the cuts in the via cut matrix for this object.

`-originOffset`

Specifies the originOffset for this `oaViaParam` object.

`-cutWidth`

Specifies the width to be used for each cut in the via cut matrix on the cut layer for this `oaViaParam` object.

`-cutHeight`

Specifies the height to be used for each cut in the via cut matrix on the cut layer for this `oaViaParam` object.

`-cutLayer`

Specifies the layer that should be used as the `cutLayer` for this `oaViaParam` object.

`-cutColumns`

Specifies the number of columns for the cut matrix on the `cutLayer` for this `oaViaParam` object.

`-cutRows`

Specifies the number of rows for the cut matrix on the cutLayer for this oaViaParam object.

Returns

<oaStdViaVariant> : Newly created standard via variant.

db::createStreamNameMap

Description

Create a new, empty dbStreamNameMap object.

Syntax

```
db::createStreamNameMap
```

Returns

<dbStreamNameMap>: New, empty dbStreamNameMap object

Examples

The following example creates an empty stream name map object.

```
db::createStreamNameMap
```

db::createSubCircularRuler

Description

Creates a disconnected sub-ruler object.

Syntax

```
db::createSubCircularRuler -radius <float> [-packet <string>] [-label  
<string>]
```

Arguments

`radius`

Sub-ruler radius.

`-packet`

Display packet or packet alias name to draw the circle.

Default: ruler.

-label

Sub-ruler label.

Returns

<dbSubCircularRuler>: Sub circular ruler created.

Examples

```
# Create a ruler object in layout
set obj [db::createSubCircularRuler -radius 0.1 -packet ruler \
    -label "RulerLabel"]

# Check the ruler attributes
puts [db::getAttr obj.label]
puts [db::getAttr obj.radius]

#Create a sub-ruler object
db::createSubCircularRuler -radius 2.4 -label 2.4 -packet ruler1
```

db::createWindowAnnotationSet

Description

Associates an annotation set with a window. To remove the annotation set from the window, destroy the associated dbWindowAnnotationSet object using db::destroy.

Syntax

```
db::createWindowAnnotationSet <string: string> -window <giWindow | int>
```

Arguments

string

Name of the annotation set object to enabled in window.

-window

Window in which to enable annotation set.

Returns

<dbWindowAnnotationSet>: Window annotation set object created.

Examples

```
# Get an annotation set
set anSet [lindex [db::getAnnotationSets ] 0]

# Create window annotation set
db::createWindowAnnotationSet $anSet -window [gi::getActiveWindow]

# Print names of the window annotation sets
```

```
set wndAnSet [db::getWindowAnnotationSets -window [gi::getActiveWindow]]
puts [db::createList [db::getattr wndAnSet.name]]
```

db::decodeSymbol

Description

Decodes an encrypted stack symbol.

Syntax

```
db::decodeSymbol <encrypted_symbol: string>
```

Arguments

`encrypted_symbol`

Encrypted stack symbol.

Returns

`<string>` : Result of decoding encrypted stack symbol

db::defineUdvDevice

Defines user-defined device for udv.

Syntax

```
db::defineUdvDevice -deviceLCV <dmCellViewName> -targetLib <oaLib |  
string> -deltaVoltageParam <string>
```

Returns

`<none>`

Arguments

`-deviceLCV`

The LCV of the defined udv.

`-targetLib`

The target library which applies to this defined udv.

`-deltaVoltageParam`

The mapping name of delta voltage parameter for this defined udv.

db::defineVflagMaxDevice

Defines user-defined device for vflagMax.

Syntax

```
db::defineVflagMaxDevice -deviceLCV <dmCellViewName> -targetLib <oaLib |  
string> -voltageParam <string>
```

Returns

<none>

Arguments

-deviceLCV

The LCV of the defined vflagMax.

-targetLib

The target library which applies to this defined vflagMax..

-voltageParam

The mapping name of voltage parameter for this defined vflagMax..

Examples

```
db::defineVflagMaxDevice -deviceLCV demoPLL_ref10/vmax_symbol/symbol  
-targetLib demoPLL_ref10_tb -voltageParam vmax
```

db::defineVflagMaxMinDevice

Define user-defined device for vflagMaxMin or vflaghl

Syntax

```
db::defineVflagMaxMinDevice -deviceLCV <dmCellViewName> -targetLib <oaLib  
| string> -vmaxParam <string> -vminParam <string>
```

Returns

<none>

Arguments

-deviceLCV

The LCV of the defined vflaghl

-targetLib

The target library which applies to this defined vflaghl

-vmaxParam

The mapping name of vmax parameter for this defined vflaghl

-vminParam

The mapping name of vmin parameter for this defined vflaghl

db::defineVflagMinDevice

Defines user-defined device for vflagMin.

Syntax

```
db::defineVflagMinDevice -deviceLCV <dmCellViewName> -targetLib <oaLib |  
string> -voltageParam <string>
```

Returns

<none>

Arguments

-deviceLCV

The LCV of the defined vflagMin.

-targetLib

The target library which applies to this defined vflagMin.

-voltageParam

The mapping name of voltage parameter for this defined vflagMin.

Examples

```
db::defineVflagMinDevice -deviceLCV demoPLL_ref10/vmin_symbol/symbol  
-targetLib demoPLL_ref10_tb -voltageParam vmin
```

db::defineVsyncDevice

Defines user-defined device for vsync.

Syntax

```
db::defineVsyncDevice -deviceLCV <dmCellViewName> -targetLib <oaLib |  
string>
```

Returns

<none>

Arguments

`-deviceLCV`

The LCV of the defined vsync.

`-targetLib`

The target library which applies to this defined vsync.

db::deleteBuiltInVias

Description

Delete all built-in custom vias.

Syntax

```
db::deleteBuiltInVias (-lib <oaLib> | -libName <dmLibName>)
```

Arguments

`-lib`

OA library where the built-in custom via definitions and cell views will be deleted.

`-libName`

Name of library where the built-in custom via definitions and cell views will be deleted.

Returns

<none>

Examples

The following example deletes build in vias from specified library.

```
db::deleteBuiltInVias -libName builtInVias
```

db::deprecateAttr

Description

Marks an attribute as deprecated or defunct.

Syntax

```
db::deprecateAttr <name: string> -of <string> [-type <enum>]  
[-replacement <string>] [-defunct <bool>]
```

Arguments

`name`

Attribute name to deprecate or make defunct.

`-of`

The type of object for which to deprecate or make defunct the attribute.

`-type`

The attribute type of attribute to deprecate or make defunct: {appDef | intrinsic}.

Default: intrinsic.

`-replacement`

Name of replacement attribute to see regarding deprecating or defuncting of this attribute.

`-defunct`

Mark the attribute defunct.

The default is false.

Returns

<none>

db::destroy

Description

Deletes objects, as described in the following table.

Libraries	Deletes the library's directory and everything it contains from disk and removes the library from the Library Definition File, if one exists. Requires write access to the library. If access is denied the command fails. If the library is deleted but, due to disk access permissions, it is not possible to remove every file associated with the library, the command succeeds, but warnings are generated to inform you about the file removal failures.
Cells	Destroys the supplied cell, removing it from disk. All cellViews contained in this cell and all files in the cell and its cellViews are also removed. Requires write access on the cell's library. If access is denied, the command fails.

Cell categories	This method destroys the specified cell category. It does not destroy the cells themselves, just their grouping into the category. Requires write access to the supplied cell category's library. If access is denied the command fails.
CellViews	Destroys the supplied cellView and the files it contains, removing them from disk. It requires write access to the cellView's library. If access is denied, the command fails.
MPP objects	Destroys the specified MPP objects in the database.
Custom Compiler objects (database objects)	Destroys specified database or Custom Compiler objects. If you pass a partial object to <code>db::destroy</code> , it deletes the entire object.
PCell definitions	Destroys the supplied <code>dbPcellDef</code> , removing it and all DMFiles referenced only by it, from disk. If the PCell definition is a C++ PCell and a PCell C++ shared object has already been loaded for the library in which the <code>dbPcellDef</code> exists, the command fails. The application must be restarted and the definition destroyed before any shared objects are loaded. It requests <code>oacWriteLibAccess</code> access on the PCell definition's library and if access is denied, the command fails.
xtJobs (Custom Compiler jobs)	Destroys the passed xtJob if the process associated with the xtJob has finished executing.

Syntax

```
db::destroy <tclObj:attrObject>
```

Arguments

oaLib

The library to delete.

dmCell

The cell to destroy.

dmCellCategory

The name of the cell category to destroy.

dmCellView

The cellView to destroy.

{leMPP}

The MPP object to delete.

`{oaObject} | {dbObject}`

The Custom Compiler or database object to destroy.

`dbPcellDef`

The PCell definition to destroy.

`xtJob`

The job to delete.

Examples

```
db::destroy [db::getVias -of $oaDesign]
db::destroy [le::getLinearRulers -design $oaDesign]
```

In these two examples, `db::destroy` deletes either all vias or all rulers in the design.

db::detectNetlistType

Description

Detects the default analog library component from the netlist.

Syntax

```
db::detectNetlistType [-netlist <dbCollection>] [-design <oaDesign>]
                        [-data <{<deFigure>} | {<oaFig>}>]
```

Arguments

`-netlist`

Netlist type detection.

`-design`

Design type detection.

`-data`

Data type detection.

Returns

<none>

Examples

```
# Detect netlist type of selected instances
db::detectNetlistType -data [de::getSelected -design [ed]]
```

db::enableTkSend

Description

Enables the Tk send command to work with all X11 authorization methods. Ensures send commands can be received from other applications

Syntax

```
db::enableTkSend [<enable: bool>]
```

Arguments

`enable`

Enable or disable. (Default is: true)

Returns

<none>

db::engToSci

Description

Converts strings to scientific notation. Note that it is not possible to exactly represent all numbers using floating point storage, so some conversions may not be intuitive. For example, 4u may be converted to "3.99990e-6".

The precision argument controls the maximum number of significant digits of the output. The actual precision of the output will be the lesser of the input's number of significant digits and the requested -precision. String input zeros preceding the decimal place will always be treated as significant digits. For example "100k" will be treated as if it has 3 significant digits.

Syntax

```
db::engToSci <input: string> [-precision <unsigned int>]
```

Arguments

<string>

The input value.

-precision <int>

The maximum number of significant digits for the result.

Default: 6.

Returns

The resultant value of converting the input to a string using Scientific notation. (<string>)

Errors

- If the input string is not numeric.
- If the precision is less than 0.

Examples

```
puts [db::engToSci 2.1n]
puts [db::engToSci 2.1M]
```

db::eval

Description

Evaluates a Tcl expression with support for block-scoped objects such as interrupt checking, busy cursors and parameter sandboxes. This command is equivalent to Tcl's 'eval' command except for this additional support.

Syntax

```
db::eval <script: variant> [-log <bool>]
```

Arguments

-script

Tcl expression to evaluate.

-log

Log the expression. The default is false.

Returns

<none>

Examples

```
set design [ed]

# Get all nets of the design
set nets [db::getNets -of $design]
set i 0

# Evaluates a Tcl expression with support for block-scoped objects such
# as interrupt checking.

# Transaction in db::eval helps to automatically undo all changes, if
# user terminates the script with Ctrl+C.
db::eval {
```

```
# Start transaction
set tr [de::startTransaction "Delete Nets" -design $design]
set progress [db::createProgress deleteCells -title "Delete Nets"\
               -maxValue [db::getCount $nets]]
db::foreach net $nets {
    db::setProgress $i -label "Deleting [db::getAttr net.name]" \
        -of $progress
    db::destroy $net
    db::checkForInterrupt $i -handler $progress
    incr i
    after 1000
}
#End transaction
de::endTransaction $tr
}
```

db::evalCallback

Description

Evaluates Callback Syntax script. This function can be called to evaluate the text stored in IL label displays, parameter fields, etc.

Syntax

```
db::evalCallback <callback: string> [-of <oaRef>] [-param <string>]
[-text <oaEvalText>] [-context <heContext>]
```

Arguments

callback

The string to be evaluated.

-of

The current instance, custom via, or standard via.

Default: "".

-param

The name of the current parameter.

Default: "".

-text

oaEvalText object.

-context

Hierarchy context.

Returns

The evaluated value of the callback string (<string>)

Errors

- If the string is not valid Callback Syntax.
- If the evaluated script generates an error.

Examples

```
# Create a blank schematic view and create an instance of reference40nm/n_4t
# Sate the instance name to I0
# Make sure the nf parameter of the instance is set to 1
set design [ed]

# Get the M0 instance
set inst [db::getInsts I0 -of $design]

# The n_4t has an parameter evaluation callback ref40_mos_cbk(nf), which
# evaluate parameter values.
set param [db::evalCallback ref40_mos_cbk(nf) -of $inst]
puts [db::getAttr param.name]
puts [db::getAttr param.value]

# In the schematic design change the nf parameter of the instance to 2
# Now execute the callback
set param [db::evalCallback ref40_mos_cbk(nf) -of $inst]
puts [db::getAttr param.name]
puts [db::getAttr param.value]
```

db::evalEnvironment

Description

Sets the values of `db::getCurrentRef`, `db::getCurrentEvalText`, and `db::getCurrentContext` for the duration of a script.

Syntax

```
db::evalEnvironment [-ref <oaRef>] [-text <oaEvalText>] [-context  
  <heContext>] -script <string>
```

Arguments

`-ref`

The current instance, custom via, or standard via.

`-text`

`oaEvalText` object.

`-context`

Hierarchy context.

`-script`

String to be evaluated.

Returns

<none>

db::evalParam

Description

Calculates the evaluated value of a parameter.

Syntax

```
db::evalParam <param: dbParam> [-context <heContext>] [-evalType <enum>]  
[-refDesignVars <string>] [-refParentParams <string>]
```

Arguments

`dbparam`

Parameter to be evaluated.

`-context`

Design context in which the parameter is evaluated.

`-evalType`

Type of parameter to evaluate: {debug | full | netlist}.

Default: full.

`-refDesignVars`

Name of a Tcl array for storing design variables.

`-refParentParams`

Name of a Tcl array for storing parent references.

Returns

The parameters (<string>)

Examples

```
# Print names and evaluated values of all parameters of the I0 instance  
set inst [db::getInsts I0 -of [ed]]  
set params [db::getParams -of $inst]
```

```
db::foreach param $params {  
    puts "[db::getAttr param.name] [db::evalParam $param]"  
}
```

db::exportImage

Description

Outputs the specified cellview or window contents to the specified format, using a maximum resolution of 1280x1024.

The depth argument is ignored if the source argument is a viewport or the dmCellView is not of type layout.

The olpGroup is only applicable to data of type layout.

Syntax

```
db::exportImage <context: dmCellView | giWindow> -includeBackground  
    <bool> [-depth <int>] [-width <unsigned int>] [-height <unsigned int>]  
    [-file <string>] [-overwrite <bool>] [-display <string>] [-format  
    <enum>] [-colorMode <enum>  
]
```

Arguments

[dmCellView](#) | [giWindow](#)

context: The cellview or window to output.

-depth

Specifies the start level from which the hierarchy should be exported.

-width

Exported image width. (Default: 0)

-height

Exported image height. (Default: 0)

-file

Name of output file.

-overwrite

Overwrites the existing file.

-display

Name of output display to use.

`-includeBackground`

Includes the background during output.

`-format`

Format to use when exporting data ({BMP | JPEG | PNG | SVG}).

`-colorMode`

Output in color, grayscale, or black and white. ({ blackAndWhite | color | grayscale })

Returns

The file that was created (*oaFile*).

Errors

- dmCellView does not exist.
- display is not provided.
- The specified file exists and overwrite is false.
- Supplied printer is not available.
- Supplied window does not contain layout, schematic or symbol oaDesign data.

Examples

```
db::exportImage [dm::getCellViews schematic -cellName amp -libName PLL] \
  -file amp.png -depth 32 -overwrite true -format PNG

# Export images for all cells in hierarchy
set cellview [dm::getCellViews schematic -cellName pll_clk_testbench \
  -libName PLL]
set context [de::createContext $cellview -readOnly true]
he::foreach h [db::getAttr context.hierarchy] {
  db::exportImage [db::getAttr h.cellView] \
    -file [db::getAttr h.cellView.cellName].png -depth 32 \
    -overwrite true -format PNG
}
```

db::exportLibertyFile

Description

Exports Liberty Data from design or cellview to the Liberty File format. If more than one corner is present in the design, the needed corner name should be specified.

Syntax

```
db::exportLibertyFile <name: dmCellView | dmCellViewName | oaDesign>  
  [-corner <string>] -filePath <string> [-defaultsFilePath <string>]  
  [-force <bool>] [-libraryName <string>]
```

Arguments

name

Cellview or design which contains Liberty Data to be exported.

-corner

Name of Liberty Data. (Default is: nominal)

-filePath

Filename to output.

-defaultsFilePath

Path to Liberty Defaults XML file.

-force

Overwrite existing file. (Default is: false)

-libraryName

Name of field 'library' in the generated file.

Errors

- Cannot create file with the specified name.
- File exists and `-force` is false.
- The corner with the specified name does not exist in the design.

Examples

```
# Export liberty data from the current edit design to the liberty.lib  
file  
db::exportLibertyFile [ed] -filePath liberty.lib
```

db::exportMCellFile

Description

Exports an MCell definition file.

Syntax

```
db::exportMCellFile <path: string> (-lib <oaLib> | -libName <string>)  
[-cdfFilePath <string>] [-overwrite <bool>]
```

Arguments

path

Path including the file name of MCell definition file to export.

-lib

OA library from where MCell superMaster data should be exported.

-libName

Name of the OA library from where MCell superMaster data should be exported.

-cdfFilePath

Path including the file name of MCell CDF file to export.

-overwrite

Overwrites the existing MCell definition and CDF file. The default is false.

Returns

<none>

Examples

The following example exports MCell devices into a definition file.

```
db::exportMCellFile ./guardRing.mcdef -libName mcellLib
```

db::exportParams

Description

Serializes parameter definitions and Netlist Info to an equivalent Tcl script. The generated script can be used to copy Netlist Info and parameter definitions between oaDMDData objects.

Syntax

```
db::exportParams -level <enum> -of <dmCell | oaCellDMDData | oaLib |  
oaLibDMDData | string> -filePath <string>
```

Arguments

`-of`

DMDData (or a cell, library, or library name that contains one) that supplies the parameters to export.

`-filePath`

The output Tcl file.

`-level`

The CDF level from which parameters will be exported ({ base | memory | overlay }, default is: base)

Returns

<none>

Examples

In the following example, `dm::findCell` finds the NMOS device in the `analogLib` library and passes it to the `db::exportParams` command, which outputs the NMOS device component description format.

```
db::exportParams -of [dm::findCell nmos -libName analogLib] \
    -filePath ./CDF/cdf.nmos
```

db::exportStream

Description

Executes **Export Stream**.

Syntax

```
db::exportStream -gds <string> -lib <string> [-batchMode <bool>]
    [-showResult <bool>] [-libDefFile <string>] [-acutePathToPolygon
    <bool>] [-autoMapObjects <unsigned int>] [-blockageType <unsigned
    int>] [-cell [<string>]] [-cellMap <string>] [-cellNamePrefix <string>]
    [-cellNameSuffix <string>] [-charLimit <bool>] [-colorMapping <enum>]
    [-convertPinToText <bool>] [-convertNetFromPinToText
    <bool>] [-convertTextBrackets <enum>] [-dbuPerUU <int>] [-donutNumSides
    <unsigned int>] [-dumpDesignTree <bool>] [-dumpRefLib <bool>]
    [-duplicateCell <enum>] [-ellipseNumSides <unsigned int>]
    [-expandArrayInsts <bool>] [-filterAcutePaths <bool>] [-filterDots
    <bool>] [-filterEmptyCells <bool>] [-filterReentrantPgns <bool>]
    [-filterSelfIntersectingPgns <bool>] [-filterText <bool>]
    [-filterUnboundInsts <bool>] [-filterZeroWidth <bool>] [-flattenPcells
    <bool>] [-flattenVias <bool>] [-fontMap <string>] [-hierDepth
    <int>] [-ignoreCellsAffix <string>] [-ignoreTopCellsAffix <bool>]
    [-keepAbuts <bool>] [-keepFullCellName <bool>] [-keepViewName <bool>]
```

```
[-keepHalfWidthPaths <bool>] [-keepPcells <bool>] [-keepVias <bool>]
[-labelDepth <unsigned int>] [-labelMap <string>] [-lakerLicense
<string>] [-layerMap [<string>]] [-logFile <string>] [-mapAllText
<bool>] [-maxVertices <unsigned int>] [-mergeCellviews <string>]
[-newMaxVerticesFunction <bool>] [-newMaxVerticesRule <string>]
[-noAutoBackupFile <bool>] [-noConvertPath <bool>] [-noExportRefLibCells
<bool>] [-noInfo [<unsigned int>]] [-noOutputBlockages <bool>]
[-noOutputBoundaries <bool>] [-noOutputTextDisplays <bool>]
[-noRoundPathToPolygon <bool>] [-noWarning [<unsigned int>]]
[-notExternalizeShapes <bool>] [-outputBBoxInfo <bool>] [-sortLayer
<bool>] [-objectMap <string>] [-pathToPolygon <bool>] [-propMap
<string>] [-propValueOnly <bool>] [-propertyInst <unsigned int>]
[-propertyLogic <unsigned int>] [-propertyNet <unsigned int>]
[-propertyVia <unsigned int>] [-propertyWideAlignment <unsigned int>]
[-propertyVia <uns
igned int>] [-rectAsBoundary <bool>] [-refLibList <string>] [-refLibs
<string>]] [-referenceMap <string>] [-roundPathToPolygon <bool>]
[-snapToMfgGrid <bool>] [-tclFile <string>] [-tclFunction [<string>]]
[-techLib <string>] [-text <string>] [-textDepth <int>] [-textHeight
<enum>] [-textMap [<string>]] [-toLower <bool>] [-toLowerLabel <bool>]
[-toUpper <bool>] [-toUpperLabel <bool>] [-unmappedLayersUseOALayers
<bool>] [-useTechLayerMap <bool>] [-useTechObjectMap <bool>] [-ver
<unsigned int>] [-view <string>]
```

Arguments

-gds

Output stream file.

-lib

Library name.

-batchMode

Specifies if exportStream is running with interactive mode. (Default is: true)

-showResult

Specifies if exportStream is finished in batch mode. (Default is: false)

-libDefFile

Uses a specific library definitions file.

-acutePathToPolygon

Converts acute paths to polygons. (Default is: false)

-autoMapObjects

Uses layer map entries to write prBoundaries, snapBoundaries, and layer/area Blockages. (Default is: 99)

- `-blockageType`
Data type for output blockages. (Default is: 0)
- `-cell`
Specifies the input cell name.
- `-cellMap`
Specifies the cell map file.
- `-cellNamePrefix`
Adds a prefix to all structure names in the stream file.
- `-cellNameSuffix`
Adds a suffix to all structure names in the stream file.
- `-charLimit`
Uses the 32-character limit on structure names. (Default is: false)
- `-colorMapping`
Specifies the layer mapping rule. Valid values are allVias, full, honor, ignore, lockOnly, and strict. (Default is: honor)
- `-convertPinToText`
Converts pin's term name to label. (Default is: false)
- `-convertNetFromPinToText`
Convert pin's net name to label (Default is: false)
- `-convertTextBrackets`
Converts text brackets to angle brackets or square brackets. Valid values are angle and square.
- `-dbuPerUU`
Specifies the dbuPerUU value. (Default is: 1000)
- `-donutNumSides`
Specifies the number of sides for the boundary of donut objects. (Default is: 64)
- `-dumpDesignTree`
Dumps the design tree to a log file. (Default is: false)
- `-dumpRefLib`
Dumps the reference library list to a log file. (Default is: false)

`-duplicateCell`

Specifies the handling for duplicate cells. Valid values are skip, withLibName, and withNumber. (Default is: withNumber)

`-ellipseNumSides`

Specifies the number of sides for the boundary of ellipse objects. (Default is: 64)

`-expandArrayInsts`

Expands Arrays to Instances. (Default is: false)

`-filterAcutePaths`

Filters acute paths. (Default is: false)

`-filterDots`

Filters dots. (Default is: false)

`-filterEmptyCells`

Filters empty cells. (Default is: false)

`-filterReentrantPgns`

Filters reentrant polygons. (Default is: false)

`-filterSelfIntersectingPgns`

Filters self-intersecting polygons. (Default is: false)

`-filterText`

Filters text. (Default is: false)

`-filterUnboundInsts`

Filters unbound instances. (Default is: false)

`-filterZeroWidth`

Filters zero width paths. (Default is: false)

`-flattenPcells`

Flattens any PCells into their geometry. (Default is: false)

`-flattenVias`

Flattens standard vias into their geometry. (Default is: false)

`-fontMap`

Specifies the font map file.

`-hierDepth`

Specifies the hierarchical depth to translate to. (Default is: 256)

`-ignoreCellsAffix`

When specified, do not add any prefix or suffix to cells.

`-ignoreTopCellsAffix`

When specified, do not add any prefix or suffix to top cells.

`-keepAbuts`

Keeps abutments. (Default is: false)

`-keepFullCellName`

Uses the original length of cell submaster (and stdVia) name. (Default is: false)

`-keepViewName`

Specifies if the translator to keep the view name. (Default: false)

`-keepHalfWidthPaths`

Keeps half width paths. (Default is: false)

`-keepPcells`

Keeps PCells. (Default is: false)

`-keepVias`

Keeps vias. (Default is: false)

`-labelDepth`

Adds labels to the specified hierarchical depth. (Default is: 1)

`-labelMap`

Specifies the label map file.

`-lakerLicense`

Permits using Laker licenses to access Custom Compiler features. Valid values are schematic, layout, and both. (Default is: " ")

`-layerMap`

Dumps the parsing information of layer mapping to a log file for debugging.

`-logFile`

Specifies the log file. (Default is: exportStream.log)

`-mapAllText`

Maps all texts from CDBA to the namespace specified with `-text`. (Default is: `false`)

`-maxVertices`

Splits polygons and paths with more than the specified number of vertices. Valid values are from 200 to 8190. (Default is: 8190)

`-mergeCellviews`

Specifies whether to merge cellviews file.

`-newMaxVerticesFunction`

Sets to use new `maxVertices` function. (Default is: `false`)

`-newMaxVerticesRule`

Sets new `maxVertices` rules. (`{ SwapXY | noSwap | debug }`). (Default is: `SwapXY`)

`-noAutoBackupFile`

When specified, do not automatically back up the output GDS file if the file exists. (Default is: `false`)

`-noConvertPath`

When specified, do not convert half width paths to boundaries. (Default is: `false`)

`-noExportRefLibCells`

Only outputs cells from the library specified by `-lib`. (Default is: `false`)

`-noInfo`

Suppresses the given INFO message IDs.

`-noOutputBlockages`

When specified, do not output any blockage objects. (Default is: `false`)

`-noOutputBoundaries`

When specified, do not output any boundaries objects. (Default is: `false`)

`-noOutputTextDisplays`

When specified, do not output any `textDisplay` objects. (Default is: `false`)

`-noRoundPathToPolygon`

When specified, do not convert round paths to polygons. (Default is: `false`)

`-noWarning`

Suppresses the given WARNING message IDs.

`-outputBBoxInfo`

Output bounding box (bbox) information for cell in the log file. (Default is: false)

`-notExternalizeShapes`

When specified, do not externalize shapes. (Default is: false)

`-sortLayer`

Sorts layer statistics in the log file. (Default is: false)

`-objectMap`

Specifies the object map file.

`-pathToPolygon`

Converts paths to polygons. (Default is: false)

`-propMap`

Specifies the prop map file.

`-propValueOnly`

Exports property values only. (Default is: false)

`-propertyInst`

Converts property in (1-127) to instance name.

`-propertyLogic`

Converts property in (1-127) to logic information.

`-propertyNet`

Converts property in (1-127) to net name.

`-propertyVia`

Converts Via to property in (1-127).

`-propertyWideAlignment`

Converts property in (1-127) to wide alignment information.

`-rectAsBoundary`

Exports rectangles as boundary records. (Default is: false)

`-refLibList`

Specifies the reference library list file.

`-refLibs`

Specifies a list of libraries that contains the reference cells.

`-referenceMap`

Exports a complete mapping of designs to stream structures.

`-roundPathToPolygon`

Converts round paths to polygons. (Default is: false)

`-snapToMfgGrid`

Snaps vertices to the manufacturing grid. (Default is: false)

`-tclFile`

Specifies the Tcl file.

`-tclFunction`

Specifies the Tcl function.

`-techLib`

Specifies the technology library.

`-text`

Text namespace. (Default is: cdba)

`-textDepth`

Text depth to translate to. (Default is: 256)

`-textHeight`

Text height mode ({ matchOABBox | matchObjectHeight }). (Default: matchOABBox)

`-textMap`

Quoted list of text map files.

`-toLower`

Converts cell and instance names to lower case. (Default is: false)

`-toLowerLabel`

Converts label names to lower case. (Default is: false)

`-toUpper`

Converts cell and instance names to upper case. (Default is: false)

`-toUpperLabel`

Converts label names to upper case. (Default is: false)

`-unmappedLayersUseOALayers`

Uses OA layer numbers if not mapped. (Default is: false)

`-useTechLayerMap`

Uses the layer map file that is in the source technology library. (Default is: false)

`-useTechObjectMap`

Use the object map file that is in the source technology library. (Default is: false)

`-ver`

Specifies the stream version to write out. (Default is: 3)

`-view`

Specifies the input view name.

Returns

<bool>: Execute exportStream.

Examples

The following example exports specified design data into GDSII file in batch mode.

```
db::exportStream -gds and.gds -batchMode true -lib DemoPLL -cell and2 \  
-hierDepth 32 -keepPcells true \  
-layerMap./PDK/techfiles/refl_layer.map
```

db::exportTechFile

Description

Exports the technology data, either local or referenced from OpenAccess database, into a *.tf technology file.

Syntax

```
db::exportTechFile <path: string> (-lib <oaLib> | -libName <string>)  
[-overwrite <bool>] [-local <bool>] [-includeEmptySections <bool>]  
[-sections <string>]
```

Arguments

path

Path including the file name of technology file to export.

`-lib`

OA library, from where technology data should be exported.

`-libName`

Name of OA library, from where technology data should be exported.

`-overwrite`

Whether to overwrite or not existing technology file. (Default is: false)

`-local`

Whether to export local or global. (Default is: false)

`-includeEmptySections`

Whether to export include empty sections. (Default is: true)

`-sections`

Sections or subsections to be exported. If you specify the name of a section, all data within that section is exported. If you specify a subsection, only the data in the subsection is exported. For the names of the sections and subsections, see Technology File Contents. (Default: all)

For example, `-sections controls` exports all data in the controls section.

`-sections {techLayers techDisplays}` exports the data in the techLayers and techDisplays subsections of the layerDefinitions section. `-sections all` exports all of the technology data.

Examples

```
db::exportTechFile ./ref40.tf -libName reference40nm -overwrite true \  
-includeEmptySections true
```

db::expr

Description

Evaluates an arithmetic expression. It is identical to Tcl's "expr" command with the exception that it includes support for Custom Compiler supported engineering suffixes.

Syntax

```
db::expr <string: string> [-of <oaCustomVia | oaInst>] [-evalType <enum>]  
[-context <heContext>] [-map <dbExpressionMap>] [-refLineageParams  
<string>] [-refParentParams <string>] [-refDesignVars <string>]  
[-designVars [<map>]]
```


Arguments

`<string>`

The input arithmetic expression. Numeric comparison "==" is supported.

`-of`

Object to be queried.

`evalType`

Parameter evaluation type: {debug | full | netlist | none}.

Default: none.

`-context`

Context for full evaluation.

`-map`

Expression map for evaluation.

`-refLineageParams`

Variable storage reference for lineage parameters.

`-refParentParams`

Variable storage reference for parent parameters.

`-refDesignVars`

Variable storage reference for design variables.

`-designVars`

Set of design variable/values.

Returns

The resultant value of the input expression. Note that the actual returned object type must be a Tcl double to preserve precision. (<float>).

Errors

If the arithmetic expression contains errors such as divide by zero or improper syntax.

Examples

```
db::expr "10.0m == 10.00e-03"
```

db::filter

Description

Returns an un-elaborated read-only dbCollection that is derived from another collection by filtering.

Syntax

```
db::filter <collection: dbCollection> (-filter <string> | -type <string>)
```

Arguments

collection

Collection to filter.

-filter

Expression to filter the collection.

-type

Filters the collection to the objects of the specified type.

Returns

<dbCollection>: Filtered collection.

Examples

The following example gets all visible LPPs in the edited design.

```
set LPPs [db::getLPPs -from [ed] -filter {%visible}]
```

The following example filters out LPPs with "drawing" purpose.

```
set drawingLPPs [db::filter $LPPs -filter {[lindex %lpp 1]=="drawing"}]
```

db::findCircuitMatches

Description

Returns a collection of dbNetlistMatch objects for each unique circuit structure that matches the given template circuit structure.

Syntax

```
db::findCircuitMatches <none: dbCollection> (-of <oaDesign> | -in  
  <dbCollection>) [-full <bool>] [-postFilter <oaDesign>]
```

Arguments

`none`

Collection of ordered templates instances representing a single circuit structure.

`-of`

Design to search.

`-in`

Circuit structure to search.

`-full`

Indicates whether the source and target circuits structures must fully match (true) or partly (false). The default is false.

`-postFilter`

If an `oaDesign` exists, this option processes all possible matches and separates them into two sets. The first set contains best matches which do not exist in the second set, and the second set contains all other matches which do not exist in the first set. Otherwise, it uses the default way. The default is false.

Returns

`<dbCollection>` : Collection of `dbNetlistMatch` objects.

Examples

```
# Get templates from the templLib library
set templates [db::getTemplates -libName templLib]

# Find circuit matches in the current edit design and print their names
set cm [db::findCircuitMatches $templates -of [ed]]
puts [db::createList [db::getattr cm.source.name]]
```

db::findConnections

Description

Collects information about all connect definitions in the hierarchy defined by the given context or design. Since the scanning for connect definitions could be quite intensive, the returned object will be a reference to this collected data. When this object goes out of scope, the resources used to hold the collected data will be freed. The data held in this object can go out of date. Because of this constraint, you should re-extract the connection information by invoking this command again.

Syntax

```
db::findConnections <none: heContext | oaDesign>
```

Arguments

`heContext` | `oaDesign`

Top level context or design to use to collect all defined connection information.

Returns

`dbConnections` object describing all defined connections in the given hierarchy (`dbConnections`).

Errors

Invalid argument.

Examples

```
# Find connection definitions from design and print their names and
# default nets
set connInfo [db::findConnections [ed]]
db::foreach connDef [db::getAttr parameters -of $connInfo] {
    puts "name: [db::getAttr connDef.name] defaultNet: [db::getAttr
    connDef.defaultNet]"
}
```

db::findConstraint

Description

Returns the first found `oaConstraint` of the given definition. This command returns *none* if no `oaConstraint` of the given definition is found.

Syntax

```
db::findConstraint <definition: dbConstraintDefName> [-layers [<int |
string>]] -of <oaDesign | oaTech>
```

Arguments

`definition`

Constraint definition name to find.

`-layers`

Tcl list of layer numbers or names.

`-of`

Database to begin search, if `oaDesign` is given and no constraint is found, the associated `oaTech` will be searched.

Returns

<dbConstraint> : dbConstraint object.

Examples

The following example returns first found constraint of the given constraint definition.

```
# Get edit design from active context
set context [de::getActiveContext]
set design [db::getAttr context.editDesign]

# Find minSpacing constrain for M1 layer (8) from design
db::findConstraint minSpacing -from $design -layers 8
```

db::findContextConnectionDef

Description

Retrieves information about a specific connect definition with respect to the given context. The command returns a dbContextConnectionDef which can be used to extract additional information about this connect definition.

Syntax

```
db::findContextConnectionDef <none: string> -of <dbContextConnections>
```

Arguments

<string>

Connection definition name to use when finding the object.

-of [dbContextConnections](#)

Context connections object for which a specific connection definition is returned.

Returns

Connection object for the context describing all defined connections in the context hierarchy ([dbContextConnectionDef](#)).

Errors

Invalid argument.

Examples

```
# Find connection definitions with name "power" from specified context
and print its state in I0 instance master
set context [de::getActiveContext]
set h [db::getAttr context.hierarchy]
set design [db::getAttr context.editDesign]
```

```
set ctxConnections [db::findContextConnections $h -of  
    [db::findConnections $h]]  
set contextDef [db::findContextConnectionDef power -of $ctxConnections]  
puts [db::getConnectionDefState -of $contextDef -instName I0]
```

db::findContextConnections

Description

Retrieves the connection information for a specific context or design. The command returns a `dbContextConnections` object which can then be used to extract additional information about connect definitions and assignments in this context.

Syntax

```
db::findContextConnections <heContext> -of <dbConnections>
```

Arguments

`heContext`

Context for which connection information is returned.

`-of dbConnections`

Defined connections object.

Returns

`dbContextConnections` object describing all defined connections in the given design context. ([dbContextConnections](#))

Errors

Invalid argument.

Examples

```
# Find connection definitions from specified context and print their  
names, default net and source type  
set context [de::getActiveContext]  
set h [db::getAttr context.hierarchy]  
set design [db::getAttr context.editDesign]  
set ctxConnections [db::findContextConnections $h -of  
    [db::findConnections $h]]  
foreach instName [db::getAttr ctxConnections.instances] {  
    puts "instName = $instName"  
    db::foreach ctxConnectDef [db::getContextConnectionDefs -from  
        $ctxConnections -instName $instName] {  
        set connectDef [db::getAttr ctxConnectDef.connectionDef]  
        puts "    name: [db::getAttr connectDef.name] defaultNet:  
[db::getAttr connectDef.defaultNet] sourceType: [db::getAttr  
connectDef.sourceType]"
```

```
}  
}
```

db::findGenerator

Description

Returns the specified cellView from cellView generator Tcl procedure name. An empty string is returned if there is no registered generator for the given `-for` viewtype name. The optional `-from` argument can be used to find a generator registered against a specific source viewtype name.

Syntax

```
db::findGenerator -for <string> [-of <string>]
```

Arguments

`-for <string>`

Destination viewtype name of the generator.

`-of <string>`

Source viewtype name of the generator.

Returns

Name of the registered Tcl procedure which implements the specified generator (<string>).

Errors

Invalid viewtype name is given.

Examples

```
db::findGenerator -for schematicSymbol -from schematic  
db::findGenerator -for schematicSymbol -from verilogAMSText  
db::findGenerator -for schematicSymbol -from verilogText  
# For all cellviews generation se::generateSymbol command is used.  
# You may create your own procedure and register your own cellview generator.
```

Examples

```
# Unregister previous cellview generator for symbol view  
db::unregisterGenerator -for schematicSymbol  
  
# Register new cellview generator for symbol view, which adds copyright  
# label in symbol view  
db::registerGenerator createSymbol \  
    -for schematicSymbol \  
    -optionSetType symbolGenerator \  
    -usesPinSides true \  
    -defaultOptionSetPref seSymbolGenOptionSetDefault \  
    -
```

```

        -supportsUpdate true
proc createSymbol {args} {
    array set opts $args
    set iface $opts(-interface)
    set gos $opts(-options)
    set dest $opts(-dest)
    se::generateSymbol -interface $iface -dest $dest -options $gos
    -update false
    set symbolLCV [file join [db::getAttr dest.libName] [db::getAttr
dest.cellName] [db::getAttr dest.name]]
    set design [db::openDesign $symbolLCV -readOnly true]
    de::createContext $design
    se::createText "Copyright (c) Synopsys" -anchor {0 0} -design $design
    -type note
    de::save $design
    db::closeDesign $design
}

# Print registered generator name for symbol view
db::findGenerator -for schematicSymbol

```

db::findInterfaceReader

Description

Returns the specified interface reader Tcl procedure name. An empty string is returned if there is no registered interface reader.

Syntax

```
db::findInterfaceReader -viewType <string>
```

Arguments

```
-viewType <string>
```

Viewtype name of the reader.

Returns

Name of the registered Tcl procedure which implements the specified reader (<string>).

Errors

Invalid viewtype name is given.

Examples

```

# Find and print interface reader tcl proc name for layout view
set int_reader [db::findInterfaceReader -viewType maskLayout]
puts $int_reader

# Unregister an interface reader

```



```
db::unregisterInterfaceReader -viewType maskLayout

# Register "new_proc" as the interface reader procedure
db::registerInterfaceReader new_proc -viewType maskLayout
puts [db::findInterfaceReader -viewType maskLayout]
```

db::findPatternMatches

Description

Finds all matches for each pattern definition given in the input collection. Each match will produce a dbPatternMatch object in the output collection if a dbPatternMap object exists in the patternMap collection for the specific pattern definition.

Syntax

```
db::findPatternMatches <pattern_defs: dbCollection> (-in <dmCellView |
  oaDesign> | -from <{<deFigure>} | {<oaFig>} | {<oaInst>}}>)
```

Arguments

pattern_defs

Ordered collection of pattern definitions to search for.

-in

Design or cell view to search for patterns in.

-from

Collection of deFigures or oalnsts defining the graph for searching. Mutually exclusive with -in.

Returns

<dbCollection> : Collection of pattern matches found.

db::flushPcellCache

Description

Flushes the persistent cache for the supplied PCell superMasters. If omitted, then all PCell superMasters are flushed for all libraries where caching is enabled

Syntax

```
db::flushPcellCache [-cellViews <{<dmCellView>}>] [-param
  [<oaParamArray>]]
```

Arguments

`-cellViews`

Set of PCell superMasters for which to flush cached subMasters.

`-param`

Specifies a set of subMasters to flush for each target dmCellView.

Returns

<none>

Examples

```
# Flush cache for reference40nm/nwellGR cell
db::flushPcellCache -cellViews [dm::findCellView layout -cellName nwellGR
    -libName reference40nm]
```

db::foreach

Description

Provides “foreach” loop control for iterating over dbCollections.

Note:

For consistency with the standard Tcl foreach syntax, the `db::foreach` command has three positional arguments, which is an exception to the command guidelines. However, unlike the standard Tcl foreach command syntax, the first positional argument can have only one variable.

Syntax

```
db::foreach <varName: string> <collection: dbCollection | dbIterator>
    <script: script>
```

Arguments

<varName>

Name of a variable, which will be iterated over the collection.

collection

An iterator over the dbCollection.

<script>

Tcl script to execute.

Examples

```
set instsCollection [db::getInsts -of $oaDesign]
    db::foreach oaInst $instsCollection {
        puts "Inst name = [db::getattr name -of $oaInst]"
    }
```

In this example, `db::foreach` traverses all instances in the design and prints all instance names.

db::freezeRefs

Description

Freezes or unfreezes all references in the design.

Syntax

```
db::freezeRefs -of <oaDesign> [-freeze <bool>]
```

Arguments

`-of`

The design which references to freeze / thaw.

`-freeze`

Freeze (true) or thaw (false) all references in the design. (Default is: true)

Returns

<none>

Examples

```
# Preference dbPcellFreezeEnabled must be true to freeze references
db::setPrefValue dbPcellFreezeEnabled -value true
```

```
# Freeze all references in design
db::freezeRefs -of [ed]
```

```
# Unfreeze all references in design
db::freezeRefs -of [ed] -freeze false
```

db::generateBuiltInVias

Description

Generate built-in custom via based on standard via definitions.

Syntax

```
db::generateBuiltInVias (-lib <oaLib> | -libName <dmLibName>)
```

Arguments

`-lib`

OA library where the built-in custom via definitions and cell views will be created.

`-libName`

Name of library where the built-in custom via definitions and cell views will be created.

Returns

{<dmCellView>}: The collection of newly created built-in custom via cellviews.

Examples

The following example generates build in vias into specified library.

```
db::generateBuiltInVias -libName builtInVias
```

db::generateCDL

Description

Generates a CDL cellview.

Syntax

```
db::generateCDL -interface <dbInterfaceSet> -dest <dmCellView |  
dmCellViewName> [-options <dbGenOptionSet>]
```

Arguments

`-interface`

The interface set object.

`-dest`

Destination cellview.

`-options`

Object containing the option values.

Examples

```
# Select a source cell which contains CDL interfaceSet  
set cellViewSource [dm::findCellView schematic -cellName amp -libName PLL]  
  
# Get Interface set from the source cell
```

```
set interfaceSet [db::getCellViewInterface -src $cellViewSource]

# Generate CDL cell view
db::generateCDL -interface $interfaceSet -dest PLL/amp/cdl
```

db::generateCellView

Description

Creates a cellView from the given cellView or interface set. If a source cellView is supplied, the interface set is constructed from this cellView and passed to the registered generator for the destination view type.

If a generator is not registered for the destination view type, a Tcl error is generated.

If `-updateSearch` changed is given, the destination cellView must exist. Otherwise, a Tcl error is generated.

Syntax

```
db::generateCellView -dest <dmCellView | dmCellViewName> [-cellView  
  <dmCellView | dmCellViewName>] [-interface <dbInterfaceSet>] [-viewType  
  <string>] [-update <bool>] [-options <dbGenOptionSet>] [-pinMap]
```

Arguments

`-dest` `dmCellView`

The destination cellView.

`-cellView` `dmCellView`

The source cellView.

`-interface` `dbInterfaceSet`

Tcl list of pin definitions. A pin definition is an ordered list of name, type, and side.

`-viewType`

Destination cellView viewType name, if `-dest` option is not specified.

`-update`

Indicates whether a existing destination cellView should be regenerated or just updated.

The default is false.

`-options` `dbGenOptionSet`

Optional option set that applies to the corresponding generator. The option set type must match the type registered with the generator.

`-pinMap`

Optional option to `givdb::getEquivalentNetse` the pin name mapping data to the generator.

Examples

The following example generates symbol in the existing symbol view from specified schematic view.

```
set srcCellView [dm::getCellViews schematic -cellName amp \
                -libName DemoDesign]
set destCellView [dm::getCellViews symbol -cellName amp \
                -libName DemoDesign]
db::generateCellView -cellView $srcCellView -dest $destCellView
```

db::generateConstraintPatterns

Description

Generates constraint patterns for foundry or user-defined constraint group of input library.

Syntax

```
db::generateConstraintPatterns -techLibName <string> -outputLibName
<string> -constrDefList <string> [-constraintGroupName <string>]
[-verboseLevel <int>] [-forAutoVia <bool>]
```

Arguments

`-techLibName`

Technology library name.

`-outputLibName`

Output library name.

`-constrDefList`

Generates constraint definitions.

`-constraintGroupName`

Generates constraint group name. The default is foundry.

`-verboseLevel`

Verbose level for report file (0 or 1). The default is 0.

`-forAutoVia`

Generates patterns for AutoVia. The default is false.

db::generateDeviceMappingTemplate

Description

Generates the device mapping file template for cells from the specified library, filtered and based on cell name patterns and the view search list.

Syntax

```
db::generateDeviceMappingTemplate -filePath <string> -libName <string>  
  [-viewSearchList <string>] [-cellName <string>] [-mapModules <bool>]  
  [-force <bool>]
```

Arguments

-filePath

File path to be created (relative or absolute).

-libName

Name of the library to parse.

-viewSearchList

List of view names to find, separated by spaces.

Default: symbol.

-cellName

Wildcard pattern to use for cell name filtering.

Default: *.

-mapModules

Option queried when the namePrefix prop is not found, if true devType is set to X, otherwise - "?".

The default is true.

-force

Force to overwrite the file if it already exists.

The default is false.

Examples

```
# Generate devMap_template.xml file for the reference40nm library  
db::generateDeviceMappingTemplate -filePath devMap.xml -libName  
reference40nm
```

```
# Import amds_lib_ldo spice netlist into DesignLib library using
  devMap.xml as the device mapping file
set deviceMapFile [./samples device_map.xml]
db::importText ./amds_lib_ldo.cir -libName DesignLib \
-viewType SPICEText -viewName hspiceD -overwrite always \
-generateSymbols default -overwriteSymbols always \
-deviceMappingFile $deviceMapFile
```

db::generateLayout

Description

Generates a design layout from a given schematic.

Syntax

```
db::generateLayout <cellView: dmCellView | dmCellViewName>
-src <dmCellView | dmCellViewName> [-mFactorNames [<string>]]
[-toolSearchList [<string>]] [-template <enum>] [-force <bool>] [-xScale
<float>] [-yScale <float>]
```

Arguments

cellView

Destination cellView.

-src

Source cellView.

-mFactorNames

Space delimited string of parameter/property names to recognize as M-Factor for the placeLikeSchematic template.

-toolSearchList

Space delimited string of CDF tool names to search for terminal and property mapping and/or terminal order for the placeLikeSchematic template.

-template

Template that contains the algorithms used as the basis for generating the layout. ({ floorPlan | placeLikeSchematic }, default is: placeLikeSchematic)

-force

Flag to write over existing top design (Default is: false)

-xScale

Scale factor for x-axis for the floorPlan template.

`-yScale`

Scale factor for y-axis for the floorPlan template.

Returns

<none>

Examples

The following example creates a flat layout design from a source design.

```
# Get destination cell view
set destCellView [ dm::getCellViews layout -cellName and2 -libName
    DemoPLL ]

# Get source cell view
set sourceCellView [ dm::getCellViews layout.config -cellName and2
    -libName DemoPLL ]

# Generate layout for existing cell view using source cell view with
# option force
db::generateLayout $destCellView -src $sourceCellView -force true

# Get number of generated objects in destination cell view
set design [db::openDesign $destCellView]
db::getCount [de::getFigures -design $design]
```

db::generateRule

Description

Converts rules.

Syntax

```
db::generateRule [-design <oaDesign>] [-lib <oaLib>] [-cg
    <oaConstraintGroup>] [-cg_name <string>] [-output <string>]
    [-genConstDef <string>] [-dumpConstDef <bool>] [-debug <bool>]
```

Arguments

`-design`

Cell view ID.

`-lib`

Library ID.

`-cg`

Constraint group ID.

`-cg_name`
Constraint group name.

`-output`
Output file name.

`-genConstDef`
Generate `tfConstDef.file` as C++ codes (.cpp)

`-dumpConstDef`
Dump system constraint definitions to `dumpConstDef.txt`.

`-debug`
Turn on/off the debug mode.

Returns

`<string>` : generateRule

db::generateSchematic

Description

Generates the schematic representation of a Verilog cellview.

Syntax

```
db::generateSchematic [<cellView: dmCellView | dmCellViewName>]  
-src <dmCellView | dmCellViewName | oaDesign | {<dmCellView>}>  
[-libName <string>] [-viewName <string>] [-overwrite <enum>]  
[-connectByName <enum>] [-netNames <string>] [-symbolViewList <string>]  
[-generateSymbols <bool>] [-symbolOptionSet <string>] [-labelAllSegments  
<bool>] [-createPortOrder <bool>] [-ignoreInsts <string>] [-makeFormGate  
<bool>] [-mergeSubPorts <enum>] [-autoCorrectIOPorts <bool>]  
[-viewSearchList <string>] [-viewStopList <string>] [-deviceMappingFile  
<string>] [-labelAllWires <bool>] [-mergeInstBits <enum>]  
[-createVirtualTerms <enum>] [-bitFormat <string>] [-byNameNetsPattern  
<enum>] [-byNameNetsCaseSensitive <bool>]
```

Arguments

`cellView`
Destination cellview. If not specified, the schematic design is created in the source cell.

`-src`
Source cellview.

`-libName`

Destination library name.

`-viewName`

Destination view name.

`-overwrite`

Controls the overwrite of schematics and symbols if targets exist. ({ false | schematicsOnly | true }, default is: false)

`-connectByName`

Indicates whether the connection is created by name or physically. ({ all | none | selected }, default is: selected)

`-netNames`

Specifies nets for name connection. (Default is:)

`-symbolViewList`

List of views for instances placed masters. (Default is: symbol)

`-generateSymbols`

Indicates whether symbol views must be generated if they are absent for instances placed masters. (Default is: true)

`-symbolOptionSet`

Options set for symbol generation. (Default is: default).

`-labelAllSegments`

Creates wire name labels on all wire segments. (Default is: false)

`-createPortOrder`

Indicates whether portOrder property should be set on generated schematics. (Default is: true)

`-ignoreInsts`

Ignore instances with regular expression, for example: `.*(Dummy).*`, which is to ignore instances with `Dummy`. (Default is:)

`-makeFormGate`

Wraps the recognized part of schematic into one additional level of hierarchy. The default is false.

`-mergeSubPorts`

Merges related bit terminals into busTerm in a specified order. ({ 0 | 1 | asc | desc | false | off | true }, default is: 0)

`-autoCorrectIOPorts`

Automatically corrects directions of I/O ports. (Default is: true)

`-viewSearchList`

The view search list to use while traversing the source hierarchy. By default, the tool does not traverse the hierarchy. (Default is: <unbound>)

`-viewStopList`

The view stop list to use while traversing the source hierarchy. (Default is:)

`-deviceMappingFile`

The devices mapping file to use for device resolution. If not specified, the default \$SYNOPSIS_CUSTOM_INSTALL/samples/device_map.xml file is taken.

`-labelAllWires`

Creates wire labels on all wire segments. (Default is: false)

`-mergeInstBits`

Merges related vector bit instances into a vector instance in a specified order. ({ asc | desc | off }, default is: off)

`-createVirtualTerms`

Controls the terminals to create for a corresponding virtual terminal found in the source. ({ asPhysicals | asPhysicalsWithConnectDef | asVirtuals }, default is: asVirtuals)

`-bitFormat`

The bus bit format used in the source cellview. For Verilog cellviews, the value is ignored. (Default is: [d])

`-byNameNetsPattern`

Pattern type to use for "byName" net names matching. ({ regexp | wildcard }, default is: wildcard)

`-byNameNetsCaseSensitive`

Case sensitivity to use while matching net names for "byName" connection. (Default is: true)

Returns

dmCellView | {<dmCellView>} : Generated cell view(s)

Examples

The following example generates the schematic cellview from provided netlist text cellview.

```
# Set device mapping file to one located in SYNOPSIS_CUSTOM_INSTALL
set deviceMapFile [file join $::env(SYNOPSIS_CUSTOM_INSTALL) \
samples device_map.xml]

# Import amds_lib_ldo spice netlist into DesignLib library
db::importText ./bias.cir -libName DesignLib -viewType SPICEText \
-viewName hspiceD -overwrite always -generateSymbols default \
-overwriteSymbols always -deviceMappingFile $deviceMapFile

# Get source text cellview
set sourceCellView [dm::getCellViews hspiceD -cellName bias \
-libName DesignLib]

# Generate schematic cellview from imported text cellview
db::generateSchematic -src $sourceCellView -autoCorrectIOPorts true \
-deviceMappingFile $deviceMapFile -generateSymbols true \
-labelAllSegments true -labelAllWires true

# Open the generated bias schematic design
de::open DesignLib/bias/schematic
```

db::generateSPICE

Description

Generates spice cellview.

Syntax

```
db::generateSPICE -interface <dbInterfaceSet> -dest <dmCellView |
dmCellViewName> [-options <dbGenOptionSet>]
```

Arguments

interface

The interface set object.

-dest

Destination cellview.

-options

Object containing the option values.

Examples

```
set cellViewSource [dm::findCellView symbol -cellName amp\
-libName pll_work]
```

```
set cell [dm::createCell SpiceFromCell -libName "pll_tb"]
set cellViewDest [dm::createCellView spice -cell $cell\
    -viewType "SPICEText"]
set interfaceSet [db::getCellViewInterface -src $cellViewSource]
db::generateSPICE -interface $interfaceSet -dest $cellViewDest
```

In this example, `db::generateSPICE` generates a `SPICEText` view from the given interface set.

db::generateSystemVerilog

Description

Generates systemVerilog cellview.

Syntax

```
db::generateSystemVerilog -interface <dbInterfaceSet> -dest <dmCellView |
    dmCellViewName> [-options <dbGenOptionSet>]
```

Arguments

`-interface`

The interface set object.

`-dest`

Destination cellView.

`-options`

Object containing the option values.

Examples

```
# Select a source cell which contains SystemVerilog interfaceSet
set cellViewSource [dm::findCellView symbol -cellName amp -libName PLL]

# Get Interface set from the source cell
set interfaceSet [db::getCellViewInterface -src $cellViewSource]

# Generate VerilogAMS cell view
db::generateSystemVerilog -interface $interfaceSet -dest PLL/amp/verilogams
```

db::generateVerilog

Description

Generates a Verilog cellView containing a module template from an interface specification.

Syntax

```
db::generateVerilog -interface <dbInterfaceSet> -dest <dmCellView |  
dmCellViewName> [-options <dbGenOptionSet>]
```

Arguments

-interface

The interface set object.

-dest

The destination cellView.

-options

The object containing the option values.

Examples

```
set cellViewSource [dm::findCellView symbol -cellName amp\  
-libName pll_work]  
set cell [dm::createCell VerilogFromCell -libName "pll_tb"]  
set cellViewDest [dm::createCellView verilog -cell $cell\  
-viewType "verilogText"]  
set interfaceSet [db::getCellViewInterface -src $cellViewSource]  
db::generateVerilog -interface $interfaceSet -dest $cellViewDest
```

In this example, `db::generateVerilog` generates a `verilogText` view from the given interface set.

db::generateVerilogA

Description

Generates a *veriloga* cellView.

Syntax

```
db::generateVerilogA -interface <dbInterfaceSet> -dest <dmCellView |  
dmCellViewName> [-options <dbGenOptionSet>]
```

Arguments

-interface

The interface set object.

-dest

The destination cellview.

-options

The object containing the option values.

Returns

<none>

Examples

```
set cellViewSource [dm::findCellView symbol -cellName amp\  
-libName pll_work]  
set cell [dm::createCell VerilogAFromCell -libName "pll_tb"]  
set cellViewDest [dm::createCellView verilogA -cell $cell\  
-viewType "verilogAText"]  
set interfaceSet [db::getCellViewInterface -src $cellViewSource]  
db::generateVerilog -interface $interfaceSet -dest $cellViewDest
```

In this example, `db::generateVerilog` generates a `verilogAText` view from the given interface set.

db::generateVerilogAMS

Description

Generate verilogAMS cellview.

Syntax

```
db::generateVerilogAMS -interface <dbInterfaceSet> -dest <dmCellView |  
dmCellViewName> [-options <dbGenOptionSet>]
```

Arguments

-interface

The interface set object.

-dest

The destination cellview.

-options

The object containing the option values.

Examples

```
# Select a source cell which contains VeriloAMS interfaceSet  
set cellViewSource [dm::findCellView symbol -cellName amp -libName PLL]  
  
# Get Interface set from the source cell  
set interfaceSet [db::getCellViewInterface -src $cellViewSource]
```



```
# Generate VerilogAMS cell view
db::generateVerilogAMS -interface $interfaceSet -dest PLL/amp/verilogams
```

db::generateVHDL

Description

Generates a VHDL cellview.

Syntax

```
db::generateVHDL -interface <dbInterfaceSet> -dest <dmCellView |
dmCellViewName> [-options <dbGenOptionSet>]
```

Arguments

-interface

The interface set object.

-dest

The destination cellview.

-options

The object containing the option values.

Examples

```
# Select a source cell which contains VHDL interfaceSet
set cellViewSource [dm::findCellView schematic -cellName amp -libName PLL]

# Get Interface set from the source cell
set interfaceSet [db::getCellViewInterface -src $cellViewSource]

# Generate VHDL cell view
db::generateVHDL -interface $interfaceSet -dest PLL/amp/vhdl
```

db::getAnnotationSets

Description

Gets annotation sets.

Syntax

```
db::getAnnotationSets
```

Returns

[<string>]: Collection of registered annotation set names.

db::getAssignmentDefs

Description

Retrieves the assignment definitions found in the specified instance from the given context.

Syntax

```
db::getAssignmentDefs -of <dbContextConnectionDef> -instName <string>
```

Arguments

-of

Context connection definition.

-instName

Instance name in the given context.

Returns

A collection of assignment definitions found in the given instance hierarchy. ([dbAssignmentDef](#)).

Errors

- Invalid argument.
- Instance given not in context.

db::getAssignments

Description

Returns a dbCollection containing all the assignments having the given name for a specific design or instance. If a name is not supplied, all assignments for the given design or instance will be returned. The collection could be empty or contain dbAssignmentDef objects.

Syntax

```
db::getAssignments [<none: string>] -of <oaDesign | oaInst> [-filter  
<string>]
```

Arguments

none: <string>

A glob-style pattern to match assignment name against.

`-of`

Design or instance from which to find assignments.

`-filter`

Standard attribute filter.

Returns

dbCollection of design or instance assignments. ([dbAssignmentDef](#)).

Errors

Invalid argument.

Examples

```
# Find assignment definitions for all instances with connection
# assignments and print instance name and assignment name
set context [de::getActiveContext]
set h [db::getAttr context.hierarchy]
set design [db::getAttr context.editDesign]
set ctxConnections [db::findContextConnections $h -of
[db::findConnections $h]]
foreach instName [db::getAttr ctxConnections.instances] {
    set inst [db::getInsts $instName -of $design]
    db::foreach assignmentDef [db::getAssignments -from $inst] {
        puts "      instName: [db::getAttr
assignmentDef.assignment.instance.name] name: [db::getAttr
assignmentDef.assignment.name]"
    }
}
```

db::getAttr

Description

Returns the value of the specified attribute of the given object.

`db::getAttr` can access both PCell subMaster and additional MCell guard ring and MOS subMaster attributes.

- Additional MCell guard ring attributes include general guard ring attributes, geometry attribute name, physical geometry, and physical geometry Id.
- Additional MCell MOS attributes include MOS attributes (including gate/active regions and polyContact or contact information), logic information within gate/active regions, geometry attribute name, physical geometry, and physical geometry Id.

Syntax

```
db::getAttr [<name: dbVarAttrName>] [-of <dbCollection | dbIterator>]  
[-type <enum>][-orient <oaOrient>]
```

Arguments

`name`

The attribute name or hierarchical sequence of names separated by periods.

`-of`

The objects for which the attribute values are obtained.

`-type`

Filters attributes to only those of the specified type: {all | appDef | intrinsic | param | prop}.

Default: all.

`-orient`

Orientation of the region. Valid values: R0, R180, MX, and MY. For an existing track region already created in a design its orient is R0.

Returns

(<variant>): The attribute value(s), possibly nested collections.

Errors

- The object does not have the given attribute.
- The object type is not supported.

Examples

```
set cellName [db::getAttr cellName -of $dmCellView]  
set shapeBbox [db::getAttr bBox -of $oaShape]
```

In this example, `db::getAttr` gets the `cellName` and `bBox` attributes and applies their attribute values to the variables `cellName` and `shapeBbox`.

The above example can be rewritten as:

```
db::getAttr dmCellView.cellName  
db::getAttr oaShape.bBox
```

The way this works is, the `db::getAttr` command's first argument will autocomplete against the defined Tcl variables.

For example:

```
set myDesign [ed]
puts [db::getAttr myDesign.cellName]
```

You can also do:

```
db::getAttr my<tab>
```

And it will give:

```
db::getAttr myDesign.
```

The insertion cursor will be ready to accept an attribute name of the design.

The following example gets a list of segments of centerline.

```
db::getAttr BodySeg -of $design
```

The following example gets a list of glue blocks of OD length.

```
db::getAttr ODLenght -of $design
```

db::getBuiltinNetlists

Description

Command to return built-in analog block component netlists.

Syntax

```
db::getBuiltinNetlists [<glob: string>] [-filter <string>]
```

Arguments

glob

Glob-style name filter. (Default is: *)

-filter

Standard filter expression. (Default is:)

Returns

<none>

Examples

```
# Get builtin analog block component netlists and print their names
set netlists [db::getBuiltinNetlists]
puts [db::createList [db::getAttr netlists.name]]
```

db::getCallbacks

Description

Gets a filtered collection of callbacks based on the matches defined.

Syntax

```
db::getCallbacks [<pattern: string>] [-filter <string>] [-callbackType  
  <string>] [-viewType <string>]
```

Arguments

<string>

Pattern to match callback names.

Default: *.

-filter

Standard attribute filter.

Default: match all.

-callbackType

Callback type for which to return the registered callbacks. The default value, the empty string (""), indicates that callbacks of all types should be returned.

Default: "".

-viewType

View type name, if applicable, for which to return the registered callbacks. The default value, the empty string (""), indicates that callbacks for all view types should be returned.

Default: "".

Returns

Filtered list of callbacks ([dbCallback](#)).

Errors

Undefined callbackType.

Examples

```
db::getCallbacks -callbackType onFirstDesignOpened
```

In this example, `db::getCallbacks` returns all registered callbacks for the `onFirstDesignOpened` callback type.

db::getCellViewInterface

Description

Creates an interface set object from the source cellView. This command is registered as the default interface reader for all OA database views.

Syntax

```
db::getCellViewInterface -src <dmCellView | dmCellViewName>  
[-ignoreParams <bool>] [-processPortOrder <bool>]
```

Arguments

-src

CellView from which to read interface information.

-ignoreParams

If true, parameter interface information will not be read from the source cellView.

The default is true.

-processPortOrder

If true - the port list in text-type cellviews will be generated according to portOrder list of source cell view. The default is false.

Returns

<dbInterfaceSet>: Interface set information from the specified cellView ([dbInterfaceSet](#)).

Errors

Invalid cellView is given.

Examples

```
# Select a source cell which contains SystemVerilog interfaceSet  
set cellViewSource [dm::findCellView symbol -cellName amp -libName PLL]  
  
# Get Interface set from the source cell  
set interfaceSet [db::getCellViewInterface -src $cellViewSource]  
  
# Generate VerilogAMS cell view  
db::generateSystemVerilog -interface $interfaceSet -dest PLL/amp/verilogams
```

db::getCircularRulerDefs

Description

Returns a collection of circular ruler definitions.

Syntax

```
db::getCircularRulerDefs [<none: string>] [-filter <string>]
```

Arguments

<none>

Pattern to match definition name.

-filter

Filter expression.

Returns

{<dbCircularRulerDef>}: Collection of ruler definition objects.

Examples

```
# Create a sub-ruler object
set subCR [db::createSubCircularRuler -radius 2.4 -label 2.4 -packet
ruler1]

# Create a circular ruler definition
db::createCircularRulerDef CRDefName -subRulers $subCR

# Create a circular ruler with CRDefName definition
db::createCircularRuler {0 0} -design [ed] -rulerDef CRDefName

# Get circular ruler definitions and print their names
set rulerDefs [db::getCircularRulerDefs]
puts [db::createList [db::getAttr rulerDefs.name]]
```

db::getComponents

Description

Retrieves a collection of components for the given object. The returned value 'dbComponent' of `db::getComponents` is one of the pair(s) that match the filter given on the command line. If no filter is given, the whole value of the cell's `lxComponentType` parameter will be returned.

Syntax

```
db::getComponents -of <dmCell | oaDesign | oaLib | oaRef | oaTech>
[-filter <string>]
```

Arguments

-of

Collection of dbComponent objects.

`-filter`

Standard filter expression.

Returns

`<dbCollection>` : Collection of `dbComponent` objects.

Examples

```
set cs [db::getComponents -of $oaLib]
db::createList [db::getAttr cs.mosType]
```

db::getConnectionDefState

Description

Retrieves the assignment state of the switched master referenced by the given instance name. The instance name given must exist in the context described by the `dbContextConnectionDef` object. The state will indicate if the give connection definition is fully, partially, or not assigned at that level.

Syntax

```
db::getConnectionDefState -of <dbContextConnectionDef> -instName <string>
```

Arguments

`-of` `dbContextConnectionDef`

Context connection definition.

`-instName` `<string>`

Instance name in the given context.

Returns

The assign state of the switched master of the given instance.

Errors

- Invalid argument.
- Instance given not in context.

Examples

```
# Find connection definitions with name "power" from specified context
# and print its state in I0 instance master
set context [de::getActiveContext]
set h [db::getAttr context.hierarchy]
set design [db::getAttr context.editDesign]
```

```
set ctxConnections [db::findContextConnections $h -of  
    [db::findConnections $h]]  
set contextDef [db::findContextConnectionDef power -of $ctxConnections]  
puts [db::getConnectionDefState -of $contextDef -instName I0]
```

db::getContextConnectionDefs

Description

Returns a collection of dbContextConnectionDef objects which exist in the switched master associated with the given instance name.

Syntax

```
db::getContextConnectionDefs [<none: string>] -of <dbContextConnections>  
    -instName <string> [-filter <string>]
```

Arguments

<string>

Glob-style pattern for matching parameter name.

-of

The connections context.

-instName

Name of instance in the given context.

-filter

Standard attribute filter.

Default: Match all.

Returns

Collection of connect definition objects relative to the given instance.
([dbContextConnectionDef](#)).

Errors

- Invalid argument.
- Instance given not in context.

Examples

```
# Find connection definitions from specified context and print their  
names, default net and source type  
set context [de::getActiveContext]  
set h [db::getAttr context.hierarchy]
```

```
set design [db::getAttr context.editDesign]
set ctxConnections [db::findContextConnections $h -of
[db::findConnections $h]]
foreach instName [db::getAttr ctxConnections.instances] {
    puts "instName = $instName"
    db::foreach ctxConnectDef [db::getContextConnectionDefs -from
    $ctxConnections -instName $instName] {
        set connectDef [db::getAttr ctxConnectDef.connectionDef]
        puts "        name: [db::getAttr connectDef.name] defaultNet:
[db::getAttr connectDef.defaultNet] sourceType: [db::getAttr
connectDef.sourceType]"
    }
}
```

db::getCount

Description

Returns the number of objects in the specified collection.

Syntax

```
db::getCount <collection: dbCollection>
```

Arguments

collection

An iterator over the collection.

Returns

<unsigned int>: The number of objects in the specified collection.

Errors

Invalid object type.

Examples

In these two examples, `db::getCount` gets a count of all vias. You can use this example to find the number of vias in a design.

```
set viaCollection [db::getVias -of $oaDesign]
set viaCount [db::getCount $viaCollection]
```

db::getCurrentContext

Description

Returns the currently active context. This method should only be called from within parameter change callbacks or interpreted label evaluation.

Syntax

```
db::getCurrentContext
```

Examples

```
# Create label evaluation procedure that changes label color based on the
# evaluated value
proc displayParam {paramName} {
    set inst [db::getCurrentRef]
    set label [db::getCurrentEvalText]
    set context [db::getCurrentContext]
    set paramValue [db::getParamValue $paramName -of $inst -evalType full
-context $context]
    if {$paramValue > 10} {
        db::setAttr label.color -value red
    } else {
        db::setAttr label.color -value green
    }
    return "$paramName = $paramValue"
}

# Register displayParam procedure as an IL function
se::registerLabelFunction "Display Parameter" -evalType IL -expr
displayParam(%param%) -layer 237 -purpose -1
```

db::getCurrentEvalText

Description

Returns the `oaEvalText` object currently being rendered or queried. This Tcl should only be called from within IL (Interpreted Label) evaluation.

Syntax

```
db::getCurrentEvalText
```

Returns

The currently rendered or queried `oaEvalText` object. (*oaEvalText*).

Errors

If the proc is called outside of evaluation of an interpreted label.

Examples

```
# Create label evaluation procedure that changes label color based on the
# evaluated value
proc displayParam {paramName} {
    set inst [db::getCurrentRef]
    set label [db::getCurrentEvalText]
    set context [db::getCurrentContext]
```

```
        set paramValue [db::getParamValue $paramName -of $inst -evalType full
-context $context]
        if {$paramValue > 10} {
            db::setAttr label.color -value red
        } else {
            db::setAttr label.color -value green
        }
        return "$paramName = $paramValue"
    }

# Register displayParam procedure as an IL function
se::registerLabelFunction "Display Parameter" -evalType IL -expr
    displayParam(%param%) -layer 237 -purpose -1
```

db::getCurrentParam

Description

Returns the parameter name involved in parameter change callbacks or attribute evaluation. This Tcl should only be called from within parameter attributes scripts.

Syntax

```
db::getCurrentParam
```

Returns

The name of the parameter. (<string>).

Errors

If the proc is called outside of a parameter change callback or dbParam attribute evaluation.

Examples

```
# Create parameter change callback procedure
proc paramCallback {} {
    set inst [db::getCurrentRef]
    set paramValue [db::getParamValue [db::getCurrentParam] -of $inst]
    db::setParamValue param2 -of $inst -value [expr $paramValue * 2]
}

# Add two new parameters to the testLib/nmos cell
set nmos [dm::findCell nmos -libName testLib]
db::createParamDef param1 -data $nmos -callback paramCallback
db::createParamDef param2 -data $nmos

# Change value of param1 of M0 instance of the nmos cell to 10
db::setParamValue param1 -of [db::getInsts M0 -of [ed]] -value 10
```

```
# Get value of param2, it should be 20
puts [db::getParamValue param2 -of [db::getInsts M0 -of [ed]]]
```

db::getCurrentRef

Description

This command should only be called from within scripts executed by either IL evaluation or the parameter subsystem. If executed from within dbParam attribute evaluation, returns the parameter's reference (instance/via). If executed from within IL evaluation, returns the reference associated with the label. If not called from within IL evaluation or from the parameter subsystem, generates an error.

Syntax

```
db::getCurrentRef
```

Returns

The reference associated with the current IL label or dbParam evaluation (*oaRef*).

Errors

If the proc is called outside of the context of either: parameter subsystem or IL label evaluation.

Examples

```
# Create parameter change callback procedure
proc paramCallback {} {
    set inst [db::getCurrentRef]
    set paramValue [db::getParamValue [db::getCurrentParam] -of $inst]
    db::setParamValue param2 -of $inst -value [expr $paramValue * 2]
}

# Add two new parameters to the testLib/nmos cell
set nmos [dm::findCell nmos -libName testLib]
db::createParamDef param1 -data $nmos -callback paramCallback
db::createParamDef param2 -data $nmos

# Change value of param1 of M0 instance of the nmos cell to 10
db::setParamValue param1 -of [db::getInsts M0 -of [ed]] -value 10

# Get value of param2, it should be 20
puts [db::getParamValue param2 -of [db::getInsts M0 -of [ed]]]
```

db::getCutClasses

Description

Returns names of cutClasses from specified tech.

Syntax

```
db::getCutClasses -of <oaTech>
```

Arguments

of

The tech to get cut classes from.

Returns

[<string>] : Names of cut classes.

db::getDeviceDisplays

Description

Retrieves settings from the dbDeviceDisplay objects associated with a given instance, master, library or design.

Syntax

```
db::getDeviceDisplays [-window <int>] [-filter <string>]
```

Arguments

-window

Window ID.

-filter <string>

Specifies a string expression to further filter the returned collection.

Default: None.

Returns

Objects defining state of device label displays ([dbDeviceDisplay](#)).

Examples

```
# Get master of I0 instance
set master [db::getAttr master -of [db::getInsts I0 -of [ed]]]

# Create device display to show model name and hide l and w parameters
```

```
db::setDeviceDisplay -of $master -params {model(parameter) w(none)
l(none)}

# Create device display to show net names on instance terminals
db::setDeviceDisplay -of $master -terms net

# Restore all labels by destroying device display
db::destroy [db::getDeviceDisplays]
```

db::getEffectiveNet

Description

Returns a context where the effective net is found, and the effective net name.

Syntax

```
db::getEffectiveNet -of <oaBitNet | oaBitTerm | string>
                    -context <heContext>
```

Arguments

-of

Object for which to search for the effective net.

-context

Hierarchy context where the source object is located.

Returns

<none>

Examples

The following example gets hierarchical context of design.

```
set context [de::getActiveContext]
set heContext [db::getAttr context.hierarchy]
db::getEffectiveNet -context $he -of vdd!
```

db::getEnums

Description

Returns a collection of registered dbEnum objects, optionally filtered by name.

Syntax

```
db::getEnums [<pattern: string>] [-filter <string>]
```


Arguments

pattern

Optional wildcard pattern to match against the name of returned enums. (Default is: *)

-filter

Standard attribute filter. (Default is:)

Returns

<dbCollection> : Collection of dbEnum objects.

Examples

```
# Create a new enumeration type
db::createEnum resEnumType -values {"HiZ" "Pull-Up" "Pull-Down"}

# Get the enumeration type with its name
set en [db::getEnums resEnumType]

# The enumeration type can be used for preference creation
db::createPref insertRestorType -type resEnumType \
    -value [lindex [db::getattr en.values] 0]

# Set preference value and check auto values with Tab key.
db::setPrefValue insertRestorType -value Pull-Down
```

db::getEquivalentNets

Description

Retrieves hierarchically equivalent nets.

Syntax

```
db::getEquivalentNets <bitNet: oaBitNet | string> [-direction <enum> |
    -signalDirection <enum>] -context <heContext> [-includeVirtualTerminals
    <bool>] [-includeSymbolPins <bool>] [-traceProc <variant>] [-sortByDepth
    <bool>] [-depth <int>]
```

Arguments

bitNet

The oaBitNet or net name to trace.

-context

The queried context to determine child context.

-direction

The hierarchy directions to trace the net: {both | down | up}. (Default is: both)

Default: both

`-signalDirection`

The signal direction to trace the net: {backward | both | forward}. (Default is: both)

Default: both

`-includeVirtualTerminals`

Include the nets connected by virtual connections. (Default is: true)

Default: true.

`-includeSymbolPins`

Include connections to symbol pins in results. (Default is: true)

Default: true.

`-traceProc`

Callback for computing additional nets to trace. The specified proc is called for each `oaInstTerm` on the traced net. The proc should return a list of additional `oaBitNets` to trace. Signature: `<list of oaBitNets> traceProc <heContext> <oaInstTerm>`. The available example is `de::probeTraceThroughLvsIgnore`.

`-sortByDepth`

If true, sorts results by hierarchy depth, and sort contexts with the same depth alphanumerically. (Default is: false)

Default: false.

`-depth`

Maximum levels to search bellow the given context. Negative value searches all levels. (Default is: -1)

Returns

The list which has the collection for instances and terminals to which the net is connected.

Examples

```
proc traceProc {context object} {  
    set result {}  
    if {"InstTerm"==[db::getAttr object.type]} {  
        set inst [db::getAttr object.inst]  
        if {"analogLib"==[db::getAttr inst.libName] &&  
"res"==[db::getAttr inst.cellName]} {  
            if {"MINUS"==[db::getAttr object.name]} {  
                set it [db::getNext [db::getInstTerms PLUS -of $inst]]
```

```
                if {"!"=$it} {
                    lappend result [db::getAttr it.net]
                }
            }
        }
    }
    return $result
}

# Get hierarchically equivalent nets including nets connected to through
# analogLib/res device
set hierarchy [db::getAttr hierarchy -of [de::getActiveContext]]
set netName pbias

foreach item [db::getEquivalentNets $netName -direction both -context
    $hierarchy -traceProc traceProc -includeSymbolPins false] {
    set context [lindex $item 0]
    set occurrence [db::getAttr context.occurrence]
    puts "Occurrence: $occurrence"
    foreach net [lrange $item 1 end] {
        puts "    Net: [db::getAttr net.name]"
    }
}
```

db::getEvaluators

Description

Gets evaluators.

Syntax

```
db::getEvaluators [-annotationSet <string>] [-filter <string>]
```

Arguments

-annotationSet

Name of annotation set object the evaluators should return.

-filter

Specifies a string/expression to further filter the returned collection.

Returns

{<dbLabelEvaluator>}: Collection of dbLabelEvaluator objects.

Examples

```
# Print names of the all label evaluators
set evs [db::getEvaluators]
puts [db::createList [db::getAttr evs.name]]
```

db::getExportOasisOptions

Description

Gets ExportOasis options.

Syntax

```
db::getExportOasisOptions
```

Returns

<string> : Get exportOasis command.

db::getExportStreamOptions

Description

Gets ExportStream options.

Syntax

```
db::getExportStreamOptions [-tclOpt <bool>]
```

Arguments

-tclOpt

Get db::exportStream options.

Returns

<string> : Get exportStream options.

db::getFeatures

Description

Returns a Tcl list of all registered application features. Checking out an application feature may correspond to an existence check or checkout of one or more lm features on the license server. Features returned may or may not be checked out.

Note that the returned license names are internally defined and may refer to one or more features on the actual license server.

Syntax

```
db::getFeatures
```

Returns

List of names of all registered features ([<string>]).

db::getFigGroupMembers

Description

Returns a collection containing of figure group member (figGroupMem) in a figure group (figGroup). Each figGroupMem references one figure (fig) that belongs to the figGroup.

Syntax

```
db::getFigGroupMembers -group <deFigure | oaFigGroup | string> [-design  
    <oaDesign>]
```

Arguments

-group

The figGroup from which to get members.

-design

The design in which the group belongs.

Returns

{<oaFig>} : Collection of fig group members.

Examples

The following example creates a figure group object from the selection and gets a member object. The last command returns object types.

```
set design [db::getAttr editDesign -of [de::getActiveContext]]
set figGroup [db::createFigGroup objGroup -members [de::getSelected  
    -design $design] -design $design ]
set mems [db::getFigGroupMembers -group $figGroup]
db::createList [db::getAttr mems.type]
```

db::getFigGroups

Description

Returns a collection of fig groups.

Syntax

```
db::getFigGroups [<pattern: string>] -of <oaDesign> [-filter <string>]
```

Arguments

pattern

Pattern to match object name.

-of

Design to fetch the groups from.

-filter

Filter expression.

Returns

{<oaFigGroup>} : Collection of fig groups

Examples

The following example gets and returns a collection of figure groups from the design.

```
# Get collection of figure groups from specified design
set figGroups [db::getFigGroups -of $design]

# Count number of figure groups and get their names
db::getCount $figGroups
db::foreach figGroup $figGroups {puts [db::getattr figGroup.name]}
```

db::getFlightlineCoordinates

Description

Get flight line coordinates.

Syntax

```
db::getFlightlineCoordinates <net: oaNet>
```

Arguments

net

Net name.

Returns

{<oaPoint>} : List of point pairs.

Examples

The following example gets flight line coordinates for a specified net.

```
# Get net from design
set net [db::getNets CLK -of $design]
```

```
# Get flightline coordinates. This will print coordinates into Console
db::getFlightlineCoordinates $net
```

db::getGenOptionSets

Description

Returns a dbCollection of all currently known option sets of the specified type, which matches the optional option name pattern and filter specification.

Syntax

```
db::getGenOptionSets [<name: string>] [-filter <string>]
```

Arguments

<string>

Glob-style pattern to match with an existing object set name.

-filter <string>

Standard attribute filter.

Default: Match all.

Returns

Collection of dbGenOptionSet objects. The ordering of the objects in the list is undefined ({dbGenOptionSet}).

Errors

None.

Examples

This example gets the Abstract Generator option set, *m1Set*, and then assigns it to variable *m1OptionSet* for further use.

```
set m1OptionSet [db::getGenOptionSets m1Set]
```

db::getGroupMembers

Description

Returns a collection of member figures.

Syntax

```
db::getGroupMembers -of [<oaGroup>] [-types <enum>]
```

Arguments

`-of`

The group to get members for.

`-types`

Specifies the type of members to return from the group. ({ all | exceptLeader | leader })

Returns

{<oaFig>} : Collection of oaFig.

Examples

The following example gets a leader member from the group object in a loop and prints an object type.

```
set design [db::getAttr editDesign -of [de::getActiveContext]]
set groups [db::getGroups -of $design ]
db::foreach group $groups {
    set leader [db::getGroupMembers -from $group -types leader]
    puts [db::getAttr leader.type]
}
```

db::getGroups

Description

Returns a collection of groups.

Syntax

```
db::getGroups [<pattern: string>] -of <oaDesign> [-groupDef <oaGroupDef>]
[-filter <string>]
```

Arguments

`pattern`

Pattern to match object name.

`-of`

Design to fetch the groups from.

`-groupDef`

Groups in the design that are associated with the specified group definition.

`-filter`

Filter expression.

Returns

{<oaGroup>}: Collection of groups.

Examples

The following example gets and returns a collection of groups from the design.

```
# Get collection of groups from specified design
set groups [db::getGroups -of $design]

# Count number of groups and get their names
db::getCount $groups
db::foreach group $groups {puts [db::getattr group.name]}
```

db::getICCCellViews

Description

Retrieves mapped ICC cellViews for the specified OA cellViews.

Syntax

```
db::getICCCellViews <cellViews: {<dmCellView>}> [-defaultLibPath
<string>]
```

Arguments

cellViews

CellViews to export.

-defaultLibPath

The default ICC library path.

Returns

<{<dmCellView>}>: CellViews being exported.

db::getIndex

Description

Returns the nth item in the specified collection. When supported, this command returns in constant or logarithmic time, although many collection types are not supported by this command. However, it is possible to copy an unsupported collection into a supported one using the `db::createCollection` or `db::sort` commands.

Syntax

```
db::getIndex <int> -of {<oaObject | dbObject>}
```

Arguments

`int`

Index to retrieve.

`-of`

Collection of objects.

Returns

`<oaObject | dbObject>`

Errors

Specified collection does not support `db::getIndex`. Use `db::createCollection` to copy it to a supported collection.

Examples

This example gets a collection of nets with net names starting with "c", and then gets the last net in the collection and prints its net name.

```
set nets [db::getNets c* -of [ed]]
# get collection of nets with net names starting with "c"

set net [db::getIndex [expr [db::getCount $nets]-1] -of $nets]
# get the last net in the collection

puts [db::getattr net.name]
# Print name of the retrieved net cpu|alu|n1877
```

db::getInstHeaders

Description

Returns a collection of `oaInstHeader` objects.

Syntax

```
db::getInstHeaders [<dmCellViewName: dbCellViewName>] -of <oaDesign>
[-filter <string>]
```

Arguments

`dmCellViewName`

Cellview name from which `instHeaders` will be returned

`-of`

`oaDesign` to search.

`-filter`

Standard attribute filter.

Returns

{<oaInstHeader>} : Collection of instHeaders

Examples

The following example gets instance header for DemoPLL/and2/layout design.

```
db::getInstHeaders DemoPLL/and2/layout -of $design
```

db::getInsts

Description

Returns a collection of instances. Returns an empty collection if no matching instances are found.

Syntax

```
db::getInsts [<pattern: string>] -of <oaDesign | {<oaInstHeader>} |  
{<oaInstTerm>} | {<oaPin>} | {oaInstHeader}> [-filter <string>]
```

Arguments

<string>

Pattern to match instance name.

Default: Match all instances.

`-of oaDesign | oaPin | {oaInstHeader}`

A design, pin, or collection of instHeaders from which to get the instances.

`-filter <string>`

Filter expression.

Default: no filter.

Returns

{<oaFig>}: Collection of instances.

Errors

- Wrong design or design is not open.
- Incorrect oaPin or oaInstHeader.
- Filter error.

Examples

The following example gets and returns the collection of all instance objects in the design.

```
# Get collection of instances from specified design
set insts [db::getInsts -of $design]

# Count number of instances and get their names
db::getCount $insts
db::foreach inst $insts {puts [db::getattr inst.name]}
```

db::getInstTerms

Description

Returns a collection of instance terminals. Returns an empty collection if no matching instance terminals are found.

Syntax

```
db::getInstTerms [<pattern: oaTerm | string>] [-includeImplicit <bool> |  
-singleBit <bool>] -of <oaDesign | {<oaFig>} | {<oaInst>} | {<oaNet>}>  
[-filter <string>]
```

Arguments

pattern

Pattern to match instTerm name or oaTerm. (Default is:)

-of

Design or objects from which to collect instance terminals.

-includeImplicit

Specifies whether to include implicit nets in the return collection. (Default is: false)

-singleBit

Specifies whether to return single bit nets. (Default is: false)

-filter

Filter expression. (Default is:)

Returns

{<oaInstTerm>} : Collection of instance terminals.

Errors

- Wrong design or design is not open.
- Wrong oaNet or oaInst.
- Filter error.

Examples

The following example returns the collection of all instTerm objects (instance terminals) in the design.

```
set instTermCollection [db::getInstTerms -of $oaDesign]
```

The following example returns the collection of instance terminals that are attached to the CLK net in the given oaDesign.

```
db::getInstTerms -of [db::getNets CLK -of $oaDesign]
```

db::getLayers

Description

Returns specified layers from specified tech.

Syntax

```
b::getLayers [<name: string | unsigned int>] (-of <oaTech> | -above  
<oaPhysicalLayer> | -below <oaPhysicalLayer>) [-local <bool>] [-material  
<string>] [-filter <string>]
```

Returns

{<oaLayer>}: Collection of oaLayer

Arguments

name

Glob-style pattern to match against layer name or a number to filter the returned collection.

-of

The technology to get layers from.

-above

Retrieve layers above the specified layer.

db::getLPPHeaders

Description

Returns a collection of oaLPPHeader objects.

Syntax

```
db::getLPPHeaders [<lpp: lpp>] [-depth <int>] -of <oaDesign> [-filter  
  <string>]
```

Arguments

`lpp`

Layer Purpose Pair. The default is `**`.

`-depth`

Depth to search for header containing at least one shape on LPP. The default is `-1`.

`-of`

oaDesign to search.

`-filter`

Standard attribute filter.

Returns

{<oaLPPHeader>} : Collection of lppHeaders

Examples

The following example gets LPP header for ?m1 drawing? LPP.

```
db::getLPPHeaders "m1 drawing" -of $design
```

db::getMarkers

Description

Returns a collection of markers filtered by values you provided.

Syntax

```
db::getMarkers -of <oaDesign> [-filter <string>]
```

Arguments

`-of`

Design in which markers are searched.

`-filter`

Filter expression. The default is: "".

Returns

{<oaMarker>} : Collection of markers.

Examples

```
# Check schematic design with some violations
se::check -design [ed]

# Get markers and print their messages and points
set markers [db::getMarkers -of [ed]]
db::foreach marker $markers {
    puts "[db::getattr marker.msg] [db::getattr marker.points]"
}
```

db::getNetlistInfo

Description

Gets the netlist info for a cell by retrieving values stored in the Netlist Info database.

If the section is `netlist`, the `-netlister` argument is required.

If the section is `display`, the `propList` section is queried. This section of the Netlist Info controls the display of parameters and simulation results within `cdsParam` labels.

If the section is `form`, the top level properties are queried. Typical entries in the form properties are `promptWidth`, `fieldHeight`, and so on. This is provided mostly for data compatibility. Custom Compiler does not use the form properties stored in the Netlist Info.

Syntax

```
db::getNetlistInfo <name: string> -level <enum> -data <dmCell |
    oaCellDMDData | oaLib | oaLibDMDData> [-netlister <string>] [-info
    <string>] [-section <enum>]
```

Arguments

`name`

The name of Netlist Info.

`-data`

The cell or library that contains a DMData where the specified Netlist Info is stored.

`-netlist`

The Netlist name, required only when a section is `netlist`. (Default is:)

`-info`

Name of the subsection, only allowed when section is `netlist`. (Default is: `simInfo`)

`-section`

The section where the specified Netlist Info is stored. ({ `display` | `form` | `netlist` }, default is: `netlist`)

`-level`

The CDF level from which value will be obtained. ({ `base` | `memory` | `overlay` }, default is: `overlay`)

Returns

(<string>): The data stored in the Netlist info for the given netlist/name.

Errors

- If the `-section` argument is `netlist` and the `-netlist` option is not provided.
- If the specified `-data` does not exist.

Examples

```
puts [db::getNetlistInfo termOrder -data [dm::getCells nmos4 \  
-libName analogLib] -netlist hspiceD -section netlist -info simInfo]  
puts [db::getNetlistInfo netlistProcedure -data [dm::getCells nmos4 \  
-libName analogLib] -netlist hspiceD -section netlist -info simInfo]  
puts [db::getNetlistInfo propMapping -data [dm::getCells nmos4 \  
-libName analogLib] -netlist hspiceD -section netlist -info simInfo]  
puts [db::getNetlistInfo termOrder -data [dm::getCells amp \  
-libName PLL] -netlist verilogams -section netlist -info viewInfo]
```

db::getNets

Description

Returns a collection of nets. If no matching nets are found, returns an empty collection.

Syntax

```
db::getNets [<pattern: string>] [-includeImplicit <bool> | -singleBit  
<bool>] -of <oaDesign | {<oaConnFig>} | {<oaInstTerm>} | {<oaPin>} |  
{<oaTerm>}> [-equivNets <bool>] [-filter <string>]
```

Arguments

`pattern`

Pattern to match object name. (Default is:)

`-of`

Design or objects from which to collect nets.

`-includeImplicit`

Includes implicit nets in the return collection. (Default is: false)

`-equivNets`

Specifies whether to return nets that are logical equivalents in the database.
(Default is: false)

`-singleBit`

Specifies whether to return single bit nets. (Default is: false)

`-filter`

Filter expression. (Default is:)

Returns

{<oaNet>}: Collection of nets.

Errors

- Wrong design or design is not open.
- Incorrect filter.

Examples

The following example gets and returns the collection of all nets in the design.

```
set netCollection [db::getNets -of $oaDesign]
```

db::getNext

Description

Returns the next object in a collection or, if the `-type` argument is specified, the next object of the given type. Returns an empty string if there are no more objects in a collection.

Syntax

```
db::getNext <collection: dbIterator>
```

Arguments

`collection`

The collection being iterated.

Returns

<variant>: The next object in the corresponding collection.

Examples

In this example, `db::getNext` works as an increment operator by returning the next object in the `dbCollection`, and then updating its object pointer to point to the next object. Specifically, the first line returns the `giToolbar` object adjacent to the schematic window Standard Toolbar. The second line adds a new action (the next toolbar item) to the object set. After the last object is returned, a null string (" ") is returned.

```
set schToolbar [db::getNext [gi::getToolbars \
    seSchStandardToolbar -from $schWindowType] ]
gi::addActions {giUserAction} -to $schToolbar
```

db::getNotifiers

Description

Gets collection of notifiers supported by an object, optionally filtered by name.

Syntax

```
db::getNotifiers [<pattern: string>] -of <dbCollection> [-filter
    <string>]
```

Arguments

`pattern`

Optional wildcard pattern to match against the name of the returned notifiers.

Default is: `"*"`.

-of

Object to fetch notifiers from (supported by).

-filter

Standard attribute filter.

Returns

<dbCollection> : Collection of notifiers.

Examples

```
# Create an object
set obj [db::createObject ]

# Add attributes to the object
db::addAttr name -of $obj -valueType string -value "nmos"
db::addAttr l -of $obj -valueType double -value 1u
db::addAttr w -of $obj -valueType double -value 0.1u

# Create notifiers for each attribute. Each notifier notifies the object
to
# set its attribyte value
set n1 [db::createNotifier nmosAttrSet_l -to $obj -args string \
-description "NMOS L parameter set notifier"]
set n2 [db::createNotifier nmosAttrSet_w -to $obj -args string \
-description "NMOS W parameter set notifier"]

# Set listeners to the notifiers
db::addListener [list nmosValueSet_l $obj ] -to $n1
db::addListener [list nmosValueSet_w $obj ] -to $n2

# Create procedures which handles object attribute
proc nmosValueSet_l {obj value} {
    puts "This is nmos l attribute set listener proc, the value is $value"
    if {$value >= 0} {
        db::setAttr obj.l -value $value
    }
}

# Create procedures which handles object attribute
proc nmosValueSet_w {obj value} {
    puts "This is nmos w attribute set listener proc, the value $value"
    if {$value >= 0} {
        db::setAttr obj.w -value $value
    }
}

# Get the object notifier
set n [db::getNotifiers nmosAttrSet_l -of $obj]

# Notify the object to set the l attribute
```

```
db::notify $n -args 5u

# Check if the attribute is set
puts [db::getAttr obj.l]

# Get the object notifier
set n [db::getNotifiers nmosAttrSet_w -of $obj]

# Notify the object to set the w attribute
db::notify $n -args 0.5u

# Check if the attribute is set
puts [db::getAttr obj.w]
```

db::getOpenDesigns

Description

Gets the open designs.

Syntax

```
db::getOpenDesigns [<cellViewName: dbCellViewName>] [-type <enum>]
                    [-filter <string>]
```

Arguments

cellViewName

Cellview name for which to get open designs

-type

Design type. ({ all | pcell | regular | scratch | stdVia }). The default is all)

-filter

Standard attribute filter.

Returns

{<oaDesign>} : Opened designs

Examples

The following example gets all opened designs.

```
db::getOpenDesigns
```

The following example gets opened DemoPLL/and2/layout designs.

```
db::getOpenDesigns DemoPLL/and2/layout
```

db::getPaperSizes

Description

Returns a collection of paper sizes matching the specified criteria.

Syntax

```
db::getPaperSizes [<pattern: string>] [-filter <string>]
```

Arguments

`pattern`

A glob-style wildcard pattern to match follower names against.

Default: `"*"`.

`-filter`

Standard attribute filter.

Returns

{<dbPaperSizeIterator>}: A collection of paper sizes.

Examples

```
# Print defined paper sizes attributes in console
db::foreach ps [db::getPaperSizes] {
    puts "Name: [db::getAttr ps.name] Width: [db::getAttr ps.width] \
        Height: [db::getAttr ps.height]"
}
```

db::getParamDefs

Description

Retrieves the list of parameter definitions for a given object. If an `oaInst/dmCell` is specified, both cell and library level parameters will be returned. Cell level parameters have higher precedence than library parameters and will hide same named definitions from the library. Note that `oaLibDMData` and `oaCellDMData` inherit from `oaDMData` so the object passed in can be either library or cell DM Data.

Syntax

```
db::getParamDefs [<pattern: string>] -level <enum> -of <dmCell |
    oaCellDMData | oaLib | oaLibDMData | oaRef | oaStdViaDef | oaTech>
```

Arguments

`pattern`

Glob-style pattern to match the parameter name. (Default is: *)

`-of`

The object that supplies the matching parameter definitions.

`-level`

The CDF level from which value will be obtained. ({ base | memory | overlay }, default is: overlay)

Returns

{<dbParamDef>}: Collection of parameter definitions for the given object that match the supplied pattern.

Examples

```
# Print all defined parameter names and default values for specified cell
db::foreach pd [db::getParamDefs -of [dm::getCells nmos4 -libName analogLib]] {
  puts "Name: [db::getAttr pd.name] \
        Default Value: [db::getAttr pd.defValue]"
}
```

db::getParamFilter

Description

Retrieves the list of Netlist Info identifiers that are active in the Parameter Filter Registry.

Syntax

```
db::getParamFilter -windowType <giWindowType>
```

Arguments

`-windowType`

Window Type.

Returns

The list of registered Netlist Info identifiers. ([<string>]).

Examples

The following example gets list of Netlisters active in Parameter registry.

```
# Get list of netlisters for a window type
set netlisters [db::getParamFilter -windowType seSchematic]
```

```
# Print netlisters names in Console
puts $netlisters
```

db::getParams

Description

Returns a collection of the parameters for the given instance, oaCustomVia or oaDMDData. If an instance, dmCell, or custom via is passed in, the returned list includes cell level and library level parameters.

Syntax

```
db::getParams [<pattern: string>] -level <enum> -of <dmCell |
oaCellDMDData | oaLib | oaLibDMDData | oaRef | oaStdViaDef | oaTech>
[-filter <string>]
```

Arguments

pattern

Glob-style pattern to match the parameter name. (Default is: *)

-of

The object that supplies the matching parameters.

-filter

Standard attribute filter. (Default is:)

-level

The CDF level from which value will be obtained. ({ base | memory | overlay }, default is: overlay)

Returns

{<dbParam>}: Collection of parameters for the given object that match the supplied pattern. ({dbParam}).

Examples

The following example gets and returns an editable parameter collection for the instance in the current design. This list can be used for getting required parameter values and making further modifications.

```
set instParams [db::getParams -of [db::getNext [db::getInsts -of \
    $oaDesign]] -filter %editable==1]
```

db::getParamValue

Description

Returns the value of a parameter. Optionally returns the database value, the netlisting value, the fully evaluated value, or performs a debug evaluation depending on the `-evalType` argument.

The database value is determined using the following search precedence:

If the parameter is of type `netSet`, returns the net name of the assignment definition. If there is no `oaAssignValue/oaAssignAssignment`, returns the empty string.

If an `oaParam` of the given name exists on the sub or super master, returns the `oaParam` value.

If an `oaProp` of the given name exists, returns the `oaProp` value.

If an Custom Compiler parameter exists of the given name, returns the `defaultValue` field.

If the `-evalType` argument is set to `none`, returns the database value.

If the `-evalType` argument is set to `full`, performs a full hierarchical evaluation. Full evaluation requires that a top context is provided with the `-context` argument. All PEL/SEL functions are evaluated. If a PEL/SEL function or design variable cannot be resolved then the maximal partial evaluation is returned.

If the `-evalType` argument is set to `debug`, performs a debug evaluation. This is useful for understanding why full evaluation failed.

If the `-evalType` argument is set to `netlisting`, performs a netlisting evaluation. During netlisting evaluation parameter and design variable references are tracked and stored in the Tcl variable name passed into the `-reLineageParams`, `-refParentParams`, and `-refDesignVars` arguments. If the `-useExponents` argument is set true then engineering suffixes will be substituted with exponents.

The `-map` argument controls mapping of real values, function call names, design variable names, and operators. It can be used to translate parameter expressions to languages that do not support Custom Compiler's engineering suffixes and AEL functions.

Syntax

```
db::getParamValue <name: string> -of <oaRef> [-evalType <enum>]  
[-context <heContext>] [-useExponents <bool>] [-map <dbExpressionMap>]  
[-refLineageParams <string>] [-refParentParams <string>] [-refDesignVars  
<string>] [-refParentListParams <string>]
```

Arguments

`name`

Parameter name.

`-of`

Object to be queried.

`-evalType`

Parameter Evaluation Type ({ debug | full | netlist | none }, default is: none)

`-context`

Context for full evaluation.

`-useExponents`

Convert engineering suffixes to scientific notation. (Default is: true)

`-map`

Expression Map for evaluation.

`-refLineageParams`

Variable storage reference for lineage parameters.

`-refParentParams`

Variable storage reference for parent parameters.

`-refDesignVars`

Variable storage reference for design variables.

`-refParentListParams`

Variable storage reference for parent list parameters.

Returns

`<string>` : The value of the named property/parameter/oaParam.

Errors

- If the named property/oaParam/parameter does not exist.
- If the evaluation type is `full` referenced parent/lineage parameters do not exist.
- If performing evaluation and there is a parse error in either SEL or PEL syntax.
- If the `-reDesignVars`, `-refParentParams` or `-refLineageParams` argument is specified and `-evalType` is not `netlisting`.

- If the `-context` argument is specified and the `-evalType` is not `full`.
- If the `-evalType` argument is `full` and `-context` is not specified.
- If the `-useExponents` argument is specified and the `-evalType` argument is not `netlisting`.

Examples

The following example returns the “w” parameter value of the X0 instance.

```
db::getParamValue w -of [db::getInsts X0 -of $oaDesign]
```

db::getPatternDefs

Description

Returns a collection of *dbPatternDef* objects currently defined and passing the given filter.

Syntax

```
db::getPatternDefs [<nameFilter: string>] [-filter <string>]
```

Arguments

`nameFilter`

Glob-style name filter.

Default is: `"*"`.

`-filter`

Standard filter expression.

Returns

`<dbCollection>` : Collection of pattern definitions.

db::getPatternInst

Description

Returns a *dbPatternInst* object for use in defining a *dbPatternDef* object.

Syntax

```
db::getPatternInst <instanceName: string> -deviceType <enum> -connections  
                  <string>
```

Arguments

`instanceName`

Instance name.

`-deviceType`

Type of device (`{ cap | diode | nmos | npn | pmos | pnp | res }`).

`-connections`

List of net/port pairs.

Returns

`<dbPatternInst>` : Pattern instance.

Errors

- Invalid port names used for specific device type.

db::getPcellCacheLocation

Description

Returns the on-disk pathname of the cache file used to cache the subMaster of the passed in dmCellview as specified by an ASSIGN 'PcellCachePath' in the lib.defs file.

Syntax

```
db::getPcellCacheLocation <CellView: dmCellView>
```

Arguments

`CellView`

A PCell dmCellView for which to determine its subMaster's cache path on disk.

Returns

`<string>`: The file path to the cache location of the supplied dmCellView, or empty if not cached.

db::getPcellCacheSize

Description

Returns the on-disk size of the cache file used to cache the subMaster of the passed in dmCellview.

Syntax

```
db::getCellCacheSize <CellView: dmCellView>
```

Arguments

CellView

A PCell dmCellView for which to determine the subMaster cache size on disk.

Returns

<int>: The size of the on disk subMaster cache. (Up to an 8-byte integer).

db::getCellDefs

Description

Returns a collection of PCell definitions defined within Custom Compiler from the specified libraries whose names match the specified pattern.

Syntax

```
db::getCellDefs [<name: string>] (-lib <{<oaLib>}> | -libName <string>)  
[-filter <string>]
```

Returns

{<dbPcellDef>}: A collection of PCell definitions.

Errors

None.

Arguments

name

The glob-style wildcard pattern to match PCell definition names against.

Default: "".

-lib {oaLib}

The collection of libraries from which to retrieve PCell definitions.

-libName <string>

The library from which to retrieve PCell definitions.

-filter <string>

Standard attribute filter.

Default: Match all.

Examples

The following example gets nfin PCell definition from reference10 library.

```
# Get nfin pcell definition from reference10 library
db::getPcellDefs nfin -libName reference10

# Check attributes for PCell definition
db::getAttr pcellDef.defFiles
db::getAttr pcellDef.lib.name
db::getAttr pcellDef.name
```

db::getPcells

Description

Returns a collection of dmCellViews that contain Custom Compiler defined PCell superMasters in the supplied library, optionally filtered by the attributes of a dmCellView.

dmCellViews are returned rather than oaDesigns because oaDesigns require the design to be open and remain open in order for the object to remain valid. By working with dmCellViews, which reference oaCellViews that only allow a single oaDesign to exist within them, the open restriction of the oaDesign can be avoided. The conversion from a dmCellView to an oaDesign is a simple conversion by name:

```
set ln [db::getAttr name -of \ [db::getAttr lib - of \ [db::getAttr cell
- of $dmCellView]]]

set cn [db::getAttr name - of \ [db::getAttr cell - of \ $dmCellView]]

set vn [db::getAttr name - of $dmCellView]

set d [oa::DesignOpen $ln $cn $vn $mode]
```

This function requests oacReadLibAccess access on the supplied library and if access is denied, the command fails.

Note: Because OpenAccess requires an oaDesign to be opened in order to determine if it is a PCell, this command can become increasingly inefficient as the number of oaDesigns in a library grows.

Syntax

```
db::getPcells -lib {<oaLib>} -libName <string> [-filter <string>]
```

Arguments

-lib {oaLib}

The collection of libraries for which the PCells are desired.

`-libName <string>`

The library for which the PCells are desired.

`-filter <string>`

standard attribute filter.

Default: match all.

Returns

A collection of dmCellViews that contain PCell superMaster oaDesigns ([dmCellView](#)).

Errors

oacLibAccessDenied.

Examples

The following example counts the number of PCells.

```
# Get PCells from specified library
set pcells [db::getPcells -libName reference40nm]

# Count number of PCells
db::getCount $pcells
```

db::getPins

Description

Returns a collection of pins. Returns an empty collection if no matching pins are found.

Syntax

```
db::getPins [<pattern: string>] -of <oaDesign | oaNet | oaTerm> [-filter  
  <string>]
```

Arguments

`<string>`

Pattern to match pin name.

Default: Match all pins.

`-of oaDesign | oaNet | oaTerm`

Design from which to get the pins, or the net or terminal to which pins are attached.

`-filter <string>`

Filter expression.

Default: no filter.

Returns

{<oaPin>}: Collection of pins.

Errors

- Wrong design or design is not open.
- Wrong oaNet or oaTerm.
- Wrong filter.

Examples

The following example returns the collection of pins with left access direction in the given oaDesign.

```
db::getPins -of [db::getTerms OUTM -of $oaDesign] -filter \  
    %accessDir==4
```

db::getPrefs

Description

Returns a collection of preference values from the specified scopes, optionally restricted by whether they are localized and by their label.

The `-scope` argument specifies the scopes to search for preferences. By default, if a preference matching name and other filters is localized in the supplied scope, the corresponding dbPref is returned, otherwise the scope tree is searched up for the closest parent where the preference is localized. This ensures that the 'effective' preference object is returned. If `-localized` is true, only those preferences localized in the supplied scope are returned and the search up the tree is not performed.

The `-label` argument lets you match preference definition labels.

Note that if multiple scopes are provided and `-localize` is false, the same preference may be matched multiple times but may only be returned once.

The `-dialog` argument lets you filter the list by preferences modified by a dialog. If `-scope` is not specified, preferences are returned using the scope specified by the dialog. If `-scope` is specified, the returned preferences are looked up relative to the given scope. In both cases, all other filters (the positional <string> argument, label, localized, and filter arguments) are applied to this set of preferences.

Syntax

```
db::getPrefs [<name: string>] [-scope <dbCollection>] [-label  
<string>] [-localized <bool>] [-dialog <giDialog>] [-filter <string>]  
[-includeDeprecated <bool>]
```

Arguments

name

Optional glob-style pattern to match against preference name.

-scope

Optional collection of scope specifications to return preference values from these scopes.

-label

Optional glob-style pattern to match against preference label.

-localized

Return only preferences localized in the given scope. (Default is: false)

-dialog

Optionally restrict list to preferences shown within a dialog.

-filter

Standard attribute filter.

-includeDeprecated

Specifies whether to include deprecated preferences in result. (Default: false)

Returns

{<dbPref>}: Collection of dbPref objects matching criteria.

Examples

The following example returns the collection of dbPref objects localized in the given cellView scope.

```
db::getPrefs -scope [db::getScopes $dmCellView] -localized true
```

db::getPrefValue

Description

Returns the value of a preference from the specified scope, if localized there, or from the lowest scope above where the preference is localized. This command is a shortcut for:

```
db::getAttr value -of [db::getPrefs $name -scope $scope]
```

Syntax

```
db::getPrefValue <name: dbPrefName> [-scope <dbScope | deContext | dmCell  
| dmCellView | oaLib | string>]
```

Arguments

name

Preference name.

-scope

The scope from which to retrieve the preference value.

Default: User scope in active environment.

Returns

<string>: Preference value.

Errors

- A preference with given name does not exist.
- Incorrect scope.

Examples

The following example returns the `giBindingSet` Tcl preference value from the project scope.

```
db::getPrefValue giBindingSet -scope project
```

db::getPrinters

Description

Returns a collection of printers whose names match the specified pattern.

Syntax

```
db::getPrinters [<pattern: string>] [-filter <string>]
```

Arguments

`<string>`

A glob-style wildcard pattern to match printer names against.

Default: `"*"`.

`-filter <string>`

Standard attribute filter.

Default: Match all.

Returns

A collection of printers (`{<dbPrinter>}`).

Examples

The following example types all printers names in the console.

```
# Get all printers
set printers [db::getPrinters]

# Type all printers names in the console
db::foreach printer $printers {puts [db::getattr printer.name]}
```

db::getProcessInfo

Description

Returns the program related information.

Syntax

```
db::getProcessInfo
```

Returns

`<dbProcessInfo>`: Process Information.

Examples

```
set pi [db::getProcessInfo]
foreach attr [db::listAttrs -of $pi] {
    puts "$attr = [db::getattr $attr -of $pi]"
}
```

db::getProps

Description

Returns a collection of oaProp objects.

Syntax

```
db::getProps [<pattern: string>] -of <attrObject> [-filter <string>]
```

Arguments

pattern

Property name

-of

OA object.

-filter

Standard attribute filter.

Returns

{<oaProp>} : Collection of props

Examples

The following example gets lxIgnore Boolean property.

```
# Get edit design from active context
set context [de::getActiveContext]
set design [db::getattr context.editDesign]

# Get single instance in the layout
set instance [db::getNext [db::getInsts -of $design]]

# Create lxIgnore Boolean property with value true for instance
db::createProp $instance -name lxIgnore -value true -type bool
# Get lxIgnore Boolean property
db::getProps lxIgnore -of $instance
```

db::getRegisteredNetlistInfo

Description

Returns the list of Netlist Info identifiers registered via db::registerNetlistInfo.

Syntax

```
db::getRegisteredNetlistInfo [<none: string>]
```

Arguments

<none>

The name of the netlist from which to retrieve its registered default CDF properties.

Default: "".

Returns

The list of registered Netlist Info identifiers. ([<string>]).

Examples

```
puts [db::getRegisteredNetlistInfo]
```

db::getRulers

Description

Returns a collection of liner and circular rulers.

Syntax

```
db::getRulers -design <oaDesign> [-filter <string>] [-addOverlay <bool>]  
[-editableDesignOnly <bool>]
```

Arguments

-design

Design for which the rulers are required.

-filter

Filter expression.

-addOverlay

Get additional objects from the overlay design.

-editableDesignOnly

Ignore the persistent rulers in the read-only design.

Returns

{<dbRuler>}: Collection of rulers.

Examples

The following example gets a collection of liner and circular rulers.

```
# Get rulers for the specified design
set rulers [db::getRulers -design $design]

# Count number of rulers and get their types
db::getCount $rulers
db::foreach ruler $rulers {puts [db::getattr ruler.type]}
```

db::getScopes

Description

Returns a collection of all scopes in the system, optionally restricted by their identifier. When specifying the scope name, a glob-style pattern may be used to identify all scopes matching the given pattern.

System-defined named scopes include install, site, project, and user.

Syntax

```
db::getScopes [<scope: deContext | dmCell | dmCellView | oaLib | string>]
               [-filter <string>]
```

Arguments

scope

Optionally return only those scopes with the specified identifier. String may use glob-style pattern matching. (Default: Match all)

-filter

Optional standard attribute filter. (Default: Match all)

Returns

Collection of scopes matching optional qualifiers ({<dbScope>}).

Errors

Incorrect environment.

Examples

In this example, the `db::getScopes` command returns project scope, and the next `db::getPrefs` command returns collection of preferences that have been modified in that scope.

```
set projectScope [db::getScopes project]
set prefs [db::getPrefs -scope $projectScope -filter {%modified}]
```

db::getScopeTypes

Description

Returns the collection of all scope types.

Syntax

```
db::getScopeTypes [<{pattern}: string>] [-filter <string>]
```

Arguments

{pattern}

Optional glob-style pattern to match against scope type names.

-filter

Standard attribute filter.

Returns

{<dbScopeType>}: Collection of scope types.

Examples

```
# Create a new scope type
set sct [db::getScopeTypes myScopeType]

# Create a new scope
set sc [db::createScope userScope -type $sct]

# get preference abd ist value from installation scope
set pref [db::getPrefs dbCapacitanceSuffix -scope [db::getScopes install]]

# localize the preference to the new created scope
db::localizePrefs $pref -scope $sc

# Set the preference value another from default
db::setPrefValue dbCapacitanceSuffix -value f

# Check the preference value in the netw scope
db::getPrefValue dbCapacitanceSuffix -scope userScope

# Check the preference value in the current scope
db::getPrefValue dbCapacitanceSuffix

# Check the preference value in the intallation scope, which is the default value
db::getPrefValue dbCapacitanceSuffix -scope install
```

db::getShapes

Description

Returns a collection of shapes. Returns an empty collection if no matching instance terminals are found.

Syntax

```
db::getShapes -of <oaDesign | oaLPPHeader | oaNet | oaPin> [-lpp <lpp>]  
[-filter <string>]
```

Arguments

`-of`

Design, net, pin, or oaLPPHeader from which to fetch the shapes.

When `-of oaDesign` is specified, returns a collection of shapes from the given design and matching to the given filter and LPP.

When `-of oaNet` is specified, returns a collection of shapes from the given net and pins/terms, which are connected to the given net and matching, to the given filter and LPP.

When `-of oaLPPHeader` is specified, returns a collection of shapes from the given LPP.

When `-of oaPin` is specified, returns a collection of shapes from the given net and matching to the given filter and LPP.

`-filter`

Standard filter expression.

Default: no filter.

`-lpp`

Restricts shapes to be returned from a given LPP.

Default: all LPPs.

Returns

Collection of shapes. (`{oaFig}`).

Errors

- Wrong design, or the design is not opened.
- Wrong oaNet, oaInst, or oaPin.
- Filter error.

Examples

The following example returns the collection of rectangular shapes on the "m1 drawing" LPP.

```
db::getShapes -of $oaDesign -lpp {m1 drawing} -filter \
    %type=="Rect"
```

The following example gets the wire objects from the current edit design and uses `se::setShapeStyle` to set them to the color red.

```
set shapes [db::getShapes -of [ed] -lpp {wire drawing}]
db::foreach shape $shapes {
    se::setShapeStyle $shape -color red
}
```

db::getTableValue

Description

Returns a string representation of the oaValue stored in the lookup table.

Syntax

```
db::getTableValue <values: [string]> -of <dbTable>
```

Arguments

`values`

Tcl list of column and row values to use for the table lookup. If the table is one-dimensional, only the first value from the list is used.

`-of`

Table object holding the lookup table to search.

Returns

`<dbCore>` : String representation of the oaValue stored in the lookup table

Examples

The following example returns string representation of the value stored in the lookup table.

```
# Get edit design from active context
set context [de::getActiveContext]
set design [db::getAttr context.editDesign]

# Find minSpacing constrain for M1 layer (8) from design
set constraint [db::findConstraint minSpacing -from $design -layers 8]

# Get table value for the specified row and column values
db::getTableValue {0.3 0.1} -from [db::getAttr constraint.value]
```

db::getTerms

Description

Returns a collection of terminals. If no matching terminals are found, returns an empty collection.

Syntax

```
db::getTerms [<pattern: string>] [-includeImplicit <bool> | -singleBit  
<bool>] -of <oaDesign | {<oaNet>}> [-filter <string>]
```

Arguments

pattern

Pattern to match object name. (Default is:)

-of

Design or objects from which to collect terminals.

-includeImplicit

Includes implicit nets in the return collection. (Default is: false)

-singleBit

Specifies whether to return single bit nets. (Default is: false)

-filter

Filter expression. (Default is:)

Returns

{<oaTerm>} : Collection of terminals.

Errors

- Wrong design or design is not open.
- Wrong oaNet.
- Filter error.

Examples

In these two examples, `db::getTerms` returns collections of terminals with the name CLK.

```
db::getTerms CLK -of $oaDesign
db::getTerms -of $oaDesign -filter %name=="CLK"
```

db::getType

Description

Returns the specific object type.

Syntax

```
db::getType -of <attrObject>
```

Arguments

`-of`

The object to get type of.

Returns

<variant> : Type of the object.

Examples

The following example demonstrates returned values.

```
set design [db::getAttr editDesign -of [de::getActiveContext]]
set insts [db::getInsts -of $design]
db::getType $insts
=> {<oaInst>}
db::getType [db::getNext $insts]
=> ScalarInst
set selected [de::getSelected -design $design]
db::getType $selected
=> {<deFigure>}
db::getType [db::getNext $selected]
=> deFigure
db::getType 123
=> string
```

db::getVDRCDDeviceList

Get the list of names of vdrc device with the specified type.

Syntax

```
db::getVDRCDDeviceList -targetLib <oaLib | string> -type <enum>
```

Returns

[<string>] : List of LCV of VDRC devices.

Arguments

-targetLib

Query VDRC devices applied in the target library.

-type

Device type ({ udv | vflagMax | vflagMin | vflaghl | vsync }).

db::getVDRCDDeviceParamName

Gets the parameter name of vdrc device with the specified paramter name type

Syntax

```
db::getVDRCDDeviceParamName -deviceLCV <dmCellViewName> -design <oaDesign>  
-paramNameType <enum>
```

Returns

<none>

Arguments

-deviceLCV

The LCV of the defined VDRC device.

-design

Design

-paramNameType

Type of param name ({ dv | vMax | vMin }).

db::getVDRCDDeviceVoltage

Gets the voltage value for the specified vdrc device.

Syntax

```
db::getVDRCDDeviceVoltage -inst <oaInst> -type <enum>
```

Returns

<none>

Arguments

-inst

Inst to query voltage value.

-type

Voltage type ({dv | vMax | vMin}).

db::getVersion

Description

Returns the current version of the tool.

Syntax

```
db::getVersion
```

Returns

<string> : Current version.

db::getViaDefs

Description

Collection of oaViaDef objects.

Syntax

```
db::getViaDefs [<name: string>] -of <oaTech> [-local <bool>] [-filter  
  <string>]
```

Returns

{<oaLayer>}: Collection of oaLayer

Arguments

name

Glob-style pattern to match against layer name or a number to filter the returned collection.

`-of`

The technology to get via defs from.

`-local`

Return local via defs Default: false

`-filter`

Standard attribute filter. Default: .

db::getViaHeaders

Description

Collection of oaViaHeader objects.

Syntax

```
db::getViaHeaders [<viaDefName: string>] -of <oaDesign> [-filter  
  <string>]
```

Arguments

viaDefName

oaViaDef name.

`-of`

oaDesign to search.

`-filter`

Standard attribute filter.

Returns

{<oaViaHeader>} : Collection of viaHeaders

Examples

The following example gets all via definitions headers from designs.

```
# Get edit design from active context  
set context [de::getActiveContext]  
set design [db::getattr context.editDesign]  
  
# Get all via definitions headers from designs  
db::getViaHeaders -of $design
```

The following example gets via12 via definition header from design.

```
# Get edit design from active context
set context [de::getActiveContext]
set design [db::getattr context.editDesign]

# Get via12 via definition header from design
db::getViaHeaders via12 -of $design
```

db::getVias

Description

Returns a collection of vias, otherwise returns an empty collection if no matching objects are found. Unnamed arguments are added as the collection of oaNet.

Syntax

```
db::getVias [<pattern: string>] -of <oaDesign> | <oaPin> | {<oaNet>} |
{<oaViaHeader>}> [-filter <string>]
```

Arguments

pattern

Glob-style pattern to match object name. The default is " ".

-of

A design, a pin, or a collection of viaHeaders or nets from which to get the vias.

-filter <string>

Standard filter expression. The default is " ".

Returns

<{<oaVia>}>: Collection of vias.

Examples

The following example gets and returns the collection of all vias in the design.

```
set viaCollection [db::getVias -of $oaDesign]
```

db::getVirtualInstTerms

Description

Returns a Tcl list of virtual instTerm name pairs for the given instance. To create logical netlists, the virtual instTerm list should be appended to the OpenAccess instTerm list for that instance.

Syntax

```
db::getVirtualInstTerms <none: dbContextConnections> -instName <string>
```

Arguments

`dbContextConnections`

An object which holds an LCV for the current block and lineage describing the instance path from the top block to this current block. It also holds the current set of config rules.

`-instName`

Name of scalar instance for which to retrieve the virtual instTerms.

Returns

Tcl list of strings representing virtual terminal names. (<string>).

Errors

- Instance name given does not exist.
- Invalid heContext.

Examples

```
# Get and print virtual instance terminals names of I0 instance
set context [de::getActiveContext]
set h [db::getAttr context.hierarchy]
set design [db::getAttr context.editDesign]
set ctxConnections [db::findContextConnections $h -of
    [db::findConnections $h]]
set virtInstTerms [db::getVirtualInstTerms $ctxConnections -instName I0]
puts $virtInstTerms
```

db::getWindowAnnotationSets

Description

Gets window annotation sets.

Syntax

```
db::getWindowAnnotationSets -window <giWindow | int> [-filter <string>]
```

Arguments

`-window`

Window from which to retrieve the dbWindowAnnotationSet objects.

`-filter`

String/expression to further filter the returned collection.

Returns

{<dbWindowAnnotationSet>}: Collection of dbWindowAnnotationSet objects.

Examples

```
# Get an annotation set
set anSet [lindex [db::getAnnotationSets ] 0]

# Create window annotation set
db::createWindowAnnotationSet $anSet -window [gi::getActiveWindow]

# Print names of the window annotation sets
set wndAnSet [db::getWindowAnnotationSets -window [gi::getActiveWindow]]
puts [db::createList [db::getattr wndAnSet.name]]
```

db::hideProps

Description

This command tells the Property Editor to hide/ignore properties matching specific names. It is intended to be used in system startup scripts to specify the list of properties which should not be displayed to the user by the Property Editor. For example, schematics use properties to store system information such as the running count of created instances and also version counters tracking connectivity extraction. These properties should not be edited by users.

The `-objType` name must be a valid OA type name. Class inheritance is used to determine if properties should be hidden. For example if a particular property should be hidden on both `oaInsts` and `oaVias`, it is sufficient to call `db::hideProps` with `-objType = "Ref"` (Note that `oaRef` is the super class of `oaVia` and `oaInst`).

Syntax

```
db::hideProps <names: [string]> [-objType <string>]
```

Arguments

{<string>}

List of property names to ignore.

`-objType`

Type of object for which the named properties should be hidden.

Default: "Design".

Examples

The following example hides `lxIgnore`, `lvsIgore` and `nlAction` properties in Property Editor.

```
db::hideProps {lxIgnore lvsIgnore nlAction}
```

db::importEDIF

Description

Imports an EDIF file.

Syntax

```
db::importEDIF -inputFile <string> [-outputLibName <string>]  
[-topCellName <string>] [-prepare <string>] [-importResultLogFile  
<string>] [-overwrite <bool>] [-ignoreUnusedParams <bool>]  
[-putLabelOnAllSegments <bool>] [-case <enum>] [-cellNamesCase  
<enum>] [-instsNamesCase <enum>] [-netsNamesCase <enum>]  
[-paramsNamesCase <enum>] [-symbolsOnly <bool>] [-cdfFilesDir <string>]  
[-notSearchExternalSymbol <bool>] [-dataSource <enum>]
```

Arguments

`-inputFile`

Specifies the input EDIF file.

`-outputLibName`

Specifies the output library name. This is only used with `-symbolsOnly`. The default is "".

`-topCellName`

Specifies the top cell name of importing EDIF file. The default is "".

`-prepare`

Specify the file to return the list of LCVs with parsed data dump. No importing is done. (Default is:)

`-importResultLogFile`

Specifies the import results log file name. The default is: "".

`-overwrite`

Overwrites the existing cellviews. The default is true.

`-ignoreUnusedParams`

Automatically removes unused parameters. The default is false.

`-putLabelOnAllSegments`

Creates wire labels on all segments of a net. The default is false.

`-case`

Specifies case sensitivity for all names. Valid values are lower, preserve, and upper. The default is preserve.

`-cellNamesCase`

Specifies case sensitivity for cell names. Valid values are lower, preserve, and upper.

`-instsNamesCase`

Specifies case sensitivity for instance names. Valid values are lower, preserve, and upper.

`-netsNamesCase`

Specifies case sensitivity for net/terminal names. Valid values are lower, preserve, and upper.

`-paramsNamesCase`

Specifies case sensitivity for parameter names/values. Valid values are lower, preserve, and upper.

`-symbolsOnly`

Extracts the symbol library only. The default is false.

`-cdfFilesDir`

Specifies the directory of the input iCDF file. The default is: "".

`-notSearchExternalSymbol`

When specified, do not search for external symbols. The default is false.

`-dataSource`

Specifies the producer tool of source data (leave default if not sure). Valid values are Composer and ViewDraw. The default is Composer.

Examples

```
# Import the adder.edif file into the imported_edif library
db::importEDIF -inputFile adder.edif -outputLibName imported_edif
```

db::importMCellFile

Description

Import MCell definitions.

Syntax

```
db::importMCellFile (-lib <oaLib> | -libName <dmLibName>) -filePath  
  <string> [-cdfFilePath <string>] [-overwrite <bool>]
```

Arguments

-lib

OA library that the MCell devices will be created into.

-libName

Name of OA library that the MCell devices will be created into.

-filePath

The MCell definition file path.

-cdfFilePath

The MCell CDF file path.

-overwrite

Overwrites the existing cell views. The default is false.

Returns

{<dmCellView>} : The collection of newly created MCell cell views.

Examples

The following example imports MCell file into existing definition library.

```
db::importMCellFile -filePath ./guardRing.mcddef -libName mcellLib
```

db::importTechFile

Description

Reads technology data from a technology file in ISTF or XML format. It runs a parser on it and loads technology data into the in-memory technology object (oaTech) of the specified library. To work successfully, this command requires that the specified OA library be already opened in the current session. If a technology object does not exist in the specified library, it will be created.

If the user attempts to import technology data from a file into an oaLib that does not have a local oaTech, but instead references another library technology through the techLibName property, then the command follows references and imports technology into a leaf library.

When the parser finds any syntax error in the technology file, it restores the in-memory database to its previous state and issues a message about the error, showing the location of the error in the technology file. The parser will report syntax errors in cases when the structure of technology file is corrupted or if an unknown keyword appears in section names. If an unknown keyword appears as a name of item in a known subsection, then it will be interpreted as legal data. That data (unknown to Custom Compiler data), will be stored in a database, so that it can be exported later to technology file, but Custom Compiler will not use that data in any other way. A warning will be issued if such data will be encountered during import.

Syntax

```
db::importTechFile <path: string> (-lib <oaLib> | -libName <string>)  
[-mode <string>] [-sections <string>]
```

Arguments

<string>

Specifies path including the file name of technology file to import.

-lib

The OA library where technology data should be loaded.

-libName

The name of the OA library where technology data should be loaded.

-mode

Importing technology mode: merge, overwrite or replace.

Default: overwrite.

-sections

Sections or subsections to be imported. If you specify the name of a section, all data within that section is imported. If you specify a subsection, only the data in the subsection is imported. For the names of the sections and subsections, see Technology File Contents. (Default: all)

For example, `-sections controls` imports all data in the controls section.

`-sections {techLayers techDisplays}` imports the data in the techLayers and techDisplays subsections of the layerDefinitions section. `-sections all` imports all of the technology data.

Returns

Technology object filled after reading the file. (*oaTech*).

Errors

- Cannot read, or incorrect format of input file.
- Permission denied.
- Bad arguments.
- Both `-lib` and `-libName` are provided.

Examples

```
db::importTechFile tech.tf -libName techLib -merge false
```

In this example, `db::importTechFile` loads technology data into an open `techLib` library. If any technology data already exists, it will be overwritten.

db::importText

Description

Imports netlist text files into the given library. The module/block names in the netlist files are used as the cell name along with the given library and view name to make up the destination cellviews.

If `-interfaceOnly` is set to true, then the associated shadow OA netlist design for each imported cellview contains only terminal information and does not have instances.

Use `-libSearchList` and `-viewSearchList` to control using which master design for each instance found in cellview text. These arguments accept a list of strings. If `-librarySearchList` is not given, the shadow OA extraction uses the library ordering from the `lib.defs` file.

To have symbols generated for each cellview imported, the `-generateSymbols` argument can be used. The argument given is a symbol generator option-set name. By default, an option-set named "default" exists in the tool. The view name used for the generated symbol is *symbol*.

Syntax

```
db::importText <filePath: [string]> -libName <string> [-viewName  
  <string>] [-libSearchList [<string>]] [-viewSearchList [<string>]]  
  [-interfaceOnly <bool>] [-generateSymbols <string>] [-topSubckt  
  <string>] [-digitalTop <string>] [-overwrite <enum>] [-overwriteSymbols  
  <enum>] [-viewType <enum>] [-deviceMappingFile <string>]  
  [-includeFiles <string>] [-defines <string>] [-parserOptions <string>]  
  [-preserveDirectives <bool>] [-importModules <string>] [-moduleMatchType
```

```
<enum>] [-excludeModules <string>] [-excludeModuleMatchType <enum>]  
[-configTemplate <heConfigTemplate | string>] [-singleSource <bool>]  
[-virtualTermMapping [<string>]] [-virtualTermsReferenceView [<string>]]  
[-skipModDefGeneration <bool>]
```

Arguments

`filePath`

List of netlist text files to import.

`-libName`

Destination library name for the imported text views.

`-viewName`

Destination view name for the imported text views.

`-libSearchList`

Library search list for instance binding.

`-viewSearchList`

View search list for instance binding.

`-interfaceOnly`

Extracts the interface of the imported text views only. The default is false.

`-generateSymbols`

Generates symbols. If not specified, no symbols are generated. (Default is:)

`-topSubckt`

Indicates whether top level SPICE/CDL statements should be imported into a cell and what cell name to use for it. (Default is:)

`-digitalTop`

Specifies the top module name. If digital is the top of the imported hierarchy, config is created on the corresponding cell. (Default is:)

`-overwrite`

Cellview overwriting mode to use during importing. ({ always | never })

`-overwriteSymbols`

Cellview overwriting mode to use during importing for symbols to be generated. ({ always | never | portOrderOnly })

`-symGenMergeSubPorts`

Merge related bit terminals into busTerm in specified order when generating symbol. ({ asc | desc | off })

`-symGenBitFormat`

The bus bit format used in source cellview that is used when generating symbol for merging ports. (Default: [d])

`-viewType`

View type for the imported text views. If not specified, it is deduced from the view name or file extensions. ({ CDLText | SPICEText | VHDLText | systemVerilogText | verilogAMSText | verilogAText | verilogText }, default is:)

`-deviceMappingFile`

Device mapping file path to be used for instance binding, for analog languages only. (Default is:)

`-includeFiles`

Filter specifying processing mode for include files. The acceptable format is `<action> or <action> {<action> <file_names> <match_mode>} ...` where `<action>` is one of the set: {parse, include, ignore}, `<file_names>` is a single name or a Tcl list of names, and `<match_mode>` is one of the set: {exact, regexp, wildcard}. (Default is: include)

`-defines`

Additional definitions list to be predeclared for the verilog/A/AMS views. (Default is:)

`-parserOptions`

Parser specific options (Default is:)

`-preserveDirectives`

Indicates whether the directives should be preserved in the imported text views. (Default is: true)

`-importModules`

The names of the modules (or module name pattern) to be imported. (Default is:)

`-moduleMatchType`

The module matching type to filter the module names using `-importModules`. ({ exact | regexp | wildcard }, default is: exact)

`-excludeModules`

The names of the modules (or module name pattern) to be excluded during import. (Default is:)

`-excludeModuleMatchType`

The module matching type to filter the module names using `-excludeModules`. ({ exact | regexp | wildcard }, default is: exact)

`-configTemplate`

Hierarchy config template to be used for config creation, when `-topSubckt` or `-digitalTop` is specified.

`-singleSource`

Indicates whether text importing should be done in single source mode. (Default is: false)

`-virtualTermMapping`

Terminal to connection definition mapping.

`-virtualTermsReferenceView`

Prioritized list of views for determining if a terminal should be virtual. Any terminal that is not found in a reference view is created as virtual.

`-skipModDefGeneration`

Generates a module definition file from the reference library. (Default is: false)

Returns

{<dmCellView>}: A collection of created CellViews.

The first cellview in the collection is the top cellview. If `-configTemplate` is specified, then in addition to importing the text modules, it also creates a config view, which is the first item in the collection.

Examples

The following example imports one or more netlist files into the given OA library.

```
# Set device mapping file to one located in SYNOPSYS_CUSTOM_INSTALL
set deviceMapFile [file join $::env(SYNOPSYS_CUSTOM_INSTALL) \
samples device_map.xml]

# Import amds_lib_ldo spice netlist into DesignLib library
db::importText ./amds_lib_ldo.cir -libName DesignLib \
-viewType SPICEText -viewName hspiceD -overwrite always \
-generateSymbols default -overwriteSymbols always \
-deviceMappingFile $deviceMapFile
```

```
# Open the design in which netlist was imported
de::open DesignLib/ldo/hspiceD
```

db::includes

Description

Checks whether an object is included in a given collection.

Syntax

```
db::includes <collection: dbCollection> -object <attrObject>
```

Arguments

collection

An iterator over the collection.

-object

An object to be searched.

Examples

The following example processes the initial selection set, then deselects objects on {m2 drawing} LPP.

```
set selected [de::getSelected -design [ed]];
# get selection in the edited design

set figs [de::getFigures [db::getattr bbox -of [ed]] -design [ed] \
  -touch true -depth 0 \
  -filter {%LPP.lpp == [list m2 drawing]}]

db::foreach fig $figs {
    if {[db::includes $selected -object $fig] } { de::deselect $fig
}
}
```

db::init

Description

Given a path relative to any startup directory, this command sources the matches from any startup directory.

Essentially, this command sources all files matched by `db::resolve`. It may be used to initialize a product with startup files spread throughout the startup directories after the full platform has been invoked.

Syntax

```
db::init [<path: string>] [-echo <bool>] [-noWarning <bool>]
```

Arguments

`path`

Path relative to any startup directory. (Default is: .)

`-echo`

Display content of sourced files. (Default is: false)

Default is: false.

`-noWarning`

Do not issue a warning when no file is found. (Default is: false)

Examples

The following example loads Tcl file from startup folder provided as environment variable and displays the content of sourced file.

```
db::init layoutData/utilities/convertLayer.tcl -echo true
```

db::instCacheTriggerParamCallbacks

Description

Triggers parameter callbacks by using cache.

Syntax

```
db::instCacheTriggerParamCallbacks <cache: dbCore> -of <oaRef>  
-triggerParamList <string>
```

Arguments

`cache`

The inst cache.

`-of`

The instance used to trigger parameter callbacks.

`-triggerParamList`

The ordered list of parameter names to trigger callbacks for.

Returns

<none>

Examples

```
set cache [db::createInstCache]
set inst [de::getSelected -design [ed]]
set instance [db::getAttr object -of $inst]
db::instCacheTriggerParamCallbacks $cache -of $instance -triggerParamList
{contRows finger}
```

db::interrupt

Description

Displays a dbInterruptException Tcl exception.

Syntax

```
db::interrupt
```

Examples

The following example interrupts the loop, closing the progress dialog, if the process lasts more than 10 seconds.

```
db::eval {
    set progress [db::createProgress progress -title Progress ]
    set begin [clock seconds]
    puts "Starts at [clock format $begin -format %H:%M:%S]"
    update
    while { true } {
        db::checkForInterrupt -handler $progress
        de::sleep 1
        set end [clock seconds]
        set delay [expr $end - $begin]
        if {$delay > 10} {
            puts "Ends at [clock format $end -format %H:%M:%S]"
            db::interrupt
        }
    }
}
```

db::isEmpty

Description

Checks whether a collection is empty.

Syntax

```
db::isEmpty <collection: dbCollection>
```

Arguments

<object>

An iterator over the collection.

Returns

1 when the collection is empty, and 0 when the collection contain any object.

Examples

This example checks whether a net with the name `vControl` exists in the design. If such a name exists, the command returns 0, otherwise it returns 1.

```
db::isEmpty [db::getNets vControl -of [ed]]
```

db::isEncrypted

Description

Checks if a file is encrypted in the Laker format.

Syntax

```
db::isEncrypted <filename: string>
```

Arguments

filename

The file to be checked.

Examples

```
puts [db::isEncrypted test.tcl]
# Encrypt Tcl script
db::encrypt test.tcl -target test.tcl.enc
# Returns true
puts [db::isEncrypted test.tcl.enc]
```

db::isObject

Description

Checks whether a Tcl variable holds a `dbObject` or an `oaObject`.

Syntax

```
db::isObject <variant>
```

Arguments

<variant>

The object to be tested.

Returns

<bool>: True if the given variable holds a dbObject or an oaObject, otherwise, returns false.

Examples

In the following example, the `db::isObject` command will return true.

```
set p [le::createBoundary [list [list 0 0 ] [list 1 2]] -type pr -design
    [ed]]
db::isObject $p
```

In the following example, the `db::isObject` command will return false.

```
set p "Hello, world"
db::isObject $p
```

db::isProhibitedName

Description

Checks whether the name contains prohibited characters defined by Tcl preference `dbProhibitedCharacters`.

Syntax

```
db::isProhibitedName <name: string>
```

Arguments

name

Object name to check.

Returns

<bool> : Returns true if the given name contains prohibited characters defined by preference `dbProhibitedCharacters`, otherwise false.

Examples

```
# Set prohibited characters, name cannot be started with "%" and should
    not contain "^"
db::setPrefValue dbProhibitedCharacters -value {%}{^}

# Check if the name starts with "%" or contains "^" and print message
```

```
set name "%abc"  
if {[db::isProhibitedName $name]} {  
    puts "The name $name contains prohibited characters"  
}
```

db::listAttrs

Description

Returns the attribute names for the given object. For a deFigure semantic type object, attributes attached to its related OpenAccess object are also listed.

`db::listAttrs` can access both PCell subMaster and additional MCell guard ring and MOS subMaster attributes.

Syntax

```
db::listAttrs [<pattern: string>] -of <attrObject> [-includeDeprecated  
    <bool>] [-type <enum>]
```

Arguments

`pattern`

Glob style pattern matching the attribute names to be listed.

Default: `*`.

`-of`

The object for which the attributes are listed.

`-includeDeprecated`

Whether deprecated attributes are listed.

The default is true.

`-type`

Filters attributes to only those of the specified type: {all | appDef | intrinsic | param | prop}.

Default: all.

Returns

List of attribute names ([<string>]).

Errors

- The object type is not supported.
- The object is invalid.

Examples

```
db::listAttrs -of [db::getNext [dm::getLibs ]]
```

In this example, `db::listAttrs` returns the following attributes of the library:

| | | | |
|-----------------|---------------|------------|-----------|
| accessLevel | groupMems | path | this |
| constraintGroup | groupsOwnedBy | props | type |
| fullPath | mode | readable | writable |
| groupLeaders | name | statusIcon | writePath |
| refLibs | | | |

The value for each attribute can be fetched using the `db::getAttr` command or can be set using the `db::setAttr` command.

db::listCallbackTypes

Description

Generates a report that lists all the defined callback types and their argument lists.

Syntax

```
db::listCallbackTypes[-pattern <string>]
```

Arguments

`-pattern`

Glob-type pattern to match callback names. The default is `*`.

Returns

Report of all the defined callback types (<string>).

db::loadDeviceDisplays

Description

Loads the label display information.

Syntax

```
db::loadDeviceDisplays [-window <int>]-filePath <string>
```


Arguments

`-window`

Window ID.

`-filePath <string>`

File from which to load the display information. If absolute path not specified, relative path is assumed.

Returns

{<dbDeviceDisplay>}: Collection of dbDeviceDisplay objects created from the file.

Examples

The following example loads the label display information from the XML file for specified window.

```
db::loadDeviceDisplays -filePath deviceDisplays.xml -window 3
```

The following example loads previously saved file to hide instance name.

```
# Create device display to hide name of I0 instance
db::setDeviceDisplay -of [db::getInsts I0 -of [ed]] -name none

# Save device displays in devDisplay.xml file
db::saveDeviceDisplays [db::getDeviceDisplays] -filePath ./devDisplay.xml

# Restore name of I0 instance by destroying device display
db::destroy [db::getNext [db::getDeviceDisplays -filter {%objectName ==
    "I0"}]]

# Load previously saved file to hide name again
db::loadDeviceDisplays -filePath ./devDisplay.xml
```

db::loadGenOptionSets

Description

Creates new dbGenOptionSet objects from the given file. If a set already exists with the given name, an error is generated, and no option set data from the file is applied. If the `-force` argument is true, any existing option set with the same name as a set from the file is overwritten.

Syntax

```
db::loadGenOptionSets -filePath <string> [-force <bool>] [-loadDefault
    <bool>] [-src <oaDesign>] [-fromTech <bool>]
```

Arguments

`-filePath`

Name of file name to read and load into the named option set.

`-force`

If true, overwrites existing option sets with the same name as a set from the file.

The default is false.

`-loadDefault`

Loads default values for abstractGenerator.

The default is false.

`-src`

The source design.

`-fromTech`

Gets constraints from the technology database, otherwise gets them from the design.

The default is false.

Returns

New collection of dbGenOptionSet objects ([dbGenOptionSet](#)).

Errors

- Set with same name already exists.
- filePath name given is not a valid dbGenOptionSet data file or non-existent.

Examples

In the following example, the symbol is generated using the schematic view as a source for the interface information and the symbol generator options are loaded from the file.

```
# Find the source view
set srcCellView [dm::findCellView schematic -cellName inv -libName testLib]

# Option set file for Symbol Generation stored in current folder
set opSetFile [file normalize opsetFile.xml]

# Load options sets for Symbol Generator
set os [db::loadGenOptionSets -file $opSetFile -force true]

# Get interface set object from source view
set is [db::getCellViewInterface -src $srcCellView]

# Generate symbol view
```

```
se::generateSymbol -interface $is -dest testLib/inv/symbol -update false \  
-options $os
```

The following example loads a new option set from the *agM1OptionSet.xml* file.

```
db::loadGenOptionSets -filePath ./agM1OptionSet.xml  
  
# The XML file format can be:  
# <dbGenOptionSet type="abstractGenerator" name="m1Set">  
# M1 Use true  
# M1 Cover true  
# M1 Pin true  
# M1 Blockage true  
# M1 pinCutout false  
# M1 effectiveWidth effectiveWidth  
# M1 Value 0.15  
# M1 ignoreAccess false  
# M1 Oversize 0.022  
# </dbGenOptionSet>
```

db::loadMCells

Description

Reloads MCell rules from superMasters in disk.

Syntax

```
db::loadMCells (-lib <oaLib> | -libName <string>)
```

Arguments

-lib

OA library from where MCell superMaster data should be loaded.

-libName

OA library from where MCell superMaster data should be loaded.

Examples

```
# Load MCell data from the reference40nm library  
db::loadMCells -libName reference40nm
```

db::loadModelFile

Description

Loads simulation model files and sections by SPICE parser.

Syntax

```
db::loadModelFile <fileSection: [string]> [-format <enum>]
```

Arguments

`fileSection`

List of model files and sections to load.

`-format`

Format of model files. The default is spice. ({ spectre | spice })

Returns

<dbGndbProfile> : Sessions matching the search criteria

db::loadPrefs

Description

Loads all preferences or a set of preferences from a scope or an arbitrary file on disk. If a collection of preferences are specified, modifies those whose name matches a preference within the loaded data. If instead a scope is specified, all values are localized to the specified scope.

If the `-filePath` argument is omitted, the scope referenced by the provided dbPref or dbScope object is loaded. Note that performing this operation with a collection of dbPref objects may be slow.

Returns all modified dbPref objects. Preferences that were specified as arguments but did not have values on disk are omitted from this list and do not cause any error.

Syntax

```
db::loadPrefs [<collection: dbCollection>] [-filePath <string>]
```

Arguments

`collection`

Collection of preferences to modify or a scope into which these values are loaded and localized.

`-filePath`

Load preferences from supplied file. If provided, matching preferences given in the unnamed argument are modified (Default is:)

Returns

{<dbPref>}: Collection of preferences which were loaded.

Errors

- Cannot open `-filePath`.
- Incorrect scope.

Examples

```
# File prefs.xml which contains saved preference values
set prefsFile [file normalize prefs.xml]

# Set collection of needed preferences
set pc [db::getPrefs seLabel* ]

# Get the scope of the cellview
set scope [db::getScopes [dm::findCellView schematic -libName design \
    -cellName Diff ]]

# Load preferences which are defined for the cellview scope
set prefs [db::loadPrefs $scope -filePath $prefsFile ]

# Or load those preferences which are selected in the pc collection
set prefs [db::loadPrefs $pc -filePath $prefsFile ]
```

db::loadStreamNameMap

Description

Loads an XML file into the dbStreamNameMap object.

Syntax

```
db::loadStreamNameMap <StreamObjMap: dbStreamNameMap> -filename <string>
```

Arguments

StreamObjMap

Cell name to stream object map.

-filename

An XML file to load into the map.

db::localizePrefs

Description

Localizes preference in the scope returning the newly localized preference. The new preference value is that of the dbPref object provided. If a localized preference already exists at the given scope, its value is overwritten with the supplied preference's value.

Syntax

```
db::localizePrefs <dbpref: dbCollection> -scope <dbScope | deContext |  
dmCell | dmCellView | oaLib | string>
```

Arguments

dbPref

Preferences to localize.

-scope

Scope where the preference is localized.

-ignoreErrors

Ignore errors localizing preferences and localize only those which can be localized.

The default is false.

Returns

{<dbCollection>}: Collection of preference objects localized in the given scope.

Errors

- Incorrect scope.
- Invalid preference object.
- One or more preferences cannot be localized at the target scope.

Examples

```
# Set list of preferences which are needed to be set  
set prefs [list {leShapeTap} {leRectangleAsSquare}]  
  
# Create a blank collection of dCollection object  
set coll [db::createCollection]  
  
# Get the preferences, set their values and add them to the collection  
foreach p $prefs {  
  
    # Get the preference from the user scope  
    set pref [db::getPrefs $p -scope [db::getScopes user]]  
  
    # set its value  
    db::setAttr pref.value -value true  
  
    # Add the preference to the collection  
    db::add $pref -to $coll  
}  
  
# Localize the preferences into the design library scope  
db::localizePrefs $coll -scope [dm::getLibs DesignLibrary]
```

```
# Save the preferences
db::savePrefs $coll
```

db::makePcellEditable

Description

Makes a general PCell to be an editable PCell by registering user-defined procedures.

Notice that the data passed to the user-defined procedures is a list of name value pairs, for example, {`inst $inst`} indicates that `inst` is the name (in string format) and `$inst` is its value.

Syntax

```
db::makePcellEditable <cellView: dmCellView | dmCellViewName>
  -pcellType <enum> [-grPinLPPs [<lpp>]] [-contourLPPs [<lpp>]]
  [-grAllowedStyles <string>] [-createProc <string>] [-grLPPsProc
  <string>] [-selectDesc <string>] [-selectProc <string>] [-stretchProc
  <string>] [-chopLPPsProc <string>] [-chopProc <string>] [-reshapeProc
  <string>] [-splitProc <string>] [-healLPPsProc <string>] [-healProc
  <string>] [-cutRemovalHaloProc <string>] [-cutRemovalProc <string>]
  [-directEditProc <string>]
```

Arguments

`dmCellView` | `dmCellViewName`

The `cellView` in which to convert.

`-pcellType`

Specifies the device type. Valid values are `flatInstance`, `guardring`, `instance`, `track`, `transmissionline`, and `via`.

The `lvsIgnore` property with value *true* is added on the `superMaster` if the device type is `guardring`.

`-grPinLPPs`

Specifies the LPPs of pins for the guard ring. This argument is valid only when the PCell type is `guardring`.

`-contourLPPs`

Specifies the LPPs that contribute the contour of the guard ring or transmission line. This argument is valid only when the PCell type is `guardring` or `transmissionline`.

`-createProc`

The callback procedure for creation. This procedure is called in the creation commands for MCells and editable PCells, such as `Create Manual Guard Ring`

and Create Auto Guard Ring. For guard rings, this procedure is called in the creation commands whenever the input points are changed.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $RunTimeInst} {centerLine $CenterLine} { edgeLine
  $EdgeLine} {interactive Boolean} }
```

where:

- `inst $RunTimeInst`: `oalnstld`.
- `$CenterLine`: User-specified point list in a fixed format `{ {x0 y0} {x1 y1} {x2 y2} ... {xn yn} }`. The point list is relative (well-transformed) to the instance's master.
- `$EdgeLine`: String of auto calculated edgeline point list in a fixed format `{ {x0 y0} {x1 y1} {x2 y2} ... {xn yn} }`. The point list is relative (well-transformed) to the instance's master.

In the creation procedure, points are grown to get the real centerline. This argument is valid when `centerLine` is not specified.

- `interactive Boolean`: When `TRUE`, indicates that the calling is under interactive mode. When `FALSE`, indicates that the calling is to commit the data.

Default: `FALSE`.

- Associated Tcl preferences:

- Preference `leAutoGRNotchValue` specifies the minimum notch value.
- Preference `leAutoGRSpacingMethod` specifies the space method. When *true*, specifies the space by Boundary method. When *false*, specifies the space by Layers method.
- Preference `leAutoGRSpacing` specifies the boundary spacing when `leAutoGRSpacingMethod` is *true* (Boundary method).
- Preference `leAutoGRLayerSpacing` specifies the layer spacing when `leAutoGRSpacingMethod` is *false* (Layers method). The value format is the same as the preference `leGRLayerSpacing`.
- Preference `leGRInteractive` enables or disables the interactive mode for guard rings. The default value is `1`.

- **Return:** A Boolean value, where:
 - **TRUE:** The procedure handles the create operation successfully.
 - **FALSE:** The oalnstld fails to update with the creating conditions.

For example, when you click points {10.0 10.0} {20.0 10.0} {20.0 15.0} {35.0 15.0} in the canvas during guard ring creation, the origin of the runtime instance is set as {10.0 10.0}, the orientation is set as R0, and the centerline passed into the creation procedure is {0.0 0.0} {10.0 0.0} {10.0 5.0} {25.0 5.0}.

`-grLPPsProc`

The callback procedure of guard ring LPPs. This argument is valid only when the PCell type is `guardring`. This procedure is called in the Create Auto Guard Ring command to set up a dialog setting for the spacing option.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $Inst} }
```

where:

- `$Inst`: oalnstld, which is the operated editable PCell device.
- Return of string with a list of choppable LPPs:

```
{ {LayerName1 PurposeName1} {LayerName2 PurposeName2} ... }
```

`-selectDesc`

Specify the selection filter descriptions. Each selection filter description should have format {simple filter with LPP, object type, selection filter procedure} in which the object type and selection filter procedure are optional.

Valid object types include `all`, `line`, `rectangle`, `polygon`, and `path`. For example:

- { stretch } means that only objects on LPP (stretch drawing) are selectable.
- { Diff line } means that only line objects on LPP (Diff, drawing) are selectable.
- { Diff line } { M1 all M1SelectProc } { PPLUS {rectangle polygon} ImpSelectProc } means that line objects on LPP (Diff, drawing) are selectable; all objects on LPP (M1, drawing) that can pass the selection filter procedure M1SelectProc are selectable; and rectangles and polygons on LPP (PPLUS, drawing) that can pass the selection filter procedure ImpSelectProc are also selectable.

- `{ Diff line } { M1 rectangle selectProc } { PPLUS all selectProc }` means that line objects on LPP (Diff, drawing) are selectable; rectangles on LPP (M1, drawing) that can pass the selection filter procedure `selectProc` are selectable; and all shapes on LPP (PPLUS, drawing) that can pass the selection filter procedure `selectProc` are also selectable.

`-selectProc`

The callback procedure for selection filter. This procedure is called in the preselect and post select of the Stretch, Chop, Reshape, and Split commands to define what can or cannot be selected.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $Inst} {figure $Figure} {selection $Selection}
  {command $ActiveCmdName} }
```

where:

- `$Inst`: `oaInstId` or `oaViald`, which is the current editable PCell instance or via.
- `$Figure`: `deFigure`, which is the object in the current editable PCell instance or via to be selected; where:

`deFigure` = hierarchy (relative to the editing design) + `oaFigId`
- `$Selection`: Selected point list of the figure in a fixed format `{{x0 y0} {x1 y1} ...}`. If the figure is fully selected, this argument will not be given.
- `$ActiveCmdName`: The current command string, which can be `none`, `stretch`, `chop`, `reshape`, or `split`.
- Return: A Boolean value, where:
 - `TRUE`: The object can be selected; and returns the selection status of this object.
 - `FALSE`: The object cannot be selected; the selection status of this object is not returned.

`-stretchProc`

The callback procedure for stretch to define the stretching behavior of the editable PCell instance.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $Inst} {selection $Selection {offset $Offset} }
  {interactive Boolean} }
```

where:

- `$Inst`: `oalnstId` or `oaViald`, which is the operated editable PCell device or via.
- `$Selection`: Selection list of (`deFigure` + `$PointList`), for example:
`{ {$Figure1 $PointList} {$Figure2 $PointList} ...}`, where:

`deFigure` = hierarchy (relative to the editing design) + `oaFigId`

`$PointList` must be specified in a fixed format like `{{x0 y0} {x1 y1} ...}` or `{{x2 y2} {x3 y3} ...}`, depending on the selected figure shape.

If this figure is fully selected, this selected point list is not given.

- `$Offset`: Absolute stretch offset in a fixed format `{dx dy}`. The offset is relative (well-transformed) to the instance's master or the via's master.
 - `interactive`: Indicates whether the command is under interactive (runtime) mode. When it is `FALSE`, it commits the data. The default is `FALSE`.
- Preference `leGRInteractive` enables or disables the interactive mode for guard rings. The default value is `1`.
 - Return: A Boolean value, where:
 - `TRUE`: The procedure directly updates the parameters of the `oalnstId` or `oaViald` for the stretch operation successfully.
 - `FALSE`: The `oalnstId` or `oaViald` is not changeable with the stretching conditions.

`-chopLPPsProc`

The callback procedure for choppable LPPs. This procedure is called in the Chop command to set up a dialog setting for choppable LPPs.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $Inst} }
```

where:

- `$Inst`: `oalnstId` or `oaViald`, which is the operated editable PCell instance or via.

- Return of string with a list of choppable LPPs and the default chop attributes in Boolean type:

```
{ {LayerName1 PurposeName1 Boolean} {LayerName2 PurposeName2
Boolean} ... }
```

where:

- `Boolean` indicates whether the LPP is set as chopped by default in command GUI settings. This field is optional, and the default is `FALSE`.
- When the heal procedure is defined but the healable LPPs procedure is not defined, the tool uses the choppable LPPs to get the healable LPPs.

`-chopProc`

The callback procedure for chop. This procedure is called in the Chop and Auto Chop Guard Ring commands to define the chopping behavior of the editable PCell device.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $Inst} {selection $Selection} {points $PointList} }
```

where:

- `$Inst`: `oalnstld` or `oaViald`, which is the operated editable PCell instance or via.
- `$Selection`: Selection list `{ $Figure1 $Figure2 ... }` of `deFigure`, where:

`deFigure` = hierarchy (relative to the editing design) + `oaFigld`
- `$PointList`: Chop points in a fixed format `{ {x0 y0} {x1 y1} ... }`. The point list is relative (well-transformed) to the instance's master or the via's master.
- Preference `leGuardRingChopLayers` is used for user-selected chopping layers.
- Return: A Boolean value, where:
 - `TRUE`: The procedure directly updates the parameters of the `oalnstld` or `oaViald` and creates new `oalnstlds` or `oaVialds` if necessary for the chop operation.
 - `FALSE`: The `oalnstld` or `oaViald` fails to update with the chopping conditions.

-reshapeProc

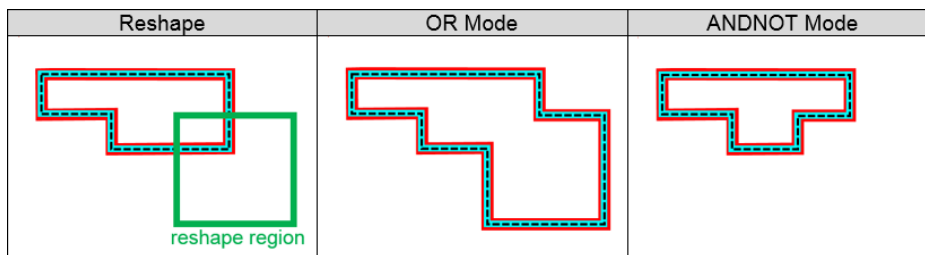
The callback procedure for reshape. This procedure is called in the Reshape command to define the reshaping behavior of the editable PCell instance or via.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $Inst} } {selection $Selection} {points $PointList} }
{interactive Boolean}
```

where:

- `$Inst`: `oalnstId` or `oaViald`, which is the operated editable PCell instance or via.
- `$Selection`: Selection list `{ $Figure1 $Figure2 ... }` of `deFigure`, where:
`deFigure` = hierarchy (relative to the editing design) + `oaFigId`
- `$PointList`: Reshape points in a fixed format `{ {x0 y0} {x1 y1} ... }`. The point list is relative (well-transformed) to the instance's master or the via's master.
- `interactive`: Indicates whether the command is under interactive (runtime) mode. When it is `FALSE`, it commits the data. The default is `FALSE`.
- Preference `letoggleShape` is used for candidate switch. In general, the reshaping result contains at most two candidates: 0, which is for OR mode, and 1, which is for ANDNOT mode.



- Return: A Boolean value, where:
 - `TRUE`: The procedure handles the reshape operation successfully.
 - `FALSE`: The `oalnstId` or `oaViald` fails to update with the reshaping conditions.

-splitProc

The callback procedure of split. The procedure is called in the Split command to define the splitting behavior of an editable PCell instance or via.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $Inst} } {selection $Selection} } {points $PointList}
  {offset $Offset {interactive Boolean} }
```

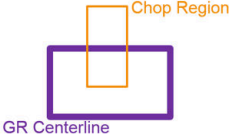
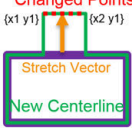
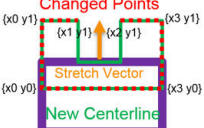
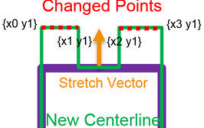
where:

- `$Inst`: `oalnstld` or `oaViald`, which is the operated editable PCell instance or via.
- `$Selection`: Selection list `{ $Figure1 $Figure2 ... }` of `deFigure`, where:
`deFigure` = hierarchy (relative to the editing design) + `oaFigld`
- `$PointList`: Split points in a fixed format `{ {x0 y0} {x1 y1} ... }`. The point list is relative (well-transformed) to the instance's master or the via's master.
- `$Offset`: Absolute stretch offset in a fixed format `{ dx dy }`. The offset is relative (well-transformed) to the instance's master or the via's master.
- `interactive`: Indicates whether the command is under interactive (runtime) mode. When it is `FALSE`, it commits the data. The default is `FALSE`.
- Associated Tcl preferences:
 - Preference `leSplitMode` is used for operand switch. In general, the splitting result contains at most three candidates: 0, which is for inside mode; 1, which is for outside mode; and 2, which is for adjacent mode.
 - Preference `leGRInteractive` enables or disables the interactive mode for guard rings. The default value is 1.
- Return: A list containing 2 elements; the second element is optional.

The first element is a Boolean value, where:

- `TRUE`: The procedure handles the splitting operation successfully.
- `FALSE`: The `oalnstld` or `oaViald` fails to update with the splitting conditions.

The second element is a list of points that indicates the changed points.

| Split - Chop | Mode | Split - Stretch | Output of Split Procedure |
|---|----------|---|---|
|  | Inside |  | 1 { {{x1 y1} {x2 y1}} } |
| | Outside |  | 1 { {{x1 y1} {x0 y1} {x0 y0} {x3 y0} {x3 y1} {x2 y1}} } |
| | Adjacent |  | 1 { {{x0 y1} {x1 y1}} {{x2 y1} {x3 y1}} } |

-healLPPsProc

The callback procedure for healable LPPs. This procedure is called in the Heal command to set up a dialog setting for healable LPPs. With this procedure, the tool can handle healing using proper LPPs.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $Inst} }
```

where:

- \$Inst: oalnstId or oaViald, which is the operated editable PCell instance or via.
- Return of string with a list of healable LPPs and the default heal attributes in Boolean type:

```
{ {LayerName1 PurposeName1 Boolean} {LayerName2 PurposeName2 Boolean} ... }
```

where:

- Boolean indicates whether the LPP is set as healed by default in command GUI settings. This field is optional, and the default is FALSE.

-healProc

The callback procedure for heal. This procedure is called in the Heal operations to define the healing behavior of the editable PCell instance or via.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $Inst} } {selection $Selection} {points $PointList} }
```

where:

- `$Inst`: `oalnstId` or `oaViald`, which is the operated editable PCell instance or via.
- `$Selection`: Selection list `{ $Figure1 $Figure2 ... }` of `deFigure`, where:
`deFigure` = hierarchy (relative to the editing design) + `oaFigId`
- `$PointList`: Heal points in a fixed format `{ {x0 y0} {x1 y1} ... }`. The point list is relative (well-transformed) to the instance's master or the via's master. If this argument is not given, the heal target is the whole instance or via.
- Preference `leGuardRingHealLayers` is used for user-selected healing layers.
- Return: A Boolean value, where:
 - `TRUE`: The procedure handles the healing operation successfully.
 - `FALSE`: The `oalnstId` or `oaViald` fails to update with the healing conditions.

`-cutRemovalHaloProc`

Specify the procedure of cut removal halo

`-cutRemovalProc`

Specify the cut removal procedure

`-directEditProc`

The callback procedure of direct editing filter. The procedure is called in the Stretch command to define whether allow 1-level (1x) direct editing on editable PCells. You cannot edit 1x editable PCells if the procedure is not specified.

- Input with one argument with the following data in a Tcl list:

```
{ {inst $Inst} } {figure $Figure} } {selection {{x0 y0} {x1  
y1} ...}} {command $activeCmdName}}
```

where:

- `$Inst`: `oalnstId` or `oaViald`, which is the operated editable PCell instance or via.
- `$Figure`: Corresponds to `deFigure` that is the object in the operated editable PCell instance or via to be selected, where:

`deFigure = hierarchy (relative to the editing design) + oaFigId`
- `selection`: Selected point list of the figure. This argument is not given if the figure is fully selected.
- `command`: Current command string, which can be `stretch`.
- Return: A Boolean value, where:
 - `TRUE`: Allows to perform 1x direct editing on the object and the selection status of the object.
 - `FALSE`: Not allow to perform 1x direct editing on the object and the selection status of the object.

Returns

{<dmCellView>} : The cellview of the upgraded editable PCell.

Examples

Example 1: Using the Tcl

```
db::makePcellEditable $lib/$cell/layout -pcellType guardring \  
-createProc mpp::grCreationCB \  
-grLPPsProc mpp::grLPPsCB \  
-grContourLPPs { PPLUS NWELL NPLUS POLY } \  
-selectDesc { {stretch line} } \  
-stretchProc mpp::grStretchCB \  
-chopProc mpp::grChopCB \  
-chopLPPsProc mpp::grLayersToChopCB \  
-healLPPsProc mpp::grLayersToHealCB \  
-healProc mpp::grHealCB \  
-reshapeProc mpp::grReshapeCB \  
-splitProc mpp::grSplitCB
```

Example 2: Callback Procedures

```
namespace eval ::mpp {  
  proc grStretchCB { data } {
```

```
#puts "grStretchCB $data"
    array unset selArr
    array set selArr [join $data]
    set selection $selArr(selection)
    set inst $selArr(inst)
    if { [info exists selArr(interactive)] } {
        set interactive $selArr(interactive)
    } else {
        set interactive 0
    }

    set pointlist [db::getParamValue "pointList" -of $inst]
    set pointlist [convert2TclPts $pointlist]

    set closed_path true
    if { [lindex $pointlist 0] != [lindex $pointlist end] } {
        set closed_path false
    }

    set xOriginPitch [expr [db::engToSci [db::getParamValue
xOriginPitch -of $inst]]*1e6]
    set yOriginPitch [expr [db::engToSci [db::getParamValue
yOriginPitch -of $inst]]*1e6]
    set xPitch [expr [db::engToSci [db::getParamValue xPitch -of
$inst]]*1e6]
    set yPitch [expr [db::engToSci [db::getParamValue yPitch -of
$inst]]*1e6]

    #set offset $selArr(offset)
    foreach {dx dy} $selArr(offset) {}
    set dx [expr int($dx/$xPitch)*$xPitch]
    set dy [expr int($dy/$yPitch)*$yPitch]
    set offset [list $dx $dy]

    foreach shape $selection {
        set selSegPoints [lindex $shape 1]
        set result [po::transform $pointlist -selected $selSegPoints
-closed $closed_path -offset $offset -dbu 1000]
#puts "po::transform pointlist=$pointlist selSegPoints=$selSegPoints
closed_path=$closed_path offset=$offset dbu=1000\n\tresult=$result"

        if { $closed_path } {
            set result [linsert $result [llength $result] [lindex
$result 0]]
        }

        set result [db::snapPoints $result -offset [list
$xOriginPitch $yOriginPitch] -spacing [list $xPitch $yPitch]]
        set result [fixPoints $result]
        set result [convertFromTclPts $result]

        db::eval {
            set sb [db::createParamSandbox]
```

```

        if { $interactive == 1 } {
            db::setParamValue "shEnable" -of $inst -value 1
        } else {
            db::setParamValue "shEnable" -of $inst -value 0
        }
        db::setParamValue "pointList" -of $inst -value $result
        stretchChopBoxParam $inst $selSegPoints $offset
    }
}
return 1
}

proc stretchChopBoxParam {inst selSegPoints offset} {
    set chopParamNames {}
    set offsetX [lindex $offset 0]
    set offsetY [lindex $offset 1]
    lappend chopParamNames "m1_chopBox"
    lappend chopParamNames "ct_chopBox"
    lappend chopParamNames "m0po_chopBox"
    foreach chParam $chopParamNames {
        set prevChopList [db::getParamValue "$chParam" -of $inst]
        set prevChopList [convert2TclPts $prevChopList]
        set chopBoxes {}
        foreach prevChopBox $prevChopList {
            set chopEgdes [convertBoxToEdges $prevChopBox]
            if { [po::isVertical $selSegPoints] } {
                set intersection1 [po::getIntersection $selSegPoints
-with [lindex $chopEgdes 2]]
                set intersection2 [po::getIntersection $selSegPoints
-with [lindex $chopEgdes 3]]
            } else {
                set intersection1 [po::getIntersection $selSegPoints
-with [lindex $chopEgdes 1]]
                set intersection2 [po::getIntersection $selSegPoints
-with [lindex $chopEgdes 2]]
            }
            if { $intersection1 != "" || $intersection2 != "" } {
                set newBox [normalizePointList $prevChopBox [list
[expr -$offsetX] [expr -$offsetY]]]
                lappend chopBoxes $newBox
            } else {
                lappend chopBoxes $prevChopBox
            }
        }
        set chopBoxes [convertFromTclPts $chopBoxes]
        db::setParamValue "$chParam" -of $inst -value $chopBoxes
    }
}

```

```

proc convertBoxToEdges {box} {
    set x1 [lindex [lindex $box 0] 0]
    set y1 [lindex [lindex $box 0] 1]
    set x2 [lindex [lindex $box 1] 0]
    set y2 [lindex [lindex $box 1] 1]
    set left [list [list $x1 $y1] [list $x1 $y2]]
    set right [list [list $x2 $y1] [list $x2 $y2]]
    set top [list [list $x1 $y2] [list $x2 $y2]]
    set bottom [list [list $x1 $y1] [list $x2 $y1]]
    return [list $left $right $top $bottom]
}

proc fixPoints {points} {
    if {[llength $points] == 1} {return $points}
    set isClosed false
    if {[lindex $points 0] == [lindex $points end]} {
        set isClosed true
    }
    set pointList [oa::PointArray ]
    foreach p $points {
        foreach {x1 y1} $p {}
            oa::append $pointList [oa::Point $x1 $y1]
        }
    set pointList [oa::compress $pointList false]
    set newList {}
    set num [oa::getNumElements $pointList]
    for {set i 0} {$i<$num} {incr i} {
        set pt [oa::getElement $pointList $i]
        lappend newList $pt
    }
    return $newList
}

proc normalizePointList {pointList {org {}}} {
    set newPointList {}
    if {$org == {}} {
        foreach {offsetX offsetY} [lindex $pointList 0] {}
    } else {
        foreach {offsetX offsetY} $org {}
    }
    foreach pt $pointList {
        set newPtX [expr [lindex $pt 0] - $offsetX]
        set newPtY [expr [lindex $pt 1] - $offsetY]
        lappend newPointList [list $newPtX $newPtY]
    }
    return $newPointList
}

proc grReshapeCB { data } {
    array unset reshapeArr
    array set reshapeArr [join $data]
}

```

```

set points $reshapeArr(points)
set inst $reshapeArr(inst)

if { 0 == [llength $points] } {
    puts "Error : no points on region"
    return 0
}

set pointlist [db::getParamValue "pointList" -of $inst]
set pointlist [convert2TclPts $pointlist]

set centerline [lrange $pointlist 0 end]
set isCLClosed false
if {[lindex $centerline 0] == [lindex $centerline end]} {
    set isCLClosed true
}

if { $isCLClosed } {
    set edgePts [getEdgePoints $centerline]
    set count 0
    set plLinePt "[list [lindex $points end] [lindex $points
end]]"

    set closed_path true
    if { [lindex $points 0] != [lindex $points end] } {
        set closed_path false
    }
    if { !$closed_path } {
        foreach i $edgePts {
            if {[po::getIntersection $i -with $plLinePt] != ""} {
                incr count
            }
            if { [lindex [lindex $i 0] 0] == [lindex [lindex $i
1] 0] && [lindex [lindex $plLinePt 0] 0] == [lindex [lindex $i 1] 0]} {
                set y1 [lindex [lindex $i 0] 1]
                set y2 [lindex [lindex $i 1] 1]
                if { $y1 > $y2 } {
                    set tmp $y1
                    set y1 $y2
                    set y2 $tmp
                }
                set y_pline [lindex [lindex $plLinePt 0] 1]
                if { $y_pline > $y1 && $y_pline < $y2 } {
                    incr count
                }
            }
        }
    }
    if { $count == 0 } {
        de::sendMessage "The end point of the PolyLine needs
to exactly intersect with the centerline of the GuardRing!!!" -severity
warning
        return 0
    }
}

```

```

    }
    #####
    set toggle [db::getPrefValue leToggleShape -scope
[de::getActiveContext]]
    if { $toggle == "normal" } {
        set mode OR
    } else {
        set mode ANDNOT
    }

    set result [po::reshape $centerline -closed $isCLClosed -points
$points -dbu 2000 -mode $mode]
#puts "centerline=$centerline; isCLClosed=$isCLClosed; points=$points"
#set result [le::reshapeGeometryPoints -geometry $centerline -region
$points -design [ed] -toggled $toggle ]
    if {$result == {}} { return 0 }

    set result [convertFromTclPts $result]
    if { 0 == [llength $result] } {
        puts "Warning : centerline not changed for $toggle mode."
        return 0
    } else {
        if { [info exists selArr(interactive)] } {
            set interactive $selArr(interactive)
        } else {
            set interactive 0
        }
        db::eval {
            set sb [db::createParamSandbox]
            if { $interactive == 1 } {
                db::setParamValue "shEnable" -of $inst -value 1
-evalCallbacks false
            } else {
                db::setParamValue "shEnable" -of $inst -value 0
-evalCallbacks false
            }
            db::setParamValue "pointList" -of $inst -value $result
        }
    }
    return 1
}

proc grCreationCB {data} {
    #puts "grCreationCB with $data"
    array unset mppArr
    array set mppArr [join $data]

    if { [info exists mppArr(centerLine)] } {
        set points $mppArr(centerLine)
    } elseif { [info exists mppArr(edgeLine)] } {
        set points $mppArr(edgeLine)
    } else {
        return 0
    }
}

```

```

    }
    set points [join $points]
    set inst $mppArr(inst)
    if { [info exists mppArr(interactive)] } {
        set interactive $mppArr(interactive)
    } else {
        set interactive 0
    }
    set origin [db::getattr inst.origin]
    set cellName [db::getattr inst.master.cellName]
    set xPitch [expr [db::engToSci [db::getParamValue xPitch -of
$inst]]*1e6]
    set yPitch [expr [db::engToSci [db::getParamValue yPitch -of
$inst]]*1e6]
    set finPitch [db::getParamValue finPitch -of $inst]

    set centerLine 1
    set emptyPoints 0
    if { [info exists mppArr(centerLine)] } {
        set points $mppArr(centerLine)
    } elseif { [info exists mppArr(edgeLine)] } {
        set points $mppArr(edgeLine)
        set centerLine 0
    } else {
        return 0
    }
    set points [join $points]
    if { [llength $points] == 0 } {
        set emptyPoints 1
    }

    if { $emptyPoints == 0 && $centerLine == 0 } {
# Force to set the pointList as a workaround for layer width properties
        db::setParamValue "pointList" -of $inst -value {(0 0) (0 5)
(5 5)}

        set leAutoGRLayerSpacing [db::getPrefValue
leAutoGRLayerSpacing]
        set overSizeOffset_x 0
        set overSizeOffset_y 0
        set leAutoGRLayerSpacing_final ""
        set boundary [db::getPrefValue leAutoGRSpacingMethod]
        if { true == $boundary } {
            set grName $cellName
            set leAutoGRLayerSpacing_final "AsIs { DIFF M1 PPLUS
NPLUS }"
        } else {
            foreach leAutoGRLayerSpacing_tmp $leAutoGRLayerSpacing {
                set grName [lindex [split [lindex
$leAutoGRLayerSpacing_tmp 0] "/" ] 1]
                if { $grName == $cellName } {
                    set leAutoGRLayerSpacing_final
$leAutoGRLayerSpacing_tmp

```

```

    }
  }
}

foreach autoGRLayerRule [lindex $leAutoGRLayerSpacing_final
1] {
    set layerName [lindex [lindex $autoGRLayerRule 0] 0]
    if { $layerName != "stretch" } {
        if { $layerName == "PPLUS" || $layerName == "NPLUS" }
        {
            set propName_SegV "master.IMP_Width_SegV"
            set propName_SegH "master.IMP_Width_SegH"
        } else {
            set propName_SegV
"master.$layerName\_Width_SegV"
            set propName_SegH
"master.$layerName\_Width_SegH"
        }

        set layerWidth_x 0
        if { ![catch { set layerWidth_x [db::getAttr
$propName_SegV -of $inst]] } } {
            #puts "db::getAttr $propName_SegV $layerWidth_x"
            set layerWidth_x [expr $layerWidth_x/2]
            if { $layerWidth_x > $overSizeOffset_x } {
                set overSizeOffset_x $layerWidth_x
            }
        }

        set layerWidth_y 0
        if { ![catch { set layerWidth_y [db::getAttr
$propName_SegH -of $inst]] } } {
            #puts "db::getAttr $propName_SegH $layerWidth_y"
            set layerWidth_y [expr $layerWidth_y/2]
            if { $layerWidth_y > $overSizeOffset_y } {
                set overSizeOffset_y $layerWidth_y
            }
        }
    }
}

set newOffsetX [snapToGrid $overSizeOffset_x $xPitch]
set newOffsetY [snapToGrid $overSizeOffset_y $yPitch]
if { $newOffsetX < $overSizeOffset_x } {
    set newOffsetX [expr $newOffsetX + $xPitch]
}
if { $newOffsetY < $overSizeOffset_y } {
    set newOffsetY [expr $newOffsetY + $yPitch]
}

set poPointSet [po::createPointSet $points -dbu 2000]
set poPointSet [po::size $poPointSet -by [list $newOffsetX
0]]

```



```

        set poPointSet [po::size $poPointSet -by [list 0
$newOffsetY]]

        # Growing & Shrinking pointlist to honor notch
        set notchVal [expr ([db::getPrefValue leAutoGRNotchValue
-scope [de::getActiveContext]]/2.0)]
        set poPointSet [po::size $poPointSet -by $notchVal]
        set poPointSet [po::size $poPointSet -by [expr (-1.0 *
$notchVal)]]

        set points [po::getPoints $poPointSet]
#A proper dbu is 2000 for this tech but due to issue 9001200258 and the
snapping scale of guard ring centerline is 0.001,
#so, dbu 1000 is used here
#puts "po::transform pointlist=$points selSegPoints=$points closed_path=1
offset=$newOffsetX $newOffsetY dbu=1000"
        set points [po::transform $points -selected $points -closed
true -offset [list $newOffsetX $newOffsetY] -dbu 1000]
#puts "\tresult=$points"
        set points [linsert $points [llength $points] [lindex $points
0]]

        foreach {originX originY} [db::getAttr origin -of $inst] {}
        set newOrigin [list [expr $originX-$newOffsetX] [expr
$originY-$newOffsetY]]
        db::setAttr origin -of $inst -value $newOrigin
    }

    set points [convertFromTclPts $points]
    db::eval {
        set sb [db::createParamSandbox]
        db::setParamValue "pointList" -of $inst -value $points
        if { $interactive == 1 } {
            db::setParamValue "shEnable" -of $inst -value 1
        } else {
            db::setParamValue "shEnable" -of $inst -value 0
        }
    }
    set codeversion [db::getParamValue "codeversion" -of $inst]
    db::setParamValue "source_codeversion" -of $inst -value
$codeversion -evalCallbacks false
}
return 1
}

proc grLPPsCB {data} {
    set l1 [list "DIFF" "drawing"]
    set l2 [list "M1" "drawing"]
    set l3 [list "PPLUS" "drawing"]
    set l4 [list "NPLUS" "drawing"]
    #set l4 [list "stretch" "drawing"]
    return [list $l1 $l2 $l3 $l4 ]
}

```

```

    }

    proc grLayersToChopCB { data } {
        set l2 [list "M1" "drawing" 1]
        set l3 [list "VIA0" "drawing"]
        set l4 [list "MOPOLY" "drawing"]
        return [list $l2 $l3 $l4 ]
    }

    proc grLayersToHealCB { data } {
        set l2 [list "M1" "drawing" 1]
        set l3 [list "VIA0" "drawing"]
        set l4 [list "MOPOLY" "drawing"]
        return [list $l2 $l3 $l4 ]
    }

    proc grChopCB {data} {
        #puts "grChopCB $data"
        array unset chopArr
        array set chopArr [join $data]
        set chopLayers [db::getPrefValue leGuardRingChopLayers -scope
[de::getActiveContext ]]

        set points $chopArr(points)
        set inst $chopArr(inst)

        set chopParamNames {}
        foreach chopLayer $chopLayers {
            switch $chopLayer {
                {M1 drawing} {
                    lappend chopParamNames "m1_chopBox"
                }
                {VIA0 drawing} {
                    lappend chopParamNames "ct_chopBox"
                }
                {MOPOLY drawing} {
                    lappend chopParamNames "m0po_chopBox"
                }
            }
        }

        set codeversion [db::getParamValue "codeversion" -of $inst]
        db::setParamValue "source_codeversion" -of $inst -value
$codeversion -evalCallbacks false
        db::eval {
            set sb [db::createParamSandbox]
            foreach chParam $chopParamNames {
                set prevChopList [db::getParamValue "$chParam" -of $inst]
                set prevChopList [convert2TclPts $prevChopList]
                set chopBoxes $prevChopList
                set chopBoxes [modifyChopPointList $inst $chopBoxes
$points]

                set chopBoxes [convertFromTclPts $chopBoxes]
            }
        }
    }

```

```

        db::setParamValue "$chParam" -of $inst -value $chopBoxes
    -evalCallbacks false
    }
    }
    return 1
}

proc grHealCB {data} {
    #puts "grHealCB : $data"
    array unset healArr
    array set healArr [join $data]

    set healLayers [db::getPrefValue leGuardRingHealLayers -scope
[de::getActiveContext ]]
    set chopParamNames {}
    foreach chopLayer $healLayers {
        switch $chopLayer {
            {M1 drawing} {
                lappend chopParamNames "m1_chopBox"
            }
            {VIA0 drawing} {
                lappend chopParamNames "ct_chopBox"
            }
            {MOPOLY drawing} {
                lappend chopParamNames "m0po_chopBox"
            }
        }
    }
    if { 0 == [llength $chopParamNames] } {
        return 0
    }

    set inst $healArr(inst)
    set codeversion [db::getParamValue "codeversion" -of $inst]
    db::setParamValue "source_codeversion" -of $inst -value
$codeversion -evalCallbacks false

    set healPoints $healArr(points)
    db::eval {
        set sb [db::createParamSandbox]
        if { $healPoints == {} } {
            foreach chParam $chopParamNames {
                db::setParamValue $chParam -value {} -of $inst
            }
        } else {
            foreach chParam $chopParamNames {
                set prevChopList [db::getParamValue "$chParam" -of
$inst]
                set prevChopList [convert2TclPts $prevChopList]
                set chopBoxes $prevChopList
                set chopBoxes [modifyHealPointList $inst $chopBoxes
$healPoints]
            }
        }
    }
}

```

```

        set chopBoxes [convertFromTclPts $chopBoxes]
        db::setParamValue "$chParam" -of $inst -value
$chopBoxes -evalCallbacks false
    }
}
}
return 1
}

proc modifyHealPointList {inst chopPointList points} {
    set chopPointSet [rectListToPointSet $chopPointList]
    set chopBoxSet [po::createPointSet $points -dbu 2000]
    set resultPointSet [po::not $chopBoxSet -from $chopPointSet]
    set result [resultPointSetToRect $resultPointSet ]
    return $result
}

proc modifyChopPointList {inst chopPointList points} {
    set chopPointSet [rectListToPointSet $chopPointList]
    set chopBoxSet [po::createPointSet $points -dbu 2000]
    set resultPointSet [po::or $chopBoxSet -with $chopPointSet]
    set result [resultPointSetToRect $resultPointSet ]
    return $result
}

proc rectListToPointSet {chopPointList} {
    set chopPointSet [po::createPointSet -dbu 2000]
    foreach chopBox $chopPointList {
        foreach {x y x1 y1} [join $chopBox " "] {}
            set chopBoxPoints [list [list $x $y] [list $x1 $y] [list $x1
$y1] [list $x $y1] ]
            po::append $chopBoxPoints -to $chopPointSet -dbu 2000
        }
    }
    return $chopPointSet
}

proc resultPointSetToRect {resultPointSet} {
    set pts [po::getPoints $resultPointSet]
    set resultBoxList {}
    while {$pts != ""} {
        set rects [mpp::convertToRect $pts]
        foreach box $rects {
            lappend resultBoxList $box
        }
        set pts [po::getPoints $resultPointSet]
    }
    return $resultBoxList
}

proc convertToRect {points} {
    set xPoints {}
    set yPoints {}

```

```

    set pointSet [po::createPointSet $points -dbu 2000]
    foreach p $points {
        lappend xPoints [lindex $p 0]
        lappend yPoints [lindex $p 1]
    }
    set xPoints [lsort -unique $xPoints]
    set yPoints [lsort -unique $yPoints]
    set y1 [lindex $yPoints 0]
    set y2 [lindex $yPoints end]
    set rectBoxes {}
    for {set i 0} {$i < [expr [llength $xPoints]-1]} {incr i} {
        set x1 [lindex $xPoints $i]
        set x2 [lindex $xPoints [expr $i+1]]
        set bBox [list [list $x1 $y1] [list $x2 $y1] [list $x2 $y2]
[list $x1 $y2]]
        set bBox [po::createPointSet $bBox -dbu 2000]
        set andBox [po::and $bBox -with $pointSet]
        set boxPoints [po::getPoints $andBox]
        set bBox [list [lindex $boxPoints 1] [lindex $boxPoints 3]]
        lappend rectBoxes $bBox
    }
    return $rectBoxes
}

proc grSplitCB { data } {
    #puts "grSplitCB $data"
    array unset selArr
    array set selArr [join $data]

    set chop_region $selArr(points)
    if { 0 == [llength $chop_region] } {
        puts "Error : no points on region"
        return 0
    }
    set inst $selArr(inst)
    foreach {offsetx offsety} $selArr(offset) {}
        set xPitch [expr [db::engToSci [db::getParamValue xPitch -of
$inst]]*1e6]
        set yPitch [expr [db::engToSci [db::getParamValue yPitch -of
$inst]]*1e6]
        #set offsetx [expr int($offsetx/$xPitch)*$xPitch]
        #set offsety [expr int($offsety/$yPitch)*$yPitch]
        set offset [db::snapPoints [list [list $offsetx $offsety]]
-offset [list 0 0] -spacing [list $xPitch $yPitch]]
        set offset [lindex $offset 0]
        set mode [db::getParamValue leSplitMode -scope
[de::getActiveContext]]

        set centerline [db::getParamValue "pointList" -of $inst]
        set centerline [convert2TclPts $centerline]
        set isClosed false
        if {[lindex $centerline 0] == [lindex $centerline end]} {
            set isClosed true
        }
    }
}

```

```

    }

    #puts "po::split $centerline -closed $isClosed -points $chop_region
    -offset $offset -dbu 2000 -mode $mode"
    set _result [po::split $centerline -closed $isClosed -points
    $chop_region -offset $offset -dbu 2000 -mode $mode]
    #puts "po::split result $_result"
    set cnt [llength $_result]
    if { $cnt == 0 } {
        return 0
    }

    set result [lindex $_result 0]
    if { 0 == [llength $result] } {
        #puts "centerline not changed..."
        return 0
    } else {
        db::eval {
            set sb [db::createParamSandbox]
            if { [info exists selArr(interactive)] } {
                set interactive $selArr(interactive)
            } else {
                set interactive 0
            }
            if { $interactive == 1 } {
                db::setParamValue "shEnable" -of $inst -value 1
            } else {
                db::setParamValue "shEnable" -of $inst -value 0
            }
            if { $offsetx == 0.0 && $offsety == 0.0 } {
                #puts "centerline should not change with zero offset..."
            } else {
                set result [convertFromTclPts $result]
                db::setParamValue "pointList" -of $inst -value
                $result
            }
        }
    }
    if { $cnt > 1 } {
        return [list 1 [lindex $_result 1]]
    }

    return 1
}

proc getEdgePoints {ptList} {
    set ptListNew ""
    for {set i 0} {$i<[llength $ptList]-1} {incr i} {
        lappend ptListNew "[concat \{[lindex $ptList $i]\}
        \{ [lindex $ptList $i+1]\}]"
    }
}

```

```
        return $ptListNew
    }

    proc convert2TclPts {pointlist} {
        set pointlist [string map {( \{ } $pointlist]
        set pointlist [string map { } \}} $pointlist]
        return $pointlist
    }

    proc convertFromTclPts {pointlist} {
        set pointlist [string map {\{ ( } $pointlist]
        set pointlist [string map {\} )} $pointlist]
        return $pointlist
    }
}
```

db::mapStreamName

Description

Maps the given stream name to a streamed object.

Syntax

```
db::mapStreamName <streamName: string> -map <dbStreamNameMap>
```

Arguments

streamName

Stream name to map.

-map

The stream name that is used for a streamed object map.

Returns

<dbStreamedObject>: Mapped streamed object.

db::match

Description

Determine whether or not the pattern matches the string.

Syntax

```
db::match [<text: string>] [-pattern <string>] [-noCase <bool>]
```

Arguments

text

the text (Default is:)

-pattern

Space delimited wildcard pattern (Default is: *)

-noCase

If true, then the pattern attempts to match against the string in a case insensitive manner. The default is false.

Returns

<bool> : Returns true if the given pattern matches the string, or false if it does not.

Examples

```
# Set the example of the net
set netname "net_12<7:0>"

# Set the matching pattern
set pattern "net_*<*:*>"

# Match the string with the pattern
puts [db::match $netname -pattern $pattern]
```

db::md5sum

Description

Returns the MD5 checksum for the given file.

Syntax

```
db::md5sum <filename: string>
```

Arguments

filename

File for which to calculate checksum.

Returns

<string> : md5 checksum

Examples

```
# Set the layout design file
set fileName [file normalize ./DesignLibrary/amp/layout/layout.oa]
```



```
# get the checksum of the file. It can be used to check if the file was modified.
puts [db::md5sum $fileName]
```

db::monitorChannel

Description

Processes read events on the given channel id as soon as the data is available for read.

Syntax

```
db::monitorChannel <channelId: string>
```

Arguments

channelId

channel to watch for events

db::notify

Description

Calls listeners of a notifier.

Syntax

```
db::notify <notifier: dbNotifier> [-args [<string>]]
```

Arguments

notifier

Notifier whose listeners are to be called.

-args

Arguments to be passed to listeners.

Returns

<none>

Errors

-args is not compatible with the registered arguments %s.

Examples

```
# Create an object
set obj [db::createObject ]

# Add attributes to the object
db::addAttr name -of $obj -valueType string -value "nmos"
```

```

db::addAttr l -of $obj -valueType double -value 1u
db::addAttr w -of $obj -valueType double -value 0.1u

# Create notifiers for each attribute. Each notifier notifies the object
# to set its attribyte value
set n1 [db::createNotifier nmosAttrSet_l -to $obj -args string \
    -description "NMOS L parameter set notifier"]
set n2 [db::createNotifier nmosAttrSet_w -to $obj -args string \
    -description "NMOS W parameter set notifier"]

# Set listeners to the notifiers
db::addListener [list nmosValueSet_l $obj ] -to $n1
db::addListener [list nmosValueSet_w $obj ] -to $n2

# Create procedures which handles object attribute
proc nmosValueSet_l {obj value} {
    puts "This is nmos l attribute set listener proc, the value is $value"
    if {$value >= 0} {
        db::setAttr obj.l -value $value
    }
}

# Create procedures which handles object attribute
proc nmosValueSet_w {obj value} {
    puts "This is nmos w attribute set listener proc, the value $value"
    if {$value >= 0} {
        db::setAttr obj.w -value $value
    }
}

# Get the object notifier
set n [db::getNotifiers nmosAttrSet_l -of $obj]
# Notify the object to set the l attribute
db::notify $n -args 5u
# Check if the attribute is set
puts [db::getAttr obj.l]

# Get the object notifier
set n [db::getNotifiers nmosAttrSet_w -of $obj]
# Notify the object to set the w attribute
db::notify $n -args 0.5u
# Check if the attribute is set
puts [db::getAttr obj.w]

```

db::openDesign

Description

Opens a design.

Syntax

```

db::openDesign <cellView: dmCellView | dmCellViewName> [-enableUndo
    <bool>] [-readOnly <bool>]

```

Arguments

`cellView`

CellView for which to open design

`-enableUndo`

Enable undo flag. The default is false.

`-readOnly`

Read-only mode. The default is false.

Returns

`<oaDesign>` : Newly opened design.

Examples

The following example opens DemoPLL/and2/layout design.

```
db::openDesign DemoPLL/and2/layout
```

db::openHier

Open a design to the specified depth. If `-depth` is not provided, reopens the design hierarchy to the last used level.

Syntax

```
db::openHier <design: oaDesign> [-depth <int>]
```

Returns

`<none>`

Arguments

`design`

`oaDesign`.

`-depth`

Hier depth.

db::openDesignHierarchy

Description

Opens a design hierarchy.

Syntax

```
db::openDesignHierarchy <design: oaDesign> [-depth <int>]
```

Arguments

`design`

OA Design open depth.

`-depth`

Depth to open design.

Default: 32.

Returns

<none>

Examples

The following example opens the design for given cell view.

```
set cellView [dm::getCellViews layout -cellName pll -libName DemoPLL]
set design [db::openDesign $cellView]
db::openDesignHierarchy $design -depth 20
```

db::openParamDefEditor

Description

Opens the Parameter Definition Editor.

Syntax

```
db::openParamDefEditor [-cell <dmCell>] [-libName <string>] [-readOnly  
  <bool>]
```

Arguments

`-cell` `dmCell`

Opens the parameter definition editor for the dmCell.

Default: NULL.

`-libName` <string>

Edit parameter definitions of named library.

Default: "".

`-readOnly <bool>`

Open parameter definitions view only.

The default is false.

Returns

New Parameter Definition Editor ([giWindow](#)).

Errors

If both `-cell` and `-libName` are specified.

Examples

```
# Options '-cell' and '-libName' are mutually exclusive and they all
# cannot be specified
set cell [dm::getCells amp -libName DesignLibrary]
db::openParamDefEditor -cell $cell

# Or
db::openParamDefEditor -libName DesignLibrary
```

db::openScope

Description

Creates the OpenAccess scope and returns it. The scope is guaranteed to exist until all holders of this scope release their objects referencing it.

Syntax

```
db::openScope <object: dmCell | dmCellName | dmCellView | dmCellViewName
| oaCell | oaCellView | oaLib>
```

Arguments

`object`

OA or DM object for which the scope is opened.

Returns

<dbScopeType>: Newly opened scope.

Examples

```
# Open scope for DemoPLL
set scope [db::openScope [dm::findCellView schematic -cellName inv
    -libName DemoPLL]]

# Change preference value in the new scope
db::setPrefValue seWireNameColor -value green -scope $scope
```

```
# Print preference value in new scope, it will be green
puts [db::getPrefValue seWireNameColor -scope $scope]

# Print preference value in user scope, it will be <auto>
puts [db::getPrefValue seWireNameColor -scope user]
```

db::pdaExplainCell

Description

Explains the cell inherited information of the tree widget in the **Power Domain Analyzer** (PDA) dialog box.

Syntax

```
db::pdaExplainCell <dialog: giDialog> [-number <int>]
```

Arguments

dialog

PDA dialog.

-number

The inherited information to explain. The default is 0.

db::pdaShowCell

Description

Shows the cell of a tree widget.

Syntax

```
db::pdaShowCell <dialog: giDialog> [-number <int>]
```

Arguments

dialog

PDA dialog.

-number

The inherited information to show. The default is 0.

db::print

Description

Sends a print job to the printer or to the specified file using Postscript format. When an `oaFile` is supplied, it must be of text format.

If the output is to a file, the command returns a NULL pointer, because the Job Manager is not used.

The `-depth` argument is ignored if the positional argument is not a `dmCellView`. When an `oaLib` or `dmCell` is supplied, output of all non-layout data is with a depth of 0, and output of all layout data is with a depth of All.

The `-hierMode` argument is ignored if the `-positional` argument is not a `dmCellView` or the depth is not greater than 0. The `-viewlist` argument is only used if the `-hierMode` argument applies and is set to *inContext* and the `CellView` on which *hierarchical* printing is being performed is not of type *config*.

The `-grid`, `-rulerLeft`, `-rulerRight`, `-rulerTop`, and `-rulerBottom` arguments are ignored if the `-positional` argument is not an `oaLib`, `dmCell` or `dmCellView`.

The `-copies` and `-doubleSided` arguments are ignored if the `-to` argument is `file` and the `-overwrite` argument is ignored if the `-file` argument is supplied.

The `-doubleSided` argument is ignored if the `-scale` argument is greater than 1.

The `rulerMajorTicks`, `rulerMinorTicks`, `rulerMajorCoords` and `rulerMinorCoords` are ignored if none of the rulers (`rulerLeft`, `rulerRight`, `rulerTop` or `rulerBottom`) are enabled or the `-positional` argument is not an `oaLib`, `dmCell` or `dmCellView`.

The `title`, `titleText`, `comment` and `commentText` are ignored if the `-note` argument is disabled. Further the `titleText` and `commentText` are ignored if the `-title` or `-comment` arguments, respectively, are set to default.

The `olpGroup`, `rulerLeft`, `rulerRight`, `rulerTop`, `rulerBottom`, `rulerMajorTicks`, `rulerMinorTicks`, `rulerMajorCoords` and `rulerMinorCoords` are only applicable to data of type layout.

The `-fontSize` argument applies only to the fonts generated as a result of the Ruler, Note, and Legend functionality, and thus if none of the arguments related to that functionality are applicable, neither is the `fontSize`.

When printing text (`{<oaFile>}` or `{<string>}`), only the following options apply (all others are silently ignored): `printer`, `file`, `overwrite`, `copies`, `orientation`, `paperSize`, `doubleSided`, `leftMargin`, `rightMargin`, `topMargin`, and `bottomMargin`.

Syntax

```
db::print <context: string | {<dmCell>} | {<dmCellView>} | {<giWindow>}  
| {<oaLib>}> -viewSwitchList <string> -includeBackground <bool>
```

```
[-scaleFit <enum> | -scaleFactor <float>] -printer <dbPrinter> [-depth  
<int>] [-hierMode <enum>] [-libFilter <string>] [-cellFilter <string>]  
[-viewFilter <string>] [-file <string>] [-fileType <enum>] [-overwrite  
<bool>] [-copies <int>] [-orientation <enum>] [-paperSize <string>]  
[-doubleSided <bool>] [-leftMargin <float>] [-rightMargin <float>]  
[-topMargin <float>] [-bottomMargin <float>] [-display <string>]  
[-colorMode <enum>] [-invertBrightness [<bool>]] [-fontScale <float>]  
[-title <bool>] [-legend <enum>] [-titlePosition <enum>] [-fontSize  
<int>] [-grid <bool>] [-borderRulers <bool>] [-bookmarks <bool>]  
[-window <{<giWindow>}>] [-sheets [<string>]]
```

Arguments

context

The CellView, window, or file path to output.

-printer

The printer to which the output should be sent. If exporting to a file, a printer must be supplied to define the necessary printing properties such as color, paperSize, and so on.

-depth

The hierarchy depth.

-hierMode

Mode to use when printing hierarchical data ({ all | inContext }).

-viewSwitchList

The viewSwitchList to use when printing a non-config CellView hierarchically.

-libFilter

Library filter for cellviews printed indirectly. That is, when a cellview is printed directly with a reference from a printed design.

-cellFilter

Cell filter for cellviews printed indirectly. That is, when a cellview is printed directly with a reference from a printed design.

-viewFilter

View filter for cellviews printed indirectly. That is, when a cellview is printed directly with a reference from a printed design.

-file

If supplied, the output is directed to this file in Postscript format using the properties of the supplied printer. (Default: ".")

`-fileType`

Output file format ({ pdf | pdfWithLinks | ps}).

`-overwrite`

If outputting to file, controls if an existing file is overwritten.

`-copies`

The number of copies to output.

`-orientation`

The orientation to use ({auto | landscape | portrait}).

`-paperSize`

The paper size to use.

`-doubleSided`

Enable or disable double-sided printing.

`-leftMargin`

Sets the left margin.

`-rightMargin`

Sets the right margin.

`-topMargin`

Sets the top margin to use.

`-bottomMargin`

Sets the bottom margin.

`-display`

The display to use when outputting. If an empty string is supplied, the printer's defined display is used. If not defined, an error occurs. (Default: "").

`-colorMode`

Prints in color, grayscale, or black and white ({ blackAndWhite | color | grayscale }).

`-includeBackground`

Specifies whether to include background color when printing.

`-invertBrightness`

Specifies whether to darken light colors and brighten dark colors.

`-scaleFit`

The fit mode to use ({height | page | width}).

`-scaleFactor`

The scaling factor to use.

`-fontScale`

An additional scale factor for fonts.

`-legend`

Enables the legend and control its location ({ bottom | disabled | left | right | separatePage | top }).

`-titlePosition`

Enables or disables the title and control its location. Valid values are bottom, disabled, left, right, and top.

`-fontSize`

The font size for the created text, based on the Print configuration options.

`-grid`

Enables the grid.

`-borderRulers`

Enables the rulers on the sides of the page.

`-bookmarks`

Enables content bookmark printing in hierarchy mode.

`-window`

The design window that is used to print selected cellviews.

`-sheets`

Sheets of multi sheet design to print.

Returns

<xtJob>: The print job created.

Errors

- dmCellView does not exist.
- display is not provided and supplied dbPrinter doesn't have a default display defined.
- The supplied paperSize is not supported by the dbPrinter.

- Output is directed to a printer and doubleSided is 'true' but the dbPrinter does not support it.
- Output is directed to a printer and color is 'true' but the dbPrinter does not support it.
- A paperSize is not supplied.
- The specified file exists and overwrite is false.
- Supplied printer is not available.
- Supplied window does not contain layout, schematic or symbol oaDesign data.

Examples

```
db::print [dm::getCellViews schematic -cellName amp -libName PLL] \  
-printer [db::getPrinters defaultPS] -file amp.ps -paperSize A1 -overwrite true
```

db::purgeCDF

Description

Purges component description format (CDF) data for a cell or library.

Syntax

```
db::purgeCDF <data: dmCell | oaLib> -level <enum> [-removeCDF <bool>]
```

Arguments

data

Specifies the location to purge the CDF from the oaDMData.

-removeCDF

Specifies whether the 'cdfData' property is to be removed from the oaDMData.

The default is false.

-level

The CDF level from which parameters will be exported ({ base | memory | overlay }, default is: base)

Returns

<none>

Examples

```
# Get the cell which CDF data need to purge  
set dmCell [dm::getCells n_4t -libName reference40nm]  
  
# Purge the CDF data of the cell
```

```
db::purgeCDF $dmCell -removeCDF true -forceMaster true

# Save the CDF data File
db::saveCDF $dmCell
```

db::purgePrefs

Description

Removes any active and saved preference data at the supplied scope or from their scopes. If a scope is provided, all preference values at that scope are purged. If an environment argument is provided, all preferences from all scopes from the supplied environment are purged and the environment is destroyed. The system environment cannot be destroyed.

Note that if a preference was defined by the system to be localized to the given scope, it is reset to its inherited value.

It is not possible to delete preferences from the install or context scopes as these scopes do not have a default on-disk location. Specifying one of these scopes or a preference from one of these scopes causes an error.

Syntax

```
db::purgePrefs <argument: dbCollection | string>
```

Arguments

argument

Collection of preferences, scopes, or an environment to purge.

Returns

<none>

Examples

The following example purges all preferences stored in the PLL library, cells and cellViews.

```
set lib [dm::getLibs PLL]db::foreach cell [dm::getCells -lib $lib] {
    db::foreach view [dm::getCellViews -cell $cell] {
        db::purgePrefs [db::openScope $view]
    }
    db::purgePrefs [db::openScope $cell]}
db::purgePrefs [db::openScope $lib]
```

db::query

Description

A non-interactive command. Generates a human readable report of the attributes of any collection of objects. All the product objects and certain OpenAccess objects support the `db::query` command. If `db::query` is not supported for a given object, returns an empty string. If more than one object is supplied in the input collection, returns a concatenation of output strings for each object.

Syntax

```
db::query <object: [dbCollection]> [-verbose <bool>]
```

Arguments

`object`

The object(s) to query, a list of objects or db collections.

`-verbose`

Query collections for object(s).

The default is false.

Returns

The report (<string>).

Examples

The following example reports attributes of the selected object in the given design.

```
db::query [de::getSelected -design $oaDesign]
```

db::queryFeature

Description

Returns true if a license feature is checked out. If the feature is not recognized, an error is thrown.

Note that the license name is an internally defined license name which may refer to one or more features on the actual license server. Use `db::getFeatures` to determine which features can be checked out.

Syntax

```
db::queryFeature <name: string>
```

Arguments

<string>

Name of the licensing feature to query.

Returns

True if the license is checked out, false otherwise (<bool>).

Errors

- Feature does not exist.
- Invalid authentication key.

db::raiseLicenseTier

Description

Raises the current tier.

Syntax

```
db::raiseLicenseTier <tier: enum>
```

Arguments

tier

Tier name to check. ({ advanced | advancedPlus | apex | backend | basic | elite | frontend | mtm | router | schematic })

Examples

The following example checks out the backend license if it or one of the above licenses is not checked out.

```
db::raiseLicenseTier backend
```

db::refreshICCCache

Description

Refreshes (creates or updates) the OpenAccess translation cache for layout designs in an ICC library.

Syntax

```
db::refreshICCCache [<cellViews: {<dmCellView>}>] [-force <bool>]
```

Arguments

`cellViews`

Collection of `cellViews` to refresh.

Default: All `cellViews` in all open libraries.

`-force`

When true, all `cellViews` are retranslated even if the cache is current.

The default is false.

Returns

{<dmCellView>}: Collection of `cellViews` that were refreshed.

db::refreshMCells

Description

Refreshes the subMasters of an MCell.

Syntax

```
db::refreshMCells (-lib <{<oaLib>}> | -libName <dmLibName>) -masters  
  <{<dmCellView>}>
```

Arguments

`-masters`

The superMaster `dmCellViews`.

`-lib`

OA libraries that their instances of the MCell and editable PCell with specified superMasters will be refreshed.

`-libName`

Names of OA libraries that their instances of the MCell and editable PCell with specified superMasters will be refreshed.

Examples

```
# Refresh reference40nm/nwellGR cell subMasters  
db::refreshMCells -libName reference40nm -masters [dm::findCellView  
  layout -cellName nwellGR -libName reference40nm]
```

db::registerCallbackRegsub

Description

Registers a regular expression substitution with the Callback Syntax evaluator.

Syntax

```
db::registerCallbackRegsub <regexp: string> -substitute <string>
```

Arguments

<string>

The regular expression to use for pattern matching.

-substitute <string>

The pattern to use for substitutions.

db::registerGenerator

Description

Registers a cellView from a cellView generator. If a generator was already registered for the specific source and destination combination, this new generator replaces the existing generator.

A generator option set type can optionally be specified. This setting indicates which option set type is supported by this generator. In addition, a preference name can be given for using to select a default set name of the supported option set type. If the preference does not exist, an error is generated, and the generator is not registered.

If an option set type is specified, an optional command can be specified with the `-optionEditCommand` argument. This should correspond to a command dialog, which can be used to edit the contents of an option set of the supported option set type.

If the registered generator supports the updating of an existing destination design, the `-supportsUpdates` argument should be used and set to true. By default, it is assumed that generators do not support updating and simply regenerate. Regenerated cellViews are purged, and the data is then re-created.

If the registered generator uses the side attribute from the pin spec objects of the interface set, `-usesPinSides` can be set to true. When enabled, this allows the user to manipulate the side attribute for each pin before passing the interface set to the generator procedure.

Syntax

```
db::registerGenerator <name: string> -for <string> [-of <string>]  
[-optionSetType <string>] [-defaultOptionSetPref <string>]
```



```
[-optionEditCommand <string>] [-supportsUpdate <bool>] [-usesPinSides  
<bool>]
```

Arguments

<string>

Name of the cellView generator Tcl procedure.

-for

Destination viewtype name for the generator.

-of

Source viewtype name for the generator.

-optionSetType

Optional name for the type option set supported by this generator. If no type is given, no option set is supported.

-defaultOptionSetPref

Name of preference which holds the default set name.

-optionEditCommand

Optional command which can be invoked from the
`db::showCellViewFromCellView` command.

-supportsUpdate

Optional setting which indicates that the generator supports updating an existing destination design.

The default is false.

-usesPinSides

Indicates whether this generator uses the side information from pin specs. If true, the pin adjustment is allowed from the `db::showCellViewFromCellView` command.

The default is false.

Examples

```
# Unregister previous cellview generator for symbol view
db::unregisterGenerator -for schematicSymbol

# Register new cellview generator for symbol view, which adds copyright
label in symbol view
db::registerGenerator createSymbol \
    -for schematicSymbol \
    -optionSetType symbolGenerator \
```

```

        -usesPinSides true \
        -defaultOptionSetPref seSymbolGenOptionSetDefault \
        -supportsUpdate true
proc createSymbol {args} {
    array set opts $args
    set iface $opts(-interface)
    set gos $opts(-options)
    set dest $opts(-dest)
    se::generateSymbol -interface $iface -dest $dest -options $gos
    -update false
    set symbolLCV [file join [db::getAttr dest.libName] [db::getAttr
dest.cellName] [db::getAttr dest.name]]
    set design [db::openDesign $symbolLCV -readOnly true]
    de::createContext $design
    se::createText "Copyright (c) Synopsys" -anchor {0 0} -design $design
    -type note
    de::save $design
    db::closeDesign $design
}

# Print registered generator name for symbol view
db::findGenerator -for schematicSymbol

```

db::registerInterfaceReader

Description

Registers the interface reader for the specified view type.

Syntax

```
db::registerInterfaceReader <name: string> -viewType <string>
```

Arguments

<string>

Tcl procedure name to call and retrieve the interface set object for designs of the given view type.

-viewType

Source viewType name supported by the given reader.

Examples

```

# Find and print interface reader tcl proc name for layout view
set int_reader [db::findInterfaceReader -viewType maskLayout]
puts $int_reader

# Unregister an interface reader
db::unregisterInterfaceReader -viewType maskLayout

```

```
# Register "new_proc" as the interface reader procedure
db::registerInterfaceReader new_proc -viewType maskLayout
puts [db::findInterfaceReader -viewType maskLayout]
```

db::registerNetlistInfo

Description

Adds the Netlist Info identifier to the list of known Netlist Info sections for Custom Compiler. This affects various aspects of the system. The list is used by the Parameter Filter GUI to determine all possible Netlist Info identifiers. It is also used by Symbol Generation to determine the list of Netlist Info sections to automatically create.

Syntax

```
db::registerNetlistInfo <netlister: string> [-properties <string>]
```

Arguments

netlister

The name of the netlister to add to the registry.

properties

Registered set of default properties.

Default: componentName instParameters namePrefix netlistProcedure
permuteRule propMapping termOrder.

Examples

```
db::registerNetlistInfo verilog -properties {componentName instParameters\
  namePrefix netlistProcedure permuteRule propMapping termOrder}
```

db::registerParamEvalFunction

Description

Binds a Tcl function to the named PEL command name.

Note:

The `-evalArgs` argument is a list of booleans specifying if the passed arguments should be evaluated. If `-evalArgs` is not specified for a given positional argument, the argument will be evaluated by the PEL parser.

Syntax

```
db::registerParamEvalFunction <none: string> -cmdName <string> [-numArgs  
  <int>] [-numRequiredArgs <int>] [-evalArgs <bool>]
```

Arguments

<string>

The name of the Tcl command implementing the PEL command.

-cmdName <string>

The name of the PEL command to use.

The default is the name of the registered Tcl function.

-evalArgs [<bool>]

Specifies if arguments should be evaluated.

Default: "".

Examples

```
# Create a procedure to find maximum from the given numbers
proc max_num {i args} {
    set m 0
    foreach arg $args {
        if {$m < $arg} {
            set m $arg
        }
    }
    return $m
}

# Register max_num procedure as the param eval function
db::registerParamEvalFunction max_num -cmdName max_num -numArgs 5
    -numRequiredArgs 2 -evalArgs true

# Add a new parameter to the testLib/nmos cell
set nmos [dm::findCell nmos -libName testLib]
db::createParamDef param1 -data $nmos -parseAsPEL true

# Set parameter value an expression with the above procedure
set inst [db::getInsts M0 -of [ed]]
db::setParamValue param1 -of $inst -value "max_num(1, 3, 4, 2, 2)"

# Print evaluated value of the param1 parameter
puts [db::getParamValue param1 -of $inst -context [db::getAttr hierarchy
    -of [de::getActiveContext]] -evalType full]
```

db::relativePath

Description

Makes paths relative to the base path.

Syntax

```
db::relativePath <Path: string> -base <string>
```

Arguments

Path

Path, which should be made relative.

-base

Path, which the result path should be relative to.

Returns

<string>: Non-common suffix portion of the given paths, which is relative to the base path.

Examples

```
# Get relative path to the sample folder from current path
db::relativePath $::env(SYNOPSYS_CUSTOM_INSTALL)/samples/ -base [pwd]
```

db::releaseFeature

Description

Releases a licensing feature back to the license pool. If the feature was not previously checked out, the command does nothing. If the feature is not recognized, an error is thrown.

Note that the license name is an internally defined license name which may refer to one or more features on the actual license server. Use [db::getFeatures](#) to determine which features are defined.

Syntax

```
db::releaseFeature <name: string> [-implicit <bool>]
```

Arguments

name

Name of the licensing feature to release.

-implicit

If true, the implicit check out count is decremented. If the count is reduced to zero, the feature is released. (Default is: false.)

db::remove

Description

Removes an object from the collection.

Syntax

```
db::remove <object: attrObject | dbIterator> -of <dbIterator>
```

Arguments

object

An object or collection to remove.

-of

A dbCollection from which to remove element.

Examples

```
# Create a collection of three instances M0, M1 and M2
set m0 [db::getNext [db::getInsts M0 -of [ed]]]
set m1 [db::getNext [db::getInsts M1 -of [ed]]]
set m2 [db::getNext [db::getInsts M2 -of [ed]]]
set insts [db::createCollection [list $m0 $m1 $m2]]

# Print the names of the collection instances
puts [db::createList [db::getAttr insts.name]]

# Remove the M1 instance from the collection
db::remove $m1 -from $insts

# Print names of the collection instances
puts [db::createList [db::getAttr insts.name]]
```

db::removeAttr

Description

Removes the specified attribute from the given object.

Syntax

```
db::removeAttr <name: dbVarAttrName> [-of <dbCollection | dbIterator>]
[-type <enum>]
```

Arguments

name

Attribute name or hierarchical sequence of names to remove separated by periods.

-of

Objects from which the attribute is removed.

-type

Forces only attributes of the specified type to be removed: {all | appDef | intrinsic | param | prop}.

Default: all.

Examples

```
# Create an object of dbObject type
set obj [db::createObject ]

# Add attribites to the object
db::addAttr name -of $obj -valueType string -value "vdd!"
db::addAttr nettype -of $obj -valueType string -value "power"
db::addAttr isGlobal -of $obj -valueType boolean -value "true"

# When you want to change or remove one of object attributs you may
# remove specific attribute from the object
db::removeAttr nettype -of $obj

# Print attributes of the object
puts [db::listAttrs -of $obj]
```

db::removeFigGroupMembers

Description

Destroys the figure group member (figGroupMem) thereby removing its figure (fig) from its figure group (figGroup). It does not destroy the figure.

Syntax

```
db::removeFigGroupMembers <objects: {<deFigure>} | {<deSelectionSet>} |
{<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |
{<oaVia>}} -group <deFigure | oaFigGroup | string> [-design <oaDesign>]
```

Arguments

objects

The figure to be removed.

`-group`

The figGroup from which the figure should be removed.

`-design`

The design in which the group belongs.

Returns

<none>

Examples

The following example creates a figure group object from the selection and removes shape objects from the figure group.

```
set design [db::getattr editDesign -of [de::getActiveContext]]
set figGroup [db::createFigGroup objGroup -members [de::getSelected
    -design $design] -design $design ]
db::removeFigGroupMembers [db::getShapes -of $design] -group $figGroup]
```

db::removeMarkerObject

Description

Removes the specified objects from the list of objects associated with this marker.

Syntax

```
db::removeMarkerObject <objects: oaBlockObject | {<deFigure>} |
    {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
    {<oaMarker>} | {<oaShape>} | {<oaVia>}> -marker <deFigure | oaMarker>
```

Arguments

`objects`

The objects to remove from marker.

`-marker`

The marker of the objects.

Returns

<none>

db::removeLibertyEditorPinData

Description

Removes pin data from the pins table in the Liberty Editor dialog box.

Syntax

```
db::removeLibertyEditorPinData -dialog <giDialog> -pinName <string>
```

Arguments

`-dialog`

The Liberty Editor dialog box to remove the pin data from table.

`-pinName`

Pin name for which the data is to be removed. If true, output data for each bit pin separately. Default: false.

Returns

<none>

db::renderTemplatesParasitics

Description

Creates parasitic DSPF files for all templates in the design.

Syntax

```
db::renderTemplatesParasitics -toFolder <string> -design <oaDesign>  
[-templates <string>]
```

Arguments

`-toFolder`

Folder to output the DSPF files.

`-design`

Design for which the files are generated.

`-templates`

List of templates 'l/c/v' for which must be rendered parasitic templates.

Returns

<string> : The generated parasitic template files.

Examples

```
# Create parasitic DSPF files for all templates in the design in the  
"dspf" folder  
db::renderTemplatesParasitics -design [ed] -toFolder dspf
```

db::reopenDesign

Description

Re-opens the design.

Syntax

```
db::reopenDesign <design: oaDesign> [-changeMode <bool>]
```

Arguments

design

oaDesign option

-changeMode

Enable change mode. The default is false.

Returns

<none>

Examples

The following example reopens DemoPLL/and2/layout design in read mode.

```
# Open DemoPLL/and2/layout design
set design [db::openDesign DemoPLL/and2/layout]

# Reopen DemoPLL/and2/layout design in read mode
db::reopenDesign $design -editable false
```

db::reportCallbacks

Description

Generates reportCallbacks.

Syntax

```
db::reportCallbacks [<pattern: string>] [-regexp <bool>] [-event  
<string>] [-disable <bool>]
```

Arguments

pattern

Pattern to match callback names.

-regexp

If true, pattern is regular expression. If false, it is a glob-type pattern

Default value: false.

`-event`

Event for which to return registered callback names.

Default value: all.

`-disable`

If true, reporting is disabled for specified events.

Default value: false.

Examples

```
# Generates reportCallbacks and save the report into a file
db::reportCallbacks * -event all -file ./callbacks.txt
xt::openTextViewer -files ./callbacks.txt
```

db::reset

Description

Resets the iterator to the start of the collection.

Syntax

```
db::reset <collection: dbIterator>
```

Arguments

<object>

The iterator over the Collection.

Examples

```
set design [ed]
set insts [db::getInsts -of $design]
# Iterate through each object in collection
while {[set inst [db::getNext $insts]] != ""} {
    puts [db::getAttr inst.name]
}
puts "=== Resets the iterator to the start of the collection. ==="
db::reset $insts
while {[set inst [db::getNext $insts]] != ""} {
    puts [db::getAttr inst.name]
}
```

db::resetVDRCDefinedDevices

Reset all VDRC user-defined devices

Syntax

```
db::resetVDRCDefinedDevices -targetLib <oaLib | string>
```

Returns

<none>

Arguments

-targetLib

The target library to reset all user-defined devices

db::resolve

Description

Given a path relative to any startup directory, returns all matching absolute paths from any startup directory. If the path is not provided, returns all startup directories.

Syntax

```
db::resolve [<path: string>] [-includeResources <bool>]
```

Returns

List of matching absolute paths ([<string>]).

Arguments

<string>

Path relative to any startup directory. Default:..

-includeResources <bool>

Returns resource path if available instead of absolute path (Default: false).

Examples

```
# Print all startup locations in console
foreach dir [db::resolve] {
    puts $dir
}

foreach filePath [db::resolve custom_compiler/init.tcl] {
    puts $filePath
}
```

db::resolvePath

Description

Given a path which optionally contains relative directory references (.,..), environment variable references (\$VAR), or home directory references (~bin), this command returns the absolute path to which the relative path resolves.

This command does not guarantee that the final path exists. You can use the various Tcl 'file' commands to determine whether the resulting path is acceptable.

Note that ".." will not be resolved unless `-resolveSymlinks` is true. Since the path component which precedes ".." may be a symbolic link, it would have to be resolved in order to determine the true result of "<symlink>/..".

Syntax

```
db::resolvePath <string: string> [-relative <string>] [-resolveEnvVars  
    <bool>] [-resolveSymLinks <bool>]
```

Arguments

<string>

Relative path to resolve.

-relative

Reference path (absolute) relative to which the relative path will be resolved.

-resolveEnvVars

Resolves the environment variable. The default is: true.

-resolveSymlinks

Resolves symbolic links encountered in the specified path. This should be used when comparing two paths for equivalence, but not when showing or saving user-specified paths. The default is false.

Returns

<string>: Resolved path.

Examples

```
# Get absolute path to the sample folder from current path  
db::resolvePath $::env(SYNOPSIS_CUSTOM_INSTALL) -relative [pwd]
```

db::revertParam

Description

Removes local override values for the given dbParam/parameter. The effective value is the default value stored in the Parameter Definition. This is not equivalent to changing the parameter value to the default value through modification of the `-value` attribute or via `db::setParamValue`.

Note:

For PCells, if the default value does not match between the SuperMaster oaParam and the Custom Compiler Parameter default value, this command sets the parameter to match the Custom Compiler default value.

Syntax

```
db::revertParam <param: dbParam | string> [-of <oaRef>]
```

Arguments

`param | <string>`

The parameter or name of the parameter to revert.

`-of`

The named parameter to be reverted for this object.

Default: 0.

Examples

```
# Create a blank schematic view and create an instance of reference40nm/n_4t
# Set the instance name to I0
set design [ed]

# Get the M0 instance
set inst [db::getInsts I0 -of $design]

# The M0 l parameter default value is 0.04u. Set it to 0.08u
db::setAttr inst.l -value 0.08u

# Check the parameter value.
puts [db::getAttr inst.l]

# Revert the parameter to its default value
db::revertParam l -of $inst

# Check the parameter value.
puts [db::getAttr inst.l]
```

db::revertPrefs

Description

Reverts a set of preferences to their startup values, typically considered the default value for dialogs and interactive commands. The startup value is the value exposed to the user before any modifications.

The modified preferences are then returned to the user.

Syntax

```
db::revertPrefs <dbPref: dbCollection>
```

Arguments

dbPref

Preferences to revert.

Returns

<dbCollection>: Collection of modified preferences.

Examples

```
# Set one or several preferences
db::setPrefValue saAnalyzeResultsPlotMode -value append

# Revert value of an individual preference or several preferences at a time
db::revertPrefs [db::getPrefs saA*]

# Check the preference value
puts [db::getPrefValue saAnalyzeResultsPlotMode]
```

db::saveCDF

Description

Saves and closes the CDF for a cell or library.

Syntax

```
db::saveCDF <data: dmCell | oaLib> -level <enum> [-close <bool>]
```

Arguments

data

Specifies the location to store the CDF as oaDMDData.

-close

Specifies whether to close the oaDMDData after saving the CDF. Default is *true*.

`-level`

The CDF level from which parameters will be exported ({ base | memory | overlay }, default is: base)

Returns

<none>

Examples

```
# Get the cell which CDF data need to purge
set dmCell [dm::getCells n_4t -libName reference40nm]

# Purge the CDF data of the cell
db::purgeCDF $dmCell -removeCDF true -forceMaster true

# Save the CDF data File
db::saveCDF $dmCell
```

db::saveDesign

Description

Saves the design.

Syntax

```
db::saveDesign <design: oaDesign>
```

Arguments

design

OA Design object

Returns

<none>

Examples

The following example saves opened DemoPLL/and2/layout design.

```
# Open DemoPLL/and2/layout design
set design [db::openDesign DemoPLL/and2/layout]

# Save opened DemoPLL/and2/layout design
db::saveDesign $design
```

db::saveDesignAs

Description

Saves the design as specified.

Syntax

```
db::saveDesignAs <design: oaDesign> -destination <dmCellView |  
dmCellViewName> [-overwrite <bool>]
```

Arguments

design

OA Design object

-destination

Destination in which to save the design.

-overwrite

Overwrites the existing design. The default is true.

Returns

<dmCellView> : Saved design.

Examples

The following example saves opened DemoPLL/and2/layout design.

```
# Open DemoPLL/and2/layout design  
set design [db::openDesign DemoPLL/and2/layout]  
  
# Save opened DemoPLL/and2/layout design  
db::saveDesignAs $design -destination DemoPLL/and2_test/layout
```

db::saveDeviceDisplays

Description

Saves the label display information of the current session to an XML file.

Syntax

```
db::saveDeviceDisplays <empty: {<dbDeviceDisplay>}> -filePath <string>  
[-force <bool>]
```

Arguments

`empty`

Object defining state of device label display.

`-filePath`

File in which to save the display information. Assumed relative to the current working directory if not absolutely specified.

`-force`

Overwrite existing files.

The default is false.

Returns

`<none>`

Examples

```
# Create device display to hide name of I0 instance
db::setDeviceDisplay -of [db::getInsts I0 -of [ed]] -name none

# Save device displays in devDisplay.xml file
db::saveDeviceDisplays [db::getDeviceDisplays] -filePath ./devDisplay.xml
```

db::saveGenOptionSets

Description

Stores the type and value of the specified dbGenOptionSet object. The option set name is not stored. If the given file path references an existing file, the `-force` argument determines whether the file is overwritten or an error is generated. If the given file path points to a directory, an error is generated.

Syntax

```
db::saveGenOptionSets <none: {<dbGenOptionSet>}> -filePath <string>
[-force <bool>]
```

Arguments

`none`

Collection of generator option set objects to save to a disk.

`-filePath`

Name of file to store option set information.

`-force`

Overwrite the existing file.

The default is true.

Returns

<none>

Examples

In this example, a new symbol generation option set is created from the default one and saved into a file which can be loaded with [db::loadGenOptionSets](#) in future for specific symbol generation.

```
# Get Symbol Generator default option set value
set defaultOptionSet [ db::getGenOptionSets default \
    -filter {%setType == "symbolGenerator"}}
array set vals [db::getAttr value -of $defaultOptionSet]

# Modify default values of option set
set vals(unitSize) 0.125
set vals(pinStubLength) 1
set vals(pinSpacing) 3
set vals(pinAnnotationText) cdsTerm("%p")

# Create new option set for Symbol Generator
set gos [db::createGenOptionSet myset -type symbolGenerator]

# Update created option set with new values
db::setAttr value -of $gos -value [array get vals]

# Set file name where to save the option set
set filePath [file normalize ./symbolGenOptionSet.xml]
db::saveGenOptionSets $gos -filePath $filePath -force true
```

The following example saves the obtained option set *m1Set*, which was assigned to the *m10Set* variable, into the *agM1OptionSet.xml* file.

```
set m1OptionSet [db::getGenOptionSets m1Set]
db::saveGenOptionSets $ m1OptionSet -file ./agM1OptionSet.xml
```

db::savePrefs

Description

Saves a collection of supplied preference objects. If a `-filePath` argument is supplied, the preferences are saved to the file. If `-filePath` is omitted, preferences are stored to the scope specified by their scope attribute.

Saving preferences is always a merge operation. If a preference is already stored in the given scope, its value is overwritten, otherwise a new name/value pair is created.

Preferences already stored in the file or a scope which are not included in the input collection are not modified.

The command throws an error if the storage container is non-existent (for example, context scope), is not write-able (for example, DMOBJECT, which is write protected), or is the install level.

All the preferences in the input collection must be from the same environment. The command exits with error if the input collection contains preferences with the same name and scope. It also exits with error if the input collection contains preferences with the same name and the `-filePath` argument is supplied. Both of these conditions create "duplicate preferences" error.

Syntax

```
db::savePrefs <preferences: dbCollection> [-filePath <string>]
```

Arguments

`preferences`

Collection of preferences to save.

`-filePath`

A file to save preferences to. (Default is:)

Returns

<none>

Examples

The following example saves all preferences into the `preferences.xml` file.

```
db::savePrefs [db::getPrefs] -filePath preferences.xml
```

db::sciToEng

Description

Converts real (double/string) values to Engineering notation. Exponents will be replaced with the closest Engineering suffix from the found in this table.

The precision argument controls the maximum number of significant digits of the output. The actual precision of the output will be the lesser of the input's number of significant digits and the requested `-precision`. String input zeros preceding the decimal place will always be treated as significant digits. For example "100" will be treated as if it has 3 significant digits.

Syntax

```
db::sciToEng <input: string> [-precision <unsigned int>] [-suffix <enum>]
```

Arguments

input

Input number or string.

-precision

Precision used for result. (Default is: 6)

-suffix

Suffix for the converted value ({ E | G | K | M | MEG | Meg | N | P | T | U | X | Y |
Z | a | auto | c | f | g | k | m | meg | n | p | u | y | z | {} })

Returns

<string> : Result of converting input to a string using Engineering suffixes.

Errors

- If the input string is not a numeric value.
- If the precision is less than 0.

Examples

```
# Convert double/string values to engineering notation
puts [db::sciToEng 0.000012568 -suffix u]
```

```
# Convert 1000 value to engineering notation k with precision 2
puts [db::sciToEng 1206 -precision 3 -suffix k]
```

db::setAnalogLibCompatibleVersion

Sets analogLib preferences by release name to provide backward compatible behavior for legacy designs.

Syntax

```
db::setAnalogLibCompatibleVersion <release: string>
```

Returns

<bool>: Status of this command run.

Arguments

release

The release name to set analogLib behavior.

db::setAttr

Description

Sets the value of the specified attribute of the given objects.

Syntax

```
db::setAttr <name: dbVarAttrName> [-of <dbCollection | dbIterator>]  
-value <variant> [-type <enum>] [-orient <oaOrient>]
```

Arguments

name

The attribute name or hierarchical sequence of names to set separated by periods.

-value <variant>

The attribute value to set.

-of

The objects on which the attribute is set.

-type

Filters attributes to only those of the specified type: {all | appDef | intrinsic | param | prop}.

Default: all.

-orient

Orientation of the region. Valid values: R0, R180, MX, and MY. For an existing track region already created in a design its orient is R0.

Returns

<none>

Examples

Example 1

In the following two examples, `db::setAttr` sets corresponding attribute values for instance and shape objects.

```
db::setAttr "origin" -value {20.0 20.0} -of $oaInst  
db::setAttr "bBox" -value {{0 0.5} {5 8}} -of $oaShape
```

Example 2

In the following example, `db::setAttr` hides the Blockage object type on the Object/Layer Panel's Objects tab for the currently opened design. This does not affect the next opening design.

```
db::setAttr "shown" -value false -of [de::getObjectFilters leBlockage
    -from [ed]]
```

Example 3

In the following example, `db::setAttr` hides the Blockage object type on the Object/Layer Panel's Objects tab. This takes effect on the next opening design, it does not affect the currently opened design.

```
db::setAttr "shown" -value false -of [de::getObjectFilters leBlockage]
```

Example 4

In the following example, `db::setAttr` sets the spacing between widgets.

```
set gr [gi::createGroup -parent [gi::createDialog dlg -title test
    -stayOnTop 1 ]]
set cnt 5
while {$cnt > 0} {
    gi::createPushButton -parent $gr -label "PushButton"$cnt
    incr cnt -1
}
db::setAttr gr.spacing -value 0
db::setAttr gr.margins -value[ list 100 100 100 100]
```

db::setColorShiftLayers

Description

Sets cellviews to allow color shifting.

Syntax

```
db::setColorShiftLayers <cellviews: {<dmCellView>}>
```

Arguments

cellviews

Collection of cellviews to mark as allowing color shifts for all colorable layers.

Returns

<none>

db::setDeviceDisplay

Description

Creates the dbDeviceDisplay object within the current session. If there is an existing dbDeviceDisplay object that references the current instance/master/library/cellView/design, any attributes that are given on the command line will overwrite the attributes of the existing object.

Syntax

```
db::setDeviceDisplay [-window <int>] -of <dmCellView | oaDesign | oaInst  
| oaInstHeader | oaLib> [-in <oaDesign>] [-terms <string>] [-name  
<string>] [-params <string>]
```

Arguments

-window

Window ID.

-of

Indicates the object associated with the given display object.

-in

Optional design in which to set the specified label display.

-terms

Indicates if the cdsTerm labels of the instances evaluate to the corresponding instTerm's name or net attribute.

Default: off.

-name

Indicates if the cdsName label of the instances evaluate to the instances name or cellName attribute.

Default: off.

-params

Indicates the list of parameters for cdsParam evaluation. If the parameter name is prepended with "-", the value only will be displayed.

Default: <>.

Returns

Object defining state of device label display (dbDeviceDisplay).

Examples

```
# Get master of I0 instance
set master [db::getAttr master -of [db::getInsts I0 -of [ed]]]

# Create device display to show model name and hide l and w parameters
db::setDeviceDisplay -of $master -params {model(parameter) w(none)
l(none)}

# Create device display to show net names on instance terminals
db::setDeviceDisplay -of $master -terms net

# Restore all labels by destroying device display
db::destroy [db::getDeviceDisplays]
```

db::setInstName

Description

Sets the name for an instance. This command can also be used to turn a scalar instance into a vectored instance.

Syntax

```
db::setInstName <name: string> -of <oaInst>
```

Arguments

name

The new name for the instance in oaCdbaNS. Can be either a scalar or a vector name. Cannot be a bundle name.

-of

Object being modified.

Returns

<none>

Examples

The following example sets instances named I0 to X0 in the given design.

```
db::setInstName X0 -of [db::getInsts I0 -of $oaDesign]
```

db::setLicenseTier

Description

Changes the current tier.

Syntax

```
db::setLicenseTier <tier: enum>
```

Arguments

tier

Tier name to set. ({ advanced | advancedPlus | apex | backend | basic | elite | frontend | mtm | readonly | router | schematic })

Returns

<none>

Examples

The following example downgrades or upgrades the license tier to backend.

```
db::setLicenseTier backend
```

db::setMaster

Description

Sets the master Lib/Cell/View for an instance. Same named instance terminals retain the original net object. The operation is reversible such that changing the master to a new master, and then changing the master back to the original master preserves all oaParam/oaProp's. This is an important consideration when changing from a PCell to a non-PCell master and vice versa. When changing from a PCell to a non PCell master, non-default oaParams are saved as appropriately typed oaProp's. When changing from a non PCell master to a PCell master, all oaProp's that match oaParam names are converted to oaParam's.

Syntax

```
db::setMaster <cell: dmCellView> -of <oaInst>
```

Arguments

cell

The master to assign to the given object.

-of

The instance to re-master.

Returns

<none>

Examples

The following example changes the symbol instance references to reference the library "newLib" from the library "oldLib".

```
db::foreach inst [db::getInsts \
    -of [db::getAttr editDesign -of [de::getActiveContext]] \
    -filter %libName=="oldLib"] {
    set dmCellView [dm::findCellView symbol \
        -cellName [db::getAttr cellName -of $inst] -libName newLib]
    db::setMaster $dmCellView -of $inst
}
```

db::setNetlistInfo

Description

Creates or modifies a name/value pair within the Netlist Info database. It can be used to modify all of the sections of the CDF except for the parameter definitions. The parameter definitions can only be modified via the dbParamDef API.

If the section is `netlist`, the `-netlistner` argument is required and the corresponding `simInfo` section is modified.

If the section is `display`, the `simInfo propList` section is modified. This section of the Netlist Info controls the display of parameters and simulation results within `cdsParam` labels.

If the section is `form`, the top level properties are modified. Typical entries in the form properties are `promptWidth` and `fieldHeight`. This is provided mostly for data compatibility. Custom Compiler does not use the form properties stored in the Netlist Info.

If the specified section does not have an entry, the entry is automatically created.

Syntax

```
db::setNetlistInfo <name: string> (-value <string> | -clear <bool>)
    -level <enum> -data <dmCell | oaCellDMDData | oaLib | oaLibDMDData>
    [-netlistner <string>] [-info <string>] [-section <enum>] [-type <enum>]
```

Arguments

`name`

Name of netlist info property.

`-data`

The cell or library that contains a DMDData where the specified Netlist Info is stored.

`-netlist`

The Netlist name, required only when a section is `netlist`. (Default is:)

`-info`

Name of the subsection, only allowed when section is `netlist`. (Default is: `simInfo`)

`-section`

The section where the specified Netlist Info is stored. ({ `display` | `form` | `netlist` }, default is: `netlist`)

`-value`

The value to be associated with the specified name.

`-type`

The type of value. ({ `floating` | `integer` | `list` | `real` | `string` | `symbol` }, default is: `symbol`)

`-clear`

Specifies whether to clear the Netlist Info value.

`-level`

The CDF level from which value will be obtained. ({ `base` | `memory` | `overlay` }, default is: `overlay`)

Returns

<none>

Examples

```
db::setNetlistInfo componentName -value amp -data [dm::getCells amp \
-libName PLL] -netlist verilog
db::setNetlistInfo termOrder -value {"INP" "PBIAS" "INM" "NBIAS" "OUTM" \
"OUTP"} -data [dm::getCells amp -libName PLL] -netlist verilog
db::setNetlistInfo instParameters -value {(wn wp)} -data [dm::getCells amp \
-libName PLL] -netlist verilog
```

db::setParamFilter

Description

Sets the list of Netlist Info identifiers that are active in the Parameter Filter Registry.

Syntax

```
db::setParamFilter <tools: string> -windowType <giWindowType | string>
```

Arguments

`tools`

The list of Netlister names to register for the given window type.

`-windowType`

Window Type.

Returns

<none>

Examples

```
db::setParamFilter {hspiceD auCdl veriloga} -windowType seSchematic
```

db::setParamValue

Description

Provides a simple string based interface for modifying parameter values on instances and oaCustomVia's. It is similar to the competition's direct access to the cdfgData global variable.

This function will create/modify data based on the following rules:

If the parameter is of type netSet, creates or modifies an oaAssignValue.

If an oaParam of the given name exists on the sub or super master, sets the oaParam value. The specified type will be ignored and the input value will be converted to the type of the oaParam.

If an Custom Compiler parameter exists with the given name, creates or modifies an oaProp corresponding to the Custom Compiler parameter type. The specified type will be ignored and the input value will be converted to the type of the Custom Compiler parameter.

Otherwise an oaProp is created with the specified type.

Real type properties/parameters will accept all Engineering Suffixes performing conversion when necessary.

The -type option is ignored if the named parameter is of type netSet.

Used together with `db::getParamValue` a full read/write access to oaParam, Custom Compiler parameters, and oaProps is provided in a simple "by name" interface.

Syntax

```
db::setParamValue <name: string> -value <string> [-type <enum>] -of  
  <oaRef> [-evalCallbacks <bool>]
```

Arguments

name

Parameter name.

-value

Parameter value.

-type

Parameter value type: {application | boolean | float | integer | real | string | time}.

Default: string.

-of

Object for which parameter value is to be set.

-evalCallbacks

If parameter change callbacks should be evaluated.

The default is true.

Returns

The object created/modified by `db::setPropValue`. (oaProp | oaParam | oaAssignValue)

Errors

- If the named oaParam exists and the value is not convertible to the type of the oaParam.
- If the named Custom Compiler parameter exists and the value is not convertible to the parameter type.
- If the `-type` argument is set to `netSet`.

Examples

The following example sets the parameter value for the number of fingers in the X1 instance to 3.

```
db::setParamValue nf -of [db::getInsts X1 -of $oaDesign] -value 3
```

db::setPrefValue

Description

Sets a preference value in the specified scope, if localized there, or from the lowest scope above where the preference is localized. This command is a shortcut for:

```
db::setAttr value -value value -of [db::getPrefs $name \ -scope $scope]
```

Syntax

```
db::setPrefValue <name: dbDeferredPrefName> -value <string> [-scope  
  <dbScope | deContext | dmCell | dmCellView | oaLib | string>]
```

Arguments

name

Preference name.

Note:

It is an error to set a preference that does not exist.

-value

Set the preference to this value. This argument supports tab completion for setting the preference values of dbEnum types.

-scope

Optionally set value at this scope.

Default: User scope of Active environment.

Returns

Preference object which was modified by the command (dbPref).

Errors

- A preference with given name does not exist.
- Value is not valid for the specified preference.
- Incorrect scope.

Examples

The following four examples sets the preference values for corresponding objects.

```
db::setPrefValue dmLibDir -value "/user/proj/backup"  
db::setPrefValue xtHSPICEDefaultViewer -value WaveView  
db::setPrefValue xtLVSRunDir -value "./PLL_lvs"  
db::setPrefValue leInstRowNum -value 10
```

The following example uses the Tab key to automatically complete the preference value of dbEnum types.

```
db::createEnum enTermType \  
    -values {input output inputOutput switch jumper unused tristate}  
db::createPref crTerm -type enTermType -value tristate  
db::setPrefValue crTerm -value j  
# type j and press the Tab key to auto complete the value to "jumper"
```

In the following example, if the number of layer purpose pairs is greater than or equal to the specified value, the **Tap** dialog box appears. If the count is smaller than the value, the tool taps the layer purpose pair of the active figure without showing the **Tap** dialog box.

```
db::setPrefValue leTapLPPThreshold -value 3 -scope [de::getActiveContext]
```

The following example sets the pad label location to the text label's origin for exporting.

```
db::setPrefValue lePadOutputLocationMethod -value textOrigin -scope  
[de::getActiveContext]
```

The following example shows that netlists do not evaluate expressions and print them as it is.

```
db::setPrefValue nlMaximumPartialEvaluation -value false  
.subckt cell1  
m0 net1 net4 net3 net2 nmos4 w='((2u*2)+9)+(2/3)' l=2u  
.ends cell1
```

The following example shows that netlists evaluate the computation with respect to a given expression.

```
db::setPrefValue nlMaximumPartialEvaluation -value true  
.subckt cell1  
m0 net1 net4 net3 net2 nmos4 w=9.666670666666667 l=2u  
.ends cell1
```

The following examples suppress warnings of an unmapped layer during GDSII exporting and OASIS exporting.

```
db::setPrefValue dbExportStreamSuppressUnmappedLayerWarnings -value {M1  
drawing}  
db::setPrefValue dbExportOasisSuppressUnmappedLayerWarnings -value {M1  
drawing}
```

The following examples suppress warnings of multiple unmapped layers during GDSII exporting and OASIS exporting.

```
db::setPrefValue dbExportStreamSuppressUnmappedLayerWarnings -value  
{{FLIPWELL marker}  
{M1 drawing} {POLY cddummy}}  
db::setPrefValue dbExportOasisSuppressUnmappedLayerWarnings -value  
{{FLIPWELL marker} {M1 drawing}{POLY cddummy}}
```


Use one of the following examples to show timestamps in ISO 8601 time format %H:%M:%S

```
db::setPrefValue giConsoleShowTimestamp -value "%H:%M:%S "  
db::setPrefValue giConsoleShowTimestamp -value "%T "
```

db::setProgress

Description

Updates the current position in a dbProgress and redraws its progress bar, if shown.

Syntax

```
db::setProgress <position: unsigned int> [-label <string>] -of  
  <dbProgress>
```

Arguments

position

New position.

-label

Updates current sub operation label.

-of

Progress to update.

Returns

<none>

Examples

```
db::eval {  
  # Create a progress bar  
  set progress [db::createProgress inprogress -label "Lable" \  
    -title "Example Progress" -maxValue 100]  
  set i 0  
  while {$i < 100} {  
    incr i  
  
    # Set the progress bar value  
    db::setProgress $i -label Label -of $progress  
  
    # Check the Cnt+C binding  
    db::checkForInterrupt $i -handler $progress  
    puts $i  
    after 200  
  }  
}
```

db::setViaDef

Description

Sets the definition of an oaVia instantiation.

Changing the definition of an oaVia instantiation requires deletion and re-creation of the object. Via's can be of type oaStdVia or oaCustomVia. Instantiations of oaStdVia's cannot have oaParam's. Likewise, oaCustomVia instances cannot have oaViaParam's. As a result when changing from an oaStdVia to an oaCustomVia and vice versa, it is not possible to retain all of the parameterized data. In this case the pin information and same named parameter values are preserved. If changing the definition does not change the object type, then the parameterization is preserved by copying the source oaViaParam's/ oaParam's to the destination object.

Syntax

```
db::setViaDef <def: oaViaDef> -of <oaVia>
```

Arguments

def

The new object definition.

-of

Object being modified.

Returns

<none>

Examples

```
set design [ed]
set tech [db::getAttr design.tech]
set viaDef [db::getNext [db::filter [db::getAttr tech.viaDefs] -filter
    {%name=="via12"}]]
if {"!"=$viaDef} {
    set transaction [de::startTransaction "Set Vias" -design $design]
    # Find and replace all vial2Line vias by vial2
    db::foreach via [db::getVias -of $design -filter
        {%viaDef.name=="vial2Line"}] {
        db::setViaDef $viaDef -of $via
    }
    de::endTransaction $transaction
}
```

db::showAbstractGeneratorOptions

Description

Opens the abstract generator option editing dialog from which the user can load, save, create, and edit option sets.

Syntax

```
db::showAbstractGeneratorOptions -cellView <dmCellView> -design  
  <oaDesign> -window <giWindow | int> [-parent <giDialog>] [-optionSet  
  <string>]
```

Arguments

-cellView

The source cellView.

-design

The source design.

-window

The window to use for obtaining the source design.

-parent

giDialog object of the cellView from cellView dialog.

Default: NULL.

-optionSet

Option Set name.

Default: default.

Returns

Existing or newly created dialog containing abstract generator options editor (*giDialog*).

db::showAdvancedPrintOptions

Description

Launches the Advanced Print Options dialog box that is used to configure additional Printing options.

Syntax

```
db::showAdvancedPrintOptions [-parent <giWindow | int>]
```

Arguments

`-parent`

The window that this dialog is associated with.

Returns

The dialog (*giDialog*).

Errors

Supplied or active dialog is not a Print dialog.

db::showBindExternalNetlistFile

Description

Opens the **Select External Netlist File to Bind** dialog box.

Syntax

```
db::showBindExternalNetlistFile [-parent <giWindow | int>] [-fileName  
    <string>] [-browsePath <string>]
```

Arguments

`-parent`

The parent window. (Default: 0)

`-fileName`

The default file name of the dialog box. (Default is:)

`-browsePath`

The default file browse path of the dialog box. (Default is:)

Returns

`<giDialog>` : The created dialog box.

db::showBindExternalNetlistView

Description

Opens the **Select External Netlist View to Bind** dialog box.

Syntax

```
db::showBindExternalNetlistView [-parent <giWindow | int>] -libraryName  
    <string> -cellName <string> [-fileName <string>] [-viewName <string>]  
    [-browsePath <string>]
```

Arguments

-parent

The parent window. (Default: 0)

-libraryName

The library name to set the binding.

-cellName

The cell name to set the binding.

-fileName

The default file name of the dialog box. (Default is:)

-viewName

The default view name of the dialog box. (Default is:)

-browsePath

The default file browse path of the dialog box. (Default is:)

Returns

<giDialog> : The created dialog box.

db::showCellViewFromCellView

Description

Opens the **dbCellViewFromCellView** dialog box.

If a dialog box already exists for the same parent, that dialog box is shown and brought to the top of the window stack. However, the dialog box contents are not altered.

Syntax

```
db::showCellViewFromCellView [-parent <giWindow | int>] [-src <dmCellView  
    | dmCellViewName>] [-dest <dmCellView | dmCellViewName>] [-optionSet  
    <string>] [-defaultEditor <string>]
```

Arguments

`-parent`

Parent window.

`-src`

Source cellView.

`-dest`

Destination cellView.

`-optionSet`

The optional set.

`-defaultEditor`

The default editor.

Default: `dbGenerateCellViewEditor`.

Returns

Dialog window (*giDialog*).

Errors

- Invalid cellView is given.
- Invalid editor name is given.

db::showCellViewFromPinList

Description

Opens the `dbCellViewFromPinList` dialog box.

If a dialog box already exists for the same parent, that dialog box is shown and brought to the top of the window stack. However, the dialog contents are not altered.

Syntax

```
db::showCellViewFromPinList [-parent <giWindow | int>] [-dest  
    <dmCellView>] [-defaultEditor <string>]  
db::showCellViewFromPinList [-parent <giWindow | int>] [-dest <dmCellView  
    | dmCellViewName>] [-defaultEditor <string>]
```

Arguments

`-parent`

Parent window.

`-dest`

Destination `cellView`.

`-defaultEditor`

The default editor.

Default: `dbGenerateCellViewEditor`.

Returns

Dialog window (*giDialog*).

Errors

- Invalid `cellView` is given.
- Invalid editor name is given.

db::showDummyRules

Creates dummy rule editor widget in parent

Syntax

```
db::showDummyRules -parent <giDialog | giWidget | giWindow> -data  
  <string> -libName <string>
```

Returns

`<string>` : New `IxDummyRule` string value

Arguments

`-parent`

Parent widget

`-data`

`IxDummyRule` string value to edit

`-libName`

Name of library associated with parameter

db::showEditComponents

Description

Creates a component editor widget in the parent widget.

Syntax

```
db::showEditComponents -parent <giDialog | giWidget | giWindow> -data  
    <string> -libName <string>
```

Arguments

-parent

Parent widget

-data

lxCComponentType string value to edit

-libName

Name of library associated with parameter

Returns

<string> : New lxCComponentType string value

db::showExportDef

Description

Brings up the Export DEF dialog box to export DEF files. DEF files contain the instance placement and routing for a design.

Syntax

```
db::showExportDef
```

Returns

Write DEF dialog (*giDialog*).

db::showExportImage

Description

Launches the Export Image dialog box to export a CellView to an image file.

Syntax

```
db::showExportImage [-parent <giWindow | int>]
```

Arguments

-parent

The window that this dialog box is associated with.

Default: active window.

Returns

The dialog (*giDialog*).

Errors

Supplied or active window is not a Console, Schematic, Symbol, or Layout window.

db::showExportLef

Description

Brings up the Export LEF dialog box to export LEF files. LEF files contain technology data and reference cells.

Syntax

```
db::showExportLef
```

Returns

Write LEF dialog (*giDialog*).

db::showExportLiberty

Description

Opens the Export Liberty File dialog box.

Syntax

```
db::showExportLiberty [-parent <giWindow | int>]
```

Arguments

`-parent`

The window to use as parent. (Default is: 0)

Returns

<giDialog> : The Export Liberty File dialog box.

Errors

- The specified window does not exist.

db::showExportMCellFile

Description

Shows the **Export MCell** dialog box.

Syntax

```
db::showExportMCellFile [-parent <giWindow | int>]
```

Arguments

-parent

Parent widget.

Returns

<giDialog> : The Export MCell dialog box.

db::showExportMilkyway

Description

Shows the **Export to ICC** dialog box.

Syntax

```
db::showExportMilkyway [-design <oaDesign>] [-parent <giWindow | int>]
```

Arguments

-design

Source design to export.

-parent

Parent window.

Returns

<giDialog>: The Export to ICC dialog box.

db::showExportNetlist

Description

Shows the **Export to Netlist** dialog box.

Syntax

```
db::showExportNetlist
```

db::showExportNetlist::flattenDesign

Description

Creates intermediate flatten design for further netlisting

Syntax

```
<bool> db::showExportNetlist::flattenDesign <netlistName> <flattenLCV>  
<sourceLCV> <viewSearchList> <viewStopList> [<renameObjects>]  
[<useTransaction>]
```

Arguments

<return>

(bool) True if successfully created, else false.

<netlistName>

(<string>) Name of netlist to be run with flatten design.

<flattenLCV>

(<dmCellView> or <dmLCVName>) Target flatten libCellView with "oaNetlist" design type.

<sourceLCV>

(<dmCellView> or <dmLCVName>) Source libCellView.

<viewSearchList>

(<string>) View search list for hierarchy context creation with the given top-source LCV.

<viewStopList>

(<string>) View stop list for hierarchy context creation with the given top-source LCV.

<renameObjects>

(bool) (Optional)

True: Random-generated non-hierarchical names are assigned, like "G7j4y08R"

False: The net, terminal, and instance names in target design are hierarchical, like "I1/R2/net3". Default.

`<useTransaction>`

(bool) (Optional). When true, the renaming in the target design is made as one common transaction.

db::showExportOasis

Description

Shows the **Export Oasis** dialog box.

Syntax

```
db::showExportOasis [-parent <giWindow | int>]
```

Arguments

`-parent`

The parent window on which this dialog will be centered.

Returns

`<giDialog>` : The **Export Oasis** dialog box.

db::showExportStream

Description

Brings up the **Export Stream** dialog box to export a stream file. Stream files contain the layout for a set of designs.

Syntax

```
db::showExportStream
```

Returns

Stream Out dialog box (*giDialog*).

db::showFindReplace

Description

Displays the Find/Replace dialog box associated with the passed in editor window.

If `-parent` is passed in, the top cell view is set to match the windows edit design. The view search list, view stop list, occurrence path, and traversal depth are also inherited from the editor window.

Note that the CSMs in the Library Manager provide reasonable values for `-viewSearchList`, `-viewStopList`, and `-depth` depending on the view type of the `cellView`.

Syntax

```
db::showFindReplace [-parent <giWindow | int>] [-cellView <dmCellView>]  
  [-instPath <string>] [-viewSearchList <string>] [-viewStopList <string>]  
  [-depth <unsigned int>]
```

Arguments

`-parent`

The window to use for browsing and highlighting search results.

Default: "".

`-cellView`

The top cell view to use.

Default: "".

`-instPath`

The occurrence path used to limit hierarchical searches.

Default: "".

`-viewSearchList`

The view search list used for hierarchical descent.

Default: "".

`-viewStopList`

The view stop list used for hierarchical descent.

Default: "".

`-depth`

The hierarchical descent depth limit. The default is 32.

Returns

<none>

db::showGenerateAbstract

Description

Opens the **Generate Abstract** dialog box to generate the abstract view for multiple cells.

Syntax

```
db::showGenerateAbstract
```

Returns

<giDialog>: The **Generate Abstract** dialog box.

db::showGenerateSchematic

Description

Shows the **Schematic Generation** dialog box.

Syntax

```
db::showGenerateSchematic
```

Returns

<giDialog>: The **Schematic Generation** dialog box.

db::showGenerateSchematicsFromText

Description

Opens the **Schematic Generation** dialog box for generating schematics from text files.

Syntax

```
db::showGenerateSchematicsFromText
```

Returns

<giDialog> : The created dialog.

db::showGenericPrint

Description

Launches the generic Print dialog box to configure a print job and send it to the printer.

Syntax

```
db::showGenericPrint [-parent <giWindow | int | giDialog>]
```

Arguments

`-parent`

The window or dialog box that this dialog is associated with. (default is: active window)

Returns

The dialog box(*giDialog*).

db::showImportDef

Description

Brings up the Import DEF dialog box to import DEF files. DEF files contain the instance placement and routing for a design.

Syntax

```
db::showImportDef
```

Returns

The Import DEF dialog box (*giDialog*).

db::showImportEDIF

Description

Shows the Import EDIF200 dialog box.

Syntax

```
db::showImportEDIF [-parent <giWindow | int>]
```

Arguments

`-parent`

The parent window.

Returns

<giDialog> : Dialog object.

db::showImportLef

Description

Brings up the Import LEF dialog box to import LEF files. LEF files contain technology data and reference cells.

Syntax

```
db::showImportLef
```

Returns

The Import LEF dialog box (*giDialog*).

db::showImportMCellFile

Description

Shows the Import MCell dialog box.

Syntax

```
db::showImportMCellFile [-parent <giWindow | int>]
```

Arguments

`-parent`

Parent widget.

Returns

<giDialog> : The Import MCell dialog box.

db::showImportMilkyway

Description

Shows the **Import from ICC** dialog box.

Syntax

```
db::showImportMilkyway [-design <oaDesign>] [-parent <giWindow | int>]
```

Arguments

`-design`

Target design.

`-parent`

Parent window.

Returns

`<giDialog>`: The Import from ICC dialog box.

db::showImportOasis

Description

Shows the Import OASIS dialog box.

Syntax

```
db::showImportOasis [-parent <giWindow | int>]
```

Arguments

`-parent`

Parent window on which this dialog will be centered.

Returns

`<giDialog>`: Oasis in Dialog

db::showImportSimulationDataDialog

Invokes the **Import Simualtion Data** dialog box.

Syntax

```
db::showImportSimulationDataDialog -window <unsigned int> -context  
  <deContext>
```

Returns

`<none>`

Arguments

`-window`

The window to invoke the dialog box.

`-context`

The context design.

db::showImportStream

Description

Brings up the Import Stream dialog box to import a STREAM file. Stream files contain the layout for a set of designs.

Syntax

```
db::showImportStream
```

Returns

Stream In dialog box (*giDialog*).

db::showImportText

Description

Shows the **Import Text** dialog box.

Syntax

```
db::showImportText
```

Returns

<giDialog>: The Import Text dialog box.

db::showImportVerilog

Description

Brings up the **Import Verilog** dialog box to import a Verilog file. Verilog files contain a pure netlist description for a set of designs.

Syntax

```
db::showImportVerilog
```

Returns

Verilog In dialog box (*giDialog*).

db::showLibertyDataEditor

Description

Opens the **Liberty Data Editor** dialog box for the parent Design Editor window.

Syntax

```
db::showLibertyDataEditor -parent <giWindow | int>
```

Arguments

`-parent`

The window to use as parent.

Returns

<none>

Errors

- The specified window does not exist.
- The specified window is not Design Editor window.

db::showLoadPreferences

Description

Shows the **Load Preferences** dialog box. If the dialog box is already opened for the specified scope, it is raised and the existing object is returned. Otherwise, the existing dialog box is closed before reopening for the design.

Syntax

```
db::showLoadPreferences [-scope <dbScope | deContext | string>]
```

Arguments

`-scope`

The scope from which to retrieve the preference value.

Returns

<none>

db::showManageTechnology

Description

Opens the Custom Compiler Technology Manager.

Note:

Only a single instance of the Custom Compiler Technology Manager window will be created. Multiple calls to `db::showManageTechnology` should not create a new instance of the dialog box, if one already exists.

Syntax

```
db::showManageTechnology [-lib <oaLib | string>]
```

Arguments

`-lib`

Library to be selected in the libraries table of the window.

Returns

Dialog to manage technology (*giDialog*).

db::showPrint

Description

Launches the **Print** dialog box to configure a print job and send it to a printer or file.

Syntax

```
db::showPrint [-parent <giWindow | int>]
```

Arguments

`-parent`

The window that this dialog box is associated with. (default is: active window)

Returns

The dialog (*giDialog*).

Errors

Supplied or active window is not a Console, Schematic, Symbol or Layout window.

db::showPrintQueueStatus

Description

Launches the Print Queue Status window. If the window already exists, it is raised and made active.

Syntax

```
db::showPrintQueueStatus
```

Returns

The window (giWindow).

db::showPurgePreferences

Description

Shows the **Purge Preferences** dialog box. If the dialog box is already opened for the specified scope, it is raised and the existing object is returned.

Syntax

```
db::showPurgePreferences [-scope <dbScope | deContext | string>]
```

Arguments

-scope

The scope from which to retrieve the preference value.

Returns

<none>

db::showRefreshMCells

Description

Opens the **Refresh MCell and Editable PCell** dialog box.

Syntax

```
db::showRefreshMCells [-parent <giWindow | giWindowId>]
```

Arguments

-parent

Parent Window (Id).

Returns

<giDialog> : The **Refresh MCell and Editable PCell** dialog box.

db::showSavePreferences

Description

Shows the **Save Preferences** dialog box. If the dialog box is already opened for the specified scope, it is raised and the existing object is returned. Otherwise, the existing dialog box is closed before reopening for the design.

Syntax

```
db::showSavePreferences [-scope <dbScope | deContext | string>]
```

Arguments

-scope

The scope from which to retrieve the preference value.

Returns

<none>

Errors

Invalid scope.

db::showUnitDisplayScale

Description

Displays the dialog box for editing preferred unit scale factors. Scale factors are used to display evaluated parameters in intuitive scale ranges. For example, designers may prefer to view length values in either microns or nanometers depending on the technology node.

Syntax

```
db::showUnitDisplayScale -parent <giDialog | giWindow | int>
```

Arguments

-parent

Parent Window (Id).

Returns

<giDialog>: Created unit display scale.

db::showUpdateCellView

Description

Interactively displays the **Update CellView** dialog box. If a dialog box already exists for the same parent, then the existing dialog box is shown and brought to the top of the window stack. However, the dialog box contents are not altered.

Syntax

```
db::showUpdateCellView [-parent <giWindow | int>]
```

Arguments

-parent

The parent window. (Default is: 0)

Returns

<giDialog> : The created dialog box.

Errors

Invalid cellview is given.

db::showVDRCManagerWindow

Open the VDRC Manager window

Syntax

```
db::showVDRCManagerWindow -context <deContext>
```

Returns

<giWindow> : The created window

Arguments

-context

Parent design context of VDRC Manager window

db::snapPoints

Description

Snap points with given offset and spacing.

Syntax

```
db::snapPoints <points: [point]> [-design <oaDesign>] -offset <point>  
-spacing <point> [-round <enum>]
```

Arguments

points

The list of points that form snap points.

-design

The design to get DBUPerUU.

-offset

The distance between the origin of the manufacturing grid and the origin of the snap grid.

-spacing

The distance between snap coordinates.

-round

Rounding method. ({ down | rounding | up } (Default is: rounding))

Returns

<point> : The snapped points.

db::sort

Description

Creates a new dbCollection that contains items from another collection in sorted order.

Syntax

```
db::sort <collection: dbIterator> -attr <string> [-reverse <bool>]
```

Arguments

collection

A list of objects to be converted.

-attr

Object attribute to sort by.

-reverse

Sort in reverse order. (Default is: false)

Returns

<dbCollection | dbIterator> : New collection in sorted order.

Examples

The following example gets the cell names of the library and sorts them with the name attribute.

```
# Create a list of cells from the given library
set cells [dm::getCells -libName DesignLibrary]

# Print the cell names of the list
puts [db::createList [db::getAttr cells.name]]

# Sort the list of cells with name attribute
set sortedCells [db::sort $cells -attr name]

# Print the name attribute of the sorted list elements
puts [db::createList [db::getAttr sortedCells.name]]
```

The following example creates a list of instances of the design and sorts them with the name attribute. The list can also be sorted with other attributes.

```
# Create a list of instances from the open design
set des [ed]
set ints [db::getInsts -of $des]

# Print the instance name values of the list
puts [db::createList [db::getAttr ints.name]]

# Sort the instance list with origin values
set sortedInsts [db::sort $ints -attr origin]

# Print the name attribute of the instance list elements
puts [db::createList [db::getAttr sortedInsts.name]]

# Sort the instance list elements according to name attribute
set sortedInsts [db::sort $ints -attr name]

# Print the name attribute of the sorted list elements
puts [db::createList [db::getAttr sortedInsts.name]]
```

db::source

Description

Sources the Tcl command file encrypted by the `synenc` shell command (see [Encrypting Tcl Files](#)).

Syntax

```
db::source <filePath: string>
```

Arguments

`filePath`

Path of the `.tcl.e` file.

Returns

<none>

Examples

```
db::source ./calc_dencity.tcl
# Source Tcl file encrypted with the Synenc executable
db::source ./calc_dencity.tcl.e
```

db::startStatistics

Description

Starts the statistics gathering.

Syntax

```
db::startStatistics [-openhier <bool>] [-designopen <bool>] [-oainit  
<bool>] [-rqinit <bool>] [-render <bool>] [-pcellcaching <bool>] [-file  
<string>]
```

Arguments

`-openhier`

If true, times the total open hierarchy which includes design open and init.

The default is true.

`-designopen`

If true, times the opening of oa designs.

The default is true.

`-oainit`

If true, times the OA init dynamic tables.

The default is true.

`-rqinit`

If true, times the RQ initialization.

The default is true.

`-render`

If true, times the first draw of the rendering after a hierarchical open.

The default is true.

`-pcellcaching`

If true, times Pcell caching.

The default is false.

`-file`

Output to the given file.

Returns

<none>

db::stopStatistics

Description

Stops the statistics gathering.

Syntax

```
db::stopStatistics
```

Returns

<none>

db::transform

Description

Transforms the specified points with the transform from specified instances.

Syntax

```
db::transform <points: [point]> -occurrence <oaHierPath | string> -design  
             <oaDesign> [-inverse <bool>]
```

Arguments

`points`

The list of points that form transformed input.

`-occurrence`

Instance hierarchical path to transform.

`-design`

Design in which to create the transformed input.

`-inverse`

Specifies whether transform top points to design in hierarchy. (Default is: false)

Returns

[<point>] : Transformed User Points

Examples

The first example transforms instance master coordinates to the top design. The second example transforms top coordinates to instance master coordinates.

```
set design [db::getAttr editDesign -of [de::getActiveContext]]
set inst [le::createInst -viewName layout \
-cellName n_4t -libName reference40nm \
-design $design -origin {1 1}]
set instName [db::getAttr inst.name]
db::transform {{0.2 0.2}} -design $design -occurrence $instName
=> {1.2 1.2}
db::transform {{1.2 1.2}} -design $design -occurrence $instName -inverse
1
=> {0.2 0.2}
```

db::teeLogFile

Description

Executes a Tcl expression and writes the console output to a given file.

Syntax

```
db::teeLogFile <script: variant> -logFile <string>
```

Arguments

`script`

Tcl expression to execute.

`-logFile`

Log file path.

Returns

<none>

db::triggerParamCallback

Description

Forces the evaluation of parameter change callbacks. It should only be required in special circumstances when the default values stored at the cell or library level are modified.

Syntax

```
db::triggerParamCallback <param: dbParam>
```

Arguments

param

The parameter.

Returns

<string>: The evaluated value of the callback string.

Errors

If an error occurs when evaluating the callback field of the given parameter.

Examples

```
# Create a blank schematic view and create an instance of reference40nm/n_4t
# Set the instance name to I0
# Make sure the nf parameter of the instance is set to 1
set design [ed]

# Get the I0 instance
set inst [db::getInsts I0 -of $design]

# The n_4t has an parameter evaluation callback ref40_mos_cbk(nf), which
# evaluate parameter values
set param [db::evalCallback ref40_mos_cbk(nf) -of $inst]
puts [db::getAttr param.name]
puts [db::getAttr param.value]

# In the schematic design change the nf parameter of the instance to 2
# Now execute the callback
set param [db::getParams nf -of $inst -forceMaster true]
puts [db::getAttr param.name]
puts [db::getAttr param.value]

set res [db::triggerParamCallback $param]
puts [db::getAttr res.name]
puts [db::getAttr res.value]
```

The following example shows how to trigger a parameter change callback of an instance in a schematic design by opening a schematic design and creating an M0 instance of an NMOS device cell from any PDK library.

```
# Get M0 instance
set des [ed]
set inst [db::getInsts M0 -of $des]

# Get the w parameter of the instance
set param [db::getParams 1 -of $inst -forceMaster true]

# Trigger parameter change callback which calculates the value of
# depended parameter
set evalParam [db::triggerParamCallback $param]

# Print the name and the value of evaluated parameter
puts "[db::getAttr evalParam.name] = [db::getAttr evalParam.value]"
```

db::unregisterGenerator

Description

Unregisters a cellView generator for the given view name from the session. The `-from` argument must be used to remove a generator registered against a specific source viewtype name.

Syntax

```
db::unregisterGenerator -for <string> [-of <string>]
```

Arguments

`-for`

Destination viewtype name for the generator.

`-of`

Source viewtype name for the generator.

Returns

<none>

Examples

```
# Unregister previous cellview generator for symbol view
db::unregisterGenerator -for schematicSymbol

# Register new cellview generator for symbol view, which adds copyright
label in symbol view
db::registerGenerator createSymbol \
    -for schematicSymbol \
    -optionSetType symbolGenerator \
```

```

        -usesPinSides true \
        -defaultOptionSetPref seSymbolGenOptionSetDefault \
        -supportsUpdate true
proc createSymbol {args} {
    array set opts $args
    set iface $opts(-interface)
    set gos $opts(-options)
    set dest $opts(-dest)
    se::generateSymbol -interface $iface -dest $dest -options $gos
    -update false
    set symbolLCV [file join [db::getAttr dest.libName] [db::getAttr
dest.cellName] [db::getAttr dest.name]]
    set design [db::openDesign $symbolLCV -readOnly true]
    de::createContext $design
    se::createText "Copyright (c) Synopsys" -anchor {0 0} -design $design
    -type note
    de::save $design
    db::closeDesign $design
}

# Print registered generator name for symbol view
db::findGenerator -for schematicSymbol

```

db::unregisterInterfaceReader

Description

Unregisters an interface reader.

Syntax

```
db::unregisterInterfaceReader -viewType <string>
```

Arguments

-viewType

Viewtype name for the reader to unregister.

Returns

<none>

Examples

```

# Find and print interface reader tcl proc name for layout view
set int_reader [db::findInterfaceReader -viewType maskLayout]
puts $int_reader

# Unregister an interface reader
db::unregisterInterfaceReader -viewType maskLayout

# Register "new_proc" as the interface reader procedure

```

```
db::registerInterfaceReader new_proc -viewType maskLayout  
puts [db::findInterfaceReader -viewType maskLayout]
```

db::updateCDFParams

Description

Synchronizes CDF parameter information of given cell with the parameter of its schematic and text views.

Syntax

```
db::updateCDFParams <cell: dmCell>
```

Arguments

cell

Cell for which CDF parameter information should be updated.

Returns

<none>

db::vdrcMgrAddNet

Adds Net to VDRC Manager.

Syntax

```
db::vdrcMgrAddNet (-name <string> | -names <string> | -all <bool> |  
-voltageRange <string>) -window <giWindow | int> -context <heContext>
```

Returns

<none>

Arguments

-name

Net Name to Add to VDRC Window

-names

Net name pattens to add to the VDRC Window.

-all

Adds all schematic nets to the VDRC Window

`-voltageRange`

Adds nets with simulation voltage range to the VDRC Window

`-window`

The VDRC manager window.

`-context`

Schematic hierarchy context.

db::vdrcMgrAddNetPair

Adds Net Pair to VDRC Manager.

Syntax

```
db::vdrcMgrAddNetPair -window <giWindow | int> -context <heContext> -name  
  <string> -udv <bool>
```

Returns

<none>

Arguments

`-window`

VDRC Manager Window.

`-context`

Schematic Hierarchy Context.

`-name`

Net Pair Name to Add to VDRC Window.

`-udv`

When the net pair UDV type is:

- `true`: UDV
- `false`: Vsync

db::vdrcMgrAnnotateNet

Annotates simulation value to schematic.

Syntax

```
db::vdrMgrAnnotateNet -window <giWindow | int> -context <heContext>  
-names <string>
```

Returns

<none>

Arguments

-window

VDR Manager Window.

-context

Schematic Hierarchy Context.

-names

Net Names to Annotate to Schematic Design.

db::vdrMgrAnnotateNetPair

Annotates simulation value or user-defined value to schematic design.

Syntax

```
db::vdrMgrAnnotateNetPair -window <giWindow | int> -context <heContext>  
-names <string> -types <string> [-userValue <float>]
```

Returns

<none>

Arguments

-window

VDR Manager Window.

-context

Schematic hierarchy context.

-names

Net pair names to annotate to schematic design.

-types

Net pair types to annotate to schematic design.

`-uservalue`

User-defined value to annotate to schematic design.

db::vdrcMgrCheckHierarchyMismatch

Updates hierarchy mismatch reports to VDRC Manager

Syntax

```
db::vdrcMgrCheckHierarchyMismatch -window <giWindow | int>
```

Returns

<none>

Arguments

`-window`

VDRC Manager Window

db::vdrcMgrCheckSymbolPlacement

Checks Layout Symbol Placement.

Syntax

```
db::vdrcMgrCheckSymbolPlacement -window <giWindow | int> -context  
    <deContext>
```

Returns

<none>

Arguments

`-window`

VDRC Manager Window.

`-context`

Design Context.

db::vdrcMgrClearAllData

Clears all data in the VDRC Manager.

Syntax

```
db::vdrcMgrClearAllData -window <giWindow | int> [-context <deContext>]
```

Returns

<none>

Arguments

-window

VDRG Manager Window.

-context

Layout design context.

db::vdrMgrDeleteNet

Deletes net in VDRG Manager.

Syntax

```
db::vdrMgrDeleteNet -window <giWindow | int> -name <string>
```

Returns

<none>

Arguments

-window

VDRG Manager Window.

-name

Net Name to Delete from VDRG Window.

db::vdrMgrDeleteNetPair

Removes net pair in VDRG Manager.

Syntax

```
db::vdrMgrDeleteNetPair -window <giWindow | int> -name <string> -udv  
<bool>
```

Returns

<none>

Arguments

`-window`

VDRG Manager window.

`-name`

Net pair name to delete from VDRG Window.

`-udv`

When the net pair UDV type is:

- `true`: UDV
- `false`: Vsync

db::vdrMgrImportLayout

Import Layout Data to VDRG Manager

Syntax

```
db::vdrMgrImportLayout -window <giWindow | int> -context <deContext>  
[-hierarchy <bool>]
```

Returns

<none>

Arguments

`-window`

VDRG Manager Window.

`-context`

Design Context.

`-hierarchy`

Import all hierarchies of layout designs.

db::vdrMgrImportSchematic

Import Schematic Data to VDRG Manager.

Syntax

```
db::vdrMgrImportSchematic -window <giWindow | int> -context <deContext |  
heContext> [-hierarchy <bool>]
```

Returns

<none>

Arguments

-window

VDRRC Manager Window.

-context

Design or Hierarchy Context of schematic design.

-hierarchy

Import all hierarchies of schematic designs.

db::vdrcMgrLoadDRCViolations

Loads DRC Violations to VDRRC Manager.

Syntax

```
db::vdrcMgrLoadDRCViolations -window <giWindow | int> -context  
    <deContext>
```

Returns

<none>

Arguments

-window

VDRRC Manager Window.

-context

Design Context.

db::vdrcMgrRefresh

Imports Schematic Data to VDRRC Manager.

Syntax

```
db::vdrcMgrRefresh [-window <giWindow | int>]
```

Returns

<none>

Arguments

`-window`

VDRC Manager Window to refresh. Refresh all the VDRC Manager Windows if no window specified.

db::viaDefExists

Description

Determines if the named via definition exists within a tech object. If the value of the `-of` argument is an `oaVia`, the `oaTech` object associated with the given via is used for the definition name search.

Note:

The following example implements `db::findViaDef` in Tcl:

```
proc db::viaDefExists {name obj local} {  
    if {[db::getAttr type -of $obj] == oaVia} {  
        set tech [oa::getTech [oa::getDesign $obj]]  
    } else {  
        set tech $obj  
    }  
    return [oa::ViaDefFind $tech $name $local]  
}
```

Syntax

```
db::viaDefExists <name: string> -of <oaTech | oaVia> [-local <bool>]
```

Arguments

`name`

The name of the via definition to search for.

`-of`

The via/tech object searched to find the specified via definition.

`-local`

Specifies if the via def should be looked for in only the passed in `oaTech` or the `oaTech` associated with the passed in `oaVia`. If false, then all referenced databases are searched as well.

The default is true.

Returns

If the named via definition exists (<bool>).

Examples

```
set design [ed]
puts [db::viaDefExists via12 -of [db::getAttr tech -of $design]]
```

db::viewExists

Description

Determines if the named view exists for the specified instance/cell. If an instance is passed as the value of the -of argument, the master of the instance is used to determine the cell to search for the given view name.

Syntax

```
db::viewExists <name: string> -of <dmCell | oaInst>
```

Arguments

name

The name of the view to search for.

-of

The instance/cell searched to find the specified view.

Returns

If the named view exists for the master of the specified instance (<bool>).

Examples

The command implements the logic described in the following example:

```
proc db::viewExists {view cell} {
  if {[db::getAttr type -of $cell] == oaInst} {
    set inst $cell
    set cell [dm::findCell [db::getAttr cellName -of $inst] \
      -lib [dm::findLib [db::getAttr libName -of $inst]]]
  }
  return [expr![db::isEmpty [dm::getCellViews $view -cell $cell]]]
}
```

db::vwait

Description

Blocks the script execution until variable changes.

Syntax

```
db::vwait <name: string>
```

Arguments

name

Variable name which should be monitored for changes.

Returns

<none>

idb::createImage

Description

Creates an image object.

Syntax

```
idb::createImage [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

Examples

```
set design [ed]
db::createImage ./amp.png -design $design -origin {1 1} -size 1
```

idb::query

Description

An interactive command. Generates a report of the attributes of any object in the design hierarchy. Lets you choose any visible and selectable object anywhere in the hierarchy and generate a report to the console.

Syntax

```
idb::query [<window: unsigned int>]
```

Arguments

window

The window in which to query.

Default: active window.

Returns

<none>

Enumerations

Table 9 Enumerations

| Name | Values | Description |
|----------------------|--|---|
| enImageFormat | PNG JPEG BMP | The format for image output. |
| enNameDisplay | off default inst cell | Defines choices for cdsName label display |
| enNetlistInfoSection | netlister display form | The section of the netlist info to read/write. |
| enNetlistInfoType | integer real string symbol list | The type of field stored in the netlist info. |
| enParamEvalType | full netlist debug none | The method used for evaluation. |
| enParamType | string boolean float integer radio
cyclic button netSet | The types of parameter editors. |
| enParamValueType | string boolean float double integer | The types of parameter storage. |
| enPcellPlugin | oaTcl oaCPP | The plug-in to be associated with a PCell Definition. |
| enPinSide | left right top bottom unknown | Locations for pins around the core rectangle. |
| enPinType | input output inputOutput switch
jumper unused tristate | Pin type names supported by OA. These names are consistent with the OA API. |
| enPrefScope | user library cell cellView context
custom | Scope types available for automatic localization in <code>db::createPref</code> . |
| enPrefType | bool int string float point lcv | Preference value type. |

Table 9 Enumerations (Continued)

| Name | Values | Description |
|------------------------------|--|---|
| enPrintFit | page width height | The scaling fit mode. |
| enPrintHierMode | inContext all | The mode of the hierarchical traversal. |
| enPrintLocation | disabled left right top bottom
separatePage | The location of the legend. |
| enPrintOrientation | auto portrait landscape | The orientation mode. |
| enTechFormat | XML Legacy | Format of technology file to be imported. |
| enTermDisplay | off default term net | Defines choices for cdsTerm label display. |
| enValueType | bool int float double string time enum
intRange floatRange doubleRange
timeRange | Type of value. |
| netlistParserParamUpdateMode | always never prompt | Mode to overwrite the default of CDF parameters during text import. |

Objects

This section covers the following:

- [dbAssignAssignment](#)
- [dbAssignmentDef](#)
- [dbAssignValue](#)
- [dbCallback](#)
- [dbConnections](#)
- [dbContextConnectionDef](#)
- [dbContextConnections](#)
- [dbDeviceDisplay](#)
- [dbGenOptionSet](#)
- [dbInstCache](#)
- [dbInterfaceParam](#)

- [dbInterfacePin](#)
- [dbInterfaceSet](#)
- [dbParam](#)
- [dbParamDef](#)
- [dbPcellDef](#)
- [dbPref](#)
- [dbPrinter](#)
- [dbScope](#)
- [dmCell](#)
- [dmCellCategory](#)
- [dmCellView](#)
- [giWindow](#)
- [heContext](#)
- [leMPP](#)
- [oaInst](#)
- [oaInstTerm](#)
- [oaNet](#)
- [oaPin](#)
- [oaShape](#)
- [oaTerm](#)
- [oaVia](#)
- [saConvergenceAid](#)
- [saDesignVariable](#)
- [saDesignVariables](#)
- [saDesignVariableSet](#)
- [saIncludeFile](#)
- [saModelFile](#)

- [saOutput](#)
- [saOutputs](#)
- [saOutputSet](#)
- [saPair](#)
- [saSession](#)
- [saTemperature](#)
- [saTestbench](#)
- [wfDataFile](#)
- [xtJob](#)

dbAssignAssignment

Corresponds to an `oaDesign` object that defines a connect definition assignment to a new definition. The assignment can correlate to an instance or to the design in general. This object correlates to an actual OA database object. If this object is passed to `db::destroy`, not only this object's resources are freed/released, but the underlying OA database object is destroyed (that is, removed from the design). To free up the resources used by this object without deleting the OA object, all Tcl variables holding this object should be unset.

Table 10 *Attributes*

| Name | Type | Access | Description |
|------------|-----------------|------------|--|
| name | string | Read/Write | Name of connection definition being assigned. |
| newname | string | Read/Write | New name for connect definition to continue up the hierarchy. |
| defaultNet | string | Read/Write | Default net for this override. |
| instance | <i>oaInst</i> | Read-only | The instance object in the design if the assignment is instance level, else this value will be NULL. |
| design | <i>oaDesign</i> | Read-only | Design containing the assignment object. |

dbAssignmentDef

Contains assignment information for a specific `dbConnectionDef` object.

Table 11 *Attributes*

| Name | Type | Access | Description |
|------------|---|-----------|--|
| design | <i>oaDesign</i> | Read-only | Design containing assignment. |
| instance | string | Read-only | Instance name in design which has the assignment. If this is a design wide assignment, this string will be empty or not set. |
| isOverride | bool | Read-only | Indicates this is an override assignment (dbAssignAssignment). |
| assignment | <i>dbAssignValue</i>
<i>dbAssignAssignment</i> | Read-only | Custom Compiler assignment object. |

dbAssignValue

Corresponds with an *oaDesign* object which defines a connect definition assignment to a net name. The assignment can correlate to an instance or to the design in general. This object correlates to an actual OA database object. If this object is passed to `db::destroy`, not only the object resources are freed/released, but the underlying OA database object is destroyed (that is, removed from the design). To free up the resources used by this object without deleting the OA object, all Tcl variables holding this object should be unset.

Table 12 *Attributes*

| Name | Type | Access | Description |
|----------|-----------------|------------|--|
| name | string | Read/Write | Name of connection definition being assigned. |
| value | string | Read/Write | Name of net to assign. |
| instance | <i>oaInst</i> | Read-only | The instance object in the design if the assignment is instance level, else this value will be NULL. |
| design | <i>oaDesign</i> | Read-only | Design containing the assignment object. |

dbCallback

The `dbCallback` object represents a user-defined callback for a system event exported to Tcl by the Postman. The attributes of the object are described in the following table and can be retrieved or set by `db::getAttr` and `db::setAttr` commands. The full list of available callbackTypes, and their data signatures, can be found with the `db::listCallbackTypes` command.

Table 13 *Attributes*

| Name | Type | Access | Description |
|--------------|----------|------------|--|
| name | string | Read/Write | callback name. |
| callbackType | string | Read-only | Event which will trigger this callback. |
| viewType | string | Read-only | View type name, if applicable, for which this callback is to be triggered. An empty string indicates the viewType should be fired for all viewTypes. |
| procedure | string | Read-only | Tcl procedure which is called when this callback is triggered. |
| priority | int | Read/Write | priority of execution relative to other callbacks for this event. |
| eventData | [string] | Read-only | additional arguments that will be passed to the callback. |

dbConnections

Describes the complete set of connect definitions that exist in the hierarchy defined by the given design context.

Table 14 *Attributes*

| Name | Type | Access | Description |
|------------|--------------------------|-----------|--|
| top | <i>heContext</i> | Read-only | Top design context. |
| parameters | <i>{dbConnectionDef}</i> | Read-only | Collection of all connection definitions in the hierarchy. |

dbContextConnectionDef

Describes a specific connect definition in a specific design context.

Table 15 *Attributes*

| Name | Type | Access | Description |
|---------------|------------------------|-----------|--|
| context | <i>heContext</i> | Read-only | context. |
| connectionDef | <i>dbConnectionDef</i> | Read-only | connection definition. |
| state | <i>enAssignState</i> | Read-only | The state of the connect definition in this context. |

Table 15 *Attributes (Continued)*

| Name | Type | Access | Description |
|-------------|-------------------|-----------|--|
| instances | [string] | Read-only | A Tcl list of instance object names in this context which have this connect definition in their hierarchy. |
| assignments | {dbAssignmentDef} | Read-only | Collection of assignment information objects if this connect definition has been assigned in this context. |

dbContextConnections

Describes the set of the connection definitions found in a specific context.

Table 16 *Attributes*

| Name | Type | Access | Description |
|--------------|--------------------------|-----------|---|
| context | heContext | Read-only | The context. |
| parameters | {dbContextConnectionDef} | Read-only | Collection of context connection definitions. |
| instances | [string] | Read-only | A Tcl list of oalnst object names which have any of connect definitions from this context in their hierarchy. |
| virtualTerms | [string] | Read-only | Tcl list of strings which specify virtual terminals required. |

dbDeviceDisplay

The dbDeviceDisplay object defines the state of the device label display for a given Custom Compiler session. This object may be created via the db::setDeviceDisplay or db::loadDeviceDisplay command, and destroyed via the db::destroy command. The scope attribute of the dbDeviceDisplay object is a by-name reference to the corresponding instance, design, cellView, library or master. If the name of the referenced object is changed, the scope attribute must be changed accordingly.

Table 17 *Attributes*

| Name | Type | Access | Description |
|--------|----------------------|------------|--|
| scope | string | Read-only | Scope of the label display definition for the given dbDeviceDisplay object. |
| type | <i>enScopeType</i> | Read-only | Indication of whether the scope string represents a design, library, instance or master. |
| lcv | <i>dmCellView</i> | Read-only | Corresponding design for scopes that reference an instance or that reference a design-specific master. |
| terms | <i>enTermDisplay</i> | Read/Write | Indicates if the cdsTerm labels of the instance(s) evaluate to the corresponding instTerm's name or net attribute. |
| name | <i>enNameDisplay</i> | Read/Write | Indicates if the cdsName label of the instance(s) evaluate to the instances name or cellName attribute. |
| params | string | Read/Write | Indicates the list of parameters for cdsParam evaluation. If the parameter name is prepended with "-", the value only will be displayed. |

dbGenOptionSet

Contains a set of values for use during cellView generation.

Table 18 *Attributes*

| Name | Type | Access | Description |
|---------|--------|------------|---|
| name | string | Read/Write | Name of an SCP set. |
| setType | string | Read-only | Type of the generator option set. |
| value | string | Read/Write | Data representing values for use by a generator. Manipulated as a string. |

dbInstCache

Represents the instance cache. An instance cache is a data construction, which stores modified parameters after triggering callbacks of CDF parameters.

dbInterfaceParam

Contains the interface information for use in an interface set.

Table 19 Attributes

| Name | Type | Access | Description |
|-----------|-----------------------------|------------|--|
| name | string | Read-only | Name of the interface parameter. |
| paramType | <i>enValueType</i> | Read-only | Value type of this parameter. |
| isCDF | bool | Read-only | Indicates if this parameter should be created as a CDF parameter or a general design property. |
| defValue | bool int float string | Read-only | Default value for this parameter. |
| minValue | int float | Read/Write | Minimum value if the parameter is a range type. |
| maxValue | int float | Read/Write | Maximum value if the parameter is a range type. |
| enums | {string} | Read/Write | List of valid string values if paramType is enum. |

dbInterfacePin

Contains the interface pin information for use in an interface set.

Table 20 Attributes

| Name | Type | Access | Description |
|------------|------------------|-----------|---|
| name | string | Read-only | Name of the interface pin. |
| pinType | <i>enPinType</i> | Read-only | Pin type. |
| side | <i>enPinSide</i> | Read-only | Optional side assignment for this interface pin. |
| position | int | Read-only | Optional number describing this pins order on the side. |
| connectDef | map | Read-only | Optional pair of strings defining the connect definition on the terminal. |

dbInterfaceSet

Contains the interface information for a design for use with the CTC system.

Table 21 *Attributes*

| Name | Type | Access | Description |
|------------|---------------------------|------------|--|
| pins | <i>{dbInterfacePin}</i> | Read/Write | Collection of interface pins. |
| parameters | <i>{dbInterfaceParam}</i> | Read/Write | Collection of interface parameters. |
| source | <i>dmCellView</i> | Read-only | Reference to source cellView if known. |

dbParam

Represents an instance of a parameter. It is always associated with an *oaInst*. Every *dbParam* object has a definition given by an associated *dbParamDef* object. The attributes of the object are described in the following table and can be retrieved or set by the *db::getAttr* and *db::setAttr* commands. Some of the attributes of *dbParam*'s are the result of evaluating the same named attribute in the parent *dbParamDef*.

Table 22 *Attributes*

| Name | Type | Access | Description |
|-----------|-------------------|------------|--|
| value | string | Read/Write | The non-evaluated value of the parameter. |
| ref | <i>oaRef</i> | Read-only | The parent instance/via of the <i>dbParam</i> object. |
| paramDef | <i>dbParamDef</i> | Read-only | The definition of the parameter. |
| choices | [string] | Read-only | The choices to use for the enumeration parameter types. This is the result of evaluating the parameter's <i>dbParamDef</i> -choices field. |
| display | bool | Read-only | This determines if the parameter should be visible on the Create/add instance and Edit Properties form. This is the result of evaluating the parameter's <i>dbParamDef</i> -display field. |
| editable | bool | Read-only | This determines if the parameter should be editable on the Create/add instance and Edit Properties form. This is the result of evaluating the parameter's <i>dbParamDef</i> -editable field. |
| isDefault | bool | Read-only | If the parameter does not have a local override value. Always false for <i>PCell</i> parameters. |
| use | bool | Read-only | If the parameter should be displayed in the Create Instance/Property Editor forms. Also used by netlisters to determine if the parameter should be printed. This is commonly used for providing an interface that presents a variable number of elements like the Time/Voltage pairs of a piecewise linear voltage source. |

dbParamDef

Represents the definition of a parameter. It is stored within either library or cell level `oaDMDData`. `dbParam`'s are instances of `dbParamDef` objects and are always associated with an `oalnst`. The attributes of the `dbParamDef` object are described in the following table and can be retrieved and set by the `db::getAttr` and `db::setAttr` commands.

Table 23 Attributes

| Name | Type | Access | Description |
|---------------|--------------------|------------|---|
| data | <i>oaDMDData</i> | Read-only | The storage location of the <code>dbParamDef</code> . |
| callback | string | Read/Write | Parameter change callback. This is evaluated as Callback Syntax whenever the parameter value changes. |
| choices | string | Read/Write | The choices to use for the enumeration parameter types. |
| defValue | string | Read/Write | The default value of the parameter. |
| display | string | Read/Write | If the parameter should be displayed in the Create/add instance/ Edit Properties forms. |
| editable | string | Read/Write | If the parameter should be editable in the Create/add instance and Edit Properties forms. |
| name | string | Read/Write | The name of the parameter. |
| type | <i>enParamType</i> | Read/Write | The type of the parameter. |
| parseAsPEL | bool | Read/Write | If the parameter should be interpreted as a PEL expression. |
| parseAsNumber | bool | Read/Write | If the parameter should be interpreted as a number if referenced from PEL expressions. |
| prompt | string | Read/Write | The textual description for a parameter used in Property Editing and Instance creation forms. |
| units | string | Read/Write | The units of the parameter. |
| use | string | Read/Write | If the parameter should be displayed in the Create Instance/Property Editor forms.
Also used by netlisters to determine if the parameter should be printed. This is commonly used for providing an interface that presents a variable number of elements like the Time/Voltage pairs of a piecewise linear voltage source. |

dbPcellDef

Represents a PCell definition.

Table 24 Attributes

| Name | Type | Access | Description |
|--------------|----------------------|-----------|---|
| name | string | Read-only | The name of the PCell definition. |
| library | <i>oaLib</i> | Read-only | The library in which this PCell definition exists. |
| plugin | <i>enPcellPlugin</i> | Read-only | The plug-in this PCell definition uses. |
| defFiles | {dmDMFile} | Read-only | The collection of definition dmDMFiles for this PCell definition. |
| supportFiles | {dmDMFile} | Read-only | The collection of support dmDMFiles for this PCell definition. |

dbPref

Represents a localized preference within a scope. This object may be obtained with the `db::getPrefs` command and modified using `db::setAttr` or `db::setPrefValue`. If destroyed with the `db::destroy` command, the preference at the corresponding scope is removed and thus de-localized. It is not possible to destroy preferences at install scope.

Table 25 Attributes

| Name | Type | Access | Description |
|--------------|--------------------|------------|---|
| name | string | Read-only | Preference name. |
| description | string | Read-only | Preference description. |
| scope | <i>dbScope</i> | Read-only | Scope in which this value exists. |
| modified | bool | Read-only | If true, the value has been modified since being loaded or saved. |
| labels | [string] | Read-only | Preference labels. |
| defaultScope | <i>enPrefScope</i> | Read-only | Default scope at which to localize preference. |
| value | string | Read/Write | Preference value. |
| valueType | <i>enPrefType</i> | Read-only | Value type. |

dbPrinter

Defines a printer and its properties.

Table 26 Attributes

| Name | Type | Access | Description |
|-------------|----------|------------|--|
| name | string | Read-only | The name of the printer. |
| description | string | Read/Write | A description of the printer. |
| resolution | int | Read/Write | The number of addressable units per inch (DPI or PPI). |
| paperSize | [string] | Read/Write | An array of paper sizes supported by the printer, where each index contains name, width, height, offsetX and offsetY members (using data types: <string>, <int>, <int>, <int>, <int>). The data is specified according to the units defined by the resolution value. |
| spoolProc | string | Read/Write | The command used to send the print request to the printer queue. |
| queryProc | string | Read/Write | The command used to query the printer's queue. |
| removeProc | string | Read/Write | The command used to remove a queued print job. |
| doubleSided | bool | Read/Write | Specifies if printer supports double-sided printing. |
| color | bool | Read/Write | Specifies if the printer supports color. |
| display | string | Read/Write | The default display to use with this printer. |
| olpGroup | string | Read/Write | The default OLP group to use with this printer. |

dbScope

Represents a preference scope. This object may be created with the `db::createScope` command, obtained with the `db::getScopes` command. If destroyed with the `db::destroy` command, then all children of this scope are destroyed too. It is not possible to destroy system-defined scopes.

Table 27 Attributes

| Name | Type | Access | Description |
|------|--|-----------|-------------------|
| id | <code>oaLib</code> <code>dmCell</code>
 <code>dmCellView</code>
<code>deContext</code> string | Read-only | Scope identifier. |

Table 27 *Attributes (Continued)*

| Name | Type | Access | Description |
|--------|----------------|-----------|---|
| parent | <i>dbScope</i> | Read-only | Scope from which this scope inherits its values. When requesting the parent of a parentless scope such as the install scope, "" will be returned. |
| env | string | Read-only | Environment in which this scope exists. |

dmCell

A named container, in a library, that can hold files and dmCellViews. The dmCell corresponds to a specific oaCell object but is valid as long as the library that the dmCell belongs to remains open. The `getAccess` requirements of an oaCell are handled automatically by this dm object.

Table 28 *Attributes*

| Name | Type | Access | Description |
|------------|--------|------------|--|
| name | string | read-only | Cell name. |
| libName | string | read-only | Library name of the cell. |
| statusIcon | string | read/write | Incremental path to an icon supplied by the version control system. |
| shown | bool | read/write | Hides cells for the current session. This attribute works independently of the Hidden category setting. Setting the attribute does not refresh the Library/Cell/View lists in the Library Manager. |

dmCellCategory

You can group cells in libraries into categories to simplify the task of finding a particular cell or of browsing the cells in a library. Each category is represented by a dmCellCategory. You can create them and find out the set of cells in a category. Categories can contain other categories.

Table 29 *Attributes*

| Name | Type | Access | Description |
|------|--------|------------|----------------------------|
| name | string | Read/Write | Name of the cell category. |

Table 29 *Attributes (Continued)*

| Name | Type | Access | Description |
|------------|-------------------------|-----------|--|
| lib | <i>oaLib</i> | Read-only | The library where the cell category is defined. |
| cells | <i>{dmCell}</i> | Read-only | Collection of <i>oaCells</i> in the cell category. |
| categories | <i>{dmCellCategory}</i> | Read-only | Collection of cell categories that are hierarchically contained in this cell category. |

dmCellView

The *dmCellView* is a container in a library that holds files and is identified by the combination of its cell and view names. It corresponds to an *oaCellView* but it stays valid as long as its library remains open and its *dmCell* and *dmView* exist.

Table 30 *Attributes*

| Name | Type | Access | Description |
|----------|---------------|-----------|---|
| name | string | Read-only | The name of the view that this <i>cellView</i> belongs to. |
| cellName | string | Read-only | The name of the cell to which this <i>cellView</i> belongs. |
| cell | <i>dmCell</i> | Read-only | The <i>cellView</i> 's cell. |
| libName | string | Read-only | The name of the library that this <i>cellView</i> belongs to. |

giWindow

The *giWindow* object represents a single window and has a container of menus, toolbars, assistants and one status bar. Menus, toolbars and assistants may be retrieved by the API. The status bar and toolbar that are used to hold menus are available here as attributes as they are non-removable attributes of the window object. The attributes of the object are described in the following table and can be retrieved or set by `db::getAttr` and `db::setAttr` commands. Windows are created by various subsystems in Custom Compiler including Design Editor.

Table 31 *Attributes*

| Name | Type | Access | Description |
|------------|---------------------|------------|--|
| id | int | Read-only | Window identifier. |
| windowType | <i>giWindowType</i> | Read-only | Window type object. |
| title | string | Read-only | Title displayed in window border. |
| iconTitle | string | Read/Write | Title given to the window's icon and displayed in the window manager's task list. |
| geometry | string | Read/Write | Window geometry. |
| iconified | bool | Read/Write | Whether the window is iconified. |
| maximized | bool | Read/Write | Whether the window is maximized. Note that this can be true even if the window is currently iconified. |
| statusBar | <i>giStatusBar</i> | Read-only | Status bar object holding actions that will render this window's status bar. If empty, the status bar will not be displayed in the window. |

heContext

Represents a hierarchical iterator. It can be used to traverse Placed, Switched, and View Search List hierarchies.

Table 32 *Attributes*

| Name | Type | Access | Description |
|----------------|-------------------|-----------|---|
| topCellView | <i>dmCellView</i> | Read-only | The root Cellview of the context. |
| config | <i>heConfig</i> | Read-only | The heConfig object the context is traversing. Can be empty. |
| occurrence | string | Read-only | The current descent lineage of the context in the oaCdbaNS namespace. |
| design | <i>oaDesign</i> | Read-only | The current design. |
| viewSearchList | [string] | Read-only | The effective View Search List for the current descent path. |
| viewStopList | [string] | Read-only | The list of view names used as the stop list in he::isLeaf. |
| cellView | <i>dmCellView</i> | Read-only | The current cell view. |

Table 32 *Attributes (Continued)*

| Name | Type | Access | Description |
|--------------------|---------------|-----------|---|
| isRoot | bool | Read-only | True if the context is at the root of the hierarchy (occurrence == "/"). |
| parentInst | <i>oaInst</i> | Read-only | The parent instance of the context. |
| explicitParentInst | <i>oaInst</i> | Read-only | The explicit parent instance. If the parent instance is an implicit instance then the containing explicit <i>oaVectorInst</i> will be returned. |

leMPP

Represents the MPP object, which is an *oaAppDef* object with an *oaGroup* of type MPP. The attributes of the object are described in the following table and can be retrieved by `db::getAttr` command.

Table 33 *Attributes*

| Name | Type | Access | Description |
|------------|-------------------|-----------|--|
| definition | string | Read-only | The MPP definition name. |
| name | string | Read-only | The MPP object name. |
| centerline | [point] | Read-only | The list of points that define the centerline of the MPP object. |
| lines | [<i>oaLine</i>] | Read-only | List of lines that define the non-chopped parts of the centerline. Lines must be covered by centerline (there should not be a line outside of the centerline). |
| objects | [<i>oaFig</i>] | Read-only | The list of <i>oaFig</i> objects that form the MPP object. |

oaInst

Represents block domain instances. An instance represents the inclusion of one design as a part of the contents of another. Use the `db::getAttr` command to retrieve the *oaInst* attributes described in the following table.

This table is the subset of attributes supported by the object access API, `db::listAttrs`, `db::getAttr`, and `db::setAttr`. For the full list of attributes, see the Si2 OpenAccess documentation.

Table 34 *Attributes*

| Name | Type | Access | Description |
|-----------------|------------------------------|------------|---|
| name | string | Read-only | The name of the oalnst. |
| design | <i>oaDesign</i> | Read-only | Design to which the instance belongs. |
| master | <i>oaDesign</i> | Read-only | Master of the instance. |
| orient | <i>oaOrientEnum</i> | Read/Write | Orientation of the instance. |
| origin | point | Read/Write | Origin of the instance. |
| libName | string | Read-only | The library name for the master design referenced by the instance. |
| cellName | string | Read-only | The cell name for the master design referenced by the instance. |
| viewName | string | Read-only | The view name for the master design referenced by the instance. |
| header | <i>oaInstHeader</i> | Read-only | The instHeader associated with this instance. |
| numBits | int | Read-only | The number of bits of this instance. |
| params | <i>oaParamArray</i> | Read/Write | Parameters that were specified during creation. Only PCell instances have parameters. |
| placementStatus | <i>oaPlacementStatusEnum</i> | Read/Write | The placement status for the instance. |
| source | <i>oaSourceEnum</i> | Read/Write | The source for the instance. |
| isImplicit | bool | Read-only | A boolean that indicates if the instance is implicit. |

oalnstTerm

Represents all types of connections between a net and a terminal in the master of an instance. The connection is valid if the number of bits in the net equals the number of bits implied by the instTerm's terminal name. Use the `db::getAttr` command to retrieve the oalnst attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 35 *Attributes*

| Name | Type | Access | Description |
|--------------|---------------|-----------|---|
| name | string | Read-only | The name of the term associated with this instTerm. |
| net | <i>oaNet</i> | Read-only | The net associated with this instTerm. |
| inst | <i>oaInst</i> | Read-only | The inst associated with this instTerm. |
| term | <i>oaTerm</i> | Read-only | The term associated with this instTerm. |
| termPosition | int | Read-only | The term position that this instTerm connects to. |
| numBits | int | Read-only | The number of bits for this instTerm. |
| isImplicit | bool | Read-only | A boolean value indicating whether this instTerm is implicit. |

oaNet

Represents scalar and multi-bit nets that are part of the design's physical description. Nets represent the logical connectivity within a block of a design. Nets connect to terminals, which are the logical connection points on instances of the net's block. Nets also connect to instTerms that represent connections to the lower-level instances in the net's block.

Use the `db::getAttr` command to retrieve the *oaNet* attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 36 *Attributes*

| Name | Type | Access | Description |
|------------|----------------------|------------|---|
| name | string | Read-only | Name of <i>oaNet</i> . |
| design | <i>oaDesign</i> | Read-only | Design to which the net belongs. |
| instTerms | <i>oaInstTerm</i> | Read-only | The instTerms associated with this net. |
| isEmpty | bool | Read-only | A boolean value that indicates if the net has any objects attached to it or associated with it. |
| isGlobal | bool | Read/Write | A boolean value that indicates if the net is a global. |
| isImplicit | bool | Read-only | A boolean value that indicates if the net is implicit. |
| numBits | int | Read-only | The number of bits in the net. |
| sigType | <i>oaSigTypeEnum</i> | Read/Write | The signal type of the net. |

Table 36 *Attributes (Continued)*

| Name | Type | Access | Description |
|------|--------------|-----------|-----------------------------------|
| vias | <i>oaVia</i> | Read-only | The vias associated with the net. |

oaPin

Represents the physical connections of terminals to nets. Use the `db::getAttr` command to retrieve the `oaPin` attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 37 *Attributes*

| Name | Type | Access | Description |
|-----------------|------------------------------|------------|---|
| name | string | Read/Write | Name of <code>oaPin</code> . |
| design | <i>oaDesign</i> | Read-only | Design to which the net belongs. |
| term | <i>oaTerm</i> | Read/Write | The terminal to which this pin is attached. |
| accessDir | int | Read/Write | The access direction bit-field. |
| placementStatus | <i>oaPlacementStatusEnum</i> | Read/Write | The placement status of the pin. |

oaShape

Defines specific geometry at specific locations in a design. Shapes, together with routes, specify all the manufacturable geometry in a chip layout. Shapes can also be used for more abstract purposes. The Text shape classes provide viewable labels on design objects. All shapes are placed on an `oaLayer` and an `oaPurpose`. The design API identifies these by the layer and purpose numbers. The corresponding names are found in the technology database. Optionally, shapes can also belong to either an `oaNet` or an `oaPin` to provide part of the physical implementation of that object. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 38 *Attributes*

| Name | Type | Access | Description |
|--------|-----------------|-----------|---------------------------------------|
| design | <i>oaDesign</i> | Read-only | The design in which the shape exists. |
| bBox | box | Read-only | The figure bounding box. |

Table 38 *Attributes (Continued)*

| Name | Type | Access | Description |
|----------------------|--------------|------------|--|
| layerNum | int | Read/Write | The layer number associated with this shape. |
| leConnectivityAnchor | bool | Read-only | Indicates if the object is a connectivity anchor. |
| purposeNum | int | Read/Write | The purpose number associated with this shape. |
| net | <i>oaNet</i> | Read-only | The net this shape implements. Returns NULL if this figure does not implement a net, NULL is returned. Note that this function will return a net if the shape is directly assigned to a net or if the shape is in a route that is assigned to a net. It will not return a net (it will return NULL) if the shape belongs to a pin. |
| pin | <i>oaPin</i> | Read-only | The pin this shape implements. If this figure does not implement a pin, NULL is returned. |

oaTerm

Represents scalar and multi-bit terminals. Terminals represent the logical connection points for a block. The pins associated with terminals represent the physical connection points. The nets associated with the terminals are logically exported through the terminals to the next higher level in a design hierarchy. Use the `db : :getAttr` command to retrieve the `oaTerm` attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 39 *Attributes*

| Name | Type | Access | Description |
|-------------|-----------------------|------------|--|
| name | string | Read-only | The name of the terminal. |
| isImplicit | bool | Read-only | A boolean value that indicates if this terminal is implicit. |
| isInterface | bool | Read/Write | A boolean indicating whether this terminal is an interface terminal. |
| net | <i>oaNet</i> | Read-only | The net to which this terminal is attached. |
| numBits | int | Read-only | The number of bits in this terminal. |
| termType | <i>oaTermTypeEnum</i> | Read/Write | The terminal type of this terminal. |
| position | int | Read/Write | The position that this terminal is assigned to. |

oaVia

Represents vias. A via represents the physical connection between two adjacent route pathSegs on different layers. A via may be either a standard via or a custom via. A standard via references a via definition (oaViaDef) from the technology database that corresponds to the design where the via exists. A standard via has a pre-defined set of via parameters that can be tailored to create a variety of vias to cover a variety of connections. The standard via references a standard via definition. A custom via references another design. Custom vias often include a set of parameter values that can be used to tailor the master for each specific via. Although vias are normally associated with routes, a via can represent complex pin figures. Vias always have an oaViaHeader that contains the attributes that are constant for all vias of a given via definition. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 40 *Attributes*

| Name | Type | Access | Description |
|----------------------|--------------------|-----------|--|
| name | string | Read-only | The name of the corresponding oaViaDef. |
| design | <i>oaDesign</i> | Read-only | The design in which the object exists. |
| header | <i>oaViaHeader</i> | Read-only | The viaHeader associated with this via. |
| leConnectivityAnchor | bool | Read-only | Indicates if the object is a connectivity anchor |
| master | <i>oaDesign</i> | Read-only | Master design for custom vias. Returns NULL if unbound or for standard vias. |
| viaDef | <i>oaViaDef</i> | Read-only | The viaDef associated with this via. |
| purposeNum | int | Read-only | The purpose number associated with this via. |
| layer1Num | int | Read-only | The first layer associated with oaViaDef. |
| layer2Num | int | Read-only | The second layer associated with oaViaDef. |

saConvergenceAid

This object is a collection of convergence aids, initial conditions, node sets and force nodes. Each of these is a collection of name value pairs.

Table 41 Attributes

| Name | Type | Access | Description |
|----------|--------|------------|--|
| enabled | bool | Read/Write | Enabled status. |
| nodeType | string | Read/Write | Node type (node set, initial condition or force node). |
| node | string | Read/Write | Name of the circuit node. |
| voltage | string | Read/Write | Voltage to be applied. |

saDesignVariable

This object represents design variable defined in a testbench.

Table 42 Attributes

| Name | Type | Access | Description |
|-------|--------|------------|------------------------|
| name | string | Read/Write | Design Variable Name. |
| value | string | Read/Write | Design variable value. |

saDesignVariables

This object represent design variable setup on a testbench. It has two attributes. variableSet is the name of current variable set and variableSets is the collection of variableSets defined for the testbench.

Table 43 Attributes

| Name | Type | Access | Description |
|--------------|-----------------------|------------|--|
| variableSet | string | Read/Write | Name of the current design variable set. |
| variableSets | {saDesignVariableSet} | Read/Write | Collection of design variable sets. |

saDesignVariableSet

The saDesignVariableSet object represents a collection of design variables defined for a testbench or project. designVariables is a collection of design variables. name is the name of this design variable set.

Table 44 *Attributes*

| Name | Type | Access | Description |
|-----------------|--------------------|------------|-----------------------------------|
| name | string | Read/Write | Name of this design variable set. |
| designVariables | {saDesignVariable} | Read/Write | Collection of design variables. |

saIncludeFile

This object represents an include file. The attributes are include file path, include file type and the enabled status of the include file.

Table 45 *Attributes*

| Name | Type | Access | Description |
|---------|--------|------------|-------------------------------------|
| file | string | Read/Write | Include file path. |
| type | string | Read/Write | Include file type. |
| enabled | bool | Read/Write | Enabled status of the include file. |

saModelFile

saModelFile represent a model file that user has specified. The filename field is an absolute or relative path to the model file to be included. section field specifies a section or library within the model file, may be empty.

Table 46 *Attributes*

| Name | Type | Access | Description |
|----------|--------|------------|-----------------------------------|
| filename | string | Read/Write | Path to model file. |
| section | string | Read/Write | Model file section. |
| enabled | bool | Read/Write | Enabled status of the model file. |

saOutput

This object represents an entry in the outputs table in main PrimeWave Design Environment window.

Table 47 *Attributes*

| Name | Type | Access | Description |
|------------|--------------------------|------------|--|
| name | string | Read/Write | Name given to the output. |
| expr | string | Read/Write | Expression. When the output is a signal, expression would be the net or terminal name. |
| analyses | {string} | Read/Write | The analyses for which this output is evaluated/plotted. |
| plot | bool | Read/Write | Specifies if the output should be plotted when simulation is complete. |
| save | bool | Read/Write | Specifies if the output should be saved by the simulator. This will work with the simulation save options to create, probe (or simulator equivalent) commands. |
| value | string <i>wfSignal</i> | Read-only | Value of the expression. |
| outputType | <i>enOutputType</i> | Read/Write | Output type (net, terminal or expression). |
| probe | string | Read/Write | Display packet name to be used in cross-probing. |

saOutputs

This object represents the outputs and measurement setup defined for the testbench. It has two attributes. `outputSet` is the name of current outputSet. `outputSets` is a collection of `saOutputSet` objects.

Table 48 *Attributes*

| Name | Type | Access | Description |
|------------|------------------------|------------|-----------------------------|
| outputSet | string | Read/Write | Name of current output set. |
| outputSets | { <i>saOutputSet</i> } | Read/Write | Collection of output sets. |

saOutputSet

The `saOutputSet` object represents a collection of outputs defined for a testbench. The `outputs` attribute is a collection of output objects. `name` is the name of this output set.

Table 49 *Attributes*

| Name | Type | Access | Description |
|---------|------------|------------|-------------------------|
| name | string | Read/Write | Name of the output set. |
| outputs | {saOutput} | Read/Write | Collection of outputs. |

saPair

This is a generic name value pair object. It has two read/write attributes name and value.

Table 50 *Attributes*

| Name | Type | Access | Description |
|-------|--------|------------|---|
| name | string | Read-only | The name of the design variable, model file or measurement. |
| value | string | Read/Write | The value for the above. |

saSession

saSession object represents one invocation of the PrimeWave Design Environment in a Custom Compiler session. Session name can be specified by user or is auto generated by appending an integer to `saSession`. For example `saSession1`, `saSession2`, and so on. `resultsDir` is the root directory of results directory structure. `statesDir` is the root directory of states directory structure.

Table 51 *Attributes*

| Name | Type | Access | Description |
|-------------|---------------|------------|---|
| name | string | Read/Write | Unique string identifier for this session. |
| resultsDir | string | Read/Write | Results directory for PrimeWave Design Environment Session. |
| statesDir | string | Read/Write | States directory for PrimeWave Design Environment Session. |
| testbenches | {saTestbench} | Read/Write | Testbenches in the ceSession. |

saTemperature

This object represents the temperature setting for a simulator. Only values representing a real number are allowed.

Table 52 *Attributes*

| Name | Type | Access | Description |
|-------|-------------------|------------|--|
| value | string | Read/Write | The global temperature for simulation. |
| unit | <i>enTempUnit</i> | Read/Write | Temperature unit. |

saTestbench

`saTestbench` object represents a testbench defined in a `saSession`. It is owned by the `saSession`. It contains a reference to the design to be simulated, name of the simulator and the simulation setup to be used.

Table 53 *Attributes*

| Name | Type | Access | Description |
|--------------------|-----------------------------|------------|---|
| name | string | Read/Write | Testbench name. |
| design | <i>dmCellview</i> | Read/Write | The design to be simulated. |
| simulator | string | Read/Write | name of the simulator. |
| temperature | <i>saTemperature</i> | Read/Write | The global temperature for simulation. |
| options | <i>{saOption}</i> | Read/Write | Collection of simulation options. |
| designVariables | <i>{saDesignVariables}</i> | Read/Write | Design variable sets defined for this testbench. |
| convergenceAids | <i>{saConvergenceAid}</i> | Read/Write | Convergence aids defined for this testbench. |
| analyses | <i>{saAnalysis}</i> | Read/Write | Collection of analyses added to the testbench. |
| models | <i>saModelSetup</i> | Read/Write | Collection of model files. |
| environmentOptions | <i>saEnvironmentOptions</i> | Read/Write | Simulator independent options. |
| outputs | <i>saOutputs</i> | Read/Write | Collection of output sets. |
| includeFiles | <i>{saIncludeFile}</i> | Read/Write | Include files. |
| saveOptions | <i>saSaveOptions</i> | Read/Write | Output save options. |
| sweeps | <i>{saSweep}</i> | Read/Write | Set parametric analysis for details of <code>saSweep</code> object. |

wfDataFile

A wfDataFile represents a waveform data file on a disk. A wfDataFile can be open for read or write purpose, but not for both read and write via the same wfDataFile handle. This object can be also destroyed with `db::destroy` which closes the data file.

Table 54 *Attributes*

| Name | Type | Access | Description |
|--------------------|----------------------|-----------|--|
| filePath | string | Read-only | Waveform file path. |
| accessMode | <i>enWFileType</i> | Read-only | Waveform readOnly/write type. |
| format | <i>enWFileFormat</i> | Read-only | Waveform file format. |
| nSignals | int | Read-only | Number of signals. |
| nSweeps | int | Read-only | Number of sweeps. |
| variableNames | [string] | Read-only | List of variable names. |
| variableConditions | [string] | Read-only | List of variable conditions, one string for each sweep trace. |
| nSubfiles | int | Read-only | Number of sub-files. |
| signalNames | string | Read-only | The list of all signal names. |
| resultsType | string | Read-only | Results type of signal when it comes from PrimeWave Design Environment resultsMap.xml file. |
| analysisName | string | Read-only | Analysis name of signal when the signal comes from PrimeWave Design Environment resultsMap.xml file. |

xtJob

An xtJob stores information for a job launched by a Custom Compiler session. This information is used for display by the job monitor and for inter-process communication. The attributes of the object are described in the following table and can be retrieved and (in some cases) set by the `db::getAttr` and `db::setAttr` commands.

Table 55 *Attributes*

| Name | Type | Access | Description |
|------|--------|-----------|------------------|
| name | string | Read-only | Name of the job. |

Table 55 Attributes (Continued)

| Name | Type | Access | Description |
|---------------|-------------------|------------|---|
| type | string | Read-only | Job type, either “batch”, “foreign”, or an application name. |
| cmdLine | string | Read-only | Command used to launch the job. |
| runDesc | string | Read/Write | Descriptive text for the job. |
| files | [string] | Read-only | List of filenames to track. All files in this list will be deleted before the child process is launched. They are deleted to prevent confusion between old and new output data. |
| readOnlyFiles | [string] | Read-only | List of read-only files to track. These files will NOT be deleted before the child process is launched. This attribute is meant for use with input files for the child process, files that are generated before it is launched. |
| host | string | Read-only | Host on which to run the job. |
| data | string | Read/Write | Arbitrary string data associated with the job. |
| proclD | int | Read-only | UNIX pid of the process. |
| readyProc | string | Read-only | Procedure to call once the job is initialized and ready for input. Interactive jobs only. |
| status | <i>enXtStatus</i> | Read-only | Current job status. |
| exitProc | string | Read-only | Procedure to call once the job has exited. Will be called regardless of how the job shut down. |
| exitStatus | int | Read-only | Operating system issued exit status of the process. |
| startTime | string | Read-only | Time the job was started at. |
| exitTime | string | Read-only | Time the job exited at. |

Preferences

[Table 56](#) lists Custom Compiler db namespace general preferences.

Table 56 Database Preferences

| Name | Type | Scope | Default | Description |
|------------------------------|--------|-------|---------|--|
| dbAnalogLibAutoUpdatePortNum | bool | user | true | Control whether to upate port number automatically. |
| dbAnalogLibCccsVersion | string | user | | Control behavior of analogLib cccs device. |
| dbAnalogLibCcvsVersion | string | user | | Control behavior of analogLib ccvs device. |
| dbAnalogLibIndqVersion | string | user | | Control behavior of analogLib indq device. |
| dbAnalogLibIndVersion | string | user | | Control behavior of analogLib ind device. |
| dbAnalogLibIprbsVersion | string | user | | Control behavior of analogLib iprbs device. |
| dbAnalogLibIsinVersion | string | user | | Control behavior of analogLib isin device. |
| dbAnalogLibIsourceVersion | string | user | | Control behavior of analogLib isource device. |
| dbAnalogLibMTlineVersion | string | user | | Control behavior of analogLib mtline device. |
| dbAnalogLibNcsVersion | string | user | | Control behavior of analogLib ncs device. |
| dbAnalogLibNportVersion | string | user | | Control behavior of analogLib nport device. |
| dbAnalogLibNvsVersion | string | user | | Control behavior of analogLib nvs device. |
| dbAnalogLibNxPortVersion | string | user | | Control behavior of analogLib n1port ~ n4port devices. |
| dbAnalogLibPcapacitorVersion | string | user | | Control behavior of analogLib pcapacitor device. |
| dbAnalogLibPdiodeVersion | string | user | | Control behavior of analogLib pdiode device. |
| dbAnalogLibPhyresVersion | string | user | | Control behavior of analogLib phyres device. |
| dbAnalogLibPinductorVersion | string | user | | Control behavior of analogLib pinductor device. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-----------------------------------|--------|-------|---------|--|
| dbAnalogLibPortVersion | string | user | | Control behavior of analogLib port device. |
| dbAnalogLibPprbsVersion | string | user | | Control behavior of analogLib pprbs device. |
| dbAnalogLibPpwlVersion | string | user | | Control behavior of analogLib ppwl device. |
| dbAnalogLibPsinVersion | string | user | | Control behavior of analogLib psin device. |
| dbAnalogLibPulseExpV1V2Remap | bool | user | true | Control whether to remap v1/v2 parameters in pulse/exp source types in analogLib source devices. |
| dbAnalogLibSp1t2switchEmptyOffset | bool | user | true | Control whether to have empty value for offset parameter in sp1tswitch/sp2tswitch devices |
| dbAnalogLibSp1tswitchVersion | string | user | | Control behavior of analogLib sp1tswitch device. |
| dbAnalogLibSwitchVersion | string | user | | Control behavior of analogLib switch device. |
| dbAnalogLibTlineVersion | string | user | | Control behavior of analogLib tline device. |
| dbAnalogLibVccsVersion | string | user | | Control behavior of analogLib vccs device. |
| dbAnalogLibVcresVersion | string | user | | Control behavior of analogLib vcres device. |
| dbAnalogLibVcvsVersion | string | user | | Control behavior of analogLib vcvs device. |
| dbAnalogLibVprbsVersion | string | user | | Control behavior of analogLib vprbs device. |
| dbAnalogLibVsinVersion | string | user | | Control behavior of analogLib vsin device. |
| dbAnalogLibVsourceVersion | string | user | | Control behavior of analogLib vsource device. |
| dbAnalogLibWindingVersion | string | user | | Control behavior of analogLib winding device. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------------|--------|----------|----------------------|--|
| dbAutoReduceLicenseTier | bool | user | false | If set, causes the tool to change the current license tier to Read-Only when all windows are closed other than <code>giConsole</code> , <code>dmLibraryManager</code> , and <code>giHomePage</code> .
Valid values: true, false |
| dbBindExternalNetlistViewOverwrite | bool | cellview | true | Indicates if external netlist view should be overwritten.
Valid values: true, false |
| dbCreateICCLibAttachTech | string | user | | Library name whose tech is used by the created ICC library in the Custom Compiler environment. |
| dbCreateICCLibDirectory | string | user | | Directory in which IC Compiler library is created. |
| dbCreateICCLibLayerMap | string | user | | Path to the layer mapping file which is used to synchronize with the attached technology. |
| dbCreateICCLibRefLibs | string | user | | List of the reference libraries for the created IC Compiler library. |
| dbCreateICCLibSeparateOATech | bool | user | false | Boolean switch indicating that the created the IC Compiler library uses a technology of the specified library in the Custom Compiler environment. |
| dbCreateICCLibTechFile | string | user | | Path to the IC Compiler technology file. |
| dbCreateICCLibTechSource | string | user | Export OA Technology | Source of the IC Compiler library's technology. |
| dbCreateICCLibViaMap | string | user | | Path to the via mapping file which is used to synchronize with the attached technology. |
| dbCreateImageDialogMode | bool | user | true | <code>idb::createImage</code> command options dialog eject state.
Valid values: true, false |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-------------------------------------|-------------|----------|---------|--|
| dbDataIoLayerMapLog | string | user | | Output layer mapping table to the log file for debugging. |
| dbDateDisplayFormat | string | mm/dd/yy | true | Controls the display format of date. The full list of time abbreviation. |
| dbDefVersion | string | user | 5.8 | Version for exported LEF files. |
| dbDisplayEvaluatedParams | bool | user | true | Display full evaluated PEL/SEL expressions for parameters on canvas, in the Create Instance form and in the PE.
Valid values: true, false |
| dbEDIFImportCustomSchematicDBUPerUU | int | user | 160 | Specifies the custom value for schematic views DB unit per user unit scale. |
| dbEDIFImportNetDisplayExtraProcess | bool | user | false | Controls net labels to parent wires linking flow during EDIF import. If enabled, more accurate but slow processing is done.
Valid values: true, false |
| dbEDIFImportSchematicDBUPerUUSource | dbScaleType | user | default | Specifies the source for DB unit per user unit scale for schematic views when creating new libs during EDIF Import. |
| dbEnablePcellRepository | bool | user | false | Enables PCell repository.
Valid values: true, false |
| dbEnableTableBasedParamDefEditor | bool | user | true | Uses table based Parameter Definition Editor dialog box.
Valid values: true, false |
| dbExportBlockageType | string | user | | Specifies the object mapping file name used for exportOasis translation. |
| dbExportDefCellName | string | user | | Cell name of design to export to DEF. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-------------------------------|--------|-------|-------------|---|
| dbExportDefDBUOverride | string | user | | DBU override during DEF export. |
| dbExportDefDividerChar | string | user | <file path> | Divider char to express hierarchy during DEF export. |
| dbExportDefExpandArrayInsts | bool | user | false | Specifies if DEF should expand Arrays to Instances. |
| dbExportDefFileName | string | user | | Export DEF filename. |
| dbExportDefFilterPurposes | string | user | | Filters out shapes on specified purposes. |
| dbExportDefLibName | string | user | | Library name of the design to export to DEF. |
| dbExportDefMapDividerChar | string | user | ! | Specifies a map divider character to display hierarchy during DEF export. |
| dbExportDefRunDirectory | string | user | . | The run directory for exportDEF binary. |
| dbExportDefScaleFactor | string | user | | Resizes the design with a scale factor. |
| dbExportDefSuppressInfos | string | user | | List of info message ids that should be suppressed during DEF export. |
| dbExportDefSuppressWarnings | string | user | | List of warning message ids that should be suppressed during DEF export. |
| dbExportDefVersion | string | user | 5.8 | The DEF file version to create. |
| dbExportDefViewName | string | user | layout | View name of design to export to DEF. |
| dbExportLefCellBrowserLibrary | string | user | | Library to select cells to export to LEF. |
| dbExportLefCells | string | user | | The space separated list of cell names to be outputted to LEF file. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---|--------|-------|----------|---|
| dbExportLefDBUOverride | string | user | | DBU override during LEF export. |
| dbExportLefExcludeCells | string | user | | Cells from Library to ignore during export to LEF. |
| dbExportLefFileName | string | user | | The LEF file to be created during export LEF. |
| dbExportLefIncludeCells | string | user | | List of Cells from Library to export. |
| dbExportLefLibName | string | user | | The design library name used in export LEF. |
| dbExportLefRunDirectory | string | user | . | The run directory for export LEF binary. |
| dbExportLefScaleFactor | string | user | | Resizes cell with a scale factor. |
| dbExportLefSuppressInfos | string | user | | List of info message ids that during export LEF. |
| dbExportLefSuppressWarnings | string | user | | List of warning message ids that should be suppressed during LEF importing. |
| dbExportLefVersion | string | user | 5.8 | LEF syntax version. |
| dbExportLefViews | string | user | abstract | The output views names for LEF exporter. |
| dbExportLefWriteTechnology | bool | user | false | Specifies if technology information should be written to LEF file.
Valid values: true, false |
| dbExportMCellCDFFileName | string | user | | File name for exporting CDF parameters. |
| dbExportMCellFileName | string | user | | File name for exporting MCell superMasters. |
| dbExportMilkywayCEL2FRAM | bool | user | false | Remasters instances placing CEL views to use FRAM views, if available, in the Export IC Compiler. |
| dbExportMilkywayCreateHierarchyPreservation | bool | user | true | Create Hierarchy Preservation.
Valid values: true, false |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-------------------------------------|--------|-------|-------------|--|
| dbExportMilkywayCreateUPFSupplySets | bool | user | false | Implicitly creates UPF supply sets when loading a UPF file in the IC Compiler tool.
Valid values: true, false |
| dbExportMilkywayDefaultLibPath | string | user | <file path> | Default path in which to create reference libraries for Export IC Compiler. |
| dbExportMilkywayExcludeLibs | string | user | | List of libraries to exclude for IC Compiler export. |
| dbExportMilkywayHierDepth | int | user | 20 | Hierarchy depth for Export IC Compiler. |
| dbExportMilkywayInheritRefLibs | bool | user | false | If true, newly created reference libraries inherit the reference libraries from the main library.
Valid values: true, false |
| dbExportMilkywayMapMode | string | user | Merge | Merges or replaces cached map files by user map files. |
| dbExportMilkywayOpenOnCompletion | bool | user | false | Opens the translated design after completion.
Valid values: true, false |
| dbExportMilkywayOutputLibrary | string | user | | Target library for Export to the IC Compiler design. |
| dbExportMilkywayOverwriteExisting | bool | user | false | Overwrites existing cellviews in Export IC Compiler.
Valid values: true, false |
| dbExportMilkywayRunICCLCV | lcv | user | <file path> | OA design to export. |
| dbExportMilkywaySourceCells | string | user | | Source cells for Export IC Compiler. |
| dbExportMilkywaySourceLibrary | string | user | | Source library for Import IC Compiler. |
| dbExportMilkywaySourceViews | string | user | | Source views for Export IC Compiler. |
| dbExportMilkywayTechFile | string | user | | Technology file for Export Milkyway. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------------|------------------------------|-------|---------------------------|--|
| dbExportMilkywayTechSource | string | user | Open Access | Technology to export for Export Milkyway. |
| dbExportMilkywayUpdateExisting | string | user | mixed | Update existing cell Views in Export Milkyway. |
| dbExportMilkywayUPFFile | string | user | | UPF file for Export to IC Compiler design. |
| dbExportMilkywayUPFMode | string | user | No UPF | UPF mode for Export to IC Compiler design. |
| dbExportMilkywayWarnNetlistChanges | bool | user | false | Check for netlist changes. Valid values: true, false |
| dbExportNetlistCheckOpenDesigns | enCheckOpenDesignsOrNoneMode | user | none | Controls whether to show the Check and Save prompt and the set of designs involved to it, before netlisting from the Export Netlist dialog box. Valid values: none, all, topDesign, hierDesign |
| dbExportNetlistCreateTopAsModule | bool | user | true | Default value of create top as module option to use in the Export Netlist dialog box. Valid values: true, false |
| dbExportNetlistDir | string | user | current working directory | Default directory of netlist file to use in the Export Netlist dialog box. |
| dbExportNetlistFile | string | user | | Default value of netlist file to use in the Export Netlist dialog box. |
| dbExportNetlistFileNameFormat | string | user | %LIB_%CELL.%EXT | Format to compute the default netlist file name. Uses patterns: <ul style="list-style-type: none"> • %LIB: Top design library name • %CELL: Cell name • %VIEW: View name • %NL: Netlister name • %EXT: Netlister's default file extension • %: Single char % |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------------|--------|-------|--|---|
| dbExportNetlistFlattenDesign | bool | user | false | Generates a flattened netlist.
Valid values: true, false |
| dbExportNetlistFlattenRename | bool | user | true | When true, renames nets and instances in the flattened netlists.
Valid values: true, false |
| dbExportNetlistFlattenSaveCellView | bool | user | false | When true, saves the intermediate flattened cellview.
Valid values: true, false |
| dbExportNetlistFormat | string | user | CDL | Default value of netlister format to use in the Export Netlist dialog box. |
| dbExportNetlistHiddenOptionsList | string | user | maxLineLength
indentDepth
sortInstHeaders
moduleOrder
sortInsts
connection
TypeCDL
copyNetlist
FileContent | List of Netlister Options fields to hide. |
| dbExportNetlistICCVerilog | bool | user | false | Netlist ICC Verilog.
Valid values: true, false |
| dbExportNetlistIndentDepth | int | user | 4 | Default value of indent depth to use in the Export Netlist dialog box. |
| dbExportNetlistLCV | lcv | user | // | Default value of LCV to use in the Export Netlist dialog box. |
| dbExportNetlistMaxLength | int | user | 80 | Default value of max line length to use in the Export Netlist dialog box. |
| dbExportNetlistModuleOrder | string | user | leafFirst | Default value of module ordering to use in the Export Netlist dialog box. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------------|--------|-------|--------------------|--|
| dbExportNetlistSortInstanceHeaders | string | user | none | Default value of sorting instance headers to use in the Export Netlist dialog box. |
| dbExportNetlistSortInstances | string | user | none | Default value of sorting instances to use in the Export Netlist dialog box. |
| dbExportNetlistViewSearchList | string | user | auCdl
schematic | Default value of view search list to use in the Export Netlist dialog box. |
| dbExportNetlistViewStopList | string | user | auCdl | Default value of view stop list to use in the Export Netlist dialog box. |
| dbExportNetlistWriteNameMapping | bool | user | false | Saves the name mapping data to the target directory.
Valid values: true, false |
| dbExportNetlistWriteUPF | bool | user | true | Used in the export netlist dialog to control if UPF is written as a side file.
Valid values: true, false |
| dbExportOasisAcutePathAsPolygon | bool | user | false | Specifies if Export Oasis Translator should convert acute paths to polygon and output them as boundary records.
Valid values: true, false |
| dbExportOasisApplyRepetition | bool | user | false | Applies repetition for instance/shapes/text.
Valid values: true, false |
| dbExportOasisAutoBackupFile | bool | user | true | Specifies whether to auto back up the OASIS file when it exists.
Valid values: true, false |
| dbExportOasisAutoFillFileName | bool | user | true | Specifies if auto fills OASIS file name with the top cell name.
Valid values: true, false |
| dbExportOasisAutoMapObjects | bool | user | true | Use layer map entries to write prBoundaries, snapBoundaries, and layer/area Blockages.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------------|--------|-------|---------|--|
| dbExportOasisAutoMapObjectsPropNum | int | user | 99 | The property number used to record objects type when the <code>-autoMapObjects</code> option is specified. |
| dbExportOasisBatchMode | bool | user | true | If true, runs exportOasis with batch mode.
Valid values: true, false |
| dbExportOasisBlockages | bool | user | true | Specifies if blockage objects should be exported.
Valid values: true, false |
| dbExportOasisBlockageType | string | user | | The purpose number for outputted blockages that is used during OASIS translation. |
| dbExportOasisCellMapFile | string | user | | Cell mapping file name for exportOasis translation. |
| dbExportOasisCellNamePrefix | string | user | | A string to prepend to every structure name written in the Oasis file. |
| dbExportOasisCellNameSuffix | string | user | | A string to append to every structure name written in the Oasis file. |
| dbExportOasisCheckOutOfDate | bool | user | false | Checks if read-only design is modified on the disk since last open time for non batch mode.
Valid values: true, false |
| dbExportOasisCheckSave | bool | user | false | Specifies the need to pop-up the Design Save dialog box.
Valid values: true, false |
| dbExportOasisColorCellMapFile | string | user | | Specifies the cell mapping file for color in exportOasis.. |
| dbExportOasisColorMapping | string | user | Honor | Specifies color mapping type. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------------|--------|-------|--------------------|--|
| dbExportOasisCompressionLevel | string | user | | Specifies the compression level to use. 0 indicates no compression and 9 indicates the highest compression. |
| dbExportOasisConvertLabels | string | user | None | How to force the case on labels. |
| dbExportOasisConvertNames | string | user | None | Specifies if translator should convert all cell and instance names to uppercase or to lowercase or just leave it as it is. |
| dbExportOasisConvertPaths | bool | user | false | Specifies if exportOasis converts paths to polygons.
Valid values: true, false |
| dbExportOasisDbuPerUU | string | user | | Database unit per user unit to be used in exportOasis. |
| dbExportOasisDepth | int | user | 256 | Translates the hierarchical depth during oa2oasis translation. |
| dbExportOasisDisplayCompletion | bool | user | false | If true, displays information after export.
Valid values: true, false |
| dbExportOasisDonutNumSides | int | user | 64 | Specifies the number of edges used for donut to polygon conversion during exportOasis translation. |
| dbExportOasisDumpDesignTree | bool | user | false | Dumps the design tree to the log file.
Valid values: true, false |
| dbExportOasisDumpRefLibInLog | bool | user | false | Dumps reference library list in the log file.
Valid values: true, false |
| dbExportOasisDuplicateCellHandling | string | user | Append with Number | Specifies how to handle duplicate cells. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---|--------|-------|---------|---|
| dbExportOasisEllipseAsCircle | bool | user | false | Specifies if the oa2oasis translator should convert ellipses to circles and export them as circle records.
Valid values: true, false |
| dbExportOasisEllipseNumSides | int | user | 64 | Specifies the number of edges used for ellipse to polygon conversion during exportOasis translation. |
| dbExportOasisExcludeRefLibs | string | user | | Ignores the list of libraries during importing Oasis. |
| dbExportOasisExpandArrayInsts | bool | user | false | Specifies if Oasis Out Translator should expand Arrays to Instances.
Valid values: true, false |
| dbExportOasisExternalizeShapes | bool | user | true | Specifies if exportOasis adds shape recognition colors.
Valid values: true, false |
| dbExportOasisFileName | string | user | | Output OASIS file name. |
| dbExportOasisFilterAcutePaths | bool | user | false | Specifies if the translator filters out acute paths.
Valid values: true, false |
| dbExportOasisFilterDots | bool | user | false | Specifies if the translator filters out dots.
Valid values: true, false |
| dbExportOasisFilterEmptyCells | bool | user | false | Specifies if the translator filters out empty cells.
Valid values: true, false |
| dbExportOasisFilterMergeCellviewsSuper Master | bool | user | false | Specifies if the super master is filtered out from merged cellviews.
Valid values: true, false |
| dbExportOasisFilterReentrantPgns | bool | user | false | Specifies if the translator filters out reentrant polygons and paths.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------|-------|---------|---|
| dbExportOasisFilterSelfIntersectingPgns | bool | user | false | Specifies if the translator filters out self-intersecting polygons and paths.
Valid values: true, false |
| dbExportOasisFilterText | bool | user | false | Specifies if the translator filters out the text.
Valid values: true, false |
| dbExportOasisFilterUnboundInsts | bool | user | false | Specifies if the translator filters out unbound instances.
Valid values: true, false |
| dbExportOasisFilterZeroWidth | bool | user | false | Specifies if the translator filters out zero width paths.
Valid values: true, false |
| dbExportOasisFlattenPcells | bool | user | false | Specifies if Export Oasis Translator should flatten any PCell into their geometry.
Valid values: true, false |
| dbExportOasisFlattenVias | bool | user | false | Specifies if Export Oasis Translator should flatten standard vias into their geometry.
Valid values: true, false |
| dbExportOasisFontMapFile | string | user | | Font mapping file name for Export Oasis exporting. |
| dbExportOasisHierDepth | int | user | 20 | Specifies the hierarchy depth used during exportOasis translation. |
| dbExportOasisIgnoreCellsAffix | string | user | | Specifies if the translator should ignore adding affix to every structure name. |
| dbExportOasisIgnoreDuplicatedMappings | bool | user | false | Ignores duplicated mappings for Term in the object mapping file.
Valid values: true, false |
| dbExportOasisIgnoreTopCellsAffix | string | user | No | Specifies if the translator should ignore adding affix to the top structure name. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-----------------------------------|--------|-------|---------|--|
| dbExportOasisIgnoreUnmappedLayers | bool | user | true | Ignores layers if not mapped any lines in layer mapping file.
Valid values: true, false |
| dbExportOasisIncludeRefLibs | string | user | | Specifies the reference library during importing Oasis. |
| dbExportOasisKeepAbutments | bool | user | true | Specifies if export Oasis translator should abutment information into .abut file.
Valid values: true, false |
| dbExportOasisKeepFullCellName | bool | user | false | Specifies if exportOasis translator should keep cell submaster (and stdVia) name.
Valid values: true, false |
| dbExportOasisKeepInstName | bool | user | false | The option of property of instance name for OASIS exporting.
Valid values: true, false |
| dbExportOasisKeepLogicName | bool | user | false | The option of property of logic info for OASIS exporting.
Valid values: true, false |
| dbExportOasisKeepNetName | bool | user | false | The option of property of net name for OASIS exporting.
Valid values: true, false |
| dbExportOasisKeepPcells | bool | user | true | Specifies if translator should handle PCell. |
| dbExportOasisKeepVias | bool | user | true | Specifies if vias information is kept to .pcell file. |
| dbExportOasisKeepWideAlignment | bool | user | false | The option of property of alignment boundaries for OASIS exporting.
Valid values: true, false |
| dbExportOasisLabelDepth | int | user | 1 | The depth to which to create labels. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|----------------------------------|--------|-------|---------|---|
| dbExportOasisLabelMapFile | string | user | | Specifies label mapping file name for Export Oasis Translation. |
| dbExportOasisLayerMapFile | string | user | | Specifies the Layer mapping file name for exportOasis translation. |
| dbExportOasisLayerMappingMode | string | user | tech | Specifies the layer mapping mode. |
| dbExportOasisLibName | string | user | | Library name that contains the design that should be exported to OASIS format. |
| dbExportOasisLimitChars | bool | user | false | Specifies if Oasis Out Translator conform to the 32 characters limit on structure names.
Valid values: true, false |
| dbExportOasisLimitLogFiles | string | user | 5 | Limits the maximum number of the backup log files. |
| dbExportOasisLogFileName | string | user | | Specifies a log file. |
| dbExportOasisMapAllTexts | bool | user | false | Specifies if Export Oasis Translator should attempt to name map oaTexts and oaTextDisplays that are not oaNames. Corresponds to <code>mapAllTexts</code> in the <code>exportOasis</code> binary.
Valid values: true, false |
| dbExportOasisMapColorsUsingLocks | bool | user | false | Specifies whether to map shapes with a mask color attribute. Corresponds to <code>mapColorLock</code> in the <code>exportOasis</code> binary.
Valid values: true, false |
| dbExportOasisMaxVertices | string | user | 8190 | Specifies if limit the maximum vertices for path and polygon. |
| dbExportOasisMergeCellviews | string | user | | The cellview merged to the top cell during exportOasis translation. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-------------------------------------|--------|-------|---------|---|
| dbExportOasisNameSpace | string | user | cdba | Specify the namespace mapping for text displays. |
| dbExportOasisNewMaxVerticesFunction | bool | user | true | Specifies if using new function for the maximum vertices.
Valid values: true, false |
| dbExportOasisOasisVer | string | user | 3 | Specifies a version of the oasis format. |
| dbExportOasisObjectMapFile | string | user | | Object mapping file name for exportOasis translation when explicitly specified. |
| dbExportOasisObjectMappingMode | string | user | tech | Specifies the object mapping mode. |
| dbExportOasisOutputBoundaries | bool | user | true | If set to true, this preference exports boundary objects.
Valid values: true, false |
| dbExportOasisPathAsPolygon | bool | user | false | Specifies if oa2oasis translator should convert paths to polygons and output them as boundary records.
Valid values: true, false |
| dbExportOasisPathKeepHalfWithPaths | bool | user | false | Specifies if oa2oasis translator keeps half width paths.
Valid values: true, false |
| dbExportOasisPcellOptions | string | user | Keep | Specifies if the translator should handle PCells.
Valid values: None, Flatten, Keep |
| dbExportOasisPropMapFile | string | user | | Property mapping file name for Export Oasis exporting. |
| dbExportOasisPropNameForInstName | string | user | 102 | The property name of inst's name for OASIS exporting. |
| dbExportOasisPropNameForLogicName | string | user | 103 | The property name of logic info for OASIS exporting. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|--------|-------|---------|--|
| dbExportOasisPropNameForNetName | string | user | 102 | The property name of net's name for OASIS exporting. |
| dbExportOasisPropNumForVia | string | user | | The property number of via for OASIS exporting(for third-party tools to restore via) |
| dbExportOasisPropNumForWideAlignm
ent | string | user | 105 | The property number of alignment boundaries for OASIS exporting. |
| dbExportOasisPropValuesOnly | bool | user | true | Specifies if the translator should only output property values. |
| dbExportOasisReferenceMapFile | string | user | | Specifies whether to export the reference xml file.
Valid values: true, false |
| dbExportOasisRefLibCells | bool | user | true | Specifies if translator should output cells referenced from a library other than the one specified by the library argument.
Valid values: true, false |
| dbExportOasisRefLibFile | string | user | | Specifies the file containing names of reference libraries. |
| dbExportOasisRefLibNames | string | user | | Specifies the list of libraries with cells referenced in the design. |
| dbExportOasisRemasterViewName | string | user | | Specifies if the translator should remaster the view name to every cell. |
| dbExportOasisRoundPathAsPolygon | bool | user | false | Specifies if Export Oasis Translator should convert rounded extension paths to polygon and output them as boundary records.
Valid values: true, false |
| dbExportOasisRunDirectory | string | user | | The run directory for exportOasis binary. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------|-------|---------|--|
| dbExportOasisShowLockOnly | bool | user | false | Shows the Lock Only option for color mapping type in the Export OASIS dialog box.
If Lock Only is selected, any shape of colored layer, excluding text and text display, is filtered out in exportOasis if it has correspondent mapping line with lock attribute and it has no color or it is not locked.
Valid values: true, false |
| dbExportOasisSnapToMfgGrid | bool | user | false | Specifies if vertices snap to manufacturing grid resolution.
Valid values: true, false |
| dbExportOasisSuppressInfos | string | user | | List of information message ids to be suppressed during exportOasis exporting. |
| dbExportOasisSuppressUnmappedLayer Warnings | string | user | | List of LPPs for unmapped layer warning message that should be suppressed during Oasis exporting. |
| dbExportOasisSuppressWarnings | string | user | | List of warning message ids to be suppressed during Oasis exporting. |
| dbExportOasisSystemPurposeToDrawing | bool | user | false | Specifies if exportOasis maps drawing purpose when system purpose is not mapped in layer mapping.
Valid values: true, false |
| dbExportOasisTclFile | string | user | | Specifies a Tcl file. |
| dbExportOasisTclFunction | string | user | | Specifies a Tcl function. |
| dbExportOasisTextBrackets | string | user | None | Specifies if converts text brackets. |
| dbExportOasisTextDisplays | bool | user | true | Specifies if oa2oasis translator should translate text display objects.
Valid values: true, false |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--|--------|-------|---------|--|
| dbExportOasisTextMapFile | string | user | | Specifies the text mapping file name for OASIS exporting. |
| dbExportOasisTopCell | string | user | | The top level cell that should be exported during OASIS writing. |
| dbExportOasisTopCellName | string | user | | Specifies the top level cell that should be exported during OASIS exporting. |
| dbExportOasisUnmappedLayersUseOALayerNumbers | bool | user | false | Specifies whether to use Open Access layer numbers if not mapped any lines in layer mapping file.
Valid values: true, false |
| dbExportOasisUseTechLayerMap | bool | user | false | Specifies whether to use the layer map file that is in the source technology library. This is only enabled when the file exists.
Valid values: true, false |
| dbExportOasisUseTechObjectMap | bool | user | true | Specifies whether to use the object map file that is in the source technology library. This is only enabled when the file exists.
Valid values: true, false |
| dbExportOasisValidationScheme | string | user | CRC32 | Specifies the validation scheme to use.
Valid values: None, CRC32, CHECKSUM32 |
| dbExportOasisViewName | string | user | layout | The view that should be exported during OASIS out translation. |
| dbExportStreamAcutePathAsPolygon | bool | user | false | Specifies whether or not the exportStream translator should convert acute paths to polygons and output them as boundary records.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------------------|--------|-------|---------|--|
| dbExportStreamArrayToInst | bool | user | false | Specifies if Stream Out translator should convert arrays to instances.
Valid values: true, false |
| dbExportStreamAutoBackupFile | bool | user | true | Specifies whether to auto back up the GDS file when it exists.
Valid values: true, false |
| dbExportStreamAutoFillFileName | bool | user | true | Specifies if auto fills STREAM file name with top cell name.
Valid values: true, false |
| dbExportStreamAutoMapObjects | bool | user | true | Uses layer map entries to write prBoundaries, snapBoundaries, and layer/area Blockages.
Valid values: true, false |
| dbExportStreamAutoMapObjectsProp Num | int | user | 99 | The property number is used to record objects type when using the <code>-autoMapObjects</code> argument. |
| dbExportStreamBatchMode | bool | user | true | Specifies if run <code>exportStream</code> with batch mode.
Valid values: true, false |
| dbExportStreamBlockages | bool | user | true | Specifies if blockage objects should be exported.
Valid values: true, false |
| dbExportStreamBlockageType | string | user | 0 | The datatype number for blockages used during <code>exportStream</code> translation. |
| dbExportStreamCellMapFile | string | user | | Cell mapping file name for <code>exportStream</code> translation. |
| dbExportStreamCellNamePrefix | string | user | | A string to prepend to every structure name written in the stream file. |
| dbExportStreamCellNameSuffix | string | user | | A string to append to every structure name written in the stream file. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------------|--------|-------|---------|--|
| dbExportStreamCheckOutOfDate | bool | user | false | Checks if read-only design is modified on the disk since last open time for non batch mode.
Valid values: true, false |
| dbExportStreamCheckSave | bool | user | false | Checks if the Save Design dialog box should open for designs that are not saved.
Valid values: true, false |
| dbExportStreamColorCellMapFile | string | user | | Specifies the cell mapping file for color in exportStream. |
| dbExportStreamColorMapping | string | user | Honor | Specifies the color mapping type. |
| dbExportStreamConvertLabels | string | user | None | How to force the case on labels. |
| dbExportStreamConvertNames | string | user | None | Specifies if translator should convert all cell and instance names to uppercase or to lowercase or just leave as it is. |
| dbExportStreamDbuPerUU | int | user | | Database unit per user unit to be used in exportStream. |
| dbExportStreamDepth | int | user | 256 | The hierarchical depth to translate to during oa2stream translation. |
| dbExportStreamDisplayCompletion | bool | user | false | Specifies whether to display information after Export.
Valid values: true, false |
| dbExportStreamDonutNumSides | int | user | 64 | Specifies the number of edges that should be used for donut to polygon translation during oa2stream translation. |
| dbExportStreamDumpDesignTree | bool | user | false | Dump design tree to log file
Valid values: true, false |
| dbExportStreamDumpRefLibInLog | bool | user | false | Dumps reference library list in the log file. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------|-------|--------------------|--|
| dbExportStreamDuplicateCellHandling | string | user | Append with Number | Specifies how to handle duplicate cells. |
| dbExportStreamEllipseNumSides | int | user | 64 | Specifies the number of edges that should be used for ellipse to polygon translation. |
| dbExportStreamExcludeRefLibs | string | user | | List of libraries to ignore during import Stream. |
| dbExportStreamExpandArrayInsts | bool | user | false | Specifies if Stream Out translator should expand Arrays to Instances.
Valid values: true, false |
| dbExportStreamExternalizeShapes | bool | user | true | Specifies if translator externalize the shapes.
Valid values: true, false |
| dbExportStreamFileName | string | user | | Output stream file name. |
| dbExportStreamFilterAcutePaths | bool | user | false | Specifies if translator filter out acute paths.
Valid values: true, false |
| dbExportStreamFilterDots | bool | user | false | Specifies if translator filter out dots.
Valid values: true, false |
| dbExportStreamFilterEmptyCells | bool | user | false | Specifies if translator filter out empty cells.
Valid values: true, false |
| dbExportStreamFilterMergeCellviewsSuperMaster | bool | user | false | Filters out the super master from merged cellviews.
Valid values: true, false |
| dbExportStreamFilterReentrantPgns | bool | user | false | Specifies if translator filter out reentrant polygons.
Valid values: true, false |
| dbExportStreamFilterSelfIntersectingPgns | bool | user | false | Specifies if translator filter out self-intersecting polygons.
Valid values: true, false |
| dbExportStreamFilterText | bool | user | false | Specifies if translator filter out text.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|--------|-------|---------|---|
| dbExportStreamFilterUnboundInsts | bool | user | false | Specifies if translator filter out unbound instances.
Valid values: true, false |
| dbExportStreamFilterZeroWidth | bool | user | false | Specifies if translator filter out zero width paths.
Valid values: true, false |
| dbExportStreamFlattenPcells | string | user | false | Specifies if exportStream translator should convert PCells into their geometry.
Valid values: true, false |
| dbExportStreamFlattenVias | bool | user | false | Specifies if exportStream translator should flatten standard vias into their geometry.
Valid values: true, false |
| dbExportStreamFontMapFile | string | user | | Font mapping file name for the exportStream translation. |
| dbExportStreamIgnoreCellsAffix | string | user | | Specifies if translator should ignore to add affix to every structure name. |
| dbExportStreamIgnoreDuplicatedMappings | bool | user | false | Ignores duplicated mappings for Term in the object mapping file.
Valid values: true, false |
| dbExportStreamIgnoreTopCellsAffix | string | user | No | Specifies if the translator should ignore adding affix to the top structure name. |
| dbExportStreamIgnoreUnmappedLayers | bool | user | true | Ignore layers if not mapped any lines in layer mapping file.
Valid values: true, false |
| dbExportStreamIncludeRefLibs | string | user | | Library(s) to use as reference library during import Stream |
| dbExportStreamKeepAbutments | bool | user | true | Specifies if export Stream translator should abutment information into .abut file. |
| dbExportStreamKeepFullCellName | bool | user | false | Specifies if Export Stream translator should keep cell sub-master (and stdVia) name. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---------------------------------------|--------|-------|---------|--|
| dbExportStreamKeepInstName | bool | user | false | The option of property of inst's name for GDSII exporting.
Valid values: true, false |
| dbExportStreamKeepLogicName | bool | user | false | The option of property of logic's info for GDSII exporting.
Valid values: true, false |
| dbExportStreamKeepNetName | bool | user | false | The option of property of net's name for GDSII exporting.
Valid values: true, false |
| dbExportStreamKeepVias | bool | user | true | Specifies if vias information are kept to .pcell file
Valid values: true, false |
| dbExportStreamKeepViewName | bool | user | false | Specifies if exportStream translator should keep view name.
Valid values: true, false |
| dbExportStreamKeepWideAlignment | bool | user | false | The option of property of alignment boundaries for GDSII exporting.
Valid values: true, false |
| dbExportStreamLabelDepth | int | user | 1 | The depth to which to create labels. |
| dbExportStreamLabelMapFile | string | user | | Label mapping file name for exportStream translation. |
| dbExportStreamLabelMapWithMultiPinFig | bool | user | false | Label mapping file name for Export Stream.
Valid values: true, false |
| dbExportStreamLayerMapFile | string | user | | Layer mapping file name for GDSII exporting. |
| dbExportStreamLayerMappingMode | string | user | tech | Specifies the layer mapping mode. |
| dbExportStreamLibName | string | user | | Library name that contains the design that should be exported to GDSII format. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------------------|--------|-------|---------|---|
| dbExportStreamLimitChars | bool | user | false | Specifies if Stream Out translator conform to the 32 character limit on structure names.
Valid values: true, false |
| dbExportStreamLimitLogFiles | string | user | 5 | Specifies whether to limit the maximum number of log files. |
| dbExportStreamLogFileName | string | user | | Specifies log file. |
| dbExportStreamMapAllTexts | bool | user | false | Specifies if exportStream translator should attempt to map oaTexts and oaTextDisplays that are not oaNames. Corresponds to mapAllTexts in exportStream binary.
Valid values: true, false |
| dbExportStreamMapColorsUsingLocks | bool | user | false | Specifies which mapping lines in the layer map file are used to map shapes with a mask color attribute. Corresponds to mapColorLock in exportStream binary.
Valid values: true, false |
| dbExportStreamMaxVertices | string | user | 8190 | Specifies if limit the maximum vertices for path and polygon. |
| dbExportStreamMergeCellviews | string | user | | The cellviews to be merged to the top cell during GDSII Out. |
| dbExportStreamNameSpace | string | user | cdba | The namespace mapping for textDisplays. |
| dbExportStreamNetFromPinToText | bool | user | false | Specifies if the exportStream translator should convert pin's net name to texts .
Valid values: true, false |
| dbExportStreamNewMaxVerticesFunction | bool | user | true | Specifies a new function for the maximum vertices.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-------------------------------------|--------|-------|---------|--|
| dbExportStreamNewMaxVerticesRule | string | user | SwapXY | Specifies a rule for the maximum vertices.
Valid values: true, false |
| dbExportStreamObjectMapFile | string | user | | Object mapping file name for exportStream translation when explicitly specified. |
| dbExportStreamObjectMappingMode | string | user | tech | Specifies the object mapping mode. |
| dbExportStreamOutputBBoxInfo | bool | user | false | Specifies if export BBox information for cells in the log file.
Valid values: true, false |
| dbExportStreamOutputBoundaries | bool | user | true | Specifies if boundary objects should be exported.
Valid values: true, false |
| dbExportStreamPathAsPolygon | bool | user | false | Specifies if the oa2stream translator should convert paths to polygons and export them as boundary records.
Valid values: true, false |
| dbExportStreamPathKeepHalfWithPaths | bool | user | false | Specifies if oa2stream translator keeps half width paths.
Valid values: true, false |
| dbExportStreamPcellOptions | string | user | Keep | Specifies if the translator should handle PCell.
Valid Values: None, Flatten, Keep |
| dbExportStreamPinToText | bool | user | false | Specifies if exportStream translator should convert pin's term name to text.
Valid values: true, false |
| dbExportStreamPropMapFile | string | user | | Property mapping file name for GDSII exporting. |
| dbExportStreamPropNumForInstName | string | user | 102 | The property number of inst's name for GDSII exporting. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------|-------|---------|--|
| dbExportStreamPropNumForLogicName | string | user | 103 | The property number of logic's info for GDSII exporting. |
| dbExportStreamPropNumForNetName | string | user | 102 | The property number of net's name for GDSII exporting. |
| dbExportStreamPropNumForVia | string | user | | The property number of via for GDSII exporting(for third-party tools to restore via) |
| dbExportStreamPropNumForWideAlignm
ent | string | user | 105 | The property number of alignment boundaries for GDSII exporting. |
| dbExportStreamPropValuesOnly | bool | user | true | Specifies if the translator should output property values only.
Valid values: true, false |
| dbExportStreamReferenceMapFile | string | user | | The property mapping file for GDSII exporting. |
| dbExportStreamRefLibCells | bool | user | true | Specifies if the translator should output cells referenced from a library other than the one specified by the Library argument.
Valid values: true, false |
| dbExportStreamRefLibFile | string | user | | File containing names of reference libraries. |
| dbExportStreamRefLibNames | string | user | | List of libraries with cells referenced in the design. |
| dbExportStreamRemasterViewName | string | user | | Specifies if the translator should remaster the view name to every cell. |
| dbExportStreamRoundPathAsPolygon | bool | user | true | Converts rounded extension paths to polygon and output them as boundary records.
Valid values: true, false |
| dbExportStreamRouteObjects | bool | user | true | Specifies if routes should not be translated.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------------|-------|---------|--|
| dbExportStreamRunDirectory | string | user | . | The run directory for oa2stream binary. |
| dbExportStreamShowLockOnly | bool | user | false | Shows the Lock Only option for color mapping type in the Export Stream dialog box.
If Lock Only is selected, any shape of colored layer, excluding text and text display, is filtered out in exportStream if it has correspondent mapping line with lock attribute and it has no color or it is not locked.
Valid values: true, false |
| dbExportStreamSnapToMfgGrid | bool | user | false | Specifies if vertices snap to manufacturing grid resolution.
Valid values: true, false |
| dbExportStreamSortLayer | bool | user | false | Specifies if sorting layer statistics in the log file.
Valid values: true, false |
| dbExportStreamStreamVer | string | user | 3 | Specifies a version of the stream format. |
| dbExportStreamSuppressInfos | string | user | | List of info message ids that should be suppressed during GDSII exporting. |
| dbExportStreamSuppressUnmappedLayerWarnings | string | user | | List of LPPs for unmapped layer warning message that should be suppressed during GDSII exporting. |
| dbExportStreamSuppressWarnings | string | user | | List of warning message ids that should be suppressed during GDSII exporting. |
| dbExportStreamTclFile | string | user | | Specifies Tcl file. |
| dbExportStreamTclFunction | string | user | | Specifies Tcl function. |
| dbExportStreamText | <evViewType> | user | cdba | The namespace mapping for textDisplays used during exportStream translation. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--|----------|-------|---------|--|
| dbExportStreamTextBrackets | <string> | user | None | Specifies whether to convert text brackets. |
| dbExportStreamTextDepth | int | user | 256 | The text depth to translate to during oa2stream translation. |
| dbExportStreamTextDisplays | bool | user | true | Specifies if the oa2stream translator should not translate textDisplay objects.
Valid values: true, false |
| dbExportStreamTextMapFile | string | user | | Text mapping file name for GDSII exporting. |
| dbExportStreamTopCellName | string | user | | The top level cell that should be exported during GDSII writing. |
| dbExportStreamUnmappedLayersUseOALayerNumbers | bool | user | false | Use Open Access layer numbers if no lines are mapped in the layer mapping file.
Valid values: true, false |
| dbExportStreamUnmappedSystemPurposesUseDrawing | bool | user | false | Specifies if Stream Out translator maps drawing purpose when system purpose is not mapped in layer mapping.
Valid values: true, false |
| dbExportStreamUseTechLayerMap | bool | user | false | Use the layer map file that is in the source library tech library. This is enabled only if such a file exists.
Valid values: true, false |
| dbExportStreamUseTechObjectMap | bool | user | false | Use the object map file that is in the source library tech library. This is only enabled if such a file exists.
Valid values: true, false |
| dbExportStreamViaMapFile | string | user | | Via mapping file name for GDSII exporting. |
| dbExportStreamViewName | string | user | layout | The view that should be exported during GDSII out translation. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-----------------------------|---------------------------------|----------|-------------------|---|
| dbFilterParamLabelSet | bool | user | false | Filters paramLabelSet based on the current selected parameters in Parameter Filter. |
| dbFindAngleMode | string | cellview | orthogonal | Angle mode for <code>ide::_selectSearchRegion</code> command.
Valid values: anyAngle, diagonal, orthogonal, xFirst, yFirst |
| dbFindAutoSave | bool | user | false | Auto save during find replace all.
Valid values: true, false |
| dbFindAutoSelectInEditor | bool | user | false | Indicates if the Select All button in the Search/Replace GUI adds the found results to the design selection set.
Valid values: true, false |
| dbFindCaseSensitive | bool | user | true | Search in case-sensitive way.
Valid values: true, false |
| dbFindEIPModeRootDesign | dbFindEIP
ModeRoot
Design | user | Edit | The cellview to use as a search root while in Edit-In-Place mode. |
| dbFindFitView | bool | user | true | Zoom to fit when browsing through elements in the Search/Replace dialog box.
Valid values: true, false |
| dbFindHighlightCyclePackets | bool | user | false | Cycle packet names used for highlight sets creation.
Valid values: true, false |
| dbFindHighlightFoundObjects | bool | context | true | Indicates whether found objects should be highlighted. |
| dbFindHighlightPacket | string | user | SR_HighlightFound | Packet name to use for creating a highlighting of a result. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------------|--------|----------|--|---|
| dbFindHonorPropertyEditability | bool | user | true | Honor property editability when replacing in Search/Replace GUI.
Valid values: true, false |
| dbFindOnlyUnnettedShapes | bool | user | false | Finds only unnetted shapes.
Valid values: true, false |
| dbFindOnlyUnnettedVias | bool | user | false | Finds only unnetted vias.
Valid values: true, false |
| dbFindRegionType | string | cellview | Rectangle | Region type for <code>ide::_selectSearchRegion</code> command.
Valid values: Rectangle, Polygon |
| dbFindSelectType | string | user | enclose | Selection method (touch/enclose) for <code>ide::_selectSearchRegion</code> command.
Valid values: touch, enclose |
| dbGenerateCellViewBatchMode | bool | user | false | Specifies whether or not to run in batch mode.
Valid values: true, false |
| dbGenerateCellViewDefaultViewName | string | user | {schematic
symbol}
{layout
abstract}
{symbol
schematic} | Default view type name to define the cellview name to pick given a source cellview. |
| dbGenerateCellViewDestination | lcv | user | // | Indicates the default LCV for the destination cellview for the cellview from pin list dialog box. |
| dbGenerateCellViewEditor | string | user | | Default editor name to use to pre-seed the editor combo box. |
| dbGenerateCellViewOpenOnCompletion | bool | user | true | Indicates if a cellview should be opened after completion.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-------------------------------------|--------|-------|--|--|
| dbGenerateCellViewPinPositioning | string | user | source | Indicates the default positioning to use for pins. Valid values: source, default |
| dbGenerateCellViewsBatchMode | bool | user | false | Indicates whether to run on batch mode. Valid values: true, false |
| dbGenerateCellViewsDefaultViewName | string | user | {layout abstract} | Default view type name to define the cellview name to pick given a source cellview. |
| dbGenerateCellViewsDestination | lcv | user | // | Indicates the default LCV for the destination cellview. |
| dbGenerateCellViewsOpenOnCompletion | bool | user | true | Indicates if a cellview should be opened after completion. Valid values: true, false |
| dbGenerateCellViewSource | lcv | user | // | Indicates the default LCV for the source cellview. |
| dbGenerateCellViewsOverwriteAll | bool | user | false | Indicates whether to overwrite all cellviews. Valid values: true, false |
| dbGenerateCellViewsSource | string | user | // | Indicates the default LCV for the source cellview. |
| dbGenerateCellViewUniquePins | bool | user | false | Indicates whether to remove duplicate pins. Valid values: true, false |
| dbGenSimInfoProps | string | user | propMapping
namePrefix
instParameters
component
Name
netlistProcedure | The names of the properties to be created on symbol generation in simInfo sections specified in preference seGenSimInfoSectionNames. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------------|------------------|----------|----------|--|
| dbIdleLicenseTimeout | int | user | 0 | The length of time, in minutes, to wait for input before Custom Compiler releases all licenses back to the license server. If set to 0, the tool will never release licenses when idle. The minimum idle time is 15 minutes. |
| dbIgnoreUnconnectedConnectDefs | bool | user | true | When true, ignores the connection definition if the associated net does not have connected terminals or instances.
Valid values: true, false |
| dbImageAttachToObject | bool | cellview | false | Indicates whether created object should be attached to other object.
Valid values: true, false |
| dbImageColorMode | dbImageColorMode | cellview | color | Image output value.
Valid values: color, grayscale, blackAndWhite |
| dbImageCurrentViewport | bool | user | true | Control whether to output the current viewport or designs chosen from an LCV selector.
Valid values: true, false |
| dbImageDefaultDirectory | string | user | | Default directory to browse image files from. |
| dbImageDepth | int | cellview | -1 | Depth to output when creating an image. |
| dbImageDisplay | string | cellview | | The display to use. |
| dbImageDisplayMode | string | cellview | Expanded | Image display mode
Valid values: Expanded
Collapsed |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------|---------------|----------|--|--|
| dbImageFileMasks | string | user | *.bmp*
.gif *.jpg*
.jpeg
.png
.pbm
.pgm
.ppm
.xbm
.xpm | List of image file mask patterns |
| dbImageFilename | string | cellview | | Output file name including path. |
| dbImageFilePath | string | cellview | | Image file path |
| dbImageFormat | dbImageFormat | cellview | PNG | Image output format type.
Valid values: PNG, JPEG, BMP, SVG |
| dbImageHeight | int | cellview | 0 | Image height. |
| dbImageIncludeBackground | bool | cellview | true | Controls whether the background is exported.
Valid values: true, false |
| dbImageJustification | string | cellview | centerCenter | Image justification
Valid values: upperLeft
centerLeft lowerLeft
upperCenter centerCenter
lowerCenter upperRight
centerRight lowerRight |
| dbImageKeepAspectRatio | bool | cellview | false | Controls the image width/height ratio.
Valid values: true, false |
| dbImageOLPGroup | string | cellview | | The OLP group to use. |
| dbImageOmitBackground | bool | user | false | Controls whether the background is outputted.
Valid values: true, false |
| dbImageOverwrite | bool | user | false | Controls if outputting to a file should overwrite the file if it exists.
Valid values: true, false |
| dbImageSize | string | cellview | Medium | Image size.
Valid values: Small
Medium Large |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------|---------------|----------|-----------------|---|
| dbImageSizes | dbImageSize | cellview | default | Controls the exported image size. |
| dbImageSource | dbImageSource | cellview | currentViewport | Controls whether to output the current viewport, current design, or designs chosen from an Library, Cell, View selector. |
| dbImageWidth | int | cellview | 0 | Image width. |
| dbImportCreateVirtualTerms | bool | user | false | Indicates whether certain terminals (specified by the user or taken from a reference view) should be created as virtual when importing/editing a netlist. Valid values: true, false |
| dbImportDefCellName | string | user | | The output cell name for DEF importer. |
| dbImportDefCommentChar | string | user | # | Comment character for DEF parser. |
| dbImportDefCompressLevel | int | user | 1 | Specifies the compress level during DEF importing. |
| dbImportDefCreateCustomVias | bool | user | false | Specifies that the translator creates oaCustomViaDefs instead of existing oaStdViaDefs. Valid values: true, false |
| dbImportDefEnableCompression | bool | user | false | Specifies if the translator allows compressing libraries. Valid values: true, false |
| dbImportDefEnableCompression | bool | user | false | Specifies if the translator allows compressing libraries. Valid values: true, false |
| dbImportDefExcludeRefLibs | string | user | | List of libraries to ignore during import DEF. |
| dbImportDefFileName | string | user | | The DEF file for def2oa binary. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-----------------------------|--------|-------|----------|--|
| dbImportDefImportRouting | bool | user | true | Specifies if translator should ignore any routing data during translation including vias section.
Valid values: true, false |
| dbImportDefIncludeRefLibs | string | user | | Libraries to use as reference library during import DEF. |
| dbImportDefLayerMapping | string | user | | Layer mapping file for DEF importer. |
| dbImportDefLibName | string | user | | The library name where DEF file should be imported. |
| dbImportDefMapDividerChar | string | user | | Maps divider character to express hierarchy during DEF import. |
| dbImportDefOverwrite | bool | user | true | Specifies if existing macros should be replaced during DEF importing.
Valid values: true, false |
| dbImportDefRefLibs | string | user | | Provides list of library names in which translator should search for master cells. |
| dbImportDefRefViews | string | user | abstract | Specifies view names to search for master cells. |
| dbImportDefRunDirectory | string | user | . | The run directory for def2oa binary. |
| dbImportDefSuppressInfos | string | user | | List of info message ids that should be suppressed during DEF importing. |
| dbImportDefSuppressWarnings | string | user | | List of warning message ids that should be suppressed during DEF importing. |
| dbImportDefViewName | string | user | layout | The output view name for DEF importer. |
| dbImportLefBusBitChars | string | user | [] | Busbit characters for LEF parser. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|------------------------------------|--------|-------|---------|--|
| dbImportLefCommentChar | string | user | # | Comment character for LEF parser. |
| dbImportLefCompressLevel | int | user | 1 | Specifies the compress level during LEF importing. |
| dbImportLefCreatePinText | bool | user | true | Specifies layer of oaText will be placed on pin figures if Create Pin Text is true.
Valid values: true, false |
| dbImportLefFileName | string | user | | The LEF file for lef2oa binary. |
| dbImportLefLayerMapping | string | user | | Layer mapping file for LEF importer. |
| dbImportLefLayoutView | string | user | layout | Layout view to create foreign containers. |
| dbImportLefLibName | string | user | | The library name where LEF file should be imported. |
| dbImportLefOverwriteMacros | bool | user | false | Specifies if existing macros should be replaced during LEF importing. Valid values: true, false |
| dbImportLefPinPurpose | string | user | drawing | Specifies purpose of figures of pin |
| dbImportLefPinTextHeight | float | user | 0.02 | Specifies if the height of oaText is placed on pin figures if Create Pin Text is true. |
| dbImportLefPinTextLayer | string | user | text | Specifies the layer of oaText is placed on pin figures if Create Pin Text is true. |
| dbImportLefPinTextLayerInheritance | bool | user | false | Specifies if the layer of oaText will be inherited from pin figures if Create Pin Text is true |
| dbImportLefPinTextPurpose | string | user | | Specifies purpose of oaText will be placed on pin figures if Create Pin Text is true |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--|--------|-------|-------------|---|
| dbImportLefRunDirectory | string | user | . | The run directory for lef2oa binary. |
| dbImportLefSuppressInfos | string | user | | List of info message ids that should be suppressed during LEF importing. |
| dbImportLefSuppressWarnings | string | user | | List of warning message ids that should be suppressed during LEF importing. |
| dbImportLefViewName | string | user | abstract | The output view name for LEF importer. |
| dbImportMCellCDFFilename | string | user | | File name for loading MCell CDF params. |
| dbImportMCellFileName | string | user | | File name for importing MCell superMasters. |
| dbImportMilkywayCreateDisplayResources | bool | user | false | Creates display resources file for Import IC Compiler. Valid values: true, false |
| dbImportMilkywayCreateSiteDefs | bool | user | true | Creates missing Site Definitions in the OA technology from the IC Compiler technology. |
| dbImportMilkywayCreateTechLib | bool | user | false | Creates a separate technology library when creating a new library in Import IC Compiler. Valid values: true, false |
| dbImportMilkywayDefaultLibPath | string | user | <file path> | Default path in which to create reference libraries for Import IC Compiler. |
| dbImportMilkywayDisplayResourcesFile | string | user | | Displays resources file to create for Import IC Compiler. |
| dbImportMilkywayExcludeLibs | string | user | | List of libraries to exclude for Import IC Compiler. |
| dbImportMilkywayFRAM2CEL | bool | user | false | Remasters instances placing FRAM views to use CEL views, if available, in Import IC Compiler. Valid values: true, false |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--------------------------------------|--------|-------|-------------|---|
| dbImportMilkywayHierDepth | int | user | 20 | Hierarchy depth for Import IC Compiler. |
| dbImportMilkywayLoadDisplayResources | bool | user | false | Load created display resources file in Import IC Compiler.
Valid values: true, false |
| dbImportMilkywayMapMode | string | user | Merge | Merges or replaces cached map files by user map files. |
| dbImportMilkywayOverwriteExisting | bool | user | false | Overwrites existing cellviews in Import IC Compiler.
Valid values: true, false |
| dbImportMilkywaySourceCells | string | user | | Source cells for Import IC Compiler. |
| dbImportMilkywaySourceLibrary | string | user | | Source library for Import IC Compiler. |
| dbImportMilkywaySourceViews | string | user | | Source views for Import Milkyway. |
| dbImportMilkywayTechLib | string | user | | Reference technology library for creating a new library in Import IC Compiler. |
| dbImportMilkywayTechSource | string | user | ICC Library | Technology source for creating a new library in Import IC Compiler. |
| dbImportOADataModel | int | user | 4 | Data model for OA to use during translation. |
| dbImportOasisAutoDetectVias | bool | user | false | If set to true, this preference automatically detects via objects during oasis import.
Valid values: true, false |
| dbImportOasisBatchMode | bool | user | true | Specifies whether to run Import Oasis with batch mode.
Valid values: true, false |
| dbImportOasisCellMapFile | string | user | | Cell mapping file name for importing OASIS. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|--------|-------|---------|--|
| dbImportOasisColorCellMapFileList | string | user | | File list for setting color shift layers to cells. |
| dbImportOasisConvertNames | string | user | None | Specifies whether the <code>Oasis In</code> translator should convert all cell and instance names to uppercase, lowercase, or leave them as-is. <ul style="list-style-type: none"> Valid values: 0: Does not convert cell or instance names. 1: Converts cell or instance names to uppercase. 2: Converts cell or instance names to lowercase. |
| dbImportOasisConvertPathsToPathSegs | bool | user | false | Imports two point orthogonal and diagonal path records as pathSeg objects.
Valid values: true, false |
| dbImportOasisCreateViaDefs | bool | user | true | Specifies whether or not new ViaDef can be created during OASIS import.
Valid values: true, false |
| dbImportOasisDbuPerUU | string | user | | Specifies the database units per user units value. |
| dbImportOasisDepth | int | user | 256 | The hierarchical depth to translate to. |
| dbImportOasisDisplayCompletion | bool | user | false | Specifies whether to display information after Import OASIS.
Valid values: true, false |
| dbImportOasisDumpDesignTree | bool | user | false | Dumps the design tree to a log file.
Valid values: true, false |
| dbImportOasisDuplicateCellAutoHandling | string | user | | Uses action without showing dialog box. |
| dbImportOasisDuplicateCellHandling | string | user | Skip | Specifies how to handle duplicate cells. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------|-------|---------|---|
| dbImportOasisExcludeRefLibs | string | user | | List of libraries to ignore during Import Oasis. |
| dbImportOasisExistingLibAutoHandling | string | user | | Uses action without showing dialog box. |
| dbImportOasisFileName | string | user | | The STREAM file for strm2oa binary. |
| dbImportOasisFilterAcutePaths | bool | user | false | Specifies whether or not the translator should filter out acute paths.
Valid values: true, false |
| dbImportOasisFilterDots | bool | user | true | Specifies whether or not the translator should filter out dots.
Valid values: true, false |
| dbImportOasisFilterReentrantPgns | bool | user | false | Specifies whether or not the translator should filter out reentry polygons and paths.
Valid values: true, false |
| dbImportOasisFilterSelfIntersectingPgns | bool | user | false | Specifies whether or not the translator should filter out self-intersecting polygons and paths
Valid values: true, false |
| dbImportOasisFilterZeroWidth | bool | user | false | Specifies whether or not the translator should filter out zero width paths.
Valid values: true, false |
| dbImportOasisFontMapFile | string | user | | Font mapping file name for Oasis importing. |
| dbImportOasisHierDepth | string | user | 20 | Hierarchy Depth Preference. |
| dbImportOasisIgnoreBoxes | bool | user | false | Specifies whether or not Oasis Box Records should be translated.
Valid values: true, false |
| dbImportOasisIgnoreUnmappedLayers | bool | user | true | Ignores layers if they don't have any mapped lines in the layer mapping file.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------------|--------|-------|-------------|--|
| dbImportOasisIncludeRefLibs | string | user | | Specifies the library or libraries to use as reference libraries during import Oasis. |
| dbImportOasisInternalizeShapes | bool | user | true | Specifies whether or not the translator should internalize the shapes. |
| dbImportOasisKeepInstName | bool | user | false | The option of property of inst's name for OASIS importing. |
| dbImportOasisKeepLogicInfo | bool | user | false | The option of property of logic's info for OASIS exporting.
Valid values: true, false |
| dbImportOasisKeepNetName | bool | user | false | The option of property of net's name for OASIS exporting.
Valid values: true, false |
| dbImportOasisKeepWideAlignment | bool | user | false | The Option of property of alignment boundary for OASIS importing.
Valid values: true, false |
| dbImportOasisLayerMapColorMode | string | user | EnableColor | Specifies layer mapping color mode. |
| dbImportOasisLayerMapFile | string | user | | The file name of the layer mapping file for OASIS import. |
| dbImportOasisLayerMappingMode | string | user | tech | Specifies the layer mapping mode. |
| dbImportOasisLibDir | string | user | . | Default Lib path. |
| dbImportOasisLibName | string | user | | The library name where the OASIS file should be imported. |
| dbImportOasisLibraryFile | string | user | | Library File Preference. |
| dbImportOasisLibraryList | string | user | | Library List Preference. |
| dbImportOasisLimitCellsPerLib | bool | user | false | Specifies whether or not the cells are limited in a library.
Valid values: true, false |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---|--------|-------|---------|---|
| dbImportOasisLimitLogFiles | string | user | 5 | Limits the maximum number of the backup log files. |
| dbImportOasisLogFileName | string | user | | Specifies the log file for OASIS import. |
| dbImportOasisNewLibTechFile | string | user | | new lib tech type |
| dbImportOasisNewLibTechLib | string | user | | new lib tech type |
| dbImportOasisNewLibTechRefLibs | string | user | | new lib tech references |
| dbImportOasisNewLibTechType | string | user | none | new lib tech type |
| dbImportOasisOaDataModel | int | user | 3 | Data model for OA to use during translation. |
| dbImportOasisObjectMapFile | string | user | | The file name of the object mapping file for OASIS import. |
| dbImportOasisObjectMappingMode | string | user | tech | Specifies the object mapping mode. |
| dbImportOasisOverwrite | bool | user | false | Specifies whether or not the existing macros should be replaced during OASIS import.
Valid values: true, false |
| dbImportOasisOverwriteExistingDesigns | string | user | false | Overwrite Existing Designs Preference. |
| dbImportOasisPropMapFile | string | user | | Property mapping file name for OASIS importing |
| dbImportOasisPropNameForInstName | string | user | 102 | The property name of inst's name for OASIS importing. |
| dbImportOasisPropNameForLogicName | string | user | 103 | The property name of logic info for OASIS importing. |
| dbImportOasisPropNameForNetName | string | user | 102 | The property name of net's name for OASIS importing. |
| dbImportOasisPropNameForWideAlignm
ent | string | user | 105 | The property name of alignment boundary for OASIS importing |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------|-------|---------|---|
| dbImportOasisPropSeparator | string | user | , | The character to use as the separator in the property mapping file. |
| dbImportOasisRefLibFile | string | user | | The file containing the names of reference libraries. |
| dbImportOasisRefLibNames | string | user | | A list of libraries with cells references in the design. |
| dbImportOasisRefOtherLibsFirst | bool | user | false | Specifies whether or not the reference cells are from other libraries first. Valid values: true, false |
| dbImportOasisRefViewNames | string | user | | Views of reference libraries to be used to bind cells references during OASIS import. |
| dbImportOasisResizeRatio | string | user | | Resizes cells with a scale factor. |
| dbImportOasisRestorePcells | bool | user | false | Specifies whether or not to automatically detect PCell objects during OASIS import. Valid values: true, false |
| dbImportOasisRunDirectory | string | user | . | The run directory for strm2oa binary. |
| dbImportOasisSetColorShiftLayersToCells | bool | user | false | Sets color shift layers to cells. Valid values: true, false |
| dbImportOasisSnapToMfgGrid | bool | user | false | Specifies whether vertices snap with the manufacturing grid resolution. Valid values: true, false |
| dbImportOasisSuppressInfo | string | user | | A list of info message ids that should be suppressed during OASIS importing. |
| dbImportOasisSuppressWarnings | string | user | | A list of warning message ids that should be suppressed during OASIS importing. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--|--------|-------|------------------|--|
| dbImportOasisTextBrackets | string | user | None | Specifies whether to convert text brackets. |
| dbImportOasisTextHeight | string | user | | Specifies the DBU for the height of text shapes imported during OASIS import. |
| dbImportOasisTextMapFile | string | user | | Text mapping file name for OASIS importing |
| dbImportOasisTopCell | string | user | | The output cell name for OASIS importer. |
| dbImportOasisUnmappedLayersUseOasisNumbers | bool | user | false | Use Oasis layer number if not mapped any lines in layer mapping file.
Valid values: true, false |
| dbImportOasisUseTechLayerMap | bool | user | false | Use the layer map file that is in the source tech library. This is only enabled if such a file exists.
Valid values: true, false |
| dbImportOasisUseTechObjectMap | bool | user | true | Use the object map file that is in the source tech library. This is only enabled if such a file exists.
Valid values: true, false |
| dbImportOasisViewName | string | user | layout | The output view name for OASIS importer. |
| dbImportRamDiskChunkSize | string | user | | Specifies ramdisk size. |
| dbImportRamDiskUtilityPath | string | user | | Specifies path of tar command for ramdisk. |
| dbImportReferenceViewList | string | user | schematic symbol | The reference views used for selecting terminals to be created as virtual when importing or editing a netlist and the terminal selection method is set to <i>reference</i> . |
| dbImportStreamAutoDetectVias | bool | user | false | Auto detects via objects during stream import.
Valid values: true, false |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---|--------|-------|---------|--|
| dbImportStreamBatchMode | bool | user | true | Specifies whether to run Import Stream with batch mode.
Valid values: true, false |
| dbImportStreamCellMapFile | string | user | | Cell mapping file name for GDSII importing. |
| dbImportStreamColorCellMapFileList | string | user | | File list for setting color shift layers to cells. |
| dbImportStreamConvertNames | string | user | None | Specifies if Stream In translator should convert all cell and instance names to uppercase or to lowercase or just leave as it is. 0 - no conversion 1 - to uppercase 2 - to lowercase. |
| dbImportStreamConvertPathsToPathSegs | bool | user | false | Imports two point orthogonal and diagonal path records as pathSeg objects.
Valid values: true, false |
| dbImportStreamCreateViaDefs | bool | user | false | Specifies if new ViaDef can be created during Stream Import.
Valid values: true, false |
| dbImportStreamDbuPerUU | string | user | 1000 | Database units per user units value. |
| dbImportStreamDepth | int | user | 256 | The hierarchical depth to translate to. |
| dbImportStreamDisplayCompletion | bool | user | false | Specifies whether to display information after Import.
Valid values: true, false |
| dbImportStreamDumpDesignTree | bool | user | false | Dumps the design tree to a log file.
Valid values: true, false |
| dbImportStreamDuplicateCellAutoHandling | string | user | | Uses action without showing dialog box. |
| dbImportStreamDuplicateCellHandling | string | user | Skip | How to handle duplicate cell. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|--------|-------|---------|---|
| dbImportStreamExcludeRefLibs | string | user | | List of libraries to ignore during import Stream. |
| dbImportStreamExistingLibAutoHandling | string | user | | Uses action without showing dialog box. |
| dbImportStreamFileName | string | user | | The STREAM file for strm2oa binary. |
| dbImportStreamFilterAcutePaths | bool | user | false | Specifies if translator filters out acute paths.
Valid values: true, false |
| dbImportStreamFilterDots | bool | user | true | Specifies whether the translator should filter out dots.
Valid values: true, false |
| dbImportStreamFilterReentrantPgns | bool | user | false | Specifies if translator filter out reentry polygons.
Valid values: true, false |
| dbImportStreamFilterSelfIntersectingPgns | bool | user | false | Specifies if translator filter out self-intersecting polygons.
Valid values: true, false |
| dbImportStreamFilterZeroWidth | bool | user | false | Specifies if translator filter out zero width paths.
Valid values: true, false |
| dbImportStreamFontMapFile | string | user | | Font mapping file name for GDSII importing. |
| dbImportStreamIgnoreBoxes | bool | user | false | Specifies if Stream Box Records should be translated.
Valid values: true, false |
| dbImportStreamIgnoreUnmappedLayers | bool | user | true | Ignore layers if not mapped any lines in layer mapping file.
Valid values: true, false |
| dbImportStreamIncludeRefLibs | string | user | | Libraries to use as reference library during import Stream. |
| dbImportStreamInternalizeShapes | bool | user | true | Specifies if translator internalize the shapes.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------------|--------|-------|-------------|--|
| dbImportStreamKeepInstName | bool | user | false | The option of property of inst's name for GDSII importing.
Valid values: true, false |
| dbImportStreamKeepLogicName | bool | user | false | The option of property of logic's info for GDSII importing.
Valid values: true, false |
| dbImportStreamKeepNetName | bool | user | false | The option of property of net's name for GDSII importing.
Valid values: true, false |
| dbImportStreamKeepWideAlignment | bool | user | false | The Option of property of alignment boundary for GDSII importing.
Valid values: true, false |
| dbImportStreamLayerMapColorMode | string | user | EnableColor | Specifies layer mapping color mode. |
| dbImportStreamLayerMapFile | string | user | | Layer mapping file name for GDSII importing. |
| dbImportStreamLayerMappingMode | string | user | tech | Specifies the layer mapping mode. |
| dbImportStreamLibDir | string | user | . | Default Lib path. |
| dbImportStreamLibName | string | user | | The library name where Stream file should be imported. |
| dbImportStreamLimitCellsPerLib | bool | user | false | Specifies if the cells are limited in a library.
Valid values: true, false |
| dbImportStreamLimitLogFiles | string | user | 5 | Specifies whether to limit the maximum number of log files. |
| dbImportStreamLogFileName | string | user | | Specifies log file. |
| dbImportStreamMaxVertices | string | user | 8190 | Specifies if limit the maximum vertices for path and polygon. |
| dbImportStreamNewLibTechFile | string | user | | New lib tech type |
| dbImportStreamNewLibTechLib | string | user | | New lib tech type |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---------------------------------------|--------|-------|---------|---|
| dbImportStreamNewLibTechRefLibs | string | user | | New lib tech type |
| dbImportStreamNewLibTechType | string | user | none | New lib tech type |
| dbImportStreamOaDataModel | int | user | 3 | Data model for OA to use during translation. |
| dbImportStreamObjectMapFile | string | user | | Object mapping file name for importStream. |
| dbImportStreamObjectMappingMode | string | user | tech | Specifies the object mapping mode. |
| dbImportStreamOutputBBoxInfo | bool | user | false | Specifies if export BBox information for cells in the log file. |
| dbImportStreamOverwrite | bool | user | false | Specifies if existing macros should be replaced during Stream importing.
Valid values: true, false |
| dbImportStreamPropMapFile | string | user | | Property mapping file name for GDSII importing. |
| dbImportStreamPropNumForInstName | string | user | 102 | The property number of inst's name for GDSII importing. |
| dbImportStreamPropNumForLogicName | string | user | 103 | The property number of logic's info for GDSII importing. |
| dbImportStreamPropNumForNetName | string | user | 102 | The property number of net's name for GDSII importing. |
| dbImportStreamPropNumForWideAlignment | string | user | 105 | The property number of alignment boundary for GDSII importing. |
| dbImportStreamPropSeparator | string | user | , | The character to use as the separator in the property mapping file. |
| dbImportStreamPropValuesOnly | bool | user | true | Specifies if translator should input only property values.
Valid values: true, false |
| dbImportStreamRefLibFile | string | user | | Filename containing names of reference libraries. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--|--------|-------|---------|---|
| dbImportStreamRefLibNames | string | user | | Provides list of libraries with cells referenced in the design. |
| dbImportStreamRefOtherLibsFirst | bool | user | false | Specifies if the reference cells are from other libraries first.
Valid values: true, false |
| dbImportStreamRefViewNames | string | user | | Views of reference libraries to be used to bind cells referenced during stream import. |
| dbImportStreamResizeRatio | string | user | | Resize cell with a scale factor |
| dbImportStreamRestorePcells | bool | user | false | Auto detect PCell objects during stream import.
Valid values: true, false |
| dbImportStreamRestoreViewName | bool | user | false | Specifies if importstream translator should restore the view name.
Valid values: true, false |
| dbImportStreamRunDirectory | string | user | . | The run directory for strm2oa binary. |
| dbImportStreamSetColorShiftLayersToCells | bool | user | false | Set color shift layers to cells.
Valid values: true, false |
| dbImportStreamSnapToMfgGrid | bool | user | false | Specifies if the vertices snap with manufacturing grid resolution.
Valid values: true, false |
| dbImportStreamSortLayer | bool | user | false | Specifies if sorting layer statistics in the log file.
Valid values: true, false |
| dbImportStreamSuppressInfos | string | user | | List of info message ids that should be suppressed during Stream importing. |
| dbImportStreamSuppressWarnings | string | user | | List of warning message ids that should be suppressed during Stream importing. |
| dbImportStreamTextBrackets | string | user | None | Specifies whether to convert text brackets. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|-------------------------|-------|-------------------------|--|
| dbImportStreamTextHeight | string | user | | Height of text shapes imported in User Units. |
| dbImportStreamTopCellName | string | user | | The output cell name for Stream importer. |
| dbImportStreamUnmappedLayersUseStreamNumbers | bool | user | false | Use Stream layer number if there are no lines mapped in layer mapping file. Valid values: true, false
Valid values: true, false |
| dbImportStreamUseSi2ViaDetection | bool | user | false | Use si2 via detection. Valid values: true, false |
| dbImportStreamUseTechLayerMap | bool | user | false | Use the layer map file that is in the source tech library. This is only enabled if such a file exists.
Valid values: true, false |
| dbImportStreamUseTechObjectMap | bool | user | true | Use the object map file that is in the source tech library. This is only enabled if such a file exists.
Valid values: true, false |
| dbImportStreamViaMapFile | string | user | | Via mapping file name for GDSII importing. |
| dbImportStreamViewName | string | user | layout | The output view name for Stream importer. |
| dbImportTermMapping | List of List of unknown | user | | The terminal to connection definition mapping. Used when creating virtual terminals during text importing or editing. |
| dbImportTextAnalogFileExtSkipParsing | string | user | .spf
.spf.gz
.enc | Space-separated extensions of include files to be skipped during analog text view parsing. |
| dbImportTextCDLFileExtensions | string | user | cdl | Default extensions for imported CDL text files. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|------------------------------------|---------------|-------|---------|--|
| dbImportTextCheckVCSVersion | bool | user | false | Checks VCS version with CC version when importing Verilog netlist.
Valid values: true, false |
| dbImportTextCreatePreview | bool | user | true | Creates previews on text import.
Valid values: true, false |
| dbImportTextCrossViewCheck | bool | user | true | Runs Cross View Check on text importing.
Valid values: true, false |
| dbImportTextDefines | string | user | | Space-separated list of define statements to be added to all cells (Verilog specific). |
| dbImportTextDeviceMapFile | string | user | | The device map file used during SPICE and CDL import. |
| dbImportTextDeviceMapFileSimInfo | string | user | hspiceD | SimInfo section name to be used for generating the device map file. |
| dbImportTextEnableParamCallbacks | bool | user | false | Indicates whether to trigger instance parameter callbacks during text view extraction.
Valid values: true, false |
| dbImportTextExcludeModuleMatchType | <enMatchType> | user | exact | The type of module matching given in dbImportTextExcludeModules preference.
Valid values: exact, regexp, wildcard |
| dbImportTextExcludeModules | string | user | | The names of the modules that should not be imported. |
| dbImportTextFastDiskPath | string | user | | Fast disk (RAM, SCRATCH or TMP directory) path to use for creating intermediate library for fast importing of large number of modules. |
| dbImportTextFileNames | string | user | | Netlist file names to be imported. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------------------|--------|-------|--|---|
| dbImportTextGeneratedDeviceMapFile | string | user | | The device map file name to be generated. |
| dbImportTextGenerateSchematics | bool | user | false | Generates schematics for created text views.
Valid values: true, false |
| dbImportTextGenerateSymbols | string | user | false | Generates symbols for created text views.
Valid values: true, false |
| dbImportTextImportModules | string | user | | The names of the modules that should be imported. |
| dbImportTextIncludesProcessing | string | user | Include | Determines whether to process include directives (Verilog specific).
Valid values: exact, regexp. wildcard |
| dbImportTextIncludesProcessingSPICE | string | user | parse
{include
"*.*dspf *.spf
*.gz *.gzip
*.enc"
wildcard}
{ignore
"*.*lib *.vec
*.ic *.data"
wildcard} | Include files processing type to use during SPICE files import.
Valid values: exact, regexp. wildcard |
| dbImportTextIncludesProcessingVerilog | string | user | parse | Include files processing type to use during Verilog files import.
Valid values: exact, regexp. wildcard |
| dbImportTextInterfaceOnly | bool | user | false | Flag, limiting the importing file parsing to the modules interface only.
Valid values: true, false |
| dbImportTextLanguage | string | user | | The language of the imported files.
Valid values: SPICE, CDL, Verilog, Verilog-A, Verilog-Ams, VHDL, SystemVerilog |
| dbImportTextLibName | string | user | | The destination library name, where importText creates imported modules |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------------|--------------------------------|-------|---------|--|
| dbImportTextMappingType | bool | user | true | The way to use a device mapping file. The default value true means that device mapping file is generated from reference library. |
| dbImportTextMinimumCheck | bool | user | true | Indicates whether to use relaxed syntax checking. |
| dbImportTextModuleMatchType | <enMatchType> | user | exact | Indicates the type of matching for the module names specified in dbImportTextImportModules preference. |
| dbImportTextOverwrite | string | user | never | Specifies if existing designs should be overwritten during text import.
Valid values: always, never |
| dbImportTextOverwriteCDFValue | dbNetlistParserParamUpdateMode | user | always | Specifies whether to overwrite the default value of CDF parameters during text import. <ul style="list-style-type: none"> • <code>always</code> updates the CDF parameters. • <code>never</code> making any changes to CDF parameters. • <code>prompt</code> a question box to let you decide to do one of the above actions. Valid values: always, never, prompt |
| dbImportTextOverwriteDeviceMapFile | bool | user | true | Specifies if the existing device mapping file should be overwritten during text import.
Valid values: true, false |
| dbImportTextOverwriteSymbols | string | user | never | Specifies if existing symbols should be overwritten during text import. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--|--------|-------|---------|--|
| dbImportTextPreserveDirectives | bool | user | true | Determines whether to process the compiler directives (Verilog specific).
Valid values: true, false |
| dbImportTextPreserveNonModuleComments | bool | user | false | Indicates whether the comments that are out of module bodies should be preserved after importing.
Valid values: true, false |
| dbImportTextReferenceLibrary | string | user | | The reference library name for which device mapping file is generated. |
| dbImportTextRefLibs | string | user | | Space-separated list of libraries for lookup for leaf cellviews binding. |
| dbImportTextRefViews | string | user | symbol | Space-separated list of views for lookup for leaf cellviews binding. |
| dbImportTextReportVerilogAUndefinedIdent | bool | user | false | Reports undefined identifiers used in verilogA/AMS as violation.
Valid values: true, false |
| dbImportTextRunDirectory | string | user | ./ | The run directory for importText. |
| dbImportTextSingleSource | bool | user | false | Imports the netlist file as a single source text.
Valid values: true, false |
| dbImportTextSkipModDefGeneration | bool | user | false | Skips the module definition file generation from the reference library.
Valid values: true, false |
| dbImportTextSkipSubcktLocalParams | bool | user | false | Skips parameters defined in subcircuit with .param syntax when creating CDF parameters.
Valid values: true, false |
| dbImportTextSpectreFileExtensions | string | user | scs | Default extensions for imported spectre text files. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------|-------|---------|--|
| dbImportTextSpiceCaseSensitiveCellNames | bool | user | false | Indicates whether imported spice/cdl subckts should go to a case insensitively matching existing cell name (false value).
Valid values: true, false |
| dbImportTextSpiceCDLFirstLineAsComment | bool | user | true | Boolean indicating whether recognizing the first line in SPICE/CDL netlist files as comment.
Valid values: true, false |
| dbImportTextSymbolOptionSet | string | user | default | Symbol generator option set name to use for generating symbols. |
| dbImportTextSystemVerilogFileExtensions | string | user | sv | Default extensions for imported SystemVerilog text files. |
| dbImportTextTopModule | string | user | | The subcircuit name treated as the top subcircuit during SPICE parsing |
| dbImportTextUseForkModuleCount | int | user | 100 | Minimum number of imported modules to use forking for symbol generation. The minimum number is 100, where 0 value means that no imported modules are forked. |
| dbImportTextVCSParserOptions | string | user | | Space-separated list of VCS parser specific options. |
| dbImportTextVerilogAFileExtensions | string | user | va | Default extensions for imported Verilog-A text files. |
| dbImportTextVerilogAMSFileExtensions | string | user | vams | Default extensions for imported Verilog-AMS text files. |
| dbImportTextVerilogAParser | string | user | legacy | Parser to use when importing Verilog-A text files.
Valid values: legacy, vcs |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------|-------|-------------|--|
| dbImportTextVerilogAViewName | string | user | veriloga | Default name for imported Verilog views. |
| dbImportTextVerilogFileExtensions | string | user | v vp | Default extensions for imported Verilog text files. |
| dbImportTextVHDLFileExtensions | string | user | vhd vhdl | Default extensions for imported VHDL text files. |
| dbImportTextVHDLSimSetup | string | user | | SYNOPSYS_SIM_SETUP environment variable to be used on VHDL netlist import.
SYNOPSYS_SIM_SETUP is unset if the preference value is empty. |
| dbImportTextVHDLUsePrefixInArchitectureViewName | bool | user | true | Specifies whether prefix ARCH_ should be used in architecture view names when importing VHDL texts.
Valid values: true, false |
| dbImportTextVHDLUsePrefixInConfigViewName | bool | user | true | Specifies whether prefix CONF_ should be used in VHDL configuration view names when importing VHDL texts.
Valid values: true, false |
| dbImportTextVHDLViewName | string | user | vhdl_config | The view name for imported VHDL views. |
| dbImportTextViewName | string | user | | The view name for all newly created designs. |
| dbImportUseReferenceViews | bool | user | false | Indicates whether to use reference views to create virtual terminals when importing or editing a netlist.
Valid values: true, false |
| dbImportUseSpecifiedMapping | bool | user | true | Indicates whether to use the specified mapping to create virtual terminals when importing or editing a netlist.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------------|--------|-------|---------|---|
| dbImportVerilogBlackBox | bool | user | true | Specifies if a design is created for empty Verilog modules.
Valid values: true, false |
| dbImportVerilogDesignPerModule | bool | user | true | Specifies if a separate design is created for each empty module.
Valid values: true, false |
| dbImportVerilogExcludeRefLibs | string | user | | Library(s) to ignore as reference library during import. |
| dbImportVerilogFileNames | string | user | | Verilog file name to be imported. |
| dbImportVerilogGlobalNetModule | string | user | | The module name that contains all global nets. |
| dbImportVerilogIncludeRefLibs | string | user | | Library to use as reference library during Import Verilog. |
| dbImportVerilogLibName | string | user | | The library name, that Verilog import writes modules. |
| dbImportVerilogOverwrite | bool | user | true | Specifies if existing designs are overwritten during Verilog import.
Valid values: true, false |
| dbImportVerilogRefLibs | string | user | | Space-separated list of libraries translator searches for leaf cells. |
| dbImportVerilogRefViews | string | user | | Space-separated list of views translator searches for leaf cells. |
| dbImportVerilogRunDirectory | string | user | . | run directory for import Verilog binary. |
| dbImportVerilogSuppressInfos | string | user | | List of info message ids to suppress. |
| dbImportVerilogSuppressWarnings | string | user | | List of warning message ids to suppress. |
| dbImportVerilogTieHighNet | string | user | | The name of global net, to which literal ones are mapped. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------|---------------------|-------|-------------|---|
| dbImportVerilogTieLowNet | string | user | | The name of global net, to which literal zeros are mapped. |
| dbImportVerilogTolerate | bool | user | true | Specifies if un-implemented Verilog features cause warnings instead of errors. Valid values: true, false |
| dbImportVerilogTopModule | string | user | | The module name treated as the top module during Verilog parsing. |
| dbImportVerilogViewName | string | user | netlist | View name for all newly created designs. |
| dbImportVerilogViewType | string | user | netlist | View type for all newly created designs. |
| dbIncrementalDrawingOn | int | user | 1 | Incremental Drawing On. |
| dbKeepDesignsOpen | bool | user | true
ref | Keeps all read-only designs in memory, skipping only temporary and schematicSymbol designs. The maximum number of open designs is 10,000. Valid values: true, false |
| dbKeepSubMasters | bool | user | false | Keeps subMasters in memory. The maximum number of open designs is set with the dbKeepDesignsOpenLimit preference. The value of this preference is ignored if the dbKeepDesignsOpen preference is enabled. Valid values: true, false |
| dbLastPrefsFilePath | string | user | | Last-entered preference file path in Save Preferences or Load Preferences. |
| dbLefVersion | dbLefDefVersionEnum | user | 5.8 | Version for exported DEF files |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------|--------------------------|-------|---|---|
| dbLibertyFileDefaults | string | user | \$SYNOPSIS_CUSTOM_INSTALL/data_io/libertyFileDefaults.xml | The path to file with default value overrides. |
| dbLibertyFileDefCorner | string | user | nominal | The default name of the corner of liberty data. |
| dbLibertyFileDefPowerType | dbVoltageType | user | primary | The default type of power voltage used in liberty file generation. |
| dbMatchPartial | bool | user | false | Select the matching type. Valid values: true, false |
| dbMaxLogFiles | int | user | -1 | The maximum number of session log files that is retained in the user's working directory. |
| dbMilkywayBusDelim | string | user | [%d] | Default Milkyway bus delimiter. |
| dbMilkywayCellMapFile | string | user | | Cell map file for IC Compiler translation. |
| dbMilkywayCreateLayers | bool | user | true | Creates layers in IC Compiler translation. Valid values: true, false |
| dbMilkywayExcludeLibs | string | user | | List of libraries to exclude for Milkyway translation. |
| dbMilkywayLayerMapFile | string | user | | Layer map file for IC Compiler translation. |
| dbMilkywayNoLayerAutoMap | bool | user | false | Does not translate shapes on layers which do not exist in the layer map file. Valid values: true, false |
| dbMilkywayOATechSource | <enMilkywayOATechSource> | user | icc | Stores the default OA technology source for new ICC libraries. |
| dbMilkywayObjectMapFile | string | user | | Object map file for IC Compiler translation. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------|--------------------|---------|-------------|---|
| dbMilkywayPropertyMapFile | string | user | | Property map file for IC Compiler translation. |
| dbMilkywayRefLibMapFile | string | user | | Reference library map file for IC Compiler translation. |
| dbMilkywaySuppressInfolds | string | user | | List of information message ids to exclude for IC Compiler translation. |
| dbMilkywaySuppressWarningIds | string | user | | List of warning message ids to exclude for IC Compiler translation. |
| dbMilkywayViaMapFile | string | user | | Via map file for IC Compiler translation. |
| dbPackagingTechFileName | string | library | | Packaging technology file name. |
| dbParamDefaultValuesIsZero | bool | user | true | <p>If set to true, then <code>evalDefaultZeroOrEmpty("")</code> function(callback) returns 0.</p> <p>If set to false, <code>evalDefaultZeroOrEmpty("")</code> function(callback) returns an empty string.</p> <p>You can use <code>evalDefaultZeroOrEmpty("")</code> callback to evaluate CDF parameter's value.</p> <p>Valid values: true, false</p> |
| dbParamDisplayMode | dbParamDisplayMode | user | engineering | Parameter display mode.
Valid values: engineering, scientific |
| dbParamDisplayPrecision | int | user | 5 | <p>The number of significant digits to use when displaying real values in the Property Editor dialogs and on the canvas. The valid range is between 1 and <code>std::numeric_limits<double>::digits10</code> inclusive.</p> <p>Valid values: Any integer between 1 and 15</p> |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------------------|--------------------|---------|-------------|---|
| dbParameterEditorFind | bool | user | true | Shows or hides the parameter editor find bar.
Valid values: true, false |
| dbParamEvalRemoveOutermostParentheses | bool | user | true | Removes outmost parentheses.
Valid values: true, false |
| dbParamGetCurrentRefForInitAndDone | bool | library | false | Allows <code>db::getCurrentRef</code> to return an <code>oaRef</code> object when called in <code>FormInit</code> and <code>FormDone</code> callback procedures.
Valid values: true, false |
| dbParamParseAsPEL | bool | user | false | Evaluates all string parameters using <code>parseAsPEL=true</code> .
Valid values: true, false |
| dbParamStoreDefaultCompatibility | bool | user | false | If value is true then default when creating a CDF parameter the value for <code>storeDefault</code> will be 'no' (not unset).
Valid values: true, false |
| dbPatternMatchType | dbPatternMatchType | user | Exact | Select the pattern type.
Valid values: exact, reg, expr, wildcard |
| dbPcellCacheDefaultPath | string | user | <file path> | Default path to use if there is no cache path set in <code>lib.defs</code> . |
| dbPcellCacheOnByDefault | bool | user | false | Turn on PCell caching by default, using the default cache path.
Valid values: true, false |
| dbPcellCacheSizeCeiling | int | user | 2048 | Maximum size in megabytes for cache file when reached, no new PCells are cached. |
| dbPcellCacheWarnStep | int | user | 128 | Frequency of warnings at these size intervals in megabytes. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---------------------------|------------------|----------|---------------|---|
| dbPcellCacheWarnThreshold | int | user | 1024 | Cache file threshold size in megabytes at which point warnings are generated. |
| dbPcellFreezeEnabled | bool | user | false | When true, enables PCell freezing.
Valid values: true, false |
| dbPDFViewer | string | user | gnome-open | Path to the PDF viewer. |
| dbPlacementGroupSync | bool | user | true | Enables the group synchronization.
Valid values: true, false |
| dbPostscriptViewer | string | cellview | | Path to the postscript viewer. |
| dbPrintBookmarks | bool | cellview | false | Controls if bookmarks should be printed in hierarchy mode.
Valid values: true, false |
| dbPrintBorderRulers | bool | cellview | false | Controls whether to output the ruler.
Valid values: true, false |
| dbPrintBottomMargin | float | cellview | 0.5 | The bottom margin, in inches. |
| dbPrintColor | bool | cellview | true | Controls whether to print in color (vs. grayscale).
Valid values: true, false |
| dbPrintColorMode | dbPrintColorMode | cellview | blackAndWhite | Controls whether to print in color, grayscale or black & white. |
| dbPrintCopies | int | cellview | 1 | The number of copies. |
| dbPrintCurrentViewport | bool | user | true | Control whether to print the current viewport or designs chosen from an LCV selector. |
| dbPrintDepth | int | cellview | 0 | Depth to print. |
| dbPrintDisplay | string | cellview | | The display to use. |
| dbPrintDoubleSided | bool | cellview | false | Prints on both sides of a page.
Valid values: true, false |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--------------------|-----------------|----------|-----------|---|
| dbPrintFilename | string | cellview | | Output filename including path. |
| dbPrintFileType | dbPrintFileType | cellview | ps | Printed file format.
Valid values: ps, pdf, pdfWithLinks |
| dbPrintFilter | bool | user | false | Enables or disables the library/cell/view filtering.
Valid values: true, false |
| dbPrintFilterCell | bool | user | false | Enables or disables the cell list filter.
Valid values: true, false |
| dbPrintFilterCells | string | user | * | List of cell names to filter using glob-style wildcard patterns. |
| dbPrintFilterLib | bool | user | false | Enables or disables the library list filter.
Valid values: true, false |
| dbPrintFilterLibs | string | user | * | List of library names to filter using glob-style wildcard patterns. |
| dbPrintFilterView | bool | user | false | Enables or disables the view list filter.
Valid values: true, false |
| dbPrintFilterViews | string | user | * | List of view names to filter using glob-style wildcard patterns. |
| dbPrintFontScale | float | user | 1 | Additional font scaling factor. |
| dbPrintFontSize | int | user | 10 | The font size for the text generated by the Print configuration options. |
| dbPrintGrid | bool | user | false | Controls whether to output the grid.
Valid values: true, false |
| dbPrintHierMode | dbPrintHierMode | user | inContext | Mode to use when printing the hierarchical data.
Valid values: Any integer between 0 and 2 |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------|--------------------|-------|----------|---|
| dbPrintIncludeBackground | bool | user | true | Controls whether the background is printed.
Valid values: true, false |
| dbPrintInvertBrightness | bool | user | true | Controls whether to darken light colors and brighten dark colors.
Valid values: true, false |
| dbPrintLeftMargin | float | user | 0.5 | The left margin, in inches. |
| dbPrintLegend | dbPrintLegend | user | disabled | Outputted legend location.
Valid values: Any integer between 0 and 6 |
| dbPrintOLPGroup | string | user | | The OLP group to use. |
| dbPrintOmitBackground | bool | user | true | Controls whether the background is printed. |
| dbPrintOrientation | dbPrintOrientation | user | auto | The orientation to use.
Valid values: Any integer between 0 and 3 |
| dbPrintOverwrite | bool | user | false | Controls if outputting to a file should overwrite the file if it exists.
Valid values: true, false |
| dbPrintPaperSize | string | user | | The paper size to use. |
| dbPrintPrinter | string | user | | The name of the default printer to use. |
| dbPrintRightMargin | float | user | 0.5 | The right margin, in inches. |
| dbPrintScaleFactor | float | user | 1 | Controls the scale factor applied to the page. |
| dbPrintScaleFit | dbPrintScaleFit | user | page | Controls how to fit the print to the page dimensions.
Valid values: Any integer between 0 and 3 |
| dbPrintScaleMode | dbPrintScaleMode | user | fit | Whether to scale to fit or by a certain factor.
Valid values: Any integer between 0 and 2 |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|----------------------|----------|------------------|--|
| dbPrintSource | dbPrintSource | cellview | currentViewport | Controls whether to print the current viewport, current design or design(s) chosen from an Library, cell, View selector. |
| dbPrintTitlePosition | dbprintTitlePosition | user | disabled | Whether and where to print the title.
Valid values: Any integer between 0 and 5 |
| dbPrintToFile | bool | user | false | Controls whether to print to a file instead of to printer.
Valid values: true, false |
| dbPrintTopMargin | float | user | 0.5 | The top margin, in inches. |
| dbPrintViewlist | string | user | schematic symbol | The view list for switching through the hierarchy. |
| dbPrintViewSwitchList | string | user | schematic symbol | The view list for switching through the hierarchy. |
| dbProhibitedCharacters | string | user | { } | Contains two sets of characters prohibited for using in objects' names. The first set is applied only to the first symbol in the object name. The second set of characters is applied to all symbols in the name.
The syntax can be:
{{leading character restriction set}
{general restriction set}}
Use
<code>db::isProhibitedName</code> to check validity of a string. |
| dbPropertyEditorAllowChangeReadOnlyCellParameters | bool | user | false | Permits modifications of parameters in the Property Editor even if the design is read-only. The modifications are not saved.
Valid values: true, false |
| dbPropertyEditorAnnotateInherited | bool | user | false | Shows inherited parameters with italic text.
Valid values: true, false |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---|---------------------|-------|------------|--|
| dbPropertyEditorAnnotateModified | bool | user | true | Shows modified entries with bold text.
Valid values: true, false |
| dbPropertyEditorFileBrowsePath | string | user | | The default browse path of file parameters is a list of cell-path pairs: "cellName1 path1 cellName2 path2 ..." which the length must be even.
The pairs have format with wildcard:
1) cellName path
(for example: nport remote/files)
2) libName/cellName path (for example: analogLib/nport remote/files)
3) */cellName path (for all cells with cellName name)
4) libName/* path (for whole libName library)
5) */* path (for all cells)
The browse-path of file is chosen from list at the first occurrence, all other pairs are ignored. |
| dbPropertyEditorLPPFilterMethod | <enLPPFilterMethod> | user | selectable | Determines which LPPs are shown when LPP widget (deLPPInput) is made active.
Valid values: selectable, visible, valid, All |
| dbPropertyEditorResetCDFsToDefaultsOnRemaster | bool | user | false | Determines if all the CDFs are reset to defaults on remastering.
Valid values: true, false |
| dbPropertyEditorSubMasterEvalCallbackAlways | bool | user | true | Determines if parameter callback is triggered for each instance of subMaster (true) or 1 time per subMaster (false).
Valid values: true, false |
| dbPropertyEditorTextHeightFormat | dbTextHeightFormat | user | uu | The text font height format. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------------|--------|-------|---|--|
| dbPropertyEditorUnlimitedCycle | bool | user | false | Property Editor previous/next object switch change behavior.
Valid values: true, false |
| dbPropParseAsPEL | bool | user | true | Evaluate all string properties using parseAsPEL=true.
Valid values: true, false |
| dbQueryVerboseMode | bool | user | false | Verbose mode.
Valid values: true, false |
| dbRegexp | bool | user | false | Regexp mode for interactive commands.
Valid values: true, false |
| dbRemasterKeepPropsList | string | user | nlAction
lxIgnore
lxUseCell
lvslgnore
lxRemovedD
evice | Space delimited string of property names to keep when dbRemasterRemoveProps is set to true. |
| dbRemasterRemoveProps | bool | user | false | If set to true, removes all properties that are not associated with CDF parameters when remastering.
Valid values: true, false |
| dbReplaceAllMatches | bool | user | false | Replaces all matches found in each string. If disabled only the first match in each string is replaced.
Valid values: true, false |
| dbSaveHistoryAsFilename | string | user | ./history.tcl | The default value for the File name field in the Save History As dialog box. |
| dbSavePrefsOnlyModified | bool | user | false | Tied to the Save only modified preferences option in the Save Preferences dialog box.
Valid values: true, false |
| dbSchGenBitFormat | string | user | [\\d] | Specifies if a new engine is used for generating schematics from digitals. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------------|-------------------------------|-------|-------------|---|
| dbSchGenByNameCaseSensitive | bool | user | true | Case sensitivity for Connect By Name.
Valid values: true, false |
| dbSchGenByNameNetsPattern | dbSchGen
ByNamePa
ttern | user | wildcard | Pattern syntax for Connect By Name. |
| dbSchGenConnectByName | dbNameEn
ums | user | selected | Nets by name connection mode.
Valid values: none, selected, all |
| dbSchGenDestCell | string | user | | The name of the destination cell. Used only for single schematic view generation. |
| dbSchGenDestLibrary | string | user | | The name of the destination library. |
| dbSchGenDestView | string | user | schematic | The name of the created schematic view. |
| dbSchGenEnableParamCallbacks | bool | user | false | Indicates whether to trigger parameter callbacks during schematic generation.
Valid values: true, false |
| dbSchGenFromTextCreatePortOrder | bool | user | false | Indicates whether or not the portOrder property should be created during schematic generation from text.
Valid values: true, false |
| dbSchGenFromTextDefines | string | user | | Space-separated list of define statements to be considered as pre-declared when generating schematics from verilog files. |
| dbSchGenFromTextDeviceMapFile | string | user | <file path> | Device mapping file to be used during schematic generation from SPICE and CDL files. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|-------------------|-------|---|---|
| dbSchGenFromTextExcludeModuleMatchType | enMatchType | user | exact | The module matching type to filter the module names using the <code>dbSchGenFromTextExcludeModules</code> preference. Valid values: exact regexp wildcard |
| dbSchGenFromTextExcludeModules | string | user | | The names of the modules for which schematics should not be generated. |
| dbSchGenFromTextImportModules | string | user | | The names of the modules for which schematics should be generated. |
| dbSchGenFromTextIncludesProcessingSPICE | string | user | parse
{ignore
"*.dspf *.spf
*.gz *.gzip
*.enc *.lib
*.vec *.ic
*.data"
wildcard} | Include file processing types to use during SPICE/CDL file importing. |
| dbSchGenFromTextIncludesProcessingVerilog | string | user | parse | Include file processing types to use during Verilog file importing. |
| dbSchGenFromTextLang | enNetlistLanguage | user | Verilog | Sets the language of the imported files. Valid values: SPICE CDL Verilog |
| dbSchGenFromTextLibName | string | user | | Output library name for generated schematics. |
| dbSchGenFromTextModuleMatchType | enMatchType | user | exact | The module matching type to filter the module names using the <code>dbSchGenFromTextImportModules</code> preference. Valid values: exact regexp wildcard |
| dbSchGenFromTextRefLibs | string | user | | Space-separated list of library names referenced cells to be searched in. |
| dbSchGenGenerateSymbols | string | user | true | Indicates whether to generate symbols during schematic generation. Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------|---------------------|-------|-------------|---|
| dbSchGenInstanceAlignment | instAlignEnums | user | left | Alignment of symbols in each column ensuring schematic generation.
Valid values: left, right, center |
| dbSchGenMergePortSubBits | dbSchGenMergeBits | user | off | Merges port bits to buses.
Valid values: asc, desc, off |
| dbSchGenMergeVectorBitInsts | dbSchGenMergeBits | user | off | Merges matching vector bit instances to vector instance.
Valid values: asc, desc, off |
| dbSchGenMinHorizontalSpacing | int | user | 2 | Minimum horizontal spacing during schematic generation.
Valid values: Any integer between 1 and 1000 |
| dbSchGenMinVerticalSpacing | int | user | 2 | Minimum vertical spacing during schematic generation.
Valid values: Any integer between 1 and 1000 |
| dbSchGenNetNames | string | user | | Net names to use for connection. |
| dbSchGenOverwrite | <enOverwrite> | user | never | Specifies if existing designs should be overwritten during schematic generation.
Valid values: always, never |
| dbSchGenOverwriteSymbols | bool | user | false | Specifies if existing symbols should be overwritten if generating symbols.
Valid values: true, false |
| dbSchGenParamCallbacks | dbEvaluateParamType | user | editable | Specifies which instance parameter callbacks should be evaluated during schematic generation. |
| dbSchGenPatchCord | lcw | user | <file path> | The cellview to be used as patchcords for schematic generation. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---------------------------------|-------------------------|-------|------------|--|
| dbSchGenRewireCrossingWires | bool | user | true | If set, cross wire connection in the generated schematic is rewired a bit to avoid solder dots on cross wire connections.
Valid values: true, false |
| dbSchGenSourceCells | string | user | | The name of the cell (or a wildcard style pattern for cell name matching) for which schematic should be generated. |
| dbSchGenSourceLibrary | string | user | | The name of the library for the Verilog views for which schematic should be generated. |
| dbSchGenSourceViewList | string | user | verilog | The name of the Verilog views for which schematic should be generated. |
| dbSchGenSymbolOptionSet | string | user | default | Symbol generator option set name to use for generating symbols. |
| dbSchGenTopCellName | string | user | | The cell name to put the top level spice instances in. |
| dbSchGenUnroutableConnections | enUnroutableConnections | user | byName | Connection type for unroutable nets.
Valid values: flight line byName |
| dbSchGenUseNewEngineForDigitals | bool | user | true | Specifies if a new engine is used for generating schematics from digitals.
Valid values: true, false |
| dbSchGenVirtualTermsCreate | dbSchGenCreateVirtuals | user | asVirtuals | Specifies how schematic generation creates virtual terminals (terms with connection definitions) in the target schematic. |
| dbShowAllProps | bool | user | false | Shows all properties.
Valid values: true, false |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------|----------|-----------|---|
| dbShowParamRowColors | bool | user | true | Shows alternating row colors in the Parameters table within the Property Editor, Create Instance and elsewhere.
Valid values: true, false |
| dbSigTypeGroundFilter | string | cellview | vss* gnd* | New nets whose name matches the glob style case-insensitive expression are implicitly set as ground nets. |
| dbSigTypePowerFilter | string | cellview | vdd* | New nets whose name matches the glob style case-insensitive expression are implicitly set as power nets. |
| dbSyncTermNetNames | bool | user | false | Indicates whether the net name should be synchronized along with the term name.
Valid values: true, false |
| dbTechnologyImportMultiUserCheck | bool | user | false | Enables multi-user check.
Valid values: true, false |
| dbTechnologyManagerCloseDesign | bool | user | true | Enable close design on technology changes.
Valid values: true, false |
| dbTechnologyManagerExportIncludeEmptySections | bool | user | false | Specifies that export should be done only for local level of technology data, plus techArray.
Valid values: true, false |
| dbTechnologyManagerExportLocal | bool | user | true | Specifies that export should be done for empty sections with comments syntax.
Valid values: true, false |
| dbTechnologyManagerExportSections | string | user | | Exports the listed sections and its sub-sections of technology changes |
| dbTechnologyManagerFileName | string | user | | Import Technology file name. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-----------------------------------|------------------|----------|-------------|---|
| dbTechnologyManagerImportMode | leTechImportMode | user | overwrite | Import Technology mode.
Valid values: merge, replace, overwrite |
| dbTechnologyManagerImportSections | string | user | | Import sections.
Valid values: all controls, layerDefinitions, layerRules, siteDefs, viaDefs, constraintGroups, techArray, techParams, mfgGridResolution, techPermissions, viewTypeUnits, leMPPControls, colors, techPurposes, techLayers, techLayerPurposePriorities, techDisplays, techLayerProperties, techDerivedLayers, functions, mfgResolutions, routingDirections, currentDensities, viaLayers, cutClasses, trackDefs, trackPatternDefs, scalarSiteDefs, arraySiteDefs, standardViaDefs, standardViaVariants, customViaDefs, customViaVariants, bridgeVias, colorClasses, wideAlignments |
| dbTemplateMergeSchematic | bool | user | true | Merges schematic netlist.
Valid values: true, false |
| dbTimeDisplayFormat | string | user | hh:mm:ss | Controls the display format of time. The full list of time abbreviation. |
| dbUpdateCellViewPortOrderProperty | bool | user | false | Updates port order properties to the cellview.
Valid values: true, false |
| dbUpdateCellViewSource | lcv | cellview | <file path> | Indicates the default LCV for the cellview source. |
| dbUseFormatForNumerals | bool | user | true | Use format for numerals.
Valid values: true, false |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---------------------------------------|--------|-------|--|--|
| dbValidateForICCBeforeExport | bool | user | false | Indicates whether the design should be validated before export.
Valid values: true, false |
| dbVdrcMgrAutoCreateDRCNetPairsToTable | bool | user | true | Auto creates DRC Net Pairs to Table in VDRC Manager |
| dbVdrcMgrHierarchyMismatchEnable | bool | user | true | Enable of All Hierarchy Mismatch Report in VDRC Manager. |
| dbVdrcMgrHierarchyMode | bool | user | false | Show VDRC Nets information for all schematic hierarchies. |
| dbVdrcMgrHierMismatchEnable | string | user | NetVDiff {1 0 0}
NetVMax {1 1 1}
NetVMin {1 1 1}
NetPairEdv Diff {1 0 0}
NetPairEdv Udv {0 0 1}
NetPairUdv Diff {1 1 1}
NetPairUdv Edv {1 0 0} | Enable options for Hierarchy Mismatch Report in VDRC Manager. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-------------------------------------|--------------------|-------|---|--|
| dbVdrcMgrHierMismatchSeverity | string | user | NetVDiff {error info} NetVMax {warning warning} NetVMin {warning warning} NetPairEdv Diff {error info info} NetPairEdv Udv {info info warning} NetPairUdv Diff {error warning warning} NetPairUdv Edv {error info info} | Severity Options for Hierarchy Mismatch Report in VDRC Manager. |
| dbVdrcMgrLayoutDiffMasterEnable | bool | user | true | Enable of Layout Different Master Symbol Report in VDRC Manager. |
| dbVdrcMgrLayoutDiffMasterSeverity | dbMismatchSeverity | user | warning | Severity of Layout Different Master Symbol Report in VDRC Manager |
| dbVdrcMgrLayoutDiffPairTypeSeverity | mismatch severity | user | info | Severity of Layout Different Net Pair Type Symbol Report in VDRC Manager |
| dbVdrcMgrLayoutDVMismatchEnable | bool | user | true | Enable of Layout DeltaV Value Mismatch Report in VDRC Manager |
| dbVdrcMgrLayoutDVMismatchSeverity | dbMismatchSeverity | user | error | Severity of Layout DeltaV Value Mismatch Report in VDRC Manager |
| dbVdrcMgrLayoutMismatchEnable | bool | user | true | Enable of All Layout Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrLayoutMtiMismatchEnable | bool | user | true | Enable of All Layout Multiple Symbol Mismatch Report in VDRC Manager. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--|--------------------|-------|---------|--|
| dbVdrcMgrLayoutMultiSameMasterEnable | bool | user | true | Enable of Layout Multiple Same Master Symbol Report in VDRC Manager. |
| dbVdrcMgrLayoutMultiSameMasterSeverity | dbMismatchSeverity | user | warning | Severity of Layout Multiple Same Master Symbol Report in VDRC Manager |
| dbVdrcMgrLayoutVMaxMismatchEnable | bool | user | true | Enable of Layout VMax Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrLayoutVMaxMismatchSeverity | dbMismatchSeverity | user | error | Severity of Layout VMax Value Mismatch Report in VDRC Manager |
| dbVdrcMgrLayoutVMinMismatchEnable | bool | user | true | Enable of Layout VMin Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrLayoutVMinMismatchSeverity | dbMismatchSeverity | user | error | Severity of Layout VMin Value Mismatch Report in VDRC Manager |
| dbVdrcMgrSchDiffMasterEnable | bool | user | true | Enable of Schematic Different Master Symbol Report in VDRC Manager. |
| dbVdrcMgrSchDiffMasterSeverity | dbMismatchSeverity | user | warning | Severity of Schematic Different Master Symbol Report in VDRC Manager |
| dbVdrcMgrSchDiffPairTypeSeverity | mismatch severity | user | info | Severity of Schematic Different Net Pair Type Symbol Report in VDRC Manager |
| dbVdrcMgrSchDVMismatchEnable | bool | user | true | Enable of Schematic DeltaV Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchDVMismatchSeverity | dbMismatchSeverity | user | error | Severity of Schematic DeltaV Value Mismatch Report in VDRC Manager |
| dbVdrcMgrSchLayoutDVMismatchEnable | bool | user | true | Enable of Schematic and Layout DeltaV Value Mismatch Report in VDRC Manager. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--|--------------------|-------|---------|--|
| dbVdrcMgrSchLayoutDVMismatchSeverity | dbMismatchSeverity | user | error | Severity of Schematic and Layout DeltaV Value Mismatch Report in VDRC Manager |
| dbVdrcMgrSchLayoutMismatchEnable | bool | user | true | Enable of All Schematic and Layout Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchLayoutVMaxMismatchEnable | bool | user | true | Enable of Schematic and Layout VMax Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchLayoutVMaxMismatchSeverity | dbMismatchSeverity | user | error | Severity of Schematic and Layout VMax Value Mismatch Report in VDRC Manager |
| dbVdrcMgrSchLayoutVMinMismatchEnable | bool | user | true | Enable of Schematic and Layout VMin Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchLayoutVMinMismatchSeverity | dbMismatchSeverity | user | error | Severity of Schematic and Layout VMin Value Mismatch Report in VDRC Manager |
| dbVdrcMgrSchMismatchEnable | bool | user | true | Enable of All Schematic Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchMtiMismatchEnable | bool | user | true | Enable of All Schematic Multiple Symbol Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchMultiSameMasterEnable | bool | user | true | Enable of Schematic Multiple Same Master Symbol Report in VDRC Manager. |
| dbVdrcMgrSchMultiSameMasterSeverity | dbMismatchSeverity | user | warning | Severity of Schematic Multiple Same Master Symbol Report in VDRC Manager |
| dbVdrcMgrSchSimDVMismatchEnable | bool | user | true | Enable of Schematic and Simulation DeltaV Value Mismatch Report in VDRC Manager. |

Table 56 *Database Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-------------------------------------|--------------------|----------|---------|--|
| dbVdrcMgrSchSimDVMismatchSeverity | dbMismatchSeverity | user | error | Severity of Schematic and Simulation DeltaV Value Mismatch Report in VDRC Manager |
| dbVdrcMgrSchSimMismatchEnable | bool | user | true | Enable of All Schematic and Simulation Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchSimVMaxMismatchEnable | bool | user | true | Enable of Schematic and Simulation VMax Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchSimVMaxMismatchSeverity | dbMismatchSeverity | user | error | Severity of Schematic and Simulation VMax Value Mismatch Report in VDRC Manager |
| dbVdrcMgrSchSimVMinMismatchEnable | bool | user | true | Enable of Schematic and Simulation VMin Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchSimVMinMismatchSeverity | dbMismatchSeverity | user | error | Severity of Schematic and Simulation VMin Value Mismatch Report in VDRC Manager |
| dbVdrcMgrSchVMaxMismatchEnable | bool | user | true | Enable of Schematic VMax Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchVMaxMismatchSeverity | dbMismatchSeverity | user | error | Severity of Schematic VMax Value Mismatch Report in VDRC Manager |
| dbVdrcMgrSchVMinMismatchEnable | bool | user | true | Enable of Schematic VMin Value Mismatch Report in VDRC Manager. |
| dbVdrcMgrSchVMinMismatchSeverity | dbMismatchSeverity | user | error | Severity of Schematic VMin Value Mismatch Report in VDRC Manager |
| dbVdrcMgrValuePrecision | int | user | 3 | VDRC Manager Value Display Precision |
| dbVerilogGenOptionSetDefault | string | cellview | default | Default name of the option set to use when using the built-in generateVerilog generator. |

Table 56 Database Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------------------|---------|-------|---------|--|
| dbViewErrors | bool | user | true | Controls the display of error messages in the Console window.
Valid values: true, false |
| dbViewInfo | bool | user | true | Controls the display of informational messages in the Console window.
Valid values: true, false |
| dbViewWarnings | bool | user | true | Controls the display of warning messages in the Console window.
Valid values: true, false |
| dbViewWordWrap | bool | user | false | Controls the wrapping behavior of the Console's output panes.
Valid values: true, false |
| dbVisibleToTopModule | bool | user | false | New designs synchronize block and module domains.
Valid values: true, false |
| dbUpdateCellViewKeepPinPositionEnable | boolean | user | false | Controls the Keep Updated Pins Position option in Update Cellview dialog box. |
| dbExportOasisPropNumForVia | string | user | | Streams the property number used to store the via. |
| dbExportStreamPropNumForVia | string | user | | Streams the property number used to store the via. |

7

Design Editing Tcl

This chapter lists commands, enumerations, objects, and preferences in the Custom Compiler Tcl de namespace. You use the de Tcl for layout, schematic, and symbol editing, and design navigation. Interactive commands are listed after non-interactive commands in the Commands section.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

Interactive commands are listed after non-interactive commands in this section.

The commands covered are:

- [de::abortCommand](#)
- [de::activateTools](#)
- [de::addCapability](#)
- [de::addObject](#)
- [de::addPoint](#)
- [de::addTemplateView](#)
- [de::addTool](#)
- [de::applyOLPPreset](#)
- [de::audit](#)
- [de::cancelCommand](#)

- [de::cancelTransaction](#)
- [de::clearHighlights](#)
- [de::clearUnusedDesigns](#)
- [de::clone](#)
- [de::close](#)
- [de::commandOption](#)
- [de::completeShape](#)
- [de::connect](#)
- [de::connectToServer](#)
- [de::copy](#)
- [de::copyBookmark](#)
- [de::copyClipboardToLib](#)
- [de::createActivationEngine](#)
- [de::createApplication](#)
- [de::createArgument](#)
- [de::createArgumentGroup](#)
- [de::createAssociation](#)
- [de::createBookmark](#)
- [de::createCanvas](#)
- [de::createColorExpression](#)
- [de::createCommand](#)
- [de::createContext](#)
- [de::createCrossProbeTool](#)
- [de::createErrorViewerDatabase](#)
- [de::createErrorViewerMarker](#)
- [de::createFavoriteDevice](#)
- [de::createFavoriteDeviceCategory](#)

- [de::createGradient](#)
- [de::createHierObject](#)
- [de::createHighlightSelector](#)
- [de::createHighlightSet](#)
- [de::createHomePage](#)
- [de::createInterfaceElement](#)
- [de::createLPPInput](#)
- [de::createLPPSet](#)
- [de::createNetHighlightSet](#)
- [de::createObjectFilter](#)
- [de::createOLPPreset](#)
- [de::createOnCanvasWidget](#)
- [de::createOverlay](#)
- [de::createProbe](#)
- [de::createSelectionEngine](#)
- [de::createShapeEngine](#)
- [de::createTemplate](#)
- [de::createTool](#)
- [de::crossCheckViews](#)
- [de::cut](#)
- [de::cycleActiveFigure](#)
- [de::cycleTapObject](#)
- [de::deactivateTools](#)
- [de::defineOrigin](#)
- [de::deleteBookmarks](#)
- [de::deprecateCommand](#)
- [de::descend](#)

- [de::descendInPlace](#)
- [de::descendIntoGroup](#)
- [de::deselect](#)
- [de::deselectAll](#)
- [de::deselectAllLogic](#)
- [de::deselectLogic](#)
- [de::disconnect](#)
- [de::enableColorAnnotator](#)
- [de::enablePartitioningDisplay](#)
- [de::enableRotationAndScaling](#)
- [de::endDrag](#)
- [de::endLog](#)
- [de::endTransaction](#)
- [de::errorViewerLoadResult](#)
- [de::exportBookmarks](#)
- [de::exportProbeCellSettings](#)
- [de::extractInterfaceElements](#)
- [de::findMessage](#)
- [de::finishEvalTextRendering](#)
- [de::fit](#)
- [de::getActiveCommand](#)
- [de::getActiveContext](#)
- [de::getActiveEditorWindow](#)
- [de::getActiveFigure](#)
- [de::getActiveLPP](#)
- [de::getApplications](#)
- [de::getAssociations](#)

- [de::getBackgroundImporters](#)
- [de::getBBox](#)
- [de::getBinsForGradient](#)
- [de::getBookmarks](#)
- [de::getBreakpoints](#)
- [de::getCapabilities](#)
- [de::getClipboardLibs](#)
- [de::getClips](#)
- [de::getColorAnnotators](#)
- [de::getCommands](#)
- [de::getContexts](#)
- [de::getCrossProbeTools](#)
- [de::getCursor](#)
- [de::getCursorSnapPoint](#)
- [de::getDBLimits](#)
- [de::getDesignTemplates](#)
- [de::getEditors](#)
- [de::getErrorViewerDatabases](#)
- [de::getFavoriteDeviceCategories](#)
- [de::getFavoriteDevices](#)
- [de::getFigures](#)
- [de::getHighlighted](#)
- [de::getHighlightSets](#)
- [de::getInterfaceElements](#)
- [de::getLayers](#)
- [de::getLookaheadMessages](#)
- [de::getLPPs](#)

- [de::getLPPSets](#)
- [de::getMergedBBox](#)
- [de::getObjectFilters](#)
- [de::getOLPPresets](#)
- [de::getOpenFiles](#)
- [de::getOutdatedDesigns](#)
- [de::getOutdatedDMData](#)
- [de::getOverlays](#)
- [de::getPartitioner](#)
- [de::getProbeBitNames](#)
- [de::getProbeFigures](#)
- [de::getProbeNets](#)
- [de::getProbes](#)
- [de::getRecentFiles](#)
- [de::getSelected](#)
- [de::getSelectedLogic](#)
- [de::getSelectedVertex](#)
- [de::getTemplates](#)
- [de::getTools](#)
- [de::getTransactions](#)
- [de::getViewport](#)
- [de::gotoViewport](#)
- [de::help](#)
- [de::highlight](#)
- [de::importBookmarks](#)
- [de::importProbeCellSettings](#)
- [de::isEditable](#)

- [de::isHidden](#)
- [de::isInUndo](#)
- [de::isModified](#)
- [de::linkContext](#)
- [de::loadFavoriteDevices](#)
- [de::loadInterfaceElements](#)
- [de::loadOLPPresets](#)
- [de::loadProbes](#)
- [de::navigateTo](#)
- [de::next](#)
- [de::open](#)
- [de::open3DICDesign](#)
- [de::openCanvasPropertyEditor](#)
- [de::openClipboardLib](#)
- [de::openEditor](#)
- [de::openFCDesign](#)
- [de::openFile](#)
- [de::openICC2Design](#)
- [de::pan](#)
- [de::paste](#)
- [de::redoSelection](#)
- [de::redoTransaction](#)
- [de::redraw](#)
- [de::registerHelp](#)
- [de::removeBreakpoints](#)
- [de::removeObject](#)
- [de::removePoint](#)

- [de::removeTool](#)
- [de::renumberInstances](#)
- [de::repeatCommand](#)
- [de::replay](#)
- [de::replayCommand](#)
- [de::reportOpenTransactions](#)
- [de::restoreBookmark](#)
- [de::return](#)
- [de::revert](#)
- [de::run](#)
- [de::runBackgroundImporter](#)
- [de::save](#)
- [de::saveAs](#)
- [de::saveFavoriteDevices](#)
- [de::saveInterfaceElements](#)
- [de::saveOLPPresets](#)
- [de::saveProbes](#)
- [de::select](#)
- [de::selectAll](#)
- [de::selectLogic](#)
- [de::send](#)
- [de::sendComplete](#)
- [de::sendMessage](#)
- [de::setActiveLPP](#)
- [de::setBreakpoint](#)
- [de::setCursor](#)
- [de::setEditMode](#)

- [de::setErrorViewerCategory](#)
- [de::setErrorViewerDetailInfo](#)
- [de::setErrorViewerFieldName](#)
- [de::setErrorViewerTip](#)
- [de::setViewport](#)
- [de::shiftCursor](#)
- [de::showAbout](#)
- [de::showBookmarkManager](#)
- [de::showCloseDesigns](#)
- [de::showConnectionsNavigator](#)
- [de::showConnectionsNavigatorOptions](#)
- [de::showCreateBookmark](#)
- [de::showCrossCheckOptions](#)
- [de::showDescendDialog](#)
- [de::showDesignsPrompt](#)
- [de::showGeneralOptions](#)
- [de::showGradientEditor](#)
- [de::showMakeDesignsReadOnly](#)
- [de::showOpen3DICDesign](#)
- [de::showOpenDesign](#)
- [de::showOpenFCDesign](#)
- [de::showOpenICC2Design](#)
- [de::showParamFilter](#)
- [de::showProbeBitSelectionDialog](#)
- [de::showProperties](#)
- [de::showRefreshDesigns](#)
- [de::showRenumberInstances](#)

- [de::showReturnToLevelDialog](#)
- [de::showSaveDesignAs](#)
- [de::sleep](#)
- [de::snapToObject](#)
- [de::startDrag](#)
- [de::startLog](#)
- [de::startRendererStatistics](#)
- [de::startTransaction](#)
- [de::stopBackgroundImporter](#)
- [de::stopRendererStatistics](#)
- [de::suppressMessage](#)
- [de::transformPoint](#)
- [de::undoSelection](#)
- [de::undoTransaction](#)
- [de::unhighlight](#)
- [de::uniquify](#)
- [de::zoom](#)
- [ide::createProbe](#)
- [ide::defineOrigin](#)
- [ide::descend](#)
- [ide::editCanvasText](#)
- [ide::enterBox](#)
- [ide::enterLine](#)
- [ide::enterPoint](#)
- [ide::enterPolygon](#)
- [ide::highlight](#)
- [ide::pan](#)

- [ide::pick](#)
- [ide::selectByName](#)
- [ide::selectByRegion](#)
- [ide::setViewport](#)
- [ide::testEvents](#)
- [ide::testStack](#)
- [ide::zoom](#)

de::abortCommand

Description

Stops the active interactive command in the design editor window.

Syntax

```
de::abortCommand [<name: string>] [-window <giWindow | unsigned int> |  
-context <deContext>]
```

Arguments

name

The name of the command to cancel.

-window

Window in which to send the event.

-context

Context in which to send the event.

Examples

```
# Start an interactive command ise::copy# Abort the command  
de::abortCommand
```

de::activateTools

Description

Activates one or more tools within the specified context. If already activated, this command will not alter its state.

Syntax

```
de::activateTools <tools: dbCollection> -context <deContext>
```

Arguments

`tools`

Collection of deTool objects to activate in the specified context.

`-context`

Context in which to activate tools.

Examples

```
# Get parasitics tool
set context [de::getActiveContext]
set tools [db::getattr c.editor.tools]
set tool [db::getNext [db::filter $tools -filter
    {%name=="xtParasiticsTool"}]]
# Deactivate parasitics tool in schematic context
de::deactivateTools $tool
```

de::addCapability

Description

Adds `deCapability` to `deLinkedContext` to enable this capability in its primary design context.

Syntax

```
de::addCapability <object: deCapability | string> -to <deLinkedContext>
[-enabled <bool>]
```

Returns

<deCapability>: Newly added capability.

Arguments

`object`

Capability to add.

`-to`

Linked context to modify.

`-enabled`

Enables the attribute by default. Default: true

Examples

```
# Create two contexts to link one another
set context1 [de::createContext parasitics/r_ground/schematic]
set context2 [de::createContext parasitics/r_ground/symbol]

# Open main context
de::openEditor -context $context1
# Link context2 to context1
set lc [de::linkContext $context2 -to $context1 -name linkedContext
      -title "Linked Context"]

# Add split view capability linked context and open it in split editor
de::addCapability "deSplitView" -to $lc
db::setAttr context1.editor.splitContext -value $lc

# Change linked context orientation
db::setAttr context1.editor.splitOrientation -value horizontal

# Fit both contexts
de::fit -window [db::getAttr context1.window]
de::fit -window [gi::findChild linkedContext -in [db::getAttr
      centralWidget
      -of [db::getAttr context1.window]]]
```

de::addObject

Description

Selects the specified object and adds it to the command working set. If the object is unsupported, this command will throw an error.

Syntax

```
de::addObject <object: deFigure> [-window <giWindow | unsigned int> |
      -context <deContext>]
```

Arguments

object

Object to select.

-window

Window in which to send the event.

-context

Context in which to send the event.

de::addPoint

Description

Moves the cursor to the given point and sends an addPoint event to the active command in the design editor window. This event is equivalent to clicking on the canvas for data input for many interactive commands.

The point argument is in the view the designs coordinate space.

Syntax

```
de::addPoint <point: point> [-window <giWindow | unsigned int> | -context  
    <deContext>] [-relative <bool>] [-snap <bool>]
```

Arguments

point

Point to add, in user units.

-window

Window in which to send the event.

-context

Context in which to send the event.

-relative

Specifies that this point is relative to the last entered point. Default: false

-snap

Specifies that this point will be snapped to angle mode and gravity. Default: true

Examples

The following example moves a rectangle by 0.1 in the x direction relative to its initial position.

```
# Create a rectangle in the layout design
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"

# Start interactive move command
ile::move

# Use de::addPoint to pick the rectangle
de::addPoint {0.1 0}

# Use de::addPoint to place the rectangle using relative point
de::addPoint {0.1 0} -relative true
```

de::addTemplateView

Description

Adds a new view to an existing template cell.

Syntax

```
de::addTemplateView <viewName: string> (-view <dmCellView> | -data  
    <dbCollection  
    | {<deFigure>} | {<oaFig>}>) -template <deTemplate> [-sourceNetlist  
    <dbCollection  
    | {<oaFig>}>] [-staticClone <bool>]
```

Returns

<none>

Arguments

viewName

Name of new view in the template cell.

-template

Template in which to add new view.

-view

Cellview to copy into the template.

-data

Data to copy into the new template view.

-sourceNetlist

Defines circuit structure of source.

-staticClone

If adding a maskLayout view, marks the new view for use as a static clone.

Default: false

Examples

```
# Create template from selection in design schematic
de::createTemplate invTemplate -libName DesignLibrary -netlist
[de::getSelected -design [ed]]
    -description "Inverter template"

# Add layout view to the template
de::addTemplateView layout -view [dm::findCellView layout -cellName inv3
    -libName DesignLibrary]
```

```
-template [de::getTemplates invTemplate -libName DesignLibrary]  
-staticClone true
```

de::addTool

Description

Adds a tool to design editors and makes the tool valid for designs opened with the given editors.

Syntax

```
de::addTool <tool: deTool> (-editor <deEditor | string> | -workspace  
    <giWorkspace>)
```

Arguments

deTool

Tool to add to the given association.

-editor

Editor for which this tool is available.

-workspace

Workspace to be accessed by the tool.

Examples

```
# Create init proc for new tool  
proc init {context} {  
    set action {dbShowImportVerilog}  
    set menu [gi::createMenu data_menu -title DataIO]  
    gi::addActions $action -to $menu  
}  
# Create decorate proc for new tool  
proc decorate {context} {  
    set win [db::getattr window -of $context]  
    set menu [gi::getMenus data_menu]  
    set hm [db::getNext [gi::getMenus -from $win -filter {%title == "Help"}]]  
    gi::addMenu $menu -to $win -before $hm  
}  
# Create done proc for new tool  
proc done {context} {  
    set win [db::getattr window -of $context]  
    set menu [gi::getMenus -from $win -filter {%name=="data_menu"}]  
    gi::remove $menu  
}  
# Create new tool  
set tool [de::createTool dataTool -title "DataIO" -initProc init  
    -decorateProc decorate -doneProc done]
```

```
# Add new tool to Schematic Editor
de::addTool $tool -editor seSchematic
gi::addActions dataTool -to [gi::getMenus seSchToolsMenu]
```

de::applyOLPPreset

Description

Applies the given deOLPPreset to the given design.

Syntax

```
de::applyOLPPreset <title or index: string> [-operation <enum>] -to
<deContext | oaDesign>
```

Returns

<none>

Arguments

title or index

Object Layer Panel (OLP) preset title or index.

-operation

OLP presets operation: replace, or, not. ({ not | or | replace }. Default: replace

-to

Design or context which to apply settings from the given deOLPPreset.

Examples

```
# Create OLP preset with name newPreset
de::createOLPPreset newPreset -from [ed]
# Make some changes in OLP, to see the changes after applying preset
# Apply previous OLP state
de::applyOLPPreset newPreset -to [ed]
```

de::audit

Returns a formatted report of all uses of deprecated interface elements in the given file or in all files in the startup paths of the Custom Compiler tool.

Syntax

```
de::audit [-filePath <string>] [-auditCommands [<string>]]
```

Returns

<string> : Deprecated use report

Arguments

`-filePath`

Script file to analyze.

`-auditCommands`

Warnings are generated for each call to a command in the audit commands list.

de::cancelCommand

Description

Cancels the current interactive edit in a design editor window and returns the command to the start state without cancelling the command.

Syntax

```
de::cancelCommand [-window <giWindow | unsigned int> | -context  
                  <deContext>]
```

Arguments

`-window`

Window in which to send the event.

`-context`

Context in which to send the event.

Examples

```
# Start an interactive command ise::copy# Select an object for copy from  
{0 0} pointde::addPoint {0 0}# Cancel the command  
# The runtime object will be deleted but the command is still active  
de::cancelCommand
```

de::cancelTransaction

Description

Terminates the transaction.

The command undoes all operations that were already performed as part of the transaction and pops the transaction undo entry off the stack.

Syntax

```
de::cancelTransaction <transaction: deTransaction>
```

Arguments

`deTransaction`

Transaction to cancel.

Examples

```
#Example 1
# Start transaction
set design [ed]
set tr [de::startTransaction "Create Rectangles" -design $design]

# Print information about open transactions
de::reportOpenTransactions -design $design

# Cancel transaction and print information about open transactions
de::cancelTransaction $tr
de::reportOpenTransactions -design $design

#Example 2
# Start transaction in read-only design, it will correctly set design
[ed]
de::setEditMode $design -editable 0
set tr [de::startTransaction "Create Rectangles" -design $design]
try {
    le::createRectangle {{1 1} {3 3}} -design $design design -lpp {m1
drawing}
} on error {result options} {
    de::cancelTransaction $tr
} on ok {} {
    de::endTransaction $tr
}
```

de::clearHighlights

Description

Purges all highlighted objects from a highlight, or from all highlight sets that contain objects in the specified design. Note that this may modify sets bound to parent designs which contain objects with hierarchy from the given design.

Syntax

```
de::clearHighlights (-sets <{<deHighlightSet>}> | -design <oaDesign>
| -context <deContext>) [-clearPersistent <bool>]
```

Arguments

`-sets`

Highlight sets to clear.

`-design`

Design for which all highlights will be cleared.

`-context`

Context whose highlight sets will be cleared.

`-clearPersistent`

Clears persistent highlight sets. Default: false

Examples

```
# Create two rectangles in the layout design
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"
le::createRectangle {{0.3 -0.1} {0.5 0.1}} -design [ed] -lpp "m1 drawing"

# Highlight both rectangles
# They will be added to "highlight1" set
de::highlight [de::getFigures {{0.1 0}}] -design [ed]
de::highlight [de::getFigures {{0.4 0}}] -design [ed]

# Clear highlights in "highlight1" set
de::clearHighlights -sets [de::getHighlightSets highlight1] -design [ed]
```

de::clearUnusedDesigns

Description

Removes unused designs from memory.

Syntax

```
de::clearUnusedDesigns
```

de::clone

Description

Clones a Design Editor context and optionally clones the editor (if headless is false).

Syntax

```
de::clone <context: deContext> [-headless <bool>]
```

Returns

Newly created context (deContext)

Arguments

`deContext`

Context object to clone.

`-headless <bool>`

Specifies that the new context will be headless (no window).

Default: false

Examples

```
# Clone active context
de::clone [de::getActiveContext]
```

de::close

Description

Closes all open files in the context, discarding changes and destroys the context object.

If `-preserveWindow` is true, any window associated with the context is preserved such that it can be reused with the `de::setContext` command, for example. The window will be undecorated until reused.

Syntax

```
de::close <contexts: dbCollection>
```

Arguments

`contexts`

Collection of context objects whose open designs will be closed.

Examples

In the following example, `de::close` closes all opened designs.

```
de::close [de::getContexts]
```

de::commandOption

Description

Moves the pointer to the given location and sends a `commandOption` event to the interactive command in a design editor window. This event provides an event of custom type for interactive commands that require more event bindings than can be provided by the built-in events.

The point argument is in the view design's coordinate space.

Syntax

```
de::commandOption <name: string> [-window <giWindow | unsigned int> |  
-context <deContext>] [-point <point>]
```

Arguments

name

commandOption event type.

-window

Window in which to send the event.

-context

Context in which to send the event.

-point

Point to send with the event.

Examples

The following example uses `ise::createPatch` to bind the R key to the `de::commandOptions` command so that pressing R rotates patch runtime objects.

```
namespace eval ise::createPatch {  
proc init {cmd ctx args} {  
    set ovl [de::createOverlay "sePatch" \  
        -base [db::getattr editDesign -of $ctx] \  
        -shown true -drawPlane edit]  
    db::setattr cmd.clientData -value R0  
}  
proc abort {cmd} {  
    set ctx [db::getNext [db::getattr cmd.contexts]]  
    set ovl [de::getOverlays "sePatch" \  
        -base [db::getattr ctx.editDesign]]  
    db::destroy $ovl  
}  
proc createInst {design pt orient} {  
    oa::ScalarInstCreate [oa::getTopBlock $design] \  
        basic patch symbol \  
        [oa::Transform [oa::Point $pt] [oa::Orient $orient]]  
}  
proc snapMotion {cmd ctx pt} {  
    set ovl [de::getOverlays "sePatch" \  
        -base [db::getattr ctx.editDesign]]  
    set design [db::getattr ovl.design]  
    set orient [db::getattr cmd.clientData]  
    db::destroy [db::getInsts -of $design]  
    createInst $design $pt $orient  
}
```

```

}

proc addPoint {cmd ctx pt} {
    set ovl [de::getOverlays "sePatch" \
        -base [db::getAttr ctx.editDesign]]
    set design [db::getAttr ovl.baseDesign]
    set ovlDes [db::getAttr ovl.design]
    db::destroy [db::getInsts -of $ovlDes]
    set orient [db::getAttr cmd.clientData]
    set trans [de::startTransaction "Add Patch"]
    createInst $design $pt $orient
    de::endTransaction $trans
}

proc commandOption {cmd ctx pt optionName} {
    switch $optionName {
        rotate {
            set ovl [de::getOverlays "sePatch" \
                -base [db::getAttr ctx.editDesign]]
            set design [db::getAttr ovl.design]
            set inst [oa::getNext [oa::getInsts \
                [oa::getTopBlock $design]]]
            set trans [oa::concat [db::getAttr inst.transform] \
                [oa::Transform [oa::Orient R90]]]
            db::setAttr cmd.clientData \
                -value [db::getAttr trans.orient]
            snapMotion $cmd $ctx $pt
        }
    }
}

de::createCommand [namespace current] -type interactive \
    -category schematic -description "Create Patch" \
    -label "Add Patch"

}

gi::createBinding -event r \
    -command {de::commandOption rotate -point %a} \
    -activeOperation ise::createPatch

```

de::completeShape

Description

Moves the cursor to the given point and sends a completeShape event to the active command in a design editor window. This event completes an interactive edit and initiates the database transaction which commits the edit to the design.

The point argument is in the view design's coordinate space.

Syntax

```
de::completeShape [<point: point>] [-window <giWindow | unsigned int> |  
-context <deContext>]
```

Arguments

point

Point to send with event.

-window

Window in which to send the event.

-context

Context in which to send the event.

Examples

```
# Start to create rectangle in the layout design  
ile::createRectangle  
# Add start point  
de::addPoint {0 0}  
# Complete rectangle  
de::completeShape {1 1}
```

de::connect

Connect to another session, open the equivalent design and initiate a design compare features.

Syntax

```
de::connect (-localDir <string> | -hostPort <string>) [-lcv <string> |  
-remoteWindowId <int>] -context <deContext> [-side  
<deCompareDesktopSide>]
```

Returns

<none>

Arguments

-context

Design context from which to open the other design

-localDir

Local directory of the session you want.

`-lcv`

LCV to be opened in the session you want.

`-hostPort`

hostname:port of the session you want to connect.

`-side`

Side of the desktop to move the window. Default: right.

`-remoteWindowId`

Window ID on the remote session to use. Default: 0.

de::connectToServer

Description

Connects a design to a multi-user editing server.

Syntax

```
de::connectToServer <design: oaDesign>
```

Returns

<none>

Arguments

design

Design to connect to a multi-user editing server.

de::copy

Description

Copies objects into the clipboard.

Syntax

```
de::copy <objects: {<deFigure>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>}  
|  
{<oaMarker>} | {<oaShape>} | {<oaVia>}> [-anchor <point>] [-selectResult  
<bool>]
```

Returns

Collection of objects that were just copied to the clipboard.

Arguments

`objects`

List of objects to copy.

`-anchor`

Anchor point for paste.

Default: undefined

`-selectResult`

Specifies whether to add the result to the selection set.

Default: true

Errors

- An `oaGroup` object is supplied.
- The collection of input `oaObjects` or `deFigures` are not from the same `oaDesign`.
- An interactive paste command is active, such as `ile::paste` or `ise::paste`.

Examples

```
set design [ed]
# Copy selected objects to clipboard
de::copy [de::getSelected -design $design]
# Paste objects from clipboard into active design
de::paste $design -point {1 1} -cols 2 -rows 3 -dxRow 0.1 -dyRow 0.2
-orient R90
```

de::copyBookmark

Description

Create a copy of a bookmark with a new name.

An interactive version of this command is not available.

Syntax

```
de::copyBookmark <bookmark: deBookmark> -name <string> [-force <bool>]
```

Returns

The new bookmark. ([deBookmark](#))

Arguments

`deBookmark`

The bookmark to copy.

`-name <string>`

The name of the new bookmark.

`-force <bool>`

Allow overwriting of an existing bookmark.

Default: true

Errors

- The bookmark was ifvinvalid.
- The value of `-name` matches an existing bookmark but `-force` was not true.

Examples

```
# Create new bookmark
de::createBookmark newBookmark -window [gi::getWindows $id]
# Get created bookmark
set bm [de::getBookmarks newBookmark]
# Copy the bookmark
de::copyBookmark $bm -name copiedBookmark
# Open Bookmark Manager
# New bookmark with name "copiedBookmark" is added
de::showBookmarkManager
```

de::copyClipboardToLib

Description

Copies a clipboard to a clipboard library. The original clipboard is unmodified.

To get available clipboards, use `de::getClips`. To open a clipboard library, use `de::openClipboardLib`.

Syntax

```
de::copyClipboardToLib <deClipboard> -to <oaLib>
```

Returns

`<deClipboard>`: New clipboard object.

Arguments

`deClipboard`

Clipboard to copy.

`-to`

Target library.

Examples

```
# Copy I0 instance to clipboard
set inst [db::getInsts I0 -of [ed]]
de::copy $inst

# Open new clipboard library named clipLib
de::openClipboardLib clipLib -path ./clipLib

# Copy clips to new library
set clips [de::getClips]
db::foreach clip $clips {
    de::copyClipboardToLib $clip -to [de::getClipboardLibs clipLib]
}
```

de::createActivationEngine

Description

Creates an activation engine, which enables active object highlighting and responds to addPoint events by calling a specified objectPickProc if the addPoint occurred over an object.

Syntax

```
de::createActivationEngine [-objTypes <{<deObjectFilter>}> |
    -forceObjTypes <{<deObjectFilter>}>] [-objTypes <{<deObjectFilter>}>
    | -excludeObjTypes <{<deObjectFilter>}>] [-filterProc [<variant>]]
    [-objectPickProc [<variant>]] [-addPointProc [<variant>]]
    [-startDragProc [<variant>]] [-endDragProc [<variant>]] [-snapMotionProc
    [<variant>]] [-activeObjectChangeProc [<variant>]] [-lpps
    <{<deTechLPP>}>] [-depth <int>] [-mode <enum>] [-viewDesign <bool>]
    [-includeOverlays <bool>]
```

Returns

<deActivationEngine>: A deActivationEngine object.

Arguments

`-filterProc`

Procedure called for each selectable object. The `-filterProc` procedure takes two arguments: the current active command and `deFigure` under the cursor.

Returns true if the object can be selected, returns false to prevent selection.

For example:

```
proc filterProc {command figure} { ... }
```

`-objectPickProc`

Procedure called when `addPoint` events occur over an active object. Retrieves the figure that was clicked via the `figure` attribute of the command's engine.

For example:

```
proc objectPickProc {cmd} {  
    db::getAttr engine.figure -of $cmd  
}
```

`-addPointProc`

Procedure called when clicking in the canvas.

`-startDragProc`

Procedure called when initiating a drag.

`-endDragProc`

Procedure called when completing a drag.

`-snapMotionProc`

Procedure called when moving the cursor.

`-activeObjectChangeProc`

Procedure called when active object changed.

`-lpps`

The list of the lpps to activate objects on.

`-objTypes`

The list of the object types can be activated.

`-forceObjTypes`

The list of the object types which should override the types from the OLP filter.

`-excludeObjTypes`

The list of the object types cannot be activated.

`-depth`

Activation depth. (Default is: 0)

`-mode`

Activation mode. ({ align | normal | tap }, default is: normal)

`-viewDesign`

View design. (Default is: false)

`-includeOverlays`

Include overlays. (Default is: false)

Examples

In the following example, if the `user::pathToPolygon` that you created is active, only the path objects are activated by the activation engine. The object gets converted to a polygon when you click on it.

```
namespace eval user::pathToPolygon {
    proc init {self ctx} {
        db::setAttr engine -of $self \
            -value [de::createActivationEngine \
                -filterProc [namespace current]::_filterPath \
                -objectPickProc [namespace current]::_convert];
    }
    proc _filterPath {self figure} {
        if {"Path" == [db::getAttr objType -of $figure]}{
            return 1
        } else {
            return 0
        }
    }
    proc _convert {self} {
        le::convertToPolygon [db::getAttr engine.figure -of\
            $self];
    }
}
de::createCommand user::pathToPolygon \
    -description "Creates pin Name" \
    -category layout -typeinteractive
```

See [Creating Custom Tcl User Commands](#) for more information and examples.

de::createApplication

Description

Creates a new application which can be used to open the specified file name.

Syntax

```
de::createApplication <name: string> -title <string> -openProc  
[<variant>] [-canCreate <bool>] [-canOpenMultiple <bool>]
```

Returns

<deApplication>: Newly created application object.

Arguments

name

Unique string identifier for new application.

-title

String that is shown in various interfaces.

-openProc

Procedure to evaluate using syntax given in the description.

-canCreate

Indicates that this application can create new cellviews.

-canOpenMultiple

Indicates that this application can open multiple files in a single invocation.
Default: false

Examples

In the following example, `de::createApplication` creates the VIM external editor application, which is used to open or create a `systemVerilogText` view type cellview.

```
proc de::_openSV {file readOnly attrs} {  
    set cmd [list xterm -e vim [db::getAttr path -of $file]]  
    xt::createJob deText \  
        -cmdLine $cmd \  
        -type interactive \  
        -runDesc "External editor"  
}  
de::createApplication svEditor \  
    -title "System Verilog" \  
    -openProc de::_openSV \  
    -canCreate 1
```

```
de::createAssociation -fileName systemVerilogText \  
-application svEditor
```

de::createArgument

Description

Creates a new argument definition for use with `de::createCommand`. This object need not be destroyed with the `db::destroy` command.

The argument must at least specify a name, for example, `-window`. Additionally, it may accept one of a several types (string is the default) including a list or collection consisting of any of these types. If `-isList` is true, the command accepts either a list or collection, but the underlying Tcl procedure will always be called with a collection object.

Note that some types represent multiple object types.

User commands do not support nameless (positional) arguments.

If a default value is not specified for an optional argument, the command will not see the argument specified in its argument list. In this manner, it can handle the default case without a hard-coded value.

The `-hidden` argument causes the argument to not be listed in a command's help text.

Syntax

```
de::createArgument [<name: string>] [-types [<string>]]  
-description <string> [-isList <bool>] [-optional <bool>] [-hidden  
  <bool>]  
[-default <variant>]
```

Returns

Newly created argument. (deArgument)

Arguments

`name`

Name of the command argument without the preceding dash. This may be omitted for positional arguments.

Default: ""

`-types`

Argument type names which match the value returned by `db::getAttr type`.

Default: string

`-description`

Help string for this argument.

`-isList`

Specifies that the argument accepts a list or collection consisting of any (even mixed) of the types indicated by `-types`.

Default: false

`-optional`

If true, this argument must be provided to calls to the command.

Default: false

`-default`

Provides a hard-coded default value for an optional argument that will always be provided to the command. Note that this argument is ignored if the argument type is non-native Tcl (for example: `giAction` or `deFigure`).

Default: Unspecified

`-hidden`

Restricts this argument from being shown in the command's help string.

Default: false

Errors

- Default value specified for non-optional argument.
- Invalid type name.

Examples

In the following example, `de::createArgument` creates the `-angle` argument, whose valid values are `Orthogonal`, `Diagonal`, and `Any`.

```
db::createEnum enAngleMode -values {Orthogonal Diagonal Any}
set a [de::createArgument "-angle" -types enAngleMode
    -description "Angle Mode, values: {[db::getAttr values
    -of [db::getEnums enAngleMode] ]}"]
namespace eval nm::any_cmd {
    proc execute {args} {
        puts $args
    }
}

de::createCommand "nm::any_cmd"
    -description "description of created command any_cmd"
    -arguments $a -category foundation
```



```
nm::any_cmd -help
nm::any_cmd -angle Diagonal
nm::any_cmd -angle Orthogonal
nm::any_cmd -angle Any
```

In the following example, `de::createArgument` creates the `-textLayer` argument definition, which can be used in a custom Tcl command as an optional argument.

```
de::createArgument -textLayer -optional true
    -description "Defines the text layer name to use for pin"
```

In the following example, `de::createArgument` creates a `dbEnum` type and uses `dbEnum` as a command argument.

```
set e5 [db::createEnum ee5 -valueDescPairs {{aaa "decription for aaa"}}\
    {bbb "description for bbb"}} ]
set a5 [de::createArgument "-enum_type_arg" -types ee5
    -description\ "argument of enum type, ee5 , values: [db::getAttr
    values -of \
    [db::getEnums ee5]]" -optional true ]

de::createCommand "nm::cmd_ex5" -description "description of \
    created commands cmd_ex5" -arguments $a5 -category foundation

namespace eval nm::cmd_ex5 {
    proc execute {args} {
        puts $args
    }
}

nm::cmd_ex5 -help
nm::cmd_ex5 -enum_type_arg xxx
nm::cmd_ex5 -enum_type_arg aaa
```

The following example shows how to create a non-interactive Tcl command. First, the example creates a Tcl procedure. Then, the example shows how to use `de::createArgument` to register arguments for the command. Lastly, the example registers the `myCmd` non-interactive command using `de::createCommand`.

```
## Create Tcl Procedure
namespace eval myCmd::getRefLib
proc execute { args } {
    array set myArgs $args
    if [catch {set tech [db::getTechReference $myArgs (-libName)]}] {
        set techLibName $myArgs (-libName)
    } else {
        set techLibName [oa: :getName $tech]
    }

    set techLibPath " "
    if [info exists myArgs(-refLibPath)]
    if [set myArgs(-refLibPath)] (
        set techLib [db: :getNext [dm: :getLibs $techLibName]]
    )
}
```

```
        set techLibPath [oa: :getFullPath $techLib]

    set cmd [puts "The Reference Library of $myArgs(-libName) is \
        $techLibName techLibPath"]
    return [eval $cmd]

# Register arguments for commands
set args [list]
lappend args [de: :createArgument -libName -description "Library Name"]
lappend args [de: :createArgument -refLibPath -optional true -types bool \
    \
        -description "Ref. Library Path is required"]

# Register command
de::createCommand myCmd ::getRefLib -category utility -arguments $args \
    -description "Getting Name and Path of Reference
    Library"
```

de::createArgumentGroup

Description

Creates an argument group. An argument group is a list of arguments that a command considers mutually exclusive. This argument may be passed to the `de::createCommand` procedure.

All `deArgument` objects must be optional. The `deArgumentGroup` object may itself be optional.

Syntax

```
de::createArgumentGroup <arguments: [deArgument | string]> [-optional  
    <bool>]
```

Returns

Newly created argument group object (`deArgumentGroup`).

Arguments

`arguments`

Tcl-list of `deArguments` or of argument names (strings).

`-optional`

Specifies that this argument group is optional to the command.

Default: false

Examples

In the following example, `de::createArgument` creates the `-libName` and `-lib` arguments, and `de::createArgumentGroup` creates an argument group with these arguments. When passed to `de::createCommand`, the arguments will be considered mutually exclusive.

```
de::createArgument -libName -description "Output Library Name"
de::createArgument -lib -type oaLib -description "Output Library"
de::createArgumentGroup {-libName -lib}
```

de::createAssociation

Description

Creates an association between a DM file name and a `deApplication` object.

If an association already exists with the given name, it will be removed and replaced with the newly created association.

The title given to this association will be displayed in dialogs including Create Cellview and Open Design.

Syntax

```
de::createAssociation -application <deApplication | string>
                    -fileName <string> [-default <bool>]
```

Returns

Newly created association object (`deAssociation`).

Arguments

`-fileName`

DM File name to associate.

`-application`

Application to invoke for this file name.

`-default`

Sets this editor as the default editor for the specified view type.

Examples

```
# Create new viewType
if {[oa::ViewTypeFind "newText.txt"] == ""} {
  oa::ViewTypeCreate "newText.txt"
}
# Set nedit as default tex editor
```

```
db::setPrefValue deTextEditor -value nedit
# Define proc to open files using nedit and put there Custom Compiler
version
proc openNewTextView {dmFile readOnly attrs} {
    set filePath [db::getattr dmFile.path]
    if ![file size $filePath] {
        set fh [open $filePath "w"]
        puts $fh "Custom Compiler - [db::getattr version -of
            [db::getProcessInfo]]"
        close $fh
    }
    set cmd [list [db::getPrefValue deTextEditor] $filePath]
    xt::createJob deText \
        -cmdLine $cmd \
        -type interactive \
        -runDesc "External editor"
    }
# Create application which openProc is previous proc
set newText [de::createApplication "newText" \
    -title "New Text" \
    -openProc openNewTextView \
    -canCreate true \
    -canOpenMultiple false]
# Create association to open "newText.txt" files with newText application
de::createAssociation -fileName "newText.txt" -application $newText
    -default true -defaultViewName newText
```

de::createBookmark

Description

Creates a new edit-context bookmark with the specified name, or copies an existing bookmark.

Syntax

```
de::createBookmark <{name}: string> [-window <int>] [-comment <string>]
    [-force <bool>]
```

Returns

The new bookmark. ([deBookmark](#))

Arguments

<string>

The name of the new bookmark.

-window <int>

The window to save in a bookmark.

Default: active window

`-comment <string>`

A user-specified comment.

Default: Empty string

`-force <bool>`

Allow overwriting of an existing bookmark.

Default: true

Errors

- The window ID was invalid.
- The value of `-name` matches an existing bookmark but `-force` was not true.
- There was no open design in the specified window.

Examples

```
# Create new bookmark for active window
de::createBookmark opamp -window [de::getActiveEditorWindow]

# Get and delete bookmark
set bm [de::getBookmarks opamp]
de::deleteBookmarks $bm
```

de::createCanvas

Description

Creates a new deCanvas widget and adds it to the specified parent.

Syntax

```
de::createCanvas [<{none}: string>] [-parent <giAssistant | giContainer  
| giDialog | giWindow>] [-enabled <bool>] [-shown <bool>] [-toolTip  
<string>] [-context <deContext>] [-viewport <box>] [-cursor <point>]  
[-snapCursor <point>] [-display <drDisplay>] [-autoFit <bool>] [-width  
<int>] [-height <int>]
```

Returns

<deCanvas> : Newly created deCanvas object.

Arguments

{none}

Widget name.

`-parent`

Dialog, group, or other container into which this widget should be placed.

`-enabled`

Initial enabled state.

Default: true

`-shown`

Initial shown state.

Default: true

`-toolTip`

Optional widget toolTip.

`-context`

Context to depict.

`-viewport`

Initial viewport.

`-cursor`

Initial cursor position.

`-snapCursor`

Initial snap cursor position.

`-display`

Display resource display to use for rendering.

`-autoFit`

Automatically fit the viewport to the extents of the edit design whenever the canvas size, or design size, changes.

Default: false

`-width`

Canvas width in pixels, or 0 to expand to fill available space.

Default: 0

`-height`

Canvas height in pixels, or 0 to expand to fill available space.

Default: 0

Examples

```
# Create dialog box
set dialog [gi::createDialog testDialog -title "Dialog with canvas"]

# Add canvas to that dialog box
de::createCanvas -parent $dialog -context [de::createContext
  analogLib/cap/symbol]
  -autoFit true
```

de::createColorExpression

Description

Creates a color expression object.

Syntax

```
de::createColorExpression -expression <string> -color <string>
  [-propagate <enum>] [-shape <enum>]
```

Returns

<deColorExpression> : Created color expression object

Arguments

-expression

Boolean expression identifying the objects that should be colored with given color.

-color

Density color name, which should be used for coloring the objects matching the boolean expression.

-propagate

Color propagation type up the hierarchy: { none | oneColor | worstColor }.
Default: oneColor

-shape

Shape type to be used for colorizing instances: { circle | rectangle }.
Default: rectangle

de::createCommand

Description

Creates and registers a user-defined Custom Compiler Tcl interactive or procedural command with the command manager. A command name, list of arguments, and description must be specified. If a command by the same name is already registered with `de::createCommand`, it is silently overwritten by the newly created one.

Interactive command arguments will start with a positional windowId in addition to the arguments provided. This is added to the set of arguments specified.

A toolbar and context menu will be automatically created. Retrieve these objects from the returned `deCommand` object.

Syntax

```
de::createCommand <name: deCommandInterface | string> [-license  
  [<string>] | -licenseDesc [<string>]]  
-description <string> -category <string> [-type <enum>] [-scope <enum>]  
  [-arguments [<deArgument>]]  
[-customSortingComparator <string>] [-customActiveObjectToolbarProc  
  <variant>]  
[-exclusive [<deArgumentGroup>]] [-title <string>] [-label <string>]  
  [-returnTypes [<string>]]  
[-returnDesc <string>] [-requiresWriteMode <bool>]
```

Returns

<deCommand>: Newly created interactive command. Note that this command returns "" when registering procedures.

Arguments

name

Name of the command.

-license

Required license feature names.

-licenseDesc

Set of license feature names to show in the command help declaration.

-description

Short description of the command. For example: "Starts the interactive create rectangle command."

`-category`

String category name used for organizing this command with other related commands in the console help system.

`-type`

Type of command to define: procedural or interactive.

`-arguments`

List of arguments created with `de::createArgument`. The Window ID is added as a first argument for all interactive commands.

Default: {}

`-customSortingComparator`

Sorting comparator function name.

Default is: ""

`-customActiveObjectToolbarProc`

Active Object information display proc.

`-exclusive`

List of exclusive argument groups.

Default: {}

`-title`

Command title, used on the corresponding dialog box title bar, for example, Create Rectangle.

Default is: ""

`-label`

Command label, used on the toolbar, for example, Rectangle.

Default is: ""

`-returnTypes`

Object types returned by this command, or {} if the command does not return anything.

Default is: {}

`-returnDesc`

Description of command result.

Default is: ""

`-requiresWriteMode`

Indicates whether write access to the design is required.

Default is: false

`-scope`

Scope of interactive command: {context | design}

Default is: design

Examples

```
namespace eval le::deleteShapesByLPP {
    proc execute {args} {
        array set myArgs $args
        set shapes [db::getShapes -lpp $myArgs(-lpp) -of $myArgs \
            (-design)]
        return [le::delete $shapes]
    }
}
set args [list]
lappend args [de::createArgument -lpp -type LPP \
    -description "Restricts shapes to be returned from a given LPP."]
de::createCommand le::deleteShapesByLPP -category layout \
    -arguments $args -description "Deletes shapes by given LPP."
```

This is a fully scripted example where `de::createCommand` registers the new `le::deleteShapesByLPP` command, which has two required options: `-lpp` and `-design`. The new command `le::deleteShapesByLPP` deletes all shapes with the matching LPP as defined in the `-lpp` option, and in the `oaDesign` as defined in the `-design` option.

de::createContext

Description

Creates a new `deContext` object for the specified file, cellview's primary file or bookmark object.

Syntax

```
de::createContext <object: deBookmark | dmCellView | dmCellViewName |
    dmDMFile | oaDesign> [-readOnly <bool>] [-instPath <string>]
```

Returns

<deContext>: Newly created context

Arguments

`object`

Cellview (primary file), DM file or bookmark object to open.

`-readOnly`

Opens the design in read-only mode.

Default: false

`-instPath`

Optional instance path to edit.

Default: /

Examples

In the following example, `de::createContext` creates a context for the designLib/vco/schematic design. For the created context, the designLib/vco/schematic (topDesign) and X1 master (editDesign) are opened in headless mode.

```
set context [de::createContext [dm::findCellView schematic \  
    -cellName vco -libName designLib] -instPath /X1
```

de::createCrossProbeTool

Description

Creates a new cross-probe tool object and returns it. If a cross probe tool already exists with the specified name it is replaced with the new object.

Syntax

```
de::createCrossProbeTool <name: string> -contextFilterProc [<variant>]  
    -crossProbeProc [<variant>]
```

Returns

<deCrossProbeTool>: New cross probe tool object

Arguments

`name`

English name of the new cross probe tool object.

`-contextFilterProc`

Tcl or Python procedure which returns 1 if the tool is valid for a context and 0 otherwise.

`-crossProbeProc`

Tcl or Python procedure which creates and returns associated probe objects.

de::createErrorViewerDatabase

Description

Creates a `deErrorViewerDatabase` object.

Syntax

```
de::createErrorViewerDatabase <name: string> -design <oaDesign> [-title  
  <string>] [-closeable <bool>]
```

Returns

`<deErrorViewerDatabase>` : Newly created `deErrorViewerDatabase` object

Arguments

`name`

Unique identifier.

`-design`

The base design of the database.

`-title`

The title displayed in the tab widget of the Error Viewer.

`-closeable`

Indicates whether or not the database can be closed via the close button in the Error Viewer. Default: false.

Examples

```
# Create new Error Viewer database  
set design [ed]  
de::createErrorViewerDatabase newErrDB -design $design -title "New  
  Database"
```

de::createErrorViewerMarker

Description

Creates a marker for `deErrorViewerDatabase`.

Syntax

```
de::createErrorViewerMarker <of: deErrorViewerDatabase> -points [<point>]  
[-closed <bool>] [-severity <enum>] [-user1 <string>] [-user2 <string>]  
[-user3 <string>] [-user4 <string>] [-tool <string>] [-shortMsg  
<string>] [-msg <string>]
```

Returns

<oaObject> : Marker Object

Arguments

of

The deErrorViewerDatabase object.

-points

The geometry of the marker.

-closed

Specifies whether or not the geometry is closed. Default: true

-severity

Specifies the severity value of marker ({ acknowledgedWarning | annotation |
criticalError | error | fatalError | info | signedOffCriticalError | signedOffError |
warning }). Default: error

-user1

The value for user1.

-user2

The value for user2.

-user3

The value for user3.

-user4

The value for user4.

-tool

The tool name.

-shortMsg

The summary description of the error.

-msg

The detailed description of the error.

Examples

```
# Create new Error Viewer database
set design [ed]
de::createErrorViewerDatabase newErrDB -design $design -title "New
Database"
# Create new Error Viewer marker and add it to the database
set db [de::getErrorViewerDatabases newErrDB]
de::createErrorViewerMarker $db -points {{0 0} {1 1}} -msg "new Error"
-shortMsg err1 -tool new_tool
```

de::createFavoriteDevice

Description

Creates a new favorite device for use with the Device Palette Assistant.

Syntax

```
de::createFavoriteDevice -master <dmCellView> [-params [<map>]] [-alias
<string>] [-viewType <enum>]
```

Returns

<deDevice>: Newly created favorite device

Arguments

-master

Master cellview.

-params

Optional list of parameter overrides in the format: {{name value} {name value}}.

-alias

An optional alias for the favorite device.

-viewType

View type: {maskLayout | schematic}.

Examples

```
# Find device cellview
set cv [dm::findCellView symbol -cellName cap -libName analogLib]
# Add it to favorite devices
de::createFavoriteDevice -master $cv -viewType schematic
```

de::createFavoriteDeviceCategory

Description

Creates a new favorite device category for use with the Device Palette Assistant.

Syntax

```
de::createFavoriteDeviceCategory <{name}: string> [-devices  
  <dbCollection>]
```

Returns

<deDeviceCategory>: Newly created favorite device category

Arguments

{name}

Category name.

-devices

Devices to add to this category.

Examples

```
# Create favorite devices from different libraries
set cv1 [dm::findCellView symbol -cellName cap -libName analogLib]
set cv2 [dm::findCellView symbol -cellName res -libName analogLib]
set cv3 [dm::findCellView symbol -cellName Cc -libName parasitics]
de::createFavoriteDevice -master $cv1 -viewType schematic
de::createFavoriteDevice -master $cv2 -viewType schematic
de::createFavoriteDevice -master $cv3 -viewType schematic
# Find favorite devices from analogLib
set devices [de::getFavoriteDevices -filter
  {%master.libName=="analogLib"}]
# Create new favorite device category and add these devices into it
de::createFavoriteDeviceCategory fromAnalogLib -devices $devices
```

de::createGradient

Description

Creates gradient object.

Syntax

```
de::createGradient -expression <string> -minValue <string> -maxValue  
  <string> -count <unsigned int> -startColor <string> -endColor <string>  
  [-propagate <enum>]
```

Returns

<deGradient> : Created gradient object

Arguments

-expression

Expression used to evaluate the values to be represented in a color gradient.

-minValue

Minimum value of the gradient.

-maxValue

Maximum value of the gradient.

-count

The number of colors in the gradient.

-startColor

Density color name, which should be used as first color in the gradient.

-endColor

Density color name, which can be used as last color in the color gradient.

-propagate

Color propagation type up the hierarchy: { none | oneColor | worstColor }.
Default: worstColor

de::createHierObject

Description

Creates a new deHierObject.

Syntax

```
de::createHierObject <object: dbCore | oaObject> -hierPath <deContext |  
oaHierPath>
```

Returns

<deHierObject> : New deHierObject.

Arguments

object

Referenced object.

`-hierPath`

Hierarchical path from top design to object.

Examples

The following example creates a hierarchical reference to the I0 instance object and lists the `hierObject` attribute names.

```
set context [de::getActiveContext]
set ho [de::createHierObject [db::getInsts I0 -of [db::getAttr
context.editDesign]] -hierPath $context]
db::listAttrs -of [db::getAttr ho.hierPath]
=> depth levels and type of transformation
```

de::createHighlightSelector

Description

Creates a new `deHighlightSelector` widget and adds it to the specified parent.

Syntax

```
de::createHighlightSelector [<{none}: string>] [-parent <giAssistant |
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]
[-prefName <dbPrefName>] [-valueChangeProc [<variant>]] [-label
<string>] [-value <string>] [-showNames <bool>] [-showAuto <bool>]
```

Arguments

`{none}`

Widget name.

`-parent`

Dialog, group, or other container into which this widget should be placed.

`-enabled`

Specifies whether the widget function is enabled by default.

Default: true

`-shown`

Specifies whether the widget is displayed by default.

Default: true

`-prefName`

Preference name within top-level container scope, which provides the initial value, and is automatically updated with value changes according to the container update model.

`-valueChangeProc`

Procedure to execute when the value is changed. Called with the widget object.

`-label`

Widget label, shown in the left column.

`-value`

Default value.

`-showNames`

Displays highlight names.

Default: false

`-showAuto`

Enables automatic colorization.

Default: false

Returns

`<deHighlightSelector>`: New object.

Examples

```
proc valueChangeProc {widget} {  
  puts "Value change: [db::getAttr widget.value]"  
}  
# Create dialog box  
set dialog [gi::createDialog testDialog -title "Highlight Selector"]  
  
# Add highlight selector to that dialog box  
de::createHighlightSelector -parent $dialog -label "Color"  
  -showNames true -valueChangeProc valueChangeProc
```

de::createHighlightSet

Description

Creates a new highlight set within the specified design. It may be destroyed with the `db::destroy` command. If another set exists with the same name, it will be purged and returned.

If the highlight set is declared public (false, by default), it will be displayed in the highlight set selectors of the various highlighting commands.

Syntax

```
de::createHighlightSet <name: string> -design <oaDesign> [-packet  
  <string>] [-packetFill <bool>] [-persistent <bool>] [-public <bool>]  
  [-shadow <bool>] [-visible <bool>] [-drawInvisible <bool>]
```

Returns

Newly created highlight set ([deHighlightSet](#)).

Arguments

<string>

New highlight set name.

-design *oaDesign*

Design from which objects will be highlighted.

-packet <string>

Packet name or packet alias which defines the drawing style for this set.

Default: "highlight1"

-packetFill

Packet Fill defines the fill drawing style. Default: true

-public <bool>

Show this set in selectors of interactive commands.

Default: false

-shadow

Enables Shadow Mode for this set. Default: false

-visible

Determines if the highlight set is visible. Default: true

-drawInvisible

If true, this option highlights the invisible figures. Default: true

Errors

- The display resource packet alias does not resolve to a defined packet.
- The design has not been opened by the design editor.

Examples

In the following example `de::createHighlightSet` creates the **blinkRed** highlight set in the given design, using the **markerFat** packet. The created highlight set can be used for highlighting objects with [de::highlight](#) and [ide::highlight](#).

```
de::createHighlightSet blinkRed -design $oaDesign -packet \  
    markerFat -public true
```

de::createHomePage

Description

Create a new giWindowType home page. If a giWindowType exists with the specified name, the command replaces the giWindowType with the new object.

Syntax

```
de::createHomePage <windowTypeName: string> [-bannerText <string>]  
[-product <string>] [-showAllProducts <bool>]  
[-showUtilities <enShowUtilities>] [-showDesigns <bool>]
```

Arguments

windowTypeName

The new window type name.

-bannerText

Displays the application name in the optional home page banner.

-product

Defines which icons are displayed in the top section of the home page. (see [gi::createAppIcon](#) -product).

-showAllProducts

Defines whether to show all product icons in alphabetical order in a separate section in the home page. Default: *false*.

-showUtilities

Defines whether to display the Utilities icons in the home page. There are three options available:

- inProduct: displays all the Utilities icons in the top section.
- inUtilities: displays all the Utilities icons in a separate section.
- unset: does not display the Utilities icons in the home page.

-showDesigns

Displays the designs section. Default: *false*.

Returns

<giWindowType> : New giWindowType

de::createInterfaceElement

Description

Creates interface element on net or instance terminal in the given hierarchy context.

Syntax

```
de::createInterfaceElement -on <oaInstTerm | oaNet> -direction <enum>
[-params <string>] -context <heContext> -netlister <nIAMSNetlister>
```

Returns

<nIInterfaceElement> : Interface element object.

Arguments

-on

Net or instance terminal on which the IE is created.

-direction

Direction of created IE. ({ a2d | d2a | e2n | e2r | inout | n2e | r2e }).

-params

Parameter values string with format "<param_name_1> <param_value_1>
<param_name_2> <param_value_2> ..."

-context

Hierarchy context.

-netlister

AMS netlister according to which the partition is made.

Errors

- Netlister is not specified and the netlister with default '<name>' does not exist.
- The specified net is not mixed-signal.
- The specified object is not in design of the given design context.
- Partition visualization is not executed for the specified design context and netlister.

Examples

```
# Create interface element on OUT net
set context [de::getActiveContext]
set design [db::getAttr context.editDesign]
set net [db::getNets OUT -of $design]
set hc [db::getAttr context.hierarchy]
```

```
set n [nl::getAMSNetlisters AMS]
de::createInterfaceElement -on $net -direction a2d -context $hc
-netlist $n

# Save interface elements to ie.xml file
de::saveInterfaceElements [de::getInterfaceElements -context $hc
-netlist $n] -filePath ./ie.xml

# Remove interface elements and load them from ie.xml file
db::destroy [de::getInterfaceElements -context $hc -netlist $n]
de::loadInterfaceElements -filePath ./ie.xml
```

de::createLPPInput

Description

Returns a deLPPInput widget for layer-purpose pair input. The widget displays a list of layer-purpose pairs (LPPs) from the design or a subset of deLPP objects. If the list of LPPs is passed from the design, the initial list of LPPs can be filtered using the following display options: Valid, Visible, Select, and All.

Initially, the deLPPWidget displays only the current LPP. If a value or preference is not passed to deLPPWidget, then either the active LPP is displayed (if passing from a design) or the first LPP is displayed (if passing a subset of LPPs).

The widget can be expanded to display the full list of LPPs, and the current LPP is highlighted. To control the list of LPPs displayed, set the [leOLPLPPFilterByAttribute](#) Tcl preference.

Syntax

```
de::createLPPInput [<{none}>: string] [-design <oaDesign> | -tech
<oaTech> | -lpps <{<deTechLPP>}>] [-parent <giAssistant | giContainer
| giDialog | giWindow>] [-enabled <bool>] [-shown <bool>] [-prefName
<dbPrefName>] [-valueChangeProc [<variant>]] [-label <string>] [-value
<lpp>] [-readOnly <bool>] [-valid <bool>] [-required <bool>]
```

Returns

<deLPPInput>: New object.

Arguments

{none}

Widget name.

-parent

Dialog box, group or other container in which to place this widget.

`-enabled`

Initial enabled state.

Default: true

`-shown`

Initial shown state.

Default: true

`-prefName`

Optional preference name within top-level container's scope, which provides the initial value and is automatically updated with value changes according to the container update model.

Default: ""

`-valueChangeProc`

Procedure to execute when the value is changed. Called with the widget object.

`-label`

Widget label, shown in the left column.

Default: ""

`-value`

Initial value.

`-readOnly`

Initial read-only state.

Default: false

`-valid`

Initial valid state.

Default: true

`-required`

Specifies this is a required input.

Default: false

`-design`

Design from which to retrieve the layer-purpose pairs (LPPs).

`-tech`

Technology database from which to retrieve the layer-purpose pairs (LPPs).

`-lpps`

Subset of layer-purpose pairs (LPPs) to display.

Examples

```
proc valueChanged {widget} {
    puts [db::getAttr widget.value.lpp]
}

set design [ed]
set dialog [gi::createDialog test -title "LPP Input"]
de::createLPPInput lpp -design $design -parent $dialog -valueChangeProc
    valueChanged
```

de::createLPPSet

Description

Creates a new [deLPPSet](#) object. If a set with the same title already exists, it is replaced with this new object. It is not possible to replace the built-in layer-purpose pair (LPP) sets with this command.

Alternatively, the [deLPPSet](#) object can be created with a filter expression. This expression filters deLPP objects and is identical to the one passed to [de::getLPPs](#).

The new LPP set is immediately available within the Object/Layer Panel. To destroy the set, use the [db::destroy](#) command. Specified LPPs do not have to exist as this set can be applied to designs with any technology.

To get a collection of LPP sets, call the [de::getLPPSets](#) command.

Syntax

```
de::createLPPSet <title: string> (-lpps [<lpp | {<deLPP>}>] | -filter
    <string>) [-defaultLPP <deTechLPP | lpp>] [-shown <bool>]
```

Returns

<deLPPSet> : New deLPPSet object.

Arguments

`title`

Title of the new LPP Set.

`-lpps`

Tcl list of layer purpose pairs.

`-defaultLPP`

Specifies an LPP to set active when this LPP set is singly selected within the Object/Layer Panel. If unspecified, selecting this set will not alter the current active LPP unless it becomes invalid, in which case the first valid LPP is activated.

`-shown`

Displays the set in the Object/Layer Panel. Default: true

Errors

- Cannot replace built-in LPP set “%s”. You need to use a different name.

Examples

Example 1 cannot replace the built-in LPP sets.

```
de::createLPPSet "Metal Layers" -lpps {  
  {m1 drawing}  
  {m2 drawing}  
  {m3 drawing}}
```

Example 2 specifies the expression for finding LPP

```
de::createLPPSet "Metal" -filter {%layer.material ==  
  $oa::oacMetalMaterial}
```

de::createNetHighlightSet

Description

Creates a new highlight set containing all shapes on the specified net. Unlike normal highlight sets created with `de::createHighlightSet`, this command automatically highlights new shapes on the net and removes highlights from shapes as they are removed from the net. If `-shadow` is true, this highlight set will appear brighter than other shapes in the design. The `deShadowBrightness` preference can be used to configure how much other shapes are dimmed.

Syntax

```
de::createNetHighlightSet <name: string> -net <oaBitNet> -context  
  <deContext> [-packet <string>] [-shadow <bool>]
```

Returns

<deSet>: Newly created highlight set.

Arguments

name

Name of newly created set.

-net

Net to highlight.

-context

Context from which objects will be highlighted.

-packet

Packet name or packet alias that defines the drawing style for this set.

Default: highlight1

-shadow

Enables shadow mode for this set.

Default: false

Examples

```
# Create net highlight set for OUT net
set netName OUT
set net [db::getNets $netName -of [ed]]
de::createNetHighlightSet newNetHighlightSet -net $net
    -context [de::getActiveContext]
```

de::createObjectFilter

Description

Adds a custom object filter.

Syntax

```
de::createObjectFilter <name: string> -title <string> -filterProc
    <string> -design <oaDesign>
```

Returns

<none>

Arguments

name

Object filter name.

`-title`
Object filter title.

`-filterProc`
Tcl procedure name to filter objects.

`-design`
Design in which the object filter is added.

Examples

In the following example, the `de::createObjectFilter` command creates an object filter for PCell instances and sets the selectable attribute of that filter to false.

```
de::createObjectFilter userPcellFilter -design [ed] -title "PCell Filter"
\
    -filterProc {apply {{ fig } {
                        return [expr {[db::getattr fig.objType] != "Inst" ||
                                      ![db::getattr
fig.object.header.subHeader]]}}}
set objFilter [de::getObjectFilters userPcellFilter -from [ed]]
db::setattr objFilter.selectable -value false
```

de::createOLPPreset

Description

Creates a new Object/Layer Panel preset. The preset stores the current Object/Layer Panel settings in memory so that you can recall them later.

Syntax

```
de::createOLPPreset <title: string> [-id <unsigned int>] -of <deContext |
oaDesign>
```

Returns

<deOLPPreset> : New deOLPPreset object

Arguments

`title`
Name of the OLP preset.

`-id`
OLP preset ID. Default: 0

`-of`
Design or context from which to obtain settings for the new deOLPPreset

Examples

```
# Create OLP preset with name newPreset
# It saves current design OLP state.
de::createOLPPreset newPreset -of [ed]
```

de::createOnCanvasWidget

Description

Creates a new giDialog placed on a parent window canvas.

Syntax

```
de::createOnCanvasWidget <name: string> [-shown <bool>] -window <giWindow  
| int> [-viewportPoint <point>] [-closeProc [<variant>]]
```

Returns

<giDialog>: New object.

Arguments

name

Widget name.

-shown

Initial shown state. Default: true

-window

Parent window

-viewportPoint

Coordinate of widget position on a viewport.

-closeProc

Callback procedure to call when dialog closes. Called with the dialog object.

de::createOverlay

Description

Creates an overlay, which is a design that is drawn on top of another design. To retrieve existing overlays, use the [de::getOverlays](#) command. Note that the base design must be opened in the design editor when you open or create an overlay.

To delete overlays, use the `db::destroy` command. Overlays are automatically deleted when you close the design. However, they are not deleted during edit and revert operations.

Syntax

```
de::createOverlay <name: string> [-title <string>]  
-base <oaDesign> [-design <dmCellView | oaDesign>]  
[-packet <drPacket | drPacketAlias>] [-shown <bool>] [-drawPlane <enum>]  
[-ignoreDrawFilter <bool>] [-selectable <bool>] [-selectLogic <bool>]  
[-context <deContext>] [-customActiveObjectToolbarProc <string>]
```

Returns

<deOverlay>: Newly created overlay design

Arguments

`name`

Unique identifier for a given base design.

`-title`

Optional text description of the overlay design.

`-base`

Design over which the overlay is drawn.

`-design`

Overlay design. If not specified, a temporary cellview is created.

`-packet`

Packet to override normal drawing attributes.

`-shown`

Enables drawing this overlay. Default: true

`-drawPlane`

Drawing plane to use for the overlay: annotation, edit, or static. Default: annotation

`-ignoreDrawFilter`

Draws every shape in the overlay design. Default: false

`-selectable`

Permits object selection in the overlay. Default: false

`-selectLogic`

Selects logic in parent context when you select figures. Default: false

`-context`

Limits drawing the overlay to a specific context.

`-customActiveObjectToolbarProc`

Active object information display procedure. Default: <>

Examples

```
# Create overlay design over the edit design
set o [de::createOverlay newOverlay -base [ed]]
# Create coordinate mark in the overlay design
le::createCoordinateMark {0 0} -design [db::getAttr o.design]
# Create label in the overlay design
le::createLabel Overlay -lpp {m1 drawing} -origin {1 1} -parent
[db::getAttr o.design]
# Create rectangle in the overlay design
le::createRectangle {{2 2} {3 3}} -lpp {m1 drawing} -design [db::getAttr
o.design]
```

de::createProbe

Description

Creates a collection of new probe objects for the specified hierarchical context. By default, the object name comparison is performed in a case-sensitive manner. This can be changed by passing the `-caseSensitive` argument with a value of false. For case-sensitive matching, at most one new probe object will be created. The probe may be removed with the `db::destroy` command.

Syntax

```
de::createProbe <name: string> -context <deContext | heContext> -type
<enum>
[-tool <string>] [-packet <string>] [-description <string>]
[-longDescription <string>]
[-purpose <enum>] [-terminalByPosition <bool>] [-caseSensitive <bool>]
[-crossProbe <deCrossProbeTool | string>] [-allowObjectNotFoundMessage
<bool>]
[-traceProc <variant>]
```

Arguments

`name`

Object reference within the target design specified by the context.

`-context`
Hierarchical context including top design and lineage.

`-type`
Type of probe to create. ({ cellview | instTerm | instance | net | path | terminal })

`-tool`
Tool name.

`-packet`
Packet alias name of display resources.

`-description`
Probe description.

`-longDescription`
String describing a probe in detail.

`-purpose`
Probe purpose. ({ cross | error | info | other | warning })

`-terminalByPosition`
Indicates that the defined instance terminal uses terminal position. (Default: false)

`-caseSensitive`
Case-sensitive creation of probes. (Default: true)

`-crossProbe`
If specified, creates a probe in the specified context and cross probes via the specified cross probe tool.

`-allowObjectNotFoundMessage`
Enables printing information message when an object is not found. (Default: true)

`-traceProc`
Tcl procedure to enable tracing through devices in net probes highlighting. This procedure is called for each `oaInstTerm` on the probed net, and it should return a list of additional `oaBitNets` to trace. Signature: `<list of oaBitNets> traceProc <heContext> <oaInstTerm>`. Alternatively, insert `snpsCustom` to trace through devices by using **Probe Trace Through** options in the **General Options** dialog box.

Returns

deProbe: Newly created probe object.

Errors

Probe object does not exist.

Examples

Example 1

The following example probes all nets in an active schematic design by using **Probe Trace Through** options in the **General Options** dialog box.

```
set context [de::getActiveContext]
db::foreach net [db::getNets -of [db::getAttr context.editDesign]] {
    de::createProbe [db::getAttr net.name] -context $context -type net
    -description "Net [db::getAttr net.name]" -purpose info
    -packet highlight[expr round
    (rand()*10)] -tool Custom -traceProc snpsCustom }
```

Example 2

The following example probes all nets in an active schematic design by calling a customized procedure `de::probeTraceThroughLvsIgnore` to trace through devices.

```
db::foreach net [db::getNets -of [db::getAttr context.editDesign]] {
    de::createProbe [db::getAttr net.name] -context $context -type net
    -description "Net [db::getAttr net.name]" -purpose info
    -packet highlight[expr round
    (rand()*10)] -tool Custom -traceProc de::probeTraceThroughLvsIgnore }
```

de::createSelectionEngine

Description

Creates a selection engine, which enables active object highlighting and responds to semantic events by selecting shapes, for use in interactive commands created with `de::createCommand -type interactive`. To activate the selection engine, the command must set its *engine* attribute to this object, as shown in the example that follows:

```
db::setAttr engine -value [de::createSelectionEngine] -of $self
```

See [Creating Custom Tcl User Commands](#) for more information and examples.

Syntax

```
de::createSelectionEngine [-filterProc [<variant>]]
    [-startDragSelectedProc [<variant>]]
    [-endDragSelectedProc [<variant>]] [-dragMotionProc [<variant>]]
```

```
[-dragSnapMotionProc [<variant>]] [-selectionChangeProc [<variant>]]  
[-clickSelectedProc [<variant>]] [-topNet [<string>]] [-select <enum>]
```

Returns

<deSelectionEngine>: A deSelectionEngine object.

Arguments

-filterProc

Procedure called for each selectable object. The -filterProc procedure takes two arguments: the current active command and deFigure under the cursor.

Returns true if the object can be selected, returns false to prevent selection.

For example:

```
proc filterProc {command figure} { return true }
```

-startDragSelectedProc

Called when initiating a drag over selected objects.

-endDragSelectedProc

Called when completing a drag with selected objects.

-dragMotionProc

Called for each motion when dragging selected objects.

-dragSnapMotionProc

Called for each snap motion when dragging selected objects.

-selectionChangeProc

Called when selection changes.

-clickSelectedProc

Called when clicking on a selected object.

-topNet

Selects only objects on the specified net.

-select

Selection mode (deselect | replace | select).

Examples

In this example, when the `usr::copyVia` command is active, only Via objects are selectable. You can select and copy them using this command.

```
gi::createAction usrPushButton -widgetProc
    usr::copyVia::_buildButtonWidget;
gi::createAction usrDX -widgetProc
    usr::copyVia::_buildTextWidget1;
gi::createAction usrDY -widgetProc
    usr::copyVia::_buildTextWidget2;
gi::createAction usrNum -widgetProc
    usr::copyVia::_buildNumWidget;

de::createCommand usr::copyVia -type interactive \
    -description "Copies multiple vias" -category layout \
    -label "Copy Vias" -title "Copy Vias"

gi::addActions {usrNum usrDX usrDY usrPushButton} \
    -to [gi::getToolbars usrCopyViaOptions]

namespace eval usr::copyVia {
    variable ns [namespace current];

    proc init {self ctx} {
        variable ns
        if {[db::getattr ctx.editDesign.viewType] != "maskLayout"} {
            error "This command does not support non-layout view types"
        }

        set prompt "Add point to select a via"
        db::setattr prompt -of $self -value $prompt

        db::setattr engine -of $self -value \
            [de::createSelectionEngine \
                -filterProc ${ns}::_filter ]
    }

    proc _filter {self figure} {
        if {"Via" == [db::getattr objType -of $figure] } {
            return 1
        } else {
            return 0
        }
    }

    proc _buildTextWidget1 { window } {
        return [gi::createTextInput -label "DX/DY:" \
            -width 10 -value 0 ]
    }
    proc _buildTextWidget2 { window } {
        return [gi::createTextInput -width 10 -value 0 ]
    }
}
```

```
proc _buidlNumWidget { window } {
    return [gi::createNumberInput -label Num: -width 5]
}

proc _buildButtonWidget { window } {
    set ns [namespace current];
    return [gi::createPushButton -icon rules_setup_32 \
-execProc ${ns}::_copyVia ]
}

proc _copyVia { widget } {
    set t [gi::getToolbars
deCommandOptions -from [gi::getActiveWindow ]];
    set dx [db::getAttr widget.value -of [gi::getActions \
usrDX -from $t]];
    set dy [db::getAttr widget.value -of [gi::getActions \
usrDY -from $t]];
    set num [db::getAttr widget.value -of
[gi::getActions \
usrNum -from $t]];
    set selected [db::getAttr object -of
[de::getSelected \
-design [ed]]];
    set x 0; set y 0
    for {set i 0} {$i < $num} {incr i} {
        set x [expr $x + $dx] ; set y [expr $y + $dy]
        le::copy $selected -dx $x -dy $y -anchor {0 0}
    }
}
}
```

de::createShapeEngine

Description

Creates a shape engine capable of providing point, line, rectangle and polygon input from a Tcl-defined interactive command.

Syntax

```
de::createShapeEngine -shapeType <enum> [-angleMode <enum>] -completeProc
[<variant>] [-pointsChangedProc [<variant>]] [-stretchProc [<variant>]]
[-numPoints <int>] [-draw <bool>]
```

Returns

<deShapeEngine>: A deShapeEngine object

Arguments

-shapeType

Type of point input: {line | point | polygon | rectangle}.

`-angleMode`

Optional angle mode to restrict edges of multi-point objects: {anyAngle | diagonal | orthogonal | xFirst | yFirst}.

Default: anyAngle

`-completeProc`

The name of the procedure to call when input is complete.

`-pointsChangedProc`

Optional procedure to call when points are added or removed.

`-stretchProc`

Optional procedure to call when the runtime shape is stretched by a cursor motion before adding a new point.

`-numPoints`

Optional number of points requested. Specifying 0 indicates that the input is terminated manually by the user.

Default: ""

`-draw`

Draw points as a white line.

Default: true

Examples

See [Creating Custom Tcl User Commands](#) for more information and examples.

In this example, when the `usr::getDistance` command is active, you can draw a line by clicking the canvas and the distance between these two points is shown in the console.

```
de::createCommand usr::getDistance -type interactive \
  -description "Get Distance" -category user -label Distance

namespace eval usr::getDistance {
  proc init {self ctx} {
    #Start shape engine
    db::setAttr engine -of $self -value \
      [de::createShapeEngine \
        -shapeType line \
        -numPoints 2 \
        -pointsChangedProc usr::getDistance::_prompt \
        -completeProc usr::getDistance::_shapeComplete ]

    db::setAttr prompt -of $self -value "Add first point"
  }
}
```

```

proc _prompt {self} {
    db::setAttr prompt -of $self -value "Add second point \
        to calculate"
}

proc _shapeComplete {self} {
    set ctx [db::getNext [db::getAttr contexts -of $self]]
    set design [db::getAttr editDesign -of $ctx]
    set points [db::getAttr points -of [db::getAttr engine \
        -of $self]];
    set dx [expr abs([lindex $points 0 0] - [lindex $points 1 0])]
    set dy [expr abs([lindex $points 0 1] - [lindex $points 1 1])]
    puts "Distance [expr sqrt($dx*$dx + $dy*$dy)]"
    db::setAttr prompt -of $self -value "Add first point"
}
}

```

de::createTemplate

Description

Creates a new template with the given name in the given OA library.

Syntax

```

de::createTemplate <name: string> (-netlist <dbCollection | {<deFigure>}
    | {<oaFig>}> | -of <dmCellView>)
-libName <dmLibName> [-description <string>]

```

Returns

<deTemplate> : The newly created template.

Arguments

name

Name of new template cell.

-libName

Name of library to contain new template.

-netlist

Collection of oaFig | deFigure | lxInst | dbNetlistInst objects containing instances that will define the template netlist structure.

-of

Cellview to use for the template netlist structure.

-description

User supplied description of the template.

Examples

```
# Create template from selection in design schematic
de::createTemplate invTemplate -libName DesignLibrary -netlist
[de::getSelected -design [ed]]
    -description "Inverter template"

# Get templates from DesignLibrary lib and print their names
set templates [de::getTemplates -libName DesignLibrary]
puts [db::createList [db::getattr templates.name]]
```

de::createTool

Description

Creates a new tool object. If the tool already exists, it will be destroyed. Note that internally defined tools cannot be destroyed.

Procedures may be defined for each tool state change.

- Initialization: Called once per context as the tool is requested by the user.

If an error is returned from this procedure, the tool is immediately shut down without calling the shutdown procedure. This is not true for any other procedures listed here.

- Shutdown: Called once the tool has been disabled by the user.
- Activation: Called after the tool has been initialized and each time the tool become active again after being deactivated, such as when descending into an incompatible view type.
- Deactivation: Called when the context becomes invalid, such as when descending into an incompatible view type.
- Decoration: Called after activation and each time the tool needs to redecorate the window due to a descend operation changing the window type.

This may be called multiple times per context and it is the tool's responsibility to undo all modifications, even for window types which are not currently visible (such as parent designs).

The signature of all procedures is:

```
proc procName {context} {...}
```

Although contexts commonly have a window, it is possible to define tools which work on headless contexts. Each procedure may obtain the associated window via the window

attribute on the passed context object but should be careful to consider the case where this attribute is unset.

This object may be destroyed with the `db::destroy` command.

Synopsys strongly recommends that tools defer as much initialization as possible until first used. To do this, The path to a secondary initialization file using the `-startup` argument.

Syntax

```
de::createTool <name: string> -title <string> [-initProc [<variant>]]  
[-doneProc [<variant>]] [-activateProc [<variant>]] [-deactivateProc  
[<variant>]] [-decorateProc [<variant>]]
```

Returns

Newly created tool (`deTool`)

Arguments

`name`

Unique string identifier for new tool.

`-title <string>`

String that will be shown in various interfaces such as the **Tools** menu.

`-initProc <string>`

Tcl procedure, which takes one argument, the context in which the tool is being initialized. Procedure is responsible for initializing resources, checking out licenses, and so on.

`-doneProc <string>`

Tcl procedure, which takes one argument, the context in which the tool is being shutdown due to the context closing or cellview being closed. Releases resources, licenses, and so on.

`-activateProc`

Tcl procedure which is called after `-initProc` and when re-activated after previously being deactivated.

`-deactivateProc`

Tcl procedure which is called when the tool is no longer valid for the given context (that is, descend into unsupported editor).

`-decorateProc`

Tcl procedure which is called after the editor gets a new window type for decoration. This method is called after `activateProc` and may be called multiple times as context's window type changes.

Examples

```
# Create init proc for new tool
proc init {context} {
    set action {dbShowImportVerilog}
    set menu [gi::createMenu data_menu -title DataIO]
    gi::addActions $action -to $menu
}
# Create decorate proc for new tool
proc decorate {context} {
    set win [db::getattr window -of $context]
    set menu [gi::getMenus data_menu]
    set hm [db::getNext [gi::getMenus -from $win -filter {%title == "Help"}]]
    gi::addMenu $menu -to $win -before $hm
}
# Create done proc for new tool
proc done {context} {
    set win [db::getattr window -of $context]
    set menu [gi::getMenus -from $win -filter {%name=="data_menu"}]
    gi::remove $menu
}
# Create new tool
set tool [de::createTool dataTool -title "DataIO" -initProc init
    -decorateProc decorate -doneProc done]
# Add new tool to Schematic Editor
de::addTool $tool -editor seSchematic
gi::addActions dataTool -to [gi::getMenus seSchToolsMenu]
```

de::crossCheckViews

Description

Performs terminal consistency checks across a set of cellviews. The set of cellviews will be defined by a list of view names to be used from a given cell. If you do not supply the list of view names to check, the deCrossCheckViews preference value will be consulted for a list of view names. The view names will be used in conjunction with the containing dmCell of the design or cellview from the -design argument to construct the list of dmCellViews internally. If the preference value is the empty string, all views of the dmCell of the design or cellview from the -design argument will be checked.

Syntax

```
de::crossCheckViews [<name: [string]>] -design <deContext | dmCellView
    | dmCellViewName> [-checkOnly <bool>] [-caseInsensitive <bool>]
    [-severity <enum>] [-checkPortOrder <bool>] [-checkTerminalType <bool>]
    [-ignoreEmptyPortOrder <
bool>] [-verbose <bool>]
```

Returns

Collection of violations created. ({<deViolation>})

Errors

Severity of 'info' given.

Arguments

`name`

List of views or cellviews in the library/cell/view format to compare with the control design.

`-design dmCellView | oaDesign`

Design which is compared with all the given cellviews.

`-checkOnly <bool>`

Indicates whether marker objects should be created in the designs containing consistency errors.

Default: false.

`-caseInsensitive <bool>`

Indicates whether the terminal name matching should be case insensitive.

Default: deCrossCheckCaseInsensitive

`-severity`

Severity level for violations: {error | warning}.

`-checkPortOrder`

Indicates whether port orders should be checked. Default: true

`-checkTerminalType`

Indicates whether terminal types should be checked. Default: true

`-ignoreEmptyPortOrder`

Indicates whether port order check is ignored if main checked view has no portOrder property. Default: true

`-verbose`

Indicates whether message should be printed when no violation is found.
Default: true

Examples

```
set results [de::crossCheckViews {pll/vco/symbol pll/vco/veriloga}
    -design pll/vco/schematic]
# Print violations
db::foreach res $results {
```

```
puts "LCV: [db::getAttr res.cellview.libName]/[db::getAttr  
res.cellview.cellName]/[db::getAttr res.cellview.name] Description:  
[db::getAttr res.description]"  
}
```

de::cut

Description

Copies supplied objects to the clipboard and then deletes the supplied objects.

Syntax

```
de::cut <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
[-anchor <point>]
```

Returns

Collection of objects that were just copied to the clipboard. ({*oaObject*} | {*defigure*})

Arguments

objects

List of objects to cut.

-anchor

Anchor point for paste.

Errors

- An *oaGroup* object is supplied.
- The collection of inputted *oaObjects* or *deFigures* are not from the same *oaDesign*.
- An interactive paste command is active, such as *ile::paste* or *ise::paste*.

Examples

The following example copies all shapes on m1 drawing LPP into clipboard and then deletes the supplied objects.

```
set design [ed]  
de::cut [db::getShapes -of $design -lpp {m1 drawing}]
```

de::cycleActiveFigure

Description

Cycles through the figures located around the cursor coordinates. This command also changes the cursor's coordinates if the `-point` argument is provided. If `-point` is omitted, the cursor position is unchanged and this command cycles to the next nearest object. However, if `-point` is specified, the cursor is moved and the set of objects near the cursor are recomputed.

Syntax

```
de::cycleActiveFigure [<window: deCanvas | giWindow | int>] [-point  
  <point>] [-direction <enum>]
```

Returns

Active figure (defigure)

Arguments

`window`

Window or canvas to cycle in.

Default: active window

`-point <point>`

Coordinate of the center of the selection search radius.

Default: cursor position

`-direction`

Cycling direction: {next | prev}.

Default is: next

Errors

- An invalid window object is passed.
- Invalid point is passed.
- Invalid direction is specified.

Examples

```
# Create three overlapping rectangles in layout design
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m2 drawing"
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m3 drawing"
# Run de::cycleActiveFigure using point over the rectangles
```

```
set activeFigure [de::cycleActiveFigure -point {0.1 0}]
# Print the layer of the active rectangle
db::getattr activeFigure.LPP.layer.name
# Cycle to next rectangle and print its layer
set activeFigure [de::cycleActiveFigure -point {0.1 0}]
db::getattr activeFigure.LPP.layer.name
# Cycle back to previous rectangle and print its layer
set activeFigure [de::cycleActiveFigure -point {0.1 0} -direction prev]
db::getattr activeFigure.LPP.layer.name
```

de::cycleTapObject

Description

Cycles through the objects in the tap set. The tap set is the set of objects that were near the cursor when the first point was added. This command alters the initial point of the current shape as well as the object from which attributes such as initial LPP or path width were tapped.

Syntax

```
de::cycleTapObject [-window <giWindow | unsigned int> | -context
<deContext>] [-direction <enum>]
```

Returns

<deFigure> : Tap object (defigure)

Arguments

-window

Window to cycle in.

-context

Context in which to cycle.

-direction

Direction to cycle: { next | prev }. Default: next

Errors

- Invalid window ID.
- Window is not a layout, schematic or symbol window.
- Current command is not operating on a tapped object.

de::deactivateTools

Description

Deactivates one or more tools within the specified context. An already deactivated command will not alter its state.

Syntax

```
de::deactivateTools <tools: dbCollection>
```

Arguments

tools

Collection of deToolInst objects to deactivate in the specified context. To retrieve these objects, retrieve the 'tools' attribute on the deEditorInst object. The deTool argument value is deprecated.

de::defineOrigin

Description

Moves all objects within the specified design by the inverse of the vector given by the specified coordinate.

Syntax

```
de::defineOrigin <point: point> -design <oaDesign>
```

Arguments

point

Point which will be 0,0 after translating all objects.

-design

Design in which to move objects.

Examples

```
set design [ed]
de::defineOrigin {1 1} -design $design
```

de::deleteBookmarks

Description

Deletes the specified bookmarks.

An interactive version of this command is not available.

Syntax

```
de::deleteBookmarks <bookmarks: {<deBookmark>}>
```

Arguments

{[deBookmark](#)}

The collection of bookmarks to delete.

Examples

```
# Create new bookmark
de::createBookmark newBookmark -window [gi::getWindows $id]

# Get and delete that bookmarkset
bm [de::getBookmarks newBookmark]
de::deleteBookmark $bm
```

de::deprecateCommand

Description

Marks a command, argument, argument value type or namespace as deprecated or defunct.

Syntax

```
de::deprecateCommand (-namespace <string> | -command <deCommand  
| string>) [-argument <string>] [-argumentValueType <string>]  
[-replacement <string>] [-defunct <bool>]
```

Returns

<none>

Arguments

-namespace

Namespace to deprecate.

-command

Command or procedure name to deprecate.

-argument

Argument to deprecate.

-argumentValueType

Argument value type to deprecate.

`-replacement`

Name of replacement command, argument, value type, or namespace to see regarding deprecating or making this item defunct.

`-defunct`

Mark object defunct.

Default: false

Examples

```
# Create simple command and then deprecate it
de::createCommand de::testCommand -category Foundation -description
  "Command for deprecation"
de::deprecateCommand -command de::testCommand

# Check command help for deprecation message
de::testCommand -help
```

de::descend

Description

Descends into a new design, leaving the existing design open within the specified context.

An alternate association may be specified.

When descending into a vector or array instance, the specific instance index must be encoded in the instance name.

If the new edit file does not have an associated editor or the specified editor is invalid, this command will throw an error and leave the context unmodified.

If the design could not be open in write mode, this command will fall back to read-only mode.

Syntax

```
de::descend <name: string> -context <deContext> [-master <dmCellView |
  dmCellViewName | dmDMFile>] [-readOnly <bool>]
```

Returns

<deContext>: Modified context object (deContext)

Arguments

name

Instance name.

`-context`

Context object to modify.

`-master`

File or cellview into which to descend.

`-readOnly`

Open design in read-only mode.

Default: false

Examples

```
de::descend x4 -context [de::getActiveContext] -readOnly true -master  
DemoVCO/amp/starrc
```

de::descendInPlace

Description

Descends into a new design, leaving the former open within the specified context.

Syntax

```
de::descendInPlace <lineage: oaHierPath | oaInst | string> -context  
<deContext> [-readOnly <bool>]
```

Returns

<deContext>: Modified context object.

Arguments

`lineage`

Instance, instance name, or lineage specification.

`-context`

Context object to modify.

`-readOnly`

Open design in read-only mode.

Default: false

Examples

In the following example, `de::descendInPlace` opens a master design of the X1 instance, without closing the parent design or changing the viewport.

```
de::descendInPlace X1 -context [de::getActiveContext]
```

de::descendIntoGroup

Description

Descends into a figure group.

Syntax

```
de::descendIntoGroup <object: oaFigGroup | string> -context <deContext>
```

Returns

<deContext>: Modified context object

Arguments

`object`

Figure group or name of a figure group in the edit design to edit.

`-context`

Context object to modify.

Examples

```
# Descend into G1 group
de::descendIntoGroup G1 -context [de::getActiveContext]
```

de::deselect

Description

Deselects the specified object, sub-object or collection. It is expected that the object will commonly be the return value of `de::getActiveFigure` or one of the query commands defined above.

If an object is passed which contains hierarchy, the root instance will be deselected within its design.

Objects are deselected from the select set of their design.

Remaining selected objects are then returned.

Syntax

```
de::deselect <objects: {<deFigure>} | {<oaFig>} | {<oaFigGroup>} |  
{<oaInst>} | {<oaShape>} | {<oaVia>}>
```

Returns

Collection of selected figures ({deFigure}).

Arguments

```
{deFigure} | {oaFig}
```

Objects to deselect.

Examples

```
# Select all instances in active schematic design  
de::select [db::getInsts -of [ed]]  
# Deselect one instance with name "I0"  
de::deselect [db::getInsts I0 -of [ed]]  
# Other instances remain selected
```

de::deselectAll

Description

Deselects all selected objects in the edit design of the specified context.

Syntax

```
de::deselectAll <context: deContext>
```

Arguments

```
deContext
```

Design in which to deselect all objects.

Examples

```
# Select all instances in active schematic design  
de::select [db::getInsts -of [ed]]  
# Select also all wires in active schematic design  
de::select [de::getFigures -design [ed] \ -objectTypes [de::getObjectFilters seSchWire]]  
# Deselect all selected objects  
de::deselectAll [de::getActiveContext]
```

de::deselectAllLogic

Description

Deselects all logic in the specified and subsequently linked contexts.

Syntax

```
de::deselectAllLogic -context <dbCollection>
```

Returns

<none>

Arguments

-context

Context within which to deselect logic.

Examples

```
# Open Design Navigator to see logically selected objects
# Logically select I0 instance and IN net
set context [de::getActiveContext]
set design [db::getattr context.editDesign]
de::selectLogic [db::getNext [db::getInsts I0 -of $design]] -context
    $context
de::selectLogic [db::getNext [db::getNets IN -of $design]] -context
    $context
# Logically deselect all instances
de::deselectAllLogic -context [de::getActiveContext]
```

de::deselectLogic

Description

Deselects logic within the specified design context.

Syntax

```
de::deselectLogic <objects: dbCollection | {<oaFig>} | {<oaFigGroup>}
    | {<oaGroup>} | {<oaInst>} | {<oaInstTerm>} | {<oaNet>} | {<oaPin>} |
    {<oaTerm>}> -context <deContext>
```

Returns

<dbCollection> : Deselected objects which were previously selected

Arguments

objects

Logic to deselect.

-context

Context within which to deselect logic.

Examples

```
# Open Design Navigator to see logically selected objects.
# Logically select I0 instance and IN net
set context [de::getActiveContext]
set design [db::getattr context.editDesign]
de::selectLogic [db::getNext [db::getInsts I0 -of $design]] -context
    $context
de::selectLogic [db::getNext [db::getNets IN -of $design]] -context
    $context
# Logically deselect I0 instance
de::deselectLogic [db::getNext [db::getInsts I0 -of $design]] -context
    $context
```

de::disconnect

Disconnect a design compare session

Syntax

```
de::disconnect -context <deContext>
```

Returns

<none>

Arguments

-context

Design context from which to disconnect design compare session

de::enableColorAnnotator

Description

Enables a color annotator in the given design context for the given application data.

Syntax

```
de::enableColorAnnotator <context: deContext> -applicationName <string>
    -dbHandle <string> -dbTableNames [<string>] -objectType <enum>
    [-objectNameColumn <string>] [-gradientBy <dbCollection>] [-colorizeBy
    <dbCollection>] [-occurrenceColumn <string>]
```

Returns

<deColorAnnotator> : Newly created color annotator object.

Arguments

`context`

Design context.

`-applicationName`

Color annotator application name.

`-dbHandle`

Database handle.

`-dbTableNames`

List of tables to be search to determine which objects to assign color.

`-objectType`

Object type for which color annotation is done: { instance | net }.

`-objectNameColumn`

Name of the column containing the object name or occurrence/<objectName> of the objects on which color annotation is done. Default: name

`-gradientBy`

Collection of gradient objects.

`-colorizeBy`

Collection of color expression objects.

`-occurrenceColumn`

The column name (or expression) that contains the occurrence path of the objects on which color annotation is done. Should be specified if the column specified in `-objectNameColumn` field does not contain the occurrence path.

de::enablePartitioningDisplay

Description

Enables or disables the partition visualization in the given design context.

Syntax

```
de::enablePartitioningDisplay -context <deContext> -netlister  
  <nlAMSNetlister> [-enable <bool>] [-simulatorReportsPath <string>]  
  [-onlyRefresh <bool>]
```

Arguments

`-context`

Design context.

`-netlist`

AMS netlist according to which the partition is made.

`-enable`

Enables or disables partition visualization.

Default: true

`-simulatorReportsPath`

Path to simulation reports for partition recognition. Should point to folder containing subfolders `results` and `netlist`, or immediately to `results` (`../netlist` should also exist).

`-onlyRefresh`

If true, the partition domains information is not updated when enabling the visualization. Default: false

Examples

```
# Enable partition visualization in active context
set n [nl::getAMSNetlists AMS]
de::enablePartitioningDisplay -context [de::getActiveContext] -netlist
  $n
```

de::enableRotationAndScaling

Description

Defines a superMaster using oaPcellDef snpsRotateAndScale for the given collection of dmCellView objects.

Syntax

```
de::enableRotationAndScaling <object: dmCellViewName | {<dmCellView>} |
  {<dmDMFile>}>
```

Returns

<int> : int return value.

Arguments

object

One or more cellview (primary file) or DM file.

de::endDrag

Description

Moves the cursor to the given point and sends an endDrag event to the active command in a design editor window. This event ends a drag for interactive commands that support a click-drag-release use model.

The point argument is in the view design's coordinate space.

Syntax

```
de::endDrag <point: point> [-window <giWindow | unsigned int> | -context  
    <deContext>]
```

Arguments

point

Point at which to start the drag event.

-window

Window in which to send the event.

-context

Context in which to send the event.

Examples

```
# Create three rectangles in the layout design
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"
le::createRectangle {{0.3 -0.1} {0.5 0.1}} -design [ed] -lpp "m1 drawing"
le::createRectangle {{0.6 -0.1} {0.8 0.1}} -design [ed] -lpp "m1 drawing"
# Start Move command and select two of them by drag commands
ile::move
de::startDrag {-0.15 0.15} -context [de::getActiveContext]
de::endDrag {0.55 -0.2} -context [de::getActiveContext]
de::addPoint {0.25 0}
```

de::endLog

Description

Closes a secondary log file previously opened with `de::startLog`.

Syntax

```
de::endLog <file: string>
```

Arguments

<string>

Full file system path to the secondary log file.

Examples

```
de::startLog test.log -banner "My Log"
# Redirect messages to another log.file
de::sendMessage "This is information" -logFiles test.log
de::sendMessage "This is warning" -severity warning -logFiles test.log
de::sendMessage "This is error" -severity error -logFiles test.log
de::endLog test.log
```

de::endTransaction

Description

Ends the set of operations in the current transaction.

Syntax

```
de::endTransaction <transaction: deTransaction> [-dirtyies <bool>]
```

Arguments

[deTransaction](#)

Transaction to end.

-dirtyies <bool>

This transaction should be considered to be design dirtying.

Default: true.

Examples

In this example, multiple create commands call are wrapped in single transaction.

```
set design [ed]
set x 1
set y 1
set tr [de::startTransaction "Create Rectangles" -design $design]
db::foreach lpp [de::getLPPs -from $design] {
    le::createRectangle [list [list $x $y] [list [expr $x+0.5] [expr
    $y+0.5]]] -design $design -lpp [db::getAttr lpp.lpp]
    incr x
    if {$x > 10} {
```

```
        set x 1
        incr y
    }
}
de::endTransaction $tr
```

de::errorViewerLoadResult

Description

Loads result to Error Viewer.

Syntax

```
de::errorViewerLoadResult -window <int> [-createToEditDesign <bool>]
[-speedMode <bool>] [-type <enum>] [-title <string>] [-toolName <string>]
[-ruleFile <string>] [-waiveFile <string>] -resultFile <string>
[-gpdLoadType <enum>]
```

Arguments

-window

Loading result to error viewer is associated with the given window index or the current edit design if this option is not specified.

-createToEditDesign

If true, create markers to the edit design. If false, the design is overlaid.

Default: false.

-speedMode

Use speed mode to load result. This option is only valid for calibre type result.

Default: false

-type

Specifies the result file type: calibre | gpd.

Default: calibre

-title

Specifies the title of the new tab. This only valid when createToEditDesign is false.

-toolName

Specifies the tool name.

`-ruleFile`

Specifies the rule file.

`-waiveFile`

Specifies the waive file.

`-resultFile`

Specifies the result file.

`-gpdLoadType`

Specifies the GPD loading type: All | Open | Short. Default All

Examples

```
# Load results from errors.result file to the current window
de::errorViewerLoadResult -window [db::getattr id -of
  [gi::getActiveWindow]] -resultFile errors.result
```

de::exportBookmarks

Description

Exports the specified bookmarks to a file.

An interactive version of this command is not available.

Syntax

```
de::exportBookmarks <bookmarks: {<deBookmark>}> -filePath <string>
  [-force <bool>]
```

Arguments

{ [deBookmark](#) }

The collection of bookmarks to export.

`-filePath <string>`

The path to the destination bookmark file.

`-force <bool>`

Specifies whether to overwrite existing bookmarks. When true, the command will overwrite existing bookmarks. When false, the command will display an error if the specified file exists.

Default: true

Examples

```
# Create new bookmark  de::createBookmark newBookmark -window
[gi::getWindows $id]# Get created bookmark set bm [de::getBookmarks]#
Export bookmarks to xml file
de::exportBookmarks $bm -filePath ./exportedBookmarks.xml
```

de::exportProbeCellSettings

Description

Exports **Probe Trace Through Cell** settings to the specified file.

Syntax

```
de::exportProbeCellSettings [-value <string>] -file <string>
```

Arguments

-value

The **Probe Trace Through Cell** settings to save.

-file

The file name for the exported **Probe Trace Through Cell** settings.

de::extractInterfaceElements

Description

Extracts interface elements from the simulation report.

Syntax

```
de::extractInterfaceElements -context <heContext> -netlister
<nlAMSNetlister> -path <string>
```

Returns

<dbCollection> : Collection of interface element objects successfully extracted from reports.

Arguments

-context

Hierarchy context in which the interface elements are created.

-netlister

AMS netlister according to which the partition is made.

`-path`

Path to simulation reports for interface elements extraction. Should point to folder containing subfolders 'results' and 'netlist', or immediately to 'results' ('../netlist' should also exist).

Errors

- Specified simulation path does not exist (also other file system problems: missing files/folders, no access).
- Name mapping is not found in the simulation reports.
- Specified hierarchy context does not correspond to the HC in simulation reports.

Examples

```
# Extract interface elements from simulation results in latestResults
directory
set context [de::getActiveContext]
set design [db::getAttr context.editDesign]
set net [db::getNets OUT -of $design]
set hc [db::getAttr context.hierarchy]
set n [nl::getAMSNetlisters AMS]
set col [de::extractInterfaceElements -context $hc -netlist $n
-path ./latestResults]

# Print nets and directions of the extracted interface elements
db::foreach ie $col {
    puts "[db::getAttr ie.net] [db::getAttr ie.direction]"
}
```

de::findMessage

Description

Retrieves an externalized message string by its messageID and specifies arguments for formatting options.

Syntax

```
de::findMessage <messageID: string> [-args [<string>]]
```

Returns

The message string found and formatted (<string>)

Arguments

<string>

The message ID of the message to be returned.

`-args [<string>]`

List of specifications with which to fill out % arguments within the message string.

Default: {}

Errors

- Message not found.
- Missing or invalid format arguments.

Examples

```
# Find message without specifying arguments
de::findMessage COMMAND-011
```

```
# Find message with specifying arguments
de::findMessage COMMAND-011 -args {de::testCommand -design}
```

de::finishEvalTextRendering

Description

Invokes and waits for the completion of delayed text evaluations in a window canvas.

Syntax

```
de::finishEvalTextRendering [-window <deCanvas | giWindow | int>]
                             [-incremental <bool>]
```

Returns

<none>

Arguments

`-window`

Window or canvas.

`-incremental`

When true, the command will only operate on the next batch of pending labels within a `deEvalTextRenderTimeout` timeout. Default: false

de::fit

Description

Adjusts the viewport to fit the grid or any set of objects.

When the cell has no data, `de::fit` fits to the grid and the origin of the cell is at the center of the canvas.

This command displays a warning if the specified object set is empty.

There is no interactive version of this command.

Syntax

```
de::fit [-fitGrid <bool> | -fitView <bool> | -fitEdit <bool> | -set  
<{<deFigure>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaShape>} |  
{<oaVia>}}] [-window <deCanvas | giWindow | int>] [-scale <float>]
```

Returns

The new viewport (<box>).

Arguments

`-window`

The window or canvas to modify.

Default: active window

`-fitGrid <bool>`

Displays the grid within the viewport.

The default values of `-fitEdit`, `-fitGrid`, and `-fitView` are only used if all three arguments are not specified.

Default: false

`-fitView <bool>`

Displays the design within the viewport.

The default values of `-fitEdit`, `-fitGrid`, and `-fitView` are only used if all three arguments are not specified.

Default: false

`-fitEdit <bool>`

Fits the viewport to the edit design.

The default values of `-fitEdit`, `-fitGrid`, and `-fitView` are only used if all three arguments are not specified.

Default: true

`-set {<deFigure>} | {<oaFig>}`

Displays only the specified collection of objects within the viewport.

`-scale <float>`

The maximum fraction of each axis that the data extents may cover. The range is (0, 1].

Default: 0.9

Errors

- More than one of `-fitEdit`, `-fitGrid`, and `-fitView` was specified together with `-set`.
- The window ID was invalid.
- There was no open design in the specified window.

Examples

In this example, `de::fit` adjusts the viewport to fit itself to the selected object in the given design.

```
de::fit -set [de::getSelected -design $oaDesign] -scale 0.8
```

de::getActiveCommand

Description

Returns the name of the active command.

Syntax

```
de::getActiveCommand [-window <giWindow | unsigned int> | -context  
  <deContext>]
```

Returns

<deCommand>: The active command. "" if none.

Arguments

`-window`

Window(id) in which to query the command.

`-context`

Context in which to query the command.

Examples

```
# Start the move command in the layout design ile::move# Get active  
command and print its nameset command [de::getActiveCommand]  
puts [db::getattr command.name]
```

de::getActiveContext

Description

Returns the active context, which corresponds to the last focused editor window.

Syntax

```
de::getActiveContext
```

Returns

Active context (deContext).

Errors

No editor windows open.

Arguments

None.

Examples

```
# Open designLibrary/NAND/layout
de::open [dm::getCellViews layout -libName designLibrary -cellName NAND]
# Get active context and print cell name
set context [de::getActiveContext]
puts [db::getattr c.editDesign.cellName]
```

de::getActiveEditorWindow

Description

Returns the active (last focused) editor window.

Syntax

```
de::getActiveEditorWindow
```

Examples

```
# Open designLibrary/NAND/layout
de::open [dm::getCellViews layout -libName designLibrary -cellName NAND]
# Get active editor window and print its id
set activeWindow [de::getActiveEditorWindow]
puts [db::getattr activeWindow.id]
```

de::getActiveFigure

Description

Returns the active object. If a point is provided, this command approximates an active object retrieval by taking the parameters that were used to retrieve the active object (cursor location, number of objects to cycle through) and producing the same object. Returns an empty string if there is no active figure, if the figure was not found with the given arguments, or if the `-index` is a value outside of the range of the active set.

Syntax

```
de::getActiveFigure [<window: deCanvas | giWindow | int>] [-point  
  <point>] [-index <unsigned int>] [-intent <enum>]
```

Returns

Active object (defigure).

Errors

An invalid window object is passed.

Arguments

`giWindow` | `<int>`

Window or canvas to get Active Object from.

Default: active window

`-point <point>`

If specified, calculates a new active set and returns the first figure or the one specified by `-index`.

Default: Does not calculate new active figure

`-index <int>`

Returns the *n*th item in the active set.

Default: Current cycle count, or 0 if `-point` is specified

`-intent`

Specifies selection mode: {deselect | none | select}.

Default: none

Examples

```
# Create a rectangle in the layout design  
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"
```

```
# Set cursor on the rectangle and get active figure
de::setCursor -point {0.1 0}
set activeFigure [de::getActiveFigure]
# Print active figure type
puts [db::getattr activeFigure.objType]
```

de::getActiveLPP

Description

Returns the active LPP as a deLPP object.

Syntax

```
de::getActiveLPP -design <oaDesign | deContext>
```

Returns

LPP to set active ([deLPP](#)).

Arguments

-design

Design for which this applies. If a deContext is specified, the edit design is used.

Errors

- The specified design is a not a layout design.
- The specified design is not opened in a design editor.

Examples

In this example, `de::getActiveLPP` returns the active LPP object in the active context.

```
de::getActiveLPP -design [de::getActiveContext]
```

de::getApplications

Description

Returns a collection of registered applications, optionally filtered by name.

Syntax

```
de::getApplications [<name: string>] [-filter <string>]
```

Returns

<dbCollection>: Defined applications.

Arguments

`name`

Optional glob-style pattern to match against each application's name.

Default: `*`

`-filter`

Standard attribute filter.

Examples

In this example, `de::getApplications` returns the existing applications, and prints the applications names and titles on the console.

```
db::foreach app [de::getApplications] {  
    puts "[db::getAttr name -of $app] \  
        - [db::getAttr title -of $app]"  
}
```

de::getAssociations

Description

Returns a collection recently defined associations.

Syntax

```
de::getAssociations [-filter <string>]
```

Returns

`<dbCollection>`: Defined associations (`{deAssociation}`).

Arguments

`-filter`

Standard attribute filter.

Examples

```
# Get associations and print their file names and associated application  
title  
set associations [de::getAssociations]  
db::foreach association $associations {  
    puts "[db::getAttr association.filename] [db::getAttr  
    association.application.title]"  
}
```

de::getBackgroundImporters

Gets background importers for digital designs.

Syntax

```
de::getBackgroundImporters <objects: {<dmCell>} | {<dmCellView>} |  
{<oaLib>}>
```

Returns

<dbCollection> : Background importers

Arguments

objects

A collection of selected libraries, cells, or cellviews that are imported.

de::getBBox

Description

Returns a merged bounding box from a collection of figures.

Syntax

```
de::getBBox <figures: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaShape>} | {<oaVia>}>
```

Returns

<box> : Merged bBox

Arguments

figures

Collection of figures.

Examples

The following example returns a merged bounding box from a collection of figures.

```
# Get edit design from active context  
set context [de::getActiveContext]  
set design [db::getattr context.editDesign]  
  
# Get all figures from top design  
set figures [de::getFigures -design $design]  
  
# Get bBox of all figures in the top design  
de::getBBox $figures
```

de::getBinsForGradient

Description

Gets bins for the given gradient and database tables.

Syntax

```
de::getBinsForGradient -gradient <deGradient> -dbHandle <string>  
                        -dbTableNames [<string>]
```

Returns

<dbCollection> : Collection of bin objects

Arguments

-gradient

Gradient object.

-dbHandle

Database handle.

-dbTableNames

List of tables to search to determine the objects for color assignment.

de::getBookmarks

Description

Returns bookmarks with optional filtering by bookmark name. The command returns an empty collection if no bookmark matches the pattern.

An interactive version of this command is not available.

Syntax

```
de::getBookmarks [<pattern: string>] [-regexp <bool>]
```

Returns

A collection of bookmarks ({<deBookmark>}).

Arguments

<string>

A pattern for bookmark names.

Default: match all

`-regexp <bool>`

When true, the pattern is a regular expression. Otherwise, the pattern uses glob-style wildcards.

Default: false

Examples

```
# Create two new bookmarks
de::createBookmark firstBookmark -window [gi::getWindows $id]
de::createBookmark secondBookmark -window [gi::getWindows $id]
# Get all bookmarks and print their names
set bm [de::getBookmarks]
puts [db::createList [db::getattr bm.name]]
```

de::getBreakpoints

Description

Returns a Tcl list of breakpoint line numbers.

Syntax

`de::getBreakpoints`

Returns

`<string>` : Tcl list of breakpoint line numbers

Examples

```
# Replay replay.log file in interactive mode
de::replay replay.log -interactive true
# Set two breakpoints
de::setBreakpoint 20
de::setBreakpoint 32
# Get defined breakpoints
# It will return the list {20 32}
de::getBreakpoints
```

de::getCapabilities

Description

Returns a collection of all known capabilities.

Syntax

`de::getCapabilities [<name: string>] [-filter <string>]`

Returns

<dbCollection> : All known capabilities.

Arguments

name

Glob-style pattern to match against capability name. Default: *

-filter

Standard attribute filter.

Examples

```
# Get capabilities and print their names
set caps [de::getCapabilities]
puts [db::createList [db::getAttr caps.name]]
```

de::getClipboardLibs

Description

Returns a collection of clipboard libraries.

To open a clipboard library, use [de::openClipboardLib](#).

The OpenAccess library name includes a prefix (*deClipboard_*) to avoid conflicting with design libraries. The prefix is not required for the unnamed argument, but is required when filtering with *-filter*.

Syntax

```
de::getClipboardLibs <pattern: string> -filter <string>
```

Returns

<oaLib>: Collection of clipboard libraries.

Arguments

pattern

Glob-style pattern to match the clipboard name (for example, Recent).

Default: Match all

-filter

Standard filter expression.

Default: Unfiltered

Examples

```
# Open new clipboard library named clipLib
de::openClipboardLib clipLib -path ./clipLib

# Get clipboard libraries and print their names
set libs [de::getClipboardLibs]
puts [db::createList [db::getAttr libs.name]]
```

de::getClips

Description

Returns a collection of clipboards. If a clipboard library is not specified, returns recently copied clipboards.

To create a clipboard, use [de::copy](#) or [ile::copyToClipboard](#). To destroy a clipboard, use [db::destroy](#).

Syntax

```
de::getClips [<pattern: string>] [-lib <oaLib>] [-viewType <string>]
             [-filter <string>]
```

Returns

<[deClipboard](#)>: Collection of clipboards.

Arguments

pattern

Glob-style pattern to match the clipboard name (for example, clip5).

Default: Match all

-lib

Clipboard library from which to return clipboards.

Default: Recent

-viewType

Optionally filters returned clipboards by view type.

Default: All view types

-filter

Standard filter expression.

Default: Unfiltered

Examples

```
# Copy I0 and I1 instances to clipboard
set inst [db::getInsts I0 -of [ed]]
de::copy $inst
set inst [db::getInsts I1 -of [ed]]
de::copy $inst

# Get clips and print their names
set clips [de::getClips]
puts [db::createList [db::getattr clips.name]]
```

de::getColorAnnotators

Description

Gets the enabled color annotator objects.

Syntax

```
de::getColorAnnotators [-context <deContext>] [-filter <string>]
```

Returns

<dbCollection> : Enabled color annotator objects.

Arguments

-context

Design context.

-filter

Specifies a string or expression to further filter the returned collection.

de::getCommands

Description

Returns a collection of registered commands; optionally restricted by their name.

Syntax

```
de::getCommands [<pattern: string>] [-filter <string>]
[-includeDeprecated <bool>]
```

Returns

{<deCommand>}: Collection of commands that satisfy the given filters.

Arguments

`pattern`

Glob-style pattern to match the command name.

Default: match all

`-filter <string>`

Standard attribute filter.

`-includeDeprecated`

Whether to include deprecated commands in the result.

Examples

```
# Print all registered commands from "ile" namespace
set commands [de::getCommands ile::*]
puts [db::createList [db::getAttr commands.name]]
```

de::getContexts

Description

Returns all open data contexts.

Syntax

```
de::getContexts [-window <giWindow | int>] [-file <dmDMFile>] [-filter  
  <string>]
```

Returns

<dbCollection>: All open data contexts.

Arguments

`-window` [giWindow](#) | <int>

Only returns the context associated with the given window or window ID.

`-file` [dmDMFile](#)

Only returns contexts which have the specified file open in their descend stack.

`-filter <string>`

Optional standard attribute filterStandard attribute filter.

Examples

```
# Open designLibrary/NAND/layout and designLibrary/NOR/layout
de::open [dm::getCellViews layout -libName designLibrary -cellName NAND]
```

```
de::open [dm::getCellViews layout -libName designLibrary -cellName NOR]
# Get all open contexts
set contexts [de::getContexts]
# Print their cell names
puts [db::createList [db::getattr cs.editDesign.cellName]]
```

de::getCrossProbeTools

Description

Returns a collection of registered cross probe tool objects optionally filtered by name or by tools available for a given context.

Syntax

```
de::getCrossProbeTools [<name: string>] [-context <deContext |  
heContext>]
```

Returns

{<deCrossProbeTool>}: Collection of cross probe tools.

Arguments

name

Returns cross-probe tools whose names match this glob-style pattern only.

-context

Returns cross-probe tools available for this context only.

de::getCursor

Description

Retrieves the current actual cursor location.

Syntax

```
de::getCursor [<window: deCanvas | giWindow | int>]
```

Returns

<point> : The current cursor location.

Errors

An invalid window object is passed.

Arguments

window

Window or canvas to get cursor from.

Default: active window

Examples

```
# de::getCursor will return current position of the cursor
de::getCursor
```

de::getCursorSnapPoint

Description

Queries the window for current snap point. Does not recalculate.

Syntax

```
de::getCursorSnapPoint [<window: deCanvas | giWindow | int>]
```

Returns

<point> : Current window snap point.

Errors

An invalid window object is passed.

Arguments

window

Window or canvas to get snap point from.

Default: active window

Examples

```
# Set snap spacing for layout design to 0.05 db::setPrefValue
leSnapSpacing -value {0.05 0.05} -scope [de::getActiveContext ]# Set
cursor to point out of snap gridde::setCursor -point {0.16 0.27}# Get
cursor snap point
# It will be {0.15 0.25}
de:: getCursorSnapPoint
```

de::getDBLimits

Description

Gets the database limits for the given expression.

Syntax

```
de::getDBLimits -expression <string> -dbHandle <string> -dbTableNames  
[<string>]
```

Returns

[<float>] : List of minimum and max values

Arguments

-expression

Expression to be evaluated in given database tables.

-dbHandle

Database handle.

-dbTableNames

List of tables on which given expression should be evaluated.

de::getDesignTemplates

Description

Gets the master schematic templates in the given design.

Syntax

```
de::getDesignTemplates -cellView <dmCellView | dmCellViewName>
```

Returns

{<dmCellView>} : A collection of cellviews

Arguments

-cellView

The top cellview.

de::getEditors

Description

Returns all registered editors, optionally restricted to those whose name matches a pattern and those that match the supplied filter.

Syntax

```
de::getEditors [<name: string>] [-filter <string>]
```

Returns

<dbCollection>: All registered editors.

Arguments

name

Pattern to match editor names against.

Default: Match all

-filter

Optional standard attribute filter.

Default: Match all

Examples

```
# Print all registered editors names and titles in console
db::foreach editor [de::getEditors] {
    puts "[db::getattr editor.name]: [db::getattr editor.title]"
}
```

de::getErrorViewerDatabases

Description

Gets all deErrorViewerDatabases

Syntax

```
de::getErrorViewerDatabases [<pattern: string>] [-filter <string>]
```

Returns

<dbCollection> : deErrorViewerDatabase objects matching search criteria.

Arguments

pattern

Pattern to match object name.

-filter

Standard attribute filter.

Examples

```
# Create new Error Viewer database
set design [ed]
de::createErrorViewerDatabase newErrDB -design $design -title "New
Database"
```

```
# Get Error Viewer databases and print their names
set dbs [de::getErrorViewerDatabases]
puts [db::createList [db::getAttr name -of $dbs]]
```

de::getFavoriteDeviceCategories

Description

Gets a collection of device categories for use in the Device Palette Assistant.

Syntax

```
de::getFavoriteDeviceCategories [-filter <string>]
```

Returns

<dbCollection>: Collection of device categories

Arguments

-filter

Standard attribute filter.

Examples

```
# Create favorite devices from different libraries
set cv1 [dm::findCellView symbol -cellName cap -libName analogLib]
set cv2 [dm::findCellView symbol -cellName res -libName analogLib]
set cv3 [dm::findCellView symbol -cellName Cc -libName parasitics]
de::createFavoriteDevice -master $cv1 -viewType schematic
de::createFavoriteDevice -master $cv2 -viewType schematic
de::createFavoriteDevice -master $cv3 -viewType schematic
# Find favorite devices from analogLib
set devices [de::getFavoriteDevices -filter
  {%master.libName=="analogLib"}}
# Create new favorite device category and add these devices into it
de::createFavoriteDeviceCategory fromAnalogLib -devices $devices
# Create another empty favorite device category
de::createFavoriteDeviceCategory otherDevices
# Get favorite device categories and print their names
set favCats [de::getFavoriteDeviceCategories]
puts [db::createList [db::getAttr name -of $favCats]]
```

de::getFavoriteDevices

Description

Returns a collection of favorite devices defined for the Device Palette Assistant.

Syntax

```
de::getFavoriteDevices [-filter <string>]
```

Returns

<dbCore>: Collection of devices.

Arguments

-filter

Standard attribute filter.

Examples

```
# Find cellviews for three devices
set cv1 [dm::findCellView symbol -cellName cap -libName analogLib]
set cv2 [dm::findCellView symbol -cellName res -libName analogLib]
set cv3 [dm::findCellView symbol -cellName Cc -libName parasitics]
# Add them to favorite devices
de::createFavoriteDevice -master $cv1 -viewType schematic
de::createFavoriteDevice -master $cv2 -viewType schematic
de::createFavoriteDevice -master $cv3 -viewType schematic
# Get favorite devices from analogLib and print their names
set devices [de::getFavoriteDevices -filter
  {%master.libName=="analogLib"}}
puts [db::createList [db::getattr master.cellName -of $devices]]
```

de::getFigures

Description

Returns all deFigures which touch or are enclosed by a rectangular or polygonal region or that touch a point or line. It may be combined with the `de::select` command to reproduce a select by line, rectangle or polygon command. Point selection uses the active object rather than the point specifically. Objects within child designs may be returned by passing the -depth argument. Partial objects are supported by using the -partial argument.

Returned objects are not filtered by selection, visibility or drawing filters.

Syntax

```
de::getFigures <objects: point | {<deFigure> | <oaFig> | <oaFigGroup>
  | <oaInst> | <oaShape> | <oaVia>}> [-type <enum>] [-design <oaDesign>]
  [-touch <bool>] [-depth <int>] [-partial <bool>] [-lpps {<deTechLPP>}]
  [-objectTypes {<deObjectFilters>}] [-excludeObjectTypes {<deObjectTypes>}]
  [-filter <string>] [-hierPath <oaHierPath>]
```

Returns

Objects matching criteria ({defigure}).

Arguments

`objects`

Figures or query region points.

`-type`

Indicates whether points represent lines, points, polygons, or rectangles.

Default: point if only one point is provided, rectangle if two points are provided, and polygon if more than two points are provided.

`-design`

Root design in which to find objects. The design must be opened in the design editor or an overlay created using the `de::createOverlay` command. Do not specify this argument when passing a collection of `oaFigs`.

`-touch`

Returns figures which touch the region in addition to those enclosed by it. It is implied by a line selection. Default: false

`-depth`

Maximum number of hierarchical levels to traverse. A negative number indicates all levels. Default: 0

`-partial`

Return partial figures. Default: false

`-lpps`

Returns shapes on the specified layer-purpose pairs (LPPs). Boundaries, blockages, instances, and vias are not returned. This argument is not supported if the specified objects are passed. Default: All LPPs

`-objectTypes`

Returns objects matching the specified filter criteria. This argument is not supported if the specified objects are passed. Default: All object types

`-excludeObjectTypes`

Excludes the specified object types from the operation. For example, you can use it with `-objectTypes` to return all objects except rectangles.

`-filter`

Standard attribute filter. Default: Match all

`-hierPath`

Hierarchical path to the objects. This argument is optional when passing a collection of `oaFigs` and is used exclusively with `-design`.

Errors

- Invalid line, rectangle or polygon points.
- Design is not opened by a design editor.
- `-type` does not match points given.
- `-design` cannot be specified with `oaFigs`.
- `-touch` cannot be specified with `oaFigs`.
- `-depth` cannot be specified with `oaFigs`.
- `-partial` cannot be specified with `oaFigs`.
- `oaFig` is not found in `-hierPath`.
- Cannot use `-lpps` with specified figures.
- Cannot use `-objectTypes` with specified figures.

Examples

Example 1

```
# Create object filter for blockage
set design [ed]
set blockage [de::getObjectFilters leBlockage]
# Get all leBlockage figures
set block_figs [de::getFigures -design $design -objectTypes $blockage]
```

Example 2

```
# Create object filter for instance labels in the schematic design
set design [ed]
set instLabels [de::getObjectFilters seSchInstanceLabel -from $design]
# Make instance labels invisible
db::setAttr instLabels.visible -value 0
```

de::getHighlighted

Description

Returns a collection of objects from a highlight set. By default, the objects from `highlight1` are returned.

Syntax

```
de::getHighlighted [-set <deHighlightSet>] [-filter <string>]
```

Returns

Collection of objects from the set ({<defigure>}).

Arguments

`-set deHighlightSet`

Highlight set from which to return objects.

Default: highlight1 in design of the object collection

`-filter <string>`

Standard attribute filter.

Default: Match all

Examples

```
# Create a rectangle in the layout design le::createRectangle {{0
-0.1} {0.2 0.1}} -design [ed] -lpp "ml drawing"# Highlight the
rectanglede::highlight [de::getFigures {{0.1 0}} -design [ed]]#
Get highlighted object and print its bBoxset h [db::getNext
[de::getHighlighted]]
puts [db::getAttr h.object.bBox]
```

de::getHighlightSets

Description

Returns highlight sets for the specified design, optionally restricted by their name.

Syntax

```
de::getHighlightSets [<name: string>] (-design <oaDesign> | -context
<deContext>) [-filter <string>]
```

Returns

{<deHighlightSet>}: Matching highlight set.

Errors

The design has not been opened by the design editor.

Arguments

`name`

Glob-style pattern to match highlight set name against.

Default: All sets

`-design`

Design in which the highlight set is defined.

`-context`

Context from which to return all highlight sets registered between the top and edit designs.

`-filter <string>`

Standard attribute filter.

Default: Match all

Examples

```
# Create new highlight set
de::createHighlightSet newHighlightSet -design [ed] -packet markerFat
# Get all highlight sets and print their names
set hs [de::getHighlightSets -design [ed]]
puts [db::createList [db::getattr hs.name]]
```

de::getInterfaceElements

Description

Retrieves all interface elements created for the given design context for partition visualization according to the given AMS netlister.

Syntax

```
de::getInterfaceElements -context <heContext> -netlister <nlAMSNetlister>
[-hierarchy <bool>] [-filter <string>]
```

Returns

`<dbCollection>` : Collection of interface element objects.

Arguments

`-context`

Hierarchy context.

`-netlister`

AMS netlister according to which the partition is made.

`-hierarchy`

If true, not only the interface elements of the current design, but all the interface elements of the entire hierarchy are collected.

Default is: false

`-filter`

Standard attribute filter.

Examples

```
# Create interface element on OUT net
set context [de::getActiveContext]
set design [db::getAttr context.editDesign]
set net [db::getNets OUT -of $design]
set hc [db::getAttr context.hierarchy]
set n [nl::getAMSNetlisters AMS]
de::createInterfaceElement -on $net -direction a2d -context $hc
-netlistner $n

# Save interface elements to ie.xml file
de::saveInterfaceElements [de::getInterfaceElements -context $hc
-netlistner $n] -filePath ./ie.xml

# Remove interface elements and load them from ie.xml file
db::destroy [de::getInterfaceElements -context $hc -netlistner $n]
de::loadInterfaceElements -filePath ./ie.xml
```

de::getLayers

Description

Returns an ordered collection of deLayer objects for a given design, along with the object validity, selectability, and visibility settings. One deLayer object is returned for each layer referenced by a layer-purpose pair (LPP) in the specified container.

Syntax

```
de::getLayers [<name: string>] -of <deContext | oaDesign> [-filter
<string>]
```

Returns

{<deLayer>}: Ordered collection of deLayers matching the pattern and the filter

Arguments

name

Optional glob-style pattern matched against the layer name. Default: *

`-of`

Design or deContext from which the LPPs are retrieved.

`-filter`

Standard attribute filter.

Errors

- Invalid filter.
- The specified design is a not a layout design.
- The specified design is not opened in a design editor.

Examples

```
# Get valid layers and lpps
db::foreach delayer [de::getLayers -of [ed]] {
    puts "Layer Name = [db::getattr delayer.layer]"
    db::foreach lpp [db::getattr delayer.lpps] {
        puts "    [db::getattr lpp.lpp]"
    }
}
```

de::getLookaheadMessages

Description

Returns the list of look-ahead strings currently in the look-ahead buffer.

Syntax

```
de::getLookaheadMessages
```

Returns

<string>: Tcl list of look ahead messages.

de::getLPPs

Description

Returns an ordered collection of deLPP objects for a given design, along with the object validity, selectability, and visibility settings.

Syntax

```
de::getLPPs [<name: lpp>] -of <deContext | oaDesign | oaTech> [-set  
    <deLPPSet | string>]  
[-filter <string>]
```

Returns

{<deTechLPP>}: Ordered collection of layer-purpose pair (LPP) matching the pattern and the filter

Arguments

name

Name of the layer-purpose pair.

-of

Retrieves LPPs from the design, deContext, or technology database.

-set

deLPPSet object or set title.

-filter

Standard attribute filter.

Errors

- Invalid filter.
- The specified design is a not a layout design.
- The specified design is not opened in a design editor.

Examples

```
# Print information about all LPPs defined in active design tech
set design [ed]
db::foreach item [de::getLPPs -of $design] {
    puts "LPP: [db::getAttr item.lpp] Valid: [db::getAttr item.valid] \
    Layer Number: [db::getAttr item.layer.number] Purpose Number:
    [db::getAttr item.purpose.number]"
}
```

de::getLPPSets

Description

Returns a collection of layer-purpose pair (LPP) sets, optionally filtered by title. Built-in LPP sets are not returned.

Syntax

```
de::getLPPSets [<pattern: string>] [-filter <string>]
```


Returns

<dbCollection> : Collection of matching deLPPSet objects.

Arguments

pattern

Glob-style pattern to match against deLPPSet titles.

Default is: " * ".

-filter

Standard attribute filter.

Examples

```
# Get LPP sets and print their names
set lppSets [de::getLPPSets]
puts [db::createList [db::getAttr lppSets.title]]
```

de::getMergedBBox

Description

Returns a merged BBox from a collection of figures.

Syntax

```
de::getMergedBBox <figures: {<deFigure>} | {<deSelectionSet>} | {<oaFig>}
| {<oaFigGroup>} | {<oaInst>} | {<oaShape>} | {<oaVia>}}>
```

Returns

<oaBox> : oa::Box

Arguments

figures

Collection of figures.

Examples

```
set design [ed]
# Get merged BBox of selected objects in the active design
set oaBox [de::getMergedBBox [de::getSelected -design $design]]
puts "Width: [oa::getWidth $oaBox]"
puts "Height: [oa::getHeight $oaBox]"
puts "BBox: [oa::left $oaBox] [oa::bottom $oaBox] [oa::right $oaBox]
[oa::top $oaBox]"

# Get merged BBox of all shapes in the active design
```

```
set oaBox [de::getMergedBBox [db::getShapes -of $design]]
puts "Width: [oa::getWidth $oaBox]"
puts "Height: [oa::getHeight $oaBox]"
puts "BBox: [oa::left $oaBox] [oa::bottom $oaBox] [oa::right $oaBox]
[oa::top $oaBox]"
```

de::getObjectFilters

Description

Object/Layer Panel (OLP) object filters from an open design, optionally filtered by group name or by `-filter`.

Syntax

```
de::getObjectFilters [<name: string>] [-of <deContext | oaDesign |
{<deLayer>}>] [-filter <string>]
```

Returns

{<deObjectFilter>} : Collection of object filters matching the pattern and the filter.

Arguments

name

Object filter name. Default: <>.

The supported object filter names are:

- leBlockage
- leBlockageActiveLayer
- leBlockageOther
- leBlockagePin
- leBlockagePlacement
- leBlockagePlacementHard
- leBlockagePlacementPartial
- leBlockagePlacementSoft
- leBlockageRouting
- leBoundary
- leBoundaryAlignment
- leBoundaryArea

- `leBoundaryCluster`
- `leBoundaryPR`
- `leBoundaryPRCoreRow`
- `leBoundaryPRCoreRowSite`
- `leBoundarySnap`
- `leCellTypeBlackBox`
- `leCellTypeBlock`
- `leCellTypeCore`
- `leCellTypeCover`
- `leCellTypeCoverBump`
- `leCellTypePad`
- `leCellTypePhysicalOnly`
- `leCellTypeSoftMacro`
- `leColor1`
- `leColor2`
- `leColor3`
- `leColor4`
- `leColor5`
- `leColor6`
- `leColorLock`
- `leColorMask1`
- `leColorMask2`
- `leColorMask3`
- `leColorMask4`
- `leColorMask5`
- `leColorMask6`
- `leColorMaskSame`

- `leColorSame`
- `leColorUnlock`
- `leCoordinateMark`
- `leCorridorArea`
- `leFigGroup`
- `leFigGroupMember`
- `leGlobalWideAlignment`
- `leGrid`
- `leGridAxes`
- `leGridMajorMinor`
- `leGridRoute`
- `leGuardRing`
- `leGuardRingShape`
- `leInstance`
- `leInstanceHierarchy`
- `leInstancePin`
- `leMarker`
- `leMarkerAcknowledgedWarning`
- `leMarkerAnnotation`
- `leMarkerCriticalError`
- `leMarkerError`
- `leMarkerFatalError`
- `leMarkerInfo`
- `leMarkerSignedOffCriticalError`
- `leMarkerSignedOffError`
- `leMarkerWarning`
- `leMpp`

- `lePartialObject`
- `lePartialObjectCenterLine`
- `lePartialObjectCenterLineVertex`
- `lePartialObjectEdge`
- `lePartialObjectVertex`
- `lePcellShape`
- `lePin`
- `leRouteCorridor`
- `leRouteGuide`
- `leRouteTypeBus`
- `leRouteTypeClockRing`
- `leRouteTypeClockStrap`
- `leRouteTypeCustomRoute`
- `leRouteTypeDetail`
- `leRouteTypeFill`
- `leRouteTypeGlobal`
- `leRouteTypeMacroPinConn`
- `leRouteTypeRing`
- `leRouteTypeShieldDynamic`
- `leRouteTypeShieldFix`
- `leRouteTypeStdPinConn`
- `leRouteTypeStrap`
- `leRouteTypeUser`
- `leRouteTypeUserTrunk`
- `leRouteTypeZeroSkew`
- `leRow`
- `leRowSite`

- `leRuler`
- `leShape`
- `leShapeEllipse`
- `leShapePath`
- `leShapePathSegment`
- `leShapePolygon`
- `leShapeRectangle`
- `leSignalTypeAnalog`
- `leSignalTypeClock`
- `leSignalTypeGround`
- `leSignalTypeNoNet`
- `leSignalTypePower`
- `leSignalTypeReset`
- `leSignalTypeScan`
- `leSignalTypeSignal`
- `leSignalTypeTieHigh`
- `leSignalTypeTieLow`
- `leSignalTypeTieOff`
- `leSoftMacroBoundary`
- `leSoftMacroPin`
- `leSteiner`
- `leStretchHandle`
- `leStripedWire`
- `leText`
- `leTrackRegion`
- `leTransmissionLine`
- `leTransmissionLineShape`

- `leVia`
- `leViaShape`
- `leVoltageArea`
- `leVoltageAreaGuardband`
- `neInstance`
- `neInstancePin`
- `neNet`
- `nePin`
- `seSchFlightLine`
- `seSchImage`
- `seSchInstance`
- `seSchInstanceLabel`
- `seSchInstanceName`
- `seSchInstancePin`
- `seSchInstancePinName`
- `seSchMarker`
- `seSchNote`
- `seSchNoteShape`
- `seSchOverlayObject`
- `seSchPin`
- `seSchPinName`
- `seSchReferenceDesignObject`
- `seSchShapeArc`
- `seSchShapeCircle`
- `seSchShapeEllipse`
- `seSchShapePolygon`
- `seSchShapeRectangle`

- seSchTable
- seSchTemplateGroup
- seSchText
- seSchWire
- seSchWireName
- seShape
- seSymImage
- seSymInstance
- seSymMarker
- seSymNote
- seSymNoteShape
- seSymPin
- seSymPinName
- seSymSelectionShape
- seSymShape
- seSymShapeArc
- seSymShapeCircle
- seSymShapeEllipse
- seSymShapeLine
- seSymShapePolygon
- seSymShapeRectangle
- seSymTable

`-of`

Design, context, or layer from where the ObjectFilters must be obtained.

`-filter`

Standard attribute filter.

Examples

Example 1


```
# Get all leShape figures from all hierarchical levels
set shapes [de::getFigures -design [ed] -objectTypes
  [de::getObjectFilters leShape] -depth 1]
# Print bBox attribute of the shapes
db::foreach shape $shapes {
  if {[db::getAttr shape.lineage.depth] != 0} {
    set inst [lindex [db::getAttr shape.lineage.levels] 0 0]
    set masterName [db::getAttr inst.master.cellName]
    puts $masterName
  }
}
```

Example 2

```
# Get pin figures of the I0 instance
set insts [db::getInsts I0 -of [ed]]
set shapes [db::getShapes -of [db::getAttr inst.master] -filter
  {%pin!=""}]
set hp [oa::HierPath]
oa::pushLevel $hp $inst
set figs [de::getFigures $shapes -hierPath $hp]
```

Example 3

```
# Get poly shapes from all hierarchy levels and copy them to top
set design [ed]
set transaction [de::startTransaction "Copy Shapes to top" -design
  $design]
db::foreach fig [de::getFigures -design $design -depth -1 -objectTypes
  [de::getObjectFilters leShape] -lpps [de::getLPPs {poly drawing}
  -from $design]] {
  if {0==[db::getAttr fig.lineage.depth]} {continue}
  set inst [lindex [db::getAttr fig.lineage.levels] end 0]
  puts "Copy from [db::getAttr inst.libName]/[db::getAttr
  inst.cellName]/[db::getAttr inst.viewName]"
  set transform [db::getAttr fig.lineage.transform]
  le::copy [db::getAttr fig.object] -dest $design \
    -anchor {0 0} \
    -dx [db::getAttr transform.xOffset] \
    -dy [db::getAttr transform.yOffset] \
    -rotate [db::getAttr transform.orient]
}
de::endTransaction $transaction
```

de::getOLPPresets

Description

Returns a collection of all Object/Layer Panel presets defined in a session.

Syntax

```
de::getOLPPresets [<pattern: string>] [-filter <string>]
```

Returns

<dbCollection> : Collection of matching deOLPPreset objects.

Arguments

pattern

Glob-style pattern to match preset name. Default: *

-filter

Standard attribute filter.

Examples

```
# Create two OLP presets
de::createOLPPreset newPreset1 -from [ed]
de::createOLPPreset newPreset2 -from [ed]
# Get OLP presets and print their names
set presets [de::getOLPPresets]
puts [db::createList [db::getattr presets.title]]
```

de::getOpenFiles

Description

Returns all currently open files, optionally restricted by modified state or being open in a given context.

Syntax

```
de::getOpenFiles [-context <deContext>] [-modified <bool>] [-filter  
  <string>]
```

Returns

All currently open files (in any context) ({<dmDMFile>})

Arguments

-context deContext

Only return files open in the specified context.

-modified

Only return files which are either modified or unmodified by providing this flag. If unspecified, all files will be returned.

`-filter`

Optional standard attribute filter.

Examples

```
# Print list of modified designs that are open in context and their
primary files
db::foreach dmFile [de::getOpenFiles -modified true] {
    puts "LCV: [db::getAttr dmFile.container.libName]/[db::getAttr
dmFile.container.cellName]/[db::getAttr dmFile.container.name] Path:
[db::getAttr dmFile.path]"
}
```

de::getOutdatedDesigns

Description

Returns a collection of designs that have changed on disk since they were initially opened.

Syntax

```
de::getOutdatedDesigns [-filter <string>]
```

Returns

{<oaDesign>}: Designs that are newer on disk than in memory.

Arguments

`-filter`

Optional standard attribute filter.

Examples

```
# Print outdated design Library, Cell, View names; and last saved time
db::foreach design [de::getOutdatedDesigns] {
    puts "LCV: [db::getAttr design.libName]/[db::getAttr
design.cellName]/[db::getAttr design.viewName]"
    puts "    Last Save Time: [clock format [db::getAttr
design.lastSavedTime] -format {%B %d %T}]"
}
```

de::getOutdatedDMData

Description

Returns a collection of DMData that has been changed on disk since it was initially opened. This is determined by comparing the modification time on disk with the file modification time recorded when the design was first opened. Only DMData open in read-

only mode is checked. DMData that is open for write/append cannot be changed on disk by external OA processes due to OA write locking.

Syntax

```
de::getOutdatedDMData [-filter <string>]
```

Returns

{<oaDMData>} : DMData that is newer on disk than in memory.

Arguments

-filter

Standard attribute filter.

Examples

```
# Print library and cell names for outdated DM data
if {[db::isEmpty [de::getOutdatedDMData]]} {
  puts "There is no outdated DM data"
} else {
  db::foreach data [de::getOutdatedDMData] {
    puts "[db::getattr data.libName] [db::getattr data.cellName]"
  }
}
```

de::getOverlays

Description

Returns all registered overlays.

Syntax

```
de::getOverlays [<name: string>] [-base <oaDesign>] [-filter <string>]
```

Returns

<dbCollection>: All registered overlays.

Arguments

name

Glob-style pattern to match against overlay name. Default is *.

-base

If specified, returned collection returns deOverlays registered for this design.

`-filter`

Optional standard attribute filter.

Examples

```
# Create two overlay designs over the edit design
set o [de::createOverlay newOverlay1 -base [ed]]
set o [de::createOverlay newOverlay2 -base [ed]]
# Get overlays and print their names
set overlays [de::getOverlays -base [ed]]
puts [db::createList [db::getAttr overlays.name]]
```

de::getPartitioner

Description

Retrieves partitioner created for the given design context according to the given AMS netlister.

Syntax

```
de::getPartitioner -context <deContext> [-netlister <nlAMSNetlister>]
```

Returns

<nlPartitioner>: Partitioner object.

Arguments

`-context`

Design context.

`-netlister`

AMS netlister according to which the partition is made.

Examples

```
# Get partitioner from active context and check if it is simulation-based
set p [de::getPartitioner -context [de::getActiveContext]]
puts [db::getAttr p.simulationBased]
```

de::getProbeBitNames

Description

Finds all equivalent bit names for the probe.

Syntax

```
de::getProbeBitNames -probe <deProbe> -context <deContext>  
[-includeSymbolPins <bool>] [-maxDepth <int>]
```

Returns

<string> : A list of bit names involved in the probe, including equivalent bit nets.

Arguments

-probe

Probe for which the bit names are retrieved.

-context

Design context for which the bit names are retrieved.

-includeSymbolPins

Get symbol pin bit nets.

Default is: true

-maxDepth

Get the maximum level to search below the given context.

Default is -1.

de::getProbeFigures

Description

Finds figures (both 0x and 1x) highlighted for the probe in the specified design context.

The returned objects can be of the following types:

- net probe: net wire, 0x or 1x pin shape
- path probe: net wire, 0x or 1x pin shape
- instance probe: instance
- terminal probe: 0x or 1x pin shape
- cellview probe: nothing

This command should be called for canvas-type editors (SE, LE), otherwise an empty collection is returned.

Syntax

```
de::getProbeFigures -probe <deProbe> -context <deContext>
```

Returns

{<deFigure>} : Collection of figures involved in this probe highlighting.

Arguments

-probe

Probe for which the figures are retrieved.

-context

Design context for which the figures are retrieved.

Examples

```
# Create probes for all nets in the design
set context [de::getActiveContext]
set design [db::getattr context.editDesign]
db::foreach net [db::getNets -of $design] {
  de::createProbe [db::getattr net.name] -context $context -type net
}
# Get all probes from the design
set probes [de::getProbes -context $context]
# Get figures highlighted for probes and print their bBoxes
db::foreach probe $probes {
  set figures [de::getProbeFigures -context $context -probe $probe]
  puts [db::createList [db::getattr figures.object.bBox]]
}
```

de::getProbeNets

Description

Finds bit net(s) highlighted for the net probe in the specified design context.

Syntax

```
de::getProbeNets -probe <deProbe> -context <deContext>
```

Returns

{<oaNet>} : Collection of bit nets involved in the probe highlighting.

Arguments

-probe

Probe for which the nets are retrieved.

`-context`

Design context for which the nets are retrieved.

Errors

- The specified probe is not of net type.

Examples

```
# Create probes for all nets in the design
set context [de::getActiveContext]
set design [db::getAttr context.editDesign]
db::foreach net [db::getNets -of $design] {
  de::createProbe [db::getAttr net.name] -context $context -type net
}
# Get all probes from the design
set probes [de::getProbes -context $context]
# Get probes nets and print their names
db::foreach probe $probes {
  set nets [de::getProbeNets -context $context -probe $probe]
  puts [db::createList [db::getAttr nets.name]]
}
```

de::getProbes

Description

Finds probes which correspond to a specified object. If an object is not supplied, returns all probes defined relative to the specified heContext, including lineage. For example, if an heContext is passed in with no lineage, all probes will be considered. If an heContext with a lineage is passed in, only those probes which match that lineage in part or in whole will be considered. This includes probes which traverse through a lineage but whose probed object is further down the hierarchy.

If an object is given with the -object command line option, the object must exist in the current design specified by the heContext. All probes associated with that object will then be considered. If the object is an oaInst, then all probes for that object in addition to all probes that go through this instance, and exist below, will be considered.

In both cases, all probes considered are passed through the standard filtering if the -filter option is given. The final set of probes is then returned.

Syntax

```
de::getProbes -context <deContext | heContext> [-object <deFigure |  
  oaFig>] [-allHierarchy <bool>] [-filter <string>]
```

Returns

Collection of deProbe objects defined relative to the specified design ({<deProbe>}).

Errors

Object not found within context design.

Arguments

-context

Context for retrieving probes.

-object defigure | oaFig

If specified, returns probes whose highlight set includes the given object.

Default: Match all.

-allHierarchy <bool>

Returns the probes found in the entire hierarchy.

Default: false

-filter <string>

Standard attribute filter.

Default: Match all.

Examples

```
set context [de::getActiveContext]
db::foreach net [db::getNets -of [db::getAttr context.editDesign]] {
    de::createProbe [db::getAttr net.name] -context $context -type
    net -description "Net [db::getAttr net.name]" -purpose info -packet
    highlight[expr round (rand()*10)] -tool Custom
}
db::foreach probe [de::getProbes -context $context] {
    puts [db::getAttr probe.objectName]
}
```

de::getRecentFiles

Description

Returns a collection of recently opened files. The collection is ordered such that the first element is the most recently opened file and subsequent elements are increasingly less recently opened.

Syntax

```
de::getRecentFiles [-filter <string>]
```

Returns

Recently opened files in decreasingly recent order ([{{<dmDMFile>}}](#)).

Arguments

-filter <string>

Optional standard attribute filter.

Default: Match all.

Examples

```
# Open recently opened files in design editors
db::foreach file [de::getRecentFiles] {
    de::open [db::getAttr file.container]
}
```

de::getSelected

Description

Returns a collection of objects from the specified design. If `-partial` is false, this command will omit partial regions, returning a collection of full objects represented by `deFigures`. Note that an object with any number of partially selected edges will be returned once.

Syntax

```
de::getSelected -design <oaDesign> [-partial <bool>] [-filter <string>]
```

Returns

Collection of objects from the set (`{<defigure>}`)

Errors

The design has not been opened by the design editor.

Arguments

`-design oaDesign`

Design from which to return selected objects.

`-partial <bool>`

Includes partial regions in result.

Default: true

`-filter <string>`

Standard attribute filter.

Default: Match all

Examples

```
set design [ed]
de::selectAll [de::getActiveContext]
db::foreach fig [de::getSelected -design $design] {
    puts [db::getattr fig.objType]
}
```

de::getSelectedLogic

Description

Returns a collection of logic selected in the specified design context.

Syntax

```
de::getSelectedLogic -context <deContext> [-hier <bool>] [-filter  
  <string>]
```

Returns

<dbCollection> : Selected logic

Arguments

-context

Design context from which to retrieve selected logic.

-hier

If false, return only logic in the edit design. Default: false

-filter

Standard attribute filter.

Examples

```
# Open Design Navigator to see logically selected objects.  
# Logically select I0 instance and IN net  
set context [de::getActiveContext]  
set design [db::getAttr context.editDesign]  
de::selectLogic [db::getNext [db::getInsts I0 -of $design]] -context  
  $context  
de::selectLogic [db::getNext [db::getNets IN -of $design]] -context  
  $context  
# Print types of logically selected objects  
set selLogic [de::getSelectedLogic -context $context]  
puts [db::createList [db::getAttr selLogic.object.type]]
```

de::getSelectedVertex

Description

Returns the coordinates of the path or pathSeg vertex.

Syntax

```
de::getSelectedVertex <deFigure: {<deFigure>}>
```

Arguments

deFigure

The return point of the selected vertex.

Examples

```
# Select a rectangle or path in design and get its vertex coordinates
de::getSelectedVertex [de::getSelected -design [ed]]
```

de::getTemplates

Description

Returns a collection of deTemplate objects for each template found in the given library or in all libraries.

Syntax

```
de::getTemplates [<glob: string>] [-libName <dmLibName>] [-filter  
  <string>]
```

Returns

<dbCollection> : Collection of deTemplate objects.

Arguments

glob

Glob-style name filter. The default: *

-libName

Name of library to search.

-filter

Standard filter expression.

Examples

```
# Create template from selection in design schematic
de::createTemplate invTemplate -libName DesignLibrary -netlist
  [de::getSelected -design [ed]]
  -description "Inverter template"

# Get templates from DesignLibrary lib and print their names
set templates [de::getTemplates -libName DesignLibrary]
puts [db::createList [db::getAttr templates.name]]
```

de::getTools

Description

Returns all registered tools, optionally restricted to those whose name matches a given glob-style pattern. A filter may be provided. If a context is specified, only those tools active in the specified context will be returned.

Syntax

```
de::getTools [<name: string>] [-filter <string>]
```

Returns

<dbCollection>: All registered tools.

Arguments

name

Pattern to match tool names against.

Default: Match all.

-filter <string>

Optional standard attribute filter.

Default: Match all.

Examples

```
# Get all tools and print their names and titles
db::foreach tool [de::getTools] {
    puts "Name: [db::getAttr tool.name] Title: [db::getAttr tool.title]"
}
```

de::getTransactions

Description

Returns transaction objects in a design, including transactions on the undo stack and open transactions that have not yet ended. This command can also filter the returned collection by transaction name.

Note that -design and -config are mutually exclusive arguments.

Syntax

```
de::getTransactions [<pattern: string>] [-design <oaDesign> | -config  
  <heConfig>] [-regexp <bool>]
```

Returns

Transaction objects found (`{<deTransaction>}`).

Arguments

`<string>`

Pattern to filter the transaction names.

Default: Matches all strings.

`-design oaDesign`

Design in which to search for transactions.

`-config heConfig`

Config in which to search for transactions.

`-regexp <bool>`

True, if the pattern is a regular expression. False, if it uses glob-style wildcards.

Default: false

Errors

- Design is invalid.
- Undo is disabled for this container.

Examples

```
set design [ed]
set x 1
set y 1
set tr [de::startTransaction "Create Rectangles" -design $design]
db::foreach lpp [de::getLPPs -from $design] {
    le::createRectangle [list [list $x $y] [list [expr $x+0.5] [expr
    $y+0.5]]] -design $design -lpp [db::getattr lpp.lpp]
    incr x
    if {$x > 10} {
        set x 1
        incr y
    }
}
# Print all transactions from active design
db::foreach transaction [de::getTransactions -design [ed]] {
    puts "Name: [db::getattr transaction.name] Is Open: [db::getattr
    transaction.open]"
}
de::endTransaction $tr
```

de::getViewport

Description

Returns the coordinates of the current viewport.

An interactive version of this command is not available.

Syntax

```
de::getViewport [-window <deCanvas | giWindow | int>] [-ofEdit <bool>]
```

Returns

The current viewport (<box>)

Arguments

-window

The window or canvas to query.

Default: active window

-ofEdit <bool>

When true, the command returns the viewport of the active edit design.

Otherwise, the command returns the viewport of the active view design.

Default: true

Errors

- The window ID is invalid.
- There was no open design in the specified window.

Examples

```
de::getViewport -window 2 -ofEdit false
```

de::gotoViewport

Description

Navigates the viewport history created by the pan, set viewport, zoom, and fit commands. The viewport history remains unchanged.

An interactive version of this command is not available.

Syntax

```
de::gotoViewport [-window <deCanvas | giWindow | int>] [-dir <int>]
```


Returns

The new viewport (<box>).

Arguments

`-window`

The window or canvas to modify.

Default: active window.

`-dir`

Negative numbers go backward in history and positive numbers go forward. If you enter a value that is outside the history range, the oldest or newest viewport is returned, as appropriate.

Default: -1

Errors

- The window ID was invalid.
- There was no open design in the specified window.

Examples

```
de::gotoViewport -window 2 -dir 1
```

de::help

Description

Displays a Help topic for the specified item: a windowType, a command, or a specific {helpID, collectionID} pair. If no argument is provided, the Custom Compiler Home page is displayed. If no help is available for the topic requested, the Help system displays the Home page.

Notes:

- This command displays an appropriate help viewer, if necessary.
- You can specify collectionID only with helpID.
- The Custom Compiler environment does not have separate man pages.

Syntax

```
de::help [-window <giWindow | int>] [-assistant <giAssistant | string>]  
         [-dialog <giDialog | string>] [-command <string>] [-helpID <string>]  
         [-collectionID <string>]
```

Arguments

`-helpID`

Identifier of the help topic to display.

Default: NONE

`-collectionID`

Identifier of the help collection from which to display the specified help topic.

Default: cidCustomDesigner.

`-command`

Command for which to provide help.

Default: NONE

`-window`

Window for which to provide help.

Default: NONE

`-assistant`

Assistant for which to provide help.

Default: NONE

`-dialog`

Dialog box for which to provide help.

Default: NONE

Examples

```
de::help -window 1
# Call se::createInst help
de::help -command se::createInst
de::help -assistant [gi::getAssistants leObjectLayerPanel]
```

de::highlight

Description

Adds a collection of objects to the highlight set. The objects must all be within, or descend from, the design to which the highlight set belongs. Objects already in the set are ignored.

Syntax

```
de::highlight <objects: {<deFigure>} | {<oaFig>} | {<oaFigGroup>} |  
{<oaInst>} | {<oaShape>} | {<oaVia>}} [-set <deHighlightSet>]
```

Arguments

objects

Objects to highlight.

-set `deHighlightSet`

Highlight set into which to add objects.

Default: highlight1

Examples

The following example is used to highlight a rectangle. If the highlight set is not specified, the new highlight set with name *highlight1* will be created.

```
# Create a rectangle in the layout design  
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"  
# Create another rectangle  
le::createRectangle {{0.3 -0.1} {0.5 0.1}} -design [ed] -lpp "m1 drawing"  
# Highlight first rectangle  
# New "highlight1" set is created and the highlight added to that set  
de::highlight [de::getFigures {{0.1 0}} -design [ed]]  
# Highlight second rectangle  
# Highlight will be added to already created "highlight1" set  
de::highlight [de::getFigures {{0.4 0}} -design [ed]]
```

de::importBookmarks

Description

Import bookmarks from a specified file.

An interactive version of this command is not available.

Syntax

```
de::importBookmarks -filePath <string> [-force <bool>]
```

Returns

The collection of imported bookmarks ({<deBookmark>}).

Arguments

-filePath <string>

The path to an external bookmark file.

`-force <bool>`

If false, the command will throw an error if the bookmark with the same name already exists. If true, the bookmark will be silently overwritten.

Default: true

Examples

```
# Import bookmarks from previously exported xml file
de::importBookmarks -filePath ./exportedBookmarks.xml
# Open Bookmark Manager
de::showBookmarkManager
```

de::importProbeCellSettings

Description

Imports **Probe Trace Through Cell** settings from the specified file.

Syntax

```
de::importProbeCellSettings -file <string>
```

Arguments

`-file`

File name of the **Probe Trace Through Cell** settings to import.

de::isEditable

Description

Retrieves the open mode of the edit design in the specified context or an arbitrary design opened within any context.

Syntax

```
de::isEditable <object: deContext | dmCellView | dmCellViewName |
dmDMFile | oaDesign>
```

Returns

Edit mode for specified design (or edit design of specified context) (<bool>).

Arguments

`object`

Design from which to retrieve the edit mode. If a context is specified, the edit design is implied. If a dmCellView is specified, the primary file is implied.

Examples

```
set design [ed]  
de::isEditable $design
```

de::isHidden

Description

Returns true if command is hidden.

Syntax

```
de::isHidden <command: string>
```

Returns

Hidden status for a specified command (<bool>).

Arguments

command

Command whose hidden status need to get.

de::isInUndo

Description

Checks if the design is in undo or not.

Syntax

```
de::isInUndo -design <oaDesign>
```

Returns

<bool> : Is in Undo.

Arguments

-design

oaDesign option.

Errors

- Design is not valid.

de::isModified

Description

Returns true if the specified file is open and modified within any context. If a context is provided, the edit design will be queried.

Syntax

```
de::isModified <object: deContext | dmCellView | dmCellViewName |  
dmDMFile | oaDesign>
```

Returns

Whether the file is modified (<bool>).

Arguments

`object`

File or context from which to query the modification state. If a context is specified, the edit design is implied. If a `dmCellView` is specified, the primary file is implied.

Errors

The specified file is not opened within a design editor (DE) context.

Examples

```
# Save design if it was modified  
set design [ed]  
if {[de::isModified $design]} {  
    de::save $design  
}
```

de::linkContext

Description

Links a context to another context, specifying a name and title for the link.

Syntax

```
de::linkContext <context: deContext> -to <deContext> -name <string>  
-title <string>[mapToProc[<variant>]] [-mapFromProc[<variant>]]
```

Returns

<deLinkedContext> : New object.

Arguments

`context`

Context to link to another context.

`-to`

Context to link with.

`-name`

Link name, for example, `IxSchematic`.

`-title`

Title to use in the user interface, for example `SDL Schematic`.

`-mapToProc`

Maps Tcl procedure from layout object to linked context compatible object.

`-mapFromProc`

Maps Tcl procedure from linked context compatible object to layout object.

Examples

```
# Create two contexts to link one another
set context1 [de::createContext parasitics/r_ground/schematic]
set context2 [de::createContext parasitics/r_ground/symbol]
# Open main context
de::openEditor -context $context1
# Link context2 to context1
set lc [de::linkContext $context2 -to $context1 -name linkedContext
      -title "Linked Context"]
# Open linked context in the split editor
de::addCapability "deSplitView" -to $lc
db::setAttr context1.editor.splitContext -value $lc
# Change linked context orientation
db::setAttr context1.editor.splitOrientation -value horizontal
# Fit both contexts
de::fit -window [db::getAttr context1.window]
de::fit -window [gi::findChild linkedContext -in [db::getAttr
  centralWidget -of [db::getAttr context1.window]]]
```

de::loadFavoriteDevices

Description

Loads the favorite devices from a file on the disk.

Syntax

```
de::loadFavoriteDevices -filePath <string>
```

Arguments

`-filePath`

File path to load.

Examples

```
# Create favorite devices from different libraries
set cv1 [dm::findCellView symbol -cellName cap -libName analogLib]
set cv2 [dm::findCellView symbol -cellName res -libName analogLib]
set cv3 [dm::findCellView symbol -cellName Cc -libName parasitics]
de::createFavoriteDevice -master $cv1 -viewType schematic
de::createFavoriteDevice -master $cv2 -viewType schematic
de::createFavoriteDevice -master $cv3 -viewType schematic
# Save devices from analogLib to favDevices_analogLib.xml file
de::saveFavoriteDevices [de::getFavoriteDevices -filter
    {%master.libName=="analogLib"}} -filePath ./favDevices_analogLib.xml
# Save all favorite devices to favDevices_all.xml file
de::saveFavoriteDevices -filePath ./favDevices_all.xml
# Load favorite devices from previously saved file
de::loadFavoriteDevices -filePath ./favDevices_analogLib.xml
```

de::loadInterfaceElements

Description

Loads interface elements saved in the specified file.

Syntax

```
de::loadInterfaceElements -filePath <string>
```

Returns

<dbCollection> : Collection of interface element objects successfully loaded from file.

Arguments

`-filePath`

Filename to load.

Errors

- File does not exist.
- File is not a IE XML file.
- IE file format error.

Examples

```
# Create interface element on OUT net
set context [de::getActiveContext]
set design [db::getAttr context.editDesign]
set net [db::getNets OUT -of $design]
set hc [db::getAttr context.hierarchy]
set n [nl::getAMSNetlisters AMS]
de::createInterfaceElement -on $net -direction a2d -context $hc
-netlist $n

# Save interface elements to ie.xml file
de::saveInterfaceElements [de::getInterfaceElements -context $hc
-netlist $n] -filePath ./ie.xml

# Remove interface elements and load them from ie.xml file
db::destroy [de::getInterfaceElements -context $hc -netlist $n]
de::loadInterfaceElements -filePath ./ie.xml
```

de::loadOLPPresets

Description

Loads the Object/Layer Panel (OLP) presets from the given xml file.

Syntax

```
de::loadOLPPresets -filePath <string>
```

Returns

Collection of OLP presets.

Arguments

-filePath

The path to the file to load.

Examples

```
# Create two OLP presets
de::createOLPPreset metals -from [ed]
de::createOLPPreset vias -from [ed]
# Save presets to presets.xml
de::saveOLPPresets [de::getOLPPresets] -filePath ./presets.xml
# Delete all presets
db::destroy [de::getOLPPresets]
# Load presets.xml file
de::loadOLPPresets -filePath ./presets.xml
# Get OLP presets and print their names
set presets [de::getOLPPresets]
puts [db::createList [db::getAttr presets.title]]
```

de::loadProbes

Description

Loads probes defined in the specified file. The context stored in the probe file must be opened in a DE editor window, or else the command will fail. Any probe defined in the file which is not valid, that is, where either the probed object does not exist, or any part of the lineage does not exist, will be ignored and the following warning message will be sent to the console.

"Warning: Ignoring <probe type> probe because <probed object name> does not exist."

Syntax

```
de::loadProbes -file <string>
```

Returns

Collection of deProbe objects created from the file ({<deProbe>}).

Arguments

-file

Filename to load.

Errors

- File does not exist.
- File is not a probe xml file.
- Top context defined in probe file is not currently DE opened.
- Probe file format error.

Examples

```
set context [de::getActiveContext]
db::foreach net [db::getNets -of [db::getattr context.editDesign]] {
    de::createProbe [db::getattr net.name] -context $context -type
    net -description "Net [db::getattr net.name]" -purpose info -packet
    highlight[expr round (rand()*10)] -tool Custom
}
# Save all probes in external file
de::saveProbes [de::getProbes -context $context] -filePath probes.xml
# Remove all probes
db::destroy [de::getProbes -context $context]
# Recreate probes in active context from external file
de::loadProbes -filePath probes.xml
```

de::navigateTo

Description

Navigates to the specified occurrence path via a sequence of return and descend operations.

Syntax

```
de::navigateTo <lineage: heContext | oaHierPath | string> -context  
  <deContext> [-readOnly <bool>]
```

Returns

<deContext>: Modified context object.

Arguments

lineage

Lineage specification.

-context

Context in which to descend/return.

readOnly

Specifies the open mode for the final design in the lineage. Default: true

Examples

In the following example, the instance name is provided as X7. The descend operation is performed in the write mode.

```
de::navigateTo X7 -context [de::getActiveContext]
```

In the following example, `de::navigateTo` performs the descend operation, and is read into the X7 instance.

```
de::navigateTo X7 -context [de::getActiveContext] -readonly\  
  true
```

Both examples are used to descend into the hierarchy.

de::next

Description

Executes the next N commands of the currently loaded interactive replay file. If the end of the file is reached, the file is unloaded and the breakpoints cleared.

Syntax

```
de::next [<count: int>]
```

Arguments

<int>

Number of commands to execute.

Default: 1

Examples

In the following example, `de::next` executes the next five lines of `replay.log`.

```
de::replay replay.log -interactive true
de::next 5
```

de::open

Description

Spawns the registered application for the specified file or primary file of a cellview. The top, view, and edit members are initialized. If an association is registered for the DM file, it is invoked. Otherwise, the command returns an error.

If either the `-instPath` option or a bookmark is used, multiple designs will be opened. All designs except for the target design are opened in read-only mode. The target design is opened according to the `-readOnly` argument. For information about bookmarks, see Bookmark Manager Tool.

If an editor is specified, this command overrides the editor associated with the view type by default. If the specified or bound editor is inappropriate for the DM file, this command will return an error.

The `de::open` command opens cellview specified in a first positioned argument as a top design. If `-instPath` argument is defined, the command opens switched master of the instance specified with the hierarchical lineage, in a top design context. All designs above target are opened in read-only mode.

In a schematic design, the search order of the switched master is controlled by the `seViewSearchList` preference value.

Syntax

```
de::open <object: dmCellViewName | {<deBookmark>} | {<dmCellView>} |  
{<dmDMFile>}> -application <deApplication | string> [-readOnly <bool>]  
[-instPath <string>] [-depth <int>] [-headless <bool>]
```

Returns

Newly created context object (deContext).

Arguments

`object`

One or more cellviews (primary file), DM files or bookmark objects to open.

`-readOnly`

Opens the design in read-only mode.

By default, the file is opened in write mode. If `-readOnly` is set to true, the file is opened in read-only mode. If the file could not be opened in write mode, the `de::open` command falls back to read-only mode.

If the file is already open in read-only mode and `-readOnly` is false, this command attempts to change the open mode to read/write. If the file is already open in read/write mode and `-readOnly` is true, then the read/write mode is preserved.

`-instPath`

Optional instance path to edit.

Default: /

`-depth`

Instructs the editor to open the maximum number of levels from this design. If the design is already open, the editor will use this argument to update the existing setting.

`-application`

Application to open.

`-headless`

No window is created for the new context.

If the `-headless` argument is passed, the context and editor objects are created but no window is created.

Default: false

Examples

Example 1

In the following example, `de::open` opens `designLibrary/NAND/layout` cellview in edit mode without opening a design editor window.

```
de::open [dm::getCellViews layout -libName designLibrary \
        -cellName NAND] -headless true
```

Example 2

`Innv` is instantiated in the `vco_buff` design (instance name is `I2`), `vco_buff` design is instantiated in the `vco` design (instance name is `X1`).

In the following example, `de::open` opens `Innv/schematic` (symbol, if schematic does not exist) design in edit mode. `vco` and `vco_buff` designs are in context stack and open in read-only mode.

```
de::open [dm::findCellView schematic -cellName vco -libName PLL]
        -instPath /X1/I2
```

de::open3DICDesign

Opens a 3DIC Compiler design with filtering.

Syntax

```
de::open3DICDesign <dmCellView: dmCellView> [-readOnly <bool>]
[-dataRegions [<oaPointArray>]] [-excludeDesignTypes [<enDesignTypes>]]
[-excludeUnplaced [<enDesignTypes>]] [-useExistingFilters <bool>]
```

Returns

<deContext> :

Arguments

`dmCellView`

Cellview of design to open.

`-readOnly`

Initial state of the Read Only flag. Default: false

`-dataRegions`

Tcl list of coordinates that define the vertexes of the region of interest.

`-excludeDesignTypes`

Tcl list of design types to exclude from translation.

`-excludeUnplaced`

Tcl list of design types of unplaced instances to exclude from translation.

`-useExistingFilters`

Use existing 3DIC design filters.

de::openCanvasPropertyEditor

Description

Opens On-canvas Property Editor.

Syntax

```
de::openCanvasPropertyEditor -figure <deFigure> -context <deContext>
```

Arguments

`-figure`

The figure.

`-context`

Design context which contains an figure.

de::openClipboardLib

Description

Opens a clipboard library. To retrieve all open clipboard libraries, use [de::getClipboardLibs](#).

Opened clipboard libraries are automatically reopened in future sessions.

The prefix *deClipboard_* is prepended to the clipboard library name when opening in the OpenAccess database to avoid conflicting with design libraries.

Syntax

```
de::openClipboardLib <string> -path <string>
```

Returns

<oaLib>: Clipboard library.

Arguments

<string>

Library name.

`-path`

Library path.

Errors

- Library is an open library.
- Insufficient permission.

Examples

```
# Open new clipboard library named clipLib
de::openClipboardLib clipLib -path ./clipLib

# Get clipboard libraries and print their names
set libs [de::getClipboardLibs]
puts [db::createList [db::getAttr libs.name]]
```

de::openEditor

Description

Opens an editor for the specified context.

Syntax

```
de::openEditor [<editor: deEditor>] -context <deContext> [-attrs
<dbObject>] [-applyWorkspace <bool>]
```

Returns

<deEditorInst>: Newly created editor instance.

Arguments

editor

Editor to display.

-context

Context to open in an editor.

-attrs

Initial attributes for the editor.

-applyWorkspace

Applies last-used workspace. Default: true

Examples

In this example, the `de::openEditor` command creates the `<deEditorInst>` object and opens the corresponding editor with the provided context.

```
set cellView [dm::findCellView schematic -cellName amp\  
             -libName DemoPLL]  
set context [de::createContext $cellView]  
de::openEditor -context $context
```

de::openFCDesign

Description

Opens the specified Fusion Compiler design with filtering.

Syntax

```
de::openFCDesign <dmCellView: dmCellView> [-readOnly <bool>]  
        [-dataRegions [<oaPointArray>]] [-excludeDesignTypes [<enDesignTypes>]]  
        [-excludeUnplaced [<enDesignTypes>]] [-useExistingFilters <bool>]
```

Returns

`<deContext>` :

Arguments

`dmCellView`

Design cellview to open.

`-readOnly`

Initial state of the Read Only flag. Default: false

`-dataRegions`

Tcl list of coordinates that define the vertexes of the region of interest.

`-excludeDesignTypes`

Tcl list of design types to exclude from translation.

`-excludeUnplaced`

Tcl list of design types of unplaced instances to exclude from translation.

`-useExistingFilters`

Uses existing Fusion Compiler design filters.

de::openFile

Description

Opens a design file in an existing design context.

Syntax

```
de::openFile <object: dmCellView | dmCellViewName | dmDMFile | oaDesign>  
-context <deContext>
```

Returns

<none>

Arguments

object

Design file to open.

-context

Context in which to open the file.

Examples

```
# Check the count of open designs for DesignLib/amp/schematic  
puts [db::getCount [db::getOpenDesigns DesignLib/amp/schematic]]  
  
# Open the cellview in the current context  
de::openFile DesignLib/amp/schematic -context [de::getActiveContext]  
  
# Check that the count increased by one  
puts [db::getCount [db::getOpenDesigns DesignLib/amp/schematic]]
```

de::openICC2Design

Description

Opens an IC Compiler II design with filtering.

Syntax

```
de::openICC2Design <dmCellView: dmCellView> [-readOnly <bool>]  
[-dataRegions [<oaPointArray>]] [-excludeDesignTypes [<enDesignTypes>]]  
[-excludeUnplaced [<enDesignTypes>]] [-useExistingFilters <bool>]
```

Returns

<deContext> :

Arguments

`dmCellView`

Cellview of design to open.

`-readOnly`

Initial state of the Read Only flag. Default: false

`-dataRegions`

Tcl list of coordinates that define the vertexes of the region of interest.

`-excludeDesignTypes`

Tcl list of design types to exclude from translation.

`-excludeUnplaced`

Tcl list of design types of unplaced instances to exclude from translation.

`-useExistingFilters`

Uses existing IC Compiler II design filters.

de::pan

Description

Moves the viewport in the direction specified by the `-direction` argument and by a specified distance (specified either with the `-units` argument in user units or with the `-multiplier` argument as a percentage of the current viewport), or centers the given point using the `-to` argument.

Syntax

```
de::pan (-to <point> | -direction <enum>) [-units <float> | -multiplier  
<float>] [-window <deCanvas | giWindow | int>]
```

Returns

The new viewport (<box>)

Arguments

`-window`

The ID of the window or canvas to pan.

Default: active window

`-to <point>`

Centers the new coordinate but preserves the zoom level.

`-direction`

Direction to pan: {E | N | NE | NW | S | SE | SW | W}.

`-units <float>`

Shifts the viewport the specified number of user units.

`-multiplier <float>`

The distance relative to the viewport width or height by which to pan the viewport. For example, if the direction is NE, the center moves by the vector (viewport_width*multiplier, viewport_height*multiplier).

If a non-zero multiplier or units argument is provided, the minimum pan will be one database unit.

The default value of 0.5 for `-multiplier` is only used if neither `-units` nor `-multiplier` is supplied.

Default: 0.5

Errors

- Not exactly one of `-to` and `-direction` was specified.
- All three of the arguments `-direction`, `-units`, and `-multiplier` were specified.
- `-to` was specified together with `-units` or `-multiplier`.
- The value for `-multiplier` was not positive.
- The window ID was invalid.
- There was no open design in the specified window.

Examples

```
de::pan -direction NW -multiplier 1 -window 2
```

de::paste

Description

Places copies of objects from the clipboard buffer into the target design. Optionally, you can create multiple copies of the clipboard object set by using array settings.

Syntax

```
de::paste <oaDesign: oaDesign> -point <point> [-orient <enum>]  
[-connectivity <bool>] [-cols <int>] [-dxCol <float>] [-dyCol <float>]  
[-rows <int>] [-dxRow <float>] [-dyRow <float>] [-clip <deClipboard>]
```

Arguments

`oaDesign`

Design where objects will be pasted.

`-point`

Point in design at which the clipboard contents will be pasted.

`-orient`

Rotation and mirroring for objects within the clipboard: {MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90}.

`-connectivity`

Option to preserve connectivity on paste.

Default: true

`-cols`

Number of copies along the X axis to create.

Default: 1.

`-dxCol`

Distance from column to column on X direction for arrayed copy in user units.

Default: width of clipboard contents bounding box rounded to the nearest manufacturing grid width multiple

`-dyCol`

Distance from column to column on Y direction for non-aligned arrayed copy in user units.

Default: 0

`-rows <int>`

Number of copies along the Y axis to create.

Default: 1

`-dxRow <float>`

Distance from row to row on X direction for non-aligned arrayed copy in user units.

`-dyRow <float>`

Distance from row to row on Y direction for arrayed copy in user units.

Default: height of clipboard contents bounding box rounded to the nearest manufacturing grid height multiple

`-clip`

Clipboard to paste. The default is the most recently copied clipboard.

Errors

- Clipboard is empty.
- Clipboard viewType is not compatible with the edit view type.
- Design is read-only.
- Connectivity is enabled but terminal nets do not match.

Examples

```
set design [ed]
# Copy selected objects to clipboard
de::copy [de::getSelected -design $design]
# Paste objects from clipboard into active design
de::paste $design -point {1 1} -cols 2 -rows 3 -dxRow 0.1 -dyRow 0.2
-orient R90
```

de::redoSelection

Description

Redoes the last selection operation.

Syntax

```
de::redoSelection -design <oaDesign>
```

Arguments

`-design`

Design in which to redo the last selection.

Examples

```
# Create a rectangle in the layout design le::createRectangle
{{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"# Create
another rectangle le::createRectangle {{0.3 -0.1} {0.5 0.1}}
-design [ed] -lpp "m1 drawing"# Select first rectangle de::select
[de::getFigures {{0.1 0}} -design [ed]]# Change selection and change
```

```
second rectangle de::deselectAll [de::getActiveContext] de::select
[de::getFigures {{0.4 0}} -design [ed]] # Undo selection
# First rectangle will be selected de::undoSelection -design [ed] # Redo
selection
# Second rectangle will be selected
de::redoSelection -design [ed]
```

de::redoTransaction

Description

Redoes one or more recent transactions. If a transaction is specified, then all transactions up to and including that transaction are redone. If instead design/config and steps are specified, then the next N transactions (from the current transaction) in the specified design/config are redone.

Syntax

```
de::redoTransaction (-transaction <deTransaction> | -design <oaDesign> |
    -config <heConfig>) [-steps <int>]
```

Arguments

-transaction

Transaction to move forward the transaction history. All transactions up to and including this one are redone. Exclusive of design and config.

-design *oaDesign*

Design object. Exclusive of transaction and config.

-config

Config object. Exclusive of transaction and design.

-steps <int>

Number of operations to redo. Applicable only with -design or -config.

Default: 1.

Examples

```
# Create three rectangles in the layout design
le::createRectangle {{0
-0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"
le::createRectangle {{0.3
-0.1} {0.5 0.1}} -design [ed] -lpp "m1 drawing"
le::createRectangle
{{0.6 -0.1} {0.8 0.1}} -design [ed] -lpp "m1 drawing"
# Undo last three transactions
# No rectangle will remain in the design
de::undoTransaction -design [ed]
-steps 3
# Redo last two transactions
# Two rectangles will be restored
de::redoTransaction -design [ed] -steps 2
```

de::redraw

Description

Redraws the content of an design editor window.

Syntax

```
de::redraw [-window <deCanvas | giWindow | int>]
```

Arguments

-window

The window or canvas to redraw.

Examples

```
de::redraw -window 2
```

de::registerHelp

Description

Registers a new Help topic.

Syntax

```
de::registerHelp (-helpID <string> | -command <deCommand | string>  
| -windowType <giWindowType | string> | -assistant <giAssistant |  
string> | -dialog <giDialog | string>) [-collectionID <string>] -target  
[<string>] -type <enum>
```

Arguments

-helpID

Help ID of topic.

-collectionID

Collection ID of topic.

-target

Target of topic, related to type.

-type

Type of topic: {pdf | proc | url}.

-command

Command for which to create a topic.

-windowType

Window type for which to create a topic.

-assistant

Assistant for which to create a topic.

-dialog

Dialog box for which to create a topic.

Examples

```
# Create the dialog to register help for it
set dialog [gi::createDialog newDialog -title "New Dialog"]
# Set path of the help file
set helpPath "https://synopsys.com"
# Register help for the above dialog
de::registerHelp -dialog $dialog -target $helpPath -type url
```

de::removeBreakpoints

Description

Removes some or all breakpoints. If a breakpoint is not specified, removes all breakpoints.

Syntax

```
de::removeBreakpoints [<lines: [int]>]
```

Arguments

lines

A Tcl list of line numbers indicating breakpoints to remove.

Default: All breakpoints

Examples

```
# Replay replay.log file in interactive mode
de::replay replay.log -interactive true
# Set two breakpoints
de::setBreakpoint 20
de::setBreakpoint 32
# Remove breakpoints
de::removeBreakpoints
# Get breakpoints
# It will return empty list
de::getBreakpoints
```

de::removeObject

Description

Deselects the specified object and removes it from the command's working set. Ignored if the object is not selected.

Syntax

```
de::removeObject <object: deFigure> [-window <giWindow | unsigned int> |  
-context <deContext>]
```

Arguments

object

Object to deselect.

-window

Window in which to send the event.

-context

Context in which to send the event.

Examples

```
# Create three rectangles in the layout designle::createRectangle {{0  
-0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"le::createRectangle {{0.3  
-0.1} {0.5 0.1}} -design [ed] -lpp "m1 drawing"le::createRectangle {{0.6  
-0.1} {0.8 0.1}} -design [ed] -lpp "m1 drawing"# Start to move all of  
themde::selectAll [de::getActiveContext]ile::movede::addPoint {0.4 0}#  
Remove one rectangle from selection  
# Only two rectangles will remain in moving set  
de::removeObject [de::getFigures {{0 -0.1} {0.2 0.1}} -design [ed]]
```

de::removePoint

Description

Moves the cursor to the given point and sends a removePoint event to the active command in a design editor window. This event is equivalent to a canvas mouse click for data point removal in many interactive commands.

The point argument is in the view design's coordinate space.

Syntax

```
de::removePoint <point: point> [-window <giWindow | unsigned int> |  
-context <deContext>]
```

Arguments

point

Point to remove, in user units.

-window [giWindow](#) | <unsigned int>

Window in which to send the event.

-context

Context in which to send the event.

Examples

```
# Start interactive command
ile::createRectangle

# Add starting point
de::addPoint {0.1 0.1}

# Remove that point and add new starting point
de::removePoint {0.1 0.1}
de::addPoint {0.5 0.5}
```

de::removeTool

Description

Removes a tool from its association with an editor and thus makes the tool unavailable for designs opened with the given association.

Syntax

```
de::removeTool <tool: deTool> -editor <deEditor | string>
```

Arguments

tool

Tool to remove from the given association.

-editor

Editor for which tool is no longer available.

Examples

```
# Get parasitics tool
set tool [de::getTools xtParasiticsTool]
# Remove parasitics tool from schematic editor
de::removeTool $tool -editor seSchematic
```

de::renumberInstances

Description

Renumbers the instances in the specified collection.

Syntax

```
de::renumberInstances <instances: {<deFigure>} | {<oaFig>} | {<oaInst>}>
  [-ordering <enum>] [-startFrom <int>] [-customPrefix <string>]
  [-prefixToUse <enum>]
```

Returns

{<oaFig>}: Collection of modified instances.

Arguments

`instances`

Collection of instances to modify from the same design.

`-ordering`

Specifies if the renumbering will be row- or column-based ({ column | row }).

Default: row

`-startFrom`

The starting number to use for indexing. Default: 0

`-customPrefix`

The additional prefix string to append. Default: ""

`-prefixToUse`

The numeric prefix to use when indexing instances ({ current | customOnly | default }). Default: current

Examples

In this example, `de::renumberInstances` renumbers all instances in the active design.

```
de::renumberInstances [db::getInsts -of [db::getAttr editDesign \
  -of [de::getActiveContext]]]
```

de::repeatCommand

Description

Restarts a previous command.

Repeats the most recent interactive command that has been in operation in the design. The command is repeated whether it started successfully previously and whether it was aborted. In the inFix mode, an infix addPoint is sent to the command after it starts.

Syntax

```
de::repeatCommand [-window <giWindow | unsigned int> | -context  
  <deContext>]
```

Arguments

-window

Window in which to start the command.

-context

Context in which to start the command.

Examples

```
# Start an interactive command de::move# Abort it de::abortCommand# Use  
de::repeatCommand to invoke last command again  
de::repeatCommand
```

de::replay

Description

Reruns the commands and other operations in the log file or as specified by the command arguments.

Syntax

```
de::replay [<text: string>] [-filepath <string> | -body <string>]  
  [-interactive <bool>]
```

Arguments

text

File or log body to replay.

-filepath

File to replay.

-body

Replay log body.

-interactive <bool>

Enters replay in the interactive debug mode.

Default: false

Examples

These two examples each replay the `replay.log` file.

```
de::replay replay.log
de::replay replay.log interactive true; de::run
```

de::replayCommand

Description

Replays an interactive edit on the currently selected set. This may be used for example, to move the currently selected objects by the same amount as the last time an object was moved.

The command argument specifies the command to invoke. If unspecified, the last used command is assumed.

Syntax

```
de::replayCommand [<name: deCommand | string>] [-window <giWindow |
    unsigned int> | -context <deContext>]
```

Arguments

name

Command to replay.

-window

Window in which to replay the command.

-context

Context in which to replay the command.

Errors

- The window is not a design editor window.
- Invalid window id.
- Command %s is not a known command name.

Examples

```
# Create two rectangles in the layout designle::createRectangle {{0
-0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"le::createRectangle
{{0.3 -0.1} {0.5 0.1}} -design [ed] -lpp "m1 drawing"# Move one of
them ile::movede::addPoint {0.4 0}de::addPoint {1 0}de::abortCommand#
Select another rectanglele::deselectAll [de::getActiveContext]de::select
```

```
[de::getFigures {{0.1 0}} -design [ed]]# de::replayCommand will move  
this rectangle the same way as the first  
# one  
de::replayCommand
```

de::reportOpenTransactions

Description

Reports Open Transactions.

Syntax

```
de::reportOpenTransactions [-design <oaDesign>] [-type <enum>] [-close  
    <bool>]
```

Returns

<none>

Arguments

-design

Design for which to report open transactions.

-type

Transaction types to report. ({ all | design | runtime }). Default: all

-close

If true, close any open transactions. Default: false

Examples

```
# Start transaction  
set design [ed]  
de::startTransaction "Create Rectangles" -design $design  
# Print information about open transactions  
de::reportOpenTransactions -design $design
```

de::restoreBookmark

Description

Restores any edit-context bookmark.

This command warns if the specified bookmark is out-of-context.

An interactive version of this command is not available.

Syntax

```
de::restoreBookmark <bookmark: deBookmark> -window <giWindow | int>  
[-fromTop <bool>]
```

Arguments

deBookmark

The bookmark to restore.

-window giWindow | <int>

The window in which to restore the bookmark.

-fromTop <bool>

If the bookmark is out-of-context, open the partial context from the topmost design downwards or from the view design upwards.

Default: true

Examples

```
# Create new bookmark  
de::createBookmark newBookmark -window [gi::getWindows $id]  
# Zoom out the design  
de::zoom -window $id -factor 0.25  
# Restore previous bookmark  
# Viewport will be restored  
set bm [de::getBookmarks newBookmark]  
de::restoreBookmark $bm -window [gi::getWindows $id]
```

de::return

Description

Closes the edit file and returns to the parent file. If the edit file is the view file, the view file will also change.

Syntax

```
de::return <context: deContext> [-levels <int>]
```

Returns

Modified context object (deContext)

Arguments

context

Context object in which to return.

`-levels`

Number of levels to close. Use a negative number to indicate all levels.

Default: 1

Examples

```
# Descend 3 levels
de::descend cp -context [de::getActiveContext]
de::descend I52 -context [de::getActiveContext]
de::descend amp -context [de::getActiveContext]
# Return to top level
de::return [de::getActiveContext] -levels -1
```

de::revert

Description

Discards all unsaved changes in the specified files, restoring it to the on-disk version.

Syntax

```
de::revert <collection: dbCollection | dmCellViewName | {<dmCellView>} |  
{<dmDMFile>} | {<oaDesign>}> [-force <bool>]
```

Returns

<none>

Arguments

`collection`

Collection of contexts whose edit design to revert.

`-force`

Reverts the design even if there are unsaved edits and the design is open in a graphical editor.

Default: false

Examples

```
set design [ed]
# Destroy all instances in active design
db::destroy [db::getInsts -of $design]
# Revert changes
de::revert $design
```

de::run

Description

Starts executing each command in the currently loaded interactive replay file until a breakpoint is reached or the end of the file. When the end is reached, the file is unloaded and the breakpoints cleared.

Syntax

```
de::run
```

Arguments

None.

Examples

In this example, `de::run` executes the `replay.log` file loaded by `de::replay`. See also, [de::setBreakpoint](#).

```
de::replay replay.log -interactive true  
de::run
```

de::runBackgroundImporter

Imports digital designs in the background.

Syntax

```
de::runBackgroundImporter <objects: {<dmCell>} | {<dmCellView>} |  
{<oaLib>}>
```

Arguments

`objects`

A collection of selected libraries, cells, or cellviews to be imported.

de::save

Description

Saves the specified files which must be opened in a context.

Syntax

```
de::save <collection: dbCollection | dmCellViewName | oaDesign |  
{<dmCellView>} | {<dmDMFile>}>
```

Arguments

collection

Collection of contexts whose edit file will be saved or the file itself.

Examples

```
# Open schematic design in read-only mode
set context [de::open [dm::getCellViews schematic -cellName pll_top
  -libName PLL] -readOnly false]
# Add sheet symbol
se::createInst -cellName A5Sheet -libName sheets -viewName symbol -origin
  {0 0} -design [db::getattr context.editDesign]
# Save change
de::save $context
```

de::saveAs

Description

Saves the specified file into a new or existing cellview.

If a file with the same name already exists in the specified cellview, the command will terminate unless the -force argument is set to true.

The library must already exist, but the cell and view are created as necessary.

The specified file must be open within a DE context.

Syntax

```
de::saveAs <object: deContext | dmCellView | dmCellViewName | dmDMFile |
  oaDesign> -to <dmCellView | dmCellViewName> [-force <bool>]
```

Arguments

object

Context whose edit file will be saved or the file itself.

-to

Target cellview into which the file will be saved.

-force

When saving to a different cellview that already exists, terminate unless -force is true.

Examples

```
set context [de::open [dm::getCellViews schematic -cellName pll_top  
-libName PLL] -readOnly false]  
de::saveAs $context -to PLL/backup/pll_top_schematic -force true
```

de::saveFavoriteDevices

Description

Writes favorite devices to a file on disk, optionally overwriting an existing file. See also:
[de::getFavoriteDevices](#).

Syntax

```
de::saveFavoriteDevices [{<{devices}: [dbCollection]>}] -filePath <string>  
[-force <bool>]
```

Arguments

{devices}

Collection of devices to save. If omitted, all favorite devices are saved.

-filePath

Path of file to write.

-force

If true, overwrite existing files.

Default: false

Examples

```
# Create favorite devices from different libraries  
set cv1 [dm::findCellView symbol -cellName cap -libName analogLib]  
set cv2 [dm::findCellView symbol -cellName res -libName analogLib]  
set cv3 [dm::findCellView symbol -cellName Cc -libName parasitics]  
de::createFavoriteDevice -master $cv1 -viewType schematic  
de::createFavoriteDevice -master $cv2 -viewType schematic  
de::createFavoriteDevice -master $cv3 -viewType schematic  
# Save devices from analogLib to favDevices_analogLib.xml file  
de::saveFavoriteDevices [de::getFavoriteDevices -filter  
  {%master.libName=="analogLib"}] -filePath ./favDevices_analogLib.xml  
# Save all favorite devices to favDevices_all.xml file  
de::saveFavoriteDevices -filePath ./favDevices_all.xml
```

de::saveInterfaceElements

Description

Saves interface elements to the specified file.

Syntax

```
de::saveInterfaceElements <elements: dbCollection> -filePath <string>  
[-force <bool>]
```

Arguments

elements

Interface elements to save.

-filePath

Filename to save.

-force

Overwrite the existing file.

Default is: false.

Errors

- File exists and `-force` is false.
- File exists and has no write permission.
- File folder does not exist.
- File folder has no write permission.

Examples

```
# Create interface element on OUT net  
set context [de::getActiveContext]  
set design [db::getAttr context.editDesign]  
set net [db::getNets OUT -of $design]  
set hc [db::getAttr context.hierarchy]  
set n [nl::getAMSNetlisters AMS]  
de::createInterfaceElement -on $net -direction a2d -context $hc  
-netlistner $n  
  
# Save interface elements to ie.xml file  
de::saveInterfaceElements [de::getInterfaceElements -context $hc  
-netlistner $n] -filePath ./ie.xml  
  
# Remove interface elements and load them from ie.xml file
```

```
db::destroy [de::getInterfaceElements -context $hc -netlister $n]  
de::loadInterfaceElements -filePath ./ie.xml
```

de::saveOLPPresets

Description

Saves deOLPPreset state from the current design state to tagged xml file.

Syntax

```
de::saveOLPPresets <preset: dbCollection> -filePath <string> [-overwrite  
  <bool>]
```

Arguments

preset

Collection of matching deOLPPreset objects.

-filePath

File to save.

-overwrite

If true, overwrites the existing file; otherwise, prints an error message.

Default: false

Examples

```
# Create two OLP presets  
de::createOLPPreset metals -from [ed]  
de::createOLPPreset vias -from [ed]  
# Save "metals" preset to metals.xml  
de::saveOLPPresets [de::getOLPPresets metals] -filePath ./metals.xml  
# Save all presets to allPresets.xml  
de::saveOLPPresets [de::getOLPPresets] -filePath ./allPresets.xml  
# Get OLP presets and print their names  
set presets [de::getOLPPresets]  
puts [db::createList [db::getAttr presets.title]]
```

de::saveProbes

Description

Saves specified probes defined for a design to a specified file. If the filename exists, the command will throw an error unless -force is true. The file will be written even if no probes exist in the given deProbe collection. All probes in the collection must come from the same root context.

Syntax

```
de::saveProbes <probeList: {<deProbe>}> -filePath <string> [-force  
  <bool>]
```

Arguments

{[deProbe](#)}

Probes to save.

-filePath

Filename to save.

-force

Overwrite existing files.

Default: false

Examples

```
set context [de::getActiveContext]  
db::foreach net [db::getNets -of [db::getAttr context.editDesign]] {  
    de::createProbe [db::getAttr net.name] -context $context -type  
    net -description "Net [db::getAttr net.name]" -purpose info -packet  
    highlight[expr round (rand()*10)] -tool Custom  
}  
# Save all probes in external file  
de::saveProbes [de::getProbes -context $context] -filePath probes.xml
```

de::select

Description

Selects the specified object, sub-object or collection. It is expected that the object will commonly be the return value of `de::getActiveFigure` or one of the query commands defined above.

Only selectable objects will be selected. Selectable objects are visible.

If the object, sub-object or whole collection was already selected, this command does nothing.

If an object is passed which contains hierarchy, the root instance will be selected within its design.

Objects are selected within the select set of their design.

All selected objects are then returned.

Syntax

```
de::select <objects: {<deFigure>} | {<oaFig>} | {<oaFigGroup>} |  
{<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}} [-replace <bool>]
```

Arguments

objects

Collection of selected figures.

-replace

Deselect all objects not included in the specified figure set.

Default: false

Examples

In the following example, all wire labels in active schematic design get selected.

```
set design [ed]  
de::select [de::getFigures -design $design -objectTypes  
[de::getObjectFilters seSchWireName]]
```

de::selectAll

Description

Selects all objects that are visible and selectable within the specified design returning all selected objects in the set including those which were already selected.

Syntax

```
de::selectAll <context: deContext>
```

Returns

Collection of selected figures ({defigure})

Arguments

deContext

Design in which to select all objects.

Examples

In the following example, all figures in active schematic design get selected.

```
de::selectAll [de::getActiveContext]
```

de::selectLogic

Description

Select logic within the specified design context.

Syntax

```
de::selectLogic <objects: dbCollection | {<oaFig>} | {<oaFigGroup>} |  
  {<oaGroup>} | {<oaInst>} | {<oaInstTerm>} | {<oaNet>} | {<oaPin>} |  
  {<oaTerm>}}> -context <deContext>
```

Returns

<dbCollection> : Collection of selected hierarchical objects

Arguments

objects

Logic to select.

-context

Context within which to select logic.

Examples

```
# Open Design Navigator to see logically selected objects.  
# Logically select I0 instance  
set context [de::getActiveContext]  
set design [db::getattr context.editDesign]  
de::selectLogic [db::getNext [db::getInsts I0 -of $design]] -context  
  $context  
# Logically select IN net  
de::selectLogic [db::getNext [db::getNets IN -of $design]] -context  
  $context
```

de::send

Send a command to the other end of a running design compare session. Use
de::connect to establish a connection before calling this function.

Syntax

```
de::send -context <deContext> [-command <string>] [-wait <bool>]
```

Returns

<none>

Arguments

`-context`

Design context from which the other design on another session was opened

`-command`

Command to send to the other session

`-wait`

If true, wait on the response and return the result. Default: false.

de::sendComplete

Description

Sends an OnPostCommandComplete event to the tool. This command is used by Tcl Interactive Commands to signal the completion of an interactive edit.

Syntax

```
de::sendComplete <command: deCommand> -eventData <{<deFigure>} |  
{<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaShape>} | {<oaVia>}}>
```

Arguments

`<command>`

name of the Tcl Interactive Command that completes the cycle.

`-eventData {oaObject}`

event data to send with the event.

Default: "".

de::sendMessage

Description

Sends an informational, warning, or error message to the console or shell as well as to the session log and optionally also to a secondary log file.

Syntax

```
de::sendMessage <message: string> [-severity <enMessageSeverity>]  
[-logFiles [<string>]]
```

Arguments

`<string>`

The message string.

`-severity`

The message severity: {error | information | logComment | warning}.

Default: information

`-logFiles`

Tcl list of secondary log files to which the message will also be sent.

Default: No secondary log.

Examples

In each of these three examples, `de::sendMessage` prints the corresponding message in the Console and additionally sends it to the log file.

Error: This is an error message.

Warning: This is a warning message.

Information: This is an information message.

```
de::sendMessage "This is an error message." -severity error
de::sendMessage "This is a warning message." -severity warning
de::sendMessage "This is an information message." -severity \
    information
```

de::setActiveLPP

Description

Sets the active layer-purpose pair (LPP). If a layer is passed, the default LPP for that layer becomes the active LPP.

Syntax

```
de::setActiveLPP <lpp: deLayer | deTechLPP>
```

Arguments

`deLPP`

LPP to set.

Errors

- LPP does not exist.
- Owner of deLPP is not a design.
- LPP design is a not a layout design.
- LPP design is not opened in a design editor.

Examples

In this example, `de::setActiveLPP` sets the {nwell drawing} LPP to be active.

```
de::setActiveLPP [de::getLPPs {nwell drawing} -from \  
    [de::getActiveContext ]]
```

de::setBreakpoint

Description

Sets a breakpoint at the specified line number. If the line number is omitted, the breakpoint is inserted immediately after the `de::setBreakpoint` command. This is useful, for example, to insert a breakpoint directly into a log file by adding `de::setBreakpoint` where the replay should stop.

Syntax

```
de::setBreakpoint [<line: [int]>]
```

Arguments

line

Line number in the log file where the breakpoint is placed.

Default: The line immediately after the specified `de::setBreakpoint` line.

Examples

In this example, `de::replay` executes `replay.log` in the interactive debug mode, `de::setBreakpoint` inserts a breakpoint at the fifteenth line, and `de::run` replays the log file until it reaches the set breakpoint.

```
de::replay replay.log -interactive true  
de::setBreakpoint 15  
de::run
```

de::setCursor

Description

Sets the current actual cursor location and returns it.

Syntax

```
de::setCursor [<window: deCanvas | giWindow | int>] -point <point>
```

Returns

<point> : The new cursor location.

Arguments

window

Window or canvas to get cursor from.

Default: active window.

-point <point>

New cursor coordinates in user units.

Default: No default value.

Errors

- An invalid window object is passed.
- Invalid point is passed.

Examples

```
# Move cursor to {0.1 0.1} point
de::setCursor -point {0.1 0.1}
```

de::setEditMode

Description

Changes the open mode of the edit design in the specified context. If -save is false, pending changes will be discarded when changing to read-only mode.

Switching to read/write mode will revert the design to its on-disk version.

Syntax

```
de::setEditMode <collection: dbCollection | dmCellViewName |  
{<dmCellView>} | {<dmDMFile>} | {<oaDesign>}> -editable <bool> [-save  
<bool>]
```

Arguments

collection

Collection of contexts containing edit designs to make editable.

-editable

If true, sets the design to read/write mode. If false, sets the design to read-only mode.

-save

Saves changes prior to changing to read-only.

Examples

```
# Open schematic design in read-only mode
set context [de::open [dm::getCellViews schematic -cellName pll_top
  -libName PLL] -readOnly true]
# Change mode to editable (append)
de::setEditMode $context -editable true
# Add sheet symbol
se::createInst -cellName A5Sheet -libName sheets -viewName symbol -origin
  {0 0} -design [db::getattr context.editDesign]
# Save changes and make design read-only
de::setEditMode $context -editable false -save true
```

de::setErrorViewerCategory

Description

Sets the category of deErrorViewerDatabase

Syntax

```
de::setErrorViewerCategory <of: deErrorViewerDatabase> -fields [<enum>]
  [-hide [<enum>]]
```

Returns

<none>

Arguments

of

The deErrorViewerDatabase object

-fields

List of fields that forms the category ({ associatedObjects | design | message | parentMarkers | severity | shortMsg | tool | user1 | user2 | user3 | user4 })

-hide

Specify a list of fields that will not be selected by default in the category menu
({ associatedObjects | design | message | parentMarkers | severity | shortMsg |
tool | user1 | user2 | user3 | user4 })

Examples

```
# Create new Error Viewer database
set design [ed]
de::createErrorViewerDatabase newErrDB -design $design -title "New
Database"
# Create new Error Viewer marker and add it to the database
de::createErrorViewerMarker [de::getErrorViewerDatabases newErrDB]
-points {{0 0} {1 1}} -msg "new Error" -shortMsg err1 -tool new_tool
# Set the category of the database to message field
set db [de::getErrorViewerDatabases newErrDB]
de::setErrorViewerCategory $db -fields message
```

de::setErrorViewerDetailInfo

Description

Customizes the detail information for deErrorViewerDatabase

Syntax

```
de::setErrorViewerDetailInfo <of: deErrorViewerDatabase> -fields [<enum>]
```

Returns

<none>

Arguments

of

The deErrorViewerDatabase object

-fields

Displays the details of the specified fields in the Error Viewer:

{ associatedObjects | design | message | parentMarkers | severity | shortMsg |
tool | user1 | user2 | user3 | user4 }.

Examples

```
# Create new Error Viewer database
set design [ed]
de::createErrorViewerDatabase newErrDB -design $design -title "New
Database"
# Create new Error Viewer marker and add it to the database
```

```
de::createErrorViewerMarker [de::getErrorViewerDatabases newErrDB]
    -points {{0 0} {1 1}} -msg "new Error" -shortMsg err1 -tool new_tool
# Set the detail information to show severity, message and short message
  of the error
set db [de::getErrorViewerDatabases newErrDB]
de::setErrorViewerDetailInfo $db -fields {severity message shortMsg}
```

de::setErrorViewerFieldName

Description

Customizes the field name for each field.

Syntax

```
de::setErrorViewerFieldName <of: deErrorViewerDatabase> -field <enum>
    -name <string>
```

Returns

<none>

Arguments

of

The deErrorViewerDatabase object.

-field

The field: { associatedObjects | design | message | parentMarkers | severity | shortMsg | tool | user1 | user2 | user3 | user4 }.

-name

The name for the field.

Examples

```
# Create new Error Viewer database
set design [ed]
de::createErrorViewerDatabase newErrDB -design $design -title "New
  Database"
# Create new Error Viewer marker and add it to the database
de::createErrorViewerMarker [de::getErrorViewerDatabases newErrDB]
    -points {{0 0} {1 1}} -msg "new Error" -shortMsg err1 -tool new_tool
# Change "Message" field name to "Error Info"
set db [de::getErrorViewerDatabases newErrDB]
de::setErrorViewerFieldName $db -field message -name "Error Info"
```

de::setErrorViewerTip

Description

Customizes the tip for deErrorViewerDatabase.

Syntax

```
de::setErrorViewerTip <of: deErrorViewerDatabase> -fields [<enum>]
```

Returns

<none>

Arguments

of

The deErrorViewerDatabase object.

-fields

The fields to be shown in the tip: { design | message | severity | shortMsg | tool | user1 | user2 | user3 | user4 }.

Examples

```
# Create new Error Viewer database
set design [ed]
de::createErrorViewerDatabase newErrDB -design $design -title "New
Database"
# Create new Error Viewer marker and add it to the database
de::createErrorViewerMarker [de::getErrorViewerDatabases newErrDB]
  -points {{0 0} {1 1}} -msg "new Error" -shortMsg err1 -tool new_tool
# Set the tip text to show error message and severity
set db [de::getErrorViewerDatabases newErrDB]
de::setErrorViewerTip $db -fields {message severity}
```

de::setViewport

Description

Sets the viewport to be the smallest box that includes the specified bounding box.

The new viewport will not necessarily match the specified box. The viewport will be symmetrically enlarged to preserve the current aspect ratio.

Syntax

```
de::setViewport [-window <deCanvas | giWindow | int>] -box <box>
```

Returns

The new viewport (<box>)

Arguments

`-window`

The window or canvas to modify.

Default: active window

`-box <box>`

The bounding box for zooming.

Errors

- The box was invalid.
- The window ID was invalid.
- There was no open design in the specified window.

Examples

In the following example, `de::setViewport` sets the smallest bounding box, which includes the `{{0 0} {1 1}}` box, and returns the actual viewport box.

```
de::setViewport -box {{0 0} {1 1}}
```

de::shiftCursor

Description

Shifts the cursor relative to its current position in specified steps.

Syntax

```
de::shiftCursor [<window: deCanvas | giWindow | int>] [-point <point>]  
[-delta <point>] [-nsteps <int>] [-left <int>] [-right <int>] [-down  
<int>] [-up <int>]
```

Returns

Final cursor position in user units (<point>).

Arguments

`window`

Window or canvas to shift cursor in.

Default: active window

`-point <point>`

Point from which the cursor should be moved in user units.

Default: Current cursor position

`-delta <point>`

Move vector (a pair of offsets in both directions).

`-nsteps <int>`

Number of steps in which to subdivide the move vector.

Default: 1

`-left <int>`

Number of grid units to shift left.

Default: 0

`-right <int>`

Number of grid units to shift right.

Default: 0

`-up <int>`

Number of grid units to shift up.

Default: 0

`-down <int>`

Number of grid units to shift down.

Default: 0

Errors

- An invalid window object is passed.
- Invalid point is passed.
- Invalid delta offset is specified.
- invalid nsteps is specified.
- point, delta or nsteps is combined with left, right, up or down.

Examples

```
# Set cursor to {0.1 0.1} pointde::setCursor -point {0.1 0.1}# Move the  
cursor to left by 5 grid points and to up by 3 grid points  
de::shiftCursor -left 5 -up 3
```

de::showAbout

Description

Displays the **About Custom Compiler** dialog box. When the tool is in the non-graphical mode, the About information is displayed in the console.

Syntax

```
de::showAbout
```

Arguments

None.

de::showBookmarkManager

Description

Shows the Bookmark Manager window. If the window is already open, it will be displayed on top of other windows and dialog boxes.

Syntax

```
de::showBookmarkManager
```

Arguments

None

de::showCloseDesigns

Description

Shows a dialog box listing all open designs in preparation for closing them.

Syntax

```
de::showCloseDesigns [-parent <giWindow | int>]
```

Returns

<giDialog>: Newly created dialog box.

Arguments

-parent

Parent window on which this dialog will be centered. The parent window can be any window, not just design windows.

de::showConnectionsNavigator

Description

Opens the Connections Navigator dialog box.

Syntax

```
de::showConnectionsNavigator [-parent <giWindow | int>]
```

Arguments

-parent

The window to use for showing inherited connections.

de::showConnectionsNavigatorOptions

Description

Opens the options dialog box for the Connections Navigator.

Syntax

```
de::showConnectionsNavigatorOptions [-parent <giWindow | int>]
```

Arguments

-parent

The connections navigator window which options to show.

de::showCreateBookmark

Description

Opens the **Create Bookmark** dialog box for the specified window. If the dialog is already open in the given parent, it will be displayed on top of other windows and dialog boxes. One dialog can be open for each parent window, which must be a design editor window.

Syntax

```
de::showCreateBookmark [-parent <giWindow | int>]
```

Returns

The dialog which represents the newly opened dialog box (*giDialog*).

Arguments

`-parent` `giWindow` | `<int>`

The window for which to create a bookmark.

Default: active editor window

de::showCrossCheckOptions

Description

Displays the cross-view check options dialog box.

Syntax

```
de::showCrossCheckOptions [-parent <giWindow | int>]
```

Returns

`<giDialog>`: Dialog window with cross-view check options.

Arguments

`-parent`

Optional window to associate with dialog specifically to supply scope for the preference values.

de::showDescendDialog

Description

Shows the **Descend Into** dialog box, which prompts for an alternate switched master into which to descend. It also allows interactively selecting the descended index of an iterated instance.

If the context object is open in a window, the dialog box will be modal to that window. Otherwise, it will be transient to the console.

If the dialog box is already open for the given context, it will be raised and focused. The specified context and instance are ignored. Only one dialog per context may be opened.

If a position is specified and an array is passed, the row and column that corresponds to the point will be used to populate the row and column fields of the dialog.

This command only works on oaDesign data.

If `-newWindow` is true, the command will create a new window once opened.

If `-readOnly` is true, the child design will be opened in read-only mode.

Syntax

```
de::showDescendDialog -context <deContext> -inst <oaInst>  
[-position <point>] [-readOnly <bool>] [-newWindow <bool>]
```

Arguments

-context

Context object for which to open the dialog.

-inst

Instance into which to descend.

-position <point>

Specifies a coordinate over the instance, which will be used to calculate the row/
column of an array and populate the form.

Default: Index 0,0

-readOnly <bool>

Opens the design in read-only mode.

Default: false

-newWindow

Opens the design in a new window.

Default: false

de::showDesignsPrompt

Description

This command shows the **Close Designs** dialog box. You can customize the title, prompt text, buttons, and exec procedure. If `-wait` is true, the dialog will be shown as modal and the command will not return until after triggering the exec procedure.

The execution procedure has the following signature:

```
proc execProc {buttonName fileList} {  
  
}
```

The `buttonName` argument is the title of the button given in the `-buttons` argument. The `fileList` argument is a collection of all files checked within the dialog. This procedure should return 1 on success or 0 on failure. If 0 is returned, the dialog will remain open. It is the client's responsibility to display an error message. Note that if the dialog is closed through a window manager control such as the X in the title bar or a window manager key

binding, this procedure will be called with the `buttonName` set to `Cancel` regardless of what is given to the `-buttons` argument.

If a dialog with the given name already exists, it will be closed and destroyed before showing this dialog.

Syntax

```
de::showDesignsPrompt <name: string> -title <string> -prompt <string>  
-buttons [<string>] [-default <string>] [-cancelButton <string>] [-files  
<{<dmDMFile>}>] -execProc <string> [-wait <bool>]
```

Returns

<giDialog>: Newly created dialog.

Arguments

`name`

Name of the new dialog object.

`-title`

Dialog title.

`-prompt`

Dialog prompt.

`-buttons`

Button titles.

`-default`

Default button. Must match a string given in the buttons argument.

`-cancelButton`

Default button. Must match a string given in buttons.

Default: Cancel

`-files`

Files to list. These must be open in a design editor.

Default: All files open in all contexts

`-execProc`

Callback procedure called when a button is pressed (see description).

`-wait`

If true, the dialog will be shown modal and this command will not terminate until after executing a button callback.

Default: false

Errors

- The specified file is not opened in a DE context.
- Default button does not match a button in `-buttons`.

de::showGeneralOptions

Description

Shows the **General Options** dialog box. If the dialog is already opened, it will be raised and the existing object returned.

Syntax

```
de::showGeneralOptions [-parent <giWindow | int>]
```

Returns

Newly opened or already opened dialog. (*giDialog*).

Arguments

`-parent` *giWindow* | *<int>*

Parent window.

Default: Active window (any type)

de::showGradientEditor

Description

Opens the Gradient Editor widget.

Syntax

```
de::showGradientEditor [-parent <giDialog | giWindow | int>] -gradient  
  <deGradient> -dbLimits [<float>] -dbHandle <string> -tables [<string>]  
  [-precision <int>] [-execProc <string>] [-closeProc <string>]
```

Returns

<giDialog> : The created dialog

Arguments

`-parent`

The parent window or dialog box. Default: 0

`-gradient`

Gradient object to be edited.

`-dbLimits`

The minimum and maximum database values for the selected gradient expression.

`-dbHandle`

Path to the database based on which the gradient has been created.

`-tables`

List of tables whose columns are used in gradient expression.

`-precision`

Parameter display precision. Default: 5

`-execProc`

If set, the OK and Apply buttons may be added which trigger this method. Called with the editor object.

`-closeProc`

Optional procedure to execute when the Gradient Editor is closed (either using the Cancel, OK, or Apply button. Called with the editor object.

de::showMakeDesignsReadOnly

Description

Shows a dialog listing all open editable designs.

Syntax

```
de::showMakeDesignsReadOnly [-parent <giWindow | int>]
```

Returns

<giDialog> : Newly created dialog.

Arguments

`-parent`

Parent window on which this dialog will be centered. May be any window, not just design windows.

de::showOpen3DICDesign

Shows the Open 3DIC Compiler Design dialog box with filtering.

Syntax

```
de::showOpen3DICDesign [-parent <giWindow>] -design <dmCellView>
                        [-readOnly <bool>]
```

Returns

<giDialog> :

Arguments

`-parent`

Parent window.

`-design`

Design cellview to open.

`-readOnly`

Initial state of the read-only flag. Default is: false

de::showOpenDesign

Description

Shows the **Open Design** dialog box. If the dialog is already opened, it will be raised and focused.

Syntax

```
de::showOpenDesign [-window <int | giWindow>]
```

Returns

<giDialog>: Newly created dialog box.

Arguments

`-window <int> | giWindow`

Window in which to open the dialog box. Information from the window is used to pre-populate the dialog box.

Default: Active editor window

de::showOpenFCDesign

Description

Shows the open Fusion Compiler Design dialog box with filtering.

Syntax

```
de::showOpenFCDesign [-parent <giWindow>] -design <dmCellView> [-readOnly  
  <bool>]
```

Returns

`<giDialog> :`

Arguments

`-parent`

Parent window.

`-design`

Design cellview to open.

`-readOnly`

Initial state of the read-only flag. Default is: false

de::showOpenICC2Design

Description

Shows the Open IC Compiler II Design dialog box with filtering.

Syntax

```
de::showOpenICC2Design [-parent <giWindow>] -design <dmCellView>  
  [-readOnly <bool>]
```

Returns

<giDialog> :

Arguments

-parent

Parent window.

-design

Design cellview to open.

-readOnly

Initial state of the read-only flag. Default: false

de::showParamFilter

Description

Displays the Parameter Filter for the given window.

Syntax

```
de::showParamFilter -parent <giWindow | int>
```

Returns

Created Parameter Filter dialog. (*giDialog*).

Arguments

-parent *giWindow* | <int>

The parent window.

de::showProbeBitSelectionDialog

Description

Displays the bit selection dialog box for probing objects with multiple bits.

Syntax

```
de::showProbeBitSelectionDialog <name: string> -type <enum> -window  
<giWindow | int>
```

Returns

<string>: User-selected probe name.

Arguments

name

Name of bussed object.

-type

Type of probe: {instTerm | instance | net | term}.

-window

The parent window for the dialog box.

de::showProperties

Description

Shows the **Property Editor** dialog box.

Syntax

```
de::showProperties [-context <deContext>] [-parent <giWindow | int>]
```

Returns

Properties dialog object.

Arguments

-context

Context containing selection to show.

-parent

Parent design editor window.

de::showRefreshDesigns

Description

Shows a dialog box listing all open, read-only, and out-of-date designs that have changed on disk since being opened.

Syntax

```
de::showRefreshDesigns [-parent <giWindow | int>] [-quiet <bool>]  
[-designs <{<dmDMFile>} | {<oaDesign>}>]
```

Returns

<giDialog>: Shown dialog or "" if the dialog is not shown due to all designs being refreshed.

Arguments

-parent

Parent editor window for the Refresh Designs dialog box.

-quiet

If true, disables all information messages shown when designs are reverted or when no designs should be reverted.

Default: false

-designs

Specifies the set of designs or files to prompt for.

de::showRenumberInstances

Description

Opens the dialog box for renumbering instances of the active design.

Syntax

```
de::showRenumberInstances [-window <giWindow | int>]
```

Returns

<giDialog>: Renumber instances dialog box.

Arguments

`giWindow` | <int>

Window or window id for which the dialog will be associated.

Default: active editor window

de::showReturnToLevelDialog

Description

Shows the **Return to Level** dialog box.

Syntax

```
de::showReturnToLevelDialog -context <deContext>
```

Returns

<giDialog>: Newly created dialog box.

Arguments

`-context`

Parent context of the dialog box.

de::showSaveDesignAs

Description

Shows the **Save Design As** dialog box. The command returns immediately. If the dialog is already open, it will be raised and focused. The context argument holds the edit file that will be saved.

If the context object is open in a window, the dialog will be modal to that window. Otherwise, it will not have a parent.

Syntax

```
de::showSaveDesignAs -context <deContext>
```

Returns

<giDialog>: Newly created dialog.

Arguments

`-context deContext`

Context whose edit design will be saved.

de::sleep

Description

Causes the Custom Compiler tool to sleep for N seconds and then continue with execution.

Syntax

```
de::sleep <:int>
```

Arguments

<int>

Number of seconds to sleep.

Examples

```
# Sleep for 5 seconds
de::sleep 5
```

de::snapToObject

Description

Computes and returns a point snapped to the given object. Uses the Snap Regions preference setting to determine which partial objects to snap.

Syntax

```
de::snapToObject [<window: deCanvas | giWindow | int>] -object <deFigure  
| oaFig> -point <point>
```

Returns

Cursor snap point (<point>)

Arguments

window

Window or canvas to get snap point from.

Default: active window

-point <point>

Point to calculate snap point for.

Default: No default value

-object dbObject

Object to snap to (terminal, grid,)

Default: No default value

Errors

- An invalid window object is passed.
- Invalid object is specified.
- Invalid point is passed.

Examples

```
# Snap the selected object to point {1 1}
de::snapToObject -object [de::getSelected -design [ed]] -point {1 1}
```

de::startDrag

Description

Moves the cursor to the given point and sends a startDrag event to the active command in a design editor window. This event starts a drag for interactive commands that support a click-drag-release use model.

The point argument is in the view design's coordinate space.

Syntax

```
de::startDrag <point: point> [-window <giWindow | unsigned int> |  
-context <deContext>] [-intent <enum>]
```

Arguments

point

Point at which to start the drag event.

-window

Window in which to send the event.

-context

Context in which to send the event.

-intent

Specifies the intended response for this event: default, deselect, or select.

Default: default

Examples

```
# Create three rectangles in the layout design  
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"  
le::createRectangle {{0.3 -0.1} {0.5 0.1}} -design [ed] -lpp "m1 drawing"  
le::createRectangle {{0.6 -0.1} {0.8 0.1}} -design [ed] -lpp "m1 drawing"  
# Start Move command and select two of them by drag commands  
ile::move  
de::startDrag {-0.15 0.15} -context [de::getActiveContext]  
de::endDrag {0.55 -0.2} -context [de::getActiveContext]  
de::addPoint {0.25 0}
```

de::startLog

Description

Initializes a secondary log file for use with `de::sendMessage`. The optional banner message is written to this file without being placed in the primary log file. If the file already exists, it will be clobbered. Call `de::endLog` to close this log file.

Syntax

```
de::startLog <file: string> [-banner <string>]
```

Returns

Log file identifier (filePath) (<string>).

Errors

Filesystem error.

Arguments

<string>

Full filesystem path to the secondary log file.

-banner <string>

Optional banner with which to initialize the secondary log file.

Default: "".

Examples

```
de::startLog test.log -banner "My Log"
# Redirect messages to another log.file
de::sendMessage "This is information" -logFiles test.log
de::sendMessage "This is warning" -severity warning -logFiles test.log
de::sendMessage "This is error" -severity error -logFiles test.log
de::endLog test.log
```

de::startRendererStatistics

Description

Gathers renderer statistics.

Syntax

```
de::startRendererStatistics [-file <string>] [-method <bool>] [-design  
  <bool>]
```

Returns

<none>

Arguments

-file

Outputs to the specified file.

-method

Reports the drawing method. Default: true

-design

Reports the design information. Default: true

de::startTransaction

Description

Starts the transaction.

All operations performed after the start transaction call and before the end transaction are aggregated into a single undo entry.

Syntax

```
de::startTransaction [<name: string>] [-design <oaDesign> | -config  
  <heConfig>]
```

Returns

Transaction object ([deTransaction](#))

Arguments

<string>

Transaction name.

Default: Autogenerated.

-design *oaDesign*

Design object.

-config *heConfig*

Config object.

-design and -config are mutually exclusive arguments.

Errors

Design is not opened.

Examples

In this example, multiple create commands call are wrapped in single transaction.

```
set design [ed]
set x 1
set y 1
set tr [de::startTransaction "Create Rectangles" -design $design]
db::foreach lpp [de::getLPPs -from $design] {
    le::createRectangle [list [list $x $y] [list [expr $x+0.5] [expr
    $y+0.5]]] -design $design -lpp [db::getattr lpp.lpp]
    incr x
    if {$x > 10} {
        set x 1
        incr y
    }
}
de::endTransaction $tr
```

de::stopBackgroundImporter

Stops the background import process.

Syntax

```
de::stopBackgroundImporter <objects: {<dmCell>} | {<dmCellView>} |
{<oaLib>}>
```

Arguments

objects

A collection of selected libraries, cells, or cellviews being imported.

de::stopRendererStatistics

Description

Stops gathering renderer statistics.

Syntax

```
de::stopRendererStatistics
```

de::suppressMessage

Description

Suppresses message with given ID.

Syntax

```
de::suppressMessage <id: string> [-suppress <bool>]
```

Returns

<none>

Arguments

id

Suppressing message ID or wildcard.

-suppress

Message suppression flag.

Default: true

Examples

```
# Get warning message by setting value to non-existent preference
db::setPrefValue test_pref -value 1
```

```
# Suppress this warning message
de::suppressMessage PREFERENCE-016 -suppress true
```

```
# Setting the non-existent preference no longer generates warning message
db::setPrefValue test_pref -value 1
```

de::transformPoint

Description

Transforms a coordinate from the view design's coordinate space to the edit-in-place design's or vice versa. This may be used, for example, to translate a design coordinate to a view design coordinate to pass to `de::addPoint` or the other semantic event commands.

Syntax

```
de::transformPoint <point: point> -to <enum> [-window <giWindow | int>]
```

Returns

Converted point (<point>).

Arguments

<point>

Point to convert.

-to

The type of coordinate to convert to: {edit | view}.

-window

Window in which to convert the coordinates.

Default: Active editor window

Errors

- Invalid window id.
- Window is not a design editor window.

Examples

The following example shows how to:

- Convert the {0.2 0.2} edit-in-place design coordinates to the view design coordinates.
- Convert the {1.2 1.2} view design coordinates to the edit-in-place design coordinates.

```
set context [de::getActiveContext]
set design [db::getattr editDesign -of $context]
set inst [le::createInst -viewName layout \
-cellName amp -libName PLL \
-design $design -origin {1 1}]
de::descendInPlace $inst -context $context
de::transformPoint {0.2 0.2} -to view
=> 1.2 1.2
de::transformPoint {1.2 1.2} -to edit
=> 0.2 0.2
```

de::undoSelection

Description

Undoes the last selection operation.

Syntax

```
de::undoSelection -design <oaDesign>
```

Arguments

`-design`

Design in which to undo last selection.

Examples

```
# Create a rectangle in the layout design le::createRectangle
{{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"# Create
another rectangle le::createRectangle {{0.3 -0.1} {0.5 0.1}}
-design [ed] -lpp "m1 drawing"# Select first rectangle de::select
[de::getFigures {{0.1 0}} -design [ed]]# Change selection and change
second rectangle de::deselectAll [de::getActiveContext] de::select
[de::getFigures {{0.4 0}} -design [ed]]# Undo selection
# First rectangle will be selected
de::undoSelection -design [ed]
```

de::undoTransaction

Description

Undoes one or more recent transactions. If a transaction is specified, then all transactions up to but not including that transaction are undone. If instead design or config and steps are specified, then the last N transactions (from the current transaction) in the specified design or config are undone.

Notes:

- `-design`, `-config`, and `-transaction` are mutually exclusive arguments.
- (`-design` or `-config`, and `-steps`) and `-transaction` are mutually exclusive arguments.

Syntax

```
de::undoTransaction (-transaction <deTransaction> | -design <oaDesign> |
  -config <heConfig>) [-steps <int>]
```

Arguments

`-transaction` *deTransaction*

Transaction to move back the transaction history. All later transactions are undone. Exclusive of design and config.

Default: none

`-design` *oaDesign*

Design object. Exclusive of transaction and config.

`-config` [heConfig](#)

Config object. Exclusive of transaction and design.

`-steps` `<int>`

Number of operations to undo. Applicable only to `-design` or `-config`.

Default: 1

Examples

```
# Create three rectangles in the layout design
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"
le::createRectangle {{0.3 -0.1} {0.5 0.1}} -design [ed] -lpp "m1 drawing"
le::createRectangle {{0.6 -0.1} {0.8 0.1}} -design [ed] -lpp "m1 drawing"
# Undo last two transactions
de::undoTransaction -design [ed] -steps 2
```

de::unhighlight

Description

Removes a collection of objects from the highlighted set. These objects must all be within, or descend from the design to which the highlighted set belongs. Objects not found in the set will be ignored.

Syntax

```
de::unhighlight <objects: {<deFigure>} | {<oaFig>} | {<oaFigGroup>}
| {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}} [-set
<deHighlightSet>]
```

Arguments

`{deFigure}` | `{oaFig}`

Objects to remove.

`-set` [deHighlightSet](#)

Highlight set from which to remove objects.

Default: highlight1

Examples

```
# Create a rectangle in the layout design
le::createRectangle {{0 -0.1} {0.2 0.1}} -design [ed] -lpp "m1 drawing"
# Create another rectangle
le::createRectangle {{0.3 -0.1} {0.5 0.1}} -design [ed] -lpp "m1
drawing"
# Highlight both rectangles
de::highlight [de::getFigures {{0.1
```

```
0}} -design [ed]]de::highlight [de::getFigures {{0.4 0}} -design [ed]]#
Unhighlight first rectangle
de::unhighlight [de::getFigures {{0.1 0}} -design [ed]]
```

de::uniquify

Description

Duplicates an instance master and modifies the instance to point to the new master. One may use this command to modify a single instance of a master, including a pcell submaster. Note that parameters are discarded by this operation.

If -to specifies a library, the new master will be created in the specified library. The name will be the original name with a suffix that makes it unique.

If -to specifies a cellview, the instance will be modified to point to the specified cellview instead. If the cellview does not exist, it will be created and populated with the contents of the original master.

Note that if the target design already exists, it will be reused rather than overridden. If the target specified is a library, this cannot happen as the name given will be unique.

Syntax

```
de::uniquify <instance: oaInst> -to <dmCellView | oaLib>
```

Returns

<oaInst>: Newly created instance object.

Arguments

instance

Instance to uniquify.

-to

Optionally specify a specific master to use or a library within which a unique master will be automatically created.

Examples

```
# get I0 instance from design and duplicate it to design library
set design [ed]
set inst [db::getInsts I0 -of $design]
de::uniquify $inst -to [db::getAttr design.lib]
```

de::zoom

Description

Allows the user to zoom in or out by a specified factor. The center is a relative center wherein objects at that point will appear to be in the same location after zooming as they were before zooming.

The zoom factor is relative to the current viewport size. For example, the command `de::zoom -factor 2` causes the length and width of the viewport to decrease 2x, resulting in the image increasing by 2x.

Syntax

```
de::zoom [-window <deCanvas | giWindow | int>] -factor <float> [-center  
  <point>]
```

Returns

The new viewport (<box>)

Arguments

`-window`

The window or canvas to zoom.

Default: active window

`-factor <float>`

The zoom factor.

`-center <point>`

Specifies a relative center for the zoom operation.

Errors

- The zoom factor was not positive.
- The window ID was invalid.
- There was no open design in the specified window.

Examples

In this example, `de::zoom` decreases viewport height and width by a factor of two, while objects under the cursor remain fixed in place.

```
de::zoom -factor 2 -center [de::getCursor]
```

ide::createProbe

Description

Interactively specifies a new probe object across the hierarchy (0x or below) with either canvas or text entry input.

Syntax

```
ide::createProbe [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ide::defineOrigin

Description

Starts the Define Origin command in the specified window. This command supports infix mode, which sets the first point of a canvas operation at the current cursor location without requiring a click when the command is initiated from a bindkey or context-sensitive menu.

Syntax

```
ide::defineOrigin [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

ide::descend

Description

Starts the Descend command in the specified window.

Syntax

```
ide::descend [<context: deContext | unsigned int>] -name <string>  
[-newWindow <bool>] [-promptView <bool>] [-type <enum>] [-readOnly  
  <enum>]  
[-inPlace <bool>] [-promptLineage <bool>] [-shadow <bool>]
```

Returns

<none>

Arguments

`context`

Context of activation.

`-newWindow`

Default value for the **New Window** option. Specifying this value modifies the preference. (Default is: false)

`-promptView`

Default value for **Prompt for View** option. Specifying this value modifies the preference value.

`-name`

Specifies the instance or figure group name for immediate descent. (Default is:)

`-type`

Specifies if the target descend type is instance, figure group, or both. ({ both | group | instance }, default: instance)

`-readOnly`

Specifies the descend open mode. Specifying this value modifies the `deDescend` preference value. ({ 0 | 1 | auto | false | true }, default: auto)

`-inPlace`

Defines the default value for the **In Place** option. Specifying this value modifies the preference value. (Default is: false)

`-promptLineage`

Defines the default value for **Prompt for Lineage** option. Specifying this value modifies the preference value. (Default is: false)

`-shadow`

Specifies if the selected lineage is shadowed. (Default is: false)

ide::editCanvasText

Description

Starts a canvas-editing session for the given text object.

Syntax

```
ide::editCanvasText [<context: deContext | unsigned int>] -object  
  <deFigure | oaFig>
```

Returns

<none>

Arguments

context

Context of activation.

-object

Text object to edit on canvas.

ide::enterBox

Description

Initializes the `ide::enterBox` command displayed in an editor window created by the Design Manager. If the command is killed before entering two points, then an empty string is returned.

Syntax

```
ide::enterBox [<context: deContext | unsigned int>] [-prompt <string>]  
              [-draw <bool>] [-returnViewCoordinates <bool>]
```

Returns

[<point>]

Arguments

context

The context in which this command is activated.

-prompt

Command prompt. The default is Enter Box)

-draw

Draw the box in the canvas The default is true.

-returnViewCoordinates

Returns points relative to edit design (false) or view design (true) The default is false.

Errors

- The context does not have a window.
- No content or window has been passed, and there is no active editor.

Examples

The following example shows how to enter a box on the canvas:

```
# Get active context
set context [de::getActiveContext]

# Enter box into design
ide::enterBox [de::getActiveContext]

#Command wait for user input in canvas to draw a box to return a box
#coordinates in Console.
```

ide::enterLine

Description

Initializes the `ide::enterLine` command displayed in an editor window created by the Design Manager. If this command is killed before you have entered two points, it returns an empty string.

Syntax

```
ide::enterLine [<context: deContext | unsigned int>] [-prompt <string>]
[-draw <bool>] [-angle <string>]
[-returnViewCoordinates <bool>] [-commandTitle <string>] [-numPoints
<int>]
```

Returns

[<point>]

Arguments

`context`

The context in which this command is activated.

`-prompt`

Command prompt. The default is Enter Line.

`-draw`

Draws the line after first point entered. The default is true.

`-angle`

Angle mode used to enter points. The default is orthogonal.

`-width`

Width of line drawn in user units. The default is 0.

`-returnViewCoordinates`

Returns points relative to edit design (false) or view design (true). The default is false.

`-numPoints`

Completes execution after reaching the specified number of points.

Errors

- The context does not have a window.
- No context window has been passed, and there is no active editor.

Examples

The following example shows how to enter a line onto the canvas:

```
# Get active context
set context [de::getActiveContext]

# Enter line into design
ide::enterLine [de::getActiveContext]

#Command wait for user input in canvas to draw a line to return a line
#coordinates in Console.
```

ide::enterPoint

Description

Initializes the `ide::enterPoint` command displayed in an editor window created by the Design Manager. If this command is killed before you have entered a point, it will return an empty string.

Syntax

```
ide::enterPoint [<context: deContext | unsigned int>] [-prompt <string>]
               [-returnViewCoordinates <bool>]
```

Returns

[<point>]

Arguments

`context`

The context in which this command is activated.

`-prompt`

Command prompt. The default is Enter Point.

`-returnViewCoordinates`

Returns point relative to edit design (false) or view design (true). The default is false.

Errors

- The context does not have a window.
- No context window has been passed, and there is no active editor.

Examples

The following example shows how to enter a point on the canvas:

```
# Get active context
set context [de::getActiveContext]

# Enter point into design
ide::enterPoint [de::getActiveContext]

#Command wait for user input in canvas to click to return a point
#coordinates in Console.
```

ide::enterPolygon

Description

Initializes the `ide::enterPolygon` command displayed in an editor window created by the Design Manager. If this command is inactivated before you entered three points, it will return an empty string.

Syntax

```
ide::enterPolygon [<context: deContext | unsigned int>] [-prompt  
  <string>]  
[-draw <bool>] [-angle <enum>] [-returnViewCoordinates <bool>]  
[-commandTitle <string>]
```

Returns

[<point>] : List of points representing polygon entered by user on the canvas.

Arguments

`context`

The context in which this command is activated.

`-prompt`

Command prompt. The default is Enter Polygon.

`-draw`

Draws the polygon after first point entered. The default is true.

`-angle`

Angle mode used to enter points: any, diagonal, or orthogonal. Default: orthogonal

`-returnViewCoordinates`

Returns points relative to edit design (false) or view design (true). The default is false.

`-commandTitle`

Short text to instruct user on what to do next. (Default: Enter Polygon)

Errors

- The context does not have a window.
- No context window has been passed, and there is no active editor.

Examples

The following example shows how to enter a polygon on the canvas:

```
# Get active context
set context [de::getActiveContext]

# Enter polygon into design
ide::enterPolygon [de::getActiveContext]

#Command wait for user input in canvas to draw a polygon to return a
#polygon coordinates in Console.
```

ide::highlight

Description

Interactive highlight command.

Syntax

```
ide::highlight [<context: deContext | unsigned int>] [-mode <enum>]
```

Arguments

context

Context of activation.

-mode

Highlight mode: name | shapes.

ide::pan

Description

This is the interactive version of `de::pan`. Click to define the new center, or click and drag to define the direction and distance of the pan. A drag-start sets the anchor point and the drag motion pans the window dynamically such that the pan vector is always equal to the drag vector (the cursor stays at the same user coordinates during the drag). This command terminates when the abort command is called.

Syntax

```
ide::pan [<context: deContext | unsigned int>] [-point <point>]
```

Arguments

context

Context of activation.

-point <point>

Specifies the first point of a drag operation and enters one shot mode where the command toolbar is not updated.

Default: Command will prompt

ide::pick

Description

Performs an interactive pick for design objects.

Syntax

```
ide::pick [<context: deContext | unsigned int>] [-prompt <string>]  
[-oneShot <bool>] [-types [<string>]] -pickProc [<attrObject>]
```

```
[-unpickProc [<attrObject>]] [-cancelProc [<attrObject>]] [-filterProc  
[<attrObject>]] [-abortProc [<attrObject>]]
```

Arguments

`context`

Context of activation.

`-prompt`

Prompt text for design editor status bar.

`-oneShot`

Terminates command after single pick.

Default: false.

`-types`

List of selectable logical objects: {net, instance, terminal, instTerm}.

`-pickProc`

Procedure to trigger when an object is picked.

`-unpickProc`

Optional procedure triggered when an object is Ctrl-picked.

`-cancelProc`

Procedure to trigger when the command is canceled.

`-filterProc`

Preselection filter procedure.

`-abortProc`

Procedure to trigger when command is aborted.

ide::selectByName

Description

Starts the Select by Name command.

Syntax

```
ide::selectByName [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

ide::selectByRegion

Description

Starts the Select by Region command. The `-region` argument may be used to predefine the region type. The `-select` argument may be used to predefine the select mode.

As this command will be useful in bindings, a `-point` option is provided which provides such a use model. Specifying this option implies the following behavior:

- The command starts in one-shot mode, terminating after the first complete cycle.
- The specified point represents the first point of the inputted region.
- The Command Options Toolbar will be unmodified, showing the formerly active command's toolbar, or command history if no other commands are on the stack.

For example, one would bind:

```
ide::selectByRegion %w -region rectangle -point %a to the Button1-DragStart  
event. Nothing else is necessary to perform select-by-rectangle on a click-drag operation.
```

The `-select` and `-criteria` arguments mirror command options given in the toolbar for use in bindings. If specified, they provide an override for the preference value and will not modify the preference itself. However, if the preference is changed while the command is active, the new preference value will be used.

Syntax

```
ide::selectByRegion [<context: deContext | unsigned int>] [-region  
<enum>] [-select <bool>] [-criteria <enum>] [-point <point>]
```

Arguments

`context`

Context of activation.

`-region`

Specifies a region type to use: {line | point | polygon | rectangle}.

Default: Last used region type

`-select`

If true, select. If false, deselect.

`-criteria`

Specifies the select criteria: {enclose | touch}.

`-point`

Specifies the first input coordinate.

Default: {}

ide::setViewport

Description

This is the interactive version of the `de::setViewport` command, which enables zooming in or out around a specified region. This command supports infix mode, which sets the first point of a canvas operation at the current cursor location without requiring a click when the command is initiated from a bindkey or context-sensitive menu.

The new viewport will not necessarily match the given box. It will be symmetrically enlarged to preserve the current aspect ratio.

Syntax

```
ide::setViewport [<context: deContext | unsigned int>] [-point <point>]  
[-direction <enum>] [-recursive <bool>] [-log <enum>]
```

Arguments

`context`

Context of activation.

`-point`

Specifies the first point of the set viewport operation and enters "one shot mode" where the COT is not updated. Default: Command will prompt.

`-direction`

Direction to zoom ({ bidir | in | out }, default is: in)

bidir: Zooms out when you move the cursor in a bottom-up (NW,NE) direction and zooms in when you move the cursor in a top-down (SE,SW) direction while the `ide::setViewport` command is active.
in: Zooms in in all directions.
out: Zooms out in all directions.

`-recursive`

Optionally override the `deSetViewRecursive` option. Specifying this option will modify the preference value. Default: `deSetViewRecursive`

`-log`

Logging mode {auto | explicit}. Default: auto

ide::testEvents

Description

An interactive command for testing semantic events.

Syntax

```
ide::testEvents [<context: deContext | unsigned int>]
```

Returns

<none>

Arguments

`context`

Context of activation.

ide::testStack

Description

An interactive command for testing the stack.

Syntax

```
ide::testStack [<context: deContext | unsigned int>]
```

Returns

<none>

Arguments

`context`

Context of activation.

ide::zoom

Description

This is the interactive version of the `de::zoom` command. Click to pan and center the view on the cursor and zoom in by a factor of 2. Middle-click to center the view on the

cursor and zoom out by a factor of 2. If you click and drag, this command behaves like `ide::setViewport`. This command is not recursive.

Syntax

```
ide::zoom [<context: deContext | unsigned int>] [-point <point>]  
[-direction <enum>] [-log <enum>]
```

Arguments

`context`

Context of activation.

`-point <point>`

Specifies the first point of a drag operation and enters "one shot mode" where the COT is not updated.

Default: Command will prompt.

`-direction`

Direction to zoom ({ `bidir` | `in` | `out` }, default is: `in`)

`-log`

Logging mode {`auto` | `explicit`}. The default is `auto`.

Enumerations

Table 57 Enumerations

| Name | Values | Description |
|---------------------------------|---|--|
| <code>enCommandType</code> | <code>proc</code> <code>interactive</code> | Command type. |
| <code>enCoordType</code> | <code>edit</code> <code>view</code> | Coordinate system: view design or edit design. |
| <code>enCyclingDir</code> | <code>next</code> <code>prev</code> | Direction of cycling. |
| <code>enDir</code> | <code>N</code> <code>NE</code> <code>E</code> <code>SE</code> <code>S</code> <code>SW</code> <code>W</code> <code>NW</code> | Geographic direction for panning. |
| <code>enOrder</code> | <code>first</code> <code>prev</code> <code>next</code> <code>last</code> <code>all</code> | The traversing order for the debugging command. |
| <code>enOverlayDrawPlane</code> | <code>static</code> <code>annotation</code> <code>edit</code> | Plane in the drawing stack on which an overlay design is placed. |
| <code>enPointType</code> | <code>point</code> <code>line</code> <code>rectangle</code> <code>polygon</code> | Type of point input. |

Table 57 Enumerations (Continued)

| Name | Values | Description |
|------------------|---|--|
| enProbePurpose | error warning info cross other | Probe purposes. |
| enProbeType | cellview terminal instance instTerm
net path | Probe types. |
| enRegionType | point line rectangle polygon | Type of region to act on. |
| enSelectCriteria | touch enclose | Selection criteria for select by region. |
| enSeverity | info warning error abort | The severity of the reported message/probe. Corresponds to OA enums. |
| enSystemGroup | all default user | System-defined OLP groups for use with <code>de::activateOLPGroup</code> . |
| enTransform | R0 R90 R180 R270 MY MYR90 MX
MXR90 | Transforms supported by OA. These names are consistent with the OA API. |
| enZoomDirection | in out bidir | Direction to zoom. |

Objects

This section covers the following:

- [deArgument](#)
- [deArgumentGroup](#)
- [deAssociation](#)
- [deBookmark](#)
- [deCanvas](#)
- [deClipboard](#)
- [deCommand](#)
- [deContext](#)
- [deEditor](#)
- [deFigure](#)

- [deHighlightSet](#)
- [deLayer](#)
- [deLPP](#)
- [deLPPSet](#)
- [deObjectFilter](#)
- [deOverlay](#)
- [deProbe](#)
- [deTool](#)
- [deTransaction](#)
- [deViolation](#)

deArgument

Represents the specification for a command argument. These objects may be created with the [de::createArgument](#) command and need not be destroyed with [db::destroy](#).

Table 58 *Attributes*

| Name | Type | Access | Description |
|----------|----------|-----------|--|
| name | string | Read-only | Argument name, i.e., -window. Positional arguments will be unnamed. |
| types | {string} | Read-only | List of type names for accepted argument value. |
| desc | string | Read-only | Description of the argument which optionally indicates its default value. For example, Window in which to create rectangle. Default: active editor window. |
| isList | bool | Read-only | Indicates that the argument is either a Tcl list or a collection. Both will be sent to the command as collections. |
| optional | bool | Read-only | True if the argument is optional. |

Table 58 *Attributes (Continued)*

| Name | Type | Access | Description |
|-----------|--|-----------|--|
| default | <i>oaObject</i> <i>dbObject</i> string | Read-only | Default value for optional arguments. This will be "" if a default value was not provided. |
| hidden | bool | Read-only | If true, the argument will not be shown in help. |
| exclusive | { <i>deArgument</i> } | Read-only | Arguments with which this argument is exclusive. |

deArgumentGroup

Represents a set of exclusive command arguments. These objects may be created with the [de::createArgumentGroup](#) command and cannot be destroyed with [db::destroy](#). Their life span is determined by the Tcl objects and commands which reference them.

Table 59 *Attributes*

| Name | Type | Access | Description |
|----------|------|-----------|----------------------------------|
| optional | bool | Read-only | This argument group is optional. |

deAssociation

The deAssociation object represents an editor/filename association. Associations are given a name independent of the editor itself and indicate a set of tools to open. This object may be created by the [de::createAssociation](#) command and may be destroyed by the [db::destroy](#) command.

Table 60 *Attributes*

| Name | Type | Access | Description |
|----------|-----------------|-----------|--|
| name | string | Read-only | Unique association name. |
| title | string | Read-only | Association name which is displayed in graphical interfaces. |
| filename | string | Read-only | DM filename for which this association will be used. |
| editor | <i>deEditor</i> | Read-only | Editor to be invoked. |

Table 60 *Attributes (Continued)*

| Name | Type | Access | Description |
|---------|-----------------|------------|--|
| tools | <i>{deTool}</i> | Read-only | Set of tools available. |
| task | string | Read/Write | Optional task override for the given editor. |
| default | bool | Read/Write | True if this association is the default for the filename. Modifying this attribute will set to false all default attributes of other deAssociation objects valid for the filename. |

deBookmark

The deBookmark object represents viewport and edit context of a window. The attributes of the object are described in the table below and can be retrieved or set by `db::getAttr` and `db::setAttr` commands.

Table 61 *Attributes*

| Name | Type | Access | Description |
|------------|-----------------|------------|--|
| name | string | Read/Write | Bookmark name. |
| stack | string | Read-only | Context stack. |
| task | string | Read-only | Task. |
| config | <i>heConfig</i> | Read-only | Default config view list for the design's viewType. Empty if no config is defined. |
| occurrence | [string] | Read-only | Full list of Instances descended from top to the current edit design instance. |
| instEdit | string | Read-only | Edit design instance. |
| instView | string | Read-only | View design instance. |
| viewport | box | Read/Write | Viewport in the view design instance. |
| comment | string | Read/Write | A comment string for the user to use for any purpose. |

deCanvas

A deCanvas represents the central widget in both the schematic editor and layout editor. The canvas depicts the specified design context. It can be created with the [de::createCanvas](#) command and destroyed with the [db::destroy](#) command, but only if it was originally created from Tcl. Event handling is defined through *Bindings*.

deCanvas is not aligned with Opal labeled inputs and always expands both horizontally and vertically to fill available space. The deContext of a deCanvas will never be set as the active context, as accessed by the [de::getActiveContext](#) command. The active context can only be set by the design windows. Until the context has been set, the canvas remains black and does not respond to events.

The canvas supports the following interfaces:

- Bindings
- Viewport Control
- Snap & Active Object
- Rendering
- Display Resources
- Overlays
- Selection
- Highlighting
- Design Editor

Table 62 *Attributes*

| Name | Type | Read/write | Description |
|---------|---------------|------------|---|
| name | <string> | read-only | Widget name identifier. |
| parent | <giContainer> | read-only | Parent container. |
| enabled | <bool> | read/write | If set to false, the widget does not respond to input. |
| shown | <bool> | read/write | If false, the widget will not be shown. |
| toolTip | <string> | read/write | ToolTip to show when mouse hovers over this widget. If empty, no tooltip will be shown. |

Table 62 *Attributes (Continued)*

| Name | Type | Read/write | Description |
|------------|-------------|------------|--|
| context | <deContext> | read/write | Design context to depict. If unset, the canvas remains blank and disabled. |
| viewport | <box> | read/write | Current viewport, in user-units of the edit design. |
| cursor | <point> | read/write | Current cursor location, in user-units of the edit design. |
| snapCursor | <point> | read/write | Current cursor location, in user-units of the edit design. |
| display | <drDisplay> | read/write | Display to use for display packets, defaults to “display”. |
| autoFit | <bool> | read/write | Automatically fit the viewport to the extents of the edit design, whenever the canvas size or design size changes. |
| width | <int> | read/write | Canvas width in pixels, or 0 to expand to fill available space. |
| height | <int> | read/write | Canvas height in pixels, or 0 to expand to fill available space. |

deClipboard

A deClipboard is a container of objects copied from a design.

To create a clipboard, use [de::copy](#) or [ile::copyToClipboard](#).

To retrieve all copied clipboards, use [de::getClips](#).

To destroy a clipboard, use [db::destroy](#).

Table 63 *Attributes*

| Name | Type | Access | Description |
|----------|--------|-----------|------------------------------------|
| name | string | Read-only | Clipboard name, typically clip<n>. |
| lib | oaLib | Read-only | Clipboard library. |
| viewType | string | Read-only | View type of the copied data. |

deCommand

Represents an interactive command or procedure registered with Custom Compiler or an instance of such a command. This object can be destroyed with [db::destroy](#), which unregisters the command.

Table 64 *Attributes*

| Name | Type | Access | Description |
|----------------|------------------|------------|--|
| name | string | Read-only | Unique name, including namespace. |
| title | string | Read/Write | English name of the command, used for its COD title. |
| label | string | Read-only | English short-name for the command, used for its COT label. |
| toolbar | <i>giToolbar</i> | Read-only | Toolbar for this interactive command. Note that the actions within this object are embedded in the actual Command Options Toolbar shown in the window. |
| dialog | <i>giDialog</i> | Read-only | Dialog box object for this interactive command. Note that this value will always be "" for command definitions and instances for which a dialog is not present |
| dialogPosition | string | Read/Write | Dialog box position relative to the parent window. If unset, the dialog is centered on the parent window. If set, the string is interpreted as a 2-element list of integers representing positive or negative offsets from the parent window's origin. This attribute is modified when the user moves the dialog box. The default value is "", meaning that the attribute is unset and the dialog will be positioned centered on top of the parent window. |
| contextMenu | <i>giMenu</i> | Read-only | Context menu registered for this command. |
| design | <i>oaDesign</i> | Read-only | Design context for a command instance. It is always NULL ("") for command definitions. |

Table 64 Attributes (Continued)

| Name | Type | Access | Description |
|-------------|--------|------------|--|
| clientData | string | Read/Write | client data for optional use by Tcl interactive commands. It is always "" for command definitions. |
| contextMenu | giMenu | Read-only | Context menu registered for this command. |

deContext

The deContext object represents an editing context which may or may not be open in a window. These objects are created by the `de::open` command. The attributes of the object are described in the table below and can be retrieved by `db::getAttr`. This object may not be destroyed using the `db::destroy` command.

Table 65 Attributes

| Name | Type | Access | Description |
|------------|-----------|-----------|--|
| topFile | dmDMFile | Read-only | Top file. |
| topDesign | oaDesign | Read-only | Design associated with the top file. May be unset (empty string) if the top file is not an oaDesign. |
| viewFile | dmDMFile | Read-only | View file |
| viewDesign | oaDesign | Read-only | Design associated with the view file. May be unset (empty string) if the view file is not an oaDesign. |
| editFile | dmDMFile | Read-only | Edit file. |
| editDesign | oaDesign | Read-only | Design associated with the edit file. May be unset (empty string) if the edit file is not an oaDesign. |
| hierarchy | heContext | Read-only | Current descend path represented in an heContext object. The returned object is a copy and may be modified or destroyed without affecting the deContext. |

Table 65 *Attributes (Continued)*

| Name | Type | Access | Description |
|--------|-----------------|-----------|--|
| window | <i>giWindow</i> | Read-only | The window displaying this context. If opened in headless mode, this attribute will be the empty string. |

deEditor

The deEditor object represents an editor. Tcl editors may be created with the `de::createApplication` command. The attributes of the object are described in the table below and can be retrieved by `db::getAttr`. This object may be destroyed using the `db::destroy` command if an instance of the editor is not in use and the editor was defined through the Tcl API.

Table 66 *Attributes*

| Name | Type | Access | Description |
|-------|--------|-----------|---|
| name | string | Read-only | Unique editor string identifier. |
| title | string | Read-only | Editor name which is displayed in graphical interfaces. |

deFigure

Represents the whole or partial figure. The table below describes the object attributes. You can retrieve the object attributes with the `db::getAttr` command.

Table 67 *Attributes*

| Name | Type | Access | Description |
|-----------|-------------------|-----------|--|
| name | string | Read-only | The name attribute of the underlying OA object, else an empty string if the object does not have a name attribute. |
| isPartial | bool | Read-only | Flag that indicates whether the figure is a full or partial object. |
| lineage | <i>oaHierPath</i> | Read-only | Object's place in the design hierarchy. |

Table 67 *Attributes (Continued)*

| Name | Type | Access | Description |
|-----------|-----------------|-----------|--|
| objType | string | Read-only | Semantic type of the underlying object. |
| object | <i>oaObject</i> | Read-only | OA object wrapped in this object. |
| isPartial | bool | Read-only | Flag showing if this a full object or partial one. |

deHighlightSet

Represents a highlight set. It may be obtained using the [de::getHighlightSets](#) command and created using the [de::createHighlightSet](#) command. This object may be destroyed with the [db::destroy](#) command, however system-defined highlight sets may not be removed.

Table 68 *Attributes*

| Name | Type | Access | Description |
|--------|-----------------|-----------|---|
| name | string | Read-only | Highlight set name. |
| design | <i>oaDesign</i> | Read-only | Design for which this highlight set is valid. |
| packet | string | Read-only | Packet name/alias used to draw this highlight set. |
| public | bool | Read-only | If public, this set is available in the interactive commands. |

deLayer

Represents a layer within a design and abstracts the visibility and selectability settings of all its LPPs. It cannot be destroyed with the [db::destroy](#) command.

Table 69 *Attributes*

| Name | Type | Access | Description |
|------|--------|-----------|--|
| name | string | Read-only | Layer name. |
| lpps | deLPPs | Read-only | Collection of this layer's layer-purpose pairs (LPPs). |

Table 69 Attributes (Continued)

| Name | Type | Access | Description |
|------------|-----------------|------------|--|
| defaultLPP | deLPP | Read-only | Returns this layer's default LPP.
The default LPP is found by finding the first value within <code>leOLPDefaultPurposeList</code> matching an existing LPP for this layer.
If none are found, the highest priority LPP for this layer is returned. |
| selectable | enLayerTristate | Read/Write | Layer selectability.
This value represents the composite selectable state of all LPPs and object filters on this layer.
Setting this value to true or false is equivalent to setting the value on all deLPPs and layer-specific object filters.
Setting this value to 'mixed' is ignored. |
| visible | enLayerTristate | Read/Write | Layer visibility.
This value represents the composite visible state of all LPPs and object filters on this layer.
Setting this value to true or false is equivalent to setting the value on all deLPPs and layer-specific object filters.
Setting this value to 'mixed' is ignored. |

deLPP

Represents an LPP setting for a design. It can be used to change the current validity, selectability, and visibility attributes as are currently reflected in the Object/Layer Panel. This object can be obtained by [de::getLPPs](#). It cannot be destroyed with [db::destroy](#).

Table 70 Attributes

| Name | Type | Access | Description |
|------|------|-----------|-------------------------|
| lpp | lpp | Read-only | Layer and purpose name. |

Table 70 *Attributes (Continued)*

| Name | Type | Access | Description |
|------------|-----------------|------------|--|
| layer | oaPhysicalLayer | Read-only | Layout object. |
| purpose | oaPhysicalLayer | Read-only | Purpose object. |
| design | oaPhysicalLayer | Read-only | Design containing this layer-purpose pair (LPP). |
| validity | bool | Read/Write | LPP validity. |
| selectable | bool | Read/Write | LPP selectability. |
| visible | bool | Read/Write | LPP visibility. |
| fill | bool | Read/Write | If false, disables any fill pattern when rendering shapes on this LPP. |
| packet | drPacket | Read-only | Retrieves the packet used to render this LPP. |

deLPPSet

A deLPPSet is an unordered set of LPP names that can be used to filter the LPP display in the Object/Layer Panel or Display Resource Editor. This object can be created with [de::createLPPSet](#), retrieved with [de::getLPPSets](#), or destroyed with [db::destroy](#).

Table 71 *Attributes*

| Name | Type | Access | Description |
|------------|--------|------------|--|
| title | string | Read-only | Title of the layer-purpose pair (LPP) set. |
| lpps | deLPPs | Read/Write | List of LPPs. |
| defaultLPP | deLPP | Read/Write | Default LPP for this LPP set, or "" if none is defined. |
| deLPPs | deLPP | Read/Write | Collection of deLPP objects, available only if this set is obtained from a design. |

deObjectFilter

Represents an object filter and stores the supported object's selectability and visibility attributes. Represents these settings for the represented object type for a layer, design, or global default. Does not represent a database object. This object cannot be destroyed with the [db::destroy](#) command.

Table 72 *Attributes*

| Name | Type | Access | Description |
|------------|---------|------------|---|
| name | string | Read-only | Object filter name. |
| title | string | Read-only | Object filter title (shown in the Object/Layer Panel). |
| viewType | string | Read-only | View type for which this object type is valid. |
| selectable | bool | Read/Write | Selectability of this object filter. |
| visible | bool | Read/Write | Visibility of this object filter. |
| layer | deLayer | Read-only | When specific to a layer, returns the associated layer name; otherwise returns an empty string. |

deOverlay

Represents an overlay, a design that is drawn on top of another design. To create an overlay, use the [de::createOverlay](#) command. To retrieve existing overlays, use the [de::getOverlays](#) command. To delete overlays, use the [db::destroy](#) command.

Table 73 *Attributes*

| Name | Type | Access | Description |
|------------|----------|-----------|---|
| name | string | Read-only | Unique name of the overlay design. |
| title | string | Read-only | Text description of the overlay design. |
| baseDesign | oaDesign | Read-only | Design over which the overlay design is placed. The value of this attribute is "" when the base design is closed. |

Table 73 *Attributes (Continued)*

| Name | Type | Access | Description |
|------------------|--------------------------|------------|--|
| baseCellView | <i>dmCellView</i> | Read-only | Cellview over which the overlay design is placed. |
| design | <i>oaDesign</i> | Read-only | Overlay design. |
| packet | drPacket drPacketAlias | Read/Write | Optional display resource packet for the overlay design. The value of this attribute is "" when no overlay exists. |
| shadow | bool | Read/Write | Turns on the Shadow mode for the overlay. |
| shown | bool | Read/Write | Displays the overlay design on top of the base design. |
| drawPlane | enOverlayDrawPlane | Read-only | Location of the overlay in the draw stack. |
| context | <i>deContext</i> | Read-only | Context in which the overlay is drawn. If the attribute value is "", the overlay design is placed over the base design in any context. |
| ignoreDrawFilter | bool | Read/Write | Draws every shape in the overlay design. |
| selectable | bool | Read-only | Shapes in the overlay are selectable. |
| selectLogic | bool | Read-only | Selecting shapes in the overlay also selects the logic in the current design context. Note that you must set selectable to true to enable selectLogic. |

deProbe

Represents a probe. It may be obtained using the [de::getProbes](#) command and created using the [de::createProbe](#) command. This object may be destroyed with the [db::destroy](#) command.

Table 74 Attributes

| Name | Type | Access | Description |
|-----------------|-----------------------|------------|--|
| probeType | <i>enProbeType</i> | Read-only | Probe type. |
| tool | string | Read-only | Tool which created this probe object. |
| objectName | string | Read-only | The name of the probed object in the target design. |
| context | <i>heContext</i> | Read-only | Hierarchical context of probed object. |
| packet | string | Read/Write | Display packet alias name. |
| description | string | Read-only | Optional string which allows for further filtering and/or grouping of probes. Should be contain a very short description of the probe (1-4 words recommended). |
| longDescription | string | Read/Write | Detailed description of the probe object. |
| purpose | <i>enProbePurpose</i> | Read/Write | Probe purpose. |

deTool

The deTool object represents a tool. Tcl tools may be created with the `de::createTool` command. The attributes of the object are described in the table below and can be retrieved by `db::getAttr`. This object may be destroyed using the `db::destroy` command. The tool will first be deactivated in any window using it.

Table 75 Attributes

| Name | Type | Access | Description |
|-------|--------|-----------|---|
| name | string | Read-only | Unique tool string identifier. |
| title | string | Read-only | Tool name which is displayed in graphical interfaces. |

deTransaction

The deTransaction object represents a Custom Compiler transaction, as defined by the Undo Manager. The attributes of the object are described in the table below and can

be retrieved or set by `db::getAttr` and `db::setAttr` commands. A transaction that is started with `de::startTransaction` should always be ended with `de::endTransaction`. However, when a `deTransaction` object goes out of scope and is reaped by the Tcl interpreter, this will also end the transaction, thus preventing never-closed transactions inside of Tcl scripts.

Table 76 *Attributes*

| Name | Type | Access | Description |
|--------|-----------------|------------|---|
| name | string | Read/Write | transaction name. |
| design | <i>design</i> | Read-only | design in which the transaction is taking place. |
| config | <i>heConfig</i> | Read-only | hierarchy editor config in which the transaction is taking place. One of design and config will always be null. |
| open | bool | Read-only | status of transaction - if open, it is still active and has not yet been ended. |
| valid | bool | Read-only | if false, flag indicates that the transaction no longer has a valid context and cannot be undone or redone. |

deViolation

This object will describe a specific violation found in a design.

Table 77 *Attributes*

| Name | Type | Access | Description |
|------------------|--------|-----------|--|
| description | string | Read-only | Long description for this particular violation |
| shortDescription | string | Read-only | Short description for this particular violation. Typically this is a short classification description. |
| toolName | string | Read-only | Name of tool which found/detected this violation. |

Table 77 Attributes (Continued)

| Name | Type | Access | Description |
|------------|------------------------------|------------|---|
| objects | <code>{oaBlockObject}</code> | Read-only | Collection of <code>oaBlockObjects</code> which are associated with this violation. This field will be reset if the design from which these objects originate is closed. |
| points | <code>[point]</code> | Read-only | Tcl list of points describing the polygon shape for the violation region/marker. These points are in user units. |
| cellview | <code>dmCellView</code> | Read-only | Design from which this violation originated. |
| severity | <code>enSeverity</code> | Read-only | Severity of violation. |
| deleteWhen | <code>enDeleteWhen</code> | Read/Write | When creating a marker from this violation, indicates when the marker object itself should be removed from the design in relation to the objects associated with the violation. |

Preferences

[Table 78 on page 965](#) lists Custom Compiler de namespace preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 78 Design Editing Preferences

| Name | Type | Scope | Default | Description |
|--|-------------------|---------|---------|---|
| <code>deActiveObjectToolbarProc</code> | string | context | | Tcl procedure taking <code>deFigure</code> as an argument and returning formatted string to be shown on the widget. If empty or the procedure returns an empty string, the default information for the object is displayed. |
| <code>deAllowUndoAfterSave</code> | undoAfter
Save | user | always | Undo post save. |
| <code>deApplyWorkspace</code> | bool | user | true | Applies last-used workspace when opening designs. |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-----------------------------|--------|----------|---------|--|
| deAutoPopUpConsole | string | user | never | Automatically displays the Console Assistant when the specified type of violations are found.
Valid values: never, onWarningsOrErrors, onErrors |
| deBeepOnError | bool | user | false | When the deBeepOnError preference is true, the error message is preceded an audible beep.
Valid values: true, false |
| deCanvasDisplay | string | cellview | display | Name of the Display Resources display object to use for canvas rendering. See dbPrintDisplay for the equivalent printing preference. |
| deCanvasMousePanMultiplier | float | user | 0.5 | Mouse pan multiplier value. |
| deCanvasMouseZoomFactor | float | user | 2 | Mouse zoom factor value. |
| deCheckSharedMemory | int | user | 1 | Enable check of shared memory between Client and XServer. |
| deClipMask | int | user | 1 | Enables ClipMask rendering in Standard Renderer. |
| deCommandAutoFocus | bool | user | false | Automatically places the cursor in the first input field when you display a toolbar or dialog box. For example, setting the preference to <code>true</code> automatically places the cursor in the Width field (the first field) when you display the Create Interconnect or Create Bus toolbar.
Valid values: true, false |
| deCommandAutoWarp | bool | user | false | Enables auto warp feature.
Valid values: true, false |
| deCommandDialogHidePosition | string | user | left | Hides button positions in command options dialogs.
Valid values: left, right |

Table 78 *Design Editing Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--|--------------|---------|---------|--|
| deCommandRestoreDialogs | bool | user | true | Remembers whether a command dialog was shown between invocations.
Valid values: true, false |
| deCompareSyncDescendStack | bool | user | true | Enable syncing the descend stack in the Design Compare.
Valid values: true, false. |
| deCompareSyncPrefs | bool | user | true | Synchronize preferences between Design Compare sessions.
Valid values: true, false. |
| deCompareSyncSelectedLogic | bool | user | true | Synchronize selected logic (Design Navigator) between Design Compare sessions.
Valid values: true, false. |
| deCompareSyncViewport | bool | user | true | Synchronize current canvas viewport between Design Compare sessions.
Valid values: true, false. |
| deConfigEditor | ConfigEditor | user | both | Defines whether DE, HE or both are opened when opening a config.
Valid values: config, design, both |
| deConnectionsNavigatorAssignmentDetails | bool | user | true | Shows or hides assignment details.
Valid values: true, false |
| deConnectionsNavigatorAssignmentDetails Mode | string | user | Blocked | Mode of assignment details table.
Valid values: true, false |
| deConnectionsNavigatorHideFullyAssigned | bool | context | false | Hides or shows full assignment parameters in the Navigator dialog box.
Valid values: true, false |
| deConnectionsNavigatorHighlight | bool | context | false | Highlights instances in parent window.
Valid values: true, false |
| deConnectionsNavigatorParameters | bool | user | true | Shows or hides parameters.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|--------|----------|------------------|---|
| deConnectionsNavigatorParametersAdvanced | bool | user | false | Shows detailed view of parameters.
Valid values: true, false |
| deConnectionsNavigatorShowBlocking | bool | context | true | Shows blocking assignments region.
Valid values: true, false |
| deConnectionsNavigatorShowParameters | bool | context | true | Shows parameters region.
Valid values: true, false |
| deConnectionsNavigatorViewSearchList | string | context | schematic symbol | Displays search list for connections navigator. |
| deConsoleAssistantTab | string | user | Console | Specifies the Console Assistant tab to display.
Valid values: Console, Errors, History |
| deCoordMarkZoom | bool | cellview | false | Determines if viewport will be zoomed to coordinate entered (zoom level determined by preference deZoomLevel).
Valid values: true, false |
| deCreateProbeDialogMode | bool | user | false | Displays the Create Probe dialog box.
Valid values: true, false |
| deCrossCheckCaseInsensitive | bool | user | false | Specifies the case sensitivity of terminal name comparisons.
Valid values: true, false |
| deCrossCheckIgnoreEmptyPortOrder | bool | user | true | Controls whether port order check is ignored if main checked view has no portOrder property.
Valid values: true, false |
| deCrossCheckPortOrder | bool | user | true | Controls whether port orders should be checked.
Valid values: true, false. |
| deCrossCheckSeverity | string | user | error | Default severity for cross check violations.
Valid values: true, false |
| deCrossCheckTermType | bool | user | bool | Controls whether terminal types should be checked.
Valid values: true, false. |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-------------------------------|--------|---------|---------------------------------------|--|
| deCrossCheckViews | string | user | schematic
symbol hspice
verilog | List of views used by the <code>de::crossCheckViews</code> command to filter views. If empty, all views in base cell will be used. |
| deCrossFit | bool | user | false | Fits to cross selected objects
Valid values: true, false |
| deCrossHighlightPacket | string | user | highlight16 | Highlight packet for cross highlights. |
| deCrossProbe | bool | context | true | Enables cross probing in the Add Probe command when at least one cross probe tool is available.
Valid values: true, false |
| deCrossProbeTool | string | context | | Tool used for cross probing. |
| deCursorActivationDelay | int | user | 0 | Cursor activation delay in milliseconds. |
| deCursorTextHeight | int | user | 10 | Snap cursor size in pixels. |
| deDefaultClipboardLibraryPath | string | user | \$SYNOPSIS_CUSTOM_LOCAL | Library location for Recent clipboard. If the preference value is changed, the new path is used on the next startup. |
| deDefaultInstanceNaming | string | user | UserPrefix | Dictates the prefix to use when auto naming of an instance is performed. Possible values are MasterCell and UserPrefix.
Valid values: UserPrefix, MasterCell |
| deDefaultInstancePrefix | string | user | I | If the <code>deDefaultInstanceNaming</code> preference is set to UserPrefix, the value of this preference will be used as the prefix for auto naming of instances. |
| deDeleteUnusedBackups | bool | user | false | Deletes other backup and recovery cellviews.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-------------------------------|----------|-------|---------|--|
| deDescendActivateAnyLevelInst | bool | user | true | Additional filter to apply in LE-only if instances between start and stop level can be activated when descend command is prompting to specify instance to descend. Valid values: true, false |
| deDescendCaseSensitive | bool | user | true | Defines the value for the Case Sensitive Filter option in the Descend Lineage dialog box. Valid values: true, false |
| deDescendDialog | bool | user | false | Defines the default value for Prompt for View option. Valid values: true, false |
| deDescendFilter | string | user | | Specifies the value for the Filter box in the Descend Lineage dialog box. |
| deDescendInPlace | bool | user | false | Enables Edit In Place mode. This preference is ignored if <code>deDescendNewWindow</code> is true, because it takes precedence. Valid values: true, false |
| deDescendLayoutNewWindow | bool | user | false | Descends in new window with <code>ide::descend</code> in Layout View designs. |
| deDescendLineage | bool | user | false | Defines the default value for Prompt for Lineage option. Valid values: true, false |
| deDescendMode | openMode | user | auto | When the value is set to edit or readOnly, the Descend Instance enables Edit or Read-Only modes. If the value is set to auto, the descend mode is determined automatically. Valid values: auto, edit, readOnly |
| deDescendNewWindow | bool | user | false | Enables descending in new window with <code>ide::descend</code> . Mutually exclusive with <code>deDescendInPlace</code> . Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|----------------------------|------------|----------|----------|--|
| deDescendObjectType | objectType | user | instance | Specifies if the target for descend is an instance, figure group, or both when <code>ide::descend</code> is triggered in the Layout Editor.
Valid values: instance, group |
| deDescendPcellsAllowed | bool | user | true | Permits descending into PCells.
Valid values: true, false |
| deDescendRead | bool | cellview | false | If true, check buttons in Open Design and Descend Instance will enable Read-Only mode.
Valid values: true, false |
| deDescendRefresh | bool | user | false | Turns on or off the Refresh option in the Descend Lineage dialog box.
Valid values: true, false |
| deDevicePaletteShowLibrary | bool | user | false | Shows the library name for each device.
Valid values: true, false |
| deDevicePaletteShowParams | bool | user | true | Shows parameter overrides in the assistant.
Valid values: true, false |
| deDevicePaletteShowView | bool | user | false | Shows the view name for each device.
Valid values: true, false |
| deDirectApply | bool | user | false | If true, interactive commands that support Direct Apply operates in Direct Apply mode.
Valid values: true, false |
| deDisableBlinking | bool | user | false | Disable blinking.
Valid values: true, false |
| deDoubleClickTimeout | int | user | 300 | Time within which multiple clicks count as double or triple button presses.
Valid values: Any integer in a [1 2000] range. |
| deDragThreshold | int | user | 100 | Distance the cursor must move with a button depressed to consider the event a drag rather than a button press. |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------------|------|----------|---------|---|
| deDragTimer | int | user | 200 | Time, in milliseconds, that the cursor must move at least <code>deDragThreshold</code> pixels, to be considered a drag. |
| deDrawAllTextsDrafting | bool | user | false | Indicates whether all texts should be drawn with drafting option on.
Valid values: true, false |
| deDrawEllipseSides | int | user | 0 | Number of vertices to delineate the ellipse. |
| deDrawDBRulersOnTopwith | bool | user | false | Indicates whether rulers and coordinate marks in database should be drawn on top of interactive objects.
Valid values: true, false |
| deDrawRulersOnTop | bool | user | false | Indicates whether rulers and dimension marks in database should be drawn on top of interactive objects.
Valid values: true, false |
| deEnable1PixelBlockBounds | int | user | 1 | Forces standard renderer to use min 1 pixel for blocks and boundaries. |
| deEnableCrossSelect | bool | user | false | Enables cross selection.
Valid values: true, false |
| deEnableExceedOnDemandOptimization | bool | user | false | Enables drawing optimizations for Exceed on Demand.
Valid values: true, false |
| deEnableRuntimePixmaps | bool | user | true | Controls if runtime pixmap caching is ignored.
Valid values: true, false |
| deErrorViewerAutoClearHighlight | bool | cellview | true | Auto clears the highlight of the last error marker and its associated object on the canvas.
Valid values: true, false |
| deErrorViewerAutoSelectMarkerTable | bool | cellview | false | Auto Select First Item of Marker Table.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------------------|-----------------|----------|---------|---|
| deErrorViewerClearHighlightOnClose | bool | cellview | false | Clears all highlights when closing the error viewer.
Valid values: true, false |
| deErrorViewerConfigCategory | string | cellview | 0+1+3+4 | Configure categories shown in the error viewer with: <ul style="list-style-type: none"> • 0: Design • 1: Tool • 2: Severity • 3: ShortMessage • 4: Message |
| deErrorViewerCreate2EditDesign | bool | cellview | false | Creates markers for errors found by the Calibre tool.
Valid values: true, false |
| deErrorViewerDefaultDocked | DockMode | user | none | The default dock status of Error Viewer. |
| deErrorViewerDefaultLayoutStyle | LayoutStyleMode | user | none | The default layout style of Error Viewer. |
| deErrorViewerDefaultPin | PinMode | user | none | The default pin status of Error Viewer. |
| deErrorViewerDeleteWaiveTogether | bool | user | true | Delete waive marker together when close source tab. |
| deErrorViewerFilterType | FilterType | cellview | single | Filter Type |
| deErrorViewerGenericTabName | bool | cellview | false | Generic tab name |
| deErrorViewerGenericTabNameValue | string | cellview | | Generic tab name value |
| deErrorViewerExportGenericFile | string | cellview | | Export Generic Load File Name Text. |
| deErrorViewerGenericCreate2EditDesign | bool | cellview | false | Create Open / Short Error Marker to Current Edit Design.
Valid values: true, false. |
| deErrorViewerGenericEMThreshold | bool | cellview | false | Generic electromigration Threshold.
Valid values: true, false. |
| deErrorViewerGenericEMThresholdValue | float | cellview | 100 | Generic EM Threshold Value |
| deErrorViewerGenericLoadFile | bool | cellview | false | Generic Load File Name |
| deErrorViewerGenericLoadFileText | string | cellview | | Generic Load File Name Text |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-------------------------------------|-----------|----------|-------------|---|
| deErrorViewerGenericToolName | bool | cellview | false | Generic Tool Name |
| deErrorViewerGenericToolNameValue | string | cellview | | Generic Tool Name Value |
| deErrorViewerGPDPATHText | string | cellview | | Specifies the path to the GPD database. |
| deErrorViewerHideColorColumn | bool | cellview | true | Hides the Color column.
Valid values: true, false |
| deErrorViewerHideResultColumn | bool | cellview | false | Hides the Result column.
Valid values: true, false |
| deErrorViewerHierarchy | bool | cellview | false | Shows markers hierarchically from the top design's hierarchy to the hierarchy that are visible down to the view depth.
Valid values: true, false |
| deErrorViewerHighlightAfterSel | bool | cellview | false | Highlights error markers and their associated objects on the canvas when selecting them in the marker pane.
Valid values: true, false |
| deErrorViewerHighlightColorMode | ColorMode | cellview | cyclic | Specifies the Highlight Color Mode.
Valid values: fixed, cyclic, mapping |
| deErrorViewerHighlightRelatedLayers | bool | cellview | false | Highlights related layers.
Valid values: true, false |
| deErrorViewerHighlightSet | string | cellview | highlight1 | Specifies the highlight color.
Valid values: highlight n , where n is an integer between 1 and 30, inclusive. |
| deErrorViewerHighlightShadowMode | bool | cellview | true | Turns on Shadow Mode.
Valid values: true, false |
| deErrorViewerHighlightXSymbol | bool | cellview | false | Highlights X Symbol.
Valid values: true, false |
| deErrorViewerHighlightXSymbolSize | int | cellview | 1 | The Size scale of X Symbol.
Valid values: true, false |
| deErrorViewerLoadFromLCVValue | lcv | cellview | <file path> | Load from View. |
| deErrorViewerLoadResultPluginFile | string | user | | Specifies the plugin file path in the load result dialog box. |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------------------|--------|----------|---------|--|
| deErrorViewerLoadResultSpeedMode | bool | cellview | false | Applies Speed Mode when loading results files.
Valid values: true, false |
| deErrorViewerLoadResultToolName | bool | cellview | false | Tool name.
Valid values: true, false |
| deErrorViewerLoadResultToolNameValue | string | cellview | calibre | Tool name value. |
| deErrorViewerMapHighlightAssoObj | bool | cellview | true | Sets the highlight action to highlight the associated objects of markers.
Valid values: true, false |
| deErrorViewerMapHighlightMarker | bool | cellview | true | Sets the highlight action to highlight markers.
Valid values: true, false |
| deErrorViewerMapHighlightParent | bool | cellview | true | Sets the highlight action to highlight parent markers or fixing guides.
Valid values: true, false |
| deErrorViewerMaxMarkers | bool | cellview | false | Turns on Max Markers to Display.
Valid values: true, false |
| deErrorViewerMaxMarkersValue | int | cellview | 10000 | Specifies the value of Max Markers to Display. |
| deErrorViewerMaxNumDatabase | int | user | 10 | Specify the max number of deErrorViewerDatabase object for one design |
| deErrorViewerPin | bool | cellview | false | Turns on Pin Dock Widget.
Valid values: true, false |
| deErrorViewerRedHawkEMFile | bool | cellview | false | RedHawk EM File Name.
Valid values: true, false |
| deErrorViewerRedHawkEMFileText | string | cellview | | RedHawk EM Name Text. |
| deErrorViewerRedHawkEMThreshold | bool | cellview | false | RedHawk EM Threshold.
Valid values: true, false |
| deErrorViewerRedHawkEMThresholdValue | float | cellview | 100 | RedHawk EM Threshold Value. |
| deErrorViewerRedHawkOpenViaFile | bool | cellview | false | RedHawk Open Via File Name.
Valid values: true, false |
| deErrorViewerRedHawkOpenViaFileText | string | cellview | | RedHawk Via Name Text. |

Table 78 *Design Editing Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--------------------------------------|--------|----------|-------------|---|
| deErrorViewerRedHawkOpenWireFile | bool | cellview | false | RedHawk Open Wire File Name.
Valid values: <code>true</code> , <code>false</code> |
| deErrorViewerRedHawkOpenWireFileText | string | cellview | | RedHawk Wire Name Text. |
| deErrorViewerRedHawkShortFile | bool | cellview | false | RedHawk Short File Name.
Valid values: <code>true</code> , <code>false</code> |
| deErrorViewerRedHawkShortFileText | string | cellview | | RedHawk Short Name Text |
| deErrorViewerRedHawkShrink | bool | cellview | false | RedHawk Shrink.
Valid values: <code>true</code> , <code>false</code> |
| deErrorViewerRedHawkShrinkValue | float | cellview | 1 | RedHawk Shrink Value. |
| deErrorViewerRedHawkToolName | bool | cellview | false | RedHawk Tool Name.
Valid values: <code>true</code> , <code>false</code> |
| deErrorViewerRedHawkToolNameValue | string | cellview | redhawk | RedHawk Tool Name Value. |
| deErrorViewerReloadEIP | bool | cellview | true | Refresh Error Viewer Data in EIP.
Valid values: <code>true</code> , <code>false</code> |
| deErrorViewerResultFileText | string | cellview | | Result file. |
| deErrorViewerRuleFile | bool | cellview | false | Rule file.
Valid values: <code>true</code> , <code>false</code> |
| deErrorViewerRuleFileText | string | cellview | | Rule file text. |
| deErrorViewerSaveToLCVValue | lcv | cellview | <file path> | Save to View. |
| deErrorViewerShowAssociatedObj | bool | cellview | true | Show associated objects information. |
| deErrorViewerShowMsg | bool | cellview | true | Show message Information. |
| deErrorViewerShowParentMarker | bool | cellview | true | Show parent markers information. |
| deErrorViewerShowSeverity | bool | cellview | false | Show severity information. |
| deErrorViewerShowShortMsg | bool | cellview | false | Show short message information. |
| deErrorViewerShowTip | bool | cellview | false | Shows a tooltip when hovering over an error marker on the canvas.
Valid values: <code>true</code> , <code>false</code> |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------------------|----------|----------|---------|---|
| deErrorViewerShowTool | bool | cellview | false | Show tool Information. |
| deErrorViewerStarRCCreate2EditDesign | bool | cellview | false | Creates markers for errors found by StarRC GPD.
Valid values: true, false |
| deErrorViewerWaiveFile | bool | cellview | false | Waive file.
Valid values: true, false |
| deErrorViewerWaiveFileText | string | cellview | | Waive file text. |
| deErrorViewerZoom | bool | cellview | true | Zooms to each error marker and fits it to the viewport when highlighted.
Valid values: true, false |
| deErrorViewerZoomMode | ZoomMode | cellview | zoom | Select Zoom/Pan Mode. |
| deErrorViewerZoomValue | float | cellview | 7 | The Zoom Factor Value. |
| deEVStarRCErrorFilterNet | bool | cellview | false | Displays open short errors for specified net(s).
Valid values: true, false |
| deEVStarRCErrorFilterRegion | bool | cellview | false | Displays open short errors for a specified region.
Valid values: true, false |
| deEVStarRCErrorFilterRegionValue | string | cellview | | The region that contains open short errors. The open error shorts within the region is highlighted. |
| deEVStarRCErrorNetNameList | string | cellview | | List of net names. |
| deEVStarRCOALayerMap | bool | cellview | false | Specifies the StarRC and OA layer map file.
Valid values: true, false |
| deEVStarRCOALayerMapFile | string | cellview | | Specifies the StarRC and OA layer map file for mapping LVS (GPD) technology layers to OA layers. |
| deEVStarRCOpenShortErrorNum | int | cellview | 10000 | Maximum number of errors to be displayed. |
| deEVStarRCOpenShortHeatMap | bool | cellview | false | Displays color-coded visualization of different error types.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|----------------------------------|--------|----------|---------|--|
| deEVStarRCOpenShortMaxLimit | bool | cellview | false | Displays the maximum number of errors.
Valid values: true, false |
| deEVStarRCOpenShortType | string | cellview | All | Specifies the type of short errors to display: Open, Short, or All errors.
Valid values: Open, Short, All |
| deEVStarRCShortErrorBlockage | bool | cellview | true | Displays short errors for blockages.
Valid values: true, false |
| deEVStarRCShortErrorFill | bool | cellview | true | Displays short errors for fills.
Valid values: true, false |
| deEVStarRCShortErrorSkipCell | bool | cellview | true | Displays short errors for skip cells.
Valid values: true, false |
| deEVStarRCShortErrorType | bool | cellview | false | Filters short errors by type.
Valid values: true, false |
| deEVStarRCShortErrorUnselectable | bool | cellview | true | Displays short errors for unselectable nets.
Valid values: true, false |
| deEVStarRCShortErrorUnselected | bool | cellview | true | Displays short errors for unselected nets.
Valid values: true, false |
| deEVStarRCShortNet | bool | cellview | true | Displays short errors for nets.
Valid values: true, false |
| deExistingHighlightSet | string | cellview | | Existing highlight set name selected in any interactive highlight command. |
| deExitPromptSaveBindings | bool | user | false | Displays a prompt for saving key bindings when exiting the software.
Valid values: true, false |
| deExitPromptSaveBookmarks | bool | user | false | Displays a prompt for saving Bookmarks when exiting the software.
Valid values: true, false |
| deExitPromptSaveDisplayResources | bool | user | true | Displays a prompt for saving the display resources when exiting the software.
Valid values: true, false |

Table 78 *Design Editing Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-------------------------------------|------|-------|---------|--|
| deExitPromptSaveNetColorDefs | bool | user | true | If set to true, then the Schematic Net Colorization Definitions will be prompted on exit.
Valid values: true, false |
| deExitPromptSaveOLP | bool | user | true | Saves the Object/Layer Panel settings when exiting the software.
Valid values: true, false |
| deExitPromptSaveOLPPresets | bool | user | true | Displays a prompt for saving the Object/Layer Panel preset when exiting the software.
Valid values: true, false |
| deExitPromptSaveOpenedClipboardLibs | bool | user | false | If value is true, then the Opened Clipboard Libraries will be prompted on exit.
Valid values: true, false |
| deExitPromptSavePrefs | bool | user | true | If value is true, save Preferences is prompted on exit.
Valid values: true, false |
| deExitPromptSaveRecentFiles | bool | user | false | If value is true, save Recent Files is prompted on exit.
Valid values: true, false |
| deExitPromptSaveSaveRecentClips | bool | user | true | If value is true, then the Recent Clipboard Library will be prompted on exit.
Valid values: true, false |
| deExitPromptSaveTRAPresets | bool | user | true | Displays prompt to save Track Region Assistant presets when exiting the assistant.
Valid values: true, false |
| deExitSaveTRAPresets | bool | user | true | Saves the Track Region Assistant preset automatically when exiting the assistant.
Valid values: true, false |
| deExitPromptSaveWindowTypes | bool | user | false | If value is true, then the Window Types will be prompted on exit.
Valid values: true, false |

Table 78 *Design Editing Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-------------------------------|------|-------|---------|---|
| deExitPromptSaveWorkspaces | bool | user | false | If value is true, save Workspaces is prompted on exit.
Valid values: true, false |
| deExitSaveBindings | bool | user | true | If true, save bindings on exit.
Valid values: true, false |
| deExitSaveBookmarks | bool | user | true | If value is true, bookmarks are saved on exit.
Valid values: true, false |
| deExitSaveDisplayResources | bool | user | true | If value is true, display resources are saved on exit.
Valid values: true, false |
| deExitSaveNetColorDefs | bool | user | true | If set is true, then the Schematic Net Colorization Definitions will be saved on exit.
Valid values: true, false |
| deExitSaveOLP | bool | user | true | If value is true, the Object/Layer Panel is saved on exit.
Valid values: true, false |
| deExitSaveOLPPresets | bool | user | true | Saves the Object/Layer Panel presets when existing the software.
Valid values: true, false |
| deExitSaveOpenedClipboardLibs | bool | user | true | If value is true, then the Opened Clipboard Libraries will be saved on exit.
Valid values: true, false |
| deExitSavePrefs | bool | user | true | If value is true, preferences are saved on exit.
Valid values: true, false |
| deExitSaveRecentFiles | bool | user | true | If value is true, Recent Files are saved on exit.
Valid values: true, false |
| deExitSaveSaveRecentClips | bool | user | true | If value is true, then the Recent Clipboard Library will be saved on exit.
Valid values: true, false |

Table 78 *Design Editing Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-----------------------|--------|-------|---------|--|
| deExitSaveState | bool | user | true | If true, selected sets of state data are saved on exit.
Valid values: true, false |
| deExitSaveWindowTypes | bool | user | true | If value is true, then the Window Types will be saved on exit.
Valid values: true, false |
| deExitSaveWorkspaces | bool | user | true | If value is true, Workspaces are saved on exit.
Valid values: true, false |
| defaultVDRCDB | string | user | | Output the default VDRC-DB name. |
| deFontEuroStyle | string | user | | Font to use for stick text, canvas labels, and instance names. |
| deFontFixed | string | user | | Font to use for stick text, canvas labels, and instance names. |
| deFontGothic | string | user | | Font to use for stick text, canvas labels, and instance names. If set to "", the default is used. |
| deFontMath | string | user | | Font to use for stick text, canvas labels, and instance names. If set to "", the default is used. |
| deFontMilSpec | string | user | | Font to use for stick text, canvas labels, and instance names. If set to "", the default is used. |
| deFontRoman | string | user | | Font to use for stick text, canvas labels, and instance names. If set to "", the default is used. If set to 'polygon', an internal polygon font is used. |
| deFontScript | string | user | | Font to use for stick text, canvas labels, and instance names. If set to "", the default is used. If set to 'polygon', an internal polygon font is used. |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-------------------------|--------|----------|-------------|--|
| deFontStick | string | user | | Font to use for stick text, canvas labels, and instance names. If set to "", the default is used. If set to 'polygon', an internal polygon font is used. |
| deFontSwedish | string | user | | Font to use for stick text, canvas labels, and instance names. If set to "", the default is used. |
| deGlyphCacheMaxSize | int | user | 2097152 | Max byte size of glyph cache default is 2 Mb(max limit is 8Mb). |
| deHelpViewer | string | user | | Browser for viewing help topics using Webworks. |
| deHighlightActiveFigure | bool | user | true | Highlight the active figure in the other session in Design Compare |
| deHighlightCreateNewSet | bool | user | true | Creates new set (true), or adds to existing highlight connected set (false).
Valid values: true, false |
| deHighlightCycle | bool | User | false | Cycles the highlighting colors.
Valid values: true, false |
| deHighlightMode | string | cellview | name | Highlight mode in the Highlight command.
Valid values: shapes, name |
| deHighlightName | string | cellview | | Name field in the highlight by name command. |
| deHighlightPersistent | bool | user | false | Turns on persistence for selected highlight sets. Valid values: true, false |
| deHighlightSet | string | user | highlight16 | Name of selected highlight set. |
| deHighlightShadow | bool | user | false | Shadow option for interactive highlight commands.
Valid values: true, false |
| deHighlightShapesCycle | bool | user | false | Cycle option for interactive shape highlighting command.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------------|------|----------|---------|--|
| deHighlightTraceParentDesigns | bool | user | false | Displays the trace parent designs option for interactive highlight commands.
Valid values: true, false |
| deInFixMode | bool | user | false | If true, interactive commands that support InFix will operate in InFix mode.
Valid values: true, false |
| deKeepLayoutHighlights | bool | user | true | Keeps cross highlights on Layout at Deselect All or on mouse click |
| deLogMotion | bool | user | false | If true, mouse motion is written to the log.
Valid values: true, false |
| deMakeAsDefaultApplication | bool | user | false | Make it as a default application.
Valid values: true, false |
| deMappedWindows | int | user | 0 | Number of windows to store in video memory. Windows beyond this limit will be stored on the client until activated. The default value should be sufficient for nearly all users. A larger value makes sense when using large windows with remote-X over a slow network. A small value makes sense when video memory is limited and multiple CD sessions are used each with many windows. |
| deMarkerBrowserAcknowledgedWarning | bool | cellview | true | See Marker Browser Assistant.
Valid values: true, false |
| deMarkerBrowserAnnotation | bool | cellview | true | Indicates whether user generated informations should be shown in marker browser. |
| deMarkerBrowserCriticalError | bool | cellview | true | See Marker Browser Assistant.
Valid values: true, false |
| deMarkerBrowserError | bool | cellview | true | See Marker Browser Assistant.
Valid values: true, false |
| deMarkerBrowserFatalError | bool | cellview | true | See Marker Browser Assistant.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|--------|----------|---------|--|
| deMarkerBrowserHierarchy | bool | cellview | false | See Marker Browser Assistant. Valid values: true, false |
| deMarkerBrowserHighlightAssociatedObjects | bool | cellview | false | See Marker Browser Assistant. Valid values: true, false |
| deMarkerBrowserInfo | bool | cellview | true | See Marker Browser Assistant. Valid values: true, false |
| deMarkerBrowserSchematicTools | string | cellview | all | See Marker Browser Assistant. Valid values: true, false |
| deMarkerBrowserSignedOffCriticalError | bool | cellview | true | See Marker Browser Assistant. Valid values: true, false |
| deMarkerBrowserSignedOffError | bool | cellview | true | See Marker Browser Assistant. Valid values: true, false |
| deMarkerBrowserSymbolTools | string | cellview | all | See Marker Browser Assistant. Valid values: true, false |
| deMarkerBrowserWarning | bool | cellview | true | See Marker Browser Assistant. Valid values: true, false |
| deMarkerBrowserZoom | bool | cellview | true | See Marker Browser Assistant. Valid values: true, false |
| deMarkerHighlightXSize | int | user | 30 | Size of X marker symbols that are independent of zoom operations. |
| deMarkerHighlightXSymbol | bool | user | false | Highlights X marker symbols. Valid values: true, false |
| deMarkerHighlightXWidth | int | user | 2 | Width of X marker symbols that are independent of zoom operations. |
| deMaxBackupCount | int | user | 0 | Maximum number of automatic backups. |
| deMaxTexts | int | user | 1000 | Maximum number of text objects drawn within a viewport. |
| deMaxUnbondInstances | int | user | 100 | Maximum number of unbound instance to draw. Set 0 to draw all unbound instances. |
| deMBAssociatedObjects | bool | cellview | false | Highlight associated objects of markers when highlighting the marker. |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|--------|----------|---------|--|
| deMBHierarchy | bool | cellview | false | Show markers hierarchically from the edit design down to view depth. |
| deMBLayoutTools | string | cellview | all | Marker tool names for maskLayout views to manage in the Marker Browser. |
| deMBNetlistTools | string | cellview | all | |
| deMBSchematicTools | string | cellview | all | Marker tool names for schematic views to manage in the Marker Browser. |
| deMBSShowAcknowledgedWarningMarkers | bool | cellview | true | Shows markers of severity oacAcknowledgedWarning-MarkerSeverity. |
| deMBSShowAnnotationMarkers | bool | cellview | true | Shows markers of severity oacAnnotation-MarkerSeverity. |
| deMBSShowCriticalErrorMarkers | bool | cellview | true | Shows markers of severity oacCriticalError-MarkerSeverity. |
| deMBSShowErrorMarkers | bool | cellview | true | Shows markers of severity oacErrorMarkerSeverity. |
| deMBSShowFatalErrorMarkers | bool | cellview | true | Shows markers of severity oacFatalError-MarkerSeverity. |
| deMBSShowInfoMarkers | bool | cellview | true | Shows markers of severity oacInfoMarkerSeverity. |
| deMBSShowSignedOffCriticalErrorMarkers | bool | cellview | true | Shows markers of severity oacSignedOffCritical-ErrorMarkerSeverity. |
| deMBSShowSignedOffErrorMarkers | bool | cellview | true | Shows markers of severity oacSignedOffError-MarkerSeverity. |
| deMBSShowWarningMarkers | bool | cellview | true | Shows markers of severity oacWarning-MarkerSeverity. |
| deMBSymbolTools | string | cellview | all | Marker tool names for schematicSymbol views to manage in the Marker Browser. |
| deMBZoomToMarkers | bool | cellview | true | Zooms to markers selected in the Marker Browser |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------------|--------------|----------|---------|---|
| deOpenHelpWithSystemBrowser | bool | user | false | Displays the Custom Compiler Help using the system browser instead of within the product.
Valid values: true, false |
| deOpenRead | bool | user | false | Represents the "Open Read-Only" button in the Open Design dialog.
Valid values: true, false |
| deOverlayEnableSelection | bool | user | false | Enable overlay selection.
Valid values: true, false |
| dePanCoordinateMark | bool | cellview | false | Creates coordinate mark at the panning point.
Valid values: true, false |
| dePanZoom | bool | cellview | false | Zooms in during panning. The zoom level is determined by the <code>deZoomLevel</code> Tcl preference.
Valid values: true, false |
| dePeriodicSaveInterval | int | user | 0 | Periodicity of automatic backups in minutes. |
| dePolygonConversionThreshold | unsigned int | user | 2000 | The maximum size of the polygon for which the real orientation is calculated. |
| dePreSelectRepeatCommand | bool | user | false | Lets you repeat the last command in the pre-select mode. To exit the command, press Esc or use <code>de::abortCommand</code> .
Valid values: true, false |
| deProbeAssistantCSMDescription | bool | context | true | Shows Description column in the Probe Assistant.
Valid values: true, false |
| deProbeAssistantCSMLineage | bool | context | false | Shows Lineage column in the Probe Assistant.
Valid values: true, false |
| deProbeAssistantCSMObject | bool | context | true | Shows Object column in the Probe Assistant.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|--------|---------|---------|---|
| deProbeAssistantCSMPacket | bool | context | true | Shows Packet column in the Probe Assistant.
Valid values: true, false |
| deProbeAssistantFilterBySelection | bool | context | false | Show probes filtered by selection in the Probe Assistant.
Valid values: true, false |
| deProbeAssistantHierarchy | bool | context | false | Show probes hierarchically from the edit design down to view depth.
Valid values: true, false |
| deProbeAssistantShowCellViewType | bool | context | true | Shows cellview-type probes in the Probe Assistant.
Valid values: true, false |
| deProbeAssistantShowInstanceTerminalType | bool | context | true | Shows instance terminal probes in the Probe Assistant.
Valid values: true, false |
| deProbeAssistantShowInstanceType | bool | context | true | Shows instance-type probes in the Probe Assistant.
Valid values: true, false |
| deProbeAssistantShowNetType | bool | context | true | Shows net-type probes in the Probe Assistant.
Valid values: true, false |
| deProbeAssistantShowPathType | bool | context | true | Shows path-type probes in the Probe Assistant.
Valid values: true, false |
| deProbeAssistantShowTerminalType | bool | context | true | Shows terminal-type probes in the Probe Assistant.
Valid values: true, false |
| deProbeAssistantTools | string | context | all | Shows all or no probes in the Probe Assistant.
Valid values: all, none |
| deProbeAssistantZoomToProbe | bool | context | true | Zooms to probe selected in the Probe Assistant.
Valid values: true, false |
| deProbeAutoRefresh | bool | user | true | Automatically refreshes the probe objects when the design is modified.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|--------|---------|---------|--|
| deProbeCycle | bool | context | true | Probes graphical packet cycling mode for the Add Probe command.
Valid values: true, false |
| deProbeDescription | string | context | | Short description of the probe for the Add Probe command. |
| deProbeIgnoreDetailsInContextsComparison | bool | user | false | Considers only the top library/cell/view and occurrence for probe comparison.
Valid values: true, false |
| deProbeLongDescription | string | context | | Detailed description of the probe for the Add Probe command. |
| deProbeName | string | context | | Probes path field for the Add Probe command. |
| deProbeNetHighlightFullTrace | bool | user | false | Runs full hierarchy net tracing, which might affect the performance of the probe redrawing.
Valid values: true, false |
| deHighlightShadow | bool | context | false | Sets the Shadow mode for probing net operations: <ul style="list-style-type: none"> <code>true</code>: Turns on Shadow mode to increase the contrast between the nets you are probing and the remaining objects. <code>false</code>: Turns off Shadow mode. Valid values: true, false |
| deProbeNetShowFlightline | bool | user | false | Shows or hides flight lines for nets.
Valid values: true, false |
| deProbeNetShowPinFlightline | bool | user | false | Shows or hides flight lines from pin to pin.
Valid values: true, false |
| deProbeOutOfContext | bool | user | false | Propagates net probes when descending out of context.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------|--------|---------|------------|--|
| deProbePacket | string | context | highlight1 | Probes graphical packet for the Add Probe command.
Valid values: highlight n , where n is an integer between 1 and 30, inclusive |
| deProbePurpose | string | context | other | Probes purpose for the Add Probe command.
Valid values: error, warning, info, cross, other |
| deProbeTool | string | context | | Probe tool name for the Add Probe command. |
| deProbeTraceCell | bool | user | false | Traces through cells for the Add Probe command.
Valid values: true, false |
| deProbeTraceCellConfig | string | user | | Add Probe command cell settings to use during trace through operations. |
| deProbeTraceProp | string | user | lvslgnore | Properties used to determine device trace-throughs. |
| deProbeTraceThrough | bool | user | true | Traces through cells and ignores devices during Add Probe operations.
Valid values: true, false |
| deProbeTraceThroughParasitic | bool | user | true | Traces through estimated parasitic devices during Add Probe operations.
Valid values: true, false |
| deProbeType | string | context | Any | Probe type for the Add Probe command.
Valid values: Any Terminal Instance Instance Terminal Net |
| dePromptArrayInstIndex | bool | user | true | Displays a prompt for bit index when descending into array instance.
Valid values: true, false |
| dePromptSaveHier | bool | user | false | If true, the Close Designs dialog is prompted when saving open hierarchical designs.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|----------------------------------|--------|---------|---------|---|
| dePropertyEditorApplyToInstances | string | context | current | Controls the scope of instances to which a change applies. Matches the double-arrow pull down menu options in the Property Editor.
Valid values: all, current, sameMaster, sameMasterInSelected Set, selectedSet |
| dePropertyEditorApplyToMPPs | string | context | current | Current choice for 'Apply Changes to' menu item for MPPs in single item display.
Valid values: all, current, sameDef, sameDefInSelectedSet, selectedSet |
| dePropertyEditorApplyToObjects | string | context | current | Current choice for Apply Changes to' menu item for all other object types in single item display.
Valid values: current, sameLPP, sameType, sameTypeInSelectedSe, selectedSet |
| dePropertyEditorApplyToVias | string | context | current | Current choice for Apply Changes to menu item for vias in single item display.
Valid values: all, current, sameDef, sameDefInSelectedSet, selectedSet |
| dePropertyEditorApplyToWireNames | string | context | current | Current choice for Apply Changes to menu item for wire names in single item display.
Valid values: all, current, sameNet, sameNetInSelectedSet, selectedSet |
| dePropertyEditorApplyToWires | string | context | current | Current choice for Apply Changes to menu item for wires in single item display.
Valid values: all, current, sameNet, sameNetConnected, sameNetInSelectedSet, selectedSet |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---|-------------------|----------|---------|---|
| dePropertyEditorConnectionAssignmentsVisibility | <enVisiblityMode> | context | both | Default visibility for new connection assignment definitions.
Valid values: off, name, value, both |
| dePropertyEditorSetVisibility | | | | Sets the visibility of the selected object. |
| dePropertyEditorSetVisibilityBoth | | | | Sets all row's Visibility to Name=Value. |
| dePropertyEditorSetVisibilityName | | | | Sets all row's Visibility to Name. |
| dePropertyEditorSetVisibilityOff | | | | Sets all row's Visibility to Off. |
| dePropertyEditorSetVisibilityValue | | | | Sets all row's Visibility to Value. |
| dePropertyEditorShowAttributes | bool | cellview | true | Determines whether the Attributes region is expanded or collapsed.
Valid values: true, false |
| dePropertyEditorShowAttributesPrompt | bool | user | true | Determines whether to display the Attributes section Prompt column in the Property Editor.
Valid values: true, false |
| dePropertyEditorShowAttributesShown | bool | user | false | Determines whether the Attributes region Shown column is shown by default.
Valid values: true, false |
| dePropertyEditorShowAttributesValue | bool | user | true | Determines whether the Attributes region Value column is shown by default.
Valid values: true, false |
| dePropertyEditorShowAttributesVisibility | bool | user | false | Determines whether the Attributes region Visibility' column is shown by default.
Valid values: true, false |

Table 78 *Design Editing Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---|------|----------|---------|--|
| dePropertyEditorShowCommonAllRefs | bool | user | false | In the Property Editor, when in the Show Multiple Item View, displays common parameters of instances and custom vias in the All Instances or All Vias tab so that you can make changes within one tab instead of in different tabs.
Valid values: true, false |
| dePropertyEditorShowConnAssignments | bool | cellview | false | Determines whether the Assignments region is expanded or collapsed.
Valid values: true, false |
| dePropertyEditorShowConnectionAssignmentsDefaultNet | bool | user | false | Determines whether the Connection Assignments region 'Default Net' column is shown by default.
Valid values: true, false |
| dePropertyEditorShowConnectionAssignmentsName | bool | user | true | Determines whether the Connection Assignments region Name column is shown by default.
Valid values: true, false |
| dePropertyEditorShowConnectionAssignmentsNet | bool | user | true | Determines whether the Connection Assignments region Net column is shown by default.
Valid values: true, false |
| dePropertyEditorShowConnectionAssignmentsNewName | bool | user | false | Determines whether the Connection Assignments region New Name column is shown by default.
Valid values: true, false |
| dePropertyEditorShowConnectionAssignmentsVisibility | bool | user | false | Determines whether the Connection Assignments region 'Visibility' column is shown by default.
Valid values: true, false |
| dePropertyEditorShowConnectivity | bool | cellview | true | Determines whether the Connectivity region is expanded or collapsed.
Valid values: true, false |

Table 78 *Design Editing Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---|------|----------|---------|---|
| dePropertyEditorShowConnectivityPrompt | bool | user | true | Determines whether the Connectivity region Prompt column is shown by default.
Valid values: true, false |
| dePropertyEditorShowConnectivityValue | bool | user | true | Determines whether the Connectivity region Value column is shown by default.
Valid values: true, false |
| dePropertyEditorShowInstanceTerminalsAnchor | bool | user | false | Determines whether the Instance Terminals region Anchor column is shown by default.
Valid values: true, false |
| dePropertyEditorShowInstanceTerminalPrompt | bool | user | true | Determines whether the Instance Terminal region Prompt column is shown by default.
Valid values: true, false |
| dePropertyEditorShowInstanceTerminalsCurrent | bool | user | true | Determines whether the Instance Terminals region Current' column is shown by default in layouts.
Valid values: true, false |
| dePropertyEditorShowInstanceTerminalsNetName | bool | user | true | Determines whether the Instance Terminals region Net Name column is shown by default.
Valid values: true, false |
| dePropertyEditorShowInstanceTerminalsTermName | bool | user | true | Determines whether the Instance Terminals region Term Name column is shown by default.
Valid values: true, false |
| dePropertyEditorShowInstanceTerminalsTermType | bool | user | false | Determines whether the Instance Terminals region Term Type column is shown by default.
Valid values: true, false |
| dePropertyEditorShowInstTerminal | bool | cellview | true | Determines whether the Inst Terminal region is expanded or collapsed.
Valid values: true, false |

Table 78 *Design Editing Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---|------|----------|---------|--|
| dePropertyEditorShowInstTerminalValue | bool | user | true | Determines whether the Instance Terminal region Value column is shown by default.
Valid values: true, false |
| dePropertyEditorShowOAParams | bool | user | true | Shows or hides oaParams that have no corresponding CDF parameters.
Valid values: true, false |
| dePropertyEditorShowParameters | bool | cellview | true | Determines whether the Parameters section is expanded or collapsed.
Valid values: true, false |
| dePropertyEditorShowParametersPrompt | bool | user | true | Determines whether the Parameters region Prompt column is shown by default.
Valid values: true, false |
| dePropertyEditorShowParametersType | bool | user | true | Displays parameter types information in the Property Editor. |
| dePropertyEditorShowParametersValue | bool | user | true | Determines whether the Parameters region Value column is shown by default.
Valid values: true, false |
| dePropertyEditorShowParametersVisibility | bool | user | true | Determines whether the Parameters region Visibility column is shown by default.
Valid values: true, false |
| dePropertyEditorShowProperties | bool | cellview | false | Determines whether the Properties region is expanded or collapsed..
Valid values: true, false |
| dePropertyEditorShowPropertiesMaster | bool | cellview | true | Determines whether the Properties region Master column is shown by default.
Valid values: true, false |
| dePropertyEditorShowPropertiesMasterValue | bool | user | true | Determines whether the Properties region Master column is shown by default.
Valid values: true, false |

Table 78 *Design Editing Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|--|------------------------------|----------|---------|--|
| dePropertyEditorShowPropertiesName | bool | user | true | Determines whether the Properties region Name column is shown by default.
Valid values: true, false |
| dePropertyEditorShowPropertiesType | bool | user | false | Determines whether the Properties region Type column is shown by default.
Valid values: true, false |
| dePropertyEditorShowPropertiesValue | bool | user | true | Determines whether the Properties region Value column is shown by default.
Valid values: true, false |
| dePropertyEditorShowPropertiesVisibility | bool | user | false | Determines whether the Properties region Visibility column is shown by default.
Valid values: true, false |
| dePropertyEditorShowTabs | bool | user | false | Displays the Property Editor sections in tabs instead of scrollable sections.
Valid values: true, false |
| dePropertyEditorSingleMultitemMode | enSingle
Multitem
Mode | user | single | Displays single or multiple selected items in the Property Editor.
Valid values: single, multiple |
| dePropertyEditorZoom | bool | cellview | false | Zooms in on the active object shown in the Property Editor.
Valid values: true, false |
| deRecentClipboards | int | user | 10 | Number of recent clipboards to keep. |
| deRecentCount | int | user | 5 | Number of recently opened files to display in the menu defined. The maximum is 25.
Valid values: Any integer in a [1 25] range. |
| deRecoveryFileType | Recovery
FileType | user | regular | File type to use during recovery. |
| deRemoveUnusedStringsOnOpen | bool | user | false | Removes unused strings when opening designs to reduce design size.
Valid values: true, false |
| deRendererBlobSize | int | user | 2 | Size of the blob. |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-------------------------------------|---|----------|----------|---|
| deRendererCacheSize | int | user | 30 | Size of cache. |
| deRendererEnableClump | bool | user | true | Enables clump drawing for 0 filter.
Valid values: true, false |
| deRendererEnableDrawing | int | user | 1 | Enables drawing. |
| deRendererEnableReporting | int | user | 0 | Enables reporting. |
| deRendererNumThreads | int | user | 4 | Maximum number of threads to use for rendering. |
| deReNUMBERInstancesAdditionalPrefix | string | cellview | | Defines the additional prefix to use for instance renumbering. |
| deReNUMBERInstancesApplyTo | deReNUMBERInstancesApplyToEnum | cellview | all | Defines the set of instances to apply renumbering. |
| deReNUMBERInstancesNames | deReNUMBERInstancesNamesEnum | cellview | allNames | Defines the name-based set of instances to apply renumbering. |
| deReNUMBERInstancesOrderBy | deReNUMBERInstancesOrderByEnum | cellview | byRows | Defines the instance renumbering order. |
| deReNUMBERInstancesPattern | string | cellview | | Defines the pattern to use for instance filtering. |
| deReNUMBERInstancesPatternFiltering | deReNUMBERInstancesPatternFilteringEnum | cellview | matching | Defines the pattern-based instance filtering mode. |
| deReNUMBERInstancesPrefixMode | deReNUMBERInstancesPrefixModeEnum | cellview | current | Defines the prefix mode to use for instances renumbering. |
| deReNUMBERInstancesStartFrom | int | cellview | 0 | Defines the start number to use for instance renumbering. |
| deRepeatCommand | bool | user | true | If true, repeats the in post-selection mode. If false, the commands always terminate after a single cycle.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------------|--------|----------|------------|--|
| deReportUnboundMasters | bool | user | true | Indicates whether a design open in a non-headless DE context will be checked for unbound masters.
Valid values: true, false |
| deRulerFontSize | int | user | 12 | Text size of rulers and coordinate marks (min 10, max 50). |
| deRulerTextPrecision | int | user | 5 | Default number of decimal places to use in Layout ruler and dimension mark values. |
| deRuntimeUpdateFrequency | int | user | 500 | The runtime incremental updates frequency. |
| deSaveDesignAsLCV | lcv | user | // | LCV preference of save as dialog |
| deScreenCache | int | user | 1 | enable screen caching in Standard Renderer |
| deSelectAngleMode | string | cellview | orthogonal | Angle mode for interactive selection command.
Valid values: anyAngle, diagonal, orthogonal, xFirst, yFirst |
| deSelectedRecent | int | cellview | -1 | Currently selected recent clip |
| deSelectionTableCompactView | bool | context | false | Groups multiple attribute columns into a single column.
Valid values: true, false |
| deSelectionTableFit | bool | context | false | Zooms in on selected rows in the Selection Table.
Valid values: true, false |
| deSelectionTableLock | bool | user | false | Prevents the synchronization of Selection Table with Selection Set.
Valid values: true, false |
| deSelectionTableParameterNames | string | cellView | "" | Parameters to display in the selection table for instances. |
| deSelectionTableShowFilter | bool | context | false | Displays or hides the filter for the Selection Table.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--------------------------|--------|----------|-----------|--|
| deSelectionTableShowNet | bool | context | false | Displays the Net column in the Selection Table.
Valid values: true, false |
| deSelectionTableTreeView | bool | context | false | Tree view of the Selection Table.
Valid values: true, false |
| deSelectionUndoStackSize | int | user | 10 | Specifies the maximum number of undo and redo actions. |
| deSelectLCV | lcv | cellview | // | Searches LCV for interactive select by name with type == Master. |
| deSelectMode | string | cellview | Select | Selection mode for interactive selection commands.
Valid values: Select, Deselect, Replace |
| deSelectName | string | cellview | | Searches name for interactive select by name. |
| deSelectRegion | string | cellview | Rectangle | Selection type for interactive select by region.
Valid values: Point, Polyline, Rectangle, Polygon |
| deSelectSyncCloneMembers | bool | user | false | Determines if members of figure groups can be selected without using Edit in Group.
Valid values: true, false |
| deSelectType | string | cellview | Instance | Searches type for interactive select by name.
Valid values: Instance, Uneval Text, Master, Eval Text, Net Via, Def MPP, Def Terminal, Striped, Wire |
| deSetViewRecursive | bool | user | false | If true, <code>ide::setViewport</code> command remains on the command stack.
Valid values: true, false |

Table 78 *Design Editing Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-------------------------------------|--------|-------|---------|--|
| deShadowBrightness | int | user | 50 | Specifies the dimming percentage for non-shadow sets, Edit in Place and Edit in Group surrounding data when deSurroundDrawMethod is shadow.
Valid values: Any integer within a [0 100] range. |
| deShareOLPSettings | bool | user | false | Shares technology settings for all designs using the same technology library for which scope is the defined preference rather than design settings.
Valid values: true, false |
| deShowGroupsInReturnToLevelDialog | bool | user | false | Specifies whether groups should be included in Return to Dialog.
Valid values: true, false |
| deShowOpenICC2DesignExcludeCells | string | user | | List of cell types to be excluded during translation by filtering. |
| deShowOpenICC2DesignExcludeUnplaced | string | user | | List of unplaced cell types to be excluded during translation by filtering. |
| deShowOpenICC2DesignRegions | string | user | | Regions to be translated by filtering. |
| deShowOpenICC2DesignUseExisting | bool | user | false | Use existing filters from IC Compiler II designs.
Valid values: true, false |
| deShowRecoveryDialog | bool | user | false | Displays the recovery dialog box when recovery cellviews exist.
Valid values: true, false |
| deShowTooltips | bool | user | true | Displays toolTips where available.
Valid values: true, false |
| deShowTopDesignAxes | bool | user | false | Indicates whether view design axes should be drawn in Edit In Place (EIP) mode.
Valid values: true, false |
| deSingleClumpRedraw | int | user | 1 | Draws only high priority clumps in Standard renderer mode. |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-------------------------------------|------------------------|-------|---------|--|
| deSurroundDrawMethod | surround
DrawMethod | user | shadow | Enables dimming or hiding of surrounding data when in Edit in Place or Edit in Group.
Valid values:
<ul style="list-style-type: none"> <code>none</code>: Does not render objects that are surrounding the target edit hierarchy. The hidden objects are still functional (for example, snapping). <code>full</code>: Renders all objects. <code>shadow</code>: Dims objects surrounding the target edit hierarchy. The level of contrast depends on the <code>deShadowBrightness</code> setting. |
| deSyncWindowCommand | bool | user | false | Synchronizes active commands across opened designs.
Valid values: true, false |
| deSyncWindowLayout | bool | user | true | Synchronizes window layouts across opened designs.
Valid values: true, false |
| deSyncWindowLogic | bool | user | true | Synchronizes selected logic (Design Navigator selections) across opened designs.
Valid values: true, false |
| deSyncWindowViewport | bool | user | true | Synchronizes viewports across opened designs.
Valid values: true, false |
| deSyncWindows | bool | user | false | Turns on window synchronization, which links opened windows of the same type and synchronizes their attributes.
Valid values: true, false |
| deTemplateIgnoreParallelConnections | bool | user | false | Treats all types of vector instances as non-parallel vector instances.
Valid values: true, false |
| deTemplateMatchWithSDSwap | bool | user | false | Enables source-drain swap for the matcher.
Valid values: true, false |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|-------------|-------|-------------------|--|
| deTemplateMergeOtherComponent | bool | user | true | Determines whether to support merging of arbitrary components.
Valid values: true, false |
| deTemplateNetlistFullyConnected | bool | user | true | Determines whether the template netlist of a new template cell must be fully connected. |
| deTemplateTreatAllPowerGroundDeviceAsDummy | bool | user | true | Determines whether to treat all power ground device as dummy.
Valid values: true, false |
| deTextEditor | string | user | xterm -e \$EDITOR | Shell command to bring up an editor. |
| deTextHeight | matchOABBox | user | matchOABBox | Height of the text.
By default, text is sized according to the metrics of fonts used in the tool, which can draw text taller than the specified height and use bounding boxes differing from those calculated in other tools.
Valid values: default, matchOABBox, matchObjectHeight |
| deUndoClearOnSave | bool | user | false | Purges the list of commands in the Transaction History assistant after a Save design operation. |
| deUndoRestoreSelectionLimit | int | user | 500 | The maximum number of object selections that are restored immediately after an undo or redo operation. You can decrease the value to improve the undo and redo performance.
If the actual number of selections is greater than deUndoRestoreSelectionLimit, the selections are not restored after the undo or redo operation. |
| deUniquifyPCellPattern | string | user | | Prefix or suffix to add to the cell name during a Copy > Unique operation. |

Table 78 Design Editing Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-----------------------|-----------------|-------|---------|--|
| deUnrollSource | bool | user | 4 | Padding around bbox of selected object when zooming to search result. Value is between 1 and 10 inclusively. |
| deUpdateFrequency | int | user | 500 | The incremental updates frequency. |
| deViewTypesAllowWrite | List of unknown | user | | Specifies the editors that are allowed to open and modify designs.
Valid values: layout, text, schematic, symbol, config, and overlay. Include character spaces between the values. |
| deWindowScrollbars | bool | user | false | Displays scrollbars on the window.
Valid values: true, false. |
| deZoomLevel | int | user | 4 | Zoom level for Zoom to Fit slider (with check box) in the Search and Replace tool. |

8

Data Management Tcl

This chapter lists commands, objects, and preferences in the Custom Compiler Tcl dm (data management) namespace. The dm Tcl lets you manage data in libraries, library containers, and files.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

The commands covered are:

- [dm::add3DICLibrary](#)
- [dm::addFCLibrary](#)
- [dm::addICC2Library](#)
- [dm::addICCRefLibs](#)
- [dm::addToCellCategory](#)
- [dm::addToLibDefs](#)
- [dm::associateVersionControl](#)
- [dm::checkIn](#)
- [dm::checkLock](#)
- [dm::checkOut](#)
- [dm::checkStatus](#)

- [dm::convertDMSystem](#)
- [dm::compress](#)
- [dm::copyCellCategory](#)
- [dm::copyCells](#)
- [dm::copyCellsUnique](#)
- [dm::copyCellViews](#)
- [dm::copyLib](#)
- [dm::copyProps](#)
- [dm::create3DICLibrary](#)
- [dm::createCell](#)
- [dm::createCellCategory](#)
- [dm::createCellInput](#)
- [dm::createCellView](#)
- [dm::createCellViewInput](#)
- [dm::createFCLibrary](#)
- [dm::createICC2Library](#)
- [dm::createLib](#)
- [dm::createLibInput](#)
- [dm::createMilkywayLib](#)
- [dm::createMultiCellViewInput](#)
- [dm::deleteFromCellCategory](#)
- [dm::findAutoLoadFile](#)
- [dm::findCell](#)
- [dm::findCellView](#)
- [dm::findDMData](#)
- [dm::findDMFile](#)
- [dm::get3DICLayerMap](#)

- [dm::get3DICRefLibs](#)
- [dm::getCategoriesFromCategory](#)
- [dm::getCategoriesOfMember](#)
- [dm::getCellCategories](#)
- [dm::getCells](#)
- [dm::getCellsFromCategory](#)
- [dm::getCellViews](#)
- [dm::getDMFileFollowers](#)
- [dm::getDMFileLeader](#)
- [dm::getDMFiles](#)
- [dm::getFCLayerMap](#)
- [dm::getFCRefLibs](#)
- [dm::getFollowers](#)
- [dm::getICC2LayerMap](#)
- [dm::getICC2RefLibs](#)
- [dm::getLeaders](#)
- [dm::getLibDefPath](#)
- [dm::getLibs](#)
- [dm::getMilkywayRefLibs](#)
- [dm::getOpenDesigns](#)
- [dm::getSelected](#)
- [dm::getStandardLibDefPath](#)
- [dm::getTopLibDefPath](#)
- [dm::getVersionControl](#)
- [dm::getVersionControlSystems](#)
- [dm::importAutoLoadFile](#)
- [dm::isDMFileLeader](#)

- [dm::isHidden](#)
- [dm::moveCells](#)
- [dm::moveCellViews](#)
- [dm::moveLib](#)
- [dm::openLibraryDefinitionsEditor](#)
- [dm::openLibraryManager](#)
- [dm::openLibs](#)
- [dm::refreshLibraryManager](#)
- [dm::refreshLibs](#)
- [dm::registerCopyProps](#)
- [dm::registerVersionControl](#)
- [dm::removeVersionControl](#)
- [dm::renameCellCategory](#)
- [dm::revert](#)
- [dm::saveAsFCRefLib](#)
- [dm::saveAsICC2RefLib](#)
- [dm::set3DICLCVMap](#)
- [dm::set3DICRefLibs](#)
- [dm::setFCLCVMap](#)
- [dm::setFCRefLibs](#)
- [dm::setFollowers](#)
- [dm::setICC2LCVMap](#)
- [dm::setICC2RefLibs](#)
- [dm::setICCTechSync](#)
- [dm::setSelected](#)
- [dm::showAddICC2Library](#)
- [dm::showAddLibrary](#)

- [dm::showCopyCell](#)
- [dm::showCopyCellProperties](#)
- [dm::showCopyCellView](#)
- [dm::showCopyLibrary](#)
- [dm::showCopyUnique](#)
- [dm::showDeleteViewsByName](#)
- [dm::showEditLibraryCompression](#)
- [dm::showEditProperties](#)
- [dm::showLibraryManager](#)
- [dm::showMoveCell](#)
- [dm::showMoveCellView](#)
- [dm::showMoveLibrary](#)
- [dm::showNewCell](#)
- [dm::showNewCellView](#)
- [dm::showNewLibrary](#)
- [dm::showRenameCellCategory](#)
- [dm::syncToVersion](#)
- [dm::uncompress](#)
- [dm::updateLib](#)

dm::add3DICLibrary

Description

Adds the specified 3DIC Compiler Library in the Custom Compiler environment.

Syntax

```
dm::add3DICLibrary -libName <string> [-oaLibPath <string>] -3dicLibPath  
  <string>  
[-layerMapFile <string>] [-lcvMapFile <string>] [-oaTechLib <string>]  
  [-oaIncrTechFile <string>]  
[-dbuPerUU <int>]
```

Arguments

`-libName`

Library name.

`-oaLibPath`

Location of the Open Access (OA) library.

Default: digitalbridge/<libName>

`-3dicLibPath`

Location of the underlying 3DIC Compiler library.

`-layerMapFile`

File that maps 3DIC Compiler (NDM) and Custom Compiler (OA) layers.

`-lcvMapFile`

File that maps 3DIC Compiler (NDM) and Custom Compiler (OA) library, cell, and view (LCV).

`-oaTechLib`

Custom Compiler (OA) library to use.

`-oaIncrTechFile`

Custom Compiler (OA) partial technology file to use for creating an incremental library.

`-dbuPerUU`

dbuPerUU setting for the Custom Compiler (OA) library. The setting determines the scaling between Custom Compiler (OA) and 3DIC Compiler (NDM) designs.

Returns

<none>

dm::addFCLibrary

Description

Adds the specified Fusion Compiler Library in the Custom Compiler environment.

Syntax

```
dm::addFCLibrary -libName <string> [-oaLibPath <string>] -fcLibPath  
  <string> [-layerMapFile <string>]  
  [-lcvMapFile <string>] [-oaTechLib <string>] [-oaIncrTechFile <string>]  
  [-dbuPerUU <int>]
```

Arguments

`-libName`

Library name.

`-oaLibPath`

Location of the Open Access (OA) library.

Default is digitalbridge/<libName>

`-fcLibPath`

Location of the underlying Fusion Compiler library.

`-layerMapFile`

File that maps Fusion Compiler (NDM) and Custom Compiler (OA) layers.

`-lcvMapFile`

File that maps 3DIC Compiler (NDM) and Custom Compiler (OA) library, cell, and view (LCV).

`-oaTechLib`

Custom Compiler (OA) library to use.

`-oaIncrTechFile`

Custom Compiler (OA) partial technology file to use for creating an incremental library.

`-dbuPerUU`

dbuPerUU setting for the Custom Compiler (OA) library. The setting determines the scaling between Custom Compiler (OA) and Fusion Compiler (NDM) designs.

Returns

<none>

dm::addICC2Library

Description

Adds an IC Compiler II Library.

Syntax

```
dm::addICC2Library -libName <string> [-oaLibPath <string>] -icc2LibPath  
<string> [-layerMapFile <string>]
```

```
[-lcvMapFile <string>] [-oaTechLib <string>] [-oaIncrTechFile <string>]  
[-dbuPerUU <int>]
```

Arguments

-libName

Library name.

-oaLibPath

Location of the Open Access (OA) library.

Default is digitalbridge/<libName>

-icc2LibPath

Location of the underlying IC Compiler II library.

-layerMapFile

OA-IC Compiler II layer mapping file path.

-lcvMapFile

File that maps 3DIC Compiler (NDM) and Custom Compiler (OA) library, cell, and view (LCV).

-viaMapFile

Via mapping file.

-oaTechLib

OA library to use.

-oaIncrTechFile

Custom Compiler (OA) partial technology file to use for creating an incremental library.

-dbuPerUU

Specifies the dbuPerUU setting for Custom Compiler (OA) library. This argument also affects scaling between Custom Compiler (OA) and IC Compiler II (NDM) designs.

Returns

<none>

Examples

```
dm::addICC2Library -libName chip_top \  
-oaLibPath oa_chip_top \  
-icc2LibPath chip_top
```

dm::addICRefLibs

Description

Add reference libraries of the specified IC Compiler library.

Syntax

```
dm::addICRefLibs <libs: oaCollection_oaLib_oaLib | {<oaLib>}>
```

Returns

<none>

Arguments

libs

IC Compiler libraries.

dm::addToCellCategory

Description

Adds members, dmCells, or dmCellCategories to a cell category. The cell and cell category members and the cell category itself must be in the same library.

Requires write access to the supplied cell category's library. If access is denied, the command fails. It is legal to add a given cell or cell category to multiple categories.

Syntax

```
dm::addToCellCategory <cellCat: dmCellCategory> [-cell <{<dmCell>}>]  
[-cellCategory <{<dmCellCategory>}>]
```

Returns

The cell category that was updated ([dmCellCategory](#)).

Arguments

[dmCellCategory](#)

The cell category to augment.

-cell {[dmCell](#)}

A collection of oaCells to add as members.

Default: {}.

```
-cellCategory {dmCellCategory}
```

A collection of dmCellCategories to add as members.

Default: {}.

Errors

- oacGroupMemberNotInSameDB.
- oacMismatchDatabaseID.
- oacLibAccessDenied.

Examples

```
# Get cell category object from the specified library
set cat [dm::getCellCategories stdCells -libName PLL]
# Create cell category if doesn't exist
if {[db::isEmpty $cat]} {
    set cat [dm::createCellCategory stdCells -libName PLL]
}
# Add all cells started with buf into category
dm::addToCellCategory $cat -cell [dm::getCells buf* -libName PLL]
```

dm::addToLibDefs

Description

Adds a library reference to the lib.defs file in a manner similar to what dm::createLib does with the new library, and then opens the library.

Syntax

```
dm::addToLibDefs <name: string> -path <string> [-assign <string>]
[-addICCRefLibs <bool>]
```

Returns

<oaLib>: Library that is opened.

Arguments

name

Library name.

-path

Relative or absolute path to the existing library.

`-assign`

Space delimited name value pairs to be assigned (using the ASSIGN statement) to this library in the lib.defs file.

`-addICCRefLibs`

If the specified library is determined to be an IC Compiler library, definitions for each of its reference libraries are also added.

Default: true.

Examples

```
# Add library into top lib.defs
dm::addToLibDefs PDK18N -path ./PDKS/PDK18N -assign {libMode readOnly
  PDKLib true}
```

dm::associateVersionControl

Description

Specifies the name of the version control system that manages a given open library. If the version control system name is not already registered through a call to `dm::`, or if this library is already registered with a different version control system, the system generates an error message. This association is stored as a property in the library. In most cases, the association is established when the Version Control system creates the library, and this command is not needed.

Syntax

```
dm::associateVersionControl <name: string> (-lib <oaLib> | -libName
  <string>)
```

Returns

<none>

Arguments

`name`

The name of the version control system.

`-lib`

An open library to be managed by this version control system.

`-libName`

Name of the library to use.

Examples

The following example specifies SVN as the version control system for the PLL library.

```
dm::associateVersionControl SVN -libName PLL
```

dm::checkIn

Description

Triggers checkInProc if container is managed. An error message is displayed upon check in if not successful. If container is not managed, this command will return an error message.

Syntax

```
dm::checkIn <container: {<dmCellCategory>} | {<dmCellView>} |  
{<dmDMFile>}>
```

Returns

<none>

Arguments

container

Data magement collection to check in.

Examples

The following example checks in the PLL/amp/layout cellview with the registered version control system.

```
dm::checkIn [dm::findCellView layout -cellName amp -libName PLL]
```

dm::checkLock

Description

Checks whether the given files or containers are locked or tries to release abandoned locks. If `-release true`, the command tries to release the abandoned locks. If abandoned locks found are successfully released then `notLocked` is returned.

For a collection or for the `-recursive` option, the return value is determined according to the following logic:

- `noLocked`: All items are not locked.
- `lockedByCurrentProcess`: Any item is locked by the current process.

- `lockedByForeignProcess`: An item is locked by another process.
- The first occurrence of the lock is reported (lock details are printed), and the command exits.

For the `dmContainer` option the command not only checks the files contained under the directory, but also the library lock (`snps.lock`), to make sure that the library is not locked by another process. If `-recursive` is specified, the bouncing progress dialog box is prompted. The command can be interrupted using the **Cancel** button on the progress dialog box or press `Ctrl+C`.

Syntax

```
dm::checkLock <objects: dmCellName | dmCellViewName | dmLibName |
{<dmCell>} | {<dmCellView>} | {<dmDMFile>} | {<oaLib>}> [-recursive
<bool>] [-release <bool>]
```

Returns

`<enDMLockStatus>`: The lock status.

Arguments

`objects`

The container or file to check.

`-recursive`

If `true`, checks recursively, which means this command checks not only the specified container, but any containers within this container. Similarly, for libraries, the command checks every cell and cellView. For cells, the command checks every cellView of that cell. Default: `false`.

`-release`

If `true`, releases abandoned locks. Default: `false`.

Examples

```
dm::checkLock "lib/cell/layout"
dm::checkLock "lib/cell/layout" -release
dm::checkLock [dm::getCellViews -libName lib -cellName cell]
dm::checkLock [lib/cell] -release
```

dm::checkOut

Description

Triggers `checkOutProc` if container is managed. An error message is displayed upon check out if not successful. If container is not managed, this command will return an error message.

Syntax

```
dm::checkOut <container: {<dmCellCategory>} |  
{<dmCellView>} | {<dmDMFile>}>
```

Returns

<none>

Arguments

container

Data management collection to check out.

Examples

The following example checks out the PLL/amp/layout cellview with the registered version control system.

```
dm::checkOut [dm::findCellView layout -cellName amp -libName PLL]
```

dm::checkStatus

Checks status for the given object by triggering the registered checkStatusProc. Top category file cannot be queried with this command.

Syntax

```
dm::checkStatus <container: dmCellCategory | dmCellView | dmDMFile>
```

Returns

<enVersionControlStatus> : Returns the version control status (checkedOut, checkedIn) for the specified object, or unmanaged if the data management object is not managed with a version control system.

Arguments

container

CellView, DMFile, or CellCategory to query.

Examples

The following example returns the PLL/amp/layout cellview status. Possible return values are checkedIn, checkedOut, and unmanaged.

```
dm::checkStatus [dm::findCellView layout -cellName amp -libName PLL]
```

dm::convertDMSystem

Description

Converts a library from its current DM system to the specified DM system.

Renames the original library's top level directory with a '.old' suffix. The current Library Definition File is not changed. The new library has the same name and location as the original.

Requires write access to the library. If access is denied, the command fails.

Syntax

```
dm::convertDMSystem <lib: oaLib | string> -dmSystem <string>
```

Arguments

oaLib | <string>

The library or library name to convert.

-dmSystem <string>

The name of the new DM system.

Examples

```
# Convert specified library to Turbo DM
dm::convertDMSystem PLL -dmSystem oaDMTurbo

# Convert specified library to FileSys DM
dm::convertDMSystem PLL -dmSystem oaDMFileSys
```

dm::compress

Syntax

Compresses a specified library.

This command ensures the safe use of the `oazip -compress` and `oazip -check` commands, by checking for the lock files in library and ensuring that there are no designs used by other processes, before starting the compression process.

```
dm::compress <library: dmLibName | oaLib> (-compressionLevel <enum> |  
-check <bool>)
```

Arguments

`library`

A library name or a library in which the libraries should be compressed or zipped.

`-compressionLevel`

A range of 1-9 compression levels to be defined for the libraries that are created. Supports OpenAccess filesys (oaDMFileSys) and OpenAccess turbo (oaDMTurbo) libraries only.

Values are 1-9 ({ 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 }).

`-check`

Checks the compression state of the library.

Errors

- `dmclllegalLibName`.
- `dmcReadOnlySession`.
- `dmcReadOnlyLibrary`
- `oacLibAccessDenied`
- `oaOSError`.

Returns

none

dm::copyCellCategory

Description

Creates a new cell category in the same library by copying an existing category. Both the original and the new cell category have all the members that were in the original category.

Requires write access to the cell category's library. If access is denied, the command fails.

If a cell category with the new name already exists, the command fails.

Syntax

```
dm::copyCellCategory <sourceCat: dmCellCategory> -name <string> [-parent  
<dmCellCategory>]
```

Returns

The new cell category ([dmCellCategory](#)).

Arguments

[dmCellCategory](#)

The cell category to copy.

`-name <string>`

The name of the new cell category.

`-parent`

The category in which this is a subcategory.

Errors

- `dmClllegalCategoryName`.
- `oacLibAccessDenied`.
- `oacInvalidUniqueGroupName`.

Examples

```
# Get cell category object from the specified library
set cat [dm::getCellCategories stdCells -libName PLL]
# Check that category exist before performing copy
if {1==[db::getCount $cat]} {
    dm::copyCellCategory $cat -name devices
}
```

dm::copyCells

Description

Copies existing cells, including everything they contain (properties, files and cellViews), to a new location. All data files of the cells are copied. The files of the cellViews that are copied are determined by the version control system managing the destination library, or by on the leader-follower relationships that have been set using the `dm::setFollowers` command for libraries that do not have a version control system.

Does not modify the in-memory data; the source `dmCell*` pointers still point to the original locations.

Reports an error if the specified destination cell already exists. If there is such a conflict you must move or delete the destination cell using the `move` or `destroy` command to perform the desired copy.

If the specified source library does not exist, an exception is thrown.

Tcl scripts that want to be notified about the details of the copy operation as it proceeds can create callbacks using `db::createCallback` for the elements of the copy. The callback events triggered during a `copyCell` operation are shown in the example section that follows.

If the `update` parameter is set to true, the references within the cells being copied to other cells being copied are updated to refer to the new location.

If the `hier` parameter is set to true, the `oaDesign` database for each `cellView` that is copied by this command is checked for instances. For any instances that exist, the master `oaCellView` to which they refer is operated on in the same manner.

This behavior recursively applies to all of the instances in the master `cellView` operated on. Thus, copying a master `cellView` from a different library also copies all of the master `cellViews` it references.

If this results in name conflicts due to references having the same cell or `cellView` name in different libraries, the conflicts are resolved by appending an integer to the destination name. The resulting name conflict and the updated name are reported in a warning message in the console.

When the `hier` parameter is set to true, three additional parameters are supported.

The `includeViews` parameter is used to control the set of related `cellViews` to copy (using View names not `ViewType` names) in addition to the instance master's direct `cellView` reference. Alternatively, the `excludeViews` parameter can be used to list the view names that should not be copied as the hierarchy is followed.

The `includeViewTypes` parameter is used to control the set of related view types to copy (using view types names) in addition to the instance master's direct view type reference. Alternatively, the `excludeViewTypes` parameter can be used to list the view types names that should not be copied as the hierarchy is followed.

This allows copying of a 'schematic' `cellsView` to also copy the corresponding 'symbol' `cellView`.

When the `excludeLibs` parameter is provided, any cells that reside in a library in the list is not copied.

If the `hier` parameter is set to false, none of the references in the `oaDesign` databases are followed and the operation only copies the specified cells.

This command requires read access to the library of the cell being copied and write access to the library in which the cell copy is being created. If either access is denied, the command fails.

Syntax

```
dm::copyCells <sourceCells: {<dmCell>}> [-includeViews [<string>] |  
-excludeViews [<string>]]
```



```
[-update <bool> | -updateSearch <enum>] [-includeViewTypes [<string>] |  
  -excludeViewTypes [<string>]]  
-libDestName <string> [-cellDestName <string>] [-skipPcells <bool>]  
  [-hier <bool>]  
[-skipBackupFiles <bool>] [-excludeLibs [<string>]] [-renamePattern  
  <string>] [-regex <bool>]  
[-category [<string>]] [-maintainCategories <bool>]  
  [-destinationConflicts <enum>] [-sourceConflicts <enum>]  
[-cellPattern <string>]
```

Returns

The collection of new cells. (**<dmCell>**)

Arguments

sourceCells

The cell to copy.

-libDestName <string>

Destination library name.

-cellDestName <string>

The name of the destination cell, which is can be used if a single cell is supplied as input, otherwise it is invalid.

-update <bool>

Older version of **-updateSearch**. **-update** true corresponds to **-updateSearch** changed. **-update** false corresponds to **-updateSearch** none.

Default: true.

-skipPcells

Allows to skip PCell references to be copied and updated. (Default is: false)

-updateSearch

Specifies where to search for references to the source cellViews and replace those with references to the destination cellViews. (**<changed | destinationLib | none | writableLibs>**).

Available options:

- **changed**: Replaces references among the copied cellViews.
- **destinationLib**: Replaces references in all cellViews in the target library.

- none: Causes no references to be updated
- writableLibs: Replaces references in all cellViews in the open libraries that can be changed.

Default: none

`-hier <bool>`

For cellViews being copied that have oaDesigns, descend the hierarchy under that cellView copying the masters of each instance even when they belong to cells that aren't in the source list.

Default: false

`-includeViews [<string>]`

When -hier is set to true, this specifies the set of cellViews inside each copied Cell that should be copied and whose references are followed.

`-excludeViews`

When -hier is set to true, this specifies the set of cellViews inside each copied Cell that should not be copied and whose references are followed. This option cannot be used when the `-includeViews` option is used.

`-includeViewTypes [<string>]`

Specifies the set of view type names of views inside each copied cell that should also be copied.

The view type names support regular expression or wildcard matching. This option cannot be used when the `-excludeViewTypes` option is used.

`-excludeViewTypes [<string>]`

Specifies the set of view types names of views associated with each instance master cell that should not be copied.

The view type names support regular expression or wildcard matching. This option cannot be used when the `-includeViewTypes` option is used.

`-skipBackupFiles`

If true, the backup files in views are skipped while copying references. (Default: false)

`-excludeLibs [<string>]`

When -hier is enabled this prevents instance masters in these libraries from being copied. The hierarchy traversal does not descend into these masters. List members are strings that support regular expression or wildcard matching.

`-renamePattern`

This specifies the pattern for renaming a cell where conflicts exist. This option can contain %C %L and %N along with other characters that are used as found. %C substitutes the source cell name. %L substitutes the source library name. %N uses an integer that makes the destination name unique. (Default is: %C_%L).

`-regexp <bool>`

If true the pattern is interpreted as a regular expression, otherwise the pattern uses wildcards.

Default: true.

`-category <string>`

Adds all cells copied to the named category. Creates the category if necessary.

`-maintainCategories`

Adds copied cells into all categories of which the source cell is a member.

Default: true

`-destinationConflicts`

Action to take when a copied cell already exists in the destination: overwrite, rename, or skip. Default: rename

`-sourceConflicts`

Action to take when copied cells from multiple source libraries have the same name: overwrite, rename, or skip. Default: rename

`-cellPattern`

Renames cell names before `-renamePattern` is applied. This option can contain %C %L and %N characters.

Default: " "

Returns

`[{<dmCell>}]` : Collection of new cells

Errors

- `dmclllegalLibName.`
- `dmclllegalCellName.`
- `dmclllegalViewName.`
- `oacCannotFindLibrary.`

- oacLibAccessDenied.
- dmcCellExists.
- dmcCannotLockDMFiles.
- dmcCellNameRequired.
- dmcCellNameInvalidWhenMultipleCellsSupplied.
- oacDataNotCompatibleOnOpen.
- oacDataNotCompatibleOnOpenInAppendMode.

Examples

```
dm::copyCells [dm::getCells inv* -libName oldLib] -libDestName \  
    newLib -updateSearch destinationLib -hier true -includeViews {}\  
    -excludeLibs {analogLib sheets basic}
```

In this example, `dm::copyCells` copies all cells from the `oldLib` library into the `newLib` library, using an `inv*` name match pattern, and including all master cellViews from reference libraries except those in the `analogLib`, `sheets` and `basic` libraries. Searches every cellView in `newLib` and change any instances of `oldLib/inv*` to be instances of `newLib/inv*`.

dm::copyCellsUnique

Description

Copies the hierarchy of specified cellViews to the specified location

Syntax

```
dm::copyCellsUnique (-topCellView <dmCellView> | -topCell <dmCell>)  
(-leafCellView <dmCellView> | -leafCell <dmCell>) -libDestName <string>  
[-renamePattern <string>] [-destinationConflicts  
    <enum>] [-skipBackupFiles <bool>]
```

Returns

`[{<dmCell>}]` : Collection of new cells

Arguments

`topCellView`

The top cellView of the hierarchy to copy

`-topCell`

The top cell of the hierarchy to copy.

`-leafCellView`

The leaf cellView of hierarchy to copy

`-leafCell`

The leaf cell of hierarchy to copy

`-libDestName`

Destination library name

`-renamePattern`

Pattern for renaming all cells. Can contain %C %L and %N (Default is: %C_%L)

`-destinationConflicts`

Action to take when a copied cell already exists in the destination: overwrite, or skip. Default: skip

`-skipBackupFiles`

If true, the backup files in views are skipped while copying references. (Default: false)

Examples

```
set topCV [dm::getCellViews layout -cellName pll -libName PLL]set
leafCV [dm::getCellViews layout -cellName vco_buff -libName
PLL]dm::copyCellsUnique -topCellView $topCV -leafCellView $leafCV \
    -libDestName testLib -renamePattern %C_%L \
    -destinationConflicts overwrite
```

dm::copyCellViews

Description

Copies existing cellviews to a new location. The set of DMFiles that are copied depends on the currently defined Leader-Follower relationship. If the primary DMFile of a cellView is not a Leader (according to the `dm::isDMFileLeader` command), all DMFiles it contains are copied. If the primary DMFile is a Leader, only the Followers (`dm::getDMFileFollowers`) of that file, in addition to the primary DMFile itself, are copied.

Does not modify the in-memory data; the supplied `dmCellView*` pointers still point to the original locations.

Reports an error if the destination cellView already exists. Move or delete the destination cellView using the 'move' or 'destroy' command before performing the copy operation.

If the specified library does not exist, an exception is thrown.

If the update parameter is set to true, all references within the designs copied to other designs that are copied are updated to point to the new location.

If the hier parameter is set to true, the oaDesign database for this oaCellView, if one exists, is inspected for any instances. If any instances exist, the master oaCellView to which they refer is also operated on in the same manner.

This behavior subsequently applies to all of the instances in the master cellView that is operated on. Thus, copying a master cellView from a different library also copies all of the master cellViews it references.

If this results in name conflicts due to references to the same named cell or cellView from different sources, then the situation is handled according to the -source conflicts action. If this results in a conflict because a copied cell or cellView already exists in the destination, then this is handled according to the -destinationConflicts action.

When the hier parameter is set to true, two additional parameters are supported:

- The includeViews parameter is used to control the set of related cellViews to copy (using View names not ViewType names) in addition to the instance master's direct cellView reference.
- When -hier is set to true, this specifies the set of cellViews inside each copied Cell that should not be copied and whose references are followed. This option cannot be used when -includeViews is used.

For example, this allows copying of a 'schematic' cellView to also copy the corresponding 'symbol' cellView.

When the excludeLibs parameter is provided, any cellViews that reside in a library in the list are not copied.

When this command operates directly on a cellView which has a ViewType of 'config', the following behavior also applies if the hier parameter is set to true:

The config cellview as well as the root cellView to which it points are copied. Then the root cellView's instance masters are interrogated according to the rules of the config such that:

Placed masters as well as the view switched master cellViews are copied.

Unlike the netlisters, which ignore certain instance masters, all instance masters are copied even though they are not netlisted.

This is so that a schematic does not have an unbound sheet border symbol even though the symbol is not netlisted

Note that the rules in the config could lead to another config cellView being associated with an instance master as a view switched master, in which case it should be treated in the same way as a direct config cellView copy.

This is only a concern when copying a config cellview directly, because otherwise view switched masters do not exist, and placed masters are not allowed to point to config cellViews.

If the hier parameter is set to false, the oaDesign databases are not inspected and the operation only affects the data in this cellView.

Requires read access to the source cellview's library and write access to the library to which the cellView is being copied. If access is denied, the command fails.

Syntax

```
dm::copyCellViews <sourceCellViews: {<dmCellView>}> [-includeViews
    [<string>]
    | -excludeViews [<string>]] [-includeViewTypes [<string>] |
    -excludeViewTypes [<string>]]
-libDestName <string> -cellDestName <string> [-viewDestName <string>]
[-update <bool>]
[-skipPcells <bool>] [-hier <bool>] [-destinationConflicts <enum>]
[-sourceConflicts <enum>]
[-excludeLibs [<string>]] [-renamePattern <string>] [-skipBackupFiles
    <bool>] [-regex <bool>] [-cellPattern <string>]
```

Returns

The collection of newly copied cellViews ({dmCellView}).

Arguments

{dmCellView}

The collection of cellViews to copy.

-libDestName <string>

The name of the destination library.

-cellDestName <string>

The name of the destination cell.

-viewDestName <string>

Destination view name, required for copying single cellViews.

Default is: "".

-update <bool>

Issue the pre/post copy callbacks for each individual DMFile so their internal references can be updated.

Default is: true.

`-skipPcells`

Allows to skip PCell references to be copied and updated. (Default is: false)

`-hier <bool>`

Copies instance master Cells.

Default is: false.

`-includeViews <string>`

When `-hier` is used, specifies the set of cellViews associated with the instance master CellView should be copied.

`-excludeViews <string>`

When `-hier` is used, specifies the set of cellViews associated with the instance master cellView that should not be copied.

`-includeViewTypes [<string>]`

When the `-hier` option is used, specifies the set of view types names of views inside each cell that should be copied.

The view type names support regular expression or wildcard matching. This option cannot be used when the `-excludeViewTypes` option is used.

`-excludeViewTypes [<string>]`

When `-hier` is used, specifies the set of view types names of views associated with each instance master cell that should not be copied.

The view type names support regular expression or wildcard matching. This option cannot be used when the `-includeViewTypes` option is used.

`-excludeLibs <string>`

Specifies libraries whose cellViews should not be copied List members are strings that support regular expressions or glob-style wildcard matching.

Default is: "" (empty string).

`-renamePattern <string>`

This specifies the pattern for renaming a cell where conflicts exist. This option can contain %C %L and %N long with other characters that are used as found. %C substitutes the source cell name. %L substitutes the source library name. %N uses an integer that makes the destination name unique. (Default is: %C_%L).

`-destinationConflicts`

Action to take when a copied cell already exists in the destination: overwrite, rename, or skip. Default: rename

`-sourceConflicts`

Action to take when copied cells from multiple source libraries have the same name: overwrite, rename, or skip. Default: rename

`-cellPattern`

This option allows you to rename cell names by force before `-renamePattern` is applied. Can contain %C %L and %N.

`-skipBackupFiles`

If true, the backup files in views are skipped while copying references. (Default: false)

`-regexp <bool>`

If true, the pattern is interpreted as a regular expression, otherwise the pattern uses wildcards.

Default: true.

Returns

`[{<dmCellView>}]` : Collection of new cell views

Errors

- `dmclllegalLibName.`
- `dmclllegalCellName.`
- `dmclllegalViewName.`
- `oacCannotFindLibrary.`
- `oacLibAccessDenied.`
- `oacCVExists.`
- `dmcCannotLockDMFiles.`
- `dmcViewNameRequired.`
- `dmcViewNameInvalidWhenMultipleCellViewsSupplied.`
- `oacDataNotCompatibleOnOpen.`
- `oacDataNotCompatibleOnOpenInAppendMode.`

Examples

```
dm::copyCellViews [dm::getCellViews -cellName amp \  
-libName designLib \
```

```
-filter %viewType=="maskLayout"] \
-cellDestName amp -libDestName layoutLib
```

In this example, `dm::copyCellViews` copies the `amp` cell `maskLayout` and `viewType` `cellViews` from the `designLib` library into the cell with the same cell name in the `layoutLib` library.

dm::copyLib

Description

Copies an existing library, including everything it contains (properties, data files, cells, views, and `cellViews`), to a new location creating a new library. All DMFiles of the Library and Cells are copied, while the DMFiles of the `cellViews` that are copied depend on the Leader-Follower relationship that has been set using the `dm::setFollowers` command.

Does not modify in-memory data; the source `oaLib*` (if supplied) still points to the original location.

If the library contains an `oaTech` database, it is also copied. If the library references an `oaTech` database, the reference is maintained in the copied library.

Also updates the Library Definition File, if one exists, to define the new library being created.

Pre- and post-copy events can be triggered for the libraries, cells and `cellViews` that are being copied.

If the `hier` parameter is set to true, the `oaDesign` database for each `cellView` that is operated on as a result of this command is checked for instances. If any instances exist, the master `cellView` to which they refer is operated on in the same manner.

This behavior recursively applies to all of the instances in the master `cellView` that is operated on. So copying a master `cellView` from a different library also copies all of the master `cellViews` it references.

If this results in name conflicts due to references having the same cell or `cellView` name in different libraries, the conflicts are resolved by appending an integer to the destination name. The name conflict and the updated name are reported in a warning in the console.

When the `hier` parameter is set to true, two additional parameters are supported:

The `includeViews` parameter is used to control the set of related `cellViews` (using View names not ViewTypes) to copy in addition to the instance master's direct `cellView` reference.

For example, this allows a 'schematic' `cellView` to be copied together with the corresponding 'symbol' `cellView`.

When the `excludeLibs` parameter is provided, any `cellView` that resides in a library in the list is not copied.

If the `hier` parameter is set to `false`, none of the `oaDesign` databases are inspected and the operation only affects the data in this library.

The following are considered error conditions:

A library with the specified name already exists.

The path for the new library is an empty string; it specifies the name of an existing file or a non-empty directory; or the directory specified by path is not write-able.

The path is not a syntactically valid directory name.

This command requires read access to the library being copied and if access is denied, the command fails.

Syntax

```
dm::copyLib <sourceLib: oaLib | string> [-includeViews [<string>] |  
-excludeViews [<string>]]  
[-includeViewTypes [<string>] | -excludeViewTypes [<string>]] [-update  
<bool>  
| -updateSearch <enum>] -name <string> -path <string> [-skipPcells  
<bool>] [-hier <bool>]  
[-sourceConflicts <enum>] [-excludeLibs [<string>]] [-skipBackupFiles  
<bool>] [-regexp <bool>]  
[-renamePattern <string>] [-assign <string>] [-cellPattern <string>]
```

Returns

The new library (*oaLib*).

Arguments

`sourceLib` | `<string>`

Library to copy.

`-name <string>`

The name of the new library.

`-path <string>`

The location of the new library, which is created if it does not exist.

`-update <bool>`

Updates instances of copied `cellViews` to point to the new copy of that `cellView`.

Default: `true`.

`-skipPcells`

Allows to skip PCell references to be copied and updated. (Default is: false)

`-updateSearch`

Specify where to search for references to change ({ changed | destinationLib | none | writableLibs })

`-hier <bool>`

Descends the design hierarchy for each cellView that contains an oaDesign and copies instance master cells even when they outside the specified source library.

Default: false.

`-includeViews [<string>]`

Specifies the set of cellViews inside each Cell that should be copied.

Default: *.

`-excludeViews [<string>]`

Specifies the set of cellviews associated with each instance master cell that should not be copied.

`-includeViewTypes [<string>]`

Specifies the set of view types names of views inside each cell that should be copied.

The view type names support regular expression or wildcard matching. This option cannot be used when the `-excludeViewTypes` option is used.

Default: *.

`-excludeViewTypes [<string>]`

Specifies the set of view types names of views associated with each instance master cell that should not be copied.

The view type names support regular expression or wildcard matching. This option cannot be used when the `-includeViewTypes` option is used.

`-excludeLibs [<string>]`

Specifies the set of cellviews associated with each cell that should not be copied.

Default: "".

`-skipBackupFiles`

If true, the backup files in views are skipped while copying references. (Default: false)

`-regex <bool>`

If true the pattern is interpreted as a regular expression; otherwise, the pattern uses wildcards.

Default: true.

`-renamePattern <string>`

Pattern for renaming a cell where conflicts exist. This option can contain %C %L and %N.

Default: %C_%L.

`-sourceConflicts`

Action to take when copied cells from multiple source libraries have the same name {overwrite|rename|skip}. Default is *rename*.

`-sourceConflicts`

Action to take when copied cells from multiple source libraries have the same name: overwrite, rename, or skip. Default: rename

`-assign`

Library attributes.

`-cellPattern`

Allows you to rename cell names by force before applying the `-renamePattern` option. Can contain %C %L and %N.

Returns

`<oaLib>` : New library

Errors

- dmclllegalPathSyntax.
- dmclllegalLibName.
- oacLibNameExists.
- oacLibCouldNotCreate.
- oaOSError.
- oacLibAccessDenied.

- `oacDataNotCompatibleOnOpen`.
- `oacDataNotCompatibleOnOpenInAppendMode`.

Examples

```
dm::copyLib designLib -name localLib -path ./localLib
```

In this example `dm::copyLib` copies `designLib` to `localLib` in the current working directory `localLib` location and updates instances of copied `cellViews` to point to the new copy of that instance master.

dm::copyProps

Description

Copies properties from the source cell or library to the destination cell or library. Copying from cell to library or from library to cell is supported.

This command requires read access to the library being copied and if access is denied, the command fails.

The copy operation merges properties into the destination. If a property with the same name exists in the destination, its value and type are overwritten by those from the source. If the property with the same name does not exist in the destination, it is created. Properties present in the destination, but not in the source are left unchanged.

Tcl scripts within the design environment can provide special handling for certain properties that they want to manage. Those subsystems use the `dm::registerCopyProps` command to specify the names of those properties and to provide a handler to do the copy. Such properties are not copied directly by the `dm::copyProps` command, but the registered copy handler is called after all other properties have been copied.

If a registered copy handler fails, it is responsible for logging an appropriate error message and the `copyProps` command simply continues processing any remaining registered copy commands.

Library and cell properties are stored in `oaDMData` databases in their respective library container. The `copyProps` command opens the destination `oaDMData` for writing and creates it if it did not previously exist.

Upon completion, the destination's `oaDMData` is saved. If the source or destination `oaDMData` was previously open, it is left open in its previous mode. Otherwise the `oaDMData` is closed.

If this command cannot read the source's `oaDMData` and the destination's `oaDMData` in 'append mode' then the command fails. It also scans the list of properties that are to be copied and verifies that all registered copy routines exist before proceeding with the

operation. If they do not, the `dmcCopyProcDoesNotExist` error is returned which contains the list of Tcl procs that do not exist.

Syntax

```
dm::copyProps -src <oaLib | dmCell> -dest <oaLib | dmCell>
```

Arguments

`-src oaLib | dmCell`

The source.

`-dest oaLib | dmCell`

The destination.

dm::create3DICLibrary

Description

Creates the specified 3DIC Compiler Library in the Custom Compiler environment.

Syntax

```
dm::create3DICLibrary (-techFile <string> | -techLib <string>) -libName  
  <string> [-oaLibPath <string>]  
-3dicLibPath <string> [-layerMapFile <string>] [-lcvMapFile <string>]  
  [-oaTechLib <string>]  
[-oaIncrTechFile <string>] [-dbuPerUU <int>] [-scaleFactor <int>]  
[-refLibPaths [<string>]]
```

Arguments

`-libName`

Library name.

`-oaLibPath`

Location of the Open Access (OA) library. Default: `digitalbridge/<libName>`

`-3dicLibPath`

Location of the underlying 3DIC Compiler library.

`-layerMapFile`

File that maps 3DIC Compiler (NDM) and Custom Compiler (OA) layers.

`-lcvMapFile`

File that maps 3DIC Compiler (NDM) and Custom Compiler (OA) library, cell, and view (LCV).

`-oaTechLib`

Custom Compiler (OA) library to use.

`-oaIncrTechFile`

Custom Compiler (OA) partial technology file to use for creating an incremental library.

`-dbuPerUU`

dbuPerUU setting for the Custom Compiler (OA) library. The setting determines the scaling between Custom Compiler (OA) and 3DIC Compiler (NDM) designs.

`-techFile`

The 3DIC Compiler (NDM) technology file for library creation.

`-scaleFactor`

A scale factor for the created 3DIC Compiler library.

`-techLib`

The 3DIC Compiler (NDM) technology library name for creating a library.

`-refLibPaths`

List of reference library paths that are added to the created library.

Returns

<none>

dm::createCell

Description

Creates the specified cell in the specified library.

Requires write access to the supplied library. If access is denied, the command fails.

Syntax

```
dm::createCell <name: string> (-lib <oaLib> | -libName <dmLibName>)
```

Returns

Created cell ([dmCell](#)).

Arguments

`name`

The name of the cell to create.


```
-lib oaLib
```

The library in which the cell should be created.

```
-libName <string>
```

The library name in which the cell should be created.

Errors

- `dmclllegalCellName`.
- `dmclllegalLibName`.
- `oacCannotFindLibrary`.
- `oacLibAccessDenied`.
- `dmcCellExists`.

Examples

```
dm::createCell amp -libName designLib
```

In this example, `dm::createCell` creates `amp` cell in the existing `designLib` library.

dm::createCellCategory

Description

This method creates a cell category in the specified library and adds the supplied cells and cell categories to it. If a reserved cell category name is supplied ('All' or 'Uncategorized') the `dmcReservedCellCategoryName` error is returned.

Requires write access to the supplied library. If access is denied, the command fails.

Syntax

```
dm::createCellCategory <name: string> (-lib <oaLib> | -libName <string>)  
[-cell <{<dmCell>}>] [-cellCategory <{<dmCellCategory>}>] [-parent  
  <dmCellCategory>]
```

Returns

The cell category that was created ([dmCellCategory](#)).

Arguments

```
<string>
```

The name of the cell category to create.

`-lib oaLib`

The library in which to create the cell category.

`-libName <string>`

The library in which to create the cell category.

`-cell {dmCell}`

A collection of oaCells to add as members to the new category.

Default: {}.

`-cellCategory {dmCellCategory}`

A collection of dmCellCategories to add as members to the new category.

Default: {}.

`-parent`

The category in which this is a subcategory. If no parent is specified, it is a top level category.

Errors

- dmclllegalLibName.
- dmclllegalCategoryName.
- oacInvalidObjForGroup.
- oacInvalidUniqueGroupName.
- oacLibAccessDenied.
- dmcReservedCellCategoryName.

Examples

```
dm::createCellCategory inv -libName designLib \  
    -cell [dm::getCells inv* -libName designLib]
```

In this example, dm::createCellCategory creates the inv cell category in the designLib library, and adds all cells whose names start with the inv substring to the newly created cell category.

dm::createCellInput

Description

When customizing dialogs this creates a widget that allows a user to specify a lib and cell. Creates a new dmCellInput widget and adds it to the specified parent.

Syntax

```
dm::createCellInput [{none}: string] [-parent <giContainer | giDialog>]  
[-enabled <bool>] [-shown <bool>] [-prefName <dbPrefName>] [-toolTip  
<string>] [-valueChangeProc [<variant>]] [-label <string>] [-value  
<dmCell | string>] [-readOnly <bool>] [-valid <bool>] [-required <bool>]  
[-requireExisting <bool>] [-showReadOnlyLibraries <bool>] [-filterProc  
[<variant>]]
```

Returns

New <dmCellInput> widget.

Arguments

{none}

Widget name.

-parent

Dialog, group, or other container into which the widget should be placed.

-enabled

Initial enabled state.

Default: true.

-shown

Initial state shown.

Default: true.

-prefName

Optional preference name within the scope of the top-level container which provides the initial value. The value is automatically updated with changes according to the container model.

-toolTip

Optional widget tool tip.

Default: "".

-valueChangeProc

Procedure to execute when the value is changed. Called with the widget object.

-label

Label for this item, shown to the left of the entry fields.

Default: Cell.

`-value`
Initial value in the format lib/cell/view.

`-readOnly`
Initial read-only state.
Default: false.

`-valid`
Initial valid state.
Default: true.

`-required`
Specifies whether this is a required input.
Default: false.

`-requireExisting`
Specifies that the cell must exist.
Default: true.

`-showReadOnlyLibraries`
Specifies whether to show read-only libraries.
Default: true.

`-filterProc`
Procedure to run for each cellview object when populating the list of completions. Returns 1 if the cellview should be available, or 0 otherwise.

Examples

In this example, the `dmCellInput` widget is added to custom dialog box and the selected Library and Cell names are printed in the console when the user changes the value in the widget.

```
# Procedure to execute when the value of widget is changed
proc cellChangeCB {w} {
    puts "LC = [db::getAttr w.value.libName]/[db::getAttr w.value.name]"
}

# Create custom dialog
set dlg [gi::createDialog snpsWidgets -title "Widgets"]
dm::createCellInput cell \
    -parent $dlg \
    -value [dm::getCells nmos4 -libName analogLib] \
    -valueChangeProc cellChangeCB
```

dm::createCellView

Description

Creates a cellView with the specified viewType in the specified cell. If the cell and/or view do not already exist, they are created. If the view type does not already exist, an oacInvalidViewType error is returned.

Requires write access to the supplied cell's library. If access is denied, the command fails.

Syntax

```
dm::createCellView <name: string> -cell <dmCell | dmCellName> -viewType  
    <string>
```

Returns

Created cellView ([dmCellView](#)).

Arguments

name

View name of the cellView to create.

-cell [dmCell](#)

The cell in which to create the cellView.

-viewType <string>

The view type of the cellView.

Errors

- dmclllegalViewName.
- oacLibAccessDenied.
- oacCVExists.

Examples

```
# Create symbol view  
dm::createCellView symbol -cell PLL/buf -viewType schematicSymbol  
# Create layout view  
dm::createCellView layout -cell [dm::findCell buf -libName PLL] -viewType  
maskLayout
```

dm::createCellViewInput

Description

When customizing dialogs this creates a widget that allows a user to specify a lib cell and view name. Creates a new dmCellViewInput widget and adds it to the specified parent.

Syntax

```
dm::createCellViewInput [ $\{none\}$ : string] [-parent <giContainer |  
giDialog>] [-enabled <bool>] [-shown <bool>] [-prefName <dbPrefName>]  
[-toolTip <string>] [-valueChangeProc [ $\{variant\}$ ]] [-label <string>]  
[-value <dmCellView | string>] [-readOnly <bool>] [-valid <bool>]  
[-required <bool>] [-requireExisting <bool>] [-showReadOnlyLibraries  
<bool>] [-filterProc [ $\{variant\}$ ]]
```

Returns

New dmCellViewInput object.

Arguments

$\{none\}$

Widget name.

-parent

Dialog, group, or other container into which the widget should be placed.

-enabled

Initial enabled state.

Default: true.

-shown

Initial state shown.

Default: true.

-prefName

Optional preference name within the scope of the top-level container which provides the initial value. The value is automatically updated with changes according to the container model.

-toolTip

Optional widget tool tip.

Default: "".

`-valueChangeProc`

Procedure to execute when the value is changed. Called with the widget object.

`-label`

Label for this field, shown to the left of the input fields.

Default: View.

`-value`

Initial value in the format lib/cell/view.

`-readOnly`

Initial read-only state.

Default: false.

`-valid`

Initial valid state.

Default: true

`-required`

Specifies whether this is a required input.

Default: false.

`-requireExisting`

Specifies whether that the cellView must exist.

Default: true.

`-showReadOnlyLibraries`

Specifies whether to show read-only libraries.

Default: true.

`-filterProc`

Procedure to execute for each cellView object while populating the list of completions. Returns 1 if the cellView should be available or 0 otherwise.

Examples

In this example, the dmCellViewInput widget is added to the custom dialog box and the selected Library, Cell and View names are printed in the console when the user changes the value in the widget.

```
# Procedure to execute when the value of widget is changed
proc viewChangeCB {w} {
```

```
puts "LCV = [db::getAttr w.value.libName]/[db::getAttr  
w.value.cellName]/[db::getAttr w.value.name]"  
}  
  
# Create custom dialog  
set dlg [gi::createDialog snpsWidgets -title "Widgets"]  
dm::createCellViewInput view \  
-parent $dlg \  
-value [dm::getCellViews symbol -cellName nmos4 -libName analogLib] \  
-valueChangeProc viewChangeCB
```

dm::createFCLibrary

Description

Creates a Fusion Compiler library in the Custom Compiler environment.

Syntax

```
dm::createFCLibrary (-techFile <string> | -techLib <string>) -libName  
<string> [-oaLibPath <string>]  
-fcLibPath <string> [-layerMapFile <string>] [-lcvMapFile <string>]  
[-oaTechLib <string>]  
[-oaIncrTechFile <string>] [-dbuPerUU <int>] [-scaleFactor <int>]  
[-refLibPaths [<string>]]
```

Arguments

-libName

Library name.

-oaLibPath

Location of the Open Access (OA) library.

Default: digitalbridge/<libName>

-fcLibPath

Location of the underlying Fusion Compiler library. Default: " "

-layerMapFile

File that maps the Custom Compiler (OA) and Fusion Compiler (NDM) layers.

-lcvMapFile

File that maps 3DIC Compiler (NDM) and Custom Compiler (OA) library, cell, and view (LCV).

-oaTechLib

Custom Compiler (OA) library to use.

`-oaIncrTechFile`

Custom Compiler (OA) partial technology file to use for creating an incremental library.

`-dbuPerUU`

dbuPerUU setting for Custom Compiler (OA) library. The setting determines the scaling between the Custom Compiler (OA) and Fusion Compiler (NDM) designs.

`-techFile`

Fusion Compiler (NDM) technology file for library creation.

`-scaleFactor`

Scale factor for the Fusion Compiler library in the Custom Compiler environment.

`-techLib`

Fusion Compiler technology library name for library creation.

`-refLibPaths`

List of refLib paths to add to the created library.

Returns

<none>

dm::createICC2Library

Description

Creates an IC Compiler II library. Use this command if your design originated in the Custom Compiler environment, and you want to transfer it to the IC Compiler II tool for digital place-and-route.

Syntax

```
dm::createICC2Library (-techFile <string> | -techLib <string>) -libName  
  <string> [-oaLibPath <string>]  
-icc2LibPath <string> [-layerMapFile <string>] [-lcvMapFile <string>]  
  [-oaTechLib <string>]  
[-oaIncrTechFile <string>] [-dbuPerUU <int>] [-scaleFactor <int>]  
[-refLibPaths [<string>]]
```

Arguments

`-libName <string>`

Library name.

`-oaLibPath`

Location of the Open Access (OA) library.

Default: digitalbridge/<libName>

`-ICC2LibPath <string>`

Location of the underlying IC Compiler II library.

`-layerMapFile <string>`

Location of the OA-IC Compiler II layer mapping file.

`-lcvMapFile`

File that maps 3DIC Compiler (NDM) and Custom Compiler (OA) library, cell, and view (LCV).

`-oaTechLib`

OA technology library to use.

`-dbuPerUU`

dbuPerUU setting for the OA technology library. The argument affects the scaling between the OA environment and the IC Compiler II environment.

`-techFile`

IC Compiler II technology file to use for library creation.

`-oaIncrTechFile`

Custom Compiler (OA) partial technology file to use for creating an incremental library.

`-scaleFactor`

Scale factor to use when creating the IC Compiler II library.

`-techLib`

IC Compiler II technology library name for library creation.

`-refLibPaths`

List of refLib paths to add to created library.

Returns

<none>

dm::createLib

Description

Creates a library and updates the Library Definition File, if one exists, for the new library being created.

If the path argument is set to the empty string (""), an oacLibCouldNotCreate error occurs.

This command appends a DEFINE statement for the new library to the contents of the current Library Definition File. If that file is not writable, then a dmcCannotWriteDefinitionFile error occurs. If there is no current library definition file because the search at start-up didn't find one, this command creates \$SYNOPYS_CUSTOM_LOCAL ./lib.defs with the new library definition. The assign argument causes ASSIGN statements to be written to the lib.defs file after the new DEFINE statement. The argument value should be a space-delimited list of name value pairs. Each pair causes one ASSIGN statement to be written.

For example:

```
-assign "dbPcellCachePath /usr/designs/cache myAttr myValue"
```

generates two ASSIGN statements, one for dbPcellCachePath and one for myAttr.

Syntax

```
dm::createLib <name: string> -path <string> [-dmSystem <string>]  
[-assign <string>] [-attrs [<map>]] [-compressionLevel  
  <compressionLevel>]
```

Returns

Library that is created (*oaLib*)

Arguments

<string>

Library name.

-path <string>

Location of the library directory that is created if it does not exist.

-dmSystem <string>

DM system for the library.

Default: oaDMFileSys.

-assign <string>

Name value pairs to be ASSIGNED to this library in the library.

`-attrs`

A list of persistent attributes to be given to the new library.

`-compressionLevel`

Defines compression level of created library.

Values 1-9 ({ 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 }) are supported. Supports OpenAccess filesystems (oaDMFileSys) and OpenAccess turbo (oaDMTurbo) libraries only.

Errors

- dmclllegalPathSyntax.
- dmclllegalLibName.
- dmcCannotWriteDefinitionFile.
- oacLibNameExists.
- oacLibCouldNotCreate.
- oaOSError.

Examples

```
dm::createLib newLib -path ./libDir
```

In this example, dm::createLib creates the newLib library in the ./libDir directory.

dm::createLibInput

Description

When customizing dialogs this creates a widget that allows a user to specify a lib name. Creates a new dmLibInput widget and adds it to the specified parent.

Syntax

```
dm::createLibInput [<{none}: string>] [-parent <giAssistant | giContainer  
| giDialog | giWindow>] [-enabled <bool>] [-shown <bool>] [-prefName  
<dbPrefName>] [-toolTip <string>] [-valueChangeProc [<variant>]]  
[-label <string>] [-value <string>] [-readOnly <bool>] [-valid <bool>]  
[-required <bool>] [-requireExisting <bool>] [-showReadOnlyLibraries  
<bool>] [-filterProc [<variant>]]
```

Returns

New dmLibInput object.

Arguments

`{none}`

Widget name.

`-parent <giContainer | giDialog>`

Dialog, group, or other container into which the widget should be placed.

`-enabled <bool>`

Initial enabled state.

Default: true.

`-shown <bool>`

Initial state shown.

Default: true.

`-prefName <string>`

Optional preference name within the scope of the top-level container which provides the initial value. The value is automatically updated with changes according to the container model.

`-toolTip`

Optional widget tool tip.

Default: "".

`-valueChangeProc <string>`

Procedure to execute when the value is changed. Called with the widget object.

`-label <string>`

Label displayed for this item, shown to the left of the entry field.

Default: View.

`-value <string>`

Initial value in the format lib/cell/view.

`-readOnly <bool>`

Initial read-only state.

Default: false.

`-valid <bool>`

Initial valid state.

Default: true.

`-required <bool>`

Specifies whether this is a required input.

Default: false.

`-requireExisting <bool>`

Specifies whether the library must exist.

Default: true.

`-showReadOnlyLibraries`

Specifies whether to show read-only libraries.

Default: true.

`-filterProc`

Procedure to run for each cellview object when populating the list of completions. Returns 1 if the cellview should be available, or 0 otherwise.

Examples

In this example, the `dmLibraryInput` widget is added to the custom dialog box and the selected Library name is printed in the console when the user changes the value in the widget.

```
# Procedure to execute when the value of widget is changed
proc viewChangeCB {w} {
    puts "Library = [db::getattr w.value]"
}

# Create custom dialog
set dlg [gi::createDialog snpsWidgets -title "Widgets"]
dm::createLibInput library \
    -parent $dlg \
    -value analogLib \
    -valueChangeProc viewChangeCB
```

dm::createMilkywayLib

Description

Creates a IC Compiler library using the specified IC Compiler technology file.

Syntax

```
dm::createMilkywayLib <name: string> -milkywayTechFile <string> [-refLibs  
    [<string>]] [-busNameStyle <string>]
```

Arguments

`name`

Filepath of the IC Compiler library to be created.

`-milkywayTechFile <string>`

Path, including the file name of technology file to use.

`-refLibs <string>`

List of paths to reference libraries (absolute or relative, related to the main library).

`-busNameStyle <string>`

Format string using `%d` to determine how bus indexes are to be formatted, for example, "`[%d]`".

dm::createMultiCellViewInput

Description

Creates a new `dmMultiCellViewInput` widget and adds it to the specified parent.

Syntax

```
dm::createMultiCellViewInput [{<none>: string}] [-parent <giContainer |  
giDialog>] [-enabled <bool>] [-shown <bool>] [-prefName <dbPrefName>]  
[-toolTip <string>] [-filterProc [<variant>]] [-valueChangeProc  
[<variant>]] [-value <dmCellView>] [-readOnly <bool>] [-valid <bool>]  
[-required <bool>] [-requireExisting <bool>]
```

Returns

`<dmMultiCellViewInput>`: New `dmMultiCellViewInput` widget

Arguments

`{<none>}`

Widget name.

Default: "".

`-parent`

Dialog, group or other container into which this widget should be placed

`-enabled`

Initial enabled state.

Default: true.

`-shown`

Initial shown state.

Default: true.

`-prefName`

Optional preference name within top-level container's scope, which provides the initial value and is automatically updated with value changes according to the container update model.

Default: "".

`-toolTip`

Optional widget tool tip.

Default: "".

`-filterProc`

Procedure to execute for each cellView object while populating the list of completions. Returns 1 if the cellView should be available or 0 otherwise.

`-valueChangeProc`

Procedure to execute when the value is changed. Called with the widget object.

`-value`

Initial value.

`-readOnly`

Initial read-only state.

Default: false.

`-valid`

Initial valid state.

Default: true.

`-required`

Specifies if this is a required input.

Default: false.

`-requireExisting`

Specifies that the library must exist.

Default: true.

Examples

```
# Procedure to execute when the value of widget is changed
proc viewChangeCB {w} {
    db::foreach cv [db::getAttr w.value] {
        puts "LCV = [db::getAttr cv.libName]/[db::getAttr
cv.cellName]/[db::getAttr cv.name]"
    }
}

# Create custom dialog box
set dlg [gi::createDialog snpsWidgets -title "Widgets"]
dm::createMultiCellViewInput view \
    -parent $dlg \
    -value [dm::getCellViews symbol -cellName nmos4 -libName analogLib] \
    -valueChangeProc viewChangeCB
```

dm::deleteFromCellCategory

Description

Removes members, oaCells, or dmCellCategories from a cell category.

Requires write access to the supplied cell category's library. If access is denied, the command fails.

Syntax

```
dm::deleteFromCellCategory <cellCat: dmCellCategory> [-cell <{<dmCell>}>]
[-cellCategory <{<dmCellCategory>}>]
```

Returns

The cell category that was updated ([dmCellCategory](#))

Errors

oacLibAccessDenied.

Arguments

[dmCellCategory](#)

The cell category from which to remove members.

-cell {[dmCell](#)}

A collection of oaCells to remove as members.

Default: {}.

-cellCategory {[dmCellCategory](#)}

A collection of dmCellCategories to remove as members.

Default: {}.

Examples

The example removes cells those names start with "buf" substring from stdCells category in the specified library.

```
dm::deleteFromCellCategory [dm::getCellCategories stdCells -libName PLL]
-cell [dm::getCells buf* -libName PLL]
```

dm::findAutoLoadFile

Description

Finds and returns the dmDMFile representing the auto-load file for this library. Returns NULL if a file does not exist.

Requires read access to the library. If access is denied the command fails.

Syntax

```
dm::findAutoLoadFile -lib <oaLib> -libName <string>
```

Returns

The autoLoad file associated with the library, or NULL if there is no associated autoLoad file ([dmDMFile](#)).

Arguments

-lib *oaLib*

The library to search.

-libName <string>

The name of the library to search.

Errors

- dmclllegalLibName.
- oacLibAccessDenied.

Examples

```
# Get DMFile object associated with the specified library
set aFile [dm::findAutoLoadFile -libName reference10]
# Print absolute path of file if exist
if {"!"=$aFile} {
    puts [db::getattr aFile.path]
}
```

dm::findCell

Description

Returns the cell with the specified name residing in the specified library.

The cell might, or might not yet exist on the disk. This function can be used to supply a cell to a command that does not require the underlying library cell to exist.

Syntax

```
dm::findCell <name: string> (-lib <oaLib> | -libName <dmLibName>)
```

Returns

Cell object of the specified name ([dmCell](#)).

Arguments

name

The name of the cell to find.

-lib oaLib

The library in which the cell exists.

-libName <string>

The library name in which the cell exists.

Errors

- dmclllegalCellName.
- dmclllegalLibName.

Examples

```
dm::findCell amp -libName designLib
```

In this example, dm::findCell finds and returns the amp dmCell residing in the designLib library. The amp cell is found if it exists, regardless of whether it is saved on disk yet, or not.

dm::findCellView

Description

Returns a cellView of the specified name within the specified cell.

Does not check if the cellView actually exists on disk; it can be used to supply a cellView to a command that does not require the underlying library data to exist beforehand.

Syntax

```
dm::findCellView <name: string> (-cell <dmCell> | -cellName <string>)  
[-lib <oaLib>] [-libName <string>]
```

Returns

A cellView ([dmCellView](#))

Arguments

<string>

The view name of the cellView to find.

-cell [dmCell](#)

The cell in which the cellView resides.

-cellName <string>

The name of the cell in which the cellView resides.

-lib *oaLib*

The library in which to look for the specified cell and cellView.

-libName <string>

The name of the library in which the cellView from the specified cellName resides.

Errors

- dmclllegalLibName.
- dmclllegalCellName.

Examples

```
dm::findCellView layout -cellName amp -libName designLib
```

In this example, dm::findCellView returns the layout dmCellView within the designLib/amp cell.

dm::findDMData

Attempts to find open DM data for a given library, cell, or cellview. If successful, oaDMData object is returned. Otherwise, empty string is returned.

Syntax

```
dm::findDMData <dmContainer: dmCell | dmCellName | dmCellView |  
dmCellViewName | dmLibName | oaLib>
```

Returns

<oaDMData> : DM data object of the specified container

Arguments

dmContainer

DM data container.

Examples

The following example finds and returns DM data for the specified library or cell object.

```
db::openParamDefEditor -libName reference40nm
set dmdata [dm::findDMData reference40nm]
db::getattr dmdata.type
=> LibDMData
db::openParamDefEditor \
-cell [dm::findCell n_4t -libName reference40nm]
set dmdata [dm::findDMData reference40nm/n_4t ]
db::getattr dmdata.type
=> CellDMData
```

dm::findDMFile

Description

Returns a DM File with the specified name within the specified DM container object.

It does not check if the DM File actually exists on disk. It can be used to supply a DM File to a command that does not require the underlying data to exist.

Syntax

```
dm::findDMFile <name: string> -dmContainer <dmCell | dmCellName |
dmCellView | dmCellViewName | dmLibName | oaLib>
```

Returns

A DM file ([dmDMFile](#)).

Errors

dmclllegalLibName.

Arguments

<string>

The name of the DM File to find.

```
-dmContainer oaLib | <string> | dmCell | dmCellView
```

The DM container in which the DM File exists, where <string> is interpreted as a library name.

Examples

```
# Find and print analogLib sample library dm file path
set dmfile [dm::findDMFile data.dm -dmContainer [dm::getLibs
reference40nm]]
puts "Path: [db::getAttr dmfile.path]"

# Find and print analogLib/nmos4 cell dm file
set dmfile [dm::findDMFile data.dm -dmContainer [dm::getCells nmos4
-libName analogLib]]
puts "Path: [db::getAttr dmfile.path]"

# Check if data.dm file does not exist in the specified cell view
set dmfile [dm::findDMFile data.dm -dmContainer [dm::getCellViews symbol
-cellName nmos4 -libName analogLib]]
if {""==[db::getAttr dmfile.path]} {
    puts "data.dm file doesn't exist"
}
```

dm::get3DICLayerMap

Get 3DIC Compiler layer mappings.

Syntax

```
dm::get3DICLayerMap <library: oaLib>
```

Returns

<string> : List of layer name pairs

Arguments

library

Design library.

dm::get3DICRefLibs

Gets 3DIC Compiler (NDM) reference libraries from the snpsNDM library.

Syntax

```
dm::get3DICRefLibs -lib <oaLib>
```

Returns

<string> : Reflib paths

Arguments

-lib

Design library.

dm::getCategoriesFromCategory

Description

Returns a collection of cell categories that are members of the specified cell category and whose names match the specified pattern.

Requires read access to the cell category's library. If access is denied, the command fails.

Syntax

```
dm::getCategoriesFromCategory [<pattern: string>] [-filter <string>]  
-category <dmCellCategory>
```

Returns

A collection of cell categories in the cell category ([dmCellCategory](#)).

Errors

oacLibAccessDenied.

Arguments

<string>

A glob-style wildcard pattern to match cell category names against

Default: "*".

-category [dmCellCategory](#)

The cell category to query.

-filter <string>

Standard attribute filter.

Default: Match all.

Examples

```
db::foreach cat [dm::getCategoriesFromCategory -category  
[dm::getCellCategories Sources -libName analogLib]] {
```

```
# Print child category name
puts [db::getattr cat.name]
}
```

dm::getCategoriesOfMember

Description

Returns a collection of cell categories whose names match the supplied pattern and for which the supplied cell or cell category is a member.

Specify either the cell or cellCategory argument.

Requires read access to the supplied cell or cell category's library. If access is denied, the command fails.

Syntax

```
dm::getCategoriesOfMember [<pattern: string>] (-cellCategory
<dmCellCategory> | -cell <dmCell>) [-filter <string>]
```

Returns

A collection of cell categories ({[dmCellCategory](#)}).

Errors

oacLibAccessDenied.

Arguments

<string>

A glob-style wildcard pattern for cell category names.

Default: "".

-cell [dmCell](#)

Find categories containing this cell.

-cellCategory [dmCellCategory](#)

Find categories containing this cell category.

-filter <string>

Standard attribute filter.

Default: Match all.

Examples

```
puts [db::getAttr name -of [dm::getCategoriesOfMember -cell [dm::getCells  
  ccvs -libName analogLib]]]  
>> Dependent
```

dm::getCellCategories

Description

Returns a collection of cell categories in the specified library whose names match the specified pattern.

Requires read access to the supplied library. If access is denied, the command fails.

Syntax

```
dm::getCellCategories [<pattern: string>] (-lib <oaLib> | -libName  
  <string>) [-filter <string>] [-topLevel <bool>]
```

Returns

A collection of cell categories ([{dmCellCategory}](#)).

Arguments

<string>

A glob-style wildcard pattern to match cell category names against.

Default: *.

-lib

The libraries from which to retrieve the cell categories.

-libName <string>

The libraries from which to retrieve the cell categories.

-filter <string>

Standard attribute filter.

Default: Match all.

-topLevel

Only include top level categories.

Default: false.

Errors

- dmclllegalLibName.
- oacLibAccessDenied.

Examples

```
# Get collection of cell category objects in the specified library
db::foreach cat [dm::getCellCategories -libName analogLib] {
    # Print category name
    puts "Category Name = [db::getAttr name -of $cat]"
    # Print cell names belonging to category
    db::foreach cell [db::getAttr cat.cells] {
        puts "    Cell Name = [db::getAttr cell.name]"
    }
}
```

dm::getCells

Description

Returns a collection of cells from the specified library or libraries. The collection only contains cells whose names match the specified pattern.

Requires read access to the supplied libraries and if access is denied, the command fails.

Syntax

```
dm::getCells [<pattern: string>] (-lib <{<oaLib>}> | -libName
    <dmLibName>) [-filter <string>]
```

Returns

A collection of cells ({dmCell}).

Arguments

pattern

A glob-style wildcard pattern to match cell names with.

Default: "".

-lib {oaLib}

The collection of libraries from which to retrieve cells.

-libName <string>

The library name from which to retrieve cells.

`-filter <string>`

Standard attribute filter.

Default: Match all.

Errors

- `dmclllegalLibName`.
- `oacLibAccessDenied`.

Examples

```
dm::getCells inv* -lib [ oa::LibFind [oa::ScalarName \  
    [oa::CdbaNS]{designLib}]]  
dm::getCells inv* -libName designLib
```

In these two examples, `dm::getCells` returns the collection of `dmCells` whose names match the `inv*` name pattern.

dm::getCellsFromCategory

Description

Returns a collection of cells that are members of the specified cell category and whose names match the specified pattern.

Requires read access to the supplied cell category's library. If access is denied, the command fails.

Syntax

```
dm::getCellsFromCategory [<pattern: string>] [-filter <string>] -category  
    <dmCellCategory>
```

Returns

A collection of cells in the cell category (`{dmCell}`).

Errors

`oacLibAccessDenied`

Arguments

`<string>`

A glob-style wildcard pattern to match cell names against.

Default: `"*"`.

`-category dmCellCategory`

The cell category to query.

`-filter <string>`

Standard attribute filter.

Default: Match all.

Examples

```
db::getAttr cells -of [dm::getCellCategories Mos -libName \  
generic90RF]
```

In this example, `dm::getCellCategories` passes the Mos `dmCellCategory` object from the `generic90RF` library to the `db::getAttr` command, which returns collection of cells in that category.

dm::getCellViews

Description

Returns a collection of `cellViews` from the specified cell whose names match the specified pattern.

Requires read access to the cell's library. If access is denied, the command fails.

Syntax

```
dm::getCellViews [<pattern: string>] (-cellName <string> | -cell  
<{<dmCell>}>) [-libName <string>] [-lib <{<oaLib>}>] [-filter <string>]
```

Returns

A collection of `CellViews` ([{dmCellView}](#)).

Arguments

`<string>`

A glob-style wildcard pattern to match `cellView` names against.

Default: `"*"`.

`-cell {dmCell}`

A collection of cells from which to retrieve `cellViews`.

`-cellName <string>`

A cell name from which to retrieve `cellViews`.

`-lib {oaLib}`

The collection of libraries within which to retrieve cellViews with the specified names.

`-libName <string>`

The library name from which to retrieve cellViews with the specified cell name.

`-filter <string>`

Standard attribute filter.

Default: Match all.

Errors

- `dmclllegalLibName`.
- `dmclllegalCellName`.
- `oacLibAccessDenied`.

Examples

```
dm::getCellViews -cellName amp -libName codeBash -filter \  
    %viewType=~/^schematic/
```

In this example, `dm::getCellViews` returns the collection of schematic and schematicSymbol viewType `dmCellViews` from the `amp` cell in the `designLib` library.

dm::getDMFileFollowers

Description

Returns a collection of DMFiles that are considered followers of this DMFile and whose names match the specified pattern. If no followers exist, returns an empty collection.

Leadership is based on the relationships set using the `dm::setFollowers` command.

This command requires read access to the DMFile's library. If access is denied, the command fails. Note that `dm::getDMFileFollowers` operates on a specific `dmDMFile` in a specific `cellView`, whereas `dm::setFollowers` defines a pattern that can be applied to any `cellView`.

Syntax

```
dm::getDMFileFollowers [<pattern: string>] -leader <dmDMFile> [-filter  
    <string>]
```

Returns

A collection of followers of a DMFile (`{dmDMFile}`).

Errors

oacLibAccessDenied.

Arguments

<string>

A glob-style wildcard pattern to match follower names against.

Default: "*".

-leader [dmDMFile](#)

The DM file to query.

-filter <string>

Standard attribute filter.

Default: Match all.

dm::getDMFileLeader

Description

Returns the leader of this DMFile. If this file is not a follower, returns Null.

Leaders and followers are relationships within a given cellView that follow the pattern set by the dm::setFollowers command.

This command requires read access to the DMFile's library. If access is denied, the command fails.

Syntax

```
dm::getDMFileLeader <dm_file: dmDMFile>
```

Returns

The leader of the DMFile ([dmDMFile](#)).

Errors

oacLibAccessDenied.

Arguments

[dmDMFile](#)

The DM file to query.

dm::getDMFiles

Description

Returns a collection of DM Files from the specified DM container where the file names match the specified pattern.

Requires read access to the supplied container's library. If access is denied, the command fails.

Syntax

```
dm::getDMFiles [<pattern: string>] [-filter <string>] -dmContainer  
  <dmCell | dmCellView | oaLib | string>
```

Returns

A collection of DM Files ([dmDMFile](#)).

Arguments

<string>

A glob-style wildcard pattern to match DMFile names against.

Default: "*".

-dmContainer *oaLib* | <string> | [dmCell](#) | [dmCellView](#)

The DM container from which to retrieve DMFiles, where 'string' is interpreted as a library name.

-filter <string>

Standard attribute filter.

Default: Match all.

Errors

- dmclllegalLibName.
- oacLibAccessDenied.

Examples

```
db::foreach dmf [dm::getDMFiles -dmContainer [dm::getCellViews layout  
  -cellName amp -libName PLL]] {  
    puts "Name: [db::getattr dmf.name] Path: [db::getattr dmf.path]"  
}
```

Output sample:

```
>>Name: flightlines.xml Path: /projects/PLL/amp/layout/flightlines.xml
```

```
>>Name: DMData Path: /projects/PLL/amp/layout/data.dm  
>>Name: snapshot.png Path: /projects/PLL/amp/layout/snapshot.png  
>>Name: maskLayout Path: /projects/PLL/amp/layout/layout.oa
```

dm::getFCLayerMap

Description

Gets Fusion Compiler layer mappings.

Syntax

```
dm::getFCLayerMap <library: oaLib>
```

Returns

<string> : List of layer name pairs

Arguments

library

Design library.

dm::getFCRefLibs

Description

Gets Fusion Compiler (NDM) reference libraries from the snpsNDM library.

Syntax

```
dm::getFCRefLibs -lib <oaLib>
```

Returns

<string> : Reflib paths

Arguments

-lib

Design library.

dm::getFollowers

Description

Returns a list of strings which represent the DMFile names that have been defined as followers of the supplied Leader name and whose names match the specified pattern. This command does not operate on specific DMFiles; rather, it returns the names set by dm::setFollowers that establishes leader/follower patterns.

If no followers have been defined, returns an empty collection.

Syntax

```
dm::getFollowers [<pattern: string>] -leader <string>
```

Returns

A list of strings containing the names of followers ([<string>]).

Errors

None.

Arguments

<string>

A glob-style wildcard pattern to match follower names against.

Default: "".

-leader <string>

The DMFile name to query.

dm::getICC2LayerMap

Description

Get IC Compiler II layer mappings.

Syntax

```
dm::getICC2LayerMap <library: oaLib>
```

Returns

<string> : List of layer name pairs

Arguments

`library`

Design library.

dm::getICC2RefLibs

Description

Gets the IC Compiler II (NDM) reference libraries from the snpsNDM library.

Syntax

```
dm::getICC2RefLibs -lib <oaLib>
```

Returns

<string> : Reflib paths

Arguments

`-lib`

Library.

dm::getLeaders

Description

Returns a list of strings which represent the DMFile names for the currently defined DM Leaders as defined by the dm::setFollowers command.

Syntax

```
dm::getLeaders [<pattern: string>]
```

Returns

List of strings which represent the DMFile names (matching the supplied pattern) defined as Leaders in the current Custom Compiler session. ([<string>]).

Errors

None.

Arguments

<string>

A glob-style wildcard pattern for leader names.

Default: "".

dm::getLibDefPath

Description

Returns the full default path, including the filename, for the library definition file, specified by the name argument, in the user's environment. If the file is not found, an empty string is returned.

The path to the first-located file is returned. This might, or might not be the library definition file used by the current session. See [dm::getTopLibDefPath](#) for that information.

Syntax

```
dm::getLibDefPath [-name <string>]
```

Returns

The full path including the filename for the library definition file (<string>).

Arguments

-name <string>

The name of the library definition file to find.

Default: lib.defs.

Examples

```
set path [dm::getLibDefPath -name lib.defs]
puts "lib.defs path: $path"
```

dm::getLibs

Description

Returns a collection of open libraries whose names match the specified pattern. Does not return Custom Compiler specific temporary libraries.

Syntax

```
dm::getLibs [<pattern: string>] [-filter <string>]
```

Returns

A collection of libraries (*{oaLib}*).

Errors

None.

Arguments

`<string>`

A glob-style wildcard pattern to match library names against.

Default: `"*"`.

`-filter <string>`

Standard attribute filter

Default: Match all.

Examples

```
dm::getLibs *Tech*
```

In this example, `dm::getLibs` returns the collection of `oaLib` objects whose names contain the `Tech` substring.

dm::getMilkywayRefLibs

Description

Returns a Tcl list of IC Compiler reference library names from a library. This can be used for any IC Compiler library whether or not it has been opened as an IC Compiler library by the Custom Compiler tool.

Syntax

```
dm::getMilkywayRefLibs <filepath: string> [-hier <bool>]
```

Returns

`<string>`: List of `filePaths` to the reference libraries specified in this IC Compiler library.

Arguments

`filepath`

Filepath of the IC Compiler library from which to get the specified reference libraries (ref. lib. list).

`-hier <bool>`

If true, recursively extracts references from all reference libraries.

Default: false.

Errors

- Library does not exist.
- Library is inaccessible.

dm::getOpenDesigns

Description

Gets open designs.

Syntax

```
dm::getOpenDesigns
```

Returns

{<oaDesign>} : A collection of open designs

Arguments

<none>

Examples

In this example, all designs opened in memory and their modes will be printed in the console.

```
db::foreach design [dm::getOpenDesigns] {  
    puts "LCV: [db::getAttr design.libName]/[db::getAttr  
    design.cellName]/[db::getAttr design.viewName] Mode: [db::getAttr  
    design.mode]"  
}
```

dm::getSelected

Description

Returns the list of items currently selected in the specified Library Manager window. Only the item or items in the right-most column in which a selection is made are returned. Determining when the returned value is an oaLib and when it is a dmCell or dmCellView requires using "db::getAttr type" on the returned object.

Syntax

```
dm::getSelected [-window <giWindow>]
```

Arguments

-window

A Library Manager window.

Default: The most recently activated Library Manager window.

Returns

{<dmCell>} | {<dmCellCategory>} | {<dmCellView>} | {<dmDMFile>} | {<oaLib>} : A collection of Selected Items.

Examples

```
# Get selected items from the active Library Manager window and
print their attributesdb::foreach item [dm::getSelected] {      switch
[db::getAttr item.type] {      "Lib" {      puts "Library:
[db::getAttr item.name]"      }      "dmCell" {      puts
"Library: [db::getAttr item.libName] \
Cell: [db::getAttr item.name]"      }
"dmCellView" {      puts "Library: [db::getAttr item.libName] \
Cell: [db::getAttr item.cellName] \
View: [db::getAttr item.name]"      }
"dmCellCategory" {      puts "Name: [db::getAttr item.name] \
Library: [db::getAttr item.lib.name]"      }      }
}
```

dm::getStandardLibDefPath

Description

Gets the path to a version of the session's library definition file that can be read by any standard OpenAccess application. This might or might not be the same as the path returned by getTopLibDefPath.

Syntax

```
dm::getStandardLibDefPath
```

Returns

<string>: Full path, including the filename, for the current session's standard library definition file.

Arguments

<none>.

Examples

```
puts [dm::getStandardLibDefPath]
```

dm::getTopLibDefPath

Description

Returns the full path, including the filename, for the session's current library definition file. If there is no current file, an empty string is returned.

Syntax

```
dm::getTopLibDefPath
```

Returns

The full path including the filename for the current session's library definition file.

Examples

```
puts [dm::getTopLibDefPath]
```

dm::getVersionControl

Description

Returns the name of the version control system that manages the specified library. If the library does not have a registered version control system, the command returns an empty string.

Syntax

```
dm::getVersionControl <lib: oaLib>
```

Returns

<string>: Name of the version control system.

Arguments

lib

An open library.

Examples

The command in the following example returns the version control system name that is associated with the PLL library. In this case, the version control system is SVN, so the command returns SVN.

```
dm::getVersionControl [dm::getLibs PLL]  
=> SVN
```

dm::getVersionControlSystems

Description

Returns a collection of dmVersionControlSystem objects that are registered in the current session.

Syntax

```
dm::getVersionControlSystems [<name: string>] [-filter <string>]
```

Returns

{<dmVersionControlSystem>} : A collection of version control system objects

Arguments

name

The name of the version control system.

-filter

Filters the results based on the criteria you specify.

Examples

The command in the following example returns the name (attribute) of the registered version control system object. In this case, the version control system name is SVN, so the command returns SVN.

```
set vcs [dm::getVersionControlSystems ]
db::getAttr vcs.name
=> SVN
```

dm::importAutoLoadFile

Description

Imports a Tcl file from the specified location and installs it as the auto-load file for the specified library.

The file must contain Tcl only.

The auto-load file is automatically sourced when the library is opened.

The auto-load file can source additional files in the library using the oa::DMFileCreate command to import them and follow that by using the oa::DMFileFind command in combination with the oa::getPath command to obtain the actual file as it exists within the library.

```
source [oa::getPath [oa::DMFileFind $lib <name>]]
```


If an auto-load file already exists, a `dmcAutoLoadFileExists` exception is thrown if the `overwrite` argument is set to `false`.

Requires write access to the specified library. If access is denied the command fails.

Syntax

```
dm::importAutoLoadFile -lib <oaLib> -libName <string> -path <string>
[-overwrite <bool>]
```

Returns

The imported file ([dmDMFile](#)).

Arguments

`-lib oaLib`

The library into which the auto-load file is to be imported.

`-libName <string>`

The name of the library into which the auto-load file is to be imported.

`-path <string>`

The pathname of the file to import.

`-overwrite <bool>`

If true, overwrites the existing auto-load file if one exists. If false, the command fails if there is an existing file.

Default: false.

Errors

- `dmclllegalPathSyntax`.
- `dmclllegalLibName`.
- `dmcAutoLoadFileExists`.
- `oacLibAccessDenied`.

Examples

```
dm::importAutoLoadFile -libName designLib \  
-path ~/scripts/libSupport.tcl
```

In this example, `dm::importAutoLoadFile` imports the `~/scripts/libSupport.tcl` file into the `designLib` library and installs it as a file automatically loaded when the `designLib` library opens.

dm::isDMFileLeader

Description

Returns true if this DMFile is a leader, otherwise, returns false.

The DMFile is considered to be a leader if its DMFile name was specified as a 'leader' using the dm::setFollowers command.

Syntax

```
dm::isDMFileLeader <dm_file: dmDMFile>
```

Returns

True if this DMFile is a leader; otherwise false (<bool>).

Errors

None.

Arguments

dmDMFile

The DM file to query.

dm::isHidden

Description

Checks whether the cell is hidden (the cell is included in the Hidden category, or the cell's `shown` attribute is false). Also checks if the library has the INVISIBLE attribute assigned in the lib.defs file.

Syntax

```
dm::isHidden <object: dmCell | oaLib | string> [-lib <oaLib> | -libName  
<string>]
```

Returns

<bool> : Returns true if the cell is included in the Hidden category or in its subcategories, or the cell's `shown` attribute is false. Also returns true if library is defined as INVISIBLE in lib.defs.

Arguments

object

Cell or library to check.

`-lib`

Library that is checked.

`-libName`

Library name.

dm::moveCells

Description

Moves an existing cell, including everything it contains (properties, files, and cellViews), to a new location.

The function destroys the original files, including any in-memory data and points this dmCell to the new data. If any cellViews in this cell have unsaved changes, the user is prompted to save or to discard those changes.

This command reports an error if the specified destination cell already exists. In this case, the user must move or delete the destination cell using the 'move' or 'destroy' command in order to perform the desired move.

If the specified library does not exist the command fails.

If the update parameter is set to true, then references within the moved cell to other moved data are relocated to refer to the new location.

Note that this is different from Copy, because it does not support a hier parameter. The Leader-Follower relationship defined with the dm::setFollowers command has no impact on which files in the cellViews are copied.

This command moves all files in the cell and its cellViews; it does not check leader/follower relationships. It only affects the data in the cell that is actually being moved, whereas Copy can copy data from potentially different libraries by inspecting the oaDesign databases in the oaCellViews that the library contains and then copying the external references into the destination location as well.

This command requires write access to the library of the cell being moved and also on the library into which the cell is being moved and if either access is denied, the command fails.

Syntax

```
dm::moveCells <sourceCells: {<dmCell>}> [-update <bool> | -updateSearch  
  <enum>] -libDestName <string> [-cellDestName <string>] [-skipPcells  
  <bool>] [-category [<string>]] [-maintainCategories <bool>]  
  [-destinationConflicts <enum>] [-cellPattern <string>]
```

Returns

The new library being moved.

Arguments

`sourceCells`

Cells to move.

`-libDestName <string>`

Name of the destination library

`-cellDestName <string>`

Name of the destination cell.

`-update <bool>`

Older version of `-updateSearch`. `-update true` corresponds to `-updateSearch changed`. `-update false` corresponds to `-updateSearch none`.

Default: `true`.

`-skipPcells`

Allows to skip PCell references to be copied and updated. (Default is: `false`)

`-updateSearch`

Specifies where to search for references to the source cellViews and replace those with references to the destination cellViews. (`{changed | destinationLib | none | writableLibs}`).

Available options:

- `changed`: Replaces references among the copied cellViews.
- `destinationLib`: Replaces references in all cellViews in the target library.
- `none`: Causes no references to be updated
- `writableLibs`: Replaces references in all cellViews in the open libraries that can be changed.

Default: `none`.

`-category <string>`

Adds all cells moved to the named category. Creates the category if necessary.

`-maintainCategories`

Adds copied cell into all categories in which the source cell is a member.

Default: `true`.

`-destinationConflicts`

Action to take when a copied cell already exists in the destination: overwrite, rename, or skip. Default: rename

`-cellPattern`

This option allows you to rename cell names by force before `-renamePattern` is applied. Can contain %C %L and %N.

Errors

- `dmclllegalLibName`.
- `dmclllegalCellName`.
- `oacCannotFindLibrary`.
- `oacLibAccessDenied`.
- `dmcCellExists`.
- `dmcDMFileIsOpen`.
- `oacDataNotCompatibleOnOpenInAppendMode`.

Examples

```
dm::moveCells [dm::findCell amp -libName designLib] \  
              -libDestName designLib -cellDestName amplifier
```

In this example, `dm::moveCells` renames the `amp` cell from `amp` to `amplifier`.

dm::moveCellViews

Description

Moves an existing `cellView`, including all files and properties it contains, to a new location.

Destroys the original files, including any in-memory data, and points this `cellView` * to the new data. If this `cellView` has any unsaved changes, the user is prompted to save or to discard those changes.

Reports an error if the destination `cellView` already exists. Move or delete the destination `cellView` using the 'move' or 'destroy' command in order to perform the conflicting move.

If the specified library does not exist, an exception is thrown.

If the update parameter is set to true, all instances within moved designs that reference masters that are also being moved are updated to point to the new location.

Note that this command differs from the Copy command by not supporting a hier parameter. The Leader-Follower relationship defined with the dm::setFollowers command has no impact on which DMFiles in the cellViews are copied.

This command only affects the data in the cellView that is actually being moved, whereas Copy can actually copy data from potentially different libraries by inspecting the oaDesign databases in the oaCellViews that the library contains and then copying the external references into the destination location as well.

Requires write access to the source cellView's library and to the library to which the cellView is being moved. If access is denied, the command fails.

Syntax

```
dm::moveCellViews <sourceCellViews: {<dmCellView>}> -libDestName  
  <string> -cellDestName <string> [-viewDestName <string>] [-update  
  <bool>] [-skipPcells <bool>] [-cellPattern <string>]
```

Returns

The moved cellView ([dmCellView](#))

Arguments

sourceCellViews

The cellView to move.

-libDestName <string>

The name of the destination library.

-cellDestName <string>

The name of the destination cell.

-viewDestName <string>

The name of the destination view.

-update <bool>

Issue the pre- and post-move callbacks for each individual DMFile so references within the file can be updated.

Default: true.

-skipPcells

Allows to skip PCell references to be copied and updated. (Default is: false)

-cellPattern

This option allows you to rename cell names by force before -renamePattern is applied. Can contain %C %L and %N.

Errors

- dmclllegalLibName.
- dmclllegalCellName.
- dmclllegalViewName.
- oacCannotFindLibrary.
- oacLibAccessDenied.
- oacCVExists.
- dmcDMFilesOpen.
- oacDataNotCompatibleOnOpenInAppendMode.

Examples

```
dm::moveCellViews [dm::getCellViews -cellName amp \  
    -libName designLib -filter %viewType=~/*schematic*/ ] \  
    -cellDestName amp -libDestName schematicLib
```

In this example, `dm::moveCellViews` moves the `amp` cell's schematic and symbol views from the `designLib` library to the `schematicLib` library.

dm::moveLib

Description

Moves an existing library, including everything it contains (properties, cells, views and cellViews), to a new location. The new library must have a different name than the source library.

Destroys the source library including any in-memory data. If any cellViews in the source library are open in a design editor and have unsaved modifications, then the user is prompted to save those designs. Unsaved changes in any of the existing files are lost.

If the library contains an `oaTech` database, the database is also moved. If the library references an `oaTech` database, the reference is maintained in the moved library.

Also updates the Library Definition File, if one exists, to add the new library and to delete the old one. If the `lib.defs` file had attributes assigned to the old library, then those attributes direct the specified library attributes to be assigned to the new library.

If the update parameter is set to true, then references within the moved library to other cellViews in the moved library are updated.

Note that Move is different from Copy because it does not support a hier parameter. The Leader-Follower relationship defined with the dm::setFollowers command does not affect which DMFiles in the cellViews are copied.

This command only affects the data in the library that is actually being moved, whereas Copy can actually copy data from other libraries by inspecting the oaDesign databases in the oaCellViews that the library contains and then copying the external references into the destination location as well.

The following are considered error conditions:

A library with the specified name already exists.

The specified path is an empty string; it specifies the name of an existing file or a non-empty directory; or the directory specified by path is not write-able.

The specified path is not a syntactically valid directory.

This command requires write access to the library being moved and if access is denied, the command fails.

Syntax

```
dm::moveLib <sourceLib: oaLib | string> -name <string> -path <string>  
  [-update <bool>] [-skipPcells <bool>] [-assign <string>] [-cellPattern  
  <string>]
```

Returns

The library being moved (*oaLib*).

Arguments

sourceLib

Library to move

-name

Destination library name

-path

Destination library path

-update

Issue the pre/post callbacks for each dmfile (Default is: true)

-skipPcells

Allows to skip PCell references to be copied and updated. (Default is: false)

`-assign`

Library attributes

`-cellPattern`

This option allows you to rename cell names by force before `-renamePattern` is applied. Can contain %C %L and %N.

Errors

- `dmclllegalPathSyntax`.
- `dmclllegalLibName`.
- `oacLibAccessDenied`.
- `oacLibNameExists`.
- `oacLibCouldNotCreate`.
- `oaOSError`.
- `oacUnableToRemove`.
- `dmcDMFileIsOpen`.
- `oacDataNotCompatibleOnOpenInAppendMode`.

Examples

```
dm::moveLib oldLib -name newLib -path ./newLibPath
```

In this example, `dm::moveLib` moves the `oldLib` library data to the `newLibPath` directory in the current working directory and renames the `oldLib` library to `newLib`.

dm::openLibraryDefinitionsEditor

Description

Invokes the Library Definitions Editor.

Syntax

```
dm::openLibraryDefinitionsEditor
```

Returns

<none>

Arguments

None

dm::openLibraryManager

Description

Opens a new Library Manager window. Each time it is called, creates a new Library Manager window and calls the dm::refreshLibs command.

Syntax

```
dm::openLibraryManager
```

Returns

The window identifier ([giWindow](#)).

Arguments

None.

dm::openLibs

Description

Reads a set of library definitions from a specified file and replaces the set of currently open libraries known to the application with the set of libraries defined in this operation. By default the file used is \$SYNOPSIS_CUSTOM_LOCAL/lib.defs.

If the path argument is supplied:

This function reads the library definition file residing in the directory at path. The file name can be specified using the optional name argument, which defaults to "lib.defs".

Note that if a relative path is specified, it is interpreted relative to the current working directory.

If the file is found, this function reads the library definitions listed in the file and any definitions listed in other library definition files included in the file, and opens all the listed libraries.

If the path argument is not supplied:

This function searches for a default library definition file. In each of these directories it looks for the file specified with the optional name argument or "lib.defs" if name was not used.

If found, this function reads the library definitions listed in the file and any definitions listed in other library definition files included in the file, and opens all the listed libraries.

The operation is aborted (no libraries are opened) if there are any syntax errors in the library definition file.

Warnings other than these errors do not result in a termination of the function. Instead, `openLibs()` issues a load warning, then attempts to continue opening the libraries. For example, if there are two library definitions with the same logical library name in the `oaLibDefList` hierarchy, `openLibs` issues a warning, then loads the last of the duplicate defined libraries (the system uses the path to the last-listed conflicting library name to open the library).

If the previous use of `openLibs` opened a library that is not still included in the definitions seen by the current call then that library is closed. Note that libraries first opened directly by a call to `oa::LibOpen` are not closed, and these libraries are visible in the Library Manager unless they are also defined in the library definition file. Where libraries are found that were previously open but are now specified with a different path, Custom Compiler closes those libraries and reopens them in the new location. The `dm::openLibs` command is designed for use in scripts. It does not prompt the user to close open libraries and designs. See `dm::refreshLibs` for a command that asks the user to confirm such events.

Syntax

```
dm::openLibs [-name <string>] [-path <string>]
```

Arguments

`-name <string>`

The name of the Library Definition File to open.

Default: `lib.defs`.

`-path <string>`

An absolute or relative path. A relative path should be specified relative to the current working directory.

Default: `""` (empty string).

Examples

```
dm::openLibs -name myLib.defs -path ./
```

In this example, `dm::openLibs` replaces the set of currently open libraries with the libraries defined in the file `./myLib.defs`.

dm::refreshLibraryManager

Description

Updates the contents of the specified Library Manager window, or all Library Manager windows if an argument is not specified.

Syntax

```
dm::refreshLibraryManager [<window: giWindow>]
```

Arguments

window

A Library Manager window.

Default: "".

Examples

```
# Refresh all instances of Library Manager windowdb::foreach window
[gi::getWindows \
    -filter {%windowType.name=="dmLibraryManager"}} {
    dm::refreshLibraryManager $window
}
```

dm::refreshLibs

Description

Opens the libraries in the supplied library definition file, including all of its included references. If the filepath argument is not supplied, then the previously read library definition file is re-read. This means that if you specified a specific Library Definition List at startup or if a Library Definition List was found in a lower precedence directory in the startup search path, it is reread, not replaced, by calling this command. If a top-level Library Definition File does not currently exist when the filepath is not supplied, the command exits without making changes.

This command closes libraries that are no longer defined, and libraries that are defined with a different path than when last opened. When that happens, if the user has any designs from those libraries that are open with unsaved edits, it prompts the user to either save or discard the changes.

The command is aborted (no libraries opened) if there are syntax errors in the library definition file or in any of the files the library definition file references.

Warnings, other than syntax errors, do not cause the function to terminate. Instead, a warning is issued and the command continues. For example, if there are two library definitions with the same logical library name in the oaLibDefList hierarchy, openLibs issues a warning and then loads the last of the duplicate defined libraries (that is, the system uses the path to the last-listed conflicting library name to open the library).

This command is intended for interactive use. When running Custom Compiler batch scripts, use the dm::openLibs() command instead. The dm::openLibs() command won't prompt for already opened libraries whose definitions have changed. Instead, it closes those libraries and reopens them in the new location.

Syntax

```
dm::refreshLibs [<path: string>]
```

Arguments

<string>

The complete filepath to a Library Definition File.

Default: "".

Examples

```
dm::refreshLibs [dm::getLibDefPath]
```

dm::registerCopyProps

Description

This command allows a Tcl procedure to be registered for the supplied list of property names that are called by the dm::copyProps command when the property name is encountered by that command. This allows subsystems (and users) to manipulate data that is stored in the specified oaProps in the destination container using the specified routine instead of simply copying the property and its value as dm::copyProps does inherently.

If multiple Tcl procs are registered for the same property name, by calling this command multiple times, each Tcl proc is called in the order in which it was registered, unless the overwrite argument is set to 'true', in which case the list of Tcl procs for each supplied property name are cleared before adding the supplied Tcl proc.

The signature of the supplied Tcl proc must be of the form:

```
proc name prop destination  
  
<string> {<oaProp>} <oaLib | dmCell>
```

The {<oaProp>} parameter is a dbCollection.

Syntax

```
dm::registerCopyProps <names: [string]> -tclProc <string> [-overwrite  
    <bool>]
```

Arguments

[<string>]

A list of property names.

`-tclProc <string>`

Tcl proc to use to copy the supplied list of property. Proc signature should be:
`proc myProc {dboaProplter <oaLib>|<dmCell>} {}`.

`-overwrite <bool>`

If false, appends the supplied Tcl proc to the list of previously supplied Tcl procs for properties of the same name. If true, the previous list of Tcl procedures is cleared.

Default: false.

dm::registerVersionControl

Description

This command provides the Custom Compiler data management system with the name of a Version Control (VC) system integration, and also specifies if the VC system can be used for creating new libraries.

If this command sets the primary argument to "true", and if another VC system was the primary system, then the primary option is disabled and the previous one remains the primary system. If the VC system name is already registered, then the current arguments supersede the previous one.

The Proc arguments are names of Tcl functions that are evaluated by the data management system to notify the VC integrations about the requests. The Proc arguments should include a namespace specific to a given integration, and it is possible to specify arguments that are placed before the arguments that are added by the data management system.

The `checkInProc` is a Tcl function, called when the user saves a design in a library managed by this VC system and requests for it to be checked in. The function takes the collection of containers and files to be checked in. This function is called with `dmCellViews` or with a `dmFile` when a property database (data.dm file) is saved. Any errors by this function are logged or otherwise ignored.

The `checkOutProc` is a Tcl function that is called when the user requests to write or modify the data in a library managed by this VC system and requests that it be checked out. This function takes the collection of containers and files that are to be checked out. It is called with `dmCellViews` or with a `dmFile` when a property database (data.dm file) is opened. Any error by this function causes the Custom Compiler command to abort, or to change the behavior to work with read-only data.

The `testCheckOutProc` is a Tcl function that can be called by the Custom Compiler tool to see if library elements can be checked out. The function takes a collection of both containers and files. The function returns a boolean. Returning a true status indicates

that a checkout would succeed. Any error returned is treated as a failure but is otherwise ignored. This function can be used to improve the usability of commands that would otherwise query the users to perform a check out.

The `deleteProc` is called by the Custom Compiler tool with a collection of both `dmContainers` and `dmFiles`. It notifies the VC system that data is about to be deleted. Unless the `deleteByVersionControl` argument is true for the affected version control system, the Custom Compiler tool deletes the intended library elements. This allows the VC system to enter the deleted state in their repository when the changes are checked in. The Custom Compiler tool does not check in the deletions directly.

The `checkStatusProc` is an optional Tcl function that, when provided, is called before opening a file for editing. It is called with a single object, either a `dmCellView` or a `dmFile`. It returns a string that is one of: “nm”, “co”, or “ci”. These stand for “notManaged”, “checkedOut”, and “checkedIn”. When this function is provided, the Custom Compiler tool uses this function rather than checking the protection of the file or cellView to decide if it is already checked out.

This function does not distinguish between “checked in and could be checked out” and “checked in and cannot be checked out”. If the function returns an error or an unrecognizable string, then the status is treated like “nm”.

Syntax

```
dm::registerVersionControl <name: string> [-deleteProc <dbExecObject |  
    string> |  
-deleteProcWithOp <dbExecObject | string>] [-primary <bool>]  
    [-checkInProc <dbExecObject | string>]  
    [-checkOutProc <dbExecObject | string>] [-testCheckOutProc <dbExecObject  
    | string>]  
    [-deleteByVersionControl <bool>] [-createProc <dbExecObject | string>]  
    [-libraryOpenProc <dbExecObject | string>]  
    [-libraryCreateProc <dbExecObject | string>] [-libDefsUpdateProc  
    <dbExecObject | string>]  
    [-copyProc <dbExecObject | string>] [-testCopyProc <dbExecObject |  
    string>]  
    [-testCategoryProc <dbExecObject | string>] [-dataExposedProc  
    <dbExecObject | string>]  
    [-checkStatusProc <dbExecObject | string>] [-revertProc <dbExecObject |  
    string>]  
    [-syncToVersionProc <dbExecObject | string>] [-showDMFiles <bool>]
```

Arguments

name

Name of the version control system.

`-primary`

Specifies whether this is the primary version control system for this session, which allows users to associate it with new libraries.

`-checkInProc`

Name of the Tcl proc to call when library elements are to be checked in.

`-checkOutProc`

Name of the Tcl proc to call when library elements need to be editable.

`-testCheckOutProc`

Name of the Tcl proc to call to check whether library elements can be checked out.

`-deleteProc`

Name of the Tcl proc to call before deleting library data. If `deleteByVersionControl` is true, then this procedure performs the deletion. If false, it ensures that the deletion is allowed.

`-deleteProcWithOp`

Alternative delete proc that gets an operation argument.

`-deleteByVersionControl`

State if the integration deletes when `deleteProc` is called.

`-createProc`

The name of the Tcl proc to call before creating library data.

`-libraryOpenProc`

Name of the Tcl proc to call when a library managed by this version control system is opened.

`-libraryCreateProc`

Name of the Tcl proc to call when a new library is created that the user wants to be managed by this version control system.

`-libDefsUpdateProc`

The name of the function to be called with a path to a `lib.defs` file when a library is created. This is used by the version control systems that manages the user's `lib.defs` file even if it is not in a library.

`-copyProc`

Name of the Tcl proc to call when library elements are copied or moved. This procedure copies or moves that it is called for.

`-testCopyProc`

Name of the Tcl proc to call to check if library elements can be copied or moved.

`-testCategoryProc`

The name of the function to be called to check if categories can be checked out.

`-dataExposedProc`

The name of the Tcl proc to call when a view is exposed in the Library Manager. This is used to set the version and the status icon attributes on the given `cellView`.

`-checkStatusProc`

The name of the function to call to check the status. This function provides a callback that the Custom Compiler tool uses to determine if a database that is going to be edited, is checked out. For integrations that do not use this callback, the Custom Compiler tool assumes that a database is checked out if it is writable, and that it needs to be checked out if it is not writable. For integrations that provide this callback, the file protection is not used and this callback is called for each database that is opened for writing.

`-revertProc`

The name of the function to be called to revert a check out of library elements

`-syncToVersionProc`

The name of the function to be called to sync a library element to a specific version

`-showDMFiles`

Specify if show dmFiles is supported by VC system.

Returns

`<dmVersionControlSystem>` : Version Control System created

Examples

```
dm::registerVersionControl verctrl -primary true \  
-checkInProc vctrlsys::checkIn \  
-checkOutProc vctrlsys::checkout \  
-libraryCreateProc verctrl::createNewLib \  
-copyProc vctrlsys::copyData \  
-dataExposedProc vctrlsys::setVersions
```

In this example, a version control system named `verctrl` is registered.

If the opened library is associated with the version control system, and when you try to open a read-only cellView, close the modified design window, copy the library data, or expose the cellViews; the following Tcl procedures are called:

- `vctrlsys::checkIn`.
- `vctrlsys::checkOut`.
- `vctrlsys::copyData`.
- `vctrlsys::setVersions`.

The `vctrlsys::createNewLib` procedure is called when you create a new library and associate it to the version control system.

Note:

Version control integrations are written by the users or come from a third party tool.

dm::removeVersionControl

Description

Specifies that this library should no longer be associated with its version control system. Operations on this library no longer trigger VC callbacks.

Syntax

```
dm::removeVersionControl (-lib <oaLib> | -libName <string>)
```

Arguments

`-lib`

An open library managed by the version control system.

`-libName`

Name of the library to use.

Examples

```
dm::removeVersionControl -libName designLib
```

In this example, the `designLib` library is no longer associated with its version control system.

dm::renameCellCategory

Description

Renames a cell category.

Syntax

```
dm::renameCellCategory <cellCat: dmCellCategory> -name <string>
```

Returns

<dmCellCategory> : Cell category that was updated

Arguments

cellCat

The cell category to rename

-name

The new name of the cell category

Examples

```
# Get cell category object from the specified library
set cat [dm::getCellCategories "StandardCells" -libName PLL]
# Rename category
dm::renameCellCategory $cat -name "stdCells"
```

dm::revert

Description

Cancels the checkout of dmCellView, dmFile, or dmCellCategory.

Syntax

```
dm::revert <container: {<dmCellCategory>} | {<dmCellView>} |
{<dmDMFile>}>
```

Returns

<none>

Arguments

container

Data management collection to revert.

Examples

In the following example, the command cancels the PLL/amp/layout cellview checkout.

```
dm::revert [dm::findCellView layout -cellName amp -libName PLL]
```

dm::saveAsFCRefLib

Description

Saves snpsNDM library as Fusion Compiler (NDM) LibCell library.

Syntax

```
dm::saveAsFCRefLib -lib <oaLib> -path <string>
```

Returns

<none>

Arguments

-lib

Library

-path

Path

dm::saveAsICC2RefLib

Description

Saves snpsNDM library as IC Compiler II (NDM) LibCell library.

Syntax

```
dm::saveAsICC2RefLib -lib <oaLib> -path <string>
```

Returns

<none>

Arguments

-lib

Library

-path

Path

dm::set3DICLCVMap

Specifies the library/cell/view mapping file for a 3DIC Compiler library.

Syntax

```
dm::set3DICLCVMap -library <oaLib> -file <string>
```

Returns

<none>

Arguments

-library

Name of the design library.

-file

Library/cell/view mapping file.

dm::set3DICRefLibs

Sets 3DIC Compiler (NDM) reference libraries in a snpsNDM library.

Syntax

```
dm::set3DICRefLibs (-paths [<string>] | -addLib [<string>] | -addDir  
  <string> | -removeLib [<string>] | -removeAll <bool>) -lib <oaLib>
```

Returns

<none>

Arguments

-lib

Design library.

-paths

Specifies the paths of the specified libraries.

-addLib

Adds the specified libraries to the existing reference libraries list.

-addDir

Adds all libraries from specified directory to the existing reference libraries list.

`-removeLib`

Removes the given libraries from the existing reference libraries list.

`-removeAll`

Removes all reference libraries.

dm::setFCLCVMap

Description

Specifies the library/cell/view mapping file for a Fusion Compiler library.

Syntax

```
dm::setFCLCVMap -library <oaLib> -file <string>
```

Returns

<none>

Arguments

`-library`

Name of the design library.

`-file`

Library/cell/view mapping file.

dm::setFCRefLibs

Description

Sets Fusion Compiler (NDM) reference libraries in a snpsNDM library.

Syntax

```
dm::setFCRefLibs (-paths [<string>] | -addLib [<string>] | -addDir  
  <string> | -removeLib [<string>] | -removeAll <bool>) -lib <oaLib>
```

Returns

<none>

Arguments

`-lib`

Design library.

`-paths`

Specifies the paths of the specified libraries.

`-addLib`

Adds the specified libraries to the existing reference libraries list.

`-addDir`

Adds all libraries from specified directory to the existing reference libraries list.

`-removeLib`

Removes the given libraries from the existing reference libraries list.

`-removeAll`

Removes all reference libraries.

dm::setFollowers

Description

Uses DMFile names to define a set of Followers for the supplied Leader. Note that this command does not operate on a specific DMFile, instead it specifies a pattern that can be applied to any given cellView.

If the append option is set to TRUE and an existing Leader relationship has already been defined for this Leader name, the supplied set of Followers is appended to the existing list. Duplicate entries are silently ignored. If a Leader relationship does not already exist, it is created.

If the append option is set to FALSE and a previous Leader relationship exists, it is overwritten. Leader and follower names provide a template that is used on all cellViews when cells or cellViews are copied by dm::copy commands. They determine which of the DMFiles in a cellView are copied so that scratch and temporary files are left behind.

Syntax

```
dm::setFollowers -leader <string> -followers [<string>] [-append <bool>]
```

Arguments

`-leader <string>`

The DMFile name for which followers are being defined.

`-followers [<string>]`

A list of DMFile names that are followers of the leader.

`-append <bool>`

If FALSE, any previous definition of the supplied leader is overwritten; otherwise, the collection of followers is appended to the existing definition.

Default: false.

dm::setICC2LCVMap

Description

Specifies the library/cell/view mapping file for a IC Compiler II library.

Syntax

```
dm::setICC2LCVMap -library <oaLib> -file <string>
```

Returns

<none>

Arguments

`-library`

Name of the design library.

`-file`

Library/cell/view mapping file.

dm::setICC2RefLibs

Description

Sets the IC Compiler II (NDM) reference libraries in a snpsNDM library.

Syntax

```
dm::setICC2RefLibs (-paths [<string>] | -addLib [<string>] | -addDir  
<string> | -removeLib [<string>] | -removeAll <bool>) -lib <oaLib>
```

Returns

<none>

Arguments

`-lib`

Design library.

`-paths`

Specifies the paths of the specified libraries.

`-addLib`

Adds the specified libraries to the existing reference libraries list.

`-addDir`

Adds all libraries from specified directory to the existing reference libraries list.

`-removeLib`

Removes the specified libraries from the existing reference libraries list.

`-removeAll`

Removes all reference libraries.

dm::setICCTechSync

Description

Reconfigures the specified IC Compiler library library to either synchronize technology data or to use separate and independent technology data.

Syntax

```
dm::setICCTechSync <lib: oaLib> [-layerMap <string>] [-viaMap <string>]
```

Returns

<none>

Arguments

`lib`

Library to modify.

`-layerMap`

Specifies a layer mapping file. Only valid when synchronization is disabled.

Default: No layer mapping file

`-viaMap`

Specifies a via mapping file. Only valid when synchronization is disabled.

Default: No via mapping file

dm::setSelected

Description

Sets the selected library objects in the specified Library Manager window.

Syntax

```
dm::setSelected [<objects: {<dmCell>} | {<dmCellCategory>} |  
                {<dmCellView>} |  
                {<oaLib>}> | {<dmDMFile>} | {<oaLib>}>] -window <giWindow>
```

Returns

<none>

Arguments

objects

List of objects of same kind

-window

A Library Manager window

Examples

```
# Select specified library in all instances of Library Manager window
db::foreach window [gi::getWindows \
    -filter {%windowType.name=="dmLibraryManager"}]
{    dm::setSelected [dm::getLibs analogLib] -window $window}

# Select specified cell view in all instances of Library Manager window
db::foreach window [gi::getWindows \
    -filter {%windowType.name=="dmLibraryManager"}]
{    dm::setSelected [dm::getCellViews symbol -cellName nmos4 \
    -libName analogLib] -window $window}
}
```

dm::showAddICC2Library

Description

Opens the **Add Existing ICC2 Library** dialog box.

Syntax

```
dm::showAddICC2Library [-parent <giDialog | giWindow | int>]
```

Returns

<giDialog>: The dialog.

Arguments

`-parent`

parent Library Manager window.

dm::showAddLibrary

Description

Opens the **Add Existing ICC Library** dialog box.

Syntax

```
dm::showAddLibrary [-parent <giDialog | giWindow | int>]
```

Returns

<giDialog>: The dialog.

Arguments

`-parent`

parent Library Manager window.

dm::showCopyCell

Description

Opens the **Copy Cell** dialog box.

Syntax

```
dm::showCopyCell [-parent <giWindow | int>]
```

Returns

The dialog (*giDialog*).

Arguments

`-parent` *giWindow* | <int>

parent Library Manager window.

Default: Active window.

dm::showCopyCellProperties

Description

Displays the **Copy Cell Properties** dialog box.

Syntax

```
dm::showCopyCellProperties [-parent <giWindow | int>]
```

Returns

<none>

Arguments

-parent

Parent window.

dm::showCopyCellView

Description

Opens the **Copy CellView** dialog box.

Syntax

```
dm::showCopyCellView [-parent <giWindow | int>]
```

Returns

The dialog (*giDialog*).

Arguments

-parent

parent Library Manager window.

Default: Active window.

dm::showCopyLibrary

Description

Opens the **Copy Library** dialog box.

Syntax

```
dm::showCopyLibrary [-parent <giWindow | int>]
```

Returns

The dialog (*giDialog*).

Arguments

`-parent`

parent Library Manager window.

Default: Active window.

dm::showCopyUnique

Description

Shows the Copy Unique window.

Syntax

```
dm::showCopyUnique [-parent <giWindow | int>]
```

Returns

<giDialog> : The dialog

Arguments

`-parent`

parent widget

dm::showDeleteViewsByName

Description

Opens the **Delete Views by Name** dialog box.

Syntax

```
dm::showDeleteViewsByName [-parent <giWindow | int>]
```

Returns

<giDialog>: The dialog.

Arguments

`-parent`

parent Library Manager window.

dm::showEditLibraryCompression

Description

Displays the **Edit Library Compression** dialog box.

Syntax

```
dm::showEditLibraryCompression [-parent <giWindow | int>]
```

Returns

```
<giDialog> : The dialog
```

Arguments

```
-parent
```

Parent window.

dm::showEditProperties

Description

Displays the **Edit Properties** dialog box.

Syntax

```
dm::showEditProperties [-parent <giWindow | int>]
```

Returns

```
<none>
```

Arguments

```
-parent
```

Parent window.

dm::showLibraryManager

Description

Opens the Library Manager. If a Library Manager is already open, the command displays it on top of other windows and dialog boxes.

Syntax

```
dm::showLibraryManager
```

dm::showMoveCell

Description

Opens the **Move Cell** dialog box.

Syntax

```
dm::showMoveCell [-parent <giWindow | int>]
```

Returns

The dialog (*giDialog*).

Errors

None.

Arguments

`-parent`

parent Library Manager window.

Default: Active window.

dm::showMoveCellView

Description

Opens the **Move CellView** dialog box.

Syntax

```
dm::showMoveCellView [-parent <giWindow | int>]
```

Returns

The dialog (*giDialog*).

Arguments

`-parent`

parent Library Manager window.

Default: Active window.

dm::showMoveLibrary

Description

Opens the **Move Library** dialog box. The parent should be a Library Manager window, and the form is initialized with the library selected in that parent.

Syntax

```
dm::showMoveLibrary [-parent <giWindow | int>]
```

Returns

<giDialog> : The **Move Library** dialog box

Arguments

-parent

parent Library Manager window.

Default: Active window.

dm::showNewCell

Description

Opens the **New Cell** dialog box.

Syntax

```
dm::showNewCell [-parent <giWindow | int>]
```

Returns

The dialog (*giDialog*).

Arguments

-parent

parent Library Manager window.

Default: Active window.

dm::showNewCellView

Description

Opens the **New CellView** dialog box.

Syntax

```
dm::showNewCellView [-parent <giDialog | giWindow | int>]
```

Returns

The dialog (*giDialog*)

Arguments

`-parent`

Parent Library Manager window.

Default: Active window

dm::showNewLibrary

Description

Opens the **New Library** dialog box.

Syntax

```
dm::showNewLibrary [-parent <giDialog | giWindow | int>] [-libName  
  <string>] [-dmType <enum>]
```

Returns

The dialog (*giDialog*).

Arguments

`-parent`

parent Library Manager window.

Default: Active window.

`-libName`

Library name to be initialized

`-dmType`

DM type specifier: {any | filesys | icc | icc2}.

Provides the initial value for the type of library to be created.

dm::showRenameCellCategory

Description

Launches the Rename Cell Category dialog.

Syntax

```
dm::showRenameCellCategory [-parent <giWindow | int>] -cellCategory  
    <dmCellCategory>
```

Returns

The dialog (*giDialog*).

Errors

cellCategory not supplied.

Arguments

-parent

parent Library Manager window.

Default: Active window.

-cellCategory *dmCellCategory*

The cell category to rename.

dm::syncToVersion

Description

Synchronizes current (latest) version to specified version.

Syntax

```
dm::syncToVersion <container: {<dmCellCategory>} |  
{<dmCellView>} | {<dmDMFile>}> [-to <string>]
```

Returns

<none>

Arguments

container

DM collection to sync

-to

Version of container to sync to

dm::uncompress

Syntax

Uncompresses a specified library.

This command ensures the safe use of the `oazip -decompress` command, by checking for the lock files in library and ensuring that there are no designs used by other processes, before starting the compression process.

```
dm::uncompress <library: dmLibName | oaLib>
```

Returns

none

Arguments

library

A library name or the library to uncompress.

Errors

- dmclllegalLibName.
- dmcReadOnlySession.
- dmcReadOnlyLibrary
- oacLibAccessDenied
- oaOSError.

dm::updateLib

Description

Updates the IC Compiler II library.

Syntax

```
dm::updateLib <lib: oaLib>
```

Returns

<none>

Arguments

lib
IC Compiler II library.

Enumerations

Table 79 Enumerations

| Name | Values | Description |
|--------------------|---|---------------------|
| enCompressionLevel | 1-9 | Compression levels. |
| enDMLockStatus | notLocked
lockedByCurrentProcess
lockedByForeignProcess | The lock status. |

Objects

This section covers the following:

- [dmCell](#)
- [dmCellCategory](#)
- [dmCellView](#)
- [dmContainerPair](#)
- [dmDMFile](#)

dmCell

A named container, in a library, that can hold files and dmCellViews. The dmCell corresponds to a specific oaCell object but is valid as long as the library that the dmCell it belongs to remains open. The getAccess requirements of an oaCell are handled automatically by this dm object.

Table 80 Attributes

| Name | Type | Access | Description |
|---------|--------|-----------|--------------------------------|
| name | string | Read-only | The name of the Cell |
| libName | string | Read-only | The name of the Cell's library |

Table 80 *Attributes (Continued)*

| Name | Type | Access | Description |
|-------|--------|------------|---|
| shown | bool | read/write | Shows or hides cells for the current session only. The setting applies also to read-only libraries. This attribute works independently of the Hidden category setting. Setting the attribute does not refresh the Library/Cell/View lists in the Library Manager. |
| path | string | Read-only | The absolute path to the Cell directory. This is the location on the disk where the Cell exists if the cell is created. |

dmCellCategory

You can group cells in libraries into categories to simplify the task of finding a particular cell or of browsing the cells in a library. Each category is represented by a dmCellCategory. You can create them and find out the set of cells in a category. Categories can contain other categories. Any given cell and any given category might belong to multiple categories.

Table 81 *Attributes*

| Name | Type | Access | Description |
|------------|------------------|------------|--|
| name | string | Read/Write | The name of the cell category. |
| lib | oaLib | Read-only | The library where the cell category is defined. |
| cells | {dmCell} | Read-only | Collection of oaCells in the cell category. |
| categories | {dmCellCategory} | Read-only | Collection of cell categories that are hierarchically contained in this cell category. |

dmCellView

The dmCellView is a container in a library that holds files and is identified by the combination of its cell and view names. It corresponds to an oaCellView but it stays valid as long as its library remains open and its dmCell exists.

Table 82 *Attributes*

| Name | Type | Access | Description |
|----------|---------------|-----------|---|
| name | string | Read-only | The name of the view that this cellView belongs to. |
| cellName | string | Read-only | The name of the cell that this cellView belongs to. |
| cell | <i>dmCell</i> | Read-only | The cellView's cell. |
| libName | string | Read-only | The name of the library that this cellView belongs to. |
| path | string | Read-only | The absolute path to the cellView directory. This is the location on the disk where the cellView exists if the cellView is created. |

dmContainerPair

Represents a source to destination mapping for a library, cell or cellView being copied/moved. Returned as part of the event data that is available to callers that use `db::createCallback` to listen to copy and move events.

Table 83 *Attributes*

| Name | Type | Access | Description |
|--------------|--------|-----------|--|
| srcLibName | string | Read-only | The name of the source library. |
| srcCellName | string | Read-only | The name of the source cell. An empty (""), means that the dmContainer being operated on is a library. |
| srcViewName | string | Read-only | The name of the source view. An empty (""), means that the dmContainer being operated on is a library or cell. |
| destLibName | string | Read-only | The name of the destination library. |
| destCellName | string | Read-only | The name of the destination cell. This attribute is empty if the corresponding srcCellName attribute is empty. |

Table 83 *Attributes (Continued)*

| Name | Type | Access | Description |
|--------------|--------|-----------|--|
| destViewName | string | Read-only | The name of the destination view. This attribute is empty if the corresponding srcViewName attribute is empty. |

dmDMFile

Used to reference a file stored in a library. The dmDMFile corresponds to an oaDMFile but is valid as long as its library is open and the container that the dmDMFile belongs to exists on disk. Represents a place in a library that can hold an actual file. The file might, or might not actually exist on the disk. When you want to place a file in a library, you first create its dmDMFile and then use that to create the file on the disk. NOTE: Destroying a dmDMFile with db::destroy does not destroy the underlying file on disk, it only destroys the in-memory reference to the file that is embodied by the dmDMFile. To destroy the actual file in the library, use the oa::destroy command.

Table 84 *Attributes*

| Name | Type | Access | Description |
|------------|--|------------|---|
| name | string | Read-only | The name of the DM File. |
| container | string <i>dmCell</i> <i>dmCellView</i> | Read-only | The DM File's container which can be a library, cell or cellView. Determine the type of the container using the db::getType command. If that returns 'string' then the dmDMFile resides directly in a library and the string is the library name. |
| path | <string> | Read-only | The path to the DM File. This is the location on the disk where the file exists if it has been created. |
| primary | bool | Read-only | Represents whether the DM file is the primary file of a cellView. |
| version | <string> | Read/write | A version string annotated by the version control system. |
| statusicon | <string> | Read/write | Incremental path to an icon supplied by the version control system. |

Table 84 *Attributes (Continued)*

| Name | Type | Access | Description |
|------------|----------------------|-----------|---|
| lockStatus | <enDMLockStatusEnum> | Read-only | Represents the locked status of the file. |

Preferences

[Table 85](#) lists Custom Compiler dm namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 85 *Data Management Preferences*

| Name | Type | Scope | Default | Description |
|-------------------------------------|---------------------|-------|---------|--|
| dmAddICC2RefLibs | bool | user | true | Default for the Add ICC2 Reference Libraries option. Valid values: true, false |
| dmAddICCRefLibs | bool | user | true | Default for the Add ICC Reference Libraries option. Valid values: true, false |
| dmAddLibDir | string | user | ./ | Default directory from where libraries are added/opened. |
| dmAddSamplesLibs | bool | user | true | Creates ./lib.defs including samples. Valid values: true, false |
| dmAllowedCountOfBackgroundImporters | unsigned int | user | 10 | Allowed count of running background importers. |
| dmAllowedSizeOfFilesGB | float | user | 5 | Allowed total size (GB) of importing designs files. |
| dmCellPattern | string | user | | Applies the rename pattern to all cells, before resolution of conflicts |
| dmCheckInCell | string | user | auto | |
| dmCheckInInteractive | VersionControlQuery | user | query | Valid values: always, query, never |
| dmCheckOutCell | string | user | auto | |
| dmCheckOutInteractive | VersionControlQuery | user | query | Valid values: always, query, never |

Table 85 Data Management Preferences (Continued)

| Name | Type | Scope | Default | Description |
|----------------------------------|------------------|-------|-------------|--|
| dmConflictResolution | enResolutionType | user | rename | Specifies whether renaming or cell selection is used to resolve name conflicts. |
| dmCopyCellViewCellData | bool | user | false | Copy cell data.
Valid values: true, false |
| dmCopyHierRefs | bool | user | false | Copy references.
Valid values: true, false |
| dmCopyUniqueDestination | string | user | | Destination library for the Copy Unique operation. |
| dmCopyUniqueOverwrite | bool | user | true | Overwrites existing cells or skips Copy Unique operation if destination conflict exists.
Valid values: true, false |
| dmCopyUniquePattern | string | user | %C_<suffix> | Names of the newly created cells from the Copy Unique operation. |
| dmDefaultView | string | user | | The default cellview that is opened if you do not specify a view name. |
| dmDisableLibraryLocking | bool | user | false | Disables library locking.
Caution: Disabling library locking permits design data to be overwritten.
Valid values: true, false |
| dmDeleteRemoveFromVersionControl | enDeleteVCOption | user | local | Specifies how version control systems are handled when deleting a cell or cellView in the Library Manager. This preference does not affect the <code>db::destroy</code> command. |
| dmEnableLibraryCompression | bool | user | false | Determines if libraries are compressed on creation and on editing compression.
Valid values: true, false |
| dmEqualColumnWidths | bool | user | false | Controls if the LCV Selector defaults to equal width columns.
Valid values: true, false |
| dmExcludeLibs | string | user | | Exclude libraries. |
| dmExcludeViews | bool | user | false | Exclude views.
Valid values: true, false |

Table 85 Data Management Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------|--------|-------|---------|---|
| dmExcludeViewTypes | string | user | false | Decides whether the view type names are included or excluded.
Valid values: true, false |
| dmHideViews | bool | user | false | Displays (false) or hides (true) the Views pane in the Library Manager. Valid values: true, false |
| dmICC2DBUPerUU | string | user | | Resolution to be used in the Custom Compiler (OpenAccess) technology database. |
| dmICC2LayerMap | string | user | | Default IC Compiler II layer map file. |
| dmICC2LibDir | string | user | ./ | Location where the IC Compiler II library is created. Instead of storing OA cache information in <code>synopsys_custom</code> , you can use this preference to specify a different directory. |
| dmICC2LibraryPath | string | user | | Path to the IC Compiler II library. |
| dmICC2MapAbstractToLayout | bool | user | false | Convert abstract views to layout when copying from IC Compiler II to fileSys library
Valid values: true, false |
| dmICC2OAIncrTechFile | string | user | | Partial tech file that will be used to create an incremental technology for new added ICC2 libraries |
| dmICC2OALayerMap | string | user | | Location of the layer map file that contains newly added OpenAccess layers. |
| dmICC2OALibCellViewMap | string | user | | Path to l/c/v mapping file to be used in IC Compiler II translation. |
| dmICC2OATechLib | string | user | | Library name of the referenced technology for the added OA data. |

Table 85 Data Management Preferences (Continued)

| Name | Type | Scope | Default | Description |
|------------------------------|--------|-------|---------|---|
| dmICC2RecognitionColorLayers | string | user | | Lists recognition layers that require non-OA color models. If an IC Compiler II library uses recognition layers, you must set this preference before you add the library on Custom Compiler. |
| dmICC2ScaleFactor | string | user | | Sets the design scale factor when you create IC Compiler II libraries on Custom Compiler.
Important: Only one scale factor may be set for all IC Compiler II libraries in a session. |
| dmICC2SearchPath | string | user | | Search list in IC Compiler II to reference library. |
| dmICC2SeparateOATech | bool | user | false | Specifies whether or not to use a separate OA technology for new ICC libraries.
Valid values: true, false |
| dmICC2TechDisplayHandling | string | user | merge | Determines what to do with the display data (<code>display.tcl</code>) when creating Custom Compiler libraries that reference IC Compiler II libraries: <ul style="list-style-type: none"> <code>merge</code>: Merges the display data from the two libraries. <code>overwrite</code>: Overwrites the existing display data with the data from the IC Compiler II library. <code>ignore</code>: Keep the existing display data and ignore the data from the IC Compiler II library. Valid values: merge, overwrite, ignore |
| dmICC2TechFile | string | user | | Default IC Compiler II technology file. |
| dmICC2TechLib | string | user | | Default IC Compiler II technology library. |
| dmICC2ViaMap | string | user | | Default IC Compiler II via map file. |

Table 85 *Data Management Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-----------------------------------|--------|-------|-------------|---|
| dmICCBusNaming | string | user | [%d] | Default IC Compiler bus naming style. |
| dmICCLayerMapFile | string | user | | Default IC Compiler layer mapping file. |
| dmICCTechFile | string | user | | Default IC Compiler technology file. |
| dmICCViaMapFile | string | user | | Default IC Compiler via mapping file. |
| dmImportCellViewsInBackground | bool | user | false | Imports all cell views from the digital design database when adding a digital library in the Custom Compiler environment. Valid values: true, false |
| dmIncludeViews | string | user | * | Include views. |
| dmIncludeViewTypes | bool | user | * | List of view type names to include or to exclude depending on the Exclude View Types check box . |
| dmKeepFilterTextOnSelectionChange | bool | user | false | Retains the library, cell, and view filters you entered. Valid values: true, false |
| dmLibDefEditorUseAbsolutePaths | bool | user | false | Determines whether to use absolute or relative path. |
| dmLibDir | string | user | <file path> | Default directory where libraries are created, copied, and moved. |
| dmLibraryCompressionLevel | int | user | 1 | Defines the compression level of library. Supported values are 1-9. |
| dmMaintainCategories | bool | user | true | Controls category handling for copy and move cells. Valid values: true, false |
| dmManageNewLibrary | bool | user | false | Controls whether new libraries should be VC managed by default. Valid values: true, false |
| dmManageViews | bool | user | false | Whether views are listed in the Cell Conflicts dialog. Valid values: true, false |

Table 85 Data Management Preferences (Continued)

| Name | Type | Scope | Default | Description |
|----------------------------|--------|-------|---------|---|
| dmMarkSavedDesigns | bool | user | false | Enables marking designs saved by Custom Compiler
Valid values: true, false |
| dmNewLibEnablePCellCache | bool | user | false | Retains the Enable PCell Caching selection in the New Library dialog box.
Valid values: true, false |
| dmNewLibEnableParamOverlay | bool | user | false | Retains the Enable Parameter Overlay selection in the New Library dialog box.
Valid values: true, false |
| dmNewLibTechFile | string | user | | Technology file to use when creating a library. |
| dmNewLibTechLib | string | user | | Technology library to use when creating a library. |
| dmNewLibTechType | string | user | none | Technology type to use when creating a library. |
| dmNewLibType | string | user | filesys | DM system to use when creating a library. |
| dmNewLibVCArguments | string | user | | Default arguments for VC systems. |
| dmOATechSource | string | user | | Default OpenAccess technology source for ICC libraries. |
| dmOpenOnCopy | bool | user | false | Controls if the new CellView is automatically opened in its appropriate editor when copied.
Valid values: true, false |
| dmOpenOnCreate | bool | user | true | Controls if the new cellview is automatically opened in its appropriate editor when created.
Valid values: true, false |
| dmOpenOnMove | bool | user | false | Controls if the new CellView is automatically opened in its appropriate editor when moved.
Valid values: true, false |

Table 85 *Data Management Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-----------------------------|------------------------|-------|--|--|
| dmPreviewPosition | previewPosition | user | right | Specifies the location of the Preview pane in the Library Manager.
Valid values: right, bottom |
| dmPropertyIgnoredProperties | string | user | techLibName
dmVersionControl
cdfData | Space-delimited string of property names to ignore. |
| dmPropertyUpdateMethod | dmPropertyUpdateMethod | user | overwrite | Overwrites or merges properties during copy and save-as operations.
Valid values: overwrite, merge |
| dmRemoveDigitalLibs | bool | user | false | Deletes the specified digital library and its cache directory from disk.
Valid values: true, false |
| dmRenameAllCells | bool | user | false | Rename all Cells.
Valid values: true, false |
| dmSearchMutualReferences | bool | user | true | Controls search on Find and Update References dialog. If the value is true only mutual references are looked when searching in multiple libraries. If the value is false, mutual references are also included in results.
Valid values: true, false |
| dmSelectOverwrite | bool | user | true | Overwrites selected cells or skips Copy Cell operation if destination conflict exists. |
| dmShowCellCategories | bool | user | true | Shows cell categories.
Valid values: true, false |
| dmShowCellsInParentCategory | bool | user | true | Displays cells from subcategory in parent cell category.
Valid values: true, false |
| dmShowCellsInParentLibrary | bool | user | true | Decides whether cells from combined libraries are shown under top library or not.
Valid values: true, false. |
| dmShowFileBrowser | bool | user | false | Displays DMFiles in the Library Manager.
Valid values: true, false |

Table 85 *Data Management Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|------------------------|------|-------|---------|--|
| dmShowHiddenItems | bool | user | false | Shows or hides items that are in the Hidden category.
Valid values: true, false |
| dmShowSelectedInFilter | bool | user | false | Displays selected objects in the Library Manager filter boxes.
Valid values: true, false |
| dmShowVersions | bool | user | true | Displays version information.
Valid values: true, false |
| dmSkipBackupFiles | bool | user | false | Controls if the backup files in views are skipped while copying references. |
| dmSkipPcells | bool | user | false | Doesn't update references to pcells or in CustomViaDefs
Valid values: true, false |
| dmSkipCopySNPSRefLibs | bool | user | true | Determines whether the instance master references found in the SYNOPSIS_CUSTOM_INST ALL directory are skipped while copying references.
The following libraries are found under the SYNOPSIS_CUSTOM_INST ALL directory at the time of writing: <ul style="list-style-type: none"> • analogLib • basic • parasitics • rfLib • sample • sheet • snpsDefTechLib • snpsDeviceLib • verilogaLib Note:
Those Libraries (Cells or CellViews) are still possible to copy directly. |

Table 85 Data Management Preferences (Continued)

| Name | Type | Scope | Default | Description | | | | | | |
|---------------------------|---|-------|---------|---|------------------|------|-------|------------------------------------|------|---|
| dmUpdateFilesOnSave | bool | user | false | <div><div>Determines when auxiliary files in library directory hierarchy are written (created or updated). False is on close, true is on save.</div><div>When <code>false</code>, update with save and update with close are not mutually exclusive</div><table><tr><th>Design open mode</th><th>Read</th></tr><tr><td>false</td><td>Updates files while closing design</td></tr><tr><td>true</td><td>Skips updating files while closing design</td></tr></table><div>ly.</div><div>Valid values: true, false</div></div> | Design open mode | Read | false | Updates files while closing design | true | Skips updating files while closing design |
| Design open mode | Read | | | | | | | | | |
| false | Updates files while closing design | | | | | | | | | |
| true | Skips updating files while closing design | | | | | | | | | |
| dmUpdateOnlyToExisting | bool | user | false | <div>Updates the reference library only if the cell names exist in the new library. If the names do not exist in the new library, the Update References command skips the update. This preference is associated with the Update only to existing designs option in the Find and Update References dialog box.</div> <div>Valid values: true, false</div> | | | | | | |
| dmUpdateReferenceViewType | string | user | all | <div>Defines default view type for update references dialog.</div> | | | | | | |
| dmUpdateRefs | bool | user | true | <div>Update references.</div> <div>Valid values: true, false</div> | | | | | | |
| dmUpdateScope | string | user | changed | <div>Searches for instances to update.</div> <div>Valid values: changed, destinationLib, writableLibs</div> | | | | | | |
| dmUseShmForCopy | bool | user | true | <div>Controls whether use RAM drive during copy.</div> <div>Valid values: true, false</div> | | | | | | |

Table 85 *Data Management Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|---------------|--------|-------|--|---|
| dmViewTypeMap | string | user | lay* maskLayout
sch* schematic
sym*
schematicSym
bol | Specifies the view name and view type pairs. The view name can contain a wildcard (*). For example, you can specify sch* schematic lay* maskLayout. If the user enters layout1, then the view type is maskLayout. |

9

Display Resource Tcl

This chapter lists commands, enumerations, and objects in the Custom Compiler Tcl dr namespace. You use the dr Tcl to administer display resources. Interactive commands are listed after non-interactive commands in the Commands section.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

The commands covered are:

- [dr::createColor](#)
- [dr::createColorSelector](#)
- [dr::createDisplay](#)
- [dr::createLineStyle](#)
- [dr::createLineStyleSelector](#)
- [dr::createPacket](#)
- [dr::createPacketAlias](#)
- [dr::createStipple](#)
- [dr::createTextPacket](#)
- [dr::deleteColor](#)
- [dr::deleteLineStyle](#)

- [dr::deleteStipple](#)
- [dr::getColors](#)
- [dr::getDisplays](#)
- [dr::getLineStyle](#)
- [dr::getPacketAliases](#)
- [dr::getPackets](#)
- [dr::getStipples](#)
- [dr::getTextPackets](#)
- [dr::loadDisplayResources](#)
- [dr::mergeDisplayResources](#)
- [dr::renameColor](#)
- [dr::renameLineStyle](#)
- [dr::renameStipple](#)
- [dr::saveDisplayResources](#)
- [dr::showDisplayResourceEditor](#)

dr::createColor

Description

Creates a new color in the given display, or redefines an existing color.

Syntax

```
dr::createColor <name: string> -display <drDisplay> -red <int> -green  
  <int> -blue <int> [-blink <bool>]
```

Arguments

<string>

The name of the color to create.

-display [drDisplay](#)

The display object in which to create the color.

-red <int>

The red value of this color, in the range 0 through 255.

`-green <int>`

The green value of this color, in the range 0 through 255.

`-blue <int>`

The blue value of this color, in the range 0 through 255.

`-blink <bool>`

If true, the color blinks.

Default: false.

Examples

In the example, `dr::createColor` defines a new color in the RGB format and adds it to the `drDisplay` object.

```
dr::createColor myColor -display $dp -red 127-green 127 -blue 127
```

dr::createColorSelector

Description

Creates a new `drColorSelector` widget and adds it to the specified parent.

Syntax

```
dr::createColorSelector [{none}: string] [-parent <giAssistant  
| giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown  
<bool>] [-prefName <dbPrefName>] [-toolTip <string>] [-valueChangeProc  
[<variant>]] [-label <string>] [-value <string>] [-showNames <bool>]  
[-display <string>] [-filter <string>] [-iconWidth <int>] [-iconHeight  
<int>]
```

Arguments

{none}

Widget name. (Default: "")

`-parent`

Dialog box, group or other container into which this widget should be placed.

`-enabled`

Initial enabled state. (Default: true)

`-shown`

Initial shown state. (Default: true)

`-prefName`

Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model. (Default: "")

`-toolTip`

Optional widget tool tip. (Default: "")

`-valueChangeProc`

Procedure to execute when the value is changed. Called with the widget object.

`-label`

Widget label, shown in the left column. (Default: "")

`-value`

Initial value. (Default: "")

`-showNames`

Display color names. (Default: false)

`-display`

Display name. (Default: display)

`-filter`

Regular expression to filter colors by name (Default: "")

`-iconWidth`

Icon Width. (Default: 16)

`-iconHeight`

Icon Height. (Default: 16)

Returns

<deColorSelector> : New object.

dr::createDisplay

Description

Creates a new display. Attempting to create a display with the same name as an existing display is an error unless `-force` is true.

Display objects created by this command can be deleted with `db::destroy`. The default display, named "display", cannot be deleted. A display owns all of the packets,

packetAliases, colors, stipples, and lineStyles that are created in it, and those objects are destroyed when their owning display is destroyed.

Syntax

```
dr::createDisplay <name: string> [-maxColors <int>] [-maxStipples <int>]  
[-maxLineStyles <int>] [-force <bool>]
```

Arguments

<string>

The object name, which must be unique at user session scope.

-maxColors <int>

An integer to specify the fixed size of the color table, or 0 if there is no fixed limit.

Default: 0 - means no fixed limit.

-maxStipples <int>

An integer to specify the fixed size of the stipple table, or 0 if there is no fixed limit.

Default: 0 - means no fixed limit.

-maxLineStyles <int>

An integer to specify the fixed size of the linestyle table, or 0 if there is no fixed limit.

Default: 0 - means no fixed limit.

-force <bool>

If set to true, then overwrite an existing display with the specified name if one exists.

Default: false.

Returns

The new display object ([drDisplay](#)).

Errors

- Display name is invalid.
- Display with given name already exists.

Examples

In the example, `dr::createDisplay` creates the new `drDisplay` object called `testDisplay` with 256 colors, stipples and line styles.

```
set dp [dr::createDisplay testDisplay -maxColors 256 \  
      -maxStipples 256 -maxLineStyles 256 -force true ]
```

dr::createLineStyle

Description

Creates a new line style in the specified display or redefines an existing line style.

Syntax

```
dr::createLineStyle <name: string> -display <drDisplay> -width <int>  
      -pattern [<int>]
```

Arguments

<string>

The name of the `lineStyle` to create.

-display `drDisplay`

The display object in which to create the line style.

-width <int>

The line thickness, in the range 1 through 9.

-pattern [<int>]

A list of 0's and 1's to define the pattern used to draw the `lineStyle`.

Examples

In the example, `dr::createLineStyle` defines a new line style and adds it to the `drDisplay` object.

```
dr::createLineStyle myStyle -display $dp -width 1 -pattern \  
      {1 0 0 1 0 1}
```

dr::createLineStyleSelector

Description

Creates a new `drLineStyleSelector` widget and adds it to the specified parent.

Syntax

```
dr::createLineStyleSelector [<{none}: string>] [-parent <giAssistant  
| giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown  
<bool>] [-prefName <dbPrefName>] [-toolTip <string>] [-valueChangeProc  
[<variant>]] [-label <string>] [-value <string>] [-showNames <bool>]  
[-display <string>] [-iconWidth <int>]
```

Arguments

{none}

Widget name.

-parent

Dialog, group or other container into which this widget should be placed.

-enabled

Initial enabled state. (Default: true)

-shown

Initial shown state. (Default: true)

-prefName

Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model.

-toolTip

Optional widget toolTip.

-valueChangeProc

Procedure to execute when the value is changed. Called with the widget object.

-label

Widget label, shown in the left column.

-value

Initial value.

-showNames

Display line style names. (Default: false)

-display

Display name. (Default: display)

`-iconWidth`

Icon Width. (Default: 50)

Returns

`<deLineStyleSelector>` : New object.

dr::createPacket

Description

Creates a new display packet in the specified display. An existing display packet or packet alias with the same name will be silently overwritten. Packets created by this command can be deleted with `db::destroy`.

Syntax

```
dr::createPacket <name: string> -display <drDisplay> -stipple <string>
[-lineStyle <string>] -fill <string> -outline <string> [-fillStyle
<string>]
```

Arguments

`<string>`

The name of the display packet to create.

`-display` `drDisplay`

The display object in which to create the display packet.

`-stipple` `<string>`

The existing stipple name.

`-lineStyle` `<string>`

The existing linestyle name.

Default: No lineStyle

`-fill` `<string>`

The existing fill color name.

`-outline` `<string>`

The existing outline color name.

`-fillStyle` `<string>`

The name of the fill style.

Default: outlineStipple.

Returns

The new display packet ([drPacket](#)).

Errors

- Nonexistent display object.
- Invalid packet name.
- Color, lineStyle or stipple is nonexistent.
- invalid fileStyle name.

Examples

In this example, `dr::createPacket` defines a new packet and adds it to the `drDisplay` object.

```
dr::createPacket myPacket -display $dp -stipple myStipple -fill \
    red -outline yellow -lineStyle myStyle -fillStyle \
    outlineStipple
```

dr::createPacketAlias

Description

Creates a display packet alias in a display and associates the new alias with a display packet by name. A packet with that name may or may not currently exist; if it does not, then the packetAlias will resolve to the default packet. If a packetAlias exists with that name, because a packetAlias cannot resolve to another packetAlias, such a packetAlias would resolve to the default packet. If a packet with the specified name is later created, then the packetAlias will then resolve to that new packet. An existing packet alias with the same name in the display will be silently overwritten. A packetAlias cannot be named 'defaultPacket', since that would overwrite the default packet. PacketAliases created by this command can be deleted with `db::destroy`.

Syntax

```
dr::createPacketAlias <name: string> -packet <string> -display  
    <drDisplay>
```

Arguments

<string>

The name of the display packet alias to create.

-packet <string>

The name of the display packet to which the new display packet alias will point.

`-display drDisplay`

The display object in which to create the packet alias.

Returns

The new display packet alias (`drPacketAlias`).

Errors

- nonexistent display.
- invalid packetAlias name.
- packetAlias name is 'defaultPacket'.

Examples

In this example, `dr::createPacketAlias` creates an alias for the packet called `myPacket`.

```
dr::createPacketAlias myAlias -packet myPacket -display $dp
```

dr::createStipple

Description

Creates a new stipple pattern in the specified display, or redefines an existing stipple pattern.

Syntax

```
dr::createStipple <name: string> -display <drDisplay> -pattern <matrix>
```

Arguments

`<string>`

The name of the new stipple pattern.

`-display drDisplay`

The display object in which to create the new stipple pattern.

`-pattern <matrix>`

The lists of 0's and 1's that define the stipple pattern.

Examples

In this example, `dr::createStipple` defines a stipple pattern and adds it to the `drDisplay` object.

```
dr::createStipple myStipple -display $dp -pattern { {0 0 0 0 0} \
  {0 0 1 0 0} {0 1 1 1 0} {0 0 1 0 0} {0 0 0 0 0} }
```

dr::createTextPacket

Description

Creates a text packet.

Syntax

```
dr::createTextPacket <name: string> -display <drDisplay> -color <string>  
[-backgroundColor <string>] [-fontName <string>] [-height <string>]  
[-bold <string>] [-italic <string>] [-underline <string>]
```

Arguments

name

Name of the text packet.

-display

Display object.

-color

Foreground color.

-backgroundColor

Background color.

Default: inherited

-fontName

Name of font, either euroStyle, fixed, gothic, math, milSpec, roman, script, or swedish.

Default: inherited

-height

Height of text in points.

Default: inherited

-bold

Bold.

Default: inherited

-italic

Italic.

Default: inherited

`-underline`

Underline.

Default: inherited

Returns

`<drTextPacket>`: The new display textPacket.

dr::deleteColor

Description

Deletes a color from a display object unless the color is referenced by any packet in the display.

Syntax

```
dr::deleteColor <name: string> -display <drDisplay>
```

Arguments

`name`

The name of the color to delete.

`-display` `drDisplay`

The display object that contains the color.

Examples

In the example, `dr::deleteColor` deletes the color called `myColor` from the `drDisplay` object.

```
dr::deleteColor myColor -display $dp
```

dr::deleteLineStyle

Description

Deletes a line style from a display unless the line style is referenced by an packet in the display.

Syntax

```
dr::deleteLineStyle <name: string> -display <drDisplay>
```

Arguments

`<string>`

The name of the `lineStyle` to delete.

`-display` `drDisplay`

The display object that contains the line style.

dr::deleteStipple

Description

Deletes a stipple pattern from a display unless the stipple pattern is referenced by any packet in the display.

Syntax

```
dr::deleteStipple <name: string> -display <drDisplay>
```

Arguments

`<string>`

The name of the stipple pattern to delete.

`-display` `drDisplay`

The display object that contains the stipple pattern.

dr::getColors

Description

Returns a collection of the display colors (`drColors`) that satisfy the given patterns and filters.

Syntax

```
dr::getColors [<string>] [-display <drDisplay>] [-filter <string>]
```

Arguments

`<string>`

Color names.

Default: ""

`-display`

Filters the results to include only display colors from the specified display object.
By default, all display objects are searched.

Default: ""

`-filter`

Standard attribute filter.

Default: match all

Returns

The returned collection of the display colors in the specified display object.

dr::getDisplays

Description

Returns a collection of the displays satisfying the given name pattern and filter.

Syntax

```
dr::getDisplays [<name: string>] [-filter <string>]
```

Arguments

`name`

A pattern to match display names.

Default: ""

`-filter`

Standard attribute filter.

Default: match all

Returns

The returned collection of all existing display objects (`{<drDisplay>}`).

Examples

In the example, `dr::getDisplays` returns the `drDisplay` object with the name `testDisplay`.

```
set display_object [db::getNext [dr::getDisplays -filter "%name \\  
== {testDisplay}"]]
```

dr::getLineStylees

Description

Returns a collection of the line styles (drLineStylees) that satisfy the given patterns and filters.

Syntax

```
dr::getLineStylees [<string>] [-display <drDisplay>] [-filter <string>]
```

Arguments

<string>

Line style names.

Default: ""

-display

Filters the results to include only line styles from the specified display object. By default, all display objects are searched.

Default: ""

-filter

Standard attribute filter.

Default: match all

Returns

The returned collection of the line styles in the specified display object.

dr::getPacketAliases

Description

Returns a collection of the packet aliases satisfying the given patterns and filters.

Syntax

```
dr::getPacketAliases [<name: string>] [-display <drDisplay>] [-packet  
  <drPacket>] [-filter <string>]
```

Returns

The collection of display packet aliases in the specified display ({<drPacketAlias>}).

Arguments

<string>

A pattern for display packet alias names.

Default: ""

-display `drDisplay`

Filter the results to include only display packets aliases from the specified display object. By default, all display objects are searched for aliases.

Default: NULL

-packet <string>

Filters the results to include only display packet aliases of the specified packet name. By default, aliases for all display packets are returned.

Default: ""

-filter <string>

Standard attribute filter.

Default: match all

Errors

- Invalid display object.
- Both 'name' and 'packet' are specified.

Examples

The example returns the collection of display packets satisfying the given patterns and filter.

```
set c [dr::getPacketAliases -display $dp -packet \  
    [dr::getPackets myPacket -display $dp]]  
db::foreach d $c {  
    puts [db::getattr name -of $d]  
}
```

dr::getPackets

Description

Returns a collection of the display packets satisfying the given patterns and filter.

Syntax

```
dr::getPackets [<name: string>] [-display <drDisplay>] [-filter <string>]
```

Returns

The collection of display packets in the specified display ({<drPacket>}).

Errors

Invalid display object.

Arguments

name

A pattern for display object names.

Default: *

-display drDisplay

Filters the results to include only display packets from the specified display object. By default, all display objects are searched.

Default: NULL

-filter <string>

Standard attribute filter.

Default: match all

Examples

The example returns the collection of display packets satisfying the given patterns and filter.

```
dr::getPackets myPacket -display $dp
```

dr::getStipples

Description

Returns a collection of the stipple patterns (drStipples) that satisfy the given patterns and filters.

Syntax

```
dr::getStipples [<string>] [-display <drDisplay>] [-filter <string>]
```

Arguments

<string>

Stipple pattern names.

Default: ""

`-display <drDisplay>`

Filters the results to include only stipple patterns from the specified display object. By default, all display objects are searched.

Default: ""

`-filter <string>`

Standard attribute filter.

Default: match all

Returns

The returned collection of the stipple patterns in the specified display object.

dr::getTextPackets

Description

Gets text packets.

Syntax

```
dr::getTextPackets [<name: string>] [-display <drDisplay>] [-filter  
  <string>]
```

Returns

<dbCollection>: Collection of display text packets.

Arguments

name

Name of text packet to get.

Default: *

-display

Display object.

-filter

Standard attribute filter.

Default:.

dr::loadDisplayResources

Description

Opens the specified display resource file and merges it into the existing display resource data in memory. This command reads DRF, DSP, Tcl files. To load a display resource file, you can alternately source the file. The reload option reloads all display resource files in the same manner as during application startup. Specifically, within each path returned by `db::resolve`, any `display.drf`, `display.dsp`, and then `display.tcl` files are loaded (in that order).

Syntax

```
dr::loadDisplayResources [-reload <bool> | -reloadDefault <bool>]  
    [-filePath <string>] [-format <enum>]
```

Arguments

`-filePath <string>`

The full or relative path of the display resource file to load.

Default: ""

`-reload <bool>`

Reloads last-saved display resource data if set to `true`.

Default: `false`

`-reloadDefault <bool>`

Reloads factory-default display resource data if set to `true`.

Default: `false`

`-format`

Format of the display resources file to load: {`auto` | `drf` | `dsp` | `tcl`}

Valid values are `tcl` or `drf` for those formats, or `auto` to select the format based on the file name extension.

Default: `auto`

Examples

The example loads a DRF file into the Custom Compiler environment.

```
dr::loadDisplayResources -filePath dr.drf -format drf
```

dr::mergeDisplayResources

Description

Merges the data in a list of display resource files and saves them to a new display resource file. Optionally loads the merged result into memory.

Syntax

```
dr::mergeDisplayResources -srcFilePaths [<string>] [-srcFormats  
[<string>]] -destFilePath <string> [-load <bool>] [-destFormat <enum>]  
[-force <bool>]
```

Arguments

`-srcFilePaths [<string>]`

A list of full or partial paths to display resource files.

`-destFilePath <string>`

The full or relative path of the display resource file to save.

`-load <bool>`

Load the merged display resource data into memory.

Default: false.

`-destFormat`

Identifies the file format for the output (merged) file: {auto | drf | tcl}.

Valid values are tcl or drf for those formats, or auto to select the format based on the file name extension of destFilePath.

Default: auto

`-force <bool>`

If set to true, overwrite the existing file.

Default: false

`-srcFormats [enDRFileFormat]`

Identifies the source file format(s). If only a single value is specified, it applies to all source files. The length of the list must either be 1 or be equal to the length of the list specified in srcFilePaths.

Default: auto

dr::renameColor

Description

Renames a color unless the color is referenced by any packet in the display.

Syntax

```
dr::renameColor <name: string> -to <string> -display <drDisplay>
```

Arguments

<string>

The name of the color to rename.

-to <string>

The new name for the color.

-display *drDisplay*

The display object that contains the color.

Examples

In the example, `dr::renameColor` renames the color `blue` to the color `myColor` in the `drDisplay` object.

```
dr::renameColor blue -to myColor -display $dp
```

dr::renameLineStyle

Description

Renames a line style, unless the line style is referenced by a packet in the display.

Syntax

```
dr::renameLineStyle <name: string> -to <string> -display <drDisplay>
```

Arguments

<string>

The name of the `lineStyle` to rename.

-to <string>

The new name for the `lineStyle`.

-display *drDisplay*

The display object that contains the line style.

dr::renameStipple

Description

Renames a stipple pattern, unless the stipple pattern is referenced by a packet in the display.

Syntax

```
dr::renameStipple <name: string> -to <string> -display <drDisplay>
```

Arguments

<string>

The name of the stipple pattern to rename.

-to <string>

The new name for the stipple pattern.

-display [drDisplay](#)

The display object that contains the stipple pattern.

dr::saveDisplayResources

Description

Saves in-memory display resource data to an external Tcl script or DRF file.

Syntax

```
dr::saveDisplayResources -filePath <string> [-modifiedOnly <bool>] [-tcl  
<bool>] [-force <bool>]
```

Arguments

-filePath <string>

The full or relative path of the display resource file to save.

-modifiedOnly <bool>

If set to true, only save display resource modifications made after the last reload;
otherwise, save all display resources.

Default: false.

-tcl <bool>

If set to true, save the resources as a Tcl script; otherwise, create a DRF file.

Default: true

`-force <bool>`

If set to true, overwrite the existing file.

Default: false

Examples

The following example saves in-memory display resource data to an external Tcl script.

```
dr::saveDisplayResources -filePath dr.tcl -force 1 -tcl 1
```

The following example saves in-memory display resource data to an external display resources file called `dr.drf`.

```
dr::saveDisplayResources -filePath dr.drf -force 1 -tcl 0
```

dr::showDisplayResourceEditor

Description

Opens the Display Resource Editor.

Syntax

```
dr::showDisplayResourceEditor [-lpp <deTechLPP | lpp> | -packet <drPacket  
| string>] [-parent <giWindow | unsigned int>]
```

Arguments

`-parent`

The window this dialog is associated with.

`-lpp`

LPP to automatically select.

`-packet`

Packet to automatically select.

Returns

The created or existing Display Resource Editor window. ([giWindow](#)).

Enumerations

Table 86 Enumerations

| Name | Values | Description |
|----------------|--------------|--------------------|
| enDRFileFormat | drf tcl auto | Format of DR file. |

Objects

This section covers the following:

- [drColor](#)
- [drDisplay](#)
- [drLineStyle](#)
- [drPacket](#)
- [drPacketAlias](#)
- [drStipple](#)

drColor

Complete specification of a display color.

Table 87 Attributes

| Name | Type | Access | Description |
|---------|-----------|------------|------------------------------------|
| name | string | read/write | Color name. |
| display | drDisplay | read-only | Display associated with the color. |
| red | int | read/write | Red value of color. |
| green | int | read/write | Green value of color. |
| blue | int | read/write | Blue value of color. |
| blink | bool | read/write | Determines if color blinks. |

drDisplay

Container for all display resource objects. The "display" display exists by default and cannot be deleted.

Table 88 Attributes

| Name | Type | Access | Description |
|--------------|--------|------------|---|
| name | string | Read/Write | Display name. |
| maxColors | int | Read-only | Fixed size of color table of display. |
| maxStipples | int | Read-only | Fixed size of stipple table of display. |
| maxLineStyle | int | Read-only | Fixed size of linestyle table of display. |

drLineStyle

Complete specification of a display stipple.

Table 89 Attributes

| Name | Type | Access | Description |
|---------|-----------|------------|---|
| name | string | read/write | Line style name. |
| display | drDisplay | read-only | Display associated with the line style. |
| pattern | {binary} | read/write | List of binary values (0 1). |
| width | int | read/write | Width, in pixels, of line style. |

drPacket

Complete specification of the visual attributes of an object in a given display. It completely defines how objects appear in the display. Renaming a packet to a name that already exists is an error. When a packet is renamed, any drPacketAliases that might refer to the old or new names are not affected.

Table 90 *Attributes*

| Name | Type | Access | Description |
|--------------|------------------|------------|--|
| name | string | Read/Write | packet name. |
| display | <i>drDisplay</i> | Read-only | The owner display object. |
| stipple | string | Read/Write | stipple name. |
| lineStyle | string | Read/Write | linestyle name. |
| fillColor | string | Read/Write | color name for draw fill. |
| outlineColor | string | Read/Write | color name for draw outline. |
| fillStyle | string | Read/Write | Fill style name of packet: outline, solid, X, stipple, outlineStipple. |

drPacketAlias

Reference to a packet by name. It can be redirected to point to any other packet by the user. A drPacketAlias's name cannot be set to 'defaultPacket', and renaming a packetAlias to a name that already exists is an error.

Table 91 *Attributes*

| Name | Type | Access | Description |
|---------|------------------|------------|----------------------------|
| name | string | Read/Write | packetAlias name. |
| display | <i>drDisplay</i> | Read-only | The owning display object. |
| packet | string | Read/Write | packet name. |

drStipple

Complete specification of a display stipple.

Table 92 *Attributes*

| Name | Type | Access | Description |
|---------|-----------|------------|--------------------------------------|
| name | string | read/write | Stipple name. |
| display | drDisplay | read-only | Display associated with the stipple. |

Table 92 *Attributes (Continued)*

| Name | Type | Access | Description |
|---------|-------|------------|-------------------------------|
| pattern | array | read/write | Array of binary values (0 1). |

Preferences

[Table 93](#) lists the Custom Compiler dr namespace general preferences.

Table 93 *Display Resource Preferences*

| Name | Type | Scope | Default | Description |
|---------------------|--------|-------|---------|--|
| drSearchPath | string | user | | The Tcl-list of paths, relative to each directory searched for DR files, within which the <code>display.tcl</code> and <code>display.drf</code> files are located. |
| drSelectorShowNames | bool | user | false | Shows names in the Display Resource Editor.
Valid values: true, false |

10

Design Technology Co-Optimization Tcl

This chapter describes the commands and preferences in the Custom Compiler Tcl dtco namespace. These are enabled from various computation tabs in the overall DTCO graphical user interface. Typically, the DTCO graphical user interface collects various setups and arguments through user's inputs, and packs them all, and passes on to suitable tcl command.

This chapter includes the following sections:

- [Commands](#)
- [Preferences](#)

Commands

The commands covered are:

- [dtco::checkAllTextLabelsInsidePRBoundary](#)
- [dtco::compileAllUDDs](#)
- [dtco::createCompactModel](#)
- [dtco::createInterconnectModel](#)
- [dtco::createIVCV](#)
- [dtco::createParasiticExtractedNetlist](#)
- [dtco::createSplit](#)
- [dtco::createUDD](#)
- [dtco::showSetupAndRunDialog](#)

dtco::checkAllTextLabelsInsidePRBoundary

Description

Check if all text labels inside PRBoundary.

Syntax

```
dtco::checkAllTextLabelsInsidePRBoundary -design <oaDesign>
```

Arguments

-design
Design.

Returns

<none>

dtco::compileAllUDDs

Description

Compiles all UDDs in the library.

Syntax

```
dtco::compileAllUDDs -libName <string>
```

Arguments

-libName
Name of library.

Returns

<none>

dtco::createCompactModel

Description

Creates a compact spice model using the IV, CV data for the transistor geometry and its corresponding material description.

Syntax

```
dtco::createCompactModel -runRootDir <string> -libName <string>  
-projectName <string> -templateDirectory <string> -pathFile <string>  
[-tcadQueue <string>] -saveInLibrary <bool> [-cellName <string>]
```

Arguments

`-runRootDir`

Name of Run Root Directory.

`-libName`

Library with dtco parameters.

`-projectName`

Name of project.

`-templateDirectory`

TCAD Compact Model template directory.

`-pathFile`

File containing locations of IVCV data.

`-tcadQueue`

Valid values are local:default or local:priority or from
\$env(SWB_SITE_SETTINGS_DIR)/gqueues.dat. (Default: local:default)

`-saveInLibrary`

Save interconnect model in the library.

`-cellName`

Name of cell for storing model. (Default:)

Returns

<none>

dtco::createInterconnectModel

Description

Creating or updating a nxtgrd file which in turn is used in Fusion Compiler for parasitic extraction during the implementation flow.

Syntax

```
dtco::createInterconnectModel -runRootDir <string> -libName <string>  
-projectName <string> -templateDirectory <string> -flowFile <string>  
-headerFile <string> -materialsFile <string> [-tcadQueue <string>]  
-saveInLibrary <bool> [-cellName <string>]
```

Arguments

`-runRootDir`

Name of Run Root Directory.

`-libName`

Library with dtco parameters.

`-projectName`

Name of project.

`-templateDirectory`

TCAD interconnect characterization template directory.

`-flowFile`

TCAD interconnect characterization flow file.

`-headerFile`

TCAD interconnect characterization header file.

`-materialsFile`

TCAD interconnect characterization materials file.

`-tcadQueue`

Valid values are local:default or local:priority or from
env(SWB_SITE_SETTINGS_DIR)/gqueues.dat. (Default: local:default)

`-saveInLibrary`

Save interconnect model in the library.

`-cellName`

Name of cell for storing model. (Default:)

Returns

<none>

dtco::createIVCV

Description

Generates the IV, CV data by launching Sentaurus QTX using the specified parameters and the template directory which contains the device geometry information.

Syntax

```
dtco::createIVCV -runRootDir <string> -libName <string> -projectName  
  <string> -templateDirectory <string> -techType <string> -threads <int>  
  [-tcadQueue <string>]
```

Arguments

-runRootDir

Name of Run Root Directory.

-libName

Library with dtco parameters.

-projectName

Name of project.

-templateDirectory

TCAD IVCV template directory.

-techType

Technology Type.

-threads

Number of threads.

-tcadQueue

Valid values are local:default or local:priority or from
\$env(SWB_SITE_SETTINGS_DIR)/gqueues.dat. (Default: local:default)

Returns

<none>

dtco::createParasiticExtractedNetlist

Description

Create the post layout extraction netlist so it can be used for library characterization.

Syntax

```
dtco::createParasiticExtractedNetlist -runRootDir <string>  
  -exportStream <bool> [-gdsLayerMapFile <string>] [-gdsObjectMapFile  
  <string>] [-layoutLCV <string>] [-extractionCellName <string>]  
  [-gdsFileName <string>] -exportNetlist <bool> [-schematicLCV  
  <string>] [-viewSearchList <string>] [-viewStopList <string>]  
  [-preExtractionNetlist <string>] -templateDirectory <string>
```

```
-materialsFile <string> -processFile <string> -layerConfigFile <string>  
-raphaelConfigFile <string> -layoutLayersFile <string> [-tcadQueue  
<string>] -saveInLibrary <bool> [-netlistLibName <string>]
```

Arguments

`-runRootDir`

Name of Run Root Directory.

`-exportStream`

Creates GDS from layout.

`-gdsLayerMapFile`

GDS layer map file. (Default: false)

`-gdsObjectMapFile`

GDS object map file. (Default: false)

`-layoutLCV`

Layout LCV. (Default:)

`-extractionCellName`

Name of extraction cell. (Default:)

`-gdsFileName`

GDS file. (Default:)

`-exportNetlist`

Creates pre-extraction netlist from schematic.

`-schematicLCV`

Schematic LCV. (Default:)

`-viewSearchList`

Netlisting view search list. (Default: hspice hspiceD cmos.sch cmos_sch
schematic veriloga)

`-viewStopList`

Netlisting view stop list. (Default: hspice hspiceD veriloga)

`-preExtractionNetlist`

Pre-Extraction netlist file. (Default:)

`-templateDirectory`

TCAD parasitic extraction template directory.

`-materialsFile`
TCAD parasitic extraction materials file.

`-processFile`
TCAD parasitic extraction process file.

`-layerConfigFile`
TCAD parasitic extraction layer config file.

`-raphaelConfigFile`
TCAD parasitic extraction raphael config file.

`-layoutLayersFile`
TCAD parasitic extraction layout layers file.

`-tcadQueue`
Valid values are local:default or local:priority or from
env(SWB_SITE_SETTINGS_DIR)/gqueues.dat. (Default: local:default)

`-saveInLibrary`
Save netlist in the library.

`-netlistLibName`
Name of netlist library. (Default:)

Returns

<none>

dtco::createSplit

Description

Create split does a copy of Source Library to Target Library name with new parameter values and recompile all UDDs in the Target Library.

Syntax

```
dtco::createSplit -sourceLibName <string> -targetLibName <string>  
-targetLibDir <string> [-excludeViews <string>] [-overwrite <bool>]  
-params [<map>] [-postCallbackProc <string>]
```

Arguments

`-sourceLibName`
Name of Source Library.

`-targetLibName`
Name of Target Library.

`-targetLibDir`
Directory of Target Library.

`-excludeViews`
Specifies the set of cellviews associated with each cell that should not be copied. (Default:)

`-overwrite`
Overwrite Target Library if it already exists. (Default: false)

`-params`
List of split parameter {name value} pairs.

`-postCallbackProc`
Post Callback Proc Name. (Default:)

Returns

<none>

dtco::createUDD

Description

Creates UDDs from layouts.

Syntax

```
dtco::createUDD -libName <string> -cellName <string> -viewName <string>  
-file <string> [-createMarkers <bool>]
```

Arguments

`-libName`
Name of library.

`-cellName`
Name of cell.

`-viewName`
Name of view.

`-file`

Name of UDD Grid Info file.

`-createMarkers`

Create Markers on design. (Default: false)

Returns

<none>

dtco::showSetupAndRunDialog

Description

Shows DTCO Setup and Run dialog box.

Syntax

`dtco::showSetupAndRunDialog`

Returns

<giDialog> : Shows DTCO Setup and Run dialog box.

Preferences

[Table 94](#) lists Custom Compiler dtco namespace general preferences.

Table 94 Design Technology Co-Optimization Preferences

| Name | Type | Scope | Default | Description |
|---------------------------|--------|---------|--------------|--|
| dtcoCompactCategory | string | context | compactModel | Compact Model category. |
| dtcoCompactImportCellName | string | context | | Cell name to save in library. |
| dtcoCompactImportResults | bool | context | true | The option to create cell view of the final spice model in the DTCO Library by checking this option and providing the Cell Name. |
| dtcoCompactLoadLib | string | context | | The DTCO parameters for compact model generation are picked from the specified DTCO Library. |

Table 94 *Design Technology Co-Optimization Preferences (Continued)*

| Name | Type | Scope | Default | Description |
|-------------------------------------|-----------------|---------|---------------|---|
| dtcoCompactPathFile | string | context | | This field will be enabled if Compact Model is checked and IV CV is not checked. User needs to specify path file containing directory information for IV CV run. |
| dtcoCompactProjectName | string | context | | The final spice file of Compact Model run will be stored with the Project Name directory in the Run Root Directory. |
| dtcoCompactRun | bool | context | true | Checkbox to select Compact Model run. If not selected, then only IV CV curve generation is done. |
| dtcoCompactRunMode | dtcoJobTypeEnum | context | batch | The GUI mode will launch the Sentaurus Workbench GUI and user needs to manually run the Compact Models. |
| dtcoCompactTcadQueue | string | context | local:default | Transistor modeling TcadQueue default, priority. |
| dtcoCompactTemplateDirectory | string | context | | The template directory contains the files needed for Sentaurus TCAD tool to run compact model generation. |
| dtcoCreateSplitExcludeViews | string | user | | During a split operation design data will be copied from the source library to the target library. View names specified on the Exclude Views input will not be copied during a split. |
| dtcoCreateSplitPostCallbackProcName | string | user | | Create Split post callback proc name. |
| dtcoCreateSplitSourceLibraryName | string | user | | The source library name is the library with DTCO parameters which defines the baseline for the experiment. |
| dtcoCreateSplitTargetLibraryName | string | user | | The target library name is the name of the library which will contain the split design data. |
| dtcoDialogMode | bool | user | false | Dialog Mode of DTCO |
| dtcoFCCustomScriptFile | string | user | <file path> | Path of Custom Script File. |

Table 94 Design Technology Co-Optimization Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------------|--------|---------|-------------|--|
| dtcoFCDesignsList | string | user | | List of designs to be executed for QOR. |
| dtcoFCParasiticFilesList | string | user | | List of nxtgrd/TLU+ files are the collaterals from the Interconnect Characterization flow. This file contains the modeled layers of a circuit. |
| dtcoFCParasiticFilesDefaultPath | string | context | <file path> | Parasitics File default file open path. |
| dtcoFCParasiticLayerMapFile | string | user | | The Layer Map file contains physical layer mapping information between the input database and the specified nxtgrd file. |
| dtcoFCQORExecFile | string | user | | Quality of result executable path. |
| dtcoFCRefLibrariesDefaultPath | string | context | <file path> | Reference Libraries default directory open path |
| dtcoFCRefLibrariesList | string | user | | List of NDM libraries that hold the timing, power and physical information related to standard cells. |
| dtcoFCReportFilesDefaultPath | string | user | | Compare Split Default file open path. |
| dtcoFCReportFilesList | string | user | | Compare Splits YAML file paths. |
| dtcoFCRunDirRoot | string | user | <file path> | This file contains technology-specific information such as the names, characteristics for each metal layer, and design rules. This information are required to route a design. |
| dtcoFCTechFile | string | user | | This file contains technology-specific information such as the names, characteristics for each metal layer, and design rules. This information are required to route a design. |
| dtcoFCUseCustomScript | bool | user | false | Custom Script file use check box selection. |

Table 94 Design Technology Co-Optimization Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-----------------------------------|--------|---------|-------------|---|
| dtcoFCVerilogFilesDefaultPath | string | context | <file path> | Verilog File default file open path. |
| dtcoInterconnectFlowFile | string | context | | Flow file contains the layer information in the order layers would be fabricated. It is the user's responsibility to put layers in the right order. Flow file contains the layer information in the order layers would be fabricated. It is the user's responsibility to put layers in the right order. |
| dtcoInterconnectHeaderFile | string | context | | This file contains information regarding the ITF header (technology, foundry, process node, process version, process type, process corner, temperature) |
| dtcoInterconnectImportCellName | string | context | | Cell Name for Save in library. |
| dtcoInterconnectImportResults | bool | context | true | The output of Interconnect characterization (nxtgrd, ITF) files will be placed in interconnect_results directory. |
| dtcoInterconnectMaterialsFile | string | context | | Materials File contains information about the physical properties specific to the material needs to be simulated in the Sentaurus Workbench. It contains information about dielectrics, conductors, bulk resistivity model |
| dtcoInterconnectNumberOfProcesses | int | user | 16 | Number of threads to run Interconnect Characterization. |
| dtcoInterconnectParamsLibrary | string | context | | Location where files related to IV CV curve and Compact Model generation would be stored. |
| dtcoInterconnectProjectName | string | context | | The directory with Project name is created in Run Root Dir to run the Interconnect characterization. |
| dtcoInterconnectRunDirRoot | string | user | <file path> | Location where files related to Interconnect Characterization run will be stored. |

Table 94 Design Technology Co-Optimization Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-----------------------------------|-----------------|---------|---------------|--|
| dtcoInterconnectRunMode | dtcoJobTypeEnum | context | batch | The GUI mode will launch the Sentaurus Workbench GUI and user needs to manually run the Interconnect Characterization. Batch mode will run in the background and status can be checked in Job Monitor. |
| dtcoInterconnectTcadQueue | string | context | local:default | Interconnect TcadQueue default, priority. |
| dtcoInterconnectTemplateDirectory | string | context | | The template directory contains the files needed for Sentaurus tcad tool to run Interconnect Characterization. |
| dtcoIVCVCategory | string | context | transistor | IVCV category. |
| dtcoIVCVLoadLib | string | context | | The DTCO parameters for transistor modeling are picked from the specified DTCO Library. |
| dtcoIVCVRun | bool | context | true | Checkbox to select IV CV run. If not selected, then only Compact Model generation is done. |
| dtcoIVCVRunMode | dtcoJobTypeEnum | context | batch | The GUI mode will launch the Sentaurus Workbench GUI and user needs to manually run the Transistor Modeling. Batch mode will run in the background and status can be checked in Job Monitor. |
| dtcoIVCVTcadQueue | string | context | local:default | Transistor modeling TcadQueue default, priority. |
| dtcoIVCVTechType | string | context | BulkFinFET | IVCV tech type. |
| dtcoIVCVTemplateDirectory | string | context | | The template directory contains the files needed for Sentaurus TCAD tool to run transistor modeling. |
| dtcoIVCVThreads | int | context | 16 | Number of threads to run transistor modeling. |
| dtcoIVCVVariability | string | context | Nominal | Transistor modeling variability. |
| dtcoJobLimit | int | user | 20 | Limit on Jobs executed |

Table 94 Design Technology Co-Optimization Preferences (Continued)

| Name | Type | Scope | Default | Description |
|--|--------|---------|---|--|
| dtcoParamsExportFileDefaultPath | string | context | <file path> | The default path for exporting parameters. |
| dtcoParamsImportFileDefaultPath | string | context | <file path> | The default path for Importing parameters. |
| dtcoParamsLoadLib | string | context | | Library name to load DTCO parameters. |
| dtcoParamsSaveLib | string | context | | DTCO parameters library name. |
| dtcoParasiticExportNetlist | bool | context | true | Export is checked user needs to specify standard cell LCV of schematic, View Search List, View Stop List and the tool will run HSPICE Netlister to generate netlist file. If Export is unchecked user needs to specify the netlist (*.spi *.lvs) file name which will be passed as parameter to run cell parasitic extraction. |
| dtcoParasiticExportStream | bool | context | true | Export is checked user needs to specify standard cell LCV of layout and the tool will export GDS stream before running cell parasitic extraction. If Export is unchecked user needs to specify the GDS stream (*.GDS) file and cell name which will be passed as parameter to the run. |
| dtcoParasiticHSPICEViewSearchList | string | user | hspice hspiced
cmos.sch
cmos_sch
schematic
veriloga | The view search list for Schematic LCV. |
| dtcoParasiticHSPICEViewStopList | string | user | hspice hspiced
veriloga | The view stop list for Schematic LCV. |
| dtcoParasiticImportResults | bool | context | true | Checkbox selection to import results. |
| dtcoParasiticImportResultsGenerateSymbol | bool | context | false | Generate Symbol |
| dtcoParasiticImportResultsGenerateSymbol Overwrite | string | context | Always | Generate Symbol overwrite |

Table 94 Design Technology Co-Optimization Preferences (Continued)

| Name | Type | Scope | Default | Description |
|-----------------------------------|-----------------|---------|-------------|--|
| dtcoParasiticImportResultsLibName | string | context | | Library name to import final spice file. |
| dtcoParasiticLayerConfigFile | string | context | | Layer config file for Cell Parasitic Extraction. |
| dtcoParasiticLayoutLayersFile | string | context | | Layout layers file for Cell parasitics extraction. |
| dtcoParasiticLayoutLCV | lcv | context | <file path> | Layout LCV for Cell Parasitic extraction. |
| dtcoParasiticMaterialsFile | string | context | | Materials File contains information about the physical properties specific to the material needs to be simulated in the Sentaurus Workbench. |
| dtcoParasiticNetlister | string | context | HSPIICE | Netlister for schematic lcv cell parasitic extraction. |
| dtcoParasiticNetlisters | string | context | HSPIICE | List of netlisters for schematic lcv cell parasitic extraction. |
| dtcoParasiticNetlistFile | string | context | | Netlist (*.spi *.lvs) file name which will be passed as parameter to run cell parasitic extraction. |
| dtcoParasiticNumberOfProcesses | int | user | 16 | Number of threads to run Cell parasitic extraction. |
| dtcoParasiticProcessFile | string | context | | Process file for Cell Parasitic Extraction. |
| dtcoParasiticRaphaelConfigFile | string | context | | This file contains user settings for Process Explorer (meas_RC) and Raphael used for FinFET flow. |
| dtcoParasiticRunDirRoot | string | user | <file path> | Location where files related to Cell Parasitic Extraction run will be stored. |
| dtcoParasiticRunMode | dtcoJobTypeEnum | context | batch | The GUI mode will launch the Sentaurus Workbench GUI and user needs to manually run the Interconnect Characterization. Batch mode will run in the background and status can be checked in Job Monitor. |

Table 94 Design Technology Co-Optimization Preferences (Continued)

| Name | Type | Scope | Default | Description |
|---------------------------------------|-----------------------|---------|---------------|--|
| dtcoParasiticRunNetlisterInBackground | bool | user | false | Checkbox to run nelister in background for schematic lcv cell parasitic extraction. |
| dtcoParasiticSchematicLCV | lcv | context | <file path> | Schematic LCV for Cell Parasitic Extraction. |
| dtcoParasiticSetupMode | dtcoFileInputModeEnum | context | Library | Mode for Setup files input. If Libraray is specified setup files are used form tcad_prefs.xml from library directory, otherwise user needs to specify the setup files. |
| dtcoParasiticStreamCellName | string | context | | Cell name which will be passed as parameter to run Cell parasitic extraction. |
| dtcoParasiticStreamFile | string | context | | GDS stream (*.GDS) file which will be passed as parameter to run Cell parasitic extraction. |
| dtcoParasiticTcadQueue | string | context | local:default | Parasitic Cell Extraction TcadQueue default, priority. |
| dtcoParasiticTemplateDirectory | string | context | | The template directory contains the files needed for Sentaurus TCAD tool to run Cell Parasitic Extraction. |
| dtcoRunSWBTclFile | string | user | <file path> | Path of run_swb.tcl file for running Tcad tools. |
| dtcoTcadPreShellScriptFile | string | user | | TCAD Source Shell Script File Path to run before starting any Sentaurus Workbench tool. |
| dtcoTransistorRunDirRoot | string | user | <file path> | Location where files related to IV CV curve and Compact Model generation would be stored. |
| dtcoUDDGridInfoFile | string | user | | Path for UDD Grid Info File. |

11

Graphical Infrastructure Tcl

This chapter lists commands, enumerations, objects, and preferences in the Custom Compiler Tcl `gi` namespace. This namespace provides Tcl for graphical infrastructure operations supporting the Custom Compiler window framework and for creating custom user interfaces.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

The commands covered are:

- [gi::addActions](#)
- [gi::addAssistant](#)
- [gi::addMenu](#)
- [gi::addToolbar](#)
- [gi::addWindow](#)
- [gi::applyWorkspace](#)
- [gi::closeFrames](#)
- [gi::closeWindows](#)
- [gi::collapse](#)
- [gi::configureTable](#)
- [gi::createAction](#)

- [gi::createActionCategory](#)
- [gi::createAppGrid](#)
- [gi::createAppIcon](#)
- [gi::createBinding](#)
- [gi::createBindingSet](#)
- [gi::createBooleanInput](#)
- [gi::createCarousel](#)
- [gi::createCarouselItem](#)
- [gi::createCheckableGroup](#)
- [gi::createColumn](#)
- [gi::createDataType](#)
- [gi::createDbTable](#)
- [gi::createDialog](#)
- [gi::createDrag](#)
- [gi::createDynamicList](#)
- [gi::createFileInput](#)
- [gi::createGroup](#)
- [gi::createInlineGroup](#)
- [gi::createLabel](#)
- [gi::createListInput](#)
- [gi::createMenu](#)
- [gi::createMutexInput](#)
- [gi::createNumberInput](#)
- [gi::createPointListInput](#)
- [gi::createPushButton](#)
- [gi::createRow](#)
- [gi::createScrollArea](#)

- [gi::createTabGroup](#)
- [gi::createTable](#)
- [gi::createTextInput](#)
- [gi::createTheme](#)
- [gi::createToolbar](#)
- [gi::createWebView](#)
- [gi::createWindow](#)
- [gi::createWorkspace](#)
- [gi::createX11Window](#)
- [gi::drop](#)
- [gi::execDialog](#)
- [gi::executeAction](#)
- [gi::expand](#)
- [gi::exportTable](#)
- [gi::findChild](#)
- [gi::getActionCategories](#)
- [gi::getActions](#)
- [gi::getActiveDialog](#)
- [gi::getActiveTab](#)
- [gi::getActiveWindow](#)
- [gi::getAppIcons](#)
- [gi::getAssistants](#)
- [gi::getBindings](#)
- [gi::getBindingSets](#)
- [gi::getCells](#)
- [gi::getChildren](#)
- [gi::getColumns](#)

- [gi::getContextMenu](#)
- [gi::getDataTypes](#)
- [gi::getDialogs](#)
- [gi::getField](#)
- [gi::getFrames](#)
- [gi::getMenus](#)
- [gi::getRows](#)
- [gi::getSectionSizes](#)
- [gi::getThemes](#)
- [gi::getToolbars](#)
- [gi::getWindows](#)
- [gi::getWindowTypes](#)
- [gi::getWorkspaces](#)
- [gi::getXidUsage](#)
- [gi::hideContextMenu](#)
- [gi::layout](#)
- [gi::loadBindings](#)
- [gi::loadWorkspaces](#)
- [gi::moveColumns](#)
- [gi::moveRows](#)
- [gi::openTerminal](#)
- [gi::pickWindow](#)
- [gi::pressButton](#)
- [gi::prompt](#)
- [gi::registerWebViewScheme](#)
- [gi::remove](#)
- [gi::reportWidgetState](#)

- [gi::saveBindings](#)
- [gi::saveWorkspaces](#)
- [gi::scrollTo](#)
- [gi::sendGUIEvent](#)
- [gi::setActiveDialog](#)
- [gi::setActiveTab](#)
- [gi::setActiveWindow](#)
- [gi::setContextMenu](#)
- [gi::setCurrentIndex](#)
- [gi::setDropAction](#)
- [gi::setField](#)
- [gi::setFocus](#)
- [gi::setItemSelection](#)
- [gi::setSectionSizes](#)
- [gi::showBusyCursor](#)
- [gi::showCommandsHelp](#)
- [gi::showConsole](#)
- [gi::showContextMenu](#)
- [gi::showCustomizeWindow](#)
- [gi::showDbTableFilter](#)
- [gi::showDbTableFilterExpressions](#)
- [gi::showDbTableSort](#)
- [gi::showKeyboardShortcuts](#)
- [gi::showSaveHistoryAs](#)
- [gi::sortItems](#)
- [gi::unregisterWebViewScheme](#)

gi::addActions

Description

Adds a list of actions to the specified container. Actions may be added to a container multiple times. Each time a new action reference is created, referring to the same base action definition.

Actions may be specified by object or by action name. For example, the following is legal:

```
set a [db::getNext [gi::getActions giSeparator]] gi::addActions [list $a  
giCloseWindow] -to $menu
```

Within the container, a new reference to the base action definition is created and inserted. As a shorthand for renaming these after adding them, one may specify an alternate name for the newly created action references inline by using a 2-element list instead of the action name. This is particularly useful in cascades where the action definition's name would be redundant. For example:

```
gi::addActions {{deOpenDesign "Open.."} giSeparator giCloseWindow} -to  
$menu
```

A Tcl list of giAction objects is returned corresponding to the references created for each element within the input list. These may be removed from the container with the `gi::remove` command but may not be destroyed with the `db::destroy` command.

Syntax

```
gi::addActions <actions: [giAction | string]> [-after <giAction> |  
-before <giAction>] -to <giActionCategory | giMenu | giStatusBar |  
giToolBar>
```

Returns

Tcl list of giAction objects representing the references created within the container ([giAction]).

Arguments

`actions`

Tcl list of giAction objects or action definition names to add to the container. One may specify an alternate title by using a 2-element list of {name title}.

`-to`

Container object to which to add the action.

`-before`

Action from the same container before which the new action is inserted.

`-after`

Action from the same container after which the new action is inserted.

Errors

- More than one `-after` or `-before` is specified.
- The action does not support the specified container type.
- The container has been added to a window type which the action does not support.
- Specified action '%s' does not exist.

Examples

```
gi::addActions deOpenRecent -after [gi::getActions deOpenDesign]\  
                                -to [gi::getToolbars giConsoleStandard ]
```

In this example, `gi::addActions` adds the `deOpenRecent` action next to the `deOpenDesign` action in the Console standard toolbar.

gi::addAssistant

Description

Adds the specified assistant to a container (window type or window). An assistant object representing the assistant within the container is returned.

The returned object may be removed from the window, window type or task with the `gi::remove` command but may not be destroyed with the `db::destroy` command.

Syntax

```
gi::addAssistant <assistant: giAssistant> [-dock <enum> | -after  
      <giAssistant> | -before <giAssistant>] -to <giWindow | giWindowType |  
      giWorkspace> [-floating <bool>] [-tabbed <bool>]
```

Returns

Object representing assistant referenced by container (`giAssistant`).

Errors

Assistant does not support the specified window type.

Arguments

`assistant`

Assistant to add.

`-to`

Window or window type to which to add the assistant.

`-dock`

In which dock area to place assistant: {bottom | left | right | top}.

Default: left.

`-floating`

Whether the assistant is floating.

Default: false.

`-after`

Assistant after that is added.

`-before`

Assistant before that is added.

`-tabbed`

Whether the assistant is tabbed.

Default: true.

Examples

The following example adds an assistant created by the `gi::createAssistant` command to `dmLibraryManager` window type.

```
set assistant [gi::createAssistant vcFilesAssistant
                                -createProc _createFilesTable
                                -orientation horizontal
                                -title "Library Files"
                                -windowTypes dmLibraryManager]
gi::addAssistant $assistant
  -to [gi::getWindowTypes dmLibraryManager]
  -dock bottom -tabbed false
```

gi::addMenu

Description

Adds the specified menu to a container (window type or window).

Only one of `-position`, `-before` or `-after` may be used to specify the menu location. If none are provided, the menu is placed at the end.

If a reference to the specified menu already exists within the container, it is repositioned rather than adding the menu a second time.

This command returns a menu object representing the reference within the container. This reference may be manipulated independently from the base menu object.

The returned object may be removed from the window, window type or task with the `gi::remove` command but may not be destroyed with the `db::destroy` command.

Syntax

```
gi::addMenu <menu: giMenu> [-after <giMenu> | -before <giMenu>] -to  
  <giWindow | giWindowType | giWorkspace>
```

Returns

Object representing menu within the container ([giMenu](#)).

Arguments

[giMenu](#)

Menu

-to [giWindow](#) | [giWidget](#)

Container to which to add the menu.

-before [giMenu](#)

Menu within the same container before which to place this menu.

Default: "".

-after [giMenu](#)

Menu within the same container after which to place this menu.

Default: "".

Errors

- Menu contains an action that does not support this window type.
- More than one `-after` or `-before` is specified.
- One of `-after` or `-before` arguments references an object from a different container.

Examples

The following example adds the physical verification menu to the Console to the left of the Help menu.

```
gi::addMenu [gi::getMenus leVerificationMenu ] \
            -to [gi::getWindowTypes giConsole] \
            -before [gi::getMenus giHelpMenu]
```

gi::addToolbar

Description

Adds the specified toolbar to a container (window type or window).

Only one of -before or -after may be used to specify the toolbar location. If none are provided, the toolbar is placed at the bottom/right of the dock region. Toolbars are automatically spaced according to Qt's default size policy.

If a reference to the specified toolbar already exists within the container, its position, dock state and show state are overridden.

This command returns a toolbar object representing the reference within the container.

The returned object may be removed from the window, window type or task with the `gi::remove` command but may not be destroyed with the `db::destroy` command.

Syntax

```
gi::addToolbar <toolbar: giToolbar> [-after <giToolbar> | -before
<giToolbar>] -to <giWindow | giWindowType | giWorkspace> [-dock <enum>]
[-newline <bool>] [-show <bool>]
```

Returns

Object representing the toolbar reference within the container ([giToolbar](#)).

Arguments

[giToolbar](#)

Toolbar to add.

-to [giWindow](#) | [giWidget](#)

Window in which toolbar is added.

-dock

Toolbar dock area: {bottom | left | right | top}.

Default: top.

-before [giToolbar](#)

Toolbar within the same dock area before which to place this toolbar.

Default: "".

`-after giToolbar`

Toolbar within the same dock area after which to place this toolbar.

Default: "".

`-newline <bool>`

Create a new line within the dock area and add this toolbar there.

Default: false.

`-show <bool>`

Show or hide the toolbar.

Default: true.

Errors

- Toolbar contains an action that does not support this window type.
- More than one of -after or -before specified.
- One of -after or -before references an object from a different container.
- Undocked dock area is not supported.

Examples

```
gi::addToolbar [gi::getToolbars deMouseBindings] \  
-to [db::getattr window -of [de::getActiveContext]] -dock bottom
```

In this example, `gi::addToolbar` places the `deMouseBindings` toolbar at the lower edge of the active editing window.

gi::addWindow

Description

Adds the specified window to a frame and removes it from its current frame.

Syntax

```
gi::addWindow <window: giWindow | int> [-layout <enum> | -before  
<giWindow | int> | -after <giWindow | int>] [-to <giFrame | int>] [-side  
<enum>]
```

Arguments

`window`

Window to add.

`-to`

Frame into which to add the window.

`-side`

Frame side to place the window (`{ bottom | left | right | top }`).

`-layout`

Expected final layout of the window (`{ left-right | left-right-bottom | left-top-bottom | top-bottom | top-bottom-right | top-left-right }`).

`-before`

Window within the same frame before which to place this window.

`-after`

Window within the same frame after which to place this window.

Errors

- `-before <giWindow>` is not in the same window stack.
- `-after <giWindow>` is not in the same window stack.
- Window `%d` does not exist.
- Frame `%d` does not exist.
- Frame tabs are hidden, set *giShowFrameTabs* to true.

Examples

```
set frame [gi::createFrame]
db::setAttr frame.geometry -value 1800x950+50+70
set context1 [de::open topLib/inv/schematic]
set context2 [de::open topLib/inv_tb/schematic]
set lmwindow [dm::openLibraryManager]
gi::addWindow [db::getAttr context1.window] -to $frame
# Add second window to the frame from right side
gi::addWindow [db::getAttr context2.window] -layout "left-right" -to
$frame -side "right"
# Add Library Manager to frame from bottom side
gi::addWindow $lmwindow -layout "left-right-bottom" -to $frame -side
"bottom"
db::setAttr size -of [db::filter [db::getAttr frame.stacks] -filter
{%side=="left"}} -value 900
db::setAttr size -of [db::filter [db::getAttr frame.stacks] -filter
{%side=="bottom"}} -value 500
```

gi::applyWorkspace

Description

Applies a workspace to the specified window and any others linked in the workspace object.

Syntax

```
gi::applyWorkspace <name: giWorkspace | string> -to <giWindow | unsigned  
int>
```

Arguments

name

Workspace object or workspace name to apply.

-to

Window to modify.

Examples

The following example activates the SDL workspace in the current window (assuming the current window is a Layout Editor).

```
gi::applyWorkspace lxSDL -to [gi::getActiveWindow]
```

gi::closeFrames

Description

Closes the specified frames and all the contained windows.

Note that this command closes each of its contained windows first, possibly prompting for changes in the process. If any prompt is cancelled, the command returns the "User Interrupt" error.

To destroy the window without first prompting, use the `db::destroy` command instead.

Syntax

```
gi::closeFrames <frames: {<giFrame>}>
```

Arguments

frames

Collection of frames to close.

Errors

User Interrupt.

Examples

```
# Create giFrame
set frame [gi::createFrame]
# Open Library Manager window
set lm [dm::openLibraryManager]
# Open Job Monitor window
set jm [xt::showJobMonitor]
# Add Library Manager window to new frame
gi::addWindow $lm -to $frame
# Add Job Monitor window to new frame
gi::addWindow $jm -to $frame
# Close frame - all child windows will be closed
gi::closeFrames $frame
```

gi::closeWindows

Description

Closes the specified windows. Note that this closes the window without prompting to save changes if used on a design editor window. Users should use the WindowClose action to close the window from a binding, which prompts to save before closing the window.

Syntax

```
gi::closeWindows <windows: giDialog | {<giWindow>}>
```

Arguments

windows

Collection of windows to close.

Examples

This example closes all Schematic Editor windows.

```
gi::closeWindows [gi::getWindows -filter
  {%windowType.name=="seSchematic"}]
```

gi::collapse

Description

Collapses an item in a dialog, assistant, window or toolbar.

Syntax

```
gi::collapse <name: string> -in <giAssistant | giBanner | giContainer |  
giDialog | giStatusBar | giToolbar | giWindow> -index <string>
```

Arguments

name

Tree name within container to collapse given item.

-in

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

-index

Item index in the tree widget to collapse.

gi::configureTable

Description

Sets up table configurations.

Syntax

```
gi::configureTable <operation: enum> -parent [<giAssistant | giDialog |  
giWindow>] -data <string> -tableName [<string>]
```

Arguments

operation

Specify the operation to log. ({ disableFilter | enableFilter | export |
freezeColumn | groupByColumn | hideColumn | moveColumn | setColumnFilter |
showColumn | sortColumn | unfreezeColumn | ungroup })

-parent

Parent widget to which table belongs.

-data

Extra data for some operations.

-tableName

Table name.

Returns

<none>

gi::createAction

Description

Creates an action or redefines an existing Tcl action valid for all window types. If the name references an internally defined action, this command throws an error.

The action may be a command action which invokes a Tcl command, a menu action which posts a cascading menu or menu button within a toolbar or a widget action which places an Opal widget within a toolbar or status bar. Note that widgets may not be added to menus.

The widget build procedure takes a giWindow object representing the window in which the action is being constructed and returns a new widget to appear within the toolbar or status bar.

This command supports token substitution. For a list of available tokens, see

[gi::createBinding](#).

Syntax

```
gi::createAction <name: string> (-command <script> | -menu <giMenu>
| -widgetProc [<variant>] | -checkProc [<variant>]) [-defunct <bool>
| -deprecated <bool>] [-menu <giMenu> | -log <enum>] [-widgetProc
<variant>] | -log <enum>] [-log <enum> | -checkProc [<variant>]]
[-title <string>] [-label <string>] [-icon <string>] [-largeIcon
<string>] [-prompt <string>] [-toolTip <string>] [-autoDisable <bool>]
[-history <bool>] [-enabled <bool>] [-replacement <string>]
```

Returns

New action (giAction).

Arguments

name

Unique identifier of new action.

-title

Text that appears in menus.

-command

Tcl script to evaluate when actions are executed.

Default: "".

-menu

Menu this action references (for creating menu buttons and cascades).

Default: "".

`-widgetProc`

Tcl procedure that takes the action reference as an argument and creates and returns an Opal widget.

Default: "".

`-label`

Toolbar suffix text.

Default: "".

`-icon`

File path to a 16x16 graphical image (pixmap) to show in menus and toolbars.

Default: "".

`-largeIcon`

File path to a 24x24 graphical image (icon).

Default: "".

`-prompt`

Status bar prompt string.

Default: "".

`-toolTip`

Mouse-over bubble text.

The `-toolTip` argument only works for actions that are added to toolbars. It is not supported for actions which are added to menus.

Default: "".

`-autoDisable`

If true, disable this action in read-only editor windows.

Default: false.

`-history`

If true, this action may be displayed in action history widgets.

Default: false.

`-enabled`

Initial enabled state. The default is true.

`-log`

Action logging method. ({ action | command | none } Default is: command)

`-checkProc`

Creates a checkable action and calls this procedure with the giAction when toggled.

`-defunct`

True if the action is defunct. The default is false.

`-deprecated`

True if the action is deprecated. The default is false.

`-replacement`

Name of the replacement action to view, regarding deprecate/ defunct of this action.

Errors

- Action was defined internally and may not be redefined.
- More than one -menu and -command are specified.
- Icon file not found.

Examples

```
gi::createAction deZoomInCursor \  
    -title "Zoom Out (At Point)" \  
    -command {de::zoom -window %w -factor 2.0 -center %a} \  
    -prompt "Zoom in by 2x" \  
    -icon "zoom_in"
```

In this example, `gi::createAction` creates the `deZoomInCursor` action to zoom in by a factor of 2.0 around a cursor position.

gi::createActionCategory

Description

Creates a new action category.

Syntax

```
gi::createActionCategory -title <string> -windowType <giWindowType>  
    -actions [<giAction | string>]
```

Returns

<giActionCategory>: The new category.

Arguments

-title

Unique title of action category.

-windowType

Window type for which this category is valid.

-actions

Tcl list of giAction objects or names of giAction objects to include in the category.

Examples

The following example creates a *User* action category in the **Customize Window** dialog box from the **Library Manager** window. The *Options* title will be displayed in the **Category** field and the prompt option of the action will be displayed in the description field.

```
gi::createAction dmOption -command showDMOption -title Options
    -prompt "Version Control Options"
gi::createActionCategory -title User
    -windowType [gi::getWindowTypes dmLibraryManager]
    -actions {dmOption}
```

gi::createAppGrid

Description

Creates a new giAppGrid widget.

Syntax

```
gi::createAppGrid [<{none}>: string] [-parent <giAssistant | giContainer
| giDialog | giWindow>] [-enabled <bool>] [-shown <bool>] [-iconSize
<int>]] [-iconNames <dbCollection>]
```

Arguments

{none}

Widget name. (Default is:)

-parent

Dialog, group or other container into which this widget should be placed.

-enabled

Initial enabled state. (Default is: true)

`-shown`

Initial shown state. (Default is: true)

`-iconSize`

Icon size for grid items.

`-iconNames`

Specifies collection of icons to add on grid.

Returns

`<giAppGrid>` : New object.

Examples

The following example creates application grids on the dialog placing Library Manager and Search & Replace application icons.

```
set dialog [gi::createDialog appDialog -title Applications]
gi::createAppGrid appGrid -parent $dialog \
    -iconNames {dmLibraryManager dbFindReplace}
```

gi::createAppIcon

Description

Creates a new giAppIcon and shows in all giAppGrids.

Syntax

```
gi::createAppIcon <name> [-label <string>] [-icon <string>] [-toolTip  
    <string>] [-category <string> | -product <string>] [-enabled <bool>]  
    [-shown <bool>] [-execProc <proc>] [-contextMenuProc <proc>]
```

Arguments

`{name}`

Widget name. (Default is:)

`-label`

Label to show. (Default is:)

`-icon`

Icon file name, searched within the icon search path, which is shown on button.
(Default is:)

`-toolTip`

Optional widget toolTip. (Default is:)

`-category`

Category on which giAppIcon is added. (Default is:)

`-product`

Product icons to show. (Default is:)

`-enabled`

Initial enabled state. (Default is: true)

`-shown`

Initial shown state. (Default is: true)

`-execProc`

Procedure to execute when clicked.

`-contextMenuProc`

Procedure to execute when right-clicking on this widget to request a context menu.

Returns

<giAppIcon> : New object.

Examples

The following example creates a doc icon, which opens the Technology Manager tool, on the Home Page.

```
gi::createAppIcon techManager -icon doc_128 \  
    -label "Technology Manager" \  
    -execProc {apply {{widget} {db::showManageTechnology }}}}
```

gi::createBinding

Description

Creates a binding within a binding set (or the active set if one is not specified). In the ordered list of bindings in the binding set, new bindings are added last. However, if the specified binding already exists with a different target (Tcl procedure or action), this command replaces the existing binding with the new one without changing the order of the bindings.

Custom Compiler Tcl commands support token substitution, where implemented, in which the command obtains a value during execution, such as a window ID, the coordinates of a point, or a value returned by a Tcl command, and executes using this value.

[Table 95](#) lists and describes available tokens and their substitution values, which you can use with the `gi::createBinding` and `gi::createAction` Tcl commands.

Table 95 *Custom Compiler Tcl Token Substitution*

| Token | Value Substituted | Description |
|-------|--|---|
| %a | (x,y) coordinate point | Cursor location, in user units. |
| %c | Value returned by <code>de::getContexts</code>
-window %w | Current data context. See de::getContexts . |
| %o | Value returned by
<code>de::getActiveFigure</code> | Active figure object. See de::getActiveFigure . |
| %p | (x,y) coordinate point | Cursor snap point, in user units. |
| %s | (x,y) coordinate point | Cursor location, in pixels relative to the screen. |
| %w | giWindow | Window that generated the event. |
| %x | (x,y) coordinate point | Cursor location, in pixels relative to window origin. |
| %% | % | Percent symbol. |

Syntax

```
gi::createBinding [-windowType <giWindowType | giWindowTypeName> |
  -assistant <giAssistant | giAssistantName>] (-command <string> | -action
  <giAction | giActionName>) -event <string> [-activeOperation <string>]
  [-activeObject <string>] [-activeTool <string>] [-activeWorkspace
  <string>] [-widgetType <string>] [-keyword <string>] [-set <string> |
  {<giBindingSet>}>] [-userCreated <bool>] [-autoRepeat <bool>]
```

Arguments

`-event`

GUI event to which action is bound.

`-windowType`

Optionally restricts this binding to the specified window type.

Default: any string.

`-assistant`

Optionally restricts this binding to the specified assistant.

Default: empty string.

`-activeOperation`

Regular expression to match against the top command on the command stack.

Default: any string.

`-activeObject`

Regular expression to match against the type of active object, which is window dependent.

Default: any string.

`-activeTool`

Regular expression to match against design editor tools active within the current window.

`-activeWorkspace`

Regular expression to match against the workspace active within the current window.

`-widgetType`

Widget type name to which this binding applies.

`-keyword`

Binding keyword.

`-set`

Binding set in which to define this binding.

Default: Currently active binding set.

`-command`

Tcl procedure to evaluate.

`-action`

Action to execute.

`-userCreated`

If true, binding is automatically saved and loaded through xml file. The default is false.

`-autoRepeat`

If true, binding can be automatically repeated. The default is false.

Examples

```
gi::createBinding -event "Alt-e" -command {ide::descend -inPlace true}
```

This example creates the key binding Alt-e and defines it to edit in-place.

gi::createBindingSet

Description

Creates and returns a new binding set.

Syntax

```
gi::createBindingSet <name: string> [-of <giBindingSet | string>]
```

Arguments

name

Name of the binding set to create.

-of

Optionally copies an existing binding set into the new set.

Examples

```
gi::createBindingSet "myBindingSet" -from "Custom Compiler"
```

In this example, `gi::createBindingSet` creates a new binding set by copying binding definitions from the "Custom Compiler" set.

gi::createBooleanInput

Description

Creates a new `giBooleanInput` widget and adds it to the specified parent.

Syntax

```
gi::createBooleanInput [<{none}: string>] [-parent <giContainer |  
giDialog>] [-enabled <bool>] [-shown <bool>] [-prefName <dbPrefName>]  
[-toolTip <string>] [-valueChangeProc [<variant>]] [-label <string>]  
[-value <bool>]
```

Returns

New object (<giBooleanInput>).

Arguments

{none}

Widget name.

`-parent`

Dialog, group, or other container into which this widget should be placed.

`-enabled`

Initial enabled state.

Default: true.

`-shown`

Initial state shown.

Default: true.

`-prefName`

Optional preference name within the top-level container's scope, which provides the initial value. This value is automatically updated with changes according to the container update model.

`-toolTip`

Optional widget tooltip.

Default: "".

`-valueChangeProc`

Procedure to execute when the value is changed. Called with the widget object.

`-label`

Widget label, shown next to the check box.

`-value`

Initial value.

Default: false.

Examples

```
proc biValueChangeProc {widget} {  
    set dialog [db::getAttr widget.root]  
    set fl [gi::findChild fl -in $dialog]  
    set fs [gi::findChild fs -in $dialog]  
    if {[db::getAttr widget.value]} {  
        db::setAttr fl.value -value true  
        db::setAttr fs.value -value true  
    }  
}  
  
set dialog [gi::createDialog instNaming -title "Boolean Input"]
```

```
set crossSelection [gi::createLabel -parent $dialog -label "Cross  
Selection:"]  
  
gi::createBooleanInput cv -parent $dialog -label "Cross View"  
-valueChangeProc biValueChangeProc  
gi::createBooleanInput fl -parent $dialog -label "From Layout"  
gi::createBooleanInput fs -parent $dialog -label "From Schematic"
```

gi::createCarousel

Description

Creates a new *giCarousel* widget.

Syntax

```
gi::createCarousel [<{none}>: string] [-parent <giAssistant | giContainer  
| giDialog | giWindow>] [-enabled <bool>] [-shown <bool>] [-label  
<string>] [-iconSize [<int>]]
```

Returns

<giCarousel> : New object.

Arguments

{none}

Widget name. (Default is: ".")

-parent

Dialog, group, or other container into which this widget should be placed.

-enabled

Initial widget state enabled. (Default is: true.)

-shown

Initial widget state shown. (Default is: true.)

-label

Label to show. (Default is: ".")

-iconSize

Icon size for carousel items.

Examples

```
proc ciExecProc {widget} {  
    switch [db::getAttr widget.label] {  
        "Home" {
```

```

        gi::createFrame
    }
    "Library Manager" {
        dm::openLibraryManager
    }
    "Search" {
        db::showFindReplace
    }
}

set dialog [gi::createDialog carousel -title Carousel]
set carousel [gi::createCarousel homeCarousel -parent $dialog]
gi::createCarouselItem -icon home_128 -parent $carousel -label Home
    -execProc "ciExecProc"
gi::createCarouselItem -icon library_manager_128 -parent $carousel -label
    "Library Manager" -execProc "ciExecProc"
gi::createCarouselItem -icon search_128 -parent $carousel -label "Search"
    -execProc "ciExecProc"

```

gi::createCarouselItem

Description

Creates a new *giCarouselItem* within a *giCarousel* widget.

The signature for the `execProc` argument is:

```
proc myExecProc {item} {}
```

Syntax

```
gi::createCarouselItem [<{none}>: string] [-enabled <bool>] [-shown
<bool>] [-toolTip <string>] [-label <string>] [-icon <string>] [-parent
<giCarousel>] [-execProc [<variant>]] [-contextMenuProc [<variant>]]
```

Returns

<giCarouselItem> : New object.

Arguments

{none}

Widget name. (Default is: "").

-enabled

Initial widget state enabled. (Default is: true.)

-shown

Initial widget state shown. (Default is: true.)

`-toolTip`

Optional widget toolTip. (Default is: "").

`-label`

Label to show. (Default is: "").

`-icon`

Icon filename is searched within the icon search path, which is shown on the button (Default is: "").

`-parent`

Parent carousel.

`-execProc`

Procedure to execute when clicked.

`-contextMenuProc`

Procedure to execute when you right-click on this widget to request for a context menu.

Examples

The following example creates a shortcut on the Custom Compiler Home tab for launching an xterm window.

```
db::setPrefValue giShowFrameTabs -value true
proc onPostWindowCreated {win} {
    if {"giHomePage"==[db::getAttr win.windowType.name]} {
        set c [gi::findChild applications -in $win]
        gi::createCarouselItem xterminal -parent $c \
            -label "XTerm" \
            -icon console_128 \
            -execProc [list ::gi::_home::_execute {openXterm}]
    }
}

proc openXterm {} {
    set logFile .terminal.log
    set cmd "xterm > $logFile 2>&1"
    xt::createJob terminal -type interactive -cmdLine $cmd \
        -runDesc "Open Terminal"
}

db::createCallback onPostWindowCreated -callbackType \
onPostWindowCreated -priority 100 -procedure onPostWindowCreated
```

gi::createCheckableGroup

Description

Creates a new giCheckableGroup container and adds it to the specified parent.

Syntax

```
gi::createCheckableGroup [<{none}: string>] [-parent <giAssistant |  
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]  
[-prefName <dbPrefName>] [-toolTip <string>] [-label <string>] [-value  
<bool>] [-collapsible <bool>] [-valueChangeProc [<variant>]]
```

Returns

New object (<giCheckableGroup>)

Arguments

{none}

Widget name.

-parent

Dialog, group, or other container into which this widget should be placed.

-enabled

Initial enabled state.

Default: true.

-shown

Initial state shown.

Default: true.

-prefName

Optional preference name within the scope of the top-level container, which provides the initial value. This value is automatically updated with changes according to the container update model.

-toolTip

Optional widget tooltip.

Default: "".

-label

Group box label.

`-value`

Initial value.

Default: false.

`-collapsible`

Renders a collapsible group box instead of a checkable one.

Default: false.

`-valueChangeProc`

Procedure to execute when the value is changed. Called with the widget object.

Examples

```
set dialog [gi::createDialog checkgroupDialog -title "Checkable Group"]
set gr [gi::createCheckableGroup
    -label "giCheckableGroup"
    -parent $dialog]
set w1 [gi::createFileInput
    -parent $gr
    -label "Mapping File"
    -fileMasks {"XML (*.xml)" "TEXT (*.txt)"}
    -mode save]
set w2 [gi::createFileInput
    -parent $gr
    -label "Run Dir"
    -fileType directory
    -mode open]
set w3 [gi::createMutexInput
    -parent $gr
    -label "Pin Type"
    -enum {input output inputOutput switch tristate}
    -value input
    -viewType combo]
db::setAttr dialog.geometry -value 20%X20%+40%+40%
```

gi::createColumn

Description

Adds a column to the specified table.

Syntax

```
gi::createColumn [-before <giColumn> | -after <giColumn>] -parent
<giTable> [-label <string>] [-dataType <giDataType | string>] [-stretch
<bool>] [-readOnly <bool>] [-shown <bool>] [-tooltip <string>]
[-decorateProc [<variant>]]
```

Returns

<giColumn>: Newly created column.

Arguments

-parent

Table within which to add the new column.

-label

Column header. Note that column headers are hidden for the table until at least one column has a non-empty label. (Default is:)

-dataType

Cell-specific data type. (Default is: string)

-stretch

If true, this column automatically resizes when the container is resized. Note that columns do not automatically resize when the table has columns the user can resize. (Default is: false)

-readOnly

Forces all contained cells to be considered read-only. (Default is: false)

-shown

Initial shown state for this column. (Default is: true)

-before

Column position.

-after

Column position.

-tooltip

Tooltip for column header.

-decorateProc

Procedure to modify style attributes of displayed giCells, called with giCells as they are revealed.

Examples

This example shows the name/value pairs of ::env global array in the table widget.

```
set dialog [gi::createDialog testDialog -title "Create Column"]
# Create table
```

```
set table [gi::createTable testTable -parent $dialog
    -alternatingRowColors true]
set col1 [gi::createColumn -parent $table -label "Name" -stretch false
    -readOnly true]
set col2 [gi::createColumn -parent $table -label "Value" -stretch true]
set c 1
foreach key [lsort [array name ::env]] {
    set row [gi::createRow -parent $table]
    db::setAttr row.label -value $c
    set cell [gi::getCells -row $row -column $col1]
    db::setAttr cell.value -value $key
    set cell [gi::getCells -row $row -column $col2]
    db::setAttr cell.value -value $::env($key)
    incr c
}
db::setAttr dialog.geometry -value "30%x60%+35%+20%"
```

gi::createDataType

Description

Creates a new data type object.

Syntax

```
gi::createDataType <name: string> [-toDisplayProc [<variant>]]
    [-toValueProc [<variant>]] [-updateValueProc [<variant>]] [-editProc
    [<variant>]] [-compareProc [<variant>]]
```

Returns

<giDataType>: Newly created data type object.

Arguments

name

Data type name consisting of characters from the set [A-Z][a-z].

-toDisplayProc

Optional display procedure which takes a value object and returns a display string. If not specified, the value is returned as a string.

-toValueProc

Optional reverse-display procedure which takes user-entered data and converts it back to a value. When updateValueProc is specified, this procedure is only called when the current value is blank.

Default: "".

`-editProc`

Optional custom procedure that returns an edit widget for a value. If unspecified, a text input widget is assumed.

`-compareProc`

Optional procedure that compares two values. Returns a negative number if the first is less than the second, 0 if they are equal, and a positive number if the first is larger than the second. If unspecified, an alphanumeric string comparison is used.

Examples

Example 1

The datatype object created with the `gi::createDataType` command, specified at column creation command with `-dataType` option, allowing users to select the desired cell view with `dmCellViewInput` edit widget.

```
set fileMode [gi::createDataType dmLCV
    -toDisplayProc _display
    -editProc _edit]

proc _display { value } {
    return $value
}
proc _edit { value } {
    return [dm::createCellViewInput]
}

set dlg [gi::createDialog sampleDialog
    -title "Data Type"]
set tbl [gi::createTable table -parent $dlg ]
gi::createColumn -parent $tbl -dataType dmLCV -label "LCV" -stretch true
for {set i 0} {$i < 2} {incr i} {
    gi::createRow -parent $tbl
}
```

Example 2

A new `dataType` object, created with the `gi::createDataType` command and defined at **Mutex Input** creation command, maps the `deDefaultInstanceNaming` preference values (`UserPrefix`, `MasterCell`) with displayed values ("Inst Name", "Cell Name") on the dialog box.

```
proc toDisplay {value} {
    switch $value { UserPrefix {return "Inst Name"}
                  MasterCell {return "Cell Name"} }
}
proc toValue {display} {
    switch $display { "Inst Name" {return UserPrefix}
                    "Cell Name"  {return MasterCell}}
}
```

```
}
gi::createDataType instNaming -toDisplayProc toDisplay -toValueProc
toValue
set d [gi::createDialog instNaming -title "Instance Naming"]
gi::createMutexInput -parent $d -prefName deDefaultInstanceNaming
-enum {"Inst Name" "Cell Name"} -dataType instNaming -viewType radio
```

gi::createDbTable

Description

Creates a new giTable widget viewing a database.

Syntax

```
gi::createDbTable [<{none}: string>] [-parent <giAssistant |
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown
<bool>] [-label <string>] [-dbPath <string>] [-dbHandle <string>]
[-dbTableName <string>] [-query <string>] [-filter <string>]
[-filterExpressions <string>] [-filterExpressionsOp <string>] [-sort
<string>] [-allowMoveColumns <bool>] [-allowSortColumns <bool>]
[-alternatingRowColors <bool>]
[-columnResizeMode <enum>] [-allowFilterColumns <bool>]
[-allowFreezeColumns <bool>] [-allowGroupByColumns <bool>]
[-allowEexportToCSV <bool>] [-selectionModel <enum>] [-saveColumnConfig
<bool>] [-editTrigger <enum>] [-selectionChangeProc [<variant>]]
[-sortChangeProc [<variant>]] [-doubleClickProc [<variant>]]
[-filterChangeProc [<variant>]] [-toolTip <string>] [-contextMenuProc
<variant>]]
```

Arguments

{none}

Widget name. (Default is:)

-parent

Dialog, group, or other container into which this widget should be placed.

-enabled

Initial enabled state. (Default is: true)

-shown

Initial shown state. (Default is: true)

-label

Widget label, shown in the left column. If a name is not specified, the widget is "unlabeled" and extends into the label column (Default is:)

`-dbPath`

Database file to populate table.

`-dbHandle`

Handle for the database to populate table.

`-tableName`

Database table name.

`-query`

Overrides `tableName`, filters and sorts with an explicit SQL query, also disables `-allowSortColumns`.

`-filter`

Table filter specifications. (Default is:)

`-filterExpressions`

SQL filter expressions. (Default is:)

`-filterExpressionsOp`

Operator for joining SQL filter expressions. (Default is: AND)

`-sort`

Table sorting specification. (Default is:)

`-allowMoveColumns`

If true, moves columns by dragging column headers. (Default is: false)

`-allowSortColumns`

If true, sorts rows by clicking column headers. (Default is: false)

`-showColumnFilter`

Specifies if column filtering is enabled. (Default is: false)

`-allowFreezeColumns`

Allows you to select the freeze or unfreeze state of each column. (Default is: true)

If true, right-click on the column header to display context menu.

`-allowGroupByColumns`

Specifies if grouping is enabled for the table. (Default is: false)

`-allowExportToCSV`

Specifies if export table in .csv file is enabled. (Default is: false)

`-alternatingRowColors`

If true, shows rows with alternating background colors. Use this for large tables.
(Default is: false)

`-selectionModel`

Default selection model. ({ multipleCells | multipleRows | singleCell | singleRow }, default is: singleRow)

`-columnResizeMode`

Specifies column resize mode of the table. ({ fixed | interactive | resizeToContents | stretch }, default is: interactive)

`-saveColumnConfig`

Remembers the column width, order, and the shown and frozen states. (Default is: false)

`-selectionChangeProc`

Procedure to execute when the selection is changed. Called with the table object. Retrieve selection from attribute.

`-sortChangeProc`

Procedure to execute when the sort is changed by the user clicking header. Called with the table object. Retrieve sort spec from attribute.

`-doubleClickProc`

Procedure to execute when a double click is done over a cell. Called with the cell over which a double click was detected.

`-filterChangeProc`

Procedure to execute when the filter spec is changed.

`-toolTip`

Optional widget toolTip. (Default is:)

`-contextMenuProc`

Procedure to execute when right-clicking on this widget to request a context menu.

Returns

<giDbTable> : Newly created table.

Examples

```
# Create sqlite database with opened libraries cellViews
file delete LCV.db
```



```
sqlite db LCV.db
db eval {CREATE TABLE LCV(Library, Cell, View); }
db::foreach lib [dm::getLibs] {
    set libName [db::getAttr lib.name]
    db::foreach cell [dm::getCells -lib $lib] {
        set cellName [db::getAttr cell.name]
        db::foreach view [dm::getCellViews -cell $cell] {
            set viewName [db::getAttr view.name]
            db eval { INSERT INTO LCV VALUES($libName, $cellName,
$viewName); }
        }
    }
}

# Create dialog with DB Table
set dialog [gi::createDialog lcvDialog -title "LCV DB"]
set table [gi::createDbTable -parent $dialog \
    -allowSortColumns 1 \
    -dbTableName LCV \
    -dbPath LCV.db]

# Get selected row values
db::getAttr table.itemSelectionValues
```

gi::createDialog

Description

Creates a new giDialog object.

Syntax

```
gi::createDialog <{none}>: string> [-enabled <bool>] [-shown <bool>]
[-parent <giDialog | giWindow | unsigned int>] -title <string>
[-prefScope <dbScope>] [-styleSheet <string>] [-showApply <bool>]
[-showHelp <bool>] [-stayOnTop <bool>] [-execProc [<variant>]]
[-closeProc [<variant>]] [-validProc [<variant>]] [-defaultsProc
<variant>]] [-defaultsMenuProc [<variant>]] [-preLoadPrefsProc
<variant>]] [-postSavePrefsProc [<variant>]] [-topicId <string>]
```

Returns

New giDialog object.

Arguments

{none}

Dialog name.

-enabled

Initial enabled state. (Default is: true)

`-shown`

Initial state shown. (Default is: true)

`-parent`

Parent window or dialog box.

`-title`

Dialog title.

`-prefScope`

Preference scope for contained widgets with preference defaults.

`-styleSheet`

Dialog style sheet. (Default is:)

`-showApply`

Shows an **Apply** button in the dialog box. Only applies if `-execProc` is also specified. (Default is: true)

`-showHelp`

If true, shows a Help button in the button bar. (Default is: true)

`-stayOnTop`

When true, the dialog stays on top of all other windows and dialogs.

When false, the dialog stays on top of its parent window and no other dialogs.

If the `giDialogStayOnTop` Tcl preference is false, then the dialog does not stay on top of any other dialog or window, regardless of this argument. (Default is: true)

`-execProc`

If set, **OK** and **Apply** buttons may be added that trigger this method. Called with the dialog object.

`-closeProc`

Optional procedure to execute when the dialog is closed (either via **Cancel** or after **OK** and **Apply**). Called with the dialog object.

`-validProc`

Optional procedure to execute when the valid state of a child required widget changes. Called with the dialog object only.

`-defaultsProc`

Optional procedure to call when the **Defaults** button is pressed. Called with the dialog object only.

`-defaultsMenuProc`

Optional procedure to call when an action of **Defaults** button's menu is pressed. Called with the dialog object and the action name.

`-preLoadPrefsProc`

Optional procedure to call before loading preferences from the specified file. Called with the dialog object and file path.

`-postSavePrefsProc`

Optional procedure to call after saving preferences from the specified file. Called with the dialog object and file path.

`-topicId`

Topic ID.

Examples

Example 1

```
proc dialogExecProc {dialog} {  
    set prefix [gi::findChild prefix.value -in $dialog]  
    puts "Prefix = $prefix"  
}  
  
set dialog [gi::createDialog testDialog -title "Test" -execProc  
    dialogExecProc]  
gi::createTextInput prefix -label Prefix -parent $dialog -value "abc"
```

Example 2

```
set gr [gi::createGroup -parent [gi::createDialog dlg -title test  
    -stayOnTop 1 ]]  
set cnt 5  
while {$cnt > 0} {  
    gi::createPushButton -parent $gr -label "PushButton"$cnt  
    incr cnt -1  
}  
db::setAttr gr.spacing -value 0  
db::setAttr gr.margins -value [ list 100 100 100 100]
```

gi::createDrag

Description

Creates a new *giDrag* object for drag and drop. These objects represent the data being dragged and their types are represented in the mime data of the drag. The mime type of the dragged data is *application/x-<typeName>*.

This command should be called when the cursor has initiated a drag. Some Opal widgets support a callback procedure, called *-dragProc*, that can be used to identify when drags are started. Additionally, some non-Opal widgets support initiating drags by binding actions to the *Button1-DragStart* event type using the Binding Manager.

Destroying the *giDrag* object cancels the Drag and Drop operation.

Syntax

```
gi::createDrag -source <giAssistant | giWidget | giWindow> -data  
  <dbCollection>
```

Returns

<giDrag> : New object.

Arguments

-source

Specifies the initiating widget, window, or assistant. For non-Opal containers, the assistant or window is a shorthand for the central widget.

-data

Objects to drag.

Examples

The following example is a fully implemented example of the **Drag and Drop** operation for two list input widgets utilizing *gi::createDrag* and *gi::setDropAction* commands.

```
set dlg [gi::createDialog dragDrop -title "Drag & Drop"]  
set lst1 [gi::createListInput lst1  
  -parent $dlg  
  -selectionModel multiple  
  -dragProc _dragCell  
  -items {item1 item2 item3}]  
set lst2 [gi::createListInput lst2  
  -parent $dlg  
  -selectionModel multiple  
  -dragProc _dragCell]  
gi::setDropAction $lst1 -dropProc _dropCell -type giListInput  
gi::setDropAction $lst2 -dropProc _dropCell -type giListInput  
proc _dragCell {lst} {
```

```

        gi::createDrag -source $lst -data $lst
    }
    proc _dropCell {drag} {
        set src      [db::getattr drag.source]
        set trg      [db::getattr drag.target]
        set data      [db::getattr src.value]
        set sItems    [db::getattr src.items]
        set tItems    [db::getattr trg.items]

        foreach it $data {
            set sind [lsearch $sItems $it]
            set tind [lsearch $tItems $it]
            set sItems [lreplace $sItems $sind $sind]
            set tItems [lreplace $tItems $tind $tind]
            lappend tItems $it
        }
        db::setattr src.items -value $sItems
        db::setattr trg.items -value $tItems
    }
}

```

gi::createDynamicList

Description

Creates a new giDynamicList widget and adds it to the specified parent.

Syntax

```

gi::createDynamicList [<{none}>: string] [-parent <giAssistant |
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]
[-prefName <dbPrefName>] [-toolTip <string>] [-valueChangeProc
<variant>]] [-label <string>] [-value [<string>]] [-selection [<int>]]
[-editWidget <giWidget>] [-selectionChangeProc [<dbCore | giDialog |
string>]] [-readOnly <bool>] [-valid <bool>] [-required <bool>] [-unique
<bool>] [-contextMenuProc [<variant>]]

```

Arguments

{none}

Widget name. (Default is:)

-parent

Dialog, group, or other container into which this widget should be placed.

-enabled

Initial enabled state. (Default is: true)

-shown

Initial state shown. (Default is: true)

`-prefName`

Optional preference name within the scope of the top-level container, which provides the initial value. This value is automatically updated with changes according to the container update model. (Default is:)

`-toolTip`

Optional widget tooltip. (Default is:)

`-valueChangeProc`

Procedure to execute when the value is changed. Called with the widget object.

`-label`

Widget label, shown in the left column. (Default is:)

`-value`

Initial value.

`-selection`

List of initial selections.

`-editWidget`

Widget used to edit items in the list. If not set, a text input widget is assumed.

`-selectionChangeProc`

Procedure to execute when the selection is changed. Called with the widget object. Selection is retrieved from the attribute.

`-readOnly`

Initial read-only state. (Default is: false)

`-valid`

Initial valid state. (Default is: true)

`-required`

Specifies whether this input is required. (Default is: false)

`-unique`

Specifies whether the list should contain unique values. (Default is: false)

`-contextMenuProc`

Procedure to execute, when you right-click on this widget to request for a context menu.

Returns

New object (<giDynamicList>).

Examples

```
proc valueChangeProc {widget} {
    puts "Value change: [db::getattr widget.value]"
}

proc selectionChangeProc {widget} {
    puts "Selection Change: [db::getattr widget.selection]"
}

set dialog [gi::createDialog numberDialog -title "List Input"]
set items [db::createList [db::getattr name -of [dm::getCells -libName
    analogLib]]]
gi::createDynamicList
    -parent $dialog
    -label Cells
    -value $items
    -valueChangeProc valueChangeProc
    -selectionChangeProc selectionChangeProc
```

gi::createFileInput

Description

Creates a new giFileInput widget and adds it to the specified parent.

Syntax

```
gi::createFileInput [<{none}: string>] [-parent <giAssistant |
    giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]
    [-prefName <dbPrefName>] [-toolTip <string>] [-valueChangeProc
    <variant>] [-label <string>] [-value <string>] [-fileType <enum>]
    [-fileMasks [<string>]] [-mode <enum>] [-prompt <string>] [-relativePath
    <string>] [-defaultBrowsePath <string>] [-readOnly <bool>] [-valid
    <bool>] [-required <bool>] [-ignoreValidation <bool>]
```

Returns

New <giFileInput> object.

Arguments

{none}

Widget name.

`-parent <giContainer | giDialog>`

Dialog, group, or other container into which this widget should be placed.

`-enabled <bool>`

Initial enabled state.

Default: true.

`-shown <bool>`

Initial shown state.

Default: true.

`-prefName <string>`

Optional preference within the scope of the top-level container which provides the initial value. This value is automatically updated with changes according to the container update model.

`-toolTip`

Optional widget toolTip.

`-valueChangeProc <string>`

Procedure to execute when the value is changed. Called with the widget object.

`-label <string>`

Widget label, shown in the left column.

`-value <string>`

Initial value.

`-fileType <enum>`

Specifies the requested input: {directory | either file}.

Default: file.

`-fileMasks {<string>}`

List of file masks in the formatLabel (pattern) used for the browse dialog.

`-mode <enum>`

Initial file browsing mode: {open | save | select}.

Default: select.

`-prompt <string>`

Initial browse dialog prompt string.

Default: Select <label>.

`-relativePath`

Specifies an alternate path from which to provide completions and validation.

`-defaultBrowsePath`

Default path for the browse dialog in case one is not provided in the text input field.

`-readOnly <bool>`

Initial valid state.

Default: true.

`-valid <bool>`

Initial valid state.

Default: true.

`-required <bool>`

Specifies whether this is a required input.

Default: false.

`-ignoreValidation`

Specify this to ignore validation.

Default: false.

Examples

```
proc valueChangeProc {widget} {  
    puts [db::getattr widget.value]  
}  
  
set dialog [gi::createDialog instNaming -title "File Input"]  
gi::createFileInput -parent $dialog -mode select -fileType file  
    -label "Input File" -fileMasks {"Text File" *.txt} {"All" *.*}  
    -valueChangeProc valueChangeProc  
gi::createFileInput -parent $dialog -mode select  
    -fileType directory -label "Run Dir" -valueChangeProc valueChangeProc
```

gi::createGroup

Description

Creates a new giGroup widget and adds it to the specified parent.

Syntax

```
gi::createGroup [<{none}: string>] [-parent <giContainer | giDialog>]  
  [-enabled <bool>] [-shown <bool>] [-toolTip <string>] [-label <string>]  
  [-decorated <bool>]
```

Returns

<giGroup> New object.

Arguments

{none}

Widget name.

-parent

Dialog, group, or other container into which this widget should be placed.

-enabled

Initial enabled state.

Default: true.

-shown

Initial shown state.

Default: true.

-toolTip

Optional widget tooltip.

-label

Group box header. If set to an empty string, a frame is shown instead.

-decorated

Initial decorated state.

Default: true.

Examples

Example 1

```
proc btnExecProc {widget} {  
    set dialog [db::getattr widget.root]  
    puts "dialog: [db::getattr dialog.name]\n  
           [db::getattr widget.name] [db::getattr widget.label]"  
}  
  
set dialog [gi::createDialog groupsDialog -title "Groups"]
```

```
set gr1 [gi::createGroup -parent $dialog -label "Options"]
foreach label {"Run in Background" "Add Leabls" "Add Pins"} {
    set btnName [regsub -all {\s+} [string tolower $label] ""]
    gi::createBooleanInput $btnName -parent $gr1 -label $label
    -valueChangeProc btnExecProc
}
set gr2 [gi::createGroup -parent $dialog -label "Actions"]
foreach label {Run Stop "Add Vias"} {
    set btnName [regsub -all {\s+} [string tolower $label] ""]
    gi::createPushButton $btnName -parent $gr2 -label $label -execProc
    btnExecProc
}
gi::layout $gr2 -rightOf $gr1
```

Example 2

```
set gr [gi::createGroup -parent [gi::createDialog dlg -title test
    -stayOnTop 1 ]]
set cnt 5
while {$cnt > 0} {
    gi::createPushButton -parent $gr -label "PushButton"$cnt
    incr cnt -1
}
db::setAttr gr.spacing -value 0
db::setAttr gr.margins -value[ list 100 100 100 100]
```

gi::createInlineGroup

Description

Creates a new giInlineGroup container and adds it to the specified parent.

Syntax

```
gi::createInlineGroup [<{none}: string>] [-parent <giContainer |
    giDialog>] [-enabled <bool>] [-shown <bool>] [-toolTip <string>] [-label
    <string>]
```

Returns

<giInlineGroup>: New object.

Arguments

{none}

Widget name.

-parent <giContainer | giDialog>

Dialog, group, or other container into which this widget should be placed.

`-enabled <bool>`

Initial enabled state.

Default: true.

`-shown <bool>`

Initial state shown.

Default: true.

`-toolTip`

Optional widget tool tip.

Default: "".

`-label <string>`

Widget label, shown in the left column.

Examples

```
proc btnExecProc {widget} {
    set dialog [db::getattr widget.root]
    puts "dialog: [db::getattr dialog.name]\n
           [db::getattr widget.name] [db::getattr widget.label]"
}

set dialog [gi::createDialog inGroupsDialog -title "Inline Groups"]
set gr [gi::createInlineGroup -parent $dialog]
foreach label {Run Stop "Add Vias"} {
    set btnName [regsub -all {\s+} [string tolower $label] ""]
    gi::createPushButton $btnName -parent $gr -label $label -execProc
    btnExecProc
}

set gr [gi::createInlineGroup -parent $dialog]
foreach label {"Run in Background" "Add Leabls" "Add Pins"} {
    set btnName [regsub -all {\s+} [string tolower $label] ""]
    gi::createBooleanInput $btnName -parent $gr -label $label
    -valueChangeProc btnExecProc
}
```

gi::createLabel

Description

Creates a new giLabel widget and adds it to the specified parent.

Syntax

```
gi::createLabel [{none}: string] [-parent <giAssistant | giContainer  
| giDialog | giWindow>] [-enabled <bool>] [-shown <bool>] [-toolTip  
<string>] [-label <string>] [-icon <string>]
```

Returns

New <giLabel> object.

Arguments

{none}

Widget name.

-parent

Dialog, group, or other container into which to place this widget.

-enabled

Initial enabled state.

Default: true.

-shown

Initial state shown.

Default: true.

-toolTip

Optional widget tooltip.

Default: "".

-label

Label to show.

-icon

Icon filename, searched within the icon search path, which overrides -label.

Examples

This example adds three labels to the dialog object.

```
set dialog [gi::createDialog testDialog -title "Labels"]  
set w1 [gi::createLabel -label "Hello World!!!" -parent $dialog]  
# Show icon  
set w2 [gi::createLabel -icon "push_to_table" -parent $dialog]  
# Show HTML  
set w3 [gi::createLabel -label "<font color='#0000FF' size='15'>This is  
<font color=red><b>HTML</b></font> Text</font>" -parent $dialog]
```

gi::createListInput

Description

Creates a new giListInput widget and adds it to the specified parent.

Syntax

```
gi::createListInput [{<{none}: string}] [-parent <giAssistant |  
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]  
[-prefName <dbPrefName>] [-toolTip <string>] [-valueChangeProc  
[<variant>]] [-label <string>] [-value [<string>]] [-readOnly <bool>]  
[-valid <bool>] [-required <bool>] [-doubleClickProc [<variant>]]  
[-header <string>] [-filterLabel <string>] [-items [<string>]]  
[-showFilter <bool>] [-allowSort <bool>] [-selectionModel <enum>]  
[-viewType <enum>] [-showToggleButton <bool>] [-contextMenuProc  
[<variant>]] [-dragProc [<variant>]]
```

Returns

<giListInput>: New object

Arguments

{none}

Widget name.

Default: "".

-parent

Dialog, group or other container into which this widget should be placed.

-enabled

Initial enabled state.

Default: true.

-shown

Initial shown state.

Default: true.

-prefName

Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model.

Default: "".

`-toolTip`

Optional widget tooltip.

Default: "".

`-valueChangeProc`

Procedure to execute when the value is changed. Called with the widget object.

`-label`

Widget label, shown in the left column.

Default: "".

`-value`

Initial value.

`-readOnly`

Initial read-only state.

Default: false.

`-valid`

Initial valid state.

Default: true.

`-required`

Specifies that this is a required input.

Default: false.

`-doubleClickProc`

Procedure to execute when double clicking on a row. Called with the `giListInput` and the double-clicked item.

`-header`

Optional header title to show the above contained elements. If clicked, it sorts.

Default: "".

`-filterLabel`

Label shown to the left of the filter input. Allowed to be unset to remove the label.

Default: Filter.

`-items`

Items shown within the list.

`-showFilter`

Shows a filter field above the list.

Default: false.

`-allowSort`

If true, items are sorted. The default is true.

`-selectionModel`

Selection model: {single | multiple}.

Default: single.

`-viewType`

View type (list, checkbox or dualList) ({ checkbox | dualList | list }, default is: list)

`-showToggleButton`

Shows toggle buttons.

`-contextMenuProc`

Procedure to execute when you right-click on this widget to request for a context menu.

`-dragProc`

Procedure to invoke when a drag starts. Called with the widget object.

Examples

```
proc valueChangeProc {widget} {
    puts "Value change: [db::getAttr widget.value]"
}

proc doubleClickProc {widget value} {
    puts "Double click: $value"
}

set dialog [gi::createDialog listDialog -title "List Input"]
set items [db::createList [db::getAttr name -of [dm::getCells -libName
    analogLib]]]
set w1 [gi::createListInput
    -parent $dialog
    -items $items
    -label "List"
    -header "cells"
    -showFilter true
```



```
-selectionModel multiple
-valueChangeProc valueChangeProc
-doubleClickProc doubleClickProc ]
set w2 [gi::createListInput
-parent $dialog
-items $items
-label "Checkbox"
-header "cells"
-viewType checkbox
-showFilter true
-selectionModel multiple
-showToggleButton true
-valueChangeProc valueChangeProc
-doubleClickProc doubleClickProc]
set w3 [gi::createListInput
-parent $dialog
-items $items
-label "dualList"
-header "cells"
-viewType dualList
-showFilter true
-selectionModel multiple
-valueChangeProc valueChangeProc
-doubleClickProc doubleClickProc]
```

gi::createMenu

Description

This command creates a new menu. If the menu already exists, it is removed and replaced with the new, empty menu definition. The resulting menu may be destroyed using the `db::destroy` command.

A Tcl command may be registered to invoke immediately before displaying an instance of this menu definition. The registered procedure name is invoked with the Tcl object of the menu being shown and may be used to implement dynamic menus such as those that display recently opened files, windows, and so on.

The signature of the `preShow` Tcl command is as follows:

```
proc procName {menu} {}
```

Where the passed menu object represents the menu being opened.

Syntax

```
gi::createMenu <name: string> -title <string> [-preShow <string>]
[-dynamic <bool>] [-save <bool>] [-shown <bool>]
```

Returns

The newly created menu ([giMenu](#))

Errors

Name is not a valid menu name.

Arguments

`name`

Unique name for menu.

`-title`

Menu title displayed in menu bar.

`-preShow`

Procedure to invoke immediately before rendering this menu or any instance.
Procedure called with the menu Tcl object being rendered.

Default: "".

`-dynamic`

If true, indicates that the contents of this menu change automatically. Specifically excludes it from graphical customization.

Default: false.

`-save`

If true, the menu may be recreated by a Workspace object.

Default: false.

`-shown`

If true, indicates to show the menu.

Examples

```
gi::createMenu userMenu -title User
```

In this example `gi::createMenu` creates the User menu, which can be added to any window.

gi::createMutexInput

Description

Creates a new `giMutexInput` widget and adds it to the specified parent.

Syntax

```
gi::createMutexInput [<{none}: string>] [-parent <giAssistant |  
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]  
[-prefName <dbPrefName>] [-toolTip <string>] [-valueChangeProc  
[<variant>]] [-label <string>] -enum [<string>] [-viewType <enum>]  
[-value <string>] [-radioRows <unsigned int>] [-comboWidth <unsigned  
int>] [-dataType <giDataType | string>]
```

Returns

<giMutexInput> : New object.

Arguments

{none}

Widget name.

-parent

Dialog, group or other container into which this widget should be placed.

-enabled

Initial enabled state. The default is true.

-shown

Initial shown state. The default is true.

-prefName

Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model.

-toolTip

Optional widget tooltip.

-valueChangeProc

Procedure to execute when the value is changed. Called with the widget object.

-label

Widget label, shown in the left column.

-enum

Possible values.

-viewType

View type ({ auto | combo | radio }, default is: auto)

-value

Initial value.

-radioRows

Initial number of rows to show when multiple radio buttons are used. (Default is: 1)

-comboBoxWidth

Initial minimum number of character width to show in the combo box view. (Default is: 10)

-dataType

Data type for performing display to value conversions (Default is: string)

Examples

```
proc valueChangeProc {widget} {  
    puts "Value change: [db::getattr widget.value]"  
}  
  
set dialog [gi::createDialog mutexDialog -title "Mutex Inputs"]  
set w1 [gi::createMutexInput  
    -parent $dialog  
    -label "Radio Group"  
    -enum {1 2 3 4 5 6}  
    -radioRows 2  
    -viewType radio  
    -value 1  
    -valueChangeProc valueChangeProc]  
set w2 [gi::createMutexInput  
    -parent $dialog  
    -label "Combo"  
    -enum {User1 User2 User3}  
    -value User2  
    -viewType combo  
    -valueChangeProc valueChangeProc]
```

gi::createNumberInput

Description

Creates a new giNumberInput widget and adds it to the specified parent.

Syntax

```
gi::createNumberInput [<{none}>: string] [-parent <giAssistant |  
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]  
[-prefName <dbPrefName>] [-toolTip <string>] [-valueChangeProc  
[<variant>]] [-label <string>] [-value <string>] [-width <int>]
```

```
[-valueType <enum>] [-viewType <enum>] [-sliderLabels <enum>]  
[-sliderOrientation <enum>] [-minValue <string>] [-maxValue <string>]  
[-stepValue <string>] [-suffix <string>] [-readOnly <bool>] [-valid  
<bool>] [-required <bool>]
```

Returns

<giNumberInput>: New object.

Arguments

{none}

Widget name.

-parent

Dialog, group, or other container into which to place this widget.

-enabled

Initial enabled state.

Default is true.

-shown

Initial state shown.

Default is true.

-prefName

Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model.

-toolTip

Optional widget tooltip.

-valueChangeProc

Procedure to execute when the value is changed. Called with the widget object.

-label

Widget label, shown in the left column.

-value

Initial value

-width

Maximum width in characters to show at a time in the widget.

Default is 0.

`-valueType`

Value type of integer or floating point: {engineering | float | int}.

Default is int.

`-viewType`

Number input view type (text or slider). ({slider | text}.

Default is text.

`-sliderLabels`

Labels to show to the left/right of the slider { minCurrent | minMax | none }. The default is minMax.

`-sliderOrientation`

Slider orientation ({ horizontal | vertical }). The default is horizontal.

`-minValue`

Initial minimum value.

`-maxValue`

Initial maximum value.

`-stepValue`

Initial step value.

Default is 1.

`-suffix`

Suffix label to show to the right of the input, for example, *ms*.

`-readOnly`

Initial read-only value.

Default is false.

`-valid`

Initial valid state.

`-required`

Specifies this as a required input.

Default is false.

Examples

```
proc valueChangeProc {widget} {
    puts "Value change: [db::getattr widget.value]"
}

set dialog [gi::createDialog numberDialog -title "Number Inputs"]
set w1 [gi::createNumberInput
    -parent $dialog
    -label "Integer"
    -value 10
    -width 20
    -valueType int
    -minValue 0
    -maxValue 100
    -stepValue 1
    -valueChangeProc valueChangeProc]
set w2 [gi::createNumberInput
    -parent $dialog
    -label "Float"
    -value 10
    -width 20
    -valueType float
    -minValue 0
    -maxValue 100
    -stepValue 1
    -valueChangeProc valueChangeProc]
gi::createNumberInput
    -parent $dialog
    -label "Slider"
    -value 10
    -width 300
    -valueType int
    -viewType slider
    -minValue 0
    -maxValue 100
    -stepValue 1
    -suffix "ms"
    -valueChangeProc valueChangeProc
```

gi::createPointListInput

Description

Creates a new giPointListInput widget and adds it to the specified parent.

Syntax

```
gi::createPointListInput [<{none}>: string] [-parent <giContainer |
    giDialog>] [-enabled <bool>] [-shown <bool>] [-prefName <dbPrefName>]
    [-toolTip <string>] [-valueChangeProc [<variant>]] [-label
```

```
<string>] [-value <string>] [-readOnly <bool>] [-required <bool>]  
[-isRectilinearPolygon <bool>] [-showEditorButton <bool>]
```

Returns

New <giPointListInput> object.

Arguments

{none}

Widget name.

-parent

Dialog, group, or other container into which this widget should be placed.

-enabled

Initial enabled state.

Default: true.

-shown

Initial state shown.

Default: true.

-prefName

Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model.

-toolTip

Optional widget tooltip.

-valueChangeProc

Procedure to execute when the value is changed. Called with the widget object.

-label

Widget label shown in the left column.

-value

Initial value.

-readOnly

Initial read-only state.

-required

Specifies this is a required input. Default is *false*.

`-isRectilinearPolygon`

Specifies whether polygon should be checked to be rectilinear. Default is *false*.

`-showEditorButton`

Specifies whether edit button should be shown. Default is *true*.

Examples

```
proc valueChangeProc {widget} {  
    puts "Value change: [db::getAttr widget.value]"  
}  
  
set dialog [gi::createDialog scrollAreaDialog -title "Point List"]  
gi::createPointListInput -parent $dialog -label "Points" -valueChangeProc  
    valueChangeProc  
db::setAttr dialog.geometry -value 20%X40%+40%+30%
```

gi::createPushButton

Description

Creates a new giPushButton widget and adds it to the specified parent.

Syntax

```
gi::createPushButton [<{none}: string>] [-parent <giContainer |  
    giDialog>] [-enabled <bool>] [-shown <bool>] [-toolTip <string>] [-label  
    <string>] [-icon <string>] [-execProc [<variant>]]
```

Returns

New <giPushButton> object.

Arguments

`{none}`

Widget name.

`-parent`

Dialog, group, or other container into which this widget should be placed.

`-enabled`

Initial enabled state.

Default: true.

`-shown`

Initial state shown.

Default: true.

-toolTip

Optional widget tooltip.

-label

Button text.

-icon

Icon filename, searched within the icon search path.

-execProc

Procedure to execute when the button is pressed. Called with the button widget.

Examples

Example 1

```
proc btnExecProc {widget} {
    set dialog [db::getattr widget.root]
    puts "dialog: [db::getattr dialog.name]\n
             [db::getattr widget.name] [db::getattr widget.label]"
}

set dialog [gi::createDialog testDialog -title "Push Buttons"]
set gr [gi::createInlineGroup -parent $dialog]
foreach label {Run Stop "Add Vias"} {
    set btnName [regsub -all {\s+} [string tolower $label] ""]
    gi::createPushButton $btnName
        -parent $gr
        -label $label
        -execProc btnExecProc
}
}
```

Example 2

```
set gr [gi::createGroup -parent [gi::createDialog dlg -title test
    -stayOnTop 1 ]]
set cnt 5
while {$cnt > 0} {
    gi::createPushButton -parent $gr -label "PushButton"$cnt
    incr cnt -1
}
db::setattr gr.spacing -value 0
db::setattr gr.margins -value[ list 100 100 100 100]
```

gi::createRow

Description

Adds a row to the specified table.

Syntax

```
gi::createRow [-before <giRow> | -after <giRow>] -parent <giRow |  
giTable> [-label <string>] [-expanded <bool>] [-shown <bool>] [-height  
<int>]
```

Returns

giRow>: New object.

Arguments

-parent

The new row can be added to a table or as a child of an existing row to create a tree widget.

-label

Row header. Row headers are hidden for the table until at least one row has a non-empty label.

Default: " ".

-expanded

Indicates expansion state for a tree structure.

Default: false.

-shown

Initial shown state.

Default: true.

-before

Row position relative to parent.

-after

Row position relative to parent.

-height

Height of the row in a table.

Default: -1.

Examples

This example shows the name/value pairs of `::env` global array in the table widget.

```
set dialog [gi::createDialog testDialog -title "Create Row"]
# Create table
set table [gi::createTable testTable -parent $dialog
    -alternatingRowColors true ]
set col1 [gi::createColumn -parent $table -label "Name" -stretch false
    -readOnly true]
set col2 [gi::createColumn -parent $table -label "Value" -stretch true]
set c 1
foreach key [lsort [array name ::env]] {
    set row [gi::createRow -parent $table]
    db::setAttr row.label -value $c
    set cell [gi::getCells -row $row -column $col1]
    db::setAttr cell.value -value $key
    set cell [gi::getCells -row $row -column $col2]
    db::setAttr cell.value -value $::env($key)
    incr c
}
db::setAttr dialog.geometry -value "30%x60%+35%+20%"
```

gi::createScrollArea

Description

Creates a `giScrollArea` widget and adds it to the specified parent.

Syntax

```
gi::createScrollArea [<{none}>: string] [-parent <giAssistant |
    giContainer | giDialog | giWindow>] [-rowsToShow <unsigned int>]
    [-showHorizontalScrollBar <enum>] [-showVerticalScrollBar <enum>]
```

Returns

<giScrollArea> : New object.

Arguments

{none}

Widget name.

-parent

Dialog, group, or other container into which this widget should be placed.

`-rowsToShow`

Maximal row count which `giScrollArea` must show without adding scrollbars.
Default is 0 means, scroll area freely resizes in the vertical direction. Default is: 0.

`-showHorizontalScrollBar`

Shows horizontal scrollbar ({ auto | false | true }, default is: auto).

`-showVerticalScrollBar`

Shows vertical scrollbar ({ auto | false | true }, default is: auto).

Examples

```
set dialog [gi::createDialog scrollAreaDialog -title "Scroll Area"]
set sa [gi::createScrollArea -parent $dialog -rowsToShow 10
    -showHorizontalScrollBar false]
for {set i 0} {$i < 30} {incr i} {
    set w1 [gi::createPushButton -parent $sa -label "Corner [expr $i + 1]"]
    set w2 [gi::createMutexInput -parent $sa -value 1 -enum {1 2 3}
        -viewType combo]
    set w3 [gi::createMutexInput -parent $sa -value 2 -enum {1 2 3}
        -viewType combo]
    set w4 [gi::createTextInput -parent $sa -value ""]
    gi::layout $w2 -rightOf $w1
    gi::layout $w3 -rightOf $w2
    gi::layout $w4 -rightOf $w3
}
db::setAttr dialog.geometry -value 20%X30%+40%+35%
```

gi::createTabGroup

Description

Creates a new `giTabGroup` widget and adds it to the specified parent.

Syntax

```
gi::createTabGroup [<{none}: string>] [-parent <giContainer | giDialog>]
    [-enabled <bool>] [-shown <bool>] [-toolTip <string>] [-tabRaiseProc
    [<dbCore | giDialog | string>]]
```

Arguments

{none}

Widget name.

`-parent`

Dialog, group, or other container into which to place this widget.

`-enabled`

Initial enabled state.

Default: true.

`-shown`

Initial state shown.

Default: true.

`-toolTip`

Optional widget tooltip.

`tabRaiseProc`

Tcl expression to evaluate when a tab is raised.

Examples

```
set dialog [gi::createDialog dialog -title "Tab Group" -showHelp false]
set tabGroup [gi::createTabGroup tabGroup -parent $dialog]

# Mutex Input Group
set group [gi::createGroup -parent $tabGroup -label "Mutex Input Tab
Group"]
gi::createMutexInput -parent $group -enum {1 2 3} -label "First Input"
gi::createMutexInput -parent $group -enum {f s t} -viewType combo -label
"Second Input"

# File Input Checkable Group
set group [gi::createGroup -parent $tabGroup -label "File Input Tab
Group"]
set checkgroup [gi::createCheckableGroup -parent $group -label "File
Input Checkable Group"]
gi::createFileInput -parent $checkgroup -mode select -fileType file
gi::createFileInput -parent $checkgroup -mode select -fileType directory

# Push button Inline Group
set group [gi::createGroup -parent $tabGroup -label "Button Tab Group"]
set inlineGroup [gi::createInlineGroup -parent $group -label "Button
Inline Group"]
gi::createPushButton -label "Button1" -parent $inlineGroup
gi::createPushButton -label "Button2" -parent $inlineGroup
db::setAttr dialog.geometry -value 20%x30%+40%+35%
```

gi::createTable

Description

Creates a new giTable widget and adds it to the specified parent.

Syntax

```
gi::createTable [<{none}: string>] [-parent <giAssistant |  
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown  
<bool>] [-label <string>] [-readOnly <bool>] [-allowAddRows <bool>]  
[-allowMoveRows <bool>] [-allowMoveColumns <bool>] [-allowResizeColumns  
<bool>] [-allowSortColumns <bool>] [-allowHideColumns <bool>]  
[-allowFreezeColumns <bool>] [-alternatingRowColors <bool>]  
[-selectionModel <enum>] [-showAllContent <bool>] [-saveColumnConfig  
<bool>] [-editTrigger <enum>] [-selectionChangeProc [<variant>]]  
[-valueChangeProc [<variant>]] [-addRowProc [<variant>]]  
[-doubleClickProc [<variant>]] [-toolTip <string>] [-contextMenuProc  
<variant>]] [-dragProc [<variant>]]
```

Arguments

{none}

Widget name. (Default is:)

-parent

Dialog, group, or other container into which this widget should be placed.

-enabled

Initial enabled state. (Default is: true)

-shown

Initial state shown. (Default is: true)

-label

Widget label, shown in the left column. If a name is not specified, the widget is "unlabeled" and extends into the label column (Default is:)

-readOnly

Initial read-only state. (Default is: false)

-allowAddRows

If true, "Click to Add" appears in the last row of the table allowing to add additional rows. (Default is: false)

-allowMoveRows

If true, moves rows by dragging row headers. (Default is: false)

-allowMoveColumns

If true, moves columns by dragging column headers. (Default is: false)

-allowResizeColumns

Allows to resize columns. (Default is: false)

`-allowSortColumns`

If true, sorts rows by clicking column headers. (Default is: false)

`-allowHideColumns`

If true, right clicking on the column header shows a context menu allowing to toggle the show/hide state of each column. (Default is: false)

`-alternatingRowColors`

If true, shows rows with alternating background colors. Use this for large tables. (Default is: false)

`-selectionModel`

Default selection model. ({ multipleCells | multipleRows | singleCell | singleRow }, default is: singleRow)

`-showAllContent`

Show all contents of the table. (Default is: false)

`-saveColumnConfig`

Remembers the column width, order, and the shown and frozen states. (Default is: false)

`-editTrigger`

editTrigger mode. ({ doubleClick | select }, default is: select)

`-selectionChangeProc`

Procedure to execute when the selection is changed. Called with the widget object. Retrieve selection from attribute.

`-valueChangeProc`

Procedure to execute when the value of any cell is changed. Called with the widget object. Retrieve selection from attribute.

`-addRowProc`

Procedure to call when a row is added via "Click to add". Called with the new giRow object.

`-doubleClickProc`

Procedure to execute when a cell is double clicked. Called with the cell over when a double click was detected.

`-toolTip`

Optional widget toolTip.

`-contextMenuProc`

Procedure to execute when right-clicking on this widget to request for a context menu.

`-dragProc`

Procedure to invoke when a drag starts. Called with the widget object.

Returns

`<giTable>`: New object.

Examples

The following example shows files in `$SYNOPSIS_CUSTOM_LOCAL` as tree view.

```
proc createTree {path parent table} {
    set fileList [glob -nocomplain $path/*]
    foreach f $fileList {
        set row [gi::createRow -parent $parent -label " " ]
        set cell [gi::getCells -column [gi::getColumns -parent $table]
        -row $row]
        db::setAttr value -of $cell -value [file tail $f]
        if [file isdirectory $f] {
            createTree [file join $path [file tail $f]] $row $table
        }
    }
}

set d [gi::createDialog testDialog -title "Tree"]
set table [gi::createTable testTable -parent $d -alternatingRowColors
    true ]
set fc [gi::createColumn -parent $table -label "File name" -stretch
    true ]

createTree $::env(SYNOPSIS_CUSTOM_LOCAL) $table $table
```

gi::createTextInput

Description

Creates a new `giTextInput` widget and adds it to the specified parent.

Syntax

```
gi::createTextInput [<{none}>: string] [-parent <giContainer |
giDialog>] [-enabled <bool>] [-shown <bool>] [-prefName <dbPrefName>]
[-toolTip <string>] [-valueChangeProc [<variant>]] [-completions
<string>]] [-label <string>] [-value <string>] [-height <int>] [-width
<int>] [-echo <bool>] [-wordWrap <bool>] [-readOnly <bool>] [-valid
<bool>] [-required <bool>]
```

Returns

<giTextInput>: New object.

Arguments

`{none}`

Widget name.

`-parent`

Dialog, group, or other container into which to place this widget.

`-enabled`

Initial enabled state.

Default: true.

`-shown`

Initial state shown.

Default: true.

`-prefName`

Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model.

`-toolTip`

Optional widget tooltip.

Default: "".

`-valueChangeProc`

Procedure to execute when the value is changed. Called with the button widget.

`-completions`

Initial list of completions.

`-label`

Widget label, shown in the left column.

`-value`

Initial value.

`-height`

Maximum height, in characters, to show at a time in the widget. Defaults to 1, meaning a single-line text input is used. A value of 0 implies a vertically expanding text field.

Default: 1.

`-width`

Maximum width, in characters, to show at a time in the widget. 0 means the widget expands horizontally to fill the available space.

Default: 0.

`-echo`

Specifies whether asterisks are displayed instead of entered characters.

Default: false.

`-wordWrap`

Enables or disables word wrap in multi-line text inputs. Has no impact when height is equal to 1.

Default: true.

`-readOnly`

Sets the read-only state. When set to `true`, the background color of fields in the user interface are gray to indicate that they are read-only.

Default: false.

`-valid`

Initial valid state.

`-required`

Specifies that this is a required input.

Default: false.

`-parent`

Optional parent window.

Examples

```
proc valueChangeProc {widget} {  
    puts "Value change: [db::getattr widget.value]"  
}  
  
set dialog [gi::createDialog scrollAreaDialog -title "Text Inputs"]
```

```
gi::createTextInput -label "Single-Line" -parent $dialog -completions  
    [info commands] -valueChangeProc valueChangeProc  
gi::createTextInput -label "Password" -parent $dialog -valueChangeProc  
    valueChangeProc -echo true  
gi::createTextInput -label "Multi-Line" -parent $dialog -height 10 -value  
    [exec custom_compiler -help] -valueChangeProc valueChangeProc  
db::setAttr dialog.geometry -value 40%X40%+30%+30%
```

gi::createTheme

Description

Create a new theme, optionally based on an existing one returned by `gi::getThemes`.

Syntax

```
gi::createTheme <name: string> [-copy <giTheme>]
```

Returns

<giTheme> : New giTheme

Arguments

name

Theme name.

-copy

Copy colors from an existing theme.

Examples

The following example creates a Dark giTheme object copying all attributes from the Light giTheme. It automatically adds a new Theme to Theme combo box of the **General Options** dialog box Interface section. Later, you can update normal, inactive and disabled giThemeState object attributes.

```
set darkTheme [gi::createTheme Dark -copy [gi::getThemes Light]]  
db::setAttr darkTheme.normal.windowTitleColor -value #333  
db::setAttr darkTheme.inactive.windowTitleColor -value #333
```

gi::createToolbar

Description

Creates a new toolbar and returns its Tcl object. One may specify whether the toolbar supports specific orientations. Default toolbar position should be defined in the window type. If a toolbar with the same name already exists, it is removed and replaced with

the new, empty toolbar. The resulting toolbar may be destroyed using the `db::destroy` command.

Syntax

```
gi::createToolbar <name: string> [-title <string>] [-orientation <enum>]  
[-dynamic <bool>] [-save <bool>]
```

Returns

Newly created toolbar ([giToolbar](#)).

Errors

Invalid toolbar name.

Arguments

`name`

Unique name for the toolbar.

`-title`

Toolbar title.

Default: "".

`-orientation`

Restricts toolbar orientation: {both | horizontal | vertical}.

Default: both.

`-dynamic`

If true, indicates that the contents of this toolbar change automatically.
Specifically excludes it from graphical customization.

Default: false.

`-save`

If true, the toolbar may be recreated by a Workspace object.

Default: false.

Examples

```
gi::createToolbar oftenUseCommands -title "User Commands"
```

In this example, `gi::createToolbar` creates the User Commands toolbar, which can be a container for actions, and can be added to a specified window or all windows of a window type.

gi::createWebView

Description

Creates a new giWebView widget and adds it to the specified parent.

Syntax

```
gi::createWebView [{<none>: string}] [-parent <giAssistant | giContainer  
| giDialog | giWindow>] [-label <string>] [-enabled <bool>] [-shown  
<bool>] -source <string> [-showNavigation <bool>] [-showLocation <bool>]  
[-newWindowProc [<variant>]]
```

Returns

<giWebView> : New object.

Arguments

{none}

Widget name.

-parent

Dialog, group or other container into which this widget should be placed.

-label

Widget label, shown in the left column.

-enabled

Initial enabled state. The default is true.

-shown

Initial shown state. The default is true.

-source

HTML File or Url.

-showNavigation

Specifies if navigation is shown. The default is false.

-showLocation

Specifies if location is shown. The default is false.

-newWindowProc

Procedure to execute when creating new window.

Examples

```
set dialog [gi::createDialog scrollAreaDialog -title "Web View Area"]
gi::createWebView -parent $dialog -source "https://google.com"
    -showNavigation true -showLocation true
db::setAttr dialog.geometry -value 40%X40%+30%+30%
```

gi::createWindow

Description

Creates a window of a window type if the specified window type was created in Tcl.

Syntax

```
gi::createWindow [-windowType <giWindowType>] [-showTabs <bool>] [-parent  
    <giWindow>]
```

Returns

<giWindow> : Newly created window.

Arguments

-windowType

Window type of which to create window. If not specified, a window of type specified by giNewWindowType preference is created.

-showTabs

Indicates if window may be placed in a frame with more than one window. The default is true.

-parent

Parent window.

Examples

The following example creates and opens the customWindow calling by the decorateWindow procedure, specified at the windowType creation command, to decorate window with widgets.

```
proc decorateWindow {win} {
    gi::createLabel -parent $win
        -label "<h1>This is Custom Window</h1>"
}

gi::createWindowType customWindow
    -createProc decorateWindow -title "Custom Window"
gi::createWindow -windowType [gi::getWindowTypes customWindow]
```

gi::createWorkspace

Description

Creates a new workspace object from one or more open windows.

Syntax

```
gi::createWorkspace <name: string> (-window <giWindow | unsigned int> |  
-windowType <giWindowType | string>) -title <string> [-default <bool>]  
[-initProc [<variant>]] [-doneProc [<variant>]]
```

Arguments

{name}

Name, unique identifier.

-title

Title to show in the menu.

-window

Window layout to capture in the workspace.

-windowType

Window type for which to create workspace.

-default

Make the workspace the default. (Default is: false)

-initProc

The initProc is called after the workspace layout has been applied.

-doneProc

The doneProc is called before removing the workspace layout (and applying some other workspace).

Returns

<giWorkspace>: Newly created workspace object.

Examples

The following example creates a new *myLayout* workspace. Other commands are used to configure workspace and add Design Navigator to the new workspace.

```
set ws [gi::createWorkspace myLayout  
-title "My Layout"  
-windowType leLayout
```



```
        -default false]  
gi::addAssistant [gi::getAssistants heDesignNavigator] -to $ws
```

gi::createX11Window

Description

Creates a new giX11Window widget and adds it to the specified parent.

Syntax

```
gi::createX11Window [-parent <giAssistant | giContainer | giDialog |  
    giWindow>]
```

Arguments

-parent

Dialog, group, or other container into which this widget should be placed.

Returns

<giX11Window> : New giX11Window object.

Examples

Example 1

```
db::enableTk  
set dialog [gi::createDialog tkDialog -title TK]  
  
# Create giX11Window container widget  
set x1lwin [gi::createX11Window -parent $dialog]  
  
# Add Tk widget into giX11Window container widget  
toplevel .top -use [db::getAttr id -of $x1lwin]  
canvas .top.myCanvas -width 400 -height 200  
.top.myCanvas create text 180 50 -text "My Canvas" -fill magenta -font  
    {Helvetica -50 bold}  
.top.myCanvas create oval 50 130 100 160 -fill yellow  
.top.myCanvas create rectangle 220 120 300 180 -fill yellow  
grid .top.myCanvas -row 0 -column 0  
db::setAttr dialog.geometry -value 425x250
```

Example 2

```
db::enableTk  
set dialog [gi::createDialog xtermDialog -title XTERM]  
  
# Create giX11Window container widget  
set x1lwin [gi::createX11Window -parent $dialog]  
  
# Add xterm into giX11Window container widget
```

```
exec xterm -into [db::getAttr id -of $x11win] -fa 'Monospace' -fs 14 &  
db::setAttr dialog.geometry -value 690x375
```

gi::drop

Description

Replays a drag and drop operation.

Syntax

```
gi::drop <indexes: [string]> -in <giAssistant | giDialog | giToolbar  
| giWindow> -target <string> -position <point> [-enDropAction <enum>]  
[-source <string>] [-sourceContainer <giAssistant | giDialog | giToolbar  
| giWindow>]
```

Arguments

`indexes`

Indexes of items to drag and drop.

`-in`

Dialog, toolbar, assistant, or window that contains the target widget.

`-target`

Target widget name within container to drop items.

`-position`

Point where the drop occurs.

`-enDropAction`

Proposed drop action ({ copy | ignore | link | mask | move }, default is: ignore)

`-source`

Source widget name.

`-sourceContainer`

Dialog, toolbar, assistant or window that contains source widget.

Returns

<none>

gi::execDialog

Description

Waits until the dialog is closed and returns false if cancelled, true if closed by any other button.

Syntax

```
gi::execDialog <{none}>: giDialog>
```

Returns

<bool>: True if dialog accepted, false if cancelled.

Arguments

{none}

Dialog on which to wait.

Examples

```
proc dialogExecProc {dialog} {  
    puts dialogExecProc  
}  
  
set dialog [gi::createDialog testDialog -title "Test" -execProc  
    dialogExecProc]  
set w1 [gi::createLabel -label "Hello World!!!" -parent $dialog]  
set res [gi::execDialog $dialog]  
puts $res
```

gi::executeAction

Description

Executes a giAction object as if its menu or toolbar entry/button were pressed. The action may be an internally or externally defined Tcl action or an internally defined 'item action'. Note that item actions often prompt for input with modal dialogs which prevent further execution of the Tcl script until dismissed. These dialogs may replay using a log file, but cannot be dismissed from Tcl at this time.

If the specified action is a checkable action, it is toggled.

Although clicking on the action within a menu or toolbar may have resulted in one or more Tcl commands being logged, when executed through this command these commands are not logged. Only the `gi::executeAction` command and any modifications made to a resulting dialog are logged.

Syntax

```
gi::executeAction <action: giAction | giActionName> -in <deCanvas  
| giAssistant | giDialog | giWindow | int>
```

Arguments

action

Action to execute.

-in

Assistant, window, canvas or dialog in which to execute the action.

gi::expand

Description

Expands an item in a dialog, assistant, window, or toolbar.

Syntax

```
gi::expand <name: string> -in <giAssistant | giBanner | giContainer |  
giDialog | giStatusBar | giToolbar | giWindow> -index <string>
```

Arguments

name

Name of a tree widget within container to expand given item.

-in

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

-index

Item index in the tree widget to expand.

gi::exportTable

Exports table data

Syntax

```
gi::exportTable <{none}: string> -parent <giAssistant | giDialog |  
giWindow> -filePath <string>
```

Returns

<none>

Arguments

`{none}`

Table name.

`-parent`

Parent window or dialog box.

`-filePath`

File path to export table data.

gi::findChild

Description

Find and returns a child widget object from a container. To specify an absolute path relative to the container, use a slash (/) for the first character. Relative paths may also be specified. The command returns an error if more than one match is found within the container.

Syntax

```
gi::findChild <none: string> -in <giContainer | giDialog>
```

Returns

`<dbCore | string>`: Object or attribute value with the specified name, or "" if it could not be found.

Arguments

`none`

Widget name to find.

`-in`

Container object to search.

Examples

```
proc execProc {dialog} {  
    set runBg [gi::findChild runBg -in $dialog]  
    if {[db::getAttr runBg.value]} {  
        set tclfile [gi::findChild tclfile.value -in $dialog]  
        puts $tclfile  
    } else {  
        set view [gi::findChild view.value -in $dialog]  
        puts $view  
    }  
}
```

```
set dialog [gi::createDialog test -title "Find Child" -execProc execProc]
gi::createBooleanInput runBg -parent $dialog -label "Run in Background"
gi::createFileInput tclfile -parent $dialog -label "Tcl file"
gi::createMutexInput view -parent $dialog -enum {both schematic layout}
-value both -viewType combo
```

gi::getActionCategories

Description

Returns a collection of action categories, optionally restricted to a given windowType.

Syntax

```
gi::getActionCategories [<expression: string>] [-windowType
<giWindowType>] [-filter <string>]
```

Returns

{<giActionCategory>}: Collection of matching action categories.

Arguments

`expression`

A wildcard-style pattern to match action categories name against

Default: `*`.

`-windowType`

Optionally restricts the returned collection to those categories for the specified window type.

`-filter`

Standard attribute filter.

Examples

Example 1

The following example returns existing action categories in the Layout Editor and then prints the title.

```
set actionsCategories [gi::getActionCategories
    -windowType [gi::getWindowTypes leLayout]]
db::createList [db::getAttr actionsCategories.title]
```

Example 2

The following example returns the **Edit** action category object in the Layout Editor.

```
set editCat [gi::getActionCategories Edit -windowType [gi::getWindowTypes  
leLayout]]
```

gi::getActions

Description

Returns a collection of globally defined actions or references to actions defined within the system. Callers may retrieve action references by specifying a parent container rather than global definitions.

Syntax

```
gi::getActions [<expression: string>] [-of <giStatusBar | giWindow |  
{<giMenu>} | {<giToolbar>}>] [-filter <string>] [-includeDeprecated  
<bool>]
```

Returns

Collection of actions ({<giAction>}).

Arguments

expression

A wildcard-style pattern to match action name against.

Default: *

-of

Containers from which to return action references. By default, action definitions are returned.

Default: "".

-filter

Standard attribute filter.

Default: Match all.

-includeDeprecated

Whether to include deprecated actions in the result.

Examples

```
gi::getActions dm* -from [gi::getMenus dmLibraryContextMenu]
```

In this example, `gi::getActions` uses `dm*` name glob-style matching to return all matching actions found in the library view list in the Library Manager.

gi::getActiveDialog

Description

Returns the most recently focused dialog in the system, or returns "" if the dialog has been since destroyed or no dialog has been focused.

Syntax

```
gi::getActiveDialog
```

Returns

Most recently focused giDialog object or "" if not found (*giDialog*).

gi::getActiveTab

Description

Returns the active tab for the given tab widget.

Syntax

```
gi::getActiveTab <name: string> -in <giAssistant | giBanner | giContainer  
| giDialog | giStatusBar | giToolbar | giWindow>
```

Returns

Active tab name (<string>).

Arguments

name

Tab widget name to get the active tab.

-in

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

Errors

- Tabbed widget with given name does not exist within the dialog or assistant.
- Specified widget is not a tabbed widget.

gi::getActiveWindow

Description

Returns the active window. Note that this is not necessarily an editor window.

Syntax

```
gi::getActiveWindow
```

Returns

Active window ([giWindow](#)).

Errors

No windows open.

Examples

```
set window [gi::getActiveWindow]
# Set custom title
db::setAttr window.userTitle -value "Project 1"
# Set geometry of parent frame
db::setAttr window.frame.geometry -value 1600x800+100+100
```

gi::getAppIcons

Description

Returns all giAppIcon objects.

Syntax

```
gi::getAppIcons [<pattern: string>] [-filter <string>]
```

Arguments

`pattern`

Glob-style pattern matched against the icon name. (Default is: *)

`-filter`

Standard attribute filter.

Returns

<giAppIcon> : A collection of matching icons

Examples

The following example returns a list of app icon names.

```
db::createList [db::getAttr name -of [gi::getAppIcons]]
dmLibraryManager saConsole dbFindReplace xtJobMonitor giConsole deHelp
deWhatsNew dcReviewManager xtDtco
```

gi::getAssistants

Description

Returns a collection of global assistant definitions or references within a container.

Syntax

```
gi::getAssistants [<assistant: string>] [-of <giWindow | giWindowType |  
giWorkspace>] [-filter <string>]
```

Returns

Collection of dockable assistants either defined globally or referenced by a container ({<giAssistant>}).

Arguments

`assistant`

A glob-style pattern to match assistant name against.

Default: all assistant names

`-of`

Container of menus from which to return assistants. Default is to return definitions.

Default: "".

`-filter`

Standard attribute filter.

Default: Match all.

Examples

```
gi::getAssistants -from [db::getattr window -of \  
[de::getActiveContext]] -filter %shown==1
```

In the above example, `gi::getAssistants` returns all assistants; they are shown in the current active editor window.

```
set assistants [gi::getAssistants]  
db::createList [db::getattr assistants.name]
```

In the above example, `gi::getAssistants` returns a collection of all existing assistants, and the `db::createList` command returns the names of those assistants.

gi::getBindings

Description

Returns a collection of all bindings within a binding set, optionally filtered.

Syntax

```
gi::getBindings [-set <giBindingSet | string>] [-filter <string>]
```

Returns

{<giBinding>}: A collection of all bindings within a binding set.

Arguments

`-set`

The binding set or the name of the binding set from which to retrieve bindings.

`-filter`

Optionally filter expression.

Examples

```
gi::getBindings -set Maestro -filter {%event == "Shift-p" && [db::getAttr  
  name -of %container] == "leLayout" }
```

In this example, `gi::getBindings` returns the binding definition bound to the Shift+P key event from the "Maestro" binding definition set, as applied in the Layout Editor.

gi::getBindingSets

Description

Returns a collection of existing binding sets, optionally filtered.

Syntax

```
gi::getBindingSets [<name: string>] [-filter <string>]
```

Arguments

`name`

Optional glob-style pattern to match against binding set names and filter the returned list.

`-filter`

Optionally copy an existing binding set into the new set.

Examples

The following example returns a collection of existing bind sets and the name binding sets.

```
set bindSets [gi::getBindingSets]
db::createList [db::getattr bindSets.name]
```

gi::getCells

Description

Retrieves a collection of cell objects from a table, row and/or column object.

Syntax

```
gi::getCells [-parent <giTable>] [-row <giRow>] [-column <giColumn>]
```

Returns

<dbCollection>: A collection of matching cells.

Arguments

-parent

Table from which to retrieve cells.

-row

Retrieve cells from the specified row.

-column

Retrieve cells from the specified column.

Examples

The following example returns giCell, which is the intersection cell of column B and row 2.

```
set dlg [gi::createDialog sampleDialog
    -title "Sample Dialog"]
set tbl [gi::createTable tbl -parent $dlg ]
foreach col {A B C} {
    gi::createColumn -parent $tbl -label $col
}
foreach row {1 2 3} {
    gi::createRow -parent $tbl -label $row
}
set col [gi::getColumns -parent $tbl -filter {%label == "B"}]
set row [gi::getRows -parent $tbl -filter {%label == "2"}]
set giCell [gi::getCells -column $col -row $row]
```

gi::getChildren

Description

Returns collection of children widgets, found hierarchically.

Syntax

```
gi::getChildren [<pattern: string>] -parent <giAssistant | giContainer |  
giDialog | giWindow> [-filter <string>]
```

Returns

<dbCollection>: Collection of children widgets.

Arguments

pattern

Glob-style pattern to match against widget name. Default is: "".

-parent

Container object to search.

-filter

Standard attribute filter.

Examples

The following example returns children widgets of the dialog box and the widget names.

```
set dlg [gi::createDialog sampleDialog -title "Sample Dialog"]  
set gr [gi::createGroup group -parent $dlg -label "Files" \  
-decorated true]  
gi::createFileInput fileWidget -parent $gr -fileType file  
set fileChild [gi::getChildren -parent $dlg]  
db::createList [db::getAttr fileChild.name]
```

The following example returns the **Close** button from the dialog box and the type of widget.

```
set closeButton [gi::getChildren close \  
-parent [db::getAttr dlg.buttonBar]]  
db::getAttr closeButton.type
```

gi::getColumns

Description

Returns all columns in the specified table in display order, optionally filtered by a standard attribute filter string.

Syntax

```
gi::getColumns -parent <giDbTable | giTable> [-filter <string>]
```

Returns

<dbCollection>: A collection of matching columns.

Arguments

-parent

Parent table.

-filter

Standard attribute filter.

Examples

The following example returns column which label is B.

```
set dlg [gi::createDialog sampleDialog
        -title "Sample Dialog"]
set tbl [gi::createTable tbl -parent $dlg ]
foreach col {A B C} {
    gi::createColumn -parent $tbl -label $col
}
foreach row {1 2 3} {
    gi::createRow -parent $tbl -label $row
}
set col [gi::getColumns -parent $tbl -filter {%label == "B"}]
```

The following example prints the columns in a table.

```
db::foreach column [gi::getColumns -parent $table] {
    puts [db::getattr column.label]
}
```

The following example finds a column by its label.

```
set column [gi::getColumns -parent $table -filter {%label == "Name"}]
```

gi::getContextMenu

Description

This command returns the context menu associated with a given context name. For editor windows, specify an object type name.

Syntax

```
gi::getContextMenu [<name: string>] -windowType <giWindowType
| giAssistant>
```

Returns

Menu associated with the given context name or "" if no menu is defined (giMenu)

Arguments

name

Context name with which this menu is associated. For editor windows, specify the object type name.

Default: "".

-windowType

Window type or assistant in which this menu is registered.

Examples

```
gi::getContextMenu dmCellCSM -windowType \  
    [gi::getWindowTypes dmLibraryManager]
```

In this example: `gi::getContextMenu` returns the context sensitive menu object for the Library Manager cell list view.

gi::getDataTypes

Description

Returns a collection of defined data types; optionally restricted by their names.

Syntax

```
gi::getDataTypes [<name: string>] [-filter <string>]
```

Returns

<dbCollection>: A collection of matching data types.

Arguments

pattern

Optional glob-style pattern which restricts the returned collection to those data types matching this pattern.

-filter

Standard attribute filter.

Examples

The following example returns a collection of all registered datatype objects in the current session and the list of datatype names.

```
set dataTypes [gi::getDataTypes ]
db::createList [db::getattr dataTypes.name]
```

gi::getDialogs

Description

Returns a collection of open dialogs, optionally restricted by their name and parent window or dialog. Names are unique for a given parent window. If the parent is not specified, parentless dialogs are returned.

Syntax

```
gi::getDialogs [<name: string>] [-parent <giDialog | giWindow>] [-filter  
  <string>]
```

Returns

Collection of dialogs optionally restricted by the -name pattern. (*{<giDialog>}*).

Arguments

<string>

Glob-style pattern to match the dialog name against.

Default: (match all).

-parent

Dialogs whose parent is the specified window or dialog.

Default: "".

-filter <string>

Standard attribute filter.

Default: match all.

Examples

```
gi::getDialogs dmCopyCell -parent [gi::getWindows -type \  
  [gi::getWindowTypes dmLibraryManager]]
```

In this example, `gi::getDialogs` returns the Copy Cell dialog object, if it is open.

gi::getField

Description

Presses button in dialog, assistant, window or toolbar.

Syntax

```
gi::getField <name: string> -in <giAssistant | giBanner | giContainer |  
giDialog | giStatusBar | giToolbar | giWindow> [-index <string>]
```

Returns

Field value (<string>)

Arguments

`name`

Widget name to get.

`-in`

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

`-index`

Specifies the index into a list, tree or table to retrieve. Required for these widgets.

Default: "".

Errors

- Widget with given name does not exist within the dialog, toolbar or assistant.
- Invalid value.

gi::getFrames

Description

Returns a collection of all the frames.

Syntax

```
gi::getFrames [<index: int>] [-filter <string>]
```

Returns

{<giFrame>} : Collection of frames.

Arguments

`index`

A pattern matching frame id.

`-filter`

The standard attribute filter.

Examples

This example prints the Id and geometry of each `giFrame` object in the console.

```
db::foreach frame [gi::getFrames] {  
    puts "ID = [db::getattr frame.id] Geometry: [db::getattr  
    frame.geometry]"  
}
```

gi::getMenus

Description

Returns a collection of globally defined menu definitions or references within a container (window type or window). If a container is specified, the set is ordered by position.

Syntax

```
gi::getMenus [<expression: string>] [-from <dbCollection | {<giWindow>} |  
    {<giWindowType>}>] [-filter <string>]
```

Returns

Collection of menus defined globally or referenced by a container (`{<giMenu>}`).

Arguments

`expression`

A glob-style pattern to match menu name against.

Default: all menu names.

`-from`

Window or window type from which to return menu references. By default, definitions are returned.

Default: "".

`-filter`

Standard attribute filter.

Default: Match all.

Examples

```
gi::getMenus dmFile* -from [gi::getWindowTypes dmLibraryManager]
```

In this example, `gi::getMenus` returns the `dmFileMenu` object.

gi::getRows

Description

Returns all rows in the specified table or those which are children of a row within a tree.

Syntax

```
gi::getRows -parent <giRow | giTable> [-filter <string>]
```

Returns

<dbCollection>: A collection of matching rows.

Arguments

-parent

Parent table or hierarchical row.

-filter

Standard attribute filter.

Examples

The following example returns all row objects in the parent table.

```
set dlg [gi::createDialog sampleDialog  
        -title "Sample Dialog"]  
set tbl [gi::createTable tbl -parent $dlg ]  
foreach col {A B C} {  
    gi::createColumn -parent $tbl -label $col  
}  
foreach row {1 2 3} {  
    gi::createRow -parent $tbl -label $row  
}  
set rows [gi::getRows -parent $tbl]
```

gi::getSectionSizes

Description

Gets splitter or column stretch information.

Syntax

```
gi::getSectionSizes <name: string> -in <giAssistant | giBanner |  
giContainer | giDialog | giStatusBar | giToolbar | giWindow>
```

Returns

[<int>]: Section sizes.

Arguments

name

Widget name within container to get section sizes.

-in

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

Examples

The following example returns table header section sizes.

```
set dlg [gi::createDialog sampleDialog  
-title "Sample Dialog"]  
set tbl [gi::createTable tbl -parent $dlg  
-allowResizeColumns 1]  
gi::createRow -parent $tbl  
foreach col {A B C} {  
    set col$col [gi::createColumn -parent $tbl -label $col]  
}  
gi::getSectionSizes tbl -in $dlg
```

gi::getThemes

Description

Returns a collection of existing giTheme objects.

Syntax

```
gi::getThemes [<name: string>] [-filter <string>]
```

Returns

<dbCollection> : Collection of giThemes

Arguments

name

A glob-style pattern matching the theme name.

`-filter`

Standard attribute filter.

Examples

The following example returns all available Themes and their names.

```
set themes [gi::getThemes]
db::createList [db::getattr themes.name]
```

gi::getToolbars

Description

Returns a collection of global toolbar definitions or references within a container (window type or window).

Syntax

```
gi::getToolbars [<expression: string>] [-of <dbCollection | {<giWindow>}  
| {<giWindowType>}}] [-filter <string>]
```

Returns

Collection of toolbars either defined globally or referenced by a container ({<giToolbar>}).

Arguments

`expression`

A glob-style pattern to match toolbar name against.

Default: all toolbar names.

`-of`

Containers from which toolbars should be returned. Default is to return definitions.

Default: "".

`-filter`

Standard attribute filter.

Default: Match all.

Examples

```
gi::getToolbars -from [gi::getWindowTypes leLayout] -filter \  
%shown==1
```

In this example `gi::getToolbars` returns collection of all toolbars they are shown in the layout editor.

gi::getWindows

Description

Returns a collection of open windows.

Syntax

```
gi::getWindows [<index: string>] [-type <{<giWindowType>}>] [-frame  
  <giFrame | int>] [-filter <string>]
```

Returns

Collection of open windows, with names or ids matching the pattern (`{<giWindow>}`).

Arguments

`index`

A pattern matching window id.

Default: All open window.

`-type`

Return only windows of specified window types.

Default: All window type.

`-frame`

Return only windows contained within the specified frame.

`-filter`

Standard attribute filter.

Default: Match all.

Examples

```
gi::getWindows -type [gi::getWindowTypes leLayout]
```

In this example, `gi::getWindows` returns the collection of all opened layout windows.

gi::getWindowTypes

Description

Returns a collection of defined window types.

Syntax

```
gi::getWindowTypes [<name: string>] [-filter <string>]
```

Returns

Collection of window types (`{<giWidget>}`).

Arguments

`name`

A glob-style pattern matching window type name.

Default: All window types.

`-filter`

Standard attribute filter.

Default: Match all.

Examples

```
gi::getWindowTypes le*
```

In this example, `gi::getWindowTypes` returns the `leLayout` window type.

gi::getWorkspaces

Description

Returns a collection of defined workspaces.

Syntax

```
gi::getWorkspaces [<expression: string>] [-windowType <giWindowType>]  
[-filter <string>]
```

Returns

`<dbCollection>`: Collection of workspaces.

Arguments

`expression`

A glob-style pattern matching the workspace name.

Default: `*`.

`-windowType`

Window type filter.

`-filter`

Standard attribute filter.

Examples

The following example returns a collection of workspaces in the Layout Editor and the name of workspaces.

```
set ws [gi::getWorkspaces -windowType [gi::getWindowTypes leLayout]]
db::createList [db::getAttr ws.name]
```

gi::getXidUsage

Description

Retrieves xid usage statistics

Syntax

```
gi::getXidUsage <type: enum>
```

Arguments

type

type : resource type ({ GCs | all | dump | glyphsets | pictures | pixmaps | windows })

gi::hideContextMenu

Description

Hides any open context menu in any window.

Syntax

```
gi::hideContextMenu
```

gi::layout

Description

Redefines a widget's position and layout attributes within its container. Widgets are allowed to be aligned with other widgets in the same container.

Syntax

```
gi::layout <{none}: giWidget> [-before <giWidget> | -after <giWidget> |  
-leftOf <giWidget> | -rightOf <giWidget>] [-align <giWidget>] [-justify  
<enum>] [-equalWidth <bool>]
```


Arguments

`{none}`

Widget or container to layout.

`-align`

Optionally horizontally-align this widget with another in the same container. This effectively creates a column.

`-justify`

Justify the widget against the position of its column: {center | left | right}. Default: left.

`-equalWidth`

Specifies that this widget should be the same size as the widget to which it is relative. Requires `-leftOf` or `-rightOf`.

Default: false.

`-before`

Place this widget in a new row before another widget.

`-after`

Place this widget in a new row before after widget.

`-leftOf`

Place this widget in the same row as another widget and to its left.

`-rightOf`

Place this widget in the same row as another widget and to its right.

Examples

```
set dialog [gi::createDialog scrollAreaDialog -title "Layout"]
set highlight [gi::createLabel -parent $dialog -label "Highlight"]
set crossView [gi::createBooleanInput -parent $dialog -label "Cross
View"]
set unplaced [gi::createBooleanInput -parent $dialog -label "Unplaced"]
set crossSelection [gi::createLabel -parent $dialog -label "Cross
Selection"]
set fromLayout [gi::createBooleanInput -parent $dialog -label "From
Layout"]
set fromSchematic [gi::createBooleanInput -parent $dialog -label "From
Schematic"]

gi::layout $highlight -justify right
gi::layout $crossView -rightOf $highlight
gi::layout $unplaced -rightOf $crossView
```

```
gi::layout $fromLayout -rightOf $crossSelection -align $crossView  
gi::layout $fromSchematic -rightOf $fromLayout -align $unplaced
```

gi::loadBindings

Description

Loads bindings from an XML file into the specified set.

Syntax

```
gi::loadBindings -filePath <string> [-set <giBindingSet | string>]
```

Arguments

-filePath

Filename to write.

-set

Binding set into which bindings are created.

Examples

```
gi::loadBindings -filePath newBindings.xml -set Maestro
```

In this example, `gi::loadBindings` loads bindings from a `newBindings.xml` file into the Maestro binding set.

gi::loadWorkspaces

Description

Loads a collection of workspaces from a file on disk.

Syntax

```
gi::loadWorkspaces -filePath <string>
```

Arguments

-filePath

File path to save.

Returns

<dbCollection>: Collection of workspaces that were loaded.

Examples

```
# Create new workspace
set ws [gi::createWorkspace myworkspace -windowType [gi::getWindowTypes
  seSchematic] -title "My Workspace"]
# Add assistants
gi::addAssistant [gi::getAssistants heDesignNavigator] -to $ws
gi::addAssistant [gi::getAssistants seNetTracerAssistant] -to $ws
# Add custom menu
set menu [gi::createMenu mymenu -title "My Menu"]
gi::addActions deOpenDesign -to $menu
gi::addMenu $menu -to $ws
# Save into external file
gi::saveWorkspaces $ws -filePath myworkspace.xml
# Load workspace from external file
set ws [gi::loadWorkspaces -filePath myworkspace.xml]
```

gi::moveColumns

Description

Moves one or more columns within a table as if you have dragged the column headers manually. In all cases, the relative positioning of the specified columns is retained. After moving, all columns will be adjacent in the same relative order as before the move.

Note that the ordering of the positional column collection is irrelevant. It is the relative positioning within the table which dictates the new column order. Thus, it is not an error if the same column is specified multiple times in the collection.

This command works with both giTable and giDbTable.

Syntax

```
gi::moveColumns <columns: dbCollection> (-before <giColumn> | -after
  <giColumn>)
```

Arguments

columns

Columns to move.

-before

Specifies another column in the table before which all columns are placed.

-after

Specifies another column in the table after which all columns are placed.

Examples

The following example moves column A after column C in the created table.

```
set dlg [gi::createDialog sampleDialog
        -title "Sample Dialog"]
set tbl [gi::createTable tbl -parent $dlg
        -alternatingRowColors true]
foreach col {A B C} {
    set col$col [gi::createColumn -parent $tbl -label $col -stretch 1]
}
for {set i 1} {$i < 4} {incr i} {
    set row$i [gi::createRow -parent $tbl -label $i]
}
gi::moveColumns $colA -after $colC
```

The following example shows column 3 before column 1.

```
set columns [gi::getColumns -parent $table]
set c1 [db::getIndex 0 -of $columns]
set c3 [db::getIndex 2 -of $columns]
gi::moveColumns $c3 -before $c1
```

gi::moveRows

Description

Moves one or more rows within a table.

Syntax

```
gi::moveRows <rows: dbCollection> (-before <giRow> | -after <giRow> |
    -relative <enum>)
```

Arguments

rows

Rows to move.

-before

Specifies another row in the table before which all rows are placed.

-after

Specifies another row in the table after which all rows are placed.

-relative

Specifies a relative offset (up, down, top or bottom) to move rows within the table: {bottom | down | top | up}.

Default: up.

Examples

The following example moves the last row to the first row within the table.

```
set dlg [gi::createDialog sampleDialog
        -title "Sample Dialog"]
set tbl [gi::createTable tbl -parent $dlg
        -alternatingRowColors true]
foreach col {A B C} {
    set col$col [gi::createColumn -parent $tbl -label $col -stretch 1]
}
for {set i 1} {$i < 16} {incr i} {
    set row$i [gi::createRow -parent $tbl -label $i]
}
gi::moveRows $row15 -relative top
```

gi::openTerminal

Description

Launches terminal command in given path, or in the current working directory if the path is not given. The terminal command is given by the preference `giTerminalCommand`, which defaults to `xterm`.

Syntax

```
gi::openTerminal [-path <string>]
```

Arguments

`-path`

The path to open the terminal. (Default:)

Returns

`<xtJob>`: The created job terminal.

gi::pickWindow

Prompts you to select a window.

Syntax

```
gi::pickWindow [-title <string>] [-prompt <string>] [-windows
    <{<giWindow>}>]
```

Returns

`<giWindow>` : Returns selected window.

Arguments

`-title`

Dialog box title.

`-prompt`

(Optional) Prompt to show in the dialog box above the window list.

`-windows`

Specifies the list of windows to show.

gi::pressButton

Description

Presses a button within a dialog, toolbar or assistant.

Syntax

```
gi::pressButton <name: string> -in <giAssistant | giBanner | giContainer  
| giDialog | giStatusBar | giToolbar | giWindow>
```

Arguments

`name`

Button name within container to press.

`-in`

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

gi::prompt

Description

Prompts the user with a modal dialog box, which returns the pressed button when clicked.

One may define the text, title, any number of buttons, an icon, the default button and the cancel button (which corresponds to escape or closing the dialog as well).

The buttons are right aligned and the dialog is not re-sizable, conforming to human interface guidelines for layout.

Syntax

```
gi::prompt <bodyText: string> -title <string> -buttons [<string>]  
[-default <string>] [-cancel <string>] [-icon <enum>] [-leftButtons  
[<string>]] [-name <string>] [-parent <giAssistant | giDialog |  
giWindow>]
```

Returns

Pressed button (<string>).

Arguments

`bodyText`

Text shown in body of dialog.

`-title <string>`

Text shown in title bar of dialog.

`-buttons [<string>]`

List of button names.

`-default <string>`

Name of button in `-buttons` which is equivalent to hitting return.

Default: First button in list of buttons.

`-cancel <string>`

Name of button in `-buttons`, which is equivalent to hitting escape.

Default: Last button in list of buttons.

`-icon <string>`

Icon severity: {error | information | question | warning}.

Default: information.

`-leftButtons`

List of left-aligned buttons.

`-name`

Name for this dialog. By default, the name is `giPrompt`, but a unique suffix may be appended if more than one instance of this dialog exists at a time.

Default: "".

`-parent`

Specifies the prompt dialog's parent dialog, window or assistant. This causes the prompt to appear centered over its parent.

Errors

- Default button not given in list of buttons.
- Cancel button not given in list of buttons.
- Invalid icon.

Examples

```
set prompt [gi::prompt "DRC will be run in a window of design \nDo you
want to clear the window value?"
    -title "Window Based DRC"
    -buttons { Clear Continue }
    -cancel Continue
    -icon question]
if { "Clear"==$prompt } {
    puts "Cleaning"
}
```

gi::registerWebViewScheme

Description

Creates and registers a new scheme for WebView.

Syntax

```
gi::registerWebViewScheme <name: string> -loadProc [<variant>]
```

Arguments

name

Scheme name

-loadProc

Procedure to run when the user clicks a hyperlink.

Examples

The following example registers the cmd scheme. When you click on the hyperlink specified with `cmd:// scheme` in the HTML file, the Linux command specified in the authority section will be executed utilizing the specified path as an argument and will print the result in the Console.

```
# Creates scheme.html file in the current working directory,
# with the following content (replacing /CWD with the absolute
# path to current working directory).
<!DOCTYPE html><html><h3>Cmd Scheme.</h3>
<li><a href='cmd://ls/CWD/synopsys_custom'>Local Directory</a></li>
</html>
```



```
proc test_load {widget authority path} {
    set cmd "exec $authority $path"
    puts [eval $cmd]
}
gi::registerWebViewScheme cmd -loadProc test_load

# Runs the following commands in the Console.
# When user clicks the "Local Directory" hyperlink, prints the content of
# the
# synopsys_custom directory.

set dialog [gi::createDialog webViewDlg -title "WEB View"]
gi::createWebView sampleWebView -parent $dialog -source [pwd]/scheme.html
```

gi::remove

Description

Removes an action, menu, toolbar or assistant reference from its container previously added with one of the gi::add* commands. Use db::destroy to destroy definitions created with the gi::create* commands.

Syntax

```
gi::remove <item: giAction | giAssistant | giMenu | giToolbar>
```

Arguments

item

Collection of action, menu, toolbar or assistant references to remove.

Examples

```
# Find Schematic Editor Options menu id
set menu [gi::getMenus -from [gi::getWindowTypes seSchematic] -filter
    {%title=="Options"}}
# Remove Options menu from Schematic Editor window
gi::remove $menu
# Find Schematic Editor Hierarchy toolbar id
set toolbar [gi::getToolbars -from [gi::getWindowTypes seSchematic]
    -filter {%title=="Hierarchy"}}
# Remove Hierarchy toolbar from Schematic Editor window
gi::remove $toolbar
```

gi::reportWidgetState

Description

Returns a string representation of the current state of a widget and its children.

Syntax

```
gi::reportWidgetState <widget: giAssistant | giDialog | giMenu |  
    giStatusBar | giToolbar | giWindow> [-filePath <string>]
```

Arguments

`widget`

Widget to report.

`-filePath`

XML string representing the widget state.

gi::saveBindings

Description

Saves a collection of bindings to an XML file. By default, all bindings in the active binding set are assumed. If the file already exists, it is overwritten.

Syntax

```
gi::saveBindings [<bindings: giBindingSet | {<giBinding>}>] -filePath  
    <string>
```

Arguments

`bindings`

Bindings to save.

`-filePath`

Filename to write.

Examples

```
gi::saveBindings [gi::getBindings -set Maestro] -filePath \  
    maestro.xml
```

In this example, `gi::saveBindings` saves the binding definitions from the Maestro binding set to a `maestro.xml` file.

gi::saveWorkspaces

Description

Saves a collection of workspaces to a file on disk.

Syntax

```
gi::saveWorkspaces <workspaces: dbCollection> -filePath <string> [-force  
  <bool>]
```

Arguments

`workspaces`

Collection of workspaces.

`-filePath`

File path to save.

`-force`

Overwrite existing files. (Default is: false)

Examples

```
# Create new workspace
set ws [gi::createWorkspace myworkspace -windowType [gi::getWindowTypes  
  seSchematic] -title "My Workspace"]
# Add assistants
gi::addAssistant [gi::getAssistants heDesignNavigator] -to $ws
gi::addAssistant [gi::getAssistants seNetTracerAssistant] -to $ws
# Add custom menu
set menu [gi::createMenu mymenu -title "My Menu"]
gi::addActions deOpenDesign -to $menu
gi::addMenu $menu -to $ws
# Save into external file
gi::saveWorkspaces $ws -filePath myworkspace.xml
# Load workspace from external file
set ws [gi::loadWorkspaces -filePath myworkspace.xml]
```

gi::scrollTo

Description

Cell, row, or column to scroll to within the table.

Syntax

```
gi::scrollTo <{target}: giCell | giColumn | giRow>
```

Arguments

`{target}`

Target cell, row, or column.

Examples

The following example scrolls to the fifteenth row of the table.

```
set dlg [gi::createDialog sampleDialog
    -title "Sample Dialog"]
set tbl [gi::createTable tbl -parent $dlg
    -alternatingRowColors true]
foreach col {A B C} {
    gi::createColumn -parent $tbl -label $col -stretch 1
}
for {set i 1} {$i < 30} {incr i} {
    gi::createRow -parent $tbl -label $i
}
gi::scrollTo [gi::getRows -parent $tbl -filter {%label == "15"}]
```

gi::sendGUIEvent

Description

Activates the specified window (or assistant's window) and sends a GUI event as if generated by user input.

This command does not accept event data. Instead, you must first script the correct application state using commands such as `de::setCursor`.

Syntax

```
gi::sendGUIEvent <event: string> -to <deCanvas | giAssistant | giWindow>
```

Arguments

event

GUI event to send.

-to

Window, assistant or canvas to receive the event.

Errors

- Unrecognized GUI Event.
- Assistant object specified is not from a window.

Examples

```
# Open the schematic design
# Send Shift-o keys event to the active window
gi::sendGUIEvent Shift-o -to [gi::getActiveWindow]
```

gi::setActiveDialog

Description

Raises the specified dialog and marks it the 'active' dialog in the system. It is returned by the `gi::getActiveDialog` command.

Syntax

```
gi::setActiveDialog <dialog: giDialog>
```

Arguments

giDialog

Dialog to set as the active dialog.

Examples

```
set dialog1 [gi::createDialog testDialog1 -title "Test 1"]
gi::createLabel -label "Dialog 1" -parent $dialog1

set dialog2 [gi::createDialog testDialog2 -title "Test 2"]
gi::createLabel -label "Dialog 2" -parent $dialog2

gi::setActiveDialog $dialog1
```

gi::setActiveTab

Description

Activates the specified tab within a tabbed widget.

Syntax

```
gi::setActiveTab <name: string> -in <giAssistant | giBanner | giContainer
| giDialog | giStatusBar | giToolbar | giWindow> -tabName <string>
```

Arguments

name

Tab widget name within container to set active tab.

-in

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

-tabName

Tab name of tab widget to set as active.

Examples

```
set dialog [gi::createDialog tabGroupDialog -title "Tab Group"]
set tabGroup [gi::createTabGroup tabGroup -parent $dialog]

# Create three tabs

set group1 [gi::createGroup main -parent $tabGroup -label "Main"]
gi::createFileInput fil -parent $group1 -label "Netlist File"

set group2 [gi::createGroup options -parent $tabGroup -label "Options"]
gi::createBooleanInput bi1 -parent $group2 -label "Interface Only"
gi::createBooleanInput bi2 -parent $group2 -label "Parser Options"

set group3 [gi::createGroup generation -parent $tabGroup -label
"Generation"]
gi::createBooleanInput bi3 -parent $group3 -label "Generate Symbol View"
gi::createBooleanInput bi4 -parent $group3 -label "Generate Schematic
View"

db::setAttr dialog.geometry -value 500x300+300+300

gi::setActiveTab /tabGroup -in $dialog -tabName /tabGroup/[db::getAttr
group2.name]

set activeTab [gi::getActiveTab /tabGroup -in $dialog]
puts "Active Tab: $activeTab"
```

gi::setActiveWindow

Description

Sets the active window and optionally raises it.

Syntax

```
gi::setActiveWindow <window: giWindow | giWindowId> [-raise <bool>]
[-force <bool>]
```

Arguments

window

Window (id) to set active.

-raise

Raise the window as well.

Default: false.

`-force`

Forces external window manager to activate this window. The default is false.

Examples

```
set lmwindow [dm::openLibraryManager]
set drwindow [dr::showDisplayResourceEditor]
# Make Library Manager window active
gi::setActiveWindow $lmwindow -raise true
```

gi::setContextMenu

Description

This command associates a menu with a context state name. For editors, the name is the name of a semantic type. If a menu is already defined for the specified name, it is replaced.

Syntax

```
gi::setContextMenu [<name: string>] -windowType <giAssistant |
giWindowType> -menu <giMenu | string>
```

Arguments

`name`

Context name (type name for editor windows) with which to associate the given menu.

Default: "".

`-windowType`

Window type or assistant for which this menu is to be registered.

`-menu`

Menu to register. May be empty (empty string) to un-register the menu definition.

Examples

```
gi::setContextMenu dmLibCSM -windowType \
    [gi::getWindowTypes dmLibraryManager] -menu \
    [gi::getMenus dmLibraryContextMenu]
```

In this example, `gi::setContextMenu` associates an already created `dmLibraryContextMenu` menu with a `dmLibCSM` context state name. [More? Why would users want to do this?] "This is useful for.."

gi::setCurrentIndex

Description

Sets the `currentIndex` in a table/tree widget.

Syntax

```
gi::setCurrentIndex <name: string> -in <giAssistant | giBanner |  
    giContainer | giDialog | giStatusBar | giToolbar | giWindow> -index  
    <string>
```

Arguments

`name`

Name of a table/tree widget in the specified dialog, toolbar, assistant or window.

`-in`

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

`-index`

Item index in the table widget to make current.

gi::setDropAction

Description

Specifies a Tcl procedure to invoke when a drop occurs over `-target` which contains data type `-type`. Although multiple drop actions can be specified for different `-type`, only one drop action can be specified for a `-target` and `-type`. If one has already been specified, this command overrides the currently registered `-dropProc` and `-dragEnterProc` with the new values.

The `-dropProc` is called with the *giDrag* object when the drop occurs. The signature is:

```
proc dropProc {giDrag} {}
```

The `-dragEnterProc` is called with the *giDrag* object when the cursor enters the target widget. It should return true to indicate that the drop is supported, or false otherwise. If unspecified, true is assumed. The signature is:

```
proc dragEnterProc {giDrag} {  
    return true  
}
```

Both the *source* and *target* widgets are stored as attributes in the *giDrag* object.

Syntax

```
gi::setDropAction <{target}: giWidget> -type <string> -dropProc  
[<variant>] [-dragEnterProc [<variant>]] [-dragLeaveProc [<variant>]]
```

Returns

<giDrag> : New object.

Arguments

{target}

Target widget.

-type

Supported data type.

-dropProc

Procedure to invoke when drop occurs. Called with the `giDrag` object.

-dragEnterProc

Procedure to invoke when the cursor enters the target widget. Called with the `giDrag` object. Returns true to accept the drag, false to reject.

-dragLeaveProc

Procedure to invoke when the cursor leaves the target widget. Called with the `giDrag` object.

Examples

The following example is a fully implemented example of the **Drag and Drop** operation for two list input widgets utilizing `gi::createDrag` and `gi::setDropAction` commands.

```
set dlg [gi::createDialog dragDrop -title "Drag & Drop"]
set lst1 [gi::createListInput lst1
          -parent $dlg
          -selectionModel multiple
          -dragProc _dragCell
          -items {item1 item2 item3}]
set lst2 [gi::createListInput lst2
          -parent $dlg
          -selectionModel multiple
          -dragProc _dragCell]
gi::setDropAction $lst1 -dropProc _dropCell -type giListInput
gi::setDropAction $lst2 -dropProc _dropCell -type giListInput
proc _dragCell {lst} {
    gi::createDrag -source $lst -data $lst
}
proc _dropCell {drag} {
    set src [db::getattr drag.source]
```

```
set trg      [db::getattr drag.target]
set data     [db::getattr src.value]
set sItems   [db::getattr src.items]
set tItems   [db::getattr trg.items]

foreach it $data {
    set sind [lsearch $sItems $it]
    set tind [lsearch $tItems $it]
    set sItems [lreplace $sItems $sind $sind]
    set tItems [lreplace $tItems $tind $tind]
    lappend tItems $it
}
db::setattr src.items -value $sItems
db::setattr trg.items -value $tItems
}
```

gi::setField

Description

Presses button in dialog, assistant, window or toolbar.

Syntax

```
gi::setField <name: string> -in <giAssistant | giBanner | giContainer |
giDialog | giStatusBar | giToolbar | giWindow> [-index <string>] -value
<string>
```

Arguments

name

Field name to set.

-in

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

-index

Specifies the index into a list, tree or table to retrieve. Required for these widgets.

Default: "".

-value

Value to set the widget to.

Examples

```
gi::setField {fileName} -value {design.gds} -in \
[gi::getDialogs {dbExportStream}]
```

In this example, `gi::setField` enters the `design.gds` file name in the Stream File entry of the Export Stream dialog box.

gi::setFocus

Description

Sets focus on passed `giWidget`

Syntax

```
gi::setFocus <{target}: giWidget>
```

Arguments

{target}

Target widget.

Examples

```
proc setFocus {widget button} {
    gi::setFocus $widget
}

proc exec {dialog} {
    # Set focus on widget if its value is empty
    db::foreach widget [gi::getChildren -parent $dialog -filter
        {%type=="giTextInput"}] {
        set value [string trim [db::getattr widget.value]]
        if {""==$value} {
            gi::setFocus $widget
            return
        }
    }
}

set dialog [gi::createDialog setFocus -title "Set Focus" -execProc exec]
set user [gi::createTextInput user -label "User" -parent $dialog]
set password [gi::createTextInput password -label "Password" -parent
    $dialog -echo true]
set btn1 [gi::createPushButton -label "Set Focus User" -parent $dialog
    -execProc [list setFocus $user]]
set btn2 [gi::createPushButton -label "Set Focus Password" -parent
    $dialog -execProc [list setFocus $password]]
gi::layout $btn2 -rightOf $btn1
gi::setFocus $user
```

gi::setItemSelection

Description

Set Selection in dialog, assistant, window, or toolbar.

Syntax

```
gi::setItemSelection <name: string> -in <giAssistant | giBanner |  
giContainer | giDialog | giStatusBar | giToolbar | giWindow> -index  
[<string>]
```

Arguments

name

Widget name within container to set selection given items.

-in

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

-index

Index of item to set selection.

Examples

```
gi::setItemSelection {heTreeWidget} -index {0.1,all} -in \  
[gi::getWindows -type [gi::getWindowTypes heEditor]]
```

In this example, `gi::setItemSelection` selects the second row of the first expanded row in the Hierarchy Editor tree widget `heTreeWidget`.

gi::setSectionSizes

Description

Replays a splitter or column stretch operation.

Syntax

```
gi::setSectionSizes <name: string> -in <giAssistant | giBanner |  
giContainer | giDialog | giStatusBar | giToolbar | giWindow> -values  
[<int>]
```

Arguments

name

Widget name within container to set section sizes.

`-in`

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

`-values`

List of section sizes.

Examples

```
set dialog [gi::createDialog setSectionSizes -title "Set Section Sizes"
            -execProc exec]
set table [gi::createTable table -parent $dialog -allowResizeColumns
          true]
gi::createColumn -parent $table -label "User"
gi::createColumn -parent $table -label "Permission"
gi::createColumn -parent $table -label "Comment"
gi::createRow -parent $table
gi::createRow -parent $table
gi::createRow -parent $table
db::setAttr dialog.geometry -value {800x600}
# Set table's column widths
gi::setSectionSizes table -values {300 150 200} -in $dialog
```

gi::showBusyCursor

Description

Changes the cursor graphic to a busy cursor.

Syntax

```
gi::showBusyCursor
```

Examples

The following example shows the busy cursor, running in `db::eval` body, until while the loop ends.

```
set i 0
db::eval {
    while { $i < 10 } {
        gi::showBusyCursor
        de::sendMessage "Iteration $i"
        update
        incr i
        de::sleep 1
    }
}
```

gi::showCommandsHelp

Description

Shows Tcl Help Utility with commands and preferences.

Syntax

```
gi::showCommandsHelp
```

Returns

<giWindow> : Tcl Help window.

gi::showConsole

Description

Shows and activates console window.

Syntax

```
gi::showConsole
```

gi::showContextMenu

Description

Displays the context menu for the specified window or widget.

The command returns the menu being shown, or "" if no menu is shown. After displaying the context menu, use the [gi::hideContextMenu](#) command to hide the context menu.

Syntax

```
gi::showContextMenu <{none}>: giAssistant | giWidget | giWindow>
```

Returns

<giMenu> : Menu associated with the given context name, or "" if no menu is defined.

Arguments

{none}

Window, assistant, or widget whose context menu is shown.

gi::showCustomizeWindow

Description

Shows the Customize Window dialog for the specified window.

Syntax

```
gi::showCustomizeWindow [<window: giWindow | int>]
```

Arguments

window

Window to customize.

gi::showDbTableFilter

Description

Opens the filter dialog for the DB table widget.

Syntax

```
gi::showDbTableFilter -parent <giDialog | giWindow | int> -table  
  <dbObject> [-dbColumns <string>] [-dbPath <string>] [-dbTableName  
  <string>] [-specs <string>] [-execProc <string>] [-excludeColumns  
  <string>] [-loadProc <string>] [-saveProc <string>] [-advProc <string>]  
  [-bkFilter <string>]
```

Arguments

-parent

Parent window or dialog box.

-table

Parent table widget.

-dbColumns

DB column names and types.

-dbPath

DB file name to get column names and types.

-dbTableName

DB table name (required if DB file is specified).

`-specs`

Initial specifications in the dialog box.

`-execProc`

OK/Apply proc, receives the new specs and the table widget as arguments.

`-excludeColumns`

List of column names to exclude from filtering (accepts glob patterns per tcl string match, ignored if `-dbColumns` is specified).

`-loadProc`

Register the callback function for loading file in the dialog box.

`-saveProc`

Register the callback function for saving file in the dialog box.

`-advProc`

Adds the input text for users to edit and sends query filter to the table.

`-bkFilter`

Filter setting backup record.

Returns

`<giDialog>` : Dialog object.

Examples

```
# Create sqlite database with opened libraries cellViews
file delete LCV.db
sqlite db LCV.db
db eval {CREATE TABLE LCV(Library, Cell, View); }
db::foreach lib [dm::getLibs] {
    set libName [db::getAttr lib.name]
    db::foreach cell [dm::getCells -lib $lib] {
        set cellName [db::getAttr cell.name]
        db::foreach view [dm::getCellViews -cell $cell] {
            set viewName [db::getAttr view.name]
            db eval { INSERT INTO LCV VALUES($libName, $cellName,
$viewName); }
        }
    }
}
```

```
# Create dialog with DB Table
set dialog [gi::createDialog lcvDialog -title "LCV DB"]
set table [gi::createDbTable -parent $dialog \
    -allowSortColumns 1 \
    -dbTableName LCV \
```



```
-dbPath LCV.db]

# Open filter dialog allowing to filter specified table by library, cell,
# and view name
gi::showDbTableFilter -parent $dialog -table $table -dbTableName LCV
-dbPath LCV.db
```

gi::showDbTableFilterExpressions

Description

Opens the dialog for DB table filter expressions.

Syntax

```
gi::showDbTableFilterExpressions [-parent <giDialog | giWindow | int>]
-table <dbObject> [-specs <string>]
```

Returns

<giDialog>: Dialog object.

Arguments

-parent

Parent window or dialog.

-table

Parent DB table widget.

-specs

Initial specifications in the dialog.

gi::showDbTableSort

Description

Opens the sort dialog for the DB table widget

Syntax

```
gi::showDbTableSort -parent <giDialog | giWindow | int> -table <dbObject>
[-dbColumns <string>] [-dbPath <string>] [-dbTableName <string>]
[-excludeColumns <string>] [-specs <string>] [-execProc <string>]
[-excludeColumns <string>]
```

Arguments

-parent

Parent window or dialog

-table

Parent table widget

-dbColumns

DB column names and types

-dbPath

DB file name

-dbTableName

DB table name (required if DB file specified)

-excludeColumns

List of column names to exclude from sorting (accepts glob patterns per tcl string match; ignored if -dbColumns is specified)

-specs

Initial specifications in the dialog

-execProc

OK/Apply proc, receives the new specs and the table widget as arguments.

-excludeColumns

List of column names to exclude from filtering (accepts glob patterns per tcl string match, ignored if -dbColumns is specified).

Returns

<giDialog> : Dialog object

Examples

```
# Create sqlite database with opened libraries cellViewsfile delete
LCV.db
sqlite db LCV.db
db eval {CREATE TABLE LCV(Library, Cell, View); }
db::foreach lib [dm::getLibs] {
    set libName [db::getattr lib.name]
    db::foreach cell [dm::getCells -lib $lib] {
        set cellName [db::getattr cell.name]
        db::foreach view [dm::getCellViews -cell $cell] {
            set viewName [db::getattr view.name]
```

```
        db eval { INSERT INTO LCV VALUES($libName, $cellName,
$viewName); }
    }
}

# Create dialog with DB Table
set dialog [gi::createDialog lcvDialog -title "LCV DB"]
set table [gi::createDbTable -parent $dialog \
    -allowSortColumns 1 \
    -dbTableName LCV \
    -dbPath LCV.db]

# Open sort dialog allowing to sort by Library, Cell, or View column
gi::showDbTableSort -parent $dialog -table $table -dbTableName LCV
    -dbPath LCV.db
```

gi::showKeyboardShortcuts

Description

Shows the Keyboard Shortcuts dialog.

Syntax

```
gi::showKeyboardShortcuts [-parent <giWindow | giWindowId>]
```

Returns

<giDialog> : Keyboard Shortcuts dialog.

Arguments

-parent

Window to modify.

Examples

```
gi::showKeyboardShortcuts -parent [db::getNext [gi::getWindows ]]
```

gi::showSaveHistoryAs

Description

Launches the **Save History As** dialog box that is used to save the current selection in the history window.

Syntax

```
gi::showSaveHistoryAs
```

Returns

The dialog (*giDialog*).

Arguments

None.

gi::sortItems

Description

Sorts table or tree column items in a dialog, assistant, window, or toolbar.

Syntax

```
gi::sortItems <name: string> -in <giAssistant | giBanner | giContainer |  
giDialog | giStatusBar | giToolbar | giWindow> -column <string> -order  
<enum>
```

Arguments

name

Name of the widget whose column should be sorted

-in

Dialog, toolbar, banner, status bar, assistant, or window that contains the widget.

-column

Column that should be sorted.

-order

The sort order: {ascending | descending}.

gi::unregisterWebViewScheme

Description

Unregisters a scheme for WebView.

Syntax

```
gi::unregisterWebViewScheme <name: string>
```

Arguments

name

Scheme name

Enumerations

Table 96 Enumerations

| Name | Values | Description |
|-------------------|--|--|
| enActionLogMethod | none command action | Action logging method, whether to disable logging, log the action's command, or to log the action itself, rather than its command. |
| enAssistantOrient | horizontal vertical | Assistant orientation. |
| enDockAreas | left bottom right top | Dock areas within a window. |
| enDockOrient | horizontal vertical both | Toolbar or assistant restricted docking orientations. |
| enDockState | undefined undocked left bottom right top | Docking state for toolbars and assistants. |
| enFrameSide | left bottom right top | Window placement sides within a frame. |
| enLoggingMode | auto explicit | Container's logging mode, whether to log automatically by GUI logging methods, or disable automatic logging and log explicitly. |
| enTristate | true false partial | Represents a tri-state variable such as the window's enabled state. |

Objects

This section covers the following:

- [dbScope](#)
- [giAction](#)
- [giAssistant](#)
- [giContainer](#)
- [giMenu](#)
- [giStatusBar](#)

- [giToolbar](#)
- [giWidget](#)
- [giWindow](#)
- [giWindowType](#)

dbScope

Represents a preference scope. This object may be created with the `db::createScope` command, obtained with the `db::getScopes` command. If destroyed with `db::destroy` command, then all children of this scope are destroyed, too. It is not possible to destroy system-defined scopes.

Table 97 *Attributes*

| Name | Type | Access | Description |
|--------|--|-----------|--|
| id | <code>oaLib</code> <code>dmCell</code> <code>dmCellView</code> <code>deContext</code> string | Read-only | Scope identifier. |
| parent | <code>dbScope</code> | Read-only | Scope from which this scope inherits its values. When requesting the parent of a parentless scope such as the install scope, "" is returned. |
| env | string | Read-only | Environment in which this scope exists. |

giAction

The `giAction` object represents a reusable action that may be placed within a menu, toolbar or status bar. If the container is not specified, the action has not been added to any containers and thus acts as a definition rather than a reference to another action. The attributes of the object are described in the following table and can be retrieved or set by `db::getAttr` and `db::setAttr` commands. This object is created using the `gi::createAction` command and deleted using the generic `db::destroy` command. One may add a reference to this action to a menu or toolbar with the `gi::addActions` command and remove the reference with the `gi::remove` command (`db::destroy` only works on the root definition).

Table 98 *Attributes*

| Name | Type | Access | Description |
|-------------|---|------------|--|
| name | string | Read-only | Action name. |
| container | <i>giMenu</i> <i>giToolbar</i> <i>giStatusBar</i> | Read-only | A container in which the action resides. May be empty for actions that have not been added. |
| title | string | Read/Write | Text shown in menu entries and the binding manager's mouse button indicator toolbar. |
| label | string | Read/Write | Shorter text label used in toolbar entries. |
| icon | string | Read/Write | Filepath to a 16x16 icon pixmap to be displayed in menus and toolbars. |
| largeIcon | string | Read/Write | Filepath to a 24x24 icon pixmap to be displayed in menus and toolbars. |
| toolTip | string | Read/Write | A short help message displayed when the cursor pauses over a toolbar or status bar widget. |
| prompt | string | Read/Write | A slightly longer help message displayed in the window's prompt (either within the status toolbar or status bar) when the cursor hovers over a menu, toolbar or status bar widget. |
| enabled | bool | Read/Write | Whether the action is enabled or disabled. |
| autoDisable | bool | Read/Write | Flag used by other subsystems, for example DE, which indicates that the action should be automatically disabled when the editing mode is read-only. |
| shown | bool | Read/Write | Whether the action is shown. |

Table 98 Attributes (Continued)

| Name | Type | Access | Description |
|---------|---------------|------------|---|
| command | string | Read/Write | Tcl script to evaluate when actions of this type are executed. This may be the empty string for actions defined in C++. |
| menu | <i>giMenu</i> | Read-only | Menu referenced by this action. Is "" if this action does not represent a cascade. |
| history | bool | Read/Write | Whether this action may be shown in the action history toolbar. |

giAssistant

The `giAssistant` object represents a dockable assistant that may be placed within a window or window type. An assistant has a menu and toolbar, both of which are optional and may be represented by the empty string here. The attributes of the object are described in the following table and can be retrieved or set by `db::getAttr` and `db::setAttr` commands. This object cannot be created or deleted in Tcl and can only be created using C++ API. One may add an assistant to a menu with the `gi::addAssistant` command or remove an assistant from a window with the `gi::remove` command. Note that the typical user interaction is showing and hiding the assistant, not adding and removing which also removes it from the assistant menu and destroys any cached data.

Table 99 Attributes

| Name | Type | Access | Description |
|--------------|---------------------------------------|------------|---|
| name | string | Read-only | Assistant name. |
| container | <i>giWindow</i> <i>giWindowType</i> | Read-only | A container in which the assistant resides. May be empty for assistants that have not been added. |
| title | string | Read/Write | Assistant title (shown in the Assistants menu and undocked window border). This is read-only for internally defined assistants. |
| orientations | <i>enDockOrient</i> | Read-only | Restricted orientation attribute for this assistant. |

Table 99 Attributes (Continued)

| Name | Type | Access | Description |
|---------------|--------------------------------------|------------|--|
| dockState | <i>enDockState</i> | Read-only | Represents the doc state for the assistants under a container. For the assistants without a parent container, the value is always <i>undefined</i> . |
| toolbar | <i>giToolbar</i> | Read-only | Toolbar object representing the row of buttons at the top of the assistant. Set it to {} to remove the toolbar. |
| menu | <i>giMenu</i> | Read-only | Menu object representing assistant cascade menu. Set it to {} to remove the menu. |
| shown | bool | Read/Write | Whether the assistant is shown or hidden. |
| geometry | string | Read/Write | Assistant geometry for undocked assistants with W*H+X+Y format. Set it to "" to unset the geometry. The assistant internally decides the appropriate geometry when next shown. |
| dockSize | string | Read/Write | Assistant width and height when docked with W*H format. Set it to "" to unset the dock size. The assistant internally decides the appropriate geometry when next shown. |
| centralWidget | <i>giWindow</i> <i>giContainer</i> | Read-only | Top level central widget. Set it to "" if the window does not support this attribute. |

giContainer

A `giContainer` object is the base class for all container classes. Although `giContainer` can be referred to as a return type, there are no instances of the base class. Returned objects have the attributes of their actual derived types.

Table 100 Attributes

| Name | Type | Access | Description |
|------------|--------------------------------------|------------|---|
| name | string | Read-only | Container name. |
| parent | <i>giContainer</i> | Read-only | Parent container. |
| children | <i>giWidget</i> <i>giContainer</i> | Read-only | Child widgets. |
| styleSheet | string | Read/Write | CSS-structured style sheet defining the look and feel of this group and its children. |
| margins | string | Read/Write | Left, top, right, and bottom margins to use around the container. |
| spacing | string | Read/Write | Spacing between widgets inside the container. |

giMenu

The `giMenu` object represents a menu that may be placed within a window, window type or assistant. A menu is a container of action references. The attributes of the object are described in the following table and can be retrieved or set by `db::getAttr` and `db::setAttr` commands. This object is created using `gi::createMenu` command and deleted using the generic `db::destroy` command. Destroying a menu also destroys all contained action references. One may add a reference to this menu to a window, window type or task with the `gi::addMenu` command and remove the reference with the `gi::remove` command (`db::destroy` only works on the root definition).

Table 101 Attributes

| Name | Type | Access | Description |
|-----------|--|------------|---|
| name | string | Read-only | Menu name. |
| container | <i>giWindow</i> <i>giWindowType</i> <i>giAssistant</i> | Read-only | A container in which the menu resides. May be empty for menus that have not been added. |
| title | string | Read/Write | Menu title |

Table 101 *Attributes (Continued)*

| Name | Type | Access | Description |
|---------|--------|-----------|---|
| preShow | string | Read-only | Procedure to invoke immediately before rendering this menu or any instance. Procedure called with the menu Tcl object being rendered. |

giStatusBar

The giStatusBar object represents the status bar region of a window or window type. A status bar is a container of actions. These objects have no attributes. Status bars cannot be created or destroyed by the user. Instead, they are created and destroyed with the window. The status bar object may be obtained using an attribute of either the giWindowType or the giWindow object. The db::destroy command throws an error if one of these objects is passed.

Table 102 *Attributes*

| Name | Type | Access | Description |
|------|--------|-----------|--------------|
| name | string | Read-only | Widget name. |

giToolbar

The giToolbar object represents a toolbar that may be placed within a window, window type or assistant. A toolbar is a container of action references. The attributes of the object are described in the following table and can be retrieved or set by db::getattr and db::setattr commands. This object is created using the gi::createToolbar command and deleted using the generic db::destroy command. Destroying a toolbar also destroys all contained action references. One may add a reference to this toolbar to a window, window type or task with the gi::addToolbar command and remove the reference with the gi::remove command (db::destroy only works on the root definition).

Table 103 *Attributes*

| Name | Type | Access | Description |
|------|--------|-----------|---------------|
| name | string | Read-only | Toolbar name. |

Table 103 Attributes (Continued)

| Name | Type | Access | Description |
|--------------|--|------------|--|
| container | <i>giWindow</i> <i>giWindowType</i> <i>giAssistant</i> | Read-only | A container in which the toolbar resides. May be empty for toolbars that have not been added. |
| title | string | Read/Write | Toolbar title (shown in the toolbars menu). |
| orientations | <i>enDockOrient</i> | Read-only | Restricted orientation attribute for this toolbar. |
| dockState | <i>enDockState</i> | Read-only | Represents the doc state for the assistants under a container. For the toolbars without a parent container, the value is always <i>undefined</i> . |
| shown | bool | Read/Write | Whether the toolbar is shown or hidden. |
| dynamic | bool | Read/Write | If true, indicates that the contents of the toolbar change automatically. Specially excludes it from the graphical customization. |
| save | bool | Read/Write | If true, recreates the toolbar with a Workspace object. Note that dynamic toolbars are never saved, regardless of this attribute. |

giWidget

The `giWidget` object is the base class for all widget classes. Although `giWidget` can be referred to as a return type, there are no instances of the base class. Returned objects have the attributes of their actual derived types.

Table 104 Attributes

| Name | Type | Access | Description |
|--------|--------------------|-----------|-------------------------|
| name | string | Read-only | Widget name identifier. |
| parent | <i>giContainer</i> | Read-only | Parent container. |

Table 104 Attributes (Continued)

| Name | Type | Access | Description |
|------------|--|------------|---|
| root | <i>giDialog</i> <i>giWindow</i> <i>giAssistant</i> | Read-only | Top-level container containing this widget. |
| style | <i>giStyle</i> | Read-only | Widget style, including the color and font attributes. Although this attribute is read-only, its attributes are read/write. |
| styleSheet | string | Read/Write | Qt style sheet defining the look and feel of the widget. |
| takesFocus | bool | Read/Write | If set to false, the widget does not accept focus at all. |

giWindow

The *giWindow* object represents a single window and has a container of menus, toolbars, assistants and one status bar. Menus, toolbars and assistants may be retrieved by the API. The status bar and toolbar that are used to hold menus are available here as attributes as they are non-removable attributes of the window object. The attributes of the object are described in the following table and can be retrieved or set by `db::getAttr` and `db::setAttr` commands. Windows are created by various subsystems in the platform.

Table 105 Attributes

| Name | Type | Access | Description |
|------------|---------------------|------------|---|
| id | int | Read-only | Window identifier. |
| windowType | <i>giWindowType</i> | Read-only | Window type object. |
| title | string | Read-only | Title displayed in window border. |
| iconTitle | string | Read/Write | Title given to the window's icon and displayed in the window manager's task list. |
| geometry | string | Read/Write | Window geometry. |
| iconified | bool | Read/Write | Whether the window is iconified. |

Table 105 *Attributes (Continued)*

| Name | Type | Access | Description |
|-----------|--------------------|------------|---|
| maximized | bool | Read/Write | Whether the window is maximized. Note that this can be true even if the window is currently iconified. |
| statusBar | <i>giStatusBar</i> | Read-only | Status bar object holding actions that renders this window's status bar. If empty, the status bar is not displayed in the window. |

giWindowType

The `giWindowType` object represents a window type from which windows can be created. A window type has a container of menus, toolbars, assistants and zero or one status bar. The attributes of the object are described in the table that follows and can be retrieved or set by `db::getAttr` and `db::setAttr` commands. This object cannot be created or destroyed using the Tcl API.

Table 106 *Attributes*

| Name | Type | Access | Description |
|-----------|--------------------|------------|--|
| name | string | Read-only | Window type name. |
| geometry | point | Read/Write | Default window geometry for newly created windows with this type. |
| statusBar | <i>giStatusBar</i> | Read-only | Status bar object holding actions which renders the status bar portion of windows created with this type. If empty, the status bar is not displayed in windows of this type. |
| icon | string | Read/Write | Filename of a graphical image to use as the window icon. |

Preferences

[Table 107](#) lists Custom Compiler gi namespace general preferences.

Table 107 Graphical Infrastructure Preferences

| Name | Type | Scope | Default | Description |
|---------------------------------------|--------|-------|--|---|
| giAllowFloatingAssistants | bool | user | true | Allowing floating assistants.
Valid values: true, false |
| giAutoFocus | bool | user | true | Focus central widgets of windows and assistants when cursor enters.
Valid values: true, false |
| giBindingKeyTimeout | int | user | 300 | Time to wait for sequential key events when working with multi-character bindings. |
| giBindingSet | string | user | Custom Compiler | Binding set to activate. |
| giConsoleScrollHistoryOnlyWhenVisible | bool | user | true | Disables scroll to the last history entry if the history tab is not visible.
Valid values: true, false |
| giConsoleSuppressPopupMessageIDs | string | user | LICENSE-003
LICENSE-004
WINDOW-015
WINDOW-118
SAE-651
SAE-652
SAE-653
SAE-654 | Message IDs to suppress from the Console.
You can specify a space-delimited list of message IDs with this preference.
Additionally, you can use wildcards when listing the message IDs.
For example:
<pre>db::setPrefValue giConsoleSuppressPopup MessageIDs -value "LICENSE-001 WINDOW-30" db::setPrefValue giConsoleSuppressPopup MessageIDs -value "NETLISTING-010 ITK-003 LICENSE-003 OPAL-*"</pre> |
| giConsoleTimestampFormat | string | user | | Defines the timestamp format in the Console pane and the Console assistant. For basic HH:MM:SS, use %T. Refer to std::strftime() function for more details. |

Table 107 Graphical Infrastructure Preferences (Continued)

Name	Type	Scope	Default	Description
giConsoleUseTclDebugger	bool	user	false	Show tooltips for menu items. Applicable only if deShowTooltips is enabled. Valid values: true, false
giDialogDefaultStyleSheet	string	user		Style sheet to apply to custom dialogs.
giDialogsStayOnTop	bool	user	true	Dialogs stay on top of their parent window. Valid values: true, false
giFilterPatternStyle	giPattern Style	user	partial	Controls pattern matching within filtered lists. Valid values: matchStart, partial, regexp, wildcard
giHideAppIcons	string	user	giConsole	Defines hidden app icons.
giHelpExportPrefsScope	string	user	user	Exports scope of preferences.
giHelpFilterCommandsIncludeDescription	bool	user	true	Filters commands with Include Description . Valid values: true, false
giHelpHideOACommands	bool	user	true	Hides OA commands in Help. Valid values: true, false
giHelpSaveCommandsToFileDir	string	user	pwd	Saves commands to a file.
giHelpSavePrefsHTMLDir	string	user	pwd	Saves preferences a html file.
giHelpShowCommands	string	user	All	Shows commands in Help.
giHelpShowDeprecatedPrefs	bool	user	true	Shows deprecated preferences. Valid values: true, false
giHelpShowPrefs	string	user	All	Shows preferences in Help.
giHelpSortCommandsBy	string	user	Category	Sorts commands in Help.
giHideAutoDisabled	bool	user	false	Controls if disabled buttons and menu commands are grayed out or hidden. The preference applies to actions that are disabled for a given window, such as actions that modify the design in a read-only editor window. Valid values: true, false

Table 107 Graphical Infrastructure Preferences (Continued)

Name	Type	Scope	Default	Description
giKeepProgressDialogOntop	bool	user	true	Keeps the progress dialog on top of all other windows. Valid values: true, false
giLowBandwidth	bool	user	false	In low-bandwidth, remote display environments, set to true in order to improve the responsiveness of canvas operations. In particular, when drawing simple shapes, such as rectangles, paths and polygons, the cursor moves more fluidly with <code>giLowBandwidth</code> set to true. This option delays the updates to the coordinate display and active object toolbar as long as the cursor continues to move in order to prioritize updates to the canvas. In a high-latency, low-CPU test environment, cursor performance improved by up to 5X. In most environments, this improvement is less noticeable. Example: <pre>db::setPrefValue giLowBandwidth -value true</pre> Valid values: true, false
giMaxRecentFrames	int	user	0	Maximum number of frames which should keep their renderer buffers. Defaults is 0, in which case every renderer will keep their resources as long it exists. If the value is N, then such visible top level windows will keep resources, which belong to some of last N frames interacted and visible.
giNewWindowType	string	user	giHomePage	Window type to create when new windows are created via the new tab button or <code>gi::createWindow</code> .
giPreserveCustomization	bool	user	true	Preserves window customization for future sessions. Valid values: true, false

Table 107 Graphical Infrastructure Preferences (Continued)

Name	Type	Scope	Default	Description
giPrinter	string	user		Printer used in printing dialogs.
giPromptCloseMultipleTabs	bool	user	true	Prompts when closing a frame with multiple tabs. Valid values: true, false
giSaveWorkspacesFile	string	user	""	Default file path to save workspaces to when saving interactively.
giScrollFrameTabs	bool	user	false	Indicates whether tab size should be calculated according to the text size on it. Valid values: true, false
giShowConsolePopups	bool	user	true	Shows messages of the Console assistant. Valid values: true, false
giShowDialogsAsWindows	bool	user	false	When set to true, Custom Compiler creates dialogs as separate top-level windows. Valid values: true, false
giShowFrameDecoration	bool	user	true	Enables custom frame decorations. Valid values: true, false
giShowFrameTabs	bool	user	true	Shows tab bar in window frames. Valid values: true, false
giShowMenuTooltips	bool	user	false	Show tooltips for menu items. Applicable only if deShowTooltips is enabled. Valid values: true, false
giShowProgressDialogs	bool	user	true	Shows progress dialogs for long-running operations. Valid values: true, false
giTerminalCommand	string	user	xterm	Controls terminal command used by the <code>gi::openTerminal</code> command.
giTheme	string	user	Light	Graphical User Interface theme.
giVerticalTabsForAssistants	bool	user	false	Arranges assistant tabs vertically.

Table 107 Graphical Infrastructure Preferences (Continued)

Name	Type	Scope	Default	Description
giWorkspacesIncludeGeometry	bool	user	false	Includes window geometries in workspaces object. Valid values: true, false

12

Hierarchy Editing Tcl

This chapter lists commands, enumerations, and objects in the Tcl he namespace. This namespace provides Tcl for editing hierarchy configuration (config) views.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

Interactive commands are listed after non-interactive commands in the Commands section.

The commands covered are:

- [he::addRule](#)
- [he::addViewGroup](#)
- [he::clearBindings](#)
- [he::clone](#)
- [he::closePropDB](#)
- [he::computeId](#)
- [he::createConfig](#)
- [he::createConfigTemplate](#)
- [he::createContext](#)
- [he::createLogicalSelectionEngine](#)

- [he::createNetInput](#)
- [he::createOverlayConfig](#)
- [he::createOverlayView](#)
- [he::createRule](#)
- [he::createViewGroup](#)
- [he::deleteRule](#)
- [he::deleteViewGroup](#)
- [he::expandViewList](#)
- [he::findPropDB](#)
- [he::findSwitchedMaster](#)
- [he::foreach](#)
- [he::getConfigTemplates](#)
- [he::getConfigView](#)
- [he::getExternalFileName](#)
- [he::getInstHeaders](#)
- [he::getInsts](#)
- [he::getProps](#)
- [he::getPropValue](#)
- [he::getRules](#)
- [he::getSelected](#)
- [he::getViewGroups](#)
- [he::isLeaf](#)
- [he::listPropDBs](#)
- [he::openConfig](#)
- [he::openPropDB](#)
- [he::popContext](#)
- [he::pushContext](#)

- [he::queryViewSelection](#)
- [he::saveAsPropDB](#)
- [he::saveConfig](#)
- [he::savePropDB](#)
- [he::setPropValue](#)
- [he::setRuleFromDE](#)
- [he::showExportToFile](#)
- [he::showHierarchyEditor](#)
- [he::showLogicalSelectionOptions](#)
- [he::showObject](#)
- [he::undoRedoHistory](#)

he::addRule

Description

This function adds a rule to the specified heConfig object. In the case of a conflicting rule the value used for the final configuration rule comes from the passed in rule object. For example if a rule for selecting the view name "old" for cell "lib/cell" exists in the configuration, and the passed in rule object selects view "new" for cell "lib/cell" then the final selected view for the configuration is "new". However, rules are only replaced if they are exact matches in type and parameters. If a Bind to Open rule exists for "lib/cell" and a rule is added to select view "new" for "lib/cell" then the final configuration contains both rules for "lib/cell".

Syntax

```
he::addRule <rule: heRule> -config <heConfig>
```

Arguments

[heRule](#)

The rule to be added to the heConfig object.

-config [heConfig](#)

The heConfig object modified.

Examples

```
set topCellView [dm::findCellView schematic -cellName pll
  -libName designLib]
set config [he::createConfig -topCellView $topCellView ]
set cell [dm::findCell vco -libName designLib]
set vslRule [he::createRule -cell $cell -viewSearchList
  "schematic verilog"]
set selectRule [he::createRule -cell $cell -selectView vhdl]
he::addRule $vs1Rule -config $config
he::addRule $selectRule -config $config
```

This example creates viewSearchList and selectView rules for the designLib/vco cell and adds them to the hierarchy configuration for the designLib/pll/schematic top cellview.

he::addViewGroup

Description

Adds a view name group to the specified heConfig object.

Note that if a view name group exists with the same name as the added view name group then the old group is automatically removed. The final definition of the view name group matches the view name group passed into he::addViewGroup.

Syntax

```
he::addViewGroup <viewGroup: heViewNameGroup> -config <heConfig>
```

Arguments

[heViewNameGroup](#)

The view name group.

-config [heConfig](#)

The config object to modify.

Examples

```
set topCellView [dm::findCellView schematic -cellName pll \
  -libName designLib]
set config [he::createConfig -topCellView $topCellView ]
set vhd1Group [he::createViewGroup vhd1 \
  -views [list vhd1 vhd vhdAMS schematic symbol]]
he::addViewGroup $vhd1Group -config $config
```

This example creates the "vhdl" View Group and adds the group to the designLib/pll/config hierarchy configuration.

When using group names in the Hierarchy Editor, append the percent character (%) before the name in the View Search List entry field (for example, %vhdl).

he::clearBindings

Description

Clears the occurrence bindings for a branch of hierarchy.

Syntax

```
he::clearBindings <context: deContext> [-inst <oaInst | string>]
```

Arguments

context

The design context.

-inst

If specified, clears bindings from the instances hierarchy.

Returns

<none>

he::clone

Description

This command creates a copy of the specified hierarchy context. The implementation is optimized so that the amount of time and memory required to clone a context is independent of the occurrence path depth.

Syntax

```
he::clone <context: heContext>
```

Arguments

context

Context object to clone.

Returns

<heContext>: Newly created context.

Examples

```
#Print all of the Cell Views in a design with a representative occurrence
path array set occurrenceMap {}
he::foreach child [db::getAttr hierarchy -of [de::getActiveContext]] {
  set occurrenceMap([db::getAttr occurrence -of $child]) [he::clone $child]
}
foreach key [lsort [array names occurrenceMap]] {
  set cellView [db::getAttr cellView -of $occurrenceMap($key)]
  set cellViewName [file join [db::getAttr libName -of $cellView]\
    [db::getAttr cellName -of $cellView] \
    [db::getAttr name -of $cellView]]
  puts "$key => $cellViewName"
}
```

he::closePropDB

Description

Closes the specified *hePropDB* database. Opens and closes are counted. This command decrements the open count for the specified *hePropDB* database. When the open count is reduced to 0, the database is purged from memory.

Syntax

```
he::closePropDB <hePropDB: hePropDB>
```

Arguments

hePropDB

The database to close.

Returns

<none>

Examples

To close a hierarchical property database:

```
set ctx [de::getActiveContext]
set heCtx [db::getAttr ctx.hierarchy]
set cellView [db::getAttr ctx.editFile.container]
#Open or create hier database files
set myPropDB [he::openPropDB myPropDB -topCellView $cellView]
he::setPropValue myPropName -db $myPropDB -value "myPropValue" -context
  $heCtx
he::savePropDB $myPropDB
he::closePropDB $myPropDB
```

This example adds a property with a value to a hierarchical property database, then saves and closes it.

he::computeId

Description

This command calculates a unique string for the given heContext/heInstHeader object.

Syntax

```
he::computeId [-context <heContext>] [-instHeader <heInstHeader>]
```

Returns

<string> : The unique identifier of the passed-in object

Arguments

-context

Queried heContext. You can set only <-context> or <-instHeader>, but not both.

-instHeader

Queried heInstHeader. You can set only <-context> or <-instHeader>, but not both.

he::createConfig

Description

Creates a new heConfig object with the specified properties. The object must either be explicitly destroyed or all referencing variables must go out of scope to free allocated resources.

If the -cellView argument is specified then the config is created within the specified library cell view (LCV). If the LCV does not exist in the database it is created. If the LCV exists but is not a configuration cellview type, the command fails.

Syntax

```
he::createConfig [-viewSearchList [<string>] | -templateName <string>]  
[-viewStopList [<string>] -templateName <string>] [-libSearchList  
[<string>-templateName <string>] -topCellView <dmCellView> [-bindProp  
<string>] [-cellView <dmCellView>] [-libOpenList [<string>]]  
[-cellOpenList [<dmCell | string>]] [-overlayViewName <string>]
```

Arguments

`-viewSearchList [<string>]`

The global view search list.

`-viewStopList [<string>]`

The list of views used to indicate leaf cellviews.

Default: "".

`-topCellView` `dmCellView`

Cellview used as root of configured design hierarchy.

`-bindProp`

Property binding.

`-cellView`

The cellview where the newly created config should be stored.

Default: NULL.

`-libSearchList`

lib search list.

`-libOpenList`

lib open list.

`-cellOpenList`

cell open list.

`-overlayViewName`

Default overlay view name.

`-templateName`

Configuration template name.

Returns

`<heConfig>` New Config object.

Errors

- If `-cellView` is not writable.
- If `-cellView` specifies an existing OA cellview that is not a Config View.

Examples

```
set topCellView [dm::findCellView $viewName -cellName $cellName \  
    -libName $libName]  
set con [he::createConfig -topCellView $topCellView \  
    -viewSearchList "schematic hspice" \  
    -viewStopList hspice]
```

This example creates a new configuration with the top cell view \$libName/\$cellName/\$viewName, the viewSearchList "schematic hspice," and the viewStopList "hspice." Note that the configuration view is not yet saved.

he::createConfigTemplate

Description

Creates a new configuration template. If the template name matches an existing template then the original object is deleted before the new object is created.

The names of the templates are displayed in the HE GUI template menu and usually correspond to the name of a simulator. The name of the template can contain spaces but should not contain newlines.

Syntax

```
he::createConfigTemplate <name: string> -filePath <string>
```

Arguments

Name <string>

The name of the configuration template.

-filePath <string>

File path of the configuration template.

Returns

The created heConfigTemplate object.

Examples

```
he::createConfigTemplate vhdl -filePath ./vhdl.xml
```

This example creates a template with the name "vhdl". The XML file ./vhdl.xml can have, for example, the following content:

```
<config version="1">  
  <stop_list>vhdl vhd symbol</stop_list>  
  <view_search_list>vhdl vhd schematic </view_search_list>  
</config>
```

he::createContext

Description

This function creates a new context object for navigating a design hierarchy. If the top cellview is a configuration view then the configured hierarchy is navigated and the provided `<view search list>` is ignored.

If the top cellview is not a configuration cellview then the specified `<view search list>` is used to traverse the hierarchy.

If the top cellview is not a config view and no `<view search list>` is provided then the placed hierarchy is traversed.

The `-viewStopList` argument is used to determine stopping points in the hierarchy for view search list and placed hierarchies. For configured hierarchies the view stop list comes from the `heConfig` object and the passed in `-viewStopList` argument is ignored. Regardless of the stop list designs not containing instances are treated as stop points.

Note that the context API allows traversing configured, view search list and placed hierarchies using a single set of commands. This allows generic code to be written that operates on both Layout and Schematic hierarchies. If the master does not match the master determined by the current ruleset then the context object is said to be "out of context." This is provided primarily to support common editor use models.

Syntax

```
he::createContext -cellView <dmCellView | dmCellViewName>  
  [-viewSearchList [<string>]] [-viewStopList [<string>]]
```

Arguments

`-cellView` [dmCellView](#)

The cellview used as the root of the created `heContext`.

`-viewSearchList` [`<string>`]

The view search list used if the specified cellview is not a Config cellview.

`-viewStopList` [`<string>`]

List of views used to determine stop points.

Default: "".

Returns

The created context object ([heContext](#)).

Examples

```
set dmCV [dm::findCellView $viewName -libName $libName -cellName \
    $cellName]
set h [he::createContext -cellView $dmCV -viewSearchList \
    "hspice schematic" -viewStopList hspice]
```

In this example, `dm::findCellView` returns the DM cellview object passed to the `he::createContext` command as the top cell view, creating a hierarchy context object.

he::createLogicalSelectionEngine

Description

Creates a logical selection engine for use in an interactive command.

Syntax

```
he::createLogicalSelectionEngine [-filterProc [<variant>]]
    [-selectionChangeProc [<variant>]] [-types [<string>]] [-hier <bool>]
```

Arguments

`-filterProc`

Procedure called for each selectable object. Return false to prevent activation.

`-selectionChangeProc`

Procedure called when the logical selection changes, even externally.

`-types`

List of logical types to select {Inst | InstTerm | Net | Term | Pin | FigGroup}.

`-hier`

If true, select logical objects at any level of hierarchy. If false, select only 0x logical objects.

Default: false.

Returns

<heLogicalSelectionEngine>, the newly created logical selection engine object.

Examples

```
namespace eval ie::getNetName {
proc selectionChangeProc {cmd heContext} {
set ocrs [db::getattr occurrence -of $heContext]
set slct [he::getSelected -context $heContext -hier false]
if {[db::isEmpty $slct]} {
```

```

set nm [db::getattr object.name -of $slct]
puts "$ocrs:$nm"
}
}
proc init {cmd context} {
    set ns [namespace current]
    db::setattr engine -of $cmd -value [\
        he::createLogicalSelectionEngine \
        -selectionChangeProc ${ns}::selectionChangeProc\
        -types "Net"\
        -hier true]
    # Supported types: Inst, InstTerm, Net, Term, Pin, FigGroup
}
}
de::createCommand ie::getNetName -type interactive\
-description "Prints hierarchal net name " -category\
    layout -label GetNetName

```

In this example, createLogicalSelectionEngine registers the ie::getNetName command. The ie::getNetName command prints the hierarchical name when you click on the net in a layout design.

he::createNetInput

Description

Creates a new heNetInput widget and adds it to the specified parent.

Syntax

```

he::createNetInput [<{none}>: string] [-parent <giAssistant | giContainer
| giDialog | giWindow>] [-enabled <bool>] [-shown <bool>] [-toolTip
<string>] [-valueChangeProc [<variant>]] [-label <string>] [-hier
<bool>] [-requireExisting <bool>] [-logicalTypes [<enum>]] -context
<deContext> [-allowSelectionSync <bool>] [-textChangeProc [<variant>]]
[-allowPattern <bool>] [-readOnly <bool>] [-valid <bool>] [-required
<bool>]

```

Arguments

{none}

Widget name. (Default is:)

-parent

Dialog, group or other container into which this widget should be placed.

-enabled

Initial enabled state. (Default is: true)

`-shown`
Initial shown state. (Default is: true)

`-toolTip`
Optional widget toolTip. (Default is:)

`-valueChangeProc`
Procedure to execute when the value is changed. Called with the widget object

`-label`
Widget label, shown in the left column. (Default is:)

`-hier`
Support selection of hierarchical nets above and below the current design
(Default is: false)

`-requireExisting`
Automatically invalidate entered names which do not exist. (Default is: true)

`-logicalTypes`
Ordered list of object types to select and synchronize with hierarchical selection.
({ inst | instTerm | net | netGroup | pin | term })

`-context`
Hierarchical context in which to select nets

`-allowSelectionSync`
Automatically synchronize objects with logical selection (Default is: true)

`-textChangeProc`
Procedure to execute when the widget text is changed. Called with the widget
object.

`-allowPattern`
If true, the widget follows the `giFilterPatternStyle` preference. (Default is:
false)

`-readOnly`
Initial read-only state. (Default is: false)

`-valid`
Initial valid state. (Default is: true)

`-required`

Specifies this is a required input. (Default is: false)

Returns

`<heNetInput>` : New heNetInput object.

he::createOverlayConfig

Description

Creates an overlay config.

Syntax

```
he::createOverlayConfig <src: dmCellView | dmCellViewName> (-name  
  <string> | -dest <dmCellView | dmCellViewName>) -template <string>  
  [-overwrite <bool>]
```

Arguments

`src`

The cellview to use as source for config's top.

`-name`

Overlay config's name to create.

`-dest`

The destination overlay config to create.

`-template`

Template to determine viewSearchList and viewStopList.

`-overwrite`

Overwrites the target, if it exists. The default is false.

Returns

`<dmCellView>` : New Overlay Config.

he::createOverlayView

Description

Creates an overlay view.

Syntax

```
he::createOverlayView [<name: string>] [-src <dmCellView |  
dmCellViewName>] [-overwrite <bool>] -context <heContext>
```

Arguments

name

The overlay view name. If not specified, the occurrence path of the context is used.

-src

The cellview to copy to create an overlay view. If not specified, the context's cellview is copied.

-overwrite

Overwrite the existing design. (Default is: false)

-context

The overlay config in which to create the overlay view.

Returns

<dmCellView> : New Overlay Cell View.

he::createRule

Description

Creates a single new rule. It can be used to create Selection, View Search List, and Bind to Open rules at the Cell, Instance, and Occurrence level.

Rule objects are used for adding, searching, and removing configuration rules.

The level of the rule and the type of rule must be uniquely specified.

Note that rule objects are memory-managed by the system. It is not necessary to delete/destroy rule objects explicitly.

Syntax

```
he::createRule [-viewSearchList [<string>]] [-selectView <string>]  
[-selectCell <string>] [-selectLib <string>] [-open <bool>] [-stop  
<bool>] [-cell <dmCell>] [-cellView <dmCellView>] [-instName <string>]  
[-occurrence <string>]
```

Returns

The created rule ([heRule](#)).

Arguments

`-viewSearchList {<string>}`

The list of views used for view search list rules. You can only set one of the following arguments at a time: `-viewSearchList`, `-selectView`, `-selectCell`, `-selectLib`, `-open`, or `-stop`.

Default is: "".

`-selectView`

Select View name. You can only set one of the following arguments at a time:

`-viewSearchList`, `-selectView`, `-selectCell`, `-selectLib`, `-open`, or `-stop`.

`-selectCell`

Select cell name. You can only set one of the following arguments at a time:

`-viewSearchList`, `-selectView`, `-selectCell`, `-selectLib`, `-open`, or `-stop`.

`-selectLib`

Select library name. You can only set one of the following arguments at a time:

`-viewSearchList`, `-selectView`, `-selectCell`, `-selectLib`, `-open`, or `-stop`.

`-open <bool>`

The flag used to specify Bind to Open rules. You can only set one of the following arguments at a time: `-viewSearchList`, `-selectView`, `-selectCell`, `-selectLib`, `-open`, or `-stop`.

Default is: false.

`-stop`

Stop Flag. You can only set one of the following arguments at a time:

`-viewSearchList`, `-selectView`, `-selectCell`, `-selectLib`, `-open`, or `-stop`.

`-cell dmCell`

Cell used to specify cell-level rules.

You can only set one of the following arguments/combination of arguments at a time: `-occurrence`, `-cell`, or `-cellView` and `-instName`.

Default is: "".

`-cellView dmCellView`

Cellview used to specify instance level rules. Must be used together with `-instName`.

You can only set one of the following arguments/combination of arguments at a time: `-occurrence`, `-cell`, or `-cellView` and `-instName`.

Default is: "".

`-instName <string>`

The instance name used to specify instance level rules. Must be used together with `-cellView`.

You can only set one of the following arguments/combination of arguments at a time: `-occurrence`, `-cell`, or `-cellView` and `-instName`.

Default is: "".

`-occurrence <string>`

The occurrence path used to specify occurrence level rules.

You can only set one of the following arguments/combination of arguments at a time: `-occurrence`, `-cell`, or `-cellView` and `-instName`.

Default is: "".

Examples

Example 3

This example creates a view search list for the designlib vco cell. It contains the views schematic and hspice. The example also applies a Bind to Open rule for the X0 instance of the designlib vco schematic cellview, omitting it from evaluation in processes such as netlisting.

```
set cell [dm::findCell vco -libName designLib]
set cellView [dm::findCellView schematic -cellName vco \
  -libName designLib]

he::createRule -cell $cell -viewSearchList "schematic hspice"
he::createRule -cellView $cellView -instName X0 -open true
```

Example 4

This example creates cell and instance scoped rules and saves in the config view. The instance l12 is bound to veriloga view and other instance of nand2 circuit is bound to versions from another libraries.

```
set tbLibName "topLib"
set top [dm::findCellView schematic -cellName nand2_tb \
  -libName $tbLibName]
set config [he::createConfig -topCellView $top -templateName HSPICE]
set cell [dm::findCell nand2 -libName $tbLibName]
foreach cellName {nand2_x1 nand2_x2} {
  # Create cell scoped rule
  set rule [he::createRule -cell $cell -selectLib DUTLib]
  he::addRule $rule -config $config
}
```

```
set rule [he::createRule -cell $cell -selectCell $cellName]
he::addRule $rule -config $config

set rule [he::createRule -cell $cell -selectView schematic]
he::addRule $rule -config $config

# Create instnace scoped rule
set rule [he::createRule -cellView $top -instName I12 \
    -selectLib DUTLib]
he::addRule $rule -config $config

set rule [he::createRule -cellView $top -instName I12 \
    -selectCell nand2]
he::addRule $rule -config $config

set rule [he::createRule -cellView $top -instName I12 \
    -selectView veriloga]
he::addRule $rule -config $config

set cfgCV [dm::findCellView config_${cellName} -cellName nand2_tb \
    -libName $tbLibName]
he::saveConfig $config -cellView $cfgCV
}
```

he::createViewGroup

Description

This function creates a new View Name Group. The group can then be added to an heConfig object using he::addViewGroup.

Note that it is not an error if a member of the passed in views references a view name group which does not exist in the heConfig. This is done to allow copying of view name groups between configs without regard for ordering.

Syntax

```
he::createViewGroup <name: string> -views [<string>]
```

Arguments

<string>

The name of the new view name group.

-views [<string>]

The list of view names to associate with the new view name group.

Returns

The newly created view name group object. ([heViewNameGroup](#)).

Examples

```
he::createViewGroup vhdl -views [list vhdl vhd vhdAMS \  
    schematic symbol]
```

This example creates the %vhdl View Group with a "vhdl vhd vhdAMS schematic symbol" view search list.

he::deleteRule

Description

This function removes the specified rule from an heConfig.

Syntax

```
he::deleteRule <rule: heRule> -config <heConfig>
```

Arguments

heRule

The rule to be deleted from the heConfig object.

-config *heConfig*

The heConfig object to be modified.

Returns

<none>

Examples

```
set top [dm::findCellView config -cellName vco -libName DemoPLL]  
set cfg [he::openConfig -cellView $top]  
set openRules [he::getRules $cfg -open true ]  
while { [set rule [db::getNext $openRules]] != "" } {  
    he::deleteRule $rule -config $cfg  
}
```

This example deletes all Bind to Open rules from the hierarchy configuration whose top cellview is DemoPLL/vco/schematic.

he::deleteViewGroup

Description

This function deletes a named View Name Group in the specified heConfig.

Syntax

```
he::deleteViewGroup <viewGroup: heViewNameGroup> -config <heConfig>
```

Arguments

`heViewNameGroup`

The view name group to be deleted.

`-config heConfig`

The heConfig to remove the named view name group from.

Returns

<none>

Examples

```
set topCellView [dm::findCellView schematic -cellName test \
  -libName fifo_lib]
set config [he::openConfig -cellView $topCellView]
set vhdlGroup [he::getViewGroups VHDL -config $config]
he::deleteViewGroup [db::getNext $vhdlGroup] -config $config
```

This example deletes the VHDL View Group from the open hierarchy configuration.

he::expandViewList

Description

This function expands all view name groups within the specified view list. If a view name group contains additional view name groups they are also expanded.

Syntax

```
he::expandViewList -config <heConfig> -views [<string>]
```

Arguments

`-config heConfig`

The heConfig used to evaluate the view name groups within the input views.

`-views [<string>]`

The list of view names to expand.

Returns

The fully expanded list of views [<string>].

Errors

If a member of the view list references a non-existent view name group.

Examples

To list all views which form a specified view group:

```
set cv [dm::findCellView schematic -cellName Opamp_transient -libName
MonteCarlo]
set config [he::createConfig -topCellView $cv -templateName HSPICE]
set vg [he::createViewGroup SPICE -views [list schematic spf]]
he::addViewGroup $vg -config $config
he::expandViewList -config $config -views %SPICE
```

This example creates a config view for given cellview, creates a new view group named `SPICE` which contains `schematic spf` views, and adds that view group to a config. Then it uses the [he::expandViewList](#) command to list views forming the `SPICE` view group.

he::findPropDB

Description

Finds the specified *hePropDB* database in memory. If the named property database is not in memory, then the empty string is returned.

Syntax

```
he::findPropDB <name: string> -topCellView <dmCellView | dmCellViewName>
```

Arguments

`name`

The name of the database to search for.

`-topCellView`

The cellview to search.

Returns

<none>

he::findSwitchedMaster

Description

This command determines the switched master *heContext* for the passed in instance or *helInstHeader* and calculates the child context. Note that for placed master followers the Switched master is equal to the placed master.

Returns

<heContext> : The found context object

Syntax

```
he::findSwitchedMaster [-context <heContext>] [-inst <oaInst>]  
                        [-instHeader <heInstHeader>]
```

Arguments

-context

The parent context used to determine the associated master. You can only set one of the following arguments/combination of arguments at a time: <-instHeader>, or <-context> and <-inst>.

-inst

The instance used to determine the switched master. You can only set one of the following arguments/combination of arguments at a time: <-instHeader>, or <-context> and <-inst>.

-instHeader

The heInstHeader used to determine the switched master. You can only set one of the following arguments/combination of arguments at a time: <-instHeader>, or <-context> and <-inst>.

Errors

- If no options are provided.
- If -instHeader and either -context or -inst are also specified.
- If -context is specified and -inst is not specified.
- If -inst is specified and -context is not specified.

Examples

```
set sm [he::findSwitchedMaster -context $h -inst $i]  
set cv [db::getAttr cellView -of $sm]  
puts "Switched master is [db::getAttr libName -of $cv] \  
    /[db::getAttr cellName -of $cv]/[db::getAttr name -of $cv]"
```

In this example, he::findSwitchedMaster returns the switched master of instance \$i. Then, by getting the cellview attribute of the switched master object, it prints the switched master library cellview.

he::foreach

Description

A foreach loop for the hierarchy.

Syntax

```
he::foreach <variable: string> <context: dmCellView | heContext> <script:
script>
```

Arguments

-variable

The variable name of the Context loop.

-context

CellView or context to iterate on.

-script

A Tcl script to execute.

Returns

<none>

Examples

To loop through the design hierarchy:

```
set hierarchy [db::getAttr hierarchy -of
[de::getActiveContext ]]he::foreach hr $hierarchy {
    set design    [db::getAttr design -of $hr]
    set cellView  [db::getAttr cellView -of $hr]
    set vieName   [db::getAttr name -of $cellView]
    set cellName  [db::getAttr cellName -of $cellView]
}
```

This example gets the hierarchy from active context, iterates through it, and gets the design and cellview for each hierarchical context.

he::getConfigTemplates

Description

Gets the list of defined configuration templates.

Syntax

```
he::getConfigTemplates <pattern: string> [-filter <string>]
```

Arguments

`pattern <string>`

A pattern to use for matching template names.

`-filter <string>`

A glob style filter for the returned collection.

Default: "".

Returns

Matching configuration templates ([heConfigTemplate](#)).

Examples

```
db::getattr filePath -of [he::getConfigTemplates HSPICE ]
```

This example returns the file path of the "HSPICE" configuration template.

he::getConfigView

Gets the associated configuration view object.

Syntax

```
he::getConfigView -window <giWindow | int>
```

Returns

`<heConfig>` : The config object

Arguments

`-window`

Hierarchy Editor window.

he::getExternalFileName

Description

Returns the external file name of the current cell view of the hierarchy context. If the cell view is not linked to an external file, returns empty string.

Syntax

```
he::getExternalFileName <context: heContext> [-nameType <bool>]
```

Arguments

`context`

The `heContext` to get the current cell view.

`-nameType`

The option to get the file name with absolute path or relative path. If it is set to `true`, it returns relative path. (Default: `true`)

Returns

`<none>`

`he::getInstHeaders`

Description

This function determines the `heInstHeaders` referenced for a given `heContext` object. `heInstHeaders` can be used to get the associated instances and the child `heContext`.

Syntax

```
he::getInstHeaders -context <heContext>
```

Arguments

`-context`

The context queried to determine the `heInstHeader`'s.

Returns

A collection of child `heInstHeader` objects (`{<heinstheader>}`).

Examples

```
set ihs [he::getInstHeaders -context $h]
while {[set ih [db::getNext $ihs]] != ""} {
    set placedMaster [db::getAttr placedMaster -of $ih]
    puts "Placed Master cell name is: [db::getAttr cellName -of \
        $placedMaster]"
}
```

In this example, `he::getInstHeaders` returns all instance headers from the given `$h` context. Then, by walking through the instance headers, it prints all placed master cell names.

he::getInsts

Description

This function determines the instances bound to the given heInstHeader. This function is used in conjunction with he::getInstHeaders to walk the instances of a specific master.

Syntax

```
he::getInsts -instHeader <heInstHeader> [-singleBit <bool>]
```

Arguments

-instHeader [heInstHeader](#)

The instance header queried.

-singleBit

Flag to indicate if should iterate over all singleBit instances. (Default is: false)

Returns

The collection of instances associated with the passed in master. ([{{oaInst}}](#)).

Examples

```
set is [he::getInsts -instHeader $ih]
while {[set i [db::getNext $is]] != ""} {
    puts "Instance name is: [oa::getName $i]"
}
```

In this example he::getInstances returns all instances from the given \$ih instance header. Then, by walking through the instances, it prints all instance names.

he::getProps

Description

Returns the collection of properties for a given instance/context/scope.

If -scope is not specified, all the properties that match the given object are returned. If -scope is specified, only properties from the specified scope table are returned. Returned properties are sorted both by name and scope. If multiple same named properties are returned, the sorted order ensures highest precedence properties are returned first.

This command is provided for managing hePropDBs. The effective value is typically obtained using the [he::getPropValue](#) command.

Syntax

```
he::getProps [<name: string>] -db <hePropDB> [-context <heContext>]  
  [-inst <oaInst | string>] [-scope <enum>]
```

Arguments

name

Filters the returned properties by name.

-db

The hierarchical property database.

-context

The occurrence.

-inst

The instance to check within the specified occurrence.

-scope

The scope to inspect. Defaults to all scopes. ({ cell | instance | occurrence })

Returns

<dbCollection> : The collection of heProps that matches the specified query.

Errors

- If `-inst` is specified and `-context` is not specified.
- If the specified `-context` has a different top cellview then the specified `-db`.
- If the specified `-inst` is not found inside the specified context.

Examples

```
# Get all properties in an hePropDB  
set props [he::getProps -db hdb]  
# Get all the cell level "accuracy" properties:  
set props [he::getProps "accuracy" -db $db -scope cell]  
# Get the "accuracy" prop for the cell of the switched master of $ctx  
set props [he::getProps "accuracy" -context $ctx -scope cell -db $db]  
# Get the "annotationIndex" for "i<0:3>" in the current active context  
set ctx [db::getattr hierarchy -of [de::getActiveContext]]  
set cellView [db::getattr topCellView -of $ctx]  
set idx [he::getProps "annotationIndex" -inst "i<0:3>" -context $ctx \  
-db [he::findPropDB "annotationIndex" -topCellView $cellView]]
```

To get all properties from the hierarchical property database:

```
set myPropDB [he::openPropDB myPropDB -topCellView $cellView]
db::foreach prop [he::getProps -db $myPropDB] {
    puts "[db::getAttr prop.name]=[db::getAttr prop.value]"
}
```

This example iterates on all properties in the specified hierarchical property database and prints their names and values.

he::getPropValue

Description

Returns the effective value of the named property. If the property does not exist, the empty string is returned.

The full hierarchical path is specified by the `-context` and `-inst` arguments. If `-inst` is not specified, the path is given by the `[db::getAttr occurrence -of $context]` command. If specified, then the value of `-inst` must reference an instance within the given `-context`. The full hierarchical path is the concatenation of the occurrence of the `-context` and the name of the specified `-inst`.

Syntax

```
he::getPropValue <name: string> -db <hePropDB> -context <heContext>
[-inst <oaInst | string>]
```

Arguments

`name`

The name of the queried property.

`-db`

The *hePropDB* to search.

`-context`

The occurrence.

`-inst`

Specified to check if the specified instance is inside the specified `-context`.

Returns

`<string>` : The collection of `heProps` that match the specified query.

Errors

- If the specified context does not have the same top cellview as the *hePropDB*.
- If the instance cannot be found in the specified context.

Examples

```
# $ctx points to /i1/i2/i3
he::getPropValue "accuracy" -context -db $hdb
# The following example is identical to the above:
set parentCtx [he::clone $ctx]
he::popContext $parentCtx
he::getPropValue "accuracy" -context $parentCtx -inst "i3"
# annotation index example
he::getValue "annotationIndex" -context $ctx -inst "i<0:3>" -db $hdb
```

To get data from the hierarchical property database:

```
set ctx [de::getActiveContext]
set heCtx [db::getattr ctx.hierarchy]
set cellView [db::getattr ctx.editFile.container]
#Open or create hier database files
set myPropDB [he::openPropDB myPropDB -topCellView $cellView]
set propVal [he::getPropValue myPropName -db $myPropDB -context $heCtx]
```

This example gets the value of the property named *myPropName* from the specified *propDB*.

he::getRules

Description

This function returns an iterator over the list of Rule objects which match the passed in options.

The matching is performed by finding all of the Rule objects which have all of the specified options. Note that if no options are supplied then all rules are returned.

Most of the options are considered a match if and only if the value of the option and the value of the rule are identical. However the *-occurrence* value is considered a match if the value is identical or if the rule's value is a child instance of the passed in value. For example: an occurrence path of */I1* matches rules with occurrence paths of */I1/I2* and so on. It does not match rules with occurrence paths such as */I2*. Using an occurrence path of */* returns all occurrence rules in the hierarchy. This feature is provided because netlisters need to be able to determine if any occurrence bindings exist for child instances of a given path.

Syntax

```
he::getRules <config: heConfig> [-viewSearchList [<string>] | -selectView  
<string> | -selectCell <string> | -selectLib <string> | -open <bool>  
| -stop <bool>] [-cell <dmCell>] [-cellView <dmCellView>] [-instName  
<string>] [-occurrence <string>]
```

Arguments

heConfig

The heConfig searched for rules.

-viewSearchList {<string>}

The list of views used for view search list rules. You can only set one of the following arguments at a time: -viewSearchList, -selectView, -selectCell, -selectLib, -open, or -stop.

Default: "".

-selectView <string>

Selected view name used for view selection rules. You can only set one of the following arguments at a time: -viewSearchList, -selectView, -selectCell, -selectLib, -open, or -stop.

Default: "".

-selectCell <string>

Selected cell name. You can only set one of the following arguments at a time: -viewSearchList, -selectView, -selectCell, -selectLib, -open, or -stop.

Default: "".

-selectLib <string>

Selected library name. You can only set one of the following arguments at a time: -viewSearchList, -selectView, -selectCell, -selectLib, -open, or -stop.

Default: "".

-open <bool>

The flag used to specify Bind to Open rules. You can only set one of the following arguments at a time: -viewSearchList, -selectView, -selectCell, -selectLib, -open, or -stop.

Default: false.

`-stop`

Stop Flag. You can only set one of the following arguments at a time:

`-viewSearchList`, `-selectView`, `-selectCell`, `-selectLib`, `-open`, or `-stop`.

`-cell` `dmCell`

Cell used to specify cell level rules.

You can only set one of the following arguments/combination of arguments at a time: `-occurrence`, `-cell`, or `-cellView` and `-instName`.

Default: "".

`-cellView` `dmCellView`

Cellview used to specify instance level rules.

You can only set one of the following arguments/combination of arguments at a time: `-occurrence`, `-cell`, or `-cellView` and `-instName`.

Default: "".

`-instName` `<string>`

The instance name used to specify instance level rules.

You can only set one of the following arguments/combination of arguments at a time: `-occurrence`, `-cell`, or `-cellView` and `-instName`.

Default: "".

`-occurrence` `<string>`

The occurrence path used to specify occurrence level rules.

You can only set one of the following arguments/combination of arguments at a time: `-occurrence`, `-cell`, or `-cellView` and `-instName`.

Default: "".

Returns

The found rules. (`{<heRule>}`).

Examples

```
set top [dm::findCellView config -cellName vco -libName DemoPLL]
set cfg [he::openConfig -cellView $top]
set selectRules [he::getRules $cfg -selectView abstract]
set openRules [he::getRules $cfg -open true]
set occRules [he::getRules $cfg -occurrence "/"]
```

In this example, `he::getRules` returns `selectView`, `Bind to Open` and `occurrence` rules from the open hierarchy configuration.

he::getSelected

Description

Lists the selected items in Hierarchy Editor of <heContext> object.

Syntax

```
{<heContext>} he::getSelected -window [<giWindow> | <int>] ]
```

Arguments

-window

The index of Hierarchy Editor.

Errors

- If the selected item does not exist.
- If the selected option is not Hierarchy Editor.

he::getViewGroups

Description

This function returns a collection of view name groups filtered by the passed in name field.

Syntax

```
he::getViewGroups [<filter: string>] -config <heConfig>
```

Arguments

<string>

A glob style filter for the returned collection.

Default: .*

-config [heConfig](#)

The configuration queried.

Returns

The collection of heViewNameGroup's matching the passed in filter ([{<heViewNameGroup>}](#)).

Examples

```
set topCellView [dm::findCellView schematic -cellName test \  
-libName fifo_lib]
```

```
set config [he::openConfig -cellView $stopCellView]
set groups [he::getViewGroups V* -config $config]
```

In this example, `he::getViewGroups` returns all View Groups whose names start with "V" in the open hierarchy configuration.

he::isLeaf

Description

This function is a predicate to determine if an instance is a leaf or if an `heContext` is a leaf. An instance is considered a leaf if the switched master is within the configuration view's Stop Cell List. A cellview can also be considered a leaf if it does not contain instances. If an `<oaInst>` is not provided then this function checks to see if the passed in `-context` is a leaf cell view.

Syntax

```
he::isLeaf [<inst: oaInst>] -context <heContext>
```

Arguments

`oaInst`

The instance checked to see if it is a leaf.

`-context heContext`

The `heContext` queried to determine if the passed in object or `heContext` is a leaf.

Returns

If the object/`heContext` is a leaf. (`<bool>`).

Errors

If the instance/`heContext` is unbound in the configuration hierarchy.

Examples

```
set il [he::isLeaf $i -context $h]
```

In this example, `he::isLeaf` checks to find out if the given instance `$i` is a leaf cell in the current context.

he::listPropDBs

Description

Lists the names of the *hePropDB* databases stored in a specified `oaCellView`.

Note that this command does not return in-memory *hePropDB* databases that were created by the `he::openPropDB -scratch true` command but have never been saved to OA.

Syntax

```
he::listPropDBs [<name: string>] -topCellView <dmCellView |  
dmCellViewName>
```

Arguments

`name`

If not specified, all the names are returned. If specified, the list is filtered by the given Glob-style expression.

`-topCellView`

The top cellview to search for the named `hePropDB` database.

Returns

`<dbCollection>` : The list of HDB names stored in the specified cellview.

Examples

To list all available hierarchical property databases for a given cellview:

```
set ctx [de::getActiveContext]  
set cellView [db::getattr ctx.editFile.container]  
db::createList [he::listPropDBs -topCellView $cellView]
```

This example gets a cellview for the active context and lists all available hierarchical property databases.

he::openConfig

Description

This function reads an `heConfig` object from either an OA cellview or from a file on disk. The `-readOnly` argument specifies if the cellview is to be opened in read-only or read/write mode. Note that if read/write mode is requested the specified cellview is locked to prevent other processes from modifying the cellview.

Syntax

```
he::openConfig [-cellView <dmCellView>] [-pathName <string>] [-readOnly  
<bool>]
```

Returns

The opened config object (`heConfig`).

Arguments

`-cellView`

The `dmCellView` to open as a config. You can only set `<-cellView>` or `<-pathName>`, not both.

Default: NULL.

`-pathName`

The path of the file. You can only set `<-cellView>` or `<-pathName>`, not both.

Default: "".

`-readOnly`

The access mode used to open. If true, config is opened as read-only.

Default: false.

Errors

- If both `-cellView` and `-pathName` are specified.
- If neither `-cellView` nor `-pathName` are specified.
- If the specified `-cellView` does not exist or is not a Config cellview.
- If `-readOnly` is false and the specified `-cellView` is not writable.

Examples

```
set dmcv [dm::findCellView config -libName $libName -cellName \  
    $cellName]  
set h [he::openConfig -cellView $dmcv]  
he::showHierarchyEditor -config $h
```

This example opens a configuration view and calls `he:: showHierarchyEditor` to show configuration details in the Hierarchy Editor.

he::openPropDB

Description

Opens or creates a *hePropDB* database on disk.

The `name` and `-topCellView` arguments uniquely specify a *hePropDB* database stored in OA.

The `-filePath` argument is used to create/open *hePropDB* databases stored externally to OA. It should not be used for *hePropDB* databases stored within OA.

Note:

`-filePath` supports `~` and environment variable expansion.

The `-scratch` option can be used to create/reference an in-memory *hePropDB* database without modifying the file system. If the database has already been opened (`-scratch true`), then the reference count is incremented.

If the *hePropDB* database is already in memory, then the open count is incremented. The `he::closePropDB` command decrements the open count. When the count reaches 0, the database is purged from the memory. The `-readOnly` argument can be used to “upgrade” from read only to write access. However it cannot be used to demote from write access to read only.

Use the `he::savePropDB/he::saveAsPropDB` command to save changes.

Syntax

```
he::openPropDB [<name: string>] [-topCellView <dmCellView |  
dmCellViewName>] [-filePath <string>] [-readOnly <bool>] [-scratch  
<bool>]
```

Arguments

`name`

Hierarchical Property Database.

`-topCellView`

The top cellview to search for the named *hePropDB* database.

`-filePath`

The path to an HDB file to open or create.

`-readOnly`

If true, the returned property database is not editable.

Default is: false.

`-scratch`

If true, the database is created in-memory without modifying the file system.

Default is: false.

Returns

`<hePropDB>` : Newly create property DB.

Errors

- The option `-name` is required for all cases except when `-filePath` is specified.
- If creating a new *hePropDB* and either `name` or `-topCellView` are not specified.
- If `-readOnly true` and `-filePath` does not point to a readable file.
- If `-filePath` exists but is not readable.
- If both `-readOnly` and `-scratch` are true.
- Attempting to open a database with `-scratch true` that is already open in memory with `-scratch false`.
- Attempting to open a database with `-scratch false` that is already open in memory with `-scratch true`.

Examples

To open and/or create a hierarchical property database on disk:

```
set ctx [de::getActiveContext]
set cellView [db::getattr ctx.editFile.container]
#Open or create hier database files
set myPropDB [he::openPropDB myPropDB -topCellView $cellView]
```

This example creates or opens the hierarchical property database for the specified top cellview.

he::popContext

Description

This function modifies the passed in `<heContext>` object to point to the parent of the current context lineage.

This is provided primarily for interactive editor support.

Syntax

```
he::popContext <context: heContext>
```

Arguments

`heContext`

The `heContext` representing the current cellview or design.

Returns

`<none>`

Examples

```
if {[he::isLeaf $i -context $h] != 1 } {  
    he::pushContext $h -inst $i  
    puts "The current occurrence path is: [db::getAttr \  
        occurrence -of $h]"  
    he::popContext $h  
}
```

In this example, `he::isLeaf` checks to find out if the given instance `$i` is not a leaf cell in the current context. Then the example calls `he::pushContext` to descend into the given instance. After descending, it prints the occurrence path. Finally, it calls `he::popContext` to return the context to the top context. Note that `he::pushContext` and `he::popContext` do not return any objects, instead, they change the provided `$h` context object.

he::pushContext

Description

This function descends into the given instance. If the optional `-master` argument is provided then it pushes out of context into the specified master. If the `-master` argument is not specified, then the default descend path is used.

This command is provided primarily for interactive editor support.

Note:

It is not an error to push through an instance into an unrelated `-master` cellview. Pushing into a master which does not match the default descent path results in an out of context `heContext` object.

Syntax

```
he::pushContext <context: heContext> -inst <oaInst> [-master <dmCellView  
    | dmCellViewName | oaDesign>]
```

Arguments

`heContext`

The `heContext` representing the current cellview or design.

`-inst oaInst`

The instance to descend into.

`-master dmCellView | oaDesign`

The new master for the `heContext` after the push operation. Only used for out of context descent.

Default: NULL.

Returns

<none>

Examples

```
if {[he::isLeaf $i -context $h] != 1 } {  
    he::pushContext $h -inst $i  
    puts "The current occurrence path is: [db::getattr \  
        occurrence -of $h]"  
    he::popContext $h  
}
```

In this example, `he::isLeaf` checks to find out if the given instance `$i` is not a leaf cell in the current context. Then the example calls `he::pushContext` to descend into the given instance. After descending, it prints the occurrence path. Finally, it calls `he::popContext` to return the context to the top context. Note that `he::pushContext` and `he::popContext` do not return any objects, instead, they change the provided `$h` context object.

he::queryViewSelection

Description

This function queries the specified `<heContext>` object to determine the rules used to select the switched master and meta data for an `oaInst`.

Syntax

```
he::queryViewSelection -context <heContext> -inst <oaInst>
```

Arguments

`-context` `heContext`

Context in which the passed in `oaInst` must be found.

`-inst` `oaInst`

Instance queried.

Returns

A textual description of how the final switched master for the given object is obtained.
(<string>)

Errors

If queried instance does not exist in the passed in `heContext`.

Examples

To query the specified `<heContext>` object to determine the rules used to select the switched master:

```
set heCtx [db::getAttr hierarchy -of [de::getActiveContext]]
set inst [db::getInsts I0 -of [ed]]
he::queryViewSelection -context $heCtx -inst $inst
# Sample output
# global view search list "hspice hspiceD cmos.sch cmos_sch schematic
  veriloga "
```

This example queries the instance `I0` selection rule in the specified hierarchy context.

he::saveAsPropDB

Description

Saves the specified *hePropDB* database to the specified location.

The `-name` argument can be used to change the name of the saved property database. It does not rename the in-memory property database.

Note:

The `-overwrite` option is used only if the `-filePath`, `-name`, or `-topCellView` is specified.

Syntax

```
he::saveAsPropDB <hePropDB: hePropDB> [-filePath <string>] [-topCellView  
  <dmCellView | dmCellViewName>] [-overwrite <bool>] [-name <string>]
```

Arguments

`hePropDB`

The database to save.

`-filePath`

The path to write the given database.

`-topCellView`

The cellview to save the database to.

`-overwrite`

If false, the command does not overwrite the existing database.

Default is: false.

`-name`

The new name to save the database to.

Returns

<none>

Errors

- No write permission to the file system.
- If the specified destination *hePropDB* is already open.
- If neither `-name` nor `-filePath/-topCellView` are specified.

Examples

To save a hierarchical property database with a new name:

```
set myPropDB [he::openPropDB myPropDB -topCellView $cellView]
he::saveAsPropDB $myPropDB -name dbNewName -topCellView $cellView
```

This example creates or opens a property database with the name `myPropDB` and saves it with a new name `dbNewName`.

he::saveConfig

Description

This function writes the contents of the `heConfig` object to either the OA database or directly to the specified file name. If the optional `-cellView` arguments is specified then the command operates as `SaveAs` creating the cellview if necessary using the appropriate configuration `oaViewType`.

When performing a `SaveAs` the `heConfig` object is written to the specified cellview. The passed in `heConfig` object remains associated with the previous cellview. A new config object is returned which is associated with the passed in `-cellView`. The access mode of the returned config object is `w`.

If the `heConfig` object is written directly to a file then the returned `heConfig` is not associated with a cellview. The access mode of the `heConfig` is `w`.

If neither `-cellView` nor `-pathName` is specified then the command works semantically like a "normal" save. If the `heConfig` was associated with a cellview then the `heConfig` is written to the specified cellview and the passed-in `heConfig` is returned. If the `heConfig` was associated with an external file then the external file is overwritten and the passed-in `heConfig` is returned. If the `heConfig` was not associated with either a cellview or an external file then an error is generated.

Syntax

```
he::saveConfig <config: heConfig> [-cellView <dmCellView>] [-pathName  
  <string>]
```

Returns

The saved heConfig object ([heConfig](#)).

Arguments

[heConfig](#)

The heConfig object to be saved.

`-cellView` [dmCellView](#)

The cellview in which to store the heConfig. Note that the cellview does not have to exist. You can only set `<-cellView>` or `<-pathName>`, not both.

`-pathName` <string>

The destination output file. You can only set `<-cellView>` or `<-pathName>`, not both.

Errors

- If the destination `-cellView` or file is not writable.
- If the specified `-cellView` exists and is not a configuration cellview.
- If the heConfig object was created in-memory and is not associated with a cellview or external file.

Examples

```
set cv [dm::createCellView config -cell [dm::findCell ALU2 \  
  -libName ALU8] -viewType config.xml]  
he::saveConfig $cfg -cellView $cv
```

This example creates a new cellview of `viewType config.xml`. Then it calls `he::saveConfig` to save the new config view.

he::savePropDB

Description

Saves the specified *hePropDB* database.

Syntax

```
he::savePropDB <hePropDB: hePropDB>
```

Arguments

hePropDB

The database to save.

Returns

<none>

Errors

- No write permission to the file system.
- If the database is *scratch=true*, the database is not associated with an on-disk file and the `he::saveAsPropDB` command must be used.

Examples

To save a hierarchical property database after it has been altered:

```
set ctx [de::getActiveContext]
set heCtx [db::getattr ctx.hierarchy]
set cellView [db::getattr ctx.editFile.container]
#Open or create hier database files
set myPropDB [he::openPropDB myPropDB -topCellView $cellView]
he::setPropValue myPropName -db $myPropDB -value "myPropValue" -context
    $heCtx
he::savePropDB $myPropDB
```

This example adds a property with a value to a hierarchical property database and then saves it.

he::setPropValue

Description

Sets the specified property to the given value. If a property already exists in the given scope then the property value is updated.

The full hierarchical path is specified by the `-context` and `-inst` arguments. If `-inst` is not specified, the path is given by *[db::getattr occurrence -of \$context]*. If specified, the value of `-inst` must reference an instance within the given `-context`. The full hierarchical path is the concatenation of the occurrence of the `-context` and the name of the specified `-inst`.

The `-scope` defaults to *occurrence*. To modify the cell or instance scope, `-scope` must be specified.

Syntax

```
he::setPropValue <name: string> -value <string> -db <hePropDB> -context
    <heContext> [-inst <oaInst | string>] [-scope <enum>] [-propType <enum>]
```

Arguments

`name`

The name of the property.

`-value`

The value of the property.

`-db`

The HDB to modify.

`-context`

The queried `heContext`.

`-inst`

Instance Name.

`-scope`

Specifies the scope to modify (`{ cell | instance | occurrence }`, default is: `occurrence`)

`-propType`

Specifies the type to use for creating the property (`{ bool | float | int | string }`)

Returns

`<heProp>` : The created/modified property.

Errors

- If `-type` is specified and does not match the type of the existing *heProp* property.
- If the value cannot be coerced into the type of the existing *heProp* property. For example, if the given value is "foo" but the existing property is a numeric type such as `int` or `float`.
- If the `-inst` does not exist in the specified `-context`.
- If the top cellview of `-context` does not match the `hePropDB` database.

Examples

```
# "annotationIndex" is set to 1 in the occurrence scope
```

```
he::setPropValue "annotationIndex" -value 1 -context $ctx -inst "i<0:1>"  
-db $hdb
```

```
# Set a value at the cell level for the cell specified by the given  
context
```

```
he::setValue "accuracy" -value 3 -context $ctx -db $db -scope cell
# Set a value at the instance level.
he::setValue "accuracy" -value 4 -context $ctx -db $db -scope inst

# The above refers to the inst given by [db::getattr parentInst -of $ctx]
```

To add some data to a hierarchical property database:

```
set ctx [de::getActiveContext]
set heCtx [db::getattr ctx.hierarchy]
set cellView [db::getattr ctx.editFile.container]
#Open or create hier database files
set myPropDB [he::openPropDB myPropDB -topCellView $cellView]
he::setPropValue myPropName -db $myPropDB -value "myPropValue" -context
  $heCtx
```

This example adds a new property to a propDB object with the name `myPropName` and with the value `myPropValue`.

he::setRuleFromDE

Description

Sets a view selection rule to the instance with specified name in the specified window.

Syntax

```
he::setRuleFromDE -viewName <string> -instName <string> [-occurrence
  <string>] -in <giWindow>
```

Arguments

`-viewName <string>`

The view name to select.

`-instName <string>`

The name of the active instance.

`-occurrence`

The occurrence path to add the rule to

`-in <giWindow>`

The window with a design containing a specified instance.

Returns

<none>

he::showExportToFile

Description

Invokes the `heExportToFile` dialog box.

Syntax

```
he::showExportToFile [-parent <giWindow | int>]
```

Arguments

`-parent`

Active editor window.

Returns

`<giDialog>` : Show Export To File

he::showHierarchyEditor

Description

Creates a new Hierarchy Editor window.

Syntax

```
he::showHierarchyEditor [-config <heConfig>] [-readOnly <bool>]
```

Arguments

`-config` `heConfig`

The `heConfig` to edit.

Default: NULL.

`-readOnly` `<bool>`

Sets the editor to read-only mode.

Default: false.

Returns

Newly created Hierarchy Editor window. (`giWindow`)

Examples

To open a new Hierarchy Editor window:

```
set cv [dm::findCellView schematic -cellName Opamp_transient -libName  
MonteCarlo]  
he::showHierarchyEditor -cellView $cv
```

This example opens the Hierarchy Editor for the specified top cellview.

he::showLogicalSelectionOptions

Description

Creates a new hierarchy editor window.

Syntax

```
he::showLogicalSelectionOptions -parent <giAssistant>
```

Arguments

-parent

Parent assistant.

Returns

<giDialog>, options dialog object.

Examples

```
he::showLogicalSelectionOptions -parent [ gi::getAssistants\  
heDesignNavigator -from [gi::getWindows 2 ]]
```

In this example, he::showLogicalSelectionOptions displays the Logical Selection Options window for Design Navigator that is located in the second window.

he::showObject

Description

This command shows the specified object in the Design Navigator by switching the active view and scrolling to the specified item in the list.

Syntax

```
he::showObject <object: deHierObject> -assistant <giAssistant>
```

Arguments

`object`

Object to show.

`-assistant`

Design Navigator assistant object.

Returns

<none>

he::undoRedoHistory

Description

Performs an undo or redo for the Hierarchy Editor.

Syntax

```
he::undoRedoHistory -windowId <int> [-redo <bool>] [-steps <int>]
```

Arguments

`-windowId`

The window ID of the Hierarchy Editor window.

`-redo`

If true, performs a redo; otherwise performs an undo by default.

Default: false.

`-steps`

Number of undo or redo history items. (Default is: 1)

Returns

<none>

Enumerations

Table 108 Enumerations

Name	Values	Description
enViewMode	cell, tree	

Objects

This section covers the following:

- [dmCell](#)
- [dmCellView](#)
- [giWindow](#)
- [heConfig](#)
- [heConfigTemplate](#)
- [heContext](#)
- [heInstHeader](#)
- [heRule](#)
- [heViewNameGroup](#)
- [oaInst](#)

dmCell

A named container, in a library, that can hold files and dmCellViews. The dmCell corresponds to a specific oaCell object but is valid as long as the library that the dmCell belongs to remains open. The getAccess requirements of an oaCell are handled automatically by this dm object.

Table 109 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the Cell.
libName	string	Read-only	The name of the Cell's library.

dmCellView

The dmCellView is a container in a library that holds files and is identified by the combination of its cell and view names. It corresponds to an oaCellView but it stays valid (in that the attributes of the object can be accessed even if the underlying cellview does not exist) as long as its library remains open and its dmCell and dmView exist.

Table 110 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the view that this cellview belongs to.
cellName	string	Read-only	The name of the cell that this cellview belongs to.
cell	<i>dmCell</i>	Read-only	The cellview's cell.
libName	string	Read-only	The name of the library that this cellview belongs to.

giWindow

The `giWindow` object represents a single window and has a container of menus, toolbars, assistants and one status bar. Menus, toolbars and assistants may be retrieved by the API. The status bar and toolbar that are used to hold menus are available here as attributes as they are non-removable attributes of the window object. The attributes of the object are described in the table below and can be retrieved or set by `db::getAttribute` and `db::setAttribute` commands. Windows are created by various subsystems in the Custom Compiler environment including the Design Editor.

Table 111 Attributes

Name	Type	Access	Description
id	int	Read-only	Window identifier.
windowType	<i>giWindowType</i>	Read-only	Window type object.
title	string	Read-only	Title displayed in window border.
iconTitle	string	Read/Write	Title given to the window's icon and displayed in the window manager's task list.
geometry	string	Read/Write	Window geometry.
iconified	bool	Read/Write	Whether the window is iconified.
maximized	bool	Read/Write	Whether the window is maximized. Note that this can be true even if the window is currently iconified.

Table 111 Attributes (Continued)

Name	Type	Access	Description
statusBar	<i>giStatusBar</i>	Read-only	Status bar object holding actions that renders this window's status bar. If empty, the status bar is not displayed in the window.

heConfig

The heConfig object represents a configuration in Tcl. heConfig objects store the rules for traversing design hierarchies.

Table 112 Attributes

Name	Type	Access	Description
bindProp	[string]	Read/Write	The name of the property used to determine default binding. If the string is empty then properties are not used to determine binding.
cellOpenList	[dmCell/]	Read/Write	The list of cells and cell names for designs whose switched master instantiations the configuration omits and leaves as open circuits.
cellView	<i>dmCellView</i>	Read/Write	The Cellview in which the heConfig is stored. heConfig is not required to be associated with a dmCellView. If the configuration is not associated with a storage cellview then this attribute is a dmCellView with libName/cellName and viewName set to "".
hasEditor	[bool]	Read	Indicates that the config has an editor refers to it
ignorePropNames	[string]	Read/Write	The list of names for oaStringProp(s) on instances or placed masters. When the value is "ignore", the configuration omits and leaves as open circuits.

Table 112 Attributes (Continued)

Name	Type	Access	Description
libOpenList	[string]	Read/Write	The list of libraries with designs whose switched master instantiations the configuration omits and leaves as open circuits.
libSearchList	[string]	Read/Write	The list of libraries the configuration searches to bind instances to switched masters.
mode	<i>oaChar</i>	Read/Write	Access mode of the object. If set to false the object can still be modified in memory, however <code>he::save</code> fails.
modified	[bool]	Read	Indicates that the config is modified and needs to be saved.
overlayViewName	[string]	Read/Write	The overlay view name for the config
shortPropNames	[string]	Read/Write	The list of names for <code>oaStringProp(s)</code> on instances and placed masters the configuration replaces with net aliases.
topCellView	<i>dmCellView</i>	Read/Write	Top cellview of the configured hierarchy.
viewSearchList	[string]	Read/Write	The Global View Search List.
viewStopList	[string]	Read/Write	The list of view names to be used as leaf cellviews.

heConfigTemplate

The `heConfigTemplate` is an association between a name and a configuration file saved on disk. The name field is used by the HE GUI to display the list of available templates.

Table 113 Attributes

Name	Type	Access	Description
name	string	Read-only	The name displayed in the HE GUI for the template.

Table 113 *Attributes (Continued)*

Name	Type	Access	Description
filePath	string	Read/Write	The path to a configuration file stored on disk.

heContext

The heContext represents a hierarchical iterator. It can be used to traverse Placed, Switched, and View Search List hierarchies.

Table 114 *Attributes*

Name	Type	Access	Description
topCellView	<i>dmCellView</i>	Read-only	The root Cellview of the context.
config	<i>heConfig</i>	Read-only	The heConfig object the context is traversing. Can be empty.
occurrence	string	Read-only	The current descent lineage of the context in the oaCdbaNS namespace.
design	<i>oaDesign</i>	Read-only	The current design.
viewSearchList	[string]	Read-only	The effective View Search List for the current descent path.
viewStopList	[string]	Read-only	The list of view names used as the stop list in he::isLeaf.
cellView	<i>dmCellView</i>	Read-only	The current cellview.
isRoot	bool	Read-only	True if the context is at the root of the hierarchy (occurrence == "/").
parentInst	<i>oaInst</i>	Read-only	The parent instance of the context.
explicitParentInst	<i>oaInst</i>	Read-only	The explicit parent instance. If the parent instance is an implicit instance then the containing explicit oaVectorInst is returned.

heInstHeader

The heInstHeader is the instance header object for the Hierarchy API. It provides data and methods for determining the placed master, the switched master and the associated instances.

Table 115 *Attributes*

Name	Type	Access	Description
placedMaster	<i>dmCellView</i>	Read-only	The placed master of this instance header.
switchedMaster	<i>dmCellView</i>	Read-only	The switched master of this instance header.

heRule

The heRule object represents view selection rules. heRule's are used throughout the Hierarchy API to define how design hierarchies are elaborated.

Table 116 *Attributes*

Name	Type	Access	Description
cell	<i>dmCell</i>	Read-only	Cell this rule applies to.
cellView	<i>dmCellView</i>	Read-only	Parent cellview of the instance this rule applies to.
instName	string	Read-only	Name of the instance this rule applies to.
occurrence	string	Read-only	The occurrence path of the rule.
level	<i>enRuleLevel</i>	Read-only	The level the rule applies to.
attrName	<i>enRuleType</i>	Read-only	The type of the rule.
select	string	Read-only	The view name used for a selection type rule.
viewSearchList	[string]	Read-only	The View Search List to use for viewSearchList rule types.

heViewNameGroup

The heViewNameGroup represents an ordered list of view names and other View Name Groups. It is used through out the Hierarchy API to allow user defined lists of views.

Table 117 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the view name group.
views	[string]	Read-only	The list of views associated with the view name group.

oalnst

Represents block domain instances. An instance represents the inclusion of one design as a part of the contents of another. Use the db::getAttr command to retrieve the oalnst attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 118 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the oalnst.
design	<i>oaDesign</i>	Read-only	Design to which the instance belongs.
master	<i>oaDesign</i>	Read-only	Master of the instance.
orient	<i>oaOrientEnum</i>	Read/Write	Orientation of the instance.
origin	point	Read/Write	Origin of the instance.
libName	string	Read-only	The library name for the master design referenced by the instance.
cellName	string	Read-only	The cell name for the master design referenced by the instance.
viewName	string	Read-only	The view name for the master design referenced by the instance.

Table 118 Attributes (Continued)

Name	Type	Access	Description
header	<i>oaInstHeader</i>	Read-only	The instHeader associated with this instance.
numBits	int	Read-only	The number of bits of this instance.
params	<i>oaParamArray</i>	Read/Write	Parameters that were specified during creation. Only Pcell instances have parameters.
placementStatus	<i>oaPlacementStatusEnum</i>	Read/Write	The placement status for the instance.
source	<i>oaSourceEnum</i>	Read/Write	The source for the instance.
isImplicit	bool	Read-only	A Boolean that indicates if the instance is implicit.

Preferences

Table 119 lists the Custom Compiler he namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 119 Hierarchy Editing Preferences

Name	Type	Scope	Default	Description
heAutoRefreshConfig	bool	cellview	true	Controls if the content of HED is automatically refreshed on config changes. Valid values: true, false
heAutoRefreshDesign	bool	cellview	true	Controls if the content of HED is automatically refreshed on design changes. Valid values: true, false
heCrossSelectFigGroups	bool	user	true	If enabled, fig groups are automatically selected from design fig groups in Design Navigator and in the command net inputs. Valid values: true, false

Table 119 Hierarchy Editing Preferences (Continued)

Name	Type	Scope	Default	Description
heCrossSelectInsts	bool	user	true	If enabled, instances are automatically selected from design instances in Design Navigator and in the command net inputs. Valid values: true, false
heCrossSelectInstTerms	bool	user	true	If enabled, instance terminals are automatically selected from design instances in Design Navigator and in the command net inputs. Valid values: true, false
heCrossSelectNets	bool	user	true	If enabled, nets are automatically selected from design shapes in Design Navigator and in the command net inputs. Valid values: true, false
heCrossSelectPins	bool	user	true	If enabled, pins are automatically selected from design shapes in Design Navigator and in the command net inputs. Valid values: true, false
heCrossSelectTerms	bool	user	true	If enabled, terminals are automatically selected from design shapes in Design Navigator and in the command net inputs. Valid values: true, false
heCrossSelectSize	unsigned int	user	1000	Design is considered large, if its objects' count exceeds this number.
heDescendToSelectInst	bool	user	false	Controls if it automatically descends to the cross-selected items of HED.
heDesignNavigatorFillingSize	unsigned int	user	1000000	Specifies the maximum number of objects to be visible in the Design Navigator.

Table 119 Hierarchy Editing Preferences (Continued)

Name	Type	Scope	Default	Description
heDesignNavigatorFilteringSize	unsigned int	user	500000	Displays up to the specified number of objects during a Filter operation in the Design Navigator.
heDesignNavigatorGroupsView	<enGroupsView>	context	all	Sub-view to use in the Groups tab of the Design Navigator
heDesignNavigatorInstsView	string	context	all	Sub-view to use in the Insts tab of the Design Navigator
heDesignNavigatorNetsView	<enNetsView>	context	all	Sub-view to use in the Nets tab of the Design Navigator
heDesignNavigatorShowContextSelector	bool	user	true	If enabled, Design Navigator context selector is shown. Valid values: true, false
heDesignNavigatorShowExtendedFlightlineInfo	bool	cellview	false	Displays the number of unconnected pins next to the net names in the Design Navigator. Valid values: true, false
heDesignNavigatorShowFilter	bool	user	true	If enabled, a Filter input widget is shown in the Design Navigator.
heDesignNavigatorShowInstMasterColumn	bool	user	true	Displays the Layout master column in the Design Navigator Insts tab.
heDesignNavigatorShowMasterInstCountColumn	bool	user	true	Displays the Layout Master Instances count column in the Design Navigator Instances tab.
heDesignNavigatorShowOpenNetsColumn	bool	user	true	Displays the Layout Master column in the Design Navigator Open Nets tab.
heDesignNavigatorShowShortedNetsColumn	bool	user	true	Displays the Layout Master column in the Design Navigator Shorted Nets tab.
heDesignNavigatorView	string	context	nets	Selected view.
heEditorAvailableViewList	string	user		Available view search/stop list when launching a view selection dialog.

Table 119 Hierarchy Editing Preferences (Continued)

Name	Type	Scope	Default	Description
heEditorDefaultViewSearchList	string	user		Default view search list when creating a new config
heEditorDefaultViewStopList	string	user		Default view stop list when creating a new config
heEnableCrossSelect	bool	user	false	Controls if the selection of Hierarchy Editor is automatically synchronizing to SE/SDL/TE
heExportHierDelimiter	string	user		Delimiter used when exporting hierarchy to file.
heExportHierOpenInEditor	bool	user	false	If true, opens the file created in export hierarchy from hierarchy assistant in the text editor. Valid values: true, false
heExportHierType	string	user	placedInstance InSwitchedHier archy	Criteria for exporting hierarchy to file.
heExternalNetlistFileBrowsePath	string	user		The default browse path of the external netlist file selector in the view selection dialog.
heFilterLibCellNames	string	cellview		A list of library name and cell name for HED to filter the items.
heKeepExpandStatus	bool	cellview	false	Controls if the expanded items are keep expanded after HED refresh.
heNavigatorAutoRefresh	bool	cellview	false	Controls if the Hierarchy Navigator is automatically refreshed based on design data and config changes. Valid values: true, false
heNavigatorCheckCDFIgnore	bool	cellview	true	Check CDF Parameter for Ignore Instances in Hierarchy Navigator
heNavigatorCrossHighlightSize	unsigned int	user	1000	Design is considered large, if its objects' count exceeds this number.

Table 119 Hierarchy Editing Preferences (Continued)

Name	Type	Scope	Default	Description
heNavigatorShowFind	bool	cellview	false	Controls showing or hiding the search line in the hierarchy navigator.
heNavigatorShowMaster	bool	cellview	true	Controls if the master is included on each instance line in the Hierarchy Navigator. Valid values: true, false
heNavigatorTrackContext	bool	cellview	true	Controls if the Hierarchy Navigator insures the current edit lineage is visible on context changes. Valid values: true, false
heSelectionTableFit	bool	user	false	Fit current selection immediately. Valid values: true, false
heSelectionTableShadow	bool	user	false	Shadow current selection immediately. Valid values: true, false
heShadowMode	bool	user	false	If enabled, selected objects are displayed brighter than surrounding objects. Valid values: true, false
heShadowNets	bool	user	false	If enabled, selected nets are shown brighter than surrounding objects. Valid values: true, false
heShadowPeerGroups	bool	user	false	If enabled, peers of the selected group are shown brighter than surrounding objects. Valid values: true, false
heShowBoundColumn	bool	cellview	true	Controls whether the View Found column is shown in the Hierarchy Editor. Valid values: true, false
heShowFind	bool	cellview	false	Controls whether the Find search box is shown in the Hierarchy Editor. Valid values: true, false

Table 119 Hierarchy Editing Preferences (Continued)

Name	Type	Scope	Default	Description
heShowLibrarySearchList	bool	cellview	true	Controls if the library search list editor is shown in the Hierarchy Editor. Valid values: true, false
heShowPaneBound	bool	cellview	true	Show the Hierarchy Editor pane's Bound column. Valid values: true, false.
heShowPaneConstraintView	bool	cellview	false	Show the Hierarchy Editor pane's Constraint View column. Valid values: true, false
heShowPaneConstraintViewSearchList	bool	cellview	false	Show the Hierarchy Editor pane's Constraint View Search List column. Valid values: true, false
heShowPaneSelected	bool	cellview	true	Show the Hierarchy Editor pane's Selected column. Valid values: true, false
heShowPaneSelectedConstraintView	bool	cellview	false	Show the Hierarchy Editor pane's Selected Constraint View column. Valid values: true, false
heShowPaneViewSearchList	bool	cellview	true	Show the Hierarchy Editor pane's View Search List column. Valid values: true, false
heShowPaneViewStopList	bool	cellview	false	Show the Hierarchy Editor pane's View Stop List column. Valid values: true, false
heShowSelectedColumn	bool	cellview	true	Controls whether the View Selected column is shown in the Hierarchy Editor. Valid values: true, false
heShowTopCellView	bool	cellview	true	Controls whether the top cellview editor is shown in the Hierarchy Editor. Valid values: true, false
heShowViewSearchList	bool	cellview	true	Controls if the view search list editor is shown in the Hierarchy Editor. Valid values: true, false

Table 119 Hierarchy Editing Preferences (Continued)

Name	Type	Scope	Default	Description
heShowViewStopList	bool	cellview	true	Controls if the view stop list editor is shown in the Hierarchy Editor. Valid values: true, false
heShowVSLColumn	bool	cellview	true	Controls if the View Search List column is shown in the Hierarchy Editor.
heViewMode	<enView Mode>	cellview	tree	View either the cell or the tree view panes in the hierarchy editor.

13

iPDK Tcl Procedures

This chapter lists the iPDK Tcl Procedures in Custom Compiler.

For details, see:

- [iPDK_* Procedures](#)

iPDK_* Procedures

The procedures covered are:

- [iPDK_engToSci](#)
- [iPDK_getCurrentInst](#)
- [iPDK_getInstCellName](#)
- [iPDK_getInstLibName](#)
- [iPDK_getOrient](#)
- [iPDK_getParamDef](#)
- [iPDK_getParamValue](#)
- [iPDK_islvpcell](#)
- [iPDK_isLayout](#)
- [iPDK_isSchematic](#)
- [iPDK_paramExist](#)
- [iPDK_sciToEng](#)
- [iPDK_setParamValue](#)

iPDK_engToSci

Description

Converts a number from engineering to scientific notation.

Syntax

```
iPDK_engToSci value
```

Arguments

value

The `value` to be converted.

Returns

The value in engineering notation.

iPDK_getCurrentInst

Description

Returns the object ID of the instance whose parameters are changed. This is required to set all the parameters calculated from the Tcl callback.

The returned object ID should not be passed to any standard OpenAccess API.

Note:

The returned object ID should be used only by the IPL Tcl API.

Syntax

```
iPDK_getCurrentInst
```

Returns

The Object ID of the instance whose parameters are changed.

iPDK_getInstCellName

Description

Returns the cell name of the specified instance object ID.

Syntax

```
iPDK_getInstCellName instId
```

Arguments

`InstID`

The `instId` returned by the `iPDK_getCurrentInst` procedure.

Returns

The cell name of the changed instance.

iPDK_getInstLibName

Description

Returns the library name of the specified instance object ID.

Syntax

```
iPDK_getInstLibName instId
```

Arguments

`InstID`

The `instId` returned by the `iPDK_getCurrentInst` procedure.

Returns

The library name of the changed instance.

iPDK_getOrient

Description

Gets the orientation of the instance.

Syntax

```
iPDK_getOrient instId
```

Arguments

`instId`

The `instId` returned by the `iPDK_getCurrentInst` procedure.

Returns

The orientation of the instance.

iPDK_getParamDef

Description

Gets the specified attribute value of the CDF definition of the specified CDF parameter.

Syntax

```
iPDK_getParamDef attribute instId param
```

Arguments

attribute

The parameter attribute.

instId

The `instId` returned by the `iPDK_getCurrentInst` procedure.

param

The parameter.

Returns

The attribute value of the specified parameter.

iPDK_getParamValue

Description

Gets the parameter value for the specified parameter from the instance on which the callback is triggered.

Syntax

```
iPDK_getParamValue paramName instId
```

Arguments

paramName

The parameter name

instId

The `instId` returned by the `iPDK_getCurrentInst` procedure

Returns

The value of the specified parameter of the selected instance.

iPDK_isIvpcell

Description

Checks if the cell view from which the callback was triggered is a ivPcell view.

Syntax

```
iPDK_isIvpcell
```

Returns

Returns 1 if cell view is of type ivPcell.

iPDK_isLayout

Description

Checks if the cell view from which the callback was triggered is a layout view.

Syntax

```
iPDK_isLayout
```

Returns

Returns 1 if cell view is of type layout.

iPDK_isSchematic

Description

Checks if the cell view from which the callback was triggered is a schematic view.

Syntax

```
iPDK_isSchematic
```

Returns

Returns 1 if cell view is of type schematic.

iPDK_paramExist

Description

Checks if the CDF parameter exists.

Syntax

```
iPDK_paramExist paramName instId
```

Arguments

paramName

The name of the parameter.

instID

The instance id returned by the `iPDK_getCurrentInst` procedure.

Returns

Returns 1 if parameter exist, else returns 0.

iPDK_sciToEng

Description

Converts a number from scientific to engineering notation.

Syntax

```
iPDK_sciToEng value
```

Arguments

value

The `value` to be converted.

Returns

The value in scientific notation.

iPDK_setParamValue

Description

Sets the CDF parameter value on the current instance.

Syntax

```
iPDK_setParamValue paramName paramValue  
                    instId  
                    [evalCB <true|false>]
```

Arguments

`paramName`

The parameter name.

`paramValue`

The parameter value.

`instId`

The `instId` returned by the `iPDK_getCurrentInst` procedure.

`evalCB`

If set to `true`, a callback is triggered to re-evaluate related CDF parameters.
Default is `true`.

Returns

<none>

14

Layout Editing Tcl

For details, see

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

The commands covered are

- [le::addDummy](#)
- [le::addToFigGroup](#)
- [le::addVirtualConnection](#)
- [le::align](#)
- [le::alignAngle](#)
- [le::autoFix](#)
- [le::autoVia](#)
- [le::boolean](#)
- [le::breakSync](#)
- [le::changePinRelationship](#)
- [le::checkAutoConfiguration](#)
- [le::checkCapacitance](#)
- [le::checkDRCInRegion](#)
- [le::checkDRCInViewRegion](#)

- [le::checkDRCViolations](#)
- [le::checkDRCViolationsByICV](#)
- [le::checkEMResConstraints](#)
- [le::checkOnModificationByICV](#)
- [le::checkOpens](#)
- [le::checkPinConnectivity](#)
- [le::checkShapeEM](#)
- [le::checkShield](#)
- [le::checkShorts](#)
- [le::checkTrackSnapping](#)
- [le::checkVias](#)
- [le::checkVoltageAnnotations](#)
- [le::chop](#)
- [le::clearFlightlines](#)
- [le::clearPinConnections](#)
- [le::clearVoltagePhysicalAnnotations](#)
- [le::clearVoltages](#)
- [le::cloneMaster](#)
- [le::closeICVManager](#)
- [le::closeStarRC](#)
- [le::compactBus](#)
- [le::connectICCNet](#)
- [le::convertToAliasLayer](#)
- [le::convertToArray](#)
- [le::convertToPath](#)
- [le::convertToPolygon](#)
- [le::copy](#)

- [le::copyHighlighted](#)
- [le::copyLook](#)
- [le::copyTrackRegionDef](#)
- [le::correctOffgrid](#)
- [le::createAttributeLabel](#)
- [le::createAbutGroups](#)
- [le::createBlockage](#)
- [le::createBoundary](#)
- [le::createCellViewFromSelection](#)
- [le::createCoordinateMark](#)
- [le::createCorridorArea](#)
- [le::createDRDLineEdit](#)
- [le::createDRDTreeWidget](#)
- [le::createEllipse](#)
- [le::createGlobalTrackRegion](#)
- [le::createGuardRing](#)
- [le::createICCIInst](#)
- [le::createICNet](#)
- [le::createInst](#)
- [le::createLabel](#)
- [le::createMPP](#)
- [le::createMPPDef](#)
- [le::createMPPEnclosedPathDef](#)
- [le::createMPPEnclosedShapeDef](#)
- [le::createMPPMasterPathDef](#)
- [le::createMPPOffsetPathDef](#)
- [le::createMPPRingDef](#)

- [le::createMPPSubRectDef](#)
- [le::createNetGroup](#)
- [le::createOffTrackInfo](#)
- [le::createPath](#)
- [le::createPathSeg](#)
- [le::createPin](#)
- [le::createPinsFromText](#)
- [le::createPolygon](#)
- [le::createRectangle](#)
- [le::createRouteCorridor](#)
- [le::createRow](#)
- [le::createSiteDef](#)
- [le::createStripedWire](#)
- [le::createStripeSegment](#)
- [le::createSyncFigGroup](#)
- [le::createSyncGroup](#)
- [le::createTerm](#)
- [le::createTrack](#)
- [le::createTrackGroupDef](#)
- [le::createTrackPattern](#)
- [le::createTrackPatternDef](#)
- [le::createTrackPcell](#)
- [le::createTrackRegion](#)
- [le::createTrackRegionDef](#)
- [le::createVia](#)
- [le::createVoltagePhysicalAnnotations](#)
- [le::cutCorner](#)

- [le::cycleDensity](#)
- [le::delete](#)
- [le::deleteCutMetal](#)
- [le::deleteDensityResult](#)
- [le::deleteGeneratedPins](#)
- [le::deleteVirtualConnection](#)
- [le::designSummary](#)
- [le::displayFlightlines](#)
- [le::displayPinConnections](#)
- [le::distribute](#)
- [le::EMAccuracyTool](#)
- [le::EMRCDecryptFile](#)
- [le::EMRCCEMResultToEV](#)
- [le::EMRCRevertLogFile](#)
- [le::EMRCXoutToDSPFFile](#)
- [le::enableDebugLogModule](#)
- [le::evalViaParams](#)
- [le::exportResistanceQuery](#)
- [le::extractLayoutContext](#)
- [le::extendHandleSelection](#)
- [le::externalizeColor](#)
- [le::fill](#)
- [le::fixAcuteAngle](#)
- [le::fixMinWidthShapes](#)
- [le::fixTrackSnapping](#)
- [le::flatten](#)
- [le::generateAbstract](#)

- [le::generateAntennaRunset](#)
- [le::generateCutMetal](#)
- [le::generateGuardRing](#)
- [le::generateLabel](#)
- [le::generateLayout](#)
- [le::generateLayoutContext](#)
- [le::generatePGMarkers](#)
- [le::generatePins](#)
- [le::generateShapes](#)
- [le::generateSoftAbstract](#)
- [le::genPinFigureShorts](#)
- [le::getAbuttedInsts](#)
- [le::getAbuttedMPPs](#)
- [le::getAbutterAbuttee](#)
- [le::getAllAbutterAbutteeInDes](#)
- [le::getAreaAndPerimeter](#)
- [le::getBoundaryTrackRegionDefs](#)
- [le::getCommonPathPoints](#)
- [le::getCoordinateMarks](#)
- [le::getGateDistance](#)
- [le::getGateRectangles](#)
- [le::getGlobalTrackRegions](#)
- [le::getIncompleteNets](#)
- [le::getInstanceBBox](#)
- [le::getIntersections](#)
- [le::getMPPDefs](#)
- [le::getMPPs](#)

- [le::getNearestTracks](#)
- [le::getNetGroups](#)
- [le::getNetIslands](#)
- [le::getPinFigureShorts](#)
- [le::getPseudoParallelNets](#)
- [le::getRouteCorridors](#)
- [le::getRuleInfoByICV](#)
- [le::getStretchHandleCategorizedInfo](#)
- [le::getStripedWires](#)
- [le::getSyncGroups](#)
- [le::getTrackGroupDefs](#)
- [le::getTrackPatternDefs](#)
- [le::getTrackPatterns](#)
- [le::getTrackRegions](#)
- [le::getTrackRegionDefs](#)
- [le::getTracks](#)
- [le::getTracksInRegion](#)
- [le::getUnboundGlobalTrackRegions](#)
- [le::getUnboundTracks](#)
- [le::getUnboundTrackRegions](#)
- [le::getUnnettedFigs](#)
- [le::highlightConnected](#)
- [le::importCommandBindings](#)
- [le::importResistanceQuery](#)
- [le::importVoltages](#)
- [le::isInSync](#)
- [le::merge](#)

- [le::move](#)
- [le::optimize](#)
- [le::padLabel](#)
- [le::pasteLook](#)
- [le::propagateConnectivity](#)
- [le::refreshDensityResult](#)
- [le::regenerateMPP](#)
- [le::remaster](#)
- [le::removeFromFigGroup](#)
- [le::removeOverlappingCuts](#)
- [le::removeUnboundInstanceTerms](#)
- [le::repeatCopy](#)
- [le::replaceVia](#)
- [le::reportCapacitance](#)
- [le::reportCapacitanceFromEPA](#)
- [le::reportCELayers](#)
- [le::reportConnectivityPoints](#)
- [le::reportEM](#)
- [le::reportEMBackground](#)
- [le::reportNets](#)
- [le::reportNetsFromEPA](#)
- [le::reportRC](#)
- [le::reportResistance](#)
- [le::reportResistanceFromEPA](#)
- [le::reportShield](#)
- [le::resetVoltageOverrides](#)
- [le::reshape](#)

- [le::resize](#)
- [le::rotate](#)
- [le::scalarizeArrayInst](#)
- [le::selectStretchHandle](#)
- [le::setDRDOption](#)
- [le::setNetAnchor](#)
- [le::showAlignAssistOptions](#)
- [le::showCommandBindings](#)
- [le::showCreateTrack](#)
- [le::showCreateTrackGroupDefinition](#)
- [le::showCreateTrackRegionDefinition](#)
- [le::showDefineTerminals](#)
- [le::showDesignOptions](#)
- [le::showDRDOptions](#)
- [le::showDRDRuleSelection](#)
- [le::showEditBlackBox](#)
- [le::showEditFCBlackBox](#)
- [le::showEditICC2BlackBox](#)
- [le::showEditTrack](#)
- [le::showEditTrackGroupDefinition](#)
- [le::showEditTrackRegion](#)
- [le::showElectricalOptions](#)
- [le::showEMCalculator](#)
- [le::showImportLayout](#)
- [le::showLayoutGeneratorOptions](#)
- [le::showOLPOptions](#)
- [le::showTrackRegionOptions](#)

- [le::split](#)
- [le::splitArray](#)
- [le::splitIntoBoxes](#)
- [le::syncTo](#)
- [le::stretch](#)
- [le::toggleDRDTreeWidget](#)
- [le::trimWire](#)
- [le::validateColor](#)
- [le::yank](#)
- [ile::addDummy](#)
- [ile::addToFigGroup](#)
- [ile::adjustCursor](#)
- [ile::align](#)
- [ile::alignAngle](#)
- [ile::applyColor](#)
- [ile::attachDetach](#)
- [ile::autoChopGuardRing](#)
- [ile::autoFix](#)
- [ile::autoFlipHorizontal](#)
- [ile::autoFlipVertical](#)
- [ile::autoRotate90](#)
- [ile::boolean](#)
- [ile::bridgeInterconnect](#)
- [ile::changePinRelationship](#)
- [ile::checkCongestion](#)
- [ile::checkDensity](#)
- [ile::checkVias](#)

- [ile::checkVoltageAnnotations](#)
- [ile::chop](#)
- [ile::clearDensityDisplay](#)
- [ile::clearVoltagePhysicalAnnotations](#)
- [ile::clearVoltages](#)
- [ile::cloneMaster](#)
- [ile::collapseDensityTree](#)
- [ile::color](#)
- [ile::compactBus](#)
- [ile::continueBus](#)
- [ile::convertToArray](#)
- [ile::convertToPath](#)
- [ile::convertToPolygon](#)
- [ile::copy](#)
- [ile::copyLook](#)
- [ile::copyToClipboard](#)
- [ile::correctOffGrid](#)
- [ile::createAttributeLabel](#)
- [ile::createBlockage](#)
- [ile::createBoundary](#)
- [ile::createBus](#)
- [ile::createCellViewFromSelection](#)
- [ile::createCoordinateMark](#)
- [ile::createEllipse](#)
- [ile::createFigGroup](#)
- [ile::createGlobalTrackRegion](#)
- [ile::createGuardRing](#)

- [ile::createInst](#)
- [ile::createInterconnect](#)
- [ile::createLabel](#)
- [ile::createMPP](#)
- [ile::createPin](#)
- [ile::createPinsFromSelection](#)
- [ile::createPlane](#)
- [ile::createPolygon](#)
- [ile::createRectangle](#)
- [ile::createRouteCorridor](#)
- [ile::createRow](#)
- [ile::createRuler](#)
- [ile::createTopology](#)
- [ile::createTrackRegion](#)
- [ile::createTrackRegionPCell](#)
- [ile::createTransmissionLine](#)
- [ile::createVia](#)
- [ile::createVoltageConfig](#)
- [ile::createVoltagePhysicalAnnotations](#)
- [ile::cut](#)
- [ile::cutCorner](#)
- [ile::defineInFOLibrary](#)
- [ile::delete](#)
- [ile::deleteCutMetal](#)
- [ile::deleteFigGroup](#)
- [ile::distributeAlign](#)
- [ile::editFCBlackBox](#)

- [ile::editICC2BlackBox](#)
- [ile::editICCBlackBox](#)
- [ile::eraseLayer](#)
- [ile::expandDensityTree](#)
- [ile::fill](#)
- [ile::flatten](#)
- [ile::generateCutMetal](#)
- [ile::generateGuardRing](#)
- [ile::generateLabel](#)
- [ile::generateLayoutPins](#)
- [ile::generateMPP](#)
- [ile::generatePins](#)
- [ile::generatePlacementTrackRegion](#)
- [ile::generateRoutingTrackRegion](#)
- [ile::generateShapes](#)
- [ile::heal](#)
- [ile::hierarchicalCopy](#)
- [ile::highlightPath](#)
- [ile::importVoltages](#)
- [ile::inputCoord](#)
- [ile::lineSelect](#)
- [ile::measureAngle](#)
- [ile::measureDistance](#)
- [ile::merge](#)
- [ile::move](#)
- [ile::optimize](#)
- [ile::padLabel](#)

- [ile::partialToFull](#)
- [ile::paste](#)
- [ile::pasteLook](#)
- [ile::reassignNet](#)
- [ile::removeFromFigGroup](#)
- [ile::removeOverlappingCuts](#)
- [ile::replaceVia](#)
- [ile::reportCapacitance](#)
- [ile::reportEM](#)
- [ile::reportParasitic](#)
- [ile::reportResistance](#)
- [ile::reshape](#)
- [ile::reshapeInterconnect](#)
- [ile::resize](#)
- [ile::rotate](#)
- [ile::scalarizeArrayInst](#)
- [ile::selectByLPP](#)
- [ile::selectObjects](#)
- [ile::selectRouteObjects](#)
- [ile::showDensityDisplaySetting](#)
- [ile::showEMCalculator](#)
- [ile::split](#)
- [ile::splitArray](#)
- [ile::stackInterconnect](#)
- [ile::stretch](#)
- [ile::stretchArray](#)
- [ile::stretchWidth](#)

- [ile::stripeWire](#)
- [ile::tap](#)
- [ile::trimWire](#)
- [ile::validateColor](#)
- [ile::yank](#)
- [le::pkg::checkPlaneBoundary](#)
- [le::pkg::createDegassingHole](#)
- [le::pkg::createPlaneBoundary](#)
- [le::pkg::deleteDegassingHole](#)
- [le::pkg::deleteFloatingShapes](#)
- [le::pkg::deleteRedundantVias](#)
- [le::pkg::generatePlaneBoundary](#)
- [le::pkg::generateRedundantVias](#)
- [le::pkg::generateSealRing](#)
- [le::pkg::generateZigzag](#)
- [le::pkg::getPlaneBoundaries](#)
- [le::pkg::highlightPlaneBoundary](#)
- [le::pkg::realizePlane](#)
- [le::pkg::replacePlaneBoundary](#)
- [le::pkg::reportRedundantVias](#)
- [le::pkg::reportShieldingRate](#)
- [lepkg::refreshPlane](#)
- [ile::pkg::createDegassingHole](#)
- [ile::pkg::createPlaneBoundary](#)
- [ile::pkg::generateRedundantVias](#)
- [ile::pkg::generateZigzag](#)
- [ile::pkg::replacePlaneBoundary](#)

le::addDummy

Add Dummy Device

Syntax

```
le::addDummy <box: box> -design <oaDesign> [-mode <enum>] [-PMOSGateNet  
<string>] [-PMOSSDNet <string>] [-NMOSGateNet <string>] [-NMOSSDNet  
<string>] [-prefix <string>] [-fingerCount <int>] [-libName <string>]  
[-cellName <string>] [-viewName <string>] [-interAbuttedChains <bool>]
```

Returns

<oaInst> : Newly created dummy device

Arguments

`box`

The coordinates of bounding box of Add Dummy.

`-design`

Design in which to create the Add Dummy.

`-dummyRule`

Specify whether to add dummy by `IxDummyRule` or not. (Default is: false)

`-mode`

The mode of add dummy. ({ auto | fill | insert }, default is: auto)

`-PMOSGateNet`

The gate net name of the pmos dummy device.

`-PMOSSDNet`

The S/D net name of the pmos dummy device.

`-NMOSGateNet`

The gate net name of the nmos dummy device.

`-NMOSSDNet`

The S/D net name of the nmos dummy device.

`-prefix`

The prefix name of dummy device. (Default is: `IxDummy_`)

`-fingerCount`

The finger count of dummy device. (Default is: 1)

`-libName`

The library name of the dummy device.

`-cellName`

The cell name of the dummy device.

`-viewName`

The view name of the dummy device.

`-interAbuttedChains`

Specify if only filled the inter abutted chains. (Default is: false)

le::addToFigGroup

Description

Adds objects to the figure group.

Syntax

```
le::addToFigGroup <objects: {<defigure>} | {<deSelectionSet>} | {<oaFig>}  
| {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
-to <deFigure | oaFigGroup>
```

Returns

<oaFigGroup>: The OA figure group object updated by the command.

Arguments

object

Collection of objects to add to the given oaFigGroup.

-to

oaFigGroup to add the given objects to.

Examples

The following example adds I1 instance to FG__0 fig group.

```
set design [ed]  
le::addToFigGroup [db::getInsts I1 -of $design] -to [db::getFigGroups  
FG__0 -of $oaDeign]
```

le::addVirtualConnection

Description

Adds virtual connection.

Syntax

```
le::addVirtualConnection -design <oaDesign> -settingFile <string>
[-generateLabel <bool>] [-generatePin <bool>] [-lppMethod <enum>]
[-mapFile <string>] [-textLpp <lpp>] [-height <float>] [-just <enum>]
[-orient <enum>] [-font <enum>] [-drafting <bool>] [-overbar <bool>]
[-includePinFromInstance <bool>] [-assignToShape <bool>] [-purposeList
<string>]
```

Arguments

-design

Design in which to add virtual connection.

-settingFile

Setting file for add virtual connection.

-generateLabel

Specifies if generating labels.

Default: false

-generatePin

Specifies if generating pins.

Default: false.

-lppMethod

Specifies lpp method for generated labels.

Valid Values: inherit | map | specify | techLib

Default: inherit.

-mapFile

Specifies map file for generated labels.

-textLpp

Specifies layer and purpose pair of text.

-height

Height of the text in user units.

Default: 0.2.

`-just`

Justification of the text with respect to the origin.

Valid Values: centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight

Default: centerCenter.

`-orient`

Specifies the orientation of the text.

Valid Values: MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90

Default: R0.

`-font`

Specifies the font style for the text.

Valid Values: euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish

Default: stick.

`-drafting`

Indicates whether the text should not be mirrored or rotated.

Default: true.

`-overbar`

Renders text with overbar.

Default: false.

`-includePinFromInstance`

Specifies if including pin from instance when generating pins from net.

Default: false.

`-assignToShape`

Specifies if assigning pin information to 0x shapes directly.

Default: true.

`-purposeList`

Specifies the purpose list for new shapes in generated pins.

Default: pin drawing.

le::align

Description

Aligns specified objects, and also aligns the specified side of moveable objects to the specified side of reference (source) objects.

Syntax

```
le::align <objects: {<defigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
-src <deFigure | oaFig> [-side <enum>] [-distance <float>] [-direction  
<enum>] [-individually <>] [-byPitch <bool>]
```

Returns

{<deFigure>} | {<oaFig>} : Collection of aligned objects.

Arguments

`objects`

List of target objects to align.

`-src`

Source object for alignment; can be also a sub-object.

`-side`

Alignment side. The available sides are bottom, horizontalMiddle, left, right, top, and verticalMiddle. The value of this option is used only if at least one full object is specified.

Default: bottom.

`-distance <float>`

Distance between aligning objects.

Default: 0.

`-direction`

Direction according to which the distance between the aligning edges should be applied. The available directions are bottom, left, right, and top.

Default: right.

`-individually <bool>`

Control alignment of objects as pseudo group or individually.

Default: true.

`-byPitch`

Control alignment of objects by pitch or by spacing. (Default is: false)

Errors

- Invalid design, design not opened for write.
- Not all provided shapes form the same design.

Examples

The following example aligns an objects group to another object.

```
le::align [oa::getFigGroups [oa::getTopBlock $oaDesign]] -src  
[db::getNext [db::getShapes -of $design]] -side horizontalMiddle  
-individually false
```

le::alignAngle

Description

Aligns angle.

Syntax

```
le::alignAngle -sourceInst <deFigure | oaFig> -referenceInst <deFigure |  
oaFig> -sourcePinShape <deFigure | oaFig> -referencePinShape <deFigure  
| oaFig> [-spacing <float>] [-orthogonalSpacing <float>] [-wholeChain  
<bool>]
```

Returns

<none>

Arguments

`-sourceInst`

Source instance for angle alignment.

`-referenceInst`

Reference instance for angle alignment.

`-sourcePinShape`

Source pin shape for angle alignment.

`-referencePinShape`

Reference pin shape for angle alignment.

`-spacing`

Spacing between aligning objects. (Default: 0)

`-orthogonalSpacing`

Specifies orthogonal spacing between aligning objects. (Default: 0)

`-wholeChain`

Control alignment of objects align with whole chain or individually.

Default: true.

le::autoFix

Description

Corrects process and design constraint errors.

Syntax

```
le::autoFix [<box: box>] -design <oaDesign> [-insertJogs <enum>]  
          [-lockInstances <bool>] [-lockVias <bool>] [-lockInvisibleShapes <bool>]
```

Returns

{<defigure>}: Collection of deFigures modified by autoFix.

Arguments

`<box>`

The coordinates of bounding box of region to run AutoFix.

`-design`

Design in which to run AutoFix.

`-insertJogs`

Specifies if AutoFix should insert jogs aggressively, conservatively, or never.

Default: conservatively.

`-lockInstances`

Specifies if AutoFix should move instances.

Default: false.

`-lockVias`

Specifies if AutoFix should move vias.

Default: false.

`-lockInvisibleShapes`

Specifies if AutoFix should modify invisible shapes.

Default: false.

Examples

The following example fixes DRC violations in specified area.

```
set design [ed]
le::autoFix {{0 0} {10.0 20.0}} -design $design
```

The following example fixes DRC violations, without moving instances.

```
set design [ed]
le::autoFix -lockInstances true -design $design
```

le::autoVia

Description

Automates the via creation process.

Syntax

```
le::autoVia (-box <box> | -point <point>) -design <oaDesign>
  [-depthOn0xView <bool>] [-nets <string>]
  [-sameNetOnly <bool>] [-allowExceed <bool>] [-fitToOverlappedArea <bool>]
  [-createMetalShape <bool>]
  [-allOverlappedArea <bool>] [-minCutNum <int>] [-checkEnvAware <bool>]
  [-colRowDir <enum>]
  [-viaSetting <string>] [-layerDisable <string>]
```

Arguments

`-design`

Design in which the via will be created.

`-box`

Specifies the area within which the via will be created.

`-point`

Specifies the location at which the via will be created.

`-depthOn0xView`

Specifies the depth at 0x or current stop level. Default: false (stop level)

`-nets`

Searches the net names listed in this argument. Default: empty

`-sameNetOnly`

Restricts auto via creation to routes on the same net. Default: false

`-sameNetOnlyNoNetAnyNet`

Option to create auto via only on same net shapes where no net matches any net. (Default is: true)

`-sameNetOnlyNoNetNoNet`

Option to create auto via only on same net shapes where no net matches no net. (Default is: true)

`-allowExceed`

Permits the via to exceed the overlap region. Default: false

`-fitToOverlappedArea`

Fits metal shapes of the via fit perfectly in the overlap region. Default: false

`-createMetalShape`

Creates metal shapes for layers without any metal shapes. Default: false

`-allOverlappedArea`

Creates vias between overlapping shapes within the largest possible region under the pointer. This option is available only when Use Active Object is turned off. Default: false

`-allowStackedVia`

Creates stacked vias between overlapping metal shapes. Turn off this option for faster performance, such as when the design contains a large number of overlapping shapes. Default: true

`-checkEnvAware`

Option to check environment aware. (Default is: true)

`-autoBridgeVia`

Option to enable auto bridge via. (Default is: false)

`-minCutNum`

Specifies the minimum number of cuts. Default: 1

`-colRowDir`

Specifies direction of column and row. ({ Auto | H | V }). Default: Auto

`-viaSetting`

Specifies via setting. Example valid format is:

`{m1 viadef1 cutclass1}` or `{m1 viadef1 auto}` or `{m1 auto cutclass1}` or `{{m1 viadef1 cutclass1} or {m2 viadef2 cutclass2}}`

`-layerDisable`

Disables the specified layers. Example valid format is `{m6 m7 m8}`.

Returns

`<oaVia>` : Created auto via

Examples

The following example creates a via array between the M1 and M2 layers. The via array is created using the M1_M2x via definition and the VxBAR Cut Class.

```
set design [ed]
le::autoVia -box {{0 0} {1.0 0.5}} -design $design -viaSetting {{M1
M1_M2x VxBAR}}
```

le::boolean

Description

Generates shapes from fully selected objects.

Syntax

```
le::boolean <objects: {<defigure>} | {<deSelectionSet>} | {<oaFig>} |
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>
[-operation <enum>] [-points [<point>]] [-result <enum>]
```

Returns

`<{<oaFig>}>`: Collection of Boolean objects.

Arguments

`objects`

List of selected objects.

`-operation`

Operation to be performed. (`{ and | not | or | xor }`)

`-points`

Point list to form an area for Boolean operation.

`-result`

Result type of the operation.

Examples

The following example generates new `oaShape` objects by the AND operation of `$shapes` and the area specified with `-points` argument replacing the specified `$shapes` objects.

```
set design [ed]
# Get collection of shapes
set shapes [db::getShapes -of $design -lpp {m1 drawing}]
le::boolean $shapes -operation and -points {{0 0} {2.5 3.5}} -result
replace
```

The following example generates new `oaShape` objects with active LPP by the OR operation of `$shapes` and the area specified with `-points` argument.

```
set design [ed]
# Get collection of shapes
set shapes [db::getShapes -of $design]
le::boolean $shapes -operation or -points {{0 0} {1 1}} -result
newWithActiveLPP
```

le::breakSync

Description

Breaks the synchronous relation between the given `oaFigGroup` objects.

Syntax

```
le::breakSync <objects: {<defigure>} | {<oaFig>}>
```

Returns

{<oaFig>}: Collection of `oaFigGroup` objects obtained by breaking the provided `oaFigGroup` objects.

Arguments

`objects`

Collection of `oaFigGroup`/`deFigure` objects for which the synchronous relation is to be broken.

Examples

The following example breaks the synchronization between the `syncFigGroup` objects and their peers in the same `syncGroup` by converting them to normal `oaFigGroup` objects.

```
# Get a sync figure group with name syncClone_1_2 and
# break synchronization for the figure group
```

```
set figGroup [de::getFigures -design $design \  
              -filter {%object.name == "syncClone_1_2"}]  
le::breakSync $figGroup
```

le::changePinRelationship

Description

Changes pin relationship.

Lets you change the relationships between provided pin objects which can result in a pin being deleted or a new pin or terminal being created. When an input command receives pin shapes, the command operates on the oaPin and oaTerm objects attached to the pin shapes. As a result, the command can delete pins from the provided collection. Therefore, these pins should not be used without checking for their existence.

Syntax

```
le::changePinRelationship <pins: [oaPin]> -type <enum>
```

Arguments

pins

Collection of oaPin objects on which to perform the operation.

-type

New relationship value for the provided list of pins.

Examples

The following example sets the relationship between pin A and pin B to *must join*.

```
le::changePinRelationship [list [db::getPins A -of $oaDesign]  
                                [db::getPins B -of $oaDesign]] -type mustJoin
```

le::checkAutoConfiguration

Description

Checks for the guard ring auto configuration file.

Syntax

```
le::checkAutoConfiguration <file: string>
```

Arguments

file

Auto configuration file name.

le::checkCapacitance

Description

Reports whether selected nets meet the capacitance constraints in the layout. This method provides a quick pass/fail report.

If you set the `leReportEMRCBackgroundMode` preference to `true` when the EM Reporter and Parasitic Reporter run in the background, the violations are saved in the error viewer file. You can export the file to the Error Viewer.

Syntax

```
le::checkCapacitance <scope: oaDesign | {<oaNet>}>
```

Arguments

`scope`

Design or net to be checked. Select multiple nets to extract parasitics in a single run.

Returns

`<string>` : Violations found during checking.

le::checkDRCInRegion

Description

Checks DRC in the region.

Syntax

```
le::checkDRCInRegion [<objects: oaGroup | {<defigure>} |  
{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |  
{<oaMarker>} | {<oaNet>} | {<oaShape>} | {<oaVia>}}>] [-design  
<oaDesign>] [-box <box>]
```

Returns

`{<oaFig>}`: Collection of Markers.

Arguments

`objects`

List of objects to check.

`-design`

A design where DRC is being checked.

`-box`

The coordinates of bounding box of region to check DRC.

le::checkDRCInViewRegion

Description

Checks DRC in the viewing region.

Syntax

```
le::checkDRCInViewRegion [-window <deCanvas | giWindow | int>]
```

Returns

{<oaFig>}: Collection of Markers.

Arguments

`-window`

Window or canvas.

le::checkDRCViolations

Description

Checks for design rule violations within the viewport.

Syntax

```
le::checkDRCViolations [<objects: oaGroup|{<defigure>} |  
  {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaNet>}  
  | {<oaShape>} | {<oaVia>}}>] [-design <oaDesign>] [-box <box>]
```

Returns

<{<oaFig>}>: Collection of marker objects.

Arguments

`objects`

List of objects to check.

`-design`

Design in which the design rule checks will be performed.

`-box`

Coordinates of the region within which the design rule checks will be performed.

Examples

The following example checks DRC violation in the specified area and creates error markers if there are any.

```
set design [ed]
le::checkDRCViolations -design $design -box {{0 0} {5.6 10.2}}
```

The following example checks DRC violation on net object clk and creates error markers, if there are any.

```
set design [ed]
le::checkDRCViolations [db::getNets clk -of $design]
```

le::checkDRCViolationsByICV

Description

Checks for design rule violations within the viewport by ICV Live.

Syntax

```
le::checkDRCViolationsByICV -design <oaDesign> -box <box> -runset
<string> -oriRunset <string> [-layerMap <string>] [-arguments
<string>] -format <string> [-file <string>] [-runDir <string>]
[-checkRules <string>] [-skipRules <string>] [-controlVars <string>]
[-showErrorViewer <bool>] [-background <bool>] [-checkLayersOption
<int>]
```

Arguments

-design

Design in which the design rule checks will be performed.

-box

Coordinates of the region within which the design rule checks will be performed.

-runset

The runset for DRC checking.

-oriRunset

The original runset for DRC checking.

-layerMap

The layer mapping for DRC checking.

-arguments

The arguments for DRC checking.

`-format`

The input format (OpenAccess or Stream) for DRC checking.

`-file`

The input file for DRC checking.

`-runDir`

The directory to run the DRC checking.

`-checkRules`

The rules to check for DRC checking.

`-skipRules`

The rules to skip for DRC checking.

`-controlVars`

The control variables of DRC checking.

`-showErrorViewer`

Invoke Error Viewer after DRC checking.

`-background`

Invokes DRC checking in background.

`-checkLayersOption`

Specifies check layers option.

le::checkEMResConstraints

Description

Checks the electromigration and the resistance constraints.

If you set the `leReportEMRCBackgroundMode` preference to `true` when the EM Reporter and Parasitic Reporter run in the background, the violations are saved in the error viewer file. You can export the file to the Error Viewer.

Syntax

```
le::checkEMResConstraints <objects: oaDesign | {<oaGroup>} | {<oaNet>}>  
[-EM <bool>] [-Res <bool>] [-commonPath <bool>] [-commonPathMode  
<unsigned int>]
```

Arguments

`objects`

Nets, net groups, or the design to be checked. Select multiple nets to extract parasitics in a single run.

`-EM`

Check EM constraint violations for wires and vias. (Default is: true)

`-Res`

Check resistance constraint violations. (Default is: true)

`-commonPath`

Check common path resistance constraint violations. (Default is: true)

`-commonPathMode`

Mode of common path resistance calculations. Searches its whole connection network or the shortest path, and reports the first or all violating common paths. (Default is: 0)

Returns

`<string>`: Checked electromigration and resistance constraints.

le::checkOnModificationByICV

Description

Invokes On-modification Checking by ICV Live.

Syntax

```
le::checkOnModificationByICV -design <oaDesign> -runset <string>  
-oriRunset <string> [-layerMap <string>] [-arguments <string>] -format  
<string> [-file <string>] [-runDir <string>] [-checkRules <string>]  
[-skipRules <string>] [-controlVars <string>]
```

Arguments

`-design`

Specifies the design in which to check drc.

`-runset`

The runset for drc checking.

`-oriRunset`

The original runset for drc checking.

`-layerMap`

The layer mapping for drc checking.

`-arguments`

The arguments for drc checking.

`-format`

The input format to be used for drc checking, either OpenAccess or Stream.

`-file`

The input file for drc checking.

`-runDir`

The directory in which to run drc checking.

`-checkRules`

The rules to check for drc checking.

`-skipRules`

The rules to skip for drc checking.

`-controlVars`

The control variables of drc checking.

le::checkOpens

Description

Creates open flight lines markers.

Syntax

```
le::checkOpens (-design <oaDesign> | -nets <oaNet | {<oaNet>}>)
```

Returns

{<oaMarker>} : Collection of markers.

Arguments

`-design`

Design for which to create markers.

`-nets`

Nets for which to create markers.

`-depth`
Depth

`-hierarchyMode`
Hierarchy mode ({ full | top | view })

`-followFlightlines`
Reports opens based on flightlines

Examples

The following example creates error markers for found open nets and updates the Design Navigator to show open nets.

```
le::checkOpens -design [ed]
```

le::checkPinConnectivity

Description

Checks the pin connectivity.

Syntax

```
le::checkPinConnectivity <objects: oaDesign | {<defigure>} |  
{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |  
{<oaMarker>} | {<oaShape>} | {<oaVia>}}>
```

Returns

{<oaFig>}: Collection of Markers.

Arguments

`objects`
Collection of objects to be checked for pin accessibility.

le::checkShapeEM

Description

Checks the shape electromigration (EM) information.

Syntax

```
le::checkShapeEM [<scope: oaNet>] [-layerNum <int>] [-layerName <string>]  
-type <string> [-current <float>] [-width <float>] [-length <float>]  
[-color <string>] [-columnNum <int>] [-rowNum <int>] [-cutWidth <float>]
```

```
[-cutLength <float>] [-layer1Width <float>] [-layer1Length <float>]  
[-layer2Width <float>] [-layer2Length <float>] [-currentDir <string>]
```

Arguments

scope

Net to be reported.

-layerNum

Specifies the layer number.

-layerName

Specifies the layer name.

-type

Specifies the EM type in Avg/Peak/RMS/DC.

-current

Specifies the current in mA.

-width

Specifies the width in um.

-length

Specifies the length in um.

-color

Specifies the color in A/B.

-columnNum

Specifies the cut column number.

-rowNum

Specifies the cut row number.

-cutWidth

Specifies the cut width in um.

-cutLength

Specifies the cut length in um.

-layer1Width

Specifies Layer1 width in um.

`-layer1Length`

Specifies Layer1 length in um.

`-layer2Width`

Specifies Layer2 width in um.

`-layer2Length`

Specifies Layer2 length in um.

`-currentDir`

Specifies the cut current direction in UP/DOWN.

Returns

<string> : The shape EM.

Examples

The following example returns the minimum width by given the input of layer number, current type and current.

```
# Check EM and return minimum width for shape on M1 layer
# for a net CLK rated for a current 0.015mA
set net [db::getNets CLK -of $design]
le::checkShapeEM $net -layerName M1 -type Avg -current 0.015
```

le::checkShield

Description

Checks shield coverage for the specified design, a group of objects, or selected nets in the design. The results show violations, or areas that are not shielded. The violations are displayed in the Marker Browser.

Syntax

```
le::checkShield <objects: oaDesign | {<oaGroup>} | {<oaNet>}>
```

Returns

<string> : Check net shield constraints.

Arguments

`objects`

Nets, net groups, or design to be reported.

Examples

The following example checks shield for the selected nets or all the nets in a design.

```
# Check shield for net CLK in a design
set net [db::getNets CLK -of $design]
le::checkShield $net
```

le::checkShorts

Description

Checks for and reports short circuitry in the design.

Syntax

```
le::checkShorts [-hierarchyMode <enum> depth <unsigned int>] -design
<oaDesign>
[-skipCellsWithNoConnectivity <bool>]
```

Arguments

-design

Design in which you want to run the short checks.

-hierarchyMode

Short detection mode: full | top | view

- full: Full hierarchy
- top: Top-level only
- view: Viewport only

Default: full

-depth

Depth or stop level of the checks.

Default: Stop level if -checkEntireHierarchy is full; Connectivity Engine depth for all other -checkEntireHierarchy settings.

-skipCellsWithNoConnectivity

Ignores unnetted cells.

Default: true

Returns

<none>

le::checkTrackSnapping

Description

Checks off-track and other track-related attribute mismatch issues.

To fix the issues, see [le::fixTrackSnapping](#).

Syntax

```
{<leOffTrackInfo>} le::checkTrackSnapping [<objects: {<deFigure>} |  
{<oaInst>} | {<oaVia>} | {<oaShape>} > ] [-region <box>] [-context  
<deContext>]  
[-design <oa::oaDesign>] [-createMarker <bool>] [-checkSignalType <bool>]  
[-checkColor <bool>] [-orient <oaOrient>]
```

Returns

{<leOffTrackInfo>} : Collection of off-track or attribute-mismatch route and instance objects

Arguments

objects

List of objects to be checked. If you do not specify the objects, all route and instance objects in the design will be checked.

-region

Rectangular area within which the check will be performed. If you do not specify the region, the entire design will be checked.

-context

Context in which the command is activated: the design context or the window ID. If you do not specify the context, the current context will be used. The -context and -design arguments are mutually exclusive.

-design

The design in which the track snapping is checked. The -context and -design arguments are mutually exclusive.

-createMarker

Creates error markers for off-track objects. Default: false

-checkSignalType

Checks and reports signal type mismatches. Default: true

-checkColor

Checks and reports color mismatches. Default: true

`-orient`

Orientation of the region. Valid values: R0, R180, MX, and MY. For an existing track region already created in a design its orient is R0.

le::checkVias

Checks for missing / undersized vias.

Syntax

```
le::checkVias -design <oaDesign> [-mode <enum>] [-depth <int>] [-layers  
  [<oaPhysicalLayer | string>]] [-region <box>] [-nets [<oaBitNet |  
  string>]] [-routeTypes [<string>]] [-checkSize <bool>] [-numCuts <int>]  
  [-allowExceed <bool>] [-viaStackingRule <bool>] [-miscTypes [<string>]]
```

Returns

{<oaFig>} : Collection of Markers

Arguments

`-design`

Design to check.

`-mode`

Mode for via checker ({ constraints | misc | reliability }, default is: reliability)

`-depth`

Depth of hierarchy to check.

`-layers`

Range of layers to check.

`-region`

Region to check.

`-nets`

Nets to check.

`-routeTypes`

Route types to check.

`-checkSize`

Flag to turn on/off via size check.

`-numCuts`

Specify the minimum number of cuts.

`-allowExceed`

Flag to turn on/off dropping via without full metal coverage.

`-viaStackingRule`

Set to enable via stacking rule checking (Default is: false)

`-miscTypes`

Misc. checkings to enable. Available checking: "overlapAbut".

le::checkVoltageAnnotations

Description

Checks voltage annotations for all nets.

Syntax

```
le::checkVoltageAnnotations [<context: deContext>] [-scope <enum>]  
[-nets <string | {<oaNet>}>] [-depth <enum>] [-nets_with_voltage_value  
<bool>] [-only_voltage_lpp <bool>] [-check_udv <bool>]  
[-check_vsync <bool>] [-check_vflag <bool>] [-missing_net_annotation  
<bool>] [-missing_label <bool>] [-multiple_labels <bool>]  
[-incorrect_label_position <bool>] [-incorrect_label_value <bool>]  
[-incorrect_label_lpp <bool>] [-missing_voltage_inst <bool>]  
[-extra_voltage_inst <bool>] [-mismatched_voltage_inst_connection  
<bool>] [-missing_voltage_inst_connection  
<bool>] [-mismatched_voltage_inst_params <bool>]  
[-mismatched_voltage_inst_master <bool>] [-misplaced_voltage_inst  
<bool>] [-net_has_voltage_inst_conflicts <bool>]  
[-connected_shape_layer_conflicts <bool>]
```

Arguments

`context`

Context object of the create physical voltage annotations operation.

`-scope`

Check text label annotation scope. ({ Design | Nets | Viewport }, default: Design)

`-nets`

oaNet or net name list for Nets scope.

`-depth`

Checks text label annotation depth. ({ 0x | Full | View }, default is: 0x)

`-nets_with_voltage_value`
Checks nets with voltage value only. (Default is: false)

`-only_voltage_lpp`
Checks voltage LPP only. (Default is: false)

`-check_udv`
Checks udv instances only. (Default is: true)

`-check_vsync`
Checks vsync instances only. (Default is: true)

`-check_vflag`
Checks vflag instances only. (Default is: true)

`-missing_net_annotation`
Specifies whether to report missing net annotations. (Default is: true)

`-missing_label`
Specifies whether to report missing label violations. (Default is: true)

`-multiple_labels`
Specifies whether to report multiple labels. (Default is: true)

`-incorrect_label_position`
Specifies whether to report incorrect label positions. (Default is: true)

`-incorrect_label_value`
Specifies whether to report incorrect label values. (Default is: true)

`-incorrect_label_lpp`
Specifies whether to report incorrect label LPPs. (Default is: true)

`-missing_voltage_inst`
Specifies whether to report missing voltage instances. (Default is: true)

`-extra_voltage_inst`
Specifies whether to report extra voltage instances. (Default is: true)

`-mismatched_voltage_inst_connection`
Specifies whether to report mismatched voltage instance connections. (Default is: true)

`-missing_voltage_inst_connection`

Specifies whether to report missing voltage instance connections. (Default is: true)

`-mismatched_voltage_inst_params`

Specifies whether to report mismatched voltage instance parameters. (Default is: true)

`-mismatched_voltage_inst_master`

Specifies whether to report mismatched voltage instance masters. (Default is: true)

`-misplaced_voltage_inst`

Specifies whether to report misplaced voltage instances. (Default is: true)

`-net_has_voltage_inst_conflicts`

Specifies whether to report nets with voltage instance conflicts. (Default is: true)

`-connected_shape_layer_conflicts`

Specifies whether to report connected shape layer conflicts. (Default is: true)

Returns

{<oaFig>} : Collection of Markers

le::chop

Description

Removes data cut by the given region or splits objects into pieces. Optionally, the removed data can be moved to the clipboard for further copy from clipboard operations.

Syntax

```
le::chop <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}>
[-design <oaDesign>] [-points [<point>] [-regionType <enum>] [-levels
<int>] [-anchor <point>] [-snapToGrid <enum>] [-copy <bool>] [-keep
<bool>] [-chopLayers [<lpp | string>]] [-selectResult <bool>]
```

Syntax

```
le::chop <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}>
[-design <oaDesign>] [-points [<point>] [-regionType <enum>] [-levels
<int>] [-anchor <point>] [-snapToGrid <enum>] [-copy <bool>] [-keep
<bool>] [-keepPath <bool>] [-chopLayers [<lpp | string>]] [-selectResult
<bool>]
```

Returns

<none>

Arguments

`objects`

List of objects to chop.

`-design`

Design to chop objects in

`-points`

A list of points defining the contour of shape that specifies the chop region.

`-regionType`

Type of region passed to the `-points` argument. ({ polygon | polyline | rectangle }, default is: rectangle)

`-levels`

Number of hierarchical levels beyond which to ignore instances which overlap the chop region. (Default is: 0)

`-anchor`

Anchor point for paste.

`-snapToGrid`

Specify if yanked results will be snapped to manufacturing grid. ({ abort | off | on }, default is: on)

`-copy`

Whether the chopped objects will be copied to clipboard. (Default is: true)

`-keep`

Whether chopped objects will be kept in the design. (Default is: false)

`-keepPath`

Whether to chop Paths and PathSegs based on the intersection point of the centerline and keep Paths and PathSegs in their object type. (Default is: false)

`-chopLayers`

List containing lpp for chop layers or { default | all }

`-selectResult`

Whether add the result to the selection set. (Default is: false)

Errors

- Point list is empty.
- The point list includes more than 2 points, and `-regionType` is not specified as a polygon or polyline.
- Rectangle point list does not contain exactly 2 points.
- Polygon point list does not represent a simple polygon.
- There are a duplicate (coincident) points in the point list.
- Region type is not rectangle, polygon, or polyline.
- Number of levels is negative.
- The design is not specified, is not open, is not valid, or is read-only.

Note:

Two coincident points in the cut list generate an empty chop.

Examples

In the following example, the area specified by the point list is chopped from the selection and copied to the clipboard.

```
le::chop [de::getSelected -design $oaDesign] -points [list [lindex  
[db::getAttr bBox -of [db::getNext [db::getShapes -of $oaDesign]]]  
0] [lindex [db::getAttr bBox -of [db::getNext [db::getShapes -of  
$oaDesign]]] 1]] -copy true
```

le::clearFlightlines

Description

Removes (or hides) static flight lines for the collection of nets in all windows for which their design is the current edit design.

To obtain a collection of nets, use the `db::getNets` command. To obtain net groups use the `le::getNetGroups` command.

If pin connection flight lines are currently being displayed, executing this command updates the set of standard flight lines that will be displayed when the pin connection flight lines are cleared. If no flight line exists then one will be created for each `oaNet` passed (or all `oaNet` in `oaGroup` passed.)

This command shows a warning when the Connectivity Engine is disabled and the immediately return without performing any other action.

Syntax

```
le::clearFlightlines {<oaNet | oaGroup>}
```

Arguments

<oaGroup>

Collection of group objects to remove flight line display. Sets the `flightlinesEnabled` attribute to false.

<oaNet>

Collection of net objects to remove flight line display. Sets the `flightlinesEnabled` attribute to false.

Errors

- The net is invalid.
- The design of the net is not a layout design.
- A group contains more than one `oaNet` or `oaGroup` object.

Examples

The following example clears all flight lines for net A.

```
le::clearFlightlines [db::getNets A -of $oaDesign]
```

le::clearPinConnections

Description

Clears pin connection flight lines in all windows for which their design is the current edit design. When all pin connection flight lines are cleared, the display of standard static flight lines is automatically reactivated using the previous set of enabled nets. If the pin connection flight lines of a pin are not displayed, it is silently ignored.

To obtain a collection of pin, use the [db::getPins](#) command.

Syntax

```
le::clearPinConnections {<oaPin>}
```

Arguments

<oaPin>

Collection of pins objects to remove pin connection flight line display.

Errors

- The pin is invalid.
- The design of the pin is not a layout design.

Examples

The following example clears all relationship flight lines between pins with A net name.

```
le::clearPinConnections [db::getPins A -of $oaDesign]
```

le::clearVoltagePhysicalAnnotations

Description

Clears the voltage physical annotation information in the current layout.

Syntax

```
le::clearVoltagePhysicalAnnotations [<context: deContext>]
```

Arguments

context

The context to be cleared.

le::clearVoltages

Description

Clears the voltage information on nets in the current layout.

Syntax

```
le::clearVoltages [<context: deContext>]
```

Arguments

context

The context to be cleared.

le::cloneMaster

Description

Clones master.

Syntax

```
le::cloneMaster <object: deFigure | oaInst> -cellName <string>  
[-overwrite <bool>]
```

Returns

{<oaInst>} : The remastered instance.

Arguments

`object`

Instance to clone master.

`-cellName`

Cell name for the new cloned master.

`-overwrite`

Specifies whether to overwrite in case. (Default is: false)

le::closeICVLManager

Description

Closes the ICVL manager.

Syntax

```
le::closeICVLManager
```

Returns

<bool> : icvl_manager be closed or not.

le::closeStarRC

Description

Stops the StarRC process and releases the license.

Syntax

```
le::closeStarRC
```

Returns

<none>

le::compactBus

Description

Reduces the spacing between specified objects in the compact mode for the given list of path/pathseg objects. The compact setting is applied only to the selected bits.

The `fanInOut` and `ladder` options specify additional three points to form orthogonal polyline for providing direction.

Syntax

```
le::compactBus <objects: dbCollection | {<deFigure>} | {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}> [-spacing <float> | -pitch <float>] -mode <enum> [-leadingBit <deFigure | oaFig>] [-refPoints [<point>]] [-fanInOutAlignment <enum>] [-fanInOutJogSpacing Mode <enum>] [-fanInOutJogSpacingValue <float>] [-ladderAlignment <enum>] [-ladderAlignmentOffset <float>] [-ladderJogCount <int>]
```

Arguments

`objects`

path, pathsegs to be Compress/Expand.

`-mode`

Specifies operation mode to Compress/Expand bus. (`{compact | fanInOut | ladder}`).

`-leadingBit`

Specifies a leading bit to do alignment for the compact mode.

`-spacing`

Specifies a spacing value between bits of bus in the compact mode, `fanInOut` and `ladder` mode. The default is 0.

`-pitch <float>`

Specifies a pitch value between bits of bus in the compact mode. The default is 0.

`-refPoints`

Three points orthogonal polyline point list to apply in the `fanInOut` and `ladder` mode.

`-fanInOutAlignment`

Specifies an align mode which is used in the `fanInOut` mode. ({`center` | `centerWire` | `side`}, **default:** `center`)

`-fanInOutJogSpacingMode`

Specifies a jog spacing mode which is used in the `fanInOut` mode. ({`asSource` | `asTarget` | `specify`}, **default:** `Target`).

`-fanInOutJogSpacingValue`

Specifies a jog spacing value when the `fanInOut` spacing mode is `specify`.
The default is 0.

`-ladderAlignment`

Specifies an align mode which is used in `ladder` mode. ({`centerline` | `edge` | `offset`}, **default:** `centerline`)

`-ladderAlignmentOffset <float>]`

Specifies offset between reference line and wire when ladder alignment mode is `offset`. The default is 0.

`-ladderJogCount <int>`

Specifies an amount of maximum jogs in ladder mode. The default is 0, indicating maximum.

Returns

`<none>`

le::connectICNet

Description

Creates a new net in IC Compiler designs. This command connects a net to a set of instance terminals in an IC Compiler design. The instance terminals must be associated with an instance not from the original netlist.

Syntax

```
le::connectICNet [<Net: oaNet>] -to <{<oaInstTerm>}>
```

Arguments

`Net`

Net object to connect, and is disconnected if not given specified objects.

-to

Collection of instance terminals to assign to the given net.

Errors

- Specified net or instTerm is not from the same dmMilkyway design.
- Instance associated with instTerm is from original netlist.

le::convertToAliasLayer

Converts the shapes or standard vias with source LPP to alias LPP for a given design or all designs of the given library.

Syntax

```
le::convertToAliasLayer (-design <oaDesign> | -libName <string>)
```

Returns

<bool> : return true if conversion is successfully completed.

Arguments

-design

design object

-libName

library name

le::convertToArray

Description

Converts selected scalar instances to array instances in the design.

Syntax

```
le::convertToArray <objects: {<deFigure>} | {<deSelectionSet>}  
| {<oaInst>}> -mode <enConvertToArrayMode> [-style  
<enConvertToArrayStyle>] [-matchAllProperties <bool>]
```

Returns

{<oaInst>} : Collection of instances after converting to array instances.

Arguments

`objects`

List of instances to be converted to arrays.

`-mode`

Specify a mode, either `singleArray` or `multipleArrays`. This indicates whether converting to a single array or multiple ones. ({ `multipleArrays` | `singleArray` })

`-style`

Specify a style, either `rowFirst` (default), `columnFirst`, or `auto`. This works only when `multipleArrays` mode is selected. ({ `auto` | `columnFirst` | `rowFirst` }, default is: `rowFirst`)

`-matchAllProperties`

Indicate whether all selected instances should have the same properties. (Default is: `true`)

le::convertToPath

Converts specified polygons or rectangles to paths

Syntax

```
le::convertToPath <objects: {<deFigure>} | {<oaFig>}> [-ignore <bool>]  
[-mode <enum>] [-result <enum>]
```

Returns

{<oaShape>} : Collection of converted paths

Arguments

`objects`

The list of objects that should be converted.

`-ignore`

Ignore if inconvertible to user's specified type but convertible to another type. (Default is: `true`)

`-mode`

Determine which mode to convert. ({ `auto` | `manual` }, default is: `auto`)

`-result`

Determine the object type to which the input shape is converted. ({ path | pathseg }, default is: path)

le::convertToPolygon

Description

Converts a list of objects to polygon objects. Ellipses are converted based on the `-numPoints` value.

Syntax

```
le::convertToPolygon <objects: {<defigure>} | {<deSelectionSet>} |  
{<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |  
{<oaVia>}}> [-numPoints <int>]
```

Returns

{<oaFig>}: Collection of converted polygon shapes.

Arguments

`objects`

List of objects to be converted to polygons.

`-numPoints`

Number of vertices to convert an ellipse into a polygon.

Errors

- Empty list of objects is provided.
- Non convertible object is detected.
- Ellipse is provided, but numPoints is less than or equal to 3.

Examples

The following example converts all the selected objects in the design to polygons.

```
le::convertToPolygon [de::getSelected -design $oaDesign]
```

le::copy

Description

Copies fully selected objects from a source point to a destination point. During the transformation, you can rotate and/or mirror objects around the source point.

Syntax

```
le::copy <objects: {<defigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
-anchor <point> [-dx <float>] [-dy <float>] [-rows <int>] [-dxRow  
<float>] [-dyRow <float>] [-cols <int>] [-dxCol <float>] [-dyCol  
<float>] [-rotate <enum>] [-dest <oaDesign>] [-group <bool>] [-keepSync  
<bool>] [-keepInSyncGroup <bool>] [-keepNet <bool>] [-keepColor <bool>]  
[-keepHighlight <bool>] [-keepMember <bool>] [-byPitch <bool>] [-lpp  
<string>]
```

Arguments

`objects`

List of objects to copy.

`-anchor`

Specifies the anchor point.

`-dx`

X projection of copy vector. (Default is: 0)

`-dy`

Y projection of copy vector. (Default is: 0)

`-rows`

Number of rows. (Default is: 1)

`-dxRow`

X projection of distance between rows. This value is ignored if rows=1. (Default is: 0)

`-dyRow`

Y projection of distance between rows. This value is ignored if rows=1. (Default is: 0)

`-cols`

Number of columns. (Default is: 1)

`-dxCol`

X projection of distance between columns. This value is ignored if cols=1. (Default is: 0)

`-dyCol`

Y projection of distance between columns. This value is ignored if cols=1. (Default is: 0)

`-rotate`

Rotation to apply on copied objects. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }, default is: R0)

`-dest`

Design to copy objects into.

`-group`

Specifies if the objects are added to a figure group (Default is: false)

`-keepSync`

Keeps the objects to copy and their resulting copy synchronized. (Default is: false)

`-keepInSyncGroup`

Keeps the resulting groups synchronized if the groups to copy are synchronous groups. (Default is: false)

`-keepNet`

Keeps net names in the resulting copy. (Default is: false)

`-keepColor`

Keeps the color of objects. (Default is: false)

`-keepHighlight`

Keeps the highlight of objects. (Default is: false)

`-keepMember`

Keeps the membership of objects. (Default is: false)

`-byPitch`

Places the result by pitch or spacing. (Default is: true)

`-lpp`

Specifies the LPP of the resulting copy. (Default is:)

Returns

{<deFigure> | <oaFigure> : Collection of copied objects.

Errors

- Empty list of objects is provided.
- Not all provided objects from the same design context.

- Snap/PR boundary is in the list of objects and destination cell view already contains another Snap/PR boundary.
- Negative or zero value is passed as rows/cols parameter.
- Negative value is passed as dxrow/dxcol/dyrow/dycol parameter.

Examples

The following example copies instance X0, creates an array copied instances, specifies the spacing between the instances, and rotates the instances.

```
le::copy [db::getNext [db::getInsts X0 -of $oaDesign]] -anchor {0 0} -dx  
10 -dy 10 -cols 2 -rows 2 -dxCol 1 -dxRow 1 -dyCol 1 -dyRow -1 -rotate  
MXR90
```

le::copyHighlighted

Description

Copies highlighted to cellview.

Syntax

```
le::copyHighlighted [<name: string>] -source <oaDesign> -destination  
<dmCellView | dmCellViewName | oaDesign> [-overwrite <bool>] [-copyNet  
<bool>] [-copyColor <bool>] [-packet <string>]
```

Arguments

name

Name of the highlight set.

-source

Source design to search for highlight sets containing figures.

-destination

Destination design or cellview to create or append.

-overwrite

Overwrites or appends to the existing cellview. Default is true.

-copyNet

Copy connectivity (flattened). Default is false.

-copyColor

Copy MPT color. Default is false.

`-packet`

Packet defines the drawing style.

Returns

<none>

Examples

The following example copies highlighted objects from oaDesign into designLib/dut/layout cellView.

```
set design [ed]
# Get all shapes on m1 drawing
set shapes [db::getShapes -of $design -lpp {m1 drawing}]
# Highlight found shapes
de::highlight $shapes
le::copyHighlighted -source $design -destination designLib/dut/layout
```

le::copyLook

Description

Copies fully selected parameterized cell (PCell) shapes and vias.

Syntax

```
le::copyLook -objects <{<defigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>
```

Returns

List of copied objects and vias.

Arguments

`objects`

List of objects to copy.

Errors

Returned objects are from different cell views.

le::copyTrackRegionDef

Creates an leTrackRegionDef in technology or design database by referencing to a given leTrackRegionDef object. Note that the command also copies leTrackGroupDef and leTrack objects, those are referenced by the given leTrackRegionDef, to the target database when they can not be found in the target database.

Syntax

```
le::copyTrackRegionDef [<name: string>] -to <oaDesign | oaTech>  
-reference <leTrackRegionDef> [-overwrite <enum>]
```

Returns

<leTrackRegionDef> : The newly copied leTrackRegionDef object

Arguments

name

Name of the trackRegionDef: If not specified the reference object name is taken.
A trackRegionDef with given name shouldn't exist (Default is:)

name

Name of the trackRegionDef: If not specified the reference object name is taken.
(Default is:)

-to

Design or Tech in which to copy the trackRegionDef

-reference

Reference trackRegionDef object from which to copy

-overwrite

Determines whether or not to overwrite object if an object with the same name
but different settings already exists in target database ({ full | none | rename |
skip }, default is: none)

le::correctOffgrid

Description

Corrects Off-grid objects.

Syntax

```
le::correctOffgrid <objects: {<deFigure>} | {<deSelectionSet>} |  
{<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaShape>} | {<oaVia>}}>  
[-grid <enum>]
```

Arguments

objects

List of objects to correct.

`-grid`

Grid to correct to. ({ manufacturing | snapping })

Returns

<none>

le::createAttributeLabel

Description

Creates an attribute label in a design.

Command execution produces a group containing two objects: the new label object and the specified parent object, where the specified parent object is the leader.

This command supports the following predefined attribute names:

- netName
- termName
- instName
- cellName
- libName
- viewName

Syntax

```
le::createAttributeLabel <attribute name: netName, termName, instName,  
cellName, libName, viewName> -parent <defigure | oaFig> [-valueOnly  
<bool>] -lpp <lpp> -origin <point> [-just <enum>] [-orient <enum>]  
[-font <enum>] [-height <float>] [-drafting <bool>] [-overbar <bool>]
```

Returns

<oaShape> : New label object (oaAttrDisplay or oaPropDisplay).

Arguments

<attribute name>

Text of the label.

-parent

Design to contain the label, or parent to which to attach label.

`-valueOnly`

Displays the value only, otherwise displays both the name and the value.

Default: false.

`-lpp`

Layer purpose pair associated with label.

`-origin`

Text origin location.

`-just`

Label justification related to origin.

Default: centerCenter.

`-orient`

Label orientation.

Default: R0.

`-font`

Text font.

Default: stick.

`-height`

Text height in user units.

Default: 1.

`-drafting`

Prevents text mirroring or rotating.

Default: true.

`-overbar`

Renders text with overbar.

Default: false.

Errors

- An invalid layer and purpose pair is provided.
- An invalid point is provided as an origin.

- Negative value for height is specified.
- Parent object does not have attribute or property with specified name.

Examples

The following example creates an attribute label for the selected object.

```
le::createAttributeLabel termName -valueOnly true -lpp {m1 net} -origin  
{0 0} -orient R0 -height 0.5 -overbar false -parent [db::getAttr object  
-of [db::getNext [de::getSelected -design $oaDesign]]]
```

le::createAbutGroups

Description

Creates abutment group in the given design.

Syntax

```
le::createAbutGroups <design: oaDesign> -abutter <string> -abuttee  
<string> -termA <string> -termB <string> -paramsA [<string>] -paramsB  
[<string>]
```

Arguments

design

Design to generate abutment groups.

-abutter

Abutter Instance.

-abuttee

Abuttee Instance.

-termA

Term A.

-termB

Term B.

-paramsA

Param value list that change with abutting term A.

-paramsB

Param value list that change with abutting term B.

le::createBlockage

Description

Creates a blockage of a given type (placement, routing, pin, wiring, fill, slot, screen, or feedthru) in a given design. You define the geometry blockage in a point list. If the point list contains 2 points, the command creates a rectangular blockage based on the provided points' bounding box.

Syntax

```
le::createBlockage <points: [point]> -design <oaDesign> [-parent  
  <oaInst>] -type <enum> [-layer <string>] [-hasWidth <bool>] [-width  
  <float>]
```

Returns

Newly created blockage object (*oaBlockage*).

Arguments

{<point>}

List of points that forms the blockage boundary.

-design *oaDesign*

Design where the blockage is created.

-parent *oaInst*

Parent oalnst object for blockage.

-type

Blockage type ({ feedthru | fill | pin | placement | routing | screen | slot | wiring })

-layer <int | string>

Layer name or number. Ignored for the placement type.

Default: Empty string.

-width <float>

Effective width of the blockage around the boundary of the blockage object. If empty, then the spacing rules associated with the layer apply.

Default: Empty string.

-hasWidth <bool>

Assigns an effective width to the blockage.

Default: false.

Errors

- An invalid cell view is provided.
- Number of points is less than 2.
- Non placement type is provided, but layer does not specified.

Examples

The following example creates a rectangle blockage object of the size specified by the points.

```
le::createBlockage {{-1 1} {1 -1}} -type placement -design $oaDesign
```

le::createBoundary

Description

Creates a new boundary of the given type (either PR, Snap, or Wide Alignment) for a given design using the supplied list of points that define the boundary shape.

The list of points (first argument) forms the boundary's geometry. If the list contains 2 points, the command creates a rectangular boundary based on the provided points' bounding box.

Syntax

```
le::createBoundary <points: [point]> -design <oaDesign> -type <enum>  
[-force <bool>] [-boundaryLPP <lpp>] [-snapPoints <bool>] [-endShapes  
<bool>]
```

Returns

<oaFig> : Newly created boundary object.

Arguments

points

The list of points that form boundary.

-design

Design in which to create the boundary.

-type

Sets the boundary type by pr, snap, or wideAlignment.

`-force`

Specifies if the old boundary should be replaced with the new one. The default is *false*.

`-boundaryLPP`

Specifies the boundary LPP for wide alignment boundary.

`-snapPoints`

Specifies if snap points according to the wide alignment constraint when creating wide alignment boundary. The default is *true*.

`-endShapes`

Specify the end shapes for wide alignment boundary. The default is *false*.

Errors

- An invalid `oaDesign` object is provided.
- Number of points is less than 2.
- Replace option is turned off but boundary already exist.

Examples

The following example creates a snap type rectangle boundary of the size specified by the points.

```
le::createBoundary {{-1 1} {1 -1}} -design $oaDesign -type snap -force  
on
```

le::createCellViewFromSelection

Description

Creates an `oaCellView` and an `oaDesign` in that `cellView` from the provided collection of objects. Returns a newly created `oaDesign` object. Depending on the `-replace` option value, the command either replaces objects by instance of newly created design or leaves them untouched.

Syntax

```
le::createCellViewFromSelection [-viewName <string>] -objects  
<{<defigure>} | {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} |  
{<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}} [-libName <string>]  
-cellName <string> [-instName <string>] [-replace <bool>] -origin  
<point> [-status <enum>] [-force <bool>]
```

Returns

<oaDesign> : Newly created CellView object.

Arguments

-viewName

Specifies the new view name.

Default: layout.

-objects

List of objects that should be go to the newly created cell view.

-libName

Library name were new cell view should be created.

Default: Library of the current cellView.

-cellName

Cell name for which new view should be created.

-instName

Instance name (in case if objects should be replaced by instance).

Default: Prefix from master's meta data or 'I' concatenated with integer.

-replace

Specifies if objects should be replaced by newly created instance.

Default: false.

-origin

Location of the origin of the cellView.

-status

Placement status for cellView. The available values are fixed, locked, none, placed, and unplaced.

Default: placed.

-force

Specifies if need to overwrite in case if the cell view exists.

Default: true.

Errors

- Provided objects are from different oaDesigns.
- Illegal or non existing library name.
- Illegal cell name.
- Overwrite option is turned off but cell view exists with the same name.
- Provided instName is not valid or already exist.
- oaGroup is passed that does not correspond to Guard Ring, Ruler, or Coordinate Mark.

Examples

From the selected objects, the following example creates a new cell (TestCell) in library Lib with a view of layout.

```
le::createCellViewFromSelection -cellName TestCell -objects  
[de::getFigures {{8.5 -4} {8.5 2} {11 2} {11 5} {-5 5} {-5 -4} } -design  
$oaDesign -type polygon] -origin {0 0} -libName Lib -viewName layout
```

le::createCoordinateMark

Description

Creates a coordinate mark at the specified coordinate.

Syntax

```
le::createCoordinateMark <point: point> -design <oaDesign> [-note  
<string>] [-persistent <bool>]
```

Returns

<leCoordinateMark>: Mark created.

Arguments

point

Point to be marked.

-design

Design in which to create the mark.

-note

Text note on the mark.

`-persistent`

Keeps coordinate marks in the design. (Default is: true)

Errors

- `<point>` is not specified.
- Design is not specified.
- Design is invalid, not open or does not have write access.
- The design type is not supported by this command.

Examples

The following example creates a coordinate mark in a design.

```
le::createCoordinateMark -design $oaDesign
```

le::createCorridorArea

Description

Creates a Corridor Area object for use with a routing corridor.

Syntax

```
le::createCorridorArea [<name: string>] (-rect <box> | -points [<point>])  
-routeCorridor <leRouteCorridor> [-pathWidth <float>] [-startEndStyle  
<enum>] [-stopEndStyle <enum>] -minLayer <string> -maxLayer <string>
```

Returns

`<leCorridorArea>` : Corridor Area Object.

Arguments

`name`

Optional name for the corridor area.

`-routeCorridor`

Route corridor to which area will be added.

`-rect`

Rectangle defining corridor area.

`-points`

List of points defining either a boundary or path centerline of corridor area.

`-pathWidth`

if given, define path width to be used with points (Default is: 0)

`-startEndStyle`

End style for first point of path ({ doubleExtend | extend | truncate }, default is: truncate)

`-stopEndStyle`

End style for last point of path ({ doubleExtend | extend | truncate }, default is: truncate)

`-minLayer`

Layer name or number for lower most layer for use in area.

`-maxLayer`

Layer name or number for upper most layer for use in area.

Errors

- Box is inverted.
- minLayer or maxLayer name or number does not exist.
- minLayer or maxLayer is not a metal layer.
- Corridor area with the same name already exists.

Examples

```
set net [db::getNets CLK -of $design]
set routeCorridor [le::createRouteCorridor routeCorridoe1 -design $design
-nets $net]

le::createCorridorArea corridorArea1\
-maxLayer m5 \
-minLayer m1 \
-routeCorridor $routeCorridor \
-rect {{0.0 0.0} {5.0 5.0}}
```

le::createDRDLineEdit

Description

Creates a line edit that has clear text capability and adds it to the specified parent

Syntax

```
le::createDRDLineEdit [<{none}: string>] [-parent <giAssistant |
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]
```

```
[-prefName <dbPrefName>] [-toolTip <string>] [-valueChangeProc  
[<variant>]] [-label <string>] [-value <string>]
```

Returns

<drdLineEdit> : New object.

Arguments

{none}

Widget name.

-parent

Dialog, group or other container into which this widget should be placed.

-enabled

Initial enabled state. The default is true.

-shown

Initial shown state. The default is true.

-prefName

Optional preference name within top-level container's scope which provides the initial value and is automatically updated with value changes according to the container update model.

-toolTip

Optional widget tooltip.

-valueChangeProc

Procedure to execute when the value is changed. Called with the widget object

-label

Label of this line edit widget

-value

initial value.

le::createDRDTreeWidget

Creates a new drdTreeWidget widget and adds it to the specified parent

Syntax

```
le::createDRDTreeWidget [<{none}>: string] [-parent <giAssistant |  
giContainer | giDialog | giWindow>] [-enabled <bool>] [-shown <bool>]  
[-toolTip <string>] [-contextMenuProc [<variant>]] [-selectionChangeProc
```

```
[<variant>]] [-doubleClickChangeProc [<variant>]] [-valueChangeProc  
[<variant>]] [-flat <bool>] [-hide <bool>] [-hint <string>] [-keywords  
<string>]
```

Returns

<drdTreeWidget> : New object.

Arguments

{none}

Widget name. (Default is:)

-parent

Dialog, group or other container into which this widget should be placed.

-enabled

Initial enabled state. (Default is: true)

-shown

Initial shown state. (Default is: true)

-toolTip

Optional widget toolTip. (Default is:)

-contextMenuProc

Procedure to execute when right-clicking on this widget to request a context menu

-selectionChangeProc

Procedure to invoke when selection changed

-doubleClickChangeProc

Procedure to invoke when double click item value changed

-valueChangeProc

Procedure to invoke when value changed

-flat

the tree is flat or not (Default is: false)

-hide

the tree is invisible or not (Default is: false)

-hint

the hint string for tree header

`-keywords`

the keywords string for tree item (Default is: * :)

le::createEllipse

Description

Creates an ellipse in a design using 32 data points. You must provide an LPP, center point, and horizontal and vertical radii (width and height of bounding box).

Syntax

```
le::createEllipse <box> -design <oaDesign> -lpp <lpp> [-net <string  
| oaNet>]
```

Returns

New ellipse (*oaEllipse*).

Arguments

`<box>`

Ellipse bounding box.

`-design`

Design context where the ellipse is created.

`-lpp`

The layer purpose pair.

Default: drawing.

`-net`

Net name or oaNet object to which the new ellipse is added.

Default: <none>.

Errors

- An invalid oaDesign is passed.
- A zero or negative value is specified for height/width.
- An invalid lpp is provided.
- A net name is provided which does not adhere to CDB scalar net namespace. Bus / bundle format names are not permitted in layout.

Examples

In the following example, the first line defines a bounding box; the second line creates an ellipse, using that bounding box, on the defined LPP with the defined net name.

```
set bbox [list [list 0.0 0.0] [list 0.02 0.02]]
le::createEllipse $bbox -design $oaDesign -lpp {m1 drawing} -net e1
```

le::createGlobalTrackRegion

Description

Creates a Global Track Region object in a design.

Syntax

```
le::createGlobalTrackRegion [<name: string>] -design <oaDesign>
    -regionDef <leTrackRegionDef> [-flipX <bool>] [-flipY <bool>]
    [-flipFirstX <bool>] [-flipFirstY <bool>] [-tileOffset <point>]
```

Returns

<leGlobalTrackRegion> : The newly created leGlobalTrackRegion object.

Syntax

```
le::createGlobalTrackRegion [<name: string>] -design <oaDesign>
    -regionDef <leTrackRegionDef> [-flipX <bool>] [-flipY <bool>]
    [-flipFirstX <bool>] [-flipFirstY <bool>] [-tileOffset <point>] [-orient
    <enum>]
```

Returns

<leGlobalTrackRegion> : The newly created leGlobalTrackRegion object

Arguments

name

Name of the global region object (Default is:)

-design

Design in which to create the global region

-regionDef

leTrackRegionDef object used in the global region

-flipX

Determines if tiles in the global region are flipped about the X-axis (Default is: false)

`-flipY`

Determines if tiles in the global region are flipped about the Y-axis (Default is: false)

`-flipFirstX`

Determines if the first tile in the global region is flipped about the X-axis (Default is: false)

`-flipFirstY`

Determines if the first tile in the global region is flipped about the Y-axis (Default is: false)

`-tileOffset`

Offset of the first tile in the global region

`-orient`

The orientation of the created global track region ({ MX | MY | R0 | R180 }, default is: R0)

Examples

```
set trackRegion [le::createTrack track -in $design \  
-offset 0 -trackType default]  
set trackGroupDefinition [le::createTrackGroupDef \  
trackGroupDefinition -in $design \  
-direction horizontal -layer m1 -pitch {10 100} \  
-tracks $ trackRegion]  
set trackRegionDefinition [le::createTrackRegionDef \  
trackRegionDefinition -in $design \  
-groupDefs $ trackGroupDefinition]  
  
le::createGlobalTrackRegion globalTrackRegion \  
-design [ed] \  
-regionDef $ trackRegionDefinition
```

le::createGuardRing

Description

Creates a guard ring.

Syntax

```
le::createGuardRing <points: [point]> -design <oaDesign> -master  
<dmCellView | dmCellViewName | oaDesign> [-params [<map>]]  
[-evalCallbacks <bool>] [-net <oaNet | string>]
```


Arguments

`points`

The list of points that form the guard ring.

`-design`

Design context where the guard ring is created.

`-master`

Super master of the guard ring.

`-params`

Parameters of MCell or editable PCell instance. Each item should have this format: {parameter name, parameter value}.

`-evalCallbacks`

Specifies whether to trigger callbacks. (Default: true.)

`-net`

The net name or oaNet object to which guard ring should be assigned.

Returns

<oaInst>: The newly created guard ring instance.

Examples

The following example creates a guard ring for the provided master design.

```
set guardRingCellView [dm::getCellViews -libName ref \
-cellName nwellGR]

le::createGuardRing {{0.0 0.0} {5.0 5.0}} \
-design $design \
-master $guardRingCellView
```

The following example creates a guard ring with parameters for the provided master design.

```
le::createGuardRing {{0.0 0.0} {5.0 5.0}} \
-design [ed] \
-master $guardRingCellView \
-params {{ CutRow 2}}
```

le::createICInst

Description

Creates a new cell instance in the IC Compiler design.

This command creates an instance in a IC Compiler design. If module hierarchy is included in the name, the bar character must be used as the hierarchy delimiter. A leading bar character is allowed and silently removed by the command when creating the instance. The instance is eventually exported to a Milkyway and attached to the existing hierarchy preservation structures. The origin of the instance is set to place the instance just outside the lower right of the PRBoundary object or cell extent. The placement status is set to *none* and transform to *R0*.

Instance terminal objects are created for all oaTerms in the master design with no net being assigned to the oalnstTerm objects. You need to use the `le::connectICNet` command to have the oalnstTerms connected to an appropriate net.

Syntax

```
le::createICCIInst <name: string> -design <oaDesign> [-master <oaDesign>]  
[-libName <string>] [-cellName <string>] [-viewName <string>]
```

Returns

<oaInst> : Newly created object.

Arguments

name

Instance name including optional module hierarchy.

-design

The design where the instance should be created.

-master

The master of the instance to be created.

-libName

The name of the master cell library.

-cellName

The name of the master cell.

-viewName

The view name of the master cell.

Default: *abstract*.

Errors

- Specified design is not from a dmMilkyway library.
- viewName is not *layout* or *abstract*.

- cellName or master must be given.
- libName and viewName must not be given if master is given.
- libName/cellName/viewName does not exist.
- Instance name references non-existent module hierarchy.

le::createICNet

Description

Creates a new net in the IC Compiler design.

This command creates a net in a IC Compiler design. If module hierarchy is included in the name, the bar character must be used as the hierarchy delimiter. A leading bar character is allowed and silently removed by the command when creating the net. The net is exported to the IC Compiler design and inserted into the hierarchy preservation structures.

Syntax

```
le::createICNet <name: string> -design <oaDesign> [-sigType <oaSigType>]
```

Returns

<oaNet> : Newly created object.

Arguments

name

Tcl list of net names including optional module hierarchy.

-design

The design where the net should be created.

-sigType

Signal type for the new net.

Errors

- Specified design is not from a dmMilkyway library.
- Net name references non-existent module hierarchy.

le::createInst

Description

Creates an instance or an instance array object with a given name in a given design. An instance array is created when you provide step values greater than 1. If both step values are equal to 1, and name is `oaVectorBitName`, then `oaVectorInstBit` is created. If `oaScalarName` is passed then `oaScalarInst` is created. If no name is passed then the default name will be `oaScalarName`. Note that `oaScalarInst` is derived from `oaBitInst`, which derives from `oaInst`. Otherwise `oaArrayInst` object is created, which also derives from `oaBitInst` and represents an array of instances.

If you provide a list of name/value pairs with the `-params` argument, the command applies those parameters to the instance as either `oaParams` or `oaProps`, depending on whether the master design is a `superMaster` (for example: a `PCell`).

If a given parameter corresponds to a `superMaster oaParam`, the given parameter is created as an `oaParam` and used during instance creation. If the given parameter does not correspond to a `superMaster oaParam` but does correspond to a CDF parameter, the given parameter is created as an `oaProp` on the newly created instance based on the type matching defined in the CDF. If the given parameter name does not correspond to a `superMaster oaParam` or a CDF parameter, the instance is not created.

Syntax

```
le::createInst [<name: string>] [-master <oaDesign>] [-libName <string>]  
  [-cellName <string>] [-viewName <string>] -design <oaDesign> [-orient  
  <enum>] -origin <point> [-rows <int>] [-cols <int>] [-dx <float>] [-dy  
  <float>] [-status <enum>] [-params [<map>]] [-evalCallbacks <bool>]
```

Returns

Newly created instance ([oaInst](#)).

Arguments

name

Name of instance.

Default: Prefix value from master cells meta data. If that value is empty string, then "i" concatenated with an integer, which creates unique name.

-master

Master of the instance.

-libName

Name of the master cell library.

Default: Library of the current cellView.

`-cellName`

The name of a master cell. Should be specified if `-master` not specified.

`-viewName`

The view name of the master cellView.

Default: layout.

`-design`

Design in which to create the instance.

`-orient`

Orientation of the instance. The available values are MX, MXR90, MY, MYR90, R0, R180, R270, and R90.

Default: R0.

`-origin`

Location of the origin of the instance.

`-rows`

Number of rows.

Default: 1.

`-cols`

Number of columns.

Default: 1.

`-dx`

Delta between instances in X direction.

Default: Instance width.

`-dy`

Delta between instances in the Y direction.

Default: Instance height.

`-status`

Placement status for the instance.

Default: placed.

`-params`

Properties of instance (or parameters of PCell instance). Each item should have {property name, property value} format.

Default: Empty list.

`-evalCallbacks`

Specifies if callbacks should be triggered.

Default: true.

Errors

- An invalid cell view is provided.
- An non-existing library, cell, view, or master name is provided.
- An illegal instance name is provided.
- Negative or zero value is passed as row/cols parameter.
- Negative or zero value is passed as dx/dy parameter.

Examples

The following example creates an instance (M1) of the `n_4t` n-channel mosfet device (which is defined in the `reference40nm` library) and specifies the origin.

```
set design [ed]
le::createInst M1 -design $design -libName reference40nm -cellName \
  n_4t -viewName layout -origin {1 1} -params {{1 1u} {w 3u}}
```

The following example creates a vectored instance of the `n_4t` mosfet device (which is defined in the `reference40nm` library) and specifies the origin.

```
le::createInst -design $design -libName reference40nm -cellName n_4t \
  -viewName layout -origin {1 1} -params {{1 0.5u} {w 1.5u} {nf 3}} \
  -rows 3 -cols 4 -dx 1 -dy 2
```

le::createLabel

Description

Creates a label in a design.

To create a static label you should specify the text (positional argument) and either `oaDesign` or `oaFig` as an argument for the `-parent` option. If you specify `oaDesign` as a parent, the static label is created in given design. Otherwise the label is attached to the `oaFig` object.

Syntax

```
le::createLabel <string: string> -parent <defigure | oaDesign | oaFig>  
-lpp <lpp> -origin <point> [-just <enum>] [-orient <enum>] [-font  
<enum>] [-height <float>] [-drafting <bool>] [-overbar <bool>]
```

Returns

The created label object.

Syntax

```
le::createLabel <string: string> [-font <enum> | -polygonFont <enum>]  
-parent <deFigure | oaDesign | oaFig> -lpp <lpp> -origin <point> [-just  
<enum>] [-orient <enum>] [-spacing <float>] [-height <float>] [-drafting  
<bool>] [-overbar <bool>]
```

Returns

<oaText> : New label object (oaText)

Arguments

string

Text of the label

-parent

The design in which the label should be created, or the parent to which the label should be attached.

-lpp

The lpp on which to create the text.

-origin

Location of the origin of text.

-just

Justification of the text with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }, default is: centerCenter)

-orient

Orientation of the text. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }, default is: R0)

-font

Font style with which the text is displayed. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish }, default is: stick)

`-polygonFont`

Font style of the polygon with which the text is displayed. ({ contemplate | merit | simple }, default is: simple)

`-spacing`

Spacing of the text in user units. (Default is: 0)

`-height`

Height of the text in user units. (Default is: 1)

`-drafting`

Indicates whether the text should not be mirrored or rotated. (Default is: true)

`-overbar`

Indicates whether the text will be rendered with overbar. (Default is: false)

Errors

- An invalid design is passed.
- An invalid layer and purpose pair is provided.
- An invalid point is provided as an origin.
- Negative value for height is specified.
- The `-attr` is passed, but `oaDesign` is passed as the `-parent` object.
- The `-attr` value is passed and the value does not match any valid attribute for the `-parent` object.
- Unnamed arguments cannot be passed when `-attr` is passed, while the `-valueOnly` option only makes sense when `-attr` is passed.
- Optional parameter and `-attr` parameter are not specified (or both are specified at the same time).

Examples

The following example creates a text label (Text) on the defined LPP, specifies the origin, and rotates the text label by 90 degrees.

```
le::createLabel Text -lpp {m1 text} -origin {0 0} -orient R90 -parent  
$oaDesign
```

le::createMPP

Description

Creates an leMPP object in a database. During MPP creation, you can specify a list of object properties and parameters based on which MPP is generated.

The -points argument specifies a centerline. The centerline must be Manhattan, for example, orthogonal segments only.

If a centerline's first and last points are equal, a closed (polygonal type) MPP is created. Otherwise, an open MPP is created.

Syntax

```
le::createMPP [<name: string>] -design <oaDesign> -definition <string>  
-points [<point>] [-lines [<point>]] [-properties [<string>]]
```

Returns

oaAppDef object that represents the MPP ([leMPP](#)).

Arguments

name

MPP object name.

Default: empty string.

-design

Design where MPP is created.

-definition

Name of existing MPP definition.

-points

Centerline's points.

-lines

List of lines of the created MPP object, to create a MPP with lines.

-properties

List of properties and related values for an MPP object.

Default: empty list.

Errors

- An invalid design is passed.
- A name for non existing definition is passed.
- Based on the centerline provided, the created MPP has a self-intersection.

Examples

The following example creates an MPP object (mpp1) with a centerline based on the given points.

```
le::createMPP "mpp1" -design $oaDesign -definition "psubGR" -points {{0  
0} {0 10} {10 10} {10 0}}
```

le::createMPPDef

Description

Creates an MPP definition.

Syntax

```
le::createMPPDef <name: string> -in <oaTech> -spacings [<float>] -rings  
[<leMPPRingDef | string | {<oaAppObject>}>] [-masterIndex <int>]  
[-exposedParams [<string>]]
```

Returns

<leMPPDef>: The newly created MPP definition object.

Arguments

name

The name for MPP definition being created.

-in

Technology in which to create the MPP definition.

-rings

The list of rings definition names or rings definition.

-spacings

The spacing list for each ring definition, corresponding to the ring list.

-masterindex

The index indicating the master ring.

Default: 0.

-exposedParams

The list of exposed parameters for the MPP definition.

Examples

```
set tech [oa::getTech $design]
set masterPathDef [le::createMPPMasterPathDef masterPathDef \
  -in $tech -layer m1 -purpose drawing -width 0.07 \
  -chopable true -extension truncate]
set enclosedPathDef1 [le::createMPPEnclosedPathDef \
  nclosedPathDef1 -in $tech -layer diff \
  -purpose drawing -enclosure 0.04]
set enclosedPathDef2 [le::createMPPEnclosedPathDef \
  enclosedPathDef2 -in $tech -layer pplus \
  -purpose drawing -enclosure 0.04]
set rectDef [le::createMPPSubRectDef rectDef -in $tech \
  -layer cont -purpose drawing -height 0.07 -width 0.07]
set ringDef [le::createMPPRingDef ringDef -in $tech \
  -masterPath $masterPathDef \
  -enclosedPaths [ list $enclosedPathDef1 $enclosedPathDef2 ] \
  -subRects [ list $rectDef ]]

le::createMPPDef mppDef -in $tech \
  -rings [ list $ringDef ]
```

le::createMPPEnclosedPathDef

Description

Creates an MPPEnclosedPath definition.

Syntax

```
le::createMPPEnclosedPathDef <name: string> -in <oaTech> -layer <string>
  [-purpose <string>] -enclosure <float> [-endOfLineEnclosure <float>]
  [-beginOffset <float>] [-endOffset <float>] [-connectivity <bool>]
  [-chopable <bool>] [-extension <enum>] [-exposedParams [<string>]]
```

Returns

<leMPPEnclosedPathDef> : Newly created MPPEnclosedPath definition object.

Arguments

name

The name for the MPPEnclosedPath definition to create.

-in

Technology in which to create the MPPEnclosedPath definition.

`-layer`

The layer on which to create each shape.

`-purpose`

The purpose on which to create each shape. The default is drawing.

`-enclosure`

The enclosure over the master path.

`-endOfLineEnclosure`

The endOfLine enclosure over the master path. If not specified, the enclosure value will be used.

`-beginOffset`

The begin offset from the master path.

`-endOffset`

The end offset from the master path.

`-connectivity`

Specifies whether the shape is connected to the ring net. The default is false.

`-chopable`

Specifies the whether the shape is chopable. The default is false.

`-extension`

The begin/end extension style for each path shape. Valid values: { extend | round | truncate | variable }. The default is truncate.

`-exposedParams`

The list of exposed parameters for the MPPEnclosedPath definition.

Examples

The following example creates an MPPEnclosedPath definition in the specified technology.

```
le::createMPPEnclosedPathDef diffPath \  
    -in $tech \  
    -enclosure 0.0 \  
    -layer diff \  
    -chopable false \  
    -exposedParams {name layer chopable enclosure}
```

le::createMPPEnclosedShapeDef

Description

Creates an MPPEnclosedShape definition.

Syntax

```
le::createMPPEnclosedShapeDef <name: string> -in <oaTech> -layer  
  <string> [-purpose <string>] -enclosure <float> [-connectivity <bool>]  
  [-exposedParams [<string>]]
```

Returns

<leMPPEnclosedShapeDef>: Newly created MPPEnclosedShape definition object.

Arguments

name

The name for the MPPEnclosedShape definition being created.

-in

Technology in which to create the MPPEnclosedShape definition.

-layer

The layer on which to create each shape.

-purpose

The purpose on which to create each shape. The default is drawing.

-enclosure

The enclosure over the master path.

-connectivity

Specifies whether the shape is connected to the ring net.

Default: false.

-exposedParams

The list of exposed parameters for the MPPEnclosedShape definition.

Examples

The following example creates an MPPEnclosedShape definition in the specified technology.

```
le::createMPPEnclosedShapeDef nwellShape \  
    -in $tech \  
    -enclosure 0.14 \  
    -purpose drawing
```

```
-layer nwell \  
-exposedParams {name layer enclosure}
```

le::createMPPMasterPathDef

Description

Creates an MPPMasterPath Definition.

Syntax

```
le::createMPPMasterPathDef <name: string> -in <oaDesign | oaTech> [-layer  
    <string>] [-purpose <string>] [-width <float>] [-connectivity <bool>]  
    [-chopable <bool>] [-extension <enum>] [-exposedParams [<string>]]
```

Returns

<oaAppObject>: Newly created MPPMasterPath definition object.

Arguments

name

The name for the MPPMasterPath definition being created.

-in

Technology in which to create the MPPMasterPath definition.

-layer

The layer on which to create each shape.

Default: .

-purpose

The purpose on which to create an MPPMasterPath. The default is drawing.

Default:.

-width

The master path width.

-connectivity

Specifies whether the shape is connected to the ring net.

Default: false.

-chopable

Specifies whether the shape is chopable.

Default: false.

`-extension`

The begin/end extension style for each path shape. Valid values: {extend | round | truncate | variable}. The default is truncate.

`-exposedParams`

List of exposed parameters for the MPPMasterPath definition.

Examples

The following example creates an MPPMasterPath definition in the specified technology.

```
le::createMPPMasterPathDef mppMetallPath \  
    -in $tech \  
    -layer m1 \  
    -chopable true \  
    -connectivity true \  
    -extension truncate \  
    -width 0.07 \  
    -exposedParams {name layer chopable width}
```

le::createMPPOffsetPathDef

Description

Creates an MPPOffsetPath definition.

Syntax

```
le::createMPPOffsetPathDef <name: string> -in <oaTech> -layer <string>  
    [-purpose <string>] -width <float> [-offset <float>] [-beginOffset  
    <float>] [-endOffset <float>] [-connectivity <bool>] [-chopable <bool>]  
    [-extension <enum>] [-exposedParams [<string>]]
```

Returns

<oaAppObject>: Newly created MPPOffsetPath definition object.

Arguments

`name`

The name for the MPPOffsetPath definition being created.

`-in`

Technology in which to create the MPPOffsetPath definition.

`-layer`

The layer on which to create an MPPOffsetPath definition.

Default: .

`-purpose`

The purpose on which to create each shape. The default is drawing.

`-width`

The Offset Path width.

`-offset`

The offset from the master path. The default is 0.

`-beginOffset`

The begin offset from the master path.

`-endOffset`

The end offset from the master path.

`-connectivity`

Specifies whether the shape is connected to the ring net.

Default: false.

`-chopable`

Specifies whether the shape is chopable.

Default: false.

`-extension`

The begin/end extension style for offset path. Valid values: {extend | round | truncate | variable}. The default is truncate.

`-exposedParams`

List of exposed parameters for the MPPOffsetPath definition.

le::createMPPRingDef

Description

Creates an MPPRing definition.

Syntax

```
le::createMPPRingDef <name: string> -in <oaTech> [-netName  
  <string>] -masterPath <leMPPMasterPathDef | string> [-enclosedPaths  
  [<leMPPEnclosedPathDef | string | {<oaAppObject>}>]] [-offsetPaths  
  [<leMPPOffsetPathDef | string | {<oaAppObject>}>]] [-subRects  
  [<leMPPSubRectDef | string | {<oaAppObject>}>]] [-enclosedShapes  
  [<leMPPEnclosedShapeDef | string | {<oaAppObject>}>]] [-exposedParams  
  [<string>]]
```


Returns

<oaAppObject>: Newly created MPPRing definition object.

Arguments

name

The name for the MPPRing definition being created.

-in

Technology in which to create the MPPRing definition.

-netName

The default net name for the MPPRing.

-masterPath

The master path definition name or master path definition.

-enclosedPaths

The list of enclosed path definition names or enclosed path definitions.

-offsetPaths

The list of offset path definition names or offset path definitions.

-subRects

The list of sub rectangle definition names or sub rectangle definitions.

-enclosedShapes

The list of enclosed shapes definition names or enclosed shapes definitions.

-exposedParams

The list of exposed parameters for the MPPRing definition.

Examples

The following is a fully completed example about how to create MPPRing definition.

```
#Creating MPPSubRectDef
le::createMPPSubRectDef subRects \
    -in $tech \
    -chopable true \
    -endOfLine 0.08 \
    -height 0.06 \
    -width 0.06 \
    -layer cont \
    -numRows 100 \
    -minEnclosure 0.03 \
    -spacingType fit \
```

```

        -connectivity true \
        -exposedParams {name layer chopable}

#Creating MPPMasterPathDef
le::createMPPMasterPathDef mppMetallPath \
    -in $tech \
    -layer m1 \
    -chopable true \
    -connectivity true \
    -extension truncate \
    -width 0.07 \
    -exposedParams {name layer chopable width}

# Creating MPPEnclosedPath
le::createMPPEnclosedPathDef diffPath \
    -in $tech \
    -enclosure 0.0 \
    -layer diff \
    -chopable false \
    -exposedParams {name layer chopable enclosure }

# Creating Enclosed Shape
le::createMPPEnclosedShapeDef nwellShape \
    -in $tech \
    -enclosure 0.14 \
    -layer nwell \
    -exposedParams {name layer enclosure }

# Creating MPPRingObject
le::createMPPRingDef mppRing \
    -in $tech \
    -enclosedPaths diffPath \
    -enclosedShapes nwellShape \
    -masterPath mppMetallPath \
    -subRects subRects

```

le::createMPPSubRectDef

Description

Creates an MPPMasterSubRect definition.

Syntax

```

le::createMPPSubRectDef <name: string> -in <oaTech> -layer <string>
    [-purpose <string>] -width <float> -height <float> [-offset
    <float>] [-beginOffset <float>] [-endOffset <float>] [-connectivity
    <bool>] [-chopable <bool>] [-minEnclosure <float>] [-endOfLine
    <float>] [-spacing <float>] [-numRows <int>] [-spacingType <enum>]
    [-exposedParams [<string>]]

```

Returns

<oaAppObject>: Newly created MPPMasterSubRect definition object.

Arguments

`name`

The name for the MPPMasterSubRect definition to create.

`-in`

Technology in which to create the MPPMasterSubRect definition.

`-layer`

The layer on which to create each shape.

`-layer`

The layer on which to create each shape.

`-purpose`

The purpose on which to create each shape. The default is drawing.

`-width`

Width of each sub rectangle.

`-height`

Height of each sub rectangle.

`-offset`

The offset from the master path. The default is 0.

`-beginOffset`

The begin offset from the master path.

`-endOffset`

The end offset from the master path.

`-connectivity`

Specifies whether the shape is connected to the ring net.

Default: false.

`-chopable`

Specifies whether the shape is chopable.

Default: false.

`-minEnclosure`

The minimum enclosure over the sub rectangles. If not specified, use the minimum enclosure of LPP.

`-endOfLine`

The end of line extension past the sub rectangles. If not specified, use the minimum enclosure of LPP.

`-spacing`

The spacing between each sub rectangle. If not specified, use the minimum enclosure of LPP.

`-numRows`

The number of rows of sub rectangles. The default is 1.

`-spacingType`

The spacing type between sub rectangles (`{fit | fitBegin | min | minBegin}`). The default is `fit`.

`-exposedParams`

The list of exposed parameters for the `MPPMasterSubRect` definition.

Examples

The following example creates an `MPPSubRect` definition in the specified technology.

```
le::createMPPSubRectDef subRects \  
    -in $tech \  
    -chopable true \  
    -endOfLine 0.08 \  
    -height 0.06 \  
    -width 0.06 \  
    -layer cont \  
    -numRows 100 \  
    -minEnclosure 0.03 \  
    -spacingType fit \  
    -connectivity true \  
    -exposedParams {name layer chopable}
```

le::createNetGroup

Description

Returns the `oaGroup` with `oaGroupDef le_net_group` containing only `oaNet` members.

Syntax

```
le::createNetGroup {<oaNet | oaGroup>} -name <string>
```

Returns

<oaGroup>: Created oaGroup.

Arguments

oaNet

Collection of oaNet objects to include in the created group.

oaGroup

Collection of oaGroup objects.

-name

Name of the oaGroup to create.

Errors

- Input collection is not oaNet or oaGroup.
- Any oaGroup in input collection does not have oaGroupDef le_net_group.
- Collection is not from the same oaDesign.

Examples

```
set nets [db::getNets -of [ed] -filter {%name=="clk0m" ||
%name=="clk0p"}}
le::createNetGroup $nets -name clockPair
```

le::createOffTrackInfo

Description

Creates off track info objects.

Syntax

```
le::createOffTrackInfo <object: deFigure | oaFig>
```

Returns

<leOffTrackInfo> : Off-track information object.

Arguments

object

Object to create offTrackInfo.

le::createPath

Description

Creates a path in a design. You must provide an LPP, width, and a list of centerline points.

Syntax

```
le::createPath <points: [point]> -design <oaDesign> -width <float> -lpp  
<lpp> [-style <enum>] [-beginExtendVal <float>] [-endExtendVal <float>]  
[-net <oaNet | string>] [-checkIntersection <bool>]
```

Returns

New path (*oaPath*).

Arguments

{<point>}

List of points that form the path's center line.

-design *oaDesign*

Design context where the path is created.

-width <float>

Path width.

-lpp <lpp>

The layer and purpose pair.

Default: drawing.

-style

Begin/end style for the path.

Default: flush.

-beginExtendVal <float>

Beginning extension for the variable style.

Default: 0.0.

-endExtendVal <float>

End extension for the variable style.

Default: 0.0.

```
-net <string | oaNet>
```

Net name or oaNet object to which a new path is anchored. If the net does not exist, it is created.

Default: <none>.

```
-checkIntersection
```

Check self-intersection of a path's centerline when creating paths. (Default is: true)

When false, you can create a path with its centerline intersected.

Errors

- An invalid design context is passed.
- An invalid point-list is specified (such that the created path will have a self-intersection).
- A negative or zero value is provided for path's width.
- A net name is provided which does not adhere to CDB scalar net namespace. Bus / bundle format names are not permitted in layout.

Examples

In the following example, the first line specifies a list of points; the second line creates a path using those points and specifying a width, LPP, and net name.

```
set points [list [list 0.0 0.0] [list 0.1 0.1] [list 0.2 0.13] [list 0.05  
1.35]]  
le::createPath $points -design $oaDesign -width 0.2 -lpp {m1 drawing}  
-net "netA"
```

le::createPathSeg

Description

Creates a path segment in a design with a designated LPP, width, and the start and end points.

Syntax

```
le::createPathSeg <points: [point]> -design <oaDesign> -width <float>  
-lpp <lpp> [-beginStyle <enum>] [-beginExt <float>] [-beginLDiag  
<float>] [-beginRDiag <float>] [-beginRHalfWidth <float>] [-endStyle  
<enum>] [-endExt <float>] [-endLDiag <float>] [-endRDiag <float>]  
[-endRHalfWidth <float>] [-net <oaNet | string>]
```

Returns

New path-seg (*oaPathSeg*).

Arguments

`points`

List of points that contains exactly two points, which form the path-seg's centerline.

`-design`

Design where the path-seg is created.

`-width`

Path-seg width.

`-lpp`

The layer & purpose pair.

Default: drawing.

`-beginStyle`

Begin style for the path segment.

Default: truncate.

`-endStyle`

End style for the path segment.

Default: truncate.

`-beginExt`

Beginning extension for the variable style.

Default: 0.0.

`-endExt`

Ending extension for the variable style.

Default: 0.0.

`-beginLDiag`

Begin extension's left diagonal extension.

Default: 0.0.

`-endLDiag`

End extension's left diagonal extension.

Default: 0.0.

`-beginRDiag`

Begin extension's right diagonal extension.

Default: 0.0.

`-endRDiag`

End extension's right diagonal extension.

Default: 0.0.

`-beginRHalfWidth`

Begin extension's right half width.

Default: 0.0.

`-endRHalfWidth`

End extension's right half width.

Default: 0.0.

`-net`

Net name or `oaNet` object to which a new path segment is anchored. If the net does not exist, it is created.

Default: <none>.

Errors

- An invalid design context is passed.
- Number of provided points is not equal to 2.
- A negative or zero value is provided for path segment's width.
- A net name is provided which does not adhere to CDB scalar net. namespace. Bus/bundle format names are not permitted in the layout.
- End style is not *extend*, but user provides values for `-beginExtendVal` or/and `-endExtendVal`.

Examples

In the following example, the first line specifies a list of points; the second line creates a path segment using those points and specifying a width, LPP, and net name.

```
set points [list [list 0.0 0.0] [list 1.0 1.0]]
le::createPathSeg $points -design $oaDesign -width 0.2 -lpp {ml
drawing} -net "netA"
```

le::createPin

Description

Creates and returns an oaPin object for the provided oaTerm terminal object or name and assigns the provided list of oaPinFig objects to the newly created pin.

Syntax

```
le::createPin [<name: string>] [-design <oaDesign>] -term <oaTerm  
| string> -shapes <{<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
[-accessDir <int>] [-status <enum>]
```

Arguments

name

Pin name.

Default: Automatically generated according to the OA model.

-design

Design in which to create the pin.

-term

Terminal (or terminal name) of the pin.

-shapes

Shapes to attach to the pin.

-accessDir

Access direction for a pin. Any combination of none:0, top:1, bottom:2, left:4, and right:8. (Default is: 15)

-status

Placement status for a pin. ({ fixed | locked | none | placed | unplaced }, default is: placed)

Returns

<oaPin>: Newly created pin.

Errors

- The provided pinFig shapes are not from the same design.
- Illegal terminal/pin name is provided (that is, it does not adhere to CDB namespace rules).

- Empty list of shapes are provided.
- An object is provided in `-shapes` that cannot be a pin shape.
- Terminal name is provided which differs from net name that is associated with provided shapes.

Examples

In the following example, the first line defines a pin shape on the specified LPP; the second line creates a pin using that shape.

```
set shapes [le::createRectangle {{3 1} {4 2}} -design $oaDesign -lpp  
{m1 drawing}]  
le::createPin pin1 -shapes $shapes -term pin1
```

le::createPinsFromText

Description

Assigns text net information to the physical objects. When data is imported from GDSII, the net information is supplied by creating a text object that contains the name of the net, and by placing its origin over a physical object to 'net' it. This means that the database does not contain any actual net information. If an existing object already has a net assigned, its net will be overwritten according to the text net assignment. If two or more text net assignments exist on the same object, the last one processed, according to alphanumeric order, will be used.

The association between the text object and the physical object it 'nets' is based on the following:

- The text object and physical object are on the same layer.
- The text object and physical object are on layers defined as equivalent, using the `equivalentLayers` constraint supplied by the Technology Data.

This command operates on the set of layout designs in the supplied `cellViews` or the supplied layout designs themselves by opening them for write, updating the net information and converting the directly netted physical objects to pin shapes, and then closing them. If the design was not previously open, then prior to closing the design is saved. If the design was previously opened for read, it is converted to write, the modifications are made and saved and the mode is reverted back to read-only. If the design was previously opened for write, the modifications are made and only saved if the design was not previously modified.

When an object is converted to a pin shape, a single pin for the net to which the object is assigned is created, thus making all directly netted objects strongly connected pin shapes to one another, but weakly connected to any pre-existing pin shapes. If the net already exists and contains a terminal, then the pin's name is the same as the terminal name. If no

terminal exists on the net, then the net's name is used. If the resulting pin name conflicts with an already existing pin, the next available integer, beginning with '1', is appended to the end in order to resolve the conflict. `oaShape` and `oaVia` objects are converted to pin shapes, however `oaInstTerms` are not.

If the supplied `cellView` does not contain a design, or the design it does contain is not a layout design, it is silently ignored.

If write access for a design can not be obtained, a warning is generated and the command will continue processing the rest of the inputted designs.

Note:

The functionality within this command that creates pin shapes from existing objects is identical to that which exists in the `le::createPinsFromSelection` command. Thus both commands use the same internal routines.

Syntax

```
le::createPinsFromText -design <{<dmCellView>} | {<oaDesign>} |  
{oaDesign}>
```

Arguments

`-design`

The collection of `cellViews` that contain layout designs or the layout designs themselves to operate on.

Examples

The following example creates pins in place of text labels.

```
le::createPinsFromText -design $oaDesign
```

le::createPolygon

Description

Creates a polygon in a design. You must provide a layer purpose pair and a list of points for the command to compute the polygon's geometry layer.

If the user provides 2 points such that they form the diagonal endpoints of a valid rectangle, then a rectangular polygon will be created.

Syntax

```
le::createPolygon <points: [point]> [-net <oaNet | string>] -design  
<oaDesign> -lpp <lpp>
```

Returns

New polygon (*oaPolygon*).

Arguments

`points`

List of points that forms the polygon.

`-design`

Design where the polygon is created.

`-lpp`

The layer and purpose pair.

Default: drawing.

`-net`

Net name or oaNet object to which the new polygon is anchored.

Default: <none>.

Errors

- An invalid design is passed.
- An invalid point-list is specified (the points are self-intersecting).
- An invalid LPP is provided.
- A net name is provided which does not adhere to CDB scalar net namespace.

Examples

In the following example, the first line defines a list of points describing a polygon; the second line creates the polygon object using the list of points.

```
set points [list [list 0.0 0.0] [list 0.1 0.1] [list 0.2 0.13] [list 0.05  
1.34]]  
le::createPolygon $points -design $oaDesign -lpp {m1 drawing} -net "A"
```

le::createRectangle

Description

Creates a rectangle in a design. You must provide a layer purpose pair and two points for the command to compute the rectangle's geometry.

Syntax

```
le::createRectangle <box: box> -design <oaDesign> -lpp <lpp> [-net <oaNet  
| string>] [-color <string>] [-lockColor <bool>]
```

Returns

New rectangle (*oaRect*)

Arguments

box

The coordinates of the bounding box of the rectangle.

-design

Design where the rectangle is created.

-lpp

The layer and purpose pair.

Default: drawing.

-net

Net name or oaNet object to which the rectangle is anchored.

-color

The color of the rectangle. (Default is: noColor)

-lockColor

Specifies if the color is locked for the created rectangle. The default is false.

Errors

- An invalid design is passed.
- An invalid bounding box is specified (with 0 area).
- An invalid LPP is provided.
- A net name is provided which does not adhere to CDB namespace.

Examples

In the following example, the first line defines a list of points describing a rectangle; the second line creates the rectangle object using the list of points.

```
set points [list [list 0.0 0.0] [list 0.1 0.1]]  
le::createRectangle $points -design $oaDesign -lpp {m1 drawing} -net A
```

le::createRouteCorridor

Description

Creates a `leRouteCorridor` object in a design.

Syntax

```
le::createRouteCorridor <name: string> -design <oaDesign> [-nets  
  <{<oaNet>}>]
```

Returns

<leRouteCorridor> : Newly created routing corridor object.

Arguments

`name`

Name of the routing corridor.

`-design`

The design in which you can create the routing corridor.

`-nets`

Collection of nets to assign to the routing corridor.

Errors

- Empty net collection passed in `-nets`.
- Routing corridor with the same name already exists.

le::createRow

Description

Creates a row in a design.

Syntax

```
le::createRow <points: [point]> -design <oaDesign> -siteDef  
  <oaScalarSiteDef | string> [-name <string>] [-direction <bool>]  
  [-rowOrigin <enum>] [-siteSpacing <float>] [-rowSpacing <float>]  
  [-coreToleftSpacing <float>] [-coreTobottomSpacing <float>]  
  [-coreTorightSpacing <float>] [-coreTotopSpacing <float>] [-rowPairing  
  <bool>] [-startFromFirstRow <bool>] [-flipFirstRow <bool>]
```

Returns

<oaRow>: Collection of new rows.

Arguments

`points`

The list of points that form row.

`-design`

Design in which to create the row.

`-siteDef`

Site definition or its name, for new object.

`-name`

The prefix of row name. Default is *R*.

`-direction`

Row grow direction, Horizontal if true or Vertical if false. Default is *True*.

`-rowOrigin`

The row origin {leftbottom | lefttop | rightbottom | righttop}.
Default is *leftbottom*.

`-siteSpacing`

The spacing between the sites. Default is *0*.

`-rowSpacing`

The spacing between the rows. Default is *0*.

`-coreToleftSpacing`

The left spacing between row region and boundary. Default is *0*.

`-coreTobottomSpacing`

The bottom spacing between row region and boundary. Default is *0*.

`-coreTorightSpacing`

The right spacing between row region and boundary. Default is *0*.

`-coreTotopSpacing`

The top spacing between row region and boundary. Default is *0*.

`-rowPairing`

The pairing flag of rows. Default is *false*.

`-startFromFirstRow`

The start first flag of pairing rows. Default is *false*.

`-flipFirstRow`

The flip first flag of pairing rows. Default is *false*.

Examples

The following example creates a row with the specified side definition for the standard cell design.

```
le::createSiteDef siteDef -design $design -height 1.24 -width 0.48
le::createRow {{0.0 0.0} {20.0 10.0}} \
  -design $design \
  -siteDef siteDef
```

le::createSiteDef

Description

Creates a Site Definition. This command automatically adds your change to the technology file and imports your change into the design.

Syntax

```
le::createSiteDef <name: string> -design <oaDesign> [-type <enum>] -width
<float> -height <float> [-syminx <bool>] [-syminy <bool>] [-syminrot
<bool>]
```

Returns

<oaSiteDef>: Newly created site definition object.

Arguments

`name`

The name of the site.

`-design`

Design in which to create the site definition.

`-type`

The type of site to create ({core | pad}).

`width`

The width of the site in user units (usually microns).

`-height`

The height of the site in user units (usually microns).

`-syminx`

Controls whether the site allows cells to be mirrored on the X axis (true).

Default: false.

`-syminy`

Controls whether the site allows cells to be mirrored on the Y axis (true).

Default: false.

`-syminrot`

Controls whether the site allows cells to be rotated (true). Default is false (no rotation).

Default: false.

Examples

```
le::createSiteDef newSiteDef -design [db::getAttr editDesign -of
[de::getActiveContext]]-type core -width 20 -height 4
```

To verify if this definition is added to the database, you can export the technology file from the library using **Technology Manager > File > Export**. In the technology file, the definition should look as follows:

```
siteDefs (
  scalarSiteDefs (
    ;(siteDefName siteDefType width height symInX symInY symInR90 )
    ;(-----)
    ( newSiteDef core 20 4 false false false )
  ) ;scalarSiteDefs
) ;siteDefs
```

For more information about the technology file, see the *Technology and Display Resource File Reference Manual* in the online Help.

le::createStripedWire

Description

Creates striped wire according to the input arguments.

After creating the segment objects by the [le::createStripeSegment](#) command, those Tcl objects can be packaged to a list by the “list” command as the input argument to the `le::createStripedWire` command. During the striped wire creation, the input segment sequence is honored.

Syntax

```
le::createStripedWire <segments: [leStripeSegment]> [-design <oaDesign>]  
[-net <oaNet | string>] [-beginStyle <enum>] [-endStyle <enum>]  
[-beginExtendVal <float>] [-endExtendVal <float>] [-beginStrap <bool>]  
[-endStrap <bool>] [-slot <bool>] [-stagger <bool>]
```

Returns

<leStripedWire> : Striped wire created.

Arguments

`segments`

List of stripe segments.

`-design`

Design in which to create the striped wire.

`-net`

Net name or oaNet.

`-beginStyle`

Begin style of the striped wire ({ extend | truncate | variable }).

`-endStyle`

End style of the striped wire ({ extend | truncate | variable }).

`-beginExtendVal`

Begin extension of the striped wire.

`-endExtendVal`

End extension of the striped wire.

`-beginStrap`

Begin strap of the striped wire.

`-endStrap`

End strap of the striped wire.

`-slot`

Slot of the striped wire.

`-stagger`

Stagger of the striped wire.

Errors

- Invalid design or net info is passed.
- Invalid segment info is passed.

Examples

The following example creates a striped wire with two segments over layer M1 and M3. The via definition is decided by the system.

```
set seg1 [ le::createStripeSegment {{0.05 0.05} {1.15 0.05} {1.15 1.15}}  
-lpp {M1 drawing} -stripes 2 -width 0.56 -spacing 0.15 ]  
  
set seg2 [ le::createStripeSegment {{1.15 1.15} {8.55 1.15}}  
-lpp {M3 drawing} -stripes 3 -width 0.76 -spacing 0.22 ]  
  
le::createStripedWire [list $seg1 $seg2] -design [ed] -net netA  
-beginStrap true -endStrap true
```

le::createStripeSegment

Description

Creates a disconnected striped segment object. This object is a Tcl object to supply stripe segment parameters for creating striped wire command `le::createStripedWire` as the input argument. The database update only happens until the `le::createStripedWire` command is run successfully.

Syntax

```
le::createStripeSegment <points: [point]> [-width <float>] [-spacing  
<float>] [-stripes <int>] -lpp <lpp> [-via [<oaViaDef | string>]]  
[-offset <float>] [-slotLength <float>] [-slotSpacing <float>]
```

Returns

<leStripeSegment> : Stripe segment created.

Arguments

points

Point list of the stripe segment.

-width

Stripe segment width.

-spacing

Stripe segment spacing.

`-stripes`
Stripe segment number.

`-lpp`
LPP of the stripe segment.

`-offset`
Spacing before the first bridge.

`-via`
Via definition name or `oaViaDef`.

`-slotLength`
Length of the slots (or the distance between the bridges).

`-slotSpacing`
Width of the bridges between the striped wires.

Examples

The following command creates a Tcl stripe segment object according to the input parameters.

```
le::createStripeSegment {{0.05 0.05} {1.15 0.05} {1.15 1.15}} -lpp {M1  
drawing} -stripes 2 -width 0.56 -spacing 0.15 -via {V12 V23}
```

le::createSyncFigGroup

Description

Creates `oaFigGroup` from the given layout objects and adds them to the specified `oaGroup`.

Syntax

```
le::createSyncFigGroup [<name: string>] -in <oaGroup | string> -objects  
<{<deFigure>} | {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>}  
| {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}} [-source  
<{<deFigure>} | {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} |  
{<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}] [-objectsOrder  
<{<deFigure>} | {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} |  
{<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}] [-sourceOrder  
<{<deFigure>} | {<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} |  
{<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}]
```

Returns

`<oaFigGroup>`: The `oaFigGroup` object created by the command.

Arguments

`name`

Name of the `oaFigGroup`.

`-in`

Synchronous group that the `oaFigGroup` is to be added to.

`-objects`

Collection of `oaFig/deFigure` objects or a single `oaFig/deFigure` to create the `oaFigGroup`.

`-source`

Collection of `oaFig/deFigure` objects or a single `oaFig/deFigure` from one of the peer `oaFigGroups` to associate correspondence with.

`-objectsOrder`

Order of `oaFig/deFigure` objects to sync with the `oaFigGroup`.

`-sourceOrder`

Order of `oaFig/deFigure` objects from one of the peer `oaFigGroups` to associate correspondence with.

Examples

The following example creates an `oaFigGroup` from the provided objects and adds the newly created `oaFigGroup` to the provided `syncGroup`.

```
# Create a syncClone by adding existed selected objects to the group and
# syncing with SyncGroup pear
set selectedObject [de::getSelected -design $design]
set syncClone [le::createSyncFigGroup syncCloneName -in SyncGroup \
               -objects $selectedObject]
db::getAttr syncClone.name
db::getAttr syncClone.type
```

le::createSyncGroup

Description

Creates an `oaGroup` in the given design.

Syntax

```
le::createSyncGroup [<name: string>] -design <oaDesign>
```

Returns

<oaGroup>: The oaGroup object created by the command.

Arguments

name

Name of the synchronous oaGroup.

-design

Layout design where oaGroup is to be created.

Examples

The following example creates a syncGroup in the provided design. A name can be specified optionally that should be given to the syncGroup.

```
# Create a synchronization group in a design with a name SyncGroup_1
Set syncGroup [le::createSyncGroup SyncGroup_1 -design $design]
db::getAttr syncGroup.name
db::getAttr syncGroup.type
```

le::createTerm

Description

Creates and returns a new terminal with the given name for the given oaNet object. You can specify the access direction for the terminal and create a connect definition during terminal creation.

Syntax

```
le::createTerm [<name: string>] -net <oaNet | string> [-design
<oaDesign>] [-type <enum>] [-assignmentName <string>] [-defaultNet
<oaNet | string>]
```

Returns

The created terminal object ([oaTerm](#)).

Arguments

name

The terminal name.

Default: Name of the <oaNet> that <oaTerm> is to be created on.

-net

The net or net name for which the terminal should be created.

-design

The design where the net should be found (if -net is provided).

-type

The type of terminal being created.

Default: inputOutput.

-assignmentName

The name of the connect definition for terminal.

-defaultNet

Default net of the connect definition for the terminal.

Errors

- An invalid net object is provided.
- An invalid terminal name is specified (does not adhere to CDB namespace).

Examples

The following example creates a terminal with name *A* for net *A*.

```
le::createTerm "A" -net [db::getNets A -of $oaDesign]
```

le::createTrack

Description

Creates an leTrack object. This object is used when creating an leTrackGroupDef object.

Syntax

```
le::createTrack <name: string> [-displayName <string>] -in <oaDesign  
| oaTech> -trackType <enum> -offset <float> [-width <float>] [-color  
<string>] [-visible <bool>] [-active <bool>] [-signalType <enum>]  
[-gridOffsetLow <float>] [-gridOffsetHigh <float>] [-gridPointsLow  
{<float>}] [-gridPointsHigh {<float>}] [-gridVisibleLow <bool>]  
[-gridVisibleHigh <bool>]
```

Arguments

name

Name of the leTrack object.

-displayName

Display Name of the leTrack object. (Default:)

`-in`

Design or technology in which to create the track.

`-trackType`

Track type ({ default | nonReserved | reserved }).

`-offset`

Offset of track from the region tile lower left.

`-width`

Width of the track.

Default: 0.

`-color`

Color of the track.

Default: noColor.

`-visible`

Default visible state of the track.

Default: false.

`-active`

Default active state of the track.

Default: true.

`-signalType`

Restricts nets that can snap to the track based on net sigType. Valid values: all (default), analog, clock, ground, power, reset, scan, signal, tieHi, tieLo, and tieoff.

`-gridOffsetLow`

Offset distance for grid starting points. Use this argument with `-gridPointsLow`.

Default: 0.0

`-gridOffsetHigh`

Offset distance for grid ending points. Use this argument with `-gridPointsHigh`.

Default: {}

`-gridPointsLow`

Starting points of the grid. Use `-gridPointsLow` and `-gridPointsHigh` to control where interconnects start and stop when routing on tracks.

Default: {}

`-gridPointsHigh`

Ending points of the grid. Use `-gridPointsLow` and `-gridPointsHigh` to control where interconnects start and stop when routing on tracks.

Default: {}

`-gridVisibleLow`

Default visible state of low-end track grids.

Default: true

`-gridVisibleHigh`

Default visible state of high-end track grids.

Default: true

Returns

<leTrack>: The newly created leTrack object.

Errors

- Width is less than or equal to 0 for non-reserved and reserved type tracks.

Examples

```
le::createTrack trackRed -in $design \  
-width 0.05 \  
-offset 0.0 \  
-trackType reserved \  
-visible true \  
-active true \  
-color red
```

le::createTrackGroupDef

Description

Creates an leTrackGroupDef object in a technology or design database.

Syntax

```
le::createTrackGroupDef <name: string> [-displayName <string>]  
-in <oaDesign | oaTech> -tracks <dbCollection> -layer <string>
```

```
-direction <enum> -pitch <float | point> [-allowCrossBoundary <bool>]  
[-renderPurpose <string>] [-LEGShapeSnapPurposes <string>]
```

Returns

<leTrackGroupDef>: The newly created leTrackGroupDef object.

Arguments

name

trackGroupDef name.

-displayName

trackGroupDef display name. (Default:)

-in

Technology or design database in which to create the definition.

-tracks

Collection of leTrack objects.

-layer

Layer of the trackGroupDef (all tracks in groupDef will be on this layer).

-direction

Direction of the tracks in the definition ({ horizontal | vertical }).

Direction of horizontal results in tracks parallel to the X-Axis, and vertical results in tracks parallel to the Y-Axis.

-pitch

Size of the group, in the X and Y directions. You can specify 0 for X pitch if the track direction is horizontal. You can specify 0 for Y pitch if the track direction is vertical.

-allowCrossBoundary

Determines if tracks from group can cross region boundary. The default is true.

[-renderPurpose]

Purpose to be used to render tracks. Default value is "" in which case 'drawing' purpose will be used (Default:)

[-LEGShapeSnapPurposes]

Only shapes with the purposes defined can snap to track LEGs. Default value is "" meaning all purposes are allowed (Default:)

Errors

- Name must be unique in the container. For tech it must be unique in the entire tech graph.
- Pitch is less than or equal to 0.
- Design viewType is not maskLayout.
- Owner not opened for edit.
- Color not supported for layer.

Examples

```
set track [le::createTrack trackRed -in $design \  
-width 0.05 -offset 0.0 \  
-trackType reserved -visible true -active true -color red]  
  
le::createTrackGroupDef trackGroupDef -in $design \  
-direction horizontal \  
-layer M1 \  
-pitch {0.04 0.04} \  
-tracks $track
```

le::createTrackPattern

Description

Creates leTrackPattern in a design database.

Syntax

```
le::createTrackPattern <name: string> (-of <leTrackPattern> |  
-trackPatternDef <leTrackPatternDef>) -design <deContext | oaDesign>  
[-color <string>] [-box <box>] [-blockPitch <float>] [-offset  
<float>] [-visible <bool>] [-active <bool>] [-gridOffsetLow <float>]  
[-gridOffsetHigh <float>] [-gridPointsLow [<float>]] [-gridPointsHigh  
[<float>]]
```

Returns

<leTrackPattern>: The newly created leTrackPattern.

Arguments

name

TrackPattern name

-design

Design in which to create the pattern

<code>-of</code>	Reference for unspecified options
<code>-trackPatternDef</code>	Definition for this pattern
<code>-color</code>	Color of pattern
<code>-box</code>	Optional pattern extend
<code>-blockPitch</code>	Block pitch in user units
<code>-offset</code>	Distance from design origin to first track
<code>-visible</code>	Default visibility of tracks in this pattern (Default is: true)
<code>-active</code>	Default activity for tracks in this pattern (Default is: false)
<code>-gridOffsetLow</code>	Offset for grid starting points (Default is: 0)
<code>-gridOffsetHigh</code>	Offset for grid ending points
<code>-gridPointsLow</code>	Starting points of grid
<code>-gridPointsHigh</code>	Ending points of grid

le::createTrackPatternDef

Description

Creates an `leTrackPatternDef` object in the technology or in the design database.

Syntax

```
le::createTrackPatternDef <name: string> [-of <leTrackPatternDef>]  
  [-orientation <enum>] [-layer <string>] [-purpose <string>] -in  
  <deContext | oaDesign | oaTech> [-patternType <enum>] -trackPitch  
  <float> [-offset <float>] [-numTracks <unsigned int>] [-blockPitch  
  <float>] [-width <float>] [-visible <bool>] [-active <bool>] [-color  
  <string>] [-autoGenerateTrack <bool>]
```

Returns

<leTrackPatternDef> : Newly created leTrackPatternDef

Arguments

name

TrackPatternDef name.

-of

Reference for unspecified options

-orientation

Pattern Def orientation: horizontal or vertical.

-layer

The layer which snaps to this pattern.

-purpose

Purpose on the given layer which snaps to this pattern.

Default: oaAny.

-in

Design or technology in which to create the definition.

-patternType

Pattern Type: default, nonReserved, or reserved.

Default: default.

-trackPitch

Track pitch.

-offset

Offset of the first track from the design origin.

Default: 0.

`-numTracks`

Number of tracks in a block.

Default: 0.

`-blockPitch`

Block pitch in user units.

Default: 0.

`-width`

Width parameter for non-reserved and reserved patterns.

Default: 0.

`-visible`

Default visibility of tracks in this pattern.

Default: true.

`-active`

Default activity for tracks in this pattern.

Default: false.

`-color`

Color of pattern definition. No color and *Colorn* is supported. Same Color is not supported.

Default: noColor.

`-autoGenerateTrack`

The auto generate track flag of the track pattern definition.

Default: true.

Examples

```
le::createTrackPatternDef trackPatternDefinition \  
-orientation horizontal \  
-layer M1 \  
-purpose drawing \  
-in $design \  
-patternType nonReserved \  
-trackPitch 0.1 \  
-width 0.04 \  
-visible true \  
-active true \  
-color red
```

le::createTrackPcell

Description

Creates a track PCell.

Syntax

```
le::createTrackPcell <box: box> -design <oaDesign> [-lpp <lpp>] [-name  
<string>] [-libName <string>] [-cellName <string>] [-viewName <string>]  
[-isGlobal <bool>] [-generateShape <bool>] [-snapTracks <string>]  
[-presetLibName <string>] [-presetCellName <string>] [-presetViewName  
<string>] [-preset <string>]
```

Returns

<oaInst> : The newly created track PCell object.

Arguments

box

The coordinates of bounding box of the track PCell.

-design

Design in which to create the track PCell.

-lpp

The lpp on which to create the track PCell.

-name

Name of the track PCell.

-libName

Name of the master cell library.

-cellName

Name of the master cell.

-viewName

Name of the master cellview.

-isGlobal

The global setting of the track PCell. The default is false.

-generateShape

The generate shape setting of the track PCell. The default is false.

`-snapTracks`

The parameter snap track of the track PCell.

`-presetLibName`

Name of the preset library.

`-presetCellName`

Name of the preset cell.

`-presetViewName`

Name of the preset cellView.

`-preset`

Name of the preset.

le::createTrackRegion

Description

Creates an leTrackRegion object in a design database.

Syntax

```
le::createTrackRegion [<name: string>] -design <oaDesign> -points  
  [<point>] [-flipX <bool>] [-flipY <bool>] [-flipFirstX <bool>]  
  [-flipFirstY <bool>] [-completeX <bool>] [-completeY <bool>] [-visible  
  <bool>] [-orient <oaOrient>]
```

Returns

<leTrackRegion>: The newly created leTrackRegion object.

Syntax

```
le::createTrackRegion [<name: string>] -design <oaDesign> -regionDef  
  <leTrackRegionDef> -points [<point>] [-flipX <bool>] [-flipY <bool>]  
  [-flipFirstX <bool>] [-flipFirstY <bool>] [-offsetTarget <enum>]  
  [-tileOffset <point>] [-visible <bool>] [-orient <enum>]
```

Returns

<leTrackRegion> : The newly created leTrackRegion object

Arguments

name

Name of the region object (Default is:)

`-design`
Design in which to create the region

`-regionDef`
leTrackRegionDef object used in the region

`-points`
List of points

`-flipX`
Determines if tiles in the region are flipped about the X-axis (Default is: false)

`-flipY`
Determines if tiles in the region are flipped about the Y-axis (Default is: false)

`-flipFirstX`
Determines if the first tile in the region is flipped about the X-axis (Default is: false)

`-flipFirstY`
Determines if the first tile in the region is flipped about the Y-axis (Default is: false)

`-offsetTarget`
Determines if the offset of tracks is relative to the design axis or region boundary ({ design | region }, default is: design)

`-tileOffset`
Offset of the first tile in the region

`-visible`
Determines if semantic oaAreaBoundary can be drawn (independent of deObjectFilter) (Default is: true)

`-orient`
The orientation of the created track region ({ MX | MY | R0 | R180 }, default is: R0)

Errors

- Name must be unique in the design.
- Design viewType is not maskLayout.
- Track group defs cannot be referenced, for example: owner is tech and track group def used is in the design.

- Design not opened for edit.
- Points result in a 0 area boundary.
- Points do not create a rectangle.

Examples

```
set track [le::createTrack track -in $design -offset 0 \
    -trackType reserved -visible 1 -width 0.1 \
    -active 1 -color E2]
set trackGroupDef [le::createTrackGroupDef trackGroupDef \
    -in $design -direction horizontal \
    -layer M1 -pitch {1 2} -tracks $track]
set trackRegionDef [le::createTrackRegionDef trackRegionDef \
    -in $design -groupDefs $trackGroupDef]

le::createTrackRegion trackRegion -design [ed] \
    -regionDef $reserved \
    -points {{0.0 -0.0} {0.0 100.0} {100.0 100.0} {100.0 -0.0}}
```

le::createTrackRegionDef

Description

Creates an leTrackRegionDef definition in a technology or design database.

Syntax

```
le::createTrackRegionDef <name: string> [-flipX <bool> | -flipXGroups
    <{<oaAppObject>}>] [-flipY <bool> | -flipYGroups <{<oaAppObject>}>]
    -in <oaDesign | oaTech> -groupDefs <{<oaAppObject>}> [-flipTilesX
    <bool>] [-flipTilesY <bool>] [-flipTilesXFirst <bool>] [-flipTilesYFirst
    <bool>] [-minCountX <int>] [-minCountY <int>] [-maxCountX
    <int>] [-maxCountY <int>] [-snapGrid <point>] [-snapOffset
    <point>] [-autoGenerateGlobalRegion <bool>] [-enclosedGroups
    <{<oaAppObject>}>] [-groupEnclosure <point>] [-defaultTracksVisible
    <int>] [-defaultTracksActive <int>] [-flipGroupsFirst <{<oaAppObject>}>]
    [-groupFlipTarget <enum>]
```

Returns

<leTrackRegionDef> : The newly created leTrackRegionDef object.

Arguments

name

The trackRegionDef name.

-in

Design or technology in which to create the definition.

`-groupDefs`

Collection of `leTrackGroupDef` objects.

`-flipX`

Determines if groups in the definition are flipped about the X-axis.

Default: false.

`-flipXGroups`

Collection of `leTrackGroupDef` to flip in X.

`-flipY`

Determines if groups in the definition are flipped about the Y-axis.

Default: false.

`-flipYGroups`

Collection of `leTrackGroupDef` to flip in Y.

`-flipTilesX`

Determines default value of flip X when creating `leTrackRegion` or `leGlobalTrackRegion`.

Default: false.

`-flipTilesY`

Determines default value of flip Y when creating `leTrackRegion` or `leGlobalTrackRegion`.

Default: false.

`-flipTilesXFirst`

Determines default value of `flipFirstX` when creating `leTrackRegion` or `leGlobalTrackRegion`.

Default: false.

`-flipTilesYFirst`

Determines default value of `flipFirstY` when creating `leTrackRegion` or `leGlobalTrackRegion`.

Default: false.

`-minCountX`

Minimum number of least common multiple (LCM) times groups repeated in the X direction.

Default: 1.

`-minCountY`

Minimum number of least common multiple (LCM) times groups repeated in the Y direction.

Default: 1.

`-maxCountX`

Maximum number of least common multiple (LCM) times groups repeated in the X direction.

Default: -1.

`-maxCountY`

Maximum number of least common multiple (LCM) times groups repeated in the Y direction.

Default: -1.

`-snapGrid`

Snap grid to use during interactive creation and editing of `leTrackRegion`.

`-snapOffset`

Track region offset distance from the design axes.

`-autoGenerateGlobalRegion`

Auto create a global region on first design open.

Default: false.

`-enclosedGroups`

Collection of `leTrackGroupDef` that abide by `groupEnclosure` value.

Default is an empty collection.

`-groupEnclosure`

Vector value to enclose group definitions inside the region boundary. X value is used for horizontal groups, and Y value is used for vertical groups.

Default: {0 0}

`-defaultTracksVisible`

Integer value that allows user to override active attribute for all `leTracks`.

When specifying to -1, no change to leTracks visible attribute. When specifying to 0, sets leTracks visibility attribute false. When specifying to 1, sets leTracks visible attribute true.

Default: -1

`-defaultTracksActive`

Integer value that allows user to override visible attribute for all leTracks.

When specifying to -1, no change to leTracks visible attribute. When specifying to 0, sets leTracks visibility attribute false. When specifying to 1, sets leTracks visible attribute true.

Default: -1

`-flipGroupsFirst`

Collection of leTrackGroupDef to flip first. Valid only when flipX or flipY is true or flipGroupsX or flipGroupsY is specified.

`-groupFlipTarget`

Determines if flip group inside region or tile. Valid values: region, tile. The default is tile.

Examples

```
set track [le::createTrack track -in $design -offset 0 \
  -trackType reserved -visible 1 -width 0.1 \
  -active 1 -color E2]
set trackGroupDef [le::createTrackGroupDef trackGroupDef \
  -in $design -direction horizontal \
  -layer M1 -pitch {1 2} -tracks $track]

le::createTrackRegionDef trackRegionDef -in $design \
  -groupDefs $trackGroupDef
```

le::createVia

Description

Creates a single oaVia object and displays the result in the oaViaParam field. Uses the provided oaViaDef object.

Use the -params option to specify the parameters for custom vias or to specify the number of rows and columns for a standard via. The list of parameters is provided below:

- cutWidth: Cut Width
- cutHeight: Cut Height

- cutRows: Rows
- cutColumns: Columns
- cutSpace: Spacing
- layer1Enc: Layer1 Enclosure
- layer2Enc: Layer2 Enclosure
- layer1Offset: Layer 1 Offset
- layer2Offset: Layer2 Offset
- origOffset: Origin Offset
- implant1Enc: Implant Layer1 Enclosure
- implant2Enc: Implant Layer1 Enclosure

If you specify a property that is not in the above list, the command creates a string property for the via.

Syntax

```
le::createVia -design <oaDesign> -definition <oaViaDef | string> -origin  
<point> [-params [<map>]] [-orient <enum>]
```

Returns

The newly created object ([oaVia](#)).

Arguments

-definition

Via definition (oaViaDef) or the via definition name for the new object. Also specifies the default values for the related via parameters.

-origin

The location of the via origin.

-design

The oaDesign where the via should be created.

-params

List of properties for the via object. Each item should have {name value} format, for example: {{cutRows 2} {cutColumns 2} ... }. These properties will be assigned to the via object.

Default: An empty list.

`-orient`

Via orientation.

Default: R0.

Errors

- An invalid oaViaDef object.
- An invalid cell view is provided.
- Negative or zero value is passed as row/cols parameter.
- Negative or zero value is passed as dx/dy parameter.

Examples

The following example creates a via using via definition *via12* at the defined location.

```
le::createVia -definition "via12" -design [ed] -origin {15 15} \  
  -params {{cutRows 10} {cutColumns 10}}
```

le::createVoltagePhysicalAnnotations

Description

Creates physical voltage annotations for all nets.

Syntax

```
le::createVoltagePhysicalAnnotations [<context: deContext>] [-method  
  <enum>] [-scope <enum>] [-nets <string | {<oaNet>}>] [-depth <enum>]  
  [-height <float>] [-font <enum>] [-orient <enum>] [-notation <enum>]  
  [-annotateProc <string>] [-clearProc <string>] [-overwrite <bool>]
```

Arguments

`context`

Context object of the create physical voltage annotations operation.

`-method`

Physical annotation method. Valid values: custom and label (default).

`-scope`

(Label method only) Text label annotation scope. Valid values: Design (default), Nets, and Viewport.

`-nets`

(Label method only) oaNet or net name list for Nets scope.

`-depth`

(Label method only) Text label annotation depth. Valid values: 0x (default), Full, and View.

`-height`

(Label method only) Text height in user unit. The default is 0.2.

`-font`

(Label method only) Text font. Valid values: euroStyle, fixed, gothic, math, milSpec, roman, script, stick (default), and swedish.

`-orient`

(Label method only) Text orientation. Valid values: MX, MXR90, MY, MYR90, R0 (default), R180, R270, and R90.

`-notation`

(Label method only) Number notation. Valid values: engineering, literal (default), and scientific.

`-annotateProc`

(Custom method only) Annotation procedure name for the custom method.

`-clearProc`

(Custom method only) Name of the Tcl procedure for removing the annotations.

`-overwrite`

Overwrites the existing annotations. The default is true.

le::cutCorner

Description

Cuts corners of objects with the given cut length and the number of sides.

Syntax

```
le::cutCorner <objects: {<defigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
-cutLength <float> -cutSides <int> [-remainAsPath <bool>]
```

Returns

{<oaFig>}: Collection of objects.

Arguments

`objects`

List of objects to be cut.

`-cutLength`

The distance from the corner to the point that will be cut.

`-cutSides`

The number of sides at each corner to be cut.

`-remainAsPath`

The option to control whether remain as path after cutting path.

Default: false.

Errors

- If the cut length exceeds the allowed value.
- If the cut length or cut sides value is negative.

Examples

The following example cuts corners (both, convex, and concave vertices) of the specified polygon, by 0.5u from corner, creating 32 sides.

```
set design [ed]
set polygon [le::createPolygon {{0.22 0.63} {1.485 0.63} {1.485 0.05}
 {0.93 0.05} {0.93 0.35} {0.22 0.35}} -design $design -lpp {m2 drawing}]
le::cutCorner $polygon -cutLength 0.03 -cutSides 32
```

le::cycleDensity

Description

Cycles the density tip.

Syntax

```
le::cycleDensity [<window: deCanvas | giWindow | int>]
```

Arguments

`window`

Window or canvas.

le::delete

Description

Deletes specified objects. Deletes full and partial layout objects. If the full object is provided, the command deletes the object from the design. If a partial object is provided, the command removes the provided parts from the object (thus the object will be modified).

Syntax

```
le::delete <objects: {<defigure>} | {<deSelectionSet>} | {<oaFig>} |
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>
[-partialObject <enum>]
```

Arguments

`objects`

List of objects to delete. Objects are represented by a selectable figure.

`-partialObject`

Partial object {full | ignore | partial}.

Default: ignore.

Examples

The following example deletes all selected objects in the design.

```
le::delete [de::getSelected -design $oaDesign]
```

le::deleteCutMetal

Description

Deletes cut metal shapes.

Syntax

```
le::deleteCutMetal <layer: string> -design <oaDesign> [-area <enum>]
[-points [<point>]] [-cutPurpose <string>] [-onlyShapeBelowCut <bool>]
```

Arguments

`layer`

The target layer which user needs to handle.

`-design`

Design in which to active delete cut metal command.

`-area`

The area to use delete cut metal command. ({ design | polygon | rectangle }, default is: design)

`-points`

Point list for the rectangle/polygon area, based on which area type is specified.

`-cutPurpose`

Specifies the purpose for cut-metal shapes in the ICC Co-Design flow.

`-onlyShapeBelowCut`

Specify to only delete shape below cut. (Default: false)

le::deleteDensityResult

Description

Deletes the density result.

Syntax

```
le::deleteDensityResult [<window: deCanvas | giWindow | int>] [-layer  
<string>] [-group <string>] [-all <bool>]
```

Arguments

`window`

Window or canvas.

`-layer`

Layer name in layer mode to delete density.

`-group`

Group name in the config file mode to delete density.

`-all`

Specifies whether to delete all results. If specified, this argument ignores the layer and group option. The default is false.

le::deleteGeneratedPins

Description

Deletes the pins that were generated from the **Generate Pins** command.

Syntax

```
le::deleteGeneratedPins -design <oaDesign>
```

Arguments

-design

Design in which to delete generated pins.

Errors

- The design is not of type MaskLayout.

Examples

The following example deletes pins that were generated with the Generate Pins command.

```
set design [ed]  
le::deleteGeneratedPins -design $design
```

le::deleteVirtualConnection

Description

Deletes the objects which are generated by the **Virtual Connection** command.

Syntax

```
le::deleteVirtualConnection -design <oaDesign>
```

Arguments

-design

Design in which to delete virtual connection.

le::designSummary

Description

Generates a summary of the design.

Syntax

```
le::designSummary
```

Returns

<string> : Design Summary.

le::displayFlightlines

Description

Displays all flight lines. If no flight line exists then one will be created for each oaNet passed (or all oaNet in oaGroup passed.)

To obtain a collection of nets, use the [db::getNets](#) command.

If the [le::displayPinConnections](#) command is called, the set of nets for which standard flight lines are being displayed will be temporarily disabled until:

- The [le::displayFlightlines](#) command is called.
- The [le::clearPinConnections](#) command is called and results in all pin connection flight lines being cleared.

If pin connection flight lines are currently being displayed, executing this command will clear them.

This command shows a warning when the Connectivity Engine is disabled and the immediately return without performing any other action.

Syntax

```
le::displayFlightlines {<oaNet | oaGroup >}
```

Arguments

oaGroup

Collection of group objects to display flight lines. Sets the `flightlinesEnabled` attribute to *true*.

oaNet

Collection of net objects to display flight lines. Sets the `flightlinesEnabled` attribute to *true*.

Errors

- The net is invalid.
- The design of the net is not a layout design.

Examples

The following example displays the flight lines for all the nets in the design.

```
le::displayFlightlines [db::getNets -of $oaDesign]
```

le::displayPinConnections

Description

Displays pin connection flight lines for the collection of pins in all windows for which their design is the current edit design. Once activated, the list of pins is maintained until:

- A pin's design is closed.
- The [le::displayFlightlines](#) command is called.
- The [le::clearPinConnections](#) command is used to clear all existing pin connection flight lines.

Subsequent calls to this command add new pins to the preexisting set, while silently ignoring pins that are already members.

To obtain a collection of pins, use the [db::getPins](#) command.

If standard static flight lines are currently being displayed, executing this command will temporarily clear them. To restore the standard static flight lines to their previous state, use the [le::clearPinConnections](#) command to clear all pin connection flight lines.

Syntax

```
le::displayPinConnections {<oaPin>}
```

Arguments

<oaPin>

Collection of pins to display pin connection flight lines.

Errors

- The pin is invalid.
- The design of the pin is not a layout design.

Examples

The following example displays flight lines between pins in the design.

```
le::displayPinConnections [db::getPins A -of $oaDesign]
```

le::distribute

Description

Aligns and distributes specified objects.

Syntax

```
le::distribute <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
-src <deFigure | oaFig> [-alignSide <enum>] [-direction <enum>]  
[-distance <float>] [-byPitch <bool>] [-sort <enum>] [-sortDirection  
<enum>] [-distributeFirstObject <bool>]
```

Returns

{<deFigure>} | {<oaFig>} : Collection of aligned objects.

Arguments

objects

List of target objects to distribute.

-src

Source object for distribute, can be also a sub-object.

-alignSide

Alignment side. The value of this option is used only if at least one full object is specified. ({ bottom | horizontalMiddle | left | right | top | verticalMiddle }, default is: bottom)

-direction

Direction according to which the distance between the aligning edges should be applied. ({ bottom | left | right | top }, default is: right)

-distance

Distance between distributing objects. (Default is: 0)

-byPitch

When true, distributes objects by pitch. When false, distributes objects or by spacing. (Default is: false)

-sort

Direction according to which the distance between the aligning edges should be applied. ({ cell | instance | net }, default is: instance)

-sortDirection

Direction according to sorting distribute objects. ({ fromBottom | fromLeft | fromRight | fromTop | none }, default is: none)

-distributeFirstObject

Distributes the first object. (Default is: false)

le::EMAccuracyTool

Description

Enables the EM accuracy tool.

Syntax

```
le::EMAccuracyTool [-step <string>] [-netName <string>]
```

Returns

<string> : Step result.

Arguments

-step

Specifies the tool step.

-netName

Specifies the net name.

le::EMRCDecryptFile

Description

Encrypts or decrypts files using bfs lib.

Syntax

```
le::EMRCDecryptFile [-isDecryption <bool>] [-filePath <string>]
```

Returns

<string>: The encryption/decryption result.

Arguments

-isDecryption

Specifies whether to decrypt or encrypt file. Default is true.

-filePath

Specifies the file path.

le::EMRCEMResultToEV

Description

Converts EM results to the Error Viewer.

Syntax

```
le::EMRCEMResultToEV [-dspfFilePath <string>] [-emResultFilePath  
  <string>] [-scale <float>]
```

Returns

<string>: The converted results.

Arguments

-dspfFilePath

Specifies the dspf file path.

-emResultFilePath

Specifies the EM result file path.

-scale

Specifies the unit scale in the em_config file.

le::EMRCRevertLogFile

Description

Replaces the log Id with content.

Syntax

```
le::EMRCRevertLogFile [-filePath <string>]
```

Returns

<string>: The reverted log.

Arguments

-filePath

Specifies the file path.

le::EMRCXoutToDSPFFile

Description

Converts xout file to DSPF file.

Syntax

```
le::EMRCXoutToDSPFFile [-xoutFilePath <string>] [-layer_infoFilePath  
    <string>] [-configFilePath <string>]
```

Returns

<string>: The converted result.

Arguments

-xoutFilePath

Specifies the xout file path.

-layer_infoFilePath

Specifies the layer_info file path.

-configFilePath

Specifies the config file path.

le::enableDebugLogModule

Description

Enables a debug log module without GUI icon in the debug system.

Syntax

```
le::enableDebugLogModule -moduleName <string> -isEnabled <bool>
```

Returns

<string>: The result of the command: succeeded or failed.

Arguments

-moduleName

Specifies the name of the debug log module.

-isEnabled

Specifies whether to enable the log module.

le::evalViaParams

Description

Calculates via parameters from the given arguments.

Syntax

```
le::evalViaParams (-techLib <oaLib> | -techLibName <dmLibName> | -design  
<oaDesign>) [-cutSize <string> | -cutClass <string>] -layer1 <lpp>  
-layer2 <lpp> [-layer1Box <box>] [-layer2Box <box>] [-layer1Width  
<float>] [-layer2Width <float>] [-layer1Dir <enum>] [-layer2Dir <enum>]  
-overlapBox <box>
```

Returns

<oaViaParam>: The evaluated oaViaParam.

Arguments

-techLib

OA library where to query rules from its technology database.

-techLibName

Specifies the name of the OA library where to query rules from its technology database.

-design

OA design where to query rules from.

-layer1

Bottom layer of the via.

-layer2

Upper layer of the via.

-layer1Box

Box of layer1. Valid only when both layer1Box and layer2Box are specified.

-layer2Box

Box of layer2. Valid only when both layer1Box and layer2Box are specified.

-layer1Width

Width of layer1. Valid only when both layer1Box and layer2Box are specified.
Queried from the layer box if not specified.

`-layer2Width`

Width of layer2. Valid only when both layer1Box and layer2Box are specified.
Queried from the layer box if not specified.

`-layer1Dir`

Direction of layer1. Valid only when both layer1Box and layer2Box are specified.
Queried from the layer box if not specified.

`-layer2Dir`

Direction of layer2. Valid only when both layer1Box and layer2Box are specified.
Queried from the layer box if not specified.

`-overlapBox`

Overlaps the box of layer1 and layer2.

`-cutSize`

Specifies the cut size in a single value or pair value. For example: {0.5} or {0.2
0.3}.

`-cutClass`

Specifies the cutClass name.

le::exportResistanceQuery

Description

Export resistance query data to a file.

Syntax

```
le::exportResistanceQuery <scope: oaDesign> -filePath <string>
```

Arguments

`scope`

Design to be reported.

`-filePath`

Specify the file path to export query data to.

Returns

<string>: Success to export query data or not.

le::extractLayoutContext

extract specified objects to yaml in layout context

Syntax

```
le::extractLayoutContext <context: deContext> -path <string> [-presetLCV  
  <dmCellView | dmCellViewName>]
```

Returns

<none>

Arguments

context

Context of activation.

-path

Folder path to store the extracted info.

-presetLCV

LCV that store trackPcell preset.

le::extendHandleSelection

Description

Extends the handle selection to instances on the same chain.

Syntax

```
le::extendHandleSelection <objects: {<deFigure>} | {<deSelectionSet>}  
  | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>}  
  | {<oaVia>}} [-direction <enum>] [-neighborOnly <bool>] [-selectionSet  
  <bool>]
```

Arguments

objects

Instances whose selected stretch handles are extended to other instances.

-direction

Direction (left / right / all) to search for abutment partners ({ all | left | right },
default is: all)

Default: all.

`-neighborOnly`

Return only immediate neighbors. (Default is: false)

`-selectionSet`

All instances in the selection set. (Default is: false)

le::externalizeColor

Description

Checks the design specified and represents precolored shapes in the technology's color model. It can change layers or purposes or create recognition shapes depending on the color model.

Where a design is already open, the command creates a transaction on that design named *Externalize color*.

Note:

Use the `-purpose` argument only when the technology contains a purpose color model. The specified purpose will be used for shapes that must be moved to an uncolored purpose.

Syntax

```
le::externalizeColor (-design <oaDesign> | -libName <string>)  
[-anchorsOnly <bool>] [-purpose <string>]
```

Arguments

`-design`

Specifies a design that is currently open and editable.

`-libName`

Specifies the name of an open library that will be scanned for all maskLayout oaDesigns.

`-anchorsOnly`

By default only anchors end up colored in the foundry model. When false then all colored shapes are converted. Default is *true*.

`-purpose`

Only for purpose color models: specify the purpose to use for shapes residing on a colored purpose that must be moved to this uncolored purpose. Default is drawing.

Examples

The following example checks the design or multiple designs specified and represents pre-colored shapes in the technology color model. It can change layers or purposes or create recognition shapes depending on the color model.

```
# Externalize colors for library DesignLib
le::externalizeColor -libName DesignLib
```

le::fill

Fill figures

Syntax

```
le::fill (-points [<point>] | -objects <{<deFigure>} | {<deSelectionSet>}
| {<oaFig>}) (-cellName <string> | -master <oaDesign> | -fillIn
<{<deFigure>} | {<deSelectionSet>} | {<oaFig>}) -newViewName <string>
-tiling <bool> [-design <oaDesign>] [-libName <string>] [-viewName
<string>] [-distributed <bool>] [-spacing <string>] [-enclosure
<string>] [-orient <enum>] [-just <enum>] [-array <bool>] [-newCellView
<bool>] [-newLibName <string>] [-newCellName <string>] [-excludeLPPs
<string>] [-excludeHalo <string>] [-excludeDepth <unsigned int>]
```

Returns

{<oaFig>} : Collection of filled figures.

Arguments

-points

Point list of the fill region.

-objects

Collection of oaFig for extracting fill regions.

-design

Design where figures are filled with. (Default is: editing design)

-cellName

Cell name of the master cellview to fill in. This must be specified if -fillIn is not specified.

-master

Master of the cellview to fill in.

-fillIn

Collection of oaFig to fill in.

`-libName`

Library name of the master cellview to fill in. (Default is: library of -design)

`-viewName`

View name of the master cellview to fill in. (Default is: layout)

`-distributed`

Specifies whether to distribute the available space when -tiling false is specified.
(Default is: false)

`-spacing`

The spacing value between two consecutive filling objects. (Default is: 0)

`-enclosure`

The enclosure size to the area edge. (Default is: 0)

`-orient`

Orientation of the filler figure. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }, default is: R0)

`-just`

Justification of the filler figure with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }, default is: centerCenter)

`-array`

Specifies whether to fill area by array instance (arrayInst) objects. (Default is: true)

`-newCellView`

Specifies whether to create a new cell view for filling objects. (Default is: false)

`-newLibName`

Specifies the library name of the new cell view which collects the filling instances. (Default is: library of -design)

`-newCellName`

Specifies the cell name of the new cell view which collects the filling instances. (Default is: cell of -design)

`-newViewName`

Specifies the view name of the new cell view which collects the filling instances. (Default is: layout_fill)

`-excludeLPPs`

Specifies the list of excluded LPPs to avoid filling objects on the the regions.

`-excludeHalo`

Specifies the distance between filling objects and excluded regions. (Default is: 0)

`-excludeDepth`

Specifies the depth of excluded LPP shapes. (Default is: 0)

`-tiling`

Specifies whether to use the tiling method to fill the specified area. (Default is: false)

le::fixAcuteAngle

Description

Fixes acute angle.

Syntax

```
le::fixAcuteAngle <objects: oaDesign | {<deFigure>} | {<deSelectionSet>}  
| {<oaFig>} | {<oaShape>}> [-report <bool>] [-coordMark <bool>]  
[-fixConvex <bool>] [-patchConcave <bool>] [-aggressiveness <int>]  
[-circularPatch <bool>] [-circularSpacingAware <bool>]
```

Arguments

`objects`

List of objects or design to fix acute angle

`-report`

Shows the detailed report.

`-coordMark`

Puts a coordinate mark at fix failed corner.

`-fixConvex`

Sets to true to clip acute angle in convex part of polygon.

`-patchConcave`

Sets to true to patch reflex angle in concave part of polygon.

`-aggressiveness`

Specifies how aggressive to fix corners with violation. Larger value may fix more corner, but maybe slower and have unintended result.

`-circularPatch`

Patch circular and rectangular intersection with rectangle using aggressiveness.

`-circularSpacingAware`

Adjusts patch of circular and rectangular intersection by consider spacing to other side consider spacing to other side.

Returns

<none>

le::fixMinWidthShapes

Description

Fixes Min Width shapes.

Syntax

```
le::fixMinWidthShapes -design <oaDesign> -lpp <lpp> [-minWidth <float>]  
[-d2dMinWidth <float>]
```

Arguments

`-design`

Design in which to create the findMinWidthShapes.

`-lpp`

The LPP on which to create the findMinWidthShapes.

`-minWidth`

Specifies minimum width. (Default is: 0)

`-d2dMinWidth`

Specifies D2D region minimum width. (Default is: 0)

Returns

<none>

le::fixTrackSnapping

Description

Fixes specified off-track and other track-related mismatch issues where the objects have unlocked Route Status and Placement Status.

To check and report off-track route objects, see [le::checkTrackSnapping](#).

Syntax

```
le::fixTrackSnapping [<objects: {<deFigure>} | {<oaInst>} | {<oaShape>} |  
{<oaVia>} | {leOffTrackInfo} > ] [-orient <oaOrient>]
```

Arguments

`objects`

List of off-track route or instance objects to be fixed. You can specify individual objects, or the list of objects found by [le::checkTrackSnapping](#) (leOffTrackInfo).

`-orient`

Orientation of the region. Valid values: R0, R180, MX, and MY. For an existing track region already created in a design its orient is R0.

Returns

{<leOffTrackInfo>} : Collection of off-track or attribute-mismatch route and instance objects that could not be fixed

le::flatten

Description

Flattens instances and vias in the given region.

Syntax

```
le::flatten <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}  
[-levels <int>] [-connectivity <bool>] [-pcells <bool>] [-vias <bool>]  
[-pins <bool>] [-pinText <bool>] [-mpps <bool>] [-stripedWires <bool>]  
[-figGroups <bool>] [-hierarchyName <bool>] [-returnResult <bool>]  
[-scalarizeArrayInst <bool>]
```

Arguments

`objects`

List of objects to flatten.

`-levels`

Number of hierarchical levels to flatten. (Default is: 0)

`-connectivity`

Specifies whether to preserve original connectivity. (Default is: true)

`-pcells`

Specifies whether to preserve or flatten PCells. (Default is: true)

`-vias`

Specifies whether to preserve or flatten vias. (Default is: true)

`-pins`

Specifies whether to preserve instance pins and net terminals. (Default is: true)

`-pinText`

Specifies whether to preserve pin text. (Default is: true)

`-mpps`

Specifies whether to preserve or flatten MPPs. (Default is: true)

`-stripedWires`

Specifies whether to preserve or flatten striped wires. (Default is: true)

`-figGroups`

Specifies whether to preserve or flatten figure groups.

Default: true.

`-hierarchyName`

The option to take hierarchy name as prefix.

Default: true.

`-returnResult`

Specifies whether to return a collection of flattened objects.

Default: true.

`-scalarizeArrayInst`

Specifies whether to transform the array into a group of individual instances.

Default: false.

Errors

- Object list is empty.
- The collection of oaFigs or deFigures that were input are not from the same oaDesign.
- Number of levels is negative.
- oaVectorInst was supplied.
- oaVectorInstBit was supplied.

Examples

The following example flattens all objects and instances in the defined area.

```
le::flatten [de::getFigures {{-4 -4} {4 4} } -type rectangle  
            -design $oaDesign]
```

le::generateAbstract

Description

Generates an abstract view from the source cellView, additionally using the abstract generation parameters (option sets). This command is registered as a generator for layout views by default. If the design is not opened, then the abstract generator opens to read source data and then closes.

Syntax

```
le::generateAbstract [-src <oaDesign>] [-srcLibName <string>]  
[-srcCellName <string>] [-srcViewName <string>] [-srcCategoryName  
<string>] [-destLibName <string>] [-destCellName <string>]  
[-destViewName <string>] [-options <dbGenOptionSet>] [-force <bool>]  
[-batch <bool>] [-genCellNamePrefix <string>] [-group <string |  
xtJobGroup>] [-exitProc <string>]
```

Returns

<dmCellView> : New cell view

Arguments

-src

The source design.

-srcLibName

The library name of the source design.

`-srcCellName`

The cell name(s) of the source design.

`-srcViewName`

The view name of the source design.

`-srcCategoryName`

The category name of the source design.

`-ignoreCell`

The cell names to be ignored.

`-destLibName`

The library name of the destination design.

`-destCellName`

The cell name(s) of the destination design.

`-destViewName`

The view name of the destination design.

`-options`

The options set.

`-force`

Indicates if the user wishes to overwrite an existing design. The default value is **false**.

`-batch`

Specify to run abstract generation in batch mode. The default value is **false**.

`-genCellNamePrefix`

Specify cell name prefix.

`-group`

The job group under which this job is added (for batch mode).

`-exitProc`

Procedure to call once the job has exited. (for batch mode).

`-tcl`

Specify scripts to run in batch mode.

Examples

The following example generates an abstract view from the source design. The abstract view is generated in the library *Lib*, with cell name *Cell*, and view name of *abstract*.

```
le::generateAbstract -src $oaDesign -srcLibName designLib -srcCellName  
designCell -srcViewName layout -destLibName Lib -destCellName Cell  
-destViewName abstract
```

le::generateAntennaRunset

Description

Creates a runset from the given design without running the antenna extraction. The runset and the corresponding layer map file are generated in the output directory.

Syntax

```
le::generateAntennaRunset [-src <oaDesign>] [-srcLibName <string>]  
[-srcCellName <string>] [-srcViewName <string>] [-options  
<dbGenOptionSet>] [-allLayers <bool>]
```

Arguments

`-src`

The source design.

`-srcLibName`

The library name of the source design.

`-srcCellName`

The cell name of the source design.

`-srcViewName`

The view name of the source design.

`-options`

The options set object that contains option settings to use for abstract generation.

`-allLayers`

Use all related layers, such as poly, diff, metal, cut, erasing, and fin layers, to generate the runset.

Returns

<none>

Errors

- An invalid source cellview is given.
- No tool for antenna extraction is loaded.
- The output directory cannot be written.

Examples

The following example gets an icv runset for antenna with a full layer list.

```
le::generateAntennaRunset -src $topDesign -allLayers 1
```

le::generateCutMetal

Description

Generates metal and cut-metal shapes to prevent minimum spacing violations. Use the command when the spacing between metal shapes is less than the `minUncutEndToEndSpacing` value.

Syntax

```
le::generateCutMetal <lpp: lpp> -design <oaDesign> [-area <enum>]  
[-points [<point>]] [-dir <enum>] [-onTop <bool>] [-cutPurpose  
<string>] [-space <float>] [-eraseWidth <float>] [-eraseLength <float>]  
[-eraseSpace <float>] [-generateOnInstBoundary <bool>] [-boundaryLpp  
<lpp>] [-onlyShapeBelowCut <bool>] [-increment <bool>] [-autoMode  
<bool>] [-cutLPP <lpp>] [-findSingleMetal <bool>] [-enableHalfWidthMetal  
<bool>] [-blockagePurpose <string>] [-spacingLessThanCutWidth <bool>]  
[-connectedMetalToTop <bool>] [-extendBlockage <bool>] [-checkShort  
<bool>] [-generateOnPolyEndedSide <bool>] [-checkExistingCutMetal  
<bool>] [-fillMode <bool>] [-fillMinSpacing <float>]
```

Arguments

`lpp`

The target LPP which user need to handle.

`-design`

The design in which to activate the command.

`-area`

The area in which the command applies. ({ design | polygon | rectangle }, default: design)

`-points`

Point list for the rectangle or polygon area, based on which area type is specified.

`-dir`

The track direction of the target layer. Default is preferred routing direction (`{ H | V }`)

`-onTop`

Specifies if the generated shape is on the top level (0x). (Default: true)

`-cutPurpose`

Specifies the purpose for cut-metal shapes.

`-space`

Specifies the minimum spacing for violation checking.

`-eraseWidth`

Specifies the width of the erasing layer.

`-eraseLength`

Specifies the length of the erasing layer.

`-eraseSpace`

Specifies the minimum spacing of the erasing layer.

`-generateOnInstBoundary`

Specifies if generating cut shapes on instance boundary. (Default: true)

`-boundaryLpp`

Specifies the boundary LPP. If no LPP is specified, the boundary is calculated according to `leInstBoundary`.

`-onlyShapeBelowCut`

Specifies if only generates shapes below cut metal in the ICC Co-Design flow. (Default: false)

`-increment`

Specifies if generating cut metal shapes in incremental mode. (Default: false)

`-autoMode`

Specifies if generating cut metal shapes in auto mode. (Default: false)

`-cutLPP`

Specifies the cut LPP that below-cut shapes should be inserted.

`-findSingleMetal`

Specifies whether to find a single metal for below-cut shapes insertion. (Default: false)

`-enableHalfWidthMetal`

Enables half width generation when generating shapes below cut in the ICC design. (Default: false)

`-blockagePurpose`

Specifies the blockage purpose name when enabling half width generation. The shape below cut will be half-width when blockages shapes exist.

`-spacingLessThanCutWidth`

Specify to generate shape below cut when metal spacing is less than cut width. (Default is: true)

`-connectedMetalToTop`

Specifies whether to generate shapes below cut in the instance for the connected metal. (Default: false)

`-extendBlockage`

Specify to extend blockage when generate shape below cut. Metal layer with blockage purpose will be collected for inserting cut metal. (Default is: false)

`-checkShort`

Specify to check 0x net on the metal shape. Won't generate cut metal when 0x nets are the same. (Default is: false)

`-generateOnPolyEndedSide`

When generate cut metal on instance boundary, cut metal will be placed on poly-ended side (Default is: false)

`-checkExistingCutMetal`

When generate cut metal, don't need to insert again if cut metal exists nearby. (Default is: false)

`-fillMode`

Specify to use fill cell mode when generate cut metal. (Default is: false)

`-fillMinSpacing`

Specify the minimum spacing to do fill mode. (Default is: false)

Returns

{<oaFig>} : Collection of generated objects.

le::generateGuardRing

Description

Creates guard rings using the auto mode.

Syntax

```
le::generateGuardRing <objects: {<deFigure>} | {<deSelectionSet>} |  
{<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaShape>} | {<oaVia>}>  
[-autoConfig <bool>] [-perInstance <bool>] [-master <dmCellView |  
dmCellViewName | oaDesign>] [-style <enum>] [-autoStyle <enum>]  
[-spacing <string>] [-layerSpacing [<string>]] [-individualWrap  
<bool>] [-mergeDistance <float>] [-notch <float>] [-removeJog <float>]  
[-autoNotch <float>] [-params [<map>]] [-rings <enum>] [-evalCallbacks  
<bool>] [-net [<oaNet | string>]] [-rows <int>] [-bodyWidth <float>]  
[-metWidthEqBody <enum>] [-cornerCut <enum>] [-sameMetWidth <enum>]
```

Returns

{<oaInst>} : Collection of guard rings.

Arguments

objects

List of objects to be enclosed by generated guard ring.

-autoConfig

Specifies whether to enable the auto configuration mode. The default is false.

-perInstance

(For auto configuration mode only) Generates guard rings for each selected instance. The default is false.

-master

(Not for auto configuration mode) SuperMaster of the guard ring.

-style

(Not for auto configuration mode) Style to form guard ring. The value can be `conformal` or `rectangle` (default).

-autoStyle

(For auto configuration mode only) Style to form guard ring. The value can be `auto` (default), `conformal`, or `rectangle`.

`-spacing`

(Not for auto configuration mode) Specifies the minimum spacing between enclosed objects' boundary and the boundary of the generated guard ring. The default is 0.

`-layerSpacing`

(Not for auto configuration mode) Specifies the minimum spacing between enclosed objects' layer boundary and layer boundaries of the generated guard ring. Each item should have {guard ring LPP, object LPP, spacing value} format. For example: {{{M1 drawing} {DIFF drawing} 0.5}}.

`-individualWrap`

(Not for auto configuration mode) Generates guard rings for individual selected objects.

`-mergeDistance`

(Not for auto configuration mode) The distance to control how to group the individual objects. The default is 0.

`-notch`

(Not for auto configuration mode) Specifies the minimum notch value. Not for auto-configuration mode. The default is 0.

`-removeJog`

Specifies the minimum jog value to remove. The default is 0.

`-autoNotch`

(For auto configuration mode only) Specifies the minimum notch value instead of using the predefined notch value in the auto-configuration file.

`-params`

(Not for auto configuration mode) Parameters of PCell instance. Each item should have the format {parameter name, parameter value}.

`-rings`

(For auto configuration mode only) Number of guard rings to generate, up to the maximum definition count in auto configuration file. ({ 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | max }, default is: max)

`-evalCallbacks`

Specifies whether to trigger callbacks. The default is true.

`-net`

The net names or oaNet objects to guard rings which should be assigned.

`-rows`

(For auto configuration mode only) Cut rows of MCell guard ring.

`-bodyWidth`

(For auto configuration mode only) Body width of MCell guard ring.

`-metWidthEqBody`

(For auto configuration mode only) Sets the **Metal Width Equals Body Width** option by `auto` (default), `disable`, or `enable`.

`-cornerCut`

(For auto configuration mode only) Sets the **Generate Corner Cut** option by `auto` (default), `disable`, or `enable`.

`-sameMetWidth`

(For auto configuration mode only) Sets the **Same Metal Width** option. ({ `auto` | `disable` | `enable` }, default is: `auto`)

le::generateLabel

Description

Generates labels from net, terminal, or instance.

Syntax

```
le::generateLabel -design <oaDesign> [-fromNet <bool>] [-fromTerm <bool>]
[-fromInstance <bool>] [-area <enum>] [-lppMethod <enum>] [-mapFile
<string>] [-textType <enum>] [-attach <bool>] [-box <box>] [-points
[<point>]] [-point <point>] [-objects <{<deFigure>} | {<deSelectionSet>}
| {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>}
| {<oaVia>}}] [-topMetalOnly <bool>] [-useVirtualConnection <bool>]
[-generateDeviceNamePerGate <bool>] [-textLpp <lpp>] [-instTextLpp
<lpp>] [-just <enum>] [-orient <enum>] [-font <enum>] [-height <float>]
[-drafting <bool>] [-overbar <bool>] [-style <enum>] [-location <enum>]
[-inward <float>]
```

Returns

{<oaShape>}: Collection of label objects.

Arguments

`-design`

Design in which to generate labels.

`-fromNet`

Generates labels from net. (Default is: false)

`-formTerm`

Generates labels from terminal. (Default is: false)

`-fromInstance`

Generates labels from instance. (Default is: false)

`-area`

The area to generate labels. ({ design | point | polyline | region | selection | viewPort }, default is: design)

`-lppMethod`

LPP method. ({ inherit | map | specify | techLib }, default is: inherit)

`-mapFile`

Map file.

`-textType`

Text type. ({ attribute | text }, default is: attribute)

`-attach`

Attaches text to object. (Default is: true)

`-box`

The coordinates of bounding box in region mode.

`-points`

Point list in polyline mode.

`-point`

The location of a label origin in point mode.

`-objects`

The list of objects in selection mode.

`-topMetalOnly`

Uses the top metal for island only. (Default is: false)

`-useVirtualConnection`

Uses virtual connection. (Default is: false)

`-generateDeviceNamePerGate`

Generates device name per gate. (Default is: false)

`-textLpp`

Specifies a text LPP for labels generated on shapes.

`-instTextLpp`

Specifies a text LPP for labels generated on instances.

`-just`

Label justification related to origin.

Valid values: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter (default), lowerCenter, upperRight, centerRight, and lowerRight.

`-orient`

Justification of the text with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }, default is: centerCenter)

`-orient`

Orientation of the text. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }, default is: R0)

`-font`

Font style with which the text is displayed. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish }, default is: stick)

`-height`

Text height in user units. (Default is: 1)

`-drafting`

Prevents text mirroring or rotating. (Default is: true)

`-overbar`

Renders text with overbar. (Default is: false)

`-style`

Specifies whether to generate label with automatically determined label attributes. ({ auto | manual }, default is: manual)

`-location`

Specifies location which is used to locate the label origin on the object. ({ bottom | center | left | left_right | right | top | top_bottom }, default is: center)

`-inward`

Specifies the offset in user units to move the label origin inward when generating label with user-defined location. (Default is: 0)

Errors

- An invalid design is passed.
- An invalid text layer and purpose pair is provided.
- Negative value for height is specified.

le::generateLayout

Description

Generates layout from the given schematic.

Syntax

```
le::generateLayout -dest <dmCellView | dmCellViewName> -interface  
  <dbInterfaceSet> [-options <dbGenOptionSet>]
```

Arguments

`-dest`

The destination cellview.

`-interface`

The interface.

`-options`

The options value.

le::generateLayoutContext

generate specified objects from yaml to layout context

Syntax

```
le::generateLayoutContext <context: deContext> -path <string>  
  [-techMapping <string>] [-overwrite <bool>]
```

Returns

<none>

Syntax

```
le::generateLayoutContext <context: deContext> -path <string>  
[-techMapping <string>] [-overwrite <bool>] [-skipTrackPcell <bool>]
```

Returns

<none>

Arguments

context

Context of activation.

-path

Folder path to store the extracted info.

-techMapping

Technology mapping file.

-overwrite

Overwrite duplicate object.

-skipTrackPcell

skip extracting trackPcellRegion.

le::generatePGMarkers

Description

Generates PG markers.

Syntax

```
le::generatePGMarkers (-nets <{<oaNet>}> | -design <oaDesign>) [-layers  
<string>] [-routeTypeOnly <bool>]
```

Arguments

-nets

Collection of net objects.

-design

Design to search in.

-layers

Defines on which layers PG markers are generated.

`-routeTypeOnly`

Defines whether shapes should be filtered by route type for OA design.

Returns

<none>

le::generatePins

Generate Pins

Syntax

```
le::generatePins [<objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>}
| {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}]
-design <oaDesign> [-area <enum>] [-points [<point>]] [-fromText <bool>]
[-lppMethod <enum>] [-mapFile <string>] [-extractDepth <unsigned
int>] [-textDepth <unsigned int>] [-useVirtualConnection <bool>]
[-extractThroughVia <bool>] [-pinSize <string>] [-fromNet <bool>]
[-includePinFromInstance <bool>] [-shapeDepth <enum>] [-assignToShape
<bool>] [-purposeList <string>] [-excludeLayers <string>] [-topLayerNum
<int>] [-metalOnly <bool>] [-forLogicTermsOnly <bool>] [-autoPlacePin
<bool>] [-nets <string | {<oaNet>}>] [-createSquarePin <bool>]
[-ignoreNets <string | {<oaNet>}>] [-skipUserCreatedPins <bool>]
[-enableMultithread <bool>] [-createPinLabel <bool>] [-style <enum>]
[-textType <enum>] [-attach <bool>] [-followInstPinLabel <bool>] [-just
<enum>] [-orient <enum>] [-pinLabelLppMethod <enum>] [-pinLabelMapFile
<string>] [-pinLabelLpp <lpp>] [-font <enum>] [-height <float>]
[-drafting <bool>] [-overbar <bool>]
```

Returns

{<oaShape>} : Collection of pin figures.

Arguments

`objects`

List of selected objects.

`-design`

Design in which to generate pins.

`-area`

The area to generate pins. ({ design | polygon | rectangle | selection | viewport }, default is: design)

`-points`

Point list for the rectangle/polygon area, based on which area type is specified.

`-fromText`

Specify if generate pins from text or not. (Default is: true)

`-lppMethod`

Lpp method for generate pins from text. ({ map | sameLpp | techLib }, default is: sameLpp)

`-mapFile`

Map file for generate pins from text.

`-extractDepth`

The extract shape depth to generate pins from text. (Default is: 0)

`-textDepth`

The text depth to generate pins from text. (Default is: 0)

`-useVirtualConnection`

Specify if use virtual connection in generate pins. (Default is: true)

`-extractThroughVia`

Specify if extract shapes through via in generate pins. (Default is: false)

`-pinSize`

Specify the pin size for new shapes in generate pins.

`-fromNet`

Specify if generate pins from net or not. (Default is: false)

`-includePinFromInstance`

Specify if include pin from instance when generate pins from net. (Default is: false)

`-shapeDepth`

Specify shape depth when generate pins from net. ({ includeInstPin | instPinOnly | top })

`-assignToShape`

Specify if assign pin information to 0x shapes directly. (Default is: true)

`-purposeList`

Specify the purpose list for new shapes in generate pins. (Default is: pin drawing)

- `-excludeLayers`
Specify the exclude layers in generate pins.
- `-topLayerNum`
Specify the top layer number for extract pin shapes. < 1 means search all layers.
(Default is: -1)
- `-metalOnly`
Specify only metal layers are used when specifying top layer number to generate pin. (Default is: true)
- `-forLogicTermsOnly`
Specify if generate pins for logic terms only. (Default is: true)
- `-autoPlacePin`
Specify if enable auto place pin. (Default is: false)
- `-nets`
Net names or oaNets for generate pin from net.
- `-createSquarePin`
Specify if enable create square pin. (Default is: false)
- `-ignoreNets`
Net names or oaNets for generate pin from net to ignore net.
- `-skipUserCreatedPins`
Specify to skip nets with user created pins. (Default is: false)
- `-enableMultithread`
Specify to enable multithread to trace connection. (Default is: false)
- `-createPinLabel`
Specify to create pin label for generate pins from net. (Default is: false)
- `-style`
Specifies whether to create pin label with automatically determined label attributes. ({ auto | manual }, default is: auto)
- `-textType`
Specifies pin label's text type. ({ attribute | text }, default is: attribute)
- `-attach`
Specifies whether attach label to object. (Default is: true)

`-followInstPinLabel`

Specify to create pin label by following instance pins' label. (Default is: false)

`-just`

Justification of the pin label with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }, default is: centerCenter)

`-orient`

Orientation of the pin label. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }, default is: R0)

`-pinLabelLppMethod`

Specifies LPP method for pin label. ({ map | sameLpp | specify | techLib }, default is: sameLpp)

`-pinLabelMapFile`

Map file for generating pin labels.

`-pinLabelLpp`

Specifies a text LPP for pin labels.

`-font`

Font style with which the pin label is displayed. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish }, default is: stick)

`-height`

Height of the pin label in user units. (Default is: 1)

`-drafting`

Indicates whether the pin label should not be mirrored or rotated. (Default is: true)

`-overbar`

Indicates whether the pin label will be rendered with overbar. (Default is: false)

le::generateShapes

Description

Generates shapes by applying a Boolean operation to objects on the given input layers and purposes and places them on the output LPP. If you provide an ellipse as an input object, the command converts it to a polygon, using the `-numPoints` value as a number of points to use during conversion.

Syntax

```
le::generateShapes [<objects: {<deFigure>} | {<deSelectionSet>} |  
  {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |  
  {<oaVia>}] (-lpp2 <lpp> | -lpp2s [<lpp>]) [-design <oaDesign>] -lpp1  
  <lpp> -lpp <lpp> -operation <enum> [-levels <int>] [-numPoints <int>]  
  [-region [<point>]] [-pointsCollection <bool>]
```

Returns

dbCollection | {<oaShape>}: Collection of new shapes.

Arguments

objects

Collection of objects, based on which the shapes are generated.

-design

Design based on which the shapes are generated.

-lpp1

First layer and purpose pair. All objects from the input shape collection on this LPP are the first sub-set to the shape generation operation.

-lpp2

Second layer and purpose pair. All objects from the input shape collection on this LPP are the second sub-set to the shape generation operation.

-lpp2s

Second layer and purpose pair list. All objects from the input shape collection on this LPP are the second sub-set to the shape generation operation.

-lpp

Layer and purpose pair for the output shapes.

-operation

Operation which is performed. ({ and | andnot | or | xor })

-levels

Number of hierarchical levels beyond which to ignore shapes contained by instances. (Default: 0)

-numPoints

Number of vertices to convert an ellipse into a polygon.

`-region`

Point list of generate shape region.

`-pointsCollection`

Return the points collection instead of generate shapes. (Default: false)

Errors

- Not all shapes from provided list belong to given design.
- An invalid LPP is provided.

Examples

The following example generates the shape on three LPPs and performs an *and* operation.

```
set shapes [de::getSelected -design $oaDesign]
le::generateShapes [de::getSelected -design $oaDesign]
    -lpp "diff net" -lpp1 "diff drawing" -lpp2 "diff pin"
    -operation and
```

le::generateSoftAbstract

Description

Creates a soft abstract from the given design.

Syntax

```
le::generateSoftAbstract [-src <oaDesign>] [-srcLibName <string>]
    [-srcCellName <string>] [-srcViewName <string>] [-destLibName
    <string>] [-destCellName <string>] [-destViewName <string>] [-options
    <dbGenOptionSet>] [-force <bool>]
```

Returns

<dmCellView> : New cell view.

Arguments

`-src`

The source design.

`-srcLibName`

The library name of the source design.

`-srcCellName`

The cell name(s) of the source design.

`-srcViewName`

The view name of the source design.

`-destLibName`

The library name of the destination design.

`-destCellName`

The cell name(s) of the destination design.

`-destViewName`

The view name of the destination design.

`-options`

The options set.

`-force`

Overwrites an existing design. The default value is false.

le::genPinFigureShorts

Description

Gets pin figures' shorts.

Syntax

```
le::genPinFigureShorts -lpps <{<deTechLPP>}> -design <oaDesign> [-box  
[<point>]]
```

Returns

{<string>}: Short information.

Arguments

`-lpps`

List of LPPs for querying pin figures' shorts.

`-design`

Design based on which the pin figures' shorts are detected.

`-box`

The box region to use for querying pin figures' short.

le::getAbuttedInsts

Description

Returns a collection of abutted instances.

Syntax

```
le::getAbuttedInsts <objects: {<deFigure>} | {<deSelectionSet>} |  
  {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |  
  {<oaVia>}} [-direction <enum>] [-neighborOnly <bool>] [-sameSide <bool>]
```

Returns

{<oaInst>}: Collection of abutted instances.

Arguments

`objects`

Instances to search for abutment partners.

`-direction`

Direction to search for abutment partners. ({ all | left | right }, default is: all)

`-neighborOnly`

Returns only immediate neighbors. (Default is: false)

`-sameSide`

Traces all instances on the same side. (Default is: true)

Examples

Note that the Auto Abutment engine should be turned on for these examples.

The following example returns abutted neighbor instances to specified oalnst object.

```
set design [ed]  
set inst1 [le::createInst I1 -design $design -libName reference40nm  
  -cellName n_4t -viewName layout -origin {1 1}]  
set inst2 [le::createInst I2 -design $design -libName reference40nm  
  -cellName n_4t -viewName layout -origin {1.19 1}]  
set inst3 [le::createInst I3 -design $design -libName reference40nm  
  -cellName n_4t -viewName layout -origin {1.38 1}]  
set inst3 [le::createInst I4 -design $design -libName reference40nm  
  -cellName n_4t -viewName layout -origin {1.57 1}]  
db::foreach inst [le::getAbuttedInsts $inst2 -neighborOnly true] {  
  puts [db::getattr inst.name]  
}
```

The following example returns abutted instances to right side of the specified oalnst object.

```
db::foreach inst [le::getAbuttedInsts $inst2 -direction right] {  
    puts [db::getAttr inst.name]  
}
```

le::getAbuttedMPPs

Description

Returns a collection of abutted multipath paths (MPPs).

Syntax

```
le::getAbuttedMPPs -design <oaDesign>
```

Returns

<dbCollection>: List of abutted MPPs.

Arguments

-design

Design in which to look for the abutted MPPs.

Examples

The following example gets a list of abutted MPPs that exist in the specified design.

```
le::getAbuttedMPPs -design $oaDesign  
oa:0x17ede11c oa:0x17ede11e oa:0x17ede11f
```

le::getAbutterAbuttee

Description

Returns the abutter and abuttee pair.

Syntax

```
le::getAbutterAbuttee <design: oaDesign> -inst1 <string> -inst2 <string>
```

Returns

<dbCollection> : Abutter/Abuttee pair.

Arguments

design

Design to get the abutter and abuttee pair.

```
-inst1
    Instance 1.

-inst2
    Instance 2.
```

Examples

Note that the Auto Abutment engine should be turned on for these examples.

The following example returns list of Abutter/Abuttee pair. I2 is the abutter and I1 is the abuttee.

```
set design [ed]
set inst1 [le::createInst I1 -design $design -libName reference40nm
    -cellName n_4t -viewName layout -origin {1 1}]
set inst2 [le::createInst I2 -design $design -libName reference40nm
    -cellName n_4t -viewName layout -origin {1.19 1}]
puts [le::getAbutterAbuttee $design -inst1 I1 -inst2 I2]
>> {I2 I1}
```

le::getAllAbutterAbutteeInDes

Description

Returns all abutters and abuttees in the design.

Syntax

```
le::getAllAbutterAbutteeInDes <design: oaDesign>
```

Returns

<dbCollection> : List of abutted instance pairs in a given design.

Arguments

design

Design to get abutted instance pairs.

Examples

The following example returns a list of Abutter/Abuttee pair lists in the design.

```
set design [ed]
set inst1 [le::createInst I1 -design $design -libName reference40nm
    -cellName n_4t -viewName layout -origin {1 1}]
set inst2 [le::createInst I2 -design $design -libName reference40nm
    -cellName n_4t -viewName layout -origin {1.19 1}]
set inst3 [le::createInst I3 -design $design -libName reference40nm
    -cellName n_4t -viewName layout -origin {1.38 1}]
```

```
set inst3 [le::createInst I4 -design $design -libName reference40nm
-cellName n_4t -viewName layout -origin {1.57 1}]
puts [le::getAllAbutterAbutteeInDes $design]
>> {I2 I1} {I3 I2} {I4 I3}
```

le::getAreaAndPerimeter

Description

Computes area and perimeter of source and drain regions of passed oalnst device.

Syntax

```
le::getAreaAndPerimeter <instance: oaInst> [-perFinger <bool>]
[-includeGateEdges <bool>] [-depth <int>]
```

Arguments

instance

The oalnst handle.

-perFinger

Specifies whether to scale the area values per finger.

Default: false.

-includeGateEdges

Specifies whether to include gate edges in perimeter computation.

Default: false.

-depth

Specifies the number of hierarchical levels to traverse. A negative number indicates all levels.

le::getBoundaryTrackRegionDefs

Description

Collection of leBoundaryTrackRegionDef objects.

Syntax

```
le::getBoundaryTrackRegionDefs [<pattern: string>] -tech <oaTech>
[-filter <string>]
```

Returns

{<leBoundaryTrackRegionDef>} : Collection of leBoundaryTrackRegionDef objects.

Arguments

`pattern`

Regex pattern for name matching. (Default is: *)

`-tech`

Technology to search.

`-filter`

Standard attribute filter.

Examples

The following example returns boundary track region definitions specified in the technology.

```
set tech [oa::TechFind reference10]
set bndTrackRgion [le::getBoundaryTrackRegionDefs -tech $tech]
```

le::getCommonPathPoints

Description

Creates a list of violation points.

Syntax

```
le::getCommonPathPoints -source <deFigure> -targets <{{deFigure}}>
    -distance <float>
```

Returns

[<point>] : Point for first branch relative to targets.

Arguments

`-source`

Source object.

`-targets`

Target objects.

`-distance`

Distance from source.

le::getCoordinateMarks

Description

Obtains the coordinate marks for a specified design.

Syntax

```
le::getCoordinateMarks -design <oaDesign> [-filter <string>] [-type  
    <enum>]
```

Arguments

-design

Design in which to look for the coordinate marks.

-filter

Filter the marks. (Default is: .*)

-type

Coordinate mark type. Valid values: all, nonPersistent, persistent. (Default: persistent)

Returns

<leCoordinateMark>: A collection of coordinate marks.

Errors

- Design is not specified.
- Design is invalid or not open.

Examples

The following example gets all the coordinate marks that exist in the specified design.

```
le::getCoordinateMarks -design $oaDesign
```

le::getGateDistance

Description

Computes the average distance from gate edges to the edges of target layer-purpose pairs.

Syntax

```
le::getGateDistance <object: oaDesign | oaInst> [-rectangle <box>] -lpps  
    [<string>] -direction <enum> -type <enum> [-index <int>] [-halo <float>]
```

```
[-extend <float>] [-extend_direction <enum>] [-operation <enum>] [-split  
<bool>] [-corner <enum>] [-depth <int>] [-debug <bool>]
```

Returns

<float>: Computed average distance.

Arguments

object

The oalnst | oaDesign handle.

-rectangle

Specifies the Gate rectangle.

-lpps

Specifies target LPPs.

-direction

Specifies the direction of computation by bottom, left, left_bottom,
left_top, right, right_bottom, right_top, or top.

-type

Specifies the distance type by any, external, or internal.

-index

Specifies the distance index.

Default: 1.

-halo

Specifies the neighborhood range.

-extend

Specifies the extension value for gate edge.

-extend_direction

Specifies the direction of computation for extension by bottom, left, right, or
top.

-operation

Specifies the boolean operation across layers by and, not, or or.

-split

Specifies whether to return individual pairs of overlapping lengths and distance
values (true) instead of a weighted distance (false). See Example 4.

Default: false.

`-corner`

Specifies the corner type for computation by `concave` or `convex`.

`-depth`

Specifies the number of hierarchical levels to traverse. A negative number indicates all levels.

`-debug`

Specifies whether to enable internal debugging. Default is: false.

Examples

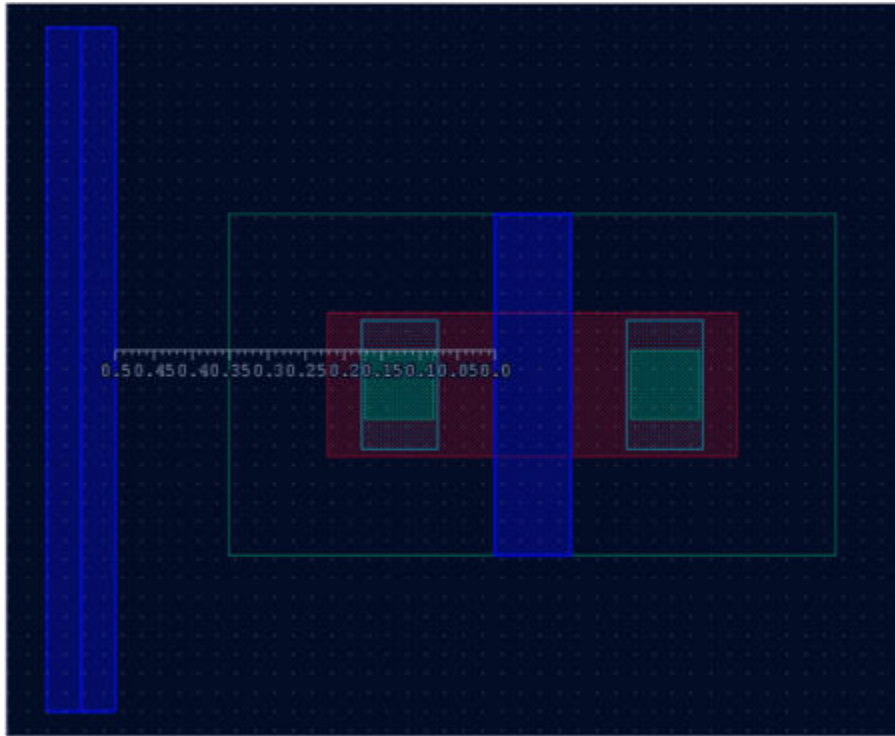
Examples are described in the following sections.

Example 1

This example shows how to compute the distance to the first {poly drawing} line to the left of the transistor.

```
le::getGateDistance $inst -lpps {poly} -direction left -type external  
-index 1
```

This example specifies only {poly} and not {poly drawing} for the LPP. If only the layer name is specified, it takes *drawing* as the purpose. The ruler in the following figure shows the computed distance.

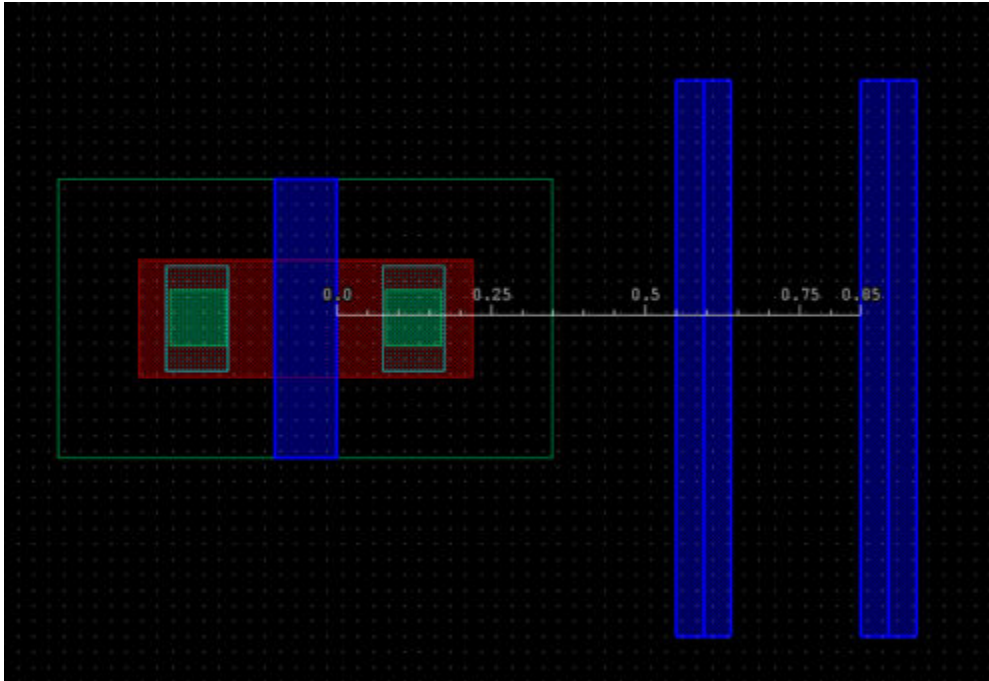


Example 2

This example shows how to compute the distance to the second poly of any purpose to the right of the transistor.

```
le::getGateDistance $inst -lpps {{poly *}} -direction right -type
external -index
```

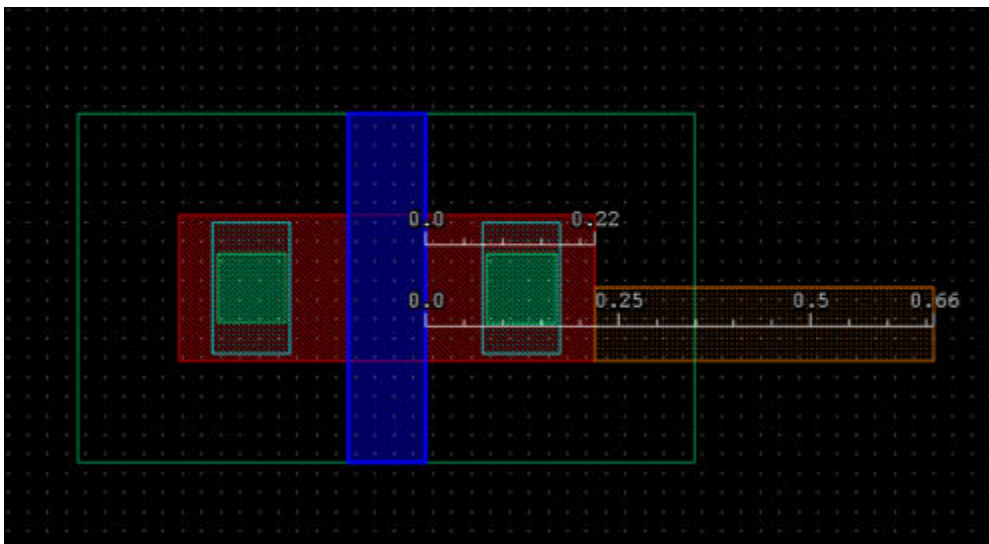
This example uses double curly braces because multiple LPPs can be specified as {{layer1 purpose1} {layer2 purpose2} ... }. For any LPP, if a purpose is not specified then it takes *drawing* as the purpose. The ruler in the following figure shows the computed distance.



Example 3

This example computes the distances to the first OD1 edge (shown in red) and to the second OD2 edge (shown in orange). The computed distances are weighted (by span) an average of the two distance rulers.

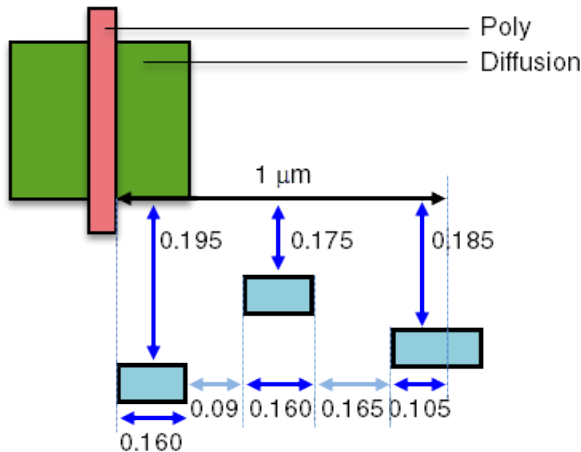
```
le::getGateDistance $inst -lpps {{OD1 *}} {OD2 *}} -direction right -type
internal -index 1
```



Example 4

This example computes the distance between the bottom gate edge and the three blue objects. The bottom gate edge is extended by 1 mm (-extend 1u) to the right (-extend_direction right).

```
le::getGateDistance $inst -lpps {poly} -extend 1u -extend_direction right
-type external -index 1 -split true
```



-split true returns the following:

```
{0.160 0.195} {0.09 5.0000} {0.160 0.175} {0.165 5.0000} {0.105 0.185}
```

Note that **0.09** and **0.165** are returned separately instead of reported as a sum of 0.255.

le::getGateRectangles

Description

Returns transistor instance gate geometry as a list of rectangles.

This command computes gate geometry as boolean AND of diffusion, and as poly silicon geometry. The command takes a transistor as `oaInst` and returns the individual finger geometry as a list of rectangles. The command does not look into the neighboring transistors or the interconnect geometry. The computation is done purely from the geometry of passed `oaInst`.

Syntax

```
le::getGateRectangles <instance: oaInst> [-depth <int>]
```

Returns

<string> : Computed gate rectangles.

Arguments

`instance`

The oalnst handle.

`-depth`

Specifies the number of hierarchical levels to traverse. A negative number indicates all levels.

le::getGlobalTrackRegions

Description

Retrieves leGlobalTrackRegion objects.

Syntax

```
le::getGlobalTrackRegions [<pattern: string>] -design <oaDesign> [-filter  
    <string>]
```

Returns

{<leGlobalTrackRegion>} : Collection of leGlobalTrackRegion objects.

Arguments

`pattern`

Regex pattern for name matching. The default is `*`.

`-design`

Container to search.

`-filter`

Standard attribute filter. The default is `<matches all>`.

Examples

The following example returns the global track region.

```
set design [ed]
set trackRegion [le::createTrack track -in $design \
    -offset 0 -trackType default]
set trackGroupDefinition [le::createTrackGroupDef \
    trackGroupDefinition -in $design \
    -direction horizontal -layer M1 -pitch {10 100} \
    -tracks $trackRegion]
set trackRegionDefinition [le::createTrackRegionDef \
    trackRegionDefinition -in $design \
    -groupDefs $trackGroupDefinition]
```

```
le::createGlobalTrackRegion globalTRackRegion \  
-design $design \  
-regionDef $trackRegionDefinition  
set globalTrackRegion [le::getGlobalTrackRegions globalTRackRegion  
-design $design]
```

le::getIncompleteNets

Description

Returns the collection of oaNet objects that either contain more than one island or that have instTerms that are bound to terminals that have no pins.

Syntax

```
le::getIncompleteNets <oaDesign | deContext> [-filter <string>]
```

Returns

<oaNet>: Collection of oaNet objects that either contain more than one island or that have instTerms that are bound to terminals that have no pins.

Arguments

oaDesign | deContext

Design or context to retrieve incomplete nets.

-filter

Net name filter expression.

Default: ""

Errors

- Input design or context does not exist.

Examples

The following example gets all the incomplete nets in the specified design.

```
le::getIncompleteNets $oaDesign
```

le::getInstanceBBox

Description

Gets bounding box (BBox) of the given instance.

Syntax

```
le::getInstanceBBox <instance: oaInst>
```

Returns

<box>: Instance BBox.

Arguments

instance

Instance to get the bounding box.

le::getIntersections

Description

Gets intersections of asked LPPs on specified query line.

Syntax

```
le::getIntersections <object: oaDesign | oaRef> -point <point> -direction  
<enum> -length <float> -lpp1 <lpp> [-operation <enum>] [-lpp2 <lpp>]  
[-halo <float>] [-depth <int>]
```

Returns

<string>: Computed intersection values.

Arguments

object

The oaRef | oaDesign handle.

-point

Specifies the starting point for query line.

-direction

Specifies the direction of query line. Valid values: bottom, left, right, and top.

-length

Specifies the length of query line.

-lpp1

Specifies the first LPP for query.

`-operation`

Specifies the Boolean operation across layers. Valid values: and, not, not_touch, or, and touch.

`-lpp2`

Specifies the second LPP for query.

`-halo`

Specifies the neighborhood range.

`-depth`

Specifies the number of hierarchical levels to traverse. A negative number indicates all levels.

`-stepSize`

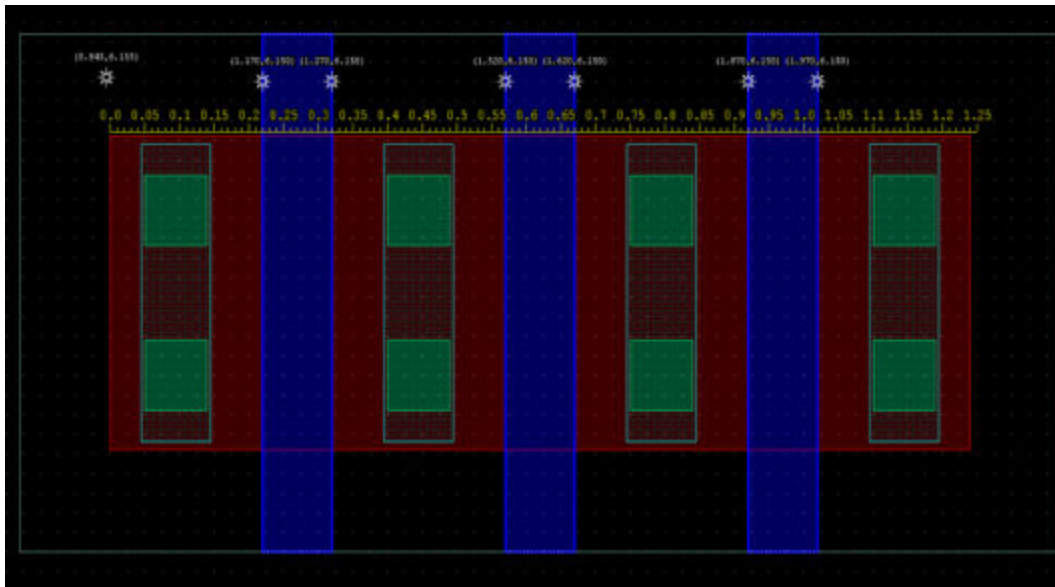
Option to specify step size in number of manufacturing grids for converting non orthogonal edges to orthogonal edges

Examples

In the following example, this command computes the intersections of Poly geometry on a defined query line.

```
le::getIntersections <object: oaDesign | oaRef> -point <point> -direction  
  <enum> -length <float> -lpp1 <lpp> [-operation <enum>] [-lpp2 <lpp>]  
  [-halo <float>] [-depth <int>] [-stepSize <int>]
```


In the following figure, coordinate marks show the intersecting points. The example command returns `{{1.170 1.270} {1.520 1.620} {1.870 1.970}}`.



le::getMPPDefs

Description

Returns a collection of MPP definitions.

Syntax

```
le::getMPPDefs [<pattern: string>] -in <oaDesign | oaTech> [-filter
<string>] [-local <bool>]
```

Returns

{<oaAppObject>}: Collection of MPP definition objects that match the search criteria.

Arguments

`pattern`

Globe-style pattern to match the object name.

Default: `*`.

`-in`

Technology or design from which to collect for the MPP objects.

`-filter`

Filter expression.

`-local`

Considers only the given technology.

Default: false.

Examples

The following example gets an innerRingMPP MPP definition from the specified technology.

```
le::getMPPDefs innerRingMPP -in $tech
```

le::getMPPs

Description

Returns all the leMPP objects from the given design that match the specified criteria.

Syntax

```
le::getMPPs [<pattern: string>] -design <oaDesign> [-filter <string>]
```

Returns

{<defigure>}: Collection of deFigure objects of type ({<leMPP>}).

Errors

An invalid design is passed.

Arguments

<string>

Glob-style pattern to match the leMPP object names.

Default: An empty string that results in “match all”.

`-design oaDesign`

Design in which to look for the leMPP objects.

`-filter`

Filter expression.

Examples

The following example gets all the MPP objects that exist in the specified design.

```
le::getMPPs -design $oaDesign
```

le::getNearestTracks

Description

Returns effective tracks which are nearest to a point.

Syntax

```
le::getNearestTracks <point: point> -design <oaDesign> -layer <string>  
[-numTracks <int>] [-color <string>] [-width <float>] [-direction  
<enum>] [-sigType <enum>] [-trackType <enum>] [-snapMode <enum>] [-orient  
<oaOrient>]
```

Returns

<dbCollection>: Collection of track information.

Arguments

point

The point at which should be nearest to the tracks.

-design

Design in which to find the tracks.

-layer

Specifies the exact layer.

-numTracks

Specifies the number of tracks. The default is 1.

-color

Specifies the exact color.

-width

Specifies the exact width.

-direction

Specifies the exact direction. Valid values are horizontal and vertical.

`-sigType`

Specifies the exact signal type. Valid values are all, analog, clock, ground, power, reset, scan, signal, tieHi, tieLo, and tieoff.

`-trackType`

Specifies the exact track type. Valid values are all, default, nonReserved, and reserved. The default is all.

`-snapMode`

Specifies the snapping mode. Valid values are down, nearest, and up. The default is nearest.

`-orient`

Orientation of the region. Valid values: R0, R180, MX, and MY. For an existing track region already created in a design its orient is R0.

le::getNetGroups

Description

Returns the collection of oaGroup objects with only oaNet and/or oaGroup members.

Syntax

```
le::getNetGroups <oaDesign | deContext> [-filter <string>]
```

Returns

<oaGroup>: Collection of group objects.

Arguments

`oaDesign | deContext`

Design or context to retrieve net groups.

`-filter`

Group name filter expression.

Default: ""

Errors

- Input design or context does not exist.

Examples

The following example gets all the incomplete nets in the specified design.

```
le::getNetGroups $oaDesign
```

le::getNetIslands

Description

Returns the list of connectivity islands for a specified net.

Syntax

```
le::getNetIslands -of <oaNet>
```

Arguments

-of

Net where you can get islands.

le::getPinFigureShorts

Description

Returns the short information of pin figures.

Syntax

```
le::getPinFigureShorts -lpps <{<deTechLPP>}> -design <oaDesign> [-box  
<box>]
```

Returns

<string> : Short information.

Arguments

-lpps

List of LPPs for querying the short information of pin figures.

-design

Design in which to activate the command for detecting pin figure shorts.

-box

The box region to use for querying the short information of pin figures.

le::getPseudoParallelNets

Description

Returns the list of nets that are connected in the schematic, but do not need to be physically connected in the layout, because those connections have equal voltage potential.

Syntax

```
le::getPseudoParallelNets -design <oaDesign>
```

Returns

List of pseudo parallel nets.

Arguments

`-design`

Design where you can get the pseudo parallel nets.

le::getRouteCorridors

Description

Returns a collection of leRouteCorridors objects from a given design that matches the pattern and the filter.

Syntax

```
le::getRouteCorridors [<pattern: string>] -of <oaDesign> [-filter  
  <string>]
```

Returns

{<leRouteCorridor>}: Collection of routing corridors in the design.

Arguments

`pattern`

Pattern for matching object names.

Default: `*`.

`-of`

Design from which routing corridors are fetched.

`-filter`

Attribute filter expression.

Default: `""`.

Errors

- Wrong design, or the design is not opened.
- Incorrect filter.

le::getRuleInfoByICV

Get Rule Information by ICV Live

Syntax

```
le::getRuleInfoByICV [-type <string>] [-runset <string>] [-oriRunset  
<string>] [-layerMap <string>] [-runDir <string>] [-arguments <string>]  
[-name <string>] [-queryDelimiter <string>] -design <oaDesign>  
[-controlVars <string>] [-background <bool>]
```

Returns

<none>

Arguments

-type

What type of information to get

-runset

Runset file to acquire information

-oriRunset

Original runset file to acquire information

-layerMap

Layer mapping file to acquire information

-runDir

Run directory

-arguments

The arguments to acquire rule information

-name

The rule name to acquire rule description

-queryDelimiter

The rule name to acquire rule delimiter

-design

Design in which to get all layer information

-controlVars

The control variables to configure runset

`-background`

Parse icv runset in the background

le::getStretchHandleCategorizedInfo

Description

Returns categorized stretch handle information of input objects.

Syntax

```
le::getStretchHandleCategorizedInfo <objects: {<defigure>} |
{<deSelectionSet>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} |
{<oaMarker>} | {<oaShape>} | {<oaVia>}}>
```

Returns

[<string>]: List of the stretch handle information.

Arguments

`objects`

Instances to get stretch handle information.

Examples

The following example returns the categorized stretch handle information of the reference40/n_4t/layout PCell instance.

```
{{diff {{{Diffusion Extension Right} CENTER_RIGHT D} {{Diffusion
Extension Left} CENTER_LEFT S}}} {m1 {{metalStretch LOWER_CENTER
D} {metalStretch UPPER_CENTER D} {metalStretch LOWER_CENTER S}
{metalStretch UPPER_CENTER S}}} {poly {{Length CENTER_RIGHT} {{Width per
Finger} UPPER_CENTER}}}} {{Bottom {{metalStretch m1 D} {metalStretch m1
S}}} {Left {{{Diffusion Extension Left} diff S}}} {Right {{{Diffusion
Extension Right} diff D} {Length poly}}} {Top {{metalStretch m1 D}
{metalStretch m1 S} {{Width per Finger} poly}}} {{D {{metalStretch
m1 LOWER_CENTER} {metalStretch m1 UPPER_CENTER} {{Diffusion Extension
Right} diff CENTER_RIGHT}}} {S {{metalStretch m1 LOWER_CENTER}
{metalStretch m1 UPPER_CENTER} {{Diffusion Extension Left} diff
CENTER_LEFT}}}}
```

```
set design [ed]
set inst [db::getNext [db::getInsts -of $design -filter \
{%libName == "reference40nm" && %cellName == "n_4t"}]]
le::getStretchHandleCategorizedInfo $inst
```

le::getStripedWires

Description

Returns a collection of striped wires.

Syntax

```
le::getStripedWires [<pattern: string>] -design <oaDesign> [-filter  
    <string>]
```

Returns

{<oaFigGroup>}: Collection of stripedWire objects that match the search criteria.

Arguments

pattern

Globe-style pattern to match the object name.

-design

Searches the specified design for the stripedWire object.

-filter

Uses the filter expression you provide in the argument.

Examples

The following example returns the collection of leStripedWire objects found in the design.

```
db::foreach sw [le::getStripedWires -design [ed]] {puts [db::getAttr  
    f.name]}
```

le::getSyncGroups

Description

Returns a collection of synchronous oaGroup objects for the design.

Syntax

```
le::getSyncGroups [<pattern: string>] -design <oaDesign> [-filter  
    <string>]
```

Returns

{<oaGroup>}: Collection of synchronous oaGroup objects found in the design.

Arguments

`pattern`

Glob style pattern to match synchronous group names.

`-design`

Design queried for synchronous oaGroup objects.

`-filter`

Filter expression.

Examples

The following example returns SyncGroup objects in the design.

```
db::foreach clone [le::getSyncGroups -design [ed]] {  
  db::foreach m [db::getattr clone.members] {  
    puts [db::getattr m.object.type]  
  }  
}
```

le::getTrackGroupDefs

Description

Returns a collection of leTrackGroupDefs matching the group defs and the filter.

Syntax

```
le::getTrackGroupDefs [<group: string>] -of <oaDesign | oaTech> [-filter  
  <string>]
```

Returns

{<leTrackGroupDef>}: Collection of leTrackGroupDefs matching the group defs and the filter.

Arguments

`group`

Regex pattern for name matching.

Default: *.

`-of`

Container to search.

`-filter`

Standard attribute filter. Leave the filter empty to match all objects.

Default:

Examples

The following example returns a track group definition.

```
set design [ed]
set trackGroupDef [le::createTrackGroupDef trackGroupDef \
-in $design -direction horizontal \
-layer M1 -pitch {1 2} -tracks $track]
set trGroupDef [le::getTrackGroupDefs trackGroupDef -of $design]
```

The following example returns all track group definitions specified in the design technology.

```
set tech [db::getattr design.tech]
set techTrackGroupDefs [le::getTrackGroupDefs -of $tech]
```

le::getTrackPatternDefs

Description

Returns a collection of leTrackPatternDefs matching the pattern and the filter.

Syntax

```
le::getTrackPatternDefs [<pattern: string>] -of <oaDesign | oaTech>
[-filter <string>]
```

Returns

{<leTrackPatternDef>} : Collection of leTrackPatternDefs matching the pattern and the filter.

Arguments

pattern

Globe style pattern for objects name matching. (Default is: *)

-of

Container to search.

-filter

Standard attribute filter. (Default is:)

Examples

The following example returns track pattern definitions.

```
set design [ed]
set trackPatternDef [le::createTrackPatternDef trackPatternDef \
```

```
-orientation horizontal -layer M1 -purpose drawing \  
-in $design -patternType nonReserved -trackPitch 0.1 \  
-width 0.04 -visible true -active true -color red]  
set trackPatternDef [le::getTrackPatternDefs trackPatternDef -from  
$design]
```

The following example returns track pattern definitions specified in the technology.

```
set tech [oa::TechFind reference10]  
set trackPatternDefs [le::getTrackPatternDefs trackPattern -from $tech]
```

le::getTrackPatterns

Description

Returns a collection of leTrackPatterns matching the pattern and the filter.

Syntax

```
le::getTrackPatterns [<pattern: string>] -of <oaDesign> [-filter  
<string>]
```

Returns

{<leTrackPattern>} : Collection of leTrackPatterns matching the pattern and the filter.

Arguments

pattern

Globe style pattern for objects name matching. (Default is: *)

-of

Design to search.

-filter

Standard attribute filter. (Default is:)

Examples

The following example returns track patterns.

```
le::getTrackPatterns  
set design [ed]  
set trackPatternDef [le::createTrackPatternDef trackPatternDef \  
-orientation horizontal -layer M1 -purpose drawing \  
-in $design -patternType nonReserved -trackPitch 0.1 \  
-width 0.04 -visible true -active true -color red]  
  
le::createTrackPattern trackPattern \  
-trackPatternDef $trackPatternDef \  
-design $design \  

```

```
-box {{0.0 0.0} {20.0 10.0}}  
set trackPattern [le::getTrackPatterns trackPattern -from $design]
```

le::getTrackRegions

Description

Returns a collection of leTrackRegions matching the name and the filter.

Syntax

```
le::getTrackRegions [<region: string>] -design <oaDesign> [-filter  
  <string>]
```

Returns

{<leTrackRegion>}: Collection of leTrackRegion objects matching the name and the filter.

Arguments

region

Regexp pattern for name matching.

Default: *.

-design

Design to search.

-filter

Standard attribute filter. Leave the filter empty to match all objects.

Default:

Examples

The following example returns the track region.

```
le::getTrackRegions  
set design [ed]  
set track [le::createTrack track -in $design -offset 0 \  
-trackType reserved -visible 1 -width 0.1 \  
-active 1 -color red]  
set trackGroupDef [le::createTrackGroupDef trackGroupDef \  
-in $design -direction horizontal \  
-layer M1 -pitch {1 2} -tracks $track]  
set trackRegionDef [le::createTrackRegionDef trackRegionDef \  
-in $design -groupDefs $trackGroupDef]  
  
le::createTrackRegion trackRegion -design $design \  
-regionDef $trackRegionDef \
```

```
-points {{0.0 -0.0} {0.0 100.0} {100.0 100.0} {100.0 -0.0}}  
set trackRegion [le::getTrackRegions trackRegion -design $design]
```

le::getTrackRegionDefs

Description

Returns a collection of leTrackRegionDefs matching the region definitions and the filter.

Syntax

```
le::getTrackRegionDefs [<regionDef: string>] -of <oaDesign | oaTech>  
[-filter <string>]
```

Returns

{<leTrackRegionDef>}: Collection of leTrackRegionDefs matching the region defs and the filter.

Arguments

regionDef

Regexp pattern for name matching.

Default: *.

-of

Container to search.

-filter

Standard attribute filter. Leave the filter empty to match all objects.

Default:

Examples

The following example returns track region definitions.

```
set design [ed]  
le::createTrack trackRed -in $design \  
-width 0.05 \  
-offset 0.0 \  
-trackType reserved \  
-visible true \  
-active true \  
-color red  
set trackGroupDef [le::createTrackGroupDef trackGroupDef \  
-in $design -direction horizontal \  
-layer M1 -pitch {1 2} -tracks $track]  
le::createTrackRegionDef trackRegionDef -in $design \  
-points {{0.0 -0.0} {0.0 100.0} {100.0 100.0} {100.0 -0.0}}
```

```
-groupDefs $trackGroupDef  
set regionDef [le::getTrackRegionDefs trackRegionDef -of $design]
```

The following example returns track region definitions specified in the design technology.

```
set tech [db::getattr design.tech]  
set regionDef [le::getTrackRegionDefs trackRegionDef -of $design]
```

le::getTracks

Description

Returns a collection of leTrack objects.

Syntax

```
le::getTracks [<pattern: string>] [-of <oaDesign | oaTech>]  
[-trackGroupDef <leTrackGroupDef>] [-trackRegion <leGlobalTrackRegion |  
oaAreaBoundary>] [-filter <string>]
```

Returns

{<leTrack>}: Collection of leTrack objects.

Arguments

pattern

Regexp pattern for name matching. (Default is: *)

-of

Container of the leTrack object to search.

-trackGroupDef

TrackGroup definition where leTrack is used

-trackRegion

leTrackRegion or leGlobalTrackRegion from which to return leTracks

-filter

Standard attribute filter. (Default is:)

Examples

The following example returns leTrack objects.

```
set design [ed]  
le::createTrack trackRed -in $design \  
-width 0.05 \  
-offset 0.0 \  
-trackType reserved \  

```



```
-visible true \  
-active true \  
-color red  
set track [le::getTracks trackRed -from $design]
```

le::getTracksInRegion

Description

Returns effective tracks in a region.

Syntax

```
le::getTracksInRegion <box: [point]> -design <oaDesign> -layer <string>  
[-color <string>] [-width <float>] [-direction <enum>] [-sigType <enum>]  
[-trackType <enum>] [-filter <string>] [-orient <oaOrient>]
```

Returns

<dbCollection> : Collection of track information.

Arguments

box

List of points

-design

Design in which should find the tracks.

-layer

The exact layer.

-color

The exact color.

-width

The exact width.

-direction

The exact direction. ({ horizontal | vertical })

-sigType

The exact signalType. ({ all | analog | clock | ground | power | reset | scan |
signal | tieHi | tieLo | tieoff })

-trackType

The exact trackType. ({ all | default | nonReserved | reserved }, default is: all)

`-filter`

Standard attribute filter. (Default is:)

`-orient`

Orientation of the region. Valid values: `R0`, `R180`, `MX`, and `MY`. For an existing track region already created in a design its orient is `R0`.

le::getUnboundGlobalTrackRegions

Description

Returns a collection of unbound `leGlobalTrackRegion` objects.

Syntax

```
le::getUnboundGlobalTrackRegions [<pattern: string>] -design <oaDesign>
[-filter <string>]
```

Returns

{<leGlobalTrackRegion>} : Collection of unbound `leGlobalTrackRegion` objects.

Arguments

`pattern`

Regexp pattern for name matching. (Default is: *)

`-design`

Design to search.

`-filter`

Standard attribute filter. (Default is:)

le::getUnboundTracks

Description

Retrieves a collection of unbound `leTrack` objects.

Syntax

```
le::getUnboundTracks [<pattern: string>] -design <oaDesign> [-filter
<string>]
```

Returns

{<leTrack>} : Collection of unbound `leTrack` objects.

Arguments

`pattern`

Regex pattern for name matching. (Default: *)

`-design`

Design to search.

`-filter`

Standard attribute filter. (Default is:)

le::getUnboundTrackRegions

Description

Returns a collection of unbound leTrackRegion objects.

Syntax

```
le::getUnboundTrackRegions [<pattern: string>] -design <oaDesign>  
[-filter <string>]
```

Arguments

`pattern`

Regex pattern for name matching. (Default: *)

`-design`

Design to search.

`-filter`

Standard attribute filter. (Default is:)

Returns

{<leTrackRegion>} : Collection of unbound leTrackRegion objects.

le::getUnnettedFigs

Description

Gets unnetted figures of the design.

Syntax

```
le::getUnnettedFigs -design <oaDesign>
```

Arguments

-design

Design to search in.

Returns

{<oaFig>} : Collection of unnetted pin figures.

le::highlightConnected

Description

Adds a collection of connected objects to the highlight set.

Syntax

```
le::highlightConnected <objects: {<deFigure>} | {<deSelectionSet>}  
| {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} |  
{<oaShape>} | {<oaVia>}> -context <deContext> [-optionSet  
<string>] [-name <string>] [-packet <string>] [-depth <enum>]  
[-highlightParents <bool>] [-propagatingShapesOnly <bool>] [-region  
<box>] [-initialColorPropagation <bool>] [-stopAt <enum>] [-point  
<point>]
```

Arguments

objects

Figures to initiate highlight connected.

-context

Context to initiate highlight.

-optionSet

Name of the dbGenOptionSet (setType highlightConnectedGenerator) to use.

Default: default.

-name

Create the name of highlight set or append to the name of highlight set .

Default: "".

-packet

Use the packet when creating deHighlightSet.

Default: inherited.

`-depth`

Depth to trace figures.

Valid Values: 0x | full | view.

Default: view.

`-highlightParents`

Trace parent designs.

Default: true.

`-propagatingShapesOnly`

Limit tracing to shapes that are propagating.

Default: true.

`-region`

Limit tracing to region.

`-initialColorPropagation`

Controls if color propagation performed prior to any erasing constraints applied.

Default: true.

`-stopAt`

Highlight connected stopping directive ({ branch | changeType | fullIsland | neighbor }). The default is fullIsland.

`-point`

Specifies coordinates of input point.

Returns

<deHighlightSet>: Highlight set which is created by `le::highlightConnected` command.

Examples

The following example highlights shapes that are physically connected to the selected object and then returns the highlighted object set.

```
set highlightSet [le::highlightConnected [de::getSelected -design [ed]] \
    -context [de::getActiveContext ] -depth full -packet highlight1]
# Returns highlighted figures from highlight set
set highlighted [de::getHighlighted -set $highlightSet]
```

le::importCommandBindings

Import interactive command key bindings.

Syntax

```
le::importCommandBindings -filePath <string>
```

Returns

<none>

Arguments

-filePath

Specify the import file.

le::importResistanceQuery

Description

Import resistance query data from a file. Input file should be in the XML format.

Syntax

```
le::importResistanceQuery <scope: oaDesign> -filePath <string> [-append  
  <bool>]
```

Arguments

scope

Design to be reported.

-filePath

Specify the file path to import query data from.

-append

Specify the mode to import query data:

- `true`: Append the imported data to existing data
- `false`: (Default) Replace existing data with imported data.

Returns

<string>: Success to import query data or not.

le::importVoltages

Description

Imports voltages.

Syntax

```
le::importVoltages [<context: deContext>] -config <dmCellView |  
dmCellViewName> [-overwrite <bool>]
```

Arguments

context

Context object of the import voltages operation.

-config

The configuration view for the import voltages operation.

-overwrite

Overwrites the existing voltages. The default is false.

le::isInSync

Description

Checks if the provided oaGroup is synchronized.

Syntax

```
le::isInSync <object: deFigure | oaFigGroup | oaGroup>
```

Returns

<bool>: If the provided oaGroup is synchronized.

Arguments

object

An oaGroup/oaFigGroup/deFigure object to check for synchronization.

le::merge

Description

Merges objects from the same lpp that overlap or abut. The command also converts ellipses to polygons during merging, using -numPoints value as a number of points to be created per ellipse during conversion.

Syntax

```
le::merge <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
[-numPoints <int>] [-preserveRouteType <bool>] [-preservePaths <bool>]  
[-preserveNets <bool>] [-preserveProperties <bool>]
```

Returns

{oaFig}: Collection of merged shapes.

Arguments

`objects`

List of objects to merge.

`-numPoints <int>`

Number of vertices to convert an ellipse into a polygon.

`-preserveRouteType`

Preserve route type if route types of source objects are the same. If true, the command does not merge shapes on different route type. If false, then the route type is not looked at, and resultant type (if mixed) is signalUser.

Default: true.

`-preservePaths`

Option to preserve path/pathSeg if the result object type is polygon. If true, then the command will not merge objects that cannot result in path or pathSeg. (Default is: false)

`-preserveNets`

Option to preserve or ignore nets. (Default is: true)

`-preserveProperties`

Option to preserve or ignore properties. (Default is: true)

Errors

- Not all provided shapes are from the same design.
- numPoints is less than 3.
- At least one of the objects became invalid because of merge (for example it would create a self-intersecting polygon).

Examples

The following example merges all shapes on the same LPPs in the specified design.

```
le::merge [de::getSelected -design $oaDesign]
```

le::move

Description

Moves one or more selected moveable objects by the specified vector. This command supports row, track, and wide alignment snapping by default.

Syntax

```
le::move <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
[-dx <float>] [-dy <float>] [-angleMode <enum>] [-anchor <point>]  
[-rotate <enum>] [-dest <oaDesign>] [-removeUnanchoredNet <bool>]
```

Returns

{<deFigure>} | {<oaFig>}: Collection of moved objects.

Arguments

`objects`

List of objects to move.

`-dx`

X projection of move vector.

Default: 0.

`-dy`

Y projection of move vector.

Default: 0.

`-angleMode`

Specify angle mode to move objects. ({ anyAngle | diagonal | orthogonal }, default is: anyAngle)

`-anchor`

Anchor point.

`-rotate`

Rotation to apply on objects. The available types are MX, MXR90, MY, MYR90, R0, R180, R270, and R90.

Default: R0.

`-dest`

Design to move objects into.

`-removeUnanchoredNet`

Option to remove unanchored net (Default is: false)

Errors

- Invalid design, design not opened for write.
- Not all provided shapes form the same design.
- Invalid command line argument.

Examples

In the following example, the first line creates a set of selected objects; the second line moves all those objects 1u for X-axes and 1u for Y-axes, and rotates them 90 degrees.

```
set objects [de::getSelected -design $oaDesign]
db::foreach object $objects {
  le::move $objects -dx 1 -dy 1 -anchor {0 0} -rotate R90
}
```

le::optimize

Description

Optimizes objects.

Syntax

```
le::optimize <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |
{<oaInst>} | {<oaShape>}>
```

Returns

{<oaFig>} : Collection of optimized objects.

Arguments

`objects`

Collection of objects to be optimized.

le::padLabel

Description

Generates pad labels.

Syntax

```
le::padLabel -design <oaDesign> -padLpp [<lpp>] -textLpp [<lpp>]  
[-textHeight <float>] [-textXOffset <string>] [-textYOffset <string>]  
[-padOperator1 <enum>] [-padOperator2 <enum>] [-padSize1 <float>]  
[-padSize2 <float>] [-insertFile [<string>]] [-outputFile [<string>]]  
[-levels <int>]
```

Arguments

-design

Design in which to create the padLabel.

-padLpp

LPP of the pad.

-textLpp

LPP of the text.

-textHeight

Text height.

-textXOffset

Text X-offset.

-textYOffset

Text Y-offset.

-padOperator1

Specifies the first search operator for the pad area. ({ equal | greater | greaterEqual | less | lessEqual })

-padOperator2

Specifies the second search operator for the pad area. ({ equal | greater | greaterEqual | less | lessEqual })

-padSize1

Specifies the first pad area size.

`-padSize2`

Specifies the second pad area size.

`-insertFile`

Insert file path.

`-outputFile`

Output file path.

`-levels`

Specifies the number of hierarchical levels for pad labels. (Default is: 0)

le::pasteLook

Description

Pastes look from selection.

Syntax

```
le::pasteLook -objects <{<defigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>
```

Returns

{<oaFig>}: Collection of pasted objects.

Arguments

`-objects`

List of objects to be pasted.

le::propagateConnectivity

Description

Propagate Connectivity.

Syntax

```
le::propagateConnectivity (-design <oaDesign> | -objects <{<oaInstTerm>}  
| {<oaNet>} | {<oaTerm>}}>) [-hierarchyMode <enum> | -depth <unsigned  
int>] [-mode <enum>]
```

Arguments

`-design`

Design object.

`-objects`

Collection of objects.

`-hierarchyMode`

Depth mode ({ full | top | view })

`-depth`

Depth

`-mode`

Connectivity propagation mode ({ netlist | shape })

Returns

<none>

le::refreshDensityResult

Description

Refreshes the density result.

Syntax

```
le::refreshDensityResult [<window: deCanvas | giWindow | int>] [-layer  
  <string>] [-group <string>] [-all <bool>]
```

Arguments

`window`

Window or canvas.

`-layer`

Layer name in layer mode to refresh density.

`-group`

Group name in the config file mode to refresh density.

`-all`

Specifies whether to refresh all results. This argument ignores the layer and group options if specified. The default is false.

le::regenerateMPP

Description

Regenerates MPPs. You can use this command when changing technology.

Syntax

```
le::regenerateMPP <objects: {<defigure>} | {<deSelectionSet>} | {<oaFig>}  
| {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>
```

Returns

{<oaFigGroup>}: Collection of regenerated MPPs.

Arguments

`objects`

List of MPPs to regenerate.

Examples

The following example regenerates MPPs in the design.

```
le::regenerateMPP [le::getMPPs -design $oaDesign]
```

le::remaster

Description

Remasters instances for color shift process. The command only changes instance's cell name to a new one according the cell mapping file. Library and view names remain unchanged.

Syntax

```
le::remaster <objects: oaDesign | {<oaInst>}> [-colorCellMap <string>]
```

Arguments

`objects`

Design or instances to be remastered.

`-colorCellMap`

Color cell mapping files.

Examples

```
le::remaster [ed] -colorCellMap MAP_FILES
```

```
le::remaster [db::getInsts -of [ed]] -colorCellMap MAP_FILES
```

le::removeFromFigGroup

Description

Removes objects from the figure group.

Syntax

```
le::removeFromFigGroup <objects: {<defigure>} | {<deSelectionSet>} |  
{<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |  
{<oaVia>}} [-depth <enum>]
```

Returns

{<oaFigGroup>}: Collection of OA figure group objects updated by the command.

Arguments

objects

Collection of objects to remove from the given figure group.

-depth

Specifies if a given objects contained by a nested oaFigGroup is removed only from the current group. ({allNested | current}).

Default: current.

Examples

The following example descends into FG__1 FigGroup and removes I1 instance from that FigGroup.

```
set design [ed]  
de::descendIntoGroup FG__1 -context [de::getActiveContext]  
le::removeFromFigGroup [db::getInsts I1 -of $design]
```

le::removeOverlappingCuts

Description

Removes overlapping cuts of guard rings.

Syntax

```
le::removeOverlappingCuts [<objects: {<deFigure>} | {<deSelectionSet>}  
| {<oaFig>} | {<oaFigGroup>} | {<oaInst>}}] -design <oaDesign> [-area  
<enum>] [-level <unsigned int>]
```

Arguments

`objects`

List of objects to consider for the overlapping cut removal of guard rings. This argument is must when area mode is set as selection.

`-design`

Design in which to create the remove overlapping cuts.

`-area`

Specifies the area mode by `design` (default) or `selection`.

`-level`

The hierarchical levels of guard rings to be considered. The default is 0.

le::removeUnboundInstanceTerms

Remove unbound instance terms

Syntax

```
le::removeUnboundInstanceTerms -design <oaDesign>
```

Returns

<none>

Arguments

`-design`

Design to remove unbound instance terms.

le::repeatCopy

Description

Repeats the last copy operation on the selected objects.

Syntax

```
le::repeatCopy [-window <giWindow | unsigned int> | -context <deContext>]
```

Arguments

`-window`

Window in which to repeat the copy.

`-context`

Context in which to repeat the copy.

Examples

The following example creates binding definition, so pressing Shift+> repeats the last copy operation.

```
gi::createBinding -event "Shift-Greater" \  
-command {le::repeatCopy -context %c} \  
-windowType leLayout
```

le::replaceVia

Description

Replaces the vias with the source cut numbers to the vias with the target cut numbers.

Syntax

```
le::replaceVia -design <oaDesign> [-sourceCutNum <int>] [-targetCutNum  
<int>]
```

Arguments

`-design`

Design to replace via.

`-sourceCutNum`

Source cut number. The default is 1.

`-targetCutNum`

Target cut number. The default is: 2.

Examples

The following example finds all standard vias with 2 cuts and increases cut numbers to 4.

```
set design [ed]  
le::replaceVia -design $design -sourceCutNum 2 -targetCutNum 4
```

le::reportCapacitance

Description

Reports the capacitance between specified nets, displays the summary (total capacitance) in the Console, and exports the summary and detailed data to a file (if specified).

Syntax

```
le::reportCapacitance <scope: oaDesign | {<oaNet>}> [-groupResult  
  <string>] [-filePath <string>] [-verbose <bool>] [-history <bool>]  
  [-blockingMode <bool>] [-exportToErrorViewer <bool>]
```

Arguments

`scope`

Net to be reported. Select multiple nets to extract parasitics in a single run.

`-groupResult`

Specifies the way to group capacitance results. ({Net | Layer | All}, default: Net)

`-filePath`

Specifies the CSV file path.

`-verbose`

Specifies whether to report polygons for point-to-point.

`-blockingMode`

Specifies whether to run capacitance reporter in blocking mode.

Set `-blockingMode` to `false` to run the Capacitance Reporter in the background mode, which supports both StarRC shared and non-shared modes. When running the reporter in the background mode, you can specify the output file name using the `-filePath` argument and set `-exportToErrorViewer` to `true` to see the results in the Error Viewer. If you do not specify these arguments, the report data can be viewed in the **History** section of the **Parasitic Reporter** dialog box.

Default: `true`

`-exportToErrorViewer`

Exports result to the Error Viewer. (Default is: `false`)

If you set the option to `true` when the EM Reporter and Parasitic Reporter run in the background, the violations are saved in the error viewer file. After the reporter job is complete, the command prints the file path of the error viewer file in the console.

`-history`

Saves the report history.

If you set the option to the following value when the EM Reporter and Parasitic Reporter run in the background:

- `true`: Adds information to history in the dialog.
- `false`: Adds information to history in the dialog only if you have not used the `-filePath` and `-exportToErrorViewer` option with the `-history` option.

Returns

<string> : The total capacitance and length of query net, detailed capacitance from the query net to other net .

Examples

The following example generates a capacitance report for net A, groups the data by net, and exports the report to a folder called output.

```
le::reportCapacitance [db::getNets "A" -of [oa::DesignFind ReportC  
regCase layout]] -filePath ./output -groupResult Net
```

The example code displays the following summary (total capacitance) in the Console:

```
{A} {E} 0.01875 18.17956  
{A} {D} 0.03845 37.28593  
{A} {Ground} 0.00941 9.12235  
{A} {C} 0.00589 5.7074  
{A} {B} 0.03063 29.70475
```

The following example shows how to generate a capacitance report for the ntail net in the background.

```
le::reportCapacitance [db::getNets ntail -of [ed]] -blockingMode false
```

le::reportCapacitanceFromEPA

Description

Reports capacitance information.

Syntax

```
le::reportCapacitanceFromEPA <scope: oaDesign | {<oaNet>}> [-groupResult  
<string>] [-filePath <string>] [-verbose <bool>] [-blockingMode <bool>]  
[-exportToErrorViewer <bool>]
```

Arguments

scope

Net to be reported.

`-groupResult`

Specifies the way to group capacitance results. ({ Net | Layer | All }, default: Net)

`-filePath`

Specifies the CSV file path.

`-verbose`

Specifies whether to report polygons for point-to-point.

`-blockingMode`

Specifies whether to run capacitance reporter in blocking mode.

`-exportToErrorViewer`

Exports result to the Error Viewer. (Default is: false)

Returns

<string> : The total capacitance and length of query net, detailed capacitance from the query net to other net.

le::reportCELayers

Description

Prints messages showing what layers are propagating in the specified design and how that set of layers are determined. Prints one message for layers used by the connectivity engine and one for layers used by highlight connected. The design must be a layout but it is not required to be extracted by CE, and the CE need not be enabled.

This command performs search to find a constraint group with a `validRoutingLayers` constraint. If found in the design, the first line of the report is:

Propagating layers are determined by the `validRoutingLayers` constraint in the *leCEConstraintGroup* in design *lib cell view*.

If the constraint is found in the tech, the first line is:

Propagating layers are determined by the `validRoutingLayers` constraint in the *leCEConstraintGroup* in the technology for *lib*, where the constraint group name, as well as *lib cell* and *view*, are replaced by the actual values in use.

In both the cases, the next line of the report is:

Layers are: *layr1 layr2 layr3 layr4 layr5...*

listing the names of all the layers in the constraint that do not have material type as "cut".

If no constraint is found by this search, then the report contains:

Propagating layers are layers that are not excluded by the
`leNonPropagatingLayerMaterials` preference. Layers are: layr1 layr2 layr3 layr4...

Where the layer names are replaced by the appropriate names for the design. The layers are all the `oaPhysicalLayers` in the tech for the design, that do not have a nonpropagating material type.

Syntax

```
le::reportCELayers <design: oaDesign>
```

Arguments

`design`

Design object.

Examples

The following example prints the following information:

Information: Connectivity Setup: Propagating layers are determined by the
`leCEConstraintGroups` preference - `validRoutingLayers` constraint in the constraint group
'leCEConstraintGroup' in the technology for 'testLib'.

Layers are: diff poly m1 m2 m3 m4 m5 m6 m7 m8 m9 m10.

Information: Highlight Connected Setup: Propagating layers are determined by the
`leHCConstraintGroups` preference - `validRoutingLayers` constraint in the constraint group
'leCEConstraintGroup' in the technology for 'testLib'.

Layers are: diff poly m1 m2 m3 m4 m5 m6 m7 m8 m9 m10.

```
set design [ed]
le::reportCELayers $design
```

le::reportConnectivityPoints

Reports the points where need to be created pins.

Syntax

```
le::reportConnectivityPoints -design <oaDesign> -file <string>
[-skipCutLayers <bool>]
```

Returns

<none>

Syntax

```
le::reportConnectivityPoints -design <oaDesign> -file <string>  
[-skipCutLayers <bool>] [-includeHierPath <bool>]
```

Returns

<none>

Arguments

-design

design object.

-file

Output file for reporting.

-skipCutLayers

Specifies whether we should report the cut layers object or not.

-includeHierPath

Specifies whether we should report the cut layers object or not.

le::reportEM

Description

Reports electro migration information.

Syntax

```
le::reportEM <scope: {<oaNet>}> [-compare <string>] [-filePath <string>]  
[-verbose <bool>]  
[-showLength <bool>] [-exportToErrorViewer <bool>] [-blockingMode <bool>]  
[-history <bool>]
```

Arguments

scope

Nets to be reported. Select multiple nets to extract parasitics in a single run.

-compare

Specifies the EM compare type by `current` (default) or `dimension`.

-filePath

Specifies the CSV file path.

`-verbose`

Specifies whether to report polygons for point-to-point.

`-showLength`

Specifies whether to show the EM length.

`-exportToErrorViewer`

Specifies whether to export the result to the Error Viewer. (Default is: false)

If you set the option to `true` when the EM Reporter and Parasitic Reporter run in the background, the violations are saved in the error viewer file. After the reporter job is complete, the command prints the file path of the error viewer file in the console.

`-blockingMode`

Specifies whether to run the EM Reporter in blocking mode.

Set `-blockingMode` to `false` to run the EM Reporter in background mode. When running the reporter in the background mode, you can specify the output file name using the `-filePath` argument and set `-exportToErrorViewer` to `true` to see the results in the Error Viewer. If you do not specify these arguments, the report data can be viewed in the **History** section of the **EM Reporter** dialog box.

Default: `true`

`-history`

Saves the report history.

If you set the option to the following value when the EM Reporter and Parasitic Reporter run in the background:

- `true`: Adds information to history in the dialog.
- `false`: Adds information to history in the dialog only if you have not used the `-filePath` and `-exportToErrorViewer` option with the `-history` option.

Returns

`<string>` : The total electromigration.

Examples

The following example reports EM of a net.

```
# Report EM for net CLK and store results into file
set net [db::getNets CLK -of $design]
le::reportEM $net -compare dimension -filePath ./emResults.csv \
  -verbose true -showLength true
```

le::reportEMBackground

Reports EMRC for background mode.

Syntax

```
le::reportEMBackground
```

Returns

<int> : Background EMRC Analysis

le::reportNets

Description

Reports the nets information within a design to the console.

Syntax

```
le::reportNets <objects: oaDesign | {<oaGroup>} | {<oaNet>}> [-ratio  
  <float>] [-length <float>] [-filePath <string>]
```

Returns

The report.

Arguments

`object`

Nets, net groups, or design to be reported.

`-ratio`

Specifies the routed / Manhattan length ratio. Nets with ratio over the value will be reported.

Default: 0.

`-length`

Specifies a length threshold. Nets with routed length over the value will be reported.

Default: 0.

`-filePath`

Specifies the .csv file path.

Examples

The following example reports information of all the nets in the design.

```
le::reportNets [db::getAttr editDesign -of\  
    [de::getActiveContext]]
```

The following example reports information of the CLK net in the design.

```
le::reportNets [db::getNets CLK -of [db::getAttr editDesign\  
    -of [de::getActiveContext]]]
```

le::reportNetsFromEPA

Description

Reports net information.

Syntax

```
le::reportNetsFromEPA <objects: oaDesign | {<oaGroup>} | {<oaNet>}>  
    [-ratio <float>] [-length <float>] [-filePath <string>]
```

Returns

<string> : The report.

Arguments

objects

Nets, net groups, or design to be reported.

-ratio

The routed / manhattan length ratio. Nets with ratio over the value are reported.
(Default is: 0)

-length

Length threshold. Nets with routed length over the value are reported. (Default
is: 0)

-filePath

Specifies the output file of CSV type.

le::reportRC

Reports resistance and capacitance information

Syntax

```
le::reportRC <scope: oaDesign | oaNet | {<oaNet>}> [-reportUsing  
  <string>] [-source <string>] [-destination <string>] [-sourceShorted  
  <bool>] [-destinationShorted <bool>] [-query <string>] [-queryAll  
  <bool>] [-depth <string>] [-region <string>] [-compare <bool>]  
  [-operatingPoint <enum>] [-spacingMode <enum>] [-densityMode  
  <enum>] [-density <float>] [-filePath <string>] [-blockingMode  
  <bool>] [-exportToErrorViewer <bool>] [-reportElmoreDelay <bool>]  
  [-reportLayerDistribution <bool>]
```

Returns

<string> : The total resistance and capacitance.

Arguments

scope

net or design to be reported.

-reportUsing

Specify the type of resistance reporter. Default: Selected Path.

-source

Specify source point and layer. It is ignored when -reportUsing is not 'Selected Path'

-destination

Specify destination point and layer. It is ignored when -reportUsing is not 'Selected Path'

-sourceShorted

Specify whether the source shorted. It is ignored when -reportUsing is not 'Selected Path'

-destinationShorted

Specify whether the destination shorted. It is ignored when -reportUsing is not 'Selected Path'

-query

Specify query set name. It is ignored when -reportUsing is not 'Selected Path'

-queryAll

Specify whether to report all query set. It is ignored when -reportUsing is not 'Selected Path'

`-depth`

Specify the depth of resistance reporter. Default is using preference 'leElectricalReporterViewDepth'

`-region`

Specify the region {{llx lly} {urx ury}} of resistance reporter. If it is not specified, the resgion is full design

`-compare`

Specify whether to report the status (pass/fail) when reporting type is 'All Constraints'

`-operatingPoint`

Specify the operating point. ({ maximum | minimum })

`-spacingMode`

Specify the spacing mode. ({ layout | minimum })

`-densityMode`

Specify the density mode. ({ layout | specified })

`-density`

Specify the density value.

`-filePath`

Specify csv file path.

`-blockingMode`

Specify whether to run resistance reporter in blocking mode.

`-exportToErrorViewer`

Specify whether to export result to error viewer (Default is: false)

`-reportElmoreDelay`

Specify whether to report elmore delay (Default is: false)

`-reportLayerDistribution`

Specify whether to report layer distributio (Default is: false)

le::reportResistance

Description

Reports the resistance information.

Syntax

```
le::reportResistance <scope: oaDesign | oaNet | {<oaNet>}> [-reportUsing  
  <string>] [-source <string>]  
[-destination <string>] [-sourceShorted <bool>] [-destinationShorted  
  <bool>] [-query <string>] [-queryAll <bool>]  
[-depth <string>] [-region <string>] [-compare <bool>] [-operatingPoint  
  <enum>] [-spacingMode <enum>]  
[-densityMode <enum>] [-density <float>] [-filePath <string>]  
[-blockingMode <bool>] [-exportToErrorViewer <bool>  
-reportElmoreDelay <bool> -reportLayerDistribution <bool>] [-history  
  <bool>]
```

Arguments

`scope`

Net or design to be reported. Select multiple nets to extract parasitics in a single run.

`-reportUsing`

Specifies the type of resistance reporter. The value must be {Selected Path, All Constraints, All Pins, All Terminals, All Instances, Selected Queries, Selected Queries All Nets}. Default: Selected Path.

`-source`

Specifies the source point and layer. It is ignored when `-reportUsing Selected Path` is not specified.

`-destination`

Specifies the destination point and layer. It is ignored when `-reportUsing Selected Path` is not specified.

`-sourceShorted`

Specifies whether the source is shorted. It is ignored when `-reportUsing Selected Path` is not specified.

`-destinationShorted`

Specifies whether the destination is shorted. It is ignored when `-reportUsing Selected Path` is not specified.

`-query`

Recalls the specified query set that is contained in the `leReportResistanceSet` Tcl preference and the queries stored in the Resistance Query Manager. This argument is ignored when `-reportUsing Selected Path` is not specified.

`-queryAll`

Reports all query sets that are contained in the `leReportResistanceSet` Tcl preference and the queries stored in the Resistance Query Manager. This argument is ignored when `-reportUsing Selected Path` is not specified.

`-depth`

Specifies the depth of resistance reporting. Valid values are:

- `0x`: The top-level only.
- `view`: The levels that are currently displayed.
- `full`: All levels of the hierarchy.

The `leElectricalReporterViewDepth` Tcl preference determines the default value.

`-region`

Specifies the region `{{llx lly} {urx ury}}` of resistance reporter. If it is not specified, the region is full design.

`-compare`

Specifies whether to report the status (pass or fail) when the reporting type is `All Constraints`.

When you run the EM Reporter and Parasitic Reporter in the background, the tool ignores the `-compare` option and prints a warning message if you have set the `-blockingMode` option to `false`.

`-operatingPoint`

Specifies the operating point. (`{ maximum | minimum }`)

`-spacingMode`

Specifies the spacing mode. (`{ layout | minimum }`)

`-densityMode`

Specifies the density mode. (`{ layout | specified }`)

`-density`

Specifies the density value.

`-filePath`

Specifies the CSV file path.

`-blockingMode`

Specifies whether to run the Resistance Reporter in blocking mode.

Set `-blockingMode` to `false` to run the Resistance Reporter in the background mode, which supports both StarRC shared and non-shared modes. When running the reporter in the background mode, you can specify the output file name using the `-filePath` argument and set `-exportToErrorViewer` to `true` to see the results in the Error Viewer. If you do not specify these arguments, the report data can be viewed in the **History** section of the **Parasitic Reporter** dialog box.

Default: `true`

`-exportToErrorViewer`

Specifies whether to export the result to the Error Viewer. Default: `false`.

If you set the option to `true` when the EM Reporter and Parasitic Reporter run in the background, the violations are saved in the error viewer file. After the reporter job is complete, the command prints the file path of the error viewer file in the console.

`-history`

Saves the report history.

If you set the option to the following value when the EM Reporter and Parasitic Reporter run in the background:

- `true`: Adds information to history in the dialog.
- `false`: Adds information to history in the dialog only if you have not used the `-filePath` and `-exportToErrorViewer` option with the `-history` option.

`-reportElmoreDelay`

Specifies whether to calculate and report elmore delay. Default: `false`.

`-reportLayerDistribution`

Specifies whether to report the layer distribution result to console. Default: `false`.

Returns

`<string>` : The total resistance.

Examples

The following example reports resistance for the entire query set (`leReportResistanceSet`) and the queries stored in the Resistance Query Manager.

```
le::reportResistance [ed] -queryAll 1
```

The following example reports resistance for a specific query that is stored in the query set (`leReportResistanceSet`) or the Resistance Query Manager.

```
le::reportResistance [ed] -query <query name>
```

The following example reports resistance for routes between the specified points (source and destination).

```
le::reportResistance <oaNet> -source <> -destination <>
```

The following example shows how to generate a resistance report for the ntail net in the background.

```
le::reportResistance [db::getNets ntail -of [ed]] -blockingMode false
le::reportResistance [db::getNets -of $des] -reportUsing "All Instances"
le::reportResistance [db::getNets -of $des] -source "inst A" -destination
    "inst B"
```

See Also

- [Reporting Resistance From Selected Queries](#)
- [Managing Resistance Report Queries](#)
- [Reporting and Checking Resistance](#)

le::reportResistanceFromEPA

Description

Reports the resistance information.

Syntax

```
le::reportResistanceFromEPA <scope: oaDesign | oaNet | {<oaNet>}>
[-reportUsing <string>] [-source <string>] [-destination <string>]
[-sourceShorted <bool>] [-destinationShorted <bool>] [-query <string>]
[-queryAll <bool>] [-depth <string>] [-region <string>] [-compare
<bool>] [-operatingPoint <enum>] [-spacingMode <enum>] [-densityMode
<enum>] [-density <float>] [-filePath <string>] [-blockingMode <bool>]
[-exportToErrorViewer <bool>]

le::reportResistanceFromEPA <scope: oaDesign | oaNet | {<oaNet>}>
[-reportUsing <string>] [-source <string>] [-destination <string>]
[-sourceShorted <bool>] [-destinationShorted <bool>] [-query <string>]
[-queryAll <bool>] [-depth <string>] [-region <string>] [-compare
<bool>] [-operatingPoint <enum>] [-spacingMode <enum>] [-densityMode
<enum>] [-density <float>] [-filePath <string>] [-blockingMode
<bool>] [-exportToErrorViewer <bool>] [-reportElmoreDelay <bool>]
[-reportLayerDistribution <bool>]
```

Arguments

`scope`

Net or design to be reported.

`-reportUsing`

Specifies the type of resistance reporter. The value must be {Selected Path, All Constraints, All Pins, All Terminals, Selected Queries, Selected Queries All Nets}. (Default: Selected Path)

`-source`

Specifies the source point and layer. It is ignored when `-reportUsing Selected Path` is not specified.

`-destination`

Specifies the destination point and layer. It is ignored when `-reportUsing Selected Path` is not specified.

`-sourceShorted`

Specifies whether the source is shorted. It is ignored when `-reportUsing Selected Path` is not specified.

`-destinationShorted`

Specifies whether the destination is shorted. It is ignored when `-reportUsing Selected Path` is not specified.

`-query`

Recalls the specified query set that is contained in the `leReportResistanceSet` Tcl preference. It is ignored when `-reportUsing Selected Path` is not specified.

`-queryAll`

Reports all query sets that are contained in the `leReportResistanceSet` Tcl preference. It is ignored when `-reportUsing Selected Path` is not specified.

`-depth`

Specifies the depth of resistance reporting. Valid values are:

- `0x`: The top-level only.
- `view`: The levels that are currently displayed.
- `full`: All levels of the hierarchy.

The `leElectricalReporterViewDepth` Tcl preference determines the default value.

`-region`

Specifies the region `{{llx lly} {urx ury}}` of resistance reporter. If it is not specified, the region is full design.

`-compare`

Specifies whether to report the status (pass or fail) when the reporting type is `All Constraints`.

`-operatingPoint`

Specifies the operating point. (`{ maximum | minimum }`)

`-spacingMode`

Specifies the spacing mode. (`{ layout | minimum }`)

`-densityMode`

Specifies the density mode. (`{ layout | specified }`)

`-density`

Specifies the density value.

`-filePath`

Specifies the CSV file path.

`-blockingMode`

Specifies whether to run resistance reporter in blocking mode.

`-exportToErrorViewer`

Specifies whether to export the result to the Error Viewer. (Default: false)

`-reportElmoreDelay`

Specifies whether to report elmore delay. (Default: false)

`-reportLayerDistribution`

Specifies whether to report layer distribution. (Default: false)

Returns

`<string>` : The total resistance.

le::reportShield

Description

Reports shield coverage for the selected nets or all nets in the design. The shield coverage is reported as percent per layer in the Console.

Syntax

```
le::reportShield <objects: oaDesign | {<oaGroup>} | {<oaNet>}> [-filePath  
  <string>]
```

Arguments

`objects`

Nets, net groups, or design to be reported.

`-filePath`

Specifies the .csv file path.

Examples

The following example reports shield coverage for the selected nets or all nets in a design.

```
# Report shield for net CLK
set net [db::getNets CLK -of $design]
le::reportShield $net
```

le::resetVoltageOverrides

Description

Removes override voltage values and return to the original source values.

Syntax

```
le::resetVoltageOverrides [<context: deContext>] [-nets [<oaBitNet |  
  string>]]
```

Arguments

`context`

Specifies the context object of the reset operation.

`-nets`

Collection of nets whose voltage values will be reset. If not specified, this command will operate on all nets.

le::reshape

Description

Reshapes a fully selected object by merging the existing shape with the "reshape" shape.

This command reshapes an object based on the point-list and shape type you provide (poly-line, polygon, or rectangle). It works only for path, pathSeg, polygon, rectangle, ellipse, boundary, and blockage.

If the shape you provide is path or pathSeg, then -type must be poly-line, the first point of -points must be on the path or pathSeg centerline, and the polyline formed by -points must not have a self-intersection. If the last point of -points is also on the centerline, then the command replaces the point-list section enclosed by the path's centerline with the new point-list you provide. Otherwise, based on the -clockwise parameter, the command replaces one part of the path's center-line with the point-list you provide.

If the shape you provide is a polygon, rectangle, ellipse, boundary, or blockage, and -type is poly-line, then the first and the last points of -points must be on the boundary of the shape. In this case, the command replaces the section enclosed by the point-list on the object's boundary with the new point-list you provide.

If the shape you provide is a polygon, rectangle, ellipse, boundary, or blockage, and -type is polygon or rectangle, then the new shape is formed by merging the existing shape with the reshape shape.

As a result of reshaping an ellipse, the ellipse will be converted to polygon, using -numPoints argument's value as a number of points to use.

If the object you pass is oaFigGroup, then it must correspond to leMPP. You cannot pass any other group object to this command.

Syntax

```
le::reshape <object: defigure | oaFig> -type <enum> -points [<point>]
[-clockwise <bool>] [-op <enum>] [-numPoints <int>]
```

Returns

Collection of reshaped objects ({<oaFig>}).

Arguments

`object`

Object to reshape.

`-type <enShapeType>`

Shape type for point list.

`-points {<point>}`

Point list for object, based on which a new shape is formed.

`-clockwise <bool>`

Portion of the path (relative to direction from first to second points from point list) to be replaced.

Default: true.

`-op <enum>`

Reshape mode used when shape is a polygon or rectangle.

Default: OR.

`-numPoints <int>`

Number of vertices to convert an ellipsis into a polygon.

Default: leNumPoints.

Errors

- Provided shape is not supported for reshaping (supported shapes are path, pathSeg, polygon, rectangle, ellipse, boundary, blockage).
- Shape is path or pathSeg, but type is not polyline.
- Shape is path or pathSeg, but first point is not on the center line.
- Shape is polygon, rectangle, ellipse, blockage or boundary, type is polyline, but first or/and the last point is not on the segment or vertex.
- Shape is ellipse but numPoints is less than 3.
- Reshape pointlist for pathseg object does not create orthogonal / diagonal pathSegs.
- oaFigGroup is passed which does not represent the leMPP object.

Examples

In the following example, the first line defines a set of points describing a box shape; the second line defines a rectangle using the defined set of points; the third line reshapes the rectangle using a different point list.

```
set box [list {10.0 10.0} {15.0 15.0}]
set rect [le::createRectangle $box -design $oaDesign -lpp "m1 drawing"]
le::reshape $rect -type rectangle -points [list {13.0 13.0} {18.0 20.0}]
```

le::resize

Description

Resizes (enlarges or reduces) a set of fully selected shape objects. As a result, objects can be destroyed. As a result of operation, no merging will happen, even if objects will overlap.

Syntax

```
le::resize <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}  
[-sizeOperation <enum>] [-size <float>] [-left <float>] [-right <float>]  
[-top <float>] [-bottom <float>] [-result <enum>]
```

Returns

<oaFig> | <deFigure> : Collection of resized objects.

Arguments

`objects`

List of objects to resize.

`-sizeOperation`

Size operation which is performed. ({ anisotropic | isotropic })

The default is isotropic.

`-size`

Resize uniform value for shape objects. If the value is positive, the object is enlarged. If the value is negative, the object is reduced.

`-left`

Resize value for the left edge.

`-right`

Resize value for the right edge.

`-top`

Resize value for the top edge.

`-bottom`

Resize value for the bottom edge.

`-result`

Result operation which is performed. ({ newWithActiveLPP | replace | replaceWithActiveLPP })

The default is replace.

Errors

- Not all provided shapes are from the same design.
- At least one of the objects would become invalid because of the resize (for example it would create a self-intersecting path object).

Examples

In the following example, the first line defines a set of points describing a box shape; the second line defines a rectangle using the defined set of points; the third line resizes the rectangle using the specified value.

```
set box [list {10.0 10.0} {15.0 15.0}]
set rect [le::createRectangle $box -design $oaDesign -lpp "m1 drawing"]
le::resize $rect -size 0.1
```

le::rotate

Rotate

Syntax

```
le::rotate <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}>
(-angle <float> | -flip <enum>) [-origin <point>] [-numPoints <int>]
[-separate <bool>] [-just <enum>]
```

Returns

{<oaFig>} : Rotated objects.

Arguments

`objects`

List of objects to be rotated.

`-angle`

Angle of rotation.

`-flip`

Mirror type of rotation. ({ horizontal | vertical })

`-origin`

Location of the origin of rotation.

`-numPoints`

Number of vertices to convert an ellipse into a polygon.

`-separate`

Flag to turn on/off rotating objects separately (Default is: false)

`-just`

Justification of the rotation with respect to the origin. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | origin | upperCenter | upperLeft | upperRight }, default is: origin)

le::scalarizeArrayInst

Description

Converts the array instances to a collection of scalar instances.

Syntax

```
le::scalarizeArrayInst <objects: {<defigure>} | {<deSelectionSet>} |  
{<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |  
{<oaVia>}}>
```

Returns

{<oaInst>}: Collection of instances after scalarization.

Arguments

`objects`

Collection of array instances to scalarize.

Examples

The following example creates scalar instances from the specified arrayInst object.

```
set design [ed]  
# Create array inst  
set arrayInst [le::createInst -design $design -libName reference40nm  
-cellName n_4t -viewName layout -origin {1 1} -cols 3 -rows 5]  
le::scalarizeArrayInst $arrayInst
```

le::selectStretchHandle

Description

Selects certain stretch handles on specified objects.

Syntax

```
le::selectStretchHandle <objects: {<deFigure>} | {<deSelectionSet>} |  
{<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>}}>
```

```
| {<oaVia>}> [-lpp <string>] [-paramName <string>] -location <enum>  
[-customLocationName <string>]
```

Arguments

objects

Instances on which stretch handles are selected.

-lpp

LPP of the shape.

-paramName

Parameter name.

-location

Location name. ({ all | bottom | center | center_center | center_left | center_right
| custom | left | lower_center | lower_left | lower_right | right | top | upper_center |
upper_left | upper_right })

-customLocationName

Custom location name.

Examples

The following example selects stretch handles on specified instance objects.

```
# Select all stretch handles of an instance I1  
set inst [db::getInsts I1 -of $design]  
le::selectStretchHandle $inst -location all
```

le::setDRDOption

Description

Sets an option for DRD Visual and Assist. Allows DRD options to be configured from the console/script. These options only apply to DRD Visual, Assist, and On Modification modes.

Syntax

```
le::setDRDOption [-interactiveTimeLimit [<int>]] [-commitTimeLimit  
[<int>]] [-outputMarkerInfo [<bool>]]
```

Arguments

-interactiveTimeLimit

Option to set time limit for DRD Visual and Assist (1 integer in milli seconds).

`-commitTimeLimit`

Option to set time limit for DRD OnModification (1 integer in milli seconds).

`-outputMarkerInfo`

Option to set the flag to output marker information for DRD Visual and Assist.

Examples

The following example allows DRD options to be configured from the console or script. These options only apply to DRD **Visual**, **Assist** and **On Modification** modes.

```
# Set DRD options for DRD Visual and Assist, interactive and commit
# time limits to 300ms, and allow to output marker info
le::setDRDOption -interactiveTimeLimit 300 -commitTimeLimit 300 \
    -outputMarkerInfo true
```

le::setNetAnchor

Description

Sets anchors to pin figures.

Syntax

```
le::setNetAnchor <collection: {<defigure>} | {<oaShape>} | {<oaVia>}>
    -value <bool>
```

Arguments

`collection`

The collection of oaPinFig or deFigures for the command to operate on.

`-value`

Value to apply to the anchor.

Examples

The following example sets anchor of the net of the shape to true. The shape must already have a net name.

```
set box [list {10.0 10.0} {15.0 15.0}]
set rect [le::createRectangle $box -design $oaDesign \
    -lpp "m1 drawing" -net "A"]
le::setNetAnchor $rect -value true
```

le::showAlignAssistOptions

Description

Shows the **Align Assist Options** dialog box. The dialog box is raised if it is already opened for the given window, and the existing object returned.

Syntax

```
le::showAlignAssistOptions
```

Returns

<none>

le::showCommandBindings

Displays the interactive command key bindings.

Syntax

```
le::showCommandBindings [-parent <giWindow | int>]
```

Returns

<none>

Arguments

-parent

Window(id) of layout editor window to show command binding

le::showCreateTrack

Description

Shows the **Create Track** dialog box.

Syntax

```
le::showCreateTrack [-parent <giWindow | int>]
```

Returns

<giDialog> : The created or existing **Create Track** dialog box.

Arguments

`-parent`

The parent window.

le::showCreateTrackGroupDefinition

Description

Shows the **Create Track Group Definition** dialog box.

Syntax

```
le::showCreateTrackGroupDefinition [-parent <giWindow | int>]
```

Returns

<giDialog> : The created or existing **Create Track Group Definition** dialog box.

Arguments

`-parent`

The parent window.

le::showCreateTrackRegionDefinition

Description

Shows the **Create Track Region Definition** dialog box.

Syntax

```
le::showCreateTrackRegionDefinition [-parent <giWindow | int>]
```

Returns

<giDialog> : The created or existing **Create Track Region Definition** dialog box.

Arguments

`-parent`

The parent window.

le::showDefineTerminals

Description

Brings up the **Define Terminals** dialog box, which lets you create or modify terminals for nets, as well as modify their attributes.

Syntax

```
le::showDefineTerminals [-parent <giWindow | int>]
```

Returns

The **Define Terminals** dialog box (*giDialog*).

Arguments

`-parent`

The parent window for the **Define Terminals** dialog box.

Default: active editor window.

Errors

- An invalid window object is passed.
- There is no design opened in specified window.
- Edit Design of the specified window is not of type MaskLayout.

le::showDesignOptions

Description

Shows the **Layout Design Options** dialog box. If the dialog box is already opened for the given window, it will be raised and the existing object returned. Otherwise, the existing dialog box will be closed before reopening for this design.

Syntax

```
le::showDesignOptions [-parent <giWindow | int>]
```

Returns

Newly opened or already opened dialog box (*giDialog*).

Arguments

`-parent <giWindow | int>`

Parent window containing the design for which the options will be edited.

Default: Last focused layout window.

Errors

- Invalid window.
- No active editor window.

le::showDRDOptions

Description

Shows the DRD Options.

Syntax

```
le::showDRDOptions [-parent <giWindow | int>]
```

Returns

<giDialog>: Dialog containing the DRD options.

Arguments

-parent

The parent window.

le::showDRDRuleSelection

Description

Displays the DRD Rule Selection dialog box.

Syntax

```
le::showDRDRuleSelection [-parent <giWindow | int>]
```

Returns

<giDialog> : Dialog containing DRD options

Arguments

-parent

The parent window.

le::showEditBlackBox

Description

Shows the black box editor dialog for the specified cellview or schematic instance.

Syntax

```
le::showEditBlackBox <target: defigure | dmCellView | oaInst> [-parent  
    <giDialog | giWindow | int>]
```

Returns

The *giDialog* dialog box.

Arguments

target

Either the cellview to edit or the schematic instance used to find the associated black box cellview.

-parent

The parent window or dialog box.

le::showEditFCBlackBox

Description

Shows the black box editor dialog box for the specified cellview or schematic instance.

Syntax

```
le::showEditFCBlackBox <target: defigure | oaInst> [-parent <giDialog |  
    giWindow | int>]
```

Returns

<giDialog> : The black box editor dialog box.

Arguments

target

The schematic instance used to find the associated black box cellview.

-parent

The parent dialog/window.

le::showEditICC2BlackBox

Description

Shows the black box editor dialog for the specified cellview or schematic instance.

Syntax

```
le::showEditICC2BlackBox <target: deFigure | oaInst> [-parent <giDialog |  
giWindow | int>]
```

Returns

<giDialog> :

Arguments

target

The schematic instance used to find the associated black box cellview.

-parent

The parent dialog/window.

le::showEditTrack

Description

Shows the **Edit Track** dialog box.

Syntax

```
le::showEditTrack [-parent <giWindow | int>]
```

Returns

<giDialog> : The created or existing **Edit Track** dialog box.

Arguments

-parent

The parent window.

le::showEditTrackGroupDefinition

Description

Shows the **Edit Track Group Definition** dialog box.

Syntax

```
le::showEditTrackGroupDefinition [-parent <giWindow | int>]
```

Returns

<giDialog> : The created or existing **Edit Track Group Definition** dialog box.

Arguments

-parent

The parent window.

le::showEditTrackRegion

Description

Shows the **Edit Track Region** dialog box.

Syntax

```
le::showEditTrackRegion [-parent <giWindow | int>]
```

Returns

<giDialog> : The created or existing **Edit Track Region** dialog box.

Arguments

-parent

The parent window.

le::showElectricalOptions

Description

Shows the electrical options dialog box.

Syntax

```
le::showElectricalOptions [-window <unsigned int>] [-parasiticOnly  
  <bool>]
```

Returns

<giDialog> : Electrical options dialog box.

Arguments

`-window`

Specifies the id of the window which this dialog is from. The default is 0.

`-parasiticOnly`

Only shows parasitic related content. The default is true.

le::showEMCalculator

Description

Shows the EM Calculator dialog box.

Syntax

```
le::showEMCalculator [-parent <giWindow | int>]
```

Returns

<giDialog> : EM Calculator dialog box.

Arguments

`-parent`

Specifies the id of the window which this dialog box is from. The default is 0.

le::showImportLayout

Description

Shows the **Import Layout** dialog box.

Syntax

```
le::showImportLayout [-parent <giWindow | int>]
```

Returns

<giDialog>: The dialog box.

Arguments

`-parent`

Window (id) of layout editor window to import layout in.

le::showLayoutGeneratorOptions

Description

Shows the **Layout Generator Options** dialog box.

Syntax

```
le::showLayoutGeneratorOptions -parent <giDialog> -optionSet <string>  
-src <dmCellView> -dst <dmCellView>
```

Arguments

-parent

Parent dialog box.

-optionSet

Option set name.

-src

The source cellView.

-dst

The target cellView.

Errors

- The source cellview is not a symbol or schematic view.
- The destination cellview is not a layout view.

le::showOLPOptions

Description

Shows the **Object/Layer Options** dialog box.

Syntax

```
le::showOLPOptions [-parent <giWindow | int>]
```

Returns

<giDialog> : The created or existing Object/Layer Options Dialog

Arguments

-parent

The parent window.

le::showTrackRegionOptions

Description

Shows the **Track Region Options** dialog box.

Syntax

```
le::showTrackRegionOptions [-parent <giWindow | int>]
```

Returns

<giDialog> : The created or existing **Track Region Options** dialog box.

Arguments

-parent

The parent window.

le::split

Description

Splits a fully selected object, based on the point-list and shape type.

Based on the boundary of the split shape, the command adds new vertices to the objects you provide. It modifies the objects by stretching all edges and vertices that intersect the split shape by the (dx, dy) vector.

If the object you pass is oaGroup, then it must correspond to MPP or Ruler. You cannot pass any other group object.

Syntax

```
le::split <objects: {<defigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
-type <enum> -points [<point>] [-dx <float>] [-dy <float>] [-lockAngles  
<bool>]
```

Returns

{oaFig}: Collection of split objects.

Arguments

objects

Collection of objects to split.

-type

Shape type for the point list.

`-points`

Point list for the object, based on which the split shape is determined.

`-dx`

Stretch in the horizontal direction. The default is 0.

`-dy`

Stretch in the vertical direction. The default is 0.

`-lockAngles`

Turns on/off angle locking. The default is true.

Errors

- Not all provided shapes form the same design.
- Less than two points are specified in point list.
- Shape type is rectangle, but number of points in list is not equal to 2.
- At least one of the objects became invalid because of split and then stretch (for example it would create a self-intersecting polygon).
- `oaGroup` is passed which does not correspond to MPP or Ruler object.

Examples

In the following example, the first line defines a set of points describing a box shape; the second line defines a rectangle using the defined set of points; the third line splits the rectangle using the given point list and moves the split part 1.5u on the X-axis.

```
set box [list {10.0 10.0} {15.0 15.0}]
set rect [le::createRectangle $box -design $oaDesign -lpp "m1 drawing"
-net "A"]
le::split $rect -type rectangle -points [list {12 13} {16 16}] -dx 1.5
```

le::splitArray

Description

Splits the array.

Syntax

```
le::splitArray <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>}
| {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}
-points [<point>] [-toggleArray <bool>]
```

Returns

{<oaInst>} : Collection of split instances or array instances.

Arguments

objects

Collection of array instances to be split.

-points

Point list for the line, based on which the split segment is determined.

-toggleArray

Toggles arrays. The default is false.

Examples

The following example splits ArrayInst instances; they are placed across the line with {{0.0 3.0} {10.0 3.0}} coordinates.

```
set design [ed]
# Create array inst
set inst [le::createInst -design $design -libName reference40nm -cellName
n_4t -viewName layout -origin {1 1} -cols 10 -rows 10]
le::splitArray [db::getInsts -of [ed]] -points {{0.0 3.0} {10.0 3.0}}
```

le::splitIntoBoxes

Description

Splits the specified oaShape (oaPolygon, oaPath, oaPathSeg, or oaRect) into its non-overlapping boxes. The set of rectangles exactly cover the original oaShape.

Syntax

```
le::splitIntoBoxes <object: oaPath | oaPathSeg | oaPolygon | oaRect>
[-type <enum>]
```

Returns

<string> : List of boxes.

Arguments

object

The object to be split.

-type

Specifies X | Y type of split by maxX or maxY.

Examples

The following example splits the specified rectilinear polygon (not physically) into vertically aligned boxes, and then creates a rectangle.

```
set design [ed]
set polygon [le::createPolygon {{0.295 0.575} {1.41 0.575} {1.41 0.07}
    {1.085 0.07} {1.085 0.35} {0.605 0.35} {0.605 0.08} {0.295 0.08}}
    -design $design -lpp {m1 drawing}]
foreach points [le::splitIntoBoxes $polygon -type maxX] {
    set x1 [lindex $points 0]
    set y1 [lindex $points 1]
    set x2 [lindex $points 2]
    set y2 [lindex $points 3]
    le::createRectangle [list [list $x1 $y1] [list $x2 $y2]] -design
    $design -lpp {m2 drawing}
}
```

le::syncTo

Description

Synchronizes a given oaFigGroup object's peers within the same synchronous group.

Syntax

```
le::syncTo <object: defigure | oaFigGroup>
```

Returns

<oaGroup>: The oaGroup object owning the input oaFigGroup and its peer.

Arguments

object

An oaFigGroup or deFigure object to be synced.

le::stretch

Description

Receives a list of partially or fully selected objects and stretches them based on the modification vector.

If you set the `-lockAngles` argument, the object retains the same angles during stretching. The `-permitJogs` argument specifies whether stretch can insert jogs to keep connectivity.

Syntax

```
le::stretch <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
[-dx <float>] [-dy <float>] [-lockAngles <bool>] [-permitJogs <bool>]  
[-keepConnected <enum>] [-keepAbutment <bool>] [-removeUnanchoredNet  
<bool>]
```

Arguments

`objects`

Collection of full or partial objects to stretch.

`-dx`

Stretch in the horizontal direction. (Default is: 0)

`-dy`

Stretch in the vertical direction. (Default is: 0)

`-lockAngles`

Turns on/off angle locking. (Default is: true)

`-permitJogs`

Turns on/off jog insertion at the end of path/pathSeg objects. (Default is: false)

`-keepConnected`

The connection mode for stretching. ({ backbone | fixedVias | individual }, default is: individual)

`-keepAbutment`

Keeps abutment. (Default is: false)

`-removeUnanchoredNet`

Option to remove unanchored net (Default is: false)

Returns

{oaFig}: Collection of stretched objects.

Errors

- Not all provided shapes form the same design.
- At least one of the objects became invalid because of stretch, for example it would create a self-intersecting polygon.
- oaFigGroup is passed as an input argument which does not correspond to an MPP or Ruler object.

Examples

In the following example, the first line defines the side of the figure defined by the points; the second line gets the partially selected object; and the third line stretches the partially selected edge of the object a distance of 2.0u on the X and Y axes without changing the angles of the object sides.

```
set figure [de::getFigures [list [list 22.0 11.0]] -design $oaDesign
    -partial true]
set selected [de::select [db::getNext $figure]]
le::stretch $selected -dx -2.0 -dy -2.0 -lockAngles true
```

le::toggleDRDTreeWidget

Description

Toggles the check state of a DRDTreeWidget by selection.

Syntax

```
le::toggleDRDTreeWidget <{target}: giWidget> [-checked <bool>]
```

Arguments

{target}

Target widget to toggle selected items.

-checked

Toggle selected items to checked or unchecked.

le::trimWire

Trim paths, branches or both

Syntax

```
le::trimWire <objects: dbCollection | {<deFigure>} | {<deSelectionSet>}
    | {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaNet>}
    | {<oaShape>} | {<oaVia>}> -method <enum> [-of <enum>] [-edge <enum>]
    [-preserveEndStyle <bool>] [-byNet <bool>] [-toConnected <bool>]
    [-toViewLevel <bool>] [-skipAndFindClosestAlignedEdge <bool>]
```

Returns

<none>

Arguments

`objects`

paths, pathsegs, vias or nets to be trimmed.

`-method`

Trimming method of Trim Wire Command. ({ both | branch | path })

`-of`

Trimming direction of path method. ({ auto | bottom | left | right | top }, default is: auto)

`-edge`

Ending edge of a via neighbor for path method. ({ path | via }, default is: path)

`-preserveEndStyle`

Preserve the original end style of path/pathseg objects after trimming. (Default is: true)

`-byNet`

Use the neighbors of the same net for trimming path/pathseg. (Default is: false)

`-toConnected`

Trim to connected object. (Default is: false)

`-toViewLevel`

Trim to view level. (Default is: true)

`-skipAndFindClosestAlignedEdge`

Skip and find next closest aligned edge. (Default is: false)

le::validateColor

Description

Validates color.

Syntax

```
le::validateColor [<objects: {<defigure>} | {<oaFig>} | {<oaNet>}}>]  
[-fix <bool>] [-scope <enum>] [-no_color <bool>] [-track <bool>]  
[-constraint <bool>] [-conflict <bool>] [-not_locked <bool>] [-waive  
<bool>] [-levels <int>]
```

Arguments

`objects`

List of objects to validate color.

`-fix`

Specifies whether to fix violations. The default is false.

`-scope`

Scope to validate color. Valid values: design, object (default), and viewport.

`-no_color`

Specifies whether to report no/same color violations. The default is false.

`-track`

Specifies whether to report track mismatch violations. The default is false.

`-constraint`

Specifies whether to report constraint mismatch violations. The default is false.

`-conflict`

Specifies whether to report color conflict violations. The default is false.

`-not_locked`

Specifies whether to report not locked violations. The default is false.

`-waive`

Specifies whether to waive violations of fully covered shapes. The default is false.

`-levels`

Specifies the number of hierarchical levels to validate. The default is 0.

Examples

The following example validates color for the set of object or entire design.

```
# Validate color for entire design all hierarchy levels, fix violations
# and report not locked violation
le::validateColor -fix true -scope design -levels 32 -not_locked true
```

le::yank

Description

Copies data cut by the given region to the clipboard.

Syntax

```
le::yank [<objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>]  
[-design <oaDesign>] -points [<point>] [-regionType <enum>] [-levels  
<int>] [-anchor <point>] [-snapToGrid <enum>] [-filterByOLP <bool>]  
[-flatten <bool>] [-selectResult <bool>]
```

Arguments

`objects`

Collection of objects to yank.

`-design`

Design in which to activate the yank command.

`-points`

A list of points defining the contour of the shape that specifies the yank region.

`-regionType`

The type of region (a shape) that is passed to `-points`. ({ polygon | rectangle }, default is: rectangle)

`-levels`

Number of hierarchical levels beyond which to ignore instances which overlap the yank region. The interactive command uses the visual depth for this parameter. (Default is: 0)

`-anchor`

Specifies the anchor point for paste.

`-snapToGrid`

Specify if snapping the yanked results to the manufacturing grid. ({ abort | off | on }, default is: on)

`-filterByOLP`

Specifies whether to filter the yank results by OLP settings. (Default is: false)

`-flatten`

Specifies to flatten all input objects. This option only works when the `-filterByOLP` option is true. (Default: false)

`-selectResult`

Specifies whether to add the result to the selection set. (Default is: true)

Returns

<none>

Errors

- Point list is empty.
- Number of levels is negative.
- The objects passed are not from the same design.
- A boundary object is cut into multiple pieces or a snap boundary is cut into a non-rectangular shape.

Examples

In the following example, the first line defines a set of points; the second line yanks the objects in the specified design within a *rectangle* shape defined by the set of points.

```
set points [list [list 0.0 0.0] [list 12.0 12.0] ]  
le::yank [db::getShapes -of $oaDesign] -points $points -regionType  
rectangle
```

ile::addDummy

Description

Starts the interactive command to add dummy devices.

Syntax

```
ile::addDummy [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::addToFigGroup

Description

Starts the interactive command to add objects to an existing OA figure group.

Syntax

```
ile::addToFigGroup [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::adjustCursor

Description

Starts the interactive command to adjust cursor.

Syntax

```
ile::adjustCursor [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::align

Description

Brings up the **Align** toolbar and starts the interactive **Align** command. The toolbar remains open until you end the command.

Syntax

```
ile::align [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

Errors

- An invalid window Id is passed.
- There is no design opened in specified window.
- Current edit design is not a layout design.
- Current edit design is read-only.

ile::alignAngle

Description

Starts the interactive **Align Angle** command.

Syntax

```
ile::alignAngle [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated.

Returns

<none>

ile::applyColor

Description

Starts the interactive **Apply Color** command.

Syntax

```
ile::applyColor [<context: deContext | unsigned int>] -action <enum>
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

-action

Apply the color action. Valid values: color1, color2, color3, colorNone, colorSame, lock, shift, and unlock.

Returns

<none>

ile::attachDetach

Description

Brings up the Attach/Detach toolbar and runs interactive attaching/detaching command.

Note:

Attaching is implemented as a grouping. So there is no separate non-interactive command for attaching. However, you can use the non-interactive commands corresponding to group creation/deletion OA Tcl commands.

Syntax

```
ile::attachDetach [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::autoChopGuardRing

Description

Starts the interactive **Auto Chop Guard Ring** command to eliminate shorts for multi part paths and both MCell and editing PCell guard rings with overlapped metal shapes.

Syntax

```
ile::autoChopGuardRing [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::autoFix

Description

Displays the AutoFix toolbar and starts the interactive DRD AutoFix command.

Syntax

```
ile::autoFix [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::autoFlipHorizontal

Description

Flips selected shapes about the y-axis.

Syntax

```
ile::autoFlipHorizontal [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::autoFlipVertical

Description

Flips selected shapes about the x-axis.

Syntax

```
ile::autoFlipVertical [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::autoRotate90

Description

Rotates selected shapes 90°.

Syntax

```
ile::autoRotate90 [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::boolean

Description

Brings up the Boolean dialog box and starts the interactive command.

Syntax

```
ile::boolean [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::bridgeInterconnect

Description

Brings up the Bridge Interconnect toolbar and starts the bridge interconnect operation.

Syntax

```
ile::bridgeInterconnect [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::changePinRelationship

Description

Brings up the Change Pin Relationship toolbar, which lets you change the relationships (strong, weak, or must join) between pins.

Syntax

```
ile::changePinRelationship [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::checkCongestion

Description

Starts the interactive command to check route congestion. See Checking Route Congestion.

Syntax

```
ile::checkCongestion [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::checkDensity

Description

Starts the interactive **Check Density** command.

Syntax

```
ile::checkDensity [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::checkVias

Description

Starts the interactive check vias command.

Syntax

```
ile::checkVias [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::checkVoltageAnnotations

Description

Starts the interactive command to check voltage annotation.

Syntax

```
ile::checkVoltageAnnotations [<context: deContext | unsigned int>]
```

Arguments

context

The context in which this command is activated.

Returns

<none>

ile::chop

Description

Starts the interactive command to chop data to the clipboard. In addition to the following command-line arguments, this command also responds to interactive options.

Explicit logging is supported. In the explicit logging mode, the command is logged as `le::chop` on command completion.

For example, the explicit logging command is shown below:

```
le::chop [de::getSelected -design [ed]] -points {{-10 0} {10.33 10.53}}
```

Syntax

```
ile::chop [<context: deContext | unsigned int>] [-log <enum>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

-log

Logging mode. Valid values: auto (default) and explicit.

Returns

<none>

ile::clearDensityDisplay

Description

Clears the density display.

Syntax

```
ile::clearDensityDisplay [<context: deContext | unsigned int>]
```

Arguments

context

The context in which this command is activated.

Returns

<none>

ile::clearVoltagePhysicalAnnotations

Description

Interactively clears the voltage physical annotation information in the layout.

Syntax

```
ile::clearVoltagePhysicalAnnotations [<context: deContext | unsigned  
int>]
```

Arguments

context

Specifies the context of activation.

Returns

<none>

ile::clearVoltages

Description

Interactively clears the voltage information on nets in the layout.

Syntax

```
ile::clearVoltages [<context: deContext | unsigned int>]
```

Arguments

`context`

Specifies the context of activation.

Returns

<none>

ile::cloneMaster

Description

Starts the interactive command to clone master.

Syntax

```
ile::cloneMaster [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::collapseDensityTree

Description

Starts the interactive command to collapse the density tree.

Syntax

```
ile::collapseDensityTree [<context: deContext | unsigned int>]
```

Arguments

`context`

The context in which this command is activated.

Returns

<none>

ile::color

Description

Displays the **Color** toolbar and starts the interactive command.

Syntax

```
ile::color [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

Errors

- An invalid window object is passed.
- No design is opened in the specified window.

ile::compactBus

Description

Brings up the **Compact Bus** toolbar and starts the operation of compact bus bits.

Syntax

```
ile::compactBus [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::continueBus

Description

Starts the interactive **Continue Bus** command.

Syntax

```
ile::continueBus [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::convertToArray

Description

Displays the **Convert to Array** toolbar and starts the interactive command.

Syntax

```
ile::convertToArray [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

Errors

- An invalid window object is passed.
- There is no design opened in specified window.
- Edit Design of the specified window is not of type MaskLayout.

ile::convertToPath

Description

Displays the **Convert to Path** toolbar and starts the interactive command.

Syntax

```
ile::convertToPath [<context: deContext | unsigned int>]
```


Arguments

context

The window ID in which the command is activated.

Returns

<none>

ile::convertToPolygon

Description

Displays the **Convert to Polygon** toolbar and starts the interactive command.

Syntax

```
ile::convertToPolygon [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::copy

Description

Displays the **Copy** toolbar and starts the interactive copying command.

Syntax

```
ile::copy [<context: deContext | unsigned int>] [-object <defigure |  
oaFig>] [-point <point>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

-object

Object for exclusive copy.

`-point`

Specifies the drag start point and the command toolbar is not displayed. This must be specified at the same time with `-object`.

Returns

<none>

Errors

- An invalid window object is passed.
- There is no design opened in the specified window.

ile::copyLook

Description

Displays the **Copy Look** toolbar in the Layout Editor.

Syntax

```
ile::copyLook [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

Errors

- An invalid window object is passed.
- No design is opened in the specified window.
- Edit Design of the specified window is not of type MaskLayout.

ile::copyToClipboard

Description

Starts the interactive command to copy selected objects to the clipboard. In addition to the following command-line arguments, this command also responds to the interactive options.

Explicit logging is supported. In the explicit logging mode, the command is logged as `de::copy` on command completion.

For example, the explicit logging command is shown below:

```
de::copy [de::getSelected -design [ed]] -anchor {-1 -1}
```

Syntax

```
ile::copyToClipboard [<context: deContext | unsigned int>] [-log <enum>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

`-log`

Logging mode ({ auto | explicit }, default is: auto)

Returns

<none>

ile::correctOffGrid

Description

Starts the interactive **Correct Off-grid** command.

Syntax

```
ile::correctOffGrid [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::createAttributeLabel

Description

Brings up the **Attribute Label** toolbar and starts the interactive label creation process.

Syntax

```
ile::createAttributeLabel [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createBlockage

Description

Brings up the **Create Blockage** toolbar and starts the interactive blockage creation process.

Syntax

```
ile::createBlockage [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createBoundary

Description

Brings up the **Create Boundary** toolbar and starts the interactive boundary creation process.

Syntax

```
ile::createBoundary [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createBus

Description

Starts the interactive command to create a bus.

Syntax

```
ile::createBus [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createCellViewFromSelection

Description

Brings up the **Create Cell View from Selection** toolbar and starts the interactive cellView creation process.

Syntax

```
ile::createCellViewFromSelection [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createCoordinateMark

Description

Starts the interactive command to create coordinate marks.

Syntax

```
ile::createCoordinateMark [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createEllipse

Description

Displays the Ellipse Creation toolbar and starts the interactive ellipse creation command.

Syntax

```
ile::createEllipse [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createFigGroup

Description

Brings up the Fig Group Creation toolbar and starts the interactive figGroup creation process.

Syntax

```
ile::createFigGroup [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createGlobalTrackRegion

Description

Starts the interactive command to manually create a global track region.

Syntax

```
ile::createGlobalTrackRegion [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createGuardRing

Description

Starts the interactive command to manually create guard rings.

Syntax

```
ile::createGuardRing [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createInst

Description

Brings up the **Create Instance** dialog box, which you can use to create an instance or an instance array.

Syntax

```
ile::createInst [<context: deContext | unsigned int>] [-device  
  <deDevice>] [-hidden <bool>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

-device

If specified, populates the Create Instance dialog box with the master and parameters from the specified device object.

-hidden

If true, starts the command in a hidden state without affecting the command toolbar or showing the dialog box.

Default: false.

Returns

<none>

ile::createInterconnect

Description

Starts Interconnect Creation and invokes the Create Interconnect toolbar.

Syntax

```
ile::createInterconnect [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createLabel

Description

Brings up the Label Creation toolbar and starts the interactive label creation process.

Syntax

```
ile::createLabel [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createMPP

Description

Brings up the Create MPP toolbar or dialog and starts the interactive MPP creation process.

Syntax

```
ile::createMPP [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createPin

Description

Brings up the pin creation toolbar and starts the interactive pin creation process.

Syntax

```
ile::createPin [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createPinsFromSelection

Description

Brings up the Create Pins from Selection toolbar, which lets you create pins from a selection.

Syntax

```
ile::createPinsFromSelection [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createPlane

Description

Starts the interactive command to create planes.

Syntax

```
ile::createPlane [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createPolygon

Description

Displays the Polygon Creation toolbar and starts the interactive polygon creation command.

Syntax

```
ile::createPolygon [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createRectangle

Description

Displays the Rectangle Creation toolbar and starts the interactive rectangle creation command.

Explicit logging is supported. In the explicit logging mode, the command is logged as `le::createRectangle` on command completion.

Syntax

```
ile::createRectangle [<context: deContext | unsigned int>] [-log <enum>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

-log

Logging mode ({ auto | explicit }, default is: auto)

Returns

<none>

ile::createRouteCorridor

Description

Starts the interactive create routing corridor command.

Syntax

```
ile::createRouteCorridor [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::createRow

Description

Starts the interactive Create Row command.

Syntax

```
ile::createRow [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createRuler

Description

Starts the interactive Create Ruler command in the specified layout window.

Syntax

```
ile::createRuler [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createTopology

Description

Starts the interactive Create Topology command in the specified layout window.

Syntax

```
ile::createTopology [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createTrackRegion

Description

Starts the interactive Create Track Region command.

Syntax

```
ile::createTrackRegion [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createTrackRegionPCell

Description

Starts the interactive Create Track Region PCell command.

Syntax

```
ile::createTrackRegionPCell [<context: deContext | unsigned int>] [-log  
  <enum>]
```

Arguments

`context`

Context of activation.

`-log`

Logging mode ({ auto | explicit }, default is: auto)

Returns

<none>

ile::createTransmissionLine

Description

Starts the interactive command to create transmission lines.

Syntax

```
ile::createTransmissionLine [<context: deContext | unsigned int>]
```

Arguments

`context`

The context in which the command is activated.

Returns

<none>

ile::createVia

Description

Brings up the Create Via toolbar and starts the interactive via creation process.

Syntax

```
ile::createVia [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::createVoltageConfig

Description

Starts the interactive command to create a new configuration view.

Syntax

```
ile::createVoltageConfig [<context: deContext | unsigned int>]
```

Arguments

context

Specifies the context of activation.

Returns

<none>

ile::createVoltagePhysicalAnnotations

Description

Starts the interactive physical annotate voltages command.

Syntax

```
ile::createVoltagePhysicalAnnotations [<context: deContext | unsigned  
int>]
```

Arguments

context

The context in which the command is activated.

Returns

<none>

ile::cut

Description

Starts the interactive command to copy selected objects to the clipboard and then delete them. In addition to the following command-line arguments, this command also responds to the interactive options.

Syntax

```
ile::cut [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::cutCorner

Description

Brings up the **Cut Corner** toolbar and starts the interactive **Cut Corner** command.

Syntax

```
ile::cutCorner [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

Errors

- An invalid window Id is passed.
- There is no design opened in the specified window.

ile::defineInFOLibrary

Description

Starts the interactive define InFO library command.

Syntax

```
ile::defineInFOLibrary [<context: deContext | unsigned int>]
```


Arguments

`context`

The context in which this command is activated.

Returns

<none>

ile::delete

Description

Brings up the **Delete** toolbar, which you can use to delete full or partial objects.

Explicit logging is supported. In the explicit logging mode, the command is logged as `le::delete` on command completion.

Syntax

```
ile::delete [<context: deContext | unsigned int>] [-log <enum>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

`-log`

Logging mode ({ auto | explicit }, default is: auto)

Returns

<none>

ile::deleteCutMetal

Description

Starts the interactive delete cut metal command.

Syntax

```
ile::deleteCutMetal [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::deleteFigGroup

Description

Brings up the Fig Group Deletion toolbar and starts the interactive figGroup deletion process.

Syntax

```
ile::deleteFigGroup [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::distributeAlign

Description

Starts the interactive Distribute Align command.

Syntax

```
ile::distributeAlign [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

Errors

- An invalid window Id is passed.
- There is no design opened in the specified window.

- Current edit design is not a layout design.
- Current edit design is read-only.

ile::editFCBlackBox

Description

Starts the interactive command to edit or create a Fusion Compiler black box.

Syntax

```
ile::editFCBlackBox [<window: int>]
```

Arguments

`window`

Window in which to start the command.

Returns

<none>

ile::editICC2BlackBox

Description

Starts the interactive command to edit or create an IC Compiler II black box.

Syntax

```
ile::editICC2BlackBox [<window: int>]
```

Arguments

<window>

Window in which to start the command.

Returns

<none>

ile::editICBlackBox

Description

Starts the interactive command to edit or create an IC Compiler black box.

Syntax

```
ile::editICCBlackBox [<window: int>]
```

Arguments

<window>

Window in which to start the command.

Returns

<none>

ile::eraseLayer

Description

Starts the interactive Erase Layer command.

Syntax

```
ile::eraseLayer [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::expandDensityTree

Description

Expands the density tree.

Syntax

```
ile::expandDensityTree [<context: deContext | unsigned int>]
```

Arguments

context

The context in which this command is activated.

Returns

<none>

ile::fill

Description

Starts the interactive command to fill instances.

Syntax

```
ile::fill [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::flatten

Description

Starts the interactive command to flatten instances. In addition to the following command-line arguments, this command also responds to the interactive options.

Syntax

```
ile::flatten [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::generateCutMetal

Description

Starts the interactive command to generate cut metal shapes.

Syntax

```
ile::generateCutMetal [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::generateGuardRing

Description

Starts the interactive command to create guard rings by auto mode.

Syntax

```
ile::generateGuardRing [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

Examples

The following example generates an nwellGR guard ring around the selected objects in a design.

```
le::generateGuardRing [de::getSelected -design $design] \  
set design [ed]  
-master reference40nm/nwellGR/layout \  
-spacing 0.6 -params {{CutRow 4}}
```

ile::generateLabel

Description

Displays the **Generate Label** toolbar and starts the interactive command.

Syntax

```
ile::generateLabel [<giWindow | int>]
```

Arguments

`giWindow | int`

Window in which the interactive command is activated.

Returns

`<none>`

Errors

- An invalid window object is passed.
- There is no design opened in specified window.
- Edit Design of the specified window is not of type MaskLayout.

ile::generateLayoutPins

Description

Generates layout pins.

Syntax

```
ile::generateLayoutPins [-parent <giWindow | int>]
```

Returns

`<giDialog>` : The dialog box.

Arguments

`-parent`

parent widget

ile::generateMPP

Description

Brings up the Generate MPP toolbar or dialog and starts the interactive MPP creation process.

Syntax

```
ile::generateMPP [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::generatePins

Description

Starts the interactive command to generate pins.

Syntax

```
ile::generatePins [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::generatePlacementTrackRegion

Interactive placement track region generation

Syntax

```
ile::generatePlacementTrackRegion [<context: deContext | unsigned int>]
```

Returns

<none>

Arguments

`context`

Context of activation

ile::generateRoutingTrackRegion

Interactive routing track region generation

Syntax

```
ile::generateRoutingTrackRegion [<context: deContext | unsigned int>]
```

Returns

<none>

Arguments

context

Context of activation

ile::generateShapes

Description

Displays the **Generate Shapes** dialog box and starts the interactive shape generation command.

Syntax

```
ile::generateShapes [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

Errors

- An invalid window object is passed.
- There is no design opened in specified window.

ile::heal

Description

Brings up the **Heal** toolbar and starts the interactive healing process.

Syntax

```
ile::heal [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::hierarchicalCopy

Description

Starts the interactive **Hierarchical Copy** command.

Syntax

```
ile::hierarchicalCopy [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::highlightPath

Description

Starts the interactive shortest path command.

Syntax

```
ile::highlightPath [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::importVoltages

Description

Starts the interactive annotate voltages command.

Syntax

```
ile::importVoltages [<context: deContext | unsigned int>]
```

Arguments

context

The context of activation.

Returns

<none>

ile::inputCoord

Description

Starts the interactive Coordinate input command.

Syntax

```
ile::inputCoord [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::lineSelect

Description

Starts the interactive in-command line selection.

Syntax

```
ile::lineSelect [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::measureAngle

Description

Starts the interactive **Measure Angle** command.

Syntax

```
ile::measureAngle [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::measureDistance

Description

Starts the interactive **Measure Distance** command. This command always nests on top of other interactive commands. This command only supports layout designs.

Syntax

```
ile::measureDistance [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

Errors

- Invalid window ID.
- Window is not a layout window.

ile::merge

Description

Displays the **Merge** toolbar and starts the interactive **Merge** command.

Syntax

```
ile::merge [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::move

Description

Brings up the **Move** toolbar and starts the interactive **Move** command. The **Move** toolbar remains open until you end the command.

Syntax

```
ile::move [<context: deContext | unsigned int>] [-object <defigure |  
oaFig>] [-point <point>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

-object

Object for exclusive move.

-point

Specifies the drag start point and the command toolbar is not displayed. Should be specified at the same time with -object.

Note:

When you specify the `-point` argument, the **Move** toolbar is not displayed. Instead, the tool prompts you to specify a destination point.

Returns

<none>

Errors

- An invalid window Id is passed.
- There is no design opened in specified window.
- Current edit design is not a layout design.
- Current edit design is read-only.

ile::optimize

Description

Optimizes the MOS OD area.

Syntax

```
ile::optimize [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::padLabel

Description

Starts the interactive pad label command.

Syntax

```
ile::padLabel [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::partialToFull

Description

Starts the interactive partial to full command.

Syntax

```
ile::partialToFull [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::paste

Description

Starts the interactive command to paste data from the clipboard. In addition to the following command-line arguments, this command also responds to the interactive options.

Syntax

```
ile::paste [-parent <giWindow | int>] [-clip <deClipboard>]
```

Arguments

`-parent`

Window in which the interactive command is activated.

`-clip`

Clipboard to paste. The default is the most recently copied clipboard.

Returns

<none>

Errors

- Clipboard is empty.
- Clipboard viewType is not compatible with the edit view type.
- Window is not a layout window.
- An invalid window Id.
- Active window cannot be killed.
- Edit design is read-only.

ile::pasteLook

Description

Displays the **Paste Look** toolbar in the Layout Editor.

Syntax

```
ile::pasteLook [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

Errors

- An invalid window object is passed.
- No design is opened in the specified window.
- Edit Design of the specified window is not of type MaskLayout.

ile::reassignNet

Description

Starts the interactive command to reassign an existing net.

Syntax

```
ile::reassignNet [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::removeFromFigGroup

Description

Starts the interactive command to remove objects from an existing OA figure group.

Syntax

```
ile::removeFromFigGroup [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::removeOverlappingCuts

Description

Starts the interactive **Remove Overlapping Cuts** command.

Syntax

```
ile::removeOverlappingCuts [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::replaceVia

Description

Starts the interactive **Replace Via** command to replace standard vias into a user-specified number of cuts with the consideration of keeping interconnect footprint and obstacles.

Syntax

```
ile::replaceVia [<giWindow | int>]
```

Arguments

giWindow | int

Window in which the interactive command is activated.

Returns

<none>

ile::reportCapacitance

Description

Displays the **Electrical Reporter** dialog box for capacitance reporting.

Syntax

```
ile::reportCapacitance [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

See Also

- [Preferences](#)
- [Generating Capacitance Reports](#) in the *Custom Compiler Electrical Checking and Reporting User Guide*

ile::reportEM

Description

Displays the **Electrical Reporter** dialog box for reporting EM (electromigration) of a net.

Syntax

```
ile::reportEM [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

See Also

- [Preferences](#)
- [Reporting Electromigration](#) in the *Custom Compiler Electrical Checking and Reporting User Guide*

ile::reportParasitic

Description

Starts the interactive parasitic reporter command.

Syntax

```
ile::reportParasitic [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::reportResistance

Description

Displays the **Electrical Reporter** dialog box for reporting resistance between two specific points in a net.

Syntax

```
ile::reportResistance [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::reshape

Description

Displays the **Reshape** toolbar and starts the interactive reshaping command.

Syntax

```
ile::reshape [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::reshapeInterconnect

Description

Starts Reshape Interconnect and invokes the Reshape Interconnect Toolbar.

Syntax

```
ile::reshapeInterconnect [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::resize

Description

Displays the **Resize** toolbar and starts the interactive resizing command.

Syntax

```
ile::resize [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::rotate

Description

Displays the **Rotate** toolbar and starts the interactive rotation command.

Syntax

```
ile::rotate [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::scalarizeArrayInst

Description

Starts the interactive command to scalarize array instances.

Syntax

```
ile::scalarizeArrayInst [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::selectByLPP

Description

Starts the **Select by LPP** command.

Syntax

```
ile::selectByLPP [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::selectObjects

Description

Starts the interactive command to select objects.

Syntax

```
ile::selectObjects [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::selectRouteObjects

Description

Starts the interactive command to select route objects.

Syntax

```
ile::selectRouteObjects [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ile::showDensityDisplaySetting

Description

Starts the interactive **Show Density Display Setting** command.

Syntax

```
ile::showDensityDisplaySetting [<context: deContext | unsigned int>]
```

Arguments

`context`

The context in which this command is activated.

Returns

<none>

ile::showEMCalculator

Description

Starts the interactive **EM Calculator** command.

Syntax

```
ile::showEMCalculator [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::split

Description

Displays the **Split** toolbar and starts the interactive command.

Syntax

```
ile::split [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::splitArray

Description

Starts the interactive command to split arrays.

Syntax

```
ile::splitArray [<context: deContext | unsigned int>]
```


Arguments

`context`

Context of activation.

Returns

`<none>`

ile::stackInterconnect

Description

Displays the Stack Interconnect toolbar and starts the interconnect command.

Syntax

```
ile::stackInterconnect [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

Returns

`<none>`

Errors

Errors

An invalid design is passed.

- An invalid window object is passed.
- No design was opened in the specified window.

ile::stretch

Description

Displays the **Stretch** toolbar and starts the interactive stretch command.

Syntax

```
ile::stretch [<context: deContext | unsigned int>] [-object <defigure |  
oaFig>] [-point <point>]
```

Arguments

`context`

Context in which the command is activated: the design context or the window Id.

`-object`

Object for exclusive stretch.

`-point`

Specifies the drag start point and the command toolbar is not displayed. Should be specified at the same time with `-object`.

Note:

When you specify the `-point` argument, the Stretch toolbar is not displayed. Instead, the tool prompts you to specify a destination point.

Returns

`<none>`

ile::stretchArray

Description

Starts the interactive command to stretch the specified array instances.

Syntax

```
ile::stretchArray [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

`<none>`

ile::stretchWidth

Description

Starts the interactive command to stretch the wire width.

Syntax

```
ile::stretchWidth [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::stripeWire

Description

Starts the interactive **Stripe Wire** command.

Syntax

```
ile::stripeWire [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::tap

Description

Runs the interactive tapping command. This command can be nested from all LE commands.

Syntax

```
ile::tap [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::trimWire

Description

Brings up the **Wire Trim** toolbar.

Syntax

```
ile::trimWire [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::validateColor

Description

Starts the interactive command to validate color.

Syntax

```
ile::validateColor [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::yank

Description

Starts the interactive command to yank data to the clipboard. In addition to the following command-line arguments, this command also responds to interactive options.

Explicit logging is supported. In the explicit logging mode, the command is logged as `le::yank` on command completion.

For example, the explicit logging command is shown below:

Need to select objects:

```
le::yank [de::getSelected -design [ed]] -points {{-10 0} {10.33 10.53}}
```

Does not need to select objects (yank by region):

```
le::yank -design [ed] -points {{-10 0} {10.33 10.53}}
```

Syntax

```
le::yank [<context: deContext | unsigned int>] [-log <enum>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

-log

Logging mode ({ auto | explicit }, default is: auto)

Returns

<none>

le::pkg::checkPlaneBoundary

Description

Checks plane boundary placement or LVS comparison results and generates reports.

Syntax

```
le::pkg::checkPlaneBoundary [-nets <oaNet | string>]
                                -design <oaDesign> -mode <enum>
                                [-coordinateMark <bool>]
                                [-floating <bool>] [-errorMarker
                                <bool>]
                                [-region [<point>]] [-filePath
                                <string>]
```

Arguments

-design

Design to check.

-mode

Mode to report: { reportOpenAfterRealize | reportOpenBeforeRealize |
reportPlaneGeneration }.

- `reportOpenAfterRealize`: Analyzes the flight lines to report floating connectivity islands, dangling edge, shorts, and LVS comparison status.
- `reportOpenBeforeRealize`: Using the Highlighting Connected feature, simulates the plane guide to check whether all same-net vias are connected before the plane boundary is realized (placed).
- `reportPlaneGeneration`: Shows the percent of plane guides that were created successfully. Shows whether plane guides are created successfully by plane layers.

`-nets`

Net name or `oaNet` object to check.

`-coordinateMark`

Creates coordinate marks for open islands. (Default: false)

`-errorMarker`

Creates error markers for open bumps. (Default: false)

`-floating`

Creates coordinate marks on floating figures. This depends on the `-coordinateMark` argument. (Default: false)

`-region`

Region in which coordinate marks or error markers are created.

`-filePath`

Specifies the output file.

Returns

{<oaFig>} : Collection of open figures.

le::pkg::createDegassingHole

Creates degassing holes and adds dummy metals (or metal fills) to ensure uniform metal density across packaging designs.

Syntax

```
le::pkg::createDegassingHole -design <oaDesign> -layerMappingFile  
  <string> -fobox <string> [-keepRunResult <bool>] [-runset <string>]  
  [-arguments <string>] [-diebox <string>] [-d2d <string>] [-ubm <string>]  
  [-ipd <string>] [-bgab <string>] [-tiv <string>]
```

Arguments

`-design`

Design in which degassing holes and metal fills are created.

`-layerMappingFile`

Specifies the layer mapping file.

`-fobox`

Fanout box (FOBOX) region layer-purpose pair (LPP).

`-keepRunResult`

Creates a `synopsys_custom/degassing_hole_<name>` directory and stores the output of the run in the directory. (Default is: false)

`-runset`

The IC Validator degassing runset file. The runset is automatically generated if `-runset` is not specified. (Default is:)

`-arguments`

Arguments passed to the IC Validator tool. (Default is: `-oa_dm5`)

`-diebox`

Die box (DIEBOX) region layer-purpose pair. (Default is:)

`-d2d`

Die-to-die (D2D) region layer-purpose pair. (Default is:)

`-ubm`

UBM layer-purpose pair. (Default is:)

`-ipd`

IPD layer-purpose pair. (Default is:)

`-bgab`

BGAB region layer-purpose pair. (Default is:)

`-tiv`

TIV layer-purpose pair. (Default is:)

Returns

<none>

le::pkg::createPlaneBoundary

Description

Creates a power and ground (PG) plane boundary in packaging designs.

Syntax

```
le::pkg::createPlaneBoundary <points: [point]> -design <oaDesign>  
-adoptTech <bool> -lpp <lpp> [-net <oaNet | string>] [-wireWireSpacing  
<float>] [-planeWireSpacing <float>] [-planePlaneSpacing <float>]  
[-planeWidthThreshold <float>] [-excludeNet <string>] [-patchConcave  
<bool>] [-priority <int>]
```

Returns

{<oaFig>}: Collection of the created plane boundaries.

Arguments

points

The list of points that form the plane.

-design

Design in which to create the plane.

-adoptTech

Look for related rules in packaging technology.

-lpp

The LPP on which to create the plane.

-net

The net name or oaNet object to which the plane should be assigned.

-wireWireSpacing

The spacing between two wires. (Default is: 10)

-planeWireSpacing

The spacing between plane to wire. (Default is: 10)

-planePlaneSpacing

The spacing between two planes. (Default is: 10)

-planeWidthThreshold

Shapes with width larger than the value are treated as planes. Others are treated as wires. (Default is: 100)

`-excludeNet`

Excludes shapes of the specified net, separated by space. (Default is:)

`-patchConcave`

Automatically patch concaves corners. (Default is: true)

`-priority`

Plane priority. (Default is: 15)

le::pkg::deleteDegassingHole

Description

Restore the design from degassing holes.

Syntax

```
le::pkg::deleteDegassingHole -design <oaDesign> [-force <bool>]
```

Arguments

`-design`

Specifies from which design to restore degassing holes.

`-force`

Forces to overwrite the design. Otherwise, a dialog appears for you to decide.
The default is `true`.

Returns

{<oaDesign>} : Collection of degassing holes.

le::pkg::deleteFloatingShapes

Description

Deletes floating shapes.

Syntax

```
le::pkg::deleteFloatingShapes (-routerConfig <bool> | -nets <string>)  
-design <oaDesign> [-length <float>]
```

Arguments

`-design`

Design in which to create the deleteFloatingShapes.

`-routerConfig`

Specifies nets from the router configure file.

`-nets`

Specifies nets to destroy floating.

`-length`

Shapes with a length narrower than the value are treated as floating to delete.
(Default is: 0)

Returns

<none>

le::pkg::deleteRedundantVias

Description

Deletes redundant vias within the specified area in packaging designs.

Syntax

```
le::pkg::deleteRedundantVias (-box <box> | -points [<point>]) -design  
    <oaDesign>
```

Arguments

`-box`

Option to draw an area on the canvas for redundant via deletion.

`-points`

Option to use coordinates to specify an area for redundant via deletion.

`-design`

Design in which you want to delete the redundant vias.

Returns

<none>

le::pkg::generatePlaneBoundary

Description

Generates a plane boundary.

Syntax

```
le::pkg::generatePlaneBoundary [-config [<string>] | -initConstraintFile
<string> | -importConstraintFile <string> | -exportConstraintFile
<string>] -design <oaDesign> [-net <oaNet | string>] [-oneIslandOnly
<bool>] [-legalIslandOnly <bool>] [-combineCluster <bool>] [-pushRoute
<bool>] [-combineRouter <bool>] [-excludeNets <string>] [-filterNets
<string>] [-boneWithContour <bool>] [-primaryOnly <bool>] [-coverDiffNet
<bool>] [-coverFOB <bool>] [-coverFOBSpacing <float>] [-escapeSpacing
<string>] [-d2dEscapeSpacing <string>] [-d2dSizing <string>]
[-surroundingComponentSizing <string>] [-surroundingDepth <float>]
[-customizedClusterSpacing <float>] [-maxBoxClusterSpacing <float>]
[-maxClusterCount <int>] [-maxBoxCount <int>] [-uTurn <string>]
[-rmstMode <string>] [-escapeMode <string>] [-escapeDirection
<string>] [-escapeNeighborSide <bool>] [-escapeFourDirection
<bool>] [-escapeAllDifferentNet <bool>] [-mergeSameNet <bool>]
[-tryNeighborSideCount <int>] [-zigzagMaxLength <float>] [-zigzagDepth
<float>] [-minWidth <float>] [-minSpacing <float>] [-mergeShieldingPaths
<bool>] [-mergeShapes <bool>] [-uniformGrow [<bool>]] [-autoOrder
<bool>] [-pointsCollection <bool>] [-patchConcave <bool>]
```

Arguments

-design

Design in which to create the plane to be generated.

-config

Configure file for generating. The format should be {{Layer Purpose} {Net Method LeftSizing RightSizing TopSizing BottomSizing}}.

Method: Border, Contour, Bone, All

-initConstraintFile

Creates plane guides with net layer assignment automatically, and exports the initial PG plane guides configuration file. Note that the net layer assignment preference `rtPackagingPlaneBoundaryNetLayers` should be set. (Default: `plane_constraint.csv`)

-importConstraintFile

Creates plane guides with the given plane constraints. (Default: `plane_constraint.csv`)

-exportConstraintFile

Exports plane constraints. (Default: `plane_constraint.csv`)

-net

The net name or `oaNet` object to which generate plane should be assigned.

`-oneIslandOnly`

Only allows to create one island plane. (Default is: true)

`-legalIslandOnly`

Only allows to create a legal island plane. (Default is: true)

`-combineCluster`

Combines Cluster to generate planes. (Default is: false)

`-pushRoute`

Post process push route. (Default is: false)

`-combineRouter`

Combines Router to generate planes. (Default is: false)

`-excludeNets`

Specifies exclude nets. (Default is:)

`-filterNets`

Specifies filter nets. Planes are not created. (Default is:)

`-boneWithContour`

Uses the contour geometry to apply bone method.

`-primaryOnly`

Considers primary from router config only.

`-coverDiffNet`

Covers all different nets when generating the plane boundary. (Default is: false)

`-coverFOB`

Covers FOB when generating the plane boundary. (Default is: false)

`-coverFOBSpacing`

Covers FOB spacing when generating the plane boundary. (Default is: 110)

`-escapeSpacing`

Specifies the escape spacing for different nets. (Default is:)

`-d2dEscapeSpacing`

Specifies the D2D escape spacing for different nets. (Default is:)

`-d2dSizing`

Sizing for the same net in the D2D region. (Default is:)

- `-surroundingComponentSizing`
Surrounding component sizing for the same net. (Default is:)
- `-surroundingDepth`
Specifies the surrounding depth. (Default is: 0)
- `-customizedClusterSpacing`
Specifies the spacing for clusters. (Default is: 0)
- `-maxBoxClusterSpacing`
Specifies the spacing for maxBox cluster. (Default is: 0)
- `-maxClusterCount`
Specifies the maximum count for clusters. (Default is: 0)
- `-maxBoxCount`
Specifies the maximum count to create boxes. (Default is: 1)
- `-uTurn`
Specifies the U-turn mode. (Default is: Default)
- `-rmstMode`
Specifies the RMST method mode. (Default is: Region)
- `-escapeMode`
Specifies the escape mode. (Default is: Nearby)
- `-escapeDirection`
Specifies the escape direction. (Default is:)
- `-escapeNeighborSide`
Escapes to the neighbor side if possible. (Default is: true)
- `-escapeFourDirection`
Escapes to four directions if possible. (Default is: true)
- `-escapeAllDifferentNet`
Escapes all different nets if possible. (Default is: false)
- `-mergeSameNet`
Merges the same net boundary. (Default is: true)
- `-tryNeighborSideCount`
Count to try the neighbor side. (Default is: 1)

`-zigzagMaxLength`

Specifies the zigzag maximum length. (Default is: 0)

`-zigzagDepth`

Specifies the zigzag depth. (Default is: 0)

`-minWidth`

Specifies the minimum width. (Default is: 0)

`-minSpacing`

Specifies the minimum spacing. (Default is: 0)

`-mergeShieldingPaths`

Merges shielding paths.

`-mergeShapes`

Merges top level shapes.

`-uniformGrow`

Uniforms grow when sizing.

`-autoOrder`

Sets automatic order. (Default is: false)

`-pointsCollection`

Returns the points collection instead of generating plane boundary. (Default is: false)

`-patchConcave`

Automatically patches concave corners. (Default is: true)

Returns

`dbCollection` | `{<oaShape>}` : Collection of generated plane boundaries.

Examples

The following example creates a Power Ground Plane boundary on CU_PP3 drawing LPP, for VDD net with the Bone method, taking in count that the plane boundary for VDD1 net also supposed to be on the same LLP.

```
le::pkg::generatePlaneBoundary -design [ed] -net VDD -config {{CU_PP3  
drawing} {VDD Bone 0} {VDD1 Bone 0}}
```

le::pkg::generateRedundantVias

Description

Generates redundant vias.

Syntax

```
le::pkg::generateRedundantVias (-box <box> | -points [<point>]) -design  
  <oaDesign> -net <oaNet | string> [-spacingFactor <float>]
```

Arguments

-box

Specifies the area for redundant via generation.

-points

Specifies the area for redundant via generation.

-design

Design in which to create the redundant vias.

-net

The net name or oaNet object to which redundant vias are generated on.

-spacingFactor

The spacing factor to generate redundant vias. (Default is: 1)

Returns

{<oaFig>} : Collection of redundant vias.

le::pkg::generateSealRing

Description

Generates seal rings.

Syntax

```
le::pkg::generateSealRing -design <oaDesign>
```

Arguments

-design

Specifies which design to perform the generate seal ring.

Returns

{<oaDesign>} : Collection of seal rings.

le::pkg::generateZigzag

Description

Generates zigzag.

Syntax

```
le::pkg::generateZigzag [<objects: {<defigure>} | {<deSelectionSet>} |  
  {<oaFig>} | {<oaShape>}] [-type <enum>] [-targetEdge <int>]
```

Returns

{<oaShape>}: Collection of objects.

Arguments

`objects`

List of objects to generate zigzag.

`-type`

Type of zigzag. Valid values are `obtuseAngle` and `rightAngle`. The default is `rightAngle`.

`-targetEdge`

Target edge to generate zigzag. Any combination of `center:0`, `top:1`, `bottom:2`, `left:4`, `right:8`. The option type is an integer. The default is 15.

le::pkg::getPlaneBoundaries

Description

Return plane boundaries in a design.

Syntax

```
le::pkg::getPlaneBoundaries -design <oaDesign>
```

Arguments

`-design`

Design where the plane is created.

Returns

{<oaFig>} : Collection of plane boundaries.

{<oaDesign>} : Collection of designs.

le::pkg::highlightPlaneBoundary

Description

Highlights plane boundary.

Syntax

```
le::pkg::highlightPlaneBoundary -design <oaDesign> [-net <oaNet |  
string>] [-nets <string>] [-excludeNets <string>] -lpp <lpp> [-plane  
<bool>] [-sameNet <bool>] [-diffNet <bool>] [-openNet <bool>]
```

Arguments

-design

Design in which to create the highlight plane.

-net

The net name or oaNet object to which highlight plane should be assigned.

-nets

Specifies plane nets. (Default is:)

-excludeNets

Specifies plane exclude nets. (Default is:)

-lpp

The LPP on which to create the highlight plane.

-plane

Highlights plane. (Default is: true)

-sameNet

Highlights same nets. (Default is: true)

-diffNet

Highlights different nets. (Default is: false)

-openNet

Highlights open net figures. (Default is: false)

Returns

{<oaFig>} : Collection of highlight plane boundary.

le::pkg::realizePlane

Description

Realizes (or places) the power and ground (PG) plane boundaries in packaging designs.

Syntax

```
le::pkg::realizePlane <objects: {<deFigure>} | {<deSelectionSet>} |  
{<oaFig>} | {<oaShape>}> -adoptTech <bool> [-wireWireSpacing <float>]  
[-planeWireSpacing <float>] [-planePlaneSpacing <float>] [-keepBoundary  
<bool>] [-planeWidthThreshold <float>] [-patchConcave <bool>]  
[-fixMinWidth <bool>] [-removeFloating <bool>] [-removeFloatingLength  
<float>] [-hasD2D <bool>] [-d2dLPP <lpp>] [-planeWidthThresholdD2D  
<float>] [-planeLineSpacingD2D <float>]
```

Arguments

`objects`

List of objects to realize.

`-adoptTech`

Looks for related rules in the packaging technology.

`-wireWireSpacing`

Specifies the spacing between two wires. (Default is: 10)

`-planeWireSpacing`

Specifies the spacing from plane to wire. (Default is: 10)

`-planePlaneSpacing`

Specifies the spacing between two planes. (Default is: 10)

`-keepBoundary`

Keeps the plane boundary after operation. (Default is: false)

`-planeWidthThreshold`

Shapes with width greater than the value are treated as planes. Others are treated as wires. (Default is: 100)

`-patchConcave`

Automatically patches concave corners. (Default is: true)

`-fixMinWidth`

Automatically fixes minimum width. (Default is: false)

`-removeFloating`

Automatically removes floating figures. (Default is: false)

`-removeFloatingLength`

Shapes with a length narrower than the value are treated as floating to remove.
(Default is: 100)

`-hasD2D`

Considers D2D-related rules. (Default is: false)

`-d2dLPP`

The LPP of the D2D region.

`-planeWidthThresholdD2D`

Shapes with a width wider than the value are treated as planes in the D2D region. Others are treated as wires.

`-planeLineSpacingD2D`

Specifies the spacing between plane and line in the D2D region.

Returns

{<oaFig>}: Collection of realized objects.

le::pkg::replacePlaneBoundary

Description

Replaces the plane boundary.

Syntax

```
le::pkg::replacePlaneBoundary <points: [point]> -design <oaDesign>  
-layers <string> [-net <oaNet | string>] [-priority <int>]
```

Arguments

`points`

The list of points that form the plane boundary.

`-design`

Design in which to create the plane boundary.

`-layers`

The layers on which the plane boundary is replaced.

`-net`

The net name or oaNet object to which the plane boundary is assigned.

`-priority`

Plane boundary priority. Default: 15

le::pkg::reportRedundantVias

Description

Reports redundant vias created in packaging designs.

Syntax

```
le::pkg::reportRedundantVias -design <oaDesign> [-net <oaNet | string>]
```

Arguments

`-design`

Design in which you want to report redundant vias.

`-net`

Net name or oaNet object for which redundant vias are reported. If you do not specify the nets, the command reports redundant vias for all nets in the design.

le::pkg::reportShieldingRate

Description

Report shielding rate of the signals in the specified D2D region.

Syntax

```
le::pkg::reportShieldingRate -design <oaDesign> [-layerMappingFile  
  <string>]  
                                     [-arguments <string>] [-diebox <string>]  
                                     [-d2d <string>] [-shieldSpacing <float>]  
                                     [-shieldSpacingTolerance <float>]  
  [-filePath <string>]
```

Arguments

`-design`

Specifies from which design to report shielding rate of the signals.

`-layerMappingFile`

The layer mapping file when exporting the design to a GDS file. If you do not use this option, it gets the value from the `lePackagingLayerMapFile` preference.

`-diebox`

LPP for a die-box region. The die-box region is used to identify whether the signal net should be shielded. If you do not use this option, it gets the value from the `lePackagingArguments` preference.

`-d2d`

LPP for a D2D region. The D2D region is used to define the region to check shielding. If you do not use this option, it gets the value from the `lePackagingD2DLPP` preference.

`-shieldSpacing`

The spacing between signal wire and shielding PG shape. The default is 10.

`-shieldSpacingTolerance`

The tolerance of shield spacing. The default is 0.001.

`-arguments`

Additional arguments that are passed to the IC Validator tool. The default is `-oa_dm5`.

`-filePath`

The file path of an output report.

Returns

{<oaDesign>} : Collection of D2D regions.

lepkg::refreshPlane

Description

Refreshes planes.

Syntax

```
lepkg::refreshPlane -design <oaDesign>
```

Arguments

`-design`

Design in which to refresh planes.

ile::pkg::createDegassingHole

Description

Starts the interactive create degassing hole command that is used for packaging designs.

Syntax

```
ile::pkg::createDegassingHole [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::pkg::createPlaneBoundary

Description

Starts the interactive command to create plane boundary.

Syntax

```
ile::pkg::createPlaneBoundary [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the window Id.

Returns

<none>

ile::pkg::generateRedundantVias

Description

Starts the interactive **Generate Redundant Vias** command.

Syntax

```
ile::pkg::generateRedundantVias [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::pkg::generateZigzag

Description

Starts the interactive **Generate Zigzag** command.

Syntax

```
ile::generateZigzag [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ile::pkg::replacePlaneBoundary

Description

Starts the interactive **Replace Plane Boundary** command.

Syntax

```
ile::pkg::replaceVia [<context: deContext | unsigned int>]
```

Returns

<none>

Arguments

context

Context of activation.

Returns

<none>

Enumerations

Table 120 Enumerations

Name	Values	Description
angle_label_size_e	small medium large	Angle label size.
angle_unit_e	degree radian both	Angle unit.
enAlignAssistMode	off explicit implicit	Align Assist mode.
enAlignAssistTargetMode	cursor object all	Target mode for Align Assist.
enArea	design point polyline region selection viewPort	Area mode.
enAxisMethod	full partial	Determines how drawn axes are rendered.
enBType	placement routing pin	Blockage type.
enBnType	snap pr	Boundary type.
enChopRegionType	rectangle polygon polyline	Type of the chop region.
enConvertToArrayMode	manual auto	Mode to convert instances to arrays.
enConvertToArrayStyle	horizontal vertical max	Style to convert instances to arrays.
enCopyDXDYMode	Specify Interactive	Mode to specify the spacing.
enCreateRulerMode	linear circular dimensionMark	Ruler type.
enDimensionMarkJustificationMode	upperLeft centerLeft lowerLeft upperCenter centerCenter lowerCenter upperRight centerRight lowerRight	Location of the text label.
enDirection	left right bottom top	Direction type.
enDRDMode	off visual assist	DRD engine modes.
enExtendSelectionPathSearchMethod	points pointsExtension	Search method for extended selection of paths or path segments.
enFilterLPPs	valid visible selectable all	Determines which LPPs are listed in the Object/Layer Panel.

Table 120 Enumerations (Continued)

Name	Values	Description
enGenShapeDepth	0x View	Depth of operation.
enGenShapeCommandAppl iesTo	selection design	Mode for operated objects.
enHealMode	full region	Heal method.
enHighlightExplore	pan zoom	Action applied when exploring figures in the highlight set.
enHighlightMode	connected shapes	Highlighting mode used in ile::highlight.
enGuardRingSelect	normal singleLPP allLPPs	Guard ring selection mode.
enInstColsRowsMode	specify byRegion	Cols/Rows mode for instance creation.
enInstDXDYMode	specify interactive	DX/DY mode for instance creation.
enInstJustification	origin upperLeft centerLeft lowerLeft upperCenter centerCenter lowerCenter upperRight centerRight lowerRight	Justification of instances.
enInstSelect	normal singleLPP allLPPs	Instance selection mode.
enInstType	scalar array	Instance type.
enJust	upperLeft centerLeft lowerLeft upperCenter centerCenter lowerCenter upperRight centerRight lowerRight	Text justification supported by OA. These names are consistent with OA API.
enLayerTristate	true false mixed	Composite value for a layer setting from its LPPs.
enLoggingMode	auto explicit	Command logging mode.
enLppMethod	inherit map specify	LPP method.
enLPPSort	priority alpha number	LPP sort order.
enOperation	or and andnot xor	Boolean operations.

Table 120 Enumerations (Continued)

Name	Values	Description
enOrient	R0 R90 R180 R270 MY MYR90 MX MXR90	Orientation supported by OA. These names are consistent with OA API.
enPadOperator	greaterEqual greater equal lessEqual less	Pad area size operator.
enPadLabelMode	Design Region	Create pad label mode.
enPadLabelSelectionType	Rectangle Polygon	Region selection type.
enPasteJustification	lowerLeft lowerCenter lowerRight centerLeft centerCenter centerRight upperLeft upperCenter upperRight	Justification for pasting objects.
enPathSegStyle	truncate extend chamfer custom variable	PathSeg end style.
enPathStyle	truncate extend round variable	Path end style.
enPinRel	strong weak mustJoin	Pin relationship enumeration.
enPIStatus	none unplaced placed fixed locked	Placement status.
enRegionType	point line rectangle polygon	Type for Select by Region.
enResizeSize	Isotropic, Anisotropic	Resize mode.
enResult	Replace, Replace with active LPP, New shape with active LPP	Result mode for Size and Boolean operations.
enRubberBandStyle	none, runtime, parent	Rubber band display style.
enRulerLineStyle	default, thin, medium, thick	Determines the style of ruler lines and labels.
enRulerLabelSize	default, small, medium, large	Determines the text size of labels.
enSearchType	instance group net terminal text evalText viaDef	Type of objects for which to search.
enSelectCriteria	touch enclose	Selection criteria for select by region.
enShape	line rectangle polygon ellipse circle arc	Shape type names.

Table 120 Enumerations (Continued)

Name	Values	Description
enShapeType	polygon polyline rectangle	Shape type.
enSide	bottom horizontalMiddle top left verticalMiddle right	Side type for alignment.
enStackViaSearchMethod	origin bBox	Search method for extended selection of stacked vias.
enSystemGroup	all default user	System group.
enTermType	input output inputOutput switch jumper unused tristate	Terminal type.
enTextFont	euroStyle gothic math roman stick script fixed swedish milSpec	Text font.
enTextType	attribute text	Text type.
enTopologyMode	PointToPoint Star Bone	Topology style.
enTraceDepth	0x view full	Trace depth for Highlight Connected.
enTransform	R0 R90 R180 R270 MY MYR90 MX MXR90	Transforms supported by OA. These names are consistent with the OA API.
enViaOrient	R0 R90 R180 R270 MY MYR90 MX MXR90	Orientation for via object.
enViaSelect	normal singleLPP cutLPP allLPPs	Via selection mode.

Objects

This section covers the following:

- [dbCircularRuler](#)
- [dbGenOptionSet](#)
- [dbLinearRuler](#)
- [dbSubCircularRuler](#)
- [dmCell](#)

- [dmCellView](#)
- [giWindow](#)
- [leCoordinateMark](#)
- [leDimensionMark](#)
- [leFlightline](#)
- [leMPP](#)
- [oaInst](#)
- [oaInstTerm](#)
- [oaNet](#)
- [oaPin](#)
- [oaShape](#)
- [oaTerm](#)
- [oaVia](#)

dbCircularRuler

This object represents the complete specification of a circular ruler. The attributes of the object are described in the following table and can be retrieved by `db::getAttr` command.

Table 121 *Attributes*

Name	Type	Access	Description
design	<i>oaDesign</i>	Read-only	Design in which the ruler resides.
origin	point	Read/Write	Center point for each circle.
scribeAngle	float	Read/Write	Direction of the radial measure scribe: [0:360].
subRulers	[dbSubCircularRuler]	Read/Write	List of subCircularRulers associated with this ruler.

dbGenOptionSet

Contains a set of values for use during cellView generation.

Table 122 *Attributes*

Name	Type	Access	Description
name	string	Read/Write	Name of a Symbol Construction Parameters (SCP) or Abstract Generation Parameters (AGP) set.
setType	string	Read-only	Type of the generator option set.
value	string	Read/Write	Data representing values for use by a generator. Manipulated as a string.

dbLinearRuler

This object represents the complete specification of the linear ruler. The attributes of the object are described in the following table and can be retrieved by `db::getAttr` command.

Table 123 *Attributes*

Name	Type	Access	Description
design	<i>oaDesign</i>	Read-only	Design in which the ruler resides.
points	[point]	Read/Write	List of points that define the ruler (the first point of the array indicates the start point).

dbSubCircularRuler

This object represents a single circle within a circular ruler. The attributes of the object are described in the following table and can be retrieved by `db::getAttr` command.

Table 124 *Attributes*

Name	Type	Access	Description
label	string	Read/Write	Ruler's name.
packet	string	Read/Write	Display packet or packet alias name for the drawing style of the circle.

Table 124 *Attributes (Continued)*

Name	Type	Access	Description
radius	float	Read/Write	Radius of the circle in user units.

dmCell

A named container, in a library, that can hold files and dmCellViews. The dmCell corresponds to a specific oaCell object but is valid as long as the library that the dmCell belongs to remains open. The getAccess requirements of an oaCell are handled automatically by this dm object.

Table 125 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the Cell.
libName	string	Read-only	The name of the Cell's library.

dmCellView

The dmCellView is a container in a library that holds files and is identified by the combination of its cell and view names. It corresponds to an oaCellView but it stays valid as long as its library remains open and its dmCell and dmView exist.

Table 126 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the view that this cellView belongs to.
cellName	string	Read-only	The name of the cell that this cellView belongs to.
cell	<i>dmCell</i>	Read-only	The cellView's cell.
libName	string	Read-only	The name of the library that this cellView belongs to.

giWindow

This object represents a single window and has a container of menus, toolbars, assistants and one status bar. Menus, toolbars and assistants may be retrieved by the API. The status bar and toolbar that are used to hold menus are available here as attributes as they are non-removable attributes of the window object. The attributes of the object are described in the following table and can be retrieved or set by `db::getAttr` and `db::setAttr` commands. Windows are created by various subsystems in Custom Compiler including the Design Editor.

Table 127 *Attributes*

Name	Type	Access	Description
id	int	Read-only	Window identifier.
windowType	<i>giWindowType</i>	Read-only	Window type object.
title	string	Read-only	Title displayed in window border.
iconTitle	string	Read/Write	Title given to the window's icon and displayed in the window manager's task list.
geometry	string	Read/Write	Window geometry.
iconified	bool	Read/Write	Whether the window is iconified.
maximized	bool	Read/Write	Whether the window is maximized. Note that this can be true even if the window is currently iconified.
statusBar	<i>giStatusBar</i>	Read-only	Status bar object holding actions that will render this window's status bar. If empty, the status bar will not be displayed in the window.

leCoordinateMark

This object represents the complete specification of the coordinate mark. The attributes of the object are described in the following table and can be retrieved by `db::getAttr` command.

Table 128 *Attributes*

Name	Type	Access	Description
design	<i>oaDesign</i>	Read-only	Design in which the mark resides.
label	string	Read-only	Text label on the mark.
point	point	Read/Write	Location of the coordinate mark.
note	string	Read/Write	Text note on the mark.

leDimensionMark

This object represents the complete specification of the dimension mark. The attributes of the object are described in the following table and can be retrieved by `db::getAttr` command.

Table 129 *Attributes*

Name	Type	Access	Description
design	<i>oaDesign</i>	Read-only	Design in which the mark resides.
points	<i>oaPointArray</i>	Read/Write	4-point array represented in an <i>oaLine</i> . The segment between points 2 and 3 represents the dimension mark. The segments between points 1 and 2, as well as points 3 and 4, represent the extensions. If the mark is created without extensions, these extension lines will be 0 length. The main segment will be retrieved if <code>includeExtension</code> is false.
includeExtension	bool	Read/Write	If false, trim the <i>points</i> line to only the main segment.
packetName	string	Read/Write	The display packet name for rendering.
justification	<i>oaTextAlign</i>	Read/Write	Text justification, relative to the coordinate mark's extents.

leFlightline

This object represents a flight line instance on the canvas. It always has an `oaNet`. If it is a topology flight line then it also has an `oaRoute` on that net.

Table 130 *Attributes*

Name	Type	Access	Description
net	oaNet	Read-only	The net of this flight line.
route	oaRoute	Read-only	For a topology flight line, the oaRoute that it represents. For other flight lines it is null.

leMPP

Represents the MPP object, which is an `oaAppDef` object with an `oaGroup` of type MPP. The attributes of the object are described in the following table and can be retrieved by `db::getAttr` command.

Table 131 *Attributes*

Name	Type	Access	Description
definition	string	Read-only	The MPP definition name.
name	string	Read-only	The MPP object name.
centerline	[point]	Read-only	The list of points that define the centerline of the MPP object.
lines	[point]	Read-only	List of lines that define the non-chopped parts of the centerline. Lines must be covered by centerline (i.e., there should not be a line outside of the centerline).
design	[oaDesign]	Read-only	Design to which the instance belongs.

oalnst

Represents block domain instances. An instance represents the inclusion of one design as a part of the contents of another. Use the `db::getAttr` command to retrieve the `oalnst` attributes described in the following table.

Table 132 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the oalnst.
design	<i>oaDesign</i>	Read-only	Design to which the instance belongs.
master	<i>oaDesign</i>	Read-only	Master of the instance.
orient	<i>oaOrientEnum</i>	Read/Write	Orientation of the instance.
origin	point	Read/Write	Origin of the instance.
libName	string	Read-only	The library name for the master design referenced by the instance.
cellName	string	Read-only	The cell name for the master design referenced by the instance.
viewName	string	Read-only	The view name for the master design referenced by the instance.
header	<i>oaInstHeader</i>	Read-only	The instHeader associated with this instance.
numBits	int	Read-only	The number of bits of this instance.
params	<i>oaParamArray</i>	Read/Write	Parameters that were specified during creation. Only PCell instances have parameters.
placementStatus	<i>oaPlacementStatusEnum</i>	Read/Write	The placement status for the instance.
source	<i>oaSourceEnum</i>	Read/Write	The source for the instance.
isImplicit	bool	Read-only	A boolean that indicates if the instance is implicit.

oalnstTerm

Represents all types of connections between a net and a terminal in the master of an instance. The connection is valid if the number of bits in the net equals the number of bits implied by the instTerm's terminal name. Use the `db::getAttr` command to retrieve the oalnst attributes described in the following table.

Table 133 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the term associated with this instTerm.
net	<i>oaNet</i>	Read-only	The net associated with this instTerm.
inst	<i>oaInst</i>	Read-only	The inst associated with this instTerm.
term	<i>oaTerm</i>	Read-only	The term associated with this instTerm.
termPosition	int	Read-only	The term position that this instTerm connects to.
numBits	int	Read-only	The number of bits for this instTerm.
isImplicit	bool	Read-only	A boolean value indicating whether this instTerm is implicit.

oaNet

Represents scalar and multi-bit nets that are part of the design's physical description. Nets represent the logical connectivity within a block of a design. Nets connect to terminals, which are the logical connection points on instances of the net's block. Nets also connect to instTerms that represent connections to the lower-level instances in the net's block. Use the `db::getAttr` command to retrieve the oaNet attributes described in the following table.

Table 134 *Attributes*

Name	Type	Access	Description
name	string	Read-only	Name of oaNet.
extracted	bool	Read/Write	Whether this net is extracted.
flightlinesEnabled	bool	Read/Write	Specifies if flight lines are rendered for this net.
flightlinePacket	bool	Read/Write	Name of the packet to display connectivity flight lines. Must be one of y0Fit to y9Fit.

Table 134 Attributes (Continued)

Name	Type	Access	Description
empty	bool	Read-only	A boolean value that indicates if the net has any objects attached to it or associated with it.
global	bool	Read/Write	A boolean value that indicates if the net is a global.
implicit	bool	Read-only	A boolean value that indicates if the net is implicit.
numBits	int	Read-only	The number of bits in the net.
sigType	<i>oaSigTypeEnum</i>	Read/Write	The signal type of the net.

Note:

The flight line and connectivity attributes (extracted, flightlinesEnabled, and flightlinePacket) are not stored in OpenAccess. They are managed in memory by the Connectivity Engine and stored persistently in a side file in the design's cellView.

oaPin

Represents the physical connections of terminals to nets. Use the `db::getAttr` command to retrieve the oaPin attributes described in the following table.

Table 135 Attributes

Name	Type	Access	Description
name	string	Read/Write	Name of oaPin.
design	<i>oaDesign</i>	Read-only	Design to which the net belongs.
term	<i>oaTerm</i>	Read/Write	The terminal to which this pin is attached.
accessDir	int	Read/Write	The access direction bit-field.
placementStatus	<i>oaPlacementStatusEnum</i>	Read/Write	The placement status of the pin.

oaShape

Defines specific geometry at specific locations in a design. Shapes, together with routes, specify all the manufacturable geometry in a chip layout. Shapes can also be used for more abstract purposes. The Text shape classes provide viewable labels on design objects. All shapes are placed on an `oaLayer` and an `oaPurpose`. The design API identifies these by the layer and purpose numbers. The corresponding names are found in the technology database. Optionally, shapes can also belong to either an `oaNet` or an `oaPin` to provide part of the physical implementation of that object.

Table 136 *Attributes*

Name	Type	Access	Description
design	<i>oaDesign</i>	Read-only	The design in which the shape exists.
bBox	box	Read-only	The figure bounding box.
layerNum	int	Read/Write	The layer number associated with this shape.
leConnectivityAnchor	bool	Read-only	Indicates if the object is a connectivity anchor.
purposeNum	int	Read/Write	The purpose number associated with this shape.
net	<i>oaNet</i>	Read-only	The net this shape implements. Returns NULL if this figure does not implement a net, NULL is returned. Note that this function will return a net if the shape is directly assigned to a net or if the shape is in a route that is assigned to a net. It will not return a net (it will return NULL) if the shape belongs to a pin.
pin	<i>oaPin</i>	Read-only	The pin this shape implements. If this figure does not implement a pin, NULL is returned.

oaTerm

Represents scalar and multi-bit terminals. Terminals represent the logical connection points for a block. The pins associated with terminals represent the physical connection

points. The nets associated with the terminals are logically exported through the terminals to the next higher level in a design hierarchy. Use the `db::getAttr` command to retrieve the `oaTerm` attributes described in the following table.

Table 137 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the terminal.
isImplicit	bool	Read-only	A boolean value that indicates if this terminal is implicit.
isInterface	bool	Read/Write	A boolean indicating whether this terminal is an interface terminal.
net	<i>oaNet</i>	Read-only	The net to which this terminal is attached.
numBits	int	Read-only	The number of bits in this terminal.
termType	<i>oaTermTypeEnum</i>	Read/Write	The terminal type of this terminal.
position	int	Read/Write	The position that this terminal is assigned to.

oaVia

Represents vias. A via represents the physical connection between two adjacent route pathSegs on different layers. A via may be either a standard via or a custom via. A standard via references a via definition (`oaViaDef`) from the technology database that corresponds to the design where the via exists. A standard via has a pre-defined set of via parameters that can be tailored to create a variety of vias to cover a variety of connections. The standard via references a standard via definition. A custom viareferences another design. Custom vias often include a set of parameter values that can be used to tailor the master for each specific via. Although vias are normally associated with routes, a via can represent complex pin figures. Vias always have an `oaViaHeader` that contains the attributes that are constant for all vias of a given via definition.

Table 138 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the corresponding oaViaDef.
design	oaDesign	Read-only	The design in which the object exists.
header	oaViaHeader	Read-only	The viaHeader associated with this via.
layer1Num	int	Read-only	The first layer associated with oaViaDef.
layer2Num	int	Read-only	The second layer associated with oaViaDef.
leConnectivityAnchor	bool	Read-only	Indicates if the object is a connectivity anchor.
master	oaDesign	Read-only	Master design for custom vias. Returns NULL if unbound or for standard vias.
purposeNum	int	Read-only	The purpose number associated with this via.
viaDef	oaViaDef	Read-only	The viaDef associated with this via.

Preferences

Table 139 lists Custom Compiler le namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 139 Layout Editing Preferences

Name	Type	Scope
emRootDir	string	cellview
le3DICArgs	string	user
leAbstractAntennaAreaFactor	string	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leAbstractAntennaBdry	bool	user
leAbstractAntennaExportStream	bool	user
leAbstractAntennaGdsFile	string	user
leAbstractAntennaGdsMode	bool	user
leAbstractAntennaMode	bool	user
leAbstractAntennaModel2LayerName	string	user
leAbstractAntennaModel3LayerName	string	user
leAbstractAntennaModel4LayerName	string	user
leAbstractAntennaRunDirNameTemplate	string	user
leAbstractAssignPinTypeForBoundaryPin	bool	user
leAbstractAutoGenerateRunset	bool	user
leAbstractBlockageDepth	int	user
leAbstractCellSubType	abstract_cell_sub_type	user
leAbstractCellType	abstract_cell_type	user
leAbstractColorLock	bool	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leAbstractConstraintGroups	string	user
leAbstractCustomLayerMapFile	string	user
leAbstractCustomRunsetFile	string	user
leAbstractDbuPerUU	string	user
leAbstractEnablePinCutChannel	bool	user
leAbstractExcludedBlockageLayers	string	user
leAbstractExtendedPinDepthMode	bool	user
leAbstractExtendedPinMode	bool	user
leAbstractExtendedPinNetMode	bool	user
leAbstractExtendedPinNets	string	user
leAbstractExtendedPinSpecifyDepth	int	user
leAbstractGenDefaultView	string	user
leAbstractGenOptionSetDefault	string	user
leAbstractIgnoreTextOpen	bool	user
leAbstractIncludeCutLayer	bool	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leAbstractIncludePGNets	bool	user
leAbstractIncludePinArea	bool	user
leAbstractKeepRunResult	bool	user
leAbstractLimitLogFiles	integer	user
leAbstractLoadDefaultLayers	bool	user
leAbstractMergeOrthogonalBlockageOnly	bool	user
leAbstractNetlistFile	string	user
leAbstractPinExtractionFile	string	user
leAbstractPinExtractionMode	bool	user
leAbstractPinExtractionOption	abstract_pin_extract_option	user
leAbstractPinMerge	bool	user
leAbstractPinTypeTerminalMapping	string	user
leAbstractPVEOptions	string	user
leAbstractPVERunDir	string	user
leAbstractPVEToolName	abstract_antenna_pve_tool	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leAbstractSetFixedMask	bool	user
leAbstractSetPinTypeByTerminal	bool	user
leAbstractSetPinTypeForBoundaryPin	bool	user
leAbstractSetTermTypeByFile	bool	user
leAbstractSignalPinHSpacing	float	cellview
leAbstractSignalPinVSpacing	float	cellview
leAbstractSiteName	string	user
leAbstractSiteNameFromTech	string	user
leAbstractSiteOption	abstract_site_option	user
leAbstractSlicePolygon	bool	user
leAbstractSpecialPinHSpacing	float	cellview
leAbstractSpecialPinVSpacing	float	cellview
leAbstractSquarePinBdry	bool	user
leAbstractSymmetry	abstract_symmetry	user
leAbstractTextToPinDepth	int	user
leAbstractTrimBBdry	bool	user
leAbstractTrimCoverBlockage	bool	user
leAbstractUseCustomLayerMap	bool	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leAbstractUseExistingLayer	bool	user
leAbstractUseIntersectionAreaWithFin	bool	user
leAbstractUseMosSourceDrain	bool	user
leAbstractUseSourceCellType	bool	user
leAbstractUseSourceSymmetry	bool	user
leAbstractUseUserCutoutSpacing	bool	user
leAbstractZeroBlockageSpacing	bool	user
leAbutmentAutoFlip	bool	cellview
leAbutmentAutoRestore	bool	cellview
leAbutmentDisableCheckLPP	bool	cellview
leAbutmentEnableDummyRule	bool	cellview
leAbutmentEquivalentClasses	string	user
leAbutmentExternalConnection	abutment_external_connection	cellview
leAbutmentPerpendicularAlignment	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leActivateFigGroupMember	bool	user
leActiveLPP	string	cellview
leActiveObjectPurposePriority	string	user
leAddDummyDialogMode	bool	user
leAddDummyFigGroup	bool	cellview
leAddDummyFigGroupPrefix	string	cellview
leAddDummyFillAsRect	bool	cellview
leAddDummyNMOSGateNetList	string	cellview
leAddDummyNMOSSDNetList	string	cellview
leAddDummyPhysicalOnly	bool	cellview
leAddDummyPMOSGateNetList	string	cellview
leAddDummyPMOSSDNetList	string	cellview
leAddDummyPrefix	string	cellview
leAddDummyRule	bool	cellview
leAddDummyUsage	add_dummy_usage	cellview
leAdditionalCapacitanceReporterSetting	string	user
leAdditionalRACmd	string	library
leAdditionalStarCmd	string	library
leAdditionalXACmd	string	library
leAddResultsToSelectionSet	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leAddVirtualConnectionGenerateLabel	bool	cellview
leAddVirtualConnectionGeneratePin	bool	cellview
leAdjustCursorDialogMode	bool	user
leAdjustCursorDistance	string	cellview
leAdjustCursorDistanceList	string	cellview
leAdjustCursorKeepLastEnterDistance	bool	cellview
leAlignAngleApplySpacing	bool	cellview
leAlignAngleDialogMode	bool	user
leAlignAngleOnlySameNet	bool	cellview
leAlignAngleOrthogonalSpacing	string	cellview
leAlignAngleOrthogonalSpacings	string	cellview
leAlignAngleSpacing	string	cellview
leAlignAngleSpacings	string	cellview
leAlignApplyDistance	bool	cellview
leAlignApplySide	bool	cellview
leAlignAsGroup	bool	cellview
leAlignAssistCursorText	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leAlignAssistEnable	bool	cellview
leAlignAssistFindTargetEdge	bool	user
leAlignAssistInCommand	bool	user
leAlignAssistMode	enAlignAssistMode	cellview
leAlignAssistModeCoordMark	bool	cellview
leAlignAssistRegionCenter	bool	cellview
leAlignAssistRegionCenterline	bool	cellview
leAlignAssistRegionCenterlineVertex	bool	cellview
leAlignAssistRegionEdge	bool	cellview
leAlignAssistRegionEdgeCenter	bool	cellview
leAlignAssistRegionOrigin	bool	cellview
leAlignAssistRegionVertex	bool	cellview
leAlignAssistShowRefIndicator	bool	user
leAlignAssistTargetAlwaysIncludeLabel	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leAlignAssistTargetMode	bool	user
leAlignDialogMode	bool	user
leAlignDirection	align_direction	cellview
leAlignDistance	string	cellview
leAlignDistances	string	cellview
leAlignDistribute	bool	cellview
leAlignEditPostSelection	bool	user
leAlignPitch	string	cellview
leAlignPitches	string	cellview
leAlignRadius	int	user
leAlignRefMode	align_ref_mode	cellview
leAlignSelectionAxes	bool	user
leAlignSelectionFigGroup	bool	user
leAlignShowAnchorNotice	bool	user
leAlignShowNoticeLabel	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leAlignShowTargetEdgeLine	bool	user
leAlignSide	align_side	cellview
leAlignSpacingMode	align_spacing_mode	cellview
leAlignUseObjectRef	bool	user
leAllowedTechFoundryConstraintGroups	string	cellview
leAlwaysDrawHighlights	bool	user
leAngleLabelSize	angle_label_size_e	cellview
leAnglePrecision	integer	cellview
leAngleShowAsTip	bool	cellview
leAngleUnit	angle_unit_e	cellview
leAnnotateColor	string	user
leAnyLayerSteiner	bool	cellview
leApplyPartialSelection	bool	cellview
leAttach	bool	cellview
leAttributeListSelector	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leAttributeListSelectorValues	string	user
leAttributeSelector	string	cellview
leAttributeSelectorValues	string	user
leAttrTextViewOnly	bool	user
leAutoAbutment	bool	cellview
leAutoAddResistanceQuery	bool	cellview
leAutoChopGuardRing	bool	user
leAutoChopGuardRingMethod	auto_chop_guard_ring_method	cellview
leAutoChopGuardRingRegionType	auto_chop_guard_ring_region_type	cellview
leAutoChopGuardRingSpacing	float	cellview
leAutoDefGRStyle	int	cellview
leAutoDetermineRegionConstraintGroupOrder	bool	user
leAutoEraseLayer	bool	cellview
leAutoFixArea	operation_mode	cellview
leAutoFixAutoExecute	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leAutoFixInsertJogs	string	cellview
leAutoFixInstStopLevelMode	enMode	cellview
leAutoFixLockInstances	bool	context
leAutoFixLockInvisibleShapes	bool	context
leAutoFixLockVias	bool	context
leAutoFixOversizeWindow	float	cellview
leAutoFlipHorizontalDialogMode	bool	user
leAutoFlipVerticalDialogMode	bool	user
leAutoGRCenterLineNotch	bool	cellview
leAutoGRJogValue	float	user
leAutoHaloOverlap	bool	cellview
leAutoHierConnect	bool	cellview
leAutoMergePaths	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leAutoMergePathsLayers	string	cellview
leAutoMGRBodyWidth	float	cellview
leAutoMGRBodyWidthMethod	int	cellview
leAutoMGRCutRow	int	cellview
leAutoPickUpNamesFromDN	bool	user
leAutoPopUpConsole	string	user
leAutoRemoveUnboundInstanceTerms	bool	user
leAutoRotate90DialogMode	bool	user
leAutoStackedViaHonorWidthConstraints	bool	user
leAutoTap	bool	cellview
leAutoTerminate	bool	cellview
leAutoTriggerRejectingEventAfterAbutment	bool	cellview
leAutoViaAllOverlappedAreas	bool	cellview
leAutoViaAllowExceed	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leAutoViaAllowHammerHead	bool	cellview
leAutoViaAllowStackedVia	bool	cellview
leAutoViaApplyMinCuts	bool	cellview
leAutoViaAutoBridgeVia	bool	user
leAutoViaAwareBlockage	bool	cellview
leAutoViaAwareCutColor	bool	user
leAutoViaAwareNonvisibleLayers	bool	cellview
leAutoViaAwareNonvisibleLayers	bool	cellview
leAutoViaAwareTrack	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leAutoViaCheckHierarchyNet	bool	cellview
leAutoViaCreateMarkers	bool	cellview
leAutoViaCutLayersWithoutCutPattern	string	user
leAutoViaCutPlacement	leAutoViaCutPlacement	cellview
leAutoViaDepth	text_depth	cellview
leAutoViaEnvAware	bool	user
leAutoViaExtendLengthDirection	bool	cellview
leAutoViaIgnoreConstraints	string	cellview
leAutoViaIgnoreForbiddenETERulesLayers	string	cellview
leAutoViaJustification	string	cellview
leAutoViaMaxCuts	bool	user
leAutoViaMinCuts	int	cellview
leAutoViaMinCutsDirection	string	cellview
leAutoViaNetNames	string	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leAutoViaNotification	bool	cellview
leAutoViaOddCycleHaloFactor	float	cellview
leAutoViaOnAreasWithoutViasOnly	bool	user
leAutoViaOnIntersectionOnly	bool	user
leAutoViaOnlyLargestCutAllowed	bool	user
leAutoViaPatternStyle	string	cellview
leAutoViaSameNetOnly	bool	cellview
leAutoViaSameNetOnlyNoNetNoNet	bool	cellview
leAutoViaSameNetOnlyVisible	bool	user
leAutoViaSetting	string	cellview
leAutoViaShowEnclosure	bool	cellview
leAutoViaShowHammerHeadOption	bool	user
leAutoViaShowMissingViaMessage	bool	user
leAutoViaShowSpacing	bool	cellview
leAutoViaUsePattern	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leAutoViaVpeDebugger	bool	cellview
leAutoViaVpeDebuggerVerbose	string	cellview
leAutoViaWithinOverlappedArea	bool	cellview
leAutoViaWithBridgeLayerShape	bool	cellview
leAutoViaWithinOverlappedArea	bool	cellview
leBlockageHasWidth	bool	cellview
leBlockageLayer	string	cellview
leBlockageType	string	cellview
leBlockageWidth	float	cellview
leBodyWidth	bool	cellview
leBooleanKeepColor	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leBooleanKeepNet	bool	cellview
leBooleanOperationMode	bool_oper	cellview
leBooleanResultMode	bool_result	cellview
leBoundaryEndShapes	bool	cellview
leBoundaryLocked	bool	user
leBoundaryMinNotch	float	cellview
leBoundaryRelaxStepY	string	user
leBoundaryStepYValues	string	user
leBoundaryType	boundary_type	cellview
leBridgeInterconnectDirection	int	cellview
leBridgeLayer	string	cellview
leBridgeViaPitch	float	cellview
leBusBits	int	cellview
leBusDefaultSpacing	float	cellview
leBusDefineMultipleWidthsSpacings	bool	cellview
leBusExtendLengthMatching	bool	cellview
leBusExtendMeshMode	bool	cellview
leBusExtendLPPMode	bus_extend_lpp_mode	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leBusExtendMergeWire	bool	cellView
leBusExtendMeshMode	bool	cellview
leBusExtendMiddleAlign	bus_extend_middle_align	cellview
leBusExtendMiddleLayer	string	cellview
leBusExtendSelectTarget	bus_extend_select	cellview
leBusExtendSpacing	string	cellview
leBusExtendSpecifyWidthSpacing	bool	cellview
leBusExtendTrimEndingEdge	trim_endection	cellview
leBusExtendUsePolygon	bool	cellView
leBusExtendViaTemplate1	bus_extend_via_template1	cellView
leBusExtendViaTemplate2	bus_extend_via_template1	cellview
leBusExtendWidth	string	cellview
leBusHorizontalSpacing	float	cellview
leBusMultipleLPPs	bool	cellview
leBusMultipleSpacings	string	cellview
leBusMultipleWidths	string	cellview
leBusSpacing	float	cellview
leBusVerticalSpacing	float	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leBusViaTemplate	bus_via_template	cellview
leCanvasDeltaOnEditDesign	bool	user
leCapacitanceFilterType	string	cellview
leCapacitanceFilterValue	float	cellview
leCapacitanceMaxFilterValue	float	cellview
leCapacitanceModel	string	cellview
leCapacitanceRangeType	string	cellview
leCapReporterTopLevelOnly	bool	cellview
leCDFDefaultValueOverride	string	user
leCEConstraintGroups	string	user
leCEDepth	int	cellview
leCEInitialPropagate	bool	cellview
leCellViewLCV	lcv	cellview
leCEPseudoParallel	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCEReloadAfter	unsigned int	cellView
leCERuntimePropagation	bool	cellview
leCheckCongestionDialogMode	bool	user
leCheckDRCInViewRegion	bool	cellview
leCheckEncloseViaMode	enclose_via_check_mode	cellview
leCheckFloatingShield	bool	cellview
leCheckLengthForCommonPath	bool	cellview
leCheckPGDistance	bool	cellview
leCheckTandemAggressorForParallelShielding	bool	cellview
leCheckViaArrayForCoaxialMode	via_array_check_mode	cellview
leCheckViasAllowExceed	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCheckViasCutNumber	bool	cellview
leCheckViasDepth	via_checker_depth	cellview
leCheckViasMinCutNumber	int	cellview
leCheckViasStackingRule	bool	user
leCheckVoltageAnnotationsDialogMode	bool	user
leChooseCustomViaFirst	bool	cellview
leChopArrayOverlapped	bool	cellview
leChopCopy	bool	cellview
leChopKeep	bool	cellview
leChopShapeType	edit_shape_type	cellview
leCircularRulerPacket	string	cellview
leCircularRulerPredefined	string	cellview
leClipboardDepth	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leClipboardSnapGrid	clipboard_snap_grid	cellview
leClipboardUseDesignOrigin	bool	user
leClipTrackRegions	bool	user
leCloneMasterCellName	string	cellview
leCloneMasterDialogMode	bool	user
leColorDepth	int	cellview
leColorLegendOrientation	density_color_legend_orientation	user
leColorLegendRangeUpdate	bool	user
leColorName	string	cellview
leColorObjects	color_object_option	cellview
leColorPropagation	bool	cellview
leColorPropagationDirection	auto	user
leColorWithLock	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCommitPolyline	bool	user
leCompactBusAdjacentBit	bool	user
leCompactBusDialogMode	bool	user
leCompactBusMode	string	cellView
leCompatibleTrack	leCompatibleTrack	user
leCongestionDisplayLayer	string	cellview
leCongestionDisplayLayers	string	user
leCongestionDisplayMode	string	cellview
leCongestionExportError	bool	cellview
leCongestionOverflowLineWidth	int	cellview
leCongestionOverflowThreshold	int	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCongestionShadowMode	bool	cellview
leConsiderFollowers	bool	user
leConstraintLevel	constraint_level	cellview
leContextSensitiveCursor	bool	cellview
leConvertToArrayDialogMode	bool	user
leConvertToArrayMatchAllProperties	bool	cellview
leConvertToArrayMode	<enConvertToArrayMode>	cellview
leConvertToArrayStyle	<enConvertToArrayStyle>	cellview
leConvertToPathMode	string	cellview
leConvertToPathZoom	bool	cellview
leCopyAutoUpdateDXDY	bool	cellview
leCopyColRowMode	string	cellview
leCopyCols	int	cellview
leCopyDefaultsDXDY	bool	cellview
leCopyDestroySource	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCopyDialogMode	bool	user
leCopyDistanceMethod	string	cellview
leCopyDX	float	cellview
leCopyDXDYMode	copy_dxdy_mode	cellview
leCopyDY	float	cellview
leCopyElectricalSimulationToConfig	bool	user
leCopyKeepAttachment	bool	cellview
leCopyKeepColor	bool	cellview
leCopyKeepCopyToClipboard	bool	cellview
leCopyKeepHighlight	bool	cellview
leCopyKeepInSyncGroup	bool	cellview
leCopyKeepMember	bool	cellview
leCopyKeepNet	bool	cellview
leCopyKeepSync	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCopyLPP	bool	cellview
leCopyLPPMethod	string	cellview
leCopyLPPSpecify	string	cellview
leCopyResetGroup	bool	cellview
leCopyResetRowsCols	bool	cellview
leCopyRows	int	cellview
leCorrectOffGridAutoZoom	bool	cellview
leCorrectOffGridDialogMode	bool	user
leCorrectOffGridGridType	grid_type	cellview
leCorrectOffGridShowByInstance	bool	cellview
leCorrectOffGridShowByShape	bool	cellview
leCorridorDrawingMode	corridorDrawingMode	cellview
leCorridorLayerValues	string	user
leCorridorMaxLayer	string	user
leCorridorMinLayer	string	user
leCorridorSelectNew	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCorridorWidth	float	cellview
leCreateAngleMode	edit_angle_mode	cellview
leCreateAttributeLabelDialogMode	bool	user
leCreateBlockageDialogMode	bool	user
leCreateBlockageShowAllLayers	bool	cellview
leCreateBoundaryDialogMode	bool	user
leCreateBusMode	bus_mode	cellView
leCreateBusSelectBy	bus_select_by	cellview
leCreateCellViewFromSelectionDialogMode	bool	user
leCreateDegassingHoleDialogMode	bool	user
leCreateEllipseDialogMode	bool	user
leCreateFigGroupDialogMode	bool	user
leCreateFRAMAdjacentRows	string	user
leCreateFRAMCell	string	user
leCreateFRAMCellType	string	user
leCreateFRAMFirstLayerParallel	string	user
leCreateFRAMFirstLayerPerpendicular	string	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCreateFRAMLayerBlockage	string	user
leCreateFRAMMergeBlockage	bool	user
leCreateFRAMOpenOnCompletion	bool	user
leCreateFRAMOverwriteExisting	bool	user
leCreateFRAMPGOrient	string	user
leCreateFRAMPreserveAllMetalBlockage	bool	user
leCreateFRAMScript	string	user
leCreateFRAMSiteDef	string	user
leCreateFRAMSmashLevel	int	user
leCreateFRAMSource	lcv	user
leCreateFRAMTargetICCLibraryPath	string	user
leCreateFRAMView	string	user
leCreateFRAMViewer	string	user
leCreateGlobalTrackRegionDialogMode	bool	user
leCreateGRNetNameList	string	cellview
leCreateInstDialog	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCreateInstDialogMode	bool	user
leCreateInstDrawRubberBand	bool	user
leCreateInstUsingTabledLCV	bool	user
leCreateInterconnectDialogMode	bool	user
leCreateLabelDialog	bool	user
leCreateLabelDialogMode	bool	user
leCreateLockColor	bool	cellview
leCreateMPPDialogMode	bool	user
leCreateOrientation	string	cellview
leCreatePinDialog	bool	user
leCreatePinDialogMode	bool	user
leCreatePinsFromSelectionDialogMode	bool	user
leCreatePlaneBoundaryDialogMode	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCreatePlaneDialogMode	bool	user
leCreatePlaneShapeType	info_create_plane_shape_type	cellview
leCreatePolygonDialogMode	bool	user
leCreateRectangleDialogMode	bool	user
leCreateRulerAutoTextJustification	bool	cellview
leCreateRulerDialogMode	bool	user
leCreateRulerDiscontinuousMeasurement	bool	user
leCreateRulerDirection	enDirection	user
leCreateRulerDiscontinuousMeasurement	bool	user
leCreateRulerIncludeExtension	bool	cellview
leCreateRulerLabelSize	create_ruler_label_size	user
leCreateRulerMeasurebyLine	bool	user
leCreateRulerMeasureKeepDistance	int	user
leCreateRulerMeasureLPP	string	user
leCreateRulerMeasureLPPMode	create_ruler_measure_lpp_mode	cellview
leCreateRulerMeasureSpace	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCreateRulerMeasureSpacing	bool	user
leCreateRulerMeasureWidth	bool	user
leCreateRulerMode	string	cellview
leCreateRulerPacket	string	cellview
leCreateRulerPointsMode	string	cellview
leCreateRulerSnapEdge	bool	user
leCreateRulerSnapMidpoint	bool	user
leCreateRulerStyle	create_ruler_style	user
leCreateRulerTextJustification	string	cellview
leCreateRuntimeDesignFiltered	bool	user
leCreateRuntimeDesignFilteredNumber	int	user
leCreateShapeType	create_shape_type	cellview
leCreateShapeWithAlignmentBoundary	bool	cellview
leCreateTrackRegionPCellDialogMode	bool	user
leCreateTransmissionLineDialogMode	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCreateViaDialogMode	bool	user
leCreateViaJustification	string	cellview
leCreateViaOrigin	bool	cellview
leCreateVoltageConfigDialogMode	bool	user
leCreateVoltagePhysicalAnnotationsDialogMode	bool	user
leCurrentScalingFactor	float	cellview
leCustomViaJustificationType	custom_via_justification_type	cellview
leCutClassAlignMode	string	user
leCutClassAlignPreferSmall	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leCutCornerOneShot	bool	cellview
leCutLength	float	cellview
leCutRemainAsPath	bool	cellview
leCutSides	int	cellview
leCycleFlightlinePackets	bool	cellview
leCycleName	bool	cellview
leCycleRulerPackets	bool	cellview
leDDActiveElectricalInfo	bool	user
leDDActiveObjectInfo	bool	user
leDDCanvasDist	bool	user
leDDCanvasDXDY	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDDCanvasSnapPoint	bool	user
leDDCanvasTopLevelPoint	bool	user
leDDCountSelection	bool	user
leDDCursorTextDXDYPositive	bool	user
leDDDrcSpacing	bool	user
leDDEllipseArea	bool	user
leDDEllipseCap	bool	user
leDDEllipsePerim	bool	user
leDDEllipseWidthHeight	bool	user
leDDPathArea	bool	user
leDDPathCap	bool	user
leDDPathLength	bool	user
leDDPathRes	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDDPathWidth	bool	user
leDDPolyArea	bool	user
leDDPolyCap	bool	user
leDDPolyPerim	bool	user
leDDRectArea	bool	user
leDDRectCap	bool	user
leDDRectPerim	bool	user
leDDRectWidthHeight	bool	user
leDDRefStretchHandle	bool	user
leDefaultBusSpacing	float	user
leDefaultPathWidth	float	user
leDefaultTemperatureFactor	float	cellview
leDefineInFOLibraryDialogMode	bool	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leDefinePackagingLibraryDialogMode	bool	user
leDefineTerminalsFillingSize	int	user
leDeleteCutMetalLayer	string	cellview
leDeleteCutMetalShapeBelowCut	bool	cellview
leDeleteLogicalWhen	string	cellview
leDeleteNested	bool	cellview
leDeletePartialMode	delete_partial_mode	cellview
leDeltaTemperature	float	cellview
leDenseLineAboveLayerILO	string	cellview
leDenseLineAboveLayerILS	int	cellview
leDenseLineAboveLayerNumber	int	cellview
leDenseLineBelowLayerILO	string	cellview
leDenseLineBelowLayerILS	int	cellview
leDenseLineBelowLayerNumber	int	cellview
leDenseLineFromSignalSpacing	float	cellview
leDenseLineSameLayerLeftNumber	int	cellview
leDenseLineSameLayerRightNumber	int	cellview
leDenseLineUserDefinedLength	float	cellview
leDenseLineUserDefinedSpacing	float	cellview
leDenseLineUserDefinedWidth	float	cellview
leDensityActivePurposes	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDensityCalculateGroup	string	cellview
leDensityCalculateLayer	string	cellview
leDensityCheckBox	string	cellview
leDensityCheckRegionType	DensityRegionType	cellview
leDensityConfigManual	DensityConfigManual	cellview
leDensityExcludedLPPs	string	cellview
leDensityGroupMethod	DensityMethod	cellview
leDensityHistoryExcludedLPPs	string	cellview
leDensityHistoryPurposes	string	cellview
leDensityLayerList	string	user
leDensityLowRange	float	cellview
leDensityOpacity	int	cellview
leDensityRedrawCategory	string	cellview
leDensityRedrawColor	string	cellview
leDensityRedrawGroup	string	cellview
leDensityRedrawLayer	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDensityStepSize	real	cellview
leDensityWindowSize	real	cellview
leDensityWindowType	DensityWindowType	cellview
leDesignSummaryApplyTo	design_summary_apply_to	user
leDesignSummaryCombineInstanceStatistics	bool	user
leDesignSummaryDepth	design_summary_depth	user
leDesignSummaryPreservePcells	bool	user
leDesignSummaryStyle	design_summary_style	user
leDesignSummaryUserDefinedValue	int	user
leDetectShorts	bool	cellview
leDetectShortsOnOpen	bool	cellview
leDevicePaletteCategory	string	context
leDiagonalWidthMethod	string	user
leDigitalMCMMSetupFile	string	user
leDigitalReportsDir	string	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDigitalSDCFile	string	user
leDimUnselected	bool	user
leDirectEditStaticIndex	bool	user
leDirectManipulation	bool	user
leDirectManipulationSelectedOnly	bool	user
leDisableSyncClonePrompt	bool	user
leDisplayDynamicFlightlines	bool	cellview
leDisplayFlightlineNetName	bool	cellview
leDisplayFlightlines	bool	cellview
leDisplayGridBasis	string	cellview
leDisplayInCommandTrackPatterns	bool	cellview
leDistanceAngleMode	string	cellview
leDistanceAuto	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDistanceConvertType	string	cellview
leDistanceDirection	string	cellview
leDistanceEdgeToEdge	bool	cellview
leDistributeAlignDialogMode	bool	user
leDistributeApplyDistance	bool	cellview
leDistributeDistance	string	cellview
leDistributeDistanceHistory	string	cellview
leDistributeFirstObject	bool	cellview
leDistributeLPP	string	cellview
leDistributeSort	bool	cellview
leDistributeSortMethod	distribute_sort_method	cellview
leDistributeSpecifyLPP	bool	cellview
leDistributeValueSet	string	cellview
leDistributeValueSetFile	string	user
leDragThreshold	int	user
leDraw1xPinsInsideFig	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDrawAllFigsPinLabels	bool	user
leDrawAllRulers	bool	user
leDrawArrayInstBorderOnly	bool	user
leDrawArrayInstMemberBoundary	bool	user
leDrawAxisMethod	drawAxisMethod	user
leDrawConnectivityLabelsAllLayers	bool	user
leDrawCurrentLevelMarkersOnly	bool	user
leDrawDirectionIndicator	bool	user
leDrawEIPBoundary	bool	user
leDrawFilter	int	user
leDrawHalfRuler	bool	user
leDrawInstLabelWhenSelected	bool	user
leDrawInstOrigin	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDrawInstPins	bool	user
leDrawInstPinsWhenHighlighted	bool	user
leDrawMergedHighlightedShapes	bool	user
leDrawMergedShapes	bool	cellview
leDrawMergedShapesMode	shapesOnSameNet	user
leDrawNonPropagatingShapes	bool	user
leDrawPathLabel	bool	user
leDrawPathLabelFullDepth	bool	user
leDrawPinLabel	drawPinLabel	user
leDrawPinLabelFullDepth	bool	user
leDrawRuntimeInstBoundary	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDrawTermLabel	bool	user
leDrawTermLabelsOnAllFigs	bool	user
leDrawTextOrigin	bool	user
leDrawViaOrigin	bool	user
leDrawZoomInvariantFlightlineText	bool	user
leDRCFilter	string	cellview
leDRCMode	enDrcMode	cellview
leDRCModeForceSlide	bool	cellview
leDRDAssistThreshold	float	cellview
leDRDAutoAssignColor	bool	cellview
leDRDCheckBlockage	bool	cellview
leDRDCheckOnModification	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leDRDCheckRange	float	cellview
leDRDCheckRuleSet	string	cellview
leDRDColorDensity	string	cellview
leDRDColorGeneration	string	cellview
leDRDConstraintFilters	<std::string>	cellview
leDRDConstraintGroup	string	cellview
leDRDDynamicRuler	bool	cellview
leDRDDynamicRulers	bool	cellview
leDRDDynamicRulerThreshold	int	cellview
leDRDDynmicMaxShapeCount	int	user
leDRDEnablePushThrough	bool	cellview
leDRDGenerateFixingGuide	bool	cellview
leDRDGenerateFixingGuideInInstance	bool	cellview
leDRDGroupFilter	string	cellview
leDRDLayerFilter	string	cellview
leDRDMaxRulerCount	int	cellview
leDRDMaxRulerRange	int	cellview
leDRDMaxThreshold	float	cellview
leDRDMinRulerRange	int	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDRDObjectAssociation	int	cellview
leDRDOutputPartialResult	bool	cellview
leDRDPreferredRuleConstraintGroup	string	cellview
leDRDPTAsPixels	int	cellview
leDRDPTAsRuleValues	float	cellview
leDRDReboundingRadius	int	cellview
leDRDRuleFile	string	cellview
leDRDRuleFilter	string	cellview
leDRDRulerDisplayingRadius	int	cellview
leDRDRuleSets	string	cellview
leDRDRuleSource	string	cellview
leDRDShowErrorInSelectedObject	bool	cellview
leDRDShowRuleNameWhenCgRuleSource	bool	cellview
leDRDTool	string	user
leDRDUseICVLive	bool	user
leDRDVisualColor	bool	cellview
leDRDVisualHalo	float	cellview
leDRDVisualTimer	int	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leDRDWindowCheckThreshold	string	cellview
leDynamicFlightlineLineStyle	string	cellview
leDynamicMeasurement	bool	user
leEditableCellEval	int	user
leEditableCellFromRepository	bool	user
leEditableCellRecovery	bool	user
leEditableCellRepository	bool	user
leEditableCellSelectionStatus	bool	user
leEditablePcellCache	bool	user
leEditablePcellRecovery	bool	user
leEditablePcellRepository	bool	user
leEditAngleMode	edit_angle_mode	cellview
leEditCutLength	float	cellview
leEditCutSides	int	cellview
leEditCutSidesFlag	bool	cellview
leEditShapeType	edit_shape_type	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leEditViaParamConstraintAwareMode	enum	user
leEditViaUpdateSpacing	enum	user
leEIPUseTopDesignCoord	bool	user
leElectricalEMRule	string	cellview
leElectricalReporterAutoHighlight	bool	cellview
leElectricalReporterCapGroup	string	cellview
leElectricalReporterElmoreDelay	bool	cellview
leElectricalReporterEMCompare	string	cell
leElectricalReporterEmdbOccurrence	string	cellview
leElectricalReporterEmdbPath	string	cell
leElectricalReporterEMFilter	string	cellview
leElectricalReporterEMFrom	string	cell
leElectricalReporterEMMode	string	cellview
leElectricalReporterEMTo	String	cell
leElectricalReporterElmoreDelay	bool	cellview
leElectricalReporterShadowMode	bool	cellview
leElectricalReporterTraceParents	bool	cellview
leElectricalReporterVdrcdbOccurrence	string	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leElectricalReporterVdrcdbPath	string	cell
leElectricalReporterViewDepth	string	cellview
leElectricalReporterZoom	bool	cellview
leElectricalResistanceReportType	string	cellview
leElectricalResistanceRule	string	cellview
leEllipseAsCircle	bool	cellview
leEMAnalysisTemperature	float	cellview
leEMCalcTabs	bool	user
leEMCalculationMode	calculateWidth	user
leEMCalculationType	avg	user
leEMColorValue	string	user
leEMConstraintModel	string	cellview
leEMCurrentDirection	string	user
leEMCurrentValue	float	user
leEMCutClass	string	user
leEMLayerValue	string	user
leEMLengthMode	calculateWidth	user
leEMLengthValue	float	user
leEMLengthValueFromLayout	float	user
leEMLowerLayerLength	float	user
leEMLowerLayerWidth	float	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leEMRCAutoSaveHistory	bool	cellview
leEMRCDebugMode	bool	user
leEMRCEnableSingleNetInBackground	bool	cellview
leEMRCExportToErrorViewer	bool	cellview
leEMRCMaxHistoryNum	int	cellview
leEMRCReporterMaxDetailDisplayNumber	int	user
leEMRCResultFilePath	string	user
leEMRCResultOpenOnCompletion	bool	cellview
leEMRCShareStarRCJob	bool	user
leReportResistanceSelectUsingType	string	cellview
leEMRCTraceAreaType	string	cellview
leEMRCTraceRegion	string	cellview
leEMReporterDisplayNumber	int	user
leEMSelfHeatingFlow	bool	cellview
leEMTemperature	string	user
leEMUpperLayerLength	float	user
leEMUpperLayerWidth	float	user
leEMViaColumnNumber	float	user
leEMViaCurrentValue	float	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leEMViaLayerValue	string	user
leEMViaRowNumber	float	user
leEMViolationThreshold	float	cellview
leEMWarningThreshold	float	cellview
leEMWidthValue	float	user
leEnableCE	bool	cellview
leEnableFlightlineJogs	bool	cellview
leEnableModifyAabut	bool	cellview
leEnableNonPcellAabutment	bool	user
leEnablePermutePins	bool	cellview
leEnableRestrictedSelection	user	cellview
leEnableZoom	bool	cellview
leEnforceShieldMaxGapCheck	bool	cellview
leEraseLayerColor	string	cellview
leEraseLayerColorValues	string	cellview
leEraseLayerRemoveShapeBelowCut	bool	cellview
leExtendSelectionPathMatchWidth	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leExtendSelectionPaths	bool	user
leExtendSelectionPathSearchMethod	path_seg_query_types	user
leExtendSelectionPathSeg	bool	user
leExtensionSelectionPathSearchMethod	string	user
leFCArgs	string	user
leFigGroupMemberStatus	fig_group	cellview
leFigGroupName	string	cellview
leFigGroupStatus	fig_group	cellview
leFillArea	fill_area	cellview
leFillConvertArrayInst	bool	cellview
leFillDialogMode	bool	user
leFillDistribute	bool	cellview
leFillEnclosure	string	cellview
leFillEnclosureList	string	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leFillExclude	bool	cellview
leFillExcludeHalo	string	cellview
leFillExcludeHaloList	string	cellview
leFillExcludeLPPs	string	cellview
leFillExcludeLPPsList	string	cellview
leFillJustification	string	cellview
leFillLCV	lcv	cellview
leFillLCVList	string	cellview
leFillNewCellView	bool	cellview
leFillNewCellViewName	string	cellview
leFillOrientation	string	cellview
leFillShowTilingHighlight	bool	user
leFillSpacing	string	cellview
leFillSpacingList	string	cellview
leFillTiling	bool	cellview
leFilterAntennaValueZero	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leFinPatternShadowBrightness	int	user
leFlattenDialogMode	bool	user
leFlightlineDisplayFilter	string	cellview
leFlightlineDisplayType	string	cellview
leFlightlineHighlightFiguresOnNet	bool	cellview
leFlightlineName	string	cellview
leFlightlinePacket	string	cellview
leFlightlinesBySelectionPhysicalOnly	bool	cellview
leFlightlineSyncSelection	bool	cellview
leFlightlineTypeFilter	string	cellview
leFlipWideWireColor	bool	cellview
leGatherBusAlignMode	gather_align_mode	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leGatherBusJogSpacing	float	cellview
leGatherBusJogSpacingMode	gather_jog_spacing_mode	cellview
leGatherSpreadBusLPPs	string	cellview
leGatherSpreadBusMode	gather_mode	cellview
leGatherSpreadBusSpacing	float	cellview
leGatherSpreadBusTarget	gather_target	cellview
leGenerateCutMetalArea	generate_cut_metal_area	cellview
leGenerateCutMetalBlockagePurpose	string	user
leGenerateCutMetalBoundaryLPP	string	cellview
leGenerateCutMetalConnectedMetalToTop	bool	user
leGenerateCutMetalCutPurpose	string	cellview
leGenerateCutMetalCustomCell	bool	cellview
leGenerateCutMetalDialogMode	bool	user
leGenerateCutMetalEraseLength	string	cellview
leGenerateCutMetalEraseSpacing	string	cellview
leGenerateCutMetalEraseWidth	string	cellview
leGenerateCutMetalFindBelowShapeMode	leGenerateCutMetalFindBelowShapeMode	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leGenerateCutMetalGenerateInstBoundary	bool	cellview
leGenerateCutMetalHalfMetalGeneration	bool	user
leGenerateCutMetalIncrementalMode	bool	cellview
leGenerateCutMetalLPP	string	cellview
leGenerateCutMetalMaxDepth	integer	cellview
leGenerateCutMetalMode	generate_cut_metal_mode	cellview
leGenerateCutMetalOnExtendedMetalBelowCut	bool	user
leGenerateCutMetalOnTop	bool	cellview
leGenerateCutMetalShapeBelowCut	bool	cellview
leGenerateCutMetalSideFile	string	cellview
leGenerateCutMetalSnapToTrack	bool	user
leGenerateCutMetalSpacing	string	cellview
leGenerateCutMetalSpacingSameWithCutWidth	bool	user
leGenerateCutMetalSpecifyBoundaryLPP	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leGenerateCutMetalTrackDir	topology_bone_direction	cellview
leGenerateGuardRingDialogMode	bool	user
leGenerateLabelArea	generate_label_area	cellview
leGenerateLabelAttach	bool	cellview
leGenerateLabelByInst	bool	cellview
leGenerateLabelByNet	bool	cellview
leGenerateLabelByTerm	bool	cellview
leGenerateLabelDialogMode	bool	user
leGenerateLabelIgnoreShort	bool	cellview
leGenerateLabelIncludeGuardRing	bool	cellview
leGenerateLabelInstLPP	string	cellview
leGenerateLabelPerGate	bool	cellview
leGenerateLabelTextType	generate_label_text_type	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leGenerateLabelTopMetalForIslandOnly	bool	cellview
leGenerateLabelUseVirtualConnection	bool	cellview
leGenerateMPPDialogMode	bool	user
leGeneratePinArea	generate_pin_area	cellview
leGeneratePinAssignToShape	bool	cellview
leGeneratePinAttachToInst	bool	cellview
leGeneratePinAutoPlacePin	bool	cellview
leGeneratePinAutoPlacePinCenterMode	bool	cellview
leGeneratePinByLabel	bool	cellview
leGeneratePinByNet	bool	cellview
leGeneratePinCheckLogicTerms	bool	cellview
leGeneratePinCreatePinLabel	bool	cellview
leGeneratePinCreateSquarePin	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leGeneratePinEnableMultithread	bool	cellview
leGeneratePinExcludedLayers	string	cellview
leGeneratePinExtractShapeLevel	int	cellview
leGeneratePinExtractThroughVia	bool	cellview
leGeneratePinFollowInstLabel	bool	cellview
leGeneratePinFollowInstPinLabel	bool	cellview
leGeneratePinForInvisibleLayers	bool	cellview
leGeneratePinIgnoreLabel	string	cellview
leGeneratePinIgnoreNet	bool	cellview
leGeneratePinIgnoreNetNames	string	cellview
leGeneratePinIncludeInstancePin	bool	cellview
leGeneratePinLabelStyle	generate_pin_label_style	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leGeneratePinLECheckLogicTerms	bool	cellview
leGeneratePinLPPMethod	string	cellview
leGeneratePinMapFile	string	cellview
leGeneratePinNetMode	generate_pin_net_mode	cellview
leGeneratePinPinSize	string	cellview
leGeneratePinSaveReport	bool	user
leGeneratePinSaveReport	bool	user
leGeneratePinSelectedNetNames	string	cellview
leGeneratePinShapeDepth	generate_pin_shape_depth	cellview
leGeneratePinShowPlaceStatus	bool	cellview
leGeneratePinShowReport	bool	user
leGeneratePinSkipUserCreatedPins	bool	cellview
leGeneratePinSpecifyExcludedLayers	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leGeneratePinSpecifyPinSize	bool	cellview
leGeneratePinSpecifyTopLayersNumber	bool	cellview
leGeneratePinTextDepth	int	cellview
leGeneratePinTopLayersMetalOnly	bool	cellview
leGeneratePinTopLayersNumber	int	cellview
leGeneratePinUseVirtualConnect	bool	cellview
leGeneratePlacementTrackRegionDialogMode	bool	user
leGenerateRoutingTrackRegionDialogMode	bool	user
leGenerateShapesDialogMode	bool	user
leGenLayoutBoundaryBasis	string	user
leGenLayoutBoundaryHeight	float	user
leGenLayoutBoundaryRatio	float	user
leGenLayoutBoundaryType	le_generate_layout_boundary_type	user
leGenLayoutBoundaryUtilization	float	user
leGenLayoutBoundaryWidth	float	user
leGenLayoutBusOrder	string	user
leGenLayoutConstraintGroup	string	user
leGenLayoutLeftRightPinLPP	string	user
leGenLayoutLeftRightPinLength	float	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leGenLayoutLeftRightPinPitch	float	user
leGenLayoutLeftRightPinWidth	float	user
leGenLayoutPinsLCV	lcv	user
leGenLayoutTopBottomPinLPP	string	user
leGenLayoutTopBottomPinLength	float	user
leGenLayoutTopBottomPinPitch	float	user
leGenLayoutTopBottomPinWidth	float	user
leGenShapeCommandAppliesTo	gen_shape_command_applies_to	cellview
leGenShapeDepth	gen_shape_depth	cellview
leGenShapeLPP1	string	cellview
leGenShapeLPP2	string	cellview
leGenShapeOperations	string	cellview
leGenShapeSelectNewObjects	bool	cellview
leGRAutoAbutment	bool	cellview
leGRAutoCategory	string	cellview
leGRAutoConfig	bool	cellview
leGRAutoConfigFile	string	user
leGRAutoConfigJog	bool	cellview
leGRAutoConfigNotch	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leGRAutoConfigStyle	gr_auto_def_style	cellview
leGRAutoDef	bool	cellview
leGRAutoDefBodyWidth	int	cellview
leGRAutoDefCornerCut	int	cellview
leGRAutoDefPerInstance	bool	cellview
leGRAutoDefRingCount	string	cellview
leGRAutoDefRingCountList	string	user
leGRAutoFilter	string	cellview
leGRAutoRule	string	cellview
leGRAutoStyle	gr_auto_style	cellview
leGRAutoWrapNonLayerSpacingObjects	bool	user
leGRCategoryList	string	cellview
leGRConfigRingCount	string	cellview
leGRFilterList	string	cellview
leGRRuleList	string	cellview
leGRUseConfig	bool	user
leGravity	bool	cellview
leGravityDepth	int	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leGravityEIPSurround	bool	cellview
leGravityOffGrid	bool	cellview
leGravityRadius	int	cellview
leGravityRegionCenter	bool	cellview
leGravityRegionCenterline	bool	cellview
leGravityRegionCenterlineVertex	bool	cellview
leGravityRegionEdge	bool	cellview
leGravityRegionEdgeCenter	bool	cellview
leGravityRegionOrigin	bool	cellview
leGravityRegionStretchHandle	bool	cellview
leGravityRegionVertex	bool	cellview
leGravityStretchHandles	bool	cellview
leGravityStripeWire	bool	cellview
leGRBodyWidth	float	cellview
leGRBodyWidthMethod	int	cellview
leGRContactRow	int	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leGRIndividualWrap	bool	cellview
leGRInteractive	bool	user
leGRJogValue	float	cellview
leGRLayerSpacing	string	user
leGRLCV	lcv	cellview
leGRMergeDistance	float	cellview
leGRNotchValue	float	cellview
leGRPerInst	bool	cellview
leGRRingList	string	user
leGRRings	string	cellview
leGRShowCellCategories	bool	user
leGRSpacing	float	cellview
leGRSpacingMethod	bool	cellview
leGRStyle	gr_style	cellview
leGRUsingTabledLCV	bool	user
leGuardRingChopLayers	string	cellview
leGuardRingHealLayers	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leHaloOverlapEnableDummyRule	bool	cellview
leHCBlackBoxDesigns	List of lcv	user
leHCConstraintGroups	string	user
leHCDisplayRuleTypes	string	user
leHCIgnorePurposes	string	user
leHCIncrementalUpdate	bool	user
leHCInitialColorPropagation	bool	user
leHCInitialColorPropagationTrace	bool	user
leHCNonPropagatingLayerMaterials	enMaterial	cellview
leHCOptionSet	string	cellview
leHCSelectLogicalNets	bool	user
leHCTraceAbstractCell	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leHCTraceLayers	string	cellview
leHierarchyRowSnapping	bool	user
leHierarchicalColorUpdates	bool	user
leHierarchicalCopyDescendClip	bool	cellview
leHierarchicalCopyDialogMode	bool	user
leHierarchicalCopyRemoveSource	bool	cellview
leHierarchicalCopyKeepColor	bool	cellview
leHierarchicalCopyKeepNet	bool	cellview
leHierarchicalCopyMode	hierarchical_copy_mode	cellview
leHierarchicalCopyReturnLevel	string	cellview
leHierarchicalCopyReturnLevelList	string	cellview
leHealRegionType	heal_region_type	cellview
leHealMethod	heal_method	cellview
leHCValidPurposes	string	user
leHighlightConnectedDialogMode	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leHighlightExplore	string	user
leHighlightFlightlineShapes	bool	cellview
leHighlightInstAsAbstractPacket	string	user
leHighlightPathDialogMode	bool	user
leHighlightPathFilterIsland	bool	user
leHighlightPathFullIsland	bool	user
leHighlightPathPacket	string	user
leHighlightPathPacket1	string	user
leHighlightPathPacket2	string	user
leHighlightSaveAppend	bool	user
leHighlightSaveColor	bool	user
leHighlightSaveConnectivity	bool	user
leHighlightSaveOpenInWindow	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leHighlightSaveOverlay	bool	user
leHighlightSaveReplaceByOverlay	bool	user
leHighlyResistiveLayerMaterials	string	cellview
leHonorErasingLayerConnectivity	bool	cellview
leHonorPreferredRoutingOnly	bool	cellview
leHonorRoutingDirection	bool	cellview
leHorizontalWidth	float	cellview
leICC2Args	string	user
leICCArgs	string	user
leICCBoundingBoxBoundary	string	user
leICCBoundingBoxCopyPinsViewStopList	string	user
leICCBoundingBoxGateCount	int	user
leICCBoundingBoxHeight	float	user
leICCBoundingBoxPinConstraintKeepBusesTogether	bool	user
leICCBoundingBoxPinConstraintNoStacking	enNoStacking	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leICCBlackBoxPinConstraintPreRouteSpacing	int	user
leICCBlackBoxPinContraintBusOrdering	enBusOrdering	user
leICCBlackBoxSizeMode	enBlackBoxSizeMode	user
leICCBlackBoxSupplyPinMinWidthMultiplier	int	user
leICCBlackBoxUtilization	float	user
leICCBlackBoxWidth	float	user
leICCCheckLibrary	bool	user
leICCGenerateBlackBoxTiming	bool	user
leICCGroundNetName	string	user
leICCImportLayoutCellView	lcv	user
leICCImportLayoutShapes	bool	user
leICCLinkLibrary	string	user
leICCLogicalSource	string	user
leICCLogicCellView	lcv	cellview
leICCPowerMode	enPowerMode	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leICCPowerNetName	string	user
leICCPROpenOnCompletion	bool	user
leICCSearchPath	string	user
leICCTargetLibrary	string	user
leICCTLUPlusMapFile	string	cellview
leICCTLUPlusMaxFile	string	cellview
leICCTLUPlusMinFile	string	cellview
leICCTLUPlusNominalFile	string	cellview
leICCUPFFile	string	user
leCCUserBlackBoxScript	string	user
leCCUserScript	string	user
leCCVerilogFile	string	user
leCCVerilogTopModule	string	user
leCCVerilogViewSearchList	string	user
leCCVerilogViewStopList	string	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leIgnoreBlockageTypesForAbstractGenerator	string	cellview
leIgnoreCEConnectionLimit	bool	user
leIgnoreConnectingLayers	string	cellview
leIgnoreCutLayers	string	cellview
leIgnoreEquivalentNumCut	bool	user
leIgnoreErasingLayers	string	cellview
leIgnoreNotFullyShieldedViolationWithoutAggressor	bool	cellview
leIgnoreParallelWidthCheckForCoaxialStyle	bool	cellview
leIgnorePurposes	<string>	cellView
leIgnoreRuntimeInstDepth	bool	user
leIgnoreRuntimeStipple	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leIgnoreTrackGridDefinitions	bool	user
leIgnoreUncoloredHierarchyObjects	bool	user
leIgnoreUnmappedLPP	bool	cellview
leIgnoreViaColors	bool	cellview
leIgnoreViaCutColor	bool	user
leIgnoreWidthOffGridChecking	bool	user
leImportVoltagesDialogMode	bool	user
leImportVoltagesOverwrite	bool	user
leIncludeColorNone	bool	cellview
leIncludeColorSame	bool	cellview
leInFOLayerMapFile	string	cellview
leInFOLCV	lcv	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leInFOLibraries	string	user
leInFOPlaneSpacing	float	cellview
leInFORunset	string	cellview
leInputCoordAddAnchorPoint	bool	cellview
leInputCoordDialogMode	bool	user
leInputCoordKeepLastValue	bool	cellview
leInputCoordUseRelativeCoordinate	bool	cellview
leInputCoordValue	string	cellview
leInstAddToGroup	bool	cellview
leInstanceLoopCheck	bool	user
leInstancePlacementGridSnappingPrecedence	string	cellview
leInstancePolySnappingHierarchy	leInstancePolySnappingHierarchy	user
leInstanceSelectionMode	string	cellview
leInstanceSnappingHonorComponentType	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leInstanceSnappingIncludeSnapToFin	bool	cellview
leInstanceSnappingMetalShapes	leInstanceSnappingMetalShapes	user
leInstArrayDisplay	int	user
leInstBoundary	string	user
leInstBoundaryLPP	string	user
leInstColNum	int	cellview
leInstColsRowsMode	inst_cols_rows_mode	cellview
leInstDefaultDXDY	bool	cellview
leInstDX	float	cellview
leInstDXDYMode	inst_dx_dy_mode	cellview
leInstDY	float	cellview
leInstJustification	string	cellview
leInstLabel	string	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leInstLabelDrawParameters	bool	user
leInstLabelDrawRealMasterCellName	bool	user
leInstLabelParameters	string	user
leInstLCV	lcv	cellview
leInstName	string	cellview
leInstOptionExpand	bool	cellview
leInstOrigin	bool	cellview
leInstParamExpand	bool	cellview
leInstRowNum	int	cellview
leInstType	bool	cellview
leInteractiveStarRCEngineBias	string	cellview
leInteractPRboundaryEditing	bool	cellview
leInterconnectAddViaShowEnclosure	bool	cellview
leInterconnectAddViaShowSpacing	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leInterconnectAdjustStartEnd	bool	cellview
leInterconnectAdjustVia	bool	cellview
leInterconnectAdjustWidth	bool	cellview
leInterconnectAllowExceedMaxWidth	bool	cellview
leInterconnectAutoUpdateWidth	bool	cellview
leInterconnectCycleStepWidth	bool	cellview
leInterconnectDropVia	bool	cellview
leInterconnectExtendPathEndToViaEnd	bool	cellview
leInterconnectHonorActiveLayer	bool	cellview
leInterconnectHonorDrawFilterSize	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leInterconnectIgnoreRuleRegionBoundary	bool	cellview
leInterconnectInfixModeEnabled	bool	user
leInterconnectInfixModeEnabled	bool	user
leInterconnectInsertJog	bool	cellview
leInterconnectKeepViaWidthHeight	bool	cellview
leInterconnectShortEndPath	bool	user
leInterconnectStepTaper	bool	cellview
leInterconnectStepWidth	string	cellview
leInterconnectStepWidthList	string	cellview
leInterconnectTap	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leInterconnectTaper	bool	cellview
leInterconnectTaperLength	float	cellview
leInterconnectTrackCheckBeginEndOnly	bool	cellview
leInterconnectUniformWidth	bool	cellview
leInterconnStackViaWithRoutingDir	bool	cellview
leInterLayerOrientation	string	cellview
leInterLayerSeparation	int	cellview
leInternalizeColor	bool	user
leIRCXMapFile	string	cellview
leIRCXMaxFile	string	cellview
leIRCXMinFile	string	cellview
leIRCXOperatingPoint	operating_point	cellview
leITFEMMapFile	string	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leITFEMMaxFile	string	cellview
leITFEMMinFile	string	cellview
leITFEMOperatingPoint	enum	cellview
leKeepSelectedObjects	bool	user
leLadderBusAlign	ladder_align_mode	cellview
leLadderBusAlignOffset	float	cellview
leLadderBusJogCount	int	cellview
leLadderBusJogCountMode	ladder_jog_count_mode	cellview
leLayerInfoPath	string	library
leLayerNameValues	string	user
leLayoutDensity	float	cellview
leLayoutGenOptionSetDefault	string	user
leLayoutSpacing	float	cellview
leLayoutViewList	string	user
leLayoutWidth	float	cellview
leLineSelectionReset	bool	cellview
leLiveDRDCheckRuleSet	string	cellview
leLiveDRDConstraintGroup	string	cellview
leLiveDRDGroupFilter	string	cellview
leLiveDRDLayerFilter	string	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leLiveDRDRuleFile	string	cellview
leLiveDRDRuleFilter	string	cellview
leLiveDRDRuleSets	string	cellview
leLiveDRDRuleSource	string	cellview
leLoadReportinResultsBrowser	bool	cellview
leLockAngles	bool	cellview
leLoopRemovalMultipleLayers	bool	cellview
leLoopRemovalSingleLayer	bool	cellview
leLPPAbbreviations	bool	user
leLPPSet	string	cellview
leLPPShowStreamLayerNumbers	bool	user
leMajorGridMult	int	cellview
leMajorGridSpacing	int	cellview
leMajorGridUnits	float	cellview
leManualStackedViaAllowExceed	bool	user
leManualStackedViaSetting	string	cellview
leManualStackedViaShowEnclosure	bool	cellview
leManualStackedViaShowSpacing	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leManualViaCalculateLayerWidth	bool	user
leManualViaKeepInputValue	bool	cellview
leManualViaUseMinParam	string	user
leMaterialBasedConnection	bool	cellview
leMaxEMNetNumber	int	cellview
leMaxLayer	string	cellview
leMaxPinPairNumberToReportNetR	int	cellview
leMCellToolVersion	string	cellview
leMeasureAngleDialogMode	bool	user
leMeasureDistanceDialogMode	bool	user
leMeasurement	bool	user
leMeasureMergeShapes	bool	user
leMeasureWireBased	bool	user
leMergePreserveNets	bool	cellview
leMergePreserveProperties	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leMGRAbutCLAlign	bool	cellview
leMGRAutoConfigBodyWidth	float	cellview
leMGRAutoConfigBodyWidthMethod	gr_method	cellview
leMGRAutoConfigBodyWidthSwitch	bool	cellview
leMGRAutoConfigCornerCut	gr_auto_status	cellview
leMGRAutoConfigCutRow	int	cellview
leMGRAutoConfigMetWidthEqBody	gr_auto_status	cellview
leMGRAutoConfigSameMetalWidth	gr_auto_status	cellview
leMGRBodyWidthMethod	gr_method	cellview
leMGRBottomLPP	string	cellview
leMGRConfigBodyWidth	float	cellview
leMGRConfigBodyWidthMethod	gr_method	cellview
leMGRConfigBodyWidthSwitch	bool	cellview
leMGRConfigCornerCut	gr_auto_status	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leMGRConfigClockwise	bool	cellview
leMGRConfigCutRow	int	cellview
leMGRConfigMetWidthEqBody	gr_auto_status	cellview
leMGRConfigSameMetalWidth	gr_auto_status	cellview
leMGRCutHVSetting	gr_cut_hv_setting	cellview
leMGRCutParams	string	cellview
leMGRCutPattern	string	cellview
leMGRCutPatternList	string	user
leMGRCutSpacingType	string	cellview
leMGRCutSpacingTypeList	string	user
leMGRDefaultParam	bool	user
leMGRSameMetWidth	bool	cellview
leMGRTopLayer	string	cellview
leMGRUseLayerResolution	bool	cellview
leMilkywayArgs	string	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leMinLayer	string	cellview
leMinorGridMult	int	cellview
leMinorGridSpacing	int	cellview
leMinorGridUnits	float	cellview
leMoveAutoAnchor	bool	cellview
leMoveStretchRemoveUnanchoredNet	bool	cellview
leMoveSymmetry	bool	cellview
leMPPAllowSelfIntersectingPaths	bool	user
leMPPDefName	string	cellview
leMPPIgnoreTechRuleChecking	bool	user
leMPPMType	mpp_manual_type	cellview
leMPPName	string	cellview
leMPPSABottomSpacing	float	cellview
leMPPSALeftSpacing	float	cellview
leMPPSANotch	float	cellview
leMPPSARightSpacing	float	cellview
leMPPSASpacing	float	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leMPPSASpacingCenterline	bool	cellview
leMPPSASpacingMode	mpp_auto_spacing_mode	cellview
leMPPSATopSpacing	float	cellview
leMPPSAType	mpp_auto_type	cellview
leMultipleInstanceSnappingMethod	singleColumnRow	cellview
leNavigatorAutoRefresh	bool	cellview
leNavigatorTrackContext	bool	cellview
leNetGroupName	string	cellview
leNetLabelDrawingStyle	onTop	user
leNetlistMode	string	cellview
leNetMatchingAware	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leNetName	string	cellview
leNetQueryWindowSize	float	library
leNetShieldingAware	bool	cellview
leNonPropagatingLayerMaterials	enMaterial	cellview
leNumPoints	int	cellview
leNXTGRDDeviceMapFile	string	cellview
leNXTGRDMetalMapFile	string	cellview
leNxtgrdOperatingPoint	operating_point	cellview
leNXTGRDSpecifyGeometryMapFile	string	cellview
leOLPAutoRedraw	bool	user
leOLPButtonsHonorFilter	bool	user
leOLPCompactView	bool	user
leOLPDefaultPurposeList	string	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leOLPDynamicUpdateDesignLPPs	bool	user
leOLPHidePurposes	string	user
leOLPKeepActiveLPPSelectable	bool	user
leOLPKeepActiveLPPVisible	bool	user
leOLPLimitLPPName	bool	user
leOLPLPPDisplayText	string	user
leOLPLPPFilterByAttribute	string	cellview
leOLPLPPFilterByAttributeEraseLayer	string	cellview
leOLPLPPFilterByAttributeGenerateCutMetal	string	cellview
leOLPLPPsForBoundaries	bool	user
leOLPMaxLayerLength	int	user
leOLPMaxPurposeLength	int	user
leOLPPresetsCount	int	user
leOLPRetainSelectability	bool	user
leOLPSetFilterCaseSensitivity	filter_sensitivity	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leOLPShowActionsToolbar	bool	user
leOLPShowActive	bool	user
leOLPShowColorToolbar	bool	user
leOLPShowFilled	bool	user
leOLPShowFilter	bool	user
leOLPShowLayer	bool	user
leOLPShowLayerBlockages	bool	user
leOLPShowLayersTab	bool	user
leOLPShowLPPs	bool	cellview
leOLPShowLPPSets	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leOLPShowPacket	bool	user
leOLPShowPresets	bool	user
leOLPShowPurpose	bool	user
leOLPShowPurposeFirst	bool	user
leOLPShowRouteGuides	bool	user
leOLPShowRoutingGrid	bool	user
leOLPShowSelectability	bool	user
leOLPShowSignalTypeFilters	bool	user
leOLPShowTrackPatterns	bool	user
leOLPShowVisibility	bool	user
leOLPSortLPPs	string	cellview
leOLPToggleColorOrMask	color_or_mask	user
leOpticalShrinkingFactor	float	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leOverlayEnableHighlight	bool	user
lePackagingAcuteAngleAggressiveness	int	cellview
lePackagingAcuteAngleCircularPatch	bool	cellview
lePackagingAcuteAngleCircularSpacingAware	bool	cellview
lePackagingAcuteAngleCoordMark	bool	cellview
lePackagingAcuteAngleFixConvex	bool	cellview
lePackagingAcuteAnglePatchConcave	bool	cellview
lePackagingAcuteAngleReport	bool	cellview
lePackagingAdoptTechnologyFile	bool	cellview
lePackagingArguments	string	cellview
lePackagingAutoGenerateRunset	bool	cellview
lePackagingBGABLPP	string	cellview
lePackagingConnectAllNets	bool	cellview
lePackagingCreateHolesMethod	create_holes_method	cellview
lePackagingD2DLPP	string	cellview
lePackagingDegassingHoleAroundTip	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
lePackagingDegassingHoleFixMinWidth	bool	cellview
lePackagingDegassingHoleFixMinWidthLayers	string	cellview
lePackagingDIEBOXLPP	string	cellview
lePackagingEnableHighlightDifferentNet	bool	user
lePackagingEnableHighlightPlane	bool	user
lePackagingEnableHighlightSameNet	bool	user
lePackagingEndPlaneLPP	string	cellview
lePackagingExcludeNet	string	cellview
lePackagingFOBOXLPP	string	cellview
lePackagingGenerateZigzag	bool	cellview
lePackagingHasD2D	bool	cellview
lePackagingHasDIEBOX	bool	cellview
lePackagingHighlightDifferentNetSet	string	user
lePackagingHighlightPlaneSet	string	user
lePackagingHighlightSameNetSet	string	user
lePackagingIPDLPP	string	cellview
lePackagingKeepBoundary	bool	cellview
lePackagingKeepRunResult	bool	cellview
lePackagingLayerMapFile	string	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
lePackagingLCV	lcv	cellview
lePackagingLibrary	string	user
lePackagingMultithreadNumber	int	user
lePackagingPatchConcaveCorners	bool	cellview
lePackagingPGCellNames	string	cellview
lePackagingPlaneGuideAssistantClusterSpacing	int	user
lePackagingPlaneGuideAssistantCombineCluster	bool	user
lePackagingPlaneGuideAssistantCompactCluster	bool	user
lePackagingPlaneGuideAssistantCoverDiffNet	bool	user
lePackagingPlaneGuideAssistantCoverFOB	bool	user
lePackagingPlaneGuideAssistantEscape	string	user
lePackagingPlaneGuideAssistantEscapeCount	int	user
lePackagingPlaneGuideAssistantEscapeFourDirection	bool	user
lePackagingPlaneGuideAssistantEscapeMode	plane_guide_assistant_escape_mode	user
lePackagingPlaneGuideAssistantEscapeNeighbor	bool	user
lePackagingPlaneGuideAssistantFOBSpacing	int	user
lePackagingPlaneGuideAssistantMethod	plane_guide_assistant_method	user
lePackagingPlaneGuideAssistantPushRoute	bool	user
lePackagingPlaneGuideAssistantRMSTMode	plane_guide_assistant_rmst_mode	user
lePackagingPlaneGuideAssistantRouter	bool	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
lePackagingPlaneGuideAssistantShowLayerAssignment	bool	user
lePackagingPlaneGuideAssistantSizing	string	user
lePackagingPlaneGuideAssistantSortByNet	bool	user
lePackagingPlaneGuideAssistantSurroundingComponentSizing	string	user
lePackagingPlaneGuideAssistantSurroundingDepth	int	user
lePackagingPlaneGuideAssistantUTurnMode	plane_guide_assistant_uturn_mode	user
lePackagingPlaneGuideAssistantZoomFit	bool	user
lePackagingPlaneLayers	string	cellview
lePackagingPlanePlaneSpacing	float	cellview
lePackagingPlanePriority	int	cellview
lePackagingPlaneShapeType	packaging_create_plane_shape_type	cellview
lePackagingPlaneWidthThreshold	float	cellview
lePackagingPlaneWidthThresholdD2D	float	cellview
lePackagingPlaneWireSpacing	float	cellview
lePackagingRealizePlaneFixMinWidth	bool	cellview
lePackagingRemoveFloating	bool	cellview
lePackagingRemoveFloatingLength	float	cellview
lePackagingRunset	string	cellview
lePackagingSpacingPlaneLineD2D	float	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
lePackagingStartPlaneLPP	string	cellview
lePackagingTargetLPP	string	cellview
lePackagingTargetMode	target mode	cellview
lePackagingTIVLPP	string	cellview
lePackagingTracePhysicalConnection	bool	cellview
lePackagingUBMLPP	string	cellview
lePackagingWidthMax	float	cellview
lePackagingWidthPlaneMinD2D	float	cellview
lePackagingWireWireSpacing	float	cellview
lePackagingZigzagTargetEdge	int	cellview
lePackagingZigzagType	packaging_zigzag_type	cellview
lePackagingZoomToViolation	bool	cellview
lePadAreaSize1	float	cellview
lePadAreaSize2	float	cellview
lePadInsertFile	string	cellview
lePadLabelDepthMode	pad_label_depth	cellview
lePadLabelDialogMode	bool	user
lePadLabelMode	pad_label_mode	cellview
lePadLabelSelectionType	enPadLabelSelectionType	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
lePadLabelUserDefinedLevel	int	cellview
lePadLabelZoomView	bool	cellview
lePadOperator1	string	cellview
lePadOperator2		
lePadOutputFile	string	cellview
lePadOutputLocationMethod	pad_output_location_method	cellview
lePadShapeLPP	string	cellview
lePadTextHeight	float	cellview
lePadTextLPP	string	cellview
lePadTextOffset	string	cellview
lePadTextXOffset	float	cellview
lePadTextYOffset	float	cellview
lePadTolerance	float	cellview
leParamNamesIgnored	string	cellview
leParasiticConductingMaterials	string	user
leParasiticReporterPreserveNeighborShapes	bool	cellview
lePartialSelectRadius	int	user
lePasteJustification	paste_justification	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
lePasteOrigin	bool	cellview
lePathBeginExtension	float	cellview
lePathEndExtension	float	cellview
lePathPresetWidths	string	user
lePathSegBeginExtension	float	cellview
lePathSegBeginLD	float	cellview
lePathSegBeginRD	float	cellview
lePathSegBeginRHW	float	cellview
lePathSegBeginStyle	path_seg_beg	cellview
lePathSegEndExtension	float	cellview
lePathSegEndLD	float	cellview
lePathSegEndRD	float	cellview
lePathSegEndStyle	path_seg_end	cellview
lePathStyle	path_style	cellview
lePdAutoBondPadCols	string	cellview
lePdAutoBondPadMethodType	string	cellview
lePdAutoBondPadPitchX	string	cellview
lePdAutoBondPadPitchY	string	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
lePdAutoBondPadRows	string	cellview
lePdAutoBondPadStartCol	string	cellview
lePdAutoBondPadStartRow	string	cellview
lePdAutoBondPadWx	string	cellview
lePdAutoBondPadWy	string	cellview
lePdAutoCreateBondPadDialogMode	bool	user
lePdMeasureMapsDrawText	string	cellview
lePdMeasureMapsDrawTextPrecision	string	cellview
lePdMeasureMapsIntervalNum	string	cellview
lePdMeasureMapsMapType	string	cellview
lePdMeasureMapsRangeMax	string	cellview
lePdMeasureMapsRangeMin	string	cellview
lePdMeasureMapsRangeType	string	cellview
lePdMeasureMapsSimTime	string	cellview
lePdMeasureMapsTestbenchLCV	string	cellview
lePdShowMeasureMapsDialogMode	bool	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
lePdTbEndTime	string	cellview
lePdTbIntervalCount	string	cellview
lePdTbLCV	string	user
lePdTbMosEndTime	string	cellview
lePdTbMosSimEndTime	string	user
lePdTbMosStartTime	string	cellview
lePdTbSimEndTime	string	user
lePdTbStartTime	string	cellview
lePdUseSAEVariables	string	user
lePhotonicFeatureEnabled	bool	cellview
lePinAccess	int	cellview
lePinAlign	bool	cellview
lePinArrayMode	bool	cellview
lePinAuto	bool	cellview
lePinAutoSize	pin_auto_size	cellview
lePinConvertLabelToAttributeLabel	bool	cellview
lePinHeight	float	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
lePinJust	string	cellview
lePinLabel	bool	cellview
lePinLabelAutoPlace	bool	cellview
lePinLabelLPP	string	cellview
lePinLabelMethod	string	cellview
lePinLabelMode	pin_label_mode	cellview
lePinLabelPosition	string	cellview
lePinLabelTextAttach	bool	cellview
lePinLabelTextType	pin_label_type	cellview
lePinLabelTextType	enPinLabelType	cellview
lePinNewShapes	bool	cellview
lePinNewShapesConfig	pin_new_shapes_config	cellview
lePinPhotonic	bool	cellview
lePinPhotonicAngle	string	cellview
lePinPhotonicConnectType	string	cellview
lePinPhotonicCrossSection	string	cellview
lePinPhotonicLocation	string	cellview
lePinPhotonicRadius	string	cellview
lePinPhotonicWidth	string	cellview
lePinPurposeList	string	user
lePinRel	pin_relationship	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
lePinShapePurposeList	string	cellview
lePinUseVirtualConnect	bool	cellview
lePinWidth	float	cellview
lePinWidthHeight	string	cellview
lePlaneGuideAssistantDisplayPatternWidget	bool	cellview
lePlaneGuideAssistantExcludeNetsText	string	cellview
lePlaneGuideAssistantLogTCLCommand	bool	cellview
lePlaneGuideAssistantPrimaryOnly	bool	cellview
lePlaneGuideAssistantSortNetBased	bool	cellview
lePlaneGuideAssistantSyncView	bool	cellview
lePolygonHoles	bool	cellview
lePolySnappingTargetHierarchy	leInstancePolySnappingHierarchy	user
lePostAutoAbutmentProcedure	bool	cellview
lePRBoundaryLPP	string	user
lePreserveAll	bool	cellview
lePreserveConnectivity	bool	cellview
lePreserveFigGroups	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
lePreserveHierarchyName	bool	cellview
lePreserveMPPs	bool	cellview
lePreservePcells	bool	cellview
lePreservePins	bool	cellview
lePreservePinText	bool	cellview
lePreserveStripedWires	bool	cellview
lePreserveTerm	bool	cellview
lePreserveViaDefParameters	bool	cellview
lePreserveVias	bool	cellview
lePreViaCreateProc	string	user
lePropertyEditorFilterViaDefByLayer1	bool	user
lePropertyEditorPathWidthMethod	string	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
lePropertyEditorParameterFilter	string	user
lePropertyEditorShownAttrsAlignmentBoundary	string	user
lePropertyEditorShownAttrsAreaBlockage	string	user
lePropertyEditorShownAttrsAreaBoundary	string	user
lePropertyEditorShownAttrsArrayInst	string	user
lePropertyEditorShownAttrsAttributeText	string	user
lePropertyEditorShownAttrsCircularRuler	string	user
lePropertyEditorShownAttrsClusterBoundary	string	user
lePropertyEditorShownAttrsCoordinateMark	string	user
lePropertyEditorShownAttrsCoreBoxSpec	string	user
lePropertyEditorShownAttrsCorridorArea	string	user
lePropertyEditorShownAttrsCustomArrayInstance	string	user
lePropertyEditorShownAttrsDimensionMark	string	user
lePropertyEditorShownAttrsDonut	string	user
lePropertyEditorShownAttrsEllipse	string	user
lePropertyEditorShownAttrsFigureGroup	string	user
lePropertyEditorShownAttrsFlightline	string	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
lePropertyEditorShownAttrsInst	string	user
lePropertyEditorShownAttrsLayerBlockage	string	user
lePropertyEditorShownAttrsLinearRuler	string	user
lePropertyEditorShownAttrsMarker	string	user
lePropertyEditorShownAttrsMPP	string	user
lePropertyEditorShownAttrsPath	string	user
lePropertyEditorShownAttrsPathSeg	string	user
lePropertyEditorShownAttrsPolygon	string	user
lePropertyEditorShownAttrsPRBoundary	string	user
lePropertyEditorShownAttrsPropertyText	string	user
lePropertyEditorShownAttrsRectangle	string	user
lePropertyEditorShownAttrsRouteGuide	string	user
lePropertyEditorShownAttrsRow	string	user
lePropertyEditorShownAttrsScalarSiteDef	string	user
lePropertyEditorShownAttrsSnapBoundary	string	user
lePropertyEditorShownAttrsSteiner	string	user
lePropertyEditorShownAttrsText	string	user
lePropertyEditorShownAttrsTrackRegion	string	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
lePropertyEditorShownAttrsVia	string	user
lePropertyEditorShowViaWidthHeight	bool	user
leQueryObjectPersistent	bool	user
leQueryObjectZoom	bool	cellview
leQueryObjectZoomView	int	cellview
leRATCLMapFile	string	cellview
leRATCLMaxFile	string	cellview
leRCAnalysisTemperature	float	cellview
leRCIgnorePurposes	string	user
leRCQueryExtension	float	library
leRCQueryWindowSize	float	library
leRCReporterByOneExtraction	bool	cellview
leRealizePlaneDialogMode	bool	user
leReassignNetDialogMode	bool	user
leReassignNetFilterSDLInstanceTerminals	bool	cellview
leReassignNetName	string	cellview
leReassignNetNameValues	string	cellview
leRectangleAsSquare	bool	cellview
leRedundantViaSpacingFactor	float	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leRelaxOLPSelectability	bool	user
leRemoveFromFigGroupDepth	enRemoveFromFigGroupDepth	cellview
leRemoveObjsCVFromSet	bool	cellview
leRemoveOverlappingCutsArea	remove_overlapping_cuts_area	cellview
leRemoveOverlappingCutsDepth	clipboard_depth	cellview
leRemoveOverlappingCutsDialogMode	bool	user
leRemovePinShapes	bool	cellview
leRenderFinGridColors	off	user
leRenderTrackGrids	bool	cellview
leRenderTrackGridsMethod	<enRenderTrackGridsMethod>	cellview
leRenderTracksOnInvisibleLPPs	bool	user
leRenderTrackWidth	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leReplaceObjects	bool	cellview
leReplacePlaneBoundaryDialogMode	bool	user
leReplaceViaBottomLayer	string	cellview
leReplaceViaCutNum	int	cellview
leReplaceViaFilterByBottomLayer	bool	cellview
leReplaceViaMode	replaced_via_mode	cellview
leReplaceViaRegionType	replaced_via_region_type	cellview
leReplaceViaSourceDef	string	cellview
leReplaceViaSpecifySourceDef	bool	cellview
leReplaceViaSpecifyTargetDef	bool	cellview
leReplaceViaTargetDef	string	cellview
leReportCapacitanceDialogMode	bool	user
leReportCapacitanceExportToConsole	bool	cellview
leReportCapacitanceExportToErrorViewer	bool	cellview
leReportCapacitanceGroupType	int	cellview
leReportEMByFullDepth	bool	library
leReportEMCurrentType	report_EM_current_type	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leReportEMRCBackgroundMode	bool	cellview
leReportMaxUnshieldedLength	bool	cellview
leReportOutOfDateMPP	bool	user
leReportParasiticDialogMode	bool	user
leReportResistanceDensityMode	density_mode	cellview
leReportResistanceDestination	string	cellview
leReportResistanceInput	report_resistance_input	cellview
leReportResistanceSet	string	cellview
leReportResistanceSource	string	cellview
leReportResistanceSpacingMode	spacing_mode	cellview
leReservedTrackRuleType	leReservedTrackRuleType	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leReshapeInterconnectDialogMode	bool	user
leReshapeInterconnectMode	reshape_interconnect	cellview
leReshapeShapeType	edit_shape_type	cellview
leResistanceExtractTargetOnly	bool	cellview
leResistanceModel	string	cellview
leResistanceReporterSuperconductive	bool	cellview
leResizeBottomSize	float	cellview
leResizeDialogMode	bool	user
leResizeKeepColor	bool	cellview
leResizeKeepNet	bool	cellview
leResizeLeftSize	float	cellview
leResizeMerge	bool	cellview
leResizeResultMode	resize_result	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leResizeRightSize	float	cellview
leResizeSize	float	cellview
leResizeSizeMode	resize_size	cellview
leResizeTopSize	float	cellview
leResMaxPinListNumber	int	cellview
leRestrictedSelectionAlignmentBoundary	bool	user
leRestrictedSelectionAlignmentBoundaryLPPs	string	cellview
leRestrictedSelectionAreaBoundary	bool	user
leRestrictedSelectionClusterBoundary	bool	user
leRestrictedSelectionLayerBlockages	bool	user
leRestrictedSelectionPinBlockages	bool	user
leRestrictedSelectionPlacementBlockages	bool	user
leRestrictedSelectionPRBoundary	bool	user
leRestrictedSelectionRoutingBlockages	bool	user
leRestrictedSelectionRow	bool	user
leRestrictedSelectionSnapBoundary	bool	user
leRestrictedSelectionTrackRegion	bool	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leRestrictViasToVariants	bool	user
leRotateAngle	float	cellview
leRotateAsGroup	bool	cellview
leRotateDialogMode	bool	user
leRotateInstDrawFullContent	bool	cellview
leRotateInteractive	bool	cellview
leRotateJustification	string	cellview
leRotateOrigin	bool	cellview
leRotateStep	float	cellview
leRouteType	route_type	cellview
leRoutingPurposeList	string	cellview
leRowGrowDirectionName	string	cellview
leRowGrowDirectionValues	string	user
leRowName	string	cellview
leRowOrient	string	cellview
leRowOrigin	string	cellview
leRowPairing	bool	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leRowStormRegion	int	cellview
leRowTopSpacing	float	cellview
leRReportTraceVisibleLPPsOnly	bool	cellview
leRubberBandPacket	string	user
leRubberBandStyle	string	user
leRulerTickOnGrid	bool	user
leSaveConnectivity	bool	cellview
leScalarizeArrayInst	bool	cellview
leSelectAreaType	string	cell
leSelectCriteria	string	user
leSelectedNxtgrdCornerName	string	cellview
leSelectionTableAttributes	string	user
leSelectLPP	string	cellview
leSelectLPPHierarchy	bool	cellview
leSelectMode	string	cellView
leSelectNetName	string	user
leSelectObjectsDialogMode	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leSelectObjectsShowCell	bool	cellview
leSelectObjectsShowLayer	bool	cellview
leSelectObjectsShowLibrary	bool	cellview
leSelectObjectsShowPurpose	bool	cellview
leSelectRadius	int	user
leSelectRouteLayers	string	cell
leSelectRouteMode	select	user
leSelectRouteObjectsDialogMode	bool	user
leSelectRouteStatus	string	cell
leSelectRouteType	string	cell
leSelectRouteVias	bool	cell
leSelectSignalType	string	cell

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leSelectStackedVias	bool	user
leShadowSelection	bool	user
leShapeTap	bool	cellview
leShapeType	string	cellview
leShieldMinimumPercentage	float	cellview
leShieldReportAboveBelowTandemSeparately	bool	cellview
leShortDetectCheckErasingRules	bool	user
leShowCellCategories	bool	user
leShowColorLegend	bool	user
leShowDensityDisplaySettingDialogMode	bool	user
leShowDistance	bool	cellview
leShowEMCalculatorDialogMode	bool	user
leShowMetalLPPSets	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leShowNonPreferredTrack	bool	cellview
leShowRecognitionLpps	bool	user
leShowTrackRegionFilter	bool	cellview
leSignalType	string	cellview
leSiteDefName	string	cellview
leSizedSteiner	bool	cellview
leSkipInstTermSync	bool	user
leSlotBeginStrap	bool	cellview
leSlotEndStrap	bool	cellview
leSmartDRDMode	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leSmartEraseLayer	bool	user
leSnapBoundaryLPP	string	user
leSnapCoordToManufacturingGrid	bool	user
leSnapCursorShape	string	user
leSnapOffset	unknown	cellview
leSnapOffsetMatch	bool	cellview
leSnapRectToTrack	bool	cellview
leSnapSpacing	unknown	cellview
leSnapSpacingMatch	bool	cellview
leSnapStripedWireToTrack	bool	cellview
leSnapToRowAllowInstOverlap	bool	cellview
leSnapToRowMatchCellHeight	bool	cellview
leSnapToRowSupportedCellType	string	cellview
leSnapToRowWithTrackAutoFlip	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leSnapToTrackGrids	bool	cellview
leSnapToTrackIntersection	bool	cellview
leSnapToTrackIntersectionCriteria	leSnapToTrackIntersectionCriteria	cellview
leSpecifyPadAreaSize	bool	cellview
leSpecifyPadFirstLocation	bool	cellview
leSpecifyPadInsertFile	bool	cellview
leSpecifyPinLabelLPP	bool	cellview
leSpecifyTextLPP	bool	cellview
leSplitBigViaLevel	int	cellview
leSplitMode	string	cellview
leSplitShapeType	string	cellview
leStackInterconnectBottomLayer	string	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leStackInterconnectGroup	bool	cellview
leStackInterconnectMinimumLength	float	cellview
leStackInterconnectTopLayer	string	cellview
leStackViaSearchMethod	string	user
leStarRCBinaryPath	string	user
leStarRCEnvSettingFile	string	cellview
leStarRCReportEM	bool	cellview
leStarRCReportResistance	bool	cellview
leStartLevel	int	cellview
leStaticMeasurement	bool	user
leStdViaStretchMethod	stretch_stdvia	cellview
leSteinerHeight	float	cellview
leSteinerWidth	float	cellview
leStopLevel	int	cellview
leStrapBegin	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leStrapEnd	bool	cellview
leStretch1xBoundary	bool	cellview
leStretchArrayInstance	bool	cellview
leStretchArrayMethod	string	cellview
leStretchConnectionAbutmentAware	bool	cellview
leStretchConnectionAware	bool	cellview
leStretchConnectionAwareMode	connect_stretch	cellview
leStretchConnectionExtend	bool	cellview
leStretchDialogMode	bool	user
leStretchHandleDisplay	string	user
leStretchIgnoreFullSelection	bool	cellview
leStretchKeepViaOriginCenter	bool	cellview
leStretchPartialSelectionFirstMode	bool	cellview
leStretchPermitJogs	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leStretchWidthMethod	string	cellview
leStringWidth	string	cellview
leStripePolicy	string	cellview
leSyncColor	bool	cellview
leSyncDNOrder	sync_net_order	user
leSyncText	bool	cellview
leTapAttributes	bool	cellView
leTapBlockagelayer	bool	cellView
leTapBlockageType	bool	cellView
leTapCollapseAttributes	bool	cellView
leTapDialogMode	bool	user
leTaperFromSelectedEdge	bool	cellview
leTaperToCoverPin	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leTapInstanceMaster	bool	cellView
leTapInstanceOientation	bool	cellView
leTapListAllLPPs	bool	cellview
leTapListMode	tap_list_mode	cellView
leTapLpp	bool	cellView
leTapLPPTreshold	int	user
leTapNet	bool	cellView
leTapPathStyle	bool	cellView
leTapPathWidth	bool	cellView
leTapText	bool	cellView
leTapTextFont	bool	cellView
leTapTextHeight	bool	cellView
leTapTextJustification	bool	cellView
leTapTextOrientation	bool	cellView
leTCadGRDMapFile	string	cellview
leTCadGRDMaxFile	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leTCadGRDMinFile	string	cellview
leTechFoundryConstraintGroups	string	cellview
leTerminateAfterColorApplied	bool	cellview
leTermName	string	cellview
leTermType	string	cellview
leTextAttach	bool	cellview
leTextDepth	text_depth	cellview
leTextDrafting	bool	cellview
leTextDrawFilter	int	user
leTextExpand	bool	cellview
leTextFont	string	cellview
leTextHeight	float	cellview
leTextJust	string	cellview
leTextLine	string	cellview
leTextLPP	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leTextLPPMapFile	string	cellview
leTextLPPMethod	string	cellview
leTextMethod	text_method	cellview
leTextMultiline	string	cellview
leTextOrientation	string	cellview
leTextOverbar	bool	cellview
leTextPolyline	bool	cellview
leTextPurposeList	string	user
leTLUPlusOperatingPoint	operating_point	library
leToggleShape	toggle_shape	cellview
leToolboxAlign	bool	cellview
leToolboxAlignSpacing	string	cellview
leToolboxCoordinateType	coordinate_type	cellview
leToolboxDistribute	bool	cellview
leToolboxDistributeMethod	distribute method	cellview
leToolboxDistributeValue	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leToolboxMeasurement	bool	cellview
leToolboxMove	bool	cellview
leToolboxMoveSpacings	string	cellview
leToolboxObjectEditing	bool	cellview
leToolboxOrient	bool	cellview
leToolboxRotate	bool	cellview
leToolboxRotateGroup	bool	cellview
leToolboxRotateJustification	toolbox_rotate_justification	cellview
leTopologyBoneDirection	topology_bone_direction	cellview
leTopologyMode	<enTopologyMode>	cellview
leTopologyNetSelection	int	cellview
leTraceArea	string	user
leTraceDepth	string	user
leTraceIgnoreErasingShapes	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leTraceParentDesigns	bool	user
leTracePropagatingShapesOnly	bool	cellview
leTraceVisibleLPPsOnly	bool	user
leTrackAutoTileOffset	bool	cellview
leTrackCompleteX	bool	cellview
leTrackCompleteY	bool	cellview
leTrackDefaultSet	string	cellview
leTrackDefinition	string	cellview
leTrackFlipFirstX	bool	cellview
leTrackFlipFirstY	bool	cellview
leTrackFlipX	bool	cellview
leTrackFlipY	bool	cellview
leTrackGenerationName	string	cellview
leTrackGlobalRegionOffset	point	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leTrackGlobalRegionOffsetMatch	bool	cellview
leTrackInteractive	bool	user
leTrackName	string	cellview
leTrackOffsetTarget	leTrackOffsetTarget	cellview
leTrackOverwritePreset	bool	user
leTrackPatternColor	bool	cellview
leTrackPatternDefActive	bool	cellview
leTrackPatternDefBlockPitch	float	cellview
leTrackPatternDefLayer	string	cellview
leTrackPatternDefNumTracks	int	cellview
leTrackPatternDefOffset	string	cellview
leTrackPatternDefOrientation	cellview	cellview
leTrackPatternDefPitch	string	cellview
leTrackPatternDefPurpose	string	cellview
leTrackPatternDefType	string	cellview
leTrackPatternDefVisible	bool	cellview
leTrackPatternDefWidth	float	cellview
leTrackPcellDrawShape	bool	user
leTrackPcellLCV	lcv	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leTrackPCellLPP	string	user
leTrackPcellName	string	cellview
leTrackPcellPresetEnable	bool	user
leTrackPcellPresetInst	string	cellview
leTrackPcellPresetInstNames	string	cellview
leTrackPcellPresetLCV	lcv	user
leTrackPcellShowRoutingLayerOnly	bool	cellview
leTrackPcellTrackType	track_pcell_track_type	user
leTrackPresetsFilePath	string	user
leTrackPresetName	string	cellview
leTrackRegionAllowedDefNames	string	library
leTrackRegionAssistantShowTrackGrids	bool	user
leTrackRegionCrossHighlight	bool	user
leTrackRegionCrossHighlightPacket	string	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leTrackRegionCrossSelect	bool	user
leTrackRegionDefFilter	leTrackRegionDefFilter	cellview
leTrackRegionDefFilterValues	string	cellview
leTrackRegionDefValues	string	cellview
leTrackRegionShareSettings	bool	user
leTrackRegionTreeFormat	string	user
leTrackRegionType	leTrackRegionType	cellview
leTrackSavePreset	bool	user
leTrackSavePresetLCV	lcv	user
leTrackShowGenerateTracksDialog	bool	user
leTrackTileOffset	unknown	cellview
leTransmissionLineBendFactor	float	cellview
leTransmissionLineChamferFactor	float	cellview
leTransmissionLineLCV	lcv	user
leTransmissionLineMode	transmission_line_mode	cellview
leTransmissionLineRadialFactor	float	cellview
leTransmissionLineResolution	int	cellview
leTransmissionLineStyle	transmission_line_style	cellview
leTransmissionLineTaperStyle	transmission_line_taper_style	cellview
leTransmissionLineWidth	float	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leTransmissionLineWidth1	float	cellview
leTransmissionLineWidth2	float	cellview
leTRAToggleMode	string	user
leTriggerHierConnectForUncoloredInstPinsOnly	bool	user
leTrimRemoveOverlapping	bool	user
leTrimWireDirection	trim_direction	cellview
leTrimWireEndEdge	trim_endection	cellview
leTrimWireMethod	trim_method	cellview
leTrimWirePreserveEndStyle	bool	cellview
leTrimWireSameNetNeighbors	bool	cellview
leTrimWireToConnected	bool	cellview
leUpdateConnectedViasAndEndStyles	bool	cellview
leUpdateConnectedViasAndEndStylesMode	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leUpdateConnectedViasFullStacked	bool	cellview
leUpdateConnectedViasPreserveVia	string	cellview
leUpdateConnectedViasWhenChangingLayers	bool	cellview
leUpdateTrackPcell	bool	user
leUpdateWideAlignment	bool	user
leUpdateWidthForAll	bool	cellview
leUseAllLayers	bool	cellview
leUseBusSpacing	bool	cellview
leUseDesignViaVariants	bool	user
leUseLocalNetShapeAsFeedthrough	bool	user
leUseNxEdit	bool	cellview
leUsePathSeg	bool	cellview
leUsePurposesForAbstractGenerator	string	cellview
leUsePurposesForAntennaExtraction	string	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leUserDefinedConstraintGroup	string	cellview
leUserDefinedConstraintGroups	string	cellview
leUserSetWidthList	string	cellview
leValidateColorCheckColorConflict	bool	cellview
leValidateColorCheckColorConstraintMismatch	bool	cellview
leValidateColorCheckConflict	bool	cellview
leValidateColorCheckConstraintMismatch	bool	cellview
leValidateColorCheckNoColor	bool	cellview
leValidateColorCheckNotLocked	bool	cellview
leValidateColorCheckRecognitionLayerNoColor	bool	cellview
leValidateColorCheckTrackMismatch	bool	cellview
leValidateColorDepth	string	cellview
leValidateColorScope	string	cellview
leValidateColorTrackMismatch	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leValidateColorWaiveCovered	bool	cellview
leValidPurposes	string	user
leVDRCAnnotationCheckTextLabel	bool	user
leVDRCAnnotationCheckUdv	bool	user
leVDRCAnnotationCheckVflag	bool	user
leVDRCAnnotationCheckVoltageInstance	bool	user
leVDRCAnnotationCheckVsync	bool	user
leVDRCAnnotationConnectedShapeLayerConflicts	bool	user
leVDRCAnnotationDepth	enum	user
leVDRCAnnotationExtraVInst	bool	user
leVDRCAnnotationIncorrectTextLabelLPP	bool	user
leVDRCAnnotationIncorrectTextLabelPosition	bool	user
leVDRCAnnotationIncorrectTextLabelValue	bool	user
leVDRCAnnotationMismatchedVInstConn	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leVDRCAnnotationMismatchedVInstMaster	bool	user
leVDRCAnnotationMismatchedVInstParams	bool	user
leVDRCAnnotationMisplacedVInst	bool	user
leVDRCAnnotationMissingNetAnnotation	bool	user
leVDRCAnnotationMissingTextLabel	bool	user
leVDRCAnnotationMissingVInst	bool	user
leVDRCAnnotationMissingVInstConn	bool	user
leVDRCAnnotationMultipleTextLabels	bool	user
leVDRCAnnotationNetHasVInstConflicts	bool	user
leVDRCAnnotationNetsWithVoltageValue	bool	user
leVDRCAnnotationOnlyVoltageLPP	bool	user
leVDRCAnnotationPhysicalErrorGroup	bool	user

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leVDRCAnnotationScope	enum	user
leVDRCAnnotationSDLErrorGroup	bool	user
leVDRCIcv	string	cellview
leVDRCPhysicalAnnotationCustomProcedureName	string	user
leVDRCPhysicalAnnotationLabelTextFontOrient	vdrc_physical_annotation_font_orient	user
leVDRCPhysicalAnnotationLabelTextFontSize	float	user
leVDRCPhysicalAnnotationLabelTextFontStyle	vdrc_physical_annotation_font_style	user
leVDRCPhysicalAnnotationLabelTextLPPs	string	user
leVDRCPhysicalAnnotationMarkerShapeLPPs	string	user
leVDRCPhysicalAnnotationMarkerShapeLPPsEditable	bool	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leVDRCPhysicalAnnotationMethod	vdrc_physical_annotation_method	user
leVDRCPhysicalAnnotationNumberNotation	vdrc_physical_annotation_num_notation	user
leVDRCPhysicalAnnotationOverwrite	bool	user
leVDRCPhysicalAnnotationShapePropertyMaxVoltageName	string	user
leVDRCPhysicalAnnotationShapePropertyMinVoltageName	string	user
leVDRCPhysicalAnnotationShapePropertyNamesEditable	bool	user
leVDRCPhysicalClearCustomProcedureName	string	user
leVDRCUdvLayerPurpose	string	user
leVDRCUdvPinPurpose	string	user
leVDRCUdvTextPurpose	string	user
leVDRCUdvUsedDeltaVoltage	string	user
leVDRCUdvUsedLayer	string	user
leVDRCUdvUsedLength	string	user
leVDRCUdvUsedTextSize	string	user
leVDRCUdvUsedWidth	string	user
leVDRCVflagHUsedLPP	string	user
leVDRCVflagHUsedMaxVoltage	string	user
leVDRCVflagHUsedMinVoltage	string	user
leVDRCVflagHUsedTextHeight	string	user
leVDRCVflagPinPurpose	string	user
leVDRCVflagUsedLayer	string	user

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leVDRCVflagUsedSize	string	user
leVDRCVflagUsedVoltagePurpose	string	user
leVDRCVflagVoltagePurposes	string	user
leVDRCVsyncLayerPurpose	string	user
leVDRCVsyncPinPurpose	string	user
leVDRCVsyncUsedLayer	string	user
leVDRCVsyncUsedLength	string	user
leVDRCVsyncUsedWidth	string	user
leVerticalWidth	float	cellview
leViaAuto	bool	cellview
leViaAutoFill	bool	cellview
leViaBottomLayer	string	cellview
leViaBtmLayers	string	cellview
leViaCheckCutMinWidth	bool	user
leViaColorPropagation	bool	cellview
leViaCols	int	cellview
leViaCreateMetalShape	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leViaCreateMethod	leViaCreateMethod	cellview
leViaCutCenterAlign	bool	cellview
leViaCutClassName	string	cellview
leViaCutClassNames	string	cellview
leViaDefineCuts	bool	cellview
leViaDefineSizeManually	bool	cellview
leViaDefineSizeMode	leViaSizeMode	cellview
leViaDefineWidthHeight	bool	cellview
leViaDefName	string	cellview
leViaDefValues	string	cellview
leViaEndLayerAuto	string	cellview
leViaEndLayerManual	string	cellview
leViaFilterByBottomLayer	bool	cellview
leViaFilterCutClass	string	cellview
leViaFilterCutClasses	string	cellview

Table 139 Layout Editing Preferences (Continued)

Name	Type	Scope
leViaFilterCutClassesManualStack	string	cellview
leViaFilterCutClassManualStack	string	cellview
leViaFilterCutClassWire	string	cellview
leViaFilterCutClassesWire	string	cellview
leViaFilterDef	string	cellview
leViaFilterDefManualStack	string	cellview
leViaFilterDefs	string	cellview
leViaFilterDefsManualStack	string	cellview
leViaFilterDefWire	string	cellview
leViaFilterDefsWire	string	cellview
leViaFilterLayer	string	cellview
leViaHeight	float	cellview
leViaHonorViaDefDefaultSpacing	bool	user
leViaHonorWidthConstraints	bool	user
leViaImplantAllowExceed	bool	user
leViaJustification	string	cellview
leViaJustificationMaterial	string	cellview
leViaJustificationType	via_justification_type	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leViaManualStackedSpecifyFromShapeByRegion	bool	cellview
leViaManualStackedSpecifyFromShapeByRegionSelection	leViaManualStackedSpecifyFromShapeByRegionSelection	cellview
leViaManualStackedSpecifyMode	leViaStackedColRowMode	cellview
leViaMatchWireWidth	bool	user
leViaMetalFitToOverlappedArea	bool	cellview
leViaMultipleEnclosures	bool	user
leViaParamExpand	bool	cellview
leViaPreset	string	cellview
leViaPresetFile	string	user
leViaPresetNames	string	user
leViaRegionHeight	float	cellview
leViaRegionWidth	float	cellview
leViaRows	int	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leViaSameMetalSize	bool	cellview
leViaSelectionMode	string	cellview
leViaSettingMethod	leViaSettingMethod	cellview
leViaStackedApplyColRow	bool	cellview
leViaStackedAuto	bool	cellview
leViaStackedManual	bool	cellview
leViaStartLayerAuto	string	cellview
leViaStartLayerManual	string	cellview
leViaTopLayers	string	cellview
leViaTraceSameLayerShapes	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leViaUseActiveObject	bool	cellview
leViaWidth	float	cellview
leVisualizeTrackWidth	bool	cellview
leVPEEnableContextCheckCorrection	bool	user
leVPEPickCutClassFromViaDef	bool	user
leVPEUseOverlapDimensions	bool	user
leWeakConnectionModeOverride	off	user
leWideAlignmentSnapFinPattern	bool	cellview
leWideAlignmentSnapping	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leWideAlignmentSnappingBoundaryShape	bool	cellview
leWideAlignmentSnappingDepth	int	cellview
leWideAlignmentSnappingHierLevel	wide_alignment_snapping_hier_level	cellview
leWideAlignmentSnappingTargetDepth	int	cellview
leWidth	float	cellview
leWidthIncrement	unsigned	user
leWireOutFromPinOnly	bool	cellview

Table 139 *Layout Editing Preferences (Continued)*

Name	Type	Scope
leYankFlatten	bool	cellview
leYankSelectObjects	bool	cellview
vdrcResultDir	string	cellview

15

Placement Tcl

For details, see

- [Commands](#)
- [Enumerations](#)
- [Preferences](#)

Commands

The commands covered are

- [Ip::adjustRelativeSpacing](#)
- [Ip::align](#)
- [Ip::applyTemplates](#)
- [Ip::autoPinPlace](#)
- [Ip::autoPlace](#)
- [Ip::checkAreaPinPlacement](#)
- [Ip::checkPinPlacement](#)
- [Ip::checkSimilarity](#)
- [Ip::cleanUpPins](#)
- [Ip::convertCustomConstraint](#)
- [Ip::copyResult](#)
- [Ip::doDeviceGrouping](#)
- [Ip::doStdCellGrouping](#)
- [Ip::dumpDMPprop](#)
- [Ip::exportLayoutPlacement](#)

- [lp::extractAlignment](#)
- [lp::extractFigGroup](#)
- [lp::extractGRParams](#)
- [lp::extractGRPitchSpacing](#)
- [lp::extractLayoutParameters](#)
- [lp::extractOrigin](#)
- [lp::extractOriginFromFile](#)
- [lp::extractOriginInfoFromFile](#)
- [lp::extractOriginToFile](#)
- [lp::extractPinPlacement](#)
- [lp::extractPlacement](#)
- [lp::extractPRBoundary](#)
- [lp::extractSEDSUBGROUPS](#)
- [lp::extractSurroundCells](#)
- [lp::extractSymmetric](#)
- [lp::extractTopology](#)
- [lp::fastPlace](#)
- [lp::generateAreaPinPlacement](#)
- [lp::generateConstraint](#)
- [lp::generatePinPlacement](#)
- [lp::generatePlacement](#)
- [lp::generatePRBoundary](#)
- [lp::generateSurroundCells](#)
- [lp::getGroupVectorName](#)
- [lp::groupByGuardRing](#)
- [lp::groupBySurroundCells](#)
- [lp::groupDeviceByArea](#)

- [lp::handleConstraintArea](#)
- [lp::importPinPlacement](#)
- [lp::insertBoundary](#)
- [lp::insertColumn](#)
- [lp::insertDecapFiller](#)
- [lp::insertDiode](#)
- [lp::insertTap](#)
- [lp::isReadOnly](#)
- [lp::optimizePins](#)
- [lp::placementDebugger](#)
- [lp::placeToRows](#)
- [lp::rebuildFigGroup](#)
- [lp::saveDesignToPlacementLib](#)
- [lp::showOptions](#)
- [lp::testGenerateGrouping](#)
- [lp::updateConstraint](#)
- [lp::updateGRSpacing](#)
- [lp::updateLayoutParameters](#)
- [ilp::autoPinPlace](#)
- [ilp::autoPlace](#)
- [ilp::copyResult](#)
- [ilp::createConstraintArea](#)
- [ilp::createPlacementArea](#)
- [ilp::extractTopology](#)
- [ilp::optimizePins](#)
- [ilp::placeToRows](#)

lp::adjustRelativeSpacing

Description

Adjusts the spacing between boxes in the relative constraint.

Syntax

```
lp::adjustRelativeSpacing -context <deContext> -constraint <string> [-x  
Factor <float>] [-yFactor <float>] [-xOffset <float>] [-yOffset <float>]
```

Arguments

-context

Specifies the context of activation.

-constraint

Specifies the name of relative constraint to adjust spacing.

-xFactor

Specifies the spacing factor of x direction (default is 1).

-yFactor

Specifies the spacing factor of y direction (default is 1).

-xOffset

Adjusts the spacing value of x direction (default is 0).

-yOffset

Adjusts the spacing value of y direction (default is 0).

Returns

<none>

lp::align

Description

Aligns instances and figGroups.

Syntax

```
lp::align -design <oaDesign> -sources [<string>] -target <string>  
-sourceEdge <string> -targetEdge <string> [-direction <string>]  
[-moveInHierarchy <bool>]
```

Arguments

`-design`

Active design.

`-sources`

List of instance or figGroup names to align to the target.

`-target`

Specifies the target instance or figGroup.

`-sourceEdge`

Specifies the source edge. Valid values: right, left, center, origin, top, bottom.

`-targetEdge`

Specifies the target edge. Valid values: right, left, center, origin, top, bottom.

`-direction`

Specifies align direction. Valid values: vertical and horizontal.

`-moveInHierarchy`

Specifies to move instances/figGroups inside the hierarchy or only on top-level.

Syntax

```
lp::align -design <oaDesign> -sources [<string>] -target <string>  
-sourceEdge <string> -targetEdge <string> [-direction <string>]  
[-moveInHierarchy <bool>] [-considerMustJoinPins <bool>]
```

Returns

<none>

Arguments

`-design`

Active design

`-sources`

List of instance names, figGroup names or terminals to align to the target

`-target`

Target instance, figGroup or terminal (reference)

`-sourceEdge`

Source edge (right | left | center | origin | top | bottom)

`-targetEdge`

Target edge (right | left | center | origin | top | bottom)

`-direction`

Align direction (vertical | horizontal)

`-moveInHierarchy`

Specify to move instances/figGroups inside the hierarchy or only on top-level

`-considerMustJoinPins`

Consider must join pins when terminal is specified (Default is: false)

Ip::applyTemplates

Description

Applies the templates found in the specified template library to the schematic, creating constraints in the layout.

Syntax

```
lp::applyTemplates <cell: deContext | dmCellView | oaDesign | string>  
-templateLibrary <oaLib | string> [-onlyIncludeRealizedInstances <bool>]
```

Arguments

`cell`

Layout cell to be constrained.

`-templateLibrary`

Library from which templates are applied.

`-onlyIncludeRealizedInstances`

Specifies to include only the realized instances.

Returns

`<dbCollection>` : Collection of cmConstraint objects.

Ip::autoPinPlace

Description

Auto pin placer.

Syntax

```
lp::autoPinPlace -mode <enum> [-file <string>] [-context <deContext>]  
[-window <string>] [-distribute <bool>]
```

Arguments

-mode

Mode of pin placer.

-file

Input constraint file for pin placer.

-context

Design context.

-window

Window Id of the top level design.

-distribute

Distribute pins on the edge.

Returns

<none>

lp::autoPlace

Description

Runs automatic placement.

Syntax

```
lp::autoPlace <cell: deContext | dmCellView | oaDesign | string>  
[-placementLibrary <oaLib | string>]
```

Arguments

cell

Cell to place.

-placementLibrary

Library to hold placement results.

Returns

<none>

Ip::checkAreaPinPlacement

Check area pin placement

Syntax

```
Ip::checkAreaPinPlacement -context <deContext>
```

Returns

<string> : Area pin placement status

Arguments

-context
Design context

Ip::checkPinPlacement

Check pin placement

Syntax

```
Ip::checkPinPlacement -context <deContext> -pinRelativeData <string>
```

Returns

<string> : Boundary pin placement status

Arguments

-context
Design context

-pinRelativeData
Pin relative data file with YAML format

Ip::checkSimilarity

Description

Checks the placement similarity of two layouts.

Syntax

```
Ip::checkSimilarity <context: deContext> [-refLayoutFile <string>] [-reference <deContext>] [-binWidth <float>] [-binHeight <float>]
```

Arguments

`-context`

Specifies the context of the target design.

`-refLayoutFile`

Specifies the layout placement file of reference design. Ignored if `-reference` is specified.

`-reference`

Specifies the context of reference design for comparison.

`-binWidth`

Specifies the width of a bin for target design in micron. (default: 1 micron).

`-binHeight`

Specifies the height of a bin for target design in micron. (default: 1 micron).

Returns

<none>

Ip::cleanUpPins

Description

Cleans up pins.

Syntax

```
Ip::cleanUpPins -context <deContext> -mode <enum>
```

Arguments

`-context`

Specifies the design context.

`-mode`

Specifies the mode to clean up pins ({ area | boundary }).

Returns

<none>

Ip::convertCustomConstraint

Convert the custom placement constraint into native placement constraint

Syntax

```
lp::convertCustomConstraint <design: oaDesign>
```

Returns

<none>

Arguments

design

Source design to convert the custom constraints.

lp::copyResult

Description

Copies placement results back to layout.

Syntax

```
lp::copyResult <cell: deContext | dmCellView | oaDesign | string> -design  
  <deContext | dmCellView | oaDesign>
```

Arguments

cell

Placement result to be copied.

-design

Design into which the result is copied.

Returns

<none>

lp::doDeviceGrouping

Description

Groups devices and generates constraint for placer.

Syntax

```
lp::doDeviceGrouping <context: deContext> [-onlyIncludeRealizedInstances  
  <bool>]
```


Arguments

`context`

The context in which this command is activated.

`-onlyIncludeRealizedInstances`

Specifies to include only the realized instances.

Returns

`<dbCollection>` : Collection of `cmConstraint` objects.

`lp::doStdCellGrouping`

Description

Groups standard cells in the given standard library for generating standard cell constraints.

Syntax

```
lp::doStdCellGrouping <context: deContext>
```

Arguments

`context`

The context in which this command is activated.

Returns

`<dbCollection>` : Collection of `cmConstraint` objects.

`lp::dumpDMPProp`

Description

Dumps the DM property.

Syntax

```
lp::dumpDMPProp -design <oaDesign>
```

Arguments

`-design`

Specifies the design into which the result is copied.

Returns

`<none>`

Ip::exportLayoutPlacement

Description

Exports the layout placement information to a YAML file.

Syntax

```
lp::exportLayoutPlacement <context: deContext> -file <string>
```

Arguments

-context

Specifies the context pointer.

-file

Specifies the name of the output file.

Returns

<none>

Ip::extractAlignment

Description

Extracts alignments.

Syntax

```
lp::extractAlignment <design: oaDesign> [-center_v <bool>] [-center_h <bool>] [-left <bool>] [-bottom <bool>] [-right <bool>] [-top <bool>]
```

Arguments

-design

Specifies the layout design.

-center_v

Extracts center alignment, (vertical).

-center_h

Extracts center alignment, (horizontal).

-left

Extracts leftEdge alignment.

`-bottom`

Extracts bottomEdge alignment.

`-right`

Extracts rightEdge alignment.

`-top`

Extracts topEdge alignment.

Returns

<none>

Ip::extractFigGroup

Extract fig group hierarchy as yaml file

Syntax

```
Ip::extractFigGroup -design <oaDesign> -file <string>
```

Returns

<none>

Arguments

`-design`

Layout design

`-file`

Export yaml file nmae

Ip::extractGRParams

Extracts GR parameters from related `oaInst` to side file.

Syntax

```
Ip::extractGRParams <context: deContext> -file <string>
```

Returns

<none>

Arguments

`context`

Layout context.

`-file`

File path storing GR params

Ip::extractGRPitchSpacing

Extracts GR spacing in pitch to side file.

Syntax

```
Ip::extractGRPitchSpacing <context: deContext> -path <string>
```

Returns

<none>

Arguments

`context`

Layout context.

`-path`

Path for side file

Ip::extractLayoutParameters

Extract layout parameters

Syntax

```
Ip::extractLayoutParameters -context <deContext> -config <string>  
[-output <string>]
```

Returns

<none>

Arguments

`-context`

Design context to extract layout parameters

`-config`

Input file to config extraction

`-output`

Output file with extracted parameters (Default is: layout_param.yaml)

lp::extractOrigin

Description

Extracts the origin of placement groups.

Syntax

```
lp::extractOrigin <design: oaDesign> [-extractToFile <string>] [-extractFromFile <string>]
```

Arguments

`design`

Specifies the design of the layout.

`-extractToFile`

Extracts origin point to the file.

`-extractFromFile`

Extracts origin point from the file.

Returns

`<giDoubleList>`: Origin point of extracted placement.

lp::extractOriginFromFile

Description

Extracts the origin of placement groups from file.

Syntax

```
lp::extractOriginFromFile <design: oaDesign> -file <string>
```

Arguments

`-design`

Specifies the layout design.

`-file`

Extracts origin point from the file.

Returns

<giDoubleList> : Origin point of extracted placement from file.

lp::extractOriginInfoFromFile

Description

Extracts detailed origin information of placement groups from file.

Syntax

```
lp::extractOriginInfoFromFile <design: oaDesign> -file <string> -mode  
    <string>
```

Arguments

-design

Specifies the layout design.

-file

Extracts origin point from the file.

-mode

Extracts information for x or y from the file.

Returns

<giDoubleList> : Origin point of extracted placement from file.

lp::extractOriginToFile

Description

Extracts the origin of placement groups to file.

Syntax

```
lp::extractOriginToFile <design: oaDesign> -file <string>
```

Arguments

-design

Specifies the layout design.

-file

Extracts origin point to the file.

Returns

<None>

Ip::extractPinPlacement

Description

Extracts pin placement.

Syntax

```
Ip::extractPinPlacement -context <deContext> -output <string> [-format  
    <enum>]
```

Arguments

-context

Specifies the design context.

-output

Specifies the output file.

-format

Specifies the output format as `csv` or `yaml`. By default, the format is `yaml`.

Returns

<none>

Ip::extractPlacement

Extract placement information for migration.

Syntax

```
Ip::extractPlacement -context <deContext> -path <string>  
    [-layoutParamConfig <string>] [-removeExisting <bool>]  
    [-extractAlignment <bool>] [-ignoreDummy <bool>] [-excludeCells  
    <string>] [-surroundCells <string>] [-extractSurroundCellsProc <string>]  
    [-surroundCellsFile <string>]
```

Returns

<none>

Arguments

`-context`

Active context.

`-path`

Path to store the extracted info.

`-layoutParamConfig`

Layout parameters config file (Default is:)

`-removeExisting`

Remove existing placement constraints (Default is: false)

`-extractAlignment`

Auto extract alignments (Default is: true)

`-ignoreDummy`

Ignore dummy device during extraction (Default is: false)

`-excludeCells`

Exclude Cells (Default is:)

`-surroundCells`

surround cell names (Default is:)

`-extractSurroundCellsProc`

User defined proc for extracting surround cells' properties (Default is:)

`-surroundCellsFile`

Surround cells info file name (Default is: surround_cells.yaml)

Ip::extractPRBoundary

Description

Extracts a PR boundary from a given design.

Syntax

```
Ip::extractPRBoundary <design: oaDesign> -output <string>
```

Arguments

`-design`

Specifies the layout design.

`-output`

Specifies the output file.

Returns

<none>

Ip::extractSEDSubgroups

Description

Extracts information of constraints from schematic of subgroups.

Syntax

```
Ip::extractSEDSubgroups <context: deContext> [-constraint <string>]
```

Arguments

`-context`

Specifies the context of activation.

`-constraint`

Specifies the constraint name to be extracted. Extracts all constraints if constraint name is not specified.

Returns

<None>

Ip::extractSurroundCells

extract surround cells, contained insts and user properties info.

Syntax

```
Ip::extractSurroundCells <context: deContext> -surroundCells <string>  
[-userDefinedProc <string>] -file <string>
```

Returns

<none>

Arguments

`context`

Layout context.

`-surroundCells`
 surround cells

`-userDefinedProc`
 user defined proc for getting user cared properties (Default is:)

`-file`
 file path to stored info

Ip::extractSymmetric

Extract symmetric constraints

Syntax

```
Ip::extractSymmetric -design <oaDesign>
```

Returns

<none>

Arguments

`-design`
 Layout design

Ip::extractTopology

Description

Extracts topology constraints from a pre-existing layout.

Syntax

```
Ip::extractTopology <cell: deContext | dmCellView | oaDesign> \  
    [-xFactor <float>] [-yFactor <float>] [-mode <string>] \  
    [-halo <bool>][-insts [<string>]]
```

Arguments

`cell`
 Cell from which layout topology is extracted.

`-xFactor`
 Horizontal spacing scale factor for spacing. (Default is: 1)

`-yFactor`

Vertical spacing scale factor for spacing. (Default is: 1)

`-mode`

Extract topology modes: relatives-only, relatives, clusters, slicing, instMatching, cmos, or resistorBank. (Default is: relatives-only)

`-halo`

Whether to use halo to try to preserve alignment and spacing: true(1) or false(0). (Default is: true)

`-insts`

Instances to extract topology.

Returns

<dbCollection> : Collection of cmConstraint objects.

Ip::fastPlace

Syntax

```
Ip::fastPlace [-context <deContext>] [-maxSolution <int>]  
[-selectedInstances <bool>] [-exploration <bool>] [-currentLevelOnly  
<bool>] [-includePassiveDevices <bool>]
```

Returns

<none>

Arguments

`-context`

Context of activation.

`-maxSolution`

Maximum solutions generated by placer

`-selectedInstances`

Place by selected instances from DN-Schematic.

`-exploration`

Explore the placement results to the placement gallery.

`-currentLevelOnly`

Place instances in current level only.

`-includePassiveDevices`

Place passive devices. (Default is: false)

Ip::generateAreaPinPlacement

Description

Generates area pin placement for a given design.

Syntax

```
Ip::generateAreaPinPlacement -context <deContext> [-createPinLabel <bool>] [-just <enum>] [-orient <enum>] [-font <enum>] [-height <float>] [-drafting <bool>] [-overbar <bool>] [-pinLabelLppMethod <enum>] [-pinLabelMapFile <string>] [-pinLabelLpp <string>]
```

Arguments

`-context`

Specifies the design context.

`-createPinLabel`

Creates pin label to generate area pins. (Default is: false)

`-just`

Specifies the justification of the `generate_area_pin_placement` with respect to the origin. ({ `centerCenter` | `centerLeft` | `centerRight` | `lowerCenter` | `lowerLeft` | `lowerRight` | `upperCenter` | `upperLeft` | `upperRight` }, default is: `centerCenter`).

`-orient`

Specifies the orientation of `generate_area_pin_placement`. ({ `MX` | `MXR90` | `MY` | `MYR90` | `R0` | `R180` | `R270` | `R90` }, default is: `R0`).

`-font`

Specifies the font style with which `generate_area_pin_placement` is displayed. ({ `euroStyle` | `fixed` | `gothic` | `math` | `milSpec` | `roman` | `script` | `stick` | `swedish` }, default is: `stick`).

`-height`

Specifies the height of `generate_area_pin_placement` in user units. (Default is: 1).

`-drafting`

Indicates whether `generate_area_pin_placement` should not be mirrored or rotated. (Default is: true)

`-overbar`

Indicates whether `generate_area_pin_placement` is rendered with overbar. (Default is: false)

`-pinLabelLppMethod`

Specifies the LPP method for pin label. ({ map | sameLpp | specify | techLib }, default is: sameLpp).

`-pinLabelMapFile`

Specifies the map file for generating pin labels.

`-pinLabelLpp`

Specifies a text LPP for pin labels.

Returns

<None>

lp::generateConstraint

Description

Generating constraints.

Syntax

```
lp::generateConstraint <context: deContext> [-applyTemplates <bool>]  
[-templateLibrary <string>] [-groupStdCells <bool>] [-stdCellLibraries  
[<string>]] [-groupDevices <bool>]  
[-onlyIncludeRealizedInstances <bool>] [-ignoreDummy <bool>]
```

Arguments

`context`

Context of activation.

`-applyTemplates`

Apply constraints in template library.

`-templateLibrary`

Template library from which templates is applied.

`-groupStdCells`
Group by standard cell libraries.

`-stdCellLibraries`
Standard cell libraries.

`-groupDevices`
Group devices.

`-onlyIncludeRealizedInstances`
Specifies to include only the realized instances.

`-ignoreDummy`
ignore dummy instances

Returns

<dbCollection> : Collection of cmConstraint objects.

lp::generatePinPlacement

Description

Generates pin placement.

Syntax

```
lp::generatePinPlacement -context <deContext> -pinRelativeData <string>  
[-layerMap <string>]  
-output <string> [-migrateWidth <bool>] [-createPins <bool>]
```

Arguments

`-context`
Design context

`-pinRelativeData`
Pin relative data file with YAML format

`-layerMap`
Layer map file with YAML format

`-output`
Pin placement file with CSV format

`-migrateWidth`

Migrate width

`-createPins`

Create pins to layout

Returns

<none>

lp::generatePlacement

Generate placement from extracted data.

Syntax

```
lp::generatePlacement -context <deContext> -path <string> [-techMapping  
  <string>] [-createPRBoundary <bool>] [-importConstraint <bool>]  
  [-realizeConstraint <bool>] [-reuseLayoutParam <bool>]
```

Returns

<none>

Syntax

```
lp::generatePlacement -context <deContext> -path <string>  
  [-techMapping <string>] [-createPRBoundary <bool>] [-importConstraint  
  <bool>] [-realizeConstraint <bool>] [-reuseLayoutParam <bool>]  
  [-generateSurroundCellsProc <string>] [-surroundCellsFile <string>]
```

Returns

<none>

Arguments

`-context`

Active context.

`-path`

Path to load the extracted info.

`-techMapping`

Technology mapping file. (Default is:)

`-createPRBoundary`

Create PR boundary (Default is: false)

`-importConstraint`
Import constraint (Default is: true)

`-fillToRectangles`
Fill placement to rectangles (Default is: false)

`-realizeConstraint`
Realize constraint (Default is: true)

`-reuseLayoutParam`
Reuse extracted layout parameters (Default is: true)

`-generateSurroundCellsProc`
User defined procedure for generating surround cells

`-surroundCellsFile`
Surround cells info file name (Default is: surround_cells.yaml)

Ip::generatePRBoundary

Generate PR boundary into the given design

Syntax

```
Ip::generatePRBoundary <design: oaDesign> -input <string> [-scalingMethod  
  <enum>]
```

Returns

<none>

Arguments

`design`

Layout design

`-input`

PR boundary config file with YAML format

`-scalingMethod`

Method to scale offset ({ exact_value | percentage }, default is: percentage)

Ip::generateSurroundCells

generate surround cells.

Syntax

```
lp::generateSurroundCells -context <deContext> -userDefinedProc <string>  
-file <string>
```

Returns

<none>

Arguments

-context

Layout context.

-userDefinedProc

user defined proc for generating surround cells

-file

file path storing surround cells info

lp::getGroupVectorName

Description

Returns the vector name for an lxInst object.

Syntax

```
lp::getGroupVectorName <inst: lxInst> -session <lxSession>
```

Arguments

inst

The lxInst object for which the vector name is returned.

-session

The lxSession object to which the lxInst belongs.

Returns

<string> : The vector name of the lxInst object

lp::groupByGuardRing

Group instances by mcell guard ring.

Syntax

```
lp::groupByGuardRing <context: deContext> -leaf <bool>
```

Returns

<dbCollection> : Collection of cmConstraint objects

Arguments

context

context pointer

-leaf

Group to a leaf constraint

Ip::groupBySurroundCells

Group instances by surround cells.

Syntax

```
Ip::groupBySurroundCells <context: deContext> -surroundCells <string>
```

Returns

<dbCollection> : Collection of cmConstraint objects

Arguments

context

Layout context.

-surroundCells

surround cells

Ip::groupDeviceByArea

Description

Creates constraint areas to give the constraint generator to create the specific constraint in the created areas.

Syntax

```
Ip::groupDeviceByArea <context: deContext>
```

Arguments

context

Context of activation.

Returns

<dbCollection> : Collection of cmConstraint objects.

lp::handleConstraintArea

Description

Save or load the constraint area for a design.

Syntax

```
lp::handleConstraintArea -lineDesign <oaDesign> -dataDesign <oaDesign>  
-mode <string>
```

Arguments

-lineDesign

Design in which the constraint area exists.

-dataDesign

Design in which the constraint area is stored.

-mode

Choose the handling mode {save, load}.

Returns

<none>

lp::importPinPlacement

Description

Imports pin placement.

Syntax

```
lp::importPinPlacement -context <deContext | oaDesign> -input <string>
```

Arguments

-context

Specifies the design context or OA design.

-input

Specifies the input file in `csv` format.

Returns

<none>

Ip::insertBoundary

Description

Insert boundary cells.

Syntax

```
Ip::insertBoundary <design: oaDesign> -mode <string> [-prefix <string>]  
[-setFixed <bool>] [-rowPattern <string>] [-honorPRBoundary <bool>]
```

Arguments

`design`

Design from which boundary cell is inserted.

`-mode`

Place boundary cells to outside or inside of boundary. Valid values: negative or positive. (Default: positive)

`-prefix`

Prefix for names of inserted boundary cells. (Default: Ip_boundary)

`-setFixed`

Set inserted boundary cells to be fixed. (Default: false)

`-rowPattern`

PN type of inserted horizontal boundary cells (starting from the bottom row).

`-honorPRBoundary`

Honor the PR Boundary if the width / height is not a multiple of 'site width' / 'row height'. By default, the value is false.

Returns

{<oaInst>} : Collection of created instances.

Ip::insertColumn

Description

Insert column cells.

Syntax

```
lp::insertColumn <design: oaDesign> [-prefix <string>] [-setFixed  
<bool>] [-minDistance <float>] -maxDistance <float> [-pitch <float>]  
[-boundaryDistance <float>] [-xRange [<float>]] [-minGap <float>]  
[-rowPattern <string>]
```

Arguments

`design`

Design from which column cells is inserted.

`-prefix`

Prefix for name of inserted column cells. (Default: `lp_column`)

`-setFixed`

Set inserted column cells to be fixed. (Default: `false`)

`-minDistance`

Minimum distance between two adjacent columns. (Default: $0.5 * \text{maxDistance}$)

`-maxDistance`

Maximum distance between two adjacent columns.

`-pitch`

Default center-to-center distance between two adjacent columns. (Default: $0.5 * (\text{minDistance} + \text{maxDistance})$)

`-boundaryDistance`

Distance between boundary and left-most columns. (Default: $0.5 * \text{maxDistance}$)

`-xRange`

Range of x coordination to insert column cells to.

`-minGap`

Minimum gap between each inserted column cell and its nearby instances.
(Default: `0`)

`-rowPattern`

PN type of inserted boundary column cells. (starting from the bottom row).

Returns

`{<oaInst>}` : Collection of created instances.

Ip::insertDecapFiller

Insert decap and filler cells

Syntax

```
Ip::insertDecapFiller <design: oaDesign> [-decapPrefix <string>]  
[-fillerPrefix <string>] [-setFixed <bool>] [-boundaryLpp <lpp>]  
[-insertDecap <bool>] [-insertFiller <bool>] [-depth <int>]
```

Returns

{<oaInst>} : Collection of created instances

Arguments

`design`

Design from which decap and filler will be inserted.

`-decapPrefix`

Prefix for names of inserted decap cells (default: lp_decap)

`-fillerPrefix`

Prefix for names of inserted filler cells (default: lp_filler)

`-setFixed`

Set inserted decap/filler cells to be fixed (default: false)

`-boundaryLpp`

LPP of boundary layer

`-insertDecap`

To insert decap cell (default is true)

`-insertFiller`

To insert filler cell (default is true)

`-depth`

The depth to traverse instances hierarchically (Default is: 0)

Ip::insertDiode

Description

Insert diode cells.

Syntax

```
lp::insertDiode -nets [<{<oaNet>}>] [-prefix <string>] [-setFixed <bool>]  
-maxDistance <float> [-checkExistence <bool>] [-minGap <float>]
```

Arguments

-nets

List of nets on which to insert diode cells.

-prefix

Prefix for names of inserted diode cells. (Default: lp_diode)

-setFixed

Set inserted diode cells to be fixed. (Default: false)

-maxDistance

Maximum distance between inserted diode cell and the primary input pin.

-checkExistence

Check if there is any pre-placed diode cell on target nets. (Default: true)

-minGap

Minimum gap between each inserted diode cell and its nearby instances.
(Default: 0)

Returns

{<oaInst>} : Collection of created instances.

lp::insertTap

Description

Insert tap cells.

Syntax

```
lp::insertTap <design: oaDesign> [-prefix <string>] [-setFixed <bool>]  
[-initialPlaceOnly <bool>] [-minDistance <float>] -maxDistance <float>  
[-pitch <float>] [-boundaryDistance <float>] [-skipAlternateRowCount  
<int>] [-skipBottomRowCount <int>] -mode <string> [-checkExistence  
<bool>] [-minGap <float>] [-type <string>] [-leftBoundary <bool>]  
[-rightBoundary <bool>]
```

Arguments

`design`

Design from which tap filler is inserted.

`-prefix`

Prefix for names of inserted tap cells. (Default: `lp_tap`)

`-setFixed`

Set inserted tap cells to be fixed. (Default: `false`)

`-initialPlaceOnly`

Insert the tap cells with input parameters and ignore cell overlaps. (Default: `false`)

`-minDistance`

Minimum distance between two adjacent tap cells. (Default: $0.5 * \text{maxDistance}$)

`-maxDistance`

Maximum distance between two adjacent tap cells.

`-pitch`

Default center-to-center distance between two adjacent tap cells. (Default: $0.5 * (\text{minDistance} + \text{maxDistance})$)

`-boundaryDistance`

Distance between boundary and lower-left tap cell. (Default: $0.5 * \text{maxDistance}$)

`-skipAlternateRowCount`

Skip every alternate N rows after inserting a rows of tap cells. (Default: 1)

`-skipBottomRowCount`

Skip first N rows before inserting tap cells. (Default: 0)

`-mode`

Tap insertion modes `columnAligned` or `checkerboard`.

`-checkExistence`

Check if there is any pre-placed tap cell. (Default: `true`)

`-minGap`

Minimum gap between each inserted tap cell and its nearby instances. (Default: 0)

`-type`

Insert tap type. Valid values: n and p. (Default: p)

`-leftBoundary`

Insert the left boundary tap cell. (Default: false)

`-rightBoundary`

Insert the right boundary tap cell. (Default: false)

Returns

{<oaInst>} : Collection of created instances.

Ip::isReadOnly

Description

Determines if input context is read-only.

Syntax

```
Ip::isReadOnly <context: deContext>
```

Arguments

`context`

Context of activation.

Returns

<string> : is read only or not.

Ip::optimizePins

Description

Optimizes pins.

Syntax

```
Ip::optimizePins -pins <{<oaPin>}> -design <oaDesign>[-keepEdge <bool>]  
[-keepOrder <bool>][-vertPinGap <int>][-horzPinGap <int>][-alignPath  
<enum>]
```

Arguments

`-pins`

Pin collection to be optimized.

`-design`

The top level design.

`-keepEdge`

Whether to keep original edge of pins.

`-keepOrder`

Whether to keep order of pins.

`-vertPinGap`

Defines the vertical gap between pins.

`-horzPinGap`

Defines the horizontal gap between pins.

`-alignPath`

Defines align path method ({ keepOriginalLayer | none | usePathLayer }, default is: none).

Returns

{<oaPin>} : Pin collection of optimized pins.

lp::placementDebugger

Description

Placement debugger.

Syntax

```
lp::placementDebugger <design: oaDesign> -file <string>
```

Arguments

`-design`

Specifies the design to draw.

`-file`

Specifies the file name that stores box information.

Returns

<None>

Ip::placeToRows

Auto row placer

Syntax

```
lp::placeToRows -design <oaDesign> [-insts <{<oaInst>}>] [-compact  
  <enum>] [-ioNetWeight <int>]  
  [-prewireNetWeight <int>] [-multiplePinNetWeight [<int>]]
```

Returns

<none>

Arguments

-design

Design

-design

Design

-insts

List of insts to place

-compact

Compact mode ({ left | none | right }, default is: none)

-ioNetWeight

I/O net weight

-prewireNetWeight

Prewire net weight

-multiplePinNetWeight

Multiple pin net weight

Ip::rebuildFigGroup

Re-build fig group hierarchy as described in yaml file

Syntax

```
lp::rebuildFigGroup -design <oaDesign> -file <string>
```

Returns

<none>

Arguments

-design

Layout design

-file

Input yaml file nmae

Ip::saveDesignToPlacementLib

Description

Save current design to place library if it is read-only.

Syntax

```
Ip::saveDesignToPlacementLib <context: deContext>
```

Arguments

context

Context of activation.

Returns

<string> : Newly created lib/cell/view name.

Ip::showOptions

Description

Opens Placement Options dialog.

Syntax

```
Ip::showOptions [-parent <giWindow | int>]
```

Arguments

-parent

Window or id of parent window from which to open Placement Options dialog.

Returns

<giDialog> : The Dialog.

Ip::testGenerateGrouping

Syntax

```
lp::testGenerateGrouping <design: oaDesign>
```

Returns

<none>

Arguments

design

Source design to perform the grouping.

lp::updateConstraint

Description

Updates constraints.

Syntax

```
lp::updateConstraint <context: deContext> -file <string>
```

Arguments

-context

Specifies the context of activation.

-file

Specifies the file name to update the constraint.

Returns

<none>

lp::updateGRSpacing

Updates GR spacing value from side file.

Syntax

```
lp::updateGRSpacing <context: deContext> -path <string> -techMap <string>
```

Returns

<none>

Arguments

`context`

Layout context.

`-path`

Path for side file

`-techMap`

Path for techMap file

lp::updateLayoutParameters

Update parameters for instances

Syntax

```
lp::updateLayoutParameters -context <deContext> -input <string>  
[-layoutDesign <oaDesign>] [-mapFile <string>]
```

Returns

<none>

Syntax

```
lp::updateLayoutParameters -design <oaDesign> [-input <string>]  
[-layoutDesign <oaDesign>] [-mapFile <string>]  
[-customFile <string>] [-instName [<string>]] [-instOrder [<string>]]
```

Returns

<none>

Arguments

`-design`

Design to update layout parameters

`-input`

Input file with parameters for update

`-layoutDesign`

Current layout design with SDL

`-mapFile`

Mapping file with parameter name/value mapping

`-customFile`

The file defined custom parameters

`-instName`

The instances to be updated parameters.

`-instOrder`

The instance order for instance mapping

ilp::autoPinPlace

Description

Interactive command to run automatic pin placement.

Syntax

```
ilp::autoPinPlace [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ilp::autoPlace

Description

Interactive command to run automatic placement.

Syntax

```
ilp::autoPlace [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ilp::copyResult

Description

Interactive command to copy placement results back to layout.

Syntax

```
ilp::copyResult [<context: deContext | unsigned int>]
```

Arguments

context

Context or window ID in which to start the **Copy Result** command.

Returns

<none>

ilp::createConstraintArea

Description

Starts the interactive Create Constraint Area command.

Syntax

```
ilp::createConstraintArea [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ilp::createPlacementArea

Description

Interactive command to define and create a placement area.

Syntax

```
ilp::createPlacementArea [<deContext | unsigned int>]
```


Arguments

`context`

Context or window ID in which to start the **Create Placement Area** command.

Returns

<none>

ilp::extractTopology

Description

Interactive command to extract the layout topology of the selected objects and created constraints.

Syntax

```
ilp::extractTopology <context: deContext | unsigned int>
```

Arguments

`context`

Context or window ID in which to start the **Extract Topology Constraints** command.

Returns

<none>

ilp::optimizePins

Description

Interactive command to perform pin optimization.

Syntax

```
ilp::optimizePins [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ilp::placeToRows

Interactive command to perform place to rows

Syntax

```
ilp::placeToRows [<context: deContext | unsigned int>]
```

Returns

<none>

Arguments

context

Context of activation

Enumerations

Table 140 Enumerations

Name	Values	Description
enBoundaryMethod	promote bBox	Indicates how oaPRBoundary is created.
enExtractMode	clusters, relatives, slicing	Indicates the mode to be used for fixing the non-slicing part when extracting topology constraints from an existing placement.
enGalleryAlignment	horizontal vertical	Indicates the alignment of the placement gallery with respect to the edit canvas.
enLabelJustification	upperLeft centerLeft lowerLeft upperCenter centerCenter lowerCenter upperRight centerRight lowerRight	Indicates justification of label text in the placement gallery.
enLabelOrientation	R0 R90 R180 R270 MX MY MXR90 MYR90	Indicates orientation of labels in the placement gallery.
enResultFilterAttribute	width height aspectRatio area connectivityMetric utilizationRatio	Indicates names of result filter attributes that can be specified in <i>lpDefaultResultFilters</i> preference.

Preferences

[Table 141](#) lists lp namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 141 Placement Preferences (Sheet 1 of 8)

Name	Type	Scope	Default	Description
lpAlignBoundaryLayersToAxis	bool	cell	false	Specifies whether to align boundary layers to the axes. Valid values: true, false
lpAutoOpenBrowser	bool	user	false	Specifies whether (true) or not (false) to automatically open the results browser when the placement gallery is opened. Valid values: true, false
lpAutoOpenGallery	bool	user	true	Specifies whether (true) or not (false) to automatically open the placement gallery when placement is complete. Valid values: true, false
lpAverageTrackPitch	float	cell	0	Specifies how connectivity cost is mapped to spacing between rows in constraints that support it: Rows, Column, Inst Matching. The larger the value, the larger the spacing. Default (0) means to use minimum-width + minimum-spacing values for the second metal layer in the technology.
lpBlockageLPPs	string	cell		Specified LPPs should be treated as blockage in placer engine.
lpBoundaryMethod	enBound aryMet hod	cell	promote	Specifies how oaPRBoundary is created.
lpBridging	bool	cell	TRUE	Perform layer bridging if lpBridging is true. Valid values: true, false
lpCalculateGRSpacingOldFlow	bool	user	false	Uses old flow when calculate GR spacing.
lpConfigFile	string	user		Placer to run with config file

Table 141 Placement Preferences (Sheet 1 of 8) (Continued)

Name	Type	Scope	Default	Description
IpConnectivityWeight	float	cell	0.5	Specifies the weight of the connectivity objective in the placement optimization function.
IpConstraintAreaActionTypePref	string	cellview	Create	Action Type.
IpConstraintAreaCMOSLPPPref	string	cellview		LPP name for CMOS.
IpConstraintAreaInstMatchingLPPPref	string	cellview		LPP name for InstMatching.
IpConstraintAreaRelativeLPPPref	string	cellview		LPP name for Relative mode.
IpConstraintAreaResistorBankLPPref	string	cellview		LPP name for ResistorBank.
IpConstraintAreaTypePref	string	cellview	InstMatching	Constraint Type.
IpCopyResultReplaceExistingObjects	bool	user	true	Specifies whether (true) or not (false) the objects copied from the placement results gallery by the Copy Result command may replace the objects already existing in the target layout. Valid values: true, false
IpCreatePRBoundaryFromOrigin	bool	user	true	Placer to create PR Boundary starting from origin.
IpCreateTemplateConstraint	bool	user	false	Specifies whether (true) or not (false) to create a template constraint in the constraint hierarchy for each template that is matched to the netlist by the Apply Templates command. Valid values: true, false
IpDefaultAspectRatio	string	cell	""	Preference for setting the min and max aspect ratio. This has lower precedence than specified BBox, prBoundary and block size intent. For example: {min:X max:Y}
IpDefaultResultFilters	string	cell	""	List of default filters to be applied in Result Browser Assistant. Default interpreted as "{width {} height {} area {} aspectRatio {}}".

Table 141 Placement Preferences (Sheet 1 of 8) (Continued)

Name	Type	Scope	Default	Description
lpDensityWeight	float	cell	0.0	Specifies the weight of the density objective in the placement optimization function.
lpDeviceBoundary	string	user	instBoundary	Placer to get device boundary (instBoundary activeLayer).
lpDeviceDelimiters	string	user	""	Preference for setting device delimiters at user scope. Default interpreted as ".".
lpEnforceDRC	bool	user	TRUE	Enforce DRC spacing between devices of same row and different rows. Valid values: true, false
lpExtractTopologyHalo	bool	user	true	Specifies whether (true) or not (false) to use a halo to try to preserve alignment when extracting topology constraints from an existing placement. Valid values: true, false
lpExtractTopologyMode	extract_mode	user	relatives-only	Specifies the mode for fixing the non-slicing part when extracting topology constraints from an existing placement. Valid values: relatives-only, relatives, clusters, slicing
lpExtractTopologyXFactor	float	user	1.0	Specifies the horizontal spacing scale factor used when extracting topology constraints from an existing placement.
lpExtractTopologyYFactor	float	user	1.0	Specifies the vertical spacing scale factor used when extracting topology constraints from an existing placement.
lpFirmConstraintsUponCopy	bool	user	true	Specifies whether (true) or not (false) to 'firm' constraints (for example, vpattern rows) when copying back from the placement results gallery to the target design. Valid values: true, false

Table 141 Placement Preferences (Sheet 1 of 8) (Continued)

Name	Type	Scope	Default	Description
IpForceOverWriteResults	bool	user	true	Specifies whether (true) or not (false). If the preference is set to (false), in the case that the results from the previous run in the placement library exist, those results are not deleted and the placement execution aborts with a warning message. The execution proceeds normally if there are no solutions in the placement library from a previous run. Valid values: true, false
IpGalleryAlignment	gallery_alignment	user	vertical	Specifies the alignment of the gallery as it pertains to the edit canvas.
IpGalleryLabelHeight	float	user	0.0	Specifies an override for the text height of labels in the placement gallery.
IpGalleryLabelJustification	label_justification	user	""	Specifies an override for the text justification of labels in the placement gallery. Valid values: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight
IpGalleryLabelLPP	string	user	"text drawing"	Specifies an override for the LPP of labels in the placement gallery.
IpGalleryLabelOrientation	label_orientation	user	""	Specifies an override for the text orientation of labels in the placement gallery. Valid values: R0, R90, R180, R270, MX, MY, MXR90, MYR90
IpHaloConstraintHorizontalLayer	string	cell		Halo constraint use the layer's pitch as horizontal unit.
IpHaloConstraintVerticalLayer	string	cell		Halo constraint use the layer's active track pitch as vertical unit.
IpHonorPRBoundary	bool	cell	false	Whether to place within a specified PRBoundary Valid values: true, false

Table 141 Placement Preferences (Sheet 1 of 8) (Continued)

Name	Type	Scope	Default	Description
IpHonorPRBoundaryDimension	bool	cell	false	Preference to control whether the prBoundary in the current design should be honored by the placer or not. This allows you to go back-and-forth between exploration and design refinement needs. Valid values: true, false
IpIdentifySideIndex	bool	user	true	Show index to identify the sides.
IpInitPinSideFromSch	bool	user	false	Initializes the pin side associated with the location in schematic. Valid values: true, false
IpLayoutAreaWeight	float	cell	0.5	Specifies the weight of the layout area objective in the placement optimization function.
IpMinDummySize	int	cell	1	Minimum poly pitch of the dummy instance.
IpMaxNumResults	int	cell	6	Specifies the maximum number of placement results generated for each design.
IpMaxNumRetryInBackground	int	user	2147483647	Maximum number to retry in background mode.
IpMaxNumUnconstrained	int	user	10	Maximum number of unconstrained instances that can be placed without a warning
IpOptimizePinAlignPath	string	user	none	Optimizes pin align path.
IpOptimizePinsDialogMode	string	user	false	<code>ilp::optimizePins</code> command options dialog eject state.
IpOptimizePinHorzPinGap	int	user	1	Minimum track gap on horizontal direction when optimizing pins.
IpOptimizePinKeepEdge	bool	user	false	Whether to keep original pin edge when pin optimization.
IpOptimizePinKeepOrder	bool	user	false	Whether to keep original pin order when pin optimization. Valid values: true, false

Table 141 Placement Preferences (Sheet 1 of 8) (Continued)

Name	Type	Scope	Default	Description
lpOptimizePinVertPinGap	int	user	1	Minimum track gap on vertical direction when optimizing pins.
lpPinTopMetal	string	cell	""	Top metal layer name on which pin is created.
lpPlaceBackground	bool	user	false	Specifies whether (true) or not (false) to run interactive command Auto Place job(s) in the background. Valid values: true, false
lpPlacedCellNameSeparator	string	user	"@"	Separator string used between target cell and library names to generate the placed cell name. When empty, the target cell name is used as the placed cell name.
lpPlacementArea	string	cellview	""	Placement area polygon coordinates
lpPlacementLibrary	string	user	""	Specifies the name of the placement library. When empty, the placement library name is <designLibrary>_HLX and the library is created in the directory specified by <i>dmAddLibDir</i> preference.
lpPlacePackedPartition	bool	user	FALSE	Use PackedPartition for placement. Valid values: true, false
lpPlacePGPins	bool	cell	false	Specifies whether to place power and ground pins. Valid values: true, false
lpPlacePinPreRouteWeight	float	user	0	Net weight for pre-route shapes.
lpPlacePinSnapToTrack	bool	user	true	Snap pins to track if <i>lpPlacePinSnapToTrack</i> is true.
lpPlaceSelected	bool	user	TRUE	Place selected objects only. Valid values: true, false
lpPlaceToRowsCompact	string	user	none	Compact mode
lpPlaceToRowsDialogMode	bool	user	false	<i>ilp::placeToRows</i> command options dialog eject state.

Table 141 Placement Preferences (Sheet 1 of 8) (Continued)

Name	Type	Scope	Default	Description
IpPlaceToRowsInstances	string	user	all	Instances mode.
IpPlaceToRowsIONets	bool	user	false	Specifies whether to set IO nets value in memory placement GUI.
IpPlaceToRowsIONetsValue	int	user	5	IO nets value in memory placement GUI.
IpPlaceToRowsPinsOnNet	bool	user	false	Specifies whether to set Pins on net value in memory placement GUI.
IpPlaceToRowsPinsOnNetValue	int	user	5	Pins on net value in memory placement GUI.
IpPlaceToRowsPinsValue	int	user	5	Pins value in memory placement GUI.
IpPlaceToRowsPrewireNets	bool	user	false	Specifies whether to set prewire nets value in memory placement GUI
IpPlaceToRowsPrewireNetsValue	int	user	5	Prewire nets value in memory placement GUI.
IpPowerMult	int	cell	0	Specifies the power multiplier.
IpRunApplyTemplates	bool	user	true	Specifies whether (true) or not (false) to apply templates when generating constraints. Valid values: true, false
IpRunGroupDevices	bool	user	true	Specifies whether (true) or not (false) to group devices when generating constraints. Valid values: true, false
IpRunGroupStdCells	bool	user	true	Specifies whether (true) or not (false) to group std cells when generating constraints. Valid values: true, false
IpShowIncompleteResults	bool	user	true	Specifies whether (true) or not (false) incomplete results are displayed in the placement gallery. Valid values: true, false
IpSoftClusterWeight	float	cell	1.0	Specifies the weight of the soft cluster objective in the placement optimization function.

Table 141 Placement Preferences (Sheet 1 of 8) (Continued)

Name	Type	Scope	Default	Description
IpSpacerPinRatio	float	cell	0	Controls spreading of the pins on the edges. Value must be between 0 and 1. Value of 1 means the pin shapes are placed as apart as possible. Value of 0 means the pin creation code does not make an effort to place them apart.
IpSquarePins	bool	cell	false	Specifies whether to create square pins. Valid values: true, false
IpStandardCellsLibList	string	user	""	Specifies a list of libraries containing standard cells.
IpTemplateLibrary	string	user	""	Specifies the name of the template library, to be used by the Apply Templates command.
IpTopDownGenerateConstraint	bool	user	false	Generates constraints from the top down during placement. Valid values: true, false
IpTopDownGenerateConstraintOnly	bool	user	false	Top down generate constraints during placement. Valid values: true, false
IpTrackPatternList	string	cell	""	List of track patterns to use.

16

Schematic Driven Layout Tcl

For details, see

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

The commands covered are

- [Ix::addPenalty](#)
- [Ix::applyPresetParams](#)
- [Ix::backAnnotateLDE](#)
- [Ix::batchCreateTemplate](#)
- [Ix::changeInstanceMapping](#)
- [Ix::checkFeature](#)
- [Ix::clearAnchors](#)
- [Ix::clearCrossHighlight](#)
- [Ix::clearHighlight](#)
- [Ix::clearNetVoltage](#)
- [Ix::clearRuler](#)
- [Ix::createParasiticView](#)
- [Ix::createPinsOnTop](#)
- [Ix::createSession](#)

- [lx::dumpMismatchButton](#)
- [lx::ecoPostUpdateLayout](#)
- [lx::ecoPreUpdateLayout](#)
- [lx::ecoUpdateLayout](#)
- [lx::establishCorrespondence](#)
- [lx::establishCorrespondenceFromLVS](#)
- [lx::execParasiticExtraction](#)
- [lx::expandSchematic](#)
- [lx::exportSdlNetlist](#)
- [lx::extractFeatures](#)
- [lx::extractParasiticTrainingData](#)
- [lx::extractTemplate](#)
- [lx::findCloneCandidates](#)
- [lx::generateEdtext](#)
- [lx::generatePlacementCatalog](#)
- [lx::getAssistantCanvas](#)
- [lx::getAssociatedObject](#)
- [lx::getAssociatedObjectByName](#)
- [lx::getBrowserCanvas](#)
- [lx::getDesignFromSED](#)
- [lx::getDeviceInfo](#)
- [lx::getFoldingParam](#)
- [lx::getFoldingThreshold](#)
- [lx::getMismatchedConstraints](#)
- [lx::getPlacement](#)
- [lx::getPseudoParallelNets](#)
- [lx::getSchematicDummies](#)

- [lx::getSessions](#)
- [lx::highlightFlightline](#)
- [lx::incrementalRealizeConstraint](#)
- [lx::importSDLMappingFile](#)
- [lx::layoutHierarchyChangeECOUpdate](#)
- [lx::loadAddShapes](#)
- [lx::loadAddShapeSettings](#)
- [lx::loadCellTemplate](#)
- [lx::parasiticBackAnnotate](#)
- [lx::plePostProcess](#)
- [lx::predictParasitic](#)
- [lx::preview](#)
- [lx::realize](#)
- [lx::realizeConstraint](#)
- [lx::realizeTemplate](#)
- [lx::rechain](#)
- [lx::registerLayoutTemplate](#)
- [lx::reloadSymbolicEditor](#)
- [lx::removePeriphery](#)
- [lx::reopenLogicalSource](#)
- [lx::replacePlacement](#)
- [lx::resetLayoutNets](#)
- [lx::saveAddShapes](#)
- [lx::saveAddShapeSettings](#)
- [lx::saveCellTemplate](#)
- [lx::saveConstraint](#)
- [lx::saveTemplateConstraint](#)

- [Ix::sedCreateSession](#)
- [Ix::sedDestroySession](#)
- [Ix::sedGetDesign](#)
- [Ix::sedGetGridSize](#)
- [Ix::sedSetParam](#)
- [Ix::sedSetPreset](#)
- [Ix::setPlacement](#)
- [Ix::setupSDL](#)
- [Ix::showAddShapeSettingsDialog](#)
- [Ix::showAllNet](#)
- [Ix::showBatchCreateTemplate](#)
- [Ix::showCellTemplate](#)
- [Ix::showCheckerDialog](#)
- [Ix::showCheckOptions](#)
- [Ix::showCreateTemplate](#)
- [Ix::showDebugMessage](#)
- [Ix::showDefineSource](#)
- [Ix::showDefineTarget](#)
- [Ix::showExpandSchematic](#)
- [Ix::showGateConnected](#)
- [Ix::showGateNet](#)
- [Ix::showGenerateLayout](#)
- [Ix::showImportLayout](#)
- [Ix::showLHCGenerateInstanceMap](#)
- [Ix::showLHCGenerateSchematic](#)
- [Ix::showODNet](#)
- [Ix::showOptions](#)

- `lx::showParasiticEstimationByMLDialog`
- `lx::showPresetSetup`
- `lx::showSDLCheckAndRepairWindow`
- `lx::showSDLMapping`
- `lx::showSymbolicDesignOptions`
- `lx::showUpdateSchematic`
- `lx::splitDevice`
- `lx::symbolic`
- `lx::templateDesigner`
- `lx::testSaveTemplateConstraint`
- `lx::unregisterLayoutTemplate`
- `lx::updateConstraints`
- `lx::updateConstraintBySDL`
- `lx::updateInstanceCache`
- `lx::updateLayout`
- `lx::verify`
- `lx::verifyHierarchy`
- `lx::verifyNetVoltage`
- `ilx::add`
- `ilx::addDummy`
- `ilx::addIncremental`
- `ilx::addPeripheralDummy`
- `ilx::alignGates`
- `ilx::alignIndividual`
- `ilx::alignRow`
- `ilx::applyPresetParams`
- `ilx::associateSpareDummy`

- [ilx::calculateGridSize](#)
- [ilx::chainSerialRes](#)
- [ilx::clone](#)
- [ilx::copy](#)
- [ilx::customizeSpacing](#)
- [ilx::delete](#)
- [ilx::deleteTrack](#)
- [ilx::distance](#)
- [ilx::establishCorrespondence](#)
- [ilx::flip](#)
- [ilx::flipRow](#)
- [ilx::foldGates](#)
- [ilx::generateGroupPlacements](#)
- [ilx::generateGuardRings](#)
- [ilx::generatePattern](#)
- [ilx::generatePlacement](#)
- [ilx::highlightFlightline](#)
- [ilx::insertColumn](#)
- [ilx::insertRow](#)
- [ilx::insertShift](#)
- [ilx::insertTrack](#)
- [ilx::manualCorrespondence](#)
- [ilx::merge](#)
- [ilx::mergeGates](#)
- [ilx::mirror](#)
- [ilx::modifyDummy](#)
- [ilx::modifyFlightline](#)

- [ilx::modifyPode](#)
- [ilx::modifyPreset](#)
- [ilx::move](#)
- [ilx::moveFlightline](#)
- [ilx::moveInstance](#)
- [ilx::optimizeOD](#)
- [ilx::pickAndPlace](#)
- [ilx::reassignNet](#)
- [ilx::rechain](#)
- [ilx::removeColumn](#)
- [ilx::removeLayoutParam](#)
- [ilx::removeRow](#)
- [ilx::removeShift](#)
- [ilx::reportSummary](#)
- [ilx::resetCustomizedSpacing](#)
- [ilx::resetIndividualAlignment](#)
- [ilx::rotate](#)
- [ilx::sedRealize](#)
- [ilx::selectEmpty](#)
- [ilx::split](#)
- [ilx::splitDevice](#)
- [ilx::swapDevice](#)
- [ilx::swapInterdigitate](#)
- [ilx::swapSD](#)
- [ilx::swapTerm](#)
- [ilx::symbolicEditor](#)
- [ilx::transpose](#)

- [ilx::unalignGates](#)
- [ilx::unsetFingerGroup](#)
- [ilx::updateLayout](#)
- [ilx::viaTrackPlanner](#)

lx::addPenalty

Description

Add Penalty Tiles.

Syntax

```
lx::addPenalty -window <int>
```

Arguments

-window

Window to add penalty tiles.

Returns

<none>

lx::applyPresetParams

Description

Applies a preset parameter to layout instances.

Syntax

```
lx::applyPresetParams <objects: {<deFigure>} | {<deSelectionSet>}  
| {<oaFig>} | {<oaFigGroup>} | {<oaInst>}}> -lcv <dmCellView |  
dmCellViewName>
```

Arguments

objects

List of instances to apply preset parameters.

-lcv

Option to specify the preset lcv that is used to apply on the layout instances.

lx::backAnnotateLDE

Description

Back-annotates LDE parameters.

Syntax

```
lx::backAnnotateLDE -cellView <dmCellView | dmCellViewName> -refFile  
  <string> [-format <enum>] [-case <enum>] -tool <string> [-netlistFile  
  <string>] [-spFile <string>] [-propertyFile <string>] -outputFile  
  <string> [-spfFile <string>]
```

Arguments

-cellView

The source cellview.

-refFile

The reference netlist file.

-format

Reference netlist format. ({ CDL | HSPICE }) (Default is: HSPICE)

-case

Case sensitive option for the reference netlist. ({ Lower | Sensitive | Upper })
(Default is: Lower)

-tool

The tool that extracted parameters.

-netlistFile

The extracted netFile.

-spFile

The extracted netlist file.

-propertyFile

The extracted property annotation file.

-outputFile

The output netlist file.

-spfFile

The input spf file.

lx::batchCreateTemplate

Batch create template for each constraint group.

Syntax

```
lx::batchCreateTemplate -context <deContext> -templateLib <string>
```

Returns

<none>

Arguments

-context

Context of activation.

-templateLib

Template library

lx::changeInstanceMapping

Description

Changes the instance mapping.

Syntax

```
lx::changeInstanceMapping -design <oaDesign> -instName <string> -lcv  
<dmCellView | dmCellViewName>
```

Arguments

-design

The target edit layout design.

-instName

The logical instance name.

-lcv

The target mapping cellview.

lx::checkFeature

Description

Checks whether the feature is defined on the license server.

Syntax

```
lx::checkFeature <name: string>
```

Returns

<bool> : Result

Arguments

name

Name of the licensing feature to check.

lx::clearAnchors

Description

Clears anchors of nets or vias.

Syntax

```
lx::clearAnchors (-design <oaDesign> | -cellView <dmCellView |  
dmCellViewName>) [-keepByNames <string>] [-keepBySignalTypes <string>]
```

Arguments

-design

Layout design is cleared via or net shape anchors.

-cellView

Layout cellView is cleared via or net shape anchors.

-keepByNames

Skip to remove anchors by net name.

-keepBySignalTypes

Skip to remove anchors by signal types.

Returns

<none>

lx::clearCrossHighlight

Description

Clears the cross highlight from the Symbolic Editor.

Syntax

```
lx::clearCrossHighlight -window <giWindow | int>
```

Arguments

-window

The Symbolic Editor The window (ID) in which to clear the highlight.

lx::clearHighlight

Description

Clears all highlights for highlighted flight lines in the Symbolic Editor.

Syntax

```
lx::clearHighlight -window <int>
```

Arguments

-window

Specifies the window in which to clear the highlighted flight lines.

Examples

```
lx::clearHighlight -windows 2
```

lx::clearNetVoltage

Description

Clears the net voltage information in a logical net.

Syntax

```
lx::clearNetVoltage -design <oaDesign>
```

Arguments

-design

Specifies the layout design containing a net voltage to be cleared.

lx::clearRuler

Description

Clears the ruler.

Syntax

```
lx::clearRuler -window <int>
```

Arguments

-window

Specifies the window in which to clear the ruler.

lx::createParasiticView

Description

Creates a parasitic view.

Syntax

```
lx::createParasiticView -src <dmCellView | dmCellViewName> -dst  
<dmCellView | dmCellViewName> [-schematic <dmCellView | dmCellViewName>]  
[-overwrite <bool>]
```

Arguments

-src

Cell view of source layout view.

-dst

Cell view of destination netlist view.

-schematic

Cell view of source schematic view.

-overwrite

Overwrite existing destination netlist view. (Default: false)

Returns

<none>

lx::createPinsOnTop

Description

Creates pins on the top.

Syntax

```
lx::createPinsOnTop <design: oaDesign>
```

Arguments

`design`

The physical design.

Returns

`<none>`

lx::createSession

Description

Creates and initializes an SDL session for the given target and source designs.

Syntax

```
lx::createSession -target <deContext> -source <deContext>
```

Arguments

`-target`

The target layout design context.

`-source`

The source design context bound by the physical configuration. This configuration represents the logical source.

Returns

`<lxSession>`: SDL session.

Examples

```
set and2source [de::open [dm::getCellViews layout.config -cellName and2  
-libName DemoPLL]]  
set and2target [de::open [dm::getCellViews layout -cellName and2 -libName  
DemoPLL]]  
lx::createSession -source $and2source \  
-target $and2target
```

lx::dumpMismatchButton

Description

Dumps the mismatch button status.

Syntax

```
lx::dumpMismatchButton -context <deContext> -outputFile <string>
```


Arguments

`-context`

Layout Design Window Context.

`-outputFile`

The output file for the result.

Returns

`<none>`

lx::ecoPostUpdateLayout

Post-update target layout from given lib/cell/view.

Syntax

```
lx::ecoPostUpdateLayout <targetLCV: string> -reference <string> -changes  
[<string>] [-placementCatalogSamples [<string>]] [-traverseHierachy  
<bool>] [-update <lxEcoUpdateLayoutTypeEnum>]
```

Returns

`<bool>` : True of False

Arguments

`targetLCV`

The target Lib/Cell/View to update layout.

`-reference`

The reference Lib/Cell/View to update layout.

`-changes`

The changed instances names.

`-placementCatalogSamples`

The sample point to communicate placement information to SED.

`-traverseHierachy`

To traverse the design hierarchy of the given targetLCV. (Default is: false)

`-update`

Layout update type (all | placementOnly). (Default is: all)

lx::ecoPreUpdateLayout

Pre-update target layout from given lib/cell/view.

Syntax

```
lx::ecoPreUpdateLayout <targetLCV: string> -reference <string> -changes  
[<string>] [-placementCatalogSamples [<string>]] [-traverseHierachy  
<bool>] [-update <lxEcoUpdateLayoutTypeEnum>]
```

Returns

<bool> : True of False

Arguments

targetLCV

The target Lib/Cell/View to update layout.

-reference

The reference Lib/Cell/View to update layout.

-changes

The changed instances names.

-placementCatalogSamples

The sample point to communicate placement information to SED.

-traverseHierachy

To traverse the design hierarchy of the given targetLCV. (Default is: false)

-update

Layout update type (all | placementOnly). (Default is: all)

lx::ecoUpdateLayout

Update target layout from given lib/cell/view.

Syntax

```
lx::ecoUpdateLayout <targetLCV: string> -reference <string> -changes  
[<string>] [-placementCatalogSamples [<string>]] [-traverseHierachy  
<bool>] [-update <lxEcoUpdateLayoutTypeEnum>]
```

Returns

<bool> : True of False

Arguments

`targetLCV`

The target Lib/Cell/View to update layout.

`-reference`

The reference Lib/Cell/View to update layout.

`-changes`

The changed instances names.

`-placementCatalogSamples`

The sample point to communicate placement information to SED.

`-traverseHierarchy`

To traverse the design hierarchy of the given targetLCV. (Default is: false)

`-update`

Layout update type (all | placementOnly). (Default is: all)

lx::establishCorrespondence

Description

Establishes the correspondence.

Syntax

```
lx::establishCorrespondence -design <oaDesign> [-setIgnored <bool>]
```

Arguments

`-design`

Layout design would be analyzed and set correspondence.

`-setIgnored`

Set ignored for the extra instances if all of logical instances are matched.
(Default: false)

Returns

<none>

lx::establishCorrespondenceFromLVS

Description

Establishes the correspondence from the LVS result.

Syntax

```
lx::establishCorrespondenceFromLVS [-skipCellFile <string> | -skipCells  
  [<string>]|-skipLibraries [<string>]]  
-layout <dmCellView | dmCellViewName> [-schematic <dmCellView |  
  dmCellViewName>]  
-path <string> [-hierarchy <bool>] [-outputFile <string>] [-targets  
  <dmCellView | dmCellViewName | {<dmCellView>}>]  
[-setIgnored <bool>] [-fixConnectivity <bool>] [-retarget <bool>]  
[-scalarizeProc  
  <string>] [-splitProc <string>]
```

Arguments

-layout

Cell view of the layout.

-schematic

Cell view of the schematic.

-path

The run directory for LVS.

-hierarchy

Set correspondence for the top design only or layout hierarchy. (Default: false)

-outputFile

The output file for the renaming result. (Default: "")

-skipCellFile

The file contains skip cell list which skips hierarchical verification.

-skipCells

The cells skip hierarchical verification.

-skipLibraries

The libraries skip hierarchical verification.

-targets

The target cell views. (Default: "")

`-setIgnored`

Set ignored for the extra instances if all of logical instances are matched.
(Default: false)

`-fixConnectivity`

Fix connectivity mismatch or not. (Default: true)

`-retarget`

Use retarget flow or not. (Default is: false)

`-scalarizeProc`

Scalarize procedure for retarget. (Default is: `le::scalarizeArrayInst`)

`-splitProc`

Split procedure for retarget. (Default is: `lx::splitDevice`)

Returns

<none>

`lx::execParasiticExtraction`

Description

Executes a parasitic extraction job.

Syntax

```
lx::execParasiticExtraction -rundir <string>
```

Arguments

`-rundir`

Output Run Directory.

Returns

<xtJob> : The executed job object.

`lx::expandSchematic`

Description

Creates and initializes an SDL session from soft macro cells.

Syntax

```
lx::expandSchematic [-ignoreLibs <string> | -ignoreAllExceptSource  
<bool>] -source <dmCellView | oaLib | string> -target <oaLib | string>  
[-copySchematic <bool>] [-excludeLibs [<string>]] [-overwrite <bool>]  
[-symbolMapFile <string>] [-layoutMapFile <string>] [-referenceLayoutLib  
<string>] [-copyOnly <bool>] [-overwriteLayout <bool>] [-sourceVSL  
[<string>]] [-logFile <string>]
```

Arguments

-source

Source to be expanded, Schematic or Config cellview or Library.

-target

Destination Library for soft-macro cells.

-copySchematic

Copy schematic from source to target (Default: false).

-excludeLibs

Specifies libraries whose cellviews should not be copied.

-overwrite

Action to take when a copied cell already exists in the destination. (Default: false)

-symbolMapFile

Specifies the symbol mapping file.

-layoutMapFile

Specifies the layout mapping file.

-referenceLayoutLib

Layout library from which layout cellviews are copied into target.

-copyOnly

Copies the schematic without creating soft macros. (Default: false)

-ignoreAllExceptSource

Ignore all libraries except source schematic library. (Default: false)

-overwriteLayout

Action to take when a reference layout already exists in the destination. (Default: false)

`-sourceVSL`

Specifies View Search List to use while elaborating the schematic.

`-logFile`

Specifies log file.

lx::exportSdlNetlist

Export SDL Netlist

Syntax

```
lx::exportSdlNetlist [-src <dmCellView | dmCellViewName>] -dst  
  <dmCellView | dmCellViewName> [-schematic <dmCellView | dmCellViewName>]  
  [-overwrite <bool>]
```

Returns

<none>

Arguments

`-src`

Cell view of source layout view

`-dst`

Cell view of destination netlist view

`-schematic`

Cell view of source schematic view

`-overwrite`

Overwrite existing destination netlist view (Default is: false)

lx::extractFeatures

Extract the features from schematic and layout

Syntax

```
lx::extractFeatures [-layout <dmCellView | dmCellViewName>] -schematic  
  <dmCellView | dmCellViewName> [-directory <string>] [-mode <enum>]  
  [-placeProc <string>] [-routeProc <string>] [-save <bool>] [-nets  
  [<string>]]
```

Returns

<none>

Arguments

-layout

Cell view of source layout view

-schematic

Cell view of source schematic view

-directory

Working directory to save data and model

-mode

Extract data for train or predict ({ predict | train }, default is: train)

-placeProc

Procedure for placement

-routeProc

Procedure for route

-save

Save as real layout in same library (Default is: false)

-nets

The nets selected to extract features.

lx::extractParasiticTrainingData

Internal extract data for Parasitic Estimation training.

Syntax

```
lx::extractParasiticTrainingData [-fileList <string>] -MLLibrary  
<string> -process <string> -data <string> -schematic <dmCellView |  
dmCellViewName> [-layout <dmCellView | dmCellViewName>] [-spf <string>]
```


Returns

<string> :

Arguments

-fileList

File list of schematic, layout and SPF.

-MLLibrary

Library for working directory

-process

Cell for working directory

-data

View for working directory

-schematic

Schematic LCV

-layout

Layout LCV

-spf

SPF file path

lx::extractTemplate

Extract template from given library

Syntax

```
lx::extractTemplate -fromLib <string> -templateLib <string>
```

Returns

<none>

Arguments

-fromLib

Library to extract template

-templateLib

Library to create template

lx::findCloneCandidates

Description

Returns clones for the given layout objects. This command is `lxCloneFrom` dependent.

Syntax

```
lx::findCloneCandidates <objects: {<deFigure>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} | {<oaVia>}}>  
[-design <oaDesign>]
```

Returns

<dbCollection>: Collection of objects from a layout design representing the layout source clone.

Arguments

objects

Collection of layout objects representing the layout source clone.

-design

Design in which to search for clone candidates.

Default: Design that the input objects are from.

Examples

```
set selected [de::getSelected -design [ed] ]  
set matches [lx::findCloneCandidates $selected]
```

Returns clones for the given layout objects. This command is `lxCloneFrom` dependent.

lx::generateEdtext

Description

Generates Edtext.

Syntax

```
lx::generateEdtext -cellView <dmCellView | dmCellViewName>  
[-textLPPMapFile <string>] -oagdsLayerMapFile <string> -outputFile  
<string>
```

Arguments

-cellView

The physical cellview.

`-textLPPMapFile`

The text LPP map file. If it is not specified, this command will use the technology library.

`-oagdsLayerMapFile`

The layer map file which records the map between OA LPPs and GDS LPPs.

`-outputFile`

The output file.

Returns

<none>

lx::generatePlacementCatalog

Description

Creates placement catalog for Design Centering.

Syntax

```
lx::generatePlacementCatalog -window <int> -group [<string>] -pattern <string> [-yaml <string>] [-wMin <int>] [-wMax <int>] [-lMin <int>] [-lMax <int>] [-wbylMin <int>] [-wbylMax <int>] [-wbylIgnore <bool>]
```

Arguments

`-window`

Specifies the SED window(id) with group for which patterns are to be created.

`-group`

Specifies the group of devices being optimized.

`-pattern`

Specifies the pattern to place device in SED matching.

`-yaml`

Specifies the name of the YAML file.

`-wMin`

Specifies the minimum width W as a % of current width of a device.

`-wMax`

Specifies the maximum width W as a % of current width of a device.

`-lMin`

Specifies the minimum length L as a % of current length of a device.

`-lMax`

Specifies the maximum length L as a % of current length of a device.

`-wbylMin`

Specifies the minimum width to length ration W/L as a % of current ratio of a device.

`-wbylMax`

Specifies the maximum width to length ration W/L as a % of current ratio of a device.

`-wbylIgnore`

Ignores W/L checks during options generation.

Returns

<none>

lx::getAssistantCanvas

Description

Gets the template schematic pointer from the template layout or template schematic.

Syntax

```
lx::getAssistantCanvas -from <string>
```

Returns

<dbCore | string> : Object or attribute value with the specified name, or "" if it could not be found.

Arguments

`-from`

Gets the pointer from the template layout or schematic.

lx::getAssociatedObject

Description

Gets the associated objects (from schematic to layout and from layout to schematic) when a database object is provided.

Syntax

```
lx::getAssociatedObject <object: deContext | oaDesign | oaInst | oaNet |  
oaTerm>
```

Returns

deContext | oaDesign | {<deFigure>} | {<oaNet>} | {<oaTerm>}: Associated objects. Fails if the logical correspondence is not from the top design.

Arguments

object

The context, design, instance, net, or term to find associated objects for.

<oaDesign | <oaFig> | {<oaNet> | <oaTerm>>

The object for which associated objects must be found.

<oaDesign | {<oaFig>} | {<oaNet>} | {<oaTerm>}>

The associated objects of the specified object.

Errors

- The specified database object is not a valid object.

Examples

```
set design [db::getAttr editDesign -of [de::getActiveContext]]  
set sDesign [lx::getAssociatedObject $design]
```

Gets the associated objects (from schematic to layout and from layout to schematic) when a database object is provided.

lx::getAssociatedObjectByName

Description

Gets the associated objects (from schematic to layout and from layout to schematic) when the name, type, and source for a database object are provided. For example, given a net name in a layout design, this command will return the corresponding net objects in the schematic.

Syntax

```
lx::getAssociatedObjectByName <object: deContext | oaDesign | oaInst |  
oaNet | oaTerm>
```

Returns

deContext | oaDesign | {<deFigure>} | {<oaNet>} | {<oaTerm>} : Associated objects.

Arguments

`object`

Context, design, instance, net, or term to find associated objects for.

Errors

- The specified database object is not a valid object.

lx::getBrowserCanvas

Description

Gets the template schematic pointer from the template layout or template schematic.

Syntax

```
lx::getBrowserCanvas -from <string>
```

Returns

`<dbCore | string>` : Object or attribute value with specified name or "" if it could not be found.

Arguments

`-from`

Specifies whether to get the pointer from the layout or schematic template.

lx::getDesignFromSED

Description

Returns the requested Design ID.

Syntax

```
lx::getDesignFromSED -window <giWindow | int> [-type <enum>]
```

Returns

`<oaDesign>` : OA Design Id

Arguments

`-window`

The window (ID) of the Symbolic Editor window to in which to invoke the `getDesign` command

`-type`

Type of design used in SED session ({ preview | sdlLayout | symbolic }, default is: sdlLayout)

lx::getDeviceInfo

Description

Displays the SED device information.

Syntax

```
lx::getDeviceInfo [-window <giWindow | int>]
```

Arguments

`-window`

Specifies the window (ID) of the symbolic editor window in which to invoke the show device information command.

lx::getFoldingParam

Description

Gets the folding parameter name used by SDL for a component. This command is for MOS superMaster only, and defines the `finger` parameter.

Syntax

```
lx::getFoldingParam -libName <string> -cellName <string>
```

Returns

Folding parameter for a component.

Arguments

`<string>`

The folding parameter name.

`-libName`

The library name of the component.

`-cellName`

The cell name of the component.

Errors

The specified component does not exist.

Examples

```
lx::getFoldingParam -libName reference90RF -cellName n_4t
```

In this example, `lx::getFoldingParam` gets the folding parameter name for a component. A PDK library name and a cell is provided in this example.

lx::getFoldingThreshold

Description

Gets the threshold value used by SDL.

Syntax

```
lx::getFoldingThreshold -libName <string> -cellName <string>
```

Returns

Folding threshold for a component.

Arguments

<string>

Folding threshold value.

-libName

The library name of the component.

-cellName

The cell name of the component.

Errors

- The specified component does not exist.

Examples

```
lx::getFoldingThreshold -cellName n_4t -libName reference90RF
```

In this example, `lx::getFoldingThreshold` gets the default folding threshold for the given PCell device from the PDK library.

lx::getMismatchedConstraints

Description

Finds and returns a list of constraint pairs. The constraint pair consists of one constraint from the front end and another from the layout that are equivalent but have different values.

Syntax

```
lx::getMismatchedConstraints <session: lxSession>
```

Arguments

`session`

SDL session.

Returns

<dbCollection>: Collection of cmConstraint objects.

lx::getPlacement

Description

Dumps the placement in MDC.

Syntax

```
lx::getPlacement [-window <giWindow | int>] [-mode <enum>] [-single  
  <bool>]
```

Arguments

`-window`

The window (ID) of the Symbolic Editor window in which to invoke get placement command

`-mode`

Shows the orientation of the device ({ detail | normal }).

`-single`

Shows the placement string as a single line.

lx::getPseudoParallelNets

Description

Gets the pseudo parallel nets from the schematic design and layout.

Syntax

```
lx::getPseudoParallelNets <design: oaDesign>
```

Arguments

design

Layout design would be analyzed and set correspondence.

Returns

<none>

lx::getSchematicDummies

Description

Gets schematic dummy instances.

Syntax

```
lx::getSchematicDummies -context <deContext> [-mosfetOnly <bool>]  
[-includeHalfDummy <bool>] [-includeStackMos <bool>]
```

Arguments

-context

Target context to get schematic dummies.

-mosfetOnly

Only gets schematic dummy for MOSFET. (Default is: false)

-includeHalfDummy

Gets the schematic half dummy for MOSFET. (Default is: true)

-includeStackMos

Gets the schematic dummy for stacking MOS. (Default is: false)

Returns

{<oaFig>} : Collection of schematic dummies.

lx::getSessions

Description

Gets the SDL sessions.

Syntax

```
lx::getSessions [-window <giWindow | int>] [-filter <string>]
```

Returns

{<lxSession>}: Sessions matching the search criteria.

Arguments

-window

Window to get the SDL session from.

-filter

Standard attribute filter.

Examples

```
lx::getSessions -window [db::getattr context.window]
```

This commands gets the SDL sessions.

lx::highlightFlightline

Description

Assigns a highlight on the flight line.

Syntax

```
lx::highlightFlightline -window <int> -net <string> -colorIndex <int>
```

Arguments

-window

Window for assigning highlight.

-net

Net name of flight lines to assign highlight.

-colorIndex

Color index of highlight color.

lx::incrementalRealizeConstraint

incremental realize constraint

Syntax

```
lx::incrementalRealizeConstraint [-context <deContext>] [-constraint  
    <string>] [-dirtyInsts [<string>]]
```

Returns

{<oaFig>} : Collection of created top fig groups

Arguments

-context

Context of activation.

-constraint

Constraint names to be realized. (default: the only top constraint)

-dirtyInsts

Dirty instances need to be re-realized.

lx::importSDLMappingFile

Description

Imports the SDL Mapping File.

Syntax

```
lx::importSDLMappingFile -lib <oaLib | string> -filePath [<string>]  
    [-mode <enum>]
```

Arguments

-lib

Destination library for the SDL mapping file.

-filePath

Specifies the SDL mapping file.

-mode

Import mode ({ append | replace }). (Default: replace.)

Returns

<none>

lx::layoutHierarchyChangeECOUpdate

Description

Updates the overlay schematic with any parameter changes.

Syntax

```
lx::layoutHierarchyChangeECOUpdate -context <deContext>
```

Arguments

-context

SDL Design Window Context.

Returns

<none>

lx::loadAddShapes

Description

Load the Add Shape Settings from the Section.

Syntax

```
lx::loadAddShapes [-window <giWindow | int>] -set <string>
```

Arguments

-window

Window(id) from which to load Add Shape Settings.

-set

Set Name to load the Add Shape Settings.

Returns

<none>

lx::loadAddShapeSettings

Description

Loads the Add Shape settings from an external file.

Syntax

```
lx::loadAddShapeSettings [-window <giWindow | int>] -set <string>
```

Arguments

-window

Specifies the Window Id to load Add Shapes.

-set

Specifies the load settings with a distinct name.

Returns

<none>

lx::loadCellTemplate

Description

Loads the cell template from the section.

Syntax

```
lx::loadCellTemplate [-window <giWindow | int>] -set <string>
```

Arguments

-window

The window (ID) from which to load the Cell Template.

-set

Set name to load the Cell Template Settings.

Returns

<none>

lx::parasiticBackAnnotate

Description

Parasitic back annotate.

Syntax

```
lx::parasiticBackAnnotate -cellView <dmCellView | dmCellViewName>  
-refFile <string> [-format <enum>] [-case <enum>] [-topCell <string>]  
-dspfFile <string> -netlistFile <string> -outputFile <string>
```

Arguments

-cellView

The physical cellview.

-refFile

The reference file.

-format

The reference netlist format, either CDL or HSPICE. The default is HSPICE.

-case

Specifies the case-sensitive option for the reference netlist. Valid values are Lower, Sensitive, and Upper. The default is Lower.

-topCell

The reference file.

-dspfFile

The reference DSPF(.SPF) file.

-netlistFile

The extracted netFile.

-outputFile

The output netlist file.

lx::plePostProcess

Description

Parasitic layout extraction post processing.

Syntax

```
lx::plePostProcess -cellView <dmCellView | dmCellViewName> -spfFile  
  <string> -flattenSpiceFile <string> [-estimatedParasiticFile <string>]  
  [-ignoreParasitic <bool>] [-checkBulkConnection <bool>] -outputFile  
  <string> [-trimParasiticsNets [<string>]]
```

Arguments

-cellView

The physical cellview.

-spfFile

The SPF file which include parasitic information.

-flattenSpiceFile

The file is one flatten spice file.

-estimatedParasiticFile

Estimated parasitic file. (Default:)

-ignoreParasitic

The flag to whether to ignore parasitics. (Default: false)

-checkBulkConnection

The flag to whether to check bulk connection. (Default: true)

-outputFile

The output file.

-trimParasiticsNets

The nets to trim parasitics.

Returns

<none>

lx::predictParasitic

Calculate Cg by Machine Learning Model.

Syntax

```
lx::predictParasitic -schematic <dmCellView | dmCellViewName> [-layout  
  <dmCellView | dmCellViewName>] -MLLibrary <string> -process <string>  
  -model <string> -nets <string> -outputFile <string> [-save <bool>]  
  [-mode <enModeType>]
```


Returns

<string> : Cg predicted value

Arguments

-schematic

Schematic LCV

-layout

Layout LCV

-MLLibrary

Library for working directory

-process

Cell for working directory

-model

Choose model

-nets

Net to be used for predicting C value

-outputFile

Complete file path to save RC prediction. (Ex. "/users/result.csv")

-save

Save the feature csv and config file or not. (Default is: false)

-mode

Different modes (Default is: RandomForest)

lx::preview

Description

Switches to the preview mode, and displays the created layout instances for the device MOS.

Syntax

```
lx::preview [-window <giWindow | int>] [-assistant <bool>] [-refresh  
  <bool>]
```

Arguments

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the preview command.

`-Assistant`

Updates the preview assistant of the Symbolic Editor window when the assistant is invoked.

`-refresh`

Refreshes the preview command.

Returns

<none>

Errors

- An invalid symbolic window is passed.
- No design is opened in the specified window.

lx::realize

Description

Creates layout objects and pastes them in the Layout Editor.

Syntax

```
lx::realize [-window <giWindow | int>]
```

Arguments

`-window`

The window (ID) of the Symbolic Editor window in which to invoke realize command.

Errors

- An invalid symbolic window object is passed.
- No design is opened in the specified window.

lx::realizeConstraint

Description

Realizes the constraint.

Syntax

```
lx::realizeConstraint [-context <deContext>] [-constraint <string>]  
[-point <point>] [-force <bool>]
```

Arguments

-context

Context of activation.

-constraint

Constraint names to be realized. (Default: the only top constraint)

-point

Realizes the constraint to the specific point. (Default: {0 0})

-force

Force to realize the constraint even the placement group is already realized in target design. (Default: true)

Returns

{<oaFig>} : Collection of created target fig group.

lx::realizeTemplate

Description

Realizes the template view.

Syntax

```
lx::realizeTemplate -context <deContext> -libName <string> -cellName  
<string> -viewName <string> [-point <point>] [-skipCells [<string>]]
```

Arguments

-context

Target context to realize template.

-libName

Template library name.

`-cellName`

Template cell name.

`-viewName`

Template view name.

`-point`

Realizes the template to specific point. The default is (0, 0).

`-skipCells`

Cell names to be skipped.

Returns

{<oaFig>} : Collection of created top fig groups.

lx::rechain

Description

Re-chains devices.

Syntax

```
lx::rechain -window <int>
```

Arguments

`-window`

Window in which to re-chain devices.

Returns

<none>

lx::registerLayoutTemplate

Description

Registers a custom layout template. If a custom layout template with the same name was already registered, the new custom layout template replaces it.

Syntax

```
lx::registerLayoutTemplate <name: string> -title <string>  
[-optionSetType <string>] [-defaultOptionSetPref <string>] [-commandType  
<enum>]
```

Arguments

`<string>`

Name of the custom layout template Tcl procedure.

`-name <string>`

Title of the custom layout template.

`-optionSetType <string>`

Name for the type option set supported by this custom layout template.

`-defaultOptionSetPref <string>`

Name of the preference which holds the default option set name.

`-commandType <enum>`

Valid command type for the layout template ({all | generateLayout | pickAndPlace}).

Returns

`<none>`

lx::reloadSymbolicEditor

Description

Reloads devices from a constraint.

Syntax

```
lx::reloadSymbolicEditor -window <int>
```

Arguments

`-window`

Window in which to reload devices.

Returns

`<none>`

lx::removePeriphery

Remove periphery

Syntax

```
lx::removePeriphery -window <giWindow | int>
```

Returns

<none>

Arguments

-window

Window(id) of symbolic editor window to invoke remove periphery command

lx::reopenLogicalSource

Description

Reopens the logical source in the context of the SDL configuration.

Syntax

```
lx::reopenLogicalSource [-context <deContext>]
```

Returns

<deContext> - the source context.

Arguments

-context

Layout Design Window Context

Errors

- Window does not exist.

Examples

Example 1

```
lx::reopenLogicalSource -parent 2
```

This command reopens the logical source (schematic) of the already initialized SDL session for given layout window.

Example 2

```
set context [de::getActiveContext]
set leWindow [db::getattr context.window]
set seContext [lx::reopenLogicalSource -parent $leWindow]
```

This command reopens the logical source in the context of the SDL configuration.

lx::replacePlacement

Description

Replaces placement for a group.

Syntax

```
lx::replacePlacement [-window <int>] [-verbosity <int>] -catalogFilename <string> -optVarName <string> -placementIndex <int>
```

Arguments

-window

Specifies the Window(id) for an SDL session.

-verbosity

Specifies the level of debug messages (Default is: 0).

-catalogFilename

Specifies the name of the placement catalog file.

-optVarname

Specifies the name of the optimization variable in the placement catalog file.

-placementIndex

Specifies the index of `optVar` placement to be used.

Returns

<none>

lx::resetLayoutNets

Reset layout of nets

Syntax

```
lx::resetLayoutNets -design <oaDesign>
```

Returns

<none>

Arguments

-design

Target edit layout design

lx::saveAddShapes

Description

Save the Add Shape Settings from the Section.

Syntax

```
lx::saveAddShapes [-window <giWindow | int>] -set <string>
```

Arguments

`-window`

Window(id) from which to save Add Shape Settings.

`-set`

Set Name to save the Add Shape Settings.

Returns

<none>

lx::saveAddShapeSettings

Description

Saves the Add Shape settings to an external file.

Syntax

```
lx::saveAddShapeSettings [-window <giWindow | int>] -set <string>
```

Arguments

`-window`

Specifies the Window Id to save Add Shapes.

`-set`

Sets the name to save Add Shape settings.

Returns

<none>

lx::saveCellTemplate

Description

Loads the cell template from the section.

Syntax

```
lx::saveCellTemplate [-window <giWindow | int>] -set <string>
```

Arguments

-window

The window (ID) from which to save the cell template.

-set

Set name to load the cell template settings.

lx::saveConstraint

Description

Saves the placement information as the Symbolic Editor constraint settings.

Syntax

```
lx::saveConstraint -window <int>
```

Arguments

-window

The window (ID) of parent window from which to save constraint settings.

lx::saveTemplateConstraint

Description

Saves a template constraint.

Syntax

```
lx::saveTemplateConstraint -dnDeviceToNetlist [<string>] -dnNetToTemplate  
[<string>] -lib <string> -cell <string> -view <string> [-name <string>]  
[-mode <string>]
```

Arguments

`-dnDeviceToNetlist`

A list to specifies the mapping between DN-device and template netlist.

`-dnNetToTemplate`

A list to specifies the net mapping between DN and template.

`-lib`

Template library.

`-cell`

Template cell.

`-view`

Template view.

`-name`

Template name.

`-mode`

Sets the mode. Valid values are `template_manager` and `template_browser`.

lx::sedCreateSession

Description

Creates an SED session.

Syntax

```
lx::sedCreateSession [-window <giWindow | int>] [-mode <enum>] -inst  
[<string>] [-pattern <string>]
```

Arguments

`-window`

The layout The window (ID) in which to create an SED session.

`-mode`

The mode of the Symbolic Editor ({ chaining | matching }).

`-inst`

The device list added into Symbolic Editor.

`-pattern`

The pattern to place device in SED matching.

Returns

`<IxSymbolicEditorSession>` : symbolic editor session

`Ix::sedDestroySession`

Description

Deletes an SED session.

Syntax

```
lx::sedDestroySession -window <giWindow | int>
```

Arguments

`-window`

The layout window (ID) to delete an SED session.

`Ix::sedGetDesign`

Description

Returns the requested Design ID.

Syntax

```
lx::sedGetDesign -window <giWindow | int> [-type <enum>]
```

Returns

`<oaDesign>` : OA Design Id

Arguments

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the `getDesign` command.

`-type`

The type of design used in an SED session (`{ preview | sdLayout | symbolic }`, default is: `sdLayout`).

lx::sedGetGridSize

Description

Fetches the number of rows and columns in the SED grid.

Syntax

```
lx::sedGetGridSize -window <giWindow | int>
```

Returns

<string> : Number of rows and columns.

Arguments

-window

The layout window (ID) to fetch the number of rows and columns in the SED grid.

lx::sedSetParam

Description

Sets the device parameters in an SED session.

Syntax

```
lx::sedSetParam -session <lxSymbolicEditorSession> -devices [<string>]  
-parameters [<map>]
```

Arguments

-session

The SED session for the devices.

-devices

The device list whose parameters are to be set.

-parameters

The parameters to be set on the device.

Returns

<none>

lx::sedSetPreset

Description

Sets the device presets in an SED session.

Syntax

```
lx::sedSetPreset -session <lxSymbolicEditorSession> -devices [<string>]  
-preset <dmCellView | dmCellViewName>
```

Arguments

-session

SED session for the devices.

-devices

The device list whose preset is to be set.

-preset

The cellview to be used as preset.

Returns

<none>

lx::setPlacement

Description

Sets the placement in MDC.

Syntax

```
lx::setPlacement [-window <giWindow | int>] -pattern <string> [-column  
<int>] [-repeatX <int>] [-repeatY <int>]
```

Arguments

-window

The window (ID) of the Symbolic Editor window in which to invoke the set placement command.

-pattern

Sets the input pattern.

-column

Sets the column number.

`-repeatX`

Sets the repeat in X.

`-repeatY`

Sets the repeat in Y.

lx::setupSDL

Description

A command to setup SDL.

Syntax

```
lx::setupSDL [-layout <dmCellView | dmCellViewName>] [-config <dmCellView  
| dmCellViewName>] [-schematic <dmCellView | dmCellViewName>]
```

Arguments

`-layout`

Cell view of the layout.

`-config`

Cell view of the layout.config.

`-schematic`

Cell view of the schematic.

Returns

<none>

lx::showAddShapeSettingsDialog

Description

Open the **Add Shape Settings** dialog box.

Syntax

```
lx::showAddShapeSettingsDialog [-parent <giWindow | int>]
```

Arguments

`-parent`

Window Id to initialize **Add Shapes**.

Returns

<giDialog> : The Dialog.

lx::showAllNet

Description

Shows all flight line connections.

Syntax

```
lx::showAllNet -window <int>
```

Arguments

-window

Window in which to show all flight lines.

lx::showBatchCreateTemplate

Description

Shows the **Batch Create Template** dialog box.

Syntax

```
lx::showBatchCreateTemplate
```

lx::showCellTemplate

Description

Opens the **Cell Template** dialog box.

Syntax

```
lx::showCellTemplate [-parent <giWindow | int>]
```

Returns

<giDialog> : The dialog box.

Arguments

-parent

The window (ID) in which to initialize the cell template.

lx::showCheckerDialog

Description

Open the **Check Errors** dialog box.

Syntax

```
lx::showCheckerDialog [-parent <giWindow | int>]
```

Arguments

`-parent`

Window(id) from which to initialize the **Check Errors** dialog box.

Returns

<giDialog> : The dialog box.

lx::showCheckOptions

Description

Displays the **ECO Setup Options** dialog box.

Syntax

```
<giDialog> lx::showVerifyOptions [-parent <giWindow | int>]
```

Returns

Dialog ID.

Arguments

`parent <giWindow | int>`

The window (ID) of parent window in which to initialize the ECO Setup preferences.

Default: Active window.

Errors

- Window does not exist.

lx::showCreateTemplate

Description

Shows the **Template Creator** dialog box. If the dialog box is already opened, it will be raised and focused.

Syntax

```
lx::showCreateTemplate
```

lx::showDebugMessage

Description

Shows SDL debug messages.

Syntax

```
lx::showDebugMessage [-startToShowMessage <bool>] [-showAllDebugMessages  
<bool>] [-countMemoryNewAndDeleteAtGlobal <bool>] [-showCountedMemory  
<bool>] [-showCallStack <bool>] [-countCallTime <bool>] [-toFile  
<bool>] [-showLogging <bool>] [-appendMode <bool>] [-toCCConsole <bool>]  
[-classAndFunc <string>] [-class <string>] [-funcInClasses <string>]  
[-file <string>] [-function <string>] [-loggingFile <string>] [-delete  
[<bool>]]
```

Returns

<none>

Arguments

-startToShowMessage

Starts showing debug messages.

-showAllDebugMessages

Shows all debug messages.

-countMemoryNewAndDeleteAtGlobal

Counts memory new and deletes them at global.

-showCountedMemory

Shows counted memory.

-showCallStack

Dumps call stack.

`-countCallTime`
Counts function call time.

`-toFile`
Decides whether logging to file (default: `cc_rd_debug.log`).

`-showLogging`
Shows current logging status.

`-appendMode`
Decides to append to file end.

`-toCCConsole`
Decides whether to print to the Console.

`-classAndFunc`
Shows debugging message in `Class::Func`.

`-class`
Shows debugging message in `Class::*`.

`-funcInClasses`
Shows debugging message in `*::Func`.

`-file`
Shows debugging message in file.

`-function`
Shows debugging message in function.

`-loggingFile`
Saves logging message to file (default: `cc_rd_debug.log`).

`-delete`
Deletes flag. Deletes debugging messages.

ix::showDefineSource

Description

Displays the **Define Logical Source** dialog box for the user to specify the source schematic design and corresponding physical configuration to use when initiating an SDL session from a layout design window.

Syntax

```
<giDialog> lx::showDefineSource [-parent <giWindow | int>]
```

Returns

Dialog window.

Arguments

```
parent <giWindow | int>
```

Window in which to start the interactive command process.

Default: Active window.

Errors

- The specified window is not a layout editor window.
- The specified layout editor window is not opened for edit.

lx::showDefineTarget

Description

Displays the **Define Physical Target** dialog box for the user to specify the target layout and physical configuration to use when initiating an SDL session from a schematic design window.

Syntax

```
<giDialog> lx::showDefineTarget [-parent <giWindow | int>]
```

Returns

Dialog window.

Arguments

```
parent <giWindow | int>
```

Window in which to start the interactive command process.

Default: Active window.

Errors

The specified window is not a Schematic Editor window.

lx::showExpandSchematic

Description

Invokes the **Example Schematic** dialog box.

Syntax

```
<giDialog> lx::showExpandSchematic [-parent <giWindow | int>]
```

Returns

Dialog window.

Arguments

parent <giWindow | int>

Window in which to start the interactive command process.

Default: Active window.

Errors

- An invalid window ID is passed.

lx::showGateConnected

Description

Shows all flight lines connected to the gate.

Syntax

```
lx::showGateConnected -window <int>
```

Arguments

-window

The window in which to show gate-to-all flight lines.

lx::showGateNet

Description

Shows the Gate-to-Gate flight lines.

Syntax

```
lx::showGateNet -window <int>
```

Arguments

`-window`

The window in which to show for gate-to-gate flight lines.

lx::showGenerateLayout

Description

Invokes the **Generate Layout** dialog box.

Syntax

```
<giDialog> lx::showGenerateLayout [-parent <giWindow | int>]
```

Returns

Dialog window.

Arguments

`parent <giWindow | int>`

Window in which to start the interactive command process.

Default: Active window.

Errors

- The SDL session has not been initialized.
- An invalid window ID is passed.

Examples

```
lx::showGenerateLayout -parent [db::getattr window -of  
[de::getActiveContext]]
```

lx::showImportLayout

Description

Invokes the **Import Layout** dialog box.

Syntax

```
<giDialog> lx::showImportLayout [-parent <giWindow | int>]
```

Returns

Dialog window.

Arguments

`parent <giWindow | int>`

Window in which to start the interactive command process.

Default: Active window.

lx::showLHCGenerateInstanceMap

Description

Opens the **SDL Layout Hierarchy Change Generate Instance Map** dialog box.

Syntax

```
lx::showLHCGenerateInstanceMap [-parent <giWindow | int>]
```

Returns

`<giDialog>` : The Dialog

Arguments

`-parent`

Window(id) of SDL window to generate instance map for.

lx::showLHCGenerateSchematic

Description

Opens the **SDL Layout Hierarchy Change Generate Schematic** dialog box.

Syntax

```
lx::showLHCGenerateSchematic [-parent <giWindow | int>]
```

Returns

`<giDialog>` : The Dialog

Arguments

`-parent`

Window(id) of SDL window to generate schematic in.

lx::showODNet

Description

Shows S/D-to-S/D flight lines.

Syntax

```
lx::showODNet -window <int>
```

Arguments

-window

Window for showing S/D-to-S/D flight lines.

lx::showOptions

Description

Displays the **SDL Options** dialog box. The dialog box allows editing of general SDL preference values.

Syntax

```
lx::showOptions [-parent <giWindow | int>]
```

Returns

Dialog window.

Arguments

parent <giWindow | int>

The window (ID) of parent window from which to initialize SDL options.

Default: Active window.

lx::showParasiticEstimationByMLDialog

Description

Shows ML model training options for parasitic estimation.

Syntax

```
lx::showParasiticEstimationByMLDialog
```

Arguments

None

Returns

<giDialog> : Created dialog.

lx::showPresetSetup

Description

Displays the **Preset Setup** dialog box.

Syntax

```
lx::showPresetSetup
```

Returns

The dialog box.

lx::showSDLCheckAndRepairWindow

Description

Shows SDL check and repair window.

Syntax

```
lx::showSDLCheckAndRepairWindow
```

Arguments

None

Returns

<none>

lx::showSDLMapping

Description

Opens the dialog box for SDL mapping.

Syntax

```
lx::showSDLMapping [-parent <giWindow | int>]
```


Returns

<giDialog> : The dialog box.

Arguments

`-parent`

The window (ID) of parent window from which to initialize the dialog.

lx::showSymbolicDesignOptions

Description

Opens the **Symbolic Design Options** dialog box.

Syntax

```
lx::showSymbolicDesignOptions [-parent <giWindow | int>]
```

Returns

<giDialog> : The dialog box.

Arguments

`-parent`

The window (ID) from which to initialize Symbolic Design Options.

Errors

- An invalid symbolic window object is passed.
- Opens dialog in preview mode.

lx::showUpdateSchematic

Description

Opens the **SDL Update Schematic from Layout** dialog box.

Syntax

```
lx::showUpdateSchematic [-parent <giWindow | int>]
```

Returns

<giDialog> : The Dialog

Arguments

`-parent`

Window(id) of layout editor window in which to update schematic from layout.

lx::splitDevice

Description

Splits the device by setting the split parameter of original device to 1 and creating a number of copies equal with the original value of the split parameter minus 1 for each instance processed.

Syntax

```
lx::splitDevice <objects: {<deFigure>} | {<deSelectionSet>} | {<oaFig>} |  
{<oaFigGroup>} | {<oaInst>}>  
[-externalConnection <enum>][-pattern <string>]
```

Returns

Collection of newly created instances after splitting. ({oaInst}).

Arguments

`-objects`

The objects to split.

`-externalConnection`

Specify to create abutment external connection. ({ auto | disable | enable },
default is: auto)

`-pattern`

Specifies the option to specify number of fingers for the newly created split
instance. (Default is: 1)

Returns

<oaInst>: Collection of newly created instances after the split.

lx::symbolic

Description

Returns to symbolic view from preview view.

Syntax

```
lx::symbolic [-window <giWindow | int>]
```

Arguments

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

lx::templateDesigner

Description

Shows the **Discovery Template** dialog box. If the dialog box is already opened, it will be raised and focused.

Syntax

```
lx::templateDesigner
```

lx::testSaveTemplateConstraint

Description

Saves the template constraint.

Syntax

```
lx::testSaveTemplateConstraint -mode <string>
```

Arguments

`-mode`

Test mode.

lx::unregisterLayoutTemplate

Description

Unregisters a custom layout template of a particular name from the session.

Syntax

```
lx::unregisterLayoutTemplate <name: string>
```

Arguments

<string>

Name of the custom layout template Tcl procedure.

Errors

- The custom layout template name provided is invalid.

lx::updateConstraints

Description

This command transfers new and/or updated existing logic constraints from front end to the layout. The list of newly transferred and/or updated constraints in the layout is returned.

Syntax

```
{<cmConstraint>} lx::updateConstraints <lxSession >  
[-mode <enConstraintUpdateMode>]
```

Returns

List of constraints in the layout that are newly transferred or updated.

Arguments

mode

Mode for constraint update.

Errors

- 1. The layout design associated with the sdl session is not open in write mode.

lx::updateConstraintBySDL

Update current constraints by SDL information.

Syntax

```
lx::updateConstraintBySDL [-context <deContext>]
```

Returns

<none>

Arguments

-context

Context of activation.

lx::updateInstanceCache

Description

Updates the instance cache used in an SDL session.

Syntax

```
lx::updateInstanceCache [-window <int>] [-action <enum>]
```

Arguments

-window

Specifies the window ID for an SDL session.

-action

Specifies the instance cache action ({ clear | disable | enable | print }, default is: print).

Returns

<none>

lx::updateLayout

Description

This command runs the SDL autoFix routine in the SDL session associated with a layout editor context.

Syntax

```
lx::updateLayout (-objects <{<deFigure>} | {<oaFig>} | {<oaFigGroup>}  
| {<oaInst>} | {<oaNet>} | {<oaShape>} | {<oaVia>}}> | -design <oaDesign>)
```

Arguments

objects

List of layout objects to be fixed.

-design

Layout design to be fixed.

Errors

- The SDL session has not been initialized.
- The window passed does not have SDL tool active.

Examples

Example 1

```
lx::updateLayout 2
```

This command calls ECO Auto Fix for the layout window.

Example 2

```
set figs [de::getSelected -design [db::getattr editDesign -of  
[de::getActiveContext]]]  
lx::updateLayout $figs
```

This command runs the SDL autoFix routine in the SDL session associated with a layout editor context.

lx::verify

Description

This command runs the SDL comparator for the SDL session associated with a layout editor context.

Syntax

```
lx::verify [-skipCellFile <string> | -skipCells [<string>] |  
-skipLibraries [<string>]] [-context <deContext>] [-cellView <dmCellView  
| dmCellViewName>] [-verifyMode <enum>] [-createMarker <bool>]  
[-hierarchy <bool>] [-outputFile <string>]
```

Arguments

-context

Layout design window context.

-cellView

The physical cellview.

-verifyMode

The option to specify which mode to verify. Valid values: PLE, SDL, VDRC
(Default is: SDL)

`-createMarker`

Whether to export violations as OA markers. (Default: true)

`-hierarchy`

The flag whether to verify designs through the hierarchy. (Default: false)

`-outputFile`

The output file. (Default: "")

`-skipCellFile`

The file contains skip cell list which skips hierarchical verification.

`-skipCells`

The cells skip hierarchical verification.

`-skipLibraries`

The libraries skip hierarchical verification.

Returns

{<oaFig>}, collection of markers.

Errors

- The SDL session has not been initialized.
- The window passed does not have SDL active.

Examples

Example 1

```
lx::verify
```

Command does verification between layout and schematic for initialized SDL session.

Example 2

```
lx::verify -context [de::getActiveContext]
```

This command runs the SDL comparator for the SDL session associated with a layout editor context.

lx::verifyHierarchy

Description

Verify designs through the hierarchy.

Syntax

```
lx::verifyHierarchy -cellView <dmCellView | dmCellViewName> [-hierarchy  
<bool>] [-outputFile <string>] [-skipCells [<string>]] [-exportMarkers  
<bool>] [-verifyMode <enum>]
```

Returns

<bool> : Returns true if designs are clean; false if errors are detected.

Arguments

-cellView

The physical cellview.

-hierarchy

The flag whether to verify designs through the hierarchy. (Default is: true)

-outputFile

The output file. (Default is:)

-skipCells

The cells to skip to verify.

-exportMarkers

Export violations as OA Markers. (Default is: false)

-verifyMode

Option to Specify which mode to verify. ({ PLE | SDL }, default is: SDL)

lx::verifyNetVoltage

Description

Verifies the net voltage range between schematic net and layout net.

Syntax

```
lx::verifyNetVoltage -design <oaDesign>
```

Arguments

-design

Layout design to be verified

ilx::add

Description

An interactive layout command that adds devices in Symbolic Editor window.

Syntax

```
ilx::add [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

ilx::addDummy

Description

An interactive layout command that adds dummy devices in Symbolic Editor window.

Syntax

```
ilx::addDummy [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

ilx::addIncremental

Description

Interactive symbolic add incremental command

Syntax

```
ilx::addIncremental [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

context

The context in which the command is activated.

-window

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::addPeripheralDummy

Description

An interactive layout command that invokes the **Add Peripheral Dummies** command in Symbolic Editor window.

Syntax

```
ilx::addPeripheralDummy [<context: deContext | unsigned int>] [-window  
    <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

ilx::alignGates

Description

An interactive layout command that invokes the **Align Gates** command in Symbolic Editor window.

Syntax

```
ilx::alignGates [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

ilx::alignIndividual

Description

Interactive symbolic align individual device command.

Syntax

```
ilx::alignIndividual [<context: deContext | unsigned int>] [-window  
    <int>]
```

Arguments

context

Context of activation.

-window

Window(id) of symbolic editor window to invoke the command.

Returns

<none>

ilx::alignRow

Description

An interactive layout command that invokes the **Align Row** command in the Symbolic Editor window.

Syntax

```
ilx::alignRow [<context: deContext | unsigned int>] [-window <int>]  
    -alignment <enum>
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

-alignment

Specify the alignment of the align row command, {bottom | center | top}.

ilx::applyPresetParams

Description

Brings up the **Apply Preset Params** toolbar and applies the preset parameters on the selected layout instance.

Syntax

```
ilx::applyPresetParams [<context: deContext | unsigned int>]
```

Arguments

context

The context in which the command is activated.

ilx::associateSpareDummy

Description

Interactive symbolic associate spare dummy command.

Syntax

```
ilx::associateSpareDummy [<context: deContext | unsigned int>] [-window  
    <int>]
```

Arguments

-context

Specifies the context of activation.

-window

Specifies the Window Id of Symbolic Editor window to invoke command.

Returns

<none>

ilx::calculateGridSize

Description

Interactive symbolic calculate grid size command.

Syntax

```
ilx::calculateGridSize [<context: deContext | unsigned int>] [-window  
    <int>]
```

Returns

<none>

Arguments

context

Context of activation.

-window

Window(id) of symbolic editor window to invoke command.

ilx::chainSerialRes

Description

Interactive symbolic chain serial resistor command.

Syntax

```
ilx::chainSerialRes [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

context

The context in which the command is activated.

-window

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::clone

Description

An interactive layout command that identifies a layout source for cloning, selects a schematic target candidate, and places the target in the layout design.

Syntax

```
ilx::clone [<context: deContext | unsigned int>]
```

Arguments

context

The context in which the command is activated.

ilx::copy

Description

An interactive layout command that invokes the **Copy** command in the Symbolic Editor window.

Syntax

```
ilx::copy [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

ilx::customizeSpacing

Description

Interactive symbolic customize spacing command.

Syntax

```
ilx::customizeSpacing [<context: deContext | unsigned int>] [-window  
  <int>]
```

Arguments

context

Context of activation.

-window

Window(id) of symbolic editor window to invoke the command.

Returns

<none>

ilx::delete

Description

An interactive symbolic command that deletes objects.

Syntax

```
ilx::delete [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

Errors

- An invalid window object is passed.
- No design is opened in specified window.

ilx::deleteTrack

Description

Interactive symbolic delete track command

Syntax

```
ilx::deleteTrack [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

context

The context in which the command is activated.

-window

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::distance

Description

An interactive symbolic command that measures distance.

Syntax

```
ilx::distance [<context: deContext | unsigned int>]
```

Arguments

`context`

The context in which the command is activated.

ilx::establishCorrespondence

Description

This command initiates the command for the user to establish correspondence between objects in the logical (schematic) netlist and physical (layout) netlist. The correspondence is name based, and establishing or modifying correspondence through this command results in modification of the name of the appropriate layout objects to match the name of the corresponding schematic object.

Syntax

```
ilx::establishCorrespondence [<context: deContext | unsigned int>]
```

Arguments

`context`

The context in which the command is activated.

Errors

- The window does not exist.
- The SDL session is not initialized.
- Command is not started from the layout deContext.

ilx::flip

Description

An interactive layout command that invokes the **Flip** command in the Symbolic Editor window.

Syntax

```
ilx::flip [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

`[int]`

The window/ID number.

context

The context in which the command is activated.

ilx::flipRow

Description

Interactive symbolic flip row command.

Syntax

```
ilx::flipRow [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

context

The context in which the command is activated.

-window

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::foldGates

Description

An interactive layout command that invokes the **Fold Gates** command in the Symbolic Editor window.

Syntax

```
ilx::foldGates [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

ilx::generateGroupPlacements

Description

Interactive symbolic group placements generator command.

Syntax

```
ilx::generateGroupPlacements [<context: deContext | unsigned int>] [-wi  
ndow <int>]
```

Arguments

-context

Specifies the context of activation.

-window

Specifies the Window Id of Symbolic Editor window to invoke command.

Returns

<none>

ilx::generateGuardRings

Description

Interactive symbolic generate guard rings command

Syntax

```
ilx::generateGuardRings [<context: deContext | unsigned int>] [-window  
<int>]
```

Arguments

context

The context in which the command is activated.

-window

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::generatePattern

Description

Interactive symbolic generate pattern command.

Syntax

```
ilx::generatePattern [<context: deContext | unsigned int>] [-window  
<int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::generatePlacement

Description

Interactive symbolic generate placement command.

Syntax

```
ilx::generatePlacement [<context: deContext | unsigned int>] [-window  
    <int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

Window(id) of symbolic editor window to invoke command

ilx::highlightFlightline

Description

Interactive symbolic highlight flight lines command.

Syntax

```
ilx::highlightFlightline [<context: deContext | unsigned int>] [-window  
    <int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

Returns

<none>

ilx::insertColumn

Description

An interactive layout command that invokes Insert Column command in Symbolic Editor window.

Syntax

```
ilx::insertColumn [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

ilx::insertRow

Description

An interactive layout command that invokes Insert Row command in Symbolic Editor window.

Syntax

```
ilx::insertRow [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

ilx::insertShift

Description

Interactive symbolic insert shift command.

Syntax

```
ilx::insertShift [<context: deContext | unsigned int>] [-window <int>]  
-direction <enum>
```

Arguments

context

Context of activation.

-window

Window(id) of symbolic editor window to invoke the command.

-direction

Direction of insert shift command {down | right}.

Returns

<none>

ilx::insertTrack

Description

Interactive symbolic insert track command

Syntax

```
ilx::insertTrack [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

context

The context in which the command is activated.

-window

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::manualCorrespondence

Description

Interactive manual correspondence command

Syntax

```
ilx::manualCorrespondence [<context: deContext | unsigned int>]
```

Arguments

context

The context in which the command is activated.

ilx::merge

Description

An interactive command that merges SD for the selected object.

Syntax

```
ilx::merge [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

ilx::mergeGates

Description

An interactive layout command that invokes Merge Gates command in Symbolic Editor window.

Syntax

```
ilx::mergeGates [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

ilx::mirror

Description

An interactive command that mirrors selected object.

Syntax

```
ilx::mirror [<context: deContext | unsigned int>] [-window <int>] -flip  
<enum>
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

-flip

Specify the mirror type of rotation, {horizontal | rotate90 | vertical}.

ilx::modifyDummy

Description

An interactive command that modifies the selected dummy gates.

Syntax

```
ilx::modifyDummy [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

Returns

<none>

ilx::modifyFlightline

Description

Interactive symbolic modify flight line command

Syntax

```
ilx::modifyFlightline [<context: deContext | unsigned int>] [-window  
<int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::modifyPode

Description

Interactive symbolic modify pode net command

Syntax

```
ilx::modifyPode [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::modifyPreset

Description

Interactive symbolic modify preset command.

Syntax

```
ilx::modifyPreset [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::move

Description

An interactive command that moves the selected object.

Syntax

```
ilx::move [<context: deContext | unsigned int>] [int]
```

Arguments

[int]

The window/ID number.

context

The context in which the command is activated.

Returns

<none>

ilx::moveFlightline

Description

Interactive symbolic move flight line command.

Syntax

```
ilx::moveFlightline [<context: deContext | unsigned int>] [-window  
<int>]
```

Arguments

context

The context in which the command is activated.

-window

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::moveInstance

Description

Moves an instance up or down.

Syntax

```
ilx::moveInstance [<context: deContext | unsigned int>]
```

Returns

<none>

Arguments

context

The context in which this command is activated.

ilx::optimizeOD

Description

An interactive command that optimizes OD sharing.

Syntax

```
ilx::optimizeOD [<context: deContext | unsigned int>] [int]
```

Arguments

Window

Window ID.

context

The context in which the command is activated.

ilx::pickAndPlace

Description

This command displays the dialog box for user to select logic sources to place them in the layout design, generate layout instances based on info from logic source, and display the generated layout in runtime set and right-mouse-button to commit them in the target layout design.

Syntax

```
ilx::pickAndPlace [<context: deContext | unsigned int>] [-inst  
[<string>]] [-net [<string>]] [-term [<string>]] [-node [<string>]]  
[-group [<string>]]
```

Arguments

`context`

The context in which the command is activated.

`-inst`

Instant names for placing.

`-net`

Net names for placing.

`-term`

Term names for placing.

`-node`

Node names for placing.

`-group`

To specify the group names that for placing

Errors

- The SDL session has not been initialized.
- The command is not started from the layout *deContext*.

ilx::reassignNet

Description

Interactive symbolic reassign net command

Syntax

```
ilx::reassignNet [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::rechain

Description

Invokes the Rechain command.

Syntax

```
ilx::rechain [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

window

Window ID.

context

The context in which the command is activated.

ilx::removeColumn

Description

Interactive symbolic remove column command.

Syntax

```
ilx::removeColumn [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

context

The context in which the command is activated.

-window

The window (ID) of the symbolic editor window in which to invoke the command.

ilx::removeLayoutParam

Description

Interactive symbolic remove layout parameters command.

Syntax

```
ilx::removeLayoutParam [<context: deContext | unsigned int>] [-window  
<int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::removeRow

Description

Interactive symbolic remove row command.

Syntax

```
ilx::removeRow [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the symbolic editor window in which to invoke the command.

ilx::removeShift

Description

Interactive symbolic remove shift command.

Syntax

```
ilx::removeShift [<context: deContext | unsigned int>] [-window <int>]  
-direction <enum>
```

Arguments

`context`

Context of activation.

`-window`

Window(id) of symbolic editor window to invoke the command.

`-direction`

Direction of remove shift command {left | up}.

Returns

<none>

ilx::reportSummary

Description

Interactive symbolic report summary command.

Syntax

```
ilx::reportSummary [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::resetCustomizedSpacing

Description

Interactive symbolic reset customized spacing command.

Syntax

```
ilx::resetCustomizedSpacing [<context: deContext | unsigned int>]  
[-window <int>]
```

Arguments

`context`

Context of activation.

`-window`

Window(id) of symbolic editor window to invoke the command.

Returns

<none>

ilx::resetIndividualAlignment

Description

Interactive symbolic reset individual alignment command.

Syntax

```
ilx::resetIndividualAlignment [<context: deContext | unsigned int>]  
    [-window <int>]
```

Arguments

context

Context of activation.

-window

Window(id) of symbolic editor window to invoke the command.

Returns

<none>

ilx::rotate

Description

Interactive symbolic rotate command.

Syntax

```
ilx::rotate [<context: deContext | unsigned int>] [-window <int>]
```

Returns

<none>

Arguments

context

The context in which this command is activated.

-window

Window(id) of symbolic editor window to invoke command.

ilx::sedRealize

Description

Realize from Symbolic Editor.

Syntax

```
ilx::sedRealize [<context: deContext | unsigned int>] [-session  
  <lxSymbolicEditorSession>]
```

Arguments

context

The context in which the command is activated.

-session

Symbolic Editor session.

ilx::selectEmpty

Description

Invokes the Select Empty command.

Syntax

```
ilx::selectEmpty [[<context: deContext | unsigned int>] [-window <int>]
```

Arguments

window

Window ID.

context

The context in which the command is activated.

ilx::split

Description

Invokes the Split command.

Syntax

```
ilx::split [[<context: deContext | unsigned int>] [-window <int>]
```


Arguments

`window`

Window ID.

`context`

The context in which the command is activated.

ilx::splitDevice

Description

Invokes the Split Device toolbar and starts splitting the instance objects.

Syntax

```
ilx::splitDevice [<context: deContext | unsigned int>]
```

Arguments

`context`

The context in which the command is activated.

ilx::swapDevice

Description

Interactive symbolic swap device command.

Syntax

```
ilx::swapDevice [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::swapInterdigitate

Description

Starts the interactive symbolic swap interdigitate command.

Syntax

```
ilx::swapInterdigitate [<context: deContext | unsigned int>] [-window  
  <int>]
```

Arguments

context

The context in which the command is activated.

-window

The window (ID) of the Symbolic Editor window in which to invoke the command.

Returns

<none>

ilx::swapSD

Description

Invokes the Swap SD command.

Syntax

```
ilx::swapSD [[<context: deContext | unsigned int>] [-window <int>]]
```

Arguments

window

Window ID.

context

The context in which the command is activated.

ilx::swapTerm

Description

Interactive symbolic swap terms command.

Syntax

```
ilx::swapTerm [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::symbolicEditor

Description

Invokes the Symbolic Editor command.

Syntax

```
ilx::symbolicEditor [[<context: deContext | unsigned int>] [-source  
  <enum>] [-mode <enum>]
```

Arguments

`context`

The context in which the command is activated.

`-source`

Specify source to get selected objects, {layout | layout DN | schematic}.

`-mode`

Specify the mode of Symbolic Editor, {chaining | matching}.

ilx::transpose

Description

Invokes the interactive symbolic transpose command.

Syntax

```
ilx::transpose [<context: deContext | unsigned int>] [-window <int>]
```

Arguments

`context`

The context in which the command is activated.

`-window`

The window (ID) of the Symbolic Editor window in which to invoke the command.

ilx::unalignGates

Description

Invokes the **Unalign Gates** command.

Syntax

```
ilx::unalignGates [[<context: deContext | unsigned int>][-window <int>]
```

Arguments

`window`

Window ID.

`context`

The context in which the command is activated.

ilx::unsetFingerGroup

Description

Invokes the **Unset Finger Group** command.

Syntax

```
ilx::unsetFingerGroup [[<context: deContext | unsigned int>][-window  
<int>]
```

Arguments

`window`

Window ID.

`context`

The context in which the command is activated.

ilx::updateLayout

Description

Invokes the **Update Layout** command.

Syntax

```
ilx::updateLayout [<context: deContext | unsigned int>]
```

Arguments

context

The context in which the command is activated.

ilx::viaTrackPlanner

Description

Interactive by way of Track Planner.

Syntax

```
ilx::viaTrackPlanner [<context: deContext | unsigned int>] [-source  
  <enum>]
```

Arguments

context

The context in which the command is activated.

-source

To specify source to get selected objects, { layout | layoutDN }. The default is layout.

Enumerations

Table 142 Enumerations

Name	Values	Description
enAnnotationStyleType	Maker Shadow	Type of highlighting to show unplaced objects in schematic.
enAnnotationType	none partial detailed	Type of highlighting to be added to instances/pins/wires in schematic.
enBusOrder	ascending descending	Bus enumeration order.
enCDFCCallbackTriggerMode	Auto Never Always	Specify callback trigger mode.

Table 142 Enumerations (Continued)

Name	Values	Description
enColorStatus	Unrealized Realized	Status of dim objects at SDL browser.
enConstraintFixAction	Create Update Delete Signoff Skip	Initial action in Constraint Conflicts dialog box and Fix Constraint dialog box.
enConstraintUpdateMode	all new exist	Constraint update mode.
enDragAndDropMOS	PnP SED	Specifies the invoked tool.
enExternalConnections	auto enable disable	External connections option values in <code>ilx::splitDevice</code> .
enGenPinInput	design selection	Specifies the region of generating pins.
enGenPinRule	net label mix	Specifies the rule of generating pins.
enMissingConstraintSource	layout schematic	Source of constraints that have no counterparts.
enNetlistRefreshMode	Immediate deferred	Mode of SDL re-initialization if the logical data change.
enPresetMessageType	none instanceMapping copiedParameter	Preset debug message type.
enResistorConnectionType	Series-parallel Parallel-Series	Specifies connection type of resistors.
enShowLogicalSourceError	dialog console	Interface for display the logical source error.
enTemplateCommand	all generateLayout pickAndPlace	Validates layout generation commands for a custom layout template.
enBasis	widthHeightRatio width length	Basis for calculating the bounding box of <code>oaBoundary</code> .
enBoundary	prBoundary snapBoundary	Types of <code>oaBoundary</code> to create in Generate Layout.
enCloneFrom	schematic layout	Where clone target candidates should be searched from.
enCorrespondenceType	all instance terminal	Correspondence object types.

Table 142 Enumerations (Continued)

Name	Values	Description
enMarkerBrowser	never onErrors onErrorsWarnings	When to launch Marker Browser assistant after <code>lx::verify</code> .
enNetlistFilterType	all unmatched	Netlist filter types.
enOrient	R0 R90 R180 R270 MY MYR90 MX MXR90	Orientations of run time designs in <code>ilx::pickAndPlace</code> .
enPlace	singleLogical singleSelected allSelected	Placement selection modes for <code>ilx::pickAndPlace</code> .
enProbeType	inst net terminal instTerm	Probe types.
enObjectType	instance net terminal	Associated object types.
enSeverity	message warning error fatal	Severity classifications. Corresponds to OA enums.
enUpdateType	none update remove	Type of update SDL date on copy/move.
enVectorSuffix	Parenthesis Underscore	Vector suffix type.

Objects

This section covers the following:

- [IxConnectionPoint](#)
- [IxSession](#)
- [IxNetlist](#)
- [IxInst](#)
- [IxTerm](#)
- [IxInstTerm](#)
- [IxPair](#)
- [IxNode](#)
- [IxNet](#)

lxConnectionPoint

Object description should include a brief explanation of the object purpose.

Table 143 Attributes

Name	Type	Access	Description
inst	<oaInst>	read	Parent PCell instance.
layer	String	Read	Name of the annotation Layer which defined the connection point.
x	<float>	read-only	X coordinate of point location in edit design.
y	<float>	read-only	Y coordinate of point location in edit design.
xInst	<float>	Read-only	X coordinate of point location in instance sub-master.
yInst	<float>	Read-only	Y coordinate of point location in instance sub-master.
net	<oaNet>	Read	oaNet associated with the oaShape that defined the connection point.
term	<oaTerm>	Read	oaTerm associated with the oaShape that defined the connection point.
differentNets	<bool>	Read	Flag indicating that shapes from different nets are incident at this connection point.
type	String	Read	Type name of the connection point. Valid values are empty filled erase local keepout.
trackPitch	<float>	Read	Pitch attribute value of the track/trackPattern on which connection point was defined.
trackWidth	<float>	Read	Width attribute value of the track/trackPattern on which connection point was defined.
trackColor	String	Read	Color attribute value of the track/trackPattern on which connection point was defined.

IxSession

Represents one invocation of SDL in a Custom Compiler session.

Table 144 Attributes

Name	Type	Access	Description
logicalCellview	<dmCellView>	Read-only	The source hierarchy editor configuration for the session.
logicalRootCellview	<dmCellView>	Read-only	The top level logical design for the source hierarchy editor configuration.
physicalCellview	<dmCellView>	Read-only	The target physical design for the session.
logicalNetlist	<IxNetlist>	Read-only	The SDL netlist representing the flattened logical design.
physicalNetlist	<IxNetlist>	Read-only	The SDL netlist representing the physical design.

IxNetlist

Represents a flattened design. The netlist contains representations of the leaf level instances and top level terminals of the flattened design.

The logical netlist is produced by elaborating the root logical design using the binding rules defined in the source hierarchy editor configuration. The elaboration determines the switch master binding for the leaf level instances.

The physical netlist is produced by elaborating the target physical design by traversing the instance placed masters.

Table 145 Attributes

Name	Type	Access	Description
insts	{<IxInst>}	Read-only	The leaf level instances for the netlist.
unboundInsts	{<IxInst>}	read-only	The unbound instances for the netlist.
terms	{<IxTerms>}	Read-only	The top level terminals for the netlist.

Table 145 Attributes (Continued)

Name	Type	Access	Description
nets	{<IxNet>}	read-only	The top level nets for the netlist.
root	<IxNode>	read-only	Netlist hierarchy.

IxInst

Represents a leaf level instance in an IxNetlist.

Table 146 Attributes

Name	Type	Access	Description
name	<string>	Read-only	The hierarchical instance name.
libName	<string>	Read-only	The library name of the switched master.
cellName	<string>	Read-only	The cell name of the switched master.
viewName	<string>	Read-only	The view name of the switched master.
x	<string>	Read-only	The x-coordinate for the instance.
y	<string>	Read-only	The y-coordinate for the instance.
orient	<string>	Read-only	The orientation of the instance.
instTerms	{<IxInstTerm>}	Read-only	The instTerms for the instance.
params	{<IxPair>}	Read-only	The parameters used for comparison during Verify, for the instance.
oaOverrideParams	{<IxPair>}	Read-only	The locally overridden PCell parameters for the instance.
mismatch	<string>	Read-only	The mismatch type for the instance.
realizedStatus	{<bool>}	Read-only	The realized status for the instance.
configBindingLcv	<dmCellView>	Read-only	The config binding library/cell/view.
placedLcv	<dmCellView>	Read-only	The placed binding library/cell/view.
lcv	<dmCellView>	Read-only	The switched library/cell/view.

Table 146 *Attributes (Continued)*

Name	Type	Access	Description
DummyFlag	{<bool>}	Read-only	Determines if the instance can be a dummy device.
node	<IxNode>	Read-only	The parent node.

IxTerm

Represents a top level terminal in an IxNetlist.

Table 147 *Attributes*

Name	Type	Access	Description
name	<string>	Read-only	The terminal name.
netName	<string>	Read-only	The terminal's hierarchical net name.
termType	<string>	Read-only	The oaTermType.
realizedStatus	<bool>	Read-only	The realized status of the term.

IxInstTerm

Represents a connection between a net and a terminal in the switched master for an IxInst.

Table 148 *Attributes*

Name	Type	Access	Description
netName	<string>	Read-only	The hierarchical net name connected to the instance.
termName	<string>	Read-only	The switched master terminal name.

IxPair

Represents a parameter name/value pair.

Table 149 Attributes

Name	Type	Access	Description
name	<string>	Read-only	The parameter name.
value	<string>	Read-only	The parameter value.

IxNode

Represents an intermediate level that constructs the hierarchy in an IxNetlist.

Table 150 Attributes

Name	Type	Access	Description
name	<string>	Read-only	The node name.
children	{<IxNode>}	Read-only	The child nodes under the hierarchy.
leaf	<IxInst>	Read-only	The leaf instance.
parent	<IxNode>	Read-only	The parent node.
associatedName	<string>	Read-only	The representation name in the physical or logical.
isMfactored	<bool>	Read-only	Determines if the node can be an m-factor node.
isVector	<bool>	Read-only	Determines if the node can be a vector node.
isExplicitMfactor	<bool>	Read-only	Determines if the node can be an explicit m-factor node.
isExplicitVectorBit	<bool>	Read-only	Determines if the node can be an explicit vector node.
isPcell	<bool>	Read-only	Determines if the node can be a PCell node.
isFinger	<bool>	Read-only	Determines if the node can be a finger node of Stack MOS.
startIndex	<int>	Read-only	The starting bit of the vector.
stopIndex	<int>	Read-only	The stopping bit of the vector.
mfactor	<int>	Read-only	The m-factor value.

Table 150 Attributes (Continued)

Name	Type	Access	Description
lcv	<dmCellView>	Read-only	The binding library/cell/view.

IxNet

Represents a top level net in an IxNetlist.

Table 151 Attributes

Name	Type	Access	Description
name	<string>	Read-only	The net name.
mismatch	<string>	Read-only	The mismatch type.
maxVoltage	<double>	Read-only	The maximum voltage.
minVoltage	<double>	Read-only	The minimum voltage.
voltageSource	<string>	Read-only	The source of voltage range.

Preferences

Table 152 lists Custom Compiler Ix namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 152 Schematic Driven Layout Preferences

Name	Type	Scope	Default	Description
IxAbutAfterEdit	bool	cellview	1	Determines to abut MOS devices automatically after editing. Valid values: true, false
IxAbutBulkAware	bool	cellview	1	Determines if devices can be abutted for different bulk nets. Valid values: true, false
IxAbutSDAware	bool	cellview	1	Determines if devices can be abutted for different source/drain nets. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxAbutUnequalWidth	bool	cellview	true	Allows sharing of different sized diffusions. Valid values: true, false
IxAddAlignDummy	bool	user	false	Adds dummy devices for alignment in chaining mode. Valid values: true, false
IxAddDummyDialogMode	bool	user	false	The <code>ilx::addDummy</code> command options dialog eject state. Valid values: true, false
IxAddShapesEnable	bool	user	false	Creates optional layers.
IxAddShapeSettingEnable	bool	user	false	Create Optional Layers.
IxAddShapeSettingFile	string	user		Add Shapes Settings Save File Name.
IxAddShapesFileName	string	user		Adds Shape Settings Save File Name.
IxAddShapesSet	string	user		Optional layer purpose and extension.
IxAddShapeSettingSet	string	user		Optional Layer Purpose and Extension.
IxAddTextPattern	string	user		Text pattern for the Add command.
IxAddTextPatternBackup	string	user		Text pattern backup for the Add command.
IxAddTextPatternEnable	bool	user	false	Adds devices with text pattern. Valid values: true, false
IxAddTextPatternRepeatX	int	user	1	X-extend for Text Pattern in the Add command.
IxAddTextPatternRepeatY	int	user	1	Y-extend for Text Pattern in the Add command.
IxAddTextPatterns	string	user		Text patterns for the Add command.
IxAdvanceNode	bool	cellview	false	Shows PODE for advanced nodes. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxAlignGate	bool	user	true	Aligns P and N gates with the same net. Valid values: true, false
IxAllowEmptyTiles	bool	cellview	false	Allows empty tiles. Valid values: true, false
IxAlwaysTriggerSchPCellCDFCallbacks	bool	user	true	Triggers CDF callbacks for schematic PCells. Valid values: true, false
IxApplyAllInstancesWithSpacing	bool	cellview	false	Applies all instances with instance spacing. Valid values: true, false
IxAssociateSpareDummyGroup	string	user		Associates spare dummy to a group.
IxAutoArrangeWindows	bool	user	true	Automatically arranges windows during initialization. Valid values: true, false
IxAutoChain	bool	cellview	true	Auto Chain. Valid values: true, false
IxAutoFold	bool	cellview	false	Auto Fold. Valid values: true, false
IxAutoUpdateConfig	bool	cellview	true	Automatically updates the config during flatten hierarchy. Valid values: true, false
IxAutoUpdateMismatchedConstraint	bool	cellview	true	Specifies if Auto Update fixes mismatched constraints. Valid values: true, false
IxAutoUpdateMissingConstraint	bool	cellview	true	Specifies if Auto Update fixes missing constraints. Valid values: true, false
IxAutoUpdateNetVoltageMismatch	bool	cellview	true	ECO Net Voltage Mismatch AutoUpdate Enable. Valid values: true, false
IxAutoUpdateTemplateOutOfDate	bool	cellview	false	ECO Template Mismatch AutoUpdate Enable. Valid values: true, false
IxAutoUpdateUnmatchedConstraint	bool	cellview	true	Specifies if Auto Update fixes unmatched constraints. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxAutoUpdateUnplacedPlacementStatusInst	bool	cellview	false	Controls whether ECO to check instance whose Placement Status is unplaced.
IxAutoVerifyDesignOpen	bool	user	false	Runs ECO verify after opening a design. Valid values: true, false
IxBoundaryBasis	string	cellview	Width to Height Ratio	Boundary mode.
IxBoundaryConfigFile	string	cellview		Config file path of Create Boundary in the Generate Layout command.
IxBoundaryHeight	float	cellview	1	Boundary height.
IxBoundaryType	boundary_type	cellview	PR Boundary	Boundary type. Valid values: Any integer in a [0 1] range
IxBoundaryUtilization	float	cellview	70	Boundary utilization.
IxBoundaryWidth	float	cellview	1	Boundary width.
IxBoundaryWidthHeightRatio	float	cellview	1	Boundary width/height ratio.
IxBrowserHighlightPacketName	string	cellview	highlight7	Specifies a color for highlighting objects.
IxBrowserShadowMode	bool	cellview	false	If enabled, selected objects are shown brighter than the surrounding objects. Valid values: true, false
IxBusOrder	string	cellview	Descending	Bus order. Valid values: Ascending, Descending
IxCalcGridDeviceSpacingGroup	bool	user	false	Collapse Device and Spacing Group.
IxCalculateGridSizeDialogMode	bool	user	true	The <code>ilx::calculateGridSize</code> command options dialog eject state. Valid values: true, false
IxCalGridLayoutMode	string	cellview	Max Height	Specifies a mode to calculate grid size.
IxCalGridLayoutSize	float	cellview	1	Specifies a layout size to calculate grid size.

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
<code>IxCalGridWhenAbut</code>	bool	cellview	true	Calculates the grid size when abutment. Valid values: true, false
<code>IxChainSerialResConnect</code>	string	user	Full Chain	Connect of chain serial resistor.
<code>IxChainSerialResHighlight</code>	string	user	highlight7	Start resistor of chain serial resistor.
<code>IxChainSerialResPattern</code>	string	user	S1	Pattern of chain serial resistor. Valid values: S1, S2, S3, S4, S5, S6, S7, S8, Z1, Z2, Z3, Z4, Z5, Z6, Z7, Z8
<code>IxChainSerialResStart</code>	string	user		Start resistor of chain serial resistor.
<code>IxCheckAbutment</code>	bool	user	false	SED checker check Abutment Checkbox.
<code>IxCheckDummy</code>	bool	user	false	SED checker check Abutment Checkbox.
<code>IxCheckInstConn</code>	bool	cellview	true	ECO Connectivity Check Enable. Valid values: true, false
<code>IxCheckInstConnSeverity</code>	eco_severity_ref	cellview	error	Eco instconnectivity severity.
<code>IxCheckInstParams</code>	bool	cellview	true	ECO InstParam Check Enable. Valid values: true, false
<code>IxCheckInstParamsSeverity</code>	eco_severity_ref	cellview	error	Eco instparams severity.
<code>IxCheckMasterMismatch</code>	bool	cellview	true	ECO Master Mismatch Check Enable. Valid values: true, false
<code>IxCheckMasterMismatchSeverity</code>	eco_severity_ref	cellview	error	Eco mastermismatch severity.
<code>IxCheckMismatchedConstraint</code>	bool	cellview	true	Specifies if <code>lx::verify</code> checks for mismatched constraints. Valid values: true, false
<code>IxCheckMissingConstraint</code>	bool	cellview	true	Specifies if <code>lx::verify</code> checks for missing constraints. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
<code>lxCheckNameMismatch</code>	bool	cellview	false	ECO Name Mismatch Check Enable. Valid values: true, false
<code>lxCheckNameMismatchMlevel</code>	int	cellview	Error	ECO <code>lxCheckNameMismatchMlevel</code> preference. Valid values: Any integer in the [0 2] range.
<code>lxCheckNameMismatchSeverity</code>	<code>eco_severity_p</code> ref	cellview	error	Eco <code>namemismatch</code> severity.
<code>lxCheckNetVoltageMismatch</code>	bool	cellview	true	ECO Net Voltage Mismatch Check Enable. Valid values: true, false
<code>lxCheckNetVoltageMismatchSeverity</code>	<code>eco_severity_p</code> ref	cellview	error	Eco <code>netvoltageismatch</code> severity.
<code>lxCheckTemplateOutOfDate</code>	bool	cellview	true	ECO Template Mismatch Check Enable. Valid values: true, false
<code>lxCheckTemplateOutOfDateSeverity</code>	<code>eco_severity_p</code> ref	cellview	warning	Eco <code>templateoutofdate</code> severity.
<code>lxCheckTermConn</code>	bool	cellview	true	ECO Terminal Connectivity Check Enable. Valid values: true, false
<code>lxCheckTermConnDef</code>	bool	cellview	true	ECO Terminal ConnectDef Check Enable. Valid values: true, false
<code>lxCheckTermConnDefSeverity</code>	<code>eco_severity_p</code> ref	cellview	error	Eco <code>termconndefmismatch</code> severity.
<code>lxCheckTermConnMlevel</code>	int	cellview	Error	ECO <code>lxCheckTermConnMlevel</code> preference. Valid values: Any integer in the [0 2] range.
<code>lxCheckTermConnSeverity</code>	<code>eco_severity_p</code> ref	cellview	error	Eco <code>termconnmismatch</code> severity.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
lxCheCkTermType	bool	cellview	true	ECO Terminal Type Check Enable. Valid values: true, false
lxCheCkTermTypeSeverity	eco_severity_p ref	cellview	error	Eco termtypemismatch severity.
lxCheCkUnmatchedConstraint	bool	cellview	true	Specifies if lX::verify checks for unmatched constraints. Valid values: true, false
lxCheCkUnmatchedInsts	bool	cellview	true	ECO Extra Layout Inst Check Enable. Valid values: true, false
lxCheCkUnmatchedInstsSeverity	eco_severity_p ref	cellview	error	Eco extralayoutinst severity.
lxCheCkUnmatchedTerms	bool	cellview	true	ECO Extra Layout Term Check Enable. Valid values: true, false
lxCheCkUnmatchedTermsSeverity	eco_severity_p ref	cellview	error	Eco extralayoutterm severity.
lxCheCkUnplacedInsts	bool	cellview	true	ECO Unplaced Inst Check Enable. Valid values: true, false
lxCheCkUnplacedInstsSeverity	eco_severity_p ref	cellview	error	Eco unplaced instance severity.
lxCheCkUnplacedPlacementStatusInst	bool	cellview	false	Controls whether ECO to auto update instance whose Placement Status is unplaced.
lxCheCkUnplacedPlacementStatusInstSeverity	eco_severity_p ref	cellview	error	Eco unplaced placement status inst severity.
lxCheCkUnplacedTerms	bool	cellview	true	ECO Unplaced Term Check Enable. Valid values: true, false

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxCheckUnplacedTermsMlevel	int	cellview	Error	ECO IxCheckUnplacedTermsMlevel preference. Valid values: Any integer in the [0 2] range
IxCheckUnplacedTermsSeverity	eco_sev erity_p ref	cellview	error	Eco unplacedterm severity.
IxCloneAutoUpdateDXDY	bool	cellview	true	Auto Update DX & DY of Clone Array. Valid values: true, false
IxCloneCols	int	cellview	1	Columns of Clone Array.
IxCloneDX	float	cellview	1	DX of Clone Array.
IxCloneDY	float	cellview	1	DY of Clone Array.
IxCloneEnableMakeNode	bool	user	true	Enables to search master of make-node. Valid values: true, false
IxCloneFrom	clone_fr om_butt ons	cellview	schematic	Indicates where (schematic or layout view) the clone command should search for the target candidates. Valid values: Any integer in the [0 1] range
IxCloneIncludeSource	bool	cellview	false	Include Clone Source. Valid values: true, false
IxCloneKeepSync	bool	cellview	true	Controls the clone wrapping. If IxCloneMatchParams is true then clones are wrapped to syncFigGroup or oaFigGroup. If IxCloneMatchParams is false, the clones are wrapped into a oaFigGroup. Valid values: true, false
IxCloneMatchParams	bool	cellview	true	Matches parameters between clone source and target. Valid values: true, false
IxCloneMode	clone_m ode_butt ons	cellview	connectivity	Clone mode.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxCloneReplaceExisting	bool	cellview	true	Replaces existing layout instances with a clone target. Valid values: true, false
IxCloneRows	int	cellview	1	Rows of Clone Array.
IxCloneSearchSchematicSwitched Master	bool	cellview	true	Clones search schematic by switched master. Valid values: true, false
IxCloneShowCandidateCheckbox	bool	cellview	false	Shows Candidate checkbox. Valid values: true, false
IxCloneUseFixedAnchorPoint	bool	user	true	Clone place candidate with fixed anchor point.
IxConfigDefaultSuffix	string	user	.config	Specifies the default suffix of a config view name.
IxConfigLCV	lcv	user	//	Configuration View LCV.
IxConnectionMismatchDisplayCount	int	cellview	30	Connection mismatch display count.
IxConstraintMismatchFixMode	enConstraintFixAction	cellview	Update	The default action for fixing mismatched constraints.
IxConstraintTransferUpdate	bool	user	true	Specifies if mismatched transferred constraints should be updated. Valid values: true, false
IxConstraintUpdateMode	enConstraintUpdateMode	cellview	All	Updates all constraints based on logic or prompt the user for choices for individual constraint inconsistency.
IxConstraintViewSearchList	string	cellview	constraint	The constraint view search list to use when creating a physical configuration.
IxContextAwarePaste	bool	cellview	true	Pastes instances at point. Valid values: true, false
IxContextAwarePoint	unknown	cellview	0 0	Context aware at point.
IxCopyColumn	int	user	1	Copies number of columns.

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxCopyDobuleBackColumn	bool	user	false	Mirrors X the objects in the even copied rows. Valid values: true, false
IxCopyDobuleBackRow	bool	user	false	Flips the objects in the even copied columns. Valid values: true, false
IxCopyRemoveOriginal	bool	user	false	Removes the original selected objects. Valid values: true, false
IxCopyRow	int	user	1	Copies number of rows.
IxCopySequenceMode	enum	user	horizontal	Specifies the order of placing copied object by horizontal or vertical first.
IxCreateBoundary	bool	cellview	false	Creates boundary. Valid values: true, false
IxCreateFigGroup	bool	cellview	true	Creates figure group for realized transistors. Valid values: true, false
IxCreateInstanceNameLabel	bool	cellview	false	Create Label to Show Instance Name. Valid values: true, false
IxCreateInstances	bool	cellview	true	Create Instances. Valid values: true, false
IxCreateLabels	bool	cellview	true	Create Labels. Valid values: true, false
IxCreatePinLabel	bool	cellview	false	Defines if labels can be created for pins. Valid values: true, false
IxCreatePins	bool	cellview	true	Create Pin. Valid values: true, false
IxCreateSoftMacro	bool	cellview	false	Determines the pick and place mode for a non-leaf node in a SDL browser hierarchy model. Valid values: true, false
IxCreateTopLevelTemplateConstra int	bool	user	true	Creates top-level template constraints in the constraint hierarchy. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxCrossHighlighting	bool	cellview	true	Controls whether mouse-over highlighting of an object in the SDL window also highlights the corresponding objects in other related windows of SDL session. Valid values: true, false
IxCTCreateGround	bool	cellview	false	Creates ground layer. Valid values: true, false
IxCTCreateOptionLayer	bool	cellview	false	Creates optional layers. Valid values: true, false
IxCTCreatePower	bool	cellview	false	Create power layer. Valid values: true, false
IxCTFileName	string	cellview		Cell Template File Name.
IxCTFoldPolicy	bool	cellview	false	Folding Policy. Valid values: true, false
IxCTGroundHeight	float	cellview	0.5	Round Rail Height.
IxCTGroundLPP	string	cellview		Ground Layer Purpose.
IxCTHonorCellHeight	bool	cellview	false	Honors Cell Height settings. Valid values: true, false
IxCTLocalGround	bool	cellview	false	Creates local ground. Valid values: true, false
IxCTLocalGroundNet	string	cellview		Local ground net.
IxCTLocalPower	bool	cellview	false	Creates local power. Valid values: true, false
IxCTLocalPowerNet	string	cellview		Local power net.
IxCTNAlignment	string	cellview	Bottom	N Row Alignment. Valid values: Top, Center, Bottom, Power, Ground
IxCTNGroundSpacing	float	cellview	0.2	Spacing of N Strip and Ground Shape.
IxCTNMOSThld	float	cellview	1	NMOS Threshold.
IxCTNMOSThldFin	int	cellview	8	NMOS Threshold Fin Number.
IxCTNPPreset	string	cellview		NP Preset.
IxCTNStripHeight	float	cellview	1	N Strip Height.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxCTNStripHeightHin	int	cellview	8	N Strip Fin Number.
IxCTOptionLayerSets	string	cellview		Optional Layer Purpose and Extension.
IxCTPAlignment	string	cellview	Top	P Row Alignment. Valid values: Top, Center, Bottom, Power, Ground
IxCTPMOSThld	float	cellview	1	PMOS Threshold.
IxCTPMOSThldFin	int	cellview	8	PMOS Threshold Fin Number.
IxCTPNPreset	string	cellview		PN Preset.
IxCTPNSpacing	float	cellview	1.5	Spacing of P Strip and N Strip.
IxCTPNSpacingFin	int	cellview	8	P N Spacing Fin Number.
IxCTPowerGroundSpacing	float	cellview	0.2	Spacing of Power and Ground Shape.
IxCTPowerHeight	float	cellview	0.5	Power Rail Height.
IxCTPowerLPP	string	cellview		Power Layer Purpose.
IxCTPPowerSpacing	float	cellview	0.2	Spacing of P Strip and Power Shape.
IxCTPStripHeight	float	cellview	1	P Strip Height.
IxCTPStripHeightFin	int	cellview	8	P Strip Fin Number.
IxCTRowAlignment	string	cellview	Bottom	Row Alignment.
IxCTRowNumber	int	cellview	2	Row Number.
IxCTRowPair	int	cellview	1	Row Pair.
IxCTRowPairPresets	bool	cellview	false	Row Pair Presets. Valid values: true, false
IxCTRowPattern	string	cellview	PN	Row Pattern. Valid values: PN, NP, PNNP, NPPN
IxCTSharePowerGround	bool	cellview	false	Share Power and Ground. Valid values: true, false
IxCTUserPattern	string	cellview		User Pattern.
IxCustomIntraSpacingValue	float	user	0	Custom intra row spacing value.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxDeleteDummy	bool	user	false	Deletes dummy objects. Valid values: true, false
IxDeviceBoundary	string	cellview	activeLayer	Device boundary. Valid values: activeLayer, instBoundary
IxDevicePanelGateHighlight	bool	user	false	Enable device Panel Gates Highlight.
IxDevicePanelGroupDummyDevices	string	user	None	Groups dummy devices in the device panel.
IxDevicePanelMasterColumnLength	int	user	100	Saves master column length in the device panel.
IxDevicePanelNameColumnLength	int	user	100	Saves name column length in the device panel.
IxDevicePanelShowParameters	bool	user	false	Shows parameters in the device panel. Valid values: true, false
IxDistanceAngleMode	string	user	orthogonal	Determines to angle mode of measuring distance. Valid values: anyAngle, diagonal, orthogonal, horizontal, vertical
IxDistanceConvertType	string	user	linearRuler	Defines the ruler type to be converted from runtime distance to database ruler. Valid values: linearRuler, circularRuler, dimensionMark
IxDistanceEdgeToEdge	bool	user	true	Determines the distance measurement method. Valid values: true, false
IxDragAndDropMOS	string	user	PnP	Decides the invoked mode when devices of drag-and-drop are all MOS devices. If the value is PnP, invokes Pick-and-place. If the value is SED, invokes SED chaining mode. Valid values: PnP, SED
IxDummyBulkNet	string	user		Net name of dummy bulk.
IxDummyCreateProcName	string	user		Dummy Create Proc Name.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxDummyFillGap	bool	user	true	Fills dummies in inner and outer gap. Valid values: true, false
IxDummyFingerNumberChaining	int	user	1	Number of fingers to be created.
IxDummyFingerNumberMatching	int	user	1	Number of fingers to be created.
IxDummyFingerNumberMode	string	user	User Defined	Specifies mode of finger numbers.
IxDummyGateLengthMode	enum	user	Adjacent Max	Specifies gate length by adjacent number or user defined value. Valid values: Adjacent, Min, User, Defined
IxDummyGateLengthUserDefValue	float	user	0.1	The user defined value of gate length for created dummies.
IxDummyGateNet	string	user	none	Net name of gate for dummy objects.
IxDummyGateWidthMode	enum	user	Adjacent Max	Specifies the gate width by adjacent number or user-defined value. Valid values: Adjacent Max, Adjacent Min, User Defined
IxDummyGateWidthUserDefValue	float	user	0.1	The user defined value of gate width for created dummies.
IxDummyLCV	lcv	user	<file path>	Dummy LCV.
IxDummyLCVDesign	lcv	cellview	<file path>	Dummy LCV.
IxDummyLCVHistory	string	cellview		Dummy LCV History.
IxDummyLeftODNet	string	user	none	Net name of left OD for dummy objects.
IxDummyLength	float	user	0.1	Length of dummy objects.
IxDummyMinusNet	string	user		Net name of dummy minus.
IxDummyModes	string	user	Normal	Mode of Add Dummy.
IxDummyNMOSGateNet	string	cellview	none	Dummy gate net name for NMOS.
IxDummyNMOSSDNet	string	cellview	none	Dummy source/drain net name for NMOS.

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxDummyODNet	string	user	none	Net name of common OD for dummy objects.
IxDummyPlusNet	string	user		Net name of dummy plus.
IxDummyPMOSGateNet	string	cellview	none	Dummy gate net name for PMOS.
IxDummyPMOSSDNet	string	cellview	none	Dummy source/drain net name for PMOS.
IxDummyResBulkNet	string	cellview		Bulk net name for dummy resistor.
IxDummyResBulkSource	string	cellview	Adjacent	Resistor bulk net name is from adjacent logic or user-defined. Valid values: true, false
IxDummyResistanceValueMode	string	user	Adjacent Max	Mode of Resistance Value.
IxDummyResistorNet	string	cellview		Net name for dummy resistor.
IxDummyResPlusMinusNet	string	cellview		Plus/Minus net name for dummy resistor.
IxDummyRightODNet	string	user	none	Net name of right OD for dummy objects.
IxDummyShareOD	bool	user	true	Specifies the od sharing for dummy MOS. Valid values: true, false
IxDummyTapRef	string	user	Top	Mode of dummy reference.
IxDummyWidth	float	user	0.1	Width of dummy objects.
IxEcFilterType	string	cellview	Name	String Filtering Type.
IxEcFitToSelected	bool	cellview	true	Fit to Selected. Valid values: true, false
IxEcIgnoreName	string	cellview	_SDL_UNSET	The name EC appends to layout during unset operation.
IxEcInstanceFilter	string	cellview	Potential match	The options turn on in instance filter. Valid values: Potential match, Matched Potential, match Matched, Potential match Ignored, Matched Ignored, Potential match Matched Ignored

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxEcoApplyPreset	bool	user	false	Apply the preset when fixing the master mismatch, parameter mismatch, and unplaced instance during the ECO update layout.
IxEcoCellTypeIgnoredNames	string	cellview	blockRing cover coverBump pad padSpacer coreSpacer coreAntenna corner via padArealO coreWellTap	List of cell types to ignore during SDL ECO if preference IxEcoIgnoreCellTypes is true.
IxEcoClearAnchors	bool	user	false	Clear anchors of nets or vias when eco updating layout.
IxEcoCreateMarker	bool	user	true	Export violations as OA markers.
IxEcoDeleteExtraDevice	bool	cellview	false	Deletes extra devices to fix unmatched devices.
IxEcoDeleteExtraDevicesWithSameBaseName	bool	cellview	false	Deletes extra devices if the base name of extra devices matches to base name of logical devices. Valid values: true, false
IxEcoFixAbuttedDummy	bool	cellview	false	Fixes abutted dummy devices when fixing the parameter mismatch. Valid values: true, false
IxEcoHierarchyMode	eco_hierarchy_mode	user	SDL	ECO hierarchy mode.
IxEcoHierarchySkip	eco_hierarchy_skip	user	none	The skip type of verify hierarchy.
IxEcoHierarchySkipCellFile	string	user		The file of skip cell list of verify hierarchy.
IxEcoHierarchySkipCells	string	user		The skip cells of verify hierarchy.
IxEcoHierarchySkipLibraries	string	user		The skip libraries of verify hierarchy.
IxEcoIgnoreCellTypes	bool	cellview	true	Enables SDL ECO to ignore the selected cell types. Valid values: true, false

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxEcoOutputFile	string	user		The output file path of verify hierarchy.
IxEcoReuseExtraDevicesWithSameBaseName	bool	cellview	false	Reuses extra devices to fix unplaced if the base name of extra devices matches to base name of logical devices. Valid values: true, false
IxEcoSyncNames	bool	cellview	true	Controls whether name-synchronization of name-mismatching instances, nets and terminals should be performed as part of ECO fix. Valid values: true, false
IxEcShowInstConnectionColumn	bool	cellview	false	Shows connection column in instance table of established correspondence. Valid values: true, false
IxEcShowInstMasterColumn	bool	cellview	true	Shows master column in the instance table of established correspondence. Valid values: true, false
IxEcShowInstNameColumn	bool	cellview	true	Shows name column in the instance table of established correspondence. Valid values: true, false
IxEcShowInstParameterColumn	bool	cellview	false	Shows parameter column in the instance table of established correspondence. Valid values: true, false
IxEcShowOnlyNetInstTerminals	string	user		Inst terminals that show only nets in the instance table of established correspondence.
IxEcTerminalFilter	string	cellview	none	The options turn on in terminal filter. Valid values: None, Ignored
IxEcUpdateMapping	bool	cellview	true	Updates to instance mapping. Valid values: true, false
IxEnableAreaEstimate	bool	user	false	Area estimate mode. Valid values: true, false

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxEqualWidth	bool	cellview	false	Enforces MOS instances of equal width to be used during chaining. Valid values: true, false
IxEquivalentNetMode	string	user	SDL	Defines the mode of net-to-net feedthrough priority handling. Valid values: SDL, ICC
IxEstablishCorrespondView	string	cellview	insts	The active object types display in panel.
IxExpandMFactor	bool		true	Specifies to expand the device with m-factor in the design browser. Valid values: true, false
IxExpandSchematicCheckOut	bool	user	true	Option to checkout required files during expand schematic.
IxExpandSchematicCopyOnly	bool	user	false	Copies schematic only without creating or updating soft macros. Valid values: true, false
IxExpandSchematicCopySchematic	bool	user	true	Copies over the symbol, schematic and constraint views to the target library. Valid values: true, false
IxExpandSchematicCopyViews	string	user		List of views to be copied during copy schematic.
IxExpandSchematicIgnoreAllButSource	bool	user	false	The option to ignore all libraries except top schematic library.
IxExpandSchematicIgnoreLibs	string	user		Ignore Libraries.
IxExpandSchematicLayoutMapFile	string	user		Layout Map File Name.
IxExpandSchematicLibTechType	string	user	none	Technology type for target library.
IxExpandSchematicLimitLogFiles	string	user	5	Number of log file backups.
IxExpandSchematicNotify	bool	user	false	Option to enable the menu when done for expand schematic.
IxExpandSchematicOverwriteCellview	bool	user	true	Overwrites schematic and symbol during expand schematic with copy. Valid values: true, false

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxExpandSchematicOverwriteLayout	bool	user	false	Overwrites an existing layout with layout from reference library. Valid values: true, false
IxExpandSchematicParamMacroNoLibOcc	bool	user	false	Option to not add occurrence library rule for parameterized soft macros.
IxExpandSchematicPlacedPriority	bool	user	false	Option to retain %PLACED in front in the Library Search List.
IxExpandSchematicSymbolMapFile	string	user		Symbol Map File Name.
IxExpandSchematicUpdateConfigSchematic	bool	user	true	Updates the existing soft macro config's top schematic during expand schematic. Valid values: true, false
IxExpandSFactor	bool	user	false	Expands S-factor during SDL initialization.
IxFingerNaming	string	user	.f	List of strings that can serve as separators for the suffix of a folded transistor name. The first string in the list has priority. Valid values: ".", ".f", ".f.", ".f.f"
IxFitTotalCount	bool	cellview	false	Fits grid size for total count. Valid values: true, false
IxFlightlineHorizontalLayer	string	cellview		Layer for horizontal flight line.
IxFlightlinePwrGnd	bool	user	false	Trigger to show the Power/Ground net between two PN-pairs. Valid values: true, false
IxFlightlineTopBottomOD	bool	user	false	Trigger to change the initial routing of flight line. Valid values: true, false
IxFlightlineTrackNum	int	user	1	Number of flight line track.
IxFlightlineType	string	user	All	Flight line type. Valid values: All, OD Gate, GateConnected
IxFlightlineVerticalLayer	string	cellview		Layer for vertical flight line.
IxFoldAllowDiscrepancy	bool	user	false	Allows discrepancy. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxFoldGateMode	bool	user	true	Specifies to fold gates by segment number or threshold.
IxFoldGateSegmentValue	int	user	2	Number of gates to be folded.
IxFoldGateThresholdfinValue	int	user	1	Number of fin to be folded.
IxFoldGateThresholdValue	float	user	1	Width of gates to be folded.
IxFoldSameGateWidth	bool	user	false	Same Gate Width. Valid values: true, false
IxFollowSchematic	bool	cellview	false	Follows schematic placement.
IxFollowSchematicOrientation	bool	cellview	false	Follows schematic orientation. Valid values: true, false
IxGateInfoGateName	bool	cellview	1	Information to be displayed in gate. Valid values: true, false
IxGateInfoGateNet	bool	cellview	1	Information to be displayed in gate. Valid values: true, false
IxGateInfoGateWidth	bool	cellview	0	Information to be displayed in gate. Valid values: true, false
IxGenerateGroupPlacementsDialogMode	bool	user	false	ilx::generateGroupPlacements command options dialog eject state Valid values: true, false
IxGenerateGuardRingsDialogMode	bool	user	true	ilx::generateGuardRings command options dialog eject state. Valid values: true, false
IxGenerateLabelPerGate	bool	cellview	false	Defines if generate device name per gate when creating label by instance. Valid values: true, false
IxGeneratePatternDialogMode	bool	user	true	ilx::generatePattern command options dialog eject state. Valid values: true, false
IxGeneratePlacementDialogMode	bool	user	true	ilx::generatePlacement command options dialog eject state. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxGRBodywidth	float	user	0	Body Width of Guard Ring.
IxGRBodywidthType	string	user	Cut Rows	Bodywidth type of Guard Ring.
IxGRBoundarySpacing	string	user	0	Boundary Spacing of Guard Ring.
IxGRFullHistory	string	user		Full history dump for Guard Ring. This preference is hidden, declaring it as deprecated.
IxGRHistory	string	user		History of Guard Ring.
IxGRIndividualWrap	bool	user	false	Individual wrap of Guard Ring.
IxGRLayerSpacing	string	user		Layer Spacing of Guard Ring.
IxGRLCV	lcv	user	<file path>	Current Guard Ring LCV.
IxGRMergeDistance	float	user	0	Merge distance of Guard Ring.
IxGRNet	string	user		Current Guard Ring Net name.
IxGRNotch	float	user	0	Notch of Guard Ring.
IxGroupPlacementsGroupName	string	user		Group for which placement catalog is to be generated.
IxGroupPlacementsIncrement	int	user	1	Increment to use for m-factor/finger/vector.
IxGroupPlacementsLengthMax	int	user	100	Maximum device length % relative to current length.
IxGroupPlacementsLengthMin	int	user	100	Minimum device length % relative to current length.
IxGroupPlacementsModifyWidthType	string	user	Mfactor	Modifies width type of the device.
IxGroupPlacementsPatternText	string	user		Pattern text string to use for generating placement catalog.
IxGroupPlacementsWbyLIgnore	bool	user	false	Ignores Width/Length checks. Valid values: true, false
IxGroupPlacementsWbyLRatioMax	int	user	100	The maximum Width/Length ratio % of the device relative to current Width/Length ratio.
IxGroupPlacementsWbyLRatioMin	int	user	100	The minimum Width/Length ratio % of the device relative to current Width/Length ratio.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxGroupPlacementsWidthMax	int	user	100	The maximum width % of the device relative to the current width.
IxGroupPlacementsWidthMin	int	user	100	The minimum width % of the device relative to the current width.
IxGroupPlacementsYAMLFile	string	user		YAML file to write the placement catalog.
IxGroupRealizedInstances	bool	cellview	false	Group Realized Objects into Created Group. Valid values: true, false
IxGroupVectorDevice	bool	cellview	false	Group Device with Bus Net. Valid values: true, false
IxGRRowNum	int	user	1	Row Number of Guard Ring.
IxGRSpacingType	string	user	Boundary	Spacing Type of Guard Ring.
IxGRStyle	string	user	Rectangle	Style of Guard Ring.
IxHonorChainConstraint	bool	user	true	Honor Chain Constraint. Valid values: true, false
IxHonorFlightlineTrack	bool	user	false	Trigger to honor flight line track num for flight line placement. Valid values: true, false
IxIgnoreNames	string	cellview	nIAction lvIsgnore IxIsgnore	Allows the user to customize the names of the device filtering properties for SDL. The properties can be set on instances to filter out specific instances, or can be set on instance master cellviews to filter out all instances of the master cellview during SDL layout generation. The default property names are nIAction, lvIsgnore, and IxIsgnore, but this Tcl preference allows the user to customize these property names. This preference affects SDL but does not affect the netlisters. The HSPICE netlister uses nIAction. The CDL netlister uses nIAction and IxIsgnore.
IxImportSDLMappingMode	replace	cellview	replace	Import SDL mapping mode.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxIncrAddPattern	string	user		Current pattern for the Add Incremental command.
IxIncrAddPatternEnable	bool	user	false	Adds devices with text pattern. Valid values: true, false
IxIncrAddPatterns	string	user		History of text patterns.
IxIncrementalPreview	bool	cellview	true	Computes preview result incrementally. Valid values: true, false
IxIncrSwapInterdigitate	bool	user	false	Swap interdigitate for added devices. Valid values: true, false
IxIndividualAlignment	string	user	Top	Individual Alignment.
IxInspectCentroidDrawAxes	bool	user	false	Draws Centroid Axes. Valid values: true, false
IxInspectEnable	bool	user	true	Enables SED Inspection. Valid values: true, false
IxInspectExtendSelection	bool	user	false	Extends Selection. Valid values: true, false
IxInspectMode	string	user	None	Specifies Inspection Mode.
IxInspectNetFilterMode	string	user	All	Net list filter.
IxInspectNetHighlightGates	bool	user	true	Highlight only Gates.
IxInspectNetHighlightSourceDrain	bool	user	true	Highlight only Source/Drain.
IxInspectPreset	string	user		Specifies Inspection Preset.
IxInstanceBoundaryCalculation	instance _bounda ry_calcul ation	cellview	BBox	Instance boundary calculation method.
IxInstanceSpacing	float	cellview	0.01	Specifies the spacing between instances in the Generate Layout and Pick & Place commands.
IxInstSpacingType	string	cellview	Microns	Instance Different Row Spacing Type.
IxInterdigitate	bool	cellview	true	Interdigitate. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxInterdigitateCrossCoupling	bool	cellview	true	Cross Coupling. Valid values: true, false
IxInterdigitateDummy	bool	cellview	false	Creates dummy devices. Valid values: true, false
IxInterdigitateInvertPattern	bool	cellview	true	Inverts patterns. Valid values: true, false
IxInterdigitateNumIslands	int	cellview	1	Number of Islands.
IxInterdigitatePattern	interdigitate_pattern	cellview	ABBA	Interdigitate Pattern. Valid values: Any integer in the [0 2] range
IxInterRowLPP	string	cellview		Inter Row LPP.
IxInterRowNumFins	string	cellview	8	Row spacing in terms of fin number between two rows.
IxInterRowSpacing	string	cellview	1	Inter Row Spacing.
IxIntraRowBoundary	string	cellview	polyLayer	Boundary for intra row spacing in terms of poly number.
IxIntraRowLPP	string	cellview		Intra Row LPP.
IxIntraRowNumPolys	int	cellview	1	X spacing in terms of poly number.
IxKeepLayoutPos	bool	cellview	true	Keeps previous layout position when realizing devices. Valid values: true, false
IxKeepLayoutShape	bool	cellview	false	Keeps layout existing shape. Valid values: true, false
IxLayoutFollowSchematicHierarchy	bool	user	false	When descending or returning to the schematic, SDL descends or returns to the corresponding hierarchy in the layout. Valid values: true, false
IxLayoutHierarchyChangeCellName	string	user	%C_%N	Layout Hierarchy Change cell name convention.
IxLayoutHierarchyChangeDelimiter	string	user	_	Uses when constructing instance names in Move Instance.
IxLayoutHierarchyChangeInstanceMapFile	string	user	instance.map	File name to emit instance map file.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxLayoutHierarchyChangeLibrary	string	user		Library where SDL layout hierarchy change cells must be created.
IxLayoutHierarchyChangeMoveDownPrefix	string	user	LHCMD	Prefix to add to instance name of instance moved down into another instance.
IxLayoutHierarchyChangeOverrideInstanceMap	bool	user	true	Overwrites an existing instance map file. Valid values: true, false
IxLayoutHierarchyChangeOverrideSchematic	bool	user	true	Overwrites updated schematic cellviews. Valid values: true, false
IxLayoutHierarchyChangeSchematicLibrary	string	user		Library where updated schematic from overlay must be generated.
IxLeftMostMOSPort	string	cellview	source	Determines the left-most MOS port in the initial placement. Valid values: source, drain
IxLeftRightPinHeight	float	cellview	0.35	Left/Right Pin Height.
IxLeftRightPinLPP	string	cellview		Left/Right Pin LPP.
IxLeftRightPinPitch	float	cellview	0.7	Left/Right Pin Pitch.
IxLeftRightPinWidth	float	cellview	0.35	Left/Right Pin Width.
IxLHCAutoCreateMakeNode	bool	user	false	Creates Make Node during LE Make Cellview. Valid values: true, false
IxLHCCellName	string	user	%C_%N	Layout Hierarchy Change cell name convention.
IxLHCDelimiter	string	user	_	Specifies delimiter to use when constructing instance names in Move Instance.
IxLHCInstanceMapFile	string	user	instance.map	Specifies file name to emit instance map file.
IxLHCLibrary	string	user		Specifies the library where SDL layout hierarchy change cells must be created.
IxLHCMakeNodeCellName	string	user	MN_%L_%N	Layout Hierarchy Change Make Node cell name convention.

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxLHCMakeNodeCreateVector	bool	user	true	Preference to create vector instance if all instances are vector instances with identical range.
IxLHCMakeNodeECOOverwrite	bool	user	false	Preference to overwrite existing Make Node cellviews.
IxLHCMakeNodeEnableReuse	bool	user	false	Enables reuse of pre-existing layout.
IxLHCMakeNodeInstName	string	user	LHCMN_I%N	Prefix to add to instance name of instance created for Make Node.
IxLHCMakeNodeInstName	string	user	LHCMN_I	Prefix to add to instance name of instance created for Make Node.
IxLHCMoveDownPrefix	string	user	LHCMD	Specifies prefix to add to instance name of instance moved down into another instance.
IxLHCOverwriteInstanceMap	bool	user	true	Overwrites an existing instance map file. Valid values: true, false
IxLHCOverwriteSchematic	bool	user	true	Overwrites updated schematic cellviews. Valid values: true, false
IxLHCReuse	bool	user	true	Reuses previously created layout hierarchy change cells. Valid values: true, false
IxLHCSameMaster	bool	user	false	Moves up selected 1x instances from all 0x instances of same master. Valid values: true, false
IxLHCSchematicLibrary	string	user		Specifies the library where updated schematic from overlay must be generated.
IxLHCSpecifyInstanceName	bool	user	false	Enables specification of a custom instance name.
IxLHCSpecifyTargetCell	bool	user	false	Enables specification of a custom target cell.
IxLHCSpecifyTargetLibrary	bool	user	false	Enables specification of a different target library. Valid values: true, false
IxLibrarySearchList	string	cellview	%PLACED	Library Search List.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxLogicalLCV	lcv	user	//	Logical View LCV.
IxMatchedFolding	bool	cellview	false	Matched folding. Valid values: true, false
IxMCUpdateMapping	bool	user	true	Update instance mapping when setting correspondence. Valid values: true, false
IxMergeTransistors	bool	cellview	true	Combines fingers, m-factors into a single transistor. Valid values: true, false
IxMFactorNames	string	cellview	m M	MFactor parameter names list.
IxMFactorNaming	string	user	.m	MFactor suffix.
IxMissingConstraintFixMode	enConstraintFixAction	cellview	Create	The default action for fixing missing logic constraints in layout.
IxMLParasiticEstimationDir	string	user	<file path>	The working directory for SDL ML project.
IxModifyDummyLCV	lcv	user	<file path>	Value of dummy LCV.
IxModifyDummyLCVHistory	string	cellview		Dummy LCV History.
IxModifyPresetName	string	user		Modifies preset.
IxMoveInstanceDialogMode	bool	user	false	ilx::moveInstance command options dialog eject state. Valid values: true, false
IxMoveKeepAlign	bool	user	false	Moves gates with alignment gates. Valid values: true, false
IxNetlistFilter	int	cellview	All	Netlist filter. Valid values: All, Potential Single Match, Potential Multiple Match, Unmatched
IxNetlistRefreshMode	string	user	immediate	The netlist refresh mode to control SDL re-initialization.
IxNMOSAlignment	string	cellview	bottom	NMOS Alignment.
IxNMOSPODENet	string	cellview		PODE gate net name for NMOS.
IxOpenConfig	bool	cellview	false	Open Configuration. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxOpenedLayoutFrameSize	string	user		Specifies the size of opened layout frame. When the value is empty, the size is calculated by SDL.
IxOpenedSchematicFrameSize	string	user		Specifies the size of opened schematic frame. When the value is empty, the size is calculated by SDL.
IxOpenInNewFrames	bool	user	false	Controls whether schematic and layout view are opened in new frames during starting SDL. Valid values: true, false
IxOpenMarkerBrowserOnError	string	cellview	On Errors	Opens the marker browser on verify errors.
IxOpenSDLWithSameCell	bool	user	false	Starts SDL with same cell without SDL configuration. Valid values: true, false
IxOptimizePatternOD	bool	cellview	1	Optimizes OD sharing after applying the pattern. Valid values: true, false
IxOverridePinPlacement	bool	cellview	true	Enables Pin Placement Overrides. Valid values: true, false
IxParamPrecision	string	cellview	0.001	Parameter Compare Precision.
IxParasiticEstimationModel	string	user		Parasitic estimation model for SDL ML project.
IxPatternFile	string	user	none	The file name of user-defined pattern file.
IxPatternPanelFilterMode	pattern_ display_ mode	user		Filter settings for pattern panel. Valid values: Any integer in the [0 2] range
IxPeripheralBottomNum	int	user	1	Number of bottom peripheral dummies to be created.
IxPeripheralDummyGateLength	string	user		The length values of peripheral dummy.
IxPeripheralDummyGateLengthModeMDC	string	user	Adjacent Max	Specifies mode of peripheral dummy length.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxPeripheralDummyGateLengthModeSDC	string	user	Adjacent Max	Specifies mode of peripheral dummy length.
IxPeripheralDummyGateWidth	string	user		The width values of peripheral dummy.
IxPeripheralDummyGateWidthModeMDC	string	user	Adjacent Max	Specifies mode of peripheral dummy width.
IxPeripheralDummyGateWidthModeSDC	string	user	Adjacent Max	Specifies mode of peripheral dummy width.
IxPeripheralFingerAdjacent	bool	user	false	Defines finger number of peripheral dummies by referring to adjacent method. Valid values: true, false
IxPeripheralFingerOne	bool	user	true	Defines finger number of peripheral dummies by the value of one. Valid values: true, false
IxPeripheralLeftNum	int	user	1	Number of left peripheral dummies to be created.
IxPeripheralLRBT	bool	user	false	Adds peripheral dummies by left, right, bottom, or top method. Valid values: true, false
IxPeripheralRightNum	int	user	1	Number of right peripheral dummies to be created.
IxPeripheralRing	bool	user	true	Adds peripheral dummies by ring method. Valid values: true, false
IxPeripheralRingNum	int	user	1	Number of peripheral ring dummies to be created.
IxPeripheralTopNum	int	user	1	Number of top peripheral dummies to be created.
IxPGAutoWildcard	bool	user	true	Auto Wildcard for Pattern of TextPattern style. Valid values: true, false
IxPGBalancedPattern	string	user		Balanced Pattern of Pattern Generator.
IxPGDeviceSequence	string	user		Device Sequence of Pattern Generator.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxPGDirection	string	user	Horizontal	Direction of Pattern Generator. Valid values: Vertical, Horizontal
IxPGInsertPattern	string	user		Input Pattern of TextPattern style.
IxPGMaxColumnNum	int	user	0	Maximal Columns Number of TextPattern Style.
IxPGMaxColumnState	bool	user	false	State of Maximal Columns Number of TextPattern Style. Valid values: true, false
IxPGPatternStyle	string	user	Interdigitate	Pattern Style of Pattern Generator. Valid values: Interdigitate, CommonCentroid, CentralSymmetrical, DeviceGroup, TextPattern
IxPGPatternTextHistory	string	user		History of insert patterns.
IxPGRepeatCount	int	user	1	Repeat Count of Pattern Generator.
IxPGRowNumber	string	user	1	Row Number of Pattern Generator.
IxPGRowNumberState	bool	user	false	State of Row Number of BalancedPattern Style. Valid values: true, false
IxPGSeriesChainDirection	string	user	Horizontal	Direction of Series Chain Style of Pattern Generator.
IxPGSeriesChainType	string	user	S1	Type of series chain.
IxPGStartDevice	string	user		Balanced Pattern of Pattern Generator.
IxPGStartingPattern	string	user		Starting Pattern of Pattern Generator.
IxPGStartOffset	int	user	0	Start Offset of Pattern Generator.
IxPGUnitPattern	string	user		Unit Pattern of Pattern Generator.
IxPGXExtend	int	user	1	X-extend for Text Pattern.
IxPGYExtend	int	user	1	Y-extend for Text Pattern.
IxPhysicalLCV	lcv	user	//	Physical View LCV.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxPhysicalLibrary	string	user		Physical library for SDL.
IxPickAndPlaceDialogMode	bool	user	true	Determines if dialog or toolbar is used in ilx::pickAndPlace .
IxPinHeight	float	cellview	0.35	Pin Height.
IxPinLocation	pin_location	cellview	Boundary Side	Defines the pins location of generate layout
IxPinLPP	string	cellview		Pin LPP.
IxPinOverrideDefault	bool	cellview	true	Enable Pin Override Default
IxPinPitch	float	cellview	0.7	Pin Pitch.
IxPinWidth	float	cellview	0.35	Pin Width.
IxPlacementTemplate	string	cellview	Place Like Schematic	Placement Template.
IxPlaceSelectionMode	string	cellview	Single Schematic Bit	Controls if the pick and place command places a single object or all objects. Valid values: Single Layout, Single Schematic Bit, Single Schematic, All Selected
IxPLEAdditionalParasiticDevices	string	cellview		The additional parasitic devices in schematic.
IxPLEAutoDetectSchStopCells	bool	cellview	true	Specifies whether to detect schematic stop cells automatically.
IxPLEAutoPrefs	bool	user	true	Automatically load prefs.xml.
IxPLECellListFileForSDLVerifySkip	string	cellview		Selected cell list file for SDL Verify Skip.
IxPLECellsForSDLVerifySkip	string	cellview		Selected cells for SDL Verify Skip.
IxPLECheckBulkConnection	bool	cellview	true	Specifies whether to check bulk connection.
IxPLECoupleToGround	bool	cellview	false	Specifies whether coupling capacitances are retained or lumped to ground.
IxPLEEnableLPEJob	bool	cellview	true	LPE job enabled status in the PLE dialog box.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxPLEEnableLVSJob	bool	cellview	true	LVS job enabled status in the PLE dialog box.
IxPLEEnablePLEJob	bool	cellview	true	PLE job enabled status in the PLE dialog box.
IxPLEEnableSDLVerifyJob	bool	cellview	false	SDL Verify job enabled status in the PLE dialog box.
IxPLEExtractionType	string	cellview	RC	Specifies the type of extraction.
IxPLEFixSubstrateConn	bool	cellview	true	The flag whether to fix substrate connection.
IxPLEHierarchicalSeparator	string	cellview		The hierarchical separator.
IxPLEHSPICEViewSearchList	string	cellview	hspice hspiceD cmos.sch cmos_sch schematic veriloga symbol	HSPICE View search list passed to PLE.
IxPLEHSPICEViewStopList	string	cellview	hspice hspiceD veriloga	HSPICE View stop list passed to PLE.
IxPLEIgnoreParasiticDevices	bool	cellview	false	Ignore estimated parasitic devices in schematic.
IxPLEInFlow	bool	cellview	false	Partial layout extraction flow.
IxPLELayCellView	lcv	cellview	<file path>	Layout cell view.
IxPLELibrariesForSDLVerifySkip	string	cellview		Selected libraries for SDL Verify Skip.
IxPLENetlisterFormat	string	cellview	HSPICE	The netlister format for partial layout extraction.
IxPLEOpenParasiticView	bool	cellview	true	Specifies whether to open parasitic view.
IxPLEOutputSPF	string	cellview		Output SPF file for parasitic extraction.
IxPLEPowerExtract	string	cellview	Yes	Specifies the extraction of power nets.
IxPLEPowerNets	string	cellview		Specifies power nets for special treatment during extraction.

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxPLERefCellView	lcv	cellview	<file path>	Reference cell view.
IxPLERefSPF	string	cellview		Reference SPF file for parasitic extraction.
IxPLERunDirBase	string	user	<file path>	The base run directory of parasitic extraction.
IxPLERunDirNameTemplate	string	user	%c.%t.%j	Template for customizing run directory name
IxPLERunInBackground	bool	cellview	false	Specifies whether to run PLE in background mode.
IxPLESchStopCells	string	cellview		Schematic Stop Cells.
IxPLESDLVerifyCreateMarker	bool	cellview	true	Specifies whether SDL Verify create marker.
IxPLESDLVerifyLaunchDebugger	bool	cellview	true	Specifies whether to open Error Viewer.
IxPLESDLVerifyOutputFile	string	cellview		Selected output file for SDL Verify.
IxPLESDLVerifyPassOnly	bool	cellview	true	Specifies whether SDL Verify must pass.
IxPLESDLVerifySkip	string	cellview	None	Skip type of SDL Verify.
IxPLESDLVerifyViewOutput	bool	cellview	true	Specifies whether to open text viewer to view output.
IxPLESelectCellListFileSkipCells	string	cellview		Selected cell list file for Skip Cells.
IxPLESelectCellListSkipCells	string	cellview		Selected cells for Skip Cells.
IxPLESelectCellSourceSkipCells	string	cellview	None	Source of Skip Cells.
IxPLESelectNetListFileTrimParasitics	string	cellview		Selected net list file to trim parasitics.
IxPLESelectNetListTrimParasitics	string	cellview		Selected net list to trim parasitics.
IxPLESelectNetSourceTrimParasitics	string	cellview	None	Source of Nets to trim parasitics.
IxPLESeparateSchDevices	bool	user	false	Specifies whether to sperate schematic devices from DSPF file.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxPLETextLPPMapFile	string	cellview		The map file path to be used in Label creation.
IxPLETextNetPrefix	string	user		Specifies the prefix for untexted nets.
IxPLEVirtualConnection	bool	cellview	false	The flag indicates whether to add virtual connection for unrouted nets or not.
IxPMOSAlignment	string	cellview	top	PMOS alignment.
IxPMOSPODENet	string	cellview		PODE gate net name for PMOS.
IxPodeNet	string	cellview		Net name for PODE gates.
IxPostPasteProcName	string	cellview		The name of post-paste procedure.
IxPostProcName	string	cellview	none	Post-process procedure name.
IxPostProcUsednames	string	cellview		The name list of post-process procedure that ever used.
IxPreserveExistingBoundary	bool	cellview	true	Determines if existing PR / Snap boundaries should be preserved during Ix::showExpandSchematic , if IxCreateBoundary is false. Valid values: true, false
IxPreserveExistingInstances	bool	cellview	true	Determines if existing instances should be preserved during Ix::showExpandSchematic , if IxCreateInstances is false. Valid values: true, false
IxPreserveExistingPins	bool	cellview	true	Determines if existing pins / terms / shapes should be preserved during Ix::showExpandSchematic , if IxCreatePins is false. Valid values: true, false
IxPresetLCV	lcv	user	<file path>	Preset LCV.
IxPreviewAutoUpdate	bool	cellview	false	Refresh preview assistant automatically after symbol editing. Valid values: true, false
IxPreviewContextAware	bool	cellview	false	Preview context aware. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxPreviewContextFinfet	string	cellview	wideAlignm ent	Preview context for finfet. Valid values: wideAlignment, layoutContext
IxPreviewFinBoundary	string	cellview		Preview fin boundary.
IxPreviewInDesign	bool	cellview	false	Preview In-Design. Valid values: true, false
IxPropagatePODNet	bool	cellview	true	Propagate nets from logic gates. Valid values: true, false
IxPseudoParallelModeRestriction	string	cellview	none	Pseudo Parallel Mode Restriction
IxReassignNetExtend	bool	user	true	Extend to connected device. Valid values: true, false
IxReassignNetPattern	string	user	S1	Pattern of reassign net. Valid values: S1, S2, S3, S4, S5, S6, S7, S8, Z1, Z2, Z3, Z4, Z5, Z6, Z7, Z8
IxReassignNetShortConnect	bool	user	true	Shortest connection between rows. Valid values: true, false
IxReassignNetStartNet	string	user	Plus	Starting net of reassign net. Valid values: Plus, Minus
IxReassignNetTerminal	string	user	Top	Starting net of reassign net. Valid values: Top, Bottom
IxRechainAppendRight	bool	user	true	Specifies to append rechain objects to right or left. Valid values: true, false
IxReconfigureDeviceFixField	sed_rec onfig_de vice_fix_ field	user	instanceCo unt	Fix Field of Reconfigure Device. Valid values: Any integer in the [0 2] range
IxRelativeSizeSymbol	bool	cellview	false	Relative size symbol. Valid values: true, false
IxReplaceExistingObjects	bool	cellview	false	Controls if the pick and place command replaces already placed objects in the target layout design. Valid values: true, false
IxReportTriggeredCallbacks	bool	user	false	Reports the triggered callbacks.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxResistorDirection	string	user	auto	Resistor Direction.
IxResistorTolerance	int	user	0	Resistor remainder tolerance.
IxReversePropMappingOrder	bool	user	false	Reverses the order of parameter mapping. Valid values: true, false
IxRowOrientation	string	user	None	Row Orientation. Valid values: None, Flipped Odd, Flipped Even, Flipped
IxSameRowSpacingType	string	cellview	Microns	Same Row Spacing Type.
IxSaveAsCopyConfig	bool	user	false	Copies config view when saving layout view to another cell in SDL. Valid values: true, false
IxSaveAsCopySchematic	bool	user	false	Copies schematic view when saving layout view to another cell in SDL. Valid values: true, false
IxSchematicAnnotationStyle	<enAnnotationStyleType>	cellview	Marker	Determines the style for schematic annotation. Valid values: Marker, Shadow
IxSchematicDimming	bool	cellview	false	Dims the schematic to the deSurroundBrightness value when routing in the layout, and enables visualization of which schematic wire is being routed. Valid values: true, false
IxSchematicInstanceRegionSelection	bool	user	false	Controls instance selection in region based filtering in schematic. Valid values: true, false
IxSchematicPinRegionSelection	bool	user	false	Controls pin selection in region based filtering in schematic. Valid values: true, false
IxSdlBrowserColorStatus	string	user	Unrealized	Color which status for object at SDL browser. Valid values: Unrealized, Realized

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxSdlBrowserDisplayDummy	bool	user	true	Displays the dummy group at SDL browser. Valid values: true, false
IxSdlBrowserDisplayExtra	bool	user	true	Displays the extra group at SDL browser. Valid values: true, false
IxSdlBrowserDisplayIgnored	bool	user	false	Displays the extra group at SDL browser. Valid values: true, false
IxSdlBrowserInstColumnOrder	string	user		Specifies the order of columns.
IxSdlBrowserMasterName	string	user	cell	Specifies browser master name format. Valid values: Cell, Cell/View, Lib/Cell/View
IxSdlBrowserNetColumnOrder	string	user		Specifies the order of columns.
IxSdlBrowserShowInstLayoutMasterColumn	bool	user	false	Displays Insts' Layout Master column at SDL browser. Valid values: true, false
IxSdlBrowserShowInstMasterColumn	bool	user	true	Displays Insts' Master column at SDL browser.
IxSdlBrowserShowInstMismatchColumn	bool	user	true	Displays Insts' Mismatch column at SDL browser.
IxSdlBrowserShowInstParameterColumn	bool	user	true	Displays Insts' Parameter column at SDL browser.
IxSdlBrowserShowParameterTip	string	user		Specifies the content of the parameter tooltip in the SDL browser.
IxSdlBrowserUnrealizedDarker	int	user	130	Darker background color for unrealized node at SDL browser.
IxSdlGroupsView	string	context	all	Sub-view to use in the Sdl Groups tab of the Sdl browser..
IxSdlInstsView	string	context	all	Sub-view to use the Sdl Instances tab of the Sdl browser.
IxSDLMappingFilePath	string	cellview	<file path>	Defines the path for SDL mapping import/export.
IxSdlNetsView	string	context	all	Sub-view to use the Sdl Nets tab of the Sdl browser.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxSDLPostProcName	string	cellview		The name of SDL post procedure.
IxSdlTermsView	string	context	all	Sub-view to use in the Sdl Terms tab of the Sdl browser..
IxSEDAncorActiveDevices	bool	user	true	Anchors the position of <code>figGroup</code> to ensure active devices do not move. Valid values: true, false
IxSEDAutoDeleteConstraint	bool	cellview	true	Auto Delete Constraint. Valid values: true, false
IxSEDCrossHighlightNet	bool	user	false	Always cross highlight net.
IxSEDCrossHighlightSymbol	bool	cellview	false	Always cross highlight symbol. Valid values: true, false
IxSEDCrossSelect	bool	cellview	false	Cross select in SED. Valid values: true, false
IxSEDHighlightPreview	bool	cellview	true	Highlights preview instance. Valid values: true, false
IxSEDHighlightPreviewShadow	bool	cellview	true	Highlights preview instance in shadow mode. Valid values: true, false
IxSEDIgnoreModifiedDevicePlacement	bool	user	false	Ignores placement on device size modification. Valid values: true, false
IxSEDInstFlipSnapping	bool	user	true	Allows to flip instance for snapping. Valid values: true, false
IxSEDInterdigitate	bool	cellview	false	Interdigitate in SED
IxSEDPartialSelLayout	bool	cellview	false	Partial select devices in the constraint group. Valid values: true, false
IxSEDPenaltyParams	string	user		Penalty parameters to pass to penalty procedure.
IxSEDPenaltyProcName	string	user		Penalty procedure to get number of penalty tiles and correction value.

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxSEDSortDeviceByGateNet	bool	user	false	Sorts devices by gate net on the device panel. Valid values: true, false
IxSEDTextPatternCenterAlign	bool	user	true	Aligns TextPattern result by center. Valid values: true, false
IxSEDTextpatternUnitExpand	bool	user	true	Expands Textpattern by Unit. Valid values: true, false
IxSelectInstancesOnErrors	bool	cellview	false	Selects 0x Instances on Initialization Errors. Valid values: true, false
IxSelectionOrderPlace	bool	cellview	false	Specifies if devices are placed on selection order. Valid values: true, false
IxSFactorNames	string	cellview	s	SFactor parameter names list.
IxShortNames	string	cellview	IxRemoveDevice	Ignores instance and shorts specified terminals property names.
IxShowDummyFullMaster	bool	user	true	Shows dummy full master. Valid values: true, false
IxShowIgnored	bool	cellview	false	Shows ignored physical instances in the netlist view. Valid values: true, false
IxShowInstanceAnnotation	string	cellview	None	Controls the highlight annotation of schematic instances when they are placed in the layout. Valid values: None, Partial, Detailed
IxShowInstances	bool	cellview	true	Shows instances. Valid values: true, false
IxShowInstTermAnnotation	enAnnotationType	cellview	none	Turns on/off the annotation of schematic instance terminals when they are placed in the layout. Valid values are: none, partial or detailed
IxShowLibraryCellView	bool	cellview	true	Shows the bound library/cell/view in the netlist view. Valid values: true, false

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxShowLogicalSourceError	string	user	dialog	Outputs logical source error message in dialog or console.
IxShowPatternFile	bool	user	false	Shows pattern source file on the pattern panel. Valid values: true, false
IxShowPinAnnotation	string	cellview	None	Controls the highlight annotation of schematic pins when they are placed in the layout. Valid values: None, Partial, Detailed
IxShowPresetMessage	none	user	none	Shows preset message in console.
IxShowPreviewAssistant	bool	cellview	false	Shows preview assistant based on workspace. Valid values: true, false
IxShowTerminals	bool	cellview	true	Shows terminals. Valid values: true, false
IxShowWireAnnotation	string	cellview	None	Controls the highlight annotation of schematic wires when they are placed in the layout. Valid values: None, Partial, Detailed
IxSignalAwareChaining	bool	cellview	false	Signal aware device chaining. Controls the sharing of diffusion on the signal rather than the power net. Valid values: true, false
IxSingleRowPlacement	bool	cellview	false	Specifies if devices are placed on a single row.
IxSkipAutoEnableCE	bool	user	false	Specifies to skip auto enable Connectivity Engine during SDL initialization. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
lxSoftMacroCellName	string	user	%C_%N	Generates file names for soft-macros during Expand Schematic and Create Soft Macro. Valid combinations are %L for library name, %C for cell name, %N for running index, and %F for effective parameter values sorted on parameter names. Valid values: %L_%C_%F_%N %C_%L_%F_%N %F_%L_%C_%N %N_%L_%F_%C %L_%C_%N_%F %C_%L_%N_%F %F_%L_%N_%C %N_%L_%C_%F %L_%F_%C_%N %C_%F_%L_%N %F_%C_%L_%N %N_%F_%L_%C %L_%F_%N_%C %C_%F_%N_%L %F_%C_%N_%L %N_%F_%C_%L %L_%N_%F_%C %C_%N_%F_%L %F_%N_%C_%L %N_%C_%F_%L %L_%N_%C_%F %C_%N_%L_%F %F_%N_%L_%C %N_%C_%L_%F %L_%C_%N %C_%L_%N %N_%L_%C %C_%F_%N %F_%C_%N %N_%F_%C %C_%N_%F %F_%N_%C %N_%C_%F %L_%N_%C %C_%N_%L %N_%C_%L %C_%N %N_%C

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxSoftMacroCellNameReplaceDotWithD	bool	user	false	Option to replace . with D in parameterized soft macro cell names.
IxSoftMacroSchematicLibrary	bool	user	true	Creates soft macros in source schematic libraries. Valid values: true, false
IxSpacingType	string	cellview	Fins	Spacing type. Valid values: Microns, Fins
IxSpecifyBoundaryLPP	bool	cellview	false	Specify Boundary LPP.
IxSpecifyConfig	bool	cellview	true	Specifies if explicitly defining config cellview when launching SDL from the schematic. Valid values: true, false
IxSplit	user	cellview	true	Auto Split Fingered Devices. Valid values: true, false
IxSplitDevicePattern	string	cellview	1	Split device pattern.
IxSpreadRechainDevices	bool	user	true	Spread Devices to Different Rows when Rechain. Valid values: true, false
IxStopLevel	int	cellview		The level drawing loop stops when the preview assistant is enabled.
IxSwapTermSingleSegmentEnable	bool	user	true	Enables Single Segment swap term. Valid values: true, false
IxSyncWithLogic	bool	cellview	false	Synchronize with logic. Valid values: true, false
IxTemplateManagerAllowOverlapMatch	bool	user	false	Finds overlaps. Valid values: true, false
IxTemplateManagerApplyPreset	bool	user	true	Applies preset during realization. Valid values: true, false
IxTemplateManagerAutoDeleteConstraint	bool	user	false	Creates fig group for realized transistors. Valid values: true, false
IxTemplateManagerBindExistingLayout	bool	user	true	Bind existing layout with template. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxTemplateManagerConfigFile	string	cellview		Template Manager config file name.
IxTemplateManagerConfigTargetHeight	string	cellview		Template Manager config target height.
IxTemplateManagerCreateTemplateExtractRp	bool	user	true	Extracts RP when creating template. Valid values: true, false
IxTemplateManagerCreateTemplateLibraryName	string	user		Library name in Create Template dialog box.
IxTemplateManagerCrossShadow	bool	user	false	Template Manager cross shadow feature. Valid values: true, false
IxTemplateManagerFilteringDiffOfMatchingDeviceCountCheckbox	bool	user	true	Static filtering item: Difference of Matching Device Count. Valid values: true, false
IxTemplateManagerFilteringDiffOfMatchingDeviceCountValueSelector	int	user	0	Static filtering. Value Selector: Diff Rate.
IxTemplateManagerFilteringFingerNum	bool	user	true	Static filtering instance parameter: No. of Finger. Valid values: true, false
IxTemplateManagerFilteringGateLength	bool	user	true	Static filtering instance parameter: Gate Length. Valid values: true, false
IxTemplateManagerFilteringGateWidth	bool	user	true	Static filtering instance parameter: Gate Width. Valid values: true, false
IxTemplateManagerFilteringMatchingRateCheckbox	bool	user	true	Static filtering item: Matching Rate of Instance Parameters. Valid values: true, false
IxTemplateManagerFilteringMatchingWithTotalM	bool	user	true	Static filtering item: Total Multiplicity. Valid values: true, false
IxTemplateManagerFilteringMatchingRateValueSelector	float	user	100	Static filtering Value Selector: Match Rate.
IxTemplateManagerFilteringMFactor	bool	user	true	Static filtering instance parameter: Multiplicity. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxTemplateManagerFilteringStaticTemplateOnly	bool	user	false	Searches static template only. Valid values: true, false
IxTemplateManagerFlippedMostType	bool	user	false	MOS flipping. Valid values: true, false
IxTemplateManagerLibraryName	string	user		Library name in Template Manager.
IxTemplateManagerMultipleSolutions	bool	user	false	Realizes with relax template cell.
IxTemplateManagerOnlyShowMatchCell	bool	user	true	Only shows match cell in cell name in the Create Template dialog box. Valid values: true, false
IxTemplateManagerPostFilterMatches	bool	user	true	Finds overlaps. Valid values: true, false
IxTemplateManagerPresetCell	string	user		Cell name to provide in the template preset list.
IxTemplateManagerRealizeToSpecifiedPoint	bool	user	false	Realizes to specified point. Valid values: true, false
IxTemplateManagerReferencePoint	unknown	user	0 0	Reference Point.
IxTemplateManagerRefineConstraint	bool	user	true	Refines constraint during template creation. Valid values: true, false
IxTemplateManagerRp	bool	user	false	Enables Relative Placement Section. Valid values: true, false
IxTemplateManagerRpGlobalSpacing	int	user	0	RP global spacing. Valid values: Any integer in the [0 50000] range.
IxTemplateManagerRpPackingDirection	string	user	Bottom Left	RP packing direction.
IxTemplateManagerRpSpacing	bool	user	true	RP spacing. Valid values: true, false
IxTemplateManagerSDLCompliant	bool	user	true	Only Find SDL Compliant Template. Valid values: true, false

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxTemplateManagerShowLCSelector	bool	user	true	Shows lib cell selector section. Valid values: true, false
IxTemplateManagerShowMapping	bool	user	false	Shows mapping table section. Valid values: true, false
IxTemplateManagerShowPresetSelector	bool	user	true	Shows preset selector section. Valid values: true, false
IxTemplateManagerShowTemplateDescription	bool	user	false	Shows template description section. Valid values: true, false
IxTemplateManagerShowTemplateLayout	bool	user	true	Shows template layout section. Valid values: true, false
IxTemplateManagerShowTemplateSchematic	bool	user	true	Shows template schematic section. Valid values: true, false
IxTemplateMaxNumResults	int	user	4	Max results for multiple solutions.
IxTemplatePostConstraintProcName	string	cellview	::IxTemplateManager::forEachConstraintProc	The post procedure name for template constraint creation.
IxTemplatePostRealizeProcName	string	cellview	::IxTemplateManager::forEachRealizationProc	The post procedure name for template realization.
IxTemplateSearchReturnConnectedABC	bool	user	false	Return connected ABC matches.
IxTemplateUsePostConstraintProc	bool	cellview	false	To use post procedure for template constraint creation.
IxTemplateUsePostRealizeProc	bool	cellview	false	To use post procedure for template realization.
IxTemplateUseRealizerContextSetting	bool	user	false	Grab context settings from each realizer.
IxTextPinLPPMapFile	string	cellview		Defines the map file path to be used in the Label creation.
IxTextPinLPPMethod	bool	cellview	Specify	Defines the text LPP method to be used in the Label creation.

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
lxtTextSpecifyPinLPP	bool	cellview	false	LPP selector mode for placed labels in the Create Label command.
lxtToolSearchList	string	cellview	sdl	Tool Search List.
lxtTopBottomPinHeight	float	cellview	0.35	Top/Bottom Pin Height.
lxtTopBottomPinLPP	string	cellview		Top/Bottom Pin LPP.
lxtTopBottomPinPitch	float	cellview	0.7	Top/Bottom Pin Pitch.
lxtTopBottomPinWidth	float	cellview	0.35	Top/Bottom Pin Width.
lxtTriggerCDFCallbackParams	string	user	Auto	Specifies the frequency for triggering the parameter callback defined in <code>lxCDFCallbackParams</code> . Valid value: Auto, Never, or Always.
lxtUniquifyConnectDefConflicts	bool	user	true	Creates virtual terminals for conflicting unresolved assignments. Valid values: true, false
lxtUnmatchedConstraintFixMode	enConst raintFixA ction	cellview	Skip	The default action for fixing unmatched layout constraints.
lxtUpdateDummyGroupSameNet	bool	cellview	false	Group dummies which have same net connection.
lxtUpdateDummyIgnoreFullDummy	bool	user	false	Specifies if ignore full dummy.
lxtUpdateDummyNotUseProtected Master	bool	cellview	false	Specifies to not use the master of active device to update the dummy in the schematic.
lxtUpdateDummyPreProcName	string	cellview		The name of update dummy pre procedure.
lxtUpdateDummyToOriginal	bool	user	false	Updates dummy to original schematic. Valid values: true, false
lxtUpdateLayoutParamFile	string	user		The file with layout parameters to be updated.
lxtUpdateSchematicApplyTermCon nectionMapping	bool	cellview	false	Specifies if applying term connection mapping. Valid values: true, false

Table 152 Schematic Driven Layout Preferences (Continued)

Name	Type	Scope	Default	Description
IxUpdateSchematicAutoOpen	bool	cellview	true	Specifies if auto opening update dummy schematic. Valid values: true, false
IxUpdateSchematicDummyDevices	bool	cellview	true	Updates dummy devices. Valid values: true, false
IxUpdateSchematicExtraDevices	bool	cellview	true	Updates extra devices. Valid values: true, false
IxUpdateSchematicExtraPrefixName	string	cellview	IxInst_	Specifies extra instance prefix name.
IxUpdateSchematicFitToSelected	bool	cellview	true	Fits to the selected object. Valid values: true, false
IxUpdateSchematicGroupExtraPrefixName	string	cellview	IxExtraGroup_	Group prefix name of extra devices.
IxUpdateSchematicGroupingDummy	bool	cellview	true	Specifies if grouping dummy. Valid values: true, false
IxUpdateSchematicGroupingExtra	bool	cellview	false	Specifies if grouping extra. Valid values: true, false
IxUpdateSchematicGroupPrefixName	string	cellview	IxDummyGroup_	Group prefix name.
IxUpdateSchematicIgnoreFullDummy	bool	cellview	false	Specifies if ignoring full dummy. Valid values: true, false
IxUpdateSchematicNetLabelHeight	int	cellview	10	Specifies net label height.
IxUpdateSchematicParameters	bool	cellview	true	Updates parameters. Valid values: true, false
IxUpdateSchematicRenameExtraInstances	bool	cellview	false	Back-annotates extra instances. Valid values: true, false
IxUpdateSchematicShowDummy	tableViewType	cellview	Cell	Specifies to show which type of dummy table.
IxUpdateSchematicShowExtra	tableViewType	cellview	Cell	Specifies to show which type of extra table.
IxUpdateSchematicSpecifySchematicTarget	bool	cellview	false	Updates dummy to specify a schematic target. Valid values: true, false

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxUpdateSchematicSyncDummyName	bool	cellview	false	Specifies if renaming layout instances. Valid values: true, false
IxUpdateSchematicTargetView	string	user	schematic_eco	Updates dummy view.
IxUpdateSchematicTermConnectionMapping	string	cellview		Term Connection Mapping.
IxUpdateSDLDataOnCopyMove	string	user	update	Controls whether or not to update or remove SDL data when copy/move lib, cell or cell view.
IxUsePostPlacementProc	bool	cellview	false	Uses post-process procedure after a device is placed.
IxUsePostProc	bool	cellview	false	Uses post-process procedure. Valid values: true, false
IxUseSDLPstProc	bool	cellview	false	Enables SDL post procedure. Valid values: true, false
IxUseTermOrderForTermMapping	bool	user	true	Uses termOrder for terminal mapping. Valid values: true, false
IxUseUpdateDummyPreProc	bool	cellview	false	Enables update dummy pre procedure. Valid values: true, false
IxVectorSuffix	vector_suffix_type	cellview	Parenthesis	Specify vector bracket for layout instance name.
IxVerifyAllParamsWithoutInstParameters	bool	user	true	Verifies all parameters without instameters/instParametersIgnore defined. Valid values: true, false
IxVerifyTotalWidthOnly	bool		false	Checks the total width of device. Valid values: true, false
IxViaTrackPlannerAutoAbutment	bool	user	true	Triggers abutment after VTP click. Valid values: true, false
IxViaTrackPlannerCPSize	float	user	0	Defines the size of connection point.

Table 152 *Schematic Driven Layout Preferences (Continued)*

Name	Type	Scope	Default	Description
IxViaTrackPlannerDialogMode	bool	user	false	ilx::viaTrackPlanner command options dialog eject state. Valid values: true, false
IxViaTrackPlannerHalo	float	user	0.05	Defines the size of connection point halo region.
IxViaTrackPlannerHideAnnotation	bool	user	false	Hides the annotation while Via Track Planner session is active. Valid values: true, false
IxViaTrackPlannerHideLayers	bool	user	true	Hides the drawing layers while Via Track Planner session is active. Valid values: true, false
IxViaTrackPlannerNetAware	string	user	None	Displays annotations only for selected nets. Valid values: None, Annotations, Annotations and CPs
IxViaTrackPlannerShortAware	bool	user	true	Controls whether Via Track Planner considers net shorts for peer detection.
IxViaTrackPlannerShowNullCPs	bool	user	true	Controls whether connection points with Null terminals are active. Valid values: true, false
IxViaTrackPlannerSnapToCP	bool	user	false	Controls whether snap cursor to connection point center while Via Track Planner session is active. Valid values: true, false
IxViewSearchList	string	cellview	abstract layout schematic	Search list.
IxViewStopList	string	cellview	abstract layout	Stop list.
IxWidthSum	bool	cellview	false	Width sum. Valid values: true, false

17

Netlist Editing Tcl

This chapter lists the commands, enumerations, and objects in the Custom Compiler Tcl namespace. This namespace provides Tcl for editing netlist representations of designs.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

Interactive commands are listed after non-interactive commands in the Commands section.

The commands covered are:

- [ne::getLogicals](#)
- [ne::showDesignOptions](#)

ne::getLogicals

Description

Returns the logical objects found at the given position of the given window. The first number in the position pair denotes list type and the second number denotes position in the list. Both numbers start at 1. If the position argument is not given then all objects in the design will be returned. If a position is given, the object at that position is returned. If the second value of the position pair is zero then all objects on the given list type will be returned in the collection. If both numbers are zero then all objects in the document are returned. The ordering of the objects in the collection is arbitrary.

Syntax

```
ne::getLogicals [<window: int>] [-position <point>]
```

Arguments

`window`

Window ID.

`-position`

Column and row in the neNetlist window of logical to fetch.

Returns

{<neLogical>}: Collection of logicals.

Errors

- Window is not of type neNetlist.
- Position does not exist in canvas.

ne::showDesignOptions

Description

Opens the Netlist Editor Design Options dialog for the given window.

Syntax

```
ne::showDesignOptions [<window: giWindow | int>]
```

Arguments

`-window`

Parent netlist editor window (neNetlist).

Returns

<giDialog>: giDialog object.

Errors

Window is not of type neNetlist.

Enumerations

Table 153 Enumerations

Name	Values	Description
enLogicalType	unknown instance net term	Logical type.

Objects

This section covers the following:

- [neLogical](#)

neLogical

This logical object is associated with specific item in view list and holds an associated logical OpenAccess block object.

Table 154 Attributes

Name	Type	Access	Description
name	<string>	Read-only	Name of the block object.
logicalType	<enLogicalType>	Read-only	Logical type.
object	<oaBlockObject>	Read-only	Associated block object.

Preferences

[Table 155](#) lists ne namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 155 Netlist Editing Preferences

Name	Type	Scope	Default	Description
neCheckCrossViews	bool	user	true	Indicates whether cross view check should be run on netlist views. Valid values: true, false

Table 155 *Netlist Editing Preferences (Continued)*

Name	Type	Scope	Default	Description
nePropertyEditorParameterFilter	string	user	hspiceD	Specifies the default values used in the Parameter Filter dialog for Netlist window type.

18

Netlist Tool Tcl

This chapter lists the commands, enumerations, objects, and preferences in the Custom Compiler Tcl nl namespace. This namespace provides Tcl for netlisting tool operations.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Netlisting Functions](#)
- [Netlisting Procedures](#)
- [Preferences](#)

Commands

Interactive commands are listed after non-interactive commands in the Commands section.

The commands covered are:

- [nl::addMapping](#)
- [nl::addVariableReference](#)
- [nl::addViewStopList](#)
- [nl::append](#)
- [nl::calculateMapping](#)
- [nl::checkInstHeader](#)
- [nl::checkSplitBuses](#)
- [nl::checkVHDLPortConsistency](#)

- [nl::checkVHDLUniqueEntityDefinitions](#)
- [nl::clearBuffer](#)
- [nl::computeTermOrder](#)
- [nl::computeTermOrder::getTermsFromDesign](#)
- [nl::computeTermOrder::sortTermsByType](#)
- [nl::computeVirtualTerminalName](#)
- [nl::createAMSNetlister](#)
- [nl::createHierMap](#)
- [nl::createNameMap](#)
- [nl::createNetlister](#)
- [nl::createOANameMap](#)
- [nl::createParameterizedNameMap](#)
- [nl::createPartitioner](#)
- [nl::createTerm](#)
- [nl::extractDigitalPowerDomains](#)
- [nl::finalizeNameMap](#)
- [nl::findAMSDomain](#)
- [nl::findAMSDomainObjects](#)
- [nl::findInstNamePrefix](#)
- [nl::findModuleDir](#)
- [nl::findModuleName](#)
- [nl::findNetlistProcedure](#)
- [nl::findParamMap](#)
- [nl::findParamValue](#)
- [nl::flattenDesign](#)
- [nl::generateEvent](#)
- [nl::getAlwaysWriteParamDefault](#)

- `nl::getAMSNetlisters`
- `nl::getBuffer`
- `nl::getEffectiveNetlistInfo`
- `nl::getExternalEntities`
- `nl::getICCSupplyName`
- `nl::getInstTerms`
- `nl::getMappedContext`
- `nl::getMappings`
- `nl::getModuleDescription`
- `nl::getModuleNameMapForAMS`
- `nl::getNamesForMapping`
- `nl::getNetlisters`
- `nl::getNetMembers`
- `nl::getNetType`
- `nl::getParamNames`
- `nl::getReferencedVariables`
- `nl::getShortedNets`
- `nl::getSupplyType`
- `nl::getTerms`
- `nl::getWireBitOrder`
- `nl::getWorkerNetlisters`
- `nl::indent`
- `nl::instHeaderHandler`
- `nl::isLeaf`
- `nl::isModuleUpToDate`
- `nl::isNetNameGlobal`
- `nl::isStringParam`

- `nl::isTextModule`
- `nl::lookup`
- `nl::map`
- `nl::mapModuleNames`
- `nl::mapPath`
- `nl::mapTerminalNames`
- `nl::message`
- `nl::moduleBodyHandler`
- `nl::moduleHandler`
- `nl::modulePreprocessHandler`
- `nl::netlistBeginHandler`
- `nl::netlistBodyHandler`
- `nl::netlistEndHandler`
- `nl::netlistMainHandler`
- `nl::nlPrintCdInst`
- `nl::nlPrintCdInstance`
- `nl::openNameMap`
- `nl::preloadModuleNames`
- `nl::printSpectreInst`
- `nl::printSpiceInst`
- `nl::printSpiceInstTerms`
- `nl::refreshPartitioner`
- `nl::revertOutdatedDesigns`
- `nl::runNetlister`
- `nl::saveModuleState`
- `nl::saveNameMap`
- `nl::setTermOrder`

- [nl::setWireBitOrder](#)
- [nl::uniqueify](#)
- [nl::writeUPF](#)

nl::addMapping

Description

Adds an entry to the name map table associated with the given module and scope.

Note:

This command does not enforce a one-to-one mapping in the name map. This is necessary to allow for languages that do not natively support name aliases.

```
nl::addMapping $nm -src net1 -dest net1
nl::addMapping $nm -src net1_alias -dest net1
puts "net1 => [nl::lookup $nm -src net1]"
# net1 => net1
puts "net1_alias => [nl::lookup $nm -src net1_alias]"
# net1_alias => net1
puts "[nl::lookup $nm -dest net1] <= net1"
# net1_alias <= net1
```

For aesthetics, the netlister/name map author can install the preferred name into the tables as a final step in netlisting a block. For example, the ordering of token addition for net aliases depends on the random order in which the netlister encounters the nets.

Suppose the order for a net with preferred name of "a" is:

```
Add a->a
Add b->a
```

Then a lookup from the destination token "a" returns "b". "b" is not the preferred name for the net in the design database. After netlisting the entire block, the netlister as a final step could iterate over the preferred net names and add them back into the tables, thus, controlling the name returned for reverse lookups, without modifying the order of netlisting. In this case, the netlister would Add a->a after the rest of the block is netlisted.

The `-type` argument should only be used for constructing character maps for the Parameterized Name Transform.

Syntax

```
nl::addMapping -map <nlNameMap> -src <string> [-dest <string>] [-type
<enSpecialChar>] [-module <heContext>] -scope <string>
```

Arguments

`-map`

Name map object to be modified.

`-src`

Database namespace identifier to be mapped to destination namespace.

`-dest`

Target namespace identifier to be reverse mapped to the database namespace.

`-type`

Specifies special states in character maps: {badChar | delete}.

`-module`

Context specifying the module level tables.

Default: NULL.

`-scope`

Scope of the identifier: commonly one of "Net", "Module", "Model", "Parameter", "Instance", "Terminal" or "Default". If a scope is not supplied, then the entry is added in the "Default" scope.

Returns

Input identifier mapped into the correct namespace. The return value is never empty.
(<string>)

Examples

The following example maps "\$" to "_" within the scope of the currently selected net.

```
nl::addMapping $nm -scope Net -src "$" -dest "_"
```

The following example deletes the exclamation character (!) from tokens within the scope of the currently selected terminal.

```
nl::addMapping $nm -scope Term -src '!' -type delete
```

The following example specifies characters for further escaping or renaming within the scope of the currently selected net.

```
nl::addMapping $nm -scope Net -src '<' -type badChar
```

nl::addVariableReference

Description

Adds an entry into the netlister's set of referenced variables. The netlisting framework detects the usage of design variables and parent()/lineage() parameters through tracking the execution of `nl::findParamValue` function calls. However, this command can be used to explicitly add variable references to the automatically generated set.

Syntax

```
nl::addVariableReference <name: string> -netlister <nlNetlister> -master  
  <heContext> -type <enum>
```

Arguments

`name`

The name of the reference added.

`-netlister`

The netlister object modified.

`-master`

The context object referencing the input variable.

`-type`

The type of variable reference: {all | design | inst | lineage | parent}

Returns

<none>

nl::addViewStopList

Description

Associates the netlister with view stop list.

Syntax

```
nl::addViewStopList -netlister <nlNetlister> -viewStopList [<string>] -to  
  <nlAMSNetlister> [-master <bool>]
```

Arguments

`-netlister`

The netlister to use for the given view stop list.

`-viewStopList`

The view stop list.

`-to`

The AMS netlist to add the given view stop list.

`-master`

If the added view stop list is for the master netlist.

Default: false.

Returns

<none>

Examples

The following example adds the HSPICE netlist to view the stop list {hspice symbol schematic} in the AMS netlist.

```
set nlr [nl::getNetlisters HSPICE]
set ams [nl::getAMSNetlisters AMS]
nl::addViewStopList -netlist $nlr -viewStopList\
    {hspice symbol schematic} -to $ams -master true
```

nl::append

Description

Appends the given string to the buffer of the given netlist. If the text exceeds the maximum line length, then it wraps the text, using the `lineWrapSuffix` of the currently executing netlist. If `-comment` is true, then the `commentPrefix` is used in place of the `lineWrapSuffix`. New `-comment` lines begin at the buffer's current indentation depth.

Syntax

```
nl::append <string: string> -netlist <nlNetlist> [-comment <bool>]
          [-wrap <bool>] [-format <bool>]
```

Arguments

`string`

The text to append to the buffer.

`-netlist`

The netlist to append the output to.

`-comment`

If the appended text is a comment.

Default: false.

`-wrap`

Wrap long lines using the `lineWrapSuffix` when the text exceeds the maximum line length. (Default is: true)

`-format`

Format appended text using current indentation, comment formatting, and line wrapping. (Default is: true)

Returns

<none>

Examples

The following example appends the text in quotes to the specified buffer of the `$nl` netlister.

```
nl::append "input text to netlist file" -netlister $nl \  
-wrap true -comment false
```

The following example appends the text to the buffer without any additional formatting or line wrapping.

```
nl::append "input text to netlist file" -netlister $nl \  
-wrap false -comment false -format false
```

nl::calculateMapping

Description

This command calculates and optionally adds a source to destination association to a name map.

The basename for bit and bus identifiers are also added to the name tables. The `nl::map` function relies on having mapping information for bus basenames to provide vectored/bit mapping support.

Note:

The `-fixedFirstChar` argument enforces that the first character of the mapped identifier is the specified character. This argument is provided to simplify mapping of SPICE instance names.

Syntax

```
nl::calculateMapping -map <nlNameMap> -add <bool> -src <string> [-module  
  <heContext>] -scope <string>  
-fixedFirstChar <string>
```

Arguments

-map

Name map to be modified.

-add

If set to true, the name tables are updated with the result of the calculation.

-src

Database name. The database namespace identifier to be mapped to destination namespace.

-module

Context specifying the module level tables.

Default: NULL.

-scope

Scope of the identifier, commonly one of "Net", "Module", "Model", "Parameter", "Instance", "Terminal", or "Default".

-fixedFirstChar

First character.

Returns

Input identifier mapped into the proper namespace. The name map tables are updated if necessary (<string>).

Errors

1) If the name map object is finalized and the -add argument is set to true.

nl::calculateMapping relies on information that is stripped from the name tables by nl::finalizeNameMap. Therefore it is not possible to add to a finalized name map.

Examples

The following example calculates and adds the source-to-destination association for a name map within the scope of the selected instance.

```
set mapInstName [nl::calculateMapping \  
  -map $mapObj \  
  -src $instName \  
  -scope $scope]
```

```
-scope Instance \  
-add true]
```

nl::checkInstHeader

Description

Verifies the compatibility between placed and switched masters. This command verifies that the terminals are compatible between two designs. It is called by the netlisting subsystem to verify compatibility between placed and switched masters of an instance header.

If the masters are incompatible a Tcl error is generated containing the error reason.

Syntax

```
nl::checkInstHeader <instHeader: heInstHeader>
```

Arguments

instHeader

The instance header verified.

Returns

<none>

nl::checkSplitBuses

Description

Checks whether the module has split buses – bus terminals with names that cannot be referenced in the Verilog by the base name only.

This is because either:

- The terminal does not contain all the bits of the bus range.

Or

- The terminal bits are connected to nets which cannot be represented by one base name in Verilog.

Syntax

```
nl::checkSplitBuses <nlNetlister> <heContext>
```

Arguments

`nlNetlister`

The netlister.

`heContext`

The module context.

Returns

<int>: 1 if there are split buses, and 0 otherwise.

nl::checkVHDLPortConsistency

Description

Checks the port names and directions between the designs of given contexts and model that is being bound, if such entity exists in VHDL models external directory.

Syntax

```
nl::checkVHDLPortConsistency -contexts [<heContext>]
```

Arguments

`-contexts`

List of contexts to be checked.

Returns

<none>

nl::checkVHDLUniqueEntityDefinitions

Description

Checks the entity names that differ only by case for 2 leaf cells or for text module and leaf cell, if entity of leaf cell exists in VHDL models external directory.

Syntax

```
nl::checkVHDLUniqueEntityDefinitions -leafList [<heContext>] [-textList  
[<heContext>]]
```

Arguments

`-leafList`

List of leaf contexts to be checked.

`-textList`

List of text contexts to be checked vs. leaf contexts.

Returns

<none>

nl::clearBuffer

Description

Resets the netlister's buffer to the empty string. Note that the procedures registered with the nlNetlister object are responsible for writing the buffer contents to disk. Also, the buffer is not directly associated with a file descriptor by the netlisting framework. The netlisting event handlers are responsible for writing the buffer to disk.

Syntax

```
nl::clearBuffer -netlister <nlNetlister> [-reset <bool>]
```

Arguments

`-netlister`

The netlister to manipulate.

`-reset`

If true, the current buffer position and indent level are set to zero. (Default is: true)

Returns

<none>

Examples

The following example resets the netlister's buffer position and indent level to zero.

```
nl::clearBuffer -netlister $nl
```

The following example does not modify the current buffer indent level and position. Only the contents of the buffer are cleared.

```
nl::clearBuffer -netlister $nl -reset false
```

nl::computeTermOrder

Description

Computes the term order for the specified context. Note that this command does not change the terminal order returned by the `nl::getTerms` or `nl::getInstTerms` command. The `nl::setTermOrder` command must be called to modify the term order used by the netlister.

The Custom Compiler netlister supports two general modes for computing terminal orders: “Compatible” and “Consistent”. Compatible port ordering matches industry standard port ordering algorithms. “Consistent” port ordering is similar to “Compatible” except that it ignores `simInfo` `portOrder` properties for hierarchical designs because they are commonly out of sync with the design. Port names in the `portOrder`/`termOrder` are specified within single quotations, double quotations or without any quotations.

Lookup Precedence for Text Views:

- `viewInfo-><switched master view name>->termOrder`
- `portOrder` property (Switched Master, Placed Master)
- `simInfo-><netlistInfo>->termOrder`
- OA Terminals (sorted by `–sort algorithm`)
 - Switched Master
 - Placed Master (if Switched Master has no design)

Lookup Precedence for hierarchical designs (Consistent):

- `portOrder` property (Switched Master, Placed Master)
- OA Terminals (sorted by `–sort algorithm`)
 - Switched Master
 - Placed Master (if Switched Master has no design)
- Virtual terminals are added to the end of the list in sorted alphanumeric order.

Lookup Precedence for hierarchical designs (Compatible):

- `simInfo-><netlistInfo>->termOrder`
- `<matches lookup precedence for hierarchical designs (Consistent)>`

Lookup Precedence for leaf designs:

- `<matches precedence for hierarchical designs (Compatible)>`

Note:

The portOrder property and termOrder simInfo/netlistInfo may contain fewer or more terminals than what exist in OA. Extra terminals in portOrder/termOrder should be quietly ignored. This is required for PCell's which have a variable number of terminals.

If there are fewer terminals in portOrder/termOrder:

- For leaf designs, the extra OA terminals are quietly ignored.
- For hierarchical designs, the extra OA terminals must be appended to the end of the terminal list (but before the inherited connections/virtual terminals).

Syntax

```
nl::computeTermOrder -of <heContext> -netlist <nlNetlist>  
  [-compatible <bool>] [-sortByType <string>] [-appendVirtualTerms  
  <bool>] [-ignorePortOrderProp <bool>]
```

Arguments

-of

The context for which terminal order is computed.

-netlist

The netllister used to determine the netlistInfo name for looking up termOrder.

-compatible

If compatible port ordering lookup precedence should be used. Default is: true.

-sortByType

If specified, the terminals will be grouped by type in the specified order. Terminals within each group are sorted alphanumerically. Terminals which do not correspond to any of the specified types are implicitly added in the last group preceding virtual terminals. The list of type names are used to group terminals.

-appendVirtualTerms

Set to false to leave virtual terminals out of the terminal list. Default is true.

-ignorePortOrderProp

If enabled, the portOrder property will be ignored for hierarchical schematics. Default is false.

Returns

<string>: The computed list of terminal names.

nl::computeTermOrder::getTermsFromDesign

Description

Gets terminals from the design. Gets the OA terminals and creates tcl list collection of the corresponding nlTerm objects and if portOrder parameter is not empty, sorts the terminals according to the portOrder.

Syntax

```
nl::computeTermOrder::getTermsFromDesign <oaDesign> <nlNetlister>  
    <string>
```

Arguments

oaDesign

The design.

nlNetlister

The netlister.

string

The port order to sort the terminal objects.

Returns

[<nlTerm>] : The list of created terminals.

nl::computeTermOrder::sortTermsByType

Description

Sorts terminals according to the sequence of the types for the given list of terminal types (for example {"input" "output" "inputOutput" }). Same type of terminals will appear in an alphanumerical order. If any terminal type does not exist in the list, those terminals will be added at the end of the list. If the terminal types list is empty, terminals will be sorted in the following order: inputs, outputs, inputOutputs, switches, jumpers, unused, and tristate.

Syntax

```
nl::computeTermOrder::sortTermsByType [<nlTerm>] [<string>]
```

Arguments

nlTerm

The initial terms list.

`string`

List of terminal types, for example, {"input" "output" "inputOutput" }.

Returns

[<nlTerm>] : The list of sorted terminals.

nl::computeVirtualTerminalName

Description

Computes virtual terminal names for the specified context.

Syntax

```
nl::computeVirtualTerminalName connDefName defNetName
```

Returns

<none>

nl::createAMSNetlister

Description

Creates an analog mixed-signal (AMS) netlister object.

Syntax

```
nl::createAMSNetlister <name: string> [-masterNetlister <string>]  
[-AMSBindings [<string>]] [-unifiedNamespace <bool>]]
```

Arguments

`name`

Netlister name.

`-masterNetlister`

Master netlister name.

Default: HSPICE.

`-AMSBindings`

The collection of nlNetlister name to view stop list bindings.

`-unifiedNamespace`

If set to true, a common namespace is used for all name mapping. (Default is: true)

Returns

<nlAMSNetlister>: Newly created AMSNetlister object.

nl::createHierMap

Description

This command creates an nlHierMap object. The nlHierMap object is used to map hierarchical instance/net names between namespaces. Destroying the referenced nlNameMap object invalidates the nlHierMap object.

Syntax

```
nl::createHierMap <map: nlNameMap | nlOANameMap | nlParameterizedNameMap>
```

Arguments

nlNameMap

Name map used for the transforming of names.

Returns

New nlHierMap. (nlHierMap)

Errors

If the passed name map does not have both a rootContext and a hierarchical delimiter and error will occur.

nl::createNameMap

Description

This command creates a new nlNameMap to store table-based mappings

For parameterized name map objects, see [nl::createParameterizedNameMap](#). For oaNameSpace based name map objects, see [nl::createOANameMap](#).

Note that you can query all attributes of a name map using the common `db::getAttr` command. However, you cannot modify the attributes after the name map is created.

You can delete a name map object using the `db::destroy` command.

The option `-scopeAliases` accepts a list of lists. The format is:

```
[list [list scope1 [list conflict1 conflict2 ...]] [list scope2 [list  
conflict1 conflict2 ...]] ...]
```

For example the `-scopeAliases` for Verilog are:

```
set aliases [list [list Net [list Instance Parameter]] [list Term [list  
Instance Parameter]] [list Instance [list Parameter Net Term]] [list  
Parameter [list Instance Net Term]]]
```

An equivalent way of writing this is:

```
set aliases {  
    {Net {Instance Parameter}}  
    {Term {Instance Parameter}}  
    {Instance {Parameter Net Term}}  
    {Parameter {Instance Net Term}}  
}
```

Syntax

```
nl::createNameMap [-openBrace <string>] [-closeBrace <string>]  
[-hierDelimiter <string>] [-rootContext <heContext>] [-leadDelimiter  
<bool>] [-scopeAliases [<string>]]
```

Arguments

`-openBrace`

Character to indicate the beginning of a bit index in the destination namespace.

`-closeBrace`

Character to indicate the end of a bit index in the destination namespace.

`-hierDelimiter`

Optional hierarchy delimiter/separator in the destination namespace. Only needed for name maps, which are used with the Hierarchical Name Map API.

Default: "/".

`-rootContext`

Optional context object to elaborate the design hierarchy. It is needed for name maps, which are written and read from the disk, and required if the object will be used for hierarchical name mapping using `nl::mapPath`.

Default: NULL.

`-uniqueifyProc`

Procedure in the write API to make identifiers unique in case of a scope conflict.

Default: `nl::uniqueify`.

`-scopeAliases`

Scope check for identifier aliasing.

Default: All name scopes are independent.

Returns

New name map object (nlNameMap)

nl::createNetlister

Description

Creates a netlister object. Typically one netlister object is created for each output format; that is, a netlister is created for HSPICE, for Verilog, for ICV, etc. The created object is not bound to a cellView and is intended to be used to run all of the netlisting jobs for a given output format.

The `db::setAttr` command should be used to configure an `nlNetlister` object after creation. The `nlNetlister` object has many attributes and it is not convenient to set them all from a single command line.

Syntax

```
nl::createNetlister <name: string> [-type <enum>]
```

Arguments

`name`

Netlister name.

`-type`

The type of the parameters to return. ({ hspice | user | verilog }, default is: user)

Returns

<nlNetlister> : Newly created netlister object

Errors

If the named netlister object already exists.

Examples

```
set nl [nl::createNetlister myHspice -type hspice]
```

This example creates a netlister called myHspice.

```
set nl [nl::createNetlister myNetlister -type user]
```

In this example, `nl::createNetlister` creates the netlister myNetlister of the user type.

```
set nl [nl::createNetlister customSPICE -type hspice]
```

In this example, `nl::createNetlister` creates the `customSPICE` netlister of the `hspice` type.

nl::createOANameMap

Description

Creates a new `nlNameMap` using the Open Access procedural name transforms.

For parameterized name transforms, see [nl::createParameterizedNameMap](#).

You must provide the generic `openBrace/closeBrace` and `hierarchyDelimiter` arguments because `oaNamespace` objects cannot programmatically supply this information. Also note the `oaNameSpace` system is a plug-in system, and you can add your own name transforms at runtime.

Note that you can query all attributes of a name map object using the common `db::getAttribute` command. However, you cannot modify the attributes after the name map object is created.

You can delete a name map using the `db::destroy` command.

The option `-scopeAliases` accepts a list of lists. The format is:

```
[list [list scope1 [list conflict1 conflict2 ...]] [list scope2 [list  
conflict1 conflict2 ...]] ...]
```

For example the `-scopeAliases` for Verilog are:

```
set aliases [list [list Net [list Instance Parameter]] [list Term [list  
Instance Parameter]] [list Instance [list Parameter Net Term]] [list  
Parameter [list Instance Net Term]]]
```

An equivalent way of writing this is:

```
set aliases {  
  {Net {Instance Parameter}}  
  {Term {Instance Parameter}}  
  {Instance {Parameter Net Term}}  
  {Parameter {Instance Net Term}}  
}
```

Syntax

```
nl::createOANameMap [-openBrace <string>] [-closeBrace <string>]  
  [-hierDelimiter <string>] [-rootContext <heContext>] [-leadDelimiter  
  <bool>] [-scopeAliases [<string>]] [-algo <bool>] -oaNameSpace  
  <oaNameSpace>
```

Arguments

`-oaNameSpace`

Sets the destination namespace.

`-algo`

True, if the new name map is algorithmic. False, if it contains table data.

Default: true.

`-openBrace`

Character to indicate the beginning of a bit index in the destination namespace.

`-closeBrace`

Character to indicate the end of a bit index in the destination namespace.

`-hierDelimiter`

Optional hierarchy delimiter/separator in the destination namespace. Only needed for name maps that are used with the Hierarchical Name Map API.

Default: "/".

`-rootContext`

Optional context to elaborate the design hierarchy. Only needed for name maps that are written and read from disk, or used with hierarchical name maps.

Default: NULL.

`-uniqueifyProc`

Procedure in the write API to generate unique identifiers in the case of a scope conflict.

Default: nl::uniqueify.

`-scopeAliases`

Scope check for identifier aliasing.

Default: All name scopes are independent.

Returns

New name map (nIONameMap).

nl::createParameterizedNameMap

Description

Creates a new name map using the parameterized name transform. The parameterized name map is able enough to cover most netlisting languages. As such it is the most commonly used name mapping object for creating netlists.

To use the Open Access name transforms, see [nl::createOANameMap](#).

You can query all attributes of a name map object using the common `db::getAttribute` command. However, you cannot modify the attributes after the name map is created.

You can delete a name map object using the `db::destroy` command.

The option `-scopeAliases` accepts a list of lists. The format is:

```
[list [list scope1 [list conflict1 conflict2 ...]] [list scope2 [list  
conflict1 conflict2 ...]] ...]
```

For example the `-scopeAliases` for Verilog are:

```
set aliases [list [list Net [list Instance Parameter]] [list Term [list  
Instance Parameter]] [list Instance [list Parameter Net Term]] [list  
Parameter [list Instance Net Term]]]
```

An equivalent way of writing this is:

```
set aliases {  
    {Net {Instance Parameter}}  
    {Term {Instance Parameter}}  
    {Instance {Parameter Net Term}}  
    {Parameter {Instance Net Term}}  
}
```

Syntax

```
nl::createParameterizedNameMap [-openBrace <string>] [-closeBrace  
<string>] [-hierDelimiter <string>] [-rootContext <heContext>]  
[-leadDelimiter <bool>] [-scopeAliases [<string>]] [-escapePrefix  
<string>] [-escapeSuffix <string>] [-renamePrefix <string>] -maxLength  
<int> [-startCharMap <nlNameMap>] -validCharMap <nlNameMap> [-keywords  
[<string>]] [-algo <bool>] [-prependFirstChar <enum>]
```

Arguments

`-openBrace`

Character beginning multi-bit index.

`-closeBrace`

Character ending multi-bit index.

`-hierDelimiter`
Hierarchy delimiter.
Default: `/`.

`-rootContext`
Root context for hierarchy path mapping.

`-leadDelimiter`
If the destination path begins with a delimiter.
Default: `false`.

`-scopeAliases`
Scope check for identifier aliasing.

`-escapePrefix`
String to prepend to invalid identifiers.

`-escapeSuffix`
String to append to invalid identifiers.

`-renamePrefix`
Prefix string for renaming invalid identifiers.

`-maxLength`
The maximum identifier length.

`-startCharMap`
Name map object to transform each start character to a valid destination string.

`-validCharMap`
Name map object to transform each character to a valid destination string.

`-keywords`
List of reserved identifiers.

`-algo`
If mapping purely algorithmic, or can use stored table data.
Default: `false`.

`-prependFirstChar`
The algorithm used to determine if `-fixedFirstChar` should be prepended:
{`always` | `auto`}.

Returns

New name map (nlParameterizedNameMap)

Examples

In this example, `nl::createParameterizedNameMap` creates a parametrized name map. The `openBrace` and `closeBrace` attributes specify the characters at the beginning and end of a bit index in the destination namespace. The `hierDelimiter` argument defines the optional hierarchy delimiter/separator in the destination namespace to be ".".

The `escapePrefix` and `escapeSuffix` attributes specify that a string will be prepended and appended to invalid identifiers. The `renamePrefix` argument designates that string to be `rnm`.

The `maxLength` argument specifies a maximum identifier length of 1024 characters. The `startCharMap` argument specifies the name map that transforms the first character of each name to a valid destination string. The `validCharMap` argument specifies the name map that transforms each character to a valid destination string. The `keywords` argument defines the reserved identifiers to be `list`, `output`, `input`, and `inout`.

```
set nlParameterizedNameMap [nl::createParameterizedNameMap \
    -openBrace  {} \
    -closeBrace {} \
    -hierDelimiter {.}\
    -escapePrefix "\\\" \
    -escapeSuffix "" \
    -renamePrefix "rnm" \
    -maxLength 1024 \
    -startCharMap $startmap \
    -validCharMap $validmap \
    -keywords [list output input inout]]
```

nl::createPartitioner

Description

Creates partitioner for given hierarchy context based on given AMS netlister.

Syntax

```
nl::createPartitioner -netlister <nlAMSNetlister> -context <heContext>
```

Arguments

`-netlister`

AMS netlister which settings are used for partition.

`-context`

Hierarchy context for which the partitioner is destined.

Returns

<nlPartitioner>: Partitioner object.

nl::createTerm

Description

Creates an nlTerm object. nlTerm objects represent terminals in the netlisting connectivity model. The netlisting infrastructure typically creates the correct set of terminals for each module in the design hierarchy.

Syntax

```
nl::createTerm <noname: nlTerm | oaTerm | string> [-context <heContext>]  
[-net <oaNet | string>] [-virtual <bool>] [-defaultNet <oaNet | string>]  
[-termType <enum>] [-scalarize <bool>]
```

Arguments

noname

The terminal or terminal name.

-context

The context used to look up terminals/nets specified by name.

-net

The net or net name from the terminal.

-virtual

If true, the created terminal (scalarize) should propagate as an inherited connection. Default is: false.

-defaultNet

The default net used if '-virtual' is true.

-termType

Type of probe to create. ({input | inputOutput | jumper | output | switch | tristate | unused})

-scalarize

If true, the input terminal will be split into individual bit terms. Default is: false.

Returns

{<string>}: Contents of the printed string.

nl::extractDigitalPowerDomains

Description

Scans a design hierarchy and extracts power supply information.

Syntax

```
nl::extractDigitalPowerDomains -topCellView <heConfig | heContext>  
    [-createDomains <bool>]
```

Arguments

-topCellView

The design to traverse

-createDomains

Indicates whether power domains should be created in the return data structure.
(Default is: true)

Returns

<nIICCSupplyInfo> : The power domains found in the design hierarchy

nl::finalizeNameMap

Description

Removes identity mappings from the name map object. Identity mappings are defined as entries where the source and destination tokens are the same. Finalized module tables are read only and cannot be used with the name mapping Write API. Note, individual modules can be finalized separately using the optional -module argument.

Syntax

```
nl::finalizeNameMap <map: nlNameMap | nlOANameMap |  
    nlParameterizedNameMap> [-module <heContext>]
```

Arguments

nlNameMap

Name map object to query or modify.

-module heContext

Context specifying the module level tables. If not supplied then all existing module tables are finalized.

Default: NULL.

Returns

<none>

nl::findAMSDomain

Description

Returns the domain type of specified object or context.

Syntax

```
nl::findAMSDomain [-of <deFigure | oaInst | oaInstTerm | oaNet | oaShape  
| oaTerm>] -context <heContext> -partitioner <nlPartitioner>
```

Arguments

-of

Object.

-context

Hierarchy context.

-partitioner

Partitioner to be used.

Returns

<string>: Domain type.

nl::findAMSDomainObjects

Description

Returns collection of OA objects of corresponding domain type in the current design of hierarchy context.

Syntax

```
nl::findAMSDomainObjects <name: enum> -context <heContext> -partitioner  
<nlPartitioner>
```

Arguments

name

Domain type ({ AMS | analog | digital | mixed | unknown | verilogA }).

`-context`

Hierarchy context.

`-partitioner`

Partitioner to be used.

Returns

{<oaBlockObject>} : Object container.

nl::findInstNamePrefix

Description

Determines the instance name prefix used for invoking netlist procedures. This utility looks in the specified simInfo/Netlist info for the instNamePrefix property. If it is not found, then the empty string is returned.

The equivalent Tcl implementation of the command is as follows:

```
proc find_inst_name_prefix {inst_header tool} {  
    set pm [db::getAttr placedMaster -of $inst_header]  
    set lib [db::getAttr lib -of $pm]  
    set lib_name [db::getAttr name -of $lib]  
    set cell [db::getAttr cell -of $pm]  
    set cell_name [db::getAttr name -of $cell]  
    set data [oa::CellDMDDataOpen $lib_name $cell_name r]  
    return [db::getNetlistInfo instNamePrefix -cell $data \  
-netlist $tool]  
}
```

Syntax

```
nl::findInstNamePrefix -instHeader <heInstHeader> -tool <string>
```

Arguments

`-instHeader`

The instance header being processed.

`-tool`

The tool name to use for querying the Netlist info `instNamePrefix` field.

Returns

The prefix used for all instance names bound to the given `heInstHeader`. (<string>)

nl::findModuleDir

Description

This command retrieves the directory name in the UNIX namespace for the given `heContext`. If a mapping does not exist, a new mapping is created and returned guaranteeing that calculated mappings will not conflict with previously mapped `heContexts`. The directory returned is a relative path, typically "module_name". This directory name can be used to store per-module metadata. The netlister uses this directory to store partial netlists and timestamps for incremental netlisting.

Syntax

```
nl::findModuleDir <heContext: heContext> -map <nlNameMap | nlOANameMap |  
nlParameterizedNameMap>
```

Arguments

`heContext`

Context object.

`-map`

The name map object used to find or calculate the destination namespace module name.

Returns

Unique directory name for the given context. (<string>)

nl::findModuleName

Description

Retrieves the module name in the destination namespace for the given `heContext`. If a mapping does not exist, a new mapping is created and returned guaranteeing that calculated mappings will not conflict with previously mapped `heContexts`. This command uses the mapped cell name of the context if possible. If necessary this command makes the module name unique to ensure that contexts with different IDs map to different module names.

Syntax

```
nl::findModuleName <heContext: heContext> -map <nlNameMap | nlOANameMap |  
nlParameterizedNameMap> [-prefix <string>] [-suffix <string>]
```

Arguments

heContext

Context object.

-map

The name map used to find or calculate the destination namespace module name.

-prefix

Prefix to add.

-suffix

Suffix to add.

Returns

Unique string for the given context (<string>).

nl::findNetlistProcedure

Description

Determines the netlist procedure for the given instance header. It searches the simInfo/Netlist info for the given tool and returns the field stored in the netlistProcedure field. If the field does not exist, then the empty string is returned.

The equivalent Tcl implementation of the command is as follows:

```
proc find_netlist_procedure {inst_header tool} {  
    set pm [db::getAttr placedMaster -of $inst_header]  
    set lib [db::getAttr lib -of $pm]  
    set lib_name [db::getAttr name -of $lib]  
    set cell [db::getAttr cell -of $pm]  
    set cell_name [db::getAttr name -of $cell]  
    set data [oa::CellDMDDataOpen $lib_name $cell_name r]  
    return [db::getNetlistInfo netlistProcedure -cell $data \
```



```
-netlister $tool]  
}
```

Syntax

```
nl::findNetlistProcedure -instHeader <heInstHeader> -tool <string>
```

Arguments

`-instHeader`

The instance header being processed.

`-tool`

The tool name to use for querying the Netlist info netlistProcedure field.

Returns

The name of the Tcl procedure to use for netlisting instances of the given heInstHeader (<string>).

nl::findParamMap

Description

Utility determining the mapping from database parameter name to netlisted parameter. The mapping is computed by parsing the "propMapping" field in the Netlist Info of the given master. It involves finding the CDF data, parsing the string stored in the CDF data, creating an nmMap object, filling that map object and returning the final computed object. The returned object can then be used to determine the mapped parameter name for the given simInfo/simulator.

Syntax

```
nl::findParamMap -netlistInfo <string> -master <heContext>
```

Arguments

`-netlistInfo`

Netlist Info identifier to use for finding the "propMapping" field.

`-master`

The component queried.

Returns

Name map object containing the mapping from database parameter name to simulator parameter name. ([nlNameMap](#))

nl::findParamValue

Description

Determines the Netlist Evaluated Value of the named parameter/property. It also tracks parameter lookups during parameter evaluation to compile the lists of passed parameters for each module.

If an instance is not provided, then -master's default value is returned. Otherwise the value of the parameter on the given instance is returned.

To retrieve the netlisting evaluated value of a parameter without adding parameter and design variable references to an nlNetlister object, use `db::getParamValue -eval netlisting`.

Syntax

```
nl::findParamValue <param_name: string> -netlister <nlNetlister> -master  
<heContext> [-inst <oaInst>] [-parent <heContext>]
```

Arguments

`param_name`

The name of the parameter to look up.

`-netlister`

The netlister used to track parameter/variable references.

`-master`

The switched master.

`-inst`

The instance used to find parameters.

Default: NULL.

`-parent`

The parent context of the instance. Referenced parameters are inserted to the set of parameters maintained by the netlister for the parent context.

Default: NULL.

Returns

The evaluated value of the named parameter/property. (<string>)

Errors

- If -inst is specified and -parent is not.
- If -parent is specified and -inst is not.

Examples

The following example retrieves the value of the parameter `w` for the specified master object from within parent context `$ctx` of the selected instance.

```
set paramValue [nl::findParamValue "w" -netlist $nl \  
-master $master -parent $ctx]
```

nl::flattenDesign

Description

Creates a logical design which is electrically equivalent to the input design hierarchy. The new design has one level of hierarchy and contains an instance for each occurrence from the input design hierarchy.

Hierarchically passed parameters and inherited connections are resolved and their effective values are stored on the flattened instances.

The new design contains only the set of objects necessary to logically represent the design. Shapes for example are not copied. The logic design is created as an `oacNetlistViewType`.

Syntax

```
nl::flattenDesign <cellView: dmCellView | dmCellViewName> [-toolNames  
<string>] [-netlistName <string>] -src <deContext | dmCellView  
| dmCellViewName | heContext> [-overwrite <bool>] [-viewSearchList  
<string>] [-viewStopList <string>]
```

Arguments

`cellView`

The location to write the flattened design.

`-toolNames`

The tool names of netlister to find the parameters map for multiplication factor processing.

`-netlistName`

The name of the netlister to find the ignore settings.

`-src`

The source design.

`-overwrite`

Controls if an existing destination cellview should be overwritten. (Default: false)

`-viewSearchList`

If the input design is specified as a dmCellView the viewSearchList is used to elaborate the hierarchy. (Default: schematic symbol)

`-viewStopList`

If the input design is specified as a dmCellView the viewStopList is used to elaborate the hierarchy. (Default: symbol)

Returns

<none>

nl::generateEvent

Description

Calls the function registered with the netlister for the specified event. When a context or instance header is specified they are installed as the active context/instance header for the netlister.

Syntax

```
nl::generateEvent <type: enum> -netlister <nlNetlister> [-module  
  <heContext>] [-instHeader <heInstHeader>]
```

Arguments

`type`

The event to generate: {instHeader | instPost | instance | module | moduleBody | moduleFooter | moduleHeader | modulePreprocess | netlistBegin | netlistBody | netlistEnd | netlistMain | textModule}

`-netlister`

The netlister to use for generating events.

`-module`

The module currently being processed.

Default: null.

`-instHeader`

The instance header currently being processed.

Default: null.

Returns

<none>

nl::getAlwaysWriteParamDefault

Determines whether the instance parameter with default value should be printed even with the `nl<name>WriteParamDefaults` preference set to `false`.

Syntax

```
nl::getAlwaysWriteParamDefault <name: string> -netlister <nlNetlister>  
-context <heContext>
```

Returns

<bool>: True if parameter should always be printed

Arguments

`name`

The name of the parameter.

`-netlister`

The current netlister.

`-context`

The master context of instance.

nl::getAMSNetlisters

Description

Returns the analog, mixed signal (AMS) netlister object.

Syntax

```
nl::getAMSNetlisters [<pattern: string>] [-filter <string>]
```

Arguments

`pattern`

Pattern to match AMS netlister name.

Default: *.

-filter

Filter expression.

Default: match all.

Returns

{<nlAMSNetlister>} : Collection of AMS netlisters with names matching input pattern.

Examples

```
set nl [db::getNext [nl::getAMSNetlisters "AMS"]]
db::setAttr AMSBindings -of $nl
    -value HSPICE {hspice hspiceD veriloga
                  {Verilog {verilog behavioral functional verilogams}}
                  {VHDL {vhdl}}
    }
db::setAttr AMSViewTypeBindings -of $nl
    -value HSPICE {SPICEText verilogaText
                  {Verilog {verilogText verilogAMSText
                           systemVerilogText}}
                  {VHDL {VHDLText}}
    }
```

See also SolvNetPlus article [#000042230 "How to handle netlisting errors related to AMS bindings in AMS simulation \(NETLISTING-072/NETLISTING-001\)"](#).

nl::getBuffer

Description

Gets all the text in the current netlister's output buffer.

Syntax

```
nl::getBuffer -netlister <nlNetlister>
```

Arguments

-netlister

The netlister to manipulate.

Returns

The contents of the current netlister's output buffer. (<string>)

Examples

The following example retrieves the text in the currently active netlister's output buffer.

```
set inputBuffer [nl::getBuffer -netlister $nl]
```

nl::getEffectiveNetlistInfo

Description

Gets the `netlistInfo` field to be used for the given device, for case of multiple info names declared. For only one name returns this sole name, like `db::getAttr netlistInfo ....`

Syntax

```
nl::getEffectiveNetlistInfo <name: nlNetlister> -context <heContext>
```

Arguments

`name`

Netlister

`-context`

The context for which `netlistInfo` is asked.

Returns

`<string>` : Value of the `netlistInfo` name.

nl::getExternalEntities

Description

Gets all entity and architecture pairs for all the files in the specified external directory.

Syntax

```
nl::getExternalEntities -directory <string>
```

Arguments

`-directory`

The name of the external directory.

Returns

`<string>`: List of entity and architecture names existing in the specified external directory.

nl::getICCSupplyName

Description

Computes a valid net name for the given object. The name is guaranteed to be a valid net name in UPF and ICC.

Syntax

```
nl::getICCSupplyName <oaNetName: oaBitNet | string>
```

Arguments

oaNetName

The net/net name to map.

Returns

<string>: The mapped net name.

nl::getInstTerms

Description

Returns the list of nlInstTerms in the order corresponding to the order of nlTerms for the switched master. This list includes the virtual terminals required to implement inherited connections.

Syntax

```
nl::getInstTerms -netlist <nlNetlist> -inst <oaBitInst> -master  
<heContext> [-filter <string>]
```

Arguments

-netlist

The netlist object providing the Simplified Connectivity Model.

-inst

The instance inspected.

-master

The switched master of the given instance.

-filter

Standard attribute filter.

Default: match all.

Returns

The list of instance terminals. ([[nlInstTerm](#)])

Errors

If the given master does not exist in the netlister's terminal data.

Examples

The following example retrieves instance terminals from the inspected object `inst` within the specified switched master.

```
set instPorts [nl::getInstTerms -inst $inst -master $master \  
-netlister $nl]
```

nl::getMappedContext

Description

Takes as an unnamed argument a hierarchical path name and computes the `heContext` for the given hierarchical path name. The hierarchical path name is typically given in the simulator's namespace. The passed in `nlHierMap` is used to transform the path name into the database namespace. By default, name comparison is performed in a case-sensitive manner. This can be changed by passing the `caseSensitive` argument with a value of `false`. For case-insensitive matching, the first `heContext` that matches is returned.

Syntax

```
nl::getMappedContext <string> -map <nlHierMap> [-caseSensitive <bool>]
```

Arguments

`string`

Hierarchical path name.

`-map`

Hierarchical name map queried.

`-caseSensitive`

Controls whether name comparison is case-sensitive.

Default: `true`.

Returns

Context that matches the hierarchical path name ([heContext](#)).

Errors

Throws an error if no context matching the hierarchical path name is found.

nl::getMappings

Description

Retrieves all of the mappings from the specified name scope. The returned value is a list of source to destination identifier pairs. Note, if the module is finalized the results will not contain identity mappings (mappings where the source and destination identifiers are the same).

This command does not modify the nlNameMap.

The return format is identical to Tcl's format used for "array get" and "array set":

```
array set myarray [nl::getMappings -map <nlNameMap> -module $ctx -scope
Net]
    puts "Mappings in the Net name scope:
    foreach src [array names myarray] {
        puts "\t$src => $mymappings($src) "
    }
```

Syntax

```
nl::getMappings -map <nlNameMap> [-module <heContext>] -scope <string>
```

Arguments

-map

Name map to be queried.

-module

Context specifying the module level tables.

Default: NULL.

-scope

Scope of the identifier: commonly one of "Net", "Module", "Model", "Parameter", "Instance", "Terminal", or "Default".

Returns

The list of source/destination mappings for the specified module and name scope.
([<map>])

nl::getModuleDescription

Description

Generates a standard description of the module including the lib/cell/view and the effective view search list and the Stop List. It uses the passed-in commentPrefix for to customize for the target language.

The equivalent Tcl implementation of the command is as follows:

```
proc nl::getModuleDescription {ctx commentPrefix} {  
    set prefix "$commentPrefix "  
  
    set div "$cchar-----\n"  
    set out $div  
  
    set cv [db::getAttr cellView -of $ctx]  
    set lib [db::getAttr lib -of $cv]  
    set cell [db::getAttr lib -of $cv]  
    set view [db::getAttr lib -of $cv]  
  
    set vsl [db::getAttr viewSearchList -of $ctx]  
    set stop_l [db::getAttr stopList -of $ctx]  
  
    append out "${prefix}Library: [db::getAttr name -of $lib]\n"  
    append out "${prefix}Cell : [db::getAttr name -of $cell]\n"  
    append out "${prefix}View : [db::getAttr name -of $view]\n"  
    append out "${prefix}View Search List: $vsl\n"  
    append out "${prefix}View Stop List: $stop_l\n"  
    append out $div  
  
    return $out  
}
```

Syntax

```
nl::getModuleDescription -module <heContext> -commentPrefix <string>
```

Arguments

`-module`

The module to describe.

`-commentPrefix`

The comment prefix.

Returns

The generated module description. (<string>)

nl::getModuleNameMapForAMS

Returns name map of worker netlister used for the given module in AMS netlisting.

Syntax

```
nl::getModuleNameMapForAMS -netlister <nlAMSNetlister | nlNetlister>  
-context <heContext>
```

Returns

<nlNameMap> : Name map for given module.

Arguments

`-netlister`

Netlister object

`-context`

The context to retrieve the effective netlister for.

nl::getNamesForMapping

Retrieves terminal or net names to be mapped.

Syntax

```
nl::getNamesForMapping -netlister <nlAMSNetlister | nlNetlister> -context  
<heContext> -objectType <enum> -purpose <enum>
```

Returns

<string> : List of names mapping data. The data has format: {<action_type> <first_name>
[<second_name>] [<other_names>]}

Arguments

`-netlister`

The netlister object to query.

`-context`

The context to retrieve the names for.

`-objectType`

The object type (net or terminal). The name of which is retrieved. ({ net | terminal })

`-purpose`

The purpose for which the name mapping is needed. Can be `shorted` for net; `shorted` or `globalForRemoveBang` for terminal. ({ globalForRemoveBang | shorted })

nl::getNetlisters

Description

Returns the collection of netlister objects specified by the given name pattern.

Syntax

```
nl::getNetlisters [<pattern: string>] [-filter <string>]
```

Arguments

`pattern`

Pattern to match netlister name.

Default: `*`.

`-filter`

Standard attribute filter.

Default: match all.

Returns

{<nlNetlister>}: Collection of netlisters with names matching the input pattern.

Examples

```
set nl [nl::getNetlisters HSPICE]
```

In this example, `nl::getNetlisters` retrieves the HPICE netlister object.

nl::getNetMembers

Description

Splits an nlNet into component buses/bundles guaranteeing that all components have oaSimpleName's. Bundles that do not contain repeat operators are broken into constituent buses and scalar nets. If a bundle uses the repeat operator, then some components may still be bundles, but they are guaranteed to have scalar names.

If the `-scalarize` option is true, then only scalar nets are returned.

Syntax

```
nl::getNetMembers <net: nlNet> [-scalarize <bool>]
```

Arguments

net

Queried net.

-scalarize

Return the constituent scalar nets.

Default: false.

Returns

List of bundle or bus members. ([nlNet])

Examples

This function could be used to implement based connectivity for Verilog netlisting. The following Tcl is an example implementation.

```
proc get_inst_term_text {nl it} {  
    set n [db::getAttr net -of $it]  
    set wire_name ""  
    # split into buses  
    foreach nm [nl::getNetMembers $n] {  
        lappend wire_name [get_verilog_net_name $nl $nm]  
    }  
    if {[llength $wire] != 1 } {  
        # more then one member, concatenate  
        set wire_name "[join $wire_name ,]"  
    } else {  
        set wire_name [lindex $wire_name 0]  
    }  
    set term [db::getAttr term -of $it]  
    set conn [map $nl [db::getAttr name -of $term] Term]  
    return ".${conn}($wire_name)"  
}
```

```

proc map {nl str scope}{
    set ctx [db::getAttr module -of $nl]
    set nm [db::getAttr nameMap -of $nl]
    return [nl::calculateMapping -map $nm -add true -src $str - module
    $ctx \
        -scope $scope]
}
proc get_verilog_bit_name {nl net} {
    return [map $nl [db::getAttr name -of $net]]
}
proc get_verilog_bus_name {nl net} {
    set basename [db::getAttr baseName -of $net]
    set bo [nl::getWireBitOrder -wire $basename -netlister $nl]
    set net_bo [db::getAttr bitOrder -of $net]
    if {"none" == $net_bo } {
        nl::setWireBitOrder $net_bo \
            -wire $basename -netlister $nl
        set bo $net_bo
    }
    if {$bo != $net_bo } {
        # reverse part select
        set name ""
        foreach b [nl::getNetMembers $n -scalarize 1] {
            append name [get_verilog_bit_name $nl $b]
            append name ","
        }
        set name [string trimright $name ","]
        return $name
    } else {
        # just call name mapping
        return [map $nl [db::getAttr name -of $net] Net]
    }
}
proc get_verilog_net_name {nl net} {
    switch [db::getAttr type -of $net] {
        "global" {
            # get name for global in global top module
        }
        "anonymous" {
            # anonymous nets in verilog can be blank
            return ""
        }
        "scalar" -
        "bit" {
            return [get_verilog_bit_name $nl $net]
        }
        "bus" {
            return [get_verilog_bus_name $nl $net]
        }
        "bundle" {
            # must have a repeat
            set part [get_verilog_bus_name $nl $net]
            set repeat [db::getAttr repeat -of $net]
    }
}

```

```
        set name ""
        while {0 < $repeat} {
            lappend name $part
            incr repeat -1
        }
        return [join $name ,]
    }
}
```

nl::getNetType

Description

Returns the `netType` property value for the specified net, set directly or calculated based on lower hierarchy.

Syntax

```
nl::getNetType <name: string> -netlister <nlNetlister> -context  
               <heContext>
```

Arguments

`name`

Net name.

`-netlister`

The netlister object.

`-context`

The context in which this command is activated.

Returns

<string> : Net type name to be printed

nl::getParamNames

Description

Retrieves the list of parameters which should be considered for netlisting an instance of the given module. The netlister computes the list of important parameters in two different ways. If the module is a leaf, then the list of parameter names stored in the `NetlistInfo` `instParameters` field is returned.

The results can be filtered by both variable type and by name. If the variable type is `Inst`, then parameters found within the `instParameters` field of the tool's `simInfo/Netlist` info

are returned. If the variable type is Parent, then parameters referenced via the parent() PEL command are returned. If the variable type is Lineage then parameters referenced by the lineage() PEL function are returned. If the variable type is All then the union of all parameter types is returned. Parameters can be both type Inst and type Lineage/Parent. However, the returned list is guaranteed to only contain unique elements.

Syntax

```
nl::getParamNames -netlist <nlNetlist> -master <heContext> [-type  
  <enum>]
```

Arguments

`-netlist`

The netlist used to find the list of applicable parameters.

`-master`

The module to use to determine the list of relevant parameter names.

`-type`

The type of the parameters to return. ({ all | design | inst | lineage | parent }, default is: all)

Returns

<string> : List of parameter names.

Errors

If the `-type` argument is set to design.

Examples

The following example returns parameter names for the specified master object.

```
set params [nl::getParamNames -netlist $nl -master $master]
```

nl::getReferencedVariables

Description

Retrieves a list of the names of parameters referenced via "parent"/"lineage" PEL expressions for a given module. It can also be used to retrieve names of the design variables referenced by a given module. It is only valid to call this function from a Netlisting Event. Also note that the list is populated by monitoring the evaluation of parameters using the `nl::findParamValue` command. As such, the information is not complete until all instances within the given module have been netlisted.

Note that the ModuleHeader event is triggered after the ModuleBody event. As such, it is safe to call `nl::getReferencedVariables` for the current module from within the ModuleHeader event and at any time after.

Note:

At the end of netlisting, this command can be used to get all of the referenced design variables by querying the Design type from the top module of the design.

Syntax

```
nl::getReferencedVariables [<name: string>] -netlister <nlNetlister>  
-master <heContext> [-type <enum>]
```

Arguments

`name`

A glob style regular expression matched against the parameter/variable names.
(Default is: *)

Default: *.

`-netlister`

The queried netlister object.

`-master`

The context containing the variable references.

`-type`

Type of tracked variable ({ all | design | inst | lineage | parent }, default is: all)

Returns

<string> : List of variable names referenced during netlisting of the given heContext.

Errors

- If the command is called outside of netlisting execution.
- If the value of `-type` is Inst.

nl::getShortedNets

Description

Gets the list of shorted net names and their preferred equivalents due to patch cords, ignored/removed devices, and shorted instance pins.

Syntax

```
nl::getShortedNets -netlister <nlNetlister> -context <heContext>
```

Arguments

`-netlister`

The netlister object to query.

`-context`

The context for which to retrieve the shorted nlNets.

Returns

<string> : The list of shorted net names and their preferred equivalent net names in the following format: [list net1 preferred1 net2 preferred2 ...]

nl::getSupplyType

Description

Determines the supply type based on the net name. If the name matches the preference dbSigTypeGroundRegexp, “ground” is returned. If the name matches the preference dbSigTypePowerRegexp, “power” is returned. If the net name matches neither of the patterns or if it matches both of the patterns, the result is “signal”.

Syntax

```
nl::getSupplyType <object: oaBitNet | oaBitTerm | string>
```

Arguments

`object`

The bit net or bit terminal or name to check.

Returns

<string> : The type of the net based on the name.

nl::getTerms

Description

Returns the ordered nlTerms for a given context. The netlisting framework will calculate scalarized nlTerms if the scalarizeTerms setting is true.

Syntax

```
nl::getTerms [<pattern: string>] -netlister <nlNetlister> -context  
             <heContext> [-filter <string>]
```

Arguments

pattern

Pattern to match term name.

Default: *.

-netlister

The netlister object to query.

-context

The context for retrieving the list of nlTerms.

-filter

Standard attribute filter. Set `nl::getTerms -filter "%isVirtual == 1` to retrieves all virtual terminals of the given context.

Default: match all.

Returns

The list of nlTerms. Note that the order is important and is guaranteed match the order of the nlInstTerms returned by `nl::getInstTerms`. ([[nlTerm](#)])

Examples

The following example returns an ordered list of nlTerm objects for the given context, which in this case is the `$ctx` object.

```
set cellPorts [nl::getTerms -context $ctx -netlister $nl]
```

nl::getWireBitOrder

Description

Finds the wire bit order according to the terminals of the context. If the supplied wire name is not a terminal then it looks in a per-context table stored within the netlister. The table can be modified via `nl::setWireBitOrder`. If the bit order for the given wire name has not been specified by `nl::setWireBitOrder`, then OA's BusTermDef and BusNetDef's are used to determine the bit order. If a BusTermDef exists, it takes precedence over BusNetDef's.

OA sets the Bus*Def's bit order according to the usage in the design. If the bus is used in both descending and ascending order, then OA sets the Bus*Def's value to "none". As such "none" may be returned by this command.

This command can be used in languages for which bit order must be maintained for wiring connections. For example in Verilog if a wire is defined as "foo[0:3]" then it is illegal to use "foo[3:0]" in a connection. The individual bits must be concatenated: "{foo[3],foo[2],foo[1],foo[0]}".

Syntax

```
nl::getWireBitOrder -wire <string> -netlister <nlNetlister> -module  
                    <heContext>
```

Arguments

-wire

The basename of the wire.

-netlister

The netlister to query.

-module

The context to query.

Returns

The bit order of the given wire name. (*oaBitOrder*)

Errors

- If the supplied name is not a scalar name.
- If the supplied net name does not have an oaBusNetDef or oaBusTermDef.

nl::getWorkerNetlisters

Description

Returns the list of child netlisters for the given AMS netlister.

Syntax

```
nl::getWorkerNetlisters -netlister <nlAMSNetlister> [-view <string>]
```

Arguments

-netlister

The netlister to use for the given view.

`-view`

The view name.

Returns

{<nlNetlister>} : Collection of worker netlisters associated with the given AMS netlister.

nl::indent

Description

Adjusts the left indent margin of the netlister's output buffer by the number of spaces stored in the netlister's `-indentDepth` attribute. If the current cursor location is at the beginning of a line, then the cursor will be adjusted to the new indentation depth. Otherwise the current cursor location will not be modified.

Syntax

```
nl::indent <string: string> -netlister <nlNetlister>
```

Arguments

`string`

The script to execute.

`-netlister`

The netlister to have its buffer indented.

Examples

The following example adjusts the left margin indent of the netlister's output buffer by the number of spaces stored in the netlister's `indentDepth` attribute.

```
nl::indent "Text with indent in netlist" -netlister $nl
```

nl::instHeaderHandler

Description

Processes an instance header. It first determines the netlist procedure to use by calling `nl::findNetlistProcedure`. Next, it determines the instance name prefix by calling `nl::findInstNamePrefix`. It then optionally scalarizes the instances bound to the instance header depending on the `nlNetlister`'s `-scalarizeInsts` attribute. Finally, it calls the netlist procedure for each `oaBitInst`.

The order of instance processing is controlled by the `nlNetlister`'s `-sortInsts` option.

Syntax

```
nl::instHeaderHandler <netlist: nlNetlist> <context: heContext>  
  <header: heInstHeader>
```

Arguments

`netlist`

The netlist object.

`-context`

The module being netlisted.

`-header`

The instance header being processed.

Returns

<none>

nl::isLeaf

Description

Checks the `-leafList` attribute of the passed in netlist, and returns true if the given context is contained in the specified netlist's list of leaf contexts.

Syntax

```
nl::isLeaf -netlist <nlNetlist> -master <heContext> [-domain <enum>]
```

Arguments

`-netlist`

The netlist object.

`-master`

The switched master of the given instance.

`-domain`

Optionally check to see if the master is bound to the given domain. This check is only applied in AMS netlisting. ({ analog | any | digital }, default is: any)

Returns

<bool>: Boolean.

nl::isModuleUpToDate

Description

Default implementation for the isModuleUpToDate event.

Syntax

```
nl::isModuleUpToDate <netlist: nlNetlist> <context: heContext>
```

Arguments

netlist

The netlist object.

-context

The module object.

Returns

<bool>: True if the module is up-to-date, otherwise false.

nl::isNetNameGlobal

Description

Determines if the given net name is global anywhere in the design hierarchy.

Syntax

```
nl::isNetNameGlobal <net_name: string> -netlist <nlNetlist>
```

Arguments

net_name

The name of the net to check.

-netlist

The netlist object.

Returns

True if the passed in net name is global anywhere in the design hierarchy. (<bool>)

Examples

The following example determines whether the given net name `vout` is global anywhere in the design hierarchy.

```
set isGlob [nl::isNetNameGlobal "vout" -netlist $nl]
```

nl::isStringParam

Description

Queries the netlister to determine if the given parameter name is a string parameter. The parameter is considered to be a string parameter if it is in the netlistInfo's stringParameters list or if it is a passed parameter that is referenced by a string parameter.

The netlister tracks string parameters and string parameter references.

Syntax

```
nl::isStringParam <param_name: string> -netlister <nlNetlister> -master  
  <heContext>
```

Arguments

`param_name`

The name of the parameter to lookup.

`-netlister`

The netlister used to track string parameters.

`-master`

The master used to determine if the named parameter is a string parameter.

Returns

If the given parameter is a string parameter or is used as input to a string parameter within the passed in master's design hierarchy. (<bool>)

nl::isTextModule

Description

Determines if the passed-in module is a text module. The name of a text module cannot be modified and thus must not be name mapped. It is also important for some formatters to know if a module is a text module in order to track text files which should be included in the final netlist.

The following view OA types will be considered "text views":

`oacVerilogAMSText`

`oacVHDLAMSText`

`oacVerilogText`

`oacVHDLText`

oacVerilogAText

[Link to OA Reserved View Types.](#)

This is the default implementation for the allowNameMapProc field in the nlNetlister object.

Syntax

```
nl::isTextModule <context: heContext>
```

Arguments

context

The module inspected.

Returns

True if the passed in module is a text module, false otherwise. (<bool>)

nl::lookup

Description

Queries the name map tables to determine if a mapping is stored for the passed-in identifier.

This command is typically only used in name map creation (that is, during netlisting). Typically `nl::map` should be used instead. If the name map data is available the `nl::map` command is used to retrieve mapped identifiers.

This command does not modify the nlNameMap.

Syntax

```
nl::lookup -map <nlNameMap> [-src <string>] [-dest <string>] [-module  
<heContext>] -scope <string>
```

Arguments

-map

Name map object to be queried.

-src

Database namespace identifier to be mapped to destination namespace. If you provide the `-src` option, the table mapping source names to the destination names are checked for the identifier.

`-dest`

Destination namespace identifier to be reverse mapped to the database namespace. If you provide the `-dest` option, the table mapping destination names to the source names are checked for the identifier.

`-module`

Context specifying the module level tables.

Default: NULL.

`-scope`

Scope of the identifier: commonly one of "Net", "Module", "Model", "Parameter", "Instance", "Terminal", or "Default".

Returns

Text associated with the input identifier. If the input identifier is not in the name map tables, an empty string is returned (<string>).

Errors

Either the `-src` or `-dest` argument must be provided but not both, otherwise an error will result.

nl::map

Description

Maps an identifier from either the database namespace to the destination namespace or vice versa. This command does not modify the name map.

If the name map is algorithmic, this command does not check tables; instead the name map returns only calculated names. If the name map is table-based (non-algorithmic), the command checks only the tables for the passed-in identifier.

This command also provides vectored instance and net support.

You must use `-src` to map the argument string in to the destination namespace, or use `-dest` to reverse-map the argument string into the database namespace.

Syntax

```
nl::map -map <nlNameMap> [-src <string>] [-dest <string>] [-module  
  <heContext>] -scope <string>
```

Arguments

`-map`

Name map to be queried.

`-src`

Database namespace identifier to be mapped to destination namespace.

`-dest`

Destination namespace identifier to be reverse mapped to the database namespace.

`-module`

Context specifying the module level tables.

Default: NULL.

`-scope`

The scope of the identifier. Commonly one of "Net", "Module", "Model", "Parameter", "Instance", "Terminal", or "Default". For the module scope, the global tables before the module specific tables is checked.

Returns

Input identifier mapped into the correct namespace. The return value is never empty.
(<string>)

Errors

Either the `-src` or `-dest` argument must be provided but not both, otherwise an error will result.

Examples

The following example adds an exclamation point (!) identifier to the destination namespace for the specified `$ctx` context object within the scope of the currently selected net.

```
set map [nl::map -map $mapObj -src "!" -module $ctx -scope Net]
```

nl::mapModuleNames

Description

Fills the name mapping table for the module names: first the texts, then the leafs, then the hierarchical modules. Should be called only from the netlisting callbacks.

Syntax

```
nl::mapModuleNames <nlNetlister: nlNetlister> [-mapParts <bool>]  
  [-mapLeafs <bool>] [-mapTexts <bool>] [-prefix <string>] [-suffix  
  <string>] [-skipAffixPatterns <string>]
```

Arguments

`nlNetlister`

The current netlister.

`-mapParts`

If true, maps the hierarchical module names. (Default is: 1)

`-mapLeafs`

If true, maps the leaf module names. (Default is: 1)

`-mapTexts`

If true, maps the text module names. (Default is: 1)

`-prefix`

Prefix to add. (Default is:)

`-suffix`

Suffix to add. (Default is:)

`-skipAffixPatterns`

List of wildcard patterns to exclude from suffix/prefix additions when mapping.
(Default is:)

Returns

<none>

nl::mapPath

Description

Returns the mapped hierarchical path.

Syntax

```
nl::mapPath -map <nlHierMap> [-src <string>] [-dest <string>] -scope  
  <string>
```

Arguments

`-map`

Hierarchical name map to be queried.

`-src`

Database namespace identifier to be mapped to destination namespace.

`-dest`

Target namespace identifier to be reverse mapped to the database namespace.

`-scope`

Scope of the identifier. Commonly one of "Net", "Module", "Model", "Parameter", "Instance", "Terminal", or "Default".

Returns

Mapped hierarchical path (<string>).

Errors

Either the `-src` or `-dest` argument must be provided but not both, otherwise an error will result.

nl::mapTerminalNames

Maps all terminal names in all modules.

Syntax

```
nl::mapTerminalNames -netlister <nlAMSNetlister | nlNetlister>
```

Returns

<none>

Arguments

`-netlister`

Netlister object.

nl::message

Description

Logs error, warning, and informational messages generated by the netlister. If an error is logged then the netlisting continues until either the job completes or the maximum number

of errors is encountered. The end result of the netlisting job is flagged as a failure. If the `-severity abort` is specified, then the netlisting job immediately ends and the result becomes a netlisting failure.

Syntax

```
nl::message <message: string> -netlister <nlNetlister> [-severity <enum>]
```

Arguments

`message`

The message to be reported.

`-netlister`

The netlister object used to log messages.

`-severity`

The severity of the message, error is default. ({ abort | error | info | warning }, default is: error)

Returns

<none>

Examples

The following example prints the specified error message from the netlister.

```
nl::message "This is an Error" -netlister $nl -severity "error"
```

The following example prints the specified warning message from the netlister.

```
nl::message "This is a Warning" -netlister $nl -severity "warning"
```

nl::moduleBodyHandler

Description

Iterates over the instance headers of the given module context invoking `InstHeader` events. It optionally sorts the instance headers by the switched `lib/cell/view` name. The sorting is controlled by the passed-in `nlNetlister`'s `-sortInstHeaders` setting. (This is the default event handler registered as the `-moduleBodyProc` attribute for netlisters. It typically does not require re-implementation by netlister authors.)

Syntax

```
nl::moduleBodyHandler <netlister: nlNetlister> <context: heContext>
```

Arguments

`netlister`

The netlister object.

`-context`

The module being netlisted.

Returns

<none>

nl::moduleHandler

Description

Generates events for ModulePreprocess, ModuleHeader, ModuleBody, and ModuleFooter. The order of event generation is preprocess, body, header, and footer, as required by parameter aggregation. This command also handles the re-ordering of netlisting output to match the expected order of: preprocess, header, body, footer. (This is the default event handler registered as the `-moduleProc` attribute for netlisters. It typically does not require re-implementation by netlister authors.)

If the module is determined to be a text module, then only TextModule will be emitted. The command determines if the module is a text module by calling the command registered in the `-isTextModule` attribute of the passed in nlNetlister object.

The equivalent Tcl implementation of the command is as follows:

```
proc nl::moduleHandler {nl ctx} {
  nl::generateEvent "ModulePreprocess" -netlister $nl -module $ctx
  set preprocess [nl::getBuffer -netlister $nl]
  nl::clearBuffer -netlister $nl
  nl::generateEvent "ModuleBody" -netlister $nl -module $ctx
  set body [nl::getBuffer -netlister $nl]
  nl::clearBuffer -netlister $nl
  nl::append $preprocess -netlister $nl -wrap 0
  nl::generateEvent "ModuleHeader" -netlister $nl -module $ctx
  nl::append $body -netlister $nl -wrap 0
  nl::generateEvent "ModuleFooter" -netlister $nl -module $ctx
}
```

Syntax

```
nl::moduleHandler <netlister: nlNetlister> <context: heContext>
```


Arguments

`netlister`

The netlister object.

`-context`

The module to netlist.

Returns

<none>

nl::modulePreprocessHandler

Description

Appends the textual description of the module obtained from `nl::getModuleDescription` to the netlister's output buffer. This is the default event handler registered as the `-modulePreprocessProc` attribute for netlisters.

The equivalent Tcl implementation of the command is as follows:

```
proc nl::modulePreprocessHandler {netlister module} {  
    set comment_prefix [db::getattr commentPrefix \  
-of $netlister]  
    nl::append [nl::getModuleDescription -module $module]\  
-commentPrefix $comment_prefix]  
-wrap false  
}
```

Syntax

```
nl::modulePreprocessHandler <netlister: nlNetlister> <context: heContext>
```

Arguments

`netlister`

The netlister object.

`-context`

The module object.

Returns

<none>

nl::netlistBeginHandler

Description

Insures that module names are correctly mapped by calling `nl::preloadModuleNames`. This is the default event handler registered as the `-netlistBeginProc` attribute for netlisters. It typically does not require re-implementation by netlister authors.

The equivalent Tcl implementation of the command is as follows:

```
proc nl::netlistBeginHandler {netlister} {  
  nl::preloadModuleNames -netlister $netlister  
}
```

Syntax

```
nl::netlistBeginHandler <netlister: nlNetlister>
```

Arguments

netlister

The netlister object passed to `nl::preloadModuleNames`.

Returns

<none>

nl::netlistBodyHandler

Description

Walks the parts list stored in the `-partsList` flag of the `nlNetlister` object and calls the structural event `NetlistModule` for each module. Each module will be written to a separate file. The `nlNetlister` setting `-createTopAsModule` controls whether the `ModuleHeader/ModuleFooter` events are generated for the top module. This is the default event handler registered as the `-moduleBodyProc` attribute for netlisters. It typically does not require re-implementation by netlister authors.

Syntax

```
nl::netlistBodyHandler <netlister: nlNetlister>
```

Arguments

netlister

The netlister queried.

Returns

<none>

nl::netlistEndHandler

Description

Walks and concatenates the output files generated in the NetlistBody event into the final netlist file. It also saves the name mapping tables alongside the netlist file. The nlNetlister attribute `-moduleOrder` controls the order in which modules appear in the final netlist. This is the default event handler registered as the `-netlistEndProc` attribute for netlisters.

Syntax

```
nl::netlistEndHandler -netlister <netlister: nlNetlister>
```

Arguments

`-netlister`

The netlister object.

Returns

<none>

nl::netlistMainHandler

Description

Functions as the default event handler registered as the `-netlistMainProc` attribute for netlisters. It typically does not require re-implementation by netlister authors. This command generates NetlistBegin, NetlistBody and NetlistEnd events.

The equivalent Tcl implementation of the command is as follows:

```
proc nl::netlistMainHandler {netlister} {  
    nl::generateEvent "netlistBegin" -netlister\ $netlister  
    nl::generateEvent "netlistBody" -netlister\ $netlister  
    nl::generateEvent "netlistEnd" -netlister\ $netlister  
}
```

Syntax

```
nl::netlistMainHandler <netlister: nlNetlister>
```

Arguments

`netlister`

The netlister used to generate events.

Returns

<none>

nl::nlPrintCdlInst

Description

Handles the (non-leaf) subcircuit's instantiations. It is called from the `nl::nlPrintCdlInstance` procedure.

The `nl::nlPrintCdlInst` procedure can be directly called from another netlisting procedure. In order to be used as a netlisting procedure, it should be set in the `simInfo/auCdl/netlistProcedure`.

Syntax

```
nl::nlPrintCdlInst <nlNetlister> <oaInst> <master> <instNamePrefix>  
[<params>] [<parent>]
```

Arguments

`nlNetlister`

The netlister.

`oaInst`

The instance object.

`master`

The instance master context.

`instNamePrefix`

Instance name prefix.

`params`

Parameters list.

`parent`

Parent context.

Returns

<none>

nl::nlPrintCdlInstance

Description

Default netlisting procedure will be called if simInfo/auCdl/netlistProcedure is not defined for a module.

The `nl::nlPrintCdlInstance` procedure can be directly called from another netlisting procedure. In order to be used as a netlisting procedure it should be set in `simInfo/auCdl/netlistProcedure`.

Syntax

```
nl::nlPrintCdlInstance <nlNetlister> <oaInst> <master> <instNamePrefix>  
    [<params>] [<parent>]
```

Arguments

`nlNetlister`

The netlister.

`oaInst`

The instance object.

`master`

The instance master context.

`instNamePrefix`

Instance name prefix.

`params`

Parameters list.

`parent`

Parent context.

Returns

<none>

nl::openNameMap

Description

Loads the name map files into memory.

Syntax

```
nl::openNameMap -filePath <string>
```

Arguments

`-filePath`

Directory path where name map files are stored. The `-filePath` argument accepts both tilde and environment variables embedded within the path.

Returns

New name map with the contents of the directory loaded. ([nlNameMap](#))

Errors

- If the directory does not exist.
- If the directory does not contain valid name map object files.

Examples

The following example loads the map file `map_file.txt` into memory.

```
set nameMap [nl::openNameMap -filePath map_file.txt]
```

nl::preloadModuleNames

Description

Scans the netlister Parts and Leaf Lists and ensures that text views are not name mapped by installing entries in the netlister's name mapping tables for all Text Views.

Syntax

```
nl::preloadModuleNames -netlister <netlister: nlNetlister>
```

Arguments

`-netlister`

The netlister scanned for text views.

Returns

<none>

nl::printSpectreInst

Description

Composes the instance line for Spectre netlister.

Syntax

```
nl::printSpectreInst -netlister <nlNetlister> -inst <oaInst> -master  
<heContext> [-format <string>] [-subParam <string>] [-defaultParams  
<bool>] [-filterParamsByCDFUse <bool>] [-returnString <bool>]
```

Arguments

-netlister

The netlister object.

-inst

The netlisted instance.

-master

The master of the given instance.

-format

The format of instance line.

-subParam

Subparams list.

-defaultParams

Write default params. (Default is: true)

-filterParamsByCDFUse

If true, do not print the CDF parameters which `use` attribute is false. If false, ignore the `use` value.

-returnString

Compose return string instead of printing. (Default is: false)

Returns

<string> : Contents of the printed string.

nl::printSpiceInst

Description

Determines the Spice instance line.

Syntax

```
nl::printSpiceInst -netlist <nlNetlist> -inst <oaInst> -master  
  <heContext> [-format <string>] [-subParam <string>] [-defaultParams  
  <bool>] [-filterParamsByCDFUse <bool>] [-returnString <bool>]  
  [-wrapQuotesForParamValue <bool>]
```

Arguments

-netlist

The netlist object.

-inst

The netlisted instance.

-master

The master of the given instance.

-format

The format of instance line.

-subParam

Subparams list.

-defaultParams

Write default params. (Default is: true)

-filterParamsByCDFUse

If true, do not print the CDF parameters which `use` attribute is false. If false, ignore the `use` value.

-returnString

Composes return string instead of printing. The default is false.

-wrapQuotesForParamValue

Wrap parameter value with single quotes (")when printing the HSPICE netlist result. (Default is: true)

Returns

<string>: The contents of the printed string.

nl::printSpiceInstTerms

Description

Prints the instance terminals connections notation as a part of the SPICE-style instance line.

Syntax

```
nl::printSpiceInstTerms -netlister <nlNetlister> -inst <oaInst> -master  
  <heContext> [-maxNumber <int>]
```

Arguments

-netlister

The netlister object.

-inst

The netlisted instance.

-master

The switched context of the given instance.

-maxNumber

The maximum number of printed connections. A value of -1 means no limitation.
The default is -1.

Returns

<none>

nl::refreshPartitioner

Description

Recalculates cached partitioning within the partitioner object.

Syntax

```
nl::refreshPartitioner <name: nlPartitioner>
```

Arguments

name

Partitioner to be updated.

Returns

<none>

nl::revertOutdatedDesigns

Description

Reverts read-only designs that have changed on disk since they were loaded into memory. If you open a design in read-only mode and someone else opens it for editing and changes it, you have an outdated version. This command reverts your outdated version to the current version, retaining read-only mode. In the case where an outdated design is open in a design editor, the command will prompt to confirm.

Syntax

```
nl::revertOutdatedDesigns
```

Returns

<bool>: Returns true if OK is clicked. False if Cancel is clicked.

nl::runNetlister

Description

Runs the netlister on the specified top cellView. The final netlist is stored at the specified -filePath. Associated name mapping files are stored with the final netlist.

Incremental netlisting can accelerate netlisting speed by re-using unchanged portions of previous netlisting runs. Incremental netlisting is off by default. It can be enabled by using the -reference argument and specifying the location of a previous netlisting run.

Syntax

```
nl::runNetlister <netlister: nlAMSNetlister | nlNetlister> -cellView  
  <dmCellView | dmCellViewName> -filePath <string> [-viewSearchList  
  [<string>]] [-viewStopList [<string>]] [-reference <string>]  
  [-writeNameMapping <bool>]
```

Arguments

netlister

Netlister object.

-cellView

CellView to netlist.

-viewSearchList

View search list to use for design elaboration (not used if the top cellView is a config view).

Default is: "".

`-viewStopList`

View stop list to use for design elaboration (not used if the top cellView is a config view).

Default is: "".

`-filePath`

Main output filename for the final netlist data. The parent directory must be write-enabled.

`-reference`

Reference path for data to be used for incremental netlisting.

`-writeNameMapping`

Saves the name mapping data to the target directory. The default is true.

Returns

<bool> : Completion status of netlisting job - true if complete.

Errors

If the output directory is not write-enabled by the netlisting process.

Examples

The following example invokes the netlisting process, passes in a previously created nlNetlister object and a top cell view, a view search list, and a view stop list.

```
set cv [dm::findCellView $viewName -cellName $cellName \  
    -libName $libName]  
nl::runNetlister $nl -cellView $cv -filePath ./netlist.sp \  
    -viewSearchList "hspice schematic" -viewStopList "hspice"
```

nl::saveModuleState

Description

Default implementation for the saveModuleState event.

Syntax

```
nl::saveModuleState <netlister: nlNetlister> <context: heContext>
```

Arguments

`netlister`

The netlister object.

`-context`

The module object.

Returns

<bool>: True if the data saving passed successfully, otherwise false

nl::saveNameMap

Description

This command writes a name map object to disk for future use or use by other sub systems.

Syntax

```
nl::saveNameMap <map: nlNameMap | nlOANameMap | nlParameterizedNameMap>  
-filePath <string>
```

Arguments

`map`

Name map to save to disk.

`-filePath`

File path of the directory for saved name map files. The `-filePath` argument accepts both tilde and environment variables embedded within the path.

Returns

<none>

nl::setTermOrder

Description

Sets the default terminal order for the given context. The terminal order is affects the following:

- Formatting terminal/port declarations in hierarchical modules.
- Formatting net connectivity on instances of the given context.

Note:

The order of instance terms can be overridden by creating a portOrder property on individual instances.

Syntax

```
nl::setTermOrder <termNames: string | {<nlTerm>}> -netlister  
  <nlNetlister> -of <heContext> [-appendVirtualTerminals <bool>]
```

Arguments

termNames

The ordered terminals or terminal names.

-netlister

The netlister instance.

-of

The context for the specified terminal order.

-appendVirtualTerminals

Append the default virtual terminals to the specified terminal order. (Default is: true).

Returns

<none>

Examples

The following example overrides the default order of terminals for HSPICE netlister from "<input terminals> <inout terminals> <output terminals>" to "<inout terminals> <output terminals> <input terminals>"

Default:

```
.subckt block1 in1 in2 inout<0> inout<1> inout<2> out1 out2  
.ends block1
```

After applying customization:

```
.subckt block1 inout<0> inout<1> inout<2> out1 out2 in1 in2  
.ends block1
```

nl::setWireBitOrder

Description

Sets the bit order for a wire in the given context. This command is only necessary for formats that restrict wire connections to follow the bit order used to define the wire (Verilog).

Syntax

```
nl::setWireBitOrder <oaBitOrder> -wire <string> -netlister <nlNetlister>  
-module <heContext>
```

Arguments

`oaBitOrder`

The new bit order of the given wire name.

`-wire`

The basename of the wire.

`-netlister`

The netlister to modify.

`-module`

The module where the wire name exists.

Returns

<none>

nl::uniqueify

Description

Calculates alternate names for identifiers if there are mapped name conflicts.

Users can override the default behavior of `nl::uniqueify` by providing a procedure name for the `-uniqueifyProc` argument at name map creation time.

If a user-defined procedure is not provided when creating the name map, the following is used:

```
proc permute {nm name index} {  
    if {0 == $index } {  
        return $name
```

```
}

return "${name}_${index}"

}
```

The following example illustrates the use of a custom unique function:

Assume the token "\$a" is mapped to "_a" algorithmically, and nl::calculateMapping calculates that the token "!a" should also be mapped to "_a". This is a scope conflict. nl::calculateMapping then calls nl::uniqueify with increasing indexes until it returns an identifier, which does not conflict with the previously mapped tokens.

Any user-defined procedure for resolving mapped name conflicts must ensure that if a valid destination namespace identifier is passed in, a valid destination namespace identifier is returned.

Syntax

```
nl::uniqueify <identifier: string> -index <int>
```

Arguments

identifier

Identifier to compute.

-index

Index used to compute a unique identifier.

Returns

Unique string (<string>).

nl::writeUPF

Description

Creates UPF (Unified Power Format) from the specified power supply information (nlICCSupplyInfo).

Syntax

```
nl::writeUPF <nlICCSupplyInfo: nlICCSupplyInfo> -filePath <string>
[-overwrite <bool>] [-map <nlNameMap>]
```

Arguments

nlICCSupplyInfo

The supply information.

`-filePath`

The file to create.

`-overwrite`

If set to true overwrite existing files.

Default: 1.

`-map`

The name map object used to make instance names consistent between the UPF file and the associated Verilog netlist. The `Instance` scope is used for mapping `-element` identifiers.

Returns

<string>: The absolute path to the generated UPF file.

Enumerations

Table 156 Enumerations

Name	Values	Description
enEventType	netlistMain netlistBegin netlistBody netlistEnd module modulePreprocess moduleHeader moduleBody moduleFooter instHeader	The types of Events generated by the netlister.
enNetlisterType	hspice verilog user	The type of netlister object to create.
enSeverity	info warning error abort	The severity of the reported message/probe. Corresponds to OA enums.
enSpecialChar	delete badChar	Used specifying special states in character maps.
enVariableType	all inst parent lineage design	The types of parameters tracked by the netlisting framework's Parameter Aggregation.

Objects

This section covers the following:

- [dmCell](#)
- [dmCellView](#)
- [heContext](#)
- [heInstHeader](#)
- [nlAMSNetlister](#)
- [nlHierMap](#)
- [nlInstTerm](#)
- [nlNameMap](#)
- [nlNet](#)
- [nlNetlister](#)
- [nlOANameMap](#)
- [nlParameterizedNameMap](#)
- [nlTerm](#)
- [oaInst](#)

dmCell

A named container, in a library, that can hold files and dmCellViews. The dmCell corresponds to a specific oaCell object but is valid as long as the library that the dmCell belongs to remains open. The getAccess requirements of an oaCell are handled automatically by this dm object.

Table 157 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the Cell.
libName	string	Read-only	The name of the Cell's library.

dmCellView

The dmCellView is a container in a library that holds files and is identified by the combination of its cell and view names. It corresponds to an oaCellView but it stays valid as long as its library remains open and its dmCell and dmView exist.

Table 158 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the view that this cellView belongs to.
cellName	string	Read-only	The name of the cell that this cellView belongs to.
cell	<i>dmCell</i>	Read-only	The cellView's cell.
libName	string	Read-only	The name of the library that this cellView belongs to.

heContext

The heContext represents a hierarchical iterator. It can be used to traverse Placed, Switched, and View Search List hierarchies.

Table 159 *Attributes*

Name	Type	Access	Description
topCellView	<i>dmCellView</i>	Read-only	The root Cellview of the context.
config	<i>heConfig</i>	Read-only	The heConfig object the context is traversing. Can be empty.
occurrence	string	Read-only	The current descent lineage of the context in the oaCdbaNS namespace.
design	<i>oaDesign</i>	Read-only	The current design.
viewSearchList	[string]	Read-only	The effective View Search List for the current descent path.
viewStopList	[string]	Read-only	The list of view names used as the stop list in he::isLeaf.

Table 159 Attributes (Continued)

Name	Type	Access	Description
cellView	<i>dmCellView</i>	Read-only	The current cell view.
isRoot	bool	Read-only	True if the context is at the root of the hierarchy (occurrence == "/").
parentInst	<i>oaInst</i>	Read-only	The parent instance of the context.
explicitParentInst	<i>oaInst</i>	Read-only	The explicit parent instance. If the parent instance is an implicit instance then the containing explicit <i>oaVectorInst</i> will be returned.

helInstHeader

The *helInstHeader* is the instance header object for the Hierarchy API. It provides data and methods for determining the placed master, the switched master and the associated instances.

Table 160 Attributes

Name	Type	Access	Description
placedMaster	<i>dmCellView</i>	Read-only	The placed master of this instance header.
switchedMaster	<i>dmCellView</i>	Read-only	The switched master of this instance header.

nIAMSNetlister

Provides the framework for registering custom event handling Tcl procedures. For each structural event as defined in Events and Handlers, a Tcl procedure can be registered by setting the corresponding <event name>Proc attribute to the name Tcl procedure. If the value registered for a procedure is the empty string ("") then the event will be skipped. The netlister framework also uses the values of attributes stored in the *nIAMSNetlister* object to control framework functionality such as global net collection and feedthrough removal. Netlister objects are typically created at tool startup and a single object is created for each output format. A single netlister object can be used to create all the netlists for a specific

output format in a Custom Compiler session. The attributes of the object are described in the table below and can be retrieved or set by the `db::getAttr` and `db::setAttr` commands.

Table 161 Attributes

Name	Type	Access	Description
name	string	Read-only	AMS netlister Name.
masterNetlister	string	Read/Write	Specifies the master netlister.
nameMapProc	string	Read/Write	The name of the Tcl procedure called to create a new name mapping object.
netlistEndProc	string	Read/Write	The name of the Tcl procedure called at the end of AMS netlisting.
bindings	{<nIAMSBinding>}	Read/Write	The collection of bindings: nINetlister name to list of cellView names.
AMSViewType Bindings	{<nIAMSBinding>}	Read/Write	The collection of bindings: nINetlister name to list of view type names.

nIHierMap

The nIHierMap object provides an interface for transforming hierarchical references between namespaces.

Table 162 Attributes

Name	Type	Access	Description
nameMap	nINameMap	Read-only	The nINameMap used for performing name transforms.

nIInstTerm

Represents the connection from an nINet in a parent context to an nITerm in a child context.

Table 163 Attributes

Name	Type	Access	Description
term	<i>nITerm</i>	Read-only	The terminal definition for this instance terminal.
net	<i>nINet</i>	Read-only	The net connecting to this instance terminal.

nINameMap

The nINameMap object provides utilities and a common interface for creating, storing and retrieving mapping data, including serialization/de-serialization to or from disk and runtime storage and retrieval of name table data. nINameMap does not provide algorithms for transforming identifiers between namespaces. The algorithms for transforming names are separate code and plug into the name map in a generic fashion.

See nIONameMap and nIParameterizedNameMap for implementations of the nINameMap interface.

Table 164 Attributes

Name	Type	Access	Description
algo	bool	Read-only	If the new name map is algorithmic or if it contains table data.
openBrace	string	Read-only	Character used to indicate the beginning of a bit index in the destination namespace.
closeBrace	string	Read-only	Character used to indicate the end of a bit index in the destination namespace.
hierDelimiter	string	Read-only	The hierarchy delimiter/separator used in the destination namespace by nIHierMap objects.
rootContext	<i>heContext</i>	Read-only	The context object used to elaborate the design hierarchy by nIHierMap objects.
uniqueifyProc	string	Read-only	The procedure used in the write API to make identifiers unique when scope conflicts occur.

nlNet

Represents nets, buses and bundles in the Simplified Connectivity API. Due to virtual terminals, some referenced nets may not exist in the given module. Furthermore, this object can represent an "anonymous" net. This is needed for over-declared wires in Verilog. The names for anonymous nets are automatically generated and are guaranteed to not connect by name to any other net in the design or to any globals throughout the design hierarchy.

Table 165 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the net/bus/bundle.
netType	<i>enNetType</i>	Read-only	The type of this net.
minIndex	int	Read-only	The Minimum numeric index of the net. Only valid for Bus type nets. This corresponds to the first bit in the bus if the bit order is ascending.
maxIndex	int	Read-only	The maximum numeric index of the net. Only valid for Bus type nets. This corresponds to the last bit in the bus if the bit order is ascending.
repeat	int	Read-only	The number of times the bit/bus/bundle name is repeated.
bitOrder	<i>oaBitOrderEnum</i>	Read-only	The bit order of the bus. Only valid for Bus type nets. This attribute will always be either ascending or descending. It will never be none.
width	int	Read-only	The number of members of this net. This is computed as: $\text{repeat} * (\text{maxIndex} - \text{minIndex} + 1)$.
index	int	Read-only	The index of this bit. Only valid for Bit type nets.

nlNetlister

Provides the framework for registering custom event handling Tcl procedures. For each structural event as defined in Events and Handlers, a Tcl procedure can be registered by setting the corresponding <event name>Proc attribute to the name Tcl procedure. If the value registered for a procedure is the empty string ("") then the event will be skipped. The netlister framework also uses the values of attributes stored in the nlNetlister object to control framework functionality such as global net collection and feedthrough removal. Netlister objects are typically created at tool startup and a single object is created for each output format. A single netlister object can be used to create all the netlists for a specific output format in a Custom Compiler session. The attributes of the object are described in the table below and can be retrieved or set by the `db::getAttr` and `db::setAttr` commands.

Table 166 Attributes

Name	Type	Access	Description
name	string	Read-only	Netlister Name.
collectGlobals	bool	Read/Write	Specifies whether all global nets should be collected before the Parts List is processed. This is needed for Spice, but not needed for Verilog.
supportsFeedThrou ghs	bool	Read/Write	Specifies whether the formatter supports feed-throughs. If it does not support feed-throughs, then the netlister will handle shorted terminals by installing name aliases in the name mapping object and removing all but one of the shorted terminals from the nlTerm list returned by nl::getTerms.
scalarizeTerminals	bool	Read/Write	Specifies whether the netlister should create scalarized terminal lists.
processNetTypes	bool	Read/Write	Specifies whether to process the netType property of nets in the hierarchy. This is false by default, and set true for Verilog to support SystemVerilog net types.

Table 166 Attributes (Continued)

Name	Type	Access	Description
commentPrefix	string	Read/Write	The prefix to use for single line comments.
indentDepth	int	Read/Write	The number of spaces to use for each level of indentation.
maxLineLength	int	Read/Write	The maximum number of characters to print before wrapping a line.
lineWrapSuffix	string	Read/Write	The string to use to wrap long lines.
netlistInfo	string	Read/Write	The identifier used for querying the Netlist Info for netlisting information. This is used by various netlisting utility functions such as nl::findInstNamePrefix, nl::getTerms and any other logic that requires Netlist Info data.
nameMapProc	string	Read/Write	The name of the Tcl procedure called to create a new name mapping object.
netlistMainProc	string	Read/Write	The name of the Tcl procedure called for NetlistMain events.
netlistBeginProc	string	Read/Write	The name of the Tcl procedure called for NetlistBegin events.
netlistBodyProc	string	Read/Write	The name of the Tcl procedure called for NetlistBody events.
createTopAsModule	bool	Read/Write	Controls whether ModuleHeader/ModuleFooter events are generated for the top module.
modulePreprocessProc	string	Read/Write	The name of the Tcl procedure called for ModulePreprocess events.
moduleHeaderProc	string	Read/Write	The name of the Tcl procedure called for ModuleHeader events.

Table 166 Attributes (Continued)

Name	Type	Access	Description
textModuleProc	string	Read/Write	The name of the Tcl procedure called for TextModule events.
moduleBodyProc	string	Read/Write	The name of the Tcl procedure called for ModuleBody events
sortInstHeaders	<i>enSortOrder</i>	Read/Write	Controls whether instance header events should be generated in a sorted order based on switched master LCV.
instHeaderProc	string	Read/Write	The name of the Tcl procedure called for InstHeader events.
scalarizeInsts	bool	Read/Write	Controls if instances are scalarized.
sortInsts	<i>enSortOrder</i>	Read/Write	Controls if instances are sorted by instance name.
moduleFooterProc	string	Read/Write	The name of the Tcl procedure called for ModuleEnd events.
netlistEndProc	string	Read/Write	The name of the Tcl procedure called for NetlistEnd events.
moduleOrder	<i>enModuleOrder</i>	Read/Write	The order in which modules will be written to the final netlist file. Note that this has no effect on the order that modules are netlisted.
defaultNetlistProc	string	Read/Write	The default netlist procedure for cells that do not have a netlistProcedure defined in their Netlist Info. (This affects assembly of the final netlist and has no effect on the order the netlister uses to process modules.)
isTextModuleProc	string	Read/Write	The name of the Tcl command to use to determine if an heContext is a text module.
nameMap	<i>nlNameMap</i>	Read-only	The name map object used for a specific netlister invocation.

Table 166 Attributes (Continued)

Name	Type	Access	Description
currentContext	<i>heContext</i>	Read-only	The current context in a netlisting run. This is set internally whenever the Module event is generated.
fileName	string	Read-only	The final netlist filename. This attribute is set by nl::runNetlister at the beginning of the netlisting process.
partsList	[<i>heContext</i>]	Read-only	The list of non-leaf modules found during the first pass of netlisting.
leafList	[<i>heContext</i>]	Read-only	The list of leaf modules found during the first pass of netlisting.
textList	[<i>heContext</i>]	Read-only	The list of text modules found during the first pass of netlisting.
globals	string	Read-only	The list of global terminal names.
exprMap	[<i>dbExpressionMap</i>]	Read/write	The expression mapping for netlister
keepBitOrderForSorting	bool	Read/write	Indicates if the bit ordering should be considered in terminal name sorting.
logFile	string	Read/write	The log file name (Neither relative nor absolute path, just filename).
domain	[<i>nlDomainEnum</i>]	Read/write	The domain of netlister.

nIONameMap

The nIONameMap object implements the nlNameMap interface using OA's built-in namespace translation routines. The nIONameMap supports all of the attributes of the base nlNameMap. nIONameMap also supports an additional attribute for storing the destination oaNameSpace. nIONameMap is used throughout the Name Mapping API as

an `nlNameMap`. `nlOANameMap` is an `nlNameMap` object and can be used throughout the name mapping API wherever an `nlNameMap` object is required.

Table 167 Attributes

Name	Type	Access	Description
<code>oaNameSpace</code>	<code>oaNameSpace</code>	Read-only	The <code>oaNameSpace</code> used for name conversions.
<code>algo</code>	<code>bool</code>	Read-only	If this flag is true then the name map object is similar to OA name mappings in that mapping does not depend on previously mapped identifiers.
<code>openBrace</code>	<code>string</code>	Read-only	Character used to indicate the beginning of a bit index in the destination namespace.
<code>closeBrace</code>	<code>string</code>	Read-only	Character used to indicate the end of a bit index in the destination namespace.
<code>hierDelimiter</code>	<code>string</code>	Read-only	The hierarchy delimiter/separator used by <code>nlHierMap</code> in the destination namespace.
<code>rootContext</code>	<code>heContext</code>	Read-only	The context used by <code>nlHierMap</code> to elaborate the design hierarchy.
<code>uniqueifyProc</code>	<code>string</code>	Read-only	The procedure used in the write API to make identifiers unique when scope conflicts occur.

nlParameterizedNameMap

The `nlParameterizedNameMap` object implements the `oaNameMap` interface using a parameterized algorithmic name transform. The `nlParameterizedNameMap` supports all of the attributes of the base `nlNameMap`. `nlParameterizedNameMap` also supports additional attributes used to parameterize the name transform algorithm. `nlParameterizedNameMap` can be used throughout the Name Mapping API as an `nlNameMap` object wherever an `nlNameMap` object is required.

Table 168 *Attributes*

Name	Type	Access	Description
keywords	[string]	Read-only	List of reserved identifiers.
startCharMap	<i>nINameMap</i>	Read-only	A name map used to transform the first character of each name to a valid destination string.
validCharMap	<i>nINameMap</i>	Read-only	A name map used to transform each character to a valid destination string. Note, if a startCharMap is provided then the startCharMap is used in place of the validCharMap for the first character of identifiers.
escapePrefix	string	Read-only	The string to use for prefixing invalid identifiers.
escapeSuffix	string	Read-only	The string to use for appending to invalid identifiers.
algo	bool	Read-only	If this flag is true then the name map object is similar to OA name mappings in that mapping does not depend on previously mapped identifiers.
openBrace	string	Read-only	Character used to indicate the beginning of a bit index in the destination namespace.
closeBrace	string	Read-only	Character used to indicate the end of a bit index in the destination namespace.
hierDelimiter	string	Read-only	The hierarchy delimiter/separator used by nIHierMap in the destination namespace.
rootContext	<i>heContext</i>	Read-only	The context used by nIHierMap to elaborate the design hierarchy.
uniqueifyProc	string	Read-only	The procedure used in the write API to make identifiers unique when scope conflicts occur.

nlTerm

Represents a terminal in the Simplified Connectivity API. It can represent a real `oaTerm` or a Virtual Terminal. It also differs from `oaTerms` in that they cannot be a bundle and they do not support split ranges. The Simplified Connectivity API will automatically provide floating anonymous `nlInstTerms` for missing bits in `oaTerm` ranges. Many of the `nlTerm` attributes are writable to allow modification of terminal definitions by individual netlister implementations. Due to the ordering of netlisting it is safe to make modifications to `nlTerm` objects for a given module within the `ModuleHeader` event handler. Note, however, that this can have undesirable consequences. In general, care must be taken to ensure that all modifications are made before any `nlInstTerms` that reference the `nlTerm` are used.

Table 169 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The basename of the terminal.
bitOrder	<i>oaBitOrderEnum</i>	Read/Write	The bit order of the terminal. It is an error to retrieve this attribute if the terminal is not a bus.
minIndex	int	Read/Write	The Minimum numeric index of the terminal.
maxIndex	int	Read/Write	The maximum numeric index of the terminal.
isGlobal	bool	Read/Write	If the terminal is global.
isVirtual	bool	Read-only	If the terminal is virtual.
isBus	bool	Read-only	If this terminal is a bus.
termType	<i>oaTermTypeEnum</i>	Read/Write	The terminal's type.

oaInst

Represents block domain instances. An instance represents the inclusion of one design as a part of the contents of another. Use the `db::getAttr` command to retrieve the `oaInst` attributes described in the following table. (For the full list of attributes, see the Si2 `OpenAccess` documentation.)

Table 170 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the oalnst.
design	<i>oaDesign</i>	Read-only	Design to which the instance belongs.
master	<i>oaDesign</i>	Read-only	Master of the instance.
orient	<i>oaOrientEnum</i>	Read/Write	Orientation of the instance.
origin	point	Read/Write	Origin of the instance.
libName	string	Read-only	The library name for the master design referenced by the instance.
cellName	string	Read-only	The cell name for the master design referenced by the instance.
viewName	string	Read-only	The view name for the master design referenced by the instance.
header	<i>oaInstHeader</i>	Read-only	The instHeader associated with this instance.
numBits	int	Read-only	The number of bits of this instance.
params	<i>oaParamArray</i>	Read/Write	Parameters that were specified during creation. Only Pcell instances have parameters.
placementStatus	<i>oaPlacementStatusEnum</i>	Read/Write	The placement status for the instance.
source	<i>oaSourceEnum</i>	Read/Write	The source for the instance.
isImplicit	bool	Read-only	A boolean that indicates if the instance is implicit.

Netlisting Functions

The following netlisting functions are available in the Custom Compiler tool. These functions can be used to customize netlisters, along with the procedures described in [Netlisting Procedures](#).

- [nl::cdl::printSpiceParam](#)
- [nl::cdl::setBadChars](#)
- [nl::HSPICE::getModelParamValue](#)
- [nl::HSPICE::getModuleNameForTextView](#)
- [nl::HSPICE::getSpiceInstTermName](#)
- [nl::HSPICE::spiceAppendTermNames](#)
- [Spectre::exprMapFuncProc](#)
- [VerilogAMS::printInst](#)
- [VerilogAMS::printInstParameters](#)
- [vlg::warnFilteredNetTypes](#)

nl::cdl::printSpiceParam

Description

Dumps the parameter of the given instance to the netlist and returns the parameter value.

Syntax

```
<string> nl::cdl::printSpiceParam <string> <string> <nlNetlister>  
<heContext> <nlParamMap> [<oaInst>] [<heContext>] [<int>] [<int>]  
[<int>]
```

Arguments

param

<string> The name of the parameter.

dbName

<string> The mapped parameter name.

nl

<nlNetlister> The netlister.

`master`

`<heContext>` The instance master context.

`pmap`

`<nlParamMap>` Parameter mapping object.

`Inst`

`<oaInst>` The instance object.

`parent`

`<heContext>` The parent context.

`writedefaults`

`<int>` Write the default value to the netlist if not 0.

`writename`

`<int>` Write the parameter name with format `paramName=paramValue` if not 0.

`writeError`

`<int>` Write the error to the console if not 0.

Returns

`<string>` The value of the parameter value of the given instance.

nl::cdl::setBadChars

Description

Sets the name mapping object to map the given characters to a string with format:
`%<charcode>%`.

Syntax

```
nl::cdl::setBadChars <nlNameMap> <string>
```

Arguments

`<namemap>`

`<nlNameMap>` The name mapping object.

`<charlist>`

`<string>` The characters to be mapped.

nl::HSPICE::getModelParamValue

Description

Calculates the model parameter value of the following instances:

- If the "model" parameter name is not mapped, gets the modelName instance parameter value, otherwise gets the value of the mapped parameter.
- if the model name is not found, gets the instance's model parameter value.

Syntax

```
<string> nl::HSPICE::getModelParamValue <nlNetlister> <heContext>  
      <oaInst> <heContext>
```

Arguments

nl

<nlNetlister> The netlister.

master

<heContext> The master cell context.

inst

<oaInst> The instance.

parent

<heContext> The parent design.

Returns

<string> The calculated subckt name.

nl::HSPICE::getModuleNameForTextView

Description

Calculates the module name for the given text view:

- Look at the `moduleName` property of `viewInfo`.
- If not found, look at the `moduleName` property of a shadow OA design.
- If not found, look at the `moduleName` property of `simInfo`.
- If not found, use the cell name.

Syntax

```
<string> nl::HSPICE::getModuleNameForTextView <nlNetlister> <heContext>
```

Arguments

nl

<nlNetlister> The netlister.

ctx

<heContext> The module context.

Returns

<string> The calculated subckt name.

nl::HSPICE::getSpiceInstTermName

Description

Finds the instTerm's net name to be netlisted:

- If the net is global then map it in the global scope name mapping, excluding the virtual terminals case. For the virtual terminals case, mapping should be done inside the module.
- If the net is not global, but its name coincides with the global net name, then uniquely identify it in the global name space.
- Otherwise, just map it in its context name mapping.

Syntax

```
<string> nl::HSPICE::getSpiceInstTermName <nlNetlister> <nlNameMap>  
<heContext> <nlInstTerm>
```

Arguments

netlister

<nlNetlister> The netlister.

nm

<nlNameMap> The name map object.

parent

<heContext> The parent context.

`instterm`

`<nInstTerm>` The instance terminal.

Returns

`<string>` The net name to be netlisted for the given instance terminal.

nl::HSPICE::spiceAppendTermNames

Description

Dumps the given module's terminals to the netlist.

Syntax

```
nl::HSPICE::spiceAppendTermNames <nInstTerm> <heContext>
```

Arguments

`nl`

`<nInstTerm>` The netlist.

`ctx`

`<heContext>` The module context.

Spectre::exprMapFuncProc

Description

This function provides the implementation for the `funcProc` attribute of the Spectre `nlExprMap`. In other words, it formats function calls found in parameter expressions for Spectre language netlisting.

VerilogAMS::printInst

Description

This function works as the default netlist procedure for the VerilogAMS netlist or language.

VerilogAMS::printInstParameters

Description

This function prints instance parameters for VerilogAMS hierarchical designs. In addition to formatting normal passed parameters, this function also handles the formatting requirements for inherited connection assignments.

vlg::warnFilteredNetTypes

Description

This function prints a standard warning message for any net types that were ignored during netlisting. Note that Verilog ignores any net types not present in the `nlVerilogWireNetTypeList` preference.

Netlisting Procedures

The following netlisting procedures are available in the Custom Compiler tool. These functions can be used to customize netlisters, along with the functions described in [Netlisting Functions](#).

- [nlPrintauCdlBJT](#)
- [nlPrintauCdlDevice](#)
- [nlPrintauCdlDiode](#)
- [nlPrintauCdlMOSFET](#)
- [nlPrintauCdlPrimDevice](#)
- [nlPrintauCdlSubckt](#)

nlPrintauCdlBJT

Description

Netlisting procedure for BJT devices in the following format:

- `Qxxx C B E model <M=val> <$EA=val>`
- `Qxxx C B E model <M=val> <$EA=val> <$SUB=val>`

This procedure can be directly called from another netlisting procedure. In order to be used as a netlisting procedure it should be set in the `simInfo/auCdl/netlistProcedure`.

Syntax

```
nlPrintauCdlBJT <nlNetlister> <oaInst> <heContext> <string> [<string>]  
[<heContext>]
```

Arguments

netlister

<nlNetlister> The netlister.

inst

<oaInst> The instance object.

master

<heContext> The instance master context.

instNamePrefix

<string> Instance name prefix.

params

<string> Parameters list.

parent

<heContext> Parent context.

Examples

```
proc myBJTNetlistProc {nl inst context instNamePrefix params parent} {  
    nl::append " * Custom text added before instance  
[db::getAttr inst.name]" -netlister $nl  
    return [nlPrintauCdlBJT $nl $inst $context  
$instNamePrefix $params $parent]  
}
```

nlPrintauCdlDevice

Description

Default netlisting procedure for passive devices C, L, and R in the following format:

- ## Cxxx PLUS MINUS <C> <M=val> <\${modelName}> <\${W=val}> <\${L=val}>
- ## Cxxx PLUS MINUS <C> <M=val> <\${modelName}> [\${SUB=val}] <\${W=val}> <\${L=val}>
- ## Lxxx PLUS MINUS <L> <M=val> \${modelName}

- ## Rxxx PLUS MINUS R <M=val>[\$RP] <\$W=val> <\$L=val>
- ## Rxxx PLUS MINUS R <M=val>[\$RP] <\$SUB=val> <\$W=val> <\$L=val>

This procedure can be directly called from another netlisting procedure. In order to be used as a netlisting procedure it should be set in the simInfo/auCdl/netlistProcedure.

Syntax

```
nlPrintauCdlDevice <nlNetlister> <oaInst> <heContext> <string> [<string>]
[<heContext>]
```

Arguments

netlister

<nlNetlister> The netlister.

inst

<oaInst> The instance object.

master

<heContext> The instance master context.

instNamePrefix

<string> Instance name prefix.

params

<string> Parameters list.

parent

<heContext> Parent context.

nlPrintauCdlDiode

Description

Default netlisting procedure for diode devices in the following format:

- Dxxx PLUS MINUS <model> <AREA=val> <PJ=val> <M=val>
- Dxxx PLUS MINUS <model> <AREA=val> <PJ=val> <M=val> [\$SUB=val]

This procedure can be directly called from another netlisting procedure. In order to be used as a netlisting procedure it should be set in the simInfo/auCdl/netlistProcedure.

Syntax

```
nlPrintauCdlDiode <nlNetlister> <oaInst> <heContext> <string> [<string>]
[<heContext>]
```

Arguments

`netlister`

<nlNetlister> The netlister.

`inst`

<oaInst> The instance object.

`master`

<heContext> The instance master context.

`instNamePrefix`

<string> Instance name prefix.

`params`

<string> Parameters list.

`parent`

<heContext> Parent context.

nlPrintauCdlMOSFET

Description

Default CDL netlisting procedure for the following MOSFET devices in format:

- Mxxx D G S B model <W=val> <L=val> <M=val>

This procedure can be directly called from another netlisting procedure. In order to be used as a netlisting procedure it should be set in the `simInfo/auCdl/netlistProcedure`.

Syntax

```
nlPrintauCdlMOSFET <nlNetlister> <oaInst> <heContext> <string> [<string>]  
[<heContext>]
```

Arguments

`netlister`

<nlNetlister> The netlister.

`inst`

<oaInst> The instance object.

`master`

<heContext> The instance master context.

`instNamePrefix`
 <string> Instance name prefix.

`params`
 <string> Parameters list.

`parent`
 <heContext> Parent context.

nlPrintauCdlPrimDevice

Description

Netlisting procedure for the following format netlisting:

- Rxxx PLUS MINUS <modelName> <instParam1=val> <instParam2=val>
\$SUB=<netName3>

This procedure can be directly called from another netlisting procedure. In order to be used as a netlisting procedure it should be set in the `simInfo/auCdl/netlistProcedure`.

Syntax

```
nlPrintauCdlPrimDevice <nlNetlister> <oaInst> <heContext> <string>  
                          [<string>] [<heContext>]
```

Arguments

`netlister`
 <nlNetlister> The netlister.

`inst`
 <oaInst> The instance object.

`master`
 <heContext> The instance master context.

`instNamePrefix`
 <string> Instance name prefix.

`params`
 <string> Parameters list.

`parent`
 <heContext> Parent context.

nlPrintauCdlSubckt

Description

Netlisting procedure for the following format netlisting:

- `Xxxx PLUS MINUS / modelName <W=val> <L=val> <M=val>`
- `Xxxx PLUS MINUS B / modelName <W=val> <L=val> <M=val>`

Generates empty modelName subcircuits at the end of the netlist.

This procedure can be directly called from another netlisting procedure. In order to be used as a netlisting procedure it should be set in the `simInfo/auCdl/netlistProcedure`.

Syntax

```
nlPrintauCdlSubckt <nlNetlister> <oaInst> <heContext> <string> [<string>]  
[<heContext>]
```

Arguments

`netlister`

`<nlNetlister>` The netlister.

`inst`

`<oaInst>` The instance object.

`master`

`<heContext>` The instance master context.

`instNamePrefix`

`<string>` Instance name prefix.

`params`

`<string>` Parameters list.

`parent`

`<heContext>` Parent context.

Preferences

[Table 171](#) lists Custom Compiler nl namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 171 Netlist Tool Preferences

Name	Type	Scope	Default	Description
nIAMSAIwaysGetScalarizeTermsFromWorker	bool	user	false	When <code>true</code> , always uses the <code>scalarizeTerminals</code> flag of the worker netlister for module terminals in AMS netlisting. When <code>false</code> , uses the worker's flag only in the case when it is set on both for this module and its parent module, else uses the AMS netlister's flag. Valid values: true, false
nIAMSCompatiblePortOrder	bool	user	true	Port ordering in a way that is compatible with industry standards. Valid values: true, false
nIAMSErrorOnDuplicateTextViews	bool	user	true	Defines the behavior of the AMS netlister in case when more than one text-type module exists in the hierarchy with the same cell name. When true, shows an error and terminates the netlisting. When false, performs the netlisting having the same cell name for all modules. Valid values: true, false
nIAMSGroupModulesByLibrary	bool	user	false	When <code>true</code> , groups all modules for each OA library into a separate file (per netlist language). The contents of each file is described in a <code>config.yaml</code> file. Valid values: true, false
nIAMSIgnorePropNames	string	user		The list of property names to check for determining whether an instance or master should be ignored.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nlAMSIncludeTextViews	string	user		Text views to be included in the netlist. Format: l/c/v or c/v or cellname. If the library name is not given, the top library of the cellview is used. If the view is not given and only one SPICE/CDL text view exists, that view is used, otherwise the text view is chosen randomly.
nlAMSMapMultiViewCells	bool	user	false	When false, if the same-cell-name devices are present in the hierarchy, but are used by different worker netlisters of the AMS netlister, these cell names are not mapped for uniquifying. When true, these cell names are always mapped. Valid values: true, false
nlAMSResolveXInstPrefix	bool	user	false	Removes X prefix from Spice instance names. Valid values: true, false
nlCDLAddCellNameAffix	bool	user	false	When true, adds prefix or suffix to the cell name in CDL netlisting. Valid values: true, false
nlCDLBusStyle	string	user	<>	Indicates the style of bus parentheses.
nlCDLCellNamePrefix	string	user		Prefix for cell name in CDL netlisting.
nlCDLCellNameSuffix	string	user		Suffix for cell name in CDL netlisting.
nlCDLCellsAffixFilterType	cellsAffixFilterTypes	user	ignore	When 'process', the value of preference 'nlCDLProcessCellsAffix' is used to add the prefix/suffix to only the given cells. When 'ignore', the value of preference 'nlCDLIgnoreCellsAffix' is used to not add the prefix/suffix to the given cells.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nICDLCheckBusTerminalTypeConsistency	bool	user	false	When true, bus terminal bits are checked to be of the same terminal type. Valid values: true, false
nICDLCompatiblePortOrder	bool	user	true	Port ordering in a way that is compatible with industry standards. For further details, see Netlister Preference Examples . Valid values: true, false
nICDLErrorOnGlobalToTerminalShort	bool	user	true	Defines behavior in CDL netlisting when instance ignoring causes nets short, and one net is global and other is connected to terminal. When false, the global net stays and terminal is ignored. When true, an error is shown and the netlisting fails. Valid values: true, false
nICDLErrorOnTerminalShort	bool	user	false	Defines the behavior in CDL netlisting when instance ignoring causes nets short, and these nets both are connected to different terminals. When false, one of these terminals stays and another is ignored. When true, an error is shown and the netlisting fails. Valid values: true, false
nICDLGenerateEmptySubckts	bool	user	false	Specifies whether an empty subcircuit should be generated. Valid values: true, false
nICDLGlobalNets	string	user	None	Defines global nets for CDL netlister.
nICDLIgnoreCellsAffix	string	user		Space-separated list of cell name wildcards for which the prefix and suffix should not be added in CDL netlisting.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nICDLIgnorePropNames	string	user	lvslgnore	The list of property names to check for determining if an instance or master should be ignored for CDL.
nICDLIncludeCDLFile	bool	user	false	Controls how text views are netlisted. When false, includes text views with an <code>.include</code> statement. When true, writes the contents of text views directly into the final netlist. Valid values: true, false
nICDLIncludeCDLNetlistFile	Include FileEu mType	user	false	Controls how the netlister handles the <code>CDL_NETLIST_FILE</code> property. When false, the property is ignored. Otherwise, the netlister looks for a property <code>CDL_NETLIST_FILE</code> on all leaf cellviews. The property should be set to the path of an associated text file that contains the <code>.subckt</code> for the cellview. When true, the netlister writes the contents of the file directly into the final netlist. When include, it writes an <code>.include</code> statement in the final netlist that points to the associated file. Valid values: true, false, include
nICDLIncludeFiles	string	user		Specifies the space-separated list of files to include in all CDL netlists.
nICDLIsCaseSensitive	string	user	Sensitive	Indicates whether the CDL name mapping is case-sensitive.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nICDLMapNetNamesData	string	user		Pairs of source and destination names for net names mapping in CDL netlister. Used when the <code>nICDLSupportMapNetNames</code> preference is true.
nICDLPortConnectionType	string	user	byOrder	Defines connection by name or order for subcircuit instances. Valid values: byOrder, byName
nICDLPrependFirstChar	string	user	always	Specifies whether to prepend the <code>-fixedFirstChar</code> option.
nICDLPrintLDD	bool	user	true	Prints *.LDD statement at the start of CDL netlist. Valid values: true, false
nICDLPrintMFactor	bool	user	true	When true, the CDL netlister exports multiplier factor m. Valid values: true, false
nICDLProcessCellsAffix	string	user		Space-separated list of cell name wildcards only for which the prefix and suffix should be added in CDL netlisting.
nICDLRemoveBangs	bool	user	false	When true, removes ! from all net/terminal and instance names in the CDL netlist. Valid values: true, false
nICDLScalarizeBitOrder	string	user	autoAscending	The order of scalarized terminal bits in the CDL netlist.
nICDLScale	string	user	meter	Specifies the insertion of the *.SCALE statement. Valid values: none, meter, micron

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nICDLShortPreferredOrder	string	user	descending	Defines the behavior in the CDL netlisting when instance ignoring causes nets short, and these nets have equal priorities (global, term-connected, labeled, and so on). When descending, the alphanumerically higher name wins. When ascending, the alphanumerically lower name wins.
nICDLSupportMapNetNames	bool	user	false	When true, enables net names mapping in CDL netlister according to the value of preference nICDLMapNetNamesData. Valid values: true, false
nICDLTopVirtualTerms	string	user	useDefaultNetName	Controls the virtual terminals creation in the top module for CDL netlister with one of the following: • useDefaultNetName adds virtual terminals with the the default net name like gnd!. • useVirtualTermName adds virtual terminals with a special virtual name like vt_ground_gnd! for non-top modules. • assignDefaultNet does not add any virtual terminals in the top module at all.
nICDLUseSingleQuote	bool	user	true	Specifies whether single quotes surround parameter values in the CDL netlist. Valid values: true, false
nICDLWriteParamDefaults	bool	user	false	Specifies whether the instance parameters should be exported with default values in the CDL netlist. Valid values: true, false

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nICheckDesign	nICheckDesign	user	timestamp	Mode to specify whether to check ERC/SRC when netlisting.
nICheckNonNetlistableCellSeverity	nINonNetlistableSeverity	user	error	Severity of non-netlistable cells check on netlisting.
nICheckOutdatedDesignsSeverity	nIOutdatedSeverity	user	warning	Severity of outdated designs check on netlisting.
nIConnDefDefaultsAsGlobals	bool	user	false	Used in virtual terminals creation for parameterized connections. When false, additional virtual terminals are created for non-global default nets (for global ones they aren't created). When true, the additional terminals for default nets are never created. Valid values: true, false
nIEldoBusStyle	string	user	<>	Indicates the style of bus parentheses.
nIEldoCompatiblePortOrder	bool	user	true	Port ordering in a way that is compatible with industry standards.
nIEldoDefaultStringParameters	string	user	model	The list of standard parameter names which should always be formatted as Eldo string parameters.
nIEldoGlobalNets	string	user	0 gnd! ground gnd	Defines global nets for Eldo netlister.
nIEldoHierDelimiter	string	user	.	Hierarchy delimiter.
nIEldoIgnorePropNames	string	user		The list of property names to check for determining whether an instance or master should be ignored.
nIEldoIsCaseSensitive	string	user	Lower	Indicates whether the Eldo name mapping is case-sensitive.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nIEldoScalarizeBitOrder	string	user	autoAscending	The order of scalarized terminal bits in the Eldo netlist.
nIEldoWriteParamDefaults	bool	user	false	Specifies whether the instance parameters should be exported with default values in the Eldo netlist. Valid values: true, false
nIEnableVirtualTerminals	bool	user	true	When true, enables virtual terminals processing in netlisters. Valid values: true, false
nIErrorOnAbsentInstTerm	bool	user	true	When true, shows errors and stops netlisting on an absent instance terminal. When false, displays a warning. Valid values: true, false
nIErrorOnCrossViewCheckViolation	bool	user	false	When false, the Cross View Check violations are all shown as warnings and the netlisting is not stopped when violation occurs. When true, the Cross View Check violations shows error and terminates netlisting.
nIFilterInstParamsByCDFUse	bool	user	false	When true, the instance parameters with attribute 'use=false' are not printed. Valid values: true, false
nIHSPICEBusStyle	string	user	<>	Indicates the style of bus parentheses.
nIHSPICECompatiblePortOrder	bool	user	true	Port ordering in a way that is compatible with industry standards. For further details, see Netlister Preference Examples . Valid values: true, false

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nIHSPICECopyIncludedTextFile	bool	user	false	Controls how the text views are included in HSPICE. When true, copies the text file to the same folder of the target netlist file, and prints in <code>.INCLUDE</code> statement only the file name without path. When false, prints in <code>.INCLUDE</code> statement the full path of the text file located in the text view folder. Valid values: true, false
nIHSPICEDefaultLeafNetlistByTemplate	bool	user	true	When true, performs netlisting for leaf devices with undeclared netlist procedures using the template-based command <code>nl::printSpiceInst</code> . Valid values: true, false
nIHSPICEDefaultStringParameters	string	user	model	The list of standard parameter names which should always be formatted as HSPICE string parameters.
nIHSPICEDiffStbProbeFile	string	user	<file path>	The file path of the HSPICE definition for the <code>diffstbprobe</code> element.
nIHSPICEGlobalNets	string	user	0 gnd! ground gnd	Defines global nets for the HSPICE netlister.
nIHSPICEIgnoreBehavioralModules	bool	user	false	When true, observes behavioral modules as leaves. When false, behavioral modules cause errors and netlisting termination. Valid values: true, false
nIHSPICEIgnoreLeavesInAutoSetXPrefix	bool	user	false	When true, do not autoset the X value of the name prefix to the leaf devices when the prefix is not specified. Valid values: true, false

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nIHSPICEIgnorePropNames	string	user		The list of property names to check for determining whether an instance or master should be ignored.
nIHSPICEIncludeFiles	string	user		Specifies the include file list for the HSPICE netlister.
nIHSPICEIncludeHSPICEFile	bool	user	true	Controls how text views are netlisted. When false, includes text views with an <code>.include</code> statement. When true, writes the contents of text views directly into the final netlist. Valid values: true, false
nIHSPICEIncludeHSPICENetlistFile	Include FileEu mType	user	false	Controls how the netlister handles the <code>HSPICE_NETLIST_FILE</code> property. When false, the property is ignored. Otherwise, the netlister looks for a property <code>HSPICE_NETLIST_FILE</code> on all leaf cellviews. The property should be set to the path of an associated text file that contains the <code>.subckt</code> for the cellview. When true, the netlister writes the contents of the file directly into the final netlist. When include, it writes an <code>.include</code> statement in the final netlist that points to the associated file. Valid values: true, false, include
nIHSPICEIsCaseSensitive	string	user	Lower	Indicates whether the HSPICE name mapping is case-sensitive.
nIHSPICELocalsAsGlobals	string	user		Defines the nets to be processed as globals in HSPICE netlister.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nIHSPICEPrependFirstChar	string	user	auto	When always, always adds the fixed first char to the instance name. When auto, adds only if the existing first char is not the needed one in the HSPICE netlisting. Valid values: always, auto
nIHSPICERemoveBangs	bool	user	false	When true, removes ! from all net/terminal and instance names in the HSPICE netlist. Valid values: true, false
nIHSPICEScalarizeBitOrder	string	user	autoAsc ending	The order of scalarized terminal bits in the HSPICE netlist.
nIHSPICEWriteParamDefaults	bool	user	false	Specifies whether the instance parameters should be exported with default values in the HSPICE netlist. Valid values: true, false
nIIgnoreCDFModuleName	bool	user	false	When true, the moduleName value in CDF parameters is not considered when getting the module name of a text module for netlisting.
nIIgnoreCell	string	user	""	Declares cellviews as: <i>ignored</i> with or without terminal shorts; observed as <i>stop</i> modules even if they are not netlisted as leaf; or <i>not ignored</i> even if they are declared as <i>ignored</i> in the database. The value should be text in YAML format. For further details, see nIIgnoreCell .
nIIgnoreParasiticDevices	bool	user	false	Ignores parasitic devices found in the design. Valid values: true, false
nILibrariesIgnoringEvalErrors	string	user	sheets	List of libraries where PCell evaluation errors are ignored.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nlMapParameterCase	bool	user	false	Indicates whether parameter names, values, and model names should be mapped. Valid values: true, false
nlMaximumPartialEvaluation	bool	user	false	When true, uses the maximum partial evaluation of expressions. When false, prints expressions as is and not evaluate them. Valid values: true, false
nlMaxNumConsoleWarnings	int	user	256	The maximum number of warning messages of the same ID reported to the console by netlisting. Set to -1 to print all warnings without limitations; set to 0 to suppress any warnings.
nlMultiplicationParamName	string	user	m	Names of parameters with meaning of device multiplication used in flatten netlisting.
nlNanoSimBusStyle	string	user	<>	Indicates the style of bus parentheses.
nlNanoSimCompatiblePortOrder	bool	user	true	Port ordering in a way that is compatible with industry standards. For further details, see Netlister Preference Examples . Valid values: true, false
nlNanoSimGlobalNets	string	user	gnd	Defines global nets for the NanoSim netlister.
nlNanoSimIgnorePropNames	string	user		The list of property names to check for determining whether an instance or master should be ignored.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nlNanoSimIncludeNanoSimFile	bool	user	false	Controls how text views are netlisted. When false, includes text views with an <code>.include</code> statement. When true, writes the contents of text views directly into the final netlist. Valid values: true, false
nlNanoSimIsCaseSensitive	string	user	Lower	Indicates whether the NanoSim name mapping is case-sensitive.
nlNanoSimScalarizeBitOrder	string	user	autoAsc ending	The order of scalarized terminal bits in the NanoSim netlist.
nlNanoSimWriteParamDefaults	bool	user	false	Specifies whether the instance parameters should be exported with default values in the NanoSim netlist. Valid values: true, false
nlOpenNetlistViewer	bool	user	true	When true, opens the text viewer. Valid values: true, false
nlPerformanceReport	bool	user	false	When true, creates a performance report on netlisting. Valid values: true, false
nlPrintMessages	string	user	to console	Shows where messages should printed. Valid values: to console, to file, none
nlProcessOutdatedConnectivity	nlProc essOut datedD esigns	user	update	Controls the behavior for schematics modified by third-party tools.
nlProcessUnconnectedSymbolPinForVirtual	bool	user	false	When true, forces processing for virtual terminals from a switched master if the symbol terminal is floating.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nlReportIgnoredDevices	bool	user	false	When true, the ignored devices and instances are printed to the <code>ignored.txt</code> file Valid values: true, false
nlRevertOutdatedDesigns	bool	user	false	When true, reverts outdated designs on netlisting. Valid values: true, false
nlRevertOutdatedDesigns	bool	user	false	When true, reverts outdated designs on netlisting. Valid values: true, false
nlRunInBackground	bool	user	false	When true, runs the netlister in background. Valid values: true, false
nlShortExprPropNames	string	user	lXRemoveDevice lvsRemoveDevice	Property or parameter names that define how to short devices that have been removed during netlisting.
nlSimulatorAliases	string	user	{hspiceD hspice} {spectre spectre} {auCdl auCdl} {verilog verilog}	Aliases for simInfo names.
nlSpectreAlwaysRemasterVerilogATextModules	bool	user	false	When true, all text modules of the verilogA view type have their names changed to <code><libName>_<cellName>_<viewName></code> notation, and the corresponding <code>-master</code> option is used in the <code>INCLUDE</code> statement. When false, this renaming is used only in the case where there is more than one text module of the verilogA view type with the same cell name, to avoid name ambiguity. Valid values: true, false

Table 171 *Netlist Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
nISpectreAMSCompatiblePortOrder	bool	user	false	Port ordering in a way that is compatible with industry standards.
nISpectreAMSErrorOnDuplicateTextViews	bool	user	true	Defines the behavior of the SpectreAMS netlister in case when more than one text-type module exists in the hierarchy with the same cell name. When true, shows an error and terminates the netlisting. When false, performs the netlisting having the same cell name for all modules. Valid values: true, false
nISpectreAMSGroupModulesByLibrary	bool	user	false	When true, groups all modules for each OA library into a separate file (per netlist language). The contents of each file is described in a <i>config.yaml</i> file. Valid values: true, false
nISpectreAMSIgnorePropNames	string	user		The list of property names to check for determining whether an instance or master should be ignored.
nISpectreAMSMapMultiViewCells	bool	user	false	When false, if the same-cell-name devices are present in the hierarchy, but are used by different worker netlisters of the SpectreAMS netlister, these cell names are not mapped for uniquifying. When true, these cell names are always mapped. Valid values: true, false
nISpectreCompatiblePortOrder	bool	user	true	Port ordering in a way that is compatible with industry standards. For further details, see Netlister Preference Examples. Valid values: true, false

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nlSpectreDefaultStringParameters	string	user	model	The list of the parameter names for which the value is always observed as string to be wrapped in double quotes.
nlSpectreDiffStbProbeFile	string	user	<file path>	The file path of the Spectre definition for element diffstbprobe.
nlSpectreIgnorePropNames	string	user		The list of property names to check for determining whether an instance or master should be ignored.
nlSpectreIncludeFiles	string	user		Specifies the include file list for Spectre netlister.
nlSpectreIncludeSpectreFile	bool	user	false	Controls how text views are netlisted. When false, includes text views with an <code>.include</code> statement. When true, writes the contents of text views directly into the final netlist. Valid values: true, false
nlSpectreScalarizeBitOrder	string	user	autoAsc ending	The order of scalarized terminal bits in the Spectre netlist.
nlSpectreWriteParamDefaults	bool	user	true	Specifies whether the instance parameters should be exported with default values in the Spectre netlist. Valid values: true, false
nlStopOnConnectivityImplicitChange	bool	user	false	Controls the netlister behavior for connectivity re-extracted designs. When true, shows errors and stops netlisting on a design with connectivity changes done implicitly by SE connectivity engine. Valid values: true, false
nlSupplyTypeGroundRegex	string	user	[vV][sS][sS]	The regular expression for ground nets. This is used in nl::getSupplyType .

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nlSupplyTypePowerRegexp	string	user	[vV][dD][dD]	The regular expression for power nets. This is used in nl::getSupplyType .
nlTextViewCheckMethod	string	user	None	Specifies the method to validate if the text view design data is up to date.
nlTopVirtualTerms	string	user	useDefaultNetName	Controls the virtual terminals creation in the top module with one of the following: <code>useDefaultNetName</code> adds virtual terminals with the the default net name like <code>gnd!</code>. <code>useVirtualTermName</code> adds virtual terminals with a special virtual name like <code>vt_ground_gnd!</code> for non-top modules. <code>assignDefaultNet</code> does not add any virtual terminals in the top module at all. This works for the netlists which have no specific <code>nl<NAME>TopVirtualTerms</code> preference.
nlUseDirectionInShortPriority	bool	user	false	Controls how the winner net name is calculated when internal (that is. not connected to terminals) non-global nets are shorted together. - False: The alphabetical priority is used (according to the <code>nl<name>ShortPreferredOrder</code> preference value). - True: The signal source net wins. That is, the <code>output inst.term</code> wins <code>inout</code>, and <code>inout</code> wins <code>input</code>. For the same type, alphabetical priority is used.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nlUseViewInfoTermOrder	bool	user	true	Indicates whether to use the viewInfo termOrder for text views.
nlVerilogCheckBusTerminalTypeConsistency	bool	user	true	When true, bus terminal bits are checked to be of the same terminal type. Valid values: true, false
nlVerilogCheckLeafTerminals	bool	user	false	Controls whether to do a cross-view check for the leaf cells.
nlVerilogCompatiblePortOrder	bool	user	true	Port ordering in a way that is compatible with industry standards. For further details, see Netlister Preference Examples . Valid values: true, false
nlVerilogCopyIncludedTextFile	bool	user	false	Controls how the text views are included in Verilog. When true, copies the text file to the same folder of the target netlist file, and prints only the file name without path in an <code>.include</code> statement. When false, prints the full path of the text file located in the text view folder in an <code>.include</code> statement. Valid values: true, false
nlVerilogDefaultNetType	string	user	wire	The default net type for wire declarations.
nlVerilogDesignVariableMode	string	user	macro	Defines how the design variables are specified in the Verilog netlist. When macro, prepends the variable name with "" when used, and declares the variable value in macro. When xmr, prepends the variable name with "snps_design_vars." when used, and declares values out of netlisting. Valid values: macro, xmr

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nlVerilogDesignVariables	string	user		List of name value pairs to specify design variables and their default values.
nlVerilogGlobalParameterDefault	string	user	0	Default value to use for parameters that have no default value specified.
nlVerilogICCAddCellNameAffix	bool	user	false	When true, add prefix and/or suffix to the cell name in VerilogICC netlisting.
nlVerilogICCCellNamePrefix	string	user		Prefix for cell name in VerilogICC netlisting.
nlVerilogICCCellNameSuffix	string	user		Suffix for cell name in VerilogICC netlisting.
nlVerilogICCCellsAffixFilterType	cellsAffixFilterTypes	user	process	When 'process', the value of preference 'nlVerilogICCProcessCellsAffix' is used to add the prefix/suffix to only the given cells. When 'ignore', the value of preference 'nlVerilogICCIgnoreCellsAffix' is used to not add the prefix/suffix to the given cells.
nlVerilogICCCompatiblePortOrder	bool	user	true	Port ordering in a way that is compatible with industry standards. Valid values: true, false
nlVerilogICCGroundConnectDefs	string	user		Includes the definition names of ground supply connections in the design found during VerilogICC supply extraction.
nlVerilogICCIgnoreCellsAffix	string	user		Space-separated list of cell name wildcards for which the prefix and suffix should not be added in VerilogICC netlisting.
nlVerilogICCIgnoreConnectDef	string	user	[vV][dD][dD]	Excludes the definition names of ground supply connections in the design found during VerilogICC supply extraction.

Table 171 *Netlist Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
nIVerilogICCIgnorePropNames	string	user		The list of property names to check for determining whether an instance or master should be ignored.
nIVerilogICCIncludeSupplies	bool	user	false	When true, the VerilogICC netlister writes power supplies to the Verilog netlist. When false, the VerilogICC netlister removes power supplies from the Verilog netlist. Valid values: true, false
nIVerilogICCPortConnectionType	string	user	byName	Defines connection by name or order for modules. Values are: byOrder, byName
nIVerilogICCPowerConnectDefs	string	user		Includes the definition names of power supply connections in the design found during VerilogICC supply extraction.
nIVerilogICCPrintInstanceParameters	bool	user	true	Controls whether to print the instance parameters in VerilogICC netlisting. Valid values: true, false
nIVerilogICCProcessCellsAffix	string	user		Space-separated list of cell name wildcards only for which the prefix and suffix should be added in VerilogICC netlisting.
nIVerilogICCRangeDelimiterCharacter	string	user		The range delimiter character for scalarized instance names in VerilogICC netlister: '<base><delimiter><index>'. If empty, the bit notation '<base>[<index>]' is used.
nIVerilogICCSupplyNameMap	string	user	false	Contains pairs of supply name in the design and the supply name in ICC.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nIVerilogCCVectorExpansion	bool	user	true	When true, all vector ranges are netlisted as individual bits and the range delimiter character refers to the nIVerilogCCRangeDelimiterCharacter Tcl preference value.
nIVerilogIgnoredNetTypesForPropagation	string	user	reg	The netType property values of the net to be ignored in the net type propagation in the hierarchy for systemVerilog. <i>Net type</i> is the value of the oaProp netType, set on the oaNet object. This setting is visible in the Parameters section of the Property Editor.
nIVerilogIgnoredSigTypes	string	user	""	Controls how the Verilog netlister handles net signal types in the design and whether to add or ignore supply0 or supply1 signal types in the final netlist. <i>Signal type</i> is the value of the oaNet's signal type returned by oaNet::getSignalType. This is only accessible via the db::getAttr Tcl command on the Console. Valid values: List of any names from: { signal power ground clock tieoff tieHi tieLo analog scan reset }
nIVerilogIgnorePortOrderProp	string	user	false	When true, Verilog netlisting ignores portOrder properties for all hierarchical schematics. portOrder is still checked for leaf devices and text views.
nIVerilogIgnorePropNames	string	user		The list of property names to check for determining whether an instance or master should be ignored.

Table 171 *Netlist Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
nIVerilogIllegalCharacterMapping	string	user	escaped	<code>mapped</code> maps illegal characters to another character while <code>escaped</code> escapes any illegal characters in a string. Valid values: <code>escaped</code> , <code>mapped</code>
nIVerilogIncludeSupplies	bool	user	true	When true, the Verilog netlister writes power supplies to the Verilog netlist. When false, the Verilog netlister removes power supplies from the Verilog netlist. Valid values: <code>true</code> , <code>false</code>
nIVerilogIncludeVerilogFile	bool	user	false	Controls how text views are netlisted. When false, includes text views with an <code>.include</code> statement. When true, writes the contents of text views directly into the final netlist. Valid values: <code>true</code> , <code>false</code>
nIVerilogMSBLSB	bool	user	true	When true, applies MSB:LSB. When false, applies LSB:MSB. Valid values: <code>true</code> , <code>false</code>
nIVerilogNetTypeConflictSolutions	string	user		Defines the resulting systemVerilog net type for cases when the net has connections to different net types. If not defined, or the case is not found in definitions, the net type 'interconnect' is used.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nIVerilogNetTypesAsArray	string	user	real wreal scalar voltage_r current_r wreal1dri ver wreal4st ate wreal sum wrealavg wrealmin wreal max	The netType property values of the net, for which the bus nets are printed as arrays (like "wreal a [0:1];" but not "wreal [0:1] a") in the systemVerilog netlist.
nIVerilogNumericFormat	string	user	asls	Defines how parameters are scaled in the final netlist. Valid values: asls, decimal, scientific
nIVerilogPortConnectionType	string	user	byOrder	Defines connection by name or order for modules. Valid values: byName, byOrder
nIVerilogPrintParamName	bool	user	true	When true, prints the names of parameters at the instantiation: modulename # When false, not printing the names of parameters: modulename #(val1, val2) Valid values: true, false
nIVerilogPrintPortsByLine	bool	user	false	When true, uses formatting style 'print ports by line' for printing. When false, no formatting style applies. Valid values: true, false
nIVerilogPrintScalarWires	bool	user	true	When true, prints scalar nets. When false, no scalar nets printed. Valid values: true, false

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nIVerilogPrintTimeScale	bool	nIVerilog PrintTime Scale	true	When true, prints the time scale data (time units, scale precision) in Verilog netlisting. Valid values: true, false
nIVerilogRangeDelimiterCharacter	string	user	_	The range delimiter character for scalarized instance names in Verilog netlister: ' '. If empty, the bit notation [] is used.
nIVerilogRemoveBangs	bool	user	false	When true, removes ! from all net/terminal and instance names in the Verilog netlist. Valid values: true, false
nIVerilogRenameBusNetsWithPartialTerm	string	user	false	When true, renames buses that are wider than their corresponding terminals in Verilog netlist. Valid values: true, false
nIVerilogRenameSameBaseVectorInsts	bool	user	false	When true, renames vector instances with the same base name to keep their vectors in the netlist instead of scalarizing them. Valid values: true, false
nIVerilogScopeCollisions	string	user	error	Defines the behavior on scope collisions. Valid values: error, warning, silent
nIVerilogSplitModules	bool	user	false	When true, splits final netlist by including separate netlist files. Valid values: true, false
nIVerilogSupply0NetName	string	user	""	Prints net names as supply0 type in the Verilog netlist. Patterns * and ? are supported.
nIVerilogSupply1NetName	string	user	""	Prints net names as supply1 type in the Verilog netlist. Patterns * and ? are supported.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nlVerilogTie0NetName	string	user	""	Prints net names as tie0 type in the Verilog netlist. Patterns * and ? are supported.
nlVerilogTie1NetName	string	user	""	Prints net names as tie1 type in the Verilog netlist. Patterns * and ? are supported.
nlVerilogTimeScalePrecision	string	custom	1ns	Verilog stimulator inter-time step precision.
nlVerilogTimeScaleTimeUnits	string	custom	1ns	Verilog simulator base time step granularity.
nlVerilogVectorExpansion	bool	user	false	When true, all vector ranges are netlisted as individual bits and the range delimiter character refers to the nlVerilogRangeDelimiterCharacter Tcl preference. Valid values: true, false
nlVerilogWireNetTypeList	string	user	trireg tri0 tri1 supply0 supply1 triand trior wand wor wire real wreal scalar voltage_r current_r wreal1dri ver wreal4st ate wreal sum wrealavg wrealmin wreal max interconnect	The supported values of the netType property of a net.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nIVHDLBitOrderAscending	bool	custom	true	When true, arranges bus bits in ascending order. When false, arranges bus bits in descending order. Valid values: true, false
nIVHDLBusBitOrder	string	custom	Ascending	Sets the Bit Order of Bus option in the VHDL Options dialog box. In VHDL netlisting, uses the <code>nIVHDLBitOrderAscending</code> Tcl preference instead.
nIVHDLCompatiblePortOrder	bool	user	true	Port ordering in a way that is compatible with industry standards. Valid values: true, false
nIVHDLCopyDirectory	string	user		Indicates a destination directory where stores the copied text views.
nIVHDLIgnorePropNames	string	user		The list of property names to check for determining whether an instance or master should be ignored.
nIVHDLIllegalCharacterMapping	string	user	escaped	<code>renamed</code> changes illegal characters to other characters while <code>escaped</code> escapes any illegal characters in a string. Valid values: escaped, renamed
nIVHDLInheritedConnectionDataType	string	user	bit	Specifies the port or signal data types of inherited connections in the VHDL netlist.
nIVHDLModelsDirectory	string	user		Specifies the directory for models.
nIVHDLNetlistFiles	string	user	single	Option for netlist files.
nIVHDLPackageOptions	string	user	{Standard.all 0} {IEEE.S TD_Logi c_1164. all 0}	Specifies types of VHDL packages and their use states in the VHDL netlist.

Table 171 Netlist Tool Preferences (Continued)

Name	Type	Scope	Default	Description
nlVHDLScalarDataType	string	user	bit	Specifies the scalar port or signal data types in the VHDL netlist.
nlVHDLUserDataTypeList	string	user		Specifies default values for signals in the VHDL netlist.
nlVHDLVectorDataType	string	user	bit_vector	Specifies bus port or signal data types in the VHDL netlist.
nlVirtualTerminalsFormat	string	user	%PREFIX_%NAME_%DEFAULT	Specifies the format of virtual terminal names with the following patterns: • %NAME: Connection definition name • %DEFAULT: Default net name • %PREFIX: Value of the <code>nlVirtualTerminalsPrefix</code> Tcl preference • %% : Single percent
nlVirtualTerminalsPrefix	string	user	vt	Specifies the prefix to use when composing a virtual terminal name.

Netlister Preference Examples

The following section gives examples for some of the netlister preferences.

- [nl<netlister>CompatiblePortOrder](#)
- [nlIgnoreCell](#)

nl<netlister>CompatiblePortOrder

Custom Compiler supports two different port ordering mechanisms for netlisting: *compatible* and *consistent*.

- [Compatible Netlisting](#)
- [Consistent Netlisting](#)
- [Examples](#)

Compatible Netlisting

During compatible netlisting, the port order is determined according to the following:

1. First, the netlister looks for the existence of the `termOrder` information in the corresponding simulator section under `simInfo` (`hspiceD`, `auCdl`, `spectre`, and so on) in the Parameter Definition Editor of the cell. This is done for all cells, whether or not they contain hierarchy.
2. If `termOrder` is not found, the netlister searches in the following order for the `portOrder` property on the:
 - a. instance
 - b. switched master (usually a schematic view)
 - c. placed master (usually a symbol view)

If the `portOrder` property exists on an instance, then it should exist on all instances of that cell and properties should have the same values. Otherwise, the netlister issues an error message.

3. If the previous two steps do not result in finding a valid port order, the port order is determined according to the following:
 - For the HSPICE netlister, the netlist ordering is alphanumeric.
 - For the Verilog netlister, the netlist ordering is by port type in the following order: inputs, outputs, inout, virtual terms.

Within each port type, the ordering is alphanumeric.

Note:

- In the results of netlisting when the Compatible Port Order mode is set to *false* and the given module has no `portOrder` or `termOrder` specified, the ports are grouped in the following order: outputs, inputs, inout, and virtual terms.
- For the AMS netlisting, the Compatible Port Order mode is always *false*. For the single (non-AMS) netlisting, this mode is controlled by `nl<name>CompatiblePortOrder` Tcl preference with the default value *true*.

Consistent Netlisting

During consistent netlisting, the proper ordering of ports is determined as follows:

1. For non-leaf views, the netlister searches in the following order for the portOrder property on the:
 - a. instance
 - b. switched master (usually a schematic view)
 - c. placed master (usually a symbol view)

Similar to compatible mode, if portOrder is found on one instance but not on another of the same master, or if the properties are not the same, a netlisting error results.

2. If portOrder is not found, the order of ports for both HSPICE and Verilog netlisters is by port type in the following order: outputs, inputs, inout, virtual terms.

Within each port type, the ordering is alphanumeric.

3. For leaf-level views, data compatibility with industry standards is preserved. This means termOrder is honored first, then portOrder, then netlister-specific port ordering as described in Step 2.

Note:

- For both compatible and consistent port ordering of non-leaf cells, if additional ports are found on the view that are not in the portOrder or termOrder, they are ordered alphanumerically with no regard to terminal type and appended to the term list defined by mentioned properties.

Examples

The following examples use an instance with the following terminals:

```
Inputs:  a_in1 d_in2
Outputs: c_out1 f_out2
InputOutputs: b_io1 e_io2
```

Example #1:

```
nl<netlister>CompatiblePortOrder      true
portOrder empty
termOrder empty
```

For Example #1, netlist examples for HSPICE and Verilog are as follows:

```
HSPICE xi0 a_in1 b_io1 c_out1 d_in2 e_io2 f_out2 cella
Verilog cellA I0( c_out1, f_out2, a_in1, d_in2, b_io1, e_io2 );
```

Example #2:

Note that port ordering in HSPICE and Verilog netlists is the same.

```
nl<netlist>CompatiblePortOrder      false
portOrder empty
termOrder empty
```

For Example #2, netlist examples for HSPICE and Verilog are as follows:

```
HSPICE xi0 a_in1 d_in2 c_out1 f_out2 b_io1 e_io2 cella
Verilog cellA I0( a_in1, d_in2, c_out1, f_out2, b_io1, e_io2 );
```

nIgnoreCell

Use Tcl preference `nlIgnoreCell` to declare cellviews as one of the following:

- *ignored* with or without terminal shorts;
- observed as *stop* modules even if they are not netlisted as leaf;
- *not ignored* even if they are declared as *ignored* in the database.

The preference value should be text in YAML format which contains the declarations of above mentioned actions (`ignore`, `stop`, and `notIgnore`) with the following structure:

```
common :
  - <action_description>
  - . . .
  - <action_description>
<netlist_name> :
  - <action_description>
  - . . .
  - <action_description>
. . .
<netlist_name> :
  - <action_description>
  - . . .
  - <action_description>
```

The section with keyword `common` is related to all netlisters. Each section with keyword `<netlist_name>` is related only to the netlister with this name. Each action description should have one of the following formats:

- For action `ignore`:
 - without terminal shorts:
 - `action_:` `ignore`
 - `cellviews :` `<LCV_declarations>`

- with terminal shorts:

```
- action_ : ignore
  cellviews : <LCV_declarations>
  short : <short_declaration>
```

where <LCV_declaration> can contain one or multiple L/C/V strings like these:

```
'Lib1/Cell12/View3'
'Lib1/Cell12/*'
['Lib1/Cell12/View3', 'Lib2/Cell12/View3']
```

Wildcards are allowed, for example: Lib1/*/* refers to all views of all cells of library Lib1.

<short_declaration> has the same format as the value of property lvsRemoveDevView like these:

```
'(short(Term1 Term2))'
'(short(Term1 Term2)) (short(Term3 Term4))'
```

The following example shows that the corresponding action is applied to the given instances in the *parent designs* (specified by the cellviews key). Note that without the *instances* key, the *cellviews* means the switched master designs of the instances in this action.

```
common:
- action : ignore
  cellviews : 'lib1/cell11/*'
  instances : ['I3', 'I7']
- action : notIgnore
  cellviews : 'lib1/cell12/*'
  instances : 'I4'
```

- For action stop:

- with inherited connections calculation (for example: with virtual terminals addition:

```
- action : stop
  cellviews : <LCV_declarations>
```

or

```
- action : stop
  cellviews : <LCV_declarations>
  inhConn : true
```

- without inherited connections calculation:

```
- action : stop
  cellviews : <LCV_declarations>
  inhConn : false
```


- For action `notIgnore`:

```
- action : notIgnore  
cellviews : <LCV_declarations>
```

The actions declared by this preference have higher priority than the actions declared by the database (property, parameter). If there are conflicting actions declared for the same LCV in the `common` and `netlister-scope` sections of the preference value, the `netlister-scoped` action wins. If there are conflicting actions declared for the same LCV within the same scope, the `action` declared later wins.

19

Geometry Manipulation Tcl

This chapter describes the commands and preferences in the Custom Compiler Tcl po namespace. This namespace provides a fast and easy geometry manipulation capability directly in Custom Compiler, without the need to write a runset or execute physical verification tools.

The Tcl po namespace allows you to do complex geometry operations using a daisy-chain method. This chapter includes the following sections:

- [Commands](#)
- [Preferences](#)
- [Geometry Manipulation Example](#)

Commands

The commands covered are:

- [po::and](#)
- [po::angle](#)
- [po::append](#)
- [po::area](#)
- [po::chop](#)
- [po::coinEdge](#)
- [po::compress](#)
- [po::convexEdge](#)
- [po::createCollection](#)
- [po::createPointSet](#)
- [po::cut](#)

- [po::cutCorner](#)
- [po::density](#)
- [po::donut](#)
- [po::enclose](#)
- [po::encloseRect](#)
- [po::enclosure](#)
- [po::expandEdge](#)
- [po::extent](#)
- [po::extents](#)
- [po::external](#)
- [po::getAngle](#)
- [po::getArea](#)
- [po::getContours](#)
- [po::getDistance](#)
- [po::getHoles](#)
- [po::getIntersection](#)
- [po::getMaxBox](#)
- [po::getPerimeter](#)
- [po::getPoints](#)
- [po::getSpecifiedBoxes](#)
- [po::inside](#)
- [po::insideEdge](#)
- [po::interact](#)
- [po::internal](#)
- [po::isClockwise](#)
- [po::isHorizontal](#)
- [po::isVertical](#)

- [po::length](#)
- [po::makeClockwise](#)
- [po::manhattanContour](#)
- [po::not](#)
- [po::offGrid](#)
- [po::or](#)
- [po::outside](#)
- [po::outsideEdge](#)
- [po::pathLength](#)
- [po::perimeter](#)
- [po::polyToTrap](#)
- [po::rectangle](#)
- [po::rectangles](#)
- [po::rectEnclosure](#)
- [po::reshape](#)
- [po::shapePoints](#)
- [po::shift](#)
- [po::size](#)
- [po::snapGrid](#)
- [po::split](#)
- [po::straddle](#)
- [po::touch](#)
- [po::touchEdge](#)
- [po::transform](#)
- [po::vertex](#)
- [po::withEdge](#)
- [po::withNeighbor](#)

- [po::withWidth](#)
 - [po::xor](#)
-

po::and

Description

Run the Boolean AND operation.

Syntax

```
po::and <input: db:Collection | poPointSet> -with <dbCollection |  
poPointSet>
```

Arguments

input

Specify the operand1 points container.

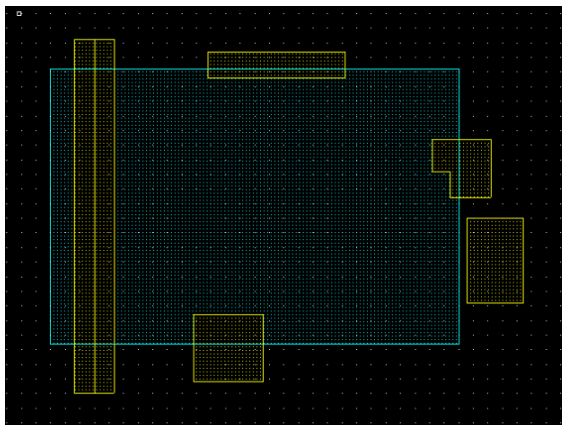
-with

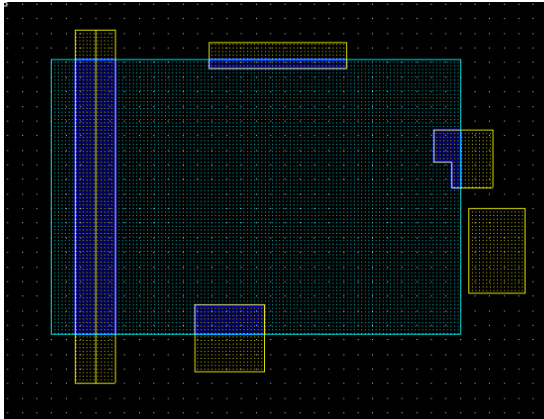
Specify the operand2 points container.

Returns

<dbCollection | poPointSet> : The result points container.

Examples





```
# po::and
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shapes on m2 drawing
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
# and all polygons from m2 layer with shapes on m1 layer
set and [po::and $cm2 -with $cm1]
# select shapes in and collection
db::foreach p [db::getattr and.pointArray] {
    #create shapes on poly layer
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

po::angle

Description

Selects all edges from input-collections that have a smaller angle magnitude with respect to the x-axis of the coordinate system, which conform to the constraint specified by `-value`.

Syntax

```
po::angle <input: dbCollection> -value <string> [-not <bool>]
```

Arguments

input

Specifies the input-collection.

`-value`

Specifies the small angle magnitude constraint interpreted in degrees. (For example: `==90`)

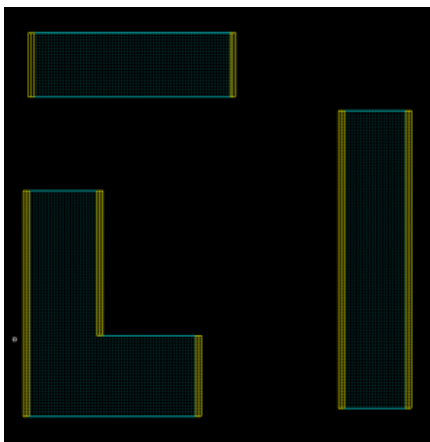
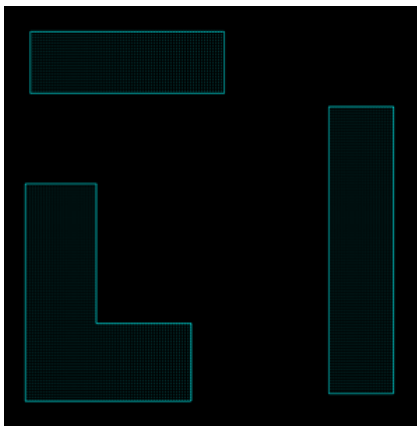
`-not`

`-not true` is the complement of the angle operation. (Default: false)

Returns

`<dbCollection>` : The result points container.

Examples



In this example, this command returns points of edges from source collection for those angle is equal to 90. The path on resulting points is created.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
# Creates source collection of points for shapes on m1 layer
set srcPts [po::createCollection $m1Shape]
set edges [po::angle $srcPts -value ==90]
```

```
# Create path objects on resulting points
db::foreach edge $edges {
    le::createPath [db::getattr edge.pointArray] -design $design \
        -lpp m2 -width 0.07
}
```

po::append

Description

Appends a polygon or polyline or polygons into a polygon or polyline container.

Syntax

```
po::append <input: [oaShape | point | {<deFigure>}]> -to <dbCollection |
    poPointSet> [-dbuPerUU <int>]
```

Arguments

input

Specify the input objects.

-to

Specify the polygons or polylines container.

-dbuPerUU

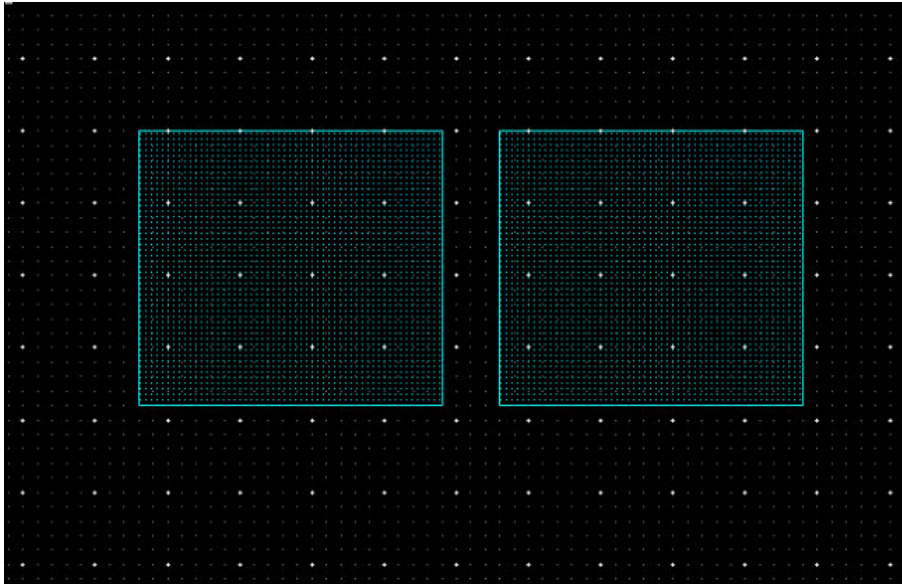
Specify dbuPerUU for points input. (Default: 1000)

Returns

<none>

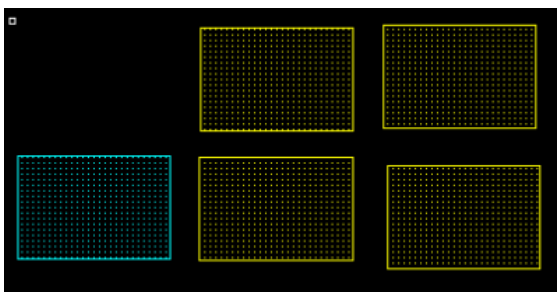
Examples

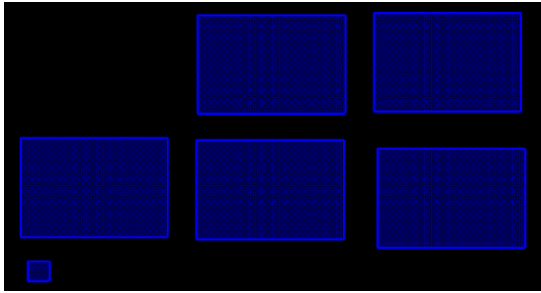
Example 1



```
# po::createCollection
# create collection with oaShapes
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
set shapes [db::getShapes -of $design]
set rec1 [po::createCollection [db::getNext $shapes]]
set rec2 [po::createCollection [list {0.125 0.0} \
    {0.125 0.095} {0.23 0.095} {0.23 0.0}] -closed true]
db::getattr rec1.pointArray
po::getDistance [db::getattr rec1.pointArray] \
    -to [db::getattr rec2.pointArray]
```

Example 2





```
# po::createCollection
# create collection with oaShapes
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]

#Add new oa shape into existing collection
set shapes [db::getShapes -lpp {m1 drawing} -of $design]
set col [po::createCollection $shapes]
po::append [list {0.125 0.0} {0.125 0.095} {0.23 0.095} {0.23 0.0}] \
    -to $col
set shapes [db::getShapes -lpp {m2 drawing} -of $design]

db::foreach oaShape $shapes {
    po::append $oaShape -to $col
}
set points [db::createList [db::getattr col.pointArray]]
puts "$points"
# create polygones with points from collection
foreach point $points {
    le::createPolygon $point -lpp {poly drawing} -design $design
}
```

po::area

Description

Select all merged-polygons in input-collection having areas that conform to the constraint specified by "-value".

Syntax

```
po::area <input: dbCollection> -value <string> [-not <bool>]
```

Arguments

input

Specify input-collection.

`-value`

Specify the constraint representing the area of merged polygons in user units squared. (For example: ≥ 2)

`-not`

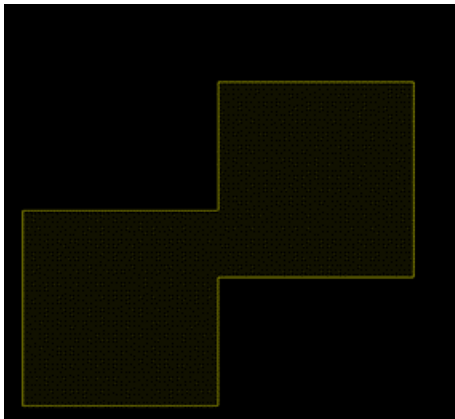
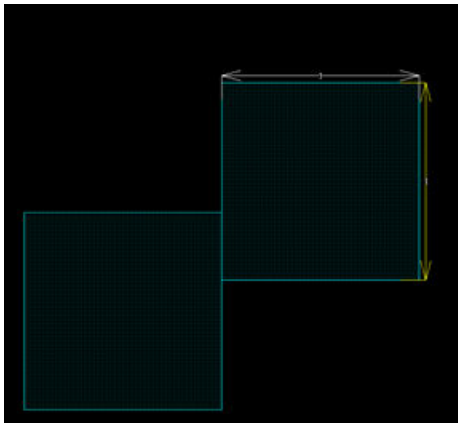
"-not true" is the complement of the area operation. (Default: false)

Returns

`<dbCollection>` : The result points container.

Examples

Example 1



In this example, this command returns collection of merged area (that is equal to $2.0u^2$) points. The polygon with resulting points is created.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
# Creates source collection of points for shapes on m1 layer
set srcPts [po::createCollection $m1Shape]
```

```
set mergeArea [po::area $srcPts -value ==2]
le::createPolygon [db::getAttr mergeArea.pointArray] \
    -lpp m2 -design $design
```

Example 2

In this example, this command returns an empty collection.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
# Creates source collection of points for shapes on m1 layer
set srcPts [po::createCollection $m1Shape]
set mergeArea [po::area $srcPts -value ==2 -not true ]
db::isEmpty $mergeArea => 1
```

po::chop

Description

Chop polylines by a box.

Syntax

```
po::chop <input: dbCollection | poPointSet> -by <box>
```

Arguments

input

Specify the polyline points container.

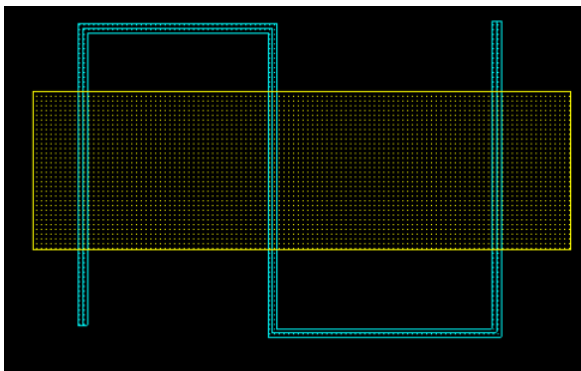
-by

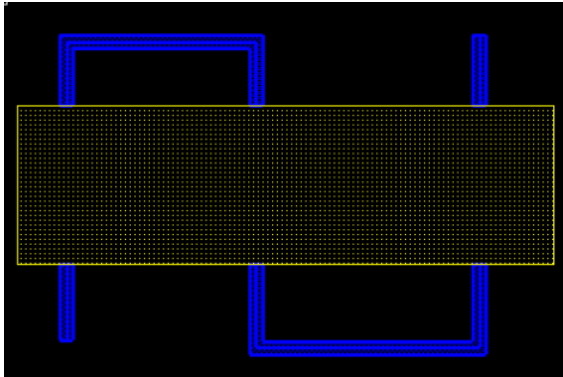
Specify the box to chop polylines.

Returns

<poPointSet> : The result points container.

Examples





```
# po::chop
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# Get Path shapes on m1 drawing lpp
set m1 [db::getShapes -filter {%type == "Path"} -lpp {m1 drawing} \
    -of $design]
#create point set with selected shape
set poSet1 [po::createPointSet -closed false]
set inputPoints [db::getattr m1.points]
po::append $inputPoints -to $poSet1
# Get shape on m2 drawing lpp
set m2 [db::getNext [db::getShapes -lpp {m2 drawing} -of $design]]
# Get the bbox of the shape on m2 layer.
set bbox [db::getattr m2.bBox]
set resultPointSet [po::chop $poSet1 -by $bbox]
db::foreach point [db::getattr resultPointSet.polygons] {
    #create shapes on poly layer
    le::createPath [db::getattr point.pointArray] -lpp {poly drawing} \
        -design $design -width 0.1
}
```

po::coinEdge

Description

Selects all edges or edge portions from the input-collection that coincide with edges of the with-collection.

Syntax

```
po::coinEdge <input: dbCollection> -with <dbCollection> [-dir <enum>]
[-not <bool>]
```

Arguments

input

Specifies the input-collection.

`-with`

Specifies the with-collection.

`-dir`

Specifies the touch direction. Valid values: all, inside, outside. (Default: all)

`-not`

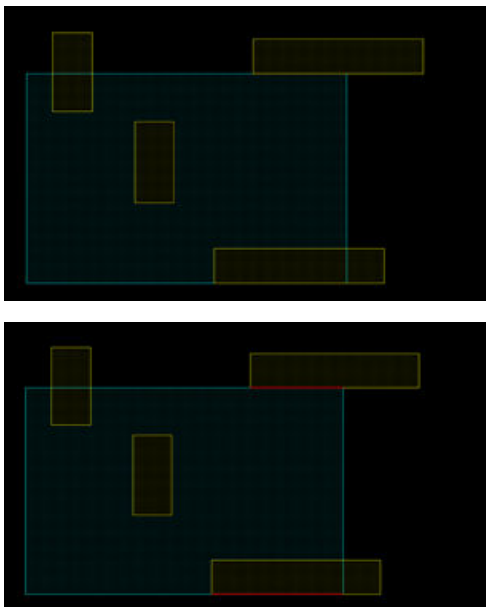
`-not true` is the complement of the `coinEdge` operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples

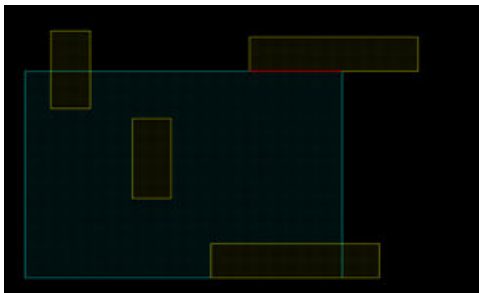
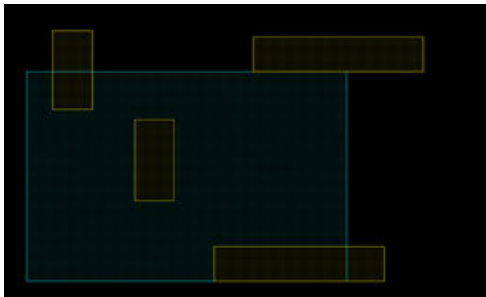
Example 1



In this example, this command returns collection of point objects of the m2 metal layer shape coincident edges with m1 layer shape. The result edges are highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]
set resPoints [po::coinEdge $srcPts -with $withPts -dir all -not 0]
db::foreach pt $resPoints {
    xt::highlightViolation [db::getattr pts.pointArray] -design $design \
        -toolName po -packet highlight2 -isClosed
}
```

Example 2

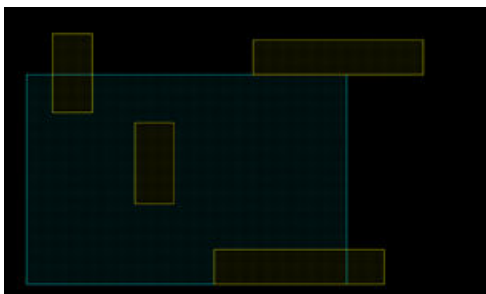


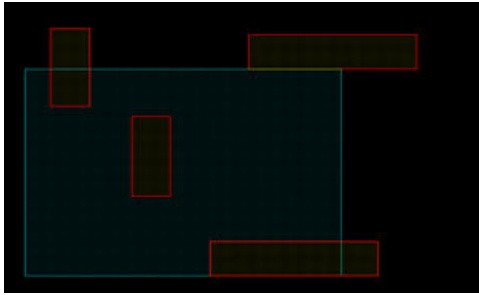
In this example, this command returns collection of point objects of the m2 layer outside shape coincident edges with m1 layer shape. The result edge is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]

set resPoints [po::coinEdge $srcPts -with $withPts -dir outside -not 0]
db::foreach pt $resPoints {
    xt::highlightViolation [db::getattr pts.pointArray] -design $design \
        -toolName po -packet highlight2 -isClosed
}
```

Example 3



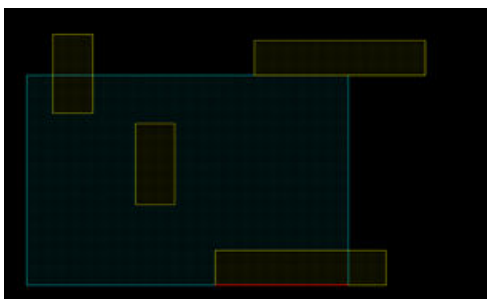
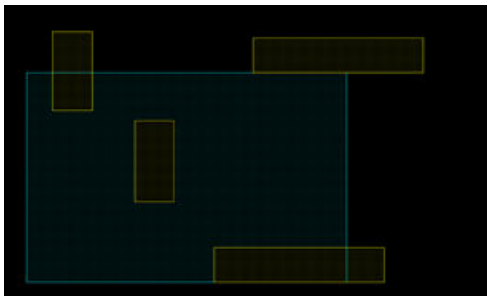


In this example, this command returns collection of point objects of the m2 layer outside shape non-coincident edges with m1 layer shape. The result edges are highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]

set resPoints [po::coinEdge $srcPts -with $withPts -dir outside -not 1]
db::foreach pt $resPoints {
    xt::highlightViolation [db::getattr pts.pointArray] -design $design \
        -toolName po -packet highlight2 -isClosed
}
```

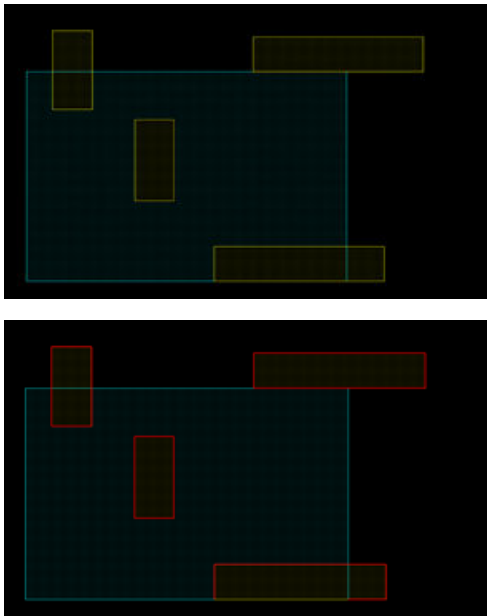
Example 4



In this example, this command returns collection of point objects of the m2 layer inside shape coincident edges with m1 layer shape. The result edges are highlighted.


```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]
set resPoints [po::coinEdge $srcPts -with $withPts -dir inside -not 0]
db::foreach pt $resPoints {
    xt::highlightViolation [db::getattr pts.pointArray] -design $design \
        -toolName po -packet highlight2 -isClosed
}
```

Example 5



In this example, this command returns collection of point objects of the m2 layer inside shape non-coincident edges with m1 layer shape. The result edges are highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]
set resPoints [po::coinEdge $srcPts -with $withPts -dir inside -not 1]
db::foreach pt $resPoints {
    xt::highlightViolation [db::getattr pts.pointArray] -design $design \
        -toolName po -packet highlight2 -isClosed
}
```

po::compress

Description

Removes all collinear and coincident points.

Syntax

```
po::compress <points: [point]> [-closed <bool>] [-dbuPerUU <int>]
```

Arguments

`points`

The list of points that form compressing points.

`-closed`

Specify the input points is closed or not. (Default: true)

`-dbuPerUU`

Specify dbuPerUU for points input. (Default: 10000)

Returns

[<point>] : The compressed points.

Examples

In this example, this command removes {1 1} {2 2} {3 2} {3 1} collinear points.

```
po::compress {{0 0} {1 1} {2 2} {3 3} {3 2} {3 1} {3 0}} => {0.0 0.0}
{3.0 3.0} {3.0 0.0}
```

po::convexEdge

Description

Selects edges from the input-collection based upon the number of convex endpoints the edges have.

Syntax

```
po::convexEdge <input: dbCollection> (-value <string> | -angle1 <string>)
[-length1 <string>] [-angle2 <string>] [-length2 <string>] [-withLength
<string>]
```

Arguments

`input`

Specifies the input collection.

`-value`

Specifies the constraint representing the number of endpoints (0, 1, or 2), which must lie at a convex corner for an edge to be selected (For example: `==1`).

`-angle1`

Specifies the angle constraint of one endpoint of edges (For example: `==90`).

`-length1`

Specifies the length constraint of edges abutting the endpoint satisfying `-angle1` (For example: `<=0.1`). This option cooperates with `-angle1` and is specified in user units.

`-angle2`

Specifies the angle constraint of the other endpoint of the edges (For example: `==270`). This option cooperates with `-`.

`-length2`

Specifies the length constraint of edges abutting the endpoint satisfying `"-angle2"` (For example: `<=0.1`). This option cooperates with `"-angle2"` and is specified in user units.

`-withLength`

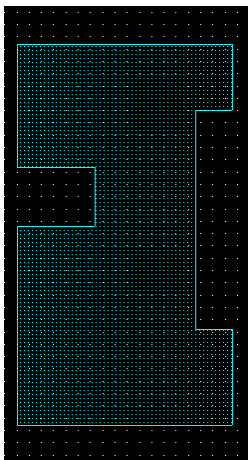
Specifies the length constraint in user units to limit the output edges (For example: `==0.05`).

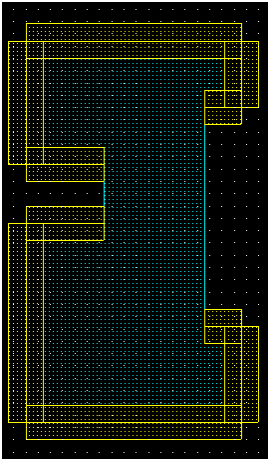
Returns

`<dbCollection>` : The result points container.

Examples

Example 1

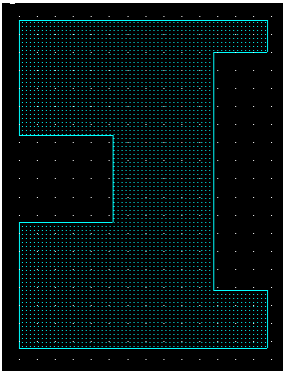


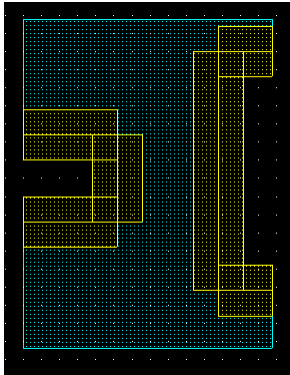


In this example, this command returns edges neighboring convex points of source polygon. The paths on resulting points are created.

```
set design [ed]
set m1Shape [db::getShapes -of $design]
set srcPts [po::createCollection $m1Shape]
set con [po::convexEdge $srcPts -value >=1]
# Create path objects on resulting points
db::foreach edge $con {
    le::createPath [db::getattr edge.pointArray] -design $design \
        -lpp m2 -width 0.07
}
```

Example 2





In this example, this command returns edges neighboring concave points of source polygon. The paths on resulting points are created.

```
set design [ed]
set m1Shape [db::getShapes -of $design]
set srcPts [po::createCollection $m1Shape]

set con [po::convexEdge $srcPts -angle1 >180 -angle2 <360]
# Create path objects on resulting points
db::foreach edge $con {
    le::createPath [db::getattr edge.pointArray] -design $design \
        -lpp m2 -width 0.07
}
```

po::createCollection

Description

Creates a container to contain multiple one-shot polygons or polylines.

Syntax

```
po::createCollection [<input: [oaShape | point | {<deFigure>} |
    {<oaShape>}]>] [-dbuPerUU <int>] [-closed <bool>]
```

Arguments

input

Specify the input points, objects or collections.

-dbuPerUU

Specify dbuPerUU for points input. (Default: 1000)

-closed

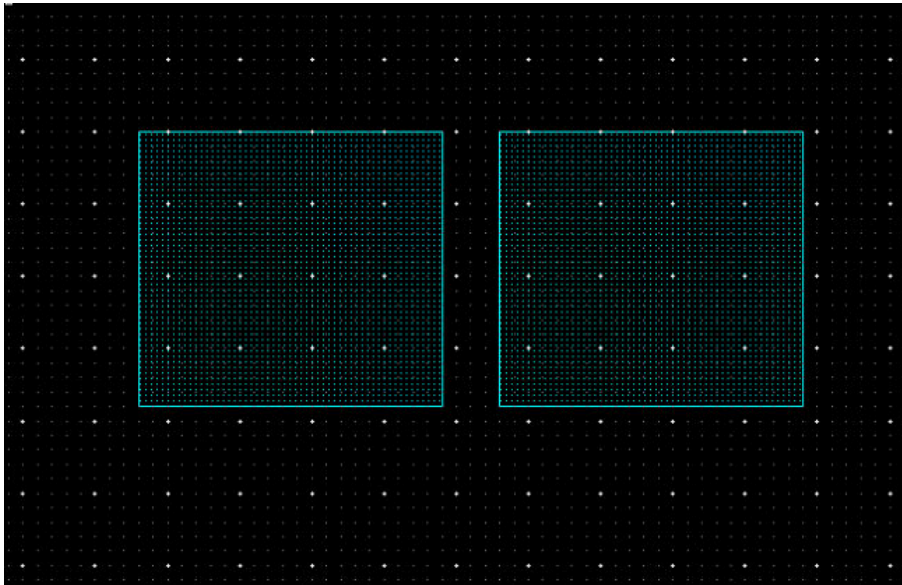
Indicate the container containing multiple polygons points or multiple polylines points. (Default: true)

Returns

<dbCollection> : The points collection.

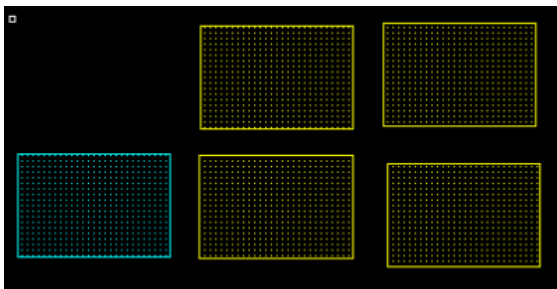
Examples

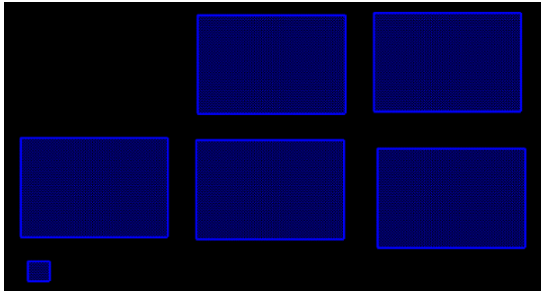
Example 1



```
# po::createCollection
# create collection with oaShapes
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
set shapes [db::getShapes -of $design]
set rec1 [po::createCollection [db::getNext $shapes]]
set rec2 [po::createCollection \
    [list {0.125 0.0} {0.125 0.095} {0.23 0.095} {0.23 0.0}] \
    -closed true]
db::getattr rec1.pointArray
po::getDistance [db::getattr rec1.pointArray] \
    -to [db::getattr rec2.pointArray]
```

Example 2





```
# po::createCollection
# create collection with oaShapes
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]

#Add new oa shape into existing collection
set shapes [db::getShapes -lpp {m1 drawing} -of $design]
set col [po::createCollection $shapes]
po::append [list {0.125 0.0} {0.125 0.095} {0.23 0.095} {0.23 0.0}] \
    -to $col
set shapes [db::getShapes -lpp {m2 drawing} -of $design]

db::foreach oaShape $shapes {
    po::append $oaShape -to $col
}
set points [db::createList [db::getattr col.pointArray]]
puts "$points"
# create polygones with points from collection
foreach point $points {
    le::createPolygon $point -lpp {poly drawing} -design $design
}
```

po::createPointSet

Description

Create a points set to contain multiple one-shot polygons.

Syntax

```
po::createPointSet [<input: [oaShape | point]>] [-dbuPerUU <int>][-closed
    <bool>]
```

Arguments

input

Specify the input oaShapes or points.

-dbuPerUU

Specify dbuPerUU for points input. (Default:1000)

`-closed`

Indicate the `poPointSet` containing multiple polygon points of multiple polyline points. (Default:true)

Returns

`<poPointSet>` : The points set.

po::cut

Description

Selects all polygons from the input-collection that share some (but not all) of their area with polygons of the with-collection.

Syntax

```
po::cut <input: dbCollection> -with <dbCollection> [-value <string>]  
[-hasEvenOdd <enum>] [-not <bool>]
```

Arguments

`input`

The input-collection.

`-with`

The with-collection.

`-value`

The selection criterion (For example: `>=2`).

`-hasEvenOdd`

Cooperates with `"-value"` to specify the extra selection criterion. Valid values: even, none, odd. (Default: none)

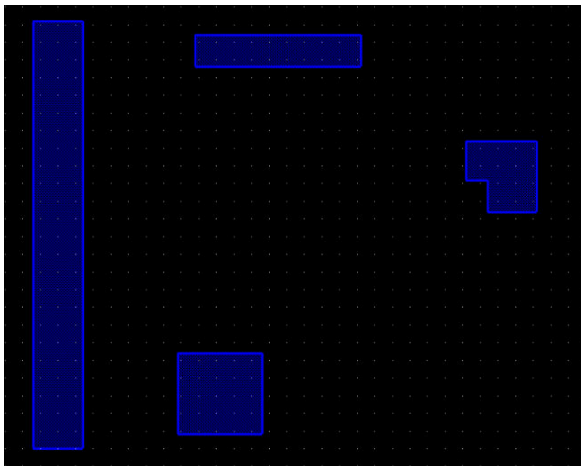
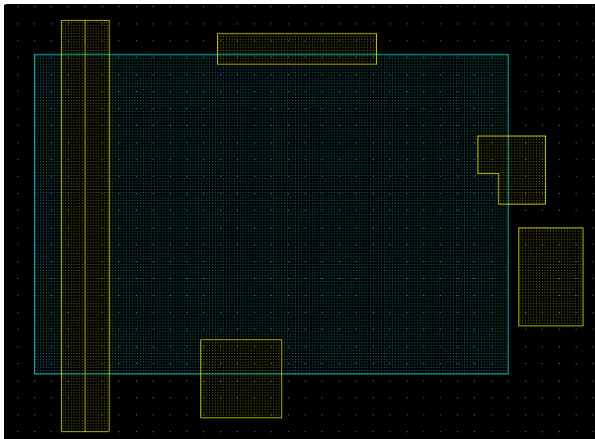
`-not`

`"-not true"` is the complement of the cut operation. (Default: false)

Returns

`<dbCollection>` : The result points container.

Examples



```
# po::cut
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
#get shape on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shape on m2 drawing
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
# cut selects all polygons from m2 layer that share
# some of their area with polygons of the m1 layer
set cut [po::cut $cm2 -with $cm1]

# select shapes in "cut" collection
db::foreach p [db::getattr cut.pointArray] {
    #create similar shapes on m4 layer
```

```
le::createPolygon $p -lpp {m4 drawing} -design $design  
}
```

po::cutCorner

Description

Cuts the selected corners of polygons based on cut-length and number-of-sides.

Syntax

```
po::cutCorner <input: dbCollection | poPointSet> [-cutLength <float>]  
[-numSides <int>] [-selected <box>]
```

Arguments

input

Specify the operand1 points container.

-cutLength

Specify the cut Length for a corner.

-numSides

Specify the number of sides after cut a corner. (Default: 1)

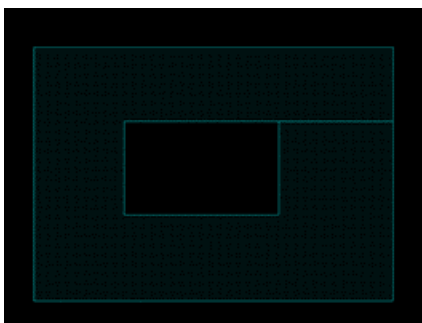
-selected

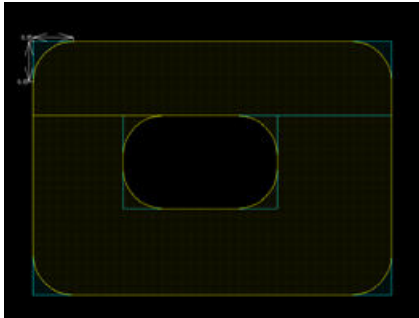
Specify the bounding box to determine the selected corners.

Returns

<dbCollection | poPointSet> : The result points container.

Examples





In this example, this command cuts corners of source polygon with specified 0.15 length and 8 number of sides. And, the polygon with result points on m2 layer is created.

```
set design [ed]
set m1Shape [db::getShapes -of $design]
set srcPts [po::createCollection $m1Shape]
set resPolygon [po::cutCorner $srcPts -cutLength 0.15 -numSides 8]
le::createPolygon [db::getAttr resPolygon.pointArray] -lpp m2 \
    -design $design
```

po::density

Description

Output the data capture windows whose area ratio (density) meets the constraint specified by "-value".

Syntax

```
po::density <input: dbCollection> [-insideOf <box | dbCollection> |
    -design <oaDesign>] -value <string> [-window <point>] [-step <point>]
    [-slideMode <enum>] [-gradient <string>] [-gradientMode <enum>] [-corner
    <bool>] [-outputAll <bool>]
```

Arguments

input

Specify input-collection.

-value

Specify the density (area ratio) constraint. (For example: >0.5)

-insideOf

If specify <box>, the box is the data capture extent. If specify <dbCollection>, the extents of polygons in <dbCollection> are the data capture extents.

-design

If specify <oaDesign>, the bounding box of <oaDesign> is the data capture extent.

-window

Specify the dimensions of the data capture window within the data capture extent.

-step

Specify the grid increment for which data capture windows are tiled within the data capture extent.

-slideMode

Cooperate with "-window" and "-step" to define how to handle data capture windows overlapping the right/top edges of data capture extents. Valid values: backup, ignore, truncate. (Default: backup)

-gradient

Specify the gradient constraint representing the density value change from one window to adjacent nonoverlapping windows. (For example: >0.3)

-gradientMode

Specify how to calculate the gradient of the data capture window. Valid values: absolute, relative. (Default: relative)

-corner

If specify "-corner true", the set of windows in the gradient calculation is extended to include the four windows touching current window on each corner. (Default: false)

-outputAll

If specify "-outputAll true", output all data capture windows. (Default: false)

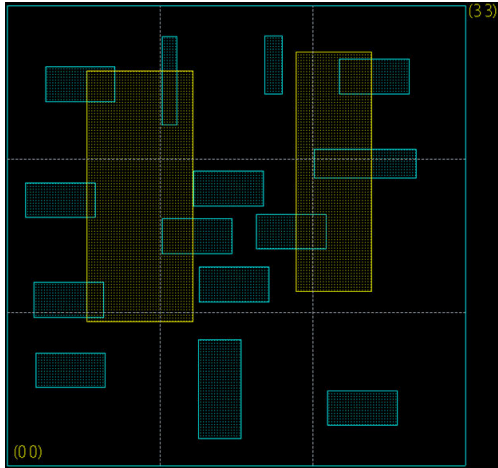
Returns

<dbCollection> : The result points container.

Examples

```
set design [ed]
# Get m1 layer shapes, those will be used as an input collection to
# calculate density in examples below
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]
# Get m2 layer shapes, those will be used as a data capture window
# extents to calculate density.
```

```
set m2Shape [db::getShapes -lpp m2 -of $design]
set insidePts [po::createCollection $m2Shape]
```



Example 1

In this example, the `po::density` command returns m1 layer shapes density in a design.

```
set denPts [po::density $srcPts -design $design -value >0.1]
puts [db::getAttr denPts.density]
0.147004
```

Example 2

In this example, the `po::density` command returns m1 layer shapes density inside of m2 shapes area.

```
set denPts [po::density $srcPts -insideOf $insidePts -value >0.1]
db::foreach pts $denPts {
    puts [format "Density: %0.4f Points:%s" [db::getAttr pts.density] \
        [db::getAttr pts.pointArray]]
}
=>
Density: 0.2096 Points: {1.89 1.135} {2.385 1.135} {2.385 2.7} {1.89
2.7}
Density: 0.1341 Points: {0.52 0.94} {1.215 0.94} {1.215 2.575} {0.52
2.575}
```

Example 3

In this example, the `po::density` command returns m1 layer shapes density inside of {0 0} {3 3} window for each data capture (1x1) windows, if the density value is greater than 0.1.

```
set denPts [po::density $srcPts -insideOf {{0 0} {3 3}} -value >0.1 \
    -window {1 1}]
db::foreach pts $denPts {
```

```
puts [format "Density: %0.4f Points:%s" [db::getAttr pts.density] \
      [db::getAttr pts.pointArray]]
}
```

```
Density: 0.1456 Points: {2.0 2.0} {3.0 2.0} {3.0 3.0} {2.0 3.0}
Density: 0.1024 Points: {0.0 2.0} {1.0 2.0} {1.0 3.0} {0.0 3.0}
Density: 0.1022 Points: {2.0 1.0} {3.0 1.0} {3.0 2.0} {2.0 2.0}
Density: 0.3904 Points: {1.0 1.0} {2.0 1.0} {2.0 2.0} {1.0 2.0}
Density: 0.1911 Points: {0.0 1.0} {1.0 1.0} {1.0 2.0} {0.0 2.0}
Density: 0.1024 Points: {2.0 0.0} {3.0 0.0} {3.0 1.0} {2.0 1.0}
Density: 0.1792 Points: {1.0 0.0} {2.0 0.0} {2.0 1.0} {1.0 1.0}
Density: 0.1160 Points: {0.0 0.0} {1.0 0.0} {1.0 1.0} {0.0 1.0}
```

Example 4

In this example, the `po::density` command returns m1 layer shapes density inside of {0 0} {3 3} window for each data capture (1x1) windows, if the density value is greater than 0.1 and relative gradient greater than 0.4.

```
set denPts [po::density $srcPts -insideOf {{0 0} {3 3}} -value >0.1 \
          -window {1 1} -gradientMode relative -gradient >0.4 ]

db::foreach pts $denPts {
    puts [format "Density: %0.4f Gradient: %0.4f Points:%s" \
        [db::getAttr pts.density] [db::getAttr pts.gradient] \
        [db::getAttr pts.pointArray]]
}
```

```
Density: 0.1024 Gradient: 0.4643 Points: {0.0 2.0} {1.0 2.0} {1.0 3.0}
{0.0 3.0}
Density: 0.1022 Gradient: 0.7381 Points: {2.0 1.0} {3.0 1.0} {3.0 2.0}
{2.0 2.0}
Density: 0.3904 Gradient: 0.7481 Points: {1.0 1.0} {2.0 1.0} {2.0 2.0}
{1.0 2.0}
Density: 0.1911 Gradient: 0.5105 Points: {0.0 1.0} {1.0 1.0} {1.0 2.0}
{0.0 2.0}
Density: 0.1024 Gradient: 0.4287 Points: {2.0 0.0} {3.0 0.0} {3.0 1.0}
{2.0 1.0}
Density: 0.1792 Gradient: 0.5410 Points: {1.0 0.0} {2.0 0.0} {2.0 1.0}
{1.0 1.0}
```

po::donut

Description

Selects all polygons from the input-collection which have interior cycles (holes) that are fully enclosed within a polygon.

Syntax

```
po::donut <input: dbCollection> [-value <string>] [-not <bool>]
```

Arguments

`input`

The input-collection.

`-value`

The specific number of interior cycles for a polygon. (For example: ≥ 2)

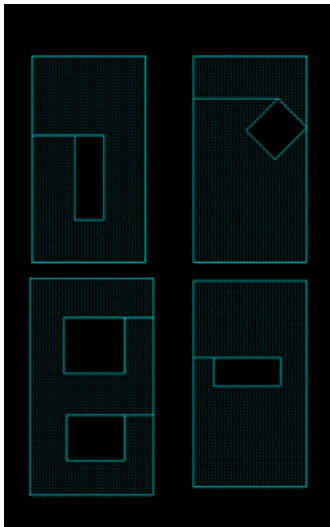
`-not`

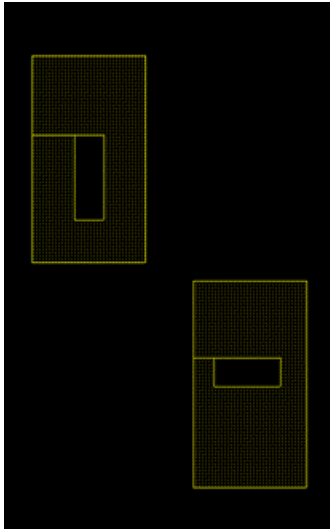
"-not true" is the complement of the donut operation. (Default: false)

Returns

`<dbCollection>` : The result points container.

Examples





In this example, this command returns polygons those have completely enclosed one hole, from source points collection. The polygons with resulting points are created.

```
set design [ed]
set m1Shape [db::getShapes -of $design]
set srcPts [po::createCollection $m1Shape]
set resPts [po::donut $srcPts -value 1]
db::foreach points [db::getattr resPts.pointArray] {
    le::createPolygon $points -design [ed] -lpp m2
}
```

po::enclose

Description

Selects the polygons from the input-collection containing any polygons of with-collection.

Syntax

```
po::enclose <input: dbCollection> -with <dbCollection> [-value <string>]
[-hasEvenOdd <enum>] [-not <bool>]
```

Returns

<dbCollection> : The result points container

Arguments

input

Specify input-collection.

-with

Specify with-collection.

-value

Specify the selection criterion. (For example: ≥ 2)

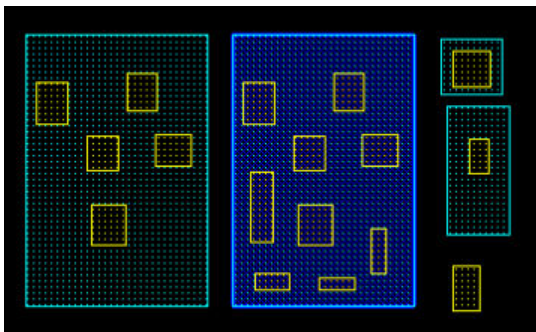
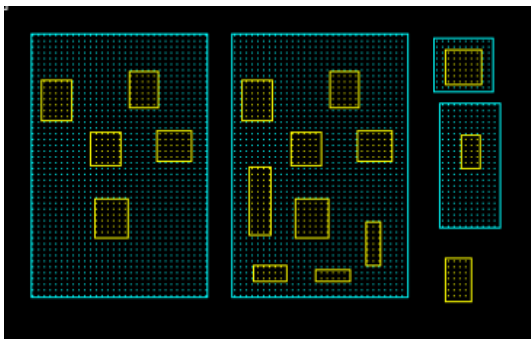
-hasEvenOdd

Cooperate with "-value" to specify the extra selection criterion. Valid values: even, none, odd. (Default: none)

-not

"-not true" is the complement of the enclose operation. (Default: false)

Examples



```
# po::enclose
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shapes on m2 drawing
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
```

```
# select shapes on m1 layer which enclose more than 5 shapes in m2 layer
set en [po::enclose $cm1 -with $cm2 -value >5]
# select shapes in "en" collection
db::foreach p [db::getattr en.pointArray] {
    #create shapes on poly layer
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

po::encloseRect

Description

Selects all polygons from the input-collection that can enclose an orthogonal rectangle of the specified width and length.

Syntax

```
po::encloseRect <input: dbCollection> -width <float> -length <float>
[-isOrthogonalOnly <bool>] [-not <bool>]
```

Arguments

input

Specify input-collection.

-width

Specify the width of the enclosed orthogonal rectangle in user units.

-length

Specify the length of the enclosed orthogonal rectangle in user units.

-isOrthogonalOnly

"-isOrthogonalOnly false" selects polygons containing orthogonal/diagonal rectangles. (Default: true)

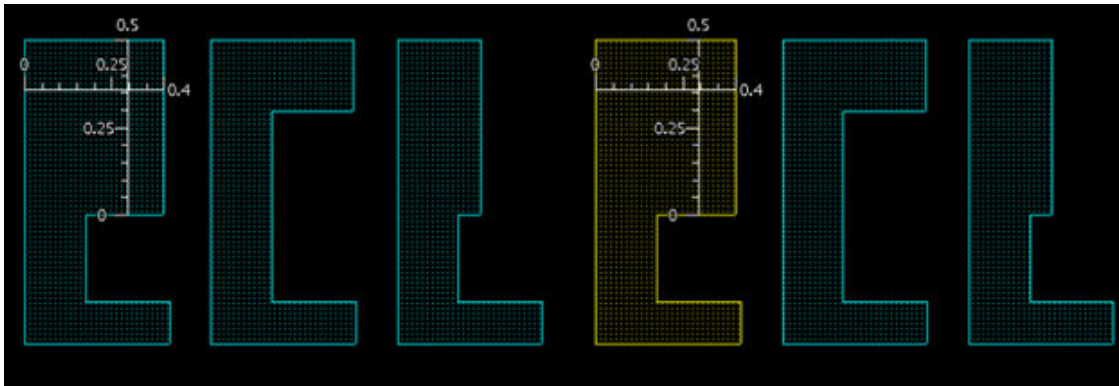
-not

"-not true" is the complement of the encloseRect operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples



In this example, this command selects polygon that encloses 0.5x0.4u rectangle. The polygons with resulting points are created.

```
set design [ed]
set m1Shape [db::getShapes -of $design]
set srcPts [po::createCollection $m1Shape]
set boxes [po::encloseRect $srcPts -length 0.5 -width 0.4]
db::foreach points [db::getattr boxes.pointArray] {
    le::createPolygon $points -design [ed] -lpp m2
}
```

po::enclosure

Description

Measures the separation between the exterior-facing sides of edges in an input-collection and the interior-facing sides of edges in a with-collection.

Syntax

```
po::enclosure <input: dbCollection> -with <dbCollection> -value <string>
[-metric <enum>] [-oppositeExt <float>] [-acute <enum>] [-parallel
<enum>] [-perpendicular <enum>] [-obtuse <enum>] [-projectionFilter
<enum>] [-projLength <string>] [-angled <string>] [-cornerFilter
<enum>] [-abut <string>] [-singular <bool>] [-intersectingOnly <bool>]
[-measureAll <bool>] [-outputType <enum>]
```

Arguments

input

The input-collection.

-with

The with-collection.

`-value`

The measuring constraint in user units. (For example: <0.05)

`-metric`

Measures edge pairs based on the specific metric. Valid values: euclidean, opposite, square. (Default: euclidean)

`-oppositeExt`

Cooperates with "-metric opposite". A non-negative number in user units specifies the amount to extend both ends of an edge. (Default: 0)

`-acute`

Measures edge pairs based on the acute condition. Valid values: also, none, only. (Default: also)

`-parallel`

Measures edge pairs based on the parallel condition. Valid values: also, none, only. (Default: also)

`-perpendicular`

Measures edge pairs based on the perpendicular condition. Valid values: also, none, only. (Default: none)

`-obtuse`

Measures edge pairs based on the obtuse condition. Valid values: also, none, only. (Default: none)

`-projectionFilter`

Specifies the condition of the mutual edge projection. Valid values: all, notProj, proj. (Default: all)

`-projLength`

Cooperates with "-projectionFilter proj". Specify the projection length constraint and the constraint value as a non-negative number in user units. (For example: >0) (Default: >=0)

`-angled`

Measures the two edges only when the number of non-orthogonal edges in the pair meets the given constraint. (For example: ==0)

`-cornerFilter`

Measures edge pairs based on corner-to-corner or corner-to-edge orientation. Valid values: Valid values: all, corner, cornerToCorner, cornerToEdge, notCorner. (Default: all)

`-abut`

Specifies that the separation between intersecting edges should also be checked under the specified constraint (For example: <90).

`-singular`

Handles polygon singularities that are point-to-edge or point-to-point polygon intersections. (Default: false)

`-intersectingOnly`

Cooperates with `-abut` or `-singular`. Specifies whether or not to measure only between intersecting edges and to ignore non-intersecting edges. (Default: false)

`-measureAll`

By default, considers polygon containment criteria. If the value of this option is `-measureAll true`, it will ignore polygon containment criteria. (Default: false)

`-outputType`

Specifies the output type. Valid values: `negEdgeA`, `negEdgeB`, `posEdgeA`, `posEdgeB`, `region`. (Default: `region`)

Returns

`<dbCollection>` : The result points container.

Examples

```
set design [ed]
set outer [db::getShapes -lpp m1 -of $design]
set inner [db::getShapes -lpp m2 -of $design]
set outPts [po::createCollection $outer]
set inPts [po::createCollection $inner]
```

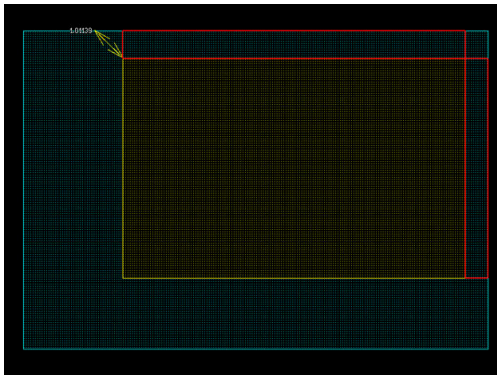
Example 1



In this example, the `po::enclosure` command measures enclosure and returns points collection with euclidean metric, if it is less than 1u. The result polygon is highlighted.

```
set respts [po::enclosure $inPts -with $outPts -value <1 \
    -metric euclidean ]
db::foreach pts [db::getattr respts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
    highlight2 -isClosed
}
```

Example 2



In this example, the `po::enclosure` command measures enclosure and returns points collection with opposite metric, if it is less than 1u. The result polygons are highlighted.

```
set respts [po::enclosure $inPts -with $outPts -value <1 \
    -metric opposite]
db::foreach pts [db::getattr respts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
    highlight2 -isClosed
}
```

Example 3



In this example, the `po::enclosure` command measures enclosure and returns enclosed polygon edge points, if the enclosure is less than 1u. The result edges are highlighted.

```
set respts [po::enclosure $inPts -with $outPts -value <1 \
    -metric opposite -outputType posEdgeA ]
db::foreach pts [db::getattr respts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
    highlight2 -isClosed
}
```

po::expandEdge

Description

Expands the edges of polygons from the input-collection into rectangles in the specified direction and amounts.

Syntax

```
po::expandEdge <input: dbCollection> -by <float> [-dir <enum>] [-extendBy
    <float>] [-factor <bool>] [-cornerFill <bool>]
```

Arguments

input

The input collection.

-by

Specifies the expanding value in user units.

-dir

Specifies the expanding direction. Valid values: all, inside, outside.
(Default: all)

-extendBy

Specifies the amount to extend/retract the length of edges from the input-collection prior to expanding them into rectangles. (Default: 0)

-factor

If false, the values in -by and -extendBy are in user units. (Default: false)

-cornerFill

If true, fills the gaps between rectangles formed by the operation. (Default: false)

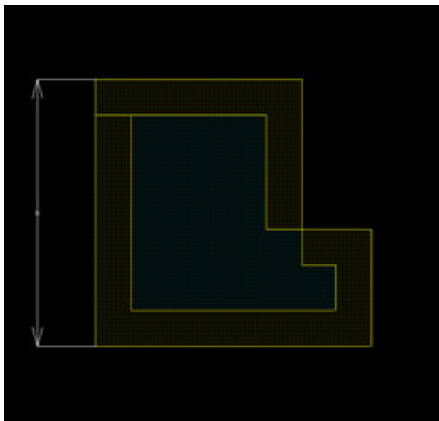
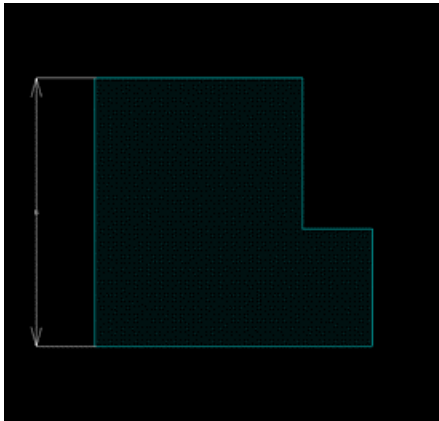
Returns

<dbCollection> : The result points container.

Examples

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]
```

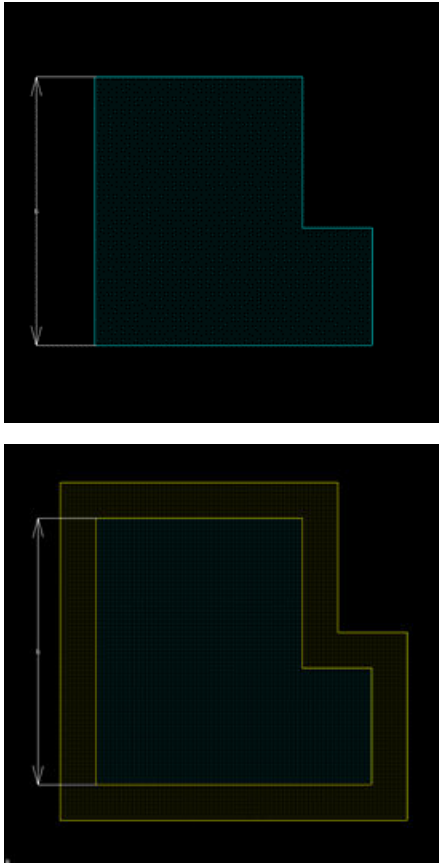
Example 1



In this example, this command expands source polygon edges in inside direction. The polygons with resulting points are created.

```
set resPts [po::expandEdge $srcPts -by 0.8 -dir inside \
    -cornerFill false ]
db::foreach points [db::getAttr resPts.pointArray] {
    le::createPolygon $points -design [ed] -lpp m2
}
```

Example 2



In this example, this command expands source polygon edges in outside direction, filling corners. The polygons with resulting points are created.

```
set resPts [po::expandEdge $srcPts -by 0.8 -dir outside \  
-cornerFill true ]  
db::foreach points [db::getAttr resPts.pointArray] {  
    le::createPolygon $points -design [ed] -lpp m2  
}
```

po::extent

Description

Generate a bounding box that represents the minimum bounding box containing all polygons or polylines of input-collection.

Syntax

```
po::extent <input: dbCollection> [-outputBox <bool>]
```

Arguments

`input`

Specify input-collection.

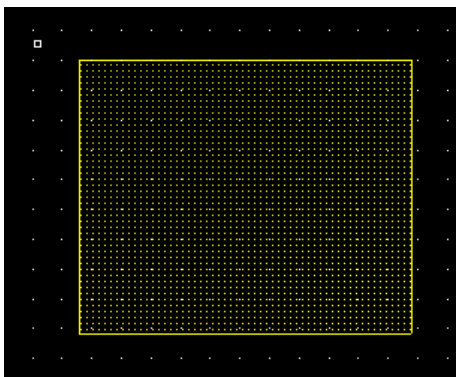
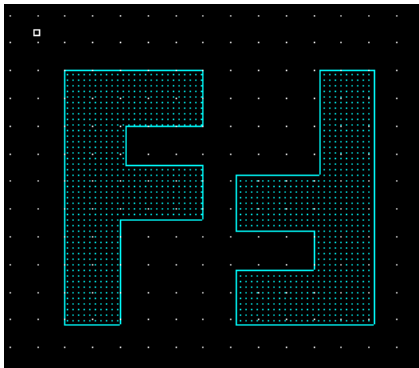
`-outputBox`

By default, output points container. If specify "-outputBox true", output `oa::Box`.
(Default: false)

Returns

`<dbCollection | oaBox>` : The result points container or `oa::Box`.

Examples



```
# po::extent
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
set shapes [db::getShapes -of $design]
#create collection with selected shapes
set col [po::createCollection $shapes]
# extent collection
set e [po::extent $col]
```

```
le::createPolygon [db::getattr e.pointArray] -lpp {m2 drawing} \  
-design $design
```

po::extents

Description

Creates the minimum bounding boxes for each merged polygon in an input-collection and merges the minimum bounding boxes to generate the operation results.

Syntax

```
po::extents <input: dbCollection>
```

Arguments

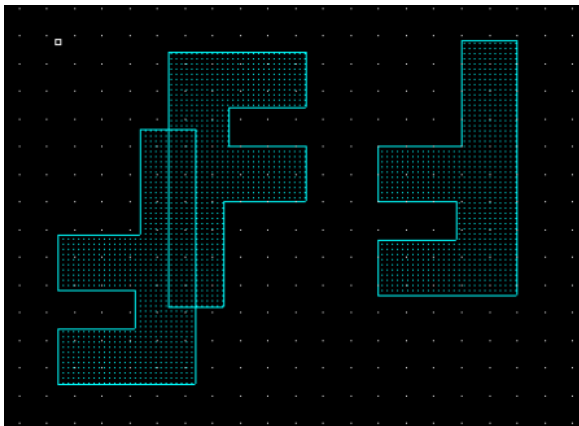
input

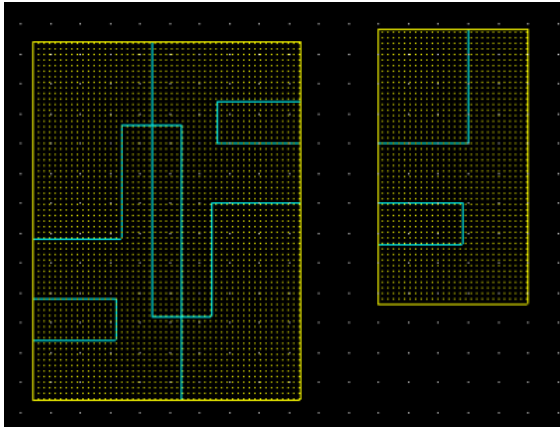
The input collection.

Returns

<dbCollection> : The result points container.

Examples





```
# po::extents
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
set shapes [db::getShapes -of $design]
#create collection with selected shapes
set col [po::createCollection $shapes]
# extent collection
set e [po::extents $col]
db::foreach i $e {
    le::createPolygon [db::getattr i.pointArray] -lpp {m2 drawing} \
        -design $design
}
}
```

po::external

Description

Measures the separations between the exterior-facing sides of edges. This command is principally used for spacing checks.

Syntax

```
po::external <input: dbCollection> [-with <dbCollection>] -value <string>
    [-metric <enum>] [-oppositeExt <float>] [-acute <enum>] [-parallel
    <enum>] [-perpendicular <enum>] [-obtuse <enum>] [-projectionFilter
    <enum>] [-projLength <string>] [-angled <string>] [-cornerFilter
    <enum>] [-abut <string>] [-singular <bool>] [-intersectingOnly <bool>]
    [-measureAll <bool>] [-outputType <enum>]
```

Arguments

input

The input collection.

`-with`

The with collection.

`-value`

Specifies the measuring constraint in user units. (For example: <0.05)

`-metric`

Measure edge pairs based on the specific metric. Valid values: Valid values: euclidean, opposite, square. (Default: euclidean)

`-oppositeExt`

Cooperates with `-metric opposite`. A non-negative number in user units specifies the amount to extend both ends of an edge. (Default: 0)

`-acute`

Measures edge pairs based on the acute condition. Valid values: Valid values: also, none, only. (Default: also)

`-parallel`

Measures edge pairs based on the parallel condition. Valid values: Valid values: also, none, only. (Default: also)

`-perpendicular`

Measures edge pairs based on the perpendicular condition. Valid values: Valid values: also, none, only. (Default: none)

`-obtuse`

Measures edge pairs based on the obtuse condition. Valid values: Valid values: also, none, only. (Default: none)

`-projectionFilter`

Specifies the condition of the mutual edge projection. Valid values: Valid values: all, notProj, proj. (Default: all)

`-projLength`

Cooperate with `"-projectionFilter proj"`. Specifies the projection length constraint and the constraint value is a non-negative number in user units. (For example: >0) (Default: >=0)

`-angled`

Specifies to measure the two edges only when the number of non-orthogonal edges in the pair meets the given constraint. (For example: ==0)

`-cornerFilter`

Measure edge pairs based on corner-to-corner or corner-to-edge orientation.
Valid values: all, corner, cornerToCorner, cornerToEdge, notCorner. (Default: all)

`-abut`

Specifies that separation between intersecting edges should also be checked under the specified constraint. (For example: <90)

`-singular`

Handle polygon singularities that are point-to-edge or point-to-point polygon intersections. (Default: false)

`-intersectingOnly`

Cooperate with "-abut" or "-singular". Specifies to measure only between intersecting edges and to ignore non-intersecting edges. (Default: false)

`-measureAll`

By default, consider polygon containment criteria. If Specifies "-measureAll true", ignore polygon containment criteria. (Default: false)

`-outputType`

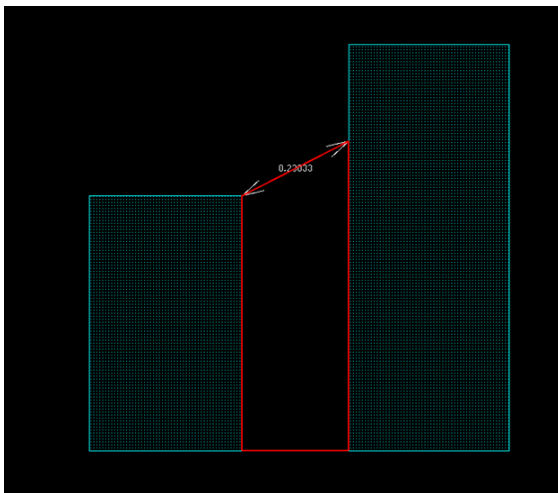
Specifies the output type. Valid values: negEdgeA, negEdgeB, posEdgeA, posEdgeB, region. (Default: region)

Returns

<dbCollection> : The result points container.

Examples

Example 1

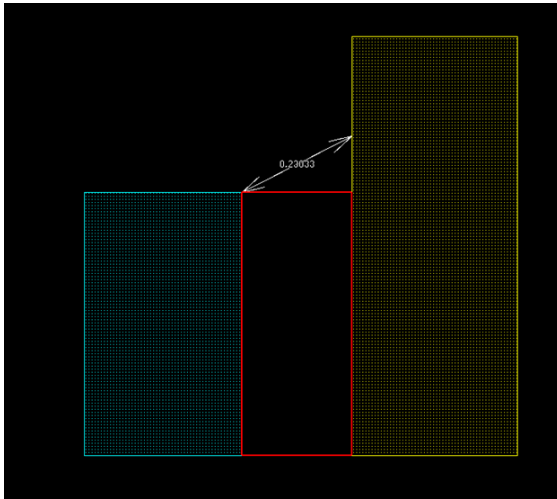


In this example, if the spacing is less than 0.23u, this command measures the spacing between exterior-facing sides of input points collection and returns Euclidean points collection. The returned area is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]

set resPts [po::external $srcPts -value <0.23]
db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
    highlight2 -isClosed
}
```

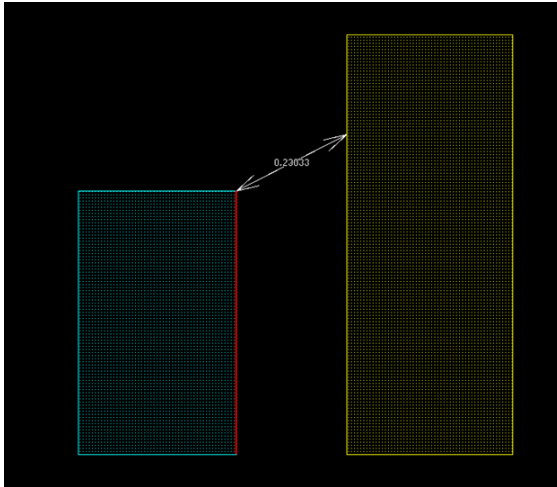
Example 2



In this example, if the spacing is less than 0.23u, this command measures the spacing between m1 layer and m2 layer shape sides and returns opposite points collection. The returned area is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]
set m2Shape [db::getShapes -lpp m2 -of $design]
set withPts [po::createCollection $m2Shape]
set resPts [po::external $srcPts -with $withPts -value <0.23 \
    -metric opposite]
db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
    highlight2 -isClosed
}
```

Example 3



In this example, if the spacing is less than 0.23u, this command measures the spacing between m1 layer and m2 layer shape sides and returns m1 layer shape edge points collection. The returned area is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]
set m2Shape [db::getShapes -lpp m2 -of $design]
set withPts [po::createCollection $m2Shape]
set resPts [po::external $srcPts -with $withPts -value <0.23 \
    -metric opposite -outputType posEdgeA ]
db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
        highlight2 -isClosed
}
```

po::getAngle

Description

Get the smaller angle degrees of the two edges.

Syntax

```
po::getAngle <input: [point]> -with [<point>] [-obtuse <bool>]
```

Arguments

input

The points represent one edge. This list only has two points.

-with

The points represent the other edge. This list only has two points.

`-obtuse`

If specify "`-obtuse true`", get the larger angle magnitude (>90 and <180) of the two edges. (Default: false)

Returns

`<float>` : The smaller angle degrees of the two edges.

Examples

Example 1

```
# This command returns the smaller angle magnitude ( $\geq 0$  and  $\leq 90$ ) between
# the two edges
po::getAngle {{0 0} {1 0}} -with {{0 0} {-1 1}}
→ 45.0
```

Example 2

```
# This command returns the larger angle magnitude ( $\geq 90$  and  $\leq 180$ )
# between
# the two edges
po::getAngle {{0 0} {1 0}} -with {{0 0} {-1 1}} -obtuse true
→ 135.0
```

po::getArea

Description

Get the total merged area of the input polygons.

Syntax

```
po::getArea <input: dbCollection | poPointSet>
```

Arguments

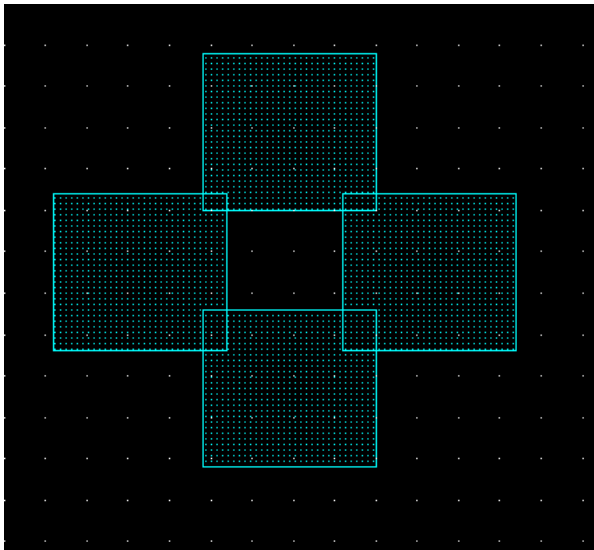
`input`

Specify the input points container.

Returns

`<float>` : The total merged area of the input polygons.

Examples



```
# po::getArea
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
set shapes [db::getShapes -of $design]
#create collection with selected shapes
set col [po::createCollection $shapes]
# get area
set area [po::getArea $col]
```

po::getContours

Description

Merge the input polygons and get the contours.

Syntax

```
po::getContours <input: dbCollection | poPointSet>
```

Arguments

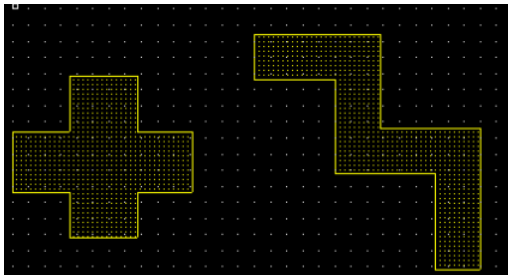
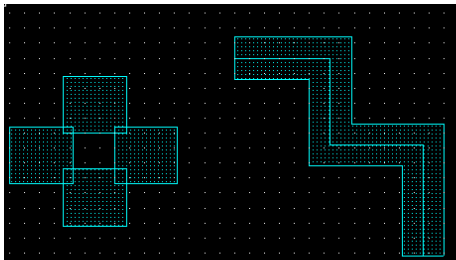
input

Specify the operand1 points container.

Returns

<poPointSet> : The result points container.

Examples



```
# po::getContours
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
set shapes [db::getShapes -of $design]
#create collection with selected shapes
set col [po::createCollection $shapes]
# create contour
set cnt [po::getContours $col]

# create shape with m2 drawing lpp
le::createPolygon [db::getattr cnt.pointArray] -lpp {m2 drawing} \
    -design $design
```

po::getDistance

Description

Get the Euclidean distance of the two things.

Syntax

```
po::getDistance <points: [point]> -to [<point>]
```

Arguments

points

The list of points that form getting distance points.

-to

The other points to measure the distance.

Returns

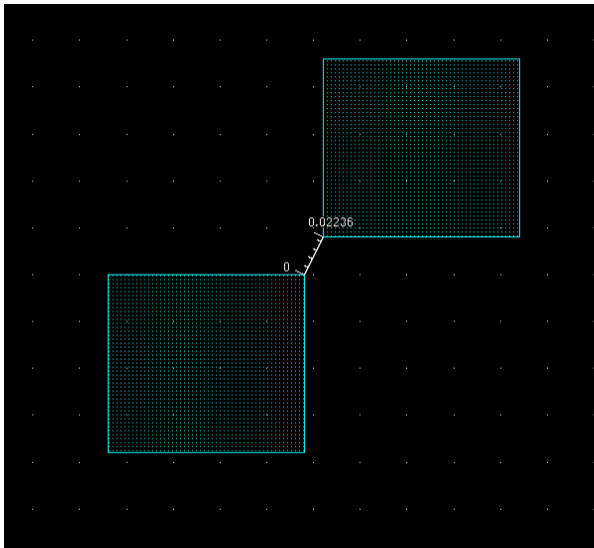
<float> : The Euclidean distance between the two input points.

Examples

Example 1

```
# Get the Euclidean distance of the two points
po::getDistance {{0 0}} -to {{0 3}}
# Get the Euclidean distance of the two virtual rectangles
po::getDistance {{0 0} {1 1}} -to {{0 0} {3 3}}
```

Example 2



```
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
set shapes [db::getShapes -of $design]
set p1 [db::getattr bbox -of [db::getNext $shapes]]
set p2 [db::getattr bbox -of [db::getNext $shapes]]
# Get the Euclidean distance of the two rectangles
po::getDistance $p1 -to $p2
```

po::getHoles

Description

Merge the input polygons and get the holes.

Syntax

```
po::getHoles <input: dbCollection | poPointSet>
```

Arguments

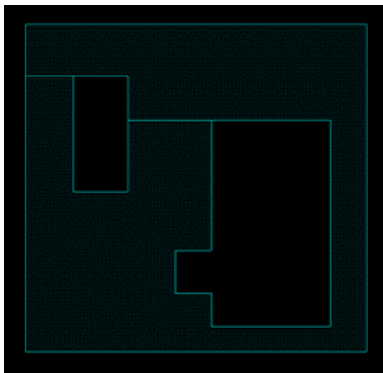
input

Specify the operand1 points container.

Returns

<poPointSet> : The result points container.

Examples



```
set design [ed]
# Get polygon on m1 layer that has holes
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]
# Returns hole points
set resPts [po::getHoles $srcPts]
# Create polygon with result points on m2 layer
db::foreach points [db::getattr resPts.pointArray] {
    le::createPolygon $points -design $design -lpp m2
}
```

po::getIntersection

Description

Get the intersection of the two lines or two segments.

Syntax

```
po::getIntersection <points: [point]> -with [<point>] [-onSegment <bool>]
```

Arguments

`points`

The list of points that form line points.

`-with`

The other line or segment points.

`-onSegment`

Calculate the intersection of the two segments. (Default: true)

Returns

`<point>` : The intersection of the two lines or two segments.

Examples

Example 1

```
# The command returns an empty list, since the specified segments do not  
# intersect  
po::getIntersection {{0 0} {1 1}} -with {{0 1} {0.3 0.7}}
```

Example 2

```
# The command returns intersection point of specified two lines  
po::getIntersection {{0 0} {1 1}} -with {{0 1} {0.3 0.7}} \  
    -onSegment false  
=> 0.5 0.5
```

po::getMaxBox

Description

Get the maximum area enclosed box.

Syntax

```
po::getMaxBox <input: dbCollection> [-anchorPoint <point>]
```

Arguments

`input`

Specify input-collection.

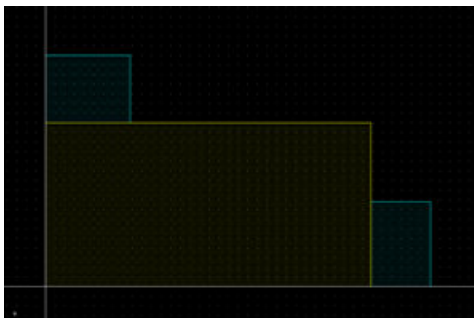
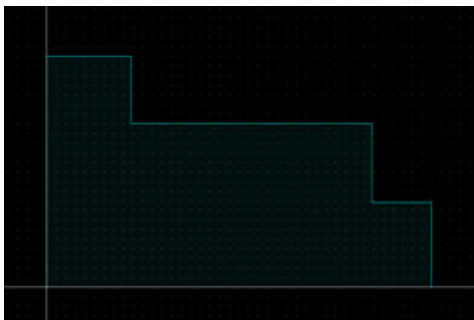
`-anchorPoint`

Specify the anchor point in user units.

Returns

`<dbCollection>` : The result points container.

Examples



In this example, this command returns the maximum area enclosed box. The polygons with those points on m2 layer are created.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
# Creates source collection of points for shapes on m1 layer
set srcPts [po::createCollection $m1Shape]
po::getMaxBox $srcPts
# Creates polygons with resulting points on m2 layer
le::createPolygon [db::getAttr poBox.pointArray] -design $design -lpp m2
```

po::getPerimeter

Description

Get the total merged perimeter of the input polygons.

Syntax

```
po::getPerimeter <input: dbCollection | poPointSet>
```

Arguments

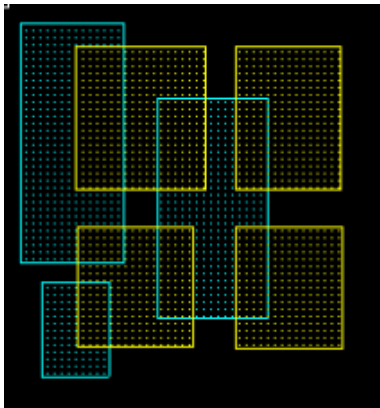
input

Specify the input points container.

Returns

<float> : The total merged perimeter of the input polygons.

Examples



```
# po::getPerimeter
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# Get shape on m2 drawing lpp
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
set cm2 [po::createCollection $m2]
#get shapes from m1 which interacts with shapes on m2 layer
set per [po::getPerimeter $cm2 ]
puts $per
> 85.57
```

po::getPoints

Description

Get the points(in user units) of the points set iteratively.

Syntax

```
po::getPoints <input: poPointSet> [-reset <bool>]
```

Arguments

input

Specify the points set.

-reset

Reset the iterator of points set. (Default: false)

Returns

[<point>] : The points (in user units) in the points set.

po::getSpecifiedBoxes

Description

Get the specified boxes which match the criteria based on a given anchor point.

Syntax

```
po::getSpecifiedBoxes <input: dbCollection> -minWidth <float> -minHeight  
<float> -anchorPt <point> [-maxWidth <float>] [-maxHeight <float>]  
[-opType <enum>]
```

Arguments

input

Specify input-collection.

-minWidth

Specify the minimum width of the specified box.

-minHeight

Specify the minimum height of the specified box.

-anchorPt

Specify the anchor point in user units.

`-maxWidth`

Specify the maximum width of the specified box. Only applied in `opType = horizontal` or `vertical`. (Default: 0)

`-maxHeight`

Specify the maximum height of the specified box. Only applied in `opType = horizontal` or `vertical`. (Default: 0)

`-opType`

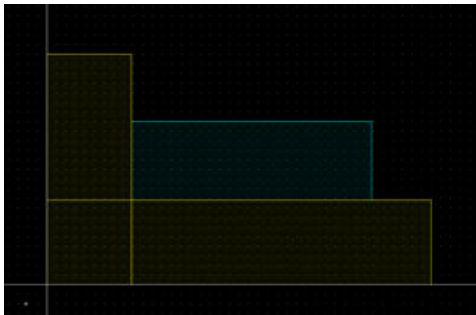
Specify the operation type. Valid values: `all`, `convexCorner`, `horizontal`, `vertical`. (Default: `all`)

Returns

`<dbCollection>` : The result points container.

Examples

Example 1

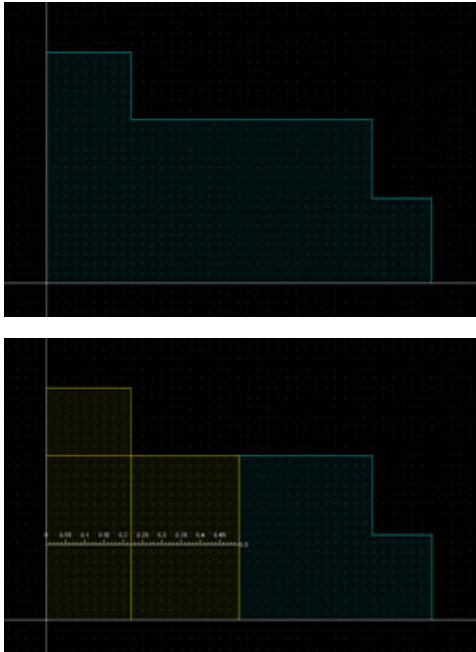


In this example, this command returns boxes with edges from `{0 0}` anchor point.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
# Creates source collection of points for shapes on m1 layer
set srcPts [po::createCollection $m1Shape]
set boxes [po::getSpecifiedBoxes $src -anchorPt {0 0} -minWidth 0 \
    -minHeight 0]
```

```
# Creates polygons with resulting points on m2 layer
db::foreach points [db::getattr boxes.pointArray] {
    le::createPolygon $points -design $design -lpp m2
}
```

Example 2



In this example, this command returns vertical boxes with edges from {0 0} anchor point for those width does not exceeds 0.5u.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
# Creates source collection of points for shapes on m1 layer
set srcPts [po::createCollection $m1Shape]
set boxes [po::getSpecifiedBoxes $src -anchorPt {0 0} -minWidth 0 \
    -minHeight 0 -maxWidth 0.5 -opType vertical]
db::foreach points [db::getattr boxes.pointArray] {
    le::createPolygon $points -design $design -lpp m2
}
```

po::inside

Description

Selects all polygons from the input-collection that share their entire areas with polygons of with-collection.

Syntax

```
po::inside <input: dbCollection> -with <dbCollection> [-not <bool>]
```

Arguments

input

Specify input-collection.

-with

Specify with-collection.

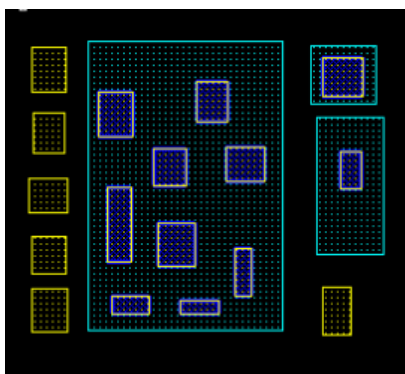
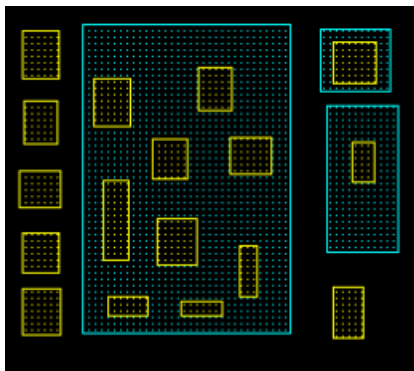
-not

"-not true" is the complement of the inside operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples



```
# po::inside
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
```

```
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shapes on m2 drawing
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
# select shapes on m2 layer which are inside in a shape on m1 layer
set in [po::inside $cm2 -with $cm1]
# select shapes in "in" collection
db::foreach p [db::getAttr in.pointArray] {
    #create shapes on poly layer
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

po::insideEdge

Description

Selects all edges or edge segments from input-collection that lie completely inside polygons of with-collection.

Syntax

```
po::insideEdge <input: dbCollection> -with <dbCollection> [-not <bool>]
```

Arguments

input

The input collection.

-with

The with collection.

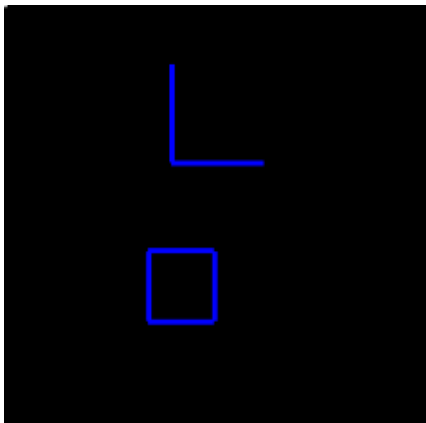
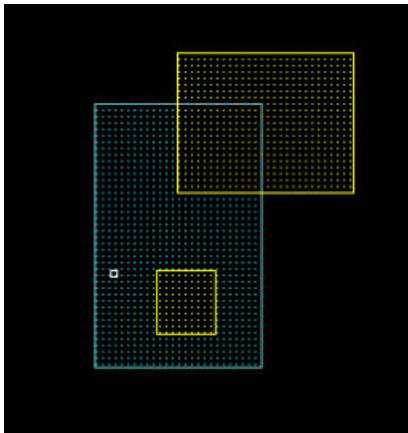
-not

"-not true" is the complement of the insideEdge operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples



```
# po::insideEdge
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shapes on m2 drawing
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
# select edges of the shapes on m2 layer which are inside in a shape on
# m1 layer
set in [po::insideEdge $cm2 -with $cm1]
# select points in "in" collection
db::foreach p [db::getattr in.pointArray] {
    #create path shapes on poly layer with points from "in" collection
    le::createPath $p -width 0.1 -lpp {poly drawing} -design $design
}
```

po::interact

Description

Selects all polygons from input-collection based on the type and number of intersections they have with polygons of with-collection.

Syntax

```
po::interact <input: dbCollection> -with <dbCollection> [-value <string>]  
[-hasEvenOdd <enum>] [-hasSingular <enum>] [-not <bool>]
```

Arguments

input

Specify input-collection.

-with

Specify with-collection.

-value

Specify the selection criterion. (For example: >=3)

-hasEvenOdd

Cooperate with "-value" to specify the extra selection criterion. Valid values: even, none, odd. (Default: none)

-hasSingular

Indicate how to handle point-to-point and point-to-edge interactions. Valid values: also, none, only. (Default: none)

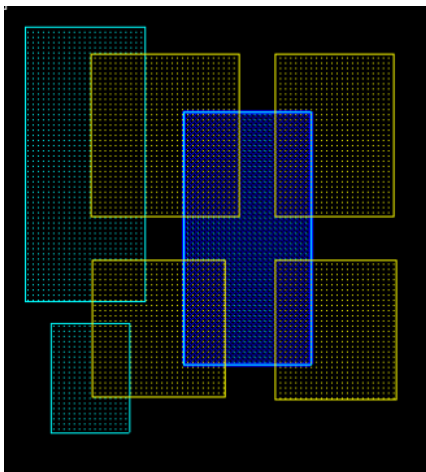
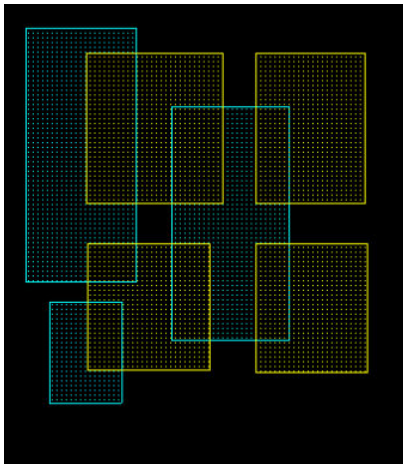
-not

"-not true" is the complement of the interact operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples



```
# po::interact
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
#get shapes on m1 drawing lpp
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
# Get shape on m2 drawing lpp
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
set cm2 [po::createCollection $m2]
#get shapes from m1 which interacts with 4 shapes on m2 layer
set p [po::interact $cm1 -with $cm2 -value 4]
db::foreach point [db::getattr p.pointArray] {
    #create shapes on poly layer
    le::createPolygon $point -lpp {poly drawing} -design $design
}
```

po::internal

Description

Measures the separations between the interior-facing sides of edges. This command is principally used for width and overlap checks.

Syntax

```
po::internal <input: dbCollection> [-with <dbCollection>] -value <string>
[-metric <enum>] [-oppositeExt <float>] [-acute <enum>] [-parallel
<enum>] [-perpendicular <enum>] [-obtuse <enum>] [-projectionFilter
<enum>] [-projLength <string>] [-angled <string>] [-cornerFilter
<enum>] [-abut <string>] [-singular <bool>] [-intersectingOnly <bool>]
[-measureAll <bool>] [-outputType <enum>]
```

Arguments

`input`

The input collection.

`-with`

The with collection.

`-value`

Specifies the measuring constraint in user units. (For example: <0.05>)

`-metric`

Measure edge pairs based on the specific metric. Valid values: euclidean, opposite, square. (Default: euclidean)

`-oppositeExt`

Cooperate with `-metric opposite`. A non-negative number in user units specifies the amount to extend both ends of an edge. (Default: 0)

`-acute`

Measure edge pairs based on the acute condition. Valid values: also, none, only. (Default: also)

`-parallel`

Measure edge pairs based on the parallel condition. Valid values: also, none, only. (Default: also)

`-perpendicular`

Measure edge pairs based on the perpendicular condition. Valid values: also, none, only. (Default: none)

`-obtuse`

Measure edge pairs based on the obtuse condition. Valid values: also, none, only. (Default: none)

`-projectionFilter`

Specifies the condition of the mutual edge projection. Valid values: all, notProj, proj. (Default: all)

`-projLength`

Cooperate with "-projectionFilter proj". Specifies the projection length constraint and the constraint value is a non-negative number in user units. (For example: >0) (Default: >=0)

`-angled`

Specifies to measure the two edges only when the number of non-orthogonal edges in the pair meets the given constraint. (For example: ==0)

`-cornerFilter`

Measure edge pairs based on corner-to-corner or corner-to-edge orientation. Valid values are all, corner, cornerToCorner, cornerToEdge, notCorner. (Default: all)

`-abut`

Specifies that separation between intersecting edges should also be checked under the specified constraint. (For example: <90)

`-singular`

Handle polygon singularities that are point-to-edge or point-to-point polygon intersections. (Default: false)

`-intersectingOnly`

Cooperate with "-abut" or "-singular". Specifies to measure only between intersecting edges and to ignore non-intersecting edges. (Default: false)

`-measureAll`

By default, consider polygon containment criteria. If Specifies "-measureAll true", ignore polygon containment criteria. (Default: false)

`-outputType`

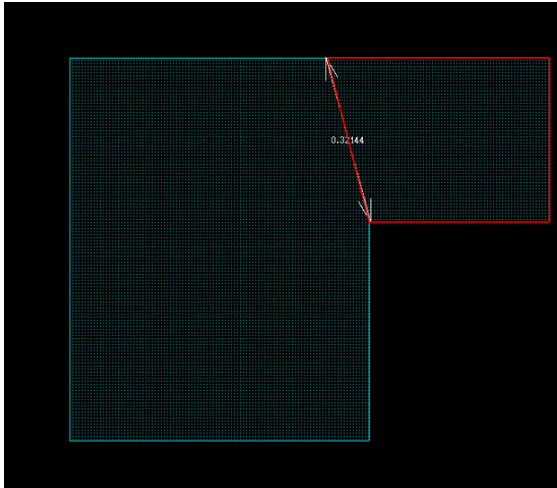
Specifies the output type. Valid values: negEdgeA, negEdgeB, posEdgeA, posEdgeB, region. (Default: region)

Returns

<dbCollection> : The result points container.

Examples

Example 1

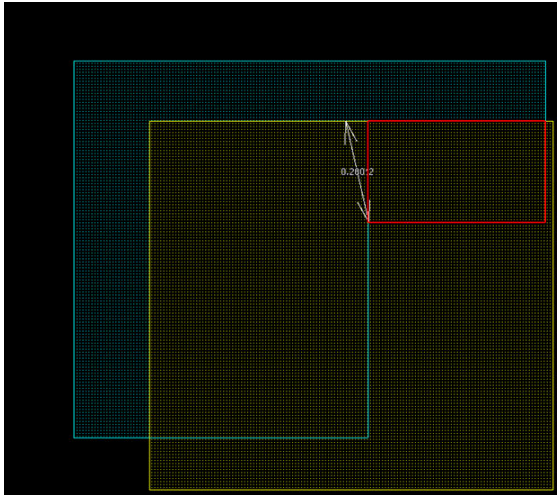


In this example, if the spacing is less than 0.32u, this command measures the spacing between interior-facing sides of input points collection and returns Euclidean points collection. The returned area is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]

set resPts [po::internal $srcPts -value <0.32]
db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po \
        -packet highlight2 -isClosed
}
```

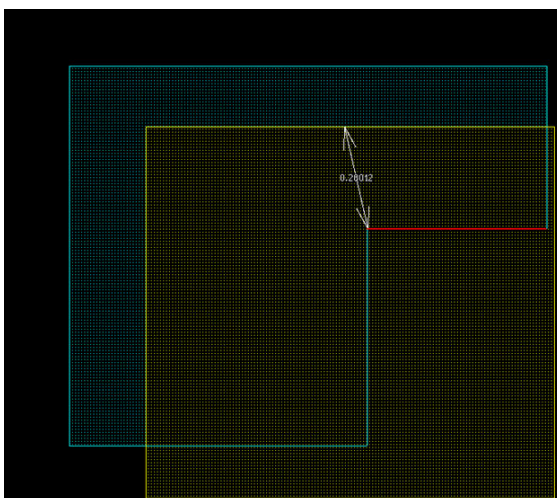
Example 2



In this example, if the spacing is less than 0.2u, this command measures the spacing between m1 layer shape interior side and m2 layer shape side and returns opposite points collection. The returned area is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]
set m2Shape [db::getShapes -lpp m2 -of $design]
set withPts [po::createCollection $m2Shape]
set resPts [po::internal $srcPts -with $withPts -value <0.2 \
    -metric opposite]
db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po \
        -packet highlight2 -isClosed
}
```

Example 3



In this example, if the spacing is less than 0.2u, this command measures the spacing between m1 layer shape interior side and m2 layer shape side and returns m1 layer side points collection. The returned area is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]
set m2Shape [db::getShapes -lpp m2 -of $design]
set withPts [po::createCollection $m2Shape]
set resPts [po::internal $srcPts -with $withPts -value <0.23 \
    -metric opposite -outputType posEdgeA ]
db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po \
    -packet highlight2 -isClosed
}
```

po::isClockwise

Description

Check the input points satisfying clockwise direction or not .

Syntax

```
po::isClockwise <points: [point]>
```

Arguments

points

The list of points that form checking clockwise points.

Returns

<bool> : Honor clockwise direction or not.

Examples

Example 1

```
# counter-clockwise ordered points
po::isClockwise {{0 0} {1 0} {1 1} {0 1}}
→ 0
```

Example 2

```
# clockwise ordered points
po::isClockwise {{0 0} {0 1} {1 1} {1 0}}
→ 1
```

po::isHorizontal

Description

Check the input points satisfying horizontal direction or not .

Syntax

```
po::isHorizontal <points: [point]> [-dbuPerUU <int>]
```

Arguments

points

The list of points that form checking horizontal points.

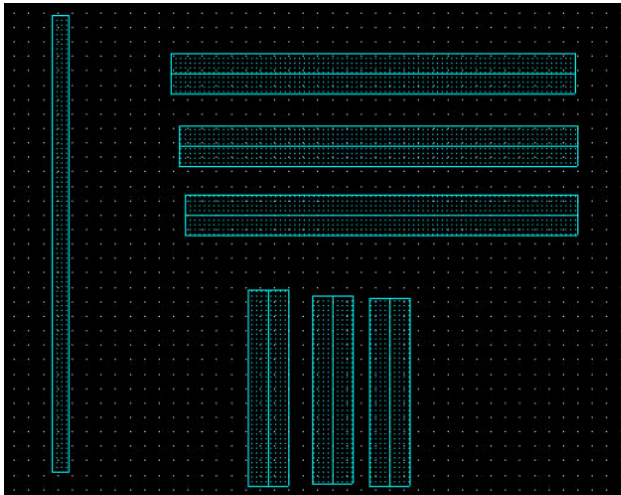
-dbuPerUU

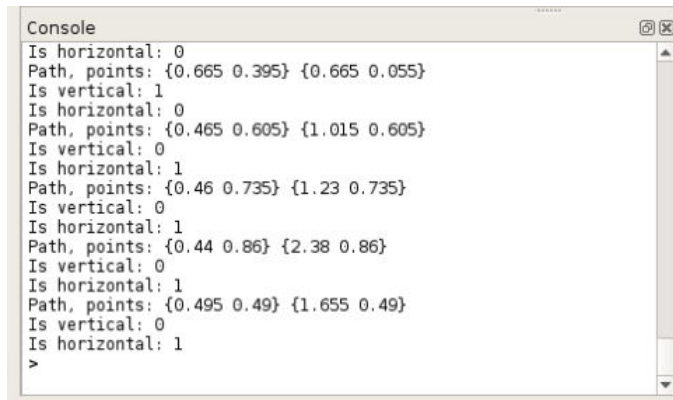
Specify dbuPerUU for points input. (Default: 1000)

Returns

<bool> : Honor horizontal direction or not.

Examples





```

Console
Is horizontal: 0
Path, points: {0.665 0.395} {0.665 0.055}
Is vertical: 1
Is horizontal: 0
Path, points: {0.465 0.605} {1.015 0.605}
Is vertical: 0
Is horizontal: 1
Path, points: {0.46 0.735} {1.23 0.735}
Is vertical: 0
Is horizontal: 1
Path, points: {0.44 0.86} {2.38 0.86}
Is vertical: 0
Is horizontal: 1
Path, points: {0.495 0.49} {1.655 0.49}
Is vertical: 0
Is horizontal: 1
>

```

```

# po::isHorizontal
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getAttr ctx.editDesign]
# Get Path shapes on m1 drawing lpp
set m1 [db::getShapes -filter {%type == "Path"} -lpp {m1 drawing} \
    -of $design]

db::foreach shape $m1 {
    set cml [po::createCollection $shape]
    # get shapes from m1 which interacts with shapes on m2 later
    set points [db::getAttr shape.points ]
    puts "[db::getAttr shape.type], points: $points"
    set orientation [po::isVertical $points]
    puts "Is vertical: $orientation"
    set orientation [po::isHorizontal $points]
    puts "Is horizontal: $orientation"
}

```

po::isVertical

Description

Check the input points satisfying vertical direction or not .

Syntax

```
po::isVertical <points: [point]> [-dbuPerUU <int>]
```

Arguments

points

The list of points that form checking vertical points.

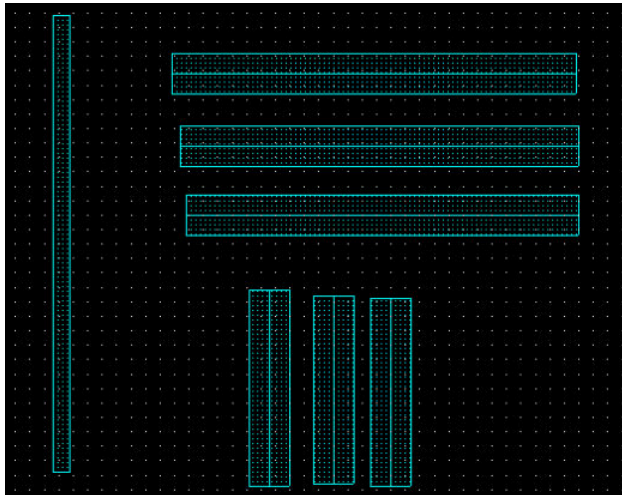
-dbuPerUU

Specify dbuPerUU for points input. (Default: 1000)

Returns

<bool> : Honor vertical direction or not.

Examples



```

Console
Is horizontal: 0
Path, points: {0.665 0.395} {0.665 0.055}
Is vertical: 1
Is horizontal: 0
Path, points: {0.465 0.605} {1.015 0.605}
Is vertical: 0
Is horizontal: 1
Path, points: {0.46 0.735} {1.23 0.735}
Is vertical: 0
Is horizontal: 1
Path, points: {0.44 0.86} {2.38 0.86}
Is vertical: 0
Is horizontal: 1
Path, points: {0.495 0.49} {1.655 0.49}
Is vertical: 0
Is horizontal: 1
>

```

```

# po::isVertical
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# Get Path shapes on m1 drawing lpp
set m1 [db::getShapes -filter {%type == "Path"} -lpp {m1 drawing} \
-of $design]

db::foreach shape $m1 {
    set cm1 [po::createCollection $shapes]
    # get shapes from m1 which interacts with shapes on m2 later
    set points [db::getattr shape.points ]
    puts "[db::getattr shape.type], points: $points"
    set orientation [po::isVertical $points]
    puts "Is vertical: $orientation"
}

```



```
    set orientation [po::isHorizontal $points]
    puts "Is horizontal: $orientation"
}
```

po::length

Description

Selects all edges from input-collection having length that conforms to the constraint specified by "-value".

Syntax

```
po::length <input: dbCollection> -value <string> [-not <bool>]
```

Arguments

input

The input collection.

-value

Specifies the length constraint in user units. (For example ≤ 0.2)

-not

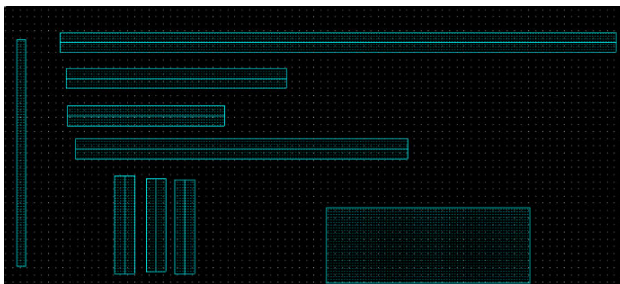
"-not true" is the complement of the length operation. (Default: false)

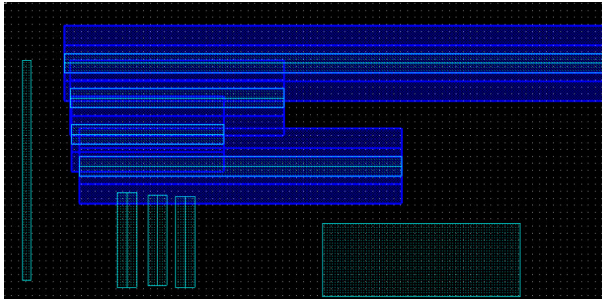
Returns

<dbCollection> : The result points container.

Examples

Example 1

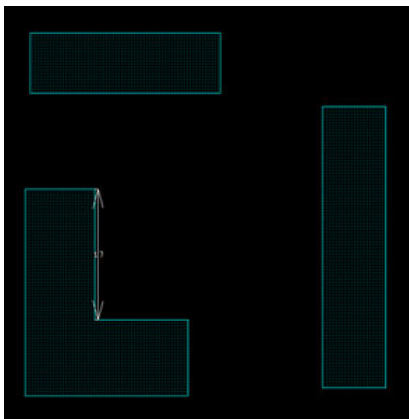


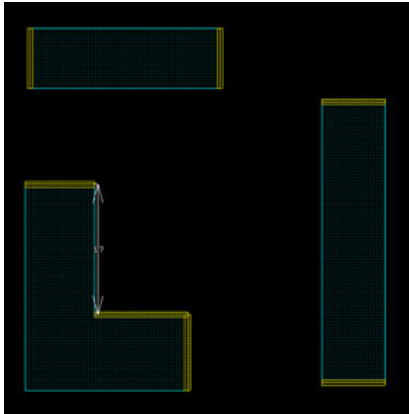


```
# po::length
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# Get Path shapes on m1 drawing lpp
set m1 [db::getShapes -filter {%type == "Path"} -lpp {m1 drawing} \
    -of $design]
set cm1 [po::createCollection $m1]
set l [po::length $cm1 -value >0.5]

db::foreach p [db::getattr l.pointArray] {
    # create shapes on poly layer
    le::createPathSeg $p -width 0.2 -lpp {poly drawing} -design $design
}
```

Example 2





In this example, this command returns points of edges from source collection for those length is less than 1.7u. The paths on resulting points are created.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
# Creates source collection of points for shapes on m1 layer
set srcPts [po::createCollection $m1Shape]
set edges [po::length $srcPts -value <1.7]
# Create path objects on resulting points
db::foreach edge $edges {
    le::createPath [db::getattr edge.pointArray] \
        -design $design -lpp m2 -width 0.07
}
```

po::makeClockwise

Description

Guarantee the ordering of input points.

Syntax

```
po::makeClockwise <points: [point]> [-reverse <bool>]
```

Arguments

points

The list of points that form making clockwise points.

-reverse

Guarantee the ordering of input points satisfying counter-clockwise. (Default: false)

Returns

[<point>] : The points keep the specific ordering.

Examples

Example 1

```
# Makes points order clockwise
po::makeClockwise {{0.0 0.0} {1.0 0.0} {1.0 1.0} {0.0 1.0}}
→ {0.0 0.0} {0.0 1.0} {1.0 1.0} {1.0 0.0}
```

Example 2

```
# Makes points order counter-clockwise
po::makeClockwise {{0.0 0.0} {0.0 1.0} {1.0 1.0} {1.0 0.0}} -reverse 1
→ {0.0 0.0} {1.0 0.0} {1.0 1.0} {0.0 1.0}
```

po::manhattanContour

Description

For orthogonal polygons, form a connected component and get its contour.

Syntax

```
po::manhattanContour <input: dbCollection> [-notchGap <float>] [-jogLen  
  <float>]
```

Arguments

input

Specify input-collection.

-notchGap

Specify the notchGap length in user units.

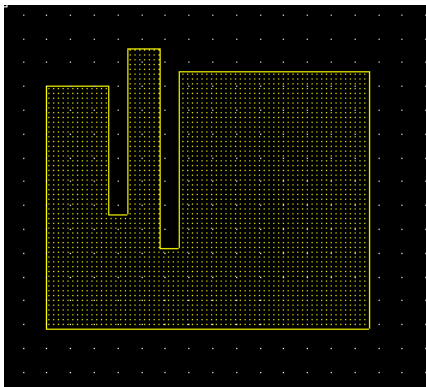
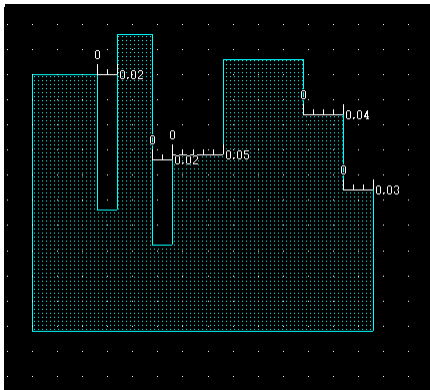
-jogLen

Specify the jog length in user units.

Returns

<dbCollection> : The result points container.

Examples



```
# po::manhattanContour
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
set shapes [db::getShapes -of $design]
#create collection with selected shapes
set col [po::createCollection $shapes]
# create contour
set cnt [po::manhattanContour $col -jogLen 0.2 ]
# create shape with m2 drawing lpp
le::createPolygon [db::getattr cnt.pointArray] -lpp {m2 drawing} \
    -design $design
```

po::not

Description

Run the Boolean NOT operation.

Syntax

```
po::not <input: dbCollection | poPointSet> -of <dbCollection |  
poPointSet>
```

Arguments

input

Specify the operand2 points container.

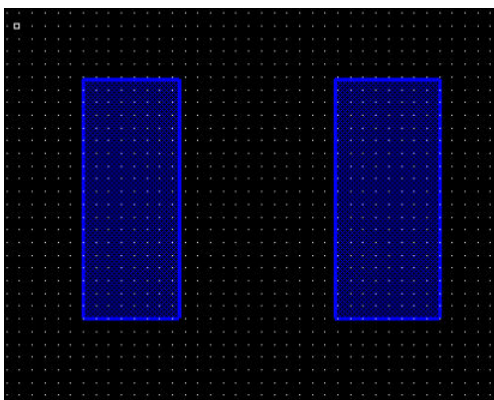
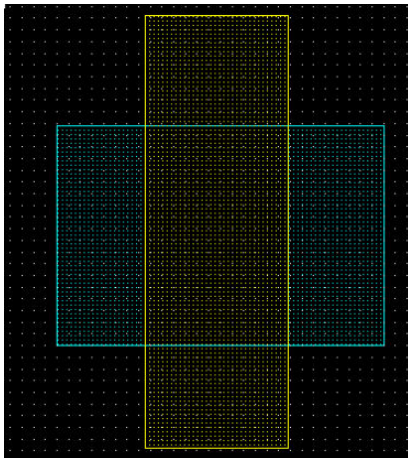
-of

Specify the operand1 points container.

Returns

<dbCollection | poPointSet> : The result points container.

Examples



```
# po::not  
#  
set ctx [de::getContexts -window [gi::getActiveWindow ]]
```

```
set design [db::getattr ctx.editDesign]
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shapes on m2 drawing
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
# not all polygons from m2 layer with shapes on m1 layer
set or [po::not $cm2 -from $cm1]
# select shapes in "or" collection
db::foreach p [db::getattr or.pointArray] {
    #create shapes on poly layer
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

po::offGrid

Description

Output off-grid polygons or edges based upon a user-specified grid in user units and parameter settings.

Syntax

```
po::offGrid <input: dbCollection> -resolution <point> [-offset <point>]
[-edge <bool>]
```

Arguments

input

The input collection.

-resolution

Specifies the grid resolution in user units. x-coordinate is applied in the x-direction and y-coordinate is applied in the y-direction.

-offset

Specifies an offset displacement to apply to the objects in input-collection before checking them against the grid resolution. (Default: {0 0})

-edge

If its value is true, output the off-grid edges of objects in input-collection. (Default: false)

Returns

<dbCollection> : The result points container.

Examples

```
# Creating points collection
set pts [po::createCollection {{0.0 0.0} {0.0 0.1} {0.1 0.1} {0.1 0.0}}]
# Adding off-grid points
po::append {{0.002 0.002} {0.002 0.201} {0.201 0.201} {0.201 0.002}} \
    -to $pts
# The po::offGrid command selects off-grid points from collection
set offGrid [po::offGrid $pts -resolution {0.5 0.5}]
db::getAttr offGrid.pointArray
→ {0.201 0.201} {0.201 0.002} {0.1 0.002} {0.1 0.0} {0.0 0.0} {0.0 0.1}
   {0.002 0.1} {0.002 0.201}
```

po::or

Description

Run the Boolean OR operation.

Syntax

```
po::or <input: dbCollection | poPointSet> [-with <dbCollection |
    poPointSet>]
```

Arguments

input

Specify the operand1 points container.

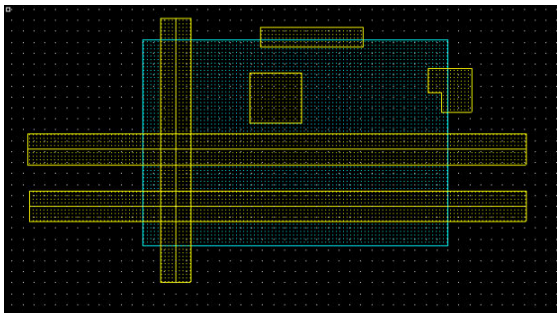
-with

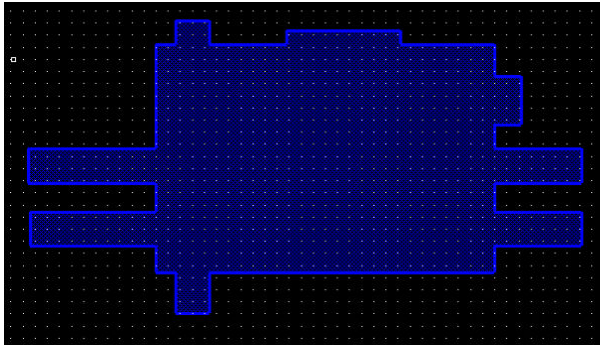
Specify the operand2 points container.

Returns

<poPointSet> : The result points container.

Examples





```
# po::or
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shapes on m2 drawing
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
# and all polygons from m2 layer with shapes on m2 layer
set or [po::or $cm2 -with $cm1]
# select shapes in "or" collection
db::foreach p [db::getattr or.pointArray] {
    #create shapes on poly layer
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

po::outside

Description

Selects all polygons from input-collection with areas that lie completely outside all polygons of with-collection.

Syntax

```
po::outside <input: dbCollection> -with <dbCollection> [-not <bool>]
```

Arguments

input

Specify input-collection.

-with

Specify with-collection.

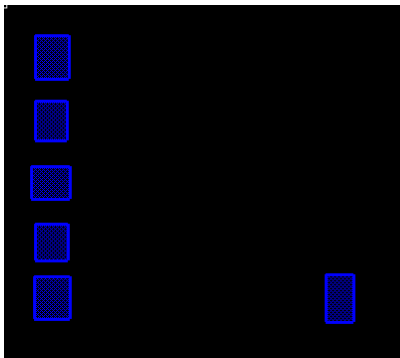
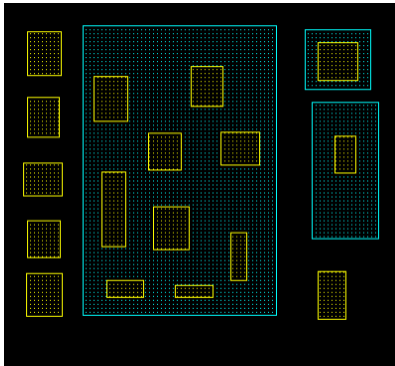
-not

"-not true" is the complement of the outside operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples



```
# po::outside
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shapes on m2 drawing
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
# select shapes on m2 layer which are outside of shapes m1 layer
set os [po::outside $cm2 -with $cm1]
# select shapes in withh points in "os" collection
db::foreach p [db::getattr os.pointArray] {
    #create shapes on poly layer
```

```
le::createPolygon $p -lpp {poly drawing} -design $design  
}
```

po::outsideEdge

Description

Select all edges or edge segments from input-collection that lie completely outside polygons of with-collection.

Syntax

```
po::outsideEdge <input: dbCollection> -with <dbCollection> [-not <bool>]
```

Arguments

input

The input collection.

-with

The with collection.

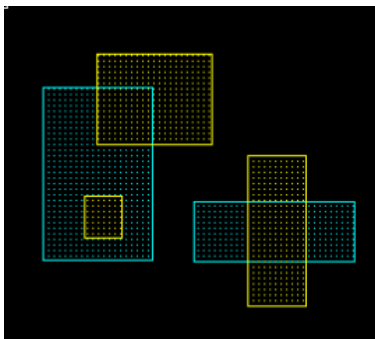
-not

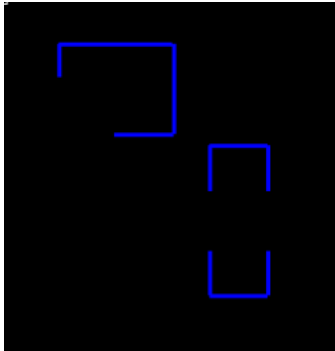
"-not true" is the complement of the outsideEdge operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples





```
# po::outsideEdge
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getAttr ctx.editDesign]
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shapes on m2 drawing
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
# select edges of the shapes on m2 layer which are outside of shapes on
# m1 layer
set in [po::outsideEdge $cm2 -with $cm1]
# select points in "in" collection
db::foreach p [db::getAttr in.pointArray] {
    #create path shapes on poly layer with points from "in" collection
    le::createPath $p -width 0.1 -lpp {poly drawing} -design $design
}
```

po::pathLength

Description

For each polyline in input-collection, select a maximum-length contiguous path of distinct edges satisfying the constraint specified by "-value".

Syntax

```
po::pathLength <input: dbCollection> -value <string>
```

Arguments

input

Specify input-collection.

-value

Specify the constraint representing the total length of polylines in user units. (For example: <0.5)

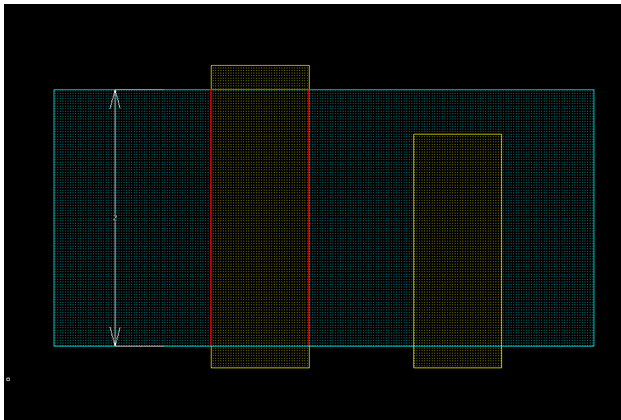
Returns

<dbCollection> : The result points container.

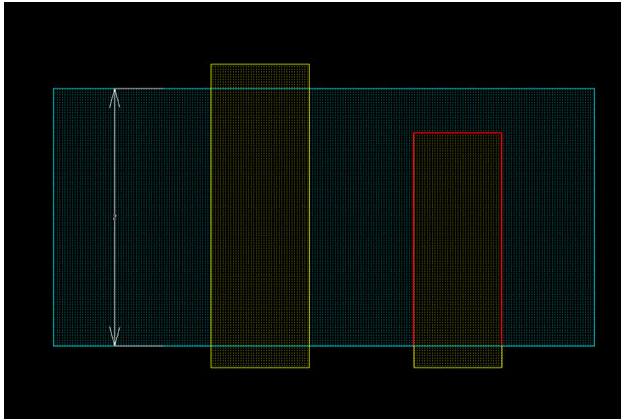
Examples

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]
# Gets m2 layer shape edges inside of the m1 layer rectangle
set insideEdges [po::insideEdge $srcPts -with $withPts]

# Returns edges those length is less or equal to 2.0u
set edges [po::pathLength $insideEdges -value <=2.0]
# Highlights found edges
db::foreach pts [db::getattr edges.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po \
        -packet highlight2 -isClosed
}
```



```
# Returns edges those length is greater than 2.0u
set edges [po::pathLength $insideEdges -value >2.0]
# Highlights found edges
db::foreach pts [db::getattr edges.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po \
        -packet highlight2 -isClosed
}
```



po::perimeter

Description

Select all merged-polygons in input collection with perimeters conforming to the constraint specified by "-value".

Syntax

```
po::perimeter <input: dbCollection> -value <string> [-not <bool>]
```

Arguments

input

Specify input-collection.

-value

Specify the perimeter constraint for a merged-polygon. (For example: ≥ 2)

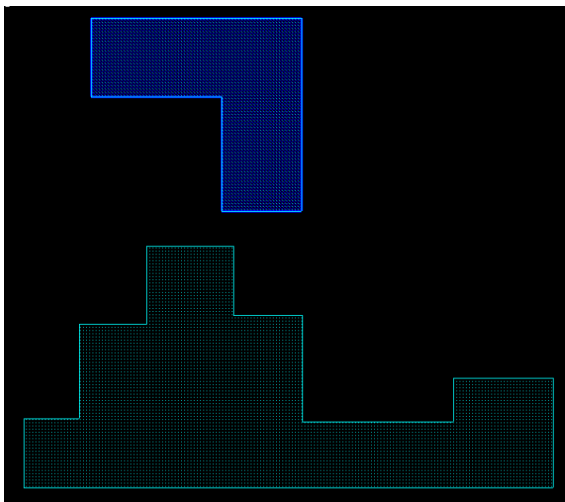
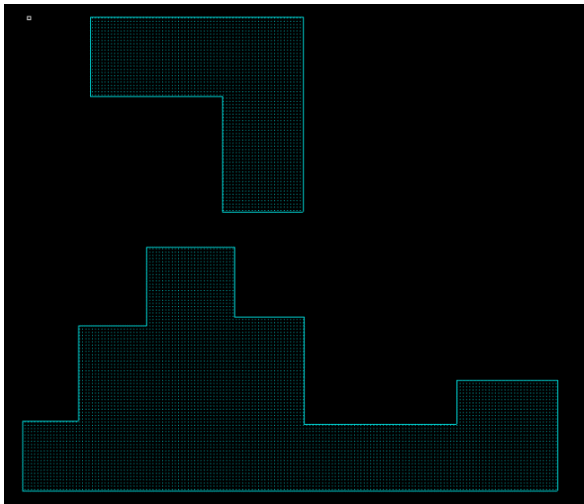
-not

"-not true" is the complement of the perimeter operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples



```
# po::perimeter
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# Get Path shapes on m1 drawing lpp
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
set cm1 [po::createCollection $m1]
#get polygones which perimeter is greater than 5DBU
set l [po::perimeter $cm1 -value <5]
db::foreach p [db::getattr l.pointArray] {
    # create shapes on poly layer
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

po::polyToTrap

Description

Splits polygons into trapezoids by horizontal/vertical slicing.

Syntax

```
po::polyToTrap <input: dbCollection | poPointSet> [-hSlicing <bool>]
```

Arguments

input

Specify the operand1 points container.

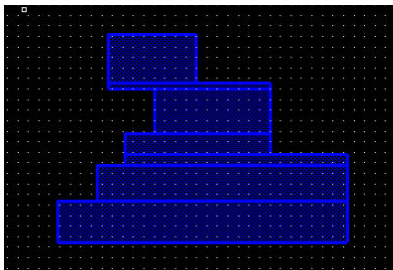
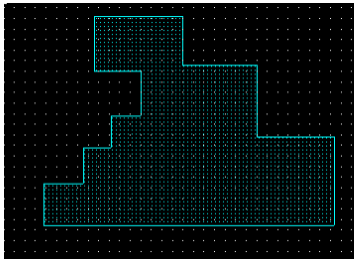
-hSlicing

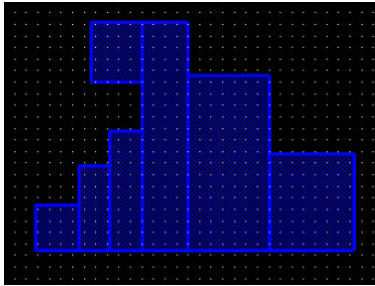
Slice polygons by horizontal direction or not. (Default: true)

Returns

<dbCollection | poPointSet> : The result points container.

Examples





```
# po::polyToTrap
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# Get Path shapes on m1 drawing lpp
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
set cml [po::createCollection $m1]
# Splits polygons into trapezoids by horizontal/vertical slicing
set l [po::polyToTrap $cml -hSlicing true]
//set l [po::polyToTrap $cml -hSlicing false]
db::foreach p [db::getattr l.pointArray] {
    # create shapes on poly layer
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

po::rectangle

Description

Selects rectangles from input-collection based on geometric properties.

Syntax

```
po::rectangle <input: dbCollection> [-width <string>] [-length <string>]
[-aspectRatio <string>] [-measure <enum>] [-not <bool>]
```

Arguments

input

Specify input-collection.

-width

Specify the width criterion of selected rectangles in user units. (For example:
>=0.05)

-length

Specify the length criterion of selected rectangles in user units. (For example:
>=0.08)

`-aspectRatio`

Specify the aspect ratio constraint of the selected rectangles. (For example: >2)

`-measure`

Specify the measuring criterion. Valid values: extents, orthogonal. (Default: orthogonal)

`-not`

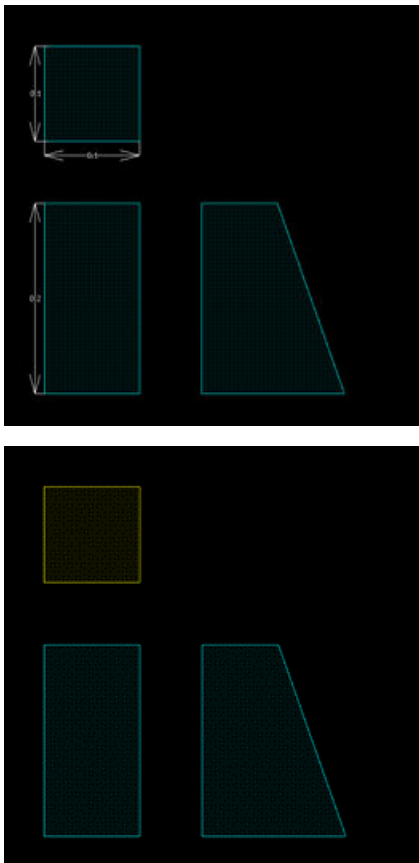
"-not true" is the complement of the rectangle operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples

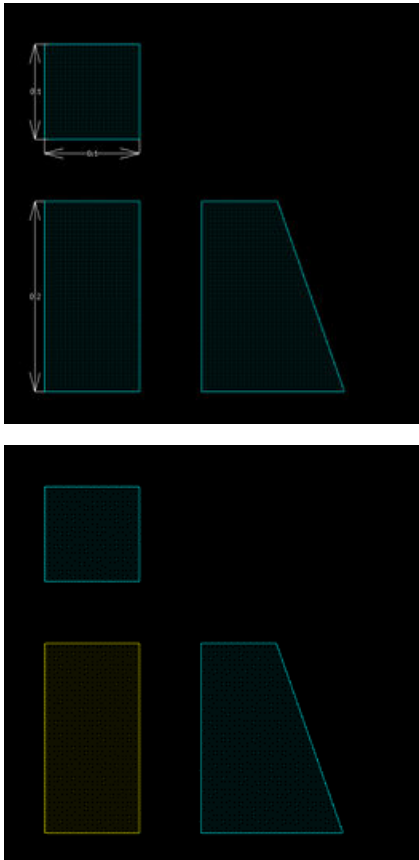
Example 1



In this example, this command selects rectangle which aspect ratio is equal to 0.5. The rectangle with resulting points is created.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]
set resPts [po::rectangle $srcPts -aspectRatio 1]
db::foreach points [db::getattr resPts.pointArray] {
    le::createPolygon $points -design $design -lpp m2
}
```

Example 2



In this example, this command selects rectangle which length equal to 0.2 and width equal to 0.1. The rectangle with resulting points is created.

```
set resPts [po::rectangle $srcPts -length 0.2 -width 0.1]
db::foreach points [db::getattr resPts.pointArray] {
    le::createPolygon $points -design $design -lpp m2
}
```

po::rectangles

Description

Creates an array of rectangles of specified "-width", "-length" and "-space" in user units.

Syntax

```
po::rectangles -width <float> -length <float> -space <point>  
-insideOf <box | dbCollection> [-offset <point>]  
[-maintainSpacing <bool>] [-dbuPerUU <int>]
```

Arguments

-width

Specify the size of the created rectangles in x-coordinate direction in user units.

-length

Specify the size of the created rectangles in y-coordinate direction in user units.

-space

Specify the x/y spacing in user units between created rectangles.

-insideOf

Specify the extents to create rectangles.

-offset

Specify the x or y offset when keep spacing in y or x-coordinate direction.

-maintainSpacing

If specify "-maintainSpacing true", no other rectangle may fall within the spacing halo. (Default: false)

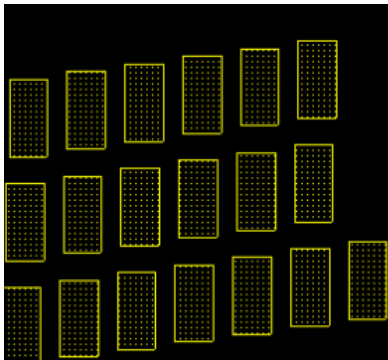
-dbuPerUU

Specifies dbuPerUU for points in output-collection. (Default is: 1000)

Returns

<dbCollection> : The result points container.

Examples



```
# po::rectangles
#
set res [po::rectangles -width 1 -length 2 -space {0.5 0.7} \
    -insideOf {{0 0} {10 10}} -offset {0.1 0.2}]
db::foreach r $res {
    le::createPolygon [db::getattr r.pointArray] -lpp {m2 drawing} \
        -design [ed]
}
```

po::rectEnclosure

Description

Provides efficient checking of multiple-condition enclosure rules for orthogonal rectangular polygons which edges parallel to database coordinate axes.

Syntax

```
po::rectEnclosure <input: dbCollection> -with <dbCollection> -goodRules
    <string> [-abut <string>] [-singular <bool>] [-outsideAlso <bool>]
```

Arguments

input

The input collection.

-with

The with collection.

-goodRules

Specifies the good rules based on the good rule format. (For example: { {{0.15} 0.15} {0.15} {0.15}} {{0} {0.5 opposite} {0} {0.5 opposite}})

-abut

Specifies that separation between intersecting edges should also be checked under the specified constraint. (For example: <90)

-singular

Handle polygon singularities that are point-to-edge or point-to-point polygon intersections. (Default: false)

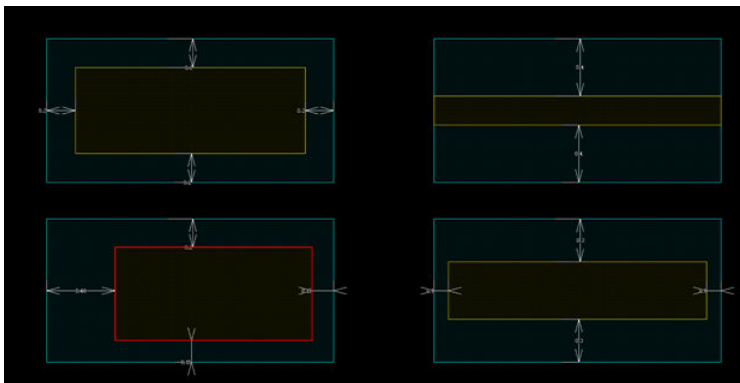
-outsideAlso

If its value is true, outputs rectangles from input-collection that are not enclosed by rectangles of with-collection. (Default: false)

Returns

<dbCollection> : The result points container.

Examples



In this example, this command returns points collection that violated specified -goodRules. The violated shape is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set outPts [po::createCollection $m1Shape]
set inPts [po::createCollection $m2Shape]

set resPts [po::rectEnclosure $inPts -with $outPts -goodRules {
    {{0.2} {0.2} {0.2} {0.2}}
    {{0.3} {0.1} {0.3} {0.1}}
    {{0.4} {0} {0.4} {0}}}]
db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po \
    -packet highlight2 -isClosed
}
```

po::reshape

Description

Reshape points with the given region and mode.

Syntax

```
po::reshape <points: [point]> [-closed <bool>] -points [<point>] [-mode  
  <enum>] [-dbu <int>]
```

Arguments

points

Specify the list of points that form reshape.

-close

Specify whether the input points are closed. (Default: true)

-points

Specify the point list for reshape, based on which a new point list is formed.

-mode

Specify the reshape mode. Valid values: ANDNOT, OR. (Default: OR)

-dbu

Specify dbuPerUU for input points. (Default: 1000)

Returns

<point> : The reshaped points.

po::shapePoints

Description

Shape input points to remove jogs and output orthogonal points.

Syntax

```
po::shapePoints <points: [point]> -jogLen <float> [-closed <bool>]  
  [-convexConcave <bool>] [-dbuPerUU <int>]
```

Arguments

points

The list of points that form shape points.

`-jogLen`

Specify the jog length in user units.

`-closed`

Specify the input points is closed or not. (Default: true)

`-convexConcave`

Specify the two corners of jog edge satisfying 90 and 270 degrees. (Default: true)

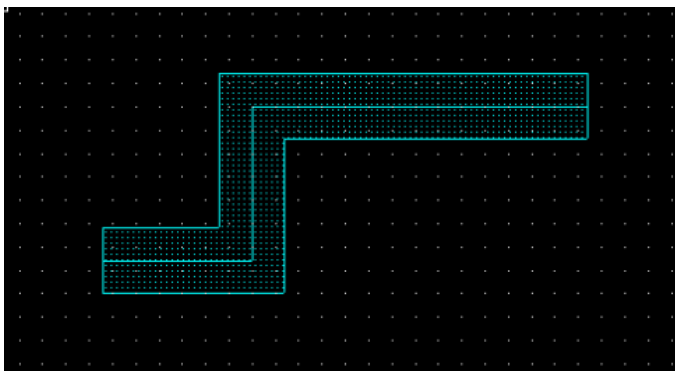
`-dbuPerUU`

Specify dbuPerUU for points input. (Default: 1000)

Returns

[<point>] : The orthogonal points having no jogs.

Examples



```
> puts $sh
{0.935 0.435} {0.935 0.365} {0.61 0.365} {0.61 0.2} {0.415 0.2} {0.415 0.27} {0.54 0.27} {0.54 0.435}
>
```

```
# po::shapePoints
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# Get rectangle shapes on m1 drawing lpp
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
set cm1 [po::createCollection $m1]
set sh [po::shapePoints [db::getattr cm1.pointArray] -jogLen 0.001 \
    -closed true]
```

po::shift

Description

Shifts all polygons of input-collection by the specified x-coordinate offset and y-coordinate offset.

Syntax

```
po::shift <input: dbCollection> -by <point>
```

Arguments

input

The input collection.

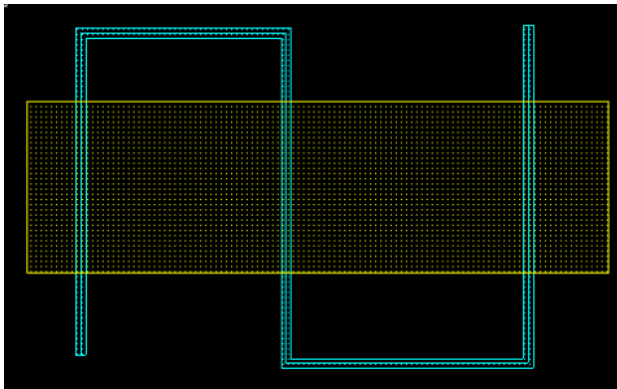
-by

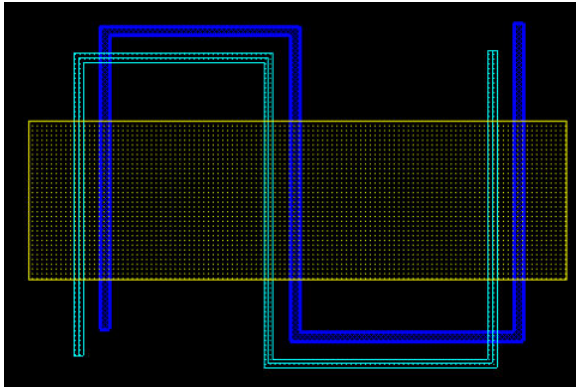
Specifies the shifting offsets for x-coordinate and y-coordinate.

Returns

<dbCollection> : The result points container.

Examples





```
# po::shift
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# Get rectangle shapes on m1 drawing lpp
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
# create collection
set cml [po::createCollection $m1]
# get collection of shifted object points
set res [po::shift $cml -by {0.1 0.1}]
db::foreach p $res {
    le::createPolygon [db::getattr p.pointArray] -lpp {poly drawing} \
        -design $design
}
```

po::size

Description

Do sizing uniformly or non-uniformly.

Syntax

```
po::size <input: dbCollection | poPointSet> -by <box | float | point>
[-stepping <bool>]
```

Arguments

input

Specify the operand1 points container.

-by

Specify the values to do sizing uniformly or non-uniformly.

-stepping

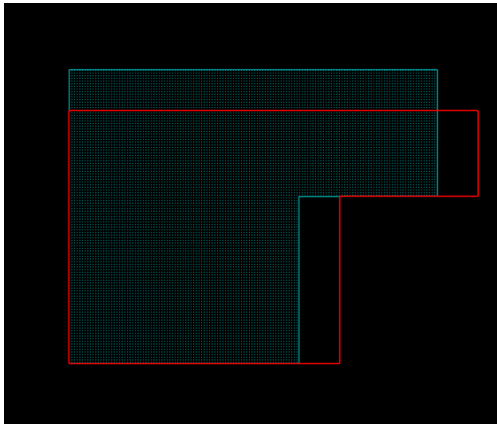
If specify "-stepping true", do growing or shrinking step by step. (Default: true)

Returns

<dbCollection | poPointSet> : The result points container.

Examples

Example 1

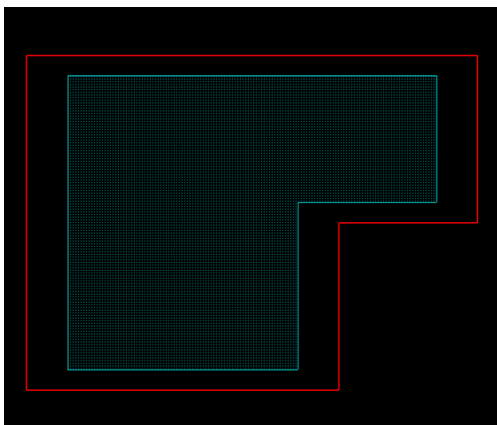


```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]

set resPts [po::size $srcPts -by {{0 0} {0.1 -0.1}}]

db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
        highlight2 -isClosed
}
```

Example 2

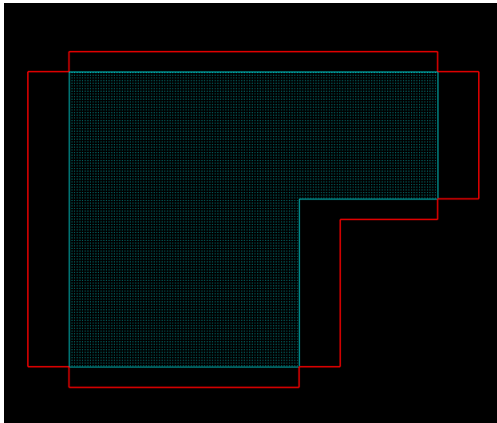


```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
```

```
set srcPts [po::createCollection $m1Shape]

set resPts [po::size $srcPts -by {0.1 0.05}]
db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
    highlight2 -isClosed
}
```

Example 3

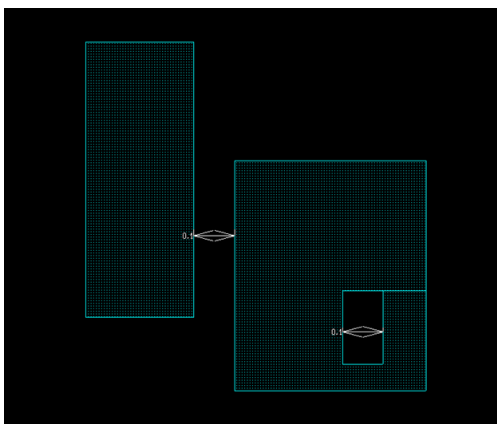


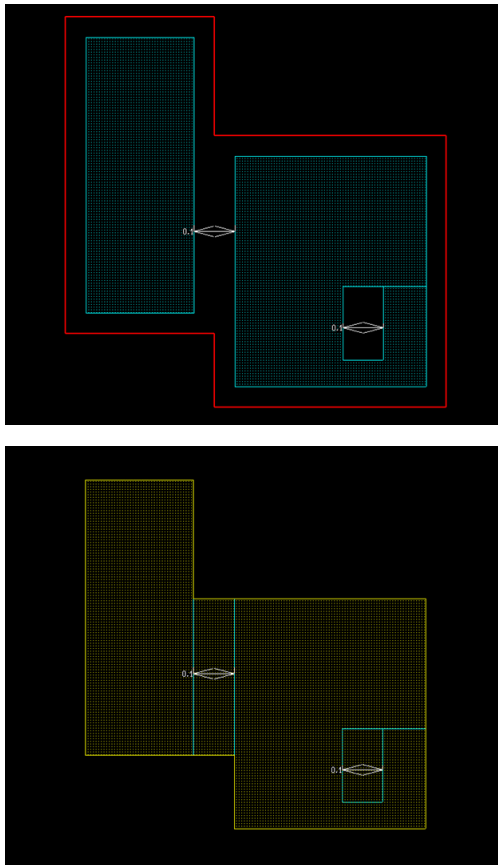
```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]

set resPts [po::size $srcPts -by {0.1 0.05} -stepping false]

db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
    highlight2 -isClosed
}
```

Example 4





In this example, polygons with a spacing of 0.1u are combined and a 0.1 μm wide hole is removed by increasing them by 0.05u and then shrinking them by the same amount.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]
# Input points increased by 0.05u
set resPts [po::size $srcPts -by 0.05 ]

db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
    highlight2 -isClosed
}

# Input points shrunk by 0.05u
set resPts [po::size $srcPts -by -0.05 ]
db::foreach pts [db::getattr resPts.pointArray] {
    le::createPolygon $pts -design $design -lpp m1
}
```

po::snapGrid

Description

Snaps the input polygons on grid users specified.

Syntax

```
po::snapGrid <input: dbCollection> -resolution <point> [-overSize <bool>]  
[-overSize <bool>] [-underSize <bool>]
```

Arguments

input

Specify input-collection.

-resolution

Specify the grid resolution in user units. x-coordinate is applied in the x-direction and y-coordinate is applied in the y-direction.

-overSize

Specify the operation result would enclose the input polygons or not. "-overSize true" and "-underSize true" are exclusive. (Default: true)

-underSize

Specify the operation result would be inside the input polygons or not. "-overSize true" and "-underSize true" are exclusive. (Default: false)

Returns

<dbCollection> : The result points container.

Examples

```
# Creating off-grid points collection  
set pts [po::createCollection \  
    {{0.002 0.002} {0.002 0.201} {0.201 0.201} {0.201 0.002}}]  
# The po::snapGrid command snaps points to nearest grid  
set onGrid [po::snapGrid $pts -resolution {0.05 0.05}]  
db::getattr onGrid.pointArray  
{0.0 0.25} {0.25 0.25} {0.25 0.0} {0.0 0.0}
```

po::split

Description

Split points with the given region and mode.

Syntax

```
po::split <points: [point]> [-closed <bool>] -points [<point>] [-offset  
<point>] [-mode <enum>] [-dbu <int>]
```

Arguments

points

Specify the list of points that form for split.

-close

Specify whether the input points are closed. (Default: true)

-points

Specify the point list for split, based on which the split shape is determined.

-offset

Specify the x-offset and y-offset in user units.

-mode

Specify the split mode. Valid values: adjacent, inside, outside. (Default: inside)

-dbu

Specify dbuPerUU for input points. (Default: 1000)

Returns

<string> : The split points and the changed points compared to the original ones.

Examples

```
> set gr_centerline { {0.0 0.0} {0.0 5.0} {5.0 5.0} {5.0 0.0} }  
{0.0 0.0} {0.0 5.0} {5.0 5.0} {5.0 0.0}  
> set chop_region { {2.0 6.0} {3.0 6.0} {3.0 4.0} {2.0 4.0}  
{2.0 6.0} } {2.0 6.0} {3.0 6.0} {3.0 4.0} {2.0 4.0} {2.0 6.0}  
> set stretch_vector {0.0 3.0}  
0.0 3.0  
> set result [po::split $gr_centerline -closed true \  
-points $chop_region -offset $stretch_vector -mode inside \  
-dbu 100]  
{ {0.0 0.0} {0.0 5.0} {2.0 5.0} {2.0 8.0} {3.0 8.0} {3.0 5.0}  
{5.0 5.0} {5.0 0.0} {0.0 0.0} } { {2.0 8.0} {3.0 8.0} } }  
> set cnt [llength $result]  
2  
> set new_centerline [lindex $result 0]  
{0.0 0.0} {0.0 5.0} {2.0 5.0} {2.0 8.0} {3.0 8.0} {3.0 5.0}  
{5.0 5.0} {5.0 0.0} {0.0 0.0}  
> set highlighting [lindex $result 1]  
{ {2.0 8.0} {3.0 8.0} }
```

po::straddle

Description

Select all polygons from input-collection that straddle a polygon of with-collection.

Syntax

```
po::straddle <input: dbCollection> -with <dbCollection> [-not <bool>]
```

Arguments

input

Specify input-collection.

-with

Specify with-collection.

-not

"-not true" is the complement of the straddle operation. (Default: false)

Returns

<dbCollection> : The result points container.

po::touch

Description

Selects all polygons from input-collection that are completely outside all polygons of with-collection and share a coincident edge, or segment, with a polygon of with-collection.

Syntax

```
po::touch <input: dbCollection> -with <dbCollection> [-value <string>]  
[-hasEvenOdd <enum>] [-not <bool>]
```

Arguments

input

Specify input-collection.

-with

Specify with-collection.

-value

Specify the selection criterion. (For example: >=2)

`-hasEvenOdd`

Cooperate with `"-value"` to specify the extra selection criterion. Valid values: even, none, odd. (Default: none)

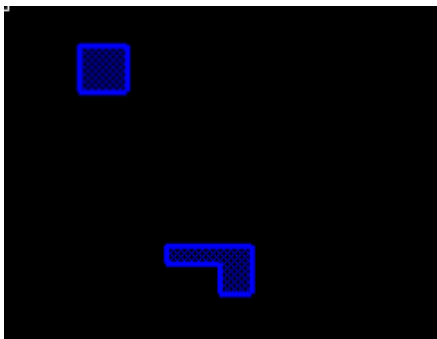
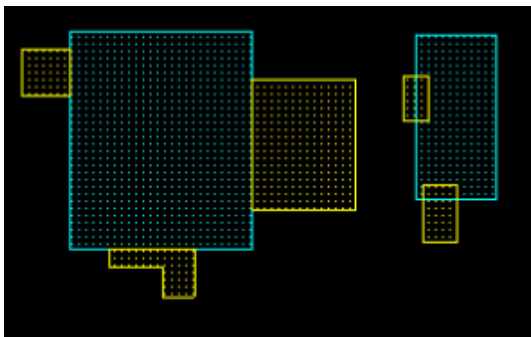
`-not`

`"-not true"` is the complement of the touch operation. (Default: false)

Returns

`<dbCollection>` : The result points container.

Examples



```
# po::touch
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shapes on m2 drawing
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
# select shapes on m2 layer which touches shapes on m1 layer
set t [po::touch $cm2 -with $cm1]
# get points of "t" collection
db::foreach p [db::getattr t.pointArray] {
```

```
#create shapes on poly layer
le::createPolygon $p -lpp {poly drawing} -design $design
}
```

po::touchEdge

Description

Selects all complete edges from input-collection that are coincident with any portion of edges in with-collection.

Syntax

```
po::touchEdge <input: dbCollection> -with <dbCollection> [-dir <enum>]
[-not <bool>]
```

Arguments

input

The input collection.

-with

The with collection.

-dir

Specifies the touch direction. Valid values: all, inside, outside. (Default: all)

-not

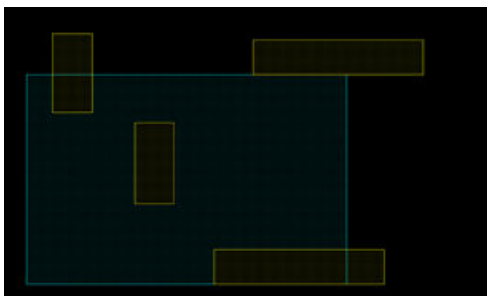
"-not true" is the complement of the touchEdge operation. (Default: false)

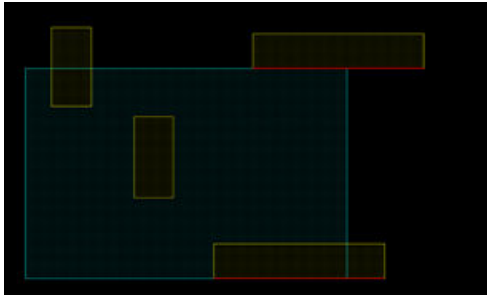
Returns

<dbCollection> : The result points container.

Examples

Example 1



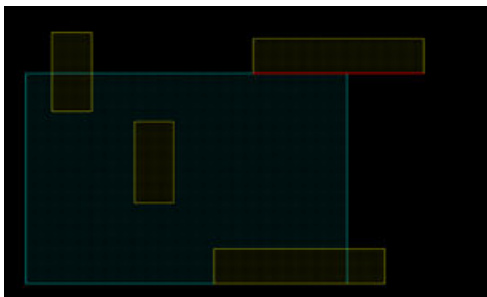
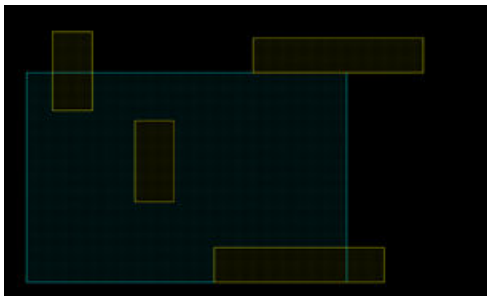


In this example, this command returns collection of point objects of the m2 metal layer shape touching edges with m1 layer shape. The result edges are highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]
set resPoints [po::touchEdge $srcPts -with $withPts -dir all -not 0]

db::foreach edge $edges {
    xt::highlightViolation [db::getattr pts.pointArray] -design $design \
        -toolName po -packet highlight2 -isClosed
}
```

Example 2



In this example, this command returns collection of point objects of the m2 metal layer shape touching edges outside m1 layer shape. The result edge is highlighted.

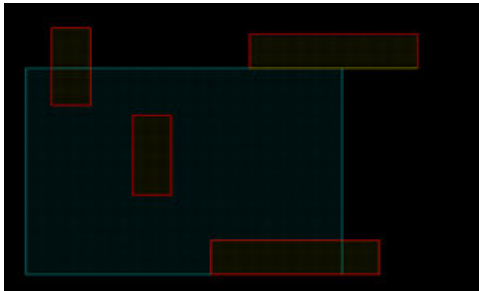
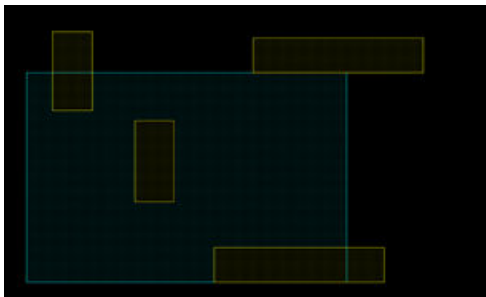
```
set design [ed]
```

```
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]

set resPoints [po::touchEdge $srcPts -with $withPts -dir outside -not 0]

db::foreach edge $edges {
    xt::highlightViolation [db::getAttr pts.pointArray] -design $design \
        -toolName po -packet highlight2 -isClosed
}
```

Example 3

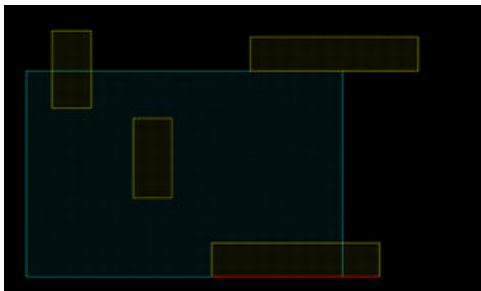
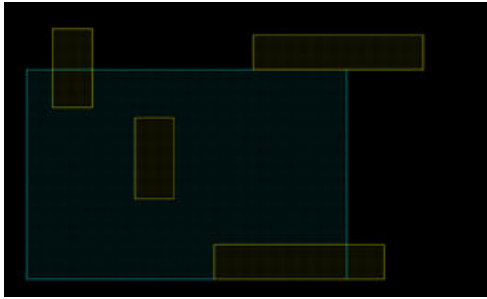


In this example, this command returns collection of point objects of the m2 metal layer shape not touching edges outside m1 layer shape. The result edges are highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]
set resPoints [po::touchEdge $srcPts -with $withPts -dir outside -not 1]

db::foreach edge $edges {
    xt::highlightViolation [db::getAttr pts.pointArray] -design $design \
        -toolName po -packet highlight2 -isClosed
}
```

Example 4

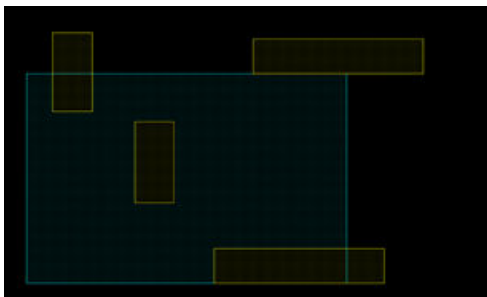


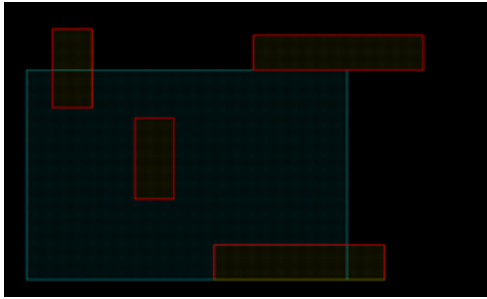
In this example, this command returns collection of point objects of the m2 metal layer shape touching edges inside m1 layer shape. The result edge is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]
set resPoints [po::touchEdge $srcPts -with $withPts -dir inside -not 0]

db::foreach edge $edges {
    xt::highlightViolation [db::getattr pts.pointArray] -design $design \
        -toolName po -packet highlight2 -isClosed
}
```

Example 5





In this example, this command returns collection of point objects of the m2 metal layer shape not touching edges inside m1 layer shape. The result edges are highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set m2Shape [db::getShapes -lpp m2 -of $design]
set srcPts [po::createCollection $m2Shape]
set withPts [po::createCollection $m1Shape]
set resPoints [po::touchEdge $srcPts -with $withPts -dir inside -not 1]

db::foreach edge $edges {
    xt::highlightViolation [db::getattr pts.pointArray] -design $design \
        -toolName po -packet highlight2 -isClosed
}
```

po::transform

Description

Transform points based on the selected points, some flags, offset and orientation.

Syntax

```
po::transform <points: [point]> [-selected [<point>]] [-offset <point>]
[-closed <bool>] [-lockAngles <bool>] [-keepLength <bool>] [-keepHole
<bool>] [-lockEnds <bool>] [-formLoop <bool>] [-dbuPerUU <int>] [-orient
<enum>]
```

Arguments

points

The list of points that form transforming points.

-selected

Transform the selected points. If no selected points, transform all input points.

-offset

Specify the x-offset and y-offset in user units.

`-closed`

Specify the input points is closed or not. (Default: true)

`-lockAngles`

Indicate a flag maintaining the corresponding angles of the selected points when transform points. (Default: true)

`-keepLength`

Keep the length of adjacent selected points. (Default: false)

`-keepHole`

Keep the number of the holes in the input closed points. (Default: true)

`-lockEnds`

Lock the endpoints of the input polyline when the selected points exclude endpoints. (Default: true)

`-formLoop`

Only for the polyline. Refine the start or end point when the input polyline is self intersecting. (Default: false)

`-dbuPerUU`

Specify dbuPerUU for points input. (Default: 1000)

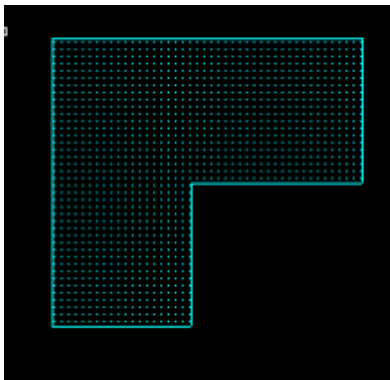
`-orient`

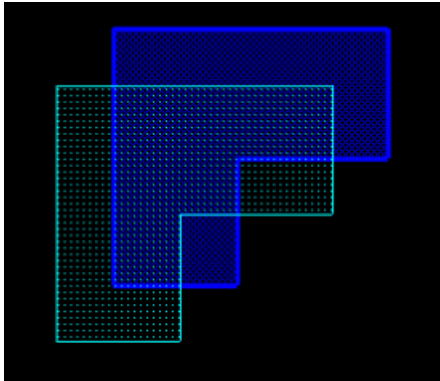
Orientation of the transforming points. Valid values: MX, MXR90, MY, MYR90, R0, R180, R270, R90. (Default: R0)

Returns

[<point>] : The transformed points. If the result is self-intersecting, returns empty list.

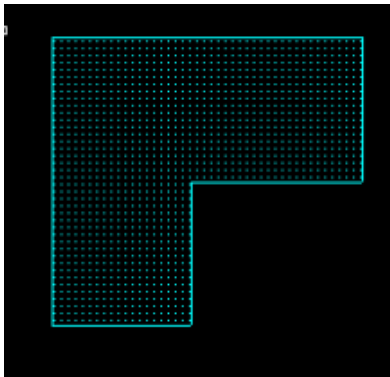
Examples

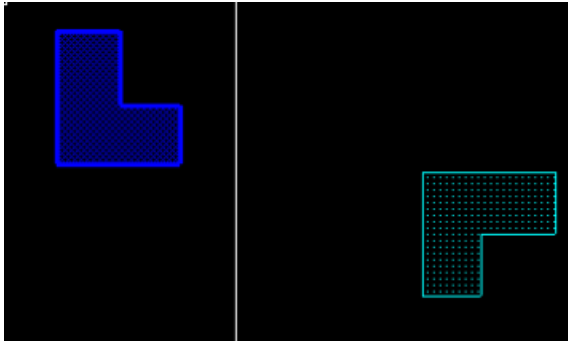




```
# po::transform
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# Get rectangle shapes on m1 drawing lpp
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#create collection
set cml [po::createCollection $m1]
#get points of transformed shapes
set tr [po::transform [db::getattr cml.pointArray] -offset {0.1 0.1}]
#Create polygon on poly drawing lpp with transformed points
le::createPolygon $tr -lpp {poly drawing} -design $design
```

Or





```
# get points of trasformed shapes
set tr [po::transform [db::getattr cml.pointArray] -offset {0.1 0.1} \
    -orient R90]
# Create polygon on poly drawing lpp with transformed points
le::createPolygon $tr -lpp {poly drawing} -design $design
```

po::vertex

Description

Selects all polygons from input-collection having vertex counts conforming to the value.

Syntax

```
po::vertex <input: dbCollection> -value <string>
```

Arguments

input

Specify input-collection.

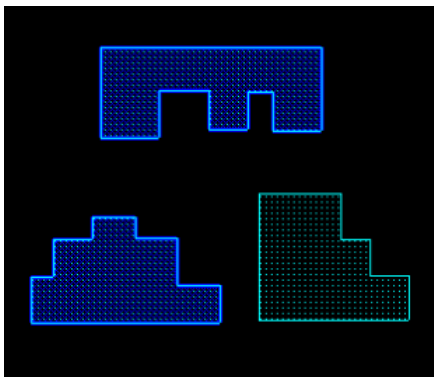
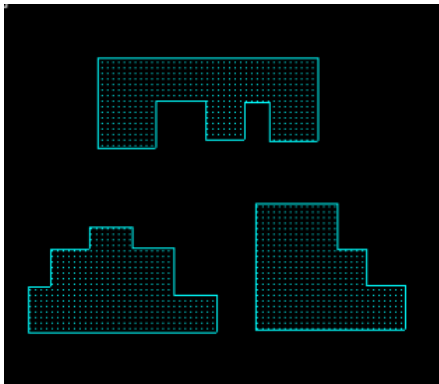
-value

Specify the specific number of vertices for a polygon. (For example: <=4)

Returns

<dbCollection> : The result points container.

Examples



```
# po::vertex
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
# Get points of shapes which has more than 5 verteses
set v [po::vertex $cm1 -value >10]

db::foreach p [db::getattr v.pointArray] {
    #create shapes on poly layer
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

po::withEdge

Description

Select polygons from input-collection that have edges or edge segments on with-collection.

Syntax

```
po::withEdge <input: dbCollection> -with <dbCollection> [-value <string>]  
[-not <bool>]
```

Arguments

input

The input collection.

-with

The with collection.

-value

Specifies the specific number of edges or segments of with-collection. (For example: >=2)

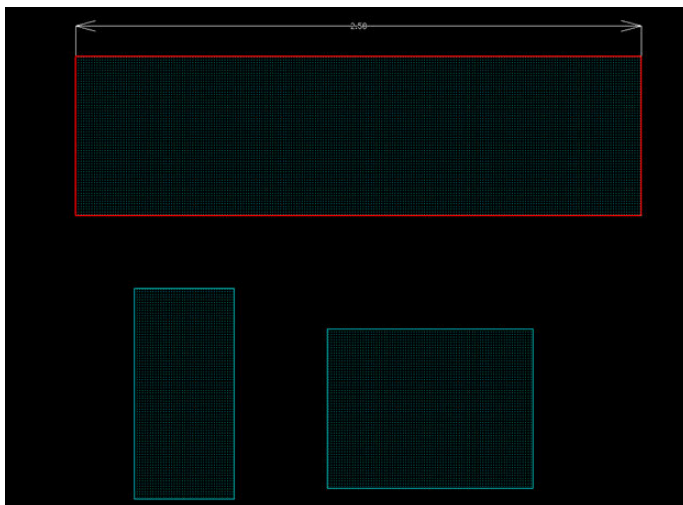
-not

"-not true" is the complement of the withEdge operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples



In this example, this command returns rectangle points that has edges with length greater than 2.0u. The found rectangle is highlighted.

```
set design [ed]  
set m1Shape [db::getShapes -lpp m1 -of $design]  
set srcPts [po::createCollection $m1Shape]
```

```
set resPts [po::withEdge $srcPts -with [po::length $srcPts -value >2]]

db::foreach pts [db::getAttr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
    highlight2 -isClosed
}
```

po::withNeighbor

Description

Selects all orthogonal rectangles from input-collection if they have the specified number of orthogonal rectangles within the specified distance.

Syntax

```
po::withNeighbor <input: dbCollection> [-with <dbCollection>] -value  
    <string> -space <string> [-metric <enum>] [-not <bool>]
```

Arguments

input

Specify input-collection.

-with

Specify with-collection.

-value

Specify the selection criterion to represent the number of neighboring rectangles. (For example: ==2)

-space

Specify the within distance in user units. (For example: <=0.1)

-metric

Cooperate with "-space" to specify the measuring criterion. Valid values: center, euclidean, square. (Default: euclidean)

-not

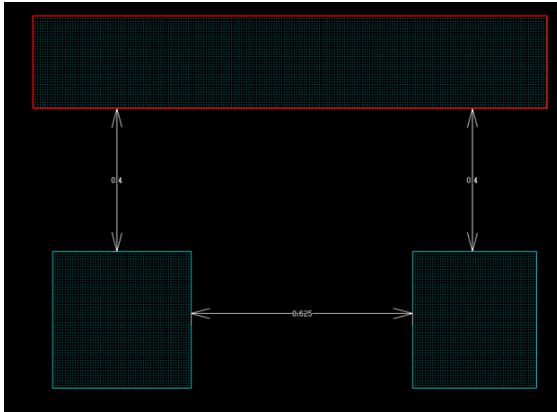
"-not true" is the complement of the withNeighbor operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples

Example 1

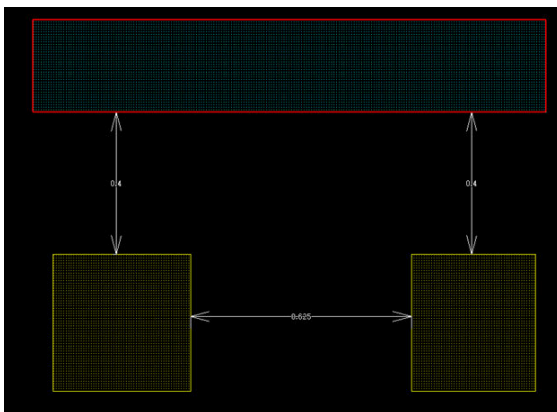


In this example, this command returns rectangle points that has two neighbor with 0.4u distance in a same collection. The result rectangle is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]

set resPts [po::withNeighbor $srcPts -space ==0.4 -value 2]
db::foreach pts [db::getattr resPts.pointArray] {
    xt::highlightViolation $pts -design $design -toolName po -packet \
    highlight2 -isClosed
}
```

Example 2



In this example, this command returns rectangle points that has two neighbors with 0.4u distance in layer m2 collection. The result rectangle is highlighted.

```
set design [ed]
set m1Shape [db::getShapes -lpp m1 -of $design]
set srcPts [po::createCollection $m1Shape]
set m2Shape [db::getShapes -lpp m2 -of $design]
set withPts [po::createCollection $m2Shape]
```

```
set resPts [po::withNeighbor $srcPts -with $withPts -space ==0.4 \  
    -value 2]  
db::foreach pts [db::getattr resPts.pointArray] {  
    xt::highlightViolation $pts -design $design -toolName po -packet \  
    highlight2 -isClosed  
}
```

po::withWidth

Description

Select all regions of polygons in input-collection that satisfy the width constraint to generate output polygons.

Syntax

```
po::withWidth <input: dbCollection> [-value <string>] [-not <bool>]
```

Arguments

input

The input collection.

-value

Specifies the width constraint in user units. (For example: <0.3)

-not

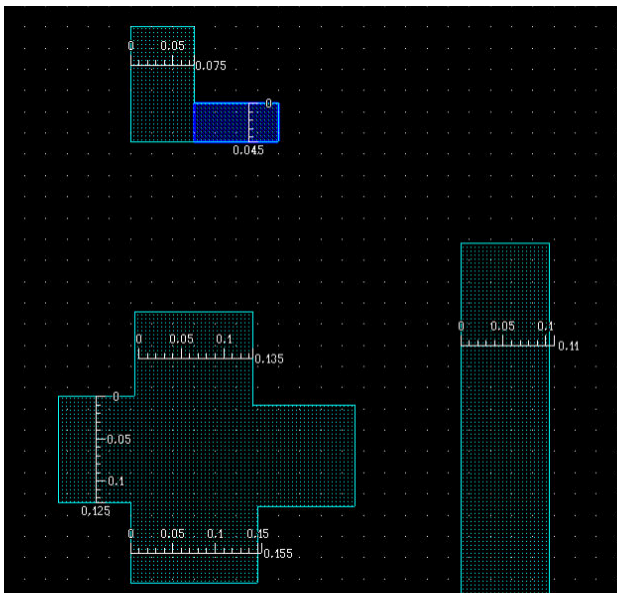
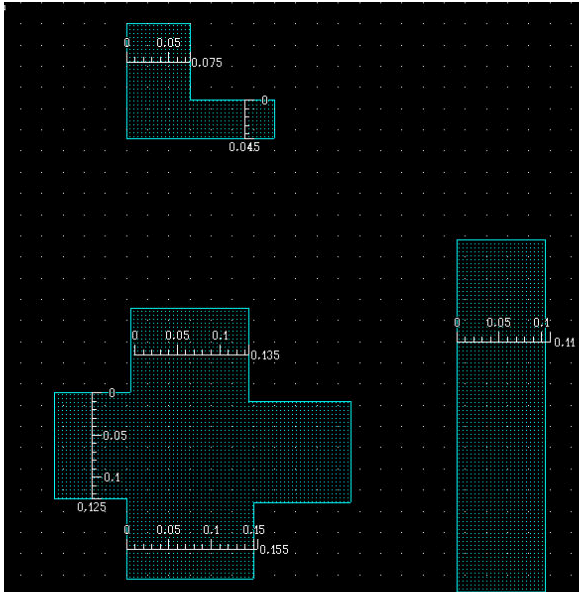
"-not true" is the complement of the withWidth operation. (Default: false)

Returns

<dbCollection> : The result points container.

Examples

Example 1

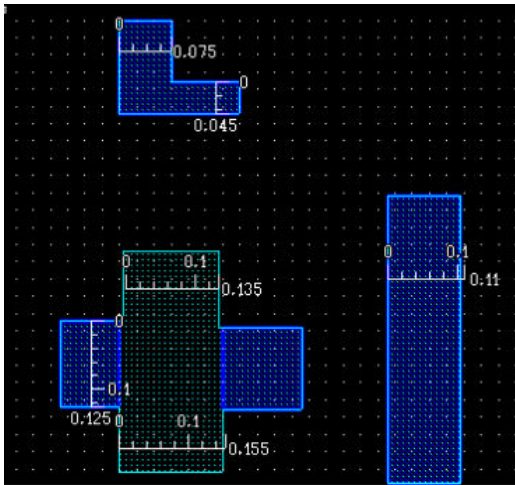


```
# po::withWidth
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# get shapes on m1 drawing layer
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
# create collections with selected shapes
set cml [po::createCollection $m1]
# select all polygons of the shapes that
# satisfy the width constraint to generate output polygons
# their edges with polygons of the m1 layer
```

```
set we [po::withWidth $cm1 -value >0.01<0.05]

# create resulting polygons on poly drawing layer
db::foreach p [db::getattr we.pointArray] {
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

Example 2



```
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
# get shapes on m1 drawing layer
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
# create collections with selected shapes
set cm1 [po::createCollection $m1]
# select all polygons of the shapes that
# satisfy the width constraint to generate output polygons
# their edges with polygons of the m1 layer
set we [po::withWidth $cm1 -value >0.128 -not true]
# create resulting polygons on poly drawing layer
db::foreach p [db::getattr we.pointArray] {
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

po::xor

Description

Run the Boolean XOR operation.

Syntax

```
po::xor <input: dbCollection | poPointSet> -with <dbCollection |
poPointSet>
```


Arguments

`input`

Specify the operand1 points container.

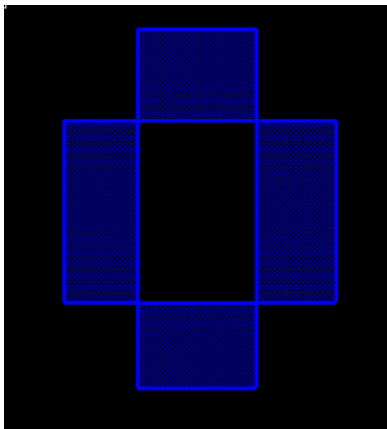
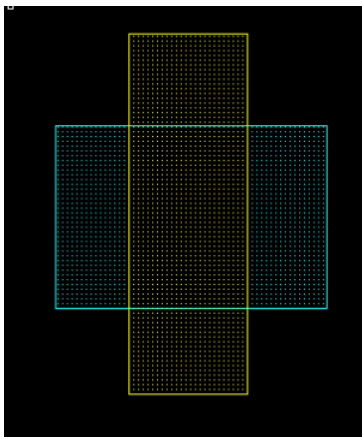
`-with`

Specify the operand2 points container.

Returns

`<dbCollection | poPointSet>` : The result points container.

Examples



```
# po::xor
#
set ctx [de::getContexts -window [gi::getActiveWindow ]]
set design [db::getattr ctx.editDesign]
#get shapes on m1 drawing
set m1 [db::getShapes -lpp {m1 drawing} -of $design]
#get shapes on m2 drawing
```

```
set m2 [db::getShapes -lpp {m2 drawing} -of $design]
#create collections with selected shapes
set cm1 [po::createCollection $m1]
set cm2 [po::createCollection $m2]
# xor all polygons from m2 layer with shapes on m1 layer
set or [po::xor $cm2 -with $cm1]
# select shapes in "or" collection
db::foreach p [db::getattr or.pointArray] {
    #create shapes on poly layer
    le::createPolygon $p -lpp {poly drawing} -design $design
}
```

Preferences

Table 172 lists Custom Compiler po namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 172 Geometry Manipulation Preferences

Name	Type	Scope	Default	Description
poDBUpperUU	int	user	1000	Defines the default dbuPerUU for po::* commands.
poEllipseNumSides	int	user	32	Defines the default number of sides to transform an ellipse to a polygon for po::* commands

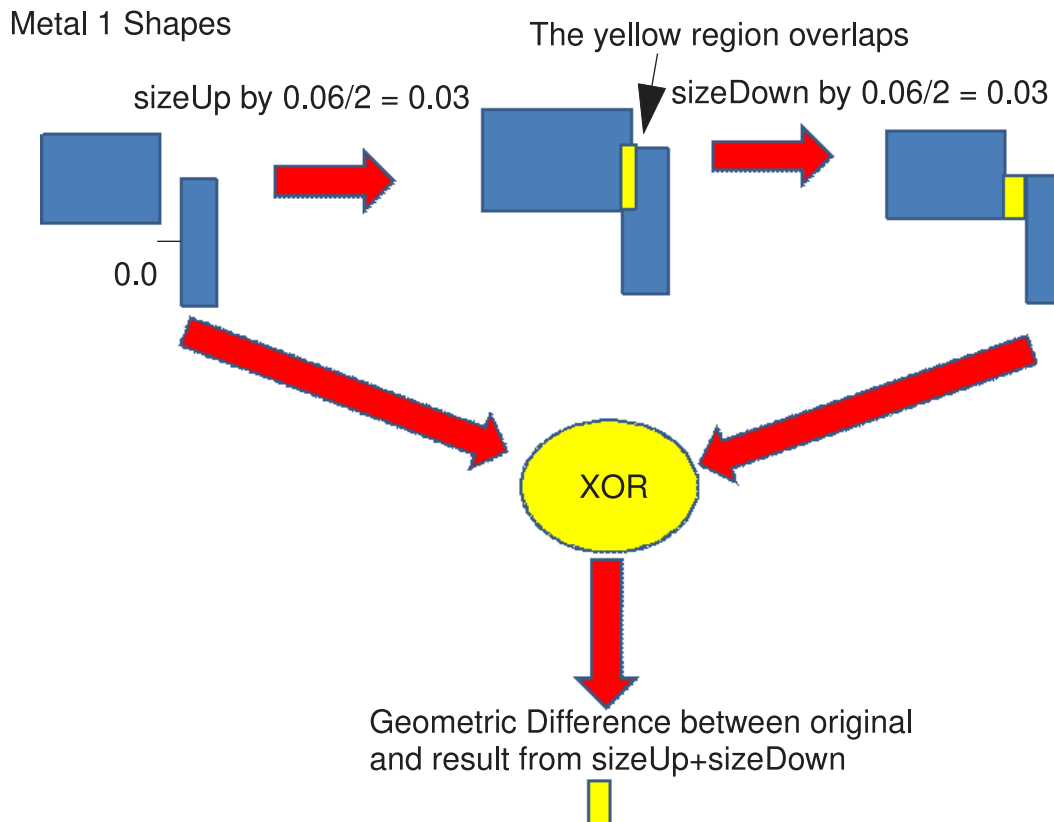
Geometry Manipulation Example

The geometry engine consists of Tcl APIs that:

- Are geometric operators for operating on geometric operands called poPointSets or points. These include po::and, po::or, po::not, po::xor, po::getContours, po::size, and po::transform.
- Construct or enlarge poPointSets from layout shapes or from Tcl lists of points. These include po::creatPointSet, po::append, po::compress, and po::makeClockwise.
- modify input points. These include po::compress and po::makeClockwise.
- Retrieve Tcl lists of points from poPointSets for layout shape creation or further processing. This includes po::getPoints.
- Measure or control utilities on geometry operands called poPointSet or points. These include po::getArea, po::getDistance, po::getIntersection, po::getPerimeter, po::isClockwise, po::is Horizontal, and po::isVertical.

A typical example used in physical verification tools and other geometry engines is to make your own DRC checker. You do this by collecting all shape data in your design on a certain layer, then growing their dimensions by 1/2 of the minimum allowable spacing. Any shapes that are spaced by less than the minimum spacing will produce regions of overlap. When this geometry then gets sized back down to its original dimensions, the overlap regions remain as violation markers.

Figure 2 Example Using Minimum Spacing Equal to 0.06



The following sample script uses the `po::` commands to create a simple `minSpacing` DRC checker.

```
set minSpacing 0.07
set cxt [de::getActiveContext]
set design [db::getattr cxt.editDesign]
#get the metal 1 shapes in the design
set m1Figures [de::getFigures [db::getattr design.bBox] -type rectangle
  -design $design \
    -touch 1 -depth 32 -filter {[lsearch {Path PathSeg Rectangle}
    %objType]>=0}]

#create a poPointSet representing all metal1 geometry in the design
set m1Geometry [po::createPointSet]
```

```
db::foreach fig $m1Figures {po::append [db::getattr fig.object] -to
    $m1Geometry}

#size up, size dow, and xor to find spacing violations
set upSize [po::size $m1Geometry -by [expr {$minSpacing/2}]]
set downSize [po::size $upSize -by [expr {- $minSpacing/2}]]
set violations [po::xor $downSize -with $m1Geometry]

#generate error markers to see and debug violation in the Error Viewer
set t [de::startTransaction "creating violations shapes" -design $design]
while {[set p [po::getPoints $violations]]!=""} {
    set severity [oa::MarkerSeverity "error"]
    set shortMessage "minSpacing violation found: $p"
    set longMessage "Minimum Spacing Must be >= $minSpacing for layer m1 found
    at: $p"
    oa::MarkerCreate [oa::getTopBlock $design] $p $longMessage $shortMessage "
    leLayout" 1 0 $severity
}
de::endTransaction $t
```

The following figures show you before and after running the minSpacing DRC checker script.

Figure 3 *Before Running minSpacing DRC Checker Script*

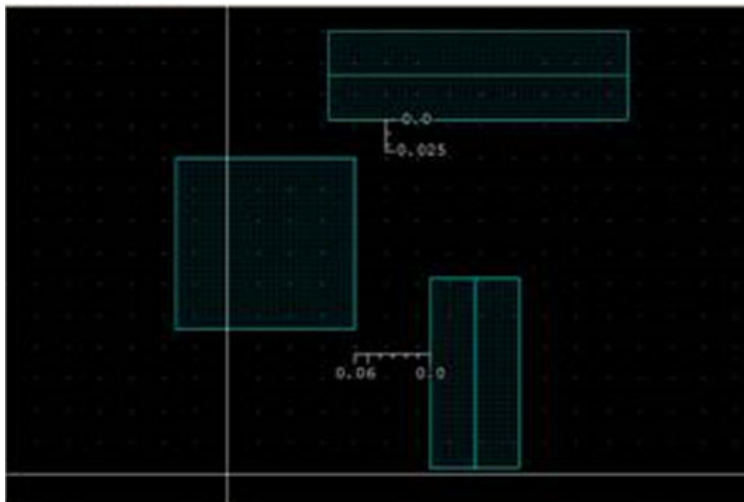
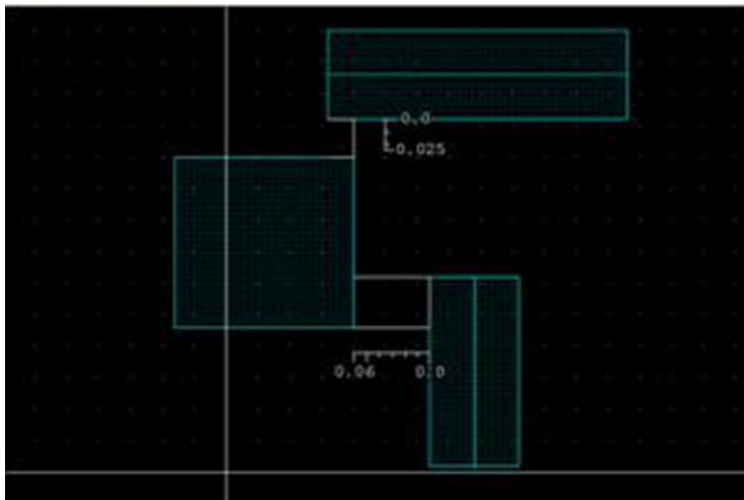


Figure 4 After Running minSpacing DRC Checker Script



20

Relative Placement Language Tcl

This chapter lists the commands in the Custom Compiler Tcl rpl namespace. This namespace provides Tcl for relative placement language operations.

For details, see:

- [Introduction to Relative Placement Language Tcl](#)
- [Commands](#)
- [Preferences](#)

Introduction to Relative Placement Language Tcl

The Relative Placement Language (RPL) captures layout creation process based on user-driven scripts. The user script is used to place instances by grouping and abutment. The RPL language abstracts the complexity of Custom Compiler language to simple easy to use commands. The commands keep track of the design under construction as well at its context.

Note:

Stdcells and leaf-cells must have prBoundary defined.

This chapter covers the following topics:

- [Using Relative Placement Language](#)
- [Relative Placement Language Basics](#)
- [Using RPL Commands to Build a Circuit](#)

 [Show me](#) how to create instance arrays using RPL commands.

 [Show me](#) how to create memory instances using RPL scripts.

Using Relative Placement Language

The Relative Placement Language commands create layout designs. If a schematic view is available, RPL uses it in SDL mode to establish the netlists and the connections between instances. Otherwise, the schematic and symbol views can be derived from the layout views.

This section covers the following flows:

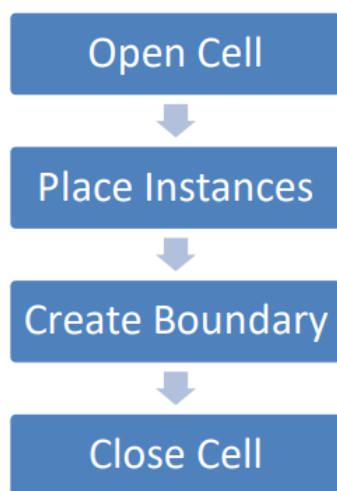
- [A Cell With Schematic](#)
- [A Cell Without Schematic](#)

A Cell With Schematic

Cells with schematic are created with SDL. Schematics can be generated by importing a netlist (Verilog, VHDL, or SPICE). Instances are automatically instantiated in the layout view with proper connectivity. The RPL user script can have only placement commands to place the design.

Terminals are also created automatically without pin shapes. The user can create pin shapes using native Custom Compiler commands.

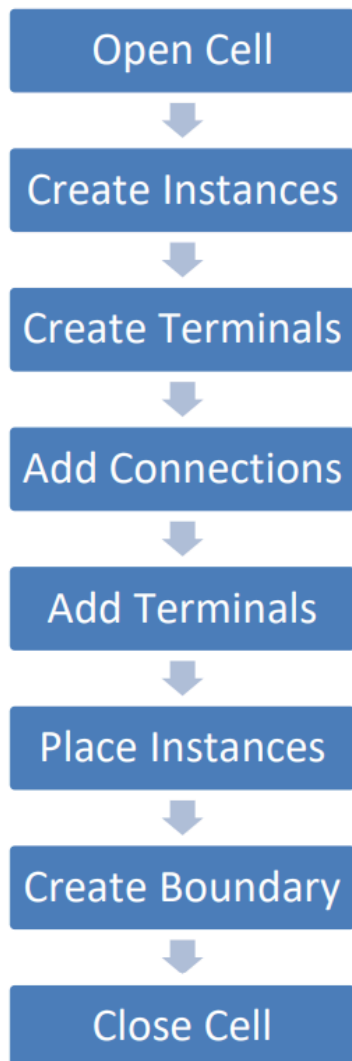
Example of cell created with schematic:



A Cell Without Schematic

When the schematic is not available for the cell creation, the instances and the connectivity must be provided by the RPL commands.

Example of cell created without schematic:



Relative Placement Language Basics

RPL is a language that does not require a design pointer to operate on. It assumes a design in context always from the opened design. RPL does not require to pass around the Open Access objects. All commands refer to object by names.

Examples

```
rpl::createCell myNewCell  
rpl::pack "i0 i1 i2"
```



```
rpl::createBoundary  
rpl::closeCell
```

This concept is very important to simplify the language. The goal is to make the language simple to use to non-programmer designers.

Net and Instance Name Definition

The RPL commands support vector notation for instance and net names. An instance named `memCell<63:0>` can be used in the commands.

Bits are defined with a range or individually. For example: `net1<32:10,5,4,2:0>`. Bit 3,6,7,8, and 9 are not part of this net name.

The RPL commands supports `<>` as well as `[ ]`. A net called `a[1]` is equivalent to a net called `a<1>`.

The RPL commands do a smart match between the instance or net passed to the command and the real instance or net name used in the database.

Using RPL Commands to Build a Circuit

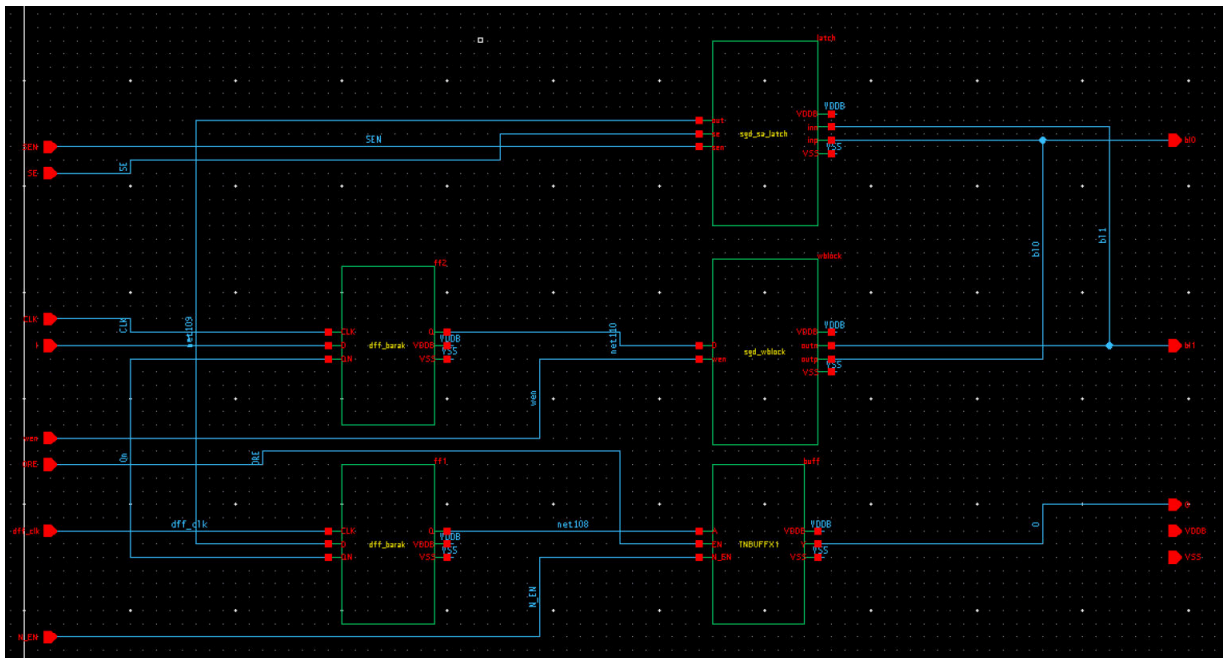
In this example, a simple RAM which is composed of rows of bits and ioreg for column decoders is built. This example shows how a circuit can be built using the RPL commands.

First, sets a few variables so the code is parameterized.

```
# Get the rpl package  
package require rpl  
  
# Set a few variables  
set rows 64  
set cols 64  
set rows_1 [expr $rows-1]  
set cols_1 [expr $cols-1]  
  
# Set the working library and the reference libraries  
rpl::setup::setWorkingLibrary demoRAM  
rpl::setup::setReferenceLibraries [list sram_primitive demoRAM RefLib]
```

In the following section, a simple column circuit is built, called `ioreg`, by placing all instances in a column. Because this block already has a schematic, the layout is prepopulated with all instances and terminals.

Here is the schematic:

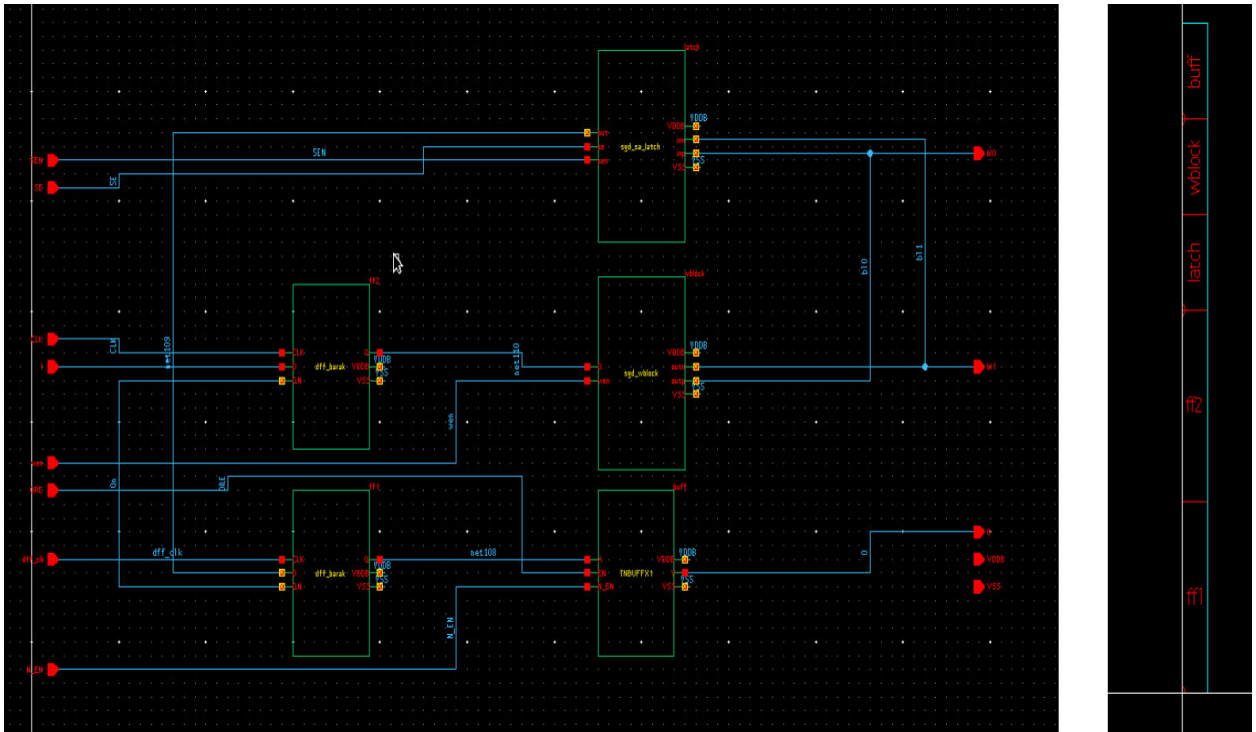


```
# Create new cell
# All instances and terms will be created based on the schematic
# automatically
rpl::createCell ioreg

# Place cells by flipping every other even cells
rpl::pack "ff1 ff2 latch wblock buff" -template "if" -direction up

# Create a PR boundary and close the cell
rpl::createBoundary
rpl::closeCell
```

Here is the cell layout:



Now, you can build a bitcell row to be instantiated into multiple rows. In this example, the schematic does not exist. Each memory cell, terminals and nets must be created by this script. The script is a bit longer but not complex.

```
#####
# Create a cell containing row of bit cells
#####
rpl::createCell bitcell_row${cols}

# Create ports
rpl::createTerm blp<${cols}_1:0> -type inputOutput
rpl::createTerm bln<${cols}_1:0> -type inputOutput
rpl::createTerm wl -type input
rpl::createTerm VDD
rpl::createTerm VSS

# Create bit cell instances
rpl::createInstance "bitcell<${cols}_1:0>" -cellName sgd_bitcell_2fins

# Place the bitcells in a row
rpl::pack "bitcell<${cols}_1:0>" -direction right -template {i f}

# Assign nets to bit cells in the row
for {set c 0} {$c < $cols} {incr c} {
    rpl::createInstPinConnection bitcell<${c}> -pinName wlprim -netName wl
```

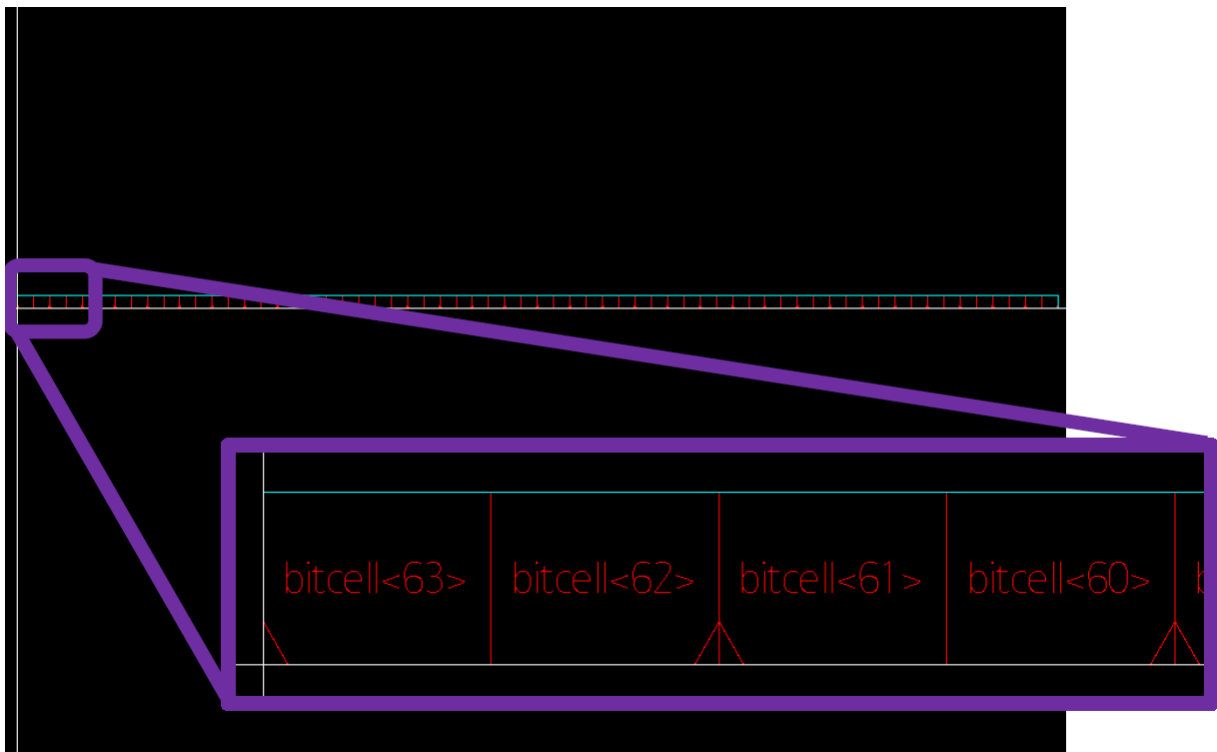
```

        rpl::createInstPinConnection bitcell<$c> -pinName prim0 \
-netName bln<$c>
        rpl::createInstPinConnection bitcell<$c> -pinName prim1 \
-netName blp<$c>
        rpl::createInstPinConnection bitcell<$c> -pinName VDDM -netName VDD
        rpl::createInstPinConnection bitcell<$c> -pinName VSS -netName VSS
    }

    # Create prBoundary, generate schematic, and close cell
    rpl::createBoundary
    rpl::generateSchematic
    rpl::closeCell

```

Here is the layout result. Every other cells are flipped (MY).



The memory array is build of rows of memory slices. Uses `rpl::pack` to pack the slices in the up direction by flipping every other rows.

```

#####
# Create a memory bank or matrix cell
#####
::rpl::createCell bitcell_matrix${rows}X${cols}

# Create ports
rpl::createTerm blp<$cols_1:0> -type inputOutput
rpl::createTerm bln<$cols_1:0> -type inputOutput
rpl::createTerm wl<$rows_1:0> -type input

```

```
rpl::createTerm VDD
rpl::createTerm VSS

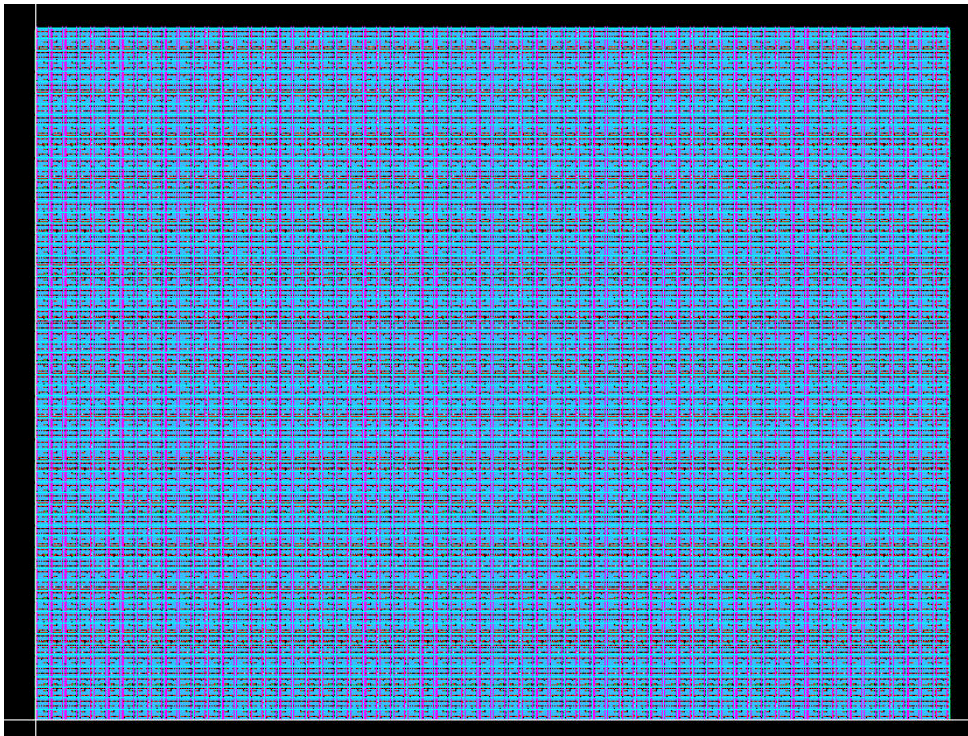
# Create row cell instances
rpl::createInstance "bitcell_row<$rows_1:0>" -cellName bitcell_row${cols}

# Place the row cells in a column
rpl::pack "bitcell_row<$rows_1:0>" -direction up -template "i f"

# Assign nets to row cells
for {set r 0} {$r < $rows} {incr r} {
    for {set c 0} {$c < $cols} {incr c} {
        rpl::createInstPinConnection bitcell_row<$r> -pinName bln<$c> \
        -netName bln<$c>
        rpl::createInstPinConnection bitcell_row<$r> -pinName blp<$c> \
        -netName blp<$c>
    }
    rpl::createInstPinConnection bitcell_row<$r> -pinName wl \
    -netName wl<$r>
    rpl::createInstPinConnection bitcell_row<$r> -pinName VDD \
    -netName VDD
    rpl::createInstPinConnection bitcell_row<$r> -pinName VSS \
    -netName VSS
}

# Create prBoundary, generate schematic, and close cell
rpl::createBoundary
rpl::generateSchematic
rpl::closeCell
```

Here is the layout of the full 64x64 memory array:



A memory block and column decoders are created. You can assemble the whole design.

```
#####
# Assemble the memory
#####
set blockName SRAM_${rows}X${cols}

rpl::createCell $blockName

rpl::createInstance "ioreg<${cols}_1:0>" -cellName ioreg
rpl::createInstance "mem_core" -cellName "bitcell_matrix${rows}X${cols}"

# Place the blocks
rpl::pack "ioreg<${cols}_1:0>" -direction right -name allIORegs
rpl::pack "allIORegs mem_core" -direction up

# Add connections
for {set r 0} {$r < $rows} {incr r} {
    rpl::createTerm wl<${r}>
    rpl::createInstPinConnection mem_core -pinName wl<${r}> -netName wl<${r}>
}
for {set c 0} {$c < $cols} {incr c} {
    rpl::createTerm bln<${c}>
    rpl::createTerm blp<${c}>
    rpl::createTerm CLK
```

```

rpl::createTerm I
rpl::createTerm N_EN
rpl::createTerm O
rpl::createTerm ORE
rpl::createTerm SEN
rpl::createTerm SE
rpl::createTerm VDD
rpl::createTerm VSS
rpl::createTerm dff_clk
rpl::createTerm wen

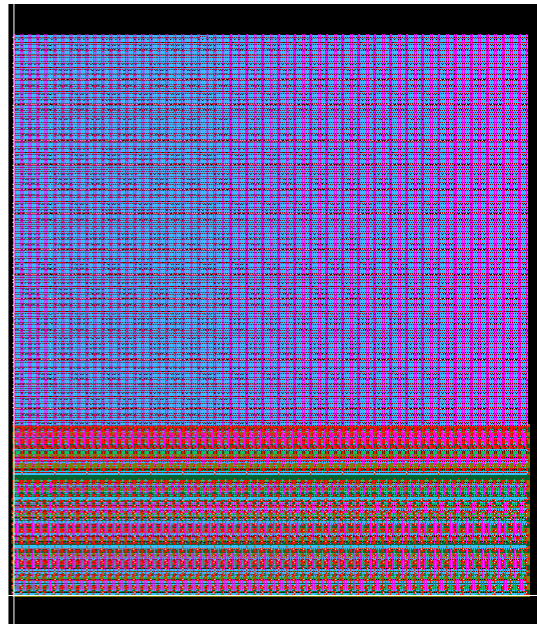
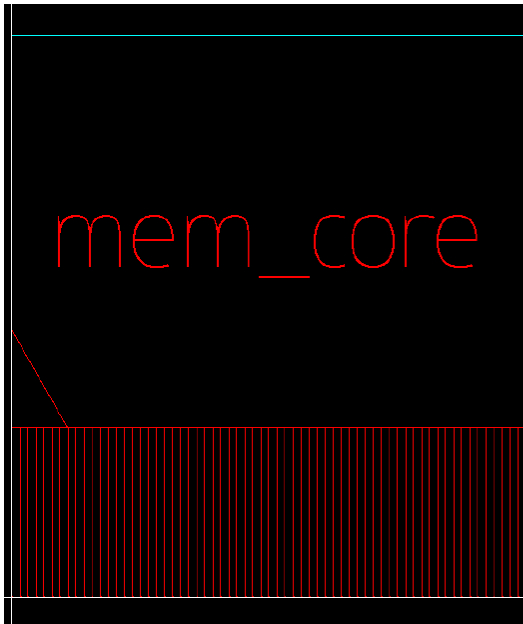
rpl::createInstPinConnection mem_core -pinName bln<$c> \
-netName bln<$c>
rpl::createInstPinConnection mem_core -pinName blp<$c> \
-netName blp<$c>
rpl::createInstPinConnection ioreg<$c> -pinName bl0 -netName bln<$c>
rpl::createInstPinConnection ioreg<$c> -pinName bl1 -netName blp<$c>
rpl::createInstPinConnection ioreg<$c> -pinName CLK -netName CLK
rpl::createInstPinConnection ioreg<$c> -pinName I -netName I
rpl::createInstPinConnection ioreg<$c> -pinName N_EN -netName N_EN
rpl::createInstPinConnection ioreg<$c> -pinName O -netName O
rpl::createInstPinConnection ioreg<$c> -pinName ORE -netName ORE
rpl::createInstPinConnection ioreg<$c> -pinName SEN -netName SEN
rpl::createInstPinConnection ioreg<$c> -pinName SE -netName SE
rpl::createInstPinConnection ioreg<$c> -pinName VDD -netName VDD
rpl::createInstPinConnection ioreg<$c> -pinName VSS -netName VSS
rpl::createInstPinConnection ioreg<$c> -pinName dff_clk \
-netName dff_clk
rpl::createInstPinConnection ioreg<$c> -pinName wen -netName wen
}

# Create prBoundary, generate schematic, and close cell
rpl::createBoundary
rpl::generateSchematic
rpl::closeCell

# Close all opened designs
rpl::closeCell -all true -force true

```

Here is the final layout:



Commands

The commands covered are:

- `rpl::addTracks`
- `rpl::align`
- `rpl::closeCell`
- `rpl::createBoundary`
- `rpl::createCell`
- `rpl::createInstance`
- `rpl::createInstPinConnection`
- `rpl::createLibrary`
- `rpl::createNet`
- `rpl::createTerm`
- `rpl::createTermLabel`
- `rpl::deleteCell`

- [rpl::discardDesigns](#)
- [rpl::error](#)
- [rpl::evaluate](#)
- [rpl::generateSchematic](#)
- [rpl::getTracks](#)
- [rpl::group::assign](#)
- [rpl::group::clear](#)
- [rpl::group::get](#)
- [rpl::pack](#)
- [rpl::param](#)
- [rpl::place](#)
- [rpl::reset](#)
- [rpl::setOrientation](#)
- [rpl::setup::addReferenceLibraries](#)
- [rpl::setup::getCurrentContext](#)
- [rpl::setup::getCurrentDesign](#)
- [rpl::setup::getOpenContexts](#)
- [rpl::setup::getOpenContextsCount](#)
- [rpl::setup::getReferenceLibraries](#)
- [rpl::setup::getWorkingLayoutLibrary](#)
- [rpl::setup::getWorkingLibrary](#)
- [rpl::setup::getWorkingSchematicLibrary](#)
- [rpl::setup::setReferenceLibraries](#)
- [rpl::setup::setWorkingLayoutLibrary](#)
- [rpl::setup::setWorkingLibrary](#)
- [rpl::setup::setWorkingSchematicLibrary](#)
- [rpl::tracks](#)

- [rpl::utils::createContext](#)
- [rpl::utils::deletePinFigs](#)
- [rpl::utils::expandNames](#)
- [rpl::utils::getBbox](#)
- [rpl::utils::getNets](#)
- [rpl::utils::layers::getMetalLayerStack](#)
- [rpl::utils::PRBoundaryFind](#)

rpl::addTracks

Description

Creates global track regions based and activates the specified tracks. The goal is to allow the user to script the track creation easily using a simple list of strings. The command deletes all global track regions before adding the new ones.

The string syntax is on the form of `regionName/groupName/trackName`. The command creates a global track region for each unique `regionName` in the list. Then the command activates the specified `groupName/trackName`. A wildcard character (*) can be used to get all the `groupName` or the `trackName` active.

Examples

Example 5

Creates a track region from the definition `analog_region`. Then it enables all the tracks on all the groups of that newly created global region.

```
rpl::addTracks {analog_region/*/*}
```

Example 6

Creates a track region from the definition `analog_region`. Then it enables all the tracks on M1 and M2 groups.

```
rpl::addTracks {analog_region/M1/* analog_region/M2/*}
```

Example 7

Creates a track region from the definition `analog_region`. Then it enables only the blue tracks on M1 group and the red tracks on M2 group.

```
rpl::addTracks {analog_region/M1/blue analog_region/M2/red}
```

The string supports also a region offset using the following syntax:

```
regionName (xOffset,yOffset) /groupName/trackName
```

where `xOffset` and `yOffset` are floating point numbers.

The option `-preserveExistingGlobalTracks` prevents the deletion of global tracks before creating new ones. The `-points <box>` argument creates a local track region within that `<box>`. Existing tracks are not deleted when adding local track regions. The recommended flow is to create all global tracks in a single `rpl::addTracks` command first. Multiple `rpl::addTracks` commands can then be used for local track regions.

See `le::createGlobalTrackRegion` and `le::createTrackRegion` for the other options.

Syntax

```
rpl::addTracks <trackRegionNames: [string]> [-design <oaDesign>]
[-basename <string>] [-points <box>] [-preserveExistingGlobalTracks
<bool>] [-flipX <bool>] [-flipY <bool>] [-flipFirstX <bool>]
[-flipFirstY <bool>] [-offsetTarget <rplOffsetTargetEnum>]
```

Arguments

`trackRegionNames`

Track region name to create in the current design. It is in the form of a list of `regionName/groupName/trackName`.

`-design`

When not specified (normal use model), the design is the current context opened by `rpl::createCell`.

`-basename`

Sets the basename of the track region. An index number (for example, `_0`) is appended to this basename.

`-points`

Creates a local track region when defined.

`-preserveExistingGlobalTracks`

Do not delete existing global tracks. (Default: false)

`-flipX`

Determines if tiles in the region are flipped about the X-axis. (Default: false)

`-flipY`

Determines if tiles in the region are flipped about the Y-axis. (Default: false)

`-flipFirstX`

Determines if the first tile in the region is flipped about the X-axis. (Default: false)

`-flipFirstY`

Determines if the first tile in the region is flipped about the Y-axis. (Default: false)

`-offsetTarget`

Sets this Offset Target for local track regions. Valid values: region, design
(Default: region)

Syntax

```
rpl::addTracks <trackRegionNames: [string]> [-design <oaDesign>]  
[-basename <string>] [-points <box>] [-preserveExistingGlobalTracks  
<bool>] [-flipX <bool>] [-flipY <bool>] [-flipFirstX <bool>]  
[-flipFirstY <bool>] [-offsetTarget <rplOffsetTargetEnum>]
```

Returns

`<none>`

Arguments

`trackRegionNames`

Global Track Region name to create in the current design. It is in the form of a list of 'regionName/groupName/trackName'

`-design`

When not specified (normal use model), the design is the current context opened by `rpl::createCell`

`-basename`

Sets the basename of the track region. An index number, like `_0`, will be append to this basename

`-points`

Creates a local track region when defined

`-preserveExistingGlobalTracks`

Do not delete existing global tracks (Default is: false)

`-flipX`

Determines if tiles in the region are flipped about the X-axis (Default is: false)

`-flipY`

Determines if tiles in the region are flipped about the Y-axis (Default is: false)

`-flipFirstX`

Determines if the first tile in the region is flipped about the X-axis (Default is: false)

`-flipFirstY`

Determines if the first tile in the region is flipped about the Y-axis (Default is: false)

`-offsetTarget`

Set this Offset Target for local track regions (region | design) (Default is: region)

rpl::align

Description

Aligns instances and figGroups. Hierarchical instance or figGroup names use the / or . (slash or dot) as a hierarchy delimiter. The option `-moveInHierarchy` forces `rpl::align` to move the instance or figGroup inside the hierarchy when this option sets to true. When this option sets to false, the top level object is moved using the object in the hierarchy as a reference.

Syntax

```
rpl::align [-design <oaDesign>] -sources <string> -target <string>
           -sourceEdge <rplAlignEdgeEnum> -targetEdge <rplAlignEdgeEnum>
           [-direction <rplAlignDirectionEnum>]
           [-moveInHierarchy <bool>] [-considerMustJoinPins <bool>]
```

Returns

<none>

Arguments

`-design`

When not specified (normal use model), the design is the current context opened by `rpl::createCell`

`-sources`

List of instance names, figGroup names or terminals to align to the target.

`-target`

Target instance, figGroup or terminal (reference)

`-sourceEdge`

Source edge (right | left | center | origin | top | bottom)

`-targetEdge`

Target edge (right | left | center | origin | top | bottom)

`-direction`

Direction (vertical | horizontal)

`-moveInHierarchy`

When set to true, align moves instances/figGroups inside the hierarchy. When false, align moves the top-level instances/figGroups based (Default is: 0)

`-considerMustJoinPins`

Consider must join pins when terminal is specified (Default is: 0)

Details on Instance Names

For the `-sources` argument, you can specify a set of instance or figGroup names as input. But for the `-target` argument, only one instance or figGroup name is allowed. Hierarchical names can be specified by using the symbol `/` as a hierarchy delimiter. For example, an instance name `M0/M1` indicates an instance named M1 inside instance M0. There is no limitation on the level of hierarchy.

Details on Alignment Protocol

When specifying `-sourceEdge` and `-targetEdge` arguments, point of source instance or figGroup is only acceptable to align to point of target instance or figGroup. (For example, center to center or origin to origin.)

The edge of source instance or figGroup is also only acceptable to align edge of target instance or figGroup. Moreover, vertical and horizontal edges cannot align with each other. (For example, right/left source edge cannot align to top/bottom target edge.)

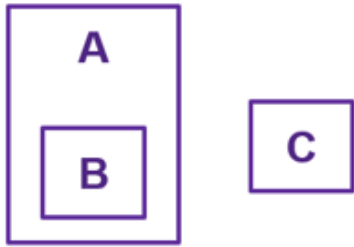
When performing point alignment (center or origin), the argument `-direction` is mandatory to decide the align direction. Otherwise, the argument is ignored. If `-direction` is set to vertical, instances are aligned vertically according to the specified point. If `-direction` is set to horizontal, source instances or figGroups are moved to align horizontally.

Details on Move in Hierarchy

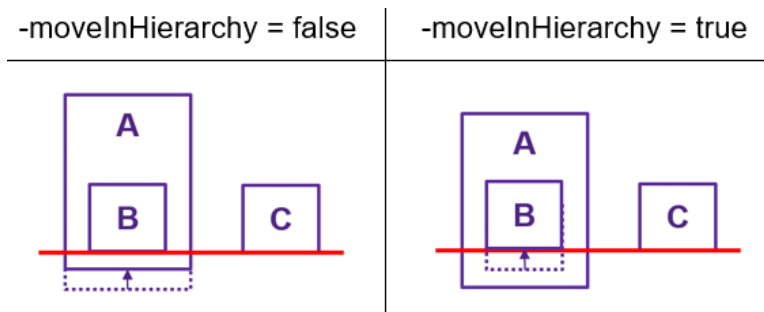
The default of `-moveInHierarchy` is false, which means only top-level instances or figGroups are moved for alignment. When setting the value to true, instances or figGroups inside hierarchy are moved instead of top-level instances or figGroups.

Examples

```
rpl::pack -source "A/B" -sourceEdge bottom -target C -targetEdge bottom
```



When specifying `-moveInHierarchy`:



rpl::closeCell

Description

Saves and closes the cell layout view under the current context. The context stack is displayed. If the `-cellName` is specified, the command closes all opened cells in the context stack.

Syntax

```
rpl::closeCell [<cellName: string>] [-libName <string>] \
    [-viewName <string>] [-all <bool>] [-force <bool>]
```

Arguments

`cellName`

Name of the cell to close. If not specified, the command closes the active design in context. (Default: "")

`-libName`

Specifies the library name. Uses the working library if not defined.

`-viewName`

Cellview name to create. (Default: layout)

`-all`

Closes all opened cells. (Default: false)

`-force`

Forces to close all opened cells. Works only with `-all`. (Default: false)

Returns

<none>

rpl::createBoundary

Command to create a prBoundary based on the bBox of all instance prBoundaries. The command creates a boundary based on the preference lInstBoundaryLPP if defined at the design context scope. The option `-boundaryLPP` forces the use of this LPP. prBoundary oaObject is the default boundary if LPP is not defined

Syntax

```
rpl::createBoundary [<instanceNames: string>] [-design <oaDesign>]  
  [-orig <point>] [-width <float>] [-height <float>] [-boundaryLPP  
  <string>] [-useLPPWhenDefined <bool>] [-reportInstOutsideBoundary  
  <bool>] [-forceOAObject <bool>]
```

Returns

<PRBoundary> : prBoundary

Arguments

`instanceNames`

List of instance names to consider. All instances are consider if not specified (Default is:)

`-design`

When not specified (normal use model), the design is the current context opened by `rpl::createCell`

`-orig`

Origin of the prBoundary (Default: 0 0)

`-width`

Width of the prBoundary

`-height`

Height of the prBoundary

`-boundaryLPP`

Defines the boundary LPP. This option superseeds the preference `lclInstBoundaryLPP` if defined. It also forces the use of this `boundaryLPP` and ignores the `-useLPPWhenDefined` option (Default is:)

`-useLPPWhenDefined`

Use the LPP defined by preference `lclInstBoundaryLPP` if defined. Object `prBoundary` is created otherwise (Default is: 1)

`-reportInstOutsideBoundary`

Check if instances are outside of the boundary and issue an error if such found (Default is: false)

`-forceOAObject`

Create a `prBoundary` even if an LPP is defined (Default is: true)

rpl::createCell

Description

Creates a new layout cellview. If a layout cellview exists already, then the command opens it. If the cell has a schematic view, the layout is opened in the SDL mode and the layout is generated and updated automatically to match the instances and connectivity of the schematic.

Details on Alignment Protocol

The command depends on the availability of the schematic and the layout view of the given cell. The table shows the various conditions:

	Without Schematic View	With Schematic View
Without Layout View	When no views are available, the command creates the cell and the layout cell view automatically.	A new layout view is created. The layout is in SDL mode. All instances are placed at (0,0)/R0 to ensure a known starting point.
With Layout View	Layout view is opened. No changes to the layout are done. Instances are not moved or changed.	Layout view is opened. No changes to the layout are done. Instances are not moved or changed.

In all cases, the new context is added to the context stack. Nested `rpl::createCell` can be used to add more and more context to the stack. All RPL commands works on

the topmost cell of the stack. The context can be closed by using `rpl::closeCell`. The `-openInCanvas` option opens the design context in the Layout Editor. Otherwise, the context is created in memory only. Contexts are opened with view depth = 32. This command may open the same cell more than once on the same cell. Multiple contexts are placed on the context stack.

Syntax

```
rpl::createCell <cellName: string> [-libName <string>] [-schematicLibName  
<string>] [-viewName <string>] [-isLayoutOnlyCell <bool>] [-openInCanvas  
<bool>
```

Arguments

`cellName`

Name of the cell to close.

`-libName`

Name of the library. Using the working library if not defined.

`-schematicLibName`

Name of the schematic library. Use the schematic working library if not defined.

`-viewName`

Cellview name to create. (Default: layout)

`-isLayoutOnlyCell`

Creates a layout-only cell. (Default: false)

`-openInCanvas`

Opens the design in a GUI canvas. The default is to open in headless mode.
(Default: false)

Returns

`<dmCellView>`: DM CellView.

rpl::createInstance

Description

Creates a new instance in the design. Layout-only instances can be created using this command as well as instances that require connections. Instance names are expanded as they could be vectored instances. For example: `inv<3:0>`

A property named `rplCreated` is added to the newly added instances. The library of the cell is coming from the list of reference libraries. The command looks up for the `cellName` until a first match. Throws an error if not found.

Syntax

```
rpl::createInstance <instName: string> -cellName <string> [-design  
  <oaDesign>]  
  [-libName <string>] [-viewName <string>]
```

Arguments

`instName`

Name of the instance(s). Also supports iterated and vectored instances. For instance: `inv<7:0>`

`-cellName`

Name of the cell to instantiate

`-design`

When not specified (normal use model), the design is the current design opened by `rpl::createCell`

`-libName`

Name of the library of the cell to instantiate

`-viewName`

Name of the cellview of the cell to instantiate

Returns

`<oaInst>`: Instance object.

rpl::createInstPinConnection

Description

Adds nets to instance pins. It should be used in schematic-less layout creation.

Syntax

```
rpl::createInstPinConnection <instName: string> [-design <oaDesign>]  
  -pinName <string> -netName <string>
```

Arguments

`instName`

Specifies the instance Name.

-design

Design to create the terminal.

-pinName

Specifies the name on the instance pin.

-netName

Specifies the net name to connect the pin.

Returns

<none>

rpl::createLibrary

Description

Creates a new library and attaches a technology library to the newly created library.

Syntax

```
rpl::createLibrary <libName: string> [-techLibName <string>] [-path  
  <string>]
```

Arguments

libName

Name of the library to be created.

-techLibName

Technology library name. Using the default techLib if not defined.

-path

Directory path of the new library. (Default: "")

Returns

<oaLib> : Library object.

rpl::createNet

Description

Creates a new net in the design that does not connect to an existing terminal.

Syntax

```
rpl::createNet <netName: string> [-design <oaDesign>]
```

Arguments

`netName`

Specifies the name of the net.

`-design`

Design to create a net.

Returns

`<oaNet>` : Net object.

rpl::createTerm

Description

Creates a logical terminal in the layout. A new net of the same name is created at the same time.

Syntax

```
rpl::createTerm <termName: string> [-design <oaDesign>] [-type <string>]
```

Arguments

`termName`

Specifies the name of the terminal.

`-design`

Design to create the terminal.

`-type`

Specifies the type of the terminal.

Valid values: input, inputOutput, jumper, output, switch, tristate, unused (Default: inputOutput)

Returns

`<oaTerm>` : Newly created terminal object.

rpl::createTermLabel

Description

Creates labels on terminal pinFig .

Syntax

```
rpl::createTermLabel [<termName: string>] [-layer <string>] [-purpose  
  <string>] [-height <float>] [-orient <rplCreateTermLabelEnum>]
```

Arguments

`termName`

Specifies the terminal to add a label. (Default: *)

`-layer`

Layer to generate the label. Use the pinFig layer if not specified. (Default:)

`-purpose`

Layer purpose to generate the label. Use the pinFig LPP if not specified.
(Default:)

`-height`

Text height in user units. (Default: 0.02)

`-orient`

Text orientation (MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90). (Default:
R0)

Returns

<none>

rpl::deleteCell

Description

Deletes a cell in the working library.

Syntax

```
rpl::deleteCell <cellName: string> [-libName <string>]
```

Arguments

`cellName`

Specifies the name of the cell.

`-libName`

Specifies the name of the library.

Returns

<none>

rpl::discardDesigns

Description

Closes all designs and context without saving. It also resets the state of the RPL context stack.

Syntax

```
rpl::discardDesigns
```

Returns

<none>

rpl::error

Description

Error handling based on Tcl preference `rplErrorHandling`.

Syntax

```
rpl::error [<message: string>]
```

Arguments

`message`

Message to print if it needs to stop. (Default: Script stopped on error)

Returns

<string>

rpl::evaluate

Description

Opens the given `cellName` and execute the block of Tcl code.

Syntax

```
rpl::evaluate [-cellName <string>] [-libName <string>] [-viewName  
  <string>] [-readOnly <bool>] -script <string>
```

Arguments

`-cellName`

Cell name to be opened and processed.

`-libName`

Name of the library. Using the working library if not defined.

`-viewName`

Name of the cellView. (Default: layout)

`-readOnly`

Open the design in read-only mode. (Default: false)

`-script`

Block to Tcl code to execute on the cell.

Returns

<none>

Examples

In this example, the cell `myTopDesign` is opened and the code in the script block is executed on it (which routing the whole design). The context and the design is saved and closed at the end of the process.

```
rpl::evaluate -cellName myTopDesign -script {  
    set design [rpl::setup::getCurrentDesign]  
    set context [rpl::setup::getCurrentContext]  
    rt::route -design $design  
}
```

rpl::generateSchematic

Description

Generates the schematic and symbol views based on the layout view. No schematic is created if the cell has no nets or the cell has the `lxIgnore` property set to true (layout-only cell).

Syntax

```
rpl::generateSchematic [-design <oaDesign>] [-overwrite  
    <rplGenerateSchematicOverwriteEnum>]
```


Arguments

`-design`

Design on which to generate the schematic.

`-overwrite`

Controls schematics and symbols overwrite if targets exist.

Valid values: false, schematicsOnly, true (Default: false)

Returns

`dmCellView` | `{<dmCellView>}` : Generated cell views.

`rpl::getTracks`

Description

Prints the global tracks created in the design. The printout is in the form to be used by `rpl::addTracks` directly. For example, uses the following command to recreate tracks:

```
rpl::addTracks {  
  CORE_M1M2_G010/M2_w22p132/*  
  analogGateLength146/M1_w60p240/*  
  analogGateLength146/M4_w60p250/w60o125_blue  
}
```

If the `-outputFormat` is `list`, the command returns a list of lists of all track regions, local and global, in the form of key-value pairs. The key names are named as the arguments for the `rpl::addTracks` command.

Syntax

```
rpl::getTracks [-design <oaDesign>] [-outputFormat  
  <rplGetTracksOutputEnum>]
```

Arguments

`-design`

When not specified (normal use model), the design is the current context opened by `rpl::createCell`.

`-outputFormat`

Method of outputting the tracks. The `print` format prints the information in the Console. This command returns a list of tracks used when `-outputFormat` is `list`. Valid values: `print` or `list` (Default: `print`)

Syntax

```
rpl::getTracks [-design <oaDesign>] [-outputFormat  
  <rplGetTracksOutputEnum>]
```

Returns

<string> : Nothing when 'print', list of tracks when 'list' option is selected

Arguments

-design

When not specified (normal use model), the design is the current context opened by rpl::createCell

-outputFormat

Method of outputting the tracks. The 'print' format prints the information in the Console. This command returns a list of tracks used when outputFormat is 'list'. (print | list) (Default is: print)

rpl::group::assign

Description

This command creates virtual groups by name. This command will not create a figGroup or any other Open Access type of objects.

Syntax

```
rpl::group::assign [<groupName: string>] -groupList [<string>]
```

Arguments

groupName

Group name.

-groupList

List on instance names.

Returns

<string> : Group name.

rpl::group::clear

Description

Clears all the grouped instances.

Syntax

```
rpl::group::clear [<groupName: string>]
```

Arguments

groupName

Optional group name to be deleted.

Returns

<none>

rpl::group::get

Description

This command retrieves the list of instance names by group name.

Syntax

```
rpl::group::get [<groupName: string>]
```

Arguments

groupName

Group name.

Returns

<string> : List of instances in the named group.

rpl::pack

Description

Packs instances in a precise order. Packed instances are placed in a figGroup. The command returns the figGroup to allow nested `rpl::pack` commands.

Syntax

```
rpl::pack <insts: [string]> [-design <oaDesign>] [-template <string>]  
[-direction <rplPackDirectionEnum>] [-align <rplPackAlignNum>] [-padLow  
<float>] [-padHigh <float>] [-absoluteSize <float>] [-x <float>] [-y  
<float>] [-name <string>] [-stopOnError <bool>]
```

Arguments

`insts`

List of instances and figGroup names.

`-design`

Design on which to pack.

`-template`

Packing directive to pack the objects. (Default: i)

`string` is a series of characters to represent the flipping and the spacing for each instance. Each character can be complemented by a number to multiply the character. Supported characters are:

- i: instance (non-flipped version).
- f: flipped in the orientation of the pack command.
- m: mirrored (flipped in the other packing direction).
- r: reflected (both flipped and mirrored).
- sn: space by n, where n can be a real number.

For example: if the user input is {ifs0.3i3f4}, the tool converts the input to be {i f s 0.3 i i i f f f f}.

- Instance 1 placed as is (not moved nor flipped).
- Instance 2 abutted and flipped.
- Adding a 0.3 um spacing.
- Instance 3, 4, and 5 abutted as is.
- Instance 6, 7, 8, and 9 abutted and flipped.

If the list of instances is greater than the template string, the command recycles the template string from the beginning. For example, if the template is {if} and 4 instances are packed by the command, the first instance is placed normally and the second one flipped. The third instance has no template directive therefore will restart the template at the beginning and be placed as is. The forth instance will be flipped. All characters are case-sensitive.

`-direction`

Direction of the packing.

Valid values: right, left, up, down (Default: right)

`-align`

Align mode for instances.

Valid values: origin, center, high, low, top, bottom, left, right (Default: origin)

`-padLow`

Adds padding to the left or bottom of the first instance. (Default: 0.0)

`-padHigh`

Adds padding to the right or top of the last instance. (Default: 0.0)

`-absoluteSize`

Adds padding to meet an overall absolute size in the padding direction. (Default: 0.0)

`-x`

Moves the group origin to the X value.

`-y`

Moves the group origin to the Y value.

`-name`

Group name to be created. A default name is generated automatically if `-name` is not provided. (Default: "")

`-stopOnError`

Stops if an instance is specified in the list of instances but missing from the design. If this argument is false, the `rpl::pack` command reports and continues. (Default: true)

Returns

<string>: Returns the figGroup name.

Details of Created figGroup

A figGroup is created with the specified name, and all packed instances are placed under the created figGroup.

A property with name `rplPadding` is appended to the figGroup in a format:

```
{left_padding, bottom_padding} {right_padding, top_padding}
```

The values of property are determined by spacing value in template string. When option `-padLow` or `-padHigh` is given, they overwrite spacing value in the original template string.

Examples

Example 8

```
rpl::pack "A" -template "s0.2 i s0.6" -direction right
```

Placement result = [0.2 A]

Property(rplPadding) = {0.2 0} {0 0}

Example 9

```
rpl::pack "A" -template "s0.2 i s0.6" -direction right -padLow 0.3  
-padHigh 0.5
```

Placement result = [0.3 A 0.5]

Property(rplPadding) = {0.3 0} {0.5 0}

Example 10

```
rpl::pack "A B" -template "s0.2 i s0.6" -direction right -padLow 0.3  
-padHigh 0.5
```

Placement result = [0.3 A 0.6 0.2 B 0.5]

Property(rplPadding) = {0.3 0} {0.5 0}

rpl::param

Description

Simple lookup dictionary utility.

Syntax

```
rpl::param <command: rplPackParamEnum> [<paramName: string>] [<paramVal:  
string>]
```

Arguments

command

Specifies command name.

Valid values: set, get, reset, exist

paramName

Specifies parameter name.

paramVal

Specifies parameter value.

Returns

<string> : Returns the param value.

rpl::place

Description

Places instances at a specific location. The instance lower or left corner is placed at the `-point` location.

Syntax

```
rpl::place <insts: [string]> [-design <oaDesign>] -point <point>
```

Arguments

`insts`

List on instances.

`-design`

When not specified (normal use model), the design is the current context opened by `rpl::createCell`.

`-point`

Point to move objects. For example, `-point 1.0 2.0`.

Returns

<none>

rpl::reset

Description

Resets the instance orientation and location to 0,0.

Syntax

```
rpl::reset <insts: [string]> [-design <oaDesign>]
```

Arguments

`insts`

List on instances.

`-design`

Design.

Returns

<none>

rpl::setOrientation

Description

Specifies the orientation of the given instances. Orientations are relative. This command does not move the overall prBoundary bBox of the instances.

Syntax

```
rpl::setOrientation <insts: [string]> [-design <oaDesign>] [-orient  
  <rplSetOrientationEnum>]
```

Arguments

insts

List on instances.

-design

Design.

-orient

Orientation.

Valid values: R0, R90, R180, R270, MX, MY, MXR90, MYR90 (Default: R0)

Returns

<none>

rpl::setup::addReferenceLibraries

Description

Adds the reference libraries used by the `rpl::createInstance` command. This command needs to know where to find cells like stdcells, tap cells, and blocks in other libraries. This setting is global to the session.

Syntax

```
rpl::setup::addReferenceLibraries <libNames: [string]>
```


Arguments

`libNames`

List of library names.

Returns

List of library names.

rpl::setup::getCurrentContext

Description

When a cell is opened using the command `rpl::createCell`, its context is placed on the a stack of contexts. This command returns the top-most context which is the current context.

Syntax

```
rpl::setup::getCurrentContext
```

Returns

<string> : Current context of the opened cell.

rpl::setup::getCurrentDesign

Description

When a cell is opened using the command `rpl::createCell`, its context is placed on the a stack of contexts. This command returns design of the top-most context.

Syntax

```
rpl::setup::getCurrentDesign
```

Returns

<oaDesign> : Current design.

rpl::setup::getOpenContexts

Description

This command returns the list of opened contexts in the current session in the reverse order that they were opened. The newest opened design is first in the list.

Syntax

```
rpl::setup::getOpenContexts
```

Returns

<string> : List of opened contexts.

rpl::setup::getOpenContextsCount

Description

This command returns the number of opened contexts.

Syntax

```
rpl::setup::getOpenContextsCount
```

Returns

<string> : Number of opened contexts.

rpl::setup::getReferenceLibraries

Description

Gets the reference libraries used by the `rpl::createInstance` command. This command needs to know where to find cells like stdcells, tap cells, and blocks in other libraries. This setting is global to the session.

Syntax

```
rpl::setup::getReferenceLibraries
```

Returns

List of library names.

rpl::setup::getWorkingLayoutLibrary

Description

Specifies on which library to operate the RPL commands on.

This command is used when the schematic and the layout are located in two different libraries.

Syntax

```
rpl::setup::getWorkingLayoutLibrary
```

Returns

Working library name.

rpl::setup::getWorkingLibrary

Description

Specifies on which library to operate the RPL commands on.

This command is used when the schematic and the layout are in the same library.

Syntax

```
rpl::setup::getWorkingLibrary
```

Returns

Working library name.

rpl::setup::getWorkingSchematicLibrary

Description

Specifies on which library to operate the RPL commands on.

This command is used when the schematic and the layout are located in two different libraries.

Syntax

```
rpl::setup::getWorkingSchematicLibrary
```

Returns

Working library name.

rpl::setup::setReferenceLibraries

Description

Sets the reference libraries used by the `rpl::createInstance` command. This command needs to know where to find cells like stdcells, tap cells, and blocks in other libraries. This setting is global to the session.

Syntax

```
rpl::setup::setReferenceLibraries <libNames: [string]>
```

Arguments

`libNames`

List of library names.

Returns

List of library names.

rpl::setup::setWorkingLayoutLibrary

Description

Specifies on which library to operate the RPL commands on.

This command is used when the schematic and the layout are located in two different libraries.

Syntax

```
rpl::setup::setWorkingLayoutLibrary <libName: string>
```

Arguments

`libName`

Library name.

Returns

Working library name.

rpl::setup::setWorkingLibrary

Description

Specifies on which library to operate the RPL commands on.

This command is used when the schematic and the layout are in the same library.

Syntax

```
rpl::setup::setWorkingLibrary <libName: string>
```

Arguments

`libName`

Library name.

Returns

Working library name.

rpl::setup::setWorkingSchematicLibrary

Description

Specifies on which library to operate the RPL commands on.

This command is used when the schematic and the layout are located in two different libraries.

Syntax

```
rpl::setup::setWorkingSchematicLibrary <libName: string>
```

Arguments

libName

Library name.

Returns

Working library name.

rpl::tracks

Description

Template command.

Syntax

```
rpl::tracks [<design: oaDesign>] (-addTrackSet <string> | -addTrackNames  
<string> | -listTrackSets <bool> | -getCurrentTracks <bool>)
```

Arguments

design

Design. (Default: current context design)

-addTrackSet

Track set.

-addTrackNames

Track names (Regions/Groups/Tracks).

`-listTrackSets`

Lists the available track sets.

`-getCurrentTracks`

Returns the tracks that are used in the design.

Returns

<string>

rpl::utils::createContext

Description

Opens a new context of a Library/Cell/View in the Layout Editor canvas.

Syntax

```
rpl::utils::createContext <libCellView: string> [-readOnly <bool>]  
[-openInCanvas <bool>]
```

Arguments

`libCellView`

Specifies libName/cellName/viewName. Use / as delimiters.

`-readOnly`

Opens the context in read-only. (Default: false)

`-openInCanvas`

Opens the design in the editor canvas. (Default: false)

Returns

<deContext> : Return lcv context.

rpl::utils::deletePinFigs

Description

Deletes the pin figures without deleting the terms.

Syntax

```
rpl::utils::deletePinFigs [<netNames: string>] [-design <oaDesign>]
```

Arguments

`netNames`

List of net names to delete the terms from. (Default:)

`-design`

Design

Returns

`<deContext>` : Return lcv context.

rpl::utils::expandNames

Description

Expand the names of buses or array of instances. Directives are:

- `*` (asterisk): all.
- `:` (colon): range of bits. For example, `in<2:0>` is expanded to `in<2>` `in<1>` `in<0>`.
- `()` (curly brackets): non vector matching. For example, `inv(0:2)` is expanded to `inv0`, `inv1`, and `inv2`.
- `,` (comma): separate list of single items. For example, `inv(0,3)` is expanded to `inv0` and `inv3`.

The directives can be combined. For example, `inv<8,6,3:0>` is expanding to `{inv<8>` `inv<6>` `inv<3>` `inv<2>` `inv<1>` `inv<0>}`.

This command does not do any design lookup. It is simply expanding the input strings into a list of unique names.

Syntax

```
rpl::utils::expandNames <names: string>
```

Arguments

`names`

Name.

Returns

`<string>` : Expanded names.

rpl::utils::getBbox

Description

Returns the BBox of an object based on the prBoundary of the instance, the figGroup, or the design. The BBox of a figGroup is the combined BBox of each individual instances in that group.

Syntax

```
rpl::utils::getBbox <object: string>
```

Arguments

object

Objects.

Returns

<string> : Box of the object: {x0 y0} {x1 y1}.

rpl::utils::getNets

Description

Returns a dbCollection of oaNets based on the net names. The net names can be in the form of the inv<8,6,3:0>, where six nets are described in this string. The command returns a collection these six nets at once.

Syntax

```
rpl::utils::getNets [<netNames: string>] [-of <oaDesign>] [-verbose  
  <bool>]
```

Arguments

netNames

List of net names. (Default:)

-of

Design from which to collect nets.

-verbose

Issue warning if the net does not exists. (Default: true)

Returns

<<oaNet>>

rpl::utils::layers::getMetalLayerStack

Description

This function returns the real layer metal stack from the lowest layer to the top metal layer. Backside metal layers appear below POLY layers.

Syntax

```
rpl::utils::layers::getMetalLayerStack <design: oaDesign> [-includePoly  
  <bool>] [-includeContactLessMetal <bool>]
```

Arguments

design

Design to get the layers from.

-includePoly

Include poly layers. (Default: false)

-includeContactLessMetal

Include contact-less metal layers. (Default: false)

Returns

<string>

rpl::utils::PRBoundaryFind

Description

Function that returns the PRBoundary of an object or its equivalent. It returns the BBox when a design has a layer-based boundary shape.

Syntax

```
rpl::utils::PRBoundaryFind <object: string>
```

Arguments

object

Objects.

Returns

<string> : PRBoundary of an object or its equivalent.

Preferences

The table lists rpl namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 173 *Relative Placement Language Preferences*

Name	Type	Scope	Default	Description
rplErrorHandling	enCCAE rrorHand ling	project	stopOnError	Indicates how to handle errors. Valid values: stopOnError, continueOnError
rplVerboseMode	bool	project	false	Enables the extra information to be printed in the Console.

21

Routing Tcl

For details, see

- [Commands](#)
- [Preferences](#)
- [Route Pattern Description Reference](#)

Commands

The commands covered are

- [rt::autoGenerateTrunk](#)
- [rt::cdIsRoute](#)
- [rt::checkPostRoute](#)
- [rt::checkPreRoute](#)
- [rt::checkRoute](#)
- [rt::checkRoutingStyle](#)
- [rt::checkTopology](#)
- [rt::connectCellRails](#)
- [rt::createFreeFormTopology](#)
- [rt::createGroupPattern](#)
- [rt::createP2PRoute](#)
- [rt::createPatternRoute](#)
- [rt::createPowerStaples](#)
- [rt::createPowerStripes](#)
- [rt::createTrunkTopology](#)

- [rt::createUniblock](#)
- [rt::createViaLadder](#)
- [rt::deleteCustomRouteShapes](#)
- [rt::deleteShield](#)
- [rt::deleteTopology](#)
- [rt::deleteTrunkAndShield](#)
- [rt::deleteTrunkBox](#)
- [rt::estimateRoute](#)
- [rt::exportTopology](#)
- [rt::extractShielding](#)
- [rt::generateTrunk](#)
- [rt::getStrappableNets](#)
- [rt::getTopologies](#)
- [rt::getTopologyInfo](#)
- [rt::getTopologyNets](#)
- [rt::importTopology](#)
- [rt::insertCutMetalPatch](#)
- [rt::insertVias](#)
- [rt::powerHookup](#)
- [rt::realizeGlobalRoute](#)
- [rt::reportSimilarity](#)
- [rt::route](#)
- [rt::routeHierarchy](#)
- [rt::shield](#)
- [irt::addStripe](#)
- [irt::autoRoute](#)
- [irt::autoShield](#)

- [irt::createRoute](#)
- [irt::createP2PShield](#)
- [irt::createRoute](#)
- [irt::createRoutingArea](#)
- [irt::createTopology](#)
- [irt::deleteCustomRouteShapes](#)
- [irt::deleteMesh](#)
- [irt::deleteShield](#)
- [irt::generateMeshRoute](#)
- [irt::generateTrunk](#)
- [irt::previewPattern](#)
- [irt::query](#)

rt::autoGenerateTrunk

Generates trunk routes automatically.

Syntax

```
rt::autoGenerateTrunk (-design <oaDesign> | -nets <{<oaNet>}>)  
[-topTrunkLayer <string>] [-insertVia <bool>] [-mode <string>]  
[-minTopTrunkLength <float>] [-maxTrunkLevel <unsigned int>] [-align  
  <enum>]  
[-startSide <enum>] [-trimLengthLow <float>] [-trimLengthHigh <float>]
```

Returns

{<oaFig>} : Collection of created objects

Arguments

-design

The design in which the trunk routes are added.

-nets

Collection of nets to route

-topTrunkLayer

Specifies the top trunk layer

`-insertVia`

Adds vias between overlapping trunks (Default: true)

`-mode`

Specifies the mode to use when generating trunks:

- `clean`: Honors non-default rules, avoids obstructions, but allows opens. The clean mode avoids shorts and does not violate spacing rules.
- `dirty`: Avoids opens, but allows shorts and violations between trunks or with obstructions.

(Default: clean)

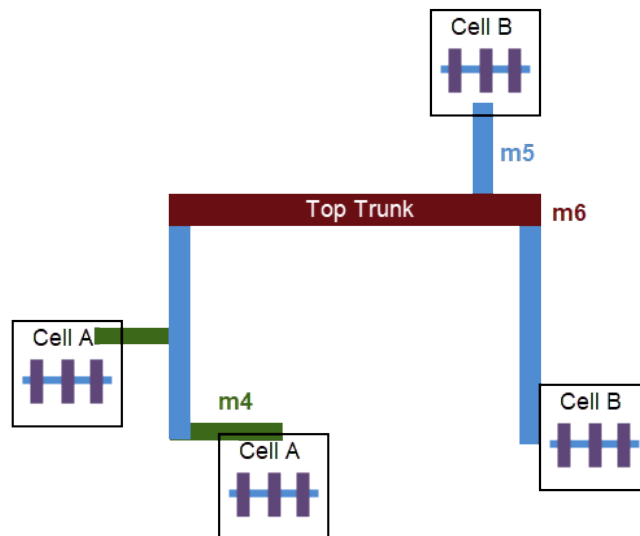
`-minTopTrunkLength`

Specifies the minimum top-trunk length.

`-maxTrunkLevel`

Specifies the maximum number of trunk levels permitted. For example, if you specify 1, trunk routes are created only on one layer. If you specify 2, trunk routes are created on two adjacent layers.

In the following example, `-topTrunkLayer` is m6 and `-maxTrunkLevel` is set at 3, so trunk routes are created on m6, m5, and m4.



`-align`

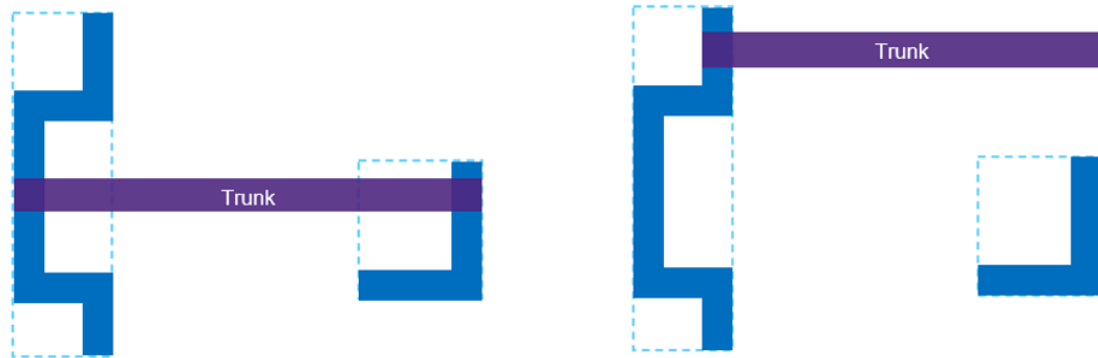
Aligns top-level trunk to the specified pins: { all | driver | receiver }.

`-startSide`

Starting point where the trunk generation looks for a valid location for trunk generation: ({ auto | center | gravity | high | low })

`-trimLengthLow`

The length the trunk extends (< 0) or shortens (> 0) to cover a non-rectangular pin shape when the trunk overlaps with the bounding box of the lowest or left-most pin. If you do not specify a value, the trunk extends to the farthest pin shape.



`-trimLengthHigh`

The length the trunk extends (< 0) or shortens (> 0) to cover a non-rectangular pin shape when the trunk overlaps with the bounding box of the highest or right-most pin. If you do not specify a value, the trunk extends to the farthest pin shape.

rt::cdlsRoute

CDLS Route Command

Syntax

```
rt::cdlsRoute (-design <oaDesign> | -nets <{<oaGroup>} | {<oaNet>}>)
```

Returns

<none>

Arguments

`-design`

Route all nets in this oaDesign

-nets

Collection of oaNets or oaGroups to route

rt::checkPostRoute

Runs specified checks on 0x route shapes and generates summary and detailed reports (checkPostRoute.csv). This command is primarily for digital and block-level designs.

Syntax

```
rt::checkPostRoute [-design <oaDesign>] [-nets [<string>]]  
[-trunkNetOnly <bool>] [-ratio <bool>] [-ratioTolerance <int>]  
[-includeTrunk <bool>] [-minOptimalLength [<float>]] [-ndrLayer <bool>]  
[-redundant <bool>] [-jumper <bool>] [-jumperThreshold <float>]  
[-layerAboveTrunk <bool>] [-viaLadder <bool>] [-viaLadderSpecFile  
  <string>]  
[-report <string>] [-error <string>] [-maxErrorMessage <int>]
```

Arguments

-design

The design to check. Default: current_design

-nets

Nets to be checked. Default: all_nets

-trunkNetOnly

Checks only nets that contain trunks. Default: off

-ratio

Checks the routing ratio and opens.

For routing ratios, the command reports errors if:

$$\frac{((\text{actual length} - \text{optimal length}))}{(\text{optimal length})} > \text{ratioTolerance}$$

Length is measured as follows:

- Pin connections: Length is measured from the edge of the pin.
- Via connections: Length is measured from the center of the via.
- Trunk connections: Length is measured from the center of the trunk.

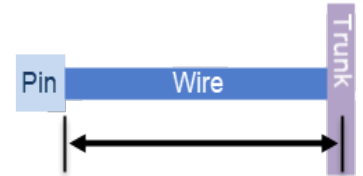
Pin Connection



Via Connection



Trunk Connection



The check applies to pin-to-pin, pin-to-trunk, and trunk-to-trunk connections.

Default: on

`-ratioTolerance`

Sets the routing ratio tolerance, in percent. Default: 20

`-includeTrunk`

Includes trunks in length calculations. Default: off

`-minOptimalLength`

Defines the threshold value (μm) for displaying the “exceed route ratio” error message. The Postroute Checker will display the error message if an optimal length value in the calculated route ratio (actual/optimal length) is greater than the threshold. Default: 0

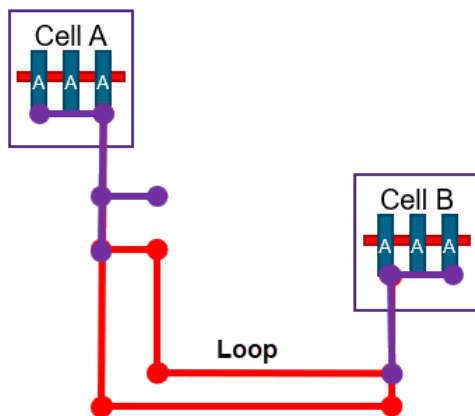
`-ndrLayer`

Checks for shape layers that are not within the non-default-rule range. The check applies to pin-to-pin and pin-to-trunk connections.

Default: off

`-redundant`

Checks for redundant connections that result in loops in a given net.



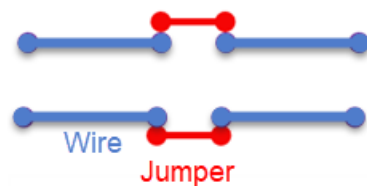
The check applies to pin-to-pin, pin-to-trunk, and trunk-to-trunk connections.

Default: off

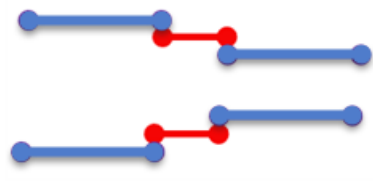
`-jumper`

Checks jumper connections. Reports violations if jumper wire lengths are shorter than `jumperThreshold`, or if the two ends of the jumper wire are not both above or below the connecting wires. The check applies to pin-to-pin, pin-to-trunk, and trunk-to-trunk connections.

Valid Jumper Connections



Not Valid Jumper Connections



Default: off

`-jumperThreshold`

Sets jumper wire length threshold. Default: $10 \times \text{minWidth}$

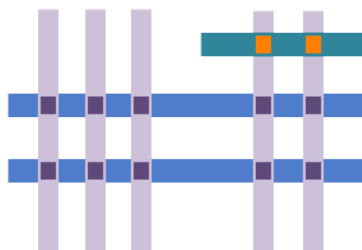
`-layerAboveTrunk`

Checks for shapes that are on layers higher than the trunk layer. The check applies to pin-to-pin, pin-to-trunk, and trunk-to-trunk connections. Default: off

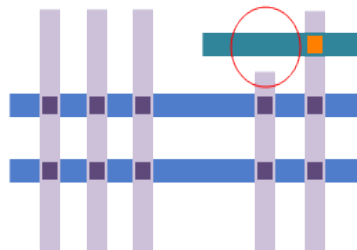
`-viaLadder`

Checks for vias that do not comply with the definitions in the via ladder specifications file (`-viaLadderSpecFile`). Also checks for missing paths, missing vias, and duplicate vias. The checks are performed only if the file is specified, and the file lists the net that you are checking. The check applies to pins only. Default: off

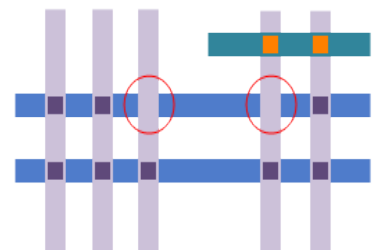
Complete



Missing Path



Missing Vias



`-viaLadderSpecFile`

Via ladder specification file. The syntax is:

`<cellName> <netName> {<ladderVia >}`

`-report`

File name of the postroute checking report. Default: checkPostRoute.csv

`-error`

Error message file name. Default: checkPostRoute.err

`-maxErrorMessage`

Sets the error message line limit. Default: unlimited

Returns

{<deFigure>} : Statistics reporting file and error message file

rt::checkPreRoute

Checks for preroute design violations.

Syntax

```
rt::checkPreRoute (-design <oaDesign> | -nets <{{oaGroup}} | {{oaNet}}>)<br>[-checks <string>]
```

Arguments

`-design`

OA design.

`-nets`

Collection of oaNets or oaGroups.

`-checks`

Types of checks to be performed. Valid values: all, errors, warnings, techErrors.

Returns

<none>

rt::checkRoute

Description

Checks for DRC violations in shapes added by the Custom Router.

Syntax

```
rt::checkRoute -design <oaDesign> [-layers <string>]  
[-routeTypes [<enRouteType>]] [-rules <string>] [-exportMarkers <bool>]  
[-printMarkers <bool>]  
[-checkFrozenShapes <bool>]
```

Arguments

-design

Design to be checked.

-layers

Specific layers to be checked. (Default: all)

-routeTypes

Type of route to be checked. This argument is ignored if -checkFrozenShapes is true.

Default: customRoute

-exportMarkers

Exports violations as OA markers. (Default: true)

-printMarkers

Logs summary of each violation. (Default: false)

-checkFrozenShapes

Checks for DRC violations in shapes that are not added by the Custom Router. (Default: false)

Syntax

```
rt::checkRoute -design <oaDesign> [-rules <string>] [-layers <string>]  
[-routeTypes [<enRouteType>]] [-rules <string>] [-exportMarkers <bool>]  
[-printMarkers <bool>] [-checkFrozenShapes <bool>]
```

Returns

<none>

Arguments

-design

Design for which checkRoute is to be executed

`-rules`

Limit checkRoute on the specified rules. If specified, overrides rtEnableRules preference (Default is:)

`-layers`

Limit checkRoute on the specified layers (Default is: all)

`-routeTypes`

Limit checkRoute on the specified route types (Default is: customRoute). Ignored if -checkFrozenShapes=true is specified

`-exportMarkers`

Export violations as OA Markers (Default is: true)

`-printMarkers`

Log summary of each violation (Default is: false)

`-checkFrozenShapes`

Check violations for non-Router added shapes also (Default is: false)

rt::checkRoutingStyle

Check routing style command

Syntax

```
rt::checkRoutingStyle (-design <oaDesign> | -nets <{{<oaNet>}}>)  
  [-skipSupplyNets <bool>] -style <string> [-boneLayer <string>]  
  [-fingerLayer <string>] [-exportMarkers <bool>] [-filePath <string>]
```

Returns

<none>

Arguments

`-design`

Specify the design to check

`-nets`

Specify the nets to check

`-skipSupplyNets`

Skip supply nets (default: true)

`-style`
Specify the routing style

`-boneLayer`
Specify the bone layer (default: the default bone layer)

`-fingerLayer`
Specify the finger layer (default: the default finger layer)

`-exportMarkers`
Export violations as OA Markers (default: true)

`-filePath`
Specify the file path of the result file

rt::checkTopology

Check for Topology violations

Syntax

```
rt::checkTopology (-design <oaDesign> | -nets <{{<oaGroup>}} | {{<oaNet>}}>)  
[-exportMarkers <bool>] [-checkMapping <bool>]
```

Returns

<none>

Arguments

`-design`
OA design

`-nets`
Collection of oaNets or oaGroups

`-exportMarkers`
Export errors as OA Markers (Default is: false)

`-checkMapping`
Check topology to shape mapping (Default is: false)

rt::connectCellRails

Connects cell rails.

Syntax

```
rt::connectCellRails -design <oaDesign> -nets [<string>]  
-layer <string> [-orient <enum>] [-extendToGrid <bool>] [-autoConnect  
  <bool>]  
[-insts [<string>]] [-cells [<string>]] [-trackRegionName <string>]  
[-regionPoints [<point>]] [-excludeRegionPoints [<point>]] [-lockcolor  
  <bool>]  
[-routeType <enum>] [-rules <string>] [-verbose <int>]
```

Returns

{<deFigure>} : Collection of Newly Created Shapes

Syntax

```
rt::connectCellRails -design <oaDesign> -nets [<string>] -layer  
  <string> [-orient <enum>] [-extendToGrid <bool>] [-autoConnect <bool>]  
  [-insts [<string>]] [-cells [<string>]] [-trackRegionName <string>]  
  [-regionPoints [<point>]] [-excludeRegionPoints [<point>]] [-lockColor  
    <bool>] [-routeType <enum>] [-rules <string>] [-verbose <int>]
```

Returns

{<deFigure>} : Collection of Newly Created Shapes

Arguments

-design

The design

-nets

The list of net names. The power stripes or staples will alternate between the nets if more than 1 net is specified.

-layer

The layer on which to create the power stripe

-orient

The orientation of power stripes ({ horizontal | vertical })

-extendToGrid

Specifies true to extend the rail stripes to power grid, else to the PrBoundaries of all instances.

-autoConnect

Specifies true to insert vias to connect cell rails with other power stripes on adjacent metal layers

`-insts`

List of regular expressions to specify which instances should be connected by this command.

`-cells`

List of regular expressions to specify which cells should be connected by this command.

`-trackRegionName`

Name of the track region in which the stripes or staples should be created. The stripes or staples should follow the local tracks in the track region

`-regionPoints`

List of points of the rectilinear region in which the stripes or staples should be created.

`-excludeRegionPoints`

List of points of the rectilinear region in which the stripes or staples should not be created.

`-lockColor`

Specifies if lock color for created shapes.

`-routeType`

Specifies the route type for created shapes. ({ PGFollowPin | PGPinConnect | PGRing | PGStripe | customRoute | signalUser })

`-rules`

Specifies the set of rules to be honored.

`-verbose`

Verbose level

rt::createFreeFormTopology

Create Free Form Topology

Syntax

```
rt::createFreeFormTopology [<name: string>] (-points [<point>] | -pattern  
<string>) -net <oaNet | string> [-design <oaDesign>] -source <string>  
-target <string> [-connect_overlap <bool>]
```

Returns

<string> : Name of newly created topology

Syntax

```
rt::createFreeFormTopology [<name: string>] (-points [<point>] |  
-pattern <string>) -net <oaNet | string> [-design <oaDesign>] -source  
<deFigure | oaShape | string> -target <deFigure | oaShape | string>  
[-connect_overlap <bool>]
```

Returns

<string> : Name of newly created topology

Arguments

name

The name of topology

-net

The net name or oaNet object

-design

The design

-source

The name of source object for the free form topology.

-target

The name of target object for the free form topology.

-points

The list of points that form topology.

-pattern

The pattern string

-connect_overlap

Control whether to create connection to the overlapped pin or topology

rt::createGroupPattern

Create group pattern

Syntax

```
rt::createGroupPattern [<name: string>] -net <oaNet | string> [-design  
<oaDesign>] -group <string> [-pattern <string>]
```

Returns

<string> : Name of newly created pattern

Arguments

name

The name of topology

-net

The net name or oaNet object

-design

The design

-group

The name of group object for group pattern.

-pattern

The pattern string

rt::createP2PRoute

Description

Runs point-to-point routing in batch mode.

Syntax

```
rt::createP2PRoute (-design <oaDesign> | -net <oaNet>) [-startPoint  
  <point>]  
[-endPoint <point>] [-startObject <deFigure | oaInstTerm | oaShape |  
  oaVia>]  
[-endObject <deFigure | oaInstTerm | oaShape | oaVia>] [-taperMode  
  <string>]  
[-wayPoints [<point>]] [-layers <string>] [-autoViaDrop <bool>]  
[-autoViaDropDep  
  <int>] [-autoViaDropDirection <string>] [-minCutNum <int>]  
[-strictPreferredD  
  irection <bool>] [-coverPin <string>] [-layerCostSetting <string>]  
[-widthSettin  
  g <string>] [-viaSetting <string>] [-routeType <string>]
```

Arguments

-design

Design to be routed.

`-net`

Net to be routed.

`-startPoint`

Location (point) from which the routing starts.

`-endPoint`

Location (point) at which the routing ends.

`-startObject`

Shape (or object) from which the routing starts.

`-endObject`

Shape (or object) at which the routing ends.

`-taperMode`

Adjusts the width of the first and last path segment of the route to match the dimensions of the connecting pin. You can taper at the starting pin, ending pin, or both pins. Valid values: off, both, start, end. (Default: off)

`-wayPoints`

Specifies intermediate points to control the course of the routes.

`-layers`

Specifies the layers used for the routing.

`-autoViaDrop`

Automatically adds vias to connect to same-net shapes below routing wires.
(Default: false)

`-autoViaDropDepth`

Specifies the layer depth for automatic via drop. (Default: 1)

`-autoViaDropDirection`

Specifies the direction of the automatic via drop: down, up, or both. (Default: down)

`-minCutNum`

Specifies the minimum number of cuts in the vias. (Default: 1)

`-strictPreferredDirection`

Allows only preferred-direction routing. (Default: off)

`-coverPin`

Cover the starting and ending shape with via. Valid values are off, both, start, and end. (Default: off)

`-layerCostSetting`

Specifies the layer cost: low, neutral, or high. Specify the layer cost in paired values: {layerName cost}. For example, to specify the cost for m1, m2, and m3, specify the paired values as follows: {{m1 low} {m2 high} {m3 neutral}}.

`-widthSetting`

Specifies the layer width. The format is {layerName width}. For example, to set the width for m1 and m2, specify the paired values as follows: {{m1 0.1} {m2 0.2}}.

`-viaSetting`

Specifies the via definition to use by layer. The format is {layerName viadef cutclass}. For example, to set the via definitions for m1, m2, and m3, specify the values as follows: {{m1 viadef1 cutclass1} {m2 viadef2 auto} {m3 auto cutclass3}}.

`-routeType`

Specifies routeType of created OA shapes.

To determine the starting and ending routing locations, the command assesses the available information from the arguments. The following table summarizes how the command establishes the starting and ending locations based on the available information: net, points (startPoint, endPoint), and objects (startObject, endObject).

Net	startPoint endPoint	startObject endObject	Behavior
x	x	x	The command uses the specified point, which must be within the boundary of the specified object, and the object must be on the specified net. Errors occur if: • The object is not on a routing layer. • The object is not found. • The point is not within the boundary of the specified object.
x	x		The command uses the point to locate the object at the highest layer in the metal stack. The object must be on the specified net. Error occurs if no shape is found on the specified net at the specified point.

Net	startPoint endPoint	startObject endObject	Behavior
x		x	The command uses the specified object as the starting or ending location. Errors occur if: • The object is not on a routing layer. • The object is not on the specified net.
x			The command cannot determine the starting or ending point based on only the net information. You must also provide the point, object, or both information.
	x	x	The command uses the specified point, which must be within the boundary of the specified object. Net information is derived from the object. Errors occur if: • The object is not on a routing layer. • The point is not within the boundary of the specified object.
	x		The command cannot determine the starting or ending point with only the point information. You must also provide the net, object, or both information.
		x	The command uses the object as the starting or ending location. Error occurs if the object is not on a routing layer.

Returns

{<dbCollection>} : Collection of Newly Created Routes.

rt::createPatternRoute

Description

Applies the specified route pattern when routing the selected net.

Syntax

```
rt::createPatternRoute -net <oaNet> -pattern <string>
```

Arguments

-net

Net to be routed.

-pattern

Route pattern to be applied during the routing operation. See [Route Pattern Description Reference](#).

Returns

<none>

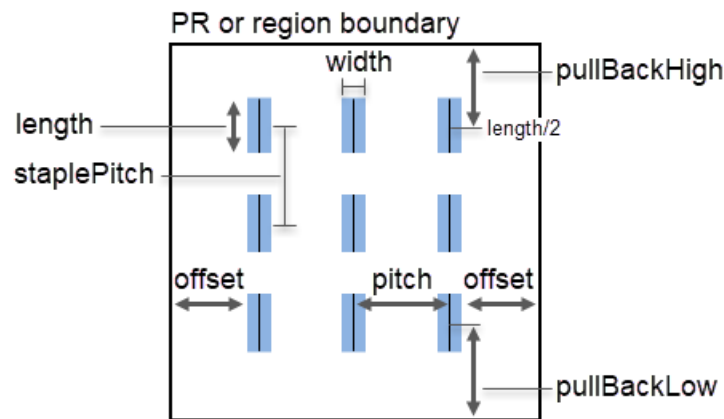
Examples

```
rt::createPatternRoute -net $oaNet -pattern "fishbone row 2 track 1 / 0  
layer M2 tapoff width 0.06;"
```

rt::createPowerStaples

Creates short metal shapes that connect adjacent VDD or VSS rails to minimize IR drop.

Power staple components are placed with reference to the PR boundary or the boundary of the specified region.



Syntax

```
rt::createPowerStaples -design <oaDesign> -nets [<string>]  
[-create_net <bool>] -layer <string> [-pitch <float>] [-offset <float>]  
[-width <float>] [-length <float>] [-staplePitch <float>] [-orient  
  <enum>]  
[-pullBackLow <float>] [-pullBackHigh <float>] [-snapTrack <bool>]  
[-useTrackSigType <bool>] [-keepBoundary <bool>] [-trackRegionName  
  <string>]  
[-regionPoints [<point>]] [-excludeRegionPoints [<point>]] [-lockcolor  
  <bool>]  
[-routeType <enum>] [-rules <string>]  
[-verbose <int>]
```

Returns

{<deFigure>} : Collection of Newly Created Shapes

Syntax

```
rt::createPowerStaples -design <oaDesign> -nets [<string>] [-create_net  
<bool>] -layer <string> [-pitch <float>] [-offset <float>] [-width  
<float>] [-length <float>] [-staplePitch <float>] [-orient <enum>]  
[-pullBackLow <float | string>] [-pullBackHigh <float | string>]  
[-snapTrack <bool>] [-useTrackSigType <bool>] [-keepBoundary  
<bool>] [-trackRegionName <string>] [-regionPoints [<point>]]  
[-excludeRegionPoints [<point>]] [-lockColor <bool>] [-routeType  
<enum>] [-rules <string>] [-verbose <int>] [-ported <bool | oaPin |  
<oaPin>>] [-pinPurposeNames <string>] [-createPinShapes <bool>]
```

Returns

{<deFigure>} : Collection of Newly Created Shapes

Arguments

-design

The design

-nets

The list of net names. The power stripes or staples will alternate between the nets if more than 1 net is specified.

-create_net

Create a net if the specified net name doesn't exist.

-layer

The layer on which to create the power stripe

-pitch

The pitch at which the power stripe should be repeated

-offset

The offset of the first stripe from origin of the PRBoundary or the region.

-width

The width of the power stripe

-length

The staple length

-staplePitch

The staple pitch

`-orient`

The orientation of power stripes ({ horizontal | vertical })

`-pullBackLow`

The distance by which the stripe or staple should be pulled back from the beginning of region or PRBoundary (float value, 'halfDR', 'fullDR')

`-pullBackHigh`

The distance by which the stripe or staple should be pulled back from the end of region or PRBoundary (float value, 'halfDR', 'fullDR')

`-snapTrack`

Snap to track

`-useTrackSigType`

Use track sigType to find where the power stripe should be created.

`-keepBoundary`

Control whether the stripe with centerline coincident with region will be created

`-trackRegionName`

Name of the track region in which the stripes or staples should be created. The stripes or staples should follow the local tracks in the track region

`-regionPoints`

List of points of the rectilinear region in which the stripes or staples should be created.

`-excludeRegionPoints`

List of points of the rectilinear region in which the stripes or staples should not be created.

`-lockColor`

Specifies if lock color for created shapes.

`-routeType`

Specifies the route type for created shapes. ({ PGFollowPin | PGPinConnect | PGRing | PGStripe | customRoute | signalUser })

`-rules`

Specifies the set of rules to be honored.

`-verbose`

Verbose level

`-ported`

Create pin for the stripes or staples

`-pinPurposeNames`

A string of purpose names separated by spaces

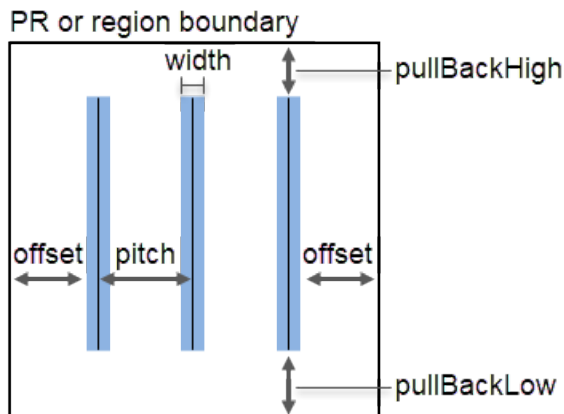
`-createPinShapes`

Create new shapes when creating pin. The original stripes/staples are kept.

rt::createPowerStripes

Creates power stripes (or rails) based on defined tracks. The power rails are created within the PR boundary or a region you specify. Use this command with the [rt::createPowerStaples](#) command to create power mesh in your design.

Power stripe components are placed with reference to the PR boundary or the boundary of the specified region.



Syntax

```
rt::createPowerStripes -design <oaDesign> -nets [<string>]
[-create_net <bool>] -layer <string> [-pitch <float>] [-offset <float>]
[-width <float>] [-useTrackSigType <bool>] [-keepBoundary <bool>]
[-snapTrack <bool>] [-orient <enum>] [-pullBackLow <float>]
[-pullBackHigh <float>] [-count <int>] [-trackRegionName <string>]
[-regionPoints [<point>]] [-excludeRegionPoints [<point>]] [-lockcolor
<bool>]
[-routeType <enum>] [-rules <string>] [-verbose <int>]
```

Returns

{<deFigure>} : Collection of Newly Created Shapes

Syntax

```
rt::createPowerStripes -design <oaDesign> -nets [<string>] [-create_net  
<bool>] -layer <string> [-pitch <float>] [-offset <float>] [-width  
<float>] [-useTrackSigType <bool>] [-keepBoundary <bool>] [-snapTrack  
<bool>] [-orient <enum>] [-pullBackLow <float | string>] [-pullBackHigh  
<float | string>] [-count <int>] [-trackRegionName <string>]  
[-regionPoints [<point>]] [-excludeRegionPoints [<point>]] [-lockColor  
<bool>] [-routeType <enum>] [-rules <string>] [-verbose <int>]  
[-ported <bool | oaPin | {<oaPin>}>] [-pinPurposeNames <string>]  
[-createPinShapes <bool>]
```

Returns

{<deFigure>} : Collection of Newly Created Shapes

Arguments

-design

The design

-nets

The list of net names. The power stripes or staples will alternate between the nets if more than 1 net is specified.

-create_net

Create a net if the specified net name doesn't exist.

-layer

The layer on which to create the power stripe

-pitch

The pitch at which the power stripe should be repeated

-offset

The offset of the first stripe from origin of the PRBoundary or the region.

-width

The width of the power stripe

-useTrackSigType

Use track sigType to find where the power stripe should be created.

-keepBoundary

Control whether the stripe with centerline coincident with region will be created

`-snapTrack`

Snap to track

`-orient`

The orientation of power stripes ({ horizontal | vertical })

`-pullBackLow`

The distance by which the stripe or staple should be pulled back from the beginning of region or PRBoundary (float value, 'halfDR', 'fullDR')

`-pullBackHigh`

The distance by which the stripe or staple should be pulled back from the end of region or PRBoundary (float value, 'halfDR', 'fullDR')

`-count`

Number of stripes to be created in the region or PRBoundary.

`-trackRegionName`

Name of the track region in which the stripes or staples should be created. The stripes or staples should follow the local tracks in the track region

`-regionPoints`

List of points of the rectilinear region in which the stripes or staples should be created.

`-excludeRegionPoints`

List of points of the rectilinear region in which the stripes or staples should not be created.

`-lockColor`

Specifies if lock color for created shapes.

`-routeType`

Specifies the route type for created shapes. ({ PGFollowPin | PGPinConnect | PGRing | PGStripe | customRoute | signalUser })

`-rules`

Specifies the set of rules to be honored.

`-verbose`

Verbose level

`-ported`

Create pin for the stripes or staples

`-pinPurposeNames`

A string of purpose names separated by spaces

`-createPinShapes`

Create new shapes when creating pin. The original stripes/staples are kept.

rt::createTrunkTopology

Create Trunk Topology

Syntax

```
rt::createTrunkTopology [<name: string>] (-orient <enum> | -pattern  
<string>) [-object_names [<string>]] -net <oaNet | string> [-design  
<oaDesign>]
```

Returns

<string> : Name of newly created topology

Syntax

```
rt::createTrunkTopology [<name: string>] (-orient <enum> | -pattern  
<string>) [-object_names [<string>]] [-objects <{<deFigure>} | {<oaFig>}  
| {<oaShape>}}] -net <oaNet | string> [-design <oaDesign>]
```

Returns

<string> : Name of newly created topology

Arguments

`name`

The name of topology

`-object_names`

The objects, which are used to create the topology.

`-objects`

List of objects.

`-net`

The net name or oaNet object

`-design`

The design

`-orient`

The orientation for trunk topology. ({ auto | horizontal | vertical })

`-pattern`

The pattern string

rt::createUniblock

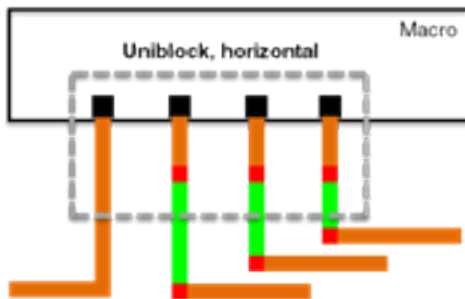
Description

Creates an area within which routes of the specified direction is prohibited. The area is specified on a per layer and per net basis. Be aware that no spacing rule is applied within the specified area during routing.

Note:

The area is reserved in memory only and is not displayed in the design.

In the following example, the command creates an area within which horizontal routes are prohibited. The router is only permitted to create vertical routes within the specified area.



Syntax

```
rt::createUniblock <box: box> -design <oaDesign> -direction <enum>
[-layer <oaLayer | string>] [-net <oaNet | string>]
```

Returns

<none>

Arguments

`box`

The coordinates of the bounding box of the uniblock.

`-design`

Design in which to create the uniblock.

`-direction`

The direction of the uniblock: {both | horizontal | vertical}.

`-layer`

The layer on which to create the uniblock.

`-net`

The net name or the oaNet object to which the uniblock should be assigned.

Examples

In the following example, the command creates an area on the M5 layer. The router is only permitted to add horizontal routes within the area.

```
rt::createUniblock {{0 0} {260 2100}} -design [ed] -direction vertical  
-layer "M5"
```

rt::createViaLadder

Create via ladders for given nets.

Syntax

```
rt::createViaLadder [-nets <{<oaNet>}>] [-config <string>] [-pin_type  
<enum>] [-trim <bool>] [-instTerms <{<oaInstTerm>}>] [-viaLadderSpec  
<string>] [-dirty <bool>]
```

Returns

<none>

Arguments

`-nets`

nets

`-config`

via ladder definition file

`-pin_type`

pin type used when creating via ladders. Values: [output] | input | all ({ all | input | output })

`-trim`

trim extra single straps on upper layers

- `-instTerms`
Specifies the collection of instance terms
 - `-viaLadderSpec`
Specifies the via ladder spec
 - `-dirty`
Create via ladder without checking design rule
-

rt::deleteCustomRouteShapes

Description

Deletes routes and accompanying shielding created by the Custom Router (Auto Router, Interactive Router, and pattern routing processes).

Syntax

```
rt::deleteCustomRouteShapes (-design <oaDesign> | -nets <{<oaGroup>} |  
    {<oaNet>}>) [-deleteShield <bool>]
```

Arguments

- `-design`
Removes route shapes created by the Custom Router in the specified OA design.
- `-nets`
Removes route shapes on the specified OA nets or in OA groups.
- `-deleteShield`
Removes the shielding around the routes you are deleting. (Default: true)

Returns

<none>

rt::deleteShield

Description

Deletes shields automatically.

Syntax

```
rt::deleteShield (-design <oaDesign> | -nets <{<oaGroup>} | {<oaNet>}>)
```

Arguments

-design

unshield all nets in this oaDesign.

-nets

Collection of oaNets or oaGroups to unshield.

Returns

<none>

rt::deleteTopology

Delete Topology

Syntax

```
rt::deleteTopology [<name: string>] (-nets <{<oaNet>}> | -design  
  <oaDesign>) [-PR <bool>]
```

Returns

<none>

Arguments

name

The name of topology

-nets

nets

-design

The design

-PR

Control whether to preserve existing PR constraints

rt::deleteTrunkAndShield

Deletes trunk and shield shapes created by the Generate Trunk command.

Syntax

```
rt::deleteTrunkAndShield [-design <oaDesign>] -nets <string | {<oaNet>}>  
  [-layer <string>]
```


Returns

<none>

Arguments

-design

Specifies the design

-nets

List of net names.

-layer

Layers for which the command applies.

rt::deleteTrunkBox

Deletes search boxes created from the Generate Trunk command.

Syntax

```
rt::deleteTrunkBox -design <oaDesign>
```

Returns

<none>

Arguments

-design

Design in which the search boxes are deleted.

rt::estimateRoute

Description

Estimates routes.

Syntax

```
rt::estimateRoute [-design <oaDesign> | -nets <{<oaNet>}>] [-allowTrunk  
  Overlap <bool> |  
-relaxDRC <bool>][-relaxDRC <bool> | -honorTrunk <bool>] [-all  
  owTrunkOverlap <bool> |  
-removeShort <bool>] [-removeShort <bool> | -honorTrunk <bool>]  
  [-groupDistance <float>]  
  [-groupDriverPin <bool>] [-excludeTrunkLayers <string>] [-ignoreBlockage  
  <bool>]
```

Arguments

`-design`

Routes all nets in the specified design.

`-nets`

Collection of Nets to route.

`-groupDistance`

Sets pin group distance.

`-groupDriverPin`

Groups driver pins with receiver pins.

`-excludeTrunkLayers`

Excludes trunk layers.

`-allowTrunkOverlap`

Allows trunk overlap.

`-ignoreBlockage`

Routes in Force mode. Default: `true`

`-relaxDRC`

Relax DRC. Default: `false`

`-removeShort`

Remove short. Default: `false`

Returns

`{<oaFig>}` : Collection of created objects.

rt::exportTopology

Export Topology

Syntax

```
rt::exportTopology (-nets <{<oaNet>}> | -design <oaDesign>) [-filePath  
  <string>] [-overwrite <bool>] [-verbose [<bool>]] [-exportName <bool>]
```

Returns

`<none>`

Arguments

<code>-nets</code>	<code>nets</code>
<code>-design</code>	The design
<code>-filePath</code>	The name of file
<code>-overwrite</code>	Specifies whether to overwrite
<code>-verbose</code>	verbose
<code>-exportName</code>	Specifies whether to export topology name

rt::extractShielding

Extracts shielding constraints.

Syntax

```
rt::extractShielding (-design <oaDesign> | -nets <{<oaGroup>}  
| {<oaNet>}}) [-create_constraint <bool>] [-max_gap <string>]  
[-max_gap_by_pitch <int>]
```

Returns

<none>

Arguments

<code>-design</code>	OA design.
<code>-nets</code>	Collection of oaNets or oaGroups.
<code>-create_constraint</code>	Export extracted constraints to Constraint Manager. (Default: 0)

`-max_gap`

Query distance of shielding extraction. (Default: 0.0)

`-max_gap_by_pitch`

Query distance in terms of pitch of shielding extraction. (Default: 0)

rt::generateTrunk

Description

Creates route trunks based on options you specify.

Syntax

```
rt::generateTrunk [-shareTrack <bool> | -groupMode <bool>] [-design  
  <oaDesign>]  
-nets <string | {<oaNet>}> -layer <string> [-startSide <enum>] [-align  
  <enum>]  
[-extensionSide <enum>] [-freedom <float>] [-ratio <int>] [-num <int>]  
[-coord <float | string>] [-coordFreedom <float>] [-begin <float |  
  string>]  
[-end <float | string>] [-shieldSide <enum>] [-shieldNetName [<string>]]  
[-shieldRatio <int>]  
[-includeInsts [<string>]] [-excludeInsts [<string>]] [-trimLengthLow  
  <float>]  
[-trimLengthHigh <float>] [-trimLengthDriver <float>]  
[-trimLengthReceiver <float>]  
[-shareShield <bool>] [-mergeShields <bool>] [-trackIndexes [<int>]]  
[-override <bool>]  
[-tag <string>] [-alignLayers [<string>]] [-trunkOffset <float>]  
[-autoConnect <bool>]  
[-connectPositions [<point>]] [-ported <bool>] [-drawBox <bool>] [-rules  
  <string>] [-snapTrack <bool>]  
[-verbose <int>] [-width <float>] [-wrongWay <bool>]  
[-extendToExistingTrunk <string>]  
[-groupPinByRow <int>] [-checkPinConnectivity <bool>] [-checkTrunkLength  
  <float>]  
[-sameTrackRegion <bool>]
```

Arguments

`-design`

Specifies a design.

`-nets`

List of nets.

`-layer`

Layer on which to create the trunk route.

`-startSide`

Starting point where the trunk generation looks for a valid location for trunk generation: { center | gravity | high | highpin | low | lowpin | majority }.

`-align`

Type of pins to consider for trunk route generation: { all | driver | instPin | receiver | topPin }.

`-extensionSide`

Extends the trunk beyond the pin based on the following specification: { both | bottom | left | pinEdge | right | top }. Default: pinEdge

`-freedom`

Length and width, in μm , to increase the search box for trunk placement.
Default: 0.0

`-ratio`

Required trunk length that is created without violations (DRC-clean). Trunk routing is considered successful if the percent of the routes meet the ratio you specify. Default: 100

`-num`

Number of trunks created together. The trunks are on adjacent tracks and have the same length. Default: 1

`-groupMode`

Creates trunks for two or more nets on consecutive tracks. If consecutive tracks are not available, group trunk will not be created.

Trunks are placed within the search location, and the order of trunks is based on the order specified by the `-nets` option. Placement sequence of horizontal trunks is from the bottom to the top. Placement sequence of vertical trunks is from left to right.

If you specify `-shieldNetName`, shield shapes are automatically added on the outside of the group, and the length of the shield shapes match the maximum length of the group. When you specify `-groupMode` with `-shieldNetName`, the `-shieldSide` argument is ignored.

Default: false

`-coord`

Starting point where the trunk generation looks for a valid location for trunk generation. This argument has higher precedence than `-startSide`.

`-coordFreedom`

Enlarges the search box from the `-coord` line to create a trunk near the specified coordinate. This argument must be used with and overrides `-coord`.

`-begin`

Starts the trunk extension at the specified location.

`-end`

Ends the trunk extension at the specified location.

`-shieldSide`

Side of the trunk route to add shielding: { any | both | high | low }. Default: any

`-shieldNetName`

Shield net to use when adding shields to trunk routes.

`-shieldRatio`

Percent of trunks that should have shielding. Default: 100

`-includeInsts`

List of instances to consider during trunk routing.

`-excludeInsts`

List of instances to be excluded from consideration for trunk routing.

`-trimLengthLow`

Increases or decreases the trunk length on the low side of the trunk (left side of vertical trunks, and bottom side of horizontal trunks). A value > 0 decreases the length, and a value < 0 increases the length.

`-trimLengthHigh`

Increases or decreases the trunk length on the high side of the trunk (right side of vertical trunks, top side of horizontal trunks). A value > 0 decreases the length, and a value < 0 increases the length.

`-trimLengthDriver`

Increases or decreases the trunk length on the side close to the driver pins. A value > 0 decreases the length, and a value < 0 increases the length.

`-trimLengthReceiver`

Increases or decreases the trunk length on the side close to the receiver pins. A value > 0 decreases the length, and a value < 0 increases the length.

`-shareTrack`

Permits trunks across multiple nets to use the same track. To use track sharing, you must specify two or more nets. In addition, the search boxes of the nets cannot overlap. Default: false

`-trackIndexes`

Places trunk routes only on the tracks you specify.

`-shareShield`

Enables shield sharing. Default: true.

`-mergeShields`

Enables merging of new shields and existing shields. Default: false.

`-override`

Replaces existing trunks routes, associated shielding, and via shapes on the specified layer. Default: false

`-tag`

An identifier (string) that you can add to a trunk route. You can use the tag to locate a trunk route in the design.

In the following example, the first command adds a tag (tagA) to a newly created trunk route (A). The second command creates another trunk route (B) that aligns precisely with trunk A.

```
rt::generateTrunk -nets [db::getNets A -of [ed]] -layer m1 -tag  
tagA  
rt::generateTrunk -nets [db::getNets B -of [ed]] -layer m1 -begin  
tagA:begin -end tagA:end
```

`-alignLayers`

Considers only the pin shapes on the specified layers when determining the search box.

`-trunkOffset`

Displacement or adjusted distance from the original `-startSide` location. The `-coord` argument has higher precedence than `-trunkOffset`. Default: 0.0

`-autoConnect`

Automatically adds vias to connect to adjacent layers when the newly created trunks overlaps with pins or existing trunks. Default: false

`-connectPositions`

Specifies where to insert vias for the overlapped areas between newly created trunk and pins or the existing trunks.

`-ported`

Creates pin objects and pin labels (net name) for the newly created trunks. By default, the pin layer purpose is used if it is defined in the technology database. Otherwise, the drawing layer purpose is used. Default: false

`-drawBox`

Draws the search box in the Layout Editor regardless of whether trunk routes are created. Default: false

The search box is a runtime object that overlays the design. It provides a visual indicator, primarily for troubleshooting purposes. For example, an empty search boxes might indicate that a trunk route is expected at the location. You can delete the search box using the [rt::deleteTrunkBox](#) Tcl command.

`-rules`

Specifies the set of rules to be honored.

`-snapTrack`

Creates trunk routes on the closest track (true). If false, creates trunk routes at the calculated location. Default: true

This option works with `-rules`.

`-verbose`

Verbose level for event logging. Default: 1

`-width`

Specifies the trunk width. Default: 0.0

`-wrongWay`

Specifies to generate the wrong way trunk. Default: false.

`-extendToExistingTrunk`

Specifies the layer to extend to connect.

`-groupPinByRow`

Specifies the number of rows that are going to be taken as one unit for trunk generation.

`-checkPinConnectivity`

Checks if pins are on the layer that are already connected. Default: false.

`-checkTrunkLength`

Checks the minimum track length. Default: 0.0.

`-sameTrackRegion`

Enables the same track region preference. Default: false.

Returns

{<deFigure>} : Collection of Newly Create Shapes

Syntax

```
rt::generateTrunk [-shareTrack <bool> | -groupMode <bool>] [-design  
  <oaDesign>] -nets <string | {<oaNet>}> -layer <string> [-startSide  
  <enum>] [-align <enum>] [-extensionSide <enum>] [-freedom <float>]  
  [-ratio <int>] [-num <int>] [-coord <float | string>] [-coordFreedom  
  <float>] [-begin <float | string>] [-end <float | string>] [-shieldSide  
  <enum>] [-shieldNetName [<string>]] [-shieldWidth <float>] [-shieldGap  
  <float>] [-ignoreTrackForShield <bool>] [-shieldRatio <int>]  
  [-includeInsts [<string>]] [-excludeInsts [<string>]] [-trimLengthLow  
  <float>] [-trimLengthHigh <float>] [-trimLengthDriver <float>]  
  [-trimLengthReceiver <float>] [-shareShield <bool>] [-mergeShields  
  <bool>] [-trackIndexes [<int>]] [-override <bool>] [-tag <string>]  
  [-alignLayers [<string>]] [-trunkOffset <float>] [-autoConnect <bool>]  
  [-connectPositions [<point>]] [-ported <bool>] [-drawBox <bool>] [-rules  
  <string>] [-snapTrack <bool>] [-verbose <int>] [-width <float>]  
  [-wrongWay <bool>] [-extendToExistingTrunk <string>] [-groupPinByRow  
  <int>] [-checkPinConnectivity <bool>] [-checkTrunkLength <float>]  
  [-sameTrackRegion <bool>] [-allowSameNetOverlap <bool>]
```

Returns

{<deFigure>} : Collection of newly created shapes

Arguments

`-design`

Specifies the design

`-nets`

Specifies the collection of nets

`-layer`

Specifies the layer

`-startSide`

Specifies where to start the search. Default: center ({ center | gravity | high |
highpin | low | lowpin | majority })

`-align`

Specifies what kind of pins to be considered when calculate the location to start the search. Default: all ({ all | driver | instPin | receiver | topPin })

`-extensionSide`

Specifies the way to extend the trunk. Default: pinEdge ({ both | bottom | left | pinEdge | right | top })

`-freedom`

Specifies to enlarge the search box by the value. Default: 0.0

`-ratio`

Specifies the minimal percentage of required trunk coverage. Default: 100

`-num`

Specifies the number of trunks to be create. Default: 1

`-coord`

Specifies the coordinate to start the search

`-coordFreedom`

Specifies search range of the option -coord

`-begin`

Specifies the start endpoint of the trunk.

`-end`

Specifies the stop endpoint of the trunk.

`-shieldSide`

Specifies the parallel shielding side. Default: any ({ any | both | high | low })

`-shieldNetName`

Specifies the shield net name or 'power'

`-shieldWidth`

Specifies the shield width for shielding

`-shieldGap`

Specifies the default shield gap for shielding

`-ignoreTrackForShield`

Boolean option to ignore tracks for shielding. Default: false

`-shieldRatio`

Specifies the minimal percentage of required shielding coverage. Default: 100

`-includeInsts`

Specifies a list of instance names, and only pins from these instances will be considered

`-excludeInsts`

Specifies a list of instance names, and pins from these instances will be skipped

`-trimLengthLow`

Specifies the length the trunk to trim or grow at low end

`-trimLengthHigh`

Specifies the length the trunk to trim or grow at high end

`-trimLengthDriver`

Specifies the length the trunk to trim or grow at driver end

`-trimLengthReceiver`

Specifies the length the trunk to trim or grow at receiver end

`-shareTrack`

Boolean option to enable track sharing. Default: false

`-shareShield`

Boolean option to enable shield sharing. Default: true

`-mergeShields`

Boolean option to merge new shields and existing shields. Default: false

`-trackIndexes`

Specifies the track indexes

`-groupMode`

Specifies whether to generate trunks for multiple nets on consecutive tracks.
Default: false

`-override`

Specifies whether to override existing trunk. Default: false

`-tag`

Specifies tag name added to the trunk.

`-alignLayers`

Specifies a list of layers to filter pin shapes

`-trunkOffset`

Specifies the offset to adjust the start search location. Default: 0.0

`-autoConnect`

Specifies whether to insert vias for the overlap areas between newly created trunk and pins or existing trunks. Default: false

`-connectPositions`

Specifies where to insert vias for the overlap areas between newly created trunk and pins or existing trunks.

`-ported`

Specifies whether to create pin object. Default: false

`-drawBox`

Boolean option to draw the search box in canvas. Default: false

`-rules`

Specifies the set of rules to be honored.

`-snapTrack`

Specifies whether to snap to track in the rules mode. Default: true

`-verbose`

Specifies verbose level. Default: 1

`-width`

Specifies the trunk width to create. Default: 0.0

`-wrongWay`

Boolean option to generate wrong way trunk. Default: false

`-extendToExistingTrunk`

Specifies the layer to extend to connect

`-groupPinByRow`

Specifies the number of rows that are going to be taken as one unit for trunk generation.

`-checkPinConnectivity`

Boolean option to check if pins are the layer are already connected. Default:
`false`

`-checkTrunkLength`

Specifies the mininum track length . Default: 0.0

`-sameTrackRegion`

Boolean option to enable same track region preference. Default: `false`

`-allowSameNetOverlap`

Boolean option to allow generating trunks overlapping same-direction same-net
trunks. Default: `true`

rt::getStrappableNets

Description

Gets net that has strappable pins.

Syntax

```
rt::getStrappableNets [-nets <{<oaNet>}>] [-design <oaDesign>]
```

Arguments

`-nets`

`nets`

`-design`

Custom Router command for this oaDesign.

Returns

`<none>`

rt::getTopologies

Get Topologies

Syntax

```
rt::getTopologies -net <oaNet | string> [-design <oaDesign>]
```

Returns

`<string>` : List of Topology Names

Arguments

`-net`

The net name or oaNet object

`-design`

The design

rt::getTopologyInfo

Get Topology Info

Syntax

```
rt::getTopologyInfo -net <oaNet | string> [-design <oaDesign>] -name  
  <string>
```

Returns

<string> : Topology Information

Arguments

`-net`

The net name or oaNet object

`-design`

The design

`-name`

The name of topology

rt::getTopologyNets

Get Topology Nets

Syntax

```
rt::getTopologyNets -design <oaDesign>
```

Returns

{<oaNet>} : Collection of OA Nets

Arguments

`-design`

The design

rt::importTopology

Import Topology

Syntax

```
rt::importTopology (-nets <{<oaNet>}> | -design <oaDesign>) -filePath  
  <string> [-overwrite <bool>] [-skipError <bool>] [-verbose [<bool>]]
```

Returns

<none>

Arguments

-nets

nets

-design

The design

-filePath

The name of file

-overwrite

Specifies whether to overwrite

-skipError

Control whether to skip data errors and continue importing other topologies.
(Default: false)

-verbose

verbose

rt::insertCutMetalPatch

Inserts wire patches for cut metal.

Syntax

```
rt::insertCutMetalPatch -design <oaDesign> -layers [<string>]  
[-colors [<int>]] [-eraseWidths [<float>]] [-minExtensions [<float>]]  
[-maxExtensions [<float>]] [-eraseSpaces [<float>]] [-e2eSpacing  
  [<float>]]  
[-maxMetalWidth [<float>]] [-minMetalWidth [<float>]] [-maxLength  
  [<float>]]  
[-minLength [<float>]] [-preprocess <bool>] [-extendPinToLEGs <bool>]  
[-verbose <int>]
```

Arguments

`-design`

The design.

`-layers`

The list of layer names.

`-colors`

The list of colors.

`-eraseWidths`

The list of cut metal widths.

`-minExtensions`

The list of cut metal extensions.

`-maxExtensions`

The list of cut metal extensions.

`-eraseSpaces`

The list of cut metal spacings.

`-e2eSpacing`

The minimum end-to-end spacing without cut metal.

`-maxMetalWidth`

The maximum wire width for cut metal.

`-minMetalWidth`

The minimum wire width for cut metal.

`-maxLength`

The maximum cut metal length.

`-minLength`

The minimum cut metal length.

`-preprocess`

Preprocess for potential conflict of cut metals.

`-verbose`

Provides detailed status of operations.

`-extendPinToLEGs`

Extends pin shapes onto line-end grids.

Returns

{<deFigure>} : Collection of Newly Created Shapes.

rt::insertVias

Creates vias.

Syntax

```
rt::insertVias -design <oaDesign> -nets [<string>] [-minLayer <string>]
[-maxLayer <string>]
[-trackRegionName <string>] [-regionPoints [<point>]]
[-excludeRegionPoints [<point>]] [-addPatch <bool>]
[-allowExceed <bool>] [-routeType <enum>] [-rules <string>] [-verbose
<int>]
```

Returns

{<deFigure>} : Collection of Newly Created Shapes

Syntax

```
rt::insertVias -design <oaDesign> -nets [<string>] [-minLayer <string>]
[-maxLayer <string>] [-trackRegionName <string>] [-regionPoints
<point>]] [-excludeRegionPoints [<point>]] [-addPatch <bool>]
[-allowExceed <bool>] [-routeType <enum>] [-rules <string>] [-verbose
<int>]
```

Returns

{<deFigure>} : Collection of Newly Created Shapes

Arguments

`-design`

The design

`-nets`

The list of net names.

`-minLayer`

The min layer

`-maxLayer`

The max layer

`-trackRegionName`
Name of the track region in which the vias should be created

`-regionPoints`
List of points of the rectilinear region in which vias should be created.

`-excludeRegionPoints`
List of points of the rectilinear region in which vias should not be created.

`-addPatch`
Control whether to add wire patch when via covers protrude wire path or to fix DRCs for the added via.

`-allowExceed`
Allows the via metal shapes to extend beyond the overlapping paths. (Default is: true)

`-routeType`
Specifies the route type for created shapes. ({ PGFollowPin | PGPinConnect | PGRing | PGStripe | customRoute | signalUser })

`-rules`
Specifies the set of rules to be honored.

`-verbose`
Verbose level

rt::powerHookup

Connects the power stripes and staples.

Syntax

```
rt::powerHookup -design <oaDesign> -nets [<string>] [-orient <enum>]  
[-allowed_layers [<string>]] [-allowed_stripe_layers [<string>]]  
[-stripe_search_distance <float>] [-use_1lp <bool>] [-trackRegionName  
  <string>]  
[-regionPoints [<point>]] [-excludeRegionPoints [<point>]] [-verbose  
  <int>] [-rules <string>]
```

Returns

{<deFigure>} : Collection of Newly Created Shapes

Arguments

`-design`

Design in which the power stripes or staples are connected.

`-nets`

List of net names. The power stripes or staples will alternate between the nets if more than one net is specified.

`-orient`

Orientation of power stripes ({ both | horizontal | vertical })

`-allowed_layers`

List of allowed routing layers for making the connections.

`-allowed_stripe_layers`

List of allowed stripe layers for making the connections.

`-stripe_search_distance`

stripe search distance.

`-use_llp`

Uses the LLP engine.

`-trackRegionName`

Name of the track region in which the stripes or staples are created. The stripes or staples should follow the local tracks in the track region.

`-regionPoints`

List of points that define the rectilinear region in which the stripes or staples are created.

`-excludeRegionPoints`

List of points that define the rectilinear region in which the stripes or staples should not be created.

`-verbose`

Verbose event log level.

`-rules`

Specifies the set of rules to be honored.

rt::realizeGlobalRoute

Description

Realizes the global routes.

Syntax

```
rt::realizeGlobalRoute [-design <oaDesign>] [-nets <{<oaGroup>} |  
{<oaNet>}>]
```

Arguments

-design

Design in which all nets are routed in global routing.

-nets

The nets or net groups to be routed.

Returns

<none>

rt::reportSimilarity

Report routing similarity between this design and a source design

Syntax

```
rt::reportSimilarity -targetDesign <oaDesign> [-sourceFilePath <string>]  
[-outputFilePath <string>]
```

Returns

<none>

Arguments

-targetDesign

Target design to be compared with source

-sourceFilePath

Extracted data from source design. Use migrationSource property if not provided

-outputFilePath

Output report file path

`-scorers`

Decide the scorers to be ran

rt::route

Description

Starts automatic routing.

Syntax

```
rt::route (-design <oaDesign> | -nets <{<oaGroup>} | {<oaNet>}>)  
  [-optionSet <string>]  
  [-autoCreateTopology <bool>]
```

Arguments

`-design`

Routes all nets in this oaDesign.

`-nets`

Collection of oaNets or oaGroups to be routed.

`-optionSet`

Specifies the name of option set.

`-autoCreateTopology`

Automatically creates topology for selected nets or entire design.

Returns

<none>

Examples

```
# Auto-create topology for net A  
rt::route -autoCreateTopology true -nets [db::getNets A -of [ed]]
```

rt::routeHierarchy

Description

Routes the design hierarchy from the bottom level to the top level. When routing at lower levels, the command considers the instances at the top level to avoid creating shorts or design rule violations.

Syntax

```
rt::routeHierarchical -topDesign <oaDesign> -nets <{<oaNet>}> [-order  
    <enum>]
```

Arguments

`-topDesign`

Top-level design to be routed.

`-nets`

Collection of oaNets to be routed. Use a comma to separate the net names.

`-order`

Specifies the routing order. Value can be one of the following:

- `specifiedNets`: Route nets to follow the order specified by the `-nets` collection
- `bottomUp` (the default): Route nets to follow the bottom-up order

Returns

<none>

Examples

```
rt::routeHierarchical -topDesign -nets <A0, A2, A1>
```

```
rt::routeHierarchical -topDesign -nets <A0, A2, A1> -order specifiedNets
```

See Also

- [Hierarchical Routing](#)

rt::shield

Description

Adds shielding automatically.

Syntax

```
rt::shield (-design <oaDesign> | -nets <{<oaGroup>} | {<oaNet>}>)
```

Arguments

`-design`

The design in which shielding is added.

`-nets`

Collection of nets or net groups to apply the shielding.

Returns

<none>

irt::addStripe

Interactive command to add stripes to router with added shapes

Syntax

```
irt::addStripe [<context: deContext | unsigned int>]
```

Returns

<none>

Arguments

`context`

Context of activation

irt::autoRoute

Description

Starts auto routing.

Syntax

```
irt::autoRoute [<context: deContext | unsigned int>]
```

Arguments

`context`

Context in which the command is activated: the design context or the ID of the window.

Returns

<none>

irt::autoShield

Description

Creates shields automatically.

Syntax

```
irt::autoShield [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the ID of the window.

Returns

<none>

irt::createRoute

Description

Creates routes interactively using the new Interactive Router.

Syntax

```
irt::createIRoute [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

irt::createP2PShield

Description

Creates point-to-point shields.

Syntax

```
irt::createP2PShield [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the ID of the window.

Returns

<none>

irt::createRoute

Description

Creates routes interactively.

Syntax

```
irt::createRoute [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the ID of the window.

Returns

<none>

irt::createRoutingArea

Description

Creates routing area.

Syntax

```
irt::createRoutingArea [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the ID of the window.

Returns

<none>

irt::createTopology

Description

Displays the Topology toolbar.

Syntax

```
irt::createTopology [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the ID of the window.

Returns

<none>

Examples

```
irt::createTopology [de::getActiveContext]
```

irt::deleteCustomRouteShapes

Description

Deletes shapes created by the Custom Router.

Syntax

```
irt::deleteCustomRouteShapes [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

irt::deleteMesh

Delete Mesh

Syntax

```
irt::deleteMesh [<context: deContext | unsigned int>]
```

Returns

<none>

Arguments

context

Context of activation

irt::deleteShield

Description

Deletes shield.

Syntax

```
irt::deleteShield [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the ID of the window.

Returns

<none>

irt::generateMeshRoute

Command for Mesh Route generation

Syntax

```
irt::generateMeshRoute [<context: deContext | unsigned int>]
```

irt::generateTrunk

Displays the **Trunk** toolbar for generating trunk routes.

Syntax

```
irt::generateTrunk [<context: deContext | unsigned int>]
```

Returns

<none>

Arguments

context

Context of activation.

See Also

- [Trunk Routing \(Manual Method\)](#)
- [rt::generateTrunk](#)

irt::previewPattern

Displays a preview of the route pattern.

Syntax

```
irt::previewPattern [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the ID of the window.

Returns

<none>

irt::query

Queries the Custom Router database.

Syntax

```
irt::query [<context: deContext | unsigned int>]
```

Arguments

context

Context in which the command is activated: the design context or the ID of the window.

Returns

<none>

Preferences

[Table 174 on page 2449](#) lists rt namespace general preferences and the Custom Router preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 174 rt Preferences

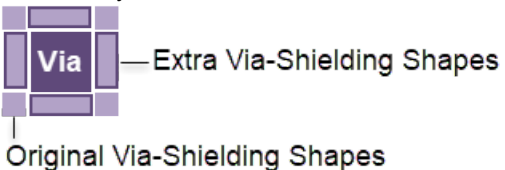
Name	Type	Scope	Default	Description
rtAddExtraShieldCenterVia	bool	cellview	false	Adds additional via-shielding shapes to ensure vias are fully enclosed.  Valid values: true, false

Table 174 rt Preferences (Continued)

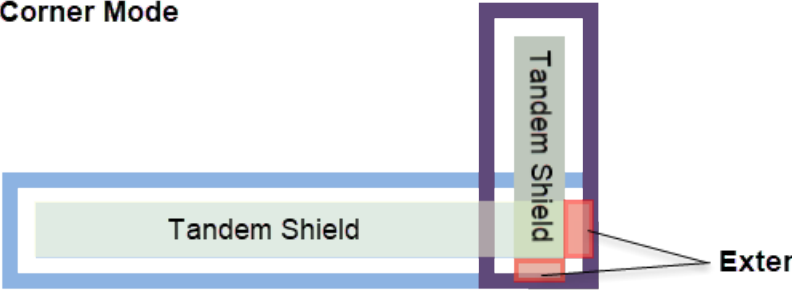
Name	Type	Scope	Default	Description
rtAlignCoaxialShield	string	cellview	on	Aligns coaxial shielding shapes with parallel shielding shapes: • Off : Does not align coaxial shield shapes with parallel shield shapes. • On : Aligns coaxial shield shapes with parallel shield shapes when possible. • Both : Aligns coaxial shield shapes with parallel shield shapes when bottom and top shield shapes are available. • Corner : Aligns coaxial shield shapes with parallel shield shapes. In addition, extends the tandem shield shapes at the corners. Corner Mode  Valid values: off, on, both, corner
rtAllowCreateSiderunSmallLoop	bool	cellview	false	Creates small side-run loops during auto routing to optimize length matching results. This preference is applicable when: • One or both of the following non-default routing constraints are defined: Length, Wire Matching. • The Wire Matching constraint specifies the Length or Length Per Layer style. • In the Auto Route dialog box Matching tab, the Lengthening Style is Endrun or Endrun Multi Layer. Valid values: true, false
rtAllowFillExtension	bool	cellview	true	Extends shapes during routing to ensure rule compliance. Valid values: true, false
rtAllowOffsetVia	bool	cellview	false	Aligns the connecting path and via using one of the following methods: • true: (via edge) Aligns the edge of the connecting path with the edge of the via. • false: (via center) Aligns the connecting path's centerline with the center of the via. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtAllowPartialSolution	bool	cellview	true	(Point-to-point interactive routing only) Routes to a location near the target when the target is not accessible. Valid values: true, false
rtAllowRedundantWelding	bool	cellview	false	Allows welding to connect pins while interactively routing. Valid values: true, false
rtAllowTaperUp	bool	cellview	false	Tapes up to pin width when NDR width is smaller than pin width if the value is true. The default is false so taper width can only be smaller or equal to pin width (taper down). Valid values: true, false
rtAllowViaAccessBoundary Pin	bool	cellview	false	Connects the routed wire to the pins at the edge of macros from the pin layer, or connect through a via to access the pin. Valid values: true, false
rtAllowWrongWayShield	bool	cellview	true	Controls if wrong way shields are allowed. If it is not allowed, only shields following the layer direction are created. Valid values: true, false
rtAllViasNonRotatable	bool	cellview	false	(Auto Route only) Always creates vias in the R0 orientation. Valid values: true, false
rtAntennaDetailedReport	bool	cellview	false	Generates a detailed report showing all nets affected by the antenna fix operation, in addition to the summary report. Valid values: true, false
rtAntennaFixViolations	bool	cellview	false	Fixes any process antenna violations on metal and cut layers. Valid values: true, false
rtAntennaReportFileName	string	cellview	antenna.txt	Antenna violation report file name. Valid values: <file name>
rtAssistedDeviceRouting	bool	cellview	false	Forces the Interactive Router to behave as if the following preferences were set to true without actually setting the preferences: rtAutoViaDrop, rtAutoWeldingMode, and rtAutoCloning. Valid values: true, false
rtAutoCloning	bool	cellview	false	Enables the Interactive Router to create duplicate pin connections on the same net. For details, see Cloning Routes. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtAutoDataSync	bool	cellview	true	Controls whether the router synchronizes with design changes (net, shape, constraint, and so on) immediately (true; default) or after the Auto Router starts (false). Valid values: true, false
rtAutoGroupInterleave	bool	cellview	true	Enables separate interleave when enabling auto grouping. Valid values: true, false
rtAutoGroupSize	string	cellview	off	Specifies the size used for auto-grouping when enabling planning feature. Valid values: auto, small, medium, large.
rtAutoRouteDialogMode	bool	user	false	Opens the Auto Route dialog box by default when you choose Routing > Auto Route . Valid values: true, false
rtAutoShieldDialogMode	bool	user	false	Opens the Add Shield dialog box by default when you choose Routing > Add Shield . Valid values: true, false
rtAutoTaperMode	string	cellview	Off	Specifies whether tapering should occur. If tapering should occur, the preference specifies whether tapering should occur at both ends of a connection or only at the start or end of the connection. You must click one end of the connection to specify the start or end. Valid values: off, both, start, end
rtAutoViaDrop	bool	cellview	false	Automatically adds vias to connect to same-net shapes below routing wires. Valid values: true, false
rtAutoViaDropDepth	int	cellview	1	Specifies the layer depth when automatically adding vias (rtAutoViaDrop).
rtAutoViaDropDirection	string	cellview	down	Adds via in the specified direction: down, up, or both. When adding the via, the tool looks for same-net shapes with respect to the current wire(s) being routed under the Auto Via Drop option. Valid values: down, up, both
rtAutoViaFlag	bool	cellview	true	Specifies whether the router automatically flags vias (changes orientation and offset) when required. Valid values: true, false
rtAutoWelding	bool	cellview	false	Enable auto welding Valid values: true, false

Table 174 *rt* Preferences (Continued)

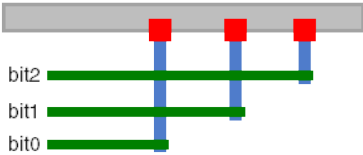
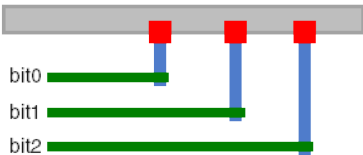
Name	Type	Scope	Default	Description
rtAutoWeldingMode	bool	cellview	false	Enables the Interactive Router to create branch connections (tap-offs) from the main interconnect (trunk) to neighboring same-net pins within the defined routing area. For details, see how to use welding during Follow-the-Cursor Routing. Valid values: true, false
rtBinSizeOverride	int	cellview	0	Option to override bin size to change data preparation memory usage.
rtBusCornerType	string	cellview	auto	Specifies how the bus bits are routed at corners. For details, see the Auto Route Bus/Diff Pair Tab. Valid values: auto, river, cross
rtBusFlipBitOrder	string	cellview	—	Specifies the order of the bus bits. You can use this preference to reverse the bit order. For example, reversing the bit order can prevent tap-off segments from crossing over the main trunk. Syntax: db::setPrefValue rtBusFlipBitOrder -value <bus_names> Example: db::setPrefValue rtBusFlipBitOrder -value "bus0 bus1 bus2" Original Order  Preference-Specified Order 
rtBusKeepPinSpacing	bool	cellview	false	Uses pin spacing as bus bit spacing. Valid values: true, false
rtBusRelativeMode	bool	cellview	false	Routes bits relative to the pin location so that after bending, the ends of the bits can cascade and not align. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtBusSplitEvenBits	bool	cellview	false	Controls auto bus splitting to leave an even number of nets between preroutes (power stripes). Valid values: true, false
rtBusSplitIgnoreWidth	float	cellview	0	Ignores preroute width less than specified value in auto bus split mode. Valid values: float (≥ 0)
rtBusTapOff	bool	cellview	true	Connects pins to bus routes. Valid values: true, false
rtBusTapoffFastMode	bool	cellview	false	Controls bus tap-off router using a faster algorithm to connect trunks to pins. Note that route quality might be impacted. Valid values: true, false
rtBusTapOffShielding	bool	cellview	true	Adds shielding on the tapoffs. If you do not select the option, only the bus trunk has shielding. Valid values: true, false
rtBusTapoffWithinUseLayer	bool	cellview	false	Forces bus tap-off to follow use_layer if set to true. Valid values: true, false
rtCloneMirrorCols	int	cellview	0	Flips the cloned routes horizontally about the y-axis after the specified number of rows. 0 turns off mirroring.
rtCloneMirrorRows	int	cellview	0	Flips the cloned routes vertically about the x-axis after the specified number of rows. 0 turns off mirroring.
rtCoaxialType	string	cellview	Solid	Specifies the coaxial shield type. Valid values: Solid, Split, Center Only
rtCoDesignViaSelectionMode	string	cellview	native	The via engine to use during routing. Valid values: native, vpe, icc
rtColumnNbCuts	int	cellview	1	The number of columns of cuts to add in the via.
rtConnectivityAware	bool	cellview	true	Enables the Auto Router to be connectivity aware during routing operations.
rtConnectPreroutes	bool	cellview	false	Enables the router to connect to existing 0x routes as if it were a 0x pin. Setting the preference to true favors connections to the path ends. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtConnectShield	bool	cellview	false	Connects shields to supply. Valid values: true, false
rtConnectShieldToAlignedPin	bool	cellview	false	Connects shields to VSS instance pins. Valid values: true, false
rtConnectWithinPin	string	cellview		Controls pin connection mode per layer: • Off – No pin connection requirements. • ViaAllPins – Any via connecting to any pin must be contained within the pin shape (via must be completely inside the pin, wires are allowed to route to/from the pin on the pin layer). • ViaWireAllPins – Any via or wire connecting to pins must be contained within the pin shapes (both vias and wires must be completely inside the pin). • ViaStandardCellPins – Any via connecting to standard cell pins must be contained within the pin shapes (used in mixed macro/stdcell routing to control different macro/stdcell pin access). • ViaWireStandardCellPins – Any via or wire connecting to standard cell pins must be contained within the pin shape (used in mixed macro/stdcell routing to control different macro/stdcell pin access). Valid values: Off, ViaAllPins, ViaWireAllPins, ViaStandardCellPin, ViaWireStandardCellPins Example: <code>db::setPrefValue rtConnectWithinPin -value {{m1 ViaAllPins} {m2 ViaAllPins} {m3 ViaAllPins}}} -scope [de::getActiveContext]</code>
rtControlShieldArrayViaByUser	bool	cellview	false	When Connect to Aligned Pin is selected, adds via arrays (true) or multiple optimal non-default rule vias (false). Valid values: true, false
rtConvergeUnroutesAllowHideBlockage	bool	cellview	true	Hides blockages during the ripup-reroute phase of routing. Valid values: true, false
rtConvergeUnroutesAllowHidePin	bool	cellview	true	Hides pins during the ripup-reroute phase of routing. Valid values: true, false
rtConvergeUnroutesAllowHidePowerMesh	bool	cellview	true	Hides power meshes during the ripup-reroute phase of routing. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtConvergeUnroutesAllowHidePreExistingShape	bool	cellview	true	Hides pre-existing shapes during the ripup-reroute phase of routing. Valid values: true, false
rtConvergeUnroutesAllowRelaxNDRSpacing	bool	cellview	true	Ignores non-default spacing rules during the ripup-reroute phase of routing. Valid values: true, false
rtConvergeUnroutesAllowRelaxNDRWidth	bool	cellview	true	Ignores non-default width rules during the ripup-reroute phase of routing. Valid values: true, false
rtConvergeUnroutesAllowSqueezeMode	bool	cellview	true	Uses the rule that permits 1 database unit of spacing around all shapes during the ripup-reroute phase of routing. Valid values: true, false
rtConvergeUnroutesEffortLevel	string	cellview	low	Controls the effort level of the ripup-reroute phase of routing: • Off: The fastest mode and runs only the initial routing phase, not allowing shorts or spacing conflicts. • Low: The default mode in which the initial routing phase is run, along with a number of converge passes, allowing spacing and shorts to previously routed connections. • Medium: Invokes additional converge passes that can allow spacing and shorts to more shapes around the source/target pins. • High: Allows spacing violations of shapes in a bigger window around the source and target pins. Valid values: off, low, medium, high
rtConvergeUnroutesReport	bool	cellview	false	Generates the report that shows the rules that were relaxed, obstructions that were hidden, the nets that were affected, and errors that occurred during the ripup-reroute phase of routing. Valid values: true, false
rtConvergeUnroutesReportFilename	string	cellview	gcr_converge_unroutes.log	The file name of the converge unroute report.

Table 174 rt Preferences (Continued)

Name	Type	Scope	Default	Description
rtCoverPin	string	cellview	off	Extends the route to cover the entire length of the pin or rectangular shape, regardless of where you clicked within the pin or rectangular shape boundary when starting or ending the route. If vias are required to connect to the pins, extending the route can result in via arrays. Specify one of the following: • off: Does not extend the route. • start: Extends the route at the starting shape only. • end: Extends the route at the ending shape only. • both: Extends the route at both the starting and ending shapes. rtCoverPin = off rtCoverPin = start, end, both Valid values: off, start, end, both
rtCoverPinWithVia	string	cellview	off	Valid values: off (default), both, start, and end. Controls if the pin should be covered by the routing Via or not.
rtCreateIRouteDialogMode	bool	user	false	(New Interactive Router) Displays the Create Route dialog box when you click Eject on the Route toolbar. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtCreateP2PShieldDialogMode	bool	cellview	false	Controls whether the P2P Shield dialog box opens by default when you choose GCR > P2P Shield . Valid values: true, false
rtCreateRouteDialogMode	bool	user	false	Displays the Create Route dialog box when you use the <code>irt::createRoute</code> command. Valid values: true, false
rtCustomRouteDesignType	string	CellView	default	Type of design to be routed. Valid values: default, digitalBlock, analogBlock
rtCutBlockagesAtPinsMinSpacing	float	cellview	0	Creates cutouts in pin blockages so that the router can access the pins during routing. You can specify the distance between the pins and the cutout, in micrometers (um). See the description of the Auto Route Advanced Tab.
rtDefaultOptionSetDir	string	cellview	<file path>	Specifies the default directory from which to load the default option set when initializing the router.
rtDefineNbCutsManually	bool	cellview	false	Lets you manually specify the number of via rows and columns when true. When false, calculates the number of via rows and columns automatically. Valid values: true, false
rtDeleteCustomRoute	string	cellview	All	Deletes specified shapes created by the Custom Router: All, Selected nets.
rtDeleteFloatingShield	bool	cellview	false	Deletes shields that are not connected to power and ground nets. Valid values: true, false
rtDeleteMeshModePref	string	cellview	Mesh and Vias	Deletes mesh mode setup.
rtDeleteMeshNetsPref	string	cellview	All	Deletes mesh for "All" or "Selected" nets.
rtDeleteShieldOption	string	cellview	All	Deletes shielding. Valid values: All, Selected
rtDeleteShortingWires	bool	cellview	false	Deletes wires created by the Custom Router that are causing short circuitry. Valid values: true, false
rtDeriveCellBound	bool	cellview	false	Redefines the cell boundary based on pin locations. Valid values: true, false
rtDiffPairTwisted	bool	cellview	false	Enables twist jumper on diff-pair. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtDiffPairTwistedInterval	float	cellview	0	Diff-pair twisted jumper interval, in microns.
rtDiffPairTwistedOffset	float	cellview	0	Diff-pair twisted jumper offset, in microns.
rtDiffPairTwistedStyle	string	cellview	orthogonal	Defines diff-pair jumper style.
rtDisableAVMRuleInShield	string	cellview	known	Turns off the specified rule checks during shielding operations: <code>known</code>: Disables all known rule checks. <code>all</code>: Disables all rule checks. <code>basic</code>: Disables BasicSpacing checks. <code>via</code>: Disables IsoDenseVia checks. <code>none</code>: Enables all rule checks. Valid values: known, all, basic, via, none
rtDisplayMessageOnCursor	bool	cellview	true	Shows a warning message when the pointer rests on a component on the layout canvas that might require adjustments. Valid values: true, false
rtDisplayRuleInfo	string	cellview	on	Displays rule violation information during routing. Valid values are: <code>on</code>: Displays rule names, and one or more rectangular boxes to visually indicate the areas where violations will occur. <code>off</code>: Does not display rule names or visual indicators. <code>box</code>: Displays one or more rectangular boxes to visually indicate the areas where violations will occur. Does not display rule names.

Table 174 *rt Preferences (Continued)*

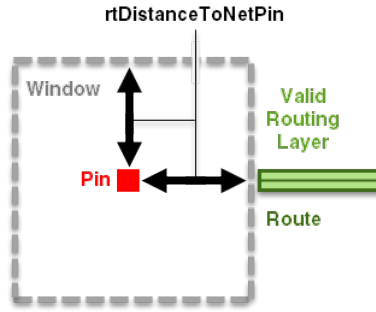
Name	Type	Scope	Default	Description
rtDistanceToNetPin	string	context		Specifies the net name and one of the following: - The distance between the edge of the pin and the edge of the valid routing layer in the context of the <code>rtPinEscapeMaxValidLayerMode</code> and <code>rtPinEscapeMinValidLayerMode</code> Tcl preferences. The default value is 10 average track pitches. - The distance between the edge of the pin and end of the dangling route in the context of the <code>rtPinEscapeControl</code> Tcl preference. By default, no window is created.
				 Make sure you set a value large enough so that the router can complete as many routes as possible. If <code>rtDistanceToNetPin</code> is too small, the window will be too close to the pin, and the router will not be able to connect to the pin.
rtEnablepackagingRouter	bool	cellView	false	Turns on the packaging design router. Valid values: true, false
rtEnableRules	string	cellview	all	Specifies the set of rules to be applied during interactive routing. Valid values: all, lvs, none
rtEnableRulesList	string	cellview		Specifies the set of rules to be applied
rtEnableRulesViewType	string	cellview	constraintCategory	Specifies the view type for displaying the rules
rtEnableWrongWayLimitForPinAccess	bool	cellview	false	Applies wrong way limit for pin access. Valid values: true, false
rtExpandedViaSetting	string	cellview		Controls the expanded via setting.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtExtendRouteToCoverBoundaryPin	bool	cellview	false	Extends route to cover the boundary pin, thus preventing LVS problems when pin shapes are not streamed out. Valid values: true, false
rtExtendShieldToEnclose	string	cellview	off	Adds shielding based on one of the following if a via does not fully cover the overlapping paths: • off: Does not consider the connecting shapes, and adds shielding to the paths only. • via: If the connecting vias are the same width as the paths, extends the shielding to enclose the vias. • pin: If the connecting pins are the same width as the paths, extends the shielding to enclose the pin. • both: If the connecting vias and pins are the same width as the connecting paths, extends the shielding to enclose the vias and pins. Valid values: off, via, pin, both
rtExtendToEndGrid	bool	cellview	false	Enables Custom Router commands to automatically extend all Custom Route shapes to match the track grids. Valid values: true, false
rtFingerPinLayer	string	cellview		Finger pin layer for memory routing
rtFlipTrackColorForNdrRouting	bool	cellview	false	If a route does not meet the layer's <code>minWidth</code> requirement, permits the route color to change so that it can be placed closer to a neighboring route. This preference is only applicable to the Auto Router. Valid values: true, false
rtFollowTrackSigType	bool	cellview	false	If enabled, the Custom Router will follow (or match) the <code>sigType</code> of tracks and nets exactly. Valid values: true, false
rtForceBiasChannelCenter	bool	cellview	false	Forces routing to bias towards channel center even if longer wire length is required. Valid values: true, false
rtForceInsideTrunkBox	bool	cellview	false	Forces the routing within the user-defined selection for trunk routing. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtForceTaperRuleInTaperWindow	bool	cellview	true	Forces the router to use only the taper rule within the taper window when set to true. Setting the preference to false uses both the NDR and taper rules within the taper window. The router will choose the taper width based on either the NDR or taper rule, whichever costs less. Valid values true, false
rtGenerateMeshRouteAreaXmax	float	cellview	0	Mesh Route area maximum x value.
rtGenerateMeshRouteAreaXmin	float	cellview	0	Mesh Route area minimum x value.
rtGenerateMeshRouteAreaYmax	float	cellview	0	Mesh Route area maximum y value.
rtGenerateMeshRouteAreaYmin	float	cellview	0	Mesh Route area minimum y value.
rtGenerateMeshRouteCreatePinShapes	bool	cellview	false	Create New Pin Shape Option
rtGenerateMeshRouteDialogMode	bool	user	true	The <code>irt::generateMeshRoute</code> command options dialog eject state. Valid values: true, false
rtGenerateMeshRouteInsertVias	bool	cellview	true	Insert Vias option.
rtGenerateMeshRouteInstanceCellName	string	cellview	meshRoute	Instance Cell Name input values
rtGenerateMeshRouteInstanceViewName	string	cellview	layout	Instance View Name input values
rtGenerateMeshRoutePinShapePurposeNames	string	cellview	pin	New Pin Shape Purpose Name Option
rtGenerateMeshRouteDirty	bool	cellview	false	The Dirty mode option. Valid values: true, false
rtGenerateMeshRouteHierarchyGroup	bool	cellview	true	Creates Hierarchy option.
rtGenerateMeshRouteHierarchyGroupOption	string	cellview	FigGroup	Create Hierarchy group option.
rtGenerateMeshRouteHookupNets	string	cellview		Nets for hookup mesh route.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtGenerateMeshRouteHookupSelectTable	string	cellview		Layer table for hookup mesh route.
rtGenerateMeshRouteHookupTabEnable	bool	cellview	false	Hookup Tab Enable option. Valid values: true, false
rtGenerateMeshRouteInsertVias	bool	cellview	true	Insert Vias option. Valid values: true, false
rtGenerateMeshRouteKeepBoundary	bool	cellview	true	Keep Boundary option. Valid values: true, false
rtGenerateMeshRouteKeepViasOnBoundary	bool	cellview	true	Keep vias on Boundary option.
rtGenerateMeshRouteLayerSelectTable	string	cellview		Layers selection table and pull back values.
rtGenerateMeshRouteMeshTabEnable	bool	cellview	true	Mesh Tab Enable option. Valid values: true, false
rtGenerateMeshRouteRegion	string	cellview	PR Boundary	Mesh Route Region option.
rtGenerateMeshRouteRegionPoints	string	cellview		Region Points for Rectangle and Polygon.
rtGenerateMeshRouteRemoveFloating	bool	cellview	false	Remove Floating Path option. Valid values: true, false
rtGenerateMeshRouteSnapTrack	bool	cellview	true	Snap Track option. Valid values: true, false
rtGenerateMeshRouteTrackRegion	string	cellview		Track Region Name selection.
rtGenerateMeshRouteViaRules	string	cellview		Layer table for via rules
rtGenerateTrunkAlign	string	cellview	All	Specifies the kind of pins to be considered when calculating the location to start the search: Default: All
rtGenerateTrunkAutoConnect	bool	cellview	false	Creates vias between newly created trunks and existing trunks or pins. Valid values: true, false
rtGenerateTrunkBegin	float	cellview	0	Specifies the starting point of the trunk.
rtGenerateTrunkCoordinate	float	cellview	0	Specifies the coordinate to start the search for generating trunk routes.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtGenerateTrunkCoverage	string	cellview	Full	Specifies whether full trunk coverage is needed. Default: Full
rtGenerateTrunkDialogMode	bool	user	false	Displays the Generate Trunk dialog box from the Trunk toolbar. You display the Trunk toolbar using the irt::generateTrunk command. Valid values: true, false
rtGenerateTrunkEnd	float	cellview	0	Specifies the stopping point of the trunk route.
rtGenerateTrunkExcludeInstances	string	cellview		Excludes the specified list of instances and pins during trunk routing.
rtGenerateTrunkExtensionSide	string	cellview	PinEdge	Specifies the way to extend the trunk route. Default: PinEdge
rtGenerateTrunkFreedom	float	cellview	0	Enlarges the search box by the specified value. Default: 0.0
rtGenerateTrunkIncludeInstances	string	cellview		Includes the list of specified instances and only pins in these instances during trunk routing.
rtGenerateTrunkInstance	bool	cellview	false	Enables instance filter options. Default: false Valid values: true, false
rtGenerateTrunkLayer	string	cellview		Specifies the routing layer to use for trunk routing.
rtGenerateTrunkNum	int	cellview	1	Specifies the number of trunks to create.
rtGenerateTrunkOverride	bool	cellview	false	Overrides existing trunk. Default: false Valid values: true, false
rtGenerateTrunkRatio	int	cellview	100	Minimal percentage of required trunk coverage. Default: 100
rtGenerateTrunkShareTrack	bool	cellview	false	Enables track-sharing. Default: false Valid values: true, false
rtGenerateTrunkShieldCoverage	string	cellview	Full	Adds full shielding coverage. Default: Full
rtGenerateTrunkShieldEnable	bool	cellview	false	Enables the shield options. Default: false Valid values: true, false
rtGenerateTrunkShieldNet	string	cellview		Shielding net name. Default:
rtGenerateTrunkShieldRatio	int	cellview	100	Minimal percentage of required shielding coverage. Default: 100
rtGenerateTrunkShieldSide	string	cellview	Any	Side to add parallel shielding. Default: Any

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtGenerateTrunkStartSide	string	cellview	Center	Specifies where to start the search. Default: Center
rtGenerateTrunkTrackIndexes	string	cellview		Track indexes.
rtGenerateTrunkTrackSelection	string	cellview	Any	Specifies whether any track can be used, or only specific tracks which match the track indexes can be used. Default: Any
rtGenerateTrunkTrimLengthBy	string	cellview	<file path>	Enables the low and high trimming options or driver and receiver trimming options.
rtGenerateTrunkTrimLengthDriver	float	cellview	0	Length the trunk to trim or grow on the driver side.
rtGenerateTrunkTrimLengthEnable	bool	cellview	false	Enables the trimming options. Default: false Valid values: true, false
rtGenerateTrunkTrimLengthHigh	float	cellview	0	Length of the trunk to trim or grow at the high end.
rtGenerateTrunkTrimLengthLow	float	cellview	0	Length the trunk to trim or grow at the low end.
rtGenerateTrunkTrimLengthReceiver	float	cellview	0	Length of the trunk to trim or grow on the receiver side.
rtGenerateTrunkWidth	float	cellview	0	Specifies the width of trunk.
rtGlobalRouteFlexiblePinEscape	bool	cellview	false	Boolean input to enable global router to relax pin escape distance. Valid values: true, false
rtIgnoreForbiddenETELayers	string	cellview	*	Specifies the layers on which allowedEndToEndSpacing rule should be ignored
rtIgnoreSameNetDRCs	bool	cellview	false	Ignores Same Net DRCs during interactive routing. Valid values: true, false
rtIgnoreTracksInRuleRegion	string	cellview		LPPs associated with region constraints where tracks are be disabled.
rtImportExcludeCells	string	cellview		Option to exclude cells during import
rtImportPartialLayers	string	cellview		Imports only the shapes on the specified layers to improve data preparation performance.
rtImportPolygonEdgeStep	float	cellview	0	Width of rectangles into which a non-axis-oriented polygon is split during data preparation.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtInsertCutMetalPatchAreaSelection	string	cellview	Design	The type of area to insert cut metal patch.
rtInsertCutMetalPatchColorSelection	string	cellview	All	The type of color selection.
rtInsertCutMetalPatchDialogMode	bool	user	true	The <code>irt::insertCutMetalPatch</code> command options dialog eject state. Valid values: true, false
rtInsertCutMetalPatchExtraOptions	string	cellview		The manual options.
rtInsertCutMetalPatchInputManualOptions	bool	cellview	false	Input manual options. Valid values: true, false
rtInsertCutMetalPatchLayerSelection	string	cellview	All	The type of layer selection.
rtInsertCutMetalPatchNetsSelection	string	cellview	All	The type of nets selection.
rtInsertCutMetalPatchRegionPoints	string	cellview		The region to insert cut metal patch.
rtInsertCutMetalPatchSelectedColors	string	cellview		The selected colors.
rtInsertCutMetalPatchSelectedLayers	string	cellview		The selected layers.
rtInsertCutMetalPatchSelectedNets	string	cellview	All	The selected nets.
rtInsertCutMetalPatchSetFixedStatus	bool	cellview	false	Set the inserted patches to fixed status or not. Valid values: true, false
rtInsertCutMetalPatchStage	string	cellview	Preprocess pin shapes	The stage of insert cut metal patch.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtInterGroupATCConnection Method	string	cellview	topHVLa yers	The trunk and spine layers that can be used to connect device groups during topology routing: • <code>topLayer</code>: Only the trunks and spines on the topmost layer in one group can connect to trunks and spines on the topmost layer in another group. • <code>topHVLayers</code>: Only the horizontal trunks or spines on the topmost layer in one group can connect to the vertical trunk or spine on the topmost layer in another group. • <code>anyTrunkAndSpine</code>: The trunks or spines on any layer in one group can connect to the trunks or spines on any layer in another group. Valid values: <code>topLayer</code>, <code>topHVLayers</code>, <code>anyTrunkAndSpine</code>
rtIntraShieldPlacement	string	cellview	interle ave	The method in which the bus trunk is to be shielded: • <code>outside</code>: Places shields on the outside of the trunk. • <code>interleave</code>: Inserts shared shielding between two adjacent bits. • <code>doubleInterleave</code>: Shields each bit individually. Valid values: <code>outside</code>, <code>interleave</code>, <code>doubleInterleave</code>
rtIRBus	bool	cellview	false	Bus routing during follow-the-cursor interactive routing. Valid values: <code>true</code>, <code>false</code>
rtIRFreeSpace	bool	cellview	false	Turns on the free-space follow-the-cursor interactive routing. Use this method to start routing from anywhere on the canvas. Turn off this preference to start routing only from a pin. Valid values: <code>true</code>, <code>false</code>
rtIRP2P	bool	cellview	false	Turns on point-to-point interactive routing. Valid values: <code>true</code>, <code>false</code>
rtIRStrap	bool	cellview	false	Turns on strap interactive routing. Valid values: <code>true</code>, <code>false</code>
rtIRStrapAllowStackedVia	bool	cellview	false	Allow stacked via during IR Strap

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtLayerCost	string	cellview		(Interactive Router only) Controls layer length cost at per layer basis. <pre>rtLayerCost { {<layer1> <userCost1>} {<layer2> <userCost2>} ... {<layerN> <userCostN>} }</pre> where • <code>layer_n</code> is the name of the layer. • <code>userCost_n</code> is the cost value of the layer. Valid values are neutral, high, and low. If you do not specify any value, the router assumes a cost of neutral for all layers.
rtLayerCostAR	string	cellview		(Auto Router only) Controls layer length cost at per layer basis. <pre>rtLayerCostAR { {<layer1> <userCost1>} {<layer2> <userCost2>} ... {<layerN> <userCostN>} }</pre> where • <code>layer_n</code> is the name of the layer. • <code>userCost_n</code> is the cost value of the layer. Valid values are 0 to 100, where 0 costs the least and 100 costs the most. The Auto Router will use the lower cost layers more than the higher cost layers. To use all of the layers equally, assign all layers a cost of 0. In the following example, the Auto Router will use layers m1 and m2 most of the time, and layers m3 and m4 sometimes. The Auto Router will infrequently use layers m5, m6, m7, and m8. <pre>rtLayerCostAR { {m1 0} {M2 0} {m3 50} {M4 50} {m5 100} {M6 100} {m7 100} {M8 100} }</pre> If you do not specify any value, the router assumes a cost of 0 for all layers.
rtLayerCutClass	string	cellview		Controls layer cut class per layer.
rtLayerDirection	string	cellview		Controls layer direction per layer.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtLayerGridTrack	string	cellview		Specifies whether the route and corresponding parallel shields, jumpers for parallel shields, and bus shields will snap to active wire tracks or routing grid. You must specify the layer, and then one of the following: • on: Snaps routes and shields to active tracks. If the layer does not have active tracks, the routes and shields will snap to the routing grid. If you choose this option, the Cost value is set to 4 automatically, and you cannot change it. • off: Does not snap routes and shields to active tracks or grids. Choose this option for gridless routing. • <costValue>: Snaps routes and shields to active tracks. However, the routes can move off tracks and grids to connect to off-track or off-grid pins. Specify a costValue between 4 and 20, inclusive. The higher the value, the more likely the route will remain on the track or grid. If tracks and routing grids are not defined and you specify On or <costValue>, the Auto Router uses an internally determined grid. The grid pitch is route width + minSpacing between shapes. The grid offset is 0 or pitch/2, whichever permits more pins on the grid. Syntax: <pre>{{layerName [on off n]}}, where n is an integer between 4 and 20, inclusive}</pre> To specify a global value that applies to all layers, replace the layerName with the track adherence value. For example: <pre>rtLayerGridTrack -value {on {M2 off} {M3 4}}</pre>
rtLayerSpacing	string	cellview		Controls layer spacing at per layer basis.
rtLayerValid	string	cellview		Controls valid layer at per layer basis.
rtLayerViaDef	string	cellview		Controls layer via definition at per layer basis.
rtLayerWidth	string	cellview		Controls layer width at per layer basis.
rtMainModePointToPoint	bool	cellview	false	Uses the point-to-point interactive routing mode. Valid values: true, false
rtMatchBox	string	cellview		Specifies the boundary box for wire length matching. Valid values: {x0 y0} {x1 y1}

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtMatchCenterChannel	bool	cellview	false	Option to control the loops created by match lengthening to be in the center of routing channel if the value is true. Valid values: true, false
rtMatchShieldStyle	integer	cellview	1	Option to control the shielding style for the loops created by match lengthening. • 0: No shield between loops. • 1: One shield between lengthen loops. • 2: Two shields between lengthen loops. Valid values: 0, 1, 2
rtMaxCloning	int	cellview	64	The maximum number of clones.
rtMaxPinNumRoutePGLimit	int	cellview	30	Sets maximum number of pins that a PG net without supply network can be routed as signal net.
rtMaxRoutingTime	int	cellview	0	Maximum routing time. Routing operations are stopped when the specified maximum time is reached.
rtMaxShapesForCloning	int	cellview	64 000	The maximum number of shapes permitted for cloning.
rtMaxTopologyStripeNum	int	cellview	5	The maximum number of stripes in the topology route segment.
rtMaxWeldingConnections	int	cellview	256	The maximum number of welding connections.
rtMaxWrongWayPinAccessLength	string	cellView		Set layer-specific wrong way length for pin access.
rtMinConnectedShieldLength	float	cellview	0	The minimum shield length threshold. If the signal wire length is less than this value, shield wires are not created.
rtMinimizeCross	bool	cellview	false	Enables cross-minimization routing. Valid values: true, false
rtMinimizeCrossCompactRouting	bool	cellview	false	Enables compact routing for cross minimization. Valid values: true, false
rtMinimizeCrossTraverseBlockShape	bool	cellview	true	Enables traverse shapes inside blocks or just use block boundary. Valid values: true, false
rtMinimumWidthIncrement	float	cellview	0.002	Specifies the width increment when using Ctrl+scroll. The default value is two times the manufacturing grid.
rtMinJumperLength	float	cellview	0	Minimal jumper length value.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtMinOptimalLength	float	cellview	0	Minimal optimal routing length value.
rtMinShieldSupply	float	cellview	0	The length of a shield line needs to be longer than min_shield_to_supply before it can be tied to supply. Valid values: float (≥ 0)
rtNetPriorityDefFile	string	cellview		Path of the file that contains via net routing priority and the associated layer priorities.
rtNetsSelection	string	cellview	all	Nets to be routed: all, or selected. Valid values: all, selected
rtNumProbes	integer	cellview	200000	Option to set probe number for routing. The more probe number is set, the more detail searching and longer runtime will be taken. Valid values: integer (> 0)
rtOptimizeMPTColor	bool	cellview	true	Optimizes shape color assignment for multiple-patterning. Valid values: true, false
rtOptionSet	string	cellview		The option to control active option set.
rtP2PShieldMultipleNets	bool	cellview	false	Enables multiple nets support in bus routes during point-to-point shielding. Valid values: true, false
rtPackagingAllowShort	bool	cellview	true	Permits shorts in packaging routing. Valid values: true, false
rtPackagingAutoCreatePlaneBoundary	bool	cellview	true	Automatically creates PG plane boundaries. Valid values: true, false
rtPackagingD2dBumpArrangement	string	cellview	array	Die-to-die bump arrangement.
rtPackagingD2dRegionLpp	string	cellview		Die-to-die (D2D) region LPP.
rtPackagingD2dShieldingNets	string	cellview		Nets for die-to-die shielding.
rtPackagingD2dUseShielding	bool	cellview	false	Adds shielding in the die-to-die (D2D) region. Valid values: true, false
rtPackagingD2dUseTandem	bool	cellview	false	Adds tandem shielding in the die-to-die (D2D) region. Valid values: true, false
rtPackagingDie2DieCreateShieldingProperty	bool	cellview	true	Creates die-to-die shielding property. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtPackagingDie2DieCreateShieldingShapes	bool	cellview	true	Creates die-to-die shielding shapes. Valid values: true, false
rtPackagingDie2DieEscapeStyles	string	cellview		Specifies the die-to-die (or via) escape styles. By default, vias are in a staggered pattern when stacked. This preference lets you specify custom styles.
rtPackagingDieboxRegionLpp	string	cellview		Silicon die box (DIEBOX) region LPP.
rtPackagingExtendPlaneBoundaryNets	string	cellview		Extends PG plane boundaries for the specified nets.
rtPackagingFanoutMultipleVia	string	cellview		Creates multiple vias for fanout bumps.
rtPackagingFoboxRegionLpp	string	cellview		Fanout box (FOBOX) region LPP.
rtPackagingLdiLayers	string	cellview		Local die interconnect (LDI) layers.
rtPackagingLdiRegionLpp	string	cellview		Local die interconnect (LDI) region LPP.
rtPackagingPartialViaWithinBumpAdditionalEscapeDistance	string	cellview		Partial via within bump escape distance.
rtPackagingPlaneBoundaryNetLayers	string	cellview		List of layers on which PG plane boundaries can be created.
rtPackagingPushRouteAllowCreateTaper	bool	cellview	true	Allows Push Route to create tapers if tapering does not exist. Otherwise, Push Route would only modify existing tapers. Valid values: true, false
rtPackagingPushRouteOnlyCommitCleanResults	bool	cellview	false	Specifies if Push Route only commits routing results when DRC is clean (except skipped violations) on modified shapes. Valid values: true, false
rtPackagingPushRouteTimeout	int	cellview	5	Pushes route timeout in seconds. Interrupts if running time exceeds for each single run. Use 0 to turn off.
rtPackagingRoutePGFrontsideBumpToNearestBacksideBump	bool	cellview	false	Routes power and ground (PG) frontside bump to nearest backside bump. Valid values: true, false
rtPackagingShortRouteLayers	string	cellview		Short route layers.
rtPackagingSupplyPlaneLpp	string	cellview		Supply plane guide LPP.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtPackagingUseD2dRegionLpp	bool	cellview	false	Uses the die-to-die (D2D) region on the specified LPP. Valid values: true, false
rtPackagingUseDieboxRegionLpp	bool	cellview	false	Uses the die box (DIEBOX) region on the specified LPP. Valid values: true, false
rtPackagingUseFoboxRegionLpp	bool	cellview	false	Uses the fanout box (FOBOX) region on the specified LPP. Valid values: true, false
rtPackagingUseLdiRegionLpp	bool	cellview	false	Uses the local die interconnect (LDI) region on the specified LPP. Valid values: true, false
rtPackagingUseRoundPath	bool	cellview	true	Adds paths that have round end styles (lePathStyle). To add paths that have other end styles, such as truncate end style, set the preference to false. Valid values: true, false
rtPackagingUseShortRoute	bool	cellview	false	Connects neighboring micro bumps (pins) on the specified layers to reduce the number of escape vias, thus saving routing resources. The Packaging Router only connects the micro bumps using single any-angle straight paths. Valid values: true, false
rtPackagingUseSupplyPlane	bool	cellview	false	Uses the supply plane guides on the specified LPP. Valid values: true, false
rtPackagingUseSupplyRoute	bool	cellview	false	Routes selected supply nets. Valid values: true, false
rtPatternRouteAbutPolyContact	bool	cellview	false	Abuts the poly contact against the poly pin. Set the preference to false if you want the poly contact to be placed over the poly pin. Valid values: true, false
rtPatternRouteAssistantAddRoutesIntoGroup	bool	cellview	true	Adds routes into group. Valid values: true, false
rtPatternRouteAssistantAddSinglePinNets	bool	cellview	false	Add nets with a single pin to the Pattern Route Dashboard. Valid values: true, false
rtPatternRouteAssistantAllowPatternOverDevice	bool	cellview	true	Allows pattern over device. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtPatternRouteAssistantAllowRouteOverDevice	bool	cellview	true	Specifies whether route over the device is allowed. Valid values: true, false
rtPatternRouteAssistantAllowTapoffJog	bool	cellview	false	Specifies whether to allow jog for tapoff connection. Valid values: true, false
rtPatternRouteAssistantCreateRoutesDuringRowSpacingUpdate	bool	cellview	true	Creates routes when updating the row spacing. Valid values: true, false
rtPatternRouteAssistantCreateSpine	bool	cellview	true	Defines if the spines connecting the trunks/spines should be created when the route is being realized. Valid values: true, false
rtPatternRouteAssistantCustomPatternFile	string	cellview		Specifies the name and path of the custom route pattern file.
rtPatternRouteAssistantDirtyMode	bool	cellview	false	Routes the pattern without checking design rule compliance. Valid values: true, false
rtPatternRouteAssistantDisplayAllPatterns	bool	cellview	false	Shows all the patterns in the catalog. Valid values: true, false
rtPatternRouteAssistantDisplayCustomPatternsOnly	bool	cellview	false	Shows only the custom patterns in the catalog in the Pattern Route Assistant. Valid values: true, false
rtPatternRouteAssistantEMAware	bool	cellview	false	Specifies whether pattern routing is EM-aware. Valid values: true, false
rtPatternRouteAssistantExcludeIntermittentPatterns	bool	cellview	false	Shows or hides the intermittent patterns in the catalog. Valid values: true, false
rtPatternRouteAssistantExcludeMatchPatterns	bool	cellview	false	Shows or hides the match patterns in the catalog. Valid values: true, false
rtPatternRouteAssistantExtractNoSpine	bool	cellview	false	Specifies whether pattern extraction should stop at the trunks and skip the spines. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtPatternRouteAssistantSpinePosition	bool	cellview	default	Determines how spines are placed: <code>default</code>: Considers all possible spine positions and honors the layer pitch requirements. <code>gateOnly</code>: Considers only the spine positions that overlap with the gates on poly layers. <code>sdOnly</code>: Considers only the spine positions that overlap with source and drain pins. <code>terminalAware</code>: Considers spine positions that overlap with gates, source pins, and drain pins first, and then evaluates the spacing between terminals. Valid values: default, gateOnly, sdOnly, terminalAware
rtPatternRouteAssistantForceGroupSymmetry	bool	cellview	false	Forces the routes to be symmetric within the channel. Valid values: true, false
rtPatternRouteAssistantForceGroupSymmetryOverDevices	bool	cellview	false	Considers the number of trunks to be routed in an area and ensure that routes are distributed evenly over devices. Valid values: true, false
rtPatternRouteAssistantGateCoverageOfContacts	int	cellview	100	Specifies gate coverage of contacts.
rtPatternRouteAssistantGroup	string	cellview		Pattern route placement groups.
rtPatternRouteAssistantGroupSettingsFile	string	cellview		Specifies the group settings file.
rtPatternRouteAssistantImportCustomOverwrite	bool	cellview	true	Overwrites existing custom pattern in the catalog. Valid values: true, false
rtPatternRouteAssistantMakePatternDescriptionEditable	bool	cellview	false	Make Pattern description editable. Valid values: true, false
rtPatternRouteAssistantNets	string	cellview		Pattern route nets.
rtPatternRouteAssistantNewCustomName	string	cellview		Specifies the custom pattern name.
rtPatternRouteAssistantOverDeviceTrunkTrunkSpacing	float	cellview	0	Specifies the over device trunk-to-trunk spacing.
rtPatternRouteAssistantOverDeviceViaMaximization	bool	cellview	false	Specifies whether to maximize the via for the over-device connection Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtPatternRouteAssistantPattern	string	cellview		Route pattern description.
rtPatternRouteAssistantPlanSelectedNetsOnly	bool	cellview	false	Enables the Group Planner to plan routes for selected nets only.
rtPatternRouteAssistantPreservePattern	bool	cellview	true	Prevents the pattern route planner from changing the existing patterns and attributes except for trunk positions. Valid values: true, false
rtPatternRouteAssistantRemoveConstraintAlongWithRoute	bool	cellview	false	Specifies whether or not to remove route pattern constraints when deleting the routes. Valid values: true, false
rtPatternRouteAssistantRemoveExistingRoutes	bool	cellview	true	Removes existing routes. Valid values: true, false
rtPatternRouteAssistantRemoveGroupConstraintAlongWithRoute	bool	cellview	false	Specifies whether to remove the group constraint with route. Valid values: true, false
rtPatternRouteAssistantRemoveRouteAndConstraintBeforePlanning	bool	cellview	true	Removes existing route and route pattern constraint before planning. Valid values: true, false
rtPatternRouteAssistantRouteSelectedNets	bool	cellview	false	Specifies whether to route selected nets only. Valid values: true, false
rtPatternRouteAssistantRowSpacing	string	cellview	1	Specifies the row spacing.
rtPatternRouteAssistantRowSpacingNumFins	string	cellview	10	Specifies the number of fins for row spacing.
rtPatternRouteAssistantRowSpacingType	string	cellview	Microns	Row spacing type. Valid values: Microns, Fins
rtPatternRouteAssistantSettingScope	string	cellview	scopeGroup	Specifies the scope of settings to be saved or loaded.
rtPatternRouteAssistantShareTracks	bool	cellview	false	Enables track sharing. Valid values: true, false
rtPatternRouteAssistantShareTracks	bool	cellview	false	Permits routes to share tracks during group planning. Valid values: true, false
rtPatternRouteAssistantShowPatternDescription	bool	cellview	false	Show route pattern description. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtPatternRouteAssistantSpineOnly	bool	cellview	false	Defines if only the spines should be created to connect trunks predefined inside PCells. Valid values: true, false
rtPatternRouteAssistantSpineSpineSpacing	float	cellview		Specifies the spine to spine spacing.
rtPatternRouteAssistantStripesVisible	bool	cellview	false	Creates only spines to connect trunks that are predefined inside PCells. Valid values: true, false
rtPatternRouteAssistantTaperTapoffToPinWidth	bool	cellview	false	Tapers the tapoff width to match the width of the connecting pin. Valid values: true, false
rtPatternRouteAssistantTapoffClusterNum	string	cellview	auto	Specifies the number of pins for tapoff cluster.
rtPatternRouteAssistantTapoffClusterOver	bool	cellview	false	Specifies if the tapoff cluster bone should be created over the pins. Valid values: true, false
rtPatternRouteAssistantTapoffClusterTrunkOver	bool	cellview	false	Specifies if the tapoff cluster trunk should be created over the pins. Valid values: true, false
rtPatternRouteAssistantTrunkDeviceSpacing	float	cellview	0	Specifies the spacing between trunk and device.
rtPatternRouteAssistantTrunkDeviceSpacingRef	string	cellview	default	Side of the device channel from which device-trunk spacing is determined: <code>default</code>: Both the top and bottom sides of the channel. <code>top</code>: Top side of the channel. <code>bottom</code>: Bottom side of the channel. Valid values: none, top, bottom
rtPatternRouteAssistantTrunkSpacingEnable	bool	cellview	false	Controls the trunk spacing. Valid values: true, false
rtPatternRouteAssistantTrunkTrunkSpacing	float	cellview	0	Specifies the trunk to trunk spacing.
rtPatternRouteAssistantViaMaximization	bool	cellview	false	Specifies whether to maximize the via for the source/drain coverage Valid values: true, false
rtPatternRouteIgnorePreRoute	bool	cellview	false	Turns off existing-route check during pattern routing. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtPatternRouteIncreaseTrackByFactor	int	cellview	1	Increases the number of tracks by the specified factor. For example, if the existing number of tracks is 10, specifying 2 increases the number of tracks to 20, and specifying 3 increases the number of tracks to 30. Set this preference if you want to move a pattern route component to a location that currently does not have a track. See Moving Pattern Components.
rtPatternRoutePreviewDisplayTapoff	bool	cellview	false	Specifies whether to display the tapoff segments during pattern preview. Valid values: true, false
rtPatternRoutePreviewUseActualLayer	bool	cellview	false	Displays the preview routes on the layers that will actually be used during routing. Valid values: true, false
rtPatternRouteRealizerType	string	cellview	standalone	Parameter to specify pattern route realizer: sroute, or standalone.
rtPatternRouteSymTrunksInChannel	bool	cellview	false	Forces the number of trunk positions to be symmetric within the channel. Valid values: true, false
rtPatternRouteTaperPolyTapoffByDefault	bool	user	true	Taper the poly tapoffs to pin width by default. Valid values: true, false
rtPinEscapeControl	rtPinEscapeControl	user	default	Specifies how to route to pins when access is blocked: <code>reportBlocked</code>: Runs pseudo routing with all user constraints and router settings. Adds markers and reports blockages that prevent pin access. Does not create actual routes. You might want to use this option to assess the number of pin blockages in the design before spending time creating actual routes. <code>forceIfBlocked</code>: Relaxes rules at blocked pins to permit connections. <code>skipIfBlocked</code>: Routes close to blocked pins. Adds markers between the pins and the dangling routes. You might want to use this option to route most of the net automatically, leaving a short segment to route interactively using either the Interactive Route or Create Interconnect command. <code>default</code>: Routes to blocked pins when pin escape succeeds. Valid values: default, reportBlocked, forceIfBlocked, skipIfBlocked,

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtPinEscapeMaxValidLayerMode	string	cellview	strict	Permission to use non-routing layers that are above the maximum valid routing layer. Specify one of the following permission modes: <code>auto</code>: Permits routing above the maximum valid routing layer only when necessary. The router will always try to use a layer closest to the maximum valid routing layer. To control the cost routing above the maximum valid routing layer, use the <code>rtPinEscapeMaxValidLayerModeAutoCost</code> Tcl preference. <code>strict</code>: Prohibits routing above the maximum valid routing layer. <code>window</code>: Permits routing to or from a pin using a layer above the maximum valid routing layer within the specified area (window). Use this mode when the pin is located on a non-valid routing layer. The window encloses the pin, and the default distance from the edge of the pin to the edge of the area is 10 average track pitches. To change the distance, use the <code>rtDistanceToNetPin</code> Tcl preference. Valid values: auto, strict, window
rtPinEscapeMaxValidLayerModeAutoCost	string	cellview	medium	Specifies the cost of routing above the maximum valid routing layer: <code>low</code>: The route contains long paths and fewer turns. <code>medium</code>: The route contains shorter paths and more turns than those that are low cost, thus increasing the signal travel time. <code>high</code>: The route contains shorter paths and more turns than those that are medium cost, thus increasing the signal travel time. Valid values: low, medium, high

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtPinEscapeMinValidLayerMode	string	cellview	auto	Permission to use non-routing layers that are below the minimum valid routing layer. Specify one of the following permission modes: <code>auto</code>: Permits routing below the minimum valid routing layer only when necessary. The router will always try to use a layer closest to the minimum valid routing layer. To control the cost routing below the minimum valid routing layer, use the <code>rtPinEscapeMinValidLayerModeAutoCost</code> Tcl preference. <code>strict</code>: Prohibits routing below the minimum valid routing layer. <code>window</code>: Permits routing to or from a pin using a layer below the minimum valid routing layer within the specified area (window). Use this mode when the pin is located on a non-valid routing layer. The window encloses the pin, and the default distance from the edge of the pin to the edge of the area is 10 average track pitches. To change the distance, use the <code>rtDistanceToNetPin</code> Tcl preference. Valid values: auto, strict, window
rtPinEscapeMinValidLayerModeAutoCost	string	cellview	medium	Specifies the cost of routing below the minimum valid routing layer: <code>low</code>: The route contains long paths and fewer turns. <code>medium</code>: The route contains shorter paths and more turns than those that are low cost, thus increasing the signal travel time. <code>high</code>: The route contains shorter paths and more turns than those that are medium cost, thus increasing the signal travel time. Valid values: low, medium, high
rtPinShapeIgnoreLayer	string	cellview	"" (any legal metal layer name)	Ignores pin shapes that are at lower layers than the specified layer.
rtPinTrunkOffset	float	cellview	0	Controls the offset of auto-generated bus virtual pin. Valid values: float (≥ 0)
rtPkgAutoRouteDialogMode	bool	user	false	Opens the Auto Route dialog box. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtPostLayoutSpeedup	bool	cellview	true	Postpones extracting connectivity during router data model loading (true; default). If the value is false, router will extract connectivity immediately during data model loading. Valid values: true, false
rtPreciseCursorTarget	bool	cellview	false	Enables multi-layer follow-the-cursor cursor target. Valid values: true, false
rtPreciseP2PGuidance	bool	cellview	false	Strict interpretation of P2P guidance (no margin for wire placement). Valid values: true, false
rtPrefBarDoubleCutViaOrient	string	cellview	none	Preferred bar or double-cut via direction. • <code>none</code>: No preferred via direction. • <code>H</code>: Places vias in the horizontal direction whenever possible. • <code>V</code>: Places vias in the vertical direction whenever possible. Valid values: none, horizontal, vertical
rtPrefViaDirection	string	cellview	Auto	Preferred via direction: • <code>Auto</code>: Automatically sets the via direction to optimize routing. • <code>SV</code>: Places vias strictly in the vertical direction. • <code>V</code>: Places vias in the vertical direction whenever possible. • <code>SH</code>: Places vias strictly in the horizontal direction • <code>H</code>: Places vias in the horizontal direction whenever possible. Valid values: Auto, SV, V, SH, H

Table 174 rt Preferences (Continued)

Name	Type	Scope	Default	Description
rtPreRouteCheckers	string	cellview	Design Errors, Pin Accessibility, Off Track Pin, Pin and Track Color, Pin Short, Technology Errors, Missing Via Definition, Missing Width Constraints, Missing Spacing Constraints, Design Warnings, Single Pin in Routing Area, Track Region and Layer Color, Pin Size, Unnetted Shapes, Label on Shapes Without Pins, Pin Off Grid Points, Nets Without Shapes	Runs Preroute Checker on all nets or selected nets. The Preroute Checker will check for specified design errors. Valid values: Design Errors, Pin Accessibility, Off Track Pin, Pin and Track Color, Pin Short, Technology Errors, Missing Via Definition, Missing Width Constraints, Missing Spacing Constraints, Design Warnings, Single Pin in Routing Area, Track Region and Layer Color, Pin Size, Unnetted Shapes, Label on Shapes Without Pins, Pin Off Grid Points, Nets Without Shapes
rtProgressOnReload	bool	cellview	true	When true, displays the progress dialog box when the router is reloaded. Valid values: true, false
rtPromptCloseWindow	bool	user	true	Displays a Close Window prompt if the design contains uncommitted route. Valid values: true, false
rtRatioTolerance	int	cellview	0	Acceptable routing ratio tolerance value (%).
rtRcRoutingStyle	string	cellview	endrun	Specifies the routing style (endrun, accordion, antenna, endrunMultiLayer) in match routing. Valid values: endrun, accordion, antenna, endrunMultiLayer
rtRedundantViaEffortLevel	string	cellview	high	Controls the redundant via mode. Valid values: off, low, medium, high
rtReportCongestionDialogMode	bool	user	false	Displays the Check Congestion dialog box. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtRouteCenterChannel	bool	cellview	true	Lets you control whether routes between macro blocks are centered within the free space in the channel. By default, the preference is set to <code>true</code> . However, routing in the center of channels in older technologies can create slow zones, causing routes to be detoured. To prevent these unintended route detours, you can set the preference to <code>false</code> .
rtRouteClosestPins	bool	cellview	false	Connects pins to the closes pre-route using the shortest routes. The option is useful for DDR ECO routing after buffer insertion. Valid values: true, false
rtRouteNetOaRoutesOnly	bool	cellview	false	Parameter to route oaRoutes only. Valid values: true, false
rtRouterRipupPasses	int	cellview	10	Specifies the number of ripup passes for the router.
rtRouteWithOutsideShields	bool	cellview	true	Routes bus trunks with interleaved shields but without the outer shields. Valid values: true, false
rtRoutingArea	string	cellview		Specifies the coordinates of the polygon vertices.
rtRoutingAreaXmax	float	cellview	0	Defines the x coordinate of the upper right corner of the routing window. Valid values: Positive or negative float value.
rtRoutingAreaXmin	float	cellview	0	Defines the x coordinate of the lower left corner of the routing window. Valid values: Positive or negative float value.
rtRoutingAreaYmax	float	cellview	0	Defines the y coordinate of the upper right corner of the routing window. Valid values: Positive or negative float value.
rtRoutingAreaYmin	float	cellview	0	Defines the y coordinate of the lower left corner of the routing window. Valid values: Positive or negative float value.
rtRowNbCuts	int	cellview	1	Number of rows of cuts in the vias.
rtIRStrap	bool	cellview	false	Strap creation with single click interactive routing. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtShieldCoaxChain	float	cellview	0	Parameter to specify the distance of inserting jumpers to connect coaxial shielding segments together with path and via. Valid values: float (≥ 0)
rtShieldConnectAtMeshOverlap	bool	cellview	false	Parameter to enable creating multiple connections from shields to power mesh. Valid values: true, false
rtShieldConnectLayerOffset	string	cellview	either	Parameter to set the layer offset from the signal layer. Valid values: above, either, below
rtShieldConnectTieCell	bool	cellview	false	Boolean to connect shield shapes to tie cells. Valid values: true, false
rtShieldConnectTieCellSearchDistance	float	cellview	0	Search distance to connect shield shapes to tie cells.
rtShieldConnectWithRoute	bool	cellview	false	Connects shields to ground when true. This option creates long ties that might block future routing. Enable this option only if you must connect shields to ground using the Auto Router. Note that these shields are marked as dynamic shields in the ICC designs. Valid values: true, false
rtShieldEnclosePin	bool	cellview	true	Parameter to control enclosure around pin when shielding. If enabled, pin will be enclosed by shielding. Valid values: true, false
rtShieldEncloseVia	bool	cellview	true	Parameter to control enclosure around via when shielding. If enabled, via will be enclosed by shielding. Valid values: true, false
rtShieldMaxGap	float	cellview	0.0	Specifies the maximum distance between the wire and the shared shielding. The shielding must be placed within the specified distance to be considered a shared shield.
rtShieldMinSignalSeg	float	cellview	0	Parameter to control If the signal wire length is less than this value, shield wires are not created. Valid values: Any float number greater than or equal to ($\geq$) 0.0.
rtShieldNet	string	cellview	""	Parameter to specify shield net when shield net is not specified in shield constraint. Valid values: net name

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtShieldOnPoly	bool	cellview	false	Controls shield placement on poly shapes. Valid values: true, false
rtShieldOnValidLayersOnly	bool	cellview	false	Adds shielding on valid layers only. Valid values: true, false
rtShieldOption	string	cellview	All	Applies shielding to all nets or selected nets. Valid values: All, Selected
rtShieldParallelJumper	float	cellview	0	Parameter to specify the distance of inserting jumpers to connect two parallel shielding segments together with path and via. Valid values: float (≥ 0)
rtShieldReportCoverage	bool	cellview	false	Creates a shield coverage report at the end of the auto-route run. Valid values: true, false
rtShieldSecondNet	string	cellview	""	Parameter to specify the second shield net when it is not specified in shield constraint. Valid values: net name
rtShieldSelectedShapesOnly	bool	cellview	false	Adds shielding to selected shapes only. Valid values: true, false
rtShieldSharing	bool	cellview	true	Enables shared shielding when set to true. The router checks this preference when the Sharing parameter is set to Default in the Net Shielding constraint. Valid values: true, false
rtShieldTandemJumper	float	cellview	0.0	Spacing between consecutive jumpers that connect two tandem shields. Valid values: float (≥ 0)

Table 174 rt Preferences (Continued)

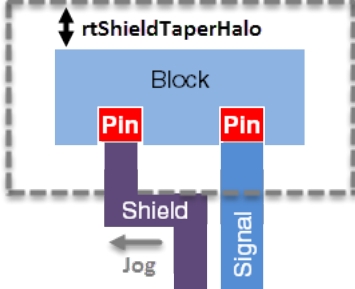
Name	Type	Scope	Default	Description
rtShieldTaperHalo	float	cellview	0	Distance from the macro block at which a shield route introduces a jog to align with the pin inside the block. Aligning with the pin ensures that the shield route can connect to the pin.  For example, to jog the shield route 2.5 mm from the block, set the preference as follows: db::setPrefValue rtShieldTaperHalo -value 2.5
rtShieldTiePowerSearch	float	cellview	0	Option specifies a search distance for power supply. Valid values: float (≥ 0)
rtShieldTieSpacing	float	cellview	0	Option specifies the spacing between shield ties on shield wires. Valid values: float (≥ 0)
rtShieldUseValidViasOnly	bool	cellview	false	Uses vias defined in the technology file to connect to the shielding when no via is selected in Net Shielding constraints. Valid values: true, false
rtShieldUseViaArrayForCoaxial	bool	cellview	false	Controls using via array to tie solid coaxial shields instead of jumper vias. Valid values: true, false
rtShieldViaWithOverhang	bool	cellview	false	Adds overhanging shielding around via shapes on tandem layers. Valid values: true, false
rtSingleConnectionToPin	string	cellview	"off"	Parameter to specify single connection to pin, "NDR" or "All" or "off". Valid values: "ndr", "all", "off"
rtSingleLayerMode	bool	cellview	false	Single layer mode preference Valid values: true, false
rtSingleLoopMatch	bool	cellview	false	Enables single-loop length matching if set to true. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtSingleLoopMatchForceOnGrid	bool	cellview	false	Lengthens the wire on the grid during single-loop length matching if set to true. Valid values: true, false
rtSingleLoopMatchMaxSpacing	float	cellview	0.0	Specifies the maximum spacing requirement for single-loop length matching. By default, the router will use the full channel width during lengthening.
rtSingleLoopMatchMinSpacing	float	cellview	0.0	Specifies the minimum spacing requirement for single-loop length matching. By default, the router will use the full channel width during lengthening.
rtSingleLoopMatchOffsetLayer	bool	cellview	false	Uses the offset layer during single-loop length matching if set to true. Valid values: true, false
rtSnapMode	bool	cellview	true	Aligns the path to the center of existing same-net shapes. Note: Routing tracks always has priority. Valid values: true, false
rtStrictFollowPrefBarDoubleCutViaOrient	bool	cellview	false	Enforce the orient of preferred bar/double cut via. Valid values: none, vertical, horizontal
rtStrictPreferredDirection	bool	cellview	false	Creates only preferred-direction routes. Select the option if you want to prevent the Interactive Router from creating wrong-way routes.
rtSyncShieldViasWithSignalVias	bool	cellview	true	Option to use signal via definitions for shield via creation. Valid values: true, false
rtTandemAllowBreaks	bool	cellview	true	Indicates that IR tandem routes can be created even if not fully complete. Valid values: true, false
rtTandemCornerVias	bool	cellview	true	Indicates whether corner vias should be inserted on IR tandem routes. Valid values: true, false
rtTandemJumperSpacing	float	cellview	0	Defines spacing between jumper vias on IR tandem routes. Valid values: 0, 100
rtTandemJumperVias	string list	cellview	""	Defines jumper vias on tandem routes.
rtTandemLayers	string	cellview	Both	Specifies the tandem shield layers. Valid values: Both, Above, Below

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtTandemObeyPrefDir	bool	cellview	false	Defines if partial tandem connections should be realized while ignoring preferred direction settings. Valid values: true, false
rtTandemOffset	string	cellview	1	Places the tandem shield the specified number of layers from the signal wire layer. If you specify 1, the shield layer is adjacent to the wire layer.
rtTandemStack	string	cellview	"No Tandem"	Defines whether the tandem route spawns 2 or 3 layers. Specifies the IR routing tandem mode. Valid values: No Tandem, Two Layer Tandem, Three Layer Tandem
rtTaperUntilFirstTurn	bool	cellview	false	Disables tapering on first turn from source or target boundary pin. Valid values: true, false
rtTieFloatCenterShield	bool	cellview	false	Connects the center shield of split coaxial shields to neighboring shields on tandem layers. Valid values: true, false
rtTopologyAddToNetGroupMode	string	cellview	select	Adds the nets you specify to the net group: <code>all</code>: All nets that have topologies are added. <code>select</code>: Only the nets you selected in the Topology Assistant are added. <code>current</code>: Only the nets currently displayed in the Topology Assistant are added. Valid values: all, select, current
rtTopologyAddToNetGroupName	string	cellview		Specifies the name of the net group.
rtTopologyAddToNetGroupOverwrite	bool	cellview	true	Overwrites the existing net group topology. Valid values: true, false
rtTopologyAutoCreateConnectPrecisely	bool	cellview	false	Connects ATC flow directly to selected object or connected component. Valid values: true, false
rtTopologyConnectOverlap	bool	cellview	false	Connects to the overlapped pin or topology. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtTopologyDeleteMode	string	cellview	select	Deletes the nets you specify: • all: All nets that have topologies are deleted. • select: Only the nets you selected in the Topology Assistant are deleted. • current: Only the nets currently displayed in the Topology Assistant are deleted. Valid values: all, select, current
rtTopologyDisplay	bool	cellview	false	Displays the topology on the canvas. Valid values: true, false
rtTopologyDisplayAnchorPoint	bool	cellview	true	Displays the topology anchor point. Valid values: true, false
rtTopologyDisplayCompleteNet	bool	cellview	false	Control whether to show topology shapes when the net is complete
rtTopologyEMAware	bool	cellview	false	Runs the Electrical Reporter to check for electromigration violations. If EM violations are detected, the width of the topology segments will be increased to avoid the violations. Valid values: true, false
rtTopologyEMRAwareBalanceWidth	bool	cellview	false	Widens more topology segments during EM- and R-aware topology routing. When the preference is false, only segments that would result in larger electrical changes are widened to minimize the total number of modifications. When the preference is true, other segments might be widened to ensure a balanced number of changes in the design. Valid values: true, false
rtTopologyEMRAwareMaxIterationsPerNet	int	cellview	50	Specifies the maximum number of routing iterations per net in EM- and R-aware topology routing.
rtTopologyEMRAwareMaxIterationsPerPath	int	cellview	50	Specifies the maximum number of routing iterations per pin-to-pin path in R-aware topology routing.
rtTopologyEMRAwareMaxSecondsPerNet	int	cellview	10000	Specifies the maximum runtime (in seconds) per net in EM- and R-aware topology routing. The time limit is checked after each iteration is complete. Routing is not interrupted until the current iteration is finished.
rtTopologyExportFilePath	string	cellview		Specifies the path and name of the topology file you are importing or exporting.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtTopologyExportMode	string	cellview	select	Exports the nets you specify: • <i>all</i>: All nets that have topologies are exported. • <i>select</i>: Only the nets you selected in the Topology Assistant are exported. • <i>current</i>: Only the nets currently displayed in the Topology Assistant are exported. Valid values: all, select, current
rtTopologyExportNetOption	string	cellview	All	Specifies whether to export topology for all or select nets. Valid values: All, Selected
rtTopologyFilterClockNet	bool	cellview	true	Displays clock nets and hides all other nets. Valid values: true, false
rtTopologyFilterGroundNet	bool	cellview	true	Displays ground nets. Valid values: true, false
rtTopologyFilterPowerNet	bool	cellview	true	Displays power nets and hides all other nets. Valid values: true, false
rtTopologyFilterSignalNet	bool	cellview	true	Displays signal nets and hides all other nets. Valid values: true, false
rtTopologyImportExportFilePath	string	cellview		The path of the topology file for importing and exporting.
rtTopologyImportOverwrite	bool	cellview	true	Overwrites existing topologies for the net during importing. Valid values: true, false
rtTopologyIncludeStdCellNets	bool	cellview	false	Generates topologies for nets that connect to standard cells. When <i>false</i> , the Topology Auto Create command skips standard cell nets. Valid values: true, false
rtTopologyIncrementalRoute	bool	cellview	false	Creates partial topologies and lets you complete the topology for the entire net incrementally. The Auto Router will not create routes for pins that do not have topologies. Select this option if the net is large and complex. Valid values: true, false
rtTopologyMaxNumStripes	int	cellview	5	Maximum number of stripes allowed in the topology.
rtTopologyNet	string	cellview		Specifies the selected nets on the Topology toolbar.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtTopologyRAware	bool	cellview	false	Runs the Electrical Reporter to check for pin-to-pin maximum resistance violations. If resistance constraint violations are detected, the width of the topology segments will be increased to comply with the constraint. Valid values: true, false
rtTopologyReroute	bool	cellview	false	Removes route shapes as new topologies are created. Valid values: true, false
rtTopologyRoutingArea	bool	cellview	true	Displays the routing area while in the Topology command is active. Valid values: true, false
rtTopologySelectRegion	string	cellview	Rectangle	Object selection method: • Rectangle: Selects objects within the rectangular region that you draw on the canvas. • Line: Selects objects that intersect with the line that you draw on the canvas. Valid values: Rectangle, Line
rtTopologyShieldDefault	bool	cellview	true	Sets the default value of shield properties of newly generated topologies. Valid values: true, false
rtTopologyShieldTapoffAsTrunk	bool	cellview	true	Adds shielding to trunk tapoff segments, applying the same shield settings used by the corresponding trunk. Valid values: true, false
rtTopologyUseActualLayer	bool	cellview	false	Draws topology using the display packet and width of the assigned layer. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtTopologyUseStackingVia	bool	cellview	false	During topology routing, creates stacked vias to connect paths that are multiple layers apart. When true, vias are stacked between connecting paths. For example: When false, vias are not stacked between connecting paths. For example:
rtTopologyViolationMode	bool	cellview	false	Creates routes even if design rule violations occur. Valid values: true, false
rtTrimConnectedTrunk	bool	cellview	false	After <code>rt::route</code> routes selected nets, trims dangling User Trunk shapes on all fully routed nets in the design to the nearest via, regardless of when the nets were fully routed. The trimming operation is DRC aware. Valid values: true, false
rtTrimTrunks	bool	cellview	false	After <code>rt::route</code> routes each selected net, trims dangling User Trunk shapes on that net to the nearest via, regardless of whether they are fully routed. The trimming operation is DRC aware. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtTrimWireEnds	bool	cellview	false	After <code>rt::route</code> routes selected nets, trims dangling Custom Route paths and path segments to the nearest via, regardless of whether the nets are fully routed. The trimming operation does not check for or correct design rule violations that it might have been introduced. Valid values: true, false
rtTrunkBox	bbox	cellview	""	Option to specify the area of trunk-pin routing. Valid values: {x0 y0} {x1 y1}
rtTrunkUsePinWidth	bool	cellview	false	Option to use pin width as wire width in trunk-pin routing. Valid values: true, false
rtUse202003IR	bool	cellview	false	Switches to the 2020.03 Interactive Router. Valid values: true, false
rtUseAdvancedNodeEngine	string	cellview	auto	Control use of advanced node engine in routing
rtUseDesignConstraint	bool	cellview	true	use layer setting of design constraint Valid values: true, false
rtUseExpandedViaSetting	bool	cellview	false	Boolean to use the expanded via setting Valid values: true, false
rtUseGlobalRoute	bool	cellview	true	Approximates the course of the routes before detailed routes are added. Doing so can reduce the number of rip-up and reroutes, thus improving overall routing performance and results, especially in block-level routing. Valid values: true, false
rtUseKeepoutWidth	bool	cellview	true	Option to specify to have router using the shape of the top level keepout to determine the correct spacing from the spacing table. If the value is false (default), router just uses the layer spacing. Valid values: true, false
rtUseLEStopLevel	bool	cellview	true	Applies the same the view level specified in the Layout Editor, or displays the entire hierarchy. Valid values: true, false
rtUseLEViaSelection	bool	user	true	Checks the <code>leChooseCustomViaFirst</code> preference. If the preference is <code>true</code> , then the router will use custom vias, if they are defined in the technology. Set the preference to <code>false</code> if you do not want to check the <code>leChooseCustomViaFirst</code> preference. Valid values: true, false

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtUseMatchBox	bool	cellview	false	Specifies an area for match lengthening. Valid values: true, false
rtUseOnlyUserDefinedRuleFile	bool	cellview	false	Requires that the Custom Router commands use only the technology file that you specify for rule definitions. Valid values: true, false
rtUsePowerStrapAsShield	bool	cellview	false	Routes wires close to power straps and uses them as shared shielding. To use the preference, you must turn on shield sharing. The preference applies only to the Auto Router. Valid values: true, false
rtUserDefinedAutoNetGroup	string	cellView		Specify the net group for auto grouping.
rtUserDefinedConstraint	bool	cellview	false	Applies the shield style as specified in the Constraint Editor. Valid values: true, false
rtUserDefinedRuleFile	string	cellview	""	Option to load user-defined rule files instead of CD process and design intent rules.
rtUserRoutingLayers	string	cellview		Specifies the layers to use for routing in cases where multiple layers have the same mask number but different layer numbers. The layers specified by this preference override the specifications in the <code>validRoutingLayers</code> constraint in the technology library and any specifications in the Constraint Editor. The syntax is as follows: <pre>rtUserRoutingLayers {metall1, metal2, metal3,...}</pre> Where <code>metall1</code> , ..., <code>metaln</code> are the routing layer names.
rtUseTrunkBox	bool	cellview	false	Option to enable specifying an area that all pins within the area will run trunk-pin routing. Valid values: true, false
rtViaCost	string	cellview	""	Control layer via cost at per layer basis. The syntax is as follows: <pre>rtLayerCost { {<layer1> <userCost1>} {<layer2> <userCost2>} ... {<layerN> <userCostN>} }</pre> where <code>layer_n</code> is the name of the layer. <code>userCost_n</code> is the cost value of the layer. Valid values are neutral, high, and low. If you do not specify any value, the router assumes a via cost of neutral for all layers.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtViaDownSelection	string	cellview	""	Specifies a via to add below the current layer during single-layer interactive routing.
rtViaFillIntersection	bool	cellview	false	Option to allow maximized via array to be used for wire-to-wire and wire-to-pin connections if the value is true. Valid values: true, false
rtViaFilterCutClass	string	cellview		Via cut class filter.
rtViaFilterCutClasses	string	cellview	auto	Via cut class filter input history.
rtViaFilterCutClassesExpand	string	cellview		Via cut class filter input history.
rtViaFilterCutClassExpand	string	cellview		Via cut class filter.
rtViaFilterDef	string	cellview		Via definition filter.
rtViaFilterDefExpand	string	cellview		Via definition filter.
rtViaFilterDefs	string	cellview	auto	Via definition filter input history.
rtViaFilterDefsExpand	string	cellview		Via definition filter input history.
rtViaLadderColorChange	bool	cellview	false	Allows Custom Router commands to change the color assignments of vias in via ladders to improve coverage. Valid values: true, false
rtViaLadderDefFile	string	cellview		Path of via ladders definition file.
rtViaLadderFlow	string	cellview	during_r oute	Specifies the via ladder flow. Valid values: during_route, no_route, before_route
rtViaLadderPinType	string	cellview	output	Types of pins for which via ladders are created.
rtViaMinimumCut	int	cellview	1	Set minimum cut for all layers.
rtViaPreferredEnclosure	string	cellview	auto	Specifies how the via's metal layers are positioned: • <code>auto</code>: The router determines the positions. • <code>cross</code>: The layers are perpendicular. • <code>nonCross</code>: The layers are parallel. Valid values: auto, cross, nonCross
rtViaUpSelection	string	cellview	""	Specifies a via to add above the current layer during single-layer interactive routing.
rtWeldingLayers	string	cellview	all	Specifies the layers to be considered during welding operations.

Table 174 *rt Preferences (Continued)*

Name	Type	Scope	Default	Description
rtWrongWayCost	string	cellview	""	Control wrong way cost at per layer basis. The syntax is as follows: <pre>rtLayerCost { {<layer1> <userCost1>} {<layer2> <userCost2>} ... {<layerN> <userCostN>} }where</pre> • <code>layer_n</code> is the name of the layer. • <code>userCost_n</code> is the cost value of the layer. Valid values are neutral, high, and low. If you do not specify any value, the router assumes a wrong way cost of neutral for all layers.

Route Pattern Description Reference

You can add custom route patterns for use in the Pattern Route Assistant. The custom patterns you want to add must be described in text format in an ASCII file.

The following sections specify the rules for the route pattern descriptions:

- [Route Pattern Description Syntax](#)
- [trunk](#)
- [spine](#)
- [backbone / fishbone](#)
- [mirror](#)
- [intermittent / match](#)
- [track](#)
- [pin](#)
- [row](#)
- [tapoff](#)
- [layer](#)
- [width](#)
- [cluster](#)

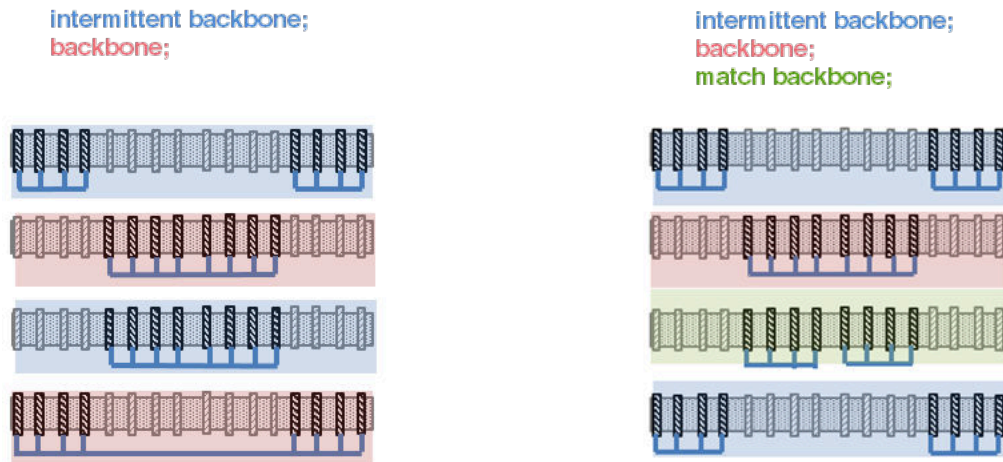
See Also

- [Adding Custom Patterns in the Pattern Catalog](#)
- [Choosing Route Patterns](#)

Route Pattern Description Syntax

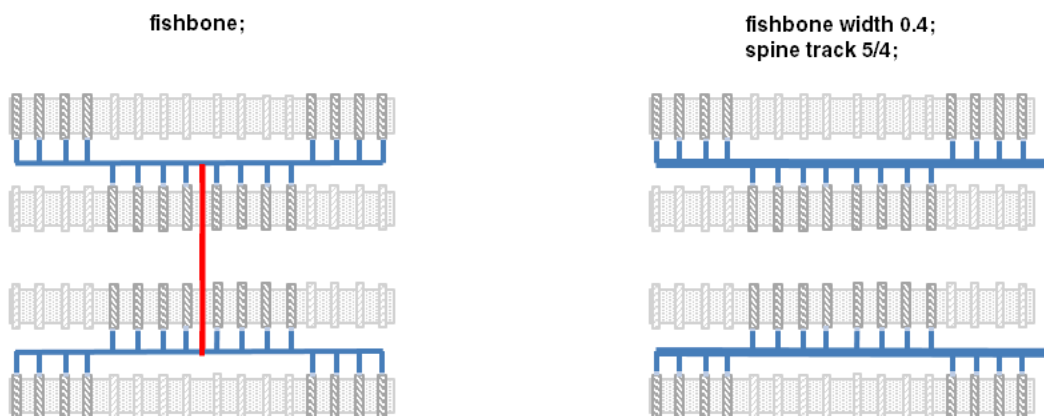
Description

A route pattern description can consist of one or more lines that specify how the pattern is constructed. If the description contains only one line, the line applies to the entire design or defined routing area. If the description contains multiple lines, each line applies to a routing channel. If the number of channels is greater than the number of lines, the Custom Router will cycle through the lines.



Each line specifies the characteristics of the route, such as the type of welding style that is used when branching from the main interconnect (route trunk) to pins. Each line must end with a semicolon (;).

Default route pattern descriptions result in generic route patterns, as shown in the example on the left. If you have special routing requirements, you can use the optional description settings to achieve the desired results, as shown in the example on the right.



Syntax

```
<trunk> [spine] <backbone / fishbone> [layer <string>] [width <float>]
[mirror];
```

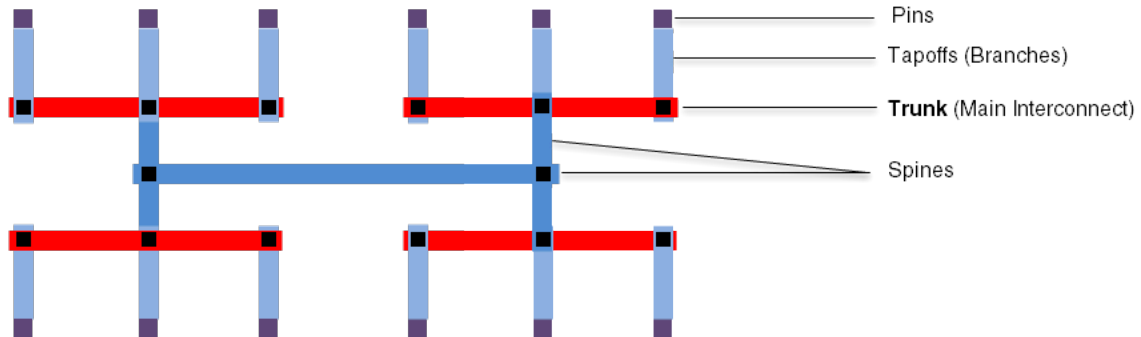
Examples

```
intermittent backbone mirror;
backbone;
mirror;
```

trunk

Description

Describes how the routing trunk is constructed. Structurally, the routing trunk is the main interconnect from which tapoffs branch to reach pins. In the context of a route pattern, the trunk description includes the trunk structure ([backbone](#) / [fishbone](#)), the pin grouping ([intermittent](#) / [match](#)), and other details as shown in the syntax.

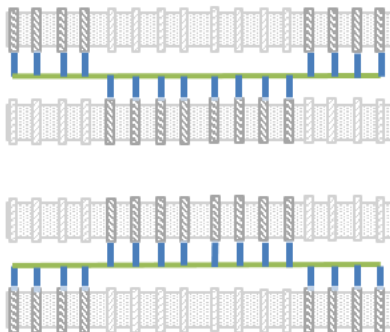


Syntax

```
[<intermittent / match>] <backbone / fishbone> [track]  
[row][layer <string>] [width <float>] [tapoff]
```

Examples

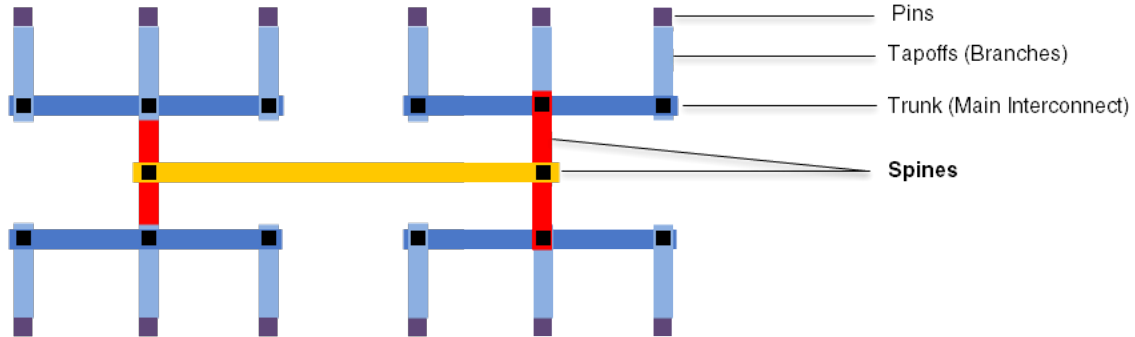
fishbone layer M3 tapoff width 0.3;



spine

Description

Specifies the paths that connect [backbone](#) / [fishbone](#) structures. Spines also connect to other spines.



When a spine spans across multiple channels, you have the option of extending the spine to align with the outermost connecting trunks.

You can specify the location of the spines based on [track](#) or [pin](#) locations.

Syntax

```
spine [track | pin][extend]
```

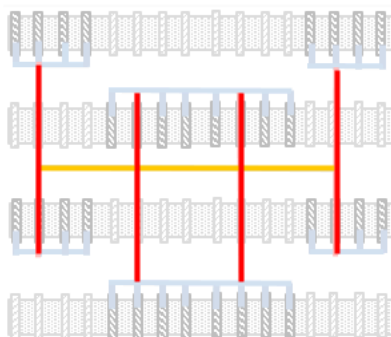
Examples

The first example shows the default spine specification. The red spines connect the backbones at the second pin position out of every four pin positions. The orange spines connect at the center-most channel.

The second example is almost exactly like the first example. The main difference is the description extends the red spines to the outermost backbone trunks, as shown in the diagram.

```
intermittent backbone mirror; spine pin 2 / 4;  
intermittent backbone mirror; spine pin 2 / 4 extend;
```

**intermittent bckbone mirror;
spine pin 2/4;**



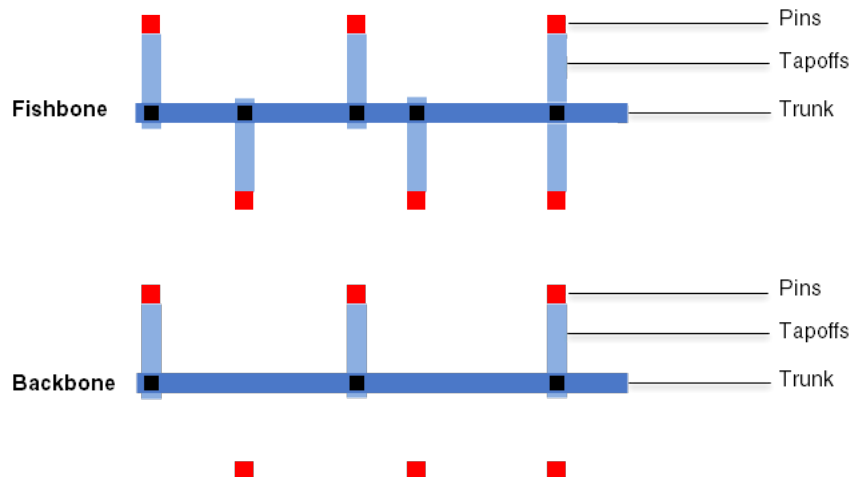
**intermittent bckbone mirror;
spine pin 2/4 extend;**



backbone / fishbone

Description

Describes the Welding mode of the route: fishbone, or backbone.



Syntax

<fishbone | backbone>

Examples



mirror

Description

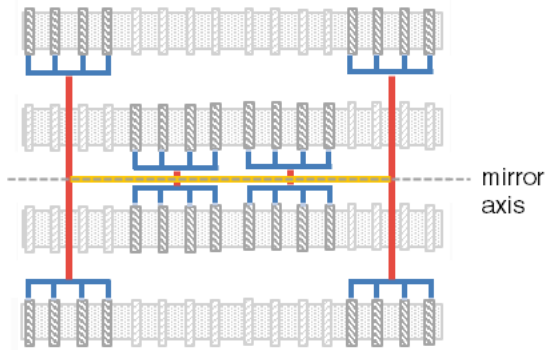
Repeats a part of or an entire route pattern using the mirror reflection of the original pattern.

Syntax

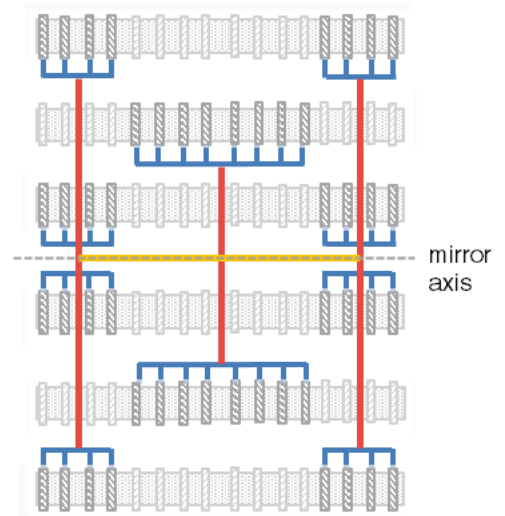
[mirror]

Examples

match backbone;
mirror;



match backbone;
backbone;
mirror;



intermittent / match

Description

Describes how the pins are connected.

Syntax

```
<default | intermittent | match>
```

Arguments

`default`

Within each channel, a single trunk connects all the pins.

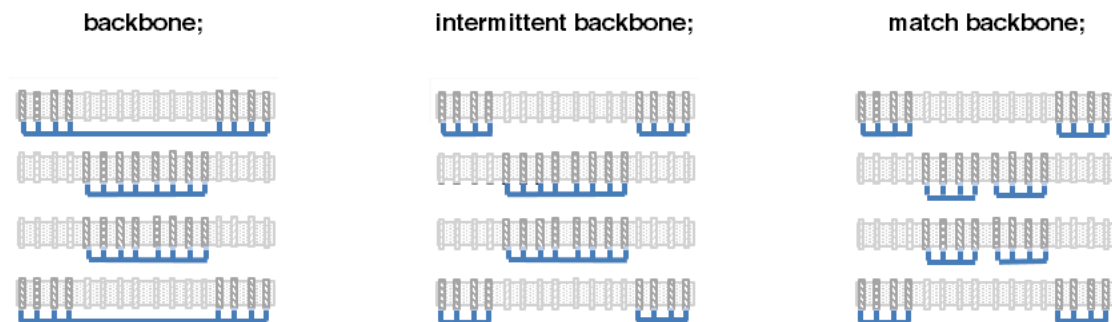
`intermittent`

Within each channel, each trunk connects the pins that are positioned consecutively.

`match`

Within the route pattern, match the smallest intermittent pin-trunk group.

Examples

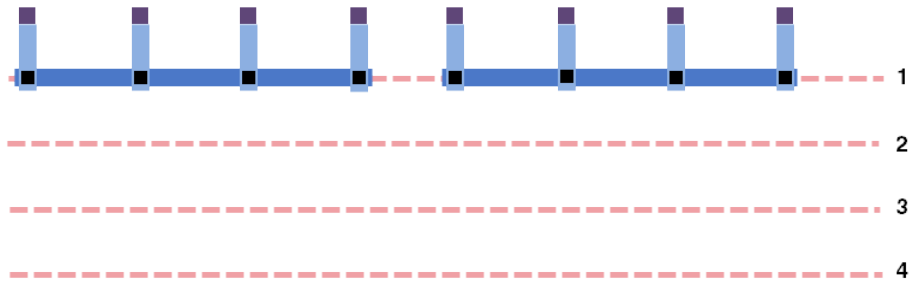


track

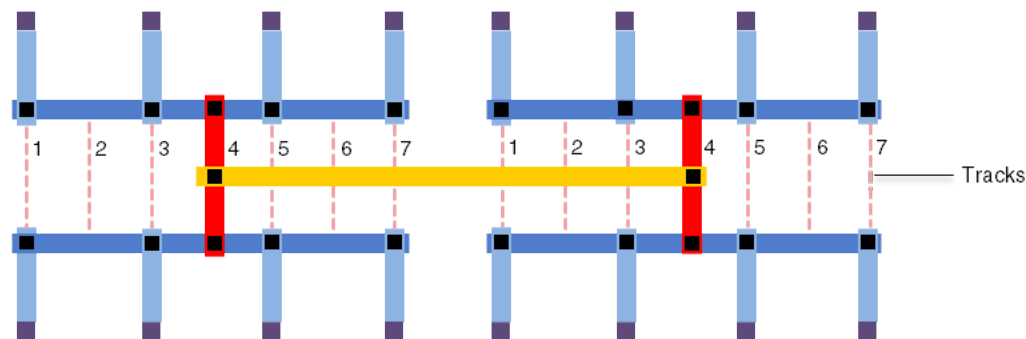
Description

Specifies the location of the routing trunks or [spines](#), based on track locations. By default, the Custom Router will route the backbone on the track closest to the pins. In the following example, because no obstacles are between the pins and track 1, the nearest track, the Custom Router routes the backbone on track 1.

Default



If you do not specify a track-based location, the Custom Router will place the spine at the center-most location. In the following example, the center-most location is at track location 4/7. If you want the spine to be placed at 4/7, you would not need to explicitly specify the track location.



Syntax

```
track <int>/<int>
```

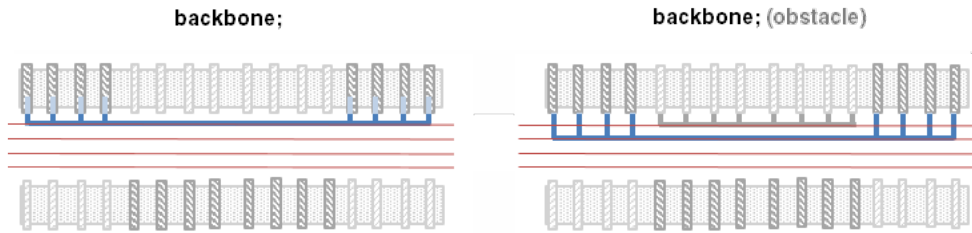
Note:

The horizontal track index for pattern routing is different from the track index for trunk routing. See [Before You Start Trunk Routing](#).

Examples

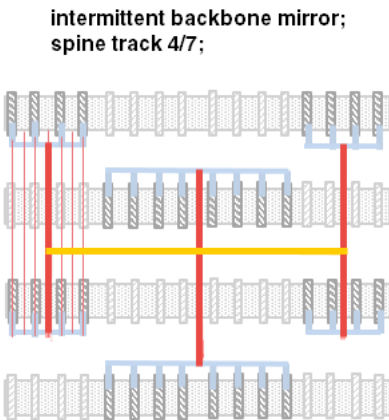
Because the first example does not specify the track location, the Custom Router places the backbone on the track closest to the pins (track 1/4). In the second example, an obstacle (existing backbone) is blocking the default track location. So, the Custom Router will place the backbone on the next available track (2/4), as shown in the second example.

```
backbone;
```



The following example explicitly specifies that the first red spine be placed on track 4/7.

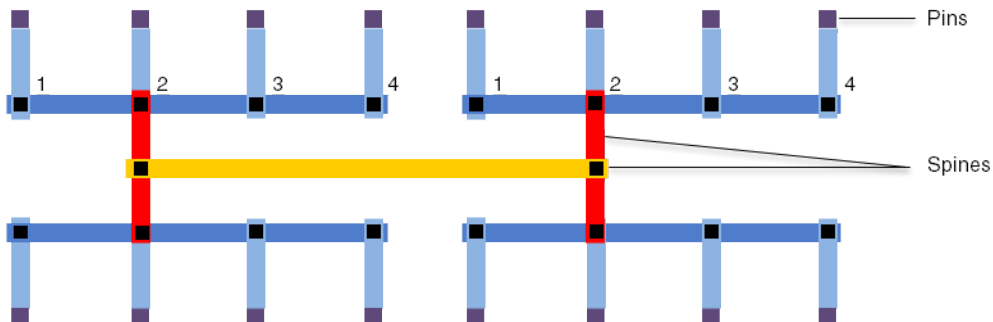
```
intermittent backbone mirror; spine track 4/7;
```



pin

Description

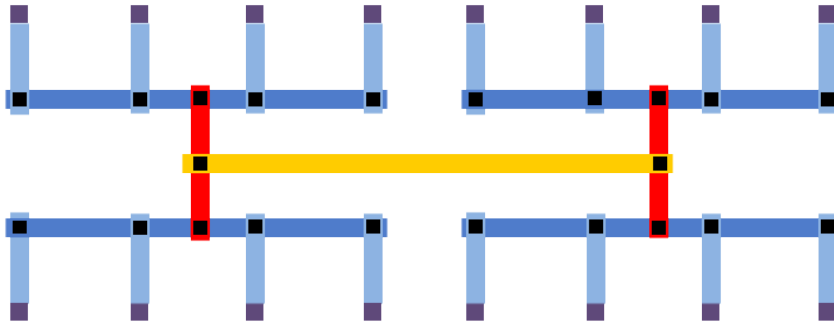
Specifies the location of the [spines](#), based on pin locations. In the following example, the red spine is placed at pin location 2/4 (second pin position for every four pin positions).



If the number of pins within a pin group is greater than the denominator value you specify, the Custom Router will place the spines at the same intervals. For example, if you specify

a pin-based location of 2/4, but the pin group contains 8 pins, then the spine will be placed at pin positions 2 and 6.

If you do not specify a pin-based location, the Custom Router will place the spine at the center-most location.



Syntax

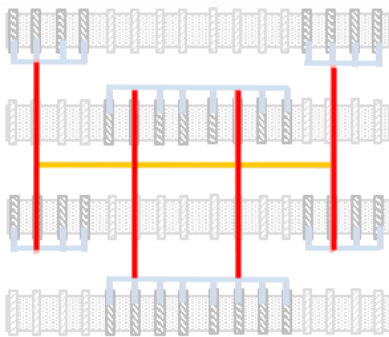
```
pin <int>/<int>
```

Examples

The following example specifies that the red spine be placed at pin location 2/4. Because there is no orange spine specification, the orange spines are placed at the center-most position of the structure by default.

```
intermittent backbone mirror; spine pin 2 / 4;
```

```
intermittent bckbone mirror;  
spine pin 2/4;
```



row

Description

Specifies the number of rows of pins to be connected. The pins must be on the same net. You can optionally specify whether the connections must be from the top (up) or bottom (down) of the pins. By default, all connections will be at the bottom of the pins.

Syntax

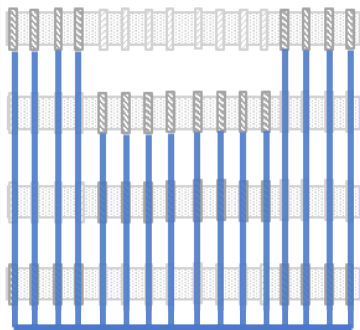
```
row <all | int> [up | down]
```

Examples

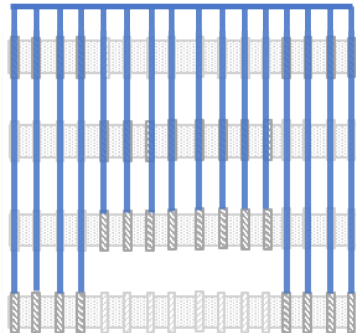
In the first example, pin connections will occur across all rows from the bottom of the pins (default). In the second example, pin connections will occur across all rows from the top of the pins. In the third example, only pins in the first three rows are connected from the bottom of the pins (default).

```
backbone row all;  
backbone row all up;  
backbone row 3 mirror;
```

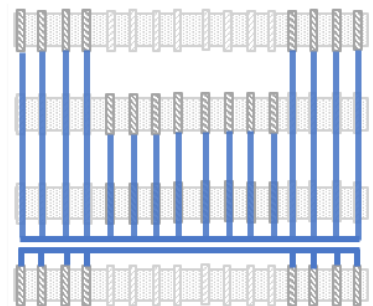
backbone row all;



backbone row all up;



backbone row 3 mirror;



tapoff

Description

Describes the layer and width of the tapoff branches.

Syntax

```
tapoff [layer] [width] [cluster]
```

Examples

```
fishbone tapoff width 0.4 cluster auto;
```

See also the example for [width](#).

layer

Description

Specifies the layer that hosts the routing trunk, tapoff, spine, or cluster connection. By default, the Interactive Router minimum cost layer is used.

Syntax

```
layer <string>
```

Examples

In the following example, the tapoff is placed on the M3 layer.

```
fishbone layer M3 tapoff width 0.3;
```

width

Description

Specifies the width of the routing trunk, tapoff, spine, or cluster connection. The default width is the layer width value specified in the technology library.

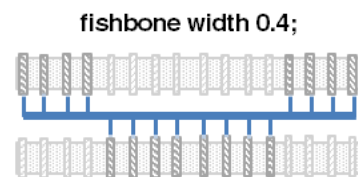
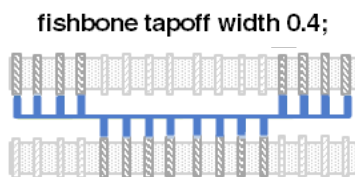
Syntax

```
width <float>
```

Examples

In the first example, the tapoff width is set to 0.4. In the second example, the routing trunk width is set to 0.4.

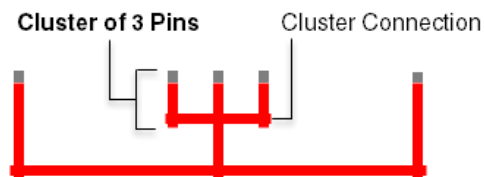
```
fishbone tapoff width 0.4;  
fishbone width 0.4;
```



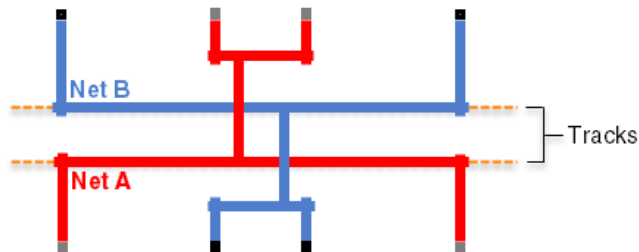
cluster

Description

A cluster is a set of neighboring pins that are on the same net and are grouped to reduce the number of routes in channels.



With a less congested channel, routes from different nets can co-exist in the same channel. In the following example, routes for net A and net B can be routed in the same channel after some of the pins are clustered.



Syntax

```
cluster <int> [layer] [width]
```

Examples

In the example, cluster is 3. The cluster connection path segment is on m5 and the width of that segment is 0.4.

```
cluster 3 layer m5 width 0.4;
```

22

Schematic Editing Tcl

This chapter lists commands, enumerations, objects, and preferences in the Custom Compiler Tcl se namespace. This namespace provides Tcl for schematic editing.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

Interactive commands are listed after non-interactive commands in the Commands section.

The commands covered are:

- [se::activateSheet](#)
- [se::addSheets](#)
- [se::addToOverlayRendering](#)
- [se::align](#)
- [se::annotateOP](#)
- [se::attachText](#)
- [se::calculateVDRCDeltaV](#)
- [se::check](#)
- [se::chopWires](#)
- [se::clearNetColorizations](#)

- [se::clearUnboundOverlays](#)
- [se::colorizeNets](#)
- [se::compareDesigns](#)
- [se::connectWires](#)
- [se::copy](#)
- [se::createArc](#)
- [se::createBlock](#)
- [se::createCheckOption](#)
- [se::createConnectDef](#)
- [se::createConstraintNote](#)
- [se::createEllipse](#)
- [se::createEMDB](#)
- [se::createInst](#)
- [se::createLine](#)
- [se::createMask](#)
- [se::createNetColorizationDefinition](#)
- [se::createOPDB](#)
- [se::createPin](#)
- [se::createPolygon](#)
- [se::createRectangle](#)
- [se::createRule](#)
- [se::createSelectionShape](#)
- [se::createTable](#)
- [se::createText](#)
- [se::createVDRADB](#)
- [se::createViolation](#)
- [se::createWire](#)

- [se::createWireName](#)
- [se::createWireStubs](#)
- [se::delete](#)
- [se::disconnectWires](#)
- [se::enableDynamicNetLabels](#)
- [se::extractConnectivity](#)
- [se::findMaskedInstances](#)
- [se::findReferenceDifferences](#)
- [se::generateEstRCSchematic](#)
- [se::generateSchematic](#)
- [se::generateSymbol](#)
- [se::generateSymbolTransformPCell](#)
- [se::getActiveSheet](#)
- [se::getBillOfMaterials](#)
- [se::getColorizationDefinitions](#)
- [se::getDynamicNetLabels](#)
- [se::getNetComponents](#)
- [se::getOverlayObjects](#)
- [se::getPinWiringPoint](#)
- [se::getReferenceCellView](#)
- [se::getRules](#)
- [se::getShapeStyle](#)
- [se::getSheets](#)
- [se::getTables](#)
- [se::importConfig](#)
- [se::importSymbol](#)
- [se::isDesignNetlistable](#)

- [se::isNetlistable](#)
- [se::isOverlay](#)
- [se::loadNetColorizationDefinitions](#)
- [se::move](#)
- [se::moveSheetToPosition](#)
- [se::populateDesignSummary](#)
- [se::propagatePortOrder](#)
- [se::registerLabelFunction](#)
- [se::registerParasiticTopology](#)
- [se::registerPinShape](#)
- [se::renameNet](#)
- [se::resizeSheets](#)
- [se::saveNetColorizationDefinitions](#)
- [se::schematicMigration](#)
- [se::setConnectivityValid](#)
- [se::setShapeStyle](#)
- [se::setSheetColumns](#)
- [se::showAnnotateOP](#)
- [se::showBillOfMaterials](#)
- [se::showCheckOptions](#)
- [se::showCompareDesigns](#)
- [se::showCreateTemplateFromSelected](#)
- [se::showEditSheets](#)
- [se::showEditTable](#)
- [se::showLibraryCheck](#)
- [se::showNetColorizationDefinitionDialog](#)
- [se::showPlotSignalSettings](#)

- [se::showPowerDomainAnalyzer](#)
- [se::showSchDesignOptions](#)
- [se::showSchematicMigration](#)
- [se::showSymDesignOptions](#)
- [se::showSynchronizePortOrder](#)
- [se::stretch](#)
- [se::syncOverlayDesign](#)
- [se::unregisterLabelFunction](#)
- [se::updatePinLabels](#)
- [ise::align](#)
- [ise::alignToColumn](#)
- [ise::attachText](#)
- [ise::check](#)
- [ise::chopWires](#)
- [ise::colorizeNets](#)
- [ise::connectWires](#)
- [ise::copy](#)
- [ise::copyToClipboard](#)
- [ise::createBlock](#)
- [ise::createColumn](#)
- [ise::createConnectDef](#)
- [ise::createConnector](#)
- [ise::createConstraintNote](#)
- [ise::createInst](#)
- [ise::createLabel](#)
- [ise::createMask](#)
- [ise::createNote](#)

- [ise::createOverlayInstanceWithMask](#)
- [ise::createRuler](#)
- [ise::createSchematicPin](#)
- [ise::createSelectionShape](#)
- [ise::createShape](#)
- [ise::createSymbolPin](#)
- [ise::createTable](#)
- [ise::createTemplate](#)
- [ise::createWire](#)
- [ise::createWireName](#)
- [ise::createWireStubs](#)
- [ise::cut](#)
- [ise::delete](#)
- [ise::disconnectWires](#)
- [ise::ignoreInst](#)
- [ise::importSymbol](#)
- [ise::manageLabels](#)
- [ise::move](#)
- [ise::paste](#)
- [ise::peek](#)
- [ise::plotSignal](#)
- [ise::printOP](#)
- [ise::renameInst](#)
- [ise::route](#)
- [ise::stretch](#)

se::activateSheet

Description

Makes the selected sheet active. The command resets the active sheet if as a sheet, a "0" number is specified. Also, the command fits the viewport to the newly activated sheet, or to all the sheets if the active sheet is not set.

Syntax

```
se::activateSheet <none: dbSheet | int> [-window <giWindow | int>]
```

Arguments

none

The sheet to activate.

-window

The window in which you can activate the sheet.

Returns

<none>

Errors

- The specified sheet is not found.
- The specified window is not a Schematic Editor window.

Examples

```
set design [ed]  
se::addSheets -design $design -numStart 1 -numEnd 3 -border  
  sheets/ASheet/symbol  
se::activateSheet 2
```

se::addSheets

Description

Adds pages to a multisheet cellview.

For the first one, also converts the cellview into a multisheet cellview. The command inserts the specified range of new sheets into a specified position, and all the other sheets that are started from the <numStart> position are shifted up by the number of newly added sheets.

Syntax

```
se::addSheets -design <oaDesign> [-numStart <int>] [-numEnd <int>]  
[-border <dmCellView | dmCellViewName>] [-text <string>]
```

Arguments

-design

The multisheet design to update.

-numStart

The start number of the sheets to add. This should be specified if -numEnd argument is set.

-numEnd

The end number of the sheets to add.

-border

The border's cellview to use.

-text

The text description to set on a sheet. (Default is: "")

Returns

<dbCollection> : The list of newly created multisheet pages.

Errors

- Unable to open the cellview.
- Unable to open border cell's symbol view.
- Incorrect sheet numbers are specified. The sheet numbers should be greater than 0, and should not be an empty range.

Examples

```
set design [ed]  
se::addSheets -design $design -numStart 1 -numEnd 3 -border  
sheets/ASheet/symbol
```

se::addToOverlayRendering

Description

Adds a packet or a specific LPP to the SE overlay rendering system.

Syntax

```
se::addToOverlayRendering [-packetName <string>] -renderAs <enum> [-layer  
  <int>] [-purpose <int>]
```

Arguments

`-packetName`

Display resource packet name. You can only set one of the following arguments/combination of arguments at a time: `<-packetName>` or `<-layer>` and `<-purpose>`.

`-renderAs`

Overlay objects rendering to use. ({ OverlayDevices | OverlayWiring })

`-layer`

Layer number, must be used together with `<-purpose>`. You can only set one of the following arguments/combination of arguments at a time: `<-packetName>` or `<-layer>`, and `<-purpose>`.

`-purpose`

Purpose number, must be used together with `<-layer>`. You can only set one of the following arguments/combination of arguments at a time: `<-packetName>` or `<-layer>` and `<-purpose>`.

se::align

Description

Aligns the specified objects.

Syntax

```
se::align <objects: {<deFigure>} | {<oaFig>}> -src <deFigure | oaFig>  
  [-direction <enum>] [-stretch <bool>]
```

Arguments

`objects`

Set of objects for alignment.

`-src`

Source object for alignment, can be also a 1x object or partial object.

`-direction`

Direction of the alignment line towards which the alignment should be made. ({auto | horizontal | vertical}, Default: auto)

`-stretch`

Controls how the object motion must be made: as stretch (true) or as move (false).

Default: true

Returns

{<deFigure>} | {<oaFig>}: Collection of aligned objects

Examples

```
set design [ed]
set inst1 [se::createInst -design $design -origin {2 2} -libName
analogLib -cellName res -viewName symbol]
set inst2 [se::createInst -design $design -origin {4 3} -libName
analogLib -cellName nmos4 -viewName symbol -orient R90]
se::align $inst1 -src $inst2 -direction horizontal
```

se::annotateOP

Description

Back-annotates operating point (OP) results onto a schematic.

Syntax

```
se::annotateOP (-db <string> | -clear <bool>) -window <giWindow | int>
[-voltage <bool>] [-params <bool>]
```

Arguments

`-db`

The database to back-annotate. Accepts either a db handle or the path to OP database. Use `se::createOPDB` to create a database from simulation results.

`-clear`

Clears window annotations.

`-window`

The window to annotate.

`-voltage`

Controls annotating of voltages. The default is 1.

`-params`

Controls annotating of device parameters. The default is 1.

Returns

<none>

Examples

```
# Create annotation DB from simulator OP results.
se::createOPDB op.db -opFile ./latestResults/results/hspice_wdf.op0
  -nameMap ./latestResults/netlist/cnl
# Back annotate OP results onto an active schematic window
se::annotateOP -db ./op.db -window [gi::getActiveWindow] -params true
  -voltage true
```

se::attachText

Description

Creates a parent/child grouping between a text object and a shape object, where the text object is the child and the shape object is the parent. If the text object was already a child of another object, that relationship is first destroyed before grouping with the new parent object.

This command operates only on note and wire name semantic types. When attaching a note to a shape, the shape must be a note shape. When attaching a wire name to a shape, the shape must be a wiring object.

Syntax

```
se::attachText <name: oaText> -to <oaShape>
```

Arguments

oaText

Text to attach to the shape.

-to

Shape to attach to the text.

Examples

This example attaches a text object to a shape.

```
set r [se::createRectangle {{1 1} {2 2}} -design $oaDesign]
set t [se::createText "TEST" -design $oaDesign -anchor {2 2} -type note
]
se::attachText $t -to $r
```

se::calculateVDRCDeltaV

Description

Calculates the delta voltage for two nets.

Syntax

```
se::calculateVDRCDeltaV -files [<string>] -nameMap <nlNameMap | string>  
-netPair [<string>] [-dutInstName <string>] [-startTime <string>]  
[-stopTime <string>] [-measType <eMeasType>]
```

Arguments

-files

The input simulator waveform files.

-nameMap

The name map for the specified -files. Accepts either a name map object or the directory where name mapping data is stored.

-netPair

Two nets to calculate delta voltage.

-dutInstName

The instance name of the device under test. (Default is:)

-startTime

Computes voltages based on the given start time. (Default is:)

-stopTime

Ignores values after the given stop time. (Default is:)

-measType

Measurement type. ({ minMax | baseTopLine }, default is: minMax)

Returns

<string> :

se::check

Description

Runs the specified checks from the given collection of seRule objects. If no collection is given, then all currently registered enabled checks are invoked on the given design.

If a `dmCellView` is supplied as an argument, the design is opened for the command to execute, and closed after it completes.

The returned violation collection can be used to determine the outcome of the command.

Syntax

```
se::check [<none: {<seRule>}}] -design <dmCellView | dmCellViewName |  
  oaDesign> [-autoSave <bool>] [-forceChecks <bool>] [-checkOnly <bool>]  
  [-crossViews <bool>] [-deletePreexistingMarkersMode <enum>] [-quiet  
  <bool>] [-viewSearchList [<string>]] [-viewStopList [<string>]]
```

Arguments

`none`

A collection of rules to invoke on the given design. If not given, invokes all registered rules with severity not set to `ignore`.

`-design`

Design to process.

`-autoSave`

Indicates whether any marker objects for a design should be saved.

Default: `false`

`-forceChecks`

Indicates whether a determination should first be made whether the design needs checking.

Default: `seCheckForced`

`-checkOnly`

Indicates whether only checks should be run, with no designs being modified in any way.

Default: `seCheckNoMarkers`

`-crossViews`

Indicates whether cross view checking is performed before the rules.

Default: `seCheckCrossViews`

`-deletePreexistingMarkersMode`

Deletes error markers created by all tools or SRC/ERC after performing **Check** or **Check and Save**. ({ `all` | `srcErcOnly`}, default is: `all`)

`-quiet`

Indicates whether messages or warnings in console are suppressed (Default: false)

`-viewSearchList`

View Search List for hierarchical rules.

`-viewStopList`

View Stop List for hierarchical rules.

Returns

All violations found from all requested rule checks. ({<deViolation>})

Errors

Design does not exist.

Examples

The following example runs the check only for one rule.

```
set rules [se::getRules "Floating Nets"]
se::check $rules -design $oaDesign -forceChecks false -checkOnly true
```

The following example performs a cross view check for schematic and symbol views.

```
se::check -design $oaDesign -forceChecks false -checkOnly true
-crossViews true
```

se::chopWires

Description

Chops all wires in the specified area.

Syntax

```
se::chopWires -design <oaDesign> -box <box>
```

Arguments

`-design`

Design in which to create the wire chopping.

`-box`

Wire chopping area.

Examples

```
set design [ed]
# Create wire segments
se::createWire {{1 1} {1 3}} -design $design
se::createWire {{0 2} {2 2}} -design $design
se::chopWires -box {{0.5 1.5} {1.5 2.5}} -design $design
```

se::clearNetColorizations

Description

Clears net colorizations for specified nets or definitions.

Syntax

```
se::clearNetColorizations [-nets [<string>] | -definitions [<string>]]
                           -design <oaDesign>
```

Arguments

-nets

Net names.

-definitions

Colorization definition.

-design

The design in which to apply a net colorization.

Examples

```
set design [ed]
# Create Net Colorization Definition
se::createNetColorizationDefinition clock -description Clock -color red
  -lineStyle longDash
se::createNetColorizationDefinition ground -description Ground -color
  blue -lineStyle thickLine
set tr [de::startTransaction "Create Wire" -design $design]
# Create wire segment
set w [se::createWire {{1 1} {3 1}} -design $design]
# Add wire name
se::createWireName clk -wire $w -abut true
de::endTransaction $tr
# Colorize net
se::colorizeNets clk -design $design -definitions clock
set tr [de::startTransaction "Create Wire" -design $design]
set w [se::createWire {{1 2} {3 2}} -design $design]
se::createWireName gnd! -wire $w -abut true
de::endTransaction $tr
```



```
se::colorizeNets gnd! -design $design -definitions ground
se::clearNetColorizations -design $design -nets clk
```

se::clearUnboundOverlays

Description

Processes and destroys unbound overlay views of the specified config.

Syntax

```
se::clearUnboundOverlays <overlay: deContext | dmCellView |
dmCellViewName>
```

Arguments

overlay

The context or a cellview of overlay config to in which to clear unbounds.

Returns

<none>

se::colorizeNets

Description

Colorizes the specified nets according to color definitions (specified or currently active).

Syntax

```
se::colorizeNets [<names: [string]>] -design <oaDesign> [-definitions
[<string>]]
```

Arguments

names

Net names.

-design

The design in which to apply a net colorization.

-definitions

Specifies the colorization definition.

Examples

```
set design [ed]
# Create Net Colorization Definition
```

```
se::createNetColorizationDefinition clock -description Clock -color red
    -lineStyle longDash
se::createNetColorizationDefinition ground -description Ground -color
    blue -lineStyle thickLine
set tr [de::startTransaction "Create Wire" -design $design]
# Create wire segment
set w [se::createWire {{1 1} {3 1}} -design $design]
# Add wire name
se::createWireName clk -wire $w -abut true
de::endTransaction $tr
# Colorize net
se::colorizeNets clk -design $design -definitions clock
set tr [de::startTransaction "Create Wire" -design $design]
set w [se::createWire {{1 2} {3 2}} -design $design]
se::createWireName gnd! -wire $w -abut true
de::endTransaction $tr
se::colorizeNets gnd! -design $design -definitions ground
```

se::compareDesigns

Description

Runs logical comparison of designs and then writes the report to a specified file.

Syntax

```
se::compareDesigns -cellView1 <dmCellView> -cellView2 <dmCellView>
    -filePath <string> [-hierarchy <bool>] [-filterExpr <string>]
    [-filterColumn <enumFilterColumn>] [-showSystemNet <bool>]
    [-matchParamByString <bool>] [-showOnlyDifference <bool>]
```

Arguments

-cellView1

The first cell view (on the left) to compare.

-cellView2

The second cell view (on the right) to compare with.

-filePath

The file path for comparing results to save. The report file is in text format.

-hierarchy

Compares designs in all hierarchies. (Default is: false)

-filterExpr

Filter expression. Rows that match the given expression are saved to the file.
(Default is:)

`-filterColumn`

The column to apply the filter expression. Values: {All Instance Name {CellView 1} {CellView 2}} (Default is: All)

`-showSystemNet`

Shows mismatches caused by system nets in the difference report. (Default is: true)

`-matchParamByString`

Compares parameter values by string match. For example, 1k and 1000 are considered mismatches if true. (Default is: true)

`-showOnlyDifference`

Skips details of matched cellviews in the difference report. (Default is: false)

Returns

<none>

se::connectWires

Description

Connects two given wire objects if they are orthogonal and intersect perpendicularly. Splits wire segments at the intersection point and re-parents any associated wire names to the new segments. Creates an oaEllipse object (a solder dot) at the intersection point using the size specified in the tool environment.

Syntax

```
se::connectWires -design <oaDesign> -wire1 <oaLine | oaPath> -wire2  
  <oaLine | oaPath>
```

Arguments

`-design`

Design to contain the new object.

`-wire1`

First oaLine or oaPath wiring object.

`-wire2`

Second oaLine or oaPath wiring object, which intersects the first wiring object.

Returns

Intersection point of given wires (<point>).

Errors

- Invalid design, design not opened for write.
- Given object is not a wiring object.
- Given segments do not intersect.
- Given segments are not orthogonal to one another.

Examples

This example connects two overlapped wires.

```
set tr [de::startTransaction tr1 -design $oaDesign]
set w1 [se::createWire {{1 1} {3 1}} -design $oaDesign]
set w2 [se::createWire {{2 0} {2 2}} -design $oaDesign]
de::endTransaction $tr
se::connectWires -design $oaDesign -wire1 $w1 -wire2 $w2
```

se::copy

Description

This command copies the given set of objects relative to the supplied anchor point to the destination point. Optionally, the set of the objects can be copied multiple times if the `-rows` or `-cols` argument is given. The spacing between rows and columns can be controlled by the command-line options.

Syntax

```
se::copy <objects: {<deFigure>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>}
| {<oaShape>}> [-anchor <point>] [-dx <float>] [-dy <float>] [-rotate
<enum>] [-rows <int>] [-cols <int>] [-dxRow <float>] [-dyRow <float>]
[-dxCol <float>] [-dyCol <float>] [-keepNets <enum>] [-chop <bool>]
[-dest <oaDesign>]
```

Arguments

`objects`

Collection of objects to be copied, all objects must be from the same `oaDesign`.

`-anchor`

Transformation origin to use.

Default: snapped center of extents

`-dx`

Distance to move the given objects in the X direction in user units.

Default: 0

`-dy`

Distance to move the given objects in the Y direction in user units.

Default: 0

`-rotate`

Transform to apply to objects. ({MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90})

Default: R0

`-rows`

Number of rows to create, must be a positive integer.

Default: 1

`-cols`

Number of columns to create, must be a positive integer.

Default: 1

`-dxRow`

X distance for row separation.

`-dyRow`

Y distance for row separation.

`-dxCol`

X distance for column separation.

`-dyCol`

Y distance for column separation.

`-keepNets`

Controls how nets and labels are copied. ({ all | labeled | none }, Default is: labeled)

`-chop`

Chops wires under the created instances. The default is false.

`-dest`

Design where objects will be pasted.

Returns

{<deFigure>} | {<oaFig>} : Collection of newly copied objects.

{<deFigure>} is returned only if the `se::copy` command is called inside the transaction, otherwise nothing is returned.

Errors

- Invalid design, design not opened for write.
- Invalid command-line argument.

Examples

The following example calls the command inside the transaction. The reference to the copied object is returned, and you can access the copied object's attributes.

```
set inst [de::getSelected -design $oaDesign]
set t [de::startTransaction copyTrans -design $oaDesign]
set g [se::copy $inst -dx 2]
de::endTransaction $t
db::listAttrs -of $g
```

The following example copies a rectangle object.

```
set r [se::createRectangle {{1 1} {2 2}} -design $oaDesign]
se::copy $r -dx 2 -dy 2 -rows 3 -cols 2 -dxRow 1 -dyRow 2 -dxCol 2 -dyCol
1
```

se::createArc

Description

Creates an arc which lays on an ellipse defined by the given bounding box. Angles are calculated relative to the positive X axis relative to the ellipse center. The `-startAngle` and `-stopAngle` arguments must both be positive and their difference must be a positive value less then or equal to 180 degrees. For symbol designs, the Schematic Editor creates the object on the *device* with layer number *231* and purpose *drawing*. For schematic designs, the Schematic Editor creates the object on the *text* layer with layer number *230* and purpose *drawing*.

Syntax

```
se::createArc <box: box> -design <oaDesign> -startAngle <float> -endAngle
<float> [-stipple <string>] [-lineStyle <string>] [-color <string>]
```

Arguments

box

Rectangle defining extents of the ellipse used to define the new arc.

`-design`

Design to contain the new object.

`-startAngle`

Starting angle of the arc in degrees; must be a positive number.

`-endAngle`

Ending angle of the arc in degrees; must be a positive number.

`-stipple`

Name of an existing stipple pattern to apply as a style override.

Default: stipple used in display packet.

`-lineStyle`

Name of an existing line style pattern to apply as a style override.

Default: lineStyle used in display packet

`-color`

Name of an existing color to apply as a style override.

Default: color used in display packet

Returns

The new `oaArc` object (`oaArc`).

Errors

- Invalid design, design not opened for write.
- Design not of view type schematic or symbol.
- Invalid box.
- Invalid stipple, linestyle or color name.
- `stopAngle` minus `startAngle` must be equal to or less than 180 degrees and must be greater than 0.
- `startAngle` and `stopAngle` must be positive.

Examples

This example creates an `oaArc` object.

```
se::createArc {{0 0} {2 1}} -design $oaDesign -startAngle 10 -endAngle 80  
-color red -lineStyle mediumLine
```

se::createBlock

Description

Helps to support "top-down" design methodology by allowing to create and place the blocks in a design prior to creating the text or schematic views.

Syntax

```
se::createBlock -design <oaDesign> -cellView <dmCellView> [-points  
  [<point>]] [-origin <point>]
```

Arguments

-design

The design in which a block should be added.

-cellView

The cell view of block to add.

-points

The points to use for creating shape of a new block.

-origin

Location of the origin of block.

Returns

<oaInst>: The instance of new created block.

Examples

The following example creates and places a block with the specified dimensions in the parent design if the block master does not exist.

```
se::createBlock -design $oaDesign -cellView [dm::findCellView\  
  symbol -cellName $cn -libName $ln] -points {{0 0} {1 1}}
```

The following example places an existing block at a specified position.

```
se::createBlock -design $oaDesign -cellView [dm::findCellView \  
  symbol -cellName $cn -libName $ln] -origin {5 5}
```

se::createCheckOption

Description

Creates a check option object.

Syntax

```
se::createCheckOption <name: string> -description <string> -value <bool>
```

Arguments

name

Option name.

-description

Option description.

-value

Option value.

Returns

<seCheckOption>: The check option that is created.

Examples

The following example creates a rule that has two options: **Case Sensitive** and **Ignore Global Symbol**. The options are passed to the `executeCheck` procedure and are shown in the **Check Options** dialog box (as shown in the figure below).

```
set optionsList [list]

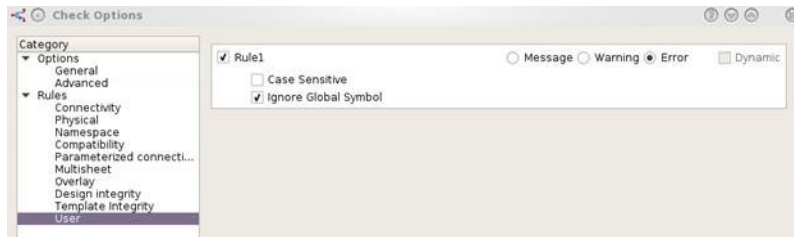
lappend optionsList [se::createCheckOption "caseSensitiveOption"
    -description "Case Sensitive" -value 0]

lappend optionsList [se::createCheckOption "ignoreGlobalOption"
    -description "Ignore Global Symbol" -value 1]

set optionsCollection [db::createCollection $optionsList]

se::createRule "Rule1" -proc executeCheck -desc "Description for Rule
1" -options $optionsCollection -prefName prefForRule

proc executeCheck { design options } {
    #...
}
```



se::createConnectDef

Description

Applies a connection definition to the given pin or wiring object. A connection definition label will be created which will effectively represent the underlying connection definition. The terminal or net associated with the pin or wiring object respectively will be used to hold the connection definition in the database. If the terminal or net already has a connection definition, this command will fail and throw a Tcl error. If the `-origin` command line argument is not given, the center of the wiring object or the top of the pin connection bounding box will be used as the location for the label object. The given object will be grouped with the label object in a parent child relationship grouping with the pin or wiring object being the parent.

Syntax

```
se::createConnectDef <name: string> -object <deFigure | oaPinFig>
    -defaultNet <string> [-origin <point>] [-just <enum>] [-orient <enum>]
    [-font <enum>] [-height <float>]
```

Arguments

`name`

Name of the connection definition parameter.

`-object`

Pin figure or wiring object for the connection definition to apply.

`-defaultNet`

Name of the connection definition default net.

`-origin`

Origin for the connection definition label.

Default: Center of wiring object or above pin

`-just`

Justification of the connection definition with respect to the origin. ({centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight}).

Default: lowerCenter

`-orient`

Orientation of the connection definition. ({MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90}).

Default: R0

`-font`

Font style with which the connection definition is displayed. ({euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish}).

Default: stick

`-height`

Height of the connection definition.

Default: 0.0625

Returns

oaEvalText object representing the connection definition label. (*oaEvalText*)

Examples

This example creates a connection definition called `power` for the given wire segment.

```
set tr [de::startTransaction tr1 -design $oaDesign]
set w [se::createWire {{1 1} {2 1}} -design $oaDesign]
de::endTransaction $tr
se::createConnectDef power -object $w -defaultNet vdd! -origin {2 1.125}
```

se::createConstraintNote

Description

Creates schematic notes formatted for Laker constraint extraction.

Syntax

```
se::createConstraintNote [<name: string>] -design <oaDesign> -anchor  
<point> [-color <string>] [-orient <string>] [-just <string>] [-font  
<string>] [-height <float>] -objects [<string>] -type <string>  
[-deviceType <string>] [-modelName <string>] [-guardRings <int>]  
[-pattern <string>] [-xSpacing <float>] [-ySpacing <float>] [-keepOrder
```

```
<bool>] [-attractToSymmetryLine <string>] [-axis <string>] [-swappable  
<string>] [-fingers <string>] [-segments <string>] [-aspectRatios  
<string>] [-variationOrientations <string>]
```

Arguments

name

The new constraint name.

-design

Design in which to create the constraint note.

-anchor

The position of the label in the user units.

-color

The color of the text.

-orient

Text Orientation. (Default is: R0)

-just

Text Justification. (Default is: lowerLeft)

-font

Font style for the text. (Default is: stick)

-height

Font height in user units. (Default is: .0625)

-objects

The objects to constrain. Objects can contain instance names and constraint names and supports wildcards.

-type

Constraint type.

-deviceType

Filter instances by device type.

-modelName

Filter instances by model name.

-guardRings

The number of guard rings to create. (Default is: 0)

`-pattern`

The pattern for cluster or matching constraint types. The expression for symmetry constraint types.

`-xSpacing`

The X spacing in user units.

`-ySpacing`

The Y spacing in user units.

`-keepOrder`

Keep order of objects in cluster. (Default is: 1)

`-attractToSymmetryLine`

Attract objects to the symmetry line. (Default is: 0)

`-axis`

Axis for symmetry constraints. (Default is: vert)

`-swappable`

Allow swapping of first and second elements in symmetry pairs. (Default is: true)

`-fingers`

List of allowable number of fingers for MOS devices in variation constraint.

`-segments`

List of allowable number of segments for resistors variation constraint.

`-aspectRatios`

List of allowable aspect ratios for capacitors in variation constraint.

`-variationOrientations`

List of allowable orientations for variation constraint. (Default is: all)

Returns

<oaText> : The constraint text object.

se::createEllipse

Description

Adds an ellipse to the given design. For schematicSymbol designs, the Schematic Editor creates the object on the *device* layer with layer number 231 and purpose *drawing*. For

schematic designs, the Schematic Editor creates the object on the *text* layer with layer number 230 and purpose *drawing*.

Syntax

```
se::createEllipse <box: box> -design <oaDesign> [-stipple <string>]  
[-lineStyle <string>] [-color <string>]
```

Arguments

`box`

Rectangle defining extents of the new ellipse object.

`-design`

Design to contain the new object.

`-stipple`

Name of an existing stipple pattern to apply as a style override for this shape

Default: stipple used in display packet

`-lineStyle`

Name of an existing line style pattern to apply as a style override.

Default: lineStyle used in display packet

`-color`

Name of an existing color to apply as a style override.

Default: color used in display packet

Returns

The new `oaEllipse` object (*oaEllipse*)

Errors

- Invalid design, design not opened for write.
- Design not of view type schematic or schematicSymbol.
- Invalid box.
- Invalid stipple, linestyle or color name.

Examples

This example creates an `oaEllipse` object.

```
se::createEllipse {{1 1} {2 3}} -design $oaDesign -stipple cross -color  
blue
```

se::createEMDB

Description

Measures terminal currents and stores the data in a database that can be used to annotate schematics and layouts.

Syntax

```
se::createEMDB <filePath: string> (-resultsDir <string> | -resultFile  
<string>) [-netlist <string>] [-nameMap <nlNameMap | string>]  
[-startTime <string>] [-stopTime <string>]
```

Arguments

`filePath`

Output file name.

`-resultsDir`

The SAE results directory.

`-resultFile`

The input simulator waveform file.

`-netlist`

The netlist associated with the specified `-resultFile`.

`-nameMap`

The name map for the specified `-resultFile`. Accepts either a name map object or the directory where name mapping data is stored.

`-startTime`

Compute currents based on given start time. (Default is:)

`-stopTime`

Ignore values after given stop time. (Default is:)

Returns

`<string>` : Database name

se::createInst

Description

Adds an instance of a specified master design in a given parent design. There are two versions of the command. One defines the master cell using a design object, and the other defines the master cell by library, cell, and view names.

If you do not provide the `-name` argument, a default scalar instance name is generated. If you provide a scalar name, or if a default name is generated, the command creates a scalar instance object. If you provide a vectored name, the command creates a vectored (iterated) instance.

If you provide a list of name/value pairs with the `-params` argument, the command applies these parameters to the instance as either `oaParams` or `oaProps`, depending on whether the master design is a `superMaster` (a `PCell`).

If a given parameter corresponds to a `superMaster oaParam`, then it is created as an `oaParam` and used during instance creation. If the parameter does not correspond to a `superMaster oaParam` but corresponds to a CDF parameter, then it is created as an `oaProp` on the newly created instance, based on the type matching defined in the CDF. If the given parameter name does not correspond to a `superMaster oaParam` or a CDF parameter, an error occurs and the instance is not created.

Syntax

```
se::createInst [<name: string>] -design <oaDesign> -origin <point>  
  [-master <oaDesign>] [-libName <string>] [-cellName <string>] [-viewName  
  <string>] [-orient <enum>] [-params [<string>]]
```

Arguments

`name`

Instance name.

`-design`

Design to contain the new object.

`-origin`

Origin of instance.

`-master`

Master design to instantiate. Must have a view type of schematic or schematicSymbol.

`-libName`

Master library name to instantiate.

Default: Library of parent design

`-cellName`

Master cell name to instantiate.

`-viewName`

Master view name to instantiate. Must have a view type of schematic or schematicSymbol.

`-orient`

Orientation of the instance. ({ MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90 }, default is: R0)

`-params`

Parameter values provided as an array {name value name value} or as a map {{name value} {name value}}.

Returns

oalnst object ([oalnst](#))

Errors

- Invalid design, design not opened for write.
- Invalid instance name.

Examples

The following example creates an instance and passes parameters.

```
se::createInst -design $oaDesign -origin {1 1} -libName analogLib  
-cellName nmos4 -viewName symbol -params {w 3u l 2u}
```

se::createLine

Description

Adds a line or path object to the given design. To create an oaPath object instead of the default oaLine object, specify a width, in user units, greater than zero. The path endtype for oaPath objects is always round. For schematicSymbol designs, the Schematic Editor creates the object on the *device* layer with layer number 231 and purpose *drawing*. For schematic designs, the Schematic Editor creates the object on *text* layer with layer number 230 and purpose *drawing*.

Syntax

```
se::createLine <[<point>]: [point]> -design <oaDesign> [-stipple  
<string>] [-lineStyle <string>] [-color <string>] [-width <float>]
```

Arguments

`point`

Vertices of the new object.

`-design`

Design to contain the new object.

`-width`

Width of the object in user units. A width of zero creates an `oaLine` object. A width greater than zero creates an `oaPath` object.

Default: 0

`-stipple`

Name of an existing stipple pattern to apply as a style override.

Default: stipple used in display packet

`-lineStyle`

Name of an existing line style pattern to apply as a style override.

Default: `lineStyle` used in display packet

`-color`

Name of an existing color to apply as a style override.

Default: color used in display packet

Returns

The new `oaLine` or `oaPath` object ([oaShape](#)).

Errors

- Invalid design, design not opened for write.
- Design not of view type schematic or schematicSymbol.
- Width value is negative.
- Invalid stipple, linestyle or color name.

Examples

This example creates an `oaLine` object.

```
se::createLine {{1 1} {2 3} {4 3} {4 2}} -design $oaDesign -stipple cross  
-color blue -width 0.25
```

se::createMask

Description

Creates an overlay mask.

Syntax

```
se::createMask <box: box> -design <oaDesign> [-type <enum>]
```

Arguments

box

The box of the mask rectangle.

-design

The design in which to create the mask.

-type

The overlay mask type. ({ clip | mask }, default is: mask)

Returns

<oaRect> : The new overlay mask added to the given design

se::createNetColorizationDefinition

Description

Creates a net colorization definition.

Syntax

```
se::createNetColorizationDefinition <name: string> [-description  
<string>] [-pattern <string>] -color <string> [-lineStyle <string>]
```

Arguments

name

Definition name.

-description

Definition description.

-pattern

Definition pattern.

`-color`

Definition color.

`-lineStyle`

Definition line style. The default is solid.

Returns

<seNetColorizationDefinition> : Net colorization definition object

Examples

```
set design [ed]
se::createNetColorizationDefinition clock -description Clock -color red
    -lineStyle longDash
se::createNetColorizationDefinition ground -description Ground -color
    blue -lineStyle thickLine
set tr [de::startTransaction "Create Wire" -design $design]
# Create wire segment
set w [se::createWire {{1 1} {3 1}} -design $design]
# Add wire name
se::createWireName clk -wire $w -abut true
de::endTransaction $tr
# Colorize net
se::colorizeNets clk -design $design -definitions clock
set tr [de::startTransaction "Create Wire" -design $design]
set w [se::createWire {{1 2} {3 2}} -design $design]
se::createWireName gnd! -wire $w -abut true
de::endTransaction $tr
se::colorizeNets gnd! -design $design -definitions ground
```

se::createOPDB

Description

Creates an annotation DB from simulator OP results.

Syntax

```
se::createOPDB <outputFile: string> -opFile <string> -nameMap <nlNameMap
    | string> [-vroot <string>] [-force <bool>]
```

Arguments

`outputFile`

The output file name.

`-opFile`

Input OP file name.

`-nameMap`

The name map for the netlist used to create the specified input file. Accepts either the name map object (result of `nl::openNameMap`) or the directory where name mapping data is stored (`cnl`).

`-vroot`

Virtual root path. The path in netlist namespace for the design under test.

`-force`

Forces the creation of databases, disregarding file timestamps. The default is 0.

Returns

<none>

Examples

```
# Create annotation DB from simulator OP results.
se::createOPDB op.db -opFile ./latestResults/results/hspice_wdf.op0
  -nameMap ./latestResults/netlist/cnl
# Back annotate OP results onto an active schematic window
se::annotateOP -db ./op.db -window [gi::getActiveWindow] -params true
  -voltage true
```

se::createPin

Description

Creates a pin in the given design. The design can be of view type schematic or symbol. This command creates an `oaTerm` object if needed, then creates the shape or instance object, adds the shape or instance to a newly created `oaPin` object, and returns this pin object.

If you provide a single point, the command uses a predefined sized shape and considers the point as the center of the shape. This is for all shape types except polygon. The predefined size is 80% of the current snap grid for the given design. Shape types of rectangle, polygon, and circle can also have a user-defined shape. For rectangle shapes, you must provide two points to define opposite corners. For polygon shapes, you must provide three or more points to describe the vertices of the closed polygon. For circle shapes, you must provide two points, where the first defines the center point and the second defines the circle radius.

For all shapes types except custom, which creates an `oaRef` object, the `oaShape` object is created on the pin layer purpose pair.

When creating a custom shape, the predefined layout, cell, and view for the given pin type is used from the tool environment, but you must provide a single point with the `-points`

argument. This point is in user units and is the origin of the layout, cell, and view in the specified design.

For all pins, a pin name label will be created in the same manner as `ise::createSchematicPin` or `ise::createSymbolPin` depending on the view type of the design.

Syntax

```
se::createPin <name: string> -design <oaDesign> -type <enum> -points  
[<point>] [-shape <enum>] [-orient <enum>] [-assignmentName <string>]  
[-defaultNet <string>] [-optical <bool>]
```

Arguments

`name`

Pin name.

`-design`

Design to contain the new pin.

`-type`

Terminal type for pin.

`-points`

Tcl list of points for pin shape definition.

`-shape`

Shape type for pin.

Default: custom

`-orient`

Orientation of the pin.

Default: R0

`-assignmentName`

Assignment name for connect definition to be added to pin.

`-defaultNet`

Name of default net for connection definition.

`-optical`

Creates an optical pin. The default is false.

Returns

oaPin object ([oaPin](#)).

Errors

- Invalid design, design not opened for write.
- Invalid point or points in <point> list.
- -defaultConnection given without -assignmentName.
- -defaultConnection and -assignmentName given with a non-scalar pin name.
- Name outside of the legal name space given.

Examples

This example creates two pins: the first example does not define a pin shape so the tool will apply the default pin shape; the second example pin defines a square pin shape.

```
se::createPin a -design $oaDesign -type input -points {{1.25 1}} -shape  
custom
```

```
se::createPin a -design $oaDesign -type input -shape square -points {{1  
1} {1.125 1.125}}
```

se::createPolygon

Description

Adds a polygon to the given design. For schematicSymbol designs, the Schematic Editor creates the object on the *device* layer with layer number 231 and purpose *drawing*. For schematic designs, the Schematic Editor creates the object on the *text* layer with layer number 230 and purpose *drawing*.

Syntax

```
se::createPolygon <[<point>]: [point]> -design <oaDesign> [-stipple  
<string>] [-lineStyle <string>] [-color <string>]
```

Arguments

point

Vertices of the new polygon object.

-design

Design to contain the new polygon object.

`-stipple`

Name of an existing stipple pattern to apply as a style override.

Default: stipple used in display packet.

`-lineStyle`

Name of an existing line style pattern to apply as a style override.

Default: lineStyle used in display packet

`-color`

Name of an existing color to apply as a style override.

Default: color used in display packet

Returns

The new `oaPolygon` object (*oaPolygon*).

Errors

- Invalid design, design not opened for write.
- Design not of view type schematic or schematicSymbol.
- Point array do not describe closed polygon.
- Less than 3 points in point array.
- Invalid stipple, linestyle or color name.

Examples

This example creates an `oaPolygon` object.

```
se::createPolygon {{1 1} {2 3} {4 3} {4 2}} -design $oaDesign -stipple  
brick -color blue
```

se::createRectangle

Description

Adds a rectangle to the given design. For `schematicSymbol` designs, the Schematic Editor creates the object on the *device* layer with layer number *231* and purpose *drawing*. For `schematic` designs, the Schematic Editor creates the object on the *text* layer with layer number *230* and purpose *drawing*.

Syntax

```
se::createRectangle <box: box> -design <oaDesign> [-stipple <string>]  
[-lineStyle <string>] [-color <string>]
```


Arguments

`box`

Rectangle defining extents of the new object.

`-design`

Design to contain the new object.

`-stipple`

Name of an existing stipple pattern to apply as a style override.

Default: stipple used in display packet

`-lineStyle`

Name of an existing line style pattern to apply as a style override.

Default: lineStyle used in display packet

`-color`

Name of an existing color to apply as a style override

Default: color used in display packet

Returns

The new `oaRect` object (*oaRect*)

Errors

- Invalid design, design not opened for write.
- Design not of view type schematic or schematicSymbol.
- Invalid stipple, linestyle, or color name.

Examples

This example creates an `oaRect` object.

```
se::createRectangle {{1 1} {2 3}} -design $oaDesign -stipple cross -color  
blue
```

se::createRule

Description

Registers a new rule check into the ERC/SRC system. The check requires a name, a Tcl procedure to execute, and a description. If a rule check already exists with the same name, the command generates a Tcl error. An `seRule` object is returned. To remove this

specific rule from the system, this object or a copy of it should be passed to `db::destroy`. Note that after destroying the object, all other Tcl variables referencing this rule will be invalid. Also note that passing an internally defined rule to `db::destroy` does not destroy or remove the rule from the system.

Syntax

```
se::createRule <string: string> -description <string> -proc <string>  
[-defaultSeverity <enum>] [-category <string>] [-allowDynamic  
<bool>] [-allowDisable <bool>] [-forceFatal <bool>] -prefName  
<string> [-isIntegrityCheck <bool>] [-viewTypes [<string>]] [-options  
<dbCollection>]
```

Arguments

`string`

Unique label for this rule check. It will be the display name presented to the user.

`-proc`

Name of Tcl procedure that implements the rule check.

`-description`

Short description of check being registered. This text is used as the short description on `oaMarker` objects and for console messages, in addition to labeling the check in the ERC/SRC options dialog box.

`-defaultSeverity`

Default severity level. ({ error | message | warning }, default is: message)

`-category`

Name of the category for this rule check. (Default is: user)

`-allowDynamic`

Indicates whether the check should be available for executing dynamically when connectivity edits are made to a design. (Default is: false)

`-allowDisable`

Indicates whether this check can be disabled. (Default is: true)

`-forceFatal`

Indicates whether violations from this rule should be forced to fatal severity level regardless of the severity settings from the preference. (Default is: false)

`-prefName`

Name of preference to correspond with this rule check. The preference is created during the command and the value is an association list with pairs for severity and dynamic.

`-isIntegrityCheck`

Indicates whether the check could be done hierarchically. (Default is: false)

`-viewTypes`

Specifies the view types on which to run the rules. Valid values are `schematic` and `schematicSymbol`.

If `-viewTypes` is set to `schematic` for a rule, it will be run on schematic views and for Hierarchy Checks it will be run on the switch masters of instances. If `-viewTypes` is set to `schematicSymbol` view, the rule can be run on symbol views. For Hierarchy and Current View checks, the placed masters of instances will also be checked.

`-options`

Passes user-defined options to the "check" (-proc argument) Tcl procedure. These options are also exposed in the **Check > Setup** dialog box to allow you to configure them through the GUI (see the example below). Each `seCheckOption` is an object that has a name (string), a description (string), and a value (bool) arguments. An `seCheckOption` is created with the `se::createCheckOption` command.

Returns

<seRule> : Newly created rule object.

Errors

- Rule already exists.
- Given preference name already exists.

Examples

In the following example, a simple rule is developed that checks whether the design contains an `oaTerm` object with the name `a`.

```
proc termNameCheck { design } {  
    set violations {}  
    set terms [db::getTerms -of $design]  
    db::foreach term $terms {  
        set name [db::getattr name -of $term]  
        if {$name=="a"} {
```

```

        set vlist [se::createViolation "Terminal $name is detected" design
$design -shortMessage "Find terminal with name a" -polygon {{1 1} {1.125
1.125} {1.25 1} {1.125 0.875}}]
        lappend violations $vlist
    }
}
if { [llength $violations] == 0 } {
    return {}
} else {
    return [db::createCollection $violations]
}
}

```

```

se::createRule termName -proc termNameCheck -desc "Find terminal with
name a" -prefName termNameCheck -defaultSeverity error

```

The following example creates a rule that has two options: **Case Sensitive** and **Ignore Global Symbol**. The options are passed to the `executeCheck` procedure and are shown in the **Check Options** dialog box (as shown in the figure below).

```

set optionsList [list]

lappend optionsList [se::createCheckOption "caseSensitiveOption"
-description "Case Sensitive" -value 0]

lappend optionsList [se::createCheckOption "ignoreGlobalOption"
-description "Ignore Global Symbol" -value 1]

set optionsCollection [db::createCollection $optionsList]

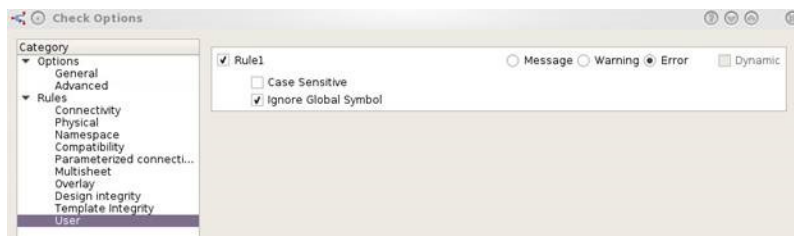
se::createRule "Rule1" -proc executeCheck -desc "Description for Rule
1" -options $optionsCollection -prefName prefForRule

proc executeCheck { design options } {

#...

}

```



The following example illustrates the use of the `-viewTypes` option.

```

se::createRule "Rule1" -viewTypes "schematic" -proc executeCheck
-desc "Description for Rule 1" -prefName prefForRule1

```

```
se::createRule "Rule2" -viewTypes "schematicSymbol" -proc
executeCheck -desc "Description for Rule 2" -prefName prefForRule2

se::createRule "Rule3" -viewTypes [list schematicSymbol schematic]
-proc executeCheck -desc "Description for Rule 3" -prefName
prefForRule3
```

Rule1: Can be run on schematic views. For Hierarchy Checks (see the figure below), the rule will be run on switch masters.

Rule2: Can be run on symbol views. For Hierarchy and Current Views checks, the rule will be run on placed masters.

Rule3: Can be run on both Schematic and Symbol views. For Current View Checks, the placed masters of instances will also be checked. For Hierarchy Checks, both placed and switch masters will be checked.



se::createSelectionShape

Description

Creates a rectangle or a polygon that represents both the selection region and the avoidance region for the design on the *instance* layer with layer number 236 and purpose *drawing*.

Syntax

```
se::createSelectionShape [<point: [point]>] -design <oaDesign>
[-overwrite <bool>]
```

Arguments

point

Vertices for the selection shape object. Creates a rectangle when the number of given points is two; creates a polygon when more than two points are given.

-design

Design to contain the new object.

`-overwrite`

If true, deletes all existing objects on layer 236 and purpose "drawing" before creating the new shape.

Default: false

Returns

The new `oaPolygon` or `oaRect` object ([oaShape](#)).

Errors

- Invalid design, design not opened for write.
- Design not of view type schematic or schematicSymbol.

Examples

This example creates a rectangular selection shape.

```
se::createSelectionShape {{1 1} {2 3}} -design $oaDesign
```

se::createTable

Description

Creates a schematic table object in the given design.

Syntax

```
se::createTable -design <oaDesign> -origin <point> [-rows <int>]  
[-columns <int>] [-name <string>] [-title <string>] [-namePosition  
<enum>] [-titlePosition <enum>] [-rotateFirstRowText <bool>] [-autoSize  
<bool>] [-width <float>] [-height <float>] [-color <string>] [-lineStyle  
<string>] [-font <enum>] [-fontHeight <float>] [-textJust <enum>]  
[-textColor <string>] [-cellData <matrix>]
```

Arguments

`-design`

Design in which to create the table.

`-origin`

Upper left corner point for table.

`-rows`

Number of rows. (Default is: 2)

`-columns`

Number of columns. (Default is: 2)

`-name`
Table name (Default is:)

`-title`
Table title (Default is:)

`-namePosition`
Table name label position. ({ bottomCenter | bottomLeft | bottomRight | hide | topCenter | topLeft | topRight }, default is: topLeft)

`-titlePosition`
Table title position. ({ bottomCenter | bottomLeft | bottomRight | hide | topCenter | topLeft | topRight }, default is: topRight)

`-rotateFirstRowText`
Indicates whether the first row of text of the table should be rotated 90 degrees. (Default is: false)

`-autoSize`
Indicates whether the cell size should be adjusted to the table content. (Default is: true)

`-width`
Cell width. (Default is: 0.125)

`-height`
Cell height. (Default is: 0.0875)

`-color`
Color of the table.

`-lineStyle`
Line style of the table.

`-font`
Font style with which the table is displayed. ({ euroStyle | fixed | gothic | math | milSpec | roman | script | stick | swedish }, default is: stick)

`-fontHeight`
Font height. (Default is: 0.0625)

`-textJust`

Justification of the table cell contents. ({ centerCenter | centerLeft | centerRight | lowerCenter | lowerLeft | lowerRight | upperCenter | upperLeft | upperRight }, default is: centerCenter)

`-textColor`

Text color.

`-cellData`

List of row contents, which in turn are lists of strings.

Returns

`<seTable>` : Schematic table object.

Errors

- Given design is not schematic or symbol.
- Invalid value specified for option `cellData`.
- `-cellData` option has more rows than table is going to have, extra values will not be used.
- `-cellData` option has more columns than table is going to have, extra values will not be used.

Examples

```
set design [ed]
se::createTable -design $design -origin {1 1} -rows 5 -columns 3 -title
  "Truth Table" \
-cellData {{in1 in2 out} {0 0 0} {0 1 0} {1 0 0} {1 1 1}} -height 0.2
  -width 0.2 \
-rotateFirstRowText true -name " " -textColor orange -autoSize false
```

se::createText

Description

Creates a text object in the given design. If the text type is 'note' or 'normal', the Schematic Editor creates an `oaText` object. For all other types, the Schematic Editor creates an `oaEvalText` object linked to the appropriate evaluator. If the text type is note, the Schematic Editor creates an object on "text" layer number 230 and purpose "drawing". For all other types, the Schematic Editor creates the object on "annotate" layer number 237 and purpose "drawing".

Syntax

```
se::createText <text: string> -design <oaDesign> -anchor <point>  
[-noteShape <oaShape>] [-color <string>] [-type <enum>] [-just <enum>]  
[-orient <enum>] [-drafting <bool>] [-overbar <bool>] [-font <enum>]  
[-height <float>]
```

Arguments

text

Label or note text.

-design

Design in which to create the label.

-anchor

Position of the label, in user units.

-noteShape

Shape object for associating with text, only considered if type is 'note'.

Default: normal

-color

Name of the color to apply as a style.

Default: color used in display packet

-type

Type of text to create.

Default: normal

-just

Justification of the label.

Default: lowerCenter

-orient

The orientation of the label.

Default: R0

-drafting

Indicates whether the text should not be mirrored or rotated.

Default: true

`-overbar`

Draws a bar over the text, if true.

Default: false

`-font`

Display font for the label.

Default: stick

`-height`

Display size, in user units, for the label.

Default: 0.0625

Returns

An `oaText` object if `evalType` is 'note' or 'normal', else an `oaEvalText` object (*oaText*).

Errors

- Invalid design, design not opened for write.
- Design not of view type schematic or schematicSymbol.
- Invalid `evalType` specified.
- Invalid justification specified.
- Invalid color specified.
- `oaShape` not on text LPP.

Examples

The following example creates a label.

```
se::createText _Label -design $oaDesign -anchor {1 1} -orient MXR90  
-overbar true
```

The following example creates an `oaEvalText` object that evaluates to the instance name.

```
se::createText cdsName() -design $oaDesign -anchor {1 1} -type IL -color  
red -font swedish -height 0.15
```

se::createVDRcdb

Description

Creates a VDRc database.

Syntax

```
se::createVDRcdb <filePath: string> (-resultsDir <string> | -resultFile  
<string>) [-netlist <string>] [-nameMap <nlNameMap | string>]  
[-startTime <string>] [-stopTime <string>] [-measType <eMeasType>]
```

Arguments

`filePath`

Output file name.

`-resultsDir`

The SAE results directory.

`-resultFile`

The input simulator waveform file.

`-netlist`

The netlist associated with the specified `-resultFile` argument.

`-nameMap`

The name map for the specified `-resultFile` argument. Accepts either a name map object or the directory where name mapping data is stored.

`-startTime`

Compute voltages based on given start time. (Default is:)

`-stopTime`

Ignore values after given stop time. (Default is:)

`-measType`

Measurement type. ({ minMax | baseTopLine }, default: minMax)

Returns

`<string>` :

se::createViolation

Description

Creates a violation container object. Rule check implementations are required to return a collection of `deViolation` objects where each object represents a specific error, warning, or message that the rule wishes to identify in the design. If the user saves violations as markers, each violation object creates an `oaMarker` object in the design during

`ise::check` or `se::check`. This object also dictates the contents of the formatted text string that are sent to the console by the `ise::check` command.

Syntax

```
se::createViolation <string: string> [-polygon [<point>]] -design  
  <heContext | oaDesign> [-objects [<oaBlockObject>]] [-deleteWhen  
  <enum>] [-severity <enum>] [-shortMessage <string>] [-toolName <string>]  
  [-probeType <enum>] [-probeObjectName <string>]
```

Arguments

`string`

Required description for the violation.

`-design`

Design where violation originates.

`-severity`

Severity level of the violation.

Default: message

`-polygon`

Required Tcl list of points describing violation region.

`-shortMessage`

Short description of the violation.

Default: ""

`-toolName`

Name of tool that found the violation.

Default: ""

`-objects`

Tcl list of block objects associated with the violation.

`-deleteWhen`

Controls when the marker object should be removed from the design with respect to the objects associated with the violation.

Default: never

`-probeType`

Type of probe to create required for context-based violations ({ cellview | instTerm | instance | net | path | terminal })

`-probeObjectName`

Probe object name required for context-based violations

Returns

Newly created violation object. ([deViolation](#))

Errors

Invalid argument.

Examples

In this example, a simple rule is developed that checks if the design contains an `oaTerm` object with the name `a`.

```
proc termNameCheck { design } {  
    set violations {}  
    set terms [db::getTerms -of $design]  
    db::foreach term $terms {  
        set name [db::getattr name -of $term]  
        if {$name=="a"} {  
            set vlist [se::createViolation "Terminal $name is detected" design  
$design -shortMessage "Find terminal with name a" -polygon {{1 1} {1.125  
1.125} {1.25 1} {1.125 0.875}}]  
            lappend violations $vlist  
        }  
    }  
    if { [llength $violations] == 0 } {  
        return {}  
    } else {  
        return [db::createCollection $violations]  
    }  
}  
  
se::createViolation termName -proc termNameCheck -desc "Find terminal  
with name a" -prefName termNameCheck -defaultSeverity error
```

se::createWire

Description

Creates a wire between two given points using the point-to-point routing engine and taking into account all the avoidance options. If you specify the `-width` argument, the command creates an `oaPath` object to represent the new wire else the command creates an `oaLine` object.

The command returns a list of `oaShape` objects created only if the command was invoked inside a transaction. Otherwise, the command returns nothing.

Syntax

```
se::createWire <points: [point]> -design <oaDesign> [-width <float>]  
[-mode <enum>] [-avoidSymbols <bool>] [-avoidShorts <bool>] [-jogStyle  
<enum>] [-color <string>]
```

Arguments

`points`

Start and end points of wire.

`-design`

The oaDesign in which to create the wire.

`-width`

Wire width in user units.

Default: 0

`-mode`

Wiring mode.

Default: auto

`-avoidSymbols`

Specifies whether to avoid symbols.

Default: seWiringAvoidSymbols

`-avoidShorts`

Specifies whether to avoid name shorts.

Default: seWiringAvoidShorts

`-jogStyle`

Specifies the initial jog style.

Default: auto

`-color`

Name of an existing color to apply as a style override.

Default: color used in the display packet

Returns

{<oaShape>} : The new wire.

Errors

- Invalid design, design not opened for write.
- Invalid command line argument.
- Number of supplied points not two.
- 'guided' given as wiring mode.

Examples

The following example creates a wire that has 2 segments. The first segment is horizontal and the second is vertical:

```
se::createWire {{2 2} {3 3}} -design $ oaDesign -jogStyle
```

The following example creates a wire that has 1 segment (wiring mode is `any`):

```
se::createWire {{2 2} {3 3}} -design $oaDesign -mode any
```

The following example creates 2 wires that do not overlap because the `-avoidShorts` argument is used:

```
se::createWire {{1 2} {1 3}} -design $oaDesign
se::createWire {{0 2} {2 2}} -design $oaDesign -avoidShorts true -width
0.0625
```

The following example creates wires within the specified transaction and then connects the wires in the specified design.

```
set tr [de::startTransaction tr1 -design $oaDesign]
set w1 [se::createWire {{1 1} {3 1}} -design $oaDesign]
set w2 [se::createWire {{2 0} {2 2}} -design $oaDesign]
de::endTransaction $tr
se::connectWires -design $oaDesign -wire1 $w1 -wire2 $w2
```

se::createWireName

Description

Adds a text string and associates it to a given wiring object. If you do not specify the wire name, the command uses the name of the current `oaNet` object assigned to the wiring object.

Syntax

```
se::createWireName [<name: string>] -wire <oaLine | oaPath> [-origin
<point>] [-just <enum>] [-orient <enum>] [-font <enum>] [-height
<float>] [-abut <bool>] [-overbar <bool>] [-color <string>]
```

Arguments

`name`

Name for the wire label.

Default: Name of oaNet of the wiring object.

`-wire`

The wire to which the label should be connected. An oaLine or oaPath wiring object.

`-origin`

Origin for the wire label.

Default: Center of wiring object

`-just`

Justification of the wire label.

Default: lowerCenter

`-orient`

Orientation of the wire label.

Default: R0

`-font`

Font for wire label.

Default: stick

`-height`

Text height of the wire label.

Default: 0.5

`-abut`

Indicates whether text bbox should abut the bbox of the given wire.

Default: True

`-abut`

Whether the wire and wire label abut.

Default: false

`-overbar`

Whether the wire label will have an overbar.

Default: false

-color

Color of the wire label.

Returns

oaText object (*oaText*)

Examples

This example adds a name to an unnamed wire. When `abut` is set to `true`, the text bounding box will abut the bounding box of the given wire.

```
set tr [de::startTransaction wire -design $oaDesign]
set w [se::createWire {{0 0} {2 0}} -design $oaDesign]
se::createWireName a -wire $w -color red -origin {1.25 1} -abut true
se::createWireName a -wire $w -color red -origin {1.5 1}
de::endTransaction $tr
```

se::createWireStubs

Description

Creates a short wire on each instance pin in the specified set of instances. Named wire "stubs" are used to reduce the routing complexity in the large designs.

Syntax

```
se::createWireStubs <Objects: {<deFigure>}> [-auto <bool>] [-size <int>]
[-wireLabel <enum>] [-pinType <enum>] [-userDefinedName <string>]
[-updateStubs <bool>]
```

Arguments

Objects

Set of objects.

-auto

Auto adjust stub to label size. (Default is: false)

-size

Size of the wire stub in snap grids. (Default is: 2)

-wireLabel

Indicates whether the nets should be auto named, should use the pin names or a user defined name. ({ none | pinName | userDefined }, default is: none).

`-pinType`

Indicates the type of pin that should be created at each end of the created stub. ({ auto | input | inputOutput | noConnPin | none | output | switch | tristate }, default is: noConnPin)

`-userDefinedName`

User defined parameterized name for nets. (Default is: %I_%P)

`-updateStubs`

Indicates whether to update already existing stubs for missed pins or labels according to the specified label or pin creation policy. (Default is: true)

Returns

{<oaShape>}: The wire stubs created.

Examples

```
set design [ed]
set inst [se::createInst -design $design -viewName symbol -cellName nmos4
-libName analogLib -origin {1 1}]
se::createWireStubs [de::getFigures $inst] -size 10 -wireLabel
userDefined -pinType auto
```

se::delete

Description

Deletes the set of specified objects. Only 0x objects are deleted. Any object in a lower level design is silently ignored.

Syntax

```
se::delete <objects: {<deFigure>} | {<oaFig>} | {<oaFigGroup>} |
{<oaInst>} | {<oaShape>}>
```

Arguments

`objects`

Collection of objects to be deleted, all objects must be from the same oaDesign.

Examples

```
se::delete [de::getSelected -design $oaDesign]
```

This example deletes all selected objects in the design.

se::disconnectWires

Description

Disconnects a set of four crossing wires at a given point. Removes the corresponding oaEllipse object (that is, the solder dot) and merges the co-linear wire segments.

The given point must be at a solder dot location and each segment must be orthogonal and have a co-linear counterpart.

Syntax

```
se::disconnectWires -design <oaDesign> -at <point>
```

Arguments

-design

Design containing wires to disconnect.

-at

Location of wire crossings to disconnect.

Examples

This example disconnects a set of four crossing wires at a given point, removes the solder dot, and merges the collinear wire segments.

```
se::createWire {{1 1} {1 3}} -design $oaDesign  
se::createWire {{0 2} {1 2}} -design $oaDesign  
se::createWire {{1 2} {2 2}} -design $oaDesign  
se::disconnectWires -design $oaDesign -at {1 2}
```

se::enableDynamicNetLabels

Description

Enables dynamic net labels and registers to get the text procedure.

Syntax

```
se::enableDynamicNetLabels -context <deContext> -applicationName <string>  
[-clientData <string>] -getTextProc <variant>
```

Arguments

-context

The design context to enable dynamic net labels.

`-applicationName`

The application name for this dynamic net label object.

`-clientData`

The client data pass to get text procedure, typically an SQL handle. (Default is:)

`-getTextProc`

Name of the procedure which evaluates the dynamic net label.

Returns

`<seTextResult>` : text result object created.

se::extractConnectivity

Description

Forces the SE connectivity engine to process the cellview for connectivity extraction.

Syntax

```
se::extractConnectivity -design <oaDesign> [-fullExtraction <bool>]
```

Arguments

`-design`

The design to extract the SE connectivity.

`-fullExtraction`

Forces the SE connectivity engine to perform the design full connectivity re-extraction regardless of timestamps set. Note that some wiring shapes, instTerms, and nets can be removed. The default is false.

Returns

`<bool>` : Boolean indicating whether or not the process was successful.

Examples

The following example gets the design from the current context and extract full connectivity.

```
# Get edit design from the current context
set des [db::getattr editDesign -of [de::getActiveContext]]

# Extract connectivity with full extraction option
se::extractConnectivity -design $des -fullExtraction true
```

se::findMaskedInstances

Description

Finds instances masked by the specified masking type.

Syntax

```
se::findMaskedInstances -design <oaDesign> -maskingType  
    <enOverlayMaskingType>
```

Arguments

-design

The design to process.

-maskingType

The effecting masking type to look for.

Returns

<oaInst> : The list of found instances.

se::findReferenceDifferences

Description

Compares reference objects in the overlay design against the reference design.

Syntax

```
se::findReferenceDifferences <overlay: oaDesign>
```

Arguments

overlay

The overlay design to check.

Returns

<string> : True if difference found, else false. If there are details of found differences, the true result is container of string information about these details: {<details_ref_vs_refInOverlay> <details_ref_vs_ovInOverlay>}, where:

- <details_ref_vs_refInOverlay> contains differences between reference design and non-modified reference objects in overlay design, and has the format:
{<instances_with_changed_masters> <added_instances> <removed_instances>
<changed_connection_to_instance_terminals> <changed_parameters_of_instances>
<instances_with_changed_positions> <added_terminals> <removed_terminals>}

- <details_ref_vs_ovlInOverlay> contains differences between reference design and overlay or modified reference objects in overlay design, and has the format: {<instances_with_changed_masters> <removed_instances> <changed_connection_to_instance_terminals> <changed_parameters_of_instances> <instances_with_changed_positions>}

se::generateEstRCSchematic

Description

Interconnect RC modeling flow command.

Syntax

```
se::generateEstRCSchematic -src <dmCellView | dmCellViewName> [-nets  
<{<oaNet>}>] -layout <dmCellView | dmCellViewName> -dst <dmCellView |  
dmCellViewName> [-estRCLib <string>]
```

Arguments

-src

Cell view of the source schematic.

-nets

Nets to generate estimated RC schematic.

-layout

Cell view of the layout.

-dst

Cell view of the destination schematic.

-estRCLib

The library for RC pcell generation.

Returns

<none>

se::generateSchematic

Description

Generates a schematic using the given interface information. It is the default generator for schematic views.

Syntax

```
se::generateSchematic -interface <dbInterfaceSet> -update <bool> -dest  
  <dmCellView | dmCellViewName> -options <dbGenOptionSet>
```

Arguments

-interface

Interface object.

-update

Indicates if the user wishes to update an existing cellview instead of regenerating it.

-dest

Destination cellview.

-options

Object containing the option values to use for schematic generation.

Examples

Example 1

In this example, the schematic is generated using the symbol view as a source for interface information and schematic generator default options.

```
# Find the source view  
set srcCellView [dm::findCellView symbol -cellName inv -libName testLib]  
# Find default options sets for Schematic Generator  
set os [db::getGenOptionSets default -filter {%setType ==  
  "schematicGenerator"}]  
# get interface set object from source view  
set is [db::getCellViewInterface -src $srcCellView]  
se::generateSchematic -interface $is -update false -dest  
  testLib/inv/schematic -options $os
```

Example 2

```
set os [db::getGenOptionSets default -filter {%setType ==  
  "schematicGenerator"}]  
set pins {}  
lappend pins [db::createInterfacePin in -type input -side left]  
lappend pins [db::createInterfacePin out -type output -side right]  
# Create new interface set  
set iso [db::createInterfaceSet -pins [db::createCollection $pins]]  
se::generateSchematic -interface $iso -update false -dest  
  testLib/inv/schematic -options $os
```

se::generateSymbol

Description

Generates a symbol using the given interface information and symbol generator options. It is registered as a generator for schematicSymbol views by default.

Syntax

```
se::generateSymbol -interface <dbInterfaceSet> -update <bool> -dest  
<dmCellView | dmCellViewName> -options <dbGenOptionSet> [-pinMap  
[<string>]]
```

Arguments

-interface

Interface object.

-update

Indicates if the user wishes to update an existing cellview instead of regenerating it.

-dest

Destination cellview.

-options

Object containing the option values to use for symbol generation.

-pinMap

The pin name mapping items.

Examples

Example 1

In this example, the symbol is generated using the schematic view as a source for interface information and symbol generator default options.

```
# Find the source view  
  
set srcCellView [dm::findCellView schematic -cellName inv -libName  
testLib]  
  
# Find default options sets for Symbol Generator  
set os [db::getGenOptionSets default -filter {%setType ==  
"symbolGenerator"}]  
  
# get interface set object from source view  
set is [db::getCellViewInterface -src $srcCellView]
```



```
# Find the destination view

set destCellView [dm::findCellView symbol -cellName inv -libName testLib]
se::generateSymbol -interface $is -dest $destCellView -update 0 -options
  $os
se::generateSymbol -interface $iso -dest $newCellView -options $gos
  -update 0
```

Example 2

In this example, the interface and symbol generator options objects are user-defined and the symbol is generated using those objects.

```
# Get Symbol Generator default option set value

set defaultOptionSet [ db::getGenOptionSets default -filter {%setType ==
  "symbolGenerator"}]
array set vals [db::getAttr value -of $defaultOptionSet]

# Modify default values of option set

set vals(unitSize) 0.125
set vals(pinStubLength) 1
set vals(pinSpacing) 3
set vals(pinAnnotationText) cdsTerm("%p")

# Create new option set for Symbol Generator

set gos [db::createGenOptionSet myset -type symbolGenerator]

# Update created option set with new values

db::setAttr value -of $gos -value [array get vals]

# Define symbol pins

lappend pins [db::createInterfacePin in -type input]
lappend pins [db::createInterfacePin out -type output]

# Create new interface set

set iso [db::createInterfaceSet -pins [db::createCollection $pins]]

# Create new cell view testLib/inv/symbol

set newCellView [dm::createCellView symbol -cell [dm::getCells inv
  -libName testLib] -viewType schematicSymbol]

# Generate symbol view

se::generateSymbol -interface $iso -dest $newCellView -options $gos
  -update 0
```

se::generateSymbolTransformPCell

Generates Pcell that allows rotation and size scaling.

Syntax

```
se::generateSymbolTransformPCell <source: dmCellView | dmCellViewName>  
[-dest <dmCellView | dmCellViewName>]
```

Returns

<none>

Arguments

source

The source symbol.

-dest

The destination cellview. Default is symbol_xform view in the same cell as source.

se::getActiveSheet

Description

Returns the currently active sheet's object or nothing.

Syntax

```
se::getActiveSheet [-window <giWindow | int>]
```

Arguments

-window

The window to get the active sheet from.

Returns

<dbSheet> : Active sheet.

Errors

- The cellview of the specified window is not a multisheet.
- The specified window is not an editor window.

Examples

```
# Get reference on sheet object from active Schematic Editor window
set sheet [se::getActiveSheet]
# Print active sheet master, description and number
puts "LCV: '[db::getAttr sheet.border.libName]/[db::getAttr
sheet.border.cellName]/[db::getAttr sheet.border.name] Description:
'[db::getAttr sheet.description]' Number: '[db::getAttr sheet.number]'"
```

se::getBillOfMaterials

Description

Computes the flattened instance count.

Syntax

```
se::getBillOfMaterials -context <heContext> [-levels <int>]
[-excludedLibraries <string>]
```

Arguments

-context

The design context in which the command is activated.

-levels

Indicates the count of hierarchy levels which will be processed. The default is 0.

-excludedLibraries

List of libraries in which to ignore cells together with their lower hierarchy.

Returns

<string> : List of {libName, cellName, viewName, count}.

Examples

```
se::getBillOfMaterials -context [db::getAttr hierarchy -of
[de::getActiveContext]]
```

se::getColorizationDefinitions

Description

Retrieves all color definitions for schematic nets colorization.

Syntax

```
se::getColorizationDefinitions [-filter <string>]
```

Arguments

`-filter`

Standard attribute filter.

Returns

`<dbCollection>` : Net colorization definitions

Examples

```
# Create Net Colorization Definition
se::createNetColorizationDefinition clock -description Clock -color red
    -lineStyle longDash
se::createNetColorizationDefinition ground -description Ground -color
    blue -lineStyle thickLine
db::foreach cd [se::getColorizationDefinitions] {
    puts [db::getattr cd.name]
}
```

se::getDynamicNetLabels

Description

Looks for registered dynamic net labels by the given context.

Syntax

```
se::getDynamicNetLabels [-context <deContext>] [-filter <string>]
```

Arguments

`-context`

The design context to look for registered dynamic net labels.

`-filter`

The filter to apply on matched result. (Default is:)

Returns

`<dbCollection>` : Collection of text result objects.

se::getNetComponents

Description

Returns a set of objects associated with a given net group. The objects are dependent on the given deFigure or oaFig given on the command line as well as the value of enNetGroup.

Syntax

```
se::getNetComponents <object: deFigure> [-of <enum>]
```

Arguments

object

Wiring object used as base for expansion.

-of

Indicates the group of objects of the net associated with given object to return.

Default: net

Returns

{<oaFig>}: The net components.

Examples

The following example selects all the net components (2 wires, 2 labels) associated with the figure under point {1 2}.

```
set tr [de::startTransaction tr1 -design $oaDesign]
set w [se::createWire {{1 1} {2 1}} -design $oaDesign]
de::endTransaction $tr
se::createWireName b -wire $w -origin {1.25 1}

set tr [de::startTransaction tr2 -design $oaDesign]
set w [se::createWire {{1 2} {2 2}} -design $oaDesign]
de::endTransaction $tr
se::createWireName b -wire $w -origin {1.25 2}

de::select [se::getNetComponents [de::getActiveFigure -point {1 2} -index
0] -of net]
```

se::getOverlayObjects

Description

Gets the list of overlay objects found in the design.

Syntax

```
se::getOverlayObjects <design: oaDesign> [-type <seOverlayObjectType>]
```

Arguments

design

The design to search for overlay objects.

-type

The types of objects to be returned. (Default: all)

Returns

<Fig | Instance> : The overlay objects found in the design.

se::getPinWiringPoint

Description

Returns the preferred wiring point for the specified pin object.

Syntax

```
se::getPinWiringPoint <pin: deFigure | oaPin | oaPinFig>
```

Arguments

pin

A pin, a schematic pin, or an instance pin.

Errors

- The specified deFigure/oaPinFig is not associated with an oaPin.
- The specified deFigure is greater than 1x deep in the hierarchy.
- The specified pin does not have any associated shapes.

Returns

<point> : The preferred wiring point for the given pin.

Examples

```
set design [ed]
# Print terminal name and wiring point for pins in the active design
db::foreach pin [db::getPins -of $design] {
    set termName [db::getAttr pin.term.name]
    set point [se::getPinWiringPoint $pin]
    puts "Term: $termName Point: $point"
}

# Print terminal name and wiring point for all instances in the active
design
db::foreach instPinFig [de::getFigures -design $design -objectTypes
[de::getObjectFilters seSchInstancePin]] {
    set instName [db::getAttr name -of [lindex [db::getAttr
instPinFig.lineage.levels] 0 0]]
    set termName [db::getAttr instPinFig.object.pin.term.name]
```

```
    set point [se::getPinWiringPoint $instPinFig]
    puts "Inst: $instName Term: $termName Point: $point"
}
```

se::getReferenceCellView

Description

Returns the reference cellview for the given overlay design.

Syntax

```
se::getReferenceCellView <design: oaDesign>
```

Arguments

design

The overlay design.

Returns

<dmCellView> : The reference cellview for the given overlay design

se::getRules

Description

Returns a collection of matching rules currently registered. The -severity and -filter arguments can be used to filter the final collection. If the -severity argument is given, the -scope argument is required.

Syntax

```
se::getRules [<string: string>] [-severity <enum>] [-scope <dbScope |  
deContext | dmCell | dmCellView | oaLib | string>] [-filter <string>]
```

Arguments

string

Style pattern for matching rule name.

Default:.

-severity

Only return those rules whose current severity setting matches.

Default: any

`-scope`

Scope used for retrieving current severity preference. Required if `-severity` is given.

`-filter`

Standard attribute filter.

Default: Match all

Returns

Collection of rules matching the name and filter expression. (`{<seRule>}`)

Errors

`-severity` is given without `-scope`.

Examples

The following example prints all rules with severity level of *error* for a given cell view.

```
set rules [se::getRules -severity error -scope [dm::findCellView  
schematic -cellName inv -libName testLib]]  
db::foreach r $rules {  
  puts [db::getattr name -of $r]  
}
```

se::getShapeStyle

Description

Gets the current color, line style, and fill for the given shape. If the shape does not have a style override, nothing is returned.

Syntax

```
se::getShapeStyle <name: oaShape> -attribute <enum>
```

Arguments

`name`

Shape object.

`-attribute`

Name of the style attribute to retrieve.

Errors

- Invalid shape.
- Design not of view type schematic or schematicSymbol.
- Invalid style attribute enumeration.

Returns

Name of the given style attribute, if the shape has a style override, else returns nothing (<string>)

Examples

```
set r [se::createRectangle {{1 1} {2 2}} -design $oaDesign -color red]
se::getShapeStyle $r -attribute color
se::getShapeStyle $r -attribute lineStyle
se::getShapeStyle $r -attribute stipple
```

This example gets the color, line style, and stipple from the given shape.

se::getSheets

Description

Returns all the sheets collected from the multisheet design.

Syntax

```
se::getSheets -design <oaDesign> [-filter <string>]
```

Arguments

-design

The design to get the sheets from.

-filter

The standard attribute filter, to filter out objects based on attribute values.

Returns

<dbCollection> : A list of collected objects. This is empty for a non-multisheet design.

Examples

```
set design [ed]
se::addSheets -design $design -numStart 1 -numEnd 3 -border
sheets/ASheet/symbol
db::foreach s [se::getSheets -design $design] {
puts [db::getattr s.number]
}
```

se::getTables

Description

Returns the schematic table objects of the design.

Syntax

```
se::getTables -design <oaDesign> [-filter <string>]
```

Arguments

-design

Design from which table objects are retrieved.

-filter

Standard attribute filter.

Returns

<dbCollection> : Collection of table objects.

Examples

```
set design [ed]
se::createTable -design $design -origin {1 1} -rows 5 -columns 3 -title
  "Truth Table" \
-cellData {{in1 in2 out} {0 0 0} {0 1 0} {1 0 0} {1 1 1}} -height 0.2
-width 0.2 \
-rotateFirstRowText true -name " " -textColor orange -autoSize false
db::foreach t [se::getTables -design $design] {
db::setAttr t.rows -value 10
db::setAttr t.columns -value 10
}
```

se::importConfig

Imports VDRC configuration for VDRC assistant.

Syntax

```
se::importConfig -window <giWindow | unsigned int> -viewName <string>
```

Returns

<none>

Arguments

`-window`

Specify the window to load the configuration.

`-viewName`

Specify the config view name. The lib and cell name of the imported config view will be the same as the design by the given window.

se::importSymbol

Description

Imports shapes, pins, labels, and the selection region from the specified source cellview into the given design. Specify a point in the destination design to coincide with the origin in the source cellview. Use the `-exclude` option to exclude certain classes of objects from the import process.

Syntax

```
se::importSymbol -design <oaDesign> -src <dmCellView | dmCellViewName>  
-point <point> [-exclude [<string>]]
```

Arguments

`-design`

Design to contain the imported objects.

`-src`

Source cellview with view type of schematicSymbol to import.

`-point`

Coordinate in the destination design which coincides with the origin of the copied objects.

`-exclude`

List of enums indicating the object types to exclude from copying.

Examples

The following example imports the shapes and labels from the source design (analogLib/nmos4/symbol) into the destination design.

```
se::importSymbol -design $destDesign -src [dm::findCellView symbol  
-cellName nmos4 -libName analogLib] -point {1 1} -exclude {pins  
selection}
```

se::isDesignNetlistable

Description

Determines whether the given design or dmCellView is ready for netlisting.

Syntax

```
se::isDesignNetlistable <design: dmCellView | oaDesign> [-open <bool>]
```

Arguments

-design

dmCellView or oaDesign to analyze.

-open

If is true, the oaDesign is opened for specified dmCellView.

Returns

<bool>.

se::isNetlistable

Description

This command checks whether the given design or the cell view is ready for netlisting. Specifically, checks whether the connectivity extraction and the ERC rules are up to date, and also issues a warning message if the instance headers are out dated. If a cell view is specified without having the OA design open, this command opens it and closes at the end of command.

Syntax

```
se::isNetlistable <design: dmCellView | dmCellViewName | oaDesign>
```

Arguments

design

Design to process.

Returns

<bool>: Indicates whether the design is ready for netlisting.

Examples

The following example checks if the schematic design is and ready to be netlisted. It needs to be checked up to date before.

```
# Get edit design from the current context
set des [db::getattr editDesign -of [de::getActiveContext]]
# Check if the design is checked and ready to generate the netlist
set isNet [se::isNetlistable $des]
```

se::isOverlay

Description

Checks to see if the specified object is an overlay cellview or overlay object.

Syntax

```
se::isOverlay <object: deFigure | dmCellView | dmCellViewName | oaDesign  
| oaFig>
```

Arguments

object

The cellview or object to check.

Returns

<bool> : True if the object is an overlay cellview/object, false if not.

Examples

The following example checks if the selected object in the **Schematic Editor** Window is overlay object.

```
# Get edit design from the current context
set des [db::getattr editDesign -of [de::getActiveContext]]
# Get OA object of the selected wire in the design
set wireShape [de::getSelected -design $des]
# Check the object
set check [se::isOverlay $obj]
puts "Check if the selected wire is overlay object: $check"
```

se::loadNetColorizationDefinitions

Description

Loads the net colorization definitions from the given file.

Syntax

```
se::loadNetColorizationDefinitions -fileName <string>
```

Arguments

`-fileName`

Specifies the file from which to load the net colorization definitions.

Returns

<dbliterator> :

Examples

```
# Create Net Colorization Definition
se::createNetColorizationDefinition clock -description Clock -color red
  -lineStyle longDash
se::createNetColorizationDefinition ground -description Ground -color
  blue -lineStyle thickLine
se::saveNetColorizationDefinitions -fileName colorsdef.tcl
se::loadNetColorizationDefinitions -fileName colorsdef.tcl
```

se::move

Description

Moves a given set of objects by the supplied X and Y distance. The `-dx` and `-dy` arguments are optional such that this command can be given just a transform and anchor point to produce a pure rotation or transformation function. If a transform is supplied that is not R0, the `-anchor` setting controls at where in the design the given objects are transformed. The `-anchor` option can accept either a <point> or the string center. If center is given, the snapped center point of the bounding box of the given objects is used.

Syntax

```
se::move <objects: {<deFigure>} | {<oaFig>} | {<oaFigGroup>} | {<oaInst>}
  | {<oaShape>}> [-dx <float>] [-dy <float>] [-rotate <enum>] [-anchor
  <point>] [-dest <oaDesign>]
```

Arguments

`objects`

Collection of objects to be moved, all objects must be from the same `oaDesign`.

`-dx`

Distance to move the given objects in the X direction in user units.

Default: 0

`-dy`

Distance to move the given objects in the Y direction in user units.

Default: 0

`-rotate`

Transform to apply to objects (`{MX | MXR90 | MY | MYR90 | R0 | R180 | R270 | R90}`).

Default: R0

`-anchor`

Coordinate of anchor for the transform or the string center.

Default: snapped center of extents

`-dest`

Design where objects will be pasted.

Returns

`{<deFigure>} | {<oaFig>}`: Collection of the moved objects.

Errors

- Invalid design, design not opened for write.
- Invalid command line argument.

Examples

The following example cuts and pastes the selected objects from the source design into the destination design.

```
set objects [de::getSelected -design $srcDesign]
se::move $objects -dx 1 -rotate R90 -destination $destDesign -anchor {4
3}
```

se::moveSheetToPosition

Description

Changes the position of the specified sheet.

The command moves the specified sheet to a new position. All the other sheets in the range from the `<selectedSheet + 1>` to `<destPosition>` are shifted one position down if the sheet's position is less than the destination position, or one position up otherwise.

Syntax

```
se::moveSheetToPosition <none: dbSheet | int> -newPos <int> -design  
    <oaDesign>
```

Arguments

`none`

The sheet to move.

`-newPos`

The new position of a sheet.

`-design`

The multisheet design to process.

Returns

`<none>`

Errors

- Unable to open the cellview.
- The specified cellview is not a multisheet.
- The specified sheet is not found in the cellview.
- The specified `newPos` is out of range of available sheets.

Examples

```
set design [ed]  
# Get reference on sheet object from active Schematic Editor window  
set sheet [se::getActiveSheet]  
# Move to new position  
se::moveSheetToPosition $sheet -newPos 2 -design $design
```

se::populateDesignSummary

Description

Processes the design hierarchy and populates the design data information.

Syntax

```
se::populateDesignSummary -context <heContext> [-levels <int>]  
    [-excludedLibraries <string>] [-mappingFile <string>] [-dbHandle  
    <string>]
```


Arguments

`-context`

The design context.

`-levels`

Indicates the count of hierarchy levels which is processed. (Default is: 0)

`-excludedLibraries`

List of libraries the cells from which are ignored together with their lower hierarchy. (Default is:)

`-mappingFile`

The mapping file to refer for equations. (Default is:)

`-dbHandle`

The database handle to populate data. If not set, the output is TCL list. (Default is:)

Returns

`<string>` : Returns list of data rows if DB handle is not provided.

Errors

- Invalid context object.
- Invalid DB handle.
- Invalid mapping file path.
- Invalid mapping format.

Examples

The following example populates the design summary information in the sqlite DB handle, and then creates a DBtable to show design summary information.

```
sqlite myDb :memory:
set context [de::createContext PLL/buff/schematic ]
# Create Design Summary table
se::populateDesignSummary \
    -context [db::getattr context.hierarchy] \
    -excludedLibraries {sheets basic} -dbHandle myDb
#Show DB Table
set dialog [gi::createDialog exDbTable \
    -title "Design Summary" -showHelp false]
set table [gi::createDbTable designSummary \
    -parent $dialog \
    -dbHandle myDb \
```

```
-dbTableName summary \  
-allowMoveColumns true]
```

se::propagatePortOrder

Description

This command propagates the string or the portOrder property value of the source design to the portOrder property of the destination design.

Syntax

```
se::propagatePortOrder (-value <string> | -source <oaDesign>)  
-destination <oaDesign>
```

Arguments

-destination

Design which portOrder property should be set.

-value

Port order value.

-source

Source design the port order value of which should be propagated to given design.

Examples

```
set design [ed]  
# Create terminals  
se::createPin in1 -design $design -points {{1 1}} -type input -shape  
custom  
se::createPin in2 -design $design -points {{1 2}} -type input -shape  
custom  
se::createPin in3 -design $design -points {{1 3}} -type input -shape  
custom  
se::createPin clk -design $design -points {{1 4}} -type input -shape  
custom  
se::createPin reset -design $design -points {{1 5}} -type input -shape  
custom  
se::createPin q -design $design -points {{2 5}} -type output -shape  
custom  
se::createPin q_n -design $design -points {{2 4}} -type output -shape  
custom  
  
# Sort terminal names  
set termNames [lsort -dictionary [db::createList [db::getAttr name -of  
[db::getTerms -of $design]]]]  
se::propagatePortOrder -value $termNames -destination $design
```

se::registerLabelFunction

Description

Registers the given function into the createLabel function list for the specified evaluator. The positional argument is the function name, which is a short description for the function expression. The same function name can be given for different evaluators. However, only one function name is valid for a single evaluator. If the given function name is already registered for an evaluator, the previous definition is overwritten. The expression string can contain a replacement string.

Syntax

```
se::registerLabelFunction <name: string> -evalType <enum> -expr  
<string> -layer <int> -purpose <int> [-position <int>] [-nameEvalProc  
<dbExecObject | string>] [-valueEvalProc <dbExecObject | string>]  
[-commitProc <dbExecObject | string>] [-validateProc <dbExecObject |  
string>]
```

Arguments

name

Function name.

-evalType

Evaluation type to use for the given expression.

-expr

Expression text to use when creating a label with `ise::createLabel`.

-layer

Layer number for creating the oaEvalText object.

-purpose

Purpose number for creating the oaEvalText object.

-position

Position of the newly registered label function.

-nameEvalProc

Name of the procedure which evaluates the name part of the eval text.

-valueEvalProc

Name of the procedure which evaluates the value part of the eval text.

`-commitProc`

Name of the procedure responsible for committing the values during on-canvas editing.

`-validateProc`

Name of the procedure which should validate the values that are edited by on-canvas editing.

Examples

In this example, a new LCV() evaluation label is defined. When creating the instance, the label will show the placed master Library, Cell, and View names on the instance.

```
proc LCV {} {  
    return "[LCVname] = [LCVvalue]"  
}  
  
proc LCVname {} {  
    return "L/C/V"  
}  
  
proc LCVvalue {} {  
    set lib [db::getAttr libName -of [db::getCurrentRef]]  
    set cell [db::getAttr cellName -of [db::getCurrentRef]]  
    set view [db::getAttr viewName -of [db::getCurrentRef]]  
    return "$lib/$cell/$view"  
}  
  
se::registerLabelFunction lcv -evalType IL -expr LCV() -layer 231  
    -purpose 237 -nameEvalProc LCVname -valueEvalProc LCVvalue
```

se::registerParasiticTopology

Description

Registers a defined custom parasitic topology.

Syntax

```
se::registerParasiticTopology <none: string> -cellview <dmCellView>  
    [-fromLib <bool>]
```

Arguments

`none`

Name.

`-cellview`

Cellview.

-fromLib

Device is from the library specified by

seEstimatedParasiticsCustomDeviceLibName. (Default is: false)

Returns

<none>

Examples

Before you can register a custom device in the Estimated Parasitics assistant, you must create the device and make sure the library is defined from the `lib.defs` file.

This example is for a device named `my_r_metal` from a library named `reference40nm`.

```
se::registerParasiticTopology MyRes -cellview [dm::getCellViews symbol  
-cellName my_r_metal -libName reference40nm]
```

If you would like this device to show up in the Estimated Parasitics assistant permanently, place the above command in the `$SYNOPSYS_CUSTOM_SITE/estimated_parasitics/startup.tcl` file.

To add a preview of the custom device in the Estimated Parasitics assistant, place the image file of the preview in the images file (for example, `$SYNOPSYS_CUSTOM_SITE/images/myres.png`). The image file name must be lower-case and fit the display.



se::registerPinShape

Description

Registers a new pin shape for use when creating pins in a symbol. The given prefix is appended with Input, Output, InputOutput, Switch, or Tristate to define the preferences to be used with this shape type for each pin type, respectively.

If the shape type name has already been registered, the previous shape type is overwritten.

Syntax

```
se::registerPinShape <none: string> -prefix <string>
```

Arguments

`none`

Name of pin shape.

`-prefix`

String used as the prefix for associated preference names.

Returns

`<none>`

Errors

Preference with name already exists.

Shape type name must not be square, circle, triangle, or polygon.

Examples

The following example shows how to define a new pin shape for the Symbol Editor.

```
se::registerPinShape myShape -prefix mySymbol
db::setPrefValue mySymbolInput -value {basic ipin symbolrOff}
db::setPrefValue mySymbolOutput -value {basic opin symbollOff}
db::setPrefValue mySymbolInputOutput -value {basic iopin symbollOff}
```

se::renameNet

Description

Renames the specified net according to rules.

Syntax

```
se::renameNet <from: oaNet | string> -to <string> [-design <oaDesign>]
```

Arguments

`from`

Source net or net name to rename.

`-to`

Target name to apply.

`-design`

Design to be modified.

Returns

<none>

Examples

The following example renames net net21 as net Q in the current edit design.

```
se::renameNet net21 -design [ed] -to Q
```

se::resizeSheets

Description

Resizes the perimeter of the border of the specified sheets, and also updates the placement of the sheets.

Syntax

```
se::resizeSheets <none: dbCollection> -newSize <dmCellView |  
dmCellViewName> -design <oaDesign>
```

Arguments

none

The collection of sheets to resize.

-newSize

The new border to use.

-design

The multisheet design to process.

Returns

<none>

Examples

```
set design [ed]  
se::addSheets -design $design -numStart 1 -numEnd 3 -border  
sheets/ASheet/symbol  
db::foreach s [se::getSheets -design $design] {  
se::resizeSheets $s -newSize sheets/BSheet/symbol -design $design  
}
```

se::saveNetColorizationDefinitions

Description

Saves the existing net colorization definitions to the given file.

Syntax

```
se::saveNetColorizationDefinitions -fileName <string> [-overwrite <bool>]
```

Arguments

`-fileName`

Specifies the file from which to load the net colorization definitions.

`-overwrite`

Overwrites the existing file. The default is false.

Returns

<dbliterator> :

Examples

```
# Create Net Colorization Definition
se::createNetColorizationDefinition clock -description Clock -color red
  -lineStyle longDash
se::createNetColorizationDefinition ground -description Ground -color
  blue -lineStyle thickLine
se::saveNetColorizationDefinitions -fileName colorsdef.tcl
```

se::schematicMigration

Migrate schematic designs by giving mapping file.

Syntax

```
se::schematicMigration -dataMode <enMigDataMode> -mapFile <string>
  [-techLib <string>] [-libraries <string>] [-cells <string>]
  [-overwriteLib <bool>] [-overwriteCell <bool>] [-librarySuffix <string>]
  [-cellSuffix <string>] [-copyHierarchy <bool>] [-copyHierExcludeLib
  <string>] [-copyHierIncludeViews <string>] [-copyHierSeparateLibs
  <bool>] [-copyHierSeprateLibSuffix <string>] [-keepSourceSymbols
  <bool>] [-keepUnmappedCells <bool>] [-localSourceLibs <string>]
  [-localLibDestMode <enMigLocalSymbolMode>] [-localLibName <string>]
  [-localLibSuffix <string>] [-localLibOverwrite <string>] [-preInstProc
  <string>] [-postInstProc <string>]
```

Returns

<none>

Arguments

`-dataMode`

Data Mode of Schematic Migration, value is `enTireLibrary` or `multipleCells`

`-mapFile`

File path of the migration mapping file.

`-techLib`

The reference technology library of target library. (Default is:)

`-libraries`

Design Library names to migrate.

`-cells`

Multiple cell names to migrate. (Default is:)

`-overwriteLib`

Overwrites the library. (Default is: 0)

`-overwriteCell`

Overwrites the cells. (Default is: 0)

`-librarySuffix`

Suffix of the design library. (Default is: `_new`)

`-cellSuffix`

Suffix of multiple cells. (Default is: `_new`)

`-copyHierarchy`

Copy hierarchy when copying the multiple cells. (Default is: 0)

`-copyHierExcludeLib`

The exclude libraries when copy hierarchy. (Default is: `analogLib sample basic sheets`).

`-copyHierIncludeViews`

The include views when copy hierachy (Default is: `*`)

`-copyHierSeparateLibs`

Copy hierarchy to separate libraries with Suffix (Default is: 0)

`-copyHierSeprateLibSuffix`

The suffix of separate libraries by copy hierarchy. (Default is: `_local`)

`-keepSourceSymbols`

Keep the source symbols in localized library (Default is: 0)

`-keepUnmappedCells`

Keep the unmapped cells in localized library (Default is: 0)

`-localSourceLibs`

The source library lames to localized (Default is:)

`-localLibDestMode`

The mode of localized library destination, value is `single` or `multiple`. (Default is: `single`)

`-localLibName`

The name of localized library. (Default is:)

`-localLibSuffix`

The suffix of localized library. (Default is:)

`-localLibOverwrite`

To overwrite the localized libraries. (Default is:)

`-preInstProc`

The procedure triggered before updating an instance. Procedure signature:
`userProc <oaInst>` (Default is:)

`-postInstProc`

The procedure triggered after updating an instance. Procedure signature:
`userProc <oaInst>` (Default is:)

se::setConnectivityValid

Description

Sets the design's connectivity status to valid.

In some circumstances, various cellviews might be modified and cause problems when performing operations that require all cellviews to share the same timestamp. For example, an extraction view might be modified and result in a netlisting failure when the netlisting operation checks whether the connectivity is valid and up-to-date.

The `se::setConnectivityValid` Tcl command helps to force connectivity checking as valid. Use this command carefully because, while it does analyze the design and extract connectivity, it does not actually change anything but the connectivity status.

Syntax

```
se::setConnectivityValid <design: oaDesign>
```

Arguments

`design`

The design to be set to a valid connectivity state.

Returns

`<bool>` : Boolean indicating whether or not connectivity has been set to valid.

se::setShapeStyle

Description

Sets or resets the color, line style, and fill for the given shape. To maintain or create style overrides, specify a style attribute, else the Schematic Editor removes any existing style overrides and renders the object using the default style for the layer purpose pair.

Syntax

```
se::setShapeStyle <name: oaShape> [-stipple <string>] [-lineStyle  
  <string>] [-color <string>]
```

Arguments

`name`

An oaShape object to set color, line style, or fill.

`-stipple`

Name of an existing stipple pattern to apply as a style override.

Default: stipple used in display packet

`-lineStyle`

Name of an existing line style pattern to apply as a style override.

Default: lineStyle used in display packet

`-color`

Name of an existing color to apply as a style override.

Default: color used in display packet

Examples

This example changes the shape style.

```
set r [se::createRectangle {{1 1} {2 2}} -design $oaDesign -color red]  
se::setShapeStyle $r -color blue
```

se::setSheetColumns

Description

Sets the number of sheet columns of multisheet cellview.

Syntax

```
se::setSheetColumns <none: int> -design <oaDesign>
```

Arguments

none

The number of columns to be set to multisheet cellview.

-design

The design to process.

Returns

<none>

Examples

```
set design [ed]  
se::addSheets -design $design -numStart 1 -numEnd 5 -border  
  sheets/ASheet/symbol  
se::setSheetColumns 3 -design $design
```

se::showAnnotateOP

Description

Shows the **Simulation Back Annotation** dialog box.

Syntax

```
se::showAnnotateOP -window <giWindow | int>
```

Arguments

-window

The schematic to back-annotate.

Returns

<giDialog> : The back-annotation dialog object.

se::showBillOfMaterials

Description

Shows the **Bill Of Materials** dialog box for the design.

Syntax

```
se::showBillOfMaterials [-parent <giWindow>]
```

Arguments

`-parent`

Parent window.

Returns

`<giDialog>` : The opened dialog box.

se::showCheckOptions

Description

Displays the main ERC control dialog box. The dialog box allows editing of preference values that control the behavior of the `ise::check` command.

Syntax

```
se::showCheckOptions [-parent <giWindow | int>]
```

Arguments

`-parent`

Optional window to associate with dialog specifically to supply scope for the preference values.

Default: Active window

Returns

Dialog window (*giDialog*)

Errors

Window does not exist.

se::showCompareDesigns

Description

Opens the **Compare Designs** window for designs logical compare.

Syntax

```
se::showCompareDesigns [-context <deContext> | -cv1 <dmCellView |  
    dmCellViewName>] [-cv2 <dmCellView | dmCellViewName>]
```

Arguments

-context

Design context to define current (left-side) design.

-cv1

First (left-side) cell view to compare.

-cv2

Second (right-side) cell view to compare with.

Returns

<giWindow> : The created window.

se::showCreateTemplateFromSelected

Description

Brings up the **Create Template From Figures** dialog box.

Syntax

```
se::showCreateTemplateFromSelected [-window <giWindow | int>]
```

Arguments

-window

The parent window.

Returns

<giDialog>: The created dialog box.

se::showEditSheets

Description

Displays the **MultiSheet Cellview** dialog box.

Syntax

```
se::showEditSheets [-window <giWindow | int>]
```

Arguments

-window

Parent window.

Returns

<giDialog> : The created dialog box.

se::showEditTable

Description

Opens the schematic table editing dialog box.

Syntax

```
se::showEditTable [-window <giWindow | int>] [-table <seTable | string>]
```

Arguments

-window

Parent window.

-table

Table object.

Returns

<giDialog> : The created dialog

Errors

- Invalid window Id.
- Window is not schematic or schematicSymbol editor.
- No table with the given name is found.

se::showLibraryCheck

Description

Displays a library wide check dialog box.

Syntax

```
se::showLibraryCheck [-parent <giWindow | int>]
```

Arguments

`-parent`

Optional window to associate with dialog specifically to supply scope for the preference values.

Returns

Dialog window (*giDialog*).

se::showNetColorizationDefinitionDialog

Description

Opens the **Net Colorization Definitions** dialog box.

Syntax

```
se::showNetColorizationDefinitionDialog
```

Returns

<giDialog>

se::showPlotSignalSettings

Description

Shows the settings dialog for plotting from a schematic.

Syntax

```
se::showPlotSignalSettings -parent <giWindow | int>
```

Arguments

`-parent`

Parent window.

Returns

<giDialog> : The dialog for configuring plot settings.

se::showPowerDomainAnalyzer

Description

Opens the **Power Domain Analyzer** dialog box for the given design context.

Syntax

```
se::showPowerDomainAnalyzer <context: deContext>
```

Arguments

context

Design context for which the Power Domain Analyzer is shown.

Returns

<giDialog> : The opened dialog box.

se::showSchDesignOptions

Description

Shows the **Schematic Design Options** dialog box. If the dialog box is already opened for the given window, it is raised and the existing object returned. Otherwise, the existing dialog box is closed before reopening for this design.

Syntax

```
se::showSchDesignOptions [-parent <giWindow | int>]
```

Arguments

-parent *giWindow* | <int>

Parent window and window containing edit design for which preferences will be edited.

Default: Last focused schematic window

Returns

Newly opened or already opened dialog box (*giDialog*)

Errors

- Invalid window.
- No active editor window.

se::showSchematicMigration

Shows the Schematic Migration dialog box.

Syntax

```
se::showSchematicMigration
```

Returns

<giDialog>: Schematic Migration dialog box

se::showSymDesignOptions

Description

Shows the **Symbol Design Options** dialog box. If the dialog box is already open for the given window, it is raised and the existing object returned. Otherwise, the existing dialog box is closed before reopening for this design.

Syntax

```
se::showSymDesignOptions [-parent <giWindow | int>]
```

Arguments

-parent [giWindow](#) | <int>

Parent window and window containing edit design for which preferences will be edited.

Default: Last focused symbol window

Returns

Newly opened or already opened dialog box (*giDialog*)

Errors

- Invalid window.
- No active editor window.

se::showSynchronizePortOrder

Description

Opens the port order synchronizing dialog box.

Syntax

```
se::showSynchronizePortOrder [<none: dmCellView | dmCellViewName>]  
[-parent <giWindow | int>]
```

Arguments

none

Source cellview.

-parent

Parent window.

Returns

<giDialog>: Schematic Generation dialog box.

se::stretch

Description

Stretches the set of objects by the supplied X and Y distance while retaining wiring. If a transform is supplied that is not R0, the `-anchor` setting controls where in the design the given objects are transformed. The `-anchor` argument can accept either a <point> or the string center. If center is given, the snapped center point of the bounding box of the given objects is used.

Syntax

```
se::stretch <objects: {<deFigure>} | {<oaFig>}> [-dx <float>] [-dy  
<float>] [-rotate <enum>] [-anchor <point>] [-avoidSymbols <bool>]  
[-avoidShorts <bool>]
```

Arguments

objects

Collection of objects to be stretched; all objects must be from the same oaDesign.

-dx

Distance to move the given objects in the X direction in user units.

Default: 0

`-dy`

Distance to move the given objects in the Y direction in user units.

Default: 0

`-rotate`

Transform to apply to objects.

Default: R0

`-anchor`

Coordinate of anchor for the transform.

Default: snapped center of extents

`-avoidSymbols`

Specifies whether to avoid symbols when creating the route.

Default: true

`-avoidShorts`

Specifies whether to avoid creating name shorts when determining the route.

Default: true

Returns

{<deFigure>} | {<oaFig>}: Collection of stretched objects.

Errors

- Invalid design, design not opened for write.
- Invalid command line argument.

Examples

```
set design [ed]
set inst [se::createInst -libName analogLib -cellName nmos4 -viewName
symbol -origin {2 1} -design $design]
se::createWire {{2 1} {0 1}} -design $design
se::stretch $inst -dx 1 -dy 1 -rotate R90
```

se::syncOverlayDesign

Description

Synchronizes the overlay design with its reference cellview.

Syntax

```
se::syncOverlayDesign [-resolveConflicts <enum>] [-force <bool>] [-window  
  <giWindow | int>]
```

Arguments

`-resolveConflicts`

Specifies how the conflicting cases should be handled.
(`{ acceptOverlayChanges | acceptReferenceChanges | autoFix }`, default is:
`autoFix`)

`-force`

Force the sync without prompts. (Default is: false)

`-window`

The editor window which the edit cellview need to be synchronized.

se::unregisterLabelFunction

Description

Removes the given function from the createLabel function list for the specified evaluator.

Syntax

```
se::unregisterLabelFunction <name: string> -evalType <enum>
```

Arguments

`name`

Function name that appears in the createLabel function combobox.

`-evalType`

Evaluation type.

Examples

The following example shows how to unregister a label function called *Pin annotation*.

```
se::unregisterLabelFunction "Pin annotation" -evalType IL
```

se::updatePinLabels

Description

Updates pins' annotation labels in a symbol design to have terminal names matched.

Syntax

```
se::updatePinLabels -design <oaDesign>
```

Arguments

`-design`

The design to process for labels fix.

Returns

`<bool>` : Boolean indicating if any change was done.

ise::align

Description

Starts the interactive alignment command. Performs alignment of passed target objects to a position of the source one.

Syntax

```
ise::align [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

`<none>`

ise::alignToColumn

Description

Aligns the selected instances to the left of a selected frame column.

Syntax

```
ise::alignToColumn [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ise::attachText

Description

Interactive command for text attaching. Detaches a text object from an existing parent and attaches it to a shape object of the same semantic type.

Syntax

```
ise::attachText [<context: deContext | unsigned int>] [-text <oaText>]
```

Arguments

context

Context of activation.

-text

Text object.

Returns

<none>

ise::check

Description

Run ERC/SRC checks on the design associated with given Window ID.

Syntax

```
ise::check [<none: giWindow | int>] [-hierarchy <bool>]
```

Arguments

none

Window object or window Id.

Default: active window.

-hierarchy

Indicates if entire hierarchy should be checked, in addition to the design associated with the given window.

Default: false.

Returns

<none>

ise::chopWires

Description

Interactive command for chapping wires in the specified area.

Syntax

```
ise::chopWires [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::colorizeNets

Description

Colorizes a net by click, selection, or name pattern.

Syntax

```
ise::colorizeNets [<context: deContext | unsigned int>]
```

Arguments

context

The context in which the command is activated.

Returns

<none>

ise::connectWires

Description

Interactively connects crossing wires.

Syntax

```
ise::connectWires [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::copy

Description

Copies the objects in the active selected set of the design associated with the specified window.

If the -point argument is given, the point will be used as the anchor point for the operation. In addition, if the -point argument is given, the Schematic Editor toolbar will not be displayed and the command terminates when the next addPoint or dragEnd event occurs. If no objects are selected, or become selected due to the anchor point, the command terminates immediately without error.

Syntax

```
ise::copy [<context: deContext | unsigned int>] [-object <deFigure |  
oaFig>] [-point <point>]
```

Arguments

context

Context of activation.

-object

Object for exclusive copy.

-point

Specifies the drag start point and enters one-shot mode, where the Schematic Editor toolbar is not updated.

Returns

<none>

ise::copyToClipboard

Description

Interactive schematic copy selected objects to clipboard.

Syntax

```
ise::copyToClipboard [<context: deContext | unsigned int>] [-log <enum>]
```

Arguments

context

Context of activation.

-log

Logging mode. ({ auto | explicit }, default is: auto)

Returns

<none>

ise::createBlock

Description

Allows you to create blocks interactively in the design. You can specify a cell view for the created block and location for the existing block or an origin in the existing design.

Syntax

```
ise::createBlock [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::createColumn

Description

Interactive command for column creation.

Syntax

```
ise::createColumn [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::createConnectDef

Description

Adds a connection definition label to the given pin or wiring object and associates it with the pin or wiring object.

Syntax

```
ise::createConnectDef [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::createConnector

Description

Creates an off-sheet connector.

Syntax

```
ise::createConnector [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::createConstraintNote

Description

Interactively creates schematic notes formatted for Laker constraint extraction.

Syntax

```
ise::createConstraintNote [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::createInst

Description

Interactively creates an instance.

Syntax

```
ise::createInst [<context: deContext | unsigned int>] [-device  
<deDevice>] [-hidden <bool>] [-singleShot <bool>] [-instNames <string>]  
[-deleteShorts <bool>]
```

Arguments

context

Context of activation.

-device

Populates the **Create Instance** dialog box with the master and parameters from the specified favorite device object.

-hidden

If true, starts the command in a hidden state without affecting the command options toolbar or showing the command options dialog box.

Default is: false

`-singleShot`

If true, the command is terminated after adding one instance.

Default is: false

`-instNames`

Instance names for the instances to be created.

`-deleteShorts`

If true, after adding an instance any net segments shorting the instance terminals are deleted.

Default is: false.

Returns

<none>

ise::createLabel

Description

Interactively adds a label to a schematic or symbol design.

Syntax

```
ise::createLabel [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::createMask

Description

Interactively creates a mask.

Syntax

```
ise::createMask [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ise::createNote

Description

Interactively adds a note to a schematic or symbol design.

Syntax

```
ise::createNote [<context: deContext | unsigned int>] [-shape <enum>]
```

Arguments

`context`

Context of activation.

`shape`

Shape of drawing object. ({arc | circle | ellipse | line | polygon | rectangle})

Returns

<none>

ise::createOverlayInstanceWithMask

Description

Interactive command for masked instance creation.

Syntax

```
ise::createOverlayInstanceWithMask [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ise::createRuler

Description

Starts the interactive **Create Ruler** command in the specified symbol or the schematic window.

Syntax

```
ise::createRuler [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::createSchematicPin

Description

Interactively adds pins to a schematic design.

Syntax

```
ise::createSchematicPin [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::createSelectionShape

Description

Creates a rectangle or polygon that represents both the selection region and the avoidance region for the design.

Syntax

```
ise::createSelectionShape [<context: deContext | unsigned int>] [-shape  
    <enum>]
```

Arguments

context

Context of activation.

-shape

Shape of selection region ({ polygon | rectangle })

Returns

<none>

ise::createShape

Description

Interactively adds a shape to a schematic or symbol design. Use the `-shape` argument to define a preset shape type.

Syntax

```
ise::createShape [<context: deContext | unsigned int>] [-shape <enum>]
```

Arguments

context

Context of activation.

-shape

Shape of drawing object. ({ arc | circle | ellipse | line | polygon | rectangle })

Returns

<none>

ise::createSymbolPin

Description

Interactively adds pins to a symbol design.

Syntax

```
ise::createSymbolPin [<context: deContext | unsigned int>]
```


Arguments

`context`

Context of activation.

Returns

<none>

ise::createTable

Description

Interactively adds schematic tables.

Syntax

```
ise::createTable [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

Errors

- Invalid window Id.
- Window is not schematic or schematicSymbol editor.

ise::createTemplate

Description

Interactively creates a template.

Syntax

```
ise::createTemplate [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ise::createWire

Description

Initiates the wire creation command which allows wires to be routed between two points interactively defined by the user.

If the `-object` option is given, the object supplied, which must be a wire or pin, will be used to determine the first point from which to begin the first wiring operation. If this object is a schematic or symbol pin, the snapped center point of the pin connection bounding box will be used. If the object is a wire, the `-point` option will be used to determine the point on the wire closest to the given point. This point will be used as the starting point for the first wiring operation.

When the `-object` command-line option is supplied, the command will be a one-shot. This means that once the second point is given, the current wiring between the first and second point will be considered complete and will be committed and the command will terminate. In addition, when the `-object` command-line option is given, this command will not post its command option toolbar.

Syntax

```
ise::createWire [<context: deContext | unsigned int>] [-type <enum>]  
                [-object <deFigure>] [-point <point>]
```

Arguments

`context`

Context of activation.

`-type`

Sets `seWiringType` preference, which determines how the wire width should be calculated. ({ auto | fat | thin })

`-object`

Indicates an object from which to define the first point to begin wiring.

`-point`

Optional reference point to be used to determine point on given object to begin wiring.

Returns

<none>

ise::createWireName

Description

Interactively adds a text string and associates it with a wire. The text string represents the net name to which the wire belongs.

Syntax

```
ise::createWireName [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::createWireStubs

Description

An interactive command for adding a wire stub.

Syntax

```
ise::createWireStubs [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::cut

Description

Interactive schematic copy selected objects to clipboard.

Syntax

```
ise::cut [<context: deContext | unsigned int>] [-log <enum>]
```

Arguments

`context`

Context of activation.

`-log`

Logging mode ({ auto | explicit }, default is: auto)

Returns

<none>

ise::delete

Description

Deletes all the selected objects if the active select set for the specified window is not empty. The command then terminates. If the select set is empty, the command waits for the select set to contain 1 or more objects. When this occurs, those objects in the select set are deleted and the command again waits for the select set to contain 1 or more objects. You can terminate the command with `de::abortCommand`.

Syntax

```
ise::delete [<context: deContext | unsigned int>] [-object <deFigure |  
oaFig>]
```

Arguments

`context`

Context of activation.

`-object`

Object for exclusive processing. If given, the select set will not be used or modified.

Returns

<none>

ise::disconnectWires

Description

Interactively disconnects crossing wires.

Syntax

```
ise::disconnectWires [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::ignoreInst

Description

Interactively ignores instances.

Syntax

```
ise::ignoreInst [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::importSymbol

Description

Imports objects from a specified cellview.

Syntax

```
ise::importSymbol [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::manageLabels

Description

Manages visibility of labels.

Syntax

```
ise::manageLabels [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ise::move

Description

Moves the objects in the active selected set of the design associated with the specified window.

If the `-point` argument is given, the point is used as the anchor point for the operation. In addition, if the `-point` argument is given, the toolbar is not displayed and the command terminates when the next `addPoint` or `dragEnd` event occurs. If no objects are selected, or become selected due to the anchor point, the command terminates immediately without error.

Syntax

```
ise::move [<context: deContext | unsigned int>] [-object <oaFig  
| deFigure>] [-point <point>]
```

Arguments

`context`

Context of activation.

`-object`

Object for exclusive processing. If given, the select set is not used or modified.

`-point`

Specifies the anchor point for the operation and also indicates that the toolbar should not be displayed.

Returns

<none>

ise::paste

Description

Starts the interactive Schematic Editor command to paste data from the clipboard into the design of the given window.

Syntax

```
ise::paste [-parent <giWindow | int>] [-clip <deClipboard>]
```

Arguments

-parent

Window in which the interactive command is activated.

-clip

Clipboard to paste. The default is the most recently copied clipboard.

Returns

<none>

Errors

- Clipboard is empty.
 - Clipboard viewType is not compatible with the edit view type.
 - Window is not a schematic or symbol window.
 - An invalid window Id.
 - Active window cannot be killed.
 - Edit design is read-only.
-

ise::peek

Description

Peeks into the master design. The shortcut key T brings up the same dialog box without command activating. When the Peek dialog box is open you can use Alt+N/Alt+P to iterate through layout views.

Syntax

```
ise::peek [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::plotSignal

Description

Plots waveforms by interactively selecting nets.

Syntax

```
ise::plotSignal [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::printOP

Description

Probes instance operating point data from the schematic design.

Syntax

```
ise::printOP [<context: deContext | unsigned int>]
```

Arguments

context

The context in which the command is activated.

Returns

<none>

ise::renameInst

Description

Interactively renames instances.

Syntax

```
ise::renameInst [<context: deContext | unsigned int>]
```

Arguments

context

The context in which the command is activated.

Returns

<none>

ise::route

Description

Connects two sets of endpoints pin-to-pin or pin-to-bus.

Syntax

```
ise::route [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ise::stretch

Description

Stretches the objects in the active selected set of the design associated with the specified window.

If the `-point` argument is given, the point will be used as the anchor point for the operation. In addition, if the `-point` argument is given, the Schematic Editor toolbar will not be displayed and the command terminates when the next `addPoint` or `dragEnd` event

occurs. If no objects are selected, or become selected due to the anchor point, then the command terminates immediately without error.

Syntax

```
ise::stretch [<context: deContext | unsigned int>] [-object <oaFig  
| deFigure>] [-point <point>]
```

Arguments

`context`

Context of activation.

`-object oaFig | defigure`

Object for exclusive stretch.

`-point <point>`

Specifies the anchor point for the operation and enters one-shot mode, where the Schematic Editor toolbar is not updated.

Returns

<none>

Enumerations

Table 175 Enumerations

Name	Values	Description
enAxisMethod	full partial	Determines how drawn axes are rendered.
enDeleteWhen	never onFirstRemove onLastRemove onModify	Settings for when to delete a marker object. Corresponds to OA enums.
enDesignSummaryApplyTo	selection design	The mode to summarize all cell objects or selected objects.
enDesignSummaryDepth	0x view full user-defined	Viewing level for summary content.
enSeDesignSummaryMappedCellsFiltering	none mapped notMapped	Mapped cells filtering.
enDesignSummaryStyle	object statistics	Summary format.
enEvalType	normal IL SEL	Evaluator types.

Table 175 Enumerations (Continued)

Name	Values	Description
enFont	euroStyle gothic math roman script stick fixed swedish milSpec	Font names supported by OA. These names are consistent with the OA API.
enJog	horizontal vertical auto	Jog styles.
enJust	upperLeft centerLeft lowerLeft upperCenter centerCenter lowerCenter upperRight centerRight lowerRight	Text justification supported by OA. These names are consistent with OA API.
enNetGroup	object branch subnet net	Defines groups of a net.
enOrient	R0 R90 R180 R270 MY MYR90 MX MXR90	Transformation.
enPinShape	square polygon triangle circle custom	Shape type names for pins.
enPinType	input output inputOutput switch tristate	Pin type names supported by OA. These names are consistent with the OA API.
enSeverity	info warning error	The severity of the reported message/probe. Corresponds to OA enums.
enShape	line rectangle polygon ellipse circle arc	Shape type names.
enStyleAttr	stipple lineStyle color	Style attribute names.
enSymbolDataType	pins shapes labels selection	Types of data to be imported.
enTextType	note normal IL SEL	Evaluator names.
enTransform	R0 R90 R180 R270 MY MYR90 MX MXR90	Transforms supported by OA. These names are consistent with the OA API.
enFilterColumn	All Instance Name "CellView 1" "CellView 2"	The column to apply the filter expression.
enWireType	auto thin fat	Type of wire.

Objects

This section covers the following:

- [dbGenOptionSet](#)
- [dbInterfaceSet](#)
- [dbScope](#)
- [deFigure](#)
- [deViolation](#)
- [dmCell](#)
- [dmCellView](#)
- [giWindow](#)
- [oaInst](#)
- [oaNet](#)
- [oaPin](#)
- [oaShape](#)
- [oaTerm](#)
- [seRule](#)

dbGenOptionSet

Contains a set of values for use during cellview generation.

Table 176 *Attributes*

Name	Type	Access	Description
name	string	Read/Write	Name of a Symbol Construction Parameters (SPC) set.
setType	string	Read-only	Type of the generator option set.
value	string	Read/Write	Data representing values for use by a generator. Manipulated as a string.

dbInterfaceSet

Contains the interface information for a design for use with the cellView generation system.

Table 177 Attributes

Name	Type	Access	Description
pins	<i>{dbInterfacePin}</i>	Read/Write	Collection of interface pins.
parameters	<i>{dbInterfaceParam}</i>	Read/Write	Collection of interface parameters.
source	<i>dmCellView</i>	Read-only	Reference to source cellview if known.

dbScope

Represents a preference scope. This object may be created with the `db::createScope` command, obtained with the `db::getScopes` command. If destroyed with `db::destroy` command, then all children of this scope are destroyed too. It is not possible to destroy system-defined scopes.

Table 178 Attributes

Name	Type	Access	Description
id	<i>oaLib dmCell dmCellView deContext string</i>	Read-only	Scope identifier.
parent	<i>dbScope</i>	Read-only	Scope from which this scope inherits its values. When requesting the parent of a parentless scope such as the install scope, "" is returned.
env	string	Read-only	Environment in which this scope exists.

deFigure

Represents the figure, either whole or partial. The following table describes the object attributes. You can retrieve the object attributes with the `db::getAttr` command.

Table 179 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name attribute of the underlying OA object, else an empty string if the object doesn't have a name attribute.
lineage	<i>oaHierPath</i>	Read-only	Object's place in the design hierarchy.
objType	string	Read-only	Semantic type of the underlying object.
object	<i>oaObject</i>	Read-only	OA object wrapped in this object.
isPartial	bool	Read-only	Flag showing if this a full object or partial one.

deViolation

This object will describe a specific violation found in a design.

Table 180 *Attributes*

Name	Type	Access	Description
description	string	Read-only	Long description for this particular violation.
shortDescription	string	Read-only	Short description for this particular violation. Typically this is a short classification description.
toolName	string	Read-only	Name of tool that detected this violation.
objects	<i>{oaBlockObject}</i>	Read-only	Collection of oaBlockObjects associated with this violation. This field will be reset if the design from which these objects originate is closed.
points	[point]	Read-only	Tcl list of points describing the polygon shape for the violation region/marker. These points are in user units.

Table 180 *Attributes (Continued)*

Name	Type	Access	Description
cellview	<i>dmCellView</i>	Read-only	Design from which this violation originated.
severity	<i>enSeverity</i>	Read-only	Severity of violation.
deleteWhen	<i>enDeleteWhen</i>	Read/Write	When creating a marker from this violation, indicates when the marker object itself should be removed from the design in relation to the objects associated with the violation.

dmCell

A named container, in a library, that can hold files and dmCellViews. The dmCell corresponds to a specific oaCell object but is valid as long as the library that the dmCell belongs to remains open. The getAccess requirements of an oaCell are handled automatically by this dm object.

dmCellView

The dmCellView is a container in a library that holds files and is identified by the combination of its cell and view names. It corresponds to an oaCellView but it stays valid as long as its library remains open and its dmCell and dmView exist.

giWindow

The giWindow object represents a single window and has a container of menus, toolbars, assistants, and one status bar.

Table 181 *Attributes*

Name	Type	Access	Description
id	int	Read-only	Window identifier.
windowType	<i>giWindowType</i>	Read-only	Window type object.
title	string	Read-only	Title displayed in window border.

Table 181 Attributes (Continued)

Name	Type	Access	Description
iconTitle	string	Read/Write	Title given to the window's icon and displayed in the window manager's task list.
geometry	string	Read/Write	Window geometry.
iconified	bool	Read/Write	Whether the window is iconified.
maximized	bool	Read/Write	Whether the window is maximized. Note that this can be true even if the window is currently iconified.
statusBar	<i>giStatusBar</i>	Read-only	Status bar object holding actions that will render this window's status bar. If empty, the status bar will not be displayed in the window.

oalnst

Represents block domain instances. An instance represents the inclusion of one design as a part of the contents of another. Use the `db::getAttr` command to retrieve the oalnst attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 182 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the oalnst.
design	<i>oaDesign</i>	Read-only	Design to which the instance belongs.
master	<i>oaDesign</i>	Read-only	Master of the instance.
orient	<i>oaOrientEnum</i>	Read/Write	Orientation of the instance.
origin	point	Read/Write	Origin of the instance.
libName	string	Read-only	The library name for the master design referenced by the instance.

Table 182 Attributes (Continued)

Name	Type	Access	Description
cellName	string	Read-only	The cell name for the master design referenced by the instance.
viewName	string	Read-only	The view name for the master design referenced by the instance.
header	<i>oaInstHeader</i>	Read-only	The instHeader associated with this instance.
numBits	int	Read-only	The number of bits of this instance.
params	<i>oaParamArray</i>	Read/Write	Parameters that were specified during creation. Only PCell instances have parameters.
placementStatus	<i>oaPlacementStatusEnum</i>	Read/Write	The placement status for the instance.
source	<i>oaSourceEnum</i>	Read/Write	The source for the instance.
isImplicit	bool	Read-only	A Boolean that indicates whether the instance is implicit.

oaNet

Represents scalar and multi-bit nets that are part of the design's physical description. Nets represent the logical connectivity within a block of a design. Nets connect to terminals, which are the logical connection points on instances of the net's block. Nets also connect to instTerms that represent connections to the lower-level instances in the net's block.

Use the `db::getAttr` command to retrieve the `oaNet` attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 183 Attributes

Name	Type	Access	Description
name	string	Read-only	Name of <code>oaNet</code> .
design	<i>oaDesign</i>	Read-only	Design to which the net belongs.

Table 183 Attributes (Continued)

Name	Type	Access	Description
isEmpty	bool	Read-only	A Boolean value that indicates if the net has any objects attached to it or associated with it.
isGlobal	bool	Read/Write	A Boolean value that indicates if the net is a global.
isImplicit	bool	Read-only	A Boolean value that indicates if the net is implicit.
numBits	int	Read-only	The number of bits in the net.
sigType	<i>oaSigTypeEnum</i>	Read/Write	The signal type of the net.

oaPin

Represents the physical connections of terminals to nets. Use the `db::getAttr` command to retrieve the `oaPin` attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 184 Attributes

Name	Type	Access	Description
name	string	Read/Write	Name of <code>oaPin</code> .
design	<i>oaDesign</i>	Read-only	Design to which the net belongs.
term	<i>oaTerm</i>	Read/Write	The terminal to which this pin is attached.
accessDir	int	Read/Write	The access direction bit-field.
placementStatus	<i>oaPlacementStatusEnum</i>	Read/Write	The placement status of the pin.

oaShape

Defines specific geometry at specific locations in a design. Shapes, together with routes, specify all the manufacturable geometry in a chip layout. Shapes can also be used for more abstract purposes. The Text shape classes provide viewable labels on design objects. All shapes are placed on an `oaLayer` and an `oaPurpose`. The design API identifies

these by the layer and purpose numbers. The corresponding names are found in the technology database. Optionally, shapes can also belong to either an `oaNet` or an `oaPin` to provide part of the physical implementation of that object. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 185 Attributes

Name	Type	Access	Description
design	<i>oaDesign</i>	Read-only	The design in which the shape exists.
bBox	box	Read-only	The figure bounding box.
layerNum	int	Read/Write	The layer number associated with this shape.
leConnectivityAnchor	bool	Read-only	Indicates whether the object is a connectivity anchor.
purposeNum	int	Read/Write	The purpose number associated with this shape.
net	<i>oaNet</i>	Read-only	The net this shape implements. Returns NULL if this figure does not implement a net, NULL is returned. Note that this function will return a net if the shape is directly assigned to a net or if the shape is in a route that is assigned to a net. It will not return a net (it will return NULL) if the shape belongs to a pin.
pin	<i>oaPin</i>	Read-only	The pin this shape implements. If this figure does not implement a pin, NULL is returned.

oaTerm

Represents scalar and multi-bit terminals. Terminals represent the logical connection points for a block. The pins associated with terminals represent the physical connection points. The nets associated with the terminals are logically exported through the terminals to the next higher level in a design hierarchy. Use the `db::getAttr` command to retrieve the `oaTerm` attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 186 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the terminal.
isImplicit	bool	Read-only	A Boolean value that indicates if this terminal is implicit.
isInterface	bool	Read/Write	A Boolean indicating whether this terminal is an interface terminal.
net	<i>oaNet</i>	Read-only	The net to which this terminal is attached.
numBits	int	Read-only	The number of bits in this terminal.
termType	<i>oaTermTypeEnum</i>	Read/Write	The terminal type of this terminal.
position	int	Read/Write	The position that this terminal is assigned to.

seRule

Describes a specific violation check capable of being performed on a design.

Table 187 Attributes

Name	Type	Access	Description
name	string	Read-only	User-visible name used to identify this rule.
proc	string	Read-only	Tcl procedure that implements the rule check. If the implementation is internal, this field is an empty string.
description	string	Read-only	Complete description of the rule.
defaultSeverity	<i>enSeverity</i>	Read-only	Default severity level for this rule.
category	string	Read/Write	Optional category name for rule.

Table 187 *Attributes (Continued)*

Name	Type	Access	Description
allowDynamic	bool	Read/Write	Indicates whether this rule is allowed to be set as a dynamic rule.
forceFatal	bool	Read/Write	Indicates whether all violations found should be created as fatal oaMarkers instead of consulting the user's setting for severity.
allowDisable	bool	Read/Write	Indicates whether the user is allowed to disable the rule.
prefName	string	Read-only	Name of preference holding current default value for severity and dynamic.

Preferences

Table 188 lists the Custom Compiler se general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 188 *Schematic Editing Preferences*

Name	Type	Scope	Default	Description
sePromptOverlayUpdateInNetlisting	bool	user	true	If true, prompt when updating the out-of-date overlay design from reference in netlisting. Valid values: true, false
seAlignDirection	<enDirection>	auto	auto	Alignment line direction. Valid values: auto, vertical, horizontal
seAlignDistribute	bool	cellview	false	Defines whether alignment should also distribute objects to equal distances. Valid values: true, false
seAlignSpacing	float	cellview	0.3125	Defines the spacing to apply between objects when distributed.
seAlignSpacingMode	enDistributeSpacingMode	cellview	Auto	Defines the spacing mode to apply between objects when distributed.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seAlignStretch	bool	user	true	Defines whether alignment motion must be made as stretch. Valid values: true, false
seAlignToColumnInstanceMaxHeight	float	cellview	62500	Defines the maximum height of instances applicable for alignment.
seAlignToColumnInstanceMaxWidth	float	cellview	62500	Defines the maximum width of instances applicable for alignment.
seAlignToColumnInstanceMinHeight	float	cellview	0.0625	Defines the minimum height of instances applicable for alignment.
seAlignToColumnInstanceMinWidth	float	cellview	0.0625	Defines the minimum width of instances applicable for alignment.
seAMSPartitionAlwaysUpdateInterfaceElements	bool	user	false	Always updates interface elements in simulator-based partition. Valid values: true, false
seAMSPartitionAMS	bool	context	true	Annotates AMS cells in simulator-based partition. Valid values: true, false
seAMSPartitionAMSPacket	string	user	highlight9	Display packet for AMS cells in simulator-based partition.
seAMSPartitionAnalog	bool	context	true	Annotates analog devices. Valid values: true, false
seAMSPartitionAnalogPacket	string	user	annotate4	Display packet for analog devices.
seAMSPartitionAnalogPrimitives	bool	context	true	Highlight analog primitives in simulator-based partition. Valid values: true, false
seAMSPartitionDigital	bool	context	true	Annotate digital devices. Valid values: true, false
seAMSPartitionDigitalPacket	string	user	annotate7	Display packet for digital devices.
seAMSPartitionDynamic	bool	user	FALSE	Dynamic objects highlighting. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seAMSPartitionInstances	bool	context	true	Highlights instances. Valid values: true, false
seAMSPartitionInterfaceElements	bool	context	true	Highlights interface elements in simulator-based partition. Valid values: true, false
seAMSPartitionMixed	bool	context	true	Annotates mixed devices. Valid values: true, false
seAMSPartitionMixedLineStyle	string	user	medium Dashed	Display line style for mixed devices. Valid values: true, false
seAMSPartitionMixedSignalNets	bool	context	false	Highlights mixed signal nets in simulator-based partition. Valid values: true, false
seAMSPartitionNetlister	string	context	AMS	AMS netlister name for partition annotation.
seAMSPartitionNets	bool	context	true	Highlight nets and terminals. Valid values: true, false
seAMSPartitionSimulationReport	string	context	current	Choice of simulation report for partition: the current or previous (from history). Valid values: current, previous
seAMSPartitionSimulationReportPath	string	context		Simulation report path for simulator-based partition.
seAMSPartitionSimulationReportPathHistory	string	context		Simulation report paths history for simulator-based partition.
seAMSPartitionType	string	context	viewBased	Partition recognition type: view-based or simulator-based. Valid values: viewBased, simulatorBased
seAMSPartitionUnknown	bool	context	true	Annotates devices of unknown type. Valid values: true, false
seAMSPartitionUnknownPacket	string	user	markerAck	Displays packet for devices of unknown type.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seAMSPartitionVerilogA	bool	context	true	Annotate VerilogA cells in simulator-based partition. Valid values: true, false
seAMSPartitionVerilogAPacket	string	user	highlight9	Display packet for VerilogA cells in simulator-based partition.
seAnalogLibPortAMI	bool	cellview	false	AMI checkbox for switch AMI_... params of port element from analogLib Valid values: true, false
seAnalogLibPortIBISAMI	bool	cellview	false	IBIS_AMI checkbox for switchIBIS_AMI_... of port element from analogLib Valid values: true, false
seAnnotateOPCutOff	bool	user	true	Enables backannotation of Cut Off working region. Valid values: true, false
seAnnotateOPCutOffColor	string	user	red	Color for Cut Off working region annotation.
seAnnotateOPFile	string	user		The OP file to backannotate onto the schematic.
seAnnotateOPLinear	bool	user	true	Enables backannotation of Linear working region. Valid values: true, false
seAnnotateOPLinearColor	string	user	yellow	Color for Linear working region annotation.
seAnnotateOPNameMappingDir	string	user		That path to the name mapping directory for translating schematic names.
seAnnotateOPNodeVoltage	bool	user	true	Enables backannotation of node voltage. Valid values: true, false
seAnnotateOPSaturation	bool	user	true	Enables backannotation of Saturation working region. Valid values: true, false
seAnnotateOPSaturationColor	string	user	green	Color for Saturation working region annotation.
seAnnotateOPSearchDir	string	user		Search directory for OP files.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seAnnotateOPSubthreshold	bool	user	true	Enables backannotation of Subthreshold working region. Valid values: true, false
seAnnotateOPSubthresholdColor	string	user	orange	Color for Subthreshold working region annotation.
seAnnotateOPValues	bool	user	true	Enables backannotation of OP values. Valid values: true, false
seAnnotateOPVirtualRoot	string	user		The simulator namespace path to the device under test in the annotation simulation results.
seAutoLabelSystemNet	bool	cellview	true	Indicates whether a system net should be auto labeled if the same name labeled net is created. Valid values: true, false
seAutoNameBitOrder	seAutoNameBitOrderEnum	user	auto	Defines the order of bit indexes for auto created bus net. Valid values: ascending, descending, auto
seCheckAmbiguousConn	string	cellview	enable true severity warning dynamic true	Controls severity level and dynamic enabling of check for ambiguous connections.
seCheckAssignAssignment	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags use of a connection definition for an assignment value. (oaAssignAssignment)
seCheckAutoPopUpConsole	string	user	never	Controls whether console assistant is automatically popped up when violations of appropriate type are found. Possible values are never, onWarningsOrErrors, and onErrors.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCheckAutoSave	string	cellview	promptOnErrors	Controls saving of designs with violations during hierarchical checking. Valid values: never, always, promptOnErrors, promptOnWarningsOrErrors
seCheckAutoSaveWhenPass	seCheckAutoSaveWhenPass	cellview	always	Controls saving of designs when no any errors or warnings found. Valid values: never, always, prompt
seCheckAutoStartBrowser	string	cellview	onErrors	Controls whether the marker browser is automatically started when the top design contains the appropriate type of violation. Valid values: never, onWarningsOrErrors, onErrors
seCheckCellViewAssign	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags presence of cellview wide connection definition assignments.
seCheckCircularReference	string	cellview	enable false severity error dynamic false	Controls the severity level and dynamic enabling of circular reference check.
seCheckConnDefDefault	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that ensures the <code>sePromptOverlayUpdateInNetlisting</code> of a connection definition is the same as the value of the net upon which it is placed.
seCheckConnDefNonGlobalDefault	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags non-global for connection definitions.
seCheckConnDefOnNamedNet	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of check for connection definitions placed on nets with wire labels.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCheckConnDefOnNonGlobal	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags connection definitions placed on non-global nets.
seCheckConnDefOnTermConflict	string	cellview	enable false severity warning dynamic false	Controls the severity level and dynamic enabling of check for terminal connection definition default net/terminal name check.
seCheckConnDefOnTermDefault	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of check for connection definitions placed on nets with wire labels.
seCheckConnDefTermExpr	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags terminal expressions (used as of connection definitions).
seCheckConnectByName	string	cellview	enable false severity warning dynamic false	Controls severity level and dynamic enabling for connect-by-name check.
seCheckConnectDefDefault	enlist	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that ensures the of a connection definition is the same as the value of the net upon which it is placed.
seCheckConnectDefNonGlobalDefault	enlist	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags non-global for connection definitions.
seCheckConnectDefOnNonGlobal	enlist	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags connection definitions placed on non-global nets.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCheckCreateMarkerInDesign	bool	cellview	true	Controls whether ERC/SRC checks create OA markers in checked designs. Set to false to create violation markers in an overlay to avoid modifying designs. Valid values: true, false
seCheckCrossViews	bool	cellview	true	Controls whether cross view checking is performed before the rules for a design. Valid values: true, false
seCheckDeletePreexistingMarkersMode	string	user	all	Deletes error markers created by all tools or SRC/ERC after performing Check or Check and Save .
seCheckDuplicateCells	string	cellview	enable false severity error dynamic false	Controls the severity level and dynamic enabling of duplicate cells check.
seCheckDuplicateCellsCaseSensitive	bool	user	true	Controls whether to compare cell names in case insensitive manner. Valid values: true, false
seCheckDuplicateCellsCaseSensitive	bool	user	true	Controls whether to compare cell names in case insensitive manner. Valid values: true, false
seCheckDynamic	bool	cellview	false	Controls whether any dynamic checking is done. Valid values: true, false
seCheckFloatingInOutInstPin	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating input/output instance pin check.
seCheckFloatingInOutPin	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating input/output pin check.
seCheckFloatingInputInstPin	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating input instance pin check.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCheckFloatingInputOutputInstPin	enlist	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating input/output instance pin check.
seCheckFloatingInputOutputPin	enlist	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating input/output pin check.
seCheckFloatingInputPin	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating input pin check.
seCheckFloatingNet	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating net check.
seCheckFloatingOutputInstPin	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating output instance pin check.
seCheckFloatingOutputPin	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating output pin check.
seCheckFloatingSwitchInstPin	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating switch instance pin check.
seCheckFloatingSwitchPin	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating switch pin check.
seCheckFloatingTristateInstPin	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating tristate instance pin check.
seCheckFloatingTristatePin	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for floating tristate pin check.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCheckForced	bool	cellview	true	Controls whether a design first be checked to see if the rules need to be executed. Valid values: true, false
seCheckGlobalHierNetShort	string	cellview	enable false severity error dynamic false	Controls the severity level and dynamic enabling of globals shorted through hierarchy check.
seCheckGlobalInst	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags presence of dedicated global indicator symbol.
seCheckGlobalNetName	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags global names not ending in "!".
seCheckGlobalNetShort	string	cellview	enable true severity error dynamic true	Controls the severity level and dynamic enabling of check for connections between global nets and non-global terminals.
seCheckHiddenWireLabel	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of check for wire labels without parent segment.
seCheckHspiceName	enlist	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of HSPICE namespace checks.
seCheckHspiceNamespace	string	cellview	enable false severity warning dynamic false	Controls the severity level and dynamic enabling of HSPICE namespace checks.
seCheckIgnoreOverlapsWithGlobals	bool	user	false	Controls whether to ignore instance overlaps with global symbols. Valid values: true, false
seCheckImplicitVectoredNet	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling of check for implicit vectored nets.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCheckInstOverlap	string	cellview	enable true severity warning dynamic true	Controls severity level and dynamic enabling for check of overlapping instances.
seCheckLibraryFilter	string	cellview		A list of zero or more names to be used to limit or control which libraries are processed.
seCheckLibraryFilterIsExclude	bool	cellview	false	Indicates whether the list from <code>seCheckLibraryFilter</code> should be considered an exclusion list.
seCheckMaximumViolationDisplay	int	cellview	20	Indicates the maximum number of violations to display in the console per rule.
seCheckMultiVFlagConnection	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for multiple vflag symbols on a single net check.
seCheckNameShort	string	cellview	enable true severity error dynamic true	Controls severity level and dynamic enabling for shorted net check.
seCheckNoMarkers	bool	cellview	false	Controls whether markers are created. Valid values: true, false
seCheckNonOrthogonalConn	string	cellview	enable true severity warning dynamic true	Controls severity level and dynamic enabling of check for non-orthogonal routing.
seCheckOnOpenChecks	string	user	{Template Self-Coherence} Out-of-Date-Intent {Connectivity Mismatch} {Missing } Out-of-Date	Specifies the list of rule names to be run on schematic open.
seCheckOrphanWireLabel	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of check for wire labels without parent segment.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCheckOverlapTypeIsPartial	bool	user	true	Controls whether to use partial or exact overlap type in instance overlap check. Valid values: true, false
seCheckOverlayClipMaskBorder	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of check for overlay clip masks border crossing the instance area in overlay design.
seCheckOverlayMaskBorder	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of check for overlay mask border crossing the instance area in overlay design.
seCheckOverlayMaskedPin	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of check for overlay mask over pin object check in overlay design.
seCheckPcellDefTypeMismatch	enlist	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of check that flags difference between types of same-named oaParam and Custom Compiler parameter.
seCheckPcellDefValMismatch	enlist	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of check that flags difference between defaults of same-named oaParam and Custom Compiler parameter.
seCheckPEL	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags PEL expressions.
seCheckPinBittermInconsistency	string	cellview	enable true severity error dynamic false	Controls the severity level and dynamic enabling of bitterm inconsistency checks.
seCheckPinOnNamedNet	string	cellview	enable false severity warning dynamic false	Controls severity level and dynamic enabling for schematic pin on conflicting net check.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCheckPortOrderMismatch	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for Port Order Property Mismatch check.
seCheckPromptCurrent	string	cellview	onErrors	Controls saving of designs in non-hierarchical checking. Valid values: never, always, onErrors, onErrorOrWarnings
seCheckShortedOutput	string	cellview	enable true severity warning dynamic true	Controls severity level and dynamic enabling for shorted output terminal check.
seCheckShortedTerms	string	cellview	enable true severity warning dynamic false	Controls the severity level and dynamic enabling of compatibility check that flags shorted terminals.
seCheckSignalWithouthSource	string	cellview	enable true severity warning dynamic false	Controls severity level and dynamic enabling for input instance pin check.
seCheckSkipReadOnlyLibraries	bool	cellview	false	Indicates if all the read-only libraries should be added to exclusion list. Valid values: true, false
seCheckSolderOnCross	string	cellview	enable true severity warning dynamic true	Controls severity level and dynamic enabling for check of solder dots on crossing wires.
seCheckTextViewCellName	string	user	warning	Severity of the module versus cell name text view check.
seCheckUnboundInstances	string	cellview	enable true severity error dynamic false	Controls severity level and dynamic enabling for unbound instances check.
seCheckUnconnectedWire	string	cellview	enable false severity warning dynamic false	Controls severity level and dynamic enabling for floating/unconnected wire check.
seCheckVerilogName	enlist	cellview	enable false severity warning dynamic false	Controls the severity level and dynamic enabling of Verilog namespace checks.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCheckVerilogNamespace	string	cellview	enable false severity warning dynamic false	Controls the severity level and dynamic enabling of Verilog namespace checks.
seCheckViewFilter	string	cellview		A list of zero or more names to be used to limit or control which libraries, cells and views are processed.
seCheckViewFilterIsExclude	bool	cellview	false	Indicates whether the list from <code>seCheckViewFilter</code> should be considered an exclusion list. Valid values: true, false
seCheckWidthMismatch	string	cellview	enable true severity error dynamic true	Controls the severity level and dynamic enabling of check that flags failed pin connections due to width mismatch.
seCheckWireLabelNetConflict	string	cellview	enable true severity error dynamic true	Controls the severity level and dynamic enabling of check for wire labels conflicting with nets of their parent wire segments.
seCircularRulerPacket	string	cellview	ruler	Colors.
seColorizationColorSelectorFilter	string	user	black blue brown cadetBlue chocolate cream cyan forest gold green lime magenta maroon navy orange pink purple red salmon silver slate tan violet white yellow	Color filter for Colorization definition colors.
seConnectDefinitionColor	string	cellview	<auto>	Default color for connect definition labels.
seConnectDefinitionDefaultNet	string	cellview		Connection definition net name preference.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seConnectDefinitionFont	string	cellview	stick	Default font for connect definition labels. Valid values: euroStyle, gothic, math, roman, script, stick, fixed, swedish, mil
seConnectDefinitionHeight	float	cellview	0.0625	Default height for connect definition labels.
seConnectDefinitionJustification	string	cellview	lowerLeft	Default justification for connect definition labels. Valid values: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight
seConnectDefinitionName	string	cellview		Connection definition name preference.
seConnectDefinitionTextAttributes	bool	cellview	false	Expand state of text attributes group box. Valid values: true, false
seConnEnableDynamicRun	bool	context	true	Enable/Disable SE connectivity engine initialization and dynamic run. Valid values: true, false
seConnEnableSymbolCheck	bool	user	true	Enable/disable SE connectivity engine in symbol view. Valid values: true, false
seConnEngineEnableForRO	bool	user	false	Enables the SE connectivity initialization and incremental processing for R/O schematics. Valid values: true, false
seConnMultiThreadOpenThreshold	int	user	0	The design's total nets count threshold for what to use a multithreaded initialization.
seConstraintViewName	string	cellview	constraints	Controls constraint view name of schematic design.
seCopyAngleMode	string	cellview	anyAngle	Indicates the angle mode for the interactive copy command. Valid values: orthogonal, diagonal, anyAngle

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCopyChop	bool	cellview	false	If true, chop the wires under placed instance. Valid values: true, false
seCopyColumns	int	cellview	1	Indicates number of columns to create.
seCopyDialogMode	bool	user	false	ise::copy command options dialog eject state. Valid values: true, false
seCopyOrigin	string	cellview	anchor	Indicates what origin to use when rotating. Valid values: anchor, center
seCopyRows	int	cellview	1	Indicates number of rows to create.
seCreateConnectDefDialogMode	bool	user	false	ise::createConnectDef command options dialog eject state. Valid values: true, false
seCreateInstDialogMode	bool	user	false	ise::createInst command options dialog eject state. Valid values: true, false
seCreateLabelDialogMode	bool	user	false	ise::createLabel command options dialog eject state. Valid values: true, false
seCreateNoteDialogMode	bool	user	false	ise::createNote command options dialog eject state. Valid values: true, false
seCreateOpenOverlayOpenInNewWindow	bool	user	false	If true, a new overlay design is opened in a new window. Valid values: true, false
seCreateOverlayConfigConfigName	string	context		Config Name for config creation.
seCreateOverlayConfigTemplate	string	user	Verilog	Template for overlay config creation.
seCreateOverlayRuleType	string	user	Occurrence	Default rule type for rule creation in overlay designs.
seCreateRulerAngleMode	string	cellview	orthogonal	Angle mode. Valid values: anyAngle, diagonal, orthogonal, xFirst, yFirst

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seCreateRulerMode	string	cellview	linear	Ruler mode. Valid values: linear, circular, dimension
seCreateRulerPacket	string	cellview	ruler	Current circular ruler packet (corresponds to Color selector in the toolbar).
seCreateRulerPointsMode	int	cellview	0	Points. Valid values: Any integer in the [0 1] range
seCreateSchematicPinDialogMode	bool	user	false	Valid values: true, false
seCreateSymbolPinDialogMode	bool	user	false	Valid values: true, false
seCreateWireDialogMode	bool	user	false	ise::createWire command options dialog eject state. Valid values: true, false
seCreateWireNameDialogMode	bool	user	false	Valid values: true, false
seCycleRulerPackets	bool	cellview	true	Cycle. Valid values: true, false
seDefaultNameDisplay	enNameDisplay	user	inst	Default display for cdsName evaluation. Valid values: none, cell, inst
seDefaultNetPrefix	string	cellview	net	Prefix to use for generated net names in the Schematic Editor.
seDefaultTerminalDisplay	enTermDisplay	user	none	Default display for cdsTerm evaluation.
seDescendUseExternalEditor	bool	user	false	Option to open the external netlist file by the external text editor when descending an instance. Valid values: true, false
seDesignNavigatorCheckCDFIgnore	bool	context	true	Check CDF Parameter for Ignore Instances in the SE design navigator.
seDesignNavigatorExpandBusNets	bool	user	false	Expand bus nets bits under the bus net nodes in the Nets tab of the SE Design Navigator. Valid values: true, false

Table 188 *Schematic Editing Preferences (Continued)*

Name	Type	Scope	Default	Description
seDesignNavigatorFlightlinePacket	string	context	""	The packet name of the s when selecting nets on SE design navigator.
seDesignNavigatorHighlightOverlap Nets	bool	user	false	Cross highlight overlapped nets on SE canvas when selecting bus bit nets on SE design navigator. Valid values: true, false
seDesignNavigatorInstancesView	string	context	se_all	Subview to use in the Insts tab of the SE Design Navigator.
seDesignNavigatorMaxExpandBits	int	user	4096	Maximum expanded bus nets bits under the bus net nodes in the Nets tab of the SE Design Navigator.
seDesignNavigatorNetsView	string	context	se_all	Subview to use in the Nets tab of the SE Design Navigator.
seDesignNavigatorTerminalsView	string	context	se_all	Subview to use in the Terminals tab of the SE Design Navigator.
seDesignSummaryCountBy	enSeDesignSummaryCountBy	context	byLCV	The results consolidation mode of SE Design Summary.
seDesignSummaryCrossHierarchyLevel	int	context	1	Defines interesting depth of hierarchy.
seDesignSummaryExcludedLibraries	string	context		List of libraries the cells from which are ignored.
seDesignSummaryMappedCellsFiltering	enSeDesignSummaryMappedCellsFiltering	context	none	Controls either counting instances without a mapping data available in the mapping file. Valid values: none, mapped, notMapped
seDesignSummaryMappingFile	string	context		Mapping file for SE Design Summary processing.
seDesignSummaryResultType	enSeDesignSummaryResultType	context	formattedText	The result type to output in the Design Summary dialog box for Schematic Editor.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seDesignSummaryScope	enBillOfMaterialsScope	context	From Current	Defines the viewing level for the summary content. Valid values: {Whole Design} {From Current} {From Top}
seDevicePaletteCategory	string	context	Uncategorized	The current and default category for the assistant in layout windows.
seDirectManipulation	bool	user	true	When enabled, click-drag stretch the active object. When disabled click-drag is window select. Valid values: true, false
seDirectManipulationSelectedOnly	bool	user	false	Only has effect when <code>seDirectManipulation</code> is set to true. If <code>seDirectManipulationSelectedOnly</code> is set to false, the active object can be click-drag stretched. If set to true, <i>only</i> selected objects can be click-drag manipulated. If the active object is not selected, click-drag be a selection event. Valid values: true, false
seDisplayCursorText	bool	user	false	Controls the display of net and instTerm information on the mouse pointer. Valid values: true, false
seDragThreshold	int	user	10	Distance the cursor must move in the Schematic and Symbol Editors with a button depressed to consider the event a drag rather than a button press.
seDrawAxisMethod	drawAxisMethod	user	full	Determines whether Axis drawn in seSchematic and seSymbol is +/- infinite or +/- 75 pixels from {0 0}.
seDrawFilter	int	user	1	Minimal size of objects to query.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seDrawPinColor	bool	user	false	Controls whether the instance pin should be colored. Five color packets are available if true: inputPin, outputPin, inputOutputPin, switchPin and tristatePin (one for each type). Valid values: true, false
seDynamicLabelAttachments	bool	user	false	Controls whether mouse-over activation of wiring or wire names display a temporary line in the Schematic Editor. Valid values: true, false
seDynamicNetHighlighting	bool	user	false	Controls whether mouse-over highlighting of wiring objects shows physical and by-name connections in the Schematic Editor. Valid values: true, false
seDynamicNetNameRatio	float	user	1	Controls dynamic net name label ratio compared with created net names.
seDynamicNetShowFlightline	bool	user	true	Controls whether mouse-over highlighting of wiring objects shows s of connections in Schematic Editor. Valid values: true, false
seDynamicNetShowFlightlineConn Def	bool	user	true	Controls whether mouse-over highlighting of wiring objects shows s of connection definition instances in schematic editor. Valid values: true, false
seEstimatedParasiticsCtotalAddVia	bool	context	false	Add via for C total. Valid values: true, false
seEstimatedParasiticsCustomDevice LibName	string	context		Library name of custom parasitic devices.
seEstimatedParasiticsCustomDevice NamePrefix	string	context		Prefix for custom parasitic devices.
seEstimatedParasiticsDenseLineAboveLayer	bool	context	false	Add dense lines at above layer.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seEstimatedParasiticsDenseLineBelowLayer	bool	context	false	Add dense lines at below layer.
seEstimatedParasiticsDenseLineDimensionUserDefined	bool	context	false	Use user-defined dimensions for dense lines.
seEstimatedParasiticsDenseLineSameLayer	bool	context	true	Add dense lines at same layer.
seEstimatedParasiticsRmetalAddVia	bool	context	true	Add via for RMetal. Valid values: true, false
seEstimatedParasiticsUseDenseLineModel	bool	context	false	Use dense line model.
seExcludeCellCategories	string	user		List of categories whose cells must be excluded from the Library, Cell, View cell list.
seFatWireWidth	float	cellview	0.0625	Width of fat wires in user units.
seFlightlineName	string	cellview		Name of the object in schematic for which flight lines should be toggled.
seGenerateLayoutSizeSymbol	bool	user	false	Specifies whether create layout-sized symbol when generate symbol from layout.
seGenSimInfoSectionNames	string	user		SimInfo section names to be created on symbol generation.
seIgnoreInstActionValue	seIgnoreInstActionValue	cellview	ignore	The value to update to nlAction property
seIgnoreInstDialogMode	bool	user	true	ise::ignoreInst command options dialog eject state. Valid values: true, false
seIgnoreInstIgnoreProperty	string	user	!xIgnore	Defines ignore property list to set on the ignored instances. Valid values: true, false
seIgnoreInstInfoRatio	float	user	1	The ratio to display ignored instance information. Valid values: true, false

Table 188 *Schematic Editing Preferences (Continued)*

Name	Type	Scope	Default	Description
selIgnoreInstMode	selIgnoreInstMode	cellview	all	Mode to control whether to set ignore properties or remove them. Valid values: true, false
selIgnoreInstNetlist	string	cellview	CDL	Defines netlist that ignore instance applies to. Valid values: true, false
selIgnoreInstShortProperty	string	user	!xRemoveDevice	Defines short property list to set on the ignored instances. Valid values: true, false
selIgnoreInstShortTerms	bool	cellview	false	Sets ignored instance to have shorted terminals for specific netlist. Valid values: true, false
selIgnoreInstShortTermsAll	bool	cellview	false	Sets ignored instance to have shorted terminals. Valid values: true, false
selIgnoreInstShortTermsAllTerms	string	cellview		Defines shorted terminals for remove device property on the ignored instances. Valid values: true, false
selIgnoreInstShortTermsTerms	string	cellview		Defines shorted terminals for remove device property on the ignored instances for specific netlist. Valid values: true, false
selImportSymbolDialogMode	bool	user	false	Valid values: true, false
selImportSymbolMaster	lcv	cellview	//	Indicates the library, cell, and view to import.
selImportSymbolOrientation	string	cellview	R0	Indicates the orientation used when placing the data. Valid values: R0, R90, R180, R270, MX, , MXR90, MYR90
selImportSymbolPins	bool	cellview	true	Indicates whether pins should be copied when importing a symbol. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
selImportSymbolSelectionRegions	bool	cellview	true	Indicates whether selection regions should be copied when importing a symbol. Valid values: true, false
selImportSymbolShapes	bool	cellview	true	Indicates whether shapes should be copied when importing a symbol. Valid values: true, false
selImportSymbolText	bool	cellview	true	Indicates whether notes and labels should be copied when importing a symbol. Valid values: true, false
selInstAngle	string	cellview	diagonal	Default angle mode for array creation. Valid values: orthogonal, diagonal, anyAngle
selInstChop	bool	cellview	false	If true, chop the wires under placed instance. Valid values: true, false
selInstCreateStub	bool	cellview	false	Creates stub on instance terminals . Valid values: true, false
selInstExpand	bool	cellview	false	Default for expanding user entered vector names. Valid values: true, false
selInstMaster	lcv	cellview	//	Default library/cell/view for instance creation.
selInstName	string	cellview		Default names applied to instances to be placed.
selInstNumColumns	int	cellview	1	Default number of columns to create.
selInstNumRows	int	cellview	1	Default number of rows to create.
selInstOrientation	string	cellview	R0	Default orientation applied to instances to be placed. Valid values: R0, R90, R180, R270, MX, , MXR90, MYR90

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seInstOverlapPercent	int	user	20	Indicates percent overlap of Instances allowed in the Schematic Editor. 0 means no overlap allowed, 100 means no restriction for overlap. Valid values: Any integer in the [0 100] range
seInstShowCellCategories	bool	user	true	Determines if showing cell categories in the Add Instance dialog box. Valid values: true, false
seInstShowCellViewTable	bool	user	false	Uses table selector in the Add Instance dialog box. Valid values: true, false
seLabelColor	string	cellview		Default color setting for label objects.
seLabelDisplayParametersType	bool	user	true	Show/hide Type column in parameters' table of Device Label Editor. Valid values: true, false
seLabelDisplayParametersValue	bool	user	true	Show/hide Value column in parameters' table of Device Label Editor. Valid values: true, false
seLabelDisplayParametersVisibility	bool	user	true	Show/hide Visibility column in parameters' table of Device Label Editor. Valid values: true, false
seLabelDisplayScope	string	cellview	Instance	Default scope for Device Label Editor.
seLabelDrafting	bool	cellview	true	Default drafting setting for label objects. Valid values: true, false
seLabelEditorShowAttributes	bool	user	true	Show/hide Attributes in Device Label Editor. Valid values: true, false
seLabelEditorShowParameters	bool	user	true	Show/hide Parameters in Device Label Editor. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seLabelFont	string	cellview	stick	Default font for note objects. Valid values: euroStyle, gothic, math, roman, script, stick, fixed, swedish, milSpec
seLabelFunction	string	cellview	N/A	Default function for labels.
seLabelHeight	float	cellview	0.0625	Default height for label objects.
seLabelJust	string	cellview	lowerCenter	Default justification for label objects. Valid values: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight
seLabelNetExpressionDefault	string	cellview		Default net for 'Net expression' label function.
seLabelNetExpressionName	string	cellview		Name for 'Net expression' label function.
seLabelOrientation	string	cellview	R0	Default orientation for label objects. Valid values: R0, R90, R180, R270, MX, , MXR90, MYR90
seLabelOverbar	bool	cellview	false	Default overbar setting for label objects. Valid values: true, false
seLabelText	string	cellview		Indicates the for label text.
seLabelTextAttributes	bool	cellview	false	Expand state of text attributes group box. Valid values: true, false
seLabelType	string	cellview	Normal	Default label type. Must be enEvalType. Valid values: Normal, IL, SEL
seLogicalCompareResizeColumn	bool	user	false	Allows column width of result table to be resizable. Valid values: true, false
seLogicalCompareZoomOnHighlight	bool	user	true	Fits on selection of an entry in the Differences table. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seLogicalCompareZoomScale	int	user	90	Maximum percentage of each axis that the data extents may cover.
seMagnifyDialogSize	enSize	cellview	Large	Magnify size of dialog box based on canvas sizes. Valid values: Small, Medium, Large
seMagnifyFitTo	enFitTo	cellview	<file path>	Magnify factor (150px/uu 250px/uu 400px/uu 600px/uu).
seMagnifyFitType	bool	cellview	false	Magnify factor (Zoom and Fit). Valid values: true, false
seMagnifyPin	bool	cellview	false	Fix magnify dialog. Valid values: true, false
seMagnifyPinLocation	enMgDlgLocation	cellview	Auto	Magnify dialog location for fixing. Valid values: Auto {Near Cursor} {Top Left} {Top Right} {Bottom Left} {Bottom Right}
seMagnifyZoomBy	enZoomBy	cellview	3x	Magnify factor. Valid values: 2x 3x 5x 10x
seMagnifyZoomType	bool	cellview	true	Magnify factor (Zoom and Fit). Valid values: true, false
seMaskClip	bool	cellview	false	Clip mode in overlay mask creation. Valid values: true, false
seMaskClipRectCreationMode	string	cellview	manual	Mask creation mode in overlay mask creation command.
seMinimalPixelsForRoutedSegment	int	user	15	The minimal length in pixels for a segment to route during interactive wiring.
seMoveAngleMode	string	cellview	anyAngle	Indicates the angle mode for the interactive move command. Valid values: orthogonal, diagonal, anyAngle
seMoveDialogMode	bool	user	false	ise::move command options dialog eject state. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seMoveOrigin	string	cellview	anchor	Indicates which origin to use when rotating. Valid values: anchor, center
seNavigatorAutoRefresh	bool	cellview	false	Controls whether the Hierarchy Navigator is automatically refreshed based on design data and config changes. Valid values: true, false
seNavigatorTrackContext	bool	cellview	true	Controls whether the Hierarchy Navigator ensures the current edit lineage is visible on context changes. Valid values: true, false
seNetColorDefsFileName	string	user	netColorDefs.tcl	Schematic nets colorization definitions for load/save.
seNetTracerDumpFileFormat	string	context	Nets Report	The file format for Net Tracer to save the net information.
seNetTracerDumpFileName	string	context		The File name for Net Tracer to save the net information.
seNetTracerDumpNetName	string	context		The Net name for Net Tracer to to the file.
seNetTracerDumpOpenEditor	bool	context	false	Control if opening file by text viewer after Net Tracer . Valid values: true, false
seNetTracerEquivalentNetFilter	string	context		The text on the filter field in Net Tracer equivalent net table.
seNetTracerLoadFileName	string	context		The file name for Net Tracer to load the net information.
seNetTracerLoadMode	string	context	Replace	Controls if replacing or appending s when loading from file. Valid values: Replace, Append
seNetTracerMaxDepth	int	context	-1	Maximum level to search below the given context n net tracer. Negative value searches all levels.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seNetTracerNetFilter	string	context		The text on the filter field in Net Tracer net table.
seNoteAttributes	bool	cellview	false	Expand state of text attributes group box. Valid values: true, false
seNoteColor	string	cellview		Default color setting for note objects.
seNoteDrafting	bool	cellview	true	Default drafting setting for note objects. Valid values: true, false
seNoteFont	string	cellview	stick	Default font for label objects. Valid values: euroStyle, gothic, math, roman, script, stick, fixed, swedish, milSpec
seNoteHeight	float	cellview	0.0625	Default height for label objects.
seNoteJust	string	cellview	centerCenter	Default justification for note objects. Valid values: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight
seNoteOrientation	string	cellview	R0	Default orientation for note objects. Valid values: R0, R90, R180, R270, MX, , MXR90, MYR90
seNoteOverbar	bool	cellview	false	Default overbar setting for note objects. Valid values: true, false
seNoteShape	string	cellview		Default note shape for note objects. Valid values: line, rectangle, polygon, circle, ellipse, arc
seNoteShapeColor	string	cellview		Default color setting for note shape.
seNoteShapeLineStyle	string	cellview		Default line style setting for note shape.
seNoteText	string	cellview		Indicates the for note text.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seOverlayAutoUpdateInNetlisting	bool	user	true	If true, the out-of-date overlay design is updated from reference automatically in netlisting. Valid values: true, false
seOverlayClearClippedInstsOnSaveAs	bool	user	true	If true, clears the nllgnore setting of clipped-out instances during Save As Schematic for overlay. Valid values: true, false
seOverlayInstancePrefixExtension	string	user	Overlay	Extension of name prefix for overlay instance name generation. ('ROverlay4')
seOverlayUpdateConflictsResolve	string	user	autoFix	Defines how to resolve conflicts in overlay design update.
seOverlayUpdateMode	string	user	explicit	Defines how to update the overlay design when its reference is modified. Values are auto, prompt, explicit. Auto: to implicitly the overlay on opening if it is . Prompt: to prompt via a dialog with options how to the overlay view. Explicit: the default, allows you to choose when to explicitly the view. The need to the view to the reference is indicated in the GUI by the X icon.
seOverlayViewNameDelimiter	string	user		The delimiter used to concatenate an occurrence to overlay design view names.
seParasiticEstAddParasiticProperties	bool	user	true	Add the parasitic properties. Valid values: true, false
seParasiticEstComponentLib	string	user		Component library of parasitic estimation.
seParasiticEstComponentParamProc	string	user		Procedure name to set parameter on estimated component.
seParasiticEstLayoutScaleFactor	unknown	context	1	Layout scale factor of parasitic estimation.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seParasiticEstSchematicPCell	string	user		Net parasitic PCell of parasitic estimation.
seParasiticEstTargetSchematic	string	context		Target schematic of parasitic estimation.
seParasiticEstTargetSchematicSameAsSource	bool	user	false	Specifies whether target schematic is the same as source schematic. Valid values: true, false
seParasiticEstUsePCell	bool	user	true	Use PCell of parasitic estimation. Valid values: true, false
seParasiticEstViaComponentMap	string	user		Via component mapping of parasitic estimation.
seParasiticEstWireComponentMap	string	user		Wire component mapping of parasitic estimation.
sePasteAnchor	se_paste_justification	cellview	lowerLeft	Anchor justification control.
sePasteAnchorIsOrigin	bool	cellview	true	Use origin as anchor of objects to paste.
sePasteAngleMode	string	cellview	anyAngle	Indicates the angle mode for the interactive paste command.
sePasteCols	int	cellview	1	Number of columns to paste from clipboard (schematic).
sePasteKeepNets	string	cellview	labeled	Net names paste control.
sePasteRows	int	cellview	1	Number of rows to paste from clipboard (schematic).
sePDABackgroundConnectionDefinition	string	context	maroon	Defines background color in Power Domain Analyzer dialog box's Inherited Connections table for connection definitions (parameters).
sePDABackgroundInheritedConnections	string	context	maroon	Defines background color in Power Domain Analyzer dialog box's Net table for supply nets found only by inherited connection.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
sePDABackgroundOfTrees	string	context	white	Defines background color of tree widgets in Power Domain Analyzer dialog box.
sePDABackgroundParameterProcessing	string	context	dimGray	Defines background color in Power Domain Analyzer dialog box's Inherited Connections table for parameters processing.
sePDABackgroundPhysicalConnections	string	context	black	Defines background color in Power Domain Analyzer dialog box's Net table for supply nets connected to any device.
sePDABackgroundPropagations	string	context	dimGray	Defines background color in Power Domain Analyzer dialog box's Net table for supply nets found only by propagation from lower hierarchy.
sePDABackgroundValuableParameters	string	context	dimGray	Defines background color in Power Domain Analyzer dialog box's Inherited Connections table for parameters processing.
sePDAGroundRegexp	string	context	<code>([vV][sS][sS][gG][nN][dD]).*</code>	Regex pattern for ground net names.
sePDAIgnoredLibraries	string	context		List of libraries whose cells are ignored (together with their lower hierarchy) in nets collecting for Power Domain Analyzer dialog box.
sePDAlnstanceHighlighting	string	context	magenta	Instance Highlighting color in case of Show Instances action in Power Domain Analyzer dialog box.
sePDAAOtherRegexp	string	context	<code>([Cc]lock [Rr]eset)</code>	Regex pattern for other (not supply) global net names.
sePDAPowerRegexp	string	context	<code>[vV][dD][dD].*</code>	Regex pattern for power net names.
sePDARowCountOfTablesInChecksTab	int	user	200	Indicates row count for each table in Checks tab in Power Domain Analyzer dialog box.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
sePDAShowEmptyRows	bool	context	true	Indicates whether to show empty row in tree widgets. Valid values: true, false
sePDAShowGround	bool	context	true	If true, show the ground nets in Power Domain Analyzer dialog box's table. Valid values: true, false
sePDAShowInheritedConnections	bool	context	true	If true, show the nets found by inherited connection in Power Domain Analyzer dialog box's table. Valid values: true, false
sePDAShowInheritedConnectionsTracer	bool	context	true	If true, show the Inherited Connections Tracer as separate tab in Power Domain Analyzer dialog box's table. Valid values: true, false
sePDAShowNonGlobals	bool	context	false	If true, also show the found non-global nets in Power Domain Analyzer dialog box's table. Valid values: true, false
sePDAShowOther	bool	context	false	If true, show other (not supply) nets in Power Domain Analyzer dialog box's table. Valid values: true, false
sePDAShowPhysicalConnections	bool	context	true	If true, show the nets physically connected to in Power Domain Analyzer dialog box's table. Valid values: true, false
sePDAShowPower	bool	context	true	If true, show the power nets in Power Domain Analyzer dialog box's table. Valid values: true, false
sePDAShowPropagations	bool	context	true	If true, show also the nets found only by propagation from lower hierarchy in Power Domain Analyzer dialog box's table. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
sePeekDialogKeepDistance	string	user	Middle	Maximum distance in pixels between the cursor and an object to keep the dialog box shown.
sePeekPin	bool	cellview	false	Fix peek dialog box. Valid values: true, false
sePeekPinLocation	enPinLocation	cellview	Auto	Peek dialog location for fixing. Valid values: Auto {Near Cursor} {Top Left} {Top Right} {Bottom Left} {Bottom Right}
sePlotSignalLowerCaseMode	bool	user	false	The Plot Signal LowerCase mode. True if in lowercase mode. Valid values: true, false
sePlotSignalMode	string	user	Append	Determines Plot Signal mode. Valid values: append new replace newPanel.
sePlotSignalNameMappingDir	string	user		Path to cnl directory generated by netlisting and used for mapping schematic names to netlist names.
sePlotSignalNetlister	string	user	HSPICE	Select Plot Signal netlister. Valid netlisters: HSPICE Verilog.
sePlotSignalPlotFilesInSeparateTabs	bool	user	true	Plot files in separate tabs. Valid values: true, false
sePlotSignalSignalType	sePlotSignalTypeEnum	user	Prefer Analog	The preferred signal type.
sePlotSignalVirtualTop	string	user		Determine text view's Virtual Top.
sePlotSignalWaveformFiles	string	user		Retrieve Waveform files.
sePlotSignalWaveformFilesData	string	user		Retrieve list for Waveform Files data.
sePlotSignalWaveformToolName	string	user	WaveView	Selects the Waveform Viewer tool name. Valid values: WaveView, nWave

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
sePortOrderSynchronizationViews	string	cellview	1	The list of view names to which port order should be propagated.
sePropagateNetNameToPins	bool	user	false	Indicates whether a change in wire name should be propagated to connected terminals. Changing a wire label changes the physically connected pin name if it matches the original wire label. Does not change physically disjointed objects. Adding a wire label does not change the pin name. Valid values: true, false
sePropagatePinNameToNets	bool	user	true	Indicates whether a change in terminal name should be propagated to connected wires. Valid values: true, false
sePropagateSourceNameOnPinAttach	bool	user	false	Indicates whether moving pins or wires should propagate the source name to the new attached wires or pins Valid values: true, false
sePropertyEditorShownAttrsArc	string	user	ellipseBBox startAngle stopAngle color lineStyle fillStyle	Shown order of schematic and symbol arc attributes.
sePropertyEditorShownAttrsCircle	string	user	color lineStyle fillStyle	Shown order of schematic and symbol circle attributes.
sePropertyEditorShownAttrsCircularRuler	string	user	origin scribeAngle circles lineType labelSize	Shown order of schematic and symbol instance circular ruler attributes.

Table 188 *Schematic Editing Preferences (Continued)*

Name	Type	Scope	Default	Description
sePropertyEditorShownAttrsConnectDefLabel	string	user	effectiveName labelName labelType visible defaultNet font height origin orient alignment drafting overbar color	Shown order of schematic and symbol connection definition label attributes.
sePropertyEditorShownAttrsEllipse	string	user	color lineStyle fillStyle	Shown order of schematic and symbol ellipse attributes.
sePropertyEditorShownAttrsImage	string	user	file origin size displayMode alignment	Shown order of schematic and symbol image attributes.
sePropertyEditorShownAttrsInst	string	user	name origin orient usage physicalOnly	Shown order of schematic and symbol instance attributes.
sePropertyEditorShownAttrsInstConnectDefLabel	string	user	effectiveName labelName labelType visible defaultNet font height origin orient alignment drafting overbar color	Shown order of schematic and symbol instance connection definition label attributes.
sePropertyEditorShownAttrsInstLabel	string	user	effectiveText text labelType visible font height origin orient alignment drafting overbar color	Shown order of schematic and symbol instance label attributes.
sePropertyEditorShownAttrsInstName	string	user	effectiveText text labelType visible font height origin orient alignment drafting overbar color	Shown order of schematic and symbol instance name attributes.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
sePropertyEditorShownAttrsInstPin	string	user	name accessDir net	Shown order of schematic and symbol instance pin attributes.
sePropertyEditorShownAttrsInstPinName	string	user	name origin orient usage physicalOnly placementStatus	Displayed order of schematic and symbol instance attributes.
sePropertyEditorShownAttrsLabel	string	user	text labelType visible font height origin orient alignment drafting overbar color	Shown order of schematic and symbol label attributes.
sePropertyEditorShownAttrsLine	string	user	points width color lineStyle fillStyle	Shown order of schematic and symbol line attributes.
sePropertyEditorShownAttrsLinearRuler	string	user	points mirror packet lineType labelSize	Shown order of schematic and symbol instance linear ruler attributes.
sePropertyEditorShownAttrsMarker	string	user	severity shortMsg msg tool deleteWhen points	Shown order of schematic and symbol marker attributes.
sePropertyEditorShownAttrsNote	string	user	text visible font height origin orient alignment drafting color	Shown order of schematic and symbol marker attributes.
sePropertyEditorShownAttrsNoteShape	string	user	ellipseBBox startAngle stopAngle points width color lineStyle fillStyle	Shown order of schematic and symbol instance note shape attributes.
sePropertyEditorShownAttrsPin	string	user	name accessDir shape origin orient type	Shown order of schematic and symbol pin attributes.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
sePropertyEditorShownAttrsPinName	string	user	text visible font height origin orient alignment drafting overbar color	Shown order of schematic and symbol pin name attributes.
sePropertyEditorShownAttrsPolygon	string	user	points color lineStyle fillStyle	Shown order of schematic and symbol polygon attributes.
sePropertyEditorShownAttrsRectangle	string	user	color lineStyle fillStyle	Shown order of schematic and symbol rectangle attributes.
sePropertyEditorShownAttrsWire	string	user	name nameVisibility color lineStyle width	Shown order of schematic and symbol wire attributes.
sePropertyEditorShownAttrsWireName	string	user	text visible wire font height orient alignment global drafting overbar color verticalBundle	Shown order of schematic and symbol wire name attributes. Note: wireAttach does not correspond to any attribute as it is an action and net.name corresponds to a hierarchical attribute.
seRemoveAllMarkers	bool	user	false	Controls whether all existing markers should be removed. Valid values: true, false
seRenameInstExpand	bool	cellview	false	Default for expanding user-defined vector names. Valid values: true, false
seRenameInstNames	string	cellview		Default names applied to instances to be placed.
seRenameInstScalarize	bool	cellview	false	Default for scalarizing user-defined vector/vector bit names when expand is on. Valid values: true, false
seReportAutoExtractedChanges	bool	user	false	If true, SE connectivity engine report connectivity changes done during automatic of a design. Valid values: true, false

Table 188 *Schematic Editing Preferences (Continued)*

Name	Type	Scope	Default	Description
seSchDisplayAxes	bool	user	true	Draw axis lines at x = 0 and y = 0. Valid values: true, false
seSchDisplayGrid	bool	cellview	true	Enable display grid. Valid values: true, false
seSchDisplayGridBasis	string	cellview	snapGrid	Controls which set of preferences to use when drawing the grid. Valid values: snapGrid, userUnits
seSchematicMigrationAutoStubSize	bool	user	true	Create stub by auto size in Schematic Migration flow.
seSchematicMigrationCallbackParameterList	string	user		The Callback Parameter List of Schematic Migration flow.
seSchematicMigrationCellList	string	user		The Cell List to migrate in the Schematic Migration dialog box.
seSchematicMigrationCellListEnable	bool	user	false	Enable the Cell List in the Schematic Migration dialog box.
seSchematicMigrationCellMappingFile	string	user		The Cell Mapping file in the Schematic Migration dialog box.
seSchematicMigrationCellMappingMode	string	user	Schematic	Migration mode in the cell mapping tab on Schematic Migration dialog box.
seSchematicMigrationCellMappingModeEnable	string	user	0	Enable the Cell Mapping Mode in the Schematic Migration dialog box.
seSchematicMigrationCellSuffix	string	user	_new	The suffix of migrated cells in the Schematic Migration dialog box.
seSchematicMigrationCopyCellHierarchy	bool	user	false	Enable Copy Cell Hierarchy in the Schematic Migration dialog box.
seSchematicMigrationCopyCellHierarchyExcludeLibraries	string	user	analogLib sample basic sheets	The exclude libraries in the Schematic Migration Copy Cell Hierarchy flow.

Table 188 *Schematic Editing Preferences (Continued)*

Name	Type	Scope	Default	Description
seSchematicMigrationCopyCellHierarchyIncludeViews	string	user	*	The views to copy in the Schematic Migration Copy Cell Hierarchy flow.
seSchematicMigrationCopyCellHierarchySeparateLibraries	bool	user	false	Copy hierarchy into separate libraries in the Schematic Migration Copy Cell Hierarchy flow.
seSchematicMigrationCreateStubsonMappedExtraTerms	bool	user	false	Create wire stubs on extra target terminals if they have mapping in the Schematic Migration flow.
seSchematicMigrationCreateTargetInstances	bool	user	false	Destroy and create instance in the Schematic Migration flow.
seSchematicMigrationDeleteExtraWires	bool	user	false	Delete extra unconnected wires in target design.
seSchematicMigrationDeletePropertiesfromInstances	string	user	0	Delete the properties from instances in the Schematic Migration flow.
seSchematicMigrationDesignLibrary	string	user		The Design Library to migrate in the Schematic Migration Flow.
seSchematicMigrationDesignLibraryEnable	string	user	0	Enable the Design Library options on the Schematic Migration dialog box.
seSchematicMigrationDesignSuffix	string	user	_new	The suffix of migrated design library on the Schematic Migration dialog box.
seSchematicMigrationDestLibMode	string	user	single	The mode to control destination of localize library in the Schematic Migration Flow. Valid Values: single, multiple.
seSchematicMigrationDestLibSingleLib	string	user		The library name of local library when destination of local library is single library.
seSchematicMigrationDestLibSuffix	string	user	_local	The library name suffix of local libraries to store local library for each SourceLibs separately.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSchematicMigrationEntireLibrariesMultipleCells	string	user	Entire Libraries	Enable migrate entire libraries or multiple cells on Schematic Migration dialog box. Valid value is "Entire Libraries" and "Multiple Cells".
seSchematicMigrationExcludeLibraryList	string	user	analogLib samples basic sheets	The exclude libraries in the Schematic Migration flow.
seSchematicMigrationExcludeParameterList	string	user		The Exclude Parameter List of Schematic Migration flow.
seSchematicMigrationExtraTermToPropertyMapping	string	user		Map extra terms of source instance to property on target instance in the Schematic Migration flow. Defined as a list of {term propName} pairs.
seSchematicMigrationIgnoreParameterIfNoCallback	bool	user	false	Ignore the parameter if no callback in the Schematic Migration flow.
seSchematicMigrationKeepSourceSymbols	bool	user	false	When true, keep source symbols and create an extra level of hierarchy in local library. Valid Values: true, false.
seSchematicMigrationKeepUnmappedCells	bool	user	false	When true, keep unmapped cells into local library. Valid Values: true, false.
seSchematicMigrationLayerMappingFile	string	user		The layer mapping file of Schematic Migration flow.
seSchematicMigrationLayoutViewSearchList	string	user	layout	The view search list in Schematic Migration flow for Layout mode.
seSchematicMigrationLocalLibrarySourceLibrary	string	user		The SourceLibs that need to localize library in the Schematic Migration Flow.
seSchematicMigrationLogCreation	string	user	1	Generate the log in Schematic Migration flow.
seSchematicMigrationMappingClassFile	string	user		The Mapping Class File on the Schematic Migration dialog box.

Table 188 *Schematic Editing Preferences (Continued)*

Name	Type	Scope	Default	Description
seSchematicMigrationMappingFile	string	user		The file for saving/loading design migration settings on the Schematic Migration dialog box.
seSchematicMigrationModes	string	user	Schematic	Migration mode on the Schematic Migration dialog box. Valid value is schematic, layout and both.
seSchematicMigrationOpenReport	bool	user	false	Open report file after the Schematic Migration flow is finished.
seSchematicMigrationOverwriteCells	bool	user	false	Overwrite the existing cells in the Schematic Migration Flow.
seSchematicMigrationOverwriteDest Lib	bool	user	false	When true, overwrite local libraries in the Schematic Migration Flow. Valid Values: true, false.
seSchematicMigrationOverwriteLibrary	bool	user	false	Overwrite the Design Library in the Schematic Migration Flow.
seSchematicMigrationOverwriteNet	bool	user	false	When true, overwrites the existing nets in Schematic Migration Flow. Valid Values: true, false.
seSchematicMigrationPostRemasterProc	string	user		The procedure triggered after updating an instance in Schematic Migration. Procedure Signature: userProc <oalnst>.
seSchematicMigrationPreRemasterProc	string	user		The procedure triggered before updating an instance in Schematic Migration. Procedure Signature: userProc <oalnst>.
seSchematicMigrationRevertNotMappedParams	string	user	0	Revert the not mapped parameters in the Schematic Migration flow.
seSchematicMigrationSchematicViewSearchList	string	user	symbol hspice	The view search list in Schematic Migration flow for Schematic mode.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSchematicMigrationSelectCellsEnable	bool	user	false	Enable Select Cells on the Schematic Migration dialog box.
seSchematicMigrationSelectLibraryEnable	string	user	0	Enable the Select Library on the Schematic Migration dialog box.
seSchematicMigrationShowLayoutMigration	bool	user	false	Show Layout Migration related features.
seSchematicMigrationSourceStreamFile	string	user		The source stream file of Schematic Migration flow.
seSchematicMigrationSourceTechLib	string	user		The source technology library on the Schematic Migration dialog box box.
seSchematicMigrationStubSize	string	user	2	The stub size of Schematic Migration flow (valid value: 1, 2, 3, 5, 10)..
seSchematicMigrationTargetStreamFile	string	user		The target stream file of Schematic Migration flow.
seSchematicMigrationTargetTechLib	string	user		The target technology library on the Schematic Migration dialog box box.
seSchematicMigrationTechnologyLibrary	string	user		The technology library of Schematic Migration flow.
seSchematicMigrationTrigerCallbackUseOldMethod	bool	user	false	Use old method to trigger callback in the Schematic Migration flow.
seSchematicMigrationTriggerCallbackOnAllParams	string	user	0	Trigger callbacks on all parameters in the Schematic Migration flow.
seSchematicMigrationUpdateTargetLibRefs	bool	user	true	Update references in target library.
seSchematicMigrationViaMapping	string	user	0	Enable via mapping in the Schematic Migration flow.
seSchematicMigrationWireStretchAvoidSymbols	bool	user	true	Avoiding symbols when stretch the wiles in the Schematic Migration flow.
seSchematicPhotonicPin	bool	cellview	false	Default for photonic pin creation. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSchematicPinAngle	string	cellview	anyAngle	Default angle mode for array creation. Valid values: orthogonal, diagonal, anyAngle
seSchematicPinColor	string	cellview		Default pin label color.
seSchematicPinCreationMode	seSchematicPinCreationMode	cellview	singlePin	Default for indicating how to create multiple pins.
seSchematicPinExpand	bool	cellview	false	Default for expanding user entered vector names. Valid values: true, false
seSchematicPinFont	string	cellview	stick	Default pin label font. Valid values: euroStyle, gothic, math, roman, script, stick, fixed, swedish, milSpec
seSchematicPinHeight	float	cellview	0.0625	Default pin label height.
seSchematicPinInput	string	user	basic ipin symbol	Library/cell/view to use when creating a pin of type input.
seSchematicPinInputJust	string	cellview	<default>	Default input. Valid values: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight, <default>
seSchematicPinInputOutput	string	user	basic iopin symbolr	Library/cell/view to use when creating a pin of type inputOutput.
seSchematicPinInputOutputJust	string	cellview	<default>	Default inputOutput pin label justification. Valid values: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight, <default>
seSchematicPinJumper	string	user	basic iopin symbolr	Library/cell/view to use when creating a pin of type jumper.
seSchematicPinName	string	cellview		Default schematic pin names.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSchematicPinOpticalInput	string	user	photonicLib inOpto symbol	To use when creating an optical pin of type input.
seSchematicPinOpticalInputOutput	string	user	photonicLib inoutOpto symbol	To use when creating an optical pin of type inputOutput.
seSchematicPinOpticalOutput	string	user	photonicLib outOpto symbol	To use when creating an optical pin of type output
seSchematicPinOpticalSwitch	string	user	photonicLib inoutOpto symbol	To use when creating an optical pin of type switch
seSchematicPinOpticalTristate	string	user	photonicLib inoutOpto symbol	To use when creating an optical pin of type tristate
seSchematicPinOrientation	string	cellview	R0	Default orientation. Valid values: R0, R90, R180, R270, MX, , MXR90, MYR90
seSchematicPinOutput	string	user	basic opin symbol	Library/cell/view to use when creating a pin of type output.
seSchematicPinOutputJust	string	cellview	<default>	Default output pin label justification. Valid values: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight, <default>
seSchematicPinOverbar	bool	cellview	false	Default pin label overbar setting. Valid values: true, false
seSchematicPinSwitch	string	user	basic iopin symbolr	Library/cell/view to use when creating a pin of type switch.
seSchematicPinSwitchJust	string	cellview	<default>	Default switch pin label justification. Valid values: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight, <default>

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSchematicPinTextAttributes	bool	cellview	false	Expand state of text attributes. Valid values: true, false
seSchematicPinTristate	string	user	basic iopin symbolr	Library/cell/view to use when creating a pin of type tristate.
seSchematicPinTristateJust	string	cellview	<default>	Default tristate pin label justification. Valid values: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight, <default>
seSchematicPinType	string	cellview	input	Default schematic pin type. Valid values: input, output, inputOutput, switch, tristate
seSchematicPinUnused	string	user	basic iopin symbolr	Library/cell/view to use when creating a pin of type unused.
seSchGravity	bool	cellview	false	Enables cursor gravity. Valid values: true, false
seSchGravityDepth	int	cellview	0	Hierarchy depth where the gravity works.
seSchGravityRadius	int	cellview	15	Distance from cursor where gravity is active.
seSchGravityRegionCenter	bool	cellview	false	Enables the snap to partial objects. Valid values: true, false
seSchGravityRegionCenterline	bool	cellview	false	Enables the snap to partial objects. Valid values: true, false
seSchGravityRegionCenterlineVertex	bool	cellview	true	Enables the snap to partial objects. Valid values: true, false
seSchGravityRegionEdge	bool	cellview	false	Enables the snap to partial objects. Valid values: true, false
seSchGravityRegionVertex	bool	cellview	true	Enables the snap to partial objects. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSchMajorGridMult	int	cellview	8	Major grid dots as multiple of snap grid.
seSchMajorGridUnits	float	cellview	1	Major grid dots in terms of user units.
seSchMinorGridMult	int	cellview	2	Minor grid dots as multiple of snap grid.
seSchMinorGridUnits	float	cellview	0.125	Minor grid dots in terms of user units.
seSchPropertyEditorParameterFilter	string	user	hspiceD	Specifies the used in the Parameter Filter dialog box for schematic window type.
seSchSelectCriteria	string	cellview	enclose	Touch/enclose mode for selection command. Valid values: touch, enclose
seSchSnapOffset	unknown	cellview	0 0	Snap grid offset for schematic.
seSchSnapSpacing	unknown	cellview	0.0625 0.0625	Snap grid spacing for schematic.
seSchSnapSpacingMatch	bool	cellview	true	Constrains user interfaces modifying <code>seSchSnapSpacingMatch</code> . See <code>leSnapSpacingMatch</code> in the Preferences for more information. Valid values: true, false
seSelectionShapeAngleMode	string	cellview	anyAngle	Indicates the angle used when interactively creating a selection shape of type polygon. Valid values: orthogonal, diagonal, anyAngle
seSelectionShapeOverwrite	bool	cellview	true	Indicates whether all selection shapes should be removed before a new shape is added. Valid values: true, false
seSelectionShapeType	string	cellview	rectangle	Indicates the current selection shape type to construct. Valid values: polygon, rectangle

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSelectionTableAttributes	string	user	Count	Attributes (columns) which are exposed in the Selection Table assistant (schematic).
seSelectMode	string	cellview	full	Switches the selection mode to full or partial. Valid values: full, partial
seSelectRadius	int	cellview	0	Distance from cursor to look for active objects.
seSelectRegionEdge	bool	cellview	true	Enables the activation of partial objects. Valid values: true, false
seSelectRegionPin	bool	cellview	true	Enables the activation of partial objects. Valid values: true, false
seSelectRegionVertex	bool	cellview	true	Enables the activation of partial objects. Valid values: true, false
seShadowSelection	bool	user	false	Shadow selection sets (schematic). Valid values: true, false
seShapeAngleMode	string	cellview	anyAngle	Indicates the angle used when interactively creating a shape. Valid values: orthogonal, diagonal, anyAngle
seShapeColor	string	cellview		Indicates color to use when creating a shape.
seShapeLineStyle	string	cellview		Indicates line style to use when creating a shape.
seShapeStipple	string	cellview	Any	Indicates stipple to use when creating a shape.
seShapeType	string	cellview	rectangle	Indicates the current shape type to construct. Valid values: line, polygon, rectangle, circle, arc, ellipse
seShapeWidth	float	cellview	0	Indicates the width of the path.
seShowDynamicNetNames	bool	user	false	Controls whether net names should be visible in viewport. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seShowIgnoreInstInfo	bool	user	false	Specifies whether to display ignored instance information. Valid values: true, false
seShowLabelAttachments	bool	cellview	false	Controls whether parent/child lines in schematics are shown or hidden. Valid values: true, false
seShowMarkers	bool	user	true	Controls whether OA markers are shown in the Schematic and Symbol Editors. Valid values: true, false
seShowMissedPinLabels	bool	user	false	Controls whether missed pin labels should be drawn in viewport. Valid values: true, false
seShowPinAsConnected	bool	user	false	Whether or not connected pins use packet "pinConn" rather than "pin". Valid values: true, false
seShowProbeWithX	bool	user	false	If true, fixed size cross is used to highlight small objects in the Schematic Editor. Valid values: true, false
seShowProbeWithXSize	int	user	10	Shape size threshold, less than which the highlighting is done by cross.
seSnapCursorShape	string	user	square	Sets the cursor shape in the Schematic Editor. Valid values: square, cross
seSolderDotSize	float	cellview	0.03125	Describes the radial extension beyond the widest path for defining the radius of the solder dot.
seStopLevel	int	cellview	100	Controls the stop level inside the schematic view.
seStretchAngleMode	string	cellview	anyAngle	Indicates the angle mode for the interactive stretch command. Valid values: orthogonal, diagonal, anyAngle

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seStretchDialogMode	bool	user	false	ise::stretch command options dialog eject state. Valid values: true, false
seStretchOrigin	string	cellview	anchor	Indicates what origin to use when rotating. Valid values: anchor, center
seStretchRoutesDisplayed	int	user	5	Indicates maximum number of stretched wire.
seStubOpticalNoConnRef	lcv	cellview	<file path>	used to instantiate no connection symbol at the end of a stub object .
seStubPinType	string	cellview	none	Indicates the type of pin which is instantiated at the end of each created stub.
seStubUpdateExisting	bool	cellview	true	Update existing stubs for missed pins and labels. Valid values: true, false
seSymbolDesignNavigatorTerminals View	string	context	se_symbol_all	Sub-view to use in the Terminals tab of the Symbol Editor Design Navigator.
seSymbolGenOptionSetDefault	string	user	default	Indicates the name of the option set to use by default when using the built-in generateSymbol command.
seSymbolOpticalPinColor	string	cellview	white	Default optical pin label color.
seSymbolPhotonicPin	bool	cellview	false	Default for photonic pin creation. Valid values: true, false
seSymbolPinAngle	string	cellview	anyAngle	Default angle mode for array creation. Valid values: orthogonal, diagonal, anyAngle
seSymbolPinArray	bool	cellview	false	Default for indicating how to create multiple entered names. Valid values: true, false
seSymbolPinColor	string	cellview		Default pin label color.
seSymbolPinDefaultNet	string	cellview		Default net name for parameterized connection.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSymbolPinExpand	bool	cellview	false	Default for expanding user-defined vector names. Valid values: true, false
seSymbolPinFont	string	cellview	stick	Default pin label font. Valid values: euroStyle, gothic, math, roman, script, stick, fixed, swedish, milSpec
seSymbolPinHeight	float	cellview	0.0625	Default pin label height.
seSymbolPinInput	string	user	basic ipin symbol	To use when creating a pin of type Input.
seSymbolPinInputJust	string	cellview	<default>	Default input pin label justification. Valid value: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight, <default>
seSymbolPinInputOutput	string	user	basic iopin symbolr	To use when creating a pin of type InputOutput.
seSymbolPinInputOutputJust	string	cellview	<default>	Default inputOutput pin label justification. Valid value: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight, <default>
seSymbolPinJumper	string	user	basic iopin symbolr	Library/cell/view to use when creating a pin of type jumper.
seSymbolPinName	string	cellview		Default symbol pin names.
seSymbolPinOpticalInput	string	user	photonicLib inOpto symbol	To use when creating a pin of type Input.
seSymbolPinOpticalInputOutput	string	user	photonicLib inoutOpto symbol	To use when creating a pin of type inputOutput.
seSymbolPinOpticalOutput	string	user	photonicLib outOpto symbol	To use when creating a pin of type output.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSymbolPinOpticalSwitch	string	user	photonicLib inoutOpto symbol	To use when creating a pin of type switch.
seSymbolPinOpticalTristate	string	user	photonicLib inoutOpto symbol	To use when creating a pin of type tristate.
seSymbolPinOrientation	string	cellview	R0	Default orientation. Valid value: R0, R90, R180, R270, MX, , MXR90, MYR90
seSymbolPinOutput	string	user	basic opin symbol	To use when creating a pin of type Output.
seSymbolPinOutputJust	string	cellview	<default>	Default output pin label justification. Valid value: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight, <default>
seSymbolPinOverbar	bool	cellview	false	Default pin label overbar setting. Valid values: true, false
seSymbolPinParameterizedConnection	bool	cellview	false	Expand state of parameterized connection group box. Valid values: true, false
seSymbolPinParameterName	string	cellview		Default parameterized connection name.
seSymbolPinShape	string	cellview	square	Default symbol pin shape. Valid value: square, circle, triangle, polygon
seSymbolPinSwitch	string	user	basic iopin symbolr	To use when creating a pin of type Switch.
seSymbolPinSwitchJust	string	cellview	<default>	Default switch pin label justification. Valid value: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight, <default>

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSymbolPinTextAttributes	bool	cellview	false	Expand state of text attributes group box. Valid values: true, false
seSymbolPinTristate	string	user	basic iopin symbolr	To use when creating a pin of type Tristate.
seSymbolPinTristateJust	string	cellview	<default>	Default tristate pin label justification. Valid value: upperLeft, centerLeft, lowerLeft, upperCenter, centerCenter, lowerCenter, upperRight, centerRight, lowerRight, <default>
seSymbolPinType	string	cellview	input	Default symbol pin type. Valid values: input, output, inputOutput, switch, tristate
seSymbolPinUnused	string	user	basic iopin symbol	Library/cell/view to use when creating a pin of type unused.
seSymDisplayAxes	bool	user	true	Draw axis lines at x = 0 and y = 0. Valid values: true, false
seSymDisplayGrid	bool	cellview	true	Enables display grid. Valid values: true, false
seSymDisplayGridBasis	string	cellview	snapGrid	Controls which set of preferences to use when drawing the grid. Valid values: snapGrid, userUnits
seSymGravity	bool	cellview	false	Enables cursor gravity. Valid values: true, false
seSymGravityDepth	int	cellview	0	Hierarchy depth where the gravity works.
seSymGravityRadius	int	cellview	15	Distance from cursor where gravity is active.
seSymGravityRegionCenter	bool	cellview	true	Enables the snap to partial objects. Valid values: true, false
seSymGravityRegionCenterline	bool	cellview	true	Enables the snap to partial objects. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seSymGravityRegionCenterlineVertex	bool	cellview	true	Enables the snap to partial objects. Valid values: true, false
seSymGravityRegionEdge	bool	cellview	true	Enables the snap to partial objects. Valid values: true, false
seSymGravityRegionVertex	bool	cellview	true	Enables the snap to partial objects. Valid values: true, false
seSymMajorGridMult	int	cellview	8	Major grid dots as multiple of snap grid.
seSymMajorGridUnits	float	cellview	1	Major grid dots in terms of user units.
seSymMinorGridMult	int	cellview	2	Minor grid dots as multiple of snap grid.
seSymMinorGridUnits	float	cellview	0.125	Minor grid dots in terms of user units.
seSymPropertyEditorParameterFilter	string	user	hspiceD	Specifies the used in the Parameter Filter dialog box for symbol window type.
seSymSelectCriteria	string	cellview	enclose	Touch/enclose mode for selection command. Valid values: touch, enclose
seSymSnapOffset	unknown	cellview	0 0	Snap grid offset for symbol.
seSymSnapSpacing	unknown	cellview	0.0625 0.0625	Snap grid spacing for symbol. Valid values: pin, graphics
seSymSpecifyGraphicsSnapSpacing	seGraphicSnapSpacingSpecification	cellview	ratio	Indicates the symbol design graphical snap spacing specification way from user interface: by value or by ratio.
seSymSnapSpacingMatch	bool	cellview	true	Constrains user interfaces modifying <code>seSymSnapSpacingMatch</code> . See <code>leSnapSpacingMatch</code> in the Preferences for more information. Valid values: true, false
seTemplateAssistantLibCellFilter	string	user		List of libraries and cells to Search for template matches.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seTemplateAssistantLibs	string	user		List of libraries to search for template matches.
seTemplateAssistantNameFilter	string	user		List of cells to Search for template matches.
seTemplateAssistantShowAnnotations	bool	user	false	Controls whether color coded annotations are displayed for template matches. Valid values: true, false
seTemplateAssistantStaticTemplate	bool	user	false	Determines whether a template intent constraint is intended to be Static or Elastic. Valid values: true, false
seTemplateAssistantUseBuiltIn	bool	user	false	Controls whether or not built-in circuit structures with known functionalities are searched during template match operations, or whether a user-specified list of Template libraries/cells are searched instead. Valid values: true, false
seTemplateAssistantUseGeometry	bool	user	true	Controls whether or not geometrical constraints such as schematic device alignments are used to improve template matching accuracy. Valid values: true, false
seTemplateMatchMembersInRectangle	string	user	true	Instructs to enclose template match member instances in a rectangle.
seTemplateNoteColor	string	user	white	The color used for creating template notes.
seTemplateNoteFont	string	user	stick	The OA font name used for creating template match notes.
seTemplateNoteFontHeight	string	user	10	The font height used for creating template notes.
seTemplateNoteShapeColor	string	user	white	The color used for creating bounding box for template matches.

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seTemplateNoteShapeLineStyle	string	user	dots	The line style used for creating bound box for template matches.
seTextDrawFilter	int	user	0	Text object drawing filter.
seUpdateEstParasiticsSaveAndClose	bool	user	true	Save and close design after updating estimated parasitics. Valid values: true, false
seUpdateEstParasiticsScope	string	user	0x	The scope for updating estimated parasitics.
seUseConstraintCellView	bool	user	true	If set, stores schematic constraints in a cellView named according to seConstraintViewName. Valid values: true, false
seUseErrorViewer	bool	user	true	Specifies whether to use error viewer in schematic/symbol window. Valid values: true, false
seUseStopNlActionOnInstance	bool	user	true	Controls whether the value stop of instance's property nlAction is considered in hierarchy traversing. Valid values: true, false
seVDRCAssistantShowFilter	bool	context	false	Controls whether the filter fields are shown in the VDRC Assistant. Valid values: true, false
seVDRCDisplayParamPrecision	int	user	5	Parameter Display Precision in Gradient Editor.
seVDRCHeatMapSetupValues	string	user	{1 Difference { } {volt_max-volt_min min max 10 density0 density100} density0}	Save VDRC Heat Map setup in the form of a Tcl dictionary.
seVDRCSEAssistantShowVflagColumn	bool	user	false	Shows/Hides the Vflag column in the Nets table of SE VDRC Assistant. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seViewSearchList	string	context	schematic symbol	Default switch list for design navigation in Schematic Editor.
seWireColorizationDef	string	cellview		Default for name expansion.
seWireColorizationNetNamePref	string	cellview		Default color setting for wire name objects.
seWireColorizeApplyMode	bool	cellview	false	Default apply-by-name mode for wire colorization. Valid values: true, false
seWireNameArray	bool	cellview	false	Default for polyline mode. Valid values: true, false
seWireNameColor	string	cellview	<auto>	Default color setting for wire name objects.
seWireNameExpand	bool	cellview	false	Default for name expansion. Valid values: true, false
seWireNameFont	string	cellview	stick	Default font for wire name objects. Valid values: euroStyle, gothic, math, roman, script, stick, fixed, swedish, milSpec
seWireNameHeight	float	cellview	0.0625	Default height for wire name objects.
seWireNameName	string	cellview		Default names for wire name objects.
seWireNameOverbar	bool	cellview	false	Default overbar setting for wire name objects. Valid values: true, false
seWireNameTextAttributes	bool	cellview	false	Expand state of text attributes group box. Valid values: true, false
seWireNameVerticalBundleDisplay	bool	cellview	false	Displays bundle net names as vertical.
seWiringAuxCursor	bool	user	true	Indicates whether auxiliary snap cursor is used for wiring. Valid values: true, false

Table 188 Schematic Editing Preferences (Continued)

Name	Type	Scope	Default	Description
seWiringAuxCursorShape	string	user	{{0.0125 0.0125} {0.0375 0.0375}} {{-0.0375 0.0375} {-0.0125 0.0125}} {{-0.0375 -0.0375} {-0.0125 -0.0125}} {{0.0375 -0.0375} {0.0125 -0.0125}}	Points for auxiliary snap cursor shape.
seWiringAvoidShorts	bool	cellview	true	Indicates whether name shorts are avoided when determining a route. Valid values: true, false
seWiringAvoidSymbols	bool	cellview	true	Indicates whether to avoid symbols when creating a route. Valid values: true, false
seWiringColor	string	cellview	<auto>	Default color for created wires.
seWiringCycleName	bool	cellview	true	Defines whether first net name in the net namefield is removed or remains after object is committed. Valid values: true, false
seWiringExpand	bool	cellview	false	Defines net name expansion. Valid values: true, false
seWiringMode	string	cellview	auto	Type of wiring to perform. Valid values: auto, guided, orthogonal, diagonal, anyAngle
seWiringNetName	string	cellview		Default net names for the created wires.
seWiringSingleSegment	bool	user	false	Control of Diagonal and Orthogonal modes to use only one segment at a time or do a complex route. Valid values: true, false

Table 188 *Schematic Editing Preferences (Continued)*

Name	Type	Scope	Default	Description
seWiringTapBuses	bool	cellview	false	Default mode for tapping buses during wiring start/commit. Valid values: true, false
seWiringType	string	cellview	<auto>	Indicates whether the wire to create should be thin, fat or automatically determined. Valid values: fat, thin, <auto>

23

Simulation and Analysis Environment Scripting Tcl

This chapter lists the commands, enumerations, and objects in the Custom Compiler Tcl ss namespace. (This namespace has no preferences.) This namespace provides Tcl for PrimeWave™ Design Environment scripting.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)

Commands

Interactive commands are listed after non-interactive commands in the Commands section.

The commands covered are:

- [ss::addGroupDefItem](#)
- [ss::addModelGroupDefFile](#)
- [ss::backAnnotateParasitics](#)
- [ss::beginCalibration](#)
- [ss::createAging](#)
- [ss::createAnalysis](#)
- [ss::createCorner](#)
- [ss::createGlobalModelGroup](#)
- [ss::createGroup](#)

- `ss::createGroupDef`
- `ss::createGroupSection`
- `ss::createModelGroup`
- `ss::createModelGroupDef`
- `ss::createModelGroupSection`
- `ss::createObject`
- `ss::deleteDuplicateCorners`
- `ss::deleteModelGroupDefFile`
- `ss::disable`
- `ss::editOutputs`
- `ss::enable`
- `ss::endCalibration`
- `ss::expr`
- `ss::findIncludeGroupDef`
- `ss::findModelGroupDef`
- `ss::findStimulusGroupDef`
- `ss::findVariableGroupDef`
- `ss::foreachSignal`
- `ss::generateCorners`
- `ss::getActiveSession`
- `ss::getActiveTestbench`
- `ss::getAnalyses`
- `ss::getCorners`
- `ss::getModelGroups`
- `ss::getOutputs`
- `ss::getTestbenches`
- `ss::listAnalyses`

- [ss::listCorners](#)
- [ss::listDesVars](#)
- [ss::listEnvOptions](#)
- [ss::listIncludeFiles](#)
- [ss::listMacros](#)
- [ss::listModelFiles](#)
- [ss::listModelGroupDefs](#)
- [ss::listModelGroups](#)
- [ss::listNodes](#)
- [ss::listOutputs](#)
- [ss::listResults](#)
- [ss::listSaveOptions](#)
- [ss::listSignals](#)
- [ss::listSimOptions](#)
- [ss::listStimulusSets](#)
- [ss::netlist](#)
- [ss::newSession](#)
- [ss::newTestbench](#)
- [ss::plot](#)
- [ss::print](#)
- [ss::printMeasurementSummary](#)
- [ss::printNoiseSummary](#)
- [ss::printTable](#)
- [ss::readOutputs](#)
- [ss::readTable](#)
- [ss::releaseNode](#)
- [ss::run](#)

- [ss::saveResults](#)
- [ss::selectDataDir](#)
- [ss::selectResults](#)
- [ss::setActiveNets](#)
- [ss::setActiveSession](#)
- [ss::setActiveTestbench](#)
- [ss::setAnalysisOptions](#)
- [ss::setDesign](#)
- [ss::setDesVars](#)
- [ss::setDeviceParameters](#)
- [ss::setEnvOptions](#)
- [ss::setGlobalDeviceParameters](#)
- [ss::setGlobalModelFiles](#)
- [ss::setGlobalTemperature](#)
- [ss::setGlobalVariables](#)
- [ss::setIncludeFiles](#)
- [ss::setJobOptions](#)
- [ss::setModelFiles](#)
- [ss::setNode](#)
- [ss::setPostRunProcs](#)
- [ss::setPreRunProcs](#)
- [ss::setSaveOptions](#)
- [ss::setSimOptions](#)
- [ss::setSimulator](#)
- [ss::setStimulusSet](#)
- [ss::setTemperature](#)
- [ss::setTestbenchName](#)

- [ss::simulateCalibration](#)
- [ss::trackActiveTestbench](#)

ss::addGroupDefItem

Add A Group Definition item.

Syntax

```
ss::addGroupDefItem <name: string> -item <string> [-sections  
  <string>] [-groupDef <saIncludeGroupDef | saStimulusGroupDef  
  | saVariableGroupDef>] -type <string> [-includeType <string>]  
  [-fileSection <string>]
```

Returns

<saVariableGroupDef | saIncludeGroupDef | saStimulusGroupDef> :

Arguments

name

Name of the group definition

-item

Item to add to this group

-sections

sections

-groupDef

group definition

-type

group def type. Could be either 1.variable 2. include 3. stimulus

-includeType

include type.

-fileSection

include file section.

ss::addModelGroupDefFile

Description

Adds a Model Group Definition file.

Syntax

```
ss::addModelGroupDefFile <name: string> -file <string> [-enabled <bool>]  
[-fileSections <string>]
```

Arguments

name

Name of the model group definition.

-file

Model file to be added to this group.

-enabled

Enabled state for this model file.

-fileSections

File sections.

Returns

<[saModelGroupDef]> :

ss::backAnnotateParasitics

Description

Adds a file or cellview for parasitic back annotation simulation.

Syntax

```
ss::backAnnotateParasitics <file: dmCellView | string> [-options  
<string>]
```

Arguments

file

The path to a parasitic netlist, or a cellview containing a parasitic extracted Spice text file.

-options

Simulator specific options for backannotating the file.

Returns

<saObject>

Errors

- If the specified options are not valid for the current simulator.
- If the specified cellview is not a SPICEText view type.

Examples

Example 1

```
ss::backAnnotateParasitics /home/billy/design/input.dspf
```

Example 2

```
ss::backAnnotateParasitics [dm::findCellView dspf -libName design  
-cellName PLL] \  
-options {  
minC 10pf  
enabled false  
}
```

ss::beginCalibration

Prepare environment for calibration simulation in prerun.

Syntax

```
ss::beginCalibration [<testbench: saTestbench>] [-disableAnalysis  
<string>]
```

Returns

<string> :

Arguments

testbench

The testbench to run calibration.

-disableAnalysis

Remove all analysis in testbench for calibration simulation. (Default is: 0)

ss::createAging

Description

Sets AGING analysis.

Syntax

```
ss::createAging [-options <string>]
```

Arguments

`-options`

Aging options.

Returns

`<none>`

ss::createAnalysis

Description

Creates a new analysis. Additional analysis type specific options can be specified via the `-options` argument.

Syntax

```
ss::createAnalysis <type: saAnalysisDef | string> [-options <string>]  
[-name <string>]
```

Arguments

`type`

The analysis type name.

`-options`

The list of analysis specific option names and values. (Default: {})

`-name`

The name of the new analysis. If nothing is specified, a unique name is generated based on the name of the analysis type. (Default: {})

Returns

The newly created analysis (`saAnalysis`).

Errors

- The specified analysis type is not valid for the SAE Script's current simulator.
- The name of one or more options is not valid for the given analysis type. The list of valid attribute names is included in the error message.

Note:

The integration dynamically determines the valid list of attributes at runtime.

ss::createCorner

Description

Creates a new corner.

Syntax

```
ss::createCorner <name: string> [-params <string>] [-models <string>]  
[-includes <string>]
```

Arguments

name

Name of the corner.

-params

List of name-value pairs for each parameter.

-models

Model file specification for the corner. For example, `{{models.inc}}`,
`{{single_corner.lib TT}}`, or `{{multiple_corners.lib {TT FF SS}}}`.

-includes

List of include file parameter names or file names.

Returns

<saCorner>)

ss::createGlobalModelGroup

Description

Creates a model group.

Syntax

```
ss::createGlobalModelGroup <name: string> -nominalSection <string>  
[-enabled <bool>]
```

Arguments

name

Filter model groups by name.

-nominalSection

Name of the nominal section.

`-enabled`

Enable status for the model file. (Default: true)

Returns

`<[saModelGroup]>` :

ss::createGroup

Create a Group of the given type.

Syntax

```
ss::createGroup <name: string> -nominalSection <string> [-enabled <bool>]  
-type <string>
```

Returns

`<saVariableGroup | saIncludeGroup | saStimulusGroup>` :

Arguments

`name`

Filter groups by name

`-nominalSection`

Nominal section name

`-enabled`

Enable status (Default is: true)

`-type`

type of the group

ss::createGroupDef

Create A Group Definition.

Syntax

```
ss::createGroupDef <name: string> [-items <string>] [-createGroup <bool>]  
-type <string>
```


Returns

<saVariableGroupDef | saIncludeGroupDef | saStimulusGroupDef> :

Arguments

name

Filter groups by name

-items

Items to add to this group

-createGroup

Create the group

-type

The type of the group to create

ss::createGroupSection

Create a Group Section.

Syntax

```
ss::createGroupSection <name: string> -groupDefName <string> -sections  
<string> [-groupDef <saIncludeGroupDef | saStimulusGroupDef |  
saVariableGroupDef>] -type <string>
```

Returns

<saVariableGroup | saIncludeGroup | saStimulusGroup> :

Arguments

name

Name of the Group Section

-groupDefName

Group Definition

-sections

Sections

-groupDef

The target group definition object you want to create section with

`-type`

The type of the group definition

ss::createModelGroup

Description

Creates a model group.

Syntax

```
ss::createModelGroup <name: string> -nominalSection <string> [-enabled  
    <bool>]
```

Arguments

`name`

Filter model groups by name.

`-nominalSection`

Name of the nominal section.

`-enabled`

Enable status for the model file. (Default: true)

Returns

<[saModelGroup]> : Model Group.

ss::createModelGroupDef

Description

Creates a model group definition.

Syntax

```
ss::createModelGroupDef <name: string> -includePath <string> [-files  
    <string>] [-createGroup <bool>] [-base <bool>]
```

Arguments

`name`

Filter model groups by name.

`-includePath`

The include file path.

`-files`

Model files to be added to this group.

`-createGroup`

Create the model group.

`-base`

Specify the base model group definition.

Returns

`<[saModelGroupDef]>`: Model Group Definition.

ss::createModelGroupSection

Description

Creates a model group section.

Syntax

```
ss::createModelGroupSection <name: string> -modelGroupDef <string>  
-fileSections <string> [-setNominal <bool>]
```

Arguments

`name`

Name of model group section.

`-modelGroupDef`

Model group definition.

`-fileSections`

File sections.

`-setNominal`

Set as nominal section for the appropriate testbench or session.

Returns

`<[saModelGroup]>` : Model Group Section.

ss::createObject

Description

Creates a new object.

Syntax

```
ss::createObject [-attributes <string>]
```

Arguments

`-attributes`

Attributes of the object.

Returns

<saObject>.

ss::deleteDuplicateCorners

Description

Deletes any duplicate corners. A corner is considered duplicate if the list of parameter values and models match with another corner.

Syntax

```
ss::deleteDuplicateCorners
```

Returns

<none>

ss::deleteModelGroupDefFile

Description

Deletes a model group definition file.

Syntax

```
ss::deleteModelGroupDefFile <name: string> -file <string>
```

Arguments

`name`

Name of the model group definition.

`-file`

Model file to delete from this group.

Returns

<[saModelGroupDef]>

ss::disable

Description

Disables the passed in objects. Subsequent ss::run or ss::netlist commands will ignore the disabled objects.

The ss::disable command supports the following shortcuts for specifying objects by name:

- `regexp` pattern
For example, `ss::disable ^tran.*` disables all objects with the names that begins with "tran".
- `corners`: Matches all corners
- `monteCarlo`: Specifies the Monte Carlo analysis
- `stimulusSets`: Matches all stimulus sets

Syntax

```
ss::disable <object: [dbCollection | saAnalysis | saCorner |  
saCornersSetup | saIncludeFile | saModelFile | saMonteCarlo | saOutput |  
saStimulusSets | saSweep | saTestbench | string]>
```

Arguments

`object`

The object(s) or object name/pattern to disable.

Returns

<none>

ss::editOutputs

Description

Allows you to edit the PrimeWave™ Design Environment main window's Outputs pane using the favorite text editor (for example, vim, emacs), according to the value set in the environment variable `EDITOR`.

At invocation, a text editor window with a header (ignored by the PrimeWave™ Design Environment when the file is read back) is brought up. User provided information must be formatted according to the rules listed in the header (refer the example given below). Delimiter character value can be selected by you, by setting the value of the PrimeWave™

Design Environment preference named `saTableExternalEditionSeparator`. The default value is `;` (as shown in the following example).

Example:

```
#File format:
# - Column delimiter character: ';'
# - Mandatory columns: Name, Expression
# - Optional columns: [Analyses="", Plot=1, Save=1]
#
# - Each line in the file represents a row in the table
# - No user comments are allowed
# - Columns are discriminated by looking for delimiter characters
# - Optional columns can be omitted, default values are assumed
# - 'Name' column can be blank but 'Expression' cannot
# - Column order is as shown in the next line
#Name;Expression;Analyses;Plot;Save
```

Syntax

```
ss::editOutputs [-filePath <string>]
```

Arguments

`-filePath`

File path to save the list.

Returns

<none>

ss::enable

Description

Enables the passed in objects. The `ss::enable` command also supports the following shortcuts for specifying objects by name:

- `regexp` pattern

For example, the `ss::enable ^tran.*` command enables all objects with the names beginning with "tran".

- `corners`: Matches all corners
- `monteCarlo`: Specifies the Monte Carlo analysis
- `stimulusSets`: Matches all stimulus sets

Syntax

```
ss::enable <object: [dbCollection | saAnalysis | saCorner |  
saCornersSetup | saIncludeFile | saModelFile | saMonteCarlo | saOutput |  
saStimulusSets | saSweep | saTestbench | string]>
```

Arguments

object

The object names or patterns to disable.

Returns

<none>

ss::endCalibration

Clean environment for calibration simulation in prerun.

Syntax

```
ss::endCalibration [-keepPrerun <string>]
```

Returns

<string> :

Arguments

-keepPrerun

Keep prerun directory after calibration simulation. (Default is: 0)

ss::expr

Description

Calls `wf::expr` with the current selected results. This command is not exported. It is done intentionally to avoid redeclaring the `expr` command when namespace import `ss::*` is used.

Syntax

```
ss::expr <expression: string | wfSignal> [-analysis <saAnalysis |  
string>] [-resultFile <string>] [-name <string>] [-params <string>]
```

Arguments

expression

The expression to evaluate.

`-analysis`

Limit expression to the given analysis name, type, or object.

`-name`

An output name that can be referenced by subsequent expressions. (Default: "")

`-resultFile`

Use data from the given result file.

`-params`

The default parameter conditions for the data access that functions in the expression.

Returns

The result of the expression. ({<string> | [wfSignal](#)})

Errors

- Any error in the input syntax.
- Referenced signals or results do not exist.

ss::findIncludeGroupDef

Returns the named include group definition.

Syntax

```
ss::findIncludeGroupDef <name: string>
```

Returns

<[saInlcudeGroupDef]> :

Arguments

`name`

Name of include group definition to get

ss::findModelGroupDef

Description

Returns the named model group definition.

Syntax

```
ss::findModelGroupDef <name: string>
```

Arguments

name

Name of the model group definition to get.

Returns

<[saModelGroupDef]>

ss::findStimulusGroupDef

Returns the named stimulus group definition.

Syntax

```
ss::findStimulusGroupDef <name: string>
```

Returns

<[saStimulusGroupDef]> :

Arguments

name

Name of stimulus group definition to get

ss::findVariableGroupDef

Returns the named variable group definition.

Syntax

```
ss::findVariableGroupDef <name: string>
```

Returns

<[saVariableGroupDef]> :

Arguments

name

Name of variable group definition to get

ss::foreachSignal

Description

Iterates over every signal in the active results.

Syntax

```
ss::foreachSignal <varName: string> <script: string>
```

Arguments

`varName`

The name of the variable to store each signal within.

`script`

The script to execute for each signal.

Returns

<none>

ss::generateCorners

Description

Generates corners by cross-product for the given parameter ranges.

Syntax

```
ss::generateCorners <prefix: string> [-params <string>] [-models  
  <string>] [-includes <string>] [-replace <bool>]
```

Arguments

`prefix`

Corner name prefix.

`-params`

List of parameter name or range pairs. Requires a Tcl list with an even number of elements, consisting of alternating design variable names and values. A value can be a list of values, or a sweep specifier using the `start:step:stop` syntax.

`-models`

List of model files or model groups, and optional sections. Requires a list, where each element is a list of one or two elements. If an element is a pair, the first is the model file and the second is a list of the sections to be included.

`-includes`

List of include file parameter names or file names.

`-replace`

Replace existing corners. (Default: false)

Returns

<none>

Examples

Example 1

In this example, the `ss::generateCorners` command creates 15 corners using the prefix `C` and varying the values of design variables `vdd` and `temp`.

```
ss::generateCorners C -params { k 1.2 vdd {1.0 1.2 1.3} temp {27:1:32} }
```

Example 2

In this example, the `ss::generateCorners` command creates 3 corners using the prefix `corner` and varying the section included from model 3.1.

```
ss::generateCorners corner -models { model1.1 {model2.1 TT}/  
{model3.1 {SS TT FF}} }
```

ss::getActiveSession

Description

Gets the active PrimeWave™ Design Environment script session.

Syntax

```
ss::getActiveSession
```

Returns

The active session (<string>).

Examples

```
# Print active session name in console  
puts [db::getAttr name -of [ss::getActiveSession]]
```

ss::getActiveTestbench

Description

Gets the current testbench. This command is useful for creating scripts that use a mixture of PrimeWave™ Design Environment script and core PrimeWave™ Design Environment Tcl commands.

Syntax

```
ss::getActiveTestbench
```

Returns

The current testbench used by the PrimeWave™ Design Environment script (<string>).

ss::getAnalyses

Description

Returns the collection of analyses in the current testbench.

Syntax

```
ss::getAnalyses [<filter: string>]
```

Arguments

<string>

Filter by analysis name. (Default: "")

Returns

Collection of analyses. (*{saAnalysis}*)

ss::getCorners

Description

Returns the collection of corners in the current testbench.

Syntax

```
ss::getCorners [<name: string>]
```

Arguments

name

Filter by corner name.

Returns

<[saCorner]>

ss::getModelGroups

Description

Returns model groups.

Syntax

```
ss::getModelGroups [<name: string>]
```

Arguments

name

Filter model groups by name.

Returns

<[saModelGroup]> : Model Group.

ss::getOutputs

Description

Returns the list of output.

Syntax

```
ss::getOutputs [<filter: string>]
```

Arguments

filter

Filter output by name.

Returns

<[saOutput]>

ss::getTestbenches

Description

Returns the collection of testbenches in the current session.

Syntax

```
ss::getTestbenches [<name: string>]
```

Arguments

name

Filter by corner name.

Returns

<[saTestbench]>

ss::listAnalyses

Description

Prints the list of analyses in tabular format.

Syntax

```
ss::listAnalyses [-report <giWindow | int>] [-filePath <string>]  
[-caption <string>]
```

Arguments

-report

Export list to the specified report window.

-filePath

File path to save the list.

-caption

Caption text for the table.

Returns

<none>

ss::listCorners

Description

Lists the corners.

Syntax

```
ss::listCorners [<filter: string>] [-report <giWindow | int>] [-filePath  
<string>] [-caption <string>]
```

Arguments

`filter`

Show corners whose names match this filter.

`-report`

Export list to the specified report window.

`-filePath`

File path to save corners list.

`-caption`

Caption text for the table.

Returns

<none>

ss::listDesVars

Description

Prints the list of analyses in tabular format.

Syntax

```
ss::listDesVars [-report <giWindow | int>] [-filePath <string>] [-caption  
  <string>]
```

Arguments

`-report`

Export list to the specified report window.

`-filePath`

File path to save the list.

`-caption`

Caption text for the table.

Returns

<none>

ss::listEnvOptions

Description

Prints the list of environment options in tabular format.

Syntax

```
ss::listEnvOptions [-report <giWindow | int> | -filePath <string>]  
                  [-caption <string>]
```

Arguments

-report

Export list to the specified report window.

-filePath

File path to save the list.

-caption

Caption text for the table.

Returns

<none>

ss::listIncludeFiles

Description

Prints the sessions list of include files in tabular format.

Syntax

```
ss::listIncludeFiles [-report <giWindow | int>] [-filePath <string>]  
                   [-caption <string>]
```

Arguments

-report

Export list to the specified report window.

-filePath

File path to save the list.

-caption

Caption text for the table.

Returns

<none>

ss::listMacros

Description

Lists all macros.

Syntax

```
ss::listMacros [-report <giWindow | int>] [-filePath <string>] [-caption  
<string>] [-format [<string>]] [-printHeaders <string>] [-delimiter  
<string>]
```

Arguments

-report

Exports the list to a specified report window.

-filePath

The file path to which to save the list.

-caption

The caption text for the table.

-format

The list of column headers and attribute names for formatting.

-printHeaders

The print column headers. (Default: true)

-delimiter

The column header delimiter character.

Returns

<none>

ss::listModelFiles

Description

Prints the model files in tabular format.

Syntax

```
ss::listModelFiles [-report <giWindow | int>] [-filePath <string>]  
[-caption <string>] [-includeGroups <bool>]
```

Arguments

-report

Export list to the specified report window.

-filePath

File path to save the list.

-caption

Caption text for the table.

-includeGroups

List model files contained in groups.

Returns

<none>

ss::listModelGroupDefs

Description

Prints the model group definitions in a tabular format.

Syntax

```
ss::listModelGroupDefs [-report <giWindow | int> | -filePath <string>]  
[-caption <string>]
```

Arguments

-caption

Caption text for the table.

-report

Export list to the specified report window.

-filePath

File path to save the list.

Returns

<none>

ss::listModelGroups

Description

Prints the model groups in a tabular format.

Syntax

```
ss::listModelGroups [-report <giWindow | int> | -filePath <string>]  
                    [-caption <string>]
```

Arguments

-caption

Caption text for the table.

-report

Export list to the specified report window.

-filePath

File path to save the list.

Returns

<none>

ss::listNodes

Description

Prints the list of node set/initial conditions in tabular format.

Syntax

```
ss::listNodes [-report <giWindow | int> | -filePath <string>] [-caption  
                    <string>]
```

Arguments

-report

Export list to the specified report window.

-filePath

File path to save the list.

-caption

Caption text for the table.

Returns

<none>

ss::listOutputs

Description

Prints the list of outputs in tabular format.

Syntax

```
ss::listOutputs [-report <giWindow | int>] [-filePath <string>] [-caption  
<string>] [-format [<string>]] [-printHeaders <string>] [-delimiter  
<string>]
```

Arguments

-report

Export list to the specified report window.

-filePath

File path to save the list.

-caption

Caption text for the table.

-format

The list of column headers and attribute names for formatting.

-printHeaders

Print column headers.

Default is: true

-delimiter

Column header delimiter character.

Returns

<none>

ss::listResults

Description

Returns the result files in alphabetical order. Each result is returned on a separate line.

Syntax

```
ss::listResults [-report <giWindow | int>] [-filePath <string>] [-caption  
  <string>]
```

Arguments

`-report`

Export list to the specified report window.

`-filePath`

File path to save the list.

`-caption`

Caption text for the table.

Returns

<none>

ss::listSaveOptions

Description

Prints the options for saving outputs in tabular format.

Syntax

```
ss::listSaveOptions [-report <giWindow | int>] [-filePath <string>]  
  [-caption <string>]
```

Arguments

`-report`

Export list to the specified report window.

`-filePath`

File path to save the list.

`-caption`

Caption text for the table.

Returns

<none>

ss::listSignals

Description

Prints the list of signals for each result file in tabular format. The printed names are in the waveform database namespace. This function is intended to be used interactively. Use `wf::findSignal` to programmatically walk signals.

Syntax

```
ss::listSignals [<filter: string>] [-report <giWindow | int>] [-filePath  
    <string>] [-caption <string>] [-resultType <saAnalysis | string>]
```

Arguments

`filter`

Filter for signal names.

`-report`

Export list to the specified report window.

`-filePath`

File path to save the list.

`-caption`

Caption text for the table.

`-resultType <string> | saAnalysis`

Limit the report to the given result type.

Returns

`<none>`

Errors

If results from more than one analysis are selected.

ss::listSimOptions

Description

Prints the list of simulator options in tabular format.

Syntax

```
ss::listSimOptions [-report <giWindow | int>] [-filePath <string>]  
    [-caption <string>]
```

Arguments

`-report`

Export list to the specified report window.

`-filePath`

File path to save the list.

`-caption`

Caption text for the table.

Returns

<none>

ss::listStimulusSets

Description

Lists the stimulus sets.

Syntax

```
ss::listStimulusSets [-report <giWindow | int>] [-filePath <string>]  
[-caption <string>]
```

Arguments

`-report`

Export list to the specified report window.

`-filePath`

File path to save the list.

`-caption`

Caption text for the table.

Returns

<none>

ss::netlist

Description

Generates final netlist. Regenerates structural netlist if needed or if forced via the `-createStructural` argument.

Syntax

```
ss::netlist [<testbench: dbCollection | saTestbench | string>]  
  [-createStructural <bool>] [-force <bool>] [-blocking <bool>]
```

Arguments

testbench

Testbench name pattern or collection of saTestbenches to run.

-createStructural

If set to true, a new structural netlist will be created along with any associated checks. (Default: 1)

-force <bool>

Force overwrite of hand edited netlists. (Default: 0)

-blocking

If true, script execution pauses until the netlisting jobs complete. (Default: 1)

Returns

The full path to the final netlist. (<string>)

Errors

- Netlist generation fails.
- Netlists on disk have been hand-edited and the `-force` option is false.

ss::newSession

Description

Creates a new PrimeWave™ Design Environment script session. This command is typically needed only to clear out an old session and reinitialize a new session. The previous session is not explicitly destroyed. However, if no Tcl objects are referencing the previous session, it will be implicitly destroyed. The unnamed arguments are used to specify session options. Currently, only these options are supported: session name, results directory, and states directory.

Note:

The default value for the results and states directory comes from the standard PrimeWave™ Design Environment preferences.

Syntax

```
ss::newSession [<options: string>] [-testSuite <string>]
```


Arguments

`options`

List of session option name/values.

`-testSuite`

Enable multi-testbench mode with the given test suite name.

Returns

The new session. (<saSession>)

Errors

If an invalid session option is specified. The list of valid session options is included in the error message.

ss::newTestbench

Description

Creates a new PrimeWave™ Design Environment testbench and makes it active.

Syntax

```
ss::newTestbench [<name: string>]
```

Arguments

`name`

Name for the new testbench (auto-generated by default).

Returns

<saTestbench> :

ss::plot

Description

Plots an expression. This command will also start up and connect to a waveform viewer if one is not already initialized.

If the plot color is not specified, a color will be chosen in sequence from the available plot colors.

Syntax

```
ss::plot <expression: string | wfSignal> [-analysis <saAnalysis |  
string>] [-resultFile <string>] [-mode <string>] [-color <string>]  
[-params <string>]
```

Arguments

`expression`

The expression or signal to plot.

`-mode`

The plot mode. Valid values: new, append, replace, newPanel, active, and custom (Default: append)

`-color`

The color used to plot the waveforms. (Default: auto)

`-analysis`

Limit expression to the given analysis name, type, or object.

`-resultFile`

Plot data from a given result file.

`-params`

The default parameter conditions for the data access functions in the expression.

Returns

`<string>`

ss::print

Description

Print an expression to a tab delimited text file. Multiple expressions can be passed in by using a semicolon-separated string.

If printing multiple signals all values will be printed at the same independent variable points (the X-axis). The X values of the first signal will be used to determine sampling points for all signals.

Syntax

```
ss::print <expression: [string | wfSignal]> [-report <giWindow | int>]  
[-filePath <string>] [-caption <string>] [-analysis <saAnalysis |  
string>] [-resultFile <string>]
```

Arguments

`expression`

The expressions or signals to print. Delimit multiple expressions with a semicolon.

`-report`

Export list to the specified report window.

`-filePath`

File path to save the list.

`-caption`

Caption text for the table.

`-analysis`

Limit expression to the given analysis name/type/object.

`-resultFile`

Use data from a given result file.

Returns

`<string>`

Errors

- The file path specified is not writable.
- There are errors evaluating the input expressions.
- There are multiple results found with differing X-axis.

ss::printMeasurementSummary

Description

This command prints the measurement summary.

Syntax

```
ss::printMeasurementSummary [<filter: string>] [-report <giWindow |  
int>] [-filePath <string>] [-caption <string>] [-violations <bool>]  
[-testbench <saTestbench | string>]
```

Arguments

`filter`

Show only measurement names matching this filter.

`-report`

Export list to the specified report window.

`-filePath`

File path to save the list.

`-caption`

Caption text for the table.

`-violations`

Show only measurements with violations, default is false.

Default is: 0

`-testbench`

Testbench or name pattern for printing.

Returns

`<none>`

ss::printNoiseSummary

Description

This command prints the noise summary results.

Syntax

```
ss::printNoiseSummary <noiseSummaryType: string> [-report <giWindow |  
int>] [-filePath <string>] [-topContributors <int> | -relativeThreshold  
<float> | -absoluteThreshold <float>] [-caption <string>] [-squared  
<bool>] [-startFrequency <float>] [-endFrequency <float>]  
[-spotFrequency <float>] [-deviceTypes <string>] [-includeElements  
<string>] [-excludeElements <string>] [-includeParams <bool>] [-params  
<string>] [-time <string>][noiseCategory >string>][-includeBreakdown  
<bool>]
```

Arguments

`noiseSummaryType`

Noise summary type: integrated or spot.

- `-report`
Export list to the specified report window.
- `-filePath`
File path to save the list.
- `-caption`
Caption text for the table.
- `-squared`
If true, report results in square units. For example, V^2 for integrated or V^2/Hz for spot noise. (Default: true)
- `-startFrequency`
Start frequency for integrated noise.
- `-endFrequency`
End frequency for integrated noise.
- `-spotFrequency`
Frequency to use for spot noise.
- `-deviceTypes`
Filter to only given device types.
- `-includeElements`
Filter to only given device instances.
- `-excludeElements`
Filter to exclude given instances.
- `-topContributors`
Filter to top n contributors.
- `-relativeThreshold`
Filter to contributors of $\geq n \%$.
- `-absoluteThreshold`
Filter to contributors $\geq n$.
- `-includeParams`
Device parameter contributions (in addition to totals). (Default: false)

`-params`

Parameter conditions for data access.

`-time`

Time point for noise analysis results.

`-noiseCategory`

Filter results per noise category. (Default: total)

`-includeBreakdown`

Include breakdown for noise category. (Default: false)

Returns

<none>

ss::printTable

Description

Prints collections of data into a standard tabular format. This command can be used on all Custom Compiler objects.

The `-format` option takes a list of Header Name/attribute name pairs specifying the ordered columns of the table.

For example, `{Name name Value value}` prints a two column table. The first column header is "Name" and the contents in the rows are the values stored in the attribute named "name". The second column header is "Value" and the contents in the rows are the values in the attribute named "value". Hierarchical attribute names are also supported. See the Example section for details.

Syntax

```
ss::printTable <collection: dbCollection> [-report <giWindow | int>]  
[-filePath <string>] [-caption <string>] -format [<string>] [-sortKey  
<string>] [-filterProc <string>] [-printHeaders <string>] [-delimiter  
<string>]
```

Arguments

`collection`

The collection of objects to print.

`-caption`

Caption text for the table.

`-report`

Export list to the specified Report window.

`-filePath`

File path to which to save the list.

`-format`

The list of column headers and attribute names for formatting.

`-sortKey`

Sort the table by the sortKey attribute name.

`-filterProc <string>`

Specifies whether or not to write the specified object.

`-printHeaders`

Print column headers. (Default: true)

`-delimiter`

Column header delimiter character.

Returns

`<string>`.

Examples

Example 1

In this example, `ss::printTable` prints all the analyses names and types if they are enabled.

```
ss::printTable [db::getAttr analyses -of\ [ss::getActiveTestbench]]  
-format {Name name Type type Enabled\ enabled}
```

Example 2

In this example, `ss::printTable` prints all outputs with their names and expressions.

```
ss::printTable [db::createCollection [ss::getOutputs]] -format\ {Name  
name Expression expr}
```

Example 3

In this example, `ss::printTable` prints all preferences matching saHSPICE. The preferences list is sorted based on the preference scope name.

```
ss::printTable [db::getPrefs saHSPICE* -scope [db::getScopes]] \ -format  
{Name name Scope scope.scopeType.name}  
-sortKey scope.scopeType.name
```

ss::readOutputs

Description

Replaces the current testbench's outputs with the contents of the specified text file.

Syntax

```
ss::readOutputs [-report <giWindow | int> | -filePath <string>] [-caption  
  <string>] [-delimiter <string>]
```

Arguments

-report

Export list to the specified Report window.

-filePath

File path to save the list.

-caption

Caption text for the table.

-delimiter

The delimiter character used to separate row items.

Returns

<none>

ss::readTable

Description

Allows you to read a table stored in a file and obtain from that table a collection of PrimeWave™ Design Environment objects. `ss::readTable` is meant to be the inverse of the `ss::printTable` command.

Syntax

```
ss::readTable [-report <giWindow | int>] [-filePath <string>] [-caption  
  <string>] -objectType <string> -delimiter <string>
```

Arguments

-report

Export list to the specified Report window.

`-filePath`

Specifies the input file.

`-caption`

Caption text for the table.

`-objectType`

Specifies the type of the SAE objects to be created (currently, only `saOutput` type is supported).

`-delimiter`

Sets the delimiter character used to separate columns.

Returns

<dbCollection> :

ss::releaseNode

Description

Releases all node set or initial conditions on the given node.

Syntax

```
ss::releaseNode <nodes: string>
```

Arguments

`nodes`

The name of the node to release. (Default: "")

Returns

<none>

ss::run

Description

Simulates the currently configured SAE Script session. Use of non-blocking run is useful when many simulations need to be started in a distributed environment.

Note:

The results might be incomplete unless the `wait` command is used.

Syntax

```
ss::run [<testbench: dbCollection | saTestbench | string>] [-blocking  
<bool>]
```

Arguments

`-testbench`

Testbench name pattern or collection of saTestbenches to run.

`-blocking`

If true, script execution will pause until the simulation is completed. (Default: 1)

Returns

The directory where simulation results will be stored. This can be used in conjunction with the wait command. (<string>)

Errors

- Netlisting failed.
- Simulation failed.
- Final netlist is hand-edited and the `-force` option is false.

Examples

```
# Simulate all testbenches in current session  
ss::run [sa::getTestbenches -of [ss::getActiveSession ]]
```

ss::saveResults

Description

Saves the current simulation results to a new directory. If the directory does not exist, it is created. If the directory already exists, then the old contents are overwritten.

Note:

To avoid recursively copying directory contents, the destination directory must not be a subdirectory of the current results directory.

Syntax

```
ss::saveResults <directory: string>
```

Arguments

`directory`

The directory to save the results.

Returns

<none>

ss::selectDataDir

Description

Specifies the default directory for retrieving simulation results. It is only necessary to use this command if a non-default data directory is to be used. Setting the data directory also resets the list of results that `ss::plot` and `ss::expr` operate on. As such, it is common to follow `ss::selectDataDir` with `ss::selectResults`.

Syntax

```
ss::selectDataDir <directory: string>
```

Arguments

`directory`

The directory used to read simulation results.

Returns

The selected data directory. (<string>)

Errors

The specified data directory is not a valid results directory.

ss::selectResults

Description

Sets the default results for `expr` and `plot` commands. If no results are specified, all available outputs will be selected.

Note:

The analysis names, analysis types, and paths to waveform databases are all valid input.

Syntax

```
ss::selectResults <results: [string]>
```

Arguments

`results`

The list of results on which to operate. (Default: {})

Returns

The selected results files. (*{saResultsaResultFile}*)

ss::setActiveNets

Description

Sets the options for the active nets flow. The format for the active nets files is a list of file names. If the file names are specified without the path, they are assumed to be relative to the netlist directory.

Syntax

```
ss::setActiveNets [-mode <string>] [-analyses <string>] [-files <string>]
```

Arguments

-mode

Active nets flow mode.

-analyses

Simulator-specific active nets analysis options.

-files

List of paths to the active nets file.

Returns

<saObject> :

Errors

If the specified options are not valid for the current simulator.

ss::setActiveSession

Description

Sets the session operated on by the PrimeWave™ Design Environment script.

Syntax

```
ss::setActiveSession <session: saSession | string>
```

Arguments

session

The new default session for the PrimeWave™ Design Environment script.

Returns

The session used by the PrimeWave™ Design Environment script. ()

ss::setActiveTestbench

Description

This command sets the active testbench for scripts.

Syntax

```
ss::setActiveTestbench <testbench: saTestbench | string>
```

Arguments

testbench

The new active testbench.

Returns

<none>

ss::setAnalysisOptions

Description

Set analysis options.

Syntax

```
ss::setAnalysisOptions <name: string> -options <string>
```

Arguments

name

Analysis name.

-options

Analysis-specific options.

Returns

<none>

Examples

In this example, `ss::SetAnalysisOptions` sets the transient analysis options for simulation.

```
set an [ss::createAnalysis tran -name tran]
ss::setAnalysisOptions tran -options {enabled 1 start 0 step 1n\
    stop 10n}
```

ss::setDesign

Description

Sets the design for simulation. The design can either be specified as a dmCellView, an oaDesign or a Library/Cell/View triplet formatted as "libName/cellName/viewName".

The newly set dmCellView is returned.

Syntax

```
ss::setDesign <design: dmCellView | oaDesign | string>
```

Arguments

design

The design to simulate.

Returns

The set design.

ss::setDesVars

Description

Sets design variables for simulation. The format for design variables is a list of design variable names and values: [list desVarName1 desVarValue1 desVarName2 desVarValue2...]

Syntax

```
ss::setDesVars <designVars: string>
```

Arguments

designVars

The list of design variable names and values.

Returns

<saDesVar>.

Errors

If the input list has an odd number of elements.

Examples

```
# Add new design variables into active testbench
ss::setDesVars {wp lu lp 0.1u}
```

ss::setDeviceParameters

Parameterize the devices

Syntax

```
ss::setDeviceParameters <deviceInfo: string> [-validate <bool>] [-sync  
  <bool>]
```

Returns

<salInstanceParameters> : List of instance parameter objects

Arguments

deviceInfo

List containing the library/cell/view, list of instance names, list of CDF parameter names, list of parameterized expressions and group name

-validate

Check for instantiation of the device in the design hierarchy (Default is: 0)

-sync

If true, create the variables for the parameters and also synchronize with group (Default is: 0)

ss::setEnvOptions

Description

Sets the environment options. Environment options are typically displayed in the **Environment Options** dialog box. Typical options include: **Stop View List**, **Search View List**, command-line options, and so forth. Additional options are integration-specific.

Note:

If an invalid option is specified, an error message is generated that includes all of the valid save option names.

Syntax

```
ss::setEnvOptions <options: string>
```

Arguments

`options`

The list of option names and values.

Returns

`<none>`

Errors

If any of the specified options do not exist for the current simulator.

ss::setGlobalDeviceParameters

Parameterize the devices in global session

Syntax

```
ss::setGlobalDeviceParameters <deviceInfo: string> [-validate <bool>]  
[-sync <bool>]
```

Returns

`<salInstanceParameters>` : List of instance parameter objects

Arguments

`deviceInfo`

List containing the library/cell/view, list of instance names, list of CDF parameter names, list of parameterized expressions and group name

`-validate`

Check for instantiation of the device in the design hierarchy (Default is: 0)

`-sync`

If true, create the variables for the parameters and also synchronize with group (Default is: 0)

ss::setGlobalModelFiles

Description

Sets global model files.

Syntax

```
ss::setGlobalModelFiles [<modelFiles: string>]
```


Arguments

`modelFiles`

Model file path or entry pairs.

Returns

<none>

ss::setGlobalTemperature

Description

Sets global temperature.

Syntax

```
ss::setGlobalTemperature <temperature: string>
```

Arguments

`temperature`

Temperature in Celsius, Kelvin, or Fahrenheit (C, K, or F).

Returns

<string> :

ss::setGlobalVariables

Description

Sets or creates global design variables.

Syntax

```
ss::setGlobalVariables <designVars: string>
```

Arguments

`designVars`

Design variable name or value pairs.

Returns

<saDesVar> :

ss::setIncludeFiles

Replace the list of include files for simulation.

Syntax

```
ss::setIncludeFiles [<includeFiles: string>] [-merge <bool>]
```

Returns

<none>

Arguments

includeFiles

Include file path/type pairs.

-merge

merge with original files

ss::setJobOptions

Set job options on session or testbench.

Syntax

```
ss::setJobOptions [<options: string>] [-isGlobal <bool>]
```

Returns

<string> :

Arguments

options

List of job option name/values.

-isGlobal

Set options to session. (Default is: false)

ss::setModelFiles

Description

Sets the model files for simulation. All previously configured model files are removed. Passing in the empty list clears all model files.

The format for model files is a list of file name and library entry pairs: [list file1 entry1 file2 entry2...]

Syntax

```
ss::setModelFiles [<modelFiles: string>]
```

Arguments

modelFiles

The list of model files and library entries.

Returns

<none>

Errors

If the input list is an odd number of elements. Every file requires the specification of a library entry. Use the empty string (either "" or {}) to indicate that there is no library entry.

ss::setNode

Description

Adds node set or IC statements providing convergence aids for simulation. Node sets and initial conditions seed the initial value for the simulator's iterative DC algorithm. Initial conditions additionally hold the specified nodes to the specified voltages throughout the DC solution.

The format for initial conditions is a list of node name and values.

[list caName1 value1 caName2 value2 caName3 value3...]

Syntax

```
ss::setNode [<nodeSet: string>] [-ic <string>]
```

Arguments

nodeSet

Node names and voltages for node sets.

`-ic`

Specify initial conditions.

Returns

<none>

ss::setPostRunProcs

Description

Sets post run procs.

Syntax

```
ss::setPostRunProcs <scripts: string>
```

Arguments

`scripts`

List of post run procs.

Returns

<none>

ss::setPreRunProcs

Description

Sets pre run procs.

Syntax

```
ss::setPreRunProcs <scripts: string>
```

Arguments

`scripts`

List of pre run procs.

Returns

<none>

ss::setSaveOptions

Description

Sets the save options. Save options are typically displayed in the **Save Options** dialog box. Typical options include: save all currents, save all power, save all voltage, and the number of levels to save. The option names are simulator integration dependent.

Note:

If an invalid save option is specified an error message is generated that includes all of the valid save option names.

Syntax

```
ss::setSaveOptions <options: string>
```

Arguments

options

The list of option names and values.

Returns

<none>

Errors

If any of the specified save options do not exist for the current simulator. A list of valid options for the current simulator is included in the error message.

ss::setSimOptions

Description

Sets simulator options.

Syntax

```
ss::setSimOptions <options: string> [-collectionName <string>]
```

Arguments

options

List of simulator option names and values.

-collectionName

Options Collection testbench attribute. (Default: options)

Returns

<none>

Errors

If one or more of the specified options do not exist for the given simulator.

ss::setSimulator

Description

Sets the simulator integration used by the current PrimeWave™ Design Environment script session.

Syntax

```
ss::setSimulator <simulator: saSimulator | string>
```

Arguments

simulator

The simulator to use for the current PrimeWave™ Design Environment script session.

Returns

The name of the set simulator. (<string>)

Errors

The specified simulator name does not match any of the installed simulators.

Examples

```
# Set simulator in active testbench
ss::setSimulator PrimeSim
```

ss::setStimulusSet

Description

Sets the active stimulus set.

Syntax

```
ss::setStimulusSet <stimulusSet: string>
```

Arguments

`stimulusSet`

Stimulus set creation commands.

Returns

<none>

ss::setTemperature

Description

Sets the simulation temperature. Units of Celsius, Kelvin, and Fahrenheit (C, K, and F respectively) are supported. If no unit is specified, Celsius is assumed.

Syntax

```
ss::setTemperature <temperature: string>
```

Arguments

`temperature`

The new temperature.

Returns

<string>

Examples

```
# Set simulation temperature in active testbench
ss::setTemperature 27
```

ss::setTestbenchName

Description

Sets the testbench name for the current session.

Syntax

```
ss::setTestbenchName <name: string>
```

Arguments

`name`

The testbench name to use.

Returns

<string> :

ss::simulateCalibration

Run calibration simulation in prerun.

Syntax

```
ss::simulateCalibration
```

Returns

<string> :

ss::trackActiveTestbench

Description

Tracks the active saTestbench through the GUI. The GUI's last-active session and testbench are used as the information source for the `ss::list*` commands. The `ss::trackActiveTestbench` command turns this tracking on or off, allowing the user to set this manually (`ss::setActiveSession` and `ss::setActiveTestbench`).

Syntax

```
ss::trackActiveTestbench [<track: bool>]
```

Arguments

track

New tracking state. (Default: 1)

Returns

<string> :

Enumerations

Table 189 Enumerations

Name	Values	Description
enGraphType	rectangular polar zsmith ysmith	Waveform graph type.
enPlotMode	new append replace overlay	Waveform plotting mode.

Objects

This section covers the following:

- [dmCell](#)
- [dmCellView](#)
- [saCorner](#)
- [saCornersSetup](#)
- [saOutput](#)
- [saResultFile](#)
- [saResults](#)
- [saSession](#)
- [saSimulator](#)
- [saSweep](#)
- [wfSignal](#)

dmCell

A named container, in a library, that can hold files and dmCellViews. The dmCell corresponds to a specific oaCell object but is valid as long as the library that the dmCell belongs to remains open. The getAccess requirements of an oaCell are handled automatically by this dm object.

Table 190 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the cell.
libName	string	Read-only	The name of the cell's library.
statusIcon	string	Read/Write	Incremental path to an icon supplied by the version control system.

dmCellView

The dmCellView is a container in a library that holds files and is identified by the combination of its cell and view names. It corresponds to an oaCellView but it stays valid as long as its library remains open and its dmCell and dmView exist.

Table 191 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the view to which this cellView belongs.
cellName	string	Read-only	The name of the cell to which this cellView belongs.
cell	dmCell	Read-only	The cellView's cell.
version	string	Read/Write	A version string annotated by the version control system.
statusIcon	string	Read/Write	Incremental path to an icon supplied by the version control system.
libName	string	Read-only	The name of the library to which this cellView belongs.

saCorner

The saCorner object represents a column in the corners setup table. The attributes of the object are described in the table below and can be retrieved or set by db::getAttribute and db::setAttribute commands.

Table 192 Attributes

Name	Type	Access	Description
name	string	Read/Write	The corner name.
parameters	{saPair}	Read/Write	The collection of saPair. Each saPair represents a design variable or model file or temperature and its value for the corner.
enabled	bool	Read/Write	The enable and disable status of the corner.

saCornersSetup

The saCornersSetup object represents the setup data of the corners analysis tool. The attributes of the object are described in the table below and can be retrieved or set by db::getAttribute and db::setAttribute commands.

Table 193 Attributes

Name	Type	Access	Description
paramDefs	{saCornerParamDef}	Read/Write	The collection of corner parameter definitions
corners	{saCorner}	Read/Write	The collection of corner objects.
enabled	bool	Read/Write	The enable and disable status of the corner analysis tool. Modifying the value of this attribute does not change the enable/disable status of the individual corners.

saOutput

This Object represents an entry in the outputs table in the main PrimeWave™ Design Environment window.

Table 194 Attributes

Name	Type	Access	Description
name	string	Read/Write	Name given to the output..
expr	string	Read/Write	Expression. When the output is a signal, expression would be the net or terminal name.
analyses	{string}	Read/Write	The analyses for which this output is evaluated/plotted.
plot	bool	Read/Write	Specifies whether the output should be plotted when simulation is complete.

Table 194 Attributes (Continued)

Name	Type	Access	Description
save	bool	Read/Write	Specifies whether the output should be saved by the simulator. This will work with the simulation save options to create <code>.probe</code> (or simulator equivalent) commands.
value	string <i>wfSignal</i>	Read-only	Value of the expression.
outputType	<i>enOutputType</i>	Read/Write	Output type (net, terminal or expression).
probe	string	Read/Write	Display packet name to be used in cross-probing.

saResultFile

The `saResultFile` is created for each simulation output file.

Table 195 Attributes

Name	Type	Access	Description
filename	string	Read-only	Simulation output file name.
resultType	string	Read-only	Result type like <code>tran</code> , <code>opBegin</code> , <code>opTran</code> , and so forth.
analysisName	string	Read-only	Analysis name like <code>tranA</code> , <code>tranB</code> , and so forth.
format	string	Read-only	Output file format like <code>psf</code> , <code>wdf</code> , and so forth.

saResults

The `saResults` is created at the end of a (successful or failed) simulation run. Note: The parametric, corners and statistical analysis tools can run multiple simulations. In that case, there will be multiple `saResults` objects.

Table 196 Attributes

Name	Type	Access	Description
name	string	Read-only	results.xml
version	string	Read-only	Version number.
simulator	string	Read-only	Simulator name.
netlistDir	string	Read-only	The netlist directory.
resultsDir	string	Read-only	The simulation result directory where the simulation output files are created.
runTime	string	Read-only	The simulation completion date.
resultFiles	<i>{saResultFile}</i>	Read-only	The collection of saResultFile object.

saSession

saSession object represents one invocation of the PrimeWave™ Design Environment in a session. Session name can be specified by user or is auto generated by appending an integer to saSession. For example "saSession1", "saSession2" etc. resultsDir is the root directory of results directory structure. statesDir is the root directory of states directory structure.

Table 197 Attributes

Name	Type	Access	Description
name	string	Read/Write	Unique string identifier for this session.
resultsDir	string	Read/Write	Results directory for a PrimeWave™ Design Environment session.
statesDir	string	Read/Write	States directory for aPrimeWave™ Design Environment session.
testbenches	<i>{saTestbench}</i>	Read/Write	Testbenches in the ceSession.

saSimulator

The saSimulator object is created for each simulator integrated into the PrimeWave™ Design Environment.

Table 198 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the simulator.
finalNetlist	string	Read/Write	The name of the final netlist file. The default value is finalNetlist.
structuralNetlist	string	Read/Write	The name of the structural netlist. The default value is netlist.
languages	{string}	Read/Write	Collection netlist formats (languages). The default netlist format is <simulator_name>.
includeTypes	{string}	Read/Write	The external file types supported in the Include Files dialog for the simulator integration.
logFile	string	Read/Write	The name of the simulator log file. The default log file name is <simulator_name>.out.
netlistTemplate	string	Read/Write	Name of the template file containing the overall structure of the final netlist. The include files in the template are specified relative to the netlist directory.
analysisDefs	{saAnalysisDef}	Read/Write	Collection of saAnalysisDef objects.
integrationVersion	int	Read/Write	The field for the integrator to specify the integration version.
optionsDef	{saOptions}	Read/Write	Collection of saOptions.
optionGroup	saOptionGroup	Read/Write	saOptionGroup.

saSweep

A saSweep represent a sweep specification.

Table 199 Attributes

Name	Type	Access	Description
name	string	Read-only	Name of the sweep.
variable	string	Read/Write	The variable or parameter that is being swept.
enabled	bool	Read/Write	Whether current sweep is enabled.
sweepRange	saSweep	Read-only	The sweep range object with which the sweep is specified.

wfSignal

A wfSignal represents a signal in a waveform file.

Table 200 Attributes

Name	Type	Access	Description
name	string	Read-only	Signal name identifier.
signalType	enWFSigType	Read-only	Signal type.
file	wfDataFile	Read-only	Parent file.
width	int	Read-only	Number of bits for real or logic bus.
alias	string	Read/Write	Signal name alias.
nSweeps	int	Read-only	Number of sweeps.
variableNames	{string}	Read-only	List of variable names.
variableConditions	{string}	Read-only	List of variable conditions, one string for each sweep trace.
unit	string	Read-only	Unit of signal values.
xUnit	string	Read-only	Unit of x-axis of signal.

24

MTB Simulation Scripting Tcl

This chapter lists the commands, enumerations, and objects in the Custom Compiler Tcl ssm namespace. This namespace provides Tcl for PrimeWave™ Design Environment multiple-testbench (MTB) simulation scripting.

For details, see:

- [Commands](#)

Commands

Interactive commands are listed after non-interactive commands in the Commands section.

The commands covered are:

- [ssm::createCorner](#)
- [ssm::getCornerName](#)
- [ssm::getCornerParams](#)
- [ssm::getCorners](#)
- [ssm::getDesVarVal](#)
- [ssm::getOutputs](#)
- [ssm::getScalarValue](#)
- [ssm::getSpecifications](#)
- [ssm::getSweepParams](#)
- [ssm::listDesVars](#)
- [ssm::setDesVars](#)
- [ssm::setScalarValue](#)

ssm::createCorner

Description

Creates a new corner.

Syntax

```
ssm::createCorner <name: string> [-params <string>] [-models <string>]  
[-includes <string>] [-testbench <saTestbench | string>]
```

Arguments

name

Corner name.

-params

List of name-value pairs for each parameter.

-models

Model file specification for the corner. For example, `{{models.inc}}`,
`{{single_corner.lib TT}}` or `{{multiple_corners.lib {TT FF SS}}}`.

-includes

List of include file parameter names or file names.

-testbench

Testbench.

Returns

<saCorner>

ssm::getCornerName

Description

Gets the corner name for the current child run.

Syntax

```
ssm::getCornerName
```

Returns

<none>

ssm::getCornerParams

Description

Gets the corner parameters for the current child run.

Syntax

```
ssm::getCornerParams
```

Returns

<none>

ssm::getCorners

Description

Returns the collection of corners in the current testbench or named testbench.

Syntax

```
ssm::getCorners [<name: string>] [-testbench <saTestbench | string>]
```

Arguments

name

Filter by the name of the corner.

-testbench

Testbench.

Returns

<[saCorner]>

ssm::getDesVarVal

Description

Returns the collection of corners in the current testbench or named testbench.

Syntax

```
ssm::getDesVarVal <name: string> [-testbench <saTestbench | string>]
```

Arguments

name

Design variable name.

-testbench

Testbench.

Returns

<[saCorner]>

ssm::getOutputs

Description

Gets the list of outputs.

Syntax

```
ssm::getOutputs [<filter: string>] [-testbench <saTestbench | string>]
```

Arguments

filter

Filter outputs by name.

-testbench

Testbench.

Returns

<[saOutput]> :

ssm::getScalarValue

Description

Gets the value of the scalar output.

Syntax

```
ssm::getScalarValue <name: string> -testbenchName <string> [-cornerName  
<string>] [-historyPoint <string>] [-testbench <saTestbench>]
```

Arguments

`name`

Name of the scalar output.

`-testbenchName`

Name of the testbench.

`-cornerName`

Name of the corner.

`-historyPoint`

Name of the history point.

`-testbench`

Current testbench.

Returns

`<string>`

ssm::getSpecifications

Description

Gets the list of specifications.

Syntax

```
ssm::getSpecifications [<filter: string>] [-testbench <saTestbench |  
string>]
```

Arguments

`filter`

Filter specifications by name.

`-testbench`

Testbench.

Returns

`<[saOutput]>`

ssm::getSweepParams

Description

Gets the sweep parameters for the current child run.

Syntax

```
ssm::getSweepParams
```

Returns

<none>

ssm::listDesVars

Description

Lists the design variables.

Syntax

```
ssm::listDesVars [-report <giWindow | int> | -filePath <string>]  
[-caption <string>] [-testbench <saTestbench | string>]
```

Arguments

-caption

Caption text for the table.

-report

Export list to the specified report window.

-filePath

File path to which to save the list.

-testbench

Testbench.

Returns

<none>

ssm::setDesVars

Description

Sets and creates the design variables.

Syntax

```
ssm::setDesVars <designVars: string> [-testbench <saTestbench | string>]
```

Arguments

`designVars`

Design variable name or value pairs.

`-testbench`

Testbench.

Returns

`<saDesVar>`

ssm::setScalarValue

Description

Gets the value of scalar output.

Syntax

```
ssm::setScalarValue <name: string> -value <string> [-testbench  
  <saTestbench | string>]
```

Arguments

`name`

Name of the scalar output.

`-value`

Scalar value.

`-testbench`

Testbench.

Returns

`<string>`

25

Text Editing Tcl

This chapter lists commands, objects, and preferences in the Custom Compiler Tcl `te` namespace. This namespace provides Tcl for editing text views of designs.

For details, see:

- [Commands](#)
- [Objects](#)
- [Preferences](#)

Commands

Interactive commands are listed after non-interactive commands in the Commands section.

The commands covered are:

- [te::bindExternalNetlistView](#)
- [te::getActiveObject](#)
- [te::getCursor](#)
- [te::getTokens](#)
- [te::insertText](#)
- [te::removeText](#)
- [te::setCursor](#)
- [te::showDesignOptions](#)
- [te::showExternalSyntaxCheckOptions](#)

te::bindExternalNetlistView

Description

Binds the cellview to an external netlist file.

Syntax

```
te::bindExternalNetlistView -file <string> -language <enum> [-module  
  <string>] -cellView <dmCellView | dmCellViewName> [-overwrite <bool>]  
  [-virtualTermMapping [<string>]] [-virtualTermsReferenceView [<string>]]
```

Arguments

-file

Path of the file to which to bind the cellview.

-language

Simulation language of the netlist file ({ CDL | SPF | SPICE | SystemVerilog |
VHDL | Verilog | VerilogA | VerilogAMS })

-module

Module name to which to bind the cellview. If not specified, the cell name will be
used instead.

-cellView

dmCellView to bind to the external netlist.

-overwrite

Indicates whether the cellview can be overwritten. (Default is: false)

-virtualTermMapping

Terminal-to-connection definition mapping.

-virtualTermsReferenceView

Prioritized list of views for determining whether or not a terminal should be
virtual. Any terminal that is not found in a reference view will be created as
virtual.

Returns

<dmCellView> : Updated CellView.

Examples

Example 1

The following example creates a cell view of externalNetlist type, and binds the netlist file to that cell view.

```
set file ./Extracted_Netlist/pll.spf
set lcv [dm::createCellView spice -cell PLL/pll \
        -viewType externalNetlist]
te::bindExternalNetlistView -file $file -language SPICE -cellView $lcv \
    -overwrite true
```

Example 2

In this case, the external spice netlist has extra vdd and vss terminals, which are missing in the cell's symbol view. In this example virtual terminals are automatically created in the text view.

```
set fiele ./Extracted_Netlist/pll.spf
set lcv [dm::createCellView spice -cell PLL/pll \
        -viewType externalNetlist]
te::bindExternalNetlistView -file $fiele -language SPICE -cellView $lcv \
    -virtualTermsReferenceView symbol -overwrite true
```

Example 3

In this case, the cell symbol has connection definitions for power nets. The external spice netlist gets binded to the cell view and the spice vdd and vss terminals get mapped to the virtual terminals. The vdd! terminal is mapped to pwr connection definition and the default net name is 1.8VDC.

```
set fiele ./Extracted_Netlist/pll.spf
set lcv [dm::createCellView spice -cell PLL/pll \
        -viewType externalNetlist]
te::bindExternalNetlistView -file $fiele -language SPICE -cellView $lcv \
    -virtualTermsReferenceView symbo \
    -virtualTermMapping " "vdd! pwr 1.8VDC"" -overwrite true
```

te::getActiveObject

Description

Returns the active OA object in the Text Editor. The active object is determined based on the text under the mouse cursor.

Syntax

```
te::getActiveObject [<window: int>]
```

Arguments

window

Window Id.

Returns

<oaBlockObject> : Current active OA object.

Examples

In this example, `execProc` commands gets the object under cursor in the Text Editor window, and printouts its properties. The `execProc` command executes with Ctrl+f shortcut key in the netlist editor window.

```
proc execProc {} {  
    set winId [db::getattr id -of [gi::getActiveWindow]]  
    set obj [te::getActiveObject $winId]  
    puts "[db::getattr obj]"  
}  
  
set bindingSet [gi::getBindingSets "Custom Compiler"]  
gi::createBinding -event " Ctrl-f" -windowType teText\  
    -command execProc -set $bindingSet
```

te::getCursor

Description

Returns the current cursor position in the document as a row, column pair. If there is no cursor then -1,-1 is returned. The row and column numbers starts at 1.

Syntax

```
te::getCursor [<window: int>]
```

Arguments

window

teText window.

Default: Active window

Returns

Row and column of the cursor. (<point>)

Errors

Window is not of type teText.

Examples

Example 1

Returns the position of the cursor (row, column) from active Text Editor.

```
te::getCursor
```

Example 2

In this example, `execProc` procedure gets cursor position in the Text Editor window, and printouts its value. The `execProc` procedure executes with Ctrl+f shortcut key in the Text Editor window.

```
proc execProc {} {  
    set winId [db::getattr id -of [gi::getActiveWindow]]  
    set obj [te::getCursor $winId]  
    puts $obj  
}  
  
set bindingSet [gi::getBindingSets "Custom Compiler"]  
gi::createBinding -event " Ctrl-f " -windowType teText\  
    -command execProc -set $bindingSet
```

te::getTokens

Description

Returns the token found at the given position. If the position is not given, then all the tokens in the document are returned. If a position is given, the token at that position is returned. The line and column numbers starts at 1. If the second value of the position pair is zero, then all the tokens on the given line is returned in the collection. If both line and column are zero then all the tokens in the document are returned. The order of tokens in the collection is the same order found in the document.

Syntax

```
te::getTokens [<window: int>] [-position <point>]
```

Arguments

`window`

teText window.

Default: Active window

`-position`

Row and column in teText window of token to fetch.

Returns

A collection of tokens found at the given position (`{teToken}`).

Errors

- Window is not of type teText.
- Position does not exist in canvas.

Examples

Example 1

Returns the token found at the given position. In this example, teToken object type and text are printed in the console.

```
set token [te::getTokens 2 -position {1 1}]
puts [db::getattr tokenType -of $token]
puts [db::getattr text -of $token]
```

Example 2

In this example, `execProc` commands executes the `te::getTokens` command with cursor position in the Text Editor window and printouts its token. The `execProc` command executes with Ctrl+f shortcut key in the Text Editor window.

```
proc execProc {} {
    set winId [db::getattr id -of [gi::getActiveWindow]]
    set position [te::getCursor $winId]
    set token [te::getTokens $winId -position $position]
    puts [db::getattr tokenType -of $token]
    puts [db::getattr text -of $token]
}

set bindingSet [gi::getBindingSets "Custom Compiler"]
gi::createBinding -event " Ctrl-f " -windowType teText\
    -command execProc -set $bindingSet
```

te::insertText

Description

Inserts the given string into the teText canvas at the given position. If the teText editor is not in editing mode, this command returns an error.

Syntax

```
te::insertText <string: string> [-window <int>] -position <point>
```

Arguments

string

Text to insert into teText canvas.

`-window`

teText window to receive text.

Default: Active window

`-position`

Position in canvas to insert text.

Returns

A collection of tokens found at the given position (`{teToken}`).

Errors

- Window is not of type teText.
- Position does not exist on canvas.

Examples

Example 1

In this example, `te::insertText` inserts the given string into the teText canvas at the given position.

```
te::insertText "Hello World!!!" -window 2 -position {2 1}
```

Example 2

In this example, `execProc` commands executes the `te::insertText` command which inserts a string at the cursor position in the Text Editor window. The `execProc` command executes with Ctrl+f shortcut key in the Text Editor window. The Text Editor needs to be in editing mode.

```
proc execProc {} {  
    set winId [db::getattr id -of [gi::getActiveWindow]]  
    set position [te::getCursor $winId]  
    set str "Hello World!!!"  
    set point [split $position]  
    te::insertText $str -window $winId -position $point  
}  
  
set bindingSet [gi::getBindingSets "Custom Compiler"]  
gi::createBinding -event " Ctrl-f " -windowType teText\  
    -command execProc -set $bindingSet
```

te::removeText

Description

Removes the given numbers of characters from the teText canvas at the given position. If the teText editor is not in editing mode, this command returns an error.

Syntax

```
te::removeText [-window <int>] -position <point> [-count <unsigned int>]
```

Arguments

-window

teText window to receive text. (Default: Active window)

-position

Position in canvas to insert text.

-count

Number of characters to remove from the teText canvas. (Default: 1)

Returns

<none>

Errors

- Window is not of type teText.
- Position does not exist in canvas.
- Count is a non-positive value.

Examples

In this example, `execProc` procedures executes `te::removeText` command which removes characters at the cursor position in the Text editor window. The `execProc` procedure executes with Ctrl+f shortcut key in the Text Editor window. The Text Editor needs to be in editing mode.

```
proc execProc {} {  
    set winId [db::getattr id -of [gi::getActiveWindow]]  
    set position [te::getCursor $winId]  
    set point [split $position]  
    te::removeText -window $winId -position $point -count 1  
}  
  
set bindingSet [gi::getBindingSets "Custom Compiler"]  
gi::createBinding -event " Ctrl-f " -windowType teText\  
    -command execProc -set $bindingSet
```

te::setCursor

Description

Sets the current cursor position in the document to the row, column pair given. If the position given is not valid, then an error is returned. The row and column numbers start at 1.

Syntax

```
te::setCursor [<window: int>] -position <point>
```

Arguments

window

teText window.

Default: Active window

-position

Row and column position to set cursor.

Returns

<none>

Errors

- Window is not of type teText.
- Point does not exist in canvas.

Examples

The following example sets the current cursor position in the document for the specified row and column pair.

```
te::setCursor 2 -position {3 2}
```

te::showDesignOptions

Description

Brings up the Text Editor **Design Options** dialog box for the given window.

Syntax

```
te::showDesignOptions [<window: giWindow | int>]
```

Arguments

`window`

Parent Text Editor window.

Returns

`giDialog` object of new options dialog.

Errors

Window is not of type `teText`.

Examples

The following example brings up the Text Editor **Design Options** dialog box.

```
te::showDesignOptions [gi::getActiveWindow]
```

te::showExternalSyntaxCheckOptions

Description

Opens the **External Parser Options** dialog box.

Syntax

```
te::showExternalSyntaxCheckOptions -window <giWindow | int>
```

Arguments

`-window`

Text editor window for which to show external parser options.

Returns

`<giDialog>` : Created dialog box

Objects

This section covers the following:

- [giDialog](#)
- [teToken](#)

giDialog

The `giDialog` object represents an instantiated, but not necessarily visible dialog. This object might have been constructed with Opal or Qt, in which case the `children` attribute returns an empty list.

Table 201 *Attributes*

Name	Type	Access	Description
name	string	Read-only	Dialog name.
parent	<i>giWindow</i>	Read-only	Parent window or "" if the dialog is does not have a parent.
title	string	Read/Write	Title shown in the banner.
geometry	string	Read/Write	Geometry string of the format {width}x{height}+{xOffset}+{yOffset}.
shown	bool	Read/Write	Whether the dialog is currently shown.
enabled	bool	Read/Write	If set to false, all contained widgets are grayed out and cannot be modified.
extraButtons	{string}	Read/Write	List of extra buttons shown to the right of the OK/Apply buttons and to the left of the Cancel button.
prefScope	<i>dbScope</i>	Read/Write	Preference scope for contained preference-bound widgets.
children	<i>{giWidgetgiContainer}</i>	Read-only	List of immediate child widgets.

teToken

A token object holds information about a specific string found in a netlist.

Table 202 *Attributes*

Name	Type	Access	Description
tokenType	<i>enTokenType</i>	Read-only	Token type.

Table 202 *Attributes (Continued)*

Name	Type	Access	Description
text	string	Read-only	String.
line	int	Read-only	Line number in file containing token.
start	int	Read-only	Starting column number on line.
end	int	Read-only	Ending column number on line.

Preferences

Table 203 lists Custom Compiler te namespace preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 203 *Text View Editing Preferences*

Name	Type	Scope	Default	Description
teBackgroundColor	string	user	black	Indicates which color should be fill in Text Editor canvas's background.
teCdITemplate	string	user	true	Default text to add to a newly created CDLText cellview's primary file.
teCheckCrossViews	bool	cellview	true	Indicates whether cross view consistency checks will be run when extracting the netlist text to shadowOA. Valid values: true, false
teDesignNavigatorInstances View	string	context	te_all	Sub-view to use in the Insts tab of the Design Navigator in the Text Editor.
teDesignNavigatorNetsView	string	context	te_all	Sub-view to use in the Nets tab of the Design Navigator in the Text Editor.
teDesignNavigatorTerminals View	string	context	te_all	Sub-view to use in the Terminals tab of the Design Navigator in the Text Editor.
teDeviceMappingFile	string	cellview		The path of the mapping file used for device creation.

Table 203 Text View Editing Preferences (Continued)

Name	Type	Scope	Default	Description
teEditExternal	bool	user	true	Indicates whether the defined external editor should be launched when the user elects to edit the text. Valid values: true, false
teFindMatchCase	bool	context	false	Indicates whether searching should be done in case-sensitive manner. Valid values: true, false
teFindString	string	context		String for which to search.
teFindStringsIsRegExp	bool	context	false	Indicates whether search string is a regular expression. Valid values: true, false
teParseExtraTermsAsVirtual	bool	user	false	Indicates whether port names given in the mapping file that do not exist in the design should be considered as virtual terminals. Valid values: true, false
tePvaDisplayOutputLog	displayLogEnum	user	onErrors	Specifies when to display the output of the external parser tool. Valid values: onErrors onErrorsOrWarnings always never
tePvaExtraOptions	string	user		Extra options to pass to pva when parsing a text view.
tePvaIncludes	string	user		Include file paths to pass to pva when parsing a text view.
tePvaRunOnSave	bool	user	false	Indicates whether pva should be run saving a text view. Valid values: true, false
teRelaxedChecking	bool	user	true	Indicates whether relaxed checking is used. Valid values: true, false
teShowLineNumbers	bool	cellview	true	Indicates whether the line numbers should be shown at the left side. Valid values: true, false

Table 203 Text View Editing Preferences (Continued)

Name	Type	Scope	Default	Description
teSpiceTemplate	string	user	* Created: %date% .subckt %cell% .ends %cell%	Default text to add to a newly created SPICEText cellview's primary file.
teSystemVerilogTemplate	string	user		Default text to add to a newly created systemVerilogText cellview's primary file.
teTextHeight	int	user	12	Indicates how many pixel should be shown as text height.
teVerilogAMSTemplate	string	user	// Created %date% module %cell% (); analog begin end endmodule	Default text to add to a newly created verilogAMSText cellview's primary file.
teVerilogATemplate	string	user	// Created %date% module %cell% (); analog begin end endmodule	Default text to add to a newly created verilogAText cellview's primary file.
teVerilogTemplate	string	user	// Created %date% module %cell% (); always begin end endmodule	Default text to add to a newly created verilogText cellview's primary file.
teVerilogTieHighNet	string	user	tie1	The net name in OA to use for the Verilog constant '1'.
teVerilogTieLowNet	string	user	tie0	The net name in OA to use for the Verilog constant '0'.
teVhdlTemplate	string	user		Default text to add to a newly created VHDLText cellview's primary file.
teVloganDisplayOutputLog	displayLogEnum	user	onErrors	Specifies when to display the output of the external parser tool. Valid values: onErrors onErrorsOrWarnings always never
teViewSearchList	string	context	schematic symbol %PLACED	Default switch list for design navigation in the text editor.
teVloganExtraOptions	string	user		Extra options to pass to vlogan when parsing a text view.
teVloganIncludes	string	user		Include file paths to pass to vlogan when parsing a text view.

Table 203 Text View Editing Preferences (Continued)

Name	Type	Scope	Default	Description
teVlogonRunOnSave	bool	user	false	Indicates whether vlogon should be run saving a text view. Valid values: true, false

26

Technology Tcl

*This chapter lists technology preferences available in Custom Compiler Tcl. These preferences operate with the Technology Manager. The system accepts preferences as entered at the Console command line, added to prefs.xml, and saved using the **Defaults** button menu in dialog boxes. The Technology Manager uses database (db namespace) Tcl.*

For details, see:

- [Database Tcl](#)
- [Preferences](#)

Preferences

[Table 204](#) lists Custom Compiler technology preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 204 Technology Preferences

Name	Type	Scope	Default	Description
techExportLocal	<bool>	user	false	Specifies that export should be done only for local level of technology data, plus techArray.
techManagerFileName	<string>	user		File name for importing/exporting technology.
techManagerImportMode	<i>enTech Import Mode</i>	user	merge	Specifies if technology should be merged or overwritten during importing.

27

UDD Editing Tcl

This chapter lists the commands and preferences in the Custom Compiler Tcl `ue` namespace. This namespace provides Tcl for UDD editing. Interactive commands are listed at the end of the chapter.

For details, see:

- [Commands](#)
- [Preferences](#)

Commands

The commands covered are:

- [ue::cndbg](#)
- [ue::compile](#)
- [ue::debug](#)
- [ue::getDesign](#)
- [ue::preview](#)
- [ue::searchAndReplace](#)
- [ue::showOptions](#)
- [ue::showSearchReplaceDialog](#)
- [ue::updateCanvas](#)
- [iue::align](#)
- [iue::boolean](#)
- [iue::copy](#)
- [iue::debug](#)

- [iue::fill](#)
 - [iue::move](#)
 - [iue::remove](#)
 - [iue::setOrigin](#)
 - [iue::size](#)
 - [iue::stretch](#)
-

ue::cndbg

Description

Invokes the Ciranova's debugger *cndbg* on the UDD PCell.

Syntax

```
ue::cndbg [-window <giWindow | int>]
```

Arguments

-window

Window Id.

Returns

<none>

ue::compile

Description

Compiles a PyCell for the current edit UDD design. The Python code for the cell resides in the <working_dir>/udd_cells/ directory.

Syntax

```
ue::compile [-window <giWindow | int>] [-cellView <dmCellView |  
dmCellViewName>]
```

Arguments

-window

Window Id.

`-cellView`

The cellview to be compiled

Returns

<none>

ue::debug

Description

Provides debug controls to the UDD assistant.

Syntax

```
ue::debug [-window <int>] [-action <enum>] [-verbosity <int>]  
          [-printToConsole <bool>] [-strictCDF <bool>] [-keepAll <bool>]
```

Arguments

`-window`

Window Id for the session.

`-action`

Specifies debug action. Valid values are clearStats, printDb, printExpressions, printFlags, printPy, and printStats.

`-verbosity`

Level of debug messages. The default is 0.

`-printToConsole`

Enables debug printing at console. The default is false.

`-strictCDF`

Strict handling of CDF parameter values. The default is true.

`-keepAll`

Retains all un-supported OA figs into the PyCell. The default is false.

Returns

<none>

ue::getDesign

Description

Returns the requested design Id.

Syntax

```
ue::debug [-window <giWindow | int>] [-type <enum>]
```

Arguments

-window

Window Id.

-type

Specifies the type of design used in the UDD session. Valid values: `preview` and `source`. The default is `source`.

Returns

<{<oaDesign>}>: OA Design Id.

ue::preview

Description

Previews an instance of Python PCell in a separate session.

Syntax

```
ue::preview [-window <giWindow | int>]
```

Arguments

-window

Window Id.

Returns

<none>

ue::searchAndReplace

Description

Finds `value` in all attribute expressions and replaces it with `newValue`.

Syntax

```
ue::searchAndReplace -window <giWindow | int> -value <string> -newValue  
    <string> [-scope <enum>]
```

Arguments

-window

Window Id.

-value

The value to be replaced.

-newValue

The new value as a replacement.

-scope

The scope of substitution ({ global | objects | operations | userParameters },
default is: global).

Returns

<none>

ue::showOptions

Description

Opens the **UDD Options** dialog box.

Syntax

```
ue::showOptions [-window <giWindow | int>]
```

Arguments

-window

Window Id.

Returns

<none>

ue::showSearchReplaceDialog

Description

Opens the **Search and Replace** dialog box.

Syntax

```
ue::showSearchReplaceDialog [-window <giWindow | int>]
```

Arguments

`-window`

Window Id.

Returns

<none>

ue::updateCanvas

Description

Updates canvas per object attribute defaults.

Syntax

```
ue::updateCanvas [-window <giWindow | int>]
```

Arguments

`-window`

Window Id.

Returns

<none>

iue::align

Description

Starts the interactive **Align** command and shows the procedure table in the **Operations** tab.

Syntax

```
iue::align [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

iue::boolean

Description

Starts the interactive **Boolean** command and shows the procedure table in the **Operations** tab.

Syntax

```
iue::boolean [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

iue::copy

Description

Starts the interactive **Copy** command and shows the procedure table in the **Operations** tab.

Syntax

```
iue::copy [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

iue::debug

Description

Starts the interactive UDD Debugger.

Syntax

```
iue::debug [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

iue::fill

Description

Starts the interactive **Fill** command and shows the procedure table in the **Operations** tab.

Syntax

```
iue::fill [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

iue::move

Description

Starts the interactive **Move** command and shows the procedure table in the **Operations** tab.

Syntax

```
iue::move [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

iue::remove

Description

Starts the interactive **Remove** command and shows the procedure table in the **Operations** tab.

Syntax

```
iue::remove [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

iue::setOrigin

Description

Starts the interactive **Set Origin** command.

Syntax

```
iue::setOrigin [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

iue::size

Description

Starts the interactive **Size** command and shows the procedure table in the **Operations** tab.

Syntax

`iue::size [<context: deContext | unsigned int>]`

Arguments

`context`

Context of activation.

Returns

`<none>`

iue::stretch

Description

Starts the interactive **Stretch** command and shows the procedure table in the **Operations** tab.

Syntax

`iue::stretch [<context: deContext | unsigned int>]`

Arguments

`context`

Context of activation.

Returns

`<none>`

Preferences

[Table 205](#) lists ue namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 205 UDD Editing Preferences

Name	Type	Scope	Default	Description
ueAlignMode	string	cell	Relative	Specifies the spacing mode for align operation. Valid values: Relative, Absolute
ueAlignSelectType	string	user	Edge	Specifies the selection type for align operation. Valid values: Edge, Point

Table 205 UDD Editing Preferences (Continued)

Name	Type	Scope	Default	Description
ueBooleanOperand	ueBooleanOperand	user	first	Specifies the operand for boolean operation. Valid values: first, second
ueDebuggerStepSize	int	user	1	Specifies the step size for previous and next debugger actions.
ueEnableDebugger	bool	user	false	Enables debugging for generated PyCells. Valid values: true, false
ueEnableFinShapes	bool	cell	false	Enables fin shapes in UDD object list. Valid values: true, false
ueEnableHierarchicalCallbacks	bool	user	false	Enables supporting callbacks on instances. Valid values: true, false
ueFillOperand	ueFillOperand	user	source	Specifies the operand for fill operation. Valid values: source, reference
uePostProcName	string	user		Specifies the python post procedure name to be called from generated PyCells.
uePycellPackageMode	string	user	Cell	Specifies the mode of PyCell packaging. Valid values: Cell, Library
ueRetainPythonCode	bool	user	false	Keeps the generated Python code in user's work area. Valid values: true, false
ueStoreVersionedViews	bool	user	false	Enables the creation of udd_cellviews. Valid values: true, false
ueTargetLibName	string	user		Specifies the name of pre-existing target library for UDD PyCells.
ueUserPythonFiles	string	user		Specifies the libraries to be imported in the generated PyCells.
ueVersionParamName	string	cell		Specifies the parameter name for PyCell versioning.

28

Integrated Tool Tcl

This chapter lists commands, enumerations, objects, and preferences in the Custom Compiler Tcl xt and ixt namespaces. These namespaces provide Tcl for netlisting, general Custom Compiler environment based simulation, parasitic extraction, physical verification, and external tool integration. Interactive commands are listed at the end of the chapter.

For details, see:

- [Commands](#)
- [Enumerations](#)
- [Objects](#)
- [Preferences](#)

Commands

The commands covered are:

- [xt::addJobClassPrefix](#)
- [xt::addPlotPacket](#)
- [xt::annotateParasitics](#)
- [xt::cdplJobProc](#)
- [xt::cdplJobsProc](#)
- [xt::checkLegality](#)
- [xt::checkLVS](#)
- [xt::checkPGConnectivity](#)
- [xt::checkPGMissingVias](#)
- [xt::checkRoutability](#)
- [xt::checkRoutes](#)

- [xt::clearPlotPackets](#)
- [xt::clearViolations](#)
- [xt::closeSimRun](#)
- [xt::closeTextViewerTab](#)
- [xt::compareVersionString](#)
- [xt::configureCDPL](#)
- [xt::configureQueueEngine](#)
- [xt::connectProgressServer](#)
- [xt::createCDPL](#)
- [xt::createDebugSession](#)
- [xt::createFCFrame](#)
- [xt::createGPDRef](#)
- [xt::createICC2Frame](#)
- [xt::createICCFRAM](#)
- [xt::createIntegration](#)
- [xt::createJob](#)
- [xt::createJobClass](#)
- [xt::createJobGroup](#)
- [xt::createParDBfromQuery](#)
- [xt::createPDKConfig](#)
- [xt::createPlot](#)
- [xt::createPlotQueueItem](#)
- [xt::createPlotViewer](#)
- [xt::createPlotViewerDef](#)
- [xt::createPlotViewerInst](#)
- [xt::createPveJob](#)
- [xt::createPWJobClass](#)

- [xt::createQueueEngine](#)
- [xt::createSimRun](#)
- [xt::createSimulator](#)
- [xt::createTool](#)
- [xt::crossprobe](#)
- [xt::displayPlotSet](#)
- [xt::editFinalNetlist](#)
- [xt::editTemplate](#)
- [xt::exec3DIC](#)
- [xt::execICC](#)
- [xt::execICC2](#)
- [xt::execTerminal](#)
- [xt::executePve](#)
- [xt::exportFloorplan](#)
- [xt::exportToICC](#)
- [xt::exportToReport](#)
- [xt::findExecutable](#)
- [xt::findFeature](#)
- [xt::findFeatureManifest](#)
- [xt::generateSimOutputCbK](#)
- [xt::getAnalysisList](#)
- [xt::getCalibreSocketPort](#)
- [xt::getCallingJob](#)
- [xt::getCrossRefs](#)
- [xt::getCDPLStatus](#)
- [xt::getDebugSessions](#)
- [xt::getEquivalentNets](#)

- [xt::getGPDCorners](#)
- [xt::getGPDCouplingCapacitors](#)
- [xt::getGPDGroundCapacitors](#)
- [xt::getGPDLayers](#)
- [xt::getGPDOObjects](#)
- [xt::getGPDRefs](#)
- [xt::getGPDResistors](#)
- [xt::getInputFiles](#)
- [xt::getInputFilesTimeStamps](#)
- [xt::getIntegrations](#)
- [xt::getJobClasses](#)
- [xt::getJobClassOptionPrefName](#)
- [xt::getJobHostStatus](#)
- [xt::getJobs](#)
- [xt::getManifestFeatureValue](#)
- [xt::getNetlistInputFiles](#)
- [xt::getOpenFile](#)
- [xt::getOutOfDateFiles](#)
- [xt::getPackages](#)
- [xt::getPDKConfigs](#)
- [xt::getPDKLibs](#)
- [xt::getPlotViewers](#)
- [xt::getPveJobs](#)
- [xt::getPWJobClasses](#)
- [xt::getQueueEngines](#)
- [xt::getRunDirName](#)
- [xt::getSaveFile](#)

- [xt::getSelectedJobGroups](#)
- [xt::getSelectedJobs](#)
- [xt::getSimRuns](#)
- [xt::getSimulators](#)
- [xt::getTextViewerActiveTab](#)
- [xt::getTextViewerTabs](#)
- [xt::getTools](#)
- [xt::getWalkupPort](#)
- [xt::hideJobMonitorColumn](#)
- [xt::highlightViolation](#)
- [xt::initCalibreConnection](#)
- [xt::initializeQueueEngine](#)
- [xt::initIntegration](#)
- [xt::initRunDir](#)
- [xt::initSimulatorIntegration](#)
- [xt::invokeExecutableForVersion](#)
- [xt::isValidRunDir](#)
- [xt::launchJobs](#)
- [xt::launchPlotViewer](#)
- [xt::launchSimulatorCbK](#)
- [xt::launchViewerCbK](#)
- [xt::loadPackage](#)
- [xt::loadPlotSet](#)
- [xt::loadPlugin](#)
- [xt::loadPvePrefs](#)
- [xt::loadRunDir](#)
- [xt::loadSimRun](#)

- [xt::openFeatureManifest](#)
- [xt::openTextViewer](#)
- [xt::openTextViewerTab](#)
- [xt::plotWaveform](#)
- [xt::postProcessSimResults](#)
- [xt::processLPEResults](#)
- [xt::processMarkupTags](#)
- [xt::queryParasitics](#)
- [xt::quitPvDebugger](#)
- [xt::readGPDParasitics](#)
- [xt::readJob](#)
- [xt::refreshTextViewer](#)
- [xt::reloadJobs](#)
- [xt::removeGPDParasitics](#)
- [xt::removeJobClassPrefix](#)
- [xt::reportGPDConfig](#)
- [xt::reportGPDProperties](#)
- [xt::resetGPDConfig](#)
- [xt::runMultiplePveJobs](#)
- [xt::runNetListAndSimulate](#)
- [xt::runNetlister](#)
- [xt::runNetlisterCbK](#)
- [xt::runSimulator](#)
- [xt::runSimulatorCbK](#)
- [xt::saveJobClassOption](#)
- [xt::savePlotSet](#)
- [xt::saveSimRun](#)

- [xt::sendCommand](#)
- [xt::setGPDConfig](#)
- [xt::setGPDPathForRead](#)
- [xt_setSaveRequiredValue](#)
- [xt::show3DICLibraryAttributes](#)
- [xt::show3DICRefLibEditor](#)
- [xt::showCapReportDialog](#)
- [xt::showCheckRoutes](#)
- [xt::showColorDecompositionSetup](#)
- [xt::showCreateFrame](#)
- [xt::showCreateICC2Design](#)
- [xt::showCreateICCFRAM](#)
- [xt::showCreateNewClass](#)
- [xt::showCreateSchematicFromVerilog](#)
- [xt::showCrossTalkNetSelector](#)
- [xt::showDRCSetup](#)
- [xt::showExport](#)
- [xt::showFCFrontEndFlow](#)
- [xt::showFCLibraryAttributes](#)
- [xt::showFormalityVerification](#)
- [xt::showICC2LibraryAttributes](#)
- [xt::showICC2RefLibEditor](#)
- [xt::showICCPlaceAndRoute](#)
- [xt::showInitializeParasitics](#)
- [xt::showInitLoad](#)
- [xt::showInitNew](#)
- [xt::showJobClassesDefinition](#)

- [xt::showJobHostStatus](#)
- [xt::showJobManagementOptions](#)
- [xt::showJobMonitor](#)
- [xt::showLPESetup](#)
- [xt::showLVSSetup](#)
- [xt::showNetBasedCheck](#)
- [xt::showNetBasedShieldCheck](#)
- [xt::showNetBasedViaCheck](#)
- [xt::showOpenICC2](#)
- [xt::showParasiticsCompare](#)
- [xt::showParasiticsOptions](#)
- [xt::showPveAboutTools](#)
- [xt::showPVEDialog](#)
- [xt::showPveOptions](#)
- [xt::showPveResults](#)
- [xt::showQueryParasiticsReport](#)
- [xt::showReport](#)
- [xt::showRun3DIC](#)
- [xt::showRunICC2](#)
- [xt::showSessionJobOptions](#)
- [xt::showSetCalibreHighlightColorDialog](#)
- [xt::showTestbenchJobOptions](#)
- [xt::showValidateForDigital](#)
- [xt::showValidateForICC](#)
- [xt::signalJob](#)
- [xt::splitVersionString](#)
- [xt::startPvDebugger](#)

- [xt::testPackage](#)
- [xt::unloadPlugin](#)
- [xt::updateCDPL](#)
- [xt::updateJob](#)
- [xt::validateForDigital](#)
- [xt::validateForICC](#)
- [xt::viewRunLog](#)
- [xt::viewSimInput](#)
- [xt::viewSimOutput](#)
- [xt::wait](#)
- [xt::writeJob](#)
- [xt::writeParasitics](#)
- [ixt::crossProbe](#)
- [ixt::editPlotSet](#)
- [ixt::showGPDParasiticsOptions](#)
- [ixt::showGPDParasiticsOptions](#)
- [ixt::plotWaveform](#)
- [ixt::queryGPDParasitic](#)
- [ixt::queryParasitics](#)

xt::addJobClassPrefix

Description

Adds a job prefix to the given job class.

Syntax

```
xt::addJobClassPrefix <prefix: string> -jobClass <xtJobClass>
```

Arguments

`-prefix`

The job ID prefix to add to the job class.

`-jobClass`

The `xtJobClass` object.

Returns

<none>

xt::addPlotPacket

Description

Adds another display resource packet to the list of plotting packets. These packets are used as a color wheel by the system to uniquely mark probes in a design as well as draw the waveform with the same respective color. This list of packets is cycled through when using `ixt::plotWaveform` and `ixt::editPlotSet` commands.

Syntax

```
xt::addPlotPacket -packet <packet: string> [-position <int>]
```

Arguments

`-packet <string>`

The name of a packet, which should be defined in the default display at the time of plotting.

`-position <int>`

The position at which to insert this new packet into the list. Negative numbers append the packet to the end of the list.

Default: Append to the end of the list.

Returns

<none>

Examples

```
xt::addPlotPacket annotatel -position 1
```

This example adds the name of a display resource packet to the list of plotting packets.

xt::annotateParasitics

Description

Enables and disables parasitic annotations in the current context.

Syntax

```
xt::annotateParasitics [<enable: bool>] [-window <giWindow | int>]
```

Arguments

<bool>

Toggles the viewing of parasitic annotation values to the current cellview.

Default: True.

enable

Enables viewing of parasitic annotation values to the current cellview.

Default: true.

-window

Schematic design editor window.

Returns

<none>

xt::cdplJobProc

Description

Creates CDPL Master.

Syntax

```
xt::cdplJobProc -procType <int> -CDPL <xtCDPL> [-job <xtJob>] [-progInfo  
<string>]
```

Arguments

-procType

The process type for CDPL task.

-CDPL

The xtCDPL object.

-job

The xtJob object.

-progInfo

Progress Info: hostname|port.

Returns

<int> : The return code of CDPL create or update, < 0 means fail, for add_job, it's job_vector_index

xt::cdplJobsProc

Description

CDPL Jobs related processes.

Syntax

```
xt::cdplJobsProc -procType <int> -CDPL <xtCDPL> [-jobs <dbCollection>]
[-inKilling <int>]
```

Arguments

-procType

The process type for CDPL task.

-CDPL

The xtCDPL object.

-jobs

The xtJob objects list.

-inKilling

In killing flag.

Returns

<int> : The return code of CDPL jobs process < 0 means fail, 0 means success.

xt::checkLegality

Checks the legality of the current placement in digital tool.

Syntax

```
xt::checkLegality -design <oaDesign> [-verbose <bool>] [-outputTcl
<bool>] [-infoViolation <bool>] [-cellViolation <bool>] [-maxInfo <int>]
[-checkDistance <float>] [-cells [<string>]]
```

Arguments

-design

Design for which legality check should be performed.

`-verbose`

Prints a detailed report of all possible violations. (Default is: false)

`-outputTcl`

Specifies whether need to write a Tcl file that contains all illegal cells with different violation types. (Default is: false)

`-infoViolation`

Specifies whether need to return check legality violation info. (Default is: false)

`-cellViolation`

Specifies whether need to return check legality violation cell. (Default is: false)

`-maxInfo`

Specifies whether need to return maximum violation info. (Default is: 0)

`-checkDistance`

Specifies the checking distance in number row heights. (Default is: 0.0)

`-cells`

Specifies the cells to check.

Returns

<none>

xt::checkLVS

Checks for inconsistencies in the physical layout of the current design in digital tool.

Syntax

```
xt::checkLVS -design <oaDesign> [-nets [<string>]] [-checkChildCells  
<bool>] [-checkZeroSpacingBlockages <bool>] [-checkTopLevelBlockages  
<bool>] [-reportFloatingPins <bool>] [-treatTerminalAsVoltageSource  
<bool>] [-maxErrors <int>] [-openReporting <enReporting>]  
[-excludeChildCellTypes <enCellTypes>] [-checks [<enCheckTypes>]]
```

Arguments

`-design`

Design for which LVS check should be performed.

`-nets`

Collection of nets to check. By default, all nets are checked.

`-checkChildCells`

Specifies whether to check shorts between child cells and top-level shapes.
(Default is: false)

`-checkZeroSpacingBlockages`

Specifies whether to check shorts on zero spacing blockages. (Default is: false)

`-checkTopLevelBlockages`

Specifies whether to check shorts on top-level blockages. (Default is: true)

`-reportFloatingPins`

Specifies whether to report floating pins when checking open or floating_routes violations. (Default is: false)

`-treatTerminalAsVoltageSource`

Specifies whether all terminals of pg nets are voltage sources. (Default is: false)

`-maxErrors`

Specifies the maximum number of errors for each type of violations. (Default is: 20)

`-openReporting`

Specifies how open violations are shown in the error browser.

`-excludeChildCellTypes`

Specifies cell types to ignore when checking shorts between child cell and top-level shapes.

`-checks`

Specifies check types: open, short, floatingRoutes or all.

Returns

<none>

xt::checkPGConnectivity

Verifies physical connectivity of the power ground network in digital tool.

Syntax

```
xt::checkPGConnectivity -design <oaDesign> [-nets [<string>]] [-netGroups  
  [<string>]] [-writeConnectivityFile <string>] [-errorViewName  
  <string>] [-checkMacroPins <enCheckPins>] [-checkBlockPins  
  <enCheckPins>] [-checkPadPins <enCheckPins>] [-checkStdCellPins  
  <enCheckPins>] [-maxFloatingClusterSize <int>]
```

Arguments

`-design`

Design for which PG connectivity check should be performed.

`-nets`

Specifies the PG nets to be checked. By default, the command checks all PG nets.

`-netGroups`

Specifies the group of PG nets to be checked. By default, the command checks all PG nets.

`-writeConnectivityFile`

Specifies the output file name. If specified, the tool writes detailed net connectivity information to the file. (Default is:)

`-errorViewName`

Specifies the error view name. If not specified, the default error view name will be `<block_name>_floatingPG.err`. (Default is:)

`-checkMacroPins`

Specifies how macro pins are checked for the specified nets.

`-checkBlockPins`

Specifies how block pins are checked for the specified nets.

`-checkPadPins`

Specifies how pad pins are checked for the specified nets.

`-checkStdCellPins`

Specifies how std cell pins are checked for the specified nets.

`-maxFloatingClusterSize`

Specify the maximum size of floating cluster to be returned by the command and reported by the error browser. (Default is: 0)

Returns

`<none>`

xt::checkPGMissingVias

Checks missing vias within the power and ground network in digital tool.

Syntax

```
xt::checkPGMissingVias -design <oaDesign> [-nets [<string>]] [-netGroups  
 [<string>]] [-outputFile <string>] [-viaRuleFile <string>]  
 [-writeDefaultViaRuleFile <bool>] [-ignoreShieldRoute <bool>]  
 [-ignoreSmallIntersections <bool>] [-honorRoutingBlockage <bool>]
```

Arguments

-design

Design for which PG missing vias check should be performed.

-nets

Specifies the PG nets to be checked. By default, the command checks all PG nets.

-netGroups

Specifies the group of PG nets to be checked. By default, the command checks all PG nets.

-outputFile

Specifies the output file name. (Default is:)

-viaRuleFile

Specifies the via rule file which includes all the via rules to be checked. (Default is:)

-writeDefaultViaRuleFile

Specifies whether need to write the default via rule to defaultViaRule.txt. (Default is: false)

-ignoreShieldRoute

Specifies whether not need to check vias against shapes with shape use shield_route. (Default is: false)

-ignoreSmallIntersections

Specifies whether not need to report error if the via site is smaller than the minimum contact code on the layer. (Default is: false)

-honorRoutingBlockage

Specifies whether not need to report error if the via site is covered by routing blockages. (Default is: false)

Returns

<none>

xt::checkRoutability

Performs routability check in digital tool. The checking is for many known issues that degrade the performance or quality of routing commands.

Syntax

```
xt::checkRoutability -design <oaDesign> [-checkStandardCellBlockedPorts  
<bool>] [-checkNonStandardCellBlockedPorts <bool>] [-checkPGBlockedPorts  
<bool>] [-checkFrozenNetBlockedPorts <bool>] [-obeyAccessEdges <bool>]  
[-accessEdgeWholeSide <bool>] [-obeyDirectionPreference <bool>]  
[-reportNoAccessEdge <bool>] [-blockedRange <int>] [-blockedRangeViaSide  
<int>] [-standardCellSearchRange <int>] [-checkCongestionMapOverflow  
<int>] [-checkMinGrid <bool>] [-checkOutOfBoundary <bool>] [-checkShield  
<bool>] [-allowViaRotation <bool>] [-connectStandardCellsWithinPins  
<bool>] [-checkNoNetPins <bool>] [-honorLayerConstraints <bool>]  
[-checkViaCutBlockage <bool>] [-checkLibViaCutBlockage <bool>]  
[-checkRoutingTrackSpace <bool>] [-checkRedundantPGShapes <bool>]  
[-checkRealMetalBlockageOverlapPin <bool>] [-viaLadder <bool>]
```

Arguments

-design

Design for which routability check should be performed.

-checkStandardCellBlockedPorts

Controls whether need to check blocked standard cell ports. (Default is: true)

-checkNonStandardCellBlockedPorts

Controls whether need to check blocked macro, top-level, and non-standard-cell ports. (Default is: true)

-checkPGBlockedPorts

Controls whether need to check blocked power and ground ports. (Default is: false)

-checkFrozenNetBlockedPorts

Controls whether need to check blocked frozen net ports. (Default is: false)

-obeyAccessEdges

Controls whether pins can be connected only at their access edges. (Default is: false)

-accessEdgeWholeSide

Controls the access points when access edges are obeyed. (Default is: true)

`-obeyDirectionPreference`

Controls whether only preferred direction access is allowed. (Default is: false)

`-reportNoAccessEdge`

Controls whether need to report the unconnected pins without predefined access edges. (Default is: false)

`-blockedRange`

Specifies the number of pitches for the same-layer accessibility check for macro and top-level ports. (Default is: 10)

`-blockedRangeViaSide`

Specifies the number of pitches for the accessibility check on the other side of an access via. (Default is: 10)

`-standardCellSearchRange`

Specifies the number of pitches for the same-layer accessibility search of standard cell pins. (Default is: 2)

`-checkCongestionMapOverflow`

Checks global route congestion map overflow larger than or equal to the given value. (Default is: 0)

`-checkMinGrid`

Controls whether need to check and report design objects that are not on the user (manufacturing) minimum grid. (Default is: true)

`-checkOutOfBoundary`

Controls whether need to check and report the pins that are outside of the design boundary. (Default is: true)

`-checkShield`

Controls whether need to perform check on the routing shapes related to the shielding flow. (Default is: true)

`-allowViaRotation`

Controls whether the tool can use rotated vias at pin connections. (Default is: true)

`-connectStandardCellsWithinPins`

Controls whether via connections to standard cell pins must be fully inside the pins. (Default is: false)

`-checkNoNetPins`

Controls whether need to check the accessibility for pins without an assigned net. (Default is: false)

`-honorLayerConstraints`

Controls whether need to consider the layer constraints when checking the accessibility of ports and the connectivity of nets. (Default is: true)

`-checkViaCutBlockage`

Controls whether need to check incorrect real metal via cut blockages in the design. (Default is: true)

`-checkLibViaCutBlockage`

Controls whether need to check incorrect real metal via cut blockages in the reference libraries. (Default is: false)

`-checkRoutingTrackSpace`

Controls whether need to check PG via obstacles in the space around each routing track by using the default or nondefault spacing requirement. (Default is: false)

`-checkRedundantPGShapes`

Controls whether need to check redundant PG shapes that overlap at the same location. (Default is: true)

`-checkRealMetalBlockageOverlapPin`

Controls whether need to check real metal routing blockages that touch or overlap a pin inside the reference libraries. (Default is: false)

`-viaLadder`

Controls whether need to perform the via ladder flow sanity checks. (Default is: false)

Returns

<none>

xt::checkRoutes

Performs routes check in digital tool. The command verifies and reports routing design rule checking (DRC) violations, net opens, antenna rule violations, voltage-area rule violations, and via conversion rates.

Syntax

```
xt::checkRoutes -design <oaDesign> [-nets [<string>]] [-drc  
<bool>] [-openNet <bool>] [-antenna <bool>] [-voltageArea <bool>]  
[-checkFromUserShapes <bool>] [-checkFromFrozenShapes <bool>]  
[-reportAllOpenNets <bool>]
```

Arguments

-design

Design for which routes check should be performed.

-nets

Collection of nets to check for opens, antenna rules, voltage area violations.

-drc

Determines whether design should be checked for design rule violations.
(Default is: true)

-openNet

Determines whether design should be checked for open nets. (Default is: true)

-antenna

Determines whether design should be checked for charge-collecting antenna
rule violations. (Default is: true)

-voltageArea

Determines whether design should be checked for voltage-area rule violations.
(Default is: true)

-checkFromUserShapes

Determines whether design should be checked for DRC violations for user-
created shapes, as well as routing shapes. (Default is: true)

-checkFromFrozenShapes

Determines whether design should be checked for DRC violations for frozen
shapes, as well as routing shapes. (Default is: true)

-reportAllOpenNets

Determines whether all open net should be reported. If false maximum of 200
open nets will be reported. (Default is: true)

Returns

<none>

xt::clearPlotPackets

Description

Clears out the entire list of plotting packets.

Syntax

```
xt::clearPlotPackets
```

Returns

<none>

xt::clearViolations

Description

Removes all violation highlights within the specified design, optionally restricted by a given tool name.

Syntax

```
xt::clearViolations -design <oaDesign> [-toolName <string>]
```

Arguments

`-design` *oaDesign*

Design in which violations are to be cleared.

`-toolName` <string>

Optionally clears only those violations created by the specified tool. If unspecified, all violations are cleared.

Default: "".

Returns

<none>

xt::closeSimRun

Description

Performs the cleanup and shutdown procedures on the run directory. Original bindkeys will be restored for the context. The context passed in will be removed from the xtSimRun::contexts list. If the xtSimRun is no longer associated with any context, the

simulator object and any viewer objects are freed. If no contexts are given, all contexts from the `xtSimRun` will be removed.

Syntax

```
xt::closeSimRun <simRun: xtSimRun> [-context <deContext>]
```

Arguments

`simRun`

The `simrun` object.

`-context`

The context associated with this `xtSimRun` that is being closed. This context should be in the `xtSimRun::contexts` list.

Default: Nil.

Returns

<none>

xt::closeTextViewerTab

Description

Closes the named tab in a Text Viewer. Note: closing a tab changes the indexes for all later tabs.

Syntax

```
xt::closeTextViewerTab <tabname: string> -window <giWindow>
```

Arguments

<string>

The name of the tab to close.

`-window` `giWindow`

The Text Viewer window.

Returns

<none>

xt::compareVersionString

Description

Compares one Synopsys tool version string with another.

Syntax

```
xt::compareVersionString <firstVersionString: string> -to <string>
```

Arguments

`firstVersionString`

First version string.

`-to`

Second version string.

Returns

`<string>`: -1 if the first version string is lower (older) than the second, 0 if they are equal, or 1 if otherwise.

xt::configureCDPL

Description

Configures CDPL Master.

Syntax

```
xt::configureCDPL -CDPL <xtCDPL> [-slotInfo <string>] [-initialSetting  
<string>] [-scheduleRule <string>] [-extraInfo <string>]
```

Arguments

`-CDPL`

The new CDPL Master.

`-slotInfo`

Slot option needed.

`-initialSetting`

Timeout info & Ratio of acquiring slots.

`-scheduleRule`

Rule for scheduling tasks.

`-extraInfo`

Extra information.

Returns

`<int>` : The return code of configure CDPL. Lesser than 0 indicates FAIL.

xt::configureQueueEngine

Description

Configures queue engine settings for the given job class.

Syntax

```
xt::configureQueueEngine <engine: string | xtQueueEngine> -jobClass  
  <string | xtJobClass> -parent <giDialog | giWindow | unsigned int>  
  [-submitHost <string>]
```

Arguments

`engine`

The queue engine to configure.

`-jobClass`

Configures the settings for this xtJobClass.

`-parent`

Parent window for the configuration dialog box.

`-submitHost`

Configures the settings for this submit host.

Default is: localhost.

Returns

`<none>`

xt::connectProgressServer

Connects the job to the specific progress server

Syntax

```
xt::_connectProgressServer -job <xtJob> -host <string> -progressPort  
  <unsigned int> -id <string> [-addr <string>]
```

Arguments

`-job`
 `job`

`-host`
 `Host`

`-progressPort`
 `Port`

`-id`
 `Server ID`

`-addr`
 `address (Default is: )`

Returns

`<none>`

xt::createCDPL

Description

Creates CDPL Master.

Syntax

```
xt::createCDPL <name: string> -jobClass <xtJobClass> -newMaster <bool>
```

Arguments

`name`
 The name for CDPL master.

`-jobClass`
 The xtJobClass object.

`-newMaster`
 Creates a new master.

Returns

`<dbCore>` : xtCDPL master object for the CDPL run.

xt::createDebugSession

Description

Creates an xtDebugSession object representing the state of an active debugger session.

Syntax

```
xt::createDebugSession (-job <string | xtJob> | -clientId <string>)  
  [-cellMap <string> | -layoutTopCellView <dmCellView>] [-nameMap  
  <nlnNameMap> | -schematicTopCellView <dmCellView>] -tool <string  
  | xtTool> [-rundir <string>] [-layoutEditCellView <dmCellView>]  
  [-schematicEditCellView <dmCellView>]
```

Arguments

-tool

The debugger tool.

-job

The job object representing the running debugger.

-clientId

Client identifier used by socket based connections to identify the running debugger.

-rundir

Verification results directory.

-cellMap

Verification layout cell to OpenAccess design map.

-layoutTopCellView

The top verification layout cell view.

-layoutEditCellView

The edit verification layout cell view.

-nameMap

Map object used to store schematic to netlist name mappings.

-schematicTopCellView

The top verification schematic cell view.

-schematicEditCellView

The edit verification schematic cell view.

Returns

<xtDebugSession>: Newly created xtDebugSession object.

xt::createFCFrame

Description

Creates Fusion Compiler frame views for the Custom Compiler design blocks (instances, macros, and I/O devices). The command uses the Custom Compiler design blocks as input (source), copies the designs into a Fusion Compiler library, generates a create-frame Tcl script, and runs the script in the Fusion Compiler tool. See About Frame Views and the Create Frame Tcl Script.

Syntax

```
xt::createFCFrame [-source [<{<dmCellView>}>]] [-script <string>]
[-targetLib <oaLib>] [-depth <int>] [-cellType <enCreateFrameCellType>]
[-siteDef <string>] [-blockAll <enBlockMode>] [-connectWithinPin
<bool>] [-hierarchicalExtraction <bool>] [-mergeBlockages <bool>]
[-preserveBlockages <enPreserveBlockages>] [-blockCoreMargin [<string>]]
[-maskColors [<string>]] [-extractAntenna <bool>] [-topCellPinOnly
<bool>] [-treatSourceDrainAsDiodes <bool>] [-generateSingleFileLib
<string>] [-timingFiles [<string>]] [-timingFilesScript <string>]
[-createUnmappedLPPs <bool>]
```

Arguments

-source

List of Custom Compiler cellviews for which you want to create Fusion Compiler frame views. The cellviews must belong to a single fileSys library.

-script

Path of the create-frame script that you want to run in the Fusion Compiler tool. If you do not specify the path, the default `fc_create_frame_template.tcl` script will be used.

-targetLib

Location where the script will be run and the Fusion Compiler frame view will be stored.

-depth

Maximum hierarchy depth for the design export. (Default: 32)

-cellType

Cellview type. (Default: block.)

`-siteDef`

Site definition. This argument is required for the core cell type.

`-blockAll`

Creates zero-spacing routing blockages in the frame view for the entire block, except for the area around the pins.

Valid values:

- `true`: Creates zero-spacing routing blockages on all layers.
- `false`: Does not create any zero-spacing routing blockages.
- `used_layers`: Creates zero-spacing routing blockages only on layers that are used by the cell.
- `topBlockedLayerName`: Creates zero-spacing routing blockages only on the specified layer and all the layers below it. The specified layer must be a metal or poly layer defined in the technology database.
- `auto`: Lets the tool determine when to create zero-spacing blockages based on the cell's design type. The tool sets `blockAll = true` for analog, black-box, and module design types. The tool sets `blockAll = used_layers` for macro cells. For all other design types, no zero-spacing blockages are created.

(Default: `auto`)

`-connectWithinPin`

Requires that the via enclosure be within the pin shape. (Default: `true`)

`-hierarchicalExtraction`

Extracts geometries in sub-blocks. (Default: `true`)

`-mergeBlockages`

Merges metal shapes and regular routing blockages to reduce the number of shapes. (Default: `false`)

`-preserveBlockages`

Preserves all metal shapes and routing blockages around pins as they exist in the design view. (Default: `auto`)

`-blockCoreMargin`

Creates zero-spacing routing blockages on the specified layers with the specified margin between the core blockage and the cell boundary.

Specify this argument using the following syntax: `{ { layer_name margin } ... }`

`-maskColors`

Number of color masks per layer. Specify this argument using the following syntax: { { layer_name number} ... }

`-extractAntenna`

Extracts antenna data. (Default: false)

`-topCellPinOnly`

Extracts top-cell pins only. (Default: true)

`-treatSourceDrainAsDiodes`

Treats the MOS drain as diodes. (Default: true)

`-generateSingleFileLib`

Path for single-file library generation.

`-timingFiles`

List of timing database files to load to the library defined by `generateSingleFileLib`.

When processing the timing database files, the argument uses either the default timing script, `fc_laod_timing.tcl`, or a timing script that you specify.

`-timingFilesScript`

Path to the script that will load the timing database files to run in the Fusion Compiler tool.

`-createUnmappedLPPs`

Creates unmapped LPPs in the Fusion Compiler technology database.

Returns

<none>

Examples

The command creates a frame view from the Custom Compiler `snps_mpll/msip_dcc/layout` and saves the frame view in the `test_fc` library. Because `blockAll` is `true`, the command creates zero-spacing routing blockages on all layers.

```
xt::createFCFrame -source [dm::getCellViews layout -libName snps_mpll  
-cellName msip_dcc] -targetLib [dm::getLibs test_fc] -blockAll true
```

xt::createGPDRef

Description

Creates association for GPD referencing.

Syntax

```
xt::createGPDRef [-schLCVName <string>] [-maskLCVName <string>]  
  -bindingType <string> [-schWinID <int>] [-maskWinID <int>] [-gpdPath  
  <string>] [-keepCrossCap <int>] [-lppMap <string>]
```

Arguments

-schLCVName

Schematic Lib Cell View Name (Default is:).

-maskLCVName

Mask Lib Cell View Name (Default is:).

-bindingType

Binding Type.

-schWinID

Schematic Window ID (Default is: -1).

-maskWinID

Mask Window ID (Default is: -1).

-gpdPath

Golden Parasitic Database Directory (Default is:).

-keepCrossCap

Keep Capacitive Coupling (Default is: -1).

-lppMap

LPP Map File (Default is:).

Returns

<bool> : Newly specified GPD path.

xt::createICC2Frame

Description

Creates IC Compiler II frame views for the Custom Compiler design blocks (instances, macros, and I/O devices). The command uses the Custom Compiler design blocks as input (source), copies the designs into an IC Compiler II library, generates a create-frame Tcl script, and runs the script in the IC Compiler II tool. See About Frame Views and the Create Frame Tcl Script.

Syntax

```
xt::createICC2Frame [-source [<{<dmCellView>}>]] [-script <string>]
  [-targetLib <oaLib>] [-depth <int>] [-cellType <enCreateFrameCellType>]
  [-siteDef <string>] [-blockAll <enBlockMode>] [-connectWithinPin
  <bool>] [-hierarchicalExtraction <bool>] [-mergeBlockages <bool>]
  [-preserveBlockages <enPreserveBlockages>] [-blockCoreMargin [<string>]]
  [-maskColors [<string>]] [-extractAntenna <bool>] [-topCellPinOnly
  <bool>] [-treatSourceDrainAsDiodes <bool>] [-generateSingleFileLib
  <string>] [-timingFiles [<string>]] [-timingFilesScript <string>]
  [-createUnmappedLPPs <bool>]
```

Arguments

-source

List of Custom Compiler cellviews for which you want to create IC Compiler II frame views. The cellviews must belong to a single fileSys library.

-script

Path of the create-frame script that you want to run in the IC Compiler II tool. If you do not specify the path, the default `icc2_create_frame_template.tcl` script will be used.

-targetLib

Location where the script will be run and the IC Compiler II frame view will be stored.

-depth

Maximum hierarchy depth for the design export. (Default: 32)

-cellType

Cellview type. (Default: block.)

-siteDef

Site definition. This argument is required for the core cell type.

`-blockAll`

Creates zero-spacing routing blockages in the frame view for the entire block, except for the area around the pins.

Valid values:

- `true`: Creates zero-spacing routing blockages on all layers.
- `false`: Does not create any zero-spacing routing blockages.
- `used_layers`: Creates zero-spacing routing blockages only on layers that are used by the cell.
- `topBlockedLayerName`: Creates zero-spacing routing blockages only on the specified layer and all the layers below it. The specified layer must be a metal or poly layer defined in the technology database.
- `auto`: Lets the tool determine when to create zero-spacing blockages based on the cell's design type. The tool sets `blockAll = true` for analog, black-box, and module design types. The tool sets `blockAll = used_layers` for macro cells. For all other design types, no zero-spacing blockages are created.

(Default: `auto`)

`-connectWithinPin`

Requires that the via enclosure be within the pin shape. (Default: `true`)

`-hierarchicalExtraction`

Extracts geometries in sub-blocks. (Default: `true`)

`-mergeBlockages`

Merges metal shapes and regular routing blockages to reduce the number of shapes. (Default: `false`)

`-preserveBlockages`

Preserves all metal shapes and routing blockages around pins as they exist in the design view. (Default: `auto`)

`-blockCoreMargin`

Creates zero-spacing routing blockages on the specified layers with the specified margin between the core blockage and the cell boundary.

Specify this argument using the following syntax: `{ { layer_name margin } ... }`

`-maskColors`

Number of color masks per layer. Specify this argument using the following syntax: { { layer_name number} ... }

`-extractAntenna`

Extracts antenna data. (Default: false)

`-topCellPinOnly`

Extracts top-cell pins only. (Default: true)

`-treatSourceDrainAsDiodes`

Treats the MOS drain as diodes. (Default: true)

`-generateSingleFileLib`

Path for single-file library generation.

`-timingFiles`

List of timing database files to load to the library defined by `generateSingleFileLib`.

When processing the timing database files, the argument uses either the default timing script, `icc2_laod_timing.tcl`, or a timing script that you specify.

`-timingFilesScript`

Path to the script that will load the timing database files to run in the IC Compiler II tool.

`-createUnmappedLPPs`

Creates unmapped LPPs in the IC Compiler II technology database.

Returns

<none>

Examples

The command creates a frame view from the Custom Compiler `snps_mpll/msip_dcc/layout` and saves the frame view in the `test_icc2` library. Because `blockAll` is true, the command creates zero-spacing routing blockages on all layers.

```
xt::createICC2Frame -source [dm::getCellViews layout -libName snps_mpll  
-cellName msip_dcc] -targetLib [dm::getLibs test_icc2] -blockAll true
```

xt::createICCFRAM

Description

Creates an IC Compiler FRAM View.

Spawns a batch job, using Milkyway, which creates an IC Compiler FRAM view. The cellView must already be in an ICC library. This command will create a FRAM view which is a sibling of the specified cellView. Use the [xt::exportToICC](#) command to export an OA design to IC Compiler.

Syntax

```
xt::createICCFRAM <source: dmCellView | dmCellViewName>  
  [-script <string>] [-cellType <string>] [-smashLevel <int>]  
  [-preserveMetalBlockage <bool>] [-mergeThresholds <string>]  
  [-createPRBoundary <bool>] [-siteDef <string>] [-layerParallelToPG  
  <string>] [-layerPerpendicularToPG <string>] [-sharePG <bool>]  
  [-pgOrient <string>] [-extractAntenna <bool>]
```

Arguments

source

IC Compiler layout cellView.

-script

Milkyway script which creates FRAM view.

-cellType

Cell type for FRAM creation.

Default is: macro.

-smashLevel

Smash level.

Default is: 0.

-preserveMetalBlockage

Indicates whether metal blockages should be preserved.

Default is: false.

-mergeThresholds

Specifies the merge thresholds: list of {layerName x y} for "macro", list of {layerName x asThin} for "core".

`-createPRBoundary`

Create PR Boundary.

Default is: false.

`-siteDef`

OA site definition name to determine tile size.

`-layerParallelToPG`

OA layer name of first metal layer parallel to PG.

`-layerPerpendicularToPG`

OA layer name of first metal layer perpendicular to PG.

`-sharePG`

Shares PG rails with createPRBoundary.

Default is: true.

`-pgOrient`

PG rail orientation.

Default is: horizontal.

`-extractAntenna`

Extracts Antenna information.

Default is: false.

Returns

<xtJob> : Returns the created FRAM batch job.

Errors

- Specified cellView is not from an IC Compiler library.
- Specified cellView is not a layout view.
- Unknown layer %s specified in `-mergeThresholds`.
- Site definition "%s" does not exist.
- `mergeThreshold` can be specified only when `preserveMetalBlockage` is *false*.

xt::createIntegration

Description

Creates and registers a simulator integration by name and returns the integration object. The created integration is initialized by calling xt::initIntegration. An integration can be created for a directory that does not exist, however, in this case, a warning message will be issued to the console.

Syntax

```
xt::createIntegration <name: string> -type <enum> -filePath <string>
```

Arguments

name

The name of the integration.

-type

The type of integration to be registered; note that only simulator is valid:
{simulator}.

-filePath

The full path to the Tcl source file that defines the integration.

Returns

The newly created xtIntegration object. (xtIntegration).

xt::createJob

Description

Starts a job. Launches the job as specified by -cmdLine on the given host of the type batch, foreign or native. The xtJob is tracked by the Job Monitor. The caller of this command must determine the invocation directory on the remote host.

The job name attribute of the xtJob will be generated by appending an underscore followed by a number to the given job name prefix. For example, if the prefix is "HSPICE", the job name will be "HSPICE_1", or "HSPICE_2" if "HSPICE_1" is already in use.

Syntax

```
xt::createJob <name: string> -type <enum> [-aimType <string>] -cmdLine  
<string> -runDesc <string> [-files [<string>]] [-defaultViewFile  
<string>] [-readOnlyFiles [<string>]] [-host <string>] [-data <string>]  
[-readyProc <dbExecObject | string | xtProcedure>] [-dataReadyProc  
<dbExecObject | string | xtProcedure>] [-menuProc <dbExecObject | string  
| xtProcedure>] [-exitProc <dbExecObject | string | xtProcedure>]
```

```
[-killOnExit <bool>] [-queueEngine <string>] [-group <xtJobGroup>]  
[-deferred <bool>] [-jobClass <xtJobClass>] [-gridOptions <string>]
```

Arguments

`name`

Prefix for the name to display in the job monitor.

`-type <string>`

Job type: ({aim | batch | dp | interactive | tclSocket}).

- **aim:** Interactive job type used for programs that integrate with the Custom Compiler environment using AimAppCom.
- **batch:** Runs processing to completion without manual intervention. Batch jobs are not terminated when the session exits. The Custom Compiler environment continued to track batch jobs when it is restarted.
- **dp:** Interactive job type used for programs that integrate with the Custom Compiler environment using Tcl/DP.
- **tclSocket:** interactiveJob type for programs that read and write messages from stdin and stdout. Jobs of this type are terminated when the session exits.

`-aimType`

Aim job type; integrator defined.

`-cmdLine <string>`

Command used to launch the job, using Bourne shell syntax.

`-runDesc <string>`

Descriptive text to show in the Job Monitor.

`-files {<string>}`

List of pairs (display name, file name) to track. All files in this list are deleted before the child process is launched.

`-defaultViewFile [=(<string>)]`

Name of the file that is set as the default active file tab in the viewoutput files action.

`-readOnlyFiles {<string>}`

List of read-only pairs to track, such as display name and file name. These files are not be deleted before the child process is launched.

`-host <string>`

Host on which to run the job. Default: Localhost.

`-data <string>`

Arbitrary string data associated with the job.

`-readyProc <string>`

Procedure to call after the job is initialized and ready for input. Interactive jobs only. Takes a single argument of the `xtJob` object.

`-dataReadyProc`

Procedure to call whenever there is data in the job's buffer.

`-menuProc`

Procedure to call when the context-sensitive menu is displayed.

`-exitProc <string>`

Procedure to call after the job has exited. This procedure is called regardless of how the job shut down. Takes a single argument of the `xtJob` object.

`-killOnExit`

Indicates the job should be terminated when the application shuts down. Default is 'false' for batch jobs and 'true' for all others.

`-queueEngine`

Specifies which registered queue engine to use to launch the job. If unspecified, the default is set to the value of the `xtJobQueueEngine` preference.

`-group`

The job group under which this job is added.

`-deferred`

Defer the job launch. Default is: false.

`-jobClass`

Job Class to use for the launch settings.

`-gridOptions`

Additional grid options (only used for queue-managed jobs).

Returns

Job object for communication and tracking. ([xtJob](#))

Errors

This command will not raise any errors.

Examples

```
set jobID [xt::createJob verilog2oa -type batch -cmdLine \  
    "verilog2oa -lib testLib -verilog test.v" -runDesc \  
    "Translate Verilog design into OA"]
```

In this example, `xt::createJob` calls the `verilog2oa` utility for converting verilog modules, which are described in the file `test.v`, into the OpenAccess database.

xt::createJobClass

Description

Create a job class.

Syntax

```
xt::createJobClass <name: string> [-prefName <string>] [-labels  
    [<string>]]
```

Arguments

`name`

The name of the job class.

`-prefName <string>`

Name to use in the preferences.

`-labels`

The Tcl list of arbitrary string labels for this job class.

Returns

`<xtJobClass>`: `xtJobClass` object for the job class.

Errors

This command will not raise any errors.

xt::createJobGroup

Description

Creates a job group.

Syntax

```
xt::createJobGroup <name: string> [-parent <xtJobGroup>] [-runDesc  
  <string>] [-detailedStatusProc <dbExecObject | string | xtProcedure>]  
  [-expanded <bool>] [-trackProgress [<bool>]] [-showProgress [<bool>]]  
  [-initOptionsDialogProc <dbExecObject | string | xtProcedure>]  
  [-exitProc <dbExecObject | string | xtProcedure>] [-data <string>]
```

Arguments

name

The name of the job group.

-parent

The parent job group.

-runDesc

Descriptive text to show in the Job Monitor.

-detailedStatusProc

Tcl procedure to produce the detailed text message.

-expanded

Group expands in the Job Monitor.

-trackProgress

Group tracks the job progress.

-showProgress

Group displays the job progress.

-initOptionsDialogProc

Tcl procedure to initialize the Job Management Options dialog for this filtered group.

-exitProc

Procedure to call after all jobs in the group have exited.

-data

Client data to associate with the job group.

Returns

<xtJobGroup>: The new xtJobGroup object.

xt::createParDBfromQuery

Description

Create database parasitics from query.

Syntax

```
xt::createParDBfromQuery <objects: {<deFigure>} | {<deSelectionSet>} |  
    {<oaFig>} | {<oaFigGroup>} | {<oaMarker>} | {<oaInst>} | {<oaShape>}  
    | {<oaVia>}} [-filename <string>] [-queried [<string>]] [-crossRef  
    <xtCrossRef>] [-eqvNets [<string>]]
```

Arguments

`objects`

Collection of parasitic instances.

`-filename`

File to store db (Default: parQuery.db).

`-queried`

List of queried nets.

`-crossRef`

CrossRef containing schematic and parasitic views to be probed.

`-eqvNets`

Tcl array of equivalent nets.

Returns

`<string>` : path to db file

xt::createPDKConfig

Description

Creates an xtPDKConfig object encapsulating the PDK configuration data for the specified PDK library.

Syntax

```
xt::createPDKConfig <name: string> (-lib <oaLib> | -libName <string>)  
    [-fileName <string>]
```

Arguments

`name`

Name or identifier.

`-lib`

PDK library.

`-libName`

PDK library name.

`-fileName`

PDK configuration file name.

Default: `pdk.xml`

Returns

`<xtPDKConfig>`: Newly created `xtPDKConfig` object.

xt::createPlot

Description

Creates an `xtPlot` object for plotting waveforms.

Syntax

```
xt::createPlot -name <string> -type <enPlotType> [-analysis <string>]  
               -packet <string>
```

Arguments

`-name`

The full hierarchical name of the property of interest.

`-type`

The type of the property being plotted (net, port, and so on).

`-analysis`

The name of the analysis to which this plot should apply. Plot sets do not have an analysis type and should be applied to all.

`-packet`

The display resource packet name to use when plotting.

Returns

The xtPlot created. (xtPlot)

Examples

```
set xtplot [xt::createPlot -name CLK -type net -analysis \  
    TRAN -packet annotatel]
```

In this example, xt::createPlot creates the xtPlot Tcl object for the transient measurement of net CLK.

xt::createPlotQueueItem

Description

Creates an xtPlotQueueItem object

Syntax

```
xt::createPlotQueueItem <signal|plot: xtPlot> -command <string> -mode  
    <enum>
```

Arguments

signal|plot

wfSignal or xtPlot object to plot.

-command

Command to plot.

-mode

Plotting mode: {append | new | replace}.

Returns

<xtPlotQueueItem>: Newly created xtPlotQueueItem

xt::createPlotViewer

Description

Creates an xtPlotViewer object. The xtPlotViewer must be further configured before use.

Syntax

```
xt::createPlotViewer <name: string>
```

Arguments

`name`

The name of the viewer integration.

Returns

The `xtViewer` created. (`xtPlotViewer`).

xt::createPlotViewerDef

Description

Creates an `xtPlotViewerDefinition` object.

Syntax

```
xt::createPlotViewerDef <name: string>
```

Arguments

`name`

Name of the `xtPlotViewerDefinition`.

Returns

<`xtPlotViewerDefinition`>: Newly created `xtViewerDefinition`

xt::createPlotViewerInst

Description

Creates an `xtPlotViewerInst` object.

Syntax

```
xt::createPlotViewerInst <viewerDef: xtPlotViewerDefinition>
```

Arguments

`viewerDef`

`xtPlotViewerDefinition` associated with the instance.

Returns

<`xtPlotViewerInst`>: Newly created `xtViewerInst`.

xt::createPveJob

Description

Creates an xtPveJob object.

Syntax

```
xt::createPveJob <name: string> [-title <string>] [-prefId <string>]  
  -jobType <enum> -toolDefault <string | xtTool> [-toolList [<string |  
  xtTool>]] [-windowTypes [<giWindowType | string>]]
```

Arguments

name

Name / identifier.

-title

Name as displayed in the interface.

Default: <name>.

-prefId

Identifier used to encode preference names.

Default: <name>.

-jobType

PVE Job type. ({ DFM | DRC | IDRC | LPE | LVS | LVSLPE | PERC })

-toolDefault

Default tool used to run the PVE job.

-toolList

List of tools capable of running the PVE job.

-windowTypes

List of window types in which PVE job related actions are available.

Returns

<xtPveJob>: Newly created xtPveJob object.

xt::createPWJobClass

Description

Creates a PrimeWave job class.

Syntax

```
xt::createPWJobClass <name: string> [-prefName <string>] [-labels  
    <string>]
```

Arguments

name

The name of the PrimeWave job class.

-prefName

Name to use in preferences.

-labels

Tcl list of arbitrary string labels for this job class.

Returns

<xtJobClass>: xtJobClass object for the PrimeWave job class.

xt::createQueueEngine

Description

Creates and registers a queue engine for starting batch jobs.

Syntax

```
xt::createQueueEngine <none: string> -initProc <dbExecObject | string  
    | xtProcedure> -createPrefsProc <dbExecObject | string | xtProcedure>  
-startProc <dbExecObject | string | xtProcedure> -statusProc  
    <dbExecObject | string | xtProcedure> -killProc <dbExecObject | string  
    | xtProcedure> -suspendProc <dbExecObject | string | xtProcedure>  
-resumeProc <dbExecObject | string | xtProcedure> -configurationProc  
    <dbExecObject | string | xtProcedure> [-arraysEnabledProc <dbExecObject  
    | string | xtProcedure>] [-launchArrayProc <dbExecObject | string |  
    xtProcedure>] [-bulkKillProc <dbExecObject | string | xtProcedure>]
```

Arguments

<none>

Name of the queue engine.

-initProc

Procedure called to check if the queue engine is available in the user's environment. If this procedure returns true for the particular engine, then this engine will be available to select for launching jobs. If the procedure returns false or raises an error, the engine is not available for launching jobs.

`-createPrefsProc`

This procedure is called when a `xtJobClass` is created to create the preferences used by that class.

`-startProc`

Procedure to call when starting a batch job using a queue engine. This procedure must have at least one argument and the final argument must be the shell command line for the job. This procedure must return an identifier provided by the queue engine to control and monitor the job. The returned identifier is stored in the corresponding `xtJob` `queueIdentifier` attribute.

`-statusProc`

Procedure to call to query the status of the queue-managed jobs. This procedure is called periodically to update the Job Monitor. Provided procedure must have at least one argument and the final argument must accept a list of running `xtJob` objects currently managed by this queue engine. This procedure is expected to call `xt::updateJob` on each `xtJob` in the list passed in.

`-killProc`

Procedure to call to terminate the queue-managed job (or remove it from the queue). This procedure must have at least one argument and its final argument must accept an `xtJob`.

`-suspendProc`

Procedure to suspend the queue-managed job. This procedure must have at least one argument and its final argument must accept an `xtJob`.

`-resumeProc`

Procedure to resume a suspended queue-managed job. This procedure must have at least one argument and its final argument must accept an `xtJob`.

`-configurationProc`

Procedure to call when the Configure button is pressed in the Job Options dialog. It should be used to spawn an Opal-based GUI where users can make changes to settings for the queue engine.

`-arraysEnabledProc`

Procedure to call to check if job arrays are enabled.

`-launchArrayProc`

Procedure to call to launch a collection of jobs as a job array.

`-bulkKillProc`

Procedure to call to terminate several jobs simultaneously.

Returns

<xtQueueEngine>: The new queue engine.

xt::createSimRun

Description

Creates a new xtSimRun object. The directory provided to the filePath argument will be the run directory used by this xtSimRun.

Syntax

```
xt::createSimRun -dir <string> [-cellView <dmCellView>] [-context  
  <deContext>] -simulator <string> [-netlistLanguage <string>] [-packet  
  <string>]
```

Arguments

-dir

The directory to be associated with the new xtSimRun.

-cellView

The cellview to be associated with the new xtSimRun. It contains the top design for netlisting. Either cellView or a context is required.

-context

The context to be associated with the new xtSimRun. The cellview of the context's top design will be the cellView associated with the xtSimRun. Either context or -cellview must be provided.

-simulator

The name of the simulator integration to be associated with the new xtSimRun.

-netlistLanguage

The netlist language to be used with the new xtSimRun. This will typically be set by the integrator during the integration.

-packet

Alias name for the display resource packet.

Returns

The xtSimRun created. (xtSimRun)

Examples

```
xt::createSimRun -context [de::getActiveContext] -dir sim \  
    -simulator HSPICE
```

In this example, `xt::createSimRun` creates a new `xtSimRun` Tcl object for the HSPICE simulator.

xt::createSimulator

Description

Creates an `xtSimulator` object. The returned `xtSimulator` must be further configured before use.

Syntax

```
xt::createSimulator <name: string>
```

Arguments

`name`

The name of the simulator integration.

Returns

The `xtSimulator` created. (`xtSimulator`)

xt::createTool

Description

Creates an `xtTool` object. Used to create a new physical verification tool for custom flows.

Syntax

```
xt::createTool <name: string> [-title <string>] [-prefId <string>]  
    -namespace <string> [-binary <string>] [-walkupToken <string>] [-aliases  
    [<string>]]
```

Arguments

`name`

Name or identifier.

`-title`

Name as displayed in the interface.

Default: `<name>`

`-prefId`

Identifier used to encode preference names.

Default: <name>

`-namespace`

Fully qualified Tcl namespace for named interface procedures.

`-binary`

Command line name used to execute the tool.

Default: <name>

`walkupToken`

Token to identify the tool during a walk-up Tcl-DP connection.

`-aliases`

Tcl list of alias names consulted for matching user input.

Returns

<xtTool>: Newly created xtTool object.

xt::crossprobe

Description

Creates a probe in the schematic and parasitic views, if both views are open. If only one view is opened, only that view is probed and no messages are given. Device or net cross probes can be created. The clear option clears all probes. If a device or net is specified with the clear option, all probes are cleared and then a new one is created.

Syntax

```
xt::crossprobe [-type <enProbeType>] [-object <oaNet | oaInst>] [-clear  
  <bool>] -crossRef <xtCrossRef> [-packet <string>] [-bus <enum>]
```

Arguments

`-type`

Type of probe to create: {device | net}

`-object` `oaNet` | `oaInst`

Specifies net or instance to use to create the crossprobes.

`-context`

Context of the object to be probed.

`-clear <bool>`

Specifies whether existing probes should be cleared. (Default: `false`).

`-crossRef`

Cross references schematic and parasitic views that are to be probed.

`-packet`

Displays the alias of the resource packet.

`-bus`

Cross probes a bus net.

Returns

Collection of probes created in the schematic or parasitic views. ([deProbe](#))

Errors

- Invalid net or instance.
- Missing net or device required for the specified probe type.

Examples

```
set h1 [he::createContext -cellView [dm::getCellViews \  
    schematic -cellName inv -libName testLib]]  
set h2 [he::createContext -cellView [dm::getCellViews \  
    starrcxt -cellName inv -libName testLib]]  
set cr [xt::createCrossRef -schematicContext $h1 \  
    -parasiticContext $h2 -schematicWindowID 1 \  
    -parasiticWindowID 2 -parasiticRunDir inv.starrcxt.lpe  
    -parasiticTool starrcxt]  
set oaNet [db::getNets a -of [oa::DesignFind testLib inv \  
    starrcxt]]  
xt::crossProbe -crossRef $cr -type net -object $oaNet -context $h2
```

In this example, `xt::crossProbe` creates a probe in the schematic and parasitic views for net `a`, if the views are both opened.

xt::displayPlotSet

Description

Displays the plot set in the plot viewer. Corresponds to the menu choice for this action.

Syntax

```
xt::displayPlotSet <simRun: xtSimRun> -plotSet [<xtPlot>]
```

Arguments

`<simRun>`

The simulation run object.

`-plotSet`

The list of plots to be displayed.

Returns

`<none>`

xt::editFinalNetlist

Description

Opens the final netlist for editing. Corresponds to the menu choice for this action.

Syntax

`xt::editFinalNetlist`

Returns

`<none>`

xt::editTemplate

Description

Opens the input template for editing.

Syntax

`xt::editTemplate`

Returns

`<none>`

xt::exec3DIC

Description

Executes 3DIC in the background with a list of commands to execute on the edit design of the specified context.

Syntax

```
xt::exec3DIC <script: string> [-cellView <deContext | dmCellView>]
```

Arguments

`script`

Commands to execute.

`-cellView`

CellView open in 3DIC.

If a design context is passed, it may prompt to save changes before executing 3DIC.

If a cellView is passed, any unsaved edits are discarded before executing 3DIC.

Returns

<xtJob> : Launched job.

xt::execICC

Description

Executes IC Compiler in the background with a list of commands to execute on the edit design of the specified context.

Syntax

```
xt::execICC <script: string> [-cellView <deContext | dmCellView>]
```

Arguments

`script`

Commands to execute.

`-cellview`

CellView to open in IC Compiler. If a context is passed, it may prompt you to save changes before executing IC Compiler. If a cellView is passed, any unsaved edits are discarded before executing IC Compiler.

Returns

<xtjob> The executed job object.

xt::execICC2

Description

Executes IC Compiler II in the background with a list of commands to execute on the edit design of the specified context.

Syntax

```
xt::execICC2 <script: string> [-cellView <deContext | dmCellView>]
```

Arguments

`script`

Commands to execute.

`-cellView`

The cellview to open in IC Compiler II. If a context is passed, it may prompt to save changes before executing IC Compiler II. If a cellview is passed, any unsaved edits are discarded before executing IC Compiler II.

Returns

<xtJob> : Launched job

xt::execTerminal

Description

Launches terminal command in the specified path or in the current working directory if the path is not specified. The terminal command is specified with the `xtTerminalCommand` preference, which defaults to `xterm`.

Syntax

```
xt::execTerminal <path: string>
```

Arguments

`script`

The path where to open the terminal.

Errors

- Failed to start the terminal.
- Directory not found.

Returns

<xtJob> : Launched job

xt::executePve

Description

Executes a physical verification job process.

Syntax

```
xt::executePve -job <string | xtPveJob> [-tool <string | xtTool>]  
[-rundir <string>] [-launcher <string>] [-callback <string>] [-window  
<giWindow | int>]
```

Arguments

-job

PVE job.

-tool

PVE tool.

rundir

Output run directory.

-launcher

PVE Launcher.

-callback

Script to call when the run completes.

-window

Design editor window. (Default is: -1)

Returns

<xtjob> The executed job object.

Errors

- Invalid preferences

xt::exportFloorplan

Description

Exports routes for digital implementation tools.

Syntax

```
xt::exportFloorplan -output <string> -design <oaDesign>  
[-nets [<string>]] [-objects <dbCollection>] [-types [<enTypes>]]  
[-overrideRoutes <bool>] [-excludeShapesWithNoNet <bool>]  
[-exportShieldRoutes <bool>] [-checkAndCreateNet <bool>]  
[-exportModifiedNets <bool>] [-exportModifiedRoutes <bool>]  
[-openFile <bool>] [-mergeAutoGeneratedOutputs <bool>]
```

Arguments

-output

Specifies an output file name.

-design

Design from which to export routes.

-nets

Specifies a list of net names.

-objects

Specifies a collection of route objects (shape or via).

-types

Types of objects: {boundary instancePlacement pin placementBlockage route routingBlockage routingCorridor row text figGroup}.

-overrideRoutes

Replaces existing routes in the digital design.

-excludeShapesWithNoNet

Excludes shapes with no net.

-exportShieldRoutes

Exports shielded routes.

-checkAndCreateNet

Indicates whether there is a need to export check and create nets. (Default: true)

-exportModifiedNets

Exports modified nets only.

`-exportModifiedRoutes`

Exports modified routes only.

`-openFile`

Opens output file on completion.

`-mergeAutoGeneratedOutputs`

Merges all automatically generated Tcl output in the current session with the final output.

Returns

<none>

xt::exportToICC

Description

Exports the specified cellView(s) to an existing IC Compiler library.

If the target design already exists in the IC Compiler library, and `-overwrite` is true, it is overwritten. Otherwise, it is ignored.

The command returns the *xtJob* object performing the batch export. To wait on the completion of this process, use the [xt::wait](#) command.

Syntax

```
xt::exportToICC <cellView: dmCellView | dmCellViewName> -to <dmLibName  
| oaLib> [-overwrite <bool>] [-createNetList <bool>] [-createLayers  
<bool>] [-cel2fram <bool>] [-hierDepth <int>] [-excludeLibs <dmLibName  
| string>] [-defaultLibPath <string>] [-refLibMap <string>] [-cellMap  
<string>] [-propertyMap <string>] [-objectMap <string>] [-layerMap  
<string>] [-viaMap <string>] [-noInfo <string>] [-noWarning <string>]
```

Arguments

`cellView`

CellView to export.

`-to`

Target IC Compiler library.

`-overwrite`

Overwrites existing cellViews.

Default is: false.

`-createNetList`

Creates MW hierarchy preservation structures from the OA-block netlist.

Default is: false.

`-createLayers`

Modifies the IC Compiler technology to create missing layers.

Default is: 0.

`-cel2fram`

Remasters instances from CEL to FRAM.

Default is: false.

`-hierDepth`

In addition to the specified design, translate this many levels of its hierarchy.

Default is: 0.

`-excludeLibs`

List of OA library names to skip during export.

`-defaultLibPath`

Default directory in which to create new IC Compiler reference libraries, used when `-hierDepth > 0`.

`-refLibMap`

Reference library map file.

`-cellMap`

Cell map file.

`-propertyMap`

Property map file.

`-objectMap`

Object map file.

`-layerMap`

Layer map file.

`-viaMap`

Via map file.

`-noInfo`

List of information message IDs to skip during export.

`-noWarning`

List of warning message IDs to skip during export.

Returns

`<xtJob>` : ed job.

Errors

- Target library is not an IC Compiler library.

xt::exportToReport

Description

Creates and returns a new linked or cached report element in the Report window for the given cell or path. If the Report window does not exist for the cell, a new window is created. If the optional `-updateProc` argument is omitted, the indicated data is inserted statistically into the Report window and the report element is not returned.

Syntax

```
xt::exportToReport (-cell <dmCell | oaCell> | -reportPath <string>)  
  [-textData <string> | -tableData [<string>] | -imagePath <string>]  
  [-tableSize [<int>]] [-caption <string>] [-comment <string>]  
  [-updateProc [<string>]]
```

Arguments

`-cell`

Cell whose Report window to show.

`-reportPath`

Reports path whose Report window to show.

`-textData`

Text data of a text-type report element.

`-tableSize`

Dimensions in rows and columns of a table-type report element.

`-tableData`

Tcl list of strings of the text in a table-type report element.

`-imagePath`

Path to the image file for an image-type report element.

`-caption`

Caption for the report element.

`-comment`

Note or textual description of the report element.

`-updateProc`

Tcl callback to update the element data.

Returns

`<giWindow>` : Report window.

Errors

- `-tableSize` argument is required if the `-tableData` argument is given.
- `-tableSize` argument value must be a Tcl list consisting of a pair of positive integers in the form `{<rowCount> <columnCount>}`.
- `-imagePath` argument must specify the path to an image file in one of the following formats: BMP, GIF, JPEG, PNG, TIFF.

xt::findExecutable

Description

Finds the absolute path to the named executable from the user's PATH environment variable.

Syntax

```
xt::findExecutable <executable: string> [-all <bool>]
```

Arguments

`executable`

The name of the executable (matched exactly) to be searched for and the absolute path to the file is return if found in the \$PATH.

`-all`

Option to return all paths for named executables when set to true, by default this option is false and only the first absolute path found is returned.

Default: false.

Returns

<string>: the absolute path to the specified executable.

xt::findFeature

Description

Finds the named feature in the given xtFeatureManifest.

Syntax

```
xt::findFeature <feature: string> -in <xtFeatureManifest>
```

Arguments

feature

The name of the feature to search for.

-in

The xtFeatureManifest to search for the given feature.

Returns

<xtFeature>: returns a xtFeature for the passed feature name.

xt::findFeatureManifest

Description

Finds named executable manifest file, located with the executable, by searching through the user's PATH environment variable or by the nearest matching version manifest in a given search path.

Syntax

```
xt::findFeatureManifest <executable: string> [-version <string>]  
[-searchPaths [<string>]] [-versionCompareProc <variant>]
```

Arguments

executable

The name of the executable (matched exactly) to be searched for in the user's \$PATH. If executable.manifest is co-located, the absolute path will be returned to the manifest file will be returned.

-version

The current version of the executable.

`-searchPaths`

The list of paths to search for manifest files with names in the format
<executable>.<version>.manifest.

`-versionCompareProc`

The Tcl procedure to use to compare version strings (must work like Tcl's string
compare).

Returns

<string>: The absolute path to the manifest file corresponding to the given executable
name.

xt::generateSimOutputCbK

Description

Processes simulator output by performing any required name mappings after the simulator
is run, through a call to xt::processMarkupTags.

Syntax

```
xt::generateSimOutputCbK <simRun: xtSimRun>
```

Arguments

simRun

The xtSimRun object.

Returns

<none>

xt::getAnalysisList

Description

Creates a list of analyses for the plot dialog box to display.

If no simulation results exist, the list is empty.

The results returned form the getAvailableAnalysesProc when defined and when
simulation results exist. The getAvailableAnalysesProc callback is found on the
xtSimulator.

Syntax

```
xt::getAnalysisList <simRun: xtSimRun>
```

Arguments

`simRun`

The `simRun` object.

Returns

Names of available analyses.

xt::getCalibreSocketPort

Description

Gets the port number of the current socket opened for Calibre RVE.

Syntax

```
xt::getCalibreSocketPort
```

Returns

<int> : Calibre RVE port number.

xt::getCallingJob

Description

Gets the job object for the current externally invoked command.

Syntax

```
xt::getCallingJob
```

Returns

<xtJob>: Calling job object. Returns "" if not called from an externally invoked command.

xt::getCrossRefs

Description

Returns a list of `xtCrossRef` objects that currently exist matching specified criteria.

Syntax

```
xt::getCrossRefs [-window <int>] [-filter <string>]
```


Arguments

`-window`

Gets xtCrossRef objects associated with the window.

`-filter`

Standard attribute filter.

Returns

<dbCollection>: xtCrossRef objects matching search criteria.

xt::getCDPLStatus

Description

Displays the status of the new CDPL Master

Syntax

```
xt::getCDPLStatus -CDPL <xtCDPL> -type <string>
```

Arguments

`-CDPL`

The new CDPL Master.

`-type`

The type is either `workerCount` or `queueTaskCount`

Returns

<string> : Status of new CDPL Master.

xt::getDebugSessions

Description

Gets all the registered xtDebugSession objects.

Syntax

```
xt::getDebugSessions [<pattern: string>] [-filter <string>]
```

Arguments

`pattern`

Pattern to match object job name/client ID. Empty pattern matches all.

`-filter`

Standard attribute filter.

Returns

<dbCollection>: xtDebugSession objects matching search criteria.

xt::getEquivalentNets

Description

Provides aliases for names that cannot be found by the waveform viewer. The returned names are in the simulator namespace, based on the destination namespace symbol passed in. Note that `db::getEquivalentNets` and `nl::lookup` are used to perform the actual look-ups. The correct context is found by using the file from the `-resultsDir` argument to locate the correct xtSimRun.

Syntax

```
xt::getEquivalentNets -net <string> -resultsDir <string>
```

Arguments

`-net <string>`

The hierarchical net name, in the simulator namespace, to find aliases for.

`-resultsDir <string>`

The name of the results directory that contains the waveform data. This will be used to determine the correct simRun and design.

Returns

A string with the list of hierarchical names in the simulator namespace that are aliases of the supplied name.

xt::getGPDCorners

Description

This command will return a string of corners for the specified GPD.

Syntax

```
xt::getGPDCorners -gpd <string>
```

Arguments

`-gpd`

Specifies the path to GPD directory

Returns

`<string>` : Corners for the specified GPD.

xt::getGPDCouplingCapacitors

Description

This command returns a string of coupling capacitors for the given nets or nodes on the command line.

Syntax

```
xt::getGPDCouplingCapacitors [-of <string>] [-from <string>] [-to  
  <string>] [-corners <string>]
```

Arguments

`-of`

Reports coupling capacitors associated with the specified nets.

`-from`

Constrains the selected capacitors to be part of a path starting from the specified node.

`-to`

Constrains the selected capacitors to be part of a path ending with the specified node.

`-corners`

Indicates the parasitic corner ordering for coupling capacitance values.

Returns

`<string>` : Coupling capacitors for the given nets or nodes.

xt::getGPDGroundCapacitors

Description

This command returns a string of ground capacitors for the given nets or nodes on the command line.

Syntax

```
xt::getGPDGroundCapacitors [-of <string>] [-from <string>] [-to <string>]  
    [-corners <string>]
```

Arguments

`-of`

Reports ground capacitors associated with the specified nets.

`-from`

Constrains the selected capacitors to be part of a path starting from the specified node.

`-to`

Constrains the selected capacitors to be part of a path ending with the specified node.

`-corners`

Indicates the parasitic corner ordering for ground capacitance values.

Returns

`<string>` : Ground capacitors for the given nets or nodes.

xt::getGPDLayers

Description

This command returns a string of layers for the specified GPD.

Syntax

```
xt::getGPDLayers -gpd <string> [-type <enum>]
```

Arguments

`-gpd`

Specifies the path to GPD directory.

`-type`

Specifies the type of layers to report ({ all | routing | via }).

Returns

`<string>` : Layers for the specified GPD.

xt::getGPDOObjects

Description

This command returns a string of objects for the given type.

Syntax

```
xt::getGPDOObjects [-cell <string>] [-net <string>] [-port <string>] [-pin  
  <string>] [-exact <bool>] [-verbose <bool>]
```

Arguments

-cell

Specifies pattern for cell object query.

-net

Specifies pattern for net object query.

-port

Specifies pattern for port object query.

-pin

Specifies pattern for pin object query.

-exact

Disables wildcard pattern matching.

-verbose

Indicates report detailed information.

Returns

<string> : GPD object names for the specified type.

xt::getGPDRefs

Description

Returns the associated Golden Parasitic Database that currently exists.

Syntax

```
xt::getGPDRefs [-lcvName <string>]
```

Returns

<string> : The associated Golden Parasitic Database.

Arguments

`-lcvName`

Lib Cell View Name. (Default is:)

xt::getGPDResistors

Description

This command returns a string of resistors for the given nets or nodes on the command line.

Syntax

```
xt::getGPDResistors [-of <string>] [-from <string>] [-to <string>]  
                    [-corners <string>] [-geometry <bool>]
```

Arguments

`-of`

Reports resistors associated with the specified nets.

`-from`

Constrains the selected resistors to be part of a path starting from the specified node.

`-to`

Constrains the selected resistors to be part of a path ending with the specified node.

`-corners`

Indicates the parasitic corner ordering for ground capacitance values.

`-geometry`

Reports geometry information of resistor.

Returns

`<string>` : Resistors for the given nets or nodes.

xt::getInputFiles

Description

Recursively scans the named file and any included files to form a list of all input file dependencies. Only files found through the markup tags will be included. Files using a simulator include syntax will be ignored.

Note:

Typically, this command uses the netlist.template file.

Syntax

```
xt::getInputFiles -file <string>
```

Arguments

-file <string>

The name of the file to scan.

Returns

The list of input files. ([<string>])

xt::getInputFilesTimeStamps

Description

Recursively scans the named file and any included files to form a time-stamped list of all input file dependencies. Only files found through the markup tags are included. Files using a simulator include syntax are ignored. Notes to user:

Note:

This command is typically used on the netlist.template file.

This command is called in the following cases:

- During run directory initialization
- Prior to netlisting
- Prior to simulating

Syntax

```
xt::getInputFilesTimeStamps -file <string>
```

Arguments

`-file <string>`

The name of the file to scan.

Returns

The list of input files and timestamp pairs. ([<map>])

xt::getIntegrations

Description

Returns a collection of all registered integrations matching the passed filter argument.

Syntax

```
xt::getIntegrations [[<string: string>]] [-filter <string>]
```

Arguments

`string`

Glob-style pattern to match integration name.

Default: Match all.

`-filter`

Standard filter argument.

Default: Match all.

Returns

Collection of all the integrations matching the given filter. ({xtIntegration})

Examples

```
db::foreach a [xt::getIntegrations] {puts [db::getattr name \  
-of $a]}
```

In this example, xt::getIntegrations returns all defined integrations and prints their names in console.

xt::getJobClasses

Description

Returns a collection of the registered job classes.

Syntax

```
xt::getJobClasses [<pattern: string>] [-filter <string>]
```

Arguments

`pattern`

Pattern to match against the object names. Empty pattern matches all.

`-filter`

Standard attribute filter.

Returns

`<dbCollection>`: The collection of registered job classes.

xt::getJobClassOptionPrefName

Description

Gets the preference name of the given job class option.

Syntax

```
xt::getJobClassOptionPrefName <jobClass: string | xtJobClass> -option  
  <string>
```

Arguments

`jobClass`

The xtJobClass object

`-option`

The option name

Returns

`<string>`: Name of the preference containing the given option.

xt::getJobHostStatus

Description

Gets job host status information.

Syntax

```
xt::getJobHostStatus -job <xtJob>
```

Arguments

`-job`

The xtJob object.

Returns

`<int>` : Get job host status success or fail.

xt::getJobs

Description

Returns the list of all jobs known to the job manager, subject to the optional filter provided.

Syntax

```
xt::getJobs [<pattern: string>] [-filter <string>]
```

Arguments

`<string>`

Pattern to match the job name against. Empty pattern matches all jobs.

`-filter <string>`

Standard attribute filter.

Default: Matches all.

Returns

`<dbCollection>`: A collection of xtJobs matching the filter criteria.

Examples

```
set jobs [db::getNext [xt::getJobs verillog2oa*]]
```

In this example, xt::getJobs finds jobs corresponding to verillog2oa* filter requirements.

xt::getManifestFeatureValue

Description

This command gets the manifest feature value.

Syntax

```
xt::getManifestFeatureValue <execName: string> -versionStr <string>  
-featureName <string> [-searchPaths <string>] [-homeEnvVarName <string>]
```

Arguments

`execName`

Executable name.

`-versionStr`

Version of executable.

`-featureName`

Name of feature whose value to return.

`searchPaths`

Extra paths to search for manifest files.

`-homeEnvVarName`

Home variable name to search for manifest files.

Returns

<string>: Manifest file feature value.

xt::getNetlistInputFiles

Description

Returns a list of input files as specified in the xtSimRun's netlist.template.

Syntax

```
xt::getNetlistInputFiles <simRun: xtSimRun>
```

Arguments

`simRun`

The simulator object.

Returns

Names of all of the files which are referenced through the netlist.template. ([<string>])

xt::getOpenFile

Description

Opens a file selection dialog to select an existing file or directory.

Syntax

```
xt::getOpenFile [-multiple <bool> | -initialFile <string> | -directory  
<bool>] [-title <string>] [-fileTypes [<string>]] [-initialDir <string>]  
[-window <giWindow | int>]
```

Arguments

-title

File selector dialog title.

-multiple

Allows multiple selection.

-fileTypes

File types filters. For example, { "Images (*.png *.jpg *.jpeg)" "Text files (*.txt)" "All files (*)" }

-initialDir

Initial directory.

-initialFile

Initial file.

-directory

Toggles directory selection.

-window

Window ID.

Returns

<none>

xt::getOutOfDateFiles

Description

Given the list of input files and timestamps, checks each file on disk against the associated time stamp. Each out-of-date file is appended to a list returned by this command. If an input file is not found in the list of timestamps, it is assumed to be out-of-date and is returned in the resulting list.

The command [xt::getInputFilesTimeStamps on page 2867](#) is provided to generate the timestamps map.

Syntax

```
xt::getOutOfDateFiles -files [<string>] -timestamps [<map>]
```

Arguments

`-files` [<string>]

The input files to be checked.

`-timestamps` [<map>]

File or timestamps pairs.

Returns

Names of all of all the files that have changed relative to the timestamps passed in.
([<string>])

xt::getPackages

Description

Returns a collection of packages found in \$PATH, optionally filtered by a glob-style pattern matched against the package name

Syntax

```
xt::getPackages [<pattern: string>] [-filter <string>]
```

Arguments

`pattern`

Glob-style pattern matched against the package name. (Default is: *)

`-filter`

Optional standard filter expression.

Returns

<dbCollection> : Collection of xtPackages

xt::getPDKConfigs

Description

Gets all registered xtPDKConfig objects.

Syntax

```
xt::getPDKConfigs [<pattern: string>] [-filter <string>]
```

Arguments

`<pattern>`

Pattern to match object name.

`-filter`

Standard attribute filter.

Returns

`<dbCollection>`: xtPDKConfig objects matching search criteria.

xt::getPDKLibs

Description

Returns a list of all oaLib objects designated as PDK libraries.

Syntax

```
xt::getPDKLibs
```

Returns

`<{<oaLib>}>`: A collection of PDK libraries.

xt::getPlotViewers

Description

Returns a collection of xtPlotViewer Tcl objects matching the passed filter argument.

Syntax

```
xt::getPlotViewers [<string>] [-filter <string>]
```

Arguments

`<string>`

The glob style regular expression used to match existing viewer names.

Default: `*`.

`-filter`

Standard attribute filter.

Default: match all.

Returns

A collection of xtViewers matching the given expression. ({xtPlotViewer})

xt::getPveJobs

Description

Gets all registered xtPveJob objects.

Syntax

```
xt::getPveJobs [<pattern: string>] [-filter <string>]
```

Arguments

<pattern>

Pattern to match object name.

-filter

Standard attribute filter.

Returns

<dbCollection>: xtPveJob objects matching search criteria.

xt::getPWJobClasses

Description

Returns a collection of the registered PrimeWave job classes.

Syntax

```
xt::getPWJobClasses [<pattern: string>] [-filter <string>]
```

Arguments

pattern

Pattern to match against the names of the PrimeWave job classes. Empty pattern matches all.

-filter

Standard attribute filter.

Returns

<dbCollection>: The collection of registered PrimeWave job classes.

xt::getQueueEngines

Description

Returns the currently registered queue engines.

Syntax

```
xt::getQueueEngines [<pattern: string>] [-filter <string>]
```

Arguments

pattern

Pattern to match against the object names. Empty pattern matches all.

-filter

Standard attribute filter.

Returns

<dbCollection>: Collection of registered objects.

xt::getRunDirName

Description

Gets default run directory name for physical verification and parasitic extraction jobs.

Syntax

```
xt::getRunDirName <cell: deContext | dmCellView | dmDMFile | heContext |  
string> -job <string | xtPveJob> -tool <string | xtTool>
```

Arguments

cell

Cell name or an object from which a cell name can be extracted.

-job

PVE job.

-tool

PVE tool.

Returns

<string> Run directory name.

xt::getSaveFile

Description

Opens a file selection dialog box for selecting a file name for saving.

Syntax

```
xt::getSaveFile [-title <string>] [-fileTypes [<string>]] [-initialDir  
<string>] [-initialFile <string>] [-window <giWindow | int>]
```

Arguments

-title

File selector dialog title.

-fileTypes

File type filters. For example, {"Images (*.png *.jpg *.jpeg)" "Text files (*.txt)" "All files (*)"}.

-initialDir

Initial directory.

-initialFile

Initial file.

-window

Window ID.

Returns

<none>.

xt::getSelectedJobGroups

Description

Returns a list of groups selected in the given Job Monitor window.

Syntax

```
xt::getSelectedJobGroups <window: giWindow | int> [-jobSummary <bool>]
```

Arguments

window

A Job Monitor window or ID.

`-jobSummary`

If it is triggered from job summary. (Default is: false)

Returns

`<dbCollection>` : Collection of Groups selected in the given Job Monitor window

xt::getSelectedJobs

Description

Returns a list of jobs selected in the given Job Monitor window.

Syntax

```
xt::getSelectedJobs <window: giWindow| int> [-jobSummary <bool>]
```

Arguments

giWindow

The Job Monitor Window.

`-jobSummary`

If it is triggered from job summary. (Default is: false)

Returns

`<dbCollection>`: Collection of Jobs selected in the given Job Monitor window.

xt::getSimRuns

Description

Returns the registered simulation runs matching the passed filter argument.

Syntax

```
xt::getSimRuns [-filter <string>] [-context <deContext>]
```

Arguments

`-filter`

Standard attribute filter.

Default: Match all.

`-context`

The context to which the returned xtSimRuns are optionally restricted.

Default: All active contexts.

Returns

A collection of xtSimRuns matching the given filter. ({xtSimRun})

xt::getSimulators

Description

Returns the registered simulator objects matching the passed filter argument.

Syntax

```
xt::getSimulators [<string: string>] [-filter <string>]
```

Arguments

string

The glob style regular expression used to match existing simulator names.

Default: *.

-filter

Standard attribute filter.

Returns

A collection of xtSimulator objects matching the given expression. ({xtSimulator})

xt::getTextViewerActiveTab

Description

Returns the active tab name in the given Text Viewer and returns empty string if there are no tabs in the window.

Syntax

```
xt::getTextViewerActiveTab -window <giWindow | int>
```

Arguments

-window [giWindow](#)

The Text Viewer window.

Returns

Name of the active tab. (<string>)

Errors

An error occurs if the window argument is not a Text Viewer window.

xt::getTextViewerTabs

Description

Returns a list of all tab names optionally filtered using the pattern.

Syntax

```
xt::getTextViewerTabs [<string>] -window <giWindow>
```

Arguments

<string>

Pattern to match against. Empty pattern matches all tabs.

-window *giWindow*

The Text Viewer window.

Returns

List of tab names matching the pattern. ([<string>])

Errors

An error occurs if the window argument is not a Text Viewer window.

xt::getTools

Description

Gets all registered xtTool objects.

Syntax

```
xt::getTools [<pattern: string>] [-filter <string>]
```

Arguments

pattern

Pattern to match object name.

-filter

Standard attribute filter.

Returns

<dbCollection> xtTool objects matching search criteria.

xt::getWalkupPort

Description

Returns the port number the Custom Compiler tool is listening on for incoming walkup connections (from downstream tools).

Syntax

```
xt::getWalkupPort
```

Returns

<int>: The port number the Custom Compiler tool is listening on for incoming walkup connections.

xt::hideJobMonitorColumn

Description

Hides or shows a table column in the Job Monitor Window.

Syntax

```
xt::hideJobMonitorColumn <window: giWindow | int> -column <enum> [-hide  
    <bool>]
```

Arguments

window

The Job Monitor Window.

-column

The column to show or hide: {end_time | host | job | output_files | process_id | run_description | start_time | status | type}.

-hide <bool>

Hides the column (true), or show it (false).

Default: True.

Returns

<none>

xt::highlightViolation

Description

Creates a temporary violation shape for the specified design. The design is not modified and the shape may not be activated or selected.

The point array may be either open (in the case that it is a polyline) or closed (in which it is considered a polygon). In the case of a polygon, the line connecting the nth point to the first is implied and not necessary.

Note:

The `-isClosed` option is currently not implemented.

Syntax

```
xt::highlightViolation <points: [point]> -design <oaDesign> -isClosed  
  <bool> -toolName <string> [-packet <string>]
```

Arguments

`points`

List of points that forms the polygon or polyline (depending on the value for `-closed`).

`-design oaDesign`

Design in which to highlight the violation.

`-toolName <string>`

Tool which owns this marker.

`-isClosed <bool>`

Indicates whether the point array is a closed polygon or an open polyline.

Note:

This option is currently not implemented.

`-packet <string>`

Packet or packet alias name defining how this violation should be drawn.

Default: `markerErr`.

Returns

`<none>`

xt::initCalibreConnection

Description

Gets or creates a socket for Calibre RVE and returns its port number.

Syntax

```
xt::initCalibreConnection [-port <int>] [-force <bool>] [-autoFindOnError  
    <bool>]
```

Arguments

-port

Port number. (Default is: 9189)

-force

Indicates whether or not to close existing calibre server before starting a new one. The default is false.

-autoFindOnError

Indicates whether or not to try with a different port if the specified one fails. (Default is: true)

Returns

<int> : Calibre Server port number.

xt::initializeQueueEngine

Description

Initializes a queue engine for use.

Syntax

```
xt::initializeQueueEngine <engine: string | xtQueueEngine>
```

Arguments

engine

The queue engine to initialize.

Returns

<bool>: Initialization result.

xt::initIntegration

Description

Loads all code for the names integration. Initializes the named integration and should only be called after the integration system is initialized.

To carry out the initialization, this command:

- Sources the primary integration Tcl file.
- Looks for and sources any netlister definition Tcl files.
- Looks for and sources any viewer definition Tcl files.
- Loads any custom menus and bindings Tcl files.
- Loads the default integration preferences at the INSTALL scope.

Note:

Integration-specific overrides of attributes are done in the setPreferencesProc callback.

Syntax

```
xt::initIntegration <name: string> -type <enum>
```

Arguments

name

The name of the simulator integration.

-type <enum>

The type of integration to be registered; note that only simulator is valid: {simulator}.

Returns

<none>

xt::initRunDir

Description

Initializes the specified directory either by selecting an existing directory, copying settings from an existing run directory, or creating a new directory. Two active simulation sessions may not use the same run directory, however, child and parent windows are allowed to use the same run directory.

- If the "-copyDir" option is used, the "-simulator" option overrides the simulator setting of the run dir being copied.
- If the "-copyDir" option is used, the topLib, topCell and topView attributes are updated with the default options of the new directory being created.
- The "-copyDir" option only copies files at the top level of the run directory, ignoring the files in the results and the netlist directories as well as log files.
- When -copyDir is not used, a call to [xt::initSimulatorIntegration on page 2886](#) copies all of the input files specified by the simulator integrator to the target directory.

The command [xt::createSimRun on page 2845](#) is called to create and return the xtSimRun.

Syntax

```
xt::initRunDir <dir_name: string> -simulator <string> [-cellView  
  <dmCellView>] [-context <deContext>] [-of <string>]
```

Arguments

`dir_name`

The directory to contain all of the simrun and simulation information.

`-cellView`

The cellview with which to initialize the run directory. Either cellView or a deContext is required.

`-context`

The design editor to associate the xtSimRun with. The context's top design will be the xtSimRun's cellView. If a context is provided, the simulation menu items become active for the integration selected. Either context or a cellView is required.

`-simulator`

The name of the simulator to use in the run directory.

`-of`

An existing run directory from which to copy initial settings.

Returns

xtSimRun: The xtSimRun object associated with the newly created run directory.

Errors

- InvalidSimulator: an unrecognized name is supplied for the simulator.
- OSError: either an invalid directory name is given, or there are not appropriate permissions for the target or source directory.

Examples

```
xt::initRunDir sim1 -context [de::getActiveContext ] \  
    -simulator HSPICE  
xt::initRunDir sim2 -simulator HSPICE -cellView \  
    [dm::getCellViews schematic -cellName and2_testbench \  
    -libName DemoPLL]
```

In this example, `xt::initRunDir` initializes the specified directory, and then `xt::createSimRun` is called to create and return the `xtSimRun` object.

xt::initSimulatorIntegration

Description

Called automatically during the run directory initialization, and completes the following operations:

- Copies the netlist.template into the run directory if the run directory is new
- Calls the `xtSimulator::getInitFilesProc` callback and copies the files that are specified for the run directory
- Calls `xt::updateInputFileList`
- Calls the `sim::initRunDirectoryProc` callback

Syntax

```
xt::initSimulatorIntegration <simRun: xtSimRun>
```

Arguments

`simRun`

The `xtSimRun` object.

Returns

<none>

xt::invokeExecutableForVersion

Description

This command invokes a given executable to output its version and returns what comes after a given marker.

Syntax

```
xt::invokeExecutableForVersion <execName: string> [-versionArg <string>]  
[-markerStr <string>]
```

Arguments

`execName`

Executable name.

`-versionArg`

Argument to output executable version banner.

Default: -v.

`-markerArg`

String that precedes version number.

Default: version.

Returns

<string>: Executable version string.

xt::isValidRunDir

Description

Verifies that the specified directory to is a complete and valid run directory. If valid, it is safe to create an xtSimRun by calling the command [xt::loadSimRun on page 2893](#).

Syntax

```
xt::isValidRunDir <rundir: string>
```

Arguments

`rundir`

The name of the directory to validate.

Returns

True, if the named directory is a valid run directory. (<bool>)

xt::launchJobs

Description

Launches deferred jobs.

Syntax

```
xt::launchJobs <jobs: dbCollection> [-arrayName <string>]
```

Arguments

jobs

The jobs to launch.

-arrayName

Queues engine job array name.

Returns

<none>

xt::launchPlotViewer

Description

Launches a new plot viewer, if one is not open already. If called more than one time, this command reuses an already opened plot viewer. Corresponds with the menu choice for this action.

Syntax

```
xt::launchPlotViewer <simRun: xtSimRun>
```

Arguments

simRun

simRun to which to register the launched viewer.

Returns

xtPlotViewerInst: The launched viewer instance.

Examples

```
xt::launchPlotViewer [xt::getSimRuns]
```

In this example, `xt::launchPlotViewer` launch a new plot viewer, if one is not open already.

xt::launchSimulatorCbk

Description

Launches the simulation process by calling the Tcl callback returned by `getLaunchCommandProc`. The Tcl procedure `getLaunchCommandProc` is an attribute of the `xtSimulator` defined by an integrator. If `getLaunchCommandProc` is not given a value, this command fails. When the simulation successfully finishes, `xt::postProcessSimResults` is called.

Syntax

```
xt::launchSimulatorCbk <simRun: xtSimRun>
```

Arguments

`simRun`

The simulation run to which the launched viewer is to be registered.

Returns

<none>

xt::launchViewerCbk

Description

Launches the waveform viewer using the tool integrator's defined `getViewerCommand` callback.

If the `xtPlotViewer::getCommandProc` is defined by the integrator, the IPC routine `xt::createJob` is used and allows `readyProc` and `exitProc` callback routines to be defined:

The `readyProc` determines when the viewer is ready to accept additional commands, thus avoiding potentially long startup times.

The `exitProc` disables plot commands when the viewer exits and sends a status message to the user.

Syntax

```
xt::launchViewerCbk <simRun: xtSimRun>
```

Arguments

`simRun`

The viewer's `xtSimRun` object.

Returns

xtJob: The job object of the launched viewer.

xt::loadPackage

Description

Loads a package.

Syntax

```
xt::loadPackage <package: [string | xtPackage]>
```

Arguments

package

Package name or object to load.

Returns

<none>

xt::loadPlotSet

Description

Loads the plot set from the specified file and returns an associated collection of xtPlots.

Syntax

```
xt::loadPlotSet -filePath <string>
```

Arguments

-filePath

The name of file to load the plot set from.

Returns

The collection of xtPlots loaded from the file. ({xtPlot})

xt::loadPlugin

Description

Loads the user-defined plugin.

Syntax

```
xt::loadPlugin <filePath: string>
```

Arguments

filePath

The path to the shared object to load.

Returns

<none>

Errors

- Invalid shared object: “%s”
- Shared object “%s” does not have a valid CDK signature. Use cdk sign to sign the shared object.
- Shared object “%s” does not support dynamic loading (missing dlsym)

xt::loadPvePrefs

Description

Loads the content of the file specified in the "filePath" to the given scope.

Syntax

```
xt::loadPvePrefs <scope: dbScope> -filePath <string> [-cv <dmCellView>]
```

Arguments

scope

Scope where the given configuration file is loaded.

-filePath

Path of the configuration file to be loaded into the given scope.

-cv

Preferred dmCellView for LCV related preferences.

Returns

<dbCollection> : Collection of the loaded preferences.

xt::loadRunDir

Description

Loads and initializes an existing run directory from the specified directory.

Only the directory is required for this operation. Additional arguments override the initialization behavior. The optional arguments are not used by the default GUI system. The simulator integration, if provided, overrides the name of the integration in the specified run directory. If the cellView or context is provided, it overrides the cell view specified in the run directory.

This command ensures that no other simulation session is using the passed in run directory. This command calls [xt::createSimRun on page 2845](#) to create the returned xtSimRun. The menu is initialized if the context is provided.

Syntax

```
xt::loadRunDir <dirName: string> [-cellView <dmCellView> | -context  
    <deContext>] [-simulator <string>]
```

Arguments

dirName

The directory containing the simRun and simulation information.

-cellView

The cellView to initialize the run directory.

-context

Loads and initializes an existing run directory.

-simulator

The name of the simulator to use in the run directory.

Returns

The simRun object associated with the newly created run directory.(xtSimRun)

Errors

- InvalidSimulator: an unrecognized name is supplied for the simulator.
- OSError: either an invalid directory name was given or there are not. appropriate permissions for the target or source directory.
- InvalidRunDirectory: Run directory does not contain valid data or is not complete.

xt::loadSimRun

Description

Loads an xtSimRun object from an existing run directory by de-serializing the data in the specified run directory. After this command executes, the directory for the name map can be found in the netlistDir attribute of the xtSimRun.

Syntax

```
xt::loadSimRun -filePath <string> [-context <deContext>] [-cellView  
  <dmCellView>]
```

Arguments

-filePath

The name of the directory from which to load data.

-context

The design context associated with the created xtSimRun. If the top design of the context does not match the LCV data stored in the run-directory, a message is given and the LCV of the context's top design is used, and the run-directory data over written.

Default: None.

-cellView

A cellView to override the LCV information stored in the run directory. This option cannot be used with the context option.

Returns

A new xtSimRun. (xtSimRun)

xt::openFeatureManifest

Description

Reads the manifest file specified by the absolute file path and returns an xtFeatureManifest object.

Syntax

```
xt::openFeatureManifest <filePath: string>
```

Arguments

`filePath`

The absolute path to a manifest file to be read.

Returns

`<xtFeatureManifest>`: the `xtManifestFeature` object for the specified manifest file or an error is thrown.

xt::openTextViewer

Description

Opens a text viewer displaying the given files in its own tab. The tab name is created by taking the basename of the file displayed in the tab.

The `-text` argument allows the user to display a fixed text string in its own tab named `Text`.

If a tab name is not unique, a colon and a numerical suffix are appended to make the tab name unique.

Syntax

```
xt::openTextViewer [-text <string>] [-files [<string>]] [-titles  
  [<string>]] [-includePaths [<string>]]
```

Arguments

`-text`

List of string to view.

`-files`

List of file names to open.

`-titles`

List of tab titles to assign.

`-includePaths`

List of include directory paths to search.

Returns

The new Text Viewer window. ([giWindow](#))

xt::openTextViewerTab

Description

Opens a file in a new tab in an existing Text Viewer window. The tab name is created by taking the basename of the file displayed in the tab.

The `-text` argument allows the user to display a fixed text string and the tab name is Text.

If a tab name is not unique, a colon and a numerical suffix are appended to make the tab name unique.

Syntax

```
xt::openTextViewerTab [-text <string>] [-filePath <string>] -window  
    <giWindow>
```

Arguments

`-text`

The string to display in the tab.

`-filePath`

The file name to open.

`-window`

The Text Viewer window.

Returns

The name of the new tab. (<string>)

Errors

An error occurs if the window argument is not a Text Viewer window.

xt::plotWaveform

Description

Displays a waveform and launches a waveform viewer window, if one has not been launched.

This command also name maps the arguments to the simulator namespace and calls the integrator-defined `getPlotScriptProc` callback to get the commands to send to the viewer.

Commands are sent to the waveform viewer using the `xt::sendCommand` function. The integrator is expected to implement the plotting asynchronously, so that control will return quickly.

Syntax

```
xt::plotWaveform <simRun: xtSimRun> -command <string> [-plot <xtPlot>]  
[-packet <string>]
```

Arguments

`simRun`

The xtSimRun holding the viewer object.

`-command`

The manner in which the new plot is added to the viewer.

`-plot`

A list of hierarchical names in the simulator namespace that are aliases of the supplied name.

`-signal`

The wfSignal object to plot.

`-packet`

The packet that is used to plot.

Returns

<none>

Examples

```
set xtplot [xt::createPlot -name CLK -type net -analysis TRAN \  
-packet annotatel]  
xt::plotWaveform $sim -command new -plot $ xtplot
```

In this example, xt::plotWaveform plots the CLK signal in the default wave viewer.

xt::postProcessSimResults

Description

This command is called after the simulation job has finished and should never be called directly by the user. This command is required by an integrator when overriding the default netlisting callback.

This command checks for successful execution of the simulator and calls the procedure xtSimulator::generateSimOutputProc to perform integrator-defined post processing of results and required name mapping.

This command also unlocks the run directory, logs the exit status and associated error messages, and sends commands gathered from xtSimRun->xtPlotViewer->getReplotScriptProc to the waveform viewer.

Syntax

```
xt::postProcessSimResults <simRun: xtSimRun> -job <xtJob>
```

Arguments

simRun

The simulator object.

-job

The simulation job object.

Returns

The exit code of the simulation job, xtJob::exitStatus (<int>).

xt::processLPEResults

Description

Creates parasitic nets and net cross-reference database.

Syntax

```
xt::processLPEResults [<Context: deContext>] [-cnlDir <string>]
```

Arguments

Context

Context with LPE extracted view.

-cnlDir

Custom path to netlister cnl directory.

Returns

<bool> : Capacitance Report

xt::processMarkupTags

Description

Recursively processes the named netlist template for all supported markup (file inclusions and name mapping). The system uses this command to create the file netlist.final from the file netlist.template.

Note:

This command overwrites any edits in the output file, so check the timestamp before calling this command.

Syntax

```
xt::processMarkupTags -inFile <string> -outFile <string>  
    -map <nlNameMap | nlOANameMap | nlParameterizedNameMap> [-direction  
    <enum>] [-workingDir <string>]
```

Arguments

-inFile

The name of the input file to be processed.

-outFile

The name of the output file to receive output results. This is relative to the workingDir unless it is an absolute path.

-map

Name map object to be queried or modified.

-direction

If set to "Forward", maps names using the -src argument. If set to "Reverse", maps names using the -dst argument: {Forward | Reverse}

Default: Forward.

-workingDir

The directory for output, for PrimeWave™ Design Environment iterative runs.

Default: "".

Returns

<none>

xt::queryParasitics

Description

Returns a collection of instances of parasitic devices connected to the specified net(s) or between the specified terminals.

If the `-nets` argument is supplied, the command returns all parasitics connected to any net from the input collection corresponding to the net query. If the `-from`, `-to` pair of arguments is specified, the command returns parasitics between supplied nets or terminals, corresponding to `netToNet` query or `pointToPoint` query.

The `-nets` argument is mutually exclusive with the pair of `-from` `-to` arguments.

Parasitic values are retrieved from the instances by getting the value of the "r" property for a "presistor", or the "c" property for a "pcapacitor". For example, assuming the instance id is in a property "inst", the values can be retrieved by:

```
set val [oa::getValue [oa::PropFind $inst c]]
```

And

```
set val [oa::getValue [oa::PropFind $inst r]]
```

Syntax

```
xt::queryParasitics [-type <enum>] [-nets <{<oaNet>}>] [-of <oaInstTerm  
| oaNet | oaRect>] [-to <oaInstTerm | oaNet | oaRect>] [-context  
<heContext>] [-fromContext <heContext>] [-toContext <heContext>]  
[-crossRef <xtCrossRef>] [-queryR <bool>] [-queryCc <bool>] [-queryCg  
<bool>] [-queryL <bool>] [-thresholdR <string>] [-thresholdCc <string>]  
[-thresholdCg <string>] [-thresholdL <string>] [-queryRFilter <bool>]  
[-corner <string>]
```

Arguments

`-type`

Type of query to perform.

`-nets`

Specifies nets to which the parasitic instances must connect to satisfy the query criteria.

`-of`

Specifies first net or terminal to which parasitic device must be connected.

`-to`

Specifies second net or terminal to which parasitic device must be connected.

`-tool`

Parasitic extraction tool name.

`-runDir`

Parasitic extraction run directory.

`-design`

Design to query.

`-queryR`

Includes parasitic resistors in the query.

Default: `xtLPEQueryRes`.

`-queryCc`

Includes parasitic coupled capacitors in the query.

Default: `xtLPEQueryCc`.

`-queryCg`

Includes parasitic grounded capacitors in the query.

Default: `xtLPEQueryCg`.

`-queryL`

Include parasitic inductors in the query.

Default: `xtLPEQueryInd`.

`-thresholdR`

Threshold value below which parasitic resistors are ignored.

Default: `xtLPEResThresh`.

`-thresholdCc`

Threshold value below which parasitic coupled capacitors are ignored.

Default: `xtLPECcThresh`.

`-thresholdCg`

Threshold value below which parasitic grounded capacitors are ignored.

Default: `xtLPECgThresh`.

`-thresholdL`

Threshold value below which parasitic inductors are ignored.

Default: `xtLPEIndThresh`.

`-queryRFilter`

Filters parasitic resistors by the threshold value.

`-corner`

Corner map value to calculate the property value of parasitics

Returns

Collection of parasitic instances associated with the specific query. (`{<oaFig>}`).

Errors

- Invalid net or terminal.
- Net or terminal does not belong to a maskLayout cellview.
- Missing net or terminal required for the specified query type.

xt::quitPvDebugger

Description

Quits a running physical verification tool's debugger process.

Syntax

```
xt::quitPvDebugger -job <xtJob>
```

Arguments

`-job` `xtJob`

The job object.

Returns

`<none>`

xt::readGPDParasitics

Description

This command reads GPD parasitics data with the specified arguments.

Syntax

```
xt::readGPDParasitics [-read <string>] [-current <bool>]  
[-keep_capacitive_coupling <bool>]
```

Arguments

`-read`

Reads GPD from specified path.

`-current`

Returns current GPD directory.

`-keep_capacitive_coupling`

Indicates that the cross capacitors are to be kept in the RC networks data structure.

Returns

`<string>` : Result of GPD data.

xt::readJob

Description

Returns the data written to stdout by the remote job, waiting a number of seconds for data as specified by the time-out argument. If the time-out is not specified, or is 0, then the readJob command does not wait.

Syntax

```
xt::readJob <job: xtJob> [-timeout <int>]
```

Arguments

`job`

The job to read data from.

`-timeout`

Time, in seconds, to wait for data. Timeout of 0 returns immediately.

Default: 0.

Returns

Data delivered by the interactive job. (`<string>`)

Errors

Any failure to connect to the job or read the data generates a Tcl error.

xt::refreshTextViewer

Description

Refreshes the text viewer contents from their sources.

Syntax

```
xt::refreshTextViewer <window: giWindow | int>
```

Arguments

window

The Text Viewer window.

Returns

<none>

Examples

```
xt::refreshTextViewer 3
```

In this example, xt::refreshTextViewer refreshes the text viewer contents of the window ID 3.

xt::reloadJobs

Description

Reloads jobs from disk.

Syntax

```
xt::reloadJobs [<filePath: string>]
```

Arguments

filePath

File from which to load jobs into the job manager.

Default: "".

Returns

<none>

xt::removeGPDParasitics

Description

This command removes GPD parasitics data.

Syntax

```
xt::removeGPDParasitics
```

Returns

<string> : Result of removing GPD.

xt::removeJobClassPrefix

Description

Removes a job prefix from the given job class.

Syntax

```
xt::removeJobClassPrefix <prefix: string> -jobClass <xtJobClass>
```

Arguments

`prefix`

The prefix to remove from the job class.

`-jobClass`

The xtJobClass object.

Returns

<none>

xt::reportGPDConfig

Description

Reports the original or modified GPD configuration options for reading GPD parasitic data.

Syntax

```
<string> xt::reportGPDConfig -gpd <string> [-type <enum>]  
[-include_starrc_options]
```

Arguments

`<-return>`

Returns configurations.

`-gpd`

Specifies absolute or relative path to GPD directory.

`-type`

Specifies the type of GPD configuration to report: ({original | user_defined | active }).

`-includeStarRCOptions`

Includes StarRC GPD configuration options in the report.

Returns

Errors

Specifies an invalid path of GPD directory.

xt::reportGPDProperties

Description

Returns a string of properties for the specified GPD.

Syntax

```
xt::reportGPDProperties -gpd <string> [-all <bool>] [-layers <bool>]  
[-corners <bool>]
```

Arguments

`-gpd`

Specifies the path to GPD directory.

`-all`

Reports all types of information.

`-layers`

Reports only the layers.

`-corners`

Reports only the parasitic corners.

Returns

<string> : Properties for the specified GPD.

xt::resetGPDConfig

Description

Cancels custom GPD configuration options previously set by the xt::setGPDConfig command.

Syntax

```
xt::resetGPDConfig -gpd <string>
```

Arguments

-gpd

Specifies absolute or relative path to GPD directory.

Returns

Errors

Specifies an invalid path of GPD directory.

xt::runMultiplePveJobs

Description

Opens the dialog for running multiple PVE jobs.

Syntax

```
xt::runMultiplePveJobs [-window <giWindow | int>]
```

Arguments

-window

Design editor window. The default is -1.

Returns

<giDialog> : Returns the "Run Multiple PVE Jobs" dialog.

xt::runNetListAndSimulate

Description

Runs both the netlister and a simulation process on the design associated with the passed simRun. Corresponds to the menu choice for this action.

Syntax

```
xt::runNetListAndSimulate <simRun: xtSimRun>
```

Arguments

simRun

simRun of the design to be netlisted and simulated.

Returns

<none>

Examples

```
set sim [xt::initRunDir sim3 -context [de::getActiveContext ] \  
        -simulator Verilog]  
xt::runNetlistAndSimulate $sim
```

In this example, xt: runNetlistAndSimulate runs both the netlister and a simulation on the design associated with the passed in simRun.

xt::runNetlister

Description

Performs the interactive portion of the netlisting procedure. This command ensures that all open cell views were saved. If there are open cell views, a check and save dialog asks the user to save the open cell. If the user chooses not to save, a warning that simulation data may not match design data is displayed. To ensure that no user edited files are overwritten when netlisting, a dialog will be raised when an edited file is about to be overwritten. The user must choose between overwriting the file or using the edited file.

After checks are done, this command will call the xtSimulator netlisting callback xt::runNetlisterCbk. In background mode, this command will not prompt the user.

If none of the input to the netlist has changed, then a new netlist will not be generated.

Syntax

```
xt::runNetlister <simRun: xtSimRun> [-generateFinal <bool>]  
[-generateStructural <bool>]
```

Arguments

`simRun`

The `simRun` object.

`-generateFinal`

If true, generates a new final netlist along with any associated checks.

Default: True.

`-generateStructural`

If true, generates a new structural netlist along with any associated checks.

Default: True.

Returns

<none>

Examples

```
set sim [xt::initRunDir sim3 -context [de::getActiveContext ] \
    -simulator Verilog]
xt::runNetlister $sim -generateFinal 1
```

In this example, `xt::runNetlister` generates a Verilog netlist for the specified `xtSimRun` object.

`xt::runNetlisterCb`

Description

Builds the structural netlist, invoking the netlister. This command is the default netlisting Tcl callback used to build the netlist.

The netlister completes these steps:

1. Locks the run directory to prevent file overwrites.
2. Generates a new structural netlist when requested.
3. Generates the final netlist by processing the template markup tags, when requested.
4. Unlocks the run directory.

If your integration overrides the default callback, you must ensure that the above steps are carried out.

The following xtSimRun attributes control the netlister output:

cellView : <dmCellView>

stopList : [<string>]

searchList : [<string>]

netlister : <nI Netlister>

Syntax

```
xt::runNetlisterCbk <simRun: xtSimRun> [-generateFinal <bool>]  
[-generateStructural <bool>]
```

Arguments

simRun

The xtSimRun.

-generateFinal

If true, generates a new final netlist along with any associated checks.

Default: True.

-generateStructural

If true, generates a new structural netlist along with any associated checks.

Default: True.

Returns

<none>

xt::runSimulator

Description

Runs a simulation in the background or in the foreground. This command is called from the menus or from the Netlist or Run dialog box.

Note: This command calls the global ::runSimulator command in background mode.

Syntax

```
xt::runSimulator <simRun: xtSimRun>
```

Arguments

`simRun`

`xtSimRun` object.

Returns

<none>

xt::runSimulatorCbk

Description

Performs all of the steps necessary to run the simulator and post-process the results.

These are the basic steps:

1. Generate or regenerate the final netlist, if required.
2. Lock the run directory.
3. Call `xtSimulator::initSimulatorProc` to prepare the run directory for execution.
4. Call `xtSimulator::launchSimulatorProc` to launch the simulation.

Note:

This routine is called by the global command `::runSimulator`.

Syntax

```
xt::runSimulatorCbk <simRun: xtSimRun>
```

Arguments

`simRun`

The `simRun` object.

Returns

<none>

xt::saveJobClassOption

Description

Opens the job management options dialog box.

Syntax

```
xt::saveJobClassOption [-parent <giDialog | giWindow | int>] [-jobClass  
    <string | xtJobClass>] [-execProc <string>]
```

Arguments

-parent

Parent window or dialog.

-jobClass

Job class

-execProc

Process to be executed after saving settings in this dialog.

Returns

<giDialog> : Dialog object.

xt::savePlotSet

Description

Saves the passed plot set into the specified file for use in a later session. You can clear the plot sets previously stored in a given file by calling this command with no plot sets provided.

Syntax

```
xt::savePlotSet -plots <{<xtPlot>}> -filePath <string>
```

Arguments

-plots

Collection of plots to save.

-filePath

The name of the file where the plot set is stored. The file name is assumed to be relative to the current run directory unless a full path is specified.

Default: plotSet.plt.

Returns

<none>

Examples

```
set xtplot1 [xt::createPlot -name CLK -type net -analysis TRAN \
    -packet annotate1]
set xtplot2 [xt::createPlot -name RESET -type net -analysis \
    TRAN -packet annotate2]
xt::savePlotSet -plots [db::createCollection [list $xtplot1 \
    $xtplot2]] -filePath plotSet.txt
```

In this example, `xt::savePlotSet` saves the plot sets into the `plotSet.txt` file.

xt::saveSimRun

Description

Saves the simulation run data to disk, saves the current run state to the run directory.

All commands that change the state data automatically call `xt::saveSimRun` when the user exits or deactivates a simulation session, creates a new `xtSimRun`, netlists or simulates a design, or changes settings via menu entries or dialogs.

To ensure proper operation, integrators should explicitly make a call to this command in their scripts to respond to modified states.

Syntax

```
xt::saveSimRun <simRun: xtSimRun>
```

Arguments

`simRun`

`xtSimRun` object.

Returns

<none>

xt::sendCommand

Description

Sends a Tcl command to the specified job and waits a number of seconds as specified by the timeout argument. If the timeout is not specified or is 0, then the caller is not blocked.

Important: Calling `sendCommand` on a job that you are currently processing a command from, or vice versa, creates deadlock and cause Custom Compiler to freeze unless the underlying delivery mechanism times out. It is very important that the implementation include a timeout to prevent deadlock.

Syntax

```
xt::sendCommand <command: string> -job <xtJob> [-timeout <int>]
```

Arguments

`command`

Command for the interactive job to execute.

`-job`

The job to execute the command.

`-timeout`

Time, in seconds, to wait for a response (return value) to the command. Timeout of 0 will return immediately.

Default: 0.

Returns

Return value of the command which was sent to the interactive job. (<string>)

Errors

- Timeout expires or message cannot be delivered.
- Failure to send the command to the remote job generates a Tcl error.
- Failure to execute the remote command generates a separate error.

Examples

```
set jobs [xt::getJobs vue*]  
xt::sendCommand "vue_select {mainTab} 0" -job $job
```

In this example, `xt::sendCommand` sends an interactive command to Hercules VUE.

xt::setGPDConfig

Description

Specifies the parameters for reading GPD parasitic data. This command allows the user to override certain parameters for reading parasitic data from a GPD directory using the `xt::readGPDParasitics` command. This command requires write access to the GPD directory.

Syntax

```
<string> xt::setGPDConfig -gpd <string> [-absolute_coupling_threshold  
<float>] [-relative_coupling_threshold <float>]
```

```
[-coupling_threshold_operation <enum>] [-coupling_threshold_operation  
<enum>] [-netlist_select_nets {<string>}] [-netlist_type {<string>}]
```

Arguments

-gpd

Specifies absolute or relative path to GPD directory.

-absoluteCouplingThreshold

Specifies the absolute coupling threshold for filtering coupling capacitors. A capacitor having less than this capacitance value is split into two capacitors to ground.

-relativeCouplingThreshold

Specifies the relative coupling threshold for filtering coupling capacitors. A capacitor is split to ground if its capacitance value divided by the total net capacitance is less than this threshold.

-couplingThresholdOperation

Specifies whether coupling capacitors are filtered when the capacitance value is less than both the absolute AND relative thresholds, or filtered when the value is less than either the absolute OR relative threshold.

-netlistSelectNets

Specifies a subset of nets to be read from the GPD database.

-netlistType

Specifies the parasitics network form to consider when reading the GPD. The allowed settings are no_couple, R, CG, CC, RCG, and RCC.

Returns

Errors

Specifies an invalid path of GPD directory.

xt::setGPDPATHForRead

Description

Specifies GPD Database.

Syntax

```
xt::setGPDPATHForRead [-window <giWindow | int>]
```

Arguments

-window

Design editor window (Default is: -1).

Returns

<none>

xt::setSaveRequiredValue

Description

Sets attribute of job manager used to determine if jobs information file is required to be saved.

Syntax

```
xt::setSaveRequiredValue -value <bool>
```

Arguments

-value

Value to be set for attribute saveRequired.

Returns

<none>

xt::show3DICLibraryAttributes

Description

Shows 3DIC Compiler library attributes.

Syntax

```
xt::show3DICLibraryAttributes -parent <giWindow | giWindowId> [-lib  
  <oaLib>]
```

Arguments

-parent

Parent window.

-lib

3DIC Compiler library.

Returns

<giDialog> :

xt::show3DICRefLibEditor

Description

Shows 3DIC Compiler reference library editor.

Syntax

```
xt::show3DICRefLibEditor -parent <giWindow | giWindowId> [-lib <oaLib>]
```

Arguments

-parent

Parent window.

-lib

3DIC Compiler library.

Returns

<giDialog> :

xt::showCapReportDialog

Description

Opens the **Cap Report** dialog box.

Syntax

```
xt::showCapReportDialog -dbPath <string> -window <giWindow | int>  
[-crossTalk <bool>] [-exclude <string>] [-queryCorner <string>]
```

Arguments

-dbPath

Path to query DB.

-window

Window for the probing.

-crossTalk

Controls whether or not to use crossTalk format.

Default: false.

`-exclude`

List of nets to exclude from victims or net2.

`-queryCorner`

The corner in which to calculate the view.

Returns

<giDialog> : Capacitance Report

xt::showCheckRoutes

Description

Imports DRC Violations from IC Compiler II.

Syntax

```
xt::showCheckRoutes -context <string>
```

Arguments

`-context`

Design context with an open NDM design

Returns

<none>

xt::showColorDecompositionSetup

Description

Sets up and runs a DRC type job.

Syntax

```
xt::showColorDecompositionSetup [-window <giWindow | int>] [-job <string  
| xtPveJob>] [-tool <string | xtTool>]
```

Arguments

`-window`

The Design Editor window. (Default is: -1)

`-job`
PVE Job. (Default is: colorDecomposition)

`-tool`
PVE Tool.

Returns

<giDialog>

The Setup and Run dialog box.

xt::showCreateFrame

Description

Shows Create Data Prep Flow dialog box.

Syntax

```
xt::showCreateFrame (-context <string> | -window <string>)
```

Arguments

`-context`
Design context with an open design.

`-window`
Parent window

Returns

<none>

xt::showCreateICC2Design

Description

Creates a new IC Compiler II design from a Schematic or Verilog netlist.

Syntax

```
xt::showCreateICC2Design -parent <giWindow | giWindowId>
```

Arguments

`-parent`
Parent window.

Returns

<none>

xt::showCreateICCFRAM

Description

Shows the *Create ICC FRAM* dialog box.

Syntax

```
xt::showCreateICCFRAM [-parent <giWindow | int>]
```

Arguments

-parent

Layout window.

Default is: -1.

Returns

<giDialog> : Create IC Compiler FRAM dialog object.

Errors

- Invalid window Id.
- Specified window is not a layout window.

xt::showCreateNewClass

Description

Opens the job management options dialog.

Syntax

```
xt::showCreateNewClass [-parent <giDialog | giWindow | int>] [-jobClass  
  <string | xtJobClass>] [-execProc <string>]
```

Arguments

-parent

Parent window or dialog.

-jobClass

Job class.

`-execProc`

Process to be executed after saving settings in this dialog.

Returns

<giDialog> : Dialog object

xt::showCreateSchematicFromVerilog

Description

Creates schematic from Verilog exported from IC Compiler II.

Syntax

```
xt::showCreateSchematicFromVerilog -context <string>
```

Arguments

`-context`

Design context with an open NDM design.

Returns

<none>

xt::showCrossTalkNetSelector

Description

Opens the crosstalk checker net selector.

Syntax

```
xt::showCrossTalkNetSelector [-window <giWindow | int>]
```

Arguments

`-window`

Design editor window. (Default is: -1)

Returns

<giDialog> : CrossTalk Checker.

xt::showDRCSetup

Description

Shows the DRC Setup and Run dialog box for running a DRC type job.

Syntax

```
xt::showDRCSetup [-window <giWindow | int>] [-job <string | xtPveJob>]  
                 [-tool <string | xtTool>]
```

Arguments

window

Design editor window.

Default: 1.

-job

Physical verification or parasitic extraction job.

Default: drc.

-tool

Physical verification or parasitic extraction tool.

Default: "".

Returns

Setup and Run Dialog. (*giDialog*)

Errors

If the prefs.xml file contains invalid data.

xt::showExport

Description

Shows export dialog.

Syntax

```
xt::showExport -context <string>
```

Arguments

-context

Digital design context with an open design.

Returns

<none>

xt::showFCFrontEndFlow

Description

Shows Fusion Compiler front-end flow.

Syntax

```
xt::showFCFrontEndFlow -parent <giWindow | giWindowId>
```

Arguments

-parent

Parent window.

Returns

<giDialog>:

xt::showFCLibraryAttributes

Description

Displays the Fusion Compiler library attributes.

Syntax

```
xt::showFCLibraryAttributes -parent <giWindow | giWindowId> [-lib  
  <oaLib>]
```

Arguments

-parent

Parent window.

-lib

Fusion Compiler library.

Returns

(*giDialog*)

xt::showFormalityVerification

Description

Runs Formality Verification.

Syntax

```
xt::showFormalityVerification -parent <giWindow | giWindowId>
```

Arguments

-parent

Parent window.

Returns

<giDialog> :

xt::showICC2LibraryAttributes

Description

Displays the IC Compiler II library attributes.

Syntax

```
xt::showICC2LibraryAttributes -parent <giWindow | giWindowId> [-lib  
  <oaLib>]
```

Arguments

-parent

Parent window.

-lib

IC Compiler II library.

Returns

(*giDialog*)

xt::showICC2RefLibEditor

Description

Displays the **ICC2 Reference Library** Editor dialog box.

Syntax

```
xt::showICC2RefLibEditor -parent <giWindow | giWindowId> [-lib <oaLib>]
```

Arguments

-parent

Parent window.

-lib

IC Compiler II library.

Returns

ICC2 Reference Library Editor dialog box.

xt::showICCPlaceAndRoute

Description

Shows the Place & Route With ICC dialog box.

Syntax

```
xt::showICCPlaceAndRoute [-parent <giWindow | int>]
```

Arguments

-parent

Layout editor window.

Default:-1.

Returns

Place & Route With ICC Dialog. (*giDialog*)

Errors

If the prefs.xml file contains invalid data.

xt::showInitializeParasitics

Description

Displays the initialize dialog for setting up parasitic exploration and queries.

Syntax

```
xt::showInitializeParasitics [-window <giWindow | int>]
```


Arguments

`-window`

Design editor window.

Default: -1.

Returns

The Dialog. (*giDialog*)

Errors

Some preferences on the dialog are undefined.

xt::showInitLoad

Description

Shows the initialized load dialog box, which allows the user to start a new netlisting and simulation session using an existing run directory.

Syntax

```
xt::showInitLoad [<window: int>]
```

Arguments

`<window>`

Window ID.

Returns

The dialog handle. (*giDialog*)

xt::showInitNew

Description

Shows the initialize new dialog box, which allows the user to start a new netlisting and simulation session with a new run directory.

If the directory path passed exists, but is not a run directory, a warning dialog is displayed asking if the user would like to overwrite the contents of the directory or cancel the operation.

If the directory path passed exists, and the copy option is also used, the user is asked if they would like to overwrite the existing run or cancel it.

The command [xt::initRunDir on page 2884](#) is called unless canceled by the user.

Syntax

```
xt::showInitNew [<window: int>]
```

Arguments

<window>

Window ID.

Returns

The dialog handle. (*giDialog*)

xt::showJobClassesDefinition

Description

Opens the job management options dialog.

Syntax

```
xt::showJobClassesDefinition [-parent <giDialog | giWindow | int>]  
[-jobClass <string | xtJobClass>] [-execProc <string>]
```

Arguments

-parent

Parent window or dialog.

-jobClass

Job class.

-execProc

Process to be executed after saving settings in this dialog.

Returns

<giDialog> : Dialog object

xt::showJobHostStatus

Description

Creates Background netlist and run progress dialog.

Syntax

```
xt::showJobHostStatus [-parent <giDialog | giWindow | int>] [-job  
  <xtJob>] [-type <int>] [-execProc <string>]
```

Arguments

-parent

Parent window or dialog.

-job

cdpl Job.

-type

Flag to indicate to create dialog, get host status and display host status, or just create and display dialog.

-execProc

Procedure to be executed after saving settings in this dialog.

Returns

<giDialog> : Dialog object.

xt::showJobManagementOptions

Description

Opens the job management options dialog box.

Syntax

```
xt::showJobManagementOptions [-parent <giDialog | giWindow | int>]  
  [-jobClass <string | xtJobClass>] [-execProc <string>]
```

Arguments

-parent

Parent window or dialog.

-jobClass

Job class.

-execProc

Process to be executed after saving settings in this dialog.

Returns

giDialog: Dialog object.

xt::showJobMonitor

Description

Opens Job Monitor and displays the jobs currently stored in the job manager.

Syntax

```
xt::showJobMonitor
```

Returns

The Job Monitor window. ([giWindow](#))

xt::showLPESetup

Description

Shows the dialog box for setting up and running a layout parasitic extraction job process. The options on the dialog are seeded using the current preference values. The OK and Apply buttons save the preferences for the options in the prefs.xml file in the run directory, and the xt::executePv command runs the extraction job.

Syntax

```
xt::showLPESetup [-window <giWindow | int>] [-job <string | xtPveJob>]  
                 [-tool <string | xtTool>]
```

Arguments

-window

Design Editor window.

Default: -1.

-job

PVE job.

Default: lpe.

-tool

PVE tool.

Default: "".

Returns

Setup and Run dialog box (*giDialog*).

Errors

If there are un-set preferences for contents of the options dialog.

xt::showLVSSetup

Description

Shows the dialog box for setting up and a running a layout versus schematic (LVS) comparison job process. It seeds the setup form with values that are described in the Options for LVS run section. OK and Apply buttons execute the xt::executePv command, which saves the preferences for the run in the prefs.xml file in the specified run directory if this preference is set.

Syntax

```
xt::showLVSSetup [-window <giWindow | int>] [-job <string | xtPveJob>]  
                 [-tool <string | xtTool>]
```

Arguments

-window

Design Editor window.

Default: -1.

-job

PVE job.

Default: lpe.

-tool

PVE tool.

Default: "".

Returns

Setup and Run dialog box. (*giDialog*)

Errors

If the prefs.xml file contains invalid data.

xt::showNetBasedCheck

Description

Setup and Run Net Based Check.

Syntax

```
xt::showNetBasedCheck [-window <giWindow | int>]
```

Arguments

-window

Design editor window. (Default is: -1)

Returns

<giDialog> : Setup and Run Dialog.

xt::showNetBasedShieldCheck

Description

Setup and Run Net Based Shield Check.

Syntax

```
xt::showNetBasedShieldCheck [-window <giWindow | int>]
```

Arguments

-window

Design editor window. (Default is: -1)

Returns

<giDialog> : Setup and Run Dialog.

xt::showNetBasedViaCheck

Description

Setup and Run Net Based Via Check.

Syntax

```
xt::showNetBasedViaCheck [-window <giWindow | int>]
```

Arguments

`-window`

Design editor window. (Default is: -1)

Returns

`<giDialog>` : Setup and Run Dialog.

xt::showOpenICC2

Description

Opens the design in IC Compiler II.

Syntax

```
xt::showOpenICC2 -context <string>
```

Arguments

`-context`

Design context with an open NDM design

Returns

`<xtJob>` : Launched job

xt::showParasiticsCompare

Description

Compares Parasitic Value Between Constraints and Layout.

Syntax

```
xt::showParasiticsCompare
```

Arguments

None.

Returns

`<giDialog>` : Compare Parasitic Values dialog

xt::showParasiticsOptions

Description

Displays the options dialog for setting parasitics preferences.

Syntax

```
xt::showParasiticsOptions
```

Returns

The Dialog. (*giDialog*)

Errors

Some preferences on the dialog are undefined.

xt::showPveAboutTools

Description

Show about tools prompt.

Syntax

```
xt::showPveAboutTools
```

Arguments

None

Returns

<none>

xt::showPVEDialog

Description

Setup and Run LVS or LPE job.

Syntax

```
xt::showPVEDialog [-window <giWindow | int>] [-job <string |  
  xtPveJob>] [-tool <string | xtTool>]
```


Arguments

`-window`

Design editor window. (Default is: -1)

`-job`

PVE Job. (Default is LVS)

`-tool`

PVE tool.

Returns

<giDialog>: Setup and Run Dialog

xt::showPveOptions

Description

Shows the physical verification options dialog for configuring general physical verification process settings.

Syntax

```
xt::showPveOptions
```

Returns

<giDialog> Returns the PVE options dialog.

xt::showPveResults

Description

Analyzes and report PVE job results.

Syntax

```
xt::showPveResults -job <xtJob>
```

Arguments

`-job`

The job object

Returns

<bool>: Returns true if job results are clean; false if errors are detected.

xt::showQueryParasiticsReport

Description

Opens the **Query Parasitics Report** dialog box.

Syntax

```
xt::showQueryParasiticsReport <objects: {<deFigure>} | {<deSelectionSet>}  
| {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>} |  
{<oaVia>}> [-title <string>] -window <giWindow | int> [-corner <string>]  
[-point1 <string>] [-point2 <string>]
```

Arguments

objects

Collection of parasitic instances.

-title

Title to display in the results table.

Default: Parasitic Query Results.

-window

The window to use as the parent of the dialog. Because the dialog references database objects from the design in the query window, this argument must be specified.

Default: NONE.

-corner

The corner map value used to calculate the property value of parasitics.

-point1

Instance terminal or subnet among the collection of parasitic instances to use as the starting point for calculating equivalent resistance.

-point2

Instance terminal or sub net among the collection of parasitic instances to use as the ending point for calculating equivalent resistance.

Returns

The dialog box. (*giDialog*)

Errors

Invalid instance list.

xt::showReport

Description

Creates and shows the Report window for the given cell or path.

Syntax

```
xt::showReport (-cell <dmCell | oaCell> | -reportPath <string>)
```

Arguments

-cell

Cell whose Report window to show.

-reportPath

Report path whose Report window to show.

Returns

<giWindow> : Report window.

xt::showRun3DIC

Description

Process the specified design using 3DIC.

Syntax

```
xt::showRun3DIC -context <string>
```

Arguments

-context

Design context with an open NDM design.

Returns

<none>

xt::showRunICC2

Description

Processes the specified design using IC Compiler II.

Syntax

```
xt::showRunICC2 -context <string>
```

Arguments

`-context`

Design context with an open NDM design

Returns

<none>

xt::showSessionJobOptions

Opens the job management options dialog.

Syntax

```
xt::showSessionJobOptions [-parent <giDialog | giWindow | int>] -session  
  <saSession> [-execProc <string>] [-btnName <string>]
```

Arguments

`-parent`

Parent window or dialog.

`-session`

Session.

`-execProc`

Process to be executed after saving settings in this dialog.

`-btnName`

btn Name for the measurementOption or simulationOption.

Returns

<giDialog> : Dialog object

xt::showSetCalibreHighlightColorDialog

Description

Shows the **Set Calibre Highlight Color** dialog box.

Syntax

```
xt::showSetCalibreHighlightColorDialog
```

Arguments

None.

Returns

<none>

xt::showTestbenchJobOptions

Opens the job management options dialog box.

Syntax

```
xt::showTestbenchJobOptions [-parent <giDialog | giWindow | int>]  
    -testbench <saTestbench> [-execProc <string>] [-btnName <string>]
```

Arguments

-parent

Parent window or dialog.

-testbench

Testbench.

-execProc

Process to be executed after saving settings in this dialog.

-btnName

btn Name for the measurementOption or simulationOption.

Returns

<giDialog> : Dialog object

xt::showValidateForDigital

Description

Opens dialog box, which allows the given design to select from a set of requirements which should be met in the design. Upon verification, if any of these rules are violated, appropriate markers are created in the design.

Syntax

```
xt::showValidateForDigital [<cellView: deContext | dmCellView |  
    dmCellViewName>]
```

Arguments

`cellView`

Design which needs to be checked. If not specified, the **Options** dialog box is opened.

Returns

<none>

xt::showValidateForICC

Description

Opens modal **Design Validation Options** dialog box. If `cellView` is specified, then the dialog box has an additional *Validate* button to run validation directly from the dialog box.

The **Design Validation Options** dialog box allows the given design to select from a set of requirements which should be met in the design. Upon verification, if any of these rules are violated, appropriate markers are created in the design.

Syntax

```
xt::showValidateForICC [<cellView: deContext | dmCellView |  
    dmCellViewName>]
```

Arguments

`cellView`

Design which needs to be checked. If not specified, the **Options** dialog box is opened.

Returns

<none>

xt::signalJob

Description

Suspends, resumes, or stops the remote job depending on the supplied arguments. This is not the same as sending a UNIX signal to the process.

Syntax

```
xt::signalJob <job: dbCollection | xtJob> -signal <enum>
```

Arguments

`{xtJob}`

Collection of job objects to which to send signal.

`-signal`

Signal to send to the process: {forcekill | kill | resume | suspend}.

Returns

<none>

Examples

```
xt::signalJob [xt::getJobs] -signal kill
```

In this example, `xt::signalJob` gets and stops all jobs. If any jobs were stopped previously, the system reports an error.

xt::splitVersionString

Description

Splits a Synopsys tool version string into its components, if possible.

Syntax

```
xt::splitVersionString <versionString: string>
```

Arguments

`versionString`

Version string to split.

Returns

<string>: Tcl list of components of the version string, if valid; otherwise, returns an empty string.

xt::startPvDebugger

Description

Launches a physical verification tool's debugger using the `xt::createJob` command. It will open the debugger of the specified PV tool (set in the `xt<runType><tool>Debug` preference), or of the last executed tool.

Syntax

```
xt::startPvDebugger -job <string | xtJob | xtPveJob> -rundir <string>  
[-tool <string | xtTool>] [-window <giWindow | int>]
```

Arguments

-job [xtJob](#)

Previously completed physical verification job containing data for debugging.

-tool <string>

The PV tool.

Default: "".

-runDir <string>

Output run directory of the design to debug. Either the rundir and runType must be specified, or the job.

-window

Design editor window.

Default: -1.

Returns

The running PV debugger job object. ([xtJob](#))

Errors

- If the design cannot be found.
- If the tool was not run.

Examples

```
xt::startPvDebugger -job [xt::getJobs hercules_drc_2]
```

In this example, xt::startPvDebugger launches VUE for a previously completed physical verification job.

xt::testPackage

Description

Tests feature manifest file and prints warning or error messages as it tests the specified file.

Syntax

```
xt::testPackage <filePath: string>
```

Arguments

`filePath`

Package manifest file to test.

Returns

`<bool>` : Return true if warnings or errors are discovered, false otherwise.

xt::unloadPlugin

Description

Unloads a user-defined plugin

Syntax

```
xt::unloadPackage <filePath: string>
```

Arguments

`filePath`

Path to the shared object to unload.

Returns

`<none>`

xt::updateCDPL

Description

Updates CDPL master according to job class.

Syntax

```
xt::updateCDPL -CDPL <xtCDPL> -jobClass <xtJobClass>
```

Arguments

`-CDPL`

The `xtCDPL` object.

`-jobClass`

The `xtJobClass` object.

Returns

<int> : The return code of CDPL create or update, < 0 means fail.

xt::updateJob

Description

Allows a queue engine integrator to update during the queue engine statusProc callback. This command can only be used within the statusProc callback for . It generates an error if it is used at any other time.

Syntax

```
xt::updateJob <none: xtJob> [-status <enum>] [-isQueued <bool>]  
    [-host <string>] [-procId <int>] [-startTime <int>] [-endTime <int>]  
    [-exitStatus <int>] [-progress <string>]
```

Arguments

none

The job to update.

-status

The updated status of the job: {FAILED | FINISHED | | RUNNING |
SUSPENDED | UNKNOWN}.

-isQueued

Indicates if the job is currently queued.

-host

Host where the job is running.

-procId

Process ID (PID) of the job.

-startTime

The time at which the job started. Expressed as the number of seconds since the epoch.

-endTime

The time at which the job finished. Expressed as the number of seconds since the epoch.

-exitStatus

The exit code of the job.

`-progress`

Displays progress of the job.

Returns

<none>.

xt::validateForDigital

Description

Validates design for the requirements stored in the preferences set by GUI version of this command. Upon verification, if any of these rules are violated, appropriate markers are created in the design.

Syntax

```
xt::validateForDigital <cellView: deContext | dmCellView |  
dmCellViewName>
```

Arguments

`cellView`

Design that needs to be checked.

Returns

<bool> :

xt::validateForICC

Description

Validates design for the requirements stored in the preferences set by the GUI version of this command. Upon verification, if any of these rules are violated, appropriate markers are created in the design.

Syntax

```
xt::validateForICC <cellView: deContext | dmCellView | dmCellViewName>
```

Arguments

`cellView`

Design that needs to be checked.

Returns

<none>

xt::viewRunLog

Description

Opens the run log in the Text Viewer. Corresponds to the menu choice for this action.

Syntax

```
xt::viewRunLog
```

Returns

<none

xt::viewSimInput

Description

Opens the final netlist in the Text Viewer. Corresponds to the menu choice for this action.

Syntax

```
xt::viewSimInput
```

Returns

<none

xt::viewSimOutput

Description

Opens all the simulation output files in the text viewer. Corresponds to the menu choice for this action.

Syntax

```
xt::viewSimOutput
```

Returns

<none

xt::wait

Description

This command waits for the job to finish.

Syntax

```
xt::wait <target: dbCollection | script | xtJob>
```

Arguments

target

The jobs to wait for, or script that creates jobs.

Returns

<string>: Job status.

xt::writeJob

Description

Writes data to stdin of the remote job, waiting a number of seconds for the write process to complete, as specified by the timeout argument. If the timeout is not specified, or is 0, then the writeJob command does not wait.

Syntax

```
xt::writeJob <job: xtJob> -data <string> [-timeout <int>]
```

Arguments

xtJob

The job to write data to.

-data <string>

Data to send to the interactive job.

-timeout <int>

Time, in seconds, to wait for the data to be sent. Timeout of 0 returns immediately.

Default: 0.

Returns

<none>

xt::writeParasitics

Description

Writes a list of parasitics to an output file.

Syntax

```
xt::writeParasitics <objects: {<deFigure>} | {<deSelectionSet>} |  
  {<oaFig>} | {<oaFigGroup>} | {<oaInst>} | {<oaMarker>} | {<oaShape>}  
  | {<oaVia>}} -file <string> [-format <enum>] [-description <string>]  
  [-totalResistance <string>] [-corner <string>]
```

Arguments

`objects`

Collection of parasitic instances.

`-file`

Output file name

`-format`

Format of the output file ({`ascii` | `csv` | `xml`}). Default is: `ascii`.

`-description`

Description of query details. (Default is:)

`-totalResistance`

Total resistance between subNodes associated with instTerms. ("fromNetName toNetName", Default is:)

`-corner`

Corner for parasitic value. (Default is:)

Returns

<none>

ixt::crossProbe

Description

Prompts the user to select the required input in the current active window; a net for the net probe, and a device for the device probe. This object is used as the object argument for the `xt::crossprobe` command and will be highlighted in the other window.

Syntax

```
ixt::crossProbe [<context: deContext | unsigned int>] [-type <enum>]  
  [-bus <enum>]
```

Arguments

`context`

Context of activation.

`-type`

Type of probe to create. Select the required input for the type of cross probe specified (`{ auto | device | net }`)

`-bus`

Mode of bus net probe. Select the optional input for the bus net mode of cross probe specified (`{ all | selectBits }`)

Returns

<none>

ixt::editPlotSet

Description

Adds and removes nets and ports from the active plotset. Interactively selects and deselects nets and terminals for inclusion in the current plot set. Upon startup, the contents of the current plot set are highlighted with deProbes. If the net or terminal the user clicks on does not exist in the xtSimRun's plot set, a xtPlot is not generated for the net/terminal.

An xtPlot is allotted a packet from the color wheel and given an empty analysis, which means all available analyses will be plotted. The xtPlot is then added to the xtSimRun's plotSet and a deProbe put on the schematic to highlight the net or terminal. If there are one or more xtPlots with the chosen terminal or net, or an equivalent net, these are removed from the plotSet and their corresponding deProbes are removed. At the end of the interactive command, the plotSet's deProbes are cleared.

Syntax

```
ixt::editPlotSet [<context: deContext | unsigned int>]
```

Arguments

`context`

Context of activation.

Returns

<none>

ixt::showGPDParasiticsOptions

Description

Opens the LE GPD Parasitic option dialog box.

Syntax

```
ixt::showGPDParasiticsOptions [-parent <giWindow | int>]
```

Arguments

-parent

Window(id) of layout editor window to query GPD Parasitic

Returns

<giDialog> : The Dialog.

ixt::showGPDParasiticsOptions

Description

Opens the LE GPD Parasitic option dialog box.

Syntax

```
ixt::showGPDParasiticsOptions [-parent <giWindow | int>]
```

Arguments

-parent

Window(id) of layout editor window to query GPD Parasitic

Returns

<giDialog> : The Dialog.

ixt::plotWaveform

Description

Interactively plots waveforms selected from schematic. The user clicks on the pins and nets they want to plot and the signals are sent to the waveform viewer for graphing. A deProbe will mark the net or terminal with the same color as used in the waveform viewer.

From the Command Options Dialog, a user can:

- Select the analysis they would like plotted.
- Plot the signals in a new graph in the waveform viewer.
- Append to an existing graph in the waveform viewer.
- Replace the existing graphs contents in the waveform viewer.

The user can also choose to append a selected plot to the current plot set.

Syntax

```
ixt::plotWaveform [<context: deContext | unsigned int>]
```

Arguments

context

Context of activation.

Returns

<none>

ixt::queryGPDParasitic

Description

Opens the LE Query Parasitic dialog box.

Syntax

```
ixt::queryGPDParasitic [-parent <giWindow | int>]
```

Arguments

-parent

Window(id) of layout editor window to query GPD Parasitic

Returns

<giDialog> : The Dialog

ixt::queryParasitics

Description

Starts the interactive Query Parasitics process. Prompts the user to select the required inputs in the current active window: auto, a net for the net query, two nets for the netToNet

query, and two instance terminals for the pointToPoint query. These objects are used as arguments for the `xt::queryParasitics` command. A report dialog is displayed by calling the `xt::showQueryParasiticsReport` command on the resulting list of parasitics. If no parasitic devices are found, an information dialog is displayed with this information instead of creating an empty report dialog.

Syntax

```
ixt::queryParasitics [<context: deContext | unsigned int>] [-type  
  <string>]
```

Arguments

`context`

Context of activation.

`-type`

Type of query to perform. Select the required inputs for the type of query specified (`{ auto | net | netToNet | pointToPoint }`) (Default is: `auto`)

Returns

`<none>`

Enumerations

Table 206 Enumerations

Name	Values	Description
<code>enCrossprobeType</code>	<code>device net</code>	The type of probe to add in the schematic and parasitic views.
<code>enIntegrationType</code>	<code>simulator</code>	The type of integration. NOTE: Currently, only simulator integrations can be registered, but this enumeration is kept for future compatibility.
<code>enMapDir</code>	<code>forward reverse</code>	Direction to use when passing names to name map object.
<code>enParasiticsFormat</code>	<code>ascii csv</code>	The format of the parasitics output or input file.
<code>enParasiticsType</code>	<code>net netToNet pointToPoint allNets</code>	The type of parasitic query or report.

Table 206 Enumerations (Continued)

Name	Values	Description
enPlotCommand	new append replace	The type of plot operation requested by the user.
enPlotType	net port instParam	The type of design object selected for a plot.
enProbeType	cellview terminal instance instTerm net path	Probe types.
enPVDBFormat	openAccess stream netlist	The database format of the schematic or layout.
enPVRunType	drc lvs lpe	The type of the verification run.
enXtColumn	name jobType cmdLine runDesc files host proclId status startTime exitTime	Columns in the Job Monitor Spreadsheet widget.
enXtSignal	kill suspend resume	Signal to send to a process.

Objects

This section covers the following:

- [deContext](#)
- [deProbe](#)
- [dmCell](#)
- [dmCellView](#)
- [giWindow](#)
- [nlNameMap](#)
- [nlNet](#)
- [nlNetlister](#)
- [oalnst](#)
- [oalnstTerm](#)
- [oaNet](#)
- [xtlIntegration](#)

- [xtJob](#)
- [xtJobClass](#)
- [xtJobGroup](#)
- [xtJobLimit](#)
- [xtPlot](#)
- [xtPlotViewer](#)
- [xtQueueEngine](#)
- [xtSimRun](#)
- [xtSimulator](#)

deContext

The deContext object represents an editing context which may or may not be open in a window. These objects are created by the de::open command. The attributes of the object are described in the table below and can be retrieved by db::getAttribute. This object may not be destroyed using the db::destroy command.

Table 207 *Attributes*

Name	Type	Access	Description
topFile	<i>dmDMFile</i>	Read-only	Top file.
topDesign	<i>oaDesign</i>	Read-only	Design associated with the top file. May be un-set (empty string) if the top file is not an oaDesign.
viewFile	<i>dmDMFile</i>	Read-only	View file.
viewDesign	<i>oaDesign</i>	Read-only	Design associated with the view file. May be un-set (empty string) if the view file is not an oaDesign.
editFile	<i>dmDMFile</i>	Read-only	Edit file.
editDesign	<i>oaDesign</i>	Read-only	Design associated with the edit file. May be un-set (empty string) if the edit file is not an oaDesign.

Table 207 *Attributes (Continued)*

Name	Type	Access	Description
hierarchy	<i>heContext</i>	Read-only	Current descend path represented in an <i>heContext</i> object from the Hierarchy Editor Functional Specification. The returned object is a copy and may be modified or destroyed without affecting the <i>deContext</i> .
window	<i>giWindow</i>	Read-only	The window displaying this context. If opened in headless mode, this attribute will be the empty string.

deProbe

Represents a probe. It may be obtained using the `de::getProbes` command and created using the `de::createProbes` command. This object may be destroyed with the `db::destroy` command.

Table 208 *Attributes*

Name	Type	Access	Description
probeType	<i>enProbeType</i>	Read-only	Probe type.
tool	string	Read-only	Tool which created this probe object.
objectName	string	Read-only	The name of the probed object in the target design.
context	<i>heContext</i>	Read-only	Hierarchical context of probed object.
packet	string	Read/Write	Display packet alias name.
description	string	Read-only	Optional string which allows for further filtering and/or grouping of probes. Should be contain a very short description of the probe (1-4 words recommended).
longDescription	string	Read/Write	Detailed description of the probe object.

Table 208 *Attributes (Continued)*

Name	Type	Access	Description
purpose	<i>enProbePurpose</i>	Read/Write	Probe purpose.

dmCell

A named container, in a library, that can hold files and dmCellViews. The dmCell corresponds to a specific oaCell object but is valid as long as the library that the dmCell belongs to remains open. The getAccess requirements of an oaCell are handled automatically by this dm object.

Table 209 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the Cell.
libName	string	Read-only	The name of the Cell's library.

dmCellView

The dmCellView is a container in a library that holds files and is identified by the combination of its cell and view names. It corresponds to an oaCellView but it stays valid as long as its library remains open and its dmCell and dmView exist.

Table 210 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the view that this cellView belongs to.
cellName	string	Read-only	The name of the cell that this cellView belongs to.
cell	<i>dmCell</i>	Read-only	The cellView's cell.
libName	string	Read-only	The name of the library that this cellView belongs to.

giWindow

The `giWindow` object represents a single window and has a container of menus, toolbars, assistants and one status bar. Menus, toolbars and assistants may be retrieved by the API. The status bar and toolbar that are used to hold menus are available here as attributes as they are non-removable attributes of the window object. The attributes of the object are described in the table below and can be retrieved or set by `db::getAttribute` and `db::setAttribute` commands. Windows are created by various subsystems in Custom Compiler including Design Editor.

Table 211 *Attributes*

Name	Type	Access	Description
id	int	Read-only	Window identifier.
windowType	<i>giWindowType</i>	Read-only	Window type object.
title	string	Read-only	Title displayed in window border.
iconTitle	string	Read/Write	Title given to the window's icon and displayed in the window manager's task list.
geometry	string	Read/Write	Window geometry.
iconified	bool	Read/Write	Whether the window is iconified.
maximized	bool	Read/Write	Whether the window is maximized. Note that this can be true even if the window is currently iconified.
statusBar	<i>giStatusBar</i>	Read-only	Status bar object holding actions that will render this window's status bar. If empty, the status bar will not be displayed in the window.

nlNameMap

The `nlNameMap` object provides utilities and a common interface for creating, storing and retrieving mapping data, including serialization/de-serialization to or from disk and runtime storage and retrieval of name table data. `nlNameMap` does not provide algorithms for transforming identifiers between namespaces. The algorithms for transforming names is

separate code and plugs into the name map in a generic fashion. See `nIONameMap` and `nIParameterizedNameMap` for implementations of the `nINameMap` interface.

Table 212 *Attributes*

Name	Type	Access	Description
algo	bool	Read-only	If the new name map is algorithmic or if it contains table data.
openBrace	string	Read-only	Character used to indicate the beginning of a bit index in the destination namespace.
closeBrace	string	Read-only	Character used to indicate the end of a bit index in the destination namespace.
hierDelimiter	string	Read-only	The hierarchy delimiter/separator used in the destination namespace by <code>nIHierMap</code> objects.
rootContext	<i>heContext</i>	Read-only	The context object used to elaborate the design hierarchy by <code>nIHierMap</code> objects.
uniqueifyProc	string	Read-only	The procedure used in the write API to make identifiers unique when scope conflicts occur.

nINet

Represents nets, buses and bundles in the Simplified Connectivity API. Due to virtual terminals, some referenced nets may not exist in the given module. Furthermore, this object can represent an "anonymous" net. This is needed for over-declared wires in Verilog. The names for anonymous nets are automatically generated and are guaranteed to not connect by name to any other net in the design or to any globals throughout the design hierarchy.

Table 213 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the net/bus/bundle.
netType	<i>enNetType</i>	Read-only	The type of this net.

Table 213 *Attributes (Continued)*

Name	Type	Access	Description
minIndex	int	Read-only	The Minimum numeric index of the net. Only valid for Bus type nets. This corresponds to the first bit in the bus if the bit order is ascending.
maxIndex	int	Read-only	The maximum numeric index of the net. Only valid for Bus type nets. This corresponds to the last bit in the bus if the bit order is ascending.
repeat	int	Read-only	The number of times the bit/bus/bundle name is repeated.
bitOrder	<i>oaBitOrderEnum</i>	Read-only	The bit order of the bus. Only valid for Bus type nets. This attribute will always be either ascending or descending. It will never be none.
width	int	Read-only	The number of members of this net. This is computed as: repeat*(maxIndex-minIndex+1)
index	int	Read-only	The index of this bit. Only valid for Bit type nets.

nlNetlister

Provides the framework for registering custom event handling Tcl procedures. For each structural event as defined in Events and Handlers, a Tcl procedure can be registered by setting the corresponding <event name>Proc attribute to the name Tcl procedure. If the value registered for a procedure is the empty string ("") then the event will be skipped. The netlister framework also uses the values of attributes stored in the nlNetlister object to control framework functionality such as global net collection and feed-through removal. Netlister objects are typically created at tool startup and a single object is created for each output format. A single netlister object can be used to create all the netlists for a specific output format in an Custom Compiler session. The attributes of the object are described in the table below and can be retrieved or set by the db::getAttr and db::setAttr commands.

Table 214 Attributes

Name	Type	Access	Description
name	string	Read-only	Netlister Name.
collectGlobals	bool	Read/Write	If all global nets should be collected before the Parts List is processed. This is needed for Spice, but not needed for Verilog.
supportsFeed Throughs	bool	Read/Write	If the formatter supports feed-throughs. If it does not support feed-throughs, then the netlister will handle shorted terminals by installing name aliases in the name mapping object and removing all but one of the shorted terminals from the nlTerm list returned by nl::getTerms.
scalarizeTerminals	bool	Read/Write	If the netlister should create scalarized terminal lists.
commentPrefix	string	Read/Write	The prefix to use for single line comments.
indentDepth	int	Read/Write	The number of spaces to use for each level of indentation.
maxLineLength	int	Read/Write	The maximum number of characters to print before wrapping a line.
lineWrapSuffix	string	Read/Write	The string to use to wrap long lines.
netlistInfo	string	Read/Write	The identifier used for querying the Netlist Info for netlisting information. This is used by various netlisting utility functions such as nl::findInstNamePrefix, nl::getTerms and any other logic that requires Netlist Info data.
nameMapProc	string	Read/Write	The name of the Tcl procedure called to generate a Name Mapping Object.

Table 214 Attributes (Continued)

Name	Type	Access	Description
netlistMainProc	string	Read/Write	The name of the Tcl procedure called for NetlistMain events.
netlistBeginProc	string	Read/Write	The name of the Tcl procedure called for NetlistBegin events.
netlistBodyProc	string	Read/Write	The name of the Tcl procedure called for NetlistBody events.
createTopAsModule	bool	Read/Write	Controls whether ModuleHeader/ModuleFooter events are generated for the top module.
modulePreprocessProc	string	Read/Write	The name of the Tcl procedure called for ModulePreprocess events.
moduleHeaderProc	string	Read/Write	The name of the Tcl procedure called for ModuleHeader events.
textModuleProc	string	Read/Write	The name of the Tcl procedure called for TextModule events.
moduleBodyProc	string	Read/Write	The name of the Tcl procedure called for ModuleBody events.
sortInstHeaders	<i>enSortOrder</i>	Read/Write	Controls whether instance header events should be generated in a sorted order based on switched master LCV.
instHeaderProc	string	Read/Write	The name of the Tcl procedure called for InstHeader events.
scalarizeInsts	bool	Read/Write	Controls if instances are scalarized.
sortInsts	<i>enSortOrder</i>	Read/Write	Controls if instances are sorted by instance name.
moduleFooterProc	string	Read/Write	The name of the Tcl procedure called for ModuleEnd events.
netlistEndProc	string	Read/Write	The name of the Tcl procedure called for NetlistEnd events.

Table 214 Attributes (Continued)

Name	Type	Access	Description
moduleOrder	<i>enModuleOrder</i>	Read/Write	The order in which modules will be written to the final netlist file. Note that this has no effect on the order that modules are netlisted.
defaultNetlistProc	string	Read/Write	The name of the Tcl command to use for helnstHeaders that do not specify a netlistProcedure field within their associated CDF.
allowNameMapProc	string	Read/Write	The name of the Tcl procedure to call to determine if it is valid to name map the module's name.
isTextModuleProc	string	Read/Write	The name of the Tcl command to use to determine if an heContext is a text module.
nameMap	<i>nlNameMap</i>	Read-only	The name map object used for a specific netlister invocation.
currentContext	<i>heContext</i>	Read-only	The current context we are netlisting. This is set internally whenever the Module event is generated.
fileName	string	Read-only	The invocation specific output filename.
partsList	[<i>heContext</i>]	Read-only	The list of non-leaf modules found during the first pass of netlisting.
leafList	[<i>heContext</i>]	Read-only	The list of leaf modules found during the first pass of netlisting.

oalnst

Represents block domain instances. An instance represents the inclusion of one design as a part of the contents of another. Use the db::getAttr command to retrieve the oalnst attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 215 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the oalnst.
design	<i>oaDesign</i>	Read-only	Design to which the instance belongs.
master	<i>oaDesign</i>	Read-only	Master of the instance.
orient	<i>oaOrientEnum</i>	Read/Write	Orientation of the instance.
origin	point	Read/Write	Origin of the instance.
libName	string	Read-only	The library name for the master design referenced by the instance.
cellName	string	Read-only	The cell name for the master design referenced by the instance.
viewName	string	Read-only	The view name for the master design referenced by the instance.
header	<i>oaInstHeader</i>	Read-only	The instHeader associated with this instance.
numBits	int	Read-only	The number of bits of this instance.
params	<i>oaParamArray</i>	Read/Write	Parameters that were specified during creation. Only Pcell instances have parameters.
placementStatus	<i>oaPlacementStatusEnum</i>	Read/Write	The placement status for the instance.
source	<i>oaSourceEnum</i>	Read/Write	The source for the instance.
isImplicit	bool	Read-only	A boolean that indicates if the instance is implicit.

oalnstTerm

Represents all types of connections between a net and a terminal in the master of an instance. The connection is valid if the number of bits in the net equals the number of bits implied by the instTerm's terminal name. Use the db::getAttr command to retrieve the

oaInst attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 216 Attributes

Name	Type	Access	Description
name	string	Read-only	The name of the term associated with this instTerm.
net	<i>oaNet</i>	Read-only	The net associated with this instTerm.
inst	<i>oaInst</i>	Read-only	The inst associated with this instTerm.
term	<i>oaTerm</i>	Read-only	The term associated with this instTerm.
termPosition	int	Read-only	The term position that this instTerm connects to.
numBits	int	Read-only	The number of bits for this instTerm.
isImplicit	bool	Read-only	A boolean value indicating whether this instTerm is implicit.

oaNet

Represents scalar and multi-bit nets that are part of the design's physical description. Nets represent the logical connectivity within a block of a design. Nets connect to terminals, which are the logical connection points on instances of the net's block. Nets also connect to instTerms that represent connections to the lower-level instances in the net's block. Use the `db::getAttr` command to retrieve the *oaNet* attributes described in the following table. (For the full list of attributes, see the Si2 OpenAccess documentation.)

Table 217 Attributes

Name	Type	Access	Description
name	string	Read-only	Name of <i>oaNet</i> .
design	<i>oaDesign</i>	Read-only	Design to which the net belongs.

Table 217 *Attributes (Continued)*

Name	Type	Access	Description
isEmpty	bool	Read-only	A boolean value that indicates if the net has any objects attached to it or associated with it.
isGlobal	bool	Read/Write	A boolean value that indicates if the net is a global.
isImplicit	bool	Read-only	A boolean value that indicates if the net is implicit.
numBits	int	Read-only	The number of bits in the net.
sigType	<i>oaSigTypeEnum</i>	Read/Write	The signal type of the net.

xtlIntegration

Provides an interface to register the location of simulator integrations. The automatic registration of integrations occurs during startup, however, integrations can be explicitly registered by users at any time. Once registered, the actual initialization of the integration is deferred until needed by a simulator tool.

Table 218 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the integration.
integrationType	<i>enIntegrationType</i>	Read-only	The type of the integration.
location	string	Read-only	The directory location closest to the root in the installation hierarchy where the integration is stored. The interpretation of the directory contents during initialization is dependent on the integration type.
isInitialized	bool	Read-only	True, if the integration has already been initialized.

xtJob

An xtJob stores information for a job launched by a Custom Compiler session. This information is used for display by the job monitor and for inter-process communication. The attributes of the object are described in the table below and can be retrieved and (in some cases) set by the db::getAttr and db::setAttr commands.

Table 219 *Attributes*

Name	Type	Access	Description
name	string	Read-only	Name of the job.
type	string	Read-only	Job type, either "batch", "foreign", or an application name.
cmdLine	string	Read-only	Command used to launch the job.
runDesc	string	Read/Write	Descriptive text for the job.
files	[string]	Read-only	List of filenames to track. All files in this list will be deleted before the child process is launched. They are deleted to prevent confusion between old and new output data.
readOnlyFiles	[string]	Read-only	List of read-only files to track. These files will NOT be deleted before the child process is launched. This attribute is meant for use with input files for the child process, files that are generated before it is launched.
host	string	Read-only	Host on which to run the job.
data	string	Read/Write	Arbitrary string data associated with the job.
procId	int	Read-only	UNIX pid of the process.
readyProc	string	Read-only	Procedure to call once the job is initialized and ready for input. Interactive jobs only.
ready	<bool>	Read-only	Indicates if the job is initialized and ready for input. Interactive jobs only.

Table 219 *Attributes (Continued)*

Name	Type	Access	Description
status	<i>enXtStatus</i>	Read-only	Current job status.
exitProc	string	Read-only	Procedure to call once the job has exited. Will be called regardless of how the job shut down.
exitStatus	int	Read-only	Operating system issued exit status of the process.
startTime	string	Read-only	Time the job was started.
exitTime	string	Read-only	Time the job exited.
creationTime	<string>	Read-only	Time the job was created.
isLocallyQueued	<bool>	Read-only	Indicates if the job is currently locally queued waiting on a concurrent job slot.
queueEngine	<xtQueueEngine>	Read-only	Queue engine used to start the job. This attribute will be "" if this is not a batch job, or if no queue engine was used to start the job.
isQueued	<bool>	Read-only	Indicates if this job is queued by a queue engine or not. If the job is not a batch job, or was not started using a queue engine, this attribute will be false.
queueIdentifier	<string>	Read-only	String used to associate the xtJob with an entry in the selected queueEngine (for example, SGE job id). This attribute will be "" if this is not a batch job, or if no queue engine was used to start the job.
deferred	<bool>	Read-only	Indicates if the job launch has been deferred.
arraySize	<int>	Read-only	If this job is part of a queue engine job array, this attribute stores the number of sibling jobs within the array.

Table 219 *Attributes (Continued)*

Name	Type	Access	Description
group	<xtJobGroup>	Read-only	This job's parent job group. This attribute will be "" if there is no parent job group.

xtJobClass

The xtJobClass is a one-to-many association or mapping of xtJob prefixes used to allow different jobs' prefixes to be associated with different job distribution configuration attributes.

Table 220 *Attributes*

Name	Type	Access	Description
name	string	Read-only	The name of the Job Class, which is set on creation of the object.
prefixes	string	Read-only	A collection of xtJob prefixes that are associated with this xtJobClass. Prefixes can only be added or removed via the xt::addJobClassPrefix and xt::removeJobClassPrefix commands.

xtJobGroup

An xtJobGroup is used to arrange jobs into a hierarchical organization, for user convenience when displayed in the Job Monitor. The attributes of the object are described in the table below and can be retrieved and (in some cases) set by the db::getAttr and db::setAttr commands.

Table 221 *Attributes*

Name	Type	Access	Description
name	<string>	Read-only	Name of the job group.
status	<string>	Read-only	Status of job group.
detailedStatus	<string>	Read-only	Status for group-filtered view.

Table 221 *Attributes (Continued)*

Name	Type	Access	Description
detailedStatusProc	[<string>]	Read/Write	Callback to produce the detailed status message.
runDesc	<string>	Read/Write	Descriptive string for the group.
expanded	<bool>	Read/Write	The group should be displayed expanded in the Job Monitor tree.
trackProgress	<bool>	Read/Write	If true, the group computes the summarized progress of all its contained jobs, and any other contained groups with trackProgress true.
showProgress	<bool>	Read/Write	If true, displays the computed progress summary (see trackProgress); if false, computation may still be done depending on trackProgress, but it is not displayed in the Job Monitor.
initOptionsDialogProc	[<string>]	Read/Write	Callback proc to initialize the Job Management Options dialog box.
parent	<xtJobGroup>	Read-only	Parent group ("" indicates top-level group).
groups	[<xtJobGroup>]	Read-only	Collection of child groups.
jobs	[<xtJob>]	Read-only	Collection of contained ("child") jobs.
exitProc	<string>	Read/Write	Procedure to call when all of the jobs contained within the job group hierarchy have exited.

xtJobLimit

An xtJobLimit represents a limit to the number of concurrent batch jobs allowed for a certain job name prefix. This object can be destroyed using db::destroy. Destroying the xtJobLimit will not destroy the associated preferences

Table 222 *Attributes*

Name	Type	Access	Description
name	<string>	Read-only	Batch job name prefix.
prefName	<string>	Read-only	Name of the preference that stores the value of the limit.
limitEnabledPref Name	<string>	Read-only	Name of the preference that indicates if the concurrent jobs of this name are limited.
concurrentJobs	[<xtJob>]	Read-only	Collection of jobs with the given prefix that are concurrently running.
queuedJobs	[<xtJob>]	Read-only	Collection of jobs that are locally queued, waiting for a concurrent job slot to be freed before they are executed.

xtPlot

A plot object is a Tcl object associated with a waveform viewer probe.

Table 223 *Attributes*

Name	Type	Access	Description
name	string	Read/Write	The hierarchical name of the net, port, or parameter in the tool namespace.
plotType	<i>enPlotType</i>	Read/Write	The type of the plot object name.
analysis	string	Read/Write	The name of the analysis associated with the plot.
packet	string	Read/Write	The name of a drPacket to highlight when selecting, and whose outline color will be suggested to the viewer as a plot color.

xtPlotViewer

Represents an output device for use with interactive plotting. The xtPlotViewer contains a viewer command that launches a viewer, as well as a procedure to format commands used by the viewer.

Table 224 Attributes

Name	Type	Access	Description
name	string	Read-only	String identifier associated with the plot viewer type.
job	xtJob	Read/Write	The xtJob object associated with the viewer. Many routines are only valid when this attribute is non-nil. It will be nil before the viewer is launched and after it has been killed.
logfile	string	Read-only	Name of the file that logs stdout and stderr from the viewer.
getLaunchCommandProc	string	Read/Write	Returns a command that launches the viewer. This command is used by the default implementation of the launchViewerProc. This command must follow standard Bourne-style shell syntax.
launchViewerProc	string	Read/Write	Launches the viewer using Tool Integration routines to create an xtJob.
getActivateScriptProc	string	Read/Write	The name of the Tcl procedure called after the viewer is launched to give the integrator a chance to return a script to put it in a usable state.
getDeactivateScriptProc	string	Read/Write	Routine will be called when the netlisting environment is deactivated, giving the integrator a chance to return a script to save the viewer state.

Table 224 Attributes (Continued)

Name	Type	Access	Description
getPlotScriptProc	string	Read/Write	The name of the Tcl procedure to return the command that should be sent to the viewer to perform the requested plotting. NOTE: The same IPC call can be used regardless of the type of underlying communication channel used.
getReplotScriptProc	string	Read/Write	The name of the Tcl procedure for return commands and/or code to be sent to for the viewer to trigger a reload or replot of data.
getEquivalentNetsProc	string	Read/Write	Name of the Tcl procedure called when the waveform viewer cannot find the named signal.

xtQueueEngine

An xtQueueEngine represents an integrated third-party distributed computing software tool. The object stores the names of integrator-defined procedures to realize the integration.

Table 225 Attributes

Name	Type	Access	Description
name	<string>	Read-only	Name of the queue engine.
initProc	[<string>]	read/write	The procedure to call to detect if the queue engine is available in the user's environment.
startProc	[<string>]	read/write	Procedure to call to launch a job using the queue engine
statusProc	[<string>]	read/write	Procedure to call to obtain status information about the job from the third party software.
killProc	[<string>]	read/write	Procedure to call to terminate the job or remove it from the queue.

Table 225 *Attributes (Continued)*

Name	Type	Access	Description
suspendProc	[<string>]	read/write	Procedure to call to suspend the job.
resumeProc	[<string>]	read/write	Procedure to call to resume the job.
configurationProc	[<string>]	read/write	Procedure to call when the Configure button is clicked in the Job Options dialog. This should spawn an Opal dialog meant to allow the user to change various settings for the third party software.
enabled	[<string>]	read-only	This attribute indicates whether the queue engine was successfully initialized. If it is false, the queue engine will be unavailable to start new jobs.
createPrefsProc	[<string>]	read/write	The procedure to call on startup to create the preferences for this integration.
arraysEnabledProc	[<string>]	read/write	The procedure to call to check if job arrays are enabled for this queue engine.
launchArrayProc	[<string>]	read/write	The procedure to call to launch a list of jobs as a queue engine job array.
bulkKillProc	[<string>]	read/write	The procedure to call to kill a list of jobs. If the queue engine supports job arrays, it may kill the entire list of jobs in a single command.

xtSimRun

Provides an interface to all of the persistent and transient users settings associated with a particular run directory. The xtSimRun is saved out automatically to the run directory whenever a key action like netlisting or simulating is performed using the xt:::saveSimRun call. This gives the user the perception of a persistent data storage that does not require explicit saving or loading. The xtSimRun owns the run directory associated with it.

Table 226 *Attributes*

Name	Type	Access	Description
runDir	string	Read-only	The name of the directory associated with this simrun.
logFile	string	Read-only	The name of the file to be used for logging netlisting execution either in the foreground or background. The only log messages that do not appear in this file are messages directly from the simulation process.
lockFile	string	Read-only	The name of the file used as a lock file for the run directory: ".rundir_lock". The designer's UNIX user name will be written to the file.
simulator	<i>xtSimulator</i>	Read-only	The current simulator object*.
viewer	<i>xtPlotViewer</i>	Read-only	The current viewer object*.
nameMap	<i>nlNameMap</i>	Read/Write	The current namemap object, as saved to disk by the netlister.
netlistLanguage	string	Read/Write	Name of the language of structural netlist.
cellView	<i>dmCellView</i>	Read/Write	The cellview containing the top design.
searchList	[string]	Read/Write	Ordered listing of search views.**
stopList	[string]	Read/Write	Ordered listing of stop views.**
contexts	{ <i>deContext</i> }	Read/Write	Collection of contexts associated with the given simRun*.
plotSet	{ <i>xtPlot</i> }	Read/Write	A collection of plot objects that represents the active plot set. When the user plots the selected set, this set will be provided to the viewer procedures.

xtSimulator

A simulator Tcl object is used for all netlisting and simulation activities and is closely associated with an xtSimRun Tcl object. While an xtSimRun contains persistent data saved across runs, the simulator object contains transient data and data is not duplicated between these Tcl objects. The xtSimulator is instantiated from a simulation integration.

Table 227 *Attributes*

Name	Type	Access	Description
simulator	string	Read-only	Name of the simulator integration.
outputDir	string	Read-only	Name of the directory, relative to the run directory, in which to place the output files of simulations. This string name is provided as a convenience to integrators.
finalNetlist	string	Read/Write	Name of the file in the launch directory which contains the final netlist. This string name is modifiable to accommodate simulators with strict file naming requirements.
netlistTemplate	string	Read-only	Name of the netlist template file relative to the run directory: netlist.template.
netlistDir	string	Read-only	Name of the directory, relative to the run directory, in which to run the netlister. This string name is provided as a convenience to integrators.
shell	string	Read-only	Name of the shell binary used to launch batch netlisting commands.
netlistProbes	string	Read-only	Name of the file in the netlistDir which contains probes to be displayed on the schematic.
structuralNetlist	string	Read/Write	Name of the output file in the netlistDir in which to store the structural netlist.
timestamps	[map]	Read/Write	List of files, along with timestamps of when the files were last generated by the system. The purpose of the list is to detect when user changes have been made to the files and avoid overwriting these edits.

Table 227 *Attributes (Continued)*

Name	Type	Access	Description
inputFiles	[string]	Read/Write	The list of input files for the final netlist. This attribute would be usually set to the files returned by xt::getNetlistInputFiles.
logFile	string	Read/Write	Name of the file which contains stdout and stderr from the simulation process.
simulationProbes	string	Read-only	Name of the file, relative to the outputDir, which contains probes from the simulator to be displayed on the schematic.
languages	[string]	Read/Write	Names of the valid netlist languages for this simulator. This must be set by the integrator.
getInitFilesProc	string	Read/Write	The name of the Tcl procedure called during init to determine which files (with relative paths) should be copied to the run directory.
initRunDirectoryProc	string	Read/Write	The name of the Tcl procedure called at the end of the run directory initialization.
generateSimInputProc	string	Read/Write	The name of the Tcl procedure called before name mapping of the input file to the simulator namespace. This is where the integrator would mark up the files to be mapped.
getLaunchCommandProc	string	Read/Write	The name of the Tcl procedure called before the simulator is invoked. It is used by the default launchSimulatorProc function. This should be defined, unless the integrator is supplying their own launchProc. The command must follow standard Bourne-style shell syntax.

Table 227 *Attributes (Continued)*

Name	Type	Access	Description
launchSimulatorProc	string	Read/Write	The name of the Tcl procedure called to invoke the simulator. If the integrator is not using the default, they must use Tool Integration commands to create an xtJob object. If they are using the default callback for this proc, the getLaunchCommandProc should be defined.
initSimulatorProc	string	Read/Write	The name of the Tcl procedure called directly before the invocation of the simulator. This is an opportunity for the integrator to reset the state.
generateSimOutputProc	string	Read/Write	The name of the Tcl procedure called after the simulation finishes to perform markup of output files for name mapping back to the schematic namespace. This is also an opportunity to generate a errorOutput.tcl file.
getNamemapFilesProc	string	Read/Write	The name of the Tcl procedure that returns a list of simulator output files that should be name mapped. The path names returned should be absolute.
getReadableFilesProc	string	Read/Write	The name of the Tcl procedure that returns a list of files containing human readable content. These files will be made available for viewing in a text editor. The path names returned should be absolute.
getAvailableAnalysesProc	string	Read/Write	The name of the Tcl procedure that returns a list of user-recognizable analyses which are available for signal plotting based on the simulation results.

Table 227 Attributes (Continued)

Name	Type	Access	Description
getViewerAnalysisIdProc	string	Read/Write	The name of the Tcl procedure called before calling the viewer getPlotScriptProc. This allows the integrator to rename the plain language analysis names to something more meaningful, technically.
setPreferencesProc	string	Read/Write	Called at the end of the run directory initialization and before netlisting or simulation. The integrator can choose to override or update any preferences or attributes with integrator-specific preference values at this time.
runNetlisterProc	string	Read/Write	The name of the Tcl procedure used to netlist.
runSimulatorProc	string	Read/Write	The name of the Tcl procedure called to run the simulation.
initMenuBindingsProc	string	Read/Write	The name of the Tcl procedure to call when initializing a run directory (either new or load), to set up simulator-specific menus and keybindings.
undoMenuBindingsProc	string	Read/Write	The name of the Tcl procedure to call before a simulation session is removed, or when a session has a different run directory initialized.

Preferences

Table 228 lists Custom Compiler xt namespace general preferences. Preferences use camelBack notation, where the first character is always a lowercase character.

Table 228 *Integrated Tool Preferences*

Name	Type	Scope	Default	Description
xtCalibreCycleColor	bool	custom	true	Cycles calibre highlight colors while highlighting. Valid values: true, false
xtCalibreDefaultServerPort	int	custom	9189	Default port location of the Calibre Communication Server.
xtCalibreExternalTriggers	string	custom		File locations for external pre-trigger definitions.
xtCalibreHighlightPackage	string	custom	pveHighlight1	Calibre highlight color package.
xtJobMonitorStatusPollInterval	int	int	0	Interval (in milliseconds) for updating the job status
xtCalibreLayerMapFile	string	custom		Layer Mapping file name
xtCalibreLayoutExportColorLock	string	custom	Honor	Correspond to the color lock field in export stream dialog
xtCalibreLayoutExportColorLockEnabled	bool	custom	false	Enable to change the color lock value Valid values: true, false
xtCalibreLayoutExportFormat	string	custom	GDSII	The export format (GDSII/OASIS) of calibre layout export
xtCalibreLayoutExportLayerMapFile	string	custom		The layer map file of calibre layout export
xtCalibreLayoutExportLayerMapFileEnabled	bool	custom	false	Enable to change the layer map file of calibre layout export Valid values: true, false
xtCalibreLayoutExportLCVName	string	custom		The library/cell/view name of calibre layout export.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtCalibreLayoutExportObjectMapFile	string	custom		The object map file of calibre layout export
xtCalibreLayoutExportObjectMapFileEnabled	bool	custom	false	Enable to change the object map file of calibre layout export Valid values: true, false
xtCalibreLayoutExportRunDir	string	custom	.	The run directory of calibre layout export
xtCalibreLayoutExportRunDirEnabled	bool	custom	false	Enable to change the run directory of calibre layout export Valid values: true, false
xtCalibreLayoutExportTclFile	string	custom		Correspond to the tcl file field in export stream dialog
xtCalibreLayoutExportTclFileEnabled	bool	custom	false	Enable to change the pre-process tcl file value Valid values: true, false
xtCalibreLayoutExportTclFunction	string	custom		Correspond to the tcl function field in export stream dialog
xtCalibreLayoutExportTclFunctionEnabled	bool	custom	false	Enable to change the pre-process tcl function value Valid values: true, false
xtCalibreLayoutExportViewName	string	custom		The view name of calibre layout export
xtCalibreLayoutExportViewNameEnabled	bool	custom	false	Enable to change the view name of calibre layout export Valid values: true, false
xtCalibreLayoutHighlightPackage	string	custom	pveHighlight1	Calibre highlight color package for layout.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtCalibreLoadRunset	bool	custom	true	Loads the latest runset while launching Calibre Interactive.
xtCalibreLVSNetlister	string	custom	CDL	Netlister to use when exporting netlists from Custom Compiler tools.
xtCalibreNetlistExportLCVName	string	custom		The library/cell/view name of calibre netlist export.
xtCalibreNetlistExportNetlister	string	custom	CDL	Netlister
xtCalibreNetlistExportNetlisterEnabled	bool	custom	false	Enable to change the netlister Valid values: true, false
xtCalibreNetlistExportStopList	string	custom		The stop list controls when expansion stops during hierarchy traversal for netlist export
xtCalibreNetlistExportStopListEnabled	bool	custom	false	Enable to change the stop list Valid values: true, false
xtCalibreNetlistExportViewList	string	custom		The view list controls hierarchy traversal during netlist export
xtCalibreNetlistExportViewListEnabled	bool	custom	false	Enable to change the view list Valid values: true, false
xtCalibreNetlistExportViewName	string	custom		The view name of calibre layout export
xtCalibreNetlistExportViewNameEnabled	bool	custom	false	Enable to change the view name of calibre layout export Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtCalibreNetlistInfoMap	string	custom	auCdl SPICE hspiceD SPICE hspiceLVS SPICE	List of mappings for netlist info to netlist formats for Calibre.
xtCalibrePreCmdStr	string	custom	exec.calibre	Driver command.
xtCalibreOAViewOpenExportNetlist	bool	custom	false	Calibre view open Export Netlist dialog.
xtCalibrePVEViolationUseMarkers	bool	custom	true	Use oaMarkers for violations. Valid values: true, false
xtCalibreRVEIgnoreSchPrompt	bool	custom	false	Ignore schematic prompt when schematic LCV is not found.
xtCalibreRVELogLimit	int	custom	0	Limit for number of standalone RVE Log Files
xtCalibreRVEMaxQueryResultFiles	int	custom	-1	Limit for number of RVE query result files
xtCalibreRVENavigateMissingDevice	string	custom	Lowest	Navigate device when the highlighted device is missing
xtCalibreRVEOptions	string	custom		Command line options for 'Calibre-RVE'.
xtCalibreRVERaiseOnHighlight	bool	custom	false	Raise CC layout window while RVE highlighting
xtCalibreSaveRunsetOnRun	bool	custom	false	Save runset each time Calibre is run
xtCalibreSchematicHighlightPackage	string	custom	pveHighlight1	Calibre highlight color package for schematic.
xtCalibreShowLayoutExportDialog	bool	custom	false	Show calibre layout export dialog before export Valid values: true, false

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtCalibreShowNetlistExportDialog	bool	custom	false	Show calibre netlist export dialog before export Valid values: true, false
xtCalibreSkipErrorCount	int	custom	100	Number of errors ignored before PVE will stop processing a query_results file.
xtCDLGlobalSearchList	string	custom	auCdl schematic	Global search list passed to the CDL netlister
xtCDLGlobalStopList	string	custom	auCdl	Global stop list passed to the CDL netlister
xtCDLViewSearchList	string	custom	auCdl schematic symbol	Default view search list used when exporting CDL netlists from Custom Compiler. View Search List passed to the CDL Netlister during Schematic Netlisting.
xtCDLViewStopList	string	custom	auCdl	Default view stop list used when exporting CDL netlists from Custom Compiler. View Stop List passed to the CDL Netlister during Schematic Netlisting.
xtCheckOpenDesigns	enChcekOpenDesignsMode	user	all	Controls the set of designs which must be checked and saved before netlisting.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtCheckRoutesCheckAntenna	bool	user	true	Determines whether design should be checked for charge-collecting antenna rule violations. Valid values: true, false
xtCheckRoutesCheckDRC	bool	user	true	Determines whether design should be checked for design rule violations. Valid values: true, false
xtCheckRoutesCheckFShapes	bool	user	true	Determines whether design should be checked for DRC violations for frozen shapes, as well as routing shapes. Valid values: true, false
xtCheckRoutesCheckOpens	bool	user	true	Determines whether design should be checked for open nets. Valid values: true, false
xtCheckRoutesCheckUShapes	bool	user	true	Determines whether design should be checked for DRC violations for user-created shapes, as well as routing shapes. Valid values: true, false
xtCheckRoutesCheckVArea	bool	user	true	Determines whether design should be checked for voltage-area rule violations. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtCheckRoutesErrorSource	string	user	current	Specifies whether one of the error views or the current view should be used to import violations from. Value is either current or error.
xtCheckRoutesErrorView	string	user		Specifies the cell name whose error view is used to import violations.
xtCheckRoutesReportOpens	bool	user	true	Determines whether all open net should be reported. If false maximum of 200 open nets will be reported. Valid values: true, false
xtCheckRoutesRun	bool	user	true	Defines whether verify_zrt_route should be ran before violations import. Valid values: true, false
xtCrossHighlightShadow	bool	custom	false	Enables shadow mode for cross highlight from RVE or VUE.
xtCrossProbeLayoutZoomFactor	string	user	0.5	Zooming factor relative to selected elements size applied on layout elements when probing single or multiple nets from the PVE Dialogs.
xtCrossProbeMaxAmount	int	custom	10000	Max amount of parasitics cross probe objects.
xtCrossProbeOptionToolbarBus	string	context	all	Bus option of parasitic cross probe.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtCrossProbeOptionToolbarType	string	context	auto	Type option of parasitic cross probe.
xtCrossProbeSchematicZoomFactor	string	user	0.5	Zooming factor relative to selected elements size applied on schematic elements when probing single or multiple nets from the PVE Dialogs.
xtDataPrepCellType	enCellType	user		Specifies cell type.
xtDataPrepCreateUnmappedLPPs	bool	user	false	Controls whether to create unmapped LPPs.
xtDataPrepExtractAntenna	bool	user	false	Controls whether to extract antenna properties.
xtDefaultSimulator	string	user	HSPICE	Name of the default simulator to choose when creating a run directory.
xtDesignInsightsHostname	string	user	localhost	Remote Hostname
xtDesignInsightsJobLimit	int	user	3	Job limit
xtDesignInsightsJobLimitEnabled	bool	user	true	Job limit disabled (limited)
xtDesignInsightsJobQueueEngine	string	user		Queue Engine
xtDesignInsightsLSFEnableJobArrays	bool	user	true	Enable job array support.
xtDesignInsightsLSFExtraOptions	string	user		Extra command line options for LSF job launch
xtDesignInsightsLSFMaxArraySize	int	user	1000	Max job array size
xtDesignInsightsLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDesignInsightsLSFQueues	string	user		LSF Queues

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtDesignInsightsLSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtDesignInsightsLSFResources	string	user		LSF Resource types
xtDesignInsightsLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtDesignInsightsLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtDesignInsightsLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtDesignInsightsNetworkComputerEnableJob Arrays	bool	user	true	Enable job array support.
xtDesignInsightsNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtDesignInsightsNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtDesignInsightsNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtDesignInsightsNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDesignInsightsNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtDesignInsightsNetworkComputerSelectedJobClass	string	user	<None>	NetworkComputer selected job class
xtDesignInsightsNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtDesignInsightsNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDesignInsightsNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtDesignInsightsPrefixes	string	user		Job class prefixes
xtDesignInsightsPrependCommand	string	user		String to prepend to job shell command line
xtDesignInsightsSGEEnableJobArrays	bool	user	true	Enable job array support
xtDesignInsightsSGEExportEnvironment	bool	user	true	Export qsub environment to child job
xtDesignInsightsSGEExtraOptions	string	user		Extra command line options for SGE job launch
xtDesignInsightsSGEMaxArraySize	int	user	1000	Max job array size
xtDesignInsightsSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDesignInsightsSGEProjects	string	user	<None>	SGE projects.
xtDesignInsightsSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtDesignInsightsSGESelectedProject	string	user		Selected SGE project
xtDesignInsightsSGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtDesignInsightsSGESubmitHost	string	user	localhost	Submit host name for remote grid launch
xtDesignInsightsSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtDesignInsightsSLURMEnableJobArrays	bool	user	true	Enable job array support

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDesignInsightsSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtDesignInsightsSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtDesignInsightsSLURMMaxArraySize	int	user	1000	Max job array size
xtDesignInsightsSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDesignInsightsSLURMPartitions	string	user	<None>	SLURM partitions.
xtDesignInsightsSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtDesignInsightsSLURMSelectedPartition	string	user		Selected SLURM partition
xtDesignInsightsSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtDesignInsightsSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtDesignInsightsSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtDFMAutoPrefs	bool	custom	true	Auto-load prefs.xml Valid values: true, false
xtDFMCalibreEchoSVRF	bool	custom	false	Echo compiled SVRF commands and included files in the Calibre DFM job output. Valid values: true, false
xtDFMCalibreHierarchicalRunMode	bool	custom	true	Enables 'DFM' hierarchical mode of operation. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDFMCalibreOptions	string	custom		Command line options for 'Calibre' 'DFM' jobs.
xtDFMCalibreOverrideRunsetFile	bool	custom	false	Override SVRF commands in Calibre DFM primary runset file. Valid values: true, false
xtDFMCalibreResultsDatabase	string	custom		Location of the DFM results database.
xtDFMCalibreRunsetFile	string	custom		Calibre DFM primary runset file.
xtDFMCalibreRunsetOptions	string	custom		List of DFM {option value} options for 'Calibre' Interactive.
xtDFMCalibreSelectWindow	string	custom		Window to limit DRC processing. The value defines x and y coordinates of opposite corners of a window.
xtDFMCalibreSelectWindowType	string	custom	Design	Window type to limit DRC processing.
xtDFMCalibreSummaryReport	string	custom		Location of the DFM summary report.
xtDFMCalibreTranscript	string	custom	stdout.dfm.log	Output file name for the Calibre DFM Transcript.
xtDFMExportOasis	bool	custom	true	Export OA layout to Oasis Valid values: true, false
xtDFMExportStream	bool	custom	true	Export OA layout to stream Valid values: true, false
xtDFMHostname	string	user	localhost	Remote Hostname
xtDFMJobLimit	int	user	3	Job limit

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDFMJobLimitEnabled	bool	user	true	Job limit disabled (limited) Valid values: true, false
xtDFMJobQueueEngine	string	user		Queue Engine
xtDFMLaunchDebugger	bool	custom	true	Launch Debugger Valid values: true, false
xtDFMLayDBFormat	string	custom	OpenAccess	Database format of the layout
xtDFMLayLCV	lcv	custom	<file path>	Layout LCV
xtDFMLayOasisFile	string	custom		Layout Oasis file
xtDFMLayStreamFile	string	custom		Layout stream file
xtDFMLSFEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtDFMLSFExtraOptions	string	user		Extra command line options for LSF job launch
xtDFMLSFMaxArraySize	int	user	1000	Max job array size
xtDFMLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDFMLSFQueues	string	user		LSF Queues
xtDFMLSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtDFMLSFResources	string	user		LSF Resource types
xtDFMLSFSSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtDFMLSFSSubmitHost	string	user	localhost	Submit host name for remote grid launch

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDFMLSFuseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtDFMNetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtDFMNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtDFMNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtDFMNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtDFMNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDFMNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtDFMNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class
xtDFMNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtDFMNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtDFMNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtDFMPrefixes	string	user		Job class prefixes

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDFMPrependCommand	string	user		String to prepend to job shell command line
xtDFMRunDir	string	custom		Run directory
xtDFMRunDirNameTemplate	string	custom	%c.%t.%j	Template for customizing run directory name
xtDFMSGEEEnableJobArrays	bool	user	true	Enable job array support Valid values: true, false
xtDFMSGEEExportEnvironment	bool	user	true	Export qsub environment to child job Valid values: true, false
xtDFMSGEEExtraOptions	string	user		Extra command line options for SGE job launch
xtDFMSGEMaxArraySize	int	user	1000	Max job array size
xtDFMSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDFMSGEGProjects	string	user		SGE projects.
xtDFMSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtDFMSGESelectedProject	string	user		Selected SGE project
xtDFMSGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtDFMSGESubmitHost	string	user	localhost	Submit host name for remote grid launch

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDFMSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtDFMTool	string	custom	Calibre	Tool
xtDigitalCongestionEffort	string	user	high	Determines Congestions effort.
xtDigitalCongestionPlacement	bool	user	false	Enables Congestion Driven placement. Valid values: true, false
xtDigitalTool	digitalTool	user	icc2	Specifies the digital tool to use in co-design flows: IC Compiler (icc), IC Compiler II (icc2), Fusion Compiler (fc), or 3DIC Compiler (3dic). The preference configures the command line and start either icc2_shell or fc_shell. Valid values: icc2, fc, 3dic
xtDigitalPlacementEffort	string	user	high	Determines Placement effort.
xtDigitalRoutingOpt	bool	user	false	Runs Optimal Routing. Valid values: true, false
xtDigitalRunCTS	bool	user	true	Determines whether CTS should be run in ICC2. Valid values: true, false
xtDigitalRunCTSSStageFrom	string	user		Determines until which stage continue CTS validation.

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtDigitalRunCTSSStageTo	string	user		Determines from which stage start CTS validation.
xtDigitalRunPlacementFrom	string	user		Determines until which stage continue placement optimization.
xtDigitalRunPlacementTo	string	user		Determines from which stage start placement optimization.
xtDigitalScenariosList	string	user		List of Scenarios.
xtDigitalScenarioSource	string	user	File	Scenarios Selection type for creating Digital design.
xtDigitalTimingPlacement	bool	user	false	Enable Timing Driven placement. Valid values: true, false
xtDigitalTimingSetup	bool	user	false	Determines whether Timing Setup should be run in ICC2. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDigitalToolLogFile	string	user		Specifies the digital tool log file format: %library.%cell.%view.%job.%tool.%user.log. For example: <pre>db::setPrefValue xtDigitalToolLogFile -value %library.%cell.%tool.%user.log</pre> Example: If the cell name is <code>top</code> and the command is Fusion Compiler Front End Flow, then <pre>db::setPrefValue xtDigitalToolLogFile -value %cell.%tool.%job.log</pre> results in <pre>top.fc.frontend flow.log.</pre>
xtDRCAutoPrefs	bool	custom	true	Determines if the <code>prefs.xml</code> file is automatically saved and loaded for a DRC run. Valid values: true, false
xtDRCCalibreControlVars	string	custom		Calibre Control variables
xtDRCCalibreControlVariablesTableConfig	string	custom		Column size/visibility of Control Variable table
xtDRCCalibreEchoSVRF	bool	custom	false	Echo compiled SVRF commands and included files in the Calibre DRC job output. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDRCCalibreHierarchicalRunMode	bool	custom	true	Enables 'DRC' hierarchical mode of operation. Valid values: true, false
xtDRCCalibreLoadRunset	bool	custom	false	Controls parsing of the 'title' 'DRC' runset file for variables and options to populate the setup dialog. Valid values: true, false
xtDRCCalibreMultipleJobEnabled	bool	custom	true	Enable DRC job with calibre tool in running multiple jobs. Valid values: true, false
xtDRCCalibreOptions	string	custom		Command line options for 'Calibre' 'DRC' jobs.
xtDRCCalibreOutputFiles	string	custom		Output files of IC Validator DRC or LVS job
xtDRCCalibreOutputRunset	string	custom		Calibre DRC output runset control file.
xtDRCCalibreOverrideRunsetFile	bool	custom	false	Override SVRF commands in Calibre DRC primary runset file. Valid values: true, false
xtDRCCalibreResultsDatabase	string	custom		Location of the DRC results database.
xtDRCCalibreRunSummary	string	custom		Run Summary of ICV DRC/LVS job
xtDRCCalibreRunsetFile	string	custom		Calibre DRC primary runset file.
xtDRCCalibreRunsetOptions	string	custom		List of DRC {option value} options for 'Calibre' Interactive.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDRCCalibreSelectWindow	string	custom		Window to limit DRC processing. The value defines x and y coordinates of opposite corners of a window.
xtDRCCalibreSelectWindowType	string	custom	Design	Window type to limit DRC processing.
xtDRCCalibreShowRunSummary	bool	custom	false	Enable Run Summary of Calibre DRC/LVS job
xtDRCCalibreSummaryReport	string	custom		Location of the DRC summary report.
xtDRCCalibreTranscript	string	custom	stdout.drc.log	Output file name for the Calibre DRC Transcript.
xtDRCDefaultToolMultipleJobEnabled	string	user	true	Specifies if job is enabled by default in the Run Multiple dialog.
xtDRCEExportOasis	bool	custom	true	Export OA layout to Oasis Valid values: true, false
xtDRCEExportStream	bool	rundir	true	Export OpenAccess database to GDSII format. Valid values: true, false
xtDRCHerculesDebugger	string	rundir	vue	Name of the DRC debugger to use with the Hercules PV tool
xtDRCHerculesLoadRunset	bool	custom	true	Controls parsing of the 'title' 'DRC' runset file for variables and options to populate the setup dialog. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDRCHerculesMultipleJobEnabled	bool	custom	true	Enable DRC job with hercule tool in running multiple jobs. Valid values: true, false
xtDRCHerculesOptions	string	rundir	vue	Command line options specific to the Hercules verification tool to perform DRC checking
xtDRCHerculesRunsetFile	string	rundir		A Runset file for a Hercules DRC run.
xtDRCHerculesRunsetOptions	string	rundir	vue	List of user specified options to include in the Hercules runset.
xtDRCHerculesSelectWindow	string	rundir	vue	Pair of lower-left and upper-right window coordinates for the Hercules SELECT_WINDOW command.
xtDRCHerculesSelectWindowType	string	custom	Design	Window type to limit DRC processing.
xtDRCHerculesVariables	string	rundir	[]	List of DRC variable name/value pairs in the 'Hercules' runset file.
xtDRCHostList	string	rundir	local	List of machine names for selecting DRC run host.
xtDRCHostname	string	user	localhost	Remote Hostname
xtDRCICVCLFFile	string	custom		clf file for ICV
xtDRCICVControlVariablesTableConfig	string	custom		Column size or visibility of Control Variable table
xtDRCICVDebugger	string	rundir	icv_vue	Name of the DRC debugger to use with the ICV PV tool.

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtDRCICVEnvVars	string	custom	<None>	ICV environment variables
xtDRCICVExplorer	bool	custom	false	Enables ICV DRC explorer standalone mode. Valid values: true, false
xtDRCICVHeatMap	bool	custom	false	Enables ICV DRC error heat map. Valid values: true, false
xtDRCICVlgraph	bool	custom	false	Enable ICV DRC lgraph mode
xtDRCICVIncludePaths	string	rundir	icv_vue	List of include paths/directories for locating included DRC runset files for ICV.
xtDRCICVLoadRunset	bool	custom	false	Controls parsing of the 'title' 'DRC' runset file for variables and options to populate the setup dialog. Valid values: true, false
xtDRCICVMacros	string	custom	<None>	ICV macros
xtDRCICVMultiCore	string	custom	none	The number of CPUs to run on local host
xtDRCICVMultipleJobEnabled	bool	custom	true	Enable DRC job with icv tool in running multiple jobs. Valid values: true, false
xtDRCICVNvN	bool	custom	false	Use netlist versus netlist for ICV Valid values: true, false
xtDRCICVObjectMapFile	string	custom	custom	Object Mapping file name

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtDRCICVOptions	string	rundir	-oa_dm5	Command-line options specific to the ICV verification tool to perform DRC checking.
xtDRCICVOutputFiles	string	custom		Output files of ICV DRC/LVS job
xtDRCICVRunsetConfigFile	string	custom		ICV DRC runset config file.
xtDRCICVRunsetFile	string	rundir		Runset file for an ICV DRC run.
xtDRCICVRunsetOptions	string	custom		List of DRC {option value} options to be added to the 'IC Validator' runset.
xtDRCICVRunSummary	string	custom		Run Summary of ICV DRC/LVS job
xtDRCICVSelectWindow	string	custom		Window to limit DRC processing. The value defines x and y coordinates of opposite corners of a window.
xtDRCICVSelectWindowType	string	custom	Design	Window type to limit DRC processing.
xtDRCICVShowRunsetOptions	bool	custom	false	Show Runset Options table in Custom Options tab for ICV
xtDRCICVShowRunSummary	bool	custom	false	Enable Run Summary for ICV DRC/LVS job
xtDRCICVSmallCell	bool	custom	false	Use the small cell options for ICV. Valid values: true, false
xtDRCJobClass	string	custom	DRC	Job class
xtDRCJobLimit	int	user	3	Job limit. Valid values: Any integer in the [1 1000000] range.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDRCJobLimitEnabled	bool	user	true	Job limit disabled (limited) Valid values: true, false
xtDRCJobQueueEngine	string	user		Queue Engine
xtDRCLaunchDebug	bool	rundir	true	Launch debugger after job completion.
xtDRCLayCell	string	custom		Layout cell.
xtDRCLayDBFormat	string	rundir	open Access	Database format of the layout, either openAccess or stream.
xtDRCLayLCV	lcv	rundir	openAccess	LCV for the layout cellview to use for the DRC run.
xtDRCLayLCV2	lcv	custom	<file path>	Layout2 LCV
xtDRCLayOasisFile	string	custom		Layout Oasis file
xtDRCLayStreamFile	string	rundir		Name of the input layout stream file.
xtDRCLoadRunset	bool	rundir	true	Parses DRC runset file for variables and options to populate the setup dialog. Enables reload runset button.
xtDRCLSFEEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtDRCLSFEExtraOptions	string	user		Extra command line options for LSF job launch
xtDRCLSFMMaxArraySize	int	user	1000	Max job array size
xtDRCLSFNNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDRCLSFFQueues	string	user		LSF Queues

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDRCLSFRemoteSubmit	bool	user	false	Specifies if the submit host is on a remote site or not. Valid values: true, false
xtDRCLSResources	string	user		LSF Resource types
xtDRCLSFSSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtDRCLSFSSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtDRCLSUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtDRCNetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtDRCNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtDRCNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtDRCNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtDRCNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDRCNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host is on a remote site or not. Valid values: true, false
xtDRCNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class
xtDRCNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDRCNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtDRCNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtDRCPrefixes	string	user		Job class prefixes
xtDRCPrependCommand	string	user		String to prepend to job shell command line
xtDRCPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtDRCRecipeChoice	string	custom	<None>	Selected recipe in the recipe file
xtDRCRecipeFile	string	custom	<None>	Recipe file for runset
xtDRCRecipeFileNameMap	string	custom	<None>	Build a map to get full path file by file basename
xtDRCRecipeRuleFilter	string	custom	<None>	Select the specific rules by this filter
xtDRCRecipeRunHistory	string	custom	<None>	Represent recipeFile:recipe history list
XTfxtDRCReloadRecipeFiles	string	custom	<None>	Represent recipeFile history list
xtDRCRunDir	string	Cellview	true	Run directory.
xtDRCRunDirNameTemplate	string	user	%c.%t.%j	Run directory name template for DRC.
xtDRCRunHost	string	rundir	true	Name of the machine for the DRC job.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDRCSGEEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtDRCSGEEexportEnvironment	bool	user	true	Export qsub environment to child job Valid values: true, false
xtDRCSGEEExtraOptions	string	user		Extra command line options for SGE job launch
xtDRCSGEMaxArraySize	int	user	1000	Max job array size
xtDRCSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDRCSGEProjects	string	user		SGE projects.
xtDRCSGERemoteSubmit	bool	user	false	Specifies if the submit host is on a remote site or not. Valid values: true, false
xtDRCSGESelectedProject	string	user		Selected SGE project
xtDRCSGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtDRCSGESubmitHost	string	user	localhost	Submit host name for remote grid launch
xtDRCSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtDRCSLURMEnableJobArrays	bool	user	true	Enable job array support
xtDRCSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtDRCSLURMExtraOptions	string	user		Extra command line options for SLURM job launch

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtDRCSLURMMMaxArraySize	int	user	1000	Max job array size
xtDRCSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtDRCSLURMPartitions	string	user	<None>	SLURM partitions.
xtDRCSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtDRCSLURMSelectedPartition	string	user		Selected SLURM partition
xtDRCSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtDRCSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtDRCSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtDRCTool	string	rundir	IC Validator	The DRC verification tools.
xtDRCViewOutput	bool	rundir	true	Open text viewer to view DRC job output when launched. Valid values: true, false
xtEldoGlobalSearchList	string	custom	hspice hspiceD cmos.sch cmos_sch schematic veriloga	Global search list passed to the Eldo netlister.
xtEldoGlobalStopList	string	custom	hspice hspiceD veriloga	Global stop list passed to the Eldo netlister.
xtEnableCombinedLVSLPEFlow	bool	custom	true	Feature Enablement Valid values: true, false
xtExport3DICScript	string	user		Script file for 3DIC

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtExportFloorplanBoundaryType	bool	user	true	Specifies 'boundary' type which should be exported.
xtExportFloorplanCheckAndCreateNet	bool	user	true	Indicates whether need to export check and create net or not.
xtExportFloorplanExcludeShapesWithNoNet	bool	user	false	Exclude shapes with no net.
xtExportFloorplanExportShieldRoutes	bool	user	false	Indicates whether need to export shielded routes or not
xtExportFloorplanFigGroupType	bool	user	true	Specifies 'fig group' type which should be exported.
xtExportFloorplanHighlighChanges	bool	cellview	false	Highlights modified routes.
xtExportFloorplanMergeAutoGenOutputs	bool	user	false	Merge all auto generated TCL outputs in current session with the final output.
xtExportFloorplanOpenFile	bool	user	false	Open output file on completion
xtExportFloorplanOutputFilName	string	context		Specifies an output file name.
xtExportFloorplanOverrideRoutes	bool	user	false	Overrides existing routes.
xtExportFloorplanPinType	bool	user	true	Specifies 'pin' type which should be exported.
xtExportFloorplanRouteType	bool	user	true	Specifies 'route' type which should be exported.
xtExportFloorplanSelectionType	string	user	All	Specifies object selection type
xtExportFloorplanTextType	bool	user	true	Specifies 'text' type which should be exported.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtExportFloorplanTrackChanges	bool	cellview	false	Enables or disables net tracking.
xtExportFloorplanTrackedNetsGroup	string	context	username_nets_modified	Specifies name of net group which contains tracked nets.
xtExportScalarizeArrayInst	bool	user	false	Indicates whether Export to Digital scalarizes OA Array Instances in NDM database.
xtExportScalarizeArrayInstMasters	List of lcv	user		Specifies masters Library, Cell, Views of OA Array Instances to be scalarized during Export to Digital.
xtFCFrontEndFlowAnalyze	bool	user	true	Analyzes the RTL syntax in Verilog, SystemVerilog, or VHDL files. Valid values: true, false
xtFCFrontEndFlowCompileFusion	bool	user	true	Runs the Fusion Compiler <code>compile_fusion</code> command.
xtFCFrontEndFlowCompileFusionFromStage	string	user		Specifies the starting stage for the <code>compile_fusion</code> command.
xtFCFrontEndFlowCompileFusionFromStageSecondPhase	string	user		Specifies the starting stage of the second phase of the flow for the <code>compile_fusion</code> command.
xtFCFrontEndFlowCompileFusionToStage	string	user		Specifies the final stage for the <code>compile_fusion</code> command

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtFCFrontEndFlowCompileFusionToStageSecondPhase	string	user		Specifies the second phase final stage for the <code>compile_fusion</code> command
xtFCFrontEndFlowElaborate	bool	user	true	Builds a new design using an analyzed module, entity, or configuration template from an <code>hdl_library</code> . Valid values: true, false
xtFCFrontEndFlowElaborateDesignName	string	user		Determines block name for frontend flow.
xtFCFrontEndFlowGenerateOutputFiles	bool	user	false	Produces an ASCII file containing the Verilog , UPF, DEF, scripts/SDC, parasitics , and GDS data for the current block. Valid values: true, false
xtFCFrontEndFlowMCMMSetupFile	string	user		File that contains the list of corners for analysis.
xtFCFrontEndFlowParasiticTech	bool	user	true	Specifies TLUPlus files and layer map file (parasitic technology descriptions) used in the Fusion Compiler tool. Valid values: true, false
xtFCFrontEndFlowRRoutingDoubleVia	bool	user	false	Adds redundant vias. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtFCFrontEndFlowRRoutingHorizontalLayer	string	user		Specifies the horizontal layer to use when creating a power ground (PG) ring pattern. This preference is equivalent to <code>create_pg_ring_pattern -horizontal_layer</code> in the Fusion Compiler tool.
xtFCFrontEndFlowRRoutingMaxLayer	string	user		Marks all routing layers above the specified layer as fully blocked for electrical (RC) and congestion estimation. This preference is equivalent to <code>set_ignored_layers -max_routing_layer</code> in the Fusion Compiler tool.
xtFCFrontEndFlowRRoutingMinLayer	string	user		Marks all routing layers below the specified layer as fully blocked for electrical (RC) and congestion estimation. This preference is equivalent to <code>set_ignored_layers -min_routing_layer</code> in the Fusion Compiler tool.
xtFCFrontEndFlowRRoutingNetGroup	string	user		Specifies the group of nets to be routed.
xtFCFrontEndFlowRRoutingSelectNets	bool	user	false	Routes only the selected nets. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtFCFrontEndFlowRoutingOpt	bool	user	false	Optimizes the current routed design for timing, electrical DRC violations, area, and power. This preference is equivalent to the Fusion Compiler <code>route_opt</code> command. Valid values: true, false
xtFCFrontEndFlowRunFloorplan	bool	user	true	Runs automatic floorplanning (<code>compile.auto_floorplan.enable</code>). Valid values: true, false
xtFCFrontEndFlowRunRouteOpt	bool	user	false	Optimizes the current routed design for timing, electrical DRC violations, area, and power. This preference is equivalent to the Fusion Compiler <code>route_opt</code> command. Valid values: true, false
xtFCFrontEndFlowRunRouting	bool	user	true	Runs routing in the Fusion Compiler tool. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtFCFrontEndFlowScenarioSource	string	user	File	The source of the corner analysis scenarios: - List: Lets you provide the list of corner analysis scenarios. - File: Specifies a Tcl file that contains Fusion Compiler commands for corner analysis. Valid values: List, File
xtFCFrontEndFlowScenariosList	string	user		List of corner analysis scenarios.
xtFCFrontEndFlowSourceFiles	string	user		Determines source files for frontend flow.
xtFCFrontEndFlowSourceFormatType	string	user	Verilog	Determines frontend flow source format type.
xtFCFrontEndFlowTargetLib	string	user		Target Fusion Compiler library for the Fusion Compiler frontend flow.
xtFCFrontEndFlowTLUPlusLayerMapFile	string	user		TLUPlus layer map file for the Fusion Compiler frontend flow.
xtFCFrontEndFlowTLUPlusMaxFile	string	user		TLUPlus file that defines the maximum operating conditions for the Fusion Compiler frontend flow.
xtFCFrontEndFlowTLUPlusMinFile	string	user		TLUPlus file that defines the minimum operating conditions for the Fusion Compiler frontend flow.

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtFCFrontEndFlowTLUPlusNominalFile	string	user		TLUPlus file that defines the nominal operating conditions for the Fusion Compiler frontend flow.
xtFCFrontEndFlowType	bool	user	true	Determines if frontend flow type is Unified. Valid values: true, false
xtFCFrontEndFlowUpfFile	string	user		UPF file for the Fusion Compiler frontend flow.
xtFilterCommonParentForJobs	bool	user	true	Filter common parent of jobs when a new job is launched
xtFrontEndFlowFloorplanFile	string	user		Location and name of the floorplan Tcl file.
xtFrontEndFlowSearchPath	string	user		Names of directories containing design and library files that are specified without directory names.
xtFrontEndFlowSecondPhaseCompileFusion	bool	user	false	Specifies whether to run a second phase of <code>compile_fusion</code> . Valid values: true, false
xtFILLAutoPrefs	bool	custom	true	Auto-load prefs.xml
xtFILLExportOasis	bool	custom	true	Export OA layout to Oasis
xtFILLExportStream	bool	custom	true	Export OA layout to stream
xtFILLHostList	string	custom	localhost	Run hostname list
xtFILLHostname	string	user	localhost	Remote Hostname

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtFILLICVControlVars	string	custom		ICV preprocessor variables
xtFILLICVControlVariablesTableConfig	string	custom		Column size or visibility of Control Variable table
xtFILLICVEnvVars	string	custom	<None>	ICV environment variables
xtFILLICVIncludePaths	string	custom		List of FILL include directory paths used for locating 'IC Validator' runset include files.
xtFILLICVLoadRunset	bool	custom	false	Controls parsing of the 'title' 'FILL' runset file for variables and options to populate the setup dialog.
xtFILLICVMacros	string	custom	<None>	ICV macros
xtFILLICVMultipleJobEnabled	bool	custom	true	Enable FILL job with icv tool in running multiple jobs.
xtFILLICVNvN	bool	custom	false	Use netlist versus netlist for ICV
xtFILLICVObjectMapFile	string	custom		Object Mapping file name
xtFILLICVOptions	string	custom	-oa_dm5	Command line options for 'IC Validator' 'FILL' jobs.
xtFILLICVRunsetConfigFile	string	custom		IC Validator FILL runset config file.
xtFILLICVRunsetFile	string	custom		IC Validator FILL runset file.
xtFILLICVRunsetOptions	string	custom		List of FILL {option value} options to be added to the 'IC Validator' runset.

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtFILLICVSelectWindow	string	custom		Window to limit FILL processing. The value defines x and y coordinates of opposite corners of a window.
xtFILLICVSelectWindowType	string	custom	Design	Window type to limit FILL processing.
xtFILLICVShowRunsetOptions	bool	custom	false	Show Runset Options table in Custom Options tab for ICV
xtFILLICVSmallCell	bool	custom	false	Use the small cell options for ICV
xtFILLJobClass	string	custom	FILL	Job class
xtFILLJobLimit	int	user	3	Job limit
xtFILLJobLimitEnabled	bool	user	true	Job limit disabled (limited)
xtFILLJobQueueEngine	string	user		Queue Engine
xtFILLLSFEnableJobArrays	bool	user	true	Enable job array support.
xtFILLLSFExtraOptions	string	user		Extra command line options for LSF job launch
xtFILLLSFMaxArraySize	int	user	1000	Max job array size
xtFILLLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtFILLLSFQueues	string	user		LSF Queues
xtFILLLSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtFILLLSFResources	string	user		LSF Resource types
xtFILLLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtFILLLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtFILLLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtFILLLaunchDebugger	bool	custom	false	Launch Debugger
xtFILLLayCell	string	custom		Layout cell
xtFILLLayDBFormat	string	custom	OpenAccess	Database format of the layout
xtFILLLayLCV	lcv	custom	<file path>	Layout LCV
xtFILLLayLCV2	lcv	custom	<file path>	Layout2 LCV
xtFILLLayOasisFile	string	custom		Layout Oasis file
xtFILLLayStreamFile	string	custom		Layout stream file
xtFILLNetworkComputerEnableJobArrays	bool	user	true	Enable job array support.
xtFILLNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtFILLNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtFILLNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtFILLNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtFILLNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtFILLNetworkComputerSelectedJobClass	string	user	<None>	NetworkComputer selected job class
xtFILLNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtFILLNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtFILLNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtFILLOutputDBFormat	string	custom	OpenAccess	Format
xtFILLOutputLCV	lcv	custom	<file path>	Output LCV
xtFILLOutputType	string	custom	Fill + Input Design	Output Type
xtFILLOverwriteFillCell	bool	custom	false	Overwrite Fill cell
xtFILLPrefixes	string	user		Job class prefixes
xtFILLPrependCommand	string	user		String to prepend to job shell command line
xtFILLPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtFILLRunDir	string	custom		Run directory
xtFILLRunDirNameTemplate	string	custom	%c.%t.%j	Template for customizing run directory name
xtFILLRunHost	string	custom		Run hostname
xtFILLSGEEEnableJobArrays	bool	user	true	Enable job array support
xtFILLSGEEExportEnvironment	bool	user	true	Export qsub environment to child job
xtFILLSGEEExtraOptions	string	user		Extra command line options for SGE job launch
xtFILLSGEMaxArraySize	int	user	1000	Max job array size
xtFILLSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtFILLSGEProjects	string	user	<None>	SGE projects.
xtFILLSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtFILLSGESelectedProject	string	user		Selected SGE project
xtFILLSGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtFILLSGESubmitHost	string	user	localhost	Submit host name for remote grid launch
xtFILLSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtFILLSLURMEnableJobArrays	bool	user	true	Enable job array support
xtFILLSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtFILLSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtFILLSLURMMaxArraySize	int	user	1000	Max job array size
xtFILLSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtFILLSLURMPartitions	string	user	<None>	SLURM partitions.
xtFILLSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtFILLSLURMSelectedPartition	string	user		Selected SLURM partition
xtFILLSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtFILLSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtFILLSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtFILLTool	string	custom	IC Validator	Tool
xtFILLViewFillCell	bool	custom	true	Open Fill View
xtFILLViewOutput	bool	custom	true	Open text viewer.
xtHerculesNetlistInfoMap	string	user	[[ams SPICE] [auCDL CDL] [hspiceD SPICE]]	List of mappings for netlist info to netlist formats for Hercules.
xtHerculesNettranOptions	string	custom		The desired value for NETTRAN_OPTIONS in the OPTIONS section of the 'Hercules' runset.
xtHerculesPreCmdStr	string	custom	exec-oa22.04.hercules	Driver command.
xtHerculesRunsetFileSuffix	string	rundir	.ev .vc	File extensions for the runset for the Hercules tool.
xtHerculesVUEOptions	string	custom		Command line options for 'Hercules-VUE'.
xtHostname	string	user	localhost	Specifies the remote hostname.
xtHostnameCompletions	string	user	localhost	Completions for hostname widget in job management options.
xtHSIMBinResultsPageSize	string	user	50	
xtHSIMDefaultNetlister	string	user	NanoSim	The name of the default netlister to choose when generating a netlist.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtHSIMDefaultViewer	string	user	WaveView	Default waveform viewer for plotting signals.
xtHSIMGlobalSearchList	string	custom	hspice hspiceD cmos.sch cmos_sch schematic veriloga	Global search list passed to the netlister.
xtHSIMGlobalStopList	string	custom	hspice hspiceD veriloga	Global stop list passed to the netlister.
xtHSIMHSPICEJobLimit	int	user	3	Job limit.
xtHSIMHSPICEJobLimitEnabled	bool	user	true	Job limit disabled (limited).
xtHSPICEDefaultNetlister	string	user	HSPICE	The default netlister to choose when generating a netlist.
xtHSPICEDefaultViewer	string	user	WaveView	Default waveform viewer for plotting HSPICE results.
xtHSPICEGlobalSearchList	string	custom	hspice hspiceD cmos.sch cmos_sch schematic veriloga	Global search list passed to the netlister.
xtHSPICEGlobalStopList	string	custom	hspice hspiceD veriloga	Global stop list passed to the netlister.
xtHSPICEJobLimit	int	user	3	Job limit.
xtHSPICEJobLimitEnabled	bool	user	true	Job limit disabled (limited).
xtHSPICEViewSearchList	string	rundir	schematic hspice symbol	View search list used by HSPICE Netlister when traversing schematic hierarchy.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlHSPICEViewStopList	string	custom	schematic hspice symbol	View stop list used by HSPICE Netlister when traversing schematic hierarchy.
xtlCC2BBCreateBlackBox	bool	user	true	Determines whether black box should be created in ICC2. Valid values: true, false
xtlCC2BBCreateFrame	bool	user	true	Determines whether frame should be created in ICC2 in case of black box flow. Valid values: true, false
xtlCC2BBShapeBlocks	bool	user	true	Determines whether shape blocks should be run in ICC2. Valid values: true, false
xtlCC2BlackBoxArea	float	user	0	Area of Target Boundary for black box creation.
xtlCC2BlackBoxBoundary	string	user		Boundary for black box creation.
xtlCC2BlackBoxFlow	bool	user	false	BlackBox flow. Valid values: true, false
xtlCC2BlackBoxPinConstraintNoStacking	string	user	any	Corresponds to ICC2 set_block_pin_constraints - no_stacking.
xtlCC2BlackBoxPinConstraintPinSpacing	int	user	1	Corresponds to set_block_pin_constraints - pin_spacing.
xtlCC2BlackBoxSizeMode	string	user	Boundary	The default mode for specifying black box estimation mode.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlICC2ConstraintMapFile	string	user		Map file for ICC2 constraints.
xtlICC2CopyCell	bool	user	false	Determines ICC2 design should be copied. Valid values: true, false
xtlICC2CreatePGRings	bool	user	true	Determines whether PG ring should be created in ICC2. Valid values: true, false
xtlICC2CreateScript	string	user		ICC2 Script for creating ICC2 design
xtlICC2DataPrepBlockAll	string	user	Auto	Controls whether the tool creates zero-spacing routing blockages in the frame view for the entire block, except for the area around the pins
xtlICC2DataPrepCellSource	string	user		Source Cell(s) for Create ICC2 Frame
xtlICC2DataPrepConnectWithinPin	bool	user	false	Controls whether the via enclosure must be within the pin shape. Valid values: true, false
xtlICC2DataPrepHierarchicalExtraction	bool	user	false	Controls whether to extract geometries in subblocks. Valid values: true, false
xtlICC2DataPrepLibSource	string	user		Source Lib for Create ICC2 Frame
xtlICC2DataPrepMaskColors	string	user		TCL list of pairs of layer_name and number of masks

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlCC2DataPrepMergeBlockages	bool	user	false	Controls whether to merge metal shapes and regular routing blockages to reduce the number of shapes. Valid values: true, false
xtlCC2DataPrepPreserveBlockages	string	user	Auto	Controls whether to preserve all metal shapes and routing blockages around pins as they exist in the design view.
xtlCC2DataPrepScript	string	user	<file path>	Create frame NDM script
xtlCC2DataPrepSiteDef	string	user		Site definition in Create ICC2 Frame
xtlCC2DataPrepTarget	string	user		Target ICC2 Library to export to
xtlCC2DataPrepTimingFiles	string	user		TCL list of timing files
xtlCC2DataPrepTopCellPin	bool	user	true	Controls whether the tool extracts hierarchical antenna properties only for nets that connect to pins in the top-level of the hard macro. . Valid values: true, false
xtlCC2DataPrepTreatSourceDrainAsDiodes	bool	user	true	Controls whether the tool treats the MOS source and drain as protection diodes and considers any MOS source and drain regions that are connected to a net to be a part of the diode protection area available to that net. Valid values: true, false

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtlCC2DataPrepViewSource	string	user		Source View for Create ICC2 Frame
xtlCC2FloorPlanControlType	string	user	core	Corresponds to initialize_floorplan -control_type.
xtlCC2FloorplanCoreBottom	float	user	1	Corresponds to initialize_floorplan -core_offset.
xtlCC2FloorplanCoreLeft	float	user	1	Corresponds to initialize_floorplan -core_offset.
xtlCC2FloorplanCoreRight	float	user	1	Corresponds to initialize_floorplan -core_offset.
xtlCC2FloorplanCoreTop	float	user	1	Corresponds to initialize_floorplan -core_offset.
xtlCC2FloorPlanShapeType	string	user	R	Corresponds to initialize_floorplan -shape.
xtlCC2FloorplanSideLength	string	user	0.0	Corresponds to initialize_floorplan -side_length.
xtlCC2FloorplanUtilization	float	user	0.7	Corresponds to initialize_floorplan -core_utilization.
xtlCC2GroundNetName	string	user		Ground net name for creating ICC2 design
xtlCC2GroundPortName	string	user		Ground port name for creating ICC2 design
xtlCC2IMetalFillerLibCellList	string	user		Corresponds to create_stdcell_filler -lib_cells.
xtlCC2IMetalFillerPrefix	string	user		Corresponds to create_stdcell_filler -prefix.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlCC2lNonMetalFillerLibCellList	string	user		Corresponds to create_stdcell_filler-lib_cells.
xtlCC2lNonMetalFillerPrefix	string	user		Corresponds to create_stdcell_filler-prefix.
xtlCC2lInsertFillerCells	bool	user	false	Determines whether filler cells should be inserted in ICC2. Valid values: true, false
xtlCC2lInsertTapCells	bool	user	false	Determines whether tap cells should be inserted in ICC2. Valid values: true, false
xtlCC2lRRoutingSearchRepairIterations	string	user		Corresponds to route_detail-max_number_iterations.
xtlCC2lTapCellDistance	float	user	1	Corresponds to create_tap_cells-distance.
xtlCC2lTapFilterCellList	string	user		Corresponds to create_tap_cells-no_tap_cell_under_layers.
xtlCC2lTapMasterCell	string	user		Corresponds to create_tap_cells-lib_cell.
xtlCC2leCreatePGRings	bool	user	true	Determines whether PG ring should be created in ICC2. Valid values: true, false
xtlCC2leInsertFillerCells	bool	user	false	Determines whether filler cells should be inserted in ICC2. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlCC2leInsertTapCells	bool	user	false	Determines whether tap cells should be inserted in ICC2. Valid values: true, false
xtlCC2leParasiticTech	bool	user	true	Determines parasitic tech descriptions in ICC2. Valid values: true, false
xtlCC2leRRoutingDoubleVia	bool	user	false	Insert Double vias. Valid values: true, false
xtlCC2leRRoutingSelectNets	bool	user	false	Route Selected Nets. Valid values: true, false
xtlCC2leRunDrcLvs	bool	user	true	Determines whether DRC LVS should be checked in ICC2. Valid values: true, false
xtlCC2leRunFloorplan	bool	user	true	Determines whether floorplan should be run in ICC2. Valid values: true, false
xtlCC2leRunPlacement	bool	user	true	Determines whether placement should be run in ICC2. Valid values: true, false
xtlCC2leRunRouting	bool	user	true	Determines whether routing should be run in ICC2. Valid values: true, false
xtlCC2LogicalSource	string	user	Verilog File	Logical source type for creating ICC2 design

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtlICC2LogicCellView	string	user	<file path>	CellView containing logical source for creating ICC2 design
xtlICC2NewCell	string	user	<file path>	ICC2 design to be created
xtlICC2PGHorizontalLayer	string	user	metal1	Corresponds to create_pg_ring_pattern -horizontal_layer.
xtlICC2PGHorizontalLayerSpacing	float	user	0	Corresponds to create_pg_ring_pattern -horizontal_spacing.
xtlICC2PGHorizontalLayerWidth	float	user	0	Corresponds to create_pg_ring_pattern -horizontal_width.
xtlICC2PGVerticalLayer	string	user	metal2	Corresponds to create_pg_ring_pattern -vertical_layer.
xtlICC2PGVerticalLayerSpacing	float	user	0	Corresponds to create_pg_ring_pattern -vertical_spacing.
xtlICC2PGVerticalLayerWidth	float	user	0	Corresponds to create_pg_ring_pattern -vertical_width.
xtlICC2PowerMode	string	user	Single Supply	Power mode for creating ICC2 design
xtlICC2PowerNetName	string	user		Power net name for creating ICC2 design
xtlICC2PowerPortName	string	user		Power port name for creating ICC2 design
xtlICC2RRoutingDoubleVia	bool	user	false	Insert Double vias. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlCC2RRoutingHorizontalLayer	string	user	metal1	Corresponds to create_pg_ring_pattern -horizontal_layer.
xtlCC2RRoutingMaxLayer	string	user		Corresponds to set_ignored_layers -max_routing_layer.
xtlCC2RRoutingMinLayer	string	user		Corresponds to set_ignored_layers -min_routing_layer.
xtlCC2RRoutingNetGroup	string	user		Nets to Route.
xtlCC2RRoutingSelectNets	bool	user	false	Route Selected Nets. Valid values: true, false
xtlCC2RunDrcLvs	bool	user	true	Determines whether DRC LVS should be checked in ICC2. Valid values: true, false
xtlCC2RunFloorplan	bool	user	true	Determines whether floorplan should be run in ICC2. Valid values: true, false
xtlCC2RunPlacement	bool	user	true	Determines whether placement should be run in ICC2. Valid values: true, false
xtlCC2RunRouting	bool	user	true	Determines whether routing should be run in ICC2. Valid values: true, false
xtlCC2RunScript	string	user		ICC2 TCL script.
xtlCC2UPFFile	string	user		UPF file for creating ICC2 design
xtlCC2VerilogFile	string	user		Verilog source file for creating ICC2 design

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlCC2VerilogTopModule	string	user		Top module within a Verilog source file for creating ICC2 design
xtlCC2VerilogViewSearchList	string	user	blackbox verilog schematic symbol	View search list for creating ICC2-compatible Verilog netlists
xtlCC2VerilogViewStopList	string	user		View stop list for creating ICC2-compatible Verilog netlists
xtlCCImportViewTypeAbstract	string	user	true	Switch of FRAM/abstract views for import from ICC
xtlCCImportViewTypeFill	string	user	true	Switch of FILL/fill views for import from ICC
xtlCCImportViewTypeILM	string	user	true	Switch of ILM views for import from ICC
xtlCCImportViewTypeLayout	string	user	true	Switch of CEL/layout views for import from ICC
xtlCCImportViewTypeSmash	string	user	true	Switch of SMASH/smash views for import from ICC
xtlCCLoadDisplayResources	bool	user	true	Load display resources of an ICC library before opening a design from that library. Valid values: true, false
xtlCVCopyRunset	bool	custom	false	Copy the runset file to run directory.
xtlCVDefaultServerPort	int	custom	9189	Default port location of the IC Validator Communication Server.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlCVLiveAreaChoice	string	user		The current area choice
xtlCVLiveLastCheckedArea	string	cellview		Store the current checking bounding box as the last checked area
xtlCVLiveBackground	bool	user	true	Runs ICV Live checking runs in the background.
xtlCVLiveCheckLayers	string	rundir	Visible layers only	Specifies the option to be applied. Valid values: All layers, Visible layers involved, Visible layers only.
xtlCVLiveDisableRelatedLayers	bool	user	false	Control the related layers display of violations
xtlCVLiveDumpCheckingRange	bool	user	false	Creates OA markers to indicate the checking range, with or without the halo range.
xtlCVLiveHaloRange	float	user	1.0	Defines the halo range using by ICV Live checking. By default, it is 1.0 which means we extend the checking range by 1 user unit to avoid boundary condition.
xtlCVLiveHistoryRecipe	string	user	""	Records the last used recipes whether single or multiple windows were used to run ICV Live.
xtlCVLiveICVObjectMapFile	string	rundir		Specifies the object mapping file.
xtlCVLiveLaunchDebugger	bool	custom	true	Launch Debugger Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlCVLiveNotifyWhenComplete	bool	user	false	Shows notification dialog when ICV Live checking completes
xtlCVLiveOnModification	bool	cellview	false	ICV Live check on design modification or not Valid values: true, false
xtlCVLivePolygonLicense	string	user	Formatted string	Specifies the option to be applied. The default is Always. Valid values: Always, Never, Ask.
xtlCVLiveRecipeChoice	string	rundir	<No-Recipe>	Specifies the recipe name to be used under the selected recipe file
xtlCVLiveRecipeDirectory	string	user	""	Specifies the path to the directory which lists all of the defined recipes.
xtlCVLiveRecipeFile	string	""	<No-Recipe>	Refers to the selected recipe file path containing multiple recipes
xtlCVLiveRecipeHistoryMaxCount	integer	user	0	Specifies the maximum number of history recipes in the xtlCVLiveHistoryRecipe.
xtlCVNetlistInfoMap	string	user	[[ams SPICE] [auCDL CDL] [hspiceD SPICE]]	List of mappings for netlist info to netlist formats for ICV.
xtlCVNettranOptions	string	custom		Command line options for running NetTran on the input schematic netlist prior to executing IC Validator.
xtlCVPreCmdStr	string	custom	exec-oa22.50.064 .icv	Driver command

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlCVRunsetFileSuffix	string	rundir	.rs	File extensions for the runset for the ICV tool.
xtlCVStreamNameMapFile	string	custom		Overwrites default location for GDS stream name map file.
xtlCVVUEOptions	string	custom		Command line options for 'IC Validator VUE'.
xtlImportErrorsAccessEdgesWholeSide	bool	user	true	Controls the access points when access edges are obeyed.
xtlImportErrorsAllowViaRotation	bool	user	true	Controls whether the tool can use rotated vias at pin connections.
xtlImportErrorsBlockedRange	int	user	10	Specifies the number of pitches for the same-layer accessibility check for macro and top-level ports.
xtlImportErrorsBlockedRangeViaSide	int	user	10	Specifies the number of pitches for the accessibility check on the other side of an access via.
xtlImportErrorsCheckAccessibility	bool	user	false	Controls whether need to check the accessibility for pins without an assigned net.
xtlImportErrorsCheckCongestionMapOverflow	int	user	0	Checks global route congestion map overflow larger than or equal to the given value.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlImportErrorsCheckMinGrid	bool	user	true	Controls whether need to check and report design objects that are not on the user (manufacturing) minimum grid.
xtlImportErrorsCheckOutOfBoundary	bool	user	true	Controls whether need to check and report the pins that are outside of the design boundary.
xtlImportErrorsCheckPadPins	string	user		Specifies how pad pins are checked for the specified nets.
xtlImportErrorsCheckShield	bool	user	true	Controls whether need to perform check on the routing shapes related to the shielding flow.
xtlImportErrorsCheckStdCellPins	string	user		Specifies how std cell pins are checked for the specified nets.
xtlImportErrorsCheckTopLevelBlockages	bool	user	true	Specifies whether to check shorts on top-level blockages.
xtlImportErrorsCheckViaLadder	bool	user	false	Controls whether need to perform the via ladder flow sanity checks.
xtlImportErrorsConsiderLayerConstraint	bool	user	true	Controls whether need to consider the layer constraints when checking the accessibility of ports and the connectivity of nets.
xtlImportErrorsDetectDuplicatedPGShapes	bool	user	true	Controls whether need to check redundant PG shapes that overlap at the same location.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlImportErrorsDetectIncorrectViaCutBlockage	bool	user	true	Controls whether need to check incorrect real metal via cut blockages in the design.
xtlImportErrorsDetectIncorrectViaCutBlockage Ref	bool	user	false	Controls whether need to check incorrect real metal via cut blockages in the reference libraries.
xtlImportErrorsDetectPGViaObstacles	bool	user	false	Controls whether need to check PG via obstacles in the space around each routing track by using the default or nondefault spacing requirement.
xtlImportErrorsDetectPins	bool	user	false	Controls whether need to check real metal routing blockages that touch or overlap a pin inside the reference libraries.
xtlImportErrorsErrorViewFileName	string	user		Specifies the error view name. If not specified, the default error view name will be <block_name>_floatingPG.err.
xtlImportErrorsFrozenNet	bool	user	false	Controls whether need to check blocked frozen net ports.
xtlImportErrorsMaxFloatingClusterSize	int	user	0	Specify the maximum size of floating cluster to be returned by the command and reported by the error browser.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlImportErrorsNonStdCell	bool	user	true	Controls whether need to check blocked macro, top-level, and non-standard-cell ports.
xtlImportErrorsObeyAccessEdges	bool	user	false	Controls whether pins can be connected only at their access edges.
xtlImportErrorsObeyDirectionPreference	bool	user	false	Controls whether only preferred direction access is allowed.
xtlImportErrorsPG	bool	user	false	Controls whether need to check blocked power and ground ports.
xtlImportErrorsReportNoAccessEdge	bool	user	false	Controls whether need to report the unconnected pins without predefined access edges.
xtlImportErrorsStdCell	bool	user	true	Controls whether need to check blocked standard cell ports.
xtlImportErrorsStdCellSearchRange	int	user	2	Specifies the number of pitches for the same-layer accessibility search of standard cell pins.
xtlImportErrorsStopOnError	bool	user	true	Stops processing when errors occur during digital tool execution with the source command.
xtlImportErrorsViaConnInsidePins	bool	user	false	Controls whether via connections to standard cell pins must be fully inside the pins.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlImportErrorsWriteCEFileName	string	user		Specifies the output file name. If specified, the tool writes detailed net connectivity information to the file.
xtlImportMilkywayOutputLibrary	string	user		Target library for Import from ICC
xtlIncrementalNetlisting	bool	user	false	Whether netlisting will be incremental. Valid values: true, false
xtlnDesignEMRCHostname	string	user	localhost	Remote Hostname
xtlnDesignEMRCJobClass	string	cellview	InDesignEMRC	Job class name for background jobs for in-design emrc flow
xtlnDesignEMRCJobLimit	int	user	3	Job limit
xtlnDesignEMRCJobLimitEnabled	bool	user	true	Job limit disabled (limited)
xtlnDesignEMRCJobQueueEngine	string	user		Queue Engine
xtlnDesignEMRCLSFEEnableJobArrays	bool	user	true	Enable job array support.
xtlnDesignEMRCLSFEExtraOptions	string	user		Extra command line options for LSF job launch
xtlnDesignEMRCLSFSMaxArraySize	int	user	1000	Max job array size
xtlnDesignEMRCLSFSNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtlnDesignEMRCLSFSQueues	string	user		LSF Queues
xtlnDesignEMRCLSFSRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtlnDesignEMRCLSFSResources	string	user		LSF Resource types
xtlnDesignEMRCLSFSSetupCommand	string	user		Shell command to run on remote host to configure LSF

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlnDesignEMRCLSFSSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtlnDesignEMRCLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtlnDesignEMRCNetworkComputerEnableJob Arrays	bool	user	true	Enable job array support.
xtlnDesignEMRCNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtlnDesignEMRCNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtlnDesignEMRCNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtlnDesignEMRCNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtlnDesignEMRCNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtlnDesignEMRCNetworkComputerSelectedJobClass	string	user	<None>	NetworkComputer selected job class
xtlnDesignEMRCNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtlnDesignEMRCNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtlnDesignEMRCNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtlnDesignEMRCPrefixes	string	user		Job class prefixes
xtlnDesignEMRCPrependCommand	string	user		String to prepend to job shell command line

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtlnDesignEMRCSGEEnableJobArrays	bool	user	true	Enable job array support
xtlnDesignEMRCSGEExportEnvironment	bool	user	true	Export qsub environment to child job
xtlnDesignEMRCSGEExtraOptions	string	user		Extra command line options for SGE job launch
xtlnDesignEMRCSGEMaxArraySize	int	user	1000	Max job array size
xtlnDesignEMRCSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtlnDesignEMRCSGEProjects	string	user	<None>	SGE projects.
xtlnDesignEMRCSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtlnDesignEMRCSGESelectedProject	string	user		Selected SGE project
xtlnDesignEMRCSGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtlnDesignEMRCSGESubmitHost	string	user	localhost	Submit host name for remote grid launch
xtlnDesignEMRCSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtlnDesignEMRCSLURMEnableJobArrays	bool	user	true	Enable job array support
xtlnDesignEMRCSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtlnDesignEMRCSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtlnDesignEMRCSLURMMMaxArraySize	int	user	1000	Max job array size
xtlnDesignEMRCSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtlnDesignEMRCSLURMPartitions	string	user	<None>	SLURM partitions.
xtlnDesignEMRCSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtlnDesignEMRCSLURMSelectedPartition	string	user		Selected SLURM partition
xtlnDesignEMRCSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtlnDesignEMRCSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtlnDesignEMRCSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtJobManagerSaveFilename	string	user	~/synopsys_custom_jobs.xml	The name of the file where the job manager contents are stored.
xtJobMonitorBatchJobs	bool	user	true	Toggles the display of batch jobs in the Job Monitor. Valid values: true, false
xtJobMonitorCPUUsageColumn	bool	user	false	Hide / Show Job Monitor CPU Usage Column. Valid values: true, false
xtJobMonitorCreationTimeColumn	bool	user	false	Hide / Show Job Monitor Creation Time Column. Valid values: true, false
xtJobMonitorDisplayOnStartup	bool	user	false	Raise the job monitor window automatically when a batch job is started. Valid values: true, false
xtJobMonitorEndTimeColumn	bool	user	false	Hide / Show Job Monitor End Time Column. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtJobMonitorHostColumn	bool	user	true	Hide / Show Job Monitor Host Column. Valid values: true, false
xtJobMonitorInteractiveJobs	bool	user	false	Toggles the display of interactive jobs in the Job Monitor. Valid values: true, false
xtJobMonitorInitSortingColumn	string	user	Creation Time	Initial sorting column applied to Job Monitor
xtJobMonitorJobColumn	bool	user	true	Hide / Show Job Monitor Name Column. Valid values: true, false
xtJobMonitorJobIDColumn	bool	user	true	Hide / Show Job Monitor Job ID Column at startup.
xtJobMonitorOutputFilesColumn	bool	user	false	Hide / Show Job Monitor Output Files Column. Valid values: true, false
xtJobMonitorPeakRSSColumn	bool	user	false	Hide / Show Job Monitor Peak RSS Column. Valid values: true, false
xtJobMonitorProcessIDColumn	bool	user	false	Hide / Show Job Monitor Process Id Column. Valid values: true, false
xtJobMonitorRunDescriptionColumn	bool	user	true	Hide / Show Job Monitor Description Column. Valid values: true, false
xtJobMonitorStartTimeColumn	bool	user	false	Hide / Show Job Monitor Start Time Column. Valid values: true, false
xtJobMonitorStatusColumn	bool	user	true	Hide / Show Job Monitor Status Column. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtJobMonitorTypeColumn	bool	user	false	Hide / Show Job Monitor Type Column. Valid values: true, false
xtJobQueueEngine	string	user		The default queue engine to use when launching batch jobs.
xtJobQueueUpdateInterval	int	user	15	The number of seconds to wait between calls to the integrator callback to query queue engines for job status.
xtJobViewOutputActiveWindowFileName	List of unknown	user		List of filenames to set the active window priority for job view output files action.
xtLPEAddExtractionToViewName	bool	custom	true	StarRC AddExtractionToViewName command
xtLPEAutoPrefs	bool	custom	true	Auto-load prefs.xml. Valid values: true, false
xtLPECcThresh	float	rundir	0	Threshold value below which to filter parasitic coupled capacitors from query results.
xtLPECgThresh	float	rundir	0	Threshold value below which to filter parasitic grounded capacitors from query results.
xtLPECheckPorts	bool	custom	false	Check ports
xtLPECheckPortsCDLViewSearchList	string	custom	auCdl schematic	Check ports CDL view search list
xtLPECheckPortsCDLViewStopList	string	custom	auCdl	Check ports CDL view stop list

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPECheckPortsCaseSensitive	bool	custom	false	Check ports with case-sensitive
xtLPECheckPortsHSPICEViewSearchList	string	custom	hspice hspiceD cmos.sch cmos_sch schematic veriloga	Check ports HSPICE view search list
xtLPECheckPortsHSPICEViewStopList	string	custom	hspice hspiceD veriloga	Check ports HSPICE view stop list
xtLPECheckPortsNetlistFiles	string	custom		Check ports netlist files. Internal used
xtLPEEnableLPEJob	bool	custom	true	LPE job enabled status in combined LVS-LPE dialog
xtLPEEnableLVSJob	bool	custom	false	LVS job enabled status in combined LVS-LPE dialog
xtLPEGPDBindLCV	string	cellview		Lib/Cell/View of the binded design
xtLPEGPDBindMaskView	bool	user	false	Enable GPD to bind with MaskLayout viewtype design
xtLPEGPDCommandQueryCc	bool	cellview	true	Option dialog controls for GPD query coupled capacitor for query parasitics. Valid values: true, false
xtLPEGPDCommandQueryCg	bool	cellview	true	Option dialog controls for GPD query grounded capacitor for query parasitics. Valid values: true, false
xtLPEGPDCommandQueryCp	bool	cellview	true	Option dialog controls for GPD query pin capacitor for query parasitics. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEGPDCommandQueryInstCheckAll	bool	cellview	true	Option dialog controls for GPD query all devices for query parasitics. Valid values: true, false
xtLPEGPDCommandQueryR	bool	cellview	true	Option dialog controls for GPD query resistors for query parasitics. Valid values: true, false
xtLPEGPDHeatMapCorner	string	cellview		GPD Heat Map Corner
xtLPEGPDHeatMapLevel	int	cellview	20	GPD Heat Map Color Level
xtLPEGPDHeatMapType	string	cellview	Path Resistance	GPD Heat Map Type
xtLPEGPDOptionQueryCc	bool	cellview	true	Option dialog controls for GPD query coupled capacitor for query parasitics. Valid values: true, false
xtLPEGPDOptionQueryCg	bool	cellview	true	Option dialog controls for GPD query grounded capacitor for query parasitics. Valid values: true, false
xtLPEGPDOptionQueryCp	bool	cellview	true	Option dialog controls for GPD query pin capacitor for query parasitics. Valid values: true, false
xtLPEGPDOptionQueryInstCheckAll	bool	cellview	true	Option dialog controls for GPD query all devices for query parasitics. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEGPDOptionQueryR	bool	cellview	true	Option dialog controls for GPD query resistors for query parasitics. Valid values: true, false
xtLPEGPDP2PFilterR	bool	user	true	GPD query resistors for query parasitics.
xtLPEGPDQueryAll	bool	user	true	GPD query all devices for query parasitics.
xtLPEGPDQueryCg	bool	cellview	true	GPD query grounded capacitor for query parasitics. Valid values: true, false
xtLPEGPDQueryExportResultToFile	bool	cellview	true	GPD Parasitic Query Export Result to File. Valid values: true, false
xtLPEGPDQueryHistoryLimit	int	user	10	Maximum Number of GPD Parasitic Query History.
xtLPEGPDQueryHistoryLimitNumber	int	cellview	10	Maximum Number of GPD Parasitic Query History.
xtLPEGPDQueryInstCheckAll	bool	cellview	true	GPD query all devices for query parasitics. Valid values: true, false
xtLPEGPDQueryR	bool	cellview	true	GPD query resistors for query parasitics. Valid values: true, false
xtLPEGPDReadGPD	bool	user	true	Read GPD parasitics. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEGPDReadGPDPPath	string	user		Read GPD parasitics path. Valid values: true, false
xtLPEGPDReadOALayerMap	bool	user	false	Read GPD OA Layer Map. Valid values: true, false
xtLPEGPDReadOALayerMapPath	string	user		Read GPD OA Layer Map path.
xtLPEGPDQueryShowTip	bool	cellview	true	GPD Parasitic Highlight show tips. Valid values: true, false
xtLPEGPDQueryViaR	bool	user	true	GPD query resistors for query parasitics. Valid values: true, false
xtLPEGPDQueryZoomFit	bool	cellview	true	GPD Parasitic Highlight zoom fit. Valid values: true, false
xtLPEGPDThresholdCc	string	cellview	0.0	Coupled Capacitor Threshold Value.
xtLPEGPDThresholdCg	string	cellview	0.0	Grounded Capacitor Threshold Value.
xtLPEGPDThresholdR	string	cellview	0.0	Resistor Threshold Value.
xtLPEHostList	string	custom	localhost	Run hostname list.
xtLPEHostname	string	user	localhost	Remote Hostname
xtLPEIndThresh	float	rundir	0	Threshold value below which to filter parasitic inductors from query results.
xtLPEInstLimit	int	rundir	100	Maximum number of instances of each type to display in a query report table.
xtLPEJobClass	string	custom	LPE	Job class

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEJobLimit	int	user	3	Job limit. Valid values: Any integer in the [1 1000000] range.
xtLPEJobLimitEnabled	bool	user	true	Job limit disabled (limited) Valid values: true, false
xtLPEJobQueueEngine	string	user		Queue Engine
xtLPELayLCV	lcv	custom	//	Layout LCV.
xtLPELayLCV2	lcv	custom	<file path>	Layout2 LCV
xtLPELPEJobName	string	custom	LPE	LPE job name in combined LVS-LPE dialog
xtLPELSFEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtLPELSFExtraOptions	string	user		Extra command line options for LSF job launch
xtLPELSFMaxArraySize	int	user	1000	Max job array size
xtLPELSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLPELSFQueues	string	user		LSF Queues
xtLPELSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtLPELSFResources	string	user		LSF Resource types
xtLPELSFSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtLPELSFSubmitHost	string	user	localhost	Submit host name for remote grid launch

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPELSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtLPELVSTool	string	custom	IC Validator	LVS Tool for seeding LPE type jobs.
xtLPELVSTool	string	custom	IC Validator	LVS Tool for seeding LPE type jobs.
xtLPEMakeExtractedPortable	bool	custom	false	make extracted view portable
xtLPEMaxDisplayInsts	int	custom	100	Maximum number of instances to display.
xtLPENetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtLPENetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtLPENetworkComputerJobClasses	string	user		NetworkComputer job classes
xtLPENetworkComputerMaxArraySize	int	user	1000	Max job array size
xtLPENetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLPENetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not. Valid values: true, false
xtLPENetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class
xtLPENetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPENetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLPENetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtLPEOpenParasiticView	bool	custom	true	Open Parasitic View. Valid values: true, false
xtLPEParLCV	lcv	custom	//	Parasitic View LCV.
xtLPEP2PQueryGPDType	string	cellview	both	GPD Query Type for query parasitics
xtLPEParWinId	string	custom		Parasitic View Window ID.
xtLPEPLEFlow	bool	custom	false	Partial layout extraction flow Valid values: true, false
xtLPEPrefixes	string	user		Job class prefixes
xtLPEPrependCommand	string	user		String to prepend to job shell command line
xtLPEPwrGndNets	string	custom		Power/Ground Net Names.
xtLPEPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtLPEQueryCc	bool	custom	true	Query Coupled Capacitors. Valid values: true, false
xtLPEQueryCg	bool	custom	true	Query Grounded Capacitors. Valid values: true, false
xtLPEQueryCorner	string	context		Corner for query parasitics

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtLPEQueryDevices	string	custom		Query parasitic devices
xtLPEQueryGPDCCapHl	string	user	highlight14	GPD Parasitic Highlight Pocket for Coupling Capacitor in option dialog
xtLPEQueryGPDcmdCouplingCapacitorHighlight	string	cellview	highlight14	GPD Parasitic Highlight Pocket for Coupling Capacitor in command dialog box.
xtLPEQueryGPDcmdGroundCapacitorHighlight	string	cellview	highlight5	GPD Parasitic Highlight Pocket for Ground Capacitor in command dialog box.
xtLPEQueryGPDcmdResistorHighlight	string	cellview	highlight4	GPD Parasitic Highlight Pocket for Resistor in command dialog box.
xtLPEQueryGPDCornerList	string	cellview		GPD Corner list for query parasitics.
xtLPEQueryGPDCornerValue	string	cellview		GPD Corner value for query parasitics.
xtLPEQueryGPDcouplingCapacitorHighlight	string	cellview	highlight14	GPD Parasitic Highlight Pocket for Coupling Capacitor.
xtLPEQueryGPDGCapHl	string	user	highlight5	GPD Parasitic Highlight Pocket for Ground Capacitor in option dialog
xtLPEQueryGPDGroundCapacitorHighlight	string	cellview	highlight5	GPD Parasitic Highlight Pocket for Ground Capacitor.
xtLPEQueryGPDNet2Value	string	cellview		GPD Net2 Name for query parasitics.
xtLPEQueryGPDNetFrom	string	cellview	GPD	GPD Use Net From for query parasitics.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEQueryGPDNetHI	string	user	highlight3	GPD Parasitic Highlight Pocket for Net in option dialog
xtLPEQueryGPDNetList	string	cellview		GPD Net Name list for query parasitics.
xtLPEQueryGPDNetValue	string	cellview		GPD Net Name for query parasitics.
xtLPEQueryGPDOptionCouplingCapacitorHighlight	string	cellview	highlight14	GPD Parasitic Highlight Pocket for Coupling Capacitor in option dialog box.
xtLPEQueryGPDOptionGroundCapacitorHighlight	string	cellview	highlight5	GPD Parasitic Highlight Pocket for Ground Capacitor in option dialog box.
xtLPEQueryGPDOptionPinCapacitorHighlight	string	cellview		GPD Parasitic Highlight Pocket for Pin Capacitor in option dialog box.
xtLPEQueryGPDOptionResistorHighlight	string	cellview	highlight4	GPD Parasitic Highlight Pocket for Resistor in option dialog box.
xtLPEQueryGPDResHI	string	user	highlight4	GPD Parasitic Highlight Pocket for Resistor in option dialog
xtLPEQueryGPDResistorHighlight	string	cellview	highlight4	GPD Parasitic Highlight Pocket for Resistor.
xtLPEQueryGPDResultFile	string	cellview		GPD Result File for saving query results.
xtLPEQueryGPDSHadowMode	bool	cellview	true	GPD Shadow Mode for highlight parasitics. Valid values: true, false
xtLPEQueryGPDTerminal1List	string	cellview		GPD Terminal1 Name list for query parasitics.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEQueryGPDTerminal1Value	string	cellview		GPD Terminal1 Name for query parasitics.
xtLPEQueryGPDTerminal2List	string	cellview		GPD Terminal2 Name list for query parasitics.
xtLPEQueryGPDTerminal2Value	string	cellview		GPD Terminal2 Name for query parasitics.
xtLPEQueryGPDType	string	cellview	net	Type for query GPD parasitics.
xtLPEQueryInd	bool	rundir	true	Flag to determine if inductors are included in query results.
xtLPEQueryL	bool	custom	true	Query Inductors. Valid values: true, false
xtLPEQueryR	bool	custom	true	Query Resistors. Valid values: true, false
xtLPEQueryRes	bool	rundir	true	Flag to determine if resistors are included in query results.
xtLPEQueryRFilter	bool	custom	false	Query Resistors Filter Valid values: true, false
xtLPEResThresh	bool	rundir	true	Threshold value below which to filter parasitic resistors from query results.
xtLPERunDir	string	cellview		Run directory.
xtLPERunDirNameTemplate	string	user	%c.%t.%j	Run directory name template for LPE.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPERunHost	string	rundir	local	Name of the host machine to run extraction. The string "local" is a keyword meaning the job is running on the local host.
xtLPESchLCV	lcv	rundir	local	Schematic cellview to use for parasitic exploration.
xtLPESchWinId	string	rundir		Schematic View Window ID.
xtLPESGEEEnableJobArrays	bool	user	true	Enable job array support Valid values: true, false
xtLPESGEEExportEnvironment	bool	user	true	Export qsub environment to child job Valid values: true, false
xtLPESGEEExtraOptions	string	user		Extra command line options for SGE job launch
xtLPESGEMaxArraySize	int	ser	1000	Max job array size
xtLPESGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLPESGEProjects	string	user		SGE projects.
xtLPESGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtLPESGESelectedProject	string	user		Selected SGE project
xtLPESGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtLPESGESubmitHost	string	user	localhost	Submit host name for remote grid launch

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtLPESGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtLPEShowAllInsts	bool	custom	false	Overrides xtLPEInstLimit, if True, to display all parasitics in a query report table. Valid values: true, false
xtLPEShowC	bool	custom	true	Show Capacitors in Parasitic Output Report. Valid values: true, false
xtLPEShowCap	bool	rundir	true	Flag to determine if capacitors are shown in a query report table. Includes grounded and coupled capacitors.
xtLPEShowCc	bool	custom	true	Show Coupled Capacitors in Parasitic Output Report. Valid values: true, false
xtLPEShowCCap	bool	rundir	true	Flag to determine if coupled capacitors are shown in a query report table. If this is false and xtLPEShowCap is true, only grounded capacitors are shown.
xtLPEShowFlightLinesCc	bool	custom	false	Show Coupled Capacitor Flight Lines. Valid values: true, false
xtLPEShowFlightLinesCg	bool	custom	false	Show Grounded Capacitor Flight Lines. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEShowFlightLinesR	bool	custom	false	Show Resistor Flight Lines. Valid values: true, false
xtLPEShowInd	bool	rundir	true	Flag to determine if inductors are shown in a query report table.
xtLPEShowL	bool	custom	true	Show Inductors in Parasitic Output Report. Valid values: true, false
xtLPEShowR	bool	custom	true	Show Resistors in Parasitic Output Report. Valid values: true, false
xtLPEShowRes	bool	rundir	true	Flag to determine if resistors are shown in a query report table.
xtLPESLURMEnableJobArrays	bool	user	true	Enable job array support
xtLPESLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtLPESLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtLPESLURMMaxArraySize	int	user	1000	Max job array size
xtLPESLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLPESLURMPartitions	string	user	<None>	SLURM partitions.
xtLPESLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtLPESLURMSelectedPartition	string	user		Selected SLURM partition
xtLPESLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPESLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLPESLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtLPEStarAnalysis	string	custom	NONE	Specifies the analysis to configure specific set of StarRC commands.
xtLPEStarCalibreGenerateCCI	bool	custom	true	Generate CCI files in SVDB results database.
xtLPEStarCalibreMaskSVDBDir	string	custom		Location of Calibre-LVS SVDB Directory used to generate CCI files.
xtLPEStarCalibreQueryFile	string	rundir	localhost	Location of Calibre Query Server command file used to generate CCI files. Passed to Star-RCXT via the CALIBRE_QUERY_FILE command.
xtLPEStarCalibreQueryFileType	string	custom	Default	Type of Calibre Query Server Command File used to generate CCI files. Either the 'default' or a 'custom' file specified by the user.
xtLPEStarCaseSensitive	bool	custom	true	StarRC CASE_SENSITIVE command. Valid values: true, false
xtLPEStarCorner	string	custom	custom	StarRC corner case to run.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEStarCornerDef	string	pdk_pve		StarRC corner cases and the corresponding file.
xtLPEStarCornerFile	string	rundir	corners_file	StarRC corner case extract mode
xtLPEStarCornerMode	string	rundir	Single Corner	StarRC corner case extract mode
xtLPEStarCornerTemps	string	rundir	-40 25 125	StarRC corner case extract mode.
xtLPEStarCoupleToGround	bool	custom	true	StarRC COUPLE_TO_GROUND command. Valid values: true, false
xtLPEStarCouplingAbsThreshold	string	custom	3e-15	StarRC COUPLING_ABS_THRESHOLD command
xtLPEStarCouplingMultiplier	string	custom	1.0	StarRC COUPLING_MULTIPLIER command
xtLPEStarCouplingRelThreshold	string	custom	0.03	StarRC COUPLING_REL_THRESHOLD command
xtLPEStarCouplingReportFile	string	custom		StarRC COUPLING_REPORT_FILE command
xtLPEStarCouplingReportNumber	int	custom	1000	StarRC COUPLING_REPORT_NUMBER command
xtLPEStarCouplingThresholdOperation	string	custom	AND	StarRC COUPLING_THRESHOLD_OPERATION command
xtLPEStarDPT	string	custom	NO	StarRC DPT command

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEStarDensityBasedThickness	string	custom	YES	StarRC DENSITY_BASED_THICKNESS command
xtLPEStarDensityOutsideBlock	string	custom		StarRC DENSITY_OUTSIDE_BLOCK command
xtLPEStarExcludeCommands	string	custom		Command names to be excluded from showing on custom options
xtLPEStarExtraction	string	custom	RC	StarRC EXTRACTION command.
xtLPEStarExtractViaCaps	string	custom	NO	StarRC EXTRACT_VIA_CAPS command
xtLPEStarFsDpString	string	custom		StarRC FS_DP_STRING command
xtLPEStarFscompareCouplingRatio	string	custom	0.1	StarRC FSCOMPARE_COUPLING_RATIO command
xtLPEStarFscompareCouplingThreshold	string	custom	1.0e-15	StarRC FSCOMPARE_COUPLING_THRESHOLD command
xtLPEStarFscompareOptions	string	custom		StarRC FSCOMPARE_OPTIONS command
xtLPEStarFscompareThreshold	string	custom	3.0e-15	StarRC FSCOMPARE_THRESHOLD command
xtLPEStarFsExtractNets	string	custom		StarRC FS_EXTRACT_NETS command
xtLPEStarGPD	string	custom		StarRC GPD command.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEStarGdsLayerMapFile	string	custom		StarRC GDS_LAYER_MAP_FILE command
xtLPEStarHierarchicalSeparator	string	custom	<file path>	StarRC HIERARCHICAL_SEPARATOR command.
xtLPEStarHnNetlistModelName	string	custom		StarRC HN_NETLIST_MODEL_NAME command
xtLPEStarHnNetlistSpiceType	string	custom		StarRC HN_NETLIST_SPICE_TYPE command
xtLPEStarICVRunsetReportFile	string	custom		StarRC ICV_RUNSET_REPORT_FILE command.
xtLPEStarICVRunsetReportFileName	string	custom	pex_runset_report	ICV runset report file name.
xtLPEStarIgnoreCapacitance	string	custom	ALL	StarRC IGNORE_CAPACITANCE command
xtLPEStarInductanceSelectBbFile	string	custom		StarRC INDUCTANCE_SELECT_BB command
xtLPEStarInductanceSelectBbLayer	string	custom		StarRC INDUCTANCE_SELECT_BB command
xtLPEStarInductanceSelectBbRegion	string	custom		StarRC INDUCTANCE_SELECT_BB command
xtLPEStarInductanceSelectBbUseFile	bool	custom	false	StarRC INDUCTANCE_SELECT_BB command
xtLPEStarInductanceSelectSBbFile	string	custom		StarRC INDUCTANCE_SELECT_S_BB command

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEStarInductanceSelectSBbLayer	string	custom		StarRC INDUCTANCE_SELECT_S_BB command
xtLPEStarInductanceSelectSBbOutFile	string	custom		StarRC INDUCTANCE_SELECT_S_BB command
xtLPEStarInductanceSelectSBbRegion	string	custom		StarRC INDUCTANCE_SELECT_S_BB command
xtLPEStarInductanceSelectSBbUseFile	bool	custom	false	StarRC INDUCTANCE_SELECT_S_BB command
xtLPEStarKeepViaNodes	string	custom	NO	StarRC KEEP_VIA_NODES command
xtLPEStarLoadRunset	bool	custom	true	Controls parsing of the 'title' 'LPE' command file for options to populate the setup dialog. Valid values: true, false
xtLPEStarMagnificationFactor	string	custom		StarRC MAGNIFICATION_FACTOR command
xtLPEStarMagnifyDeviceParams	string	custom	YES	StarRC MAGNIFY_DEVICE_PARAMS command
xtLPEStarMappingFile	string	custom		StarRC MAPPING_FILE command.
xtLPEStarMergeInstancePorts	string	custom	NO	StarRC MERGE_INSTANCE_PORTS command
xtLPEStarMetalFillGdsFile	string	custom		StarRC METAL_FILL_GDS_FILE command

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEStarMetalFillPolygonHandling	string	custom	IGNORE	StarRC METAL_FILL_POLY YGON_HANDLING command
xtLPEStarModelType	string	custom	LAYOUT	StarRC MODEL_TYPE command
xtLPEStarMosGateDeltaResistance	string	custom	NO	StarRC MOS_GATE_DELT A_RESISTANCE command
xtLPEStarMultipleJobEnabled	bool	custom	true	Enable LPE job with starrc tool in running multiple jobs. Valid values: true, false
xtLPEStarNetlistConnectOpens	string	custom	*	StarRC NETLIST_CONNEC T_OPENS command
xtLPEStarNetlistCoupleUnselectedNets	string	custom	NO	StarRC NETLIST_COUPLE _UNSELECTED_N ETS command
xtLPEStarNetlistGroundNodeName	string	custom	0	StarRC NETLIST_GROUN D_NODE_NAME command.
xtLPEStarNetlistInstanceSection	string	custom	SELECTED	StarRC NETLIST_INSTANC E_SECTION command
xtLPEStarNetlistMincapThreshold	string	custom	0.0	StarRC NETLIST_MINCAP _THRESHOLD command
xtLPEStarNetlistMinresHandling	string	custom	MERGE	StarRC NETLIST_MINRES _HANDLING command

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEStarNetlistMinresThreshold	string	custom	0.0	StarRC NETLIST_MINRES _THRESHOLD command
xtLPEStarNetlistNodeSection	string	custom	NO	StarRC NETLIST_NODE_S ECTION command
xtLPEStarNetlistParasiticResistorModel	string	custom	NO	StarRC NETLIST_PARASIT IC_RESISTOR_MO DEL command
xtLPEStarNetlistPortOrder	bool	custom	true	Use LVS netlist port order for the 'StarRC' tool. Valid values: true, false
xtLPEStarNetlistPortOrderSkipCells	bool	custom	true	LVS netlist port order files of skip cells.
xtLPEStarNetlistSelectNets	string	custom		StarRC NETLIST_SELECT _NETS command.
xtLPEStarNetlistTailComments	string	custom	NO	StarRC NETLIST_TAIL_CO MMENTS command
xtLPEStarNetlistTotalcapThreshold	string	custom	0.0	StarRC NETLIST_TOTALC AP_THRESHOLD command
xtLPEStarNetlistFile	string	custom		StarRC NETLIST_FILE command.
xtLPEStarNetlistType	string	custom		StarRC NETLIST_TYPE command.
xtLPEStarNets	string	custom		StarRC NETS command
xtLPEStarOACellName	string	custom		StarRC OA_CELL_NAME command.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEStarOADeviceMappingFile	string	custom		StarRC OA_DEVICE_MAPPING_FILE command.
xtLPEStarOALayerMappingFile	string	custom		StarRC OA_LAYER_MAPPING_FILE command.
xtLPEStarOALibName	string	custom		StarRC OA_LIB_NAME command.
xtLPEStarOAMarkerSize	string	custom	0.01	StarRC OA_MARKER_SIZE command.
xtLPEStarOAViewName	rundir	custom	starrc	StarRC OA_VIEW_NAME command.
xtLPEStarOperatingTemperature	string	custom	0	StarRC OPERATING_TEMPERATURE command.
xtLVSICVNettranOutType	string	custom	ICV	ICV-nettran format option.
xtLPEStarOAPortAnnotationView	string	custom		StarRC OA_PORT_ANNOTATION_VIEW property for 'LPE'
xtLPEStarOASkipCellMappingFile	string	custom		Selected file for StarRC OA_SKIPCELL_MAPPING_FILE command.
xtLPEStarOASkipCellMappingFileSource	string	custom	Auto Generate	Source of StarRC OA_SKIPCELL_MAPPING_FILE command.
xtLPEStarOptions	string	rundir		Command line options for 'StarRC' 'LPE' jobs.
xtLPEStarOutputRunset	string	custom		StarRC LPE runset file.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEStarPinCutThreshold	string	custom		StarRC PIN_CUT_THRESH OLD command
xtLPEStarPowerExtract	string	custom	No	StarRC POWER_EXTRACT command.
xtLPEStarPowerNets	string	custom		StarRC POWER_NETS command
xtLPEStarPowerReduction	string	custom	YES	StarRC POWER_REDUCT ION command
xtLPEStarProbeTextFile	string	custom		StarRC PROBE_TEXT_F ILE command
xtLPEStarRcGPDTclCurrInstName	string	user		Current Instance for compatible StarRC TCL commands
xtLPEStarReduction	string	custom	LAYER	StarRC REDUCTION command.
xtLPEStarReductionMaxDelayError	string	custom	1e-12	StarRC REDUCTION_MAX _DELAY_ERROR command
xtLPEStarRemoveDanglingNets	string	custom	NO	StarRC REMOVE_DANGLI NG_NETS command
xtLPEStarRemoveFloatingNets	bool	custom	true	StarRC REMOVE_FLOATI NG_NETS command. Valid values: true, false
xtLPEStarRetainCapacitanceCapModels	string	custom		StarRC RETAIN_CAPACIT ANCE_CAP_MOD ELS command

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEStarReviewOpenShort	bool	custom	false	Loads the open/short violations from GPD database when the StarRC job is complete.
xtLPEStarRunsetFile	string	custom		StarRC LPE runset file.
xtLPEStarRunsetOptions	string	custom		List of LPE {option value} options to be added to the 'StarRC' runset.
xtLPEStarShortPins	string	custom	YES	StarRC SHORT_PINS command
xtLPEStarShowPdkConfigOptionsBool	bool	custom	true	Control visibility of pdkConfig type options in custom options table
xtLPEStarShowRunsetOptionsBool	bool	custom	true	Control visibility of runset type options in custom options table
xtLPEStarShowUserOptionsBool	bool	custom	true	Control visibility of user type options in custom options table
xtLPEStarSkipCellList	string	custom		Selected cells for StarRC SKIP_CELLS command.
xtLPEStarSkipCellListFile	string	custom		Selected cell list file for StarRC SKIP_CELLS command.
xtLPEStarSkipCellSource	string	custom	None	Source of StarRC SKIP_CELLS command.
xtLPEStarSMCOutputMode	string	rundir	Multiple Views	StarRC smc output mode.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPESpiceSubcktFile	string	custom		StarRC SPICE_SUBCKT_F ILE command. Netlist port ordering information for the 'StarRC' tool.
xtLPESpiceStarDirectory	string	custom	star	StarRC STAR_DIRECT ORY command
xtLPESpiceTcadGrdFile	string	custom	custom	StarRC TCAD_GRD_FILE command.
xtLPESpiceTranslateRetainBulkLayers	string	custom	NO	StarRC TRANSLATE_RET AIN_BULK_LAY ERS command
xtLPESpiceWideDeviceTermResistance	string	custom	NONE	StarRC WIDE_DEVICE_TE RM_RESISTANCE command
xtLPESpiceXref	string	custom	Yes	StarRC XREF command
xtLPESpiceXrefFeedthruNets	string	custom	NO	StarRC XREF_FEEDTHRU _NETS command
xtLPESpiceXrefLayoutInstPrefix	string	custom	Id_	StarRC XREF_LAYOUT_IN ST_PREFIX command
xtLPESpiceXrefLayoutNetPrefix	string	custom	In_	StarRC XREF_LAYOUT_N ET_PREFIX command
xtLPESpiceXrefUseLayoutDeviceName	string	custom	NO	StarRC XREF_USE_LAYO UT_DEVICE_NAME command
xtLPESpiceThresholdCc	string	custom	0	Coupled Capacitor Threshold Value.
xtLPESpiceThresholdCg	string	custom	0	Grounded Capacitor Threshold Value.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLPEThresholdL	string	custom	0	Inductor Threshold Value.
xtLPEThresholdR	string	custom	0	Resistor Threshold Value.
xtLPETool	string	rundir	Star-RCXT	The verification tool to use in Extraction.
xtLPEViewOutput	bool	custom	true	Open text viewer. Valid values: true, false
xtLSFAutoDetect	bool	user	true	Automatically detect LSF settings from environment. Valid values: true, false
xtLSFDebug	bool	user	false	Display debugging information for LSF. Valid values: true, false
xtLSFEnableJobArrays	bool	user	false	Enables job array support. Valid values: true, false.
xtLSFExtraOptions	string	user		Extra command line options for LSF job launch.
xtLSFMaxArraySize	int	user	1000	Sets the maximum job array size.
xtLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtLSFQueues	string	user		LSF Queues.
xtLSFRemoteSubmit	bool	user	false	Specifies if the submit host is on remote site or not. Valid values: true, false
xtLSFResources	string	user		LSF Resource types.
xtLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLSFSubmitHost	string	user	localhost	Specifies Submit host name for remote grid launch.
xtLSFUseApplicationProfilesAsQueues	bool	user	false	Use application profile name as queue name
xtLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtTerminalCommand	string	user	xterm	Controls terminal command used by the <code>xt::execTerminal</code> command.
xtLVLAutoPrefs	bool	custom	true	Auto-load prefs.xml Valid values: true, false
xtLVLComparisonType	string	custom	XOR	LVL Comparison Type
xtLVLExportOasis	bool	custom	true	Export OA layout to Oasis Valid values: true, false
xtLVLExportOasis2	bool	custom	true	Export OA layout to stream Valid values: true, false
xtLVLExportStream	bool	custom	true	Export OA layout to stream Valid values: true, false
xtLVLExportStream2	bool	custom	true	Export OA layout to stream Valid values: true, false
xtLVLHostList	string	custom	localhost	Run hostname list
xtLVLHostname	string	user	localhost	Remote Hostname
xtLVLICVControlVars	string	custom		ICV preprocessor variables

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVLICVControlVariablesTableConfig	string	custom		Column size or visibility of Control Variable table
xtLVLICVEnvVars	string	custom	<None>	ICV environment variables
xtLVLICVIncludePaths	string	custom		List of LVL include directory paths used for locating 'IC Validator' runset include files.
xtLVLICVLoadRunset	bool	custom	false	Controls parsing of the 'title' 'LVL' runset file for variables and options to populate the setup dialog. Valid values: true, false
xtLVLICVMacros	string	custom	<None>	ICV macros
xtLVLICVMultipleJobEnabled	bool	custom	true	Enable LVL job with icv tool in running multiple jobs. Valid values: true, false
xtLVLICVNvN	bool	custom	false	Use netlist versus netlist for ICV. Valid values: true, false
xtLVLICVOptions	string	custom	-oa_dm5	Command line options for 'IC Validator' 'LVL' jobs.
xtLVLICVOutputFiles	string	custom		Output files of ICV DRC/LVS job
xtLVLICVRunSummary	string	custom		Run Summary of ICV DRC/LVS job
xtLVLICVRunsetFile	string	custom		IC Validator LVL runset file.
xtLVLICVRunsetOptions	string	custom		List of LVL {option value} options to be added to the 'IC Validator' runset.

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtLVLICVSelectWindow	string	custom		Window to limit DRC processing. The value defines x and y coordinates of opposite corners of a window.
xtLVLICVSelectWindowType	string	custom	Design	Window type to limit DRC processing.
xtLVLICVShowRunSummary	bool	custom	false	Enable Run Summary for ICV DRC/LVS job
xtLVLICVSmallCell	bool	custom	false	Use the small cell options for ICV Valid values: true, false
xtLVLJobClass	string	custom	LVL	Job class
xtLVLJobLimit	int	user	3	Job limit
xtLVLJobLimitEnabled	bool	user	true	Job limit disabled (limited) Valid values: true, false
xtLVLJobQueueEngine	string	user		Queue Engine
xtLVLLaunchDebugger	bool	custom	true	Launch Debugger Valid values: true, false
xtLVLLayCell	string	custom		Layout cell
xtLVLLayCell2	string	custom		Layout2 cell
xtLVLLayDBFormat	string	custom	OpenAccess	Database format of the layout
xtLVLLayDBFormat2	string	custom	OpenAccess	Database format of the layout 2
xtLVLLayerMap	string	custom	None	Layout1 Layer Map
xtLVLLayerMap2	string	custom	None	Layout2 Layer Map
xtLVLLayerMapFile	string	custom		Layout1 Layer Map
xtLVLLayerMapFile2	string	custom		Layout2 Layer Map
xtLVLLayLCV	lcv	custom	<file path>	Layout LCV

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVLLayLCV2	lcv	custom	<file path>	Layout2 LCV
xtLVLLayOasisFile	string	custom		Layout Oasis file
xtLVLLayOasisFile2	string	custom		Layout Oasis file
xtLVLLayStreamFile	string	custom		Layout stream file
xtLVLLayStreamFile2	string	custom		Layout2 stream file
xtLVLLSFEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtLVLLSFExtraOptions	string	user		Extra command line options for LSF job launch
xtLVLLSFMaxArraySize	int	user	1000	Max job array size
xtLVLLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVLLSFQueues	string	user		LSF Queues
xtLVLLSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtLVLLSFResources	string	user		LSF Resource types
xtLVLLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtLVLLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVLLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtLVLLVLLayerMap	string	custom	None	Layout1 Layer Map
xtLVLLVLLayerMap2	string	custom	None	Layout2 Layer Map

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVLNetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtLVLNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtLVLNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtLVLNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtLVLNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVLNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtLVLNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class
xtLVLNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtLVLNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVLNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtLVLPrefixes	string	user		Job class prefixes
xtLVLPendCommand	string	user		String to prepend to job shell command line

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtLVLPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtLVLRunDir	string	custom		Run directory
xtLVLRunDirNameTemplate	string	custom	%c.%t.%j	Template for customizing run directory name
xtLVLRunHost	string	custom		Run hostname
xtLVLSGEEEnableJobArrays	bool	user	true	Enable job array support Valid values: true, false
xtLVLSGEEexportEnvironment	bool	user	true	Export qsub environment to child job Valid values: true, false
xtLVLSGEEExtraOptions	string	user		Extra command line options for SGE job launch
xtLVLSGEMaxArraySize	int	user	1000	Max job array size
xtLVLSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVLSGEProjects	string	user		SGE projects.
xtLVLSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtLVLSGESelectedProject	string	user		Selected SGE project
xtLVLSGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtLVLSGESubmitHost	string	user	localhost	Submit host name for remote grid launch

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVLSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtLVLSLURMEnableJobArrays	bool	user	true	Enable job array support
xtLVLSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtLVLSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtLVLSLURMMaxArraySize	int	user	1000	Max job array size
xtLVLSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVLSLURMPartitions	string	user	<None>	SLURM partitions.
xtLVLSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtLVLSLURMSelectedPartition	string	user		Selected SLURM partition
xtLVLSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtLVLSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVLSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtLVLTool	string	custom	IC Validator	Tool
xtLVLViewOutput	bool	custom	true	Open text viewer. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSAutoPrefs	bool	user	true	Determines if the prefs.xml file is automatically saved and loaded for an LVS run. Valid values: true, false
xtLVSCalibreAutoMatch	bool	custom	false	Use cell name to determine cell correspondence between layout and schematic. Valid values: true, false
xtLVSCalibreControlVars	string	custom		Calibre Control variables
xtLVSCalibreControlVariablesTableConfig	string	custom		Column size/visibility of Control Variable table
xtLVSCalibreEchoSVRF	bool	custom	false	Echo compiled SVRF commands and included files in the Calibre LVS job output. Valid values: true, false
xtLVSCalibreEquivCellList	string	custom		LVS Eauiv/Hcell cell list.
xtLVSCalibreEquivCellListFile	string	custom		LVS Eauiv/Hcell cell list file.
xtLVSCalibreEquivFile	string	custom		LVS Eauiv/Hcell file.
xtLVSCalibreEquivFileSource	string	custom	None	Source of LVS Eauiv/Hcell file.
xtLVSCalibreGenerateCCI	bool	custom	true	Generate CCI files in LVS SVDB results database. Valid values: true, false
xtLVSCalibreGroundName	string	custom		Calibre LVS GROUND NAME svrf Command

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSCalibreHCellsFile	string	custom		Location of file containing cell correspondences between layout and schematic.
xtLVSCalibreHierarchicalRunMode	bool	custom	true	Enables 'LVS' hierarchical mode of operation. Valid values: true, false
xtLVSCalibreIncludeFiles	string	custom		Calibre INCLUDE svrf Command
xtLVSCalibreLayInputNetlistFormat	string	custom	SPICE	Layout netlist format.
xtLVSCalibreLayOutputNetlist	string	custom		Name of the output layout netlist file.
xtLVSCalibreLoadRunset	bool	custom	false	Controls parsing of the 'title' 'LVS' runset file for variables and options to populate the setup dialog. Valid values: true, false
xtLVSCalibreMaskSVDBDir	string	custom		Location of LVS SVDB results database.
xtLVSCalibreMultipleJobEnabled	bool	custom	true	Enable LVS job with calibre tool in running multiple jobs. Valid values: true, false
xtLVSCalibreOptions	string	custom		Command line options for 'Calibre' 'LVS' jobs.
xtLVSCalibreOutputFiles	string	custom		Output files of ICV DRC/LVS job
xtLVSCalibreOutputRunset	string	custom		Calibre LVS output runset control file.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSCalibreOverrideRunsetFile	bool	custom	false	Override SVRF commands in Calibre LVS primary runset file. Valid values: true, false
xtLVSCalibrePowerName	string	custom		Calibre LVS POWER NAME svrf Command
xtLVSCalibreReport	string	custom		Location of the LVS report.
xtLVSCalibreRunsetFile	string	custom		Calibre LVS primary runset file.
xtLVSCalibreRunsetOptions	string	custom		List of LVS {option value} options for 'Calibre' Interactive.
xtLVSCalibreRunSummary	string	custom		Run Summary of ICV DRC/LVS job
xtLVSCalibreShowRunSummary	bool	custom	false	Enable Run Summary of Calibre DRC/LVS job
xtLVSCalibreSpiceCells	string	custom		LVS additional netlist files in Spice format.
xtLVSCalibreTranscript	string	custom	stdout.lvs.log	Output file name for the Calibre LVS Transcript.
xtLVSCalibreTranscriptRules	string	custom	stdout.lvs.svrf	Name to use for SVRF rules file obtained by parsing the Calibre LVS Transcript.
xtLVSCalibreUseSVRFTranscript	bool	custom	true	Specifies if the SVRF rules file should be parsed out of the Calibre LVS Transcript. Valid values: true, false
xtLVSCalibreVirtualConAttr	string	custom		Calibre LVS VIRTUAL CONNECT Command

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtLVSCalibreVirtualConBool	bool	custom	false	Flag to Calibre LVS VIRTUAL CONNECT Valid values: true, false
xtLVSCalibreVirtualConColon	string	custom	None	Calibre LVS VIRTUAL CONNECT COLON Command
xtLVSCalibreVirtualConDepth	string	custom	Primary	Calibre LVS VIRTUAL CONNECT DEPTH Command
xtLVSCalibreVirtualConName	string	custom		Calibre LVS VIRTUAL CONNECT NAME Command
xtLVSCalibreVirtualConReport	string	custom	No	Calibre LVS VIRTUAL CONNECT REPORT Command
xtLVSCalibreVirtualConReportCount	string	custom		Calibre LVS VIRTUAL CONNECT REPORT MAXIMUM Command
xtLVSCalibreVirtualConReportMaximum	string	custom	All	Calibre LVS VIRTUAL CONNECT REPORT MAXIMUM Command
xtLVSExportOasis	bool	custom	true	Export OA layout to Oasis Valid values: true, false
xtLVSExportStream	bool	rundir	true	Export OpenAccess database to GDSII format. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSEExtractOnly	bool	rundir	false	Restrict the job to only extract the layout netlist. Valid values: true, false
xtLVSEXTAutoPrefs	bool	custom	true	Auto-load prefs.xml
xtLVSEXTCalibreAutoMatch	bool	custom	false	Use cell name to determine cell correspondence between layout and schematic.
xtLVSEXTCalibreControlVars	string	custom		Calibre Control variables
xtLVSEXTCalibreControlVariablesTableConfig	string	custom		Column size/visibility of Control Variable table
xtLVSEXTCalibreEchoSVRF	bool	custom	false	Echo compiled SVRF commands and included files in the Calibre LVSEXT job output.
xtLVSEXTCalibreEquivCellList	string	custom		LVS Eauiv/Hcell cell list.
xtLVSEXTCalibreEquivCellListFile	string	custom		LVS Eauiv/Hcell cell list file.
xtLVSEXTCalibreEquivFile	string	custom		LVS Eauiv/Hcell file.
xtLVSEXTCalibreEquivFileSource	string	custom	None	Source of LVS Eauiv/Hcell file.
xtLVSEXTCalibreGenerateCCI	bool	custom	true	Generate CCI files in LVSEXT SVDB results database.
xtLVSEXTCalibreGroundName	string	custom		Calibre LVS GROUND NAME svrf Command
xtLVSEXTCalibreHCellsFile	string	custom		Location of file containing cell correspondences between layout and schematic.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSEXTCalibreHierarchicalRunMode	bool	custom	true	Enables 'LVSEXT' hierarchical mode of operation.
xtLVSEXTCalibreIncludeFiles	string	custom		Calibre INCLUDE svrf Command
xtLVSEXTCalibreLayInputNetlistFormat	string	custom	SPICE	Layout netlist format
xtLVSEXTCalibreLayOutputNetlist	string	custom		Name of the output layout netlist file
xtLVSEXTCalibreLoadRunset	bool	custom	false	Controls parsing of the 'title' 'LVSEXT' runset file for variables and options to populate the setup dialog.
xtLVSEXTCalibreMaskSVDBDir	string	custom		Location of LVSEXT SVDB results database.
xtLVSEXTCalibreMultipleJobEnabled	bool	custom	true	Enable LVSEXT job with calibre tool in running multiple jobs.
xtLVSEXTCalibreOptions	string	custom		Command line options for 'Calibre' 'LVSEXT' jobs.
xtLVSEXTCalibreOutputFiles	string	custom		Output files of ICD DRC/LVS job
xtLVSEXTCalibreOutputRunset	string	custom		Calibre LVSEXT output runset control file.
xtLVSEXTCalibreOverrideRunsetFile	bool	custom	false	Override SVRF commands in Calibre LVSEXT primary runset file.
xtLVSEXTCalibrePowerName	string	custom		Calibre LVS POWER NAME svrf Command
xtLVSEXTCalibreReport	string	custom		Location of the LVSEXT report.
xtLVSEXTCalibreRunsetFile	string	custom		Calibre LVSEXT primary runset file.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSEXTCalibreRunsetOptions	string	custom		List of LVSEXT {option value} options for 'Calibre' Interactive.
xtLVSEXTCalibreSVRFOverwrite	bool	custom	true	Flag to indicate the SVRF rules file was not manually edited.
xtLVSEXTCalibreSpiceCells	string	custom		LVS additional netlist files in Spice format.
xtLVSEXTCalibreTranscript	string	custom	stdout.lvsext.log	Output file name for the Calibre LVSEXT Transcript.
xtLVSEXTCalibreTranscriptRules	string	custom	stdout.lvsext.svrf	Name to use for SVRF rules file obtained by parsing the Calibre LVSEXT Transcript.
xtLVSEXTCalibreUseSVRFTranscript	bool	custom	true	Specifies if the SVRF rules file should be parsed out of the Calibre LVSEXT Transcript.
xtLVSEXTCalibreVirtualConAttrs	string	custom		Calibre LVS VIRTUAL CONNECT Command
xtLVSEXTCalibreVirtualConBool	bool	custom	false	Flag to Calibre LVS VIRTUAL CONNECT
xtLVSEXTCalibreVirtualConColon	string	custom	None	Calibre LVS VIRTUAL CONNECT COLON Command
xtLVSEXTCalibreVirtualConDepth	string	custom	Primary	Calibre LVS VIRTUAL CONNECT DEPTH Command
xtLVSEXTCalibreVirtualConName	string	custom		Calibre LVS VIRTUAL CONNECT NAME Command

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSEXTCalibreVirtualConReport	string	custom	No	Calibre LVS VIRTUAL CONNECT REPORT Command
xtLVSEXTCalibreVirtualConReportCount	string	custom		Calibre LVS VIRTUAL CONNECT REPORT MAXIMUM Command
xtLVSEXTCalibreVirtualConReportMaximum	string	custom	All	Calibre LVS VIRTUAL CONNECT REPORT MAXIMUM Command
xtLVSEXTExportOasis	bool	custom	true	Export OA layout to Oasis
xtLVSEXTExportStream	bool	custom	true	Export OA layout to stream
xtLVSEXTExtractOnly	bool	custom	false	Extract layout netlist only.
xtLVSEXTHostList	string	custom	localhost	Run hostname list
xtLVSEXTHostname	string	user	localhost	Remote Hostname
xtLVSEXTICVBlackBoxCellList	string	custom		LVS Eauiv/Hcell cell list.
xtLVSEXTICVBlackBoxCellListFile	string	custom		LVS Eauiv/Hcell cell list file.
xtLVSEXTICVBlackBoxFile	string	custom		LVS Eauiv/Hcell file.
xtLVSEXTICVBlackBoxFileSource	string	custom	None	Source of LVS Eauiv/Hcell file.
xtLVSEXTICVControlVars	string	custom		ICV preprocessor variables
xtLVSEXTICVEnvVars	string	custom	<None>	ICV environment variables
xtLVSEXTICVEquivCellList	string	custom		LVS Eauiv/Hcell cell list.

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtLVSEXTICVEquivCellListFile	string	custom		LVS Eauiv/Hcell cell list file.
xtLVSEXTICVEquivFile	string	custom		LVS Eauiv/Hcell file.
xtLVSEXTICVEquivFileSource	string	custom	None	Source of LVS Eauiv/Hcell file.
xtLVSEXTICVICVNettranCells	string	custom		LVS nettran input files in IC Validator format.
xtLVSEXTICVIncludePaths	string	custom		List of LVSEXT include directory paths used for locating 'IC Validator' runset include files.
xtLVSEXTICVLayoutInputNetlistFormat	string	custom	ICV	Layout netlist format
xtLVSEXTICVLoadRunset	bool	custom	false	Controls parsing of the 'title' 'LVSEXT' runset file for variables and options to populate the setup dialog.
xtLVSEXTICVMacros	string	custom	<None>	ICV macros
xtLVSEXTICVMultipleJobEnabled	bool	custom	true	Enable LVSEXT job with icv tool in running multiple jobs.
xtLVSEXTICVNettranOutType	string	custom	ICV	ICV-nettran format option.
xtLVSEXTICVNvN	bool	custom	false	Use netlist versus netlist for ICV
xtLVSEXTICVOptions	string	custom	-oa_dm5	Command line options for 'IC Validator' 'LVSEXT' jobs.
xtLVSEXTICVRunsetConfigFile	string	custom		IC Validator LVSEXT runset config file.
xtLVSEXTICVRunsetFile	string	custom		IC Validator LVSEXT runset file.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSEXTICVRunsetOptions	string	custom		List of LVSEXT {option value} options to be added to the 'IC Validator' runset.
xtLVSEXTICVRunSummary	string	custom		Run Summary of ICV DRC/LVS job
xtLVSEXTICVShowRunsetOptions	bool	custom	false	Show Runset Options table in Custom Options tab for ICV
xtLVSEXTICVShowRunSummary	bool	custom	false	Enable Run Summary for ICV DRC/LVS job
xtLVSEXTICVSmallCell	bool	custom	false	Use the small cell options for ICV
xtLVSEXTICVSpiceNettranCells	string	custom		LVS nettran input files in Spice format.
xtLVSEXTICVVerilogNettranCells	string	custom		LVS nettran input files in Verilog format.
xtLVSEXTJobClass	string	custom	LVSEXT	Job class
xtLVSEXTJobLimit	int	user	3	Job limit
xtLVSEXTJobLimitEnabled	bool	user	true	Job limit disabled (limited)
xtLVSEXTJobQueueEngine	string	user		Queue Engine
xtLVSEXTKeepLayoutWindowActive	bool	custom	false	Keep layout window active when finishing job.
xtLVSEXTLSFEnableJobArrays	bool	user	true	Enable job array support.
xtLVSEXTLSFExtraOptions	string	user		Extra command line options for LSF job launch
xtLVSEXTLSFMaxArraySize	int	user	1000	Max job array size
xtLVSEXTLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSEXTLSFQueues	string	user		LSF Queues
xtLVSEXTLSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtLVSEXTLSFResources	string	user		LSF Resource types
xtLVSEXTLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtLVSEXTLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVSEXTLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtLVSEXTLaunchDebugger	bool	custom	true	Launch Debugger
xtLVSEXTLayoutCell	string	custom		Layout cell
xtLVSEXTLayoutDBFormat	string	custom	OpenAccess	Database format of the layout
xtLVSEXTLayoutInputNetlist	string	custom		Name of the input layout netlist file
xtLVSEXTLayoutLCV	lcv	custom	<file path>	Layout LCV
xtLVSEXTLayoutLCV2	lcv	custom	<file path>	Layout2 LCV
xtLVSEXTLayoutOasisFile	string	custom		Layout Oasis file
xtLVSEXTLayoutStreamFile	string	custom		Layout stream file
xtLVSEXTNLRunInBackground	bool	custom	false	Run export Stream in background.
xtLVSEXTNetlistTopAsModule	bool	custom	true	Create top level module definition.
xtLVSEXTNetworkComputerEnableJobArrays	bool	user	true	Enable job array support.
xtLVSEXTNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSEXTNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtLVSEXTNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtLVSEXTNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVSEXTNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtLVSEXTNetworkComputerSelectedJobClass	string	user	<None>	NetworkComputer selected job class
xtLVSEXTNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtLVSEXTNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVSEXTNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtLVSEXTPrefixes	string	user		Job class prefixes
xtLVSEXTPrependCommand	string	user		String to prepend to job shell command line
xtLVSEXTPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtLVSEXTRunDir	string	custom		Run directory
xtLVSEXTRunDirNameTemplate	string	custom	%c.%t.%j	Template for customizing run directory name
xtLVSEXTRunHost	string	custom		Run hostname
xtLVSEXTSGEEEnableJobArrays	bool	user	true	Enable job array support

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSEXTSGEExportEnvironment	bool	user	true	Export qsub environment to child job
xtLVSEXTSGEExtraOptions	string	user		Extra command line options for SGE job launch
xtLVSEXTSGEMaxArraySize	int	user	1000	Max job array size
xtLVSEXTSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVSEXTSGEProjects	string	user	<None>	SGE projects.
xtLVSEXTSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtLVSEXTSGESelectedProject	string	user		Selected SGE project
xtLVSEXTSGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtLVSEXTSGESubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVSEXTSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtLVSEXTSLURMEnableJobArrays	bool	user	true	Enable job array support
xtLVSEXTSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtLVSEXTSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtLVSEXTSLURMMaxArraySize	int	user	1000	Max job array size
xtLVSEXTSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVSEXTSLURMPartitions	string	user	<None>	SLURM partitions.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSEXTSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtLVSEXTSLURMSelectedPartition	string	user		Selected SLURM partition
xtLVSEXTSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtLVSEXTSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVSEXTSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtLVSEXTSchCell	string	custom		Schematic cell
xtLVSEXTSchDBFormat	string	custom	OpenAccess	Database format of the schematic
xtLVSEXTSchInputNetlist	string	custom		Name of the input schematic netlist file
xtLVSEXTSchInputNetlistFormat	string	custom	CDL	Format of the input schematic netlist
xtLVSEXTSchLCV	lcv	custom	<file path>	Schematic LCV
xtLVSEXTSchNetlistName	string	custom	%c.%t.%n	Template for schematic netlist file name.
xtLVSEXTSchNetlister	string	custom	CDL	Netlister for creating the schematic netlist
xtLVSEXTSchOutputNetlist	string	custom		Name of the output schematic netlist file
xtLVSEXTTool	string	custom	IC Validator	Tool
xtLVSEXTViewOutput	bool	custom	true	Open text viewer.
xtLVSHerculesLayInputNetlistFormat	string	rundir	Hercules	Format of the input layout netlist.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSHerculesLoadRunset	bool	custom	true	Controls parsing of the 'title' 'LVS' runset file for variables and options to populate the setup dialog. Valid values: true, false
xtLVSHerculesMultipleJobEnabled	bool	custom	true	Enable LVS job with hercule tool in running multiple jobs. Valid values: true, false
xtLVSHerculesOptions	string	rundir	Hercules	Command line options specific to the Hercules verification tool to perform LVS checking
xtLVSHerculesRunsetFile	string	rundir	Hercules	Runset file for a Hercules LVS run.
xtLVSHerculesRunsetOptions	string	custom		List of LVS {section option value} options to be added to the 'Hercules' runset.
xtLVSHerculesVariables	string	rundir	[]	List of LVS variable name/value pairs in the 'Hercules' runset file.
xtLVSHostList	string	rundir	localhost	List of machine names for selecting LVS run host.
xtLVSHostname	string	user	localhost	Remote Hostname
xtLVSIKVDebugger	string	rundir	icv_vue	Name of the LVS debugger to use with the ICV PV tool.
xtLVSIKVEnvVars	string	custom	<None>	ICV environment variables
xtLVSIKVEquivCellList	string	custom		LVS Equiv/Hcell cell list.

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtLVSICVEquivCellListFile	string	custom		LVS Equiv/Hcell cell list file.
xtLVSICVEquivFile	string	custom		LVS Equiv/Hcell file.
xtLVSICVEquivFileSource	string	custom	None	Source of LVS Equiv/Hcell file.
xtLVSICVIncludePaths	string	rundir	icv_vue	List of LVS include directory paths used for locating 'IC Validator' runset include files.
xtLVSICVLayInputNetlistFormat	string	rundir	ICV	Layout netlist format.
xtLVSICVLoadRunset	bool	custom		Controls parsing of the 'title' 'LVS' runset file for variables and options to populate the setup dialog. Valid values: true, false
xtLVSICVMacros	string	custom	<None>	ICV macros
xtLVSICVMultipleJobEnabled	bool	custom	true	Enable LVS job with icv tool in running multiple jobs. Valid values: true, false
xtLVSICVNvN	bool	custom	false	Use netlist versus netlist for ICV. Valid values: true, false
xtLVSICVOptions	string	custom	ICV	Command line options specific to the ICV verification tool to perform LVS checking.
xtLVSICVRunsetConfigFile	string	custom		IC Validator LVS runset config file.
xtLVSICVRunsetFile	string	rundir	ICV	Runset file for an ICV LVS run.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSIcVRunsetOptions	string	custom		List of LVS {option value} options to be added to the 'IC Validator' runset.
xtLVSIcVSmallCell	bool	custom	false	Use the small cell options for ICV. Valid values: true, false
xtLVsJobClass	string	custom	LVS	Job class
xtLVsJobLimit	int	user	3	Job limit. Valid values: Any integer in the [1 1000000] range.
xtLVsJobLimitEnabled	bool	user	true	Job limit disabled (limited) Valid values: true, false
xtLVsJobQueueEngine	string	user		Queue Engine
xtLVSLPEEnableExtractionTools	string	custom		Enable hidden extraction tools.
xtLVSLaunchDebugger	bool	custom	true	Launch Debugger. Valid values: true, false
xtLVSLayCell	string	custom		Layout cell.
xtLVSLayDBFormat	string	custom	OpenAccess	Database format of the layout, either openAccess, stream, or netlist.
xtLVSLayInputNetlist	string	rundir	Previously extracted layout netlist file	Name of the input layout netlist file.
xtLVSLayInputNetlistFormat	string	rundir	Previously extracted layout netlist file	Input layout netlist format.
xtLVSLayLCV	lcv	rundir	Previously extracted layout netlist file	LCV for the layout cellview to use for the LVS run.
xtLVSLayLCV2	lcv	custom	<file path>	Layout2 LCV
xtLVSLayOasisFile	string	custom		Layout Oasis file

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSLayStreamFile	string	rundir		Name of the input layout stream file.
xtLVSLSFEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtLVSLSFExtraOptions	string	user		Extra command line options for LSF job launch
xtLVSLSFMaxArraySize	int	user	1000	Max job array size
xtLVSLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVSLSFQueues	string	user		LSF Queues
xtLVSLSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtLVSLSFResources	string	user		LSF Resource types
xtLVSLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtLVSLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVSLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtLVSNedListTopAsModule	bool	rundir	true	Specifies whether the netlister should netlist the top level cell as a subckt. Valid values: true, false
xtLVSNetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtLVSNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtLVSNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtLVSNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVSNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtLVSNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class
xtLVSNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtLVSNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVSNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtLVSNLRunInBackground	bool	custom	false	Run export Stream in background.
xtLVSPLEFlow	bool	custom	false	Partial layout extraction flow Valid values: true, false
xtLVSPrefixes	string	user		Job class prefixes
xtLVSPrependCommand	string	user		String to prepend to job shell command line

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtLVSRunDir	string	cellview	true	Run directory.
xtLVSRunDirNameTemplate	string	user	%c.%t.%j	Run directory name template for LVS.
xtLVSRunHost	string	rundir	true	Name of machine for LVS job.
xtLVSSchCell	string	custom		Schematic cell.
xtLVSSchDBFormat	enPVD BFor mat	rundir	OpenAccess	Database format of the schematic, either openAccess or Netlist.
xtLVSSchInputNetlist	string	rundir	Previously generated schematic netlist file	Name of the input schematic netlist file. (existing file).
xtLVSSchInputNetlistFormat	string	custom	CDL	Format of the input schematic netlist. (existing file format).
xtLVSSchLCV	lcv	rundir	spice	LCV for the schematic cellview to use for the LVS run.
xtLVSSchNetlister	string	rundir	CDL	nINetlister name to be used to create the output schematic netlist.
xtLVSSchOutputNetlist	string	rundir	<cell>.synopsys_ custom.sp	Name of the output schematic netlist file.
xtLVSSGEEEnableJobArrays	bool	user	true	Enable job array support Valid values: true, false
xtLVSSGEEExportEnvironment	bool	user	true	Export qsub environment to child job Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSSGEXtraOptions	string	user		Extra command line options for SGE job launch
xtLVSSGEMaxArraySize	int	user	1000	Max job array size
xtLVSSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVSSGEPjects	string	user		SGE projects.
xtLVSSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtLVSSGESelectedProject	string	user		Selected SGE project
xtLVSSGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtLVSSGESubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVSSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtLVSSLURMEnableJobArrays	bool	user	true	Enable job array support
xtLVSSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtLVSSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtLVSSLURMMaxArraySize	int	user	1000	Max job array size
xtLVSSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtLVSSLURMPartitions	string	user	<None>	SLURM partitions.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtLVSSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtLVSSLURMSelectedPartition	string	user		Selected SLURM partition
xtLVSSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtLVSSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtLVSSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtLVSTool	string	rundir	IC Validator	The default verification tool to use for LVS.
xtLVSVViewOutput	bool	rundir	false	Open text viewer to view LVS job output when launched. Valid values: true, false
xtMultipleJobExportOASIS	string	user	false	Generates a common OASIS file to use as input for all jobs from the Run Multiple series instead of having each job generating its own OASIS file. Specify the layer mapping file to proceed with the OA-to-OASIS translation through the preference xtMultipleJobLayerMapFile. To be used when this preference true.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtMultipleJobExportStream	string	user	false	Generates a common GDSII file to use as input for all jobs from the Run Multiple series instead of having each job generating its own GDSII file. Specify the layer mapping file to proceed with the OA-to-GDSII translation through the preference xtMultipleJobLayerMapFile. To be used when this preference is true.
xtMultipleJobGroupCalibreDRCReports	string	user	false	Opens all Calibre DRC reports in a single Calibre RVE session.
xtMultipleJobKillDRCJobs	string	user	false	Stops remaining DRC jobs from a Run Multiple DRC series when a completed DRC job contains one or more violations.
xtMultipleJobLaunchDebugger	string	user	false	Overwrites the corresponding Launch Debugger option in each job.
xtMultipleJobLayerMapFile	string	user	false	Specifies the layer mapping file to proceed with the OA-to-OASIS or OA-to-GDSII translation. To be used when preference xtMultipleJobExportOASIS is true and preference xtMultipleJobExportOASIS or xtMultipleJobExportStream is true.

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtMultipleJobMergeICVDRCReports	string	user	false	Merges ICV DRC reports and opens the merged report in ICV VUE.
xtMultipleJobOASISFile	string	user	false	Specifies a common OASIS file to use as input for all jobs from the Run Multiple series instead of having each job generate its own OASIS file. To be used when preference xtMultipleJobExport OASIS is true.
xtMultipleJobShareOASIS	string	user	false	Overwrites the corresponding OASIS export in each job. For performance reasons, all jobs that will use a common OASIS file specified through preference xtMultipleJobOASIS File as input instead of generating their own OASIS file.
xtMultipleJobShareStream	string	user	false	Overwrites the corresponding GDSII export in each job. For performance reasons, all jobs that will use a common GDSII file specified through preference xtMultipleJobLayer MapFile as input instead of generating their own GDSII file.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtMultipleJobStreamFile	string	user	false	Specifies a common GDSII file to use as input for all jobs from the Run Multiple series instead of having each job generate its own GDSII file. To be used when preference xtMultipleJobExportJobLayerMapFile is true.
xtMultipleJobViewOutput	string	user	false	Overwrites the current View Output option in each job.
xtNanosimDefaultNetlister	string	user	NanoSim	The name of the default netlister to choose when generating a netlist.
xtNanosimDefaultViewer	string	user	WaveView	Default waveform viewer for plotting signals.
xtNanosimGlobalSearchList	string	custom	hspice hspiceD spectre cmos.sch cmos_sch schematic veriloga	Global search list passed to the netlister.
xtNanosimGlobalStopList	string	custom	hspice hspiceD spectre veriloga	Global stop list passed to the netlister.
xtNetBasedCheckConstraintFile	string	custom		The net constraint file of net based checker.
xtNetBasedCheckExportStream	bool	custom	false	Enable this option to export stream
xtNetBasedCheckHierMode	bool	custom	false	The hierarchy mode option of net based checker. Valid values: true, false
xtNetBasedCheckHostname	string	user	localhost	Remote Hostname.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetBasedCheckJobClass	string	custom	NetBasedCheck	The job class for running net based check.
xtNetBasedCheckJobLimit	int	user	3	Job limit.
xtNetBasedCheckJobLimitEnabled	bool	user	true	Job limit disabled (limited). Valid values: true, false
xtNetBasedCheckJobQueueEngine	string	user		Queue Engine.
xtNetBasedCheckLSFEnableJobArrays	bool	user	true	Enables job array support. Valid values: true, false
xtNetBasedCheckLSFExtraOptions	string	user		Extra command line options for LSF job launch.
xtNetBasedCheckLSFMaxArraySize	int	user	1000	Max job array size.
xtNetBasedCheckLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtNetBasedCheckLSFQueues	string	user		LSF Queues.
xtNetBasedCheckLSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not. Valid values: true, false
xtNetBasedCheckLSFResources	string	user		LSF Resource types.
xtNetBasedCheckLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF.
xtNetBasedCheckLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch.
xtNetBasedCheckLSFUseNetcat	bool	user	false	Uses netcat command line utility to send status updates. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetBasedCheckLayStreamFile	string	custom		The stream file of design to run net based check.
xtNetBasedCheckNetworkComputerEnableJobArrays	bool	user	true	Enables job array support. Valid values: true, false
xtNetBasedCheckNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch.
xtNetBasedCheckNetworkComputerJobClasses	string	user		NetworkComputer job classes.
xtNetBasedCheckNetworkComputerMaxArraySize	int	user	1000	Max job array size.
xtNetBasedCheckNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtNetBasedCheckNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not. Valid values: true, false
xtNetBasedCheckNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class.
xtNetBasedCheckNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer.
xtNetBasedCheckNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch.
xtNetBasedCheckNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtNetBasedCheckPinTextMap	string	custom	Technology Library	The pin text map mode of net based checker.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetBasedCheckPinTextMapFile	string	custom		The pin text map file of net based checker.
xtNetBasedCheckPrefixes	string	user		Job class prefixes.
xtNetBasedCheckPrependCommand	string	user		String to prepend to job shell command line.
xtNetBasedCheckPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtNetBasedCheckRunDir	string	custom	windowDRC	The run directory of net based checker.
xtNetBasedCheckSGEEnableJobArrays	bool	user	true	Enables job array support. Valid values: true, false
xtNetBasedCheckSGEExportEnvironment	bool	user	true	Export qsub environment to child job. Valid values: true, false
xtNetBasedCheckSGEExtraOptions	string	user		Extra command line options for SGE job launch.
xtNetBasedCheckSGEMaxArraySize	int	user	1000	Max job array size.
xtNetBasedCheckSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtNetBasedCheckSGEProjects	string	user		SGE projects.
xtNetBasedCheckSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not. Valid values: true, false
xtNetBasedCheckSGESelectedProject	string	user		Selected SGE project.
xtNetBasedCheckSGESetupCommand	string	user		Shell command to run on remote host to configure SGE.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetBasedCheckSGESubmitHost	string	user	localhost	Submit host name for remote grid launch.
xtNetBasedCheckSGEUseNetcat	bool	user	false	Uses netcat command line utility to send status updates. Valid values: true, false
xtNetBasedCheckSLURMEnableJobArrays	bool	user	true	Enable job array support
xtNetBasedCheckSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtNetBasedCheckSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtNetBasedCheckSLURMMaxArraySize	int	user	1000	Max job array size
xtNetBasedCheckSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtNetBasedCheckSLURMPartitions	string	user	<None>	SLURM partitions.
xtNetBasedCheckSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtNetBasedCheckSLURMSelectedPartition	string	user		Selected SLURM partition
xtNetBasedCheckSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtNetBasedCheckSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtNetBasedCheckSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtNetBasedCheckTool	string	custom	IC Validator	The tool of net based checker.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetBasedCheckViewOutput	bool	custom	false	The view output option of net based checker. Valid values: true, false
xtNetBasedShieldCheckCCIData	string	custom		The CCI data of net based checker.
xtNetBasedShieldCheckConstraintFile	string	custom		The net constraint file of net based checker
xtNetBasedShieldCheckCornerClose	bool	custom	true	The corner close option of net based checker
xtNetBasedShieldCheckExcludeCells	string	custom		The string of exclude cells
xtNetBasedShieldCheckExportCCI	bool	custom	true	Enable to export CCI database.
xtNetBasedShieldCheckExportStream	bool	custom	false	Enable this option to export stream
xtNetBasedShieldCheckForbiddenRegion	bool	custom	false	The forbidden region option of net based checker
xtNetBasedShieldCheckHierMode	bool	custom	false	The hierarchy mode option of net based checker
xtNetBasedShieldCheckHostname	string	user	localhost	Remote Hostname
xtNetBasedShieldCheckJobClass	string	custom	NetBasedShieldC heck	The job class for running net based check
xtNetBasedShieldCheckJobLimit	int	user	3	Job limit
xtNetBasedShieldCheckJobLimitEnabled	bool	user	true	Job limit disabled (limited)
xtNetBasedShieldCheckJobQueueEngine	string	user		Queue Engine
xtNetBasedShieldCheckLSFEnableJobArrays	bool	user	true	Enable job array support.
xtNetBasedShieldCheckLSFExtraOptions	string	user		Extra command line options for LSF job launch

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtNetBasedShieldCheckLSFMaxArraySize	int	user	1000	Max job array size
xtNetBasedShieldCheckLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtNetBasedShieldCheckLSFQueues	string	user		LSF Queues
xtNetBasedShieldCheckLSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtNetBasedShieldCheckLSFResources	string	user		LSF Resource types
xtNetBasedShieldCheckLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtNetBasedShieldCheckLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtNetBasedShieldCheckLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtNetBasedShieldCheckLVSRunset	string	custom		The LVS runset file to generate LVS result for net based shield check
xtNetBasedShieldCheckLVSVueFile	string	custom		The LVS result VUE file
xtNetBasedShieldCheckLabelSource	string	custom	Auto Labeling	The label source for net based shield check.
xtNetBasedShieldCheckLayStreamFile	string	custom		The stream file of design to run net based shield check
xtNetBasedShieldCheckLayerMap	string	custom		The layer map file that used for generate LVS result.
xtNetBasedShieldCheckLineEndClose	bool	custom	true	The line end close option of net based checker

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetBasedShieldCheckLineEndJump	bool	custom	false	The line end jump option of net based checker
xtNetBasedShieldCheckMetaFile	string	custom		The meta file
xtNetBasedShieldCheckNetworkComputerEnableJobArrays	bool	user	true	Enable job array support.
xtNetBasedShieldCheckNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtNetBasedShieldCheckNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtNetBasedShieldCheckNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtNetBasedShieldCheckNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtNetBasedShieldCheckNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtNetBasedShieldCheckNetworkComputerSelectedJobClass	string	user	<None>	NetworkComputer selected job class
xtNetBasedShieldCheckNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtNetBasedShieldCheckNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtNetBasedShieldCheckNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtNetBasedShieldCheckNoDistTolerance	bool	custom	false	The no distance tolerance option of net based checker
xtNetBasedShieldCheckPinTextMap	string	custom	Technology Library	The pin text map mode of net based checker

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetBasedShieldCheckPinTextMapFile	string	custom		The pin text map file of net based checker
xtNetBasedShieldCheckPrefixes	string	user		Job class prefixes
xtNetBasedShieldCheckPrependCommand	string	user		String to prepend to job shell command line
xtNetBasedShieldCheckPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtNetBasedShieldCheckRefData	string	custom		The SVDB reference data of net based checker.
xtNetBasedShieldCheckRunDir	string	custom	netBasedShieldCheck	The run directory of net based checker.
xtNetBasedShieldCheckRunInBackground	bool	custom	true	Run net based shield check in background
xtNetBasedShieldCheckSGEEnableJobArrays	bool	user	true	Enable job array support
xtNetBasedShieldCheckSGEExportEnvironment	bool	user	true	Export qsub environment to child job
xtNetBasedShieldCheckSGEExtraOptions	string	user		Extra command line options for SGE job launch
xtNetBasedShieldCheckSGEMaxArraySize	int	user	1000	Max job array size
xtNetBasedShieldCheckSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtNetBasedShieldCheckSGEProjects	string	user	<None>	SGE projects.
xtNetBasedShieldCheckSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtNetBasedShieldCheckSGESelectedProject	string	user		Selected SGE project

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetBasedShieldCheckSGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtNetBasedShieldCheckSGESubmitHost	string	user	localhost	Submit host name for remote grid launch
xtNetBasedShieldCheckSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtNetBasedShieldCheckSLURMEnableJobArrays	bool	user	true	Enable job array support
xtNetBasedShieldCheckSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtNetBasedShieldCheckSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtNetBasedShieldCheckSLURMMaxArraySize	int	user	1000	Max job array size
xtNetBasedShieldCheckSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtNetBasedShieldCheckSLURMPartitions	string	user	<None>	SLURM partitions.
xtNetBasedShieldCheckSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtNetBasedShieldCheckSLURMSelectedPartition	string	user		Selected SLURM partition
xtNetBasedShieldCheckSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtNetBasedShieldCheckSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtNetBasedShieldCheckSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetBasedShieldCheckShieldRatio	bool	custom	true	The shield ration option of net based checker
xtNetBasedShieldCheckShieldRatioValue	string	custom	100	The value of shield ration
xtNetBasedShieldCheckTool	string	custom	IC Validator	The tool of net based checker
xtNetBasedShieldCheckViewOutput	bool	custom	false	The view output option of net based checker.
xtNetBasedViaCheckConstraintFile	string	custom		The net constraint file of net based via checker.
xtNetBasedViaCheckJobClass	string	custom	NetBasedCheck	The job class for running net based via check.
xtNetBasedViaCheckLayStreamFile	string	custom		The stream file of design to run net based via check
xtNetBasedViaCheckDefaultRatio	string	custom	0.8	The default of via ratio to run net based via check
xtNetBasedViaCheckExportStream	bool	custom	false	Enable this option to export stream
xtNetBasedViaCheckPinTextMap	string	custom	Technology Library	The pin text map mode of net based via checker.
xtNetBasedViaCheckPinTextMapFile	string	custom		The pin text map file of net based via checker.
xtNetBasedViaCheckRunDir	string	custom	windowDRC	The run directory of net based via checker.
xtNetBasedViaCheckTool	string	custom	IC Validator	The tool of net based via checker.
xtNetBasedViaCheckViewOutput	bool	custom	false	The view output option of net based via checker. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetBasedViaCheckWidthValue	string	custom	0	The width value of net based via checker.
xtNetlistingHostname	string	user	localhost	Specifies the remote hostname.
xtNetlistingLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtNetlistingLSFRemoteSubmit	bool	user	false	Specifies if the submit host is on remote site or not. Valid values: true, false
xtNetlistingLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch.
xtNetlistingLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtNetlistingNetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtNetlistingNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtNetlistingNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtNetlistingNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtNetlistingNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtNetlistingNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtNetlistingNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetlistingNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtNetlistingNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtNetlistingNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtNetlistingPrependCommand	string	user		String to prepend to job shell command line
xtNetlistingPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtNetlistingSGEExportEnvironment	bool	user	true	Export qsub environment to child job. Valid values: true, false
xtNetlistingSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtNetlistingSGEProjects	string	user		SGE projects.
xtNetlistingSGERemoteSubmit	bool	user	false	Specifies if the submit host is on remote site or not. Valid values: true, false
xtNetlistingSGESubmitHost	string	user	localhost	Submit host name for remote grid launch.
xtNetlistingSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtNetlistingSLURMEnableJobArrays	bool	user	true	Enable job array support

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetlistingSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtNetlistingSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtNetlistingSLURMMaxArraySize	int	user	1000	Max job array size
xtNetlistingSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtNetlistingSLURMPartitions	string	user	<None>	SLURM partitions.
xtNetlistingSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtNetlistingSLURMSelectedPartition	string	user		Selected SLURM partition
xtNetlistingSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtNetlistingSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtNetlistingSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtNetsReducerCaseSensitive	string	user	NO	Case sensitive
xtNetsReducerInputFile	string	user		Input file
xtNetsReducerNetlists	bool	user	false	Reduced netlist
xtNetsReducerOutputFile	string	user		Output file
xtNetworkComputerAutoDetect	bool	user	true	Auto detect NetworkComputer settings from environment Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtNetworkComputerDebug	bool	user	false	Display debugging information for NetworkComputer Valid values: true, false
xtNetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class
xtNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPacOpenLegalizationSummaryFile	bool	user	false	Indicates whether need to open Placement Legalization summary file after Pin Access Check is completed.
xtPacOpenRoutabilitySummaryFile	bool	user	false	Indicates whether need to open Routability summary file after Pin Access Check is completed.
xtPacSingleFileLibraryPath	string	user		Specifies reference libraries path for Pin Access Check Library.
xtPDKProcess	string	user	true	PDK Process name as defined in a PDK Configuration file.
xtPDKVariant	string	user		PDK Variant name as defined in a PDK Configuration file.
xtPERCAutoPrefs	bool	custom	true	Auto-load prefs.xml Valid values: true, false
xtPERCCalibreAutoMatch	bool	custom	false	Use cell name to determine cell correspondence between layout and schematic. Valid values: true, false
xtPERCCalibreEchoSVRF	bool	custom	false	Echo compiled SVRF commands and included files in the Calibre PERC job output. Valid values: true, false
xtPERCCalibreGenerateCCI	bool	custom	true	Generate CCI files in PERC SVDB results database. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPERCCalibreHCellsFile	string	custom		Location of file containing cell correspondences between layout and schematic.
xtPERCCalibreHierarchicalRunMode	bool	custom	true	Enables 'PERC' hierarchical mode of operation. Valid values: true, false
xtPERCCalibreLayInputNetlistFormat	string	custom	SPICE	Layout netlist format
xtPERCCalibreMaskSVDBDir	string	custom		Location of PERC SVDB results database.
xtPERCCalibreOptions	string	custom		Command line options for 'Calibre' 'PERC' jobs.
xtPERCCalibreOverrideRunsetFile	bool	custom	false	Override SVRF commands in Calibre PERC primary runset file. Valid values: true, false
xtPERCCalibreReport	string	custom		Location of the PERC report.
xtPERCCalibreRunsetFile	string	custom		Calibre PERC primary runset file.
xtPERCCalibreRunsetOptions	string	custom		List of PERC {option value} options for 'Calibre' Interactive.
xtPERCCalibreTranscript	string	custom	stdout.perc.log	Output file name for the Calibre PERC Transcript.
xtPERCExportOasis	bool	custom	true	Export OA layout to Oasis Valid values: true, false
xtPERCExportStream	bool	custom	true	Export OA layout to stream Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPERCHostname	string	user	localhost	Remote Hostname
xtPERCJobLimit	int	user	3	Job limit
xtPERCJobLimitEnabled	bool	user	true	Job limit disabled (limited) Valid values: true, false
xtPERCJobQueueEngine	string	user		Queue Engine
xtPERCLaunchDebugger	bool	custom	true	Launch Debugger Valid values: true, false
xtPERCLayDBFormat	string	custom	OpenAccess	Database format of the layout
xtPERCLayInputNetlist	string	custom		Name of the input layout netlist file
xtPERCLayLCV	lcv	custom	<file path>	Layout LCV
xtPERCLayOasisFile	string	custom		Layout Oasis file
xtPERCLayStreamFile	string	custom		Layout stream file
xtPERCLSFFEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtPERCLSFFExtraOptions	string	user		Extra command line options for LSF job launch
xtPERCLSFFMaxArraySize	int	user	1000	Max job array size
xtPERCLSFFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtPERCLSFFQueues	string	user		LSF Queues
xtPERCLSFFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtPERCLSFFResources	string	user		LSF Resource types

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPERCLSFSSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtPERCLSFSSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtPERCLSFuseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtPERCNetlistTopAsModule	bool	custom	true	Create top level module definition. Valid values: true, false
xtPERCNetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtPERCNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtPERCNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtPERCNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtPERCNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtPERCNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtPERCNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class
xtPERCNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPERCNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtPERCNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtPERCPLEFlow	bool	custom	false	Partial layout extraction flow Valid values: true, false
xtPERCPrefixes	string	user		Job class prefixes
xtPERCPrepndCommand	string	user		String to prepend to job shell command line
xtPERCRunDir	string	custom		Run directory
xtPERCRunDirNameTemplate	string	custom	%c.%t.%j	Template for customizing run directory name
xtPERCSchCell	string	custom		Schematic cell
xtPERCSchDBFormat	string	custom	OpenAccess	Database format of the schematic
xtPERCSchInputNetlist	string	custom		Name of the input schematic netlist file
xtPERCSchLCV	lcv	custom	<file path>	Schematic LCV
xtPERCSchNetlister	string	custom	CDL	Netlister for creating the schematic netlist
xtPERCSchOutputNetlist	string	custom		Name of the output schematic netlist file
xtPERCSGEEEnableJobArrays	bool	user	true	Enable job array support Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPERCSGEEExportEnvironment	bool	user	true	Export qsub environment to child job Valid values: true, false
xtPERCSGEEExtraOptions	string	user		Extra command line options for SGE job launch
xtPERCSGEMaxArraySize	int	user	1000	Max job array size
xtPERCSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtPERCSGEPjects	string	user		SGE projects.
xtPERCSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtPERCSGEGSelectedProject	string	user		Selected SGE project
xtPERCSGEGSetupCommand	string	user		Shell command to run on remote host to configure SGE
xtPERCSGESubmitHost	string	user	localhost	Submit host name for remote grid launch
xtPERCSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtPERCTool	string	custom	Calibre	Tool
xtPEXAutoPrefs	bool	custom	true	Auto-load prefs.xml Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPEXCalibreEchoSVRF	bool	custom	false	Echo compiled SVRF commands and included files in the Calibre PEX job output. Valid values: true, false
xtPEXCalibreHierarchicalRunMode	bool	custom	true	Enables 'PEX' hierarchical mode of operation. Valid values: true, false
xtPEXCalibreOverrideRunsetFile	bool	custom	false	Override SVRF commands in Calibre PEX primary runset file. Valid values: true, false
xtPEXCalibreRunsetFile	string	custom		Calibre PEX primary runset file.
xtPEXCalibreRunsetOptions	string	custom		List of PEX {option value} options for 'Calibre' Interactive.
xtPEXCalibreTranscript	string	custom	stdout.pex.log	Output file name for the Calibre PEX Transcript.
xtPEXHostname	string	user	localhost	Remote Hostname
xtPEXJobLimit	int	user	3	Job limit. Valid values: Any integer in the [1 1000000] range.
xtPEXJobLimitEnabled	bool	user	true	Job limit disabled (limited) Valid values: true, false
xtPEXJobQueueEngine	string	user		Queue Engine
xtPEXLSFEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtPEXLSFExtraOptions	string	user		Extra command line options for LSF job launch

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPEXLSFMaxArraySize	int	user	1000	Max job array size
xtPEXLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtPEXLSFQueues	string	user		LSF Queues
xtPEXLSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtPEXLSFResources	string	user		LSF Resource types
xtPEXLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtPEXLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtPEXLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtPEXNetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtPEXNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtPEXNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtPEXNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtPEXNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtPEXNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPEXNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class
xtPEXNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtPEXNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtPEXNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtPEXPLEFlow	bool	custom	false	Partial layout extraction flow Valid values: true, false
xtPEXPrefixes	string	user		Job class prefixes
xtPEXPrependCommand	string	user		String to prepend to job shell command line
xtPEXSGEEnableJobArrays	bool	user	true	Enable job array support Valid values: true, false
xtPEXRunDir	string	custom		Run directory
xtPEXSGEExportEnvironment	bool	user	true	Export qsub environment to child job Valid values: true, false
xtPEXSGEExtraOptions	string	user		Extra command line options for SGE job launch
xtPEXSGEMaxArraySize	int	user	1000	Max job array size
xtPEXSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtPEXSGEProjects	string	user		SGE projects.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPEXSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtPEXSGESelectedProject	string	user		Selected SGE project
xtPEXSGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtPEXSGESubmitHost	string	user	localhost	Submit host name for remote grid launch
xtPEXSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtPLEHostname	string	user	localhost	Remote Hostname
xtPLEJobLimit	int	user	3	Job limit
xtPLEJobLimitEnabled	bool	user	true	Job limit disabled (limited)
xtPLEJobQueueEngine	string	user		Queue Engine
xtPLELSFEnableJobArrays	bool	user	true	Enable job array support.
xtPLELSFExtraOptions	string	user		Extra command line options for LSF job launch
xtPLELSFMaxArraySize	int	user	1000	Max job array size
xtPLELSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtPLELSFQueues	string	user		LSF Queues
xtPLELSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtPLELSFResources	string	user		LSF Resource types

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPLELSFSetupCommand	string	user		Shell command to run on remote host to configure LSF
xtPLELSFSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtPLELSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtPLENetworkComputerEnableJobArrays	bool	user	true	Enable job array support.
xtPLENetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtPLENetworkComputerJobClasses	string	user		NetworkComputer job classes
xtPLENetworkComputerMaxArraySize	int	user	1000	Max job array size
xtPLENetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtPLENetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtPLENetworkComputerSelectedJobClass	string	user	<None>	NetworkComputer selected job class
xtPLENetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtPLENetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtPLENetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtPLEPrefixes	string	user	ple_netlisting ple_postProcess	Job class prefixes

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPLEPrependCommand	string	user		String to prepend to job shell command line
xtPLEPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtPLESGEEnableJobArrays	bool	user	true	Enable job array support
xtPLESGEExportEnvironment	bool	user	true	Export qsub environment to child job
xtPLESGEExtraOptions	string	user		Extra command line options for SGE job launch
xtPLESGEMaxArraySize	int	user	1000	Max job array size
xtPLESGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtPLESGEProjects	string	user	<None>	SGE projects.
xtPLESGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtPLESGESelectedProject	string	user		Selected SGE project
xtPLESGESetupCommand	string	user		Shell command to run on remote host to configure SGE
xtPLESGESubmitHost	string	user	localhost	Submit host name for remote grid launch
xtPLESGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtPLESLURMEnableJobArrays	bool	user	true	Enable job array support

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPLESLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtPLESLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtPLESLURMMaxArraySize	int	user	1000	Max job array size
xtPLESLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtPLESLURMPartitions	string	user	<None>	SLURM partitions.
xtPLESLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtPLESLURMSelectedPartition	string	user		Selected SLURM partition
xtPLESLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtPLESLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtPLESLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtPlotAddToPlotSet	bool	cellview	false	Plot should be added to current plot set. Valid values: true, false
xtPlotAnalysis	string	cellview		Analysis to plot.
xtPlotCommand	string	cellview	Append	How to display waveforms (new, append, replace).
xtPrependCommand	string	ser		tring to prepend to job shell command line

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtPVLayrMapFile	string	user		Mapping file that cross references LPPs and stream layer-datatypes.
xtPWRerunUnexpectedExitJobs	string	user	false	Enable the PrimeWave rerun jobs upon unexpected exit.
xtPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtPWSysJobTimeLimit	string	user		PrimeWave set system job run time limit, unit is hour, empty means no limit.
xtQueryParasiticsOptionToolbarAutoName	string	context		Query Parasitics COT Net/Terminal field.
xtQueryParasiticsOptionToolbarNet1Name	string	context		Query Parasitics COT Net1 Name field
xtQueryParasiticsOptionToolbarNet2Name	string	context		Query Parasitics COT Net2 Name field
xtQueryParasiticsOptionToolbarNetName	string	context		Query Parasitics COT Net Name field
xtQueryParasiticsOptionToolbarTerm1Name	string	context		Query Parasitics COT Terminal1 Name field
xtQueryParasiticsOptionToolbarTerm2Name	string	context		Query Parasitics COT Terminal2 Name field
xtQueryParasiticsOptionToolbarType	string	context	auto	
xtQueueEngine	string	user	true	The default queue engine to use when launching batch jobs.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtQueueUpdateInterval	string	user	2	The number of seconds to wait between calls to the integrator callback to query queue engines for job status.
xtRcpPath	string	user	<file path>	Path to rcp binary.
xtRshPath	string	user	<file path>	Path to rsh binary.
xtRsyncPath	string	user	<file path>	Path to rsync binary.
xtRunCreateFrame	bool	user	true	Indicates whether need to run Create Frame.
xtRunDirBase	string	user	./	The template with which to construct the run directory for all PVE job types.
xtRunMultiplePveJobsHostname	string	user	localhost	Remote Hostname.
xtRunMultiplePveJobsJobLimit	int	user	3	Job limit.
xtRunMultiplePveJobsJobLimitEnabled	bool	user	true	Job limit disabled (limited). Valid values: true, false
xtRunMultiplePveJobsJobQueueEngine	string	user		Queue Engine.
xtRunMultiplePveJobsLSFEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtRunMultiplePveJobsLSFExtraOptions	string	user		Extra command line options for LSF job launch.
xtRunMultiplePveJobsLSFMaxArraySize	int	user	1000	Max job array size.
xtRunMultiplePveJobsLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtRunMultiplePveJobsLSFQueues	string	user		LSF Queues.

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtRunMultiplePveJobsLSFRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not. Valid values: true, false
xtRunMultiplePveJobsLSFResources	string	user		LSF Resource types.
xtRunMultiplePveJobsLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF.
xtRunMultiplePveJobsLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch.
xtRunMultiplePveJobsLSFUseNetcat	bool	user	false	Uses netcat command line utility to send status updates. Valid values: true, false
xtRunMultiplePveJobsNetworkComputerEnableJobArrays	bool	user	true	Enables job array support. Valid values: true, false
xtRunMultiplePveJobsNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch.
xtRunMultiplePveJobsNetworkComputerJobClasses	string	user		NetworkComputer job classes.
xtRunMultiplePveJobsNetworkComputerMaxArraySize	int	user	1000	Max job array size.
xtRunMultiplePveJobsNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtRunMultiplePveJobsNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not. Valid values: true, false
xtRunMultiplePveJobsNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class.

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtRunMultiplePveJobsNetworkComputerSetup Command	string	user		Shell command to run on remote host to configure NetworkComputer.
xtRunMultiplePveJobsNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch.
xtRunMultiplePveJobsNetworkComputerUseNetcat	bool	user	false	Uses netcat command line utility to send status updates. Valid values: true, false
xtRunMultiplePveJobsPrefixes	string	user		Job class prefixes.
xtRunMultiplePveJobsPrependCommand	string	user		String to prepend to job shell command line.
xtRunMultiplePveJobsPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtRunMultiplePveJobsSGEEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtRunMultiplePveJobsSGEExportEnvironment	bool	user	true	Exports qsub environment to child job. Valid values: true, false
xtRunMultiplePveJobsSGEExtraOptions	string	user		Extra command line options for SGE job launch.
xtRunMultiplePveJobsSGEMaxArraySize	int	user	1000	Max job array size
xtRunMultiplePveJobsSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtRunMultiplePveJobsSGEProjects	string	user		SGE projects.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtRunMultiplePveJobsSGERemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not. Valid values: true, false
xtRunMultiplePveJobsSGESelectedProject	string	user		Selected SGE project.
xtRunMultiplePveJobsSGESetupCommand	string	user		Shell command to run on remote host to configure SGE.
xtRunMultiplePveJobsSGESubmitHost	string	user	localhost	Submit host name for remote grid launch.
xtRunMultiplePveJobsSGEUseNetcat	bool	user	false	Uses netcat command line utility to send status updates. Valid values: true, false
xtRunMultiplePveJobsSLURMEnableJobArrays	bool	user	true	Enable job array support
xtRunMultiplePveJobsSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtRunMultiplePveJobsSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtRunMultiplePveJobsSLURMMaxArraySize	int	user	1000	Max job array size
xtRunMultiplePveJobsSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtRunMultiplePveJobsSLURMPartitions	string	user	<None>	SLURM partitions.
xtRunMultiplePveJobsSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtRunMultiplePveJobsSLURMSelectedPartition	string	user		Selected SLURM partition
xtRunMultiplePveJobsSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtRunMultiplePveJobsSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtRunMultiplePveJobsSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtRunNetlister	bool	rundir	true	Boolean used to seed the Netlist toggle on the Netlist and Simulate dialog. Valid values: true, false
xtRunSimulation	bool	rundir	true	Boolean used to seed the Simulate toggle on the Netlist and Simulate dialog. Valid values: true, false
xtSAEShellLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtSAEShellLSFRemoteSubmit	bool	user	false	Specifies if the submit host is on a remote site or not. Valid values: true, false
xtSAEShellLSFSubmitHost	string	user	localhost	Submits the host name for remote grid launch.
xtSAEShellLSFUseNetcat	bool	user	false	Uses the netcat command-line utility to send status updates. Valid values: true, false
xtSAEShellNetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtSAEShellNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtSAEShellNetworkComputerJobClasses	tring	user		NetworkComputer job classes

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtSAEShellNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtSAEShellNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtSAEShellNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false
xtSAEShellNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class
xtSAEShellNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtSAEShellNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtSAEShellNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtSAEShellPrependCommand	string	user		String to prepend to job shell command line
xtSAEShellPWRerunUnexpectedExitJobs	string	user	false	Enable the PrimeWave rerun jobs upon unexpected exit.
xtSAEShellPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtSAEShellPWSYSTEMJobTimeLimit	string	user		PrimeWave set system job run time limit, unit is hour, empty means no limit.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtSAEShellSGEExportEnvironment	bool	user	true	Exports qsub environment to child job. Valid values: true, false
xtSAEShellSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtSAEShellSGEProjects	string	user		SGE projects.
xtSAEShellSGERemoteSubmit	bool	user	false	Specifies if the submit host is on a remote site or not. Valid values: true, false
xtSAEShellSGESubmitHost	string	user	localhost	Submits the host name for remote grid launch.
xtSAEShellSGEUseNetcat	bool	user	false	Uses the netcat command-line utility to send status updates. Valid values: true, false
xtSAEShellSLURMEnableJobArrays	bool	user	true	Enable job array support
xtSAEShellSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtSAEShellSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtSAEShellSLURMMaxArraySize	int	user	1000	Max job array size
xtSAEShellSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtSAEShellSLURMPartitions	string	user	<None>	SLURM partitions.
xtSAEShellSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtSAEShellSLURMSelectedPartition	string	user		Selected SLURM partition

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtSAEShellSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtSAEShellSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtSAEShellSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtSGEAutoDetect	bool	user	true	Automatically detect settings for SGE from environment. Valid values: true, false
xtSGEDebug	bool	user	false	Display debugging information for SGE. Valid values: true, false
xtSGEExportEnvironment	bool	user	true	Export qsub environment to child job. Valid values: true, false
xtSGEExtraOptions	string	user		Extra command line options for SGE job launch.
xtSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtSGEProjects	string	user		SGE projects.
xtSGERemoteSubmit	bool	user	false	Specifies if the submit host is on remote site or not. Valid values: true, false
xtSGESelectedProject	string	user		Selected SGE project.
xtSGESubmitHost	string	user	localhost	Submit host name for remote grid launch.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtShowJobKillConfirmation	bool	user	true	Show Job Kill Confirmation Dialog or not
xtSimRunDirBase	string	user	The current directory: i.e. ""./	Default base directory to select when choosing a run directory.
xtSimulationHostname	string	user	localhost	Specifies the remote hostname.
xtSimulationJobLimitEnabled	bool	user	true	Controls whether or not to limit the number of concurrent jobs. Valid values: true, false
xtSimulationJobMode	string	rundir	background	Controls if simulations are launched in the # foreground or background. Valid values: foreground, background
xtSimulationJobQueueEngine	string	user	""	Specifies which distributed processing to use. If you use a "" value, no distributing process will be used. Valid values: SGE, LSF, ""
xtSimulationLSFEnableJobArrays	bool	user	false	Enables job array support.
xtSimulationLSFExtraOptions	string	user		Extra command line options for LSF job launch.
xtSimulationLSFMaxArraySize	int	user	1000	Sets the maximum job array size.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtSimulationLSFNetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtSimulationLSFQueues	string	user		LSF Queues.
xtSimulationLSFRemoteSubmit	bool	user	false	Specifies if the submit host is on remote site or not. Valid values: true, false
xtSimulationLSFResources	string	user		LSF Resource types.
xtSimulationLSFSetupCommand	string	user		Shell command to run on remote host to configure LSF.
xtSimulationLSFSubmitHost	string	user	localhost	Submit host name for remote grid launch.
xtSimulationLSFUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtSimulationNetworkComputerEnableJobArrays	bool	user	true	Enable job array support. Valid values: true, false
xtSimulationNetworkComputerExtraOptions	string	user		Extra command line options for NetworkComputer 'nc run' job launch
xtSimulationNetworkComputerJobClasses	string	user		NetworkComputer job classes
xtSimulationNetworkComputerMaxArraySize	int	user	1000	Max job array size
xtSimulationNetworkComputerNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtSimulationNetworkComputerRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtSimulationNetworkComputerSelectedJobClass	string	user		NetworkComputer selected job class
xtSimulationNetworkComputerSetupCommand	string	user		Shell command to run on remote host to configure NetworkComputer
xtSimulationNetworkComputerSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtSimulationNetworkComputerUseNetcat	bool	user	false	Use netcat command line utility to send status updates Valid values: true, false
xtSimulationPrefixes	string	user	HSIM HSPICE Nanosim Spectre VHDL Verilog XA FineSimPro FineSimProVCS HSIMHSPICE PrimeSim PrimeSimVCS PrimeSimXA PrimeSimXAVC Sxt XAEldo XAHSPICE XAVCSHSPICE	Job class prefixes.
xtSimulationPrependCommand	string	user		String to prepend to job shell command line
xtSimulationPWRerunUnexpectedExitJobs	string	user	false	Enable the PrimeWave rerun jobs upon unexpected exit.
xtSimulationPWSimPostProcEnabled	string	user	false	Enable the PrimeWave Post Process after simulation done in PW worker side
xtSimulationPWSystemJobTimeLimit	string	user		PrimeWave set system job run time limit, unit is hour, empty means no limit.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtSimulationRunHost	string	rundir	localhost	Default host on which to run simulations.
xtSimulationSGEExportEnvironment	bool	user	true	Export qsub environment to child job. Valid values: true, false
xtSimulationSGEExtraOptions	string	user		Specifies extra command line options for SGE job launch.
xtSimulationSGENetcatPath	string	user	<file path>	Path to the netcat (nc) binary.
xtSimulationSGEProjects	string	user		SGE projects.
xtSimulationSGERemoteSubmit	bool	user	false	Specifies if the submit host is on remote site or not. Valid values: true, false
xtSimulationSGESetupCommand	string	user		Specifies Shell command to run on remote host to configure SGE.
xtSimulationSGESubmitHost	string	user	localhost	Submit host name for remote grid launch.
xtSimulationSGEUseNetcat	bool	user	false	Use netcat command line utility to send status updates. Valid values: true, false
xtSimulationSLURMEnableJobArrays	bool	user	true	Enable job array support
xtSimulationSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtSimulationSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtSimulationSLURMMaxArraySize	int	user	1000	Max job array size

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtSimulationSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtSimulationSLURMPartitions	string	user	<None>	SLURM partitions.
xtSimulationSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtSimulationSLURMSelectedPartition	string	user		Selected SLURM partition
xtSimulationSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtSimulationSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtSimulationSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtSLURMAutoDetect	bool	user	true	Auto detect settings for SLURM from environment
xtSLURMCCDisableLoad	bool	user	false	Disable loading and creating SLURM queue engine from the Custom Compiler environment.
xtSLURMDebug	bool	user	false	Display debugging information for SLURM
xtSLURMEnableJobArrays	bool	user	true	Enable job array support
xtSLURMExportEnvironment	bool	user	true	Export batch environment to child job
xtSLURMExtraOptions	string	user		Extra command line options for SLURM job launch
xtSLURMMaxArraySize	int	user	1000	Max job array size

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtSLURMNetcatPath	string	user	<file path>	Path to the netcat (nc) binary
xtSLURMPartitions	string	user	<None>	SLURM partitions.
xtSLURMRemoteSubmit	bool	user	false	Specifies if the submit host on remote site or not
xtSLURMSelectedPartition	string	user		Selected SLURM partition
xtSLURMSetupCommand	string	user		Shell command to run on remote host to configure SLURM
xtSLURMSubmitHost	string	user	localhost	Submit host name for remote grid launch
xtSLURMUseNetcat	bool	user	false	Use netcat command line utility to send status updates
xtSpectreDefaultNetlister	string	user	Spectre	The default netlister to choose when generating a netlist.
xtSpectreDefaultViewer	string	user	WaveView	Default waveform viewer for plotting Spectre results.
xtSpectreGenerateSubcktInTopModule	string	user	false	
xtSpectreGlobalSearchList	string	custom	spectre cmos.sch cmos_sch schematic veriloga	Global search list passed to the netlister.
xtSpectreGlobalStopList	string	custom	spectre veriloga	Global stop list passed to the netlister.
xtSpectreJobLimit	int	user	3	Job limit.
xtSpectreJobLimitEnabled	bool	user	true	Job limit disabled (limited).
xtStarBypassManifest	string	custom		Bypass StarRC manifest features.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtStarCalibreQueryFileSuffix	string	custom	_cmd .cmd	Query Server Command File name filters.
xtStarCalibreRunset	string	rundir	localhost	Location of Calibre-LVS rules file used during layout extraction. Passed to Star-RCXT via the CALIBRE_RUNSET command.
xtStarICVRunsetReportFileName	string	custom	pex_runset_report	ICV runset report file name.
xtStarMilkywayDatabase	string	custom		StarRC MILKYWAY_DATA BASE command.
xtStarMilkywayExtractView	bool	custom	true	StarRC MILKYWAY_EXTRACT_VIEW command. Valid values: true, false
xtStarPreCmdStr	string	custom		Driver command
xtStarRunsetFileSuffix	string	custom	_cmd .cmd	File extensions for the runset for the 'StarRC' tool.
xtStarXtorGpd	bool	custom	false	StarRC XTOR_GPD command Valid values: true, false
xtSyncOnQueueJobCompletion	bool	user	true	For queue managed jobs. Run the UNIX sync shell command on the execution host after the command finishes. This will force any output files to be written to network drives. Valid values: true, false

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtTextViewerFind	bool	user	false	Show / Hide text viewer find bar. Valid values: true, false
xtTextViewerLineNum	bool	user	false	Show / Hide text viewer find bar.
xtTextViewerPoll	bool	user	true	Toggle text viewer file polling. Valid values: true, false
xtValidateForDigitalImplTool	bool	user	false	Validates design before exporting.
xtValidateInstPlacement	bool	user	true	Defines whether instances inside PR boundary should be checked to have placement status. Valid values: true, false
xtValidateInstTermConnectivity	bool	user	true	Defines whether instance terminals should be checked to have connectivity. Valid values: true, false
xtValidateMarkerSize	float	user	0.1	Size of the global markers that are placed at the bottom-left corner of the design.
xtValidateNetSignalType	bool	user	true	Defines whether nets matching power/ground name convention should be checked to have appropriate signal type. Valid values: true, false
xtValidatePinPlacement	bool	user	true	Defines whether pins should be checked to have placement status. Valid values: true, false

Table 228 *Integrated Tool Preferences (Continued)*

Name	Type	Scope	Default	Description
xtValidateShapeConnectivity	bool	user	true	Defines whether shapes should be checked to have connectivity. Valid values: true, false
xtVerilogDefaultNetlister	string	user	Verilog	The default netlister to choose when generating a netlist.
xtVerilogGlobalSearchList	string	custom	verilog functional behavioral schematic cmos.sch cmos_sch symbol	Global search list passed to the netlister.
xtVerilogGlobalStopList	string	custom	verilog functional behavioral symbol	Global stop list passed to the netlister.
xtVerilogICCGlobalSearchList	string	custom	verilog functional behavioral cmos.sch cmos_sch schematic symbol	Global search list passed to the ICC netlister.
xtVerilogICCGlobalStopList	string	custom	functional behavioral symbol	Global stop list passed to the ICC netlister.
xtXADefaultNetlister	string	user	HSPICE	The name of the default netlister to choose when generating a netlist.
xtXADefaultViewer	string	user	WaveView	Default waveform viewer for plotting signals.
xtXAGlobalSearchList	string	custom	hspice hspiceD cmos.sch cmos_sch schematic veriloga	Global search list passed to the netlister.
xtXAGlobalStopList	string	custom	hspice hspiceD veriloga	Global stop list passed to the netlister.
xtXAHSPICEJobLimit	int	user	3	Job limit.
xtXAHSPICEJobLimitEnabled	bool	user	true	Job limit disabled (limited).

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtXASpectreJobLimit	int	user	3	Job limit.
xtXASpectreJobLimitEnabled	bool	user	true	Job limit disabled (limited).
xtZRTRouteCheckAntenna	bool	user	true	Determines if a design is checked for charge-collecting antenna rule violations. Valid values: true, false
xtZRTRouteCheckDRC	bool	user	true	Determines if a design is checked for design rule violations. Valid values: true, false
xtZRTRouteCheckFShapes	bool	user	true	Determines if a design is checked for DRC violations for frozen shapes, as well as routing shapes. Valid values: true, false
xtZRTRouteCheckOpens	bool	user	true	Determines if a design is checked for open nets. Valid values: true, false
xtZRTRouteCheckUShapes	bool	user	true	Determines if a design is checked for DRC violations for user-created shapes, as well as routing shapes. Valid values: true, false
xtZRTRouteCheckVArea	bool	user	true	Determines if a design is checked for voltage-area rule violations. Valid values: true, false
xtZRTRouteErrorSource	string	user	current	Specifies if one of the error views or the current view is used to import violations. The value is either current or error.

Table 228 Integrated Tool Preferences (Continued)

Name	Type	Scope	Default	Description
xtZRTRouteErrorView	string	user		Specifies the cell name error view that is used to import violations.
xtZRTRouteNets	string	user		Determines collection of nets to check for opens, antenna rules, voltage area violations.
xtZRTRouteReportOpens	bool	user	true	Determines if all open nets are reported. If false, a maximum of 200 open nets are reported. Valid values: true, false
xtZRTRouteRun	bool	user	true	Defines whether verify_zrt_route is run before the violations import. Valid values: true, false

29

StarRC Tcl

This chapter lists commands and preferences in the Custom Compiler Tcl rc namespace. The namespace provide Tcl to support the use of StarRC commands for parasitic resistance and capacitance analyses within the Custom Compiler environment.

For details, see:

- [Commands](#)
- [Preferences](#)

Commands

The commands covered are:

- [rc::getCells](#)
- [rc::getCouplingCapacitors](#)
- [rc::getElmoreDelay](#)
- [rc::getGPDCorners](#)
- [rc::getGPDGroundCapacitors](#)
- [rc::getGPDLayers](#)
- [rc::getGroundCapacitors](#)
- [rc::getNets](#)
- [rc::getPins](#)
- [rc::getPointToPointResistance](#)
- [rc::getPorts](#)
- [rc::getResistors](#)
- [rc::readParasitics](#)

- [rc::removeAnnotatedParasitics](#)
 - [rc::reportGPDConfig](#)
 - [rc::reportGPDProperties](#)
 - [rc::resetGPDConfig](#)
 - [rc::setGPDConfig](#)
-

rc::getCells

Description

Returns a container of rcCell objects for the user-specified query.

Syntax

```
rc::getCells [-exact <bool>] [-quiet <bool>] [-ofObjects <dbCollection |  
string>] [-pattern <string>]
```

Arguments

`-exact`

Disables the simple pattern matching for querying cells.

`-quiet`

Suppresses warning and error messages if no objects match.

`-ofObjects`

Creates a collection of cells from the specified objects.

`-pattern`

Specifies the pattern for querying cells.

Returns

`<dbCollection>` : Cells match with the specified query.

rc::getCouplingCapacitors

Description

Returns a set of rcCouplingCapacitors for the given nets or nodes on the command line.

Note:

In SMC mode, either the `-allCorners` or `-corners` argument is mandatory input for this command. Custom Compiler provides a warning message in the console when these arguments are missing.

Syntax

```
rc::getCouplingCapacitors [-of <dbCollection | string>] [-from <string>]  
[-to <string>] [-corners <string>] [-allCorners <bool>] [-filter  
<string>]
```

Arguments

`-of`

Reports coupling capacitors associated with the specified nets.

`-from`

Constrains the selected capacitors to be part of a path starting from the specified node.

`-to`

Constrains the selected capacitors to be part of a path ending with the specified node.

`-corners`

Indicates the parasitic corner ordering for coupling capacitance values.

`-allCorners`

Queries resistance values for all corners.

`-filter`

Refines the returned capacitors with standard attribute filter.

Returns

<dbCollection> : Coupling capacitors for the given nets or nodes.

rc::getElmoreDelay

Description

Returns Elmore delay for the given nodes on the command line.

Syntax

```
rc::getElmoreDelay [-of <string>] [-to <string>] [-corners <string>]  
[-allCorners <bool>]
```

Arguments

`-of`

Constrains the selected resistors to be part of a path starting from the specified node.

`-to`

Constrains the selected resistors to be part of a path ending with the specified node.

`-corners`

Specifies the corners to query resistance values.

`-allCorners`

Queries resistance values for all corners.

Returns

`<dbCollection>` : Resistors for the given nets or nodes.

rc::getGPDCorners

Description

Returns a string of corner analysis simulations for the specified Galaxy Parasitic Database (GPD).

Syntax

```
rc::getGPDCorners -gpd <string>
```

Arguments

`-gpd`

Specifies the path to the GPD directory.

Returns

`<string>` : Corner analysis simulations for the specified GPD.

rc::getGPDGroundCapacitors

Description

Returns a list of ground capacitors for the specified nets or nodes on the command line.

Syntax

```
rc::getGPDGroundCapacitors [-of <string>] [-from <string>] [-to <string>]  
[-corners <string>]
```

Arguments

`-of`

Reports ground capacitors associated with the specified nets.

`-from`

Constrains the selected capacitors to be part of a path starting from the specified node.

`-to`

Constrains the selected capacitors to be part of a path ending with the specified node.

`-corners`

Indicates the order of parasitic corner analyses for ground capacitance values.

Returns

`<string>` : Ground capacitors for the specified nets or nodes.

rc::getGPDLayers

Description

Returns a string of layers for the specified Galaxy Parasitic Database (GPD).

Syntax

```
rc::getGPDLayers -gpd <string> [-type <enum>]
```

Arguments

`-gpd`

Specifies the path to the GPD directory.

`-type`

Specifies the type of layers to report ({ all | routing | via }).

Returns

`<string>` : Layers for the specified GPD.

rc::getGroundCapacitors

Description

Returns a set of rcGroundCapacitors for the given nets or nodes on the command line.

Note:

In SMC mode, either the `-allCorners` or `-corners` argument is mandatory input for this command. Custom Compiler provides a warning message in the console when these arguments are missing.

Syntax

```
rc::getGroundCapacitors [-of <dbCollection | string>] [-from <string>]  
[-to <string>] [-corners <string>] [-allCorners <bool>] [-filter  
  <string>]
```

Arguments

`-of`

Reports ground capacitors associated with the specified nets.

`-from`

Constrains the selected capacitors to be part of a path starting from the specified node.

`-to`

Constrains the selected capacitors to be part of a path ending with the specified node.

`-corners`

Indicates the parasitic corner ordering for ground capacitance values.

`-allCorners`

Queries capacitance values for all corners.

`-filter`

Refines the returned capacitors with standard attribute filter.

Returns

<dbCollection> : Ground capacitors for the given nets or nodes.

rc::getNets

Description

Returns a container of rcNet objects for the user-specified query.

Syntax

```
rc::getNets [-exact <bool>] [-quiet <bool>] [-ofObjects <dbCollection |  
string>] [-pattern <string>]
```

Arguments

-exact

Disables the simple pattern matching for querying nets.

-quiet

Suppresses warning and error messages if no objects match.

-ofObjects

Creates a collection of nets from the specified objects.

-pattern

Specifies the pattern for querying nets.

Returns

<dbCollection> : Nets match with the specified query.

rc::getPins

Description

Returns a container of rcPin objects for the user-specified query.

Syntax

```
rc::getPins [-exact <bool>] [-quiet <bool>] [-ofObjects <dbCollection |  
string>] [-pattern <string>]
```

Arguments

-exact

Disables the simple pattern matching for querying pins.

-quiet

Suppresses warning and error messages if no objects match.

`-ofObjects`

Creates a collection of pins from the specified objects.

`-pattern`

Specifies the pattern for querying pins.

Returns

`<dbCollection>` : Pins match with the specified query.

rc::getPointToPointResistance

Description

Returns resistance for the given nodes on the command line.

Note:

In SMC mode, either the `-allCorners` or `-corners` argument is mandatory input for this command. Custom Compiler provides a warning message in the console when these arguments are missing.

Syntax

```
rc::getPointToPointResistance [-of <string>] [-to <string>] [-corners  
    <string>] [-allCorners <bool>]
```

Arguments

`-of`

Constrains the selected resistors to be part of a path starting from the specified node.

`-to`

Constrains the selected resistors to be part of a path ending with the specified node.

`-corners`

Specifies the corners to query resistance values.

`-allCorners`

Queries resistance values for all corners.

Returns

`<dbCollection>` : Resistors for the given nets or nodes.

rc::getPorts

Description

Returns a container of rcPort objects for the user-specified query.

Syntax

```
rc::getPorts [-exact <bool>] [-quiet <bool>] [-ofObjects <dbCollection |  
string>] [-pattern <string>]
```

Arguments

-exact

Disable the simple pattern matching for querying ports.

-quiet

Suppresses warning and error messages if no objects match.

-ofObjects

Creates a collection of ports from the specified objects.

-pattern

Specifies the pattern for querying ports.

Returns

<dbCollection> : Ports match with the specified query.

rc::getResistors

Description

Returns a set of rcResistor objects for the given nets or nodes on the command line.

Note:

In SMC mode, either the -allCorners or -corners argument is mandatory input for this command. Custom Compiler provides a warning message in the console when these arguments are missing.

Syntax

```
rc::getResistors [-of <dbCollection | string>] [-from <string>] [-to  
<string>] [-corners <string>] [-allCorners <bool>] [-filter <string>]
```

Arguments

`-of`

Reports resistors associated with the specified nets.

`-from`

Constrains the selected resistors to be part of a path starting from the specified node.

`-to`

Constrains the selected resistors to be part of a path ending with the specified node.

`-corners`

Specifies the corners to query resistance values.

`-allCorners`

Queries resistance values for all corners.

`-filter`

Refines the returned resistors with standard attribute filter.

Returns

`<dbCollection>` : Resistors for the given nets or nodes.

`rc::readParasitics`

Description

Reads parasitics data with the specified arguments.

Syntax

```
rc::readParasitics [-gpd <string>] [-keepCapacitiveCoupling <bool>]
```

Arguments

`-gpd`

Reads the Galaxy Parasitic Database (GPD) from the specified path.

`-keepCapacitiveCoupling`

Indicates that the cross capacitors are to be kept in the RC networks data structure.

Returns

<string> : Result of parasitics data.

rc::removeAnnotatedParasitics

Description

Removes GPD parasitics data.

Syntax

```
rc::removeAnnotatedParasitics
```

Returns

<string> : Result of removing GPD.

rc::reportGPDConfig

Description

Returns a string of configurations for the specified Galaxy Parasitic Database (GPD).

Syntax

```
rc::reportGPDConfig -gpd <string> [-type <enum>] [-includeStarRCOptions  
  <bool>]
```

Arguments

-gpd

Specifies the path to the GPD directory.

-type

Specifies the type of GPD configurations to report ({ active | original |
user_defined }).

-includeStarRCOptions

Includes StarRC GPD configuration options in the report.

Returns

<string> : Configurations for the specified GPD.

rc::reportGPDProperties

Description

Returns a string of properties for the specified Galaxy Parasitic Database (GPD).

Syntax

```
rc::reportGPDProperties -gpd <string> [-all <bool>] [-layers <bool>]  
[-corners <bool>]
```

Arguments

-gpd

Specifies the path to the GPD directory.

-all

Reports all types of information.

-layers

Reports only the layers.

-corners

Reports only the parasitic corners.

Returns

<string> : Properties for the specified GPD.

rc::resetGPDConfig

Description

Resets configurations for the specified Galaxy Parasitic Database (GPD).

Syntax

```
rc::resetGPDConfig -gpd <string>
```

Arguments

-gpd

Specifies the path to the GPD directory.

Returns

<string>

rc::setGPDConfig

Description

Sets configurations for the specified Galaxy Parasitic Database (GPD).

Syntax

```
rc::setGPDConfig -gpd <string> [-absoluteCouplingThreshold <float>]  
[-relativeCouplingThreshold <float>] [-couplingThresholdOperation  
<enum>] [-netlistSelectNets <string>] [-netlistType <string>]
```

Arguments

-gpd

Specifies the path to the GPD directory.

-absoluteCouplingThreshold

Specifies the absolute coupling threshold for filtering coupling capacitors. A capacitor having less than this capacitance value is split into two capacitors to ground.

-relativeCouplingThreshold

Includes StarRC GPD configuration options in the report.

-couplingThresholdOperation

Specifies the type of GPD configurations to report ({ and | or }).

-netlistSelectNets

Specifies a subset of nets to be read from the GPD database.

-netlistType

Specifies the parasitics network form to consider when reading the GPD. The allowed settings are no_couple, R, CG, CC, RCG, and RCC.

Returns

<string>

Preferences

No rc Tcl preferences are available.

30

Tcl Interface for Running Python Scripts, Blocks, and Expressions

You can run individual Python commands in the Console assistant. However, you need to use the Tcl API to run Python scripts, blocks, and expressions.

This topic describes the following:

- `py::eval`
- `py::exec`

`py::eval`

Description

Evaluates a single-line Python expression or statement, and returns the results.

Syntax

```
<result: string> py::eval <expression: string>
```

Returns

<string>

Arguments

expression

The Python expression to evaluate.

Examples

In the example, the `py::eval` command evaluates the `sys.version` expression and returns the result string.

```
> py::eval {sys.version}
```

```
3.8.6 (default, Feb 22 2021, 00:21:58)  
[GCC 9.2.0]
```

See Also

- [py::exec](#)
- [Python Interface for Running Tcl Scripts](#)
- [Typing Python Commands in the Console](#)

py::exec

Description

Executes one or more Python expressions without returning the result. You can write each expression on a new line. Alternatively, if you want to write multiple expressions on the same line, use a semicolon between adjacent expressions.

Syntax

```
py::exec <script: string>
```

Arguments

`script`

The Python expressions or statements to run.

Examples

In the example, the `py::exec` command executes two statements:

- Imports the system information.
- Prints to the screen the system version information.

Writing the two expressions on different lines appears as follows:

```
> py::exec {  
import sys  
print(sys.version)  
}  
3.8.6 (default, Feb 22 2021, 00:21:58)  
[GCC 9.2.0]
```

None

When writing the two expressions on the same line, a semicolon is added to separate them.

```
> py::exec {import sys; print sys.version}  
  
3.8.6 (default, Feb 22 2021, 00:21:58)  
[GCC 9.2.0]
```

None

See Also

- [py::eval](#)
- [Python Interface for Running Tcl Scripts](#)
- [Typing Python Commands in the Console](#)

31

Python Interface for Running Tcl Scripts

In the Custom Compiler environment, you have the option of using the Python interface to run Tcl commands and scripts.

This topic describes the following:

- `tcl.eval()`

tcl.eval()

Description

Evaluates or runs a Tcl script.

The Custom Compiler environment includes oaScript 4.1 so that you can access the OA database using Python using the `py::eval` command. See the Examples section.

Syntax

```
tcl.eval(script)
```

Returns

<Tcl result>

Arguments

`script`

The Custom Compiler Tcl script to evaluate or run.

Examples

In the first example, the `py::eval` command runs the `tcl.eval` Python command, which evaluates the `puts hello` Tcl command.

In the second example, the `py::exec` command executes the `print` Python statement. The object of `print` is obtained from running the `db::getattr` and `db::getProcessInfo` Tcl commands.

```
> py::eval {tcl.eval("puts hello")}
hello
> py::exec {print(tcl.eval("db::getAttr name -of [db::getProcessInfo]"))}
custom_compiler
```

In the following example, a Python function (`getEditDesign`) calls `tcl.eval('ed')`, converts the Tcl `oaDesign` object to a Python `oaDesign` object, and returns the current edit design.

```
def getEditDesign() -> oa.oaDesign:
    return oa.oaObject.fromOaTcl(tcl.eval('ed'))
```

See Also

- [py::exec](#)
- [py::eval](#)
- [Python Interface for Running Tcl Scripts](#)
- [Typing Python Commands in the Console](#)

32

Synopsys Live Recorder Tcl

This chapter describes how to use the Synopsys Live Recorder (SLR) Tcl commands to start recording Custom Compiler program runs and save the recordings so that you can pinpoint and troubleshoot problems.

For details, see:

- [Commands](#)

Commands

The commands covered are:

- [slr::flush](#)
- [slr::isActive](#)
- [slr::start](#)
- [slr::stop](#)

slr::flush

Description

Saves the temporary history of the recorded program runs to the file you specified in the [slr::start](#) command. The file is overwritten if it already exists.

To run this command, SLR recording must already be in progress. To view the log file, you must use the UDB utility. See the *SLR Developers Guide*, or contact Synopsys Support for details.

The `slr::flush` command does not reset the history in the temporary files.

Syntax

```
slr::flush
```

Errors

- When you run `slr::flush`, but SLR is not currently recording
- SLR internal error

slr::isActive

Description

Checks whether the Synopsys Live Recorder (SLR) is currently recording.

Syntax

```
slr::isActive
```

Returns

Boolean value that indicates whether the SLR is currently recording.

slr::start

Description

Starts recording program runs and saves the recording to the specified file. Temporary recordings are saved to the `/tmp` directory until they are saved to the specified file.

Syntax

```
slr::start -logName <string> [-delay <int>]
```

Arguments

`-logName`

Name of the log file that contains the recording.

`-delay`

Length of time, in seconds, to wait before recording begins.

Default: 0

Returns

None

Errors

- `-logName` is not specified
 - SLR internal error
 - SLR is already recording
-

slr::stop

Description

Stops recording program runs. Temporary files containing the recordings are deleted, and the recording from the last `slr::flush` to this `slr::stop` is also deleted.

Syntax

```
slr::stop
```

Returns

None

Errors

- SLR internal errors
- Error because SLR is already stopped

A

Deprecated Elements

This appendix lists deprecated elements for Custom Compiler. Deprecated elements are planned for removal in a future release. Use of a deprecated element generates a warning, although the element is still supported. After an element becomes defunct, use of the element generates an error.

For details, see:

- [Deprecated Tcl Commands](#)
- [Deprecated Tcl Arguments](#)
- [Deprecated Tcl Argument Types](#)
- [Deprecated Objects](#)
- [Deprecated Object Attributes](#)
- [Deprecated Preferences](#)
- [Deprecated Actions](#)

Deprecated Tcl Commands

The `de::audit` command gives you information about which replacement command to use instead of a deprecated command found in your script. For example:

```
> de::audit -filePath test_deprecation_audit.tcl
File <path>/test_deprecation_audit.tcl:
line 1:      deprecated command:   ile::cutPoly. Use 'ile::eraseLayer'
instead.
line 2:      deprecated command:   le::showPlaceAndRouteWithICC. Use
'xt::showICCPlaceAndRoute' instead.
```

[Table 229](#) lists deprecated Tcl commands in alphabetical order. It lists the current deprecation status and the release version where this status became effective. Replacement commands, where applicable, are also listed.

Table 229 *Deprecated Tcl Commands*

Command	Replacement	Current Status	Deprecated at Release Version
cm::createInstanceMatching	cm::createInstMatching	defunct	Q-2020.03-SP1
db::generateAnalogSchematics	db::generateSchematic	deprecated	T-2022.06-SP2
db::recordPcellCacheDependency	-	deprecated	W-2024.09
db::refreshPcellCache	-	defunct	U-2023.03
de::disableUndo	-	defunct	U-2023.03
de::enableUndo	-	defunct	U-2023.03
dm::create3DICLibrary Argument: <code>-viaMapFile</code>	-	defunct	V-2023.12
dm::create3DICLibrary Argument: <code>-libCellLib</code>	-	defunct	V-2023.12
he::createOverlayDesign	he::createOverlayView	defunct	Q-2020.03-SP1
ile::generateZigzag	ile::pkg::generateZigzag	deprecated	V-2023.12-SP1
ile::generateRedundantVias	ile::pkg::generateRedundantVias	deprecated	V-2023.12-SP1
ile::replacePlaneBoundary	ile::pkg::replacePlaneBoundary	deprecated	V-2023.12-SP1
ile::realizePlane	ile::pkg::realizePlane	deprecated	V-2023.12-SP1
ile::createPlaneBoundary	ile::pkg::createPlaneBoundary	deprecated	V-2023.12-SP1
ile::definePackagingLibrary	ile::pkg::definePackagingLibrary	deprecated	V-2023.12-SP1
ile::createDegassingHole	ile::pkg::createDegassingHole	deprecated	V-2023.12-SP1
le::createDegassingHole	le::pkg::createDegassingHole	deprecated	V-2023.12
le::createPlaneBoundary	le::pkg::createPlaneBoundary	deprecated	V-2023.12
le::checkPlaneBoundary	le::pkg::checkPlaneBoundary	deprecated	V-2023.12
le::deleteFloatingShapes	le::pkg::deleteFloatingShapes	deprecated	V-2023.12-SP1
le::deleteRedundantVias	le::pkg::deleteRedundantVias	deprecated	V-2023.12
le::getConnected	le::highlightConnected	deprecated	Q-2020.03
le::generatePlaneBoundary	le::pkg::generatePlaneBoundary	deprecated	V-2023.12
le::generateRedundantVias	le::pkg::generateRedundantVias	deprecated	V-2023.12
le::generateZigzag	le::pkg::generateZigzag	deprecated	V-2023.12

Table 229 *Deprecated Tcl Commands (Continued)*

Command	Replacement	Current Status	Deprecated at Release Version
le::highlightPlaneBoundary	le::pkg::highlightPlaneBoundary	deprecated	V-2023.12-SP1
le::realizePlane	le::pkg::realizePlane	deprecated	V-2023.12
le::refreshPlane	le::pkg::refreshPlane	deprecated	V-2023.12-SP1
le::replacePlaneBoundary	le::pkg::replacePlaneBoundary	deprecated	V-2023.12
le::reportRedundantVias	le::pkg::reportRedundantVias	deprecated	V-2023.12
le::reportResistanceBackground	-	deprecated	U-2023.03
le::showDisplayOptions	de::showGeneralOptions	defunct	Q-2020.03-SP1
lx::checkDesign	lx::verifyHierarchy	defunct	R-2020.12
nl::createVerilogICCPrefs	-	deprecated	R-2020.12
nl::extractICCPowerDomains	nl::extractDigitalPowerDomains	deprecated	Q-2020.03
nl::getSlaveNetlisters	nl::getWorkerNetlisters	defunct	U-2023.03
nl::getVirtualTerms	nl::getTerms	deprecated	W-2024.09
nl::spiceAllowNameMap	-	deprecated	T-2022.06-SP1
nl::verilogAllowNameMap	-	deprecated	T-2022.06-SP1
nl::vhdl::allowNameMap	-	deprecated	T-2022.06-SP1
sa::getCumulativeFrequency	sa::getCumulativeFrequency	deprecated	Q-2020.03-SP1
sa::runFailedIterations	sa::runIterations	defunct	Q-2020.03
se::showSchDisplayOptions	de::showGeneralOptions	defunct	Q-2020.03-SP1
se::showSymDisplayOptions	de::showGeneralOptions	defunct	Q-2020.03-SP1
vlg::getSignalType2NetTypeMapping	vlg::getNetTypeByName	deprecated	R-2020.12

Deprecated Tcl Arguments

[Table 230](#) lists deprecated Tcl command arguments in alphabetical order by associated command. It lists the current deprecation status and the release version where this status became effective. Replacement arguments, where applicable, are also listed.

Table 230 *Deprecated Tcl Command Arguments*

Command	Argument	Replacement	Current Status	Deprecated at Release Version
db::closeCDF	-forceMaster		deprecated	U-2023.03-SP1
db::exportParams	-forceMaster	-level	deprecated	U-2023.03-SP1
db::saveCDF	-forceMaster	-level	deprecated	U-2023.03-SP1
db::purgeCDF	-forceMaster	-level	deprecated	U-2023.03-SP1
cm::createIntentDef	- interactiveComm and		deprecated	U-2023.03
cm::createIntentDef	-description	-title	deprecated	U-2023.03
cm::createResistance	-destination		deprecated	S-2021.09
cm::createResistance	-exclude		deprecated	S-2021.09
cm::createResistance	-second		deprecated	S-2021.09
cm::createResistance	-first		deprecated	S-2021.09
cm::exportIntents	-storeDefaults		deprecated	R-2020.12-SP1
db::checkoutFeature	-passphrase		defunct	S-2021.09-SP1
db::createParamDef	-forceMaster	-level	deprecated	U-2023.03
db::generateSchematic	- minHorizontalSpac ing		deprecated	T-2022.06-SP2
db::generateSchematic	-minVerticalSpacing		deprecated	T-2022.06-SP2
db::generateSchematic	-instanceAlignment		deprecated	T-2022.06-SP2
db::generateSchematic	- unroutableConnecti ons		deprecated	T-2022.06-SP2
db::getMarkers	-points	-filter	deprecated	Q-2020.03
db::getMarkers	-tool	-filter	deprecated	Q-2020.03
db::getMarkers	-isClosed	-filter	deprecated	Q-2020.03
db::getMarkers	-design	-of	deprecated	Q-2020.03
db::getMarkers	-message	-filter	deprecated	Q-2020.03
db::getMarkers	-shortMessage	-filter	deprecated	Q-2020.03

Table 230 *Deprecated Tcl Command Arguments (Continued)*

Command	Argument	Replacement	Current Status	Deprecated at Release Version
db::getMarkers	-isVisible	-filter	deprecated	Q-2020.03
db::getMarkers	-regexp	-filter	deprecated	Q-2020.03
db::getMarkers	-severity	-filter	deprecated	Q-2020.03
db::getNetlistInfo	-forceMaster	-level	deprecated	U-2023.03
db::getParamDefs	-forceMaster	-level	deprecated	U-2023.03
db::getParams	-forceMaster	-level	deprecated	U-2023.03
db::print	-printHierarchyContent	-bookmarks	deprecated	S-2021.09-SP2
db::setNetlistInfo	-forceMaster	-level	deprecated	U-2023.03
db::showFindReplace	-stopLevel		deprecated	O-2018.09
db::showFindReplace	-startLevel		deprecated	O-2018.09
de::createProbe	-netHighlightTraceProc	-traceProc	deprecated	S-2021.09
de::startRendererStatistics	-bandwidth		deprecated	T-2022.06-SP1
dm::add3DICLibrary	-viaMapFile		defunct	S-2021.09
dm::addFCLibrary	-viaMapFile		defunct	Q-2020.03-SP1
dm::createFCLibrary	-viaMapFile		defunct	Q-2020.03-SP1
dm::createFCLibrary	-libCellLib		defunct	Q-2020.03-SP1
dm::createICC2Library	-libCellLib		defunct	Q-2020.03
gi::createDialog	-buttonProc		deprecated	S-2021.09-SP1
gi::createDialog	-extraButtons		defunct	S-2021.09
iue::align	-window		deprecated	U-2023.03
iue::boolean	-window		deprecated	U-2023.03
iue::copy	-window		deprecated	U-2023.03
iue::debug	-window		deprecated	U-2023.03
iue::fill	-window		deprecated	U-2023.03

Table 230 *Deprecated Tcl Command Arguments (Continued)*

Command	Argument	Replacement	Current Status	Deprecated at Release Version
ie::move	-window		deprecated	U-2023.03
ie::remove	-window		deprecated	U-2023.03
ie::setOrigin	-window		deprecated	U-2023.03
ie::size	-window		deprecated	U-2023.03
ie::stretch	-window		deprecated	U-2023.03
le::fill	-topEnc	-enclosure	deprecated	U-2023.03
le::fill	-rightEnc	-enclosure	deprecated	U-2023.03
le::fill	-ySpacing	-spacing	deprecated	U-2023.03
le::fill	-method	-tiling	deprecated	U-2023.03
le::fill	-leftEnc	-enclosure	deprecated	U-2023.03
le::fill	-bottomEnc	-enclosure	deprecated	U-2023.03
le::fill	-xSpacing	-spacing	deprecated	U-2023.03
le::fill	-newCellViewName	-newViewName	deprecated	V-2023.12-SP1
po::cutCorner	-angleRange		deprecated	Q-2020.03
sa::createE2RInterfaceElement	-fidelity	-minDelta	deprecated	Q-2020.03
sa::createR2EInterfaceElement	-fidelityUnit		deprecated	Q-2020.03
sa::createR2EInterfaceElement	-fidelity	-minDelta	deprecated	Q-2020.03
sa::createR2RInterfaceElement	-fidelityUnit		deprecated	Q-2020.03
sa::show3DSetup	-window	-parent	defunct	Q-2020.03-SP2
se::createRule	-desc	-description	deprecated	U-2023.03

Deprecated Tcl Argument Types

[Table 231](#) lists deprecated Tcl command argument types in alphabetical order by associated command and argument. It lists the current deprecation status and the release version where this status became effective. Replacement argument types, where applicable, are also listed.

Table 231 Deprecated Tcl Argument Types

Command	Argument	Argument Type	Replacement	Current Status	Deprecated at Release Version
de::getObjectFilters [<name: string>]		leSoftMacroBoundary		deprecated	P-2019.06-SP1
de::getObjectFilters [<name: string>]		leSoftMacroPin		deprecated	P-2019.06-SP1

Deprecated Objects

[Table 232](#) lists deprecated objects in alphabetical order by object name. It lists the current deprecation status and the release version where this status became effective. It also lists replacement objects.

Table 232 Deprecated Tcl Objects

Object	Replacement	Current Status	Deprecated at Release Version
No deprecated objects at this time.			

Deprecated Object Attributes

[Table 233](#) lists deprecated objects in alphabetical order by associated object name. It also lists the current deprecation status and the release version where this status became effective.

Table 233 Deprecated Tcl Object Attributes

Object Attribute	Object	Replacement	Current Status	Deprecated at Release Version
deHighlightSet	drawInvisible		deprecated	Q-2020.03
leStripedSegment	spacing	spacings	deprecated	Q-2020.03
leStripedSegment	width	widths	deprecated	Q-2020.03
nIAMSNetlister	slavesAttributes	workersAttributes	deprecated	S-2021.09
nIAMSNetlister	masterNetlister	defaultNetlister	deprecated	S-2021.09
nINetlister	ignoreParasitics		deprecated	Q-2020.03-SP2
nINetlister	amsSlaveMode	amsNetlister	deprecated	Q-2020.03-SP2

Table 233 *Deprecated Tcl Object Attributes (Continued)*

Object Attribute	Object	Replacement	Current Status	Deprecated at Release Version
nlNetlister	allowNameMapProc		deprecated	T-2022.06-SP1

Deprecated Preferences

[Table 234](#) lists deprecated preferences in alphabetical order. It also lists the current deprecation status and the release version where this status became effective.

Table 234 *Deprecated Tcl Preferences*

Preference	Replacement	Current Status	Deprecated at Release Version
cmConstraintEditorExportFormat		defunct	U-2023.03
csRASetupVmaxminavg		deprecated	T-2022.06
dbExportNetlistFlattenDeleteTempCellView		defunct	T-2022.06-SP2
dbImportDefImportModuleHier		defunct	T-2022.06
dbImportOasisDbuPerUu	dbImportOasisDbuPerUU	deprecated	S-2021.09-SP1
dbParamTrimTrailingZeroesAfterDecimalPoint		deprecated	U-2023.03-SP1
dbPcellCacheHoldLock		deprecated	U-2023.03-SP1
dbPcellCacheLockHiatusDelay		deprecated	U-2023.03-SP1
dbPcellCacheLockHiatusThreshold		deprecated	U-2023.03-SP1
dbPcellCacheLockRetryDelay		deprecated	U-2023.03-SP1
dbPcellCacheLockRetryLimit		deprecated	U-2023.03-SP1
dbPrintHierarchyContent	dbPrintBookmarks	deprecated	S-2021.09-SP2
dbPropertyEditorAnnotateDefault		deprecated	Q-2020.03-SP1
dbSchGenInstanceAlignment		deprecated	T-2022.06-SP2
dbSchGenMergeSubports	dbSchGenMergePortSubBits	defunct	U-2023.03
dbSchGenMinHorizontalSpacing		deprecated	T-2022.06-SP2
dbSchGenMinVerticalSpacing		deprecated	T-2022.06-SP2

Table 234 *Deprecated Tcl Preferences (Continued)*

Preference	Replacement	Current Status	Deprecated at Release Version
dbSchGenUnroutableConnections		deprecated	T-2022.06-SP2
dbSchGenUseNewEngineForDigitals		deprecated	T-2022.06-SP2
dbShowNewParameterEditor		deprecated	Q-2020.03-SP1
deCanvasPropertyEditorDefaultRegion		defunct	Q-2020.03-SP1
deCanvasPropertyEditorEnable		defunct	Q-2020.03-SP1
deCanvasPropertyEditorEnlargeDistance		defunct	Q-2020.03-SP1
deCanvasPropertyEditorLEEnable		defunct	Q-2020.03-SP1
deCanvasPropertyEditorSEEnable		defunct	Q-2020.03-SP1
deDescendShadow		defunct	U-2023.03
deEnableMinDimFilter		defunct	U-2023.03
deEnableMouseMotionInterrupt		defunct	U-2023.03
deEnableNewConnectedHighlights		defunct	T-2022.06
deEnableNewHighlights		defunct	T-2022.06
deEnableNewNetHighlights		defunct	T-2022.06
deEnableRuntimePixmapThread		defunct	U-2023.03
deHelpUseWebWorks		defunct	U-2023.03
deLaunchHelpViewerAtStartup		defunct	U-2023.03
deMarkerBrowserLayoutTools		defunct	Q-2020.03-SP1
deMultithreadDelay		defunct	U-2023.03
deMultithreadStdRenderer		defunct	U-2023.03
dePolygonConversionTreshold	dePolygonConversionThreshold	deprecated	T-2022.06-SP1
deProbeHighlightShadow	deHighlightShadow	deprecated	V-2023.12
deProbeNetHighlightShadow	deHighlightShadow	deprecated	V-2023.12
dePDFReader		defunct	U-2023.03
dePolygonConversionTreshold		deprecated	T-2022.06-SP1
dePropertyEditorParameterFilter		defunct	R-2020.12-SP1

Table 234 *Deprecated Tcl Preferences (Continued)*

Preference	Replacement	Current Status	Deprecated at Release Version
deSelectionTableShadow	leShadowSelection	defunct	Q-2020.03-SP1
deSelectionTableShowArea		defunct	Q-2020.03
deSelectionTableShowArray		defunct	Q-2020.03
deSelectionTableShowCellName		defunct	Q-2020.03
deSelectionTableShowCount		defunct	Q-2020.03
deSelectionTableShowDefinition		defunct	Q-2020.03
deSelectionTableShowFigGroupMembers		defunct	Q-2020.03
deSelectionTableShowLayer		defunct	Q-2020.03
deSelectionTableShowLength		defunct	Q-2020.03
deSelectionTableShowLibrary		defunct	Q-2020.03
deSelectionTableShowName		defunct	Q-2020.03
deSelectionTableShowOrientation		defunct	Q-2020.03
deSelectionTableShowParameters		defunct	Q-2020.03
deSelectionTableShowPurpose		defunct	Q-2020.03
deSelectionTableShowSeverity		defunct	Q-2020.03
deSelectionTableShowType		defunct	Q-2020.03
deSelectionTableShowWidth		defunct	Q-2020.03
deTemplateOtherComponent		deprecated	U-2023.03
dmICC2OAResourceMap		defunct	Q-2020.03-SP1
dmICC2OAViaMap		defunct	Q-2020.03
dmShowICC2ReferenceLibraries	dmShowDigitalReferenceLibraries	defunct	Q-2020.03
giConsoleShowTimestamp	giConsoleTimestampFormat	deprecated	U-2023.03
giMinimumTabSize		defunct	U-2023.03
giModernUI		deprecated	V-2023.12
giMotionNotifyCompress		defunct	U-2023.03
giHomeOpenSingleLM		defunct	U-2023.03

Table 234 *Deprecated Tcl Preferences (Continued)*

Preference	Replacement	Current Status	Deprecated at Release Version
heDesignNavigatorShowErrorIcon		deprecated	T-2022.06
heNavigatorFindMatchCase		defunct	U-2023.03
heNavigatorFindRegExp		defunct	U-2023.03
leActiveLPP		defunct	U-2023.03
leAlignConnectionAware	leStretchConnectionAware	deprecated	R-2020.12
leAlignWithAlignAssist		deprecated	T-2022.06-SP2
leAutoViaAllowCenterOffset		deprecated	S-2021.09-SP1
leAutoViaCheckNetAnchor		deprecated	V-2023.12
leAutoViaEnvAwareUseDRD		deprecated	V-2023.12-SP1
leAutoViaRuntimeOverlapLimit		deprecated	Q-2020.03
leCEAdjustDepthWhenContextChange		deprecated	T-2022.06
leColorModifyOverlapOnly		deprecated	Q-2020.03
leColorUpdateModified		deprecated	Q-2020.03
leDensityPorosity	leDensityOpacity	deprecated	Q-2020.03
leDensityUseGroupSize		deprecated	Q-2020.03-SP2
leDistributeConnectionAware	leStretchConnectionAware	deprecated	R-2020.12
leDrawLinearLabel		deprecated	Q-2020.03
leDrawLinearText		deprecated	Q-2020.03
leDrawMethod		defunct	S-2021.09-SP1
leDrawRubberBand		defunct	U-2023.03
leDrawUnclutteredShapes	leDrawMergedShapes	deprecated	S-2021.09-SP1
leEditViaCalculateLayerWidth	leManualViaCalculateLayerWidth	deprecated	U-2023.03-SP1
leElectricalReporterHierarchyDepth		deprecated	S-2021.09-SP1
leEMReporterAutoSave		deprecated	U-2023.03-SP1
leFillNewCellViewName	eFillNewLCV	deprecated	V-2023.12-SP1
leHCHighlightInvisibleShapes		deprecated	Q-2020.03

Table 234 *Deprecated Tcl Preferences (Continued)*

Preference	Replacement	Current Status	Deprecated at Release Version
leHonorWeakConnection	leWeakConnectionModeOverride	deprecated	U-2023.03
leInstLabelDrawParameters	leInstLabelDrawRealMasterCellName	deprecated	T-2022.06-SP1
leNXTGRDMapFile		deprecated	T-2022.06
lePackagingAutoRefreshPlanes		deprecated	U-2023.03-SP1
leReportResistanceMultiple		defunct	U-2023.03
leResistanceAnalysisTemperature		defunct	U-2023.03
leRowBotSpacing		deprecated	U-2023.03-SP1
leRuntimeVertexCheckThreshold		defunct	U-2023.03
leShortDetectionMode		deprecated	K-2015.09-SP1
leShowDistanceDuringStretch		deprecated	Q-2020.03
leSnapGrid	leClipboardSnapGrid	deprecated	U-2023.03-SP1
leStretchHandleDisplay		deprecated	Q-2020.03
leUseDigitalVias		deprecated	U-2023.03-SP2
leUsePinPairToReportNetR		defunct	U-2023.03
leVDRCPhysicalAnnotationDepth		defunct	Q-2020.03-SP1
leViaApplyEOLAsEnclosure		deprecated	U-2023.03
leVPEEnableContextCheckCorrection		deprecated	V-2023.12-SP1
IxEComMode		deprecated	T-2022.06
IxExpandSchematicCheckOut		deprecated	S-2021.09-SP2
IxExpandSchematicEnableVC		deprecated	S-2021.09-SP2
IxPseudoParallelModeRestriction		deprecated	S-2021.09
IxResistorConnection		deprecated	S-2021.09-SP2
IxSdlBrowserDisplayMaster		deprecated	S-2021.09-SP1
nIAMSAIwaysGetScalarizeTermsFromSlave	nIAMSAIwaysGetScalarizeTermsFromWorker	deprecated	S-2021.09
nIAMSShortPropNames	nIAMSSignorePropNames	deprecated	U-2023.03

Table 234 *Deprecated Tcl Preferences (Continued)*

Preference	Replacement	Current Status	Deprecated at Release Version
nIBackgroundNetlistingHost		deprecated	R-2020.12
nICDLGenerateSubcktInTopModule		deprecated	Q-2020.03-SP2
nICdlPrintLDD	nICDLPrintLDD	deprecated	R-2020.12-SP1
nICDLPromoteConnectDefsAtTop	nICDLTopVirtualTerms	deprecated	U-2023.03-SP1
nICDLShortLVSIgnoreDevices		deprecated	Q-2020.03
nICDLShortNames	nIShortExprPropNames	deprecated	Q-2020.03
nICDLShortPropNames	nICDLIgnorePropNames	deprecated	U-2023.03
nIEldoGenerateSubcktInTopModule		deprecated	Q-2020.03-SP2
nIEldoShortPropNames	nIEldoIgnorePropNames	deprecated	U-2023.03
nIHSPICEGenerateSubcktInTopModule		deprecated	Q-2020.03-SP2
nIHSPICEShortPropNames	nIHSPICEIgnorePropNames	deprecated	U-2023.03
nILvsSPICEShortLVSIgnoreDevices		deprecated	Q-2020.03
nINanoSimGenerateSubcktInTopModule		deprecated	Q-2020.03-SP2
nINanoSimShortPropNames	nINanoSimIgnorePropNames	deprecated	U-2023.03
nIPortOrderMismatchRule	seCheckPortOrderMismatch	deprecated	Q-2020.03-SP2
nIPromoteTerms		deprecated	U-2023.03-SP1
nIRenameSameBaseVectorInsts	nIVerilogRenameSameBaseVect orInsts	deprecated	Q-2020.03-SP2
nISpectreAMSShortPropNames	nISpectreAMSIgnorePropNames	deprecated	U-2023.03
nISpectreGenerateSubcktInTopModule		deprecated	Q-2020.03-SP2
nISpectreShortPropNames	nISpectreIgnorePropNames	deprecated	U-2023.03
nIVerilogGenerateSubcktInTopModule		deprecated	Q-2020.03-SP2
nIVerilogICCGenerateSubcktInTopModule		deprecated	Q-2020.03-SP2
nIVerilogICCSHORTPropNames	nIVerilogICCIgnorePropNames	deprecated	U-2023.03
nIVerilogShortPropNames	nIVerilogIgnorePropNames	deprecated	U-2023.03
nIVerilogSortWireDeclarations		defunct	Q-2020.03-SP1
nIVHDLGenerateSubcktInTopModule		deprecated	Q-2020.03-SP2

Table 234 *Deprecated Tcl Preferences (Continued)*

Preference	Replacement	Current Status	Deprecated at Release Version
nlVHDLShortPropNames	nlVHDLIgnorePropNames	deprecated	U-2023.03
rtCoverPin	rtCoverPinWithVia	deprecated	V-2023.12-SP1
rtCustomRouteDigitalBlock	rtCustomRouteDesignType	deprecated	T-2022.06
rtCustomRouteDigitalBlock		deprecated	T-2022.06
rtEnableAdvancedRules	rtEnableRules	defunct	R-2020.12
rtPatternRouteAssistantFeatureBasedSpineTrunks	rtPatternRouteAssistantSpinePosition	deprecated	U-2023.03
rtPatternRouteAssistantFeatureBasedSpineTrunksLowAccuracyMode	rtPatternRouteAssistantSpinePosition	deprecated	U-2023.03
rtPatternRouteAssistantOverDeviceViaMaximization	rtPatternRouteAssistantViaMaximization	defunct	R-2020.12-SP1
rtPatternRoutePolyContExtension	rtPatternRouteAbutPolyContact	defunct	Q-2020.03-SP1
rtPatternRouteSpineOverSD	rtPatternRouteAssistantSpinePosition	deprecated	U-2023.03
rtTopologyExportFilePath		defunct	R-2020.12-SP1
rtTopologyExportNetOption		defunct	R-2020.12-SP1
rtUseNewIR	rtUse202003IR	defunct	Q-2020.03-SP2
rtWeldingRadius		defunct	R-2020.12
saFineSimProModelGroupsCheckBox	saFineSimProShowModelGroups	deprecated	Q-2020.03-SP1
saFineSimProSaveOptionsEMTStart	saFineSimProSaveOptionsEmTStart	defunct	Q-2020.03
saFineSimProSaveOptionsEMTStop	saFineSimProSaveOptionsEmTStop	defunct	Q-2020.03
saFineSimProSaveOptionsSaveEMCurrents	saFineSimProSaveOptionsEnableEM	defunct	Q-2020.03
saFineSimProVCSModelGroupsCheckBox	saFineSimProVCSShowModelGroups	deprecated	Q-2020.03-SP1
saGlobalModelGroupsCheckBox	saGlobalShowModelGroups	deprecated	Q-2020.03-SP1
saHSIMHSPICEModelGroupsCheckBox	saHSIMHSPICEShowModelGroups	deprecated	Q-2020.03-SP1

Table 234 *Deprecated Tcl Preferences (Continued)*

Preference	Replacement	Current Status	Deprecated at Release Version
saHSPICEModelGroupsCheckBox	saHSPICEShowModelGroups	deprecated	Q-2020.03-SP1
saHSPICESaveOptionsEMTStart	saHSPICESaveOptionsEmTStart	defunct	Q-2020.03
saHSPICESaveOptionsEMTStop	saHSPICESaveOptionsEmTStop	defunct	Q-2020.03
saHSPICESaveOptionsSaveEMCurrents	saHSPICESaveOptionsEnableEM	defunct	Q-2020.03
saHSPICESaveOptionsSaveVDRCVoltages	saHSPICESaveOptionsEnableVDRC	defunct	Q-2020.03
saHSPICESaveOptionsVDRCTStart	saHSPICESaveOptionsVdrcTStart	defunct	Q-2020.03
saHSPICESaveOptionsVDRCTStop	saHSPICESaveOptionsVdrcTStop	defunct	Q-2020.03
saHSPICETRANNOISESamples	saHSPICETRANNOISESamplesNum	defunct	Q-2020.03
saPrimeSimModelGroupsCheckBox	saPrimeSimShowModelGroups	deprecated	R-2020.12-SP1
saPrimeSimVCSCCommands		defunct	R-2020.12-SP1
saPrimeSimVCSModelGroupsCheckBox	saPrimeSimVCSShowModelGroups	deprecated	R-2020.12-SP1
saPrimeSimXACCommands		defunct	S-2021.09
saPrimeSimXAModelGroupsCheckBox	saPrimeSimXAShowModelGroups	deprecated	S-2021.09
saPrimeSimXAVCSCCommands		defunct	S-2021.09-SP2
saPrimeSimXAVCSModelGroupsCheckBox	saPrimeSimXAVCSShowModelGroups	deprecated	S-2021.09-SP2
saSpectreModelGroupsCheckBox	saSpectreShowModelGroups	deprecated	Q-2020.03-SP1
saXAEIModelGroupsCheckBox	saXAEIModelShowModelGroups	deprecated	Q-2020.03-SP1
saXAHSPICEModelGroupsCheckBox	saXAHSPICEShowModelGroups	deprecated	Q-2020.03-SP1
saXASpectreModelGroupsCheckBox	saXASpectreShowModelGroups	deprecated	Q-2020.03-SP1
saXAVCSHSPICEModelGroupsCheckBox	saXAVCSHSPICEShowModelGroups	deprecated	Q-2020.03-SP1
seChekDuplicateCellsCaseSensitive	seCheckDuplicateCellsCaseSensitive	defunct	U-2023.03

Table 234 *Deprecated Tcl Preferences (Continued)*

Preference	Replacement	Current Status	Deprecated at Release Version
seLogicalCompareDialogHierCompare		defunct	U-2023.03
seLogicalCompareDialogShowSystemNet		defunct	U-2023.03
seLogicalCompareDialogZoomOnHighlight	seLogicalCompareZoomOnHighlight	defunct	U-2023.03
seLogicalCompareDialogZoomScale	seLogicalCompareZoomScale	defunct	U-2023.03
seNetTracerTraceProc		deprecated	Q-2020.03-SP2
seNetTracerTraceThrough		deprecated	Q-2020.03-SP2
sePatchCord	dbSchGenPatchCord	defunct	Q-2020.03-SP1
seSchematicOpticalPin	seSchematicPhotonicPin	defunct	U-2023.03
seSchematicPinArray	seSchematicPinCreationMode	defunct	U-2023.03
seSymbolOpticalPin	seSymbolPhotonicPin	defunct	U-2023.03
seWiringVerticalBundleLabel		deprecated	T-2022.06
wfWVButtonAddHCursor	wfWVCursorType	deprecated	Q-2020.03-SP1
wfWVPanelFSMargin	wfWVPanelFSMarginY	deprecated	Q-2020.03-SP1
xtLPEStarInductanceSelectSBbInput	xtLPEStarInductanceSelectBbRegion	defunct	T-2022.06-SP2
xtLPEStarInductanceSelectSBbInput	xtLPEStarInductanceSelectSBbRegion	defunct	T-2022.06-SP2
xtLVSEXTCalibreVariables	xtLVSEXTCalibreControlVars	defunct	T-2022.06
xtLVSEXTICVVariables	xtLVSEXTICVControlVars	defunct	T-2022.06
xtStarCalibreGenerateCCI		deprecated	T-2022.06-SP1
xtStarCalibreMaskSVDBDir		deprecated	K-2015.09-SP1
xtStarCalibreQueryFile		deprecated	T-2022.06-SP1
xtStarCalibreQueryFileType		deprecated	K-2015.09-SP1
xtStarCaseSensitive		deprecated	T-2022.06-SP1
xtStarCoupleToGround		deprecated	T-2022.06-SP1
xtStarExtraction		deprecated	T-2022.06-SP1
xtStarHierarchicalSeparator		deprecated	T-2022.06-SP1

Table 234 *Deprecated Tcl Preferences (Continued)*

Preference	Replacement	Current Status	Deprecated at Release Version
xtStarICVRunsetReportFile		deprecated	T-2022.06-SP1
xtStarMappingFile		deprecated	T-2022.06-SP1
xtStarNetlistFile		deprecated	T-2022.06-SP1
xtStarNetlistGroundNodeName		deprecated	T-2022.06-SP1
xtStarNetlistInductance		defunct	T-2022.06-SP2
xtStarNetlistPortOrder		deprecated	T-2022.06-SP1
xtStarNetlistSelectNets		deprecated	T-2022.06-SP1
xtStarNetlistType		deprecated	T-2022.06-SP1
xtStarOADeviceMappingFile		defunct	U-2023.03
xtStarOALayerMappingFile		deprecated	T-2022.06-SP1
xtStarOAMarkerSize		deprecated	T-2022.06-SP1
xtStarOAViewName		deprecated	T-2022.06-SP1
xtStarOperatingFrequency		defunct	T-2022.06-SP2
xtStarOperatingTemperature		deprecated	T-2022.06-SP1
xtStarPowerExtract		deprecated	T-2022.06-SP1
xtStarReduction		deprecated	T-2022.06-SP1
xtStarRemoveFloatingNets		deprecated	T-2022.06-SP1
xtStarSpiceSubcktFile		deprecated	T-2022.06-SP1
xtStarTcadGrdFile		deprecated	T-2022.06-SP1
xtStarXref		deprecated	T-2022.06-SP1
xtXACTCalibreVariables	xtXACTCalibreControlVars	defunct	R-2020.12

Deprecated Actions

[Table 235](#) lists deprecated actions in alphabetical order. It also lists the current deprecation status and the release version where this status became effective.

Table 235 Deprecated Tcl Command Actions

Action	Replacement	Current Status	Deprecated at Release Version
dbLoadPrefsFromFile	dbShowLoadPrefs	deprecated	R-2020.12
dbSavePrefsToFile	dbShowSavePrefs	deprecated	R-2020.12
deProbeShadow	deCrossProbe	deprecated	V-2023.12